Privacy-Preserving Machine Learning

Manuel V. Loureiro

Version 2026-10-15

Table of contents

Preface **1**
How this book is organized . . . 1

I. Prerequisites **2**
Who this part is for . . . 3
The common thread . . . 3
How to read this part . . . 4
Notation . . . 5
What you will be able to do . . . 6

1. Probability distributions **7**
1.1. Why this matters . . . 7
1.2. Toolkit: what to know for each distribution . . . 8
1.3. Bernoulli and binomial . . . 11
1.4. Poisson . . . 12
1.5. Exponential . . . 15
1.6. Laplace . . . 20
1.7. Normal . . . 23
1.8. Comparing a density with its shifted copy . . . 28
1.9. Code: check the facts by simulation . . . 31
1.10. References . . . 31
1.11. Cheat sheet . . . 32

2. Concentration inequalities **33**
2.1. Why this matters . . . 33
2.2. Toolkit: five facts every derivation uses . . . 34
2.3. Markov and Chebyshev . . . 37
2.4. The Chernoff method . . . 39
2.5. Sub-Gaussian variables and Hoeffding . . . 43
2.6. Gaussian and Laplace tails . . . 48
2.7. McDiarmid and Azuma . . . 53
2.8. Code: check every bound by simulation . . . 56
2.9. References . . . 57
2.10. Cheat sheet . . . 57

3. Divergences **59**
3.1. Why this matters . . . 59
3.2. Toolkit: three facts every derivation uses . . . 61
3.3. KL divergence . . . 63

3.4. Rényi divergence . . . 67
3.5. Two Gaussians with equal variance, by hand . . . 73
3.6. Two Laplace distributions with shifted location . . . 75
3.7. Total variation . . . 78
3.8. Hockey-stick divergence . . . 82
3.9. f-divergences . . . 85
3.10. The data-processing inequality . . . 88
3.11. Pinsker's inequality . . . 91
3.12. Code: check every divergence by computation and simulation . . . 94
3.13. References . . . 96
3.14. Cheat sheet . . . 96

4. Hypothesis testing **98**
4.1. Why this matters . . . 98
4.2. Toolkit: four facts every derivation uses . . . 99
4.3. Simple-versus-simple testing . . . 101
4.4. The Neyman–Pearson lemma . . . 104
4.5. Trade-off functions and ROC curves . . . 108
4.6. The Gaussian trade-off curve . . . 110
4.7. The Laplace trade-off curve . . . 113
4.8. Best advantage equals total variation . . . 116
4.9. ROC summaries: why AUC hides the low-FPR regime . . . 119
4.10. The Bayesian view . . . 121
4.11. Estimating error rates from finite trials . . . 124
4.12. Code: check every result by computation and simulation . . . 127
4.13. References . . . 128
4.14. Cheat sheet . . . 129

5. Information theory **130**
5.1. Why this matters . . . 130
5.2. Toolkit: three facts every derivation uses . . . 131
5.3. Shannon entropy . . . 133
5.4. Mutual information . . . 138
5.5. Channels and capacity . . . 141
5.6. Min-entropy and guessing . . . 143
5.7. Conditional min-entropy and min-entropy leakage . . . 145
5.8. Shannon leakage versus min-entropy leakage . . . 149
5.9. Fano's inequality . . . 152
5.10. Code: check every formula numerically . . . 155
5.11. References . . . 156
5.12. Cheat sheet . . . 156

6. Optimization **158**
6.1. Why this matters . . . 158
6.2. Toolkit: five facts every derivation uses . . . 159
6.3. Convexity, Lipschitz continuity and smoothness . . . 162
6.4. Gradient descent and SGD . . . 166
6.5. SGD for convex Lipschitz objectives . . . 170

6.6. Strongly convex objectives . . . 173
6.7. Gradient noise and minibatches . . . 175
6.8. Non-convex objectives: convergence to stationary points . . . 179
6.9. Sensitivity of regularized ERM . . . 184
6.10. Code: check every bound by simulation . . . 188
6.11. References . . . 189
6.12. Cheat sheet . . . 189

7. Algorithmic stability and generalization **191**
7.1. Why this matters . . . 191
7.2. Toolkit: five facts every derivation uses . . . 192
7.3. Risk, empirical risk and the gap . . . 195
7.4. Uniform stability and the expected gap . . . 196
7.5. High probability: the Bousquet–Elisseeff bound . . . 199
7.6. Regularized ERM is stable . . . 202
7.7. SGD is stable if you stop early . . . 207
7.8. An unstable algorithm: 1-nearest neighbour . . . 210
7.9. Overfitting is memorization of individual examples . . . 212
7.10. Code: check every bound by simulation . . . 214
7.11. References . . . 215
7.12. Cheat sheet . . . 216

8. Cryptography basics **218**
8.1. Why this matters . . . 218
8.2. Toolkit: five facts every derivation uses . . . 219
8.3. Security parameter, adversaries and negligible functions . . . 221
8.4. Computational indistinguishability and the hybrid argument . . . 225
8.5. Concrete versus asymptotic security . . . 228
8.6. Pseudorandom generators and functions . . . 229
8.7. Encryption: perfect secrecy and IND-CPA . . . 234
8.8. The simulation paradigm . . . 238
8.9. Learning with errors . . . 240
8.10. Code: check the numbers by simulation . . . 245
8.11. References . . . 246
8.12. Cheat sheet . . . 247

II. Threat models and attacks **249**
Who this part is for . . . 250
The common thread . . . 250
How to read this part . . . 251
Notation . . . 252
What you will be able to do . . . 253

9. Threat models and privacy games **254**
9.1. Why this matters . . . 254
9.2. Toolkit: three facts every derivation uses . . . 255
9.3. Threat models . . . 259

9.4. Privacy games . . . 263
9.5. Advantage, total variation and the trade-off curve . . . 267
9.6. Average case and worst case . . . 270
9.7. Code: check every number by computation and simulation . . . 273
9.8. References . . . 274
9.9. Cheat sheet . . . 275

10. Membership inference **277**
10.1. Why this matters . . . 277
10.2. Toolkit: three facts every derivation uses . . . 279
10.3. The loss attack and the generalization gap . . . 281
10.4. Shadow models . . . 283
10.5. LiRA: a likelihood-ratio test per example . . . 287
10.6. Evaluating attacks: TPR at a low FPR . . . 292
10.7. Why outliers are most exposed . . . 296
10.8. The full-size pipeline: LiRA on CIFAR-10 . . . 300
10.9. Code: run every attack on the running example . . . 301
10.10. References . . . 303
10.11. Cheat sheet . . . 303

11. Attribute inference and model inversion **305**
11.1. Why this matters . . . 305
11.2. Toolkit: three facts every derivation uses . . . 307
11.3. The attribute inference game . . . 308
11.4. The imputation baseline . . . 311
11.5. Model inversion: the MAP attack . . . 313
11.6. Confidence masking . . . 319
11.7. Code: the game on a toy model . . . 322
11.8. References . . . 323
11.9. Cheat sheet . . . 324

12. Property inference **325**
12.1. Why this matters . . . 325
12.2. Toolkit: three facts every derivation uses . . . 327
12.3. The property-inference game . . . 329
12.4. Why a property leaks when no record does . . . 331
12.5. Shadow models and meta-classifiers . . . 335
12.6. Permutation invariance . . . 339
12.7. Code: check every result by computation and simulation . . . 344
12.8. References . . . 345
12.9. Cheat sheet . . . 345

13. Memorization and training-data extraction **347**
13.1. Why this matters . . . 347
13.2. Toolkit: three facts every derivation uses . . . 348
13.3. Verbatim, approximate, discoverable, extractable . . . 351
13.4. Duplication and model size: the copies a string needs . . . 353
13.5. Canaries and the exposure metric . . . 357

13.6. Extraction without the prefix: generate, rank, verify . . . 362
13.7. Code: check every number by computation and simulation . . . 366
13.8. References . . . 367
13.9. Cheat sheet . . . 368

14. Gradient inversion **370**
14.1. Why this matters . . . 370
14.2. Toolkit: three facts every derivation uses . . . 372
14.3. Exact recovery from a fully connected layer . . . 374
14.4. Label recovery from the last layer . . . 377
14.5. Deep Leakage from Gradients . . . 380
14.6. Batches and averaging . . . 383
14.7. Code: check every attack by computation . . . 388
14.8. References . . . 389
14.9. Cheat sheet . . . 390

15. Linkage and re-identification **392**
15.1. Why this matters . . . 392
15.2. Toolkit: three facts every derivation uses . . . 393
15.3. Quasi-identifiers and the linkage attack . . . 395
15.4. How many people are unique? A birthday calculation . . . 399
15.5. Sparse, high-dimensional data: the Netflix attack . . . 404
15.6. Does releasing a sample protect? . . . 408
15.7. Code: check every formula by simulation . . . 414
15.8. References . . . 415
15.9. Cheat sheet . . . 415

16. Syntactic anonymity and how it breaks **417**
16.1. Why this matters . . . 417
16.2. Toolkit: three facts every derivation uses . . . 418
16.3. k-anonymity . . . 420
16.4. Homogeneity, background knowledge and l-diversity . . . 425
16.5. Skewness, similarity and t-closeness . . . 428
16.6. The composition attack . . . 431
16.7. Code: anonymize, then attack . . . 435
16.8. References . . . 436
16.9. Cheat sheet . . . 437

17. Reconstruction attacks **439**
17.1. Why this matters . . . 439
17.2. Toolkit: three facts every derivation uses . . . 440
17.3. The exponential attack . . . 442
17.4. The linear-programming decoder . . . 445
17.5. Least squares under worst-case and random error . . . 449
17.6. The Fundamental Law of Information Recovery . . . 454
17.7. Reconstructing a census block . . . 458
17.8. Code: check every attack by computation and simulation . . . 462
17.9. References . . . 463

17.10. Cheat sheet 463

III. Differential privacy foundations **465**
Who this part is for 466
The common thread 466
How to read this part 467
Notation 468
What you will be able to do 468

18. The definition of differential privacy **470**
18.1. Why this matters 470
18.2. Toolkit: four facts from Part 1 472
18.3. Neighbouring datasets 473
18.4. Pure and approximate differential privacy 476
18.5. Randomized response 479
18.6. Divergence forms and the privacy loss 482
18.7. The hypothesis-testing interpretation 485
18.8. Bound on the membership advantage 488
18.9. The Bayesian interpretation 491
18.10. Why δ must be much smaller than $1/n$ 494
18.11. What differential privacy does not protect 497
18.12. Code: check every number by enumeration and simulation 498
18.13. References 500
18.14. Cheat sheet 500

19. Post-processing, group privacy and composition **502**
19.1. Why this matters 502
19.2. Toolkit: three facts every derivation uses 504
19.3. Post-processing 505
19.4. Group privacy 509
19.5. Basic sequential composition 514
19.6. Parallel composition 519
19.7. What composes and what doesn't 521
19.8. Code: check every property by exact computation and simulation 523
19.9. References 524
19.10. Cheat sheet 525

20. The Laplace and Gaussian mechanisms **526**
20.1. Why this matters 526
20.2. Toolkit: three facts every derivation uses 528
20.3. Sensitivity 529
20.4. The Laplace mechanism 532
20.5. The Gaussian mechanism: classic calibration 535
20.6. The exact privacy curve of the Gaussian mechanism 538
20.7. Laplace against Gaussian in many dimensions 542
20.8. Code: check every mechanism by computation and simulation 545
20.9. References 546

20.10. Cheat sheet . . . 546

21. Advanced and optimal composition **548**
21.1. Why this matters . . . 548
21.2. Toolkit: three facts every derivation uses . . . 550
21.3. The privacy loss of a composition . . . 552
21.4. Advanced composition . . . 554
21.5. Optimal composition . . . 557
21.6. Adaptive composition . . . 563
21.7. Code: check every composition bound . . . 567
21.8. References . . . 568
21.9. Cheat sheet . . . 568

22. Selection: exponential mechanism, report-noisy-max, sparse vector **570**
22.1. Why this matters . . . 570
22.2. Toolkit: four facts every derivation uses . . . 572
22.3. The exponential mechanism . . . 574
22.4. Utility of the exponential mechanism . . . 577
22.5. Report-noisy-max . . . 580
22.6. Gumbel noise: report-noisy-max is the exponential mechanism . . . 583
22.7. The sparse vector technique . . . 586
22.8. When the sparse vector technique goes wrong . . . 590
22.9. Code: check every mechanism by computation and simulation . . . 593
22.10. References . . . 594
22.11. Cheat sheet . . . 595

23. Beyond global sensitivity **597**
23.1. Why this matters . . . 597
23.2. Toolkit: four facts every derivation uses . . . 599
23.3. Local sensitivity, and why it leaks . . . 600
23.4. Smooth sensitivity . . . 603
23.5. Propose-test-release . . . 609
23.6. Subsample-and-aggregate . . . 612
23.7. Clipping . . . 615
23.8. Code: check every number by computation . . . 617
23.9. References . . . 619
23.10. Cheat sheet . . . 619

24. Discrete and floating-point-safe mechanisms **621**
24.1. Why this matters . . . 621
24.2. Toolkit: exact randomness from uniform integers . . . 622
24.3. Floating-point Laplace breaks differential privacy . . . 623
24.4. The snapping mechanism . . . 626
24.5. Exact Bernoulli($e^{-\gamma}$) . . . 628
24.6. The discrete Laplace mechanism . . . 630
24.7. The discrete Gaussian mechanism . . . 634
24.8. Sampling the discrete Gaussian exactly . . . 639
24.9. Code: check every sampler against its exact distribution . . . 641

24.10. References . . . 643
24.11. Cheat sheet . . . 644

25. Local, shuffle and distributed trust models **646**
25.1. Why this matters . . . 646
25.2. Toolkit: three facts every derivation uses . . . 648
25.3. Central and local differential privacy . . . 649
25.4. The price of local privacy: a $\sqrt{n}$ gap . . . 651
25.5. Frequency estimation: k-ary randomized response and unary encoding . . . 654
25.6. The shuffle model and amplification by shuffling . . . 659
25.7. Distributed DP: splitting the noise . . . 664
25.8. Code: frequency estimation in four trust models . . . 668
25.9. References . . . 670
25.10. Cheat sheet . . . 671

26. Lower bounds: what privacy costs **673**
26.1. Why this matters . . . 673
26.2. Toolkit: three facts every derivation uses . . . 675
26.3. Packing: pure DP pays the logarithm of the number of candidates . . . 678
26.4. Sample complexity: how many records privacy costs . . . 682
26.5. Fingerprinting: the $\sqrt{d}$ lower bound for approximate DP . . . 685
26.6. Tracing attacks: the lower bound is a membership inference attack . . . 691
26.7. Dinur–Nissim as a lower bound . . . 695
26.8. Code . . . 697
26.9. References . . . 698
26.10. Cheat sheet . . . 699

IV. Privacy accounting **701**
Who this part is for . . . 702
The common thread . . . 702
The running example . . . 704
How to read this part . . . 704
Notation . . . 704
What you will be able to do . . . 705

27. Rényi differential privacy **707**
27.1. Why this matters . . . 707
27.2. Toolkit: four facts from Part 1 . . . 709
27.3. Rényi differential privacy . . . 710
27.4. RDP curves of the basic mechanisms . . . 713
27.5. Composition adds curves . . . 717
27.6. Post-processing and group privacy . . . 720
27.7. The moments accountant is RDP in disguise . . . 723
27.8. From RDP to (ε, δ) . . . 725
27.9. Code: check every curve and every conversion . . . 729
27.10. References . . . 730
27.11. Cheat sheet . . . 730

28. From Rényi DP to (ε, δ) and to tests **732**
28.1. Why this matters . . . 732
28.2. Toolkit: four facts every derivation uses . . . 733
28.3. Where Mironov's conversion loses . . . 735
28.4. The improved conversion . . . 737
28.5. RDP as a constraint on tests . . . 741
28.6. Choosing the order . . . 745
28.7. The slack no conversion removes . . . 748
28.8. Code: check every conversion by computation . . . 752
28.9. References . . . 753
28.10. Cheat sheet . . . 753

29. Concentrated differential privacy **755**
29.1. Why this matters . . . 755
29.2. Toolkit: four facts used below . . . 756
29.3. The definition of zCDP . . . 757
29.4. Pure DP implies zCDP . . . 761
29.5. From zCDP to (ε, δ) . . . 766
29.6. Group privacy . . . 768
29.7. Approximate zCDP . . . 771
29.8. Subsampling, and truncated CDP . . . 773
29.9. Code: check every number . . . 778
29.10. References . . . 779
29.11. Cheat sheet . . . 780

30. f-DP and Gaussian differential privacy **782**
30.1. Why this matters . . . 782
30.2. Toolkit: three facts every derivation uses . . . 783
30.3. f-DP . . . 785
30.4. Gaussian differential privacy . . . 787
30.5. Duality with (ε, δ) . . . 790
30.6. Composition . . . 793
30.7. The central limit theorem for composition . . . 796
30.8. The subsampled Gaussian and DP-SGD . . . 800
30.9. RDP as a lossy summary of the curve . . . 803
30.10. Code: check every result by computation and simulation . . . 807
30.11. References . . . 808
30.12. Cheat sheet . . . 808

31. Amplification by subsampling **810**
31.1. Why this matters . . . 810
31.2. Toolkit: three facts every derivation uses . . . 811
31.3. Amplification for (ε, δ)-DP under Poisson sampling . . . 813
31.4. The sampling scheme must match the neighbouring relation . . . 816
31.5. RDP of the Poisson-subsampled Gaussian . . . 822
31.6. Small q and the moments accountant . . . 826
31.7. The running example: DP-SGD . . . 829
31.8. No tight amplification theorem for every mechanism . . . 832

31.9. Code: check every number by computation . . . 834
31.10. References . . . 835
31.11. Cheat sheet . . . 837

32. Numerical accounting with privacy loss distributions 839
32.1. Why this matters . . . 839
32.2. Toolkit: three facts the derivations reuse . . . 841
32.3. The privacy loss distribution . . . 842
32.4. Composition is convolution . . . 845
32.5. Discretization: pessimistic and optimistic . . . 847
32.6. Composition by FFT . . . 850
32.7. The subsampled Gaussian . . . 854
32.8. The running example: DP-SGD . . . 857
32.9. Code . . . 861
32.10. References . . . 862
32.11. Cheat sheet . . . 862

33. Amplification by shuffling and by iteration 864
33.1. Why this matters . . . 864
33.2. Toolkit: three facts every derivation uses . . . 866
33.3. Shuffled binary randomized response, exactly . . . 867
33.4. The clone bound for any local randomizer . . . 871
33.5. Amplification by iteration . . . 876
33.6. Hidden state: privacy that stops growing . . . 881
33.7. An exact example: noisy GD on a quadratic . . . 884
33.8. Code: check every number by computation . . . 887
33.9. References . . . 888
33.10. Cheat sheet . . . 889

34. Adaptive composition: filters, odometers and individual accounting 891
34.1. Why this matters . . . 891
34.2. Toolkit: stopping a supermartingale . . . 893
34.3. Why adaptively chosen parameters break the usual theorems . . . 895
34.4. Filters for pure differential privacy . . . 898
34.5. The Rényi filter . . . 903
34.6. Odometers . . . 907
34.7. Individual privacy accounting in DP-SGD . . . 910
34.8. Code: filters, odometers and per-record spend . . . 914
34.9. References . . . 915
34.10. Cheat sheet . . . 916

35. Choosing an accountant 918
35.1. Why this matters . . . 918
35.2. Every accountant on one run . . . 919
35.3. The inverse problem: noise for a target ε . . . 925
35.4. Where each accountant is tight . . . 928
35.5. Matching the accountant to the code . . . 931
35.6. What to report . . . 934

35.7. Checking your accountant . . . 937
35.8. Code . . . 939
35.9. References . . . 940
35.10. Cheat sheet . . . 941

V. Differential privacy in machine learning **943**
Who this part is for . . . 944
The common thread . . . 944
The running examples . . . 945
How to read this part . . . 946
Notation . . . 946
What you will be able to do . . . 947

36. Private empirical risk minimization **948**
36.1. Why this matters . . . 948
36.2. Toolkit: five facts every derivation uses . . . 950
36.3. Output perturbation . . . 951
36.4. Objective perturbation . . . 954
36.5. Gradient perturbation: noisy gradient descent . . . 957
36.6. How the three compare . . . 960
36.7. When the model is not convex . . . 963
36.8. Code: check every number by computation . . . 964
36.9. References . . . 965
36.10. Cheat sheet . . . 966

37. DP-SGD **968**
37.1. Why this matters . . . 968
37.2. Toolkit: four facts every derivation uses . . . 969
37.3. The algorithm and its guarantee . . . 970
37.4. Clipping: bias, noise and the choice of C . . . 973
37.5. Batch size and the number of steps at a fixed budget . . . 975
37.6. Per-example gradient norms without per-example gradients . . . 978
37.7. The running example: privacy against accuracy . . . 980
37.8. Code: check every number by computation . . . 982
37.9. References . . . 983
37.10. Cheat sheet . . . 984

38. Hyperparameter tuning and private selection **986**
38.1. Why this matters . . . 986
38.2. Toolkit: four facts every derivation uses . . . 987
38.3. Best of k is k times worse . . . 989
38.4. Paying by composition . . . 991
38.5. Random stopping: the best of a geometric number of runs . . . 992
38.6. Rényi bounds for a random number of runs . . . 995
38.7. Selecting by a noisy validation score . . . 1000
38.8. Tuning the running example . . . 1002
38.9. What to do and what to report . . . 1006

38.10. Code: check every number by computation . . . 1008
38.11. References . . . 1009
38.12. Cheat sheet . . . 1009

39. DP-FTRL and matrix factorization **1011**
39.1. Why this matters . . . 1011
39.2. Toolkit: four facts every derivation uses . . . 1013
39.3. Training is a stream of prefix sums . . . 1014
39.4. The matrix mechanism . . . 1016
39.5. Tree aggregation and DP-FTRL . . . 1019
39.6. The square-root factorization . . . 1021
39.7. Several participations . . . 1025
39.8. The running example: matrix mechanisms against DP-SGD . . . 1027
39.9. Code: check every number by computation . . . 1031
39.10. References . . . 1032
39.11. Cheat sheet . . . 1033

40. Private prediction: PATE and in-context learning **1034**
40.1. Why this matters . . . 1034
40.2. Toolkit: four facts every derivation uses . . . 1036
40.3. The noisy vote: LNMax and GNMax . . . 1037
40.4. How accurate is a noisy vote . . . 1040
40.5. Private answers against a private model . . . 1042
40.6. PATE: from private answers to a private model . . . 1045
40.7. Data-dependent analysis and Confident-GNMax . . . 1048
40.8. Private in-context learning . . . 1053
40.9. Code: check every number by computation . . . 1057
40.10. References . . . 1058
40.11. Cheat sheet . . . 1058

41. Public data and private fine-tuning **1060**
41.1. Why this matters . . . 1060
41.2. Toolkit: four facts every derivation uses . . . 1062
41.3. Why dimension costs, and when it does not . . . 1063
41.4. Good features: the last layer as private convex ERM . . . 1066
41.5. Low-rank adapters . . . 1068
41.6. Gradients near a public subspace . . . 1071
41.7. Pretraining and fine-tuning the running example . . . 1073
41.8. What public data does not buy . . . 1077
41.9. Code: check every number by computation . . . 1079
41.10. References . . . 1080
41.11. Cheat sheet . . . 1081

42. User-level differential privacy **1082**
42.1. Why this matters . . . 1082
42.2. Toolkit: three facts every derivation uses . . . 1083
42.3. The privacy unit . . . 1085
42.4. A user with k records in example-level DP-SGD . . . 1087

42.5. User-level DP-SGD . . . 1091
42.6. Contribution bounding . . . 1094
42.7. Heterogeneous users . . . 1096
42.8. Comparing the strategies on the running example . . . 1098
42.9. Code: check every number by computation . . . 1100
42.10. References . . . 1101
42.11. Cheat sheet . . . 1101

43. Disparate impact **1103**
43.1. Why this matters . . . 1103
43.2. Toolkit: four facts every derivation uses . . . 1104
43.3. Disparate impact, measured . . . 1106
43.4. Clipping reweights the groups . . . 1109
43.5. The noise floor of a small group . . . 1113
43.6. Measuring per-group accuracy . . . 1116
43.7. Mitigations and what they cost . . . 1119
43.8. Code: check every number by computation . . . 1122
43.9. References . . . 1123
43.10. Cheat sheet . . . 1123

44. Choosing ε **1125**
44.1. Why this matters . . . 1125
44.2. Toolkit: four facts every derivation uses . . . 1126
44.3. What one (ε, δ) pair says about a membership test . . . 1129
44.4. The whole curve of DP-SGD . . . 1130
44.5. The Bayesian reading . . . 1136
44.6. Reconstruction . . . 1139
44.7. Interpreting δ . . . 1141
44.8. Deployed values and their units . . . 1143
44.9. Justifying a choice . . . 1146
44.10. Code: check every number by computation . . . 1149
44.11. References . . . 1150
44.12. Cheat sheet . . . 1151

45. Auditing private training **1153**
45.1. Why this matters . . . 1153
45.2. Toolkit: four facts every derivation uses . . . 1155
45.3. From an attack to a lower bound on ε . . . 1156
45.4. Canaries and threat models . . . 1158
45.5. Tight audits of the mechanism . . . 1160
45.6. One-run auditing . . . 1165
45.7. Black-box audit of the trained network . . . 1167
45.8. Finding bugs . . . 1170
45.9. Code: run every audit . . . 1173
45.10. References . . . 1173
45.11. Cheat sheet . . . 1174

Appendices **1176**

A. Glossary **1176**

B. Index **1179**

Preface

Notes and code for a book on privacy-preserving machine learning, written alongside the study plan.

The figures and the numbers quoted in the text are produced by each chapter's script, in the repository's `code/` directory, one folder per part.

How this book is organized

The book has five parts, each building on the ones before it. Each part covers one phase of the study plan, and each of its chapters one topic of that phase.

1. **Prerequisites** collects the mathematics: distributions, concentration, divergences and hypothesis testing, then information theory, optimization and stability, and the basics of cryptography.
2. **Threat models and attacks** shows how models and data leak before any defence is introduced. It first attacks trained models, from one bit of membership up to whole training inputs, then attacks released data, from records with names removed to records hidden in crowds to aggregate statistics alone.
3. **Differential privacy foundations** defines the guarantee those attacks call for, builds its mechanisms, composes them, and ends with what privacy costs.
4. **Privacy accounting** tracks the guarantee through many steps of training, as accurately as possible.
5. **Differential privacy in machine learning** trains models with the guarantee: where the noise goes, how the knobs of noisy, clipped gradient descent trade privacy for accuracy, what else spends the budget, who pays for the noise, which ε to choose, and how to audit the result.

The appendices hold a glossary of acronyms and an index; the web edition also has the study plan. The exercises, with a worked solution for each, are in the exercise book, a companion website, and each chapter ends with a link to its page there.

Part I.

Prerequisites

This part collects the mathematics the rest of the book stands on. It contains no new ideas about privacy. Everything here is standard probability, statistics, optimization, information theory and cryptography, and you may have met most of it before. What is different is the selection and the angle: each topic is chosen because a later part needs it, and each is developed only as far as that need goes, but always with full derivations.

Who this part is for

You are expected to know undergraduate calculus and linear algebra, the basics of probability (random variables, expectation, variance, independence, conditional probability), and enough machine learning to recognise a loss function, a training set and stochastic gradient descent. You should be able to read short Python programs that use NumPy. Nothing beyond that is assumed. Every inequality, divergence or definition that the later parts rely on is stated and derived here, so you can check each step for yourself instead of taking it on trust.

If you already know a topic well, skim its cheat sheet and try its exercises in the exercise book. If they go smoothly, move on.

The common thread

Later parts study algorithms whose output is random, and keep asking the same few questions about them. How far is a random output from its expected value? How different are the output distributions on two slightly different inputs, and how would anyone tell them apart? How much does a trained model depend on a single training example? How much does an observation reveal about something hidden, and what can an observer with limited computing power learn from it? Each chapter of this part supplies the vocabulary and tools for one of these questions (Table 1).

Table 1.: The chapters of this part, the question each one answers, and the earlier chapters it uses.

Chapter	The question it answers	Builds on
Probability distributions	Which few distributions model counts, waiting times and additive noise, and what are their moments and tails?	—
Concentration inequalities	How far can a random quantity be from its mean, and with what probability?	1
Divergences	How different are two probability distributions, as a single number?	1, 2
Hypothesis testing	Given one observation, how well can you tell which of two distributions produced it?	1, 2, 3

Chapter	The question it answers	Builds on
Information theory	How much does an observation tell you about a hidden value, on average and for a single best guess?	3
Optimization	How fast does (stochastic) gradient descent converge, and how far does the minimizer move when one example changes?	1, 2
Algorithmic stability and generalization	Why does a model that depends little on each training example generalize?	2, 6
Cryptography basics	What does it mean that an efficient observer learns nothing it was not supposed to learn?	2, 3, 4

Two ideas run through all eight chapters. The first is the **likelihood ratio** $p(x)/q(x)$: how many times more likely an observation is under one distribution than under another. Divergences are averages of functions of it, the best test thresholds it, and mutual information is a divergence in disguise. The second is the **effect of one input**: how much one example shifts an average, a minimizer, a loss or a test statistic. Concentration inequalities bound it for sums, stability bounds it for learning algorithms, and the sensitivity of regularized empirical risk minimization makes it a concrete number. Watch for both as you read. Most of the later parts are built from them.

How to read this part

Order. Read chapters 1 to 4 in order, since each one uses the previous ones. Together they form the core of this part: tail bounds, then a number for how different two distributions are, then what that number means for anyone trying to tell them apart. Chapter 5, information theory, continues that thread: it measures how much an observation reveals, on average and for a single best guess. Then read chapters 6 and 7, which go together: how training moves when one example changes, and why that matters for generalization. Chapter 8 is used by three later chapters: chapter 9 on threat models uses its security games and advantage, chapter 15 on linkage uses its birthday bound, and chapter 25 on trust models builds secure aggregation from its secret sharing. A planned later part on cryptographic approaches will use all of it. You can defer it until you reach chapter 9.

Structure of a chapter. Every chapter follows the same shape, so you always know where to look:

- **Why this matters** states the question the chapter answers, the one idea that organizes it, and a table of its results and where each is used.
- **Toolkit** collects the few facts that the derivations share, each proved once.

- **One section per result**, each with the same arc: the context, the statement, a step-by-step derivation that names the fact used at each step, an interpretation that says when the result is tight and when it is loose, a worked example with real numbers, and two *check yourself* questions with their answers.
- **Code** runs a script that checks every quoted number by exact computation or simulation.
- **References** and a one-table **Cheat sheet** close the chapter.

The exercises, each with a worked solution, are collected in the exercise book, a companion website. Each chapter ends with a link to its exercises.

Abstract steps are made concrete. Where a derivation takes an abstract step, such as writing a probability as the expectation of an indicator or changing the distribution an expectation is taken under, the text works the step out on a small case first, usually a die or a coin with a table of every outcome. If a step feels like a trick, look for that table. It is there to show that the step is just bookkeeping.

Run the code. Each chapter has a script in `code/` that reproduces every number and figure in its text. From the repository root, run, for example,

```
uv run python code/part01-prerequisites/concentration.py
```

The scripts use fixed random seeds, so your output matches the tables in the text. Change the parameters and watch how the bounds respond. That is the quickest way to develop a feel for how loose each bound is.

Notation

The chapters share the conventions in Table 2. Where a chapter departs from them, it says so at its start.

Table 2.: Notation shared by the chapters of this part.

Symbol	Meaning
$\Pr[A]$, $\mathbb{E}[X]$, $\mathrm{Var}(X)$	Probability, expectation, variance
$\mathbf{1}_A$	Indicator of the event A: 1 if A happens, 0 otherwise
$\mathcal{N}(\mu, \sigma^2)$, $\mathrm{Lap}(b)$	Normal distribution with mean μ and variance σ^2; Laplace distribution with scale b
P, Q and p, q	Two distributions and their probability mass functions or densities
ln, log	Natural logarithm (in nats); base-2 logarithm (in bits), used in information theory
$\lVert\cdot\rVert$	Euclidean norm
$D = (z_1, \dots, z_n)$, $\ell(w, z)$	A training set of n examples; the loss of parameter w on example z

What you will be able to do

By the end of this part you should be able to do each of the following on paper:

- derive exponential tail bounds for sums of independent variables, and for normal and Laplace noise;
- compute the Kullback–Leibler (KL), Rényi, total-variation and hockey-stick divergences between two normal or two Laplace distributions, and explain why processing data cannot increase any of them;
- derive the optimal test between two distributions and its trade-off curve between the two kinds of error;
- prove convergence rates for stochastic gradient descent, and bound how far a regularized minimizer moves when one training example is replaced;
- prove that a uniformly stable algorithm generalizes;
- compute entropy, mutual information and min-entropy leakage, and say when the last two disagree;
- state a security definition as a game between two worlds, and prove a simple scheme secure with a hybrid argument or a simulator.

Each of these is a result derived in one of the chapters that follow.

1. Probability distributions

1.1. Why this matters

A handful of distributions do almost all the work in this book. Counts of events (errors, successes, examples drawn into a batch) are binomial, and Poisson when the events are rare. Additive noise is usually normal or Laplace. Every later chapter — concentration, divergences, hypothesis testing — is stated in terms of these few families. So you need to know each one well: its density, mean, variance, moment generating function (MGF), tails, and how it relates to the others.

The idea that ties the chapter together is that these families are not a zoo of unrelated formulas. They are built from one another. A binomial is a sum of coin flips. A Poisson is a binomial with many trials and a small success probability. An exponential is the waiting time for the first event in that same Poisson setting. A Laplace is two exponentials placed back to back. A normal is what any sum of many small independent pieces looks like. Because each family is built from simpler ones, its mean, variance and MGF can be computed from the pieces, and that is how this chapter derives them.

This chapter covers five families (Table 1.1):

Table 1.1.: The five families in this chapter, what each models, and the fact about it that later chapters use.

Family	Models	Key fact for later
Bernoulli and binomial	One yes/no outcome; the count of successes in n trials	Mean and variance add over independent trials
Poisson	Counts of rare events	Its MGF $e^{\mu(e^{\lambda}-1)}$ drives the Chernoff bound
Exponential	Waiting times; a building block	A difference of two exponentials is Laplace
Laplace	Symmetric noise with exponential tails	Its log density is piecewise linear, so shifted copies differ by a bounded factor
Normal	Sums of many small effects; symmetric noise with light tails	Closed under sums; a d-dimensional normal vector has norm $\approx \sigma\sqrt{d}$

A last section puts the normal and the Laplace side by side and asks how different a density is from a copy of itself moved sideways. That comparison is where the next chapters begin.

Each section follows the same arc: context, definition, derivations of the moments, interpretation, an example from machine learning, and a self-check.

1.2. Toolkit: what to know for each distribution

For every family below you will meet the same three summaries: its moments, its moment generating function and its tail. This section says what each one is, why it is worth computing, and the few rules for working with it.

1.2.1. Moments: where the distribution sits and how wide it is

The **mean** $\mathbb{E}[X]$ is the centre of mass of the distribution: the long-run average of many independent draws. The **variance** measures how far draws typically land from the mean. It squares each draw's distance from the mean, so that distances on either side count the same, and averages. Expanding the square $(X-\mathbb{E}X)^2 = X^2 - 2X\,\mathbb{E}X + (\mathbb{E}X)^2$ and averaging each term gives a second form that is often easier to compute, "mean of the square minus square of the mean":

$$\operatorname{Var}(X) = \mathbb{E}\big[(X-\mathbb{E}X)^2\big] = \mathbb{E}[X^2] - (\mathbb{E}X)^2. \tag{1.1}$$

Its square root, the **standard deviation**, is in the same units as X, so it is the number to compare with the mean. For a fair die, $\mathbb{E}X = (1+\cdots+6)/6 = 3.5$ and $\mathbb{E}X^2 = (1+4+\cdots+36)/6 = 91/6$, so $\operatorname{Var} X = 91/6 - 3.5^2 = 35/12 \approx 2.92$ and the standard deviation is about 1.71.

Three rules cover almost every calculation in this chapter. The first is that the mean of a sum is the sum of the means, $\mathbb{E}[X+Y] = \mathbb{E}X + \mathbb{E}Y$, always, even when X and Y are dependent. The reason is that an expectation is a weighted sum over outcomes, and a weighted sum of $X+Y$ splits into the weighted sum of X plus the weighted sum of Y.

The second rule says what scaling does to the spread: $\operatorname{Var}(cX) = c^2 \operatorname{Var}(X)$. Multiplying X by c multiplies every deviation $X - \mathbb{E}X$ by c, and squaring the deviation turns that into c^2.

The third rule is that variances add when the variables are independent: $\operatorname{Var}(X+Y) = \operatorname{Var} X + \operatorname{Var} Y$. To see where independence enters, expand the square in the definition. In general $\operatorname{Var}(X+Y) = \operatorname{Var} X + \operatorname{Var} Y + 2\operatorname{Cov}(X,Y)$, where the cross term is the **covariance** $\operatorname{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}X\,\mathbb{E}Y$. Independence makes $\mathbb{E}[XY] = \mathbb{E}X\,\mathbb{E}Y$, so the covariance vanishes and only the two variances remain.

The second and third rules together give the $1/\sqrt{n}$ law. The average of n independent copies of X is their sum times $1/n$. The sum has variance $n\operatorname{Var}(X)$ by the third rule, and multiplying by $1/n$ divides that by n^2 by the second, so the average has variance $n\operatorname{Var}(X)/n^2 = \operatorname{Var}(X)/n$. Its standard deviation is therefore the standard deviation of one draw divided by $\sqrt{n}$.

1.2.2. The moment generating function

The mean and variance summarize a distribution by two numbers. The moment generating function summarizes it by a whole function, and that function turns out to be the right tool for sums of independent variables and for tails. The idea is to average not X itself but an exponential of it, $e^{\lambda X}$, for a number λ of your choice. The **moment generating function** (MGF) of X is

$$M_X(\lambda) = \mathbb{E}[e^{\lambda X}]. \tag{1.2}$$

Here λ is a real number that *you* choose. It is not a parameter of the distribution; it is a dial. For each setting of the dial you get one number, $\mathbb{E}[e^{\lambda X}]$, and the MGF is the whole function $\lambda \mapsto M_X(\lambda)$. With $\lambda > 0$, the weight $e^{\lambda X}$ grows exponentially in X, so large values of X dominate the average; the larger λ, the further into the right tail it looks. With $\lambda < 0$ it looks at the left tail instead. At $\lambda = 0$ every weight is $e^0 = 1$, so $M_X(0) = 1$ for every distribution.

A concrete case: where e^λ comes from. Take a coin $X \sim \text{Bernoulli}(p)$, which is 1 with probability p and 0 otherwise. To compute $\mathbb{E}[e^{\lambda X}]$, list each outcome, the value $e^{\lambda X}$ takes there, and its probability (Table 1.2).

Table 1.2.: The MGF of a coin, outcome by outcome. The factor e^λ is just $e^{\lambda x}$ evaluated at $x = 1$.

Outcome x	$\Pr[X = x]$	$e^{\lambda x}$
0	$1 - p$	$e^{\lambda \cdot 0} = 1$
1	p	$e^{\lambda \cdot 1} = e^\lambda$

An expectation is the value at each outcome times its probability, added up, so

$$M_X(\lambda) = (1-p) \cdot 1 + p \cdot e^\lambda = 1 + p(e^\lambda - 1). \tag{1.3}$$

So the e^λ in the Bernoulli, binomial and Poisson MGFs is nothing mysterious: it is the weight $e^{\lambda x}$ of the outcome $x = 1$. For a count k the weight is $e^{\lambda k} = (e^\lambda)^k$, which is why e^λ keeps reappearing for the counting distributions. With $p = 0.3$ and $\lambda = 1$, for instance, $M_X(1) = 0.7 + 0.3e \approx 1.52$.

Why it "generates" moments. The k-th **moment** of X is $\mathbb{E}X^k$; the mean is the first. The MGF contains all of them, and the way to see this is to expand the exponential in its power series, $e^u = 1 + u + u^2/2! + u^3/3! + \cdots$, with $u = \lambda X$, and take the expectation of each term separately (the first rule of Section 1.2.1, extended to many terms):

$$M_X(\lambda) = \mathbb{E}\Big[1 + \lambda X + \frac{\lambda^2 X^2}{2!} + \frac{\lambda^3 X^3}{3!} + \cdots\Big] = 1 + \lambda\,\mathbb{E}X + \frac{\lambda^2}{2!}\mathbb{E}X^2 + \cdots. \tag{1.4}$$

The k-th moment $\mathbb{E}X^k$ is $k!$ times the coefficient of λ^k, and differentiating k times at $\lambda = 0$ picks it out, because differentiation kills the lower powers of λ and setting $\lambda = 0$ kills the higher ones: $M_X'(0) = \mathbb{E}X$, $M_X''(0) = \mathbb{E}X^2$. For the coin, expanding e^λ the same way gives $1 + p(e^\lambda - 1) = 1 + p\lambda + p\lambda^2/2! + \cdots$, so every moment equals p, as it must, since $X^k = X$ when $X \in \{0, 1\}$.

The log-MGF gives mean and variance directly. The derivatives of M_X give the raw moments $\mathbb{E}X$ and $\mathbb{E}X^2$, and the variance then needs a subtraction. Taking the logarithm first does the subtraction for you, and it will also turn products of MGFs into sums. Let

$\psi_X(\lambda) = \ln M_X(\lambda)$. The chain rule gives $\psi' = M'/M$, and the quotient rule applied to M'/M gives $\psi'' = M''/M - (M'/M)^2$. At 0, where $M = 1$, $M' = \mathbb{E}X$ and $M'' = \mathbb{E}X^2$,

$$\psi_X'(0) = \mathbb{E}X, \qquad \psi_X''(0) = \mathbb{E}X^2 - (\mathbb{E}X)^2 = \operatorname{Var} X. \tag{1.5}$$

The derivatives of ψ_X at 0 are called the **cumulants** of X: the first is the mean, the second the variance, and the higher ones measure finer features of the shape. For the Poisson below, ψ turns out so simple that both derivatives can be read off in one line.

Sums become products. The main reason to use MGFs is that adding independent variables multiplies their MGFs. The exponential turns the sum into a product, and independence lets the expectation of that product split. If X and Y are independent, then

$$M_{X+Y}(\lambda) = \mathbb{E}[e^{\lambda X}e^{\lambda Y}] = \mathbb{E}[e^{\lambda X}]\,\mathbb{E}[e^{\lambda Y}] = M_X(\lambda)M_Y(\lambda). \tag{1.6}$$

The first step is $e^{a+b} = e^a e^b$; the second uses independence, which lets the expectation of a product split into a product of expectations. In log form, $\psi_{X+Y} = \psi_X + \psi_Y$. This is how you get the binomial MGF from the Bernoulli one without touching the probability table, and it is the starting point of every exponential tail bound in Concentration inequalities.

The MGF pins down the distribution. If two distributions have the same MGF on an interval around 0, they are the same distribution. This is a standard theorem of probability (it is proved in Blitzstein & Hwang, listed in the references); its proof needs more analysis than this book uses, so take it as given. We use it to identify sums: compute the MGF of $X + Y$ as a product, recognise it as the MGF of a known family, and conclude that $X + Y$ belongs to that family ("the sum of independent Poissons is Poisson"). The MGF can be infinite for some λ, and for heavy-tailed distributions it is infinite for every $\lambda > 0$; the identification needs it finite near 0.

1.2.3. Tails: how fast large values become rare

The **tail** of X is $\Pr[X \geq t]$ as a function of t, for t well above the mean. The mean and variance describe the bulk of the distribution; the tail describes the rare, large outcomes, which are what a worst-case guarantee has to control. What matters is the *shape* of the decay, and three shapes recur. Evaluate each at $t = 5$. A polynomial decay such as $1/t^2$ gives $1/25 = 0.04$. An exponential decay such as e^{-t} gives e^{-5}, about 6.7×10^{-3}. A Gaussian decay such as $e^{-t^2/2}$ gives $e^{-12.5}$, about 3.7×10^{-6}.

The gap widens quickly as t grows, because the exponent of the Gaussian decay grows like t^2 while that of the exponential grows like t. Exponential tails are those of the exponential and Laplace families below; Gaussian tails are those of the normal.

The tail and the MGF are tied together, and the link is a one-line argument. Fix $\lambda > 0$. Whenever $X \geq t$, the weight $e^{\lambda X}$ is at least $e^{\lambda t}$, and it is never negative, so averaging gives $M_X(\lambda) = \mathbb{E}[e^{\lambda X}] \geq e^{\lambda t}\Pr[X \geq t]$. Rearranged, $\Pr[X \geq t] \leq M_X(\lambda)e^{-\lambda t}$. So if $M_X(\lambda)$ is finite for some $\lambda > 0$, the tail decays at least exponentially in t. Concentration inequalities turns this into explicit bounds by choosing λ well. Which decay a distribution has decides which concentration inequalities apply to it.

1.3. Bernoulli and binomial

1.3.1. Context

The simplest random quantity is a yes/no outcome: a test example is misclassified or not, a coin lands heads or not, a record is included in a sample or not. Count the yeses over n independent trials and you get a binomial. Every estimated rate in machine learning — accuracy, error rate, false-positive rate — is a binomial count divided by n.

1.3.2. Definitions

$X \sim \text{Bernoulli}(p)$ takes value 1 with probability p and 0 with probability $1-p$.

$S \sim \text{Binomial}(n, p)$ is the number of ones in n independent Bernoulli(p) trials. The probability of exactly k ones is the number of ways to place k ones among n trials, times the probability of any one such placement. That gives the probability mass function (PMF), the table of probabilities of each count,

$$\Pr[S = k] = \binom{n}{k} p^k (1-p)^{n-k}, \qquad k = 0, 1, \ldots, n. \tag{1.7}$$

Derivation of the PMF. Any particular sequence with exactly k ones has probability $p^k(1-p)^{n-k}$, because the trials are independent and the probability of a sequence is the product of the probabilities of its entries: p for each one and $1-p$ for each zero. There are $\binom{n}{k}$ such sequences — choose which k positions hold the ones — and they are disjoint events, so their probabilities add.

1.3.3. Moments

Bernoulli. $\mathbb{E}X = 1 \cdot p + 0 \cdot (1-p) = p$. Since $X^2 = X$, $\mathbb{E}X^2 = p$, so $\operatorname{Var} X = p - p^2 = p(1-p)$. The MGF, computed outcome by outcome in Table 1.2, is $M_X(\lambda) = (1-p) + pe^{\lambda} = 1 + p(e^{\lambda} - 1)$.

Binomial. Rather than work with the PMF, write the count as a sum of the trials, $S = X_1 + \cdots + X_n$ with independent Bernoullis, and apply the rules of the Toolkit to the sum. The mean of a sum is the sum of the n means, each p. The trials are independent, so the variances add, n times $p(1-p)$. And the MGF of an independent sum is the product of n identical Bernoulli MGFs. Together,

$$\mathbb{E}S = np, \qquad \operatorname{Var} S = np(1-p), \qquad M_S(\lambda) = \left(1 + p(e^{\lambda} - 1)\right)^n. \tag{1.8}$$

No sums over the PMF are needed: decomposing into simple pieces is almost always easier than computing with the PMF directly.

1.3.4. Interpretation

The variance $p(1-p)$ is a downward parabola in p that is zero at $p=0$ and $p=1$, so it is largest halfway between, at $p=\frac{1}{2}$, where it is $\frac{1}{2}\cdot\frac{1}{2}=\frac{1}{4}$, and small near 0 or 1. A rate near 0 is therefore estimated much more precisely than one near $\frac{1}{2}$. Bounds that use only the fact that each trial lies between 0 and 1, such as Hoeffding's inequality in Concentration inequalities, cannot see this difference, and it matters later.

The estimated rate $\hat{p}=S/n$ is the count scaled by $1/n$. By the scaling rules of Section 1.2.1 it has mean $np/n=p$ and variance $np(1-p)/n^2=p(1-p)/n$, so its standard deviation is $\sqrt{p(1-p)/n}$. Precision improves like $1/\sqrt{n}$: four times the data halves the error, since $\sqrt{4}=2$.

1.3.5. Example: estimating a classifier's error rate

A classifier makes an error on each test example independently with probability $p=0.05$. On $n=2{,}000$ test examples the error count is Binomial$(2000, 0.05)$, with mean $2000\times 0.05=100$ and standard deviation $\sqrt{2000\times 0.05\times 0.95}\approx 9.7$. Divided by n, that standard deviation is about $9.7/2000\approx 0.005$ of the rate, so the measured error rate is typically within about ± 0.5 percentage points of the true 5%. At $p=0.001$ the count has mean 2 and standard deviation $\sqrt{2000\times 0.001\times 0.999}\approx 1.4$: the *relative* error, $1.4/2$, is now about 70%, and a single test set can easily show 0 errors (that happens with probability $0.999^{2000}\approx e^{-2}$, about one test set in seven).

1.3.6. Check yourself

- Why is $\operatorname{Var}(X_1+\cdots+X_n)=n\operatorname{Var}(X_1)$ wrong if the X_i are copies of the same variable? *(Then the sum is nX_1, whose variance is $n^2\operatorname{Var}X_1$. Independence is what makes variances add.)*
- What is $\Pr[S=0]$ for Binomial(n,p), and roughly how large must n be before you expect to see at least one success? *($(1-p)^n\approx e^{-np}$, using $1-p\approx e^{-p}$ for small p; this is small only once np is around 1 or more, so you need n on the order of $1/p$.)*

1.4. Poisson

1.4.1. Context

When n is large and p is small, the binomial depends on n and p only through the mean $\mu=np$, as the limit below shows. The limit is the Poisson distribution: the law of rare events. It describes the number of errors in a large test set with a tiny error rate, the number of rare tokens in a document, or the size of a minibatch when each of many examples is included independently with small probability.

1.4.2. Definition

$N \sim \text{Poisson}(\mu)$ for $\mu > 0$ counts events whose expected number is μ. It takes values $k = 0, 1, 2, ...$ with

$$\Pr[N = k] = e^{-\mu}\frac{\mu^k}{k!}. \tag{1.9}$$

The factor $e^{-\mu}$ makes the probabilities add up to 1, since $\sum_k \mu^k/k! = e^{\mu}$ is the power series of the exponential.

1.4.3. The Poisson limit of the binomial

Hold the expected count μ fixed and let the number of trials grow, so each trial has success probability $p = \mu/n$. Put that into the binomial PMF for a fixed count k, write $\binom{n}{k} = n(n-1)\cdots(n-k+1)/k!$, and group the result into four factors, each of which has a simple limit as $n \to \infty$:

$$\binom{n}{k}\left(\frac{\mu}{n}\right)^k\left(1-\frac{\mu}{n}\right)^{n-k} = \underbrace{\frac{n(n-1)\cdots(n-k+1)}{n^k}}_{\to 1}\cdot\frac{\mu^k}{k!}\cdot\underbrace{\left(1-\frac{\mu}{n}\right)^n}_{\to e^{-\mu}}\cdot\underbrace{\left(1-\frac{\mu}{n}\right)^{-k}}_{\to 1}. \tag{1.10}$$

The first factor is a product of k fractions $n/n, (n-1)/n, ...$, each tending to 1, and there are only k of them. The middle factor does not depend on n. The third is the classic limit that defines e: its logarithm is $n\ln(1-\mu/n)$, and since $\ln(1-u) \approx -u$ for small u, that is close to $n\cdot(-\mu/n) = -\mu$. The last factor has a fixed exponent $-k$ and a base tending to 1. The limit is $e^{-\mu}\mu^k/k!$. Figure 1.1 shows the convergence: at $n = 10$ the binomial is visibly narrower than the Poisson, while at $n = 100$ they almost coincide.

1.4.4. Moments via the MGF

The MGF is the sum, over all counts k, of the weight $e^{\lambda k}$ times the probability of k. Gathering the powers of k into $(\mu e^{\lambda})^k$ leaves the power series of the exponential, evaluated at μe^{λ}:

$$M_N(\lambda) = \sum_{k\geq 0} e^{\lambda k}e^{-\mu}\frac{\mu^k}{k!} = e^{-\mu}\sum_{k\geq 0}\frac{(\mu e^{\lambda})^k}{k!} = e^{-\mu}e^{\mu e^{\lambda}} = \exp\left(\mu(e^{\lambda}-1)\right). \tag{1.11}$$

So the log-MGF is $\psi_N(\lambda) = \mu(e^{\lambda}-1)$. Its derivatives are all μe^{λ}, which equals μ at $\lambda = 0$. Since $\psi'(0)$ is the mean and $\psi''(0)$ the variance (Section 1.2.2),

$$\mathbb{E}N = \operatorname{Var} N = \mu. \tag{1.12}$$

Sums. Take independent $N_1 \sim \text{Poisson}(\mu_1)$ and $N_2 \sim \text{Poisson}(\mu_2)$. The MGF of the sum is the product of the two MGFs, and multiplying two exponentials adds their exponents, which gives $\exp((\mu_1+\mu_2)(e^{\lambda}-1))$. That is the Poisson MGF with mean $\mu_1+\mu_2$, and since the MGF pins down the distribution, $N_1 + N_2 \sim \text{Poisson}(\mu_1+\mu_2)$.

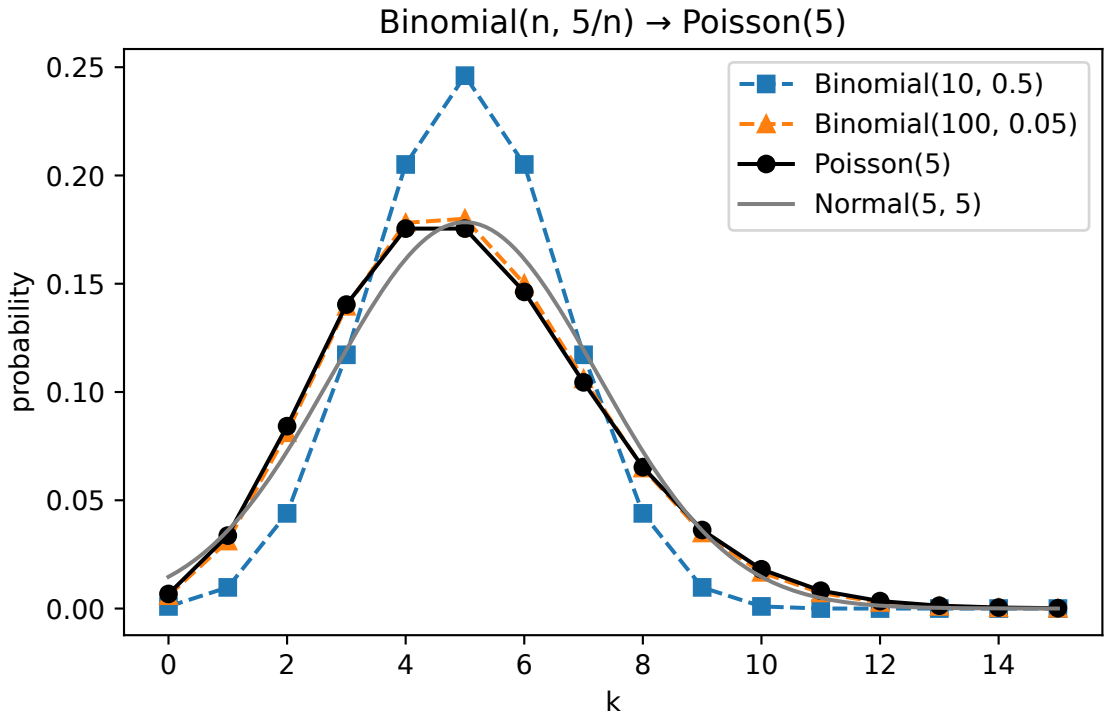


Figure 1.1.: Binomial$(n, 5/n)$ for $n = 10$ and 100, against Poisson(5) and its normal approximation.

1.4.5. Interpretation

Mean equals variance. The standard deviation is $\sqrt{\mu}$, so the spread relative to the mean, $\sqrt{\mu}/\mu = 1/\sqrt{\mu}$, shrinks only once you expect many events. With $\mu = 4$ the standard deviation is 2, so a count of 0 or 8, two standard deviations from the mean, is unremarkable.

The Poisson MGF bounds the Bernoulli MGF. The inequality $1 + x \le e^x$ holds for every x: the exponential is convex, and $1 + x$ is its tangent line at 0, which a convex curve always lies above. Apply it with $x = p(e^\lambda - 1)$:

$$M_{\text{Bern}(p)}(\lambda) = 1 + p(e^\lambda - 1) \le \exp(p(e^\lambda - 1)) = M_{\text{Poisson}(p)}(\lambda). \tag{1.13}$$

Now take independent Bernoullis with probabilities $p_1, \dots, p_n$ and mean $\mu = p_1 + \cdots + p_n$. The MGF of their sum is the product of their MGFs, each at most $\exp(p_i(e^\lambda - 1))$, and the product of those is $\exp(\mu(e^\lambda - 1))$. So the MGF of any sum of independent Bernoullis with mean μ is at most the Poisson(μ) MGF. This one-line inequality is the first step of the Chernoff bound in Concentration inequalities, which bounds the tail of a count through its MGF, as in Section 1.2.3.

Right tail. Count data are right-skewed: the count cannot go below 0 but can run far above the mean. For large k, $\Pr[N = k]$ decays like $(e\mu/k)^k$. To see this, use the bound $k! \ge (k/e)^k$ (it follows from $e^k = \sum_j k^j/j! \ge k^k/k!$), which gives $\Pr[N = k] \le e^{-\mu}\mu^k(e/k)^k \le (e\mu/k)^k$. Written as $e^{-k\ln(k/(e\mu))}$, the exponent grows like $k \ln k$, so the decay is faster than any exponential e^{-ck}, but slower than the $e^{-k^2/(2\mu)}$ of a normal with the same variance.

1.4.6. Example: minibatch sizes under independent sampling

Some training procedures form a minibatch by including each of $n = 60{,}000$ examples independently with probability $q = 0.01$. The batch size is Binomial$(60000, 0.01)$: many trials, each unlikely, so it is very close to Poisson(600), with mean $60000 \times 0.01 = 600$ and standard deviation $\sqrt{600}$, about 24.5. Batch sizes between 550 and 650, about two standard deviations either side, are typical; Concentration inequalities shows how unlikely anything outside $600 \pm 20\%$ is.

1.4.7. Check yourself

- Using only the MGF, show that $\mathbb{E}[N(N-1)] = \mu^2$. *(Differentiate M_N twice at 0 to get $\mathbb{E}N^2 = \mu^2 + \mu$, then subtract $\mathbb{E}N$.)*
- The binomial variance $np(1-p)$ tends to μ under the Poisson limit. Which factor disappears, and why? *($(1-p) \to 1$ because $p = \mu/n \to 0$.)*

1.5. Exponential

1.5.1. Context

The Poisson counts how many rare events land in a fixed window. Turn the question around: starting now, *how long* until the first one? A model in production makes a rare error, a cache entry is evicted, a request arrives. The answer is the exponential distribution, and it comes

out of the same construction as the Poisson: chop time into many short slots, give each slot a small, independent chance of holding an event, and let the slots shrink. For us it is mainly a building block: the Laplace distribution is two exponentials glued back to back, and the simplest exponential-type tail bounds are calculations with it.

1.5.2. Derivation: the waiting time in the Poisson limit

Fix a scale $b > 0$, which will turn out to be the average waiting time. Split each unit of time into n slots of width $1/n$. As the slots get finer, each one must get less likely to hold an event, or the number of events per unit of time would blow up. So choose the per-slot probability to keep the expected number of events per unit of time at $1/b$, whatever n is. With n slots per unit of time, that means each slot holds an event independently with probability

$$p = \frac{1}{bn}, \tag{1.14}$$

since then $np = 1/b$. This is the Poisson setup: in the Poisson limit we held the mean $\mu = np$ fixed and let $n \to \infty$, and here we hold the mean *rate* $1/b$ fixed in the same way.

1. **The wait in slots is geometric.** Let G be the index of the first slot that holds an event. $G > j$ means the first j slots are all empty, each with probability $1 - p$. The slots are independent, so these probabilities multiply:

$$\Pr[G > j] = (1-p)^j, \qquad j = 0, 1, 2, ... \tag{1.15}$$

   Table 1.3 lists the first few outcomes. This is the **geometric distribution**: the number of coin flips up to and including the first head.

Table 1.3.: The waiting time G, in slots, until the first event. Each row is one outcome. $G > j$ exactly when the first j slots are empty.

j	Slots $1, ..., j$ when $G = j$	$\Pr[G = j]$	$\Pr[G > j]$
1	event	p	$1-p$
2	empty, event	$(1-p)p$	$(1-p)^2$
3	empty, empty, event	$(1-p)^2p$	$(1-p)^3$
j	$j-1$ empty, then event	$(1-p)^{j-1}p$	$(1-p)^j$

2. **Convert slots to time.** Each slot lasts $1/n$, so the first event happens at time $T_n = G/n$. Time t covers $\lfloor nt \rfloor$ whole slots (the floor $\lfloor \cdot \rfloor$ rounds down to a whole number), so $T_n > t$ means those slots are all empty:

$$\Pr[T_n > t] = \Pr[G > \lfloor nt \rfloor] = \left(1 - \frac{1}{bn}\right)^{\lfloor nt \rfloor}. \tag{1.16}$$

3. **Let the slots shrink.** The goal is to bring this into the form $(1 - \mu/m)^m$, whose limit $e^{-\mu}$ we already know from the Poisson limit. Two rewrites do it. Write the exponent as $nt - r$,

where $0 \le r < 1$ is the fractional part of nt, and write $\frac{1}{bn}$ as $\frac{t/b}{nt}$, which is the same number. Then

$$\left(1-\frac{1}{bn}\right)^{\lfloor nt \rfloor} = \underbrace{\left(1-\frac{t/b}{nt}\right)^{nt}}_{\to e^{-t/b}} \cdot \underbrace{\left(1-\frac{1}{bn}\right)^{-r}}_{\to 1}. \tag{1.17}$$

The first factor is the limit $(1-\mu/m)^m \to e^{-\mu}$ with $\mu = t/b$ and $m = nt$ slots. The second has a bounded exponent and a base tending to 1. So the limiting waiting time T satisfies

$$\Pr[T > t] = e^{-t/b}, \qquad t \ge 0. \tag{1.18}$$

4. **Differentiate to get the density.** The probability that the wait is at most t is $\Pr[T \le t] = 1 - e^{-t/b}$, and a density is the derivative of this function, here $\frac{1}{b}e^{-t/b}$.

The same answer from the Poisson count. The first event comes after t exactly when no event falls in $[0, t]$. The number of events in $[0, t]$ is a count over $\lfloor nt \rfloor$ independent slots, Binomial($\lfloor nt \rfloor, \frac{1}{bn}$), with mean about t/b; by the Poisson limit it tends to Poisson(t/b), and $\Pr[\text{Poisson}(t/b) = 0] = e^{-t/b}$. Counting events in a window gives a Poisson count, and waiting for the first event gives an exponential time. Both come from the same slots.

1.5.3. Definition

$Y \sim \text{Exp}(b)$ with *scale* $b > 0$ has the density just derived,

$$f(y) = \frac{1}{b}e^{-y/b}, \qquad y \ge 0. \tag{1.19}$$

(Some texts use the rate $1/b$ instead. NumPy's `rng.exponential(b)` uses the scale.)

Figure 1.2 shows the limit for $b = 2$. With one slot per unit of time, each slot holds an event with probability $p = 1/(2 \cdot 1) = \frac{1}{2}$, so $\Pr[T_n > t] = (\frac{1}{2})^{\lfloor t \rfloor}$ is a staircase that halves at every integer. With five slots it already hugs $e^{-t/2}$. At $t = 4$ the script gives $\Pr[T_n > 4] = 0.0625$, 0.1216 and 0.1347 for $n = 1, 5, 100$, against $e^{-2} = 0.1353$. The first two follow from the formula of step 2: $(\frac{1}{2})^4 = 0.0625$, and with $n = 5$, $p = 0.1$ and $(0.9)^{20} = 0.1216$.

1.5.4. Moments and tail

Everything about the exponential follows from its density by short integrals.

The **tail** is the area under the density to the right of t: $\Pr[Y \ge t] = \int_t^\infty \frac{1}{b}e^{-y/b}\,dy = e^{-t/b}$, which is Equation 1.18 again.

The **mean** is easiest from the tail. A non-negative variable can be written as $Y = \int_0^\infty \mathbf{1}[Y \ge t]\,dt$, where $\mathbf{1}[\cdot]$ is 1 when the condition holds and 0 otherwise: the integrand is 1 for t from 0 up to Y, so the integral is the length Y. Taking expectations turns each indicator into a probability, $\mathbb{E}Y = \int_0^\infty \Pr[Y \ge t]\,dt = \int_0^\infty e^{-t/b}\,dt = b$. So b is indeed the average wait.

The **second moment** uses the Gamma integral $\int_0^\infty y^k e^{-y/b}\,dy = k!\,b^{k+1}$. Substituting $u = y/b$ turns it into $b^{k+1}\int_0^\infty u^k e^{-u}\,du$, and integrating by parts k times reduces the last integral to $k!$. With $k = 2$ and the factor $\frac{1}{b}$ from the density, $\mathbb{E}Y^2 = 2b^2$, so $\text{Var}\,Y = 2b^2 - b^2 = b^2$.

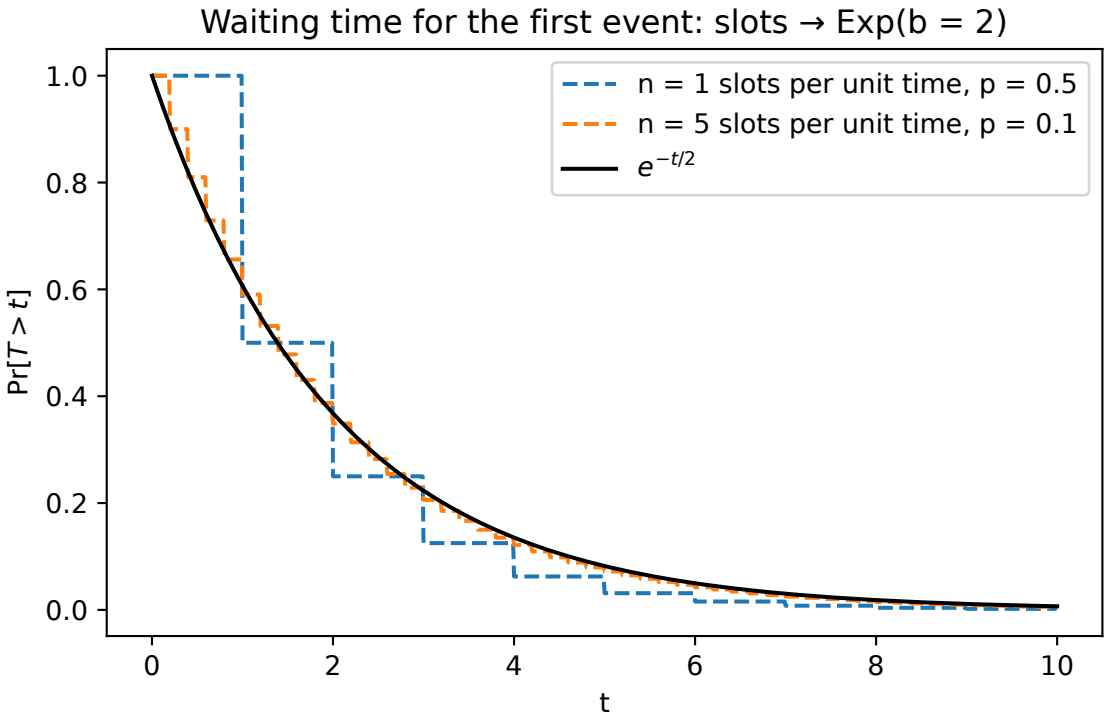


Figure 1.2.: $\Pr[T_n > t]$ for the slotted waiting time with $b = 2$ and $n = 1$ or 5 slots per unit of time, against the exponential tail $e^{-t/2}$.

The **MGF** multiplies the density by $e^{\lambda y}$, which combines the two exponents into $-(1/b-\lambda)y$. As long as $\lambda < 1/b$ that exponent is negative and the integral converges:

$$M_Y(\lambda) = \frac{1}{b}\int_0^\infty e^{-(1/b-\lambda)y}\,dy = \frac{1}{1-b\lambda}. \tag{1.20}$$

For $\lambda \geq 1/b$ the exponent is zero or positive, the integrand does not shrink, and the integral diverges. The MGF exists only on part of the line: the hallmark of an exponential, rather than Gaussian, tail.

1.5.5. Interpretation: memorylessness

The exponential forgets how long you have waited. Given that you have already waited s without an event, the chance of waiting at least t more is the same as the chance of waiting t from scratch. In symbols, the conditional probability is the ratio of the two tails,

$$\Pr[Y \geq s+t \mid Y \geq s] = e^{-(s+t)/b}/e^{-s/b} = e^{-t/b} = \Pr[Y \geq t]. \tag{1.21}$$

Having already waited s tells you nothing about the remaining wait. The derivation explains why: the slots after time s are independent of the empty slots before it, so the wait starts afresh. The geometric distribution is memoryless for the same reason. The exponential is also the only continuous distribution with this property: memorylessness says the tail $g(t) = \Pr[Y \geq t]$ satisfies $g(s+t) = g(s)g(t)$, and the only decreasing functions with that property are exponentials $e^{-t/b}$.

1.5.6. Example: time to a model's first error

A deployed classifier serves 50 requests per second and misclassifies each one independently with probability 10^{-4}. Each request is a slot, so errors arrive at rate $50 \times 10^{-4} = 0.005$ per second, and the time to the first error is approximately $\mathrm{Exp}(b)$ with $b = 1/0.005 = 200$ seconds. A 10-minute window is 600 seconds, so the probability of a clean window is $e^{-600/200} = e^{-3} \approx 0.050$. The exact slotted answer is $(1-10^{-4})^{30000}$, one factor for each of the $50 \times 600 = 30{,}000$ requests, which agrees to three decimals because p is tiny. Memorylessness says that after 10 error-free minutes, the expected wait for the first error is still 200 seconds.

1.5.7. Check yourself

- Show that $\mathbb{E}Y^k = k!\,b^k$ by differentiating the MGF k times. *($M^{(k)}(\lambda) = k!\,b^k(1-b\lambda)^{-k-1}$, which is $k!\,b^k$ at $\lambda = 0$.)*
- The slotted wait has $\mathbb{E}T_n = \mathbb{E}G/n = b$ for every n, and $\mathrm{Var}\,T_n = (1-p)b^2$. Which factor disappears in the limit, and why? *($(1-p) \to 1$ because $p = 1/(bn) \to 0$, just as for the binomial variance under the Poisson limit. The script's samples give variances 2.001, 3.596 and 3.984 for $n = 1, 5, 100$, against $b^2 = 4$.)*

1.6. Laplace

1.6.1. Context

The Laplace distribution is symmetric noise with exponential tails. Compared with a normal of the same variance it puts more mass near zero *and* more mass far out. In machine learning it appears as the noise model behind absolute-error loss: assuming Laplace noise and maximizing the likelihood gives least-absolute-deviation regression, just as normal noise gives least squares. In later parts it is one of the two standard noise distributions.

1.6.2. Definition

The Laplace density is an exponential density on each side of a centre μ, mirrored, with each side carrying half the probability. $X \sim \mathrm{Lap}(\mu, b)$ with location μ and scale $b > 0$ has density

$$f(x) = \frac{1}{2b} e^{-|x-\mu|/b}. \tag{1.22}$$

We write $\mathrm{Lap}(b)$ for $\mathrm{Lap}(0, b)$. The cumulative distribution function (CDF), the probability $\Pr[X \le x]$, comes from integrating the density up to x. Left of the centre only the left half contributes, and right of it you subtract the remaining right tail from 1: the CDF is $\frac{1}{2}e^{(x-\mu)/b}$ for $x < \mu$ and $1 - \frac{1}{2}e^{-(x-\mu)/b}$ for $x \ge \mu$.

1.6.3. Moments and tail

Take $\mu = 0$. The density takes the same value at x and $-x$, so the size $|X|$ collects the density from both sides: its density is $2 \cdot \frac{1}{2b} e^{-x/b} = \frac{1}{b} e^{-x/b}$ on $x \ge 0$, which is the $\mathrm{Exp}(b)$ density. So $|X| \sim \mathrm{Exp}(b)$, and everything follows from the exponential.

The tail of $|X|$ is the exponential tail, $\Pr[|X| \ge t] = e^{-t/b}$. The event $|X| \ge t$ is made of two equally likely pieces, $X \ge t$ and $X \le -t$, because the density is mirror-symmetric about 0, so one side alone has half of it: $\Pr[X \ge t] = \frac{1}{2}e^{-t/b}$. The mean absolute deviation is the exponential mean, $\mathbb{E}|X| = b$.

For the variance, note that $X^2 = |X|^2$, so $\mathbb{E}X^2$ is the exponential second moment $2b^2$. The mean is 0 by symmetry, so $\operatorname{Var} X = 2b^2$. Turned around, a Laplace variable with variance σ^2 has scale $b = \sigma/\sqrt{2}$.

For the MGF, split the integral at 0 into the two exponential halves. On the right, $\frac{1}{2b}\int_0^\infty e^{\lambda x - x/b}\,dx = \frac{1}{2b} \cdot \frac{1}{1/b - \lambda}$, which needs $\lambda < 1/b$; the left half is the same with $-\lambda$ in place of λ, which needs $\lambda > -1/b$. So for $|\lambda| < 1/b$, putting the two fractions over the common denominator $1/b^2 - \lambda^2$,

$$M_X(\lambda) = \frac{1}{2b}\left(\frac{1}{1/b - \lambda} + \frac{1}{1/b + \lambda}\right) = \frac{1}{1 - b^2\lambda^2}. \tag{1.23}$$

1.6.4. A Laplace variable is a difference of exponentials

The Laplace is two exponentials back to back, which suggests building it by subtracting one exponential from another: the positive one supplies the right half, the negated one the left half. The MGF confirms it. Let $Y_1, Y_2 \sim \mathrm{Exp}(b)$ be independent. Negating a variable flips the dial, $\mathbb{E}[e^{\lambda(-Y_2)}] = M_{Y_2}(-\lambda)$, and the MGF of an independent sum is the product, so the MGF of $Y_1 - Y_2$ is

$$M_{Y_1}(\lambda)\, M_{Y_2}(-\lambda) = \frac{1}{1-b\lambda} \cdot \frac{1}{1+b\lambda} = \frac{1}{1-b^2\lambda^2}, \tag{1.24}$$

which is the Laplace MGF. Since the MGF pins down the distribution, $Y_1 - Y_2 \sim \mathrm{Lap}(b)$. The script checks this numerically: over a million samples, the empirical CDF of $\mathrm{Exp}(1) - \mathrm{Exp}(1)$ is within 0.0006 of the $\mathrm{Lap}(1)$ CDF everywhere on $[-5, 5]$ (Figure 1.3).

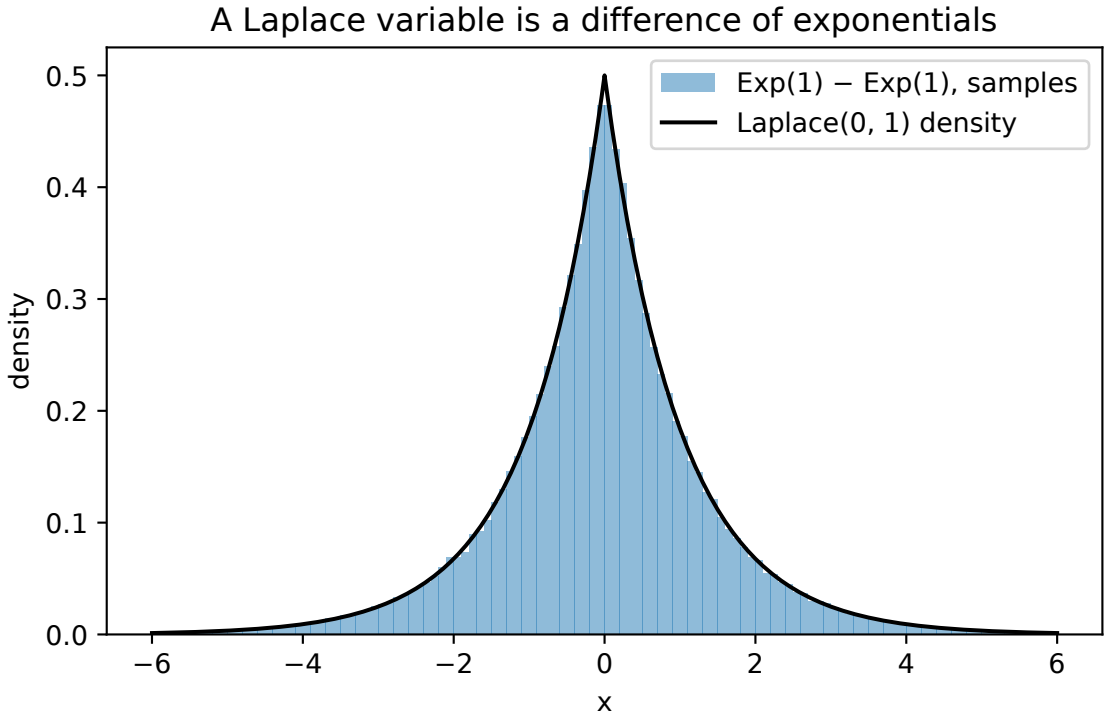


Figure 1.3.: Histogram of $\mathrm{Exp}(1) - \mathrm{Exp}(1)$ samples against the $\mathrm{Lap}(0, 1)$ density.

1.6.5. Interpretation: Laplace against normal

Put a Laplace and a normal side by side with the same variance, here 2, which means a normal with $\sigma = \sqrt{2}$ and a Laplace with $b = 1$ (Figure 1.4, left). The Laplace has a sharp peak at 0 and fatter tails. On a log scale (right) the difference is structural. Taking the logarithm of each density removes the exponential and leaves its exponent, plus a constant: the normal log density is a parabola, $-x^2/(2\sigma^2)$, while the Laplace log density is two straight lines, $-|x|/b$. Far out, a parabola always falls below a line, so the normal's tails are much lighter.

The same fact explains why the Laplace MGF exists only for $|\lambda| < 1/b$. Inside the MGF integral the weight $e^{\lambda x}$ multiplies the density, so the exponents add: $\lambda x - |x|/b$. For large positive x that

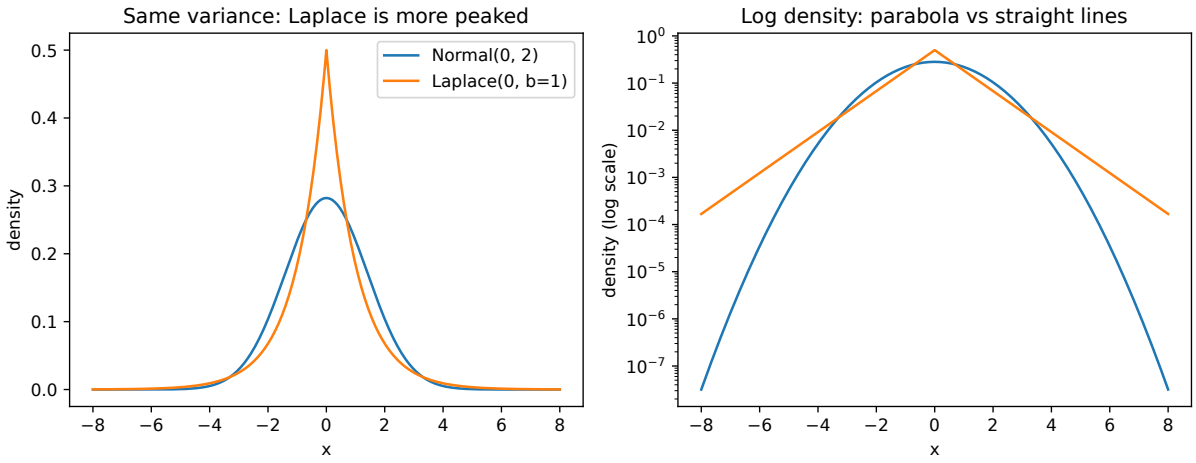


Figure 1.4.: Normal and Laplace densities with variance 2, on linear and log scales.

is $(\lambda - 1/b)x$, which goes to $-\infty$, and keeps the integral finite, only when $\lambda < 1/b$; the left side needs $\lambda > -1/b$. A quadratic decay $-x^2/(2\sigma^2)$ beats the linear term λx for every λ, which is why the normal MGF is finite everywhere.

1.6.6. Example: absolute-error regression

Suppose $y_i = w^\top x_i + \varepsilon_i$ with $\varepsilon_i \sim \mathrm{Lap}(b)$ independent. The likelihood of the data is the product of the n Laplace densities of the residuals $y_i - w^\top x_i$, and its logarithm is the sum of their log densities, $-n \ln(2b) - \frac{1}{b} \sum_i |y_i - w^\top x_i|$. Only the second term depends on w, and it enters with a minus sign, so the maximum-likelihood w minimizes the sum of absolute errors. With no features this estimates the location by the *median*: moving a candidate location m to the right lowers the absolute error of every point above m and raises it for every point below, so the sum stops decreasing exactly when half the points are on each side. The median is far less sensitive to outliers than the mean, because moving one extreme point further out does not change which point is in the middle. That insensitivity is the same heavy-tail property seen in the figure: under a Laplace model, large residuals are not surprising enough to dominate the fit.

1.6.7. Check yourself

- If $X \sim \mathrm{Lap}(b)$, what is $\Pr[|X| \geq 3b]$, and what is the corresponding probability for a normal with the same variance? *($e^{-3b/b} = e^{-3} \approx 0.050$; for the normal, $\sigma = \sqrt{2}b$, so $3b$ is $3/\sqrt{2}$ standard deviations and $\Pr[|Z| \geq 3/\sqrt{2}] \approx 0.034$. The Laplace wins this far out, and the gap grows quickly.)*
- Derive $\mathbb{E}|X| = b$ from the exponential. *($|X| \sim \mathrm{Exp}(b)$.)*

1.7. Normal

1.7.1. Context

The normal distribution appears whenever many small independent effects add up — that is the central limit theorem (CLT) — and it is the default model for measurement noise, weight initialization and random projections. It is the light-tailed counterpart of the Laplace: closed under sums and scalings, with an MGF that is finite everywhere and a tail that decays like $e^{-t^2/2}$.

1.7.2. Definition

The normal density is the bell curve: its logarithm is a downward parabola centred at the mean μ, with width set by the standard deviation σ. $X \sim \mathcal{N}(\mu, \sigma^2)$ has density

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}. \tag{1.25}$$

Standardization: subtracting the mean and dividing by the standard deviation gives $Z = (X - \mu)/\sigma \sim \mathcal{N}(0, 1)$, the standard normal, with density φ and CDF Φ. So $\Phi(t)$ is the probability that a standard normal is below t, and every normal probability reduces to it: $\Pr[X \le x] = \Phi((x-\mu)/\sigma)$. There is no closed form for Φ. In code, use the complementary error function erfc that every maths library provides, through $\Phi(x) = \frac{1}{2}\operatorname{erfc}(-x/\sqrt{2})$. It stays accurate far into the tail, where computing a small tail as $1 - \Phi$ by subtraction would lose all its digits.

1.7.3. MGF and moments

The MGF integrates $e^{\lambda x}$ against the density, so the two exponents add, and the trick is to recognize the sum as another normal density. Take $\mu = 0$ first. Completing the square in the combined exponent gives $\lambda x - \frac{x^2}{2\sigma^2} = -\frac{(x-\lambda\sigma^2)^2}{2\sigma^2} + \frac{\lambda^2\sigma^2}{2}$. The first term is the exponent of a normal density centred at $\lambda\sigma^2$, which integrates to 1 against the normalizer $\frac{1}{\sigma\sqrt{2\pi}}$, so only the factor $e^{\lambda^2\sigma^2/2}$ survives. A mean μ adds the constant μ to X, which multiplies $e^{\lambda X}$ by $e^{\lambda\mu}$. So

$$M_X(\lambda) = e^{\mu\lambda + \sigma^2\lambda^2/2}. \tag{1.26}$$

Then $\psi(\lambda) = \mu\lambda + \sigma^2\lambda^2/2$ gives $\psi'(0) = \mu$ and $\psi''(0) = \sigma^2$. Because ψ is a quadratic, its third and higher derivatives are zero, so all higher cumulants (the higher derivatives of the log-MGF at 0, Section 1.2.2) are zero.

1.7.4. Sums and linear maps

Scaling and adding independent normals gives another normal. The MGF shows it: scaling X_i by a_i evaluates its MGF at $a_i\lambda$, and adding independent variables multiplies MGFs, which adds the exponents. For independent $X_i \sim \mathcal{N}(\mu_i, \sigma_i^2)$, the MGF of $\sum_i a_i X_i$ is therefore $\exp(\lambda \sum_i a_i\mu_i +$

$\frac{\lambda^2}{2}\sum_i a_i^2\sigma_i^2$). That is the normal MGF with mean $\sum_i a_i\mu_i$ and variance $\sum_i a_i^2\sigma_i^2$, and since the MGF pins down the distribution,

$$\sum_i a_i X_i \sim \mathcal{N}\Big(\sum_i a_i\mu_i,\ \sum_i a_i^2\sigma_i^2\Big). \tag{1.27}$$

Among distributions with finite variance, the normal is the only one for which every linear combination of independent copies stays in the same family, up to location and scale (a classical result of Lévy on what are called stable distributions). The Laplace, for instance, fails: the sum of two independent Lap(b) variables has MGF $1/(1-b^2\lambda^2)^2$, which is not a Laplace MGF.

1.7.5. The normal in many dimensions

$X \sim \mathcal{N}(0, \sigma^2 I_d)$ means the d coordinates are independent $\mathcal{N}(0,\sigma^2)$; I_d is the $d \times d$ identity matrix. Two facts are used constantly.

Rotation invariance. The noise looks the same in every direction, so a question about one direction is a one-dimensional question. To see it, project X onto a unit vector u, one with $\sum_i u_i^2 = 1$. The projection $\langle u, X\rangle = \sum_i u_i X_i$ is a linear combination of independent normals, so by the rule above $\langle u, X\rangle \sim \mathcal{N}(0, \sigma^2\sum_i u_i^2) = \mathcal{N}(0,\sigma^2)$, whatever the direction u.

The norm concentrates at $\sigma\sqrt{d}$. The squared length $\|X\|^2 = \sum_i X_i^2$ is a sum of d independent terms, so its mean and variance are d times those of one term. Write $X_i = \sigma Z$ with Z standard normal. Then $\mathbb{E}X_i^2 = \sigma^2$, and $\operatorname{Var} X_i^2 = \sigma^4(\mathbb{E}Z^4 - (\mathbb{E}Z^2)^2) = \sigma^4(3-1) = 2\sigma^4$. The value $\mathbb{E}Z^4 = 3$ comes from the MGF: $e^{\lambda^2/2} = 1 + \lambda^2/2 + \lambda^4/8 + \cdots$, and the coefficient of λ^4 times $4!$ is $24/8 = 3$. So

$$\mathbb{E}\|X\|^2 = d\sigma^2, \qquad \operatorname{Var}\|X\|^2 = 2d\sigma^4, \qquad \frac{\|X\|^2}{d\sigma^2} = 1 \pm O\Big(\frac{1}{\sqrt{d}}\Big). \tag{1.28}$$

The last statement divides by the mean: the ratio has mean 1 and standard deviation $\sqrt{2d\sigma^4}/(d\sigma^2) = \sqrt{2/d}$, and $O(1/\sqrt{d})$ means "at most a constant times $1/\sqrt{d}$". In high dimension a normal vector is almost never near the origin: it lives on a thin shell of radius $\sigma\sqrt{d}$. The script measures $\|X\|/\sqrt{d}$ for $\sigma = 1$ (Table 1.4 and Figure 1.5):

Table 1.4.: Mean and standard deviation of $\|X\|/\sqrt{d}$ over 2,000 draws of $X \sim \mathcal{N}(0, I_d)$.

d	1	10	100	1,000	10,000
mean of $\|X\|/\sqrt{d}$	0.802	0.977	0.998	1.000	1.000
std of $\|X\|/\sqrt{d}$	0.611	0.220	0.069	0.022	0.007

The standard deviation falls like $1/\sqrt{2d}$. That is half the $\sqrt{2/d}$ found for the squared norm, because taking a square root halves small relative deviations: $\sqrt{1+e} \approx 1 + e/2$. At $d = 10{,}000$, for instance, $1/\sqrt{20000} \approx 0.007$. At $d = 1$ the mean is $\mathbb{E}|Z| = 2\int_0^\infty z\varphi(z)\,dz = 2/\sqrt{2\pi} = \sqrt{2/\pi} \approx 0.798$.

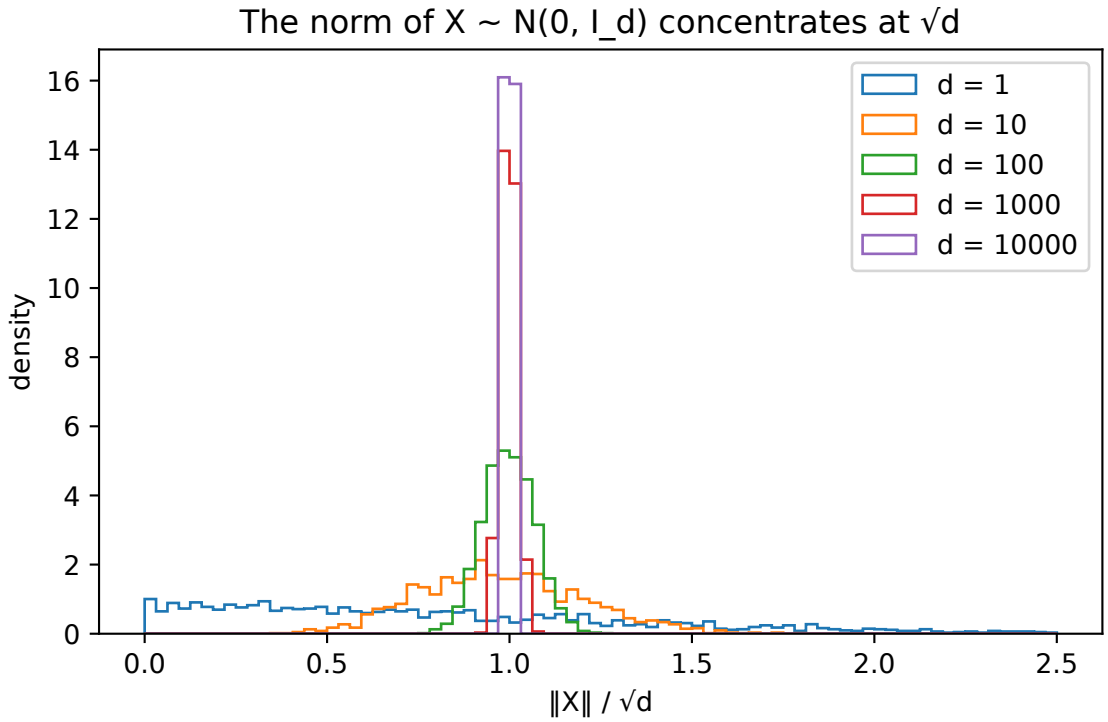


Figure 1.5.: Histograms of $\|X\|/\sqrt{d}$ for $X \sim \mathcal{N}(0, I_d)$ and d from 1 to 10,000.

1.7.6. The central limit theorem

Take a sum of n independent draws from the same distribution, with mean μ and finite variance σ^2. By the rules of Section 1.2.1 the sum has mean $n\mu$ and standard deviation $\sigma\sqrt{n}$. Subtract the first and divide by the second, and the result always has mean 0 and variance 1. The central limit theorem says more: whatever the distribution of each draw, the *shape* of this standardized sum approaches the standard normal. In symbols, if $X_1, X_2, \ldots$ are independent and identically distributed with mean μ and finite variance σ^2, then

$$\frac{X_1 + \cdots + X_n - n\mu}{\sigma\sqrt{n}} \to \mathcal{N}(0,1) \quad \text{in distribution,} \tag{1.29}$$

which means that its CDF approaches Φ at every point. The theorem holds whatever the shape of the X_i, and the best way to see this is to start from a distribution that looks nothing like a normal. Take $X_i \sim \text{Exp}(1)$, so $\mu = \sigma^2 = 1$. One draw is badly skewed: it can't go below 0, its density is highest at 0, and it has a long right tail. The sum $S_n = X_1 + \cdots + X_n$ has a known density, the Gamma$(n, 1)$ density $x^{n-1}e^{-x}/(n-1)!$ (its MGF is $(1-\lambda)^{-n}$, the product of n exponential MGFs), so you can watch the standardized sum $(S_n - n)/\sqrt{n}$ change shape as n grows (Figure 1.6).

At $n = 1$ it is the exponential itself, shifted to mean 0. The hard wall at $z = (0-1)/1 = -1$ is the constraint $X \geq 0$. At $n = 2$ the wall moves to $(0-2)/\sqrt{2} = -\sqrt{2}$, and the peak rounds off, because two draws are rarely both near 0. At $n = 5$ the bump is in the right place but leans left, with a longer right tail. At $n = 30$ the curve is close to $\mathcal{N}(0, 1)$, though a small lean to the left remains.

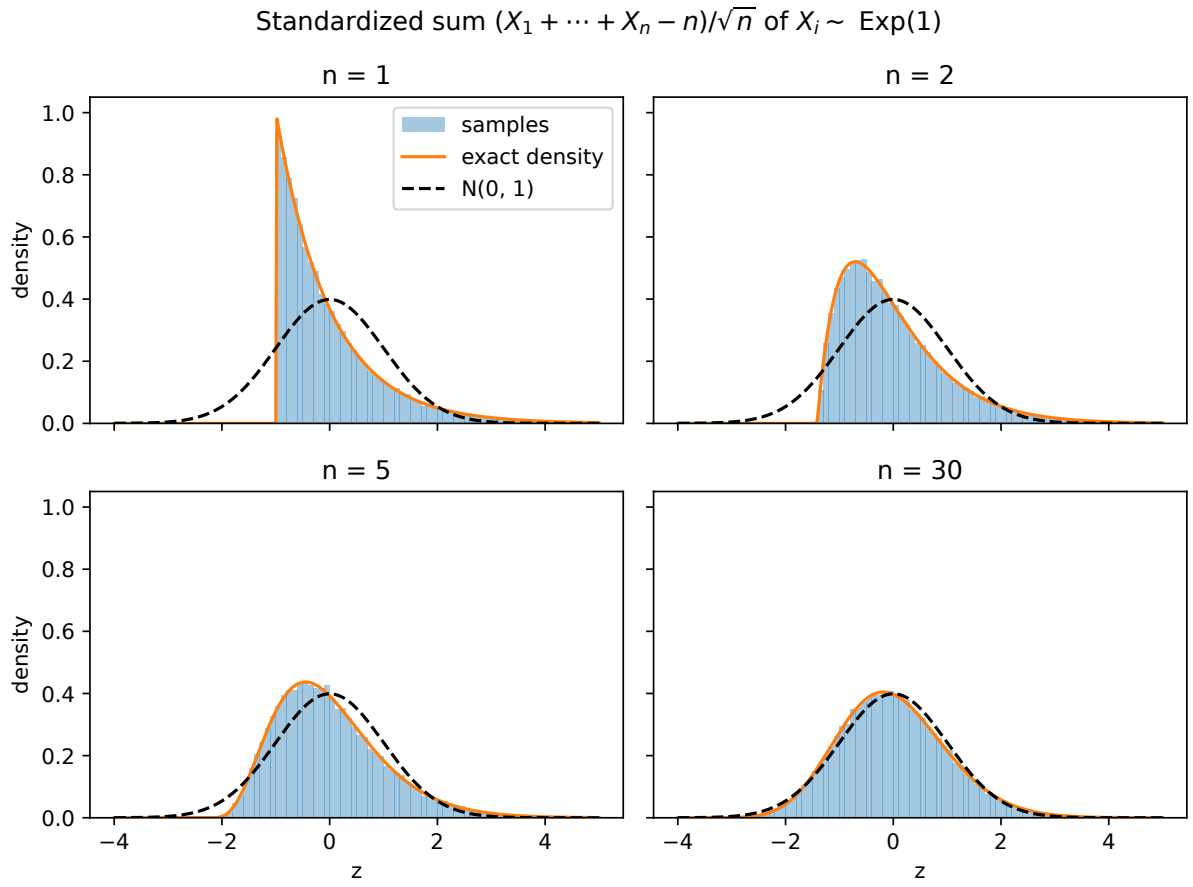


Figure 1.6.: The standardized sum of n independent Exp(1) variables for $n = 1, 2, 5, 30$: histogram of 50,000 simulated sums, the exact density from Gamma$(n, 1)$, and the $\mathcal{N}(0, 1)$ density. The skewed exponential becomes a normal step by step.

The leftover lean is measured by the **skewness** $\mathbb{E}[(X-\mu)^3]/\sigma^3$, the average cubed deviation in units of the standard deviation. Cubing keeps the sign, so a long right tail makes it positive; it is 0 for every normal, which is symmetric. The third central moment, like the variance, adds over independent terms: expand $\mathbb{E}[(\sum_i (X_i-\mu))^3]$, and every cross term contains some $\mathbb{E}[X_i-\mu]=0$ as a factor. For Exp(1) the third central moment is 2: with $\mathbb{E}Y^k = k!$, it is $\mathbb{E}Y^3 - 3\mathbb{E}Y^2 + 3\mathbb{E}Y - 1 = 6-6+3-1=2$. So S_n has third central moment $2n$ and standard deviation $\sqrt{n}$, and its skewness is $2n/n^{3/2} = 2/\sqrt{n}$. The skewness falls, but only like $1/\sqrt{n}$. The largest gap between the CDF of the standardized sum and the normal CDF, which the script computes exactly from the Gamma CDF, falls at the same rate (Table 1.5).

Table 1.5.: How far the standardized sum of n Exp(1) variables is from $\mathcal{N}(0,1)$. The CDF gap is the maximum of $|F_n(z)-\Phi(z)|$ over $z \in [-5,5]$ in steps of 0.01.

n	Skewness $2/\sqrt{n}$	Largest CDF gap to $\mathcal{N}(0,1)$
1	2.000	0.1587
2	1.414	0.0945
5	0.894	0.0596
30	0.365	0.0243

Multiply each gap by $\sqrt{n}$ and you get 0.159, 0.134, 0.133 and 0.133: the product settles to a constant, so the error decays like $1/\sqrt{n}$. This rate holds in general. The Berry–Esseen theorem bounds the gap by $C\,\mathbb{E}|X-\mu|^3/(\sigma^3\sqrt{n})$ for a universal constant $C < 0.48$ (Shevtsova, 2011). The numerator is a third moment, like the skewness, so the more skewed the summands, the more terms you need.

This is also why the normal curve in Figure 1.1 fits so well: Poisson(5) is a sum of five independent Poisson(1) variables, since Poisson means add, so the CLT already applies to it. A common rule of thumb is that the normal approximation to a binomial is reasonable once $np(1-p) \gtrsim 10$. The CLT is a statement about the *bulk*. In the far tails, where worst-case guarantees operate, it can be badly wrong, because the gap it controls is an absolute error of order $1/\sqrt{n}$, far larger than the tiny tail probabilities themselves; that is the job of the concentration inequalities in the next chapter.

1.7.7. Example: noise in high dimension

Add $\mathcal{N}(0, \sigma^2 I_d)$ noise to a model with $d = 10^6$ parameters and $\sigma = 0.01$. The noise vector has norm about $\sigma\sqrt{d} = 0.01 \times 1000 = 10$, almost surely within a fraction of a percent of that: the relative standard deviation is $1/\sqrt{2d} = 1/\sqrt{2\times 10^6}$, under 0.1%. Yet by rotation invariance its projection onto any fixed unit vector is only $\mathcal{N}(0, 0.01^2)$. Large in norm, small in every direction: both facts come from rotation invariance and norm concentration.

1.7.8. Check yourself

- If $X \sim \mathcal{N}(0, \sigma^2 I_d)$ and u, v are orthogonal unit vectors, are $\langle u, X\rangle$ and $\langle v, X\rangle$ independent? *(Yes. They are jointly normal with covariance $\sigma^2\langle u, v\rangle = 0$, and for jointly normal variables zero covariance means independence.)*

- Why does the mean of $\|X\|/\sqrt{d}$ approach 1 from below? *(Jensen's inequality says that for a concave function g, such as the square root, $\mathbb{E}g(Y) \le g(\mathbb{E}Y)$. With $Y = \|X\|^2$ this gives $\mathbb{E}\|X\| \le \sqrt{\mathbb{E}\|X\|^2} = \sqrt{d}$.)*

1.8. Comparing a density with its shifted copy

1.8.1. Context

You take a noisy measurement $x = \mu + X$ of an unknown value μ: a sensor reading, a timing measurement, or a statistic computed from a random sample. The noise X has density f centred at 0. Suppose μ can only be one of two values, 0 or s. The number $s \in \mathbb{R}$ is the **shift**, the distance between the two candidate values. It is fixed and known, and it may be positive or negative.

Each candidate gives x its own density. If $\mu = 0$, then $x = X$, with density $f(x)$. If $\mu = s$, then $x = s + X$, with density $f(x - s)$: the same curve, moved sideways by s.

The second claim is the step to check. The CDF of $s + X$ at a point t is $\Pr[s + X \le t] = \Pr[X \le t - s] = F(t - s)$, where F is the CDF of X, and differentiating in t gives the density $f(t - s)$. So $f(x - s)$ is the graph of f moved right by s: its value at x is the value f had at $x - s$. With $s = 1$, for instance, $f(x - 1)$ at $x = 1$ equals $f(0)$, so the shifted copy has its peak at 1.

To decide between the two candidates from one observation x, compare the two densities at x. Their ratio $f(x)/f(x - s)$ says how many times more likely x is under $\mu = 0$ than under $\mu = s$. Taking its logarithm turns the ratio into a difference, and since both densities below are exponentials of something, it leaves just the difference of the exponents. That logarithm is the **log ratio**

$$L(x) = \ln \frac{f(x)}{f(x - s)} = \ln f(x) - \ln f(x - s). \tag{1.30}$$

It is positive when x favours $\mu = 0$, negative when x favours $\mu = s$, and zero when x says nothing.

The question this section answers is: **how large can $|L(x)|$ get?** If it is bounded, no single observation can make one candidate much more likely than the other. If it is unbounded, some observations settle the question almost for certain. The normal and the Laplace give opposite answers. Later parts compare distributions in exactly this way, and the next chapters build on it: divergences average the log ratio over x, and hypothesis testing uses it to decide.

1.8.2. Statement

The answer follows from the shapes of the two log densities. A normal log density is a parabola, and the difference of a parabola and the same parabola shifted is a straight line, which grows without bound. A Laplace log density is a pair of straight lines with slopes $\pm 1/b$, and the difference of two such shapes can never exceed what the shift moves it, $|s|/b$. For any shift $s \in \mathbb{R}$, if f is the $\mathcal{N}(0, \sigma^2)$ density, then

$$L(x) = \frac{s^2 - 2sx}{2\sigma^2}, \tag{1.31}$$

which is linear in x and unbounded. If f is the Lap(b) density, then

$$L(x) = \frac{|x-s| - |x|}{b}, \qquad |L(x)| \le \frac{|s|}{b} \quad \text{for every } x. \tag{1.32}$$

1.8.3. Derivation

Normal. Start from the normal density with $\mu = 0$, $f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-x^2/(2\sigma^2)}$.

1. *Write the shifted density.* Replace x by $x - s$: $f(x-s) = \frac{1}{\sigma\sqrt{2\pi}} e^{-(x-s)^2/(2\sigma^2)}$. The constant in front is the same; only the exponent changes.
2. *Take the ratio.* The constants cancel, and $e^a/e^c = e^{a-c}$:

$$\frac{f(x)}{f(x-s)} = \frac{e^{-x^2/(2\sigma^2)}}{e^{-(x-s)^2/(2\sigma^2)}} = \exp\left(\frac{(x-s)^2 - x^2}{2\sigma^2}\right). \tag{1.33}$$

3. *Take the logarithm.* Since $\ln e^c = c$, the log ratio is the exponent: $L(x) = \dfrac{(x-s)^2 - x^2}{2\sigma^2}$.
4. *Expand the square.* $(x-s)^2 = x^2 - 2sx + s^2$, and the x^2 cancels against $-x^2$, leaving $L(x) = \dfrac{s^2 - 2sx}{2\sigma^2}$.

The x^2 terms cancel because both densities have the same curvature; what is left is linear in x, with slope $-s/\sigma^2$. A line with nonzero slope has no bound, so $|L(x)| \to \infty$ as $|x| \to \infty$.

Laplace. Start from $f(x) = \frac{1}{2b} e^{-|x|/b}$.

1. *Write the shifted density.* $f(x-s) = \frac{1}{2b} e^{-|x-s|/b}$.
2. *Take the ratio and the logarithm,* as for the normal: the constants cancel and the log leaves the difference of exponents, $L(x) = -\dfrac{|x|}{b} + \dfrac{|x-s|}{b} = \dfrac{|x-s| - |x|}{b}$.
3. *Bound it from above.* The triangle inequality $|a+c| \le |a| + |c|$ says that the size of a sum is at most the sum of the sizes. With $a = x$, $c = -s$ it gives $|x-s| \le |x| + |s|$, so $|x-s| - |x| \le |s|$.
4. *Bound it from below.* The same inequality with $a = x - s$, $c = s$ gives $|x| \le |x-s| + |s|$, so $|x-s| - |x| \ge -|s|$.
5. *Combine.* $\big||x-s| - |x|\big| \le |s|$, so $|L(x)| \le |s|/b$ for every x, and the ratio of the two densities never exceeds $e^{|s|/b}$.

The bound is reached, not just approached. For $s > 0$ and any $x \le 0$, $|x-s| - |x| = (s-x) - (-x) = s$; for any $x \ge s$, it is $(x-s) - x = -s$. Between 0 and s the log ratio falls linearly from s/b to $-s/b$.

Table 1.6 puts numbers on both formulas for $s = 1$, $\sigma = 1$ and $b = 1$, where they read $L(x) = (1 - 2x)/2$ and $L(x) = |x-1| - |x|$.

Table 1.6.: The log ratio $L(x)$ of a density and its copy shifted by $s = 1$, for $\mathcal{N}(0, 1)$ and Lap(1), at a few observations x.

x	-2	0	0.5	1	3	10
Normal, $L(x) = (1 - 2x)/2$	2.5	0.5	0	-0.5	-2.5	-9.5
Laplace, $L(x) = \lvert x - 1\rvert - \lvert x\rvert$	1	1	0	-1	-1	-1

Both vanish at the midpoint $x = 0.5$, where the two candidates are equally plausible. Far out they part ways. At $x = 10$ the normal log ratio is -9.5, so the observation is $e^{9.5} \approx 13{,}360$ times more likely under $\mu = 1$ than under $\mu = 0$. The Laplace never says more than $e^1 \approx 2.72$ times, however far out x is (Figure 1.7).

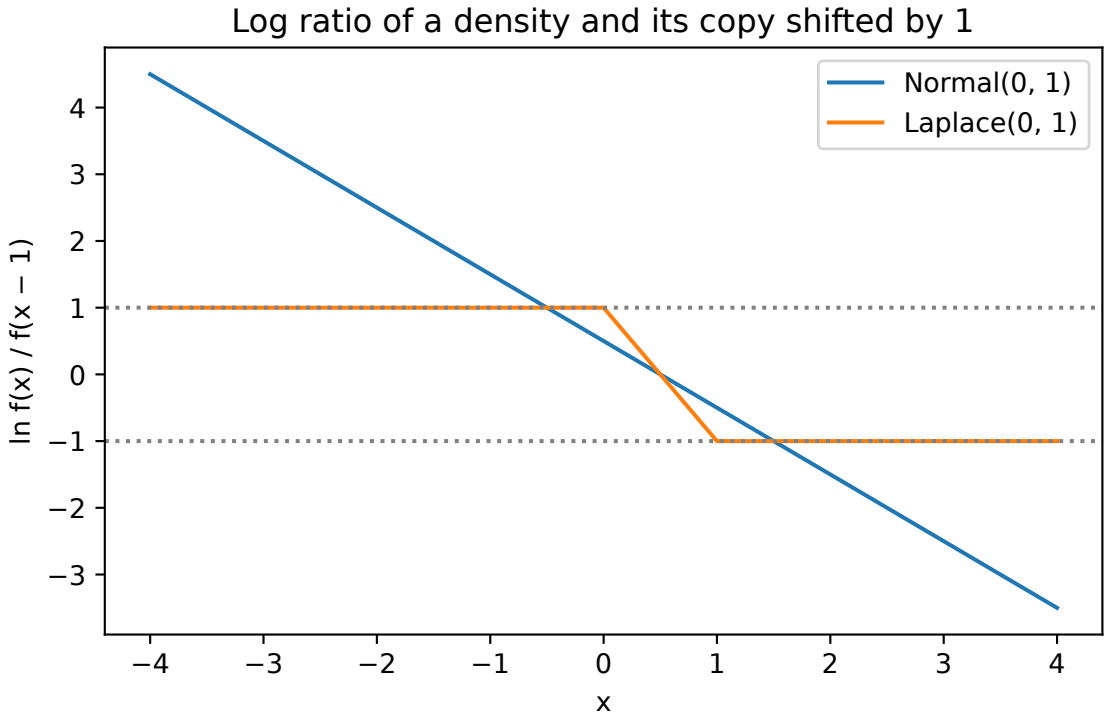


Figure 1.7.: Log ratio $L(x)$ of a density and its copy shifted by $s = 1$, for the standard normal and Lap(1).

1.8.4. Interpretation

For the Laplace, the two copies are within a fixed factor of each other everywhere. For the normal, they are close in the bulk, but the ratio is unbounded in the tails — where, however, there is very little probability. How to make "very little probability" precise is the subject of the next three

chapters: concentration bounds it, divergences summarize it, and hypothesis testing says what an observer can do with it.

1.8.5. Check yourself

- For the normal, at which x is $L(x) = 0$, and why is that point natural? *($x = s/2$, the midpoint, where the two densities cross.)*
- Under $x \sim \mathcal{N}(0, \sigma^2)$, what is the distribution of $L(x)$? *(It is linear in a normal, so normal: mean $s^2/(2\sigma^2)$, variance $(s/\sigma^2)^2\sigma^2 = s^2/\sigma^2$.)*

1.9. Code: check the facts by simulation

The script code/distributions.py draws every figure in this chapter and checks the formulas against simulation. Each probability function and each simulated quantity is a small typed function; the script calls them once for the tables and loops them over parameters for the figures. From the repository root:

```
uv run python code/part01-prerequisites/distributions.py
```

Table 1.7 compares each formula with 1,000,000 samples:

Table 1.7.: Mean and variance from the formulas against 1,000,000 samples (seed 0).

Distribution	Mean (formula / sample)	Variance (formula / sample)
Bernoulli(0.3)	0.300 / 0.301	0.210 / 0.210
Binomial(20, 0.3)	6.000 / 6.001	4.200 / 4.198
Poisson(4)	4.000 / 3.998	4.000 / 4.001
Exp(2)	2.000 / 2.001	4.000 / 4.019
Lap(0, 2)	0.000 / −0.000	8.000 / 7.993
$\mathcal{N}(1, 3^2)$	1.000 / 1.002	9.000 / 8.977

The formulas in each row are the ones derived above: $p(1-p)$, $np(1-p)$, μ, b^2, $2b^2$ and σ^2. Every sample value is within a few thousandths of its formula, which is the sampling error you expect from a million draws. The other four checks are reported in their sections above: the slotted waiting time, the difference of exponentials, the norm of a normal vector, and the convergence of standardized exponential sums to the normal. tests/test_distributions.py checks that the output still matches.

1.10. References

- Blitzstein & Hwang, *Introduction to Probability* (2nd ed., 2019), ch. 3–5 and 7: every distribution here, with worked examples
- Wasserman, *All of Statistics* (2004), ch. 2–3: a compact reference

- Vershynin, *High-Dimensional Probability* (2018), §3.1: concentration of the norm of a normal vector
- Shevtsova, "On the absolute constants in the Berry–Esseen type inequalities for identically distributed summands" (2011): the constant $C \le 0.4748$ in the Berry–Esseen bound

1.11. Cheat sheet

Table 1.8 collects the facts above.

Table 1.8.: Summary of the five families: density, moments, MGF and tail.

Distribution	Density or PMF	Mean	Variance	MGF	Tail
Bernoulli(p)	$p^x(1-p)^{1-x}$, $x \in \{0,1\}$	p	$p(1-p)$	$1 + p(e^\lambda - 1)$	—
Binomial(n,p)	$\binom{n}{k} p^k (1-p)^{n-k}$	np	$np(1-p)$	$(1 + p(e^\lambda - 1))^n$	Chernoff, see chapter 2
Poisson(μ)	$e^{-\mu}\mu^k/k!$	μ	μ	$e^{\mu(e^\lambda - 1)}$	$\approx (e\mu/k)^k$
Exp(b)	$\frac{1}{b} e^{-y/b}$, $y \ge 0$	b	b^2	$\frac{1}{1-b\lambda}$, $\lambda < \frac{1}{b}$	$e^{-t/b}$
Lap(μ,b)	$\frac{1}{2b} e^{-\lvert x-\mu\rvert/b}$	μ	$2b^2$	$\frac{e^{\mu\lambda}}{1-b^2\lambda^2}$, $\lvert\lambda\rvert < \frac{1}{b}$	$\Pr[\lvert X-\mu\rvert \ge t] = e^{-t/b}$
$\mathcal{N}(\mu,\sigma^2)$	$\frac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$	μ	σ^2	$e^{\mu\lambda + \sigma^2\lambda^2/2}$	$\Pr[X-\mu \ge t] \le \frac{1}{2} e^{-t^2/(2\sigma^2)}$

Relationships

- Binomial($n, \mu/n$) $\to$ Poisson(μ) as $n \to \infty$.
- Geometric($\frac{1}{bn}$)$/n \to$ Exp(b) as $n \to \infty$: the waiting time for the first event in the same slots as the Poisson limit.
- Binomial(n,p) $\approx \mathcal{N}(np, np(1-p))$ in the bulk when $np(1-p) \gtrsim 10$.
- Exp(b) $-$ Exp(b) $\sim$ Lap(b), and $\lvert$Lap(b)$\rvert \sim$ Exp(b).
- Independent Poissons add to a Poisson, and independent normals add to a normal.
- CLT: $(S_n - n\mu)/(\sigma\sqrt{n}) \to \mathcal{N}(0,1)$ for any i.i.d. summands with finite variance. The CDF error is $O(1/\sqrt{n})$ and grows with the skewness (Berry–Esseen).
- $\mathcal{N}(0, \sigma^2 I_d)$ has norm $\approx \sigma\sqrt{d}$ and is $\mathcal{N}(0,\sigma^2)$ along every unit direction.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

2. Concentration inequalities

2.1. Why this matters

Randomness enters machine learning everywhere: the sample you trained on, the minibatches you drew, the test set you evaluated on, and later in this book, noise that is added on purpose. In every case the question is the same: **how far can a random quantity be from what you expect, and with what probability?** Concentration inequalities answer it. Treat them as tools for that one job, not as a list to memorize.

The whole subject is one idea, applied with more and more information about the random variable. To bound the chance that a variable lands far out, you compare it with something whose average you know, and the more you know, the sharper the comparison. If you know only the mean, you get Markov's inequality, and the tail can only be shown to decay like $1/t$, where t is how far out you look. If you also know the variance, the average squared distance from the mean, you get Chebyshev's inequality and a decay like $1/t^2$. If you know the **moment generating function (MGF)** $\mathbb{E}[e^{\lambda X}]$, which averages an exponential of the variable and so feels every moment at once, you get the Chernoff bound, which decays exponentially in t. When that MGF is no larger than a normal distribution's, the variable is called **sub-Gaussian** and its tail decays like a normal's, as e^{-t^2}; Hoeffding's inequality is the special case of bounded variables, which are always sub-Gaussian. Finally, when the quantity is not a sum of independent terms but a function of many inputs, or a running total whose steps depend on the past (a martingale), McDiarmid's and Azuma's inequalities give the same exponential bounds as long as each input or step can move the result only a bounded amount.

Table 2.1 lists where each one shows up:

Table 2.1.: The inequalities in this chapter and where each is typically used.

Inequality	Typical use
Markov	The base step inside every other proof
Chebyshev	Quick variance-based error bars
Chernoff	Counts of events: error counts, sampled batch sizes, rare outcomes
Hoeffding	Confidence intervals for averages of bounded quantities; generalization bounds
Gaussian tail	How large normal noise can be
Laplace tail	How large Laplace noise can be; maxima over many draws
McDiarmid	Functions of many independent inputs, such as an empirical risk; stability

Inequality	Typical use
Azuma	Sums whose terms are chosen adaptively, based on the past

The distributions used here (binomial, Poisson, Laplace, normal) and their MGFs are derived in Probability distributions. This chapter restates each fact where it uses it, so you do not need that chapter open.

2.2. Toolkit: five facts every derivation uses

Each section below follows the same arc: the question the bound answers, the statement, a derivation, an interpretation, an example, and a self-check. The derivations reuse five facts. We prove them once here.

2.2.1. Fact 1 — Probability is the expectation of an indicator

The probability of an event is the average of a variable that is 1 when the event happens and 0 when it does not. Write $\mathbf{1}_A$ for that 0/1 variable, the **indicator** of the event A, and $\mathbb{E}$ for the expectation (the probability-weighted average). Then

$$\Pr[A] = \mathbb{E}[\mathbf{1}_A]. \tag{2.1}$$

This matters because it turns probabilities into averages, and averages respect inequalities: any pointwise inequality between a random variable and an indicator becomes a probability bound after taking expectations. Markov's inequality and the union bound are both this move. Because everything later depends on it, here it is slowly, with a die.

Step 1: an indicator is a 0/1 random variable. For an event A, $\mathbf{1}_A$ equals 1 on outcomes where A happens and 0 elsewhere. Roll a fair die and let A = "the roll is at least 5" (Table 2.2):

Table 2.2.: The indicator of the event "roll at least 5" on each outcome of a fair die.

Roll	1	2	3	4	5	6
$\mathbf{1}_A$	0	0	0	0	1	1

The average of that row is 2/6, which is exactly $\Pr[A]$. Averaging a 0/1 variable counts the fraction of outcomes where the event happens.

Step 2: if one variable is never larger, its average is never larger. If $Y \le Z$ on *every* outcome, then $\mathbb{E}[Y] \le \mathbb{E}[Z]$: you are averaging two rows of numbers where each entry of the first is at most the matching entry of the second.

Step 3: put them together. Take X = the roll and compare $5 \cdot \mathbf{1}[X \ge 5]$ with X itself, outcome by outcome (Table 2.3):

Table 2.3.: Outcome-by-outcome comparison of $5 \cdot \mathbf{1}[X \geq 5]$ with the die roll X.

Roll	1	2	3	4	5	6	Average
$5 \cdot \mathbf{1}[X \geq 5]$	0	0	0	0	5	5	$5\Pr[X \geq 5]$
X	1	2	3	4	5	6	3.5

Every entry in the top row is at most the one below it. Where the event fails, the top is 0 and $X \geq 0$. Where it holds, the top is 5 and $X \geq 5$ by definition of the event. So the averages are ordered: $5\Pr[X \geq 5] \leq 3.5$, and dividing by 5 gives $\Pr[X \geq 5] \leq 0.7$. The true value is $2/6 = 1/3$; the bound is weak, but it is correct and used nothing except the mean. Figure 2.1 shows the comparison.

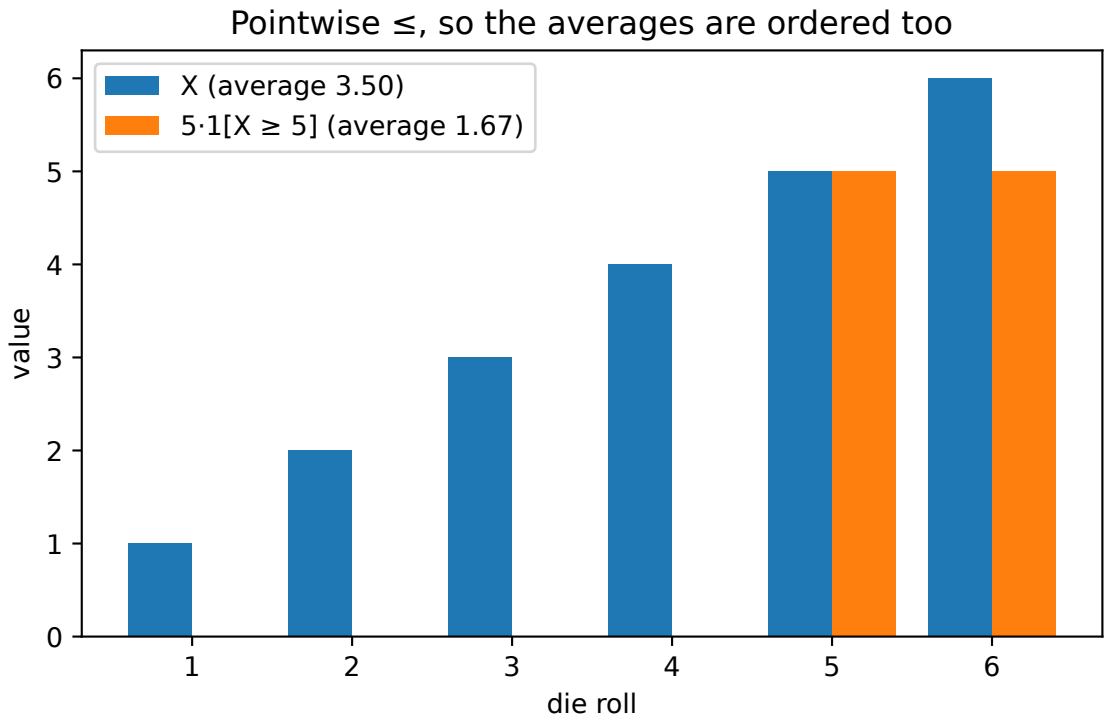


Figure 2.1.: The scaled indicator $5 \cdot \mathbf{1}[X \geq 5]$ never exceeds the die roll X, so its average never exceeds 3.5.

The pattern for bounding Pr[event]: write it as the average of an indicator, find a variable that is at least as large as that indicator on every outcome and whose average you know, and average both sides.

2.2.2. Fact 2 — Union bound

The chance that at least one of several events happens is at most the sum of their separate chances. Adding the chances can only overcount, never undercount, because an outcome that lies in several

events gets counted several times. In symbols, for events $A_1, \dots, A_k$,

$$\Pr[A_1 \cup \cdots \cup A_k] \le \sum_i \Pr[A_i]. \tag{2.2}$$

To prove it, use Fact 1 and compare indicators outcome by outcome: $\mathbf{1}[\cup_i A_i] \le \sum_i \mathbf{1}[A_i]$. On an outcome where none of the events happens, both sides are 0. On an outcome where at least one happens, the left side is 1 and the right side counts how many of the events happen, which is at least 1. The right side can be larger than 1 on outcomes that lie in several events at once: that double counting is the only slack. Taking expectations of both sides turns each indicator into a probability and gives the bound.

You will use it whenever you need k estimates, k steps or k hypotheses to all behave at once. If each one fails with probability at most β/k, all of them succeed together except with probability at most $k \cdot \beta/k = \beta$. In the exponential bounds of this chapter, asking for failure probability β/k instead of β replaces $\ln(1/\beta)$ by $\ln(k/\beta) = \ln k + \ln(1/\beta)$, so the cost of controlling k things is an extra $\ln k$ inside every exponential bound.

2.2.3. Fact 3 — $1 + x \le e^x$ for every real x

The straight line $1 + x$ touches the curve e^x at $x = 0$ (both equal 1 there and both have slope 1) and stays below it everywhere else. The reason is that e^x is **convex**: its second derivative e^x is positive, so the curve bends upward, away from any tangent line, on both sides of the point of contact. A convex function therefore lies above all its tangents, and $1 + x$ is its tangent at 0.

You will use it to turn a sum into an exponential. In the Chernoff bound it turns $1 + p(e^\lambda - 1)$ into $\exp(p(e^\lambda - 1))$, by taking $x = p(e^\lambda - 1)$. A product of such factors then becomes a single exponential whose exponent is a sum, which is far easier to optimize.

2.2.4. Fact 4 — MGFs multiply under independence

The moment generating function of a random variable X is the average of $e^{\lambda X}$, viewed as a function of a number λ that you choose:

$$M_X(\lambda) = \mathbb{E}[e^{\lambda X}]. \tag{2.3}$$

With $\lambda > 0$ the weight $e^{\lambda X}$ grows exponentially in X, so large values of X dominate the average, and the larger λ, the further into the right tail it looks. That is why the MGF is the right object for tail bounds.

The MGF of a sum of independent variables is the product of their MGFs. The reason is that $e^{\lambda(X+Y)} = e^{\lambda X} e^{\lambda Y}$, and when X and Y are independent so are $e^{\lambda X}$ and $e^{\lambda Y}$; the average of a product of independent variables is the product of their averages, so

$$\mathbb{E}[e^{\lambda(X+Y)}] = \mathbb{E}[e^{\lambda X}]\,\mathbb{E}[e^{\lambda Y}]. \tag{2.4}$$

Scaling is just as simple. Multiplying X by a constant c is the same as multiplying the dial λ by c, because $\mathbb{E}[e^{\lambda(cX)}] = \mathbb{E}[e^{(c\lambda)X}]$, so $M_{cX}(\lambda) = M_X(c\lambda)$.

You will use it to reduce a sum of n variables to n one-variable calculations, one MGF per term.

2.2.5. Fact 5 — Inverting a tail bound

A tail bound answers "how likely is a deviation of size t?". In practice you usually want the reverse: "how large a deviation must I allow so that it is exceeded with probability at most β?". That number is the **width** at failure probability β, and you get it by setting the bound equal to β and solving for t.

Concretely, suppose the deviation D satisfies $\Pr[D \geq t] \leq e^{-g(t)}$ for an increasing function g. The bound is at most β exactly when $g(t) \geq \ln(1/\beta)$, that is, when $t \geq g^{-1}(\ln(1/\beta))$. So with probability at least $1 - \beta$,

$$D < g^{-1}(\ln(1/\beta)). \tag{2.5}$$

For example, the Gaussian-shaped bound with $g(t) = t^2/(2\sigma^2)$ gives $t^2/(2\sigma^2) = \ln(1/\beta)$, so $D < \sigma\sqrt{2\ln(1/\beta)}$. Every "width at failure probability β" in this chapter is this step.

2.3. Markov and Chebyshev

2.3.1. Context

You have a random estimate and want the crudest answer to "how far can it be from the truth?", using as little as possible about its distribution. Markov uses only the mean, and Chebyshev uses only the variance. Both are weak. But every stronger bound in this chapter is Markov applied to a better function, so the derivation pattern matters more than the bounds themselves.

2.3.2. Markov's inequality

A non-negative variable cannot often be much larger than its mean, because those large values alone would already pull the average up. If $X \geq 0$ were at least a with probability p, those outcomes would contribute at least ap to the mean, so $ap \leq \mathbb{E}[X]$. For $X \geq 0$ and $a > 0$:

$$\Pr[X \geq a] \leq \frac{\mathbb{E}[X]}{a}. \tag{2.6}$$

Derivation. This is the die argument of Fact 1, with a general a in place of 5.

1. Compare X with a scaled indicator: $a\,\mathbf{1}[X \geq a] \leq X$ pointwise. If $X \geq a$, the left side is a, which is at most X. If $X < a$, the left side is 0, which is at most X because $X \geq 0$. This is the only place non-negativity is used.
2. Take expectations, which preserve $\leq$ (Fact 1, Step 2). The average of the indicator is a probability (Fact 1), so $a \Pr[X \geq a] \leq \mathbb{E}[X]$.
3. Divide both sides by $a > 0$.

Tightness. You cannot do better with only the mean. Write μ for the mean and take the variable that puts all its mass on two points: $X = a$ with probability μ/a and 0 otherwise. Its mean is $a \cdot \mu/a = \mu$, and $\Pr[X \geq a] = \mu/a$ exactly, so Markov's bound holds with equality.

2.3.3. Generalized Markov

Markov's inequality applies to any non-negative variable, so you can apply it to a transformed version of X instead of X itself. If g is non-negative and strictly increasing, then $X \geq t$ and $g(X) \geq g(t)$ are the same event, because an increasing function keeps the order. Applying Markov to the non-negative variable $g(X)$ at the level $a = g(t)$ gives

$$\Pr[X \geq t] = \Pr[g(X) \geq g(t)] \leq \frac{\mathbb{E}[g(X)]}{g(t)}. \tag{2.7}$$

The rest of this chapter is a search for good g. A fast-growing g makes the threshold $g(t)$ in the denominator huge compared with typical values of $g(X)$, so the bound gets small; the price is that $\mathbb{E}[g(X)]$ also grows, and you must be able to compute or bound it. $g(x) = x^2$ gives Chebyshev, and $g(x) = e^{\lambda x}$ gives Chernoff.

2.3.4. Chebyshev's inequality

The variance σ^2 is the average squared distance from the mean μ. A large deviation $|X - \mu| \geq t$ makes the squared distance at least t^2, and if that happened often, the average squared distance would exceed σ^2. So the probability of a deviation of t or more is at most σ^2/t^2. For X with mean μ and variance σ^2:

$$\Pr[|X - \mu| \geq t] \leq \frac{\sigma^2}{t^2}. \tag{2.8}$$

Derivation.

1. Turn the two-sided event into an event about a non-negative variable: $|X - \mu| \geq t$ holds exactly when $(X - \mu)^2 \geq t^2$, since squaring preserves the order of non-negative numbers.
2. Apply Markov to $Y = (X - \mu)^2$, which is non-negative and has mean σ^2 by the definition of the variance, at the level $a = t^2$: $\Pr[Y \geq t^2] \leq \mathbb{E}[Y]/t^2 = \sigma^2/t^2$.

Means of samples. Averaging n independent samples shrinks the variance by a factor of n. Two rules give this. Variances add for independent terms, because the cross terms (covariances) in the expansion of the square vanish, so a sum of n terms has variance $n\sigma^2$. And $\mathrm{Var}(cX) = c^2 \operatorname{Var} X$, because every deviation is multiplied by c and then squared, so dividing the sum by n divides its variance by n^2. Together, the mean $\bar{X}$ has $\mathrm{Var}(\bar{X}) = n\sigma^2/n^2 = \sigma^2/n$. Chebyshev then gives $\Pr[|\bar{X} - \mu| \geq t] \leq \sigma^2/(nt^2)$. To turn this into a width, set the bound equal to β and solve for t, as in Fact 5: $\sigma^2/(nt^2) = \beta$ gives $t = \sigma/\sqrt{n\beta}$. So with probability at least $1 - \beta$, $|\bar{X} - \mu| < \sigma/\sqrt{n\beta}$.

2.3.5. Interpretation: the price of knowing only the variance

How wide must an error bar be to fail with probability at most β? With only the variance known, solving $\sigma^2/t^2 = \beta$ gives $t = \sigma/\sqrt{\beta}$. With a Gaussian-type tail $e^{-t^2/(2\sigma^2)}$, Fact 5 gives $t = \sigma\sqrt{2\ln(1/\beta)}$. Table 2.4 compares the two, in units of σ. For instance, at $\beta = 0.05$, $1/\sqrt{0.05} = 4.5$, while $\ln(1/0.05) = \ln 20 \approx 3.0$ gives $\sqrt{2 \times 3.0} = 2.4$.

Table 2.4.: Width needed to reach failure probability β, in units of σ, with only the variance known (Chebyshev) and with a Gaussian-type tail.

β	Chebyshev, $1/\sqrt{\beta}$	Gaussian-type, $\sqrt{2\ln(1/\beta)}$
0.05	4.5	2.4
10^{-3}	31.6	3.7
10^{-6}	1,000	5.3

The Chebyshev width grows like a power of $1/\beta$, the Gaussian-type width only like the square root of a logarithm. Polynomial bounds are fine for a quick 95% error bar. They are useless when the failure probability must be tiny, as it usually must be in security and in the later parts of this book, often 10^{-5} or smaller.

2.3.6. Example: the error of Laplace noise

A measurement is corrupted by additive noise from the **Laplace distribution** $\mathrm{Lap}(b)$, which has density $\frac{1}{2b}e^{-|x|/b}$: symmetric around 0, with scale b. Its variance is $2b^2$; you can check this by integrating, since $\mathbb{E}[X^2] = 2\int_0^\infty x^2 \frac{1}{2b}e^{-x/b}\,dx = 2b^2$ (Probability distributions). Chebyshev with $\sigma^2 = 2b^2$ gives $\Pr[|\text{error}| \geq t] \leq 2b^2/t^2$, and setting this to β gives the $1-\beta$ width $b\sqrt{2/\beta}$. With $b = 1$ and $\beta = 0.05$ that is $\sqrt{2/0.05} = \sqrt{40} \approx 6.3$.

Now use the shape of the noise. Integrating the density gives the exact tail $\Pr[|X| \geq t] = e^{-t/b}$, and setting $e^{-t/b} = \beta$ gives the width $b\ln(1/\beta) = \ln 20 \approx 3.0$. At $\beta = 10^{-6}$ the two widths are 1,414 (that is, $\sqrt{2\times 10^6}$) and 13.8 (that is, $\ln(10^6)$): ignoring the shape of the noise costs a factor of 100.

2.3.7. Check yourself

- Why can't you apply Markov directly to $X - \mu$? *(It can be negative, so step 1 of the Markov derivation fails.)*
- Which step of the sample-mean argument used independence, and is it more than you need? *(Only the vanishing of covariances. Pairwise uncorrelated samples suffice.)*

2.4. The Chernoff method

2.4.1. Context

Small failure probabilities like 10^{-5} are out of reach for polynomial bounds at sensible widths. The fix is to apply generalized Markov with $g(x) = e^{\lambda x}$. Its average is the MGF, which contains every moment of X at once, so the bound uses everything about the distribution, not just the mean or the variance. The resulting Chernoff bound is exponentially tight: Cramér's theorem says that for a sum of n independent copies of a variable, the probability of a large deviation decays exponentially in n with exactly the rate the optimized Chernoff bound gives, so no method can improve its exponent for large sums.

2.4.2. The recipe

The master bound. For any $\lambda > 0$, $e^{\lambda x}$ is positive and increasing, so generalized Markov applies with $g(x) = e^{\lambda x}$. Dividing $\mathbb{E}[e^{\lambda X}]$ by $e^{\lambda t}$ gives the first form below. Writing the MGF as $e^{\psi(\lambda)}$, where ψ is the logarithm of the MGF (the **log-MGF**), puts the whole bound into one exponent:

$$\Pr[X \geq t] \leq e^{-\lambda t}\,\mathbb{E}[e^{\lambda X}] = \exp\big(-(\lambda t - \psi(\lambda))\big), \qquad \psi(\lambda) = \ln \mathbb{E}[e^{\lambda X}]. \tag{2.9}$$

This holds for every $\lambda > 0$, so you are free to pick the λ that makes the bound smallest, which is the one that makes $\lambda t - \psi(\lambda)$ largest. The best exponent, $\sup_\lambda(\lambda t - \psi(\lambda))$, viewed as a function of t, has a name: the **Legendre transform** $\psi^*(t)$ of the log-MGF. It is the exponent Cramér's theorem says is exact.

In practice every Chernoff bound in this chapter is computed by the same five steps:

1. Write down the MGF of X.
2. If X is a sum of independent terms, factorize the MGF (Fact 4).
3. Bound each factor by something simple, usually with Fact 3.
4. Minimize the exponent over λ by setting its derivative to zero.
5. Simplify the result into a usable form.

Think of $e^{\lambda X}$ as a lens on the tail. A large λ magnifies the far tail but inflates $\mathbb{E}[e^{\lambda X}]$. A small λ keeps the expectation tame but magnifies little. Figure 2.2 shows the trade-off for a binomial: the bound as a function of λ has a clear minimum, and the relaxation of step 3 costs a little but moves the optimum to a closed form.

2.4.3. Warm-up: the Gaussian

Start with the case where every step is exact, a normal variable $X \sim \mathcal{N}(0, \sigma^2)$ with mean 0 and variance σ^2. Its MGF is $\mathbb{E}[e^{\lambda X}] = e^{\lambda^2\sigma^2/2}$ (Probability distributions). To see why, multiply the density, which is proportional to $e^{-x^2/(2\sigma^2)}$, by $e^{\lambda x}$ and complete the square in the exponent: $\lambda x - x^2/(2\sigma^2) = -(x - \lambda\sigma^2)^2/(2\sigma^2) + \lambda^2\sigma^2/2$. The first term is a normal density shifted to mean $\lambda\sigma^2$, which still integrates to 1, and the constant $e^{\lambda^2\sigma^2/2}$ is left over. So the log-MGF is the quadratic $\psi(\lambda) = \lambda^2\sigma^2/2$.

1. **Master bound.** Substitute the MGF: $\Pr[X \geq t] \leq \exp(-\lambda t + \lambda^2\sigma^2/2)$.
2. **Minimize.** The exponent is a parabola in λ. Its derivative $-t + \lambda\sigma^2$ vanishes at $\lambda = t/\sigma^2$. Substituting this λ gives the exponent $-t^2/\sigma^2 + t^2/(2\sigma^2) = -t^2/(2\sigma^2)$, so

$$\Pr[X \geq t] \leq e^{-t^2/(2\sigma^2)}. \tag{2.10}$$

Remember the shape of this calculation: a quadratic log-MGF gives a quadratic exponent. The sub-Gaussian section reuses it verbatim.

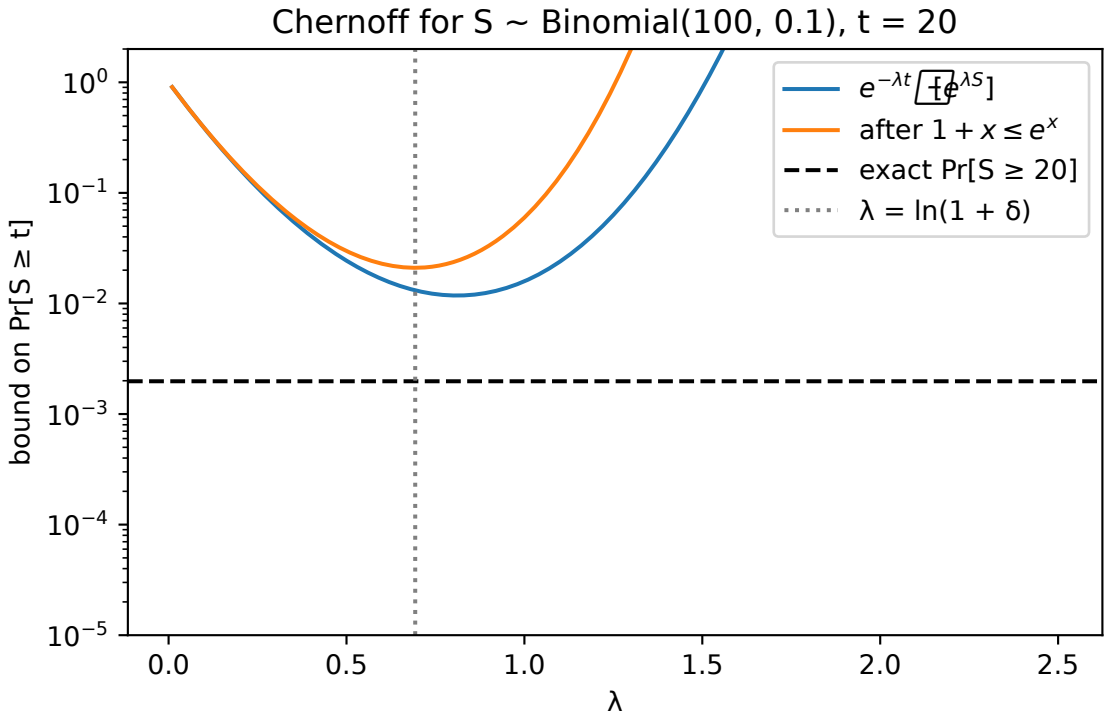


Figure 2.2.: The Chernoff bound $e^{-\lambda t}\mathbb{E}[e^{\lambda S}]$ for $S \sim \text{Binomial}(100, 0.1)$ and $t = 20$, as a function of λ, before and after the relaxation $1 + x \le e^x$. Both stay above the exact tail.

2.4.4. Main case: sums of Bernoullis

Now the case the method is most used for: counting how many of many independent yes/no events happen. Let $X_i \sim \text{Bernoulli}(p_i)$ be independent, each equal to 1 with probability p_i and 0 otherwise, let $S = \sum_i X_i$ be the count, and let $\mu = \mathbb{E}[S] = \sum_i p_i$ be its mean. The question is how likely S is to land a fraction δ above or below μ.

Upper tail, $\Pr[S \geq (1+\delta)\mu]$ for $\delta > 0$. Follow the recipe.

1. **One factor.** X_i is 1 with probability p_i and 0 otherwise, so $\mathbb{E}[e^{\lambda X_i}] = p_i e^{\lambda} + (1-p_i) = 1 + p_i(e^{\lambda}-1)$. Fact 3 with $x = p_i(e^{\lambda}-1)$ bounds this by $\exp(p_i(e^{\lambda}-1))$. The right side is the MGF of a Poisson variable with mean p_i, since a Poisson(μ) variable has MGF $\exp(\mu(e^{\lambda}-1))$ (Probability distributions). In words: as far as the MGF can tell, a Bernoulli sum is dominated by a Poisson count with the same mean.
2. **Product.** By Fact 4 the MGF of the sum is the product of the factors, and the exponents add: $\mathbb{E}[e^{\lambda S}] \leq \exp\big(\sum_i p_i(e^{\lambda}-1)\big) = \exp(\mu(e^{\lambda}-1))$.
3. **Master bound** with $t = (1+\delta)\mu$: $\Pr[S \geq (1+\delta)\mu] \leq \exp(\mu(e^{\lambda}-1) - \lambda(1+\delta)\mu)$.
4. **Minimize.** The derivative of the exponent, $\mu e^{\lambda} - (1+\delta)\mu$, vanishes at $e^{\lambda} = 1+\delta$, so $\lambda = \ln(1+\delta)$, which is positive as required. Substituting, the exponent becomes $\mu\delta - (1+\delta)\mu\ln(1+\delta)$, so

$$\Pr[S \geq (1+\delta)\mu] \leq \left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}}\right)^{\mu} = \exp\big(-\mu[(1+\delta)\ln(1+\delta) - \delta]\big). \tag{2.11}$$

5. **Simplify.** The bracket is awkward to invert, so replace the logarithm by a simpler lower bound, $\ln(1+\delta) \geq \frac{2\delta}{2+\delta}$ for $\delta \geq 0$. To see that this holds, let $f(\delta) = \ln(1+\delta) - \frac{2\delta}{2+\delta}$. It starts at $f(0) = 0$ and never decreases, because its derivative is $f'(\delta) = \frac{1}{1+\delta} - \frac{4}{(2+\delta)^2} = \frac{\delta^2}{(1+\delta)(2+\delta)^2} \geq 0$; so $f(\delta) \geq 0$. Putting the lower bound into the bracket gives $(1+\delta)\frac{2\delta}{2+\delta} - \delta = \frac{\delta^2}{2+\delta}$, so

$$\Pr[S \geq (1+\delta)\mu] \leq \exp\left(-\frac{\delta^2\mu}{2+\delta}\right). \tag{2.12}$$

Lower tail, $\Pr[S \leq (1-\delta)\mu]$ for $0 < \delta < 1$. The same recipe works if you look at $-S$ instead of S, because a small S is a large $-S$.

1. **Master bound for the lower tail.** For $\lambda > 0$, $S \leq s$ holds exactly when $e^{-\lambda S} \geq e^{-\lambda s}$, since $e^{-\lambda x}$ is decreasing. Markov applied to $e^{-\lambda S}$, and the same per-factor bound as before with $-\lambda$ in place of λ, give $\Pr[S \leq s] \leq e^{\lambda s}\,\mathbb{E}[e^{-\lambda S}] \leq \exp(\lambda s + \mu(e^{-\lambda}-1))$.
2. **Minimize.** With $s = (1-\delta)\mu$, the derivative $s - \mu e^{-\lambda}$ vanishes at $e^{-\lambda} = 1-\delta$, that is, $\lambda = -\ln(1-\delta) > 0$. Substituting gives $\exp\big(-\mu[\delta + (1-\delta)\ln(1-\delta)]\big)$.
3. **Simplify.** Expand the logarithm in its Taylor series, $\ln(1-\delta) = -\sum_{k\geq 1}\delta^k/k$, multiply by $1-\delta$ and collect powers of δ: $(1-\delta)\ln(1-\delta) = -\delta + \sum_{k\geq 2}\frac{\delta^k}{k(k-1)}$. Every term of the sum is non-negative, so keeping only the first one ($k=2$, which is $\delta^2/2$) gives the lower bound $-\delta + \frac{\delta^2}{2}$. The bracket is then at least $\delta^2/2$, so

$$\Pr[S \leq (1-\delta)\mu] \leq \exp\left(-\frac{\delta^2\mu}{2}\right). \tag{2.13}$$

2.4.5. Interpretation

The bounds depend on the expected count μ, not on the number of trials n. A million trials with $p = 10^{-6}$ expect a single event, and the count is far from concentrated. Rare events concentrate in relative terms only once you expect enough of them, because the exponent $\delta^2\mu$ grows with μ.

Chernoff also beats Hoeffding (the range-only bound of the next section) when p is small, and the reason is in step 1. That step keeps each p_i in the bound instead of replacing it by the worst case, so the bound implicitly knows the variance of the count, $\mu(1-p) \approx \mu$, which is small when p is small. A bound that only knows the range has to assume the largest variance the range allows. Figure 2.3 shows the gap.

Finally, the two tails differ. For large δ, the upper-tail exponent $\delta^2\mu/(2+\delta)$ behaves like $\delta\mu$, so the tail decays like $\exp(-\delta\mu)$, not like $\exp(-\delta^2)$. Counts have Poisson-like right tails, not Gaussian ones: they can overshoot their mean by a lot more easily than they can undershoot it.

2.4.6. Example: sampled minibatch sizes

A training loop forms each minibatch by including each of $n = 60{,}000$ examples independently with probability $q = 0.01$. The batch size B is a sum of Bernoulli(q) variables, so the bounds above apply with $\mu = 60{,}000 \times 0.01 = 600$. How likely is the batch to be more than 20% off, that is, $\delta = 0.2$, so that $\delta^2 = 0.04$?

The upper-tail bound is $\exp(-0.04 \times 600/2.2) = e^{-10.9} \approx 1.8 \times 10^{-5}$, and the lower-tail bound is $\exp(-0.04 \times 600/2) = e^{-12} \approx 6.1 \times 10^{-6}$. The two events cannot happen together, so adding them gives the chance of either: B stays within $600 \pm 20\%$ except with probability about 2.4×10^{-5} per step. Over 1,000 steps, the union bound (Fact 2) adds the 1,000 per-step failure probabilities, $1{,}000 \times 2.4 \times 10^{-5}$, so every batch stays in that range except with probability about 0.024.

2.4.7. Check yourself

- Why must λ be positive for the upper tail? *(For $\lambda < 0$, $e^{\lambda x}$ is decreasing, so $X \geq t$ no longer implies $e^{\lambda X} \geq e^{\lambda t}$.)*
- Where exactly did independence enter? *(Only in factorizing the MGF in step 2.)*

2.5. Sub-Gaussian variables and Hoeffding

2.5.1. Context

Redoing the Chernoff calculation for every new distribution is tedious. The Gaussian warm-up showed that a quadratic log-MGF alone produces a Gaussian-shaped tail: nothing else about the normal distribution was used. So define a class by that one property, prove tail and sum bounds for the class once, and from then on only check membership. Hoeffding's inequality is the result for the most common member: bounded variables.

2.5.2. Definition

The idea is to call a variable sub-Gaussian when its MGF, centred at its mean μ, is never larger than the MGF of a normal variable, because then the Chernoff argument for the normal goes through with "$\leq$" in place of "=". Formally, X is **σ-sub-Gaussian** if its centred MGF is dominated by that of $\mathcal{N}(0, \sigma^2)$:

$$\mathbb{E}\big[e^{\lambda(X-\mu)}\big] \leq e^{\lambda^2\sigma^2/2} \quad \text{for all } \lambda \in \mathbb{R}. \tag{2.14}$$

The parameter σ, called the **variance proxy** when squared, is the effective standard deviation for tail purposes. It is at least the true standard deviation. To see why, expand both sides around $\lambda = 0$: the left side is $1 + \lambda^2 \operatorname{Var}(X)/2 + ...$ (the linear term vanishes because $X - \mu$ has mean 0) and the right side is $1 + \lambda^2\sigma^2/2 + ...$, so for small λ the inequality forces $\operatorname{Var}(X) \leq \sigma^2$. It can be much larger than the true standard deviation, and that slack is where these bounds lose accuracy.

2.5.3. Three properties, each one line

Tail. A sub-Gaussian variable has a tail no heavier than the normal's. The Gaussian warm-up used the MGF only through the inequality in the master bound, so repeating it with the sub-Gaussian inequality in place of the equality gives the same result:

$$\Pr[X - \mu \geq t] \leq e^{-t^2/(2\sigma^2)}. \tag{2.15}$$

The same holds for the lower tail, because $-X$ is also σ-sub-Gaussian (replace λ by $-\lambda$ in the definition). Adding the two one-sided bounds, the two-sided bound $\Pr[|X-\mu| \geq t]$ carries a factor of 2.

Sums. Variance proxies add under independence, like variances. The reason is Fact 4: the centred MGF of a sum of independent terms is the product of their centred MGFs, and each factor is bounded by a Gaussian MGF, whose exponents add. For independent X_i, each σ_i-sub-Gaussian with mean μ_i,

$$\mathbb{E}\big[e^{\lambda\sum_i(X_i-\mu_i)}\big] = \prod_i \mathbb{E}\big[e^{\lambda(X_i-\mu_i)}\big] \leq \prod_i e^{\lambda^2\sigma_i^2/2} = e^{\lambda^2(\sum_i \sigma_i^2)/2}, \tag{2.16}$$

so the sum is $\sqrt{\sum_i \sigma_i^2}$-sub-Gaussian.

Scaling and means. By Fact 4, $M_{cX}(\lambda) = M_X(c\lambda)$, so substituting $c\lambda$ for λ in the definition gives $e^{c^2\lambda^2\sigma^2/2}$: cX is $|c|\sigma$-sub-Gaussian. Now take the mean of n independent σ-sub-Gaussian variables. Their sum is $\sqrt{n\sigma^2} = \sigma\sqrt{n}$-sub-Gaussian by the sum rule, and dividing by n makes the mean $(\sigma\sqrt{n}/n) = (\sigma/\sqrt{n})$-sub-Gaussian. That is where the $1/\sqrt{n}$ rate comes from.

2.5.4. Hoeffding's lemma: bounded variables are sub-Gaussian

Statement. A variable trapped in an interval $[a, b]$ cannot have a heavy tail, since it cannot go beyond b at all. The lemma makes this quantitative: its MGF is bounded as if it were normal with standard deviation half the width of the interval. If $X \in [a, b]$, then X is $\frac{b-a}{2}$-sub-Gaussian:

$$\mathbb{E}[e^{\lambda(X-\mu)}] \leq \exp\left(\lambda^2(b-a)^2/8\right). \tag{2.17}$$

Derivation. Centre X so that $\mathbb{E}[X] = 0$ (shifting X and the interval by μ changes nothing on either side). A mean-zero variable must take values on both sides of 0, so $a \le 0 \le b$. The plan is to bound $e^{\lambda X}$ by something linear in X, whose average depends only on the mean, and then show that the resulting expression has a quadratic logarithm.

1. **Use convexity.** On $[a, b]$, the convex curve $e^{\lambda x}$ lies below the straight line (the chord) joining its values at the two endpoints. That line is the weighted average of the endpoint values, with weights that say how close x is to each end:
$$e^{\lambda x} \le \frac{b-x}{b-a} e^{\lambda a} + \frac{x-a}{b-a} e^{\lambda b}. \tag{2.18}$$
2. **Take expectations.** The right side is linear in x, so its average only needs $\mathbb{E}[X] = 0$: $\mathbb{E}[e^{\lambda X}] \le \frac{b}{b-a} e^{\lambda a} - \frac{a}{b-a} e^{\lambda b}$. This is exactly the MGF of the two-point variable that sits at a or b with mean 0.
3. **Reparametrize** to make the bound a function of one variable. Let $p = -a/(b-a) \in [0, 1]$, so that $\frac{b}{b-a} = 1 - p$ and $\lambda a = -hp$, where $h = \lambda(b-a)$; then $e^{\lambda b} = e^{\lambda a} e^h$. The right side becomes $e^{-hp}(1 - p + pe^h) = e^{\varphi(h)}$, where
$$\varphi(h) = -hp + \ln\left(1 - p + pe^h\right). \tag{2.19}$$
It remains to show $\varphi(h) \le h^2/8$, by a Taylor expansion around $h = 0$.
4. **Value and slope at 0.** $\varphi(0) = 0 + \ln 1 = 0$. The derivative is $\varphi'(h) = -p + \frac{pe^h}{1-p+pe^h}$, so $\varphi'(0) = -p + p = 0$.
5. **Bound the curvature.** Write $u = \frac{pe^h}{1-p+pe^h}$, a number in $[0, 1]$, so that $\varphi'(h) = -p + u$. Differentiating again gives $\varphi''(h) = u(1-u)$. A product of two non-negative numbers that add up to 1 is largest when both are $\frac{1}{2}$, so $\varphi''(h) \le 1/4$.
6. **Taylor with remainder.** Taylor's theorem says $\varphi(h) = \varphi(0) + h\varphi'(0) + \frac{h^2}{2}\varphi''(\xi)$ for some ξ between 0 and h. With the value and slope 0 and the curvature at most $\frac{1}{4}$, $\varphi(h) \le h^2/8 = \lambda^2(b-a)^2/8$.

The worst case is a fair coin on the two endpoints, whose variance is exactly $((b-a)/2)^2$: it puts all its mass as far from the mean as the interval allows.

2.5.5. Hoeffding's inequality

The average of n bounded independent variables is sub-Gaussian with a variance proxy that shrinks like $1/n$, so its tail is Gaussian-shaped with that proxy. Concretely, combine the lemma with the rule for means: the mean of n independent variables in $[a, b]$ is $\frac{b-a}{2\sqrt{n}}$-sub-Gaussian, so $\sigma^2 = (b-a)^2/(4n)$. Putting this into the tail bound $e^{-t^2/(2\sigma^2)}$ gives

$$\Pr\left[\bar{X}_n - \mu \ge t\right] \le \exp\left(-\frac{2nt^2}{(b-a)^2}\right). \tag{2.20}$$

As a two-sided confidence interval, add the matching lower tail to get $2\exp(-2nt^2/(b-a)^2)$, set it equal to β and solve for t (Fact 5): with probability at least $1 - \beta$, $|\bar{X}_n - \mu| < (b - a)\sqrt{\ln(2/\beta)/(2n)}$.

2.5.6. Interpretation

Hoeffding needs only the range, not the variance. That is its strength, since there is nothing to estimate, and its weakness, since it assumes the worst-case variance $(b-a)^2/4$, the variance of the fair coin on the endpoints.

When the true variance σ^2 is much smaller than that, use **Bernstein's inequality**, which keeps the variance in the bound. It is proved by the same Chernoff recipe, with a sharper bound on each factor's MGF that uses the variance and the size of the largest deviation (Boucheron, Lugosi & Massart, ch. 2, give the derivation). For $|X_i - \mu| \le M$:

$$\Pr\left[\bar{X}_n - \mu \ge t\right] \le \exp\left(-\frac{nt^2}{2\sigma^2 + \frac{2}{3}Mt}\right). \tag{2.21}$$

The denominator shows the two regimes. For small t the $2\sigma^2$ term dominates and the bound looks Gaussian with the true variance; for large t the $\frac{2}{3}Mt$ term dominates and the bound decays only exponentially in t. Setting the bound equal to β and solving the quadratic in t shows that its width is about $\sqrt{2\sigma^2 \ln(1/\beta)/n} + 2M\ln(1/\beta)/(3n)$: driven by the variance for moderate deviations, and by the range only at the far end.

Figure 2.3 puts all the bounds so far side by side for the mean of 200 Bernoulli(0.1) variables. Hoeffding even loses to Chebyshev at small deviations here, because the true variance $0.1 \times 0.9 = 0.09$ is far below Hoeffding's worst case of $1/4 = 0.25$ for variables in $[0, 1]$.

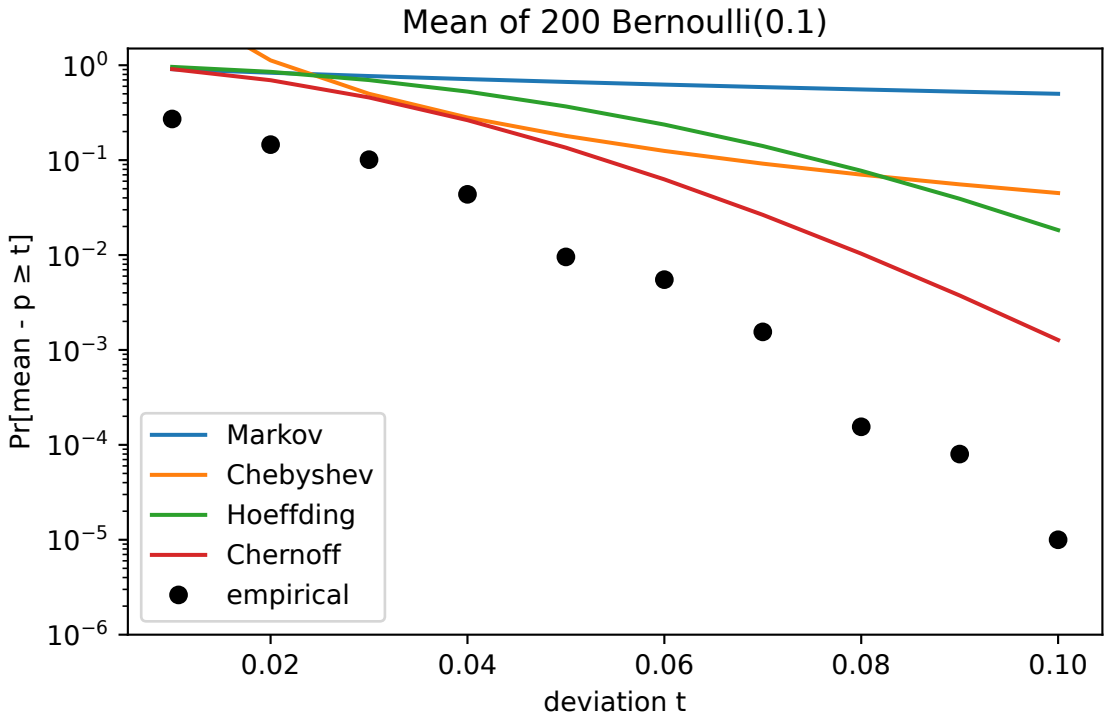


Figure 2.3.: Empirical tail of the mean of 200 Bernoulli(0.1) variables against the Markov, Chebyshev, Hoeffding and Chernoff bounds.

2.5.7. Example: generalization for a finite class

How far can a model's error on the training sample be from its error on the whole population? For one fixed model h with loss in $[0, 1]$, the **empirical risk** $\hat{R}(h)$, its average loss on n independent examples, is a mean of n bounded terms whose expectation is the **true risk** $R(h)$, the average loss on the population. Two-sided Hoeffding with $b - a = 1$ gives $\Pr[|\hat{R}(h) - R(h)| \geq t] \leq 2e^{-2nt^2}$.

But a learner picks the model after seeing the data, so you need the bound for every model in the class at once. For a finite class H, apply the union bound (Fact 2) over its $|H|$ models: the chance that any one of them is off by t or more is at most $2|H|e^{-2nt^2}$. Setting this to β and solving for t gives $t = \sqrt{\ln(2|H|/\beta)/(2n)}$, so with probability at least $1 - \beta$,

$$|\hat{R}(h) - R(h)| \leq \sqrt{\frac{\ln(2|H|/\beta)}{2n}} \quad \text{for all } h \in H. \tag{2.22}$$

The size of the class enters only through $\ln |H|$, which is the $\ln k$ cost of Fact 2.

2.5.8. Example: estimating a small error rate

A classifier is tested on 1,000 negative examples and makes 1 false positive. You want a 95% upper bound on its true false-positive rate, so the failure probability is $\beta = 0.05$ and $\ln(1/\beta) = \ln 20 \approx 3.0$. Three methods give very different answers.

Hoeffding, one-sided, adds the half-width $\sqrt{\ln(1/\beta)/(2n)} = \sqrt{\ln 20/2000} \approx 0.039$ to the measured rate of 1/1,000. That is about 40 times the rate you measured: the range $[0, 1]$ forces it to assume a variance of 0.25, when the true variance of a rate near 0.001 is about 0.001.

Bernstein uses that small variance. Plugging $\sigma^2 \approx 0.001$, $M = 1$ and $n = 1{,}000$ into its width gives $\sqrt{2 \times 0.001 \times 3.0/1000} + 2 \times 3.0/3000 \approx 0.0045$, almost ten times tighter. This is a heuristic: σ^2 was estimated from the same data, and a rigorous version (empirical Bernstein) is wider.

The exact answer comes from the binomial distribution itself. The **Clopper–Pearson** upper bound is the largest true rate p under which seeing 1 or fewer false positives in 1,000 trials still has probability at least 0.05; it is 0.0047.

Measuring rare rates is expensive: range-based bounds are useless near zero, and even exact intervals need many trials. This returns in later parts, wherever a small rate has to be certified empirically.

2.5.9. Check yourself

- Is a Laplace variable sub-Gaussian? *(No. Its MGF is infinite for $|\lambda| \geq 1/b$, so no quadratic bound holds for all λ. See the next section.)*
- Why is the fair coin on $\{a, b\}$ the worst case in Hoeffding's lemma? *(It maximizes the variance among variables on $[a, b]$, and step 5 is tight at $u = 1/2$.)*

2.6. Gaussian and Laplace tails

2.6.1. Context

The normal and Laplace distributions are the two standard models of additive noise, and their tails behave very differently: the normal tail decays like $e^{-t^2/2}$, the Laplace tail only like e^{-t}. This section gives sharp tail bounds for each, rules for the maximum of many draws, and the two-regime behaviour of sums of Laplace variables.

2.6.2. Gaussian tails

For the normal distribution you know the density exactly, so you can bound the tail integral directly instead of going through the MGF, and do better than Chernoff. Write $Z \sim \mathcal{N}(0,1)$ for a standard normal variable, with density $\varphi(z) = e^{-z^2/2}/\sqrt{2\pi}$. Two bounds hold for $t \geq 0$: the first is the Chernoff bound with an extra factor $\frac{1}{2}$, and the second, the **Mills ratio** bound, replaces the exponential by the density at t divided by t:

$$\Pr[Z \geq t] \leq \tfrac{1}{2} e^{-t^2/2}, \qquad \Pr[Z \geq t] \leq \frac{\varphi(t)}{t}. \tag{2.23}$$

Derivation of the first bound. The idea is to slide the tail back to start at 0 and compare it with the right half of the whole distribution, which has probability exactly $\frac{1}{2}$. Substitute $u = z - t$, so that the tail integral runs over $u \geq 0$. Expanding $(t+u)^2 = t^2 + 2tu + u^2$ splits the density into the factor $e^{-t^2/2}$, a factor e^{-tu}, and the standard normal density in u:

$$\Pr[Z \geq t] = \int_0^\infty \varphi(t+u)\,du = e^{-t^2/2} \int_0^\infty \frac{e^{-tu-u^2/2}}{\sqrt{2\pi}}\,du \leq e^{-t^2/2} \int_0^\infty \varphi(u)\,du = \tfrac{1}{2} e^{-t^2/2}. \tag{2.24}$$

The inequality drops the factor e^{-tu}, which is at most 1 for $t, u \geq 0$, and the last integral is $\frac{1}{2}$ because the normal density is symmetric around 0. This beats the Chernoff bound $e^{-t^2/2}$ of the warm-up by a factor of $\frac{1}{2}$.

Derivation of the second (Mills ratio). The idea is to multiply the density by a factor that is at least 1 on the tail and makes the integral easy. On $z \geq t$ the factor z/t is at least 1, and $z\varphi(z)$ has the simple antiderivative $-\varphi(z)$, because differentiating the density gives $\varphi'(z) = -z\varphi(z)$. So

$$\Pr[Z \geq t] = \int_t^\infty \varphi(z)\,dz \leq \int_t^\infty \frac{z}{t}\varphi(z)\,dz = \frac{1}{t}\big[-\varphi(z)\big]_t^\infty = \frac{\varphi(t)}{t}. \tag{2.25}$$

This bound is asymptotically exact as t grows, because the tail mass sits ever closer to t, where the factor z/t is close to 1. Figure 2.4 compares the bounds with the exact tail. At $t = 3$ the exact tail is 1.35×10^{-3}. Chebyshev gives $1/3^2 = 0.111$, Chernoff $e^{-9/2} = 0.0111$, the halved bound 0.0056, and Mills $\varphi(3)/3 = 0.00148$.

Working rules. These are the two widths you will use most for a normal variable $X \sim \mathcal{N}(0, \sigma^2)$; a normal with standard deviation σ is σZ, so every threshold scales by σ. For one draw, the first bound doubled covers both tails, $\Pr[|X| \geq t] \leq e^{-t^2/(2\sigma^2)}$, and setting this to β gives $\Pr[|X| \geq \sigma\sqrt{2\ln(1/\beta)}] \leq \beta$. For the maximum of k draws, ask each draw to fail with probability β/k and apply the union bound (Fact 2): every $|X_i| < \sigma\sqrt{2\ln(k/\beta)}$ with probability at least $1 - \beta$. Since $\ln(k/\beta) = \ln k + \ln(1/\beta)$, controlling k draws costs only a factor of about $\sqrt{\ln k}$ in the width.

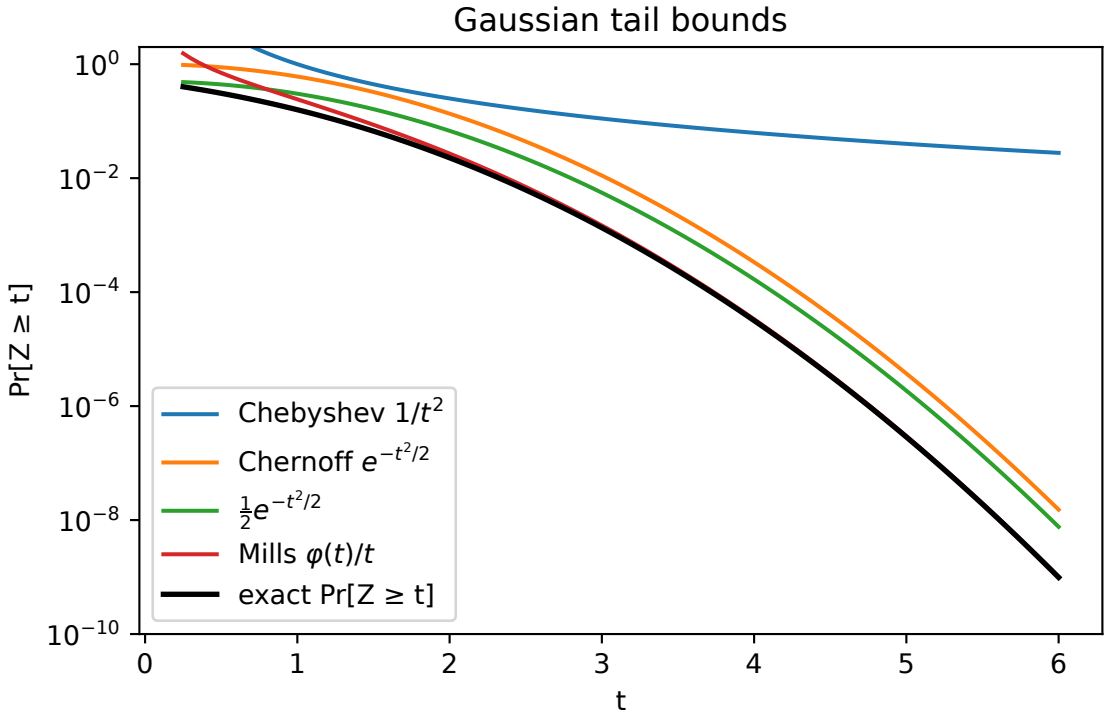


Figure 2.4.: The exact standard normal tail against the Chebyshev, Chernoff, halved Chernoff and Mills-ratio bounds.

2.6.3. Laplace tails

For the Laplace distribution the tail is exactly computable, so no bound is needed. Lap(b) has density $\frac{1}{2b}e^{-|x|/b}$, and integrating it from $t \geq 0$ to infinity gives

$$\Pr[X \geq t] = \tfrac{1}{2}e^{-t/b}, \qquad \Pr[|X| \geq t] = e^{-t/b}, \tag{2.26}$$

where the second form doubles the first because the density is symmetric.

Working rules. Setting $e^{-t/b} = \beta$ and solving gives the width for one draw: with probability at least $1-\beta$, $|X| \leq b\ln(1/\beta)$. For the maximum of k independent draws, ask each to fail with probability β/k and use the union bound: with probability at least $1-\beta$, every $|X_i| \leq b\ln(k/\beta)$.

The union bound is nearly exact here. All k independent draws stay below t with probability $(1-e^{-t/b})^k$, so $\Pr[\max_i |X_i| > t] = 1-(1-e^{-t/b})^k$, and when the per-draw tail $x = e^{-t/b}$ is small, $(1-x)^k \approx 1-kx$, so this is $\approx ke^{-t/b}$, exactly the union bound's sum. Figure 2.5 confirms it: the empirical failure rate sits at β for every k from 1 to 1,000.

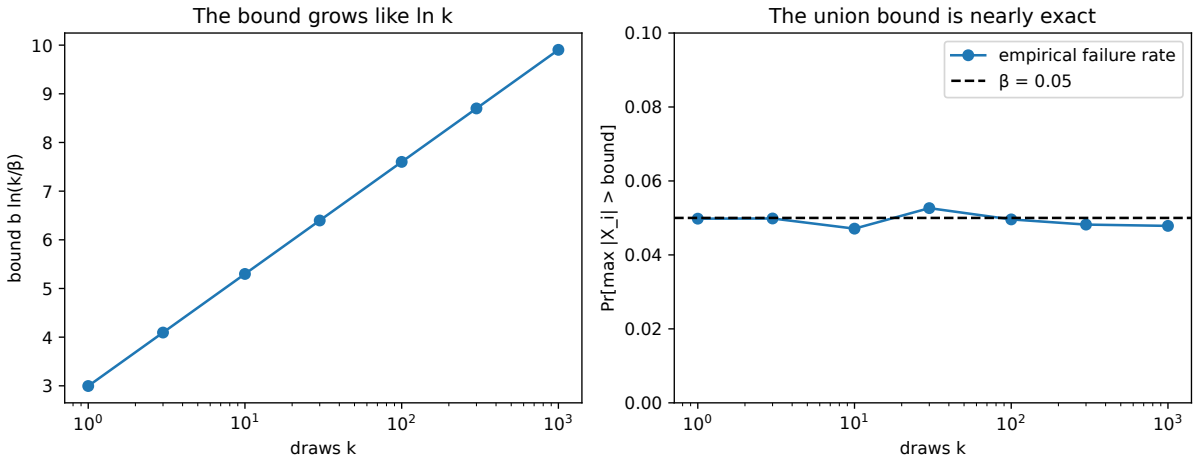


Figure 2.5.: Left: the bound $b\ln(k/\beta)$ grows like $\ln k$. Right: the empirical probability that the maximum of k Laplace draws exceeds it stays at $\beta = 0.05$.

2.6.4. Laplace is sub-exponential, not sub-Gaussian

MGF. The Laplace MGF is finite only for small λ, which rules out the sub-Gaussian definition (it must hold for every λ). Integrating $e^{\lambda x}$ against the density, the right half gives $\int_0^\infty \frac{1}{2b}e^{\lambda x - x/b}\,dx = \frac{1}{2(1-\lambda b)}$, which is finite only for $\lambda < 1/b$, and the left half gives $\frac{1}{2(1+\lambda b)}$. Adding them, for $|\lambda| < 1/b$,

$$\mathbb{E}[e^{\lambda X}] = \frac{1}{1-\lambda^2 b^2} \tag{2.27}$$

(Probability distributions). It blows up as $|\lambda| \to 1/b$, so no bound of the form $e^{\lambda^2\sigma^2/2}$ can hold for all λ.

A quadratic bound near zero. For small λ, though, the log-MGF is still at most quadratic, which is all the Chernoff recipe needs as long as you only use those λ. The log-MGF is $-\ln(1-x)$

with $x = \lambda^2 b^2$, and $-\ln(1-x) = x + x^2/2 + x^3/3 + ... \le x + x^2 + \cdots = \frac{x}{1-x}$. For $0 \le x \le \frac{1}{2}$, $\frac{1}{1-x} \le 2$, so $-\ln(1-x) \le \frac{x}{1-x} \le 2x$. For $|\lambda| \le 1/(2b)$ you have $x \le \frac{1}{4}$, well inside that range, so

$$\mathbb{E}[e^{\lambda X}] \le e^{2\lambda^2 b^2}. \tag{2.28}$$

Sum of k independent Lap(b)**.** Let S be the sum of k independent Lap(b) variables. By Fact 4 the MGF of S is the product of k factors, each at most $e^{2\lambda^2 b^2}$, so $\mathbb{E}[e^{\lambda S}] \le e^{2k\lambda^2 b^2}$ for $|\lambda| \le 1/(2b)$. Run Chernoff with λ restricted to $(0, 1/(2b)]$. The result depends on whether the best λ is inside the allowed range.

1. **Find the unconstrained optimum.** The exponent to minimize is $-\lambda t + 2k\lambda^2 b^2$, a parabola whose derivative $-t + 4k\lambda b^2$ vanishes at $\lambda^* = t/(4kb^2)$.
2. **Small deviations.** $\lambda^* \le 1/(2b)$ exactly when $t \le 2kb$. Then λ^* is allowed, and substituting it gives the exponent $-t^2/(4kb^2) + t^2/(8kb^2) = -t^2/(8kb^2)$, so the bound is $\exp(-t^2/(8kb^2))$: Gaussian-like.
3. **Large deviations.** If $t > 2kb$, the best allowed λ is the boundary $\lambda = 1/(2b)$, which gives $\exp(-t/(2b) + k/2)$. In this regime $k/2 < t/(4b)$, so the exponent is at most $-t/(2b) + t/(4b) = -t/(4b)$, and the bound is at most $\exp(-t/(4b))$.

The two cases combine into one formula, because in each regime the smaller of the two expressions inside the minimum is the one that applies:

$$\Pr[S \ge t] \le \exp\left(-\min\left(\frac{t^2}{8kb^2},\ \frac{t}{4b}\right)\right). \tag{2.29}$$

Small deviations look Gaussian, with variance on the order of kb^2. Large deviations decay only exponentially. That two-regime shape is what *sub-exponential* means.

Figure 2.6 shows why the distinction matters. Take $k = 10$ and $b = 1$. The central limit theorem (CLT) says a sum of many independent terms is approximately normal with the same mean and variance, here mean 0 and variance $10 \times 2b^2 = 20$. That normal approximation matches the bulk: $\Pr[S \ge 10]$ is 1.42×10^{-2} empirically and 1.27×10^{-2} from the normal. But it badly underestimates the tail: $\Pr[S \ge 20]$ is 3.4×10^{-5} empirically and only 3.9×10^{-6} from the normal. Both deviations are in the Gaussian regime of the bound, since $t \le 2kb = 20$, so the bound gives $e^{-100/80} = 0.287$ at $t = 10$ and $e^{-400/80} = e^{-5} = 6.7 \times 10^{-3}$ at $t = 20$. The sub-exponential bound is loose, but it is valid everywhere.

2.6.5. Check yourself

- Why does the maximum of k normals grow like $\sqrt{\ln k}$ but the maximum of k Laplace draws like $\ln k$? *(Invert the tails: $e^{-t^2/2} = \beta/k$ gives $t = \sqrt{2\ln(k/\beta)}$, while $e^{-t/b} = \beta/k$ gives $t = b\ln(k/\beta)$.)*
- What does dropping the $\frac{1}{2}$ in the Gaussian tail bound cost in the width? *($\ln(1/(2\beta))$ becomes $\ln(1/\beta)$: $\ln 2 \approx 0.7$ more inside the square root.)*

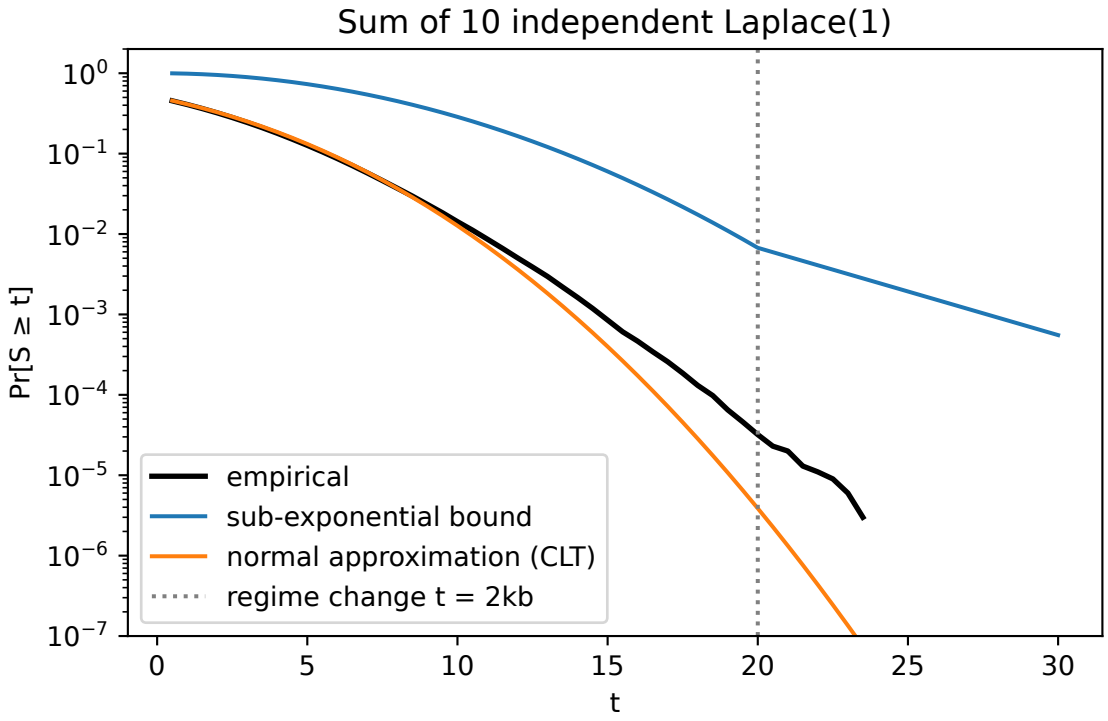


Figure 2.6.: Empirical tail of a sum of 10 Laplace(1) variables, against the sub-exponential bound and the normal approximation.

2.7. McDiarmid and Azuma

2.7.1. Context

So far, everything was a sum of independent terms. Two common situations break that. First, quantities such as a trained model's empirical risk are complicated functions of the data, not sums, so there is no MGF to factorize. Second, in sequential procedures each step can be chosen after seeing earlier outcomes, so the terms are dependent and Fact 4 no longer applies.

Azuma's inequality handles the second situation: dependent sums whose terms are "fresh" given the past, in the sense that each new term averages to zero whatever happened before. Such a running total is called a martingale. McDiarmid's inequality handles the first: it follows from Azuma by revealing the inputs of the function one at a time, and it applies to any function that no single input can move by much.

2.7.2. Azuma–Hoeffding

Setup. A **martingale** is a running total whose next step is zero on average given everything that has happened so far, like your fortune in a sequence of fair bets where you may choose each bet based on the previous ones. Formally, $M_0, M_1, \dots, M_n$ is a martingale if each increment $D_k = M_k - M_{k-1}$ has conditional mean zero given the past, $\mathbb{E}[D_k \mid \text{past}] = 0$. Assume each D_k lies in an interval $[A_k, A_k + d_k]$, where the position A_k may depend on the past but the length d_k is fixed in advance.

Statement. The total drift $M_n - M_0$ obeys the same tail bound as a sum of independent bounded terms, with the step lengths d_k playing the role of the ranges:

$$\Pr[M_n - M_0 \geq t] \leq \exp\left(-\frac{2t^2}{\sum_k d_k^2}\right). \tag{2.30}$$

Derivation. The Chernoff recipe needs the MGF of the total. Without independence it does not factorize, but you can still bound it one step at a time, starting from the last step and working back.

1. **Bound one step given the past.** Given the past, D_k is a mean-zero variable on an interval of length d_k, so Hoeffding's lemma (previous section) applies to it, with every expectation taken given the past: $\mathbb{E}[e^{\lambda D_k} \mid \text{past}] \leq e^{\lambda^2 d_k^2/8}$. The proof is the one in the previous section, run conditionally.
2. **Peel off the last increment.** The **tower property** says you can average in two stages: first average over the last step with the past held fixed, then average over the past; the result is the ordinary average. Split $M_n - M_0 = (M_{n-1} - M_0) + D_n$. With the past held fixed, $e^{\lambda(M_{n-1} - M_0)}$ is a constant and comes out of the inner average, which is then bounded by step 1:

$$\mathbb{E}\big[e^{\lambda(M_n - M_0)}\big] = \mathbb{E}\Big[e^{\lambda(M_{n-1} - M_0)}\, \mathbb{E}\big[e^{\lambda D_n} \mid \text{past}\big]\Big] \leq e^{\lambda^2 d_n^2/8}\, \mathbb{E}\big[e^{\lambda(M_{n-1} - M_0)}\big]. \tag{2.31}$$

3. **Repeat.** Apply the same step to $M_{n-1} - M_0$, then $M_{n-2} - M_0$, and so on, n times: $\mathbb{E}[e^{\lambda(M_n - M_0)}] \leq \exp(\lambda^2 \sum_k d_k^2/8)$. Peeling replaces Fact 4, which needed independence.

4. **Chernoff.** The exponent to minimize is $-\lambda t + \lambda^2 \sum_k d_k^2/8$. Its derivative vanishes at $\lambda = 4t/\sum_k d_k^2$, and substituting gives $\exp(-2t^2/\sum_k d_k^2)$.

Hoeffding is the special case of independent increments: there the past tells you nothing, and the conditional bound of step 1 is the ordinary lemma.

2.7.3. Example: an adaptive coin game

You have two coins, with heads probabilities 0.3 and 0.7. You flip 200 times; before each flip you pick the 0.7 coin if the previous flip was heads, and the 0.3 coin otherwise. The flips are dependent, and the number of heads is not binomial, so Hoeffding does not apply.

What you can control is how far the heads count drifts from what your own choices predicted. Let X_k be 1 if flip k is heads, let p_k be the bias of the coin chosen at step k, and let $M_n = \sum_{k \le n}(X_k - p_k)$: heads minus the heads you "expected" given your choices. Given the past, the coin is fixed, so $X_k - p_k$ has mean $p_k - p_k = 0$ and lies in $[-p_k, 1 - p_k]$, an interval of length 1. So M_n is a martingale with $d_k = 1$, $\sum_k d_k^2 = 200$, and Azuma gives $\Pr[M_{200} \ge t] \le e^{-2t^2/200}$. At $t = 20$ the exponent is $2 \times 400/200 = 4$ and the bound is $e^{-4} = 1.83 \times 10^{-2}$, while simulation gives 1.12×10^{-3} (Figure 2.7). The bound pays for assuming the worst-case variance, exactly as Hoeffding does: each step's true variance is $p_k(1 - p_k) = 0.21$, not the 0.25 the interval allows.

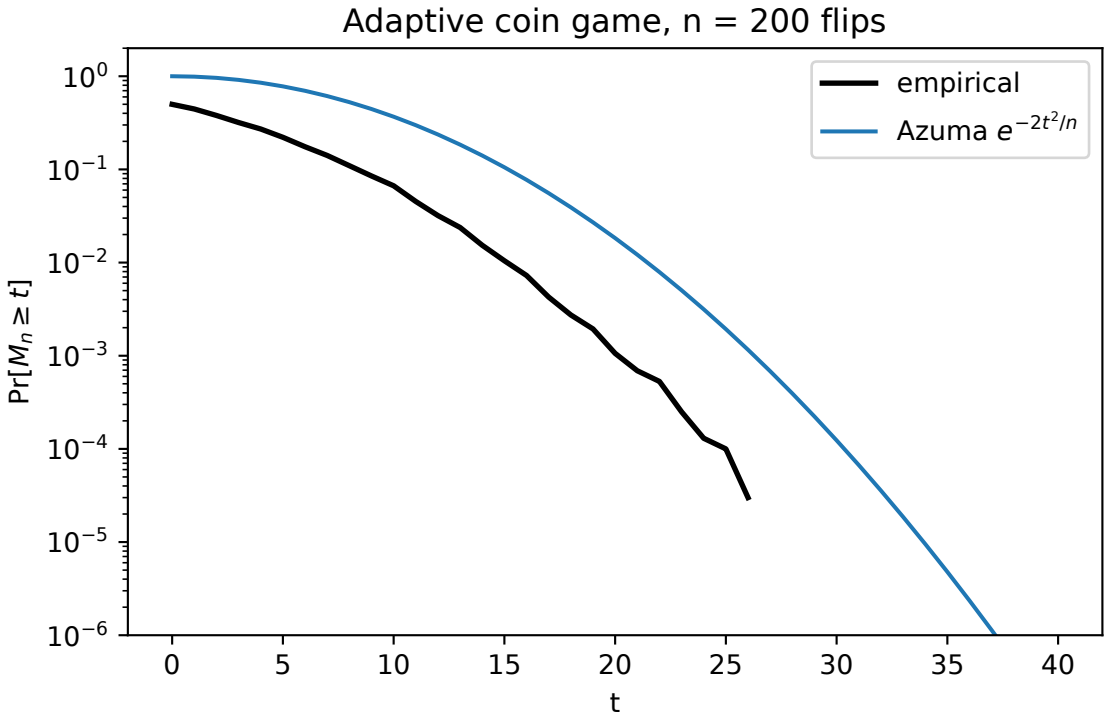


Figure 2.7.: Empirical tail of the martingale in the adaptive coin game against the Azuma bound.

2.7.4. McDiarmid (bounded differences)

Statement. If a function of many independent inputs cannot be moved much by any single input, it concentrates around its mean as tightly as a sum of bounded terms would. Let $X_1, \dots, X_n$ be independent, and suppose that changing only the i-th input, with all others held fixed, changes f by at most c_i. Then

$$\Pr\left[f(X) - \mathbb{E}f(X) \geq t\right] \leq \exp\left(-\frac{2t^2}{\sum_i c_i^2}\right). \tag{2.32}$$

Derivation via the Doob martingale. The idea is to turn f into a martingale by revealing the inputs one at a time and tracking your best guess of f as you go. Each revealed input moves the guess by a bounded amount, and Azuma does the rest.

1. **Build the martingale.** After seeing $X_1, \dots, X_k$, your best guess of $f(X)$ is its conditional average, $M_k = \mathbb{E}[f(X) \mid X_1, \dots, X_k]$. Before seeing anything, the guess is $M_0 = \mathbb{E}f$; after seeing everything, it is $M_n = f(X)$. So $M_n - M_0$ is exactly the deviation you want to bound. (M_k) is a martingale by the tower property: averaging the next guess M_k over the not-yet-seen X_k gives back the current guess M_{k-1}, so each increment has mean zero given the past.
2. **Bound the range of each increment.** By independence, the unseen inputs do not depend on the seen ones, so M_k is the average of f over fresh copies of $X_{k+1}, \dots, X_n$, with $X_1, \dots, X_k$ fixed at their observed values. Hold $X_1, \dots, X_{k-1}$ fixed. Then M_k depends only on X_k, and changing X_k changes every term of that average by at most c_k, so it changes the average by at most c_k. As X_k varies, M_k therefore stays in an interval of length at most c_k, and so does $D_k = M_k - M_{k-1}$, whose position depends on the past through M_{k-1}.
3. **Apply Azuma** with $d_k = c_k$.

Interpretation. The bounded-difference constant c_i measures how much one input can move the output: a per-record influence. Small per-record influence forces the output to concentrate. The same quantity returns in later parts under the name *sensitivity*.

Example: stability. The empirical risk of a fixed model with loss in $[0, 1]$ is an average of n losses, so changing one example changes it by at most $1/n$: $c_i = 1/n$, $\sum_i c_i^2 = n/n^2 = 1/n$, and McDiarmid gives concentration $\exp(-2nt^2)$. That is Hoeffding again, now as a special case. The real power is for functions that are not sums. Bousquet & Elisseeff apply McDiarmid to the risk of a model *trained* on the data: if retraining after changing one example moves the loss by at most β (here a stability constant, not a failure probability), each input has bounded influence on the trained model's risk, and the generalization gap concentrates (Algorithmic stability and generalization).

2.7.5. Check yourself

- Why can't you use Hoeffding in the adaptive coin game? *(The coin chosen at each step depends on earlier flips, so the flips are not independent. Only their conditional structure is controlled.)*
- In the McDiarmid derivation, where is the independence of the X_i used? *(In step 2: the average over the unrevealed inputs must not change when X_k changes.)*

2.8. Code: check every bound by simulation

The script code/concentration.py draws every figure in this chapter and computes the numbers quoted in it. Each bound and each simulated quantity is a small typed function; the script calls them once for the table and loops them over parameters for the figures. From the repository root:

```
uv run python code/part01-prerequisites/concentration.py
```

Table 2.5 shows the output with seed 0.

Table 2.5.: Results of the chapter's script with seed 0.

Experiment	Result
Bernoulli mean, $n = 200$, $p = 0.1$, $t = 0.05$	Empirical 9.6×10^{-3}; Markov 0.67; Chebyshev 0.18; Hoeffding 0.37; Chernoff 0.135
Maximum of $k = 100$ draws of Lap(1), $\beta = 0.05$	Bound 7.60; empirical failure rate 0.0486
Standard normal tail at $t = 3$	Exact 1.35×10^{-3}; Chebyshev 0.111; Chernoff 0.0111; halved 0.0056; Mills 0.00148
Sum of $k = 10$ draws of Lap(1)	$t = 10$: empirical 1.42×10^{-2}, bound 0.287, normal 1.27×10^{-2}. $t = 20$: empirical 3.4×10^{-5}, bound 6.7×10^{-3}, normal 3.9×10^{-6}
Adaptive coin game, $n = 200$, $t = 20$	Empirical 1.12×10^{-3}; Azuma 1.83×10^{-2}

tests/test_concentration.py checks that the output still matches this table.

What to take from it

The first row shows that variance matters. The mean of 200 Bernoulli(0.1) variables has mean 0.1 and variance 0.09/200, and the question is how likely it is to reach 0.15. Markov gives $0.1/0.15 = 0.67$, Chebyshev $0.09/(200 \times 0.05^2) = 0.18$, Hoeffding $e^{-2\times 200\times 0.05^2} = e^{-1} = 0.37$, and Chernoff, with $\mu = 20$ and $\delta = 0.5$, $e^{-0.25\times 20/2.5} = e^{-2} = 0.135$. Hoeffding loses to Chebyshev because $p(1-p) = 0.09$ is far below the worst case 0.25: range-based bounds pay for ignoring the variance.

The second row shows that a union bound can be nearly exact. The bound $\ln(100/0.05) = 7.60$ fails at the empirical rate 0.0486, close to the target 0.05, because for independent draws with small individual tails the chance that any one fails is almost the sum of the individual chances.

The third row shows that the Mills ratio is the bound to use far out. At $t = 3$ it is within 10% of the exact tail, while the Chernoff bound is 8 times too large.

The fourth row shows that the CLT is not a tail bound. For the Laplace sum it underestimates $\Pr[S \geq 20]$ by a factor of 9, since 3.4×10^{-5} is about nine times 3.9×10^{-6}. When the answer must be a guarantee, use a bound.

The last row shows that Azuma handles adaptivity, at the price of the worst-case variance.

2.9. References

- Mitzenmacher & Upfal, *Probability and Computing*, ch. 4: the clearest Chernoff chapter, with worked computer-science examples
- Vershynin, *High-Dimensional Probability* (2018), ch. 2: sub-Gaussian and sub-exponential variables in depth. Read it after this chapter rather than before.
- Boucheron, Lugosi & Massart, *Concentration Inequalities* (2013), ch. 2 and 6: reference depth
- Bousquet & Elisseeff, "Stability and Generalization" (JMLR 2002): McDiarmid applied to learning algorithms

2.10. Cheat sheet

Table 2.6 collects the bounds. Tails are one-sided unless noted, and β is the failure probability.

Table 2.6.: Summary of the bounds in this chapter.

Bound	Assumption	Tail $\Pr[X-\mu \geq t] \leq$	Width at failure probability β
Markov	$X \geq 0$	μ/t (for $\Pr[X \geq t]$)	μ/β
Chebyshev	Variance σ^2	σ^2/t^2 (two-sided)	$\sigma/\sqrt{\beta}$
Sub-Gaussian	MGF $\leq e^{\lambda^2\sigma^2/2}$	$e^{-t^2/(2\sigma^2)}$	$\sigma\sqrt{2\ln(1/\beta)}$
Hoeffding	Mean of n independent variables in $[a,b]$	$e^{-2nt^2/(b-a)^2}$	$(b-a)\sqrt{\ln(1/\beta)/(2n)}$
Chernoff (multiplicative)	Sum of independent Bernoullis, mean μ	$e^{-\delta^2\mu/(2+\delta)}$ at $t=\delta\mu$	$\approx \sqrt{2\mu\ln(1/\beta)}$ for small δ
Gaussian $\mathcal{N}(0,\sigma^2)$	—	$\frac{1}{2}e^{-t^2/(2\sigma^2)}$	$\sigma\sqrt{2\ln(1/\beta)}$, two-sided
Laplace Lap(b)	—	$\frac{1}{2}e^{-t/b}$; two-sided exactly $e^{-t/b}$	$b\ln(1/\beta)$, two-sided
McDiarmid	Independent inputs, bounded differences c_i	$e^{-2t^2/\sum_i c_i^2}$	$\sqrt{\sum_i c_i^2 \ln(1/\beta)/2}$
Azuma–Hoeffding	Martingale, increments in intervals of length d_i	$e^{-2t^2/\sum_i d_i^2}$	$\sqrt{\sum_i d_i^2 \ln(1/\beta)/2}$
Sum of k Laplace	Independent Lap(b)	$e^{-\min(t^2/(8kb^2),\ t/(4b))}$	$b\sqrt{8k\ln(1/\beta)}$ while that is $\leq 2kb$; else $4b\ln(1/\beta)$

Rules of thumb

- Variance-only bounds cost $1/\sqrt{\beta}$; exponential bounds cost $\sqrt{\ln(1/\beta)}$; heavy (sub-exponential) tails cost $\ln(1/\beta)$.
- k estimates add $\ln k$ inside the logarithm, via the union bound.
- If the variance is much smaller than the range allows, use Chernoff or Bernstein, not Hoeffding.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

3. Divergences

3.1. Why this matters

Someone hands you a die. It is either fair, or loaded so that a six comes up half the time. You may roll it as often as you like. Three questions come up at once:

- **On average,** how much does one roll tell you about which die you hold?
- **In the worst case,** how strongly can a single roll point the wrong way?
- **With one roll,** how often can the best possible guess be right?

Each question has its own answer, and each answer is a **divergence**: a number $D(P\|Q)$ that is zero when two distributions P and Q are equal and grows as they become easier to tell apart. The same questions come up all over machine learning. When you fit a model q_θ to data from p, you need to know how far the fit is from the truth. When you run an A/B test or watch a feature drift in production, you need to know how many samples it takes to see the change. When you perturb the data a little, you need to know how much any downstream statistic can move. Later parts ask the same questions about the outputs of an algorithm run on two different inputs.

A divergence is not a distance in general. Asking "how fast do rolls of the loaded die expose it?" is a different question from "how fast do rolls of the fair die clear it?", so a divergence can be asymmetric, $D(P\|Q) \neq D(Q\|P)$, and it can fail the triangle inequality.

3.1.1. The one idea: the likelihood ratio

You see an outcome x and want to know which distribution it came from. The natural thing to compare is how likely x is under each one. Their quotient, the **likelihood ratio**

$$r(x) = \frac{p(x)}{q(x)}, \tag{3.1}$$

says how many times more likely x is under P than under Q. Its logarithm, $\ln r(x)$, is the **evidence** that x gives for P: positive when x favours P, negative when it favours Q, zero when x says nothing. The logarithm is used because evidence from independent observations should add up, and logarithms turn the product of their ratios into a sum. Natural logarithms measure evidence in units called **nats**. Table 3.1 shows the ratio and the evidence for the die, with P the loaded die and Q the fair one.

Table 3.1.: The loaded die P against the fair die Q, outcome by outcome. This is the running example of the chapter.

Roll x	1	2	3	4	5	6
$p(x)$ (loaded)	0.1	0.1	0.1	0.1	0.1	0.5
$q(x)$ (fair)	1/6	1/6	1/6	1/6	1/6	1/6
$r(x) = p(x)/q(x)$	0.6	0.6	0.6	0.6	0.6	3
$\ln r(x)$	-0.511	-0.511	-0.511	-0.511	-0.511	1.099

A six is three times more likely under the loaded die ($0.5/\frac{1}{6} = 3$). It is fairly strong evidence for "loaded", $\ln 3 = 1.10$ nats. Any other roll is $0.1/\frac{1}{6} = 0.6$ times as likely under the loaded die, weak evidence for "fair", $\ln 0.6 = -0.51$ nats. For continuous pairs, $r(x)$ is a curve. Figure 1.7 in Probability distributions draws $\ln r$, written $L(x)$ there, for a normal density and for a Laplace density, each against a copy of itself shifted sideways. The normal's log-ratio is a straight line that grows without bound. The Laplace's is flat beyond a point, so it never exceeds a fixed value. That difference comes back in every section of this chapter.

Every divergence in this chapter is a summary of the likelihood ratio. The divergences differ only in which summary they take, and each summary answers one question (Table 3.2). In the table, $\mathbb{E}_P$ means an average over outcomes drawn from P, $\mathbb{E}_Q$ an average over outcomes drawn from Q, and $(y)_+ = \max(y, 0)$ keeps only the positive part of a number.

Table 3.2.: Each divergence answers one question, by summarizing the likelihood ratio $r = p/q$ in its own way.

Question	Divergence	Summary of r
How much evidence does one observation give, on average?	Kullback–Leibler (KL)	$\mathbb{E}_P[\ln r]$, the average evidence
The same, but counting rare large ratios more	Rényi of order α	the log of a power mean of r, from typical ($\alpha \to 1$) to largest ($\alpha \to \infty$)
How strong is the strongest evidence any outcome can give?	Max-divergence	$\ln \max r$
How often is the best single guess right?	Total variation (TV)	$\mathbb{E}_Q[(r-1)_+]$, the mass where P beats Q
The same, when one answer must be γ times more convincing	Hockey-stick E_γ	$\mathbb{E}_Q[(r-\gamma)_+]$, the mass where P beats γQ
All of the above at once	f-divergences	$\mathbb{E}_Q[f(r)]$ for a convex f

Because every divergence is a function of r, there is one rule they all obey. Processing the data, by binning it, computing a feature or adding noise, can only average likelihood ratios together. It can never sharpen them, so no divergence can grow. That is the **data-processing inequality**.

Table 3.3 lists the results of the chapter and where each is used.

Table 3.3.: The results of this chapter and where each is used.

Result	Where it is used
KL, Gibbs' inequality	Maximum likelihood, variational inference, the cost of a wrong model
Additivity over independent samples	Sample sizes for telling two distributions apart
Rényi divergence, its limits and monotonicity	Tail bounds on the evidence; a single family from average case to worst case
Max-divergence	The worst-case ratio of probabilities of any event
Total variation	Best single-shot test; how much any bounded statistic can change
Hockey-stick E_γ	Tests with unequal priors or costs; rejection sampling with a low envelope
f-divergences	One definition and one proof for every divergence above
Data-processing inequality	Coarse-graining, features and summaries never increase a divergence
Pinsker, Bretagnolle–Huber	Turning an easy KL calculation into a bound on TV

How to read this chapter. On a first pass, read the KL, Rényi, total variation, data-processing and Pinsker sections, and in each one read the context, statement, interpretation and example before the derivation. The Gaussian and Laplace sections are worked examples; do the Gaussian one by hand. Two derivations are there for completeness and can wait for a second pass: the Laplace Rényi integral and the hockey-stick representation of f-divergences (Equation 3.27).

The distributions used here (normal, Laplace) and the comparison of a density with its shifted copy are in Probability distributions. The Chernoff method and the log of the moment generating function (MGF) are in Concentration inequalities. Where this chapter uses a fact from those pages, it restates it.

Conventions. P and Q have probability mass functions or densities p and q. We write sums $\sum_x$; for densities, read them as integrals. Logarithms are natural, so divergences are in nats. We use $0 \ln 0 = 0$. When P puts mass where Q puts none, $r = \infty$ there and most divergences below are infinite.

3.2. Toolkit: three facts every derivation uses

Each section below follows the same arc: the question the divergence answers, its definition and properties, derivations, an interpretation, an example, and a self-check. The derivations reuse three facts. Skim them now and come back when a derivation cites one.

3.2.1. Fact 1 — Change of measure: $\mathbb{E}_P[g] = \mathbb{E}_Q[r\,g]$

Some divergences are averages over outcomes drawn from P, others over outcomes drawn from Q. This fact lets you switch between the two. The idea: an outcome that is three times as likely under P as under Q should count three times as much when you average under Q but want the answer that P would give. Multiplying each term by $r(x) = p(x)/q(x)$ does exactly that reweighting. For any function g, as long as Q gives positive probability to every outcome that P does,

$$\mathbb{E}_P[g(X)] = \sum_x p(x)g(x) = \sum_x q(x)\frac{p(x)}{q(x)}g(x) = \mathbb{E}_Q[r(X)\,g(X)]. \tag{3.2}$$

The middle step only multiplies and divides by $q(x)$.

Two special cases come up again and again. With $g = 1$ the left side is 1, so $\mathbb{E}_Q[r] = 1$: the likelihood ratio always averages to 1 under Q. On the die, $\frac{1}{6}(5 \times 0.6 + 3) = 1$. In words, if the data really come from Q, the outcomes that favour P are rare enough to balance how strongly they favour it. With $g = \mathbf{1}_A$, the indicator that is 1 on an event A and 0 elsewhere, the fact reads $P(A) = \mathbb{E}_Q[r\,\mathbf{1}_A]$. For the event "roll is 6" this gives $\frac{1}{6} \times 3 = 0.5$, the loaded die's probability of a six.

3.2.2. Fact 2 — Jensen's inequality

A function φ is **convex** when its graph bends upward, like a bowl, so that every chord lies above the graph. For such a function, averaging first and then applying φ gives less than applying φ first and then averaging:

$$\varphi(\mathbb{E}X) \le \mathbb{E}\varphi(X). \tag{3.3}$$

If φ is strictly convex (no straight pieces), equality holds only when X is constant.

The reason is that a convex function lies above each of its tangent lines. Let $m = \mathbb{E}X$ and take the tangent at m: $\varphi(x) \ge \varphi(m) + c(x - m)$ for every x, where c is the slope there. Average both sides over X. The term $c(X - m)$ averages to 0, because X averages to m, which leaves $\mathbb{E}\varphi(X) \ge \varphi(m)$. If φ is strictly convex, the tangent touches the graph only at m, so equality forces $X = m$ with probability 1.

A small case shows the gap. Let X be 0.6 or 3 with probability $\frac{1}{2}$ each, the die's two likelihood ratios, and $\varphi(x) = x^2$. Then $\varphi(\mathbb{E}X) = 1.8^2 = 3.24$, while $\mathbb{E}\varphi(X) = \frac{1}{2}(0.36 + 9) = 4.68$. Jensen is what makes every divergence non-negative, what orders the Rényi divergences by their order, and what drives the data-processing inequality.

3.2.3. Fact 3 — The best event is $\{r > \gamma\}$

Several divergences ask for the event A that makes $P(A) - \gamma Q(A)$ as large as possible, for some weight $\gamma \ge 0$. The answer is to collect every outcome where p beats γq and nothing else. Writing $(y)_+ = \max(y, 0)$,

$$\max_A \left[P(A) - \gamma Q(A)\right] = \sum_x \left(p(x) - \gamma q(x)\right)_+, \tag{3.4}$$

attained at $A^* = \{x : p(x) > \gamma q(x)\} = \{r > \gamma\}$.

The reason is that the quantity is a sum over the outcomes in A: $P(A) - \gamma Q(A) = \sum_{x \in A}(p(x) - \gamma q(x))$. An outcome adds a positive term exactly when $p(x) > \gamma q(x)$, and a negative or zero term otherwise. So the sum is largest when A contains all the outcomes with positive terms and none of the others, and its value is then the sum of the positive parts.

Randomized choices do no better. Instead of a set, you could include each outcome with some probability $\phi(x) \in [0, 1]$, and get $\sum_x \phi(x)(p(x) - \gamma q(x))$. Each term is a number between 0 and the bracket, so the sum is largest when $\phi(x) = 1$ where the bracket is positive and $\phi(x) = 0$ elsewhere: that is the indicator of A^* again.

On the die with $\gamma = 1$, only the six has $p > q$, so $A^* = \{6\}$ and the maximum is $0.5 - \frac{1}{6} = \frac{1}{3}$. This fact gives the total variation, the hockey-stick divergence and Pinsker's inequality.

3.3. KL divergence

3.3.1. Context

Roll the die n times and add up the evidence from each roll: $\sum_{i=1}^{n} \ln r(x_i)$. If the die is loaded, sixes come up often and the sum climbs. If it is fair, the many small negative terms win and the sum falls. Figure 3.1 shows eight runs of each. The runs are noisy, but each family drifts along a straight line. The slope of that line, the average evidence per roll, is the **KL divergence**. It is the speed at which data from P convince you that they are not from Q.

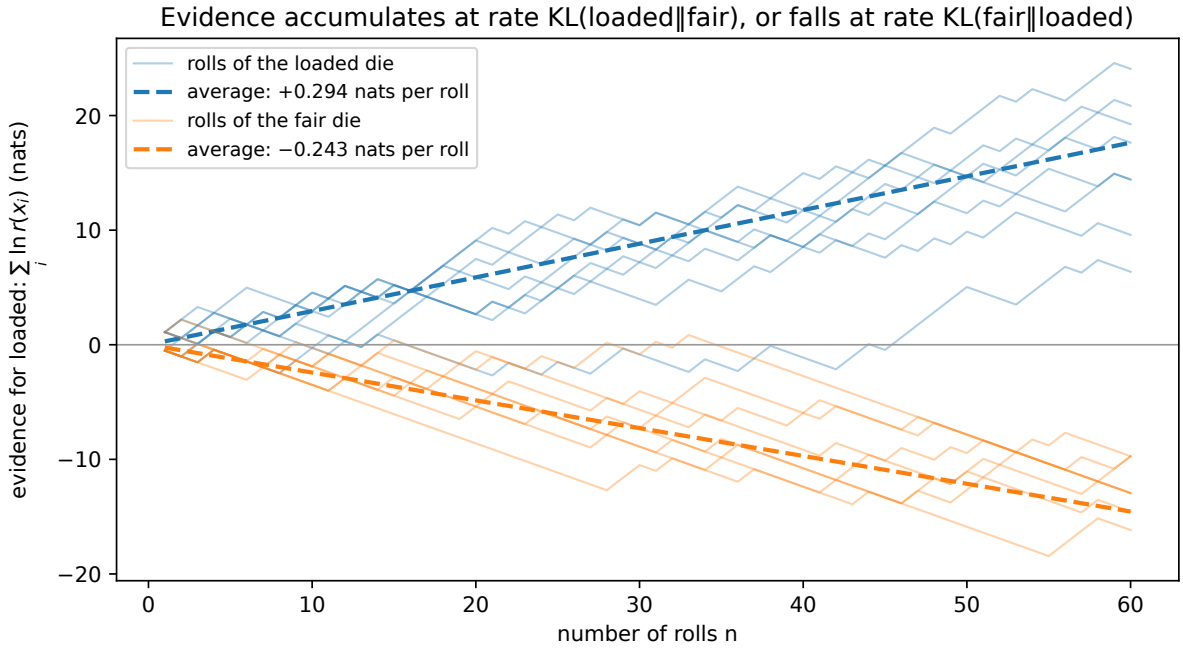


Figure 3.1.: Cumulative evidence for "loaded", $\sum_i \ln r(x_i)$, over 60 rolls. Blue: eight runs with the loaded die, drifting up at $D_{\mathrm{KL}}(P\|Q) = 0.294$ nats per roll. Orange: eight runs with the fair die, drifting down at $D_{\mathrm{KL}}(Q\|P) = 0.243$ nats per roll. Dashed lines are the averages.

The same number measures a model's fit. If the data come from p and you model them with q_θ, then $\ln \frac{p(x)}{q_\theta(x)}$ is the evidence that x gives for the truth over the model. KL is its average: how much worse the model explains the data than the truth does. Minimizing it over θ is maximum likelihood.

3.3.2. Statement

The KL divergence is the average evidence for P that one observation gives, when the observation really comes from P:

$$D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P\left[\ln \frac{p(X)}{q(X)}\right] = \sum_x p(x) \ln \frac{p(x)}{q(x)}. \tag{3.5}$$

Write $L(x) = \ln r(x)$ for the **log-likelihood ratio**, the evidence of a single outcome. Then $D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P[L]$. It has three properties, each proved in the derivation.

1. **Gibbs' inequality:** $D_{\mathrm{KL}}(P\|Q) \geq 0$, with equality only if $P = Q$. Data from P never favour Q on average.
2. **Not symmetric:** in general $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$, because the two directions average different evidence over different data.
3. **Additive over independent coordinates:** evidence from independent observations adds up, so their divergences add too,

$$D_{\mathrm{KL}}(P_1 \times P_2 \,\|\, Q_1 \times Q_2) = D_{\mathrm{KL}}(P_1\|Q_1) + D_{\mathrm{KL}}(P_2\|Q_2). \tag{3.6}$$

   Here $P_1 \times P_2$ is the distribution of a pair drawn independently from P_1 and P_2. In particular, n i.i.d. samples carry n times the KL divergence of one. That is the straight line in Figure 3.1.

3.3.3. Derivation

Gibbs' inequality. The claim is that data from P cannot, on average, favour Q. The proof turns KL into an average of a convex function and applies Jensen.

1. **Flip the ratio.** $\ln \frac{p}{q} = -\ln \frac{q}{p}$, so $D_{\mathrm{KL}}(P\|Q) = \mathbb{E}_P[-\ln \frac{q(X)}{p(X)}]$. The function $-\ln$ is strictly convex: its graph bends upward.
2. **Apply Jensen (Fact 2)**, which says that for a convex function the average of the function is at least the function of the average: $\mathbb{E}_P[-\ln \frac{q}{p}] \geq -\ln \mathbb{E}_P[\frac{q}{p}]$.
3. **Compute the inner average.** $\mathbb{E}_P[q/p] = \sum_{x:p(x)>0} p(x)\frac{q(x)}{p(x)} = \sum_{x:p(x)>0} q(x)$. This is Q's mass on the outcomes P can produce, which is at most 1. The logarithm of a number at most 1 is at most 0, so $-\ln$ of it is ≥ 0.
4. **When is it zero?** Equality needs equality in step 2, which for a strictly convex function means q/p is constant on the outcomes P can produce. It also needs equality in step 3, so Q has no mass outside those outcomes. A constant ratio between two distributions that both sum to 1 on the same outcomes must be 1, so $p = q$.

Additivity. Evidence from independent observations adds up, so its average does too.

1. **Densities multiply.** Under a product distribution the probability of a pair is the product of the probabilities, $p_1(x_1)p_2(x_2)$ under P and $q_1(x_1)q_2(x_2)$ under Q.
2. **So the log-likelihood ratio is a sum.** The ratio of products is the product of ratios, and the logarithm of a product is a sum: $L(x_1, x_2) = L_1(x_1) + L_2(x_2)$.
3. **Average.** Expectations are linear, and the P-average of $L_i(X_i)$ depends only on the distribution P_i of that coordinate, so it equals $D_{\mathrm{KL}}(P_i \| Q_i)$. Adding the two gives Equation 3.6.

Maximum likelihood minimizes KL. Split the logarithm of the ratio into the log of the truth minus the log of the model. Only the second part depends on the model's parameter θ:

$$D_{\mathrm{KL}}(P\|Q_\theta) = \underbrace{\mathbb{E}_P[\ln p(X)]}_{\text{does not depend on } \theta} - \mathbb{E}_P[\ln q_\theta(X)]. \tag{3.7}$$

So minimizing KL over θ is the same as maximizing the expected log-likelihood $\mathbb{E}_P[\ln q_\theta(X)]$. You cannot compute that expectation, because you do not know p, but its sample version $\frac{1}{n}\sum_i \ln q_\theta(x_i)$ is the average log-likelihood that maximum likelihood maximizes.

3.3.4. Interpretation

Signal against noise. After n rolls of the loaded die, the total evidence is a sum of n independent per-roll terms, so its mean is n times the per-roll mean and its variance is n times the per-roll variance. That gives a mean of $0.2939\,n$ (the drift) and a standard deviation of $0.8047\sqrt{n}$ (the spread of $\ln r$ is 0.8047 per roll, from the script). The drift grows like n and the noise like $\sqrt{n}$, so the drift overtakes the noise once $0.2939\,n > 0.8047\sqrt{n}$, that is once $n > (0.8047/0.2939)^2 = 7.50$ rolls. The same argument with any pair says that the number of observations you need scales like 1/KL. The total variation and Pinsker sections make this precise.

Which direction you minimize matters. KL is an average under its *first* argument, so the two directions punish different mistakes. There are two options when you fit a model q_θ to a target p:

- $D_{\mathrm{KL}}(P\|Q_\theta)$ **(forward, maximum likelihood)** averages over the data. Where $p > 0$ but $q_\theta \approx 0$, the term $\ln(p/q_\theta)$ explodes. The model is forced to cover every region the data visits. It is **mass-covering**.
- $D_{\mathrm{KL}}(Q_\theta\|P)$ **(reverse)** averages over the model. Regions the model ignores cost nothing, but putting model mass where $p \approx 0$ is heavily punished. It is **mode-seeking**. Variational inference minimizes this direction, because it only needs samples and densities from the model q_θ, and p only up to a constant.

Figure 3.2 shows both fits of a single Gaussian $\mathcal{N}(\mu, \sigma^2)$ to data from two bumps, $p = \frac{1}{2}\mathcal{N}(-2, 0.5^2) + \frac{1}{2}\mathcal{N}(2, 0.5^2)$.

The forward fit has a closed form. For a Gaussian q, the negative log-density is $\ln\sigma + \frac{1}{2}\ln(2\pi) + \frac{(x-\mu)^2}{2\sigma^2}$, so its average under p is $-\mathbb{E}_p[\ln q(X)] = \ln\sigma + \frac{1}{2}\ln(2\pi) + \frac{\mathbb{E}_p(X-\mu)^2}{2\sigma^2}$. Setting its derivative in μ to zero makes μ the mean of p, $\mathbb{E}_p X = 0$, since the bumps sit symmetrically at ± 2. Setting its derivative in σ to zero makes σ^2 the variance of p. That variance has two parts: the bump centres are 2 away from the mean, which contributes 2^2, and each bump has its own spread 0.5^2. So $\sigma^2 = \mathrm{Var}_p\, X = 2^2 + 0.5^2$, and $\sigma = 2.06$. This is **moment matching**: the forward fit copies

the mean and variance of the data. The result is a wide bump that puts most of its mass in the valley between the modes, where there is no data.

The reverse fit has no closed form, and a grid search finds $\mu = 2.00$, $\sigma = 0.50$: the fit sits exactly on one bump and ignores the other. Its reverse KL is $0.693 = \ln 2$. The reason is that on that bump, p is the bump's own density times its weight $\frac{1}{2}$, while the model is that density with weight 1. So $q/p \approx 2$ everywhere the model puts mass, and the average of $\ln(q/p)$ under q is about $\ln 2$.

Each fit is bad in the other's direction. The forward fit has forward KL 0.724 but reverse KL 2.097. The reverse fit has reverse KL 0.693 but forward KL 15.307, because it gives almost no probability to the left bump, where half the data lie.

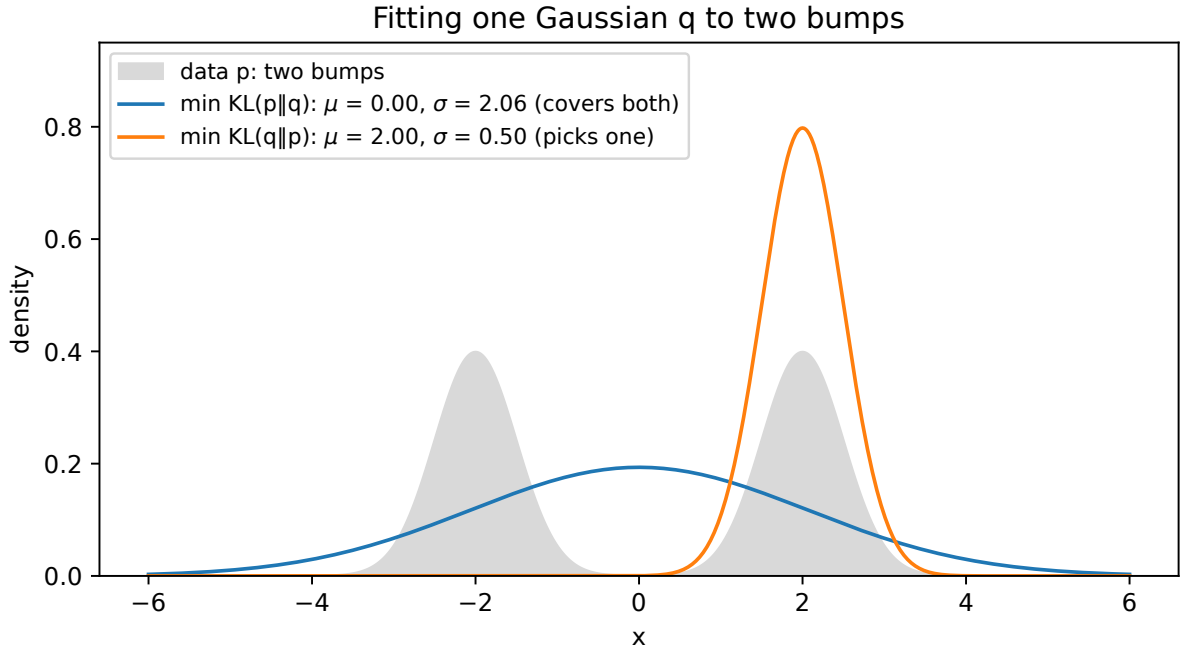


Figure 3.2.: Fitting one Gaussian to a two-bump density p. Minimizing $D_{\mathrm{KL}}(p\|q)$ matches the mean and variance and covers both bumps; minimizing $D_{\mathrm{KL}}(q\|p)$ locks onto one bump.

3.3.5. Example: the loaded die

Use Table 3.1. Under P, each of the five non-sixes has probability 0.1 and evidence $\ln 0.6$, and the six has probability 0.5 and evidence $\ln 3$. The five non-sixes together have probability $5 \times 0.1 = 0.5$, the same as the six, so both terms carry weight 0.5 and the logarithms combine:

$$D_{\mathrm{KL}}(P\|Q) = 5 \times 0.1 \ln 0.6 + 0.5 \ln 3 = 0.5 \ln(0.6 \times 3) = 0.5 \ln 1.8 = 0.2939. \tag{3.8}$$

In the other direction the average is under the fair die, which gives the non-sixes total weight $\frac{5}{6}$ and the six weight $\frac{1}{6}$, and the evidence is now for "fair", $\ln(q/p)$:

$$D_{\mathrm{KL}}(Q\|P) = \tfrac{5}{6} \ln \tfrac{1}{0.6} + \tfrac{1}{6} \ln \tfrac{1}{3} = 0.4257 - 0.1831 = 0.2426. \tag{3.9}$$

The two directions differ. Rolls of the loaded die produce sixes, where the evidence for "loaded" is strong ($\ln 3 = 1.10$ nats). Rolls of the fair die produce mostly non-sixes, where the evidence for "fair" is weaker ($\ln(1/0.6) = 0.51$ nats). So a loaded die gives itself away faster than a fair one proves its innocence. By additivity, 20 rolls of the loaded die give an expected $20 \times 0.2939 = 5.88$ nats of evidence for "loaded". Turning nats back into a likelihood ratio means exponentiating, which gives $e^{20 \times 0.5 \ln 1.8} = 1.8^{10} = 357$.

3.3.6. Check yourself

- If $q(x) = 0$ at a point where $p(x) > 0$, what is $D_{\mathrm{KL}}(P\|Q)$, and what does that mean for maximum likelihood? *(It is infinite. A model that gives zero probability to an observed outcome has log-likelihood $-\infty$, which is why practical models keep every probability positive.)*
- Why is the forward-KL fit to the two bumps centred at 0, where p is almost zero? *(The forward KL only rewards matching the mean and variance of p. Having mass where $p \approx 0$ costs nothing in that direction.)*

3.4. Rényi divergence

3.4.1. Context

KL is the average of $\ln r$ under P. Exponentiating an average of logarithms gives a **geometric mean**, so $e^{D_{\mathrm{KL}}}$ is the geometric mean of the likelihood ratio. Nothing forces you to use the geometric mean. Other means of the same numbers weight large values more, or less. All the usual ones belong to one family, the **power mean** with exponent s, $M_s = \left(\mathbb{E}_P[r^s]\right)^{1/s}$: raise each value to the power s, average, and undo the power. The harmonic mean is $s = -1$, the arithmetic mean is $s = 1$, the geometric mean is the limit $s \to 0$, and $s \to \infty$ picks out the largest value. On the die, under P the ratio is 0.6 or 3, each with probability $\frac{1}{2}$ (Table 3.1). Its means are in Table 3.4.

Table 3.4.: Different means of the die's likelihood ratio under P. Their logarithms are the Rényi divergences of orders 0, 1, 2 and ∞.

Mean of r under P	Value	Its logarithm	Divergence
Harmonic, $1/\mathbb{E}_P[1/r]$	1.0000	0	D_0
Geometric, $e^{\mathbb{E}_P[\ln r]}$	1.3416	0.2939	$D_1 = D_{\mathrm{KL}}$
Arithmetic, $\mathbb{E}_P[r]$	1.8000	0.5878	D_2
Largest value, $\max r$	3.0000	1.0986	D_∞

The **Rényi divergence of order** α is the logarithm of one of these means, the power mean with exponent $s = \alpha - 1$. The order is a dial. Near $\alpha = 1$ you get the typical evidence (KL). As α grows, the mean is pulled toward the largest ratio, and you get the worst case. The family keeps KL's best property, additivity over independent samples. It also controls how often the evidence can be large, which KL alone cannot tell you.

3.4.2. Statement

Written out, the log of the power mean with exponent $\alpha - 1$ is the log of $\mathbb{E}_P[r^{\alpha-1}]$ divided by $\alpha - 1$. Because averaging under P is averaging under Q with an extra factor r (Fact 1), the same number can be written as an average under Q or as a plain sum. For $\alpha \in (0,1) \cup (1,\infty)$,

$$D_\alpha(P\|Q) = \frac{1}{\alpha-1} \ln \sum_x p(x)^\alpha q(x)^{1-\alpha} = \frac{1}{\alpha-1} \ln \mathbb{E}_Q\big[r^\alpha\big] = \frac{1}{\alpha-1} \ln \mathbb{E}_P\big[r^{\alpha-1}\big]. \tag{3.10}$$

Two equivalent forms do most of the work. The first is the power-mean form of the context. With $M_s = \left(\mathbb{E}_P[r^s]\right)^{1/s}$ and, for $s = 0$, the geometric mean $M_0 = e^{\mathbb{E}_P[\ln r]}$,

$$D_\alpha(P\|Q) = \ln M_{\alpha-1}. \tag{3.11}$$

The second connects Rényi divergence to the Chernoff method of Concentration inequalities. The **moment generating function** of a random variable Y is $\lambda \mapsto \mathbb{E}[e^{\lambda Y}]$, and its logarithm is the quantity the Chernoff method optimizes to bound tails. Apply it to the evidence $L = \ln r$ under P, and call the result $\psi(\lambda) = \ln \mathbb{E}_P[e^{\lambda L}]$. Since $e^{\lambda L} = r^\lambda$, $\psi(\alpha - 1)$ is exactly the logarithm inside the definition, so

$$D_\alpha(P\|Q) = \frac{\psi(\alpha-1)}{\alpha-1}. \tag{3.12}$$

The family has five properties, each derived below.

1. **Limit $\alpha \to 1$:** $D_\alpha \to D_{\mathrm{KL}}$, because the power mean tends to the geometric mean. So we set $D_1 = D_{\mathrm{KL}}$.
2. **Limit $\alpha \to \infty$:** a power mean with a huge exponent is dominated by the largest value, so $D_\alpha \to D_\infty(P\|Q) = \ln \max_x r(x)$, the **max-divergence**. (For densities, the maximum is an essential supremum: the largest value that r exceeds with positive probability.) The same number is the largest ratio of probabilities that P and Q give to any event:

$$D_\infty(P\|Q) = \ln \max_{A:\, Q(A)>0} \frac{P(A)}{Q(A)}. \tag{3.13}$$

3. **Monotone:** D_α is non-decreasing in α, because power means grow with their exponent.
4. **Additive over independent coordinates:** $D_\alpha(P_1 \times P_2\|Q_1 \times Q_2) = D_\alpha(P_1\|Q_1) + D_\alpha(P_2\|Q_2)$ for every α, as for KL.
5. **Skew symmetry:** for $0 < \alpha < 1$, $D_\alpha(P\|Q) = \frac{\alpha}{1-\alpha} D_{1-\alpha}(Q\|P)$. In particular $D_{1/2}$ is symmetric, since then $\frac{\alpha}{1-\alpha} = 1$.

Table 3.5 lists the orders that have names of their own.

Table 3.5.: Rényi divergences of special orders.

Order	$D_\alpha(P\|Q)$	What it measures
$\alpha \to 0$	$-\ln Q(p > 0)$	Only whether Q puts mass outside the support of P
$\alpha = \frac{1}{2}$	$-2 \ln \sum_x \sqrt{p(x)q(x)}$	Symmetric; the sum is the Bhattacharyya coefficient, the overlap of $\sqrt{p}$ and $\sqrt{q}$

Order	$D_\alpha(P\|Q)$	What it measures
$\alpha = 1$	$\mathbb{E}_P[\ln r]$	KL: the average evidence
$\alpha = 2$	$\ln \mathbb{E}_Q[r^2] = \ln\left(1 + \chi^2(P\|Q)\right)$	The second moment of r, with $\chi^2 = \mathbb{E}_Q[(r-1)^2]$
$\alpha \to \infty$	$\ln \max r$	The worst case

3.4.3. Derivation

The forms agree. Fact 1 says an average under Q of r times something equals the average under P of that something. With "something" $= r^{\alpha-1}$, $\mathbb{E}_Q[r^\alpha] = \mathbb{E}_Q[r \cdot r^{\alpha-1}] = \mathbb{E}_P[r^{\alpha-1}]$, and writing either average as a sum gives $\sum_x p^\alpha q^{1-\alpha}$. Dividing $\ln \mathbb{E}_P[r^{\alpha-1}]$ by $\alpha - 1$ is the same as taking the logarithm of $\left(\mathbb{E}_P[r^{\alpha-1}]\right)^{1/(\alpha-1)} = M_{\alpha-1}$, which gives Equation 3.11. And $r^{\alpha-1} = e^{(\alpha-1)L}$, so $\ln \mathbb{E}_P[r^{\alpha-1}] = \psi(\alpha-1)$, which gives Equation 3.12.

Monotonicity is the power-mean inequality. A larger exponent leans harder on the large values, so power means grow with their exponent: $M_{s_1} \le M_{s_2}$ for $s_1 < s_2$. Each case is one use of Jensen (Fact 2): a convex function of an average is at most the average of the function, and for a concave function (one that bends downward) the inequality is reversed.

1. *Both exponents positive.* The function $\varphi(y) = y^{s_2/s_1}$ is convex for $y \ge 0$, because its exponent s_2/s_1 is above 1. Jensen applied to $Y = r^{s_1}$ gives $\left(\mathbb{E}_P[r^{s_1}]\right)^{s_2/s_1} \le \mathbb{E}_P\left[(r^{s_1})^{s_2/s_1}\right] = \mathbb{E}_P[r^{s_2}]$. Raising both sides to the positive power $1/s_2$ keeps the order and gives $M_{s_1} \le M_{s_2}$.
2. *Against the geometric mean.* ln is concave, so Jensen runs the other way: for $s > 0$, $\ln \mathbb{E}_P[r^s] \ge \mathbb{E}_P[\ln r^s] = s\,\mathbb{E}_P[\ln r]$. Dividing by $s > 0$ gives $\ln M_s \ge \ln M_0$. For $s < 0$ the same inequality holds before dividing, and dividing by a negative s flips it: $\ln M_s \le \ln M_0$.
3. *Both exponents negative.* Now $s_2/s_1 \in (0,1)$, so φ is concave and step 1's inequality reverses. Raising to the negative power $1/s_2$ reverses it again, so the conclusion $M_{s_1} \le M_{s_2}$ is the same.

With $s = \alpha - 1$, this says D_α is non-decreasing in α. On the die, the rows of Table 3.4 increase from top to bottom.

ψ is convex, with $\psi(0) = 0$ and $\psi'(0) = D_{\mathrm{KL}}$. This is what ties Rényi divergence to the Chernoff method, and what gives the limit $\alpha \to 1$ below.

1. **Value at 0.** $\psi(0) = \ln \mathbb{E}_P[e^0] = \ln 1 = 0$.
2. **First derivative.** Differentiate the logarithm of the average, passing the derivative inside the average: $\psi'(\lambda) = \mathbb{E}_P[Le^{\lambda L}]/\mathbb{E}_P[e^{\lambda L}]$. This is an average of L in which each outcome x gets weight proportional to $p(x)e^{\lambda L(x)} = p(x)r(x)^\lambda$. Those weights define the **tilted** distribution: P reweighted toward outcomes with large r when $\lambda > 0$. At $\lambda = 0$ there is no tilt, and $\psi'(0) = \mathbb{E}_P[L] = D_{\mathrm{KL}}(P\|Q)$.
3. **Second derivative.** Differentiating the quotient once more gives $\psi''(\lambda) = \frac{\mathbb{E}_P[L^2 e^{\lambda L}]}{\mathbb{E}_P[e^{\lambda L}]} - \left(\frac{\mathbb{E}_P[Le^{\lambda L}]}{\mathbb{E}_P[e^{\lambda L}]}\right)^2$, the mean square of L minus its squared mean under the tilted distribution. That is the variance of L under the tilted distribution. A variance is never negative, so ψ is convex. In terms of the divergence, $(\alpha-1)D_\alpha = \psi(\alpha-1)$ is convex in α.

Limit $\alpha \to 1$. Since $\psi(0) = 0$, Equation 3.12 can be written $D_\alpha = \frac{\psi(\alpha-1)-\psi(0)}{(\alpha-1)-0}$, the slope of ψ between 0 and $\alpha - 1$. As $\alpha \to 1$ that slope tends to the derivative $\psi'(0)$, which is D_{KL} by step 2. (This needs ψ to be finite near 0. van Erven & Harremoës prove the limit in general.)

Limit $\alpha \to \infty$. Let $M = \max_x r(x)$, the largest likelihood ratio on the support of P. A high power of r is dominated by its largest value, so squeeze D_α between two bounds that both tend to $\ln M$.

1. *Upper bound.* Every $r(x) \le M$, so $\mathbb{E}_P[r^{\alpha-1}] \le M^{\alpha-1}$, and taking logs and dividing by $\alpha - 1 > 0$ gives $D_\alpha \le \ln M$ for every $\alpha > 1$.
2. *Lower bound.* Let x^* be a point where $r(x^*) = M$. Dropping every other term of the average can only make it smaller: $\mathbb{E}_P[r^{\alpha-1}] \ge p(x^*)M^{\alpha-1}$, so $D_\alpha \ge \ln M + \frac{\ln p(x^*)}{\alpha-1}$.
3. *Squeeze.* The last term is a fixed number divided by $\alpha - 1$, so it tends to 0 as $\alpha \to \infty$, and $D_\alpha \to \ln M$. (For densities, replace x^* by a set of positive probability on which $r > M - \eta$, and let $\eta \to 0$.)

Limit $\alpha \to 0$. As $\alpha \to 0$, $p^\alpha \to 1$ wherever $p > 0$, and the terms with $p = 0$ are 0 for every $\alpha > 0$. So $\sum_x p^\alpha q^{1-\alpha} \to \sum_{x:\, p(x)>0} q(x) = Q(p > 0)$, the mass Q puts where P can land. Divide its logarithm by $\alpha - 1 \to -1$ to get $-\ln Q(p > 0)$.

Max-divergence over events. The largest ratio for a single outcome is also the largest ratio for any event, because an event's probability is a sum of outcomes' probabilities, each at most M times its Q-probability. Formally, Fact 1 with the indicator of A gives $P(A) = \mathbb{E}_Q[r\mathbf{1}_A] \le M\,\mathbb{E}_Q[\mathbf{1}_A] = M\,Q(A)$, so $P(A)/Q(A) \le M$. The event $A = \{x^*\}$ attains it.

Additivity. As for KL, the log-likelihood ratio of a pair is a sum, $L_1(X_1) + L_2(X_2)$, and under $P_1 \times P_2$ the two terms are independent. For independent variables the moment generating function of a sum is the product of their moment generating functions: $e^{\lambda(L_1+L_2)} = e^{\lambda L_1}e^{\lambda L_2}$, and the average of a product of independent factors is the product of their averages (Fact 4 of Concentration inequalities). Taking logarithms, $\psi = \psi_1 + \psi_2$. Divide by $\alpha - 1$.

Skew symmetry. The defining sum is symmetric under swapping the roles of p and q together with α and $1 - \alpha$. Indeed $\sum_x p^\alpha q^{1-\alpha} = \sum_x q^{1-\alpha}p^{1-(1-\alpha)}$, and the right side is the sum that defines $D_{1-\alpha}(Q\|P)$. Its logarithm is $(\alpha-1)D_\alpha(P\|Q)$ on one hand and $((1-\alpha)-1)D_{1-\alpha}(Q\|P) = -\alpha D_{1-\alpha}(Q\|P)$ on the other. Divide by $\alpha - 1$.

The same facts as a picture. Equation 3.12 says D_α is the slope of the chord of ψ from $(0, 0)$ to $(\lambda, \psi(\lambda))$, with $\lambda = \alpha - 1$. For a convex function through the origin, chords get steeper as λ grows (monotonicity), and they approach the tangent at 0 as $\lambda \to 0$ (the KL limit). Figure 3.3 draws this for the die.

3.4.4. Interpretation

Where each order looks. D_α is built from the sum $\sum_x p(x)^\alpha q(x)^{1-\alpha}$. Divide each term by the total and you get a distribution. It is the tilted distribution of the derivation with $\lambda = \alpha - 1$, since $p^\alpha q^{1-\alpha} = p\,r^{\alpha-1}$, and the slope $\psi'(\alpha - 1)$ is the average evidence L under it. So the tilted distribution shows which outcomes the order α pays attention to. At $\alpha = 1$ it is P itself. As α grows, it moves toward the outcomes with the largest r.

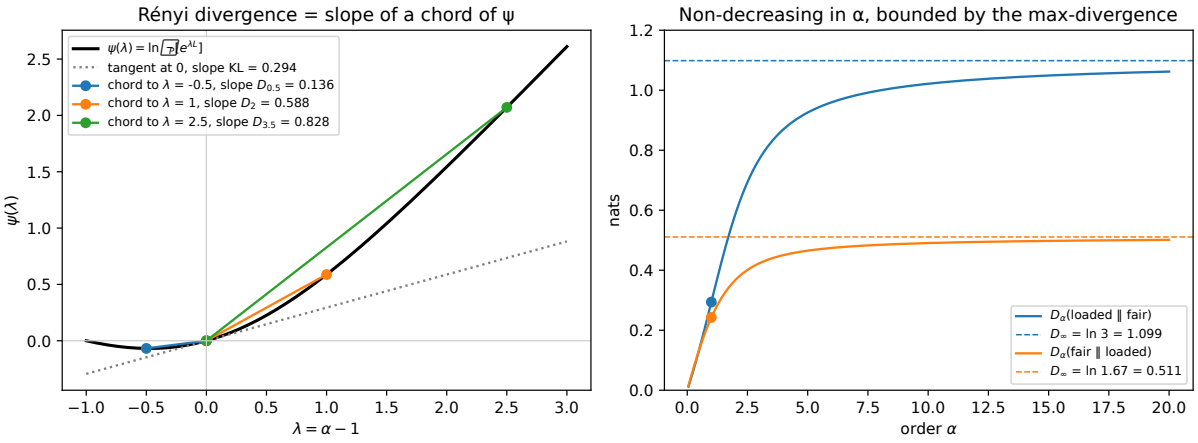


Figure 3.3.: Left: the log-MGF ψ of the log-likelihood ratio for the loaded die against the fair die. The Rényi divergence D_α is the slope of the chord from the origin to $\lambda = \alpha - 1$, and the tangent at 0 has slope KL. Right: D_α as a function of α, in both directions, rising to the max-divergence $\ln \max r$.

Figure 3.4 draws it for two pairs with the same shift and variance, a Gaussian pair and a Laplace pair, and the difference between them is the difference between a log-ratio that keeps growing and one that stops. For the Gaussian pair, the log-ratio is a straight line in x, so the tilt keeps finding larger ratios further out. The tilted distribution is again normal, centred at $(1-\alpha)s$ for a shift s (derived in the next section). It slides into the tail at a constant speed and never stops, so D_α grows without bound and $D_\infty = \infty$. For the Laplace pair with shift t (measured in units of the Laplace scale), the log-ratio already equals its maximum t on all of $x \le 0$, and no outcome has a larger one. The tilted distribution piles up on that region and stops moving, so D_α levels off at $D_\infty = t$.

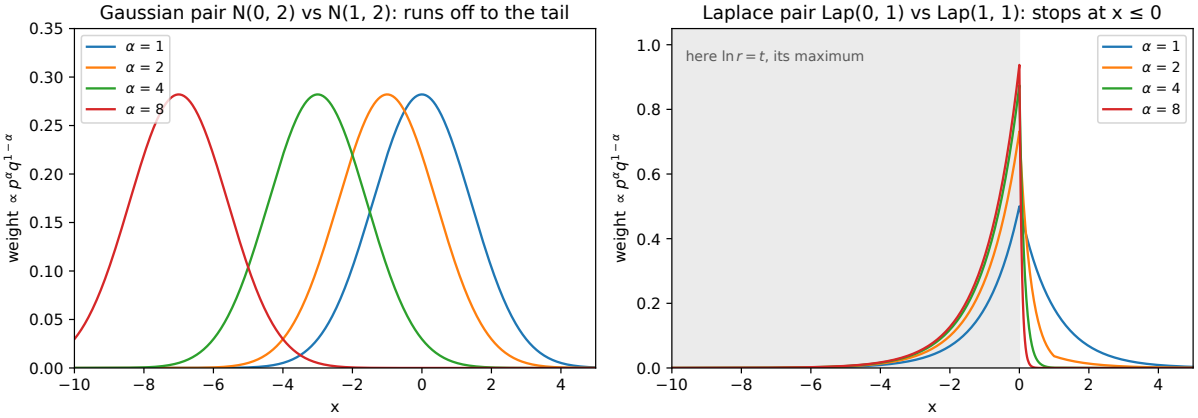


Figure 3.4.: Where the Rényi divergence of order α looks: the normalized weights $p^\alpha q^{1-\alpha}$ for $\alpha = 1, 2, 4, 8$. Left: $\mathcal{N}(0, 2)$ against $\mathcal{N}(1, 2)$; the weight is $\mathcal{N}(1-\alpha, 2)$ and runs off into the left tail. Right: $\mathrm{Lap}(0, 1)$ against $\mathrm{Lap}(1, 1)$; the weight collects on $x \le 0$ (shaded), where the log-likelihood ratio is already at its maximum, and stops.

Rényi divergence controls the tail of the evidence. How often does a sample from Q look

like strong evidence for P, at least c nats? The Chernoff method answers this kind of question. Markov's inequality says that a non-negative variable exceeds a level k with probability at most its mean divided by k. Applying it to $e^{\alpha L}$, which is non-negative and exceeds $e^{\alpha c}$ exactly when $L \geq c$, gives a bound in terms of the average of $e^{\alpha L} = r^{\alpha}$ under Q. That average is exactly what the Rényi divergence of order α measures: by the definition, $\mathbb{E}_Q[r^{\alpha}] = e^{(\alpha-1)D_{\alpha}(P\|Q)}$. So for any $\alpha > 0$,

$$Q[L \geq c] \leq e^{-\alpha c}\,\mathbb{E}_Q\big[e^{\alpha L}\big] = e^{-\alpha c}\,\mathbb{E}_Q[r^{\alpha}] = \exp\big(-\alpha c + (\alpha-1)D_{\alpha}(P\|Q)\big). \tag{3.14}$$

At $\alpha = 1$ the divergence term drops out and this is $Q[r \geq e^{c}] \leq e^{-c}$: Markov's inequality applied to r itself, whose Q-mean is 1 (Fact 1). It holds for *any* pair of distributions. Larger α uses more about the pair and can do much better. KL alone cannot give a bound like this, because an average says nothing about how often a variable is far from it.

3.4.5. Example: the die across orders

The script computes D_{α} for the die in both directions:

Table 3.6.: Rényi divergences between the loaded and the fair die.

α	0.5	1 (KL)	2	5	10	∞
$D_{\alpha}(P\|Q)$, loaded ‖ fair	0.1362	0.2939	0.5878	0.9257	1.0216	1.0986
$D_{\alpha}(Q\|P)$, fair ‖ loaded	0.1362	0.2426	0.3677	0.4653	0.4906	0.5108

Check two entries by hand. At order 2 the definition is $\ln \mathbb{E}_Q[r^2]$, and under the fair die r is 0.6 with probability $\frac{5}{6}$ and 3 with probability $\frac{1}{6}$, so $D_2(P\|Q) = \ln\big(\frac{5}{6} \times 0.36 + \frac{1}{6} \times 9\big) = \ln 1.8 = 0.5878$, the log of the arithmetic mean in Table 3.4. The max-divergences are the logs of the largest ratios in each direction, $\ln 3 = 1.0986$ and $\ln(1/0.6) = \ln \frac{5}{3} = 0.5108$. Every row increases, as monotonicity requires. The $\alpha = 0.5$ entries agree, as skew symmetry requires. The order-10 value is already within 7% of the worst case ($1.0216/1.0986 = 0.93$), because the die has only two values of r and the larger one has probability $\frac{1}{2}$ under P, so the power mean reaches the maximum quickly.

3.4.6. Check yourself

- Using Equation 3.12, why does a quadratic ψ give a Rényi divergence that is linear in α? *(If $\psi(\lambda) = a\lambda + b\lambda^2$, then $\psi(\alpha-1)/(\alpha-1) = a + b(\alpha-1)$. The Gaussian pair below is exactly this case.)*
- Can $D_2(P\|Q)$ be finite while $D_{\infty}(P\|Q)$ is infinite? *(Yes. D_{∞} is infinite whenever r is unbounded, even if large ratios are rare enough that $\mathbb{E}_Q[r^2]$ is finite. The Gaussian pair below is an example.)*

3.5. Two Gaussians with equal variance, by hand

3.5.1. Context

The most common pair in practice is a distribution and a shifted copy of it: the same noise around two different centres, the same feature distribution before and after a drift, two classes with the same spread. For two normals with equal variance, every divergence has a closed form. The computation is the standard exercise for this chapter, so try it yourself before reading the derivation.

3.5.2. Statement

Let $P = \mathcal{N}(0, \sigma^2)$ and $Q = \mathcal{N}(s, \sigma^2)$: the same bell curve with standard deviation σ, centred at 0 and at s. What matters is how far apart the centres are compared with the width of the curves, so write $\mu = s/\sigma$ for the shift in standard deviations. KL is half the square of that shift, each Rényi divergence is α times the KL, and the max-divergence is infinite because the likelihood ratio is unbounded:

$$D_{\mathrm{KL}}(P\|Q) = \frac{s^2}{2\sigma^2} = \frac{\mu^2}{2}, \qquad D_\alpha(P\|Q) = \frac{\alpha s^2}{2\sigma^2} = \frac{\alpha\mu^2}{2}, \qquad D_\infty(P\|Q) = \infty. \tag{3.15}$$

Shifting both distributions by the same amount changes nothing, so the same holds for $\mathcal{N}(m_0, \sigma^2)$ against $\mathcal{N}(m_1, \sigma^2)$, with means m_0 and m_1 and $s = m_1 - m_0$.

3.5.3. Derivation

The log-likelihood ratio (also in Probability distributions). Everything follows from the evidence $L(x)$, so compute it first. The normal density with mean m is $\frac{1}{\sigma\sqrt{2\pi}} e^{-(x-m)^2/(2\sigma^2)}$. In the ratio p/q the constants in front cancel, and the log of the ratio of exponentials is the difference of the exponents:

$$L(x) = \ln \frac{p(x)}{q(x)} = \frac{(x-s)^2 - x^2}{2\sigma^2} = \frac{s^2 - 2sx}{2\sigma^2}. \tag{3.16}$$

The last step expands $(x-s)^2 = x^2 - 2sx + s^2$, and the x^2 cancels. So L is a straight line in x. For $s > 0$ it is positive left of the midpoint $s/2$, where x is closer to P's centre than to Q's.

KL. KL is the average of this line when x comes from P.

1. **Set up.** $D_{\mathrm{KL}} = \mathbb{E}_P[L(X)]$ with $X \sim \mathcal{N}(0, \sigma^2)$.
2. **Average a line.** L is linear in x, so its average is L at the average of X. Under P that average is $\mathbb{E}_P X = 0$, and $L(0) = \frac{s^2}{2\sigma^2}$.

Rényi, route 1: complete the square. The plan is to write $p^\alpha q^{1-\alpha}$ as a constant times a normal density, because a normal density integrates to 1 and only the constant survives.

1. **Pull out the constants.** We need $\int p^\alpha q^{1-\alpha}\, dx$. Both densities share the factor $\frac{1}{\sigma\sqrt{2\pi}}$, and $(\frac{1}{\sigma\sqrt{2\pi}})^\alpha (\frac{1}{\sigma\sqrt{2\pi}})^{1-\alpha} = \frac{1}{\sigma\sqrt{2\pi}}$. What remains is $\exp\big(-\frac{1}{2\sigma^2}[\alpha x^2 + (1-\alpha)(x-s)^2]\big)$.
2. **Expand the bracket.** $\alpha x^2 + (1-\alpha)(x^2 - 2sx + s^2) = x^2 - 2(1-\alpha)sx + (1-\alpha)s^2$.

3. **Complete the square** with $m = (1-\alpha)s$. Since $(x-m)^2 = x^2 - 2mx + m^2$, the bracket is $(x-m)^2 + (1-\alpha)s^2 - (1-\alpha)^2 s^2 = (x-m)^2 + \alpha(1-\alpha)s^2$, using $(1-\alpha) - (1-\alpha)^2 = (1-\alpha)\alpha$.
4. **Integrate.** The factor with $(x-m)^2$, together with $\frac{1}{\sigma\sqrt{2\pi}}$, is a normal density with mean m and integrates to 1. (That normal, $\mathcal{N}((1-\alpha)s, \sigma^2)$, is the tilted distribution of Figure 3.4.) Only the constant part of the bracket is left:

$$\int p^\alpha q^{1-\alpha}\, dx = \exp\left(-\frac{\alpha(1-\alpha)s^2}{2\sigma^2}\right). \tag{3.17}$$

5. **Take the logarithm and divide by** $\alpha - 1$**.** Since $\frac{-\alpha(1-\alpha)}{\alpha-1} = \alpha$, this gives $D_\alpha = \frac{\alpha s^2}{2\sigma^2}$.

Rényi, route 2: the log-likelihood ratio is normal. The plan is to use Equation 3.12, which needs the log of the moment generating function of L under P, and for a normal variable that is known in closed form.

1. **The distribution of** L**.** Under P, $L(X) = \frac{s^2}{2\sigma^2} - \frac{s}{\sigma^2}X$ is a linear function of a normal variable, so it is normal. Its mean is $\frac{s^2}{2\sigma^2}$ (because $\mathbb{E}X = 0$) and its variance is $\frac{s^2}{\sigma^4}\sigma^2 = \frac{s^2}{\sigma^2}$ (a constant factor a multiplies a variance by a^2).
2. **Its log moment generating function.** For a normal variable Y with mean m and variance v, $\ln \mathbb{E}[e^{\lambda Y}] = \lambda m + \frac{\lambda^2}{2}v$ (Probability distributions). With the mean and variance of step 1, $\psi(\lambda) = \lambda\frac{s^2}{2\sigma^2} + \frac{\lambda^2}{2}\frac{s^2}{\sigma^2}$.
3. **Divide.** By Equation 3.12, $D_\alpha = \frac{\psi(\alpha-1)}{\alpha-1} = \frac{s^2}{2\sigma^2} + (\alpha-1)\frac{s^2}{2\sigma^2} = \frac{\alpha s^2}{2\sigma^2}$.

Max-divergence. $L(x)$ is a line with non-zero slope, so it is unbounded above (as $x \to -\infty$ when $s > 0$). The largest evidence is infinite, so $D_\infty = \infty$.

Checks. At $\alpha = 1$ the Rényi formula gives $s^2/(2\sigma^2)$, the KL value. It is increasing in α, as monotonicity requires. And it is additive: for n independent coordinates with shifts s_i, the divergences add to $\alpha\sum_i s_i^2/(2\sigma^2) = \alpha\|s\|^2/(2\sigma^2)$. It depends on the shift vector only through its length. That is what it must do, because the standard normal in many dimensions looks the same in every direction: rotating the space turns any shift into any other shift of the same length, without changing the pair.

3.5.4. Interpretation

Only the signal-to-noise ratio matters. Every divergence depends on s and σ only through $\mu = s/\sigma$, so doubling the noise has the same effect as halving the shift.

The dependence is quadratic in the shift, so a small shift costs very little: $\mu = 0.1$ gives KL $= 0.1^2/2 = 0.005$. This is why a small drift is hard to see in one sample and needs many.

The average case is finite but the worst case is infinite. Every D_α with finite α is finite, but the likelihood ratio is unbounded in the tails (Probability distributions, the shifted-copy section). Large ratios exist but are rare, and Equation 3.14 makes "rare" precise, as the example shows.

3.5.5. Example: how often does a sample from Q favour P strongly?

Take $\mu = 1$. Then Equation 3.15 gives $D_{\mathrm{KL}} = \frac{1}{2} = 0.5$, $D_2 = 1$ and $D_{10} = 5$. The script also estimates KL as the average of L over a million draws from P and gets 0.4990.

Now ask how often an observation from Q gives at least $c = 2$ nats of evidence for P. That is a likelihood ratio of $e^2 \approx 7.4$ in the wrong direction. The exact answer comes from solving $L \geq c$ for x. Since $L(x) = \frac{s^2 - 2sx}{2\sigma^2}$ decreases in x, $L \geq c$ when $x \leq \frac{s}{2} - \frac{c\sigma^2}{s}$. Under Q, $x = s + \sigma Z$ with Z standard normal, so the condition is $Z \leq -\frac{\mu}{2} - \frac{c}{\mu}$. Write Φ for the standard normal cumulative distribution function, so that $\Phi(z)$ is the probability that $Z \leq z$. Then $Q[L \geq c] = \Phi(-\frac{\mu}{2} - \frac{c}{\mu}) = \Phi(-0.5 - 2) = \Phi(-2.5) = 0.0062$.

How close do the bounds of Equation 3.14 come? With $\alpha = 1$ the bound is Markov's, which holds for any pair and gives $e^{-2} = 0.1353$. The Gaussian pair lets you do better by choosing α. With $D_\alpha = \alpha\mu^2/2$, the exponent of the bound is $-\alpha c + \alpha(\alpha - 1)\mu^2/2$. It is a parabola in α, and setting its derivative $-c + (2\alpha - 1)\mu^2/2$ to zero gives the best order, $\alpha = c/\mu^2 + \frac{1}{2} = 2.5$. There the exponent is $-2.5 \times 2 + 2.5 \times 1.5/2 = -3.125$, so the bound is $e^{-3.125} = 0.0439$.

The Rényi bound is 7 times the exact value (0.0439/0.0062), against 22 times for Markov (0.1353/0.0062). The remaining gap is the usual price of the Chernoff method (Concentration inequalities): Markov's inequality is loose for any variable that is not concentrated on two values.

3.5.6. Check yourself

- What are D_{KL} and D_2 between $\mathcal{N}(0, I_d)$ and $\mathcal{N}(v, I_d)$ for $v = (0.1, \ldots, 0.1) \in \mathbb{R}^{100}$? *($\|v\|^2 = 100 \times 0.01 = 1$, so $KL = 0.5$ and $D_2 = 1$: a hundred tiny shifts add up to one unit shift.)*
- Why is $D_{\mathrm{KL}}(P\|Q) = D_{\mathrm{KL}}(Q\|P)$ for this pair, although KL is not symmetric in general? *(Reflecting $x \mapsto s - x$ swaps P and Q, and divergences do not change under a bijection of the outcome space: by the data-processing inequality, applying the bijection cannot increase a divergence, and neither can undoing it.)*

3.6. Two Laplace distributions with shifted location

3.6.1. Context

The Laplace distribution is the other standard noise model. Its density with centre m and scale b is $\frac{1}{2b}e^{-|x-m|/b}$: a sharp peak with tails that fall off exponentially, more slowly than the normal's. Probability distributions showed that its log-likelihood ratio against a shifted copy is bounded, unlike the normal's. This section computes the full Rényi family for that pair and compares it with a Gaussian pair of the same variance. The comparison turns "bounded against unbounded likelihood ratio" into a difference you can see in the divergences.

3.6.2. Statement

Let $P = \mathrm{Lap}(0, b)$ and $Q = \mathrm{Lap}(s, b)$ with $s \geq 0$, and write $t = s/b$ for the shift in units of the scale. The max-divergence is just the shift t, KL is slightly less, and every Rényi divergence has a closed form:

$$D_{\mathrm{KL}}(P\|Q) = t + e^{-t} - 1, \qquad D_\infty(P\|Q) = t, \tag{3.18}$$

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln\left(\frac{\alpha}{2\alpha - 1} e^{(\alpha-1)t} + \frac{\alpha - 1}{2\alpha - 1} e^{-\alpha t}\right) \qquad (\alpha \neq \tfrac{1}{2}, 1). \tag{3.19}$$

At $\alpha = \frac{1}{2}$ both coefficients divide by zero, and the bracket is replaced by its limit $(1 + \frac{t}{2})e^{-t/2}$.

3.6.3. Derivation

Rescaling $x \mapsto x/b$ is a bijection: it can be undone, so it loses nothing and does not change any divergence. Take $b = 1$ and $s = t$.

Max-divergence. With $b = 1$ the densities are $\frac{1}{2}e^{-|x|}$ and $\frac{1}{2}e^{-|x-t|}$, so the evidence is the difference of the exponents, $L(x) = |x - t| - |x|$. Two numbers whose distance is t cannot have absolute values that differ by more than t (the triangle inequality), so $|L(x)| \leq t$. For every $x \leq 0$, both x and $x - t$ are negative, so $|x - t| = t - x$ and $|x| = -x$, and $L(x) = t$ exactly. So $\max L = t$ (Probability distributions).

KL. KL is the average of L under P, and the work is to average $|X - t|$.

1. **Set up.** $D_{\mathrm{KL}} = \mathbb{E}_P[L] = \mathbb{E}|X - t| - \mathbb{E}|X|$ with $X \sim \mathrm{Lap}(0, 1)$. The second term is $\mathbb{E}|X| = 1$, because $|X|$ has the exponential density e^{-y} on $y \geq 0$, whose mean is 1.
2. **Split off the negative part.** For any number y, $|y| = y + 2(-y)_+$: if $y \geq 0$ both sides are y, and if $y < 0$ the right side is $y - 2y = -y$. With $y = X - t$, and $\mathbb{E}X = 0$, this gives $\mathbb{E}|X - t| = -t + 2\,\mathbb{E}(t - X)_+$.
3. **Write the positive part as an integral.** $(t - X)_+$ is the length of the interval from X up to t, which is $\int_{-\infty}^{t} \mathbf{1}[X \leq x]\, dx$. Averaging inside the integral gives $\mathbb{E}(t - X)_+ = \int_{-\infty}^{t} \Pr[X \leq x]\, dx$. The Laplace cumulative distribution function is $\Pr[X \leq x] = \frac{1}{2}e^{x}$ for $x < 0$ and $1 - \frac{1}{2}e^{-x}$ for $x \geq 0$, so this is $\int_{-\infty}^{0} \frac{1}{2}e^{x}\, dx + \int_{0}^{t} (1 - \frac{1}{2}e^{-x})dx = \frac{1}{2} + t - \frac{1}{2}(1 - e^{-t}) = t + \frac{1}{2}e^{-t}$.
4. **Combine.** $\mathbb{E}|X - t| = -t + 2(t + \frac{1}{2}e^{-t}) = t + e^{-t}$, and $D_{\mathrm{KL}} = t + e^{-t} - 1$.

Rényi (can wait for a second reading; only the result is used later). We need $I(\alpha) = \int p^\alpha q^{1-\alpha} = \frac{1}{2}\int \exp(-\alpha|x| - (1 - \alpha)|x - t|)dx$. The absolute values change sign at 0 and t, and between those points each one is a plain linear function, so split the line into the three pieces of Table 3.7. On each piece the exponent is linear in x, and the integral of an exponential of a linear function is elementary.

Table 3.7.: The three pieces of the Laplace Rényi integral.

Piece	Exponent $-\alpha\|x\| - (1-\alpha)\|x-t\|$	Integral of e^{exponent}
$x < 0$	$x - (1 - \alpha)t$	$e^{-(1-\alpha)t}$
$0 \leq x \leq t$	$(1 - 2\alpha)x - (1 - \alpha)t$	$\dfrac{e^{-\alpha t} - e^{-(1-\alpha)t}}{1 - 2\alpha}$

Piece	Exponent $-\alpha\lvert x\rvert - (1-\alpha)\lvert x-t\rvert$	Integral of e^{exponent}
$x > t$	$-x + (1-\alpha)t$	$e^{-\alpha t}$

1. **Collect the $e^{-\alpha t}$ terms.** Add the three integrals and halve. The coefficient of $e^{-\alpha t}$ is $\frac{1}{2}(1 + \frac{1}{1-2\alpha}) = \frac{1-\alpha}{1-2\alpha} = \frac{\alpha-1}{2\alpha-1}$.
2. **Collect the other terms.** The coefficient of $e^{-(1-\alpha)t} = e^{(\alpha-1)t}$ is $\frac{1}{2}(1 - \frac{1}{1-2\alpha}) = \frac{-\alpha}{1-2\alpha} = \frac{\alpha}{2\alpha-1}$.
3. **Take the logarithm.** $\frac{1}{\alpha-1} \ln I(\alpha)$ is Equation 3.19.
4. **Check against KL.** The Rényi divergence must tend to KL as $\alpha \to 1$. $I(1) = \int p = 1$, so $D_\alpha = \frac{\ln I(\alpha) - \ln I(1)}{\alpha-1}$ is a difference quotient of $\ln I$ and tends to its derivative at 1, $I'(1)/I(1)$. Differentiating the closed form of I at $\alpha = 1$ gives $I'(1) = -1 + t + e^{-t}$, so the limit is $t + e^{-t} - 1$, as above.

3.6.4. Interpretation: Laplace against Gaussian at the same variance

To compare the two noise models fairly, give them the same variance. $\text{Lap}(0, b)$ has variance $2b^2$, so compare it with the Gaussian pair $\mathcal{N}(0, 2b^2)$ against $\mathcal{N}(s, 2b^2)$. Its standardized shift is $\mu = s/(\sqrt{2}\,b) = t/\sqrt{2}$, so $D_\alpha = \alpha\mu^2/2 = \alpha t^2/4$ by Equation 3.15.

For small shifts, the Laplace pair is further apart. Expanding $e^{-t} = 1 - t + \frac{t^2}{2} - \ldots$, the Laplace KL $t + e^{-t} - 1$ is $\approx t^2/2$: twice the Gaussian's $t^2/4$. The Laplace density's sharp peak makes small shifts easier to detect, because the peak moves visibly.

For large orders the comparison reverses. The Gaussian D_α grows linearly in α forever. The Laplace D_α rises and saturates at t, because the likelihood ratio can never exceed e^t (Figure 3.5, and Figure 3.4 for where each order looks). At $t = 1$ the script solves for the order where the two curves cross: for orders above 2.98, the Gaussian pair is the *more* distinguishable one in the Rényi sense.

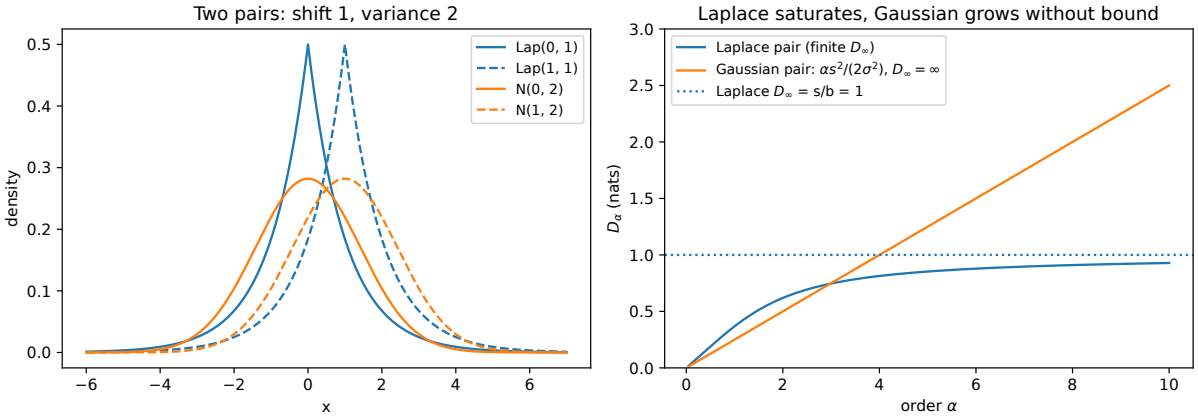


Figure 3.5.: Left: a Laplace pair and a Gaussian pair, both with shift 1 and variance 2. Right: their Rényi divergences as a function of the order α. The Gaussian's grows linearly without bound; the Laplace's levels off at its max-divergence $s/b = 1$.

3.6.5. Example: shift of one scale unit

Table 3.8 compares $t = 1$ ($b = 1$, $s = 1$) with the Gaussian pair of variance 2, for which $D_\alpha = \alpha/4$.

Table 3.8.: Rényi divergences of a Laplace pair and a Gaussian pair with the same shift and variance.

	KL	$D_{0.5}$	D_2	D_{10}	D_∞
Laplace, $t = 1$	0.3679	0.1891	0.6191	0.9287	1.0000
Gaussian, same variance	0.2500	0.1250	0.5000	2.5000	∞

By hand: at $t = 1$ the Laplace KL is $1 + e^{-1} - 1 = e^{-1} = 0.3679$, and the Monte Carlo average of L over a million draws gives 0.3684. At $\alpha = 2$ the coefficients in Equation 3.19 are $\frac{2}{3}$ and $\frac{1}{3}$ and the prefactor is 1, so $D_2 = \ln\left(\frac{2}{3}e + \frac{1}{3}e^{-2}\right) = \ln 1.8573 = 0.6191$. In the bulk (small α) the Laplace pair is further apart. In the worst case the Gaussian pair is.

3.6.6. Check yourself

- Why does D_α for the Laplace pair stay below t for every α? *(Monotonicity in α and the limit $D_\infty = t$: a non-decreasing function that tends to t never exceeds it.)*
- For which x is the Laplace log-likelihood ratio at its maximum t, and how likely is that region under P? *(For every $x \le 0$, which has probability $\frac{1}{2}$ under P. The worst case is not rare at all, unlike for the Gaussian.)*

3.7. Total variation

3.7.1. Context

KL and Rényi divergences are convenient to compute, but they do not directly answer operational questions. **Total variation** does. Draw p and q on the same axes. Where the curves overlap, an observation cannot tell you which distribution it came from. Where they don't, it can. Total variation is the part that does not overlap: the mass of P that sits above Q. It answers three questions at once. It is the largest difference in the probability that P and Q give to any event. It is the largest amount by which any bounded statistic can change when you replace Q by P. And it is how well the best possible test can tell P from Q with one observation.

3.7.2. Statement

The total variation has four equal forms, and each says the same thing from a different side: the largest gap between the probabilities of one event, the mass where p sits above q, half the total absolute difference between the two probability tables, and the average under Q of how far r exceeds 1:

$$\mathrm{TV}(P,Q) = \max_A |P(A) - Q(A)| = \sum_x \big(p(x) - q(x)\big)_+ = \frac{1}{2}\sum_x |p(x) - q(x)| = \mathbb{E}_Q\big[(r-1)_+\big]. \tag{3.20}$$

Equivalently, the overlap of the two curves is $\sum_x \min(p(x), q(x)) = 1 - \mathrm{TV}(P,Q)$. It has three uses and one structural property.

1. **Bounded statistics.** For every g with values in $[a,b]$, $|\mathbb{E}_P g - \mathbb{E}_Q g| \le (b-a)\,\mathrm{TV}(P,Q)$.
2. **Best single-shot test.** Nature picks P or Q with probability $\frac{1}{2}$ each and shows you one draw. Guessing blind is right half the time, and seeing the draw adds half the TV. The best possible guess is correct with probability
$$\Pr[\text{correct}] = \frac{1}{2} + \frac{1}{2}\mathrm{TV}(P,Q). \tag{3.21}$$
3. **TV is a metric:** symmetric, zero only if $P = Q$, and it satisfies the triangle inequality (it is half an ℓ_1 distance, the sum of absolute differences). It is always between 0 and 1.

3.7.3. Derivation

The forms of Equation 3.20. Each form follows from the previous one.

1. **The best event.** Fact 3 with $\gamma = 1$ says the event that maximizes $P(A) - Q(A)$ collects every outcome where $p > q$, so $\max_A[P(A) - Q(A)] = \sum_x (p-q)_+$.
2. **Absolute values come for free.** The differences $p - q$ sum to $1 - 1 = 0$, so the positive and negative parts have equal mass: $\sum(p-q)_+ = \sum(q-p)_+$. So $\max_A[Q(A) - P(A)]$ has the same value, which lets you put absolute values on $P(A) - Q(A)$.
3. **Half the absolute difference.** $|y| = y_+ + (-y)_+$, so $\sum|p-q| = \sum(p-q)_+ + \sum(q-p)_+ = 2\sum(p-q)_+$.
4. **An average under Q.** $p - q = q(r-1)$, so $\sum_x (p-q)_+ = \sum_x q\,(r-1)_+ = \mathbb{E}_Q[(r-1)_+]$.
5. **Overlap.** At each x, $\min(p,q)$ is p minus whatever of p sticks out above q: $\min(p,q) = p - (p-q)_+$. Sum over x: $\sum\min(p,q) = 1 - \mathrm{TV}$.

Bounded statistics. The idea is that a statistic with values in an interval is a "soft event", and Fact 3 bounds soft events as well as sharp ones. Shift g by a constant, which does not change $\mathbb{E}_P g - \mathbb{E}_Q g$, so that it takes values in $[0, b-a]$. Then $\mathbb{E}_P g - \mathbb{E}_Q g = \sum_x g(x)(p(x) - q(x))$. The function $\phi = g/(b-a)$ has values in $[0,1]$, and the randomized version of Fact 3 says $\sum_x \phi(x)(p(x) - q(x)) \le \sum(p-q)_+$. Multiplying by $b-a$ gives the bound $(b-a)\sum(p-q)_+$, and the same argument with g replaced by $-g$ bounds the other sign.

Best single-shot test.

1. **Score a test.** A test says "P" when $x \in A$ and "Q" otherwise. It is right when the draw came from P and lands in A, or came from Q and lands outside A, so it is correct with probability $\frac{1}{2}P(A) + \frac{1}{2}(1 - Q(A)) = \frac{1}{2} + \frac{1}{2}(P(A) - Q(A))$.

2. **Choose the best event.** Maximize over A with Fact 3: the best test is $A^* = \{p > q\}$, "say P when the observation is more likely under P", and it achieves Equation 3.21.
3. **Randomizing does not help.** A randomized test says "P" with probability $\phi(x) \in [0, 1]$, and by the second part of Fact 3 it does no better.

Concretely, on the die. Table 3.9 evaluates Equation 3.20 outcome by outcome.

Table 3.9.: Total variation between the loaded and the fair die, outcome by outcome.

Roll	1–5 (each)	6	Sum
$p - q$	$0.1 - \frac{1}{6} = -\frac{1}{15}$	$0.5 - \frac{1}{6} = \frac{1}{3}$	0
$(p - q)_+$	0	$\frac{1}{3}$	$\frac{1}{3}$
$\lvert p - q \rvert$	$\frac{1}{15}$	$\frac{1}{3}$	$\frac{2}{3}$

So TV $= \frac{1}{3}$, and half the absolute sum, $\frac{1}{2} \times \frac{2}{3}$, agrees. The best test says "loaded" exactly when it sees a six, and it is right with probability $\frac{1}{2} + \frac{1}{2} \times \frac{1}{3} = \frac{1}{2} + \frac{1}{6} = \frac{2}{3}$.

3.7.4. Interpretation

TV is the bottom line for one observation. No classifier, however clever, can beat $\frac{1}{2} + \frac{1}{2}$TV accuracy from a single draw with equal priors, because the best test of the derivation already reaches it.

It also bounds how much any fixed procedure can change when the data distribution changes. If the test distribution moves from Q to P with TV ≤ 0.01, then the accuracy of *any* fixed classifier moves by at most 0.01. Its accuracy is the average of a statistic with values in $[0, 1]$ (1 for a correct prediction, 0 for a wrong one), so the bounded-statistics property applies with $b - a = 1$.

TV is not additive. For n i.i.d. samples, $\mathrm{TV}(P^n, Q^n)$ is not $n\,\mathrm{TV}(P, Q)$. It is capped at 1 and has no simple formula. That is why you usually compute KL or a Rényi divergence for n samples, where additivity makes it trivial, and convert to TV with Pinsker's inequality (Section 3.11).

3.7.5. Example: telling two Gaussians apart from n samples

For $P = \mathcal{N}(0, \sigma^2)$ and $Q = \mathcal{N}(s, \sigma^2)$ with $s > 0$, the best event collects the outcomes more likely under P. The two bell curves cross at the midpoint $s/2$, so $A^* = \{p > q\} = \{x < s/2\}$. Write Φ for the standard normal cumulative distribution function, $\Phi(z) = \Pr[Z \leq z]$ for a standard normal Z. In standard units the midpoint is $\mu/2$ above P's centre and $\mu/2$ below Q's, with $\mu = s/\sigma$, so $P(A^*) = \Phi(\frac{\mu}{2})$ and $Q(A^*) = \Phi(-\frac{\mu}{2})$. The normal curve is symmetric, so $\Phi(-z) = 1 - \Phi(z)$, and

$$\mathrm{TV} = P(A^*) - Q(A^*) = \Phi\Big(\frac{\mu}{2}\Big) - \Phi\Big(-\frac{\mu}{2}\Big) = 2\Phi\Big(\frac{\mu}{2}\Big) - 1, \qquad \mu = s/\sigma. \tag{3.22}$$

At $\mu = 1$, TV $= 0.3829$, and by Equation 3.21 the best test is right with probability $\frac{1}{2} + \frac{1}{2}(2\Phi(0.5) - 1) = \Phi(0.5) = 0.6915$ (Figure 3.6, left).

Now suppose each observation is shifted by only $\mu = 0.1$ standard deviations, and you see n i.i.d. samples. The evidence of the whole sample is the sum of the evidence of each sample,

$\sum_i L(x_i) = \sum_i \frac{s^2 - 2sx_i}{2\sigma^2} = \frac{n(s^2 - 2s\bar{x})}{2\sigma^2}$, which depends on the data only through the sample mean $\bar{x}$. So nothing is lost by looking only at $\bar{x}$. The sample mean has the same centre as one observation and a variance n times smaller: it is $\mathcal{N}(0, \sigma^2/n)$ under P and $\mathcal{N}(s, \sigma^2/n)$ under Q. That is a Gaussian pair with standardized shift $s/(\sigma/\sqrt{n}) = \mu\sqrt{n}$. So the best test thresholds the sample mean at $s/2$. Table 3.10 gives its accuracy, exactly and by simulation.

Table 3.10.: Telling $\mathcal{N}(0, 1)$ from $\mathcal{N}(0.1, 1)$ with n samples.

n	$\mu\sqrt{n}$	TV	Best accuracy	Simulated (20,000 trials)
100	1	0.3829	0.6915	0.6916
400	2	0.6827	0.8413	0.8424

A shift of a tenth of a standard deviation is invisible in one sample ($\text{TV} = 2\Phi(0.05) - 1 = 0.0399$) but clear with a few hundred. The accuracy depends on n only through $\mu\sqrt{n}$, so reaching a given accuracy needs n proportional to $1/\mu^2$, that is to $1/\text{KL}$, since $\text{KL} = \mu^2/2$.

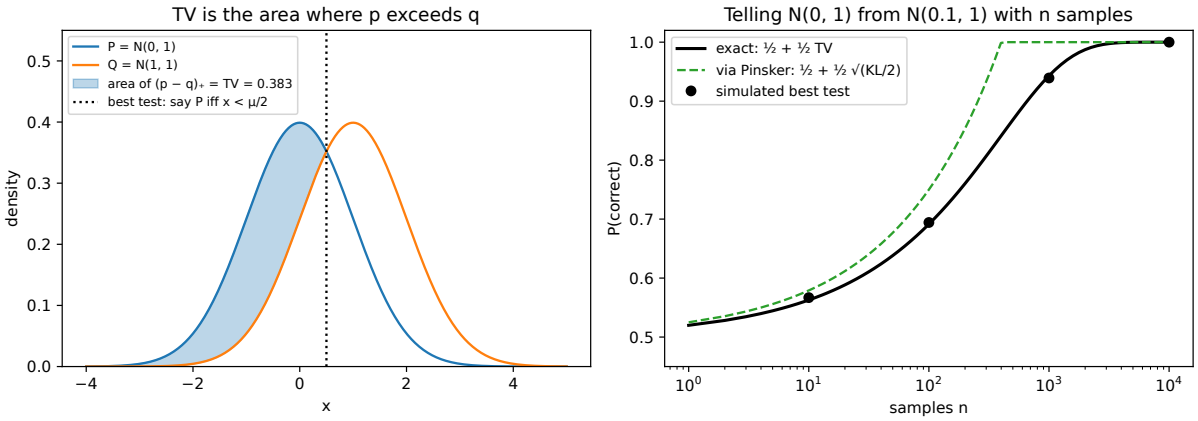


Figure 3.6.: Left: two unit-variance Gaussians one standard deviation apart. The shaded area, where p exceeds q, is the total variation, and the best test thresholds at the midpoint. Right: the best accuracy for telling $\mathcal{N}(0, 1)$ from $\mathcal{N}(0.1, 1)$ as the number of samples grows, exactly, by simulation, and as bounded through Pinsker's inequality.

3.7.6. Check yourself

- Two distributions have disjoint supports. What is their TV, and what does the best test do? *($\text{TV} = 1$: the event "x is in the support of P" has probability 1 under P and 0 under Q. The test looks at which support x falls in and is always right.)*
- Why is $\text{TV}(P, Q) \le 1 - e^{-D_\infty(P\|Q)}$? *(Hint: $p/q \le e^{D_\infty}$ everywhere, so $q \ge pe^{-D_\infty}$, and $\sum(p - q)_+ \le \sum p(1 - e^{-D_\infty})$.)*

3.8. Hockey-stick divergence

3.8.1. Context

TV measures the mass of p that sticks out above q. Now scale q up by a factor $\gamma \geq 1$ and measure the mass of p that *still* sticks out above γq (Figure 3.7). Only outcomes where p is more than γ times q count, so the **hockey-stick divergence** E_γ measures how much of the difference between P and Q comes from likelihood ratios above γ. It shows up whenever one kind of error must be avoided more than the other: tests with unequal priors or unequal costs. It also shows up in **rejection sampling**, a way to draw samples from a target p using draws from an easier q: propose from q and keep each proposal with a probability that corrects for the difference. This needs an envelope $\gamma q \geq p$. If your γ is too small, the part of p above γq is exactly what you lose.

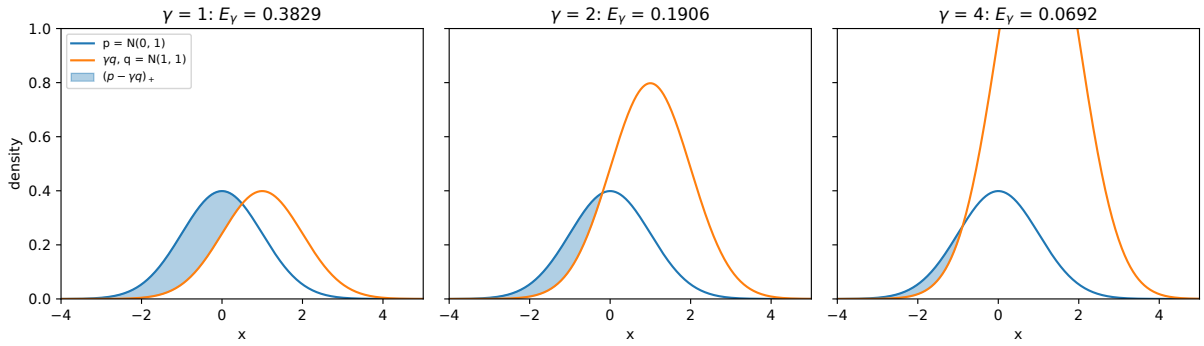


Figure 3.7.: The hockey-stick divergence as an area, for $p = \mathcal{N}(0,1)$ and $q = \mathcal{N}(1,1)$. Raising γ lifts the curve γq, and E_γ is the shaded mass of p still above it: 0.3829 (the total variation) at $\gamma = 1$, then 0.1906 and 0.0692. It shrinks but never reaches zero, because r is unbounded in the left tail.

3.8.2. Statement

The hockey-stick divergence is the TV with q replaced by γq. Like the TV, it is the largest gap $P(A) - \gamma Q(A)$ over events, the mass of p above γq, and an average under Q of how far r exceeds the threshold. For $\gamma \geq 1$,

$$E_\gamma(P\|Q) = \max_A \left[P(A) - \gamma Q(A)\right] = \sum_x \left(p(x) - \gamma q(x)\right)_+ = \mathbb{E}_Q\left[(r-\gamma)_+\right]. \tag{3.23}$$

The name comes from the shape of $t \mapsto (t-\gamma)_+$: flat, then a straight line. It has four properties.

1. $E_1(P\|Q) = \mathrm{TV}(P,Q)$: with no scaling it is the TV.
2. E_γ is non-increasing and convex in γ, and $E_\gamma = 0$ exactly when $\gamma \geq \max r = e^{D_\infty(P\|Q)}$: once γq covers p everywhere, nothing sticks out.
3. In terms of the log-likelihood ratio, it is a difference of two tail probabilities: $E_\gamma(P\|Q) = P[L > \ln\gamma] - \gamma\, Q[L > \ln\gamma]$.

4. **Unequal priors.** If P has prior probability $\pi \le \frac{1}{2}$ and Q has $1-\pi$, the smallest possible probability of a wrong guess from one observation is $\pi(1 - E_\gamma(P\|Q))$ with $\gamma = (1-\pi)/\pi$, the prior odds against P. Always guessing Q errs with probability π, and the observation removes a fraction E_γ of that.

3.8.3. Derivation

1. **The forms.** Fact 3 with weight γ gives the maximum and the sum, and $p - \gamma q = q(r-\gamma)$ turns the sum into $\sum q(r-\gamma)_+ = \mathbb{E}_Q[(r-\gamma)_+]$.
2. **Property 1:** set $\gamma = 1$ in Equation 3.20.
3. **Property 2:** for each outcome, $(r(x)-\gamma)_+$ as a function of γ is a hockey stick turned around, a straight descending line that flattens at 0, so it is non-increasing and convex. An average of non-increasing convex functions is non-increasing and convex. Every term vanishes exactly when $\gamma \ge r(x)$ for all x, that is when $\gamma \ge \max r$.
4. **Property 3:** the terms of $\sum(p-\gamma q)_+$ are non-zero only on $\{r > \gamma\} = \{L > \ln\gamma\}$. On that set the sum of p is $P[L > \ln\gamma]$ and the sum of γq is $\gamma Q[L > \ln\gamma]$.
5. **Property 4:** guessing "P" on A errs when the draw came from P and fell outside A, or came from Q and fell inside A, so it errs with probability $\pi P(A^c) + (1-\pi)Q(A)$. Factoring out π and writing $\gamma = (1-\pi)/\pi$, this is $\pi - \pi\big[P(A) - \gamma Q(A)\big]$. Minimizing the error means maximizing the bracket, which by Fact 3 is E_γ.

Closed forms for the shifted pairs. Property 3 turns E_γ into two tail probabilities, and for the Gaussian and Laplace pairs both are known. Write $c = \ln\gamma$.

For the Gaussian pair ($\mu = s/\sigma$), the evidence is the line $L(x) = \frac{s^2 - 2sx}{2\sigma^2}$, and $L > c$ when $x < \frac{s}{2} - \frac{c\sigma^2}{s}$, that is when $x/\sigma < \frac{\mu}{2} - \frac{c}{\mu}$. Under P, x/σ is standard normal, so $P[L > c] = \Phi(\frac{\mu}{2} - \frac{c}{\mu})$. Under Q, x/σ is a standard normal plus μ, so $Q[L > c] = \Phi(-\frac{\mu}{2} - \frac{c}{\mu})$. Therefore

$$E_\gamma = \Phi\Big(\frac{\mu}{2} - \frac{c}{\mu}\Big) - \gamma\,\Phi\Big(-\frac{\mu}{2} - \frac{c}{\mu}\Big). \tag{3.24}$$

It is positive for every γ, because $D_\infty = \infty$: no finite γ covers p.

For the Laplace pair ($t = s/b$, $b = 1$), the evidence $L(x) = |x-t| - |x|$ is t on $x \le 0$, $-t$ on $x \ge t$, and $t - 2x$ in between. For $0 \le c < t$, $L > c$ happens on that middle piece when $x < \frac{t-c}{2}$. Under $P = \mathrm{Lap}(0,1)$ the probability of $X < \frac{t-c}{2}$, a positive point, is $P[\cdot] = 1 - \frac{1}{2}e^{-(t-c)/2}$. Under Q, $X - t$ is a $\mathrm{Lap}(0,1)$ variable that must be below $\frac{t-c}{2} - t = -\frac{t+c}{2}$, so $Q[\cdot] = \frac{1}{2}e^{-(t+c)/2}$. Since $\gamma e^{-(t+c)/2} = e^c e^{-(t+c)/2} = e^{-(t-c)/2}$, the two halves combine:

$$E_\gamma = 1 - e^{-(t-\ln\gamma)/2} \quad \text{for } 1 \le \gamma \le e^t, \qquad E_\gamma = 0 \text{ beyond.} \tag{3.25}$$

At $\gamma = 1$ this gives $\mathrm{TV} = 1 - e^{-t/2}$.

3.8.4. Interpretation

The curve $\gamma \mapsto E_\gamma$ shows how much of the difference between P and Q comes from large likelihood ratios. TV is its value at $\gamma = 1$, and the max-divergence is where it hits zero (Figure 3.8). For the die, the only outcome with $r > \gamma$, for $1 \le \gamma < 3$, is the six, which contributes $p - \gamma q = 0.5 - \gamma/6$.

So $E_\gamma = 0.5 - \gamma/6$ falls linearly to zero at $\gamma = 3$. For the Laplace pair it reaches zero at $\ln\gamma = t$. The Gaussian curve never reaches zero, but it gets very small. In this sense E_γ separates a likelihood ratio that is *never* large (Laplace) from one that is large only *with small probability* (Gaussian).

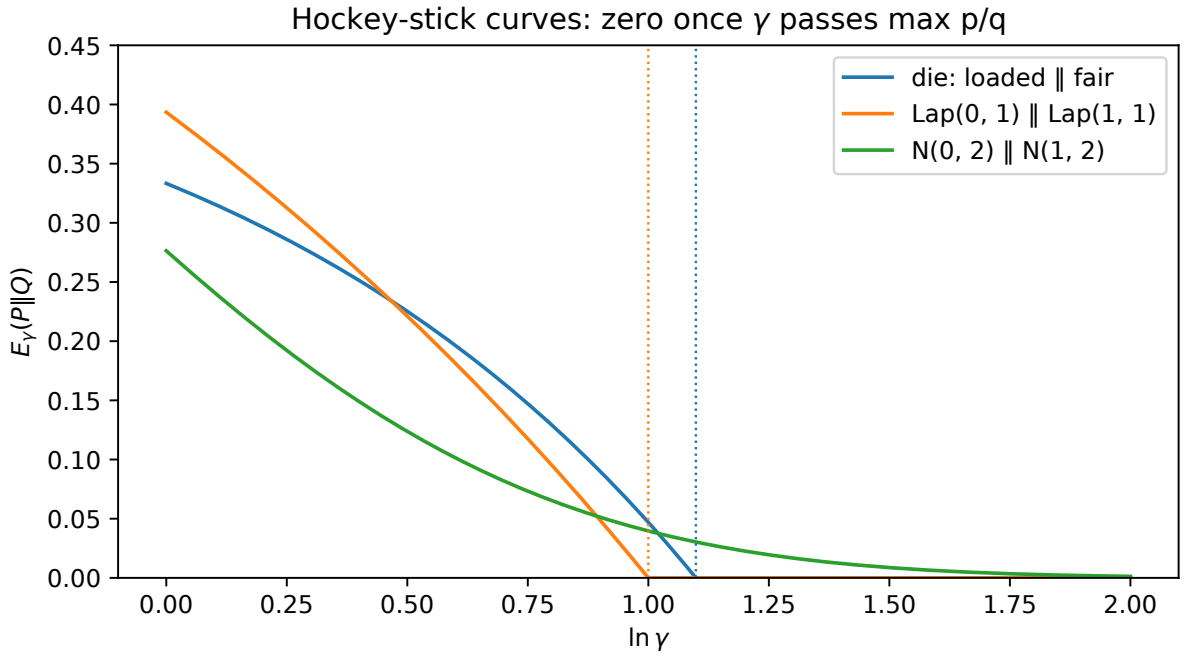


Figure 3.8.: The hockey-stick divergence $E_\gamma(P\|Q)$ as a function of $\ln\gamma$ for the loaded die, a Laplace pair and a Gaussian pair with the same variance. The die and Laplace curves reach zero at their max-divergences ($\ln 3$ and 1, dotted lines); the Gaussian curve stays positive.

3.8.5. Example: rejection sampling with an envelope that is too low

You want samples from the loaded die p, using rolls of the fair die q as proposals, with envelope γq. Propose $x \sim q$ and accept it with probability $\min\left(1, \frac{p(x)}{\gamma q(x)}\right)$. The chance of proposing x and accepting it is $q(x)\min(1, p/(\gamma q)) = \min(p, \gamma q)/\gamma$, so the accepted rolls have distribution $m \propto \min(p, \gamma q)$. Where the envelope covers p this is p; where it does not, the excess of p is cut off.

With $\gamma = 3 = \max r$, the envelope covers p, so $m = p$ exactly. Each proposal is accepted with total probability $\sum \min(p, 3q)/3 = 1/\gamma = \frac{1}{3}$.

With $\gamma = 2$, the envelope $2q = \frac{1}{3}$ is below $p(6) = 0.5$. The uncovered mass is $E_2(P\|Q) = 0.5 - \frac{1}{3} = \frac{1}{6}$ (the script gives 0.1667). The acceptance rate rises to $\sum \min(q, p/\gamma) = (1 - E_2)/\gamma = \frac{5}{12}$, since $\sum \min(p, \gamma q)$ is all of p except the part above γq, which is $1 - E_\gamma$. But the output is wrong: $\min(p, 2q) = (0.1, \ldots, 0.1, \frac{1}{3})$ sums to $\frac{5}{6}$, and dividing by $\frac{5}{6}$ gives $m = (0.12, \ldots, 0.12, 0.4)$. A six now comes up 40% of the time instead of 50%: $\mathrm{TV}(P, M) = 0.5 - 0.4 = 0.1$.

In general $\mathrm{TV}(P, M) \le E_\gamma(P\|Q)$ (Exercise 4 in the exercise book). The hockey-stick divergence is exactly the price of a cheap envelope.

3.8.6. Check yourself

- Why must $\gamma \ge 1$ in the definition? *(For $\gamma < 1$, $\max_A[P(A) - \gamma Q(A)] \ge 1 - \gamma > 0$ even when $P = Q$, by taking A to be everything. The quantity no longer vanishes at $P = Q$.)*
- For the Gaussian pair with $\mu = 1$, the script gives $E_2 = 0.1906$, and a Monte Carlo average of $(r-2)_+$ over a million draws from Q gives 0.1892. Which form of E_γ does the simulation use, and why does it need samples from Q, not P? *(The form $\mathbb{E}_Q[(r-\gamma)_+]$, which is an average under Q, so a sample average estimates it only when the samples come from Q.)*

3.9. f-divergences

3.9.1. Context

KL, TV and E_γ were defined separately, but each is an average under Q of some function of the likelihood ratio. The function says how much a given ratio counts as "difference". Naming it gives one family, the **f-divergences**, with one proof of non-negativity and one proof of the data-processing inequality for all its members. Rényi divergence is not a member, but it is an increasing function of one, so it inherits everything.

3.9.2. Statement

Pick a convex function f that is zero at $t = 1$, so that a ratio of 1 (no evidence either way) counts as no difference. The f-divergence averages f of the likelihood ratio under Q. For a convex $f : (0, \infty) \to \mathbb{R}$ with $f(1) = 0$,

$$D_f(P\|Q) = \mathbb{E}_Q[f(r(X))] = \sum_x q(x)\, f\Big(\frac{p(x)}{q(x)}\Big). \tag{3.26}$$

Table 3.11 lists the common choices of f, called **generators**.

Table 3.11.: Common f-divergences, their generators, and their values for the loaded die against the fair die.

Divergence	$f(t)$	Die, loaded ‖ fair
KL, $D_{\mathrm{KL}}(P\|Q)$	$t \ln t$	0.2939
Reverse KL, $D_{\mathrm{KL}}(Q\|P)$	$-\ln t$	0.2426
Total variation	$\frac{1}{2}\lvert t-1\rvert$	0.3333
Hockey-stick E_γ ($\gamma = 2$)	$(t-\gamma)_+$	0.1667
χ^2	$(t-1)^2$	0.8000
Squared Hellinger H^2	$(\sqrt{t}-1)^2$	0.1317

Rényi as a transform. The Rényi divergence is a logarithm of $\mathbb{E}_Q[r^\alpha]$, and $\mathbb{E}_Q[r^\alpha]$ minus 1 is an f-divergence. For $\alpha > 1$, $f(t) = t^\alpha - 1$ is convex with $f(1) = 0$, so $D_f = \mathbb{E}_Q[r^\alpha] - 1$ and $D_\alpha = \frac{1}{\alpha-1}\ln(1 + D_f)$, an increasing function of D_f. For $0 < \alpha < 1$, t^α is concave, so use the convex $f(t) = 1 - t^\alpha$. Then $D_f = 1 - \mathbb{E}_Q[r^\alpha]$ and $D_\alpha = \frac{1}{\alpha-1}\ln(1 - D_f)$, again increasing in D_f (the logarithm decreases in D_f, and $\frac{1}{\alpha-1}$ is negative).

Non-negativity. $D_f(P\|Q) \geq 0$, with equality when $P = Q$.

The hockey-stick curves determine every f-divergence (can wait for a second reading). The idea is that any convex f can be built from hockey-stick shapes $(t-\gamma)_+$ and $(\gamma-t)_+$, one bend at each γ, weighted by how sharply f bends there, its second derivative $f''(\gamma)$. Averaging each hockey stick under Q gives a hockey-stick divergence, so the f-divergence is a weighted sum of them. If f is twice differentiable,

$$D_f(P\|Q) = \int_1^\infty f''(\gamma)\, E_\gamma(P\|Q)\, d\gamma + \int_0^1 \gamma f''(\gamma)\, E_{1/\gamma}(Q\|P)\, d\gamma. \tag{3.27}$$

3.9.3. Derivation

Non-negativity. Jensen (Fact 2) says the average of a convex function is at least the function of the average, and Fact 1 says r averages to 1 under Q. So $\mathbb{E}_Q[f(r)] \geq f(\mathbb{E}_Q[r]) = f(1) = 0$.

Equation 3.27. The plan is to write $f(t)$ as hockey sticks for each fixed t, then put in $t = r(X)$ and average.

1. **Taylor's theorem around $t = 1$.** With the remainder written as an integral, $f(t) = f(1) + f'(1)(t-1) + \int_1^t f''(\gamma)(t-\gamma)\, d\gamma$: the value at 1, the tangent line, and the curvature accumulated between 1 and t.
2. **Rewrite the remainder with hockey sticks,** so that one formula holds for all t at once: it equals $\int_1^\infty f''(\gamma)(t-\gamma)_+\, d\gamma + \int_0^1 f''(\gamma)(\gamma-t)_+\, d\gamma$. For $t \geq 1$, $(t-\gamma)_+$ is zero past $\gamma = t$, so the first integral is the remainder and the second is zero. For $t < 1$, swapping the limits of $\int_1^t$ flips the sign of $(t-\gamma)$ into $(\gamma-t)$ on $[t, 1]$, which is the second integral, and the first is zero.
3. **Average under Q.** Put $t = r(X)$ and average. $f(1) = 0$, and $\mathbb{E}_Q[r-1] = 0$ by Fact 1, so the first two terms vanish.
4. **The first integral.** $\mathbb{E}_Q[(r-\gamma)_+] = E_\gamma(P\|Q)$ by definition.
5. **The second integral.** $\mathbb{E}_Q[(\gamma-r)_+] = \sum(\gamma q - p)_+ = \gamma\sum(q - p/\gamma)_+ = \gamma E_{1/\gamma}(Q\|P)$, the hockey-stick divergence in the other direction.

Concretely, KL on the die. For $f(t) = t\ln t$, $f'(t) = \ln t + 1$ and $f''(\gamma) = 1/\gamma$. From Section 3.8, $E_\gamma(P\|Q) = \frac{1}{2} - \frac{\gamma}{6}$ for $1 \leq \gamma \leq 3$ and 0 beyond. In the other direction, $q/p = \frac{5}{3}$ on rolls 1–5 and $\frac{1}{3}$ on the six. For $\gamma \leq 1$, the term $(q - p/\gamma)_+$ is positive on a non-six when $\frac{1}{6} > \frac{0.1}{\gamma}$, that is when $\gamma > 0.6$, and never on the six. So $E_{1/\gamma}(Q\|P) = 5(\frac{1}{6} - \frac{0.1}{\gamma})_+ = \frac{5}{6} - \frac{1}{2\gamma}$ for $\gamma > 0.6$. The weight $\gamma f''(\gamma)$ is 1. Then

$$\int_1^3 \frac{1}{\gamma}\Big(\frac{1}{2} - \frac{\gamma}{6}\Big)d\gamma + \int_{0.6}^1 \Big(\frac{5}{6} - \frac{1}{2\gamma}\Big)d\gamma = \Big(\tfrac{1}{2}\ln 3 - \tfrac{1}{3}\Big) + \Big(\tfrac{1}{3} - \tfrac{1}{2}\ln\tfrac{5}{3}\Big) = \tfrac{1}{2}\ln 1.8 = 0.2939, \tag{3.28}$$

the KL computed directly. The first integral is $\frac{1}{2}\ln 3 - \frac{2}{6}$; the second is $\frac{5}{6} \times 0.4 - \frac{1}{2}\ln\frac{1}{0.6}$; and $\ln 3 - \ln\frac{5}{3} = \ln 1.8$. The script repeats the integral numerically and also gets 0.2939.

3.9.4. Interpretation

Each f weights the likelihood ratio differently (Figure 3.9). χ^2 grows quadratically in t, so it is dominated by outcomes with large ratios; on the die it is 0.80, the largest entry in Table 3.11. TV and E_γ grow linearly and ignore everything below a threshold. Reverse KL blows up as $t \to 0$, so it punishes outcomes that P almost never produces.

The E_γ curves are complete information. By Equation 3.27, if you know E_γ in both directions for all $\gamma \geq 1$, you know every f-divergence, and so every Rényi divergence. Later parts use this to move between different ways of saying two distributions are close.

f-divergences are also used to train models. They have a variational form, $D_f(P\|Q) = \sup_T \left(\mathbb{E}_P[T] - \mathbb{E}_Q[f^*(T)]\right)$, where $f^*(u) = \sup_t \left(ut - f(t)\right)$ is the convex conjugate of f and the supremum runs over functions T of the outcome. Both expectations can be estimated from samples, so a divergence can be estimated without knowing either density. Estimating the supremum with a neural network T from samples is how a variant of the generative adversarial network (GAN), the f-GAN, trains generative models (Nowozin, Cseke & Tomioka, 2016).

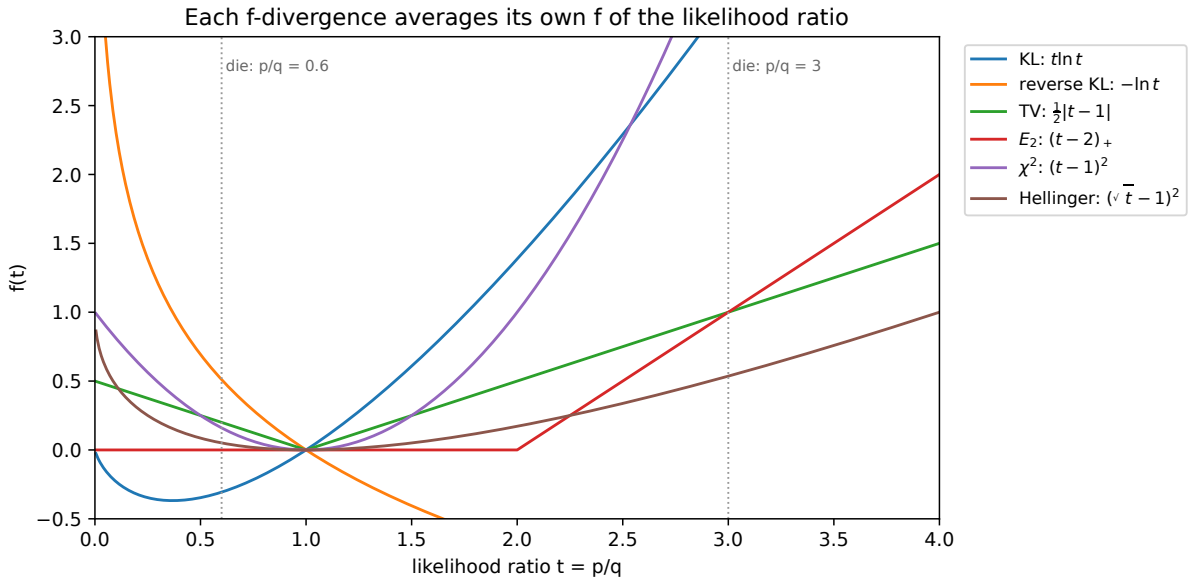


Figure 3.9.: The generator functions f of six f-divergences, as functions of the likelihood ratio $t = p/q$. All are convex and vanish at $t = 1$. The dotted lines mark the two likelihood ratios of the loaded die.

3.9.5. Check yourself

- Why does $f(t) = t \ln t$ give $D_{\mathrm{KL}}(P\|Q)$ when the definition averages under Q? *($\mathbb{E}_Q[r \ln r] = \mathbb{E}_P[\ln r]$ by Fact 1.)*
- Adding $c(t-1)$ to f gives a different function. Does it change D_f? *(No: $\mathbb{E}_Q[c(r-1)] = 0$. So TV can equally use $f(t) = (t-1)_+$.)*

3.10. The data-processing inequality

3.10.1. Context

You rarely look at raw data. You bin it, compute features, threshold a score, or pass it through a randomized algorithm. The **data processing inequality** (DPI) says that none of this can make two distributions easier to tell apart. The reason is simple. When processing maps several inputs to the same output, the output's likelihood ratio is an *average* of the inputs' likelihood ratios. Averaging pulls strong evidence toward weak evidence: a six (ratio 3) reported together with a two and a four (ratio 0.6 each) becomes a moderate ratio of $(3 + 0.6 + 0.6)/3 = 1.4$. Every divergence is a convex summary of the ratios, and by Jensen, averaging first can only make a convex summary smaller. This is the most used property of divergences. It turns "any processing" into "no gain", without knowing what the processing is.

3.10.2. Statement

Processing may be random, so describe it by its output probabilities. A **channel** (or Markov kernel) K maps each input x to a distribution $K(\cdot \mid x)$ over outputs y. A deterministic function is the special case $K(y \mid x) = \mathbf{1}[y = g(x)]$. Feeding $X \sim P$ through K gives the output distribution $PK(y) = \sum_x p(x)K(y \mid x)$, and likewise QK. The inequality says the outputs are never further apart than the inputs. For every f-divergence and every channel,

$$D_f(PK \| QK) \le D_f(P \| Q). \tag{3.29}$$

The same holds for every Rényi divergence D_α, including D_∞. Equality holds when the processing keeps the likelihood ratio: when $r(x)$ can be computed from y. Such a y is a **sufficient statistic**.

A useful special case. The coarsest processing reports only whether an event A happened. That turns P and Q into two coins, with probabilities of heads $P(A)$ and $Q(A)$, and the KL between two coins with heads probabilities a and b is the **binary KL** $\mathrm{kl}(a, b)$. So for every event A,

$$D_{\mathrm{KL}}(P \| Q) \ge \mathrm{kl}(P(A), Q(A)), \qquad \mathrm{kl}(a, b) = a \ln \frac{a}{b} + (1 - a) \ln \frac{1 - a}{1 - b}. \tag{3.30}$$

3.10.3. Derivation

1. **The output's likelihood ratio is an average of the input's.** For each output y, write $p(x) = q(x)r(x)$ inside $PK(y)$ and divide by $QK(y)$:

$$r_Y(y) = \frac{PK(y)}{QK(y)} = \sum_x \frac{q(x)K(y \mid x)}{QK(y)}\, r(x) = \mathbb{E}\big[r(X) \mid Y = y\big], \tag{3.31}$$

   where (X, Y) is drawn as $X \sim Q$, then $Y \sim K(\cdot \mid X)$. The weights $q(x)K(y \mid x)/QK(y)$ are exactly the conditional probabilities of X given $Y = y$ (Bayes' rule), so the sum is a conditional average.
2. **Jensen, conditionally.** Fact 2 holds for any distribution, in particular for the conditional distribution of X given $Y = y$. For convex f, the function of an average is at most the average of the function: $f(r_Y(y)) = f(\mathbb{E}[r(X) \mid Y = y]) \le \mathbb{E}\big[f(r(X)) \mid Y = y\big]$.

3. **Average over $Y \sim QK$.** The left side becomes $\mathbb{E}_{QK}[f(r_Y)] = D_f(PK\|QK)$. On the right, averaging the conditional averages over Y gives the overall average (the tower property), $\mathbb{E}_Q[f(r(X))] = D_f(P\|Q)$.
4. **Rényi.** D_α is an increasing function of an f-divergence (Section 3.9), so the inequality carries over. For D_∞, take $\alpha \to \infty$.
5. **Equality.** If $r(x)$ is a function of y, then $r(X)$ is constant given $Y = y$, and Jensen in step 2 holds with equality.

Concretely, on the die. Report only whether the roll is odd or even (Table 3.12).

Table 3.12.: Processing the die roll into its parity averages the likelihood ratio within each output.

Y	Rolls	$P(Y)$	$Q(Y)$	$r_Y = P(Y)/Q(Y)$	Average of r over the rolls, Q-weighted
even	2, 4, 6	0.7	0.5	1.4	$(0.6 + 0.6 + 3)/3 = 1.4$
odd	1, 3, 5	0.3	0.5	0.6	$(0.6 + 0.6 + 0.6)/3 = 0.6$

The ratios 0.6 and 3 on the even rolls are averaged into 1.4: the strong evidence of a six is diluted by the twos and fours. The KL of the parity is computed like any KL, as the P-weighted average of $\ln r_Y$, and drops from 0.2939 to $0.7 \ln 1.4 + 0.3 \ln 0.6 = 0.0823$. If instead you report only whether the roll is a six, nothing is lost: r depends only on that bit, so the bit is sufficient, and the KL stays at $\mathrm{kl}(0.5, \frac{1}{6}) = 0.5 \ln 3 + 0.5 \ln 0.6 = 0.2939$.

3.10.4. Interpretation

There is no free lunch from processing. A feature extractor, a summary statistic or a quantizer can only lose information about which of two distributions produced the data. Any accuracy you get from a processed output, you could have got from the raw input, by applying the same processing yourself.

Randomized processing is included. The channel can use its own randomness, as long as that randomness does not depend on whether the input came from P or Q: the channel K is the same in both worlds.

The special case gives lower bounds from a single event. $D_{\mathrm{KL}} \geq \mathrm{kl}(P(A), Q(A))$ says: if the divergence is small, then no event can have very different probabilities under P and Q. Hypothesis testing uses this form to bound the errors of any test.

3.10.5. Example: binning a Gaussian

$P = \mathcal{N}(0, 1)$ and $Q = \mathcal{N}(1, 1)$ have KL $1^2/2 = 0.5$. Suppose a sensor reports x only to a resolution w: it rounds x into bins of width w, with an edge at the midpoint $\frac{1}{2}$. Within each bin, the log-likelihood ratio, a straight line for the raw data, is replaced by a single level (Figure 3.10).

The wider the bins, the more evidence is averaged away. Table 3.13 gives the KL between the binned distributions, from the script.

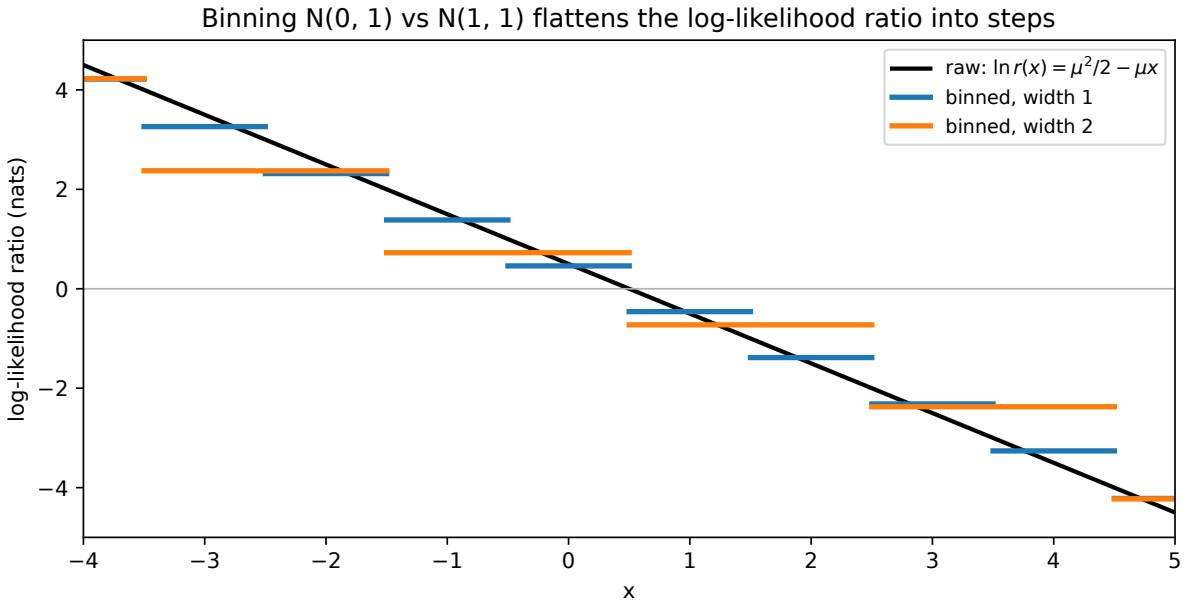


Figure 3.10.: The log-likelihood ratio of $\mathcal{N}(0,1)$ against $\mathcal{N}(1,1)$, raw (black line) and after rounding x into bins of width 1 and 2. Each bin reports one value, the log of the Q-weighted average of r over the bin, so the line becomes a staircase. Wider bins give flatter steps and less evidence.

Table 3.13.: KL between $\mathcal{N}(0,1)$ and $\mathcal{N}(1,1)$ after rounding x into bins.

Bin width w	0.5	1	2	one bit: $x < \frac{1}{2}$?	raw
KL after binning	0.4898	0.4618	0.3781	0.3090	0.5

Coarser bins lose more, but slowly: a single bit, the threshold at the midpoint, keeps 62% of the KL ($0.3090/0.5 = 0.618$, Figure 3.11). That bit is the best single-shot test from the total variation section, and it keeps all of the TV, since TV is attained by that very event. So the same processing can keep one divergence whole while losing 38% of another: different divergences lose different amounts.

3.10.6. Check yourself

- Is the DPI violated if a processing step increases the *accuracy* of some particular classifier? *(No. The DPI bounds the best accuracy over all tests. A processing step can help a bad classifier, but it cannot beat the best test on the raw data.)*
- Why is the sample mean of n Gaussian observations lossless for telling $\mathcal{N}(0,1)$ from $\mathcal{N}(\mu,1)$? *(The log-likelihood ratio of the sample is a function of the sample mean, so it is sufficient and the DPI holds with equality.)*

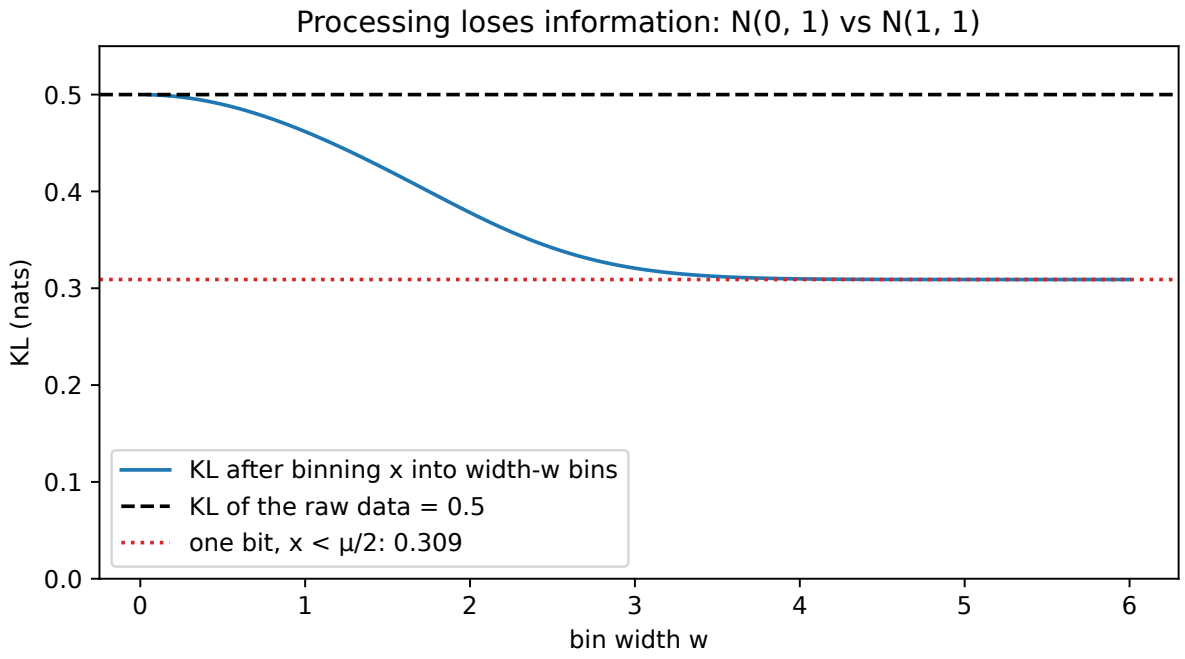


Figure 3.11.: KL between $\mathcal{N}(0,1)$ and $\mathcal{N}(1,1)$ after rounding the observation into bins of width w. It never exceeds the raw-data value 0.5, and it falls to the one-bit value 0.309 as the bins grow.

3.11. Pinsker's inequality

3.11.1. Context

KL is easy to compute and adds up over samples. TV is what you want: it is the accuracy of the best test and the largest change in any bounded statistic. **Pinsker's inequality** converts the first into a bound on the second.

3.11.2. Statement

Pinsker's inequality says that TV is at most the square root of half the KL. The square root is there because, for nearby distributions, TV grows linearly in the difference while KL grows quadratically, as the derivation shows:

$$\mathrm{TV}(P,Q) \le \sqrt{\tfrac{1}{2} D_{\mathrm{KL}}(P\|Q)}. \tag{3.32}$$

For large KL, where Equation 3.32 exceeds 1 and says nothing (TV is never above 1), use the **Bretagnolle–Huber** inequality, which stays below 1 for every KL:

$$\mathrm{TV}(P,Q) \le \sqrt{1 - e^{-D_{\mathrm{KL}}(P\|Q)}}. \tag{3.33}$$

3.11.3. Derivation

Why a square root. Before the proof, here is where the shape comes from. Let P be close to Q, and write $r = 1 + h$ with h small: $h(x)$ is the relative difference between p and q at x. Then $\mathbb{E}_Q[h] = \mathbb{E}_Q[r] - 1 = 0$ by Fact 1.

1. *TV is linear in h:* $\mathrm{TV} = \frac{1}{2}\mathbb{E}_Q|r-1| = \frac{1}{2}\mathbb{E}_Q|h|$, by the form $\frac{1}{2}\sum|p-q| = \frac{1}{2}\sum q|r-1|$.
2. *KL is quadratic in h:* $D_{\mathrm{KL}} = \mathbb{E}_P[\ln r] = \mathbb{E}_Q[r \ln r]$ by Fact 1. For small h, $\ln(1+h) = h - \frac{h^2}{2} + O(h^3)$, so $(1+h)\ln(1+h) = h + \frac{h^2}{2} + O(h^3)$. The first term averages to 0, so $D_{\mathrm{KL}} \approx \frac{1}{2}\mathbb{E}_Q[h^2]$.
3. *Compare:* the average of $|h|$ is at most the root of the average of h^2 (Cauchy–Schwarz, or Jensen with the convex function y^2): $\mathbb{E}_Q|h| \le \sqrt{\mathbb{E}_Q[h^2]}$. So $\mathrm{TV} \lesssim \frac{1}{2}\sqrt{2D_{\mathrm{KL}}} = \sqrt{D_{\mathrm{KL}}/2}$.

That is Pinsker's inequality for nearby distributions, constant included. The proof below makes it exact for all P and Q.

Pinsker. The plan is to reduce any pair to a pair of coins, where the inequality is a one-variable calculus fact.

1. **Reduce to two outcomes.** Let $A^* = \{p > q\}$, $a = P(A^*)$ and $b = Q(A^*)$. By Fact 3, this event attains the TV, so $\mathrm{TV} = a - b$. Reporting only whether $x \in A^*$ turns P and Q into coins with heads probabilities a and b, and by the special case of the data-processing inequality (Section 3.10) this can only lower KL: $D_{\mathrm{KL}}(P\|Q) \ge \mathrm{kl}(a,b)$. So it is enough to show $\mathrm{kl}(a,b) \ge 2(a-b)^2$ for all $a, b \in (0,1)$.
2. **Fix a and vary b.** Let $g(b) = \mathrm{kl}(a,b) - 2(a-b)^2$, the gap we want to show is non-negative. Then $g(a) = 0$, because two equal coins have zero KL.
3. **Differentiate:** $\frac{d}{db}\mathrm{kl}(a,b) = -\frac{a}{b} + \frac{1-a}{1-b} = \frac{b-a}{b(1-b)}$, and $\frac{d}{db}\big(-2(a-b)^2\big) = 4(a-b)$, so
$$g'(b) = (b-a)\left(\frac{1}{b(1-b)} - 4\right). \tag{3.34}$$
4. **Sign:** $b(1-b) \le \frac{1}{4}$ (a product of two numbers that add to 1 is largest when both are $\frac{1}{2}$), so the bracket is ≥ 0. Then $g' \le 0$ for $b < a$ and $g' \ge 0$ for $b > a$: g decreases to its minimum at $b = a$, where it is 0, and increases after. So $g \ge 0$.
5. **Combine:** $D_{\mathrm{KL}} \ge \mathrm{kl}(a,b) \ge 2(a-b)^2 = 2\,\mathrm{TV}^2$, which rearranges to Equation 3.32.

Bretagnolle–Huber. The plan is to bound the overlap $\sum\sqrt{pq}$ from both sides: above by TV, below by KL.

1. **Split into min and max.** At each x, $\min(p,q)\cdot\max(p,q) = pq$, so $\sqrt{pq} = \sqrt{\min(p,q)}\sqrt{\max(p,q)}$. The Cauchy–Schwarz inequality, $\big(\sum_x u_x v_x\big)^2 \le \sum_x u_x^2 \sum_x v_x^2$, then gives $\big(\sum_x \sqrt{pq}\big)^2 \le \sum_x \min(p,q) \cdot \sum_x \max(p,q)$.
2. **Express in TV.** $\sum\min(p,q) = 1 - \mathrm{TV}$ and $\sum\max(p,q) = 1 + \mathrm{TV}$, since $\min + \max = p + q$ sums to 2 and $\max - \min = |p-q|$ sums to $2\,\mathrm{TV}$. So $\big(\sum\sqrt{pq}\big)^2 \le (1-\mathrm{TV})(1+\mathrm{TV}) = 1 - \mathrm{TV}^2$.
3. **Express in KL.** $\sum_x \sqrt{pq} = \sum_x p\sqrt{q/p} = \mathbb{E}_P[e^{-L/2}]$. The function $e^{-y/2}$ is convex, so by Jensen $\mathbb{E}_P[e^{-L/2}] \ge e^{-\mathbb{E}_P[L]/2} = e^{-D_{\mathrm{KL}}/2}$.
4. **Combine:** squaring step 3 and using step 2, $e^{-D_{\mathrm{KL}}} \le 1 - \mathrm{TV}^2$, which rearranges to Equation 3.33.

3.11.4. Interpretation: how tight are these?

Near zero, Pinsker has the right shape. Take two coins with heads probabilities $\frac{1}{2} \pm \frac{\delta}{2}$. They differ by δ on each side, so TV $= \delta$. For coins near $\frac{1}{2}$, the binary KL is about the squared difference divided by $2 \times \frac{1}{2} \times \frac{1}{2}$, so KL $\approx 2\delta^2$, and Equation 3.32 gives $\sqrt{\delta^2} = \delta$: it is tight as $\delta \to 0$, and the constant $\frac{1}{2}$ cannot be improved. In the square-root argument above, this is the case where $|h|$ is the same on every outcome, so Cauchy–Schwarz is an equality. For the Laplace pair, KL $\approx t^2/2$ and TV $= 1 - e^{-t/2} \approx t/2$ for small t, and Pinsker gives $\sqrt{t^2/4} = t/2$, so it is also nearly tight there.

For large KL, Pinsker is useless. It passes 1 at KL $= 2$ ($\sqrt{2/2} = 1$), while TV can never exceed 1. Bretagnolle–Huber is always at most 1, and solving for where the two bounds cross shows it is better than Pinsker once KL is above 1.59 (Figure 3.12).

For Gaussians, Pinsker loses a constant factor. KL $= \mu^2/2$ gives the bound $\sqrt{\mu^2/4} = \mu/2$, while the exact TV is $2\Phi(\mu/2) - 1$. For small μ the normal density near 0 is about its peak height $1/\sqrt{2\pi}$, so $2\Phi(\mu/2) - 1 \approx \mu/\sqrt{2\pi} \approx 0.40\mu$. Here $|h|$ varies across outcomes, so Cauchy–Schwarz has slack.

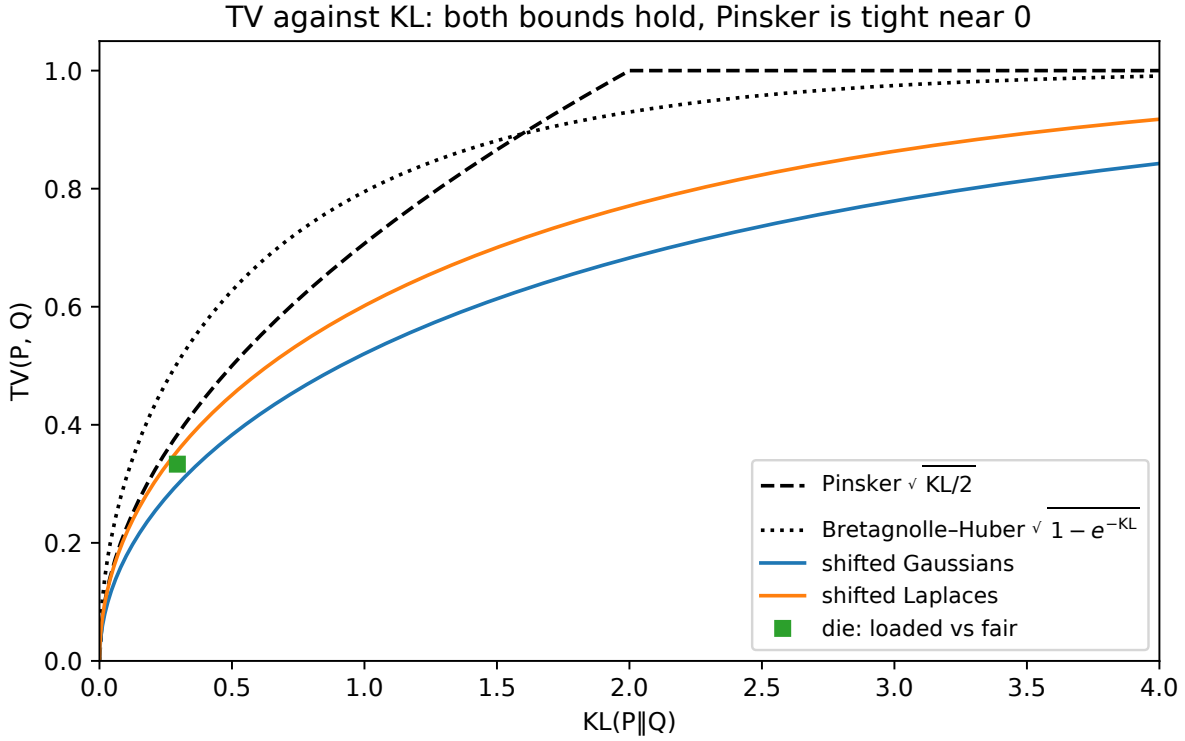


Figure 3.12.: Total variation against KL for shifted Gaussian pairs, shifted Laplace pairs and the die, with the Pinsker and Bretagnolle–Huber bounds. Every pair lies below both bounds; Pinsker is close near KL $= 0$ and vacuous beyond KL $= 2$.

3.11.5. Example: how many samples to tell two distributions apart?

KL is additive, so n i.i.d. samples have $\mathrm{KL}_n = n\,\mathrm{KL}_1$, and Pinsker gives $\mathrm{TV}_n \le \sqrt{n\,\mathrm{KL}_1/2}$. By Equation 3.21, the best accuracy is at most $\frac{1}{2} + \frac{1}{2}\sqrt{n\,\mathrm{KL}_1/2}$. For that bound to get close to 1, $n\,\mathrm{KL}_1$ must be of order 1, so getting accuracy close to 1 needs n on the order of $1/\mathrm{KL}_1$, whatever test you use.

For $\mathcal{N}(0,1)$ against $\mathcal{N}(0.1,1)$, $\mathrm{KL}_1 = 0.1^2/2 = 0.005$, and Table 3.14 compares the bounds with the exact TV. For example, at $n = 100$ Pinsker gives $\sqrt{0.5/2} = 0.5$ and Bretagnolle–Huber gives $\sqrt{1 - e^{-0.5}}$.

Table 3.14.: Exact total variation between n samples of $\mathcal{N}(0,1)$ and of $\mathcal{N}(0.1,1)$, and the two bounds computed from KL.

n	KL_n	Exact TV	Pinsker	Bretagnolle–Huber
100	0.50	0.3829	0.5000	0.6273
400	2.00	0.6827	1.0000	0.9299

At $n = 100$ Pinsker overstates TV by 31% ($0.5000/0.3829 = 1.31$). At $n = 400$ it is vacuous, and Bretagnolle–Huber still gives a non-trivial bound. Figure 3.6 (right) shows the Pinsker bound on accuracy over the whole range of n. The same comparison for single pairs: Gaussian $\mu = 1$, TV 0.3829 against Pinsker 0.5000; Laplace $t = 1$, TV 0.3935 against 0.4289; the die, TV 0.3333 against 0.3833.

3.11.6. Check yourself

- Why is the reduction to two outcomes in step 1 of the Pinsker derivation free? *(TV is attained by the single event A^*, so processing the data into the bit "$x \in A^*$?" keeps TV exactly, and the DPI says it can only lower KL.)*
- Pinsker bounds TV by KL. Can you bound KL by TV? *(Not in general. Let Q move probability 10^{-9} from P onto an outcome where P puts none. Then TV $= 10^{-9}$, but $D_{\mathrm{KL}}(Q\|P) = \infty$.)*

3.12. Code: check every divergence by computation and simulation

The script code/divergences.py computes every number in this chapter and draws every figure. Each divergence is a small typed function. The closed forms for Gaussian and Laplace pairs are checked against Monte Carlo averages and against each other, and the figures loop the same functions over parameters. From the repository root:

```
uv run python code/part01-prerequisites/divergences.py
```

Output (seed 0), summarized in Table 3.15:

Table 3.15.: The script's output with the default seed.

Experiment	Result
Die, loaded ‖ fair	KL 0.2939; reverse KL 0.2426; TV 0.3333; χ^2 0.8000; H^2 0.1317; E_2 0.1667; E_3 0; Rényi orders in Table 3.6
Die, power means of r under P	Harmonic 1.0000; geometric 1.3416; arithmetic 1.8000; maximum 3.0000
Die, evidence per roll	Loaded die: mean 0.2939, standard deviation 0.8047, $(\text{sd}/\text{mean})^2 = 7.50$. Fair die: evidence for "loaded" has mean -0.2426, standard deviation 0.5998
Die, KL rebuilt from the E_γ curves (Equation 3.27)	0.2939
Die, after processing	Parity: KL 0.0823. "Is it a six?": KL 0.2939
Gaussians, $\mu = 1$	KL 0.5000 (Monte Carlo 0.4990); D_2 1.0000; D_{10} 5.0000; TV 0.3829; best accuracy 0.6915; E_2 0.1906 (Monte Carlo 0.1892); E_4 0.0692
Gaussians, $\mu = 1$, $Q[L \geq 2]$	Exact 0.0062; Markov 0.1353; Rényi–Chernoff 0.0439 at $\alpha = 2.5$
Gaussians, $\mu = 1$, binned	Width 0.5: 0.4898; width 1: 0.4618; width 2: 0.3781; one bit: 0.3090
Laplace, $t = 1$	KL 0.3679 (Monte Carlo 0.3684); $D_{0.5}$ 0.1891; D_2 0.6191; D_{10} 0.9287; D_∞ 1; TV 0.3935; E_2 0.1422
n samples, $\mu = 0.1$	$n = 100$: accuracy 0.6915, simulated 0.6916. $n = 400$: accuracy 0.8413, simulated 0.8424
Fitting one Gaussian to two bumps	Forward: $\mu = 0.00$, $\sigma = 2.06$. Reverse: $\mu = 2.00$, $\sigma = 0.50$

tests/test_divergences.py checks that the output still matches.

What to take from it. First, the closed forms are right. Every Monte Carlo average agrees with its formula to within 0.0015, about the sampling error of a million draws, and KL rebuilt from the hockey-stick curves matches KL computed directly. Second, Rényi divergences really are means of one ratio: on the die, the harmonic, geometric and arithmetic means and the maximum of r give D_0, KL, D_2 and D_∞.

Third, processing only loses. Parity throws away more than 70% of the KL of the die, while the sufficient bit "is it a six?" keeps all of it. Fourth, the bulk and the worst case can disagree. The Laplace pair is further apart than the Gaussian pair at low orders and closer at high orders, so which pair is "more different" depends on the divergence you ask. Finally, direction matters: forward and reverse KL give opposite fits to the same data.

3.13. References

- van Erven & Harremoës, "Rényi Divergence and Kullback–Leibler Divergence" (IEEE Transactions on Information Theory, 2014), §§1–3. Everything this book uses from it is derived above: the definition, the special orders, the limits $\alpha \to 0, 1, \infty$, monotonicity in α, skew symmetry, additivity and data processing. Go to the paper for one of two things: proofs for general measures (densities with respect to arbitrary reference measures, where the limits need care), or properties not covered here, such as continuity in α, joint convexity in (P, Q) for $\alpha \leq 1$, and the Pinsker-type inequality for Rényi divergences of order $\alpha \leq 1$.
- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2: KL, the chain rule and Jensen's inequality, with exercises; §11.6 for Pinsker's inequality.
- Polyanskiy & Wu, *Information Theory: From Coding to Learning* (2025), ch. 7: f-divergences, their variational forms and the data-processing inequality, at reference depth.
- Tsybakov, *Introduction to Nonparametric Estimation* (2009), §2.4: Pinsker, Bretagnolle–Huber and other inequalities between divergences, used to prove lower bounds for estimation.
- Bishop, *Pattern Recognition and Machine Learning* (2006), §10.1: forward against reverse KL in variational inference, with the mass-covering and mode-seeking pictures.
- Liese & Vajda, "On Divergences and Informations in Statistics and Information Theory" (IEEE Transactions on Information Theory, 2006): f-divergences and their representation through the curves E_γ.
- Nowozin, Cseke & Tomioka, "f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization" (NeurIPS 2016): f-divergences estimated from samples with neural networks.

3.14. Cheat sheet

$r = p/q$ is the likelihood ratio and $L = \ln r$ the log-likelihood ratio, the evidence for P. Gaussian pair: $\mathcal{N}(0, \sigma^2) \parallel \mathcal{N}(s, \sigma^2)$ with $\mu = s/\sigma$. Laplace pair: $\mathrm{Lap}(0, b) \parallel \mathrm{Lap}(s, b)$ with $t = s/b$. Table 3.16 collects every divergence.

Table 3.16.: Every divergence in this chapter, with its closed form for the shifted Gaussian and Laplace pairs.

Divergence	Definition	Gaussian pair	Laplace pair	Key property
KL	$\mathbb{E}_P[L]$	$\mu^2/2$	$t + e^{-t} - 1$	≥ 0; additive; maximum likelihood minimizes it
Rényi D_α	$\frac{1}{\alpha-1} \ln \mathbb{E}_Q[r^\alpha] = \ln \left(\mathbb{E}_P[r^{\alpha-1}]\right)^{1/(\alpha-1)}$	$\alpha\mu^2/2$	Equation 3.19	Log of a power mean of r; non-decreasing in α; additive; \$D_1 = \$ KL
Max-divergence D_∞	$\ln \max r = \ln \max_A \frac{P(A)}{Q(A)}$	∞	t	Worst-case ratio over all events

Divergence	Definition	Gaussian pair	Laplace pair	Key property
TV	$\max_A \lvert P(A) - Q(A)\rvert = \frac{1}{2}\sum \lvert p - q\rvert$	$2\Phi(\mu/2) - 1$	$1 - e^{-t/2}$	Best accuracy $\frac{1}{2} + \frac{1}{2}$TV; \$1 - \$ overlap; a metric
Hockey-stick E_γ	$\max_A [P(A) - \gamma Q(A)] = \mathbb{E}_Q[(r-\gamma)_+]$	$\Phi(\frac{\mu}{2} - \frac{\ln\gamma}{\mu}) - \gamma\Phi(-\frac{\mu}{2} - \frac{\ln\gamma}{\mu})$	$\left(1 - e^{-(t-\ln\gamma)/2}\right)_+$	\$E_1 = \$ TV; 0 for $\gamma \geq e^{D_\infty}$
f-divergence D_f	$\mathbb{E}_Q[f(r)]$, f convex, $f(1) = 0$	—	—	Contains KL, TV, E_γ, χ^2, H^2

Inequalities

- Data processing: $D(PK\|QK) \leq D(P\|Q)$ for every f-divergence and every D_α; equality for sufficient statistics.
- Pinsker: TV $\leq \sqrt{\mathrm{KL}/2}$. Bretagnolle–Huber: TV $\leq \sqrt{1 - e^{-\mathrm{KL}}}$.
- Log-likelihood tail: $Q[L \geq c] \leq \exp(-\alpha c + (\alpha - 1)D_\alpha(P\|Q))$ for every $\alpha > 0$.
- Skew symmetry: $D_\alpha(P\|Q) = \frac{\alpha}{1-\alpha} D_{1-\alpha}(Q\|P)$ for $0 < \alpha < 1$.

Rules of thumb

- KL is the rate at which evidence accumulates; telling two distributions apart takes about 1/KL samples.
- Compute in KL or Rényi, where independent samples add; convert to TV at the end.
- The Rényi order is a dial from typical evidence ($\alpha \to 1$) to the worst case ($\alpha \to \infty$).
- A bounded likelihood ratio (Laplace) gives a finite max-divergence; an unbounded one (Gaussian) gives finite D_α for every finite α but $D_\infty = \infty$.
- Processing averages likelihood ratios together, so it can never increase a divergence.
- Maximum likelihood minimizes forward KL and covers all modes; variational inference minimizes reverse KL and picks one.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

4. Hypothesis testing

4.1. Why this matters

A spam filter sees an email and must say "ham" or "spam". A fraud detector sees a transaction and must say "normal" or "fraud". A medical test reads a biomarker and must say "healthy" or "ill". A monitoring job sees a batch of sensor readings and must say "calibrated" or "drifted". Each one is the same problem: **an observation came from one of two distributions, and you must decide which.** Hypothesis testing is the mathematics of that decision. It starts by naming the two kinds of mistake a decision rule can make and measuring how often it makes each. It then asks what the best possible rule is and how good it can be. Finally, it asks how to summarize and compare real rules, and how to estimate their error rates from a finite amount of data.

The whole chapter rests on one idea. Write $p(x)$ for the probability of observing x under the first distribution and $q(x)$ for its probability under the second. The ratio $q(x)/p(x)$, called the **likelihood ratio**, says how many times more likely the observation is under the second distribution than under the first, and **it is the only thing about the observation that matters.** The Neyman–Pearson lemma, proved below, says that the best rule compares this ratio with a threshold. Moving the threshold trades one kind of mistake for the other, and the best rules at all thresholds together trace a curve that is fixed by the two distributions alone. Every one-number summary of a rule, from the best advantage to the area under the curve (AUC) to the probability that an alarm is right, is read off that curve. Table 4.1 lists the results and where each is used. Later parts ask how well *any* test can distinguish two distributions; the trade-off curve built here is the answer.

Table 4.1.: The results in this chapter and where they are used.

Result	Typical use
Type I/II errors, power	Reading any detector's confusion matrix
Neyman–Pearson lemma	Designing the optimal detector; proving that nothing beats it
Likelihood-ratio test	Score thresholds, sequential and multi-sample tests
Trade-off function $T(P, Q)$	The whole error trade-off between two distributions, as one curve
Gaussian and Laplace trade-off curves	Closed-form reference curves for shifted noise
Best advantage = total variation	A one-number summary of distinguishability

Result	Typical use
Receiver operating characteristic (ROC) curve, AUC, true positive rate at a low false positive rate	Comparing classifiers and detectors honestly
Bayesian view	Turning error rates into posterior probabilities at a given base rate
Confidence intervals for rates	Reporting measured error rates with their uncertainty

The distributions used here are in Probability distributions, the tail bounds and confidence intervals in Concentration inequalities, and total variation and Kullback–Leibler (KL) divergence in Divergences. Each is restated below where it is used.

4.2. Toolkit: four facts every derivation uses

The derivations below reuse four facts. Each is stated and proved once here, with a small running example.

The running example. A spam filter counts suspicious words in an email, capped at 3. The default assumption is that an email is ham; this is the **null hypothesis** H_0, under which the count X has distribution P. The **alternative** H_1 is that the email is spam, under which X has distribution Q. Table 4.2 gives both, and their likelihood ratio $\Lambda(x) = q(x)/p(x)$, with the alternative on top so that large values point to H_1. (Divergences writes the ratio the other way up, as $r(x) = p(x)/q(x)$, and its logarithm as $L(x)$; here Λ is always the plain ratio q/p.) Spam tends to have more suspicious words, so the ratio grows with the count: a count of 3 is four times as likely in spam as in ham, a count of 0 a quarter as likely.

Table 4.2.: Word-count distributions under ham (P) and spam (Q), and their likelihood ratio.

x	0	1	2	3
$p(x)$, ham	0.4	0.3	0.2	0.1
$q(x)$, spam	0.1	0.2	0.3	0.4
$\Lambda(x) = q(x)/p(x)$	0.250	0.667	1.500	4.000

4.2.1. Fact 1 — A test is a function, and its errors are expectations

The simplest decision rule picks a set R of outcomes, its **rejection region**, and says "spam" (rejects H_0) exactly when x lands in R. It is useful to allow a little more: on some outcomes the rule may say "spam" only with some probability, by flipping a biased coin. So a **test** is a function $\phi : \mathcal{X} \to [0, 1]$ on the set $\mathcal{X}$ of possible outcomes, where $\phi(x)$ is the probability of rejecting H_0 after observing x. A **deterministic test** takes only the values 0 and 1, so it is the indicator of its rejection region: $\phi(x) = 1$ on R and 0 off it. A **randomized test** may take values strictly between 0 and 1.

A test makes two kinds of mistake, and both are averages of ϕ. The first is to reject when the email is ham. Given $X = x$, the test rejects with probability $\phi(x)$, so averaging over X drawn from P gives the probability of rejecting under H_0. The second is to keep H_0 when the email is spam: averaging ϕ over X drawn from Q gives the probability of rejecting under H_1, and one minus that is the probability of failing to. Writing $\mathbb{E}_P$ and $\mathbb{E}_Q$ for averages under P and Q, the two error probabilities are

$$\alpha_\phi = \mathbb{E}_P[\phi(X)], \qquad \beta_\phi = 1 - \mathbb{E}_Q[\phi(X)]. \tag{4.1}$$

For a deterministic test this is the familiar fact that the probability of an event is the average of its indicator, $\Pr[X \in R] = \mathbb{E}[\mathbf{1}_R(X)]$: the indicator is 1 on the event and 0 off it, so its average is the probability mass of the event (Concentration inequalities builds Markov's inequality on the same fact).

For example, the test "reject when $X \geq 2$" has $\phi = (0, 0, 1, 1)$ on the outcomes $0, 1, 2, 3$. Its first error is the ham mass on $\{2, 3\}$, $\alpha_\phi = 0.2 + 0.1 = 0.3$. It rejects spam with probability $\mathbb{E}_Q\phi = 0.3 + 0.4 = 0.7$, so its second error is $\beta_\phi = 0.3$.

4.2.2. Fact 2 — Change of measure

The likelihood ratio converts averages under P into averages under Q. The reason is simple: wherever $p(x) > 0$, the spam probability $q(x)$ equals the ham probability $p(x)$ multiplied by $\Lambda(x) = q(x)/p(x)$. So any average under Q can be computed as an average under P in which each outcome is reweighted by Λ. The one catch is outcomes that P never produces, where $p(x) = 0$ and the ratio is undefined; those have to be counted separately. For any function $g \geq 0$,

$$\mathbb{E}_Q[g(X)] = \mathbb{E}_P[g(X)\Lambda(X)] + \mathbb{E}_Q\big[g(X)\mathbf{1}[p(X) = 0]\big]. \tag{4.2}$$

To prove it, split the sum (or integral) $\sum_x g(x)q(x)$ into the points with $p(x) > 0$, where $q(x) = \Lambda(x)p(x)$ turns the sum into $\sum_x g(x)\Lambda(x)p(x) = \mathbb{E}_P[g\Lambda]$, and the points with $p(x) = 0$, which give the second term.

In the spam example $p > 0$ everywhere, so only the first term appears. For "reject when $X = 3$", $\phi = (0, 0, 0, 1)$ and $\mathbb{E}_P[\phi\Lambda] = 0.1 \times 4 = 0.4$, which is exactly $\mathbb{E}_Q\phi$, the spam mass at 3. One consequence is used later: taking $g = 1$, the left side is 1 and the second term is $Q[p = 0]$, the Q-probability of the outcomes P never produces, so $\mathbb{E}_P[\Lambda] = 1 - Q[p = 0] = Q[p > 0] \leq 1$.

4.2.3. Fact 3 — Mixing tests mixes their errors

Suppose you flip a coin that lands heads with probability $\lambda \in [0, 1]$, and run test ϕ_1 on heads and ϕ_2 on tails. On outcome x this rejects with probability $\lambda\phi_1(x) + (1 - \lambda)\phi_2(x)$, so it is the test $\phi = \lambda\phi_1 + (1 - \lambda)\phi_2$, which takes values in $[0, 1]$. Its errors are the same mixture of the two tests' errors,

$$\alpha_\phi = \lambda\alpha_{\phi_1} + (1 - \lambda)\alpha_{\phi_2}, \qquad \beta_\phi = \lambda\beta_{\phi_1} + (1 - \lambda)\beta_{\phi_2}, \tag{4.3}$$

because by Fact 1 both errors are averages of ϕ, and the average of a weighted sum is the weighted sum of the averages. Geometrically, if you plot each test as a point $(\alpha_\phi, \beta_\phi)$, every point on the segment between two achievable points is achievable too. So the set of achievable error pairs is convex, and randomization fills in the segments between deterministic tests.

4.2.4. Fact 4 — The inverse normal distribution function

Write Φ for the standard normal cumulative distribution function (CDF): $\Phi(x)$ is the probability that a standard normal variable is at most x. Write Φ^{-1} for its inverse, so $\Phi^{-1}(u)$ is the point below which a standard normal has probability u. The normal density is symmetric about 0, so the area above x equals the area below $-x$: $1-\Phi(x) = \Phi(-x)$. Applying Φ^{-1} to both sides of this identity, with $x = \Phi^{-1}(\alpha)$, gives $\Phi^{-1}(1-\alpha) = -\Phi^{-1}(\alpha)$. The values used below are $\Phi^{-1}(0.9) = 1.2816$, $\Phi^{-1}(0.95) = 1.6449$, $\Phi^{-1}(0.975) = 1.9600$, $\Phi^{-1}(0.99) = 2.3263$ and $\Phi^{-1}(0.999) = 3.0902$.

Φ has no closed-form inverse, so these values are computed numerically. The script solves $1-\Phi(x) = \alpha$ by bisection, evaluating the upper tail as $\frac{1}{2}\operatorname{erfc}(x/\sqrt{2})$ with the complementary error function erfc. Solving for the tail directly, rather than computing Φ^{-1} at $1-\alpha$, keeps full precision when α is tiny, because $1-\alpha$ then rounds to a number very close to 1 and loses the digits that matter.

4.3. Simple-versus-simple testing

4.3.1. Context

The simplest testing problem has exactly two candidate distributions, each fully specified: **simple** hypotheses, as opposed to composite ones such as "the mean is positive", which allow a whole family of distributions. Most practical detection problems reduce to it once you fix what "normal" and "abnormal" data look like. This section fixes the vocabulary.

4.3.2. Definitions

You observe X and test whether it came from P or from Q:

$$H_0 : X \sim P \qquad \text{against} \qquad H_1 : X \sim Q. \tag{4.4}$$

A test can go wrong in two ways, and each has a name. A **type I error**, or false positive, is rejecting H_0 when it is true: flagging a ham email as spam. Its probability is α_ϕ from Fact 1, also called the **size** of the test or its **false positive rate (FPR)**. A **type II error**, or false negative, is keeping H_0 when H_1 is true: letting spam through. Its probability is β_ϕ, the **false-negative rate**.

The complement of each error is a success rate. The **power** $1-\beta_\phi$ is the probability of detecting H_1 when it holds; in classification it is called the **true positive rate (TPR)**, **recall** or **sensitivity**. The complement of the first error, $1-\alpha_\phi$, is the **specificity**, the probability of correctly passing a ham email. Finally, a test has **level** α if its false positive rate is at most α, $\alpha_\phi \le \alpha$: the level is a budget, the size is what the test actually spends.

The convention is asymmetric on purpose. H_0 is the default ("this email is fine", "this patient is healthy"), and a false alarm is the error you control first. You fix the level, then look for the most powerful test at that level.

4.3.3. The two errors pull in opposite directions

For each rejection region in the spam example, Fact 1 gives the errors directly: α is the ham mass p summed over the region, and the power is the spam mass q summed over it. For the region $\{2, 3\}$, $\alpha = 0.2 + 0.1 = 0.3$ and the power is $0.3 + 0.4 = 0.7$, so $\beta = 0.3$. Table 4.3 lists the threshold tests "reject when $X \geq k$" and two regions that are not thresholds.

Table 4.3.: Errors of deterministic tests in the spam example. The first five are thresholds on the count; the last two are not.

Rejection region	α	β	Power
none	0.00	1.00	0.00
$\{3\}$	0.10	0.60	0.40
$\{2, 3\}$	0.30	0.30	0.70
$\{1, 2, 3\}$	0.60	0.10	0.90
all	1.00	0.00	1.00
$\{0\}$	0.40	0.90	0.10
$\{0, 3\}$	0.50	0.50	0.50

Going down the first five rows, the rejection region grows: each added outcome adds its ham mass to α and its spam mass to the power, so α rises and β falls. You cannot lower both errors by changing the region; you can only trade one for the other.

The last two rows are bad regions, and a useful yardstick shows why. A test that ignores the email and rejects with probability α by flipping a coin has, by Fact 1, size α and power α, so $\beta = 1 - \alpha$. Any test worth using must beat this. The region $\{0\}$ does not: at $\alpha = 0.4$ it has $\beta = 0.9$, worse than the coin flip's $1 - 0.4 = 0.6$, because it rejects on the outcome most typical of ham. The region $\{0, 3\}$ does exactly as well as the coin flip, $\beta = 0.5 = 1 - 0.5$.

4.3.4. Why randomize

The threshold tests reach only $\alpha \in \{0, 0.1, 0.3, 0.6, 1\}$. If your spam policy allows a false-positive rate of 0.25, no deterministic threshold test uses that budget: $\{3\}$ spends only 0.1, and $\{2, 3\}$ overspends at 0.3. A randomized test spends it exactly: reject on $\{3\}$ always and on $\{2\}$ with probability 0.75. This is the same as flipping a coin that picks the region $\{3\}$ with probability 0.25 and the region $\{2, 3\}$ with probability 0.75 (count 3 is then rejected either way, and count 2 with probability 0.75). So it is the mixture $0.25 \cdot \mathbf{1}\{3\} + 0.75 \cdot \mathbf{1}\{2, 3\}$, and by Fact 3 its size is $\alpha = 0.25 \times 0.1 + 0.75 \times 0.3 = 0.25$ exactly. In practice, randomization matters for discrete data, where the threshold can only jump from one outcome to the next. For continuous data such as scores, moving a threshold changes α smoothly, so a threshold can hit any α on its own.

Figure 4.1 shows the two errors for a continuous problem: a biomarker that is $\mathcal{N}(0, 1)$ in healthy patients and $\mathcal{N}(2, 1)$ in ill ones, with the test "reject when $x > t$". The false positive rate α is the area under the healthy curve to the right of t, and the miss rate β is the area under the ill curve to the left of t. Moving t right shrinks the blue area α and grows the orange area β.

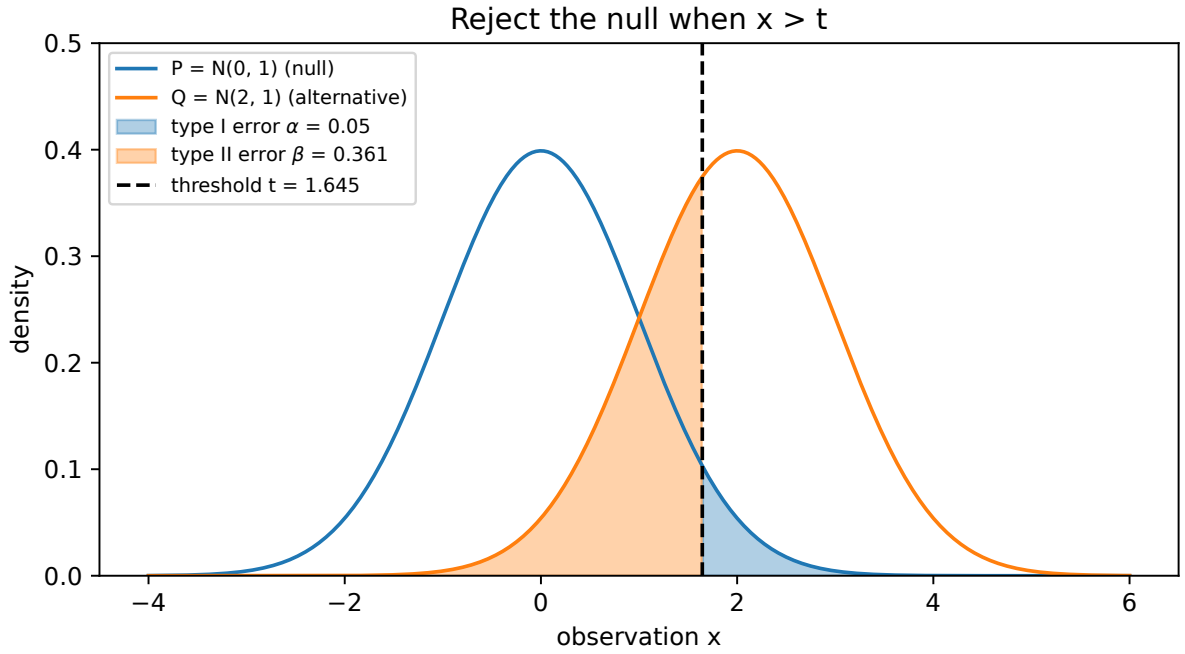


Figure 4.1.: The two errors of the threshold test "reject when $x > t$" for $P = \mathcal{N}(0,1)$ against $Q = \mathcal{N}(2,1)$, with t set so that $\alpha = 0.05$. The blue area under P beyond t is α; the orange area under Q before t is $\beta = 0.361$.

4.3.5. Example: a medical screening test

A screening test flags a patient when the biomarker exceeds $t = 1.645$, the threshold drawn in Figure 4.1. A healthy patient's biomarker is standard normal, so it exceeds t with probability $1 - \Phi(1.645)$, and by Fact 4 that is 0.05 because $\Phi^{-1}(0.95) = 1.6449$. So one healthy patient in 20 is sent for a follow-up. An ill patient's biomarker is centred at 2, so it falls below t with probability $\beta = \Phi(t-2) = \Phi(-0.355) = 0.361$. The test's sensitivity is therefore $1 - \beta = 0.6388$: about two ill patients in three are caught. Lowering the threshold catches more ill patients and alarms more healthy ones. Which point to pick is a policy decision. What the mathematics can say is which points are *achievable*, and that is the subject of Section 4.4.

4.3.6. Check yourself

- A test rejects with probability α regardless of the data. What are its errors? *($\alpha_\phi = \alpha$ and $\beta_\phi = 1 - \alpha$. This "coin flip" test is the baseline every useful test must beat.)*
- Why is the test that rejects on $\{0\}$ in Table 4.3 worse than the coin flip at the same α? *(It rejects where $\Lambda(x) = 0.25 < 1$: on that region ham is four times as likely as spam, so it spends false-positive budget where spam is rare.)*

4.4. The Neyman–Pearson lemma

4.4.1. Context

Among all tests at level α, which one is the most powerful? The space of tests is huge: every function $\phi : \mathcal{X} \to [0, 1]$. The Neyman–Pearson lemma says that you only need to look at one family, thresholds of the likelihood ratio, and that the best test in that family beats every other test.

The intuition is a budget. Each outcome x you decide to reject costs $p(x)$ of false-positive budget and buys $q(x)$ of power. The exchange rate at x is $q(x)/p(x) = \Lambda(x)$, units of power per unit of budget. If you want the most power for a fixed budget, you should spend it on the outcomes with the best exchange rate first, and that means rejecting exactly where $\Lambda(x)$ is above some cut-off.

4.4.2. Statement

Let P and Q have densities (or probability mass functions) p and q. A **likelihood ratio (LR) test** with threshold $\eta \geq 0$ rejects wherever $q(x)$ is more than η times $p(x)$, keeps H_0 wherever it is less, and on the tie set, where $q(x) = \eta\, p(x)$ exactly, may reject with any probability $\gamma(x) \in [0, 1]$:

$$\phi^*(x) = \begin{cases} 1 & \text{if } q(x) > \eta\, p(x), \\ \gamma(x) & \text{if } q(x) = \eta\, p(x), \\ 0 & \text{if } q(x) < \eta\, p(x). \end{cases} \tag{4.5}$$

Writing the condition as $q > \eta p$ rather than $\Lambda > \eta$ also covers points where $p(x) = 0$: there any $q(x) > 0$ counts as above every threshold. The lemma has two parts.

1. **Optimality.** Every test ϕ with $\alpha_\phi \leq \alpha_{\phi^*}$ has $\beta_\phi \geq \beta_{\phi^*}$: no test that spends at most as much false-positive budget misses less.
2. **Existence.** For every $\alpha \in (0, 1]$ there is an LR test with a constant γ and size exactly α.

Together they say that the most powerful level-α test is the LR test of size α. (For $\alpha = 0$ the best test is $\mathbf{1}[p(x) = 0]$: with no budget at all, you can only reject where H_0 is impossible.)

4.4.3. Derivation of optimality

The plan is to compare ϕ^* with any other test ϕ outcome by outcome, weighting each outcome by $q(x) - \eta\, p(x)$, which is positive exactly where the LR test rejects. A pointwise comparison then adds up to a comparison of the errors.

1. **A pointwise inequality.** For every x,

$$(\phi^*(x) - \phi(x))(q(x) - \eta\, p(x)) \geq 0. \tag{4.6}$$

   To see it, check the three cases of Equation 4.5. If $q > \eta p$, then $\phi^* = 1 \geq \phi$, so both factors are ≥ 0. If $q < \eta p$, then $\phi^* = 0 \leq \phi$, so both are ≤ 0. If $q = \eta p$, the second factor is 0. This is the only place the form of ϕ^* is used, and it does not care what γ is.

2. **Sum (or integrate) over x.** A sum of non-negative terms is non-negative. Expanding the product, $\sum_x \phi(x)q(x) = \mathbb{E}_Q[\phi]$ and $\sum_x \phi(x)p(x) = \mathbb{E}_P[\phi]$ by Fact 1, and the same for ϕ^*, so
$$\mathbb{E}_Q[\phi^*] - \mathbb{E}_Q[\phi] \geq \eta(\mathbb{E}_P[\phi^*] - \mathbb{E}_P[\phi]). \tag{4.7}$$
3. **Use the level constraint.** The other test spends no more budget: $\mathbb{E}_P[\phi] = \alpha_\phi \leq \alpha_{\phi^*} = \mathbb{E}_P[\phi^*]$. Together with $\eta \geq 0$, the right side is ≥ 0.
4. **Read off the errors.** So $\mathbb{E}_Q[\phi^*] \geq \mathbb{E}_Q[\phi]$. Both sides are powers, $1 - \beta_{\phi^*} \geq 1 - \beta_\phi$, which is $\beta_\phi \geq \beta_{\phi^*}$.

Step 2 is the budget intuition of the Context written out. η is the exchange rate at the cut-off: every unit of false-positive probability spent at x buys $\Lambda(x)$ units of power. The LR test spends its budget only where the exchange rate is at least η, so no other allocation of the same budget buys more power.

4.4.4. Derivation of existence

The idea is to lower the cut-off on Λ until the outcomes strictly above it use at most α of the budget, while including the outcomes at the cut-off would use at least α. Then randomizing on the outcomes exactly at the cut-off fills the gap. To make this precise, let $X \sim P$ and let $G(c) = \Pr_P[\Lambda(X) > c]$ for $c \geq 0$ be the ham probability of the outcomes whose ratio exceeds c, which is the size of the test that rejects strictly above c. (Points with $p(x) = 0$ have probability 0 under P, so $\Lambda(X)$ is finite.)

1. **How G behaves.** Raising c removes outcomes from $\{\Lambda > c\}$, so G is non-increasing. It is right-continuous, because as $c' \downarrow c$ the sets $\{\Lambda > c'\}$ grow to $\{\Lambda > c\}$, and the probability of a growing sequence of sets tends to the probability of their union. In the same way, as $c' \uparrow c$ the sets $\{\Lambda > c'\}$ shrink to $\{\Lambda \geq c\}$, so the left limit is $G(c^-) = \Pr_P[\Lambda(X) \geq c]$.
2. **Choose the cut-off.** Set $\eta = \inf\{c \geq 0 : G(c) \leq \alpha\}$, the smallest cut-off whose strict rejection region fits in the budget. Right-continuity gives $G(\eta) \leq \alpha$. Just left of η, $G > \alpha$, so $\Pr_P[\Lambda \geq \eta] \geq \alpha$ (when $\eta = 0$ this reads $1 \geq \alpha$).
3. **Randomize on the ties.** The LR test with constant γ rejects $\{\Lambda > \eta\}$ always and $\{\Lambda = \eta\}$ with probability γ, so its size is $G(\eta) + \gamma \Pr_P[\Lambda = \eta]$. If $\Pr_P[\Lambda = \eta] > 0$, pick $\gamma = (\alpha - G(\eta))/\Pr_P[\Lambda = \eta]$ to make the size exactly α. It lies in $[0, 1]$: it is ≥ 0 because $G(\eta) \leq \alpha$, and ≤ 1 because $G(\eta) + \Pr_P[\Lambda = \eta] = \Pr_P[\Lambda \geq \eta] \geq \alpha$, both by step 2. If $\Pr_P[\Lambda = \eta] = 0$, step 2 gives $G(\eta) = \alpha$ and any γ works.

Concrete: the spam example at $\alpha = 0.25$. The distinct ratios, from the largest, are 4 (mass 0.1 under P), 1.5 (mass 0.2), 0.667 and 0.25. For any cut-off c with $1.5 \leq c < 4$, only the count 3 has a ratio above c, so $G(c) = 0.1$, which fits in the budget. Just below 1.5 the count 2 joins, and $G = 0.1 + 0.2 = 0.3$, which does not. So $\eta = 1.5$. The test rejects the count 3 always and must spend the remaining $0.25 - 0.1 = 0.15$ on the tie set $\{2\}$, whose ham mass is 0.2: $\gamma = (0.25 - 0.1)/0.2 = 0.75$. This is the randomized test of Section 4.3. Its power is the spam mass it rejects, $0.4 + 0.75 \times 0.3 = 0.625$, so $\beta = 0.375$.

The script checks the lemma by brute force. It draws 200,000 random tests, $\phi(x)$ uniform on $[0, 1]$ for each x; 11,727 of them have size at most 0.25, and the most powerful of those has power 0.587, below 0.625. Figure 4.2 plots 400 random tests: every one lies on or above the curve traced by the LR tests.

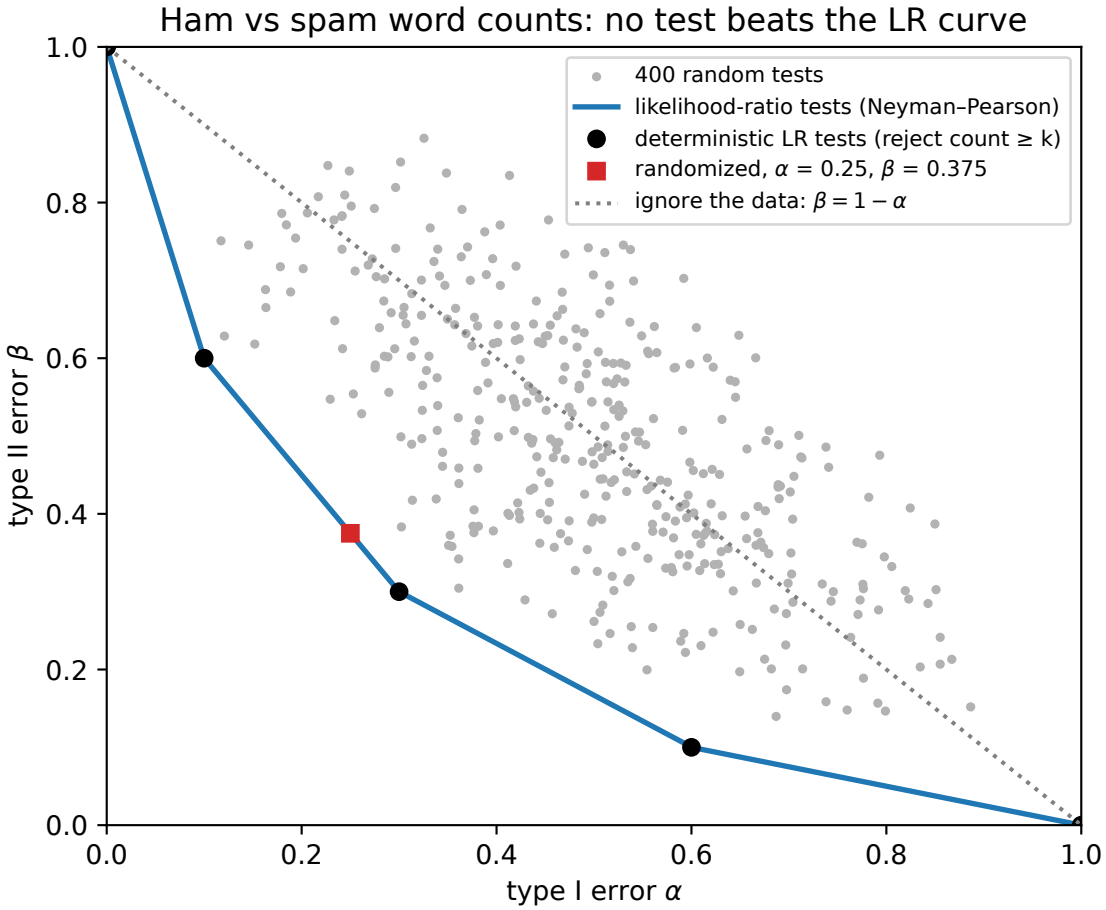


Figure 4.2.: Errors (α, β) of 400 random tests on the spam example (grey), against the curve of likelihood-ratio tests. Black dots are the deterministic thresholds, the red square is the randomized test of size 0.25, and the dotted line is the coin-flip test.

4.4.5. Interpretation

Only the likelihood ratio matters. The optimal test looks at x only through $\Lambda(x)$, so two observations with the same likelihood ratio are interchangeable for testing. Everything else about x is noise.

Monotone ratios give thresholds on x. Often $\Lambda(x)$ increases with some simple statistic $s(x)$, such as the word count in the spam example. Then "the ratio is above η" is the same event as "the statistic is above some t", $\{\Lambda > \eta\} = \{s > t\}$, and the optimal test is a plain threshold on s. The word count, the Gaussian biomarker and the Laplace shift below all work this way. This is why thresholding a well-calibrated score is the right thing to do.

The slope of the curve is $-\eta$. Plot the LR tests as points (α, β). Moving along a segment of this curve means rejecting one more outcome x with a little more probability. That spends $p(x)$ of false-positive budget per unit of extra rejection probability and gains $q(x)$ of power, so β falls by $\Lambda(x)$ for each unit α rises: the slope of the segment is $-\Lambda(x)$. In Figure 4.2 the segments have slopes -4, -1.5, -0.667 and -0.25: the likelihood ratios, largest first. Because the LR test adds outcomes in decreasing order of exchange rate, each segment is less steep than the last, and a curve whose slope keeps increasing is convex.

Many observations. For n independent observations the joint probability of the sample is the product of the individual probabilities, under P and under Q. So the likelihood ratio of the whole sample is $\prod_i \Lambda(x_i)$, and the LR test thresholds its logarithm $\sum_i \ln \Lambda(x_i)$, a sum of independent terms. Sums of independent terms concentrate, so the errors decay exponentially in n. With the false positive rate held fixed, the type II error decays like $e^{-n\,\mathrm{KL}(P\|Q)}$, where the KL divergence $\mathrm{KL}(P\|Q) = \mathbb{E}_P[\ln(p(X)/q(X))]$ is the average log-ratio by which a ham observation favours ham over spam (the Chernoff–Stein lemma; Cover & Thomas, ch. 11, and Exercise 8 in the exercise book).

What the lemma needs. Both hypotheses must be simple. A composite alternative such as "the mean is positive" contains many distributions, each with its own LR test, and one test is optimal against all of them at once only in special cases. The main one is a one-sided test in a family with a monotone likelihood ratio, where every member's LR test is the same threshold on one statistic (Karlin–Rubin; Casella & Berger, §8.3).

4.4.6. Example: why a classifier's score should be a likelihood ratio

A fraud model outputs a score $s(x)$. Suppose the score is calibrated, $s(x) = \Pr[\text{fraud} \mid x]$, with a base rate π of fraud. Bayes' rule says that the odds of fraud after seeing x are the odds before seeing it, multiplied by the likelihood ratio (Section 4.10 derives this): $s/(1-s) = \frac{\pi}{1-\pi}\Lambda(x)$. The odds $s/(1-s)$ grow with s, so s is an increasing function of $\Lambda(x)$. Thresholding s is then an LR test, and Neyman–Pearson says no other use of x can catch more fraud at the same false-alarm rate. If the score is not monotone in Λ, re-ranking the transactions by an estimate of Λ can only help.

4.4.7. Check yourself

- Where did the proof use that ϕ takes values in $[0,1]$? *(In step 1: when $q > \eta p$ it needs $\phi \le 1 = \phi^*$, and when $q < \eta p$ it needs $\phi \ge 0 = \phi^*$.)*
- In the spam example, what is the most powerful test at level $\alpha = 0.3$, and does it need randomization? *(Reject $\{2,3\}$, with power 0.70. No: 0.3 is exactly the P-mass of the outcomes with $\Lambda \ge 1.5$.)*

4.5. Trade-off functions and ROC curves

4.5.1. Context

A single test gives one point (α, β). To compare two *problems* (how hard is it to tell P from Q?) you want the best β at every α at once. That is a curve, and it depends only on P and Q, not on any particular test.

4.5.2. Definition

For each false positive budget α, look at every test that stays within it and record the smallest miss rate any of them achieves. As a function of α, that is the **trade-off function** of P and Q:

$$T(P,Q)(\alpha) = \inf\{\beta_\phi : \alpha_\phi \le \alpha\}, \qquad \alpha \in [0,1]. \tag{4.8}$$

The infimum (inf) is the largest number that is at most every β_ϕ in the set; it is the minimum when the minimum is attained. By the Neyman–Pearson lemma it is attained, by the LR test of size α, so in practice you compute T by sweeping the LR threshold. Write T for $T(P,Q)$ when the pair is clear.

The **ROC curve** of the optimal test plots its TPR against its FPR, that is, power against size: $\alpha \mapsto 1 - T(\alpha)$. It holds the same information as T, turned upside down. A particular *score*, thresholded at every possible value, also traces an ROC curve, and by Neyman–Pearson that curve lies on or below the optimal one.

4.5.3. Statement: four properties

Every trade-off function is

1. **non-increasing,**
2. **at most** $1-\alpha$**:** $T(\alpha) \le 1-\alpha$,
3. **convex,**
4. **continuous** on $[0,1]$.

In words: a bigger budget never hurts, the coin flip is always available, mixing tests fills in segments, and there are no jumps.

Conversely, every function $f : [0,1] \to [0,1]$ with these four properties is $T(P,Q)$ for some pair P, Q, so these four properties are exactly what "a possible error trade-off" means. The standard construction takes P uniform on $[0,1]$, so that any test rejecting an interval of length α has size α, and designs Q so that rejecting the top interval $(1-\alpha, 1]$ misses exactly $f(\alpha)$. Concretely, let Q have CDF $G(x) = f(1-x)$ on $[0,1]$ (so $G(1) = f(0)$, and $G(0) = f(1) = 0$ since property 2 forces $f(1) \le 0$), and put its remaining mass $1 - f(0)$ at a point outside $[0,1]$, where P has no mass. The slope of G at x is minus the slope of f at $1-x$. Because f is convex its slope increases with its argument, so as x grows and $1-x$ shrinks, the slope of G, which is the density of Q on $[0,1]$, is non-decreasing. The likelihood ratio is that density on $[0,1]$ (since $p = 1$ there) and infinite at the outside point, so it is non-decreasing, and the tests "reject the outside point and every $x > 1-\alpha$" are LR tests. Each has size α, and its type II error is the Q-mass it does not reject, $Q[0 \le X \le 1-\alpha] = G(1-\alpha) = f(\alpha)$.

4.5.4. Derivation

1. **Non-increasing.** If $\alpha \le \alpha'$, every test with $\alpha_\phi \le \alpha$ also has $\alpha_\phi \le \alpha'$. The infimum in Equation 4.8 is over a larger set at α', so it is no larger.

2. **At most $1-\alpha$.** The coin-flip test $\phi \equiv \alpha$ is always within budget, with $\alpha_\phi = \alpha$ and $\beta_\phi = 1-\alpha$ (Fact 1), so the infimum is at most $1-\alpha$.

3. **Convex.** Convexity says the curve at a mixed budget is no higher than the same mixture of the curve's values, and Fact 3 provides exactly that mixture. Fix $\alpha_1, \alpha_2, \lambda \in [0,1]$ and a small $\delta > 0$. Pick tests ϕ_i with $\alpha_{\phi_i} \le \alpha_i$ and $\beta_{\phi_i} \le T(\alpha_i) + \delta$, which exist by the definition of the infimum. By Fact 3, the mixture $\lambda\phi_1 + (1-\lambda)\phi_2$ has size at most $\lambda\alpha_1 + (1-\lambda)\alpha_2$ and type II error at most $\lambda T(\alpha_1) + (1-\lambda)T(\alpha_2) + \delta$. So $T(\lambda\alpha_1 + (1-\lambda)\alpha_2) \le \lambda T(\alpha_1) + (1-\lambda)T(\alpha_2) + \delta$. Let $\delta \to 0$.

4. **Continuous.** Inside $(0,1)$ this comes free from convexity: a convex function on an interval is continuous at every interior point, because its chords pin it between two lines on either side of the point (any analysis text, e.g. Rockafellar, *Convex Analysis*, §10). At $\alpha = 1$, the test $\phi \equiv 1$ has $\beta = 0$, so $T(1) = 0$, and property 2 squeezes $0 \le T(\alpha) \le 1 - \alpha \to 0$.

   At $\alpha = 0$ convexity alone allows a jump, so the argument needs Fact 2. First, $T(0) = 1 - Q[p = 0]$: a test of size 0 must have $\phi = 0$ wherever $p > 0$, so the most it can do is reject everywhere $p = 0$. Now take any test with $\alpha_\phi \le \alpha$ and any cut-off $K > 0$, and bound its power. By Fact 2 the power is $\mathbb{E}_Q[\phi] = \mathbb{E}_P[\phi\Lambda] + \mathbb{E}_Q[\phi\,\mathbf{1}[p = 0]]$, and the second term is at most $Q[p = 0]$. Split $\mathbb{E}_P[\phi\Lambda]$ by whether $\Lambda \le K$: where $\Lambda \le K$ it is at most $K\,\mathbb{E}_P[\phi] \le K\alpha$, and where $\Lambda > K$ it is at most $\mathbb{E}_P[\Lambda\,\mathbf{1}[\Lambda > K]]$ because $\phi \le 1$. So

$$1 - \beta_\phi = \mathbb{E}_Q[\phi] \le Q[p = 0] + \mathbb{E}_P[\phi\Lambda] \le Q[p = 0] + K\alpha + \mathbb{E}_P[\Lambda\,\mathbf{1}[\Lambda > K]]. \tag{4.9}$$

   Hence $T(\alpha) \ge T(0) - K\alpha - \mathbb{E}_P[\Lambda\,\mathbf{1}[\Lambda > K]]$. Since $\mathbb{E}_P[\Lambda] \le 1$ is finite (Fact 2), the part of that average coming from $\Lambda > K$ tends to 0 as $K \to \infty$. So choose K large to make the last term small, then α small to make $K\alpha$ small: $T(\alpha) \to T(0)$.

4.5.5. Interpretation

The two extremes of the curve are the two extremes of difficulty. $T(\alpha) = 1 - \alpha$ for all α means no test beats a coin flip, which happens exactly when $P = Q$. $T \equiv 0$ means some test makes no errors at all, which happens when P and Q live on disjoint sets.

Between the extremes, the further T bends below the diagonal, the easier the problem. A pair (P, Q) is **harder to distinguish** than (P', Q') if $T(P, Q) \geq T(P', Q')$ everywhere: at every budget, the best test misses more. This is only a partial order, because two curves can cross, and then neither problem is uniformly harder.

The curve need not start at 1. If Q puts mass where P has none, a test catches that mass with no false alarms, so $T(0) = 1 - Q[p = 0]$ is below 1 (Exercise 6 in the exercise book). And the curve depends on which distribution is the null: swapping the roles of P and Q reflects the curve across the diagonal, so $T(Q, P)$ is the inverse function of $T(P, Q)$ when $T(P, Q)$ is strictly decreasing (Exercise 2 in the exercise book).

4.5.6. Check yourself

- Why can a trade-off function never be concave and below the diagonal on part of $[0, 1]$ and above it elsewhere? *(Property 2 forbids it from ever being above the diagonal $1 - \alpha$.)*
- A deterministic score gives a staircase ROC on a finite test set. Why can you always join adjacent points with straight segments? *(Fact 3: randomizing between the two thresholds achieves every point on the segment.)*

4.6. The Gaussian trade-off curve

4.6.1. Context

The most common pair in practice is a shift of a normal distribution: a score, a biomarker or a sample mean that is normal under both hypotheses with the same spread but different centres. Its trade-off curve has a closed form, worked out here in full.

4.6.2. Statement

For a unit-variance normal under H_0 and the same normal shifted right by μ under H_1, the best test is a threshold on x. At budget α the threshold sits where P has upper tail α, which is $\Phi^{-1}(1 - \alpha)$, and the miss rate is the probability that Q, centred μ higher, falls below it. So for $P = \mathcal{N}(0, 1)$ and $Q = \mathcal{N}(\mu, 1)$ with $\mu \geq 0$,

$$T(P, Q)(\alpha) = \Phi\big(\Phi^{-1}(1 - \alpha) - \mu\big). \tag{4.10}$$

More generally, $\mathcal{N}(m_0, \sigma^2)$ against $\mathcal{N}(m_1, \sigma^2)$ has this curve with $\mu = |m_1 - m_0|/\sigma$. The reason is that standardizing the observation, $x \mapsto (x - m_0)/\sigma$, is a one-to-one relabelling of the outcomes, so every test on the original scale is a test on the new scale with the same two errors, and on the new scale the pair is $\mathcal{N}(0, 1)$ against $\mathcal{N}(\pm\mu, 1)$.

4.6.3. Derivation

The plan is to compute the likelihood ratio, see that it increases with x, and then read both errors of the threshold test off normal tails.

1. **The likelihood ratio.** With the standard normal density $\varphi(x) = e^{-x^2/2}/\sqrt{2\pi}$, the density of Q is $\varphi(x-\mu)$. The constants cancel in the ratio, and expanding the square, $x^2-(x-\mu)^2 = 2\mu x - \mu^2$:
$$\Lambda(x) = \frac{\varphi(x-\mu)}{\varphi(x)} = \exp\Big(\frac{x^2-(x-\mu)^2}{2}\Big) = \exp\Big(\mu x - \frac{\mu^2}{2}\Big). \tag{4.11}$$
2. **The LR test is a threshold on x.** For $\mu > 0$, Λ is strictly increasing in x, so $\Lambda(x) > \eta$ exactly when $x > t$, where $t = (\ln\eta + \mu^2/2)/\mu$ solves $\mu t - \mu^2/2 = \ln\eta$. Ties $\Lambda(x) = \eta$ happen only at the single point $x = t$, which has probability 0, so no randomization is needed. By Neyman–Pearson, the tests $\phi_t(x) = \mathbf{1}[x > t]$ are optimal.
3. **Size fixes the threshold.** $\alpha = \Pr_P[X > t] = 1 - \Phi(t)$, so $t = \Phi^{-1}(1-\alpha)$.
4. **Type II error.** Under Q, $X - \mu \sim \mathcal{N}(0,1)$, so the test misses with probability $\beta = \Pr_Q[X \le t] = \Pr[X - \mu \le t - \mu] = \Phi(t-\mu)$.
5. **Substitute** step 3 into step 4 to get Equation 4.10.

Checks. At $\mu = 0$ the two hypotheses coincide, and indeed $\Phi(\Phi^{-1}(1-\alpha)) = 1-\alpha$, the diagonal. As $\mu \to \infty$, $T \to 0$ for every $\alpha > 0$. And T is symmetric about the diagonal, $T(T(\alpha)) = \alpha$ (Exercise 1 in the exercise book). In short: Fact 4 turns $\Phi^{-1}(1-T(\alpha))$ into $-\Phi^{-1}(T(\alpha)) = -(\Phi^{-1}(1-\alpha)-\mu)$, so $T(T(\alpha)) = \Phi(-\Phi^{-1}(1-\alpha)) = 1-(1-\alpha) = \alpha$.

4.6.4. Interpretation

One number describes the problem. The curve depends on the two normals only through μ, the distance between the means in units of the standard deviation. Figure 4.3 shows the family. Crosses mark simulated thresholds at $\mu = 1$, with 20,000 draws per point.

Near $\alpha = 0$ the curve is vertical. The slope of the curve at the point with threshold t is $-\Lambda(t)$, as in the slope rule of Section 4.4. Here it follows from the two errors: moving the threshold from t to $t + dt$ lowers α by $p(t)\,dt$ and raises β by $q(t)\,dt$, so $T'(\alpha) = d\beta/d\alpha = -q(t)/p(t) = -\Lambda(t)$. Small α means a large threshold, and $\Lambda(t) = e^{\mu t - \mu^2/2} \to \infty$ as $t \to \infty$. So the first sliver of false-positive budget buys power at an unbounded exchange rate. The Laplace pair of Section 4.7 behaves differently.

Averaging n observations. Take n independent draws from $\mathcal{N}(0,\sigma^2)$ against $\mathcal{N}(\delta,\sigma^2)$. Step 1, with the spread σ put back, gives each draw a log likelihood ratio $(\delta x_i - \delta^2/2)/\sigma^2$, and the log ratios of independent draws add up to $\sum_i(\delta x_i - \delta^2/2)/\sigma^2$. This depends on the data only through $\sum_i x_i$, that is, through the sample mean $\bar{x}$, so the best test thresholds $\bar{x}$. The sample mean is $\mathcal{N}(0,\sigma^2/n)$ against $\mathcal{N}(\delta,\sigma^2/n)$, a shift of δ with standard deviation $\sigma/\sqrt{n}$. So the curve is Equation 4.10 with
$$\mu = \frac{\delta\sqrt{n}}{\sigma}. \tag{4.12}$$
Distinguishability grows like $\sqrt{n}$, the same rate at which a sample mean concentrates.

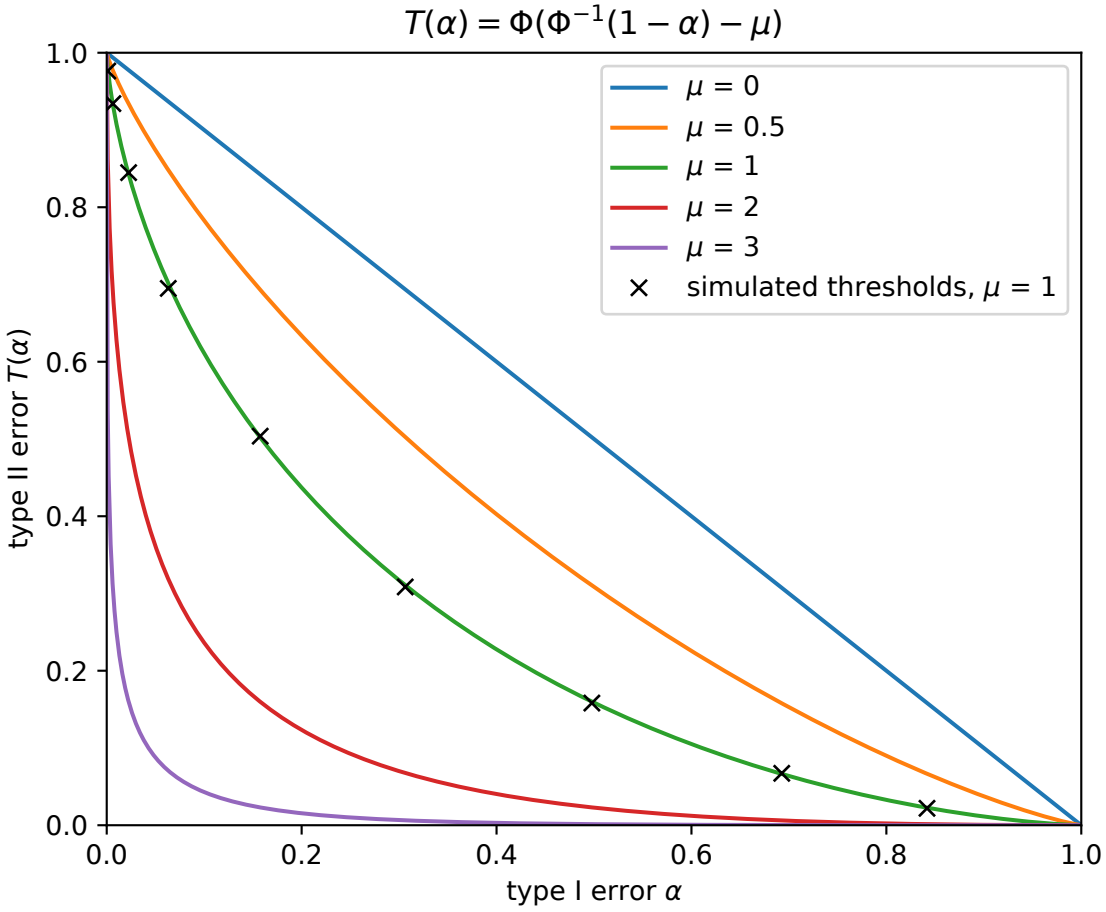


Figure 4.3.: The Gaussian trade-off curve $\Phi(\Phi^{-1}(1-\alpha) - \mu)$ for $\mu = 0, 0.5, 1, 2, 3$. Crosses are simulated errors of threshold tests at $\mu = 1$.

4.6.5. Example: detecting sensor drift

A monitoring job averages $n = 100$ readings from a sensor with noise σ and must detect a drift of $\delta = 0.2\sigma$ in the mean. Then $\mu = \delta\sqrt{n}/\sigma = 0.2 \times 10 = 2$. Table 4.4 evaluates Equation 4.10 at this shift and at $\mu = 1$, for two levels. Each row sets the threshold from the level by Fact 4 and subtracts μ; for example, at $\mu = 2$ and $\alpha = 0.05$, $\beta = \Phi(1.6449 - 2) = \Phi(-0.3551) = 0.3612$.

Table 4.4.: The Gaussian trade-off curve at two shifts and two levels.

μ	α	Threshold $\Phi^{-1}(1-\alpha)$	$\beta = T(\alpha)$	Power
1	0.05	1.6449	0.7405	0.2595
1	10^{-3}	3.0902	0.9817	0.0183
2	0.05	1.6449	0.3612	0.6388
2	10^{-3}	3.0902	0.8622	0.1378

At a 5% false-alarm rate the job catches the drift with probability 0.6388 (Table 4.4). A simulation with 1,000,000 draws per hypothesis gives $\hat{\alpha} = 0.0503$ and $\hat{\beta} = 0.3617$. If false alarms must be rare, $\alpha = 10^{-3}$, the threshold moves out to 3.0902 and the power drops to $1 - \Phi(3.0902 - 2) = 0.1378$: a stricter threshold costs a lot of power when μ is moderate.

How many readings? Turn the question around: what shift μ, and so what n, gives power at least $1 - \beta'$ at level α? The power target is $T(\alpha) \le \beta'$, that is, $\Phi(\Phi^{-1}(1-\alpha) - \mu) \le \beta'$. Applying the increasing function Φ^{-1} to both sides gives $\Phi^{-1}(1-\alpha) - \mu \le \Phi^{-1}(\beta')$, and by Fact 4, $\Phi^{-1}(\beta') = -\Phi^{-1}(1-\beta')$. So the condition is $\mu \ge \Phi^{-1}(1-\alpha) + \Phi^{-1}(1-\beta')$. Substituting $\mu = \delta\sqrt{n}/\sigma$ and solving for n,

$$n \ge \left(\frac{\Phi^{-1}(1-\alpha) + \Phi^{-1}(1-\beta')}{\delta/\sigma} \right)^2 . \tag{4.13}$$

For power 0.9 at $\alpha = 0.01$ and $\delta/\sigma = 0.25$, the two quantiles are $\Phi^{-1}(0.99) = 2.3263$ and $\Phi^{-1}(0.9) = 1.2816$, so $n \ge ((2.3263 + 1.2816)/0.25)^2 = 208.3$, and you need $n = 209$ readings.

4.6.6. Check yourself

- Two classifiers produce normal scores with equal spread. Classifier A separates the class means by 2 standard deviations, classifier B by 1. How many independent B-scores must you average to match one A-score? *(Four: μ grows like $\sqrt{n}$, and $1 \times \sqrt{4} = 2$.)*
- Why does step 2 need $\mu > 0$, and what changes for $\mu < 0$? *(For $\mu < 0$ the ratio is decreasing in x, so the optimal test rejects for small x. The curve is the same with $|\mu|$.)*

4.7. The Laplace trade-off curve

4.7.1. Context

Replace the normal by a Laplace distribution: the same shift problem, but with exponential tails. The Laplace distribution $\mathrm{Lap}(m, b)$ with centre m and scale b has density $\frac{1}{2b}e^{-|x-m|/b}$. The

trade-off curve changes shape at the ends, and the reason is the likelihood ratio. Shifting a Laplace density changes its exponent $-|x-m|/b$ by at most s/b for a shift of s (moving the centre by s changes the distance $|x-m|$ by at most s), so the shifted density is within a factor $e^{s/b}$ of the original everywhere (Probability distributions). A shifted normal is not: its likelihood ratio, computed in Section 4.6, grows without bound.

4.7.2. Statement

Because the likelihood ratio is capped, no test can buy power faster than the cap allows, and the curve's two ends are straight lines with the cap as their slope. In between, where the ratio does vary, the curve bends. For $P = \mathrm{Lap}(0, b)$ and $Q = \mathrm{Lap}(s, b)$ with $s > 0$, write $\kappa = s/b$ for the shift in units of the scale. Then

$$T(P,Q)(\alpha) = \begin{cases} 1 - e^{\kappa}\alpha, & \alpha < \frac{1}{2}e^{-\kappa}, \\ \dfrac{e^{-\kappa}}{4\alpha}, & \frac{1}{2}e^{-\kappa} \le \alpha \le \frac{1}{2}, \\ e^{-\kappa}(1-\alpha), & \alpha > \frac{1}{2}. \end{cases} \tag{4.14}$$

The three pieces correspond to a threshold above the spam-side centre s, between the two centres, and below the ham-side centre 0.

4.7.3. Derivation

Rescale x by b, which changes no test's errors (as for the normal), so that $b = 1$ and $s = \kappa$. The CDF of $\mathrm{Lap}(0, 1)$ is $F(x) = \frac{1}{2}e^{x}$ for $x < 0$ and $1 - \frac{1}{2}e^{-x}$ for $x \ge 0$, obtained by integrating the density $\frac{1}{2}e^{-|x|}$ on each side of 0. The plan is the same as for the normal: find the likelihood ratio, see that thresholds on x are LR tests, and read the errors off F, now in three cases.

1. **The likelihood ratio.** The normalizing constants cancel, so $\ln \Lambda(x) = -|x-\kappa| + |x| = |x| - |x-\kappa|$. Removing the absolute values on each side of 0 and κ gives

$$\ln \Lambda(x) = \begin{cases} -\kappa, & x \le 0, \\ 2x - \kappa, & 0 < x < \kappa, \\ \kappa, & x \ge \kappa. \end{cases} \tag{4.15}$$

   It is non-decreasing in x and clipped to $[-\kappa, \kappa]$ (Figure 4.4, left).
2. **Thresholds on x are LR tests.** Because Λ is non-decreasing, the region $\{x > t\}$ always consists of every point with $\Lambda > \eta$, for $\eta = \Lambda(t)$, plus part of the tie set $\{\Lambda = \eta\}$. When t falls in a flat piece, the tie set is a whole half-line with positive probability, and rejecting only the part of it with $x > t$ is one allowed choice of $\gamma(x)$ in Equation 4.5. So every threshold test is optimal, and its errors $\alpha = 1 - F(t)$ and $\beta = \Pr_Q[X \le t] = F(t - \kappa)$ trace T.
3. **Case $t \ge \kappa$.** Both t and $t - \kappa$ are ≥ 0. Then $\alpha = \frac{1}{2}e^{-t} \le \frac{1}{2}e^{-\kappa}$, and $\beta = 1 - \frac{1}{2}e^{-(t-\kappa)} = 1 - e^{\kappa} \cdot \frac{1}{2}e^{-t} = 1 - e^{\kappa}\alpha$.
4. **Case $0 \le t < \kappa$.** Now $t \ge 0$ but $t - \kappa < 0$. Then $\alpha = \frac{1}{2}e^{-t} \in (\frac{1}{2}e^{-\kappa}, \frac{1}{2}]$, and $\beta = \frac{1}{2}e^{t-\kappa}$. Substituting $e^{t} = 1/(2\alpha)$ from the first equation: $\beta = \frac{1}{2}e^{-\kappa}/(2\alpha) = e^{-\kappa}/(4\alpha)$.
5. **Case $t < 0$.** Both are negative. Then $\alpha = 1 - \frac{1}{2}e^{t} > \frac{1}{2}$, so $\frac{1}{2}e^{t} = 1 - \alpha$, and $\beta = \frac{1}{2}e^{t-\kappa} = e^{-\kappa}(1-\alpha)$.

Checks. The pieces agree at the joins: at $\alpha = \frac{1}{2}e^{-\kappa}$ both give $\frac{1}{2}$, and at $\alpha = \frac{1}{2}$ both give $\frac{1}{2}e^{-\kappa}$. At $\kappa = 0$ all three pieces are $1 - \alpha$, the diagonal of two identical distributions.

4.7.4. Interpretation

The ends are straight lines. Their slopes are $-e^{\kappa}$ and $-e^{-\kappa}$. That is the slope rule of Section 4.4 at work: the slope of T is $-\Lambda$ at the threshold, and on the two flat pieces of the likelihood ratio Λ is the constant e^{κ} or $e^{-\kappa}$. Because $\Lambda \le e^{\kappa}$ everywhere, no test can buy power faster than e^{κ} per unit of false-positive rate. Reading the first piece as power: for small α, the best power is exactly $1 - T(\alpha) = e^{\kappa}\alpha$.

Gaussian against Laplace. To compare the two shapes fairly, match the two problems by their best advantage, the largest gap TPR $-$ FPR any test achieves. Section 4.8 shows that it equals $1 - e^{-\kappa/2}$ for the Laplace pair and $2\Phi(\mu/2) - 1$ for the Gaussian pair. At $\kappa = 1$ the Laplace value is $1 - e^{-1/2} = 0.3935$, and solving $2\Phi(\mu/2) - 1 = 0.3935$ gives a Gaussian pair with $\mu = 1.0301$. In the bulk the two curves nearly coincide. At small α they do not, as Table 4.5 shows: the Laplace power is $e\alpha$, and the Gaussian power is $1 - \Phi(\Phi^{-1}(1 - \alpha) - 1.0301)$.

Table 4.5.: Best power at small false-positive rates for two pairs with the same total variation.

α	Laplace power, $\kappa = 1$	Gaussian power, $\mu = 1.0301$
10^{-3}	2.718×10^{-3}	1.969×10^{-2}
10^{-6}	2.718×10^{-6}	9.829×10^{-5}

At $\alpha = 10^{-6}$ the Gaussian test is 36 times more powerful, $9.829 \times 10^{-5} / 2.718 \times 10^{-6} = 36.2$ (Table 4.5). Its unbounded likelihood ratio lets a test find rare observations that are overwhelmingly more likely under Q; the clipped Laplace ratio forbids that. Figure 4.4 plots both on log axes.

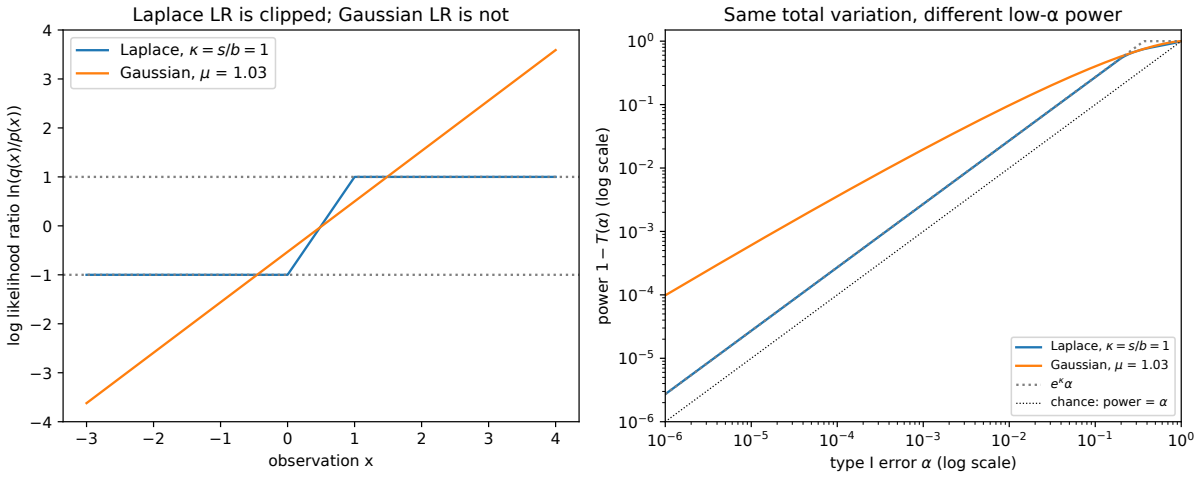


Figure 4.4.: Left: the log likelihood ratio of $\mathrm{Lap}(1, 1)$ against $\mathrm{Lap}(0, 1)$ is clipped at ± 1, while the Gaussian one with the same total variation is a straight line. Right: best power $1 - T(\alpha)$ on log axes. The Laplace curve follows $e\alpha$ for small α; the Gaussian curve rises far above it.

4.7.5. Example: absolute-error residuals

A regression model with absolute-error loss has residuals that are roughly Laplace with scale b. You suspect a batch of inputs carries a systematic offset of $s = b$, so $\kappa = 1$. The first breakpoint of Equation 4.14 is $\frac{1}{2}e^{-1} = 0.1839$ and the second is $\frac{1}{2}$, so each level falls on a different piece. At $\alpha = 0.05$, below the first breakpoint, the first piece gives $\beta = 1 - e \times 0.05 = 0.8641$, power 0.1359. At $\alpha = 0.25$, between the breakpoints, the middle piece gives $\beta = e^{-1}/(4 \times 0.25) = e^{-1} = 0.3679$, power 0.6321. At $\alpha = 0.75$, above $\frac{1}{2}$, the last piece gives $\beta = e^{-1} \times 0.25 = 0.0920$.

A simulation of the threshold test at $\alpha = 0.25$ with 1,000,000 draws per hypothesis gives $\hat{\alpha} = 0.2498$ and $\hat{\beta} = 0.3685$.

4.7.6. Check yourself

- On which piece of Equation 4.14 does the threshold t lie between the two medians 0 and s? *(The middle piece, $\frac{1}{2}e^{-\kappa} \le \alpha \le \frac{1}{2}$, where Λ is strictly increasing.)*
- Why can a test on the Laplace pair at level $\alpha = 10^{-3}$ never have power above $e^{\kappa} \times 10^{-3}$? *(Fact 2: power $= \mathbb{E}_P[\phi\Lambda] \le e^{\kappa}\,\mathbb{E}_P[\phi] \le e^{\kappa}\alpha$, since $\Lambda \le e^{\kappa}$ everywhere.)*

4.8. Best advantage equals total variation

4.8.1. Context

Sometimes you want one number for "how distinguishable are P and Q?". A natural one is the **advantage** of a test, $1 - \alpha_\phi - \beta_\phi = \mathrm{TPR} - \mathrm{FPR}$: how much more often it fires under Q than under P. A test that ignores the data, such as the coin flip, fires equally often under both and has advantage 0. (In medicine, $\mathrm{TPR} - \mathrm{FPR}$ at a threshold is Youden's index.) Its maximum over all tests turns out to be the total variation (TV) distance of Divergences. That distance is half the total absolute difference between the two probability tables, $\frac{1}{2}\sum_x |q(x) - p(x)|$, or equivalently the largest difference $Q(A) - P(A)$ in the probability the two distributions give to any one event A.

4.8.2. Statement

The advantage of a test is a sum over outcomes of $\phi(x)$ times $q(x) - p(x)$, so the best test fires exactly on the outcomes where q exceeds p, and its advantage is the total excess of q over p. That excess is the total variation distance:

$$\max_{\phi} (1 - \alpha_\phi - \beta_\phi) = \mathrm{TV}(P, Q) = \sup_{A} \left(Q(A) - P(A)\right) = \frac{1}{2}\sum_{x} |q(x) - p(x)|, \tag{4.16}$$

with the sum replaced by an integral for densities. The maximum is attained by the LR test with $\eta = 1$: reject where $q > p$.

4.8.3. Derivation

1. **Advantage as one sum.** By Fact 1, the power is $\mathbb{E}_Q\phi$ and the false positive rate is $\mathbb{E}_P\phi$, so $1-\alpha_\phi-\beta_\phi=\mathbb{E}_Q\phi-\mathbb{E}_P\phi=\sum_x\phi(x)(q(x)-p(x))$.
2. **Maximize term by term.** Each $\phi(x)$ appears in one term only, so each can be chosen separately. A term is largest with $\phi(x)=1$ where $q(x)>p(x)$ and $\phi(x)=0$ where $q(x)<p(x)$. So the maximum is $\sum_{x:q>p}(q(x)-p(x))=Q(A^*)-P(A^*)$ with $A^*=\{q>p\}$. A deterministic $\phi=\mathbf{1}_A$ gives $Q(A)-P(A)$, and the best of these is A^*, so the maximum is also $\sup_A(Q(A)-P(A))$.
3. **Rewrite as half the L^1 distance.** Both tables sum to 1, so $\sum_x(q-p)=1-1=0$, and the positive and negative parts of $q-p$ are equal: $\sum_{q>p}(q-p)=\sum_{q<p}(p-q)$. Their sum is $\sum_x|q-p|$, so each is half of it.

On the trade-off curve, the advantage at level α is $1-\alpha-T(\alpha)$: the vertical gap between the diagonal and the curve. Its derivative is $-1-T'(\alpha)$, which is zero where the slope of T is -1. By the slope rule of Section 4.4 the slope is $-\Lambda$ at the threshold, so this is the point where the LR threshold is $\eta=1$ (Figure 4.5).

4.8.4. Example: three pairs

Spam words. The differences $|q-p|$ in Table 4.2 are 0.3, 0.1, 0.1 and 0.3, so TV $=\frac{1}{2}(0.3+0.1+0.1+0.3)=0.40$. Spam is more likely than ham at the counts 2 and 3, so the best test rejects $\{2,3\}$, and indeed its advantage is $1-0.3-0.3=0.4$ (Table 4.3).

Gaussian. The best test rejects where $q>p$, that is, where $\Lambda(x)=e^{\mu x-\mu^2/2}>1$, which is $x>\mu/2$: the point halfway between the two means, where the two bell curves cross. That test has $\alpha=1-\Phi(\mu/2)$ and $\beta=\Phi(\mu/2-\mu)=\Phi(-\mu/2)=1-\Phi(\mu/2)$, so TV $=1-2(1-\Phi(\mu/2))=2\Phi(\mu/2)-1$. At $\mu=1$ this is $2\Phi(0.5)-1=0.3829$, reached at $\alpha=1-\Phi(0.5)=0.3085$. Maximizing $1-\alpha-T(\alpha)$ over a grid of 100,001 values of α gives the same 0.3829 at $\alpha=0.3085$.

Laplace. Here it is easiest to maximize the gap on the curve directly. On the middle piece of Equation 4.14 the gap is $1-\alpha-\frac{e^{-\kappa}}{4\alpha}$, and setting its derivative to zero, $-1+\frac{e^{-\kappa}}{4\alpha^2}=0$, gives $\alpha=\frac{1}{2}e^{-\kappa/2}$. There $\frac{e^{-\kappa}}{4\alpha}=\frac{1}{2}e^{-\kappa/2}$ as well, so TV $=1-e^{-\kappa/2}$. At $\kappa=1$ that is 0.3935, at $\alpha=0.3033$; the grid search agrees.

4.8.5. Interpretation

TV is the best that a summary using a single threshold, and weighting the two errors equally, can say. It measures the problem at one point of the curve, where $\eta=1$, and says nothing about the ends. Table 4.5 is the warning: two pairs with the same TV, 0.3935, differ by a factor of 36 in power at $\alpha=10^{-6}$. If the regime you care about is "very few false alarms", TV is the wrong summary, and the whole curve is the right one.

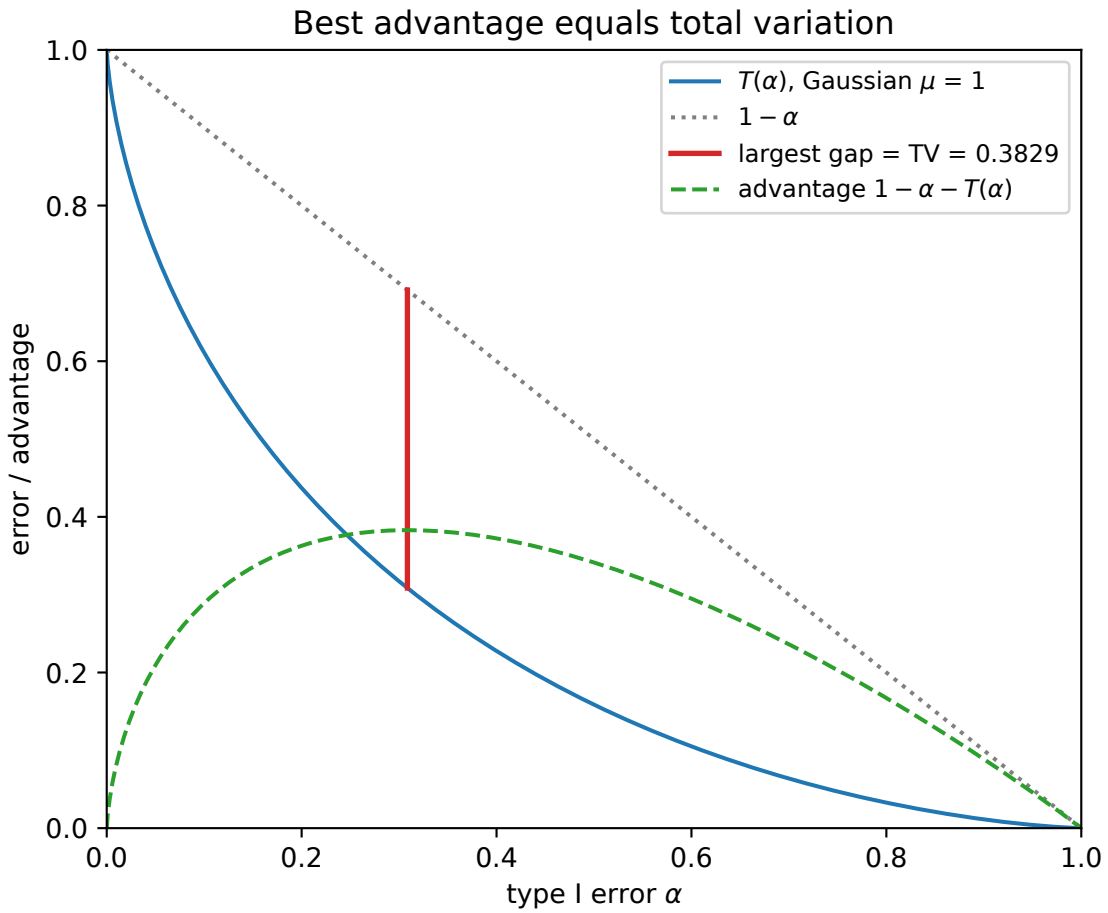


Figure 4.5.: The Gaussian trade-off curve at $\mu = 1$, the diagonal $1 - \alpha$, and the advantage $1 - \alpha - T(\alpha)$ (dashed). The red segment is the largest gap, at $\alpha = 0.3085$, and its length is the total variation 0.3829.

4.8.6. Check yourself

- Why is the advantage of any test at most 1, and when is it exactly 1? *(It is at most $Q(A^*) - P(A^*) \le 1$, with equality only if $Q(A^*) = 1$ and $P(A^*) = 0$: the distributions have disjoint supports.)*
- Is the coin-flip test's advantage ever negative? Is any test's? *(The coin flip has advantage 0. A test can have negative advantage, like rejecting $\{0\}$ in the spam example, but then its complement $1 - \phi$ has positive advantage.)*

4.9. ROC summaries: why AUC hides the low-FPR regime

4.9.1. Context

Real detectors are compared on held-out data by their ROC curves, the plot of TPR against FPR as the detector's threshold sweeps from strict to lenient. The curve is usually summarized by the **AUC**, the area under it. In many applications only a small corner of the curve matters: a fraud team can review a tiny fraction of transactions, a screening programme can afford few false alarms, a spam filter must almost never discard real mail. This section shows that two detectors with the same AUC can differ by an order of magnitude in that corner.

4.9.2. Statement: AUC is a ranking probability

The area under the ROC curve has a direct meaning: it is the probability that the detector scores a randomly chosen positive above a randomly chosen negative. For a score S that is $S_- \sim P$ on negatives and $S_+ \sim Q$ on positives (continuous and independent),

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}\, d(\mathrm{FPR}) = \Pr[S_+ > S_-]. \tag{4.17}$$

4.9.3. Derivation

The idea is to parametrize the curve by the threshold and recognize the area integral as an average over a negative's score.

1. **Parametrize the curve by the threshold.** The test "positive when $S > t$" has $\mathrm{FPR}(t) = \Pr[S_- > t]$ and $\mathrm{TPR}(t) = \Pr[S_+ > t]$. As t goes from $+\infty$ to $-\infty$, the FPR goes from 0 to 1.
2. **Change variables.** $\mathrm{FPR}(t) = \int_t^\infty p_-(s)\, ds$, where p_- is the density of S_-, so $d\,\mathrm{FPR}(t) = -p_-(t)\, dt$. Substituting, and flipping the limits to absorb the minus sign,

$$\mathrm{AUC} = \int_{+\infty}^{-\infty} \Pr[S_+ > t]\,(-p_-(t))\, dt = \int_{-\infty}^{\infty} \Pr[S_+ > t]\, p_-(t)\, dt. \tag{4.18}$$

3. **Read it as an average.** Fix a negative's score t. The chance that an independent positive beats it is $\Pr[S_+ > t]$. The last integral averages that chance over t drawn from the negatives' density, so it is $\mathbb{E}[\Pr[S_+ > S_- \mid S_-]] = \Pr[S_+ > S_-]$: the probability that a random positive outscores a random negative.

The binormal model. A common model has normal scores in both classes, $S_- \sim \mathcal{N}(0,1)$ and $S_+ \sim \mathcal{N}(m, \sigma^2)$. The difference of two independent normals is normal, with the means subtracted and the variances added, so $S_+ - S_- \sim \mathcal{N}(m, 1+\sigma^2)$. The AUC is the probability that this difference is positive, and the ROC curve follows by setting the threshold from the FPR:

$$\mathrm{AUC} = \Phi(m/\sqrt{1+\sigma^2}), \qquad \mathrm{TPR}(\mathrm{FPR}) = 1 - \Phi\Big(\frac{\Phi^{-1}(1-\mathrm{FPR}) - m}{\sigma}\Big). \tag{4.19}$$

The second formula repeats steps 3 and 4 of the Gaussian derivation in Section 4.6: the threshold is $t = \Phi^{-1}(1-\mathrm{FPR})$, and a positive exceeds it when $(S_+ - m)/\sigma > (t-m)/\sigma$, now with spread σ under Q.

4.9.4. Example: same AUC, different detectors

Detector A has positives that score $\mathcal{N}(1,1)$, so $m = 1$, $\sigma = 1$ and $\mathrm{AUC} = \Phi(1/\sqrt{2}) = 0.7602$. Detector B has positives that score $\mathcal{N}(m, 2^2)$, more spread out, with m chosen to give the same AUC: $m/\sqrt{5} = 1/\sqrt{2}$ gives $m = \sqrt{5/2} = 1.5811$ and the same AUC, 0.7602. Table 4.6 compares them using the binormal ROC formula. For example, at FPR 10^{-3} the threshold is 3.0902; A's TPR is $1 - \Phi(3.0902 - 1) = 0.0183$, and B's is $1 - \Phi((3.0902 - 1.5811)/2) = 1 - \Phi(0.7546) = 0.2253$.

Table 4.6.: Two binormal detectors with equal AUC. The simulated AUC counts correctly ranked pairs among 1,000,000.

Detector	AUC (formula / simulated)	TPR at FPR = 0.1	TPR at FPR = 10^{-3}
A, $\mathcal{N}(1,1)$	0.7602 / 0.7605	0.3891	0.0183
B, $\mathcal{N}(1.5811, 2^2)$	0.7602 / 0.7601	0.5595	0.2253

At an FPR of 10^{-3}, detector B catches 12 times as many positives, $0.2253/0.0183 = 12.3$ (Table 4.6). On linear axes the two ROC curves cross and look like comparable detectors; on log axes the gap at low FPR is obvious (Figure 4.6). B's positives are more spread out, so a few of them score far above every negative. Those are exactly the cases a strict threshold catches.

4.9.5. Interpretation

The AUC hides this because it averages over all FPRs equally. The region $\mathrm{FPR} \le 10^{-3}$ is a tenth of a percent of the horizontal axis, so it barely affects the area, and most of the AUC comes from FPRs nobody would operate at. The remedy is to report the TPR at a fixed low FPR, chosen by the application, and to plot ROC curves on log–log axes. A log–log ROC curve shows exactly the behaviour near zero that separated the Gaussian and Laplace curves in Section 4.7.

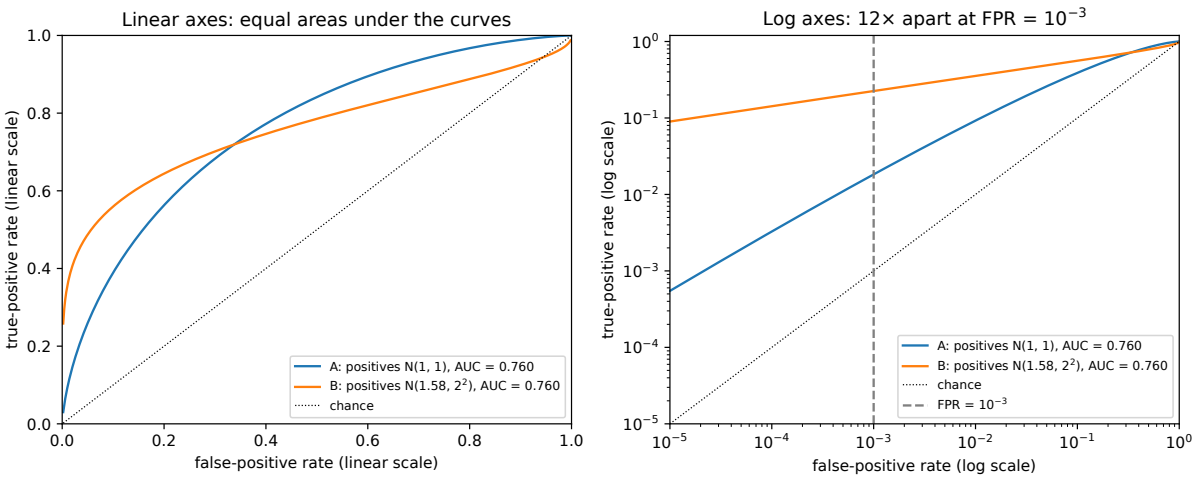


Figure 4.6.: ROC curves of two detectors with the same AUC of 0.760. Left, linear axes: the curves cross and enclose equal areas. Right, log axes: at FPR $= 10^{-3}$ detector B has 12 times the TPR of detector A.

Neither curve here is the best possible for its detector. B's score threshold is not the optimal test for its own P and Q: with unequal variances the log likelihood ratio, ln of the $\mathcal{N}(m, \sigma^2)$ density over the $\mathcal{N}(0, 1)$ density, has the term $\frac{1}{2}(1 - 1/\sigma^2)s^2$, which is quadratic in the score s and positive for $\sigma = 2$. So the ratio is large at both ends, and the optimal test also flags very *low* scores. Neyman–Pearson says the optimal ROC is at least as good as the one plotted.

An extreme case is in Exercise 5 in the exercise book: a detector with AUC only 0.525 that catches 5% of positives at FPR 10^{-3}.

4.9.6. Check yourself

- A detector's ROC curve lies on the diagonal except for a jump to TPR 0.05 at FPR 0. What is its AUC, and is it useless? *(About* $0.5 + 0.05 \times 0.5 \approx 0.525$*. It is not useless: it catches 5% of positives with no false alarms.)*
- Why does the AUC not depend on any threshold, while TPR at FPR 10^{-3} does? *(The AUC integrates over all thresholds. TPR at a fixed FPR evaluates one threshold, the one whose FPR is* 10^{-3}*.)*

4.10. The Bayesian view

4.10.1. Context

Neyman–Pearson treats the two hypotheses asymmetrically and uses no prior. In many problems you do know a base rate: 1% of patients are ill, one transaction in a thousand is fraud. Then the question becomes "given this observation, how likely is H_1?", and the likelihood ratio answers it.

4.10.2. Statement

Bayes' rule says how evidence updates a belief, and in terms of odds it is a single multiplication. Write $\pi = \Pr[H_1]$ for the prior probability of H_1, the base rate. The odds of H_1 against H_0 after seeing x are the odds before seeing it, multiplied by the likelihood ratio of x:

$$\underbrace{\frac{\Pr[H_1 \mid x]}{\Pr[H_0 \mid x]}}_{\text{posterior odds}} = \underbrace{\frac{\pi}{1-\pi}}_{\text{prior odds}} \times \underbrace{\frac{q(x)}{p(x)}}_{\Lambda(x)}. \tag{4.20}$$

If you want to make as few mistakes as possible, you should say H_1 whenever the posterior odds exceed 1, that is, exactly when $\Lambda(x) > (1-\pi)/\pi$. So the decision that minimizes the probability of error is again an LR test, with the threshold set by the prior.

4.10.3. Derivation

1. **Bayes' rule.** The probability of H_1 and x together is $\pi q(x)$, and of H_0 and x together is $(1-\pi)p(x)$. Dividing each by the total probability of x, $m(x) = \pi q(x) + (1-\pi)p(x)$, gives $\Pr[H_1 \mid x] = \pi q(x)/m(x)$ and $\Pr[H_0 \mid x] = (1-\pi)p(x)/m(x)$. Dividing one by the other cancels $m(x)$ and gives Equation 4.20.
2. **Error of a test.** A test errs when H_0 holds and it rejects, with probability $(1-\pi)\alpha_\phi$, or when H_1 holds and it misses, with probability $\pi\beta_\phi$. By Fact 1, writing $\alpha_\phi = \sum_x \phi p$ and $\beta_\phi = 1 - \sum_x \phi q$,

$$(1-\pi)\alpha_\phi + \pi\beta_\phi = \pi + \sum_x \phi(x)((1-\pi)p(x) - \pi q(x)). \tag{4.21}$$

3. **Minimize term by term.** Each $\phi(x)$ appears in one term, so set $\phi(x) = 1$ where the bracket is negative and 0 where it is positive. The bracket is negative where $\pi q(x) > (1-\pi)p(x)$, or $\Lambda(x) > (1-\pi)/\pi$.
4. **The minimum error on the curve.** Because the optimal test is an LR test, it lies on the trade-off curve, so the minimum error is $\min_\alpha [(1-\pi)\alpha + \pi T(\alpha)]$. With equal priors, $\pi = \frac{1}{2}$, it is $\frac{1}{2}\min_\alpha(\alpha + T(\alpha))$. Minimizing $\alpha + T(\alpha)$ is maximizing the advantage $1 - \alpha - T(\alpha)$, whose maximum is TV by Equation 4.16, so the minimum error is $\frac{1}{2}(1 - \mathrm{TV})$. For the Gaussian pair at $\mu = 1$ that is $\frac{1}{2}(1 - 0.3829) = 0.3085$.

The same holds for any outcome you observe, not only for x. If the evidence is just "the test fired", its likelihood ratio is the probability of firing under H_1 over that under H_0, $\Pr[\text{fire} \mid H_1]/\Pr[\text{fire} \mid H_0] = (1-\beta)/\alpha$, the power divided by the false positive rate.

4.10.4. Example: screening at 1% prevalence

Use the biomarker test with $\mu = 2$ from Table 4.4, in a population where 1% are ill (prior odds 1/99). To get the probability of illness after a positive result, multiply the prior odds by the likelihood ratio of a positive, then turn odds o back into a probability $o/(1+o)$.

At $\alpha = 0.05$ the power is 0.6388, so a positive result has likelihood ratio $0.6388/0.05 = 12.8$. The posterior odds are $12.8/99 = 0.129$, so $\Pr[\text{ill} \mid \text{positive}] = 0.129/1.129 = 0.114$. Nine in ten

positives are false alarms. At $\alpha = 10^{-3}$ the power is 0.1378, the likelihood ratio is $0.1378/0.001 = 137.8$, the posterior odds are $137.8/99 = 1.392$, and $\Pr[\text{ill} \mid \text{positive}] = 1.392/2.392 = 0.582$.

The stricter threshold catches fewer ill patients but makes a positive result far more informative (Figure 4.7). This is the second reason low-FPR behaviour matters: when positives are rare, the false-positive rate, not the power, decides whether an alarm is worth acting on.

The Bayes rule here. Minimizing the error probability at $\pi = 0.01$ means rejecting when $\Lambda(x) > (1-\pi)/\pi = 99$. For the Gaussian, $\Lambda(x) = e^{\mu x - \mu^2/2} > 99$ means $x > \mu/2 + \ln(99)/\mu = 1 + 4.5951/2 = 3.2976$. That test has $\alpha = 1 - \Phi(3.2976) = 4.88 \times 10^{-4}$ and power only $1 - \Phi(3.2976 - 2) = 0.0972$: when errors of both kinds count equally and the disease is rare, it is cheapest to rarely say "ill". If missing a case costs c times as much as a false alarm, the quantity to minimize becomes $(1-\pi)\alpha + c\pi\beta$, the same argument gives the threshold $(1-\pi)/(c\pi)$, and the test moves along the same curve.

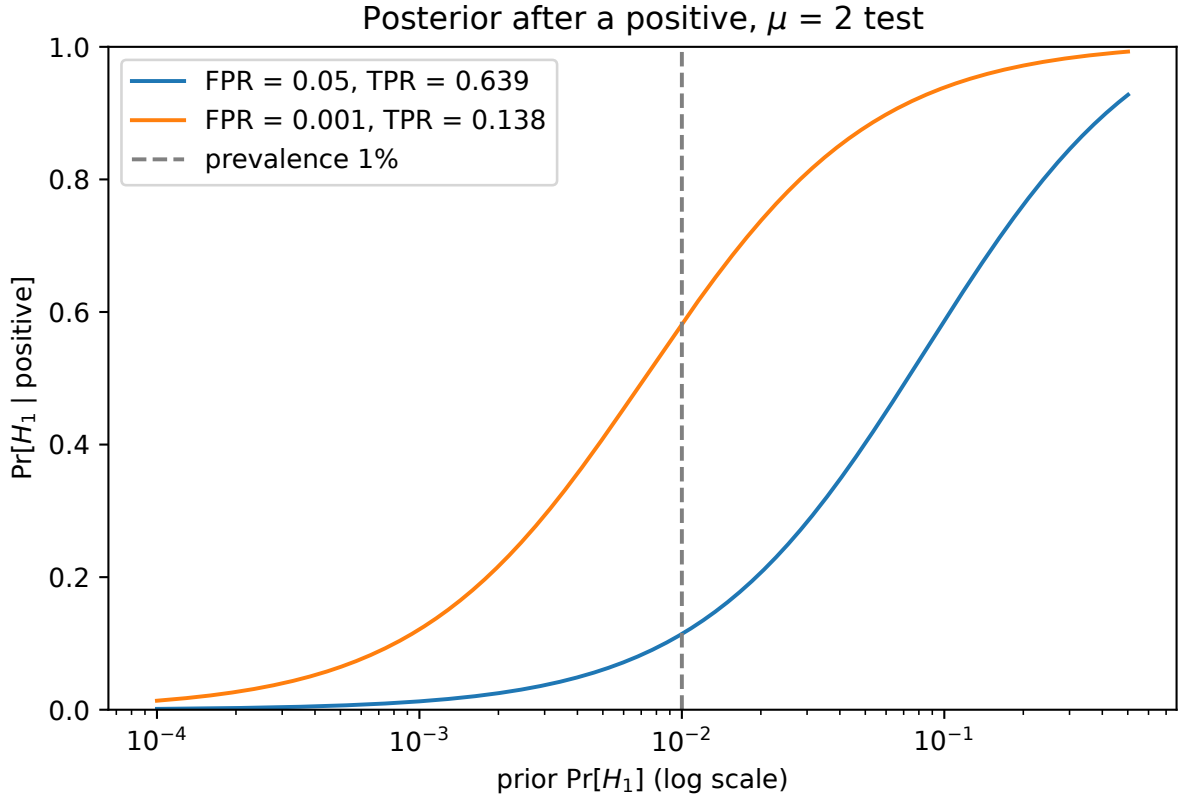


Figure 4.7.: Posterior probability of illness after a positive result, as a function of the prior, for the $\mu = 2$ biomarker test at two thresholds. At 1% prevalence (dashed line) the stricter test gives 0.582 against 0.114.

4.10.5. Check yourself

- If the prior doubles from 1% to 2%, roughly how does the posterior odds after a positive change? *(It roughly doubles: posterior odds scale linearly with prior odds,* 2/98 *against* 1/99*.)*

- Why is the Bayes-optimal test an LR test even though Bayes and Neyman–Pearson ask different questions? *(Both trade off the two errors linearly: Bayes with fixed weights $1-\pi$ and π, Neyman–Pearson with the exchange rate η, which plays the role of a Lagrange multiplier on the false-positive budget. Each linear trade-off is minimized by a threshold on Λ.)*

4.11. Estimating error rates from finite trials

4.11.1. Context

Everything so far assumed you know P and Q. In practice you measure a detector's FPR and TPR on test data: k false positives among n negatives. Each negative is falsely flagged independently with probability α, so the count k is Binomial(n, α), the number of successes in n independent trials that each succeed with probability α (Probability distributions). The measured rate k/n is therefore uncertain, and at low rates the uncertainty is large relative to the rate itself. A claim such as "TPR at FPR 10^{-3}" is only as good as the interval around it.

4.11.2. Statement: the Clopper–Pearson interval

The interval is built by asking which true rates would make the observed count unsurprising. A true rate far above k/n would make a count as small as k very unlikely; the upper end u is the rate at which seeing k or fewer has probability exactly $a/2$. Likewise the lower end l is the rate at which seeing k or more has probability $a/2$. For k successes in n trials and confidence level $1-a$, let l and u solve

$$\Pr[\mathrm{Bin}(n, l) \geq k] = \frac{a}{2}, \qquad \Pr[\mathrm{Bin}(n, u) \leq k] = \frac{a}{2}, \tag{4.22}$$

with $l = 0$ if $k = 0$ and $u = 1$ if $k = n$. Then $\Pr[l \leq p \leq u] \geq 1 - a$ for every true rate p.

4.11.3. Derivation (upper end; the lower end is symmetric)

The interval is random, because it depends on the observed count, and the claim is about how often it misses the fixed true rate p. The plan is to show that it misses above, $p > u$, with probability at most $a/2$. Write $F_p(k) = \Pr[\mathrm{Bin}(n, p) \leq k]$ for the probability of at most k successes at rate p, and let $K \sim \mathrm{Bin}(n, p)$ be the observed count.

1. **Monotonicity.** $F_p(k)$ is decreasing in p: a larger success probability makes small counts less likely. So for each k there is a single rate $u(k)$ with $F_{u(k)}(k) = a/2$, and the true rate lies at or above the upper end, $p \geq u(K)$, exactly when $F_p(K) \leq F_{u(K)}(K) = a/2$.
2. **A CDF evaluated at its own draw is rarely small.** It remains to show $\Pr[F_p(K) \leq a/2] \leq a/2$. Let k^* be the largest k with $F_p(k) \leq a/2$. Since F_p is increasing in k, $F_p(K) \leq a/2$ exactly when $K \leq k^*$, so $\Pr[F_p(K) \leq a/2] = \Pr[K \leq k^*] = F_p(k^*) \leq a/2$. (If no such k^* exists, the probability is 0.)
3. **Combine.** So $\Pr[p \geq u(K)] \leq a/2$, and the same argument gives $\Pr[p \leq l(K)] \leq a/2$ at the lower end. The probability that at least one of two events happens is at most the sum of their probabilities (the union bound), so the interval misses with probability at most a, and its coverage is at least $1 - a$.

The interval is **exact** in the sense that it uses the binomial distribution itself rather than an approximation, and **conservative**: its coverage is at least $1 - a$, often more.

Zero counts and the rule of three. If $k = 0$, the probability of seeing no successes at rate u is $(1 - u)^n$, so the one-sided upper bound at level 0.95 solves $(1 - u)^n = 0.05$. That gives $u = 1 - 0.05^{1/n}$. For large n, $0.05^{1/n} = e^{-\ln(20)/n} \approx 1 - \ln(20)/n$, so $u \approx \ln(20)/n \approx 3/n$, since $\ln 20 = 3.00$ to two decimals. With $n = 3{,}000$ negatives and no false positives, $u = 9.981 \times 10^{-4}$. Turning this around, to certify an FPR below 10^{-3} at 95% confidence you need $(1-10^{-3})^n \leq 0.05$, that is, at least $\ln 20/(-\ln(1 - 10^{-3})) = 2{,}994.2$, so 2,995, negatives with *no* false positive at all.

4.11.4. Example: measuring TPR at FPR 10^{-3}

Take the $\mu = 2$ biomarker, with the threshold $t = 3.0902$ that gives a true FPR of 10^{-3} and a true TPR of 0.1378. Test it on $n = 10{,}000$ negatives and 10,000 positives. One simulated run gives Table 4.7.

Table 4.7.: One simulated evaluation of the $\mu = 2$ test at its 10^{-3} threshold, with 10,000 examples per class.

Quantity	Count	Estimate	95% Clopper–Pearson interval
False positives	10	0.0010	$[0.00048, 0.00184]$
True positives	1,350	0.1350	$[0.1284, 0.1419]$

The TPR is pinned down to about ± 0.007, half the interval's width of $0.1419 - 0.1284 = 0.0135$. The FPR is not: the data are consistent with a true FPR anywhere from about half to almost twice the target (Table 4.7). The number of negatives limits what a low-FPR claim can mean.

Two other common ways to build the FPR interval from $k = 10$, $n = 10{,}000$ do worse. The **Wald** interval uses the normal approximation to the binomial, $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ with $\hat{p} = k/n$, where 1.96 is $\Phi^{-1}(0.975)$. Here the half-width is $1.96\sqrt{0.001 \times 0.999/10{,}000} = 0.00062$, giving $[0.00038, 0.00162]$. That is too narrow and symmetric for a skewed count, and at $k = 0$ it collapses to $[0, 0]$, claiming certainty from no data. The **Hoeffding** interval comes from Hoeffding's inequality (Concentration inequalities), which says that the average of n independent variables in $[0, 1]$ strays from its mean by at least ε with probability at most $2e^{-2n\varepsilon^2}$. Setting that to 0.05 gives the two-sided half-width $\varepsilon = \sqrt{\ln(2/0.05)/(2n)} = \sqrt{\ln(40)/20{,}000} = 0.0136$, more than 13 times the rate being measured. A bound that uses only the range $[0, 1]$ ignores the tiny variance $p(1 - p)$.

Clopper–Pearson is conservative, not loose: over 2,000 simulated evaluations with true FPR 10^{-3} and $n = 10{,}000$, its interval covered the true rate 97.20% of the time, above the nominal 95%. Figure 4.8 shows how slowly the interval tightens with n when the observed rate stays at 10^{-3}.

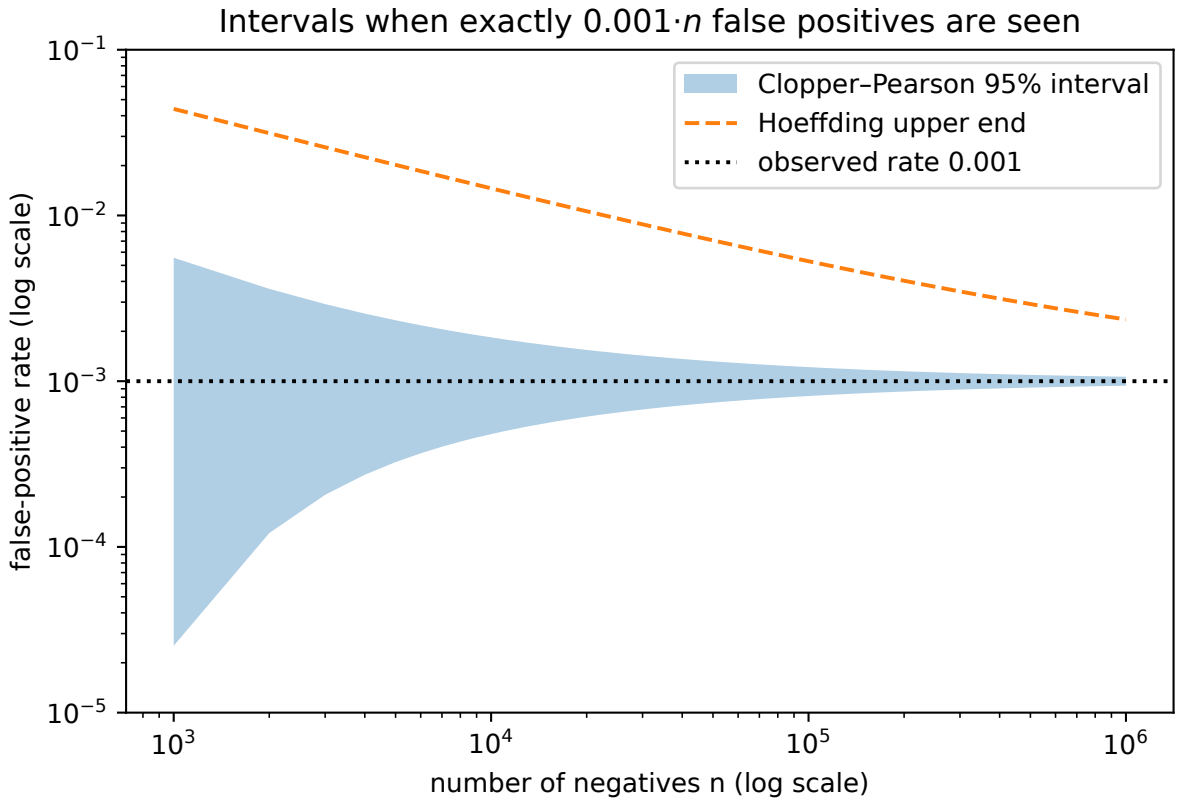


Figure 4.8.: 95% Clopper–Pearson intervals for an observed false-positive rate of exactly 10^{-3}, as the number of negatives grows from 10^3 to 10^6, with the upper end of the two-sided Hoeffding interval for comparison.

4.11.5. Interpretation

What sets the relative width of the interval is mainly the expected count $k = n\alpha$, the number of false positives you expect to see, not n alone. A count of k fluctuates by about $\sqrt{k}$, so its relative uncertainty is about $1/\sqrt{k}$. Ten events give a factor-of-two uncertainty, $[0.00048, 0.00184]$ around 10^{-3}. A hundred events, at $n = 100{,}000$, give about $\pm 20\%$: $[0.00081, 0.00122]$. This gives a rule of thumb: to measure an FPR of α to within a factor of about 1.2, you need on the order of $100/\alpha$ negatives, 10^5 for $\alpha = 10^{-3}$.

The threshold is also estimated. If you choose the threshold on the same data so that the empirical FPR is 10^{-3}, the true FPR of that threshold is uncertain in exactly the way shown above. Choose thresholds on one split and measure rates on another.

4.11.6. Check yourself

- You see 0 false positives in 300 negatives. What can you claim at 95% confidence? *(FPR $\leq$ about $3/300 = 0.01$, by the rule of three. Nothing about 10^{-3}.)*
- Why is Hoeffding so much wider than Clopper–Pearson at low rates? *(Hoeffding assumes the worst-case variance $\frac{1}{4}$ for a variable in $[0, 1]$; the true variance of a false-positive indicator is $\alpha(1 - \alpha) \approx 10^{-3}$.)*

4.12. Code: check every result by computation and simulation

The script code/hypothesis_testing.py computes every number quoted in this chapter and draws every figure. Each error rate, trade-off curve, interval and simulation is a small typed function; the script calls them once for the table and loops them over parameters for the figures. It uses only the standard library and NumPy: Φ comes from `math.erfc`, and Φ^{-1} and the Clopper–Pearson ends are found by bisection. From the repository root:

```
uv run python code/part01-prerequisites/hypothesis_testing.py
```

Output (seed 0), summarized in Table 4.8:

Table 4.8.: Results printed by the script with seed 0.

Experiment	Result
Spam example, LR test at $\alpha = 0.25$	$\eta = 1.5$, $\gamma = 0.75$, power 0.625
200,000 random tests on the spam example	11,727 with size ≤ 0.25; best power 0.587
Gaussian $\mu = 2$, $\alpha = 0.05$	$\beta = 0.3612$ exact; simulated $\hat{\alpha} = 0.0503$, $\hat{\beta} = 0.3617$
Gaussian $\mu = 2$, $\alpha = 10^{-3}$	power 0.1378
Laplace $\kappa = 1$, $\alpha = 0.25$	$\beta = 0.3679$ exact; simulated $\hat{\alpha} = 0.2498$, $\hat{\beta} = 0.3685$
Same TV 0.3935, $\alpha = 10^{-6}$	Laplace power 2.718×10^{-6}; Gaussian ($\mu = 1.0301$) 9.829×10^{-5}

Experiment	Result
Best advantage, Gaussian $\mu = 1$	Formula 0.3829; grid maximum 0.3829 at $\alpha = 0.3085$
Equal-AUC detectors A and B	AUC 0.7602 both; TPR at 10^{-3}: 0.0183 and 0.2253
Screening at 1% prevalence, $\mu = 2$	Posterior after a positive: 0.114 at $\alpha = 0.05$; 0.582 at $\alpha = 10^{-3}$
Clopper–Pearson, $k = 10$, $n = 10{,}000$	$[0.00048, 0.00184]$; coverage over 2,000 runs 0.9720

tests/test_hypothesis_testing.py checks that the output still matches Table 4.8 and every other number in the text.

Four things stand out. First, Neyman–Pearson is not just a theorem: a random search over 200,000 tests never comes close to the LR test at the same level. Second, the closed-form curves match simulation to within sampling error, for both the Gaussian and the Laplace pairs, so the derivations can be trusted at the levels where simulation is too expensive. Third, summaries can agree while the curves differ: equal TV or equal AUC says nothing about power at 10^{-3} or 10^{-6}, which is why the whole curve is the object to look at. Fourth, low rates are expensive to measure: ten false positives pin the FPR down only to a factor of two.

4.13. References

- Lehmann & Romano, *Testing Statistical Hypotheses* (3rd ed., 2005), ch. 3: most powerful tests and power functions, the rigorous Neyman–Pearson lemma (§3.2, Theorem 3.2.1) with the converse that every most powerful test is an LR test almost everywhere, and monotone likelihood ratios. The trade-off function is the smallest type II error as a function of the level, that is, one minus the power of the most powerful test.
- Casella & Berger, *Statistical Inference* (2nd ed., 2002), §8.3: a gentler proof of the lemma (Theorem 8.3.12) and the Karlin–Rubin theorem for one-sided composite tests.
- Wasserman, *All of Statistics* (2004), ch. 10: a compact tour of tests, power and p-values.
- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), §11.7–11.9: Neyman–Pearson from the information-theory side, the Chernoff–Stein lemma and Chernoff information.
- Fawcett, "An introduction to ROC analysis" (Pattern Recognition Letters, 2006): ROC curves (the trade-off function turned upside down), AUC as a ranking probability, and the convex hull of a classifier's ROC, which is the convexity property proved here.
- Clopper & Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial" (Biometrika, 1934), and Brown, Cai & DasGupta, "Interval Estimation for a Binomial Proportion" (Statistical Science, 2001): the exact interval, and why the Wald interval fails.

4.14. Cheat sheet

Table 4.9 collects every result in the chapter.

Table 4.9.: Every result in this chapter.

Result	Statement	Where it comes from
Errors of a test ϕ	$\alpha_\phi = \mathbb{E}_P\phi$, $\beta_\phi = 1 - \mathbb{E}_Q\phi$, power $= 1 - \beta_\phi$	Fact 1
Neyman–Pearson	The size-α test that rejects when $q/p > \eta$ (randomizing at $=$) is most powerful at level α	$(\phi^* - \phi)(q - \eta p) \geq 0$ pointwise
Trade-off function	$T(\alpha) = \inf\{\beta_\phi : \alpha_\phi \leq \alpha\}$; non-increasing, $\leq 1 - \alpha$, convex, continuous	Mixing tests; change of measure
Slope of T	$T'(\alpha) = -\Lambda$ at the threshold	$d\beta/d\alpha = -q(t)/p(t)$
Gaussian $\mathcal{N}(0,1)$ vs $\mathcal{N}(\mu,1)$	$T(\alpha) = \Phi(\Phi^{-1}(1-\alpha) - \mu)$; n draws: $\mu = \delta\sqrt{n}/\sigma$	LR increasing in x
Laplace $\mathrm{Lap}(0,b)$ vs $\mathrm{Lap}(s,b)$	$1 - e^\kappa\alpha$; $e^{-\kappa}/(4\alpha)$; $e^{-\kappa}(1-\alpha)$, with $\kappa = s/b$	LR clipped to $[e^{-\kappa}, e^\kappa]$
Best advantage	$\max_\phi(1-\alpha_\phi-\beta_\phi) = \mathrm{TV}(P,Q)$; Gaussian $2\Phi(\mu/2) - 1$; Laplace $1 - e^{-\kappa/2}$	Reject where $q > p$
AUC	$\Pr[S_+ > S_-]$; binormal $\Phi(m/\sqrt{1+\sigma^2})$	Change of variables in $\int \mathrm{TPR}\, d\mathrm{FPR}$
Bayes	Posterior odds = prior odds $\times\Lambda(x)$; minimum error at $\pi = \frac{1}{2}$ is $\frac{1}{2}(1 - \mathrm{TV})$	Bayes' rule
Clopper–Pearson	$\Pr[\mathrm{Bin}(n,l) \geq k] = \Pr[\mathrm{Bin}(n,u) \leq k] = a/2$; zero counts: $u \approx 3/n$	Inverting binomial tails

Rules of thumb

- To find the best test, compute the likelihood ratio and threshold it. If it is monotone in a statistic, threshold the statistic.
- Look at the whole trade-off curve, on log axes if false positives must be rare. TV and AUC are single points or averages of it.
- A bounded likelihood ratio caps power at small α: power $\leq \Lambda_{\max}\,\alpha$.
- To measure a rate near α, plan for about $100/\alpha$ trials, and report a Clopper–Pearson interval.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

5. Information theory

5.1. Why this matters

You observe something, and you want to know how much it tells you about something else you cannot see. A word in an email says something about whether the email is spam. A noisy sensor reading says something about the true state. A model's prediction says something about the label. Information theory turns "how much" into a number, and it does so in two different ways, because there are two different questions you might be asking.

The first question is: **how many bits, on average, does the observation save you when you describe the hidden value?** Shannon entropy and mutual information answer it. They are the language of compression, of the cross-entropy loss you train classifiers with, of decision-tree splits and of sending data over noisy links. The second question is: **how much more likely is a single best guess to be right once you have seen the observation?** Min-entropy and vulnerability answer that one. They are the language of password strength, and of anyone who gets only one try.

The idea that organizes the chapter is that **every quantity here is built from the logarithm of a probability, either averaged or maximized.** Shannon entropy is the average of $-\log p(x)$ over the outcomes x; min-entropy uses only the most likely outcome instead. Because of this, almost every derivation below is one of two moves. Either it applies Jensen's inequality, which compares the average of a logarithm with the logarithm of an average, or it takes the product rule $p(x, y) = p(y)\, p(x \mid y)$ (the probability of both equals the probability of y times the probability of x given y) and turns it, by taking logs, into a sum.

Table 5.1.: The results of this chapter and where they are used.

Result	Where it is used
Entropy, $0 \leq H(X) \leq \log\lvert\mathcal{X}\rvert$	Cross-entropy loss, compression, the scale of everything below
Chain rule, conditional entropy	Splitting the uncertainty of several variables into pieces
Mutual information $I(X;Y)$ as a Kullback–Leibler (KL) divergence	Feature selection, information gain; the link to Divergences
Data-processing inequality	Processing an observation cannot add information about its source
Channel capacity, binary symmetric channel $1 - h(p)$	The most information a noisy channel can carry

Result	Where it is used
Vulnerability and min-entropy	One-guess success; password strength
Conditional min-entropy, min-entropy leakage, min-capacity	How much an observation helps a guesser, and its worst case over priors
Fano's inequality	Lower bounds: no estimator can beat a given error rate

Table 5.1 lists the results. Later parts of the book measure what an observation reveals about a hidden value. They need both views, the average one and the one-guess one, and they need to know where the two disagree.

Logarithms in this chapter are base 2, so quantities are in **bits**, unless stated otherwise. Divergences uses natural logarithms, whose unit is the **nat**. To convert, note that $\log_2 x = \ln x / \ln 2$, so one nat is $1/\ln 2 = \log_2 e \approx 1.443$ bits.

5.2. Toolkit: three facts every derivation uses

5.2.1. Fact 1 — Jensen's inequality for the logarithm

The logarithm grows more and more slowly: going from 1 to 2 adds one bit, going from 2 to 3 adds much less. Because of this bend, if you average some positive numbers and then take the log, you get more than if you take the logs first and then average. Concretely, the numbers 1 and 4 have logs 0 and 2, whose average is 1, while their average 2.5 has $\log \log_2 2.5 \approx 1.32$. In general, for a positive random variable Z,

$$\mathbb{E}[\log Z] \leq \log \mathbb{E}[Z], \tag{5.1}$$

with equality if and only if Z is constant (with probability 1), since only then is there nothing for the bend to act on.

The proof uses the fact that a curve that bends downward, a **concave** function, lies below each of its tangent lines. Take the tangent to log at the point $m = \mathbb{E}Z$. Its slope is the derivative of $\log_2 z$ at m, which is $1/(m \ln 2)$, so for every $z > 0$

$$\log z \leq \log m + \frac{z - m}{m \ln 2}. \tag{5.2}$$

Now put Z in place of z and average both sides. The left side becomes $\mathbb{E}[\log Z]$. On the right, $\log m$ stays as it is and the second term averages to zero, because $\mathbb{E}[Z - m] = 0$ by the choice of m. That leaves $\mathbb{E}[\log Z] \leq \log m = \log \mathbb{E}Z$. The tangent touches log only at m, so equality needs $Z = m$ wherever Z has probability: Z must be constant.

This fact turns "an average of logs" into "a log of an average", and that is how every bound in this chapter is proved. Multiplying both sides by -1 gives the mirror version for $-\log$, which bends the other way (it is **convex**): $-\log \mathbb{E}Z \leq \mathbb{E}[-\log Z]$.

5.2.2. Fact 2 — Gibbs' inequality: KL divergence is non-negative

To say how different two distributions p and q on the same finite set are, information theory uses the **Kullback–Leibler (KL) divergence** $D(p \,\|\, q)$. It is the average, over outcomes drawn from p, of the log of how much more likely each outcome is under p than under q. If p and q are the same, every ratio is 1, every log is 0, and the divergence is 0. Gibbs' inequality says that it can never be negative:

$$D(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0, \tag{5.3}$$

with equality if and only if $p = q$. This is not obvious, because individual log ratios can be negative (wherever $q(x) > p(x)$). The point is that on average, weighted by p, they cannot be.

The proof is Jensen's inequality (Fact 1) applied to the ratio $Z = q(X)/p(X)$, with X drawn from p. The average of this ratio is easy: the $p(x)$ in the weight cancels the $p(x)$ in the denominator, leaving $\sum q(x)$ over the outcomes that p can produce, which is at most 1. So, summing only over x with $p(x) > 0$,

$$-D(p \,\|\, q) = \mathbb{E}\left[\log \frac{q(X)}{p(X)}\right] \leq \log \sum_{x:\, p(x)>0} p(x) \frac{q(x)}{p(x)} = \log \sum_{x:\, p(x)>0} q(x) \leq \log 1 = 0. \tag{5.4}$$

Both inequalities in this line must be equalities for $D = 0$. The first needs the ratio q/p to be constant where p has mass (the equality case of Fact 1), and the second needs q to put no mass outside that set. Together these force $q = p$.

Every "≥ 0" and every "$\leq \log|\mathcal{X}|$" below is this fact in disguise. Divergences studies D itself in more detail.

5.2.3. Fact 3 — Conventions

Three conventions keep the formulas short. First, $0 \log 0 = 0$. The expression $0 \log 0$ is not defined on its own, but $t \log t \to 0$ as $t \to 0$, and this choice says that outcomes of probability zero contribute nothing, which is what you want from a measure of uncertainty.

Second, a joint distribution of two variables with finitely many values is a table $p(x, y)$, with one row for each value of X and one column for each value of Y. Adding up a row gives the **marginal** probability $p(x)$ that $X = x$, whatever Y is; adding up a column gives $p(y)$.

Third, a **channel** from X to Y is a rule that, for each input x, says how likely each output y is. It is also a table, $C[x, y] = p(y \mid x)$, and each row sums to 1 because, given the input, some output must occur. A channel alone does not say how likely each input is. That comes from a **prior** $\pi(x)$, and the two together give the joint table by the product rule, $p(x, y) = \pi(x)\, C[x, y]$: the probability that the input is x, times the probability that the channel then produces y.

5.3. Shannon entropy

5.3.1. Context

How uncertain is a random variable? A fair coin should count as one unit of uncertainty, two fair coins as two, and a coin that always lands heads as zero. Shannon entropy is the measure with these properties, and it is also the average number of bits that the best possible code needs per symbol. You meet it every time you train a classifier. The cross-entropy loss, averaged over the data, equals $H(p) + D(p \,\|\, q)$: the entropy of the true label distribution p, which no model can remove, plus the KL divergence from p to the model's predicted distribution q, which training drives down.

5.3.2. Statement

The rarer an outcome, the more it tells you when it happens. So measure the information in an outcome x by its **surprisal** $\log(1/p(x))$, which is 0 for a certain outcome and grows as $p(x)$ shrinks: an outcome of probability $\frac{1}{2}$ carries 1 bit, one of probability $\frac{1}{4}$ carries 2 bits. The **entropy** of a discrete random variable X with distribution p is its average surprisal:

$$H(X) = -\sum_x p(x) \log p(x) = \mathbb{E}\left[\log \frac{1}{p(X)}\right]. \tag{5.5}$$

For a variable that is 1 with probability p and 0 otherwise, a Bernoulli(p) variable, the sum has two terms. It is called the **binary entropy function**:

$$h(p) = -p \log p - (1-p) \log(1-p). \tag{5.6}$$

Bounds. Entropy cannot be negative, because surprisals are not, and it is largest when no outcome is favoured. For X taking values in a finite set $\mathcal{X}$, with $|\mathcal{X}|$ elements,

$$0 \le H(X) \le \log|\mathcal{X}|. \tag{5.7}$$

The lower bound holds with equality exactly when X is constant, the upper bound exactly when X is uniform.

Joint and conditional entropy. For two variables, $H(X, Y)$ is the entropy of the pair, computed from the joint table. Once you observe a particular value $Y = y$, the uncertainty about X is the entropy of the distribution of X given that value, $H(X \mid Y = y) = -\sum_x p(x \mid y) \log p(x \mid y)$. Before you observe Y, you do not know which y you will see, so the natural summary is the average over y. That average is the **conditional entropy**:

$$H(X \mid Y) = \sum_y p(y)\, H(X \mid Y = y). \tag{5.8}$$

Chain rule. The uncertainty in the pair is the uncertainty in Y plus whatever uncertainty about X remains once Y is known, and the same holds with the roles swapped:

$$H(X, Y) = H(Y) + H(X \mid Y) = H(X) + H(Y \mid X). \tag{5.9}$$

5.3.3. Derivation

Lower bound. The idea is that entropy is an average of surprisals, and no surprisal is negative.

1. Each probability satisfies $0 \le p(x) \le 1$, so $1/p(x) \ge 1$ and $\log(1/p(x)) \ge 0$. Each term $p(x)\log(1/p(x))$ is therefore non-negative (Fact 3 handles $p(x) = 0$).
2. A sum of non-negative terms is zero only if every term is zero. A term $p(x)\log(1/p(x))$ is zero only when $p(x) = 0$ or $p(x) = 1$. So $H(X) = 0$ forces one value to have probability 1: X is constant.

Upper bound. The idea is that the gap between $\log|\mathcal{X}|$ and $H(X)$ is exactly the KL divergence from p to the uniform distribution, which Fact 2 says is non-negative.

1. Let $u(x) = 1/|\mathcal{X}|$ be the uniform distribution. Since $\sum_x p(x) = 1$, you can write the constant $\log|\mathcal{X}|$ as $\sum_x p(x)\log|\mathcal{X}|$. Then $\log|\mathcal{X}| - H(X) = \sum_x p(x)\log|\mathcal{X}| + \sum_x p(x)\log p(x)$.
2. Combine the two logs in each term, using $\log|\mathcal{X}| = \log(1/u(x))$: the sum becomes $\sum_x p(x)\log\frac{p(x)}{u(x)} = D(p\,\|\,u)$.
3. By Gibbs' inequality (Fact 2), $D(p\,\|\,u) \ge 0$, with equality iff $p = u$. So $H(X) \le \log|\mathcal{X}|$, with equality iff X is uniform.

Chain rule. The idea is that the product rule for probabilities becomes a sum rule for surprisals, and averaging preserves sums.

1. For every pair with $p(x,y) > 0$, the product rule gives $p(x,y) = p(y)\,p(x \mid y)$. Taking $\log(1/\cdot)$ of both sides turns the product into a sum: $\log\frac{1}{p(x,y)} = \log\frac{1}{p(y)} + \log\frac{1}{p(x|y)}$.
2. Average both sides over the pair (X,Y). The left side is the average surprisal of the pair, $H(X,Y)$. The first term on the right is $H(Y)$. The second term, grouped by y, is $\sum_y p(y)\sum_x p(x \mid y)\log\frac{1}{p(x|y)}$, which is the average over y of $H(X \mid Y = y)$, that is $H(X \mid Y)$.
3. Starting from $p(x,y) = p(x)\,p(y \mid x)$ instead, the same steps give the second form.

5.3.4. Interpretation

Figure 5.1 shows the binary entropy $h(p)$. It is 1 bit at $p = \frac{1}{2}$, where the coin is fair and you have no idea which way it will land, and zero at $p \in \{0,1\}$, where you know the outcome. It is very steep near the ends, so even a quite lopsided coin keeps a lot of uncertainty: $h(0.11) = 0.4999$, so a coin with an 11% chance of heads still carries half a bit. The figure also shows the min-entropy of the same coin, a one-guess measure that Section 5.6 defines and that is always below h.

The chain rule splits uncertainty into pieces: first reveal Y, then what remains of X. Applying Equation 5.9 repeatedly, peeling off one variable at a time, extends it to any number of variables: $H(X_1,\ldots,X_n) = \sum_i H(X_i \mid X_1,\ldots,X_{i-1})$, the uncertainty of the first variable, plus that of the second given the first, and so on.

Entropy depends only on the probabilities, not on the values themselves. Relabelling the outcomes, for instance calling them "red" and "blue" instead of 0 and 1, does not change it.

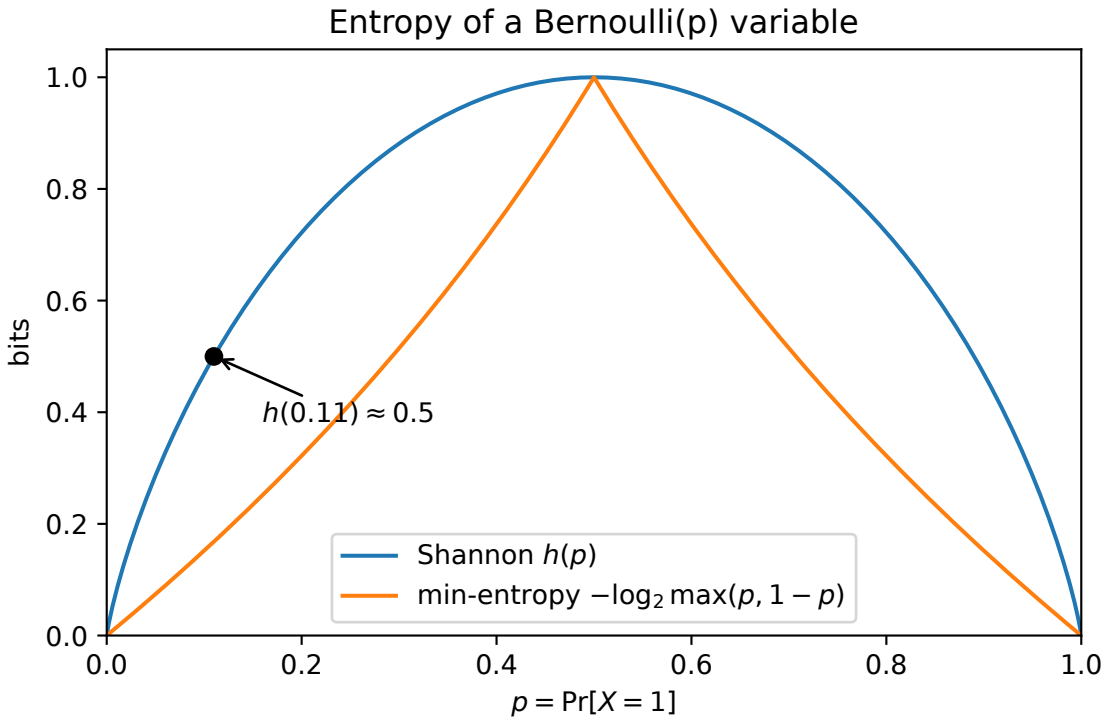


Figure 5.1.: The binary entropy $h(p)$ against the min-entropy $-\log_2 \max(p, 1-p)$ of the same Bernoulli variable. Both are 1 bit at $p = \frac{1}{2}$; the min-entropy is lower everywhere else.

5.3.5. Conditioning reduces entropy on average, not pointwise

Intuitively, learning something should not make you more uncertain. That is true on average: averaged over what you might observe, observing Y cannot increase the uncertainty about X,

$$H(X \mid Y) \leq H(X), \tag{5.10}$$

with equality if and only if X and Y are independent. The proof is short once mutual information is defined in the next section. There the gap $H(X) - H(X \mid Y)$ turns out to be a KL divergence (Equation 5.11), and KL divergences are non-negative by Gibbs' inequality (Fact 2).

But a *particular* observation can increase the uncertainty. $H(X \mid Y = y) > H(X)$ is perfectly possible, as the next example shows: if an observation is rare, it can push you from a confident belief into a toss-up, and the average stays low only because the common observations make up for it.

5.3.6. Example: a spam filter's keyword

Let X be the label of an email (ham or spam) and Y whether it contains the word "offer". One email in eight is spam. Every spam email contains the word, and so does one ham email in seven. Since ham emails are $\frac{7}{8}$ of the total, the ham emails with the word are $\frac{1}{7} \cdot \frac{7}{8} = \frac{1}{8}$ of all emails, and the ham emails without it are $\frac{7}{8} - \frac{1}{8} = \frac{3}{4}$. Table 5.2 gives the full joint distribution.

Table 5.2.: Joint distribution of an email's label and whether it contains the keyword.

	$Y = 0$ (absent)	$Y = 1$ (present)	$p(x)$
ham	3/4	1/8	7/8
spam	0	1/8	1/8
$p(y)$	3/4	1/4	

Work out each quantity from Table 5.2. Before you look at the email, the label is spam with probability $\frac{1}{8}$, so $H(X) = h(1/8) = \frac{1}{8}\log_2 8 + \frac{7}{8}\log_2 \frac{8}{7} = 0.375 + 0.1686 = 0.5436$ bits: the label is fairly predictable, because most email is ham.

If the word is absent, you are in the first column. Its only non-zero entry is ham, so the email is certainly ham and $H(X \mid Y = 0) = 0$.

If the word is present, you are in the second column, where ham and spam each have probability $\frac{1}{8}$ out of $\frac{1}{4}$. Given the word, they are equally likely, so $H(X \mid Y = 1) = h(1/2) = 1$ bit. **Seeing the word made you more uncertain** than you were before: from 0.5436 bits up to a full bit.

On average, though, the word helps. You see it absent with probability $\frac{3}{4}$ and present with probability $\frac{1}{4}$, so $H(X \mid Y) = \frac{3}{4} \cdot 0 + \frac{1}{4} \cdot 1 = 0.25$ bits, less than $H(X)$.

The chain rule can be checked on the same table. The word is present with probability $\frac{1}{4}$, so $H(Y) = h(1/4) = 0.8113$, and $H(Y) + H(X \mid Y) = 0.8113 + 0.25 = 1.0613$. That equals the entropy of the pair computed directly from the three non-zero cells $(\frac{3}{4}, \frac{1}{8}, \frac{1}{8})$.

Figure 5.2 draws the four numbers. The common observation (word absent) removes all uncertainty; the rare one (word present) increases it; the average goes down. That is the general pattern: conditioning reduces entropy only in expectation.

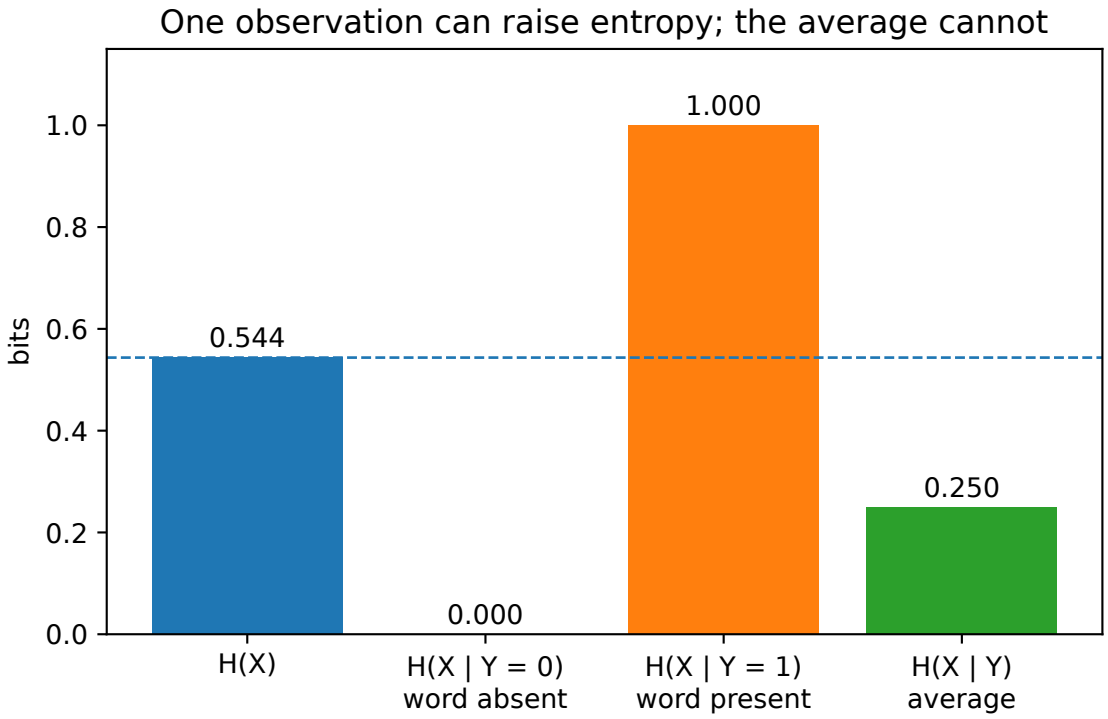


Figure 5.2.: Entropy of the spam label before observing the keyword, after each of the two possible observations, and on average. The rare observation raises the entropy above $H(X)$ (dashed line); the average falls below it.

5.3.7. Check yourself

- What is the entropy of a uniformly random 4-digit PIN, and why? *(There are 10,000 equally likely PINs, so* $H = \log_2 10{,}000 = 13.2877$ *bits: the upper bound* $\log|\mathcal{X}|$*, attained because the distribution is uniform.)*
- Using the chain rule, show that $H(X, Y) \leq H(X) + H(Y)$, with equality iff X and Y are independent. *(*$H(X, Y) = H(Y) + H(X \mid Y)$*, and* $H(X \mid Y) \leq H(X)$ *with equality iff independent.)*

5.4. Mutual information

5.4.1. Context

How much does observing Y tell you about X? A natural answer is the drop in uncertainty it causes on average, $H(X) - H(X \mid Y)$. This is the **information gain** a decision tree uses to choose a split, and the criterion many feature-selection methods use to rank features. It turns out to be a KL divergence, which makes it symmetric and non-negative, and it obeys a rule that drives a great deal of later reasoning: processing cannot create information.

5.4.2. Statement

The **mutual information** between X and Y is the average drop in uncertainty about X from seeing Y. It can also be read as a measure of dependence: the KL divergence from the true joint distribution P_{XY} to the product $P_X P_Y$ of its marginals, which is what the joint distribution would be if X and Y were independent. The two readings give the same number:

$$I(X;Y) = H(X) - H(X \mid Y) = D(P_{XY} \,\|\, P_X P_Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)\,p(y)}. \tag{5.11}$$

It follows that mutual information is symmetric, $I(X;Y) = I(Y;X) = H(Y) - H(Y \mid X)$: Y tells you as much about X as X tells you about Y. It is non-negative, and it is zero if and only if X and Y are independent.

Sometimes a third variable Z is already known. The **conditional mutual information** is the same quantity computed with every distribution conditioned on $Z = z$, then averaged over z. It measures what Y adds about X beyond what Z already told you, and it is non-negative because it is an average of KL divergences:

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y, Z) = \sum_{z} p(z)\, D(P_{XY|z} \,\|\, P_{X|z} P_{Y|z}) \geq 0. \tag{5.12}$$

Chain rule for mutual information. What a pair (Y, Z) tells you about X is what Y tells you, plus what Z adds once Y is known:

$$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y). \tag{5.13}$$

Data-processing inequality. Suppose Z is computed from Y alone, possibly with fresh randomness, so that once you know Y, also knowing X tells you nothing more about Z. In

other words, X and Z are independent given Y. This is called a **Markov chain** and written $X \to Y \to Z$. Then Z cannot know more about X than Y does:

$$I(X;Z) \leq I(X;Y). \tag{5.14}$$

5.4.3. Derivation

Mutual information is a KL divergence. The idea is to write both entropies as averages over the same pairs (x, y), so that they can be combined term by term.

1. Start from $H(X) - H(X \mid Y)$. The entropy $H(X)$ is an average over x, but since $\sum_y p(x,y) = p(x)$ it can be written as an average over pairs: $H(X) = \sum_{x,y} p(x,y) \log \frac{1}{p(x)}$. The conditional entropy is already an average over pairs, $H(X \mid Y) = \sum_{x,y} p(x,y) \log \frac{1}{p(x|y)}$.
2. Subtract, combining the logs in each term: $\sum_{x,y} p(x,y) \log \frac{p(x|y)}{p(x)}$.
3. Multiply the fraction inside the log by $p(y)/p(y)$ and use $p(x \mid y)\, p(y) = p(x,y)$: $\frac{p(x|y)}{p(x)} = \frac{p(x,y)}{p(x)\, p(y)}$. The sum is now $\sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)\, p(y)}$, which is the definition of $D(P_{XY} \,\|\, P_X P_Y)$.

Symmetry, non-negativity, independence. The expression at the end of step 3 does not change when you swap the roles of X and Y, so $I(X;Y) = I(Y;X)$, and running the same steps with the roles swapped gives $H(Y) - H(Y \mid X)$. Gibbs' inequality (Fact 2) gives $I \geq 0$, with equality iff $p(x,y) = p(x)\, p(y)$ for all pairs, which is the definition of independence. Since $I(X;Y) = H(X) - H(X \mid Y) \geq 0$, this also proves $H(X \mid Y) \leq H(X)$.

Chain rule for mutual information. The idea is to pass from knowing nothing to knowing (Y, Z) through the intermediate stage of knowing only Y.

1. By definition, $I(X; Y, Z) = H(X) - H(X \mid Y, Z)$: the total drop in uncertainty.
2. Add and subtract the uncertainty at the intermediate stage, $H(X \mid Y)$: $\big[H(X) - H(X \mid Y)\big] + \big[H(X \mid Y) - H(X \mid Y, Z)\big]$.
3. The first bracket is the drop from learning Y, which is $I(X;Y)$. The second is the further drop from learning Z when Y is already known, which is $I(X; Z \mid Y)$.

Data-processing inequality. The idea is to expand what the pair (Y, Z) tells you about X in two orders. In one order, the part Z adds after Y is zero, because Z was computed from Y.

1. Apply the chain rule with Z first and with Y first:

$$I(X;Z) + I(X;Y \mid Z) = I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y). \tag{5.15}$$

2. The Markov chain says X and Z are independent given Y, so the conditional mutual information $I(X; Z \mid Y)$ is zero (it is an average of KL divergences between identical distributions).
3. Hence $I(X;Z) = I(X;Y) - I(X;Y \mid Z) \leq I(X;Y)$, since conditional mutual information is non-negative.

Equality holds iff $I(X;Y \mid Z) = 0$: once you know Z, Y tells you nothing more about X. Such a Z has kept everything in Y that matters for X, and is called a **sufficient statistic** for X.

5.4.4. Interpretation

Mutual information measures dependence of any kind, not only linear correlation. Two variables can have zero correlation and still be strongly dependent, as Y and Y^2 are when Y is symmetric around zero. Their mutual information is positive in that case, and it is zero only under full independence.

The data-processing inequality says that no function of the data, however clever, can contain more information about the source than the data itself. Denoising, feature extraction and training a model are all processing steps. They can make information easier to use, but never larger.

Because $I(X;Y) = D(P_{XY} \,\|\, P_X P_Y)$, every property of KL divergence in Divergences applies to mutual information. In particular, KL divergence has its own data-processing inequality there: passing two distributions through the same channel cannot increase the divergence between them. That inequality implies the one here.

Mutual information is an average over both x and y. A small value can therefore hide a few outcomes in which Y reveals X completely, because those outcomes carry little weight in the average. Section 5.8 shows how far apart the average and the worst case can be.

5.4.5. Example: the keyword, then a noisy detector

In the spam table, the word reduced the uncertainty from $H(X) = 0.5436$ bits to $H(X \mid Y) = 0.25$ bits on average, so $I(X;Y) = 0.5436 - 0.25 = 0.2936$ bits. The script computes the same value from the KL formula of Equation 5.11 and from $H(Y) - H(Y \mid X)$.

Now suppose the keyword is detected by a sloppy tokenizer that gets the answer wrong with probability 0.1, whatever the email. Call its output Z. A channel on $\{0, 1\}$ that flips its input with a fixed probability is a **binary symmetric channel**, studied in the next section, so Z is Y passed through a binary symmetric channel with flip probability 0.1. The tokenizer looks only at the word, not at the label, so $X \to Y \to Z$ is a Markov chain, and the data-processing inequality predicts $I(X;Z) \le I(X;Y)$. The script finds $I(X;Z) = 0.1668$ bits. Since $0.1668/0.2936 \approx 0.57$, the noisy feature carries 57% of the information of the clean one: a 10% error rate in the detector costs over 40% of the information.

5.4.6. Check yourself

- If $Z = g(Y)$ for an invertible function g, what is $I(X;Z)$? *(Equal to $I(X;Y)$. Apply the data-processing inequality to $X \to Y \to Z$, which gives $\le$, and to $X \to Z \to Y$, which is also a Markov chain because $Y = g^{-1}(Z)$ is computed from Z, and gives $\ge$.)*
- Can $I(X;Y)$ exceed $H(X)$? *(No. $I(X;Y) = H(X) - H(X \mid Y)$ and $H(X \mid Y) \ge 0$: an observation cannot remove more uncertainty than there was.)*

5.5. Channels and capacity

5.5.1. Context

A channel turns an input X into a random output Y: a noisy storage device, a communication link, a lossy sensor. You control the input distribution but not the noise. How much information can one use of the channel carry at best? That maximum is the **capacity**. Shannon's channel coding theorem says it is exactly the rate, in bits per channel use, at which you can send data with an error probability as small as you like, using long enough codes. This chapter does not prove that theorem; it computes the capacity, which is the number the theorem is about.

5.5.2. Statement

A channel is fixed, but how much information passes through it depends on how often you send each input, the prior π. So the capacity is the mutual information between input and output, maximized over the input prior. For a channel $C[x, y] = p(y \mid x)$,

$$\mathcal{C} = \max_{\pi} I(X;Y), \qquad X \sim \pi,\ Y \mid X = x \sim C[x, \cdot]. \tag{5.16}$$

The **binary symmetric channel** $\mathrm{BSC}(p)$ has inputs and outputs in $\{0, 1\}$ and flips the input with probability p. Its capacity is one bit, the most a binary output can carry, minus the entropy of the noise:

$$\mathcal{C}_{\mathrm{BSC}}(p) = 1 - h(p), \tag{5.17}$$

attained by the uniform input.

5.5.3. Derivation

The idea is to use the form $I(X;Y) = H(Y) - H(Y \mid X)$: the uncertainty in the output, minus the part of it that comes from the noise. The noise part turns out not to depend on the input prior, so you only have to make the output as uncertain as possible. Let the input prior be $q = \Pr[X = 1]$.

1. Use the symmetric form of mutual information: $I(X;Y) = H(Y) - H(Y \mid X)$.
2. Given $X = x$, the output equals x with probability $1-p$ and is flipped with probability p, so $H(Y \mid X = x) = h(p)$ for both inputs. Averaging over x gives $H(Y \mid X) = h(p)$, whatever q is.
3. Y takes two values, so by the entropy upper bound $H(Y) \le \log 2 = 1$, with equality iff Y is uniform.
4. The output is 1 if the input is 1 and not flipped, or the input is 0 and flipped, so $\Pr[Y = 1] = q(1-p) + (1-q)p$. At $q = \frac{1}{2}$ this is $\frac{1}{2}(1-p) + \frac{1}{2}p = \frac{1}{2}$, so the uniform input makes Y uniform and the bound in step 3 is attained.
5. Hence $\max_q I(X;Y) = 1 - h(p)$.

5.5.4. Interpretation

Figure 5.3 (left) shows $I(X;Y)$ as a function of the input prior q: a concave hump peaking at $q = \frac{1}{2}$ for every p. Away from the peak you lose a lot. At $p = 0.11$ and $q = 0.2$, the output is 1 with probability $0.2(0.89) + 0.8(0.11) = 0.266$, so $H(Y) = h(0.266) = 0.8357$ and the channel carries only $0.8357 - 0.4999 = 0.3358$ bits, against 0.5001 at the uniform input.

The right panel shows the capacity against p. It is zero at $p = \frac{1}{2}$, where the output is a fair coin flip whatever the input, so the output is independent of the input. It is symmetric about $\frac{1}{2}$, because $h(p) = h(1-p)$: a channel that flips almost always is as good as one that never flips, since the receiver can flip every output back.

For channels without this symmetry, the maximum over π has no closed form. The **Blahut–Arimoto** algorithm computes it numerically, by alternately improving the prior and the receiver's best reading of the output until neither changes. The script uses it to check Equation 5.17, and to compute two capacities in the exercises in the exercise book.

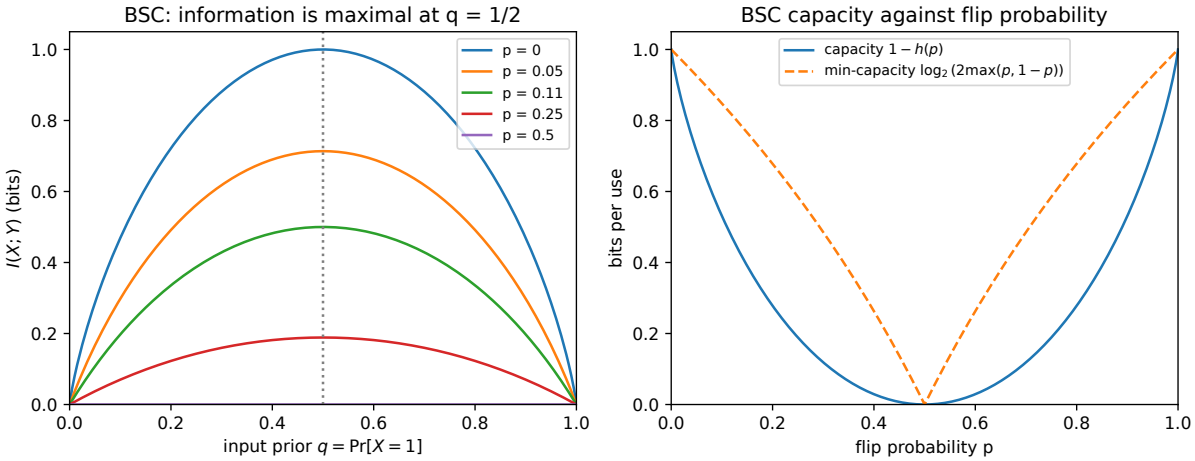


Figure 5.3.: Left: mutual information of a binary symmetric channel as a function of the input prior, for several flip probabilities; every curve peaks at $q = \frac{1}{2}$. Right: the capacity $1 - h(p)$ against p, with the min-capacity of Section 5.7 for comparison.

5.5.5. Example: a storage medium that flips bits

A cheap storage medium flips each stored bit independently with probability $p = 0.11$. Its capacity is $1 - h(0.11) = 1 - 0.4999 = 0.5001$ bits per stored bit. So with good error-correcting codes you can store about one bit of data per two raw bits, with an error rate as small as you like, and by the channel coding theorem no code can do better. At $p = 0.25$ the capacity drops to $1 - h(0.25) = 1 - 0.8113 = 0.1887$. Since $1/0.1887 \approx 5.3$, that is about five raw bits per data bit.

5.5.6. Check yourself

- Why does the derivation not need to compute $H(Y)$ for a general q? *(Because $H(Y \mid X)$ does not depend on q, so maximizing I means maximizing $H(Y)$, and its largest possible value 1 is attainable.)*
- What is the capacity of a channel whose output is independent of its input? *(Zero: $I(X;Y) = 0$ for every prior, because mutual information is zero exactly under independence.)*

5.6. Min-entropy and guessing

5.6.1. Context

Shannon entropy counts the bits needed to describe a value on average. A different question is how likely you are to guess it in **one try**. A password chosen by a person might have high entropy, because most people choose something unusual, yet be guessed one time in ten, because one popular choice dominates. For the one-try question, all that matters is the probability of the most likely value: that is the best you can do with one guess.

5.6.2. Statement

The **vulnerability** of X is the success probability of the best single guess, the largest probability of any value. To put it on the same bit scale as entropy, take its negative log; that is the **min-entropy**:

$$V(X) = \max_x p(x), \qquad H_\infty(X) = -\log V(X). \tag{5.18}$$

A fair coin has $V = \frac{1}{2}$ and $H_\infty = 1$ bit, the same as its Shannon entropy. In general, min-entropy uses the surprisal of the most likely outcome, which is the smallest surprisal, while Shannon entropy averages all surprisals. So min-entropy is never larger than Shannon entropy:

$$H_\infty(X) \le H(X) \le \log|\mathcal{X}|, \tag{5.19}$$

with equality in the first inequality iff X is uniform on its **support**, the set of values it takes with positive probability.

5.6.3. Derivation

The best single guess. If you guess g, you are right with probability $p(g)$. The best you can do is pick g with the largest $p(g)$, so the optimal success probability is $\max_x p(x)$.

$H_\infty \le H$. The idea is that an average of numbers is at least their smallest value.

1. Every outcome has $p(x) \le \max_{x'} p(x') = V(X)$, so its surprisal satisfies $\log \frac{1}{p(x)} \ge \log \frac{1}{V(X)}$.
2. Average over X: $H(X) = \mathbb{E}[\log \frac{1}{p(X)}] \ge \log \frac{1}{V(X)} = H_\infty(X)$.
3. Equality needs every surprisal to equal the smallest one, $p(x) = V(X)$ for every x with $p(x) > 0$: all outcomes in the support are equally likely.

5.6.4. Interpretation

Min-entropy has a direct reading: H_∞ is the number of bits of a uniform distribution that is exactly as easy to guess as X. A uniform k-bit string is guessed in one try with probability 2^{-k}, so a value with $H_\infty = k$ bits is as hard to guess in one try as a uniform k-bit string, no matter how large $H(X)$ is.

Both entropies belong to one family. The **Rényi entropy** of order α is $H_\alpha(X) = \frac{1}{1-\alpha} \log \sum_x p(x)^\alpha$. As α grows, the sum $\sum_x p(x)^\alpha$ is dominated more and more by the largest probability, and as $\alpha \to \infty$ the formula tends to $-\log \max_x p(x) = H_\infty$. As $\alpha \to 1$ the formula becomes $0/0$, and its limit is Shannon's H. This parallels the family of Rényi divergences in Divergences.

For a coin, Figure 5.1 shows the gap. The min-entropy $H_\infty = -\log \max(p, 1-p)$ is smooth on each side of $\frac{1}{2}$ but has a kink there, where the most likely outcome switches from tails to heads, and it lies below $h(p)$.

5.6.5. Example: a PIN with one popular choice

A uniform 4-digit PIN has $H = H_\infty = \log_2 10{,}000 = 13.2877$ bits, and a single guess succeeds with probability 10^{-4}. Now suppose 10% of people choose 1234 and the rest choose uniformly among the other 9,999 PINs. The script gives two very different numbers.

The Shannon entropy is $H = 12.4278$ bits, less than a bit below the uniform case. You can see where it comes from with the chain rule: first reveal whether the PIN is 1234, which costs $h(0.1) = 0.4690$ bits, and then, in the 90% of cases where it is not, reveal which of the 9,999 others it is, which costs $\log_2 9{,}999$ bits. That gives $0.4690 + 0.9 \log_2 9{,}999 = 0.4690 + 11.9588 = 12.4278$.

The min-entropy is $H_\infty = \log_2 10 = 3.3219$ bits, because the best guess, "1234", succeeds with probability 0.1, and $-\log_2 0.1 = \log_2 10$.

The Shannon entropy says this PIN distribution is almost as good as uniform. The min-entropy says that one guess in ten succeeds. If the guesser gets one try, only the second number matters.

5.6.6. Check yourself

- Which is larger for any X: $V(X)$ or $2^{-H(X)}$? *($V(X)$. $H_\infty \le H$ means $-\log V \le H$, so $\log V \ge -H$ and $V \ge 2^{-H}$.)*
- Give a distribution on a million values whose Shannon entropy is above 10 bits but whose min-entropy is 1 bit. *(Probability $\frac{1}{2}$ on one value and $\frac{1}{2}$ spread uniformly over the other 999,999. The best guess wins with probability $\frac{1}{2}$, so $H_\infty = 1$, while by the chain rule $H = 1 + \frac{1}{2} \log_2 999{,}999 \approx 10.97$: one bit to say which half, then, half the time, the index among 999,999.)*

5.7. Conditional min-entropy and min-entropy leakage

5.7.1. Context

Now the guesser sees an observation Y before guessing the hidden value X, which this section calls the secret. How much does that help? The natural answer, from Smith (2009), keeps the meaning of vulnerability as a success probability: the posterior vulnerability is the success probability of the best guessing strategy that may depend on what was observed. Comparing it with the vulnerability before the observation gives the **min-entropy leakage**, and maximizing that over priors gives the **min-capacity**, the one-guess analogue of channel capacity.

5.7.2. Statement

The guesser who sees y should guess the value of X that is most likely given y. Its success probability is then the average over y of $\max_x p(x \mid y)$, and since $p(y) \max_x p(x \mid y) = \max_x p(x, y)$, this is the sum of the largest entry in each column of the joint table. For a joint distribution $p(x, y)$, the **posterior vulnerability** and the **conditional min-entropy** are

$$V(X \mid Y) = \sum_y \max_x p(x, y), \qquad H_\infty(X \mid Y) = -\log V(X \mid Y). \tag{5.20}$$

How much the observation helps is the factor by which it multiplies the chance of a correct guess, measured in bits by its log. For a prior π and channel C, the **min-entropy leakage** is

$$\mathcal{L}(\pi, C) = H_\infty(X) - H_\infty(X \mid Y) = \log \frac{V(X \mid Y)}{V(X)}. \tag{5.21}$$

The leakage depends on the prior. Its largest value over all priors, the **min-capacity**, depends only on the channel. It has a closed form, the log of the sum of the column maxima of the channel table, and it is attained at the uniform prior:

$$\mathcal{ML}(C) = \max_\pi \mathcal{L}(\pi, C) = \log \sum_y \max_x C[x, y]. \tag{5.22}$$

It satisfies $0 \le \mathcal{ML}(C) \le \log \min(|\mathcal{X}|, |\mathcal{Y}|)$: an observation never hurts the guesser, and it cannot multiply the chance of success by more than the number of possible secrets or the number of possible observations.

5.7.3. Derivation

Posterior vulnerability is the success of the best strategy. The idea is that the guesser's choice for one observation does not affect its success on any other observation, so it can make the best choice column by column.

1. A strategy assigns a guess $g(y)$ to each observation y. It succeeds when $X = g(Y)$, which happens with probability $\sum_y p(g(y), y)$: the strategy picks one cell from each column of the joint table, and its success is the sum of the picked cells.

2. Each column can be optimized separately, since $g(y)$ affects only the term for y. The best choice in column y is the row with the largest entry. Dividing a column by $p(y)$ does not change which row is largest, so this is also the value of x with the largest posterior probability $p(x \mid y)$, the **maximum a posteriori (MAP) guess** $\arg\max_x p(x, y) = \arg\max_x p(x \mid y)$.
3. So the best success probability is $\sum_y \max_x p(x, y)$, which is Equation 5.20. Writing $p(x, y) = p(y)\, p(x \mid y)$ gives the equivalent form $V(X \mid Y) = \sum_y p(y) \max_x p(x \mid y)$: the average over y of the vulnerability $V(X \mid Y = y)$ of the posterior distribution.

Min-capacity. The idea is to bound the posterior vulnerability by pulling the prior out of each column maximum, and then to notice that the uniform prior loses nothing in that step.

1. Write $p(x, y) = \pi(x)C[x, y]$. In each column, the largest product is at most the largest prior times the largest channel entry: $\max_x \pi(x)C[x, y] \le (\max_x \pi(x))(\max_x C[x, y]) = V(X) \max_x C[x, y]$.
2. Sum over y and divide by $V(X)$: $\frac{V(X|Y)}{V(X)} \le \sum_y \max_x C[x, y]$, for every prior. Taking logs, $\mathcal{L}(\pi, C) \le \log \sum_y \max_x C[x, y]$.
3. At the uniform prior $\pi(x) = 1/|\mathcal{X}|$, the inequality in step 1 is an equality, because the constant factor $\pi(x) = V(X)$ comes out of the max. So the bound is attained, and Equation 5.22 follows.
4. *Range.* For the upper bound, there are two ways to bound the sum of column maxima. Each column maximum is at most 1, so the sum is at most $|\mathcal{Y}|$. Each column maximum is also at most its column sum, and all column sums together add up to $\sum_x \sum_y C[x, y] = |\mathcal{X}|$, because each of the $|\mathcal{X}|$ rows sums to 1. For the lower bound, fix any row x_0: each column maximum is at least $C[x_0, y]$, and these add up to 1 across the row, so the sum of column maxima is at least 1 and $\mathcal{ML} \ge \log 1 = 0$.

5.7.4. Interpretation

Leakage is a ratio of success probabilities on a log scale: $\mathcal{L} = 1$ bit means the observation doubles the chance of a correct first guess, and 2 bits means it quadruples it. That is a statement an engineer can act on, and it is why min-entropy is the usual measure in the field of quantitative information flow, which studies how much a program's outputs reveal about its secret inputs.

Smith's definition is not the average of $H_\infty(X \mid Y = y)$. It averages the posterior vulnerabilities and then takes the log, $-\log \sum_y p(y) V(X \mid Y = y)$, rather than averaging the logs, $\sum_y p(y)(-\log V(X \mid Y = y))$. These differ, and Jensen's inequality (Fact 1, in the mirror version for the convex $-\log$: the $-\log$ of an average is at most the average of $-\log$) says Smith's value is never larger. The reason to prefer Smith's version is its meaning: it is the log of an actual success probability, that of the MAP guesser. The average of logs is not the log of any success probability.

The min-capacity is the worst case over the prior. That matters whenever you do not know the prior: a single number, computed from the channel alone, bounds the leakage for every prior. It is also cheap to compute, one maximum per column, where the Shannon capacity needs an iterative algorithm.

5.7.5. Example: walking through a small table

A secret X takes four values with prior $\pi = (0.4, 0.3, 0.2, 0.1)$. An observation Y takes three values, through the channel in Table 5.3.

Table 5.3.: The channel of the guessing walk-through: rows are secret values, columns are observations.

$C[x, y]$	y_1	y_2	y_3
x_1	0.8	0.2	0
x_2	0.1	0.8	0.1
x_3	0	0.5	0.5
x_4	0	0.2	0.8

Multiply each row by its prior to get the joint table, Table 5.4, also drawn in Figure 5.4. For example, the first row is $0.4 \times (0.8, 0.2, 0) = (0.32, 0.08, 0)$.

Table 5.4.: The joint distribution $p(x, y) = \pi(x)C[x, y]$ of the walk-through, with the column maxima in bold.

$p(x, y)$	y_1	y_2	y_3
x_1	**0.32**	0.08	0
x_2	0.03	**0.24**	0.03
x_3	0	0.10	**0.10**
x_4	0	0.02	0.08
$p(y)$	0.35	0.44	0.21

1. **Without the observation**, the best guess is the most likely value x_1, right with probability $V(X) = 0.4$. So $H_\infty(X) = -\log_2 0.4 = 1.3219$ bits.
2. **With the observation**, guess the row with the largest entry in each column (in bold): x_1 on y_1, x_2 on y_2, x_3 on y_3. The success probability is $0.32 + 0.24 + 0.10 = 0.66 = V(X \mid Y)$, and $H_\infty(X \mid Y) = -\log_2 0.66 = 0.5995$ bits. Notice that on y_3 the guesser picks x_3 although the channel entry $C[x_4, y_3] = 0.8$ is larger than $C[x_3, y_3] = 0.5$. The guesser compares joint probabilities, not channel entries, and x_3 is twice as likely as x_4 a priori, which more than makes up the difference: $0.2 \times 0.5 = 0.10$ beats $0.1 \times 0.8 = 0.08$.
3. **Simulation.** The script draws (X, Y) a million times and lets the MAP guesser play; it wins 0.6594 of the time, close to the predicted 0.66.
4. **Leakage.** $\mathcal{L} = \log_2(0.66/0.4) = 0.7225$ bits: the observation multiplies the guessing success by $0.66/0.4 = 1.65$. For comparison, the Shannon leakage of the same table, the mutual information, is $I(X; Y) = 0.6865$ bits out of $H(X) = 1.8464$.
5. **Min-capacity.** The column maxima of the channel C (not of the joint table) are 0.8, 0.8, 0.8, so $\mathcal{ML}(C) = \log_2(0.8 + 0.8 + 0.8) = \log_2 2.4 = 1.2630$ bits. Under the uniform prior, the leakage is exactly this value, and the script confirms that no prior on a grid of step 0.05 does better. The Shannon capacity of the same channel, from Blahut–Arimoto, is 0.8269 bits.

6. **Smith versus the average of logs.** The posterior vulnerabilities are the column maxima divided by the column totals, 0.32/0.35, 0.24/0.44 and 0.10/0.21. Their $-\log_2$ are 0.1293, 0.8745 and 1.0704 bits, and averaging these with weights $p(y) = 0.35, 0.44, 0.21$ gives 0.6548 bits, larger than Smith's 0.5995, as Jensen predicts.

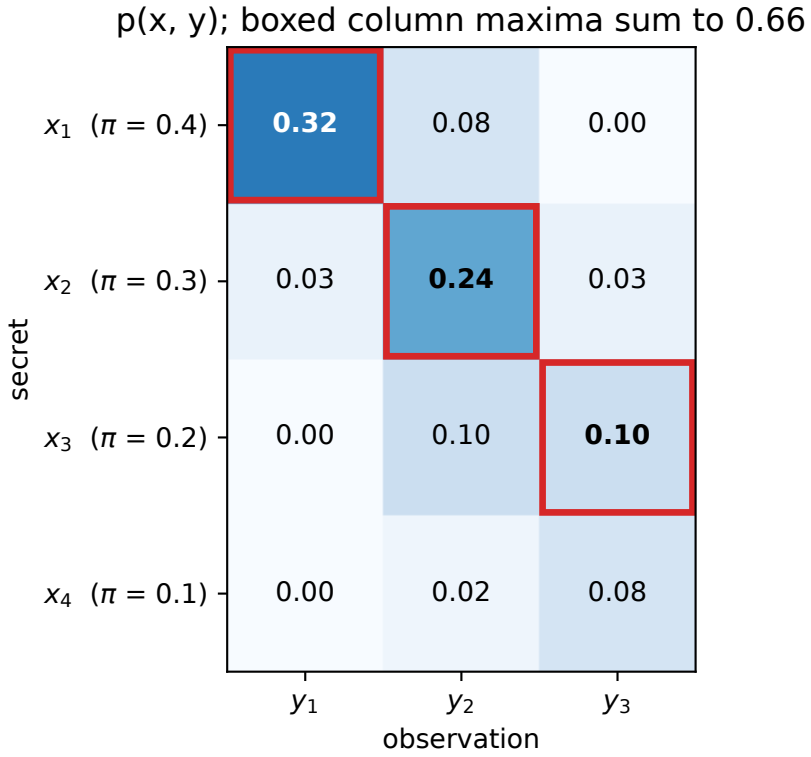


Figure 5.4.: The joint distribution $p(x, y) = \pi(x)C[x, y]$ of the walk-through. The MAP guesser picks the boxed cell in each column, and its success probability is their sum $V(X \mid Y) = 0.66$.

5.7.6. Check yourself

- What is the min-capacity of the binary symmetric channel? *($\log_2(2\max(p, 1-p))$: each of the two columns has maximum $\max(p, 1-p)$. At $p = 0.11$ it is $\log_2 1.78 = 0.8319$ bits, against a Shannon capacity of 0.5001. It is the dashed curve in Figure 5.3.)*
- Why can the min-entropy leakage for a given prior be smaller than the min-capacity? *(A skewed prior already makes the best blind guess good, so the observation adds less. Here 0.7225 bits at the given prior against 1.2630 at the uniform one.)*

5.8. Shannon leakage versus min-entropy leakage

5.8.1. Context

Shannon leakage, the mutual information $I(X;Y)$, and min-entropy leakage $\mathcal{L}$ both measure how much an observation reveals about a secret. They often agree. When they don't, the disagreement is large, and it is always the same kind: a channel that reveals the secret completely on a few outcomes and nothing on the rest. Shannon leakage averages logs, so it gives those rare outcomes little weight; min-entropy leakage asks how much the guesser's success grows, and a rare full reveal can make it grow a lot. This section works through such channels.

5.8.2. Statement

Let X be uniform on n-bit strings, and consider two families of **deterministic** channels, whose output is a fixed function of the input. The first reveals part of the secret every time; the second reveals all of it, but only rarely.

- **reveal-low(m)** always outputs the lowest m bits of X.
- **reveal-if-zero(j)** outputs X itself if its lowest j bits are all zero, which happens with probability 2^{-j}, and a fixed symbol $\perp$ otherwise.

For the first family both leakages equal m. For the second, the Shannon leakage is about n bits times the probability 2^{-j} of a reveal, plus a small term, while the min-entropy leakage is about $n - j$ bits:

$$\text{reveal-low}(m): \quad I(X;Y) = \mathcal{L} = m, \tag{5.23}$$

$$\text{reveal-if-zero}(j),\ j \geq 1: \quad I(X;Y) = 2^{-j}n + (1-2^{-j})\log\frac{1}{1-2^{-j}}, \qquad \mathcal{L} = \log\left(2^{n-j}+1\right). \tag{5.24}$$

5.8.3. Derivation

Two observations about deterministic channels make both leakages easy to compute. For Shannon leakage: the output is fixed once the input is known, so $H(Y \mid X) = 0$ and $I(X;Y) = H(Y) - H(Y \mid X) = H(Y)$. For min-entropy leakage: under the uniform prior, $V(X) = 2^{-n}$, and each column of the joint table contains only entries 2^{-n} (for inputs that produce that output) or 0. So each column maximum is 2^{-n}, and $V(X \mid Y) = 2^{-n} \times$ (number of distinct outputs). The ratio is $V(X \mid Y)/V(X) =$ number of distinct outputs, so $\mathcal{L} = \log$(number of distinct outputs).

reveal-low(m). The lowest m bits of a uniform string are uniform on 2^m values, so $I(X;Y) = H(Y) = m$. There are 2^m distinct outputs, so $\mathcal{L} = m$.

reveal-if-zero(j).

1. Count the outputs and their probabilities. There are 2^{n-j} strings whose lowest j bits are zero (the other $n-j$ bits are free). Each is output as itself, with probability 2^{-n}. Every other string gives the output $\perp$, which therefore has probability $1 - 2^{n-j} \cdot 2^{-n} = 1 - 2^{-j}$.

2. Compute $H(Y)$ as the average surprisal. The 2^{n-j} revealed outputs each have surprisal $\log 2^n = n$, and $\perp$ has surprisal $\log \frac{1}{1-2^{-j}}$. So $H(Y) = 2^{n-j} \cdot 2^{-n} \log 2^n + (1-2^{-j}) \log \frac{1}{1-2^{-j}} = 2^{-j}n + (1-2^{-j}) \log \frac{1}{1-2^{-j}}$.
3. The number of distinct outputs is the 2^{n-j} revealed strings plus $\perp$, which is $2^{n-j}+1$, and its log gives $\mathcal{L}$.

The script checks both closed forms against the general formulas applied to the explicit channel matrices for $n = 6$; they agree to 7.8×10^{-16}, which is rounding error.

5.8.4. Interpretation

Figure 5.5 plots the two leakages against each other for $n = 64$. The reveal-low family lies on the diagonal: Shannon and min-entropy agree. The reveal-if-zero family hugs the top-left corner: almost no Shannon leakage, almost total min-entropy leakage.

The reason is where the logarithm sits. Shannon leakage averages the log over outcomes, so an outcome of probability 2^{-j} that reveals n bits contributes only $2^{-j}n$. Min-entropy leakage takes the log of an averaged probability. The same outcome raises the guessing success from 2^{-n} to more than 2^{-j}, because the guesser wins whenever the reveal happens, and $\log(2^{-j}/2^{-n}) = n - j$: almost $n - j$ bits.

5.8.5. Example: two channels that Shannon ranks the wrong way

Take $n = 64$ and compare reveal-if-zero(3), which outputs the whole secret one time in eight, with reveal-low(9).

For reveal-if-zero(3), the Shannon leakage is $2^{-3} \cdot 64 + \frac{7}{8} \log_2 \frac{8}{7} = 8 + 0.1686 = 8.1686$ bits. The guesser wins whenever the secret is revealed, and otherwise has one chance in $2^{64} - 2^{61}$, so its success is just above $\frac{1}{8} = 0.1250$; in bits, $\log_2(2^{61}+1)$ is 61 to many decimal places. For reveal-low(9), both leakages are 9 bits, and the guesser, knowing 9 of the 64 bits, must still guess the other 55, so it succeeds with probability $2^{-55} \approx 2.78 \times 10^{-17}$. Table 5.5 puts these side by side.

Table 5.5.: Shannon and min-entropy leakage of two deterministic channels on a uniform 64-bit secret.

Channel	Shannon leakage	Min-entropy leakage	One-guess success
reveal-if-zero(3)	8.1686 bits	61.0000 bits	0.1250
reveal-low(9)	9.0000 bits	9.0000 bits	2.78×10^{-17}

By Shannon leakage, reveal-low(9) is the leakier channel. By one-guess success, reveal-if-zero(3) is worse by a factor of more than 10^{15}: $0.125/(2.78 \times 10^{-17}) \approx 4.5 \times 10^{15}$. Smith (2009) used a pair of programs like these to argue that Shannon leakage is the wrong measure for one-try guessing.

The extreme case: reveal the secret only if it is all zeros. Set $j = n$: the channel outputs X if $X = 0 \cdots 0$ and $\perp$ otherwise. For a 4-bit secret, the output reveals the secret with probability $\frac{1}{16}$, so the Shannon leakage is $H(Y) = h(1/16) = 0.3373$ bits. The min-entropy leakage is exactly

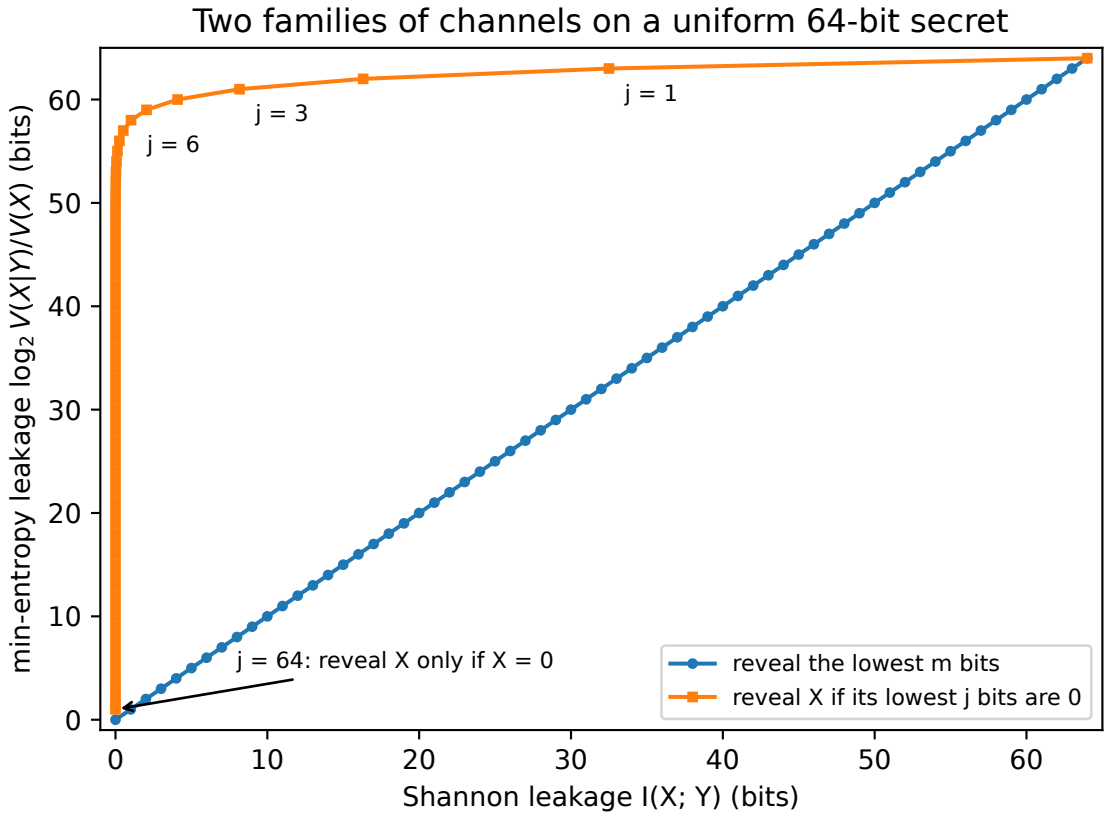


Figure 5.5.: Shannon leakage against min-entropy leakage for two families of deterministic channels on a uniform 64-bit secret. Revealing the low m bits gives equal leakages (diagonal). Revealing the whole secret with probability 2^{-j} gives a small Shannon leakage but a min-entropy leakage close to $64 - j$.

1 bit: the guesser wins if it sees 0000, and otherwise guesses one of the 15 remaining values, winning with probability $\frac{1}{16} + \frac{15}{16} \cdot \frac{1}{15} = \frac{2}{16}$. Its success doubles from $\frac{1}{16}$ to $\frac{2}{16}$, and $\log_2 2 = 1$. For a 32-bit secret the Shannon leakage falls to $h(2^{-32}) = 7.786 \times 10^{-9}$ bits, and the min-entropy leakage stays at 1 bit, by the same doubling argument.

Both numbers are small, and both miss something: for the one secret value 0000, the channel reveals everything, $H_\infty(X \mid Y = 0000) = 0$. Any measure that averages over the secret, Shannon or min-entropy, can hide a value that is fully exposed, because that value carries only its small share of the average. A worst-case question ("how much can the observation reveal about any single secret value?") needs a worst-case measure, and later parts build one.

5.8.6. Check yourself

- For reveal-if-zero(j) at $n = 64$, roughly where does the Shannon leakage drop below 1 bit? *(Between $j = 6$ and $j = 7$: the main term $2^{-j} \cdot 64$ equals 1 at $j = 6$, where the total is just above 1 bit because of the small $\perp$ term. That is the point labelled $j = 6$ in Figure 5.5.)*
- Can a channel have min-entropy leakage 0 but positive Shannon leakage? *(Yes. If the MAP guess is the same for every y, as when one secret value is very likely a priori, then $V(X \mid Y) = V(X)$, even though Y changes the posterior.)*

5.9. Fano's inequality

5.9.1. Context

Mutual information also gives **lower bounds**: statements that *no* estimator can do better than some error rate. You need these whenever you want to show that a problem is hard, rather than that a method is good. Fano's inequality is the basic tool. It says that if X is still uncertain after seeing Y, every guess of X from Y must often be wrong. The reason is that an estimator with a small error rate would leave little uncertainty: if you knew you were usually right, you would be nearly certain of X.

5.9.2. Statement

Let X take values in a finite set of size M, and let $\hat{X}$ be any estimate of X computed from Y (deterministically or with extra randomness), with error probability $P_e = \Pr[\hat{X} \neq X]$. Fano's inequality bounds how much uncertainty a guess with this error rate can leave. Describing X given the guess takes at most two pieces: one binary piece saying whether the guess was right, worth at most $h(P_e)$ bits, and, in the fraction P_e of cases where it was wrong, which of the other $M - 1$ values X is, worth at most $\log(M - 1)$ bits. So

$$H(X \mid Y) \leq h(P_e) + P_e \log(M - 1). \tag{5.25}$$

Using $h(P_e) \leq 1$ and $\log(M - 1) \leq \log M$ gives $H(X \mid Y) \leq 1 + P_e \log M$, and solving for P_e gives the weaker but explicit form

$$P_e \geq \frac{H(X \mid Y) - 1}{\log M}. \tag{5.26}$$

5.9.3. Derivation

Introduce the error indicator $E = \mathbf{1}[\hat{X} \neq X]$, which is 1 when the guess is wrong and 0 when it is right, so $\Pr[E = 1] = P_e$. The idea is to compute the uncertainty of the pair (E, X) given $\hat{X}$, written $H(E, X \mid \hat{X})$, in two ways with the chain rule. Revealing X first makes E free; revealing E first splits X's uncertainty into the "right" and "wrong" cases, which are easy to bound. Every entropy below is conditioned on $\hat{X}$.

1. **First order: X, then E.** $H(E, X \mid \hat{X}) = H(X \mid \hat{X}) + H(E \mid X, \hat{X})$. Given both X and $\hat{X}$, you know whether they differ, so E is determined and the last term is 0.
2. **Second order: E, then X.** $H(E, X \mid \hat{X}) = H(E \mid \hat{X}) + H(X \mid E, \hat{X})$.
3. **Bound $H(E \mid \hat{X})$.** Conditioning reduces entropy on average, so $H(E \mid \hat{X}) \leq H(E)$, and E is a Bernoulli variable with parameter P_e, so $H(E) = h(P_e)$.
4. **Bound $H(X \mid E, \hat{X})$** by splitting on E. If $E = 0$, then $X = \hat{X}$ and there is no uncertainty left. If $E = 1$, then X is one of the $M-1$ values other than $\hat{X}$, so its entropy is at most $\log(M-1)$ by the entropy upper bound. Averaging over E: $H(X \mid E, \hat{X}) \leq (1 - P_e) \cdot 0 + P_e \log(M-1)$.
5. **Combine** steps 1–4. The two orders give the same quantity, so $H(X \mid \hat{X}) = H(E \mid \hat{X}) + H(X \mid E, \hat{X}) \leq h(P_e) + P_e \log(M-1)$.
6. **Replace $\hat{X}$ by Y.** The estimate is computed from Y, so $X \to Y \to \hat{X}$ is a Markov chain, and the data-processing inequality gives $I(X; \hat{X}) \leq I(X; Y)$. Writing both sides as $H(X)$ minus a conditional entropy, this is the same as $H(X \mid Y) \leq H(X \mid \hat{X})$, and the bound of step 5 carries over.

5.9.4. Interpretation

The right side of Equation 5.25 increases with P_e on $[0, (M-1)/M]$, the range from a perfect estimator to one that guesses blindly. So you can invert it numerically: the **Fano bound** is the smallest P_e that satisfies the inequality for the given $H(X \mid Y)$. The script finds it by bisection, halving an interval that contains it until it is small enough.

When it is tight. Equality needs step 4 to be tight: when the guess is wrong, the true value must be uniform over the other $M-1$ values. The M-ary **symmetric channel**, which keeps the input with probability $1-\varepsilon$ and otherwise outputs one of the other $M-1$ symbols uniformly, has exactly this structure. For it, given the output, the input is the output itself with probability $1-\varepsilon$ and otherwise uniform over the rest, so $H(X \mid Y) = h(\varepsilon) + \varepsilon \log(M-1)$, and the MAP guess (the output itself) errs with probability ε. Fano's bound is exact (Figure 5.6, left).

When it is loose. If errors are concentrated on a few wrong values, $H(X \mid Y)$ is small even when P_e is not, because the leftover uncertainty after a wrong guess is small. The **shift channel**, which outputs $x + 1 \bmod M$ instead of x with probability ε, has the same MAP error ε (for $\varepsilon \leq \frac{1}{2}$). But given the output, the input is one of only two values, so $H(X \mid Y) = h(\varepsilon)$ only. At $M = 8$ and $\varepsilon = 0.2$, Fano's bound is 0.2000 for the symmetric channel, equal to the MAP error, but only 0.0954 for the shift channel, against a MAP error of 0.2000 (Figure 5.6, right). The weak form Equation 5.26 gives 0.0945 and 0 respectively. For the symmetric channel, $H(X \mid Y) = h(0.2) + 0.2 \log_2 7 = 0.7219 + 0.5615 = 1.2834$, and $(1.2834 - 1)/3 = 0.0945$. For the shift channel, $H(X \mid Y) = h(0.2) = 0.7219$ is below 1, so the weak form gives a negative number, which says nothing beyond $P_e \geq 0$. The weak form pays for replacing $h(P_e)$ by 1, and is useless whenever $H(X \mid Y) \leq 1$.

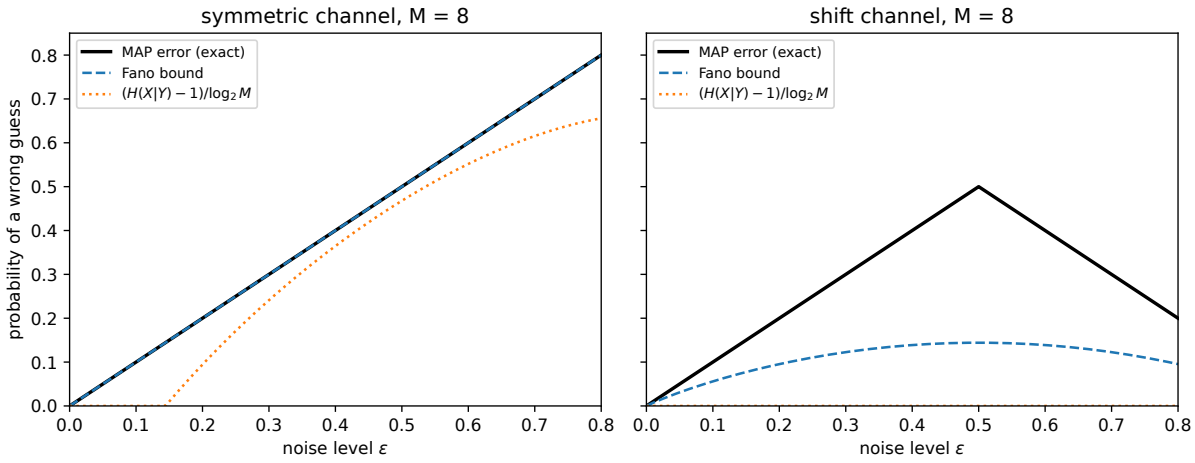


Figure 5.6.: MAP error against Fano's lower bound, for $M = 8$ and a uniform X, as the noise level ε grows. Left: on the symmetric channel the bound is exact. Right: on the shift channel, where errors go to a single wrong value, the bound is loose and the weak form is zero.

5.9.5. Example: how hard is a classification problem?

Eight classes and one bit. A label X is uniform over $M = 8$ classes, so $H(X) = \log_2 8 = 3$ bits, and you know the features Y carry at most $I(X;Y) \le 1$ bit about it. Then $H(X \mid Y) = 3 - I(X;Y) \ge 2$ bits. Every classifier, however it is trained, therefore has error at least the Fano bound at $H(X \mid Y) = 2$, the P_e that solves $h(P_e) + P_e \log_2 7 = 2$: 0.3730. The weak form gives $(2-1)/3 = 0.3333$.

Fano's method for estimation. The same idea shows that estimating a parameter needs many samples. Turn estimation into identification: pick M candidate parameter values $\theta_1, \dots, \theta_M$ that are far apart, so that estimating the parameter well would identify which candidate it is. Let X be a uniformly random index, and draw n independent samples $Y_1, \dots, Y_n$ (together written Y^n) from P_{θ_X}. If the samples carry little information about the index, Fano says no estimator can identify it, so none can estimate the parameter well. The argument has three steps.

1. **Information adds over independent samples.** Given X the samples are independent, so the chain rule gives $H(Y^n \mid X) = \sum_i H(Y_i \mid X)$ exactly. Without conditioning on X, the chain rule plus "conditioning reduces entropy" gives only $H(Y^n) \le \sum_i H(Y_i)$. Subtracting the first from the second, $I(X;Y^n) \le \sum_i I(X;Y_i) = n\, I(X;Y_1)$, since all samples have the same distribution.
2. **Each sample carries at most the largest pairwise KL divergence.** Let $\bar{P} = \frac{1}{M}\sum_x P_{\theta_x}$ be the average of the candidate distributions, which is the distribution of Y_1 when the index is random. By the KL form of Equation 5.11, $I(X;Y_1)$ is the average KL divergence from each candidate to this average, $\frac{1}{M}\sum_x D(P_{\theta_x} \,\|\, \bar{P})$. Expanding the logs shows that for any distribution Q, $\frac{1}{M}\sum_x D(P_{\theta_x} \,\|\, Q) = \frac{1}{M}\sum_x D(P_{\theta_x} \,\|\, \bar{P}) + D(\bar{P} \,\|\, Q)$, so the average divergence to $\bar{P}$ is the smallest average divergence to any Q. Taking $Q = P_{\theta_{x'}}$ gives $I(X;Y_1) \le \frac{1}{M}\sum_x D(P_{\theta_x} \,\|\, P_{\theta_{x'}})$ for each x', and hence $I(X;Y_1) \le \kappa = \max_{x,x'} D(P_{\theta_x} \,\|\, P_{\theta_{x'}})$.

3. **Apply Fano.** The index is uniform, so $H(X) = \log M$ and $H(X \mid Y^n) = \log M - I(X;Y^n) \geq \log M - n\kappa$. Putting this into Equation 5.26 gives

$$P_e \geq 1 - \frac{n\kappa + 1}{\log M}. \tag{5.27}$$

With $M = 1024$ candidates ($\log M = 10$ bits) whose sample distributions are within $\kappa = 0.01$ bits of each other, $n = 100$ samples leave $P_e \geq 1 - (100 \times 0.01 + 1)/10 = 0.8000$, and even $n = 400$ samples leave $P_e \geq 1 - (4 + 1)/10 = 0.5000$. Choosing the candidates well, so that they are many and far apart yet hard to tell apart, is the art of the method; Wainwright (2019) works through many cases.

5.9.6. Check yourself

- If X is a deterministic function of Y, what does Fano's inequality say? *($H(X \mid Y) = 0$, so it allows $P_e = 0$, as it should.)*
- Why does step 6 of the derivation need the data-processing inequality? *(Steps 1–5 only bound $H(X \mid \hat{X})$. The bound must be stated in terms of what the estimator sees, Y, and the data-processing inequality says $\hat{X}$ knows no more about X than Y does.)*

5.10. Code: check every formula numerically

The script code/information_theory.py computes every number quoted in this chapter and draws its figures. Entropy, mutual information, vulnerability, leakage, capacity and the Fano bound are small typed functions of a joint table or a channel matrix; the script calls them once for the numbers and loops them over parameters for the figures. From the repository root:

```
uv run python code/part01-prerequisites/information_theory.py
```

Table 5.6 summarizes the output (seed 0; only the guessing simulation uses the seed).

Table 5.6.: Output of the chapter's script with seed 0. All quantities are in bits except probabilities.

Experiment	Result
Spam keyword	$H(X) = 0.5436$, $H(X \mid Y = 1) = 1.0000$, $H(X \mid Y) = 0.2500$, $I(X;Y) = 0.2936$ (from $H(X) - H(X \mid Y)$ and from the KL form); $I(X;Z) = 0.1668$ through a BSC(0.1)
PIN, 10% choose 1234	$H = 12.4278$, $H_\infty = 3.3219$ (uniform: 13.2877)
BSC(0.11)	$1 - h(p) = 0.5001$; Blahut–Arimoto 0.5001 at $\Pr[X = 1] = 0.5000$; min-capacity 0.8319
Guessing walk-through	$V(X) = 0.4000$, $V(X \mid Y) = 0.6600$, simulated 0.6594; $\mathcal{L} = 0.7225$; $\mathcal{ML} = 1.2630$; Shannon capacity 0.8269

Experiment	Result
Reveal-if-zero(3) vs reveal-low(9), $n = 64$	Shannon 8.1686 vs 9.0000; min-entropy 61.0000 vs 9.0000
Fano, $M = 8$, $\varepsilon = 0.2$	Symmetric: bound 0.2000, MAP 0.2000. Shift: bound 0.0954, MAP 0.2000

tests/test_information_theory.py checks that the output still matches.

Five lessons come out of these runs. The first is that two formulas give one number: mutual information computed as an entropy difference and as a KL divergence agree, which is a useful check on any implementation. The second is that the closed forms are right. Blahut–Arimoto, which knows nothing about the binary symmetric channel, finds $1 - h(p)$ and the uniform input, and the leakage closed forms match the explicit matrices. The third is that vulnerability is a real success rate: the simulated MAP guesser wins at the rate $V(X \mid Y)$ predicts. The fourth is that Shannon and min-entropy measure different things. They can rank channels in opposite orders, and neither sees a single exposed secret value. The last is that Fano's bound is tight only when errors are spread out over the wrong values; concentrated errors make it loose.

5.11. References

- Cover & Thomas, *Elements of Information Theory* (2nd ed., 2006), ch. 2 (entropy, mutual information, the chain rules in §2.5, Jensen and its consequences in §2.6, the data-processing inequality in §2.8, Fano's inequality in §2.10) and ch. 7 (channel capacity, the binary symmetric channel in §7.1, the channel coding theorem in §7.7): the standard textbook for every Shannon result here
- MacKay, *Information Theory, Inference, and Learning Algorithms* (2003), ch. 2 and 8–10: the same material with a machine-learning slant, free online
- Smith, "On the Foundations of Quantitative Information Flow" (FoSSaCS 2009): vulnerability, conditional min-entropy $V(X \mid Y) = \sum_y \max_x p(x, y)$, and the argument against Shannon leakage for one-try guessing
- Braun, Chatzikokolakis & Palamidessi, "Quantitative Notions of Leakage for One-try Attacks" (MFPS 2009): min-capacity and the fact that the uniform prior attains it
- Alvim, Chatzikokolakis, McIver, Morgan, Palamidessi & Smith, *The Science of Quantitative Information Flow* (2020): the full theory of vulnerability, leakage and capacity, including gain functions that generalize the one-guess adversary
- Wainwright, *High-Dimensional Statistics* (2019), ch. 15: minimax lower bounds, including Fano's method, worked through for many estimation problems
- Yu, "Assouad, Fano, and Le Cam" (1997): the three classical lower-bound methods side by side

5.12. Cheat sheet

Table 5.7 collects the results.

Table 5.7.: Every result of the chapter.

Quantity	Formula	Key facts
Entropy	$H(X) = -\sum_x p(x) \log p(x)$	$0 \le H \le \log\lvert\mathcal{X}\rvert$; uniform attains the maximum
Binary entropy	$h(p) = -p \log p - (1-p)\log(1-p)$	$h(\frac{1}{2}) = 1$; $h(0.11) \approx \frac{1}{2}$
Conditional entropy	$H(X \mid Y) = \sum_y p(y) H(X \mid Y = y)$	$\le H(X)$ on average; one y can raise it
Chain rule	$H(X,Y) = H(Y) + H(X \mid Y)$	Extends to n variables
Mutual information	$I(X;Y) = H(X) - H(X \mid Y) = D(P_{XY} \parallel P_X P_Y)$	Symmetric, ≥ 0, zero iff independent
Data processing	$X \to Y \to Z \Rightarrow I(X;Z) \le I(X;Y)$	From the chain rule for I
Capacity	$\mathcal{C} = \max_\pi I(X;Y)$	BSC(p): $1 - h(p)$ at the uniform input
Vulnerability, min-entropy	$V(X) = \max_x p(x)$, $H_\infty = -\log V$	$H_\infty \le H$; one-guess success
Posterior vulnerability	$V(X \mid Y) = \sum_y \max_x p(x,y)$	Success of the MAP guesser
Min-entropy leakage	$\mathcal{L} = \log V(X \mid Y)/V(X)$	Factor by which guessing success grows
Min-capacity	$\mathcal{ML}(C) = \log \sum_y \max_x C[x,y]$	Attained at the uniform prior; $\le \log\min(\lvert\mathcal{X}\rvert, \lvert\mathcal{Y}\rvert)$
Fano	$H(X \mid Y) \le h(P_e) + P_e \log(M-1)$	Weak form $P_e \ge (H(X \mid Y) - 1)/\log M$; tight when errors are spread evenly

Rules of thumb

- Shannon quantities average log over outcomes; min-entropy quantities take log of a best-case probability. When a rare outcome reveals a lot, the two disagree, by orders of magnitude.
- Both are averages over the secret. Neither tells you how exposed a single secret value is.
- To prove an estimator must fail, bound the mutual information between the unknown and the data, then apply Fano.
- Processing never adds information: compute the bound on the raw observation, and it holds for everything computed from it.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

6. Optimization

6.1. Why this matters

Training a model means minimizing an average loss over the training set. Two questions come up every time you do it. The first is **how fast the optimizer gets there.** Gradient descent and stochastic gradient descent (SGD) are the workhorses, and you want to know how many steps they need, how to pick the step size, and what it costs you that the gradients SGD uses are noisy. The second is **how much the answer depends on any one training example.** If you replace one example and train again, does the minimizer barely move, or can it jump somewhere else entirely? The first question is about the algorithm; the second is about the problem itself, and it is the bridge to the next chapter, where a model that does not depend much on any one example is shown to generalize.

Both questions have the same kind of answer. **A few constants of the loss decide everything.** The first is how steep the loss can be anywhere, which is the largest size of its gradient: the Lipschitz constant L. The second is how fast that slope can change as you move, which is a bound on the curvature from above: the smoothness β. The third is how much the loss curves upward at least, a bound on the curvature from below: the strong convexity λ. The last one belongs to the optimizer rather than the loss: how far the random gradients SGD uses scatter around the true gradient, their variance σ^2. Every rate and every bound in this chapter is a combination of these four constants, and once you know which constant each result uses, you can tell when it applies. Table 6.1 lists the results.

Table 6.1.: The results of this chapter and where they are used.

Result	What it controls	Typical use
Lipschitz, smooth, strongly convex	The constants every later result is stated in	Checking that a loss qualifies; picking step sizes
SGD, convex and Lipschitz: $RL/\sqrt{T}$	Error of the averaged iterate after T steps	Linear models with logistic or hinge loss
SGD, strongly convex: $O(G^2/(\lambda T))$, G the gradient bound	Error with a regularizer	Ridge and regularized logistic regression
SGD, smooth non-convex	How fast the gradient norm goes to zero	Neural networks
Gradient noise and minibatches	How variance slows SGD and shrinks the best step	Choosing step size and batch size

Result	What it controls	Typical use
Sensitivity of regularized empirical risk minimization (ERM): $2L/(\lambda n)$	How far the minimizer moves when one example changes	Stability and generalization; per-example influence

The distributions behind the noise (Probability distributions) and the way averages concentrate (Concentration inequalities) are used only lightly here. This chapter is about inequalities that hold for every input, and about expectations; it never needs a tail bound.

Notation. The training data are n examples $D = (z_1, \dots, z_n)$. The model is a parameter vector $w \in \mathbb{R}^d$, and its loss on example z is $\ell(w, z)$. The **empirical risk** is the average loss over the training set, $F(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(w, z_i)$. A point where it is smallest is a **minimizer** w^*, and $F^* = F(w^*)$ is the smallest value. $\nabla F(w)$ is the gradient, the vector of partial derivatives, which points in the direction in which F increases fastest. $\langle a, b\rangle = \sum_j a_j b_j$ is the inner product of two vectors, and all norms are Euclidean, $\|a\| = \sqrt{\langle a, a\rangle}$.

6.2. Toolkit: five facts every derivation uses

Every proof in this chapter tracks the distance from the current parameters to the minimizer and asks how one step changes it. The five facts below are the pieces those proofs are built from, each stated and proved once.

6.2.1. Fact 1 — A convex function lies above its tangents

A function is **convex** when it curves upward everywhere, with no dents. The usual way to say this is that the straight segment (the chord) between any two points on its graph never dips below the graph: for all w, w' and every mixing weight $\theta \in [0, 1]$, the value at the mixed point is at most the same mix of the two values, $f(\theta w' + (1-\theta)w) \le \theta f(w') + (1-\theta) f(w)$.

For a differentiable f there is an equivalent picture, and it is the one the proofs use. The tangent at w predicts the value at another point w' by starting from $f(w)$ and adding the slope times the displacement, $\langle \nabla f(w), w' - w\rangle$. Convexity says this linear prediction never overshoots: the graph lies above every one of its tangents,

$$f(w') \ge f(w) + \langle \nabla f(w), w' - w\rangle \quad \text{for all } w, w'. \tag{6.1}$$

To see why the chord condition gives this, write the mixed point as $w + \theta(w' - w)$, a step of size θ from w towards w'. Subtract $f(w)$ from both sides of the chord condition and divide by $\theta > 0$: $\frac{f(w+\theta(w'-w)) - f(w)}{\theta} \le f(w') - f(w)$. The left side is the average slope over a short step towards w', and as $\theta \to 0$ it becomes the slope in that direction, the directional derivative $\langle \nabla f(w), w' - w\rangle$. That is Equation 6.1. (The other direction, from tangents to chords, is in the references.) For a concrete case take $f(w) = w^2$ at $w = 1$. The slope there is 2, so the tangent is $1 + 2(w' - 1)$. At $w' = 3$ the tangent predicts 5, and the function is 9, above it.

Here is why this matters. The fact turns a gradient, which you can compute, into a lower bound on how much higher the function is somewhere else. Every convex convergence proof uses it with w' set to the minimizer w^* and the inequality rearranged: $\langle \nabla f(w), w - w^* \rangle \geq f(w) - f(w^*)$. In words, the gradient at w points away from the minimizer, and its component along the direction from w^* to w is at least the current error $f(w) - f(w^*)$. So a step against the gradient makes progress towards w^* in proportion to how far from optimal you are.

6.2.2. Fact 2 — Expanding a square, and Cauchy–Schwarz

The squared length of a sum of two vectors is the two squared lengths plus a cross term, exactly as $(a+b)^2 = a^2 + 2ab + b^2$ for numbers. And the cross term can never be larger than the product of the lengths, because the inner product is the product of the lengths times the cosine of the angle between the vectors, and a cosine is at most 1 in size (the **Cauchy–Schwarz inequality**):

$$\|a+b\|^2 = \|a\|^2 + 2\langle a, b \rangle + \|b\|^2, \qquad |\langle a, b \rangle| \leq \|a\| \, \|b\|. \tag{6.2}$$

The first identity is what makes the SGD proofs work. One step moves w to $w - \eta g$, where g is the gradient used and η the step size. The new squared distance to the minimizer, $\|w - \eta g - w^*\|^2$, splits by the identity (with $a = w - w^*$ and $b = -\eta g$) into the old squared distance, a cross term with the gradient that measures progress, and a step-length term $\eta^2 \|g\|^2$ that measures overshoot. That split is the whole SGD proof.

6.2.3. Fact 3 — Projection onto a convex set never increases distances to its points

Sometimes the parameters are kept inside a set $\mathcal{W}$, such as a ball, and any step that leaves it is pulled back to the closest point of the set. This **projection** is $\Pi(u) = \arg\min_{p \in \mathcal{W}} \|u - p\|$, the point p of $\mathcal{W}$ nearest to u. The fact is that pulling back can only bring you closer to every point of the set, so it never undoes progress towards a minimizer that lies in the set. This needs $\mathcal{W}$ to be closed (so that a nearest point exists) and convex (so that the segment between two of its points stays inside). For every $w \in \mathcal{W}$,

$$\|\Pi(u) - w\| \leq \|u - w\|. \tag{6.3}$$

The picture behind the proof: if u is outside the set and p is its nearest point, then any point w of the set lies "behind" p as seen from u, at an angle of at least 90° at p. Pythagoras then makes u farther from w than p is. To make this precise, let $p = \Pi(u)$ and $w \in \mathcal{W}$. Points of the segment from p to w have the form $p + \theta(w - p)$ with $\theta \in [0, 1]$, and they are all in $\mathcal{W}$ because it is convex. Their squared distance to u is $h(\theta) = \|u - p - \theta(w - p)\|^2$. Since p is the closest point of the whole set, h is smallest at $\theta = 0$, so it cannot start decreasing as θ grows from 0: $h'(0) = -2\langle u - p, w - p \rangle \geq 0$, which says $\langle u - p, p - w \rangle \geq 0$. That is the obtuse angle. Now expand $u - w = (u - p) + (p - w)$ with Fact 2: $\|u - w\|^2 = \|u - p\|^2 + \|p - w\|^2 + 2\langle u - p, p - w \rangle$. The cross term is non-negative and so is $\|u - p\|^2$, so $\|u - w\|^2 \geq \|p - w\|^2$.

For the ball $\{\|w\| \leq R\}$ the projection is simple: $\Pi(u) = u \min(1, R/\|u\|)$, which leaves u alone if it is inside and shrinks it back to the sphere of radius R if it left. Projected SGD uses this to keep its iterates in a bounded set, and Fact 3 says this costs nothing in the analysis, as long as the minimizer w^* is in $\mathcal{W}$.

6.2.4. Fact 4 — Jensen: averaging inside a convex function helps

The chord condition of Fact 1 says that the value at a mix of two points is at most the mix of the values. Applying it repeatedly extends it to a mix of any number of points, which is **Jensen's inequality**: for a convex f and weights $\alpha_t \geq 0$ that add up to 1,

$$f\Big(\sum_t \alpha_t w_t\Big) \leq \sum_t \alpha_t f(w_t). \tag{6.4}$$

(Mix the first two points, then mix the result with the third, and so on.)

The proofs need it for the following reason. They can only bound the *average* error over the run, $\frac{1}{T}\sum_t (F(w_t) - F^*)$, which is not the error of any model you could hand to someone. Jensen with equal weights $1/T$ turns it into a bound for one model, the **averaged iterate** $\bar{w}_T = \frac{1}{T}\sum_t w_t$: its error is at most the average error.

6.2.5. Fact 5 — Unbiased gradients: the cross term averages out, the square does not

SGD does not use the true gradient $\nabla F(w_t)$, which needs a pass over the whole training set. It uses a cheap random estimate g_t, for example the gradient of one example picked at random. The estimate is **unbiased**: averaged over the random pick, it equals the true gradient, $\mathbb{E}[g_t \mid \text{past}] = \nabla F(w_t)$. Here "past" means all the random choices made before step t. Those choices determine w_t, so once you condition on them w_t is a fixed vector and only the new pick at step t is random. Two consequences follow, and they explain the title of this fact.

First, an inner product of g_t with any vector v_t fixed by the past (such as $w_t - w^*$) behaves as if g_t were the true gradient: $\mathbb{E}\langle g_t, v_t\rangle = \mathbb{E}\langle \nabla F(w_t), v_t\rangle$. The reason is that an inner product with a fixed vector is linear, so the conditional expectation passes inside it, $\mathbb{E}[\langle g_t, v_t\rangle \mid \text{past}] = \langle \mathbb{E}[g_t \mid \text{past}], v_t\rangle = \langle \nabla F(w_t), v_t\rangle$; then average over the past.

Second, the squared length of g_t does not average to the squared length of the true gradient. It is larger, by the **variance**, the mean squared error of the estimate. This is the **bias–variance split**:

$$\mathbb{E}[\|g_t\|^2 \mid \text{past}] = \|\nabla F(w_t)\|^2 + \mathbb{E}[\|g_t - \nabla F(w_t)\|^2 \mid \text{past}]. \tag{6.5}$$

To prove it, write g_t as the true gradient plus an error, $\nabla F + (g_t - \nabla F)$, and expand the square with Fact 2. The cross term is $2\langle \nabla F(w_t), g_t - \nabla F(w_t)\rangle$. The first vector is fixed by the past, and the error has conditional mean zero, so by the first consequence the cross term averages to zero. The two squares remain.

A concrete case makes both consequences visible. Take three one-dimensional examples $z = (1, 2, 6)$ and the loss $\ell(w, z) = \frac{1}{2}(w - z)^2$. The risk is $F(w) = \frac{1}{3}\sum_i \frac{1}{2}(w - z_i)^2$, whose gradient is the average of the $w - z_i$, that is $\nabla F(w) = w - 3$, since the mean of the data is 3. At $w = 4$, SGD picks one example uniformly at random and uses its gradient $w - z_i$. Table 6.2 lists the three equally likely outcomes.

Table 6.2.: SGD gradients for $F(w) = \frac{1}{3}\sum_i \frac{1}{2}(w - z_i)^2$ at $w = 4$, where $\nabla F(4) = 1$.

Example picked	$z_1 = 1$	$z_2 = 2$	$z_3 = 6$	Average
$g = w - z_i$	3	2	-2	1
g^2	9	4	4	17/3
$(g - \nabla F)^2$	4	1	9	14/3

The gradient is right on average: $(3 + 2 - 2)/3 = 1 = \nabla F(4)$. Its second moment is $(9 + 4 + 4)/3 = 17/3$, and it splits as $17/3 = 1^2 + 14/3$, the squared true gradient plus the variance $(4 + 1 + 9)/3 = 14/3$. Averaging removed the error from the gradient itself but not from its square. The variance never averages out, and every SGD rate in this chapter pays for it.

6.3. Convexity, Lipschitz continuity and smoothness

6.3.1. Context

To say anything about an optimizer, you need assumptions on the loss. Four properties are enough for this whole chapter. Two of them bound a derivative from above: Lipschitz continuity bounds the slope, and smoothness bounds the curvature. The other two bound the curvature from below: convexity says it is never negative, and strong convexity says it is at least some $\lambda > 0$. Mixing them up is the most common mistake in reading optimization results, so this section says exactly what each one means, how they are related, and how to check them on two common losses.

6.3.2. Statement

Each property below is about a function $f : \mathbb{R}^d \to \mathbb{R}$.

Lipschitz. f is **L-Lipschitz** when it can never change faster than L per unit of distance: $|f(w) - f(w')| \le L\|w - w'\|$. For differentiable f this is the same as saying the slope is at most L everywhere, $\|\nabla f(w)\| \le L$ for all w.

Smooth. f is **β-smooth** when its *gradient* is β-Lipschitz, that is, the slope changes by at most β per unit of distance: $\|\nabla f(w) - \nabla f(w')\| \le \beta\|w - w'\|$. Because the slope cannot change quickly, the tangent at w stays a good prediction of f nearby, and its error grows at most like a parabola with curvature β. This is the **descent lemma**, the upper-bound counterpart of Equation 6.1:

$$f(w') \le f(w) + \langle \nabla f(w), w' - w \rangle + \frac{\beta}{2}\|w' - w\|^2. \tag{6.6}$$

Convex. As in Fact 1, f lies above its tangents, Equation 6.1.

Strongly convex. f is **λ-strongly convex** when it lies above each tangent by at least a bowl of curvature λ: $f(w') \ge f(w) + \langle \nabla f(w), w' - w \rangle + \frac{\lambda}{2}\|w' - w\|^2$. It is convexity with a margin that grows with distance. An equivalent form compares gradients at two points: going from w' to w,

the slope in the direction of travel increases by at least λ per unit of distance. Multiplying by the distance, this says the gradient is **strongly monotone**:

$$\langle \nabla f(w) - \nabla f(w'), w - w' \rangle \geq \lambda \|w - w'\|^2. \tag{6.7}$$

For a twice-differentiable f, all four properties can be read off the Hessian $\nabla^2 f$, the matrix of second derivatives, whose eigenvalues are the curvatures of f along its principal directions. Write $A \succeq B$ when $A - B$ has no negative eigenvalue, so that the curvature of A in every direction is at least that of B. Then convex means $\nabla^2 f \succeq 0$ (no direction curves down), λ-strongly convex means $\nabla^2 f \succeq \lambda I$ (every direction curves up by at least λ), and β-smooth means every eigenvalue of $\nabla^2 f$ lies in $[-\beta, \beta]$ (no direction curves by more than β either way). Smoothness and strong convexity together sandwich the curvature, $\lambda I \preceq \nabla^2 f \preceq \beta I$. The ratio of the largest to the smallest curvature, $\kappa = \beta/\lambda$, is the **condition number**, and it measures how much more steeply the function rises in its steepest direction than in its flattest one.

6.3.3. Derivation

Steps 1 to 4 all use one move: to compare f (or ∇f) at two points, walk along the straight segment $w_s = w + s(w' - w)$, $s \in [0, 1]$, between them and add up the changes, using the fundamental theorem of calculus.

1. **Bounded gradient implies Lipschitz.** The change in f along the segment is the integral of the slope in the direction of travel, $f(w') - f(w) = \int_0^1 \langle \nabla f(w_s), w' - w \rangle \, ds$. By Cauchy–Schwarz (Fact 2) each inner product is at most $\|\nabla f(w_s)\| \, \|w' - w\| \leq L\|w' - w\|$, and integrating over an interval of length 1 gives $f(w') - f(w) \leq L\|w' - w\|$. Swapping w and w' bounds the change in the other direction, which gives the absolute value.

2. **Lipschitz implies bounded gradient.** Step a short distance h from w in the direction of the gradient, $u = \nabla f(w)/\|\nabla f(w)\|$. The Lipschitz condition says the change per unit distance is at most L: $\frac{f(w+hu) - f(w)}{h} \leq L$. As $h \to 0$ the left side becomes the slope in direction u, $\langle \nabla f(w), u \rangle = \|\nabla f(w)\|$, so $\|\nabla f(w)\| \leq L$.

3. **Smoothness implies the descent lemma.** The error of the tangent prediction is the integral of how much the slope has changed since the start of the segment: $f(w') - f(w) - \langle \nabla f(w), w' - w \rangle = \int_0^1 \langle \nabla f(w_s) - \nabla f(w), w' - w \rangle \, ds$. At position s you are a distance $s\|w' - w\|$ from w, so smoothness bounds the gradient change by $\beta s \|w' - w\|$, and Cauchy–Schwarz bounds the integrand by $\beta s \|w' - w\|^2$. Since $\int_0^1 \beta s \, ds = \beta/2$, the error is at most $\frac{\beta}{2}\|w' - w\|^2$.

4. **Hessian bound implies smoothness.** The gradient change along the segment is the integral of the Hessian applied to the direction of travel, $\nabla f(w') - \nabla f(w) = \int_0^1 \nabla^2 f(w_s)(w' - w) \, ds$. A symmetric matrix with eigenvalues in $[-\beta, \beta]$ stretches no vector by more than a factor β, so the integrand has length at most $\beta\|w' - w\|$, and so does the integral.

5. **Strong convexity implies strong monotonicity.** Write the strong convexity inequality twice, once from w to w' and once from w' to w, and add the two. The function values appear on both sides and cancel, which leaves $0 \geq \langle \nabla f(w) - \nabla f(w'), w' - w \rangle + \lambda \|w - w'\|^2$. Moving the inner product to the left (and flipping $w' - w$ to $w - w'$) gives Equation 6.7. The converse, and the Hessian forms of convexity, are proved in Nesterov's *Lectures on Convex Optimization*, §2.1.

6. **Two consequences of strong convexity.** Both are used later in the chapter.

   (a) *A small gradient means you are close to the minimizer.* At the minimizer the gradient is zero, $\nabla f(w^*) = 0$. Put $w' = w^*$ in Equation 6.7, so that the left side is $\langle \nabla f(w), w - w^* \rangle$, and bound it with Cauchy–Schwarz: $\lambda \|w - w^*\|^2 \le \|\nabla f(w)\| \, \|w - w^*\|$. Dividing by $\lambda \|w - w^*\|$ gives $\|w - w^*\| \le \|\nabla f(w)\| / \lambda$. In particular a second point with zero gradient would be at distance 0 from w^*, so the minimizer is unique.

   (b) *Adding $\frac{\lambda}{2}\|w\|^2$ to a convex function makes it λ-strongly convex.* The term $\frac{\lambda}{2}\|w\|^2$ meets its own strong convexity inequality with equality, because by Fact 2 (with $a = w$, $b = w' - w$) $\frac{\lambda}{2}\|w'\|^2 = \frac{\lambda}{2}\|w\|^2 + \lambda\langle w, w' - w\rangle + \frac{\lambda}{2}\|w' - w\|^2$, and λw is its gradient. Adding this equality to the tangent inequality Equation 6.1 of the convex function gives the strong convexity inequality for the sum.

6.3.4. Interpretation: Lipschitz is not smooth

The two conditions bound different derivatives, so neither implies the other, and two one-dimensional functions show it (Figure 6.1). The absolute value $f(w) = |w|$ is 1-Lipschitz, because its slope is always -1 or $+1$. But it is not smooth for any β, because the slope jumps from -1 to 1 at 0, an infinite rate of change. The parabola $f(w) = w^2/2$ is the opposite. Its slope is w, which changes at rate 1, so it is 1-smooth. But it is not Lipschitz on all of $\mathbb{R}$, because its slope grows without bound as $|w|$ grows.

The parabola is a special case of something general: **a strongly convex function is never Lipschitz on all of $\mathbb{R}^d$.** Consequence (a) rearranged says $\|\nabla f(w)\| \ge \lambda \|w - w^*\|$, so the slope grows at least linearly with the distance from the minimizer. That is why every strongly convex result below works on a bounded set $\mathcal{W}$, where the slope stays bounded, and projects onto it.

The two constants also do different jobs. L bounds how large one step's gradient can be. That governs SGD on losses that are not smooth, and, at the end of this chapter, how far one example can move the minimizer. β bounds how far you can trust the linear approximation of the tangent. That governs the largest safe step size, $1/\beta$, and the analysis of non-convex losses.

6.3.5. Example: logistic regression is 1-Lipschitz and 1/4-smooth

Logistic regression is the loss that has both properties, and checking them shows how the constants are found in practice: compute the gradient and bound its size, then compute the Hessian and bound its eigenvalues. Take a label $y \in \{-1, +1\}$, features x with $\|x\| \le 1$, and the **margin** $m = y\langle w, x\rangle$, which is positive when the model classifies the example correctly and large when it does so confidently. The logistic loss is $\ell(w) = \ln(1 + e^{-m})$: close to 0 for a large positive margin, and growing linearly for a large negative one. Write $\sigma(s) = 1/(1 + e^{-s})$ for the logistic (sigmoid) function, which rises from 0 to 1.

1. **Gradient.** Differentiate in m first: $\frac{d}{dm} \ln(1 + e^{-m}) = -\frac{e^{-m}}{1 + e^{-m}} = -\sigma(-m)$. The margin's gradient in w is $\nabla_w m = yx$, so by the chain rule $\nabla \ell(w) = -\sigma(-m)\, y\, x$.
2. **Lipschitz.** The gradient's length is $\|\nabla \ell(w)\| = \sigma(-m)\|x\| \le 1$, because $\sigma < 1$, $|y| = 1$ and $\|x\| \le 1$. A gradient bounded by 1 means the loss is 1-Lipschitz (step 2 of the derivation).

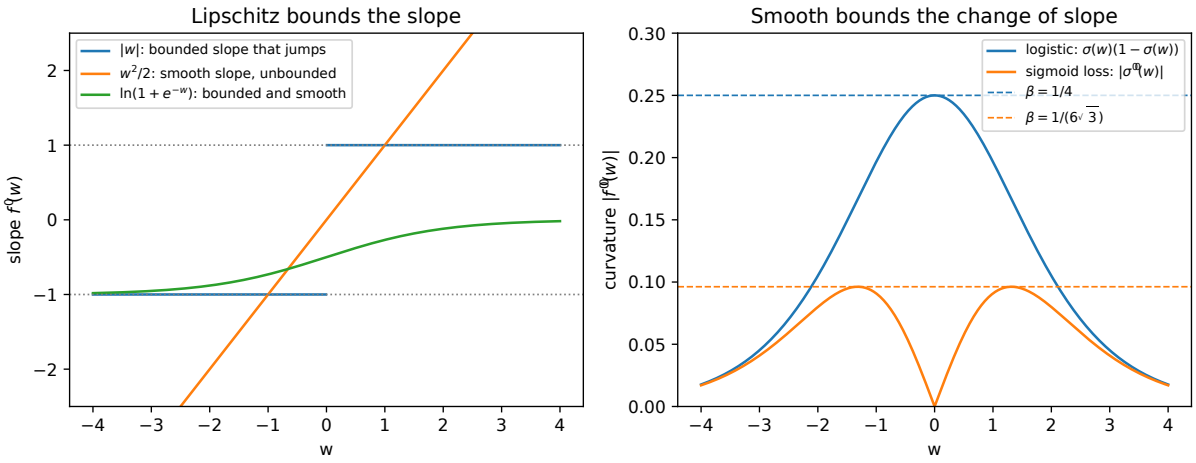


Figure 6.1.: Left: the slopes of $|w|$ (bounded, but jumps), $w^2/2$ (continuous, unbounded) and the logistic loss $\ln(1+e^{-w})$ (bounded and continuous). Right: the curvature of the logistic loss never exceeds $1/4$, and that of the non-convex sigmoid loss never exceeds $1/(6\sqrt{3})$.

3. **Hessian.** Differentiate the factor $-\sigma(-m)$ once more in m: $\frac{d}{dm}[-\sigma(-m)] = \sigma'(-m) = \sigma(m)(1-\sigma(m))$, using that the sigmoid's derivative is $\sigma' = \sigma(1-\sigma)$ and that this expression is the same at m and $-m$. The chain rule brings in yx twice, so $\nabla^2\ell(w) = \sigma(m)(1-\sigma(m))\,y^2\,xx^\top = \sigma(m)(1-\sigma(m))\,xx^\top$.
4. **Smooth and convex.** The matrix $xx^\top$ curves only along x: it has eigenvalue $\|x\|^2 \le 1$ in that direction and 0 in every direction perpendicular to it. The factor in front is $u(1-u)$ with $u = \sigma(m) \in [0,1]$, and $u(1-u) \le \frac{1}{4}$, with equality at $u = \frac{1}{2}$ (a parabola peaking at its middle). So every eigenvalue of the Hessian lies between 0 and $\frac{1}{4}$, $0 \preceq \nabla^2\ell \preceq \frac{1}{4}I$: the loss is convex and $\frac{1}{4}$-smooth.
5. **Not strongly convex.** The Hessian has rank one, so it has no curvature at all in directions perpendicular to x, and even along x its curvature $u(1-u)$ vanishes as $|m| \to \infty$. No $\lambda > 0$ bounds it from below.

With numbers: for $x = (0.6, 0.8)$ (length 1) and $y = +1$, at $w = 0$ the margin is 0, the loss is $\ln 2 = 0.6931$, the gradient norm is $\sigma(0) = 0.5000$ and the curvature is the maximum, $\frac{1}{2}\cdot\frac{1}{2} = 0.2500$. At $w = (3,4)$ the margin is $0.6\cdot 3 + 0.8\cdot 4 = 5$, and loss, gradient norm and curvature are all tiny: 0.0067, 0.0067 and 0.0066. Confidently classified examples contribute almost no curvature, so the empirical risk F is only weakly curved at a good solution, where most examples are confidently classified. On the logistic dataset used in the experiments below ($n = 1{,}000$, $d = 5$), the smallest eigenvalue of $\nabla^2 F(w^*)$ is 0.0233.

Adding a ridge term gives $\ell + \frac{\lambda}{2}\|w\|^2$. By consequence (b) it is λ-strongly convex, and since the ridge term adds λ to every eigenvalue of the Hessian it is $(\frac{1}{4}+\lambda)$-smooth. But its gradient $\nabla\ell + \lambda w$ grows with $\|w\|$, so it is no longer Lipschitz on $\mathbb{R}^d$. On the ball $\|w\| \le R$ its gradient is at most $1 + \lambda R$: at most 1 from the loss and at most λR from the ridge term.

6.3.6. Example: least squares and ridge

For the squared loss $\ell(w,(x,y)) = \frac{1}{2}(\langle w,x\rangle - y)^2$, the chain rule gives the gradient $(\langle w,x\rangle - y)x$, the prediction error times the features, and differentiating again gives the Hessian $xx^\top$. As in the logistic case, $xx^\top$ has eigenvalues $\|x\|^2$ and 0, so the loss is convex and $\|x\|^2$-smooth. It is not Lipschitz, because the prediction error, and with it the gradient, grows linearly in w. On the ball $\|w\| \le R$ with $|y| \le Y$ and $\|x\| \le 1$, the prediction error is at most $|\langle w,x\rangle| + |y| \le R + Y$, so the loss is $(R+Y)$-Lipschitz there.

Stack the n feature vectors as the rows of a matrix X and the labels into a vector y. The empirical risk then has the same Hessian everywhere, $H = \frac{1}{n}X^\top X$, the average of the $x_i x_i^\top$, called the sample second-moment matrix. It is strongly convex exactly when H is invertible (no direction with zero curvature), with $\lambda = \lambda_{\min}(H)$ and $\beta = \lambda_{\max}(H)$, its smallest and largest eigenvalues. When $d > n$, the n examples cannot span all d directions, H is singular, and F is flat along the missing directions, so it has a whole affine space of minimizers. **Ridge** regression adds $\frac{\lambda}{2}\|w\|^2$, which adds λ to every curvature. The Hessian becomes $H + \lambda I$, which is λ-strongly convex for any data. Setting the gradient $\frac{1}{n}X^\top(Xw - y) + \lambda w$ to zero and multiplying by n gives $(X^\top X + n\lambda I)w = X^\top y$, so the unique minimizer is $w^* = (X^\top X + n\lambda I)^{-1}X^\top y$.

6.3.7. Check yourself

- Is the hinge loss $\max(0, 1 - y\langle w,x\rangle)$ with $\|x\| \le 1$ Lipschitz? Smooth? *(It is 1-Lipschitz: where it is not flat its gradient is $-yx$, and in general its subgradients have norm at most $\|x\| \le 1$. It is not smooth: the gradient jumps from $-yx$ to 0 at margin 1.)*
- Why can a λ-strongly convex, β-smooth function never have $\lambda > \beta$? *(Both bound the same quantity, the error of the tangent prediction $f(w') - f(w) - \langle \nabla f(w), w' - w\rangle$. Strong convexity says it is at least $\frac{\lambda}{2}\|w' - w\|^2$ and the descent lemma says it is at most $\frac{\beta}{2}\|w' - w\|^2$, so $\lambda \le \beta$.)*

6.4. Gradient descent and SGD

6.4.1. Context

Gradient descent (GD) steps against the full gradient of the empirical risk. On a training set of a million examples, one step costs a million gradient evaluations. SGD replaces the full gradient with the gradient of one example, or of a small batch, chosen at random. Each step is cheap and right on average, but noisy. This section defines both algorithms and shows, on a quadratic, what a fast rate looks like when there is no noise, so that you can see later what the noise costs.

6.4.2. Statement

Gradient descent takes a step of size η against the full gradient: $w_{t+1} = w_t - \eta \nabla F(w_t)$.

Projected SGD does the same with a random estimate of the gradient, and then pulls the result back into the set $\mathcal{W}$ with the projection $\Pi_{\mathcal{W}}$ of Fact 3. At step t it draws an index i_t uniformly from $\{1, \dots, n\}$, takes that example's gradient $g_t = \nabla \ell(w_t, z_{i_t})$, and updates

$$w_{t+1} = \Pi_{\mathcal{W}}(w_t - \eta_t g_t), \tag{6.8}$$

where the step size η_t may change with t. With a **minibatch** S_t of B indices, $g_t = \frac{1}{B}\sum_{i \in S_t} \nabla \ell(w_t, z_i)$ is the average gradient over the batch. Either way the estimate is unbiased, $\mathbb{E}[g_t \mid \text{past}] = \nabla F(w_t)$, because averaging $\nabla \ell(w_t, z_i)$ over a uniformly chosen i gives exactly $\frac{1}{n}\sum_i \nabla \ell(w_t, z_i)$, which is the true gradient (Fact 5).

GD on a strongly convex quadratic. Take a quadratic risk $F(w) = F^* + \frac{1}{2}(w - w^*)^\top H (w - w^*)$ whose curvatures (the eigenvalues of H) lie between λ and β, $\lambda I \preceq H \preceq \beta I$. The idea is that on a quadratic the gradient is a linear function of the error $w - w^*$, so each GD step multiplies the error by a fixed matrix. With the step $\eta = 1/\beta$, the error along each principal direction shrinks by the factor $1 - h/\beta$, where h is the curvature in that direction. The steepest direction ($h = \beta$) is fixed in one step, and the flattest one ($h = \lambda$) shrinks slowest, by $1 - \lambda/\beta = 1 - 1/\kappa$ per step. So the distance to the minimizer shrinks at least geometrically:

$$\|w_t - w^*\| \le \left(1 - \frac{1}{\kappa}\right)^t \|w_0 - w^*\|. \tag{6.9}$$

6.4.3. Derivation

1. **One step multiplies the error by a matrix.** The gradient of the quadratic is $\nabla F(w) = H(w - w^*)$. Subtract w^* from both sides of the GD update: $w_{t+1} - w^* = (w_t - w^*) - \eta H(w_t - w^*) = (I - \eta H)(w_t - w^*)$.
2. **Find the factor in each direction.** $I - \eta H$ has the same principal directions as H. With $\eta = 1/\beta$, a curvature h of H becomes the factor $1 - h/\beta$ of $I - H/\beta$. Since $h \in [\lambda, \beta]$, these factors lie in $[0, 1 - \lambda/\beta] = [0, 1 - 1/\kappa]$.
3. **Bound the shrinkage, then repeat.** Write the error in the orthonormal basis of principal directions. Each coordinate is multiplied by its own factor, all of them between 0 and $1 - 1/\kappa$, so the length of the error shrinks by at least $1 - 1/\kappa$. Applying this at each of the t steps gives Equation 6.9.

The quadratic makes the factors exact, but a weaker statement holds for every β-smooth F, and the non-convex analysis later in this chapter starts from it: each GD step with $\eta = 1/\beta$ lowers F by at least $\|\nabla F\|^2/(2\beta)$. The reason is the descent lemma Equation 6.6. It says that around the current point, F lies below a parabola: the tangent plus $\frac{\beta}{2}\|w' - w\|^2$. Put in the GD step, $w' - w = -\nabla F(w)/\beta$. The tangent term $\langle \nabla F, w' - w \rangle$ becomes $-\frac{1}{\beta}\|\nabla F\|^2$, the progress, and the parabola term becomes $\frac{\beta}{2} \cdot \frac{1}{\beta^2}\|\nabla F\|^2$, the price of overshooting. Together,

$$F(w_{t+1}) \le F(w_t) - \frac{1}{\beta}\|\nabla F(w_t)\|^2 + \frac{\beta}{2} \cdot \frac{1}{\beta^2}\|\nabla F(w_t)\|^2 = F(w_t) - \frac{1}{2\beta}\|\nabla F(w_t)\|^2. \tag{6.10}$$

The overshoot eats half of the progress, and half is left.

6.4.4. Interpretation

GD on a strongly convex smooth function converges **linearly**: the error shrinks by a constant factor per step. To see how many steps that takes, use $1 - x \leq e^{-x}$: after t steps the factor is at most $e^{-t/\kappa}$, which reaches a target accuracy ϵ once $t \geq \kappa \ln(1/\epsilon)$. So reaching ϵ takes about $\kappa \ln(1/\epsilon)$ steps. Ill-conditioning, a large κ, is what makes GD slow. The step has to be small enough for the steepest direction, $\eta \leq 1/\beta$, or it overshoots there; and with that small step, the flattest direction, with curvature λ, moves only a fraction λ/β of the way per step.

SGD cannot do this with a constant step. Near w^* the true gradient vanishes, but the single-example gradients do not: in Table 6.2, even at the minimizer $w = 3$ the three examples would give gradients 2, 1 and -3. So the iterates keep jumping around w^*, at a distance set by η and the gradient variance. What makes SGD converge is averaging the iterates, which cancels the jumps, or shrinking η_t over time, which shrinks them. Both give slower rates than GD's, and the next sections derive them.

6.4.5. Example: GD and SGD on a two-dimensional least-squares problem

The script draws 200 examples with $x \sim \mathcal{N}(0, \mathrm{diag}(1, 0.1))$, so that the second feature varies much less than the first, and \$y = x_1 + x_2 + \$ noise. The empirical Hessian has $\lambda = 0.101$ and $\beta = 0.969$, so $\kappa = 9.58$. Starting at $w_0 = (-1, 3)$, Equation 6.9 predicts that GD with $\eta = 1/\beta$ needs at most 126 steps to shrink $\|w - w^*\|$ by a factor 10^6. That is the smallest t with $(1-1/9.58)^t \leq 10^{-6}$: taking logarithms, $t \geq \ln(10^6)/(-\ln(1-1/9.58)) = 13.8155/0.11024 = 125.3$, so $t = 126$. GD actually takes 122. The bound is nearly exact because the starting error has a large component along the flat direction, and that component contracts at exactly the worst-case rate $1 - 1/\kappa$.

Figure 6.2 shows the two algorithms on such a problem. GD first kills the error along the steep direction, then crawls along the flat valley. SGD with a constant step follows roughly the same route but keeps jittering around w^*.

6.4.6. Check yourself

- Why is $\eta = 2/\beta$ the largest constant step for which GD on a quadratic does not diverge? *(Along the steepest direction the error is multiplied by $1 - \eta\beta$ at each step. It stays bounded only if $|1 - \eta\beta| < 1$, that is $\eta < 2/\beta$; beyond that, each step overshoots by more than it corrects.)*
- In SGD, why does drawing i_t uniformly matter? *(It makes $\mathbb{E}[g_t \mid past] = \nabla F(w_t)$. A biased draw would give each example a different weight, so SGD would minimize a reweighted risk instead.)*

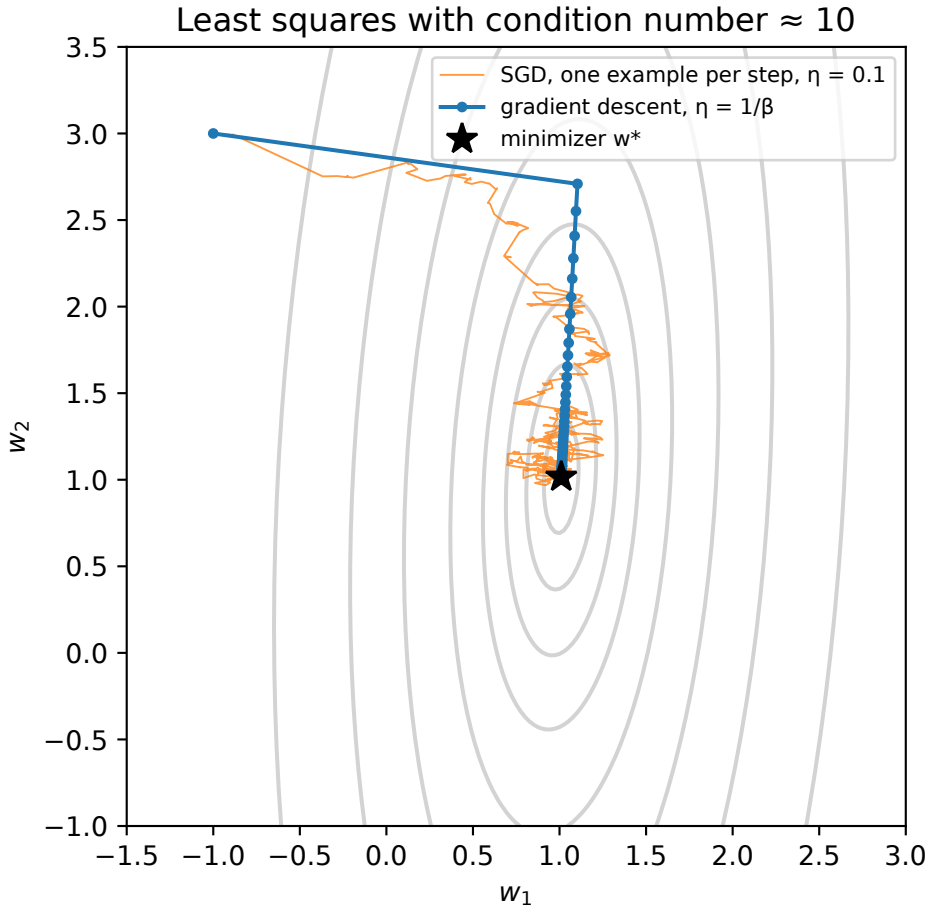


Figure 6.2.: Gradient descent (step $1/\beta$) and single-example SGD (constant step 0.1) on a two-dimensional least-squares risk with condition number about 10. The grey curves are contours of the risk.

6.5. SGD for convex Lipschitz objectives

6.5.1. Context

The cleanest guarantee for SGD needs only convexity and bounded gradients, with no smoothness at all, so it covers the hinge loss and the absolute loss as well as the logistic loss. Its proof is the template for every other proof in this chapter, so it is worth following line by line.

6.5.2. Statement

The guarantee says that SGD's error after T steps comes from two costs that pull the step size in opposite directions. The first is the cost of *travel*: you start at distance R from the minimizer, and with small steps it takes a long time to get there, so this cost falls as η grows. The second is the cost of *noise*: each step overshoots by an amount that grows with the step size and with the size of the random gradients, so this cost rises with η. The right step balances the two.

The setting is this. F is convex, and projected SGD runs inside a closed convex set $\mathcal{W}$ that contains the minimizer w^*. It starts at $w_1 \in \mathcal{W}$, at distance $\|w_1 - w^*\| \le R$ from the minimizer. The random gradients have a bounded second moment, $\mathbb{E}\|g_t\|^2 \le G^2$, so G is the typical size of a gradient used. Run projected SGD with a constant step η for T steps and return the averaged iterate $\bar{w}_T = \frac{1}{T}\sum_{t=1}^{T} w_t$. Then

$$\mathbb{E}[F(\bar{w}_T)] - F^* \le \frac{R^2}{2\eta T} + \frac{\eta G^2}{2}. \tag{6.11}$$

The first term is the travel cost and the second the noise cost. The step $\eta = R/(G\sqrt{T})$ minimizes the right side, which then becomes $RG/\sqrt{T}$. For an L-Lipschitz loss with single-example gradients, every gradient has $\|g_t\| \le L$, so $G = L$ and the bound is $RL/\sqrt{T}$.

6.5.3. Derivation

The idea is to follow the squared distance to the minimizer, $a_t = \|w_t - w^*\|^2$. One step changes it by two terms (Fact 2): a cross term that shrinks it, which convexity ties to the current error $F(w_t) - F^*$, and a step-length term that grows it, which is at most $\eta^2 G^2$ on average. Rearranging bounds the error at each step by the decrease in distance plus the noise term, and summing over the steps makes the decreases cancel in a chain.

1. **Projection can only help.** The new point is the projection of $w_t - \eta g_t$, and w^* is in $\mathcal{W}$, so by Fact 3 (projection never increases the distance to a point of the set) $a_{t+1} \le \|w_t - \eta g_t - w^*\|^2$.
2. **Expand the square** (Fact 2, with $a = w_t - w^*$ and $b = -\eta g_t$): $\|w_t - \eta g_t - w^*\|^2 = a_t - 2\eta\langle g_t, w_t - w^*\rangle + \eta^2\|g_t\|^2$.
3. **Rearrange** steps 1 and 2 to isolate the cross term, which is the progress the step makes: $\langle g_t, w_t - w^*\rangle \le \frac{a_t - a_{t+1}}{2\eta} + \frac{\eta}{2}\|g_t\|^2$.
4. **Take expectations.** The vector $w_t - w^*$ is fixed by the past, so by Fact 5 the inner product with g_t averages to the inner product with the true gradient $\nabla F(w_t)$. On the right, $\mathbb{E}\|g_t\|^2 \le G^2$. This gives $\mathbb{E}\langle \nabla F(w_t), w_t - w^*\rangle \le \frac{\mathbb{E}a_t - \mathbb{E}a_{t+1}}{2\eta} + \frac{\eta G^2}{2}$.

5. **Convexity turns progress into error.** Fact 1 with $w = w_t$ and $w' = w^*$ says the gradient's component towards w_t from w^* is at least the current error: $F(w_t) - F^* \le \langle \nabla F(w_t), w_t - w^* \rangle$. So $\mathbb{E}[F(w_t) - F^*] \le \frac{\mathbb{E}a_t - \mathbb{E}a_{t+1}}{2\eta} + \frac{\eta G^2}{2}$.
6. **Sum over $t = 1, \dots, T$.** The distance terms cancel in a chain (they **telescope**): $(a_1 - a_2) + (a_2 - a_3) + \cdots + (a_T - a_{T+1}) = a_1 - a_{T+1}$. Since $a_{T+1} \ge 0$ and $a_1 \le R^2$, the total is at most R^2. The noise term is added T times. Hence $\sum_t \mathbb{E}[F(w_t) - F^*] \le \frac{R^2}{2\eta} + \frac{\eta T G^2}{2}$.
7. **Average and apply Jensen.** Dividing by T bounds the average error over the run, and Fact 4 (Jensen: for convex F the error of the averaged iterate is at most the average error, $F(\bar{w}_T) \le \frac{1}{T}\sum_t F(w_t)$) passes the bound to $\bar{w}_T$. That is Equation 6.11.
8. **Optimize η.** The right side $\frac{R^2}{2\eta T} + \frac{\eta G^2}{2}$ has derivative $-\frac{R^2}{2\eta^2 T} + \frac{G^2}{2}$ in η, which vanishes at $\eta = R/(G\sqrt{T})$. At that step both terms equal $RG/(2\sqrt{T})$, and their sum is $RG/\sqrt{T}$.

Nothing in the proof needed F to be differentiable. For a function with kinks, such as the hinge loss, replace the gradient by a **subgradient**: any vector g whose "tangent" $F(w) + \langle g, w' - w \rangle$ stays below the graph everywhere (at the kink of $|w|$ at 0, any slope between -1 and 1). Then step 5 holds by the definition of a subgradient, and the rest of the proof is unchanged.

6.5.4. Interpretation

The two terms of Equation 6.11 are a trade-off. $R^2/(2\eta T)$ is the cost of travelling the distance R with steps of size η, and $\eta G^2/2$ is the cost of the jitter each step adds. A longer run, larger T, makes travel cheaper, so it allows smaller steps and less jitter, which is why the best step shrinks like $1/\sqrt{T}$.

The rate $1/\sqrt{T}$ cannot be improved for this class of functions. There are convex L-Lipschitz functions on which every method that only queries gradients has error at least of order $RL/\sqrt{T}$ after T queries (Bubeck 2015, §3.5). Smoothness alone does not help SGD either, because smoothness reduces the overshoot of the true gradient but not the noise term, which comes from the randomness of g_t.

The guarantee is for the averaged model, not the last one. The proof bounds the average of $F(w_t)$ over the run, and Jensen passes that bound only to the average of the w_t. So average the iterates, and do not just take the last one.

Finally, it is a worst case. Real objectives often have curvature near the optimum, which the bound does not use, and then SGD does much better than the bound, as the example shows.

6.5.5. Example: logistic regression

The script draws $n = 1{,}000$ examples in $d = 5$ with x uniform on the unit sphere and labels from a logistic model. It finds the minimizer with Newton's method ($\|w^*\| = 3.152$, $F^* = 0.5198$), and runs projected SGD on the ball of radius $R = 5$ from $w_1 = 0$. The minimizer is inside the ball, and the start is at distance $3.152 \le R$ from it, so the statement applies. The loss is 1-Lipschitz, $L = 1$, and the step is $\eta = R/(L\sqrt{T})$, which is $5/\sqrt{100} = 0.5$ at $T = 100$. Table 6.3 shows the results averaged over 20 runs.

Table 6.3.: Projected SGD on logistic regression against the convex Lipschitz bound Equation 6.11.

Steps T	Step η	Observed $F(\bar{w}_T) - F^*$	Bound $RL/\sqrt{T}$
100	0.5000	4.44×10^{-2}	0.500
1,000	0.1581	6.46×10^{-3}	0.158
10,000	0.0500	7.71×10^{-4}	0.050

The bound holds in every row. It is loose by a factor of 11 at $T = 100$ ($0.500/0.0444 = 11.3$) and 65 at $T = 10{,}000$ ($0.050/0.000771 = 64.9$). The gap grows because the observed error falls faster than the bound's $1/\sqrt{T}$. From $T = 100$ to $T = 10{,}000$, a hundredfold increase, the error falls by $0.0444/0.000771 = 57.6$, much closer to the factor 100 of a $1/T$ rate than to the factor 10 of $1/\sqrt{T}$. The reason is that this F is strongly convex near w^* (its smallest curvature there is 0.0233), which the bound does not know. The next section shows what a bound that uses curvature gives. The left panel of Figure 6.3 shows the same comparison over a range of T.

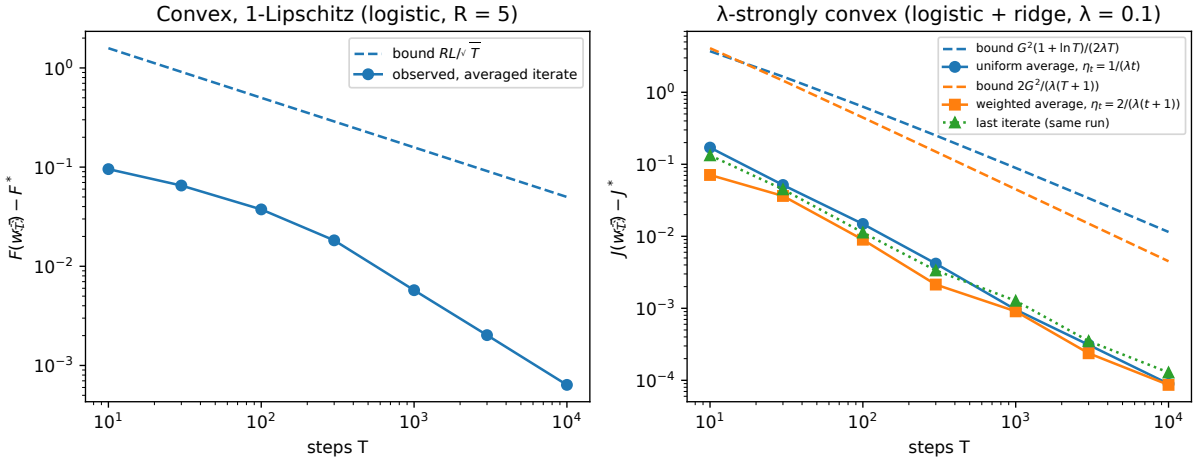


Figure 6.3.: Left: projected SGD on unregularized logistic regression, averaged iterate, against $RL/\sqrt{T}$. Right: the same data with a ridge term $\lambda = 0.1$: the uniformly averaged iterate with steps $1/(\lambda t)$, the t-weighted average with steps $2/(\lambda(t+1))$, and its last iterate, against the two strongly convex bounds.

6.5.6. Check yourself

- Where did the proof use that $w^* \in \mathcal{W}$? *(In step 1: Fact 3 only promises that projection does not increase the distance to points of $\mathcal{W}$.)*
- If you doubled the radius R used to set η but w^* stayed put, what would happen to the guarantee? *(The bound doubles, since it is $RL/\sqrt{T}$; in practice the step is just twice as large.)*

6.6. Strongly convex objectives

6.6.1. Context

Regularized models are strongly convex, and strong convexity pulls the iterates towards w^* at every step: the farther you are, the larger the gradient pointing back. That turns the $1/\sqrt{T}$ rate into $1/T$, provided you choose the step sizes and the average to match. This section shows two choices that work, and why the plain uniform average loses a factor $\ln T$.

6.6.2. Statement

The two schedules below differ only in how they weight the run. Both let the step shrink like $1/(\lambda t)$, which is the right size once strong convexity is pulling the iterates in. The first averages all iterates equally; the second gives later, better iterates more weight. Let F be λ-strongly convex on the closed convex set $\mathcal{W}$, which contains w^*, and let $\mathbb{E}\|g_t\|^2 \le G^2$ as before.

With steps $\eta_t = \frac{1}{\lambda t}$ and the uniform average $\bar{w}_T = \frac{1}{T}\sum_t w_t$, the error after T steps is

$$\mathbb{E}[F(\bar{w}_T)] - F^* \le \frac{G^2(1+\ln T)}{2\lambda T}. \tag{6.12}$$

With steps $\eta_t = \frac{2}{\lambda(t+1)}$ and the weighted average $\tilde{w}_T = \sum_t \frac{2t}{T(T+1)} w_t$, in which iterate t has weight proportional to t, the $\ln T$ disappears:

$$\mathbb{E}[F(\tilde{w}_T)] - F^* \le \frac{2G^2}{\lambda(T+1)}. \tag{6.13}$$

6.6.3. Derivation

The proof is the convex one with a better step 5. Strong convexity adds a term $\frac{\lambda}{2}\|w_t - w^*\|^2$ to the progress each step makes, and with the right steps that term makes the distance terms telescope away without leaving the R^2 of the convex case behind.

1. **The per-step inequality.** Steps 1 to 4 of the convex proof hold with a step η_t that changes over time: $\mathbb{E}\langle \nabla F(w_t), w_t - w^*\rangle \le \frac{\mathbb{E}a_t - \mathbb{E}a_{t+1}}{2\eta_t} + \frac{\eta_t G^2}{2}$, where $a_t = \|w_t - w^*\|^2$ is again the squared distance to the minimizer.
2. **Strong convexity improves step 5 by a distance term.** The strong convexity inequality at $w = w_t$, $w' = w^*$, rearranged, says $\langle \nabla F(w_t), w_t - w^*\rangle \ge F(w_t) - F^* + \frac{\lambda}{2}\|w_t - w^*\|^2$. Using this on the left of step 1 and moving the distance term to the right,

$$\mathbb{E}[F(w_t) - F^*] \le \Big(\frac{1}{2\eta_t} - \frac{\lambda}{2}\Big)\mathbb{E}a_t - \frac{1}{2\eta_t}\mathbb{E}a_{t+1} + \frac{\eta_t G^2}{2}. \tag{6.14}$$

3. **Uniform schedule.** With $\eta_t = 1/(\lambda t)$, the coefficients are $\frac{1}{2\eta_t} - \frac{\lambda}{2} = \frac{\lambda t}{2} - \frac{\lambda}{2} = \frac{\lambda(t-1)}{2}$ and $\frac{1}{2\eta_t} = \frac{\lambda t}{2}$, so the distance terms are $\frac{\lambda}{2}[(t-1)\mathbb{E}a_t - t\,\mathbb{E}a_{t+1}]$. Summed over t, each $t\,\mathbb{E}a_{t+1}$ subtracted at step t is added back at step $t+1$, and the chain telescopes to $\frac{\lambda}{2}[0 \cdot a_1 - T\,\mathbb{E}a_{T+1}] \le 0$. The starting distance has disappeared, multiplied by 0. The

noise terms are $\frac{\eta_t G^2}{2} = \frac{G^2}{2\lambda t}$, and they sum to $\frac{G^2}{2\lambda}\sum_{t\le T}\frac{1}{t} \le \frac{G^2(1+\ln T)}{2\lambda}$, because the harmonic sum $\sum_{t\le T} 1/t$ is 1 plus terms that are each below the area under $1/s$, and $\int_1^T ds/s = \ln T$. Divide by T and apply Jensen (Fact 4). The harmonic sum is where the $\ln T$ comes from: the early iterates, taken with huge steps, are bad, and the uniform average gives them full weight.

4. **Weighted schedule.** With $\eta_t = 2/(\lambda(t+1))$, the coefficients are $\frac{1}{2\eta_t} - \frac{\lambda}{2} = \frac{\lambda(t+1)}{4} - \frac{\lambda}{2} = \frac{\lambda(t-1)}{4}$ and $\frac{1}{2\eta_t} = \frac{\lambda(t+1)}{4}$. Multiply the step-2 inequality by t, which is the weight this average gives to iterate t. The distance terms become $\frac{\lambda}{4}[(t-1)t\,\mathbb{E}a_t - t(t+1)\mathbb{E}a_{t+1}]$, which telescope in the same way to at most 0. The noise term becomes $t\cdot\frac{\eta_t G^2}{2} = \frac{t}{t+1}\cdot\frac{G^2}{\lambda} \le \frac{G^2}{\lambda}$, the same for every t, so no harmonic sum appears. Summing, $\sum_t t\,\mathbb{E}[F(w_t) - F^*] \le \frac{TG^2}{\lambda}$. Divide by $\sum_t t = T(T+1)/2$ and apply Jensen with weights $2t/(T(T+1))$.

Shalev-Shwartz and Ben-David prove the uniform case (*Understanding Machine Learning*, Theorem 14.11). The weighted scheme is from Lacoste-Julien, Schmidt and Bach (2012). Rakhlin, Shamir and Sridharan (2012) get the same $O(G^2/(\lambda T))$ by averaging only the last half of the iterates, which drops the bad early ones in another way.

6.6.4. Interpretation

The rate is now $1/T$ instead of $1/\sqrt{T}$, so to halve the error you need twice the steps, not four times. Strong convexity is what buys this, and it costs a $1/\lambda$ in the constant: a weak regularizer gives a fast rate only after many steps.

The constant G depends on the domain. For a regularized loss $\ell + \frac{\lambda}{2}\|w\|^2$ with ℓ L-Lipschitz, the gradient is $\nabla\ell + \lambda w$, which on the ball $\|w\| \le R$ has length at most $L + \lambda R$. As in Section 6.3, a strongly convex function has no bound on its gradient over all of $\mathbb{R}^d$, so the bound needs a bounded set, and that is the reason for the projection.

The $1/(\lambda T)$ is not an artefact of the proof. Exercise 3 in the exercise book gives a one-dimensional noisy quadratic on which SGD's error is exactly $\sigma^2/(2\lambda T)$, where σ^2 is the gradient variance.

6.6.5. Example: regularized logistic regression

On the same data with $\lambda = 0.1$, the minimizer has $\|w^*\| = 0.839$, inside the ball of radius 5, and the gradient bound is $G = L + \lambda R = 1 + 0.1\times 5 = 1.5$. After $T = 10{,}000$ steps, averaged over 20 runs, the uniform average has error 9.09×10^{-5} against the bound 1.15×10^{-2}, and the weighted average 7.85×10^{-5} against 4.50×10^{-3} from Equation 6.13. Those bounds are the formulas with these numbers: $G^2 = 2.25$, so the uniform bound is $2.25\,(1+\ln 10^4)/(2\times0.1\times10^4) = 2.25\times10.21/2000$ and the weighted bound is $2\times2.25/(0.1\times10{,}001)$. The bounds hold but are loose by factors of about 130 and 60 ($0.0115/0.0000909 = 127$ and $0.0045/0.0000785 = 57$). The main reason is that $G = 1.5$ is the worst case over the whole ball, while the actual gradients near w^* are much smaller. In the right panel of Figure 6.3, all three iterates (uniform, weighted, and the last one) fall like $1/T$, and the weighted average is slightly ahead.

6.6.6. Check yourself

- Why do the first steps of the $1/(\lambda t)$ schedule, which can be huge, not wreck the result? *(The projection keeps every iterate in $\mathcal{W}$, and the telescoping in step 3 cancels their distance terms exactly: the first one is multiplied by $t-1=0$.)*
- What goes wrong if you use $\eta_t = 1/(\lambda t)$ with a λ larger than the true strong convexity? *(Step 2 claims more pull towards w^* than there is, so the telescoping fails, and the steps shrink too fast: the iterates can stall far from w^*.)*

6.7. Gradient noise and minibatches

6.7.1. Context

Every SGD bound so far paid for the second moment $\mathbb{E}\|g_t\|^2$, which by the bias–variance split of Fact 5 is the squared true gradient plus the variance of the estimate. Variance comes from sampling examples, and also from computing gradients in low precision or compressing them to send between machines. In later parts noise is sometimes added to gradients deliberately. This section answers two practical questions: how does the batch size change the variance, and how should the step size change when the gradients get noisier?

6.7.2. Statement

Minibatch variance. Averaging B independent random gradients divides their variance by B, just as the variance of a sample mean is the variance of one sample divided by the sample size. If the B examples are drawn without replacement, the average does a little better still, because no example can be drawn twice and the batch covers more of the dataset. To state this, let $\sigma_1^2 = \frac{1}{n}\sum_i \|\nabla \ell(w, z_i) - \nabla F(w)\|^2$ be the variance of a single-example gradient at w: the average squared distance of the n per-example gradients from their mean. For the average gradient g_B of a batch of B indices drawn uniformly,

$$\mathbb{E}\|g_B - \nabla F(w)\|^2 = \begin{cases} \dfrac{\sigma_1^2}{B} & \text{with replacement,} \\ \dfrac{\sigma_1^2}{B} \cdot \dfrac{n-B}{n-1} & \text{without replacement.} \end{cases} \tag{6.15}$$

The extra factor $\frac{n-B}{n-1}$ is 1 for $B=1$ and 0 for $B=n$, when the batch is the whole dataset and its gradient is exact.

Noisy SGD. Suppose each stochastic gradient is perturbed by extra noise ξ_t, independent of everything else, with mean zero and mean squared size $\mathbb{E}\|\xi_t\|^2 = \sigma^2$. Independent noise with mean zero adds its squared size to the second moment without changing anything else, so the convex Lipschitz bound Equation 6.11 holds with $G^2 = L^2 + \sigma^2$. Its best step and its value at that step follow as before:

$$\mathbb{E}[F(\bar{w}_T)] - F^* \le \frac{R^2}{2\eta T} + \frac{\eta(L^2+\sigma^2)}{2}, \qquad \eta^* = \frac{R}{\sqrt{(L^2+\sigma^2)T}}, \qquad \text{bound at } \eta^* = \frac{R\sqrt{L^2+\sigma^2}}{\sqrt{T}}. \tag{6.16}$$

6.7.3. Derivation

With replacement. The B draws are independent, each with mean $\nabla F(w)$ and variance σ_1^2. Variances of independent random vectors add, just as for numbers: expanding the squared length of the sum of their deviations gives the squared lengths plus cross terms, and each cross term has mean zero because the draws are independent and each deviation has mean zero. So the sum of the B gradients has variance $B\sigma_1^2$. Dividing the sum by B divides the variance by B^2, so the average has variance $B\sigma_1^2/B^2 = \sigma_1^2/B$.

Without replacement. Now the draws are not independent, and the cross terms no longer vanish. They turn out to be slightly negative, which is where the gain comes from. Write $u_i = \nabla\ell(w, z_i) - \nabla F(w)$ for the deviation of example i's gradient from the mean, so that the deviations sum to zero, $\sum_i u_i = 0$, and $\frac{1}{n}\sum_i \|u_i\|^2 = \sigma_1^2$. Let $I_1, \dots, I_B$ be the indices drawn.

1. **Each single draw behaves as before.** Each draw I_j on its own is still uniform over the n examples, so $\mathbb{E}\|u_{I_j}\|^2 = \sigma_1^2$.
2. **Two different draws are negatively related.** A pair of different draws $I_j \neq I_k$ is a uniformly random pair of distinct indices, so $\mathbb{E}\langle u_{I_j}, u_{I_k}\rangle = \frac{1}{n(n-1)}\sum_{i\neq i'}\langle u_i, u_{i'}\rangle$, the average over the $n(n-1)$ ordered pairs. That sum of cross terms is the squared length of the total minus the squares: $\sum_{i\neq i'}\langle u_i, u_{i'}\rangle = \|\sum_i u_i\|^2 - \sum_i \|u_i\|^2 = 0 - n\sigma_1^2$. So the expected cross term is $-n\sigma_1^2/(n(n-1)) = -\sigma_1^2/(n-1)$. In words: the deviations sum to zero, so once you have drawn an example that deviates one way, the ones left over deviate, on average, slightly the other way.
3. **Add up.** The squared length of the batch average $\frac{1}{B}\sum_j u_{I_j}$ is $\frac{1}{B^2}$ times B squares plus $B(B-1)$ ordered cross terms. Taking expectations with steps 1 and 2, $\frac{1}{B^2}[B\sigma_1^2 - B(B-1)\frac{\sigma_1^2}{n-1}] = \frac{\sigma_1^2}{B}(1 - \frac{B-1}{n-1}) = \frac{\sigma_1^2}{B}\cdot\frac{n-B}{n-1}$.

The toy data of Table 6.2 make this concrete. There $\sigma_1^2 = 14/3$, and the deviations $u_i = g - \nabla F$ are $3-1=2$, $2-1=1$ and $-2-1=-3$, which sum to zero. Table 6.4 lists every batch of $B = 2$ drawn without replacement, with the mean deviation of the batch and its square.

Table 6.4.: All batches of size 2 drawn without replacement from the toy data of Table 6.2.

Batch	$\{1,2\}$	$\{1,3\}$	$\{2,3\}$	Average
Mean deviation $\bar{u}$	1.5	-0.5	-1	0
$\bar{u}^2$	2.25	0.25	1	7/6

The variance is the average of the last row, $(2.25 + 0.25 + 1)/3 = 7/6$. The formula gives $\frac{14/3}{2}\cdot\frac{3-2}{3-1} = \frac{7}{6}$, matching Table 6.4. With replacement the variance would be $\frac{14/3}{2} = \frac{7}{3}$, twice as large.

Noisy SGD. The noisy gradient is $\nabla\ell + \xi_t$. Expand its squared length with Fact 2: the cross term $2\langle\nabla\ell, \xi_t\rangle$ has mean zero, because ξ_t is independent of the gradient and has mean zero (the same argument as in Fact 5). So $\mathbb{E}\|\nabla\ell + \xi_t\|^2 = \mathbb{E}\|\nabla\ell\|^2 + \mathbb{E}\|\xi_t\|^2 \leq L^2 + \sigma^2$. Substitute $G^2 = L^2 + \sigma^2$ into Equation 6.11 and optimize η as in step 8 of that derivation, which gives $\eta^* = R/(G\sqrt{T})$ and the bound $RG/\sqrt{T}$.

6.7.4. Interpretation

A batch of B divides the variance by B, and sampling without replacement removes a further fraction $(B-1)/(n-1)$ of what is left. That correction only matters once B is a sizeable fraction of n, and at $B = n$ the gradient is exact.

Noise shrinks the best step. η^* scales like $1/\sqrt{L^2+\sigma^2}$, so noisier gradients call for smaller steps and longer runs. To see how much longer, set the bound $R\sqrt{L^2+\sigma^2}/\sqrt{T}$ equal to a target accuracy ϵ and solve for T: reaching ϵ takes $R^2(L^2+\sigma^2)/\epsilon^2$ steps. Noise of size $\sigma = 3L$ makes $L^2+\sigma^2 = L^2+9L^2 = 10L^2$, so it multiplies the number of steps by 10.

Larger batches are not free. A batch of B costs B gradient evaluations and divides the variance by B, so per gradient evaluation the variance reduction is exactly cancelled. Batches help mainly because the B evaluations can run in parallel. Exercise 6 in the exercise book makes this precise for the non-convex bound: beyond a critical batch size, larger batches waste computation.

6.7.5. Example

Minibatch variance. At the logistic minimizer w^* of the experiments, $\sigma_1^2 = 0.1722$. The true gradient there is zero, so this is just the mean squared per-example gradient. The script draws 4,000 random batches of each size and measures the variance of their average gradient (Table 6.5).

Table 6.5.: Measured minibatch gradient variance at w^* for the logistic data ($n = 1{,}000$), against Equation 6.15.

Batch size B	With replacement: measured / σ_1^2/B	Without replacement: measured / Equation 6.15
1	1.75×10^{-1} / 1.72×10^{-1}	1.75×10^{-1} / 1.72×10^{-1}
10	1.74×10^{-2} / 1.72×10^{-2}	1.68×10^{-2} / 1.71×10^{-2}
100	1.73×10^{-3} / 1.72×10^{-3}	1.57×10^{-3} / 1.55×10^{-3}
500	3.47×10^{-4} / 3.44×10^{-4}	1.74×10^{-4} / 1.72×10^{-4}

The formulas in Table 6.5 match the measurements to within the Monte Carlo error. At $B = 500 = n/2$ the correction factor is $\frac{1000-500}{999}$, about one half, so sampling without replacement halves the variance. Figure 6.4 shows the full range.

Noise and step size. The script adds Gaussian noise with $\mathbb{E}\|\xi_t\|^2 = \sigma^2$ to every single-example gradient and runs projected SGD on the logistic problem for $T = 10{,}000$ steps, 10 runs per setting, sweeping η over 17 values from 10^{-3} to 10. With $R = 5$ and $L = 1$, the bound's recommended step is $\eta^* = 5/\sqrt{(1+\sigma^2)\cdot 10^4}$, which is $5/100 = 0.05$ without noise, and the bound there is $5\sqrt{1+\sigma^2}/100$. Table 6.6 compares that step with the best one on the grid.

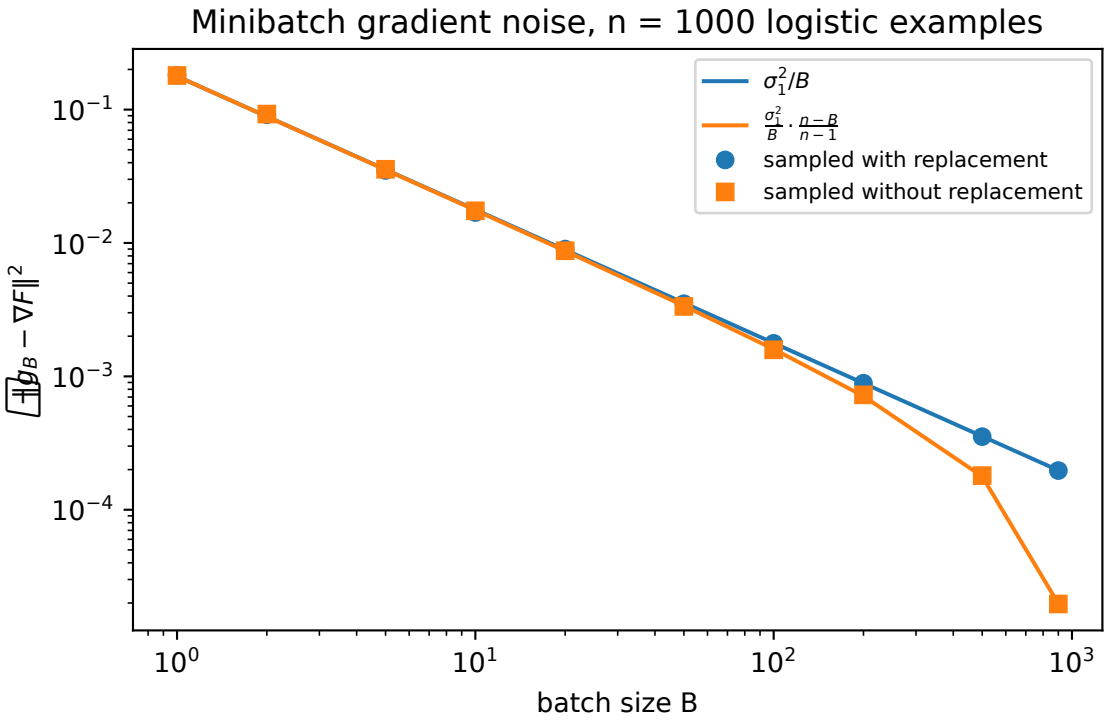


Figure 6.4.: Minibatch gradient variance at the logistic minimizer against batch size, measured and from Equation 6.15. Without replacement, the variance drops below σ_1^2/B as B approaches $n = 1000$.

Table 6.6.: Projected SGD with added gradient noise, $T = 10{,}000$: suboptimality $F(\bar{w}_T) - F^*$ at the step size the bound recommends and at the best step size on the grid.

Noise σ	η^* from the bound	Bound at η^*	Observed at η^*	Best η on the grid	Observed there
0	0.0500	0.050	4.99×10^{-4}	0.178	1.85×10^{-4}
1	0.0354	0.071	1.58×10^{-3}	0.056	1.38×10^{-3}
3	0.0158	0.158	9.58×10^{-3}	0.056	5.10×10^{-3}

Table 6.6 and Figure 6.5 show three things. First, noise costs accuracy: at the bound's step, the error grows about 20-fold from $\sigma = 0$ to $\sigma = 3$ ($9.58 \times 10^{-3} / 4.99 \times 10^{-4} = 19.2$). Second, noise pushes the best step down, from 0.178 to 0.056 on the grid, and the U-shaped curves in the figure move left. Third, the bound's η^* is conservative. It assumes the gradient is as large as L and the distance as large as R throughout, while the actual run has smaller gradients and starts closer, so the best observed step is larger: by a factor of 3.6 without noise ($0.178/0.05 = 3.56$).

6.7.6. Check yourself

- You quadruple the batch size. By how much should the variance term in a bound like $\eta\sigma^2$ drop, and what happens to the cost of a step? *(By a factor of 4, slightly more without replacement; each step costs 4 times as many gradient evaluations.)*
- Why does a very small η give the same error for every noise level in Figure 6.5? *(The iterates barely leave $w_1 = 0$, so the error is just $F(0) - F^*$, set by the travel term $R^2/(2\eta T)$, not by the noise.)*

6.8. Non-convex objectives: convergence to stationary points

6.8.1. Context

Neural network losses are not convex. There can be many local minima and saddle points, and no method that only uses gradients can promise to find the global minimum. What you can still guarantee is that SGD finds a point where the gradient is small, an **approximate stationary point**. The argument uses smoothness instead of convexity, and it shows directly how the gradient variance and the step size trade off.

6.8.2. Statement

The idea is that each step lowers F in proportion to the squared gradient, by the descent lemma (as for GD in the section on gradient descent), minus a penalty for the noise. F cannot fall forever, because it is bounded below, so the total of the squared gradients over the run is bounded, and they must be small on average.

To state it, let F be β-smooth and bounded below by F_{inf}, and let $\Delta = F(w_1) - F_{\text{inf}}$ be the most it can fall from the starting point. Let the stochastic gradients be unbiased, with variance at

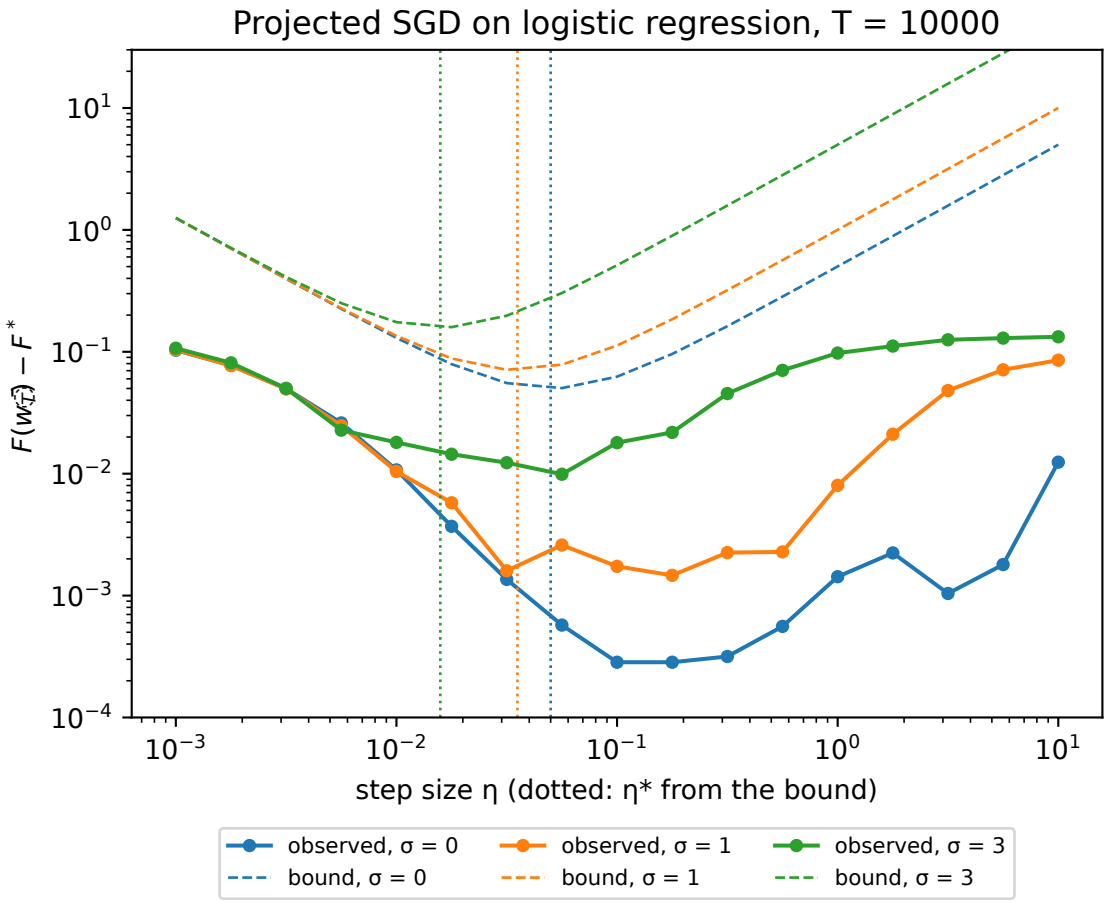


Figure 6.5.: Suboptimality of the averaged iterate after $T = 10{,}000$ steps of projected SGD on logistic regression, as a function of the step size, for three levels of added gradient noise. Dashed curves are the bound $R^2/(2\eta T) + \eta(L^2 + \sigma^2)/2$; dotted vertical lines mark its minimizer η^*.

most σ^2: $\mathbb{E}[\|g_t - \nabla F(w_t)\|^2 \mid \text{past}] \le \sigma^2$. SGD (with no projection) with a constant step $\eta \le 1/\beta$ satisfies

$$\min_{t \le T} \mathbb{E}\|\nabla F(w_t)\|^2 \le \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}\|\nabla F(w_t)\|^2 \le \frac{2\Delta}{\eta T} + \beta\eta\sigma^2. \tag{6.17}$$

As in the convex case, the first term is the travel cost and the second the noise cost. With $\eta = \min\left(1/\beta,\ \sqrt{2\Delta/(\beta\sigma^2 T)}\right)$, the step that balances them unless that would exceed $1/\beta$, the right side is at most $\frac{4\beta\Delta}{T} + 2\sqrt{\frac{2\beta\Delta\sigma^2}{T}}$.

6.8.3. Derivation

1. **Descent lemma for one step.** Apply Equation 6.6 with $w = w_t$ and $w' = w_{t+1} = w_t - \eta g_t$, so that $w' - w = -\eta g_t$: $F(w_{t+1}) \le F(w_t) - \eta\langle \nabla F(w_t), g_t\rangle + \frac{\beta\eta^2}{2}\|g_t\|^2$.
2. **Average over the random gradient.** By Fact 5, conditioned on the past the inner product averages to $\|\nabla F(w_t)\|^2$, and the bias–variance split gives $\mathbb{E}\|g_t\|^2 \le \|\nabla F(w_t)\|^2 + \sigma^2$. Collecting the two $\|\nabla F(w_t)\|^2$ terms and averaging over the past, $\mathbb{E}F(w_{t+1}) \le \mathbb{E}F(w_t) - \eta(1 - \frac{\beta\eta}{2})\mathbb{E}\|\nabla F(w_t)\|^2 + \frac{\beta\eta^2\sigma^2}{2}$.
3. **Make sure the step makes progress.** The factor $1 - \beta\eta/2$ must be positive, or the step could increase F. With $\eta \le 1/\beta$ it is at least $1/2$, so $\frac{\eta}{2}\mathbb{E}\|\nabla F(w_t)\|^2 \le \mathbb{E}F(w_t) - \mathbb{E}F(w_{t+1}) + \frac{\beta\eta^2\sigma^2}{2}$.
4. **Sum and telescope.** Summing over $t = 1, \dots, T$, the differences of F cancel in a chain to $F(w_1) - \mathbb{E}F(w_{T+1})$, which is at most Δ because F never goes below F_{inf}: $\frac{\eta}{2}\sum_{t=1}^{T}\mathbb{E}\|\nabla F(w_t)\|^2 \le F(w_1) - \mathbb{E}F(w_{T+1}) + \frac{T\beta\eta^2\sigma^2}{2} \le \Delta + \frac{T\beta\eta^2\sigma^2}{2}$.
5. **Divide by $\eta T/2$** to get the average in Equation 6.17. The minimum over t is at most the average, which gives the left inequality.
6. **Choose η.** There are two cases. If $\sqrt{2\Delta/(\beta\sigma^2 T)} \le 1/\beta$, that step makes both terms equal to $\sqrt{2\beta\Delta\sigma^2/T}$, so their sum is twice that. Otherwise $\eta = 1/\beta$, the first term is $2\beta\Delta/T$, and the condition $1/\beta < \sqrt{2\Delta/(\beta\sigma^2 T)}$, squared and rearranged, says $\sigma^2 < 2\beta\Delta/T$, so the second term σ^2 is below $2\beta\Delta/T$ too, and the sum is below $4\beta\Delta/T$. Either way the total is at most $\frac{4\beta\Delta}{T} + 2\sqrt{\frac{2\beta\Delta\sigma^2}{T}}$.

This is the argument of Ghadimi and Lan (2013, Theorem 2.1 and Corollary 2.2). They output an iterate chosen uniformly at random from the run, so that its expected squared gradient equals the average in Equation 6.17.

6.8.4. Interpretation

There are two regimes. Without noise ($\sigma = 0$) the second term vanishes, the step is $1/\beta$, and the rate is of order $\beta\Delta/T$, the deterministic GD rate. With noise the $\sqrt{\sigma^2/T}$ term dominates for large T: the squared gradient falls like $1/\sqrt{T}$, so the gradient norm, its square root, falls only like $T^{-1/4}$.

A small gradient is all you get. A stationary point can be a local minimum, a saddle point, or a flat region far from anything good, and the guarantee says nothing about $F(w_T) - F^*$.

The step size again balances travel against noise, $2\Delta/(\eta T)$ against $\beta\eta\sigma^2$, the same trade-off as in the convex case. With batches of size B, replace σ^2 by σ^2/B, by Equation 6.15.

6.8.5. Example: the sigmoid loss

The **sigmoid loss** $\ell(w) = \sigma(-y\langle w, x\rangle)$ is a smooth stand-in for the 0–1 loss: with margin $m = y\langle w, x\rangle$ it is $\sigma(-m)$, close to 1 for a badly misclassified example and close to 0 for a confidently correct one. Here σ is the sigmoid function of the logistic example, not a noise level. The loss is not convex, because σ curves up on one side of 0 and down on the other. To apply the statement you need its constants β, Δ and the noise σ^2, and each is found by bounding a derivative.

With $\|x\| \le 1$ its gradient $-y\,\sigma'(-m)\,x$ has norm at most $\max \sigma' = \frac{1}{4}$, since $\sigma' = \sigma(1-\sigma) \le \frac{1}{4}$. Its Hessian $\sigma''(-m)\,xx^\top$ has eigenvalues of size at most $\max|\sigma''|$, which is therefore the smoothness constant. Differentiating $\sigma' = \sigma(1-\sigma)$ once more gives $\sigma'' = \sigma(1-\sigma)(1-2\sigma)$. Writing $u = \sigma \in (0,1)$ and maximizing $u(1-u)(1-2u)$ by setting its derivative to zero gives $u = \frac{1}{2} - \frac{\sqrt{3}}{6}$, where $u(1-u) = \frac{1}{6}$ and $1-2u = \frac{1}{\sqrt{3}}$. So $\beta = \frac{1}{6\sqrt{3}} = 0.0962$ (Figure 6.1, right). The loss lies in $(0,1)$, so $F_{\inf} \ge 0$, and starting at $w_1 = 0$, where every margin is 0 and $F = \sigma(0) = \frac{1}{2}$, gives $\Delta \le \frac{1}{2}$. Finally, the variance is at most the second moment, which is at most the squared largest gradient: $\sigma^2 \le \mathbb{E}\|g\|^2 \le \frac{1}{16}$.

On the logistic data, $\|\nabla F(0)\|^2 = 4.06 \times 10^{-3}$. SGD runs with the step from the statement. At $T = 100$ that step is $\sqrt{2\Delta/(\beta\sigma^2 T)} = \sqrt{1/(0.0962 \times 0.0625 \times 100)} = 1.289$, well below $1/\beta = 10.4$. Table 6.7 shows the results averaged over 5 runs.

Table 6.7.: SGD on the non-convex sigmoid loss against the stationarity bound.

Steps T	Step η	Observed $\frac{1}{T}\sum_t \|\nabla F(w_t)\|^2$	Bound Equation 6.17
100	1.289	1.64×10^{-3}	1.55×10^{-2}
1,000	0.408	5.22×10^{-4}	4.90×10^{-3}
10,000	0.129	1.87×10^{-4}	1.55×10^{-3}

The bound in Table 6.7 is loose by a factor of 8 to 10 (from $0.00155/0.000187 = 8.3$ to $0.0155/0.00164 = 9.5$) and has the right slope, $1/\sqrt{T}$: each tenfold increase in T divides it by $\sqrt{10} = 3.16$. The slack comes from the worst-case constants: Δ, σ^2 and β are all upper bounds that the actual run does not reach. Figure 6.6 shows the comparison over more values of T.

6.8.6. Check yourself

- Which step of the derivation would fail with $\eta = 3/\beta$? *(Step 3: $1 - \beta\eta/2$ becomes negative, so the descent lemma no longer guarantees progress.)*
- Why can the convex proof bound $F(\bar{w}_T) - F^*$ while this one cannot? *(It used Fact 1 to turn the inner product $\langle \nabla F, w_t - w^* \rangle$ into a function gap. Without convexity there is no such link.)*

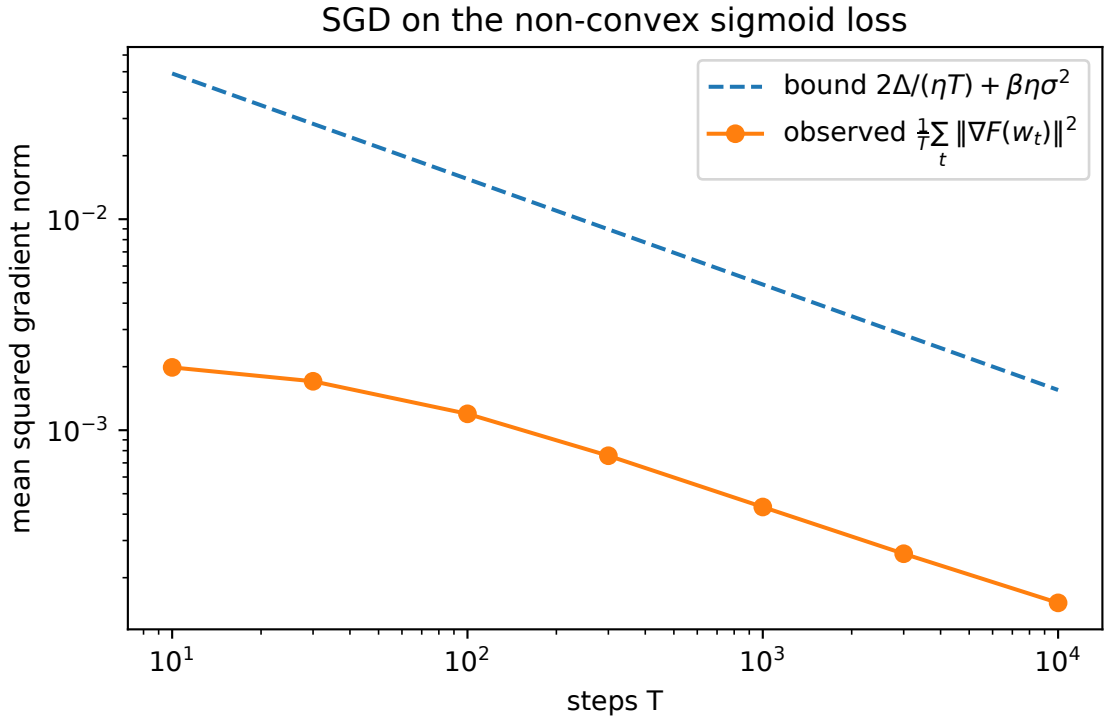


Figure 6.6.: Average squared gradient norm along SGD on the sigmoid loss, against the bound $2\Delta/(\eta T) + \beta\eta\sigma^2$ with the step chosen for each horizon T.

6.9. Sensitivity of regularized ERM

6.9.1. Context

The previous sections asked how fast an optimizer reaches the minimizer. This one asks how much the minimizer itself depends on the data. Regularized ERM picks the parameters that minimize the average training loss plus a penalty $\frac{\lambda}{2}\|w\|^2$ on their size:

$$J(w; D) = \frac{1}{n}\sum_{i=1}^{n} \ell(w, z_i) + \frac{\lambda}{2}\|w\|^2, \tag{6.18}$$

and returns the minimizer $w^*(D) = \arg\min_w J(w; D)$. Suppose you replace one training example and retrain. How far can the minimizer move? This is a question about the problem, not about the optimizer: it asks how much any single example can influence the exact solution. The answer controls how stable the learned model is, and therefore how well it generalizes (Algorithmic stability and generalization). The quantity returns in later parts under the name *sensitivity*.

6.9.2. Statement

Changing one example changes only one of the n terms of the average, so the objective's gradient moves by at most $2L/n$: that example's gradient is removed and another added, each of length at most L. Strong convexity with curvature λ then converts a small change in the gradient into a small move of the minimizer, at most the gradient change divided by λ. Formally, let each $\ell(\cdot, z)$ be convex and L-Lipschitz, and let the datasets D and D' differ in one example. Then

$$\|w^*(D) - w^*(D')\| \le \frac{2L}{\lambda n}. \tag{6.19}$$

6.9.3. Derivation

Let D' replace z_i by z'. Write $J_D = J(\cdot; D)$ and $J_{D'} = J(\cdot; D')$ for the two objectives, and $w = w^*(D)$ and $w' = w^*(D')$ for their minimizers.

1. **The two objectives differ by one term.** $J_{D'} - J_D = h$ with $h(v) = \frac{1}{n}(\ell(v, z') - \ell(v, z_i))$: the new example's loss in, the old one's out, each with weight $1/n$. Each loss has gradient of length at most L, and the length of a difference is at most the sum of the lengths (the triangle inequality), so $\|\nabla h(v)\| \le \frac{2L}{n}$ for every v.
2. **At w', the old objective has a small gradient.** Each minimizer has zero gradient for its own objective, $\nabla J_D(w) = 0$ and $\nabla J_{D'}(w') = 0$. So $\nabla J_D(w') = \nabla J_{D'}(w') - \nabla h(w') = -\nabla h(w')$. The point w' is not optimal for J_D, but the gradient of J_D there has length at most $2L/n$.
3. **Strong convexity of J_D.** J_D is an average of convex losses plus $\frac{\lambda}{2}\|w\|^2$, so it is λ-strongly convex (consequence (b) in Section 6.3: adding that term to a convex function adds curvature λ). Strong monotonicity Equation 6.7 then gives $\lambda\|w' - w\|^2 \le \langle \nabla J_D(w') - \nabla J_D(w), w' - w\rangle$.
4. **Substitute step 2 and apply Cauchy–Schwarz.** With $\nabla J_D(w) = 0$ and $\nabla J_D(w') = -\nabla h(w')$, $\lambda\|w' - w\|^2 \le \langle -\nabla h(w'), w' - w\rangle \le \frac{2L}{n}\|w' - w\|$.
5. **Divide** by $\lambda\|w' - w\|$ (if it is zero there is nothing to prove), which gives Equation 6.19.

In other words, this is consequence (a) of strong convexity, $\|w' - w\| \le \|\nabla J_D(w')\|/\lambda$ (a small gradient means you are close to the minimizer), applied to J_D at the point w', where its gradient is at most $2L/n$. With subgradients in place of gradients, the same steps work for non-differentiable losses such as the hinge loss.

The bound is tight. A loss with no curvature of its own shows it, because then the regularizer is the only curvature and nothing is lost. Take $d = 1$, labels $z_i \in \{-1, +1\}$, and the linear loss $\ell(w, z) = zw$, which is convex and 1-Lipschitz. Then $J(w; D) = \bar{z}w + \frac{\lambda}{2}w^2$ with $\bar{z}$ the mean label, and setting the derivative $\bar{z} + \lambda w$ to zero gives $w^*(D) = -\bar{z}/\lambda$. Changing one label from $+1$ to -1 changes the sum of the labels by 2, so it lowers $\bar{z}$ by $2/n$ and moves w^* by exactly $2/(\lambda n)$. For $n = 1{,}000$ and $\lambda = 0.1$ that is $2/100 = 0.02$.

6.9.4. Interpretation

Influence shrinks like $1/n$, because one example among n carries weight $1/n$ in the objective. Regularization buys stability: the curvature λ converts a small perturbation of the gradient into a small move of the minimizer, and halving λ can double the movement.

The bound also controls the loss. Because $\ell(\cdot, z)$ is L-Lipschitz, moving the parameters by at most $\frac{2L}{\lambda n}$ changes the loss on any point z by at most $L \cdot \frac{2L}{\lambda n} = \frac{2L^2}{\lambda n}$. A bound of this kind, on how much replacing one training example can change the loss at any point, is called **uniform stability**, and it is the input to the generalization bound in Algorithmic stability and generalization.

The bound is loose in two places. The proof used only the regularizer's curvature λ, while real data add curvature of their own. And it used the worst case $2L$ for the gradient difference, while two actual examples rarely have gradients pointing in opposite directions at full length. If G is μ-strongly convex with $\mu > \lambda$, because the data contribute curvature too, the same proof gives $2L/(\mu n)$.

6.9.5. Example: logistic regression

The script draws logistic datasets of size n, solves regularized logistic regression exactly with Newton's method, replaces one random example with a fresh draw, solves again, and records the largest movement over 20 replacements. The logistic loss is 1-Lipschitz, so the bound is $2/(\lambda n)$: for example $2/(0.1 \times 1{,}000) = 0.02$. Table 6.8 compares the two.

Table 6.8.: Largest movement of the regularized logistic minimizer over 20 single-example replacements, against $2L/(\lambda n)$ with $L = 1$.

n	$\lambda = 0.01$: observed / bound	$\lambda = 0.1$: observed / bound	$\lambda = 1$: observed / bound
100	2.52×10^{-1} / 2.00	5.98×10^{-2} / 0.200	8.55×10^{-3} / 0.0200
1,000	2.13×10^{-2} / 0.200	6.66×10^{-3} / 0.0200	8.54×10^{-4} / 0.00200
10,000	2.06×10^{-3} / 0.0200	6.35×10^{-4} / 0.00200	7.80×10^{-5} / 0.000200

Read Table 6.8 in both directions. Down a column, n grows tenfold and the movement shrinks tenfold, exactly as $1/n$ predicts. Across a row, the ratio of observed to bound changes with λ: it

is 0.39–0.43 at $\lambda = 1$ (for example $8.55 \times 10^{-3}/0.0200 = 0.43$), 0.30–0.33 at $\lambda = 0.1$ and 0.10–0.13 at $\lambda = 0.01$. The reason is the first source of slack above. At $\lambda = 1$ the regularizer supplies nearly all the curvature, and the remaining factor of about 2 is the gradient-difference slack. At $\lambda = 0.01$ the data supply more curvature than the regularizer (about 0.0233 at the unregularized minimizer, more for smaller w), so the bound, which only knows λ, overestimates the movement tenfold.

Figure 6.7 shows both trends. On the right, as $\lambda \to 0$ the bound blows up while the observed movement levels off at the value set by the data's own curvature.

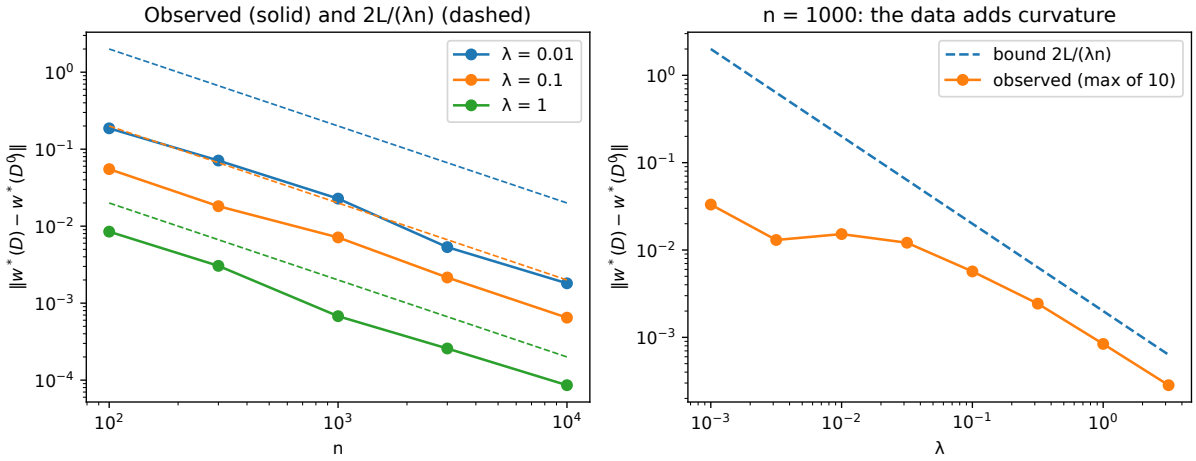


Figure 6.7.: Left: largest observed movement of the regularized logistic minimizer when one of n examples is replaced (solid), against $2L/(\lambda n)$ (dashed), for three values of λ. Right: the same at $n = 1000$ as a function of λ.

6.9.6. Why strong convexity is needed

Without the regularizer, convexity alone does not stop the minimizer from jumping, because a convex function can be flat, and on a flat stretch a tiny change in the data can move the minimizer a long way. Take the absolute loss $\ell(w, z) = |w - z|$, which is convex and 1-Lipschitz. The minimizer of $\frac{1}{n}\sum_i |w - z_i|$ is the median: moving w up by a little increases $|w - z_i|$ for each value below w and decreases it for each value above, so the risk stops decreasing when half the values are on each side. With $n = 2k + 1$ values, $k + 1$ of them 0 and k of them 1, the median is 0. Replace one 0 by a 1 and the majority switches, so the median is 1. The minimizer moves by 1 for every n: no $1/n$ at all. With $\frac{\lambda}{2}w^2$ added, Equation 6.19 caps the move at $2/(\lambda n)$ (Exercise 8 in the exercise book finds the actual move).

6.9.7. Why no such bound exists for non-convex objectives

For a non-convex loss even strong curvature in places is not enough, because the risk can have two separate valleys of nearly equal depth, and one example can decide which is deeper. Take labels $z_i \in \{-1, +1\}$ and the double-well loss $\ell(w, z) = \frac{1}{4}(w^2 - 1)^2 - zw$. The first term has two valleys, at $w = -1$ and $w = +1$, and the second tilts it towards the sign of z. On $[-2, 2]$ it is smooth and Lipschitz: its derivative $w^3 - w - z$ is at most $8 - 2 + 1 = 7$ in absolute value, and its

second derivative $3w^2 - 1$ lies in $[-1, 11]$. The empirical risk is $\frac{1}{4}(w^2 - 1)^2 - \bar{z}w$: two wells near ± 1, tilted towards the sign of the mean label $\bar{z}$.

With $n = 11$ examples, six labelled $+1$, the mean label is $\bar{z} = (6 - 5)/11 = 1/11$, the right well is lower, and the minimizer is $w = 1.0427$. Replace one $+1$ by -1: the mean label becomes $-1/11$, the tilt flips, and the minimizer jumps to -1.0427, a move of 2.0854 (Figure 6.8). With $n = 1{,}001$ (501 against 500) the move is still 2.0010. The minimizer jumps between wells regardless of n.

Adding $\frac{\lambda}{2}w^2$ does not fix this unless λ is large enough to make the whole objective convex. With $\lambda = 0.5$ and $\bar{z} = 0$ the derivative is $w^3 - w + 0.5w = w(w^2 - 0.5)$, so the wells sit at $\pm\sqrt{0.5} = \pm 0.71$, and a tilt still sends the minimizer across a gap of about 1.4. The proof of Equation 6.19 breaks at step 3, where it needs G to be strongly convex. Without it, the empirical minimizer can be a discontinuous function of the data, or not unique at all, so no bound on its movement can hold. For non-convex models, statements about the influence of one example must therefore be about the *algorithm* (for example, a fixed number of SGD steps from a fixed start), not about the exact minimizer.

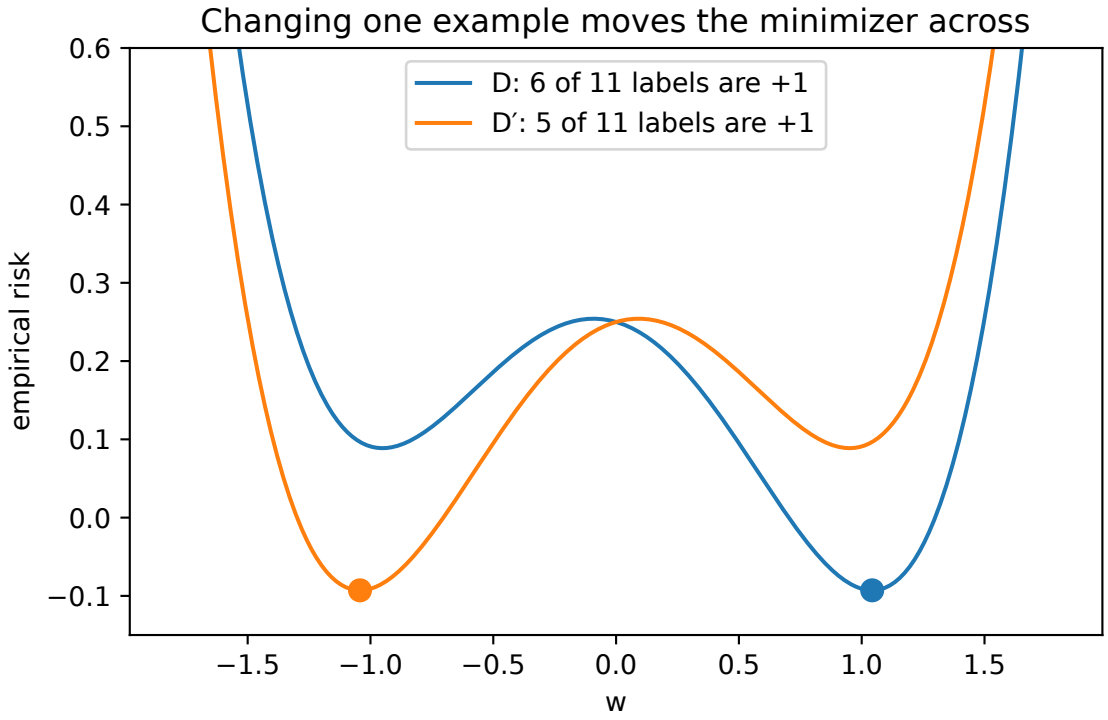


Figure 6.8.: A double-well empirical risk on 11 examples before (D, six labels $+1$) and after (D', five labels $+1$) one label changes. The dots mark the global minimizers, which jump from one well to the other.

6.9.8. Check yourself

- In the derivation, where did the factor 2 in $2L$ come from? *(From the gradient of the removed example and that of the added one, each of norm at most L, in step 1.)*

- If the loss is L-Lipschitz but you regularize with $\frac{\lambda}{2}\|w - w_0\|^2$ for some fixed w_0, does the bound change? *(No. The regularizer is still λ-strongly convex, and nothing else in the proof involved it.)*

6.10. Code: check every bound by simulation

The script code/optimization.py draws every figure in this chapter and computes every number quoted in it. Each loss, gradient, bound and simulation is a small typed function; the script calls them once for the table and loops them over parameters for the figures. From the repository root:

```
uv run python code/part01-prerequisites/optimization.py
```

Output (seed 0):

Table 6.9.: Summary of the script's output at seed 0.

Experiment	Result
Logistic example $x = (0.6, 0.8)$, $y = +1$	At $w = 0$: loss 0.6931, gradient norm 0.5000, curvature 0.2500. At $w = (3, 4)$: 0.0067, 0.0067, 0.0066
GD on least squares, $\kappa = 9.58$	122 steps to shrink the error by 10^6; Equation 6.9 predicts at most 126
Minibatch variance at w^*, $\sigma_1^2 = 0.1722$	Matches Equation 6.15 for $B = 1$ to 500 (Table 6.5)
Projected SGD, convex, $T = 10^4$	7.71×10^{-4} against the bound 0.050 (Table 6.3)
Projected SGD, $\lambda = 0.1$, $T = 10^4$	Uniform average 9.09×10^{-5} (bound 1.15×10^{-2}); weighted 7.85×10^{-5} (bound 4.50×10^{-3}); last iterate 9.61×10^{-5}
Added gradient noise, $T = 10^4$	Best grid step 0.178, 0.056, 0.056 for $\sigma = 0, 1, 3$ (Table 6.6)
Non-convex sigmoid loss, $T = 10^4$	1.87×10^{-4} against the bound 1.55×10^{-3} (Table 6.7)
Regularized ERM, one example replaced	Observed/bound between 0.10 and 0.43 (Table 6.8)
Double well, $n = 11$ and 1,001	Minimizer moves by 2.0854 and 2.0010

tests/test_optimization.py checks that the output still matches Table 6.9 and the tables above.

What to take from it

The bounds hold, with room to spare. Every worst-case constant (R, L, G, Δ, σ^2) is an upper bound that the actual run does not reach, so the guarantees are loose by factors of about 10 to 130. Their *shape*, the way they depend on T, n, λ and σ, is what to trust.

The looseness has a common cause: real problems have more curvature than the assumptions grant. Unregularized logistic regression converged like $1/T$ rather than $1/\sqrt{T}$, and its regularized minimizer moved less than the λ-only bound predicts, both because the data themselves curve the risk.

Noise costs steps, and the right step size shrinks as the noise grows, so tune η again whenever the gradient noise changes.

Finally, strong convexity is what makes the minimizer stable. With it, one example moves the solution by $O(1/(\lambda n))$. Without it, even convexity is not enough, as the median shows, and non-convexity lets the minimizer jump between wells.

6.11. References

- Shalev-Shwartz & Ben-David, *Understanding Machine Learning* (2014), ch. 12 (convexity, Lipschitzness, smoothness), ch. 13 (regularization and stability), ch. 14 (SGD, including Theorem 14.8 for the convex Lipschitz case and Theorem 14.11 for the strongly convex case): the closest match to this chapter, at the same level
- Bubeck, "Convex Optimization: Algorithms and Complexity" (2015), ch. 3 and §6.1: gradient methods for each combination of the assumptions, with lower bounds (§3.5) and SGD
- Nesterov, *Lectures on Convex Optimization* (2018), §2.1: the equivalent forms of smoothness and strong convexity, with proofs
- Ghadimi & Lan, "Stochastic First- and Zeroth-Order Methods for Nonconvex Stochastic Programming" (SIAM J. Optim. 2013): the non-convex SGD rate
- Lacoste-Julien, Schmidt & Bach, "A Simpler Approach to Obtaining an O(1/t) Convergence Rate for the Projected Stochastic Subgradient Method" (2012); Rakhlin, Shamir & Sridharan, "Making Gradient Descent Optimal for Strongly Convex Stochastic Optimization" (ICML 2012): averaging schemes without the $\ln T$
- Bottou, Curtis & Nocedal, "Optimization Methods for Large-Scale Machine Learning" (SIAM Review 2018): noise, step sizes and batch sizes in practice
- Bousquet & Elisseeff, "Stability and Generalization" (JMLR 2002): the stability of regularized ERM and what it implies

6.12. Cheat sheet

Table 6.10 collects every result in this chapter.

Table 6.10.: Every result in this chapter.

Result	Assumptions	Bound
Lipschitz	$\|\nabla f\| \le L$	$\|f(w) - f(w')\| \le L\|w - w'\|$
Descent lemma	β-smooth	$f(w') \le f(w) + \langle \nabla f(w), w' - w\rangle + \frac{\beta}{2}\|w' - w\|^2$

Result	Assumptions	Bound
Strong monotonicity	λ-strongly convex	$\langle \nabla f(w) - \nabla f(w'), w - w' \rangle \geq \lambda \|w - w'\|^2$; $\|w - w^*\| \leq \|\nabla f(w)\|/\lambda$
Logistic loss	$\|x\| \leq 1$	1-Lipschitz, $\frac{1}{4}$-smooth, convex, not strongly convex
GD, quadratic	$\lambda I \preceq H \preceq \beta I$, $\eta = 1/\beta$	$\|w_t - w^*\| \leq (1 - 1/\kappa)^t \|w_0 - w^*\|$
SGD, convex	$\mathbb{E}\|g\|^2 \leq G^2$, $\|w_1 - w^*\| \leq R$, $\eta = R/(G\sqrt{T})$	$\mathbb{E}F(\bar{w}_T) - F^* \leq RG/\sqrt{T}$
SGD, strongly convex	$\eta_t = 1/(\lambda t)$, uniform average	$G^2(1 + \ln T)/(2\lambda T)$
SGD, strongly convex	$\eta_t = 2/(\lambda(t+1))$, t-weighted average	$2G^2/(\lambda(T+1))$
SGD, non-convex	β-smooth, variance σ^2, $\eta \leq 1/\beta$	$\frac{1}{T}\sum_t \mathbb{E}\|\nabla F(w_t)\|^2 \leq \frac{2\Delta}{\eta T} + \beta\eta\sigma^2$
Minibatch variance	B of n, uniform	σ_1^2/B; times $\frac{n-B}{n-1}$ without replacement
Added gradient noise	$\mathbb{E}\|\xi\|^2 = \sigma^2$	$G^2 = L^2 + \sigma^2$; $\eta^* = R/\sqrt{(L^2+\sigma^2)T}$
ERM sensitivity	Convex L-Lipschitz loss, $\frac{\lambda}{2}\|w\|^2$	$\|w^*(D) - w^*(D')\| \leq 2L/(\lambda n)$; loss changes by $\leq 2L^2/(\lambda n)$

Rules of thumb

- Lipschitz bounds the gradient; smooth bounds the change of the gradient. Convex Lipschitz SGD needs the first, step sizes and non-convex analysis need the second.
- Without curvature, SGD's error falls like $1/\sqrt{T}$; with λ-strong convexity, like $1/(\lambda T)$. Average the iterates either way.
- Noise multiplies the number of steps by $1 + \sigma^2/L^2$ in the convex bound. Lower the step size when the gradients get noisier.
- One example moves a λ-regularized minimizer by at most $2L/(\lambda n)$. Without strong convexity there is no such bound.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

7. Algorithmic stability and generalization

7.1. Why this matters

You train a model and its training loss is low. **Will its loss on new data be low too?** The difference between the loss on new data and the loss on the training data is the **generalization gap**, and controlling it is the central question of learning theory. The classical answer counts hypotheses: it measures how rich the class of models is that the algorithm could output, through the size of a finite class, a Vapnik–Chervonenkis dimension or a Rademacher complexity, and argues that a class that is not too rich cannot fit noise. This chapter takes a different route. It looks at the *algorithm* instead of the class of models it could output.

The organizing idea fits in one sentence: **a model generalizes when it does not depend much on any single training example.** Making it precise gives three results, and each one leads to the next.

The first is that **the gap is exactly an average per-example effect.** Take one training example out, put a fresh one from the same source in its place, retrain, and see how much worse the model now does on the example you took out. Average this over the examples. The result is the expected gap, with no inequality involved: overfitting *is* dependence on individual examples.

Once the gap is written that way, the second result follows: **a bounded per-example effect gives a bounded gap.** Call β the largest amount by which replacing one example can move the loss of the trained model, at any test point. Each term of the average is then at most β, so the expected gap is at most β. McDiarmid's inequality, which says that a function of many independent inputs that no single input can move by much stays close to its mean, turns this average statement into one that holds with high probability for the one model you trained.

The third result makes this useful: **common algorithms have small β.** Regularized empirical risk minimization (ERM), which minimizes the training loss plus a penalty $\frac{\lambda}{2}\|w\|^2$ on the weights, has $\beta = 2L^2/(\lambda n)$, where n is the number of examples and L bounds how fast the loss can change with the weights. Stochastic gradient descent (SGD) with step sizes η_t has $\beta \le (2L^2/n)\sum_t \eta_t$, which grows with training time and gives a reason to stop early. Memorizing algorithms such as 1-nearest neighbour have $\beta = 1$, the largest value for a loss between 0 and 1, and when labels are noisy their gap never closes.

Where each result is used (Table 7.1):

Table 7.1.: Results of this chapter and where they are used.

Result	Typical use
Expected gap = average replace-one effect	Reading overfitting as memorization of individual examples

Result	Typical use
$\lvert\mathbb{E}[\text{gap}]\rvert \le \beta$	Generalization guarantees that need no complexity measure of the model class
High-probability bound (McDiarmid)	A guarantee for the one model you actually trained
Regularized ERM, $\beta = 2L^2/(\lambda n)$	Choosing λ; bounding how far one example moves a trained model
SGD, $\beta \le (2L^2/n)\sum_t \eta_t$	Step sizes, number of epochs, early stopping
Unstable algorithms ($\beta = 1$)	Recognizing models whose training error says nothing about test error

Notation. This chapter departs from the part's shared notation in one place. Following the learning-theory literature, it writes the training set as $S = (z_1, \dots, z_n)$ rather than D, and uses $\mathcal{D}$ for the distribution the examples are drawn from.

Later parts return to how much a trained model depends on a single example, and they reuse the quantities defined here. Every fact this chapter borrows is restated where it is used. The full derivation of McDiarmid's inequality is in Concentration inequalities, and more on the convexity facts behind the ERM bound is in Optimization.

7.2. Toolkit: five facts every derivation uses

Each section below follows the same arc: the question, the statement, a derivation, an interpretation, an example and a self-check. The derivations reuse five facts. The first turns a change in the weights into a change in the loss; the second is the symmetry behind the main identity; the third and fourth say when an optimizer cannot let two nearby weight vectors drift apart; the fifth turns an average statement into a high-probability one.

7.2.1. Fact 1 — Lipschitz losses turn parameter changes into loss changes

A loss is **L-Lipschitz** in the weights when moving the weights by a distance d changes the loss by at most Ld, whatever the example. The constant L is a speed limit: the loss cannot change faster than L per unit of distance in weight space. Written out, if $w \mapsto \ell(w, z)$ is L-Lipschitz for every example z, then

$$|\ell(w, z) - \ell(w', z)| \le L\,\|w - w'\| \quad \text{for all } z. \tag{7.1}$$

This is how every derivation below finishes. They first bound how far the trained weights move when one example changes, and this fact turns that into a bound on how far the loss moves, at every test point at once. For a differentiable loss, L can be taken as the largest norm of the gradient $\nabla_w \ell(w, z)$, because the change along a straight path from w' to w is the integral of the gradient along it, and each bit of the path contributes at most (gradient norm) $\times$ (length).

The logistic loss is the example used throughout. For features x and a label $y = \pm 1$, it is $\ell(w, (x, y)) = \ln(1 + e^{-y\langle w, x\rangle})$, which is small when the **margin** $y\langle w, x\rangle$ is large and positive. Its

gradient is $-y\,\sigma(-y\langle w, x\rangle)\,x$, where $\sigma(t) = 1/(1+e^{-t})$ is the logistic function, which takes values between 0 and 1. Since $0 < \sigma < 1$ and $|y| = 1$, the gradient norm is at most $\|x\|$. With features scaled to $\|x\| \le 1$, the logistic loss is 1-Lipschitz.

7.2.2. Fact 2 — Swapping two i.i.d. variables does not change their joint law

If U and V are independent and identically distributed (i.i.d.), then nothing in their joint behaviour can tell which one is "first". So the pair (U, V) and the swapped pair (V, U) have the same joint distribution, and as a consequence $\mathbb{E}[f(U, V)] = \mathbb{E}[f(V, U)]$ for every function f.

The reason is the joint density (or probability mass function). Independence makes it the product $p(u)\,p(v)$ of the two individual densities, and identical distribution makes those two the same function p. The product $p(u)\,p(v)$ does not change when u and v are exchanged, so the two pairs have the same law, and expectations computed from that law agree.

This is the "renaming trick" in the expected-gap theorem. The theorem section walks through it with a table of every outcome.

7.2.3. Fact 3 — Strong convexity makes the gradient strongly monotone

A function G is **λ-strongly convex** when it lies above each of its tangent planes by at least a quadratic bowl of curvature λ: for all u and v,

$$G(v) \ge G(u) + \langle \nabla G(u), v - u\rangle + \frac{\lambda}{2}\|v - u\|^2. \tag{7.2}$$

Adding $\frac{\lambda}{2}\|w\|^2$ to any convex function produces such a G. The consequence used below is that the gradient changes at least as fast as the point does, in the direction of the change: if you move from v to u, the gradient moves by at least $\lambda\|u - v\|$ along that direction. In symbols,

$$\langle \nabla G(u) - \nabla G(v),\, u - v\rangle \ge \lambda\|u - v\|^2. \tag{7.3}$$

To see why, write the strong convexity inequality as given, and again with u and v exchanged: $G(u) \ge G(v) + \langle \nabla G(v), u - v\rangle + \frac{\lambda}{2}\|u - v\|^2$. Add the two inequalities. The values $G(u)$ and $G(v)$ appear on both sides and cancel, and what is left is $0 \ge -\langle \nabla G(u) - \nabla G(v), u - v\rangle + \lambda\|u - v\|^2$, which rearranges to the claim. The stability of regularized ERM rests on this fact: two different minimizers have gradients that must differ by a lot, so if the gradients differ by little, the minimizers are close.

7.2.4. Fact 4 — For convex smooth functions, a small gradient step is non-expansive

A function f is **ρ-smooth** when its gradient ∇f is ρ-Lipschitz: the gradient changes by at most ρ per unit of distance, so the function curves no more sharply than $\frac{\rho}{2}\|w\|^2$. For a function that is both convex and smooth, the gradient satisfies a property called **co-coercivity** (Nesterov 2004, Theorem 2.1.5). In words: when two gradients differ by a lot, the two points must be far apart along the direction in which the gradients differ. Precisely,

$$\langle \nabla f(u) - \nabla f(v),\, u - v\rangle \ge \frac{1}{\rho}\|\nabla f(u) - \nabla f(v)\|^2. \tag{7.4}$$

The proof needs more than a few lines and is in Nesterov's book.

What this chapter uses is a consequence for gradient descent. Take two starting points u and v and apply the same gradient step to both, $\psi(w) = w - \eta \nabla f(w)$. If the step size satisfies $0 \le \eta \le 2/\rho$, the two points do not end up farther apart than they started:

$$\|\psi(u) - \psi(v)\| \le \|u - v\|. \tag{7.5}$$

The reason is that the step moves both points by an amount that depends on the gradient difference $\Delta = \nabla f(u) - \nabla f(v)$, and co-coercivity says this difference points enough "towards closing the distance". Expand the square of $\psi(u) - \psi(v) = (u - v) - \eta\Delta$, then use co-coercivity to replace $\langle \Delta, u - v \rangle$ by the smaller quantity $\|\Delta\|^2/\rho$:

$$\|\psi(u) - \psi(v)\|^2 = \|u - v\|^2 - 2\eta\langle \Delta, u - v\rangle + \eta^2\|\Delta\|^2 \le \|u - v\|^2 - \eta\left(\frac{2}{\rho} - \eta\right)\|\Delta\|^2 \le \|u - v\|^2. \tag{7.6}$$

The last step holds because $\eta \le 2/\rho$ makes the subtracted term non-negative.

A one-dimensional case shows why the condition on η is needed. For $f(w) = \frac{\rho}{2}w^2$, the gradient is ρw, so the step is $\psi(w) = w - \eta\rho w = (1 - \eta\rho)w$, and it multiplies every distance by $|1 - \eta\rho|$. It is non-expansive exactly when $|1 - \eta\rho| \le 1$, that is $\eta \le 2/\rho$. Larger steps overshoot the minimum and blow differences up. The stability of SGD rests on this fact.

7.2.5. Fact 5 — McDiarmid's inequality

A function of many independent inputs, none of which can move it by much, concentrates near its mean. If $Z_1, \dots, Z_n$ are independent and changing only Z_i, with the others held fixed, changes $\Phi(Z_1, \dots, Z_n)$ by at most c_i, then

$$\Pr[\Phi - \mathbb{E}\Phi \ge t] \le \exp\Big(-\frac{2t^2}{\sum_i c_i^2}\Big). \tag{7.7}$$

The derivation, via the Doob martingale and Azuma's inequality, is in Concentration inequalities. Its idea is to reveal the inputs one at a time: each revealed input shifts the conditional mean of Φ by at most c_i, and a sum of n such bounded, mean-zero shifts concentrates in the same way as a sum of independent bounded variables does.

To use it you set a failure probability δ and solve for the matching t. Setting $\exp(-2t^2/\sum_i c_i^2) = \delta$ and taking logarithms gives $t = \sqrt{\tfrac{1}{2}\sum_i c_i^2 \ln(1/\delta)}$. So with probability at least $1 - \delta$, $\Phi < \mathbb{E}\Phi + \sqrt{\tfrac{1}{2}\sum_i c_i^2 \ln(1/\delta)}$.

In this chapter β always denotes a stability parameter, and δ the failure probability.

7.3. Risk, empirical risk and the gap

7.3.1. Context

Training minimizes the loss on the data you have. What you care about is the loss on data you will see later. For a model chosen *before* looking at the data, the training loss is an unbiased estimate of the test loss. For a model *fitted* to the data, it is biased downward, because the fitting process picked the model that looks good on exactly these examples. The size of that bias is the generalization gap.

7.3.2. Definitions

A training set $S = (z_1, \dots, z_n)$ is a list of n examples drawn i.i.d. from a distribution $\mathcal{D}$, the source of all data, past and future. A learning algorithm A maps S to a model $A(S)$, and the loss $\ell(w, z) \geq 0$ measures how badly model w does on example z.

The quantity you care about is the average loss on a new example from $\mathcal{D}$. This is the **population risk**, or test loss, $R(w) = \mathbb{E}_{z\sim\mathcal{D}}\,\ell(w, z)$. You cannot compute it, because you do not know $\mathcal{D}$. What you can compute is the average loss over the training set, the **empirical risk** or training loss, $\hat{R}_S(w) = \frac{1}{n}\sum_{i=1}^{n}\ell(w, z_i)$. The **generalization gap** of the trained model is the test loss minus the training loss, $\mathrm{gap}(S) = R(A(S)) - \hat{R}_S(A(S))$; it is positive when the model looks better on its training data than it really is.

The tool for studying the gap is a training set that differs from S in one example. The **replace-one neighbour** $S^{(i)} = (z_1, \dots, z_{i-1}, z'_i, z_{i+1}, \dots, z_n)$ is the training set with example i replaced by z'_i. In probability statements, $z'_1, \dots, z'_n$ are fresh i.i.d. draws from $\mathcal{D}$, independent of S.

7.3.3. Statement

A model that was fixed before the data were drawn has a training loss that is right on average: averaging over training sets, its training loss equals its test loss. For a fixed model w that does not depend on S,

$$\mathbb{E}_S[\hat{R}_S(w)] = R(w). \tag{7.8}$$

For $w = A(S)$ this fails in general, and $\mathbb{E}[\hat{R}_S(A(S))]$ is usually smaller than $\mathbb{E}[R(A(S))]$.

7.3.4. Derivation

1. **Split the average.** The expectation of an average is the average of the expectations (linearity of expectation), so $\mathbb{E}_S[\hat{R}_S(w)] = \frac{1}{n}\sum_i \mathbb{E}[\ell(w, z_i)]$.
2. **Each term is the test loss.** Each z_i is a draw from $\mathcal{D}$ and w is fixed, so $\mathbb{E}[\ell(w, z_i)]$ is the average loss of w on a draw from $\mathcal{D}$, which is $R(w)$ by definition. The average of n copies of $R(w)$ is $R(w)$.
3. **Why it fails for a trained model.** If $w = A(S)$, step 2 breaks: z_i and $A(S)$ are dependent, because z_i was used to choose $A(S)$. The algorithm picked the model that does well on these particular z_i, so z_i is no longer a typical draw from the model's point of view.

7.3.5. Interpretation

The gap is not noise that averages out. It is a systematic bias caused by reusing the same data to choose and to evaluate. Its sign is usually positive: models look better on the data they were fitted to. The rest of this chapter bounds it through one property of the algorithm, without counting the models it could output.

7.3.6. Example: an overfit logistic regression

Take the running example of this chapter. Features x are uniform on the unit sphere in $\mathbb{R}^{10}$. Labels are $y = \pm 1$ with $\Pr[y = 1 \mid x] = \sigma(5x_1)$, so the label tends to follow the sign of the first coordinate but is noisy near $x_1 = 0$. The model is λ-regularized logistic regression, which minimizes the average logistic loss plus $\frac{\lambda}{2}\|w\|^2$ (details in Section 7.10). With $n = 50$ and $\lambda = 0.01$, averaged over 100 training sets, the training loss is 0.4603 and the test loss is 0.5781. The expected gap, estimated with the noise-reduction trick described in the regularized-ERM section, is about 0.119, close to the difference of the two averages. With $n = 1000$ it shrinks to about 0.006.

7.3.7. Check yourself

- You evaluate a fixed, pretrained model on a fresh sample. Is the sample loss biased? *(No. The model does not depend on the sample, so step 2 applies.)*
- You pick the best of 100 pretrained models on a validation set. Is the validation loss of the winner biased? *(Yes, downward. The choice depends on the validation data, so the choice is itself a learning algorithm run on that data.)*

7.4. Uniform stability and the expected gap

7.4.1. Context

To control the gap you need to say how much $A(S)$ depends on the training set. The simplest way to say it: replace one training example, retrain, and see how much the loss of the trained model changes, anywhere.

7.4.2. Definition

An algorithm is stable in this sense when replacing any one training example by any other moves the loss of the trained model by at most β, at every possible test point. The bound must hold in the worst case: for every training set, every position, every replacement and every test point. Formally, A is β-**uniformly stable** if for every training set S, every index i, every replacement z_i' and every test point z,

$$\left|\ell(A(S), z) - \ell(A(S^{(i)}), z)\right| \le \beta. \tag{7.9}$$

If A is randomized (SGD is, through the order in which it visits examples), the condition is on the expectation over A's internal randomness: $\left|\mathbb{E}_A[\ell(A(S), z) - \ell(A(S^{(i)}), z)]\right| \le \beta$.

Usually β depends on n, because with more examples each one carries less weight, and the interesting question is how fast it goes to zero. Figure 7.1 shows the definition at work on a one-dimensional logistic regression: one example's label is flipped, and the fitted model moves a lot under weak regularization and barely under strong regularization.

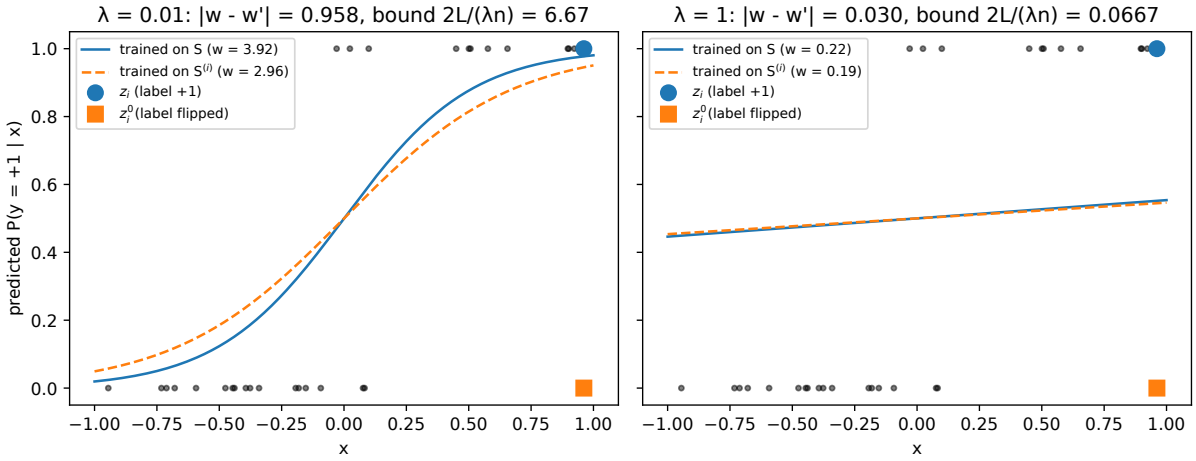


Figure 7.1.: One-dimensional logistic regression trained on 30 points, on S and on $S^{(i)}$, where the most typical positive example is replaced by the same point with its label flipped. Left: $\lambda = 0.01$. Right: $\lambda = 1$. The titles compare the change in the weight with the bound $2L/(\lambda n)$ derived later.

7.4.3. Statement

The test loss of the trained model can be measured on a fresh point z_i'. Swap the roles of z_i and z_i', and the same number becomes the loss on z_i of a model trained *without* z_i. So the expected test loss is the average loss of each example under a model that never saw it, while the training loss is the average loss of each example under a model that did. Their difference is the expected gap. For any algorithm, the expected gap is exactly this average replace-one effect:

$$\mathbb{E}_S[\mathrm{gap}(S)] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{S,z_i'}\Big[\ell(A(S^{(i)}), z_i) - \ell(A(S), z_i)\Big]. \tag{7.10}$$

Each bracket is a replace-one change at a test point, which uniform stability caps at β. Consequently, if A is β-uniformly stable,

$$\big|\mathbb{E}_S[\mathrm{gap}(S)]\big| \le \beta. \tag{7.11}$$

7.4.4. Derivation (the renaming trick)

1. **Write the test loss with a fresh point.** The test loss $R(A(S))$ is the loss on a point the model has not seen. The point z_i' is such a point: it is independent of S and has distribution $\mathcal{D}$. So, holding S fixed, its average loss is the test loss, $\mathbb{E}[\ell(A(S), z_i') \mid S] = R(A(S))$. Averaging over S as well, $\mathbb{E}[R(A(S))] = \mathbb{E}[\ell(A(S), z_i')]$ for every i.

2. **Rename $z_i \leftrightarrow z_i'$.** The two points z_i and z_i' are i.i.d. and independent of the other examples. By Fact 2 (swapping two i.i.d. variables leaves their joint distribution unchanged), exchanging them does not change the joint distribution of $(z_1, \dots, z_n, z_i')$, so it does not change any expectation. The swap turns S into $S^{(i)}$ and z_i' into z_i. Hence

$$\mathbb{E}[\ell(A(S), z_i')] = \mathbb{E}[\ell(A(S^{(i)}), z_i)]. \tag{7.12}$$

3. **Average over i.** Steps 1 and 2 hold for every i, so their average does too: $\mathbb{E}[R(A(S))] = \frac{1}{n}\sum_i \mathbb{E}[\ell(A(S^{(i)}), z_i)]$.
4. **Training loss.** By definition of the empirical risk, $\mathbb{E}[\hat{R}_S(A(S))] = \frac{1}{n}\sum_i \mathbb{E}[\ell(A(S), z_i)]$.
5. **Subtract** step 4 from step 3. The two sums run over the same i, so the difference is the average of the differences. This is Equation 7.10.
6. **Bound each term.** By Equation 7.9, applied at the test point $z = z_i$, each difference $\ell(A(S^{(i)}), z_i) - \ell(A(S), z_i)$ lies in $[-\beta, \beta]$ for every outcome. An average of numbers in $[-\beta, \beta]$ also lies there, so the expectation does, and so does the average over i. This is Equation 7.11. For a randomized A, take the expectation over A's randomness first and use the randomized definition.

Step 2 on a tiny case. Readers often stall on the renaming step, so here it is with every outcome listed. Take $n = 1$: one example $z_1 \in \{0, 1\}$ with $\Pr[z_1 = 1] = p = 0.3$, and a fresh copy z_1'. The algorithm shrinks the data towards zero, $A(S) = z_1/2$, and the loss is squared error $\ell(w, z) = (w - z)^2$. So $A(S^{(1)}) = z_1'/2$. Table 7.2 lists all four outcomes, each with its probability (the product of the two independent probabilities) and the three losses that appear in the derivation.

Table 7.2.: Every outcome of the one-example case. The renamed column is the test column with the rows $(0, 1)$ and $(1, 0)$ exchanged. Those rows have the same probability, so the averages agree.

(z_1, z_1')	Probability	Train $\ell(A(S), z_1)$	Test $\ell(A(S), z_1')$	Renamed $\ell(A(S^{(1)}), z_1)$
$(0, 0)$	$0.7 \times 0.7 = 0.49$	0	0	0
$(0, 1)$	$0.7 \times 0.3 = 0.21$	0	$(0-1)^2 = 1$	$(\frac{1}{2} - 0)^2 = 0.25$
$(1, 0)$	$0.3 \times 0.7 = 0.21$	$(\frac{1}{2} - 1)^2 = 0.25$	$(\frac{1}{2} - 0)^2 = 0.25$	$(0-1)^2 = 1$
$(1, 1)$	$0.3 \times 0.3 = 0.09$	0.25	0.25	0.25
Average		0.075	0.285	0.285

Row by row, the test and renamed columns differ. But the renamed column contains the same values as the test column, attached to the mirror-image outcome, and mirror-image outcomes have the same probability $p(1-p) = 0.21$. That is all step 2 says: averaging over a table does not care how you label its rows.

The averages are probability-weighted sums of each column. Train: $0.21 \times 0.25 + 0.09 \times 0.25 = 0.075$. Test: $0.21 \times 1 + 0.21 \times 0.25 + 0.09 \times 0.25 = 0.285$. So the expected gap is $0.285 - 0.075 = 0.21$. The right side of Equation 7.10 is the renamed average minus the train average, $0.285 - 0.075 = 0.21$ too. The stability of this algorithm is $\beta = 0.75$: changing z_1 from 0 to 1 moves the model from 0 to $\frac{1}{2}$, and at the test point $z = 1$ the loss drops from $(0-1)^2 = 1$ to $(\frac{1}{2} - 1)^2 = 0.25$, a change of 0.75. The gap 0.21 is below $\beta = 0.75$, as Equation 7.11 requires.

7.4.5. Interpretation

Equation 7.10 is an identity, not a bound. It says the expected gap *is* the average **replace-one self-influence**: how much the loss on z_i goes up when z_i is taken out of the training set and replaced by a fresh point. A model that relies heavily on individual examples to fit them overfits, and conversely, a model that overfits must be relying on individual examples. The memorization section below measures this quantity directly.

Uniform stability, by contrast, is a worst-case notion. It asks for a bound over all training sets, all replacements and all test points, while the identity only needs the average over i and over the data. This is why the bound β can be far from the true gap, and why weaker, average-case notions of stability exist (Bousquet & Elisseeff 2002; Shalev-Shwartz et al. 2010).

What makes the bound useful is what it leaves out: no model class appears. The bound holds for any algorithm, however rich the set of models it could output. This is why stability gives guarantees for algorithms such as SGD on large models, where counting the models the algorithm could output gives nothing useful.

7.4.6. Example: a memorizing lookup table

Store the training set; predict the stored label at a training point and $+1$ everywhere else, and score with the **0-1 loss**, which is 1 for a wrong prediction and 0 for a right one. With continuous features, a fresh point almost surely misses the table, so it gets the prediction $+1$. In the running example, where half the labels are -1, that prediction is wrong half the time: the test error is $\frac{1}{2}$ and the training error is 0, for every n. Now replace (x_i, y_i) by a different point. The prediction at x_i falls back to $+1$; if $y_i = -1$, the loss at $z = (x_i, y_i)$ jumps from 0 to 1. So $\beta = 1$, and the gap $\frac{1}{2} \leq 1$ is consistent with the theorem but never shrinks.

7.4.7. Check yourself

- In step 1, why is z_i' needed at all; why not write the test loss as $\mathbb{E}[\ell(A(S), z_i)]$? *(Because z_i is part of S, so $A(S)$ depends on it. Only a point independent of S gives an unbiased estimate of $R(A(S))$.)*
- In Table 7.2, what would break if z_1 and z_1' had different distributions? *(Rows $(0, 1)$ and $(1, 0)$ would have different probabilities, and the test and renamed averages would differ. Step 2 needs the two points to be exchangeable, that is, to have a joint distribution that does not change when they are swapped.)*

7.5. High probability: the Bousquet–Elisseeff bound

7.5.1. Context

Equation 7.11 controls the gap on average over training sets. You train once, on one S, so you need the gap of *your* model to be small with high probability. The gap is a function of n independent examples, and stability bounds how much one example can move it. That is exactly the situation

of McDiarmid's inequality (Fact 5): a function of independent inputs that no single input can move by much stays close to its mean, with an exponentially small chance of exceeding it.

7.5.2. Statement

The bound has three parts: the training loss you observe, the expected gap β from the previous section, and a deviation term from McDiarmid that says how far one training set can stray from the average. If A is β-uniformly stable and the loss lies between 0 and M, then with probability at least $1-\delta$ over S,

$$R(A(S)) \le \hat{R}_S(A(S)) + \beta + (2n\beta + M)\sqrt{\frac{\ln(1/\delta)}{2n}}. \tag{7.13}$$

This is Theorem 12 of Bousquet & Elisseeff (2002), rewritten for replace-one stability. They define stability by *removing* an example instead of replacing it. Replacing can be done by removing z_i and then adding z_i', two removals' worth of change, so replace-one stability is at most twice removal stability (Exercise 1 in the exercise book). Substituting $\beta = 2\beta_{\text{remove}}$ in Equation 7.13 gives their $2\beta_{\text{remove}} + (4n\beta_{\text{remove}} + M)\sqrt{\ln(1/\delta)/(2n)}$.

7.5.3. Derivation

The plan is to bound how much one example can move the gap, feed that to McDiarmid, and then use the expected-gap bound for the mean. Let $\Phi(S) = R(A(S)) - \hat{R}_S(A(S))$ be the gap as a function of the training set.

1. **Effect of one example on the test loss.** Replacing z_i moves $\ell(A(S), z)$ by at most β at every z, by uniform stability. The test loss is an average of these losses over z, and an average of numbers that each move by at most β moves by at most β: $|R(A(S)) - R(A(S^{(i)}))| \le \beta$.
2. **Effect on the training loss.** $\hat{R}_S(A(S))$ and $\hat{R}_{S^{(i)}}(A(S^{(i)}))$ share $n-1$ terms, one for each example other than i. Each shared term $\frac{1}{n}\ell(\cdot, z_j)$ is evaluated at the same point z_j under two models, so by stability it moves by at most β/n. The unshared terms are $\frac{1}{n}\ell(A(S), z_i)$ and $\frac{1}{n}\ell(A(S^{(i)}), z_i')$, evaluated at two different points, so stability says nothing about them; but both lie in $[0, M/n]$, so they differ by at most M/n. Total: at most $\frac{n-1}{n}\beta + \frac{M}{n} \le \beta + \frac{M}{n}$.
3. **Bounded differences.** Φ is the test loss minus the training loss, and by the triangle inequality ($|a-b| \le |a| + |b|$) its change is at most the sum of the two changes: replacing z_i moves Φ by at most $c_i = 2\beta + M/n$. Uniform stability holds for every training set and every replacement, so this holds for all values of the inputs, as McDiarmid requires.
4. **McDiarmid** (Fact 5). All n constants are equal, so $\sum_i c_i^2 = n(2\beta + M/n)^2$. Its square root is $\sqrt{n}(2\beta + M/n)$, and $\sqrt{n}(2\beta + M/n)/\sqrt{2} = (2n\beta + M)/\sqrt{2n}$ after multiplying inside and out by $\sqrt{n}$. So with probability at least $1-\delta$,
$$\Phi < \mathbb{E}\Phi + \sqrt{n}\Big(2\beta + \frac{M}{n}\Big)\sqrt{\frac{\ln(1/\delta)}{2}} = \mathbb{E}\Phi + (2n\beta + M)\sqrt{\frac{\ln(1/\delta)}{2n}}. \tag{7.14}$$
5. **Expected gap.** $\mathbb{E}\Phi \le \beta$ by Equation 7.11. Substitute, and move the training loss to the right-hand side.

Applying the same argument to $-\Phi$ gives the matching lower bound, so the gap is controlled from both sides.

7.5.4. Interpretation: β must be $o(1/\sqrt{n})$

The width of the bound beyond β has two parts, and they behave differently. The part $M\sqrt{\ln(1/\delta)/(2n)}$ is what Hoeffding's inequality gives for one fixed model: the average of n independent losses in $[0, M]$ exceeds its mean by more than this with probability at most δ (Concentration inequalities). It is a rate you cannot avoid even without any learning. The extra part comes from stability, $2n\beta\sqrt{\ln(1/\delta)/(2n)} = \beta\sqrt{2n\ln(1/\delta)}$, and it goes to zero only if $\beta\sqrt{n} \to 0$, which is what $\beta = o(1/\sqrt{n})$ means. Figure 7.2 shows three regimes, and the rate in each follows from putting the given β into $\beta\sqrt{2n\ln(1/\delta)}$:

- $\beta \sim 1/n$ (regularized ERM): $\beta\sqrt{n} \sim 1/\sqrt{n}$, so the width falls like $1/\sqrt{n}$, a constant factor above the fixed-model rate.
- $\beta \sim n^{-3/4}$: $\beta\sqrt{n} \sim n^{-3/4+1/2}$, so it still falls, but more slowly, like $n^{-1/4}$.
- $\beta \sim 1/\sqrt{n}$: $\beta\sqrt{n}$ is a constant, so it does not fall at all. The bound is vacuous, even though Equation 7.11 still says the *expected* gap is at most $1/\sqrt{n}$.

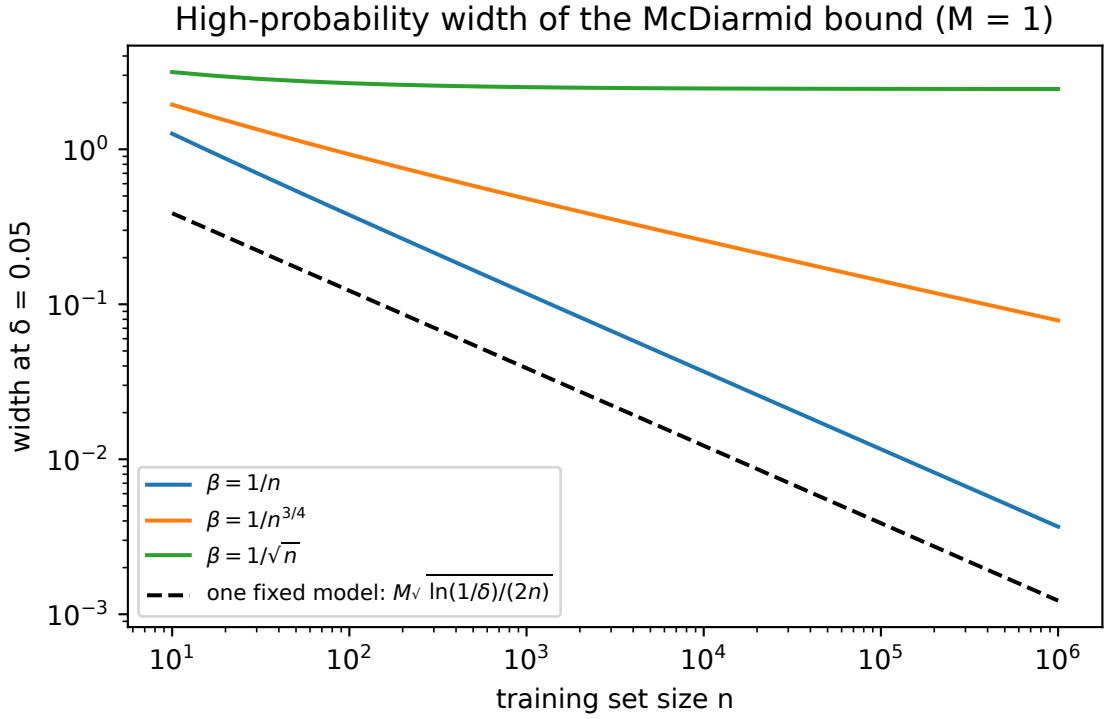


Figure 7.2.: Width of the high-probability bound, Equation 7.13, with $M = 1$ and $\delta = 0.05$, for stability $\beta = 1/n$, $n^{-3/4}$ and $n^{-1/2}$, against the Hoeffding width for a single fixed model.

The loss in step 3 comes from charging every one of the n examples the full 2β in McDiarmid, even though the effects of different examples largely average out. Later work removed most of this loss. Feldman & Vondrák (2018) proved a bound of order $\sqrt{(\beta + 1/n)\ln(1/\delta)}$, which is meaningful whenever $\beta = o(1)$, that is whenever β goes to zero at any rate. Feldman & Vondrák (2019) and then Bousquet, Klochkov & Zhivotovskiy (2020) reached, for $M = 1$ and up to constant factors

(which is what $\lesssim$ means),

$$\text{gap} \lesssim \beta \log n \, \log(1/\delta) + \sqrt{\frac{\log(1/\delta)}{n}}, \tag{7.15}$$

which is within log factors of the expected-gap bound plus the fixed-model rate. With these, $\beta \sim 1/\sqrt{n}$ also gives a $\tilde{O}(1/\sqrt{n})$ gap, where $\tilde{O}$ means "of this order up to log factors".

7.5.5. Example: regularized logistic regression

The next section shows that regularized logistic regression with $\|x\| \leq 1$ has $\beta = 2/(\lambda n)$ and weights of norm at most $1/\lambda$. To use Equation 7.13 you also need M, the largest loss. Its margin $y\langle w, x\rangle$ is at least $-\|w\|\|x\| \geq -1/\lambda$, and the logistic loss is largest at the most negative margin, so every loss is at most $M = \ln(1 + e^{1/\lambda})$. Take $n = 10{,}000$ and $\delta = 0.05$, so that $\ln(1/\delta) = \ln 20$ and the common factor is $\sqrt{\ln 20/20{,}000} = 0.01224$. Then the width beyond the training loss is $\beta + (2n\beta + M) \times 0.01224$, where $2n\beta = 4/\lambda$:

- $\lambda = 1$: $\beta = 2 \times 10^{-4}$, $M = \ln(1+e) = 1.313$, width $2 \times 10^{-4} + (4 + 1.313) \times 0.01224 = 0.065$.
- $\lambda = 0.1$: $\beta = 0.002$, $M = \ln(1 + e^{10}) = 10.000$, width $0.002 + (40 + 10) \times 0.01224 = 0.614$.

In both cases the stability term $2n\beta$ is three quarters or more of the width (4 against $M = 1.313$, and 40 against $M = 10.000$). The expected gap itself is far smaller: at $\lambda = 1$ it is already 2.53×10^{-4} at $n = 1{,}000$ (Table 7.4), and it falls like $1/n$. The high-probability bound is valid but loose, mostly because McDiarmid charges every example the worst case 2β.

7.5.6. Check yourself

- Why does step 2 pay M/n for the replaced example, not β/n? *(The replaced term is evaluated at two different points, z_i and z_i'. Stability compares two models at the same point, so it does not apply.)*
- If $\beta = 0$ (a constant algorithm), what does Equation 7.13 reduce to? *(Hoeffding for a single fixed model: $R \leq \hat{R} + M\sqrt{\ln(1/\delta)/(2n)}$.)*

7.6. Regularized ERM is stable

7.6.1. Context

Regularized ERM is the workhorse of classical machine learning: ridge regression, regularized logistic regression, support vector machines. The regularizer is usually justified as a prior or as a way to control model complexity. Stability gives a third reading: it limits how far one example can pull the solution. The penalty $\frac{\lambda}{2}\|w\|^2$ makes the objective a bowl with curvature at least λ, and one example, which carries weight $1/n$ in the objective, can only tilt such a bowl so far.

7.6.2. Statement

The setting is a convex loss whose gradient never exceeds L in norm. Let $\ell(\cdot, z)$ be convex, differentiable and L-Lipschitz for every z, and let w_S be the minimizer of the regularized training loss J_S:

$$w_S = \arg\min_w \; J_S(w), \qquad J_S(w) = \frac{1}{n}\sum_{i=1}^{n} \ell(w, z_i) + \frac{\lambda}{2}\|w\|^2. \tag{7.16}$$

Replacing one example changes J_S by a term of size about L/n in gradient, and the curvature λ converts that into a weight change of order $L/(\lambda n)$; Fact 1 then multiplies by L to get the loss change. Precisely,

$$\|w_S - w_{S^{(i)}}\| \le \frac{2L}{\lambda n} \quad \text{and} \quad \big|\ell(w_S, z) - \ell(w_{S^{(i)}}, z)\big| \le \frac{2L^2}{\lambda n} \text{ for all } z. \tag{7.17}$$

The first bound is the sensitivity of ERM, that is how far the minimizer can move when one example changes, which Optimization also derives. The second is uniform stability with $\beta = 2L^2/(\lambda n)$ (Shalev-Shwartz & Ben-David 2014, Corollary 13.6; Bousquet & Elisseeff 2002).

7.6.3. Derivation

Write $w = w_S$, $w' = w_{S^{(i)}}$, $J = J_S$, $J' = J_{S^{(i)}}$. The idea is to look at the gradient of J at the *other* minimizer w': it is small, because J and J' differ in only one term, and strong convexity says a small gradient means you are close to the minimizer.

1. **Optimality.** The gradient of a differentiable convex function is zero at its minimizer: $\nabla J(w) = 0$ and $\nabla J'(w') = 0$.
2. **The two objectives differ in one term.** Only the i-th loss differs, so $J - J' = \frac{1}{n}(\ell(\cdot, z_i) - \ell(\cdot, z_i'))$. Since $\nabla J'(w') = 0$, subtracting it changes nothing, and $\nabla J(w') = \nabla J(w') - \nabla J'(w') = \frac{1}{n}(\nabla \ell(w', z_i) - \nabla \ell(w', z_i'))$. Each gradient has norm at most L (Fact 1), so this has norm at most $2L/n$.
3. **Strong convexity.** J is a convex function plus $\frac{\lambda}{2}\|w\|^2$, so it is λ-strongly convex. Fact 3 says its gradient changes by at least $\lambda\|w' - w\|$ along the direction from w to w'. With $\nabla J(w) = 0$ from step 1,

$$\lambda\|w' - w\|^2 \le \langle \nabla J(w') - \nabla J(w),\, w' - w\rangle = \langle \nabla J(w'),\, w' - w\rangle \le \frac{2L}{n}\|w' - w\|, \tag{7.18}$$

   where the last step is the Cauchy–Schwarz inequality, $\langle a, b\rangle \le \|a\|\,\|b\|$, with the norm from step 2. Divide by $\lambda\|w' - w\|$ to get $\|w' - w\| \le 2L/(\lambda n)$.
4. **From weights to losses.** By Fact 1, a weight change of at most $2L/(\lambda n)$ changes every loss by at most L times that, $2L^2/(\lambda n)$.

A by-product is used elsewhere in this chapter. Setting the gradient of J_S to zero gives $\lambda w_S = -\frac{1}{n}\sum_i \nabla \ell(w_S, z_i)$. The right side is an average of vectors of norm at most L, so its norm is at most L, and $\|w_S\| \le L/\lambda$.

7.6.4. Interpretation

Since $\beta = 2L^2/(\lambda n)$ falls like $1/n$, which is $o(1/\sqrt{n})$, both the expected-gap and the high-probability bounds go to zero as the training set grows.

Regularization buys stability linearly: doubling λ halves β. It is not free, though. A larger λ also pulls the solution towards zero and away from the best fit, so the test loss as a function of λ is U-shaped (Figure 7.4, right): too little regularization overfits, too much underfits.

The bound is loose in three places, one per derivation step. Step 2 assumes the two gradients point in opposite directions with full norm L. Step 3 uses only the curvature λ of the regularizer and ignores the curvature of the data term, which also resists movement. Step 4 assumes the worst test point. In the experiment below, steps 2 and 4 each cost about a factor of 2, and step 3 costs more when λ is small and the data curvature matters.

7.6.5. Example: replace-one change in logistic regression

The running example uses unit-norm features, so $L = 1$ by Fact 1 and $\beta = 2/(\lambda n)$. The script fits the model on S, then on $S^{(i)}$ for 10 random i, where z_i is replaced by the same point with its label flipped. It records the largest weight change and the largest loss change. To find the test point where the loss changes most, it checks 2,000 random test points and the four points $\pm(w' - w)/\|w' - w\|$ with either label, which lie in the direction the weights moved, where the loss change is largest.

Table 7.3.: Replace-one changes of regularized logistic regression against the bounds in Equation 7.17. Ratio is the largest loss change divided by the loss bound.

n	λ	Weight bound $2L/(\lambda n)$	Largest weight change	Loss bound $2L^2/(\lambda n)$	Largest loss change	Ratio
100	0.01	2.0	0.406	2.0	0.340	0.17
100	0.1	0.2	0.0824	0.2	0.0474	0.24
100	1	0.02	0.00976	0.02	0.00505	0.25
1,000	0.1	0.02	0.00809	0.02	0.00499	0.25
10,000	0.01	0.02	0.00336	0.02	0.00285	0.14
10,000	1	2×10^{-4}	9.76×10^{-5}	2×10^{-4}	5.00×10^{-5}	0.25

Table 7.3 and Figure 7.3 show the $1/(\lambda n)$ scaling closely where the penalty dominates the curvature, that is for $\lambda \geq 0.1$: the rows with $n = 100, \lambda = 1$ and $n = 1{,}000, \lambda = 0.1$ share $\lambda n = 100$ and have nearly the same loss change (0.00505 and 0.00499). The third row with $\lambda n = 100$, at $n = 10{,}000$ and $\lambda = 0.01$, moves less (0.00336 in the weights, against 0.00976 and 0.00809), because there the data's own curvature adds to λ, as explained below. For $\lambda = 1$ the ratio is 0.25, and both factors of 2 can be read off.

The first factor of 2 comes from step 2. With margin $m = y\langle w, x\rangle$, the gradient for label y is $-y\,\sigma(-m)x$, and for the flipped label $-y$ the margin becomes $-m$ and the gradient is $y\,\sigma(m)x$. Their difference is $-y\,\sigma(-m)x - y\,\sigma(m)x = -yx$, because $\sigma(-m) + \sigma(m) = 1$. Its norm is $\|x\| = 1$,

not the worst case $2L = 2$. So the weights move by about $1/(\lambda n)$, half the bound: 0.00976 against 0.02.

The second factor of 2 comes from step 4. With $\lambda = 1$ the weights are small, so test margins are near zero. There the slope of the logistic loss is $\sigma(0) = \frac{1}{2}$, not $L = 1$, so the loss moves by about half the weight change: 0.00505 against 0.00976.

For $\lambda = 0.01$ the ratio drops to about 0.15 (0.17 and 0.14 in the table): with weak regularization the data term adds curvature beyond λ, which step 3 ignored.

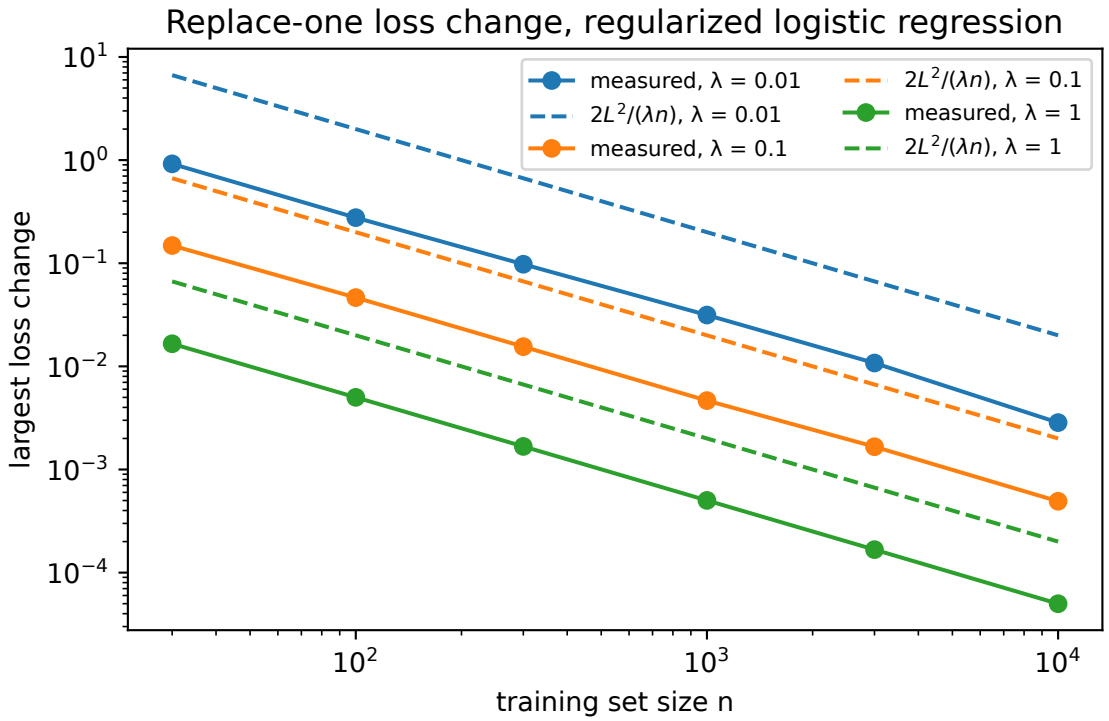


Figure 7.3.: Largest replace-one loss change of regularized logistic regression as a function of n, for three values of λ (solid), against the bound $2L^2/(\lambda n)$ (dashed). Both fall like $1/n$.

7.6.6. Example: the generalization gap against the bound

Equation 7.11 bounds the expected gap by the same β, here $2/(\lambda n)$. The script averages the gap over 100 training sets for each (n, λ). It computes the test loss exactly by numerical integration. To cancel most of the noise from the random training sets, it also subtracts, on the same training set, the gap of a fixed reference model. A fixed model's training loss is an unbiased estimate of its test loss (the first section of this chapter), so that gap has mean exactly zero and subtracting it leaves the expected value unchanged; but it rises and falls with the same lucky or unlucky training sets, so the subtraction removes much of the variation.

Table 7.4.: Expected generalization gap of regularized logistic regression, averaged over 100 training sets, against the stability bound.

λ	n	Train loss	Test loss	Expected gap	Bound $2/(\lambda n)$	Gap / bound
0.01	50	0.4603	0.5781	0.119	4.0	0.030
0.01	1,000	0.5311	0.5381	0.00596	0.2	0.030
0.1	50	0.6073	0.6401	0.0319	0.4	0.080
0.1	200	0.6288	0.6373	0.00830	0.1	0.083
0.1	1,000	0.6346	0.6365	0.00178	0.02	0.089
1	50	0.6806	0.6854	0.00459	0.04	0.115
1	1,000	0.6852	0.6855	0.000253	0.002	0.127

The gap falls like $1/n$, as the bound predicts, and stays a fixed fraction of it for each λ: about 3% for $\lambda = 0.01$ and 11% to 13% for $\lambda = 1$ (Table 7.4). Part of the slack is the factor of about 4 already seen in Table 7.3, where the ratio was 0.25. The rest comes from averaging: the gap averages over test points and replacements, while the bound takes the worst of both.

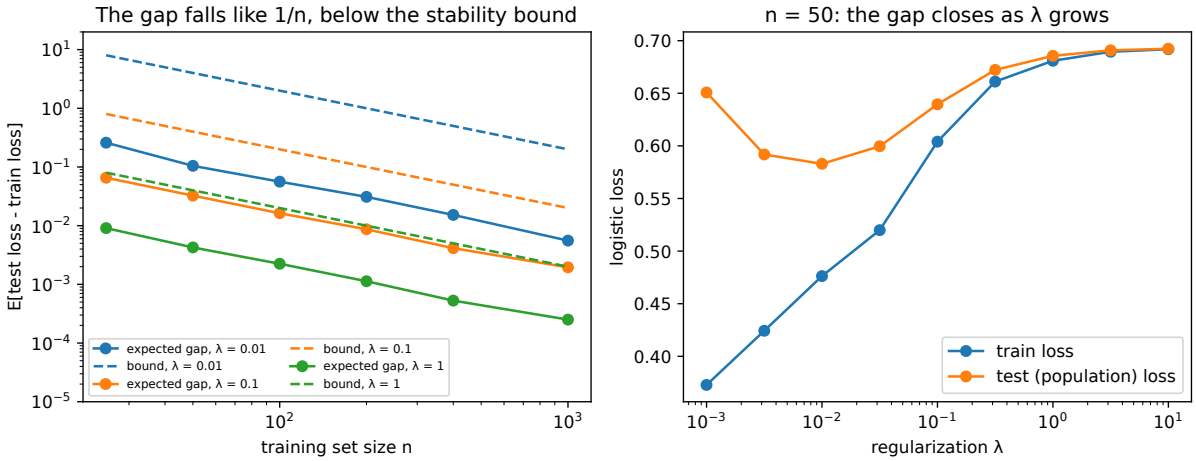


Figure 7.4.: Left: expected generalization gap of regularized logistic regression against n (solid), with the stability bound $2/(\lambda n)$ (dashed). Right: train and test loss against λ at $n = 50$. Stronger regularization closes the gap, but too much of it raises the test loss.

7.6.7. Check yourself

- Where does the derivation use convexity of ℓ, and where Lipschitzness? *(Convexity makes J_S λ-strongly convex in step 3. Lipschitzness bounds the gradients in step 2 and turns weights into losses in step 4.)*
- Squared loss on bounded data is not globally Lipschitz. How can you still apply the result to ridge regression? *(Use $\|w_S\| \leq$ a bound derived from optimality, as in the by-product, and the Lipschitz constant of the squared loss on that bounded set.)*

7.7. SGD is stable if you stop early

7.7.1. Context

Large models are not trained to an exact minimizer. They are trained by SGD for some number of steps: at each step, pick one training example at random and move the weights a little against the gradient of its loss. Hardt, Recht & Singer (2016) showed that SGD itself is stable, with a β that grows with the total step size. This turns "train for fewer epochs" into a generalization guarantee.

7.7.2. Statement

The idea is that two SGD runs, one on S and one on $S^{(i)}$, can only drift apart on the steps that happen to pick example i. Each such step pushes them apart by at most $2\eta_t L$, and it happens with probability $1/n$; every other step, by Fact 4, does not push them apart. So the expected distance grows by at most $2\eta_t L/n$ per step.

The assumptions are those of Fact 4 plus a gradient bound. Let $\ell(\cdot, z)$ be convex, ρ-smooth (gradient changes by at most ρ per unit distance) and L-Lipschitz (gradient norm at most L) for every z. Run SGD for T steps from a starting point w_0: step $t = 1, \dots, T$ updates $w_t = w_{t-1} - \eta_t \nabla \ell(w_{t-1}, z_{I_t})$, and the output is w_T. Each index I_t is drawn uniformly from $\{1, \dots, n\}$ and $\eta_t \le 2/\rho$. Then SGD is uniformly stable (in expectation over the indices) with

$$\beta \le \frac{2L^2}{n} \sum_{t=1}^{T} \eta_t. \tag{7.19}$$

This is Theorem 3.8 of Hardt, Recht & Singer (2016), who write β for the smoothness and ϵ_{stab} for the stability.

7.7.3. Derivation

Run SGD on S and on $S^{(i)}$ with the **same** index sequence $I_1, I_2, \dots$ and the same starting point. This pairing, called a **coupling**, is legitimate because the stability condition for a randomized algorithm compares the *average* loss of each run, and each run's average depends only on its own distribution. Using the same random indices for both runs does not change either run's distribution, and it makes the two runs as similar as possible. Let $\delta_t = \|w_t - w_t'\|$ be the distance between the two runs after t steps, with $\delta_0 = 0$ because both start at w_0. Step t uses the index I_t and the step size η_t and turns δ_{t-1} into δ_t. (Only here, δ_t is a distance, not a failure probability.)

1. **Same example picked** ($I_t \neq i$, probability $1 - 1/n$). Both runs take a gradient step on the same function $\ell(\cdot, z_{I_t})$. By Fact 4 (a gradient step with $\eta_t \le 2/\rho$ on a convex smooth function does not increase distances), $\delta_t \le \delta_{t-1}$.
2. **Differing example picked** ($I_t = i$, probability $1/n$). One run steps on z_i, the other on z_i', so Fact 4 does not apply. Instead, write $w_t - w_t' = (w_{t-1} - w_{t-1}') - \eta_t \nabla \ell(w_{t-1}, z_i) + \eta_t \nabla \ell(w_{t-1}', z_i')$ and use the triangle inequality on the three terms: $\delta_t \le \|w_{t-1} - w_{t-1}'\| + \eta_t \|\nabla \ell(w_{t-1}, z_i)\| + \eta_t \|\nabla \ell(w_{t-1}', z_i')\| \le \delta_{t-1} + 2\eta_t L$, since each gradient has norm at most L.

3. **Take expectations** over I_t. The index I_t is drawn fresh at step t, so it is independent of δ_{t-1}, which depends only on earlier indices. Weighting the two cases by their probabilities, $\mathbb{E}\delta_t \le (1-\frac{1}{n})\mathbb{E}\delta_{t-1} + \frac{1}{n}(\mathbb{E}\delta_{t-1} + 2\eta_t L) = \mathbb{E}\delta_{t-1} + \frac{2\eta_t L}{n}$.
4. **Unroll.** Starting from $\delta_0 = 0$ and adding the increments of steps $t = 1, \dots, T$, $\mathbb{E}\delta_T \le \frac{2L}{n}\sum_{t=1}^{T}\eta_t$.
5. **From weights to losses.** By Fact 1, at every test point z the loss difference is at most $L\delta_T$, so its expectation is at most $L\,\mathbb{E}\delta_T$: $\left|\mathbb{E}[\ell(w_T, z) - \ell(w'_T, z)]\right| \le L\,\mathbb{E}\delta_T$. Multiplying the bound of step 4 by L gives Equation 7.19.

7.7.4. Interpretation

The derivation says that the two runs only drift apart when the differing example is used. Figure 7.5 shows this as a staircase: each single run jumps when index i is drawn and otherwise does not grow. Since i is drawn with probability $1/n$ and an epoch is n steps, a jump happens on average once per epoch.

As a result, stability degrades linearly with training time. With a constant step η and E epochs, so $T = En$ steps, the sum of the step sizes is ηEn, and Equation 7.19 gives $\beta \le (2L^2/n)\eta En = 2L^2\eta E$. The n cancels: for a fixed number of epochs the bound does not improve with more data. One pass ($E = 1$) gives $\beta \le 2L^2\eta$.

This makes early stopping a trade-off. Training longer lowers the optimization error but raises β, and the excess test loss pays for both. To see the balance, split the excess test loss of the SGD output $\bar{w}$ over the best weights w^* into three pieces: $R(\bar{w}) - R(w^*) = [R(\bar{w}) - \hat{R}_S(\bar{w})] + [\hat{R}_S(\bar{w}) - \hat{R}_S(w^*)] + [\hat{R}_S(w^*) - R(w^*)]$. The first piece is the gap, at most β on average. The second is the optimization error. The third has mean zero, because w^* is a fixed model. For convex L-Lipschitz losses, SGD with constant step η reaches an optimization error of at most $\frac{\|w^*\|^2}{2\eta T} + \frac{\eta L^2}{2}$ for the averaged iterate (see Optimization; Shalev-Shwartz & Ben-David 2014, ch. 14). The first term falls with training time and the gap bound $2L^2\eta T/n$ rises with it. With $u = \eta T$, the sum to balance is $\frac{\|w^*\|^2}{2u} + \frac{2L^2 u}{n}$. Setting its derivative $-\frac{\|w^*\|^2}{2u^2} + \frac{2L^2}{n}$ to zero gives $u^* = \frac{\|w^*\|\sqrt{n}}{2L}$, and at that value both terms equal $\frac{L\|w^*\|}{\sqrt{n}}$, so the excess test loss is at most $\frac{2L\|w^*\|}{\sqrt{n}}$, plus the $\eta L^2/2$ term. Training much longer than u^* buys optimization accuracy that the growing stability term gives back.

The bound is loose because step 1 only uses that a step does not expand distances. In practice a gradient step on a curved loss often contracts them, so the true divergence saturates instead of growing linearly (Figure 7.5).

7.7.5. Example: unregularized logistic regression with SGD

Take $n = 100$ examples of the running example and a constant step $\eta = 0.25$. To apply the theorem you need the smoothness ρ. Along the margin m, the logistic loss $\ln(1 + e^{-m})$ has second derivative $\sigma(m)\sigma(-m) = \sigma(m)(1 - \sigma(m))$, a product of two numbers that sum to 1, which is at most $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$. With $\|x\| \le 1$ the curvature in weight space is no larger, so the loss is $\frac{1}{4}$-smooth, $\rho = \frac{1}{4}$, and the condition $\eta \le 2/\rho = 8$ holds. The script averages over 100 coupled pairs of runs, with z_1 replaced by a fresh draw. With $L = 1$ and $\eta = 0.25$, the loss bound $2L^2\eta E$ and the distance bound $2L\eta E$ are both $0.5E$:

Table 7.5.: SGD on unregularized logistic regression, $n = 100$, $\eta = 0.25$, averaged over 100 runs. With $L = 1$ the two bounds coincide.

Epochs	Train loss	Test loss	Gap	Bound $2L^2\eta E$	Mean $\|w_T - w'_T\|$	Bound $2L\eta E$
1	0.559	0.596	0.036	0.5	0.130	0.5
2	0.515	0.567	0.052	1.0	0.189	1.0
5	0.478	0.557	0.079	2.5	0.277	2.5
10	0.470	0.567	0.097	5.0	0.363	5.0
20	0.467	0.582	0.115	10.0	0.355	10.0
40	0.467	0.588	0.121	20.0	0.395	20.0

The qualitative prediction holds (Table 7.5, Figure 7.6): the gap grows with the number of epochs, and the test loss is lowest after about 5 epochs and rises afterwards, while the training loss keeps falling. Quantitatively the bound is loose, by a factor of 14 after one epoch (0.5/0.036) and of 165 after 40 (20.0/0.121), because the divergence saturates near 0.4 instead of growing linearly.

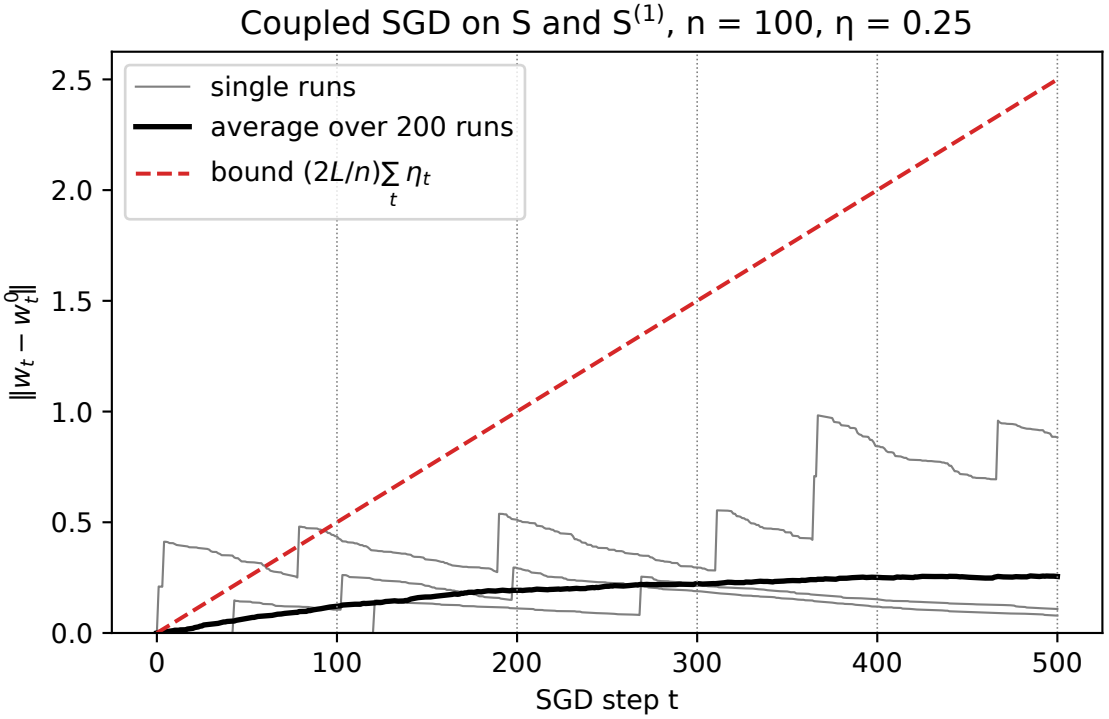


Figure 7.5.: Distance between coupled SGD runs on S and $S^{(1)}$ ($n = 100$, $\eta = 0.25$), for three single runs and the average over 200 runs, against the bound $(2L/n)\sum_t \eta_t$. Single runs jump when example 1 is drawn and contract otherwise. Dotted lines mark epochs.

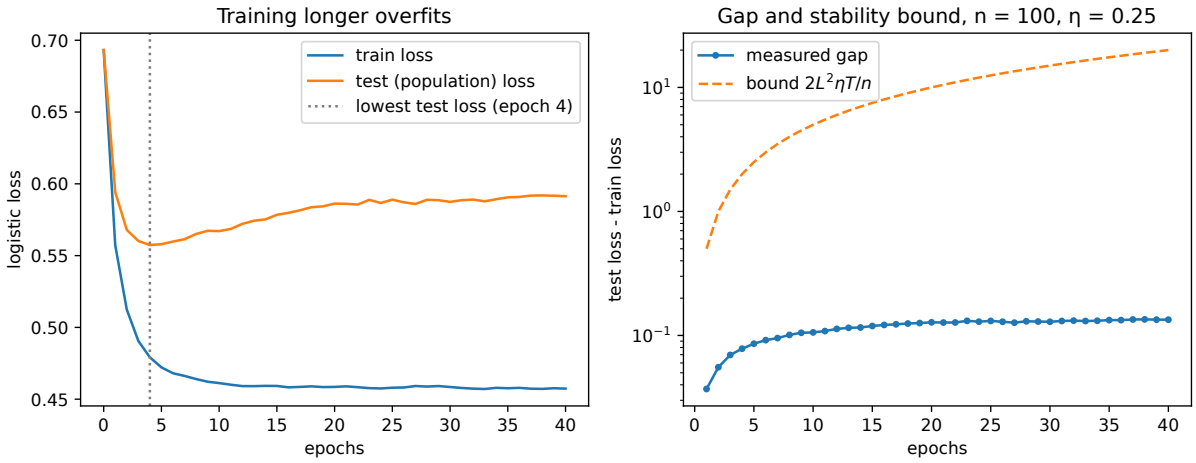


Figure 7.6.: Left: train and test loss of SGD against the number of epochs. Right: the measured gap against the stability bound $2L^2\eta T/n$, on a log scale. Both grow with training time; the bound grows faster.

7.7.6. Check yourself

- Why must both runs use the same index sequence? *(Otherwise they would step on different examples at almost every step, and step 1 could not use non-expansiveness. The bound concerns the distribution of each run, so you may couple them as you like.)*
- What breaks for non-convex losses? *(Fact 4: a gradient step on a non-convex function can expand distances, by up to a factor $1 + \eta_t\rho$. Hardt, Recht & Singer still get a bound, but it grows polynomially in T rather than linearly in $\sum_t \eta_t$.)*

7.8. An unstable algorithm: 1-nearest neighbour

7.8.1. Context

Stability is sufficient for generalization, in the sense of a vanishing gap. Seeing an algorithm without it makes clear what it rules out. The nearest neighbour (NN) rule with one neighbour, 1-NN, predicts the label of the closest training point. It fits every training example perfectly, using that example itself, which is exactly the dependence on single examples that stability forbids.

7.8.2. Statement

1-NN is as unstable as an algorithm can be, and its gap does not close. With 0-1 loss, 1-NN is β-uniformly stable only for $\beta = 1$, the largest possible value. Its training error is 0, and as $n \to \infty$ its test error tends to $\mathbb{E}[2p(x)(1 - p(x))]$, where $p(x) = \Pr[y = 1 \mid x]$ is the probability that the label at x is $+1$ (Cover & Hart 1967). So whenever the labels are noisy, its gap stays bounded away from zero.

7.8.3. Derivation

1. **Training error is 0.** Each training point is its own nearest neighbour (assume distinct points), so it is predicted with its own label.
2. $\beta = 1$**.** It is enough to find one training set, one replacement and one test point where the loss changes by 1. Take any S and the test point $z = z_i = (x_i, y_i)$. On S the prediction at x_i is y_i: loss 0. Replace z_i by $z_i' = (x_i, -y_i)$, the same point with the opposite label. On $S^{(i)}$ the nearest neighbour of x_i is z_i', so the prediction is $-y_i$: loss 1.
3. **The gap equals the test error.** By step 1 the training loss is 0, so $\text{gap}(S) = R(A(S))$. As n grows, the training points fill the space, so the nearest neighbour of a test point x lies at (essentially) x, and its label is an independent draw that is $+1$ with probability $p(x)$. The test label is another independent draw with the same probability. The two disagree when one is $+1$ and the other -1, which happens with probability $p(1-p) + (1-p)p = 2p(1-p)$. Averaging over x gives the limit.

7.8.4. Interpretation

Zero training error says nothing here. 1-NN is a reasonable classifier: the best possible classifier, the **Bayes classifier**, predicts the more likely label at each x and has error $\mathbb{E}[\min(p, 1-p)]$, and since $2p(1-p) = 2\min(p, 1-p)\max(p, 1-p) \leq 2\min(p, 1-p)$, the limiting error of 1-NN is at most twice that. But its training error carries no information about how close it gets.

The result is consistent with the theorem but not explained by it. The gap of 1-NN is below $\beta = 1$, so Equation 7.11 holds; it just says nothing useful when the loss is at most 1 anyway.

More neighbours give more stability. k-NN with majority vote averages k labels, so replacing one example rarely changes a prediction. Its worst-case β is still 1, because some training set leaves a test point with a vote split as evenly as possible, and replacing one of its neighbours flips the majority, but average-case stability notions capture its behaviour (Devroye & Wagner 1979; Exercise 6 in the exercise book).

7.8.5. Example: 1-NN against regularized logistic regression

In the running example the Bayes error is 0.250 and the 1-NN limit $\mathbb{E}[2p(1-p)]$ is 0.338. The table compares 1-NN with the stable regularized logistic regression, averaging over 10 training sets, with 0-1 error measured on 2,000 test points:

Table 7.6.: 0-1 error of 1-nearest neighbour and of regularized logistic regression.

n	1-NN train	1-NN test	Logistic ($\lambda = 0.01$) train	Logistic test
50	0.000	0.404	0.170	0.301
200	0.000	0.380	0.232	0.274
1,000	0.000	0.367	0.244	0.266
4,000	0.000	0.352	0.249	0.259

The gap of 1-NN is its test error, which falls towards 0.338 and no further. The gap of the stable logistic regression, test minus train, falls from $0.301 - 0.170 = 0.131$ at $n = 50$ to $0.259 - 0.249 = 0.010$ at $n = 4{,}000$ (Table 7.6, Figure 7.7).

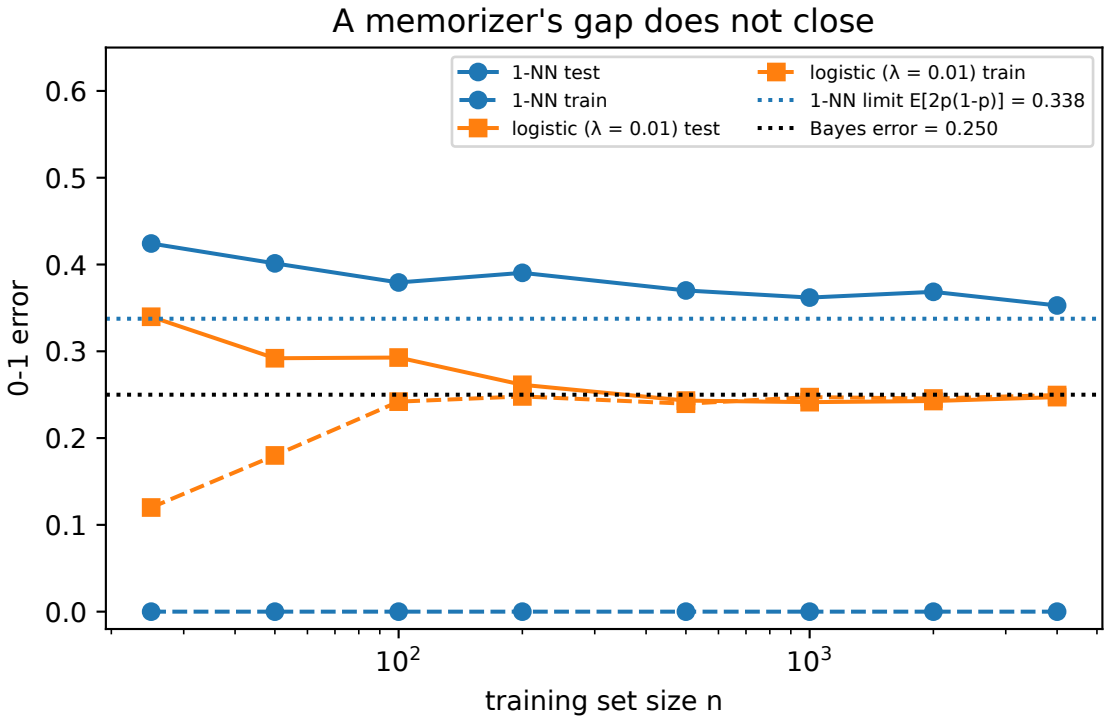


Figure 7.7.: Train (dashed) and test (solid) 0-1 error of 1-nearest neighbour and of regularized logistic regression against n, with the Bayes error and the 1-NN limit $\mathbb{E}[2p(1-p)]$.

7.8.6. Check yourself

- Is the memorizing lookup table less stable than 1-NN? *(No; both have $\beta = 1$. But 1-NN generalizes its memory to nearby points, so its test error is far better.)*
- Would label noise of zero ($p(x) \in \{0, 1\}$) rescue 1-NN's gap? *(In the limit, yes: $2p(1-p) = 0$. The gap of a memorizer is driven by examples whose label could have gone either way.)*

7.9. Overfitting is memorization of individual examples

7.9.1. Context

Equation 7.10 says the expected gap is the average, over training examples, of how much each example's own loss rises when that example is replaced. This quantity, the **self-influence** (or **memorization score**) of example i, can be measured directly by retraining. Measuring it shows *which* examples carry the gap.

7.9.2. Statement

Give the bracket of Equation 7.10 a name: for each training example, its self-influence is its loss under the model trained without it (with a fresh point in its place) minus its loss under the model trained with it,

$$\mathrm{mem}_i(S) = \ell(A(S^{(i)}), z_i) - \ell(A(S), z_i). \tag{7.20}$$

Then $\mathbb{E}[\mathrm{gap}(S)] = \frac{1}{n}\sum_i \mathbb{E}[\mathrm{mem}_i(S)]$, by Equation 7.10. Feldman (2020) uses the same idea with removal instead of replacement and accuracy instead of loss: the probability that the model is right on example i when trained with it, minus the same probability when trained without it, $\Pr[A(S) \text{ is right on } x_i] - \Pr[A(S \setminus z_i) \text{ is right on } x_i]$.

7.9.3. Derivation

Nothing new: this is Equation 7.10 with the bracket given a name. The content is in the measurement.

7.9.4. Interpretation

The gap is a sum of per-example contributions. A model with a large gap has, on average, examples whose fit depends on themselves. Uniform stability caps every contribution at β; the identity says the average is what matters for the expected gap.

The contributions are uneven. Typical examples are fitted well whether or not they are present, because their neighbours carry the same information. Atypical examples, such as mislabeled points or members of rare subgroups, are fitted only if they themselves are in the training set, so they carry most of the gap.

That does not make memorization a defect to remove at any cost, because it can be necessary. Feldman (2020) shows that when the data distribution has a long tail of rare subpopulations, each seen about once, an algorithm that does not memorize those singletons loses accuracy on the tail. So for natural data, a large gap can come with good test accuracy. The empirical evidence points the same way. Zhang et al. (2017) found that deep networks fit even randomly relabeled training data perfectly, and Feldman & Zhang (2020) estimated memorization scores on image benchmarks and found the highest scores on atypical and mislabeled examples.

7.9.5. Example: self-influence in logistic regression

The script draws training sets of $n = 40$ points, fits the model, then for each i replaces z_i by a fresh draw, refits, and records mem_i. Over 100 training sets, with $\pm$ one standard error, the measured gap and the average self-influence are close. At $\lambda = 0.01$ the gap is 0.1482 ± 0.0073 and the average self-influence is 0.1422 ± 0.0025. At $\lambda = 0.1$ the gap is 0.0420 ± 0.0017 and the average self-influence is 0.0416 ± 0.0003.

The two agree within their errors, as the identity requires. The self-influence estimate is less noisy, because it averages n refits per training set.

To see which examples carry the gap, split them by whether the label goes with the trend ($y_i x_{i1} \geq 0$, the more likely label, since $\Pr[y = 1 \mid x] = \sigma(5x_1)$ favours the sign of x_1) or against it. About a quarter go against it (24% and 25% in the two runs). At $\lambda = 0.01$, their average self-influence is 0.284, against 0.097 for the others: three times larger. At $\lambda = 0.1$ the two are 0.055 and 0.037. Figure 7.8 shows the full picture: the least typical examples are the most memorized, and regularization flattens the profile.

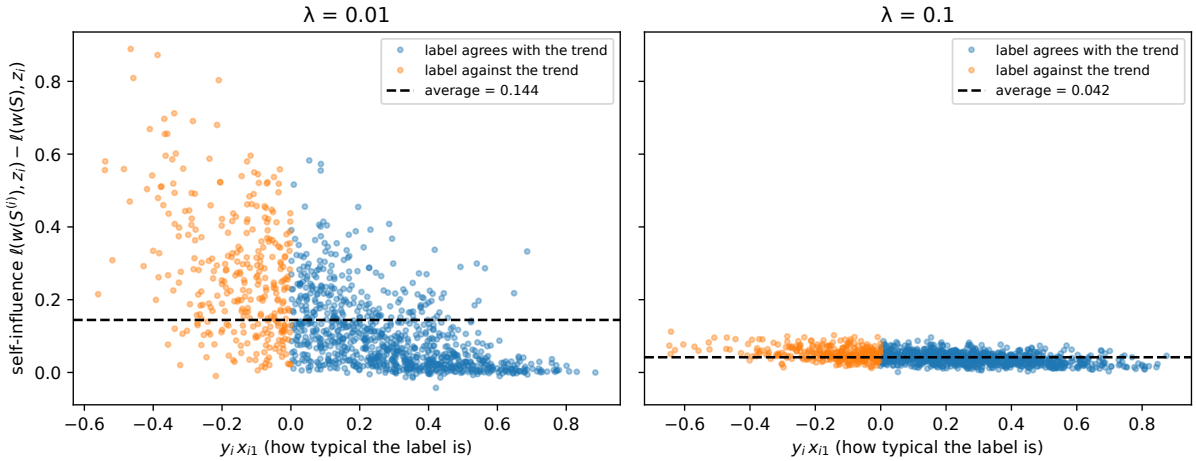


Figure 7.8.: Self-influence $\ell(w(S^{(i)}), z_i) - \ell(w(S), z_i)$ of each training example against $y_i x_{i1}$, which measures how typical its label is, for $n = 40$ and two values of λ. Examples whose label goes against the trend have the largest self-influence. The dashed line is the average, which equals the expected gap.

7.9.6. Check yourself

- Uniform stability bounds every mem_i. Why is it a stronger requirement than a small expected gap? *(A small gap only needs the average to be small. A few heavily memorized examples are compatible with a small gap but not with small uniform stability.)*
- Why does regularization shrink the self-influence of atypical examples the most? *(It caps how far any one example can pull the weights, $2L/(\lambda n)$, and atypical examples are the ones that pull hardest.)*

7.10. Code: check every bound by simulation

The script code/stability.py draws every figure in this chapter and computes the numbers quoted in it. The running example has features x uniform on the unit sphere in $\mathbb{R}^{10}$ and labels with $\Pr[y = 1 \mid x] = \sigma(5x_1)$. The model is logistic regression without intercept, fitted by Newton's method with backtracking. Because $\|x\| = 1$, the logistic loss is 1-Lipschitz and $\frac{1}{4}$-smooth. The test loss of a model is computed without sampling error. The features are spread evenly over the sphere and the label depends only on x_1, so rotating the other nine coordinates changes nothing; the test loss therefore depends only on w_1 and the norm of the other coordinates of w. That makes it a two-dimensional integral, done on a grid. From the repository root:

```
uv run python code/part01-prerequisites/stability.py
```

Output (seed 0), summarized in Table 7.7:

Table 7.7.: Summary of the script's output.

Experiment	Result
Renaming, $n = 1$, $p = 0.3$ (Table 7.2)	Train 0.0750; test 0.2850; renamed 0.2850; gap 0.2100; $\beta = 0.75$
Replace-one change, regularized logistic (Table 7.3)	Loss change between 0.14 and 0.25 of $2L^2/(\lambda n)$, for n from 100 to 10,000 and λ from 0.01 to 1
Expected gap, regularized logistic (Table 7.4)	Falls like $1/n$; about 3% of the bound at $\lambda = 0.01$, 11% to 13% at $\lambda = 1$
High-probability width, $n = 10{,}000$, $\delta = 0.05$	$\lambda = 1$: 0.065; $\lambda = 0.1$: 0.614
Self-influence, $n = 40$	$\lambda = 0.01$: gap 0.1482 ± 0.0073, self-influence 0.1422 ± 0.0025. $\lambda = 0.1$: 0.0420 ± 0.0017 and 0.0416 ± 0.0003
SGD, $n = 100$, $\eta = 0.25$ (Table 7.5)	Gap 0.036 after 1 epoch, 0.121 after 40; bound 0.5 and 20.0
1-NN against logistic (Table 7.6)	1-NN test error 0.404 $\to$ 0.352 with train error 0; limit 0.338; Bayes 0.250

tests/test_stability.py checks that the output still matches the numbers in this chapter.

What to take from it. The $1/(\lambda n)$ scaling holds, but the constant is loose: replace-one changes and gaps follow $1/(\lambda n)$ closely when $\lambda \geq 0.1$ (with weaker regularization the data's own curvature makes them smaller still), yet sit between 4 and 36 times below the bound, because the bound takes the worst case at every step. The identity, unlike the bounds, is exact: the average self-influence matches the measured gap within simulation error. The SGD bound gets the direction right and the size wrong: the gap grows with training time, but the divergence between coupled runs saturates while the bound grows linearly. Finally, memorizers do not generalize their training error: 1-NN has training error 0 and test error 0.35, while regularized logistic regression has train and test errors that meet.

7.11. References

- Bousquet & Elisseeff, "Stability and Generalization" (JMLR 2002): defines uniform and weaker stability notions; Theorem 12 is the high-probability bound
- Shalev-Shwartz & Ben-David, *Understanding Machine Learning* (2014), ch. 13: the clearest textbook treatment; Theorem 13.2 is Equation 7.10 and Corollary 13.6 the regularized-ERM bound. Chapter 14 covers SGD convergence.
- Hardt, Recht & Singer, "Train Faster, Generalize Better: Stability of Stochastic Gradient Descent" (ICML 2016): SGD stability for convex (Theorem 3.8), strongly convex and non-convex losses

- Feldman & Vondrák, "Generalization Bounds for Uniformly Stable Algorithms" (NeurIPS 2018) and "High Probability Generalization Bounds for Uniformly Stable Algorithms with Nearly Optimal Rate" (COLT 2019): the improved high-probability bounds
- Bousquet, Klochkov & Zhivotovskiy, "Sharper Bounds for Uniformly Stable Algorithms" (COLT 2020): removes a log factor from the 2019 bound
- Shalev-Shwartz, Shamir, Srebro & Sridharan, "Learnability, Stability and Uniform Convergence" (JMLR 2010): stability as the characterization of learnability
- Cover & Hart, "Nearest Neighbor Pattern Classification" (IEEE Trans. Information Theory 1967): the 1-NN limit
- Devroye & Wagner, "Distribution-Free Performance Bounds for Potential Function Rules" (IEEE Trans. Information Theory 1979): stability arguments for k-NN and local rules
- Nesterov, *Introductory Lectures on Convex Optimization* (2004), §2.1: co-coercivity and smooth convex functions (Fact 4)
- Feldman, "Does Learning Require Memorization? A Short Tale about a Long Tail" (STOC 2020): why memorizing rare examples can be necessary for accuracy
- Feldman & Zhang, "What Neural Networks Memorize and Why" (NeurIPS 2020): memorization scores measured on image benchmarks
- Zhang, Bengio, Hardt, Recht & Vinyals, "Understanding Deep Learning Requires Rethinking Generalization" (ICLR 2017): networks fit random labels

7.12. Cheat sheet

Table 7.8 collects every result. $S^{(i)}$ replaces example i by a fresh draw; β is uniform stability, the largest change in loss at any test point when one example is replaced; δ is the failure probability.

Table 7.8.: Every result of the chapter.

Result	Assumptions	Statement
Gap identity	None	$\mathbb{E}[\text{gap}] = \frac{1}{n}\sum_i \mathbb{E}[\ell(A(S^{(i)}), z_i) - \ell(A(S), z_i)]$
Expected gap	β-uniformly stable	$\lvert\mathbb{E}[\text{gap}]\rvert \le \beta$
High probability (Bousquet–Elisseeff)	β-uniformly stable, $0 \le \ell \le M$	$\text{gap} \le \beta + (2n\beta + M)\sqrt{\ln(1/\delta)/(2n)}$
Improved high probability (Feldman–Vondrák; Bousquet–Klochkov–Zhivotovskiy)	Same, $M = 1$	$\text{gap} \lesssim \beta \log n \log(1/\delta) + \sqrt{\log(1/\delta)/n}$
Regularized ERM	Convex, L-Lipschitz loss, $\frac{\lambda}{2}\lVert w\rVert^2$	$\lVert\Delta w\rVert \le 2L/(\lambda n)$, $\beta = 2L^2/(\lambda n)$
SGD (Hardt–Recht–Singer)	Convex, ρ-smooth, L-Lipschitz, $\eta_t \le 2/\rho$	$\beta \le (2L^2/n)\sum_t \eta_t$
1-NN, lookup table	0-1 loss	$\beta = 1$; training error 0

Rules of thumb

- A gap bound from stability needs $\beta \to 0$; a high-probability bound from McDiarmid needs $\beta = o(1/\sqrt{n})$.
- Regularization strength and training time are the two stability knobs: $\beta \propto 1/\lambda$ for ERM, $\beta \propto \eta T$ for SGD.
- Training error 0 is a warning, not a result: it means the model may depend entirely on individual examples.
- The examples a model memorizes most are the atypical ones.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

8. Cryptography basics

8.1. Why this matters

Some computations must hide something: a key, a message, a value split among several holders. To reason about them you need a precise answer to one question: **what does it mean that an observer with limited computing power learns nothing it was not supposed to learn?** Cryptography answers it with a small set of definitions and proof techniques. This chapter teaches them as mathematics: definitions you can state, bounds you can derive, and toy constructions you can run.

The whole subject is one idea, used over and over. **Security is a game.** A challenger secretly picks one of two worlds, runs a procedure in that world, and shows the result to an adversary, who guesses which world it is in. The challenger picks the world by flipping a fair coin, so that an adversary who learns nothing from what it sees is right exactly half the time, whatever it answers. Its **advantage** measures how much better than that coin flip it does. A scheme is then called **secure** when the advantage is tiny for every efficient adversary, and "tiny" is made precise in one of two ways: by negligible functions, which shrink faster than any power of the key length (the asymptotic view), or by explicit numbers such as 2^{-64} (the concrete view). Finally, security proofs rarely jump from the real world to the ideal one in a single move. They **change one thing at a time**, through a chain of intermediate worlds called **hybrids**, and add up the advantage gained at each step.

Table 8.1 lists where each result is used.

Table 8.1.: The results in this chapter and where each is typically used.

Result	Typical use
Negligible functions, polynomial-time adversaries	Saying "this attack essentially never works" in a way that survives polynomially many repetitions
Indistinguishability and the hybrid argument	The skeleton of every security proof
Concrete security and the birthday bound	How much data a key may protect; how long random identifiers and hashes must be
Pseudorandom generators and functions	Stretching a short random key into many random-looking values
One-time pad, indistinguishability under chosen-plaintext attack (CPA)	What encryption promises, and why it must be randomized
The simulation paradigm	Proving that what a participant sees reveals nothing beyond what it is allowed to learn
Learning with errors and Regev encryption	Lattice-based encryption that can add ciphertexts, at the price of growing noise

Later parts build protocols from these primitives. The probability tools used here (union bound, $1 + x \le e^x$, the normal tail) are proved in Concentration inequalities, and total variation (TV) distance is studied in Divergences; each is restated below where it is used.

Notation. This chapter departs from the part's shared notation in two letters. It keeps P and Q for two distributions, but lowercase q counts the queries an adversary makes (or the random draws in the birthday bound), and lowercase p is the modulus of the arithmetic in secret sharing and learning with errors, not a density. The bit length of blocks, messages and inputs is n, the number of share holders is h, and the dimension of a learning-with-errors secret is d.

A warning about the code. Every scheme in this chapter's script is a toy, written to make the definitions concrete. Its parameters are far too small to be secure, nothing runs in constant time, and NumPy's generator is not a cryptographic source of randomness. Never use it to protect anything.

8.2. Toolkit: five facts every derivation uses

8.2.1. Fact 1 — Advantage and guessing probability are the same thing

A **distinguisher** A is any procedure that receives one sample and outputs a bit, 0 or 1. Think of output 1 as "I think this sample came from Y". If A is any good at telling distribution X from distribution Y, it says 1 more often on samples from Y than on samples from X, so the natural measure of its skill is the gap between those two frequencies. That gap is its **advantage**:

$$\mathrm{Adv}_A(X, Y) = \big|\Pr[A(X) = 1] - \Pr[A(Y) = 1]\big|. \tag{8.1}$$

Here $\Pr[A(X) = 1]$ is the probability that A outputs 1 when its input is drawn from X. A distinguisher that ignores its input says 1 equally often on both and has advantage 0; one that is always right has advantage 1.

Games are often phrased differently, as a guessing contest. The challenger flips a fair bit b: if $b = 1$ it gives A a sample from Y, and if $b = 0$ a sample from X. A outputs a guess, and it **wins** if its guess equals b. So $\Pr[A \text{ wins}]$, also written $\Pr[b' = b]$ with b' the guess, is the probability that the guess is correct; it is not the probability that b is 1, which is $\frac{1}{2}$ by construction. The coin is fair on purpose. With a fair coin, an adversary who learns nothing from its sample, and so has nothing to go on, wins exactly half the time, whether it guesses at random or always says 1. Any win rate above $\frac{1}{2}$ must therefore come from information in the sample.

The two phrasings measure the same thing. There are two ways to win, and they cannot happen in the same game: the coin says Y and A says 1, or the coin says X and A says 0. Each world occurs with probability $\frac{1}{2}$, so the win probability is the average of the two conditional win rates. Writing $\Pr[A(X) = 0] = 1 - \Pr[A(X) = 1]$ and collecting terms,

$$\Pr[A \text{ wins}] = \tfrac{1}{2}\Pr[A(Y) = 1] + \tfrac{1}{2}\Pr[A(X) = 0] = \tfrac{1}{2} + \tfrac{1}{2}\big(\Pr[A(Y) = 1] - \Pr[A(X) = 1]\big). \tag{8.2}$$

So an advantage of ε means winning with probability $\frac{1}{2} + \frac{\varepsilon}{2}$: the know-nothing score, plus half the advantage. The bracket has no absolute value, so it can be negative, but then A wins less than half the time, and flipping its output (saying 0 where it said 1) turns it into a distinguisher that wins $\frac{1}{2} + \frac{\varepsilon}{2}$.

Here it is with a coin. X is a fair coin, Y lands heads with probability 0.6, and A outputs 1 on heads. Each row of Table 8.2 is one outcome of the game. Its probability is $\frac{1}{2}$ for the challenger's bit times the probability of the coin outcome in that world: $\frac{1}{2} \times 0.5 = 0.25$ for each outcome of the fair coin, $\frac{1}{2} \times 0.4 = 0.20$ for tails of the biased coin and $\frac{1}{2} \times 0.6 = 0.30$ for heads.

Table 8.2.: The distinguishing game for a fair against a 0.6-biased coin, with the distinguisher "output 1 on heads".

b	Coin	Outcome	Probability	A outputs	A wins?
0	fair	tails	0.25	0	yes
0	fair	heads	0.25	1	no
1	biased	tails	0.20	0	no
1	biased	heads	0.30	1	yes

A wins in the first and last rows, with probability $0.25 + 0.30 = 0.55$. Its advantage is $\Pr[A(Y) = 1] - \Pr[A(X) = 1] = 0.6 - 0.5 = 0.1$, and indeed $0.55 = \frac{1}{2} + \frac{0.1}{2}$.

8.2.2. Fact 2 — No distinguisher beats total variation

The **total variation distance** between two distributions P and Q on a finite set is the total amount of probability that P puts on outcomes in excess of Q:

$$\mathrm{TV}(P, Q) = \sum_{\omega:\, P(\omega) > Q(\omega)} (P(\omega) - Q(\omega)). \tag{8.3}$$

It is 0 when the distributions are equal and 1 when they never produce the same outcome. Every distinguisher, however much time it uses, has advantage at most $\mathrm{TV}(P, Q)$. The reason is that a distinguisher can only react to which outcome it sees, and the most it can do is say 1 on exactly the outcomes that P favours.

1. **Describe any distinguisher by its reaction to each outcome.** A randomized distinguisher outputs 1 on outcome ω with some probability $g(\omega) \in [0, 1]$. Nothing else about it matters for its advantage.
2. **Write the gap as a sum over outcomes.** The probability of output 1 under P is $\sum_\omega P(\omega)g(\omega)$, and likewise under Q, so $\Pr[A(P) = 1] - \Pr[A(Q) = 1] = \sum_\omega g(\omega)(P(\omega) - Q(\omega))$.
3. **Maximize term by term.** Each term is largest when $g(\omega) = 1$ where $P(\omega) - Q(\omega)$ is positive and $g(\omega) = 0$ where it is negative. That choice gives exactly the TV distance. The same argument with P and Q swapped bounds the gap of the other sign, and the excess of Q over P is the same total, because both distributions sum to 1.

So the best unbounded distinguisher attains the TV distance, and this splits security into three regimes. Two distributions with TV distance 0 are identical, and then no adversary has any advantage: this is **perfect** security. TV distance at most a small ε is **statistical** security: even an adversary with unlimited time gains at most ε. Cryptography's main business is the third case, where the TV distance is large but no *efficient* distinguisher can exploit it.

8.2.3. Fact 3 — Advantages add along a chain

If you walk from distribution H_0 to distribution H_m through intermediate distributions $H_1, \dots, H_{m-1}$, a distinguisher cannot tell the two ends apart better than the sum of how well it tells each consecutive pair apart. In symbols, for any distinguisher A,

$$\mathrm{Adv}_A(H_0, H_m) \le \sum_{i=1}^{m} \mathrm{Adv}_A(H_{i-1}, H_i). \tag{8.4}$$

The reason is a telescoping sum: in $\sum_{i=1}^{m} (\Pr[A(H_{i-1}) = 1] - \Pr[A(H_i) = 1])$, each intermediate term appears once with a plus sign and once with a minus sign and cancels, leaving $\Pr[A(H_0) = 1] - \Pr[A(H_m) = 1]$. Taking absolute values, the triangle inequality ($|a_1 + \cdots + a_m| \le |a_1| + \cdots + |a_m|$) gives the bound. This is the arithmetic behind every hybrid proof.

8.2.4. Fact 4 — A bijection maps uniform to uniform

If you shuffle the elements of a finite set by a fixed one-to-one rule, a uniformly random element stays uniformly random. Formally, if U is uniform on a finite set S and $f : S \to S$ is a bijection, then each $t \in S$ comes from exactly one u, namely $f^{-1}(t)$, so $\Pr[f(U) = t] = \Pr[U = f^{-1}(t)] = 1/|S|$. The one-time pad, secret sharing and the mask in learning with errors are all this fact, with $f(u) = u + c$ for a fixed c: adding a uniform random value to anything fixed gives a uniform result.

8.2.5. Fact 5 — Union bound and $1 - x \le e^{-x}$

Both are proved in Concentration inequalities; here is what they say and why they hold. The **union bound** says that the probability that at least one of several events happens is at most the sum of their probabilities, $\Pr[A_1 \cup \cdots \cup A_q] \le \sum_i \Pr[A_i]$: the sum counts every outcome where some event happens at least once, and outcomes where several happen more than once. The inequality $1 - x \le e^{-x}$ holds for every real x because the straight line $1 - x$ is the tangent to the convex curve e^{-x} at $x = 0$, and a convex curve lies above its tangents. For $x_i \in [0, 1]$, multiplying the nonnegative inequalities $1 - x_i \le e^{-x_i}$ gives $\prod_i (1 - x_i) \le e^{-\sum_i x_i}$.

In this chapter they bound the probability that something goes wrong in at least one of q steps: the union bound from above, and the product inequality from below (applied to the probability that nothing goes wrong).

8.3. Security parameter, adversaries and negligible functions

8.3.1. Context

A key of 40 bits can be found by trying every key; a key of 256 bits cannot. Security is therefore never absolute. It is a statement about how the attacker's success shrinks as you pay more, measured by a **security parameter** λ (think of the key length in bits). The asymptotic theory needs two notions: which attackers count as efficient, and which success probabilities count as tiny.

8.3.2. Definitions

An attacker counts as efficient if its running time grows at most polynomially with λ. Precisely, an adversary is **probabilistic polynomial time (PPT)** if it is a randomized algorithm whose running time is bounded by a polynomial in λ. Running time is always measured against the length of the input, and λ written in binary is only about $\log_2 \lambda$ bits long, so the adversary is given λ in unary, as the string of λ ones written 1^λ. Then "polynomial in its input length" means polynomial in λ.

A success probability counts as tiny if it shrinks faster than one over any polynomial. The point of this choice is that an efficient adversary can repeat its attack a polynomial number of times, so "tiny" must stay tiny after multiplication by any polynomial. A function $\mu : \mathbb{N} \to [0, 1]$ is **negligible** if, for every exponent c, it eventually falls below λ^{-c} and stays there:

$$\forall c > 0 \;\; \exists \lambda_0 \;\; \forall \lambda \geq \lambda_0 : \quad \mu(\lambda) < \lambda^{-c}. \tag{8.5}$$

Putting the two together, a scheme is **secure** if every PPT adversary has negligible advantage in its security game.

8.3.3. Closure: polynomially many tries keep it negligible

Statement. If μ is negligible and poly is a polynomial, then $\text{poly}(\lambda)\mu(\lambda)$ is negligible. The sum of two negligible functions is negligible.

Derivation. The idea is that a negligible function beats every inverse polynomial, so you can ask it to beat one that is steep enough to also cancel the polynomial poly.

1. **Bound the polynomial by a power.** A polynomial of degree $c' - 1$ is eventually below $\lambda^{c'}$, because $\lambda^{c'}$ grows faster than any of its terms. So $\text{poly}(\lambda) \leq \lambda^{c'}$ for all $\lambda \geq \lambda_1$.
2. **Ask μ for a steeper decay.** Fix any $c > 0$. Apply Equation 8.5 to μ with exponent $c + c'$ instead of c: $\mu(\lambda) < \lambda^{-(c+c')}$ for $\lambda \geq \lambda_2$.
3. **Multiply the two bounds.** For $\lambda \geq \max(\lambda_1, \lambda_2)$, $\text{poly}(\lambda)\mu(\lambda) < \lambda^{c'}\lambda^{-(c+c')} = \lambda^{-c}$. Since c was arbitrary, $\text{poly} \cdot \mu$ meets the definition of negligible.
4. **Handle a sum the same way.** Apply Equation 8.5 to each term with exponent $c + 1$: $\mu_1 + \mu_2 < 2\lambda^{-(c+1)}$, and $2\lambda^{-(c+1)} \leq \lambda^{-c}$ is the same as $2 \leq \lambda$, which holds once $\lambda \geq 2$.

This is why the definitions use polynomials and negligible functions together. A PPT adversary can repeat an attack a polynomial number of times. By the union bound (Fact 5: the chance that at least one try succeeds is at most the sum of the per-try chances), its overall success is at most the number of tries times the per-try success, which is still negligible.

8.3.4. Examples

Each example compares $\mu(\lambda)$ with λ^{-c}, usually by taking $\log_2$ of both sides, which turns the comparison of tiny numbers into a comparison of exponents.

1. $2^{-\lambda}$ **is negligible.** $\lambda^c 2^{-\lambda} = 2^{c \log_2 \lambda - \lambda}$, and the exponent goes to $-\infty$ for every c because λ grows faster than $c \log_2 \lambda$. So $\lambda^c 2^{-\lambda} \to 0$: 2^λ outgrows every polynomial.

2. $\lambda^{-\log_2 \lambda}$ **is negligible**, although it decays more slowly than any exponential. For fixed c, compare the exponents of λ: $\lambda^{-\log_2 \lambda} < \lambda^{-c}$ exactly when $\log_2 \lambda > c$, that is, for all $\lambda > 2^c$.
3. $2^{-\sqrt{\lambda}}$ **is negligible.** Taking $\log_2$, $2^{-\sqrt{\lambda}} < \lambda^{-c}$ exactly when $\sqrt{\lambda} > c \log_2 \lambda$. Since $\sqrt{\lambda}/\log_2 \lambda \to \infty$, this holds for all large λ.
4. λ^{-10} **is not negligible.** The definition must hold for every c, so one bad c is enough. Take $c = 11$: $\lambda^{-10} < \lambda^{-11}$ never holds for $\lambda \geq 1$.
5. **A function that is** $2^{-\lambda}$ **for even** λ **and** $1/\lambda$ **for odd** λ **is not negligible.** The definition needs the bound for *all* large λ, and the odd values violate it for $c = 2$, since $1/\lambda > \lambda^{-2}$ for $\lambda > 1$.

8.3.5. Interpretation: asymptotics say nothing at a fixed λ

"Negligible" is a statement about the limit. At a specific parameter it can be misleading. Table 8.3 evaluates the four examples at $\lambda = 128$. Since $128 = 2^7$, $\log_2 \lambda = 7$, so $\lambda^{-\log_2 \lambda} = (2^7)^{-7} = 2^{-49}$ and $\lambda^{-10} = (2^7)^{-10} = 2^{-70}$; and $\sqrt{128} = 11.31$.

Table 8.3.: Four functions at $\lambda = 128$. The negligible $2^{-\sqrt{\lambda}}$ is far larger than the non-negligible λ^{-10}.

$\mu(\lambda)$	Negligible?	Value at $\lambda = 128$
$2^{-\lambda}$	yes	$2^{-128} = 2.9 \times 10^{-39}$
$\lambda^{-\log_2 \lambda}$	yes	$2^{-49} = 1.8 \times 10^{-15}$
$2^{-\sqrt{\lambda}}$	yes	$2^{-11.31} = 3.9 \times 10^{-4}$
λ^{-10}	no	$2^{-70} = 8.5 \times 10^{-22}$

The non-negligible λ^{-10} is smaller than two of the negligible functions. The crossing points follow from comparing exponents, as in the examples. $2^{-\sqrt{\lambda}}$ only drops below λ^{-10} for good at $\lambda = 20{,}520$, where $\sqrt{\lambda}$ overtakes $10 \log_2 \lambda$. $\lambda^{-\log_2 \lambda}$ crosses it at $\lambda = 2^{10} = 1024$, where $\log_2 \lambda = 10$. Figure 8.1 plots the "bits of security" $-\log_2 \mu(\lambda)$ of each, the number of halvings it takes to get from 1 down to $\mu(\lambda)$. A polynomial λ^{-c} has $c \log_2 \lambda$ bits of security. The negligible functions eventually rise above every such curve, but they can take a long time to do so.

8.3.6. Example: guessing a key

Suppose the key is uniform over all 2^λ strings of λ bits. An adversary that tries t distinct keys finds the right one with probability $t/2^\lambda$, because each key it tries is the right one with probability $2^{-\lambda}$ and the tries cover disjoint possibilities. If $t = t(\lambda)$ is a polynomial, this is a polynomial times $2^{-\lambda}$, which is negligible by the closure property. Brute force is the attack every definition must allow, and it is why the best one can hope for is "negligible", never zero.

8.3.7. Check yourself

- Is $\mu(\lambda) = 2^{-(\log_2 \lambda)^2}$ negligible? *(Yes. Write* $2^{(\log_2 \lambda)^2} = (2^{\log_2 \lambda})^{\log_2 \lambda} = \lambda^{\log_2 \lambda}$*, so* $\mu(\lambda) = \lambda^{-\log_2 \lambda}$*, example 2.)*

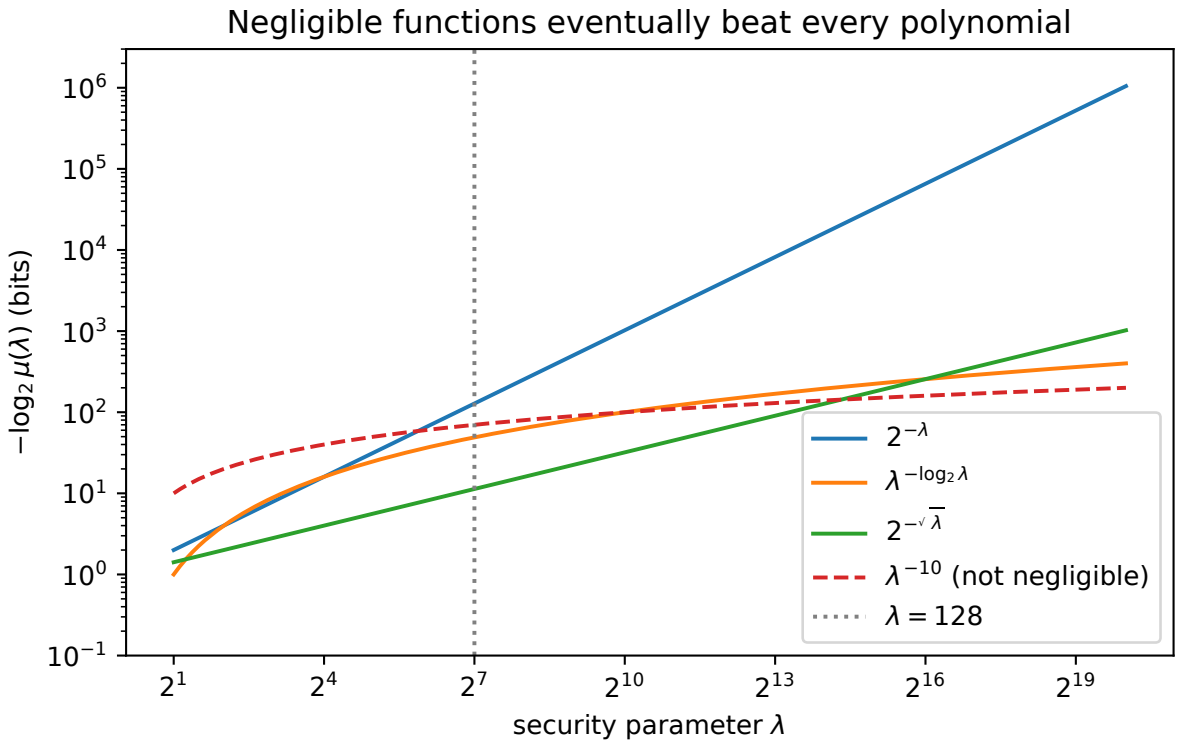


Figure 8.1.: Bits of security $-\log_2 \mu(\lambda)$ for three negligible functions and the non-negligible λ^{-10}, on log–log axes. Each negligible curve eventually overtakes the dashed one; $2^{-\sqrt{\lambda}}$ does so only beyond $\lambda = 20{,}520$.

- If an attack succeeds with probability $1/\lambda^3$ per try, does running it λ^4 times break the scheme? *(Yes, almost surely. The tries are independent, so* $\Pr[\textit{all tries fail}] = (1-\lambda^{-3})^{\lambda^4} \leq e^{-\lambda^{-3}\cdot\lambda^4} = e^{-\lambda}$ *by* $1-x \leq e^{-x}$ *(Fact 5). A non-negligible success probability can be amplified to near 1.)*

8.4. Computational indistinguishability and the hybrid argument

8.4.1. Context

Most security definitions say "these two things look the same". A ciphertext looks like random bits; the output of a generator looks like true randomness; a participant's view looks like something it could have produced alone. Computational indistinguishability makes "look the same" precise, and the hybrid argument is the tool for proving it.

You already know distinguishers from machine learning. A classifier trained to tell real images from generated ones is a distinguisher. If it outputs 1 for "real", then $\Pr[A(\text{real}) = 1]$ is its true-positive rate and $\Pr[A(\text{generated}) = 1]$ is its false-positive rate, so its advantage is its true-positive rate minus its false-positive rate at the chosen threshold. Deciding which of two distributions produced a sample is a hypothesis test, and Hypothesis testing studies the best such tests.

8.4.2. Definition

Because security is about how things behave as λ grows, the objects compared are not single distributions but families, one distribution for each value of λ. Two families $X = \{X_\lambda\}$ and $Y = \{Y_\lambda\}$ are **computationally indistinguishable**, written $X \approx_c Y$, if no efficient distinguisher has more than a negligible advantage: for every PPT distinguisher A

$$\mathrm{Adv}_A(X_\lambda, Y_\lambda) = \left|\Pr[A(1^\lambda, X_\lambda) = 1] - \Pr[A(1^\lambda, Y_\lambda) = 1]\right| \text{ is negligible in } \lambda. \tag{8.6}$$

If the TV distance itself is negligible, so that even unbounded distinguishers gain almost nothing, they are **statistically indistinguishable** ($\approx_s$), and if they are equal, **perfectly indistinguishable** ($\equiv$). By Fact 2, every distinguisher's advantage is at most the TV distance, so $\equiv$ implies $\approx_s$, which implies $\approx_c$.

8.4.3. The hybrid lemma for many samples

Statement. If one sample cannot be told apart, can many? The answer is yes, as long as "many" is polynomial. Let X and Y be efficiently samplable (an efficient algorithm can draw from each) with $X \approx_c Y$, and let $m = m(\lambda)$ be a polynomial. Then m independent copies are still indistinguishable:

$$(X_1, \ldots, X_m) \approx_c (Y_1, \ldots, Y_m). \tag{8.7}$$

Quantitatively, a distinguisher with advantage ε on m copies yields one with advantage ε/m on a single copy, with about the same running time.

Derivation. The idea is to walk from all-X to all-Y by swapping one sample at a time. If the two ends are ε apart, the m steps must add up to ε (Fact 3), so a typical step is worth ε/m. And a single step changes a single sample, so it is a one-sample problem.

1. **Define the hybrids.** Let $H_i = (Y_1, \dots, Y_i, X_{i+1}, \dots, X_m)$: the first i entries come from Y, the rest from X. Then H_0 is all-X and H_m is all-Y, and consecutive hybrids differ in a single position.
2. **Build a one-sample distinguisher.** Given a distinguisher D for m samples, let D' on a single input z pick a position i uniformly from $\{1, \dots, m\}$, sample $Y_1, \dots, Y_{i-1}$ and $X_{i+1}, \dots, X_m$ itself (this is where efficient samplability is needed), put z in position i, and output $D(Y_1, \dots, Y_{i-1}, z, X_{i+1}, \dots, X_m)$.
3. **Identify what D sees.** If $z \sim X$, the tuple handed to D has $i-1$ entries from Y followed by entries from X, so it is distributed as H_{i-1}. If $z \sim Y$, it has i entries from Y, so it is H_i.
4. **Average over the random position.** Each i is chosen with probability $\frac{1}{m}$, so $\Pr[D'(X) = 1] - \Pr[D'(Y) = 1] = \frac{1}{m} \sum_{i=1}^{m} \left(\Pr[D(H_{i-1}) = 1] - \Pr[D(H_i) = 1] \right)$.
5. **Telescope.** As in Fact 3, but without absolute values, the intermediate terms cancel and the sum equals $\Pr[D(H_0) = 1] - \Pr[D(H_m) = 1]$, whose absolute value is D's advantage ε. So D' has advantage exactly ε/m.
6. **Conclude by contradiction.** Suppose ε were not negligible. Negating Equation 8.5, there would be a c with $\varepsilon(\lambda) \geq \lambda^{-c}$ for infinitely many λ. With $m(\lambda) \leq \lambda^{c'}$ for some exponent c', this gives $\varepsilon/m \geq \lambda^{-(c+c')}$ for those same λ, so the efficient D' would have non-negligible advantage on one sample, contradicting $X \approx_c Y$.

The same argument without the running-time bookkeeping shows $\mathrm{TV}(P^{\otimes m}, Q^{\otimes m}) \leq m \, \mathrm{TV}(P, Q)$, where $P^{\otimes m}$ is the distribution of m independent samples from P.

8.4.4. Interpretation

Every hybrid proof has this shape: a chain from the real world to an ideal world, each step justified by one assumption, and a loss equal to the number of steps. The loss factor m is usually pessimistic, because changing one sample at a time is how the proof is organized, not how the best attacker works. An attacker who sees all m samples at once can pool them, and the example below shows that pooling gains far less than a factor m.

8.4.5. Example: how many flips reveal a biased coin?

X is a fair coin and Y lands heads with probability $\frac{1}{2} + \delta$, with $\delta = 0.01$. From one flip, the best advantage is the TV distance, which is the excess probability of heads, $\delta = 0.01$ (Fact 2). With m flips, the hybrid bound gives at most $m\delta$. Table 8.4 and Figure 8.2 compare that with the best possible advantage (the exact TV distance between the two binomial counts of heads) and with a simple distinguisher, majority vote: output 1 iff more than half the flips are heads.

Table 8.4.: Advantage in telling m flips of a 0.51-coin from m fair flips (seed 0, 200,000 simulated games per entry).

Flips m	Majority vote (simulation)	Best possible (TV)	Hybrid bound $\min(1, m\delta)$
1	0.008	0.010	0.010
100	0.082	0.080	1.000
10,000	0.481	0.683	1.000

The hybrid bound is tight for one sample and loose afterwards, and a normal approximation shows why. The number of heads has mean $m/2$ for the fair coin and $m/2 + m\delta$ for the biased one, and in both cases a standard deviation of about $\sqrt{m}/2$ (each flip has variance $\frac{1}{4}$ or very close to it). So the two counts are two bell curves whose centres are $m\delta/(\sqrt{m}/2) = 2\delta\sqrt{m}$ standard deviations apart. The best test puts its threshold where the two curves cross, halfway between the means, at $\delta\sqrt{m}$ standard deviations from each. Write Φ for the standard normal cumulative distribution function, $\Phi(z) = \Pr[Z \le z]$ for $Z \sim \mathcal{N}(0,1)$. The biased count lands above the threshold with probability $\Phi(\delta\sqrt{m})$ and the fair one with probability $1 - \Phi(\delta\sqrt{m})$, so the best advantage is about $2\Phi(\delta\sqrt{m}) - 1$. At $m = 100$, $\delta\sqrt{m} = 0.1$ and $2\Phi(0.1) - 1 = 0.080$; at $m = 10{,}000$, $\delta\sqrt{m} = 1$ and $2\Phi(1) - 1 = 0.683$, the two TV values in the table. The true advantage therefore grows like $\delta\sqrt{m}$, not δm: the means of the counts separate by $m\delta$, but each count fluctuates by about $\sqrt{m}/2$. To reach a constant advantage you need $\delta\sqrt{m}$ of order 1, that is about $1/\delta^2$ samples, while the hybrid bound would allow it after $1/\delta$.

Majority vote stalls at about $\frac{1}{2}$, because its threshold $m/2$ sits at the centre of the fair curve instead of halfway between the two. For large m the biased count is almost surely above $m/2$, but so is the fair count about half the time. At $m = 10{,}000$ the biased mean is $2\delta\sqrt{m} = 2$ standard deviations above $m/2$, so the biased count clears $m/2$ with probability about $\Phi(2) = 0.977$, and $0.977 - 0.5 \approx 0.48$, close to the simulated 0.481. The best test puts the threshold halfway between the two means, at $m/2 + m\delta/2$ (Exercise 3 in the exercise book).

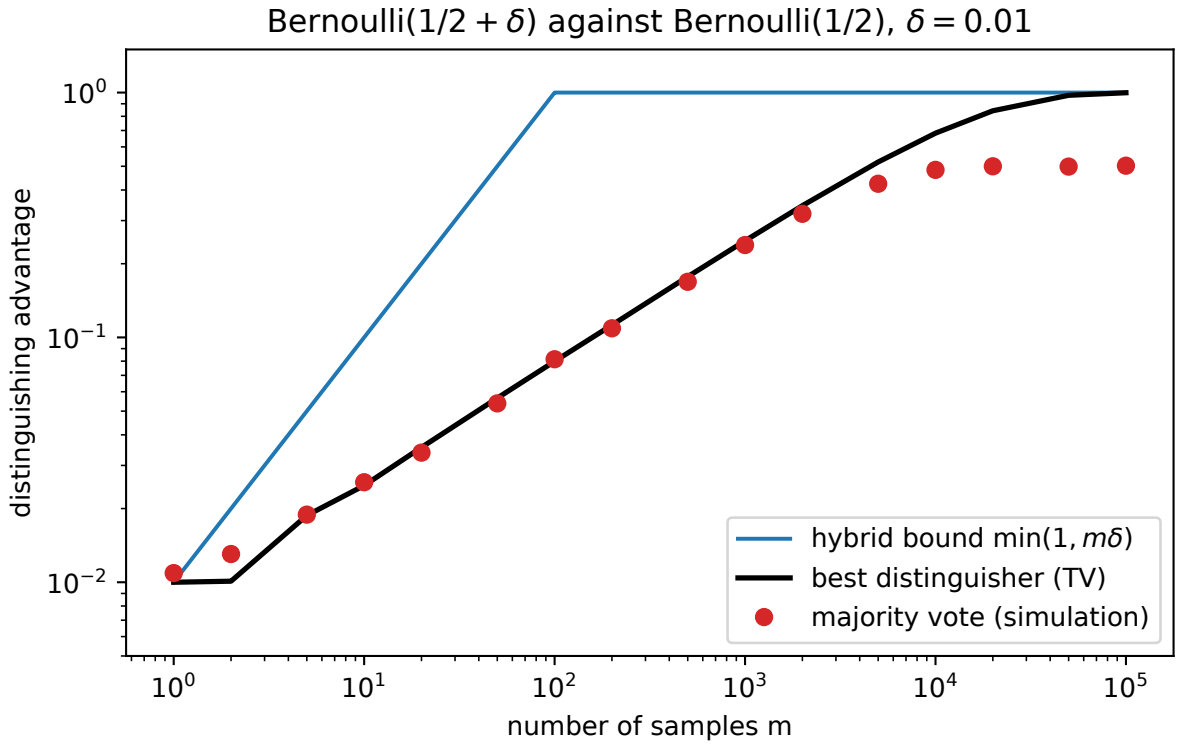


Figure 8.2.: Advantage of telling m flips of a coin with bias $\frac{1}{2} + 0.01$ from fair flips: majority vote (simulation), the best possible distinguisher (exact TV distance), and the hybrid bound $m\delta$.

8.4.6. Check yourself

- Where does the hybrid lemma use that X and Y are efficiently samplable? *(In step 2: D' must generate the other $m-1$ entries itself, in polynomial time.)*
- Why does the lemma need m to be polynomial? *(The loss is a factor m. For superpolynomial m, ε/m can be negligible even when ε is not, so step 6 no longer gives a contradiction.)*

8.5. Concrete versus asymptotic security

8.5.1. Context

Real systems fix λ: AES uses 128-bit keys, not "a key of length λ". A statement that holds "for all large enough λ" does not say whether 128 is large enough. Concrete security replaces "PPT" and "negligible" by explicit numbers.

8.5.2. Definition

Instead of "efficient" and "negligible", concrete security names the attacker's resources and the advantage they buy. A primitive is (t, q, ε)**-secure** if every adversary that runs in time at most t and makes at most q queries to its oracles (the procedures the game lets it call, such as "encrypt this message for me") has advantage at most ε. Brute force over κ-bit keys sets the benchmark: trying t keys succeeds with probability $t/2^\kappa$. A primitive has **κ-bit security** if, roughly, no attack does better than that: every adversary with time t has advantage at most $t/2^\kappa$.

Security proofs then become **reductions** with explicit losses: "if A breaks scheme S with (t, q, ε), then B breaks primitive P with (t', q', ε')". Read in reverse, this says that if P is hard to break, so is S, and the losses say how much security is lost on the way. The hybrid lemma is an example with $t' \approx t$ and $\varepsilon' = \varepsilon/m$. A reduction is **tight** when $\varepsilon' \approx \varepsilon$. A loose reduction forces larger parameters for the same guarantee, because the primitive must then be secure against advantages much smaller than the one you want for the scheme.

8.5.3. Example: what 128-bit security buys

Suppose an attacker has an exascale machine that tests 10^{18} keys per second, and runs it for a year. A year is about 3.16×10^7 seconds, so that is about 3.16×10^{25} keys, which is $2^{84.7}$ keys (since $\log_2 10 \approx 3.32$). Each key is right with probability 2^{-128}, so against a 128-bit key it succeeds with probability $2^{84.7-128} = 9.3 \times 10^{-14}$. Trying all 2^{128} keys would take $2^{128}/2^{84.7} = 2^{43.3}$ years, which is 1.08×10^{13} years. The advantage is not zero, but it is smaller than the probability of most hardware failures during the computation.

8.5.4. Example: what a q-query bound means

The next section derives that a pseudorandom permutation on n-bit blocks is also a pseudorandom function, up to an extra advantage of $q^2/2^{n+1}$ after q queries (Equation 8.10). In words, the extra advantage is the probability that q random n-bit values contain a repeat, which grows with the square of q. For a 128-bit block cipher, $q^2/2^{129} = 2^{2k-129}$ when $q = 2^k$. With 16-byte (128-bit) blocks:

- $q = 2^{32}$ blocks (2^{36} bytes, 64 GiB of data): at most $2^{64-129} = 2^{-65}$.
- $q = 2^{48}$ blocks (2^{52} bytes, 4 PiB): at most $2^{96-129} = 2^{-33}$.
- $q = 2^{64}$ blocks: the bound reaches $2^{128-129} = \frac{1}{2}$ and says nothing.

So a q-query bound is a data limit. This is why block-cipher modes recommend changing keys long before 2^{64} blocks: the guarantee degrades with the square of the data processed under one key.

8.5.5. Check yourself

- A reduction loses a factor of 2^{20} (it turns advantage ε into $\varepsilon/2^{20}$). If the primitive has 128-bit security, what does the scheme have? *(About $128 - 20 = 108$ bits: a scheme adversary with advantage ε gives a primitive adversary with advantage $\varepsilon/2^{20} \le t/2^{128}$, so $\varepsilon \le t/2^{108}$.)*
- Why is "(t, ε)-secure" usually summarized by the ratio t/ε? *(An adversary can trade time for success probability, for example by guessing a fraction of the key space: trying t of 2^κ keys gives $\varepsilon = t/2^\kappa$, so the ratio $t/\varepsilon = 2^\kappa$ stays constant along that trade-off.)*

8.6. Pseudorandom generators and functions

8.6.1. Context

Truly random bits are expensive to produce and to share. A short random key is cheap. Pseudorandom generators stretch a short key into a long string, and pseudorandom functions turn it into a huge table of random-looking values that can be looked up on demand. Both are defined by games.

8.6.2. Pseudorandom generators

A generator is pseudorandom if its output, on a random seed, cannot be told from truly random bits by any efficient distinguisher. Write U_k for the uniform distribution on $\{0,1\}^k$, the strings of k bits. A deterministic, efficient $G : \{0,1\}^\lambda \to \{0,1\}^{\ell(\lambda)}$ with $\ell(\lambda) > \lambda$, so that the output is longer than the seed, is a **pseudorandom generator (PRG)** if

$$G(U_\lambda) \approx_c U_{\ell(\lambda)}. \tag{8.8}$$

As a game: the challenger sends either $G(s)$ for a uniform seed s, or a uniform ℓ-bit string, and the adversary guesses which.

A PRG is never statistically secure. The outputs of G form a tiny subset of all ℓ-bit strings, so an adversary that can check membership in that subset wins easily. The catch is that checking takes exponential time. Let $D(y) = 1$ iff $y = G(s)$ for some seed s.

1. On $G(U_\lambda)$, D always outputs 1, because the input is an output of G by construction.
2. There are 2^λ seeds, so at most 2^λ of the 2^ℓ strings are outputs of G. On U_ℓ, D therefore outputs 1 with probability at most $2^\lambda/2^\ell = 2^{\lambda-\ell}$, which is at most $\frac{1}{2}$ because $\ell \geq \lambda + 1$.
3. So D has advantage at least $1 - \frac{1}{2} = \frac{1}{2}$, and by Fact 2 the TV distance is at least $\frac{1}{2}$. But D loops over 2^λ seeds. Pseudorandomness is only a statement about efficient distinguishers.

PRGs exist if one-way functions exist (Håstad, Impagliazzo, Levin & Luby 1999). In practice, stream ciphers such as ChaCha20 are modelled as PRGs.

8.6.3. Pseudorandom functions

A **random function** from $\mathcal{X}$ to $\mathcal{Y}$ is a table that assigns each input its own independent, uniform output. A keyed function $F : \{0,1\}^\lambda \times \mathcal{X} \to \mathcal{Y}$, written $F_k(x)$ for key k, is a **pseudorandom function (PRF)** if no efficient adversary can tell whether it is talking to F_k with a random key or to a truly random function. Because the adversary may choose its questions, the game gives it an oracle O, a box it can query, and hides behind a fair coin which function is in the box:

1. The challenger flips b. If $b = 0$ it samples a key k and sets $O = F_k$. If $b = 1$ it sets $O = f$, a uniformly random function from $\mathcal{X}$ to $\mathcal{Y}$.
2. The adversary sends queries $x_1, \ldots, x_q$, adaptively (each may depend on earlier answers), and receives $O(x_1), \ldots, O(x_q)$.
3. It outputs a guess b'. Its advantage is $|\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|$, the gap of Fact 1 between how often it says "random" in the two worlds.

A random function on 128-bit inputs is a table with 2^{128} entries, which nobody can store. A PRF gives the same appearance from a 128-bit key. In practice the hash-based message authentication code (HMAC) is modelled as a PRF; the toy code uses HMAC-SHA256.

8.6.4. From a PRG to a PRF: the GGM tree

Goldreich, Goldwasser & Micali (1986) build a PRF from any length-doubling PRG; the construction is called the GGM tree after them. Split the PRG's output into its two halves, $G(s) = G_0(s) \,\|\, G_1(s)$, each as long as the seed. The idea is to use the PRG repeatedly to grow a binary tree of keys, picking the left or right half at each level according to the next bit of the input.

1. Start at the root with the key, $k_\varnothing = k$.
2. For an input $x = x_1 x_2 \cdots x_n$, walk down the tree: the node reached after the first i bits is $k_{x_1 \cdots x_i} = G_{x_i}(k_{x_1 \cdots x_{i-1}})$, the left or right half of the PRG applied to its parent.
3. Output the leaf: $F_k(x) = k_x$.

Proof sketch. The proof replaces the tree by random values one level at a time. Let hybrid H_j replace every node at depth j by an independent uniform string, sampled when first needed, and compute everything below it with G. H_0 is the real F_k, and H_n, where every leaf is independent and uniform, is a random function. Between H_{j-1} and H_j, the only change is at depth j: in H_{j-1} the nodes there are the two halves of G applied to uniform parents, and in H_j they are uniform. Each query is a path from root to leaf, with one node per depth, so a q-query adversary touches at most q parents at depth $j-1$. Telling H_{j-1} from H_j therefore means telling q outputs of G on uniform seeds from q uniform strings. By the hybrid lemma that costs at most q times the PRG advantage. Summing over the n levels (Fact 3): $\mathrm{Adv}_{\mathrm{PRF}} \le nq \cdot \mathrm{Adv}_{\mathrm{PRG}}$. The full proof is in Goldreich, *Foundations of Cryptography*, Vol. 1, §3.6.

8.6.5. The birthday bound

Statement. Many arguments about random values fail only if two of them coincide, so you need to know how likely a coincidence is. The answer is the birthday bound: among q values there are about $q^2/2$ pairs, each pair coincides with probability $1/N$, so collisions become likely once $q^2/2$ is comparable to N, after about $\sqrt{N}$ draws rather than N. Precisely, draw q values independently and uniformly from a set of size N. The probability that two of them coincide satisfies

$$1 - e^{-q(q-1)/(2N)} \;\le\; \Pr[\text{collision}] \;\le\; \frac{q(q-1)}{2N} \le \frac{q^2}{2N}. \tag{8.9}$$

Derivation. The upper bound counts pairs; the lower bound computes the exact probability of no collision and bounds it with $1 - x \le e^{-x}$.

1. **Upper bound.** There are $\binom{q}{2} = q(q-1)/2$ pairs, and each pair coincides with probability $1/N$ (whatever the first value is, the second matches it with probability $1/N$). A collision means at least one pair coincides, so the union bound (Fact 5) gives $\Pr[\text{collision}] \le q(q-1)/(2N)$.
2. **Exact value.** Draw the values one at a time. If no collision has happened yet, the first i values are distinct, and the $(i+1)$-th avoids all of them with probability $1-i/N$. Multiplying these conditional probabilities, $\Pr[\text{no collision}] = \prod_{i=0}^{q-1}(1 - i/N)$.
3. **Lower bound.** By $1-x \le e^{-x}$ applied to each factor, $\Pr[\text{no collision}] \le \exp\left(-\sum_{i<q} i/N\right) = e^{-q(q-1)/(2N)}$, using $\sum_{i<q} i = q(q-1)/2$. Subtracting from 1 gives the lower bound on a collision.
4. **The 50% point.** Setting $1 - e^{-q^2/(2N)} = \frac{1}{2}$ means $q^2/(2N) = \ln 2$, so $q \approx \sqrt{2N \ln 2} \approx 1.18\sqrt{N}$ (since $2 \ln 2 = 1.386$ and $\sqrt{1.386} = 1.18$). Collisions appear after about $\sqrt{N}$ draws, not N.

The two bounds are close. Write $x = q(q-1)/(2N)$, so the union bound is x and the lower bound is $1 - e^{-x}$. On $[0, 1]$ the curve $1 - e^{-x}$ is concave (it bends downwards), so it lies above the straight chord joining its values at 0 and 1, which is $(1 - 1/e)x$. So when $x \le 1$, the lower bound is at least $1 - 1/e \approx 0.63$ times the union bound, and the union bound has the right order.

Examples. Table 8.5 gives the exact probability and both bounds for three cases. The union-bound column is $q(q-1)/(2N)$; for the class, $23 \times 22/730 = 0.693$.

Table 8.5.: Collision probability, exact against the two bounds of Equation 8.9.

Case	q	N	Exact	Union bound	$1 - e^{-q(q-1)/(2N)}$
Shared birthday in a class	23	365	0.507	0.693	0.500
Hashing trick: 1,000 features into 2^{20} buckets	1,000	2^{20}	0.379	0.476	0.379
Random 32-bit identifiers for 100,000 records	100,000	2^{32}	0.688	1.164	0.688

With the hashing trick, 1,000 features hashed into a million buckets already collide with probability 0.379. Random 32-bit identifiers for 100,000 records are more likely than not to clash; the union bound is above 1 and says nothing.

Figure 8.3 checks Equation 8.9 by simulation for $N = 2^{16}$. At $q = 256 = \sqrt{N}$ the exact probability is 0.3927, the simulation (100,000 trials) gives 0.3920, and the union bound gives 0.4980. The 50% point is $q = 301$, in line with $1.18\sqrt{N} = 1.18 \times 256 \approx 302$.

8.6.6. The switching lemma

A block cipher such as AES is a keyed *permutation*: different inputs never give the same output. It is modelled as a **pseudorandom permutation (PRP)**, defined by the PRF game with the random function f replaced by a uniformly random permutation of $\mathcal{X}$ (a random one-to-one shuffle of the inputs). Most proofs, however, want a PRF. The switching lemma bridges the gap. The only visible difference between a random function and a random permutation is that the function sometimes repeats an output and the permutation never does, so an adversary can tell them apart only about as often as q random values collide.

Statement. For any adversary making q queries,

$$\left| \Pr[A^{\text{random function}} = 1] - \Pr[A^{\text{random permutation}} = 1] \right| \le \frac{q(q-1)}{2|\mathcal{X}|} \le \frac{q^2}{2|\mathcal{X}|}. \tag{8.10}$$

Here A^O means the adversary run with oracle O.

Derivation. The plan is to run both oracles from the same pre-drawn list of answers, and show they behave identically unless that list contains a repeat.

1. **Simplify the adversary.** Assume its q queries are distinct: repeating a query teaches it nothing, since both oracles answer the same question the same way every time.

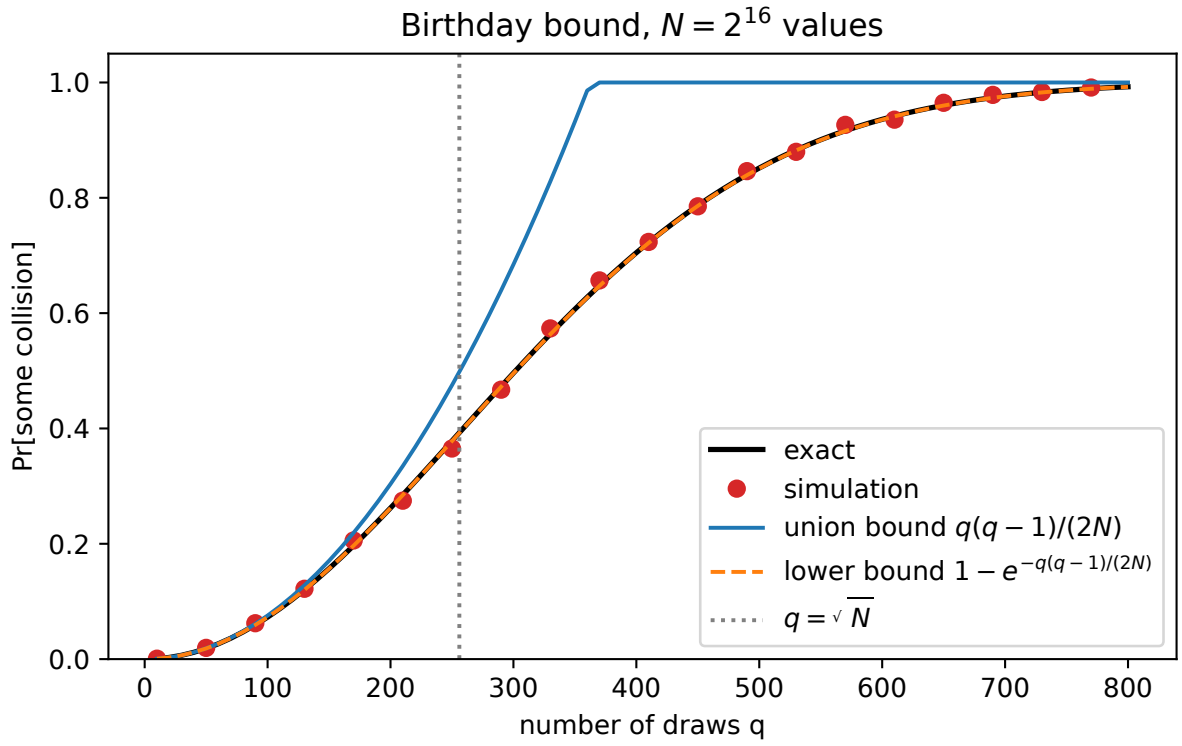


Figure 8.3.: Probability of a collision among q uniform draws from $N = 2^{16}$ values: exact, simulated, the union bound $q(q-1)/(2N)$ and the lower bound $1 - e^{-q(q-1)/(2N)}$.

2. **Pre-sample the answers.** The random-function oracle can be run by drawing $Y_1, \ldots, Y_q$ independently and uniformly from $\mathcal{X}$ in advance and answering the i-th query with Y_i. On distinct queries this is exactly a random function, because a random function gives each new input a fresh uniform output.
3. **Do the same for the permutation.** A random permutation answers the i-th distinct query with a value that is uniform over the $|\mathcal{X}| - i + 1$ values not yet used, whatever the queries are: every unused value is the image of the new query under the same number of permutations consistent with the answers so far. So the permutation oracle can be run by pre-sampling a uniformly random sequence of q *distinct* values.
4. **Compare the two.** Let C be the event that the list $Y_1, \ldots, Y_q$ of step 2 has a collision. Every list has probability $|\mathcal{X}|^{-q}$, so conditioning on no collision leaves every list of distinct values equally likely: exactly the permutation's answers. Hence $\Pr[A^{\mathrm{RF}} = 1 \mid \neg C] = \Pr[A^{\mathrm{RP}} = 1]$, writing RF for the random function and RP for the random permutation.
5. **Bound the difference.** Write $\rho = \Pr[A^{\mathrm{RP}} = 1]$. Splitting on C, $\Pr[A^{\mathrm{RF}} = 1] = \Pr[C]\Pr[A^{\mathrm{RF}} = 1 \mid C] + (1 - \Pr[C])\,\rho$. Subtracting ρ gives $\Pr[A^{\mathrm{RF}} = 1] - \rho = \Pr[C](\Pr[A^{\mathrm{RF}} = 1 \mid C] - \rho)$. The bracket is a difference of two probabilities, so its absolute value is at most 1, and the whole is at most $\Pr[C]$.
6. **Apply the birthday bound** (Equation 8.9) with $N = |\mathcal{X}|$.

The event C is defined on the pre-sampled answers, before the adversary acts, which is what makes step 4 valid for adaptive adversaries: whatever the adversary does, the answers it receives are the same list. (Bellare & Rogaway 2006 point out that a commonly given proof, which conditions on events defined through the adversary's queries, has a gap; their "identical until bad" game-playing proof is the one above.)

Corollary. If F is a PRP with advantage ε against q-query adversaries, it is a PRF with advantage at most $\varepsilon + q^2/(2|\mathcal{X}|)$. This is Fact 3 with three hybrids: going from F_k to a random permutation costs at most ε, and from a random permutation to a random function at most $q^2/(2|\mathcal{X}|)$.

The bound is tight. The adversary "output 1 iff two answers collide" never outputs 1 against a permutation and outputs 1 against a function with probability exactly $\Pr[C]$. So Figure 8.3 is also the advantage of the best adversary in the switching lemma.

8.6.7. Check yourself

- Why can the PRG distinguisher in the "never statistically secure" argument not be used against a real PRG? *(It enumerates all 2^λ seeds, which is not polynomial time.)*
- How many outputs of a 64-bit block cipher can you safely use under one key, if you want the switching term below 2^{-20}? *(Need $q^2/2^{65} \le 2^{-20}$, so $q^2 \le 2^{45}$ and $q \le 2^{22.5}$, about 6 million blocks: a 64-bit block is too small for bulk data.)*

8.7. Encryption: perfect secrecy and IND-CPA

8.7.1. Context

Encryption should hide the message. Shannon's one-time pad hides it perfectly but needs a key as long as all the data. Computational definitions accept a negligible advantage in exchange for

short keys. The definition that matters in practice, IND-CPA, has a consequence that surprises many programmers: encryption must be randomized.

8.7.2. The one-time pad and perfect secrecy

Definition. Hiding the message perfectly means that the ciphertext is no more likely to be produced by one message than by another, so seeing it tells you nothing. An encryption scheme has **perfect secrecy** if the ciphertext distribution does not depend on the message: for all messages m, m' and ciphertexts c, $\Pr[\mathrm{Enc}_K(m) = c] = \Pr[\mathrm{Enc}_K(m') = c]$, where K is the random key. Equivalently, seeing c does not change the probability of any message (Shannon 1949). To see the equivalence, let the message M be random with any prior, and apply Bayes' rule: $\Pr[M = m \mid C = c] = \Pr[C = c \mid M = m] \Pr[M = m] / \Pr[C = c]$. If $\Pr[C = c \mid M = m]$ is the same for every m, it equals its average $\Pr[C = c]$, the ratio is 1, and the posterior equals the prior.

The one-time pad. For n-bit messages, the key K is uniform on $\{0,1\}^n$ and $\mathrm{Enc}_K(m) = m \oplus K$, where $\oplus$ is bitwise exclusive or (XOR): each bit of the message is flipped where the key has a 1.

Derivation of perfect secrecy. Flipping a fixed message's bits by a uniform key gives a uniform result, by Fact 4.

1. Fix m. The map $k \mapsto m \oplus k$ is a bijection of $\{0,1\}^n$: applying it twice gives back k, since $m \oplus m = 0$, so it is its own inverse.
2. By Fact 4, $m \oplus K$ is uniform, whatever m is: $\Pr[\mathrm{Enc}_K(m) = c] = \Pr[K = m \oplus c] = 2^{-n}$, the same for every message.

Table 8.6 shows it for 2-bit messages: each message is sent to every ciphertext by exactly one key.

Table 8.6.: The one-time pad on 2-bit messages. Each row contains every ciphertext once, so the ciphertext is uniform for every message.

Key K	00	01	10	11
Ciphertext for $m = 01$	01	00	11	10
Ciphertext for $m = 10$	10	11	00	01

The price. Perfect secrecy requires at least as many keys as messages. To see why, fix a ciphertext c that occurs. Decryption must work, so each key decrypts c to one message, and at most $|\mathcal{K}|$ messages can produce c. If there are fewer keys than messages, $|\mathcal{K}| < |\mathcal{M}|$, some message m' never produces c, while some other message does, so the ciphertext distributions of those two messages differ. And the pad must not be reused: XOR-ing two ciphertexts under the same pad cancels the pad, $(m_1 \oplus K) \oplus (m_2 \oplus K) = m_1 \oplus m_2$, and reveals the XOR of the two messages.

8.7.3. IND-CPA security

Semantic security (Goldwasser & Micali 1984) says that whatever an efficient adversary can compute about the message from the ciphertext, it could compute without the ciphertext. That is hard to work with directly, because "whatever it can compute" ranges over all functions of the message. It is equivalent to the following game, which is easier to work with: the adversary picks two messages itself, receives an encryption of one of them, chosen by a fair coin, and must say which. If it cannot beat a coin flip even for messages of its own choosing, the ciphertext reveals nothing it could use.

The IND-CPA game (indistinguishability under chosen-plaintext attack):

1. The challenger samples a key k and a fair bit b.
2. The adversary may ask for encryptions $\mathrm{Enc}_k(m)$ of messages of its choice, at any time.
3. It submits two messages m_0, m_1 of equal length and receives the challenge $c^* = \mathrm{Enc}_k(m_b)$.
4. It outputs a guess b'.

The scheme is **IND-CPA secure** if every PPT adversary has negligible advantage $|\Pr[b' = 1 \mid b = 1] - \Pr[b' = 1 \mid b = 0]|$. By Equation 8.2 the win probability is $\frac{1}{2}$ plus half of this gap, so the advantage is also $2|\Pr[b' = b] - \frac{1}{2}|$. The encryption oracle in step 2 models reality: an attacker can often get chosen data encrypted, such as a request it sends or a value it plants in a database.

8.7.4. Why encryption must be randomized

Claim. No deterministic scheme is IND-CPA secure. If encryption always maps the same message to the same ciphertext, the adversary can simply encrypt m_0 itself and compare.

Attack. Ask the oracle for $c_0 = \mathrm{Enc}_k(m_0)$, then submit m_0, m_1. Output $b' = 0$ if $c^* = c_0$ and $b' = 1$ otherwise. If $b = 0$, determinism gives $c^* = c_0$, so $b' = 0$. If $b = 1$, correct decryption forces $\mathrm{Enc}_k(m_1) \neq \mathrm{Enc}_k(m_0)$ (otherwise the same ciphertext would have to decrypt to two messages), so $b' = 1$. The adversary is always right, and the advantage is 1.

The script runs this attack against $\mathrm{Enc}_k(m) = m \oplus F_k(0)$, a deterministic scheme built from the toy PRF. Over 20,000 games for each b the measured advantage is 1.000. The same weakness is visible without any oracle: deterministic encryption of a column of labels maps equal labels to equal ciphertexts, so the frequency of each ciphertext reveals the frequency of each label.

8.7.5. A randomized scheme from a PRF

Construction. The fix is to use a fresh pad for every encryption. Draw a fresh uniform r from a set $\mathcal{R}$ and use $F_k(r)$, a random-looking row of the PRF's table, as a one-time pad; send r along so the receiver can recompute the pad. So $\mathrm{Enc}_k(m) = (r,\ m \oplus F_k(r))$, and to decrypt (r, c) you compute $c \oplus F_k(r)$.

Statement. The scheme can fail in only two ways: the PRF is distinguishable from random, or the challenge reuses an r the adversary has already seen, so its pad is known. The bound adds the

two. For an adversary A making q encryption queries, there is a PRF adversary B, with about the same running time, such that

$$\Pr[A \text{ wins}] \leq \frac{1}{2} + \mathrm{Adv}_{\mathrm{PRF}}(B) + \frac{q}{2|\mathcal{R}|}. \tag{8.11}$$

Derivation.

1. **Set up two hybrids. Hybrid 0** is the real game. **Hybrid 1** is the same game with F_k replaced by a truly random function f.
2. **Hybrid 0 to 1 costs the PRF advantage.** B runs the whole game for A, using its own oracle in place of F_k, and outputs 1 iff A wins. With oracle F_k, B simulates hybrid 0; with oracle f, hybrid 1. So the winning probabilities in the two hybrids differ by at most $\mathrm{Adv}_{\mathrm{PRF}}(B)$.
3. **Inside hybrid 1, name the bad event.** Let E be the event that the challenge's r^* equals one of the q values r_i used by the oracle. Each r_i is independent of r^* and equals it with probability $1/|\mathcal{R}|$, so by the union bound $\Pr[E] \leq q/|\mathcal{R}|$.
4. **If E does not happen, the challenge is a one-time pad.** Then $f(r^*)$ is a value of the random function at a point never queried, so it is uniform and independent of everything else A sees. So $c^* = m_b \oplus f(r^*)$ is uniform whatever b is (Fact 4), and A wins with probability exactly $\frac{1}{2}$.
5. **Combine.** Split the win on whether E happens, and bound the win probability by 1 when it does: $\Pr[\text{win in hybrid 1}] \leq \frac{1}{2}\Pr[\neg E] + \Pr[E] = \frac{1}{2} + \frac{1}{2}\Pr[E] \leq \frac{1}{2} + \frac{q}{2|\mathcal{R}|}$. Add the loss from step 2.

In advantage units, $\mathrm{Adv} = 2(\Pr[\text{win}] - \frac{1}{2})$, this is $\mathrm{Adv}_{\mathrm{CPA}} \leq 2\,\mathrm{Adv}_{\mathrm{PRF}} + q/|\mathcal{R}|$. The term $q/|\mathcal{R}|$ is real, not an artefact of the proof. The script gives the adversary $q = 16$ encryptions of m_0 with 8-bit nonces ($|\mathcal{R}| = 256$); each reply $(r_i, m_0 \oplus F_k(r_i))$ reveals the pad $F_k(r_i)$. If r^* matches one of them, the adversary removes the pad, reads the challenge, and is always right; otherwise it outputs 1. So it says 1 every time when $b = 1$, and when $b = 0$ it says 1 exactly when there is no match. Its advantage is $1 - (1 - \Pr[E]) = \Pr[E]$, and since r^* misses each of the 16 independent nonces with probability $255/256$, $\Pr[E] = 1 - (255/256)^{16} = 0.0607$. The simulation measures 0.0605, and the bound $q/|\mathcal{R}|$ is 0.0625. With 128-bit nonces the same attack has advantage below $16/2^{128} = 2^{-124}$ for $q = 16$.

8.7.6. Check yourself

- Why does the IND-CPA game require $|m_0| = |m_1|$? *(Ciphertext length usually reveals message length. Without the requirement, every practical scheme would be trivially broken by comparing lengths.)*
- In the PRF scheme, where does the proof use that r is fresh for each encryption? *(In step 3: if r could repeat with high probability, E would be likely and the pad $f(r^*)$ would be known.)*

8.8. The simulation paradigm

8.8.1. Context

Encryption hides everything. Many procedures must reveal *something*: a participant learns its own share of a secret, or the final result, and should learn nothing else. "Nothing else" is hard to define directly, because the participant may combine what it sees in clever ways. The simulation paradigm defines it indirectly: whatever the participant sees, it could have produced by itself from what it is allowed to know (Lindell 2017 is a careful tutorial).

8.8.2. Definition

A randomized procedure gives a participant P a **view** $\text{View}_P(x)$: everything P sees during the run, which depends on all inputs x and on the procedure's randomness. P is allowed to learn $L(x)$, its **leakage**: for example its own input and its output. If P could fake its whole view from $L(x)$ alone, the real view cannot contain anything beyond $L(x)$. So the procedure **reveals nothing beyond** L to P if there is an efficient **simulator** S whose output, given only $L(x)$, has the same distribution as the real view:

$$S(L(x)) \equiv \text{View}_P(x) \quad \text{for every input } x. \tag{8.12}$$

Replace $\equiv$ by $\approx_s$ or $\approx_c$ for statistical or computational security.

The two sides are often called the **real world**, where the procedure runs, and the **ideal world**, where a trusted party hands P exactly $L(x)$ and the simulator fakes the rest.

Why this captures "nothing else". Suppose some function g of the view reveals a property of x. Then $g(S(L(x)))$ has the same distribution as $g(\text{View}_P(x))$, because applying the same function to two identically distributed things gives identically distributed results, and it is computed from $L(x)$ alone. So anything the view reveals is already revealed by $L(x)$. Semantic security has the same shape, with $L(m) = |m|$: the ciphertext may reveal the message's length and nothing more.

8.8.3. Worked example: additive secret sharing

Construction. Additive sharing splits a number into random pieces that add up to it, working in $\mathbb{Z}_p$, the integers $\{0, \dots, p-1\}$ with arithmetic mod p. To share a secret $x \in \mathbb{Z}_p$ among h holders, sample $r_1, \dots, r_{h-1}$ uniformly from $\mathbb{Z}_p$ and hand out

$$s_1 = r_1,\ \dots,\ s_{h-1} = r_{h-1}, \qquad s_h = x - (r_1 + \cdots + r_{h-1}) \bmod p. \tag{8.13}$$

All h shares sum to x. Shares also add: if (s_i) shares x and (t_i) shares y, then $(s_i + t_i)$ shares $x + y$.

Statement. Any $h-1$ shares look like independent uniform numbers, whatever the secret is, because the one missing share is what ties them to x. Precisely, any $h-1$ shares are independent and uniform on $\mathbb{Z}_p^{h-1}$, whatever x is. The simulator for a coalition of $h-1$ holders, with no leakage at all, outputs $h-1$ independent uniform elements of $\mathbb{Z}_p$, and Equation 8.12 holds with $\equiv$: perfect security.

Derivation by counting. The plan is to show that the coalition's shares are a one-to-one relabelling of the uniform randomness, and then apply Fact 4.

1. The randomness $(r_1, \dots, r_{h-1})$ is uniform on $\mathbb{Z}_p^{h-1}$: each of the p^{h-1} tuples has probability $p^{-(h-1)}$.
2. Fix x and a coalition that misses holder j. Its shares are a function of the randomness. If $j = h$, the coalition sees $(r_1, \dots, r_{h-1})$ itself. If $j < h$, it sees every r_i with $i \neq j$ together with s_h, and r_j can be recovered uniquely as $r_j = x - s_h - \sum_{i \neq j} r_i$, by rearranging the definition of s_h.
3. Either way the map from randomness to the coalition's shares can be undone, so it is a bijection of $\mathbb{Z}_p^{h-1}$, and every tuple of shares comes from exactly one tuple of randomness (Fact 4).
4. So every tuple of shares has probability $p^{-(h-1)}$, for every x. That is the simulator's output distribution.

Table 8.7 makes step 3 concrete for $h = 2$ and $p = 5$: the second share $x - r$ takes each value exactly once as r runs over $\mathbb{Z}_5$, for both secrets.

Table 8.7.: Two-holder additive sharing in $\mathbb{Z}_5$. Each row is a permutation of $\mathbb{Z}_5$, so the share is uniform for every secret.

r	0	1	2	3	4
Share $x - r \bmod 5$ for $x = 2$	2	1	0	4	3
Share $x - r \bmod 5$ for $x = 4$	4	3	2	1	0

The script repeats the count exhaustively for $h = 3$ and $p = 5$: for every secret and every coalition of two holders, each of the 25 pairs of shares occurs for exactly one of the 25 choices of randomness.

With leakage. If the coalition also learns x (for example because the secret is later published), the simulator receives $L = x$. It samples $h - 1$ uniform shares and sets the last one to x minus their sum. By the statement above, this matches the joint distribution of the real shares and x exactly. This is the typical form of a simulator: random where the view is random, and consistent with the leakage where the leakage pins it down.

8.8.4. Interpretation: perfect, statistical, and broken

Reducing mod p is what makes step 3 a bijection: subtracting r wraps around, so every value is hit once whatever x is. Over the integers it fails, because nothing wraps and the range of the share moves with the secret. Take $x \in \{0, \dots, 7\}$, r uniform on $\{0, \dots, R-1\}$, and the share $x - r$ without reduction. The share is uniform on $\{x - R + 1, \dots, x\}$, an interval of R values that moves with x. For secrets x and x' the two intervals overlap in $R - |x - x'|$ values, so $|x - x'|$ of the R values, each with probability $1/R$, are possible under one secret and impossible under the other. Their TV distance is therefore $|x - x'|/R$, and by Fact 2 that is the best adversary's advantage.

Figure 8.4 shows both cases. With $p = 11$, the shares of $x = 0$ and $x = 7$ have the same flat histogram; the empirical TV distance over 100,000 shares each is 0.0045, which is sampling noise

around the exact value 0. Over the integers with $R = 16$, the supports visibly differ; the empirical TV distance is 0.4423, against the exact $7/16 = 0.4375$. Taking $R = 2^{3+\sigma}$ for secrets below 2^3 makes the TV distance at most $7/2^{3+\sigma}$, which is below $2^{-\sigma}$: statistical rather than perfect security.

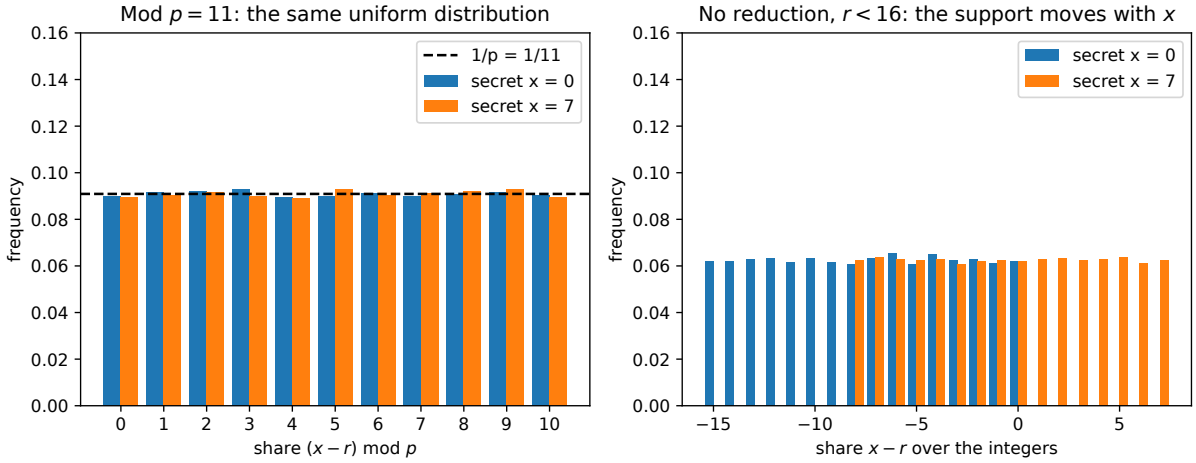


Figure 8.4.: Histograms of the second share for secrets $x = 0$ and $x = 7$. Left: reduced mod $p = 11$, both are uniform. Right: over the integers with $r < 16$, the support shifts with the secret.

8.8.5. Check yourself

- Two of three holders combine their shares of x. What do they learn? *(Nothing: two shares out of three are uniform on $\mathbb{Z}_p^2$ whatever x is. Only all three together determine x.)*
- Why must the simulator be efficient? *(An unbounded simulator could, in a computational setting, invert functions the participant cannot. The definition would then accept procedures whose views reveal information that is hard to compute from $L(x)$ but easy to read from the view.)*

8.9. Learning with errors

8.9.1. Context

Every primitive so far is generic. Learning with errors (LWE) is a specific hard problem, introduced by Regev (2005), with three attractions: it is as hard as worst-case lattice problems, no efficient quantum attack is known, and encryption built on it lets you add ciphertexts. Its one-line description is “linear algebra with noise”, and the noise is everything.

8.9.2. Definition

LWE hides a secret vector behind linear equations whose right-hand sides are slightly wrong. Fix a dimension d, a modulus p (an integer that need not be prime), and an error distribution χ on

small integers. Pick a secret s uniform in $\mathbb{Z}_p^d$, vectors of d integers mod p. An **LWE sample** is a random vector a together with the inner product $\langle a, s\rangle = \sum_j a_j s_j$, perturbed by a small error e:

$$(a,\ b = \langle a, s\rangle + e \bmod p), \qquad a \text{ uniform in } \mathbb{Z}_p^d,\ e \sim \chi. \tag{8.14}$$

There are two versions of the problem. **Search LWE** asks you to find s, given m samples. **Decision LWE** asks you to decide, given m pairs, whether they are LWE samples or uniform pairs in $\mathbb{Z}_p^d \times \mathbb{Z}_p$.

If you can solve search, you can solve decision: find a candidate s, then check whether the values $b - \langle a, s\rangle$ are small, which they are for LWE samples and are not for uniform pairs. So decision is no harder than search. Regev shows the converse for prime p polynomial in d, by guessing the coordinates of s one at a time and using the decision oracle to test each guess.

Hardness. For χ a discretized Gaussian (a normal distribution rounded to integers) of standard deviation about αp with $\alpha p > 2\sqrt{d}$, Regev (2009) proves that solving LWE is at least as hard as solving worst-case lattice problems with a quantum algorithm. The lattice problems are GapSVP, the gap version of the shortest vector problem, and the shortest independent vectors problem, both to within an approximation factor $\tilde{O}(d/\alpha)$ (d/α up to logarithmic factors). Peikert (2009) and Brakerski, Langlois, Peikert, Regev & Stehlé (2013) give classical reductions. Treat these as cited results; the proofs are long.

8.9.3. Without noise, LWE is easy

With $e = 0$, the samples are a linear system $As = b$ over $\mathbb{Z}_p$, where the rows of A are the vectors a, and Gaussian elimination solves it from d independent equations.

Here it is with $d = 2$, $p = 7$ and $s = (3, 5)$. Take $a_1 = (1, 2)$ and $a_2 = (3, 1)$, so $b_1 = 3{+}10 = 13 \equiv 6$ and $b_2 = 9 + 5 = 14 \equiv 0$.

1. The second equation $3s_1 + s_2 = 0$ gives $s_2 = -3s_1 = 4s_1$, since $-3 \equiv 4 \pmod 7$.
2. Substitute into $s_1 + 2s_2 = 6$: $9s_1 \equiv 2s_1 = 6$, so $s_1 = 3$ and $s_2 = 12 \equiv 5$. Correct.

Now add the error $e = (1, 0)$, so $b = (0, 0)$. The same steps give $s_2 = 4s_1$ and $2s_1 = 0$, so $\hat{s} = (0, 0)$, completely wrong. The reason is that elimination solves $A\hat{s} = b = As + e$ exactly, so in general it returns $\hat{s} = s + A^{-1}e$, and A^{-1} has large entries mod p. Here $A^{-1} = \begin{pmatrix} 4 & 6 \\ 2 & 4 \end{pmatrix}$ (check: the first row of A times the first column gives $1 \cdot 4 + 2 \cdot 2 = 8 \equiv 1$), and $A^{-1}(1, 0) = (4, 2)$, which shifts s to $(7, 7) \equiv (0, 0)$. A unit error is amplified into a uniformly scrambled answer.

The script does the same at $d = 16$ and $p = 4096$ with 32 samples. (Mod a power of two, elimination may only divide by odd numbers, the invertible ones, so it picks odd pivots.) Without noise it recovers all 16 coordinates of s. With errors in $\{-1, 0, 1\}$, it recovers 0 of 16.

8.9.4. Regev's symmetric bit encryption

Construction. Picture $\mathbb{Z}_p$ as a clock with p positions. To encrypt a bit, take an LWE sample and move its b halfway round the clock if the bit is 1. Whoever knows s can subtract $\langle a, s\rangle$ and see whether what remains is near 0 or near halfway; the small error does not hide that. Concretely, take p even. The key is $s \in \mathbb{Z}_p^d$. To encrypt a bit m, draw a fresh a and $e \sim \chi$ and output

$$\mathrm{Enc}_s(m) = (a,\ b = \langle a, s\rangle + e + m \cdot \tfrac{p}{2} \bmod p). \tag{8.15}$$

To decrypt, compute $v = b - \langle a, s\rangle \bmod p$ and output 1 if v is closer to $p/2$ than to 0 (counting distance round the clock, so that values just below p are close to 0), that is if $p/4 < v < 3p/4$.

Statement (correctness). Decryption is correct whenever $|e| < p/4$.

Derivation. Removing $\langle a, s\rangle$ leaves the error plus the encoded bit; the error is too small to move it across the decision boundary.

1. $v = e + m\frac{p}{2} \bmod p$, by the definition of b.
2. If $m = 0$, $v \equiv e$ with $|e| < p/4$, so $v \in [0, p/4) \cup (3p/4, p)$ (a negative e wraps to just below p) and the output is 0.
3. If $m = 1$, $v = e + p/2 \in (p/4, 3p/4)$ and the output is 1.

Security, in one hybrid. Without s, the pair $(a, \langle a, s\rangle + e)$ looks uniform by decision LWE, so the message is added to something that looks uniform: a one-time pad. Under decision LWE, the pairs $(a, \langle a, s\rangle + e)$ in all ciphertexts can be replaced by uniform pairs (a, u). Then $b = u + mq/2$ is uniform whatever m is (Fact 4, with the bijection "add $mq/2$"). So an IND-CPA adversary winning with probability $\frac{1}{2} + \varepsilon$ yields a decision-LWE distinguisher with advantage ε: it runs the game with the given pairs and outputs 1 iff the adversary wins, by the argument of step 2 in the PRF scheme.

Worked example with $d = 3$, $p = 4096$ and $s = (5, 1000, 3000)$. Here $p/2 = 2048$, and reducing mod 4096 means subtracting 4096.

- Encrypt $m = 1$ with $a_1 = (17, 2, 1)$ and $e_1 = -7$: $\langle a_1, s\rangle = 85 + 2000 + 3000 = 5085 \equiv 5085 - 4096 = 989$, so $b_1 = 989 - 7 + 2048 = 3030$. Decrypting, $v = 3030 - 989 = 2041$, which is 7 from $p/2 = 2048$: the output is 1.
- Encrypt $m = 1$ with $a_2 = (1, 1, 1)$ and $e_2 = 12$: $\langle a_2, s\rangle = 4005$, so $b_2 = 4005 + 12 + 2048 = 6065 \equiv 1969$.

8.9.5. Adding ciphertexts, and the noise budget

Statement. Because everything in a ciphertext is linear in a, the error and the encoded bit, adding ciphertexts adds all three. The bits add mod 2, since two half-turns round the clock are a full turn, and the errors pile up. So the componentwise sum of ciphertexts of $m_1, \ldots, m_k$ is a ciphertext of $m_1 \oplus \cdots \oplus m_k$ with error $e_1 + \cdots + e_k$. If every $|e_i| \le B$, the total error is at most kB, and it decrypts correctly whenever

$$k < \frac{p}{4B}. \tag{8.16}$$

Derivation.

1. The sum is $(\sum_i a_i,\ \sum_i b_i)$. By linearity of the inner product, $\sum_i b_i - \langle \sum_i a_i, s\rangle = \sum_i e_i + (\sum_i m_i)\frac{p}{2} \bmod p$.
2. Because p is even, $2 \cdot \frac{p}{2} = p \equiv 0$, so only the parity of $\sum_i m_i$ survives: $(\sum_i m_i)\frac{p}{2} \equiv (m_1 \oplus \cdots \oplus m_k)\frac{p}{2}$.
3. So the sum is an encryption of the XOR with error $\sum_i e_i$, and $|\sum_i e_i| \le kB$ by the triangle inequality. By the correctness statement, $kB < p/4$ suffices.

In the worked example, the sum is $a_1 + a_2 = (18, 3, 2)$ and $b_1 + b_2 = 4999 \equiv 903$. Then $\langle (18, 3, 2), s\rangle = 9090 \equiv 898$, and $v = 903 - 898 = 5 = e_1 + e_2$: it decrypts to $0 = 1 \oplus 1$.

Interpretation: the worst case and the typical case. Equation 8.16 is a worst-case guarantee: it assumes every error is as large as possible and all have the same sign. Independent errors with mean zero mostly cancel. Variances of independent variables add, so the sum of k errors with standard deviation σ has standard deviation $\sigma\sqrt{k}$, not kB. With χ uniform on the $2B + 1$ values $\{-B, \dots, B\}$, the variance is the average of j^2 over those values, $2\sum_{j=1}^{B} j^2/(2B + 1) = 2 \cdot \frac{B(B+1)(2B+1)}{6}/(2B + 1) = B(B + 1)/3$. By the central limit theorem the sum is approximately normal, and decryption fails when the summed error, read round the clock, lands more than $p/4$ from 0. So the normal approximation predicts

$$\Pr[\text{failure after } k \text{ additions}] \approx \Pr\left[\left|Z\sqrt{kB(B+1)/3}\right| > p/4 \text{ after wrapping mod } p\right], \quad Z \sim \mathcal{N}(0, 1). \tag{8.17}$$

Figure 8.5 and Table 8.8 compare the two for $p = 4096$ and $B = 32$, where Equation 8.16 guarantees correctness for $k < 32$ (since $p/(4B) = 4096/128 = 32$). The simulation adds k real toy ciphertexts (with $d = 4$) of random bits, 20,000 times, and checks the decrypted XOR.

Table 8.8.: Decryption failure rate of a sum of k Regev ciphertexts, $p = 4096$, errors uniform on $\{-32, \dots, 32\}$ (seed 0, 20,000 trials).

Ciphertexts added k	Empirical failure rate	Normal approximation
1	0.0000	0.0000
31	0.0000	0.0000
256	0.0007	0.0006
1,024	0.0868	0.0881
4,096	0.3814	0.3833

To see where a row comes from, take $k = 1024$: the standard deviation of the summed error is $\sqrt{1024 \times 32 \times 33/3} \approx 600$, and $p/4 = 1024$ is about 1.7 standard deviations, which a normal variable exceeds in absolute value with probability about 0.088. At $k = 256$ the standard deviation is about 300 and $p/4$ is 3.4 standard deviations away, far in the tail.

The worst-case budget is therefore very conservative. Failures only become measurable around $k = 256$, eight times the guaranteed 32, because the noise grows like $\sqrt{k}$ rather than k. But the budget is still finite. Once the noise is comparable to $p/4$, the decrypted value is spread round the whole clock, decryption becomes a coin flip, and the failure rate tends to $\frac{1}{2}$. Larger p (relative to the noise) buys more additions; that is the parameter trade-off of every LWE-based scheme that computes on ciphertexts.

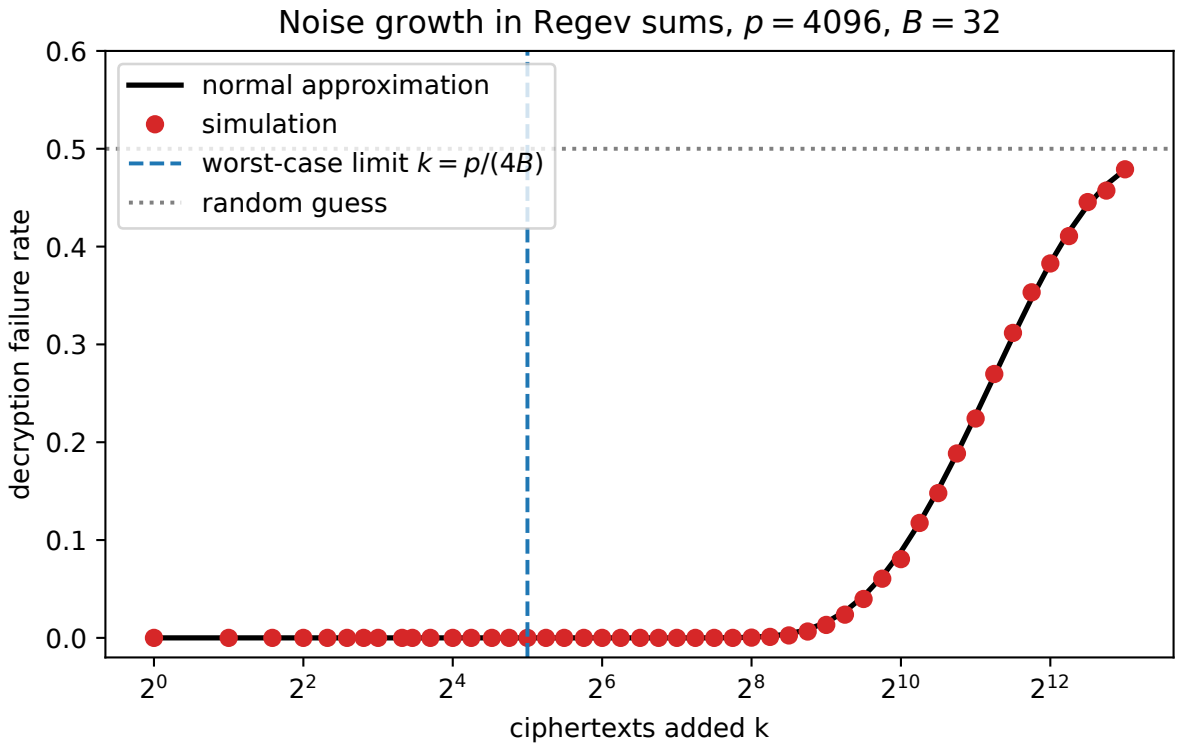


Figure 8.5.: Decryption failure rate of a sum of k Regev ciphertexts ($p = 4096$, errors uniform on $\{-32, \ldots, 32\}$): simulation against the normal approximation. The dashed line is the worst-case guarantee $k < p/(4B) = 32$.

8.9.6. Ring-LWE, briefly

A Regev ciphertext carries one bit in $d+1$ numbers, and the key has d numbers. Ring-LWE packs much more into the same space by replacing vectors with polynomials. **Ring-LWE** (Lyubashevsky, Peikert & Regev 2010) replaces vectors in $\mathbb{Z}_p^d$ by polynomials in $R_p = \mathbb{Z}_p[X]/(X^N+1)$, that is, polynomials of degree below N with coefficients mod p, multiplied with the rule $X^N = -1$, and replaces the inner product by polynomial multiplication. One ring sample $(a, a \cdot s + e)$ stands for N correlated LWE samples, carries N message coefficients, and is computed in $O(N \log N)$ time with the number-theoretic transform, a fast Fourier transform for arithmetic mod p. Its hardness rests on lattice problems restricted to ideal lattices, which have this extra structure. A middle ground, module-LWE, underlies the standardized key-encapsulation mechanism ML-KEM (FIPS 203, 2024).

8.9.7. Check yourself

- Why does p have to be even in the addition argument, and what happens for odd p with $\lfloor p/2 \rfloor$ as the encoding? *(For odd p, $2\lfloor p/2 \rfloor = p - 1 \equiv -1$, so every pair of 1-bits adds an extra error of -1. The error drifts by about $k/4$ on top of the random part.)*
- Is the LWE problem with χ uniform on $\{-B, \dots, B\}$ covered by Regev's theorem? *(Not directly. The theorem is for a Gaussian-shaped error; other small distributions need separate arguments. The toy uses a uniform error only because its worst case is easy to state.)*

8.10. Code: check the numbers by simulation

The script code/crypto_basics.py computes every number quoted in this chapter and draws its figures. Each bound, probability and toy scheme is a small typed function; the script calls them once for the table and loops them over parameters for the figures. From the repository root:

```
uv run python code/part01-prerequisites/crypto_basics.py
```

Table 8.9 summarizes the output (seed 0).

Table 8.9.: Output of the chapter's script (seed 0).

Experiment	Result
Example functions at $\lambda = 128$	$2^{-128} = 2.9 \times 10^{-39}$; $\lambda^{-\log_2 \lambda} = 1.8 \times 10^{-15}$; $2^{-\sqrt{\lambda}} = 3.9 \times 10^{-4}$; $\lambda^{-10} = 8.5 \times 10^{-22}$.
	$2^{-\sqrt{\lambda}} < \lambda^{-10}$ from $\lambda = 20{,}520$
Brute force at 10^{18} keys/s for a year	$2^{84.7}$ keys; success 9.3×10^{-14} against 128 bits; 1.08×10^{13} years for all 2^{128}
Birthday, $N = 2^{16}$, $q = 256$	Exact 0.3927; simulated 0.3920; union bound 0.4980; 50% at $q = 301$

Experiment	Result
Biased coin, $\delta = 0.01$	$m = 1$: majority 0.008, best 0.010, hybrid 0.010. $m = 100$: 0.082, 0.080, 1.000. $m = 10{,}000$: 0.481, 0.683, 1.000
IND-CPA, deterministic scheme	Advantage 1.000
IND-CPA, PRF scheme, $q = 16$, 8-bit nonce	Simulated 0.0605; exact 0.0607; bound 0.0625
Secret shares, $x = 0$ against $x = 7$	Mod 11: TV 0.0045 (exact 0). Integers, $R = 16$: TV 0.4423 (exact 0.4375)
Gaussian elimination, $d = 16$, $p = 4096$	16/16 coordinates without noise; 0/16 with errors in $\{-1, 0, 1\}$
Regev sums, $p = 4096$, $B = 32$	See Table 8.8

tests/test_crypto_basics.py checks that the output still matches.

What to take from it. Asymptotic statements need concrete numbers, because a negligible function can be large at the parameter you use: at $\lambda = 128$, $2^{-\sqrt{\lambda}}$ is still 3.9×10^{-4}. The birthday bound is the right order and nearly tight: while it is below 1, the union bound overestimates the collision probability by at most a factor of about 1.6 (that is $1/(1 - 1/e)$, from the chord argument), and the switching lemma's best adversary attains the exact collision probability. Hybrid bounds are safe but can be loose: for the biased coin they overstate the advantage by a factor up to $\sqrt{m}$, because the hybrid argument pays δ per flip while the true advantage grows like $\delta\sqrt{m}$. Determinism is fatal, and short nonces leak exactly as the proof predicts. Finally, noise is what makes LWE hard and what limits computing on its ciphertexts. Without it, elimination solves the problem; with it, every addition spends part of a finite budget.

8.11. References

- Katz & Lindell, *Introduction to Modern Cryptography* (3rd ed., 2020), ch. 2–3: perfect secrecy, computational security, PRGs, PRFs and CPA security, with complete proofs at the level of this chapter
- Boneh & Shoup, *A Graduate Course in Applied Cryptography*, ch. 2–5: semantic security, PRGs and the hybrid argument, block ciphers, PRFs, the switching lemma and the GGM tree, CPA security; written in the concrete-security style
- Goldreich, *Foundations of Cryptography*, Vol. 1 (2001), ch. 3: pseudorandomness in depth, including the GGM proof (§3.6)
- Shannon, "Communication Theory of Secrecy Systems" (1949): perfect secrecy and the key-length bound
- Goldwasser & Micali, "Probabilistic Encryption" (JCSS 1984): semantic security and the need for randomized encryption
- Goldreich, Goldwasser & Micali, "How to Construct Random Functions" (JACM 1986): the GGM construction
- Håstad, Impagliazzo, Levin & Luby, "A Pseudorandom Generator from any One-way Function" (SIAM J. Computing 1999)

- Bellare & Rogaway, "The Security of Triple Encryption and a Framework for Code-Based Game-Playing Proofs" (EUROCRYPT 2006): game-playing proofs and a careful switching lemma
- Lindell, "How to Simulate It — A Tutorial on the Simulation Proof Technique" (2017): the simulation paradigm, step by step
- Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography" (STOC 2005; JACM 2009): LWE, its quantum hardness reduction, and the original encryption scheme
- Peikert, "A Decade of Lattice Cryptography" (2016): survey of LWE, its variants and reductions
- Lyubashevsky, Peikert & Regev, "On Ideal Lattices and Learning with Errors over Rings" (EUROCRYPT 2010): ring-LWE

8.12. Cheat sheet

Table 8.10 collects the results of this chapter.

Table 8.10.: Summary of the definitions and bounds in this chapter.

Result	Statement
Advantage	$\mathrm{Adv} = \lvert\Pr[A(X)=1] - \Pr[A(Y)=1]\rvert = 2\lvert\Pr[\mathrm{win}] - \frac{1}{2}\rvert$
Best unbounded distinguisher	Advantage $= \mathrm{TV}(X, Y)$
Negligible	$\mu(\lambda) < \lambda^{-c}$ eventually, for every c; closed under $\times$ poly and $+$
Hybrid lemma	m samples: advantage at most m times the one-sample advantage
Birthday bound	$1-e^{-q(q-1)/(2N)} \le \Pr[\mathrm{collision}] \le q(q-1)/(2N)$; 50% at $q \approx 1.18\sqrt{N}$
PRP/PRF switching	Extra advantage $\le q^2/(2\lvert\mathcal{X}\rvert)$, and tight
GGM	$\mathrm{Adv}_{\mathrm{PRF}} \le nq \cdot \mathrm{Adv}_{\mathrm{PRG}}$
One-time pad	Perfect secrecy; needs $\lvert\mathcal{K}\rvert \ge \lvert\mathcal{M}\rvert$; never reuse the pad
IND-CPA	Deterministic schemes have advantage 1; $(r, m \oplus F_k(r))$ has $\Pr[\mathrm{win}] \le \frac{1}{2} + \mathrm{Adv}_{\mathrm{PRF}} + q/(2\lvert\mathcal{R}\rvert)$
Simulation	Secure if $S(L(x)) \equiv$ (or $\approx$) $\mathrm{View}(x)$ for all x
Additive sharing in $\mathbb{Z}_p$	Any $h-1$ shares uniform: perfect security
LWE	$(a, \langle a, s\rangle + e)$; easy without e by elimination
Regev bit encryption	Correct if $\lvert e\rvert < p/4$; sums of k ciphertexts correct if $kB < p/4$; typical noise $\sim \sigma\sqrt{k}$

Rules of thumb

- Security of κ bits means an attacker with time t succeeds with probability about $t/2^{\kappa}$.
- Anything random of κ bits starts colliding after about $2^{\kappa/2}$ draws: budget identifiers, nonces and block counts accordingly.
- Encryption that hides equality must be randomized.
- In LWE schemes, noise is a budget: each operation spends some, and decryption fails when it runs out.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

Part II.

Threat models and attacks

This part teaches you to break models before it teaches you to protect them. Every defence in the rest of the book is measured against an attack: a guarantee is only as meaningful as the strongest adversary it stops, and an evaluation is only as honest as the attack it runs. So you start here, with the adversaries, what they see, what they want, and how well the best of them can do.

Who this part is for

You are expected to have read the Prerequisites, or to know their content. This part leans hardest on hypothesis testing (the Neyman–Pearson lemma, trade-off curves, receiver operating characteristic (ROC) curves read at a low false positive rate), on the total variation distance, and on the link between stability and generalization. Information theory appears in the chapters on attribute inference, memorization and linkage. You should also be comfortable with logistic regression, gradients of a small neural network, and an n-gram language model at the level of writing them in NumPy.

Differential privacy, the defence most of the book is about, is introduced in the next part, and it is designed against exactly the games defined here. The only privacy definitions in this part are the syntactic ones of the last chapter, k-anonymity and its successors, and they appear as targets: the chapter shows what each one promises and the attack that breaks it.

The common thread

Every attack in this part is the same object: a **game** in which a challenger trains or releases something that depends on a hidden **secret**, and an adversary guesses the secret from what the **threat model** lets it see. What changes from chapter to chapter is the secret: one bit of membership, a sensitive attribute, a memorized string, the whole dataset, a client's input, a property of the population behind the data, or a person's name. Table 8.11 lists them.

Table 8.11.: The chapters of this part, the question each one answers, and the earlier chapters it uses.

Chapter	The question it answers	Builds on
Threat models and privacy games	What can an adversary see and do, and how is an attack stated and scored?	3, 4, 8
Membership inference	Was this record in the training set?	4, 7, 9
Attribute inference and model inversion	Does the model reveal a person's sensitive attribute beyond what the population already does?	4, 5, 9, 10
Property inference	Can a model reveal a fact about its training population that no single record reveals?	3, 4, 9, 10

Chapter	The question it answers	Builds on
Memorization and training-data extraction	Can the model be made to emit its training text, and when?	5, 7, 9, 10
Gradient inversion	What does a gradient computed on private data reveal about that data?	6, 9
Linkage and re-identification	When does removing names fail to hide who a record belongs to?	1, 2, 5, 8, 9
Syntactic anonymity and how it breaks	What do k-anonymity, l-diversity and t-closeness guarantee, and which attacks get past each?	1, 3, 5, 11, 15
Reconstruction attacks	How many accurate statistics does it take to rebuild a whole private dataset?	2, 5, 9

Two ideas run through all nine chapters.

- **An attack is a hypothesis test.** When the secret is one bit, the best attack is the likelihood-ratio test of the Neyman–Pearson lemma, its **advantage** (true positive rate (TPR) minus false positive rate (FPR)) is at most the total variation distance between the adversary's two possible views, and its whole error trade-off is bounded by one curve. Membership inference, attribute inference and property inference are each this statement with a different pair of views. The strongest practical attacks, such as the likelihood ratio attack (LiRA) on membership, are explicit approximations of the optimal test.
- **The worst case is not the average case.** Attacks succeed first on outliers, on duplicated strings, on unique combinations of attributes and on the one example in a batch that a neuron isolates. An average over records can look harmless while a few records are fully exposed. This is why attacks are reported as the TPR at a low FPR rather than as accuracy or area under the curve (AUC), and why the defence of the next part has to hold for every record and every dataset.

A third idea closes the part and frames what comes after it. Reconstruction attacks show that answering too many statistics too accurately destroys privacy no matter how the answers are computed: this is the **Fundamental Law of Information Recovery**. Some noise is not optional, and the question becomes how little noise is enough.

How to read this part

Order. The part opens with chapter 9, which sets up the vocabulary, and then has two halves. Chapters 10 to 14 attack trained models, from the smallest secret to the largest: one bit of membership, a sensitive attribute, a property of the training population, memorized text, and whole inputs recovered from gradients. Chapters 15 to 17 attack released data, from the weakest

defence to the strongest: names removed (linkage), records hidden in crowds (syntactic anonymity), and only aggregate statistics published (reconstruction).

Read chapter 9 first. It fixes the vocabulary (challenger, adversary, view, game, advantage, baseline) that the others use without redefining. Then read chapter 10, since membership inference is the reference attack that chapters 11, 12 and 13 extend. Chapters 11 to 14 can then be read in any order. Read chapters 15 to 17 in order: chapter 16 starts from the linkage game of chapter 15 and uses the imputation baseline of chapter 11, and chapter 17 shows that even exact aggregate statistics, with no records released at all, give the data away. Chapters 16 and 17 matter most for what comes next: the attacks of chapter 16 are the requirements the next part's definition is built to meet, and the results of chapter 17 return as lower bounds there.

Structure of a chapter. Every chapter has the same shape as in the prerequisites: why the topic matters, a toolkit, one section per result with a context, statement, derivation, interpretation, worked example and two *check yourself* questions, then code, references and a cheat sheet. Exercises with worked solutions are collected in the exercise book, a companion website, with a link at the end of each chapter.

Toy experiments, real pipelines. The attacks in the literature run on image classifiers and large language models trained on GPUs. Here every experiment is a NumPy toy that runs in seconds: logistic regression on synthetic data, Gaussian mean estimation, a small multilayer perceptron written by hand, an n-gram language model with planted canaries. Each toy is chosen so that the optimal attack can be computed exactly and compared with the practical one. Where a chapter's study-plan exercise needs the full pipeline (such as LiRA with 16 shadow models on CIFAR-10), the chapter describes that pipeline in prose and runs a faithful scaled-down version.

Run the code. From the repository root, for example,

```
uv run python code/part02-attacks/membership_inference.py
```

The scripts use fixed seeds, so their output matches the tables in the text.

Notation

The chapters share the conventions in Table 8.12, set in Threat models and privacy games. Where a chapter departs from them, it says so at its start.

Table 8.12.: Notation shared by the chapters of this part.

Symbol	Meaning
D, π	A dataset of records; the population they are drawn from
$\mathcal{T}$, $\theta = \mathcal{T}(D)$	A (possibly randomized) training algorithm, and the model it outputs
b	The hidden bit of a two-world game; $b = 1$ is the world where the target record is used (the member world)

Symbol	Meaning
V; P, Q	The adversary's view; its distributions when $b = 0$ and $b = 1$
TPR, FPR	$\Pr[b' = 1 \mid b = 1]$ and $\Pr[b' = 1 \mid b = 0]$ for the guess b'
Adv	The advantage, TPR $-$ FPR, at most $\mathrm{TV}(P, Q)$
$T(P, Q)$	The trade-off function: the smallest false negative rate $1 - \mathrm{TPR}$ of any test at each FPR
$\ell(\theta, z)$	The loss of model θ on record z

What you will be able to do

By the end of this part you should be able to:

- state any privacy attack as a game, name its threat model and its baseline, and bound its advantage by a total variation distance;
- derive the loss-threshold membership attack from the generalization gap and LiRA from the Neyman–Pearson lemma, and evaluate both by the TPR at a low FPR;
- separate attribute inference from what the population already reveals, and say why masking confidences does not stop it;
- explain how a property of the training population can leak when no individual record does;
- define verbatim, discoverable and extractable memorization, and compute a canary's exposure;
- recover an input exactly from the gradient of a fully connected layer, and say why averaging a batch does not remove the leak;
- compute the fraction of a population made unique by a set of quasi-identifiers, and run a Netflix-style de-anonymization;
- say what k-anonymity, l-diversity and t-closeness each guarantee, and break each one with homogeneity, skewness and composition attacks;
- prove that answers with error $o(\sqrt{n})$ to random subset sums allow almost all of a dataset to be reconstructed.

Each of these is a result derived in one of the chapters that follow.

9. Threat models and privacy games

9.1. Why this matters

A hospital trains a model on patient records and offers it to doctors through a web service. Is the model "private"? The question has no answer until you say **against whom**. Can the attacker only send inputs and read predicted labels, or does it hold the weights? Does it know the other patients in the training set, or only what the population looks like? Does it follow the rules of the service, or does it also slip crafted records into the training data? And what does it want: to learn that one person was a patient, to learn their diagnosis, or to rebuild the whole table? A **threat model** answers these questions. Without one, "the model leaks" and "the model is safe" are both untestable claims.

Once the threat model is fixed, an attack can be written down as a **game**, the way cryptographers write down attacks on encryption. A **challenger** runs the training pipeline with some hidden **secret**, such as whether one patient's record was used. It hands the **adversary** what the threat model allows it to see, and the adversary guesses the secret. An attack is good if it guesses better than someone who never saw the model.

That is the one idea of this chapter: **an attack is a guess about a hidden secret, made from a view that the threat model fixes.** When the secret is a single bit, such as "was this patient in the training set?", the game is a hypothesis test, so everything in Hypothesis testing carries over. The attack's **advantage** is its true positive rate (TPR) minus its false positive rate (FPR). No attack can push it above the total variation (TV) distance between what the adversary sees in the two worlds, and one curve bounds the attack's whole error trade-off.

The chapter builds this up in four steps. It first lists what a threat model has to specify, and proves the one ordering that always holds: more access never makes the adversary weaker. It then writes the attacks of this part as games, each with its own baseline. Next it turns the hypothesis-testing results into ceilings on any attack, and explains why the low false-positive end of the curve is where privacy is really lost. Finally it shows how far apart the average-case and worst-case versions of the same game can be. Every later chapter of this part is an instance of these games: membership inference, attribute inference, dataset properties and reconstruction. Table 9.1 lists the results and where they are used.

Table 9.1.: The results of this chapter and where each is used.

Result	Where it is used
Threat-model taxonomy: access, behaviour, trust, side information	Stating what an attack assumes, before reading any number it reports

Result	Where it is used
More access never lowers the best advantage	Comparing label-only, black-box and white-box attacks
Privacy games: membership, attribute, reconstruction, distinguishing	A precise definition of every attack in this part
Baselines and the Bayes-optimal guess	Telling an attack apart from a good guess about the population
Advantage $\leq$ TV; attack's error curve below the trade-off curve	The ceiling every attack runs into; the target defences aim at
TPR at a low FPR, precision	How attacks are evaluated in practice
Average case $\leq$ worst case	Why a defence must hold for every record and every dataset

What this chapter assumes. The hypothesis-testing vocabulary (type I and II errors, the Neyman–Pearson lemma, the trade-off function $T(P, Q)$, receiver operating characteristic (ROC) curves) is in Hypothesis testing. The total variation distance and the hockey-stick divergence E_γ are in Divergences. Security games and advantage, in their cryptographic form, are in Cryptography basics. No privacy definition is introduced here: this part only attacks. The definition that defends against these games comes in Part 3.

Conventions. D is a dataset of records drawn from a population π. A training algorithm $\mathcal{T}$ maps D to a model $\theta = \mathcal{T}(D)$, possibly at random. In a two-world game a hidden bit b selects the world: $b = 1$ is the world in which the target's record is used ("member", the alternative hypothesis), and $b = 0$ the world in which it is not (the null). Everything the adversary gets to look at, including what it knew beforehand, is its **view** V. P and Q are the distributions of V when $b = 0$ and $b = 1$. An attack outputs a guess b', 1 for "member" and 0 for "not a member". Two numbers describe how it behaves. Its **true positive rate** is the fraction of members it correctly flags, and its **false positive rate** is the fraction of non-members it wrongly flags:

$$\text{TPR} = \Pr[b' = 1 \mid b = 1], \qquad \text{FPR} = \Pr[b' = 1 \mid b = 0]. \tag{9.1}$$

9.2. Toolkit: three facts every derivation uses

9.2.1. Fact 1 — A game with a hidden bit is a hypothesis test

The challenger flips the hidden bit before anything else happens: $b = 1$ with probability π_1 and $b = 0$ with probability $\pi_0 = 1 - \pi_1$. The adversary then sees its view and announces b'. It wins when its guess matches the hidden bit, whichever value the bit has: $b' = b$ means "$b' = 1$ and $b = 1$, or $b' = 0$ and $b = 0$". So $\Pr[b' = b]$ is the probability that the guess is correct, taken over the challenger's coin flip and everything else that is random in the game. It is not the probability that b is 1, which is just π_1. The probability of a correct guess is

$$\Pr[b' = b] = \pi_1 \,\mathrm{TPR} + \pi_0 \,(1 - \mathrm{FPR}). \tag{9.2}$$

Derivation. The idea is to split the games by which world the challenger chose, count the wins in each world, and add.

1. **Split the win into the two worlds.** There are exactly two ways to win: the bit is 1 and the attack says 1, or the bit is 0 and the attack says 0. In a single game the bit has one value, either 1 or 0, so a game can win in one of these ways or the other, never both. Events that cannot happen together are called **disjoint**, and the probability that one of two disjoint events happens is the sum of their probabilities. Counting games shows why: if you count the games won with $b = 1$ and the games won with $b = 0$, every won game is counted exactly once, because no game belongs to both groups. (For events that can overlap you would have to subtract the overlap, which here is empty.) So

$$\Pr[b' = b] = \Pr[b = 1 \text{ and } b' = 1] + \Pr[b = 0 \text{ and } b' = 0]. \tag{9.3}$$

2. **Write each joint probability as "prior times conditional".** For any two events, $\Pr[A \text{ and } B] = \Pr[A]\Pr[B \mid A]$. So $\Pr[b = 1 \text{ and } b' = 1] = \Pr[b = 1]\Pr[b' = 1 \mid b = 1] = \pi_1 \,\mathrm{TPR}$, by the definition of the TPR.
3. **Do the same in the other world.** $\Pr[b = 0 \text{ and } b' = 0] = \pi_0 \Pr[b' = 0 \mid b = 0]$. When the bit is 0 the attack says either 0 or 1, so $\Pr[b' = 0 \mid b = 0] = 1 - \Pr[b' = 1 \mid b = 0] = 1 - \mathrm{FPR}$.
4. **Add the two worlds.** This gives Equation 9.2.

A concrete count. Play 1,000 games with an attack whose TPR is 0.6 and FPR is 0.2. Table 9.2 counts the wins for a fair bit, $\pi_1 = \frac{1}{2}$, and for a bit that is 1 only one time in ten, $\pi_1 = 0.1$.

Table 9.2.: Expected wins in 1,000 games of an attack with TPR 0.6 and FPR 0.2. The win rate in each world is fixed by the attack; the prior decides how many games are played in each world.

Prior π_1	Games with $b = 1$	Wins there ($b' = 1$)	Games with $b = 0$	Wins there ($b' = 0$)	Wins in total
$\frac{1}{2}$	500	$0.6 \times 500 = 300$	500	$0.8 \times 500 = 400$	700, so $\Pr[b' = b] = 0.7$
0.1	100	$0.6 \times 100 = 60$	900	$0.8 \times 900 = 720$	780, so $\Pr[b' = b] = 0.78$

The formula gives the same numbers: $\frac{1}{2}(0.6) + \frac{1}{2}(0.8) = 0.7$ and $0.1(0.6) + 0.9(0.8) = 0.78$. The second row is a warning. The attack wins more often there, but only because most games are played in the world where saying 0 is easy. Always saying 0, with no attack at all, would win 900 of those games. Accuracy alone cannot tell a good attack from a good guess about the prior, which is why every game below is scored against a baseline.

Most evaluations use a **fair bit**, $\pi_1 = \frac{1}{2}$: half the targets are members and half are not, so that neither answer is right more often by default. In that game an adversary who never looks at its

view, and so has nothing to go on, wins exactly half the games, whatever it answers; a coin flip, "always say member" and "always say non-member" all score $\frac{1}{2}$. That is the sense of "coin flip" below: not a model of any attack, but the score of knowing nothing in a fair game. With a fair bit, Equation 9.2 becomes $\frac{1}{2}\mathrm{TPR} + \frac{1}{2}(1 - \mathrm{FPR}) = \frac{1}{2} + \frac{1}{2}(\mathrm{TPR} - \mathrm{FPR})$: the know-nothing score, plus half of the difference between the two rates. That difference is the **advantage**,

$$\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR} = 2\Pr[b' = b] - 1, \tag{9.4}$$

and it measures how much better the attack does than an adversary who ignores its view. The second equality is the fair-bit formula solved for TPR−FPR: from $\Pr[b' = b] = \frac{1}{2} + \frac{1}{2}(\mathrm{TPR} - \mathrm{FPR})$, subtract $\frac{1}{2}$ and multiply by 2.

Why does this number measure the gain over knowing nothing? Look at what it is: $\mathrm{TPR} - \mathrm{FPR}$ is how much more often the attack says "member" when the target is a member than when it is not. Three cases show what that means.

1. **An attack that ignores its view scores 0.** A coin flip, or "always say member", does not look at anything that differs between the two worlds, so it says 1 with the same probability in both. Its TPR equals its FPR, its advantage is 0, and by the fair-bit formula it wins with probability exactly $\frac{1}{2}$.
2. **A perfect attack scores 1.** It says 1 every time the target is a member and never otherwise: $\mathrm{TPR} = 1$, $\mathrm{FPR} = 0$, advantage 1, and it wins every game.
3. **Every attack in between is scored by its extra wins.** Rearranged, Equation 9.4 says $\mathrm{Adv} = 2(\Pr[b' = b] - \frac{1}{2})$: the amount by which the attack's win probability exceeds the know-nothing score $\frac{1}{2}$, doubled so that a perfect attack scores 1. An attack can only win more than half the time if its answer depends on which world it is in, and the advantage measures exactly how much it does.

A negative advantage means the attack says "member" more often for non-members; flipping its answers turns it into an attack with the same advantage, positive. In the first row of Table 9.2 the advantage is $0.6 - 0.2 = 0.4 = 2(0.7) - 1$: the attack wins 0.2 more often than a coin flip, and 0.2 doubled is 0.4. Unlike the win rate, the advantage does not depend on the prior, because TPR and FPR are each measured inside one world. That matters because real targets are rarely half members. With an unequal prior the know-nothing score is no longer $\frac{1}{2}$ but the larger of π_0 and π_1, which is 0.9 in the second row of Table 9.2, where always saying 0 wins 900 games of 1,000. The advantage needs no adjustment: an attack that ignores its view still has $\mathrm{TPR} = \mathrm{FPR}$ and advantage 0, at any prior. Every two-world game in this part is scored by its advantage, and the best possible guess with an unequal prior (Equation 9.10) starts from Equation 9.2. The same fact, in the language of security games, is Fact 1 of Cryptography basics.

9.2.2. Fact 2 — Processing the view cannot help the adversary

If the adversary throws information away, or adds noise of its own, it cannot do better. Formally, let $V' = g(V, R)$ for any function g and any randomness R independent of b. Then every attack that sees V' has the same TPR and FPR as some attack that sees V, so the best advantage from V', and the best TPR at every FPR, are at most those from V.

The reason is that whoever holds V can always produce V' themselves. Given an attack A' on V', build an attack A on V that draws R, computes $g(V, R)$ and runs A' on the result. In each world A's output has the same distribution as A''s, so it has the same TPR and FPR. For instance, if a service returns a confidence score s and you only look at whether $s > 0.5$, you can do no better than someone who looks at s itself, because they could apply the same cut-off.

This is the data processing inequality of Divergences, seen from the attacker's side. It is what orders the access levels of a threat model.

9.2.3. Fact 3 — Mixing hides, and knowing the mixture index undoes it

This fact compares adversaries who know different amounts, so it needs a number for "the best any attack can do against views P and Q". That number is the **total variation distance**. For views with probabilities $p(x)$ and $q(x)$, it is half the total absolute difference between the two probability tables,

$$\mathrm{TV}(P, Q) = \tfrac{1}{2} \sum_x |p(x) - q(x)|. \tag{9.5}$$

It is the best possible advantage for a simple reason. An attack that says "member" on a set A of views has $\mathrm{TPR} = Q(A)$ and $\mathrm{FPR} = P(A)$, so its advantage is $Q(A) - P(A) = \sum_{x \in A} (q(x) - p(x))$. That sum is largest when A contains exactly the views where $q(x) > p(x)$, the views that are more likely in the member world, and its largest value is the total excess of q over p, which is half the total absolute difference. Section 9.5 returns to this with the full error curve.

Now the fact itself. Often the adversary's view depends on something it does not know, such as the other records in the dataset. Not knowing it works like extra noise: the adversary sees an average of several possible situations, and averaging two distributions can only blur the difference between them. Knowing it removes that noise, and being allowed to choose it lets the adversary pick the situation where the difference is largest.

In symbols, call the unknown Θ, with weights w_ϑ, and let P_ϑ and Q_ϑ be the views in the two worlds when $\Theta = \vartheta$. An adversary who does not see Θ faces the averages $\bar{P} = \sum_\vartheta w_\vartheta P_\vartheta$ and $\bar{Q} = \sum_\vartheta w_\vartheta Q_\vartheta$. An adversary who sees it faces the pair (Θ, V). Their best advantages are ordered, from not knowing, to knowing, to choosing:

$$\mathrm{TV}(\bar{P}, \bar{Q}) \;\le\; \sum_\vartheta w_\vartheta \,\mathrm{TV}(P_\vartheta, Q_\vartheta) \;=\; \mathrm{TV}\big((\Theta, V)_{b=0}, (\Theta, V)_{b=1}\big) \;\le\; \max_\vartheta \mathrm{TV}(P_\vartheta, Q_\vartheta). \tag{9.6}$$

Each step has a short proof. The first inequality is the triangle inequality, $|a + b| \le |a| + |b|$, which says that differences can cancel when they are added up before taking absolute values, but not after: $\mathrm{TV}(\bar{P}, \bar{Q}) = \frac{1}{2} \sum_x \big| \sum_\vartheta w_\vartheta (p_\vartheta(x) - q_\vartheta(x)) \big| \le \frac{1}{2} \sum_x \sum_\vartheta w_\vartheta |p_\vartheta(x) - q_\vartheta(x)|$, and swapping the sums gives the middle term. The equality holds because the pair (Θ, V) has probability $w_\vartheta p_\vartheta(x)$ in world 0 and $w_\vartheta q_\vartheta(x)$ in world 1, so half the L^1 distance between the two worlds is exactly $\sum_\vartheta w_\vartheta \mathrm{TV}(P_\vartheta, Q_\vartheta)$. The last inequality says that an average is at most its largest term.

A tiny case shows how much the unknown can hide. Let there be one other record, with a bit $\vartheta \in \{0, 1\}$ that is 0 or 1 with probability $\frac{1}{2}$ each, and let the mechanism release the exact count $\vartheta + b$. An adversary who knows ϑ subtracts it and reads off b, so for each ϑ the TV is 1. An adversary who does not know it sees counts 0 or 1 in world 0 and 1 or 2 in world 1, each with

probability $\frac{1}{2}$. A count of 1 could come from either world, and the TV drops to $\frac{1}{2}$ (Exercise 7 in the exercise book). This fact is what separates the average-case and worst-case games, and it is how the chapter prices side information.

9.3. Threat models

9.3.1. Context

Papers on privacy attacks report numbers such as "membership advantage 0.2" or "90% of records reconstructed". Those numbers mean nothing until you know what the attacker was assumed to see, know and do. A threat model fixes these assumptions. This section lists what a threat model has to specify, and proves the one ordering that always holds among the choices: an adversary with more access is never weaker.

9.3.2. Statement

A threat model answers five questions.

1. **Goal: what is the secret?** Whether a given record was in the training data (membership), the value of a sensitive field of a known record (attribute), the records themselves (reconstruction), or a statistic of the dataset as a whole (property).
2. **Access: what does the adversary observe?** From least to most: only the model's decisions on inputs it chooses, such as a predicted class or an "anomaly" flag (**label-only**); the model's scores on those inputs, such as confidences, losses or log-probabilities (**black-box**); the parameters θ themselves (**white-box**); or everything the training process exposed along the way, such as intermediate models and per-step gradients (the **transcript**).
3. **Behaviour: does it follow the rules?** An **honest-but-curious** (semi-honest) adversary follows the protocol and only analyses what it legitimately sees. A **malicious** (active) adversary may deviate: choose its queries adaptively, insert crafted records into the training data (**poisoning**), or, if it runs part of the system, send tampered models to the other participants.
4. **Trust: who sees the raw data?** In the **central** model a trusted curator holds the data, and the adversary sees only what is released. In the **local** model nobody is trusted: each person's data leave their device already protected, and the adversary may be the collector itself. Distributed settings, where many parties each hold a slice, sit in between.
5. **Side information: what does it know already?** The population distribution, a public dataset drawn from it, some or all of the other records, the target's non-sensitive fields.

Of these, access is the one with a natural order, and the reason is Fact 2. Each level of access lets the adversary compute the level below it: from the weights it can compute any score, and from a score it can compute the label. So whatever an attack does with less access, an attack with more access can do too, and the best attack can only get better as access grows. Write $\mathrm{Adv}^*(V) = \max_A \mathrm{Adv}(A)$ for the best advantage of any attack whose input is the view V. For the same model and the same side information S,

$$\mathrm{Adv}^*(\text{labels}, S) \le \mathrm{Adv}^*(\text{scores}, S) \le \mathrm{Adv}^*(\theta, S) \le \mathrm{Adv}^*(\text{transcript}, S), \tag{9.7}$$

and the same chain holds for the best TPR at every fixed FPR. In the same way, more side information never lowers the best advantage, $\mathrm{Adv}^*(V) \leq \mathrm{Adv}^*(V, S)$, and neither does allowing malicious behaviour.

9.3.3. Derivation

1. **Each view can be computed from the next one.** A label is a function of the scores: the class with the largest score, or "score above a cut-off". A score on a query x is a function of θ and x, and the adversary chose x itself. The final θ is the last entry of the transcript.
2. **So Fact 2 applies at each link.** Each view on the left of Equation 9.7 is $g(V, R)$ for the view V to its right. Its achievable $(\mathrm{FPR}, \mathrm{TPR})$ pairs are therefore a subset of those to its right, and a maximum over a smaller set is smaller.
3. **Side information works the same way.** V is what you get from (V, S) by forgetting S, so Fact 2 applies again.
4. **Behaviour is a larger set of strategies.** An honest-but-curious adversary is a malicious one that chooses not to deviate, and a maximum over more strategies is at least as large.

9.3.4. Interpretation

The chain compares the *best* attacks at each level, not the attacks people actually run. A particular white-box attack can do worse than a well-designed black-box one, and the example below shows the gap inside a single access level can be larger than the gap between levels.

The access levels map directly onto deployments. Label-only and black-box access describe a model behind an application programming interface (API), such as a cloud prediction service or a chatbot. White-box access describes released weights, and also every participant who receives the global model in federated learning. The transcript describes a federated-learning server that sees each client's updates, the setting of Gradient inversion. Table 9.3 places four common deployments on all the axes. The same trained model can sit in several rows at once, and its risk is set by the weakest one.

Table 9.3.: Four deployments read as threat models.

Deployment	Access	Behaviour	Trust	Typical side information
Prediction service that returns the top class	label-only	honest-but-curious user, adaptive queries	central	public data from the same population
Service that returns confidences or log-probabilities	black-box	honest-but-curious user, adaptive queries	central	public data; the target's record
Model weights published for download	white-box	anyone; can run any computation	central	public data; possibly some training records

Deployment	Access	Behaviour	Trust	Typical side information
Federated training with a coordinating server	transcript (per-client updates)	server may be malicious	distributed	the global model at every round

Trust works differently from the other axes: it decides *who* the adversary is, not how strong it is. In the central model the threat is whoever sees the released model, and the curator is outside the threat model. In the local model the curator is inside it, so protection has to be applied before data leave the user. Part 3 compares the two.

Side information is the hardest axis to bound, because the defender usually does not know what the adversary knows. That is why the strongest assumption, "the adversary knows every record except the target's", is the one a guarantee should survive (Section 9.6).

9.3.5. Example: a centroid anomaly detector at four access levels

A security team fits the simplest anomaly detector there is: the mean $\theta = \frac{1}{n}\sum_i x_i$ of $n = 100$ records $x_i \sim \mathcal{N}(0, I_d)$ with $d = 50$ features. A point x gets the anomaly score $\|x - \theta\|^2$ and is flagged when the score exceeds $\tau = d$. The adversary holds a target z and plays the membership game: if $b = 1$, z is one of the n records; if $b = 0$, it is a fresh draw from the population. Here is what the adversary can do at each level of access.

With the weights (**white-box**), the adversary knows θ. The best thing it can do with θ is to ask how much more likely this exact θ is in the member world than in the non-member world. That ratio of the two probability densities is the **likelihood ratio**, and the Neyman–Pearson lemma of Hypothesis testing says that, for any target false positive rate, no attack catches more members than one that says "member" when the likelihood ratio is above a threshold. Intuitively, the views that are most typical of members relative to non-members are the ones to flag first.

Here the two densities are Gaussians. If z was used, $\theta = (z + S)/n$, where $S \sim \mathcal{N}(0, (n-1)I_d)$ is the sum of the other records, so θ is pulled a little towards z. If it was not, $\theta \sim \mathcal{N}(0, I_d/n)$. Expanding the squares in the two Gaussian densities gives the log of the likelihood ratio,

$$\ln \frac{q(\theta \mid z)}{p(\theta \mid z)} = \frac{n}{2(n-1)}(2\langle z, \theta\rangle - \|\theta\|^2) - \frac{\|z\|^2}{2(n-1)} + \text{const}, \tag{9.8}$$

and the best white-box attack thresholds it. The bracket $2\langle z, \theta\rangle - \|\theta\|^2$ is the part that depends on θ: it grows when θ points towards z, which is exactly what training on z does.

With only the score (**black-box**), the adversary gets $\|z - \theta\|^2 = \|z\|^2 - (2\langle z, \theta\rangle - \|\theta\|^2)$. It knows z, so the **calibrated score** $\|z\|^2 - \|z - \theta\|^2$ gives back exactly the bracket that the white-box likelihood ratio needs, and the remaining term $-\|z\|^2/(2(n-1))$ it can compute itself. The calibrated score drops that term, which only shifts the threshold from one target to the next, so it is almost, not exactly, the likelihood-ratio test: in Table 9.4 the two agree to four decimals. In this model the score carries everything about θ that matters for the decision (it is a **sufficient**

statistic: the likelihood ratio can be computed from it alone), so white-box access adds nothing. A simpler black-box attack ignores $\|z\|^2$ and says "member" when the raw score is low. That attack is confused by targets that are simply far from the mean: they have a high score whether or not they were used, so it mixes a target's typicality with its membership. With only the flag (**label-only**), the adversary says "member" when z is not flagged.

The script plays 100,000 games in each world and reports the best advantage over thresholds of each score (Table 9.4).

Table 9.4.: Membership advantage against the centroid detector with $n = 100$ and $d = 50$, from 100,000 games per world.

View	Attack	Best advantage TPR − FPR
White-box	likelihood ratio of θ	0.2789
Black-box	calibrated score $\|z\|^2 - \|z - \theta\|^2$	0.2789
Black-box	raw loss $-\|z - \theta\|^2$	0.0407
Label-only	not flagged, $\tau = d$	0.0400

The chain Equation 9.7 holds: $0.0400 \leq 0.2789 \leq 0.2789$. But the large gap is *inside* the black-box level, between two attacks on the same view. Calibrating each target against its own typicality multiplies the advantage by about seven. That is the idea behind the per-example attacks of Membership inference. Sablayrolles et al. (2019) prove a general version: if training draws the parameters with a probability that decreases exponentially in the training loss, the Bayes-optimal membership attack depends on the parameters only through the target's loss, and white-box access then adds little, as here.

Figure 9.1 shows the calibrated scores in the two worlds, and the advantage at each level as the dimension grows. With more parameters per record, a single record moves the model in more directions at once, and the best advantage climbs from 0.0326 at $d = 1$ to 0.7407 at $d = 500$ (100,000 games per world).

9.3.6. Check yourself

- A service adds a little random noise to every confidence score it returns. Can that make some attack stronger? *(No. The noisy score is $g(\text{score}, R)$ with R independent of b, so by Fact 2 every attack on it is matched by an attack on the clean score. Noise can only lower the best advantage, though it may not lower it by much.)*
- Why can a black-box attack beat a white-box attack in practice, despite Equation 9.7? *(The chain compares the best attack at each level. A particular white-box attack can be badly designed, just as the raw-loss attack is far worse than the calibrated one in Table 9.4.)*

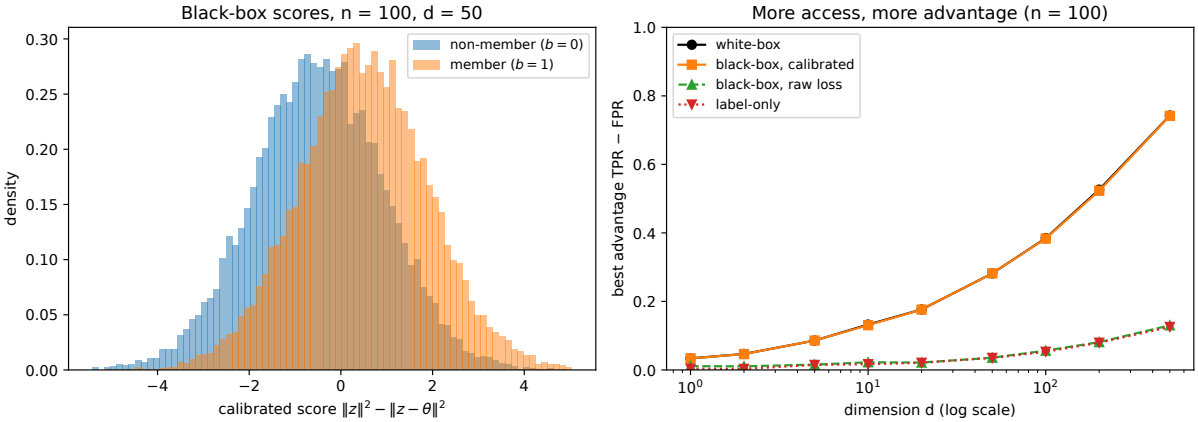


Figure 9.1.: Left: the calibrated black-box score $\|z\|^2 - \|z-\theta\|^2$ for members and non-members of the centroid detector, $n = 100$, $d = 50$. Right: the best advantage at each access level against the dimension d, from 20,000 games per world and point. White-box and calibrated black-box coincide; raw loss and label-only are far below.

9.4. Privacy games

9.4.1. Context

The same name, such as "membership inference", is used for attacks with different assumptions and different ways of scoring success. Salem et al. ("SoK: Let the Privacy Games Begin!", 2023) collect the attacks in the literature and write each one as a game between a challenger and an adversary. A game pins down three things: how the secret and the data are drawn, what the adversary sees, and how its guess is scored against a baseline. This section defines the games used in the rest of the part, and works out the best possible guess when the secret is a bit.

9.4.2. Statement

Every game in this part has the same four steps. The challenger draws a secret s from a known distribution, together with a dataset D whose distribution may depend on s. It trains $\theta = \mathcal{T}(D)$ and gives the adversary the view V that the threat model allows, along with the side information S. The adversary outputs a guess s' and earns a **gain** $g(s, s')$, usually $\mathbf{1}[s' = s]$. Finally, the attack is compared with a **baseline**: the best expected gain of a guess made from S alone, without looking at V. Figure 9.2 draws the membership game, and Table 9.5 lists the others.

Table 9.5.: The privacy games of this part. The first three are average-case: data come from the population. The distinguishing game is worst-case: the adversary picks the data.

Game	Secret s	Data in the two worlds	Gain	Baseline
Membership	bit b	$b = 1$: target z drawn from D; $b = 0$: z fresh from π	$\mathbf{1}[b' = b]$	$\frac{1}{2}$
Attribute	sensitive field a of a known member $z = (u, a)$	$z \in D$; adversary knows u	$\mathbf{1}[a' = a]$	best guess of a from u and π; or the same attack on non-members
Reconstruction	the records of D (or one column)	D drawn from π	fraction of entries recovered	guessing from π
Distinguishing	bit b	adversary chooses D_0, D_1 differing in one record; challenger trains on D_b	$\mathbf{1}[b' = b]$	$\frac{1}{2}$
Property	bit b	D drawn from π_{t_b}, populations differing in a statistic t	$\mathbf{1}[b' = b]$	$\frac{1}{2}$

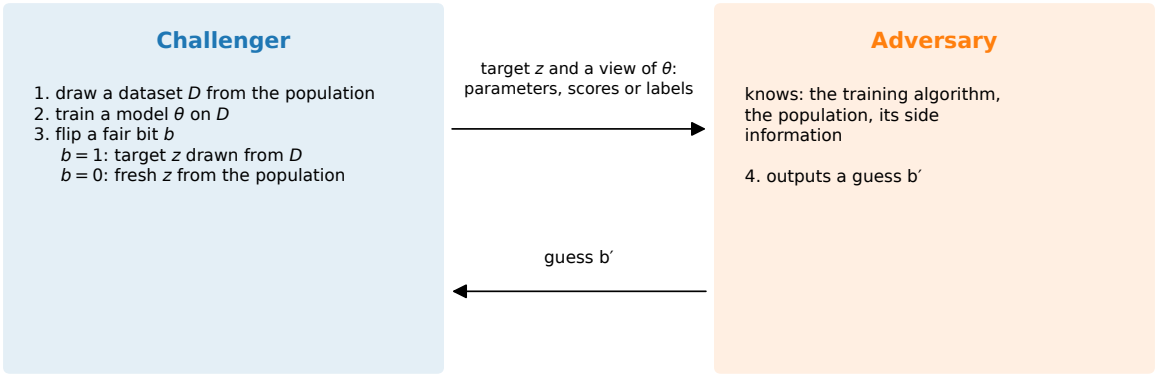


Figure 9.2.: The membership game. The challenger trains on a dataset, flips a bit that decides whether the target was in it, and shows the adversary the target and a view of the model. The adversary's guess is scored by its advantage.

When the secret is a bit that is not equally likely to be 0 or 1, the best possible guess has a closed form, and the idea behind it is simple. Say the bit is 1 only rarely, with prior $\pi_1 < \frac{1}{2}$. Then the safe default is to guess 0, which already wins with probability π_0. The view should overturn that

default only where it is strong enough evidence for 1 to outweigh the prior: where a view is more than $\gamma = \pi_0/\pi_1$ times as likely when the bit is 1 as when it is 0. With $\pi_1 = 0.1$ that is 9 times as likely. The best attack's gain over the default is the probability mass on those strongly convincing views.

The quantity that measures this mass is the **hockey-stick divergence**. With p and q the probabilities of a view when the bit is 0 and 1,

$$E_\gamma(Q\|P) = \sum_x \max\left(q(x) - \gamma p(x),\, 0\right), \tag{9.9}$$

the total by which q exceeds γ times p, counted only on the views where it does. At $\gamma = 1$ it is the total variation distance of Fact 3. The best attack wins with probability

$$\max_A \Pr[b' = b] = \pi_0 + \pi_1\, E_\gamma(Q\|P), \qquad \gamma = \frac{\pi_0}{\pi_1}. \tag{9.10}$$

The first term is the baseline of always guessing 0; the second is everything the view adds.

9.4.3. Derivation

The Bayes-optimal guess (Equation 9.10).

1. **Write an attack as a set.** A deterministic attack says 1 on some set A of views. By Equation 9.2 it wins with probability $\pi_1 Q(A) + \pi_0(1 - P(A))$.
2. **Pull out the baseline.** This equals $\pi_0 + \pi_1(Q(A) - \gamma P(A))$ with $\gamma = \pi_0/\pi_1$.
3. **Choose the best set.** $Q(A) - \gamma P(A) = \sum_{x\in A} (q(x) - \gamma p(x))$ is a sum over the views in A. To make it as large as possible, put a view in A exactly when its term is positive, that is when $q(x) > \gamma p(x)$. The maximum is then the sum of the positive terms, which is $E_\gamma(Q\|P)$. In words: say 1 when the likelihood ratio q/p exceeds the prior odds γ against the bit being 1. A randomized attack says 1 on view x with some probability $\phi(x)$ and gets $\sum_x \phi(x)(q(x) - \gamma p(x))$, which is also largest with $\phi = 1$ on the positive terms and 0 elsewhere, so randomizing does not help.
4. **Check the fair case.** With $\pi_1 = \frac{1}{2}$, $\gamma = 1$ and $E_1 = \mathrm{TV}$, so the best attack wins with probability $\frac{1}{2} + \frac{1}{2}\mathrm{TV}$, matching Fact 1 with the best advantage TV of Fact 3.

Reconstruction from one exact count. The secret is m uniform bits $x_1, \ldots, x_m$, and the release is their sum K.

1. **What the count says about each bit.** Given $K = k$, every arrangement of k ones among m places is equally likely, so each bit is 1 with probability k/m.
2. **The best guess for each bit.** Guess 1 when $k/m > \frac{1}{2}$ and 0 otherwise. Every bit gets the same guess, the majority value, and the expected number right is $\max(k, m-k)$.
3. **Average over the count.** With $K \sim \mathrm{Binomial}(m, \frac{1}{2})$, the expected fraction recovered is $\mathbb{E}[\max(K, m-K)]/m = \frac{1}{2} + \mathbb{E}|K - \frac{m}{2}|/m$.
4. **How large the gain is.** The count is a sum of m fair bits, each with variance $\frac{1}{4}$, so by the central limit theorem $K - \frac{m}{2}$ is approximately normal with mean 0 and variance $m/4$. The average distance of a normal variable from its mean is its standard deviation times $\sqrt{2/\pi}$ (integrate $|x|$ against the normal density), here $\frac{\sqrt{m}}{2}\sqrt{2/\pi}$. Dividing by m, the fraction recovered is approximately $\frac{1}{2} + \frac{1}{\sqrt{2\pi m}}$.

9.4.4. Interpretation

The baseline is part of the game, not an afterthought. A model that lets an attacker guess a rare diagnosis "with 90% accuracy" reveals nothing if 90% of people do not have that diagnosis. Accuracy only means something next to the baseline of Equation 9.10.

Attribute inference needs particular care, because it has two possible baselines that answer different questions. Comparing with a guess from the population asks "does the model help?". Comparing members with non-members asks "does the model help *because this person was in the training data*?". Only the second is a breach of that person's privacy; the first may be ordinary statistics that any study of the population would reveal. Attribute inference and model inversion develops this.

The games also differ in where the data come from. In the membership, attribute and reconstruction games the data are drawn from the population, so the score is an average over datasets and targets. In the distinguishing game the adversary chooses both datasets, so the score is a worst case. Section 9.6 shows that the first is never larger than the second, and how far apart they can be. Finally, games and threat models combine freely: every row of Table 9.5 can be played at every access level, so "label-only membership" and "white-box reconstruction" are both well defined.

9.4.5. Example: one noisy count, three games

A data team publishes how many people in a dataset have a sensitive condition, with Gaussian noise added: $Y = \sum_{x \in D} x + \sigma Z$, with bits $x \in \{0, 1\}$, $Z \sim \mathcal{N}(0, 1)$ and $\sigma = 1$. Suppose the adversary knows every record except the target's, so the rest of the count is a known number c. The same release can be attacked in three games.

In the **membership** game, the target has the condition. Without the target, $Y \sim \mathcal{N}(c, 1)$; with it, $Y \sim \mathcal{N}(c + 1, 1)$: the same bell curve, shifted right by one. The best attack says "member" on the views more likely in the member world, which are the counts to the right of the point where the two curves cross, $c + \frac{1}{2}$. Write Φ for the standard normal cumulative distribution function (CDF), so $\Phi(t)$ is the probability that a standard normal variable is below t. A member's count lands above $c + \frac{1}{2}$ with probability $\Phi(\frac{1}{2})$, its TPR, and a non-member's with probability $1 - \Phi(\frac{1}{2})$, its FPR. The advantage is $2\Phi(\frac{1}{2}) - 1$, and by Fact 1 the attack wins with probability $\frac{1}{2} + \frac{1}{2}(2\Phi(\frac{1}{2}) - 1) = \Phi(\frac{1}{2}) = 0.6915$.

In the **attribute** game with a fair prior, the target is known to be in D and its bit is the secret. The view has the same two distributions, so the best guess is again right with probability 0.6915 (0.6925 in 200,000 simulated games). Now make the condition rare, with prior $\pi_1 = 0.1$. Always guessing "no" already wins with probability 0.9, and by Equation 9.10 the release should overturn that only where the count is at least $\gamma = 9$ times as likely with the condition as without. For two unit-variance normals one apart, the likelihood ratio at y is $e^{(y-c)-1/2}$, which exceeds 9 when $y > c + \frac{1}{2} + \ln 9$. On those counts, Q puts mass $\Phi(\frac{1}{2} - \ln 9)$ and P puts mass $\Phi(-\frac{1}{2} - \ln 9)$, so $E_9 = \Phi(\frac{1}{2} - \ln 9) - 9\Phi(-\frac{1}{2} - \ln 9) = 0.0134$. The best accuracy is $0.9 + 0.1 \times 0.0134 = 0.9013$ (simulated: 0.9028). The same release that lifts a fair guess from 0.5 to 0.69 lifts this one by about a tenth of a percentage point, because the likelihood ratio is rarely large enough to overturn 9-to-1 odds.

In the **reconstruction** game, release the exact count of m uniform bits instead. The best reconstruction recovers a fraction 0.6230 of the bits for $m = 10$ (0.6236 in simulation), 0.5398 for $m = 100$ and 0.5126 for $m = 1000$. The approximation $\frac{1}{2} + 1/\sqrt{2\pi m}$ gives 0.6262, 0.5399 and 0.5126. One aggregate reveals little about each record, and less as m grows. Many aggregates are a different matter: Reconstruction attacks shows how a few times as many noisy subset counts as there are bits give away almost every bit.

9.4.6. Check yourself

- In the distinguishing game the adversary picks D_0 and D_1. Why does that make its best advantage at least the best advantage in the membership game, for the same mechanism? *(It can pick the pair that is hardest to hide, for example the dataset and target on which the mechanism leaks the most. The membership game averages over pairs drawn from the population, and a maximum is at least an average, as Section 9.6 makes precise.)*
- An attribute attack is right 75% of the time on members and 75% of the time on non-members. Is it a privacy attack on the members? *(No. It does equally well on people who were never in the data, so it is inferring the attribute from the population, not from the person's participation.)*

9.5. Advantage, total variation and the trade-off curve

9.5.1. Context

A two-world game is a test of $H_0 : V \sim P$ ("not a member") against $H_1 : V \sim Q$ ("member"). So everything Hypothesis testing proved about tests becomes a statement about attacks. This section draws out three: a ceiling on the advantage, a ceiling on the whole ROC curve, and a warning about which point of that curve to report.

9.5.2. Statement

Fact 3 already showed that no attack's advantage exceeds the total variation distance between the two worlds' views. The first bound restates that. The second goes further: it bounds the attack's TPR at *every* false positive rate, not just at the one point where TPR − FPR is largest. The bound comes from the **trade-off function** $T(P, Q)$: for each false positive rate α, $T(P, Q)(\alpha)$ is the smallest fraction of members that any attack with false positive rate α must miss. An attack can catch at most the rest. For every attack in a two-world game,

$$\text{Adv} = \text{TPR} - \text{FPR} \leq \text{TV}(P, Q), \qquad \Pr[b' = b] \leq \tfrac{1}{2} + \tfrac{1}{2}\text{TV}(P, Q), \tag{9.11}$$

$$\text{TPR} \leq 1 - T(P, Q)(\text{FPR}). \tag{9.12}$$

Both bounds are attained by likelihood-ratio attacks, which flag the views that are most typical of members relative to non-members first.

The third quantity answers the question a person who is flagged cares about: given that the attack says "member", how likely is that to be true? Of all the targets the attack flags, some are members (a fraction π_1 of targets are members, and it flags a fraction TPR of them) and some are non-members (a fraction π_0, of which it flags FPR). The **precision** is the members' share of the flagged:

$$\Pr[b = 1 \mid b' = 1] = \frac{\pi_1 \,\mathrm{TPR}}{\pi_1 \,\mathrm{TPR} + \pi_0 \,\mathrm{FPR}}. \tag{9.13}$$

9.5.3. Derivation

1. **The attack is a test.** It says "member", rejecting H_0, with probability $\phi(V)$. Its type I error is the FPR and its type II error is $1 - \mathrm{TPR}$.
2. **The advantage ceiling** (Equation 9.11). The advantage of an attack that says "member" with probability $\phi(x)$ on view x is $\sum_x \phi(x)(q(x) - p(x))$. It is largest when $\phi = 1$ exactly where $q > p$, that is, where the likelihood ratio q/p is above 1, and that largest value is $\mathrm{TV}(P, Q)$, as in Fact 3. The winning probability follows from Fact 1.
3. **The curve ceiling** (Equation 9.12). The fraction of members an attack misses is $1 - \mathrm{TPR}$. By the definition of the trade-off function, no attack with false positive rate FPR misses fewer than $T(P, Q)(\mathrm{FPR})$, so $1 - \mathrm{TPR} \geq T(P, Q)(\mathrm{FPR})$. Which attack attains it? The Neyman–Pearson lemma answers: with a budget of false positives to spend, spend it on the views where members are most over-represented, that is, flag views in decreasing order of the likelihood ratio q/p until the false positive rate reaches the budget. Each view flagged that way buys the most members per false positive.
4. **Precision** (Equation 9.13). The precision is a conditional probability, so it is the probability that a target is a member *and* flagged, divided by the probability that it is flagged. The first is $\pi_1 \mathrm{TPR}$, as in step 2 of Fact 1. A target is flagged either as a member, $\pi_1 \mathrm{TPR}$, or as a non-member, $\pi_0 \mathrm{FPR}$, and these are disjoint, so the second is their sum.

9.5.4. Interpretation

The advantage is only one point of the ROC curve, the plot of an attack's TPR against its FPR as its threshold varies. Because the advantage is $\mathrm{TPR} - \mathrm{FPR}$, it is the height of that curve above the diagonal, and TV is the largest such height, reached at the likelihood-ratio threshold 1, and it says nothing about what happens at the low false-positive end. Yet that end is where privacy breaks. An attack that flags 1% of members with no false alarms has an advantage of only about 0.01, yet it identifies those members with certainty. An attack with advantage 0.1, right on 55% of members and wrong on 45% of non-members, identifies nobody with confidence. Carlini et al. (2022) argue from this that membership attacks should be reported by their TPR at a low FPR, such as 10^{-3}, on ROC curves with log axes. The rest of this part does so.

Precision tells the same story from the attacker's side. If only one target in a hundred is a member, even a good attack produces mostly false alarms (Jayaraman et al. 2021). A low FPR is what keeps precision high when members are rare.

The two ceilings depend only on P and Q, so they bound every attack, including ones nobody has invented yet. That is what makes them the right target for a defence: a mechanism whose trade-off curve $T(P, Q)$ stays close to the diagonal for every target defeats every attack at once.

9.5.5. Example: the noisy count, worst case

Take the noisy count of the previous example, $\sigma = 1$, with the other $m = 100$ bits known (the script fixes them at 50 ones). Then $P = \mathcal{N}(50, 1)$, $Q = \mathcal{N}(51, 1)$, and as computed there the best attack says "member" when $Y > 50.5$, the point where the two curves cross, with advantage $\mathrm{TV} = 2\Phi(\frac{1}{2}) - 1 = 0.3829$. Over 200,000 games in each world it scores TPR $= 0.6901$ and FPR $= 0.3092$: advantage 0.3809 and winning probability 0.6905, against the ceilings 0.3829 and 0.6915.

Now hold the false positive rate at 10^{-3}. The attack still flags large counts, but it has to move its threshold t up until only one non-member in a thousand exceeds it: $\Pr[\mathcal{N}(50, 1) > t] = 10^{-3}$, so $t = 50 + \Phi^{-1}(0.999) = 50 + 3.0902 = 53.09$, where Φ^{-1} is the inverse of Φ. A member's count, centred one higher, exceeds t with probability $1 - \Phi(t - 51) = 1 - \Phi(3.0902 - 1) = 0.0183$. That is the TPR, and by the Neyman–Pearson argument above it is the best possible at this FPR (simulated: 0.0184, at an FPR of 0.00109). The attack now catches fewer than 2% of members, but when it fires it is right with precision $0.0183/(0.0183 + 0.001) = 0.948$ at a fair prior. If only one target in a hundred is a member, the precision drops to 0.156.

The black curve in Figure 9.3 is this ROC curve on log axes. The green one is the same game when the adversary does not know the other records, which is the subject of the next section.

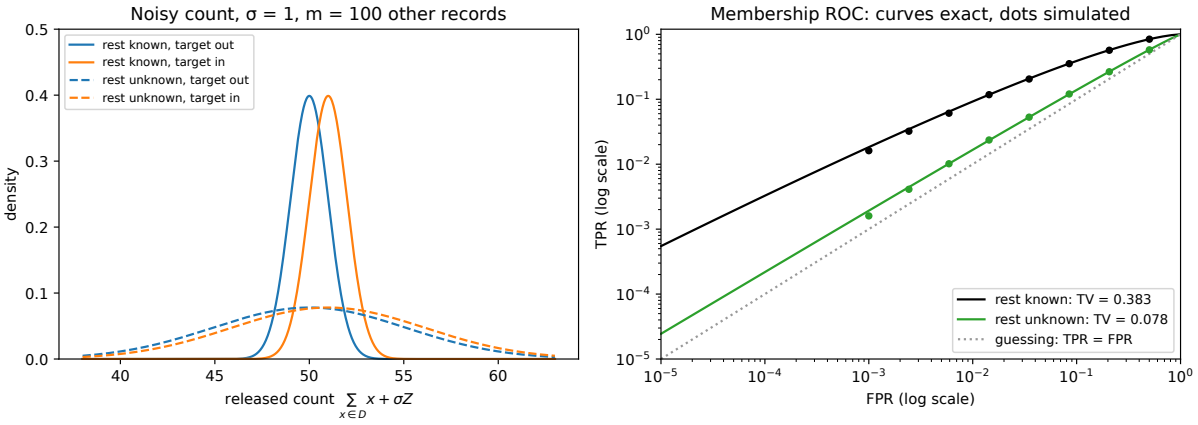


Figure 9.3.: Left: the released count in the two worlds when the other 100 records are known (solid) and unknown (dashed). Right: the membership ROC curves of the best attacks on log axes, exact (lines) and from 100,000 simulated games per world (dots). The dotted diagonal is random guessing.

9.5.6. Check yourself

- Attack A has TPR 0.5 at FPR 0.3; attack B has TPR 0.02 at FPR 10^{-4}. Which has the larger advantage, and which is the bigger privacy threat? *(A has advantage 0.2, B*

about 0.02. B is the bigger threat to the people it flags: its precision at a fair prior is $0.02/(0.02 + 0.0001) = 0.995$*, against* $0.5/0.8 = 0.625$ *for A.)*

- Why can no attack, however clever, beat Equation 9.12? *(The bound is a property of the two view distributions. Any attack is a test of P against Q, and the Neyman–Pearson lemma says no test with that FPR has more power than the likelihood-ratio test.)*

9.6. Average case and worst case

9.6.1. Context

A membership game can be scored in two ways. In the **average case**, the data and the target come from the population, the adversary knows only the population, and the advantage is an average over all of that randomness. In the **worst case**, the adversary knows, or even picks, the rest of the data and the target, and the advantage is the maximum over those choices. Evaluations of real models usually measure the first; privacy guarantees have to control the second. This section shows how far apart the two can be.

9.6.2. Statement

The comparison is Fact 3 again, with the unknown being the rest of the game. Let ϑ collect everything that the average-case game draws at random and that a worst-case adversary is allowed to fix: the other records, and the target. An average-case adversary does not know ϑ, so it sees an average over its possible values, which blurs the two worlds together. An adversary who knows ϑ sees each situation sharply, and one who chooses ϑ picks the sharpest. Let P_ϑ, Q_ϑ be the views given ϑ, and $\bar{P}, \bar{Q}$ their averages. Then, by Fact 3,

$$\underbrace{\mathrm{TV}(\bar{P}, \bar{Q})}_{\text{adversary does not know } \vartheta} \;\le\; \underbrace{\mathbb{E}_\vartheta\, \mathrm{TV}(P_\vartheta, Q_\vartheta)}_{\text{adversary knows } \vartheta} \;\le\; \underbrace{\max_\vartheta\, \mathrm{TV}(P_\vartheta, Q_\vartheta)}_{\text{adversary chooses } \vartheta}. \tag{9.14}$$

Both gaps can be as large as possible: there are mechanisms for which the left side is close to 0 and the right side is 1.

9.6.3. Derivation

1. **Three adversaries, three views.** An adversary that does not know ϑ sees V, distributed as the average $\bar{P}$ or $\bar{Q}$, so its best advantage is $\mathrm{TV}(\bar{P}, \bar{Q})$ by Equation 9.11. One that knows ϑ sees the pair (ϑ, V), and by Fact 3 its best advantage is $\mathbb{E}_\vartheta \mathrm{TV}(P_\vartheta, Q_\vartheta)$. One that chooses ϑ picks the most revealing value, which gives the maximum.
2. **The inequalities** are the two inequalities of Equation 9.6.
3. **The gaps can be extreme.** For the left gap, release the exact count, with the target's bit 1 and m other bits the adversary does not know. An adversary who knew the others would read b off the count, so the middle term is 1. One who does not know them sees the count K in world 0 and $K + 1$ in world 1, where $K \sim \mathrm{Binomial}(m, \frac{1}{2})$ is the unknown sum of the others. The two distributions are the same list of binomial weights $w_k = \Pr[K = k]$,

the second shifted one place to the right. Their total variation is the total excess of the shifted list over the original, counted where it is positive. The binomial weights rise up to the middle value $\lfloor m/2 \rfloor$ and fall after it, so the shifted list is the larger one exactly to the right of the middle, and there its excess at k is $w_{k-1} - w_k$. Adding these differences from the middle to the end, every weight cancels except the first, so the total is the largest weight, $\binom{m}{\lfloor m/2 \rfloor} 2^{-m}$. For large m the binomial is close to a normal curve with standard deviation $\sqrt{m}/2$, whose peak height is $1/(\sqrt{2\pi} \cdot \sqrt{m}/2) = \sqrt{2/(\pi m)}$ (Exercise 2 in the exercise book works this out). It tends to 0 as m grows. For the right gap, release one record of D chosen at random, worked out in the example below.

9.6.4. Interpretation

Averaging over datasets hides the target in a crowd, but only from an adversary who does not know the crowd. In the noisy count, the unknown other records act as extra noise with variance $m/4$. A defence that relies on that noise fails against an adversary who knows the other records, or who planted them, and inserting records the adversary controls is exactly what a malicious adversary of Section 9.3 can do.

Averaging over targets hides the worst-treated person. If a mechanism exposes one person in a thousand completely and everyone else not at all, its average advantage is about 0.001. Privacy is a promise to each person, so the number that matters is the advantage against that one person. Outliers and duplicated records are the usual victims, as Membership inference and Memorization and training-data extraction show.

Average-case numbers are still useful. They measure the risk that a typical record faces from a realistic attacker, and they are what an audit can estimate from data. But they are lower bounds on the worst case, never upper bounds. An audit narrows the gap by playing the adversary that chooses the target: it plants **canaries**, records built to be as exposed as possible, and measures the attack on those (Memorization and training-data extraction).

This is where the part is heading. A guarantee that holds for every pair of neighbouring datasets bounds the right-hand side of Equation 9.14, and therefore every membership-style game in this part at once, every game whose two worlds differ in one record. A game whose two worlds differ in many records, such as the property games of Property inference, is bounded only through a weaker guarantee for groups of records, which degrades with the size of the group. That is how the definition in Part 3 is designed.

9.6.5. Example: the noisy count and the random-record release

The noisy count. Keep $\sigma = 1$ and the target's bit 1, but now let the $m = 100$ other bits be fresh Bernoulli($\frac{1}{2}$) draws that the adversary does not see. The count is then $K + b + Z$ with $K \sim \text{Binomial}(100, \frac{1}{2})$. In the two worlds the count has the same distribution, shifted by 1, and that distribution is log-concave, because a binomial and a normal are both log-concave and so is their sum. A shifted log-concave density has an increasing likelihood ratio, so the best attack is still a threshold, and by symmetry the two densities cross at $m/2 + \frac{1}{2} = 50.5$. Table 9.6 compares the adversaries of Equation 9.14, and adds the case where the target's own bit is random.

Table 9.6.: Membership advantage on a noisy count with 100 other records, for adversaries with different knowledge.

Adversary	Best advantage (exact)	Simulated (200,000 games per world)
Knows the other records ($\sigma = 1$)	0.3829	0.3809
Does not know them, target's bit 1	0.0779	0.0751
Does not know them, target's bit random	0.0390	0.0398
Knows the other records, exact count ($\sigma = 0$)	1	—
Does not know them, exact count ($\sigma = 0$)	0.0796	—

Not knowing the crowd cuts the advantage by a factor of five. A normal approximation explains the number: treating K as $\mathcal{N}(50, 25)$ makes the two worlds Gaussians with shift 1 and standard deviation $\sqrt{25+1}$, which gives $2\Phi(1/(2\sqrt{26})) - 1 = 0.0781$. When the target's own bit is random, the count does not depend on b for the half of targets whose bit is 0, and the advantage halves to $0.0779/2 = 0.0390$. At an FPR of 10^{-3} the average-case attack has TPR 0.0019 (simulated 0.0016), against 0.0183 for the adversary who knows the rest; these are the green and black curves of Figure 9.3. Figure 9.4 shows the average-case advantage falling as the crowd grows, while the worst case does not move.

The random-record release. A mechanism publishes one record of D, chosen uniformly among $n = 1000$ records that are all different. The attack says "member" if the published record is z. If z is in D, it is published with probability $1/n$, so TPR $= 0.001$; if not, it is never published, so FPR $= 0$. The script measures TPR $= 0.00098$ and FPR $= 0$ over 200,000 games per world. By the average-case measure, an advantage of $1/n = 0.001$, the mechanism looks almost perfectly safe. Yet every run exposes one person completely, and when the attack fires its precision is 1. Against the target that is published, the advantage is 1. Only the right side of Equation 9.14, which fixes the dataset and asks about the worst target, reveals the problem.

9.6.6. Check yourself

- A team audits its model by running a membership attack on 10,000 random training and held-out examples and reports advantage 0.01. What can you conclude about the worst-case risk? *(Only that it is at least 0.01. The audit averages over targets and datasets. A small group of outliers could be exposed far more, and the average would barely show it.)*
- In the noisy count with unknown others, how could a malicious adversary recover the worst-case advantage? *(By controlling the other records, for example by flooding the dataset with records whose bits it knows. Then the rest of the count is known, and the advantage goes back to the 0.3829 of the first row of Table 9.6.)*

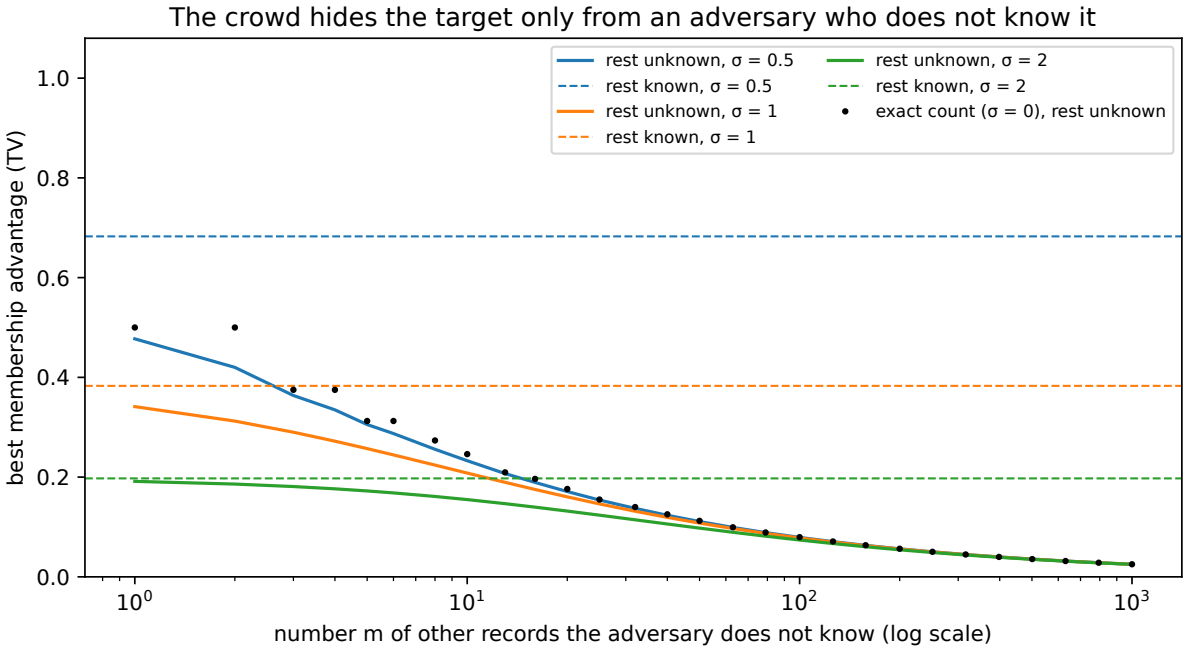


Figure 9.4.: Best membership advantage on the noisy count against the number m of other records the adversary does not know, for three noise levels (solid), and the same mechanism against an adversary who knows them (dashed). Dots: the exact count, $\sigma = 0$, whose worst case is 1.

9.7. Code: check every number by computation and simulation

The script code/threat_models.py computes every number in this chapter and draws every figure. Exact values come from closed forms (normal tails via `math.erfc`, binomial weights) and are checked against games played by simulation. From the repository root:

```
uv run python code/part02-attacks/threat_models.py
```

Output (seed 0), summarized in Table 9.7:

Table 9.7.: The script's output with the default seed.

Experiment	Result
Centroid detector, $n = 100$, $d = 50$	Best advantage: white-box 0.2789; calibrated black-box 0.2789; raw loss 0.0407; label-only 0.0400
Noisy count, rest known, $\sigma = 1$	TV 0.3829; simulated TPR 0.6901, FPR 0.3092, advantage 0.3809, winning probability 0.6905
Same, at FPR 10^{-3}	TPR 0.0183 (simulated 0.0184); precision 0.948 at prior $\frac{1}{2}$, 0.156 at prior 0.01

Experiment	Result
Noisy count, 100 unknown others	TV 0.0779 (normal approximation 0.0781; simulated 0.0751); TPR at FPR 10^{-3} 0.0019 (simulated 0.0016)
Same, random target bit	Advantage 0.0390 (simulated 0.0398)
Exact count, 100 unknown others	TV 0.0796; $\sqrt{2/(\pi m)} = 0.0798$
Attribute from the noisy count	Prior $\frac{1}{2}$: 0.6915 (simulated 0.6925). Prior 0.1: baseline 0.9, best 0.9013, $E_9 = 0.0134$ (simulated 0.9028)
Reconstruction from one exact count	$m = 10$: 0.6230 (simulated 0.6236); $m = 100$: 0.5398; $m = 1000$: 0.5126
Random-record release, $n = 1000$	Simulated TPR 0.00098, FPR 0; $1/n = 0.001$

tests/test_threat_models.py checks that the output still matches.

Four things stand out. First, the ceilings are reached: every simulated likelihood-ratio membership attack lands within 0.003 of its TV ceiling, within two standard errors for 200,000 games per world, and the small differences in either direction are sampling error. Second, access is not the only lever: in the centroid example the biggest gap is between two attacks on the same black-box view, not between access levels. Third, knowledge changes everything: the same noisy count has advantage 0.38 against an adversary who knows the other records and 0.08 against one who does not, and the exact count goes from 1 to 0.08. Fourth, one number is not enough: advantage, TPR at a low FPR, precision and accuracy against a baseline each answer a different question, and they can disagree by orders of magnitude.

9.8. References

- Salem, Cherubin, Evans, Köpf, Paverd, Suri, Tople & Zanella-Béguelin, "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023). The source of the game-based view of this chapter: membership, attribute, reconstruction and property inference written as games, with the relations between them. Read §§3–5.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018). The membership and attribute games with advantage TPR − FPR, and the loss-threshold attack.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022). Why attacks should be reported by TPR at low FPR on log-scaled ROC curves; the attack of Membership inference.
- Sablayrolles, Douze, Schmid, Ollivier & Jégou, "White-box vs Black-box: Bayes Optimal Strategies for Membership Inference" (ICML 2019). Conditions under which the optimal attack depends only on the loss, so white-box access does not help.
- Jayaraman, Wang, Knipmeyer, Gu & Evans, "Revisiting Membership Inference Under Realistic Assumptions" (PETS 2021). Membership inference when members are rare: priors, precision and threshold choice.

- Choquette-Choo, Tramèr, Carlini & Papernot, "Label-Only Membership Inference Attacks" (ICML 2021). Attacks that see only predicted labels, and how close they come to score-based ones.
- Nasr, Shokri & Houmansadr, "Comprehensive Privacy Analysis of Deep Learning: Passive and Active White-box Inference Attacks against Centralized and Federated Learning" (IEEE S&P 2019). A taxonomy of white-box and federated adversaries, honest-but-curious and active.
- Boneh & Shoup, *A Graduate Course in Applied Cryptography*, ch. 2–3: security games, advantage and the challenger–adversary vocabulary this chapter borrows.

9.9. Cheat sheet

P and Q are the adversary's view without and with the target ($b = 0$ and $b = 1$); π_1 is the prior of $b = 1$. Table 9.8 collects the results.

Table 9.8.: The results of this chapter.

Result	Statement	Where
Winning and advantage	$\Pr[b' = b] = \pi_1\mathrm{TPR} + \pi_0(1 - \mathrm{FPR})$; fair prior: $\frac{1}{2} + \frac{1}{2}\mathrm{Adv}$, $\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR}$	Equation 9.2, Equation 9.4
Access chain	labels $\leq$ scores $\leq$ parameters $\leq$ transcript, for the best attack	Equation 9.7
Ceiling on advantage	$\mathrm{Adv} \leq \mathrm{TV}(P, Q)$	Equation 9.11
Ceiling on the ROC curve	$\mathrm{TPR} \leq 1 - T(P, Q)(\mathrm{FPR})$	Equation 9.12
Bayes-optimal guess	$\max \Pr[b' = b] = \pi_0 + \pi_1 E_{\pi_0/\pi_1}(Q\|P)$ for $\pi_1 \leq \frac{1}{2}$	Equation 9.10
Precision	$\pi_1\mathrm{TPR}/(\pi_1\mathrm{TPR} + \pi_0\mathrm{FPR})$	Equation 9.13
Average against worst case	$\mathrm{TV}(\bar{P}, \bar{Q}) \leq \mathbb{E}_\vartheta \mathrm{TV}(P_\vartheta, Q_\vartheta) \leq \max_\vartheta \mathrm{TV}(P_\vartheta, Q_\vartheta)$	Equation 9.14
Noisy count, shift 1, noise σ	Rest known: $2\Phi(\frac{1}{2\sigma}) - 1$. m unknown others: $\approx 2\Phi(1/(2\sqrt{m/4 + \sigma^2})) - 1$	Table 9.6

Rules of thumb

- Before reading an attack's number, read its threat model: access, behaviour, trust, side information, and the game.
- Compare every attack with its baseline. Accuracy near the prior is no attack.
- Report TPR at a low FPR, with the ROC curve on log axes; advantage and accuracy hide the cases that matter.

- An average-case number is a lower bound on the worst case. A defence has to hold for the worst record, the worst dataset and the best-informed adversary.
- A better attack on the same view can matter more than more access: calibrate scores per target.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

10. Membership inference

10.1. Why this matters

A hospital trains a model to predict a disease and publishes it. You hold one patient's record. **Was that record in the training set?** If it was, you have learned that the patient was treated at that hospital for that disease, and the model never output a diagnosis. This is **membership inference**, the simplest question an attacker can ask about a trained model and the one asked most often. A membership inference attack is also the standard way to measure how much a model leaks: if an attacker cannot even tell whether a record was used, it can do little else with it. The reverse does not hold. A model that leaks membership may leak more, and the later chapters of this part show how.

An attack is scored by two rates. Its **true positive rate (TPR)** is the fraction of members it flags as members, and its **false positive rate (FPR)** is the fraction of non-members it wrongly flags. A useful attack flags many members while flagging few non-members. With these two numbers in hand, the chapter answers four questions, each building on the one before.

The first is why a model leaks membership at all. The answer is that it fits its training data better than other data, so its loss on a record it trained on is usually lower than on a fresh one. The simplest attack says "member" when the loss is low, and its advantage, the TPR minus the FPR, turns out to equal the generalization gap: the difference between the model's average loss on fresh data and on its training data. The second question is how an attacker can learn what members look like when it cannot see the training data. It trains **shadow models** of its own, on data where it knows who is a member, and watches how they behave. The third is what the best attack is. It is a likelihood-ratio test run separately for each record: the **likelihood ratio attack (LiRA)** estimates, for each record, how the model's output on that record is distributed when the record was trained on and when it was not, and asks which of the two the observed output fits better. The fourth is how an attack should be scored and who is at risk. Attacks should be compared by their TPR at a low FPR, because only there does a "member" verdict carry confidence, and the people an attack identifies at that end are almost all outliers.

10.1.1. The one idea: membership inference is a test about one record

Fix a target record z. Train a model on a dataset that contains z, and look at the model's output on z. Train it again on a dataset that does not, and look again. The two outputs will not repeat exactly if you retrain, because training is random: the rest of the data, the initialization and the order of the examples all change. So each case gives a *distribution* of outputs, the **IN** distribution and the **OUT** distribution of z. The attacker sees one output and must say which distribution it came from.

That is a test between two fully specified hypotheses, and the **Neyman–Pearson lemma** says how to do it best. Compute the **likelihood ratio**, the density of the observed output under IN divided by its density under OUT, and say "member" when it is above a threshold. For any allowed FPR, no test flags more members. The reason is a budgeting argument. Each possible output costs its OUT probability in false positives if you call it "member", and earns its IN probability in true positives. With a fixed false-positive budget, you earn the most by calling "member" on the outputs with the best ratio of earnings to cost first, and that ratio is the likelihood ratio.

The IN and OUT distributions differ from record to record. A typical record is predicted well whether or not it was in the training set, so its two distributions nearly coincide. An atypical record is predicted badly unless the model saw it, so its two distributions are far apart. Every result in this chapter follows from taking that seriously. The loss attack uses one threshold for every record, so it confuses "easy record" with "member". Shadow models are how the attacker estimates the two distributions, record by record. LiRA fits them per record, and once you account for the record's identity it is the Neyman–Pearson test for the whole game. And the low-FPR end of the curve is made of the records whose two distributions are far apart, which are the outliers.

Table 10.1 lists the results and where each is used.

Table 10.1.: The results of this chapter and where each is used.

Result	Where it is used
Loss attack: advantage = generalization gap $/B$	Why overfit models leak; a lower bound on leakage from train and test error alone
Shadow models	Every practical attack that needs to know what members look like
Shadow attack model estimates the membership posterior	Why an attack model trained on shadows is optimal among attacks that ignore the record's identity
LiRA (online and offline) as a per-example likelihood-ratio test	The strongest black-box attack; the attack used to audit trained models
One threshold on the per-example likelihood ratio is optimal	Why calibrating per example beats any global threshold
TPR at a low FPR on log-scaled receiver operating characteristic (ROC) curves	Reporting any privacy attack honestly
Per-example separation and self-influence	Predicting which records are exposed; why outliers are

10.1.2. The running example

Every number in this chapter comes from one experiment, small enough to run in seconds and rich enough to show every effect. It has a **pool** of 1,000 labelled points in $\mathbb{R}^{100}$, of two kinds, built so that some points are easy to predict and a few are not.

The 950 **typical points** have features $x \sim \mathcal{N}(0, I/100)$: each of the 100 coordinates is normal with variance $1/100$, so the squared length $\|x\|^2$ is a sum of 100 terms that average $1/100$

each, and $\|x\| \approx 1$. Each gets a label $y = \pm 1$ that follows its first coordinate with some noise: $\Pr[y = 1 \mid x] = \sigma(30x_1)$, where $\sigma(t) = 1/(1 + e^{-t})$ is the logistic function, which turns any real number into a probability. The 50 **outliers** have the same kind of x scaled by 5, so they are five times longer, and a label drawn by a fair coin. Nothing else in the data predicts an outlier's label, so a model can only get it right by fitting that point itself.

Each **model** is L_2-regularized logistic regression: it chooses the weight vector w that minimizes the average logistic loss on its training points plus a penalty $\frac{\lambda}{2}\|w\|^2$ that keeps the weights small, with $\lambda = 10^{-3}$. Each model trains on a random half of the pool ($n = 500$). Models come in complementary pairs: one trains on a random half, the next on the other half, so every point is a member of exactly half the models. The attacker targets 128 such models. It also trains 64 **shadow models** of its own in the same way. The attacker knows the pool and the training recipe but not which half any target model used. Every target model can be asked about all 1,000 pool points, and each (target model, point) pair is one game: 500 of the points are members of that model and 500 are not. Over 128 target models, that is $128 \times 500 = 64{,}000$ member pairs and 64,000 non-member pairs.

This is the setup of Carlini et al. (2022), scaled down from an image classifier to a linear model. Section 10.8 describes the full-size pipeline.

What this chapter assumes. The membership game, the advantage $\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR}$ and its ceiling, and the difference between average-case and worst-case games are in Threat models and privacy games. The Neyman–Pearson lemma, trade-off curves, ROC curves, the Bayesian view and confidence intervals for rates are in Hypothesis testing. The generalization gap and the self-influence of a training example are in Algorithmic stability and generalization. Each is restated below where it is used.

Conventions. As in Threat models and privacy games, $b = 1$ is the world in which the target is a member (the alternative hypothesis) and $b = 0$ the world in which it is not (the null). A model is θ, with weight vector w. A record is $z = (x, y)$, a feature vector and its label. An attack outputs a score and says "member" when the score exceeds a threshold. Its TPR is the fraction of members it flags and its FPR the fraction of non-members it flags. Logarithms are natural.

10.2. Toolkit: three facts every derivation uses

10.2.1. Fact 1 — The membership game and its advantage

Membership inference is scored as a game, so that "the attack works" has a precise meaning. The challenger draws a training set and flips a fair coin to get a hidden bit b. The coin decides the target z: if $b = 1$, z is a member of the training set; if $b = 0$, it is a fresh record from the same population. The challenger trains θ and gives the adversary z and query access to θ. The adversary guesses b', 1 for "member" and 0 for "not a member".

The coin is fair so that neither answer is right more often by default: an adversary that ignores the model and always says "member", or flips a coin of its own, wins exactly half the games. Any win rate above $\frac{1}{2}$ must come from what the model reveals. $\Pr[b' = b]$ is the probability that the guess is correct, whichever value the bit has; it is not the probability that b is 1.

To compute it, split the games by the value of the bit. Half the games have $b = 1$, and the adversary wins those when it says "member", which it does with probability TPR. The other half have $b = 0$, and it wins those when it says "not a member", with probability $1 - \text{FPR}$. Adding the two halves gives $\frac{1}{2}\text{TPR} + \frac{1}{2}(1 - \text{FPR})$, which rearranges to

$$\Pr[b' = b] = \tfrac{1}{2} + \tfrac{1}{2}\text{Adv}, \qquad \text{Adv} = \text{TPR} - \text{FPR}. \tag{10.1}$$

The **advantage** is how much more often the attack says "member" to members than to non-members. An attack that ignores the model says "member" equally often in both worlds and scores 0; a perfect attack scores 1. This is Fact 1 of Threat models and privacy games.

In the running example, "pick a target model and a pool point" is this game with the pool as the population: the point is a member of that model's training set with probability $\frac{1}{2}$, because each model trains on half the pool. The loss attack, the shadow attack and the comparison of attacks by advantage all use this fact.

10.2.2. Fact 2 — The best test of two Gaussians, at any FPR

Most of the chapter reduces to one test: an observation X is a standard normal variable in one world and the same bell curve shifted right by s in the other. Which world produced it? Write $H_0 : X \sim \mathcal{N}(0, 1)$ and $H_1 : X \sim \mathcal{N}(s, 1)$, with $s > 0$. The number s is the **separation**: the distance between the two means in units of the standard deviation. A large s means the two bell curves barely overlap.

By the Neyman–Pearson lemma, the best test thresholds the likelihood ratio. Here the log of the ratio of the two normal densities is

$$\ln \frac{e^{-(X-s)^2/2}}{e^{-X^2/2}} = \frac{X^2 - (X-s)^2}{2} = sX - \frac{s^2}{2}, \tag{10.2}$$

which grows with X. So "the likelihood ratio is large" is the same as "X is large", and the best test says "H_1" when X exceeds some cut-off.

The cut-off is set by the FPR you allow. Write Φ for the standard normal cumulative distribution function: $\Phi(t)$ is the probability that a standard normal variable is below t, and Φ^{-1} is its inverse. To flag a fraction α of H_0 draws, the cut-off must be the point that a standard normal exceeds with probability α, which is $\Phi^{-1}(1-\alpha)$. Under H_1, $X - s$ is standard normal, so X passes the cut-off with probability $1 - \Phi\big(\Phi^{-1}(1-\alpha) - s\big)$. That is the TPR:

$$\text{TPR}(\alpha) = 1 - \Phi\big(\Phi^{-1}(1-\alpha) - s\big). \tag{10.3}$$

This is the Gaussian trade-off curve of Hypothesis testing, written as TPR against FPR: the most members any test can catch, at each FPR.

To get a feel for it, take $\alpha = 10^{-3}$, so the cut-off is $\Phi^{-1}(1-\alpha) = 3.0902$: a standard normal exceeds 3.0902 one time in a thousand. With a separation of 0.5, a member's X must still clear $3.0902 - 0.5 = 2.59$ standard deviations above its own mean, so $\text{TPR} = 1 - \Phi(2.59) = 0.0048$, barely five times the chance rate of 0.001. With a separation of 4, the cut-off sits $3.0902 - 4 = -0.91$ standard deviations from the member mean, below it, so most members pass: $1 - \Phi(-0.91) = 0.8185$. LiRA's Gaussian fits, the two-type example and the exposure of outliers all use this formula.

10.2.3. Fact 3 — For logistic regression, the logit of the confidence is the margin

An attack has to pick some number from the model's output to look at. Three natural choices are the **confidence** p_y, the probability the model gives to the true label; the **loss** $\ell = -\ln p_y$; and the **logit of the confidence**, $\phi = \ln \frac{p_y}{1-p_y}$, the log of the odds the model gives to the true label. For logistic regression the last one is especially simple. The model gives the true label y the probability $p_y = \sigma(y\langle w, x\rangle)$, and the logit undoes the logistic function, $\ln \frac{\sigma(t)}{1-\sigma(t)} = \ln \frac{1/(1+e^{-t})}{e^{-t}/(1+e^{-t})} = t$. So ϕ is the **margin** $y\langle w, x\rangle$, how far the point lies on the correct side of the decision boundary, and the loss is a function of it:

$$\phi = \ln \frac{p_y}{1-p_y} = y\langle w, x\rangle, \qquad \ell = -\ln p_y = \ln\left(1 + e^{-\phi}\right). \tag{10.4}$$

The second formula is $-\ln \sigma(\phi)$ written out, since $1/\sigma(\phi) = 1 + e^{-\phi}$.

The three numbers behave differently at the confident end. A model that gives the true label probability 0.9 has $\phi = \ln(0.9/0.1) = \ln 9 = 2.197$ and loss $\ln(1/0.9) = 0.105$. Probability 0.99 gives $\phi = 4.595$; 0.999 gives 6.907. The probability crowds against 1, while ϕ keeps spreading the values out.

Yet as decision rules they are interchangeable. The confidence and ϕ rise together, and the loss falls as they rise, so each is a monotone function of the others. **A threshold on one is a threshold on the others,** and all three give the same ROC curve. They differ only when a statistic is *combined* across records, for example averaged or fitted with a Gaussian, which is where LiRA's choice of ϕ matters.

10.3. The loss attack and the generalization gap

10.3.1. Context

A model is fitted to its training data, so it usually has a lower loss on a training point than on a fresh one. Yeom et al. (2018) turned this into an attack: **say "member" when the loss is low**. The attack needs no shadow models and no knowledge of the population beyond the loss threshold. They also showed that its advantage is exactly the generalization gap: overfitting and membership leakage are, for this attack, the same number.

10.3.2. Statement

The cleanest version of the attack turns the loss directly into a probability of saying "member": the lower the loss, the more likely the verdict "member". For that to be a probability, the loss must have a known range, so let it take values in $[0, B]$. The **bounded-loss attack** says "member" with probability $1 - \ell(\theta, z)/B$: always at loss 0, never at loss B, and in proportion in between. In the membership game of Fact 1, its advantage is the expected loss on non-members minus the expected loss on members, divided by B. That difference is the **generalization gap**: the model's

expected loss on fresh data, its **risk** $R(\theta_S)$, minus its average loss on its own training set S, its **empirical risk** $\hat{R}_S(\theta_S)$. So

$$\text{Adv} = \frac{\mathbb{E}[\ell(\theta, z) \mid b = 0] - \mathbb{E}[\ell(\theta, z) \mid b = 1]}{B} = \frac{\mathbb{E}_S\big[R(\theta_S) - \hat{R}_S(\theta_S)\big]}{B}, \tag{10.5}$$

the expected generalization gap divided by B. For the 0-1 loss ($B = 1$), which is 0 on a correct prediction and 1 on a wrong one, the attack is deterministic, "member if and only if the model classifies z correctly", and its advantage is the test error minus the training error.

10.3.3. Derivation

The plan is to compute the attack's TPR and FPR as average losses, subtract them as Fact 1 says, and then recognize the two averages as test loss and training loss.

1. **TPR.** Given $b = 1$, the attack says "member" with probability $1 - \ell/B$ for whatever loss the target has. Averaging over targets and training runs, $\text{TPR} = \mathbb{E}[1 - \ell/B \mid b = 1] = 1 - \mathbb{E}[\ell \mid b = 1]/B$.
2. **FPR.** The same calculation with $b = 0$ gives $\text{FPR} = 1 - \mathbb{E}[\ell \mid b = 0]/B$.
3. **Subtract.** By Fact 1 the advantage is TPR − FPR. The 1s cancel and $\text{Adv} = (\mathbb{E}[\ell \mid b = 0] - \mathbb{E}[\ell \mid b = 1])/B$.
4. **Identify the two averages.** When $b = 1$, z is a uniformly chosen training point, so its expected loss is the expected training loss, $\mathbb{E}[\ell \mid b = 1] = \mathbb{E}_S[\hat{R}_S(\theta_S)]$. When $b = 0$, z is a fresh draw independent of S, so its expected loss is the expected test loss, $\mathbb{E}[\ell \mid b = 0] = \mathbb{E}_S[R(\theta_S)]$. Their difference is the expected gap of Algorithmic stability and generalization.
5. **The 0-1 loss.** With $\ell \in \{0, 1\}$ and $B = 1$, "member with probability $1 - \ell$" says "member" with probability 1 when $\ell = 0$ and 0 when $\ell = 1$, that is, exactly when z is classified correctly. The members it flags are the training points the model gets right, so TPR = 1 − train error; the non-members it flags are the fresh points it gets right, so FPR = 1 − test error; and Adv = test error − train error.

10.3.4. Interpretation

The first lesson is that **overfitting implies leakage.** Any model with a generalization gap leaks membership through its loss, by at least the gap divided by the loss range. You can read this off from training and test error alone, without running any attack.

The gap also has a finer structure. The stability chapter shows that the expected gap equals the average **self-influence** of the training examples: how much each example's own loss rises when it is taken out of training and replaced by a fresh record. The reason is that a training point's loss and a fresh point's loss differ exactly by whether the model saw that point. So the loss attack's advantage is the average, over records, of how much each record is memorized. An average hides who is memorized, and Section 10.7 measures it record by record.

The converse fails: **a small gap does not mean a private model.** The loss attack is one attack. It is the best attack that sees only the loss and ignores *which* record it is looking at, but a record-aware attack can beat it by a wide margin, as the rest of this chapter shows. Yeom et al. also construct models that leak membership with no generalization gap at all.

Finally, the randomized rule is convenient for the proof, but an attacker would rather use a plain threshold. Yeom et al. also propose a deterministic version: say "member" when the loss is below the expected training loss τ. That needs only τ, which an attacker can estimate from shadow models or a reported training loss.

10.3.5. Example: the running example

Over the 128 target models, the training error is 0.1495 and the test error is 0.2663. "Member if correctly classified" flags the members it gets right, so $\text{TPR} = 1 - 0.1495 = 0.8505$, and the non-members it gets right, so $\text{FPR} = 1 - 0.2663 = 0.7337$. Its advantage is $0.8505 - 0.7337 = 0.1168$, exactly the gap $0.2663 - 0.1495$.

The logistic loss is unbounded, so the bounded-loss attack needs it clipped: replace any loss above $B = 2$ by 2. The mean clipped loss is 0.4158 on members and 0.5433 on non-members. Equation 10.5 predicts an advantage of $(0.5433 - 0.4158)/2 = 0.0638$. Drawing the attack's coins gives 0.0639. With Yeom's threshold, the mean training loss $\tau = 0.4159$, the deterministic attack has $\text{TPR} = 0.5936$, $\text{FPR} = 0.4834$ and advantage 0.1102.

By Fact 1, an advantage between 0.0638 and 0.1168 means winning between $\frac{1}{2} + \frac{1}{2}(0.0638) = 0.532$ and $\frac{1}{2} + \frac{1}{2}(0.1168) = 0.558$ of games: the three attacks win between 53% and 56% of games. None identifies anyone with confidence: an FPR near one half means that half of the non-members are flagged too. Figure 10.1 shows why. The two distributions of ϕ (equivalently, of the loss, by Fact 3) are shifted but almost entirely overlap.

10.3.6. Check yourself

- A model has training accuracy 99% and test accuracy 70%. What advantage does "member if correctly classified" reach? *(The test error minus the training error,* $0.30 - 0.01 = 0.29$*. It wins the membership game with probability* $\frac{1}{2} + \frac{1}{2} \times 0.29 = 0.645$*.)*
- Why does the bounded-loss attack need a randomized decision rather than a threshold? *(Randomizing with probability* $1 - \ell/B$ *makes the probability of saying "member" linear in the loss, so its expectation is the expected loss. A threshold gives an advantage that depends on the whole distribution of the loss, not only its mean.)*

10.4. Shadow models

10.4.1. Context

An attacker who wants a better rule than "low loss means member" has to know what members' outputs look like, and it has no labelled examples: it does not know who is in the target's training set. Shokri et al. (2017) solved this with **shadow models**. The attacker trains models of its own that imitate the target: same architecture, same training recipe, data from the same distribution. For its own models it *knows* who was a member. It collects the shadow models' outputs on members and non-members, labels them "in" and "out", and trains an **attack model**, a classifier from outputs to membership. Shokri et al. train one attack model per class, with the prediction vector as input.

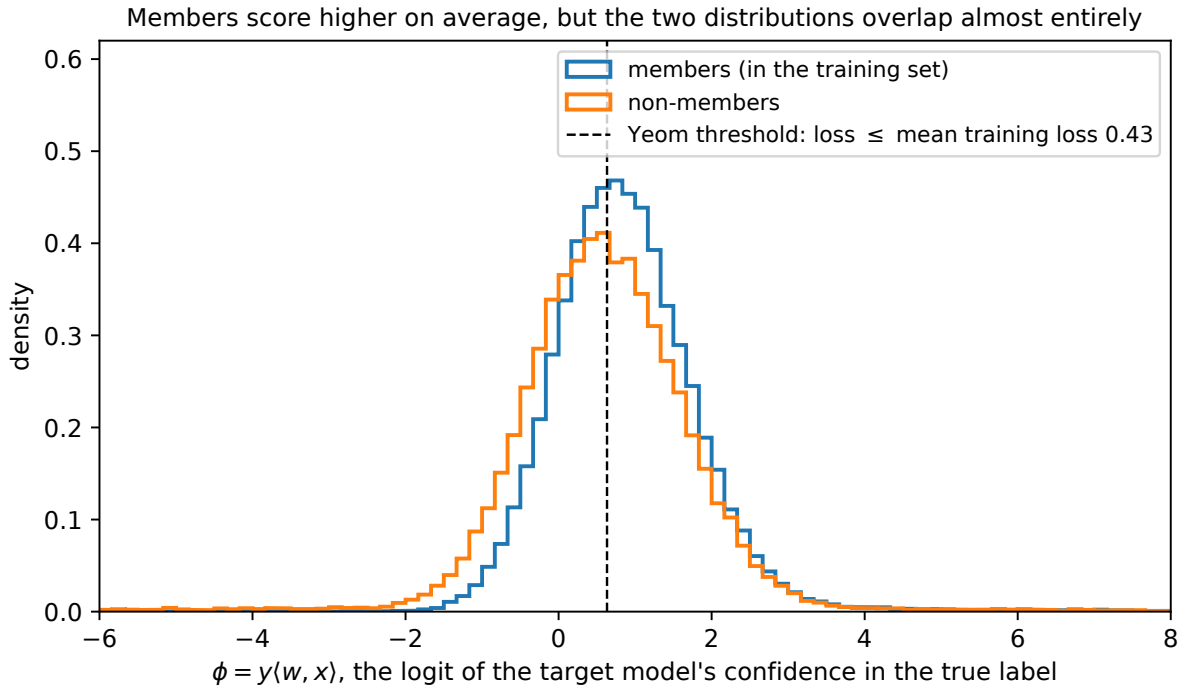


Figure 10.1.: Distribution of ϕ, the logit of the target model's confidence in the true label, over member pairs and non-member pairs of the 128 target models. The dashed line is Yeom's threshold: the attack says "member" to its right, where the loss is below the mean training loss.

10.4.2. Statement

The claim is that, with enough shadow data, the attack model learns the best rule available from the output alone. Suppose each shadow model is trained exactly as the target is. Let o be the output the attacker sees (here $o = (y, \phi)$, the class and the logit of the confidence). Train the attack model $a(o) \in (0, 1)$, a predicted probability of "in", by minimizing the cross-entropy, the usual classification loss $-\ln a$ on "in" pairs and $-\ln(1 - a)$ on "out" pairs, over the shadow pairs (o, m), with $m = 1$ for "in".

As the number of shadow pairs grows, a tends to the **posterior probability of membership**, the probability that the record is a member given that the output is o. With a fair prior, the posterior odds equal the likelihood ratio, so the posterior is an increasing function of it:

$$a(o) \to \Pr[b = 1 \mid o], \qquad \frac{\Pr[b = 1 \mid o]}{\Pr[b = 0 \mid o]} = \frac{q(o)}{p(o)}, \tag{10.6}$$

where q and p are the densities of o for members and for non-members, *pooled over all records*. So thresholding the attack model is the Neyman–Pearson test for the observation o, and no attack that sees only o has a higher TPR at any FPR.

10.4.3. Derivation

The argument has four links: the shadow data look like the real game, cross-entropy training recovers the posterior, the posterior is a monotone function of the likelihood ratio, and thresholding the likelihood ratio is optimal.

1. **The shadow pairs have the right distribution.** A shadow model is trained like the target, on data from the same population, and each record is in or out with probability $\frac{1}{2}$. So a shadow pair (o, m) has the same joint distribution as a pair (target output, membership bit) in the game. Learning from shadow pairs is learning about the real game.
2. **Cross-entropy is minimized by the true posterior.** Fix one output o and let $\rho = \Pr[m = 1 \mid o]$ be the true fraction of "in" among pairs with that output. If the attack model predicts a there, its expected loss is $-\rho \ln a - (1 - \rho) \ln(1 - a)$. Its derivative in a is $-\rho/a + (1 - \rho)/(1 - a)$, which is zero at $a = \rho$, and the function is convex, so that zero is the minimum. Since this holds separately at every o, the minimizer over all functions of o is $a(o) = \rho(o)$. With enough shadow data and a flexible enough attack model, training finds it.
3. **Bayes' rule.** Posterior odds equal prior odds times the likelihood ratio (Hypothesis testing): $\Pr[b = 1 \mid o] / \Pr[b = 0 \mid o] = \frac{\Pr[b=1]}{\Pr[b=0]} \frac{q(o)}{p(o)}$, because the joint probability of a world and an output is the prior of the world times the density of the output in that world. With prior odds 1, $\Pr[b = 1 \mid o] / \Pr[b = 0 \mid o] = q(o)/p(o)$. The posterior a and the odds $a/(1 - a)$ rise together, since $a \mapsto a/(1 - a)$ is increasing.
4. **Neyman–Pearson.** So a threshold on $a(o)$ is a threshold on $q(o)/p(o)$. By the Neyman–Pearson lemma, a likelihood-ratio threshold has the highest TPR at its FPR among all tests that see o.

10.4.4. Interpretation

The attack model is **optimal for the wrong question.** The densities q and p in step 4 are *pooled over records*: the attack model sees (y, ϕ), not which record produced it. So it is the best attack among those that treat all records alike. If $q(o)/p(o)$ is increasing in ϕ within each class, which is what you expect when members tend to have higher ϕ, the attack is a threshold on ϕ per class, which is the loss attack with a class-dependent threshold. It cannot know that $\phi = 2$ is a high value for one record and a low value for another.

What shadow models really buy, then, is **calibration**. They let the attacker choose a threshold that gives a target FPR on the real model, without any member of the real training set: the attacker picks the threshold at which the desired fraction of its own shadow non-members pass, and trusts that the real non-members behave the same way. That is the job they keep in LiRA.

The imitation can also be crude. Salem et al. (2019) found that one shadow model is often enough, and that shadow data from a different distribution still works, because what the attack model learns (lower loss means member) transfers across tasks.

10.4.5. Example: the attack model on the running example

For each class, the attack model is a logistic regression of "in" on $(1, \phi)$, trained on the roughly 32,000 shadow pairs of that class (64 shadow models times the pool points with that label). Both slopes are positive (0.238 for $y = -1$, 0.158 for $y = +1$), so within each class the predicted probability of "in" rises with ϕ, and a threshold on it is a threshold on ϕ. It is a per-class threshold on the loss. Its performance across all thresholds is summarized by the **area under the curve (AUC)**, the area under its ROC curve, which equals the probability that a random member gets a higher score than a random non-member. Its AUC is 0.577, the same as the loss attack's, and so is its TPR at every FPR to within sampling noise (Table 10.4).

Calibration works. Setting the attack model's threshold on the shadow models so that their FPR is 1% gives, on the 128 target models it never saw, an FPR of 0.0095 and a TPR of 0.0098. The attacker hits the FPR it aimed for. At that FPR the TPR is no better than chance, since 0.0098 is about the 0.0095 fraction of non-members flagged, because a global threshold on ϕ has nothing to work with at the low end.

10.4.6. Check yourself

- The attacker's shadow data come from a different hospital than the target's. Which step of the derivation fails, and is the attack useless? *(Step 1: the shadow pairs no longer have the target's joint distribution, so the attack model estimates the wrong posterior. It is not useless. Calibration is off, but "lower loss means member" usually still holds.)*
- Why does feeding the attack model a richer input, the whole prediction vector instead of ϕ, not help in the running example? *(For a binary linear model the prediction vector is $(\sigma(\langle w, x\rangle), 1 - \sigma(\langle w, x\rangle))$, and with the label it determines exactly ϕ and y, and ϕ and y determine it back. Each view can be computed from the other, so by Fact 2 of Threat models and privacy games, which says that processing a view cannot help, they give the same best attack.)*

10.5. LiRA: a likelihood-ratio test per example

10.5.1. Context

The pooled attacks of the last two sections compare a record's output with everyone else's. The right comparison is with the *same record's* output under models that did and did not train on it. Carlini et al. (2022) built the **likelihood ratio attack** (LiRA) on that idea. For a target record z, the attacker uses shadow models to estimate two distributions of ϕ_z, the logit of the confidence on z: the IN distribution $\tilde{Q}_z$ (models trained with z) and the OUT distribution $\tilde{P}_z$ (models trained without z). It then computes the likelihood ratio of the observed ϕ_z and thresholds it.

10.5.2. Statement

The attacker needs densities to form a likelihood ratio, and it only has a few dozen shadow values of ϕ_z per record. So it summarizes each set by a bell curve: it fits a Gaussian to each distribution, $\tilde{Q}_z = \mathcal{N}(\mu_{\text{in},z}, \sigma^2_{\text{in},z})$ and $\tilde{P}_z = \mathcal{N}(\mu_{\text{out},z}, \sigma^2_{\text{out},z})$, from the shadow models that did and did not train on z. Here μ and σ are the mean and standard deviation of ϕ_z over those models.

Online LiRA asks how much more likely the target model's ϕ_z is under the IN fit than under the OUT fit. Its score is the natural log of that ratio, where $\mathcal{N}(\phi; \mu, \sigma^2)$ denotes the normal density at ϕ:

$$\ln \Lambda_z(\phi_z) = \ln \mathcal{N}(\phi_z;\, \mu_{\text{in},z}, \sigma^2_{\text{in},z}) - \ln \mathcal{N}(\phi_z;\, \mu_{\text{out},z}, \sigma^2_{\text{out},z}) \tag{10.7}$$

It says "member" when $\ln \Lambda_z$ exceeds a threshold c that is **the same for every record**.

Offline LiRA uses only OUT models, which can be trained before the target record is known. It cannot compute an IN density, so it asks only how far above its usual OUT value the observed ϕ_z lies, measured in OUT standard deviations:

$$\frac{\phi_z - \mu_{\text{out},z}}{\sigma_{\text{out},z}}, \tag{10.8}$$

again with one threshold for all records. When the Gaussian fits are exact, online LiRA is the Neyman–Pearson test for the membership game: no attack that sees z and ϕ_z has a higher TPR at any FPR.

10.5.3. Derivation

Why one threshold on the per-example ratio is optimal. The adversary does not see ϕ_z alone; it also knows which record it is testing. So the observation is the pair (record, output), and the Neyman–Pearson lemma should be applied to the pair. The point of the steps below is that the record part of the pair carries no evidence and drops out of the ratio, leaving each record's own ratio.

1. **The view is a pair.** In the membership game the adversary sees the target record z and the output ϕ_z. The density of the pair is the probability of drawing that record times the density of the output given the record. Write it as $g(z)\, q_z(\phi)$ when $b = 1$ and $g(z)\, p_z(\phi)$ when $b = 0$, where q_z and p_z are the IN and OUT densities of ϕ_z.

2. **The record's own distribution is the same in both worlds.** The challenger picks z from the same pool whether or not it ends up in the training set, so g is the same factor in both. (In the running example every point is in the pool and is a member with probability $\frac{1}{2}$.)
3. **The likelihood ratio of the pair** is $\frac{g(z)\,q_z(\phi)}{g(z)\,p_z(\phi)} = \frac{q_z(\phi)}{p_z(\phi)} = \Lambda_z(\phi)$. The factor $g(z)$ cancels.
4. **Neyman–Pearson.** The best test of the pair thresholds its likelihood ratio, $\Lambda_z(\phi)$, at a single value. Different records are compared through their ratios, never through their raw ϕ.

This is the mixture argument of Fact 3 in Threat models and privacy games. An attacker that does not know which record it is testing sees an average over records of their IN and OUT distributions, and averaging blurs the differences, as when a large shift for a hard record is diluted among the small shifts of easy ones. An attacker that knows the record can undo that mixing: its best advantage rises from the total variation distance of the pooled distributions (the best advantage of any test between two distributions) to the average of the per-record ones.

Why Gaussian fits on ϕ. The fits are only as good as the bell-curve shape they assume, so it matters that ϕ, rather than the loss or the confidence, is close to Gaussian.

5. **ϕ is linear in the weights.** By Fact 3, $\phi_z = y\langle w, x\rangle$, a fixed linear function of w for a fixed record. The regularized logistic-regression estimate w solves the equation "average gradient over the training points plus λw equals zero". Because that is an average of many independent terms, the central limit theorem makes w approximately Gaussian over the randomness of the training set (statisticians call such an estimate an M-estimator), and any fixed linear function of a Gaussian vector is Gaussian too. For a deep network the same scaling turns the confidence p_y, which piles up against 1, into a quantity whose distribution over retrainings Carlini et al. found close to Gaussian.
6. **Estimate the parameters from shadows.** In the running example every point is in exactly 32 of the 64 shadow models, because the shadows come in complementary pairs. The sample means and standard deviations over the 32 IN and the 32 OUT values give $\mu_{\text{in},z}, \sigma_{\text{in},z}, \mu_{\text{out},z}, \sigma_{\text{out},z}$.

Offline as a one-sided test. Offline LiRA is a weaker test, and the last two steps say what it tests and when it loses nothing.

7. **Without IN models**, the attacker cannot write down q_z. It can still test "ϕ_z is from $\tilde{P}_z$" against "ϕ_z is larger than that", because training on z raises ϕ_z: the model moves to fit z, which increases its margin on z. The standardized score Equation 10.8 is the natural statistic, and $1 - \Phi$ of it is a one-sided p-value: the probability that a non-member's output would lie at least this far above its OUT mean.
8. **When it loses nothing.** Suppose the two fits have the same width, $\sigma_{\text{in},z} = \sigma_{\text{out},z} = \sigma_z$. Measure everything in units of σ_z from the OUT mean: the offline score is $t_z = (\phi_z - \mu_{\text{out},z})/\sigma_z$, and the IN mean sits at $s_z = (\mu_{\text{in},z} - \mu_{\text{out},z})/\sigma_z$. Then t_z is standard normal under OUT and normal with mean s_z under IN, which is exactly the setting of Fact 2, and Equation 10.7 becomes $\ln \Lambda_z = s_z t_z - s_z^2/2$. For one record, s_z is a fixed positive number, so both scores rank outputs the same way. Across records, they differ: online LiRA multiplies t_z by s_z, so it turns a given t_z into strong evidence when s_z is large and weak evidence when s_z is small; offline LiRA treats every t_z the same.

10.5.4. Example: two kinds of record, in closed form

A population with two kinds of record shows how much the per-record comparison matters, with numbers you can check by hand. Take unit variances everywhere (Table 10.2). A typical record is predicted confidently whether or not the model trained on it, so its OUT and IN means are both high and close together. An atypical record has a negative ϕ (it is misclassified) unless the model trained on it.

Table 10.2.: Two kinds of record. A typical record is predicted confidently whether or not it was trained on; an atypical one is predicted wrongly unless it was. The last column is Equation 10.3.

Kind	Share	OUT: $\phi \sim$	IN: $\phi \sim$	Separation s	Best TPR at FPR 10^{-3}, this kind alone
Typical	95%	$\mathcal{N}(3,1)$	$\mathcal{N}(3.5,1)$	0.5	0.0048
Atypical	5%	$\mathcal{N}(-2,1)$	$\mathcal{N}(2,1)$	4	0.8185

The separations are the gaps between the means, $3.5-3=0.5$ and $2-(-2)=4$, and the last column is the Fact 2 calculation done above for exactly these two separations. Now run three attacks, each set to a pooled FPR of 10^{-3}: over all non-members together, one in a thousand is flagged.

One threshold on ϕ is the loss attack. The OUT values of typical records are high, and they make up 95% of non-members, so the threshold must sit high to let through only 0.1% of non-members: at $\phi = 6.075$, typical non-members pass with probability $1-\Phi(6.075-3) = 1-\Phi(3.075)$, about 0.00105, and times their share of 0.95 that is the whole budget of 10^{-3}. Atypical non-members, centred at -2, essentially never pass. An atypical member scores around 2 and almost never passes. Only typical members, whose IN mean is 3.5, occasionally do, and the pooled TPR is 0.0048.

One threshold on the z-score is offline LiRA. Each record is standardized against its own OUT distribution, so every record gets FPR 10^{-3}, at $t_z \geq 3.0902$. Each kind then gets its own best TPR from the table, and the pooled TPR is their weighted average, $0.95 \times 0.0048 + 0.05 \times 0.8185 = 0.0455$.

One threshold on $\ln\Lambda$ is online LiRA. The threshold is $\ln\Lambda \geq 1.497$. With equal variances $\ln\Lambda = st - s^2/2$ (step 8), so for typical records the threshold is $t \geq (1.497+0.125)/0.5 = 3.244$, which lets through an FPR of only 5.90×10^{-4}; for atypical records it is $t \geq (1.497+8)/4 = 2.374$, which lets through FPR 0.0088. The pooled FPR is $0.95\times5.90\times10^{-4} + 0.05\times0.0088 = 10^{-3}$. The pooled TPR is 0.0503, the best possible at that FPR by the derivation above.

The global threshold is ten times worse than either per-example test. The likelihood ratio does better still than the z-score because it moves FPR budget from records where it buys little to records where it buys a lot. A typical record's extra false positives buy almost no true positives, since its two curves overlap; an atypical record's buy many, since its IN curve lies far to the right. The likelihood ratio makes that trade automatically.

Figure 10.2 shows the IN and OUT distributions for two real points of the running example: a typical point and an outlier.

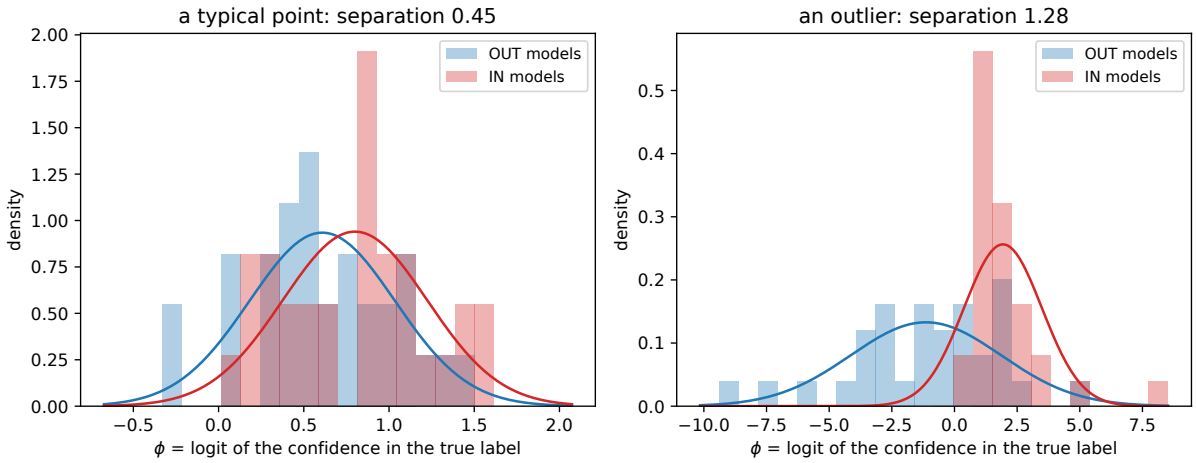


Figure 10.2.: The logit ϕ of one point over the 64 shadow models, split into the 32 that trained on it (IN) and the 32 that did not (OUT), with the fitted Gaussians. Left: a typical point, whose two distributions overlap. Right: an outlier, which the models predict wrongly unless they trained on it.

10.5.5. Interpretation

Online against offline. Online LiRA needs models trained *with* each target record, so to attack a new record you must train new shadow models. Offline LiRA's OUT models can be trained once, before the targets are known, and reused for every query. That is why it matters in practice, at some cost in power.

Per-example or global variance. Each per-example variance is estimated from half the shadow models, and with few shadow models those estimates are noisy. Noise in the variance is most harmful exactly where it counts. A record whose spread happens to be underestimated has its OUT curve drawn too narrow, so an ordinary output looks far out in the tail, its likelihood ratio is inflated, and it becomes a false positive at the strict thresholds that define the low-FPR end. Carlini et al. recommend one **global variance**, shared by all records, when shadow models are few: it averages over every record, so it is estimated from far more values and is stable. Figure 10.3 shows the effect on the running example.

More queries. For image models, Carlini et al. query each target on several augmented copies (flips, shifts) and fit a multivariate Gaussian. More observations of the same record give more evidence.

10.5.6. Check yourself

- The IN and OUT distributions of a record are identical. What is its likelihood ratio, and can LiRA ever flag it? *(The ratio is 1,* $\ln \Lambda = 0$*, for every output. With a positive threshold it is never flagged, which is right: no test can tell member from non-member for that record.)*

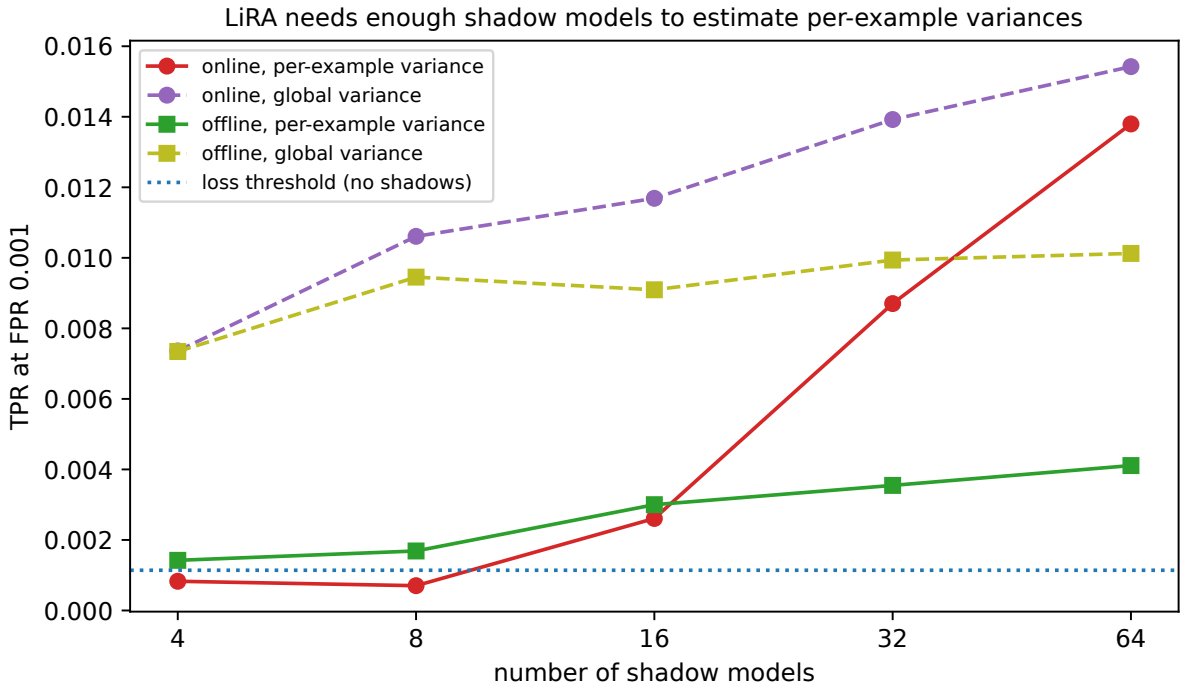


Figure 10.3.: TPR at FPR 10^{-3} of four LiRA variants on the 128 target models, as the attacker uses the first 4, 8, 16, 32 or 64 of its shadow models. With a global variance the attack is strong even with few shadow models; with per-example variances it needs many. The dotted line is the loss attack, which needs no shadow models.

- Why does the derivation need the target record to be drawn the same way in both worlds (step 2)? *(Otherwise the factor $g(z)$ does not cancel, and the record's identity is itself evidence. If, say, members were drawn from one hospital and non-members from another, the record alone would reveal b.)*

10.6. Evaluating attacks: TPR at a low FPR

10.6.1. Context

Two numbers dominate older papers on membership inference: the attack's **accuracy** (equivalently its advantage, by Fact 1) and its **AUC**, the probability that a random member scores higher than a random non-member. Both average over all members and non-members. A privacy breach is not an average. It happens when an attacker can say *with confidence* that a particular person was in the data. That means a low FPR, because at a low FPR almost every "member" verdict is right. Carlini et al. (2022) argued that attacks should be reported by their TPR at a fixed low FPR, such as 10^{-3}, and by the full ROC curve on log axes. The ROC curve plots TPR against FPR as the threshold sweeps from strict to lenient.

10.6.2. Statement

Two attacks can have the same AUC and nearly the same accuracy, yet TPRs at FPR 10^{-3} that differ by a factor of 38. The two attacks below are built to show how. Both concern a population of members and non-members; q is a fraction of members, and α is the FPR at which a TPR is read off.

Attack A exposes a few members completely and learns nothing about anyone else. It recognizes a fraction q of members with certainty and gives every other record, member or not, the same random score. Its AUC is $\frac{1}{2} + \frac{q}{2}$, its best advantage is q, and $\mathrm{TPR}(\alpha) = q + (1-q)\alpha$.

Attack B learns a little about everyone. It gives members a score $\mathcal{N}(\mu, 1)$ and non-members $\mathcal{N}(0, 1)$, with the shift μ chosen so that its AUC is the same as A's: $\Phi(\mu/\sqrt{2}) = \frac{1}{2} + \frac{q}{2}$. Its best advantage is $2\Phi(\mu/2) - 1$ and its TPR is Equation 10.3 with $s = \mu$.

10.6.3. Derivation

1. **AUC of A.** The AUC is the probability that a random member outscores a random non-member, with ties counted as one half (Hypothesis testing). Split on whether the member is one of the recognized ones. With probability q it is, and it wins. Otherwise both scores have the same distribution, so neither is more likely to be higher and the member wins with probability $\frac{1}{2}$. Total: $q + (1-q)\frac{1}{2} = \frac{1}{2} + \frac{q}{2}$.
2. **TPR of A.** At any threshold with FPR α, the recognized members always pass, and the others have the same score distribution as non-members, so they pass with the same probability α. So $\mathrm{TPR} = q + (1-q)\alpha$, and $\mathrm{TPR} - \mathrm{FPR} = q + (1-q)\alpha - \alpha = q(1-\alpha)$, at most q.

3. **AUC of B.** A member outscores a non-member when the difference of their scores is positive. That difference is a member's $\mathcal{N}(\mu, 1)$ minus an independent $\mathcal{N}(0, 1)$, so it is normal with mean μ and variance $1 + 1 = 2$. It is positive with probability $\Phi(\mu/\sqrt{2})$, the chance that a normal variable lies less than $\mu/\sqrt{2}$ of its standard deviations below its mean.
4. **Advantage of B.** The best advantage of a test between two distributions is their **total variation distance**: the most that the probability of any set of scores can differ between the two, reached by the set where one density exceeds the other. For two unit-variance Gaussians at distance μ the densities cross halfway, at $\mu/2$, so the best test says "member" above $\mu/2$. Its TPR is $\Pr[\mathcal{N}(\mu, 1) > \mu/2] = \Phi(\mu/2)$, its FPR is $\Pr[\mathcal{N}(0, 1) > \mu/2] = 1 - \Phi(\mu/2)$, and its advantage is the difference, $2\Phi(\mu/2) - 1$ (both facts are in Hypothesis testing).

With $q = 0.05$ (Table 10.3), both AUCs are $\frac{1}{2} + \frac{0.05}{2} = 0.5250$, and solving $\Phi(\mu/\sqrt{2}) = 0.5250$ gives $\mu = 0.0887$. A's best advantage is $q = 0.0500$ and B's is $2\Phi(0.0887/2) - 1 = 0.0354$. At FPR 10^{-3}, A has TPR $0.05 + 0.95 \times 10^{-3} = 0.05095$, while B has $1 - \Phi(3.0902 - 0.0887) = 0.00134$, a factor of 38.

Table 10.3.: Two attacks with the same AUC. Only the TPR at a low FPR tells them apart.

	AUC	Best advantage	TPR at FPR 10^{-3}
A: 5% of members exposed	0.5250	0.0500	0.05095
B: every member shifted by $\mu = 0.0887$	0.5250	0.0354	0.00134

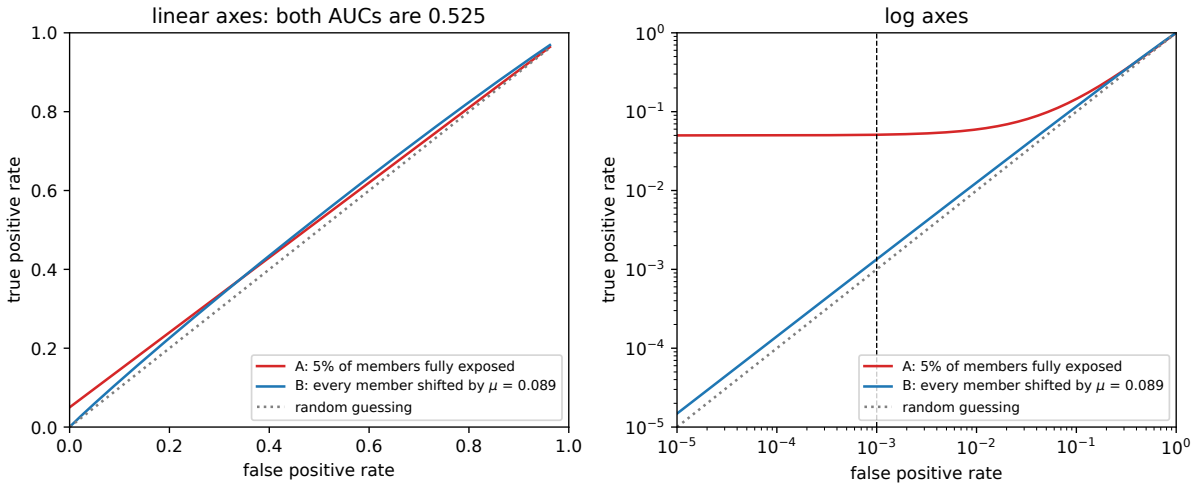


Figure 10.4.: ROC curves of attacks A and B. Left, linear axes: the two curves are indistinguishable from each other and from the diagonal. Right, log axes: A identifies 5% of members at every FPR, B is barely above chance. The dashed line marks FPR 10^{-3}.

10.6.4. Interpretation

Who is exposed matters more than how many. Attack A is a severe breach for one person in twenty, and B is a negligible one for everyone. AUC cannot tell them apart, and their best

advantages differ by only $0.0500 - 0.0354$, about 0.015. Only the low-FPR end of the ROC curve shows the difference.

Log axes are how you see that end. On linear axes, everything below FPR 0.01 is squeezed into one pixel column (Figure 10.4). On log axes, chance is the diagonal TPR = FPR at every scale, and an attack that is useful at low FPR stands out as a curve that flattens above it.

Measuring a low FPR takes many non-members. An FPR is estimated by counting false positives, and a count of k events has a relative error of about $1/\sqrt{k}$, so the estimate is only as good as the number of false positives behind it (Hypothesis testing). To see k false positives at FPR α you need about k/α non-members. That is why the running example evaluates many target models: 64,000 non-member pairs give about $64{,}000 \times 10^{-3} = 64$ false positives at 10^{-3}.

Precision is the fraction of "member" verdicts that are right. At a fair prior, members and non-members are equally many, so the right verdicts are proportional to TPR and the wrong ones to FPR, and the precision is TPR/(TPR + FPR). At TPR 0.05 and FPR 10^{-3} that is $0.05/0.051 = 0.98$. Real targets are rarely members with probability one half. Threat models and privacy games shows how the prior changes this.

10.6.5. Example: the five attacks on the running example

Table 10.4 scores every attack on the 128 target models, and Figure 10.5 draws their ROC curves on log axes. "Best advantage" is the largest TPR − FPR over all thresholds.

Table 10.4.: Every attack on the running example: 128 target models, 64 shadow models, 64,000 member and 64,000 non-member pairs.

Attack	AUC	Best advantage	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}
Loss threshold (Yeom)	0.577	0.122	0.0009	0.0103
Shadow attack model (Shokri)	0.577	0.121	0.0008	0.0104
LiRA offline	0.632	0.189	0.0030	0.0273
LiRA online	0.629	0.175	0.0149	0.0386
LiRA online, global variance	0.623	0.169	0.0177	0.0396

By AUC the attacks are close, 0.577 against about 0.63, and by best advantage offline LiRA is even the strongest. At FPR 10^{-3} they are not close at all. The loss and shadow attacks are at chance: their TPRs of 0.0009 and 0.0008 are no higher than the FPR of 0.001, which is what a coin weighted to say "member" one time in a thousand would score. Online LiRA reaches 0.0149, 17.0 times the loss attack. It finds 952 of the 64,000 members, and 952/64,000 is the 0.0149.

How precise are those numbers? The **Clopper–Pearson interval** is an exact 95% confidence interval for a proportion estimated from a count: the range of true rates under which the observed

count would not be unusually high or low. For the TPR it is $[0.0140, 0.0158]$. The FPR is measured from 64 false positives among 64,000 non-members, with interval $[0.00077, 0.00128]$: the FPR itself is only known to lie between 0.77 and 1.28 times its nominal value. That is the $1/\sqrt{k}$ rule at work: 64 counts give a relative standard error of about $1/\sqrt{64}$, an eighth, and a 95% interval reaches about two standard errors to each side.

One caveat: **the threshold was chosen on the same data** that measure the rates, as is usual in this literature. A real attacker would set it on its shadow models, as the shadow-attack example did.

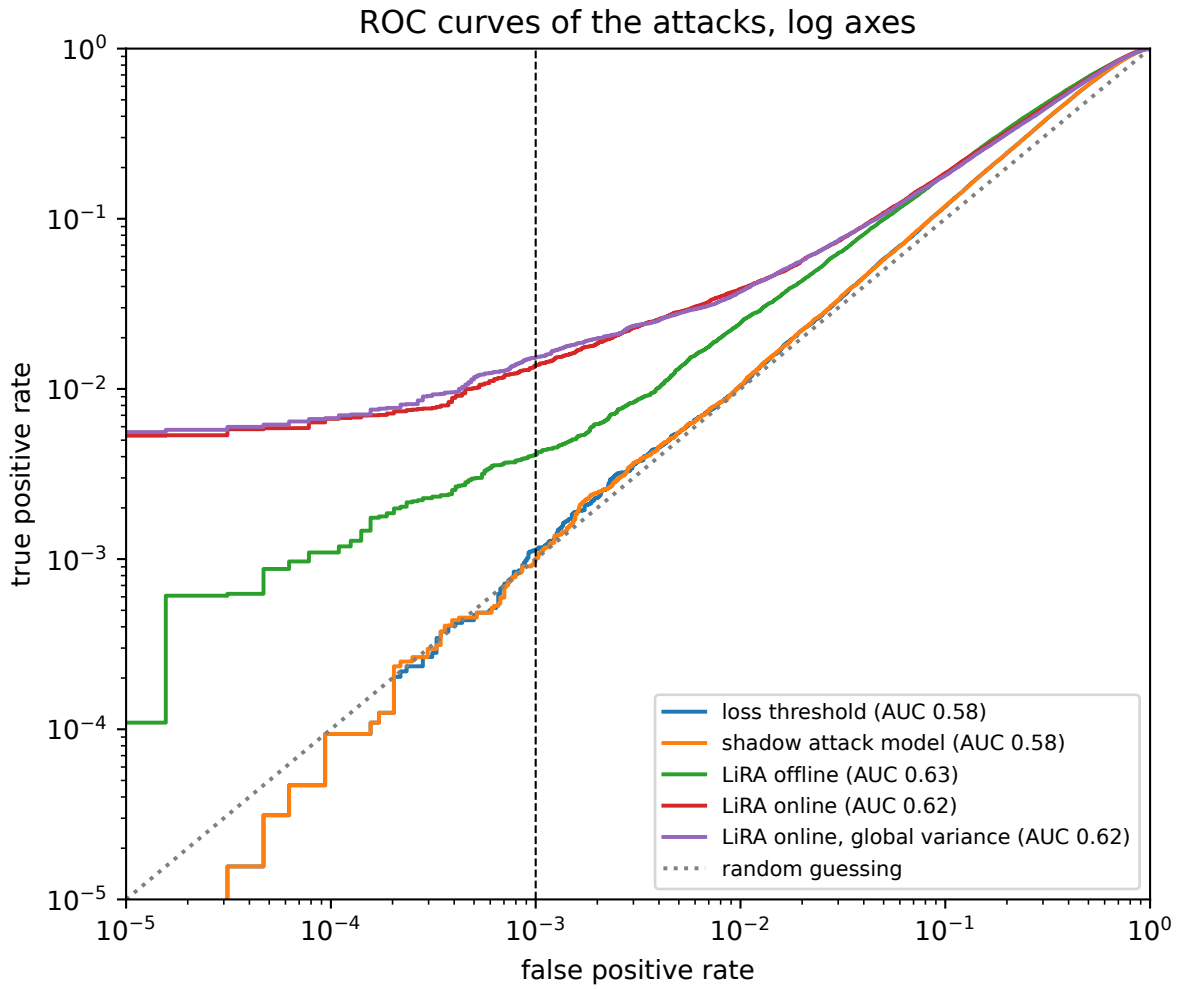


Figure 10.5.: ROC curves of the five attacks on log axes, over the 128 target models. The loss threshold and the shadow attack model follow the diagonal below FPR 0.01; online LiRA stays far above it down to the smallest FPR the 64,000 non-member pairs can measure, $1/64{,}000 \approx 1.6 \times 10^{-5}$. The dashed line marks FPR 10^{-3}.

10.6.6. Check yourself

- An attack has AUC 0.9 and TPR 10^{-3} at FPR 10^{-3}. Is it a privacy threat? *(Not to anyone in particular. It ranks members above non-members on average, but it cannot pick out any member with confidence: at FPR 10^{-3} it is at chance.)*
- With 5,000 non-members, can you report a TPR at FPR 10^{-4}? *(No. The threshold would have to admit $5{,}000 \times 10^{-4}$, that is half a false positive, so it can only admit none. With zero false positives among 5,000, the rule of three only bounds the FPR by about $3/5{,}000 = 6 \times 10^{-4}$. The rule comes from asking which rates make zero events in n trials still 5% likely: $(1-p)^n \approx e^{-pn} = 0.05$ gives $p = \ln 20/n \approx 3/n$.)*

10.7. Why outliers are most exposed

10.7.1. Context

Table 10.2 already shows the pattern: the low-FPR end of the ROC curve is made of records with a large separation between their IN and OUT distributions. Which records are those? The ones the model cannot predict without having seen them: rare, atypical or mislabelled records. This section measures that on the running example and connects it to self-influence, the amount by which training on a record changes the model's fit to that record.

10.7.2. Statement

For one record, the best test is the two-Gaussian test of Fact 2, so the best TPR is set by that record's own separation. For a record z with Gaussian IN and OUT distributions of a common width σ_z, the best TPR of any test on z alone at FPR α is

$$\mathrm{TPR}_z(\alpha) = 1 - \Phi(\Phi^{-1}(1-\alpha) - s_z), \qquad s_z = \frac{\mu_{\mathrm{in},z} - \mu_{\mathrm{out},z}}{\sigma_z}, \tag{10.9}$$

and the numerator $\mu_{\mathrm{in},z} - \mu_{\mathrm{out},z}$ is the **self-influence** of z measured in ϕ: the average amount by which training on z raises the model's logit on z.

10.7.3. Derivation

1. **Per-example test.** For a fixed record, standardize the output: subtract $\mu_{\mathrm{out},z}$ and divide by σ_z. The result is standard normal under OUT and normal with mean s_z and variance 1 under IN, which is the setting of Fact 2 with $s = s_z$. Fact 2 gives Equation 10.9.
2. **Self-influence.** The stability chapter defines the self-influence of a training example z_i as $\ell(\theta_{S^{(i)}}, z_i) - \ell(\theta_S, z_i)$: the loss of the model trained on $S^{(i)}$, the training set with z_i replaced by a fresh record, minus the loss of the model trained on S itself. It is how much z_i's loss rises when z_i is left out, and its average over records is the expected generalization gap. Measure the same thing with ϕ instead of ℓ, with the sign flipped because ϕ rises where the loss falls, and average over the training randomness. Models trained with z give ϕ_z the mean $\mu_{\mathrm{in},z}$ and models trained without it give $\mu_{\mathrm{out},z}$, so it is $\mathbb{E}[\phi_z \mid z \in S] - \mathbb{E}[\phi_z \mid z \notin S] = \mu_{\mathrm{in},z} - \mu_{\mathrm{out},z}$.

3. **Who has large self-influence.** If the rest of the data already predict z's label, $\mu_{\text{out},z}$ is high and training on z adds little. If they predict the opposite label, $\mu_{\text{out},z}$ is low and the model must move toward z to fit it. How far it can move is limited by the regularization. The stability bound for regularized empirical risk minimization in Algorithmic stability and generalization says that one example can move the weights by at most $2L/(\lambda n)$, where L bounds the length of one example's loss gradient. For logistic regression that gradient has length at most $\|x\|$, so a longer x can pull harder. The effect on $\phi_z = y\langle w, x_z\rangle$ of a move in w is the move times up to $\|x_z\|$ again. So a long x_z both pulls the weights harder and turns that pull into a larger change in its own margin. Points with an unpredictable label and a large norm, the outliers of the running example, have both.

10.7.4. Interpretation

Leakage is concentrated. The loss attack's advantage is the *average* self-influence (Section 10.3). The low-FPR end of the ROC curve is governed by the *largest* ones, because a strict threshold is passed only by the records with the largest separations. A model can have a small generalization gap and still expose a few records completely.

Memorizing outliers can be what makes a model accurate. Feldman (2020) shows that on long-tailed data, where many subpopulations are rare, a rare subpopulation seen only once in training must be memorized for good test accuracy, because the next example from it will look like that one. The records a model needs to memorize are the ones it exposes.

Removing the outliers does not fix it. Carlini et al. (2022b), "The Privacy Onion Effect", removed the most exposed records and retrained: the next layer of records became exposed. Exposure is relative to the rest of the dataset, since a record's OUT distribution depends on what the other records predict about it. Exercise 6 in the exercise book reproduces this on the running example.

Average-case evaluation understates the risk to individuals. This is the average-case versus worst-case gap of Threat models and privacy games, seen from the data: a score averaged over records drawn from the population is never more than the score on the worst record, and can be far less. A guarantee that protects every record, the subject of Part 3, has to bound the self-influence of the worst record, not the average.

10.7.5. Example: outliers in the running example

Table 10.5 splits the running example by kind of point.

Table 10.5.: The running example, split into typical points and outliers. Separations and self-influences are estimated from the 64 shadow models; TPR and FPR are measured on the 128 target models.

	Typical points (950)	Outliers (50)
Mean separation s_z	0.461	1.539
Mean loss self-influence, $\mathbb{E}[\ell \mid \text{out}] - \mathbb{E}[\ell \mid \text{in}]$	0.0897	2.0999

	Typical points (950)	Outliers (50)
Mean of Equation 10.9 at $\alpha = 10^{-3}$	0.0074	0.2145
Online LiRA at pooled FPR 10^{-3}: TPR	0.0020	0.2597
Online LiRA at pooled FPR 10^{-3}: FPR	0.00086	0.00375

Outliers carry the gap. The mean loss self-influence over all points weights the two kinds by their shares: $0.95 \times 0.0897 + 0.05 \times 2.0999 = 0.1902$. The outliers' part of that is $0.05 \times 2.0999 \approx 0.105$, a share of 0.552 of the total, from 5% of the points.

Outliers are the true positives. At FPR 10^{-3}, online LiRA flags 26% of outlier members and 0.2% of typical members. Each target model has about 25 outlier members and 475 typical members, so it finds about $0.2597 \times 25 \approx 6.5$ outliers for every $0.0020 \times 475 \approx 0.95$ typical points. So roughly $6.5/(6.5 + 0.95)$, about seven in eight, of the members it finds are outliers; counting the actual flags on the 128 target models gives a share of 0.873, from 5% of the pool.

The likelihood ratio moves FPR budget to them, exactly as in the two-type example: typical points get an FPR of 0.00086, outliers 0.00375, and the weighted average $0.95 \times 0.00086 + 0.05 \times 0.00375$ is the pooled 10^{-3}.

Not every outlier is exposed. Figure 10.6 plots each point's separation against $\mu_{\text{out},z}$. An outlier's label is a coin flip, so about half the time it happens to agree with the trend in the first coordinate. Those outliers are predicted correctly without being seen ($\mu_{\text{out},z} \gg 0$) and are no more exposed than typical points. The exposed ones are those the model gets wrong unless it trains on them.

10.7.6. Check yourself

- Two records have the same loss under the target model. Can one be far more exposed than the other? *(Yes. Exposure depends on the separation between the record's own IN and OUT distributions, not on the observed loss. A low loss is normal for an easy record and strong evidence for a hard one.)*
- A team deletes the 50 outliers from the pool and retrains. Will the typical points be as safe as before? *(Not necessarily. Some typical points now become the least predictable records, and their self-influence rises. This is the onion effect; Exercise 6 in the exercise book measures it.)*

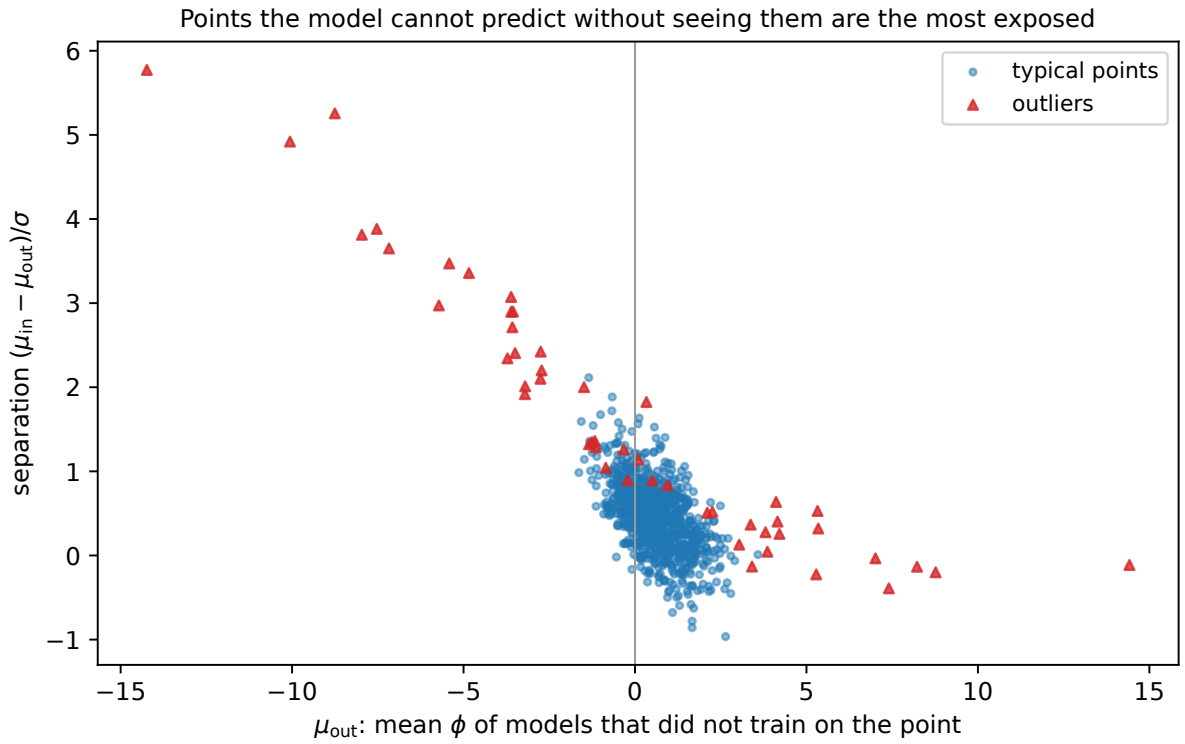


Figure 10.6.: Each pool point's separation $(\mu_{\text{in}} - \mu_{\text{out}})/\sigma$ against μ_{out}, the mean ϕ of the shadow models that did not train on it. Outliers (triangles) whose label the rest of the data contradict have the largest separations.

10.8. The full-size pipeline: LiRA on CIFAR-10

10.8.1. Context

The study plan's exercise for this phase is to train an image classifier on CIFAR-10, a standard benchmark of small colour images in ten classes, run LiRA with 16 shadow models, and report the TPR at FPR 0.1%, and to keep the pipeline for auditing private training later. That takes hours of GPU time, so this book does not run it. The running example is the same protocol with logistic regression in place of a ResNet, a deep convolutional network for images. This section lists what changes at full size.

10.8.2. Statement: the protocol

The protocol is the running example's, step for step. The steps are a recipe to follow in order, so they are listed.

1. **Data split.** Take the 50,000 CIFAR-10 training images as the pool. For each of $N+1$ models, choose a random half of the pool, so that each image is in about half of the models. One model plays the target, the others the shadows. Rotating which model is the target gives many target models and many (model, image) pairs to evaluate, as in the running example.
2. **Training.** Train every model with the same recipe: architecture (Carlini et al. use wide ResNets), optimizer, number of epochs, data augmentation. Imitating the target's recipe is what makes the shadow models valid (Step 1 of the shadow-model derivation: the shadow pairs must have the same distribution as the real ones).
3. **Statistic.** For each model and image, compute the logits on the image and on a few augmented copies (horizontal flips and small shifts). Convert each to $\phi = \ln \frac{p_y}{1-p_y}$, computed stably from the logits as $\phi = f_y - \ln \sum_{y' \neq y} e^{f_{y'}}$, where f are the logits. This follows from the softmax $p_y = e^{f_y} / \sum_{y'} e^{f_{y'}}$: the ratio $p_y/(1-p_y)$ is e^{f_y} over the sum of the other classes' $e^{f_{y'}}$, and the log of it never forms $1-p_y$, which rounds to 0 for a confident model.
4. **Fits.** For each image, fit IN and OUT Gaussians over the shadow models (a mean vector over the augmentations, and per-example or global variances).
5. **Scores.** For each target model and image, compute the online likelihood ratio Equation 10.7, or the offline z-score Equation 10.8.
6. **Evaluation.** Pool all (target model, image) pairs, sweep one threshold, and plot the ROC curve on log axes. Report the TPR at FPR 10^{-3}, with the number of non-member pairs it rests on.

10.8.3. Interpretation

With 16 shadow models, use a global variance. Each image is then in about 8 shadows and out of about 8. Per-example variances from 8 values are too noisy for the low-FPR end, as Figure 10.3 shows for 16 shadow models in the running example.

What to expect. Carlini et al. report that online LiRA is about ten times more powerful than earlier attacks at low FPR on CIFAR-10, and that the loss attack is close to chance there, which

is what Table 10.4 shows in miniature. The exact TPR depends on the model, its training length and the augmentations.

Why it is reused later. The auditing part of the book uses the same pipeline to measure privacy leakage empirically. A privately trained model takes the place of the target, and inserted canaries play the members. Canaries are records chosen to be as exposed as possible, such as mislabeled or out-of-distribution images, so that the attack sees the worst case rather than the average.

10.8.4. Check yourself

- Why must the shadow models use the same data augmentation as the target? *(Augmentation changes how strongly the model fits each image, so it changes μ_{in} and the variances. Shadow models with a different recipe estimate the wrong IN and OUT distributions.)*
- With 25,000 non-member images per target and one target model, how many false positives does FPR 10^{-3} allow? *(25. Rotating over several target models multiplies the count and tightens the interval.)*

10.9. Code: run every attack on the running example

The script code/membership_inference.py builds the pool, trains the 128 target and 64 shadow models, runs every attack, and prints every number in this chapter. It also computes the closed-form examples. Each attack is a small typed function of the margins ϕ and the shadow models' fits, and the figures loop over those functions. It runs in about ten seconds. From the repository root:

```
uv run python code/part02-attacks/membership_inference.py
```

Output (seed 0), summarized in Table 10.6:

Table 10.6.: The script's output with the default seed.

Experiment	Result
Setup	Pool 1,000 points in $\mathbb{R}^{100}$, 50 outliers, $\lambda = 10^{-3}$; 128 target models, 64 shadow models; 64,000 member and 64,000 non-member pairs
Generalization gap	Train error 0.1495, test error 0.2663; "correct means member": TPR 0.8505, FPR 0.7337, advantage 0.1168
Bounded loss, $B = 2$	Clipped loss: members 0.4158, non-members 0.5433; gap/B 0.0638; simulated advantage 0.0639
Yeom threshold	$\tau = 0.4159$: TPR 0.5936, FPR 0.4834, advantage 0.1102

Experiment	Result
Shadow attack model	Slopes 0.238 and 0.158; threshold set on shadows for FPR 1%: target FPR 0.0095, TPR 0.0098
Attacks	Table 10.4
LiRA online at FPR 10^{-3}	952 true positives of 64,000, interval $[0.0140, 0.0158]$; 64 false positives of 64,000, interval $[0.00077, 0.00128]$; 17.0 times the loss attack
Two kinds of record	Global threshold at $\phi = 6.075$: TPR 0.0048. z-score: 0.0455. Likelihood ratio, $\ln \Lambda \geq 1.497$: 0.0503, with FPRs 5.90×10^{-4} and 0.0088
Same AUC	A: AUC 0.5250, advantage 0.0500, TPR 0.05095. B: $\mu = 0.0887$, advantage 0.0354, TPR 0.00134
Outliers	Table 10.5; mean loss self-influence 0.1902, outliers' share 0.552; outliers' share of true positives 0.873
Shadow models, TPR at 10^{-3} for 4, 8, 16, 32, 64	Online, per-example variance: 0.0015, 0.0009, 0.0023, 0.0086, 0.0149. Online, global: 0.0092, 0.0127, 0.0151, 0.0171, 0.0177. Offline, per-example: 0.0018, 0.0023, 0.0029, 0.0035, 0.0030. Offline, global: 0.0098, 0.0111, 0.0112, 0.0114, 0.0118

tests/test_membership_inference.py checks that the output still matches.

What to take from it

Start with the loss attack, where theory and experiment meet exactly. **Yeom's identity is exact:** the 0-1 attack's advantage equals the error gap to four decimals, and the simulated bounded-loss attack matches gap/B to within sampling noise.

That identity measures an average, and the average is misleading where it matters. **Pooled attacks are at chance at the low-FPR end.** The loss threshold and the shadow attack model have AUC 0.577 and best advantage above 0.12, yet their TPR at FPR 10^{-3} is below 10^{-3}.

The fix is not a better statistic. **Per-example calibration is the whole gain:** LiRA uses the same statistic ϕ as the loss attack. It only compares each record with itself, and that multiplies the TPR at FPR 10^{-3} by 17.

How well it compares each record with itself depends on the shadow models. **The variance estimate decides the low end.** With 16 shadow models, online LiRA with per-example variances barely beats chance (0.0023), while with a global variance it reaches 0.0151. Per-example variances only catch up at 64 shadow models.

Finally, the gain is not spread evenly over the records. **The exposed are the outliers:** 5% of the pool provides 87% of the true positives at FPR 10^{-3}.

10.10. References

- Shokri, Stronati, Song & Shmatikov, "Membership Inference Attacks against Machine Learning Models" (IEEE S&P 2017). Shadow models and attack models, one per class; the first systematic study of membership inference on trained classifiers and on machine-learning services.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018). The membership experiment, the loss attack and its advantage as the generalization gap; also attribute inference, used in Attribute inference and model inversion.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022). LiRA online and offline, logit scaling, global variance, and the case for TPR at low FPR on log-scale ROC curves. The model for the running example and the CIFAR-10 pipeline.
- Salem, Zhang, Humbert, Berrang, Fritz & Backes, "ML-Leaks: Model and Data Independent Membership Inference Attacks and Defenses on Machine Learning Models" (NDSS 2019). One shadow model, or none, is often enough.
- Sablayrolles, Douze, Schmid, Ollivier & Jégou, "White-box vs Black-box: Bayes Optimal Strategies for Membership Inference" (ICML 2019). Under assumptions on training noise, the optimal attack depends on the parameters only through the loss, calibrated per example.
- Watson, Guo, Cormode & Sablayrolles, "On the Importance of Difficulty Calibration in Membership Inference Attacks" (ICLR 2022). Subtracting a per-example reference loss: the step from the loss attack toward offline LiRA.
- Ye, Maddi, Murakonda, Bindschaedler & Shokri, "Enhanced Membership Inference Attacks against Machine Learning Models" (CCS 2022). A hypothesis-testing framework that unifies population, reference-model and per-example attacks.
- Zarifzadeh, Liu, Backes & Shokri, "Low-Cost High-Power Membership Inference Attacks" (ICML 2024). Comparing the target with population records gives strong attacks with few reference models.
- Feldman, "Does Learning Require Memorization? A Short Tale about a Long Tail" (STOC 2020). Why accurate models on long-tailed data must memorize rare examples.
- Carlini, Jagielski, Zhang, Papernot, Terzis & Tramèr, "The Privacy Onion Effect: Memorization is Relative" (NeurIPS 2022). Removing the most exposed records exposes the next layer.

10.11. Cheat sheet

$\phi = \ln \frac{p_y}{1-p_y}$ is the logit of the confidence in the true label ($y\langle w, x\rangle$ for logistic regression). IN and OUT are the distributions of ϕ_z over models trained with and without z. Table 10.7 collects the results.

Table 10.7.: Every result in this chapter.

Result	Formula	Note
Advantage	$\mathrm{Adv} = \mathrm{TPR} - \mathrm{FPR} = 2\Pr[b' = b] - 1$	Fair prior
Loss attack (Yeom)	$\mathrm{Adv} = \mathbb{E}[\mathrm{gap}]/B$; 0-1 loss: test error − train error	Average-case; ignores which record it is
Shadow attack model	$a(o) \to \Pr[b = 1 \mid o]$, increasing in pooled $q(o)/p(o)$	Best among record-blind attacks; calibrates thresholds
LiRA online	$\ln \Lambda_z = \ln \mathcal{N}(\phi; \mu_{\text{in}}, \sigma^2_{\text{in}}) - \ln \mathcal{N}(\phi; \mu_{\text{out}}, \sigma^2_{\text{out}})$, one threshold for all z	Neyman–Pearson for the game when the fits are right
LiRA offline	$(\phi - \mu_{\text{out}})/\sigma_{\text{out}}$	OUT models only, reusable across targets
Equal variances	$\ln \Lambda_z = s_z t_z - s_z^2/2$	Online and offline rank one record alike, records differently
Per-example best TPR	$1 - \Phi(\Phi^{-1}(1-\alpha) - s_z)$, $s_z = (\mu_{\text{in}} - \mu_{\text{out}})/\sigma$	$\mu_{\text{in}} - \mu_{\text{out}}$ is the self-influence in ϕ
Same AUC, A vs B	A: $\mathrm{TPR} = q + (1-q)\alpha$; B: $\Phi(\mu/\sqrt{2}) = \frac{1}{2} + \frac{q}{2}$	AUC hides who is exposed

Rules of thumb

- Report the full ROC curve on log axes and the TPR at FPR 10^{-3} or lower, with the number of non-members behind it. Never report accuracy or AUC alone.
- A global threshold on loss or confidence is near chance at low FPR. Calibrate per example.
- With fewer than about 64 shadow models, use a global variance in LiRA.
- The records an attack exposes are the ones the rest of the data cannot predict: outliers, rare subgroups, mislabelled points.
- Overfitting implies membership leakage; no overfitting does not imply its absence.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

11. Attribute inference and model inversion

11.1. Why this matters

A hospital trains a model that predicts whether a patient will develop a lung disease from their age band and whether they smoke, and puts it behind an interface that anyone can query. An insurer knows your age and, from a claim, that you have the disease. Can it use the model to find out whether you smoke?

It can make a better guess than it could without the model. But that is not yet a privacy failure. If smokers get the disease more often, the insurer learns something about every person with the disease, whether or not their record was used for training. A published study saying "smoking causes this disease" would let it make the same guess. The question that matters for *you* is different: **does the model reveal more about your attribute because your record was in its training data?**

That question needs a baseline, and this chapter builds it and measures attacks against it. The attacks have a name. **Attribute inference** asks an adversary to recover one sensitive field of a record, such as smoking status, a genotype or an income band, from the other fields of the record and access to a model. **Model inversion** is the classic way to do it: you query the model once for each candidate value of the sensitive field, and keep the value whose prediction best explains the label you already know. In the insurer's case, you ask the model "how likely is the disease for a smoker of this age?" and "for a non-smoker of this age?", and lean towards whichever answer makes the known diagnosis less surprising.

11.1.1. The one idea: compare with the guesser who never sees the model

Imagine a second insurer who has no access to the model but knows the population perfectly: how common smoking is, how it relates to age and how it relates to the disease. For each combination of the fields it can see, it picks the value of the sensitive field that is most probable in the population. That guess is called the maximum a posteriori (MAP) guess, "a posteriori" because it is made after seeing the other fields, and its success rate is the **imputation baseline**. No model is involved in computing it. (Information theory calls this success rate the posterior vulnerability.)

The chapter rests on one claim about this baseline. **For a record that was not in the training set, no attack can beat the imputation baseline. For a record that was, an attack can.** The reason is that a model trained without your record was built from other people's data only, so it cannot carry any information about you specifically; at most it teaches the adversary about the population, and the baseline already assumes the population is known perfectly. So everything an attack achieves up to the baseline is population inference. What it achieves on training records beyond what it achieves on fresh ones is the leak about individuals. Measuring that leak means comparing people who were in the training set with people who were not, so

attribute inference turns out to be a membership question in disguise, which ties this chapter to Membership inference.

Table 11.1 lists the results and where each is used.

Table 11.1.: The results of this chapter and where each is used.

Result	Where it is used
The attribute inference game and its advantage	Measuring an attack as a difference between training and fresh records (Threat models)
Attribute advantage is a membership advantage	Turning any attribute attack into a membership test; bounding it by total variation
The non-member ceiling: imputation is the best possible on fresh records	Separating population inference from individual leakage
MAP model inversion; it equals imputation for a calibrated model	Reading an inversion attack as Bayes' rule with the model as likelihood
Overfitting lets members beat the baseline	Why the leak shrinks with the training-set size
Masking the confidences cannot help the best attacker	What rounding or label-only outputs do, and what they don't

What you need. Everything this chapter borrows is restated where it is used, so the links below are pointers for more detail. The MAP guess, vulnerability and min-entropy leakage are in Information theory. Bayes' rule, true and false positive rates, and the fact that the best advantage of a test is the total variation distance are in Hypothesis testing. Games and the vocabulary of adversaries are in Threat models, and the loss-threshold attack and the generalization gap are in Membership inference.

Notation. A record is $z = (s, x, y)$: a sensitive attribute s that takes a few values (smoker or not), public features x (the age band), and a label y (the disease). Records come from a population distribution $\mathcal{D}$, and $p(s, x, y)$ is the probability of a record (or its density, when x is continuous). A training set $S \sim \mathcal{D}^n$ is n records drawn independently from $\mathcal{D}$, and a training algorithm turns it into a model M. The model's **confidence** in label y at input (x, s) is $f_y(x, s)$, the probability the model assigns to y.

The chapter keeps returning to one number: how often the best possible guess of s is right when the guesser sees x and y and knows the population. For each pair (x, y), the best guess is the value of s with the largest probability $p(s, x, y)$, and it is right on exactly that much probability. Adding over all pairs gives the success rate, written

$$V(S \mid X, Y) = \sum_{x,y} \max_s p(s, x, y). \tag{11.1}$$

For continuous x, read it as $\mathbb{E}\big[\max_s p(s \mid X, Y)\big]$: the average, over records, of the probability of the most likely value of s given that record's x and y. The capital letters S, X, Y stand for the random fields; the letter S does double duty as the training set, and the context always says which is meant.

11.2. Toolkit: three facts every derivation uses

11.2.1. Fact 1 — The MAP guess is the best guess

Suppose you must guess s after seeing some observation o. For each value of o, whatever you answer is right exactly when the true s is your answer, so the best you can do is answer the value of s that is most probable together with that o. That answer is the **MAP guess**, $\arg\max_s p(s \mid o)$, and its success probability is the sum of the largest probabilities, one per observation:

$$V(S \mid O) = \sum_o \max_s p(s, o) = \mathbb{E}_O\Big[\max_s p(s \mid O)\Big]. \tag{11.2}$$

The second form is the same sum written as an average: $p(s, o) = p(o)\, p(s \mid o)$, and $p(o)$ does not depend on s, so it comes out of the maximum. V stands for **vulnerability**, the name Information theory gives this success rate.

The proof is the counting argument made precise. A deterministic guessing rule g maps each observation o to a guess $g(o)$. It is right when o occurs and $s = g(o)$, which has probability $p(g(o), o)$, so its success is $\sum_o p(g(o), o)$, one term per value of o. Each term is at most $\max_s p(s, o)$, and equals it when $g(o)$ is the MAP guess. A randomized rule picks among deterministic rules at random, so its success is an average of their successes, and an average is never above the best one.

For a concrete case, take no observation at all and a prior of 0.3 that a person smokes. The rule "always say non-smoker" is right 70% of the time and "always say smoker" 30%, so the best guess is "non-smoker", with $V(S) = 0.7$.

11.2.2. Fact 2 — An independent observation is useless

If an extra observation M has nothing to do with s and o, seeing it cannot change which value of s is most likely, so it cannot raise the best success. In symbols: if M is independent of (S, O), then $V(S \mid O, M) = V(S \mid O)$.

Independence means the probabilities multiply, $p(s, o, m) = p(s, o)\, p(m)$. Put that into Fact 1 with the observation (o, m):

$$V(S \mid O, M) = \sum_{o,m} \max_s p(s, o)\, p(m) = \sum_m p(m) \sum_o \max_s p(s, o) = V(S \mid O). \tag{11.3}$$

The middle step pulls $p(m)$ out of the maximum, which is allowed because $p(m) \geq 0$ does not depend on s; the last step uses that the $p(m)$ add up to 1. For instance, a coin flip you see before guessing does not change which smoking status is more likely, so it cannot change your success. This fact is what gives the non-member ceiling: a model trained without you is, as far as your record goes, such an independent observation.

11.2.3. Fact 3 — Processing an observation cannot help the best guesser

If you only see a transformed version $g(O)$ of the observation, you have at most the information in O itself, so you cannot do better: for any function g, $V(S \mid g(O)) \leq V(S \mid O)$.

The reason is that whoever sees o can compute $g(o)$ and then guess. A rule h that guesses from $g(o)$ is therefore also a rule on o, namely "apply g, then h", written $h \circ g$. So every rule on $g(O)$ is among the rules on O, and by Fact 1 none of those beats $V(S \mid O)$. For example, rounding a confidence of 0.62 to 0.6 can only merge observations that a guesser could have told apart; it never separates two that were the same. This is the guessing version of the data-processing inequality of Divergences, and the chapter uses it for confidence masking.

11.3. The attribute inference game

11.3.1. Context

"The attack recovered 78% of smoking statuses" sounds alarming, but it means nothing until you know two other numbers: how often the attribute could be guessed without the model, and how often the attack succeeds on people who were not in the training set. Yeom et al. (2018) made this precise with a game in the style of Threat models: a challenger sets up a hidden situation, the adversary gets a fixed view of it, and its guess is scored. Here the challenger flips a coin to decide whether the target record comes from the training set or fresh from the population. The coin is not a model of anything an attacker does; it is just a way to run both situations, member and non-member, under one set of rules, so that the attack's success in one can be compared with its success in the other.

11.3.2. Statement

The game (Yeom et al. 2018) is played for a training algorithm, a population $\mathcal{D}$, a training-set size n and an adversary $\mathcal{A}$. It has three steps.

1. The challenger draws a training set $S \sim \mathcal{D}^n$ and trains a model M on it.
2. It draws a hidden bit $b \in \{0, 1\}$, each value with probability $\frac{1}{2}$. If $b = 1$, it picks the target $z = (s, x, y)$ uniformly from S, so the target is a **member** of the training set. If $b = 0$, it draws $z \sim \mathcal{D}$ afresh, so the target is a **non-member**.
3. The adversary receives the target's public fields and label (x, y), but not s, and access to M. It may also know some or all of $\mathcal{D}$: that is its auxiliary knowledge. It outputs a guess $\hat{s}$ of the sensitive attribute.

The score should measure how much better the attack does on people who were in the training data than on people who were not, because that difference is what the model knows about its training records beyond the population. So the **attribute advantage** is the success rate on members minus the success rate on non-members,

$$\mathrm{Adv}^{\mathrm{attr}}(\mathcal{A}) = \Pr[\hat{s} = s \mid b = 1] - \Pr[\hat{s} = s \mid b = 0]. \tag{11.4}$$

Here $\Pr[\hat{s} = s \mid b = 1]$ is the probability that the guess equals the target's true attribute, over everything random in the game, given that the target is a member.

Attribute advantage is membership advantage. Any attribute attack can be turned into a membership attack, by a simple observation: if the attribute attack is right more often on members, then "the attribute attack was right" is itself evidence of membership. Build the membership adversary $\mathcal{B}$ that receives the whole record $z = (s, x, y)$ and M, runs $\mathcal{A}$ on (x, y) and M, and says "member" exactly when $\hat{s} = s$. A membership attack is described by two rates: its true positive rate (TPR), the probability that it says "member" for a member, and its false positive rate (FPR), the probability that it says "member" for a non-member. Its advantage is their difference. For $\mathcal{B}$ these are

$$\mathrm{TPR}_{\mathcal{B}} = \Pr[\hat{s} = s \mid b = 1], \qquad \mathrm{FPR}_{\mathcal{B}} = \Pr[\hat{s} = s \mid b = 0], \qquad \mathrm{TPR}_{\mathcal{B}} - \mathrm{FPR}_{\mathcal{B}} = \mathrm{Adv}^{\mathrm{attr}}(\mathcal{A}). \tag{11.5}$$

Membership attacks have a ceiling: no attack can tell two distributions apart better than their **total variation distance**, which is half the total absolute difference between their probabilities, $\mathrm{TV}(P, Q) = \frac{1}{2}\sum_v |p(v) - q(v)|$. Because $\mathcal{B}$ is a membership attack, its ceiling is also a ceiling on every attribute attack:

$$\mathrm{Adv}^{\mathrm{attr}}(\mathcal{A}) \le \mathrm{TV}(P_{\mathrm{in}}, P_{\mathrm{out}}), \tag{11.6}$$

where P_{in} and P_{out} are the distributions of the pair (z, M), the target record together with the model, when z is a member and when it is not.

11.3.3. Derivation

1. **The membership test fires when the attribute guess is right.** $\mathcal{B}$ is given s, so it can check whether $\hat{s} = s$, and it says "member" on exactly that event.
2. **Read off its rates.** Its TPR is the probability that it says "member" when $b = 1$, and it says "member" exactly when the guess is right, so the TPR is $\Pr[\hat{s} = s \mid b = 1]$. In the same way its FPR, the probability of "member" when $b = 0$, is $\Pr[\hat{s} = s \mid b = 0]$.
3. **Subtract.** TPR − FPR is the right-hand side of Equation 11.4 term by term.
4. **Bound by total variation.** Why can no test beat the total variation distance? A test that says "member" on a set A of views $v = (z, M)$ has $\mathrm{TPR} = P_{\mathrm{in}}(A)$ and $\mathrm{FPR} = P_{\mathrm{out}}(A)$, so its advantage is $\sum_{v \in A} (p_{\mathrm{in}}(v) - p_{\mathrm{out}}(v))$. That sum is largest when A contains exactly the views with a positive term, those more likely for members, and its largest value is the total excess of p_{in} over p_{out}. Since both distributions add up to 1, the total excess one way equals the total excess the other way, so it is half the total absolute difference, $\mathrm{TV}(P_{\mathrm{in}}, P_{\mathrm{out}})$. A randomized test averages such sets and does no better. (Hypothesis testing proves this in general.) Applied to $\mathcal{B}$, this gives Equation 11.6.

11.3.4. Interpretation

The subtraction in Equation 11.4 is what makes the advantage a measure of privacy. Any success that comes from knowing the population, for example "people with the disease tend to smoke", is the same for members and non-members, so it appears in both terms and cancels. What is left is what the model knows about its own training records.

The reduction also says which leak implies which. If a model reveals nothing about membership, in the sense that P_{in} and P_{out} are equal and their total variation is 0, then Equation 11.6 forces the attribute advantage of every adversary to 0. The converse fails. A membership leak does not have to come with an attribute leak: if s is irrelevant to the label, a model can memorize who was in its training set without its predictions depending on s at all, and then no attribute attack gains anything on members.

Because the advantage is a difference, it is not enough to report it alone. Report the success on members and on non-members, as a membership attack reports its TPR and FPR. The advantage by itself hides how much of the success is population inference, which is the part common to both.

Finally, the membership test $\mathcal{B}$ that the reduction produces is a weak one. It outputs a single bit, so when it says "member" the only evidence it carries is how much more often that happens for members than for non-members, the ratio TPR/FPR. An attack that is right on most people of the population, members or not, has an FPR close to its TPR and a ratio close to 1, so at a low FPR it is much weaker than the dedicated attacks of Membership inference (Exercise 3 in the exercise book).

11.3.5. Example: one model, 200 times

The toy experiment of this chapter (Section 11.7) has a binary sensitive attribute with prior $\Pr[s = 1] = 0.3$, ten public features and a binary label. A logistic regression with one weight vector per value of s is trained on $n = 100$ records, and this is repeated 200 times. Each time, the MAP inversion attack of Section 11.5 guesses s for the 100 members and for 100 fresh records. It is right on 0.7832 of members and 0.7262 of non-members, and the attribute advantage is the difference (computed before rounding the two rates),

$$\text{Adv}^{\text{attr}} = 0.7832 - 0.7262 = 0.0569, \tag{11.7}$$

with a standard error of 0.0045 over the 200 models. Read as a membership test, the same attack has TPR 0.7832 at FPR 0.7262. Its likelihood ratio when it fires is $0.7832/0.7262 = 1.08$: being right is barely more likely for a member than for a non-member. Its balanced accuracy, the average of its accuracy on members (the TPR) and on non-members ($1 - \text{FPR}$), is $\frac{1}{2}(\text{TPR} + 1 - \text{FPR}) = \frac{1}{2} + \frac{1}{2} \times 0.0569 = 0.5285$, hardly better than the $\frac{1}{2}$ of a coin flip. Most of its 78% success would be there even if the person had never been in the training data.

11.3.6. Check yourself

- An attack guesses the right attribute for 90% of members and 90% of non-members. What is its attribute advantage, and does it leak anything about individuals? *(Zero, since $0.9 - 0.9 = 0$. It is doing population inference: whatever it learned applies equally to people outside the training set.)*
- Why does the membership adversary $\mathcal{B}$ need the sensitive attribute s of the target, while $\mathcal{A}$ does not get it? *(The membership game hands the adversary the whole record and asks only whether it was used. $\mathcal{B}$ uses s to check whether $\mathcal{A}$'s guess was right.)*

11.4. The imputation baseline

11.4.1. Context

The game removes population inference by subtracting it. It is just as useful to measure it directly, because it tells you how much of an attack's success any model trained on the same population would give, or any published statistics about it. The measure is the success of the best guess of s from (x, y) when the population law is known. Statisticians call filling in a missing field from the others **imputation**, so this is the imputation baseline. Jayaraman & Evans (2022) found that many published attribute inference attacks do no better than imputation from similar data, and argued that this baseline is the one an attack must beat.

11.4.2. Statement

On a non-member, the model was trained without the target, so it is an independent observation (Fact 2) and the best any adversary can do is the best guess from (x, y) alone (Fact 1). That gives the **non-member ceiling**: for every adversary in the game,

$$\Pr[\hat{s} = s \mid b = 0] \le V(S \mid X, Y) = \sum_{x,y} \max_{s} p(s, x, y). \tag{11.8}$$

The second result is about attacks that never look at the model. A training record, picked at random from a training set drawn from the population, looks exactly like a fresh draw if you ignore the model, so such attacks cannot tell the two worlds apart. **Model-free attacks have zero advantage:** if $\hat{s}$ depends on (x, y) only, not on M, then $\Pr[\hat{s} = s \mid b = 1] = \Pr[\hat{s} = s \mid b = 0]$, and the Bayes imputation guess $\arg\max_s p(s \mid x, y)$, the MAP guess under the true population, attains the bound Equation 11.8 in both worlds.

11.4.3. Derivation

The ceiling.

1. **A non-member is independent of the model.** When $b = 0$, z is drawn afresh, separately from S. The model depends only on S and the randomness of training, so M is independent of $z = (s, x, y)$.
2. **So the model is a useless observation for the guess.** Fact 2 says that adding an observation independent of (s, x, y) does not change the best success: $V(S \mid X, Y, M) = V(S \mid X, Y)$.
3. **No guessing rule beats the MAP guess.** The adversary's rule is a function of (x, y, M), possibly randomized. By Fact 1 its success is at most the best success from that observation, $V(S \mid X, Y, M)$, which step 2 showed is $V(S \mid X, Y)$.

Model-free attacks.

1. **A member is a population record.** S consists of n independent draws from $\mathcal{D}$, and the target is one of them chosen uniformly at random. Whichever one is chosen, it was drawn from $\mathcal{D}$, so on its own (ignoring the model) the target has distribution $\mathcal{D}$, the same as a fresh draw.

2. **A rule that ignores M sees the same distribution in both worlds.** Its success is $\Pr[g(X,Y) = S]$ for a record drawn from $\mathcal{D}$ either way, so the two terms of Equation 11.4 are equal and the advantage is 0.
3. **Imputation attains the ceiling.** With g the MAP guess under $\mathcal{D}$, Fact 1 says this common success is $V(S \mid X, Y)$.

11.4.4. Interpretation

Below the ceiling, the model only teaches you the population. An adversary who does not know $\mathcal{D}$ can use the model to learn it, and so raise its success on non-members towards $V(S \mid X, Y)$, but it cannot go further. For a person outside the training data, the model is no more dangerous than a perfect description of the population.

This is the point of the "smoking causes cancer" example in Dwork & Roth (2014, §1). A study shows that smoking causes cancer. An insurer who reads it can now infer more about any smoker's health, whether or not that smoker took part in the study. The study disclosed a fact about the world, not about its participants, and a privacy definition that forbade this would forbid learning anything from data. The privacy question is the gap between members and non-members, which the ceiling isolates.

That does not make population inference harmless. Being guessed to be a smoker hurts whether or not the guess used your record. But no mechanism that limits what one record contributes can prevent it, since the harm would be there with your record removed. It is a question for policy and fairness, not for the attacks of this part.

Note also that the ceiling is a property of the population, not of the model, and it can be high. If x and y nearly determine s, every model and every statistic about the population "reveals" s. That is why attacks should be reported relative to it.

11.4.5. Example: the smoking table

Let $s = 1$ mean "smoker", x the age band (young or old) and $y = 1$ "has the disease". The population has $\Pr[s = 1] = 0.3$. Among non-smokers a fraction $\Pr[\text{old} \mid s] = 0.4$ is old, and among smokers 0.6. The disease probabilities $\Pr[y = 1 \mid x, s]$ are 0.1 (young non-smoker), 0.3 (old non-smoker), 0.4 (young smoker) and 0.7 (old smoker). The probability of a full record is the chance of the smoking status, times the chance of the age band given it, times the chance of the disease outcome given both; multiplying gives the joint table, Table 11.2. For example, $p(\text{smoker}, \text{old}, \text{disease}) = 0.3 \times 0.6 \times 0.7 = 0.126$, and $p(\text{non-smoker}, \text{young}, \text{no disease}) = 0.7 \times 0.6 \times 0.9 = 0.378$.

Table 11.2.: The smoking table: the joint distribution of smoking status s, age band x and disease y, with the column maxima in bold and the posterior probability of smoking in each column.

$p(s, x, y)$	young, no disease	young, disease	old, no disease	old, disease
non-smoker ($s = 0$)	**0.378**	0.042	**0.196**	0.084
smoker ($s = 1$)	0.072	**0.048**	0.054	**0.126**

$p(s, x, y)$	young, no disease	young, disease	old, no disease	old, disease
Pr[smoker $\mid x, y$]	0.1600	0.5333	0.2160	0.6000

The last row is each column's smoker entry divided by the column total; for young people with the disease, $0.048/(0.042 + 0.048) = 0.5333$. Now watch the best guess improve as the guesser sees more.

With no observation, the best guess is "non-smoker", right with probability $V(S) = 0.7$, as in Fact 1.

With age alone, the guesser adds the two columns of each age band. Young smokers have probability $0.3 \times 0.4 = 0.12$ and young non-smokers $0.7 \times 0.6 = 0.42$, so a young person is a smoker with probability $0.12/0.54 = 0.2222$. Old smokers have $0.3 \times 0.6 = 0.18$ and old non-smokers $0.7 \times 0.4 = 0.28$, so an old person is a smoker with probability $0.18/0.46 = 0.3913$. Both are below $\frac{1}{2}$, so the guess stays "non-smoker" in both age bands and $V(S \mid X) = 0.7$. Age changes the odds but never the decision, so it adds nothing to the success.

With age and disease, the posterior crosses $\frac{1}{2}$ in the two disease columns (Figure 11.1), so the MAP guess becomes "smoker" there. By Fact 1 the success is the sum of the largest entry in each column, the bold ones: $V(S \mid X, Y) = 0.378 + 0.048 + 0.196 + 0.126 = 0.748$.

The **min-entropy leakage** of Information theory turns this gain into bits: it is the base-2 logarithm of how many times more likely the best guess is to succeed with the observation than without. Here the disease label, given age, raises the success from 0.7 to 0.748, a leakage of $\log_2(0.748/0.7) = 0.0957$ bits.

Every one of these numbers comes from the population. A model trained on 10,000 patients from it, if it predicts well, lets an insurer reach 0.748 on *anyone* drawn from the population, and the ceiling Equation 11.8 says nobody outside the training set can be attacked more successfully than that.

11.4.6. Check yourself

- In the smoking table, which cell would the imputation guess get wrong most often, measured by probability mass? *(Old, with disease: the guess there is "smoker", which is wrong on the 0.084 of non-smokers in that cell, the largest non-bold entry.)*
- Does the ceiling Equation 11.8 hold for an adversary that sees the whole training set S, not only the model? *(Yes. The argument only needs the observation to be independent of the fresh target, and S is.)*

11.5. Model inversion: the MAP attack

11.5.1. Context

The imputation guess needs $p(y \mid x, s)$, the link between the label and the sensitive attribute, and the adversary usually does not know it. But a model that predicts y from x and s is an estimate of exactly that link. Fredrikson et al. (2014) used this in a study of warfarin dosing: a linear

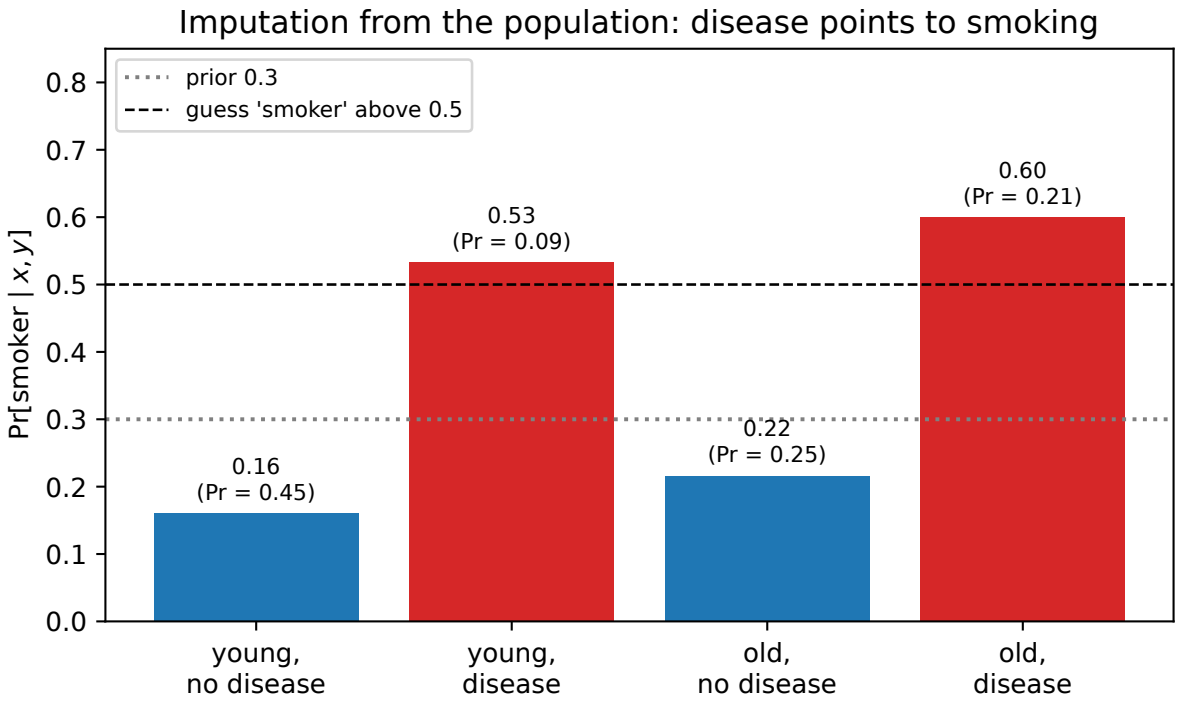


Figure 11.1.: The posterior probability of smoking in each cell of the smoking table, with each cell's probability in brackets. Red bars cross $\frac{1}{2}$, where imputation guesses "smoker"; together they add 0.048 to the 0.7 of the blind guess.

model predicted the stable dose of the drug from demographics and two genetic markers, and an adversary who knew a patient's demographics and dose could infer their genotype. They called the attack **model inversion**: run the model "backwards" from its output to an input. Fredrikson, Jha & Ristenpart (2015) extended it to models that release **confidences**, the predicted probability of each label, and showed that confidences make the attack much stronger.

11.5.2. Statement

The idea is Bayes' rule. The value of s that best explains what you see is the one with the largest product of "how plausible is s for someone with these features" and "how likely is the observed label if the attribute were s". The adversary supplies the first factor from what it knows about the population, and gets the second by querying the model with each candidate value of s. The **MAP inversion attack** guesses

$$\hat{s}_{\text{inv}}(x, y) = \arg\max_{s} \; p(s \mid x)\, f_y(x, s). \tag{11.9}$$

Here $p(s \mid x)$ is what the adversary knows about how s relates to the public features, and $f_y(x, s)$ is the model's confidence in the observed label y when it is fed the candidate s. Fredrikson et al. used the marginal $p(s)$ in place of $p(s \mid x)$, which treats s as independent of x.

If the model's confidences were exactly the population's probabilities, the second factor would be the true likelihood, and the attack would be the imputation guess. A model with that property is called **calibrated**. So, **for a calibrated model, inversion is imputation**: if $f_y(x, s) = p(y \mid x, s)$ for all s, x, y, then $\hat{s}_{\text{inv}}$ is the Bayes imputation guess, and its success is $V(S \mid X, Y)$ on members and non-members alike.

11.5.3. Derivation

1. **Bayes' rule.** The joint probability of s and y given x can be built in two stages, first s from x and then y from both: $p(s, y \mid x) = p(s \mid x)\, p(y \mid x, s)$. Dividing by $p(y \mid x)$ turns it into the posterior of s:
$$p(s \mid x, y) = \frac{p(s \mid x)\, p(y \mid x, s)}{p(y \mid x)}. \tag{11.10}$$
2. **The denominator does not depend on s.** It is the same number for every candidate, so it cannot change which candidate is largest: $\arg\max_s p(s \mid x, y) = \arg\max_s p(s \mid x)\, p(y \mid x, s)$.
3. **Plug in the model.** Replace the unknown $p(y \mid x, s)$ by the model's estimate $f_y(x, s)$. This is Equation 11.9.
4. **Calibrated case.** If $f_y = p(y \mid x, s)$ exactly, step 3 changes nothing and the guess is the MAP guess of Fact 1, with success $V(S \mid X, Y)$. The guess then uses nothing about M beyond the population's probabilities, so it is a model-free attack, and the argument of Section 11.4 gives equal success on members and non-members.

11.5.4. Interpretation

The attack is Bayes' rule with the model as the likelihood, so everything it does well or badly comes from how the model's $f_y(x, s)$ differs from the population's $p(y \mid x, s)$. Two differences matter, one on members and one on non-members.

On members, the model is too sure of the truth. Training lowers the loss $-\ln f_y(x, s)$ on each training record at its *true* s, which pushes $f_y(x, s)$ up towards 1. The other candidate value s' was never paired with this record during training, so $f_y(x, s')$ stays closer to an honest prediction. (In the toy model below this holds exactly: each value of s has its own weight vector, so the record never touched the weights used at s'.) The ratio $f_y(x, s)/f_y(x, s')$ that decides the attack is therefore tilted towards the truth for members. This is the generalization gap of Membership inference, the difference between a model's loss on its training records and on fresh ones, seen through the sensitive attribute. In the toy experiment the mean log-loss is 0.2965 on members and 0.6690 on non-members.

On non-members, an overfit model does worse than imputation. Its confidences carry the noise of a small training set rather than the population's exact probabilities, so the attack's guesses fall below the ceiling.

More data closes both gaps. As n grows, the model approaches $p(y \mid x, s)$ and both success rates approach $V(S \mid X, Y)$ (Figure 11.3). The population inference gets better and the individual leak gets smaller.

The same idea can invert the whole input rather than one field. Fredrikson, Jha & Ristenpart (2015) also attacked a face recognition model in which each class is one person. They searched for the input image that maximizes the confidence of a chosen class, by gradient descent on the image, and obtained a recognizable, blurred image of that person. It is the same idea with s replaced by the whole input, and the prior $p(s \mid x)$ replaced by an image regularizer that keeps the search among plausible images. What it recovers is a typical input of the class, which is population inference about the class; it is a privacy problem because the class is a single person.

11.5.5. Example: walking through the inversion

On the smoking table, with a calibrated model. Take an old patient with the disease. From the age-alone step of the smoking-table example, the adversary knows $\Pr[\text{smoker} \mid \text{old}] = 0.3913$, so $\Pr[\text{non-smoker} \mid \text{old}] = 1 - 0.3913 = 0.6087$. It queries the model at both values of s, and a calibrated model returns the disease probabilities of the population: $f_1(\text{old}, \text{smoker}) = 0.7$ and $f_1(\text{old}, \text{non-smoker}) = 0.3$. The scores of Equation 11.9 are $0.3913 \times 0.7 = 0.2739$ for "smoker" and $0.6087 \times 0.3 = 0.1826$ for "non-smoker", so it guesses "smoker". Dividing the smoker score by the sum of the two, which is the normalization that step 1 of the derivation leaves out, gives $0.2739/(0.2739 + 0.1826) = 0.6000$, the posterior of Table 11.2. The attack is imputation.

On the toy model. The population draws s with $\Pr[s = 1] = 0.3$ and ten standard normal features, the first shifted by 0.5 when $s = 1$. The label follows a logistic model whose weights depend on s. Guessing from x alone is right 0.7045 of the time, and Bayes imputation, which knows the true logistic weights, is right 0.7517 of the time (both on a million fresh records). Over 200 models trained on $n = 100$ records, imputation is right on 0.7508 of members and 0.7555 of non-members. The difference, -0.0046, is about one standard error from zero, as the

model-free argument predicts. MAP inversion, which uses the trained model's confidences, is right on 0.7832 of members, $0.7832 - 0.7517 = 0.0315$ *above* the imputation ceiling, and on 0.7262 of non-members, 0.0254 below it (computed before rounding).

The member success above the ceiling is something no population-level knowledge could produce, since the ceiling is the best that perfect population knowledge achieves. Figure 11.2 shows where it comes from. It plots the attack's evidence for the true value s over the other value s', the log of the ratio of their two scores, $\ln \frac{p(s|x) f_y(x,s)}{p(s'|x) f_y(x,s')}$. The attack is right when this is positive. For members the distribution has a thinner left tail: training has removed most of the cases where the model strongly prefers the wrong value. Figure 11.3 repeats the game for training sets from 25 to 1600 records. At $n = 25$ the attack is right on 0.8420 of members and 0.6810 of non-members, an advantage of $0.8420 - 0.6810 = 0.1610$. At $n = 400$ the advantage is 0.0111, and at $n = 1600$ it is 0.0052.

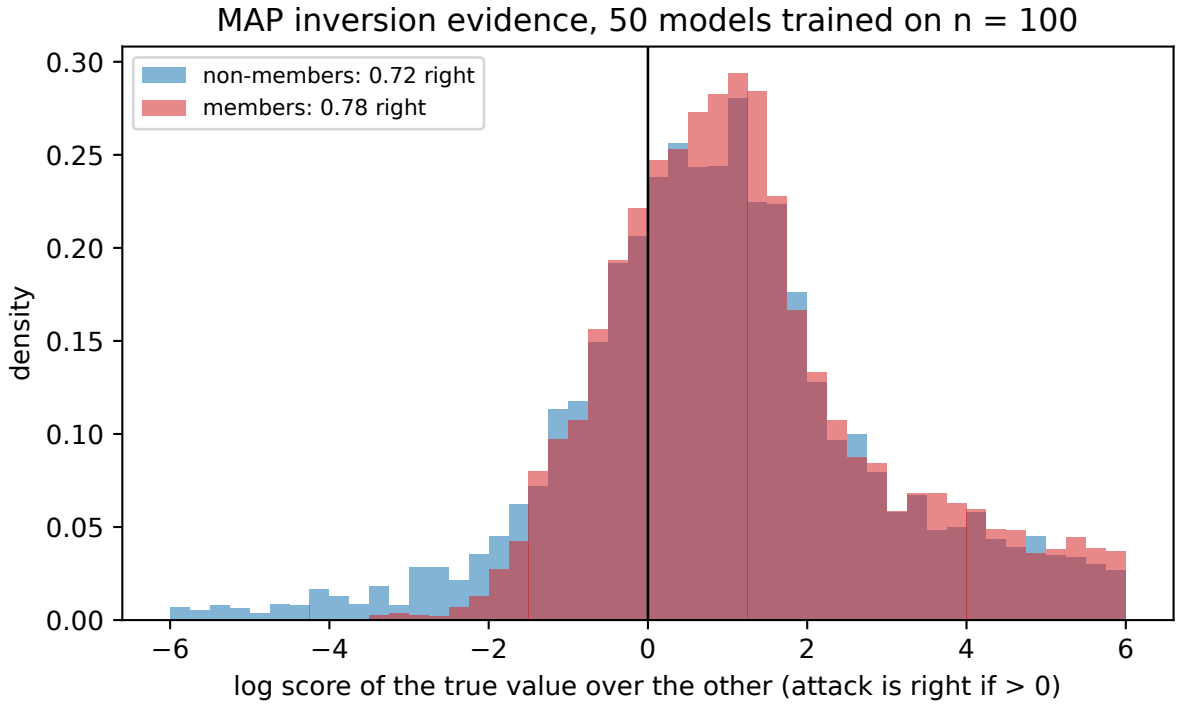


Figure 11.2.: The evidence of MAP inversion for the true attribute, the log of the score of the true value over the other, for members and non-members of 50 models trained on 100 records each. The attack is right when the evidence is positive; members have far fewer large negative values.

11.5.6. Check yourself

- If the model ignored s entirely, $f_y(x, s) = f_y(x, s')$, what would MAP inversion guess? *(The MAP guess from x alone, $\arg\max_s p(s \mid x)$: the confidence is the same factor for every candidate in Equation 11.9, so it cannot change which candidate wins.)*
- Why does using the marginal $p(s)$ instead of $p(s \mid x)$ in Equation 11.9 lose success even with a calibrated model? *(It drops the evidence that x carries about s, so the score is no longer*

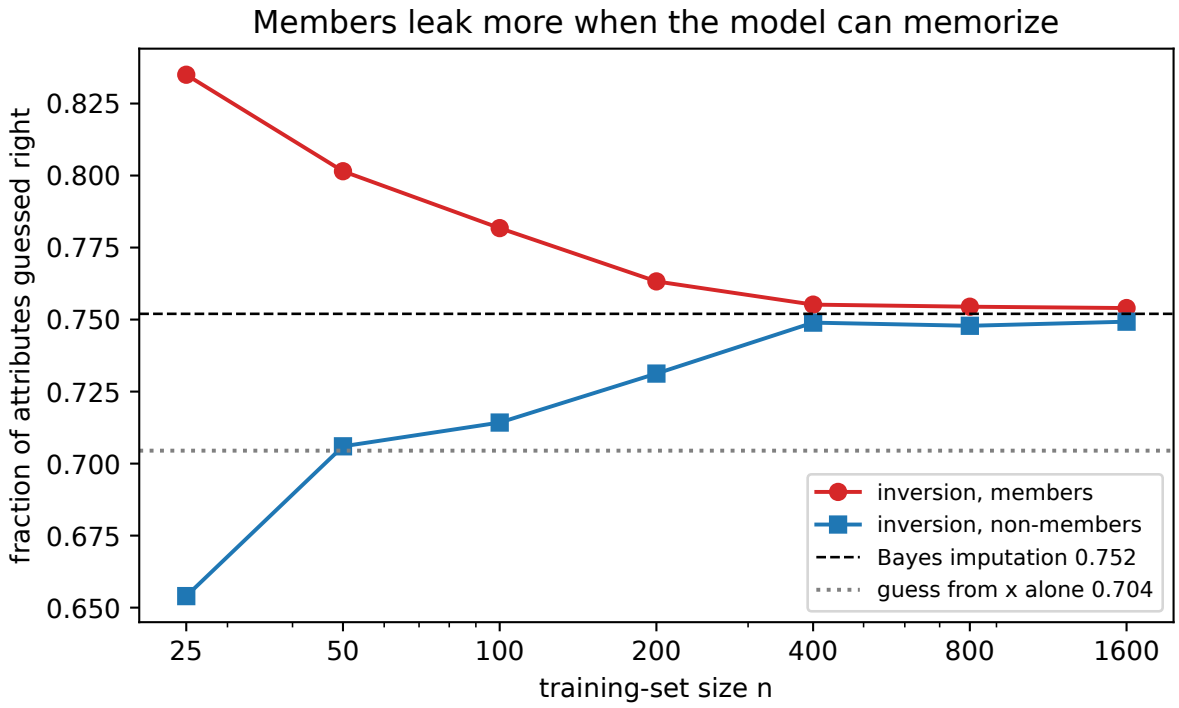


Figure 11.3.: Success of MAP inversion on members and non-members against the training-set size n, 40 models per point. Members start above the Bayes imputation ceiling and non-members below it; both converge to the ceiling as the model approaches the population.

proportional to $p(s \mid x, y)$. Exercise 2 in the exercise book computes the loss on the smoking table.)

11.6. Confidence masking

11.6.1. Context

If confidences make inversion stronger, why release them? Fredrikson, Jha & Ristenpart (2015) suggested rounding the confidences an interface returns, and many deployed interfaces return only the top label. Such **confidence masking** is cheap and leaves the predicted label unchanged. Does it close the leak?

11.6.2. Statement

Masking only coarsens what the adversary sees, so by Fact 3 it can never help the best attacker. To write this down, let $F = \big(f(x, s)\big)_s$ be the list of answers the adversary gets by querying the model at every candidate value of s, where each answer $f(x, s)$ is the model's vector of confidences over the labels. Let g be any masking function applied to these answers: rounding, keeping only the top label, adding noise. Then the best attack on the masked answers does no better than the best attack on the full ones, on members and on non-members alike:

$$V(S \mid X, Y, g(F)) \le V(S \mid X, Y, F). \tag{11.11}$$

The inequality is about the *best* attacker in each world separately. It says nothing about the *difference* between members and non-members, and masking need not shrink it.

11.6.3. Derivation

1. **A masked attack is an unmasked one.** Anyone holding the full answers can apply the mask themselves. So a rule h that guesses from $(x, y, g(F))$ is also the rule $(x, y, F) \mapsto h(x, y, g(F))$ on the unmasked answers.
2. **The best unmasked rule is at least as good.** This is Fact 3, with the observation $O = (X, Y, F)$ and the processing function $(x, y, F) \mapsto (x, y, g(F))$. It gives Equation 11.11, separately for $b = 0$ and for $b = 1$, since the argument never used which world the target came from.
3. **Why the difference can survive.** Write V_{in} and V_{out} for the best success on members and on non-members. Masking lowers both, by amounts that depend on g and on how the confidences of members and non-members differ. Nothing forces the two drops to be equal, so $V_{\text{in}} - V_{\text{out}}$ can rise or fall.

11.6.4. Interpretation

Labels alone still leak, for the same reason confidences do. With only the top label, the adversary still sees, for each candidate s, whether the predicted label $\text{label}(x, s)$ equals the known label y. On a member, the model usually predicts the training label at the true s, because it fit that record. At the other value it gives an honest prediction, which matches y less often. So "the label matches only at one candidate" points to that candidate, and on members it usually points to the truth. Non-members rarely get this help: the two predicted labels usually agree, and the attack falls back on the guess from x alone. Label-only membership inference (Choquette-Choo et al. 2021) exploits the same asymmetry.

This is why masking hurts the population inference more than the individual leak. The toy experiment below shows it: non-member success drops almost to the x-only guess, while member success stays above the imputation ceiling.

Do not read too much into small changes either way. The inequality is about the optimal attacker, and MAP inversion with an overfit model is not optimal, so a particular masking can even raise its success slightly. That does not contradict Equation 11.11.

Above all, masking is obfuscation, not a guarantee. It removes one attack's input without bounding what any attack can learn about a member. Part 3 introduces differential privacy, which bounds the member/non-member difference itself, for every attacker and every output format.

11.6.5. Example: rounding and labels on the toy model

The same 200 models of Section 11.5, attacked through three interfaces, give Table 11.3 and Figure 11.4.

Table 11.3.: MAP inversion success through three interfaces, averaged over 200 models trained on 100 records each. Guessing from x alone gives 0.7045 and Bayes imputation 0.7517.

Interface	Members	Non-members	Advantage
Full confidences	0.7832	0.7262	0.0569
Confidences rounded to one decimal	0.7836	0.7253	0.0583
Top label only	0.7741	0.7092	0.0650

Rounding to one decimal changes almost nothing. The attack only compares the scores of the two candidates, and most confidences are far enough from the rounding boundaries that the comparison keeps its sign. With labels only, non-member success drops from 0.7262 to 0.7092, close to the 0.7045 of guessing from x alone. Member success drops less, to 0.7741, still $0.7741 - 0.7517 = 0.0224$ above the imputation ceiling. So the advantage *grows*, from 0.0569 to 0.0650. The interface now leaks less about the population and as much as before about the people in the training set.

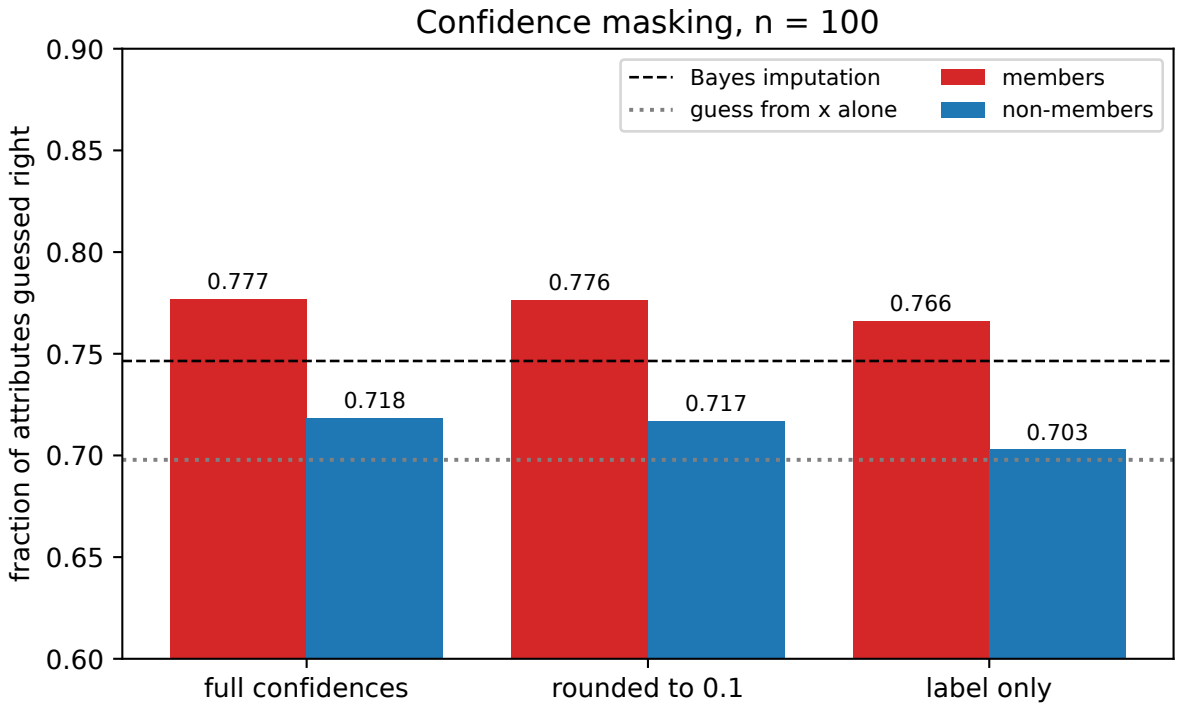


Figure 11.4.: MAP inversion success on members and non-members through three interfaces, for models trained on 100 records. The dashed line is Bayes imputation, the dotted line the guess from the public features alone.

11.6.6. Check yourself

- Can adding Gaussian noise to the returned confidences increase the best attacker's success? *(No. Noise is a randomized function of F, so a rule on the noisy answers is a randomized rule on F: it draws the noise itself and then guesses. Fact 1 shows a randomized rule does no better than the best deterministic one.)*
- Why does the label-only attack on a non-member usually fall back on the guess from x alone? *(For a fresh record, the predicted labels at the two candidate values usually agree. Both then match y or both miss, the masked confidences tie, and the tie is broken by $p(s \mid x)$.)*

11.7. Code: the game on a toy model

The script code/attribute_inference.py computes every number in this chapter and draws every figure. It works out the smoking table exactly. For the toy model it samples the population, trains a ridge-regularized logistic regression with one weight vector per value of s by Newton's method, and plays the game: each run trains a model on n records and attacks those n members and n fresh non-members with MAP inversion, Bayes imputation and the guess from x alone. The real attacks of Fredrikson et al. used a warfarin dosing model and face recognition networks; the toy keeps the structure (a sensitive input, a known label, a model queried at each candidate value) and runs in a few seconds. From the repository root:

```
uv run python code/part02-attacks/attribute_inference.py
```

Output (seed 0), summarized in Table 11.4:

Table 11.4.: The script's output with the default seed. Pairs are success on members / non-members.

Experiment	Result
Smoking table	$V(S) = 0.7000$; $V(S \mid X) = 0.7000$; $V(S \mid X, Y) = 0.7480$; leakage 0.0957 bits; posteriors 0.1600, 0.5333, 0.2160, 0.6000
Toy population, 10^6 fresh records	Guess from x alone 0.7045; Bayes imputation 0.7517
Game, $n = 100$, 200 models: mean log-loss	Members 0.2965; non-members 0.6690; gap 0.3725
Guess from x alone	Members 0.7017; non-members 0.7089
Bayes imputation	Members 0.7508; non-members 0.7555; advantage −0.0046
MAP inversion	Members 0.7832; non-members 0.7262; advantage 0.0569 (standard error 0.0045)
MAP inversion against the ceiling	Members +0.0315; non-members −0.0254
As a membership test	TPR 0.7832; FPR 0.7262; balanced accuracy 0.5285

Experiment	Result
Masking	Rounded: 0.7836 / 0.7253, advantage 0.0583. Label only: 0.7741 / 0.7092, advantage 0.0650
Training-set size, 40 models each	$n = 25$: 0.8420 / 0.6810. $n = 400$: 0.7611 / 0.7500. $n = 1600$: 0.7550 / 0.7497

tests/test_attribute_inference.py checks that the output still matches.

What to take from it. First, the ceiling holds: non-member success never exceeds Bayes imputation, at any training-set size, as the independence argument of Section 11.4 requires. Second, members beat it. At $n = 100$ the attack is 0.0315 above the best population-level guess on members, and since no knowledge of the population can do better than that guess, only individual leakage can explain the excess. Third, the model-free attacks show no advantage, as they should. Imputation and the x-only guess differ between members and non-members by less than 0.01, which is sampling noise: each rate is an average over $200 \times 100 = 20{,}000$ records, and a success rate near 0.75 averaged over that many records has a standard error of about $\sqrt{0.75 \times 0.25/20{,}000} = 0.0031$. Fourth, overfitting drives the leak. The advantage falls from 0.1610 at $n = 25$ to 0.0052 at $n = 1600$, as the gap between member and non-member loss closes. Finally, masking is not a fix: label-only outputs lower success on both members and non-members, and the advantage stays.

11.8. References

- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (IEEE CSF 2018). The attribute inference game with members and non-members, its advantage, the reduction between attribute and membership advantage, and the link to the generalization gap.
- Fredrikson, Lantz, Jha, Lin, Page & Ristenpart, "Privacy in Pharmacogenetics: An End-to-End Case Study of Personalized Warfarin Dosing" (USENIX Security 2014). The first model inversion attack: inferring genetic markers from a dosing model and a patient's demographics and dose.
- Fredrikson, Jha & Ristenpart, "Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures" (CCS 2015). Inversion from confidences for decision trees and for face recognition, and rounding as a countermeasure.
- Wu, Fredrikson, Jha & Naughton, "A Methodology for Formalizing Model-Inversion Attacks" (IEEE CSF 2016). Model inversion as a game, with the distinction between what a model reveals about its training data and about the population.
- Jayaraman & Evans, "Are Attribute Inference Attacks Just Imputation?" (CCS 2022). The imputation baseline made central: many attacks do no better, and what it takes to beat it.
- Choquette-Choo, Tramèr, Carlini & Papernot, "Label-Only Membership Inference Attacks" (ICML 2021). Why returning only labels does not stop inference about members.
- Salem, Cherubin, Evans, Köpf, Paverd, Suri, Tople & Zanella-Béguelin, "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023). Attribute inference among the other privacy games, with their relationships.

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §1. The smoking example and why learning about the population is not a privacy breach of the individual.

11.9. Cheat sheet

A record is (s, x, y); $f_y(x, s)$ is the model's confidence in label y; $V(S \mid O) = \sum_o \max_s p(s, o)$ is the success of the best guess of s from O. Table 11.5 collects the results.

Table 11.5.: Every result in this chapter.

Result	Statement	Where it comes from
Attribute advantage	$\Pr[\hat{s} = s \mid \text{member}] - \Pr[\hat{s} = s \mid \text{non-member}]$	Yeom et al.'s game
Reduction to membership	Fire when $\hat{s} = s$: TPR − FPR = attribute advantage $\leq \mathrm{TV}(P_{\text{in}}, P_{\text{out}})$	Best advantage is total variation
Non-member ceiling	$\Pr[\hat{s} = s \mid \text{non-member}] \leq V(S \mid X, Y)$	Model independent of a fresh record; MAP is optimal
Model-free attacks	Equal success on members and non-members; imputation attains $V(S \mid X, Y)$	A member is marginally a population draw
MAP inversion	$\hat{s} = \arg\max_s p(s \mid x)\, f_y(x, s)$	Bayes' rule with the model as likelihood
Calibrated model	Inversion = imputation, zero advantage	$f_y = p(y \mid x, s)$
Masking	$V(S \mid X, Y, g(F)) \leq V(S \mid X, Y, F)$, but the member/non-member gap can grow	Processing cannot help the best guesser

Rules of thumb

- Report success on members and non-members, and the imputation baseline. A single success rate says nothing about privacy.
- Success up to the imputation baseline is population inference; any model or statistic of the population gives it.
- Success on members beyond what non-members get is the individual leak. It comes from overfitting and shrinks as the training set grows.
- An attribute attack is a membership test that outputs one bit. It is weak at low false positive rates.
- Rounding confidences or returning labels only changes the attacks, not the fact that the model has memorized its training records.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

12. Property inference

12.1. Why this matters

A hospital trains a readmission model on its 1,000 patients and shares it with a research partner. Suppose you have checked that no attack on the model can tell, much better than a coin flip, whether any one patient was in the training set. The partner can still learn whether 45% or 55% of the hospital's patients are over 65, and be right almost every time. A company that shares a fraud model may reveal the share of its customers in a market it has not announced. A model trained on a lab's machines may reveal whether those machines had been patched. None of these secrets is about a record. Each is a **property** of the training set as a whole: a statistic $t(D)$ of the dataset D, such as the fraction of records that belong to some subgroup.

Property inference is the attack that recovers such a statistic from a trained model. Ateniese et al. (2015) introduced it. In their examples, the attacker learned whether a speech recognizer had been trained on speakers with a particular accent, and whether a network-traffic classifier had seen a particular kind of traffic. Ganju et al. (2018) carried the attack to fully connected neural networks, and showed that the attacker must first deal with a nuisance: the same network can be written with its hidden units in any order.

This chapter works through four questions, each building on the one before. It starts by writing the attack as a game, because only then can you ask what the most any attacker can win is. With that ceiling in hand, it explains why a property leaks when no single record does, and why the leak grows with the size of the dataset. Real models have no formula for their distribution, so it then shows how a practical attacker builds its test by training models of its own, and predicts how well that test does. Finally it turns to neural networks, whose raw weights hide the property from a simple attacker, and finds the representation that brings it back.

12.1.1. The one idea: a model is a noisy statistic, and a property moves it further than any record does

A trained model θ is a function of its training set, plus training noise. Think of it as an average over the n records. Changing one record out of n moves an average by about $1/n$. The randomness of which records were sampled moves it by about $1/\sqrt{n}$, the usual size of the error of an average. Since $1/n$ is much smaller than $1/\sqrt{n}$ when n is large, a single record is lost in the noise. That is why membership inference gets *harder* with more data.

A property is different. Raising the subgroup's share from t_0 to t_1 changes about $n|t_1 - t_0|$ records at once. Each moves the average by about $1/n$, so together they move θ by an amount proportional to $|t_1 - t_0|$ that does not shrink with n. Against noise of size $1/\sqrt{n}$, the signal-to-noise ratio is about $|t_1 - t_0|/(1/\sqrt{n}) = \sqrt{n}\,|t_1 - t_0|$, so property inference gets *easier* with more data.

Everything else in this chapter is that argument made precise. The attack is a hypothesis test that decides whether the model was trained under t_0 or under t_1, and the attacker approximates the best such test by training models of its own. Table 12.1 lists the results.

Table 12.1.: The results of this chapter and where each is used.

Result	Where it is used
The property game; advantage at most the total variation (TV) distance between the models, and between the datasets	Stating what a property attack achieves and what no attack can exceed
Membership advantage $\approx 1/\sqrt{n}$, property advantage $\to 1$ for the same released count	Why protecting records does not protect properties
Trained parameters $\approx \mathcal{N}(\theta(t), \Sigma_1/n)$; best accuracy $\Phi(\kappa\sqrt{n}\,\lvert t_1 - t_0\rvert/2)$	Predicting an attack's accuracy from the model and the data; the $\sqrt{n}\,\lvert t_1 - t_0\rvert$ scaling law
Shadow models and a linear meta-classifier	The practical attack, when the law of θ is unknown
Hidden-unit symmetries; invariant representations	Attacking neural networks through their weights or through their outputs

What this chapter assumes. Every tool the derivations need is restated where it is used, so you can read this chapter on its own. For more detail: the game view, the advantage (how much more often an attack says 1 when the answer is 1 than when it is 0) and the fact that processing the view cannot help the attacker are in Threat models and privacy games. Shadow models first appear in Membership inference. The TV distance, the data processing inequality and sufficient statistics are in Divergences. The Neyman–Pearson lemma and the Gaussian trade-off curve are in Hypothesis testing. The binomial distribution and its normal approximation are in Probability distributions.

Conventions. Records come from a population that mixes a majority group A and a subgroup B. Write π_A and π_B for the distributions of a record from each group. A population in which a fraction t of people belong to B is the mixture $\pi_t = (1-t)\pi_A + t\,\pi_B$: to draw a record, first pick B with probability t and A otherwise, then draw from that group. A dataset D is n independent records from π_t, written $D \sim \pi_t^n$. A training algorithm $\mathcal{T}$ maps it to parameters $\theta = \mathcal{T}(D)$, possibly using random choices of its own (initialization, mini-batch order). $P_{n,t}$ is the distribution of θ, over both the random data and the training randomness. The secret is which of two shares, t_0 or t_1, was used. A challenger picks it with a hidden bit b, which is 0 or 1 with probability $\frac{1}{2}$ each (a **fair bit**); $b = 0$, share t_0, is the null hypothesis, and $b = 1$, share t_1, the alternative. The attacker outputs a guess b'. Its **true positive rate** (TPR), $\mathrm{TPR} = \Pr[b' = 1 \mid b = 1]$ is how often it says 1 when the bit is 1, and its **false positive rate** (FPR), $\mathrm{FPR} = \Pr[b' = 1 \mid b = 0]$ is how often it says 1 when the bit is 0. Its **advantage** is $\mathrm{TPR} - \mathrm{FPR}$: how much more often it says 1 in the world where 1 is right. Its **accuracy** is $\Pr[b' = b]$, the probability that the guess matches the hidden bit, whichever value the bit has. It is not the probability that b is 1. With a fair bit, half the games have $b = 1$, where the attacker wins with probability TPR, and half have $b = 0$, where it wins with probability $1 - \mathrm{FPR}$, so $\Pr[b' = b] = \frac{1}{2}\mathrm{TPR} + \frac{1}{2}(1 - \mathrm{FPR}) = \frac{1}{2}(1 + \text{advantage})$. An attacker who ignores the model says 1 equally often in both worlds, so its advantage is 0 and its

accuracy is $\frac{1}{2}$: that is the coin-flip baseline. $\Phi(z) = \Pr[Z \le z]$ is the standard normal distribution function, for $Z \sim \mathcal{N}(0,1)$.

12.2. Toolkit: three facts every derivation uses

12.2.1. Fact 1 — The best advantage is the total variation distance

The attacker sees a view, such as the trained parameters, whose distribution is P when $b = 0$ and Q when $b = 1$. Some views are more likely in one world than in the other, and that difference is all the attacker has to go on. The best it can do is say 1 exactly on the views that are more likely under Q. How well that works is measured by the **total variation distance**, half the total absolute difference between the two probability tables p and q:

$$\mathrm{TV}(P,Q) = \tfrac{1}{2}\sum_x |p(x) - q(x)|. \tag{12.1}$$

(For continuous views, the sum is an integral over densities.) The fact is that every attack has advantage at most $\mathrm{TV}(P,Q)$, and the **likelihood-ratio test** "guess 1 when $q > p$" attains it. With a fair bit, the best accuracy is therefore $\frac{1}{2} + \frac{1}{2}\mathrm{TV}(P,Q)$.

The argument takes two lines. An attack may randomize, so describe it by $\phi(x)$, the probability that it says 1 on view x. Its TPR is $\sum_x \phi(x)q(x)$ and its FPR is $\sum_x \phi(x)p(x)$, so its advantage is $\sum_x \phi(x)(q(x) - p(x))$. Each term is largest when $\phi(x) = 1$ where $q(x) > p(x)$ and $\phi(x) = 0$ elsewhere, and the maximum is the total excess of q over p. Both tables sum to 1, so the excess of q over p equals the excess of p over q, and each is half the total absolute difference: the TV distance. This is the best-advantage result of Hypothesis testing, restated for games in Threat models and privacy games.

A Gaussian case gives a number you will see again. Let $P = \mathcal{N}(0,1)$ and $Q = \mathcal{N}(1,1)$. The two densities cross halfway between the means, at $\frac{1}{2}$, and $q > p$ to the right of it, so the best test says 1 when the view exceeds $\frac{1}{2}$. Its TPR is $\Pr[\mathcal{N}(1,1) > \frac{1}{2}] = \Phi(\frac{1}{2})$, because $\frac{1}{2}$ is half a standard deviation below the mean 1. Its FPR is $\Pr[\mathcal{N}(0,1) > \frac{1}{2}] = 1 - \Phi(\frac{1}{2})$. So $\mathrm{TV} = \Phi(\frac{1}{2}) - (1 - \Phi(\frac{1}{2})) = 2\Phi(\frac{1}{2}) - 1 = 0.3829$, and the best accuracy is $\frac{1}{2} + \frac{1}{2}(0.3829) = 0.6915$, which is $\Phi(\frac{1}{2})$ itself. This fact sets the ceiling on the property game and turns every advantage in this chapter into an accuracy.

12.2.2. Fact 2 — Training cannot add information about the property

The attacker sees the model, not the data, and the model is computed from the data. Whatever the attacker could learn from θ, someone holding D could learn too: they could train θ themselves, using the same algorithm and fresh random choices, and run the attacker's test on it. So the best test on θ can be no better than the best test on D. In terms of Fact 1, if $\theta = \mathcal{T}(D)$, where $\mathcal{T}$ may use randomness that does not depend on b, then

$$\mathrm{TV}(P_{n,t_0}, P_{n,t_1}) \le \mathrm{TV}(\pi^n_{t_0}, \pi^n_{t_1}). \tag{12.2}$$

This is the **data processing inequality** of Divergences: passing a view through any channel whose behaviour does not depend on the secret cannot increase its TV distance. It is the same as the "processing cannot help the adversary" fact of Threat models and privacy games.

Equality holds when θ is a **sufficient statistic** for t, which means that the likelihood ratio between the two worlds can be computed from θ alone, so θ keeps everything in D that bears on t. The two extreme cases show the range. A model that ignores its data entirely has $P_{n,t_0} = P_{n,t_1}$, and TV 0. Releasing the dataset itself attains the ceiling. Fact 2 gives the ceiling in the game section, and it is why a released count turns out to be the worst case.

12.2.3. Fact 3 — Two Gaussians with a shared covariance: the best test is linear

The trained parameters of many models are approximately Gaussian, as Section 12.5 shows. So the most useful special case of Fact 1 is two Gaussians in $\mathbb{R}^d$ with the same covariance Σ and different means, $P = \mathcal{N}(m_0, \Sigma)$ and $Q = \mathcal{N}(m_1, \Sigma)$. How hard are they to tell apart? In one dimension, the answer depends on how far apart the means are, measured in standard deviations. In d dimensions the noise can be larger in some directions than in others, so a shift along a noisy direction counts for less than the same shift along a quiet one. The **Mahalanobis distance** measures the shift in exactly this way, in standard deviations along its own direction:

$$\Delta = \sqrt{(m_1 - m_0)^\top \Sigma^{-1} (m_1 - m_0)}. \tag{12.3}$$

For $d = 1$ and $\Sigma = \sigma^2$, it is $|m_1 - m_0|/\sigma$.

The fact has two parts. First, the log of the likelihood ratio is a linear function of θ: it measures how far θ lies past the midpoint of the two means, along a direction w,

$$\ln \frac{q(\theta)}{p(\theta)} = w^\top \Big(\theta - \frac{m_0 + m_1}{2}\Big), \qquad w = \Sigma^{-1}(m_1 - m_0). \tag{12.4}$$

So the best test of Fact 1, "say 1 when $q > p$", is a linear test: say 1 when θ lies on m_1's side of a hyperplane through the midpoint. Second, its accuracy with a fair bit depends only on the Mahalanobis distance,

$$\Phi(\Delta/2). \tag{12.5}$$

Derivation. The idea is to write out the two Gaussian log-densities, watch the quadratic parts cancel because the covariances are equal, and then find the distribution of what is left.

1. **Expand the two quadratics.** A Gaussian log-density is $-\frac{1}{2}(\theta - m)^\top \Sigma^{-1}(\theta - m)$ plus a constant that depends only on Σ, and that constant is the same for P and Q. So $\ln q - \ln p = -\frac{1}{2}(\theta - m_1)^\top \Sigma^{-1}(\theta - m_1) + \frac{1}{2}(\theta - m_0)^\top \Sigma^{-1}(\theta - m_0)$. Multiplying out, the terms $\theta^\top \Sigma^{-1} \theta$ appear with opposite signs and cancel, which leaves $(m_1 - m_0)^\top \Sigma^{-1}\theta - \frac{1}{2}(m_1^\top \Sigma^{-1} m_1 - m_0^\top \Sigma^{-1} m_0)$.
2. **Factor the constant.** $m_1^\top \Sigma^{-1} m_1 - m_0^\top \Sigma^{-1} m_0 = (m_1 - m_0)^\top \Sigma^{-1}(m_1 + m_0)$, because when you multiply out the right-hand side, the cross terms $m_1^\top \Sigma^{-1} m_0$ and $m_0^\top \Sigma^{-1} m_1$ are equal (Σ^{-1} is symmetric) and cancel. Pulling out $w^\top = (m_1 - m_0)^\top \Sigma^{-1}$ gives the linear form.
3. **Find its distribution.** $L = w^\top(\theta - \frac{m_0 + m_1}{2})$ is a linear function of a Gaussian, so it is Gaussian. Under Q, θ has mean m_1, so L has mean $w^\top(m_1 - \frac{m_0 + m_1}{2}) = w^\top \frac{m_1 - m_0}{2} = \frac{\Delta^2}{2}$, by the definition of Δ. Under P the same computation with m_0 gives $-\frac{\Delta^2}{2}$. Under both, its variance is $w^\top \Sigma w = (m_1 - m_0)^\top \Sigma^{-1} \Sigma \Sigma^{-1}(m_1 - m_0) = \Delta^2$, so its standard deviation is Δ.
4. **Compute the accuracy of the best test.** By Fact 1, the best test guesses 1 when $q > p$, that is, when $L > 0$. Under Q, L has mean $\frac{\Delta^2}{2}$ and standard deviation Δ, so 0 lies $\frac{\Delta^2/2}{\Delta} = \frac{\Delta}{2}$ standard deviations below the mean, and the test is right with probability

$\Pr[\mathcal{N}(\Delta^2/2, \Delta^2) > 0] = \Phi(\Delta/2)$. Under P, L has mean $-\frac{\Delta^2}{2}$, and the test is right when $L < 0$, which lies $\frac{\Delta}{2}$ standard deviations above that mean: the same probability, $\Phi(\Delta/2)$. Averaging the two worlds gives Equation 12.5.

In one dimension with $\Sigma = 1$, $\Delta = |m_1 - m_0|$, and Equation 12.5 is the Gaussian pair of Fact 1: $\Delta = 1$ gives $\Phi(0.5) = 0.6915$. In many dimensions, only the length of the mean shift measured with Σ^{-1} matters, so a shift along a direction where θ is noisy counts for little. This fact gives the accuracy of every attack on a model whose parameters are approximately Gaussian, and it says which meta-classifier to train: a linear one.

12.3. The property-inference game

12.3.1. Context

"Can the attacker learn the property?" needs the same care as any attack: who draws the data, what the attacker sees and knows, and how its guess is scored. Threat models and privacy games writes attacks as games between a **challenger**, who runs the training pipeline with a hidden secret, and an **adversary**, who tries to guess the secret. It follows Salem et al. (2023). The property game is the version in which the secret is one bit about the population that produced the training set: was the subgroup's share t_0 or t_1? Suri & Evans (2022) call it **distribution inference** and study it in this form.

12.3.2. Statement

The **property-inference game** with parameters (n, t_0, t_1) runs in four steps. The secret is drawn at random by a fair bit so that neither answer is right more often by default, and an attacker who learns nothing scores exactly $\frac{1}{2}$.

1. The challenger draws a fair bit b.
2. It draws a dataset $D \sim \pi_{t_b}^n$ and trains $\theta = \mathcal{T}(D)$.
3. The adversary receives a view of θ: the parameters (white-box), or answers to queries (black-box). It knows π_A, π_B, n, t_0, t_1 and the training algorithm $\mathcal{T}$.
4. It outputs b' and wins if $b' = b$.

What limits the adversary? Its view is a model whose distribution is P_{n,t_0} or P_{n,t_1}, so by Fact 1 its advantage is at most the TV distance between them. The model is computed from the data, so by Fact 2 that distance is at most the TV distance between the two dataset distributions. For every adversary,

$$\mathrm{TPR} - \mathrm{FPR} \le \mathrm{TV}(P_{n,t_0}, P_{n,t_1}) \le \mathrm{TV}(\pi_{t_0}^n, \pi_{t_1}^n). \tag{12.6}$$

The right-hand side becomes concrete when the group of each record can be read off the record (π_A and π_B have disjoint supports: no record could come from both groups). Then the only thing in D that says anything about t is how many records come from the subgroup. Call that count S. It is a sufficient statistic, and S is binomial: each of the n records is from the subgroup with probability t, independently. So the right-hand side is exactly

$$\mathrm{TV}(\mathrm{Bin}(n, t_0), \mathrm{Bin}(n, t_1)). \tag{12.7}$$

12.3.3. Derivation

1. **The game is a test.** The view has distribution P_{n,t_0} when $b = 0$ and P_{n,t_1} when $b = 1$. By Fact 1, no attacker's advantage exceeds the TV distance between them. This is the left inequality of Equation 12.6.
2. **Training is a channel.** The view is a randomized function of D, with randomness that does not depend on b. Fact 2 says such processing cannot increase the TV distance, which gives the right inequality.
3. **Write the likelihood ratio of a dataset.** To find what in D matters, compare how likely D is under the two shares. With disjoint supports, a record x from B has density $t\,\pi_B(x)$ under π_t (pick group B, then draw x), and a record from A has $(1-t)\pi_A(x)$. The records are independent, so the density of D is the product over its n records. Divide the t_1 version by the t_0 version. The factors $\pi_A(x)$ and $\pi_B(x)$ are the same in both and cancel. Each of the S subgroup records leaves a factor t_1/t_0, and each of the $n - S$ others a factor $(1-t_1)/(1-t_0)$:
$$\frac{d\pi^n_{t_1}}{d\pi^n_{t_0}}(D) = \Big(\frac{t_1}{t_0}\Big)^S \Big(\frac{1-t_1}{1-t_0}\Big)^{n-S}. \tag{12.8}$$
4. **Conclude that the count is sufficient.** The likelihood ratio depends on D only through S, so S is sufficient: the best test of Fact 1 can be computed from S alone, and the data processing inequality of Fact 2 holds with equality. The TV distance between the datasets therefore equals the TV distance between the laws of S, which are $\mathrm{Bin}(n, t_0)$ and $\mathrm{Bin}(n, t_1)$. That is Equation 12.7.

12.3.4. Interpretation

The first lesson is that **releasing the count is the worst case**. When the group is visible in each record, no model trained on D can leak more about t than the count of subgroup records, because the count already carries everything D says about t. A model that does not see the group, like the logistic regression of Section 12.5, leaks less, and how much less is a number you can measure.

The game also says **what the attacker needs**. It gives the adversary π_A, π_B and $\mathcal{T}$. In practice this means auxiliary data from both groups and the ability to train models the way the victim does. Ateniese et al. and Ganju et al. assume exactly this. It is the same assumption that makes shadow models work in Membership inference: the attacker trains its own copies of the victim's pipeline on data it controls.

The secret belongs to a group, not a person, and that changes how the attack is scored. The game has no target record. Its baseline is a coin flip, and the score is accuracy, not the true positive rate at a low false positive rate. In membership inference, the attacker faces many targets, and a few records exposed with high confidence matter even if most are not, so the end of the curve with few false positives is what counts. Here each dataset carries one secret, so the plain probability of guessing it is the natural score.

You may ask whether this **is an attack at all**. Learning that smokers get more lung cancer is the purpose of a medical study, not a breach. Property inference is a harm when the property describes the *data owner* rather than the world: the share of a company's customers in a region,

the demographics of one hospital's patients, the configuration of one lab's machines. When D is a small sample from a public population, its property is public knowledge anyway.

Finally, the game has **variants**. The secret can be a real number (estimate t rather than choose between t_0 and t_1), or the presence of any record with a given feature. Some games fix the subgroup count at exactly $\lfloor tn \rfloor$ instead of drawing it at random. With a fixed count and the group visible, the ceiling is 1, because the count itself differs between the two worlds and reveals the bit. The attacker's difficulty then comes only from the training noise and from the group being hidden.

12.3.5. Example: one hospital's patients

A hospital has $n = 1{,}000$ patients. Whether 45% or 55% of them are over 65 is the secret. If age group is part of each record, Equation 12.7 gives the ceiling $\mathrm{TV}(\mathrm{Bin}(1000, 0.45), \mathrm{Bin}(1000, 0.55)) = 0.9985$ for every model trained on the data. You can see why it is so close to 1. The count has mean 450 in one world and 550 in the other, and its standard deviation is $\sqrt{1000 \times 0.45 \times 0.55} \approx 15.7$, so the two means are more than six standard deviations apart and the two distributions barely overlap. A model from which the count can be read off reaches the ceiling: by Fact 1 its attacker is right with probability $\frac{1}{2} + \frac{1}{2}(0.9985)$, more than 99.9% of the time.

At $n = 100$ the means are 45 and 55, the standard deviation is about 5, and the distributions overlap much more. The ceiling is 0.6827, and 200,000 simulated games with the likelihood-ratio test on the count give an advantage of 0.6811, within sampling error of the formula.

12.3.6. Check yourself

- Why can no attacker on any model trained on D beat Equation 12.7 when the group is visible in every record? *(The model is a randomized function of D, so by the data processing inequality its TV is at most that of D, and the count S is sufficient, so the TV of D equals that of S.)*
- A defender trains on a dataset from which the group column has been deleted. Does Equation 12.6 still hold, and does Equation 12.7? *(Equation 12.6 holds for any training algorithm. Equation 12.7 still bounds it, because the defender's dataset is a function of the original one; it may no longer be attained.)*

12.4. Why a property leaks when no record does

12.4.1. Context

Take the simplest model of all: the training algorithm releases the count S of subgroup records. Two attackers look at the same release. One asks about a record: "is my target, a member of the subgroup, in D?" The other asks about the property: "was D drawn with t_0 or t_1?" For a count, both best advantages can be computed exactly, and they move in opposite directions as n grows. This makes the one idea of the chapter exact.

12.4.2. Statement

Membership. In the membership game, D holds $n-1$ records from π_t plus one more: either the target (a subgroup member) or a fresh record from π_t. Let $X \sim \mathrm{Bin}(n-1, t)$ be the subgroup count among the $n-1$ other records. The target adds exactly 1 to the count, and a fresh record adds 1 only a fraction t of the time. So the target is detectable only through how much one extra unit changes the distribution of a count that already varies by about $\sqrt{n}$. That change is governed by the height of the most likely count, and the best advantage is

$$\mathrm{Adv}_{\mathrm{mem}}(n) = (1-t)\max_k \Pr[X=k] \approx \frac{1-t}{\sqrt{2\pi(n-1)t(1-t)}}. \tag{12.9}$$

Property. Under the two hypotheses the fraction S/n centres on t_0 or on t_1, with standard deviation about $\sqrt{t(1-t)/n}$. The best advantage is the TV distance between the two binomials, and by the normal approximation it depends on how many standard deviations apart the two centres are. With the midpoint share $\bar{t} = \frac{t_0+t_1}{2}$,

$$\mathrm{Adv}_{\mathrm{prop}}(n) = \mathrm{TV}(\mathrm{Bin}(n,t_0), \mathrm{Bin}(n,t_1)) \approx 2\Phi\Big(\frac{\sqrt{n}\,|t_1-t_0|}{2\sqrt{\bar{t}(1-\bar{t})}}\Big) - 1. \tag{12.10}$$

The first falls like $1/\sqrt{n}$. The second rises to 1 once $\sqrt{n}\,|t_1-t_0|$ is a few times $\sqrt{\bar{t}(1-\bar{t})}$.

12.4.3. Derivation

Membership. The plan is to write the two views as distributions of the count, notice that one of them is a mixture containing the other, and reduce the TV distance to the effect of shifting a binomial by one.

1. **Write the two views.** With the target in D, $S = X+1$. With a fresh record, $S = X + B'$, where $B' \sim \mathrm{Bernoulli}(t)$ is 1 when the fresh record is from the subgroup.
2. **See the second view as a mixture.** $X + B'$ equals $X+1$ with probability t and X with probability $1-t$, so its law is $t\,\mathcal{L}(X+1) + (1-t)\,\mathcal{L}(X)$, where $\mathcal{L}(\cdot)$ means "the distribution of". A fraction t of the time the non-member world looks exactly like the member world.
3. **Mixing scales the TV distance.** For any P and Q, $\mathrm{TV}(P, tP + (1-t)Q) = \frac{1}{2}\sum_x |p(x) - tp(x) - (1-t)q(x)| = \frac{1}{2}\sum_x |(1-t)(p(x)-q(x))| = (1-t)\mathrm{TV}(P,Q)$. By Fact 1, the best advantage is therefore $(1-t)\,\mathrm{TV}(\mathcal{L}(X+1), \mathcal{L}(X))$.
4. **Shift a single-peaked distribution by one.** Write $p_k = \Pr[X=k]$. Then $\Pr[X+1=k] = p_{k-1}$, so $\mathrm{TV}(\mathcal{L}(X+1), \mathcal{L}(X)) = \frac{1}{2}\sum_k |p_k - p_{k-1}|$, half the total size of the steps between neighbouring probabilities. A binomial probability mass function is **unimodal**: it rises to its most likely value, the mode k^*, and then falls. So the upward steps add up to the climb from 0 to the top, p_{k^*}, and the downward steps add up to the descent back to 0, also p_{k^*}. Half of the total is $p_{k^*} = \max_k p_k$. Table 12.2 walks through $X \sim \mathrm{Bin}(4, \frac{1}{2})$.
5. **Estimate the height of the mode.** By the normal approximation of Probability distributions, a binomial with many trials is close to a normal with the same mean and variance, here $\mathcal{N}((n-1)t, (n-1)t(1-t))$. A normal density with variance s^2 has height $1/\sqrt{2\pi s^2}$ at its peak, here $1/\sqrt{2\pi(n-1)t(1-t)}$. The probability of the most likely integer is close to that peak height, because the integers are spaced one apart and the density is nearly flat over a unit interval at the top. That gives Equation 12.9.

Table 12.2.: Shifting $X \sim \mathrm{Bin}(4, \frac{1}{2})$ by one. The positive differences add up to 6/16, the height of the mode, and so do the negative ones. Half the total absolute difference is $\frac{1}{2} \times \frac{12}{16} = \frac{6}{16} = 0.375 = \max_k \Pr[X = k]$.

k	0	1	2	3	4	5
$16 \Pr[X = k]$	1	4	6	4	1	0
$16 \Pr[X + 1 = k]$	0	1	4	6	4	1
16× difference	1	3	2	−2	−3	−1

Property. Here the plan is to use the exact answer from the game section, then approximate both binomials by normals with a common spread so that the Gaussian pair of Fact 1 applies.

1. **Start from the exact answer.** By Section 12.3, the released count S is a sufficient statistic, so the best advantage is the TV distance between the two binomials.
2. **Approximate by two normals.** By the normal approximation, S/n is close to $\mathcal{N}(t, t(1-t)/n)$. The two variances $t_0(1-t_0)/n$ and $t_1(1-t_1)/n$ are close when the gap is small, so replace both by the one at the midpoint, $\bar{t}(1-\bar{t})/n$. Now divide S/n by its standard deviation $\sqrt{\bar{t}(1-\bar{t})/n}$. The problem becomes a pair of unit-variance Gaussians whose means are $\mu = \sqrt{n}\,|t_1 - t_0|/\sqrt{\bar{t}(1-\bar{t})}$ apart.
3. **Use the Gaussian pair.** As in the example of Fact 1, two unit-variance Gaussians with means μ apart have densities that cross at the midpoint, $\mu/2$ from each mean, so the best test has TPR $= \Phi(\mu/2)$ and FPR $= 1 - \Phi(\mu/2)$, and the TV distance is $2\Phi(\mu/2) - 1$. Substituting μ gives Equation 12.10.

The general pattern. Neither argument is special to counts. Most estimators behave like averages: $\theta \approx \theta(\pi) + \frac{1}{n}\sum_i \psi(x_i)$, where $\theta(\pi)$ is the value the estimator would take on the whole population and $\psi(x)$ is the **influence** of one record x, how far it pulls the estimate (Section 12.5 derives this for logistic regression). One record z shifts θ by $\psi(z)/n$, while the sampling noise has size $\sigma/\sqrt{n}$, where σ is the standard deviation of $\psi(x)$. The shift measured in noise units is $(|\psi(z)|/n)/(\sigma/\sqrt{n}) = |\psi(z)|/(\sigma\sqrt{n}) \to 0$. The property instead shifts the centre from $\theta(\pi_{t_0})$ to $\theta(\pi_{t_1})$, a fixed distance, so in noise units the shift grows like $\sqrt{n}$.

12.4.4. Interpretation

The headline is that **more data protects records and exposes properties**. Figure 12.1 (right) shows both advantages against n on log axes. The membership advantage falls along a line of slope $-\frac{1}{2}$, which is what $1/\sqrt{n}$ looks like on log axes. The property advantage climbs to 1, sooner for a larger gap $|t_1 - t_0|$.

The reason is that **a property is a group of records**. Moving from t_0 to t_1 changes about $n|t_1 - t_0|$ records: for the hospital, $1{,}000 \times 0.1 = 100$ of its 1,000 patients. A protection measured one record at a time says little about a change to 100 records at once. Part 3 makes this precise with its group-privacy property, whose guarantee weakens with the size of the group.

It follows that **hiding the property means hiding the population**. To push the property advantage down, the released statistic must be noisy on the scale of $n|t_1 - t_0|$, not on the scale of

one record. That much noise also destroys what the statistic was for (Exercise 6 in the exercise book).

The approximations are good here. At $n = 100$ the normal approximation to the property advantage is exact to four decimals, and the one to the membership advantage is off by less than 1% (Table 12.3).

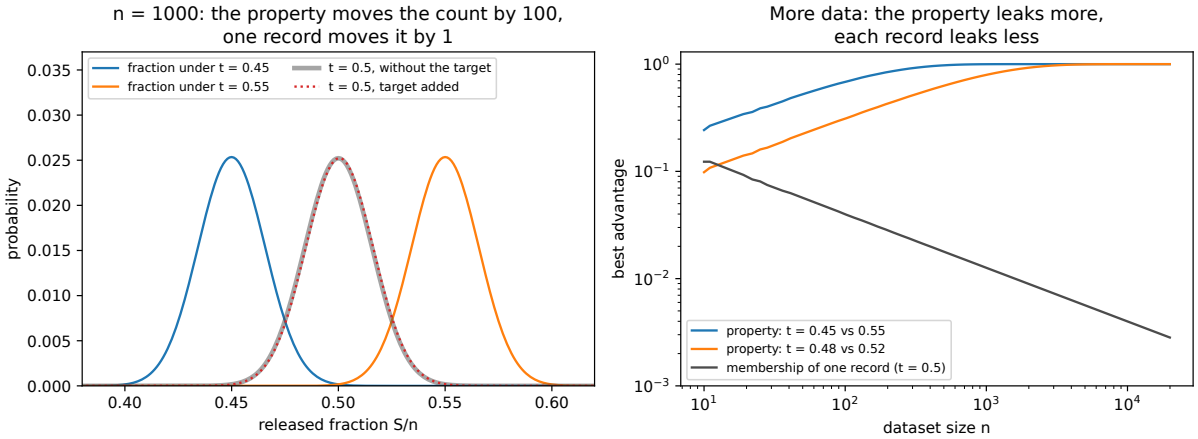


Figure 12.1.: Left: the released fraction S/n at $n = 1{,}000$ under $t = 0.45$ and $t = 0.55$, and under $t = 0.5$ with and without one target from the subgroup; the last two curves are indistinguishable. Right: the best property advantage for two gaps, and the best membership advantage of one record, against n.

12.4.5. Example: the same release, two attackers

Table 12.3 gives both advantages for $t_0 = 0.45$, $t_1 = 0.55$, and for membership at $t = 0.5$. You can check the first row by hand. For the property, $\bar{t} = 0.5$, so $\sqrt{\bar{t}(1-\bar{t})} = 0.5$ and the argument of Φ in Equation 12.10 is $\sqrt{100} \times 0.1/(2 \times 0.5) = 1$; then $2\Phi(1) - 1 = 2(0.8413) - 1 = 0.6827$. For membership, Equation 12.9 gives $0.5/\sqrt{2\pi \times 99 \times 0.25} = 0.5/12.47 = 0.0401$.

Table 12.3.: Best advantages from a released count of subgroup records. Property: $t_0 = 0.45$ against $t_1 = 0.55$. Membership: one subgroup record, population share $t = 0.5$.

n	Property, exact	Property, Equation 12.10	Membership, exact	Membership, Equation 12.9
100	0.6827	0.6827	0.0398	0.0401
1,000	0.9985	0.9984	0.0126	0.0126
10,000	1.0000	1.0000	0.0040	0.0040

At $n = 10{,}000$, an attacker who wants to know whether one patient is in the data has advantage 0.0040: with a fair bit it is right with probability $\frac{1}{2} + \frac{1}{2}(0.0040) = 0.502$. An attacker who wants the share of patients over 65 is never wrong to four decimals. The data, the release and the attacker's knowledge are the same; only the question differs.

12.4.6. Check yourself

- In step 4 of the membership derivation, why does the argument need the probability mass function to be unimodal? *(For one that rises and falls several times, the climbs and descents add up to more than twice the maximum. For X uniform on $\{0, 2\}$, $\mathrm{TV}(X, X+1) = 1$ while the maximum is $\frac{1}{2}$.)*
- Doubling the gap $|t_1 - t_0|$ has the same effect on Equation 12.10 as multiplying n by what? *(By 4: the formula depends on n and the gap only through $\sqrt{n}\,|t_1 - t_0|$.)*

12.5. Shadow models and meta-classifiers

12.5.1. Context

For a real model nobody can write down $P_{n,t}$, the distribution of the trained parameters, so the likelihood-ratio test of Fact 1 is out of reach. Ateniese et al. (2015) gave the recipe every later attack uses: if you cannot compute the two distributions, sample from them. Draw many datasets from π_{t_0} and from π_{t_1}, using auxiliary data. Train a **shadow model** on each, exactly as the victim trains. Label each shadow model's parameters with the bit that produced them, and train a classifier on these labelled parameter vectors: the **meta-classifier**. Then apply it to the victim's model. This is the shadow-model idea of Membership inference, with whole models as the data points.

Which meta-classifier should you train, and how accurate can it be? For models fitted by minimizing an average loss, the parameters turn out to be approximately Gaussian with a covariance that shrinks like $1/n$. Fact 3 then answers both questions: train a linear classifier, and expect accuracy $\Phi(\Delta/2)$.

12.5.2. Statement

The setting is a model fitted by minimizing an average loss. Let θ minimize the average loss $\frac{1}{n}\sum_i \ell(x_i, \theta)$ over the training records, and let $\theta(t)$ minimize the expected loss under π_t: the model you would get from infinitely many records. How noisy θ is around $\theta(t)$ depends on two matrices. H is the average curvature of the loss at $\theta(t)$: a sharply curved loss pins the minimizer down. G is the covariance of the per-record gradients at $\theta(t)$: how much each record pulls in its own direction. Together they give $\Sigma_1(t)$, the covariance of the fit for a single record's worth of data (the subscript 1):

$$H = \mathbb{E}_{\pi_t} \nabla^2 \ell(x, \theta(t)), \qquad G = \mathbb{E}_{\pi_t}[\nabla \ell(x, \theta(t)) \nabla \ell(x, \theta(t))^\top], \qquad \Sigma_1(t) = H^{-1} G H^{-1}. \quad (12.11)$$

Σ_1 is called the **sandwich covariance**, because G sits between two copies of H^{-1}. Then, for large n:

1. **The parameters are approximately Gaussian,** centred on the population fit with covariance shrinking like $1/n$: $\theta \approx \mathcal{N}(\theta(t), \Sigma_1(t)/n)$.

2. **The best attack is linear,** and its accuracy is that of Fact 3 with the shift $\theta(t_1) - \theta(t_0)$ measured against the noise $\bar{\Sigma}_1/n$:

$$\Phi(\Delta/2), \qquad \Delta = \sqrt{n\,(\theta(t_1) - \theta(t_0))^\top \bar{\Sigma}_1^{-1} (\theta(t_1) - \theta(t_0))}, \tag{12.12}$$

   where $\bar{\Sigma}_1$ is the average of $\Sigma_1(t_0)$ and $\Sigma_1(t_1)$.
3. **For a small gap, only $\sqrt{n}\,|t_1 - t_0|$ matters.** The shift is then about the slope $\theta'(\bar{t}) = \frac{d\theta}{dt}$ times the gap, so Δ is the gap times $\sqrt{n}$ times a constant κ, which is the length of the slope measured against the noise:

$$\Delta \approx \kappa\sqrt{n}\,|t_1 - t_0|, \qquad \kappa = \sqrt{\theta'(\bar{t})^\top \Sigma_1(\bar{t})^{-1} \theta'(\bar{t})}. \tag{12.13}$$

   For the released fraction $\theta = S/n$, $\kappa = 1/\sqrt{\bar{t}(1-\bar{t})}$, which is $1/\sqrt{0.25} = 2$ at $\bar{t} = \frac{1}{2}$, and Equation 12.12 becomes Equation 12.10.

12.5.3. Derivation

The parameters are approximately Gaussian. This is the standard asymptotic normality of **M-estimators**, estimators defined as the minimizer of an average loss (van der Vaart 1998, ch. 5). The idea: near its minimum, the fitted parameter is a linear function of an average of per-record gradients, and an average is approximately Gaussian. Four steps make this precise.

1. **The fit solves an average equation.** At a minimizer the gradient is zero, so $\frac{1}{n}\sum_i \nabla\ell(x_i, \theta) = 0$.
2. **Linearize around $\theta(t)$.** For large n, θ is close to $\theta(t)$, so a first-order Taylor expansion of the gradient around $\theta(t)$ is accurate: $0 \approx \frac{1}{n}\sum_i \nabla\ell(x_i, \theta(t)) + H(\theta - \theta(t))$. Here the sample average of the Hessians has been replaced by its expectation H, which the law of large numbers allows for large n. Solving for θ,

$$\theta - \theta(t) \approx -H^{-1}\,\frac{1}{n}\sum_i \nabla\ell(x_i, \theta(t)). \tag{12.14}$$

   This is the average form of Section 12.4, with influence $\psi(x) = -H^{-1}\nabla\ell(x, \theta(t))$: each record pulls the fit by its gradient, scaled by the inverse curvature.
3. **The average gradient has mean zero.** $\theta(t)$ minimizes the expected loss, so the expected gradient there is zero, $\mathbb{E}_{\pi_t}\nabla\ell(x, \theta(t)) = 0$. Each gradient therefore has mean zero and covariance G, and an average of n independent copies has covariance G/n.
4. **Apply the central limit theorem.** An average of many independent terms is approximately Gaussian, so the average gradient is approximately $\mathcal{N}(0, G/n)$. Multiplying a Gaussian vector by the matrix $-H^{-1}$ multiplies its covariance by H^{-1} on both sides (H is symmetric), which gives $\mathcal{N}(0, H^{-1}GH^{-1}/n)$.

The best attack and its accuracy. Under the two hypotheses, θ is approximately $\mathcal{N}(\theta(t_0), \Sigma_1(t_0)/n)$ and $\mathcal{N}(\theta(t_1), \Sigma_1(t_1)/n)$. Fact 3 needs a shared covariance; when the two covariances are close, replace both by $\bar{\Sigma}_1/n$. Fact 3 then applies with $m_b = \theta(t_b)$ and $\Sigma = \bar{\Sigma}_1/n$. Since $\Sigma^{-1} = n\,\bar{\Sigma}_1^{-1}$, the factor n comes out of the Mahalanobis distance, which gives Equation 12.12. Fact 3 also says the best test is linear in θ: a **linear discriminant**, which is the meta-classifier to train.

The scaling law. For a small gap, the population fit changes almost linearly in t, so $\theta(t_1)-\theta(t_0) \approx \theta'(\bar{t})(t_1 - t_0)$, and the covariance hardly changes, so $\bar{\Sigma}_1 \approx \Sigma_1(\bar{t})$. Substituting into Equation 12.12 gives $\Delta^2 \approx n(t_1 - t_0)^2\, \theta'(\bar{t})^\top \Sigma_1(\bar{t})^{-1}\theta'(\bar{t})$, whose square root is Equation 12.13.

The released fraction. The count fits the same mould, which checks the general formula against Section 12.4. Take $\ell(x, \theta) = \frac{1}{2}(\theta - a)^2$, where $a \in \{0, 1\}$ marks the subgroup. The minimizer of the average of $\frac{1}{2}(\theta - a_i)^2$ is the mean of the a_i, which is $\theta = S/n$. Its population version is $\theta(t) = \mathbb{E}[a] = t$, so $\theta' = 1$. The second derivative of the loss is $H = 1$, and the gradient $\theta(t) - a = t - a$ has variance $G = t(1 - t)$, the variance of a Bernoulli(t). So $\Sigma_1 = t(1 - t)$ and $\kappa = 1/\sqrt{t(1 - t)}$. With this κ, the accuracy $\Phi(\Delta/2)$ turns into the advantage $2\Phi(\Delta/2) - 1$ of Equation 12.10, because advantage is twice accuracy minus 1.

12.5.4. Interpretation

Two numbers set the leak. $\theta'(t)$ says how much the property moves what the model learns, and Σ_1 says how noisy that is. If the property does not affect what the model fits, then $\theta' = 0$ and, to first order, nothing leaks, however large n is. If it does, the leak is a side effect of fitting the data well, and no amount of care in the training code removes it.

κ compares any model with the count. With the group visible, the count has $\kappa = 1/\sqrt{\bar{t}(1 - \bar{t})}$. Suppose a model has a smaller κ. How many records n' would the count need to leak as much as the model does on n? Setting the two values of Δ equal, $\kappa\sqrt{n} = \sqrt{n'}/\sqrt{\bar{t}(1 - \bar{t})}$, so $n' = n(\kappa\sqrt{\bar{t}(1 - \bar{t})})^2$, a fraction of the real n. Suri & Evans (2022) measure leakage in exactly these units, as the number of records an attacker would need to see directly to match the model (Exercise 3 in the exercise book).

Where the Gaussian picture fails. The derivation used a Taylor expansion and the central limit theorem, both of which need large n. When n is small or the model is far from its large-n regime, the parameters are skewed and the linear meta-classifier is no longer optimal. A second gap comes from the attacker's own sampling. When the meta-classifier is trained on few shadow models, it estimates its direction w with error, and falls short of $\Phi(\Delta/2)$. That shortfall is largest when the true effect is small, because then the noise in the shadow models swamps the shift it is trying to find.

The real pipeline. Ganju et al. (2018) attack networks trained on census income data and other datasets, with many shadow models per hypothesis and a neural network as the meta-classifier. The toy below keeps every step but runs in seconds.

12.5.5. Example: logistic regression with a hidden subgroup

Each record has two features $x \sim \mathcal{N}(0, I_2)$ and a label. The majority labels with $\Pr[y = 1 \mid x] = \sigma(2x_1 + x_2)$, the subgroup with $\sigma(2x_1 - x_2)$, where $\sigma(z) = 1/(1 + e^{-z})$ is the logistic function. Think of a symptom x_2 that raises risk in most patients and lowers it in a minority. **The group is never a feature**: the model sees (x_1, x_2, y) only. A logistic regression with weights (w_1, w_2) and an intercept is trained on n records, by Newton's method with a tiny ridge penalty 10^{-3}.

The script computes $\theta(t)$ and $\Sigma_1(t)$ exactly, by Gauss–Hermite quadrature over x (a numerical rule for integrals against a Gaussian), as Table 12.4 shows.

Table 12.4.: The population fit $\theta(t)$ of the logistic regression, against the subgroup share t. The intercept is 0 for every t.

t	0	0.25	0.5	0.75	1
$w_1(t)$	2.000	1.767	1.704	1.767	2.000
$w_2(t)$	1.000	0.442	0.000	−0.442	−1.000

To use Equation 12.13 you need the slope θ' and the noise Σ_1 at the midpoint $t = \frac{1}{2}$. Start with the slope, which says **the property moves** w_2. At $t = \frac{1}{2}$ the two rules cancel in w_2: half the population pushes it up and half down. As t grows, the subgroup's rule wins and w_2 goes negative, with slope $\theta'(\frac{1}{2}) = (0, -1.704, 0)$ for $(w_1, w_2, \text{intercept})$. The table agrees: between $t = 0.25$ and $t = 0.75$, w_2 falls by $0.442 - (-0.442) = 0.884$ over a step of 0.5 in t, a slope of -1.768, close to the slope at $\frac{1}{2}$. Mixing the rules also makes the labels noisier, which shrinks w_1 from 2 to 1.704. But the table shows it shrinks symmetrically in t (1.767 at both 0.25 and 0.75), so w_1 is flat at $\frac{1}{2}$ and carries no information there. The intercept is 0 for every t, so it carries none either.

Next, **the noise.** $\Sigma_1(\frac{1}{2})$ has diagonal $(12.59, 6.94, 6.04)$ for $(w_1, w_2, \text{intercept})$, and zero off-diagonal entries. The zeros come from two symmetries of the population at $t = \frac{1}{2}$. Flipping the sign of x_2 swaps the two groups' rules, and with half of each the mixture is unchanged. The flip turns a fitted $(w_1, w_2, \text{intercept})$ into $(w_1, -w_2, \text{intercept})$, so the two are equally likely, and w_2 is therefore uncorrelated with the other two parameters. Flipping the sign of x and swapping the labels $y \to 1 - y$ also leaves the population unchanged and flips only the sign of the intercept, which makes the intercept uncorrelated with the weights. To test the Gaussian picture, the script trains two thousand models on $n = 1{,}000$ records at $t = \frac{1}{2}$. n times their variances are $(11.92, 6.80, 6.08)$, and their mean w_1 is 1.693. The Gaussian picture is right to within about 5%; the rest is the bias of a finite sample and of the ridge penalty.

Now **the constant.** Only w_2 has a nonzero slope, and Σ_1 is diagonal, so Equation 12.13 reduces to the slope of w_2 over its standard deviation: $\kappa = 1.704/\sqrt{6.94} = 0.647$, against 2 for the count. By the comparison in the interpretation, the model leaks the property as well as $(0.647/2)^2 = 0.105$ of its records would: about one record in ten.

Figure 12.2 shows the trained models for $t_0 = 0.4$ and $t_1 = 0.6$. At $n = 100$ the two clouds overlap. At $n = 1{,}000$ they have separated along w_2, and a line splits them.

Now the attack itself. For each n and gap, with t_0, t_1 centred on $\frac{1}{2}$, the script trains 150 shadow models per hypothesis, fits the linear discriminant on their three parameters, and measures its accuracy on 150 fresh target models per hypothesis (Table 12.5). Each accuracy is an average over 300 right-or-wrong outcomes, each with variance at most $\frac{1}{2} \cdot \frac{1}{2} = 0.25$, so its standard error is at most $\sqrt{0.25/300} = 0.029$.

Table 12.5.: Accuracy of the meta-classifier against logistic regressions, by dataset size n and gap $|t_1 - t_0|$, with the prediction $\Phi(\Delta/2)$ of Equation 12.12 in parentheses.

n	gap 0.05	gap 0.1	gap 0.2	gap 0.4
50	0.523 (0.546)	0.520 (0.590)	0.653 (0.676)	0.830 (0.818)
100	0.523 (0.564)	0.670 (0.627)	0.733 (0.741)	0.863 (0.901)
200	0.503 (0.590)	0.670 (0.676)	0.803 (0.819)	0.957 (0.965)

n	gap 0.05	gap 0.1	gap 0.2	gap 0.4
500	0.633 (0.641)	0.787 (0.765)	0.910 (0.926)	0.997 (0.998)
1,000	0.713 (0.695)	0.867 (0.847)	0.980 (0.979)	1.000 (1.000)

The prediction holds. Most cells agree with $\Phi(\Delta/2)$ to within two standard errors. The exceptions are at small effects ($n \leq 200$, gap ≤ 0.1), where the meta-classifier, fitted to 300 noisy shadow models, finds the direction of w badly. There the attack is weaker than the best test, as the interpretation predicted.

The scaling law holds. Equation 12.13 says the accuracy depends on n and the gap only through $\sqrt{n}\,|t_1 - t_0|$. Plotted against that product, the four curves fall on one, the curve $\Phi(\kappa\sqrt{n}\,|t_1 - t_0|/2)$ with $\kappa = 0.647$ (Figure 12.3). So a gap of 0.05 at $n = 1{,}000$ is about as easy as a gap of 0.1 at $n = 200$ or so: $\sqrt{1000} \times 0.05 = 1.58$, against $\sqrt{200} \times 0.1 = 1.41$.

12.5.6. Check yourself

- If the subgroup used the majority's label rule $\sigma(2x_1 + x_2)$, what would κ be? *(Zero. $\pi_A = \pi_B$, so π_t does not depend on t, $\theta' = 0$, and no attack beats a coin flip; the property is then undetectable even from the data.)*
- Why does the best meta-classifier ignore the intercept and w_1 in this example, even though they vary from model to model? *(Near $t = \frac{1}{2}$ their means do not depend on t, and they are uncorrelated with w_2, so they add noise without signal; the linear discriminant gives them weight $\Sigma^{-1}(m_1 - m_0)$, which is zero in those coordinates.)*

12.6. Permutation invariance

12.6.1. Context

A logistic regression has one parameter vector per function: different weights give different predictions. A neural network has many. Swap two hidden units, together with their incoming and outgoing weights, and the network computes exactly the same function. Ganju et al. (2018) observed that a meta-classifier fed the flattened weights must then learn the same pattern in every slot and every order, and does poorly. Their remedy is a representation that does not depend on the order of the units. This section works it out for a network with one hidden layer of tanh units, which has a second symmetry as well: because tanh is odd, flipping the signs of a unit's weights in and out also leaves the function unchanged.

12.6.2. Statement

Let $f_\theta(x) = \sigma(v^\top \tanh(Wx + b) + c)$ be a network with h hidden units. Row j of W and entry b_j compute unit j's input, v_j is its outgoing weight, and c is the output bias. Collect unit j's parameters as $u_j = (W_{j\cdot}, b_j, v_j)$. The two symmetries are a reordering of the units and a sign flip of some of them. A **permutation matrix** Π reorders the entries of a vector, and a diagonal

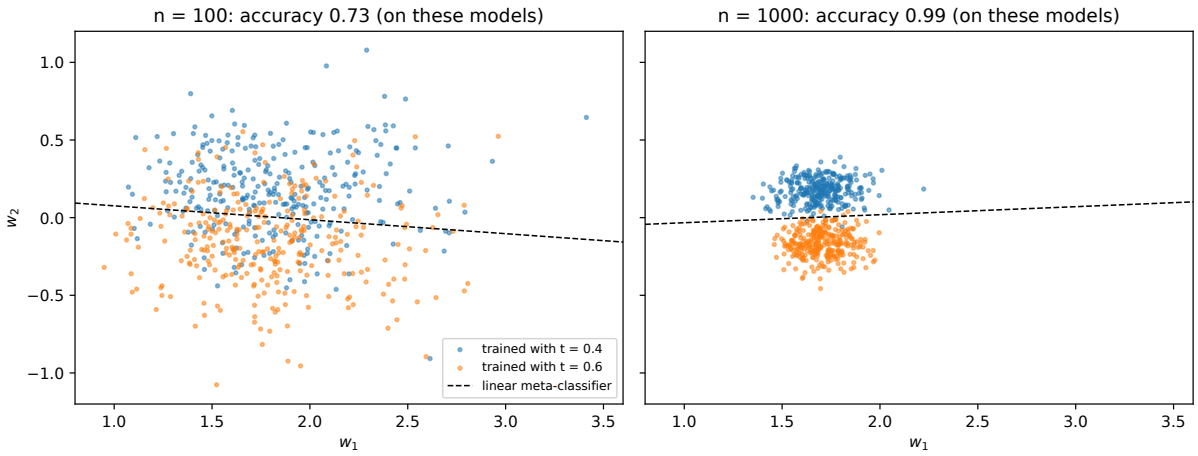


Figure 12.2.: The parameters (w_1, w_2) of 300 logistic regressions trained with subgroup share $t = 0.4$ and 300 with $t = 0.6$, at $n = 100$ (left) and $n = 1{,}000$ (right), with the linear meta-classifier fitted to them. The clouds shrink like $1/\sqrt{n}$; the gap between their centres stays fixed.

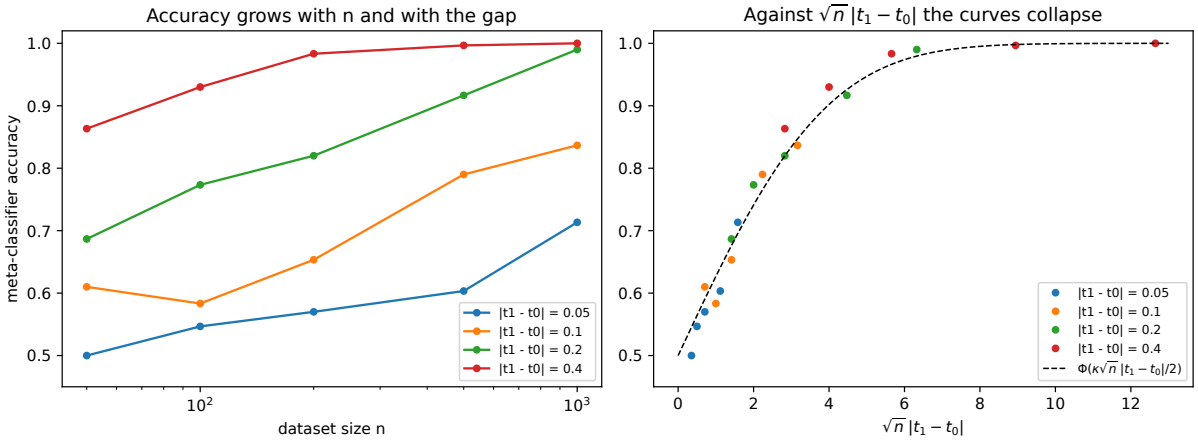


Figure 12.3.: Left: accuracy of the meta-classifier against n for four gaps $|t_1 - t_0|$ (150 shadow and 150 target models per hypothesis, seed 1). Right: the same accuracies against $\sqrt{n}\,|t_1 - t_0|$, with the Gaussian prediction $\Phi(\kappa\sqrt{n}\,|t_1 - t_0|/2)$, $\kappa = 0.647$.

matrix S with entries ± 1 flips the sign of some of them. Applying $g = S\Pi$ to the rows of W and the entries of b and v reorders the units and flips some. It does not change the function:

$$f_{(gW,\ gb,\ gv,\ c)} = f_{(W,\ b,\ v,\ c)}. \tag{12.15}$$

There are $h!$ orders and 2^h sign patterns, so each generic network has $h!\,2^h$ parameter vectors: $4! \times 2^4 = 24 \times 16 = 384$ for $h = 4$. Suppose the initialization is invariant under every g (for example, independent entries drawn from a distribution symmetric about 0) and training is gradient descent. Then the symmetry carries over from the function to the distribution of the trained weights, with three consequences:

1. the law of the trained parameters is invariant under every g, under both hypotheses;
2. every coordinate of every u_j has mean zero under both hypotheses, so a linear meta-classifier on the raw parameters can use none of them;
3. the second-moment features $\sum_j u_j u_j^\top$ are invariant under every g, and keep information that the raw means lose.

12.6.3. Derivation

The plan is to show that g passes through tanh, so it leaves the network's output unchanged; then that it passes through every step of gradient descent; and then to read off what this means for averages.

1. **tanh commutes with g.** tanh acts entry by entry, so permuting its input permutes its output in the same way. tanh is odd, $\tanh(-z) = -\tanh(z)$, so flipping the sign of an entry flips the sign of its output. Hence $\tanh(gz) = g\tanh(z)$.
2. **The output is unchanged.** Using step 1, $(gv)^\top \tanh(gWx + gb) = v^\top g^\top g \tanh(Wx + b) = v^\top \tanh(Wx + b)$, because $g^\top g = \Pi^\top S^2 \Pi = I$: $S^2 = I$ since each entry is ± 1, and $\Pi^\top \Pi = I$ since undoing a reordering gives back the original. This is Equation 12.15. Distinct Π and S give distinct parameter vectors when the units are distinct and no v_j is 0, which gives the count $h!\,2^h$.
3. **Gradient descent commutes with g.** Let $\mathcal{L}$ be the training loss and let g also denote its action on the whole parameter vector, which is linear and orthogonal (it only reorders entries and flips signs). Step 2 says $\mathcal{L}(g\theta) = \mathcal{L}(\theta)$ for every θ. Differentiating both sides with the chain rule gives $g^\top \nabla\mathcal{L}(g\theta) = \nabla\mathcal{L}(\theta)$, so $\nabla\mathcal{L}(g\theta) = g\nabla\mathcal{L}(\theta)$. One step of gradient descent from $g\theta$ therefore lands at $g\theta - \eta g\nabla\mathcal{L}(\theta) = g(\theta - \eta\nabla\mathcal{L}(\theta))$, where η is the step size: g applied to the step from θ. Repeating, if gradient descent from θ_0 reaches θ_k, then from $g\theta_0$ it reaches $g\theta_k$.
4. **The law is invariant.** By assumption, the initial parameters θ_0 have the same law as $g\theta_0$. Training maps θ_0 to θ_k and, by step 3, $g\theta_0$ to $g\theta_k$, so θ_k and $g\theta_k$ have the same law, for every dataset. Averaging over datasets drawn under either hypothesis keeps this.
5. **Raw means vanish.** Take the g that flips unit j only. Invariance says u_j and $-u_j$ have the same law, so $\mathbb{E}[u_j] = \mathbb{E}[-u_j] = -\mathbb{E}[u_j]$, which forces $\mathbb{E}[u_j] = 0$, under both hypotheses. The linear discriminant of Fact 3 puts weight $\Sigma^{-1}(m_1 - m_0)$ on the features, where m_0 and m_1 are the mean parameter vectors under the two hypotheses; their difference is zero in every unit coordinate, so the weight is zero apart from the output bias c, which no g touches.
6. **Second moments are invariant.** A permutation only reorders the terms of $\sum_j u_j u_j^\top$, which does not change the sum. A sign flip turns u_j into $-u_j$, and $(-u_j)(-u_j)^\top = u_j u_j^\top$.

12.6.4. Interpretation

Raw weights hide the property from a linear attacker. Because of the symmetries, each unit slot holds a random one of the network's units, with a random sign. The property shows in *which* units the network has, not in which slot holds which unit, so averaging over networks slot by slot washes it out.

There are **three ways to undo the symmetry**. A **canonical form** picks one parameter vector per function: flip each unit so that $v_j > 0$, then sort the units by v_j. Sorting the neurons of each layer is the first of the two remedies Ganju et al. propose; their networks use ReLU units, which have no sign symmetry, so for them sorting alone gives the canonical form, and it beats raw weights in their experiments. It is fragile, because two units with nearly equal v_j swap places from one network to the next. A **set representation** feeds each unit through the same map ϕ, adds the results, and applies a second map: $\rho(\sum_j \phi(u_j))$. The sum does not depend on the order of the units. Ganju et al. use this DeepSets form (Zaheer et al. 2017), with ϕ and ρ learned networks. The features $\sum_j u_j u_j^\top$ are the case $\phi(u) = uu^\top$, which is also unchanged by sign flips. **Query features** use the network's outputs on fixed probe inputs. A function does not depend on how it is parameterized, so these are invariant automatically, and they need only black-box access.

Deeper networks have the same problem at each layer. Each hidden layer has its own permutations, and they compose, so a set representation is applied layer by layer.

Finally, **black-box can beat white-box.** Fact 2 of Threat models and privacy games says that processing the view cannot help the attacker: whoever holds the weights can compute any query output, so white-box access is never weaker *for the best attack*. That says nothing about a fixed simple attack. For a linear meta-classifier, a better representation of the same model can matter more than more access, and the example shows probe outputs beating raw weights. Zhang, Tople & Ohrimenko (2021) attack properties through queries alone.

12.6.5. Example: a tanh network with four hidden units

The data are as in Section 12.5, with $t_0 = 0.2$ and $t_1 = 0.8$. Each network has $h = 4$ tanh units and is trained on $n = 200$ records by 300 steps of full-batch gradient descent from a random start. On each side, 150 shadow networks fit the linear discriminant and 150 target networks measure its accuracy. The script also checks Equation 12.15 directly: a network with permuted and sign-flipped units gives the same logits on 1,000 inputs, to 10^{-12}.

Each unit has four parameters (two input weights, a bias and an output weight), so the raw parameters number $4 \times 4 + 1 = 17$, counting the output bias c. The canonical form reorders the same 17. The second-moment matrix $\sum_j u_j u_j^\top$ is a symmetric 4×4 matrix with $4 \times 5/2 = 10$ distinct entries, plus c, which gives 11. To see which features carry signal, Table 12.6 also reports each representation's largest **standardized mean gap**: for each feature, the difference of its means under t_0 and t_1, divided by its pooled standard deviation.

Table 12.6.: The linear meta-classifier against tanh networks, by representation. The standardized gap of a feature is the difference of its means under t_0 and t_1, over its pooled standard deviation, on the shadow networks.

Representation	Features	Largest standardized mean gap	Accuracy
Raw parameters	17	0.34	0.497
Canonical order (flip so $v_j > 0$, sort by v_j)	17	0.66	0.807
Unit second moments $\sum_j u_j u_j^\top$	11	2.24	0.877
Outputs on 16 probe inputs	16	2.81	1.000

Raw parameters are at chance, as step 5 predicts. Their largest standardized gap, 0.34, is the size expected from noise alone. A standardized feature has variance 1, so its mean over 150 networks has variance $1/150$, and the difference of two such means has variance $2/150$, a standard deviation of $\sqrt{2/150} = 0.115$. The largest of 17 such gaps is typically two to three times that.

Canonical order recovers most of the signal, with one feature carrying most of it (Figure 12.4, left). **Second moments do better with fewer features,** because they need no tie-breaking between units with nearly equal v_j. **Probe outputs are best.** The property changes the function the network computes, and probe outputs measure that function directly, without passing through any parameterization.

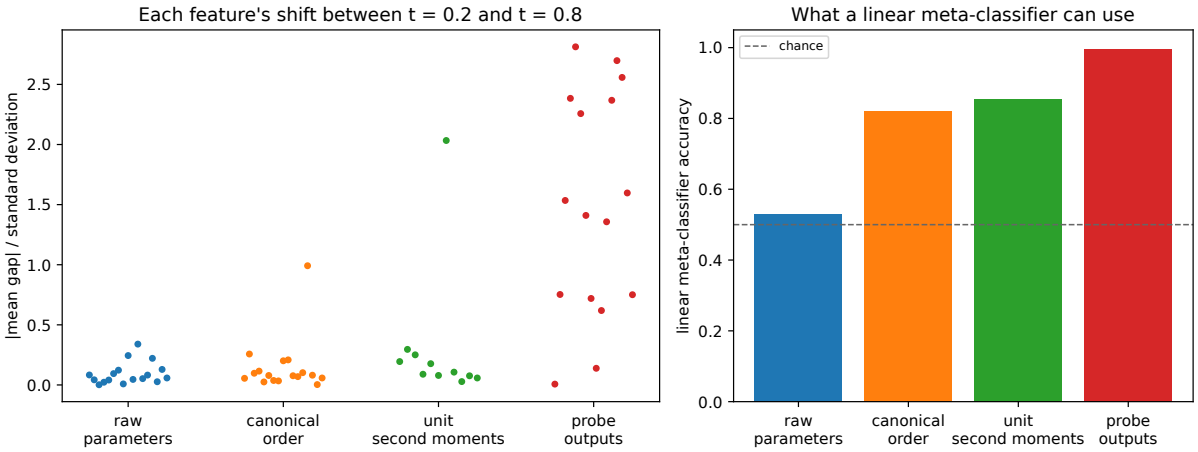


Figure 12.4.: Left: for each representation of 100 tanh networks per hypothesis ($t = 0.2$ against $t = 0.8$), the standardized mean gap of every feature. Right: accuracy of the linear meta-classifier on 100 further networks per hypothesis (seed 1). Raw parameters carry no mean signal; invariant representations do.

12.6.6. Check yourself

- Would the raw means vanish for a network with ReLU units instead of tanh? *(Not by this argument. ReLU is not odd, so the sign flip is not a symmetry; only the permutations remain. They force every unit slot to have the same mean, but that common mean can differ between t_0 and t_1.)*
- Why does a linear meta-classifier on the sum $\sum_j u_j$ fail for tanh networks, even though the sum is permutation invariant? *(The sign flips still apply: $\mathbb{E}[u_j] = 0$ for every j, so the sum has mean zero under both hypotheses. An invariant feature needs to be even in each u_j, like $u_j u_j^\top$.)*

12.7. Code: check every result by computation and simulation

The script code/property_inference.py computes every number in this chapter and draws every figure. The binomial advantages are exact sums. The population fit and the sandwich covariance of the logistic regression are computed by Gauss–Hermite quadrature on a 40×40 grid. Shadow and target models are trained in batches with NumPy: logistic regressions by Newton's method, tanh networks by full-batch gradient descent. From the repository root:

```
uv run python code/part02-attacks/property_inference.py
```

Output (seed 0), summarized in Table 12.7:

Table 12.7.: The script's output with the default seed.

Experiment	Result
Released count, $t_0 = 0.45$, $t_1 = 0.55$	Property advantage 0.6827, 0.9985, 1.0000 at $n = 100$, 1,000, 10,000 (Table 12.3)
Released count, membership at $t = 0.5$	0.0398, 0.0126, 0.0040 at the same n
Simulated property game, $n = 100$	Advantage 0.6811 over 200,000 games (exact 0.6827)
Logistic regression, population view at $t = \frac{1}{2}$	$\theta = (1.704, 0, 0)$; $\theta' = (0, -1.704, 0)$; Σ_1 diagonal $(12.59, 6.94, 6.04)$; $\kappa = 0.647$
2,000 logistic regressions, $n = 1{,}000$, $t = \frac{1}{2}$	Mean $w_1 = 1.693$; $n\times$ variances $(11.92, 6.80, 6.08)$
Meta-classifier sweep	Table 12.5
tanh networks, $h = 4$, $n = 200$, $t = 0.2$ against 0.8	Accuracy 0.497 (raw), 0.807 (canonical), 0.877 (second moments), 1.000 (probes); symmetry check passed

tests/test_property_inference.py checks that the output still matches.

What to take from it. Four things stand out. First, more data helps the property attacker: the same release that hides each record better as n grows reveals the property better. Second,

the Gaussian picture predicts real attacks: two numbers computed from the population, θ' and Σ_1, predict the meta-classifier's accuracy across a 20-fold range of n and an 8-fold range of gaps. Third, hiding the group column does not hide the property: the logistic regression never sees the group, and still leaks the property as well as a tenth of its records would. Fourth, representation matters before access does: the same networks are opaque to a linear attacker in raw form and transparent once the hidden-unit symmetries are removed.

12.8. References

- Ateniese, Mancini, Spognardi, Villani, Vitali & Felici, "Hacking Smart Machines with Smarter Ones: How to Extract Meaningful Data from Machine Learning Classifiers" (International Journal of Security and Networks, 2015). The first property-inference attack: shadow models trained with and without the property, and a meta-classifier on their parameters. Read it for the recipe and its threat model.
- Ganju, Wang, Yang, Gunter & Borisov, "Property Inference Attacks on Fully Connected Neural Networks using Permutation Invariant Representations" (CCS 2018). The permutation problem for neural networks, neuron sorting, and the DeepSets meta-classifier. The source of Section 12.6.
- Suri & Evans, "Formalizing and Estimating Distribution Inference Risks" (PoPETs 2022). The property game as a test between two training distributions, and leakage measured in records. Exercise 3 in the exercise book follows their definition.
- Zhang, Tople & Ohrimenko, "Leakage of Dataset Properties in Multi-Party Machine Learning" (USENIX Security 2021). Black-box property inference from query outputs, including properties of attributes that were removed from the training data.
- Mahloujifar, Ghosh & Chase, "Property Inference from Poisoning" (IEEE S&P 2022). An attacker who can add a few records to the training set can make a property far easier to infer.
- Melis, Song, De Cristofaro & Shmatikov, "Exploiting Unintended Feature Leakage in Collaborative Learning" (IEEE S&P 2019). Properties of other participants' data, inferred from the updates they share during collaborative training.
- Salem, Cherubin, Evans, Köpf, Paverd, Suri, Tople & Zanella-Béguelin, "SoK: Let the Privacy Games Begin! A Unified Treatment of Data Inference Privacy in Machine Learning" (IEEE S&P 2023). Property inference as one game among the others of this part.
- Zaheer, Kottur, Ravanbakhsh, Póczos, Salakhutdinov & Smola, "Deep Sets" (NeurIPS 2017). The $\rho(\sum_j \phi(u_j))$ form of permutation-invariant functions.
- van der Vaart, *Asymptotic Statistics* (1998), ch. 5. Asymptotic normality of M-estimators and the sandwich covariance $H^{-1}GH^{-1}$, with the regularity conditions this chapter skips.

12.9. Cheat sheet

π_t mixes the majority and a subgroup of share t; $D \sim \pi_t^n$; $\theta = \mathcal{T}(D) \sim P_{n,t}$; $\bar{t} = \frac{t_0+t_1}{2}$. Table 12.8 collects the results.

Table 12.8.: Every result of the chapter.

Result	Formula	Note
Ceiling on any attack	$\text{Adv} \le \text{TV}(P_{n,t_0}, P_{n,t_1}) \le \text{TV}(\pi^n_{t_0}, \pi^n_{t_1})$	Equals $\text{TV}(\text{Bin}(n,t_0), \text{Bin}(n,t_1))$ when the group is visible
Property, released count	$2\Phi(\sqrt{n}\,\lvert t_1 - t_0\rvert/(2\sqrt{\bar{t}(1-\bar{t})})) - 1$	$\to 1$ as n grows
Membership, released count	$(1-t)\max_k \Pr[\text{Bin}(n-1,t) = k] \approx (1-t)/\sqrt{2\pi(n-1)t(1-t)}$	$\to 0$ like $1/\sqrt{n}$
Two Gaussians, shared Σ	Best accuracy $\Phi(\Delta/2)$; linear test $w = \Sigma^{-1}(m_1 - m_0)$	Δ the Mahalanobis distance
Trained parameters	$\theta \approx \mathcal{N}(\theta(t), H^{-1}GH^{-1}/n)$	M-estimators, large n
Meta-classifier accuracy	$\Phi(\kappa\sqrt{n}\,\lvert t_1 - t_0\rvert/2)$, $\kappa = \lvert\theta'\rvert_{\Sigma_1^{-1}}$	Count: $\kappa = 1/\sqrt{\bar{t}(1-\bar{t})}$
Hidden-unit symmetry	$f_{(gW,gb,gv,c)} = f_{(W,b,v,c)}$, $g = S\Pi$	$h!\,2^h$ copies; raw means vanish for tanh

Rules of thumb

- Property inference gets easier with more data, membership inference harder. What matters for a property is $\sqrt{n}\,\lvert t_1 - t_0\rvert$.
- A property changes about $n\lvert t_1 - t_0\rvert$ records at once. Defences sized for one record do not cover it.
- Removing the sensitive column does not remove the property. Whatever correlates with it moves the model.
- Train the meta-classifier on a representation that respects the model's symmetries: canonical order, sums over units, or outputs on probe inputs.
- Report accuracy for property attacks: there is one secret per dataset, not a population of targets.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

13. Memorization and training-data extraction

13.1. Why this matters

A language model trained on scraped text sees phone numbers, addresses, keys and private messages. Ask it to continue "Anna Smith's phone number is" and it may finish the sentence with the real number. Nobody queried a database and nothing was leaked by a breach: the number is stored in the weights, and it comes out through the model's normal interface. That is what **training-data extraction** means. Membership inference (Membership inference) asks "was this record in the training set?" and needs the record in hand. Extraction asks for more: the attacker gets the record itself, often without knowing it existed.

This chapter answers three questions, in order. The first is what it means for a model to memorize a string. There are several definitions, from "greedy decoding reproduces it when you prompt with the text that preceded it in training" to "some prompt makes the model emit it", and they differ in what the attacker must already know. Once you know what to measure, the second question is which strings get memorized. Two things decide it: duplication and model capacity. A string repeated often, or seen by a model large enough to give it a context of its own, comes back verbatim. The third question is how you measure memorization and how an attacker finds it. Planted random secrets called canaries, and a score called exposure, measure how much a model has memorized a secret you put there yourself. Generate-rank-verify pipelines find real training data among a model's samples.

13.1.1. The one idea: the rank of the secret under the model

A language model is a distribution over strings: it gives every possible string a probability. Memorization is the model putting a training string *far above* the other strings that could have stood in its place. To make that precise, fix the context in which the secret appears, say "my pin is", and list all possible fillers r (all five-digit pins) in order of the model's probability $p(r \mid \text{context})$, most likely first. The **rank** of the true secret is its position in that list. It is also the number of guesses an attacker needs if they try fillers from most to least likely, so a low rank means the model has done most of the attacker's work.

Every measure in this chapter reads that rank, in a different way. Greedy extraction, which writes out the model's single most likely next symbol at each step, reproduces the secret when the secret's next symbol is the top choice, rank 1, at every step. Exposure compares the secret's rank with the size of the space, on a logarithmic scale: it is $\log_2$ of the number of candidates minus $\log_2$ of the rank, which counts how many halvings of the search the model has saved the attacker, in bits. Sampling-based extraction, which draws random outputs from the model, hits the secret quickly when the secret has high probability, and high probability is what puts it near the top of the list.

Table 13.1 lists the results of the chapter and where each is used.

Table 13.1.: The results of this chapter and where each is used.

Result	Where it is used
Definitions: verbatim, approximate, discoverable, extractable, k-eidetic	Reading and comparing extraction papers; choosing what to measure
Samples needed to extract a string, $m \approx \ln(1/\beta)/p$	Turning a probability into an attack budget
Copies needed for greedy extraction, k^*	Why duplication and model size drive memorization; why deduplication helps
Exposure, its null distribution	Auditing a trained model with planted canaries
$\Pr[\text{exposure} \geq e] \leq 2^{-e}$	
Exposure as bits of guessing saved	Linking memorization to min-entropy and guessing
Ranking generations by a likelihood ratio to a reference model	Extraction attacks without the prefix; the same calibration as in membership inference

The chapter builds on four earlier ones, and restates what it uses from them where it uses it. Algorithmic stability and generalization defined memorization as self-influence and showed that atypical examples are memorized most. Information theory introduced min-entropy and guessing, which give exposure its meaning. Threat models and privacy games set up attacks as games, and membership inference introduced the likelihood-ratio tests and reference models that extraction attacks reuse. The defence that bounds every quantity here, differential privacy, starts in Part 3.

The running example. Every experiment uses one toy that runs in seconds. The training set is a contact list: each line is a user name made of three random syllables, a space, and a five-digit pin, such as `toneka 14524`. There are 5,000 background lines, each seen once, and 600 secret lines, 100 each duplicated 1, 2, 4, 8, 16 and 32 times: 11,300 lines and 146,900 symbols in all, counting the end of each line. The model is a character n-gram language model with add-α smoothing, which predicts each character from the few characters before it by counting (Fact 2 below says exactly how). The order n (the model sees the last $n-1$ characters) plays the role of model size: an order-3 model stores 2,734 distinct n-grams and an order-8 model 54,614. It is the smallest model that has every property this chapter needs: a probability for every string, greedy decoding, sampling, and a capacity you can turn up. When a context is written out, `_` stands for a space: `gule_` is the five characters before the first digit of `migule 16430`.

13.2. Toolkit: three facts every derivation uses

13.2.1. Fact 1 — A language model is the chain rule

A language model never scores a whole string at once. It answers one smaller question many times: given everything written so far, how likely is each possible next symbol? Write $p(x_t \mid x_{<t})$

for the probability it gives to the next symbol x_t after the symbols $x_{<t} = x_1 \dots x_{t-1}$ that come before it. To get the probability of a whole string $x = x_1 \dots x_L$ of length L after a prompt c, you multiply these one-step probabilities. That is the chain rule of probability: the chance that the first symbol is x_1, *and then* the second is x_2 given the first, and so on, is the product of the conditional chances. Taking logarithms turns the product into a sum:

$$p(x \mid c) = \prod_{t=1}^{L} p(x_t \mid c, x_{<t}), \qquad \ln p(x \mid c) = \sum_{t=1}^{L} \ln p(x_t \mid c, x_{<t}). \tag{13.1}$$

For the pin `16430` after the prompt `migule`, the model multiplies five numbers: the probability of `1` after `migule_`, of `6` after `migule_1`, of `4` after `migule_16`, of `3` after `migule_164`, and of `0` after `migule_1643`.

The negative of the log in base 2, $-\log_2 p(x \mid c)$, is called the **log-perplexity** of x in bits. It measures how surprised the model is by x: a string the model finds likely has low log-perplexity, and each extra bit means the model found it half as likely. Every score in this chapter (exposure, the probability that a sample reproduces a string, the ranking of generations) is built from this sum of per-symbol log-probabilities.

13.2.2. Fact 2 — An n-gram model is a table of counts

An order-n model looks only at the last $n-1$ symbols, which it calls the context c. Training is nothing more than counting: for every context c and every possible next symbol w, the model stores the count $N(c, w)$ of times w followed c in the training text. The natural prediction would be the fraction of times w followed c, but that gives probability 0 to anything never seen, and one zero in the chain rule makes a whole string impossible. **Add-α smoothing** fixes this by pretending every symbol was seen α extra times in every context. With an alphabet of V symbols, the α is added to each of the V counts, so the total grows by αV:

$$p(w \mid c) = \frac{N(c, w) + \alpha}{N(c) + \alpha V}, \qquad N(c) = \sum_{w} N(c, w). \tag{13.2}$$

Three consequences of this formula do most of the work in the chapter.

The first is that **training is counting**. A line of text passes through a sequence of (context, next symbol) pairs, and training on it adds 1 to each of those counts. Training on k copies of a line adds k to each, and changes no other count.

The second is that **smoothing does not change the most likely symbol**. In a fixed context c, every count gets the same $+\alpha$, and the denominator $N(c) + \alpha V$ is the same for every w. So the symbol with the largest count still has the largest probability: $\arg\max_w p(w \mid c) = \arg\max_w N(c, w)$, where $\arg\max_w$ means "the w that makes this largest". Greedy decoding, which emits the most likely symbol, therefore just follows the largest counts.

The third is what happens at a **context that occurs only in the copies of one line**. Say the line has been inserted k times and its context c appears nowhere else. Then the line's own next symbol has count k, every other symbol has count 0, and the total is $N(c) = k$. Putting those counts into Equation 13.2 gives

$$p(\text{true} \mid c) = \frac{k + \alpha}{k + \alpha V}, \qquad p(\text{other} \mid c) = \frac{\alpha}{k + \alpha V}. \tag{13.3}$$

The two share a denominator, so the true symbol is $(k+\alpha)/\alpha$ times as likely as any other, whatever V is. A context never seen at all has every count 0, and gives every symbol $\alpha/(\alpha V) = 1/V$.

The toy uses $\alpha = 0.1$ and $V = 39$ symbols: 26 letters, 10 digits, the space, a start symbol and an end-of-line symbol, $26 + 10 + 1 + 1 + 1 = 39$. Table 13.2 evaluates Equation 13.3. For one copy, $\alpha V = 3.9$, so $p(\text{true} \mid c) = 1.1/4.9 = 0.2245$ and $p(\text{other} \mid c) = 0.1/4.9 = 0.0204$, a ratio of $1.1/0.1 = 11$. The last column turns the ratio into bits, $\log_2 11 = 3.46$: one copy makes the true digit as much more likely than any rival as 3.46 halvings would.

Table 13.2.: Add-α probabilities at a context that occurs only in the k copies of one line ($\alpha = 0.1$, $V = 39$).

Copies k	$p(\text{true} \mid c)$	$p(\text{other} \mid c)$	Ratio	Bits per digit, $\log_2$ ratio
1	0.2245	0.0204	11.0	3.46
4	0.5190	0.0127	41.0	5.36
16	0.8090	0.0050	161.0	7.33

So a single copy already makes the true digit 11 times more likely than any other. That is the mechanism of memorization in this toy: a context that belongs to one training line alone, followed by that line's next symbol.

13.2.3. Fact 3 — Guessing in order of probability takes "rank" guesses

Let a secret r come from a finite set $\mathcal{R}$ of possible values, with $|\mathcal{R}|$ elements. An attacker who has a score for every candidate, here the model's probability, should guess candidates from the highest score down, because each guess then has the best remaining chance of being right. The number of guesses this takes is the rank of the true secret: the number of candidates at least as likely as it,

$$\text{rank}(r) = \big|\{r' \in \mathcal{R} : p(r') \geq p(r)\}\big|, \tag{13.4}$$

counting r itself and every candidate tied with it (ties are counted against the attacker, who may have to try them all first).

Without a model, the attacker has no reason to prefer one order over another. A uniform secret is then equally likely to sit at each position $1, \ldots, |\mathcal{R}|$ of whatever order is used, so the number of guesses is on average the middle position, $(|\mathcal{R}| + 1)/2$. Information theory measures the same difficulty with one guess. The best single guess is the most likely value, and it succeeds with probability $\max_r p(r)$, which Information theory calls the **vulnerability**. Its negative logarithm, the **min-entropy** $H_\infty = -\log_2 \max_r p(r)$, expresses that difficulty in bits: a secret with H_∞ bits is as hard to guess in one try as a uniform choice among 2^{H_∞} values. A uniform 5-digit pin has 10^5 equally likely values, so $H_\infty = \log_2 10^5 = 5 \log_2 10 = 16.61$ bits.

The rank is what the rest of the chapter measures. Exposure is defined from the rank of a secret among all secrets of its format, and greedy decoding asks for rank 1 among the possible next symbols at every step.

13.3. Verbatim, approximate, discoverable, extractable

13.3.1. Context

"The model memorized its training data" can mean several things. A model that completes a famous quotation has memorized it, but so has every reader, and nothing private leaked. A model that emits a phone number seen once in training has leaked it. Some definitions assume the attacker knows the beginning of the training string. Others ask whether any prompt at all brings it out. Before measuring anything you need to say which notion you measure, because the numbers differ by orders of magnitude.

13.3.2. Statement

Let f be a language model trained on a dataset X of documents, and let $\text{greedy}_f(c)$ be the string produced by greedy decoding from the prompt c: at each step, append the most likely next symbol. The definitions below are parallel options that differ in what the attacker gets to use and in how close the output must be.

1. **Extractable** (Carlini et al. 2021): a string s is extractable from f if there is some prompt c, any prompt at all, such that $\text{greedy}_f(c)$ begins with s.
2. **k-eidetic** (Carlini et al. 2021): s is k-eidetically memorized if it is extractable and it appears in at most k documents of X. Small k is the dangerous case: a secret seen once and given back.
3. **Discoverable** (Carlini et al. 2023, named by Nasr et al. 2023): if the training data contain $[p \,\|\, x]$, a prefix p followed by a suffix x, then x is discoverably memorized if prompting with that same prefix makes greedy decoding produce the suffix: $\text{greedy}_f(p)$ begins with x.
4. **Approximate** (Ippolito et al. 2023): x is approximately memorized if $\text{greedy}_f(p)$ is within distance τ of x in some string distance d. In this chapter's toy d is the number of wrong digits and $\tau = 1$, so a pin with at most one wrong digit counts. Definitions 1 to 3 are **verbatim**: the output must match exactly.

Two relations follow directly from the definitions. Discoverable is a special case of extractable, with the training prefix as the prompt, and an exact match is a special case of a close one:

$$\text{discoverable} \;\Rightarrow\; \text{extractable}, \qquad \text{verbatim} \;\Rightarrow\; \text{approximate}. \tag{13.5}$$

Real systems sample rather than decode greedily, so it is useful to know how many samples bring a string out. Each sample is a fresh, independent try that produces the string x with its probability $p = p(x \mid c)$ after the prompt c, like a biased coin that lands heads with probability p. The chance that at least one of m samples is x is one minus the chance that all of them miss, $1 - (1-p)^m$, and this is at least $1 - e^{-mp}$. Asking that to be at least $1 - \beta$, for a small failure probability β, gives a sample budget:

$$m \geq \frac{\ln(1/\beta)}{p} \quad \text{samples suffice to extract } x \text{ with probability at least } 1 - \beta. \tag{13.6}$$

13.3.3. Derivation

Discoverable implies extractable. If $\text{greedy}_f(p)$ begins with x, then $c = p$ is a prompt that makes the model emit x, which is the definition of extractable. The converse fails: an extractable string may come out after a prompt that is not its training prefix, or after a prompt the attacker cannot know, while its own prefix leads elsewhere.

Verbatim implies approximate. A verbatim match has distance 0, and $0 \le \tau$.

Samples needed. The idea is to compute the probability that every sample misses, and bound it by an exponential that is easy to solve for m.

1. **One sample hits with probability p.** A temperature-1 sample from c draws each next symbol with exactly the model's probabilities $p(x_t \mid c, x_{<t})$. By the chain rule (Equation 13.1), the chance that it writes out exactly x is the product of those probabilities, which is $p = p(x \mid c)$.
2. **All m samples miss with probability $(1-p)^m$.** Each sample misses with probability $1-p$, and the samples are independent, so the misses multiply.
3. **Replace $1-p$ by e^{-p}.** The curve e^{-p} is convex, and the straight line $1-p$ is its tangent at $p = 0$ (both equal 1 there, with slope -1). A convex curve lies above its tangent lines, so $1 - p \le e^{-p}$, and raising both sides to the power m gives $(1-p)^m \le e^{-mp}$.
4. **Solve for m.** The miss probability is then at most β as soon as $e^{-mp} \le \beta$. Taking logarithms, $-mp \le \ln \beta$, that is $m \ge \ln(1/\beta)/p$.

13.3.4. Interpretation

The definitions answer different threat models. Discoverable memorization assumes an attacker who holds the training prefix. That is a strong assumption, but it makes the quantity easy to measure: prompt with every training prefix and compare the output with the true suffix. Extractable memorization fits the real attacker, who has only the model, but it is hard to measure, because you would have to search over all prompts. So the discoverable rates are what papers report on open models, and attacks then show how much of that an outsider can actually reach.

Verbatim matching is a weak notion of leakage, because it misses near copies. A model that emits a phone number with one digit changed, or a paragraph with its punctuation changed, has leaked it to any reader. Ippolito et al. (2023) show that a filter blocking verbatim training text is easy to bypass with prompts that change the style, while the approximate copies still come out. A verbatim rate is therefore a lower bound on the leak, not a measure of it.

Greedy decoding is a convenience for measurement; real deployments sample. Equation 13.6 converts a probability into an attack budget. For a string with probability 10^{-3} after its prefix and 99% confidence ($\beta = 0.01$), the budget is $\ln(100)/10^{-3} = 4.605/0.001$, so the string comes out within about 4,600 samples.

13.3.5. Example: one secret, six model sizes

Take the secret line `migule 16430`, inserted 4 times, and prompt each model with its prefix `migule`. Greedy decoding gives `07` followed by an end of line at order 3, `81554` at order 4, `14032` at order 5, and the true `16430` at orders 6, 7 and 8. The order-5 output gets the first and fourth digits right and the other three wrong, so it is not approximately memorized either. The reason the switch happens at order 6 is Fact 2. An order-6 model reads five characters of context, and from the first digit on its contexts (`gule_`, then `ule_1`, and so on) occur in this line and nowhere else. In such a context the line's own next digit has count 4 and every rival digit has count 0, so greedy decoding, which follows the largest count, emits the line's digit every time. Shorter contexts, such as the four characters `ule_` an order-5 model reads, are shared with other lines, whose digits can outvote the four copies.

Sampling gives a graded version of the same picture. At order 5, the probability that one sample from the true prefix reproduces a secret's pin, averaged over the 100 secrets of each group, is 1.4×10^{-3} at 4 copies and 0.142 at 32 copies. With $\beta = 0.01$ that means 3,252 and 31 samples for a 99% chance of extracting the pin. These are the exact budgets, the smallest m with $(1-p)^m \leq \beta$; Equation 13.6 gives slightly larger ones, since it replaces $1-p$ by the larger e^{-p}. For $p = 0.142$, for instance, $\ln(0.01)/\ln(1-0.142) = 4.605/0.153 = 30.1$, which rounds up to 31. Repetition turns a needle in a haystack into the model's favourite answer.

13.3.6. Check yourself

- Can a string be extractable but not discoverable? *(Yes. Another prompt, say a different name that shares the last syllables, may lead greedy decoding into it while its true prefix leads elsewhere.)*
- A model emits a training string with probability 10^{-6} after its prefix. Is it memorized? *(Not in the greedy sense, and by Equation 13.6 an attacker needs about* $\ln(100)/10^{-6} \approx 4.6 \times 10^6$ *samples for 99% success. Whether that counts as a leak depends on the attacker's budget, which is why exposure, which uses the rank rather than the probability, is useful.)*

13.4. Duplication and model size: the copies a string needs

13.4.1. Context

Carlini et al. (2023) measured discoverable memorization on a family of open models and found that it grows log-linearly with three things: the number of times a string is duplicated in the training set, the size of the model, and the length of the prompt. Kandpal, Wallace & Raffel (2022) found that a sequence repeated 10 times is generated about 1,000 times more often than one seen once. The n-gram toy lets you derive why, for every string, from counts.

13.4.2. Statement

The idea is a vote. Greedy decoding follows the largest count at each step (Fact 2). Each copy of a line adds one vote for the line's own next symbol in every context the line passes through. So the line comes out when, at every step, its own votes plus whatever the rest of the corpus already gave its symbol beat the strongest rival symbol. The number of copies needed is set by the step where the rival is furthest ahead.

To write this down, let s be a training line with prompt p and secret suffix $x = x_1 \dots x_L$, inserted k times. Let c_t be the context at step t (the last $n-1$ symbols of the prompt followed by the first $t-1$ symbols of the suffix, $p\,x_{<t}$), and let $N_{-}(c, w)$ be the counts in the rest of the training set, *without* the copies of s. At step t, the **gap** is how far the strongest rival $w \neq x_t$ is ahead of the true symbol in those counts, $\max_{w \neq x_t} N_{-}(c_t, w) - N_{-}(c_t, x_t)$. Assume each context c_t occurs only once within s, so a copy of s adds to the true pair (c_t, x_t) and to no rival pair (c_t, w); this holds for almost every line. Then greedy decoding from p emits x if and only if $k \geq k^*(s)$, where the copies needed are the largest gap plus one:

$$k^*(s) = \max_{t \leq L} \Big[\max_{w \neq x_t} N_{-}(c_t, w) - N_{-}(c_t, x_t) \Big] + 1. \tag{13.7}$$

There is one refinement for ties. Greedy decoding breaks a tie toward the symbol with the lowest code, so at a step where the true symbol has the lower code, a tie is already a win, and the bracket is reduced by one there.

Over a population of lines, the fraction memorized at k copies is then the fraction of lines whose k^* is at most k:

$$\Pr[\text{discoverably memorized at } k \text{ copies}] = \Pr[k^*(s) \leq k]. \tag{13.8}$$

In particular, a line whose contexts are unique to it ($N_{-} = 0$ everywhere, so every gap is 0) has $k^* = 1$: it is memorized from a single copy.

13.4.3. Derivation

1. **Greedy picks the largest count.** By Fact 2, adding α to every count does not change which count is largest, so at context c_t greedy decoding emits $\arg\max_w N(c_t, w)$, the symbol with the largest count.
2. **What the copies add.** Training is counting (Fact 2), so k copies of s add k to $N(c_t, x_t)$. By the assumption that c_t occurs once in s, they add nothing to $N(c_t, w)$ for $w \neq x_t$. With the copies, the true symbol's count is $N_{-}(c_t, x_t) + k$ and each rival's is still $N_{-}(c_t, w)$.
3. **Winning one step.** The true symbol wins strictly when its count beats the largest rival count, $N_{-}(c_t, x_t) + k > \max_{w \neq x_t} N_{-}(c_t, w)$. Moving $N_{-}(c_t, x_t)$ to the right, this is $k > \text{gap}_t$, with gap_t the bracket in Equation 13.7. Since k and the gap are whole numbers, that is $k \geq \text{gap}_t + 1$.
4. **Every step must win.** Greedy decoding stays on the line's path $c_1, c_2, \dots$ only while it keeps emitting the true symbols. One wrong symbol puts it in a different context, from which in general it does not return to x. So x comes out exactly when every step wins, that is when $k \geq \text{gap}_t + 1$ for every t, which is $k \geq \max_t(\text{gap}_t + 1)$, Equation 13.7.
5. **Unique contexts.** If $N_{-}(c_t, \cdot) = 0$ for every t, every gap is $0 - 0 = 0$, so $k^* = 1$.

6. **The population rate.** Each line is memorized at k copies exactly when $k^*(s) \leq k$. The fraction of lines memorized is therefore the fraction for which this holds, which is $\Pr[k^*(s) \leq k]$ for a line drawn at random from the population.

Figure 13.1 draws steps 3 and 4 for one line.

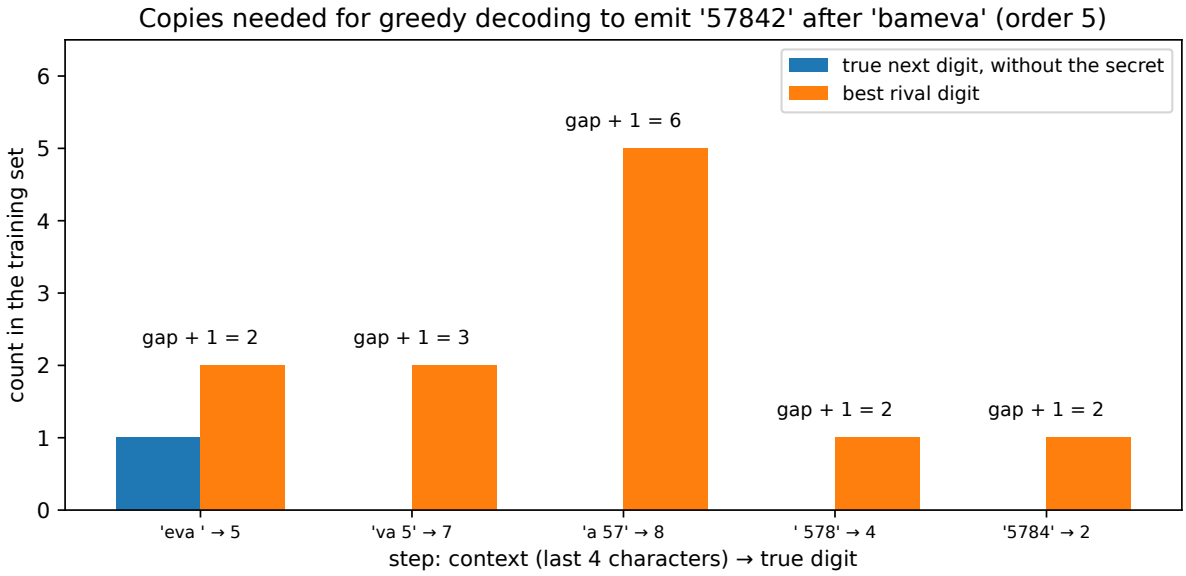


Figure 13.1.: For one secret line inserted once, an order-5 model: at each step of its pin, the count of the true next digit (blue) and of its strongest rival (orange) in the rest of the training set. The label is the gap plus one, the copies that step needs. The line needs as many copies as its hardest step.

13.4.4. Interpretation

Duplication works as a vote. Each copy is one more vote for the line's continuation in every context it passes through, and a line comes out when it outvotes the rest of the corpus at its most contested step.

Model size works by splitting the vote. A longer context is a finer partition of the training positions: fewer positions share any one context of length 7 than of length 2. So the rival counts in a longer context are smaller, while the line's own k votes stay with it in every context it passes through. In the limit every line has contexts of its own and $k^* = 1$. This is the toy version of "larger models memorize more": a larger model can afford to give rare strings their own parameters.

For the same reason, rare data are exposed first. A line whose name shares few syllables with other names reaches a context of its own sooner. This is the pattern Algorithmic stability and generalization found for **self-influence**, how much a model's prediction on a training example changes when that one example is removed: atypical examples can only be fitted by memorizing them, so they have the largest self-influence.

Deduplication is the cheapest defence, because it lowers k for every repeated string. Lee et al. (2022) found that models trained on deduplicated data emit memorized text about ten times less often, without losing quality, and Kandpal et al. (2022) found that deduplication sharply lowers the success of extraction attacks. It does nothing for singletons in a high-capacity model, where $k = 1$ already and $k^* = 1$.

13.4.5. Example: discoverable memorization against copies and order

The script prompts each of the 600 secrets with its true prefix (the name and the space), decodes five symbols greedily, and compares them with the pin. Table 13.3 gives the verbatim rate and the approximate rate (at most one digit wrong) for each order and number of copies. It also computes k^* for every secret from Equation 13.7, and the prediction $\Pr[k^* \le k]$ matches the verbatim rate exactly in all 36 cells, as the derivation says it must: the formula is not an approximation but a restatement of what greedy decoding does with counts.

Table 13.3.: Fraction of 100 secret lines per cell whose pin greedy decoding reproduces from the true prefix, verbatim / with at most one digit wrong. Rows are model orders, columns copies in the training set.

Order n	$k = 1$	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$
3	0.00 / 0.00	0.00 / 0.03	0.00 / 0.00	0.00 / 0.00	0.00 / 0.02	0.00 / 0.05
4	0.00 / 0.00	0.00 / 0.00	0.00 / 0.01	0.00 / 0.00	0.00 / 0.00	0.13 / 0.17
5	0.00 / 0.00	0.00 / 0.00	0.05 / 0.06	0.11 / 0.15	0.42 / 0.42	0.70 / 0.70
6	0.08 / 0.13	0.65 / 0.69	0.76 / 0.78	0.80 / 0.85	0.90 / 0.91	0.96 / 0.97
7	0.70 / 0.78	0.95 / 0.96	0.98 / 0.98	0.98 / 0.99	0.99 / 0.99	0.97 / 0.98
8	0.98 / 0.98	1.00 / 1.00	1.00 / 1.00	0.99 / 0.99	1.00 / 1.00	1.00 / 1.00

Read it along both axes. Along a row, more copies means more memorization: at order 5 the rate climbs from 0 at one copy to 0.70 at 32. Down a column, a larger model memorizes more: a line seen once is safe at orders up to 5, memorized 8% of the time at order 6, 70% at order 7 and 98% at order 8. The order-8 model has memorized its training set almost completely, singletons included. In the language of the definitions, those singletons are 1-eidetic: seen in one document and given back. Figure 13.2 plots the same table for a second training set.

The approximate rates are barely above the verbatim ones. In this toy, a pin either sits in contexts of its own and comes out whole, or it loses a contest early, after which greedy decoding is in the wrong context and wanders off. Natural text has many near-ties (synonyms, punctuation), and that is where approximate memorization adds the most (Ippolito et al. 2023).

The real experiment of Carlini et al. (2023) has the same shape at scale. Take GPT-Neo models from 125 million to 2.7 billion parameters and GPT-J (6 billion), all trained on the Pile, a large public collection of web and book text. Sample training sequences, grouped by how many times they are duplicated in the Pile. Prompt each model with the first 50 tokens and decode 50 tokens greedily. Count exact matches. They found that the largest model reproduces at least 1% of its training data this way, and that the rate rises log-linearly with model size, duplication and prompt length. The toy replaces tokens by characters, parameters by the context length and the Pile by a contact list, and keeps the measurement.

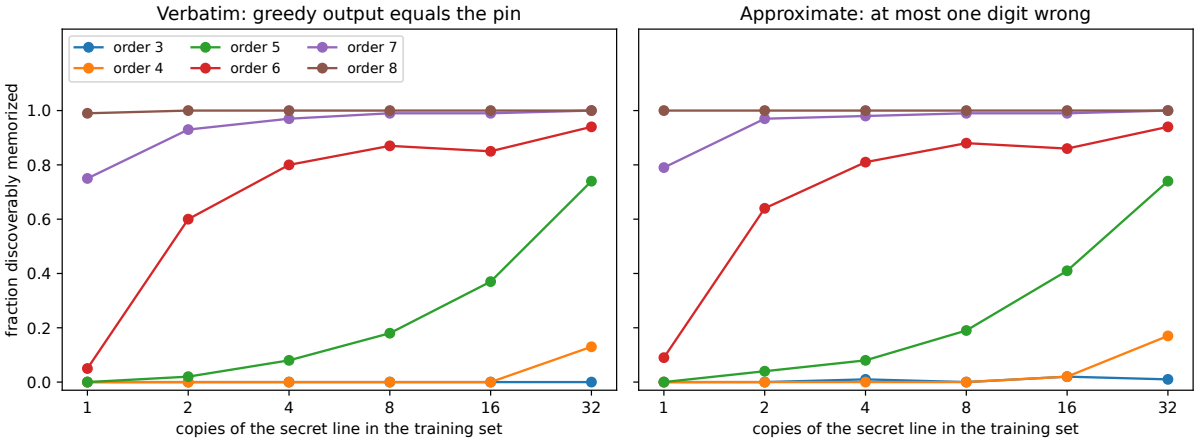


Figure 13.2.: Discoverable memorization against copies of the secret in the training set, for n-gram models of order 3 to 8, on an independent draw of the corpus. Left: greedy decoding reproduces the pin exactly. Right: at most one digit is wrong. Both rise with duplication and with order, and the approximate rate is only slightly higher.

13.4.6. Check yourself

- In Figure 13.1, why does the line need as many copies as its worst step, and not the sum over steps? *(The copies count at every step at once: each copy adds one to every pair on the line's path. So k copies close every gap up to k, and the largest gap decides.)*
- Does deduplication help an order-8 model on this corpus? *(Barely. Its singletons are already memorized 98% of the time, because their contexts are unique. Deduplication works by lowering k, and here $k^* = 1$.)*

13.5. Canaries and the exposure metric

13.5.1. Context

Discoverable memorization is all or nothing: the secret comes out or it does not. An auditor wants a graded answer: how much closer to the secret does the model put an attacker? Carlini et al. (2019, "The Secret Sharer") proposed planting **canaries**, random secrets of a fixed format such as "my pin is" followed by five random digits, in the training data, then measuring where the trained model ranks the planted secret among all secrets of the same format. Because the canary is random and planted, you know the right answer and the size of the space, so you can quantify memorization exactly.

13.5.2. Statement

A canary has a format $s[\cdot]$ with a hole, and a randomness space $\mathcal{R}$ of values that can fill the hole. Here $s[r]$ is "my pin is r", with r one of the $|\mathcal{R}| = 10^5$ pins. A value $\hat{r}$ is drawn uniformly at random from $\mathcal{R}$, and $s[\hat{r}]$ is inserted k times into the training data. After training, order all of $\mathcal{R}$

by the model's probability $p(r \mid \text{"my pin is"})$; the **rank** of $\hat{r}$ is its position in that order (Fact 3), the number of guesses an attacker who follows the model needs.

The exposure turns the rank into a number of bits saved. Without the model, the attacker faces all $|\mathcal{R}|$ candidates, which is $\log_2 |\mathcal{R}|$ bits of search. With the model, it faces $\text{rank}(\hat{r})$ candidates, $\log_2 \text{rank}(\hat{r})$ bits. The **exposure** is the difference:

$$\text{exposure}(\hat{r}) = \log_2 |\mathcal{R}| - \log_2 \text{rank}(\hat{r}). \tag{13.9}$$

Two properties follow from this definition directly. Because the rank lies between 1 and $|\mathcal{R}|$, the exposure lies between 0 and $\log_2 |\mathcal{R}|$, with the maximum when the canary is the model's top candidate. And exposure counts guesses saved: an attacker who guesses in the model's order needs $\text{rank}(\hat{r}) = 2^{\log_2 |\mathcal{R}| - \text{exposure}}$ guesses instead of $|\mathcal{R}|$, so each bit of exposure halves the work.

The third property is what makes exposure an audit tool: you know how it behaves when nothing was memorized. If the model does not depend on $\hat{r}$, for instance because the canary was never inserted, then $\hat{r}$ is just a random point in a fixed ordering, and a high exposure is unlikely. Precisely, for every $e \geq 0$,

$$\Pr\left[\text{exposure}(\hat{r}) \geq e\right] \leq 2^{-e}, \tag{13.10}$$

and, when no two candidates tie, its average is a fixed small number of bits:

$$\mathbb{E}\big[\text{exposure}(\hat{r})\big] = \log_2 |\mathcal{R}| - \frac{\log_2(|\mathcal{R}|!)}{|\mathcal{R}|} \;\longrightarrow\; \log_2 e = 1.4427 \text{ bits}. \tag{13.11}$$

So exposure is $-\log_2$ of a p-value. A **p-value** is the probability, under the assumption that nothing happened, of seeing a result at least as extreme as the one observed. Here it is the probability that a canary that was never inserted would look at least this memorized, and Equation 13.10 says it is at most $2^{-\text{exposure}}$.

13.5.3. Derivation

Range and guesses. The rank is between 1 and $|\mathcal{R}|$, so $\log_2 \text{rank}$ is between 0 and $\log_2 |\mathcal{R}|$. Subtracting from $\log_2 |\mathcal{R}|$ gives an exposure between $\log_2 |\mathcal{R}|$ and 0. Solving Equation 13.9 for the rank gives $\text{rank} = 2^{\log_2 |\mathcal{R}| - \text{exposure}}$.

Null distribution. The idea is that, without the canary, the model's scores are fixed before $\hat{r}$ is drawn, so $\hat{r}$ lands at a uniformly random position in their order.

1. **The order is fixed.** If the model was trained without the canary, its scores $p(r \mid \cdot)$ are fixed numbers that do not depend on $\hat{r}$. With no ties they order $\mathcal{R}$ into positions $1, \ldots, |\mathcal{R}|$.
2. **The canary's position is uniform.** $\hat{r}$ is uniform on $\mathcal{R}$ and independent of the scores, so its position is equally likely to be any of $1, \ldots, |\mathcal{R}|$.
3. **Count the positions with high exposure.** By Equation 13.9, exposure $\geq e$ means $\log_2 \text{rank} \leq \log_2 |\mathcal{R}| - e$, that is $\text{rank} \leq |\mathcal{R}| 2^{-e}$. There are $\lfloor |\mathcal{R}| 2^{-e} \rfloor$ such positions (the whole numbers up to that bound), each with probability $1/|\mathcal{R}|$, so the probability is $\lfloor |\mathcal{R}| 2^{-e} \rfloor / |\mathcal{R}| \leq 2^{-e}$.
4. **Ties only help the bound.** The rank counts every candidate at least as likely as $\hat{r}$, so a tie can only raise the rank of $\hat{r}$. A higher rank means a lower exposure, so the probability of a high exposure can only fall, and the bound still holds.

5. **The mean.** With the rank uniform on $1, \dots, |\mathcal{R}|$, the average of $\log_2$ rank is $\frac{1}{|\mathcal{R}|}\sum_{i=1}^{|\mathcal{R}|}\log_2 i$. A sum of logarithms is the logarithm of the product, and $1 \cdot 2 \cdots |\mathcal{R}| = |\mathcal{R}|!$, so this is $\log_2(|\mathcal{R}|!)/|\mathcal{R}|$; subtracting it from $\log_2|\mathcal{R}|$ gives Equation 13.11. For the limit, Stirling's approximation of the factorial says $\ln N! = N\ln N - N + O(\ln N)$, where $O(\ln N)$ is a correction that grows no faster than a multiple of $\ln N$. Dividing by $N\ln 2$ to get base-2 logarithms per candidate, $\log_2(N!)/N = \log_2 N - 1/\ln 2 + O(\ln N/N) = \log_2 N - \log_2 e + O(\ln N/N)$, since $1/\ln 2 = \log_2 e$. The correction vanishes as N grows, so the mean exposure tends to $\log_2 e$.

A space of four candidates makes all of this concrete. With $|\mathcal{R}| = 4$ the rank is 1, 2, 3 or 4 with probability $\frac{1}{4}$ each, and the exposure is $2 - \log_2(\text{rank})$:

Table 13.4.: The four equally likely exposures of a canary from a space of four candidates that was never inserted.

Rank	1	2	3	4
Exposure $2 - \log_2(\text{rank})$	2	1	0.415	0

The third entry is $2 - \log_2 3 = 2 - 1.585 = 0.415$. The mean is $\frac{1}{4}(2 + 1 + 0.415 + 0) = 0.854$, which matches $2 - \log_2(24)/4 = 2 - 4.585/4$, since $4! = 24$. The probability of exposure at least 1 is $\frac{1}{2} = 2^{-1}$ (ranks 1 and 2), and of exposure at least 2 is $\frac{1}{4} = 2^{-2}$ (rank 1 only): the bound is tight at integer exposures.

13.5.4. Interpretation

Exposure is a membership test with a known null. "Was this canary inserted?" is a membership question (Membership inference), and the exposure is the score of a test that says "inserted" when the exposure is at least a threshold e. By Equation 13.10 that test has false positive rate (the chance of saying "inserted" about a canary that was not) at most 2^{-e}. In membership inference, calibrating a test means training **shadow models**, extra models trained with and without a record, to learn how the score behaves for non-members. Here none are needed: the randomness of the canary does the calibration, playing the role that the population of non-members plays in membership inference.

Exposure is also guessing work saved, in the sense of Fact 3. A uniform canary has min-entropy $\log_2|\mathcal{R}|$: before seeing the model, the attacker's best single guess succeeds with probability $1/|\mathcal{R}|$. After the model, the attacker who guesses in the model's order needs $2^{\log_2|\mathcal{R}| - \text{exposure}}$ tries. Carlini et al. (2019) note that exposure close to $\log_2|\mathcal{R}|$ means the secret can actually be extracted, for example by a shortest-path search: treat each digit position as a step, weight each choice of digit by its negative log-probability, and find the cheapest path through the model's predictions, which is the most likely pin.

For large spaces exposure has to be estimated, because enumerating $\mathcal{R}$ is impossible for, say, a 9-digit number. Two estimators replace enumeration. The first is sampling: score m random candidates and use the fraction of them at least as likely as the canary as the estimate of rank $/|\mathcal{R}|$. It cannot resolve exposures above $\log_2 m$, because a canary above all m sampled candidates only tells you its rank fraction is below about $1/m$. The second is extrapolation: fit a distribution

to the sampled scores and read off its tail beyond the canary's score; Carlini et al. (2019) fit a skew-normal, a normal distribution with an extra parameter that lets one tail be heavier than the other. The fit is only as good as its tail, and the tail is exactly the part you cannot sample.

Ties hide memorization. A model that has never seen the canary's contexts gives every candidate the same score. Every candidate is then tied with all the others, so every rank is $|\mathcal{R}|$ and every exposure 0. On this corpus the order-8 model without a canary gives every pin exactly the same probability after "my pin is", and the exposure of every candidate is 0.

Finally, canaries test defences, not only models. Because you control what was planted, you can compare exposure across two training runs that differ in one thing. Train with and without deduplication, and the change in exposure measures what deduplication bought. Train with a differentially private algorithm (Part 3), and exposure near the null mean says the planted secrets were not memorized; Carlini et al. (2019) found that private training kept their canaries at the exposure of canaries never inserted. Plant canaries, unlearn them, and measure exposure again, and you have a test of an unlearning method. Auditing differentially private training uses the same tool, with canaries designed to be as exposed as possible.

13.5.5. Example: one canary, three model sizes

The script plants "my pin is 58745" and computes the exact exposure by scoring all 10^5 pins. Table 13.5 gives it for orders 2, 3 and 4.

Table 13.5.: Exposure in bits of the canary "my pin is 58745" against the number of copies k in the training set. The maximum is $\log_2 10^5 = 16.61$.

Order n	$k=0$	$k=1$	$k=2$	$k=4$	$k=8$	$k=16$	$k=32$	$k=64$
2	0.40	0.41	0.43	0.47	0.55	0.73	1.21	2.54
3	0.89	4.05	4.44	4.71	5.24	6.59	9.69	15.61
4	3.76	9.98	12.15	15.02	15.61	16.61	16.61	16.61

The jump from $k=0$ to $k=1$ follows from Fact 2: what matters is whether a digit of the canary sits in a context that only the canary's copies fill. No background line contains `s_`, an `s` followed by a space, because names end in a vowel. So the three orders behave differently.

At order 2 the model reads one character of context, and the first digit's context is a single space, shared by all 11,300 lines. No digit of the canary has a context of its own, so one copy is a single vote among thousands, and it moves the exposure by $0.41 - 0.40 = 0.01$ bits.

At order 3 the first digit's context is `s_`, which occurs only in the canary. By Fact 2 and Table 13.2, one copy makes the first digit 11 times as likely as each other first digit, a factor worth $\log_2 11 = 3.46$ bits. The exposure goes from 0.89 to 4.05, a rise of $4.05 - 0.89 = 3.16$ bits: close to 3.46 but not equal, because the rank also depends on how the other pins' scores are spread.

At order 4 the first two digits have unique contexts, `is_` and `s_5`, worth about $2 \times 3.46 = 6.92$ bits. The exposure goes from 3.76 to 9.98, a rise of $9.98 - 3.76 = 6.22$ bits.

The remaining digits share their contexts with the 5,000 background pins and move only as the copies outvote them. Figure 13.3 shows the same movement on the full distribution of scores, and Figure 13.4 averages several canaries.

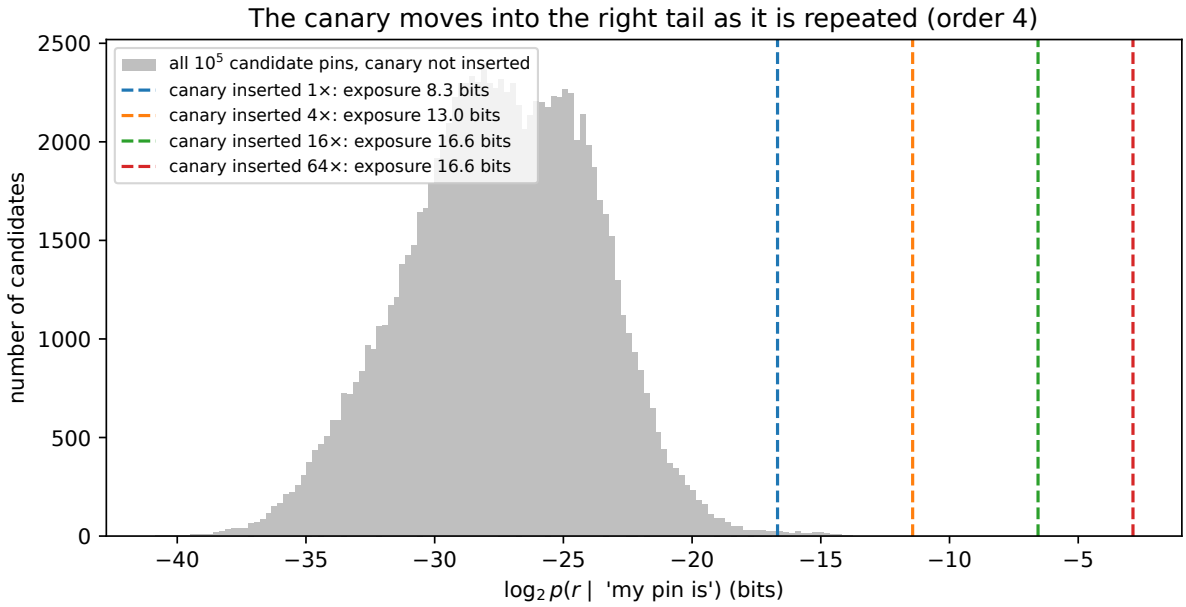


Figure 13.3.: Histogram of $\log_2 p(r \mid \text{my pin is})$ over all 10^5 pins for an order-4 model without the canary (grey), and the score of the canary after it is inserted 1, 4, 16 and 64 times (dashed). Each insertion pushes the canary further into the right tail, past more and more of the other pins.

The null in practice. For a pin that was never inserted, the derivation predicts a uniform rank. For orders 2, 3 and 4 the scores have no ties, so the exposures of all 10^5 uninserted pins are exactly the uniform-rank values: mean 1.443 bits, against 1.4426 from Equation 13.11, and a fraction $0.03125 = 1/32 = 2^{-5}$ with exposure at least 5, exactly on the bound Equation 13.10.

Estimating instead of enumerating. At order 3, the script estimates the canary's exposure from 10,000 random pins, by the two estimators of the interpretation: counting how many sampled pins score at least as high as the canary, and fitting a normal distribution to the sampled scores and reading off its tail (Table 13.6).

Table 13.6.: Exposure of the canary at order 3, computed exactly over all 10^5 pins and estimated from 10,000 random pins, by counting and by fitting a normal to their scores.

Copies k	Exact exposure	Rank	Sampling estimate	Normal fit
1	4.05	6,018	4.13	4.47
2	4.44	4,606	4.43	5.57
4	4.71	3,823	4.68	6.81
8	5.24	2,648	5.35	8.28

Counting is accurate here, because every exposure is well below $\log_2 10^4 = 13.3$ bits, the largest value 10,000 samples can resolve. The normal fit overestimates by up to 3 bits (at 8 copies,

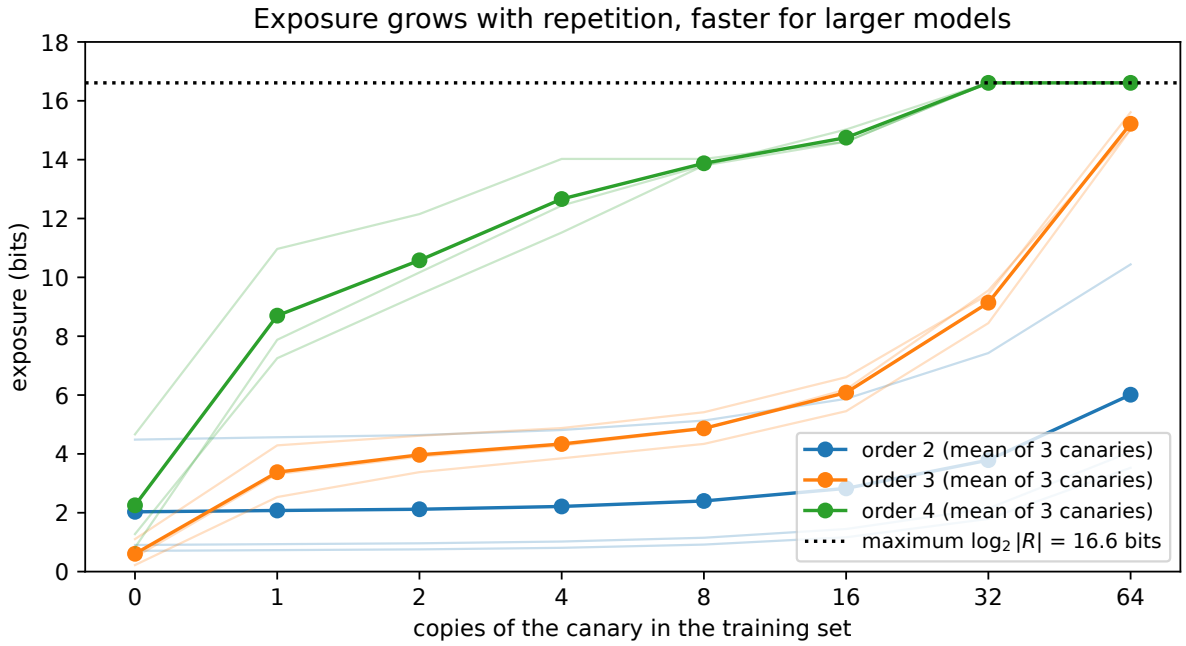


Figure 13.4.: Exposure against copies of the canary for n-gram models of order 2, 3 and 4. Thin lines are individual canaries and markers their mean; the dotted line is the maximum, $\log_2 10^5$. Larger models reach the maximum with fewer copies.

$8.28 - 5.24 = 3.04$). The score distribution has a heavier right tail than a normal, so the normal puts too few pins above the canary, which makes the canary's rank look smaller and its exposure larger. This is why the choice of tail model matters, and why Carlini et al. (2019) fit a skewed distribution.

13.5.6. Check yourself

- An auditor plants one canary, trains, and measures exposure 10 bits on a space of 10^9. What is the chance a non-inserted canary scores that high? *(At most $2^{-10} = 1/1024 \approx 0.001$, by Equation 13.10. The size of the space does not enter.)*
- Why does the order-2 model barely react to the canary even at 64 copies? *(Every context it sees is a single character, shared with thousands of training positions. The 64 votes are a small fraction of the counts in each context. The model is too small to give the canary anything of its own.)*

13.6. Extraction without the prefix: generate, rank, verify

13.6.1. Context

The attacker of Section 13.3 had the training prefix. A real attacker has only the model. Carlini et al. (2021) showed how such an attacker extracts training data from GPT-2: generate a large

number of samples, rank them by how likely they are to be memorized, and check the top of the ranking. The checking step is easy for whoever has the training data (a search) and, for an outside attacker, can be done against public sources. The hard step is the ranking, and it is a hypothesis test: for each sample, decide between two explanations of why the model produced it.

13.6.2. Statement

A model can find a string likely for two different reasons. The string may be ordinary, the kind of text any model of the language would find likely, or the model may have memorized it from its training set. Only the second is a leak. So for a generated string x, test H_0: "x is likely because text like it is common" against H_1: "x is likely because the target model memorized it". To score the two explanations you need a model of each. A **reference model** p_{ref}, trained on data from the same distribution but not on the target's training set, stands in for "common text", the distribution under H_0. The target model p_θ itself stands in for H_1. The best evidence for memorization is then how much more likely the target finds x than the reference does, which on a log scale is a difference. The most powerful test at every false positive rate ranks generations by this log-likelihood ratio,

$$\Lambda(x) = \ln p_\theta(x) - \ln p_{\text{ref}}(x), \tag{13.12}$$

not by $\ln p_\theta(x)$ alone.

13.6.3. Derivation

1. **The two hypotheses give two probabilities.** Under H_0 the string has probability $p_{\text{ref}}(x)$, and under H_1 it has probability $p_\theta(x)$.
2. **The best test thresholds their ratio.** The Neyman–Pearson lemma of Hypothesis testing says that, among all tests with a given false positive rate (the fraction of H_0 strings wrongly flagged), the one with the highest true positive rate (the fraction of H_1 strings caught) flags a string when the likelihood ratio $p_\theta(x)/p_{\text{ref}}(x)$ is above a threshold. The reason is a budget argument: each flagged string costs false positives in proportion to $p_{\text{ref}}(x)$ and gains true positives in proportion to $p_\theta(x)$, so for a fixed budget of false positives you catch the most by flagging the strings with the largest ratio first.
3. **Take logarithms.** The logarithm is increasing, so it keeps the order of the ratios, and thresholding the ratio is the same as thresholding its logarithm, Equation 13.12.
4. **Why raw likelihood falls short.** Ranking by $\ln p_\theta(x)$ alone is the same as using Equation 13.12 with p_{ref} constant, as if every string were equally common a priori. That assumption is wrong exactly for strings that are likely under any model, such as boilerplate, licences and famous quotations, and those are the strings raw likelihood puts first.

This is the same move as the per-example calibration of Membership inference: there, a record's raw loss under the target model is compared with the loss that models which did not see the record would give it.

13.6.4. Interpretation

Raw likelihood finds the common, not the private. The most likely generations of a large model are text that appears everywhere: software licences, news boilerplate, number sequences. These are in the training data, but extracting them reveals nothing. A reference model knows them too, so their ratio is close to 1 and the difference in Equation 13.12 cancels them.

The reference can be crude. Carlini et al. (2021) used three: a smaller model of the same family, the length of the text after compression with zlib (a general-purpose compressor, so the compressed length is a model-free measure of how predictable the text is), and the target's own likelihood on the lowercased text. Each gives a different idea of "generic", and each catches strings the others miss.

What the attacker cares about is precision, not recall. An attacker who checks the top 100 candidates by hand cares about how many of those are real training data, the precision of the top of the ranking, not about how many of all memorized strings it eventually finds. A high precision among a few flagged strings is a high true positive rate at a tiny false positive rate, the same operating point that matters in membership inference.

13.6.5. Example: extracting secrets from the contact-list model

The attacker samples 20,000 lines from the order-6 model, starting each at the beginning of a line. To make the ranking problem realistic, 50 **public** lines (think of a published directory) are added 100 times each to the target's training set; the attacker's own reference data, an independent contact list of 5,000 lines from the same generator, also contain them. The reference model is trained on those reference data. A generation counts as a hit if it is a private training line: in the target's training set and not public.

Of the 20,000 samples, 14,852 are distinct. Among them are 197 private training lines and all 50 public lines. By duplication group, the samples contain none of the secrets inserted 1, 2 or 4 times, 11% of those inserted 8 times, 86% of those inserted 16 times and all of those inserted 32 times. Compare the discoverable rates at order 6 (Table 13.3): 8% of the singletons and 65% of the lines seen twice come out when you hold their prefix. Without the prefix, a sample must first produce the right name before it can produce the pin, and only heavily duplicated lines make their names likely enough. What this attack extracts is much less than what is discoverable.

Ranking matters (Figure 13.5). Among the top 100 distinct generations by target likelihood, 45 are private training lines; the rest are mostly the 50 public lines, which, repeated 100 times each, are the most likely strings the model knows. Ranking by Equation 13.12, 85 of the top 100 are private training lines, because the reference model also finds the public lines likely and cancels them.

13.6.6. The production attack in prose

Nasr et al. (2023) carried this to deployed models. Their measurements separate the two notions of Section 13.3, discoverable and extractable. On open models whose training data are public (Pythia, GPT-Neo and others), they measured how much of what an attacker generates is verbatim training data, by checking every 50-token window of the output against a suffix array of the

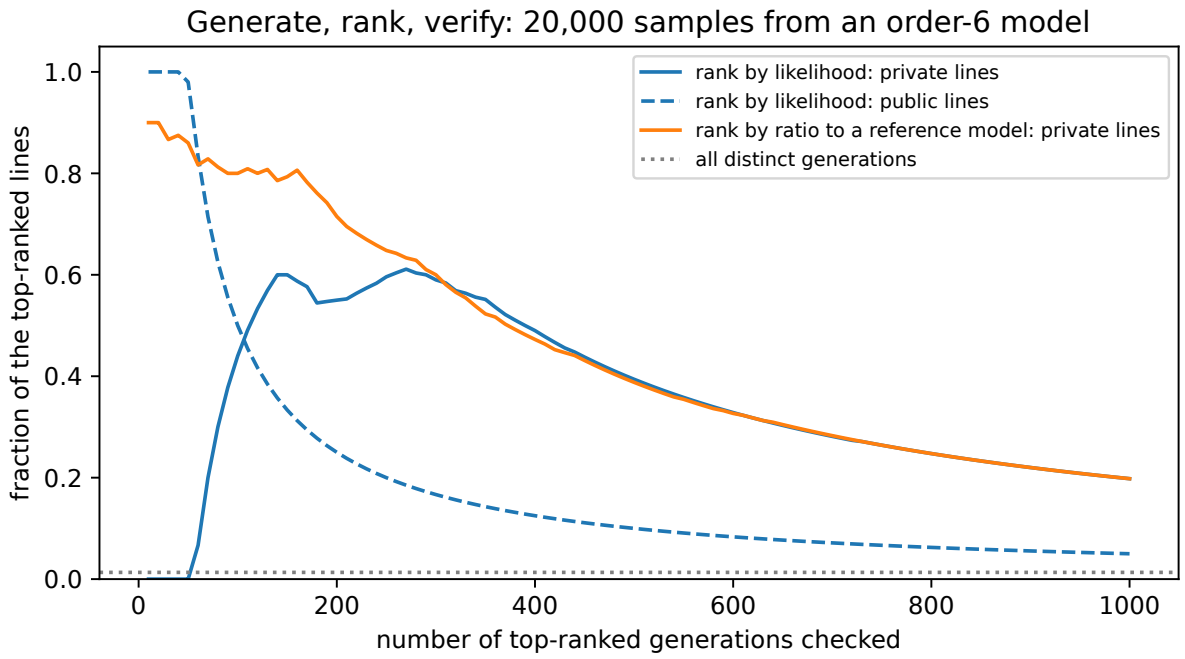


Figure 13.5.: Fraction of the top-ranked distinct generations that are private training lines (solid) or public lines (dashed), from 20,000 samples of an order-6 model trained with 50 public lines repeated 100 times. Ranking by likelihood puts the public lines first. Ranking by the likelihood ratio to a reference model trained on other data puts the private lines first.

training set, an index that finds any substring of a huge text quickly. For closed models, whose training data are unknown, they built a 9-terabyte auxiliary corpus of public web text and counted 50-token matches against it as a lower bound.

The aligned chat model they attacked, ChatGPT (gpt-3.5-turbo), rarely emitted training data when used normally: alignment training, a later stage that fine-tunes the model to follow instructions, had taught it to answer as an assistant, not to continue documents. Their **divergence attack** asked it to repeat a single word forever. After many repetitions the model diverged from the chat behaviour and began to emit text that looked like pretraining documents, and a sizeable part of that text matched the web corpus verbatim. They report that the attack made the model emit training data at a rate about 150 times higher than normal use, and that about 200 US dollars of queries extracted over 10,000 unique memorized training examples, including personal contact details. The structure is the one of this chapter: generate (with a prompt that escapes the aligned behaviour), verify (against a corpus), and estimate the total from what was found.

13.6.7. Check yourself

- Why are none of the secrets inserted 4 times generated, although at order 6 76% of them are discoverably memorized? *(An unprompted sample must first produce the secret's name. With 343,000 possible names, a sample hits a particular one only if that name is likely, which takes many copies.)*
- When would ranking by raw likelihood be as good as the ratio? *(When nothing is generically likely: if every string had the same prior probability, p_{ref} would be constant and the two rankings would agree.)*

13.7. Code: check every number by computation and simulation

The script code/memorization.py builds the contact list, trains the n-gram models by counting, and runs every experiment in this chapter: greedy and sampled extraction from true prefixes, the copies needed k^* for every secret, exact and estimated exposure of canaries over all 10^5 pins, and the generate-rank-verify attack. Every model is a small typed dataclass of sorted count arrays, so scoring all 10^5 pins is one vectorized lookup. From the repository root:

```
uv run python code/part02-attacks/memorization.py
```

Output (seed 0), summarized in Table 13.7:

Table 13.7.: The script's output with the default seed.

Experiment	Result
Corpus	5,000 background lines, 600 secrets (100 each at 1, 2, 4, 8, 16, 32 copies), 11,300 lines, 146,900 symbols
Model size, distinct n-grams	Order 3: 2,734; 4: 19,129; 5: 42,407; 6: 52,610; 7: 54,334; 8: 54,614

Experiment	Result
One secret, `migule 16430` × 4	$k^* = 151, 43, 9, 1, 1, 1$ for orders 3 to 8; greedy reproduces it from order 6
Discoverable memorization	Table 13.3; $\Pr[k^* \leq k]$ matches greedy in 36 of 36 cells
Sampling from the prefix, order 5	Mean $p(\text{pin} \mid \text{prefix})$ 1.4×10^{-3} at 4 copies, 0.142 at 32; 3,252 and 31 samples for 99%
Exposure null, orders 2–4	Mean 1.443 bits (theory 1.4426); $\Pr[\text{exposure} \geq 5] = 0.03125$ (bound 2^{-5}); order 8: all tied, exposure 0
Canary exposure	Table 13.5
Exposure estimates, order 3	Table 13.6
Extraction, order 6	14,852 distinct of 20,000 samples; 197 private lines; top 100 precision 0.45 by likelihood, 0.85 by ratio

tests/test_memorization.py checks that the output still matches.

What to take from it. Four things stand out. First, the count formula is exact: k^* from Equation 13.7 predicts which secrets greedy decoding reproduces in every cell, ties included, because it is a restatement of how greedy decoding reads counts. The `migule 16430` row shows the mechanism in one line: the secret, inserted 4 times, would need 151 copies at order 3, 43 at order 4 and 9 at order 5, but only 1 from order 6 on, which is why greedy decoding reproduces it from order 6. Second, duplication and model size both matter, and they trade off against each other: a small model needs dozens of copies, a large one needs one. Third, exposure behaves like a p-value. Without the canary, exposures follow the uniform-rank law exactly; with it, each digit of the canary that has a context of its own adds about 3.5 bits at one copy, the $\log_2 11$ of Fact 2. Fourth, the attacker without the prefix gets less, and needs a reference. Only heavily duplicated secrets come out of unprompted samples, and ranking by the likelihood ratio nearly doubles the precision of the top 100, from 0.45 to 0.85.

13.8. References

- Carlini, Liu, Erlingsson, Kos & Song, "The Secret Sharer: Evaluating and Testing Unintended Memorization in Neural Networks" (USENIX Security 2019): canaries, the exposure metric, its estimation by sampling and skew-normal extrapolation, and extraction of planted secrets by search.
- Carlini et al., "Extracting Training Data from Large Language Models" (USENIX Security 2021): extractable and k-eidetic memorization; the generate-rank-verify attack on GPT-2 with perplexity, reference-model, zlib and lowercase ranking.
- Carlini, Ippolito, Jagielski, Lee, Tramèr & Zhang, "Quantifying Memorization Across Neural Language Models" (ICLR 2023): memorization measured by prompting with training prefixes; log-linear growth with model size, duplication and prompt length.

- Nasr et al., "Scalable Extraction of Training Data from (Production) Language Models" (2023): discoverable against extractable memorization, measurement on open models against their training sets, and the divergence attack on an aligned chat model.
- Kandpal, Wallace & Raffel, "Deduplicating Training Data Mitigates Privacy Risks in Language Models" (ICML 2022): how the rate of regenerated sequences grows with duplication.
- Lee et al., "Deduplicating Training Data Makes Language Models Better" (ACL 2022): near-duplicate detection at scale and its effect on emitted training text and model quality.
- Ippolito et al., "Preventing Generation of Verbatim Memorization in Language Models Gives a False Sense of Privacy" (INLG 2023): approximate memorization and why verbatim filters fail.
- Feldman, "Does Learning Require Memorization? A Short Tale about a Long Tail" (STOC 2020): why rare examples must be memorized for accuracy, which explains why they are exposed first.
- Jurafsky & Martin, *Speech and Language Processing* (3rd ed. draft), ch. 3: n-gram language models, smoothing and perplexity.

13.9. Cheat sheet

p is a training prefix and x its suffix, k the number of copies of a string in the training set, n the order of an n-gram model, $\mathcal{R}$ a canary's randomness space. Table 13.8 collects the results.

Table 13.8.: Every result in this chapter.

Result	Statement	Key fact
Extractable	Some prompt c makes greedy decoding emit s	The real attacker's notion; hard to measure
Discoverable	Greedy from the true prefix p emits x	Easy to measure; implies extractable
k-eidetic	Extractable and in at most k documents	$k = 1$: a secret seen once comes back
Approximate	Greedy output within distance τ of x	Verbatim rates understate the leak
Samples needed	$m \geq \ln(1/\beta)/p(x \mid c)$	From $(1-p)^m \leq e^{-mp}$
Copies needed (n-gram)	$k^* = \max_t[\max_{w \neq x_t} N_{-}(c_t, w) - N_{-}(c_t, x_t)] + 1$	Largest gap on the path decides; unique contexts give $k^* = 1$
Exposure	$\log_2 \lvert\mathcal{R}\rvert - \log_2 \text{rank}$	Bits of guessing saved
Exposure null	$\Pr[\text{exposure} \geq e] \leq 2^{-e}$; mean $\to \log_2 e = 1.443$	Exposure is $-\log_2$ of a p-value
Ranking generations	$\ln p_\theta(x) - \ln p_{\text{ref}}(x)$	Neyman–Pearson; cancels generically likely text

Rules of thumb

- Memorization is a rank: the secret above the strings it could have been.

- Duplication is a vote in every context the string passes through; the most contested context decides.
- Larger models split contexts and give rare strings their own parameters, so they memorize more and from fewer copies.
- Deduplicate first: it is cheap and helps, but not for singletons in a high-capacity model.
- Report exposure, not just extraction: it is graded and has a known null.
- Rank generations against a reference, never by raw likelihood.
- An attacker without the prefix gets less than the discoverable rate suggests, but production attacks show "less" is still thousands of records.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

14. Gradient inversion

14.1. Why this matters

A phone keyboard learns from what its users type. The raw text never leaves the phones. Instead, a server sends the current model θ to each phone, the phone computes the gradient of the loss on its own few examples, and only that gradient goes back. The server averages the gradients of many phones and takes a step. This is the setting of **federated learning (FL)**, and it is often described as private because "the data stay on the device". Later parts study FL itself. This chapter asks one question about it: **what does a single client's gradient reveal about the client's data?**

The answer is: often, everything. A gradient is not a summary statistic chosen to hide the data. It is the derivative of a loss with respect to millions of parameters, and many of those derivatives are simple functions of the input. This chapter shows four ways to read the data back out of the gradient, going from the exact to the approximate. The first is **exact recovery**: for a fully connected layer with a bias, dividing one row of the weight gradient by one entry of the bias gradient gives back the layer's input, with no search at all. The second is **label recovery**: the signs of the last layer's gradient name the label, so the label can be taken off the list of unknowns. The third handles layers where no such closed form exists. You guess an input, compute the gradient it would produce, and move the guess until that gradient matches the one you saw. This is **Deep Leakage from Gradients (DLG)**. The fourth asks what happens when the client sends the average gradient of a whole batch of examples. Averaging mixes the examples together, but much less than you might hope.

14.1.1. The one idea: a gradient is a sum of outer products

Why should a gradient contain the input at all? Take a fully connected layer, which computes $a = Wx + b$: each output a_j is a weighted sum of the inputs x_k, with weights W_{jk}, plus a bias b_j. The weight W_{jk} multiplies the input x_k, so a small change in W_{jk} changes a_j by x_k times that change. The loss ℓ in turn responds to a change in a_j at some rate, $\delta_j = \partial \ell / \partial a_j$, which depends on everything that happens after this layer. The chain rule multiplies the two rates: the loss changes with W_{jk} at the rate $\delta_j x_k$. Collecting these numbers into a matrix with the shape of W, the gradient of the loss with respect to W is

$$\nabla_W \ell = \delta \, x^\top, \qquad \delta = \frac{\partial \ell}{\partial a}. \tag{14.1}$$

A matrix whose (j, k) entry is $\delta_j x_k$ is called the **outer product** of δ and x. Here it is the outer product of what flows backward (δ) with what flowed forward (x). Row j is the input x multiplied by the single number δ_j, so the layer's input sits, unchanged up to a scale, in every row of the gradient. Every attack in this chapter exploits that. With one example the scale can be

divided out. With a batch of B examples the gradient is a sum of B such outer products, and the attacker's job is to pull the sum apart.

Table 14.1 lists the results and where each is used.

Table 14.1.: The results of this chapter and where each is used.

Result	Where it is used
$\nabla_W \ell = \delta x^\top$, $\nabla_b \ell = \delta$ (Toolkit)	Every attack below; per-example gradients in later parts
Exact recovery $x = (\nabla_W \ell)_{j,:}/(\nabla_b \ell)_j$	Inverting any model whose first layer is fully connected with a bias
Labels from the sign of $\nabla_{b_2} \ell = p - e_y$; counts from its size	Removing the label from the unknowns; label leakage in batches
Deep Leakage from Gradients: $\min_{\hat{x}} \|\nabla_\theta \ell(\hat{x}) - g\|^2$	Inverting layers with no closed form, such as convolutions
Rank and isolation in a batch: $1 - (1 - 2^{-B})^m$	How much batching protects, and why it is not a defence

14.1.2. The setting and the game

The server is **honest-but-curious** in the sense of Threat models and privacy games: it runs the training protocol as specified, sends the real model and does not tamper with anything, and only analyses what it receives. Its access is the **transcript**, the strongest level of that chapter: it sees each client's own update, which is more than the final model, and it knows the architecture and every parameter of θ because it chose them. A client holds a batch of B labelled examples $(x_1, y_1), \dots, (x_B, y_B)$. Its training loss is the average of the B per-example losses, and what it sends is the gradient of that average with respect to all the parameters θ. Since the derivative of an average is the average of the derivatives, this is the average of the per-example gradients:

$$g = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x_i, y_i). \tag{14.2}$$

This is the **reconstruction game** of that chapter, played on one client. A game there has three parts: a challenger who draws a hidden secret, an adversary who sees only what the threat model allows, and a score for the adversary's guess. Here the challenger draws the batch from the population and the secret is the inputs. The adversary sees (θ, g) and outputs reconstructions $\hat{x}_1, \dots, \hat{x}_B$, and its gain is how close they come to the true inputs. We measure closeness by the **relative error** $\|\hat{x} - x\|/\|x\|$: the length of the mistake as a fraction of the length of the input, so that 0 is a perfect reconstruction and 1 is a mistake as large as the image itself. Each true example is matched to its closest reconstruction, because an average does not change when you reorder the examples, so the gradient carries no record of which example came first.

The baseline is a guess from the population alone, such as the average image of a shape, which knows nothing of the client's own position, noise or pixels. The attacks below either recover the client's exact pixels or, for large enough batches, produce blends; there is little in between.

The model. All experiments use a network with one hidden layer, written in NumPy. An input $x \in \mathbb{R}^d$ first goes through a fully connected layer (W_1, b_1) that gives one number a_j per hidden unit. An activation σ is applied to each unit, giving h. A second fully connected layer (W_2, b_2) turns h into one score z_k per class, called a **logit**. The **softmax** turns the logits into probabilities by exponentiating and normalizing, $p_k = e^{z_k} / \sum_{k'} e^{z_{k'}}$, so that every p_k is positive and they sum to 1. The loss is the **cross-entropy**, minus the log of the probability the model gives to the true label y, which is small when the model is confidently right. In one line:

$$a = W_1 x + b_1, \quad h = \sigma(a), \quad z = W_2 h + b_2, \quad p = \mathrm{softmax}(z), \quad \ell = -\ln p_y, \tag{14.3}$$

with m hidden units, K classes, $W_1 \in \mathbb{R}^{m \times d}$ and $W_2 \in \mathbb{R}^{K \times m}$. The activation σ is either the ReLU $\max(a, 0)$, which passes positive values and replaces negative ones by 0, or the sigmoid $1/(1 + e^{-a})$, which squashes every value smoothly into $(0, 1)$. The inputs are synthetic 8×8 grey-scale images ($d = 64$) of $K = 4$ shapes (a horizontal bar, a vertical bar, a diagonal and a square outline), shifted at random and with pixel noise. The network has $m = 32$ hidden units and is at its random initialization, as in the first round of training. Counting the $64 \cdot 32$ weights and 32 biases of the first layer and the $32 \cdot 4$ weights and 4 biases of the second, it has $64 \cdot 32 + 32 + 32 \cdot 4 + 4 = 2{,}212$ parameters, so the gradient has 2,212 entries, against 64 unknown pixels per example.

What this chapter assumes. Gradients, stochastic gradient descent (SGD), minibatches and the limits of non-convex optimization are in Optimization. The threat-model vocabulary and the games are in Threat models and privacy games. The facts this chapter needs from both are restated where they are used. For a different route to reconstruction, from noisy answers to many queries rather than from one gradient, see Reconstruction attacks.

Conventions. e_y is the y-th standard basis vector: 1 in position y and 0 elsewhere, the "one-hot" encoding of the label. $\odot$ is the elementwise product, which multiplies two vectors entry by entry. For a matrix M, $M_{j,:}$ is its j-th row. Gradients are column vectors and $\nabla_W \ell$ has the shape of W. We write $\delta_1 = \partial \ell / \partial a \in \mathbb{R}^m$ and $\delta_2 = \partial \ell / \partial z \in \mathbb{R}^K$ for the backward signals at the two layers: how fast the loss changes with each hidden unit's pre-activation and with each logit.

14.2. Toolkit: three facts every derivation uses

14.2.1. Fact 1 — The gradient of a fully connected layer is an outer product

This is the fact behind the whole chapter: in a fully connected layer, the gradient of each weight is the backward signal of its output times the input it multiplies. Let $a = Wx + b$ with $W \in \mathbb{R}^{m \times d}$, and let $\delta = \partial \ell / \partial a$ be the backward signal. Then

$$\nabla_W \ell = \delta \, x^\top, \qquad \nabla_b \ell = \delta. \tag{14.4}$$

Why. Write out one output: $a_j = \sum_k W_{jk} x_k + b_j$. The weight W_{jk} appears in a_j only, with coefficient x_k, so $\partial a_j / \partial W_{jk} = x_k$. The loss depends on W_{jk} only through a_j, so the chain rule gives $\frac{\partial \ell}{\partial W_{jk}} = \frac{\partial \ell}{\partial a_j} \frac{\partial a_j}{\partial W_{jk}} = \delta_j x_k$, which is the (j, k) entry of $\delta x^\top$. The bias b_j enters a_j with coefficient 1, so $\frac{\partial \ell}{\partial b_j} = \delta_j \cdot 1$.

A concrete case. With $m = 2$, $d = 3$, $\delta = (-0.3279, 0.9838)$ and $x = (0.5, 1, 0.2)$, the two rows of $\nabla_W \ell$ are $-0.3279\,x = (-0.1640, -0.3279, -0.0656)$ and $0.9838\,x = (0.4919, 0.9838, 0.1968)$. Both are multiples of x. This is the worked example of Table 14.2.

Every attack in this chapter uses this fact. The point to keep is the split between the two factors: δ depends on everything above the layer (the rest of the network, the loss, the label), but the factor $x^\top$ does not. Whatever the rest of the network does, it can only rescale the rows, never change their direction.

14.2.2. Fact 2 — Softmax with cross-entropy has gradient $p - e_y$

At the last layer the backward signal has a very simple form: the predicted probabilities minus the one-hot label. In words, raising the logit of a wrong class k raises the loss at the rate p_k, the probability the model already gives that class, and raising the logit of the true class lowers the loss at the rate $1 - p_y$. With $p = \mathrm{softmax}(z)$ and $\ell = -\ln p_y$,

$$\delta_2 = \frac{\partial \ell}{\partial z} = p - e_y. \tag{14.5}$$

Why. Write the loss without the fraction. Since $p_y = e^{z_y} / \sum_k e^{z_k}$, its log is $z_y - \ln \sum_k e^{z_k}$, so $\ell = -z_y + \ln \sum_k e^{z_k}$. Now differentiate in one logit z_k. The first term depends only on z_y, so it contributes -1 when $k = y$ and 0 otherwise, which is $-\mathbf{1}[k = y]$. The second term is the log of a sum, and the derivative of $\ln u$ is u'/u: here $u' = e^{z_k}$, so the term contributes $e^{z_k} / \sum_{k'} e^{z_{k'}} = p_k$. Together, $\partial \ell / \partial z_k = p_k - \mathbf{1}[k = y]$, which is entry k of $p - e_y$.

A concrete case. With logits $z = (-0.35, 0.295)$ and $y = 1$ (the first class), the first probability is $p_1 = e^{-0.35}/(e^{-0.35} + e^{0.295}) = 1/(1 + e^{0.645}) = 1/(1 + 1.906) = 0.3441$, so $p = (0.3441, 0.6559)$ and $\delta_2 = (0.3441 - 1, 0.6559) = (-0.6559, 0.6559)$. The entries sum to 0, and only the true class is negative.

This sign pattern is what label recovery uses. The softmax never gives a probability of exactly 0 or 1, since every e^{z_k} is positive, so $0 < p_y < 1$. The entry at the true label is therefore $p_y - 1 < 0$, and every other entry is $p_k > 0$.

14.2.3. Fact 3 — Backpropagation and averaging

To use Fact 1 at the first layer you need its backward signal δ_1, and that is obtained by sending δ_2 back through the network, which is what **backpropagation** does. Each hidden unit j feeds every logit z_k through the weight $W_{2,kj}$, so it collects the logits' signals weighted by those connections, $\sum_k \delta_{2,k} W_{2,kj}$. That sum is how fast the loss changes with the unit's output h_j. The unit's output is $h_j = \sigma(a_j)$, so to get the rate with respect to a_j you multiply by the activation's slope $\sigma'(a_j)$. In vector form:

$$\delta_1 = (W_2^\top \delta_2) \odot \sigma'(a). \tag{14.6}$$

The second half of the fact concerns batches: the gradient of an average loss over a batch is the average of the per-example gradients.

Why. $z_k = \sum_j W_{2,kj} h_j + b_{2,k}$, so h_j enters each z_k with coefficient $W_{2,kj}$, and the chain rule sums over the logits: $\frac{\partial \ell}{\partial h_j} = \sum_k \delta_{2,k} W_{2,kj} = (W_2^\top \delta_2)_j$. Then $h_j = \sigma(a_j)$ multiplies this by $\sigma'(a_j)$. The

claim about batches holds because differentiation is linear: the derivative of a sum is the sum of the derivatives, and a constant factor $\frac{1}{B}$ comes out unchanged.

A concrete case. For ReLU the slope is $\sigma'(a_j) = \mathbf{1}[a_j > 0]$: 1 if the unit is switched on, 0 if it is switched off. So a unit that is switched off on an input has $\delta_{1,j} = 0$ and, by Fact 1, a zero row in $\nabla_{W_1}\ell$. For a batch, Fact 1 applied to each example and then averaged gives the first-layer gradient $\frac{1}{B}\sum_i \delta_{1,i}x_i^\top$: a sum of B outer products, one per example.

Label recovery uses this fact to know which rows of the weight gradient can be negative, and every batch result below uses the averaging.

14.3. Exact recovery from a fully connected layer

14.3.1. Context

The server sees g. Is there any computation, short of a search, that gives back the client's input? For the most common first layer of all, a fully connected layer with a bias, there is: one division. The observation goes back at least to Phong et al. (2018), who pointed it out for exactly this federated setting.

14.3.2. Statement

Suppose the client sends the gradient of a single example, and the first layer is fully connected, $a = W_1x + b_1$. By Fact 1, row j of the weight gradient is the input x scaled by the number $\delta_{1,j}$, and entry j of the bias gradient is that same number $\delta_{1,j}$. Dividing the first by the second removes the scale. So for every unit j with $(\nabla_{b_1}\ell)_j \neq 0$,

$$x = \frac{(\nabla_{W_1}\ell)_{j,:}}{(\nabla_{b_1}\ell)_j}. \tag{14.7}$$

Moreover, since every row is a multiple of the same vector, $\nabla_{W_1}\ell$ has **rank one** (all its rows lie on a single line), and that line, its row space, is the line through x.

14.3.3. Derivation

1. **Apply Fact 1 (Equation 14.4) to the first layer.** It gives the two pieces the server sees: $(\nabla_{W_1}\ell)_{j,:} = \delta_{1,j}\,x^\top$ and $(\nabla_{b_1}\ell)_j = \delta_{1,j}$.
2. **Divide one by the other.** When $\delta_{1,j} \neq 0$, the ratio is $\delta_{1,j}x^\top/\delta_{1,j} = x^\top$. Nothing about the rest of the network enters: the depth, the activations above, the loss and the label all sit inside the number $\delta_{1,j}$, and that number cancels.
3. **Read off the rank.** All rows of $\nabla_{W_1}\ell = \delta_1 x^\top$ are multiples of the same vector $x^\top$. A matrix whose rows are all multiples of one nonzero vector has rank one, so (as long as $\delta_1 \neq 0$) the rank is one and the row space is spanned by x.

14.3.4. Interpretation

The attack is exact and costs nothing. It needs no optimization, no prior knowledge of what the data look like, and no knowledge of the label. It also checks itself: every unit with a nonzero bias gradient gives the same answer, so if all the usable ratios agree, the attacker knows it has the input.

The same division works at any fully connected layer whose input you want. Applied deeper in the network, it gives that layer's input, an intermediate feature vector. Applied at the first layer, the input is the data itself.

Three things can stop it, and only one of them stops it fully. With ReLU, units that are switched off on x have $\delta_{1,j} = 0$ and give nothing, but a single active unit is enough. Without a bias there is no denominator, but the rank-one structure still gives the direction of x, so x is known up to one scalar, and a one-dimensional search finds that scalar (Exercise 2 in the exercise book). A convolutional first layer is what really breaks the outer-product form. A convolution applies the same small filter at every position of the image, so its gradient is a sum of outer products over all the image patches rather than a single one, and no division separates them. That is why images fed to convolutional networks need the search of Section 14.5. A batch of more than one example also breaks the formula, as Section 14.6 shows, but only partly.

Figure 14.1 shows the attack on one image.

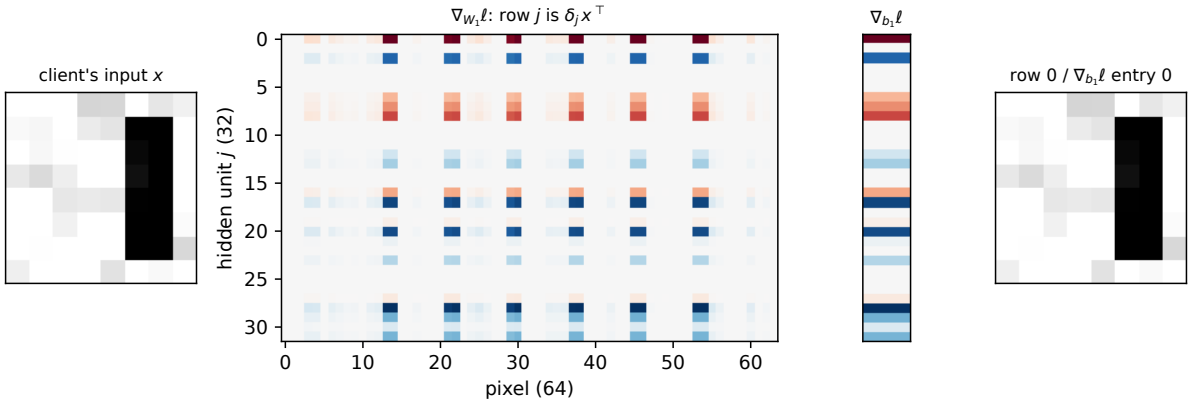


Figure 14.1.: Exact recovery from the first-layer gradient of a ReLU network with 32 hidden units. Left: the client's 8×8 input. Middle: $\nabla_{W_1}\ell$, one row per hidden unit and one column per pixel (blue positive, red negative), next to $\nabla_{b_1}\ell$. Every nonzero row is the image times the matching bias entry; switched-off units give zero rows. Right: one row divided by its bias entry, which is the input exactly.

14.3.5. Example: a network you can compute by hand

Take $d = 3$ inputs, $m = 2$ ReLU units and $K = 2$ classes, with

$$W_1 = \begin{pmatrix} 0.4 & -0.2 & 0.1 \\ 0.3 & 0.5 & -0.4 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0.1 \\ -0.1 \end{pmatrix}, \quad W_2 = \begin{pmatrix} 1 & -1 \\ 0.5 & 0.5 \end{pmatrix}, \quad b_2 = 0, \tag{14.8}$$

and the example $x = (0.5, 1, 0.2)$ with label $y = 1$ (the first class). Table 14.2 walks through the forward pass, the backward pass and the attack.

Table 14.2.: Forward pass, backward pass and exact recovery for a two-unit ReLU network. The server sees only the "Gradient" rows.

Step	Quantity	Value
Forward	$a = W_1 x + b_1$	$(0.2 - 0.2 + 0.02 + 0.1,\ 0.15 + 0.5 - 0.08 - 0.1) = (0.12, 0.47)$
	$h = \max(a, 0)$	$(0.12, 0.47)$, both units on
	$z = W_2 h$	$(0.12 - 0.47,\ 0.06 + 0.235) = (-0.35, 0.295)$
	$p = \mathrm{softmax}(z)$	$(0.3441, 0.6559)$
Backward	$\delta_2 = p - e_y$ (Fact 2)	$(-0.6559, 0.6559)$
	$\delta_1 = (W_2^\top \delta_2) \odot \sigma'(a)$ (Fact 3)	$(-0.6559 + 0.5 \cdot 0.6559,\ 0.6559 + 0.5 \cdot 0.6559) = (-0.3279, 0.9838)$
Gradient	$\nabla_{W_1} \ell = \delta_1 x^\top$ (Fact 1)	rows $(-0.1640, -0.3279, -0.0656)$ and $(0.4919, 0.9838, 0.1968)$
	$\nabla_{b_1} \ell = \delta_1$	$(-0.3279, 0.9838)$
Attack	row 1 / (-0.3279), row 2 / 0.9838	$(0.5, 1, 0.2)$ and $(0.5, 1, 0.2)$

The server never sees a, h, z or p. From the two gradient rows alone it divides and reads off x, twice. Both units are on here ($a = (0.12, 0.47)$ is positive in both entries), so the ReLU slope is 1 for both and δ_1 is just $W_2^\top \delta_2$.

On the images. For one 8×8 image and the 32-unit ReLU network, 13 of the 32 units are switched on and have a nonzero bias gradient. Every one of the 13 ratios equals the image to a relative error of at most 6.7×10^{-17}, which is rounding error. To check the rank, look at the **singular values** of $\nabla_{W_1} \ell$: they measure how much of the matrix lies along each of its independent directions, so a rank-one matrix has exactly one that is not zero. Here they are 1.6155 and then nothing: the second is 9.5×10^{-17} times the first. The gradient is rank one, as the statement says.

14.3.6. Check yourself

- The network has 20 layers above the first one and a custom loss. Does Equation 14.7 still hold? *(Yes. Everything above the first layer only changes δ_1, and $\delta_{1,j}$ cancels in the ratio. All that matters is that the first layer is fully connected with a bias.)*
- For a ReLU network, which units are useless to the attacker, and could all of them be useless? *(Units switched off on x have $\delta_{1,j} = 0$. All units are useless only if every unit is off or every active unit has $(W_2^\top \delta_2)_j = 0$, which for a random network essentially never happens; the image above had 13 usable units out of 32.)*

14.4. Label recovery from the last layer

14.4.1. Context

The gradient depends on the label too, and the label is often the most sensitive part: the diagnosis, the word typed next, the class of a photo. Zhao, Mopuri & Bilen (2020) noticed that the label can be read off the last layer in closed form. Their attack, iDLG ("improved DLG"), then searches only for the input, which makes the search of Section 14.5 easier and more reliable.

14.4.2. Statement

There are three versions of the rule, depending on what the last layer has and how many examples the gradient covers. All three come from Fact 2: the last layer's backward signal is $p - e_y$, a vector of probabilities with 1 subtracted at the true label.

From the bias, for one example. By Fact 1 the last layer's bias gradient equals its backward signal, $\nabla_{b_2}\ell = p - e_y$. Every entry is a positive probability except the one at the true label, where subtracting 1 makes it negative. So the label is the position of the smallest entry, the only negative one:

$$y = \arg\min_k (\nabla_{b_2}\ell)_k. \tag{14.9}$$

From the weights, for one example. The weight gradient tells the same story. By Fact 1, row k of $\nabla_{W_2}\ell$ is $\delta_{2,k}$ times the hidden activations h. If those activations are never negative ($h \geq 0$ and $h \neq 0$, as for ReLU or the sigmoid), multiplying by them keeps the sign of $\delta_{2,k}$. So y is the only row of $\nabla_{W_2}\ell$ whose entries are non-positive, and in particular the only row with a negative sum.

Counts, for a batch. With B examples the bias gradient is the average of $p_i - e_{y_i}$ over the batch. The e_{y_i} parts add up to the label histogram: their k-th entries sum to n_k, the number of examples of class k. The p_i parts add up to the model's total predicted probability for class k, $\sum_i p_{ik}$. So the histogram is that total minus B times the bias gradient:

$$n_k = \sum_{i=1}^{B} p_{ik} - B\,(\nabla_{b_2}\ell)_k. \tag{14.10}$$

An attacker who does not know $\sum_i p_{ik}$ replaces it by $B\bar{p}_k$, where $\bar{p}$ is a guess of the model's average prediction, and rounds to the nearest integer.

14.4.3. Derivation

1. **Bias.** Fact 1 at the last layer gives $\nabla_{b_2}\ell = \delta_2$, and Fact 2 (Equation 14.5) gives $\delta_2 = p - e_y$. Softmax probabilities lie strictly between 0 and 1, so $(\nabla_{b_2}\ell)_y = p_y - 1 < 0$ and $(\nabla_{b_2}\ell)_k = p_k > 0$ for $k \neq y$. The smallest entry is therefore at y.
2. **Weights.** Fact 1 gives $(\nabla_{W_2}\ell)_{k,:} = \delta_{2,k}\,h^\top$. With $h \geq 0$, row y is $(p_y - 1)h^\top$, a negative number times non-negative entries, so it is ≤ 0 entrywise. Every other row is $p_k h^\top \geq 0$. Summing a row multiplies $\delta_{2,k}$ by $\sum_j h_j$, which is positive because $h \geq 0$ and $h \neq 0$, so the sum keeps the sign of $\delta_{2,k}$.

3. **Counts.** By Fact 3 the batch gradient is the average of the per-example ones, so $\nabla_{b_2}\ell = \frac{1}{B}\sum_i (p_i - e_{y_i})$. Its k-th entry is $\frac{1}{B}(\sum_i p_{ik} - n_k)$, because e_{y_i} has a 1 in position k exactly when example i has class k, so $\sum_i e_{y_i}$ counts the examples of each class. Multiply by B and solve for n_k.

14.4.4. Interpretation

For one example the rule makes no errors. It is exact for every network that ends in softmax and cross-entropy with a bias, at any point of training, because the sign argument never depends on how good the model is. On 1,000 random images and random networks, both the bias rule and the weight rule name the label correctly 1.000 of the time.

For a batch, the gradient leaks the label histogram. Equation 14.10 is exact, so the only uncertainty is the total prediction $\sum_i p_{ik}$, and the attacker can estimate it well: it knows the model, so it can run the model on data of its own from the same population and average the predictions. The true counts are integers, and a number within $\frac{1}{2}$ of an integer rounds to that integer. So rounding gives the exact counts whenever the estimate of $\sum_i p_{ik}$ is off by less than $\frac{1}{2}$ for every class (Exercise 4 in the exercise book). The cruder guess $\bar{p}_k = 1/K$, which assumes the model spreads its predictions evenly over the classes, does worse, because even an untrained network is not uniform.

The label matters for inversion as well. Knowing y removes a discrete unknown from the search. Zhu et al. (2019) optimized a soft label (a probability vector standing in for the unknown label) jointly with the input, and that joint search often failed. With the label fixed by Equation 14.9, the search is over the input only.

Figure 14.2 shows both cases.

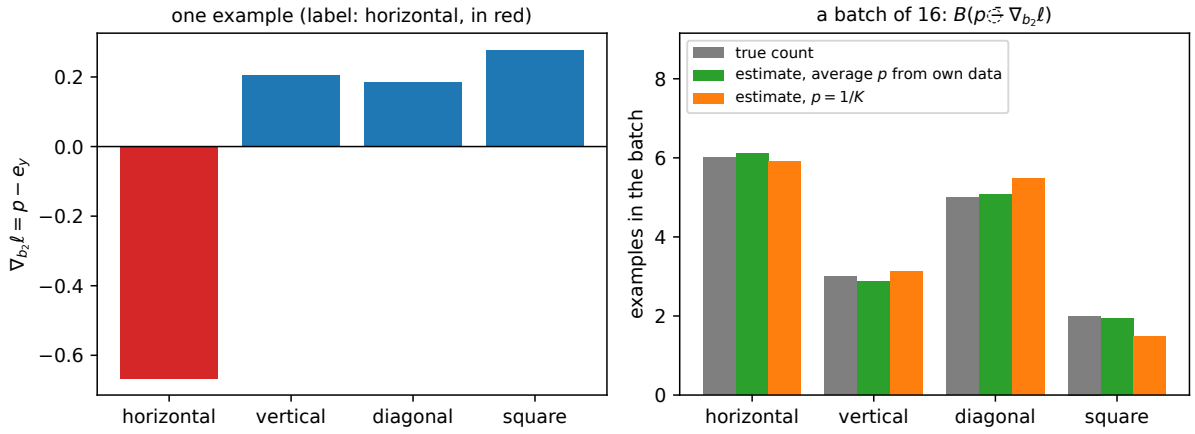


Figure 14.2.: Labels from the last-layer bias gradient. Left: for one example, $\nabla_{b_2}\ell = p - e_y$ has a single negative entry, at the true label (red). Right: for a batch of 16, the counts $B(\bar{p} - \nabla_{b_2}\ell)$ with $\bar{p}$ the model's average prediction on the attacker's own images (green) are close to the true counts (grey); with $\bar{p} = 1/K$ (orange) they are off by up to about half an example.

14.4.5. Example: a batch of 16

A batch of 16 images has true counts $(4, 2, 4, 6)$ over the four shapes. Table 14.3 reads them off the bias gradient of the 32-unit ReLU network, with three different guesses for the total prediction $\sum_i p_{ik}$ in Equation 14.10.

Table 14.3.: Label counts of one batch of 16 recovered from $\nabla_{b_2}\ell$ by Equation 14.10, under three guesses of the summed predictions.

Guess of $\sum_i p_{ik}$	Estimated counts	Rounded	All correct?
Exact, $\sum_i p_{ik}$ (not available to the attacker)	$(4.00, 2.00, 4.00, 6.00)$	$(4, 2, 4, 6)$	yes
$B\bar{p}$, $\bar{p}$ averaged over 200 of the attacker's own images	$(3.88, 2.00, 4.19, 5.92)$	$(4, 2, 4, 6)$	yes
$B/K = 4$ for every class	$(2.12, 3.01, 4.27, 6.60)$	$(2, 3, 4, 7)$	no

With the averaged guess every estimate is within $\frac{1}{2}$ of the truth (the largest miss is $4.19 - 4 = 0.19$), so rounding fixes it. With the uniform guess the first class is off by $4 - 2.12 = 1.88$, and rounding cannot help.

Over 200 random batches and networks, the rounded counts are all correct in 1.000 of batches of 4, 0.975 of batches of 16 and 0.525 of batches of 64 with the averaged guess. With the uniform guess the rates are 0.925, 0.095 and 0.000. The reason for the decline is that the error of the estimate is a sum of B deviations, one per example, each the gap between that example's prediction and the guess $\bar{p}$. These do not cancel exactly, so the error of their sum grows with B, and so does the chance that some class is off by more than $\frac{1}{2}$. Even when the counts are not all exact, they are close: Exercise 4 in the exercise book finds that at $B = 16$ the largest error over the four classes, before rounding, has median 0.233 with the averaged guess and 1.102 with the uniform one.

14.4.6. Check yourself

- Why must $p_y - 1$ be negative, even for a model that is confidently right? *(Softmax never outputs exactly 1, so $p_y < 1$. A confident model gives a tiny negative entry, but its sign is still the only negative one. In floating point a p_y that rounds to 1 would hide it, which is one reason attacks are run on early-training models.)*
- The last layer has no bias and the hidden layer uses tanh. Which rule still works? *(Neither as stated: there is no bias gradient, and tanh activations can be negative, so row y need not have a negative sum. But row y still has the opposite sign to all other rows, so for $K \geq 3$ the odd one out is the label; see Exercise 3 in the exercise book.)*

14.5. Deep Leakage from Gradients

14.5.1. Context

The closed forms need a fully connected layer with a bias. Most image models start with convolutions, and many architectures have normalization layers, attention, or no biases at all. Zhu, Liu & Han (2019) gave a generic attack that needs none of these: **guess an input, compute its gradient, and move the guess until the gradients match**. If only the true input produces the observed gradient, a matching guess is the input.

14.5.2. Statement

To turn "move the guess until the gradients match" into something you can optimize, you need one number that says how badly a guess matches. DLG uses the squared distance between the gradient the guessed batch would produce and the observed gradient g, summed over all the gradient's entries. Call it the **matching loss** R. It is zero for a perfect match and positive otherwise. Given θ, the observed gradient g and the labels $y_1, \dots, y_B$ (from Section 14.4), DLG looks for the guesses $x'_1, \dots, x'_B$ that make R smallest:

$$\hat{x}_1, \dots, \hat{x}_B = \arg\min_{x'_1,\dots,x'_B} R(x'), \qquad R(x') = \Big\| \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \ell(\theta; x'_i, y_i) - g \Big\|^2, \tag{14.11}$$

where arg min means "the inputs at which the minimum is reached". It searches by a gradient method: start from random noise and repeatedly move the guess a little downhill on R. The true batch has $R = 0$, the smallest possible value.

14.5.3. Derivation

1. **The true batch is a global minimizer of Equation 14.11.** R is a squared length, so $R \geq 0$ always. Plugging in the true x_i reproduces g exactly, so $R = 0$ there, and nothing can do better.
2. **The objective can be differentiated in x' if the network is twice differentiable.** A gradient method needs the slope of R with respect to the guess. Write $G(x') = \frac{1}{B} \sum_i \nabla_\theta \ell(\theta; x'_i, y_i)$, the function that takes a guessed batch to the gradient it would produce. Then $R = \|G(x') - g\|^2$. The squared length $\|u\|^2$ changes at the rate $2u$, and the chain rule passes this back to x' through the **Jacobian** J_G of G, the matrix of rates at which each gradient entry changes with each input pixel:

$$\nabla_{x'} R = 2\, J_G(x')^\top (G(x') - g). \tag{14.12}$$

   G is already made of first derivatives of the network, so J_G is made of second derivatives. That is why Zhu et al. replaced ReLU by the sigmoid, and the experiments here do the same. The ReLU's slope is 0 or 1, constant on each side of zero, so its second derivative is zero almost everywhere: the factor $\sigma'(a)$ in Equation 14.6 is piecewise constant in a, and the search gets no signal through it. The sigmoid's slope varies smoothly, so it does.

3. **Computing $J_G^\top v$ without forming J_G.** J_G would have one row per parameter and one column per pixel, and no one builds it. Automatic-differentiation libraries instead run backpropagation through the backward pass itself, which is called **double backpropagation**. The script writes it out by hand for the one-hidden-layer network. The forward pass computes a, h, z, p. The backward pass computes δ_2, δ_1 and the four gradient blocks. A second backward pass then sends $2(G - g)$ through every one of those steps back to x'. To check it, the script also estimates the slope by finite differences (nudge one pixel slightly and see how much R changes), and the two agree to within 1.2×10^{-10}.
4. **Optimization finds a stationary point, not necessarily the minimizer.** R is not convex in x': its surface can have dips and flat regions other than the true minimum. As the non-convex section of Optimization shows, for such a function a gradient method with a suitable step only guarantees that the slope $\|\nabla R\|$ goes to zero. That gives a **stationary point**, where the surface is flat, which may or may not be the global minimum. So success is an empirical question. The value of R at the end tells you which case you are in: $R = 0$ (up to rounding) means you found an exact match, while a value stuck well above zero means you got stuck.
5. **Uniqueness is plausible when equations outnumber unknowns.** Matching g imposes one equation per parameter: 2,212 here, against $64B$ unknown pixels. That is not a proof that the minimizer is unique, but with a single example the closed form of Section 14.3 shows that it is: the input is determined by the gradient.

14.5.4. Interpretation

DLG is a general recipe. All it needs is the ability to differentiate the gradient with respect to the input, which any automatic-differentiation library provides. So it works for convolutional networks and on real images, where no closed form exists: Zhu et al. recover CIFAR-100 pictures and text tokens from single-example gradients.

When it fails, it fails first as an optimization, not for lack of information. If the run ends with R far from zero, the attacker knows it has not found the input, and better optimizers, restarts from new random guesses, and image priors help. Geiping et al. (2020) made two changes. They replaced the squared distance by a cosine distance between gradients, which compares only the directions of the two gradients and ignores their lengths. And they added a total-variation penalty, the sum of differences between neighbouring pixels, which is small for natural images made of smooth regions. (This penalty is a measure of image roughness, not the total variation distance between distributions.) With these they recover recognizable images from trained networks and from batches.

Note what the attack does not need. The label search is gone, thanks to iDLG. And the server need not be malicious: it only computes on what it legitimately received.

Figure 14.3 shows one run.

14.5.5. Example: one image

The script starts from uniform noise and runs 600 steps of the Adam optimizer, a variant of gradient descent that scales each pixel's step by a running estimate of the size of that pixel's gradient, with step size 0.05 decaying linearly to 0. The label comes from Equation 14.9.

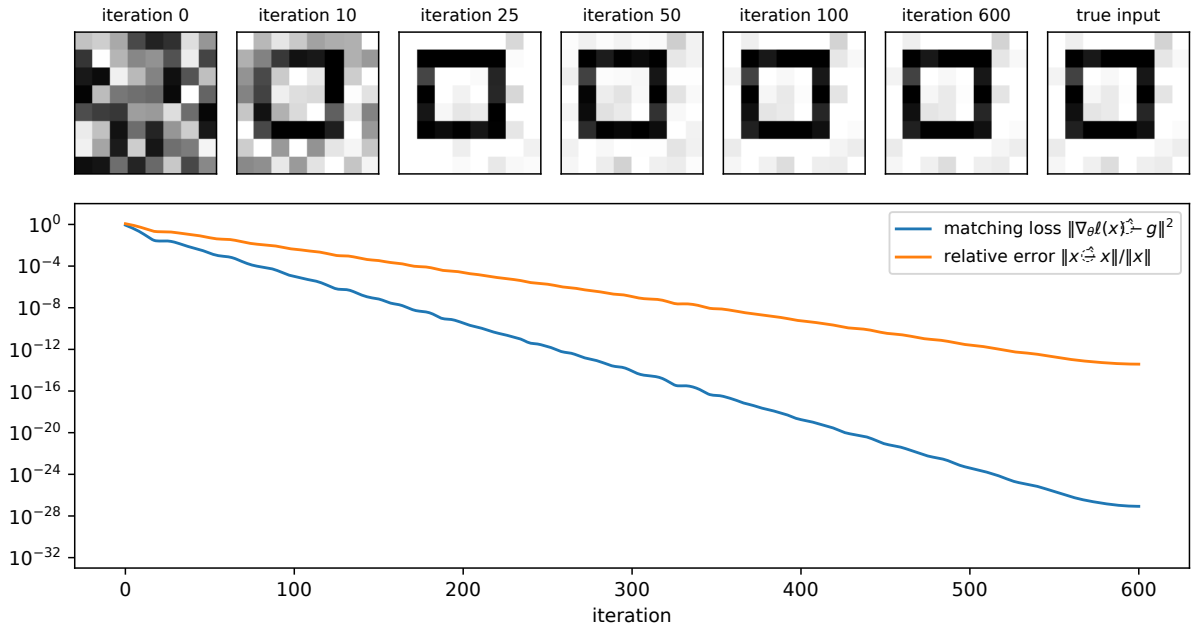


Figure 14.3.: Deep Leakage from Gradients on one 8×8 image, sigmoid network with 32 hidden units. Top: the reconstruction $\hat{x}$ at selected iterations of Adam, starting from uniform noise, and the true input. Bottom: the matching loss $R(\hat{x})$ and the relative error, on a log scale. Both fall geometrically until they reach rounding error.

Table 14.4.: Deep Leakage from Gradients on one image, sigmoid network with 32 hidden units: relative error of the reconstruction and the matching loss over the iterations.

Iteration	0	10	50	100	200	400	600
Relative error	1.6	0.67	7.5×10^{-2}	5.7×10^{-3}	3.4×10^{-5}	7.8×10^{-10}	5.8×10^{-14}
Matching loss R	1.3	0.20	2.5×10^{-3}	1.3×10^{-5}	4.6×10^{-10}	2.5×10^{-19}	1.3×10^{-27}

By iteration 50 the image is visually correct (7.5% error); by iteration 600 it is exact to about 13 digits. The matching loss falls about as the square of the error, and that is what you should expect close to the answer. Near the true input x, the gradient map G is nearly linear, so the mismatch is about the Jacobian times the error, $G(x') - g \approx J_G(x)(x' - x)$, and its squared length is quadratic in the error: $R \approx \|J_G(x)(x' - x)\|^2$. In the table, between iterations 50 and 100 the error shrinks by a factor $0.075/0.0057 \approx 13$ and R by $0.0025/0.000013 \approx 190$, close to $13^2 \approx 170$.

14.5.6. Check yourself

- The run ends with $R = 10^{-5}$ and a reconstruction that looks like a shape. Has the attack succeeded? *(Not certainly. At the true input $R = 0$, and in Table 14.4 $R = 1.3 \times 10^{-5}$ goes*

with an error below 1%, but a different input can also have small R. A loss that has stopped falling well above rounding error signals a stationary point that is not the input; restart from a new random guess.)

- Why does the attacker want the label before running DLG? *(With the label fixed, R depends on the input only. Searching jointly over a soft label adds a variable that interacts with every gradient entry through $p - e_y$, and Zhao et al. found that the joint search fails much more often.)*

14.6. Batches and averaging

14.6.1. Context

Clients rarely send the gradient of one example. They send the average over a batch, or the change in parameters after several local steps. Averaging is often said to protect the individual examples: the gradient now describes the batch, not anyone in it. This section measures that claim. Averaging does destroy the one-line formula of Equation 14.7, and it does make DLG harder, but three things survive: the span of the batch, every example that some unit "sees alone", and a search that still works for small batches.

14.6.2. Statement

Take a batch of B examples and a fully connected first layer, and write $\delta_{1,i}$ for the backward signal of example i at that layer. Each claim below follows from the gradient being a sum of one outer product per example.

The gradient is a sum of B outer products, and its rows lie in the span of the batch. By Facts 1 and 3, averaging adds up the per-example outer products $\delta_{1,i} x_i^\top$. Each of those has rows that are multiples of its own x_i, so every row of the sum is a weighted combination of the B inputs. It lies in their **span**, the set of all such combinations. And adding B rank-one matrices gives rank at most B:

$$\nabla_{W_1} \ell = \frac{1}{B} \sum_{i=1}^{B} \delta_{1,i}\, x_i^\top, \qquad \operatorname{rank} \nabla_{W_1} \ell \le B, \qquad \text{every row} \in \operatorname{span}\{x_1, \dots, x_B\}. \tag{14.13}$$

Unit ratios are weighted averages. With one example, dividing row j by the bias entry cancelled a single scale. With a batch, row j is a sum of different inputs with different scales $\delta_{1,ij}$, and the bias entry is the sum of those scales. The division gives each input a weight equal to its share of the total, so the result is a weighted average of the inputs. For a unit j with $(\nabla_{b_1} \ell)_j \neq 0$,

$$\frac{(\nabla_{W_1} \ell)_{j,:}}{(\nabla_{b_1} \ell)_j} = \sum_{i=1}^{B} \frac{\delta_{1,ij}}{\sum_{i'} \delta_{1,i'j}}\, x_i^\top, \tag{14.14}$$

which equals x_i exactly when only example i contributes, that is when $\delta_{1,i'j} = 0$ for every $i' \neq i$. For ReLU this happens when unit j is switched on for example i only: unit j **isolates** example i.

How often an example is isolated. A unit isolates example i when it is on for i and off for every other example. Suppose each of the m ReLU units switches on for each example independently with probability $\frac{1}{2}$, like a fair coin. Then one unit isolates example i with probability 2^{-B}, the chance of one particular pattern of B coin flips. The example is missed only if all m units miss it, which has probability $(1-2^{-B})^m$. So each example of the batch is recovered exactly by some unit ratio with probability

$$P_{\text{iso}}(B, m) = 1 - (1 - 2^{-B})^m. \tag{14.15}$$

14.6.3. Derivation

1. **Sum of outer products.** By Fact 3 the batch gradient is the average of the per-example gradients, and by Fact 1 each of those is $\delta_{1,i} x_i^\top$.
2. **Rank and span.** Stack the $x_i^\top$ as the rows of a matrix $X \in \mathbb{R}^{B \times d}$ and the $\delta_{1,i}^\top$ as the rows of $\Delta \in \mathbb{R}^{B \times m}$. Then the sum of outer products is a matrix product, $\nabla_{W_1} \ell = \frac{1}{B} \Delta^\top X$. A product has rank at most that of either factor, and X has only B rows, so the rank is at most B. Row j is $\frac{1}{B} \sum_i \Delta_{ij} x_i^\top$, a combination of the x_i, so it lies in their span.
3. **Ratios.** By step 1 and Fact 1, the bias gradient is $(\nabla_{b_1} \ell)_j = \frac{1}{B} \sum_i \delta_{1,ij}$. Divide the j-th row by it. The factors $\frac{1}{B}$ cancel, and the weights $\delta_{1,ij} / \sum_{i'} \delta_{1,i'j}$ sum to 1 because they are each example's share of the same total. If only $\delta_{1,ij}$ is nonzero, the weight on x_i is 1 and all the others are 0.
4. **One unit.** For ReLU, $\delta_{1,i'j} = 0$ whenever unit j is off on $x_{i'}$, because the ReLU slope is 0 there (Fact 3). So unit j isolates example i if it is on for i (probability $\frac{1}{2}$) and off for each of the other $B-1$ (probability $\frac{1}{2}$ each, independently). Multiplying the B independent probabilities gives 2^{-B}. (An active unit also needs $(W_2^\top \delta_{2,i})_j \neq 0$, which holds with probability 1 for random weights.)
5. **Many units.** The units are independent by assumption, so the probability that none of the m units isolates example i is the product of m factors $1 - 2^{-B}$, which is $(1-2^{-B})^m$. Subtracting from 1 gives Equation 14.15.
6. **When the independence assumption holds.** Suppose the first-layer bias is zero and the inputs are drawn from a distribution that is symmetric about zero, such as $\mathcal{N}(0, I)$. A unit with weight row w_j is on for x_i when $w_j^\top x_i > 0$. Symmetry makes $w_j^\top x_i$ equally likely to be positive or negative, and independent inputs make these signs independent across the batch. So each unit's on/off pattern over the batch is exactly a sequence of fair coin flips, and step 4 is exact. Different units look at the same inputs, though, so their patterns are not fully independent of each other, and step 5 is only approximate. Real data are not symmetric: images of the same kind are positively correlated, and Table 14.5 shows that this changes the answer substantially.

14.6.4. Interpretation

The formula is exponential in B and linear in m. A unit isolates a given example with probability 2^{-B}, so among m units the expected number that isolate it is $m2^{-B}$. With $m \gg 2^B$ almost every example is isolated; with $m \ll 2^B$ almost none is. To find the width that gives an even chance, set the miss probability $(1-2^{-B})^m$ to at most $\frac{1}{2}$ and take logs: $m \ln(1-2^{-B}) \leq -\ln 2$, so $m \geq \ln 2 / (-\ln(1-2^{-B}))$. At $B = 10$ this is $\ln 2 / (-\ln(1-2^{-10})) = 0.6931 / 0.000977 \approx 709.4$,

so you need $m \geq 710$ units. Real first layers have thousands of units, so batches of 10 are not safe from this attack.

Correlated data are easier for the attacker, not harder. With images, a unit tends to fire on many examples of a batch or on few: the number of examples it fires for is more spread out than the coin-flip model predicts (statisticians call this **overdispersion**), instead of concentrating around $B/2$. Units that fire on just one example are then much more common at large B, and isolation decays much more slowly than 2^{-B}. Exercise 5 in the exercise book works out one extreme model, in which each unit has its own firing rate q, drawn uniformly between 0 and 1, and fires on each example with that probability. Averaging the isolation probability $q(1-q)^{B-1}$ over q gives $1/(B(B+1))$ per unit, which shrinks only like $1/B^2$ instead of halving with each extra example.

The span leaks even when nothing is isolated. By Equation 14.13, the server learns the B-dimensional subspace that contains the batch. For $B \ll d$ that already rules out almost every input, since every example must be a combination of B known directions.

A malicious server does better still. The analysis above assumes random weights, as an honest server would send. A server that chooses θ can plant first-layer units that fire on rare inputs only ("trap weights", Boenisch et al. 2023), or that sort inputs into bins by a feature such as brightness (Fowl et al. 2022), so that most examples of even a large batch are isolated by construction.

DLG on batches has a harder search to do: it must untangle B inputs from one sum. In the experiments below it always recovers single examples, usually recovers pairs, and mostly fails beyond that, with the matching loss stuck well above zero. That is an optimization failure, not a shortage of information: at $B = 4$ there are $64 \cdot 4 = 256$ unknown pixels against 2,212 equations.

Finally, several local steps do not help either. If the client takes several SGD steps on its batch and sends the total change in parameters, every step's first-layer gradient is made of outer products with the same inputs x_i, and only the backward signals change from step to step. So for one example the change in W_1 is still a multiple of $x^\top$ in every row, still rank one, and Equation 14.7 still recovers x exactly (Exercise 6 in the exercise book).

14.6.5. Example: what a batch hides

Rank and isolation in one batch. A batch of 4 images through the 32-unit ReLU network gives a first-layer gradient whose singular values (the sizes of the matrix along its independent directions), relative to the largest, are 1.0, 0.84, 0.69, 0.039 and then 1.5×10^{-16}. Four directions are nonzero and the rest are rounding error: rank 4, as Equation 14.13 says. Every row lies in the span of the four images to within 4.6×10^{-16}. The unit ratios recover all 4 of the 4 images exactly. For comparison, Equation 14.15 with $B = 4$ has $1 - 2^{-4} = 15/16$ and predicts $1 - (15/16)^{32} = 1 - e^{32\ln(15/16)} = 1 - e^{-2.065} = 0.873$ per example under independence.

Isolation against batch size. Table 14.5 compares Equation 14.15 with simulation, 20 random networks and batches per cell (see Figure 14.4 for the full range).

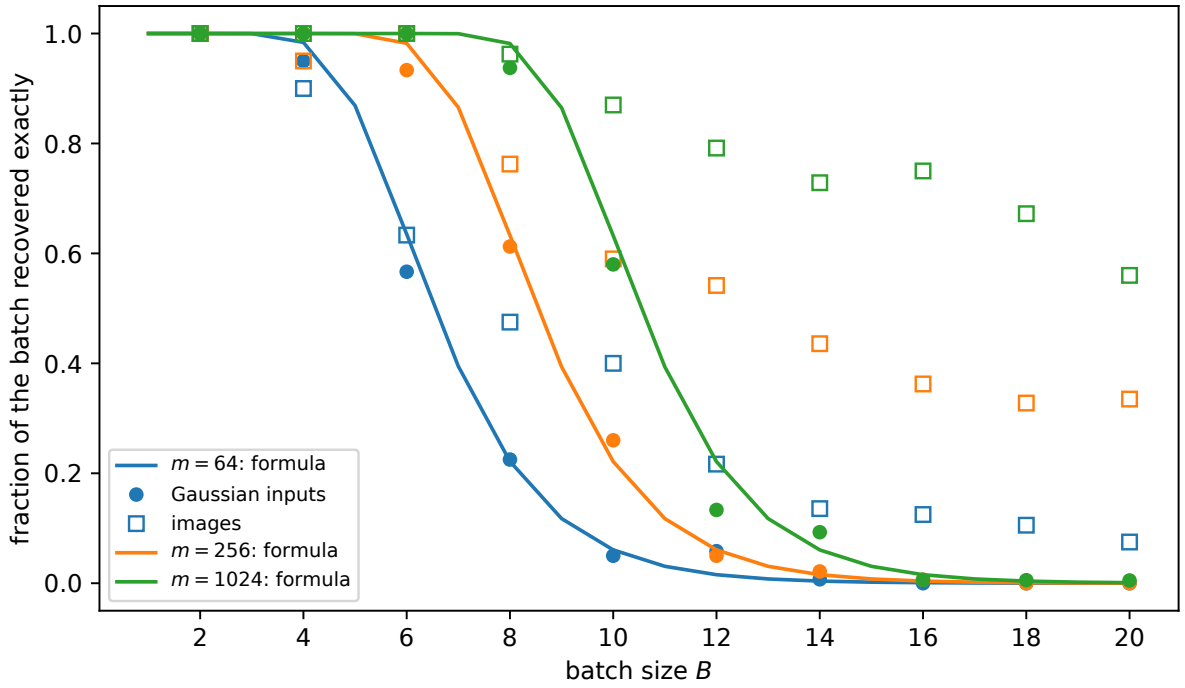


Figure 14.4.: Fraction of a batch recovered exactly by first-layer unit ratios, for ReLU networks with $m = 64$, 256 and 1024 hidden units. Lines: Equation 14.15. Filled circles: Gaussian inputs and zero first-layer bias, where each unit's on/off pattern is a sequence of fair coin flips. Open squares: the 8×8 images, which are correlated, so units fire on many or few of them and isolation decays much more slowly.

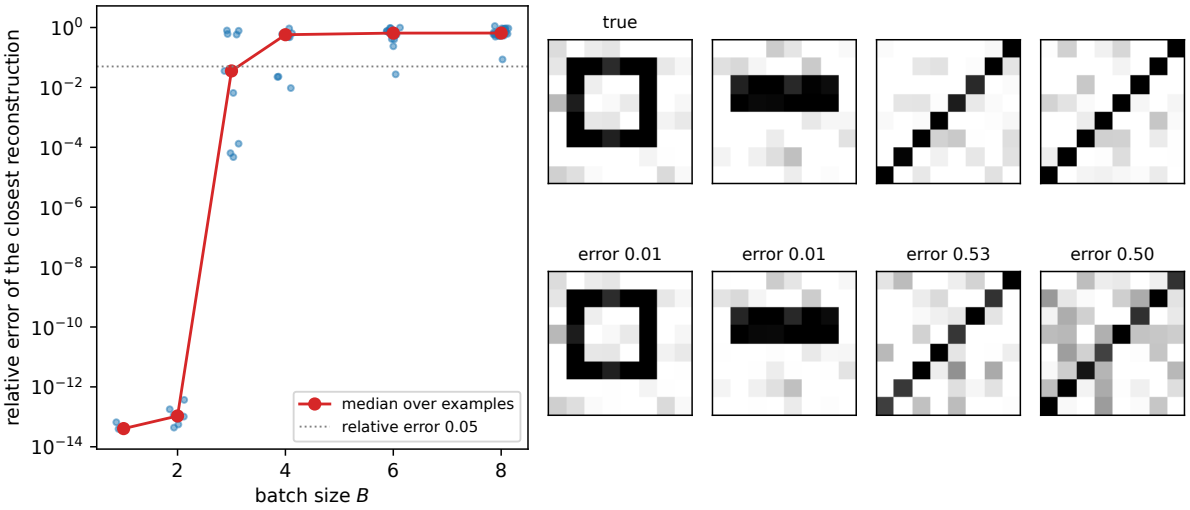


Figure 14.5.: Deep Leakage from Gradients against batch size, sigmoid network with 32 hidden units. Left: relative error of the closest reconstruction for every example (dots) and the median (red), on a log scale; the dotted line marks an error of 0.05. Right: a batch of 4, true images (top) and their closest reconstructions (bottom). Some examples are recovered and others are blends.

Table 14.5.: Fraction of batch examples recovered exactly from first-layer unit ratios of a random ReLU network, against Equation 14.15.

m	B	Equation 14.15	Gaussian inputs	Images
64	4	0.984	0.975	0.838
64	8	0.222	0.188	0.550
64	16	0.001	0.000	0.144
256	8	0.633	0.562	0.769
256	12	0.061	0.046	0.542
256	16	0.004	0.006	0.375
1024	12	0.221	0.196	0.850
1024	16	0.016	0.009	0.691

For Gaussian inputs the formula matches to within the simulation noise. It usually overestimates slightly (in seven of the eight rows), because units share inputs and so their on/off patterns are not fully independent (step 6 of the derivation). For images the formula is far too optimistic about privacy. At $B = 16$ it predicts that 1.6% of the batch is exposed in the 1024-unit network; 69.1% is.

At $B = 4$, by contrast, the images are recovered less often than the formula predicts (0.838 against 0.984 for $m = 64$). Overdispersion cuts both ways. The same spread that makes "on for exactly one image" more common at large B also makes "on for all four images" and "on for none" common at small B, and units like that isolate nothing.

DLG against batch size. Table 14.6 runs DLG on 6 random networks and batches for each batch size.

Table 14.6.: Deep Leakage from Gradients on batches, sigmoid network with 32 hidden units, 600 Adam steps, 6 trials per batch size.

B	Median relative error	Examples recovered (error < 0.05)	Median final matching loss
1	0.000	1.00	1.5×10^{-27}
2	0.000	0.67	1.3×10^{-27}
4	0.363	0.29	3.3×10^{-5}
8	0.711	0.00	7.7×10^{-5}

Batches of one and two are recovered to rounding error in most runs. From $B = 4$ the median run gets stuck: the matching loss stalls near 10^{-5}, many orders of magnitude above the 10^{-27} of a successful run, and the reconstructions are blends of the batch (Figure 14.5). Averaging has made the search harder, not the information smaller: there are still 2,212 equations, and on a ReLU network of the same width the closed-form isolation attack recovered all four images of a batch of 4 (above).

14.6.6. Check yourself

- A first layer has $m = 4096$ ReLU units and the batch size is 8. Under the independence model, what fraction of the batch is exposed? *(A unit isolates a given example with probability* $2^{-8} = 1/256$*, so the expected number of isolating units per example is* $4096/256 = 16$*, and* $P_{iso} = 1 - (1 - 1/256)^{4096} \approx 1 - e^{-16}$*: essentially the whole batch.)*
- Why does the span of the batch, revealed by Equation 14.13, matter if the attacker cannot tell which direction is which example? *(Each example is a combination of* B *known basis vectors, so any further search, such as DLG, has* B *unknowns per example instead of* d*: 4 instead of 64 here, and 4 instead of hundreds of thousands for real images.)*

14.7. Code: check every attack by computation

The script code/gradient_inversion.py computes every number in this chapter and draws every figure. The network, its forward and backward passes and the hand-written double backpropagation for DLG are small typed functions; the attacks call them, and the figures loop over them. From the repository root:

```
uv run python code/part02-attacks/gradient_inversion.py
```

It takes about half a minute. **Output** (seed 0), summarized in Table 14.7:

Table 14.7.: The script's output with the default seed.

Experiment	Result
Hand example (Table 14.2)	$p = (0.3441, 0.6559)$, $\delta_1 = (-0.3279, 0.9838)$; both unit ratios give $(0.5, 1, 0.2)$; $\nabla_{b_2}\ell = (-0.6559, 0.6559)$, label 1 (the first class; index 0 in the script)
Exact recovery, one image, ReLU, $m = 32$	13 usable units of 32; largest relative error 6.7×10^{-17}; $s_2/s_1 = 9.5 \times 10^{-17}$; 2,212 gradient entries for 64 pixels
Labels, one example, 1,000 trials	Bias rule 1.000; weight rule 1.000
Label counts, all exact, 200 batches	Averaged guess: $B = 4$: 1.000, $B = 16$: 0.975, $B = 64$: 0.525. Uniform guess: 0.925, 0.095, 0.000
DLG, one image (Table 14.4)	Relative error 5.8×10^{-14} after 600 steps; matching loss 1.3×10^{-27}
Double backpropagation against finite differences	Largest gap 1.2×10^{-10}
DLG on batches (Table 14.6)	Recovered: $B = 1$: 1.00, $B = 2$: 0.67, $B = 4$: 0.29, $B = 8$: 0.00
Batch of 4, ReLU	Rank 4; rows in the span to 4.6×10^{-16}; 4 of 4 images isolated

Experiment	Result
Isolation (Table 14.5)	Formula matches Gaussian inputs; images exposed far more at large B
Width for isolation probability $\frac{1}{2}$ at $B = 10$	710

In the table, s_1 and s_2 are the two largest singular values of the first-layer gradient, so their ratio measures how far the gradient is from rank one. tests/test_gradient_inversion.py checks that the output still matches.

What to take from it. Three things stand out. First, the closed forms are exact: unit ratios and the bias rule for labels are right to rounding error, every time, whatever the network does above the first layer, because everything above only rescales the rows that the division removes. Second, search works when the closed form is missing: DLG recovers single images to about 13 digits, and the final matching loss tells the attacker whether it has succeeded. Third, batching is not a defence. It weakens DLG on this toy, but the label histogram, the span of the batch and every isolated example still come out exactly, and real data are isolated far more often than the independence model predicts.

14.8. References

- Zhu, Liu & Han, "Deep Leakage from Gradients" (NeurIPS 2019). The gradient-matching attack of Section 14.5, with reconstructions of images and text from single-example and small-batch gradients; replaces ReLU by the sigmoid to keep the objective twice differentiable.
- Zhao, Mopuri & Bilen, "iDLG: Improved Deep Leakage from Gradients" (arXiv 2020). Label recovery from the sign of the last-layer gradient, and DLG with the label fixed.
- Phong, Aono, Hayashi, Wang & Moriai, "Privacy-Preserving Deep Learning via Additively Homomorphic Encryption" (IEEE Transactions on Information Forensics and Security, 2018). Points out that the input of a fully connected layer with a bias can be read off its gradient, Equation 14.7.
- Geiping, Bauermeister, Dröge & Moeller, "Inverting Gradients — How easy is it to break privacy in federated learning?" (NeurIPS 2020). Cosine-similarity matching with a total-variation image prior; reconstructions from trained networks, from several local steps and from batches; the proof that fully connected layers leak their inputs in closed form.
- Fowl, Geiping, Czaja, Goldblum & Goldstein, "Robbing the Fed: Directly Obtaining Private Data in Federated Learning with Modified Models" (ICLR 2022). A malicious server inserts a fully connected block whose units bin inputs by a feature, so that most examples of a large batch are isolated.
- Boenisch, Dziedzic, Schuster, Shamsabadi, Shumailov & Papernot, "When the Curious Abandon Honesty: Federated Learning Is Not Private" (IEEE EuroS&P 2023). Trap weights that make first-layer units fire for single examples; exact recovery from large batches by a malicious server.
- Wainakh et al., "User-Level Label Leakage from Gradients in Federated Learning" (PETS 2022). Label counts from batch gradients, as in Equation 14.10.
- Huang, Gupta, Song, Li & Arora, "Evaluating Gradient Inversion Attacks and Defenses in Federated Learning" (NeurIPS 2021). How the strength of these attacks depends on assumptions such as known labels, batch normalization statistics and untrained models.

- McMahan, Moore, Ramage, Hampson & Agüera y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data" (AISTATS 2017). The federated averaging protocol, where clients send updates after several local steps.

14.9. Cheat sheet

Network $a = W_1 x + b_1$, $h = \sigma(a)$, $z = W_2 h + b_2$, $p = \operatorname{softmax}(z)$, $\ell = -\ln p_y$; backward signals $\delta_2 = p - e_y$ and $\delta_1 = (W_2^\top \delta_2) \odot \sigma'(a)$. The client sends g, the gradient of its average loss over B examples. Table 14.8 collects the results.

Table 14.8.: Every result in this chapter.

Result	Formula	Needs	Fails when
Outer product (Fact 1)	$\nabla_W \ell = \delta x^\top$, $\nabla_b \ell = \delta$	A fully connected layer	—
Exact recovery, one example	$x = (\nabla_{W_1} \ell)_{j,:} / (\nabla_{b_1} \ell)_j$	First layer fully connected with bias	Convolutional first layer; batches (partly)
Label, one example	$y = \arg\min_k (\nabla_{b_2} \ell)_k$; or the non-positive row of $\nabla_{W_2} \ell$ when $h \geq 0$	Softmax and cross-entropy	p_y rounds to 1
Label counts, batch	$n_k = \sum_i p_{ik} - B(\nabla_{b_2} \ell)_k$	Last-layer bias; a guess of $\sum_i p_{ik}$	Large B with a poor guess of $\sum_i p_{ik}$
Deep Leakage from Gradients	$\min_{x'} \lVert G(x') - g \rVert^2$ by gradient descent	Twice-differentiable network; labels	Optimizer stuck (final $R \gg 0$), larger batches
Span of a batch	$\operatorname{rank} \nabla_{W_1} \ell \leq B$, rows in $\operatorname{span}\{x_i\}$	Fully connected first layer	—
Isolation, batch	$P_{\text{iso}} = 1 - (1 - 2^{-B})^m$ under independence	ReLU units, random weights	$m \ll 2^B$ for symmetric data; much slower decay for real data

Rules of thumb

- A gradient is a sum of outer products of backward signals with forward inputs. Any layer whose input you can isolate, you can read.
- One example, a fully connected layer with a bias: the input costs one division. The label costs one sign.
- When there is no closed form, match gradients. The final matching loss tells you whether it worked.
- Batching helps against search but not against structure. The label histogram and the span of the batch always leak, and examples isolated by a single unit leak exactly; with real data that is most of them for batches of 10 or more on wide layers.
- An honest server can already do all of this. A malicious server that chooses the weights can do it for large batches by design.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

15. Linkage and re-identification

15.1. Why this matters

A hospital wants to share its discharge records with researchers. It deletes the names, the addresses and the insurance numbers, and keeps what the research needs: postcode, date of birth, sex, diagnosis. Nobody's name is in the file. Is anyone in it identifiable?

Usually, yes. Most people are the only person with their combination of postcode, birth date and sex. Anyone who holds a second table with names and those same three columns, such as a voter roll, can join the two tables and put a name back on most rows. This is a **linkage attack**, and putting a name back on a "de-identified" record is **re-identification**. It needs no model, no gradients and no queries: only a release and some auxiliary data about the people in it. The other chapters of this part attack trained models. This one attacks the data itself, and it explains why removing names was never the same as anonymizing.

This chapter follows three questions, each building on the one before. The first is how many people are unique on a given set of attributes, and how you can predict that number without having the data. Once that is answered for tidy tables, the second question is what happens when the data are high-dimensional and sparse, like movie ratings or shopping baskets, so that no record matches the adversary's information exactly. The third is whether releasing only a sample helps, since a record that is unique in the sample need not be unique in the population.

15.1.1. The one idea: identifying information is measured in bits, and bits add up

Picking one person out of n takes $\log_2 n$ bits of information, because each bit halves the number of candidates and you need to halve n down to 1. That is 27.9 bits for the 248.7 million people in the United States in 1990, and 32.9 bits for 8 billion people. Each attribute you learn about someone carries some bits too: an attribute with v equally likely values is worth $\log_2 v$ bits. A sex is worth $\log_2 2 = 1$ bit, a birth month $\log_2 12 \approx 3.6$, and a birth date in a span of 80 years, one of $365 \times 80 = 29{,}200$ days, about $\log_2 29{,}200 = 14.8$. For independent attributes, the bits add: the chance of a given combination is the product of the chances of its values, and the logarithm turns that product into a sum. **Once the bits of the attributes you know exceed $\log_2 n$ by a few, almost everyone is unique.** Every result in this chapter makes that sentence precise: the expected fraction of unique people as a function of the bits (Section 15.4), the score that adds up bits from rare movie ratings (Section 15.5), and the correction for a sample (Section 15.6).

Table 15.1 lists the results and where each is used.

Table 15.1.: The results of this chapter and where each is used.

Result	Where it is used
Linkage success is the number of distinct quasi-identifier combinations over n	Scoring a release against an adversary who knows who is in it
Expected fraction unique, $\sum_c p_c(1-p_c)^{n-1}$	Predicting re-identification risk from a model of the population
Uniform cells: $e^{-n/M}$, a single curve in bits minus $\log_2 n$	The "33 bits" rule and its correction
Collision lower bound $(1-\sum_c p_c^2)^{n-1}$	A guaranteed uniqueness rate from marginals alone
Concentration of the fraction unique (McDiarmid)	Why one sample of the population is enough
Narayanan–Shmatikov score as a likelihood ratio	De-anonymizing sparse data with noisy auxiliary information
Eccentricity and its false-match rate	Deciding when the best match is the right one
Correctness of a match in a sample, $1/(1+\lambda(1-\pi))$ and $e^{-\lambda(1-\pi)}$	Why releasing a small sample does not protect the people in it

What you need. Every outside fact this chapter uses is restated where it is used; these links are for more detail. Entropy, the chain rule and the conversion between probabilities and bits are in Information theory. The binomial and Poisson distributions, and Poisson thinning (Exercise 2 in the exercise book of that chapter), are in Probability distributions. McDiarmid's inequality is in Concentration. The Neyman–Pearson lemma, the Bayesian view of a test, and why you should read a curve of true against false positive rates at small false positive rates are in Hypothesis testing. The birthday bound is in Cryptography basics. The vocabulary of adversaries, auxiliary knowledge and games is in Threat models.

Notation. A population has n people. Each person has a combination of attribute values, their **cell**. Under the model of the population, a person falls in cell c with probability p_c, independently of everyone else. There are M cells. A person is **unique** when nobody else is in their cell. U is the number of unique people. Logarithms are base 2 when the unit is bits and natural otherwise.

15.2. Toolkit: three facts every derivation uses

15.2.1. Fact 1 — Counting with indicators

To find how many people are unique on average, you do not need the distribution of U. Give each person a counter, an **indicator**, that is 1 if that person is unique and 0 otherwise. U is the sum of these counters, $U = \sum_{i=1}^{n} \mathbf{1}[\text{person } i \text{ is unique}]$. The average of a sum is the sum of the averages, and the average of a 0/1 counter is the probability that it is 1. So

$$\mathbb{E}U = \sum_{i=1}^{n} \Pr[\text{person } i \text{ is unique}]. \tag{15.1}$$

The rule "the average of a sum is the sum of the averages" is **linearity of expectation**, and it holds even when the terms depend on each other. Here they do: knowing that person 1 is unique changes the chance that person 2 is. Linearity does not care, because it only adds up how often each counter is 1, outcome by outcome. Walk through the smallest case to see it: $n = 3$ people, $M = 2$ equally likely cells A and B, so 8 equally likely outcomes (Table 15.2).

Table 15.2.: Three people, two equally likely cells: who is unique in each of the 8 outcomes.

Cells of (person 1, 2, 3)	`AAA`	`AAB`	`ABA`	`BAA`	`ABB`	`BAB`	`BBA`	`BBB`
Unique people	none	3	2	1	1	2	3	none
U	0	1	1	1	1	1	1	0

Directly, six of the eight outcomes have $U = 1$, so $\mathbb{E}U = 6/8 = 0.75$. Through the indicators, person 1 is unique when persons 2 and 3 are both in the other cell, which happens with probability $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$; the same holds for each person, so $\mathbb{E}U = 3 \times \frac{1}{4} = 0.75$. The two counts agree. This chapter uses the same move for the expected number of unique people, of sample uniques, and of false matches.

15.2.2. Fact 2 — $(1-p)^k$ is e^{-pk}, up to a factor you can bound

The question "does each of the other k people avoid my cell?" has the answer $(1-p)^k$ when each avoids it with probability $1-p$. Powers like this are awkward, and an exponential is easier to read: $(1-p)^k$ is close to e^{-pk} when p is small, because $1-p$ is close to e^{-p} for small p. The fact pins down how close, with bounds on both sides. For $0 \le p < 1$ and $k \ge 0$,

$$e^{-pk/(1-p)} \le (1-p)^k \le e^{-pk}. \tag{15.2}$$

To prove it, take logarithms, so that $(1-p)^k$ becomes $k\ln(1-p)$ and the claim becomes $-\frac{p}{1-p} \le \ln(1-p) \le -p$. Both sides come from one inequality, $\ln x \le x - 1$ for all $x > 0$, which holds because ln is concave and $x - 1$ is its tangent line at $x = 1$, and a concave curve lies below its tangents. With $x = 1 - p$ it gives the upper bound $\ln(1-p) \le -p$. With $x = \frac{1}{1-p}$ it gives $\ln \frac{1}{1-p} \le \frac{1}{1-p} - 1 = \frac{p}{1-p}$, and flipping the sign gives the lower bound $\ln(1-p) \ge -\frac{p}{1-p}$. The two bounds differ only by the factor $\frac{1}{1-p}$ in the exponent, which is close to 1 when p is small.

To see how tight this is, take $p = 10^{-4}$ and $k = 9999$. Then $pk = 0.9999$ and $pk/(1-p) = 0.9999/0.9999 = 1$, and $(1-p)^k = 0.36790$ sits between the lower bound $e^{-pk/(1-p)} = e^{-1} = 0.36788$ and the upper bound $e^{-pk} = e^{-0.9999} = 0.36792$. This is how the chapter turns "every one of the other $n-1$ people avoids my cell" into e^{-np}.

15.2.3. Fact 3 — Probabilities multiply, bits add

If attributes $A_1, \dots, A_k$ are independent, the probability of a cell is the product of the probabilities of its values, $p_c = \prod_j p_j(c_j)$, where $p_j(c_j)$ is the probability that attribute j takes the value c_j that the cell has. Three quantities that this chapter uses follow from that one product.

The **surprisal** of a cell, $\log_2(1/p_c)$, is the number of bits it takes to single out that cell: a cell with probability 2^{-10} has a surprisal of 10 bits. Since the logarithm of a product is the sum of the logarithms, surprisal adds up over attributes, $\log_2(1/p_c) = \sum_j \log_2(1/p_j(c_j))$.

The **entropy** is the average surprisal, $H = \sum_c p_c \log_2(1/p_c)$: how many bits a typical person's cell carries. Averaging the sum above term by term, it adds up too, $H(A_1, \dots, A_k) = \sum_j H(A_j)$. This is the chain rule of Information theory, which says that the entropy of a pair is the entropy of the first plus that of the second given the first, in the special case where knowing the first tells you nothing about the second.

The **collision probability** $\sum_c p_c^2$ is the chance that two independent people land in the same cell: both land in cell c with probability $p_c \cdot p_c$, and you add over cells. It multiplies over attributes, because two people share a cell exactly when they share the value of every attribute, and for independent attributes those agreements are independent events. In symbols, $\sum_c \prod_j p_j(c_j)^2 = \prod_j \sum_v p_j(v)^2$, which you can check by expanding the product of sums on the right: each term of the expansion picks one value for each attribute, that is, one cell.

Real attributes are not independent (age and number of children, ZIP code and income), and then the pair carries fewer bits than the sum of their separate bits, because part of what the second attribute says was already said by the first. The sum is therefore an upper bound on the joint entropy, and adding up bits overstates how identifying correlated attributes are.

15.3. Quasi-identifiers and the linkage attack

15.3.1. Context

Data protection practice sorts the columns of a table into three kinds. **Direct identifiers** name a person on their own: name, social security number, email address. **Sensitive attributes** are what the release is about and what must not be tied to a person: a diagnosis, a salary. **Quasi-identifiers** are everything in between: attributes that do not name anyone alone but that, combined, often single out one person, and that also appear in other datasets with names attached. ZIP code, birth date and sex are the classic example. Removing direct identifiers and keeping quasi-identifiers is the most common form of "de-identification", and the linkage attack is what breaks it. This section asks how often the attack names the right record.

15.3.2. Statement

The linkage attack. The release R holds records (quasi-identifiers, sensitive attribute). The adversary holds an auxiliary table with (name, quasi-identifiers), for example a voter roll. For each name, it finds the records of R with the same quasi-identifiers and picks one of them uniformly at random.

The game. In the language of Threat models, an attack is scored by a game. A challenger draws a target uniformly from the n people in R and gives the adversary the target's quasi-identifiers. The adversary wins if it outputs the target's record. Group the records of R into **equivalence classes**, sets of records with identical quasi-identifiers, with sizes $k_1, \dots, k_J$; there are J classes, one per distinct combination. A target alone in its class is found for sure, and a target in a class of k look-alikes is found one time in k. A class of k records is therefore hit k times as often as a single record but won only a $1/k$ share of those times, so each class contributes the same amount, one record's worth, to the wins. Averaging over the target gives the success probability of the linkage attack,

$$\Pr[\text{win}] = \frac{1}{n} \sum_{j=1}^{J} k_j \cdot \frac{1}{k_j} = \frac{J}{n} : \tag{15.3}$$

the number of distinct quasi-identifier combinations divided by the number of records. No adversary that sees only the quasi-identifiers can do better.

Sweeney's measurement. Sweeney (2000) estimated from 1990 census tables that 87% of the US population, 216 of 248 million people, were likely unique on {5-digit ZIP code, date of birth, sex}. On {city, date of birth, sex} the figure was 53%, and on {county, date of birth, sex} 18%.

15.3.3. Derivation

The idea is to fix the class the target falls in, find the best guess within it, and then average over classes.

1. **Condition on the target's class.** A target in a class of size k_j shares its quasi-identifiers with $k_j - 1$ other records. To an adversary who sees only quasi-identifiers, those records look the same as the target's. Picking one of the k_j at random wins with probability $1/k_j$.
2. **Show that nothing beats this.** Within a class, the target is equally likely to be any of the k_j records, because it was drawn uniformly and the records differ only in their sensitive attribute, which the adversary does not know. So every guess within the class wins with probability exactly $1/k_j$, and a guess outside the class never wins, since a record there has different quasi-identifiers from the target.
3. **Average over the target.** The target lands in class j with probability k_j/n, the share of records in that class. Weighting each class's win probability by how often the target lands in it gives $\Pr[\text{win}] = \sum_j \frac{k_j}{n} \cdot \frac{1}{k_j} = \frac{J}{n}$.

15.3.4. Interpretation

For this adversary, uniqueness is the whole story. A unique record is re-identified with certainty and a record in a class of 2 with probability $\frac{1}{2}$. Each unique record adds a full win and no class adds more, so the fraction of unique people is a lower bound on the success, and the success is at most 1. That is why the next section is about predicting uniqueness.

The game assumes the adversary knows the target is in the release, as a curious neighbour or an insurer with a list of patients might. Without that knowledge, a unique match in the release may

belong to someone else who happens to share the quasi-identifiers but is not in R. Whether that uncertainty protects anyone is the question of Section 15.6.

Re-identification is also not the only harm. If all records of a class share one diagnosis, the adversary learns the diagnosis without knowing which record is the target's. In the other direction, Equation 15.3 shows that a release in which every class has at least k records keeps the linkage success at most $1/k$: each class then contributes one win for at least k records, so $J \leq n/k$. That guarantee holds against this adversary only. Syntactic anonymity and how it breaks studies such releases and the attacks on the sensitive attribute that they leave open.

15.3.5. Example: one hospital release, one voter roll

Table 15.3 is a hospital release with names removed, and Table 15.4 part of a public voter roll.

Table 15.3.: A de-identified hospital release: no names, three quasi-identifiers, one sensitive attribute.

ZIP code	Birth date	Sex	Diagnosis
02138	1961-03-14	F	asthma
02138	1972-11-02	M	diabetes
02138	1972-11-02	M	hypertension
02139	1958-06-21	F	depression
02139	1990-01-01	F	migraine
02139	1990-01-01	F	asthma
02139	1990-01-01	F	fracture
02141	1983-09-30	M	pneumonia

Table 15.4.: Part of a voter roll: names with the same three quasi-identifiers.

Name	ZIP code	Birth date	Sex
Ana Silva	02138	1961-03-14	F
Ben Cole	02138	1972-11-02	M
Carla Diaz	02139	1958-06-21	F
Dan Eto	02141	1983-09-30	M

Joining on the three columns gives Ana's asthma, Carla's depression and Dan's pneumonia with certainty, because each of them matches exactly one record. Ben matches two records, so the adversary gets diabetes or hypertension and is right with probability $\frac{1}{2}$. The release has classes of sizes 1, 1, 1, 2 and 3, so $J = 5$ classes for $n = 8$ records, and Equation 15.3 gives a success of $5/8 = 0.625$ for a target drawn from the release.

This is, in miniature, how Sweeney re-identified the governor of Massachusetts in the late 1990s. The state's Group Insurance Commission had released hospital records of about 135,000 state employees and their families without names. Sweeney bought the Cambridge voter roll for twenty dollars. Six people on it shared the governor's birth date, three of them were men, and only one of those lived in his ZIP code.

In bits (Figure 15.1). Now count the bits of Sweeney's three attributes, treating each as uniform over its values, which gives an upper bound. There are about 40,000 ZIP codes, $365 \times 80 = 29{,}200$ birthdays in about 80 birth years, and two sexes, so the three attributes carry at most $\log_2 40{,}000 + \log_2 29{,}200 + 1 = 15.3 + 14.8 + 1.0 = 31.1$ bits, against $\log_2(248.7 \times 10^6) = 27.9$ bits needed to single out one American. The same count in cells: there are $M = 40{,}000 \times 29{,}200 \times 2 = 2.34 \times 10^9$ cells for $n = 248.7$ million people, so $n/M = 0.106$ people per cell. If everyone fell into a uniformly random cell, the next section shows that a fraction $e^{-n/M} = e^{-0.106} = 0.899$ would be unique. Sweeney's 87% is a little lower because people are not spread uniformly: ZIP codes range from a few residents to tens of thousands, and birth years are not equally common, so people crowd into the popular cells. Golle (2006) repeated the estimate with the 2000 census and found 63%. Both estimates have to model how birth dates spread within the age groups that census tables publish, and the answer is sensitive to that modelling.

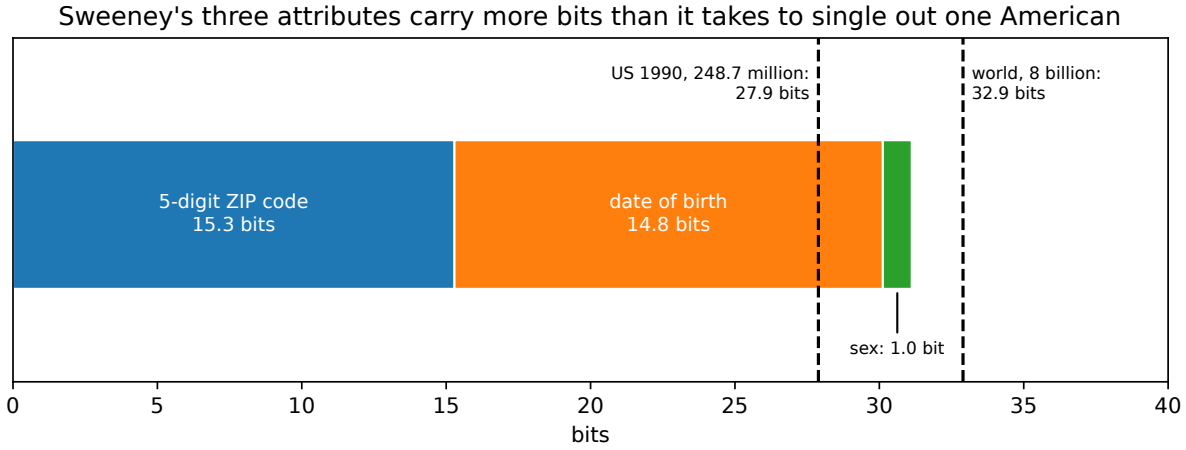


Figure 15.1.: The three classic quasi-identifiers, in bits, against the bits needed to single out one person in the US population of 1990 and in the world population. The bits are upper bounds that treat each attribute as uniform over its values.

15.3.6. Check yourself

- A release of 1,000 records has 900 distinct quasi-identifier combinations. What is the linkage success for a target drawn from the release, and what is the smallest possible number of unique records? *(0.9 by Equation 15.3. At least 800: with s singletons, the other $900 - s$ classes hold at least 2 records each, so $s + 2(900 - s) \leq 1000$, that is $s \geq 800$, attained with 100 pairs.)*
- Why does it not matter for Equation 15.3 what the sensitive attribute is? *(The adversary chooses only from the quasi-identifiers. The sensitive attribute is what it learns after winning, not what it uses to win.)*

15.4. How many people are unique? A birthday calculation

15.4.1. Context

A data custodian who wants to know the re-identification risk of a release can count unique records in it. But counting needs the data, and it says nothing about a new release or a different set of columns. A model of the population answers both. If each person falls in cell c with probability p_c, the expected fraction of unique people has a closed form. It is the birthday problem asked the other way round. The birthday problem, in Cryptography basics, asks whether *any* two of n people share a birthday; here you ask how many people share their cell with *nobody*.

15.4.2. Statement

Exact expected fraction. A person is unique when each of the other $n-1$ people lands outside their cell. If the person is in cell c, each other person avoids it with probability $1-p_c$, independently, so all of them avoid it with probability $(1-p_c)^{n-1}$. Averaging over the person's own cell, which is c with probability p_c, gives the expected fraction of unique people, for n people falling in cells independently with probabilities (p_c):

$$\frac{\mathbb{E}U}{n} = \sum_c p_c(1-p_c)^{n-1}. \tag{15.4}$$

Uniform cells. If all M cells are equally likely, every term has $p_c = 1/M$, and the formula becomes $(1-1/M)^{n-1} \approx e^{-n/M}$: the chance that nobody else lands in your cell when the average cell holds n/M people. Now count the cells in bits, $b = \log_2 M$, so that $M = 2^b$ and $n = 2^{\log_2 n}$. The average occupancy is $n/M = 2^{\log_2 n - b}$, and

$$\frac{\mathbb{E}U}{n} \approx \exp\big(-2^{\log_2 n - b}\big), \tag{15.5}$$

a single curve in $b - \log_2 n$, the bits beyond those needed to name one person. Solving $e^{-n/M} = f$ for the bits says how many you need for a fraction f of people to be unique: $b = \log_2 n + \log_2 \frac{1}{\ln(1/f)}$. That is half once b exceeds $\log_2 n$ by 0.53 bits, 90% at 3.25 bits more and 99% at 6.64 bits more.

A lower bound from the collision probability. The exact formula needs every p_c, and there can be billions of cells. A cruder question needs much less: how likely is a random person to share a cell with one other random person? That is the collision probability $\sum_c p_c^2$ of Fact 3. If you pretend every person's cell is as crowded as average in this sense, you get a guaranteed floor on uniqueness. For any (p_c),

$$\frac{\mathbb{E}U}{n} \geq \Big(1 - \sum_c p_c^2\Big)^{n-1}, \tag{15.6}$$

and for independent attributes $\sum_c p_c^2 = \prod_j \sum_v p_j(v)^2$ by Fact 3, so the bound needs only the per-attribute frequencies (the **marginals**).

Concentration. The expectation is an average over many imagined populations, but you have only one. Moving a single person to another cell changes the number of unique people by at most 2, so in a large population no one person can move the fraction much, and the fraction in the population you have is close to its expectation: for every $t > 0$,

$$\Pr\left[\left|\frac{U}{n} - \frac{\mathbb{E}U}{n}\right| \geq t\right] \leq 2e^{-nt^2/2}. \tag{15.7}$$

15.4.3. Derivation

The exact formula (Equation 15.4).

1. **Count with indicators.** By Fact 1, $\mathbb{E}U = \sum_i \Pr[\text{person } i \text{ is unique}]$. Every person draws their cell from the same probabilities (p_c), independently of the others, so nothing distinguishes person i from person j and every term of the sum is the same. It is enough to find it for one person.
2. **Condition on the person's own cell.** Call it c; it has probability p_c.
3. **Ask the others to stay away.** Given c, person i is unique if each of the other $n-1$ people lands outside c. They are independent and each avoids c with probability $1-p_c$, so this has probability $(1-p_c)^{n-1}$.
4. **Average over the cell.** $\Pr[\text{person } i \text{ is unique}] = \sum_c p_c(1-p_c)^{n-1}$. Since all n terms are equal, $\mathbb{E}U$ is n times this; divide by n.

Uniform cells (Equation 15.5).

1. **Put $p_c = 1/M$ in Equation 15.4.** There are M terms, each $\frac{1}{M}(1-\frac{1}{M})^{n-1}$, so the sum is $(1-\frac{1}{M})^{n-1}$.
2. **Replace the power by an exponential.** Fact 2 with $p = 1/M$ and $k = n-1$ squeezes $(1-\frac{1}{M})^{n-1}$ between $e^{-(n-1)/M}$ and $e^{-(n-1)/(M-1)}$. These exponents differ by $\frac{n-1}{M(M-1)}$, of order n/M^2, and replacing $n-1$ by n changes the exponent by only $1/M$. So $(1-\frac{1}{M})^{n-1} \approx e^{-n/M}$ with relative error of order $1/M + n/M^2$, negligible for M in the thousands or more.
3. **Write the exponent in bits.** $n/M = 2^{\log_2 n}/2^b = 2^{\log_2 n - b}$.
4. **Find the bits for a target fraction.** Solve $e^{-n/M} = f$ for M: taking logarithms, $n/M = \ln(1/f)$, so $M = n/\ln(1/f)$ and $b = \log_2 n - \log_2 \ln(1/f)$. At $f = \frac{1}{2}$, $\ln 2 = 0.693$ and $-\log_2 0.693 = 0.53$. At $f = 0.9$, $\ln \frac{1}{0.9} = 0.105$ and $-\log_2 0.105 = 3.25$. At $f = 0.99$, $\ln \frac{1}{0.99} = 0.0101$ and $-\log_2 0.0101 = 6.64$.

The collision lower bound (Equation 15.6). The idea is that Equation 15.4 is an average of the function $(1-x)^{n-1}$ over the cell probabilities people find themselves in, and for a curve that bends upwards the average of the function is at least the function of the average.

1. **Write Equation 15.4 as an average.** Draw a person's cell C, so $C = c$ with probability p_c, and look at the probability p_C of the cell they landed in. Then $\frac{\mathbb{E}U}{n} = \mathbb{E}[g(p_C)]$ with $g(x) = (1-x)^{n-1}$.
2. **Check that g bends upwards.** g is convex on $[0,1]$, because its second derivative $g''(x) = (n-1)(n-2)(1-x)^{n-3}$ is never negative there.

3. **Apply Jensen's inequality.** For a convex function, the average of the function is at least the function of the average, $\mathbb{E}[g(X)] \geq g(\mathbb{E}X)$. The reason is that a convex curve lies above its tangent line at the point $\mathbb{E}X$; averaging the curve gives at least the average of the tangent line, which is the tangent's value at $\mathbb{E}X$, which is $g(\mathbb{E}X)$. So $\mathbb{E}[g(p_C)] \geq g(\mathbb{E}[p_C])$.
4. **Compute the inner average.** $\mathbb{E}[p_C] = \sum_c p_c \cdot p_c = \sum_c p_c^2$, the collision probability. For independent attributes it factorizes by Fact 3.

Equality in step 3 holds when p_C is constant, that is, when all cells with positive probability are equally likely, because then there is nothing to average. The bound is tight for uniform cells and loose when a few cells hold most of the people: those crowded cells push the collision probability up and the bound down, while the many people in rare cells are still unique.

Concentration (Equation 15.7). The tool is McDiarmid's inequality (Concentration). It applies to any function f of n independent inputs such that changing input i alone, with the others fixed, moves f by at most c_i. It says that f is unlikely to be far from its mean: $\Pr[f - \mathbb{E}f \geq t] \leq \exp(-2t^2/\sum_i c_i^2)$, and the same for the lower tail. So the work is to find c_i.

1. **Change one input.** U is a function of the n independent cells. Change one person's cell from a to b; only cells a and b change their counts, so only they can change U.
2. **Bound the change.** A cell with count k contributes $\mathbf{1}[k = 1]$ to U: it holds a unique person exactly when it holds one person. Cell a's count drops by one and its contribution changes by $\mathbf{1}[k_a - 1 = 1] - \mathbf{1}[k_a = 1] \in \{-1, 0, 1\}$. Cell b's count rises by one, and its contribution likewise changes by -1, 0 or 1. So U changes by at most 2, and U/n by at most $c_i = 2/n$. (It can change by exactly 2: move a person out of a pair and into an empty cell, and both the person and their former partner become unique.)
3. **Apply McDiarmid.** With $c_i = 2/n$ for all n people, $\sum_i c_i^2 = n \cdot 4/n^2 = 4/n$, so each tail is at most $\exp(-2t^2/(4/n)) = e^{-nt^2/2}$. Adding the upper and lower tails gives the factor 2.

15.4.4. Interpretation

For uniform cells, the bits picture is exact. Equation 15.5 depends on n and M only through $b - \log_2 n$, so the curve is the same for a town and for the planet (Figure 15.2, right). The often-quoted "33 bits identify anyone" is the point $b = \log_2 n$, where the average cell holds one person and the curve is at e^{-1}. With 2^{33} uniform cells for 8 billion people, $n/M = 8 \times 10^9/2^{33} = 0.93$ and only a fraction $e^{-0.93} = 0.394$ is unique. 33 bits are enough to *give* everyone a distinct label, but random labels of 33 bits collide often; you need $32.9 + 6.64 = 39.5$ bits for 99% of the world to be unique.

Skew usually lowers uniqueness. When a few values are common (a big city, a common occupation), many people crowd into a few cells and are not unique, while many rare cells stay empty. In the census below, and for Sweeney's attributes, the uniform model with the same number of cells overstates uniqueness for this reason. This is not a theorem, though: with far fewer cells than people, a uniform spread leaves nobody unique, while a few rare cells would hold some uniques.

The entropy H gives a rule of thumb, not a bound. The rule replaces the real cells by 2^H equally likely cells, as if every cell carried the average surprisal, and gives $e^{-n2^{-H}}$. The collision bound is always below it (up to replacing n by $n - 1$), because the collision probability is at least 2^{-H}. To see why, note that $\log_2 \sum_c p_c^2$ is the logarithm of an average of p_c (weighted by p_c), while $-H = \sum_c p_c \log_2 p_c$ is the average of the logarithm, and for the concave logarithm

Jensen's inequality runs the other way: the log of an average is at least the average of the log. So $\log_2 \sum_c p_c^2 \geq -H$. The exact value can sit on either side of the entropy rule: in Table 15.5 it is above it with three and four attributes and below it with five or more.

If attributes are independent, the prediction needs only the marginals. In practice they are not, and a model of the joint distribution is needed. That is what Rocher, Hendrickx & de Montjoye (2019) fit, a **Gaussian copula**: a model that keeps each attribute's own frequencies and ties the attributes together through the correlations of a multivariate normal distribution. It is how Section 15.6 estimates risk from a sample.

Finally, one population is enough. Setting the right side of Equation 15.7 equal to a failure probability β and solving for t says that the fraction unique in the population you have is within $\sqrt{2\ln(2/\beta)/n}$ of its expectation with probability $1-\beta$. For $n = 10{,}000$ and $\beta = 0.05$ that is $\sqrt{2\ln 40/10^4} = 0.0272$. That is about five times the standard deviation of the fraction unique over the 20 simulated populations behind Table 15.5 (at most 0.0053, which the script prints next to each simulated mean), where a normal approximation would put 95% of populations within two standard deviations. McDiarmid is loose here because it charges every person the worst case, a change of 2 in U, while most changes move it by 0 or 1.

15.4.5. Example: a synthetic census

The script builds a population of $n = 10{,}000$ people with eight independent attributes, made up but shaped like real ones: sex (uniform), age in 16 five-year bands (flat, then thinning out), region (40 values with Zipf frequencies, so a few regions are large), birth month (uniform), marital status, number of children, occupation (20 values, Zipf) and blood group (real-world frequencies). It reveals them one at a time. Table 15.5 gives, for each step, the entropy H of the attributes known so far and the number of cells M, then the exact expected fraction unique from Equation 15.4, the average over 20 simulated populations, the collision lower bound of Equation 15.6, the entropy rule $e^{-n2^{-H}}$ and the uniform-cells value $e^{-n/M}$ for the same number of cells.

Table 15.5.: Fraction of 10,000 people who are unique as attributes are revealed: exact, simulated (mean of 20 populations), and three approximations. $\log_2 n = 13.29$ bits.

Attributes known	Bits H	Cells M	Exact	Simulated	Collision bound	Entropy rule	Uniform cells
sex, age band, region	9.31	1,280	0.0158	0.0160	0.0000	0.0000	0.0004
+ birth month	12.90	15,360	0.3328	0.3332	0.0834	0.2692	0.5215
+ marital status	14.60	61,440	0.5789	0.5797	0.4237	0.6690	0.8498
+ children	16.48	368,640	0.7939	0.7918	0.7443	0.8967	0.9732
+ occupation	20.39	7,372,800	0.9747	0.9737	0.9736	0.9928	0.9986

Attributes known	Bits H	Cells M	Exact	Simulated	Collision bound	Entropy rule	Uniform cells
+ blood group	22.57	58,982,400	0.9925	0.9926	0.9924	0.9984	0.9998

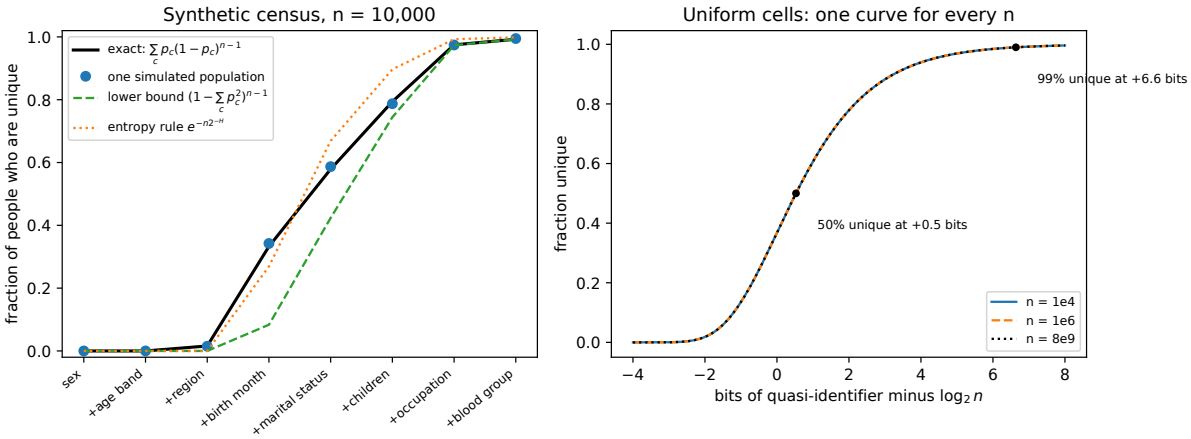


Figure 15.2.: Left: the fraction of people who are unique in the synthetic census as attributes are added, exact against one simulated population, with the collision lower bound and the entropy rule. Right: with uniform cells, the fraction unique depends only on the bits beyond $\log_2 n$; the curves for $n = 10^4$, 10^6 and 8×10^9 coincide.

Read the table along the bits column, against $\log_2 n = 13.29$. With 12.90 bits, just below it, a third of the people are unique. With 20.39 bits, $20.39 - 13.29 = 7.1$ bits beyond it, 97.5% are, close to the 99% that the uniform curve predicts at 6.64 bits beyond. Uniform cells overstate uniqueness at every step from four attributes on, by up to $0.8498 - 0.5789 = 0.27$ (with five attributes), because the attributes are skewed. The collision bound is loose with few attributes, where region and age band put most people in a few large cells, and becomes tight once most cells are nearly empty. Simulation and the exact formula agree to within 0.003.

15.4.6. Check yourself

- With $n = 1000$ people and $M = 1000$ equally likely cells, what fraction is unique, and how many bits beyond $\log_2 n$ is that? *($(1 - 0.001)^{999} = 0.368 \approx e^{-1}$, at zero bits beyond $\log_2 n$, since $M = n$.)*
- Why can the indicators "person i is unique" be dependent and still give $\mathbb{E}U = n \Pr[\text{person 1 is unique}]$? *(Linearity of expectation holds for any random variables, dependent or not. Dependence matters for the variance, which is why the concentration step needs McDiarmid rather than a sum of independent terms.)*

15.5. Sparse, high-dimensional data: the Netflix attack

15.5.1. Context

In 2006 Netflix released about 100 million movie ratings by 480,189 subscribers for a public competition. Names were replaced by random numbers, and some ratings and dates were perturbed. The data look nothing like a census table: each record is a set of (movie, rating, date) triples, 209 on average out of 17,770 movies. There are no quasi-identifier columns to join on, and the adversary's information about a person ("she watched these films around these dates and liked them") is partial and noisy. Narayanan & Shmatikov (2008) showed that this does not help. In a sparse, high-dimensional dataset, a handful of approximate facts about a person typically matches one record far better than any other. They then matched a small sample of Netflix records to public reviews on the Internet Movie Database, which carry names or pseudonyms. This section explains why a scoring rule that favours rare items works, and when to trust its answer.

15.5.2. Statement

The scoring algorithm. The adversary has auxiliary information aux: a few items i (films) with approximate values aux_i (ratings). Since no record will match exactly, it scores how well each record r of the release agrees, item by item, and adds up the agreements. An agreement on a film that half the subscribers rated says little, while an agreement on an obscure film says a lot, so each item gets a weight w_i that is larger for rarer items:

$$\text{Score}(\text{aux}, r) = \sum_{i \in \text{aux}} w_i \, \text{Sim}(\text{aux}_i, r_i), \tag{15.8}$$

where r_i is the record's value for item i and $\text{Sim} \in [0, 1]$ measures agreement (here: 1 if r rated i within one star of aux_i, else 0). Narayanan and Shmatikov use $w_i = 1/\ln|\text{supp}(i)|$, where $|\text{supp}(i)|$, the **support** of item i, is the number of records that rated it.

The eccentricity test. The top-scoring record is the best guess, but the adversary also needs to know whether to trust it. A real match should stand out: its score should be well above the second-best one, measured against how spread out all the scores are. Let $s_1 \geq s_2$ be the two largest scores and σ the standard deviation of all scores. The adversary declares a match only if the gap, in units of σ, clears a threshold φ,

$$\frac{s_1 - s_2}{\sigma} \geq \varphi, \tag{15.9}$$

and otherwise says "no match". Narayanan and Shmatikov used $\varphi = 1.5$.

Their findings on Netflix. With 8 ratings, of which 2 may be completely wrong, and dates known to within 14 days, 99% of records are uniquely identified. With 2 ratings and dates within 3 days, 68% are.

The likelihood-ratio form of the score. The weighting can be derived rather than guessed. For each record, the adversary is weighing two explanations: "this record is the target" against "this record is someone unrelated". The standard way to weigh two explanations of the same

observations is the **likelihood ratio**, the probability of what you observed under the first explanation divided by its probability under the second. Its logarithm is a sum over items when items behave independently, so it has exactly the form of Equation 15.8. To make this precise, suppose an unrelated record agrees with the auxiliary value of item i with probability s_i, independently across items, and the target's record agrees with probability $1-\varepsilon$, where ε covers wrong or stale auxiliary information. Then ranking records by the likelihood ratio is ranking them by Equation 15.8 with

$$w_i = \ln \frac{1-s_i}{s_i} + \ln \frac{1-\varepsilon}{\varepsilon} \approx \ln \frac{1}{s_i} + \ln \frac{1-\varepsilon}{\varepsilon}, \tag{15.10}$$

where the approximation uses $1-s_i \approx 1$ for rare items. The record with the highest score is the most likely to be the target.

15.5.3. Derivation

The weights. The plan is to write the probability of a record's pattern of agreements under each explanation, take the log of the ratio, and collect the terms that differ from record to record.

1. **Record the agreements.** Write $a_i(r) = \mathrm{Sim}(\mathrm{aux}_i, r_i) \in \{0,1\}$: 1 if record r agrees with the auxiliary information on item i.
2. **Write the probability of the pattern under each explanation.** If r is the target, each item agrees with probability $1-\varepsilon$ and disagrees with probability ε, independently, so the pattern has probability $\Pr[a(r)] = \prod_i (1-\varepsilon)^{a_i}\varepsilon^{1-a_i}$ (the exponents pick the right factor for each item). If r is unrelated, each item agrees with probability s_i, so $\Pr[a(r)] = \prod_i s_i^{a_i}(1-s_i)^{1-a_i}$.
3. **Take the log of the ratio.** The logarithm turns the ratio of products into a sum over items: $\sum_i \left[a_i \ln \frac{1-\varepsilon}{s_i} + (1-a_i)\ln \frac{\varepsilon}{1-s_i}\right]$.
4. **Keep only what depends on the record.** Collect the terms with a_i: $\sum_i a_i\left[\ln \frac{1-\varepsilon}{s_i} - \ln \frac{\varepsilon}{1-s_i}\right] + \sum_i \ln \frac{\varepsilon}{1-s_i}$. The last sum is the same for every record, so it does not change the ranking. The bracket, rearranged, is Equation 15.10: the log-likelihood ratio is the score with those weights, plus a constant.
5. **Why the likelihood ratio is the right thing to threshold.** For any single record, deciding between "target" and "unrelated" is a hypothesis test. The Neyman–Pearson lemma of Hypothesis testing says that, among all tests that wrongly accept an unrelated record with a given probability, the one that accepts when the likelihood ratio exceeds a threshold accepts the true target most often. The intuition: if you can afford only so many false accepts, spend them on the patterns that are most typical of the target relative to an unrelated record.

The best guess is the top score. Step 5 is about one record at a time. When the adversary knows the target is one of the n records and must pick one, Bayes' rule gives the answer. Suppose each record is equally likely a priori to be the target, and records are independent given who the target is.

1. **Write the probability of everything you see if t is the target.** Record t's pattern follows the target distribution and every other record's follows the unrelated one, so all

the agreement patterns together have probability $P_1(a(t)) \prod_{j \neq t} P_0(a(j))$, with P_1 and P_0 the two products of step 2 above.

2. **Turn it into a posterior.** Bayes' rule says the posterior probability of t is proportional to its prior times this probability, and the prior is the same for every t. Divide by $\prod_j P_0(a(j))$, which does not depend on t either: what remains is $P_1(a(t))/P_0(a(t)) = \mathrm{LR}(t)$, the likelihood ratio of record t. Normalizing so the probabilities sum to 1,

$$\Pr[\text{target} = t \mid \text{data}] = \frac{\mathrm{LR}(t)}{\sum_j \mathrm{LR}(j)}. \tag{15.11}$$

3. **Pick the most probable record.** The maximum a posteriori (MAP) guess is the record with the largest likelihood ratio, which is the top score.

Why rare items carry the attack. A false match is an unrelated record that happens to agree on every item the adversary knows. Take the clean case, $\varepsilon \to 0$ and exact agreement required. An unrelated record agrees on all of a set I of items with probability $\prod_{i \in I} s_i$, and by Fact 1 the expected number of false matches among n records is n times that. Writing both factors as powers of 2 shows the bits:

$$n \prod_{i \in I} s_i = 2^{\log_2 n - \sum_{i \in I} \log_2(1/s_i)}. \tag{15.12}$$

Each item contributes $\log_2(1/s_i)$ bits, its surprisal, and the false matches vanish once the bits exceed $\log_2 n$: the same picture as in Section 15.4. A film that 1% of subscribers rated carries $\log_2 100 = 6.64$ bits; three of them carry $3 \times 6.64 = 19.9$ bits, more than the $\log_2 480{,}189 = 18.9$ bits needed to single out one Netflix subscriber.

15.5.4. Interpretation

Sparsity is the vulnerability. In a sparse dataset most items are rated by few people, so each item a person rated is worth many bits, and nobody's record is close to anyone else's. Narayanan and Shmatikov formalize this as most records having no near neighbour. A census table has few columns with few values each; a ratings matrix has thousands of columns, and a person's "row" is a long sequence of rare events.

Because the score adds up evidence rather than demanding a perfect match, noise in the auxiliary information costs little. With a wrong item, the target loses one term of the score, while unrelated records rarely gain one. The score needs a margin, not a perfect match.

The eccentricity test answers a different question from the score. The top score is the best guess *if the target is in the release*. If it may not be, the adversary also needs to decide whether to trust the guess, and the eccentricity asks whether the top score stands out from the crowd of scores. How often it stands out by chance depends on the shape of the score distribution, so φ has to be calibrated on records known not to be the target. It should be read at a small false positive rate, as for any attack, because a match is only useful if the adversary can be confident in it (Hypothesis testing). Equation 15.11 gives the principled version. Add one more explanation, "the target is not in the release", with prior $1 - q$, and give each record prior q/n. Under the new explanation

every record is unrelated, so its likelihood, divided by $\prod_j P_0(a(j))$ as before, is 1. Bayes' rule then gives the posterior that record t is the target as $q\,\mathrm{LR}(t)/(q\sum_j \mathrm{LR}(j) + (1-q)n)$.

The weights matter less than the bits. Any weighting that favours rare items does about as well as any other; what decides success is whether the auxiliary information carries more bits than $\log_2 n$.

15.5.5. Example: a synthetic ratings matrix

The script draws a sparse matrix of 5,000 users and 4,000 items. Item popularity follows a Zipf law, the number of ratings per user is log-normal (median 30, mean 40.6), and ratings are 1 to 5 stars from a user bias, an item quality and noise. The result has 203,029 ratings and density 0.0102, close to Netflix's 0.0118. An item drawn from a random user's record carries a median of 5.83 bits, against $\log_2 5000 = 12.29$ bits needed. The auxiliary information about a target is m items from their record, with the rating off by one star half the time; in the noisy setting, a quarter of the items are replaced by random items drawn by popularity, which the target has almost never rated. Table 15.6 gives the fraction of 400 targets whose record gets the top score (ties broken at random), for three weightings. The first is equal weights. The second is the Narayanan–Shmatikov weights. The third is the likelihood-ratio weights of Equation 15.10 in their approximate form $\ln(1/s_i)$, which is close to the exact $\ln\frac{1-s_i}{s_i}$ because every item here is rated by a small fraction of users, with $s_i = |\mathrm{supp}(i)|/n$, the fraction of users who rated the item, and $\varepsilon = \frac{1}{2}$, which makes $\ln\frac{1-\varepsilon}{\varepsilon} = \ln 1 = 0$ and so drops the constant term.

Table 15.6.: Re-identification rate in a synthetic ratings matrix with 5,000 users: the fraction of 400 targets whose record gets the top score, for three weightings of Equation 15.8.

Auxiliary information	Equal weights	$1/\ln\lvert\mathrm{supp}(i)\rvert$	$\ln(n/\lvert\mathrm{supp}(i)\rvert)$
2 items	0.428	0.440	0.402
4 items	0.936	0.941	0.924
8 items	1.000	1.000	1.000
8 items, 2 of them wrong	0.943	0.989	0.978

Read the table with the bits in mind. Two items carry about $2 \times 5.83 = 11.7$ bits, a little under $\log_2 n$, and re-identify about 43% of targets. Four items carry about $4 \times 5.83 = 23$ bits and re-identify 93%. With 8 items of which 2 are wrong, the rarity weightings re-identify 98–99% of targets, matching the 99% of Narayanan and Shmatikov for 8 ratings with 2 wrong. Equal weights fall to 94% because a wrong popular item helps many unrelated records. The two rarity weightings differ by less than two standard errors; the standard error of a rate near 0.98 over 400 targets is $\sqrt{0.98 \times 0.02/400}$, which is 0.007 at these rates.

Figure 15.3 (left and centre) shows what the eccentricity measures. When the target is in the release, its record scores far above every other. When the target is not, the top score is just the high end of a long tail.

To test the eccentricity rule, the script also scores 400 outsiders, people drawn from the same model who are not in the release, so that any match it declares for them is false. With 8 items, 2

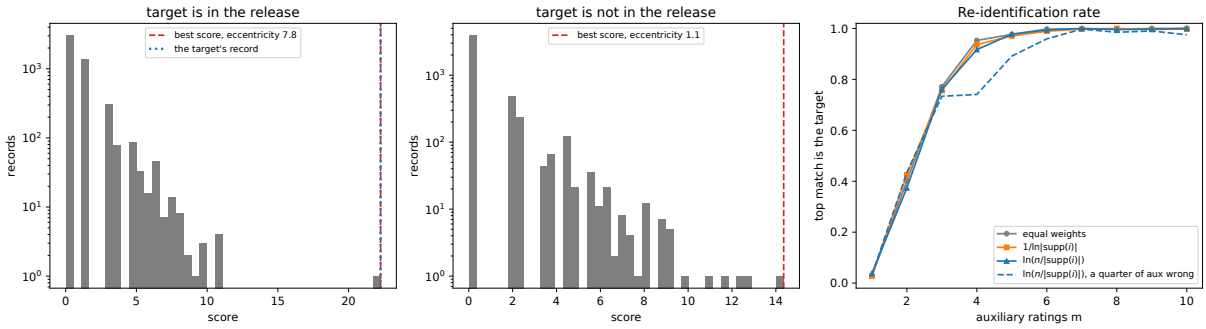


Figure 15.3.: Left and centre: the scores of all 5,000 records for one target's auxiliary information (8 items, 2 wrong), on a log scale. When the target is in the release its record stands far out; when it is not, the best score is the edge of the crowd. Right: re-identification rate against the number of auxiliary items, for three weightings, and for the likelihood-ratio weights when a quarter of the items, rounded down, are wrong.

of them wrong, and $\varphi = 1.5$, 94.8% of insiders are correctly re-identified, but 26.3% of outsiders are also "matched" to someone. The median eccentricity is 7.32 for insiders and 0.72 for outsiders, but the outsiders' distribution has a long tail. Holding the false-match rate to 1% means raising φ until only 1% of outsiders clear it, which happens at $\varphi = 4.83$, and that threshold still re-identifies 73.0% of insiders. Figure 15.4 shows the whole trade-off, the true positive rate (insiders matched) against the false positive rate (outsiders matched) as φ varies, with false positives on a log axis.

15.5.6. Check yourself

- Why do the likelihood-ratio weights in Equation 15.10 contain a term in ε that is the same for every item, and what does it do? *(It is the evidence from agreeing at all, independent of rarity. When the auxiliary information is reliable (ε small), every agreement counts a lot and a record that misses one item is heavily penalized relative to one that matches all of them.)*
- A record agrees with 3 auxiliary items, each rated by 1% of 480,189 users. How many unrelated records would you expect to agree with all three, if agreements were independent? *($480{,}189 \times 10^{-6} = 0.48$, since each unrelated record agrees on all three with probability $0.01^3 = 10^{-6}$: about half a false match, so three such items are close to, but not quite, enough.)*

15.6. Does releasing a sample protect?

15.6.1. Context

Statistical agencies and companies often release a sample: 1% of a census, 5% of the patients. The argument is plausible deniability. A record that is unique in the sample need not be unique in the population: the person you have in mind may share their attributes with someone who was not sampled, so a match is not proof. Rocher, Hendrickx & de Montjoye (2019) measured

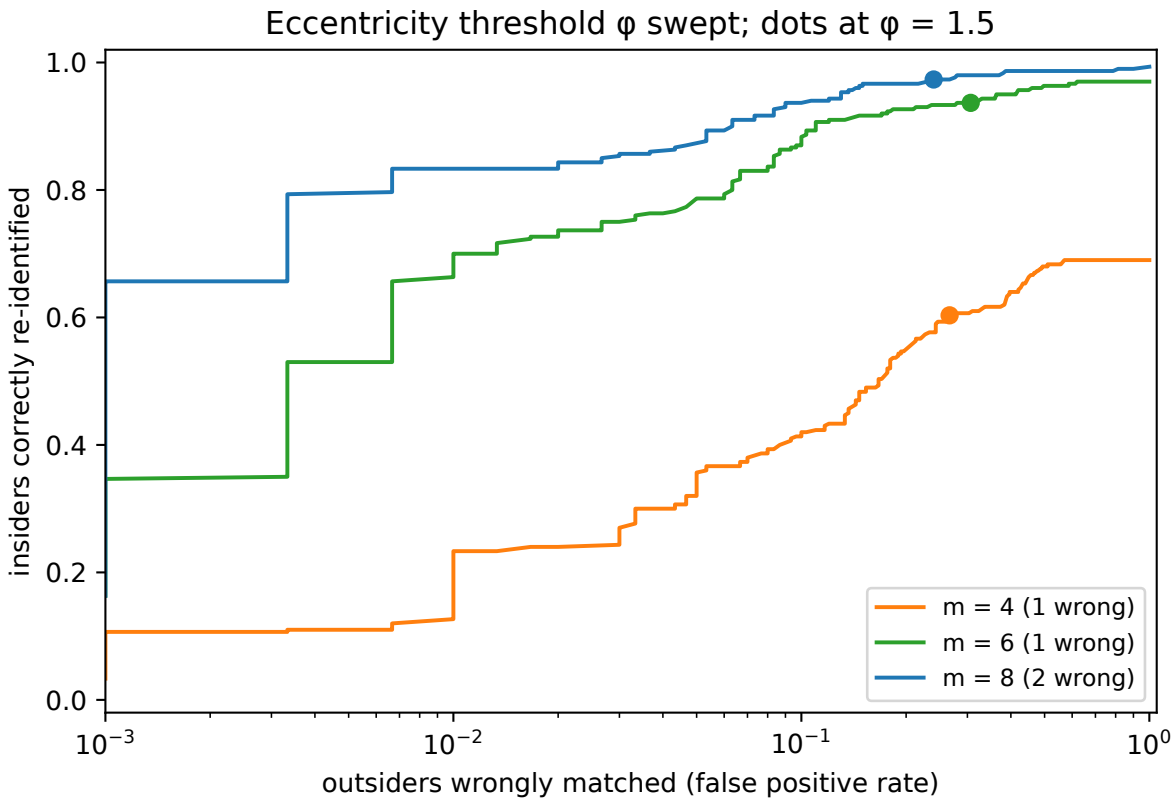


Figure 15.4.: True against false positive rates of the eccentricity test as the threshold φ is swept, for 4, 6 and 8 auxiliary items with 1, 1 and 2 of them wrong. The dots mark $\varphi = 1.5$. A false positive rate of 0 is drawn at the left edge.

how much this argument is worth. They fitted a Gaussian copula model (each attribute's own frequencies, tied together by the correlations of a multivariate normal) to a sample and used it to estimate, for each record, how likely it is to be unique in the population and so how likely a match is to be correct. Their model predicted that 99.98% of Americans would be correctly re-identified in any dataset with 15 demographic attributes, and that releasing a sample does little to change that. This section derives the two probabilities that matter and checks them.

15.6.2. Statement

Each of n people is in the release independently with probability π, the **sampling fraction**. Consider a cell with probability p, and let $\lambda = (n-1)p$ be the expected number of *other* people in it. There are two ways to attack, depending on which side the adversary starts from.

From a person to the release. The adversary knows a target's cell and finds exactly one record in that cell in the release. There are two ways that can happen. Either the target was sampled and none of the others in the cell were, and the match is right. Or the target was not sampled and exactly one of the others was, and the match is wrong. If the cell holds J others, the wrong story is J times as likely as the right one, because swapping who was sampled between the target and any one of the others leaves the probability unchanged. How many others should you expect? The release shows no second record in the cell, so the others who are there were all left out of the sample, and each of the $n-1$ others is in the cell and unsampled with probability about $p(1-\pi)$. So the cell is expected to hold about $\lambda(1-\pi)$ others, not λ, and the wrong story is about $\lambda(1-\pi)$ times as likely as the right one. A match is correct with probability "right over right plus wrong":

$$\kappa = \Pr[\text{correct} \mid \text{one match}] = \frac{1}{1 + \frac{(n-1)p(1-\pi)}{1-p\pi}} \approx \frac{1}{1+\lambda(1-\pi)}. \tag{15.13}$$

From a record to the population. The adversary takes a record that is alone in its cell in the release and looks the cell up in a register of the whole population, such as a voter roll. The record is identified if it is alone in its cell in the population too. Knowing the record is alone in the sample already tells you that nobody else in the cell was sampled. What remains is whether anyone in the cell was left out of the sample. Each of the $n-1$ others is in the cell but unsampled with probability about $p(1-\pi)$, so about $\lambda(1-\pi)$ people are expected to hide there, and the chance that nobody does is

$$\xi = \Pr[\text{population-unique} \mid \text{sample-unique}] = \left(\frac{1-p}{1-p\pi}\right)^{n-1} \approx e^{-\lambda(1-\pi)}. \tag{15.14}$$

The whole release. Averaged over the release, the fraction of sample-unique records that are unique in the population is the expected number of records that are unique in both, divided by the expected number of sample-unique records:

$$\frac{\sum_c p_c(1-p_c)^{n-1}}{\sum_c p_c(1-p_c\pi)^{n-1}}, \tag{15.15}$$

which equals the population uniqueness rate Equation 15.4 as $\pi \to 0$ and rises to 1 at $\pi = 1$.

15.6.3. Derivation

From a person (Equation 15.13). The plan is to compute the two stories, "right match" and "wrong match", given the number of others in the cell, and then average over that number.

1. **Name the crowd.** Let J be the number of other people in the target's cell. Each of the $n-1$ others is in it with probability p, independently, so $J \sim \text{Binomial}(n-1, p)$.
2. **The right match.** Given J, "exactly one match and it is the target" means the target is sampled and none of the J others are: probability $\pi(1-\pi)^J$.
3. **The wrong match.** Given J, "exactly one match and it is someone else" means the target is not sampled and exactly one of the J others is. There are J choices of which other, each sampled with probability π while the remaining $J-1$ are not: probability $(1-\pi) \cdot J\pi(1-\pi)^{J-1} = J\pi(1-\pi)^J$.
4. **Form the conditional probability.** The probability of a correct match given one match is the right story over the sum of both, each averaged over J. The common factor π cancels, leaving $\Pr[\text{correct} \mid \text{one match}] = \dfrac{\mathbb{E}[(1-\pi)^J]}{\mathbb{E}[(1+J)(1-\pi)^J]}$.
5. **Compute the first average.** The **probability generating function** of J is $\mathbb{E}[z^J]$. For a binomial it is $(1-p+pz)^{n-1}$, because z^J is a product of one factor per other person, z if they are in the cell and 1 if not, and each factor averages to $1-p+pz$ independently. At $z = 1-\pi$: $\mathbb{E}[(1-\pi)^J] = (1-p\pi)^{n-1}$.
6. **Compute the second average.** Differentiating z^J in z and multiplying by z gives Jz^J, so $\mathbb{E}[Jz^J] = z\frac{d}{dz}(1-p+pz)^{n-1} = (n-1)pz(1-p+pz)^{n-2}$. At $z = 1-\pi$: $\mathbb{E}[J(1-\pi)^J] = (n-1)p(1-\pi)(1-p\pi)^{n-2}$.
7. **Simplify.** Divide numerator and denominator of step 4 by $(1-p\pi)^{n-1}$ to get Equation 15.13. For small p, $1-p\pi \approx 1$ and $(n-1)p = \lambda$, which gives the approximation.

From a record (Equation 15.14).

1. **Sort the others into three groups.** Each other person is, independently, in the cell and sampled (probability $p\pi$), in the cell and not sampled ($p(1-\pi)$), or outside the cell ($1-p$).
2. **Condition on sample-uniqueness.** "The record is sample-unique" says that no other person is in the first group. Because each person's group is independent of the others', this condition acts on each person separately: it rules out the first group and rescales the other two by $1-p\pi$, their total probability. So, given it, each other person is outside the cell with probability $\frac{1-p}{1-p\pi}$, still independently.
3. **Ask for population-uniqueness.** Population-unique means all $n-1$ others are outside the cell: probability $\left(\frac{1-p}{1-p\pi}\right)^{n-1}$. The ratio $\frac{1-p}{1-p\pi}$ is $1 - \frac{p(1-\pi)}{1-p\pi}$, close to $1-p(1-\pi)$ for small p, so by Fact 2 the whole power is about $e^{-(n-1)(p-p\pi)} = e^{-\lambda(1-\pi)}$.

The whole release (Equation 15.15).

1. **Count the records unique in both.** By Fact 1, the expected number of records that are sample-unique and population-unique is $\sum_i \Pr[i \text{ sampled and population-unique}]$. Population-unique implies sample-unique, and sampling is independent of the cells, so each term is π times the probability of being population-unique, which is the per-person term of Equation 15.4. The total is $n\pi \sum_c p_c(1-p_c)^{n-1}$.

2. **Count the sample-unique records.** Their expected number is $n\pi \sum_c p_c(1-p_c\pi)^{n-1}$: person i is sampled, and each other person is either outside i's cell or not sampled, which fails only with probability $p_c\pi$.
3. **Divide.** The factor $n\pi$ cancels. Each cell's term in the numerator is its term in the denominator times $\left(\frac{1-p_c}{1-p_c\pi}\right)^{n-1}$, its ξ from Equation 15.14, so the ratio is an average of ξ over cells weighted by how often each is sample-unique. As $\pi \to 0$ the denominator tends to $\sum_c p_c = 1$, leaving the population uniqueness rate. At $\pi = 1$ the two sums are equal and the ratio is 1.

15.6.4. Interpretation

Sampling only helps when the cell is crowded. In both formulas, sampling enters only through the factor $1-\pi$, which multiplies λ. For rare combinations, $\lambda \ll 1$, both are close to 1 whatever π is. Because $1-\pi \le 1$, both are at least their values at $\pi = 0$: $\kappa \ge 1/(1+\lambda)$ and $\xi \ge e^{-\lambda}$ for every sampling fraction. Below a few percent, shrinking the sample further barely changes anything, because $1-\pi$ is already close to 1.

Matching from a person is more forgiving than matching from a record: $\kappa \ge \xi$, since $1/(1+x) \ge e^{-x}$ (which is $e^x \ge 1+x$, the exponential lying above its tangent line at 0). A person-to-release match can be right even when the target is not unique in the population: the others in the cell may simply not have been sampled.

At small sampling fractions, a sample unique is a population unique at the population rate. By Equation 15.15, a 1% sample gives plausible deniability exactly to the people who were not unique in the population anyway. With enough attributes, that is almost nobody.

Finally, the adversary can tell which matches to trust. λ depends on the cell, and the sample is a good sample of the population, so the adversary can estimate p from it and compute κ or ξ for each record. Rocher et al. showed this with a copula model; the script does it with the product of the marginals estimated from a 1% sample.

15.6.5. Example: a 1% sample of a synthetic census

For a single cell, Table 15.7 gives both probabilities in their Poisson forms, the approximations on the right of Equation 15.13 and Equation 15.14, for three values of λ. For instance, with $\lambda = 0.1$ and $\pi = 0.01$, $\kappa = 1/(1+0.1\times 0.99) = 1/1.099 = 0.910$.

Table 15.7.: Probability that a match in a sample is correct: from a person to the release (κ) and from a sample-unique record to the population (ξ), for three crowding levels and two sampling fractions.

λ (expected others in the cell)	κ, $\pi = 0.01$	κ, $\pi = 0.5$	ξ, $\pi = 0.01$	ξ, $\pi = 0.5$
0.1	0.910	0.952	0.906	0.951
0.5	0.669	0.800	0.610	0.779
2	0.336	0.500	0.138	0.368

Going from a 50% to a 1% sample lowers κ by $0.952 - 0.910 = 0.04$ for a rare cell and by $0.500 - 0.336 = 0.16$ for a crowded one.

The script then draws a population of $n = 100{,}000$ with the first seven attributes of Table 15.5; 82.8% of its people are unique. Equation 15.15 gives 0.8301 of sample uniques being population-unique at $\pi = 0.01$ (simulated: 0.8334 over 19,951 sample uniques in 20 samples), 0.8493 at $\pi = 0.1$ and 0.9235 at $\pi = 0.5$. As the formula predicts, the 1% value sits just above the population rate of 0.828. A 1% sample leaves 83% of its unique records identifiable against a population register.

The adversary's own estimate works too. From one 1% sample with 993 sample-unique records, it estimates each attribute's frequencies, multiplies them to get an estimate $\hat{p}$ of each record's cell probability, and computes ξ by Equation 15.14 with n and π, which are public. Its mean prediction is 0.817 against an actual 0.824. The predictions also sort the records well. The 415 records it rates above 0.95 average a prediction of 0.980 and are population-unique 99.3% of the time; the 119 it rates below 0.5 average 0.284 and are unique 30.3% of the time. Figure 15.5 shows both experiments.

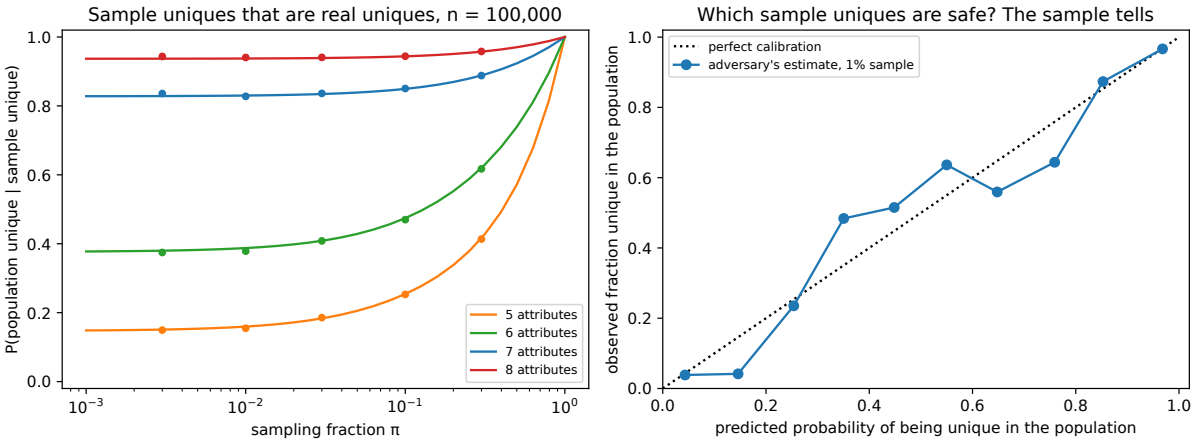


Figure 15.5.: Left: the fraction of sample-unique records that are unique in a population of 100,000, against the sampling fraction, for 5 to 8 attributes; lines are Equation 15.15, dots are simulations. As $\pi \to 0$ each curve levels off at the population's uniqueness rate. Right: the adversary's predicted probability that a sample-unique record is unique in the population, estimated from a 1% sample, against the observed fraction, in bins of width 0.1.

15.6.6. Check yourself

- A combination is expected to be shared by $\lambda = 0.05$ other people. What is κ for a 1% sample, and for a full release? *($1/(1 + 0.05 \times 0.99) = 0.953$ and $1/(1 + 0) = 1$.)*
- Why does Equation 15.15 tend to the population uniqueness rate, not to 0, as $\pi \to 0$? *(A tiny sample has almost no collisions, so nearly every sampled record is sample-unique. The fraction of them that are population-unique is then just the fraction of the population that is unique.)*

15.7. Code: check every formula by simulation

The script code/linkage.py computes every number in this chapter and draws every figure. The uniqueness formulas are computed by summing over all cells (grouped by probability, so even 59 million cells are summed quickly) and checked against simulated populations. The ratings experiment is a scaled-down Netflix: a NumPy matrix of 5,000 users, scored with Equation 15.8. From the repository root:

```
uv run python code/part02-attacks/linkage.py
```

Output (seed 0), summarized in Table 15.8:

Table 15.8.: The script's output with the default seed.

Experiment	Result
Bits	$\log_2(8 \times 10^9) = 32.9$; $\log_2(248.7 \times 10^6) = 27.9$; ZIP 15.3 + birth date 14.8 + sex 1.0 = 31.1 bits
Sweeney, uniform cells	$M = 2.34 \times 10^9$; $n/M = 0.106$; fraction unique 0.899
World, uniform cells	33 bits: fraction unique 0.394. Bits needed for 50%, 90%, 99%: 33.4, 36.1, 39.5
Toy release	Classes of sizes 1, 1, 1, 2, 3: linkage success 0.625
Synthetic census, $n = 10^4$	Table 15.5; McDiarmid 95% half-width 0.0272
Synthetic ratings, 5,000 users	203,029 ratings, density 0.0102; median bits per item 5.83; rates in Table 15.6
Eccentricity, 8 items, 2 wrong	$\varphi = 1.5$: insiders 0.948, outsiders 0.263. $\varphi = 4.83$: insiders 0.730, outsiders 0.010
Sampling, $n = 10^5$, 7 attributes	Population unique 0.828. Sample uniques that are unique: 0.8301 ($\pi = 0.01$; simulated 0.8334), 0.8493 ($\pi = 0.1$), 0.9235 ($\pi = 0.5$)
κ and ξ	Table 15.7
Adversary's estimate of ξ, 1% sample	Mean predicted 0.817, actual 0.824; above 0.95: predicted 0.980, actual 0.993; below 0.5: predicted 0.284, actual 0.303

tests/test_linkage.py checks that the output still matches.

What to take from it. First, the formulas are right: every simulated fraction agrees with its exact expectation to within 0.004. Second, the bits beyond $\log_2 n$ decide uniqueness. A few bits short, a minority is unique; seven bits beyond, almost everyone is, in a census table and in a ratings matrix alike. Third, the shortcuts err in known directions. For real, skewed attributes the uniform model usually overstates uniqueness (in the census, at every step from four attributes on) and the collision bound understates it, so compute the exact sum; it needs only a model of the population. Fourth, the decision rule needs calibration. Picking the top score is nearly always

right when the target is in the data, but the popular threshold $\varphi = 1.5$ matches a quarter of outsiders in this toy, so choose φ from the false positive rate you can accept. Last, sampling is not a defence. It protects the people in crowded cells, who were not at much risk, and the adversary can tell who they are.

15.8. References

- Sweeney, "Simple Demographics Often Identify People Uniquely" (Carnegie Mellon University, Data Privacy Working Paper 3, 2000): the 87% estimate for {ZIP code, birth date, sex} and its variants with coarser geography. Sweeney, "k-Anonymity: A Model for Protecting Privacy" (International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 2002) tells the Massachusetts story in its introduction.
- Golle, "Revisiting the Uniqueness of Simple Demographics in the US Population" (Workshop on Privacy in the Electronic Society, 2006): the same estimate from the 2000 census, and how it depends on the granularity of the attributes.
- Narayanan & Shmatikov, "Robust De-anonymization of Large Sparse Datasets" (IEEE Symposium on Security and Privacy, 2008): the scoring algorithm, the eccentricity test, the formal notion of sparsity, and the Netflix Prize experiments.
- Rocher, Hendrickx & de Montjoye, "Estimating the success of re-identifications in incomplete datasets using generative models" (Nature Communications, 2019): the Gaussian copula model, per-record correctness, and why sampling and incompleteness do not protect.
- de Montjoye, Hidalgo, Verleysen & Blondel, "Unique in the Crowd: The privacy bounds of human mobility" (Scientific Reports, 2013): four spatio-temporal points identify 95% of 1.5 million mobile phone users, the same sparsity argument for location traces.
- Ohm, "Broken Promises of Privacy: Responding to the Surprising Failure of Anonymization" (UCLA Law Review, 2010): the legal side, with the Massachusetts, search-log and Netflix cases.
- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (2006): where the response to these attacks starts. The next part introduces differential privacy, a guarantee that holds whatever auxiliary information the adversary has. The next chapter first looks at the definitions that came before it.

15.9. Cheat sheet

n people, cells c with probabilities p_c, M cells, U the number of unique people, π the sampling fraction, λ the expected number of others in the target's cell. Table 15.9 collects the results.

Table 15.9.: Every result in this chapter.

Result	Formula	Note
Linkage success	J/n, J = number of distinct quasi-identifier combinations	Target known to be in the release; at most $1/k$ if all classes have $\geq k$ records
Expected fraction unique	$\sum_c p_c(1-p_c)^{n-1}$	Exact for independent people

Result	Formula	Note
Uniform cells	$(1-1/M)^{n-1} \approx \exp(-2^{\log_2 n - b})$	50% at $b = \log_2 n + 0.53$; 99% at $\log_2 n + 6.64$
Collision bound	$\geq (1 - \sum_c p_c^2)^{n-1}$; $\sum_c p_c^2 = \prod_j \sum_v p_j(v)^2$	Jensen; tight for uniform cells
Concentration	$\Pr[\lvert U/n - \mathbb{E}U/n\rvert \geq t] \leq 2e^{-nt^2/2}$	McDiarmid with $c_i = 2/n$
Netflix score	$\sum_{i \in \text{aux}} w_i \operatorname{Sim}(\text{aux}_i, r_i)$	$w_i \approx \ln(1/s_i) + \ln \frac{1-\varepsilon}{\varepsilon}$ is the log-likelihood ratio
Eccentricity	$(s_1 - s_2)/\sigma \geq \varphi$	Calibrate φ on outsiders at a small false positive rate
Expected false matches	$n \prod_{i \in I} s_i = 2^{\log_2 n - \sum_i \log_2(1/s_i)}$	Each item is worth its surprisal in bits
Correct match, person to release	$\kappa \approx 1/(1 + \lambda(1-\pi))$	$\geq 1/(1+\lambda)$ for every π
Sample-unique is population-unique	$\xi \approx e^{-\lambda(1-\pi)}$	$\geq e^{-\lambda}$ for every π
Sample uniques that are real	$\sum_c p_c(1-p_c)^{n-1} / \sum_c p_c(1 - p_c\pi)^{n-1}$	Population uniqueness rate as $\pi \to 0$

Rules of thumb

- $\log_2 n$ bits name one person: 33 for the world, 28 for the US, 13 for a town of 10,000. About 7 bits more make 99% of people unique.
- Add up the bits of the attributes an adversary might know. If the sum exceeds $\log_2 n$, assume most people are unique; if attributes are correlated, the true sum is lower.
- In sparse data every rare item is worth $\log_2(1/s_i)$ bits, so a handful of facts is enough, noise or not.
- Removing names, releasing a sample, or perturbing a few values does not anonymize a dataset. The defence has to hold whatever auxiliary data exist. The next chapter tests the classic attempts, k-anonymity and its successors, against that requirement.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

16. Syntactic anonymity and how it breaks

16.1. Why this matters

Linkage and re-identification showed that deleting names does not anonymize a table. The columns that remain, such as age, ZIP code and sex, are **quasi-identifiers**: none of them names anyone, but together they single out most people, and anyone who holds a list of names with the same columns (a voter roll, say) can join the two tables and put a name back on most rows. Between 1998 and 2008, data custodians and researchers answered with a family of definitions that constrain the released table itself. **k-anonymity** hides every record in a crowd of at least k that look alike. **l-diversity** asks each crowd to hold several different sensitive values. **t-closeness** asks each crowd to look like the whole table. They are called **syntactic** because each one is a property of the published table that you can check by reading it, with no reference to how the table was produced or to what else the adversary knows.

These definitions are not history. Statistics offices, hospital data-sharing agreements, regulators' guidance and anonymization tools such as ARX still use them. You need to know what they promise and exactly where the promise ends, for two reasons. You will meet them in practice, and their failures are the specification of the definition the next part introduces.

The definitions form a chain, each proposed in answer to an attack on the one before it. Removing names falls to **linkage**, and k-anonymity caps linkage at $1/k$ (Section 16.3). But k-anonymity only stops the adversary from picking out the target's record; it does not stop the adversary from learning the target's diagnosis. If every record in the target's crowd has the same diagnosis (**homogeneity**), or if the adversary can rule out the other diagnoses from what it already knows about the target (**background knowledge**), the diagnosis leaks with no record picked out. l-diversity answers these two attacks (Section 16.4). A crowd can be diverse and still move the adversary's belief a long way, though: half the crowd may have a disease that is rare in the population (**skewness**), or all its different diagnoses may be diseases of the same organ (**similarity**). t-closeness answers these (Section 16.5). Finally, **composition** defeats all three at once: two releases, each satisfying the definition, intersect to reveal what neither reveals alone (Section 16.6).

16.1.1. The one idea: a syntactic definition fixes the adversary's knowledge

Each definition is a property of one table, and it protects against an adversary whose knowledge is fixed in advance. For k-anonymity the adversary knows the quasi-identifiers and nothing else. For l-diversity it knows the quasi-identifiers plus a few facts about the target. For t-closeness it knows the quasi-identifiers plus the table's overall distribution. Each attack in this chapter gives the adversary one more piece of knowledge that the definition did not anticipate, and the guarantee then says nothing. A second release, the most ordinary auxiliary information there is,

breaks all three definitions. A definition that survives has to hold whatever the adversary knows, and has to say what happens when releases are combined.

Table 16.1 lists the results and where each is used.

Table 16.1.: The results of this chapter and where each is used.

Result	Where it is used
The posterior within a class is the class's frequency	Every attack on the sensitive attribute
k-anonymity caps linkage success at $1/k$	What k-anonymity does guarantee
Uniform cells: a fraction $\Pr[\text{Poisson}(n/M) \leq k-2]$ violates k-anonymity	How coarse the quasi-identifiers must be, in bits
Recursive (c, ℓ)-diversity caps the posterior at $c/(1+c)$ after $\ell - 2$ values are ruled out	What l-diversity guarantees against background knowledge
t-closeness in total variation moves every event by at most t	What t-closeness guarantees, and why rare values escape it
Probability that two releases pin down a value	The composition attack

What this chapter uses from elsewhere. Every idea below is restated where it is used, and these links are for more detail. The linkage game, Equation 15.3, and the uniform-cells model are in Linkage and re-identification. The imputation baseline, the best guess of a sensitive attribute from the population alone, is in Attribute inference and model inversion. Total variation is in Divergences, the Poisson approximation to the binomial in Probability distributions, and entropy in Information theory.

Notation. A table has n records. Each has quasi-identifiers and one **sensitive attribute** S, such as a diagnosis, with m possible values. A **generalization** replaces each quasi-identifier by a coarser value: an age by a range, a ZIP code by a prefix. An **equivalence class** E is a set of records with the same generalized quasi-identifiers; $|E|$ is its number of records. Within a class, $n_E(v)$ is the number of records with $S = v$, and $P_E(v) = n_E(v)/|E|$ is the fraction of the class with that value, the class's distribution. $P(v)$ is the same fraction over the whole table. **Suppression** deletes records.

16.2. Toolkit: three facts every derivation uses

16.2.1. Fact 1 — Within a class, the posterior is the frequency

The adversary's **posterior** is its belief about the target's sensitive value after it has seen the release. Here is the situation. The adversary knows the target's quasi-identifiers and knows the target is in the release, so it can find the target's class E. But all records in E show the same generalized quasi-identifiers, so it cannot tell which one is the target's. If it knows nothing else, it

has no reason to favour any record, and each of the $|E|$ records is equally likely to be the target's. (This is step 2 of the derivation of Equation 15.3: the target was drawn at random, and the records of a class differ only in the sensitive attribute, which the adversary does not know.) The chance that the target has value v is then the number of records with v divided by the number of records:

$$\Pr[S_{\text{target}} = v \mid \text{release}] = \frac{n_E(v)}{|E|} = P_E(v). \tag{16.1}$$

Background knowledge works by elimination. If the adversary also knows that the target's value is not in some set B of values, the records with values in B cannot be the target's. It crosses them out and spreads its belief evenly over the records that are left. The number left is $|E|$ minus the records with values in B, so for a value v not in B,

$$\Pr[S_{\text{target}} = v \mid \text{release}, S_{\text{target}} \notin B] = \frac{n_E(v)}{|E| - \sum_{u \in B} n_E(u)}. \tag{16.2}$$

For example, take a class of three records with the diagnoses cancer, cancer, flu. Two of the three records say cancer, so the adversary's posterior is $\frac{2}{3}$ for cancer and $\frac{1}{3}$ for flu. If it knows that the target did not have flu, the flu record is crossed out, two records remain and both say cancer, so cancer has posterior $\frac{2}{2} = 1$. Every attack on the sensitive attribute in this chapter is this calculation.

16.2.2. Fact 2 — Total variation bounds every event

To say that a class "looks like the whole table" you need a distance between two distributions P and Q on the same values. The **total variation (TV)** distance is half the total absolute difference between the two probability tables, and it equals the largest difference in probability that any single question about the value can show:

$$\text{TV}(P, Q) = \frac{1}{2} \sum_{v} |P(v) - Q(v)| = \max_{A} |P(A) - Q(A)|, \tag{16.3}$$

where the maximum is over all sets of values A, and $P(A)$ is the probability that the value lands in A (Divergences).

The equality has a short reason. $P(A) - Q(A) = \sum_{v \in A} (P(v) - Q(v))$, which is largest when A holds exactly the values where $P(v) > Q(v)$, so the maximum is attained by $A = \{v : P(v) > Q(v)\}$ and equals the total excess of P over Q. Both distributions sum to 1, so the total excess of P over Q equals the total excess of Q over P, and each is half the sum of all absolute differences. (Taking A to be the values where Q is larger gives the same number with the sign flipped, which is why the absolute value does no harm.) This is what t-closeness relies on: a bound on the distance is a bound on every question the adversary can ask.

16.2.3. Fact 3 — The number of others in your cell is nearly Poisson

Fact 3 is the tool for counting how many people share your quasi-identifiers. Model the generalized quasi-identifiers as M cells, and suppose the n people fall into them independently, each cell equally likely. Fix yourself. Each of the other $n - 1$ people lands in your cell with probability

$1/M$, independently, so the number of *other* people in your cell is Binomial$(n-1, 1/M)$. When there are many trials, each with a small chance, the binomial is close to the **Poisson** distribution with the same mean, $\Pr[N = j] \approx e^{-\lambda}\lambda^j/j!$ (Probability distributions). Here the mean is $\lambda = (n-1)/M \approx n/M$, the average number of people per cell. Your class then has $1 + \text{Poisson}(\lambda)$ members: you plus the others. This is how the chapter counts how many records violate k-anonymity at a given coarseness.

16.3. k-anonymity

16.3.1. Context

Sweeney and Samarati proposed the first answer to the linkage attack in 1998: make sure that no combination of quasi-identifiers in the release singles out fewer than k people. A custodian gets there by generalizing (age 37 becomes 35–39, ZIP code 02139 becomes 021**) and by suppressing the few records that still stand out. The question is what this buys, and what it costs.

16.3.2. Statement

Definition. A release is **k-anonymous** if every equivalence class has at least k records.

What it guarantees. Recall the linkage game: a target is drawn uniformly from the release, the adversary is given the target's quasi-identifiers, and it wins if it picks out the target's record. Equation 15.3 says the best it can do is J/n, where J is the number of classes. The reason is that in a class of size $|E|$ the best guess is a random record of the class, right with probability $1/|E|$, and the target lands in that class with probability $|E|/n$, so each class contributes $1/n$ and the J classes contribute J/n. When every class has at least k records there cannot be many classes, and

$$\Pr[\text{win}] = \frac{J}{n} \le \frac{1}{k}. \tag{16.4}$$

What it costs, for uniform cells. Coarse quasi-identifiers protect more but say less, so the question is how coarse they must be. Suppose the generalized quasi-identifiers split the population into M equally likely cells. A record violates k-anonymity when its cell holds fewer than k people, and by Fact 3 the expected fraction of such records is

$$\Pr[\text{Poisson}(n/M) \le k-2]. \tag{16.5}$$

Keeping that fraction below a target α needs enough people per cell, $n/M \ge \lambda_k$, where λ_k is the mean at which the fraction is exactly α: $\Pr[\text{Poisson}(\lambda_k) \le k-2] = \alpha$. A quasi-identifier with M equally likely values carries $\log_2 M$ bits of information about a person, so in bits the quasi-identifiers may carry at most $b = \log_2 M = \log_2 n - \log_2 \lambda_k$.

16.3.3. Derivation

The cap on linkage (Equation 16.4). The idea is that n records cannot fill more than n/k classes of at least k each.

1. **Count records by class.** The J classes split the n records between them, each record in exactly one class, so $n = \sum_j |E_j|$.
2. **Use the definition.** Every class has at least k records, so the sum is at least Jk: $n \geq Jk$, that is $J/n \leq 1/k$.
3. **Read off the linkage success.** Equation 15.3 says the linkage success is J/n, so it is at most $1/k$.

The cost (Equation 16.5). The idea is to compute the chance that one person's class is too small, then note that this chance is also the expected fraction of such people.

1. **Size of a person's class.** A person's class is their cell. By Fact 3 it holds the person plus N others, with N approximately Poisson(n/M).
2. **When it is too small.** The class is smaller than k when $1 + N \leq k - 1$, that is $N \leq k - 2$, which has probability $\Pr[\text{Poisson}(n/M) \leq k - 2]$.
3. **From one person to the fraction.** The expected number of people in small classes is the sum, over people, of each one's chance of being in one (linearity of expectation: the expected count of events is the sum of their probabilities). Every person has the same chance, so the sum is n times it, and the expected fraction is that chance.
4. **Solve for the coarseness.** As λ grows, the Poisson count moves up and the chance of a small count, the CDF (cumulative distribution function) at $k - 2$, falls. So "at most α" is the same as $\lambda \geq \lambda_k$. Put $M = n/\lambda$ and take $\log_2$: $\log_2 M = \log_2 n - \log_2 \lambda$, largest at $\lambda = \lambda_k$.

16.3.4. Interpretation

The guarantee is about names only. Equation 16.4 bounds the chance of picking out the target's *record*. It says nothing about the sensitive attribute, which is the whole point of the release, and the next two sections attack exactly that gap.

k-anonymity is the mirror image of uniqueness, and the bits make the comparison exact. In the same uniform-cells model, a person is unique when nobody else is in their cell, which by Fact 3 has probability $\Pr[\text{Poisson}(n/M) = 0] = e^{-n/M}$. For 99% of people to be unique you need $e^{-n/M} = 0.99$, so $n/M = \ln(1/0.99) \approx 0.01$ and $M \approx 100n$, which is $\log_2 n + \log_2 100 = \log_2 n + 6.64$ bits (Section 15.4). At the other end, Table 16.2 shows that 2-anonymity for 99% of records needs the quasi-identifiers to carry at most $\log_2 n - 2.20$ bits. The window between "almost everyone is unique" and "almost everyone is hidden" is $6.64 + 2.20 = 8.84$, about 9 bits wide, and every attribute you publish uses some of it.

k costs more than $\log_2 k$ bits. You might expect that hiding everyone in a class of k needs k people per cell, that is $\log_2 k$ bits fewer than $\log_2 n$. But cells vary in size around their mean, and the unlucky small cells are the ones that violate the definition, so you need λ_k people per cell *on average*, and λ_k is larger than k. The margin is largest for small k. For $k = 2$ a person violates the definition when $N = 0$, which has probability $e^{-\lambda}$; setting $e^{-\lambda} = 0.01$ gives $\lambda_2 = \ln 100 = 4.61$, more than twice k.

Which columns are quasi-identifiers is a judgment, and in sparse data everything is one. Section 15.5 showed that a handful of rare movie ratings single out a subscriber. Making a ratings matrix k-anonymous means generalizing thousands of columns until little is left (Aggarwal, 2005).

Finally, the guarantee assumes the adversary knows no more than the quasi-identifiers. A neighbour who knows that the target's record is the only one in the class with a given diagnosis needs no linkage at all.

16.3.5. Example: a k-anonymous hospital table

Table 16.2 applies Equation 16.5 to a population of $n = 10{,}000$ ($\log_2 n = 13.29$ bits) with $\alpha = 1\%$. For each k it finds λ_k, then subtracts $\log_2 \lambda_k$ from $\log_2 n$. For $k = 2$, $\lambda_2 = 4.61$ from the calculation above, $\log_2 4.61 = 2.20$, and $13.29 - 2.20 = 11.08$ bits are allowed.

Table 16.2.: Uniform cells, $n = 10{,}000$: how many bits of quasi-identifier k-anonymity allows if at most 1% of records may fall in classes smaller than k.

k	People per cell λ_k	Bits allowed	Below $\log_2 n$ by	$\log_2 k$
2	4.61	11.08	2.20	1.00
5	10.05	9.96	3.33	2.32
10	17.40	9.17	4.12	3.32
20	30.58	8.35	4.93	4.32
50	66.74	7.23	6.06	5.64

The last two columns compare the true cost with the naive $\log_2 k$. The gap is about 1.2 bits at $k = 2$ ($2.20 - 1.00$) and shrinks to about 0.4 bits at $k = 50$ ($6.06 - 5.64$), because a Poisson count with a large mean varies little relative to its mean. Figure 16.1 draws the whole curve for each k.

Real tables are not uniform, so real anonymizers search for a generalization that works on the data. The script builds a synthetic hospital table of $n = 10{,}000$ patients with three quasi-identifiers: age in years (0 to 89), a five-digit ZIP code (800 values, a few areas far more common than the rest) and sex. The sensitive attribute is one of eight diagnoses, and it depends on age: asthma peaks in childhood, fractures around 18, depression around 40, diabetes, hypertension and cancer later in life, while flu is equally common at every age and HIV is rare (1.0% of records). With exact quasi-identifiers, the linkage success is 0.933: by Equation 15.3, the 10,000 patients fall into $0.933 \times 10{,}000 = 9{,}330$ distinct combinations of age, ZIP code and sex, so most of them are alone in theirs.

The script anonymizes the table with **Mondrian** (LeFevre, DeWitt & Ramakrishnan, 2006). Start with the whole table as one box. Split a box at the median of the quasi-identifier with the widest range (relative to its full range), as long as both halves still have at least k records. Stop when no split is allowed. Each final box is a class, published as a range per quasi-identifier. Mondrian recodes locally: a busy ZIP area gets narrow age ranges and a quiet one wide ranges. Because it keeps splitting until a split would leave fewer than k on one side, its classes stay close to size k, much closer than those of a single global generalization of each column. Table 16.3 gives the result.

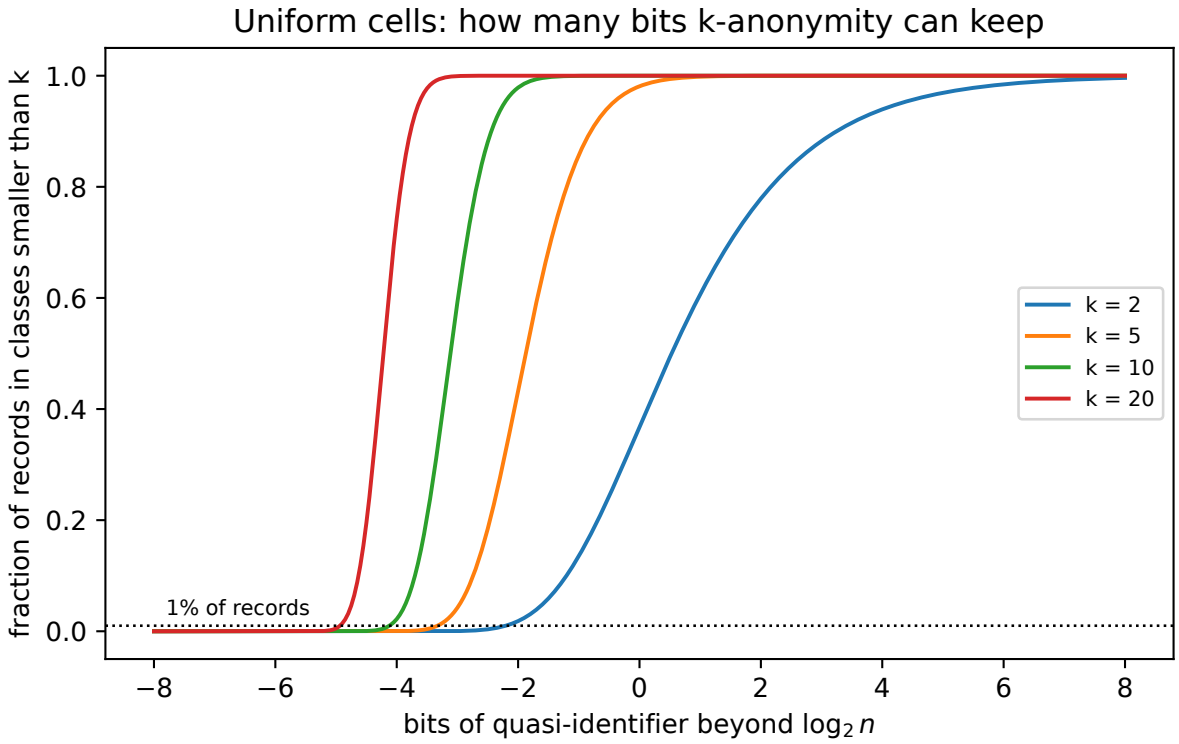


Figure 16.1.: The fraction of records in classes smaller than k, against the bits of quasi-identifier beyond $\log_2 n$, for uniform cells. Each curve is a Poisson CDF; the dotted line is 1% of records.

Table 16.3.: Mondrian k-anonymization of a synthetic hospital table of 10,000 patients. Mean width is the average, over records and quasi-identifiers, of the class's range as a fraction of the full range. Certain disclosure is the fraction of records in a class with a single diagnosis. Guess accuracy is the fraction of records whose diagnosis is the most common one in their class.

k	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
2	4,091	2–5	0.005	0.4091	0.2797	0.675
5	1,542	5–11	0.016	0.1542	0.0232	0.593
10	748	10–20	0.028	0.0748	0.0000	0.562
20	366	20–41	0.043	0.0366	0.0000	0.540
50	129	50–99	0.075	0.0129	0.0000	0.506

The linkage success sits between $1/(2k)$ and $1/k$ for every k (Figure 16.2, left). J/n is one over the average class size. No class is smaller than k, so the upper bound $1/k$ is guaranteed. The lower bound is not: Mondrian can leave classes larger than $2k$ (up to 41 records at $k = 20$), but they are rare enough that the average size stays below $2k$ in every row. At $k = 2$, for instance, 0.4091 lies between 0.25 and 0.5. The cost in precision is small. At $k = 5$ the average range covers 1.6% of each quasi-identifier's span.

The last two columns are about the diagnosis, and the next section explains them.

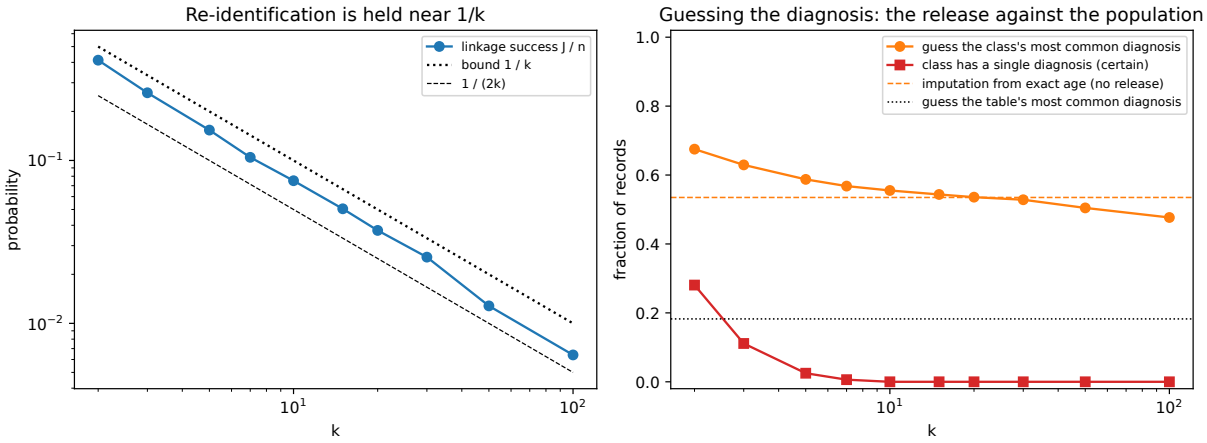


Figure 16.2.: Left: the linkage success of Mondrian releases of the synthetic hospital table against k, between $1/(2k)$ and the bound $1/k$. Right: how often the diagnosis is the most common one in the record's class, and how often the class holds a single diagnosis, against two baselines that use no release.

16.3.6. Check yourself

- A k-anonymous release of 1,000 records has 120 classes. What is the largest k it can satisfy, and what is the linkage success? *(At most* $\lfloor 1000/120 \rfloor = 8$*, since* $Jk \le n$*. The success is* $120/1000 = 0.12$*, below* $1/8 = 0.125$*.)*

- Why does 2-anonymity need about 4.6 people per cell on average, not 2? *(The number of others in your cell is roughly Poisson. With a mean of 1, a fraction $e^{-1} = 0.37$ of people would have nobody else in their cell. To get that down to 1% you need $e^{-\lambda} = 0.01$, that is $\lambda = 4.61$.)*

16.4. Homogeneity, background knowledge and l-diversity

16.4.1. Context

Machanavajjhala, Kifer, Gehrke & Venkitasubramaniam (2006) pointed out that k-anonymity hides who a record belongs to but not what it says. Table 16.4 is a 3-anonymous release.

Table 16.4.: A 3-anonymous release with three classes of three records.

ZIP code	Age	Diagnosis
130**	< 30	heart disease
130**	< 30	heart disease
130**	< 30	heart disease
148**	4*	cancer
148**	4*	flu
148**	4*	heart disease
130**	3*	cancer
130**	3*	cancer
130**	3*	flu

Alice is 26 and lives in ZIP code 13053. Her class is the first, and every record in it says heart disease. By Fact 1 her posterior for heart disease is the fraction of her class that has it, $\frac{3}{3} = 1$. This is the **homogeneity attack**: no re-identification, full disclosure. Bob is 35, lives in 13068, and is in the third class, where two of three records say cancer, so his posterior for cancer is $\frac{2}{3}$. His neighbour saw him at work through the whole flu season and rules out flu; crossing out the flu record leaves two cancer records, and cancer then has posterior 1. This is the **background knowledge attack**. The question is what condition on the classes stops both.

16.4.2. Statement

Both attacks work because one value dominates the class, either from the start or after a few values are ruled out. l-diversity asks for enough different values in each class, and its three versions differ in how they measure "enough". Sort a class's value counts from largest to smallest as $r_1 \geq r_2 \geq ... \geq r_m$, so r_1 is the count of the most common value. A class is

- **distinct ℓ-diverse** if it holds at least ℓ different values;
- **entropy ℓ-diverse** if the entropy of P_E is at least $\log_2 \ell$. The entropy $H(P_E) = -\sum_v P_E(v) \log_2 P_E(v)$ measures in bits how spread out the values are; ℓ equally common values have entropy exactly $\log_2 \ell$, so the condition says the class is at least as spread out as ℓ equal values;

- **recursive (c, ℓ)-diverse** if $r_1 < c\,(r_\ell + r_{\ell+1} + \cdots + r_m)$: the most common value is less than c times the total of all but the $\ell - 1$ most common. The right-hand side is the "tail" that is left after the $\ell - 1$ biggest values are set aside, and the condition says the top value cannot swamp that tail.

A release is ℓ-diverse if every class is.

What recursive diversity guarantees. The tail is what survives the background knowledge attack. An adversary who rules out $\ell - 2$ values can at worst remove some of the big values, and the remaining ones still include a tail at least as large as the original one. Since the top value is less than c times that tail, it holds less than c parts out of $c + 1$. Precisely: if a class is recursive (c, ℓ)-diverse, then after the adversary rules out any $\ell - 2$ values that are not the target's, every remaining value v has

$$\Pr[S_{\text{target}} = v \mid \text{release}, \text{background}] < \frac{c}{1 + c}. \tag{16.6}$$

For distinct ℓ-diversity, the corresponding statement is weaker: the adversary needs to rule out $\ell - 1$ values to be certain, but one value can still hold all but $\ell - 1$ of the records.

16.4.3. Derivation

The plan is to compare the class after elimination with the original class: its top count cannot be larger, and its tail cannot be smaller, so the defining inequality survives.

1. **What is left.** Rule out a set B of $\ell - 2$ values. The remaining $m - \ell + 2$ values keep their original counts. Call the largest of them r'_1, and the total of the other $m - \ell + 1$ remaining values T'.
2. **The top does not grow.** Ruling out values removes records but never adds any, so r'_1 is one of the original counts, and no original count exceeds r_1: $r'_1 \le r_1$.
3. **The tail does not shrink below the original tail.** T' is a sum of $m - \ell + 1$ of the original counts. The smallest such sum you could possibly get is that of the $m - \ell + 1$ smallest counts, which is $r_\ell + \cdots + r_m$. So $T' \ge r_\ell + \cdots + r_m$.
4. **Combine with the definition.** Chain the three inequalities: $r'_1 \le r_1 < c\,(r_\ell + \cdots + r_m) \le c\,T'$.
5. **Turn counts into a posterior.** By Fact 1, after elimination the posterior of a remaining value is its count over the total count left, $r'_1 + T'$, so it is at most $r'_1/(r'_1 + T')$. The map $x \mapsto x/(x + T')$ increases in x (a larger share of a pie whose other part is fixed), so with $r'_1 < cT'$ this is below $cT'/(cT' + T') = c/(1 + c)$.

16.4.4. Interpretation

l-diversity protects against certainty, not against likelihood. The bound $c/(1 + c)$ is $\frac{1}{2}$ at $c = 1$, $\frac{2}{3}$ at $c = 2$ and $\frac{3}{4}$ at $c = 3$, so even recursive diversity lets the adversary be fairly sure. Distinct and entropy ℓ-diversity do not even give that, because neither limits the share of the top value directly: an entropy 3-diverse class over 8 values can have one value in 73% of its records, since the other 27% spread over seven values add enough entropy (Exercise 2 in the exercise book).

The adversary's knowledge is assumed, not bounded. The guarantee covers an adversary who rules out up to $\ell - 2$ values. One who rules out more, or who knows something softer ("probably not flu"), is not covered.

A posterior also has to be read against what the adversary would believe with no release. A posterior of 0.6 for one diagnosis is not a leak if the population already gives 0.6 to people of the target's age. The right comparison is the **imputation baseline** of Attribute inference and model inversion: the most likely value of the sensitive attribute given the target's own quasi-identifiers, under the true population distribution, computed with no release at all. Whatever the release teaches beyond that is about the individual; up to it, it is about the population.

Some tables cannot be made diverse at all. The table's distribution P is the average of its classes' distributions P_E, each weighted by the class's share of the records. Entropy is **concave**: mixing distributions can only make the result more spread out, so the entropy of an average is at least the average of the entropies. If every class had entropy at least $\log_2 \ell$, the table would too. So a table dominated by one value, with entropy below $\log_2 \ell$, has no entropy ℓ-diverse partition.

16.4.5. Example: homogeneity in the toy and in the hospital table

In Table 16.4 the three classes hold 1, 3 and 2 different values, so they are distinct 1-, 3- and 2-diverse. Their entropy versions are $\ell = 2^{H(P_E)}$, the number of equally common values with the same entropy. The first class has one value, entropy 0 and $\ell = 1.00$. The second has three equally common values and $\ell = 3.00$. The third, $\frac{2}{3}$ cancer and $\frac{1}{3}$ flu, has entropy $\frac{2}{3}\log_2 \frac{3}{2} + \frac{1}{3}\log_2 3 = 0.918$ bits and $\ell = 2^{0.918} = 1.89$: two values, but unevenly spread, so worth fewer than two. The release as a whole is only 1-diverse, because of Alice's class.

In the hospital table, the "certain disclosure" column of Table 16.3 counts the patients in homogeneous classes. At $k = 2$ it is 28.0% of all patients. At $k = 5$ it is 2.3%, or 232 patients whose diagnosis the release states outright. The population never gives that certainty: at no age does the population law give any diagnosis more than 0.705.

The guess accuracy column needs the baselines of Figure 16.2 (right). Guessing the most common diagnosis of the whole table is right for 18.5% of patients. Imputation, which guesses the most likely diagnosis for each patient's exact age under the population law, is right for 53.6%, with no release at all. A 5-anonymous release is right for 59.3%. So most of what the release teaches the adversary is the age–diagnosis pattern, which any useful study of this table would report. The excess over imputation, $0.593 - 0.536 = 0.057$ at $k = 5$, comes from the small classes, and it is down to $0.540 - 0.536 = 0.004$ at $k = 20$.

Table 16.5 adds diversity to $k = 5$.

Table 16.5.: Mondrian with l-diversity on the same hospital table: each split is allowed only if both halves satisfy the criterion.

Criterion	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
$k = 5$	1,542	5–11	0.016	0.1542	0.0232	0.593

Criterion	Classes	Sizes	Mean width	Linkage J/n	Certain disclosure	Guess accuracy
$k = 5$, distinct $\ell = 3$	1,132	5–33	0.022	0.1132	0.0000	0.574
$k = 5$, recursive $(c = 2, \ell = 3)$	389	5–641	0.113	0.0389	0.0000	0.417

Distinct 3-diversity removes the certain disclosures for little cost. Recursive $(2, 3)$-diversity forces some classes to cover hundreds of patients and pushes the guess accuracy below imputation. The reason is that diagnosis depends on age: to keep any diagnosis from dominating a class, Mondrian must mix ages, and that erases the pattern the data were released to show.

16.4.6. Check yourself

- Is the class {cancer, cancer, cancer, flu, asthma} recursive $(2, 3)$-diverse? *(No. The sorted counts are $r_1 = 3$, $r_2 = 1$, $r_3 = 1$, and the tail beyond the two most common values is $r_3 = 1$, so the condition is $3 < 2 \times 1$, which fails. It is distinct 3-diverse.)*
- Why does the derivation of Equation 16.6 allow only $\ell - 2$ ruled-out values, not $\ell - 1$? *(If the class has exactly ℓ values, ruling out $\ell - 1$ of them leaves one, with posterior 1, whatever c is. With $\ell - 2$ ruled out, at least two values remain, and step 3 of the derivation still finds the original tail among them.)*

16.5. Skewness, similarity and t-closeness

16.5.1. Context

Li, Li & Venkatasubramanian (2007) found two attacks that l-diversity does not see. In the **skewness attack**, 1% of the table has HIV, and a class that is half HIV, half not, is 2-diverse by every definition, yet by Fact 1 it raises the adversary's belief that a member has HIV from 1% to 50%, fifty-fold. In the **similarity attack**, a class holds three different diagnoses, gastric ulcer, gastritis and stomach cancer, so it is 3-diverse, and the adversary still learns that the target has a stomach disease. Both say that diversity is the wrong yardstick: what matters is how far a class's distribution is from what the adversary would believe anyway, which is the distribution of the whole table.

16.5.2. Statement

Definition. A release is **t-close** if every class's distribution is within distance t of the whole table's: $d(P_E, P) \le t$, for a chosen distance d between distributions.

With total variation as the distance, the guarantee is that no question about the target's value gets an answer more than t away from the answer the whole table gives. By Fact 2, the total variation is the largest such gap, so for every class and every set of values A,

$$|P_E(A) - P(A)| \le t. \tag{16.7}$$

In particular, guessing the most common value of the target's class is right with probability at most $\max_v P(v) + t$: the best guess from the table alone, plus t.

With the earth mover's distance (EMD), the choice of Li et al., values that are similar are close, which is what the similarity attack needs. Think of each distribution as piles of sand on the values. The EMD is the least work needed to reshape one set of piles into the other, where moving a unit of sand between two values costs their distance apart. For m ordered values (salary bands, age groups), numbered 1 to m and placed at ground distance $|i-j|/(m-1)$ so that the two ends are distance 1 apart, the work comes out as a sum over the gaps between neighbouring values of how much sand must cross each one:

$$\mathrm{EMD}(P_E, P) = \frac{1}{m-1} \sum_{i=1}^{m-1} \Big| \sum_{j \le i} (P_E(j) - P(j)) \Big|. \tag{16.8}$$

16.5.3. Derivation

Total variation (Equation 16.7). The idea is that the adversary's belief is the class distribution, so a bound on how far the class is from the table is a bound on how far the adversary's belief moves.

1. **A distance bound is an event bound.** By Fact 2, $\mathrm{TV}(P_E, P) = \max_A |P_E(A) - P(A)|$, so a bound t on the distance is a bound t on every event.
2. **The adversary's belief is the class distribution.** By Fact 1, the adversary's belief about a target in class E is P_E. So every question it can ask about the target's value, "is it in A?", gets an answer within t of the answer from the table as a whole.
3. **Apply it to the best guess.** Take $A = \{v\}$ for the class's most common value: $P_E(v) \le P(v) + t \le \max_u P(u) + t$.

Earth mover's distance on ordered values (Equation 16.8). The idea is that on a line, sand can only get from one side of a gap to the other by crossing it, so you can count the work gap by gap.

1. **What EMD is.** EMD is the least total cost of moving the mass of P_E until it becomes P, where moving mass f from value i to j costs $f|i-j|/(m-1)$. A move from i to j crosses each of the $|i-j|$ gaps between them once, so its cost is $1/(m-1)$ per unit per gap crossed.
2. **How much must cross each gap.** Look at the gap between values i and $i+1$. The values up to i hold $\sum_{j \le i} P_E(j)$ under the class and must end up holding $\sum_{j \le i} P(j)$. The difference, $\sum_{j \le i} (P_E(j) - P(j))$, is their excess (or shortfall), and that net mass has to cross the gap, rightwards if positive and leftwards if negative. Any plan moves at least this much across the gap.
3. **A plan that does no more.** A plan that only moves mass between neighbours and never in both directions across the same gap moves exactly the net amount across each gap. So the least cost is $1/(m-1)$ times the sum, over the $m-1$ gaps, of the absolute net mass crossing it, which is Equation 16.8.

16.5.4. Interpretation

The bound is additive, and rare values need a ratio. Equation 16.7 lets a 1% value rise to $1\% + t$. At $t = 0.15$ that is 16%, a sixteen-fold increase for the people who most need protection. A difference of 0.15 looks small next to a common value such as 40%, and enormous next to 1%. A guarantee for rare values has to bound the *ratio* of the adversary's beliefs with and without the release, not their difference. That is the shape of the definition in the next part.

t-closeness also fights the purpose of the release. If age predicts diagnosis, a class of similar ages has a different diagnosis distribution from the whole table, so it is not t-close for small t. Making every class t-close means mixing ages until the pattern is gone. But the pattern is a fact about the population, the kind of finding any study is published to report, not a secret about any one patient. Attribute inference and model inversion draws the same line between population inference and individual leakage.

The choice of distance is a choice of what is "similar". EMD with a ground distance from a hierarchy of diagnoses treats gastritis and gastric ulcer as close, so a class full of stomach diseases is far from the table and fails. Total variation treats every pair of values as equally different, so it would not see the similarity attack.

16.5.5. Example: toy, skew and hospital

In Table 16.4, the table has four heart disease, three cancer and two flu records, so its distribution is $\frac{4}{9}$ heart disease, $\frac{3}{9}$ cancer, $\frac{2}{9}$ flu. Each class's total variation from it is the total excess of the class over the table (Fact 2). Alice's class is all heart disease, an excess of $1 - \frac{4}{9} = \frac{5}{9} = 0.556$. The second class is $\frac{1}{3} = \frac{3}{9}$ of each, an excess of $\frac{3}{9} - \frac{2}{9} = \frac{1}{9} = 0.111$ on flu only. Bob's class, $\frac{6}{9}$ cancer and $\frac{3}{9}$ flu, has excesses $\frac{3}{9}$ and $\frac{1}{9}$, together $\frac{4}{9} = 0.444$. So the release is t-close only for $t \geq 0.556$.

For the skewness attack (Figure 16.3), the class that is 50% HIV is at total variation $0.50 - 0.01 = 0.49$ from a table with 1% HIV (the excess on HIV). A class with 10% HIV is at $0.10 - 0.01 = 0.09$, which passes $t = 0.1$, and still makes HIV ten times as likely for its members.

Table 16.6 adds t-closeness in total variation to $k = 5$ on the hospital table.

Table 16.6.: Mondrian with t-closeness (total variation) on the hospital table. The table's most common diagnosis has frequency 0.185.

Criterion	Classes	Sizes	Mean width	Largest TV	Guess accuracy	Bound $\max_v P(v)+t$
$k = 5$	1,542	5–11	0.016	0.937	0.593	—
$k = 5$, $t = 0.3$	495	6–60	0.291	0.300	0.275	0.485
$k = 5$, $t = 0.15$	136	14–304	0.324	0.149	0.222	0.335

The bound column is $0.185 + t$: $0.185 + 0.3 = 0.485$ and $0.185 + 0.15 = 0.335$. The guess accuracy respects this bound of Equation 16.7. But the classes now span about 30% of each quasi-identifier's

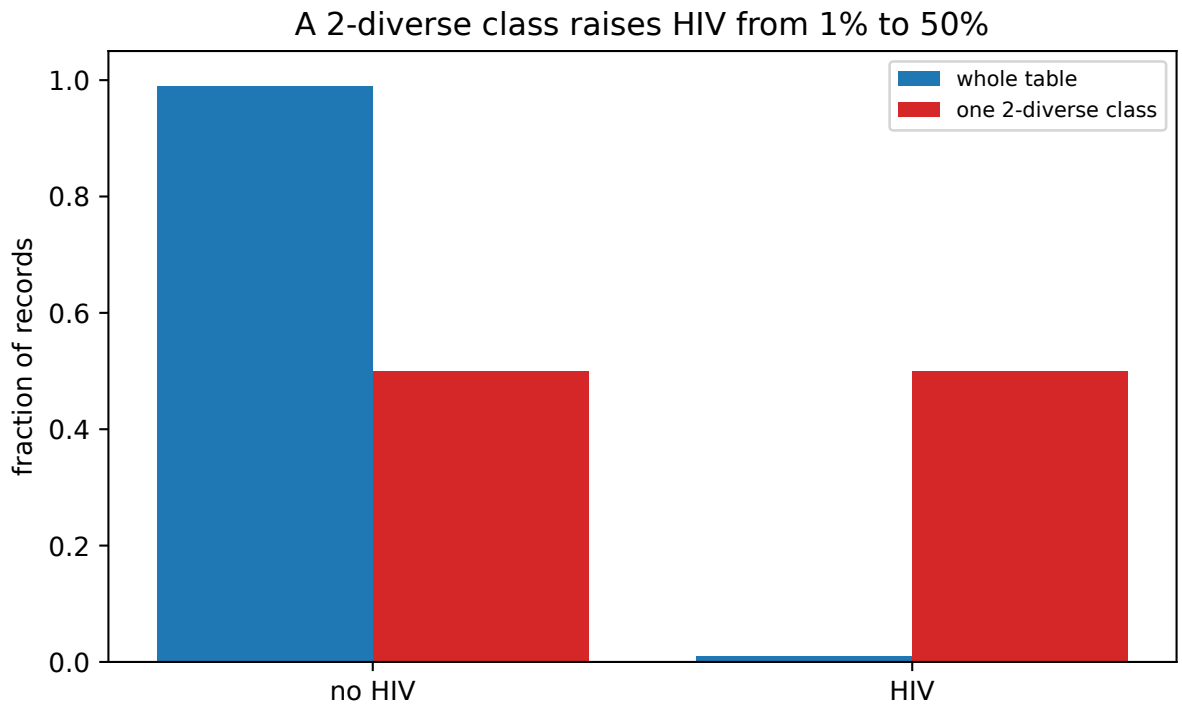


Figure 16.3.: The skewness attack: a table in which 1% of patients have HIV, and a class of it that is half HIV. The class is 2-diverse but at total variation 0.49 from the table.

range, and the release is worse than no release for learning about the population: its guess accuracy of 0.222 to 0.275 is far below the 0.536 that imputation from age achieves.

16.5.6. Check yourself

- A table has 2% cancer. A release is 0.05-close in total variation. What is the largest possible frequency of cancer in a class? *(0.07, from* $0.02 + 0.05$ *by Equation 16.7: 3.5 times the base rate, within the definition.)*
- Salaries take the 9 ordered values 3k to 11k, uniformly in the table. A class holds {3k, 4k, 5k}. Is the EMD to the table larger or smaller than for a class {6k, 8k, 11k}? *(Larger: 0.375 against 0.167. The first class is bunched at the low end, so mass has to travel far to spread out; the second is already spread out; Exercise 3 in the exercise book computes both.)*

16.6. The composition attack

16.6.1. Context

Every definition so far is about one release. Ganta, Kasiviswanathan & Smith (2008) asked what happens when there are two. Two hospitals in the same city publish their patient tables, each one properly k-anonymous, or ℓ-diverse, with no coordination. A patient treated at both

appears in a class of each. The adversary takes the diagnoses of the target's class in one release, those of the class in the other, and intersects: the true diagnosis is in both. Neither hospital did anything wrong, and each release satisfies its definition, yet the combination can reveal the diagnosis outright.

16.6.2. Statement

The attack. For a target in both releases, with class E_A in release A and E_B in release B, the candidate values are those that appear in both classes,

$$\{v : n_{E_A}(v) > 0\} \cap \{v : n_{E_B}(v) > 0\}, \tag{16.9}$$

and the true value is always among them, because the target's own record carries it in each class. The diagnosis is **pinned down** when one candidate is left.

How often, in a model. To see how often this happens, model the classes as random. Suppose the target's class has k_A records in one release and k_B in the other, and the other members' values are independent draws from the population distribution p, with p_v the probability of value v. Given that the target's true value is v, let A_v be the set of values other than v that appear among the $k_A - 1$ other members of the class in release A, and B_v the same for the $k_B - 1$ others in release B. These are the wrong values each release leaves open. The target is pinned exactly when no wrong value is left open by both, that is when A_v and B_v share nothing. So the probability is an average, over the true value v and over what release A leaves open, of the chance that every other member of the second class avoids those values. Writing $p(S) = \sum_{u \in S} p_u$ for the probability of a set of values,

$$\Pr[\text{pinned}] = \sum_v p_v \, \mathbb{E}[(1 - p(A_v))^{k_B - 1}], \qquad \Pr[A_v = S] = \sum_{T \subseteq S} (-1)^{|S| - |T|} \big(p_v + p(T)\big)^{k_A - 1}, \tag{16.10}$$

where the expectation is over the random set A_v, whose distribution is the second formula. A simpler lower bound counts, for each wrong value, the chance that both classes contain it, and subtracts the total from 1:

$$\Pr[\text{pinned}] \geq 1 - \sum_v p_v \sum_{u \neq v} \big(1 - (1 - p_u)^{k_A - 1}\big)\big(1 - (1 - p_u)^{k_B - 1}\big). \tag{16.11}$$

16.6.3. Derivation

The exact probability (Equation 16.10). The idea is to fix the true value and what the first class contains, compute the chance that the second class avoids it, and then average.

1. **Condition on the true value.** The target's true value is v with probability p_v. Everything below is for a fixed v, and the outer sum $\sum_v p_v(\cdot)$ averages over it.
2. **When the value is pinned.** The candidates are v itself together with the wrong values that appear in both classes, $\{v\} \cup (A_v \cap B_v)$. So the value is pinned exactly when A_v and B_v are disjoint.

3. **Fix what the first class leaves open.** Condition on $A_v = S$ for a particular set S. The $k_B - 1$ others in the second class are independent draws, and each avoids S with probability $1 - p(S)$. So $\Pr[B_v \cap S = \varnothing] = (1 - p(S))^{k_B - 1}$. Averaging over the possible sets S, weighted by $\Pr[A_v = S]$, gives the expectation in Equation 16.10.
4. **The distribution of the first class's set.** $A_v = S$ means two things: every one of the $k_A - 1$ draws lands in $S \cup \{v\}$, and each value of S is hit at least once. The first part is easy: "all draws land in $T \cup \{v\}$" has probability $(p_v + p(T))^{k_A - 1}$ for any set T. The second part is handled by **inclusion–exclusion**: start from "all draws land in $S \cup \{v\}$", subtract the cases where some value of S is missed (all draws land in a smaller set), add back the cases subtracted twice, and so on. The sets $T \subseteq S$ are the values not missed, and the sign $(-1)^{|S|-|T|}$ alternates with the number missed, which gives the alternating sum.

For a single wrong value, $S = \{u\}$, the sum has two terms: $\Pr[A_v = \{u\}] = (p_v + p_u)^{k_A - 1} - p_v^{k_A - 1}$, the probability that every draw is v or u, minus the probability that every draw is v (in which case u was never hit).

The lower bound (Equation 16.11). The idea is that the target fails to be pinned only if some wrong value turns up in both classes, and the chance of "some" is at most the sum of the chances of each.

1. **When it fails.** Given v, the value is not pinned when some $u \neq v$ is in both A_v and B_v.
2. **The chance for one wrong value.** u is missing from the first class when all $k_A - 1$ others avoid it, which has probability $(1 - p_u)^{k_A - 1}$, so $\Pr[u \in A_v] = 1 - (1 - p_u)^{k_A - 1}$, and likewise for B_v. The two classes are drawn independently, so the probability that u is in both is the product.
3. **Add over wrong values.** The expected number of wrong values in both sets is the sum of these products, by linearity of expectation (the expected count of events is the sum of their probabilities; Fact 1 of Linkage and re-identification). The chance that there is at least one is at most this expected number, because a count that is at least 1 whenever the event happens has expectation at least the event's probability (the union bound). Subtract from 1 and average over v.

16.6.4. Interpretation

None of the three definitions composes. Each is a condition on one table, and the intersection of two tables is not a table that either custodian checked. Ganta et al. showed the attack on k-anonymous and on ℓ-diverse releases. A custodian who checks only its own table cannot account for releases it does not know about.

That points to what is wanted instead. A definition that composes says: if release A leaks at most this much and release B at most that much, together they leak at most a known function of the two. It must also hold whatever the adversary knows, since another release is just auxiliary information. The next part starts from these two requirements.

The model understates the risk. It draws the other members of a class from the whole population. In real releases a class collects similar people (similar ages, here) who are more likely to share a diagnosis, so each class contains fewer distinct wrong values and the intersection is smaller than the model predicts.

The union bound is useful only for small classes. It charges every wrong value separately, so it is accurate when two wrong values rarely turn up in both classes at once. At $k = 2$ each class has a single other member, so it holds at most one wrong value, two wrong values can never turn up together, and the bound is exact. At $k = 5$ the expected number of shared wrong values exceeds 1, the bound is negative, and Equation 16.10 is needed.

16.6.5. Example: two hospitals

In the model, with p the population's diagnosis distribution, from which the hospital table is drawn, the probability that two releases pin down the diagnosis is 0.8734 for classes of 2, 0.1615 for classes of 5, 0.0288 for classes of 7 and 0.0015 for classes of 10. It falls fast because a larger class contains more wrong values, and each one is another chance for the two classes to share one. Simulation agrees to within 0.001 (Figure 16.4, left). The union bound gives 0.8734 at $k = 2$ and is negative from $k = 5$ on.

The script then runs the attack. A city has 15,000 people drawn from the model of the hospital table. Two hospitals each treat a random 10,000 of them, so 6,649 people are treated at both. Each hospital publishes a 5-anonymous Mondrian release of its own patients. Table 16.7 gives what the adversary learns about the 6,649.

Table 16.7.: The composition attack on two independent hospital releases, for the 6,649 people treated at both. "Pinned" is the fraction left with a single possible diagnosis.

Releases	Pinned by A alone	Pinned by B alone	Pinned by A and B	Mean candidates, A alone	Mean candidates, A and B
$k = 5$	0.0289	0.0262	0.0985	2.79	2.38
$k = 5$, distinct $\ell = 3$	0.0000	0.0000	0.0027	3.22	2.92

Two 5-anonymous releases pin down the diagnosis of 9.9% of the shared patients, more than three times either release alone (0.0985 against 0.0289 and 0.0262). For comparison, in the single-table release of Table 16.3 at $k = 5$ a patient's class has 6.8 records on average over patients, and the model at $k = 7$ predicts only 2.9%: real classes, which group patients of similar age, are more alike than random draws, as the interpretation above said. With distinct 3-diversity, no single release pins anyone down, and the two together pin down 18 patients (0.27% of 6,649).

16.6.6. Check yourself

- Two releases are each 3-diverse in the distinct sense, over 8 possible values. If each class's two other values were chosen uniformly at random from the 7 wrong ones, how likely is the intersection to pin down the truth? *($\binom{5}{2}/\binom{7}{2} = 10/21 = 0.476$: the second class must avoid both of the first class's wrong values, so its two wrong values must come from the other 5. Exercise 4 in the exercise book derives it.)*

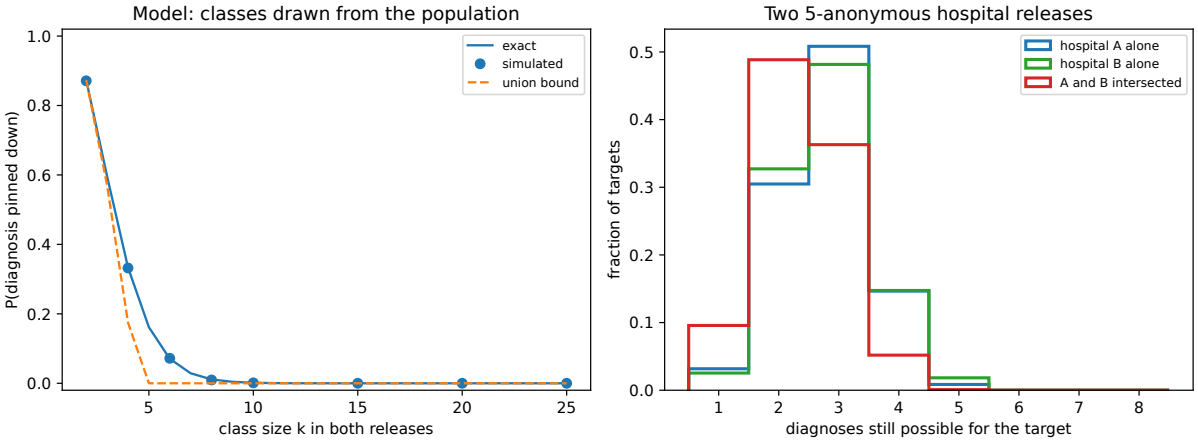


Figure 16.4.: Left: the probability that two releases pin down the diagnosis, against the class size, when the other members of each class are drawn from the population: exact (Equation 16.10), simulated, and the union bound (Equation 16.11). Right: the number of diagnoses still possible for a patient treated at both hospitals, from either 5-anonymous release alone and from their intersection.

- Why can't the second hospital prevent the attack by checking its own release more carefully? *(The attack needs only that the two classes share no wrong value. The second hospital would need to know the first hospital's classes, and every other release the patient appears in.)*

16.7. Code: anonymize, then attack

The script code/syntactic_anonymity.py computes every number in this chapter and draws every figure. It implements Mondrian with any criterion on a class (k-anonymity, distinct and recursive l-diversity, t-closeness in total variation), the uniform-cells calculation, the attacks on the sensitive attribute, and the composition attack, exactly by inclusion–exclusion, by simulation and on two synthetic hospital releases. From the repository root:

```
uv run python code/part02-attacks/syntactic_anonymity.py
```

Output (seed 0), summarized in Table 16.8:

Table 16.8.: The script's output with the default seed.

Experiment	Result
Toy release	Posteriors 1 (homogeneous class), $\frac{2}{3}$ cancer (Bob), 1 after ruling out flu; entropy $\ell = 1.00$, 3.00, 1.89; TV to the table 0.556, 0.111, 0.444
Recursive (c, ℓ) bound	$c/(1+c) = 0.500$, 0.667, 0.750 for $c = 1$, 2, 3
Skewness	50% HIV against 1%: TV 0.49, ratio 50. 10% HIV: TV 0.09, ratio 10

Experiment	Result
Uniform cells, $n = 10^4$	Table 16.2
Hospital table, $n = 10^4$	Exact quasi-identifiers: linkage 0.933. Baselines: most common diagnosis 0.185, imputation from age 0.536, largest population posterior 0.705
Mondrian	Table 16.3, Table 16.5, Table 16.6
Two hospitals	Table 16.7
Composition model	Pinned: 0.8734 ($k = 2$), 0.1615 ($k = 5$), 0.0288 ($k = 7$), 0.0015 ($k = 10$); simulation within 0.001

tests/test_syntactic_anonymity.py checks that the output still matches.

The study plan's exercise is to k-anonymize a census extract with ARX, a free anonymization tool that implements these criteria, and link it back to a public table. The script's Mondrian release plays the part of ARX, and its linkage column is that measurement: the re-identification rate against an adversary holding everyone's quasi-identifiers, the strongest auxiliary table there can be.

What to take from it. k-anonymity does what it says: linkage success stays below $1/k$, and Mondrian keeps it above $1/(2k)$ at little cost in precision. But what it says is not enough, because small classes state diagnoses outright, for 2.3% of patients at $k = 5$. Even where nothing is stated outright, most of what an anonymized table teaches is about the population, so measure attribute disclosure against imputation, not against guessing the most common value. Each fix costs utility: recursive diversity and t-closeness remove the certain disclosures by erasing the age–diagnosis pattern the data exist to show. And two releases are worse than one, under every definition here. The composition attack more than triples the pinned diagnoses, and it is the attack that a better definition has to survive.

16.8. References

- Samarati & Sweeney, "Protecting Privacy when Disclosing Information: k-Anonymity and Its Enforcement through Generalization and Suppression" (SRI technical report, 1998), and Sweeney, "k-Anonymity: A Model for Protecting Privacy" (International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems, 2002): the definition, generalization and suppression, and the linkage attack it answers.
- LeFevre, DeWitt & Ramakrishnan, "Mondrian Multidimensional k-Anonymity" (ICDE, 2006): the median-split algorithm the script uses, and why multidimensional recoding loses less than one generalization per column.
- Machanavajjhala, Kifer, Gehrke & Venkitasubramaniam, "l-Diversity: Privacy Beyond k-Anonymity" (ACM Transactions on Knowledge Discovery from Data, 2007; ICDE 2006): the homogeneity and background knowledge attacks, and the three versions of l-diversity with their Bayesian justification.

- Li, Li & Venkatasubramanian, "t-Closeness: Privacy Beyond k-Anonymity and l-Diversity" (ICDE, 2007): the skewness and similarity attacks, and the earth mover's distance with ordered and hierarchical ground distances.
- Ganta, Kasiviswanathan & Smith, "Composition Attacks and Auxiliary Information in Data Privacy" (KDD, 2008): the intersection attack on independent releases, experiments on public census data, and the argument that a definition must hold up against auxiliary information such as other releases.
- Aggarwal, "On k-Anonymity and the Curse of Dimensionality" (VLDB, 2005): why high-dimensional data cannot be k-anonymized without destroying it.
- Prasser, Kohlmayer and colleagues, the ARX data anonymization tool (arx.deidentifier.org): an open implementation of these criteria and many more, with risk models and utility measures.
- Article 29 Working Party, "Opinion 05/2014 on Anonymisation Techniques": how European regulators judge these techniques against singling out, linkability and inference, the three attacks of this chapter.

16.9. Cheat sheet

n records, classes E of size $|E|$, J classes, P_E the class's distribution of the sensitive attribute and P the table's, $r_1 \geq r_2 \geq \ldots$ a class's sorted counts. Table 16.9 collects the results.

Table 16.9.: Every result in this chapter.

Result	Formula	Note
Posterior in a class	$P_E(v) = n_E(v)/\|E\|$	Ruling out values renormalizes over the rest
k-anonymity	Every $\|E\| \geq k$; linkage $J/n \leq 1/k$	Says nothing about the sensitive attribute
Uniform cells	Fraction in classes $< k$: $\Pr[\text{Poisson}(n/M) \leq k-2]$	99% 2-anonymous at $\log_2 n - 2.20$ bits
Distinct, entropy l-diversity	$\geq \ell$ values; $H(P_E) \geq \log_2 \ell$	Neither caps the top value's share near $1/\ell$
Recursive (c, ℓ)-diversity	$r_1 < c(r_\ell + \cdots + r_m)$	Posterior $< c/(1+c)$ after ruling out $\ell - 2$ values
t-closeness (TV)	$\|P_E(A) - P(A)\| \leq t$ for every A	Additive: a 1% value may rise to $1\% + t$
EMD, ordered values	$\frac{1}{m-1}\sum_i \|\sum_{j \leq i}(P_E(j) - P(j))\|$	Similar values are close
Composition	Candidates \$= \$ values of E_A $\cap$ values of E_B	Pinned: Equation 16.10; no syntactic definition prevents it

Rules of thumb

- k-anonymity protects names, not attributes. Ask what a class's sensitive values reveal, not just how big it is.

- Measure attribute disclosure against imputation from the population, or you will count the study's findings as leaks.
- A rare sensitive value needs a bound on the ratio of beliefs, not on their difference.
- Assume another release exists. A guarantee that is only about one table is not a guarantee about a person.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

17. Reconstruction attacks

17.1. Why this matters

A hospital keeps one sensitive bit per patient: whether they tested positive. It never releases the bits. It only answers counting questions: "how many of patients 3, 17, 42, … tested positive?" To be safe, it adds a little noise to every answer. Is that enough? The answer, due to Dinur and Nissim (2003), is a hard no unless the noise is large. If every answer is within $o(\sqrt{n})$ of the truth, where n is the number of patients, an attacker who asks a number of random questions only slightly larger than n can recover almost every bit by solving a linear program. If the attacker may ask all 2^n questions, an error of $o(n)$ is already too small. (Here $o(\sqrt{n})$ means an amount that grows more slowly than $\sqrt{n}$, so that its ratio to $\sqrt{n}$ goes to 0 as n grows; $O(\cdot)$, used below, means "at most a constant times".) A table of aggregates is a system of equations whose unknowns are the people in it, and enough equations determine the unknowns.

This is the **reconstruction game** of Threat models and privacy games. In that game the secret is the whole dataset (or one column of it), and the adversary is scored by the fraction of entries it recovers, compared with the baseline of guessing from the population, which for fair bits is $\frac{1}{2}$. That chapter showed that one exact count of n fair bits lets the adversary recover only about $\frac{1}{2} + 1/\sqrt{2\pi n}$ of them: knowing the total tells you which value is the majority, and nothing about who holds it. This chapter shows what happens with many counts.

The one idea that organizes the chapter is this. **Every answer is a linear equation in the secret bits, and the error of the answers is the only thing standing between the attacker and the solution.** To hide a wrong guess, the curator's error has to be as large as the amount by which that guess changes the answers. For a random query that amount is about $\sqrt{n}$ when the guess is wrong on a constant fraction of the bits, and for the query aimed at exactly the wrong bits it is about n; errors well below those scales cannot hide the solution. Errors that are independent from answer to answer, and do not grow with the number of queries, are averaged away by asking more. Dwork and Roth call this the **Fundamental Law of Information Recovery**: overly accurate answers to too many questions destroy privacy.

Table 17.1 lists the results and where they are used.

Table 17.1.: The results of this chapter and where each is used.

Result	Where it is used
Exponential attack: error E on all 2^n subset sums leaves at most $4E$ wrong bits	Why per-answer error must be of order n when queries are unlimited
Anti-concentration of a random subset sum (Paley–Zygmund)	The step that makes a random query informative about every bad candidate

Result	Where it is used
Linear-programming decoder: error $o(\sqrt{n})$ on $O(n \log n)$ random queries leaves $o(n)$ wrong bits	The Dinur–Nissim attack; the $\sqrt{n}$ threshold
Least squares under worst-case and random error	The attack you would actually run; how noise and the number of queries trade off
Sampling curator, via Hoeffding and the union bound	Error of order $\sqrt{n \log m}$ can stop reconstruction, yet still be far from private
Information bound, via Fano	No attack can succeed with much fewer than $n/\log n$ queries
Census block reconstruction by constraint search	Why publishing many overlapping tables reveals the microdata

How to read this chapter. Read the toolkit, then the exponential attack, which is short and exact. The linear-programming section is the heart of the chapter; on a first pass read its statement, the lemma and the figure, and come back for the union-bound steps. The least-squares section is what the script runs. The Fundamental Law section puts the results together with two converses, and the census section shows the same idea on real-looking tables.

The concentration tools (Hoeffding's inequality, the union bound) are in Concentration inequalities. Entropy, mutual information, the data processing inequality (DPI) and Fano's inequality are in Information theory. Each is restated below where it is used. No privacy definition is used: Part 3 introduces the one that defends against these attacks, and Lower bounds reads this chapter's results as lower bounds on the error any defence must add.

Conventions. The secret database is a bit vector $d = (d_1, \dots, d_n) \in \{0,1\}^n$, one bit per person. A **subset-sum query** is a set $S \subseteq [n] = \{1, \dots, n\}$ of people, and it asks how many of them have bit 1. It is written as its 0/1 indicator vector q, with $q_i = 1$ when person i is in S; its true answer is $\sum_{i \in S} d_i = \langle q, d \rangle$. The attacker asks m queries, the rows of an $m \times n$ matrix A. The **curator** (the party holding d) returns $a = Ad + e$, where e is its error, one entry per query. "Error at most E" means $|e_j| \le E$ for every query j. β is a fraction of wrong bits, η a failure probability and σ the standard deviation of random noise. Logarithms are natural unless written $\log_2$. Threat models and privacy games wrote m for the number of bits; here n is the number of bits and m the number of queries.

17.2. Toolkit: three facts every derivation uses

17.2.1. Fact 1 — Subset sums are inner products

A subset sum adds up the bits of the people in the subset, so it is the inner product of the subset's indicator vector with the database: $\langle q, d \rangle = \sum_i q_i d_i$, which is linear in d. Stacking m queries as the rows of A gives $a = Ad + e$: a linear system with n unknowns (the bits) and m equations, each off by its error. Table 17.2 shows a few of the $2^4 = 16$ subset sums of the database $d = (1, 0, 1, 1)$.

Table 17.2.: Six subset sums of $d = (1, 0, 1, 1)$. Each is a linear equation in the four secret bits.

Subset S	$\{1\}$	$\{2\}$	$\{1,2\}$	$\{3,4\}$	$\{2,3,4\}$	$\{1,2,3,4\}$
Indicator q	1000	0100	1100	0011	0111	1111
$\langle q, d\rangle$	1	0	1	2	2	3

With exact answers there is nothing to attack. The singletons $\{1\}, \dots, \{4\}$ alone reveal d exactly, one bit per answer, and any n queries whose indicator vectors are linearly independent do the same, because Gaussian elimination solves the system. Everything interesting in this chapter is about answers with error.

Linearity also tells you how an attacker rules out a wrong guess. If the attacker considers a candidate database c, the answers c would produce differ from the true ones by $\langle q, c\rangle - \langle q, d\rangle = \langle q, c - d\rangle$, the subset sum of the difference vector $c - d$. If, for some query, that difference is larger than the curator's error could explain, the candidate is ruled out. Every attack below is a way of finding such a query for every bad candidate.

17.2.2. Fact 2 — Moments of a random-sign sum

A random query includes each person with probability $\frac{1}{2}$, so its answer behaves like a sum of terms that are switched on or off by fair coins. The cleanest form of such a sum uses signs instead of on/off switches. Let $s_1, \dots, s_n$ be independent fair signs ($+1$ or -1 with probability $\frac{1}{2}$ each) and $X = \sum_i s_i a_i$ for fixed numbers a_i. The sum is centred at 0, and its spread is set by $\sum_i a_i^2$, because the variances of independent terms add. What the later proofs need is that X has no heavy tails: its fourth moment is at most 3 times the square of its second moment, the same ratio as for a normal variable. In symbols,

$$\mathbb{E}X = 0, \qquad \mathbb{E}X^2 = \sum_i a_i^2, \qquad \mathbb{E}X^4 = 3\Big(\sum_i a_i^2\Big)^2 - 2\sum_i a_i^4 \le 3\Big(\sum_i a_i^2\Big)^2. \tag{17.1}$$

The proof is a count of which terms survive when you take expectations. Expand $X^4 = \sum_{i,j,k,l} s_i s_j s_k s_l a_i a_j a_k a_l$. A term in which some index appears an odd number of times (once or three times) contains a factor $\mathbb{E}s_i = 0$ or $\mathbb{E}s_i^3 = 0$, and vanishes, because the signs are independent and $s_i^3 = s_i$ has mean 0. Two kinds of terms survive. If all four indices are equal, $s_i^4 = 1$, and these terms add up to $\sum_i a_i^4$. If the indices form two distinct pairs, each $s_i^2 = 1$; the pairs can be matched as $i = j \neq k = l$ or in two other ways, so these terms add up to $3\sum_{i \neq k} a_i^2 a_k^2 = 3((\sum_i a_i^2)^2 - \sum_i a_i^4)$. Adding the two kinds gives the formula, and dropping the negative term $-2\sum_i a_i^4$ gives the bound. The second moment is the same computation with two indices: only the terms with $i = j$ survive. The mean is 0, and every odd moment is 0, because flipping all the signs turns X into $-X$ without changing its distribution.

A concrete case. Take $a = (1, 1, 1)$. Of the eight sign patterns, two give $X = \pm 3$ (all signs equal) and six give $X = \pm 1$, so $\mathbb{E}X^4 = (2 \cdot 81 + 6 \cdot 1)/8 = 21$. The formula gives $3 \cdot 9 - 2 \cdot 3 = 21$, and the bound $3 \cdot 9 = 27$.

This is how the fact is used. A random subset has $q_i = (1 + s_i)/2$: the person is in when the sign is $+1$ and out when it is -1. So a random subset sum $\sum_i q_i z_i$ is the constant $\frac{1}{2}\sum_i z_i$ plus half a

random-sign sum, and its fourth moment is at most 3 times its squared second moment. That ratio is what Fact 3 needs.

17.2.3. Fact 3 — Paley–Zygmund: a variable with a light fourth moment is not concentrated near 0

Markov's inequality (Concentration inequalities) says that a non-negative variable is rarely *far above* its mean. The attacks need the opposite: that a variable is not too often *far below* its mean, so that a random query does not usually give a tiny answer. That cannot hold for every variable. A variable that is 0 most of the time and huge once in a while has a large mean and is still usually 0. What rules this out is a second moment that is not much larger than the squared mean, since a rare huge value makes $\mathbb{E}[W^2]$ enormous. The **Paley–Zygmund inequality** makes this precise. For a random variable $W \geq 0$ with finite second moment and $0 \leq \theta < 1$,

$$\Pr\left[W > \theta\,\mathbb{E}W\right] \geq (1-\theta)^2 \frac{(\mathbb{E}W)^2}{\mathbb{E}[W^2]}. \tag{17.2}$$

A lower bound on how often a variable is not close to zero is called **anti-concentration**.

The proof splits the mean $\mathbb{E}W$ into the part that comes from the event $B = \{W > \theta\,\mathbb{E}W\}$ and the part that comes from its complement: $\mathbb{E}W = \mathbb{E}[W\mathbf{1}_{B^c}] + \mathbb{E}[W\mathbf{1}_B]$. On the complement $W \leq \theta\,\mathbb{E}W$, so the first part is at most $\theta\,\mathbb{E}W$. For the second part, use the Cauchy–Schwarz inequality $\mathbb{E}[UV] \leq \sqrt{\mathbb{E}[U^2]\,\mathbb{E}[V^2]}$ with $U = W$ and $V = \mathbf{1}_B$; since $\mathbf{1}_B^2 = \mathbf{1}_B$ has mean $\Pr[B]$, this gives $\mathbb{E}[W\mathbf{1}_B] \leq \sqrt{\mathbb{E}[W^2]\Pr[B]}$. Together, $\mathbb{E}W \leq \theta\,\mathbb{E}W + \sqrt{\mathbb{E}[W^2]\Pr[B]}$, that is, $(1-\theta)\mathbb{E}W \leq \sqrt{\mathbb{E}[W^2]\Pr[B]}$. Squaring both sides and dividing by $\mathbb{E}[W^2]$ gives Equation 17.2.

The linear-programming decoder uses this with W the square of a random subset sum: it needs a random query to move a bad candidate's answer by a lot with constant probability, and Fact 2 supplies the light fourth moment.

17.3. The exponential attack

17.3.1. Context

Start with the strongest attacker: it may ask every one of the 2^n subset sums. How large must the curator's error be before the attacker cannot pin d down? The answer is linear in n, and the proof is three lines.

17.3.2. Statement

The attacker cannot know d, but it can throw away every candidate that contradicts an answer. Call a candidate $c \in \{0,1\}^n$ **consistent** if each of its subset sums is within E of the published answer, $\left|a_S - \sum_{i\in S} c_i\right| \leq E$ for every S; the true d is always consistent. The attacker outputs any consistent candidate. The reason it cannot be far from d is that the bits where c is wrong form two subsets, the ones it missed and the ones it invented, and the attacker has asked about both. On each of them c and d disagree by the size of the subset, while both have to be within E of the

same answer, so each subset has at most $2E$ people. The number of wrong bits, the **Hamming distance** between c and d, therefore satisfies

$$\operatorname{dist}(c, d) = \#\{i : c_i \neq d_i\} \leq 4E. \tag{17.3}$$

The bound is tight: if d has at least $\lfloor 2E \rfloor$ ones and $\lfloor 2E \rfloor$ zeros, some curator with error at most E makes a candidate at distance $2\lfloor 2E \rfloor$ consistent.

So a curator with error $E = o(n)$ is **blatantly non-private** in the sense of Dinur and Nissim: the attacker recovers all but $o(n)$ of the bits.

17.3.3. Derivation

1. **A consistent candidate exists**, because d itself is one: the true answers are within E of d's sums by assumption.
2. **Count the ones that the candidate missed.** Let $S_0 = \{i : d_i = 1, c_i = 0\}$. On S_0, d sums to $|S_0|$ and c sums to 0. Both sums are within E of the one answer a_{S_0}, so by the triangle inequality they are within $2E$ of each other: $|S_0| = |S_0| - 0 \leq 2E$.
3. **Count the zeros that the candidate invented.** Let $S_1 = \{i : d_i = 0, c_i = 1\}$. Now c sums to $|S_1|$ and d to 0 on S_1, and the same argument gives $|S_1| \leq 2E$.
4. **Add.** Every disagreement is in S_0 or in S_1, so $\operatorname{dist}(c, d) = |S_0| + |S_1| \leq 4E$.

Tightness. To show a curator can really be this misleading, build a bad candidate and a curator that covers for it. Let $k = \lfloor 2E \rfloor$ and let c flip k of the ones of d to 0 and k of the zeros to 1, so c is at distance $2k$. For any S, $\sum_S (c_i - d_i)$ is the number of flipped zeros in S minus the number of flipped ones in S, so it lies in $[-k, k]$: the two databases' sums are never more than k apart. A curator that answers the midpoint $a_S = \frac{1}{2}(\sum_S c_i + \sum_S d_i)$ is then within $k/2 \leq E$ of both, so its error is at most E and c is consistent.

17.3.4. Interpretation

The proof uses only two queries, S_0 and S_1: the people the candidate got wrong in each direction. Small, well-aimed subsets do all the work. But the attacker does not know which people it got wrong, so it does not know these two subsets in advance, and that is why it needs all 2^n answers.

This is not an efficient attack. The attacker checks 2^n candidates against 2^n answers. The next section replaces "all subsets" by a few times n random ones and "check every candidate" by a linear program. The price is that a random subset cannot aim at the wrong bits the way S_0 and S_1 do, so the error must be $o(\sqrt{n})$ instead of $o(n)$.

The bound is a worst case over curators. It holds for every curator, including one that picks its errors to fool this attacker. A specific curator can be much weaker than the bound suggests. One that rounds every answer to a multiple of 4 has error at most 2, but its rounding is a known function of the true sum, and an attacker who simulates the rounding on each candidate recovers d exactly (Exercise 1 in the exercise book).

17.3.5. Example: ten people, all 1024 subsets

The script draws $d = (1, 1, 1, 0, 0, 0, 0, 0, 0, 1)$, with four ones, and checks all 2^{10} candidates against all 2^{10} subsets. It needs a test for whether a candidate c can be made consistent by *some* curator with error at most E. The tightness argument gives it: if every subset sum of $c - d$ is at most $2E$ in absolute value, the midpoint curator has error at most E and keeps c; if some subset sum of $c - d$ exceeds $2E$, no single answer can be within E of both c's sum and d's, so no curator keeps c. The script counts the candidates that pass the test (Table 17.3).

Table 17.3.: The exponential attack on $n = 10$ bits. Every surviving candidate is within $4E$ of d, and the bound is attained.

Error E	0	0.5	1	1.5	2
Candidates that survive	1	35	242	630	912
Largest distance to d	0	2	4	6	8
Bound $4E$	0	2	4	6	8

The bound is attained at every E, as the tightness argument predicts: that argument needs $\lfloor 2E \rfloor$ ones and $\lfloor 2E \rfloor$ zeros to flip, at most 4 of each here, and d has four ones and six zeros. At $E = 2$, 912 of the 1024 candidates survive, so the attacker learns little: the bound $4E = 8$ is close to $n = 10$, and a guarantee of at most 8 wrong bits out of 10 says almost nothing. Figure 17.1 draws both rows over a finer grid of E, for another random database of ten bits, so its counts can differ from those of Table 17.3.

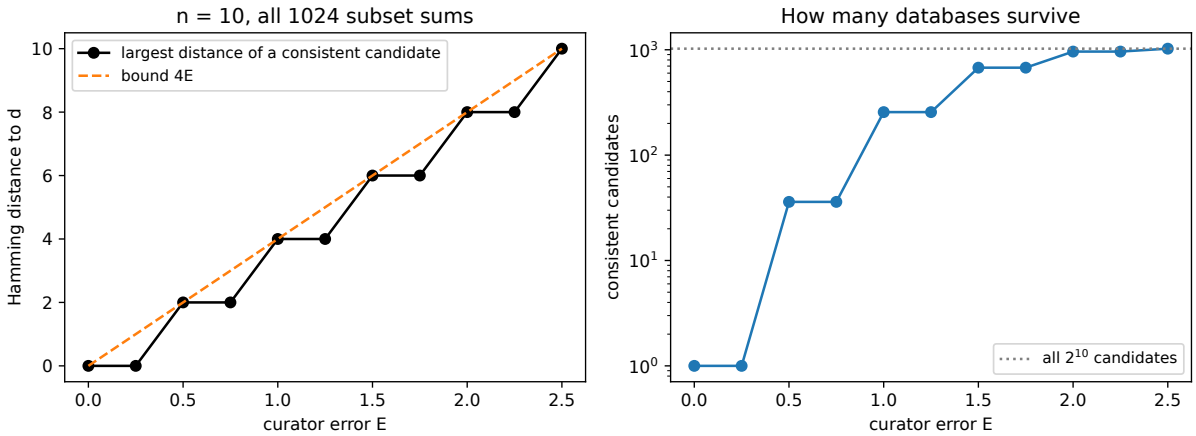


Figure 17.1.: The exponential attack on ten bits. Left: the largest Hamming distance between the true database and a candidate that some curator with error E can make consistent with all 1024 subset sums, against the bound $4E$. Right: the number of surviving candidates (log scale); at $E = 0$ only d survives.

17.3.6. Check yourself

- Why does the derivation not need the curator's errors to be random? *(It only uses $|e_S| \le E$ for the two subsets S_0 and S_1. The errors can be chosen by an adversarial curator, even*

after seeing the attacker's candidate.)

- With $n = 10^6$ people and every subset sum answered with error at most 1000, how many bits can the attacker get wrong? *(At most $4E = 4000$, or 0.4% of the bits, although each answer has error up to 1000.)*

17.4. The linear-programming decoder

17.4.1. Context

Asking 2^n questions is not realistic. Dinur and Nissim showed that a number of *random* subsets only a polylogarithmic factor above n suffices, with an efficient decoder: their proof asks $n(\log n)^2$ of them, and the version proved below needs $O(n \log n)$. Instead of checking every bit vector, the decoder lets each unknown be any number between 0 and 1, finds a fractional vector consistent with the answers by linear programming (LP), and rounds it. A linear program finds a point that satisfies a list of linear inequalities, and it can be solved in time polynomial in the number of unknowns and inequalities.

The price is that the error must now be $o(\sqrt{n})$ instead of $o(n)$, and the reason is that a random query cannot aim at the wrong coordinates. Suppose a candidate differs from d by a vector with k entries of constant size. A random subset picks up each of those entries with probability $\frac{1}{2}$, so its sum of the difference behaves like a sum of k coin-flip terms, which fluctuates by about $\sqrt{k}$, not k. An error below $\sqrt{k}$ cannot hide that fluctuation. The key step is to show that the sum does fluctuate this much with constant probability, whatever the signs of the entries, and that is where the anti-concentration of Fact 3 comes in.

17.4.2. Statement

Let the m queries be independent uniformly random subsets (each person included with probability $\frac{1}{2}$). The true d satisfies every answer to within E, so the set of vectors that do so is not empty. The **linear-programming decoder** searches that set, after relaxing each bit to the interval $[0, 1]$ so that the search becomes a linear program. It solves the feasibility problem

$$\text{find } c \in [0,1]^n \text{ with } \left|\langle q_j, c\rangle - a_j\right| \le E \text{ for } j = 1, \dots, m, \tag{17.4}$$

which is a linear program: each absolute value is two linear inequalities, which makes $2m$ of them, and each $0 \le c_i \le 1$ is a box constraint, one per person. The decoder then rounds each c_i to the nearest bit.

The analysis rests on one lemma about a single random query. Think of z as the difference between a bad candidate and d. If z has many coordinates of at least a constant size (here $\frac{1}{3}$, the size left after a rounding step in the proof), then a random subset sum of z lands far from 0 with probability at least a constant, whatever the signs of the coordinates. "Far" means on the scale $\sqrt{\beta n}$, the square root of the number of large coordinates, as the coin-flip picture above predicts.

Lemma (one random query catches a bad difference). Let $z \in \mathbb{R}^n$ have at least βn coordinates with $|z_i| \ge \frac{1}{3}$, and let q be a uniformly random subset. Then

$$\Pr\left[\,|\langle q, z\rangle| > \frac{\sqrt{\beta n}}{12}\right] \ge \frac{3}{16}. \tag{17.5}$$

The theorem turns this into a guarantee for the decoder. A bad candidate survives the linear program only if every one of the m queries fails to catch it. Each query catches it with probability at least $\frac{3}{16}$, so it survives all m independent queries with probability at most $(13/16)^m$, which shrinks exponentially in m. There are many bad candidates, but after rounding them to a grid there are only finitely many, about $(n+1)^n$. Taking m large enough that $(n+1)^n(13/16)^m$ falls below a chosen failure probability η makes every bad candidate fail at once.

Theorem (Dinur–Nissim, in the form proved here). Fix d, β and a failure probability η. If the curator's error is at most E with $2E + \frac{1}{2} \le \sqrt{\beta n}/12$, and

$$m \ge \frac{n \ln(n+1) + \ln(1/\eta)}{\ln(16/13)}, \tag{17.6}$$

then with probability at least $1 - \eta$ over the queries, every solution of the linear program rounds to a vector with at most βn wrong bits, whatever errors the curator chose.

In words: error $o(\sqrt{n})$ on $O(n \log n)$ random subset sums lets the attacker recover all but $o(n)$ bits in polynomial time.

17.4.3. Derivation

The lemma. The plan is to apply Paley–Zygmund (Equation 17.2) to $W = Y^2$, where $Y = \langle q, z \rangle$ is the random subset sum. That needs the second and fourth moments of Y, and Fact 2 provides them once Y is written as a constant plus a random-sign sum. So write the random subset as $q_i = (1 + s_i)/2$ with independent fair signs s_i.

1. **Split the sum into a constant and a random-sign part.** $Y = \langle q, z \rangle = \mu + X$, with the constant $\mu = \frac{1}{2}\sum_i z_i$ and $X = \frac{1}{2}\sum_i s_i z_i$, a random-sign sum with coefficients $a_i = z_i/2$.
2. **Compute the second moment.** By Fact 2, $\mathbb{E}X = 0$, and the variance of the random-sign part, written τ^2 so that τ is its standard deviation, is $\tau^2 := \mathbb{E}X^2 = \sum_i a_i^2 = \frac{1}{4}\sum_i z_i^2$. Expanding $(\mu + X)^2$, the cross term $2\mu\,\mathbb{E}X$ vanishes, so $\mathbb{E}Y^2 = \mu^2 + \tau^2$.
3. **Bound the fourth moment by 3 times the squared second moment.** Expand $(\mu+X)^4 = \mu^4 + 4\mu^3 X + 6\mu^2 X^2 + 4\mu X^3 + X^4$. The terms with $\mathbb{E}X$ and $\mathbb{E}X^3$ vanish, since odd moments of a random-sign sum are 0. Fact 2 gives $\mathbb{E}X^4 \le 3(\sum_i a_i^2)^2 = \frac{3}{16}(\sum_i z_i^2)^2 = 3\tau^4$. So
$$\mathbb{E}Y^4 = \mu^4 + 6\mu^2\tau^2 + \mathbb{E}X^4 \le \mu^4 + 6\mu^2\tau^2 + 3\tau^4 \le 3(\mu^2 + \tau^2)^2, \tag{17.7}$$
where the last step holds because $3(\mu^2+\tau^2)^2 = 3\mu^4 + 6\mu^2\tau^2 + 3\tau^4$ is larger by $2\mu^4 \ge 0$.
4. **Apply Paley–Zygmund** (Equation 17.2) with $W = Y^2$, whose mean is $\mu^2 + \tau^2$. Step 3 says $(\mathbb{E}Y^2)^2/\mathbb{E}Y^4 \ge \frac{1}{3}$, so $\Pr[Y^2 > \theta(\mu^2+\tau^2)] \ge (1-\theta)^2(\mathbb{E}Y^2)^2/\mathbb{E}Y^4 \ge (1-\theta)^2/3$.
5. **Drop the mean.** Since $\mu^2 + \tau^2 \ge \tau^2$, the event $Y^2 > \theta(\mu^2+\tau^2)$ implies $Y^2 > \theta\tau^2$, that is $|Y| > \sqrt{\theta}\,\tau$. So $\Pr[|Y| > \sqrt{\theta}\,\tau] \ge (1-\theta)^2/3$. A non-zero mean only helps: it pushes Y away from 0.
6. **Use the large coordinates to bound τ from below.** At least βn coordinates have $z_i^2 \ge \frac{1}{9}$, and the others contribute $z_i^2 \ge 0$, so $\tau^2 = \frac{1}{4}\sum_i z_i^2 \ge \frac{1}{4} \cdot \frac{\beta n}{9}$ and $\tau \ge \sqrt{\beta n}/6$.
7. **Choose $\theta = \frac{1}{4}$.** Then $\sqrt{\theta}\,\tau \ge \frac{1}{2} \cdot \sqrt{\beta n}/6 = \sqrt{\beta n}/12$, and the probability is at least $(3/4)^2/3 = (9/16)/3 = 3/16$. This gives Equation 17.5.

The theorem. The idea is to show that a solution with many wrong bits would give a difference vector that no random query catches, and that, with enough queries, no such vector exists.

1. **Consistent candidates differ from d by little on every query.** Both c and d are within E of every answer, so by the triangle inequality $|\langle q_j, c-d\rangle| \le 2E$ for all j, as in the exponential attack.
2. **A wrong bit is a large coordinate.** Rounding c_i gives the wrong bit only if c_i is on the far side of $\frac{1}{2}$ from d_i, that is, if $|c_i - d_i| \ge \frac{1}{2}$. So if the rounding has more than βn wrong bits, $z = c - d$ has more than βn coordinates of size at least $\frac{1}{2}$.
3. **Move to a finite set.** The linear program can return any point of the cube, and the union bound below needs a finite list of candidates. Round each c_i to the nearest multiple of $1/n$, giving c'. Each coordinate moves by at most $\frac{1}{2n}$, and a subset sum adds at most n coordinates, so $|\langle q, c-c'\rangle| \le n \cdot \frac{1}{2n} = \frac{1}{2}$ for every subset q. Hence $z' = c' - d$ satisfies $|\langle q_j, z'\rangle| \le 2E + \frac{1}{2}$ for all j, and still has more than βn coordinates of size at least $\frac{1}{2} - \frac{1}{2n} \ge \frac{1}{3}$ (for $n \ge 3$, since $\frac{1}{2} - \frac{1}{6} = \frac{1}{3}$).
4. **One grid point survives all queries only with small probability.** Fix one grid point c' whose z' has more than βn large coordinates. By the lemma (Equation 17.5), each random query has $|\langle q_j, z'\rangle| > \sqrt{\beta n}/12$ with probability at least $\frac{3}{16}$, and by the assumption $2E + \frac{1}{2} \le \sqrt{\beta n}/12$ such a query violates the bound of step 3. So each query lets z' pass with probability at most $1 - \frac{3}{16} = \frac{13}{16}$. The queries are independent, so z' passes all m of them with probability at most $(13/16)^m$.
5. **Union bound over the grid.** The union bound (Concentration inequalities, Fact 2) says that the probability that at least one of several events happens is at most the sum of their probabilities. Each coordinate of a grid point takes one of the $n+1$ values $0, \frac{1}{n}, \dots, 1$, so there are at most $(n+1)^n$ grid points, and the probability that *some* bad grid point passes every query is at most $(n+1)^n (13/16)^m$. Taking logarithms, this is at most η exactly when $n\ln(n+1) - m\ln(16/13) \le \ln\eta$, that is, when $m\ln(16/13) \ge n\ln(n+1) + \ln(1/\eta)$, which is Equation 17.6.
6. **Conclude.** Outside that event, no bad grid point passes, so by step 3 no linear-program solution with more than βn wrong bits exists. The event depends only on the queries, not on the curator's errors, so the guarantee holds for every curator at once.

17.4.4. Interpretation

The threshold is $\sqrt{n}$ because a random query cannot aim at the wrong coordinates. Its inner product with a difference vector that has k coordinates of constant size fluctuates on the scale $\sqrt{k}$, as step 2 of the lemma computes. An error below that scale cannot hide $k = \beta n$ wrong bits. An error of order $\sqrt{n}$ can, and Section 17.6 constructs a curator that does.

Among difference vectors, the balanced ones are the hard case. If the wrong coordinates of z all have the same sign, $\langle q, z\rangle$ has a large mean and escapes any small band almost surely. If they cancel, the mean is near 0 and only the fluctuation helps. Step 5 of the lemma covers both cases, because it drops the mean and relies on the fluctuation alone. Figure 17.2 shows the two cases.

The constants are poor, but the scaling is right. The theorem asks for E below about $\sqrt{\beta n}/24$, which is what $2E + \frac{1}{2} \le \sqrt{\beta n}/12$ gives after dividing by 2, and for $m \approx 33n$ at $n = 1000$ (the example below). The union bound over $(n+1)^n$ grid points costs the $\ln(n+1)$ factor. Sharper analyses need only $m = O(n)$ queries. Dwork, McSherry and Talwar (2007) go further with a variant of the decoder: instead of requiring every answer to be within E, which a single wildly wrong answer makes impossible, it minimises the total absolute error $\|Ac - a\|_1$, and it tolerates a

constant fraction of answers with arbitrarily large error. In practice, the least-squares decoder of the next section with $m = 4n$ already works at errors of about $0.2\sqrt{n}$.

Random queries are not essential either. Dinur and Nissim's queries were random to make the proof work. Real attacks use whatever queries the system answers. Cohen and Nissim (2020) ran a linear-programming reconstruction against a commercial query system that added noise to its answers, and recovered most of a secret column.

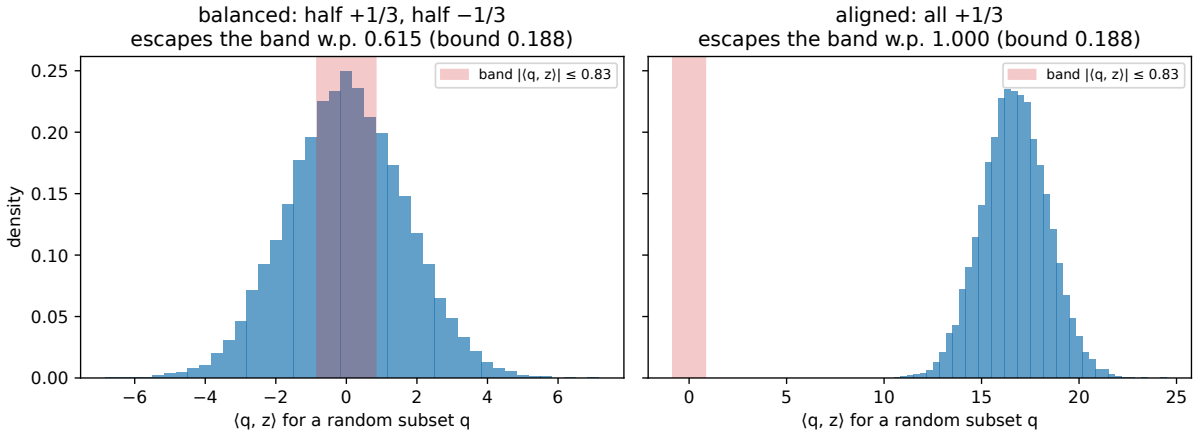


Figure 17.2.: A random subset sum of a difference vector z with $\beta n = 100$ entries of size $\frac{1}{3}$ ($n = 1000$), over 20 000 random subsets, against the band $|\langle q, z\rangle| \le \sqrt{\beta n}/12 = 0.83$ (red). Left: half the entries $+\frac{1}{3}$ and half $-\frac{1}{3}$, so the sum is centred at 0, yet it still leaves the band most of the time. Right: all entries $+\frac{1}{3}$, so the sum is centred far from the band. The lemma guarantees at least $\frac{3}{16} = 0.1875$ in both cases.

17.4.5. Example: what the theorem promises at $n = 1000$

Take $n = 1000$ people and target $\beta = 0.1$, so at most $0.1 \times 1000 = 100$ wrong bits. The lemma's threshold is $\sqrt{\beta n}/12 = \sqrt{100}/12 = 0.8333$. The script estimates the escape probability, the chance that a random subset sum lands outside the band ± 0.8333, over 100 000 random subsets for the two difference vectors of Figure 17.2: 0.6161 for the balanced one and 1.0000 for the aligned one, against the guaranteed 0.1875. The lemma is far from tight for both.

The theorem's condition $2E + \frac{1}{2} \le \sqrt{\beta n}/12$ then allows an error of $E \le \sqrt{\beta n}/24 - \frac{1}{4} = 0.4167 - 0.25 = 0.1667$. With failure probability $\eta = 0.01$, so that $\ln(1/\eta) = \ln 100$, Equation 17.6 asks for

$$m \ge \frac{1000 \ln 1001 + \ln 100}{\ln(16/13)} = 33\,295.04, \tag{17.8}$$

so at least 33 296 queries, about $33n$. Both constants are far from what the attack needs in practice, but they come with a complete proof.

17.4.6. Check yourself

- Why does the derivation round the linear program's solution to a grid before using the union bound? *(The solution can be any point in the cube, an infinite set. The union bound*

needs a finite list of candidates; the grid of step $1/n$ *has* $(n+1)^n$ *points and moves every answer by at most* $\frac{1}{2}$*.)*

- If the curator could see the queries before choosing d, would the theorem still hold? *(The proof fixes* d *before the queries. With* d *chosen after the queries, add a union bound over the* 2^n *databases: the count of bad grid points grows to at most* $2^n(n+1)^n$*, which only adds* $n \ln 2$ *to the numerator of Equation 17.6.)*

17.5. Least squares under worst-case and random error

17.5.1. Context

The linear program needs a solver and a value of E. A simpler decoder needs neither. **Least squares** picks the vector whose answers come closest to the published ones, measured by the sum of squared differences, $\hat{x} = \arg\min_x \|Ax - a\|^2$. Setting the gradient $2A^\top(Ax - a)$ to zero gives the closed form $\hat{x} = (A^\top A)^{-1}A^\top a$. The decoder clips each coordinate to $[0, 1]$ and rounds it. Because $\hat{x}$ is a linear function of the answers, its error is a linear function of the curator's error, so you can analyse it exactly in two cases: errors chosen by an adversary, and independent random noise.

17.5.2. Statement

How much least squares amplifies the curator's error depends on how much A can shrink a vector. The **smallest singular value** $s_{\min}(A)$ is the smallest length $\|Ax\|$ over unit vectors x: the factor by which A shrinks the direction it shrinks most. Assume $m \geq n$ and that A has full column rank, so that $s_{\min}(A) > 0$.

Worst-case error. The decoder's error vector has length at most $\|e\|/s_{\min}(A)$, because undoing A can stretch a vector by at most $1/s_{\min}$. Each wrong bit is off by at least $\frac{1}{2}$, so it uses up at least $\frac{1}{4}$ of the squared length. If $|e_j| \leq E$ for every j, so that $\|e\|^2 \leq mE^2$, the least-squares decoder therefore gets at most

$$\frac{4\|e\|^2}{s_{\min}(A)^2} \leq \frac{4mE^2}{s_{\min}(A)^2} \tag{17.9}$$

bits wrong. For random subsets, $s_{\min}(A) \approx \frac{1}{2}(\sqrt{m} - \sqrt{n})$, so $s_{\min}^2 \approx \frac{m}{4}(1 - \sqrt{n/m})^2$ and the bound becomes $16E^2/(1 - \sqrt{n/m})^2$; at $m = 4n$, $1 - \sqrt{1/4} = \frac{1}{2}$ and it is $64E^2$. So error $E = o(\sqrt{n})$ gives $o(n)$ wrong bits with only $m = 4n$ queries.

Random error. Now let the curator's errors be independent $\mathcal{N}(0, \sigma^2)$, normal with mean 0 and standard deviation σ. Each coordinate of the decoder's error is then also normal, and, as the derivation shows, its variance is about $\frac{4\sigma^2}{m-n}$: the noise σ^2 divided by roughly the number of equations beyond the n needed to determine d, times 4 because a random query includes each person only half the time. A bit is recovered when its error does not push it across $\frac{1}{2}$. Write Φ for the standard normal cumulative distribution function, so $\Phi(t)$ is the probability that a standard normal variable is below t. Each coordinate of $\hat{x} - d$ is approximately $\mathcal{N}(0, \frac{4\sigma^2}{m-n})$, and the expected fraction of bits recovered is approximately

$$\Phi\left(\frac{\sqrt{m-n}}{4\sigma}\right). \tag{17.10}$$

The argument of Φ is the distance $\frac{1}{2}$ to the wrong side, measured in standard deviations $2\sigma/\sqrt{m-n}$.

17.5.3. Derivation

Worst case.

1. **The decoder's error is linear in the curator's.** Substituting $a = Ad + e$, $\hat{x} = (A^\top A)^{-1}A^\top(Ad+e) = d + A^+e$, with $A^+ = (A^\top A)^{-1}A^\top$ the pseudo-inverse. The d comes back exactly, because $(A^\top A)^{-1}A^\top A = I$.
2. **Bound its size.** Write A in its singular value decomposition, $A = U\,\mathrm{diag}(s_1, \dots, s_n)\,V^\top$ with U and V having orthonormal columns. Then $A^+ = V\,\mathrm{diag}(1/s_1, \dots, 1/s_n)\,U^\top$, whose largest singular value is $1/s_{\min}(A)$. So $\|\hat{x} - d\| = \|A^+e\| \le \|e\|/s_{\min}(A)$.
3. **Clipping only helps.** Each d_i is in $[0,1]$, and moving $\hat{x}_i$ to the nearest point of $[0,1]$ cannot move it away from d_i.
4. **Count the wrong bits.** A wrong bit has $|\hat{x}_i - d_i| \ge \frac{1}{2}$, so it contributes at least $\frac{1}{4}$ to $\|\hat{x}-d\|^2$. Hence the number of wrong bits is at most $4\|\hat{x}-d\|^2 \le 4\|e\|^2/s_{\min}^2$, and $\|e\|^2 \le mE^2$ because each of the m errors is at most E in size. This is Equation 17.9.
5. **Estimate the singular value.** For a random matrix with m independent rows, $s_{\min}$ concentrates near $\sqrt{m} - \sqrt{n}$ times the scale of the rows (Vershynin, *High-Dimensional Probability*, ch. 4, for the general statement). Here the rows are $\frac{1}{2}(\mathbf{1} + s)$ with a random-sign vector s. The all-ones part sends every vector orthogonal to $\mathbf{1}$ to 0, so on those directions A acts as $\frac{1}{2}$ times a random-sign matrix, which gives the approximation $\frac{1}{2}(\sqrt{m} - \sqrt{n})$. The script checks it: at $n = 500$, $m = 2000$, $s_{\min} = 11.43$ against $\frac{1}{2}(\sqrt{2000} - \sqrt{500}) = \frac{1}{2}(44.72 - 22.36) = 11.18$. Substituting gives $4mE^2/(\frac{1}{4}(\sqrt{m} - \sqrt{n})^2) = 16E^2/(1 - \sqrt{n/m})^2$.

Random noise. The plan is to find the variance of each coordinate of $\hat{x} - d$, then ask how often that normal error crosses $\frac{1}{2}$.

1. **The error is Gaussian.** $\hat{x} - d = A^+e$ is a linear map of a Gaussian vector, so given A it is Gaussian. A linear map M turns covariance $\sigma^2 I$ into $\sigma^2 MM^\top$, so its covariance is $\sigma^2 A^+(A^+)^\top = \sigma^2(A^\top A)^{-1}A^\top A(A^\top A)^{-1} = \sigma^2(A^\top A)^{-1}$.
2. **Compute the average Gram matrix.** $A^\top A = \sum_j q_j q_j^\top$ is a sum over the m queries. For a random subset q, $q_i^2 = q_i$ because q_i is 0 or 1, so $\mathbb{E}q_i^2 = \frac{1}{2}$; and for $i \ne k$ the two memberships are independent, so $\mathbb{E}q_iq_k = \frac{1}{4}$. Hence $\mathbb{E}[qq^\top] = \frac{1}{4}(I + \mathbf{1}\mathbf{1}^\top)$ and $\mathbb{E}[A^\top A] = \frac{m}{4}(I + \mathbf{1}\mathbf{1}^\top)$.
3. **Invert it.** $(I + \mathbf{1}\mathbf{1}^\top)^{-1} = I - \frac{1}{n+1}\mathbf{1}\mathbf{1}^\top$. To check, multiply out and use $\mathbf{1}^\top\mathbf{1} = n$: the product is $I + \mathbf{1}\mathbf{1}^\top - \frac{1}{n+1}\mathbf{1}\mathbf{1}^\top - \frac{n}{n+1}\mathbf{1}\mathbf{1}^\top = I$ (Exercise 4 in the exercise book). Its diagonal is $1 - \frac{1}{n+1} = \frac{n}{n+1} \approx 1$, so replacing $A^\top A$ by its mean gives a per-coordinate variance of about $\sigma^2 \cdot \frac{4}{m} = 4\sigma^2/m$.
4. **Correct for finite m.** $A^\top A$ fluctuates around its mean, and the inverse of a fluctuating matrix is on average larger than the inverse of its mean, just as $\mathbb{E}[1/Z] \ge 1/\mathbb{E}Z$ for a positive number Z. For Gaussian rows the exact mean is known. A sum of outer products of m independent Gaussian rows with covariance Σ is called a Wishart matrix, and if $A^\top A$ is Wishart with m rows and scale Σ, then $\mathbb{E}[(A^\top A)^{-1}] = \Sigma^{-1}/(m-n-1)$ (the inverse-Wishart mean; Muirhead 1982, ch. 3). Using it with $\Sigma = \mathbb{E}[qq^\top]$ replaces m by $m - n - 1$: the variance is $4\sigma^2\frac{n}{n+1} \cdot \frac{1}{m-n-1} \approx \frac{4\sigma^2}{m-n}$. The rows here are 0/1, not Gaussian, so this is an approximation; the simulations below check it.

5. **Find when a bit goes wrong.** A 0 rounds to 1 when its error exceeds $+\frac{1}{2}$; a 1 rounds to 0 when its error is below $-\frac{1}{2}$. Either way only one tail counts. The error has standard deviation $\frac{2\sigma}{\sqrt{m-n}}$, so the probability is $\Phi(-\frac{1}{2}\big/\frac{2\sigma}{\sqrt{m-n}}) = \Phi(-\frac{\sqrt{m-n}}{4\sigma})$. The fraction recovered is one minus that, which by the symmetry $1-\Phi(-t) = \Phi(t)$ of the normal curve is Equation 17.10.

17.5.4. Interpretation

At a fixed ratio of queries to people, only $\sigma/\sqrt{n}$ matters. With $m = \kappa n$, $\sqrt{m-n} = \sqrt{\kappa-1}\sqrt{n}$, so Equation 17.10 is $\Phi(\sqrt{\kappa-1}/(4\sigma/\sqrt{n}))$. Plotted against $\sigma/\sqrt{n}$, the curves for $n = 100$, 300 and 900 fall on top of each other (Figure 17.3). The scale of the threat is $\sqrt{n}$, as in the theorem.

Noise of order $\sqrt{n}$ is not automatically safe. At $\sigma = 0.25\sqrt{n}$ and $m = 4n$, the argument of Φ is $\sqrt{3}/(4 \times 0.25) = \sqrt{3} \approx 1.73$, and least squares recovers about 96% of the bits. Recovery degrades smoothly as $\sigma/\sqrt{n}$ grows; it does not switch off at a threshold.

More queries average independent noise away. At a fixed σ, Equation 17.10 tends to 1 as m grows, because $m-n$ independent noisy equations average to an error of order $\sigma/\sqrt{m-n}$ per coordinate. To keep the attack at a fixed accuracy, the argument $\sqrt{m-n}/(4\sigma)$ must stay fixed, so the noise must grow like $\sqrt{m}$ (Figure 17.4). This is the second half of the Fundamental Law: a bound on the error of *each* answer is not enough when the number of answers is not bounded.

Worst-case errors hurt more than random ones of the same size. A curator that chooses its errors along the decoder's weakest direction, the right singular vector of $s_{\min}$, which least squares amplifies the most, does much more damage than Gaussian noise with the same scale, whose errors point in all directions equally. The bound Equation 17.9 is loose by more than a factor of 10 in the example below, but it is the only statement that holds for every curator.

17.5.5. Example: $n = 500$ people, $m = 2000$ queries

The script averages over four random databases and query matrices (Table 17.4). Here $\sqrt{n} = \sqrt{500} = 22.36$, which gives the second row.

Table 17.4.: Least-squares reconstruction of $n = 500$ bits from $m = 2000$ random subset sums with Gaussian noise.

$\sigma/\sqrt{n}$	0.1	0.25	0.5	1	2
σ	2.24	5.59	11.18	22.36	44.72
Recovered, simulated	1.0000	0.9655	0.8130	0.6550	0.5875
Recovered, Equation 17.10	1.0000	0.9585	0.8069	0.6676	0.5858

The prediction is within 0.013 of every simulated value. By hand at $\sigma = 0.5\sqrt{n}$: $\sqrt{m-n}/(4\sigma) = \sqrt{1500}/44.72 = 0.866$, and $\Phi(0.866) = 0.807$.

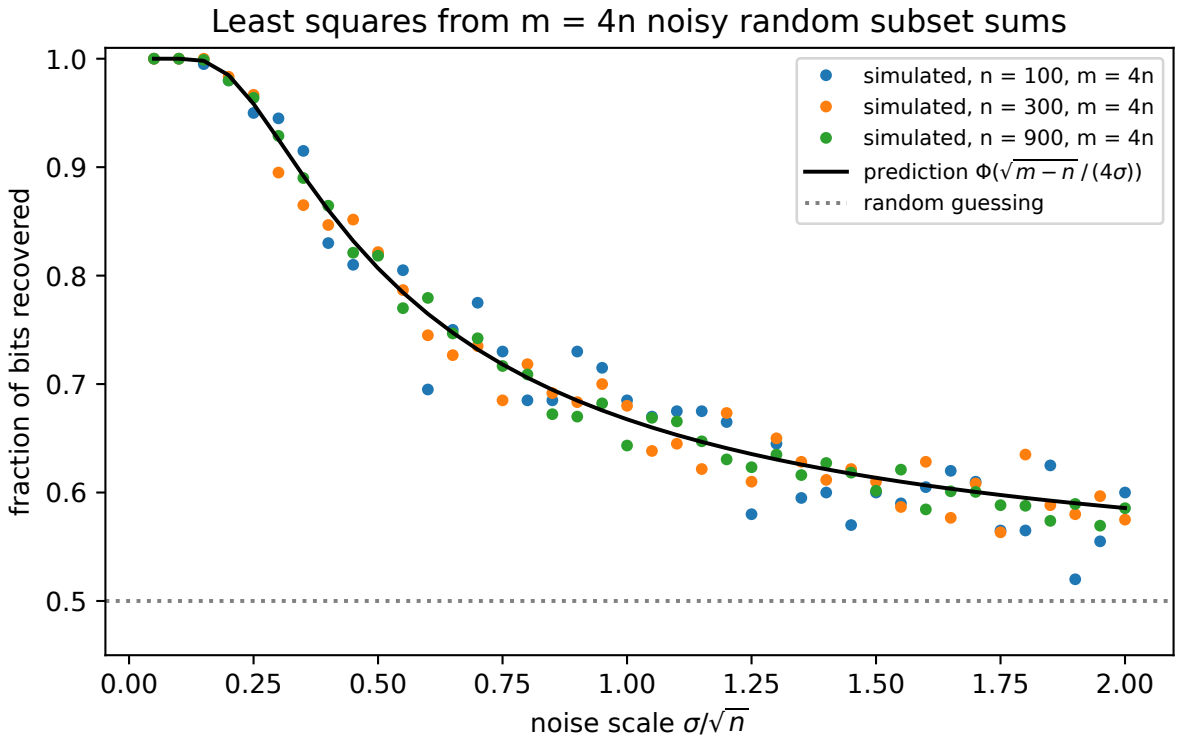


Figure 17.3.: Fraction of bits recovered by least squares from $m = 4n$ noisy random subset sums, against the noise scale in units of $\sqrt{n}$, for three database sizes. The black curve is Equation 17.10. The curves for different n coincide: the attack's success depends on $\sigma/\sqrt{n}$ only.

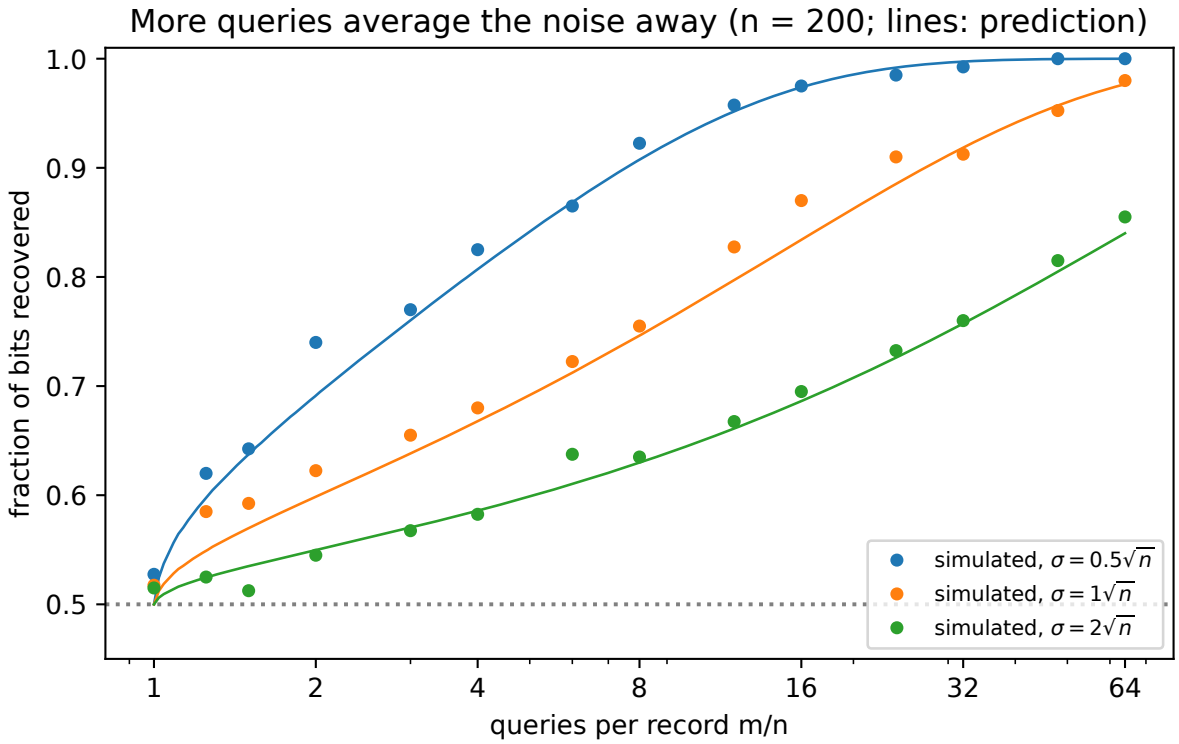


Figure 17.4.: Fraction of bits recovered by least squares at $n = 200$ as the number of queries grows, for three noise levels. Lines are Equation 17.10; dots are simulations. Near $m = n$ the system is barely determined and the noise is amplified; beyond that, more queries average the noise away, and even at $\sigma = 2\sqrt{n}$ the attack recovers most bits with $64n = 12\,800$ queries.

For worst-case errors, the script measures $s_{\min}(A) = 11.43$ and runs a curator that adds $E \cdot \text{sign}(Av)$, with v the weakest right singular vector. That curator pushes every answer by the full E in the direction that least squares turns into the largest error along v. At $E = 1$, the bound Equation 17.9 allows $4 \times 2000 \times 1/11.43^2 = 61.2$ wrong bits, and this curator causes 0. At $E = 2$, the bound is four times larger, 245.0, and this curator causes 20. At $E = 0.25\sqrt{n} = 5.59$, the bound (1914.0) exceeds n and says nothing, while this curator causes 143 wrong bits, against about 17 for Gaussian noise with $\sigma = 5.59$ ($1 - 0.9655$ of 500).

17.5.6. Check yourself

- At $m = n$ with exact answers, least squares recovers d exactly. Why does the prediction Equation 17.10 fail there with noise? *(At $m = n$ the system is square and $s_{\min}$ is close to 0, so tiny errors are amplified enormously. The factor $1/(m - n - 1)$ in the variance blows up; Figure 17.4 shows recovery near $\frac{1}{2}$ there.)*
- A curator adds independent noise with $\sigma = 2\sqrt{n}$ to every answer and lets anyone ask as many questions as they want. How many queries per person does least squares need to recover 90% of the bits? *(Solve $\Phi(\sqrt{m - n}/(4\sigma)) = 0.9$. The standard normal is below 1.2816 with probability 0.9, so $\sqrt{m - n}/(4\sigma) = 1.2816$; dividing through by $\sqrt{n}$ gives $\sqrt{m/n - 1} = 4 \times 2 \times 1.2816$, so $m/n = 106.1$.)*

17.6. The Fundamental Law of Information Recovery

17.6.1. Context

The attacks so far say when answers are too accurate. Two questions remain. Is error of order $\sqrt{n}$ really enough to stop reconstruction, or is it just where the proofs stop? And how many queries does *any* attack need? This section answers both, with one construction and one counting argument, and collects the results in the form Dwork and Roth call the Fundamental Law of Information Recovery: overly accurate answers to too many questions destroy privacy, and they do so spectacularly.

17.6.2. Statement

Table 17.5 collects the regimes. The first three rows are the attacks of the previous sections; the last two are the converses proved in this section.

Table 17.5.: The regimes of reconstruction from subset sums. The first three rows are attacks; the last two are limits on attacks.

Queries	Error per answer	Outcome	Source
All 2^n subsets	at most E	at most $4E$ wrong bits; $E = o(n)$ is blatantly non-private	Equation 17.3

Queries	Error per answer	Outcome	Source
$O(n \log n)$ random subsets	at most E, $E = o(\sqrt{n})$	$o(n)$ wrong bits, by a linear program	Equation 17.6
m random subsets	independent $\mathcal{N}(0, \sigma^2)$	fraction recovered $\approx \Phi(\sqrt{m-n}/(4\sigma))$, tending to 1 as m grows	Equation 17.10
m arbitrary	$O(\sqrt{n \log m})$, sampling curator	no attack recovers more than $\frac{3}{4}$ of the bits	Equation 17.11
m, each answer one of V values	any	at least $n(1-h(\beta))/\log_2 V$ queries are needed for βn wrong bits	Equation 17.12

A sampling curator. To show that error of order $\sqrt{n}$ can stop reconstruction, you need a curator whose errors are that small yet hide many bits completely. The idea is to answer from a random sample of the people and scale up. The curator draws a secret set T, keeping each person independently with probability p, and answers every query by $a_j = \frac{1}{p} \sum_{i \in S_j \cap T} d_i$. Dividing by p makes each answer correct on average, and its error is a sum of independent zero-mean terms, one per person, so it is of order $\sqrt{n}$. People outside T never affect any answer, so nothing reveals their bits. Then, for a failure probability η, with probability at least $1 - \eta$ all m answers have error at most

$$\max_j |a_j - \langle q_j, d \rangle| \leq \sqrt{\frac{n \ln(2m/\eta)}{2p^2}}, \tag{17.11}$$

and if d is uniform, no attacker recovers an expected fraction of more than $p + \frac{1-p}{2}$ of the bits; at $p = \frac{1}{2}$, $\frac{1}{2} + \frac{1}{4}$, three quarters.

An information bound. To bound the number of queries any attack needs, count bits of information. The secret is n uniform bits, so it carries n bits of information. An answer that takes at most V values can carry at most $\log_2 V$ bits. A guess that is wrong on a fraction β of the bits still leaves some uncertainty, measured by the **binary entropy** $h(\beta) = -\beta \log_2 \beta - (1-\beta) \log_2 (1-\beta)$, the uncertainty in bits of a coin that comes up heads with probability β. So the answers must carry at least $n(1 - h(\beta))$ bits. Precisely, let d be uniform on $\{0,1\}^n$, and let each of the m answers take at most V values (for example, exact counts take $V = n + 1$ values). If an attacker's guess $\hat{d}$ has an expected fraction $\beta \leq \frac{1}{2}$ of wrong bits, then

$$m \geq \frac{n\,(1 - h(\beta))}{\log_2 V}, \tag{17.12}$$

where h is the binary entropy function in bits.

17.6.3. Derivation

The sampling curator's error (Hoeffding and the union bound). The plan is to bound one answer's error with Hoeffding's inequality, then all m answers at once with the union bound.

1. **Write the error as a sum.** With $t_i = \mathbf{1}[i \in T]$, the answer is $\sum_{i \in S_j} d_i t_i / p$, so the error of query j is $\sum_{i \in S_j} d_i (\frac{t_i}{p} - 1)$. Only the $k \le n$ people with $d_i = 1$ in S_j contribute. Their terms are independent, have mean $p \cdot \frac{1}{p} - 1 = 0$, and lie in $[-1, \frac{1}{p} - 1]$, an interval of length $\frac{1}{p}$.
2. **Apply Hoeffding's inequality to the sum.** Hoeffding's inequality (Concentration inequalities) says that the mean of independent bounded terms is unlikely to stray far from its expectation, with a tail that falls like a Gaussian's. For a mean $\bar{X}$ of k independent terms with range $\frac{1}{p}$ and expectation μ, it reads $\Pr[\bar{X} - \mu \ge t] \le \exp(-2kt^2p^2)$. A sum deviates by u when the mean deviates by u/k, so substituting $t = u/k$ and counting both tails, $\Pr[|\text{error}| \ge u] \le 2\exp(-2p^2u^2/k) \le 2\exp(-2p^2u^2/n)$, the last step because $k \le n$.
3. **Union bound over the m queries.** The probability that some answer has error at least u is at most the sum over the m queries: $\Pr[\text{some error} \ge u] \le 2m\exp(-2p^2u^2/n)$. Set this equal to η and solve: $2p^2u^2/n = \ln(2m/\eta)$, so $u^2 = n\ln(2m/\eta)/(2p^2)$, which is Equation 17.11.

What the attacker can learn. The idea is that a person outside T is invisible to every answer.

1. **The answers ignore the people outside T.** They are a function of T and of d_i for $i \in T$ only.
2. **So a person outside T is a fair coin to the attacker.** Given $i \notin T$, the answers are independent of d_i, which is a fair bit independent of everything else. Any guess of d_i is right with probability $\frac{1}{2}$.
3. **Average over who is in T.** Each person is in T with probability p, and at best the attacker gets all of those right. Each person is outside with probability $1 - p$, and then the attacker is right half the time. The expected fraction recovered is at most $p \cdot 1 + (1-p) \cdot \frac{1}{2} = p + \frac{1-p}{2}$.

The information bound (entropy and Fano). Let $Y = (a_1, \dots, a_m)$ be the answers, with the queries fixed. The **mutual information** $I(d; Y)$ is the number of bits the answers reveal about d: the uncertainty about d before seeing Y minus the uncertainty after, $H(d) - H(d \mid Y)$, where H is entropy. The plan is to bound it from above by what the answers can carry and from below by what a good guess requires.

1. **Bound what the answers can carry.** $I(d; Y) \le H(Y) \le \sum_j H(a_j) \le m \log_2 V$. Mutual information is at most the entropy of the output, because it is $H(Y)$ minus the non-negative $H(Y \mid d)$. The entropy of a tuple is at most the sum of the entropies, with equality only when the entries are independent. And a variable with V values has entropy at most $\log_2 V$ bits, attained when all values are equally likely.
2. **Processing cannot add information.** $\hat{d}$ is computed from Y, so $I(d; \hat{d}) \le I(d; Y)$ by the DPI, which says that no function of Y, random or not, can tell you more about d than Y itself.
3. **Write what a good guess requires.** $I(d; \hat{d}) = H(d) - H(d \mid \hat{d}) = n - H(d \mid \hat{d})$, since n uniform bits have entropy n. To bound the remaining uncertainty bit by bit, use the chain rule, $H(d \mid \hat{d}) = \sum_i H(d_i \mid \hat{d}, d_1, \dots, d_{i-1})$, and the fact that conditioning on less cannot reduce entropy: dropping $d_1, \dots, d_{i-1}$ and then all of $\hat{d}$ except $\hat{d}_i$ gives $H(d \mid \hat{d}) \le \sum_i H(d_i \mid \hat{d}) \le \sum_i H(d_i \mid \hat{d}_i)$.

4. **Apply Fano for one bit.** With two values, Fano's inequality reads $H(d_i \mid \hat{d}_i) \le h(P_i)$, where $P_i = \Pr[\hat{d}_i \neq d_i]$. The reason is short: if you knew both the guess $\hat{d}_i$ and whether it was wrong, you would know d_i. So the uncertainty about d_i given $\hat{d}_i$ is at most the uncertainty about "was the guess wrong?", a coin that comes up "wrong" with probability P_i, whose entropy is at most $h(P_i)$.
5. **Average with concavity.** h is concave (its graph bends downward), so by Jensen's inequality the average of its values is at most its value at the average: $\frac{1}{n}\sum_i h(P_i) \le h(\beta)$, where $\beta = \frac{1}{n}\sum_i P_i \le \frac{1}{2}$ is the expected fraction of wrong bits.
6. **Combine.** Steps 3 to 5 give $I(d;\hat{d}) \ge n - nh(\beta)$, and steps 1 and 2 give $I(d;\hat{d}) \le m \log_2 V$. So $n - nh(\beta) \le I(d;\hat{d}) \le m \log_2 V$, which is Equation 17.12.

17.6.4. Interpretation

Error of order $\sqrt{n}$ is necessary to stop reconstruction, and it can be enough. The sampling curator's error is $O(\sqrt{n \log m})$, only a logarithm above the $o(\sqrt{n})$ that the attacks need, and it caps every attacker at $\frac{3}{4}$ of the bits.

But it is not privacy. The same curator answers exactly for the people in T: an attacker with enough queries learns their bits perfectly, because least squares recovers d_i/p for every $i \in T$. Half the people are fully exposed so that the other half can be fully hidden. Stopping blatant reconstruction is a much weaker goal than protecting everyone, and Part 3 defines a guarantee that holds for every individual.

The attacks use about as few queries as possible. With exact counts, Equation 17.12 asks for at least $n(1-h(\beta))/\log_2(n+1)$ queries, of order $n/\log n$, and least squares with exact answers uses n. Answers with error, published to a coarser precision, take fewer values, carry fewer bits each and need more queries.

The law has two halves, and a defence must control both. "Overly accurate" is the error scale: below $\sqrt{n}$ for random queries, below n for all queries. "Too many" is the count: independent noise per answer is averaged away unless it grows with the number of answers. That is why Part 3 tracks a budget that is spent across queries.

17.6.5. Example: three curators

The sampling curator at $n = 500$, $m = 2000$. With $p = \frac{1}{2}$ the script keeps 250 of the 500 people. The largest error over the 2000 queries is 28.00, or $28.00/\sqrt{500} = 1.25\sqrt{n}$. That is well under the Hoeffding bound at $\eta = 0.05$: with $p = \frac{1}{2}$, $2p^2 = \frac{1}{2}$, so Equation 17.11 is $\sqrt{2 \cdot 500 \ln(4000/0.05)} = 106.25$. (Hoeffding assumes the worst-case spread of each term; the actual terms have a smaller variance.) Least squares on these answers recovers 0.7560 of the bits, right at the ceiling of $\frac{3}{4}$.

The information bound at $n = 1000$. For at most 5% wrong bits, $h(0.05) = 0.2864$, so each person still needs $1 - 0.2864 = 0.7136$ bits. Exact answers ($V = 1001$) carry at most $\log_2 1001 \approx 9.97$ bits each and need $m \ge 1000 \times 0.7136/\log_2 1001 = 71.6$ queries, so at least 72. If the curator rounds every count to a multiple of 10, an answer takes at most $\lfloor 1000/10 \rfloor + 2 = 102$ values (the 101 multiples of 10 from 0 to 1000, plus one to be safe; the script uses $\lfloor n/w \rfloor + 2$ for every

width w), and the bound rises to 106.9. Rounding to multiples of 32 leaves $\lfloor 1000/32 \rfloor + 2 = 33$ values and a bound of 141.5. These are lower bounds for any attack, whatever its running time.

Independent noise with unlimited queries. From Table 17.4, a curator with $\sigma = 2\sqrt{n}$ lets least squares recover 0.5875 of the bits at $m = 4n$. If it answers any number of queries with the same noise level, an attacker who asks $m = 106.1\,n$ queries recovers 90% (the second Check yourself question of Section 17.5).

17.6.6. Check yourself

- Why does the information bound need the answers to take finitely many values? *(Step 1 bounds $H(a_j)$ by $\log_2 V$. A real-valued answer can carry unbounded information in its low-order digits, so the count of values is what limits it. Any published table has finite precision.)*
- A curator answers each query with an independent error of size $\sqrt{n \log m}$ chosen at random. Is it as safe as the sampling curator? *(No. Its errors are independent across queries, so least squares averages them away as m grows. The sampling curator's errors come from one secret set T shared by all answers, so asking more never reveals the people outside T.)*

17.7. Reconstructing a census block

17.7.1. Context

National statistical offices publish many tables computed from the same confidential records: counts by age, sex and race for every block, the same counts by household type, medians and means, and so on. Each table cell is a query. Garfinkel, Abowd and Martindale (2019) showed with a toy block of seven people that a modest set of published statistics can determine every person's age, sex, race and marital status, and that a constraint solver finds the solution in seconds. By then the US Census Bureau had already run the attack on its own 2010 publications at national scale, an experiment Abowd (2021) later described in detail.

This section repeats the toy experiment with a block of our own. It is a reconstruction game where the queries are not random subsets but the tables an agency actually publishes, and where the decoder is a search over all databases consistent with them.

17.7.2. Statement

A fictional block has seven people, each with an age (a whole number from 0 to 99), a sex, a race (Black or White) and a marital status, and nobody under 18 is married. Table 17.6 is everything the agency publishes about the block. Means are given to two decimals, which is enough to recover the sum of ages in each group exactly: the sum is a whole number, and $34.29 \times 7 = 240.03$, so the total is 240.

Table 17.6.: The published statistics for a fictional census block of seven people. A median over an even count is the average of the two middle ages.

Group	Count	Median age	Mean age
Total	7	33	34.29
Female	4	31	30.25
Male	3	33	39.67
Black	4	33.5	36.25
White	3	33	31.67
Married	4	43.5	47.75

Each statistic is a constraint on the unknown ages and labels, and the claim is that together they leave no freedom at all.

Claim. Exactly one block of seven people is consistent with Table 17.6, and it is the true one (Table 17.7).

Table 17.7.: The block reconstructed from Table 17.6, which is the true block.

Person	1	2	3	4	5	6	7
Age	7	15	27	33	35	52	71
Sex	F	M	F	M	F	F	M
Race	B	B	W	W	W	B	B
Married	no	no	no	yes	yes	yes	yes

17.7.3. Derivation

The decoder enumerates candidate blocks and discards those that contradict a published number. Each step uses the statistics in a way you can check by hand.

1. **Turn means into sums.** Mean $\times$ count gives the sum of ages: total 240, female 121, male 119, Black 145, White 95, married 191. The sums for female and male add to 240, so the male mean is implied by the other two; the same holds for Black and White. Single people number $7 - 4 = 3$ and their ages sum to $240 - 191 = 49$.
2. **Enumerate the women's ages.** Sort them $a \le b \le c \le d$. With four women the median is the average of the two middle ages, so $b + c = 2 \times 31 = 62$, and the sum gives $a + d = 121 - 62 = 59$. Choose $b \le 31$ (so that $c = 62 - b \ge b$) and $a \le b$; then $d = 59 - a$ must be at least $c = 62 - b$, that is $a \le b - 3$. For each b from 3 to 31 that leaves $a = 0, \dots, b - 3$, which is $b - 2$ choices, so there are $1 + 2 + \cdots + 29 = \sum_{k=1}^{29} k = 435$ age lists in all.
3. **Enumerate the men's ages.** Three sorted ages with median 33 and sum 119: the middle one is 33, so $x_1 + x_3 = 119 - 33 = 86$ with $x_1 \le 33 \le x_3 \le 99$. Every x_1 from 0 to 33 gives $x_3 = 86 - x_1$ between 53 and 86, which is allowed, so x_1 ranges over $0, \dots, 33$: 34 lists.
4. **Combine and check the total median.** The $435 \times 34 = 14\,790$ combined age vectors are all distinct; 12 906 have overall median 33.

5. **Check the race split.** For each age vector, try every way of choosing 4 of the 7 people as Black ($\binom{7}{4} = 35$ ways), and keep it if the Black ages have median 33.5 and sum 145 and the White ages have median 33 and sum 95. Only 387 age vectors admit such a split.
6. **Check the marital split** in the same way (married median 43.5, sum 191, everyone married at least 18): 33 age vectors admit one.
7. **Require all three at once.** Exactly one age vector admits a valid sex split, race split and marital split, and its splits are unique. Assigning each person the labels of the splits gives Table 17.7.

Each published statistic removes most of the remaining candidates. The block is not determined by any one table, but by their intersection.

17.7.4. Interpretation

Withholding one number rarely helps. Figure 17.5 repeats the search with each of the twelve medians and means withheld in turn. Eight of the twelve leave the block uniquely determined, because the remaining statistics imply the missing one or pin the block down without it (Exercise 8 in the exercise book). The worst case for the attacker, withholding the married mean, still leaves only 45 consistent blocks, and 51% of them have the true sex, race and marital pattern of all seven people.

Real publications are far more exposed than this toy, because published tables overlap. Real tables cross-classify many variables at many geographic levels, so the same people appear in thousands of cells. That is exactly the "many queries" condition of the Fundamental Law. The main protection of the 2010 Census, swapping a small fraction of households between blocks, changes some answers but leaves most constraints exact. Coarsening (age bands instead of single years) and suppressing small cells, common in other tables, have the same weakness: they remove some constraints and leave the rest exact.

The attack also works at scale. The Census Bureau's own reconstruction of the 2010 Census, described by Abowd (2021), rebuilt a record for each of the 308,745,538 people counted, with block, sex, age, race and ethnicity. About 46% of the reconstructed records matched a real person exactly on all five fields, and linking them to commercial data confirmed the identity of about 17% of the population. The Bureau's response was to change its disclosure-avoidance method for the 2020 Census to one based on the privacy definition of Part 3.

17.7.5. Example: what each withheld statistic leaves

Table 17.8 lists, for each statistic whose removal leaves more than one block, how many blocks remain, how many different sex/race/marital patterns they show, and what share of them has the true pattern.

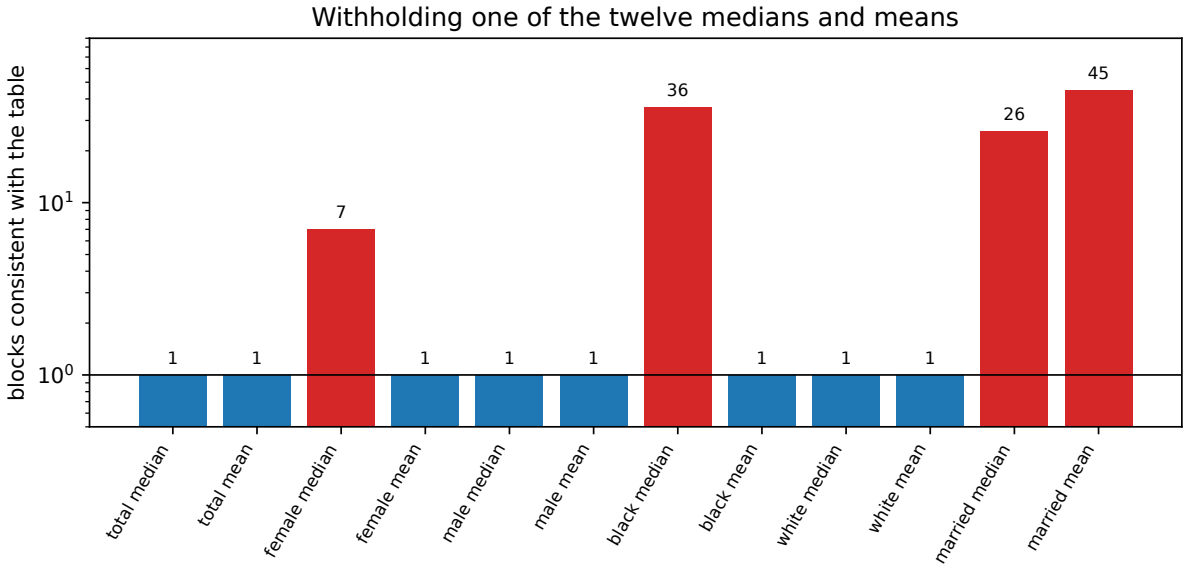


Figure 17.5.: Number of seven-person blocks consistent with Table 17.6 when one median or mean is withheld (log scale). Blue bars: the block is still unique. Red bars: a few candidates remain, still far fewer than the possible blocks.

Table 17.8.: Blocks consistent with the published table when one statistic is withheld.

Withheld	Consistent blocks	Distinct sex/race/marital patterns	Share with the true pattern
Female median	7	6	0.14
Black median	36	3	0.64
Married median	26	4	0.46
Married mean	45	7	0.51
Any of the other eight	1	1	1.00

Withholding the married mean hurts the attacker most, because it is the only statistic about the single people's ages: without it, the married and single sums are no longer known. Withholding the female mean changes nothing, because the total and male sums imply it: it equals $(240 - 119)/4$.

17.7.6. Check yourself

- Why can the attacker turn a mean published to two decimals into an exact sum? *(The sum is a whole number and the mean is the sum over a count of at most 7. Two decimals locate the sum to within* $7 \times 0.005 = 0.035$*, which leaves one whole number.)*
- If the agency published only counts and no ages, could the attacker still reconstruct something? *(Yes: counts of every cross-classification by sex, race and marital status fix how many people fall in each combination, which is the attribute pattern. Ages need age statistics, which real tables publish in single years for small groups.)*

17.8. Code: check every attack by computation and simulation

The script code/reconstruction.py computes every number in this chapter and draws every figure. Each attack, bound and curator is a small typed function, and the figures loop over them. It uses NumPy only and runs in well under a minute. From the repository root:

```
uv run python code/part02-attacks/reconstruction.py
```

Output (seed 0), summarized in Table 17.9:

Table 17.9.: The script's output with the default seed.

Experiment	Result
Fact 1, $d = (1, 0, 1, 1)$	The 16 subset sums, in binary order: 0, 1, 0, 1, 1, 2, 1, 2, 1, 2, 1, 2, 2, 3, 2, 3
Exponential attack, $n = 10$	Survivors 1, 35, 242, 630, 912 and largest distance 0, 2, 4, 6, 8 for $E = 0, 0.5, 1, 1.5, 2$; rounding to multiples of 2 or 4: one survivor
Fact 2	$\mathbb{E}(s_1 + s_2 + s_3)^4 = 21$ by enumeration and by formula
Lemma, $n = 1000$, $\beta = 0.1$	Threshold 0.8333; escape probability 0.6161 (balanced), 1.0000 (aligned), bound 0.1875
Theorem, $n = 1000$, $\beta = 0.1$, $\eta = 0.01$	$E \leq 0.1667$; $m \geq 33\,296$ ($33.3n$)
Least squares, Gaussian noise, $n = 500$, $m = 2000$	Table 17.4; m/n for 90% recovery: 106.1 at $\sigma = 2\sqrt{n}$, 657.9 at $5\sqrt{n}$
Least squares, worst case, $n = 500$, $m = 2000$	$s_{\min} = 11.43$ (approximation 11.18); wrong bits 0, 20, 143 for $E = 1, 2, 5.59$, bounds 61.2, 245.0, 1914.0
Sampling curator, $p = \frac{1}{2}$	Kept 250 of 500; max error 28.00 ($1.25\sqrt{n}$); Hoeffding bound 106.25; recovered 0.7560
Information bound, $n = 1000$	$\beta = 0.05$: 71.6 (exact), 106.9 (rounded to 10), 141.5 (rounded to 32); $\beta = 0.01$: 92.2 (exact)
Census block	435 female and 34 male age lists; 14 790 age vectors; 12 906 after the total median; 387 with a race split; 33 with a marital split; 1 block; Table 17.8

tests/test_reconstruction.py checks that the output still matches.

What to take from it. Five things stand out. First, the exponential attack's bound is exact: every surviving candidate is within $4E$ of d, and some candidate is at exactly $4E$, so no better guarantee is possible for that attack. Second, the proved constants are pessimistic, but the scaling is not. The theorem tolerates only $E \leq 0.17$ at $n = 1000$, yet least squares with $m = 4n$ recovers 96.55% of 500 bits at $\sigma = 5.59$, because both are governed by the same $\sqrt{n}$ scale and only the proof's constants differ. Third, the least-squares prediction is accurate: Equation 17.10 is within 0.013 of the simulation at every noise level, once the $m - n$ correction is included. Fourth, the

sampling curator meets its ceiling: its answers have error $1.25\sqrt{n}$, and the best decoder gets exactly the people in its sample, plus half of the rest by chance. Fifth, published tables are systems of equations: eighteen numbers determine seven people's ages, sexes, races and marital statuses.

17.9. References

- Dinur & Nissim, "Revealing Information while Preserving Privacy" (PODS 2003). The original attacks: all subset sums with error $o(n)$, and polynomially many random subset sums with error $o(\sqrt{n})$ decoded by a linear program. The "blatantly non-private" definition is theirs.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 8. The reconstruction attacks as lower bounds, and the Fundamental Law of Information Recovery.
- Dwork, McSherry & Talwar, "The Price of Privacy and the Limits of LP Decoding" (STOC 2007). The linear-programming decoder tolerates a constant fraction of arbitrarily wrong answers, with $O(n)$ queries.
- Dwork & Yekhanin, "New Efficient Attacks on Statistical Disclosure Control Mechanisms" (CRYPTO 2008). Attacks with structured queries and fast decoders.
- Kasiviswanathan, Rudelson & Smith, "The Power of Linear Reconstruction Attacks" (SODA 2013). Least-squares and linear-programming reconstruction from released statistics such as marginal tables, with the role of the smallest singular value made precise.
- Dwork, Smith, Steinke & Ullman, "Exposed! A Survey of Attacks on Private Data" (Annual Review of Statistics and Its Application, 2017). A readable survey of reconstruction and tracing attacks; the best next step after this chapter.
- Cohen & Nissim, "Linear Program Reconstruction in Practice" (Journal of Privacy and Confidentiality, 2020). A linear-programming reconstruction against a deployed query system that added noise to its answers.
- Garfinkel, Abowd & Martindale, "Understanding Database Reconstruction Attacks on Public Data" (CACM 2019). The seven-person block reconstructed with a solver; the model for Section 17.7.
- Abowd, declaration in *State of Alabama v. US Department of Commerce* (2021), Appendix B. The Census Bureau's reconstruction and re-identification experiment on the 2010 Census.
- Vershynin, *High-Dimensional Probability* (2018), ch. 4. Singular values of random matrices, used in the worst-case least-squares bound. Muirhead, *Aspects of Multivariate Statistical Theory* (1982), ch. 3, for the inverse-Wishart mean.

17.10. Cheat sheet

$d \in \{0,1\}^n$ is the secret, A the $m \times n$ 0/1 query matrix, $a = Ad + e$ the answers, E a bound on every $|e_j|$, σ the scale of random noise, β a fraction of wrong bits, η a failure probability. Table 17.10 collects the results.

Table 17.10.: Every result in this chapter.

Result	Statement	Proof idea
Exponential attack	All 2^n subsets, error E: any consistent c has $\operatorname{dist}(c,d) \le 4E$; tight	Triangle inequality on S_0 and S_1
Paley–Zygmund	$\Pr[W > \theta\,\mathbb{E}W] \ge (1-\theta)^2(\mathbb{E}W)^2/\mathbb{E}W^2$	Split $\mathbb{E}W$; Cauchy–Schwarz
Anti-concentration lemma	βn entries of z at least $\frac{1}{3}$: $\Pr[\lvert\langle q,z\rangle\rvert > \sqrt{\beta n}/12] \ge \frac{3}{16}$	Fourth moment $\le 3\times$ squared second moment
Linear-programming decoder	$2E + \frac{1}{2} \le \sqrt{\beta n}/12$ and $m \ge \frac{n\ln(n+1)+\ln(1/\eta)}{\ln(16/13)}$: at most βn wrong bits	Grid of step $1/n$, lemma, union bound
Least squares, worst case	Wrong bits $\le 4mE^2/s_{\min}^2 \approx 16E^2/(1-\sqrt{n/m})^2$	$\lVert A^+ e\rVert \le \lVert e\rVert/s_{\min}$
Least squares, Gaussian noise	Fraction recovered $\approx \Phi(\sqrt{m-n}/(4\sigma))$	Covariance $\sigma^2(A^\top A)^{-1}$; inverse-Wishart mean
Sampling curator	Error $\le \sqrt{n\ln(2m/\eta)/(2p^2)}$; recovery $\le p + \frac{1-p}{2}$	Hoeffding, union bound; unsampled bits are independent of the answers
Information bound	$m \ge n(1-h(\beta))/\log_2 V$	DPI, chain rule, Fano for one bit, concavity of h

Rules of thumb

- The threat scale of a random subset-sum query is $\sqrt{n}$; of the worst query, n. Errors well below that scale reveal the database.
- Independent noise per answer is averaged away by more answers: to keep reconstruction at bay, the noise must grow like $\sqrt{m}$.
- A curator that stops blatant reconstruction can still expose many individuals completely; stopping reconstruction is not the same as privacy.
- Exact counts carry at most $\log_2(n+1)$ bits each, so about $n/\log n$ of them can already determine n bits.
- Every published table cell is a constraint. Many overlapping tables of exact or nearly exact statistics are a solvable system.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

Part III.

Differential privacy foundations

The previous part showed that models and statistics leak: membership, attributes, training text, whole datasets. This part introduces the defence that the rest of the book is built on, **differential privacy (DP)**, and develops it far enough that you can prove a new mechanism private, calibrate its noise, compose it with others, and say what any such mechanism must cost. The finer tools for tracking privacy over many steps (Rényi DP, numerical accountants, amplification by subsampling) come in the next part, which builds directly on this one.

Who this part is for

You are expected to have read the Prerequisites and Threat models and attacks. This part uses the max-divergence and the hockey-stick divergence, the data processing inequality, the trade-off function of a hypothesis test, and tail bounds for the Laplace and normal distributions, all derived in the prerequisites. It uses the games and the advantage of the attacks part, since the definition of DP is a bound on exactly those games.

The common thread

A randomized algorithm is differentially private when changing one person's record changes the distribution of its output by at most a factor e^{ε}, up to a small slack δ. Everything in this part follows from reading that one sentence in three ways.

- **As a bound on a divergence.** Pure ε-DP bounds the max-divergence between the outputs on neighbouring datasets, and (ε, δ)-DP bounds their hockey-stick divergence. The data processing inequality then gives post-processing for free, and the **privacy loss** $L_{D,D'}$, the log of the likelihood ratio, is the quantity whose sum governs composition.
- **As a bound on every attack.** By the hypothesis-testing view, $\mathrm{FPR} + e^{\varepsilon}\,\mathrm{FNR} \geq 1 - \delta$ for any test of "was this record used". Every game of the attacks part, from membership inference to reconstruction, is therefore capped by a number you can compute before the data is touched.
- **As a budget to spend.** Privacy is lost a little at each release. Noise is calibrated to how much one record can move the answer (its **sensitivity**), losses add up under composition, and lower bounds say how much accuracy no mechanism can avoid paying.

Table 17.11 lists the chapters.

Table 17.11.: The chapters of this part, the question each one answers, and the earlier chapters it uses, by number (chapters 1–8 are the prerequisites, 9–17 the attacks part).

Chapter	The question it answers	Builds on
The definition of differential privacy	What exactly does DP promise, to whom, and what does it not protect?	3, 4, 9, 10
Post-processing, group privacy and composition	Which operations keep a guarantee, and how do guarantees add up?	3, 18

Chapter	The question it answers	Builds on
The Laplace and Gaussian mechanisms	How much noise does a numerical query need, and what is the exact privacy of Gaussian noise?	1, 2, 3, 18, 19
Advanced and optimal composition	How does privacy degrade over many releases, exactly?	2, 3, 4, 18, 19
Selection: exponential mechanism, report-noisy-max, sparse vector	How do you privately choose the best of many options, or find the few queries above a threshold?	1, 2, 18–20
Beyond global sensitivity	What if the worst case is far from the data you actually have?	1, 2, 3, 9, 18–20
Discrete and floating-point-safe mechanisms	Why do textbook mechanisms break on real computers, and how is that fixed?	1, 3, 18, 20
Local, shuffle and distributed trust models	What changes when no one is trusted with the raw data?	2, 8, 9, 18, 20
Lower bounds: what privacy costs	How much accuracy must every private mechanism give up?	2, 4, 5, 10, 17, 18–20

How to read this part

Order. Chapters 18 to 21 are the core and should be read in order: the definition, the properties that make it usable, the two mechanisms everything else is built from, and advanced and optimal composition, which finishes the story of composition that chapter 19 starts and introduces the privacy loss random variable that the next part uses throughout. Chapters 22 and 23 add mechanisms, for selection and for data whose worst case is far from typical. Chapter 24 is short and practical: read it before you implement any mechanism yourself. Chapter 25 changes who is trusted with the raw data. Chapter 26 is the most theoretical; it ties back to reconstruction and membership attacks and closes the part with what privacy costs.

Structure of a chapter. Every chapter has the same shape as in the earlier parts: why it matters, a toolkit, one section per result with a statement, derivation, interpretation, worked example and two *check yourself* questions, then code, references and a cheat sheet. Exercises with worked solutions are collected in the exercise book, a companion website, with a link at the end of each chapter.

Proofs you can check by computer. Most privacy claims in this part are checked numerically as well as proved: the exact privacy curve of a mechanism is computed and compared with its bound, a composition formula is compared with brute-force enumeration, and a buggy mechanism is shown to leak by computing its privacy loss. Running the scripts is the fastest way to see how loose each bound is. From the repository root, for example,

```
uv run python code/part03-dp-foundations/laplace_gaussian.py
```

Notation

The chapters share the conventions in Table 17.12, set in The definition of differential privacy. The next part uses the same notation, except that σ there is the noise multiplier (the noise standard deviation per unit of sensitivity) and an attacker's two error rates are written a and b.

Table 17.12.: Notation shared by the chapters of this part and the next.

Symbol	Meaning
M	A randomized mechanism, mapping a dataset to a random output
$D \simeq D'$	Neighbouring datasets: by default one is the other with one record added or removed; "replace one record" is named explicitly when used
$D' = D \cup \{x\}$	Under add/remove, D' is the dataset that contains the record x and D the one without it. A membership test says "D'" when it guesses that the record is in; its false positive rate is the chance of saying "D'" when the data were D, and its true positive rate the chance of saying "D'" when they were D'
ε, δ	The privacy parameters of (ε, δ)-DP; pure DP is $\delta = 0$
$\Delta_1 f$, $\Delta_2 f$	The ℓ_1 and ℓ_2 sensitivities of a query f
$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}$	The privacy loss of output y; $L_{D,D'}(Y)$ with $Y \sim M(D)$ is the privacy loss random variable
$E_{e^\varepsilon}(P \,\|\, Q)$	The hockey-stick divergence; (ε, δ)-DP means $E_{e^\varepsilon}(M(D) \,\|\, M(D')) \le \delta$ for all $D \simeq D'$
$\mathrm{Lap}(b)$, $\mathcal{N}(0, \sigma^2)$	Laplace noise with scale b; normal noise with variance σ^2

What you will be able to do

By the end of this part you should be able to:

- state pure and approximate DP for a stated neighbouring relation and unit of privacy, and translate a guarantee into a bound on membership advantage and on posterior odds;

- prove post-processing, group privacy and basic composition, and say what does not compose;
- calibrate the Laplace and Gaussian mechanisms, including the exact (analytic) Gaussian calibration;
- derive advanced composition and compute optimal composition exactly;
- prove the exponential mechanism, report-noisy-max and the sparse vector technique private, and recognise the common broken variants of the last;
- handle data-dependent sensitivity with smooth sensitivity, propose-test-release, subsample-and-aggregate and clipping;
- implement discrete Laplace and discrete Gaussian samplers exactly, and explain why naive floating-point noise is not private;
- design frequency estimation in the local model and explain what shuffling and secure aggregation buy;
- prove packing and fingerprinting lower bounds, and run the tracing attack they come from.

Each of these is a result derived in one of the chapters that follow.

18. The definition of differential privacy

18.1. Why this matters

Part 2 broke models. A membership attacker looked at a trained model and decided whether a given record was in its training set. A reconstruction attacker solved for the private bits behind a table of answers. Each attack was a game: the challenger ran an algorithm on one of two datasets that differ in one person, and the adversary guessed which (Threat models and privacy games). Each attack worked because the output looked different enough on the two datasets for a test to tell them apart (Membership inference).

This chapter turns that observation into a definition. If no test can tell the two datasets apart, every attack of Part 2 fails at once, whatever the adversary knows and however much it computes. **Differential privacy (DP)** asks exactly this: for every pair of datasets that differ in one person, the distribution of the output barely changes. It is a property of the algorithm, not of the data or of a particular attack, and you can check it with pencil and paper.

The chapter answers four questions:

- **What counts as "differ in one person"?** The neighbouring relation, which fixes the unit of privacy.
- **What does "barely changes" mean?** Pure ε-DP bounds the ratio of the output probabilities. Approximate (ε, δ)-DP allows the ratio bound to fail on a set of probability at most δ.
- **What does the guarantee buy?** A bound on every test's error rates, on the advantage of every membership attacker, and on how far any adversary's posterior can move.
- **What does it not buy?** It says nothing about what can be learned from the population, it treats correlated records as separate people, and it is worthless if δ is as large as $1/n$.

18.1.1. The one idea: the privacy loss is a likelihood ratio

Run a randomized algorithm M on two neighbouring datasets D and D', and look at one output y. You can ask how much more likely this output was if the data were D than if they were D'. That is a ratio of two probabilities (or densities), $p_{M(D)}(y)$ over $p_{M(D')}(y)$, and its logarithm is called the **privacy loss**:

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)} \tag{18.1}$$

If L is 0, the output is equally likely in both worlds and says nothing about which dataset was used. If L is large and positive, the output is far more likely under D, so seeing it is strong evidence for D; if it is large and negative, strong evidence for D'. This is the log-likelihood ratio

of Divergences: the evidence that y gives for "the data were D" over "the data were D'". The logarithm turns ratios into differences, so evidence from independent pieces of output adds up.

Part 1 showed that this one quantity drives everything about telling two distributions apart. The best test of "D or D'?" compares the likelihood ratio with a threshold (the Neyman–Pearson lemma of Hypothesis testing), and each divergence between two distributions is some summary of the likelihood ratio. So if you want no test to work well, the natural thing to bound is the privacy loss. **Differential privacy is a bound on the privacy loss.** Pure DP bounds it everywhere, $|L| \le \varepsilon$. The largest value of L is the **max-divergence**, so pure DP is a bound on the max-divergence. Approximate DP allows the bound to fail on outputs that carry a little probability mass δ; the divergence that measures exactly this excess mass is the **hockey-stick divergence**, defined in the Toolkit below. Everything else in the chapter is a consequence of this bound, read through the Part 1 tools (Table 18.1).

Table 18.1.: The results of this chapter and where each is used.

Result	Where it is used
Neighbouring relations (add/remove, replace; example, user, group)	Stating what a guarantee protects; amplification by subsampling in Subsampling in Part 4
Pure ε-DP and approximate (ε, δ)-DP	Every mechanism and every proof in this part
Randomized response	The first mechanism; the worst case for composition in Advanced and optimal composition; local DP in Local, shuffle and distributed trust models
Max-divergence and hockey-stick forms	Proving privacy of a mechanism; post-processing; numerical accounting
Hypothesis-testing form, $\mathrm{FPR} + e^{\varepsilon}\,\mathrm{FNR} \ge 1 - \delta$	f-DP in Gaussian differential privacy; auditing a model by attacking it
Membership-advantage bound	Turning ε into what an attacker of Part 2 can achieve
Bayesian (posterior-odds) form	Explaining ε to people who think in probabilities of a secret
Why $\delta \ll 1/n$	Choosing δ
What DP does not protect	Knowing when the guarantee does not answer your question

How to read this chapter. Read the definition section and the randomized-response section first; together they are the core. The divergence, testing and advantage sections are three readings of the same inequality, so read them in one sitting. The last two sections, on δ and on what DP does not protect, are short and matter in practice more than any proof.

Notation for the part. M is a randomized algorithm, a **mechanism**, that maps a dataset to a random output in a set $\mathcal{Y}$. $D \simeq D'$ means that D and D' are neighbours. $p_{M(D)}$ is the probability mass function or density of $M(D)$, and $\Pr[M(D) \in S]$ the probability of an event $S \subseteq \mathcal{Y}$. The

privacy parameters are $\varepsilon \geq 0$ and $\delta \in [0,1]$. $L_{D,D'}$ is the privacy loss of Equation 18.1; it is positive where the output favours D. For a function f from datasets to $\mathbb{R}^d$, the ℓ_1 and ℓ_2 **sensitivities** are $\Delta_1 f = \max_{D \simeq D'} \|f(D) - f(D')\|_1$ and $\Delta_2 f = \max_{D \simeq D'} \|f(D) - f(D')\|_2$: the most that changing one neighbour can move f, measured as a sum of absolute changes or as a Euclidean distance. The Laplace and Gaussian mechanisms uses them to calibrate noise. Logarithms are natural.

18.2. Toolkit: four facts from Part 1

The derivations below reuse four facts from Part 1. Each is restated here with its short proof, so you do not need to look them up. In the first two, P and Q are two distributions on the same finite set, with probabilities $p(y)$ and $q(y)$, and $r(y) = p(y)/q(y)$ is their likelihood ratio at the outcome y.

18.2.1. Fact 1 — The worst event for a ratio is a single outcome

Suppose you want the largest possible ratio $P(S)/Q(S)$ over all events S. An event's probability is the sum of the probabilities of its outcomes, and each outcome contributes $p(y) = r(y)\,q(y)$ to $P(S)$. So $P(S)$ is $Q(S)$ times a weighted average of the ratios $r(y)$ over the outcomes in S, with weights $q(y)/Q(S)$. An average is never larger than its largest term: $P(S) = \sum_{y \in S} r(y)q(y) \leq (\max_y r(y))\, Q(S)$. And the event $\{y^*\}$ that holds only the outcome y^* with the largest ratio reaches that value. Grouping outcomes into a bigger event can only dilute the worst ratio, never raise it:

$$\max_{S:\, Q(S)>0} \frac{P(S)}{Q(S)} = \max_y \frac{p(y)}{q(y)} = e^{D_\infty(P\|Q)}. \tag{18.2}$$

The **max-divergence** $D_\infty(P\|Q) = \ln \max_y p(y)/q(y)$ is just the logarithm of this largest ratio (Divergences). The practical consequence is that pure DP, a bound on ratios of event probabilities, can be checked outcome by outcome.

18.2.2. Fact 2 — The worst event for a difference is $\{r > \gamma\}$

Now take a number $\gamma \geq 1$ and look for the event that makes the difference $P(S) - \gamma Q(S)$ as large as possible. Each outcome y in S adds $p(y) - \gamma q(y)$ to that difference. This is positive exactly when $p(y) > \gamma q(y)$, that is, when the likelihood ratio $r(y)$ exceeds γ. So the best event takes every outcome with a positive contribution and leaves out every other one, and the best difference is the sum of the positive parts. Writing $(x)_+ = \max\{x, 0\}$,

$$\max_S \left[P(S) - \gamma Q(S)\right] = \sum_y \left(p(y) - \gamma q(y)\right)_+ = E_\gamma(P\|Q). \tag{18.3}$$

This sum is the **hockey-stick divergence** $E_\gamma(P\|Q)$, named after the shape of the function $(x)_+$: flat at zero, then rising. It is the probability mass of P that sticks out above γ times Q. A randomized choice $\phi(y) \in [0,1]$ in place of an event (include y with probability $\phi(y)$) does no better: the quantity $\sum_y \phi(y)(p(y) - \gamma q(y))$ is linear in each $\phi(y)$, so it is largest with $\phi(y) = 1$ where the coefficient is positive and $\phi(y) = 0$ where it is not, which is the event above. The practical consequence is that approximate DP, which bounds such differences, can be checked with one sum instead of a search through all $2^{|\mathcal{Y}|}$ events.

18.2.3. Fact 3 — A test's errors are probabilities of one event

An adversary who sees y and must decide between "the data were D" (the null hypothesis) and "the data were D'" uses some decision rule, and a deterministic rule is just an event S: say "D'" when $y \in S$, and "D" otherwise. Its two ways of being wrong are then probabilities of S. Its **false positive rate** is the probability of saying "D'" when the data were D, $\alpha = \Pr[M(D) \in S]$. Its **false negative rate** is the probability of saying "D" when the data were D', $\beta = \Pr[M(D') \notin S]$. A randomized test says "D'" with probability $\phi(y)$ after seeing y; then $\alpha = \mathbb{E}_{M(D)}[\phi]$ and $\beta = 1 - \mathbb{E}_{M(D')}[\phi]$, the same formulas with the indicator of S replaced by ϕ (Hypothesis testing, Fact 1). The practical consequence is that any inequality between $\Pr[M(D) \in S]$ and $\Pr[M(D') \in S]$ is an inequality between the error rates of a test.

18.2.4. Fact 4 — Posterior odds are prior odds times the likelihood ratio

Suppose someone believes, before seeing the output, that the data are D with probability $\Pr[D]$ and D' with probability $\Pr[D']$. Bayes' rule says that after seeing y, $\Pr[D \mid y] = \Pr[D]\, p_{M(D)}(y)/p(y)$, where $p(y)$ is the overall probability of y. The same holds for D' with the same $p(y)$ underneath. So when you take the ratio of the two posteriors, the normalizer $p(y)$ cancels, and what is left is the prior ratio times the likelihood ratio:

$$\frac{\Pr[D \mid y]}{\Pr[D' \mid y]} = \frac{\Pr[D]}{\Pr[D']} \cdot \frac{p_{M(D)}(y)}{p_{M(D')}(y)} = \frac{\Pr[D]}{\Pr[D']} \cdot e^{L_{D,D'}(y)}. \tag{18.4}$$

A ratio of the form $\Pr[D]/\Pr[D']$ is called the **odds** of D (Hypothesis testing, the Bayesian view). The practical consequence is that a bound on the privacy loss L is a bound on how far any belief can move.

18.3. Neighbouring datasets

18.3.1. Context

"Differ in one person" needs a precise meaning before anything can be proved. A dataset is a multiset of **records** drawn from a universe $\mathcal{X}$ of possible records. The **neighbouring relation** says which pairs of datasets the guarantee must protect. It fixes the **unit of privacy**: the thing whose presence, absence or value the output must not reveal. Choosing it is a modelling decision, and most misunderstandings of a DP claim come from not stating it.

18.3.2. Statement

There are two standard ways to say that D and D' differ in one record. They are parallel options, and each protects something different:

- **Add/remove** (unbounded DP): $D \simeq D'$ if one is obtained from the other by adding or removing one record. The size n of the dataset is itself private.
- **Replace** (bounded DP, also substitution): $D \simeq D'$ if $|D| = |D'|$ and they differ in the value of one record. The size n is public.

Separately, you choose what one record is. There are three common granularities:

- **Example-level:** a record is one training example, one row, one event.
- **User-level:** a record is all the data of one person. A user who wrote 200 messages is one record, not 200.
- **Group-level:** D and D' differ in up to k records. This is what you get for free from example-level DP, at a price (Post-processing, group privacy and composition).

The two relations are linked, because a replacement is a removal followed by an addition. To turn Alice's record into Dan's, first remove Alice's (one add/remove step), then add Dan's (a second step). So a replace-neighbour is two add/remove steps away. Pure ε-DP, defined in the next section, says that one step between neighbours can multiply the probability of any output event by at most e^{ε}. Two steps can multiply it by at most $e^{\varepsilon} \cdot e^{\varepsilon} = e^{2\varepsilon}$, which gives:

$$\varepsilon\text{-DP under add/remove} \implies 2\varepsilon\text{-DP under replace.} \tag{18.5}$$

The converse fails: a guarantee under replace only ever compares datasets of the same size, so it says nothing about pairs of different sizes.

18.3.3. Derivation

1. **Build the two-step path.** Let $D \simeq D'$ under replace: D has record x where D' has x'. Let $D_0 = D \setminus \{x\}$ be D with that record removed. Then D and D_0 differ by one removal, and D_0 and D' differ by one addition (of x'), so both pairs are add/remove neighbours.
2. **Chain the ratio bound along the path.** Pure DP (Equation 18.7 of the next section with $\delta = 0$) says that for neighbours, every event's probability under one is at most e^{ε} times its probability under the other. Apply it once for D, D_0 and once for D_0, D': for every event S,
$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D_0) \in S] \leq e^{\varepsilon} \cdot e^{\varepsilon} \Pr[M(D') \in S]. \tag{18.6}$$
3. **Check the other direction.** DP must hold with D and D' in either order. Walking the same path from D' back to D gives $\Pr[M(D') \in S] \leq e^{2\varepsilon} \Pr[M(D) \in S]$, so the pair satisfies 2ε-DP.
4. **Show the converse fails with a counterexample.** Take the mechanism "output the number of records n". Any two datasets of the same size give the same output, so it is 0-DP under replace. Under add/remove, $M(D)$ and $M(D')$ are point masses at n and $n+1$, and the event $\{n+1\}$ has probability 1 under one and 0 under the other. No finite e^{ε} times 0 reaches 1, so no ε satisfies the definition.

For approximate DP, where each step allows an additive slack δ as well, the same path gives $\Pr[M(D) \in S] \leq e^{\varepsilon}(e^{\varepsilon} \Pr[M(D') \in S] + \delta) + \delta$, that is, $(2\varepsilon, (1 + e^{\varepsilon})\delta)$ (Exercise 2 in the exercise book).

18.3.4. Interpretation

The unit of privacy is what the guarantee is about. Example-level DP on a dataset in which one person contributes 200 examples protects each message, not the person. Removing the person

changes 200 records, and the group-level price in Post-processing, group privacy and composition shows how much weaker the guarantee for a group of 200 is.

The two relations protect different things. Add/remove protects participation: the adversary cannot tell whether you are in the data at all. That is the question membership inference asks, so add/remove is the relation that matches it. Replace protects the value of your record but assumes the attacker already knows the dataset size, and with it often that you took part.

Neighbours range over every record in the universe $\mathcal{X}$, including absurd ones (a person aged 900). This is the worst case, and a mechanism is DP only if it hides even those records. That is why mechanisms clip or bound their inputs before adding noise.

Finally, Equation 18.5 can be tight, and when it is depends on the mechanism. It is tight when one replacement changes the output in two places, as the example below shows. When a replacement changes the output in only one place, replace-DP is cheaper than the factor 2 suggests.

18.3.5. Example: who are the neighbours?

Take $D = (\text{Alice: smoker}, \text{Bob: non-smoker}, \text{Carol: smoker})$, with records of the form (person, answer). Table 18.2 checks five candidate datasets against both relations.

Table 18.2.: Five candidate datasets, checked against the add/remove and the replace relation.

Candidate D'	Add/remove neighbour?	Replace neighbour?
(Alice: smoker, Bob: non-smoker)	yes: Carol removed	no: size differs
(Alice: smoker, Bob: non-smoker, Carol: smoker, Dan: non-smoker)	yes: Dan added	no: size differs
(Alice: non-smoker, Bob: non-smoker, Carol: smoker)	no: two steps	yes: Alice's answer changed
(Alice: smoker, Bob: non-smoker, Dan: smoker)	no: two steps	yes: Carol's record replaced by Dan's
(Alice: non-smoker, Bob: smoker, Carol: smoker)	no	no: two records changed

Now release the histogram (number of smokers, number of non-smokers) with independent Laplace noise of scale $b = 1/\varepsilon$ in each bin. To see what each kind of neighbour costs, you need the privacy loss of this release, which comes from two facts about the Laplace distribution (Divergences, the Laplace pair). First, the Laplace density centred at c is proportional to $e^{-|y-c|/b}$, so the log ratio of the densities centred at c and c' is $(|y-c'| - |y-c|)/b$. By the triangle inequality, $|y-c'| - |y-c| \le |c-c'|$, so the log ratio is at most s/b when the centres are s apart, with equality for y beyond both centres. Second, the two bins get independent noise, so the joint density is a product of the two bins' densities and the log ratios of the bins add.

Removing Carol changes one bin (the smokers) by 1 and leaves the other alone, so the largest privacy loss is $1/b = \varepsilon$. Changing Alice's answer moves one count from one bin to the other: both bins change by 1, each contributes up to $1/b$, and the largest loss is 2ε. The script finds both

maxima on a grid: 1.0000 and 2.0000 at $\varepsilon = 1$. Here Equation 18.5 is tight, because a replacement changes the output in two places. If you release only the number of smokers, a replacement changes one count by at most 1, and the cost under replace is ε, not 2ε.

18.3.6. Check yourself

- A hospital releases statistics with example-level DP, where an example is one visit. A patient has 30 visits. What does the guarantee say about that patient? *(Only that each visit is protected. The patient is a group of 30 records, and the guarantee for the group is much weaker; see group privacy.)*
- Why does a replace-DP guarantee not protect against learning whether you took part? *(Under replace the dataset size is public and pairs of different sizes are never compared. Whether you are in the data is a question about pairs of different sizes.)*

18.4. Pure and approximate differential privacy

18.4.1. Context

With the neighbours fixed, "the output distribution barely changes" needs a measure. Part 1 offers many, and the choice matters. An average-case measure such as the Kullback–Leibler divergence, which averages the privacy loss over outputs, can be small while one rare output exposes a person completely. Differential privacy takes the worst case over everything that can vary: over all neighbouring pairs, over all events, with no assumption on the adversary's knowledge or computing power.

18.4.2. Statement

The definition asks that no event become much more likely when one person's record changes. "Much more likely" is measured by a multiplicative factor e^{ε}, so that small ε means a ratio close to 1. On top of that, a small additive slack δ is allowed, so that events of tiny probability are not held to the ratio. Formally, a mechanism M is (ε, δ)**-differentially private** if for every pair of neighbours $D \simeq D'$ and every event $S \subseteq \mathcal{Y}$,

$$\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{18.7}$$

Because the neighbouring relation is symmetric (if $D \simeq D'$ then $D' \simeq D$), the inequality also holds with D and D' swapped.

The case $\delta = 0$ is **pure DP**, written ε**-DP**. With no slack, the ratio of any event's probabilities must stay within e^{ε}, and by Fact 1 this is the same as asking it outcome by outcome: for discrete outputs, pure DP is equivalent to the pointwise bound $p_{M(D)}(y) \leq e^{\varepsilon}\, p_{M(D')}(y)$ for every y, which together with the swapped bound is $|L_{D,D'}(y)| \leq \varepsilon$.

The case $\delta > 0$ is **approximate DP** (Dwork, Kenthapadi, McSherry, Mironov & Naor, 2006). It has **no** pointwise form. The tempting bound "$p_{M(D)}(y) \leq e^{\varepsilon}\, p_{M(D')}(y) + \delta$ for every y" is much weaker, because it pays the slack δ once per outcome instead of once per event, and it is not a privacy definition.

18.4.3. Derivation

Pure DP is pointwise. The goal is to show that the event form and the outcome form say the same thing when $\delta = 0$.

1. *Events imply outcomes.* A single outcome is an event, so take $S = \{y\}$ in Equation 18.7 to get the pointwise bound.
2. *Outcomes imply events.* An event's probability is a sum over its outcomes, so summing the pointwise bound over $y \in S$ gives $\Pr[M(D) \in S] = \sum_{y \in S} p_{M(D)}(y) \leq e^{\varepsilon} \sum_{y \in S} p_{M(D')}(y)$. This is the argument of Fact 1.
3. *For densities* the same holds with "for every y" replaced by "for almost every y", since single points have probability zero and changing the density at a few points changes no event's probability.

Approximate DP is not pointwise. The difference is that adding δ to each outcome is not the same as adding it once to each event.

1. *Events imply outcomes.* Again take $S = \{y\}$. So (ε, δ)-DP implies the pointwise bound.
2. *Outcomes do not imply events.* Summing the pointwise bound over S gives $\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + |S|\,\delta$, because each of the $|S|$ outcomes brings its own δ. The slack grows with the number of outcomes in S.
3. *A counterexample shows how bad this gets.* Let $M(D)$ be uniform on $\{1, \ldots, m\}$ and $M(D')$ uniform on $\{m+1, \ldots, 2m\}$. Every outcome has $p_{M(D)}(y) \leq 1/m$, so the pointwise bound holds with $\delta = 1/m$ for every ε, even when $p_{M(D')}(y) = 0$. But the event $S = \{1, \ldots, m\}$ has probability 1 under $M(D)$ and 0 under $M(D')$, so Equation 18.7 needs $\delta = 1$. With $m = 1000$, the script gives pointwise $\delta = 0.0010$ and event $\delta = 1.0000$. The outputs on D and D' never overlap, so one look at the output tells which dataset was used, and yet the pointwise slack looks tiny.

Why events and not outputs. The adversary does not need to see one particular output. It needs any decision rule, and by Fact 3 a decision rule is an event: "say D' when $y \in S$". Bounding every event therefore bounds every attack.

18.4.4. Interpretation

The factor e^{ε} is a multiplicative bound. For small ε, $e^{\varepsilon} \approx 1 + \varepsilon$ (the first two terms of the exponential series), so every event changes probability by at most a fraction about ε of itself. At $\varepsilon = 0.1$, an event of probability 0.5 under $M(D')$ has probability at most $0.5e^{0.1} = 0.553$ under $M(D)$. At $\varepsilon = 1$ the bound is $0.5e = 1.36$, which is above 1 and so says nothing about that event. Values used in practice range from below 1 to around 10, and large values promise little (Section 18.8).

The slack δ is additive. It allows events of probability up to δ under $M(D)$ to be arbitrarily more likely than under $M(D')$, even impossible under $M(D')$. Section 18.10 shows why it must be tiny.

The definition is worst case in every direction: over all neighbouring pairs (not typical ones), over all events (not typical outputs), and with no assumption on the adversary. That is what makes DP hold against attacks that have not been invented yet, and also what makes it expensive. The

randomness in it is the mechanism's own. The probability is over the coins of M, with the dataset held fixed, and nothing is assumed about how the data were generated.

Figure 18.1 draws the definition for a count released with noise. Pure DP asks the density on D to stay inside the band between $e^{-\varepsilon}$ and e^{ε} times the density on D'. Laplace noise stays inside everywhere, because its log density ratio is bounded, as in the histogram example of Section 18.3. Gaussian noise leaves the band in the tail, because the log ratio of two Gaussians with the same spread is a straight line in y and grows without bound. The mass outside the band is what δ must pay for.

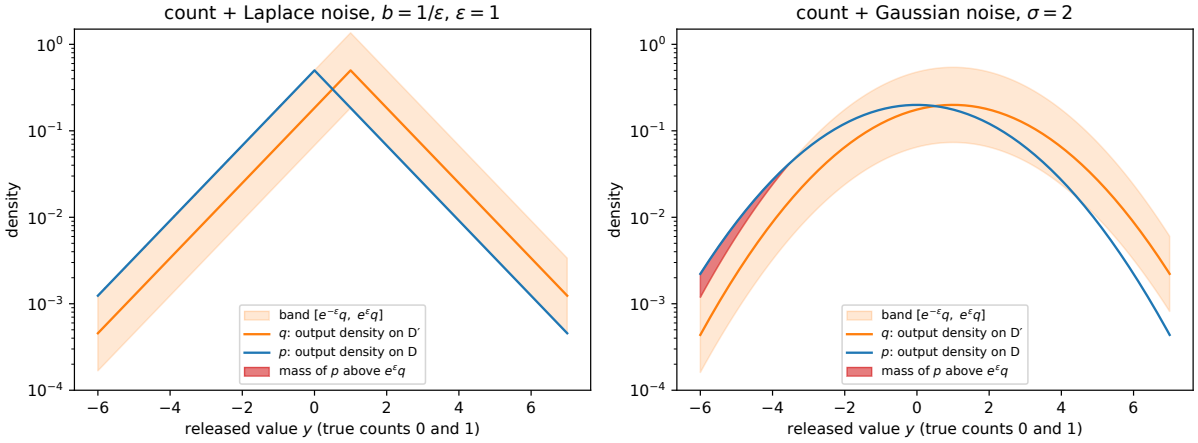


Figure 18.1.: The definition as a picture. The orange curve is the output density on D' (true count 1) and the shaded band runs from $e^{-\varepsilon}$ to e^{ε} times it, with $\varepsilon = 1$ and log vertical axes. Left: with Laplace noise of scale $1/\varepsilon$, the density on D (true count 0, blue) stays inside the band everywhere, so the release is ε-DP. Right: with Gaussian noise ($\sigma = 2$), the blue density leaves the band in the left tail; the red mass is the part that δ has to cover.

18.4.5. Example: the smallest ε for a three-person survey

Take three people with bits $(1, 0, 1)$ and a neighbour $(0, 0, 1)$ in which the first bit changed. Each person runs randomized response, the mechanism of the next section, with $\varepsilon = \ln 3$: they report their true bit with probability $\frac{3}{4}$ and the flipped bit with probability $\frac{1}{4}$, independently of each other. The output is one of $2^3 = 8$ report vectors, so there are $2^8 = 256$ events, and the script checks all of them.

The largest ratio $\Pr[M(D) \in S]/\Pr[M(D') \in S]$ is 3.0000, the same as the largest ratio for a single outcome, as Fact 1 says. You can see where the 3 comes from: only the first person's bit differs, so in the probability of any report vector the second and third factors are the same on D and D' and cancel, and the first factor is $\frac{3}{4}$ against $\frac{1}{4}$ when the first report is 1.

Likewise, the largest $\Pr[M(D) \in S] - 2\Pr[M(D') \in S]$ over the 256 events is 0.2500, equal to the hockey-stick sum of Fact 2 with $\gamma = 2$. By Fact 2, the best event collects the outcomes whose ratio exceeds 2, which are the four report vectors whose first report is 1 (ratio 3; the others have ratio $\frac{1}{3}$). Their total probability is $\frac{3}{4}$ on D and $\frac{1}{4}$ on D', so the sum is $\frac{3}{4} - 2 \cdot \frac{1}{4} = 0.25$. The brute force over events and the one-line formulas agree.

18.4.6. Check yourself

- A mechanism is 0-DP. What does that say about its output? *(Its distribution is the same on any two neighbours, so, by chaining, on any two datasets connected by a path of neighbours. Its output does not depend on the data at all.)*
- Why is it no use to check approximate DP only on single outcomes? *(The pointwise slack δ adds up over the outcomes of an event. Two distributions with disjoint supports pass the pointwise check with $\delta = 1/m$.)*

18.5. Randomized response

18.5.1. Context

You want the fraction of people who have done something embarrassing: cheated on an exam, used a drug, evaded a tax. Asked directly, people lie. Warner (1965) proposed a survey protocol in which the interviewer cannot know any individual's true answer: each respondent tosses a coin in private. On heads, they answer truthfully. On tails, they toss again and answer "yes" on heads and "no" on tails. Every "yes" now has a plausible excuse, yet the population fraction can still be estimated, because the coins' behaviour is known and averages out over many people. Randomized response is the first differentially private mechanism, half a century before the definition, and it is the reference point for the whole part.

18.5.2. Statement

The idea is to tell the truth with a probability t that is a little above $\frac{1}{2}$. Then each report leans toward the truth, which is what lets you estimate the population, but the ratio of the chances of a given report under the two possible true bits is only $t/(1-t)$. Setting that ratio to e^{ε} and solving for t gives $t = e^{\varepsilon}/(1+e^{\varepsilon})$. So **randomized response at level** ε reports a person's bit $x \in \{0,1\}$ truthfully with probability $t = e^{\varepsilon}/(1+e^{\varepsilon})$ and flipped with probability $1-t$. Applied independently to each of n people, it is ε-DP under replace.

Warner's two coins are a special case. A respondent tells the truth if the first coin lands heads (probability $\frac{1}{2}$), or if it lands tails and the second coin happens to give the true answer (probability $\frac{1}{2} \cdot \frac{1}{2}$). So $t = \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{4}$, the ratio is $t/(1-t) = 3$, and Warner's coins are randomized response at $\varepsilon = \ln 3 = 1.0986$.

To estimate the true fraction π of ones, note that the fraction of reported ones is pushed toward $\frac{1}{2}$ by the coins in a known way, so you can undo the push. Write $y_1, \dots, y_n$ for the reports and $\bar{y}$ for their mean. The expected value of $\bar{y}$ is $(1-t) + \pi(2t-1)$ (derived below), and solving that for π gives the estimate

$$\hat{\pi} = \frac{\bar{y} - (1-t)}{2t-1} \tag{18.8}$$

which is unbiased. Undoing the push also scales up the noise in $\bar{y}$ by $1/(2t-1)$, which is large when t is close to $\frac{1}{2}$, so the estimate's standard deviation is

$$\mathrm{sd}(\hat{\pi}) = \frac{\sqrt{t(1-t)}}{(2t-1)\sqrt{n}} = \frac{e^{\varepsilon/2}}{(e^{\varepsilon}-1)\sqrt{n}} \approx \frac{1}{\varepsilon\sqrt{n}} \quad \text{for small } \varepsilon. \tag{18.9}$$

The approximation uses $e^{\varepsilon/2} \approx 1$ and $e^{\varepsilon} - 1 \approx \varepsilon$ for small ε.

18.5.3. Derivation

Privacy for one person. There are two inputs and two outputs, so the whole mechanism is the table of output probabilities in Table 18.3. The privacy loss of each report is the log of the ratio between the two rows. Every ratio is $t/(1-t) = e^{\varepsilon}$ or its inverse, so $|L| \le \varepsilon$ for every output, and by Fact 1 this is ε-DP.

Table 18.3.: Randomized response as a table of output probabilities. The ratios between the two rows are the privacy loss.

True bit	Pr[report 1]	Pr[report 0]	At $\varepsilon = \ln 3$
1	$t = \frac{e^{\varepsilon}}{1+e^{\varepsilon}}$	$1-t = \frac{1}{1+e^{\varepsilon}}$	0.75, 0.25
0	$1-t$	t	0.25, 0.75
Ratio, row 1 / row 0	e^{ε}	$e^{-\varepsilon}$	3, 1/3

Privacy for n people. The point is that the other people's reports do not change between neighbours, so they cancel out of the likelihood ratio.

1. *The output distribution factors.* People toss their own coins independently, so the probability of a report vector is a product of one factor per person: $p_{M(D)}(y_1, \dots, y_n) = \prod_i p_i(y_i \mid x_i)$, where $p_i(y_i \mid x_i)$ is the probability that person i reports y_i when their true bit is x_i.
2. *Neighbours differ in one factor.* Under replace, D and D' differ only in person j's bit. All other factors are equal on the two datasets and cancel in the ratio.
3. *The loss is one person's loss.* What is left is person j's factor, $L_{D,D'}(y) = \ln \frac{p_j(y_j|x_j)}{p_j(y_j|x'_j)} \in \{-\varepsilon, 0, \varepsilon\}$, which is 0 if the two bits happen to be equal.

The estimate. A report is 1 with probability t if the true bit is 1 and $1-t$ if it is 0. Averaging over the population, in which a fraction π have bit 1, gives $\mathbb{E}\bar{y} = \pi t + (1-\pi)(1-t) = (1-t) + \pi(2t-1)$. Solving for π gives Equation 18.8, and because the estimate is a linear function of $\bar{y}$, linearity of expectation makes it unbiased. For the spread, each report is a Bernoulli variable that is 1 with probability t or $1-t$, and either way its variance is $t(1-t)$. The reports are independent, so their variances add, and $\operatorname{Var}\bar{y} = t(1-t)/n$. Dividing $\bar{y}$ by $2t-1$ divides the variance by $(2t-1)^2$, and the square root is the first form of $\operatorname{sd}(\hat{\pi})$. Substituting $t = e^{\varepsilon}/(1+e^{\varepsilon})$ gives $t(1-t) = e^{\varepsilon}/(1+e^{\varepsilon})^2$ and $2t-1 = (e^{\varepsilon}-1)/(e^{\varepsilon}+1)$, hence the closed form.

18.5.4. Interpretation

Deniability is the privacy. At $\varepsilon = \ln 3$, a "yes" is three times more likely from a true "yes" than from a true "no", never more. Anyone can say "the coin made me say it".

The noise is added per person, before the data leave them. Nobody, not even the analyst, sees a true bit. This is the **local** model, in which no one is trusted with the raw data; Local, shuffle and distributed trust models develops it.

Privacy costs samples. The error is about $1/(\varepsilon\sqrt{n})$, so halving ε doubles the error, and since the error falls like $1/\sqrt{n}$, recovering the accuracy takes four times as many people. Without privacy, counting the true bits of a fixed group has error 0, so all of this error is the price of privacy.

The guarantee holds under replace only. Under add/remove, the number of reports reveals n, and with it whether you took part. Randomized response protects the value of your answer, not the fact that you answered.

18.5.5. Example: a survey of 1000 people

Of $n = 1000$ respondents, $\pi = 0.3$ have the sensitive trait, and all use Warner's coins ($t = 0.75$). The expected fraction of "yes" reports is $0.3 \times 0.75 + 0.7 \times 0.25 = 0.225 + 0.175 = 0.4$, and Equation 18.8 turns it back into $(0.4 - 0.25)/0.5 = 0.3$. The formula gives $\mathrm{sd}(\hat{\pi}) = \sqrt{0.1875}/(0.5\sqrt{1000}) = 0.0274$, where $0.1875 = 0.75 \times 0.25$ and $0.5 = 2 \times 0.75 - 1$. Ten thousand simulated surveys give a mean estimate of 0.2996 and a standard deviation of 0.0278, close to both.

For comparison, the sampling error of a survey of 1000 people drawn from a large population is $\sqrt{0.3 \times 0.7/1000} = 0.0145$: randomized response at $\varepsilon = \ln 3$ adds noise about twice as large as the sampling noise already there. At $\varepsilon = 0.5$ the formula gives 0.0626 (Figure 18.2), and at $\varepsilon = 0.1$ it gives $9.9958/\sqrt{n}$, or 0.316 for 1000 people: an error larger than the fraction being estimated, so the survey is useless.

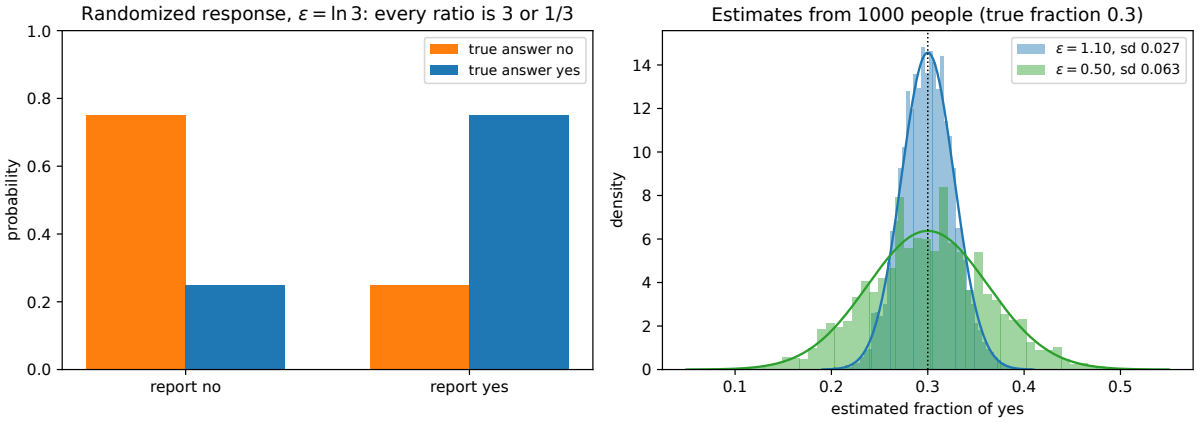


Figure 18.2.: Left: randomized response at $\varepsilon = \ln 3$ as output probabilities. Each report is 3 times more likely under one true answer than under the other. Right: estimates of the fraction of "yes" from 4000 simulated surveys of 1000 people with true fraction 0.3, at $\varepsilon = \ln 3$ and $\varepsilon = 0.5$, with the normal curves predicted by the standard-deviation formula.

18.5.6. Check yourself

- What is t at $\varepsilon = 0$, and what does the estimate do then? *($t = \frac{1}{2}$: every report is a fair coin and carries no information. The denominator $2t - 1$ is zero, so no estimate is possible.)*

- Two people answer with randomized response at $\varepsilon = \ln 3$. Is the pair of reports $\ln 3$-DP or $2\ln 3$-DP? *($\ln 3$-DP under replace, since neighbours differ in one person and the other factor cancels. A group of both people is protected only at $2\ln 3$.)*

18.6. Divergence forms and the privacy loss

18.6.1. Context

Equation 18.7 quantifies over all events. That is fine for a mechanism with two outcomes and unusable for continuous outputs, where there are infinitely many events. Facts 1 and 2 showed that the worst event is always a set of outcomes where the likelihood ratio is high. Rewriting the definition as a divergence turns "for all events" into one formula, and it shows what δ measures: the probability mass on which the privacy loss exceeds ε, discounted by how far it exceeds it.

18.6.2. Statement

For every pair of neighbours $D \simeq D'$, write $P = M(D)$ and $Q = M(D')$ for the two output distributions, with probabilities $p(y)$ and $q(y)$. There are three results, each a direct reading of a Toolkit fact.

1. **Pure DP is a max-divergence bound.** Since the worst event ratio is the worst outcome ratio (Fact 1), M is ε-DP if and only if $D_\infty(P\|Q) \le \varepsilon$ for every ordered pair of neighbours.
2. **Approximate DP is a hockey-stick bound.** The definition asks $P(S) - e^\varepsilon Q(S) \le \delta$ for every event, and Fact 2 says the largest left side is the hockey-stick divergence E_{e^ε}. Factoring $p(y)$ out of each term turns this sum into an average over outputs drawn from P: each output counts with the weight $(1 - e^{\varepsilon - L})_+$, which is 0 when its privacy loss L is at most ε and approaches 1 as L grows. So M is (ε, δ)-DP if and only if, for every ordered pair of neighbours,

$$E_{e^\varepsilon}(P\|Q) = \sum_y \left(p(y) - e^\varepsilon q(y)\right)_+ = \mathbb{E}_{y\sim P}\left[\left(1 - e^{\varepsilon - L_{D,D'}(y)}\right)_+\right] \le \delta. \tag{18.10}$$

   So the smallest δ for a given ε is a function $\delta(\varepsilon)$, the largest hockey-stick divergence over neighbouring pairs.
3. **A sufficient condition on the privacy loss.** Since each weight in that average is at most 1, and is 0 unless $L > \varepsilon$, the average is at most the probability that $L > \varepsilon$. So if $\Pr_{y\sim P}[L_{D,D'}(y) > \varepsilon] \le \delta$ for every ordered pair, then M is (ε, δ)-DP.

18.6.3. Derivation

1. **Pure.** By Fact 1, $\max_S P(S)/Q(S) = e^{D_\infty(P\|Q)}$. Requiring this largest ratio to be at most e^ε is requiring $D_\infty \le \varepsilon$.
2. **Approximate, first form.** Equation 18.7 says $P(S) - e^\varepsilon Q(S) \le \delta$ for every S. That holds for every S exactly when it holds for the worst S, and by Fact 2 with $\gamma = e^\varepsilon$, the maximum of the left side over S is $E_{e^\varepsilon}(P\|Q)$.

3. **Approximate, second form.** Factor $p(y)$ out of each term and use $q/p = e^{-L}$ from Equation 18.1: $(p - e^{\varepsilon}q)_+ = p\,(1 - e^{\varepsilon}q/p)_+ = p\,(1 - e^{\varepsilon - L})_+$. A sum of $p(y)$ times a function of y is the average of that function under P. Outcomes with $p(y) = 0$ contribute nothing to either side, so dividing by p is harmless.
4. **The sufficient condition.** For every y, $(1 - e^{\varepsilon - L})_+ \le \mathbf{1}[L > \varepsilon]$, where $\mathbf{1}[\cdot]$ is 1 when the condition holds and 0 otherwise. The left side is 0 when $L \le \varepsilon$ (then $e^{\varepsilon - L} \ge 1$), and less than 1 otherwise. Averaging both sides under P gives $E_{e^{\varepsilon}}(P\|Q) \le \Pr_P[L > \varepsilon] \le \delta$.

18.6.4. Interpretation

δ is not "the probability that privacy fails". By Equation 18.10, it is the average of $(1 - e^{\varepsilon - L})_+$, which counts an output with loss just above ε for almost nothing and an output with infinite loss in full. The tail probability $\Pr[L > \varepsilon]$ counts both in full, so it is always at least $\delta(\varepsilon)$, and often several times larger.

A mechanism has a curve, not a pair. Every ε has its own smallest δ, and a mechanism is $(\varepsilon, \delta(\varepsilon))$-DP for all ε at once (Figure 18.3). Quoting a single pair throws the rest of the curve away. Numerical accounting in Part 4 computes these curves for compositions of mechanisms.

Whether the curve reaches zero depends on whether the privacy loss is bounded. If the loss never exceeds some ε_0 (randomized response, Laplace noise), then at $\varepsilon = \varepsilon_0$ every weight $(1 - e^{\varepsilon - L})_+$ is 0 and the curve reaches $\delta = 0$. If the loss is unbounded (Gaussian noise), some outputs always have $L > \varepsilon$, so $\delta(\varepsilon) > 0$ for every ε: no pure guarantee exists, only approximate ones.

It helps to think of the privacy loss as a random variable. Draw y from $M(D)$ and compute $L_{D,D'}(y)$. The distribution of this number, the **privacy loss distribution**, determines $\delta(\varepsilon)$ through Equation 18.10. Accountants in Numerical accounting work with it directly.

18.6.5. Example: three mechanisms on the same axes

All three mechanisms release one bit or one count that changes by 1 between neighbours. For each, the recipe is the same: find where the likelihood ratio exceeds e^{ε}, and add up the excess mass $p - e^{\varepsilon}q$ there.

Randomized response at $\varepsilon_0 = \ln 3$. From Table 18.3, the two outputs have ratios 3 and $\frac{1}{3}$, so for $\varepsilon < \ln 3$ the only outcome with $p > e^{\varepsilon}q$ is the one with ratio 3, which has $p = t$ and $q = 1 - t$. So $\delta(\varepsilon) = \big(t - e^{\varepsilon}(1-t)\big)_+ = (3 - e^{\varepsilon})_+/4$, using $t = \frac{3}{4}$. The script gives $\delta(0.5) = 0.3378$, $\delta(1) = 0.0704$ and $\delta(\ln 3) = 0$.

A count plus Laplace noise of scale $b = 1$. The output densities are $\mathrm{Lap}(c, 1)$ and $\mathrm{Lap}(c + 1, 1)$, Laplace densities $\frac{1}{2}e^{-|y - \text{centre}|}$ one unit apart. Their log ratio falls from 1 to -1 as y crosses from one centre to the other, so the ratio exceeds γ on a half-line: all y below $c + y^*$, with $y^* = (1 - \ln\gamma)/2$. Integrating the two Laplace tails over that half-line gives the formula from Part 1, $E_\gamma = 1 - e^{-(1 - \ln\gamma)/2}$ for $\ln\gamma \le 1$, and $E_\gamma = 0$ once $\ln\gamma$ reaches the largest loss, 1. So $\delta(0.5) = 1 - e^{-0.25} = 0.2212$ and $\delta(1) = 0$: this count is 1-DP.

A count plus Gaussian noise with $\sigma = 2$. Here the log ratio is a straight line in y, which makes the privacy loss itself Gaussian. Measure the shift between the two outputs in units of the noise: $\mu = 1/\sigma = 0.5$. Under $M(D)$, the privacy loss is normal with mean $\mu^2/2$ and variance μ^2; under

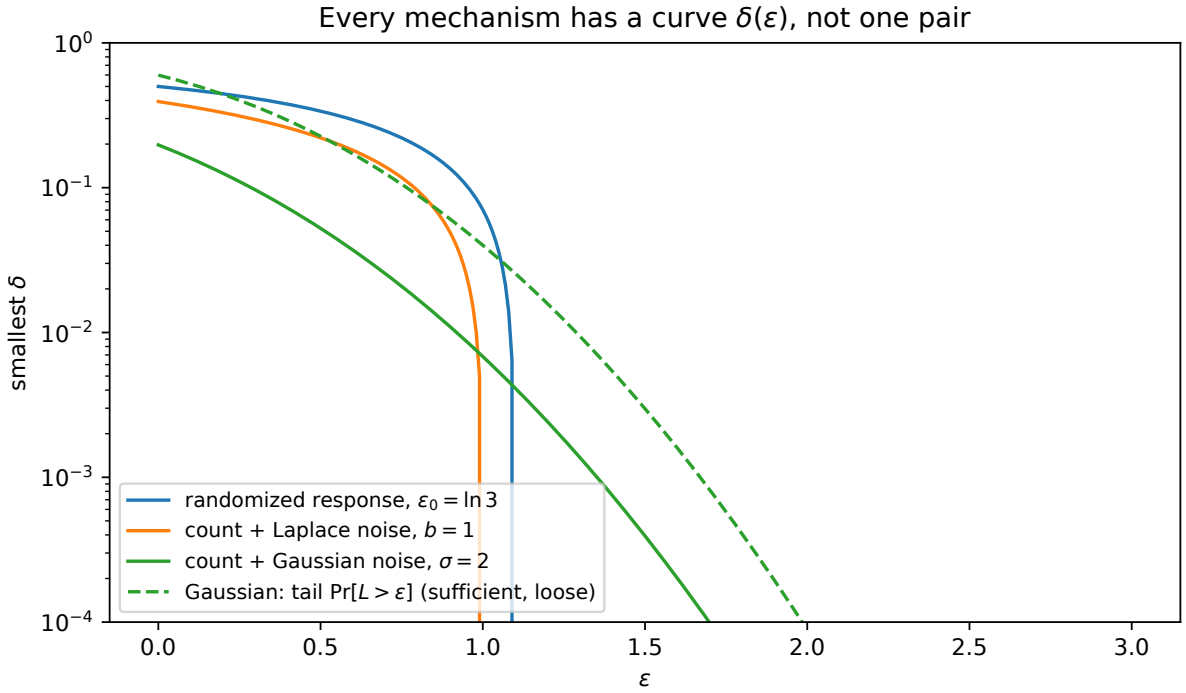


Figure 18.3.: The smallest δ as a function of ε for three mechanisms with log vertical axis: randomized response at $\varepsilon_0 = \ln 3$, a count with Laplace noise of scale 1, and a count with Gaussian noise of standard deviation 2. The first two reach $\delta = 0$ at their largest privacy loss; the Gaussian curve never does. The dashed line is the privacy-loss tail $\Pr[L > \varepsilon]$ for the Gaussian count, a valid but loose δ.

$M(D')$ it has mean $-\mu^2/2$ and the same variance. Write Φ for the standard normal cumulative distribution function, so that $\Phi(z)$ is the probability that a standard normal variable is at most z. Standardizing, $\Pr[L > \varepsilon] = \Pr\left[Z > (\varepsilon - \mu^2/2)/\mu\right] = \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu})$. The hockey-stick divergence is the P-probability of $\{L > \varepsilon\}$ minus e^{ε} times its Q-probability (Fact 2), which is the Gaussian hockey-stick formula of Part 1, $\delta(\varepsilon) = \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) - e^{\varepsilon}\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$. Table 18.4 compares the exact δ with the tail.

Table 18.4.: The exact $\delta(\varepsilon)$ of a count with Gaussian noise of standard deviation 2, against the sufficient condition $\Pr[L > \varepsilon]$.

ε	0.5	1	2	3
Exact $\delta(\varepsilon)$, Equation 18.10	5.244×10^{-2}	6.830×10^{-3}	9.439×10^{-6}	3.401×10^{-10}
Tail $\Pr[L > \varepsilon]$	2.266×10^{-1}	4.006×10^{-2}	8.842×10^{-5}	4.462×10^{-9}
Tail / exact	4.32	5.87	9.37	13.12

As a check on Equation 18.10, a Monte Carlo average of $(1 - e^{1-L})_+$ over a million draws of L gives 6.859×10^{-3}, against the exact 6.830×10^{-3}. The tail bound is valid but loose by a factor that grows with ε, from 4 to 13 here. The reason is the discount in Equation 18.10: most outputs in the tail have a loss only a little above ε, so they count almost nothing toward δ but fully toward the tail. Calibrating noise with the tail instead of the exact curve therefore wastes privacy budget, which is the subject of the analytic Gaussian calibration in The Laplace and Gaussian mechanisms.

18.6.6. Check yourself

- Why is the Laplace count 1-DP while the Gaussian count with a similar spread is not ε-DP for any ε? *(The Laplace log-likelihood ratio is bounded by $1/b = 1$, so $D_\infty = 1$. The Gaussian one is a straight line, unbounded in the tails, so $D_\infty = \infty$.)*
- In Equation 18.10, which outputs contribute most to δ? *(Those with a large privacy loss, where $(1 - e^{\varepsilon - L})_+$ is close to 1. Outputs with L just above ε contribute almost nothing.)*

18.7. The hypothesis-testing interpretation

18.7.1. Context

A membership attacker is a test. It sees the output and decides between "the data were D" and "the data were D'". Part 1 described any test by its two error rates, and the best achievable pairs by the **trade-off function**: for each false positive rate α, the smallest false negative rate any test can reach. Differential privacy turns out to be exactly a lower bound on the trade-off function, uniform over all neighbours (Wasserman & Zhou, 2010; Kairouz, Oh & Viswanath, 2015). This is the most operational reading of the definition: it says what an attacker's receiver operating characteristic (ROC) curve, its true positive rate plotted against its false positive rate, can look like.

18.7.2. Statement

Let a test say "D'" (reject the null hypothesis H_0 that the data are D) on the event S, possibly randomized. Write α for its **false positive rate (FPR)**, $\Pr[M(D) \in S]$, and β for its **false negative rate (FNR)**, $\Pr[M(D') \notin S]$. By Fact 3, both are probabilities of the event S or its complement, so the definition, which bounds the probability of every event, bounds them. Applied to the complement of S, it says a test cannot have a small FPR without paying in FNR; applied with the roles of D and D' swapped, it says the same the other way round. Precisely, M is (ε, δ)-DP if and only if every test, for every pair of neighbours, satisfies

$$\alpha + e^{\varepsilon}\beta \geq 1 - \delta \qquad \text{and} \qquad e^{\varepsilon}\alpha + \beta \geq 1 - \delta. \tag{18.11}$$

Each inequality is a line in the plane of (α, β), and no test can lie below either line. So the smallest FNR at a given FPR, which is the trade-off function, is at least the higher of the two lines (or 0, since an error rate cannot be negative). Equivalently, the trade-off function of every pair of neighbours lies above

$$f_{\varepsilon,\delta}(\alpha) = \max\left\{0,\ 1 - \delta - e^{\varepsilon}\alpha,\ e^{-\varepsilon}(1 - \delta - \alpha)\right\}. \tag{18.12}$$

In terms of the true positive rate (TPR) $1 - \beta$, the fraction of the time the test correctly says "D'", the second inequality reads: at every false positive rate α, $\text{TPR} \leq e^{\varepsilon}\alpha + \delta$.

18.7.3. Derivation

1. **The first inequality.** Apply Equation 18.7 to the pair (D, D') and the complement event S^c, the outputs on which the test says "D": $\Pr[M(D) \notin S] \leq e^{\varepsilon} \Pr[M(D') \notin S] + \delta$. The left side is $1 - \alpha$ and the probability on the right is β, so $1 - \alpha \leq e^{\varepsilon}\beta + \delta$, which rearranges to the first inequality.
2. **The second inequality.** Apply Equation 18.7 to the swapped pair (D', D) and the event S: $\Pr[M(D') \in S] \leq e^{\varepsilon} \Pr[M(D) \in S] + \delta$. The left side is $1 - \beta$ and the probability on the right is α, so $1 - \beta \leq e^{\varepsilon}\alpha + \delta$.
3. **Randomized tests.** A test $\phi(y) \in [0, 1]$ is not an event, so the definition does not apply to it directly. But Fact 2 showed that a randomized choice ϕ can make $\mathbb{E}_{M(D)}[\phi] - e^{\varepsilon}\mathbb{E}_{M(D')}[\phi]$ no larger than the best event can, which is $E_{e^{\varepsilon}} \leq \delta$. So the definition holds for ϕ as well, and steps 1 and 2 go through with ϕ and $1 - \phi$ in place of the events.
4. **The converse.** Every event S is a test. If Equation 18.11 holds for all tests, steps 1 and 2 read backwards give Equation 18.7 for every event and both orders of the pair.
5. **The trade-off form.** Rearranged for β, the two inequalities say $\beta \geq 1 - \delta - e^{\varepsilon}\alpha$ and $\beta \geq e^{-\varepsilon}(1 - \delta - \alpha)$, and $\beta \geq 0$ always. So the smallest β any test can reach at level α is at least the largest of the three, which is Equation 18.12.

18.7.4. Interpretation

The result draws a forbidden region. In the plane of (FPR, FNR), every attack against an (ε, δ)-DP mechanism lies on or above the two lines of Equation 18.11 (Figure 18.4, left). The two lines are mirror images of each other across the diagonal $\alpha = \beta$, so they cross on it: setting $\alpha = \beta$ in $\alpha + e^{\varepsilon}\beta = 1 - \delta$ gives $\alpha = \beta = (1 - \delta)/(1 + e^{\varepsilon})$.

Low FPR is where DP bites. Part 2 evaluated membership attacks by their TPR at a low FPR, because an attack that flags someone while almost never flagging non-members is one that confidently identifies people. DP caps that TPR at $e^{\varepsilon}\alpha + \delta$. At $\alpha = 10^{-3}$, $\varepsilon = 1$ and $\delta = 10^{-5}$, no attack has TPR above $e \times 10^{-3} + 10^{-5} = 0.002728$. The ratio TPR/FPR is at most $e^{\varepsilon} + \delta/\alpha$, and as $\alpha \to 0$ the δ/α term dominates (Figure 18.4, right): at very low FPR, the guarantee rests almost entirely on δ.

Auditing runs this in reverse. Measure an attack's FPR and TPR on a trained model, and Equation 18.11 gives a lower bound on the ε the training algorithm can have: solving $\text{TPR} \le e^{\varepsilon}\,\text{FPR} + \delta$ for ε gives the smallest value consistent with what you measured. The neighbouring pair is a training set with and without a planted **canary**, a record inserted on purpose to be detected. The canary is chosen to be as easy to detect as possible (a mislabeled or out-of-distribution example), because the definition bounds the worst record and an ordinary one gives a loose lower bound (memorization and extraction introduced canaries). Later parts use this to check the privacy claims of trained models.

The bound is tight for randomized response. Its test "say D' on report 0" (for neighbours $x = 1$ and $x' = 0$) has $\alpha = \beta = 1/(1 + e^{\varepsilon})$, which is exactly the crossing point, the corner of the region. Most mechanisms sit strictly inside. The trade-off curve of the Gaussian count is smooth and curved, so it touches each of the two straight lines of $f_{\varepsilon,\delta(\varepsilon)}$ at one point and lies above them elsewhere. Keeping the whole curve, instead of one $f_{\varepsilon,\delta}$, is the idea of f-DP in Gaussian differential privacy.

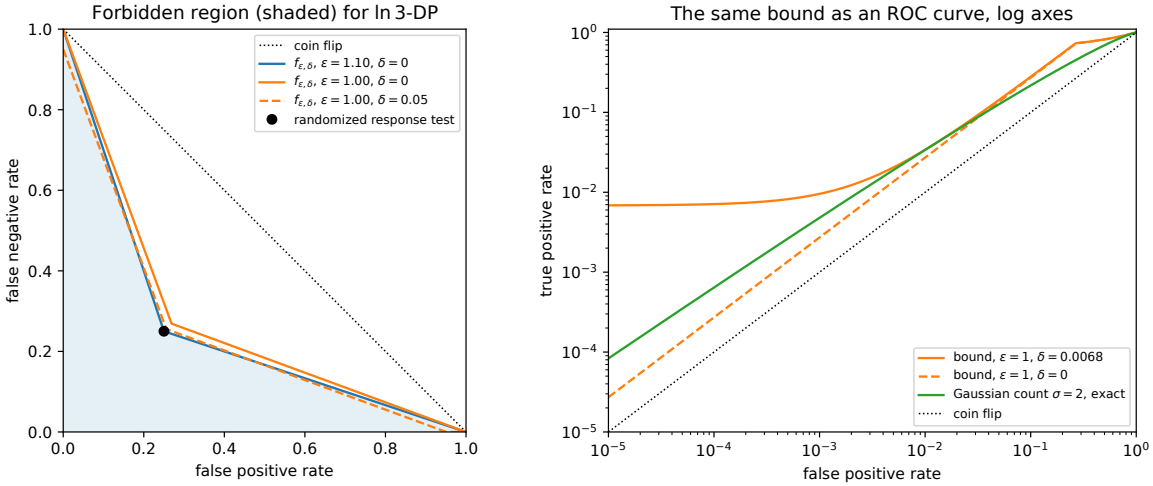


Figure 18.4.: Left: in the plane of false positive rate and false negative rate, the lower boundary $f_{\varepsilon,\delta}$ for $\varepsilon = \ln 3$ and $\delta = 0$ (shaded below: no test can be there), and for $\varepsilon = 1$ with $\delta = 0$ and $\delta = 0.05$. The randomized response test at $\varepsilon = \ln 3$ sits exactly at the corner. Right: the same bound as a curve of the largest TPR against FPR on log axes, for $\varepsilon = 1$ with and without δ, and the exact best TPR against a count with Gaussian noise of standard deviation 2, which is $(1, 0.0068)$-DP.

18.7.5. Example: randomized response and the Gaussian count, as tests

Randomized response at $\varepsilon = \ln 3$. The neighbours are a person's bit being 1 (D) or 0 (D'). The best test says "D'" (bit 0) when the report is 0. Its FPR is $\Pr[\text{report } 0 \mid 1] = 0.25$ and its FNR is $\Pr[\text{report } 1 \mid 0] = 0.25$. Then $\alpha + 3\beta = 0.25 + 0.75 = 1$: the first inequality of Equation 18.11 holds with equality. At FPR 0.25 the bound allows TPR at most $3 \times 0.25 = 0.75$, and the test achieves exactly $1 - 0.25 = 0.75$.

The Gaussian count with $\sigma = 2$ is $(1, 0.00683)$-DP by Table 18.4. At FPR 10^{-3} the bound allows TPR up to $e \times 10^{-3} + 0.00683 = 0.0095$. To compare, you need the best TPR any test can actually reach. By the Neyman–Pearson lemma, the best test thresholds the likelihood ratio, and for two Gaussians with the same spread that ratio grows steadily with y, so the best test thresholds y itself. In units of the noise, y is standard normal under D, so an FPR of 10^{-3} needs the threshold $\Phi^{-1}(0.999) \approx 3.09$, the point with 0.1% of the standard normal mass above it (Φ^{-1} is the inverse of Φ). Under D' the output is shifted by $\mu = 0.5$, so the TPR is the probability that a normal with mean 0.5 exceeds that threshold. This is the Gaussian trade-off curve of Part 1: the exact best TPR is $1 - \Phi(\Phi^{-1}(0.999) - 0.5) = 0.0048$. The bound holds, with a factor 2 of room, because at this FPR the δ term is generous: 0.00683 alone is larger than the true TPR.

18.7.6. Check yourself

- Under ε-DP with $\delta = 0$, can any test have FPR 0 and TPR above 0? *(No.* $\text{TPR} \le e^{\varepsilon} \times 0 + 0 = 0$. *Certain identification with no false alarm is ruled out entirely; only* $\delta > 0$ *allows it.)*
- An attack reports TPR 0.05 at FPR 0.001 against a model trained with a claimed $\varepsilon = 2$, $\delta = 10^{-5}$. What do you conclude? *(The claim is false, or the attack's rates are mis-measured: the bound allows TPR at most* $e^2 \times 0.001 + 10^{-5} \approx 0.0074$*.)*

18.8. Bound on the membership advantage

18.8.1. Context

Part 2 measured a membership attacker by its **advantage**. In the membership game, the challenger flips a fair coin to pick D or D', runs M on it, and the adversary guesses which dataset was used. A guesser who ignores the output is right half the time, so the useful quantity is how much better than half the adversary does, doubled so that a perfect adversary scores 1: the advantage is $2\Pr[\text{correct}] - 1$. With a fair coin, $\Pr[\text{correct}] = \frac{1}{2}\text{TPR} + \frac{1}{2}(1 - \text{FPR})$, so the advantage equals $\text{TPR} - \text{FPR} = 1 - \alpha - \beta$. Its best value over all adversaries is the total variation (TV) distance between $M(D)$ and $M(D')$, the largest difference $\Pr[M(D') \in S] - \Pr[M(D) \in S]$ over events (Threat models and privacy games). The testing form of the previous section bounds the sum of errors $\alpha + \beta$ from below, and so gives the largest advantage any DP mechanism can allow.

18.8.2. Statement

The adversary wants both error rates small at once, so it wants the smallest possible sum $\alpha + \beta$. The two lines of Equation 18.11 keep it away from the origin, and the closest it can get is the point where the lines cross, $\alpha = \beta = (1 - \delta)/(1 + e^{\varepsilon})$. Plugging that point into $1 - \alpha - \beta$ gives the bound: if M is (ε, δ)-DP, every adversary in the game between any two neighbours has

$$\text{Adv} = \text{TPR} - \text{FPR} \leq \frac{e^{\varepsilon} - 1 + 2\delta}{e^{\varepsilon} + 1}, \qquad \Pr[\text{correct}] \leq \frac{e^{\varepsilon} + \delta}{e^{\varepsilon} + 1}. \tag{18.13}$$

For $\delta = 0$ the bound is $(e^{\varepsilon} - 1)/(e^{\varepsilon} + 1)$. Multiplying top and bottom by $e^{-\varepsilon/2}$ gives $(e^{\varepsilon/2} - e^{-\varepsilon/2})/(e^{\varepsilon/2} + e^{-\varepsilon/2})$, which is the hyperbolic tangent $\tanh(\varepsilon/2)$, about $\varepsilon/2$ for small ε. Randomized response attains it.

18.8.3. Derivation

1. **Turn the advantage into a sum of errors.** The advantage is $1 - (\alpha + \beta)$, so the largest advantage comes from the smallest $\alpha + \beta$ allowed by Equation 18.11, together with $\alpha, \beta \geq 0$.
2. **Draw the feasible region.** The allowed (α, β) pairs are the points on or above both lines of Equation 18.11 with both coordinates non-negative. This region's lower boundary is a polygon with corners $(0, 1 - \delta)$ (where the first line meets the β axis), $(1 - \delta, 0)$ (where the second meets the α axis) and the crossing point $\alpha = \beta = (1 - \delta)/(1 + e^{\varepsilon})$.
3. **Check the corners.** A linear function such as $\alpha + \beta$ is smallest over such a region at one of its corners, because moving along a straight edge changes it at a constant rate, so it keeps decreasing until a corner. At the first two corners $\alpha + \beta = 1 - \delta$. At the crossing point it is $2(1 - \delta)/(1 + e^{\varepsilon})$, which is smaller since $e^{\varepsilon} \geq 1$ makes $2/(1 + e^{\varepsilon}) \leq 1$.
4. **Compute the advantage.** $1 - 2(1 - \delta)/(1 + e^{\varepsilon}) = (e^{\varepsilon} - 1 + 2\delta)/(e^{\varepsilon} + 1)$. With a fair coin the success probability is $\frac{1}{2} + \frac{1}{2}\text{Adv} = (e^{\varepsilon} + \delta)/(e^{\varepsilon} + 1)$.
5. **Show tightness.** Randomized response at level ε has $\alpha = \beta = 1/(1 + e^{\varepsilon})$, the crossing point with $\delta = 0$, so its advantage is $(e^{\varepsilon} - 1)/(e^{\varepsilon} + 1)$, exactly the bound.
6. **Extend to random datasets.** Part 2's game draws the dataset from a distribution before the coin flip. The bound holds for each fixed pair of neighbours, so it holds for the average over pairs, since an average of numbers below a bound is below the bound.

18.8.4. Interpretation

Small ε is strong. At $\varepsilon = 0.1$ the bound is $\tanh(0.05) \approx 0.05$, so the best attacker guesses right with probability at most $(1 + 0.05)/2 = 0.525$.

Large ε promises little. At $\varepsilon = 5$ the bound is 0.9866: the definition allows an attacker who is right $(1 + 0.9866)/2 = 99.3\%$ of the time. A model trained with $\varepsilon = 8$ may be far safer than that in practice, but the guarantee itself does not say so.

There is an older, looser bound. Yeom et al. (2018) showed $\text{Adv} \leq e^{\varepsilon} - 1$ for ε-DP. Since $\tanh(\varepsilon/2) \approx \varepsilon/2$ and $e^{\varepsilon} - 1 \approx \varepsilon$, the two have the same order at small ε, but the older bound is about twice as large there, and it exceeds 1, so says nothing, once $e^{\varepsilon} \geq 2$, that is, from $\varepsilon = \ln 2$ on.

Most mechanisms do not reach the bound. The bound is for the worst mechanism with this ε. A count with Laplace noise of scale $1/\varepsilon$ has best advantage $\mathrm{TV} = 1 - e^{-\varepsilon/2}$: the TV distance is the hockey-stick divergence at $\gamma = 1$, and the Laplace calculation of Section 18.6, redone for centres s apart and scale b, gives $1 - e^{-s/(2b)}$, which is $1 - e^{-\varepsilon/2}$ for $s = 1$ and $b = 1/\varepsilon$ (Part 1). It is below $\tanh(\varepsilon/2)$ at every ε (Figure 18.5).

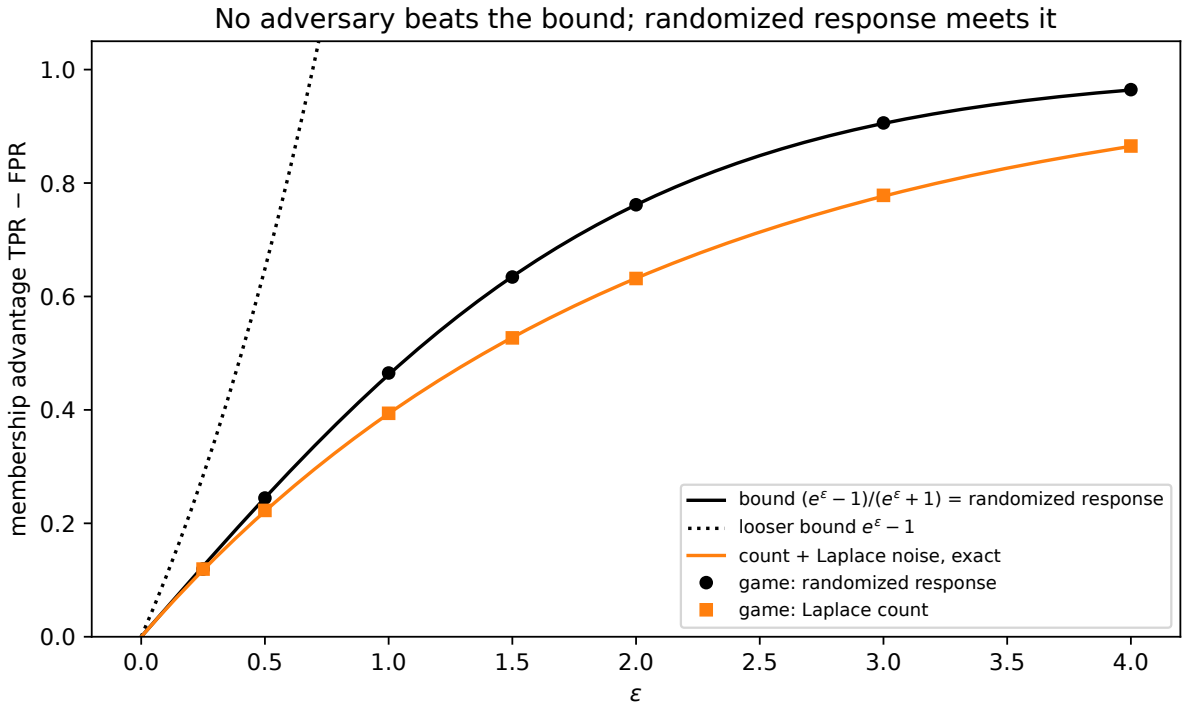


Figure 18.5.: The largest membership advantage against an ε-DP mechanism (black), the looser bound $e^{\varepsilon} - 1$ (dotted), and the exact best advantage against a count with Laplace noise of scale $1/\varepsilon$ (orange). Markers are advantages measured in 100,000 simulated membership games with the likelihood-ratio adversary; randomized response sits on the bound.

18.8.5. Example: the bound against two mechanisms

Table 18.5 compares the bound with the best advantage against randomized response and against the Laplace count.

Table 18.5.: The membership advantage bound for ε-DP and the exact best advantage against two ε-DP mechanisms.

ε	0.1	0.5	1	$\ln 3$	2	5
Bound $\tanh(\varepsilon/2)$	0.0500	0.2449	0.4621	0.5000	0.7616	0.9866

ε	0.1	0.5	1	$\ln 3$	2	5
Looser bound $e^{\varepsilon} - 1$	0.1052	0.6487	1.7183	2.0000	6.3891	147.4
Randomized response	0.0500	0.2449	0.4621	0.5000	0.7616	0.9866
Laplace count	0.0488	0.2212	0.3935	0.4226	0.6321	0.9179

The script also plays the game 200,000 times with the likelihood-ratio adversary, which guesses D when the privacy loss of the output is positive and D' otherwise. At $\varepsilon = 1$ it measures 0.4624 against randomized response (exact 0.4621) and 0.3953 against the Laplace count (exact 0.3935); at $\varepsilon = 0.5$ and 2 the measurements are within 0.002 of the exact values too.

What does δ do to this bound? With $\delta = 10^{-5}$ at $\varepsilon = 1$, it adds $2\delta/(e + 1)$ to the advantage, and the bound moves only to 0.462123, the success probability to 0.731061: a small δ barely changes the average-case advantage. The damage δ can do is at low FPR, as Section 18.7 showed, and Section 18.10 shows how bad it can get.

18.8.6. Check yourself

- Why is the advantage bound reached at $\alpha = \beta$? *(The two constraints are mirror images, so the region is symmetric about $\alpha = \beta$, and the corner that minimizes $\alpha + \beta$ is on that line.)*
- The advantage bound at $\varepsilon = 2$ is 0.76. Does this mean 76% of members are identified? *(No. Advantage is TPR − FPR averaged over the coin flip, a single-pair worst case. It says nothing direct about the TPR at a low FPR, which Equation 18.11 bounds separately and much more tightly.)*

18.9. The Bayesian interpretation

18.9.1. Context

People often ask a different question: "after seeing the output, how sure can someone be about me?" That is a question about posterior probabilities, the adversary's beliefs after seeing the output. Fact 4 says the posterior odds are the prior odds times e^L, so a bound on the privacy loss L turns directly into a bound on how far any posterior can move, whatever the prior. This is the reading that explains ε to someone who thinks in probabilities of a secret (Kasiviswanathan & Smith, 2014).

18.9.2. Statement

Consider an adversary who knows that the data are D or D', two neighbours, and believes beforehand that they are D with prior probability $\pi = \Pr[D]$. Its prior odds are $\pi/(1-\pi)$. Each output multiplies these odds by e^L, and pure DP keeps L between $-\varepsilon$ and ε, so the odds can be multiplied by at most e^ε and divided by at most e^ε. If M is ε-DP, then after any output y,

$$e^{-\varepsilon}\frac{\pi}{1-\pi} \le \frac{\Pr[D \mid y]}{\Pr[D' \mid y]} \le e^{\varepsilon}\frac{\pi}{1-\pi}, \qquad \Pr[D \mid y] \le \frac{e^{\varepsilon}\pi}{e^{\varepsilon}\pi + 1 - \pi}. \tag{18.14}$$

Under approximate DP the privacy loss is not bounded everywhere, so this cannot hold for every output. What does hold is that outputs with a loss well above ε are rare. Pick a larger level $\varepsilon' > \varepsilon$. Each output whose loss exceeds ε' is at least $e^{\varepsilon'}$ times as likely under D as under D', so it contributes at least a fraction $1 - e^{\varepsilon-\varepsilon'}$ of its probability to the excess mass that δ bounds. So such outputs can have total probability at most δ divided by that fraction. If M is only (ε, δ)-DP, the upper bounds hold with ε' in place of ε for every output except those in a set B with

$$\Pr_{y \sim M(D)}[y \in B] \le \frac{\delta}{1 - e^{\varepsilon-\varepsilon'}}, \qquad \varepsilon' > \varepsilon. \tag{18.15}$$

18.9.3. Derivation

Pure DP.

1. By Fact 4, the posterior odds are the prior odds times $e^{L_{D,D'}(y)}$.
2. Pure DP gives $|L_{D,D'}(y)| \le \varepsilon$ for every y (the pointwise form of Section 18.4), so e^L lies between $e^{-\varepsilon}$ and e^{ε}. Multiplying the prior odds by these two extremes gives the odds bound.
3. To get a bound on the probability itself, convert odds o to a probability, $o/(1+o)$. This is increasing in o, so the largest odds give the largest probability. With $o = e^{\varepsilon}\pi/(1-\pi)$, multiplying top and bottom by $1-\pi$ gives the bound on $\Pr[D \mid y]$.

Approximate DP. Let $B = \{y : L_{D,D'}(y) > \varepsilon'\}$, the outputs whose privacy loss exceeds ε'. The goal is to show B has small probability under $P = M(D)$.

1. Apply Equation 18.7 to the event B: $P(B) - e^{\varepsilon}Q(B) \le \delta$, with $Q = M(D')$.
2. On B, the loss exceeds ε', which means $p > e^{\varepsilon'}q$, or $q < e^{-\varepsilon'}p$. Summing over B gives $Q(B) < e^{-\varepsilon'}P(B)$.
3. Substitute this into step 1. Replacing $Q(B)$ by the larger $e^{-\varepsilon'}P(B)$ makes the left side smaller, so $P(B)(1-e^{\varepsilon-\varepsilon'}) < \delta$. Since $\varepsilon' > \varepsilon$, the factor in brackets is positive, and dividing by it gives Equation 18.15.
4. Outside B, the loss is at most ε', and steps 1–3 of the pure case, with ε' in place of ε, give the upper bounds.

18.9.4. Interpretation

DP bounds the *change* in belief, not the belief. It is relative, not absolute. An adversary who is already 99% sure stays about that sure; DP only promises that the output does not make it much surer.

The adversary in this statement is as strong as can be. It knows every record except the one in question, since it knows the data are D or D'. Against weaker adversaries, who are unsure about other records too, the same bound holds in a slightly different form: for the posterior about one person's record, compared with the posterior computed from the data without that record (Kasiviswanathan & Smith, 2014).

With δ, the guarantee holds for most outputs only. Taking $\varepsilon' = 2\varepsilon$ in Equation 18.15, the pure-DP reading holds at level 2ε except on outputs of probability at most $\delta/(1 - e^{-\varepsilon})$. A tiny δ keeps that exception tiny; a large one does not.

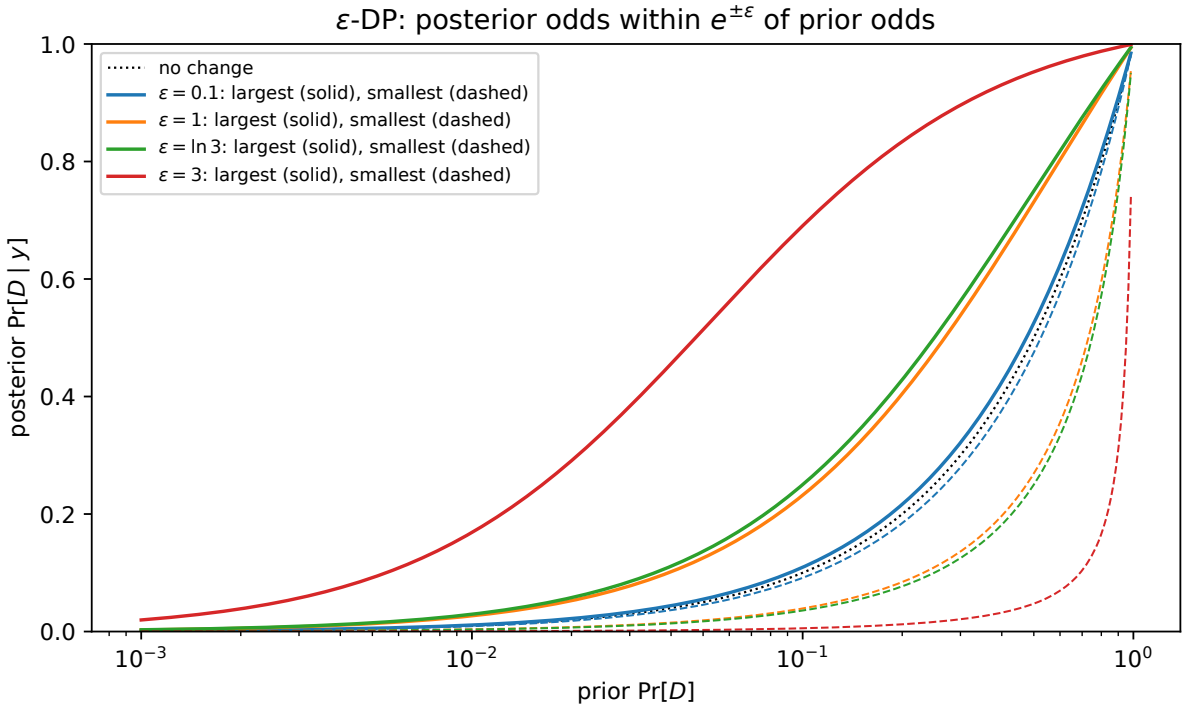


Figure 18.6.: The largest (solid) and smallest (dashed) posterior probability of D after one output of an ε-DP mechanism, against the prior, for four values of ε. The dotted diagonal is no change. Small priors stay small: even at $\varepsilon = 3$ a prior of 0.01 can rise only to 0.169.

18.9.5. Example: how sure can the interviewer be?

Back to Warner's survey at $\varepsilon = \ln 3$. The interviewer knows everyone's answer except Alice's and thinks there is a 50% chance that her true answer is "yes". She reports "yes". The prior odds are $0.5/0.5 = 1$, and a "yes" report is 3 times as likely from a true "yes", so the posterior odds are

1×3, and the interviewer is now at most $3/(3+1) = 75\%$ sure. If the prior was 1%, the posterior is at most $3 \times 0.01/(3 \times 0.01 + 0.99) = 0.0294$. Table 18.6 gives the bound for other values.

Table 18.6.: The largest posterior probability after one output of an ε-DP mechanism, from Equation 18.14.

ε	Prior 0.01	Prior 0.1	Prior 0.5
0.1	0.0110	0.1094	0.5250
1	0.0267	0.2320	0.7311
$\ln 3$	0.0294	0.2500	0.7500
3	0.1687	0.6906	0.9526

For approximate DP, take the Gaussian count with $\sigma = 2$, which is $(1, 0.0068)$-DP. Equation 18.15 with $\varepsilon' = 2$ bounds the probability of an output with privacy loss above 2 by $0.0068/(1 - e^{-1}) = 0.0108$. The exact probability is $\Pr[L > 2] = 8.842 \times 10^{-5}$, from Table 18.4. The bound holds by a wide margin, because it uses only (ε, δ), not the shape of the Gaussian tail.

18.9.6. Check yourself

- An adversary is certain ($\pi = 1$) that Alice is in the data. Can an ε-DP output change that? *(No. Prior odds are infinite, and multiplying by a factor in $[e^{-\varepsilon}, e^{\varepsilon}]$ keeps them infinite. DP limits what the output adds, not what the adversary already knows.)*
- Why does Equation 18.14 not need the adversary's prior to be "correct"? *(Fact 4 holds for any prior. The bound is on the likelihood ratio, which does not depend on the prior.)*

18.10. Why δ must be much smaller than $1/n$

18.10.1. Context

Equation 18.7 lets δ be anything in $[0, 1]$, and the advantage bound barely notices a small δ. So how small must it be? A deliberately terrible mechanism answers the question. It satisfies approximate DP with $\varepsilon = 0$, the strongest possible ε, and yet it publishes people's records verbatim.

18.10.2. Statement

The mechanism **"release each record with probability δ"** goes through the dataset and publishes each record, in full, independently with probability δ. It is DP because any one person is published only with probability δ, and otherwise the output does not depend on them at all: under add/remove it is $(0, \delta)$-DP. But over a whole dataset of n records it publishes $n\delta$ records on average. Each record escapes with probability $1 - \delta$, independently, so all of them escape with probability $(1 - \delta)^n$, and at least one is published with probability

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}. \tag{18.16}$$

So $\delta = 1/n$ exposes someone with probability about $1-e^{-1} = 0.632$. A meaningful (ε, δ) guarantee needs $\delta \ll 1/n$.

18.10.3. Derivation

1. **Couple the two runs.** To compare the outputs on D and on $D' = D \cup \{x\}$, run both with the same coins. Run the mechanism on D' by first running it on D, then tossing one more coin for x. The output on D' equals the output on D unless that last coin publishes x, which happens with probability δ.
2. **Bound one direction.** For any event S, the output on D' can land in S either because the output on D did, or because the extra coin changed it. So $\Pr[M(D') \in S] \leq \Pr[M(D) \in S] + \Pr[x \text{ published}] = \Pr[M(D) \in S] + \delta$.
3. **Bound the other direction.** With probability $1 - \delta$ the extra coin does nothing and the outputs are equal, so $\Pr[M(D') \in S] \geq (1-\delta)\Pr[M(D) \in S] \geq \Pr[M(D) \in S] - \delta$, where the last step uses $\Pr[M(D) \in S] \leq 1$.
4. **Read off the parameters.** Both directions match Equation 18.7 with factor $1 = e^0$ and slack δ, so the mechanism is $(0, \delta)$-DP.
5. **Count the exposure.** Each of the n records is published independently with probability δ, so the number published is Binomial(n, δ), with mean $n\delta$ and $\Pr[\text{none}] = (1-\delta)^n$. For small δ, $\ln(1-\delta) \approx -\delta$ (the first term of its Taylor series), so $(1-\delta)^n = e^{n\ln(1-\delta)} \approx e^{-n\delta}$.

18.10.4. Interpretation

δ is a per-person budget for catastrophe. The definition allows each person to be exposed with probability up to about δ. Over n people, that is $n\delta$ catastrophes on average. Only $n\delta \ll 1$ makes the expected number of fully exposed people small.

Approximate DP allows certain identification. In the testing picture of Section 18.7, the test "is x in the output?" never fires when x is absent, so its FPR is 0, and it fires with probability δ when x is present, so its TPR is δ. All of the δ is spent at FPR 0, where pure DP allows nothing.

In practice, pick δ well below $1/n$; $\delta = 1/n^{1.5}$ or a fixed small value such as 10^{-5} for 60,000 training examples (Abadi et al., 2016) are common. This is less costly than it sounds. Reasonable mechanisms, like the Gaussian count, spread their δ over many outputs with a moderately large loss, not over a few with infinite loss, so in practice a small δ costs little ε: in Table 18.4, going from $\varepsilon = 1$ to $\varepsilon = 3$ cuts δ from 6.830×10^{-3} to 3.401×10^{-10}.

If one person has many records, the relevant n is the number of people, not records, and the group-privacy cost of Post-processing, group privacy and composition also multiplies δ.

18.10.5. Example: ten thousand records

On a tiny dataset the script checks the claim directly. With $n = 3$ and $\delta = 0.1$, it enumerates the 2^4 possible outputs on D and on $D \cup \{x\}$ (each of the at most four records is published or not) and finds $\max_S |\Pr[M(D) \in S] - \Pr[M(D') \in S]| = 0.1000$, the hockey-stick divergence at $\varepsilon = 0$. So the mechanism is $(0, 0.1)$-DP and no better.

Now take $n = 10{,}000$ records (Table 18.7).

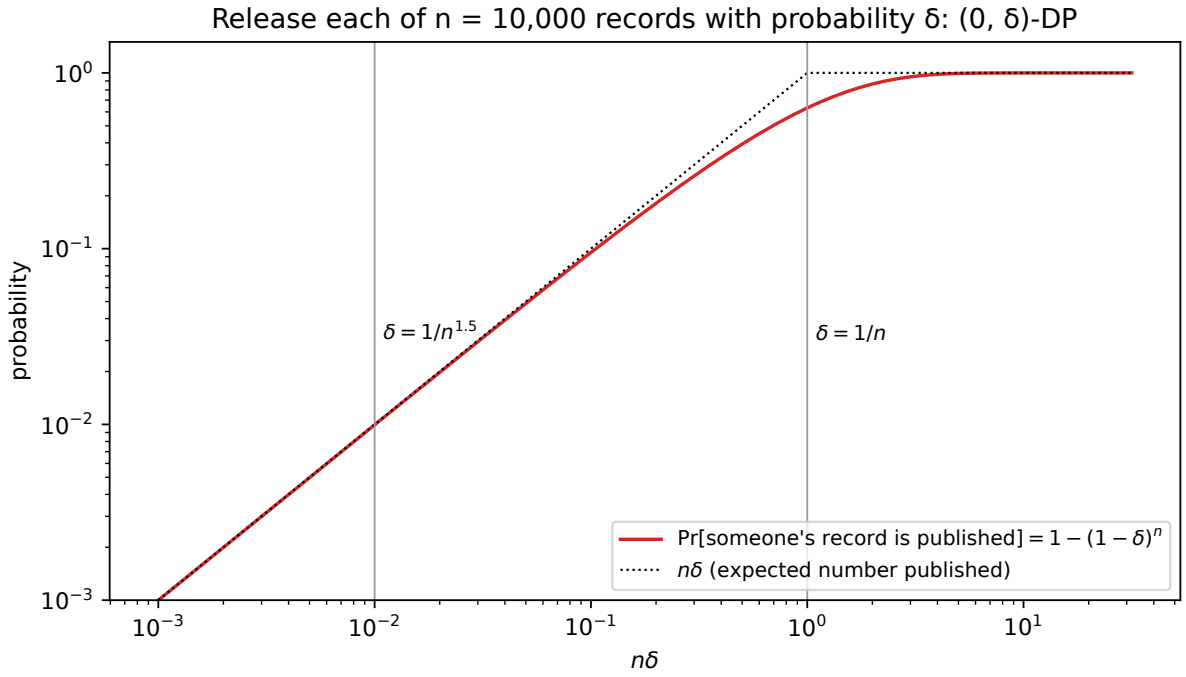


Figure 18.7.: The probability that the mechanism "release each record with probability δ" publishes at least one of $n = 10{,}000$ records (red), against $n\delta$, on log axes. Below $n\delta \approx 0.1$ the probability is essentially $n\delta$; at $\delta = 1/n$ it is already 0.63.

Table 18.7.: Exposure under the $(0, \delta)$-DP mechanism that releases each of 10,000 records with probability δ.

δ	Expected records published, $n\delta$	Pr[at least one]
$10^{-4} = 1/n$	1	0.6321
10^{-6}	0.01	9.950×10^{-3}
10^{-8}	0.0001	9.999×10^{-5}

When $n\delta$ is small, $1 - e^{-n\delta} \approx n\delta$, which is why the last two rows have Pr[at least one] almost equal to $n\delta$. In 100,000 simulated runs at $\delta = 1/n$, the mechanism published 1.0077 records on average and at least one record in a fraction 0.6351 of the runs, against 1 and 0.6321 from the formulas. A guarantee of $(0, 1/n)$-DP is compatible with publishing a random person's full record in most releases.

18.10.6. Check yourself

- Why doesn't the advantage bound Equation 18.13 warn about this mechanism? *(At $\varepsilon = 0$ it gives advantage at most δ, which is small. The harm is concentrated: a few people are exposed completely while most are untouched. An average over the coin flip hides that.)*

- Is $\delta = 10^{-5}$ appropriate for a dataset of 10 million users? *(No. $n\delta = 100$, so the definition allows about 100 people to be fully exposed. It needs δ well below 10^{-7}.)*

18.11. What differential privacy does not protect

18.11.1. Context

The definition compares two worlds that differ in one record. It does not compare the world before and after the release, and it does not say anything about records that are not independent of each other. Those are exactly the cases where people's intuitions about "privacy" go beyond what DP promises. Kifer & Machanavajjhala (2011) made this precise: no definition can protect against every inference without assumptions on how the data were generated.

18.11.2. Statement

DP bounds $\Pr[M(D) \in S]$ against $\Pr[M(D') \in S]$ for neighbours. It does not bound three kinds of inference:

1. **What the output reveals about the population.** If the data show that smokers get cancer at a high rate, everyone now believes a smoker is at higher risk, including smokers who were not in the data.
2. **What it reveals about correlated records.** If k records move together (a household, a user's duplicates, a contact network), changing all of them is a group change of size k, and for ε-DP with record-level neighbours the privacy loss of the group can be as large as $k\varepsilon$.
3. **Anything outside the mathematical mechanism.** Floating-point implementations that leak through low-order bits (Discrete and floating-point-safe mechanisms), timing side channels, the raw data stored on a server, or releases of the same data through other channels.

18.11.3. Derivation

Population inference is not a privacy loss. The point is that DP compares Alice in against Alice out, not before against after. Suppose the cancer rate θ among smokers is either 0.1 or 0.3, each with prior probability $\frac{1}{2}$. Your belief that Alice, a smoker, will get cancer is the average, $\frac{1}{2}(0.1) + \frac{1}{2}(0.3) = 0.2$. A study that excludes Alice reveals $\theta = 0.3$. Your belief about Alice rises to 0.3. The mechanism did not take Alice's record as input, so it is 0-DP with respect to her, yet your belief about her changed by half. In both worlds that DP compares, with and without Alice, the study shows $\theta = 0.3$, so DP correctly says nothing happened to Alice *because she took part*. Preventing this kind of inference would mean preventing the study from learning anything, which is the No-Free-Lunch result of Kifer & Machanavajjhala.

Correlated records. Take a count released with Laplace noise of scale $b = 1/\varepsilon$, which is ε-DP for one record. A household of k people who always answer alike moves the count by k. The log ratio of two Laplace densities whose centres are k apart is at most $k/b = k\varepsilon$, by the triangle-inequality argument of Section 18.3 (the Laplace pair of Part 1), and it reaches that value for outputs beyond

both centres. Record-level DP treats each household member as hiding behind the others, but the others carry the same information.

18.11.4. Interpretation

DP protects the individual's contribution, not the individual's secrets. If a secret can be inferred from other people's data, DP does not hide it. That is the price of letting anyone learn anything from data at all.

The unit of privacy must therefore match the unit of correlation. User-level DP, or group privacy with k set to the size of the correlated unit, is how you handle correlations you know about. Correlations you do not know about are outside the guarantee.

Finally, DP is a guarantee about an algorithm: a property of M as a mathematical object. Its implementation, its inputs and everything around it need separate care.

18.11.5. Example: a household of five

At $\varepsilon = 1$, a household of $k = 5$ correlated records faces a largest privacy loss of 5.0000 (found by the script on a grid), as the $k\varepsilon$ argument predicts. The household is therefore protected only as if by a 5-DP mechanism. The advantage bound Equation 18.13 at $\varepsilon = 5$ is 0.9866, and the exact best advantage against this Laplace count is $1 - e^{-5/2} = 0.9179$, the Laplace TV formula $1 - e^{-s/(2b)}$ with shift $s = 5$ and scale $b = 1$. An attacker who wants to know whether the household took part is right with probability up to $\frac{1}{2}(1 + 0.9179) = 0.959$, although each record is "protected at $\varepsilon = 1$".

18.11.6. Check yourself

- A model trained with DP predicts that people in a given postcode are likely to default on loans. Has it violated the privacy of those people? *(Not in the DP sense. The prediction would be the same with or without any one person's record; it reflects the population. Whether such inferences are acceptable is a question DP does not answer.)*
- A dataset contains one record per message, and users write up to 50 messages. Example-level $\varepsilon = 0.5$ is claimed. What is the user-level guarantee for a heavy user, with the group bound? *(Up to* $50 \times 0.5 = 25$*, which promises essentially nothing.)*

18.12. Code: check every number by enumeration and simulation

The script code/definition.py computes every number in this chapter and draws every figure. Each mechanism is a small typed function returning its output distribution. The definition is checked by brute force over all events where the outcome set is small, and the closed forms are checked against simulated surveys, membership games and Monte Carlo averages. From the repository root:

```
uv run python code/part03-dp-foundations/definition.py
```

Output (seed 0), summarized in Table 18.8:

Table 18.8.: The script's output with the default seed.

Experiment	Result
Histogram, Laplace noise $1/\varepsilon$ per bin, $\varepsilon = 1$	Largest privacy loss: 1.0000 under add/remove, 2.0000 under replace
Two disjoint uniform distributions, $m = 1000$	Pointwise δ 0.0010; event δ 1.0000
Randomized response, 3 people, 256 events	Largest event ratio 3.0000 = largest outcome ratio; $\max_S [P(S) - 2Q(S)]$ 0.2500 by brute force and by Fact 2
Warner's coins	$\Pr[\text{yes} \mid \text{yes}] = 0.75$, $\Pr[\text{yes} \mid \text{no}] = 0.25$, $\varepsilon = \ln 3 = 1.0986$
Survey, $n = 1000$, $\pi = 0.3$, $\varepsilon = \ln 3$	Standard deviation 0.0274 (formula), 0.0278 (10,000 simulated surveys, mean 0.2996)
$\delta(\varepsilon)$	Randomized response $\ln 3$: 0.3378 at 0.5, 0.0704 at 1. Laplace count: 0.2212 at 0.5, 0 at 1. Gaussian count: Table 18.4; Monte Carlo 6.859×10^{-3} at $\varepsilon = 1$
Testing	Largest TPR at FPR 10^{-3}: 0.002728 for $(1, 10^{-5})$. Gaussian count: 0.0048 exact against bound 0.0095
Membership advantage	Table 18.5; games at $\varepsilon = 1$: 0.4624 (randomized response), 0.3953 (Laplace count)
Bayesian	Table 18.6; Gaussian count: $\Pr[L > 2] = 8.842 \times 10^{-5}$ against bound 0.0108
Release each record with probability δ	$n = 3$, $\delta = 0.1$: exactly $(0, 0.1)$-DP. $n = 10^4$, $\delta = 1/n$: 1.0077 records and $\Pr[\geq 1] = 0.6351$ in 100,000 runs (exact 1 and 0.6321)
Household of 5 at $\varepsilon = 1$	Largest loss 5.0000; advantage bound 0.9866; Laplace exact 0.9179

tests/test_definition.py checks that the output still matches.

What to take from it. First, one formula replaces all events: the brute force over 256 events and the max-divergence and hockey-stick formulas agree exactly, and that agreement is what makes DP checkable for continuous outputs, where brute force is impossible. Second, δ depends on ε: every mechanism has a curve $\delta(\varepsilon)$, and the simple tail bound $\Pr[L > \varepsilon]$ overstates it by a factor of 4 to 13 for the Gaussian count, because it ignores the discount on outputs just past ε. Third, the bounds are tight, but not for every mechanism: randomized response meets the testing and advantage bounds exactly, while the Laplace count stays below them. Finally, a small δ is not automatically safe: $(0, 1/n)$-DP published at least one full record in 64% of runs.

18.13. References

- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (TCC 2006; journal version in the Journal of Privacy and Confidentiality, 2016). The original definition of pure DP, with sensitivity and the Laplace mechanism.
- Dwork, Kenthapadi, McSherry, Mironov & Naor, "Our Data, Ourselves: Privacy via Distributed Noise Generation" (EUROCRYPT 2006). Introduces approximate (ε, δ)-DP.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 2: the definition, randomized response, what DP promises and does not (§2.3), and the "release a random record" argument for small δ. The standard reference for this whole chapter.
- Warner, "Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias" (Journal of the American Statistical Association, 1965). The original two-coin protocol and its estimator.
- Wasserman & Zhou, "A Statistical Framework for Differential Privacy" (Journal of the American Statistical Association, 2010). DP as a bound on the power of tests.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015). The hypothesis-testing characterization Equation 18.11 and its use for optimal composition (Advanced and optimal composition).
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS Series B, 2022). Trade-off functions as the privacy definition itself (f-DP); Equation 18.12 is their $f_{\varepsilon,\delta}$.
- Kasiviswanathan & Smith, "On the 'Semantics' of Differential Privacy: A Bayesian Formulation" (Journal of Privacy and Confidentiality, 2014). The posterior reading for arbitrary priors, and the bad-set argument for approximate DP.
- Yeom, Giacomelli, Fredrikson & Jha, "Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting" (CSF 2018). Membership advantage and the bound $e^{\varepsilon} - 1$.
- Kifer & Machanavajjhala, "No Free Lunch in Data Privacy" (SIGMOD 2011). Why DP cannot protect correlated data without assumptions on how it was generated.
- Desfontaines & Pejó, "SoK: Differential Privacies" (PETS 2020). A map of the many variants of the definition, including the choice of neighbouring relation.
- Abadi et al., "Deep Learning with Differential Privacy" (CCS 2016). Example-level DP for training neural networks, with $\delta = 10^{-5}$ for 60,000 examples.

18.14. Cheat sheet

M is the mechanism, $D \simeq D'$ neighbours, $P = M(D)$, $Q = M(D')$, $L_{D,D'} = \ln(p/q)$ the privacy loss. Table 18.9 collects the results.

Table 18.9.: Every result of this chapter.

Result	Statement
(ε, δ)-DP	$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta$ for all $D \simeq D'$ and all S
Pure DP	$\delta = 0$; equivalently $\lvert L_{D,D'} \rvert \le \varepsilon$ everywhere; $D_{\infty}(P \Vert Q) \le \varepsilon$
Approximate DP	$E_{e^{\varepsilon}}(P \Vert Q) = \mathbb{E}_P[(1 - e^{\varepsilon - L})_+] \le \delta$; sufficient: $\Pr_P[L > \varepsilon] \le \delta$

Result	Statement
Add/remove vs replace	ε under add/remove $\Rightarrow 2\varepsilon$ under replace; $(\varepsilon, \delta) \Rightarrow (2\varepsilon, (1 + e^{\varepsilon})\delta)$
Randomized response	Truth with probability $e^{\varepsilon}/(1 + e^{\varepsilon})$; ε-DP; estimate error $e^{\varepsilon/2}/((e^{\varepsilon} - 1)\sqrt{n})$
Testing form	$\mathrm{FPR} + e^{\varepsilon}\,\mathrm{FNR} \geq 1 - \delta$ and $e^{\varepsilon}\,\mathrm{FPR} + \mathrm{FNR} \geq 1 - \delta$; $\mathrm{TPR} \leq e^{\varepsilon}\,\mathrm{FPR} + \delta$
Trade-off form	$T(P, Q) \geq f_{\varepsilon,\delta}(\alpha) = \max\{0, 1 - \delta - e^{\varepsilon}\alpha, e^{-\varepsilon}(1 - \delta - \alpha)\}$
Membership advantage	$\leq (e^{\varepsilon} - 1 + 2\delta)/(e^{\varepsilon} + 1)$; $\tanh(\varepsilon/2)$ for pure DP; tight for randomized response
Bayesian form	Posterior odds within $e^{\pm\varepsilon}$ of prior odds; with δ, except on outputs of probability $\leq \delta/(1 - e^{\varepsilon - \varepsilon'})$
Release each record w.p. δ	$(0, \delta)$-DP; exposes someone with probability $1 - (1 - \delta)^n$

Rules of thumb

- State the neighbouring relation and the unit of privacy with every ε.
- ε below 1 is strong, a few is moderate, and around 10 the worst-case guarantee says little.
- Pick $\delta \ll 1/n$, with n the number of people, not records.
- Low-FPR attack success is capped at $e^{\varepsilon}\,\mathrm{FPR} + \delta$: that is the number to compare with a measured membership attack.
- A bounded privacy loss gives pure DP; an unbounded one gives a whole curve $\delta(\varepsilon)$.
- DP protects your contribution, not secrets that others' data reveal about you.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

19. Post-processing, group privacy and composition

19.1. Why this matters

You have proved that one mechanism is differentially private. Now you use it. You round its output, plot it, or train a model on it. The dataset you protect has families in it, or users who each contributed fifty records. And you never run just one mechanism: an analyst asks a hundred queries, and a training run takes thousands of noisy gradient steps. Three questions come up at once:

- **After processing,** is the result still private, and do you need to prove anything new?
- **For a group** of k records, such as one user's records or one household, what does a guarantee about one record give?
- **For many mechanisms** run on the same data, what is the guarantee of all their outputs together?

The answers are the three properties that make differential privacy (DP) usable as an engineering tool. **Post-processing** is free: nothing you compute from the output without touching the data can weaken the guarantee. **Group privacy** costs a factor k in ε, and more than that in δ. **Composition** adds: running mechanisms with budgets $(\varepsilon_i, \delta_i)$ costs at most $(\sum \varepsilon_i, \sum \delta_i)$. Without these three properties, every pipeline would need its own privacy proof. With them, you prove each building block once and add up the costs.

19.1.1. The one idea: privacy losses add, processing only averages them

A mechanism M is a randomized algorithm that takes a dataset and returns an output. Two datasets are **neighbouring**, written $D \simeq D'$, when they differ in one person's record. Differential privacy asks that M behave almost the same on every neighbouring pair, so that its output reveals little about any one record. The way to measure "almost the same" at a single output y is to ask how much more likely y is when the data are D than when they are D'. If y is three times as likely under D, seeing y is evidence for D; if it is equally likely under both, seeing it tells you nothing. The logarithm of that ratio is the **privacy loss** of y, from The definition of differential privacy. Writing $p_{M(D)}(y)$ for the probability (or density) that M outputs y on data D,

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}. \tag{19.1}$$

It is positive when y favours D, negative when it favours D', and zero when y carries no evidence either way. Pure ε-DP says that no output carries much evidence: $|L_{D,D'}(y)| \leq \varepsilon$ for every output and every neighbouring pair $D \simeq D'$. Equivalently, the likelihood ratio e^L always lies in the band $[e^{-\varepsilon}, e^{\varepsilon}]$. All three properties of this chapter are statements about how L behaves.

Post-processing replaces y by $z = f(y)$. Several outputs y can lead to the same z, and the probability of z is the sum of their probabilities. So the likelihood ratio of z is a weighted average of the likelihood ratios of those y, which is the content of the data processing inequality (DPI) of Divergences: processing pools evidence, and pooling can only dilute it. An average of numbers that all lie in $[e^{-\varepsilon}, e^{\varepsilon}]$ stays in $[e^{-\varepsilon}, e^{\varepsilon}]$, so the processed output is just as private.

Group privacy compares D with a dataset D'' that differs in k records. You can walk from D to D'' one record at a time, through k neighbouring pairs. The likelihood ratio between the two ends is the product of the k ratios along the way, so the privacy loss between the ends, its logarithm, is the sum of the k step losses, each at most ε.

Composition releases $y = (y_1, \ldots, y_k)$ from k mechanisms that use independent coins. The probability of independent outcomes is the product of their probabilities, so the log-ratio of the whole release, its privacy loss, is the sum of the individual losses.

So the rule is: **along a path of datasets or across a sequence of mechanisms, privacy losses add; under processing, they only average.** Table 19.1 lists the results and where each is used.

Table 19.1.: The results of this chapter and where each is used.

Result	Where it is used
Post-processing	Publishing anything computed from a private output: rounded counts, plots, models trained on private statistics, synthetic data; every attack of Part 2 is a post-processing
Group privacy, pure and approximate	Turning a per-record guarantee into a per-user or per-household one; why correlated records weaken DP
Basic sequential composition, non-adaptive and adaptive	Budgeting an analysis of many queries; the starting point for every accountant in Advanced and optimal composition and later
Parallel composition	Histograms, per-group statistics, local randomized response: disjoint data cost the maximum, not the sum
What composes and what doesn't	Fresh noise, matching neighbouring relations, not touching the data again

How to read this chapter. Each section follows the same arc: context, statement, derivation, interpretation, example, check yourself. Read the post-processing and composition sections first; they are used everywhere. The approximate-DP derivations (Fact 2 in the toolkit, the two-step lemma in the composition section) are short but abstract; walk through the concrete tables next to them. Basic composition is tight in the worst case but loose for most purposes. How much better you can do is the subject of Advanced and optimal composition; here you only see why there is room.

Conventions. M is a randomized mechanism, $D \simeq D'$ neighbouring datasets, and $P = M(D)$, $Q = M(D')$ its two output distributions with mass functions or densities p and q. Sums over outputs $\sum_y$ are integrals for continuous outputs. Logarithms are natural. The running mechanism is **randomized response** with $\varepsilon_0 = \ln 3$: a person reports their true bit with probability $\frac{3}{4}$ and the other bit with probability $\frac{1}{4}$. Comparing a person whose bit is 1 with one whose bit is 0, the report 1 has probability $\frac{3}{4}$ against $\frac{1}{4}$, a ratio of 3, and the report 0 has the ratio $\frac{1}{3}$. So $L = \pm \ln 3$ on every output.

19.2. Toolkit: three facts every derivation uses

19.2.1. Fact 1 — Three forms of the definition

The definition of (ε, δ)-DP can be written in three ways, and each derivation below uses whichever makes its step shortest. The first form talks about events: whatever set S of outputs an observer cares about, its probability under D is at most e^ε times its probability under D', plus a small slack δ. The second form packs all events into one number, the **hockey-stick divergence**: scale q up by the factor e^ε and add up the probability mass of p that still sticks out above it. Here $(x)_+ = \max(x, 0)$ keeps only the parts that stick out. The third form, for $\delta = 0$, is the bound on the privacy loss you already met. From The definition of differential privacy, these are equivalent for a pair $P = M(D)$, $Q = M(D')$:

1. **Events:** $P(S) \le e^\varepsilon Q(S) + \delta$ for every set of outputs S.
2. **Hockey-stick:** $E_{e^\varepsilon}(P\|Q) = \sum_y \left(p(y) - e^\varepsilon q(y)\right)_+ \le \delta$.
3. **Pure case ($\delta = 0$):** $|L_{D,D'}(y)| \le \varepsilon$ for every y (on the common support).

M is (ε, δ)-DP when this holds for every ordered neighbouring pair.

Forms 1 and 2 are equivalent because the worst event is easy to find. The gap $P(S) - e^\varepsilon Q(S) = \sum_{y \in S} \left(p(y) - e^\varepsilon q(y)\right)$ is a sum of one term per output in S. It is largest when S contains exactly the outputs whose term is positive, those with $p(y) > e^\varepsilon q(y)$, and its largest value is then the sum of the positive parts, which is $E_{e^\varepsilon}(P\|Q)$. This is Fact 3 of Divergences. So "every event satisfies the bound" and "the worst event satisfies it" say the same thing. Form 3 follows from form 1 with $\delta = 0$ by taking single outputs, $S = \{y\}$: this gives $p(y) \le e^\varepsilon q(y)$, and the reversed pair gives $q(y) \le e^\varepsilon p(y)$, which together say $|L(y)| \le \varepsilon$. Conversely, summing $p(y) \le e^\varepsilon q(y)$ over the outputs $y \in S$ gives the event form back.

Since δ is the hockey-stick divergence at the scale e^ε, you can ask for every ε at once how much slack is needed. The function $\varepsilon \mapsto \delta(\varepsilon) = \max\{E_{e^\varepsilon}(P\|Q), E_{e^\varepsilon}(Q\|P)\}$, maximized over neighbouring pairs, is the **privacy profile** of M. M is (ε, δ)-DP exactly when $\delta(\varepsilon) \le \delta$. The script computes it exactly for every mechanism in this chapter.

19.2.2. Fact 2 — Tests instead of events

An event is an all-or-nothing rule: an output is in S or it is not. Later steps of a pipeline are often softer than that. A randomized test rejects with some probability that depends on the output, and a later stage of an algorithm does something with a probability that depends on what

it was given. Such a rule is a function g from outputs to $[0, 1]$. It is a mixture of events, so it should obey the same bound, and it does. If $P(S) \le e^{\varepsilon} Q(S) + \delta$ for every event S, then for every function g of the output with values in $[0, 1]$,

$$\mathbb{E}_P[g(Y)] \le e^{\varepsilon}\, \mathbb{E}_Q[g(Y)] + \delta. \tag{19.2}$$

The proof reuses the worst-event argument of Fact 1. The gap $\mathbb{E}_P[g] - e^{\varepsilon}\mathbb{E}_Q[g] = \sum_y g(y)(p(y) - e^{\varepsilon} q(y))$ is a sum in which each value $g(y)$ multiplies a fixed bracket. To make it as large as possible, you set $g(y) = 1$ where the bracket is positive and $g(y) = 0$ where it is negative; any value in between only gives up part of a positive term or adds part of a negative one. That choice of g is the indicator of the worst event, and the maximum is $E_{e^{\varepsilon}}(P\|Q) \le \delta$. This is the randomized version of Fact 3 in Divergences.

A small case shows what the fact says. Take randomized response with $\varepsilon = \ln 3$, $\delta = 0$, and a later step that acts with probability $g(1) = 0.6$ on the report 1 and $g(0) = 0.2$ on the report 0. Under bit 1 the report is 1 with probability $\frac{3}{4}$, so $\mathbb{E}[g] = \frac{3}{4}(0.6) + \frac{1}{4}(0.2) = 0.5$; under bit 0, $\frac{1}{4}(0.6) + \frac{3}{4}(0.2) = 0.3$. Indeed $0.5 \le 3 \times 0.3 = 0.9$. You will use this fact for post-processing, and in every approximate-DP proof where a later stage depends on an earlier output.

19.2.3. Fact 3 — Privacy losses of independent outputs add

When two outputs come from independent coins, the probability of seeing both is the product of their probabilities. The logarithm turns that product into a sum, so the evidence carried by the pair is the evidence of the first plus the evidence of the second. In symbols, if $Y = (Y_1, Y_2)$ where Y_1 and Y_2 use independent coins, then $p(y_1, y_2) = p_1(y_1)p_2(y_2)$ and the same for q, so

$$L(y_1, y_2) = \ln \frac{p_1(y_1)p_2(y_2)}{q_1(y_1)q_2(y_2)} = L_1(y_1) + L_2(y_2). \tag{19.3}$$

Take two randomized-response answers from a person whose bit is 1 (D) or 0 (D'). The answers $(1, 1)$ have probability $\frac{3}{4} \cdot \frac{3}{4} = \frac{9}{16}$ under D and $\frac{1}{4} \cdot \frac{1}{4} = \frac{1}{16}$ under D', so $L = \ln 9 = \ln 3 + \ln 3$: two pieces of evidence pointing the same way. The answers $(1, 0)$ have $\frac{3}{4} \cdot \frac{1}{4} = \frac{3}{16}$ under both, so $L = 0 = \ln 3 - \ln 3$: the two pieces of evidence cancel.

The fact survives when the second mechanism depends on the first output (the **adaptive** case). The chain rule of probability writes any joint probability as "first, times second given first", $p(y_1, y_2) = p_1(y_1)\, p_2(y_2 \mid y_1)$, so the same sum holds with $p_2(y_2 \mid y_1)$ in place of $p_2(y_2)$. This is what composition and parallel composition use.

19.3. Post-processing

19.3.1. Context

A private output is rarely published as is. You round a noisy count to an integer, clip it at zero, threshold it, fit a model to a table of noisy marginals, or let an analyst look at it and decide what to do next. Each of these is a function of the output, possibly randomized with its own coins, that

does not look at the data again. The question is whether such processing can undo the protection. It cannot: the data processing inequality of Part 1 says that processing averages likelihood ratios together, and averaging cannot push a ratio outside the band $[e^{-\varepsilon}, e^{\varepsilon}]$.

19.3.2. Statement

To state the result for randomized processing too, describe the processing as a **channel** K: $K(z \mid y)$ is the probability that the processing outputs z when its input is y. A deterministic f is the case $K(z \mid y) = \mathbf{1}[z = f(y)]$, which is 1 when $z = f(y)$ and 0 otherwise. The processed output on data D then has the distribution $PK(z) = \sum_y p(y)K(z \mid y)$: sum over every y the mechanism could give, weighted by how likely it is to become z. Write $L^f_{D,D'}(z)$ for the privacy loss of the processed output z.

The claim is that processing can never make the two worlds easier to tell apart. Let M be (ε, δ)-DP and let f be any randomized function whose coins are independent of the data and of M's coins. Then $f \circ M$, the mechanism $D \mapsto f(M(D))$, is (ε, δ)-DP. Equivalently, for every channel K and every neighbouring pair, the hockey-stick divergence and the largest privacy loss can only go down:

$$E_{e^\varepsilon}(PK \,\|\, QK) \le E_{e^\varepsilon}(P \| Q), \qquad \max_z \left|L^f_{D,D'}(z)\right| \le \max_y \left|L_{D,D'}(y)\right|, \tag{19.4}$$

where $PK(z) = \sum_y p(y)K(z \mid y)$ is the distribution of $f(M(D))$. In words: the privacy profile of $f \circ M$ lies on or below that of M at every ε.

19.3.3. Derivation

The plan is to turn any event about the processed output into a soft test on the original output, then apply Fact 2. Recall that $K(z \mid y)$ is the probability that f outputs z on input y.

1. **Fix an event S of processed outputs.** Let $g(y) = K(S \mid y) = \sum_{z \in S} K(z \mid y)$, the probability that processing y lands in S. It is a probability, so it takes values in $[0, 1]$. It does not depend on the data, because f does not look at the data: the same g serves for D and for D'.
2. **Average over Y.** To land in S, the mechanism first produces some y, and then processing sends it into S with probability $g(y)$. Summing over y, $\Pr[f(M(D)) \in S] = \sum_y p(y)g(y) = \mathbb{E}_P[g(Y)]$, and the same with q for D'.
3. **Apply Fact 2** to g: $\mathbb{E}_P[g] \le e^\varepsilon \mathbb{E}_Q[g] + \delta$, that is $\Pr[f(M(D)) \in S] \le e^\varepsilon \Pr[f(M(D')) \in S] + \delta$. This holds for every S and every ordered neighbouring pair, which is the event form of (ε, δ)-DP for $f \circ M$ (Fact 1). The hockey-stick inequality of Equation 19.4 is the same statement for the worst event S.
4. **The privacy-loss form.** This is step 1 of the DPI derivation in Divergences. The likelihood ratio of z is $PK(z)/QK(z)$. In the numerator, write each $p(y)$ as $q(y)$ times its likelihood ratio $e^{L(y)} = p(y)/q(y)$. That turns the ratio into a weighted average of the likelihood ratios of the y that can produce z:

$$e^{L^f(z)} = \frac{PK(z)}{QK(z)} = \sum_y \frac{q(y)K(z \mid y)}{QK(z)}\, e^{L(y)}. \tag{19.5}$$

The weight of each y is the share of the D'-probability of z that comes through y. The weights are non-negative and sum to 1, because $\sum_y q(y)K(z \mid y) = QK(z)$. So $e^{L^f(z)}$ lies between the smallest and the largest $e^{L(y)}$, which is the second inequality in Equation 19.4.

Concretely, randomized response followed by a second coin. You collect answers with $\varepsilon_0 = \ln 3$ (truth with probability $\frac{3}{4}$), then, before publishing, flip each answer with probability $f = \frac{1}{4}$ using a fresh coin. (Here f is a number, the flip probability.) The published answer $z = 1$ can be reached in two ways: the first answer was 1 and was kept, or it was 0 and was flipped. Each path's probability is the product of its two steps. Table 19.2 follows the published answer $z = 1$.

Table 19.2.: Randomized response with $\varepsilon_0 = \ln 3$, then a fresh coin that flips the answer with probability $\frac{1}{4}$. The published $z = 1$ is reached by two paths.

Path to $z = 1$	Probability under bit 1 (D)	Probability under bit 0 (D')	$e^{L(y)}$ of the first answer
$y = 1$, kept	$\frac{3}{4} \cdot \frac{3}{4} = \frac{9}{16}$	$\frac{1}{4} \cdot \frac{3}{4} = \frac{3}{16}$	3
$y = 0$, flipped	$\frac{1}{4} \cdot \frac{1}{4} = \frac{1}{16}$	$\frac{3}{4} \cdot \frac{1}{4} = \frac{3}{16}$	$\frac{1}{3}$
total	$\frac{10}{16} = 0.625$	$\frac{6}{16} = 0.375$	—

The likelihood ratio of $z = 1$ is $0.625/0.375 = \frac{5}{3}$. Step 4 predicts it as the average of 3 and $\frac{1}{3}$, weighted by each path's share of the D' column: each path contributes $\frac{3}{16}$ of the total $\frac{6}{16}$, a weight $\frac{3/16}{6/16} = \frac{1}{2}$ each, so $\frac{1}{2}(3 + \frac{1}{3}) = \frac{5}{3}$. The published $z = 0$ gets the remaining probability, $\frac{6}{16}$ under D and $\frac{10}{16}$ under D', a ratio of $\frac{3}{5}$. So the published answers are $\ln \frac{5}{3} = 0.5108$-DP, less than $\ln 3 = 1.0986$. The right panel of Figure 19.1 shows ε for every flip probability: it falls to 0 at $f = \frac{1}{2}$, where the published bit is a fair coin whatever the first answer was.

19.3.4. Interpretation

The first consequence is that nothing downstream needs a new proof. A model trained on DP statistics, synthetic data sampled from a DP model, a plot, a dashboard: all are functions of the private output that never read the data again, so all inherit the guarantee. This is why DP systems privatize at one narrow point (noisy counts, noisy gradients) and treat everything after it as public.

The same argument covers attackers. A membership-inference attacker of Part 2, who tries to tell whether a given record was in the data, sees y and outputs a guess $A(y)$, possibly using auxiliary knowledge and its own randomness. That guess is a channel applied to y, so it is a post-processing. So the bounds of The definition of differential privacy on an attack's true and false positive rates hold for every attack, including attacks invented after the release. In hypothesis-testing terms: the **trade-off curve** of Hypothesis testing gives, for each false positive rate, the smallest false negative rate any test can reach. Under processing it can only rise, because any test on z is also a test on y: compute z from y, then apply it.

How much does processing lose? Deterministic, invertible processing (relabelling, a change of units) keeps the privacy loss exactly, since each z comes from a single y and nothing is averaged. Processing that merges outputs with different privacy losses lowers it, as the coin flip does when

it mixes a ratio of 3 with a ratio of $\frac{1}{3}$. Most processing you do for utility (rounding, clipping) merges only a little, so do not count on it: the guarantee you report is the one of M.

The one condition is that f must not touch the data, because step 1 of the derivation needed the same g under D and D'. "Compute the noisy count, then look up the exact count of the winning category" is not post-processing. The second step reads the data, so it is a second mechanism, and a non-private one (Section 19.7).

19.3.5. Example: rounding and thresholding a noisy count

Add Laplace noise of scale $b = 1$ to a count. Neighbouring datasets have counts 0 and 1, so the outputs are $\mathrm{Lap}(0, 1)$ and $\mathrm{Lap}(1, 1)$, with densities $\frac{1}{2}e^{-|y|}$ and $\frac{1}{2}e^{-|y-1|}$. Their privacy loss at y is $|y-1| - |y|$. Two points 1 apart cannot differ in their distance to y by more than 1 (the triangle inequality: $|y-1| \le |y|+1$ and $|y| \le |y-1|+1$), so this loss is never more than 1 in size, and it equals $(1-y)-(-y) = 1$ for every $y \le 0$. The largest log-ratio, the max-divergence of Divergences, is therefore 1: the noisy count is 1-DP. (The Laplace mechanism in general is in The Laplace and Gaussian mechanisms.) Now post-process it in two ways: round it to the nearest integer, or publish only whether it exceeds $\frac{1}{2}$. Table 19.3 gives the exact privacy profiles from the script, and Figure 19.1 draws them.

Table 19.3.: Exact privacy profiles of a count with $\mathrm{Lap}(0, 1)$ noise, before and after two post-processings. Each row is on or below the one above it, as Equation 19.4 requires.

Published	Smallest pure ε	$\delta(0.25)$	$\delta(0.5)$
Noisy count $c + Z$	1.0000	0.3127	0.2212
Rounded to an integer	1.0000	0.3073	0.1967
Only "is $c + Z > \frac{1}{2}$?"	0.8318	0.3073	0.1967

Three things can be read from the table and the figure, and each follows from the averaging picture of step 4.

Rounding keeps the pure ε. Every output $j \le -1$ of the rounded count collects the cell $[j-\frac{1}{2}, j+\frac{1}{2}]$, which lies entirely left of 0. There the two Laplace densities have the constant ratio e, so rounding merges outputs with the same loss, and an average of equal ratios is that ratio: nothing is averaged away.

Rounding still lowers $\delta(\varepsilon)$ for smaller ε, because of the cell around 0. That cell, $[-\frac{1}{2}, \frac{1}{2}]$, mixes outputs whose losses run from -1 to 1. Its probability is $1 - e^{-1/2} = 0.3935$ when the count is 0 and $\frac{1}{2}(e^{-1/2} - e^{-3/2}) = 0.1917$ when it is 1, so its loss is $\ln(0.3935/0.1917) = 0.7191$. For ε below that value, the worst event of the rounded output is "the rounded count is at most 0". That is the same as "$c + Z < \frac{1}{2}$", exactly the threshold event, so the rounded and thresholded curves coincide there.

The one-bit summary is the most private. Its "yes" event, $c + Z > \frac{1}{2}$, needs $Z > \frac{1}{2}$ when the count is 0, which has probability $\frac{1}{2}e^{-1/2} = 0.3033$ under the Laplace tail, and $Z > -\frac{1}{2}$ when the count is 1, which has probability $1 - 0.3033 = 0.6967$. Its privacy loss is $\ln(0.6967/0.3033) = 0.8318$, and the "no" event gives the same loss with the opposite sign.

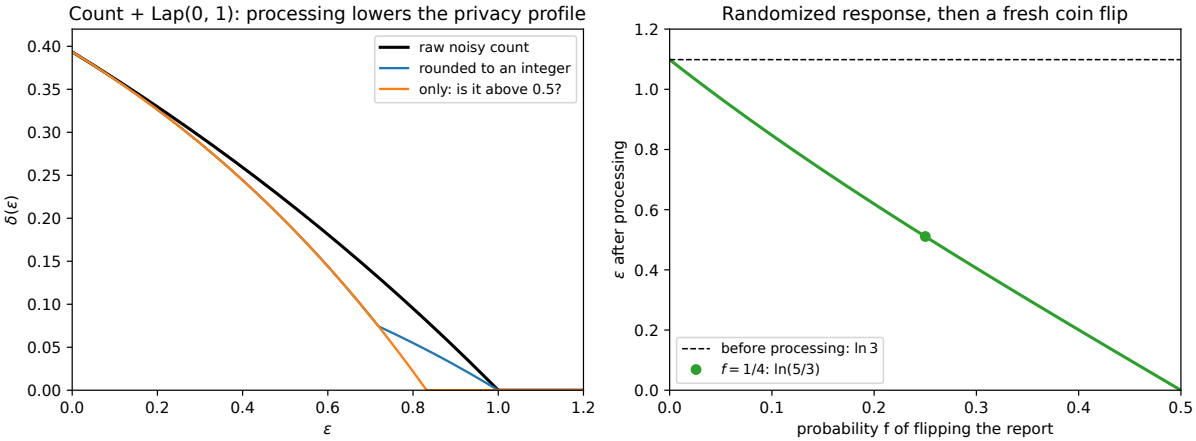


Figure 19.1.: Left: the privacy profile $\delta(\varepsilon)$ of a count with $\mathrm{Lap}(0, 1)$ noise (black), after rounding to an integer (blue), and after keeping only whether it exceeds $\frac{1}{2}$ (orange). Post-processing can only lower the curve. Right: randomized response with $\varepsilon_0 = \ln 3$ followed by a fresh coin that flips the answer with probability f; the dot is $f = \frac{1}{4}$, where $\varepsilon = \ln \frac{5}{3}$.

19.3.6. Check yourself

- A data scientist reads a DP histogram, notices that one bin looks odd, and decides to publish a second histogram with finer bins. Is the second histogram covered by post-processing? *(No. Choosing to publish is post-processing, but computing a finer histogram reads the data again. It is a second mechanism, and its cost adds by composition.)*
- Can post-processing turn a $(1, 10^{-6})$-DP mechanism into a $(0.5, 10^{-6})$-DP one? *(It can, as the coin flip shows, but only by destroying information. It can never do the reverse.)*

19.4. Group privacy

19.4.1. Context

The definition protects one record: datasets that differ in one row. But people are not rows. A user of a messaging app contributes hundreds of messages, a household shares an address and an income, and a disease runs in a family. If the neighbouring relation is example-level and you want to know what an adversary learns about all k records of one user at once, you need a guarantee for datasets that differ in k records. Group privacy gives it from the one-record guarantee, at a price.

19.4.2. Statement

Say D and D'' are at **distance** k if a path $D = D_0 \simeq D_1 \simeq \cdots \simeq D_k = D''$ joins them, changing one record per step. The idea is to apply the one-record guarantee once per step and chain the results. Each step multiplies the probability bound by e^{ε} and adds a slack δ, so after k steps the

factor is $e^{k\varepsilon}$. The slacks pile up in a geometric sum, because a δ added early is multiplied by e^{ε} again at every later step. If M is (ε, δ)-DP, then for every such pair and every event S,

$$\Pr[M(D) \in S] \le e^{k\varepsilon} \Pr[M(D'') \in S] + \delta_k, \qquad \delta_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1} \le k \, e^{(k-1)\varepsilon} \delta. \quad (19.6)$$

In particular, pure ε-DP gives pure $k\varepsilon$-DP for groups of size k. For $\delta > 0$ the bound δ_k grows exponentially in k, and it is attained by some mechanisms.

19.4.3. Derivation

Pure DP, through the privacy loss. The likelihood ratio between the two ends of the path factors into the ratios of the steps, so the losses add, and each one is at most ε.

1. **Telescope the likelihood ratio** along the path. Multiply and divide by the densities at the intermediate datasets $D_1, \dots, D_{k-1}$; each one appears once in a numerator and once in a denominator, so they cancel and only the two ends remain:

$$\frac{p_{M(D_0)}(y)}{p_{M(D_k)}(y)} = \prod_{i=1}^{k} \frac{p_{M(D_{i-1})}(y)}{p_{M(D_i)}(y)}, \qquad \text{so} \qquad L_{D_0, D_k}(y) = \sum_{i=1}^{k} L_{D_{i-1}, D_i}(y). \quad (19.7)$$

2. **Bound each term** by Fact 1, form 3: each pair $D_{i-1} \simeq D_i$ is neighbouring, so $|L_{D_{i-1}, D_i}(y)| \le \varepsilon$.
3. **Add:** a sum of k terms, each between $-\varepsilon$ and ε, lies between $-k\varepsilon$ and $k\varepsilon$, so $|L_{D_0, D_k}(y)| \le k\varepsilon$ for every y.

Concretely, a noisy count. Take a count with Lap$(0, 1)$ noise and the path of counts $0, 1, 2, 3$ ($k = 3$). The noisy count has density $\frac{1}{2} e^{-|y-c|}$ when the true count is c. The ratio of the densities at counts c and $c+1$ is $e^{|y-c-1|-|y-c|}$, so the privacy loss between counts c and $c+1$ at output y is $L_{c,c+1}(y) = |y - c - 1| - |y - c|$ (the ratio of two Laplace densities): how much farther y is from $c+1$ than from c. Table 19.4 adds the steps at three outputs.

Table 19.4.: Privacy losses along a path of three neighbouring counts, with Lap$(0, 1)$ noise. The step losses add up to the loss between the ends.

Output y	$L_{0,1}(y)$	$L_{1,2}(y)$	$L_{2,3}(y)$	Sum	Direct: $\lvert y-3\rvert - \lvert y\rvert$
-2	$3 - 2 = 1$	$4 - 3 = 1$	$5 - 4 = 1$	3	$5 - 2 = 3$
0.5	$0.5 - 0.5 = 0$	$1.5 - 0.5 = 1$	$2.5 - 1.5 = 1$	2	$2.5 - 0.5 = 2$
1.5	$0.5 - 1.5 = -1$	$0.5 - 0.5 = 0$	$1.5 - 0.5 = 1$	0	$1.5 - 1.5 = 0$

At $y = -2$ the output lies left of every count on the path, so every step points the same way and the sum reaches 3ε. At $y = 1.5$, in the middle of the path, the steps point in different directions and cancel. The script confirms that the rounded noisy count has pure ε exactly k for counts that differ by $k = 1, 2, 3, 5$: for a count, group privacy is tight.

Approximate DP, through events. With $\delta > 0$ the privacy loss is no longer bounded, so there is nothing to add up. Instead, chain the event form of the definition: bound the probability under D_0 by the one under D_1, that one by the one under D_2, and so on.

1. **One step.** For every event S, $\Pr[M(D_{i-1}) \in S] \le e^{\varepsilon} \Pr[M(D_i) \in S] + \delta$ (Fact 1, form 1).
2. **Substitute the next step** into the right side. Writing $P_i = \Pr[M(D_i) \in S]$, the bound on P_1 goes in place of P_1:

$$P_0 \le e^{\varepsilon} P_1 + \delta \le e^{\varepsilon}\left(e^{\varepsilon} P_2 + \delta\right) + \delta = e^{2\varepsilon} P_2 + \delta(1 + e^{\varepsilon}). \tag{19.8}$$

3. **Repeat k times** (induction): $P_0 \le e^{k\varepsilon} P_k + \delta(1 + e^{\varepsilon} + \cdots + e^{(k-1)\varepsilon})$. Each new δ enters before $k-1, k-2, \ldots, 0$ more multiplications by e^{ε}, which is where the geometric sum comes from.
4. **Simplify:** the geometric sum is $\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1}$ (multiply it by $e^{\varepsilon}-1$ and all terms but the first and last cancel). Each of its k terms is at most the largest one, $e^{(k-1)\varepsilon}$, giving the simpler but looser $k\, e^{(k-1)\varepsilon}\delta$.

The bound is attained. To show that no better bound holds for all (ε, δ)-DP mechanisms, build one for which every inequality in the chain is an equality. That needs an event on which each one-step bound is met exactly, so that step 2 loses nothing. Here is a mechanism on counts for which Equation 19.6 holds with equality. Fix a size K and let M output "alarm" with probability $a_c = \delta \sum_{j=0}^{K-c-1} e^{j\varepsilon}$ when the count is $c \le K$ (and $a_c = 0$ above K), and "quiet" otherwise. Pulling the first term out of the sum and e^{ε} out of the rest shows that the a_c satisfy $a_c = e^{\varepsilon} a_{c+1} + \delta$, so each neighbouring pair meets the event form with equality on "alarm", and the script checks that the other events are also within (ε, δ). For counts 0 and k the telescoping of step 3 runs with equality: $a_0 - e^{k\varepsilon} a_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} = \delta_k$.

19.4.4. Interpretation

Pure DP degrades linearly. A 0.1-DP guarantee per record is a 1-DP guarantee for a user with 10 records and a 10-DP guarantee, almost nothing, for a user with 100: a likelihood ratio of up to $e^{10} \approx 22{,}000$ lets an adversary be nearly certain. This is the formal version of the "correlated records" caveat of The definition of differential privacy: if one person's data appear in k rows, protecting one row does not protect the person.

Approximate DP degrades exponentially in δ. With $\varepsilon = 1$ and $k = 10$, the geometric sum gives $\delta_k = \frac{e^{10}-1}{e-1}\,\delta = 12818.3\,\delta$. And δ must already be much smaller than $1/n$ for a dataset of n records, because δ is a probability of failing outright: a mechanism that publishes each record in full with probability δ is $(0, \delta)$-DP and still leaks $n\delta$ records on average (The definition of differential privacy). Multiply such a δ by thousands and group privacy for large groups is often vacuous.

The bound is generic; a specific mechanism can do much better. For a count with Gaussian noise, a group of k records just shifts the count by k instead of 1, so its exact group profile is that of a Gaussian pair with a larger shift, as the Gaussian column of Table 19.5 shows. The tool that tracks this exactly for the Gaussian is in Part 4.

In practice, the better fix is to change the neighbouring relation. If you care about users, define neighbours as datasets that differ in one user's records, bound each user's contribution (clip it), and prove DP directly at that level. Group privacy is the fallback when you cannot.

19.4.5. Example: from records to users

Each record is protected at $(\varepsilon, \delta) = (0.5, 10^{-5})$. Table 19.5 compares the bound with two exact answers. The first is the alarm mechanism, built to attain the bound. The second is a count with Gaussian noise of standard deviation σ, calibrated so that $\delta(0.5) = 10^{-5}$ exactly at shift 1. For two normal distributions with standard deviation σ whose means differ by s, the privacy loss is a straight line in the output, so the event "loss above ε" is a half-line and the hockey-stick divergence is a difference of two normal tail probabilities. With $\mu = s/\sigma$ and Φ the standard normal cumulative distribution function, it is $\Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) - e^{\varepsilon}\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$, the closed form from Divergences. The script finds $\sigma = 7.0318$ by bisection on it, and for a group of k records it evaluates the same formula with the shift $s = k$. Figure 19.2 plots the same numbers up to $k = 12$.

Table 19.5.: Group privacy from a $(0.5, 10^{-5})$ guarantee per record: the bound, its simple form, and the exact δ_k of two mechanisms at $\varepsilon_k = k\varepsilon$.

Group size k	$\varepsilon_k = k\varepsilon$	δ_k from Equation 19.6	Simple form $ke^{(k-1)\varepsilon}\delta$	Alarm mechanism, exact	Gaussian count, exact
1	0.5	1.000×10^{-5}	1.000×10^{-5}	1.000×10^{-5}	1.000×10^{-5}
2	1	2.649×10^{-5}	3.297×10^{-5}	2.649×10^{-5}	2.551×10^{-5}
3	1.5	5.367×10^{-5}	8.155×10^{-5}	5.367×10^{-5}	4.858×10^{-5}
5	2.5	1.724×10^{-4}	3.695×10^{-4}	1.724×10^{-4}	1.288×10^{-4}
10	5	2.272×10^{-3}	9.002×10^{-3}	2.272×10^{-3}	7.592×10^{-4}

The geometric sum is exact for the alarm mechanism at every k, so no better bound holds for all (ε, δ)-DP mechanisms. The simple form replaces every term of the sum by the largest one, so it is looser, by a factor $3.297/2.649 = 1.24$ at $k = 2$ and $9.002/2.272 = 3.96$ at $k = 10$; use the sum. The Gaussian count is a factor $2.272/0.7592 = 2.99$ below the bound at $k = 10$, and the gap widens with k (Figure 19.2). Even so, for a user with 10 records the record-level $(0.5, 10^{-5})$ becomes $(5, 7.6 \times 10^{-4})$: a weak guarantee.

19.4.6. Check yourself

- A dataset has one row per hospital visit, and a mechanism is 0.2-DP at the visit level. A patient made 7 visits. What guarantee does the patient get? *($7 \times 0.2 = 1.4$-DP, by group privacy, assuming nothing else about the mechanism.)*
- Why does the derivation for $\delta > 0$ not just add k copies of δ? *(Each step multiplies everything before it, including the δs already accumulated, by e^{ε}. A δ that entered early is inflated by the later steps.)*

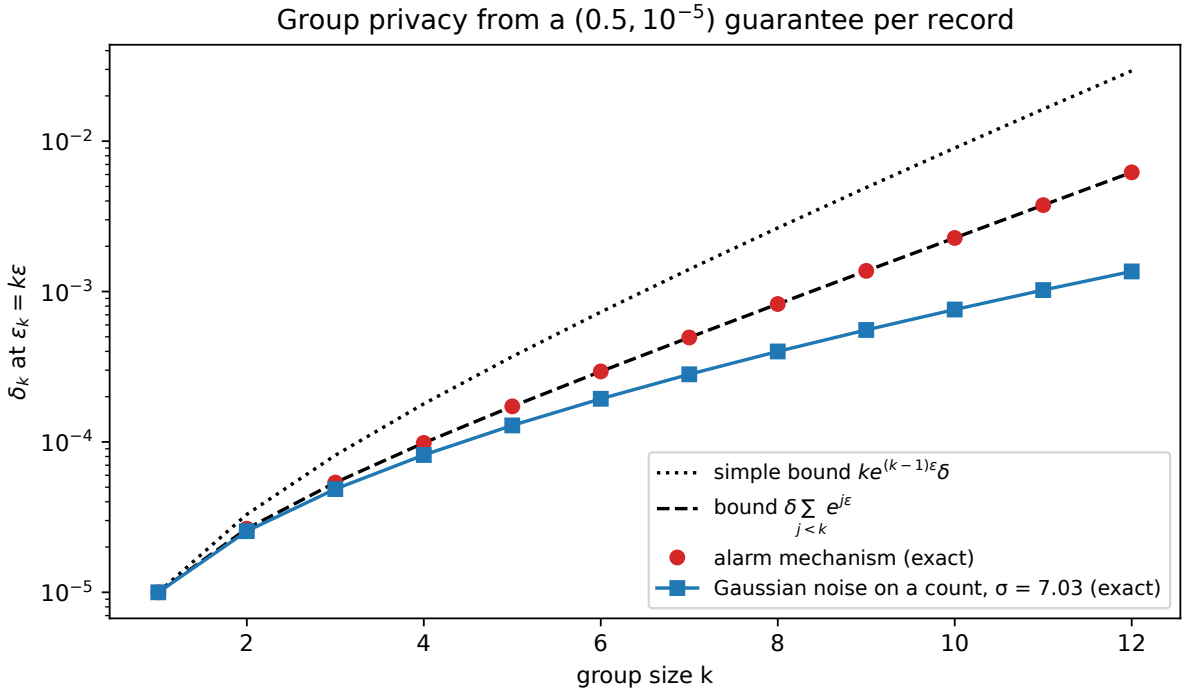


Figure 19.2.: The δ of group privacy at $\varepsilon_k = k\varepsilon$, for a per-record guarantee of $(0.5, 10^{-5})$, on a log scale. The geometric-sum bound (dashed) is attained by the alarm mechanism (red dots); the simple form (dotted) is looser by a factor that grows with k; Gaussian noise on a count (blue squares) does better than the bound but still grows quickly with k.

19.5. Basic sequential composition

19.5.1. Context

An analyst asks k queries of the same dataset, each answered by a DP mechanism with fresh noise. A training run takes k noisy gradient steps. A survey asks the same question twice. Each output alone is private, but an adversary sees all of them. The question is what guarantee holds for the whole transcript $(y_1, \dots, y_k)$, including when each mechanism is chosen after seeing the previous outputs. Fact 3 already gives the answer for pure DP: outputs with independent coins have probabilities that multiply, so their privacy losses add, and so do the budgets.

19.5.2. Statement

Let $M_1, \dots, M_k$ run on the same dataset with independent coins. Each M_i may also take the previous outputs $y_1, \dots, y_{i-1}$ as input, the way an analyst picks the next query after reading the last answer; this is **adaptive** composition, and **non-adaptive** composition is the special case where M_i ignores them. The condition is that each step is private however the earlier outputs turned out. If, for every fixed value of the previous outputs, $D \mapsto M_i(D; y_1, \dots, y_{i-1})$ is $(\varepsilon_i, \delta_i)$-DP, then the mechanism that releases $(y_1, \dots, y_k)$ is

$$\Big(\sum_{i=1}^{k} \varepsilon_i,\ \sum_{i=1}^{k} \delta_i\Big)\text{-DP}. \tag{19.9}$$

For pure DP the bound $\sum \varepsilon_i$ is the exact worst-case privacy loss whenever each M_i attains its own worst case at outputs that can occur together, as k randomized-response answers about the same bit do.

19.5.3. Derivation

Pure DP, through the privacy loss. Write the probability of a transcript as a product of one factor per step, take logs, and bound each term.

1. **Factor the probability of a transcript** by the chain rule: the probability of $(y_1, \dots, y_k)$ is the probability of y_1, times that of y_2 given y_1, and so on. Under D,

$$p_D(y_1, \dots, y_k) = \prod_{i=1}^{k} p_{M_i(D;\, y_{<i})}(y_i), \tag{19.10}$$

   where $y_{<i} = (y_1, \dots, y_{i-1})$ and each factor uses fresh coins. The same holds under D', with the same previous outputs plugged in.
2. **Take the log-ratio** (Fact 3, adaptive version): the log of a ratio of products is a sum of log-ratios, $L_{D,D'}(y_1, \dots, y_k) = \sum_{i=1}^{k} L^{(i)}_{D,D'}(y_i;\, y_{<i})$, where $L^{(i)}$ is the privacy loss of M_i with its previous outputs fixed.
3. **Bound each term.** For the fixed $y_{<i}$, M_i is ε_i-DP by assumption, so $|L^{(i)}| \le \varepsilon_i$.
4. **Add:** $|L_{D,D'}| \le \sum_i \varepsilon_i$ at every transcript, which is pure $\sum_i \varepsilon_i$-DP (Fact 1, form 3).

Concretely, an adaptive survey. Ask a person for a randomized-response answer with $\varepsilon_1 = \ln 3$. If the answer is 1, ask again with $\ln 3$; if it is 0, ask again with 0.5, which means telling the truth with probability $e^{0.5}/(1+e^{0.5}) = 0.6225$ (randomized response at level ε tells the truth with probability $e^{\varepsilon}/(1+e^{\varepsilon})$, so that truth and lie have ratio e^{ε}). The second mechanism depends on the first output, but for each value of y_1 it is at most $\ln 3$-DP. Table 19.6 lists the four transcripts, from the script. Each probability is the product of the two steps; for example $(0,1)$ under bit 1 is a lie at the first step and the truth at the second, $\frac{1}{4} \times 0.6225 = 0.1556$.

Table 19.6.: An adaptive composition of two randomized-response answers: the second one uses $\varepsilon = \ln 3$ after a 1 and $\varepsilon = 0.5$ after a 0. Each transcript's privacy loss is the sum of the two step losses.

(y_1, y_2)	Under bit 1 (D)	Under bit 0 (D')	$L = L^{(1)} + L^{(2)}$
$(1,1)$	0.5625	0.0625	$\ln 3 + \ln 3 = 2.1972$
$(1,0)$	0.1875	0.1875	$\ln 3 - \ln 3 = 0$
$(0,1)$	0.1556	0.2832	$-\ln 3 + 0.5 = -0.5986$
$(0,0)$	0.0944	0.4668	$-\ln 3 - 0.5 = -1.5986$

The worst loss is $2.1972 = \ln 3 + \ln 3$, exactly the bound with $\varepsilon_2 = \max(\ln 3, 0.5)$, the largest budget the second step can have.

Approximate DP, through a two-step lemma. With $\delta > 0$ the losses are no longer bounded, so work with events and Fact 2. The plan is to split an event over the first output, use the second mechanism's guarantee for each value of that output, and then use the first mechanism's guarantee on what is left, which is a soft test of the first output. Take $k = 2$; induction on k does the rest, since the first $k-1$ steps form one mechanism.

1. **Split an event over the first output.** For a set S of transcripts, let $S_{y_1} = \{y_2 : (y_1, y_2) \in S\}$, the second outputs that complete y_1 into S. The transcript lands in S when the first output is some y_1 and the second lands in S_{y_1}, so
$$\Pr_D[(Y_1, Y_2) \in S] = \sum_{y_1} p_1(y_1)\, P_2^{y_1}(S_{y_1}), \tag{19.11}$$
where p_1 is the distribution of $M_1(D)$ and $P_2^{y_1}$ that of $M_2(D; y_1)$. Write q_1 and $Q_2^{y_1}$ for D'.
2. **Apply the second mechanism's guarantee inside the sum.** For each y_1, $P_2^{y_1}(S_{y_1}) \le e^{\varepsilon_2} Q_2^{y_1}(S_{y_1}) + \delta_2$, and also $P_2^{y_1}(S_{y_1}) \le 1$ because it is a probability. So $P_2^{y_1}(S_{y_1}) \le g(y_1) + \delta_2$ with
$$g(y_1) = \min\left(1,\ e^{\varepsilon_2} Q_2^{y_1}(S_{y_1})\right) \in [0,1]. \tag{19.12}$$
3. **Sum over y_1:** the weights $p_1(y_1)$ add up to 1, so δ_2 comes out once, $\Pr_D[S] \le \sum_{y_1} p_1(y_1) g(y_1) + \delta_2 = \mathbb{E}_{p_1}[g] + \delta_2$.
4. **Apply the first mechanism's guarantee to g** (Fact 2, which needs values in $[0,1]$; that is why g was capped at 1): $\mathbb{E}_{p_1}[g] \le e^{\varepsilon_1} \mathbb{E}_{q_1}[g] + \delta_1$.
5. **Undo the cap:** $g(y_1) \le e^{\varepsilon_2} Q_2^{y_1}(S_{y_1})$, so $\mathbb{E}_{q_1}[g] \le e^{\varepsilon_2} \sum_{y_1} q_1(y_1) Q_2^{y_1}(S_{y_1}) = e^{\varepsilon_2} \Pr_{D'}[S]$, by step 1 under D'.
6. **Combine:** $\Pr_D[S] \le e^{\varepsilon_1 + \varepsilon_2} \Pr_{D'}[S] + \delta_1 + \delta_2$. Nothing used that M_2 ignores y_1, so this covers adaptive composition.

19.5.4. Interpretation

Basic composition is exact for the worst case. For k randomized-response answers about the same bit, the transcript "all answers equal the true bit" has loss exactly $k \ln 3$, since each answer contributes $+\ln 3$. No smaller pure ε holds.

But the worst case is rare. That transcript has probability $(\frac{3}{4})^k$ under D. Typical transcripts mix truthful answers with lies, and their losses cancel in part. Each answer contributes $+\varepsilon_0$ with probability $t = e^{\varepsilon_0}/(1+e^{\varepsilon_0})$ (it told the truth) and $-\varepsilon_0$ otherwise, so its mean is $\varepsilon_0(2t-1) = \varepsilon_0 \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1} = \varepsilon_0 \tanh(\varepsilon_0/2)$. For $\varepsilon_0 = \ln 3$ this is $\varepsilon_0 \cdot \frac{3-1}{3+1} = \frac{\varepsilon_0}{2}$. The privacy loss of k answers is a sum of k independent terms of $\pm\varepsilon_0$, so it concentrates around its mean $k\varepsilon_0 \tanh(\varepsilon_0/2)$ (half of $k \ln 3$ for $\varepsilon_0 = \ln 3$) with spread of order $\varepsilon_0\sqrt{k}$. That is the behaviour of any sum of bounded independent terms (Concentration): its standard deviation grows like $\sqrt{k}$, and deviations much larger than that are exponentially unlikely. Allowing a small δ lets you ignore that unlikely tail, and the ε you need grows like $\sqrt{k}$ rather than k. That is advanced composition, derived in Advanced and optimal composition; Exercise 6 in the exercise book previews it.

The δs add without inflation. Unlike group privacy, composition adds δs plainly: there is no chain of multiplications, because each mechanism compares the *same* pair D, D'. In group privacy each step moved to a new dataset, and the bound for the next step had to be multiplied through; here step 3 of the lemma just averages the second slack over the first output.

Adaptivity is free here, but only with fixed budgets. The theorem allows M_i to depend on previous outputs, but its budget $(\varepsilon_i, \delta_i)$ must be fixed in advance. Choosing the budgets themselves from the outputs ("stop when the answer is good enough") needs privacy filters, also in Advanced and optimal composition.

19.5.5. Example: asking the same question k times

A survey asks the same yes/no question k times, each time with fresh randomized response at $\varepsilon_0 = \ln 3$. Only the number j of answers equal to 1 matters for the privacy loss, which is $(2j-k)\ln 3$: j answers with loss $+\ln 3$ and $k-j$ with $-\ln 3$. When the true bit is 1, j follows a binomial distribution with k trials and success probability $\frac{3}{4}$; when it is 0, success probability $\frac{1}{4}$. Figure 19.3 shows the distribution of the privacy loss for $k = 10$, and Table 19.7 gives the exact numbers from the script. "Exact ε at $\delta = 10^{-6}$" is the smallest ε with $\delta(\varepsilon) \le 10^{-6}$ for the k answers, computed from the binomial distribution of the number of truthful answers.

Table 19.7.: k randomized-response answers about the same bit, $\varepsilon_0 = \ln 3$ each: the basic bound, the exact worst case, how likely the worst case is, the mean privacy loss, and the exact ε when $\delta = 10^{-6}$ is allowed.

k	Basic bound $k\varepsilon_0$	Exact worst-case loss	$\Pr_D[L = k\varepsilon_0] = (\frac{3}{4})^k$	Mean loss under D	Exact ε at $\delta = 10^{-6}$
1	1.099	1.099	0.75	0.549	1.099
10	10.986	10.986	5.631×10^{-2}	5.493	10.986
20	21.972	21.972	3.171×10^{-3}	10.986	21.972
50	54.931	54.931	5.663×10^{-7}	27.465	52.680

k	Basic bound $k\varepsilon_0$	Exact worst-case loss	$\Pr_D[L = k\varepsilon_0] = (\frac{3}{4})^k$	Mean loss under D	Exact ε at $\delta = 10^{-6}$
100	109.861	109.861	3.207×10^{-13}	54.931	94.290

A slack δ only helps if some transcripts with large loss have total probability below δ, so that they can be written off. Up to $k = 20$ the worst case has probability far above 10^{-6}, so allowing $\delta = 10^{-6}$ buys almost nothing: the exact ε equals the basic bound to three decimals. At $k = 50$ the worst case has probability 5.663×10^{-7}, below 10^{-6}, and the exact ε starts to drop. At $k = 100$ it buys $109.861 - 94.290 = 15.6$ nats. With a smaller per-answer budget the gain is much larger, because the $\sqrt{k}$ spread is then far below the $k\varepsilon_0$ worst case: at $\varepsilon_0 = 0.1$ the basic bound for $k = 1000$ answers is 100, while the exact ε at $\delta = 10^{-6}$ is 19.345 (Figure 19.4). A simulation of 200,000 runs of 10 answers gives a mean loss of 5.500 (exact 5.493) and the worst case in a fraction 0.0559 of runs (exact 0.0563).

Composing approximate-DP mechanisms. The same gap appears for the δ part. Take counts with Gaussian noise, each calibrated to $(0.5, 10^{-5})$ ($\sigma = 7.0318$). Changing one record shifts each of the k noisy counts by 1, so the two output distributions are k-dimensional normals with independent coordinates of standard deviation σ, whose centres differ by the vector $(1, \dots, 1)$ of length $\sqrt{k}$. A normal with independent equal-variance coordinates looks the same in every direction, so you can rotate the axes until that shift points along the first one. The other $k - 1$ coordinates then have the same distribution under both datasets and carry no evidence. So the exact profile of k such counts is that of one Gaussian pair with $\sqrt{k}$ times the shift, since independent Gaussian coordinates combine into one (Divergences). Table 19.8 compares it with Equation 19.9 at the same total δ.

Table 19.8.: k counts with Gaussian noise, each $(0.5, 10^{-5})$-DP: basic composition against the exact ε at the same total δ.

k	Basic: $(k\varepsilon, k\delta)$	Exact ε at the same $\delta = k \times 10^{-5}$
1	$(0.5, 10^{-5})$	0.500
2	$(1, 2 \times 10^{-5})$	0.694
4	$(2, 4 \times 10^{-5})$	0.966
10	$(5, 10^{-4})$	1.504

Basic composition is a valid bound, and the only one you need until the number of mechanisms grows. For ten Gaussian queries it already overstates ε by a factor of $5/1.504 = 3.3$.

19.5.6. Check yourself

- Two mechanisms are each $(1, 10^{-6})$-DP. What does basic composition give for running both, and what for running the first one twice with the same output reported twice? *($(2, 2 \times 10^{-6})$ for the two runs. Reporting one output twice is post-processing, so $(1, 10^{-6})$.)*

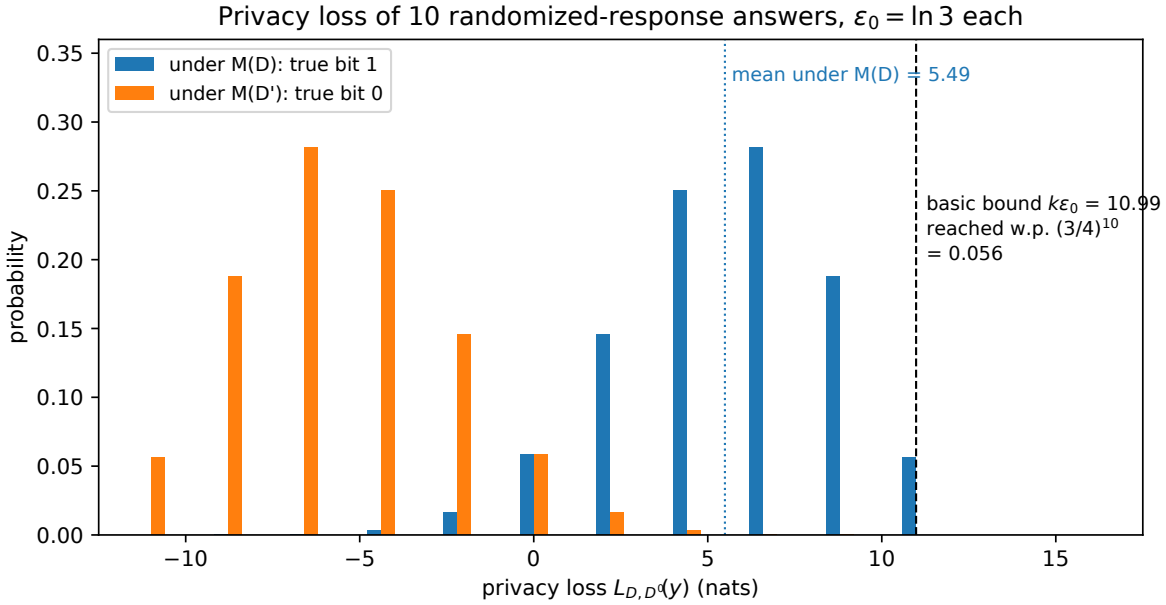


Figure 19.3.: Distribution of the privacy loss $L_{D,D'}$ of 10 randomized-response answers with $\varepsilon_0 = \ln 3$ each, when the true bit is 1 (blue) or 0 (orange). The loss takes the values $(2j - 10)\ln 3$, where j is the number of answers equal to 1. The basic bound $10\ln 3$ is reached, but only with probability $(3/4)^{10} = 0.056$; the mean under D is half of it.

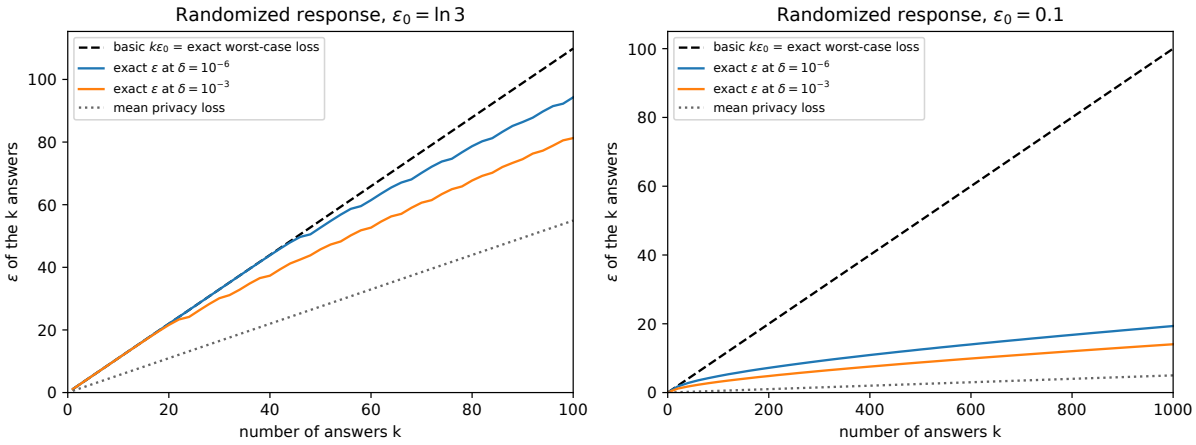


Figure 19.4.: The ε of k randomized-response answers: the basic bound $k\varepsilon_0$, which is also the exact worst-case loss (dashed), the exact ε at $\delta = 10^{-6}$ and 10^{-3}, and the mean privacy loss (dotted). Left: $\varepsilon_0 = \ln 3$; the exact curves leave the basic line only once $(3/4)^k$ drops below δ. Right: $\varepsilon_0 = 0.1$; the exact curves grow like $\sqrt{k}$.

- In Table 19.7, why is the exact ε at $\delta = 10^{-6}$ equal to the basic bound for $k \leq 20$? *(The worst transcript has probability* $(3/4)^k \geq 3.171 \times 10^{-3}$ *under* D *and* $e^{k\varepsilon_0}$ *times less under* D'*. At* $\varepsilon = k\varepsilon_0 - \eta$*, slightly below the bound, its contribution to the hockey-stick divergence is its probability under* D *minus* e^{ε} *times its probability under* D'*, which is* $(3/4)^k(1 - e^{-\eta}) \approx (3/4)^k\eta$*. That alone must stay below* δ*, so* $\delta = 10^{-6}$ *allows only* $\eta \leq 10^{-6}/(3.171 \times 10^{-3}) \approx 3 \times 10^{-4}$*, invisible at three decimals.)*

19.6. Parallel composition

19.6.1. Context

A histogram counts people in disjoint bins. Per-region statistics use disjoint subsets of the data. In local randomized response, each person randomizes only their own bit. Sequential composition would charge the sum of the budgets, $d\varepsilon$ for d bins, which is far too pessimistic: one person's record lands in one bin only, so only one output depends on it. Parallel composition makes that precise. It needs a careful look at the neighbouring relation, because there are two common ones. Under **add/remove**, neighbours differ by adding or removing one record. Under **replace**, neighbours have the same size and differ in the content of one record, which is a removal followed by an addition.

19.6.2. Statement

The idea is that a changed record touches only the part of the data it falls in, so only that part's mechanism sees a difference, and you pay only that part's budget. Split the data universe into disjoint parts $U_1, \ldots, U_d$ by a rule fixed in advance (it may depend on a record's content, but not on the rest of the dataset). Let M_i run on $D \cap U_i$, the records of D that fall in part i, with independent coins, and suppose each M_i is $(\varepsilon_i, \delta_i)$-DP under the **add/remove** relation. Then the release $(M_1(D \cap U_1), \ldots, M_d(D \cap U_d))$ is

$$\big(\max_i \varepsilon_i,\ \max_i \delta_i\big)\text{-DP under add/remove.} \tag{19.13}$$

Under replace, a record that moves from part a to part $b \neq a$ costs $(\varepsilon_a + \varepsilon_b, \delta_a + \delta_b)$, and one replaced within part a costs that part's guarantee for two add/remove steps, at most group privacy with $k = 2$ (Equation 19.6). For pure DP with equal budgets, both cases give 2ε. If the parts are fixed slots (record i always goes to part i, as in local randomized response, where each person is one slot), a replacement changes one part only and the cost is $(\max_i \varepsilon_i, \max_i \delta_i)$ again.

19.6.3. Derivation

1. **Add/remove changes one part.** If $D' = D \cup \{x\}$, then x lies in exactly one part U_j, because the parts are disjoint and cover the universe. For $i \neq j$, $D \cap U_i = D' \cap U_i$, so M_i gets the same input and has the same output distribution on both.
2. **Pure DP: the other losses are zero.** By Fact 3 the privacy loss of the whole release is $\sum_i L^{(i)}$, and $L^{(i)} \equiv 0$ for $i \neq j$ because the two distributions are equal and their ratio is 1. So $|L| = |L^{(j)}| \leq \varepsilon_j \leq \max_i \varepsilon_i$.

3. **Approximate DP: condition on the unchanged parts.** Let W be the outputs of the parts $i \neq j$. They have the same distribution under D and D' and are independent of Y_j. For an event S, fix W first: given W, the release lands in S when Y_j falls in S_W, the set of y_j completing W into S. So $\Pr_D[S] = \mathbb{E}_W[\Pr_D[Y_j \in S_W]]$. Apply M_j's guarantee inside the expectation, $\Pr_D[Y_j \in S_W] \leq e^{\varepsilon_j} \Pr_{D'}[Y_j \in S_W] + \delta_j$, and average over W, which has the same distribution under D': $\Pr_D[S] \leq e^{\varepsilon_j} \Pr_{D'}[S] + \delta_j$.
4. **Replace changes up to two parts.** If D' replaces a record $x \in U_a$ by $x' \in U_b$, then $D \cap U_a$ loses a record and $D \cap U_b$ gains one: two add/remove steps on two independent mechanisms. This is a composition of two mechanisms (Equation 19.9, with the other parts handled as in step 3), giving $(\varepsilon_a + \varepsilon_b, \delta_a + \delta_b)$. If $a = b$, part a sees a removal and an addition, two neighbouring steps, which is group privacy with $k = 2$ inside one part.

19.6.4. Interpretation

A histogram costs one ε, not d. Laplace noise of scale $1/\varepsilon$ on each of d bins is ε-DP under add/remove, however many bins there are, because adding a record changes one bin by 1 and the Laplace noise on that bin covers it. Under replace it is 2ε-DP, since moving a record changes two bins, which is why a histogram's sensitivity is often quoted as 2 under replace (The Laplace and Gaussian mechanisms).

Local randomized response composes in parallel too. Each person's report depends only on their own bit, so the whole vector of n reports is $\ln 3$-DP for any n, not $n \ln 3$. Each person is one slot, so replacing one person's bit changes only their own report.

The partition must not depend on the data. "Split the data at its median" changes when one record changes, which can move many records between parts, and then step 1 of the derivation fails: more than one part sees a difference. Use fixed bins, or account for the data-dependent split as a mechanism of its own.

19.6.5. Example: a two-bin histogram

Take two bins, each with rounded $\mathrm{Lap}(0, 1)$ noise, and let the script compute the exact pure ε of the pair of outputs under each relation. Adding or removing a record changes one bin by 1 and leaves the other alone, so only one bin contributes privacy loss, and $\varepsilon = 1.0000$, the budget of one bin. Replacing a record in bin 1 by one in bin 2 moves bin 1 up by 1 and bin 2 down by 1. Each bin then contributes a loss of at most 1, and the worst output has both noisy counts far out on the side that favours D, where both step losses equal 1 at once, so $\varepsilon = 2.0000$.

19.6.6. Check yourself

- A dataset is split into 50 states, and each state's mean income is released with 0.5-DP under add/remove. What is the guarantee for the whole release under add/remove, and under replace? *(*0.5 *under add/remove;* 1 *under replace, since a person who moves from one state to another changes two states' inputs.)*
- Why is it wrong to apply parallel composition to "the top 10% of earners" and "the rest"? *(The split depends on the data: adding one high earner can move a record across the boundary, changing both parts.)*

19.7. What composes and what doesn't

19.7.1. Context

The three properties hold under weak assumptions, which makes it tempting to apply them where even those assumptions fail. Each assumption appeared in a derivation: post-processing needs f not to read the data, composition needs independent coins and the same neighbouring relation for every step, and group privacy and composition both need the budgets fixed in advance. This section collects the ways those assumptions break in practice.

19.7.2. Statement

Table 19.9 lists what composes and what does not, with the assumption each row keeps or breaks.

Table 19.9.: What composes and what doesn't.

Situation	Composes?	Why
Function of the output, own coins	Yes, at no cost	Post-processing (Section 19.3)
Second query on the same data, fresh noise	Yes, budgets add	Basic composition (Equation 19.9)
Second query chosen from the first answer	Yes, budgets add	Adaptive composition, budgets fixed in advance
Two queries sharing one noise draw	**No**	The noise cancels in a difference; Fact 3 needs independent coins
"Post-processing" that reads the data again	**No**	It is a new mechanism, often a non-private one
Budgets chosen from earlier outputs	Not by Equation 19.9	Needs privacy filters (Advanced and optimal composition)
Mechanisms proved under add/remove, released under replace	Yes: ε doubles to 2ε and δ grows to $(1 + e^{\varepsilon})\delta$	A replacement is a removal plus an addition, so group privacy with $k = 2$ (Equation 19.6) applies, as in Equation 18.5 of The definition of differential privacy
A DP release plus an exact statistic of the same data	**No**	The exact statistic is not DP; the pair has no guarantee
Many records of one person	Yes, at $k\varepsilon$ and δ_k	Group privacy (Equation 19.6)

19.7.3. Derivation: why shared noise breaks everything

Composition assumed that each mechanism draws fresh noise. Here is what happens when two releases share one draw: the noise can be subtracted away. Release two counts with the *same* Laplace draw Z: $c_1(D) + Z$, the number of smokers, and $c_2(D) + Z$, the number of smokers aged 40 or over. Alice is a smoker aged 30.

1. **The difference is noise-free:** $(c_1 + Z) - (c_2 + Z) = c_1(D) - c_2(D)$, the exact number of smokers under 40. It is a deterministic function of the data.
2. **It changes between neighbours.** Removing Alice lowers c_1 by 1 and leaves c_2, since she is under 40. So if the difference is c under D, it is $c - 1$ under D'.
3. **So some event has probability 1 under D and 0 under D':** "the difference equals c". The event form $1 \leq e^{\varepsilon} \cdot 0 + \delta$ fails for every ε unless $\delta \geq 1$.

Each release alone is 1-DP. The pair is not DP at all. What went wrong is step 1 of the composition derivation: the transcript's probability is no longer a product of the two mechanisms' probabilities.

19.7.4. Interpretation

Fresh randomness is part of the mechanism. Seeding two "independent" mechanisms with the same random seed, caching noisy answers and re-adding the same noise to a related query, or reusing noise across the shards of a distributed system are all instances of the shared-noise bug.

Exact releases are not "public information" you can ignore. If the exact total of a column is published elsewhere, a DP release of a related quantity may be differenced against it, just as the two counts were differenced above. The DP guarantee covers your mechanism, and adds nothing to a release that is not DP.

Post-processing must be data-blind. A pipeline that takes the noisy top category, then reports its exact count, has added a non-private mechanism.

The neighbouring relation is part of the budget. A guarantee proved under add/remove and reported under replace can be off by a factor of 2 (The definition of differential privacy), because a replacement is two add/remove steps. Mixing the two inside one accountant is a bug, one that comes back with subsampling in Subsampling.

19.7.5. Example: the differencing attack in simulation

The script simulates the attacker of the derivation: Alice is in the data with probability $\frac{1}{2}$, and the attacker guesses "in" when the difference of the two released counts exceeds the midpoint. Over 200,000 runs with $\mathrm{Lap}(0, 1)$ noise, the attack with shared noise has accuracy 1.0000: the difference is exact, so the attacker is always right.

With fresh noise the same attack has accuracy 0.6206. Here $c_2 + Z_2$ does not depend on Alice at all, so the pair is just a 1-DP count plus independent noise. How well could any attack do? With Alice in or out with probability $\frac{1}{2}$ each, the best accuracy is $\frac{1}{2}$ plus half the **total variation distance** between what the attacker sees in the two cases, the largest difference in probability that the two distributions give to any one event (Divergences). For $\mathrm{Lap}(0, 1)$ and $\mathrm{Lap}(1, 1)$ the

best event is "the output is above $\frac{1}{2}$", which has probability $\frac{1}{2}e^{-1/2}$ under $\mathrm{Lap}(0,1)$ and $1-\frac{1}{2}e^{-1/2}$ under $\mathrm{Lap}(1,1)$, a difference of $1-e^{-1/2}$. So the best accuracy is $\frac{1}{2}+\frac{1}{2}(1-e^{-1/2})=0.6967$. The difference attack adds the noise of the second count, which only hurts it.

19.7.6. Check yourself

- A developer adds Laplace noise to a query answer and caches it, so that repeated identical queries return the same noisy answer. Is that safe? *(Yes. Returning the same output again is post-processing of one release. The bug is reusing the noise draw for a different query.)*
- An analysis runs 10 queries at $\varepsilon = 0.1$ each, but the eleventh is run only if the first ten look interesting. Does basic composition bound the total by 1.1? *(Yes, if the eleventh query's budget of* 0.1 *was fixed in advance, because deciding whether to run it is adaptive composition with a mechanism that may output nothing. If its budget is chosen from the answers, you need a privacy filter.)*

19.8. Code: check every property by exact computation and simulation

The script code/properties.py computes every number in this chapter and draws every figure. Each mechanism is a small typed function that returns its exact output distributions on two neighbouring datasets; the privacy profile $\delta(\varepsilon)$ is the hockey-stick divergence of those distributions, and pure ε is their largest log-ratio. Post-processing applies a channel; composition of randomized-response answers uses the binomial distribution of the number of truthful answers; the Gaussian numbers use the closed form from Part 1. From the repository root:

```
uv run python code/part03-dp-foundations/properties.py
```

Output (seed 0), summarized in Table 19.10:

Table 19.10.: The script's output with the default seed.

Experiment	Result
Randomized response $\ln 3$, then flip with probability f	$f=0.1$: $\varepsilon=0.8473$; $f=0.25$: $0.5108=\ln\frac{5}{3}$; $f=0.5$: 0
Count + $\mathrm{Lap}(0,1)$, post-processed	Raw: $\varepsilon=1$, $\delta(0.5)=0.2212$; rounded: 1.0000, 0.1967; threshold: 0.8318, 0.1967
Group privacy, pure	Rounded Laplace count, counts k apart: $\varepsilon=k$ for $k=1,2,3,5$
Group privacy, $(0.5, 10^{-5})$ per record	Table 19.5; the alarm mechanism attains the bound at every k
k randomized-response answers	Table 19.7; $\varepsilon_0=0.1$, $k=1000$: basic 100, exact 19.345 at $\delta=10^{-6}$
Simulation, 10 answers, 200,000 runs	Mean loss 5.500 (exact 5.493); worst case in 0.0559 of runs (exact 0.0563)
Adaptive two-step survey	Table 19.6; worst loss $2.1972=2\ln 3$

Experiment	Result
k Gaussian counts, $(0.5, 10^{-5})$ each	Table 19.8
Two-bin histogram, rounded Lap$(0, 1)$	Add/remove: $\varepsilon = 1.0000$; replace across bins: 2.0000
Shared against fresh noise, 200,000 runs	Attack accuracy 1.0000 against 0.6206; best possible with fresh noise 0.6967

tests/test_properties.py checks that the output still matches.

What to take from it. Post-processing never raises the profile, and sometimes lowers it: the threshold summary drops from $\varepsilon = 1$ to 0.8318, because it averages outputs with different losses. Group privacy is tight for counts in pure DP, and tight in general for approximate DP through the alarm mechanism, though specific mechanisms like the Gaussian do better. Basic composition is exact for the worst case, and the worst case is exponentially rare. That gap, $k\varepsilon_0$ against roughly $\sqrt{k}\,\varepsilon_0$, is what the accountants of the later chapters recover. Parallel composition costs one budget under add/remove and two under replace, since a replacement touches two parts. And shared noise is a total failure, not a small loss: the attack's accuracy goes from 0.6206 to 1.0000.

19.9. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014): Proposition 2.1 (post-processing), Theorem 2.2 (group privacy, pure), Theorem 3.16 (basic composition, with the proof for approximate DP in Appendix B), and §3.5 for composition in general. The standard reference for every statement here.
- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (TCC 2006): the paper that defined DP, where the guarantee for groups of records first appears.
- McSherry, "Privacy Integrated Queries" (SIGMOD 2009): parallel composition, and a system built on post-processing and composition as its only rules.
- Vadhan, "The Complexity of Differential Privacy" (2017), the opening sections: the basic properties of DP (post-processing, group privacy, composition) with careful proofs, at a more theoretical level than Dwork & Roth.
- Near & Abuah, *Programming Differential Privacy*, chapters on the properties of DP: sequential and parallel composition and post-processing, with code.
- Kifer & Machanavajjhala, "No Free Lunch in Data Privacy" (SIGMOD 2011): why correlated records defeat a per-record guarantee, the problem group privacy addresses.
- Dwork, Rothblum & Vadhan, "Boosting and Differential Privacy" (FOCS 2010), and Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): advanced and optimal composition, the subject of Advanced and optimal composition.
- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016): what changes when the budgets themselves are chosen adaptively.

19.10. Cheat sheet

M is (ε, δ)-DP: $\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$ for all S and all $D \simeq D'$; equivalently $E_{e^{\varepsilon}}(M(D) \| M(D')) \leq \delta$. The privacy loss is $L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}$. Table 19.11 collects the results.

Table 19.11.: Every result in this chapter.

Property	Statement	Proof idea	Tight?
Post-processing	$f \circ M$ is (ε, δ)-DP for any f that doesn't read the data	DPI: processing averages likelihood ratios; Fact 2 for $\delta > 0$	For invertible f; processing that merges outputs can lower ε
Group privacy, pure	$k\varepsilon$ for datasets k records apart	Losses telescope along a path	Yes, for counts with Laplace noise
Group privacy, approximate	$(k\varepsilon,\ \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$, at most $(k\varepsilon, ke^{(k-1)\varepsilon}\delta)$	Chain the event form k times	Yes, for the alarm mechanism
Basic composition	$(\sum \varepsilon_i, \sum \delta_i)$, also adaptive with fixed budgets	Losses of independent outputs add; two-step lemma for $\delta > 0$	Worst case yes; with $\delta > 0$ allowed, loose for large k
Parallel composition	$(\max \varepsilon_i, \max \delta_i)$ under add/remove; twice that under replace	Only one part sees the changed record	Yes

Rules of thumb

- Privatize once, as early as possible; everything after it is free.
- Budgets add across queries on the same data and take the maximum across disjoint data.
- Group privacy is cheap only for small groups and pure DP; for users with many records, define the neighbouring relation at the user level instead.
- Every composed mechanism needs its own fresh noise.
- Basic composition is exact for the worst case but the worst case is exponentially rare; when k is large, use the accountants of the later chapters.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

20. The Laplace and Gaussian mechanisms

20.1. Why this matters

You run a hospital database and want to publish how many patients have diabetes, the average age of a cohort, or a histogram of diagnoses, with a differential privacy (DP) guarantee. The definition of differential privacy says what the release must satisfy: changing one person's record must barely change the distribution of what you publish. It does not say how to build such a release. The two mechanisms of this chapter are the answer used almost everywhere. The idea is to compute the exact answer, call it $f(D)$ for the dataset D, and then blur it with random noise Z drawn fresh each time, so that what you publish is

$$M(D) = f(D) + Z. \tag{20.1}$$

If the noise is wide compared with how far one person can move the answer, an observer cannot tell from the output whether that person was there.

Three questions decide everything. The first is how much one record can move the answer. That is the **sensitivity** of f, and it sets the scale of the noise. The second is which noise distribution to use. Laplace noise gives pure ε-DP. Gaussian noise gives only (ε, δ)-DP, but it scales better when many numbers are released at once. The third is how much noise exactly. For Laplace noise the answer is a one-line formula. For Gaussian noise the textbook formula is loose, and the exact answer comes from solving one equation numerically.

Every later mechanism that adds noise, from the private mean to the noisy gradients of private model training in a later part, is calibrated with the tools of this chapter.

20.1.1. The one idea: privacy loss of additive noise is a shifted-copy log-ratio

Differential privacy is a bound on how much an output can favour one dataset over a neighbouring one. That is measured by the **privacy loss**: for an output y, the log of how much more likely y is when the data are D than when they are D',

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}, \tag{20.2}$$

where $p_{M(D)}$ is the density of the published output on dataset D. A loss of 0 means the output is equally likely under both datasets and says nothing about which one was used; a loss of ε means it is e^{ε} times more likely under D.

For additive noise this quantity has a simple shape. On neighbouring datasets $D \simeq D'$ (two datasets that differ in one person's record) the mechanism outputs $f(D) + Z$ and $f(D') + Z$: the *same* noise placed around two different centres. So the density of $M(D')$ is the density of $M(D)$

slid over by $f(D') - f(D)$, and the privacy loss is the log-ratio of the noise density and a copy of itself shifted by that amount. Probability distributions computed this log-ratio for the two noise densities of this chapter, and the two answers are what everything else rests on. For the Laplace density, whose log falls off like $-|z|$, the log-ratio is bounded by the shift divided by the scale, whatever y is. For the normal density, whose log falls off like $-z^2$, the log-ratio is a straight line in y: unbounded, but large only far in the tail, where outputs are rare. Table 20.1 puts the two side by side for a count, where the shift is 1, so the two centres are 0 and 1.

Table 20.1.: The privacy loss of a count released with Laplace and with Gaussian noise.

Noise	Privacy loss $L(y)$ for centres 0 and 1	Largest value	Guarantee
$\mathrm{Lap}(b)$	$(\lvert y-1\rvert - \lvert y\rvert)/b$	$1/b$, for every $y \le 0$	ε-DP with $\varepsilon = 1/b$
$\mathcal{N}(0, \sigma^2)$	$(1-2y)/(2\sigma^2)$	unbounded as $y \to -\infty$	(ε, δ)-DP, with δ set by the tail where $L > \varepsilon$

Both rows come straight from the densities. The Laplace density with scale b is $\frac{1}{2b}e^{-|z|/b}$, so the log-ratio at y of the copies centred at 0 and 1 is $(|y-1|-|y|)/b$; for $y \le 0$ this is $((1-y)-(-y))/b = 1/b$, and it never exceeds that. The normal density with variance σ^2 is proportional to $e^{-z^2/(2\sigma^2)}$, so the log-ratio is $((y-1)^2 - y^2)/(2\sigma^2) = (1-2y)/(2\sigma^2)$, which grows without limit as y moves left.

Table 20.2 lists the results of the chapter and where each is used.

Table 20.2.: The results of this chapter and where each is used.

Result	Where it is used
ℓ_1 and ℓ_2 sensitivity of counts, sums, means, histograms	Calibrating every noise-adding mechanism; clipping in Beyond global sensitivity
Laplace mechanism: $\mathrm{Lap}(\Delta_1 f/\varepsilon)$ gives ε-DP	Counts, histograms, low-dimensional statistics; report-noisy-max
Laplace accuracy: error $\le (\Delta_1 f/\varepsilon)\ln(d/\beta)$	Choosing ε for a target error
Classic Gaussian calibration $\sigma = \Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$	Reading papers and libraries that still use it
Exact $\delta(\varepsilon)$ of the Gaussian mechanism, analytic calibration	Every Gaussian release; the accountants of Part 4
Laplace against Gaussian in d dimensions	Choosing the noise for vector-valued queries and gradients

What you need. The definition of (ε, δ)-DP, its hockey-stick form and neighbouring relations are in The definition of differential privacy; post-processing and basic composition are in Post-processing, group privacy and composition. From Part 1: the Laplace and normal distributions and their shifted copies (Probability distributions), their tails (Concentration inequalities), and

the hockey-stick divergence with its closed form for two Gaussians (Divergences). Each of these is restated below where it is used.

Notation. M is a mechanism, $D \simeq D'$ are neighbouring datasets, and f is a query with values in $\mathbb{R}^d$, a vector of d numbers. Φ is the standard normal cumulative distribution function (CDF): $\Phi(t)$ is the probability that a standard normal variable is below t. Its density is $\varphi(t) = e^{-t^2/2}/\sqrt{2\pi}$. Because the normal curve is symmetric about 0, $\Phi(-t) = 1 - \Phi(t)$.

The guarantee itself says that no event can be much more likely under one dataset than under its neighbour, except for a small slack δ. Precisely, M is (ε, δ)**-DP** if for all $D \simeq D'$ and all events S,

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{20.3}$$

For one pair of datasets, write P for the distribution of $M(D)$ and Q for that of $M(D')$. Moving $e^{\varepsilon}Q(S)$ to the left, the inequality says that δ must be at least $P(S) - e^{\varepsilon}Q(S)$ for every event S. So the smallest δ that works for this pair is the largest value of that difference over all events, $\max_S[P(S) - e^{\varepsilon}Q(S)]$. That maximum has a name: it is the **hockey-stick divergence** $E_{e^{\varepsilon}}(M(D)\|M(D'))$ of Divergences, which in general is $E_{\gamma}(P\|Q) = \max_S[P(S) - \gamma Q(S)]$ for $\gamma \ge 1$. We write $\delta_M(\varepsilon)$ for the largest of these over all neighbouring pairs, the smallest δ that works for the mechanism as a whole at that ε. As a function of ε it is the **privacy curve** of M.

20.2. Toolkit: three facts every derivation uses

20.2.1. Fact 1 — Additive noise reduces to one shifted pair

When you add noise to an answer, the true answer itself should not matter to privacy, only how far apart the two answers on neighbouring datasets are. Publishing $1234 + Z$ or $1235 + Z$ hides the difference between the two datasets exactly as well as publishing Z or $1 + Z$, because subtracting 1234 from the output is something anyone can do, and it can be undone.

In symbols, let $M(D) = f(D) + Z$ where Z has density g on $\mathbb{R}^d$, and let $v = f(D') - f(D)$ be the shift between the two answers. Then

$$E_{\gamma}(M(D)\|M(D')) = E_{\gamma}(Z\|Z + v) \quad \text{for every } \gamma \ge 1. \tag{20.4}$$

The proof is that subtracting $f(D)$ just relabels the outputs. The map $y \mapsto y - f(D)$ sends $M(D)$ to Z and $M(D')$ to $Z + v$, and it is a bijection: every output corresponds to exactly one relabelled output, and back. So an event S about M's output is the same as the event $S - f(D)$ about the relabelled output, with the same probability under each dataset. The hockey-stick divergence is a maximum of $P(S) - \gamma Q(S)$ over events, and both sides range over the same values, so they are equal. (In the language of Divergences, the data-processing inequality applies in both directions, so no divergence can change.)

For the count above, with $f(D) = 1234$ and $f(D') = 1235$, the true value 1234 drops out and only the shift $v = 1$ remains. This is what makes privacy a property of the noise and of the set of shifts v that neighbouring datasets can produce, and nothing else. Section 20.3 bounds that set.

20.2.2. Fact 2 — A bounded privacy loss gives pure DP

If no single output can favour one dataset by more than a factor e^{ε}, then no event can either, because an event is a collection of outputs and its probability is a sum (or integral) over them. Formally, if $|L_{D,D'}(y)| \le \varepsilon$ for every output y and every neighbouring pair, then M is ε-DP, which is (ε, δ)-DP with $\delta = 0$.

The proof is one line. $L \le \varepsilon$ means $\ln(p_{M(D)}(y)/p_{M(D')}(y)) \le \varepsilon$, that is, $p_{M(D)}(y) \le e^{\varepsilon} p_{M(D')}(y)$ at every y. Integrating both sides over the outputs in S gives $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S]$. Only the upper bound $L \le \varepsilon$ is used here. The definition asks for the inequality in both directions, with D and D' swapped, but neighbouring is symmetric, so the bound for the pair (D', D) supplies the other direction. The Laplace mechanism is proved private exactly this way.

20.2.3. Fact 3 — The bad event, and the exact δ

Gaussian noise has an unbounded privacy loss, so Fact 2 does not apply. The way around is to look at the outputs where the loss is too large and count how much probability they carry. Call that set the **bad event**, $B = \{y : L(y) > \varepsilon\}$: the outputs that favour D over D' by more than a factor e^{ε}. With $P = M(D)$ and $Q = M(D')$,

$$E_{e^{\varepsilon}}(P\|Q) = P(B) - e^{\varepsilon} Q(B) \le P(B) = \Pr_{y\sim P}[L(y) > \varepsilon]. \tag{20.5}$$

The equality says that the worst event S in the definition of δ is the bad event itself. The reason is the argument of Fact 3 of Divergences: $P(S) - e^{\varepsilon} Q(S)$ is the integral over S of $p(y) - e^{\varepsilon} q(y)$. To make that integral as large as possible, include every output where the integrand is positive and no other. The integrand is positive exactly where $p > e^{\varepsilon} q$, that is where $L > \varepsilon$, which is B. The inequality just drops the term $e^{\varepsilon} Q(B)$, which is never negative. So the probability of the bad event under $M(D)$ is always a valid δ, though usually a loose one.

The same quantity can be written as an average of the privacy loss. Write $(x)_+ = \max(x, 0)$ for the positive part. The hockey-stick is the integral of $(p - e^{\varepsilon} q)_+$. Factor out q to get $q\,(p/q - e^{\varepsilon})_+ = q\,(e^{L} - e^{\varepsilon})_+$, or factor out p to get $p\,(1 - e^{\varepsilon} q/p)_+ = p\,(1 - e^{\varepsilon - L})_+$. Integrating, by change of measure,

$$E_{e^{\varepsilon}}(P\|Q) = \mathbb{E}_Q[(e^{L} - e^{\varepsilon})_+] = \mathbb{E}_P[(1 - e^{\varepsilon - L})_+]. \tag{20.6}$$

The last form needs only samples, or the distribution, of the privacy loss under P, and it is the starting point of Numerical accounting. In this chapter, the inequality of Equation 20.5 is the "bad event" argument behind the classic Gaussian calibration, and the equality is the exact calibration.

20.3. Sensitivity

20.3.1. Context

To hide one record, the noise has to cover the largest change one record can make to the answer. That change is the sensitivity. It depends on the query, on the norm you measure the change in,

and on the neighbouring relation (The definition of differential privacy). Under **add/remove** a neighbour has one record more or less, so the dataset size changes. Under **replace** one record is swapped for another, so the size n is fixed and public.

20.3.2. Definition

The sensitivity is the largest distance between the answers on any two neighbouring datasets. For a vector answer you need a way to measure distance, and two are used: the ℓ_1 norm $\|v\|_1 = \sum_i |v_i|$, the total change summed over coordinates, which is what Laplace noise is calibrated to, and the ℓ_2 norm $\|v\|_2 = \sqrt{\sum_i v_i^2}$, the straight-line length, which is what Gaussian noise is calibrated to. The ℓ_p **sensitivity** of f is

$$\Delta_p f = \max_{D \simeq D'} \big\| f(D) - f(D') \big\|_p, \qquad p \in \{1, 2\}. \tag{20.7}$$

The two norms are never far apart, and how far depends on how the change is spread. For every vector $v \in \mathbb{R}^d$, $\|v\|_2 \le \|v\|_1$ (squaring the sum $\sum |v_i|$ gives every v_i^2 plus non-negative cross terms) and $\|v\|_1 \le \sqrt{d}\,\|v\|_2$ (the Cauchy–Schwarz inequality applied to $\sum_i 1 \cdot |v_i|$). Taking the maximum over neighbours,

$$\Delta_2 f \le \Delta_1 f \le \sqrt{d}\,\Delta_2 f. \tag{20.8}$$

The left end is reached when one record moves a single coordinate, the right end when it moves all d coordinates by the same amount.

20.3.3. Derivation: the standard queries

Records take values in a domain $\mathcal{X}$. Table 20.3 gives the sensitivities, and the steps below derive each row by asking what the single worst record can do.

Table 20.3.: Sensitivities of the standard queries under the two neighbouring relations.

Query	Add/remove Δ_1	Add/remove Δ_2	Replace Δ_1	Replace Δ_2
Count $\lvert D\rvert$	1	1	0	0
Sum, records in $[0, B]$	B	B	B	B
Mean, records in $[0, B]$, n public	—	—	B/n	B/n
Histogram with k bins	1	1	2	$\sqrt{2}$
d threshold counts $\#\{x \ge j\}$, $j = 1..d$	d	$\sqrt{d}$	d	$\sqrt{d}$

1. **Count.** Adding or removing a record changes $|D|$ by exactly 1. Replacing a record leaves it unchanged, which is why under replace the size is not private and a count of all records needs no noise.
2. **Sum.** Adding a record $x \in [0, B]$ changes the sum by $x \le B$. Replacing x by x' changes it by $|x - x'| \le B$. Both bounds are reached with $x = B$, $x' = 0$.

3. **Mean with public n.** Under replace the mean is the sum divided by the fixed n, so its sensitivity is the sum's, B, divided by n: B/n. Under add/remove the denominator moves too, so there is no such simple bound; Exercise 1 in the exercise book releases a noisy sum and a noisy count instead.
4. **Histogram.** Under add/remove the record lands in one bin, so that bin changes by 1 and the others not at all. The change vector is $\pm e_j$, where e_j is the vector with a 1 in bin j and 0 elsewhere, and both of its norms are 1. Under replace one bin loses 1 and another gains 1: the change is $e_{j'} - e_j$, with ℓ_1 norm $1 + 1 = 2$ and ℓ_2 norm $\sqrt{1^2 + 1^2} = \sqrt{2}$.
5. **Threshold counts.** These are the empirical CDF read off at d thresholds: the count for threshold j is the number of records with $x \geq j$, so record x adds 1 to every count with $j \leq x$. A record with $x \geq d$ moves all d counts by 1, so the change vector is $(1, \ldots, 1)$, with ℓ_1 norm d and ℓ_2 norm $\sqrt{d \cdot 1^2} = \sqrt{d}$. Under replace, $x \to x'$ moves only the counts with j between the two values, at most d of them, by 1 each, so the same bounds hold.

A brute-force check. The script enumerates every dataset of one to three records with values in $\{0, 1, 2, 3\}$, every neighbour under both relations, and takes the maximum change. With $B = 3$, $k = 4$ bins and $d = 3$ thresholds it reproduces every row of Table 20.3: for example $\ell_1 = 2$ and $\ell_2 = 1.4142$ for the histogram under replace, and $\ell_1 = 3$, $\ell_2 = 1.7321$ for the threshold counts. For the mean of three records under replace it gives $3/3 = 1$.

20.3.4. Interpretation

Sensitivity is a worst case over datasets. It does not depend on the data you hold, only on what the domain allows. That is why it can be computed before looking at the data, and also why it can be large for unbounded data: the sum of arbitrary real numbers has infinite sensitivity, since one record can be as large as it likes. Clipping, in Beyond global sensitivity, forces a finite B.

The neighbouring relation changes the answer by up to a factor of 2. A histogram has $\Delta_1 = 1$ under add/remove and 2 under replace, because replacing is removing one record and adding another. Since the noise is proportional to the sensitivity, a guarantee stated without its neighbouring relation is ambiguous by that factor in ε.

Finally, dense queries separate the two norms. When one record moves many coordinates, Δ_1 grows like d but Δ_2 only like $\sqrt{d}$, because the ℓ_2 norm adds squares and then takes a square root. Section 20.7 shows that this is exactly when Gaussian noise beats Laplace noise.

20.3.5. Example: the average age of a cohort

A study releases the mean age of $n = 1000$ patients, with ages clipped to $[0, 100]$, under replace neighbours. By row 3 of Table 20.3 the sensitivity is $B/n = 100/1000 = 0.1$ year. One patient can move the mean by at most a tenth of a year, however old they are.

20.3.6. Check yourself

- Under add/remove, what is the ℓ_1 sensitivity of releasing both the count and the sum of records in $[0, 1]$? *(Adding a record changes the count by 1 and the sum by at most 1, so the ℓ_1 norm of the change is at most $1 + 1$: $\Delta_1 = 2$.)*

- A dataset has one row per user, and each user can rate up to 5 of k films. What is Δ_1 and Δ_2 of the vector of k rating counts under add/remove of a user? *(One user changes up to 5 counts by 1: $\Delta_1 = 5$, $\Delta_2 = \sqrt{5}$.)*

20.4. The Laplace mechanism

20.4.1. Context

The Laplace mechanism is the first mechanism to try. It adds independent Laplace noise to each coordinate, with scale proportional to the ℓ_1 sensitivity. It works because of the fact from the start of this chapter: the log-ratio of a Laplace density and a shifted copy is bounded by the shift divided by the scale (Probability distributions), and a bounded log-ratio is exactly what pure DP asks for (Dwork, McSherry, Nissim & Smith 2006).

20.4.2. Statement

The **Laplace distribution** $\mathrm{Lap}(b)$ with scale b has density $\frac{1}{2b}e^{-|z|/b}$: a two-sided exponential, peaked at 0. Choose the scale so that the largest shift one record can cause, $\Delta_1 f$, divided by the scale is exactly ε. For $f : \mathcal{D} \to \mathbb{R}^d$ with ℓ_1 sensitivity $\Delta_1 f$, the **Laplace mechanism**

$$M(D) = f(D) + (Z_1, \dots, Z_d), \qquad Z_i \overset{\text{i.i.d.}}{\sim} \mathrm{Lap}(b), \quad b = \frac{\Delta_1 f}{\varepsilon}, \tag{20.9}$$

is ε-DP. (i.i.d. means independent and identically distributed.) Its error per coordinate is about b, and a tail bound controls the largest error over all d coordinates: for every failure probability $\beta \in (0, 1)$,

$$\mathbb{E}|Z_i| = b, \qquad \mathrm{Var}\, Z_i = 2b^2, \qquad \Pr\Big[\max_i |Z_i| > b \ln \frac{d}{\beta}\Big] \le \beta. \tag{20.10}$$

No smaller scale works in general: for a count, $b < 1/\varepsilon$ violates ε-DP.

20.4.3. Derivation

Privacy. The plan is to write the privacy loss down, bound it at every output by the triangle inequality, and then use Fact 2.

1. **Write the two densities.** Fix $D \simeq D'$ and write $a = f(D)$ and $a' = f(D')$ for the two true answers. The coordinates of the noise are independent, so the density of $M(D)$ at y is the product $\prod_i \frac{1}{2b}e^{-|y_i - a_i|/b}$, and that of $M(D')$ is the same with a' in place of a.
2. **Take the log-ratio.** The constants $\frac{1}{2b}$ cancel, and the log of a product of exponentials is the sum of the exponents, so

$$L_{D,D'}(y) = \frac{1}{b}\sum_{i=1}^{d} \left(|y_i - a'_i| - |y_i - a_i|\right). \tag{20.11}$$

3. **Bound each term.** The distance from y_i to a'_i is at most the distance from y_i to a_i plus the distance from a_i to a'_i (the triangle inequality, $|y_i - a'_i| \le |y_i - a_i| + |a_i - a'_i|$). So each term in the sum is at most $|a_i - a'_i|$, whatever y is.

4. **Add the terms.** Summing over i gives the ℓ_1 norm of the shift, which is at most the sensitivity: $L_{D,D'}(y) \le \|a - a'\|_1 / b \le \Delta_1 f / b = \varepsilon$, for every y.
5. **Conclude.** The privacy loss is at most ε at every output, so Fact 2 gives ε-DP.

Concretely, for a count. Take $f(D) = 0$, $f(D') = 1$ and $b = 1$, so $\varepsilon = 1$. Table 20.4 evaluates the two densities and the privacy loss at a few outputs.

Table 20.4.: The Laplace mechanism for a count with $b = 1$: densities and privacy loss at five outputs.

Output y	-2	0	0.5	1	3
$p_{M(D)}(y) = \frac{1}{2}e^{-\lvert y\rvert}$	0.0677	0.5	0.3033	0.1839	0.0249
$p_{M(D')}(y) = \frac{1}{2}e^{-\lvert y-1\rvert}$	0.0249	0.1839	0.3033	0.5	0.0677
$L(y) = \lvert y-1\rvert - \lvert y\rvert$	1	1	0	-1	-1

Left of both true answers the ratio of the densities is stuck at e^1 (for example $0.0677/0.0249$ at $y = -2$), right of them at e^{-1}, and in between the privacy loss $1 - 2y$ falls linearly from 1 to -1, so the ratio e^{1-2y} passes through 1 at the midpoint $y = 0.5$. The ratio stops growing because, once y is past both centres, moving it further away multiplies both densities by the same factor. However far out an output lands, it never favours one dataset by more than a factor of e.

Accuracy. The error is the noise itself, so you need facts about the Laplace distribution (Concentration inequalities). Its absolute value $|Z_i|$ has density $\frac{1}{b}e^{-z/b}$ on $z \ge 0$, an exponential distribution with mean b and second moment $2b^2$; since Z_i has mean 0, that gives $\mathbb{E}|Z_i| = b$ and $\operatorname{Var} Z_i = 2b^2$. Integrating the same density from t to infinity gives the tail $\Pr[|Z_i| > t] = e^{-t/b}$. To bound all d coordinates at once, give each coordinate a share β/d of the failure probability: setting $e^{-t/b} = \beta/d$ gives $t = b\ln(d/\beta)$. The probability that at least one of the d coordinates fails is at most the sum of their failure probabilities (the union bound), $d \cdot \beta/d = \beta$.

Tightness. To show that no smaller scale works, find an event whose probability ratio is as large as the bound allows. The outputs where the loss is largest are those left of both answers, so take a count with $f(D) = 0$ and $f(D') = 1$ and the event $S = \{y \le 0\}$. Under $\mathrm{Lap}(b)$ noise, $\Pr[M(D) \in S] = \Pr[Z \le 0] = \frac{1}{2}$, by symmetry of the noise about 0, and $\Pr[M(D') \in S] = \Pr[Z \le -1] = \frac{1}{2}e^{-1/b}$, from the tail above. So the ratio is exactly $e^{1/b}$. On S the privacy loss is $(|y-1| - |y|)/b = 1/b$ at every point: the bound of step 4 holds with equality on half the mass of $M(D)$. If $b < 1/\varepsilon$, the ratio $e^{1/b}$ exceeds e^{ε} and the definition fails on this event (Exercise 2 in the exercise book).

20.4.4. Interpretation

The noise does not depend on n. A count gets noise of scale $1/\varepsilon$ whether the database has a hundred records or a hundred million, because one record moves a count by 1 either way. The *relative* error of a count of size n is therefore about $(1/\varepsilon)/n = 1/(n\varepsilon)$, so large populations are cheap to protect and small subgroups are expensive.

The worst case is not rare. For the Laplace mechanism the privacy loss equals ε on a set of probability $\frac{1}{2}$ (Figure 20.1, left), the event S of the tightness argument. The guarantee is tight in the strongest sense, not just at some unlikely output.

The mechanism also satisfies a whole curve of weaker guarantees. Asked for a smaller $\varepsilon' < \varepsilon$, it is not ε'-DP, but it is (ε', δ)-DP with

$$\delta = 1 - e^{-(\varepsilon - \varepsilon')/2}. \tag{20.12}$$

This is the hockey-stick divergence of a Laplace pair from Divergences, with shift in units of the scale $t = \Delta_1 f / b = \varepsilon$, and Fact 3 explains where it comes from. With $b = 1$ and centres 0 and t, the loss is t left of 0 and falls linearly as $t - 2y$ between the centres, so the bad event $\{L > \varepsilon'\}$ is $\{y < (t - \varepsilon')/2\}$. Its mass under $M(D)$ is $1 - \frac{1}{2}e^{-(t-\varepsilon')/2}$ and under $M(D')$ is $\frac{1}{2}e^{-(t+\varepsilon')/2}$; multiplying the second by $e^{\varepsilon'}$ gives $\frac{1}{2}e^{-(t-\varepsilon')/2}$, and the difference is $1 - e^{-(t-\varepsilon')/2}$. Figure 20.2 draws it as the dashed curve that hits zero at $\varepsilon' = \varepsilon$.

Why this shape of noise? What the proof used is that the log-density changes by at most $\varepsilon/\Delta_1 f$ per unit of shift. Any density with that property would do, and the Laplace log-density $-|z|/b$ has exactly that slope everywhere. Other shapes are slightly better for large ε (Geng & Viswanath's staircase mechanism), but the Laplace is within a small factor and is the standard choice.

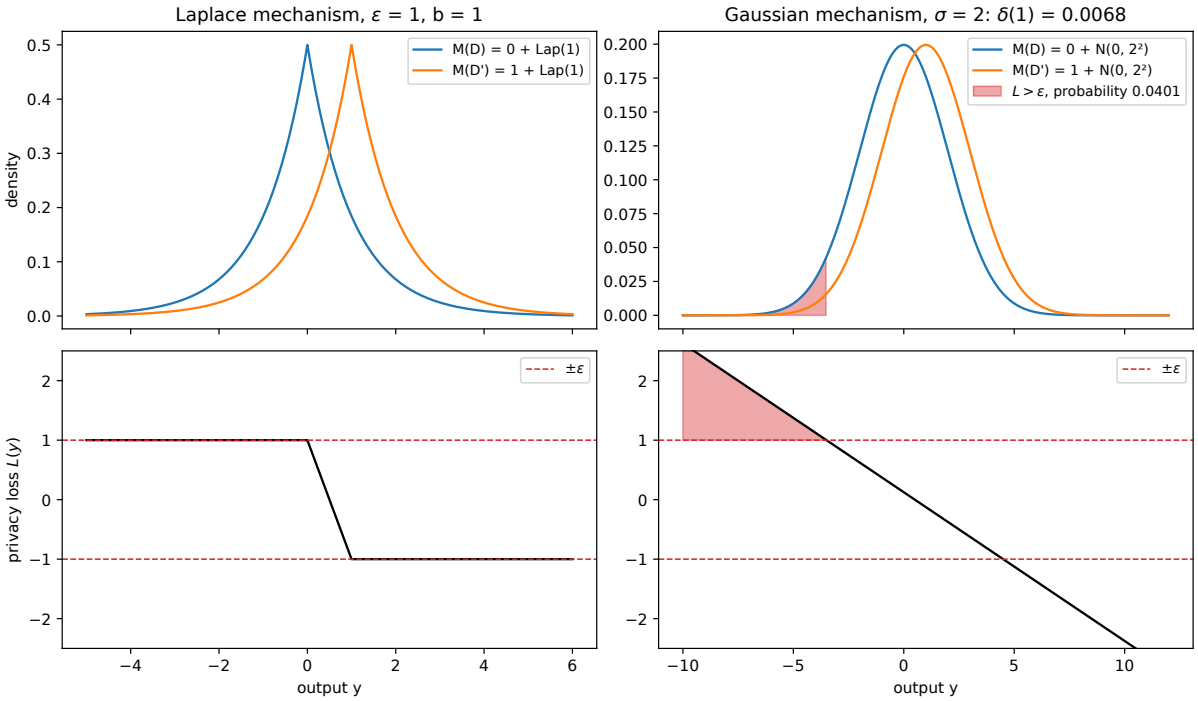


Figure 20.1.: Left column: a count released with Lap(1) noise on neighbouring datasets with true counts 0 and 1, and the privacy loss $L(y)$. It equals $+\varepsilon$ for every $y \le 0$ and $-\varepsilon$ for every $y \ge 1$, never more. Right column: the same count with $\mathcal{N}(0, 2^2)$ noise. The privacy loss is a straight line, and exceeds $\varepsilon = 1$ on the shaded tail, which has probability 0.0401 under $M(D)$. The exact $\delta(1) = 0.0068$ is smaller, because the hockey-stick subtracts e^{ε} times the mass of $M(D')$ on the same tail.

20.4.5. Example: counts and a histogram

A count. 1,234 of the hospital's $n = 10{,}000$ patients have diabetes. Released with $\varepsilon = 0.1$ under add/remove, the sensitivity is 1, so the noise is $\text{Lap}(1/0.1) = \text{Lap}(10)$. The expected absolute error is $b = 10$ (simulated over a million releases: 9.978). For a 95% guarantee take $d = 1$ and $\beta = 0.05$ in the accuracy bound: with probability 0.95 the error is below $b \ln(1/0.05) = b \ln 20 = 29.96$ (simulated 95th percentile: 29.85). That is a relative error of $29.96/1234 = 0.0243$.

The tight event, simulated. With $b = 1$ ($\varepsilon = 1$), a million draws give $\Pr[M(D) \le 0] = 0.5002$ and $\Pr[M(D') \le 0] = 0.1838$, against the exact $\frac{1}{2}$ and $\frac{1}{2}e^{-1}$. Their ratio is 2.7221 against $e = 2.7183$.

A histogram. A histogram of diagnoses with 20 bins, $\varepsilon = 1$ and add/remove has $\Delta_1 = 1$, so each bin gets Lap(1) noise. With $d = 20$ and $\beta = 0.05$, every bin is within $\ln(20/0.05) = 5.99$ of its true count with probability 0.95. In 20,000 simulated releases the maximum error exceeded 5.99 in a fraction 0.0464 of them, just under β. The union bound adds the failure probabilities as if failures never happened together; for independent coordinates they rarely do, so the bound is nearly exact.

The cohort mean. With sensitivity 0.1 from Section 20.3 and $\varepsilon = 1$, the scale is $b = 0.1/1 = 0.1$, and the mean age is accurate to $\pm 0.1 \ln 20 = \pm 0.2996$ years with probability 0.95.

20.4.6. Check yourself

- A histogram under *replace* neighbours, $\varepsilon = 1$. What Laplace scale do you need? *($\Delta_1 = 2$, so $b = 2/1 = 2$: twice the add/remove noise.)*
- You release 10 independent counts, each with $\text{Lap}(1/\varepsilon)$ noise, and every record can affect all 10. What is the guarantee of the release as a whole? *(The vector has $\Delta_1 = 10$, so by step 4 of the derivation the loss is at most $10/(1/\varepsilon) = 10\varepsilon$: the release is 10ε-DP. That is the same as basic composition of ten ε-DP releases in Post-processing, group privacy and composition, which adds up the εs.)*

20.5. The Gaussian mechanism: classic calibration

20.5.1. Context

Gaussian noise is the other natural choice. It has lighter tails than Laplace noise, a sum of independent Gaussians is again Gaussian, and it is calibrated to the ℓ_2 sensitivity, which can be much smaller than the ℓ_1 sensitivity. The price is that its privacy loss is a straight line in the output and so is unbounded (Figure 20.1, right): Fact 2 never applies, and Gaussian noise cannot give pure DP. Instead, you accept a small probability δ that the loss exceeds ε. This section derives the calibration found in the textbook of Dwork & Roth and in many papers and libraries, and shows where it is loose.

20.5.2. Statement

Write $\mathcal{N}(0, \sigma^2 I_d)$ for d independent normal coordinates, each with mean 0 and standard deviation σ. The noise must grow with the ℓ_2 sensitivity and shrink as ε grows, and it grows only slowly, like a square root of a logarithm, as δ shrinks. Let $\varepsilon \in (0, 1)$ and $\delta \in (0, 1)$. The **Gaussian mechanism** $M(D) = f(D) + Z$ with $Z \sim \mathcal{N}(0, \sigma^2 I_d)$ and

$$\sigma = \frac{\Delta_2 f \sqrt{2 \ln(1.25/\delta)}}{\varepsilon} \tag{20.13}$$

is (ε, δ)-DP (Dwork & Roth 2014, Theorem A.1).

20.5.3. Derivation

The plan is to work out the distribution of the privacy loss, which turns out to be normal, bound δ by the probability that it exceeds ε (the bad event of Fact 3), and then show with a standard normal tail bound that this probability is below δ for the σ above. The route is the same as Dwork & Roth's, in a one-sided form that is a little shorter. Write $c = \sqrt{2 \ln(1.25/\delta)}$, so $\sigma = c\, \Delta_2 f / \varepsilon$.

1. **Find the privacy loss.** By Fact 1, only the shift $v = f(D') - f(D)$ matters. Under $M(D)$, write the output as $y = f(D) + z$, where z is the noise. The density of $M(D)$ at y is proportional to $e^{-\|z\|^2/(2\sigma^2)}$, and that of $M(D')$ to $e^{-\|z-v\|^2/(2\sigma^2)}$, since y sits at distance $z - v$ from $f(D')$. The log of their ratio is the difference of exponents, and expanding $\|z - v\|^2 = \|z\|^2 - 2\langle z, v\rangle + \|v\|^2$ gives

$$L(y) = \frac{\|z - v\|^2 - \|z\|^2}{2\sigma^2} = \frac{\|v\|^2 - 2\langle z, v\rangle}{2\sigma^2}. \tag{20.14}$$

   The loss depends on z only through the inner product $\langle z, v\rangle = \sum_i z_i v_i$, the component of the noise along the shift. That is a sum of independent normals, so it is normal, with mean 0 and variance $\sum_i \sigma^2 v_i^2 = \sigma^2 \|v\|^2$. A constant minus a multiple of a normal is normal, so the privacy loss under $M(D)$ is normal, with mean $\|v\|^2/(2\sigma^2)$ and variance $4\sigma^2\|v\|^2/(4\sigma^4) = \|v\|^2/\sigma^2$. Writing $\mu = \|v\|/\sigma$ for the shift measured in standard deviations of the noise,

$$L \sim \mathcal{N}\Big(\frac{\mu^2}{2}, \mu^2\Big), \qquad \mu = \frac{\|v\|}{\sigma}. \tag{20.15}$$

   This is the one-dimensional result of Probability distributions with standardized shift μ; the other $d - 1$ directions of z, perpendicular to v, do not enter.
2. **Bound δ by the bad event** (Fact 3). The probability that the loss exceeds ε is a normal tail. Standardize it, subtracting the mean $\mu^2/2$ and dividing by the standard deviation μ: $L > \varepsilon$ exactly when a standard normal exceeds $(\varepsilon - \mu^2/2)/\mu = \frac{\varepsilon}{\mu} - \frac{\mu}{2}$. So $\delta \le \Pr[L > \varepsilon] = \Pr\left[\mathcal{N}(0,1) > \frac{\varepsilon}{\mu} - \frac{\mu}{2}\right] = \Phi(-t)$ with $t = \frac{\varepsilon}{\mu} - \frac{\mu}{2}$, using the symmetry of the normal curve to write the upper tail beyond t as $\Phi(-t)$.
3. **Take the worst shift.** A larger shift makes the bad event more likely: t decreases as μ grows, since both ε/μ and $-\mu/2$ fall. So the worst neighbouring pair has the largest shift, $\|v\| = \Delta_2 f$, that is $\mu = \Delta_2 f/\sigma = \varepsilon/c$. Substituting, $t = \varepsilon \cdot c/\varepsilon - \frac{\varepsilon}{2c} = c - \frac{\varepsilon}{2c}$.

4. **Bound the normal tail.** The Mills-ratio bound of Concentration inequalities says $\Phi(-t) \le \varphi(t)/t$ for $t > 0$. The reason is that beyond t every point z has $z/t \ge 1$, so $\int_t^\infty \varphi(z)\,dz \le \int_t^\infty \frac{z}{t}\varphi(z)\,dz$, and since the derivative of $\varphi(z)$ is $-z\varphi(z)$, the right side integrates to $\varphi(t)/t$. So $\Phi(-t) \le \frac{e^{-t^2/2}}{t\sqrt{2\pi}}$.
5. **Expand t^2.** Squaring $t = c - \varepsilon/(2c)$ gives $t^2 = c^2 - \varepsilon + \frac{\varepsilon^2}{4c^2} \ge c^2 - \varepsilon$, dropping the last term because it is positive. So $e^{-t^2/2} \le e^{-c^2/2}e^{\varepsilon/2}$. By the definition of c, $c^2/2 = \ln(1.25/\delta)$, so $e^{-c^2/2} = \delta/1.25$, and $e^{-t^2/2} \le \frac{\delta}{1.25}e^{\varepsilon/2}$.
6. **Combine.** Putting steps 4 and 5 together,

$$\Pr[L > \varepsilon] \le \delta \cdot \frac{e^{\varepsilon/2}}{1.25\sqrt{2\pi}\,t}. \tag{20.16}$$

   This is at most δ when the fraction is at most 1, that is when $t \ge e^{\varepsilon/2}/(1.25\sqrt{2\pi})$. For $\varepsilon \le 1$ the right side is at most $e^{1/2}/(1.25\sqrt{2\pi}) = 0.5262$.
7. **Check that t is large enough.** It remains to show $t \ge 0.5262$. For $\varepsilon \le 1$, $t = c - \varepsilon/(2c) \ge c - 1/(2c)$, and this lower bound grows with c. It equals 0.5262 at the positive root of the quadratic $c^2 - 0.5262c - \frac{1}{2} = 0$ (multiply $c - 1/(2c) = 0.5262$ by c). Since c grows as δ shrinks, t is large enough whenever $\delta \le 0.7448$, the value of δ that puts c at that root. Every δ of practical interest qualifies.

The constant 1.25 is there to absorb $e^{\varepsilon/2} \le e^{1/2}$ and the $1/(t\sqrt{2\pi})$ factor of step 6, and the condition $\varepsilon < 1$ is what keeps $e^{\varepsilon/2}$ under control.

20.5.4. Interpretation: three sources of slack

The proof made three approximations in a row: it replaced the exact δ by the bad-event probability, replaced that by the Mills-ratio bound, and then replaced the Mills bound by the target δ using the constants. To see how much each one costs, follow the chain at $\varepsilon = 1$ and $\delta = 10^{-5}$. There $c = \sqrt{2\ln(1.25 \times 10^5)}$, so $\sigma = 4.8448\,\Delta_2 f$ and $t = c - 1/(2c) = 4.7416$:

Table 20.5.: The classic proof at $\varepsilon = 1$, $\delta = 10^{-5}$, step by step.

Quantity	Value	What was dropped to get the next line
Exact $\delta(\varepsilon)$, the hockey-stick	4.11×10^{-8}	the term $e^{\varepsilon}Q(B)$ of Equation 20.5
$\Pr[L > \varepsilon] = \Phi(-t)$	1.06×10^{-6}	the Mills-ratio approximation
Mills bound $\varphi(t)/t$	1.10×10^{-6}	the constants $e^{\varepsilon/2}$, 1.25 and $t\sqrt{2\pi}$
Target δ	10^{-5}	

The classic σ delivers a δ about 240 times smaller than asked for (Table 20.5: $10^{-5}/(4.11 \times 10^{-8}) \approx 243$). Most of the slack is in two places. The bad-event bound throws away the $e^{\varepsilon}Q(B)$ correction, a factor of $1.06 \times 10^{-6}/(4.11 \times 10^{-8}) \approx 26$ here, and the constants of steps 5–6 lose another factor

of $10^{-5}/(1.10 \times 10^{-6}) \approx 9$. The Mills ratio itself costs almost nothing. Equivalently, you could use less noise for the same (ε, δ). Section 20.6 finds exactly how much less.

The classic theorem is stated for $\varepsilon < 1$, and its constants are designed for that range. The formula does not fail immediately above 1: the script finds that at $\delta = 10^{-5}$ the classic σ still gives a valid guarantee up to $\varepsilon = 8.42$, and violates it beyond (Figure 20.3, right). But nothing in the classic proof tells you that, so for $\varepsilon \geq 1$ you need the exact curve.

20.5.5. Check yourself

- Halving δ from 10^{-5} to 5×10^{-6}: by what factor does the classic σ grow? *(σ is proportional to $\sqrt{\ln(1.25/\delta)}$, so the factor is $\sqrt{\ln(2.5 \times 10^5)/\ln(1.25 \times 10^5)} = \sqrt{12.43/11.74} \approx 1.03$. The dependence on δ is only logarithmic.)*
- Why does step 3 take $\|v\| = \Delta_2 f$ rather than any smaller shift? *($t = \varepsilon/\mu - \mu/2$ is decreasing in μ, so the bad event $\{L > \varepsilon\}$, whose probability is $\Phi(-t)$, is most likely at the largest shift.)*

20.6. The exact privacy curve of the Gaussian mechanism

20.6.1. Context

Fact 3 says that the smallest valid δ is a hockey-stick divergence: the bad-event probability minus the correction $e^{\varepsilon}Q(B)$. For two Gaussians with the same variance, both terms are normal tail probabilities, so the exact δ has a closed form, computed in Divergences. Balle & Wang (2018) turned that observation into a calibration: instead of bounding δ, compute it exactly, and pick the smallest σ for which it is below the target.

20.6.2. Statement

Measure the sensitivity in units of the noise, $\mu = \Delta_2 f/\sigma$. The exact δ is the probability of the bad event under $M(D)$ minus e^{ε} times its probability under $M(D')$. Both are normal tails, at points that depend only on μ and ε. The Gaussian mechanism is (ε, δ)-DP **if and only if** $\delta \geq \delta_{\mu}(\varepsilon)$, where

$$\delta_{\mu}(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) = \Phi\Big(\frac{\Delta_2 f}{2\sigma} - \frac{\varepsilon\sigma}{\Delta_2 f}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\Delta_2 f}{2\sigma} - \frac{\varepsilon\sigma}{\Delta_2 f}\Big). \quad (20.17)$$

(Here "if and only if" assumes some neighbouring pair reaches $\|f(D) - f(D')\|_2 = \Delta_2 f$, as the definition of sensitivity as a maximum ensures.) Moreover, $\delta_{\mu}(\varepsilon)$ is decreasing in ε and increasing in μ. At $\varepsilon = 0$ it equals the total variation (TV) distance between the two output distributions, $\Phi(\frac{\mu}{2}) - \Phi(-\frac{\mu}{2}) = 2\Phi(\mu/2) - 1$; the TV distance is $\max_S[P(S) - Q(S)]$, the hockey-stick with $\gamma = e^0 = 1$. And it tends to 0 as $\varepsilon \to \infty$, since it is at most the first term, $\Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu})$, which goes to 0. Because δ_{μ} grows with μ, less noise always means a larger δ, and there is a single noise level that hits the target exactly. So the **analytic calibration**

$$\sigma^*(\varepsilon, \delta) = \frac{\Delta_2 f}{\mu^*}, \qquad \mu^* \text{ the unique solution of } \delta_{\mu}(\varepsilon) = \delta, \quad (20.18)$$

is the smallest σ that makes the Gaussian mechanism (ε, δ)-DP.

20.6.3. Derivation

1. **Reduce to a shifted pair** (Fact 1). For a neighbouring pair with shift v, the smallest δ at this ε is the hockey-stick of the noise against its shifted copy, $E_{e^\varepsilon}(\mathcal{N}(0,\sigma^2 I)\|\mathcal{N}(v,\sigma^2 I))$.
2. **Reduce to one dimension.** Rotate coordinates so that v points along the first axis, $v = \|v\|e_1$. A rotation is a bijection, so as in Fact 1 it changes no divergence, and the normal distribution with independent equal-variance coordinates looks the same after any rotation. Now both distributions are products: the first coordinate is $\mathcal{N}(0,\sigma^2)$ or $\mathcal{N}(\|v\|,\sigma^2)$, and the other $d-1$ are $\mathcal{N}(0,\sigma^2 I_{d-1})$ under both, with some density h. The hockey-stick is the integral of $(p-\gamma q)_+$, and here $(p_1 h - \gamma q_1 h)_+ = h\,(p_1-\gamma q_1)_+$ because $h \geq 0$. Integrating out the last $d-1$ coordinates, where h integrates to 1, leaves E_γ of the first coordinate alone. Coordinates that are identical under both hypotheses carry no evidence.
3. **Use the closed form.** For $\mathcal{N}(0,\sigma^2)$ against $\mathcal{N}(s,\sigma^2)$ with $\mu = s/\sigma$, Fact 3 says $E_\gamma = P(B) - \gamma Q(B)$ with $B = \{L > \ln\gamma\}$. Step 1 of Section 20.5 showed that under P the loss is $\mathcal{N}(\mu^2/2,\mu^2)$, so $P(B) = \Phi(\frac{\mu}{2} - \frac{\ln\gamma}{\mu})$, as in step 2 there. Under Q the noise is centred one shift further, which lowers the loss by μ^2, so the loss is $\mathcal{N}(-\mu^2/2,\mu^2)$ and $Q(B) = \Phi(-\frac{\mu}{2} - \frac{\ln\gamma}{\mu})$. This is the closed form of Divergences, $E_\gamma = \Phi(\frac{\mu}{2} - \frac{\ln\gamma}{\mu}) - \gamma\Phi(-\frac{\mu}{2} - \frac{\ln\gamma}{\mu})$. Set $\gamma = e^\varepsilon$ and $s = \|v\|$.
4. **The worst pair has the largest shift.** Intuitively, a smaller shift is harder to detect. To prove it, show that the pair with a smaller shift can be produced from the pair with a larger shift by adding noise, which cannot help anyone tell them apart. Let $\mu' < \mu$. Take Y from the pair with shift μ and unit variance, and output $Y' = \frac{\mu'}{\mu}Y + W$ with independent $W \sim \mathcal{N}(0, 1-\mu'^2/\mu^2)$. If $Y \sim \mathcal{N}(0,1)$, then Y' has mean 0 and variance $\mu'^2/\mu^2 + (1-\mu'^2/\mu^2) = 1$, so $Y' \sim \mathcal{N}(0,1)$; if $Y \sim \mathcal{N}(\mu,1)$, the mean becomes μ' and the variance is still 1, so $Y' \sim \mathcal{N}(\mu',1)$. So the pair with shift μ' is a post-processing of the pair with shift μ. By the data-processing inequality (whatever you can detect from Y', you can detect from Y by computing Y' yourself), its hockey-stick is no larger. Hence $\delta_\mu(\varepsilon)$ is non-decreasing in μ, and the maximum over neighbours is at $\|v\| = \Delta_2 f$.
5. **"If and only if."** The hockey-stick at $\gamma = e^\varepsilon$ is the smallest δ that works for one pair, as shown in the Notation paragraph above, and steps 1–4 show that the pair at shift $\Delta_2 f$ is the worst. So $\delta \geq \delta_\mu(\varepsilon)$ is sufficient (it covers the worst pair, hence all) and necessary (any smaller δ fails on that pair).
6. **Monotone in ε.** Each E_γ is non-increasing in γ (Divergences): raising γ lowers $p-\gamma q$ at every point. Exercise 3 in the exercise book computes the derivative exactly: $\frac{d}{d\varepsilon}\delta_\mu(\varepsilon) = -e^\varepsilon\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}) < 0$.
7. **The calibration is well defined.** To be sure the equation $\delta_\mu(\varepsilon) = \delta$ has exactly one root in μ, check that δ_μ runs continuously from 0 to 1 and is strictly increasing. It is continuous in μ. It tends to 0 as $\mu \to 0$ (no signal; both arguments of Φ go to $-\infty$) and to 1 as $\mu \to \infty$ (the first Φ goes to 1 and the second to 0). Step 4 shows it is non-decreasing, and the derivative $\frac{d}{d\mu}\delta_\mu(\varepsilon) = \varphi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) > 0$ (Exercise 3 in the exercise book) shows it is strictly increasing. So for $0 < \delta < 1$ the equation has exactly one root μ^*.

Solving it by bisection. Bisection finds the crossing point of an increasing function by repeatedly halving an interval that contains it. Start with an interval $[\mu_{\text{lo}}, \mu_{\text{hi}}]$ with $\delta_{\mu_{\text{lo}}}(\varepsilon) < \delta < \delta_{\mu_{\text{hi}}}(\varepsilon)$; the script uses $[e^{-12}, e^8]$. Evaluate at the midpoint and keep the half where the sign of $\delta_\mu(\varepsilon) - \delta$ changes. The midpoint is taken in $\ln\mu$, because μ^* ranges over several orders of magnitude. Each step halves the interval, so 60 steps pin μ^* to machine precision. Monotonicity is what makes this

safe: there is exactly one crossing to find. At $\varepsilon = 1$, $\delta = 10^{-5}$ the script finds $\sigma^* = 3.7306\,\Delta_2 f$, and evaluating Equation 20.17 there gives back $\delta = 1.000 \times 10^{-5}$.

Checking the closed form by direct integration. As an independent check that uses no tail formula, the script also computes $\int (p - e^{\varepsilon} q)_+ \, dy$ by Simpson's rule on a grid of step 10^{-3} over $[-40, 40]$, using only the two normal densities. Over the 30-point grid $\varepsilon \in \{0, 0.5, 1, 2, 4, 8\}$, $\mu \in \{0.25, 0.5, 1, 2, 4\}$, keeping the 24 pairs (ε, μ) with $\delta > 10^{-12}$ (below that the relative error of a grid sum is no longer meaningful), the largest relative difference from Equation 20.17 is 3.1×10^{-6}. That is the discretization error of the grid at the kink where $p = e^{\varepsilon} q$, where the integrand has a corner. The crosses on Figure 20.2, at $\varepsilon = 0, 1, \ldots, 7$ on each curve, are integrals computed the same way.

Concretely, at $\mu = 1$, $\varepsilon = 1$. Table 20.6 evaluates the two terms of Equation 20.5. Measure the noise z in units of σ. With $\mu = 1$ the loss is $L = \frac{1}{2} - z$, so the bad event $B = \{L > 1\}$ is $\{z < -\frac{1}{2}\}$. Under $M(D)$, z is standard normal, so $P(B) = \Phi(-0.5)$. Under $M(D')$ the output is centred one unit higher, so in the same coordinates z is $\mathcal{N}(1, 1)$, and $Q(B) = \Pr[z < -\frac{1}{2}] = \Phi(-\frac{1}{2} - 1) = \Phi(-1.5)$.

Table 20.6.: The exact δ at $\mu = 1$, $\varepsilon = 1$ as the bad-event probability minus its correction.

Term	Formula	Value
$P(B)$, the bad-event bound	$\Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) = \Phi(-0.5)$	0.3085
$e^{\varepsilon} Q(B)$, the correction	$e\,\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}) = e\,\Phi(-1.5)$	0.1816
$\delta_1(1)$, the exact δ	difference	0.1269

Simpson's rule gives 0.1269 as well. A Monte Carlo average of $(1 - e^{\varepsilon - L})_+$, the expectation form of Fact 3, over a million draws of $L \sim \mathcal{N}(\frac{1}{2}, 1)$ gives 0.1270.

20.6.4. Interpretation

One Gaussian mechanism gives a whole curve of guarantees, not one. It is not "(ε, δ)-DP" for a single pair; it is $(\varepsilon, \delta_\mu(\varepsilon))$-DP for every $\varepsilon \geq 0$ simultaneously, trading a larger ε for a smaller δ (Figure 20.2). At $\sigma = \Delta_2 f$ ($\mu = 1$) the curve passes through $\delta = 0.3829$ at $\varepsilon = 0$ (the TV distance, $2\Phi(\frac{1}{2}) - 1$), 0.1269 at $\varepsilon = 1$, 0.0209 at $\varepsilon = 2$ and 4.71×10^{-5} at $\varepsilon = 4$, and reaches 10^{-5} at $\varepsilon = 4.3772$. The whole curve, not a single point, is what composes tightly in Part 4.

Only the ratio $\mu = \Delta_2 f / \sigma$ matters. Doubling the sensitivity and doubling the noise give the same curve, because Equation 20.17 depends on $\Delta_2 f$ and σ only through their ratio. You can read μ as a signal-to-noise ratio: how many noise standard deviations apart the two answers are. It is exactly the parameter of the Gaussian trade-off curve in Hypothesis testing, the curve of the smallest miss rate any test can have at each false alarm rate when telling $\mathcal{N}(0, 1)$ from $\mathcal{N}(\mu, 1)$.

The exact σ is 1.3 to 1.7 times smaller than the classic one. Table 20.7 compares the two calibrations at $\delta = 10^{-5}$. Over the rows with $\varepsilon \leq 1$, the classic formula uses between 1.30 and 1.68 times the necessary noise, and the gap grows as ε shrinks. Since variance is σ^2, the factor 1.30 at $\varepsilon = 1$ is $1.299^2 \approx 1.69$ times the variance, 69% more than needed, which you could instead spend on a smaller ε.

For large ε the classic formula is wrong, not just loose. The classic σ shrinks like $1/\varepsilon$. The exact one shrinks more slowly, so the two cross. At $\delta = 10^{-5}$ they cross at $\varepsilon = 8.42$, and beyond it the classic σ is below the necessary noise.

Table 20.7.: Classic against analytic calibration of the Gaussian mechanism at $\delta = 10^{-5}$. The last column is Equation 20.17 evaluated at the classic σ.

ε	Classic $\sigma/\Delta_2 f$	Analytic $\sigma^*/\Delta_2 f$	Ratio	True δ at the classic σ
0.05	96.8961	57.7707	1.677	1.29×10^{-9}
0.1	48.4481	30.7496	1.576	2.64×10^{-9}
0.25	19.3792	13.2855	1.459	7.10×10^{-9}
0.5	9.6896	7.0318	1.378	1.61×10^{-8}
1	4.8448	3.7306	1.299	4.11×10^{-8}
2	2.4224	1.9938	1.215	1.34×10^{-7}
4	1.2112	1.0812	1.120	6.84×10^{-7}
8	0.6056	0.6002	1.009	7.97×10^{-6}

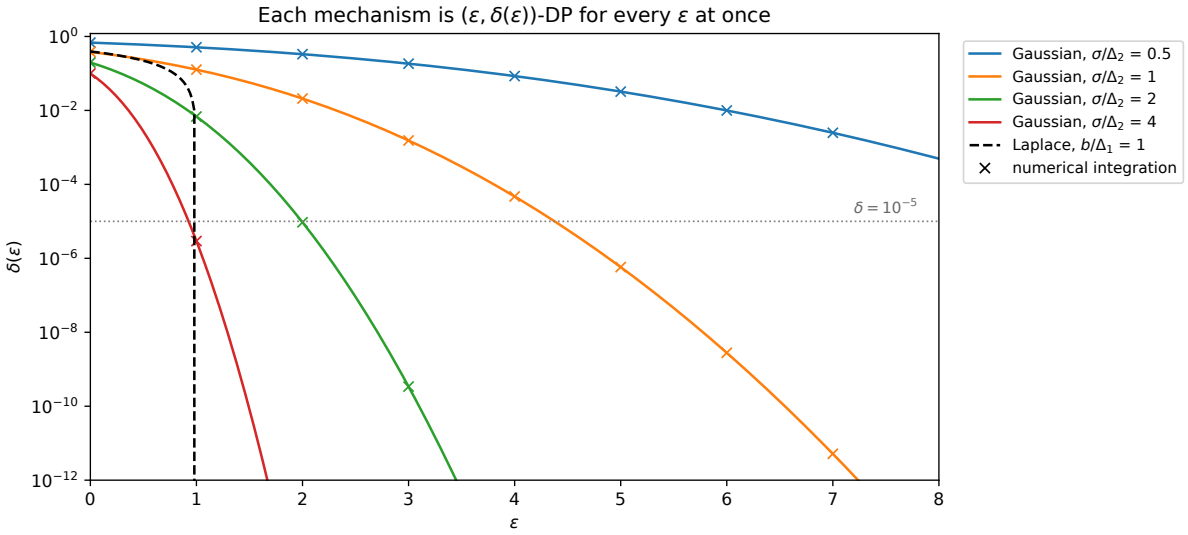


Figure 20.2.: The privacy curve $\delta(\varepsilon)$ of the Gaussian mechanism for four noise levels $\sigma/\Delta_2 f$, on a log scale, and of the Laplace mechanism with $b = \Delta_1 f$ (dashed). Crosses are the hockey-stick divergence computed by numerical integration. The Laplace curve drops to zero at $\varepsilon = 1$, its pure-DP level; every Gaussian curve stays positive but falls faster than exponentially. The dotted line marks $\delta = 10^{-5}$.

20.6.5. Example: the diabetes count again

Release the count of Section 20.4 with the Gaussian mechanism at $\varepsilon = 0.1$, $\delta = 10^{-5}$. The sensitivity is 1, so the analytic calibration gives $\sigma^* = 30.75$ (the row $\varepsilon = 0.1$ of Table 20.7). A normal variable stays within 1.96 standard deviations of its mean with probability 0.95, so the

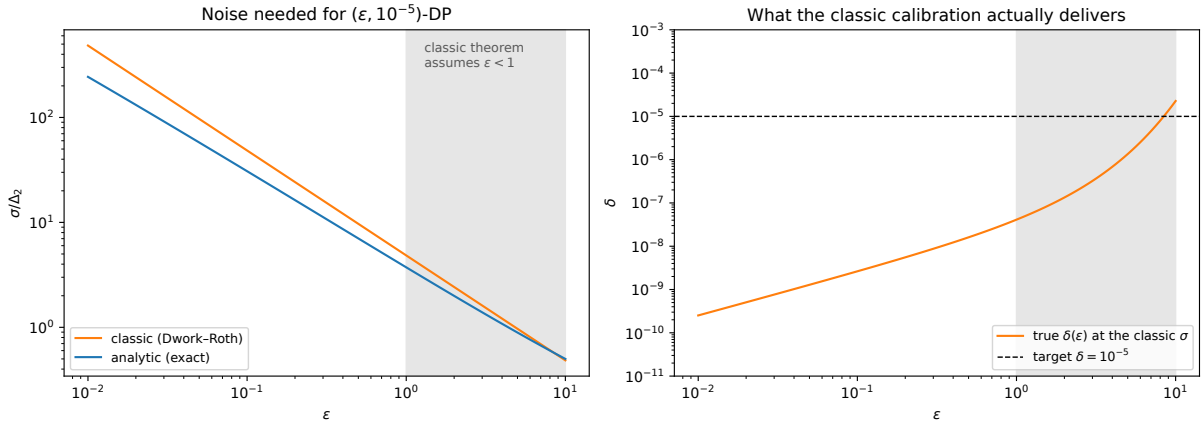


Figure 20.3.: Left: the noise needed for $(\varepsilon, 10^{-5})$-DP by the classic formula and by the analytic calibration. Right: the true δ that the classic σ delivers, from Equation 20.17. It is orders of magnitude below the target for small ε and crosses the target at $\varepsilon = 8.42$, in the shaded region where the classic theorem makes no claim.

95% error is $1.96\sigma^* = 60.27$, against 29.96 for Laplace noise with $\delta = 0$. For a single count, the Laplace mechanism is better on both counts: smaller error and a stronger guarantee. The Gaussian mechanism earns its place only when many numbers are released together, as Section 20.7 shows.

20.6.6. Check yourself

- Why is $\delta_\mu(0)$ the TV distance between $\mathcal{N}(0,1)$ and $\mathcal{N}(\mu,1)$? *(At $\varepsilon = 0$, $\gamma = e^0 = 1$, and the hockey-stick $E_1 = \max_S[P(S) - Q(S)]$ is the definition of the TV distance, its first property in Divergences.)*
- You have a Gaussian mechanism with $\sigma = \Delta_2 f$ and someone asks for its guarantee at $\delta = 10^{-5}$. What do you answer, and how would you compute it? *($\varepsilon = 4.3772$: here $\mu = 1$, so solve $\delta_1(\varepsilon) = 10^{-5}$ for ε by bisection, using that $\delta_\mu(\varepsilon)$ is decreasing in ε.)*

20.7. Laplace against Gaussian in many dimensions

20.7.1. Context

Real releases are vectors: a histogram over many bins, all the marginals of a table, a gradient with millions of coordinates. The Laplace mechanism scales with $\Delta_1 f$ and the Gaussian mechanism with $\Delta_2 f$, and by Equation 20.8 these can differ by a factor of up to $\sqrt{d}$. Which noise is better depends on how one record spreads across the coordinates.

20.7.2. Statement

To compare the two, look at the typical error in each released number, the root-mean-square error per coordinate, at the same ε. For Laplace noise it is the standard deviation of $\mathrm{Lap}(\Delta_1 f/\varepsilon)$, and for Gaussian noise it is σ^*. Because the exact Gaussian calibration depends only on $\mu = \Delta_2 f/\sigma$, it is enough to solve it once for unit sensitivity: let $\sigma_1(\varepsilon, \delta)$ be the analytic σ for unit ℓ_2 sensitivity, so that $\sigma^* = \Delta_2 f\, \sigma_1(\varepsilon, \delta)$. Then the errors per coordinate are

$$\text{Laplace: } \frac{\sqrt{2}\,\Delta_1 f}{\varepsilon}, \qquad \text{Gaussian: } \Delta_2 f\, \sigma_1(\varepsilon, \delta). \tag{20.19}$$

Two cases matter. For **sparse queries**, where one record moves one coordinate, $\Delta_1 f = \Delta_2 f$ (for example a histogram under add/remove). The ratio of the two errors does not depend on d, and Laplace wins whenever $\sqrt{2} < \varepsilon\, \sigma_1(\varepsilon, \delta)$. For **dense queries**, where one record moves every coordinate, $\Delta_1 f = d$ and $\Delta_2 f = \sqrt{d}$ (for example the threshold counts). The Laplace error grows like d and the Gaussian error like $\sqrt{d}$, and Laplace is more accurate only while

$$d < \frac{\left(\varepsilon\, \sigma_1(\varepsilon, \delta)\right)^2}{2}. \tag{20.20}$$

20.7.3. Derivation

1. **Laplace error.** A $\mathrm{Lap}(b)$ coordinate has mean 0 and variance $2b^2$, so its root-mean-square error is $\sqrt{2b^2} = \sqrt{2}\, b$, which with $b = \Delta_1 f/\varepsilon$ is $\sqrt{2}\, \Delta_1 f/\varepsilon$.
2. **Gaussian error.** A Gaussian coordinate has variance σ^2, so its root-mean-square error is σ. The analytic calibration gives $\sigma^* = \Delta_2 f\, \sigma_1(\varepsilon, \delta)$, because only $\mu = \Delta_2 f/\sigma$ enters Equation 20.17: scaling the sensitivity scales the solution by the same factor.
3. **Sparse queries.** With $\Delta_1 f = \Delta_2 f$, Laplace is better when $\sqrt{2}\, \Delta_2 f/\varepsilon < \Delta_2 f\, \sigma_1$; dividing by $\Delta_2 f$ and multiplying by ε leaves $\sqrt{2} < \varepsilon \sigma_1$, with no d.
4. **Dense queries.** Substitute $\Delta_1 f = d$ and $\Delta_2 f = \sqrt{d}$: Laplace is better when $\sqrt{2}\, d/\varepsilon < \sqrt{d}\, \sigma_1$. Dividing both sides by $\sqrt{d}$ and rearranging gives $\sqrt{d} < \varepsilon \sigma_1/\sqrt{2}$. Square both sides.

20.7.4. Interpretation

Use Laplace noise for sparse queries and Gaussian noise for dense ones. When a record touches one coordinate, the two sensitivities agree, and the stronger guarantee of Laplace ($\delta = 0$) comes with less noise too. When a record touches all coordinates, $\Delta_1 f$ pays d while $\Delta_2 f$ pays $\sqrt{d}$, and the Gaussian wins by a factor that grows like $\sqrt{d}$ (Figure 20.4).

This is why private training of neural networks uses Gaussian noise. One example's gradient moves every coordinate of a model with millions of parameters, the densest query there is. Clipping bounds its ℓ_2 norm, not its ℓ_1 norm, and the Gaussian mechanism is calibrated to exactly that.

Keep in mind that the comparison is at equal ε but unequal guarantees. The Laplace release is ε-DP; the Gaussian release has an extra δ. The Gaussian mechanism also has a second advantage that this comparison does not show: many Gaussian releases compose into another Gaussian mechanism, exactly, because stacking independent Gaussian answers gives one Gaussian answer whose ℓ_2 sensitivity is the root-sum-of-squares of theirs (Exercise 5 in the exercise book).

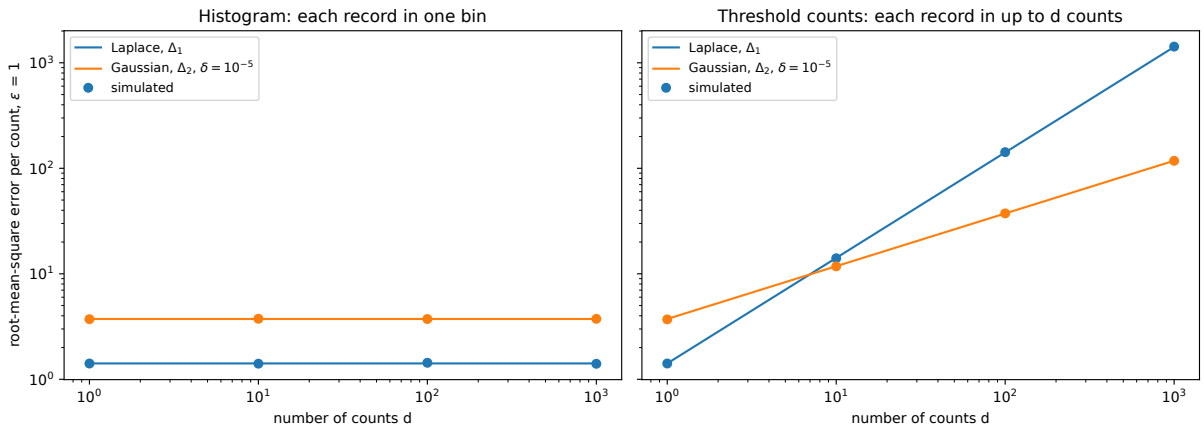


Figure 20.4.: Root-mean-square error per count at $\varepsilon = 1$ for the Laplace mechanism (calibrated to Δ_1) and the Gaussian mechanism (analytic calibration to Δ_2, $\delta = 10^{-5}$), as the number of released counts d grows; dots are simulations. Left: a histogram, where each record falls in one bin, so neither error depends on d and Laplace noise is smaller. Right: threshold counts, where a record can move all d counts; the Laplace error grows like d, the Gaussian like $\sqrt{d}$, and they cross near $d = 7$.

20.7.5. Example: releasing an empirical CDF

At $\varepsilon = 1$ and $\delta = 10^{-5}$, $\sigma_1 = 3.7306$, the value found by bisection in Section 20.6. For one count, Laplace noise has root-mean-square error $\sqrt{2} = 1.4142$ against 3.7306 for Gaussian noise, so Laplace wins. For threshold counts the crossover is at $d = (3.7306)^2/2 = 6.96$. Table 20.8 shows the errors, exactly (from $\sqrt{2}\,d$ and $3.7306\sqrt{d}$) and by simulation.

Table 20.8.: Root-mean-square error per count for d threshold counts at $\varepsilon = 1$, $\delta = 10^{-5}$.

d	Laplace, exact	Laplace, simulated	Gaussian, exact	Gaussian, simulated	Laplace / Gaussian
1	1.41	1.42	3.73	3.73	0.38
10	14.14	14.09	11.80	11.78	1.20
100	141.42	141.07	37.31	37.28	3.79
1000	1414.21	1412.90	117.97	118.11	11.99

An empirical CDF of ages read off at 100 thresholds is 3.79 times more accurate with Gaussian noise; at 1,000 thresholds, 11.99 times.

20.7.6. Check yourself

- For a histogram under *replace* neighbours, $\Delta_1 = 2$ and $\Delta_2 = \sqrt{2}$. Does that change which mechanism wins at $\varepsilon = 1$, $\delta = 10^{-5}$? *(Laplace error $\sqrt{2} \times 2 = 2\sqrt{2} = 2.83$, Gaussian $\sqrt{2} \times 3.73 = 5.28$: Laplace still wins.)*

- Why does the Gaussian advantage for dense queries grow like $\sqrt{d}$ and not like d? *(Both errors are per coordinate. The Laplace one is proportional to $\Delta_1 = d$, the Gaussian one to $\Delta_2 = \sqrt{d}$; their ratio is $d/\sqrt{d} = \sqrt{d}$ up to constants.)*

20.8. Code: check every mechanism by computation and simulation

The script code/laplace_gaussian.py computes every number in this chapter and draws every figure. Sensitivities are found by brute force over small datasets. The Laplace mechanism's privacy and accuracy are checked by simulation. For the Gaussian mechanism, the closed form Equation 20.17 is checked against direct numerical integration of the hockey-stick and against a Monte Carlo average of the privacy loss, and the analytic calibration is solved by bisection. From the repository root:

```
uv run python code/part03-dp-foundations/laplace_gaussian.py
```

Output (seed 0), summarized in Table 20.9:

Table 20.9.: The script's output with the default seed.

Experiment	Result
Sensitivities, brute force over all datasets of up to 3 records	Every row of Table 20.3: e.g. histogram under replace $\ell_1 = 2$, $\ell_2 = 1.4142$; threshold counts $\ell_1 = 3$, $\ell_2 = 1.7321$
Laplace privacy loss, count, $\varepsilon = 1$	Ranges over exactly $[-1, 1]$
Laplace tight event	$\Pr[M(D) \le 0] = 0.5002$, $\Pr[M(D') \le 0] = 0.1838$, ratio 2.7221 against $e = 2.7183$
Laplace count, $\varepsilon = 0.1$	Mean absolute error 9.978 (exact 10); 95th percentile 29.85 (bound 29.96)
Laplace histogram, 20 bins	Maximum error above 5.99 in a fraction 0.0464 of releases ($\beta = 0.05$)
Classic proof chain, $\varepsilon = 1$, $\delta = 10^{-5}$	Exact δ 4.11×10^{-8}; $\Pr[L > \varepsilon]$ 1.06×10^{-6}; Mills bound 1.10×10^{-6}
Gaussian closed form vs integration	Largest relative difference 3.1×10^{-6} over 24 pairs (ε, μ) with $\delta > 10^{-12}$
Gaussian, $\mu = 1$, $\varepsilon = 1$	$\delta = 0.1269$ (integrated 0.1269, Monte Carlo 0.1270)
Analytic calibration, $\varepsilon = 1$, $\delta = 10^{-5}$	$\sigma^* = 3.7306$, classic 4.8448; Table 20.7 for other ε
Classic σ fails at $\delta = 10^{-5}$	For $\varepsilon > 8.42$
Laplace vs Gaussian, threshold counts	Crossover at $d = 6.96$; Table 20.8

tests/test_laplace_gaussian.py checks that the output still matches.

Four things stand out. First, the formulas are right: the closed form of the Gaussian privacy curve agrees with direct integration to six digits, and the simulated errors and event probabilities agree with their formulas to within sampling error. Second, the Laplace guarantee is tight: the simulated ratio on the event $\{y \le 0\}$ is e^{ε}, not something smaller, so no smaller noise scale could work. Third, the classic Gaussian calibration over-delivers: at $\varepsilon = 1$ it gives a δ about 240 times smaller than requested, which is the same as saying that it adds 30% more noise than necessary. Fourth, dimension decides the noise: for sparse queries use Laplace noise, and for dense ones the Gaussian error is smaller by a factor that grows like $\sqrt{d}$.

20.9. References

- Dwork, McSherry, Nissim & Smith, "Calibrating Noise to Sensitivity in Private Data Analysis" (TCC 2006): the paper that introduced global sensitivity and the Laplace mechanism.
- Dwork, Kenthapadi, McSherry, Mironov & Naor, "Our Data, Ourselves: Privacy via Distributed Noise Generation" (EUROCRYPT 2006): approximate (ε, δ)-DP and Gaussian noise.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 for the Laplace mechanism and its accuracy, and Appendix A for the classic Gaussian calibration (Theorem A.1) with its two-sided proof.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018): the exact privacy curve Equation 20.17, the bisection calibration, and why the classic formula fails for large ε.
- Near & Abuah, *Programming Differential Privacy*, the chapters on sensitivity and on the Laplace and Gaussian mechanisms: the same material with code, and the practical pitfalls of computing sensitivity.
- Geng & Viswanath, "The Optimal Noise-Adding Mechanism in Differential Privacy" (IEEE Transactions on Information Theory, 2016): the staircase mechanism, optimal for pure DP, and how close the Laplace mechanism comes to it.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS-B 2022): the Gaussian mechanism as a trade-off curve, which is where Gaussian differential privacy takes $\mu = \Delta_2 f/\sigma$ next.

20.10. Cheat sheet

f has values in $\mathbb{R}^d$; $\Delta_1 f$, $\Delta_2 f$ are its ℓ_1, ℓ_2 sensitivities; $\mu = \Delta_2 f/\sigma$. Table 20.10 collects the results.

Table 20.10.: Every result in this chapter.

Result	Formula	Notes
Sensitivity	$\Delta_p f = \max_{D \simeq D'} \lVert f(D) - f(D') \rVert_p$	$\Delta_2 \le \Delta_1 \le \sqrt{d}\,\Delta_2$; depends on the neighbouring relation
Count, sum in $[0, B]$, mean with public n	1; B; B/n	Add/remove, add/remove or replace, replace

Result	Formula	Notes
Histogram	$\Delta_1 = \Delta_2 = 1$ (add/remove); 2, $\sqrt{2}$ (replace)	One record, one bin
Laplace mechanism (Equation 20.9)	$f(D) + \mathrm{Lap}(\Delta_1 f/\varepsilon)^{\otimes d}$ is ε-DP	Tight: $L = \varepsilon$ on half the mass
Laplace accuracy	$\Pr[\max_i \lvert Z_i\rvert > b\ln(d/\beta)] \le \beta$; $\mathbb{E}\lvert Z_i\rvert = b$	Error independent of n
Privacy loss of Gaussian noise	$L \sim \mathcal{N}(\mu^2/2, \mu^2)$ under $M(D)$	Unbounded, so no pure DP
Bad-event bound	$\delta \le \Pr[L > \varepsilon] = \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu})$	Loose by the term $e^{\varepsilon}Q(L > \varepsilon)$
Classic calibration (Equation 20.13)	$\sigma = \Delta_2 f\sqrt{2\ln(1.25/\delta)}/\varepsilon$, $\varepsilon < 1$	1.3–1.7 times too much noise; wrong for large ε
Exact privacy curve	$\delta_\mu(\varepsilon) = \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) - e^{\varepsilon}\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$	If and only if; $\$\delta_\mu(0) = \$ TV
Analytic calibration	$\sigma^* = \Delta_2 f/\mu^*$ with $\delta_{\mu^*}(\varepsilon) = \delta$, by bisection	δ_μ is increasing in μ, so one root
Laplace vs Gaussian, per coordinate	$\sqrt{2}\,\Delta_1 f/\varepsilon$ vs $\Delta_2 f\,\sigma_1(\varepsilon, \delta)$	Dense queries: Laplace wins only for $d < (\varepsilon\sigma_1)^2/2$

Rules of thumb

- First compute the sensitivity, and say which neighbouring relation it is for.
- One number or a sparse vector: Laplace noise, pure DP. A dense vector: Gaussian noise, calibrated to Δ_2.
- Never use the classic Gaussian formula when you can solve Equation 20.17. Solving it costs one bisection, and the classic formula adds 30–70% more noise than needed.
- At $\varepsilon = 1$, $\delta = 10^{-5}$ the Gaussian mechanism needs $\sigma \approx 3.73\,\Delta_2 f$; Laplace needs $b = \Delta_1 f$.
- A Gaussian mechanism has a whole privacy curve; quote the curve (or μ), not just one (ε, δ) pair.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

21. Advanced and optimal composition

21.1. Why this matters

A differentially private system rarely answers one query. A dashboard refreshes a hundred counts, an analyst explores a dataset with dozens of queries, and a training run takes thousands of noisy gradient steps. Each step satisfies (ε, δ)-differential privacy (DP): for any two **neighbouring** datasets D and D', which differ in one person's record, the probability of any set of outputs changes by at most a factor e^{ε}, plus a small slack δ. You need the same kind of guarantee for the whole run. Post-processing, group privacy and composition gave the first answer, **basic composition**: k mechanisms that are each ε-DP are together $k\varepsilon$-DP, because the factors e^{ε} of the k steps multiply. That answer is correct, and for large k it is far too pessimistic. A hundred queries at $\varepsilon = 0.1$ give $k\varepsilon = 10$. Since ε-DP says that no output is more than e^{ε} times likelier under one dataset than under the other, an attacker's odds that the data are D rather than D' can be multiplied by up to $e^{10} \approx 22{,}000$ by what it sees. Nothing that weak is worth stating. Yet at $\delta = 10^{-5}$ the same hundred queries are $(4.31, 10^{-5})$-DP, and no analysis that knows only each query's ε can say less (Section 21.5).

This chapter explains where the gap comes from and how to close it, in three steps. The first question is how fast privacy really degrades under composition. The answer is like $\sqrt{k}$, not k, once you allow a small δ; this is **advanced composition**. The second question is what the best possible bound is. It turns out to be one you can compute exactly, because one simple mechanism, randomized response, is the worst (ε, δ)-DP mechanism there is; this is **optimal composition**. The third question is whether any of this survives an analyst who chooses each query after seeing the earlier answers. It does; this is **adaptive composition**.

21.1.1. The one idea: the privacy loss of a composition is a sum

Run a mechanism M on the dataset D and look at the output Y. Ask how much more likely this output was under D than under the neighbour D'. The ratio of the two probabilities (or densities) says it, and its logarithm turns it into an additive amount of evidence: positive when Y favours D, negative when it favours D', zero when it says nothing. This log ratio is the **privacy loss random variable**,

$$L = L_{D,D'}(Y) = \ln \frac{p_{M(D)}(Y)}{p_{M(D')}(Y)}, \qquad Y \sim M(D). \tag{21.1}$$

It is a random variable because Y is random, and its distribution, with Y drawn from $M(D)$, is the **privacy loss distribution**. Pure ε-DP says that no output is more than e^{ε} times likelier under one dataset than under the other, in either direction, so $|L| \le \varepsilon$ for every output. Now run k mechanisms. The probability of the whole sequence of outputs is the product of the probabilities

of the individual outputs, a ratio of two products is the product of the ratios, and the logarithm turns that product into a sum. So the total privacy loss is the sum $L_1 + \cdots + L_k$ of the per-step losses (Fact 2 below).

Basic composition bounds that sum by $k\varepsilon$, as if every step produced its worst output. But a sum of k bounded random terms concentrates around its mean, and the extreme value needs every term to be extreme at once. Take $k = 100$ steps of **randomized response** at $\varepsilon = 0.1$: each step reports a person's bit truthfully with probability $t = e^{0.1}/(1 + e^{0.1}) = 0.5250$ and flips it otherwise. The total loss has mean 0.50 and standard deviation 1.00 (Section 21.3 derives both). The value $k\varepsilon = 10$ sits $(10 - 0.50)/1.00 = 9.5$ standard deviations above the mean, and it needs all 100 reports to be truthful, which has probability $t^{100} = 1.0 \times 10^{-28}$. The slack δ in (ε, δ)-DP is exactly an allowance for rare bad outputs. So **every result in this chapter bounds the tail of a sum of privacy losses, instead of its maximum, and pays for the tail with δ.** Table 21.1 lists the results and where each is used.

Table 21.1.: The results of this chapter and where each is used.

Result	Where it is used
The privacy loss random variable; $\delta(\varepsilon)$ as its average	Every accountant in Privacy accounting
The loss of a composition is a sum	All composition theorems, adaptive or not
Advanced composition (Dwork, Rothblum & Vadhan)	Quick $\sqrt{k}$ budgets; the classic analysis of iterative algorithms
Randomized response is the worst (ε, δ) mechanism	Optimal composition; the sharp mean loss $\varepsilon \tanh(\varepsilon/2)$
Optimal composition (Kairouz, Oh & Viswanath), computed exactly	The best bound for any k mechanisms known only by their (ε, δ)
Adaptive composition	Interactive query systems, iterative training, hyperparameter search

What you need. The definition of (ε, δ)-DP, randomized response and the hockey-stick form are in The definition of differential privacy. Basic composition is in Post-processing, group privacy and composition. From Part 1 you need the hockey-stick divergence, the Kullback–Leibler (KL) divergence and the data-processing inequality (Divergences), trade-off functions (Hypothesis testing) and Azuma's inequality (Concentration inequalities). Each is restated below where it is used.

Notation. M is a mechanism and $D \simeq D'$ are neighbours. $P = M(D)$ and $Q = M(D')$ are the two output distributions, with mass functions or densities p and q. L is the privacy loss of Equation 21.1. The definition of (ε, δ)-DP asks that $P(S) \le e^{\varepsilon} Q(S) + \delta$ for every set of outputs S, so for one ordered pair the smallest δ that works is the largest gap $P(S) - e^{\varepsilon} Q(S)$ over all sets. That gap is largest when S holds exactly the outputs with $p(y) > e^{\varepsilon} q(y)$, since each such output adds a positive amount and every other output would subtract. The largest gap is the **hockey-stick divergence** $E_{e^{\varepsilon}}(P\|Q) = \sum_y \left(p(y) - e^{\varepsilon} q(y)\right)_+$, where $(x)_+ = \max\{x, 0\}$, and we

write $\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q)$ for the smallest δ that works for a given ε, in one direction. DP asks for this in both directions, for every pair of neighbours. Randomized response at level ε reports a bit truthfully with probability $t = e^\varepsilon/(1+e^\varepsilon)$. Logarithms are natural.

21.2. Toolkit: three facts every derivation uses

21.2.1. Fact 1 — $\delta(\varepsilon)$ is an average of the privacy loss

The hockey-stick sum looks like a property of two whole distributions, but each output's contribution depends only on its own privacy loss. An output whose loss is at most ε contributes nothing. An output that is impossible under D', with infinite loss, contributes its whole probability. Outputs in between contribute part of their probability, more the larger the loss. So $\delta(\varepsilon)$ is an average over the output, drawn from P, of a function of the loss, and it is at most the probability that the loss exceeds ε. For any pair (P, Q) and any ε,

$$\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q) = \sum_y \big(p(y) - e^\varepsilon q(y)\big)_+ = \mathbb{E}\big[(1 - e^{\varepsilon - L})_+\big] \le \Pr[L > \varepsilon]. \tag{21.2}$$

To see the middle equality, factor $p(y)$ out of each term: $(p - e^\varepsilon q)_+ = p\,(1 - e^\varepsilon q/p)_+$, and $e^\varepsilon q/p = e^{\varepsilon - \ln(p/q)} = e^{\varepsilon - L(y)}$. A sum of $p(y)$ times something over all y is the expectation of that something when Y is drawn from P. For the inequality, look at the bracket: when $L \le \varepsilon$ the exponent $\varepsilon - L$ is non-negative, $e^{\varepsilon - L} \ge 1$, and the bracket is 0. When $L > \varepsilon$ the bracket is $1 - e^{\varepsilon - L}$, which is below 1. So the bracket is at most the indicator of $L > \varepsilon$, and its average is at most $\Pr[L > \varepsilon]$.

A concrete case: randomized response at $\varepsilon = \ln 3$ has $t = 3/(1+3) = \frac{3}{4}$. Take a person whose bit is 0 in D and 1 in D'. A report of 0 has probability $\frac{3}{4}$ under D and $\frac{1}{4}$ under D', so its loss is $\ln 3$; a report of 1 has loss $-\ln 3$. Under D the loss is therefore $+\ln 3$ with probability $\frac{3}{4}$ and $-\ln 3$ with probability $\frac{1}{4}$. At $\varepsilon = 0$ the bracket is $1 - e^{-\ln 3} = 1 - \frac{1}{3}$ for the first value and $(1-3)_+ = 0$ for the second, so $\delta(0) = \frac{3}{4}(1 - \frac{1}{3}) = \frac{1}{2}$. At $\varepsilon = 0$ the hockey-stick sum is $\sum_y (p-q)_+$, the largest difference $P(S) - Q(S)$ over sets of outputs, which is the total variation distance between the two output distributions. Here it is $\frac{3}{4} - \frac{1}{4} = \frac{1}{2}$, as it should be.

The use of this fact is that a composition theorem only has to bound the distribution of the total privacy loss. The tail bound $\Pr[L > \varepsilon]$ is the cheap version, and Equation 21.2 is the exact one.

21.2.2. Fact 2 — The privacy loss of a composition is a sum

Run $M_1, \dots, M_k$ in sequence, where M_i may depend on the earlier outputs $y_{<i} = (y_1, \dots, y_{i-1})$. The probability of a whole sequence of outputs is built one step at a time: the probability of y_1, times the probability of y_2 given y_1, and so on (the chain rule of probability). Write $p_i(y_i \mid y_{<i})$ for the probability that step i outputs y_i on D after the history $y_{<i}$, and q_i for the same on D'. Then the joint output has density $p(y_1, \dots, y_k) = \prod_i p_i(y_i \mid y_{<i})$ on D, and likewise with q_i on D'. The ratio of the two products is the product of the step-by-step ratios, and the logarithm of a product is the sum of the logarithms:

$$L(y_1, \dots, y_k) = \sum_{i=1}^{k} L_i(y_i \mid y_{<i}), \qquad L_i(y_i \mid y_{<i}) = \ln \frac{p_i(y_i \mid y_{<i})}{q_i(y_i \mid y_{<i})}. \tag{21.3}$$

Each term L_i is the privacy loss of step i alone, for the mechanism that was run after that history.

A concrete case: two rounds of randomized response at $\varepsilon = \ln 3$, on a person whose bit is 0 in D and 1 in D'. Each round reports 0 with probability $\frac{3}{4}$ on D and $\frac{1}{4}$ on D', independently, so for instance $p(0,0) = \frac{3}{4} \cdot \frac{3}{4} = \frac{9}{16}$ and $q(0,0) = \frac{1}{4} \cdot \frac{1}{4} = \frac{1}{16}$. Table 21.2 lists the four outputs. Each loss is the sum of two per-round terms $\pm \ln 3$: the output $(0,1)$ has loss $\ln 3 - \ln 3 = 0$, because one report favours D as much as the other favours D'.

Table 21.2.: Two rounds of randomized response at $\varepsilon = \ln 3$. The privacy loss of each output is the sum of the two per-round losses.

Output (y_1, y_2)	$(0,0)$	$(0,1)$	$(1,0)$	$(1,1)$
p (bit 0)	9/16	3/16	3/16	1/16
q (bit 1)	1/16	3/16	3/16	9/16
$L = \ln(p/q)$	$2\ln 3$	0	0	$-2\ln 3$

So the total loss is a sum of terms, and each term is bounded by its own step's guarantee, whatever the history: if step i is ε-DP for every history, then $|L_i| \le \varepsilon$. Adding k such bounds gives $|L| \le k\varepsilon$, which is basic composition. The rest of this chapter bounds the same sum more cleverly.

21.2.3. Fact 3 — The mean privacy loss of an ε-DP pair is second order in ε

The loss can be as large as ε, but on average it is much smaller. When ε is small, an output that favours D is only slightly more likely than one that favours D', so positive and negative losses nearly cancel, and what is left over is of order ε^2. The average loss under P is $\sum_y p(y) \ln(p(y)/q(y))$, which is by definition the KL divergence $D_{\mathrm{KL}}(P\|Q)$. If $|L| \le \varepsilon$ everywhere, then

$$\mathbb{E}[L] = D_{\mathrm{KL}}(P\|Q) \le \varepsilon(e^{\varepsilon} - 1). \tag{21.4}$$

The derivation makes the cancellation visible by pairing each p with its q.

1. **Symmetrize.** The KL divergence is never negative (Divergences): by Jensen's inequality for the concave logarithm, $-D_{\mathrm{KL}}(Q\|P) = \sum_y q \ln(p/q) \le \ln \sum_y q\,(p/q) = \ln 1 = 0$. So adding $D_{\mathrm{KL}}(Q\|P) \ge 0$ to $\mathbb{E}[L]$ can only increase it, and the two sums combine term by term: $\mathbb{E}[L] \le \sum_y p \ln \frac{p}{q} + \sum_y q \ln \frac{q}{p} = \sum_y (p-q) \ln \frac{p}{q}$.
2. **Use the bound on the loss.** Each $|\ln(p/q)|$ is at most ε, so the sum is at most $\varepsilon \sum_y |p-q|$. This is where the cancellation pays off: the factor $\sum_y |p-q|$ is itself small when P and Q are close.
3. **Bound the total difference.** Write $\sum_y |p-q| = \sum_y q\,|p/q - 1|$. The ratio $r = p/q$ lies in $[e^{-\varepsilon}, e^{\varepsilon}]$, and on that interval $|r-1|$ is largest at $r = e^{\varepsilon}$, where it is $e^{\varepsilon} - 1$ (at the other end it is $1 - e^{-\varepsilon}$, which is smaller). Since the $q(y)$ sum to 1, $\sum_y |p-q| \le e^{\varepsilon} - 1$, and the bound follows.

Why this matters: the loss is bounded by ε, but its average is of order ε^2, because $e^\varepsilon - 1 \approx \varepsilon$ for small ε. At $\varepsilon = 0.1$ the bound is $0.1(e^{0.1} - 1) = 0.0105$. Over k steps the mean grows like $k\varepsilon^2$, much slower than the worst case $k\varepsilon$.

How tight is it? Not very. The optimal-composition section (Section 21.5) shows that the best bound is $\varepsilon \tanh(\varepsilon/2)$, attained by randomized response. Since $\tanh(\varepsilon/2) \approx \varepsilon/2$ while $e^\varepsilon - 1 \approx \varepsilon$, the sharp bound is about half of Equation 21.4 for small ε: it is 0.0050 at $\varepsilon = 0.1$. At $\varepsilon = 1$ the gap is wider, 0.4621 against 1.7183.

21.3. The privacy loss of a composition

21.3.1. Context

Basic composition charges $k\varepsilon$, the largest value the total loss can take. Before improving on it, look at the whole distribution of the total loss for the simplest mechanism, randomized response composed k times. It shows how unlikely the worst case is, and it turns out to be the worst case for every later result.

21.3.2. Statement

Compose k rounds of randomized response at level ε, on a person whose bit is 0 in D and 1 in D'. A report of 0 is the truth under D, so it **points to** D; a report of 1 points to D'. Under D, each report points to D' only when it is a lie, which happens with probability $1 - t$. Let J be the number of rounds whose report points to D'. Each such round contributes $-\varepsilon$ to the loss and each other round $+\varepsilon$, so the whole loss is fixed by J. Under D,

$$J \sim \mathrm{Bin}(k, 1 - t), \qquad L = (k - 2J)\varepsilon, \tag{21.5}$$

and the loss has mean, standard deviation and worst-case probability

$$\mathbb{E}[L] = k\varepsilon \tanh(\varepsilon/2), \qquad \mathrm{sd}(L) = \varepsilon\sqrt{k(1 - \tanh^2(\varepsilon/2))}, \qquad \Pr[L = k\varepsilon] = t^k. \tag{21.6}$$

21.3.3. Derivation

1. **One round.** A truthful report has probability t under D and $1 - t$ under D', so its loss is $\ln \frac{t}{1-t} = \ln e^\varepsilon = \varepsilon$, because $t/(1 - t) = e^\varepsilon$ by the definition of t. A lie has the two probabilities swapped and loss $-\varepsilon$.
2. k **rounds.** By Fact 2 the total is the sum of the k per-round losses, and the rounds are independent. J rounds contribute $-\varepsilon$ and $k - J$ contribute $+\varepsilon$, so $L = (k - J)\varepsilon - J\varepsilon = (k - 2J)\varepsilon$. Each round points to D' independently with probability $1 - t$, so J counts successes in k independent trials and is binomial.
3. **Mean.** One round has mean $\varepsilon t - \varepsilon(1 - t) = \varepsilon(2t - 1)$. Substituting t gives $2t - 1 = \frac{2e^\varepsilon - (1 + e^\varepsilon)}{1 + e^\varepsilon} = \frac{e^\varepsilon - 1}{e^\varepsilon + 1}$, and this is $\tanh(\varepsilon/2)$, because $\tanh x = \frac{e^{2x} - 1}{e^{2x} + 1}$. The mean of a sum is the sum of the means, so multiply by k.

4. **Standard deviation.** One round takes the two values $\pm\varepsilon$, so its square is always ε^2 and its variance is $\varepsilon^2 - (\text{mean})^2 = \varepsilon^2(1 - \tanh^2(\varepsilon/2))$. Variances of independent terms add, so the total variance is k times this; take the square root.
5. **The worst case** $L = k\varepsilon$ needs $J = 0$: every report truthful, which has probability t^k.

21.3.4. Interpretation

The worst case is absurdly rare. For $k = 100$ and $\varepsilon = 0.1$, $t = 0.5250$ and $t^k = 1.0 \times 10^{-28}$. Basic composition guards against an event that will never happen.

What does happen is set by the mean and the spread. For small ε, $\tanh(\varepsilon/2) \approx \varepsilon/2$, so the mean is about $k\varepsilon^2/2$ and the standard deviation about $\sqrt{k}\,\varepsilon$ (the factor $1 - \tanh^2$ is close to 1). With $k = 100$ and $\varepsilon = 0.1$, the mean is 0.4996 and the standard deviation 0.9988. Both are far below $k\varepsilon = 10$ (Figure 21.1).

A sum of many small, independent, bounded terms is nearly normal: that is the central limit theorem. So the total loss is close to a normal distribution with this mean and spread. The optimal ε' at $\delta = 10^{-5}$ that Section 21.5 computes, 4.31, sits $(4.3068 - 0.4996)/0.9988 = 3.8$ standard deviations above the mean, far enough into the tail that only a sliver of a normal distribution lies beyond it. The rest of the chapter turns this picture into theorems. The normal approximation itself is the starting point of the Gaussian view of privacy in Privacy accounting.

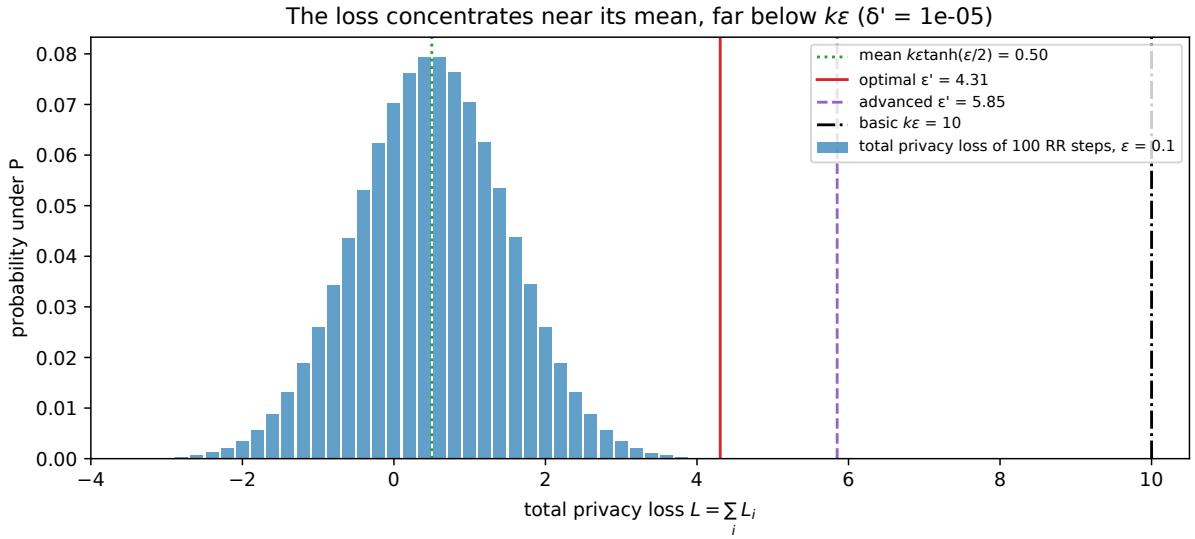


Figure 21.1.: The distribution of the total privacy loss of 100 rounds of randomized response at $\varepsilon = 0.1$, under D. The vertical lines mark its mean, the ε' of optimal composition at $\delta' = 10^{-5}$, the ε' of advanced composition, and basic composition's $k\varepsilon = 10$, which the loss reaches with probability 10^{-28}.

21.3.5. Example: a hundred noisy counts

A dashboard releases 100 counts, each with its own randomized-response or Laplace noise at $\varepsilon = 0.1$. Basic composition reports $\varepsilon = 10$. The loss distribution of composed randomized

response, which Section 21.5 shows is the worst case for 0.1-DP steps, says that the total privacy loss of any one person's data exceeds 4 only rarely. The loss $(k-2J)\varepsilon = (100-2J)\times 0.1$ is above 4 only when $100-2J > 40$, that is $J < 30$. But J has mean $100(1-t) = 47.5$ and standard deviation $\sqrt{100\,t(1-t)} = 5$, so $J < 30$ is about 3.5 standard deviations below its mean. The next two sections compute how rarely, first with a simple bound and then exactly.

21.3.6. Check yourself

- Why is the mean of the total loss non-negative, even though half the per-round losses can be negative? *(The mean is the KL divergence between the two output distributions of the whole run, which is non-negative. Each round is more likely to point to the truth than away from it.)*
- If you double k, how do the mean and the standard deviation of the total loss change? *(The mean doubles; the standard deviation grows by $\sqrt{2}$, because the variance doubles.)*

21.4. Advanced composition

21.4.1. Context

Dwork, Rothblum & Vadhan (2010) were the first to exploit the concentration of the total loss. Their theorem needs only the per-step guarantee (ε, δ), holds for any mechanisms chosen in any adaptive way, and gives a bound that grows like $\sqrt{k}$. The tool is Azuma's inequality, a version of Hoeffding's inequality for sums whose terms may depend on the earlier terms. With it the bound takes five lines.

21.4.2. Statement

The total loss is a sum of k terms, each in $[-\varepsilon, \varepsilon]$, with a small positive mean. So it is the sum of a drift, the k means added up, and a fluctuation around the drift. The fluctuation of k terms of range 2ε is of order $\varepsilon\sqrt{k}$, and Azuma's inequality says how far above its mean it goes with probability at most δ'. The drift is at most $k\varepsilon(e^{\varepsilon}-1)$ by Fact 3. Adding the two gives the bound. For every $\delta' > 0$, the composition of k mechanisms that are each (ε, δ)-DP, chosen adaptively, is $(\varepsilon', k\delta + \delta')$-DP with

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^{\varepsilon}-1). \tag{21.7}$$

With the sharp mean loss of the next section, the drift term improves to $k\varepsilon\tanh(\varepsilon/2)$:

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon\tanh(\varepsilon/2). \tag{21.8}$$

21.4.3. Derivation

Take $\delta = 0$ first: every step is ε-DP for every history. The case $\delta > 0$ is at the end of the derivation in Section 21.5.

1. **The loss is a sum.** By Fact 2, $L = \sum_i L_i$, where L_i is the loss of step i given the history $y_{<i}$.
2. **Each term is bounded, given the past.** For every fixed history, step i is ε-DP, so $L_i \in [-\varepsilon, \varepsilon]$.
3. **Each term has a small conditional mean.** Given the history, the average of L_i over step i's output is $m_i = \mathbb{E}[L_i \mid y_{<i}] = \sum_{y_i} p_i \ln(p_i/q_i)$, which is the KL divergence of step i's pair. So $m_i \le \varepsilon(e^\varepsilon - 1)$ by Fact 3.
4. **Centre the terms to get a martingale.** A **martingale** is a running sum whose next increment has mean zero given everything that happened before. Subtracting each term's conditional mean makes one: $M_k = \sum_i (L_i - m_i)$ has increments with conditional mean zero. Given the past, $L_i - m_i$ lies in $[-\varepsilon - m_i, \varepsilon - m_i]$, an interval of length $d_i = 2\varepsilon$, whose position depends on the past but whose length does not.
5. **Azuma.** The Azuma–Hoeffding inequality (Concentration inequalities) says: if each increment of a martingale has conditional mean zero and lies, given the past, in an interval of length d_i fixed in advance, then the sum exceeds s with probability at most $\exp(-2s^2/\sum_i d_i^2)$. It holds for the same reason as Hoeffding's inequality: Hoeffding's lemma bounds the conditional moment generating function of each increment using only its mean and the length of its range, and these bounds multiply one step at a time. With $d_i = 2\varepsilon$, so $\sum_i d_i^2 = 4k\varepsilon^2$,

$$\Pr[M_k \ge s] \le \exp\left(-\frac{2s^2}{\sum_i d_i^2}\right) = \exp\left(-\frac{2s^2}{4k\varepsilon^2}\right) = \exp\left(-\frac{s^2}{2k\varepsilon^2}\right). \tag{21.9}$$

6. **Choose the level.** Setting the right side to δ' and solving, $s^2 = 2k\varepsilon^2 \ln(1/\delta')$, gives $s = \varepsilon\sqrt{2k\ln(1/\delta')}$. The loss is the fluctuation plus the drift, $L = M_k + \sum_i m_i \le M_k + k\varepsilon(e^\varepsilon - 1)$. So if L exceeds $\varepsilon' = s + k\varepsilon(e^\varepsilon - 1)$ from Equation 21.7, then M_k must exceed s, and $\Pr[L > \varepsilon'] \le \delta'$.
7. **Convert.** By Fact 1, $\delta(\varepsilon') \le \Pr[L > \varepsilon'] \le \delta'$. The same argument with D and D' swapped gives the other direction, so the composition is (ε', δ')-DP.

Equation 21.8 is the same derivation with step 3 replaced by the bound $m_i \le \varepsilon \tanh(\varepsilon/2)$, derived in Section 21.5.

21.4.4. Interpretation

The gain is $\sqrt{k}$ instead of k. The leading term grows like $\varepsilon\sqrt{k}$. The price is δ' and the factor $\sqrt{2\ln(1/\delta')}$, which is $\sqrt{2 \times 11.51} = 4.80$ at $\delta' = 10^{-5}$. So advanced composition only helps once k is large enough for $\sqrt{k}$ to beat this factor. Dividing both bounds by ε, basic composition loses when $k > \sqrt{2k\ln(1/\delta')} + k(e^\varepsilon - 1)$, that is when $k > 2\ln(1/\delta')/(2 - e^\varepsilon)^2$. At $\varepsilon = 0.1$ and $\delta' = 10^{-5}$ this is $23.03/0.8948^2 = 28.8$, so advanced composition beats basic composition from $k = 29$ on (Exercise 2 in the exercise book).

The mean term is second order. For small ε, $k\varepsilon(e^\varepsilon - 1) \approx k\varepsilon^2$. Compared with the leading term $\varepsilon\sqrt{k}$, it matters when $k\varepsilon^2$ is not small next to $\varepsilon\sqrt{k}$, that is, when $\varepsilon\sqrt{k}$ is of order 1 or more.

This gives a rule of thumb for budgeting. To be $(\varepsilon_{\text{total}}, \delta')$-DP after k steps, set the leading term equal to the target and solve for the per-step budget: give each step roughly $\varepsilon = \varepsilon_{\text{total}}/\sqrt{2k\ln(1/\delta')}$. With basic composition you would need $\varepsilon_{\text{total}}/k$, which is much smaller.

The bound is loose in three places. Azuma assumes each increment could have the largest variance its range allows, ε^2, as if the loss were always at one end of its range or the other. The mean bound of Fact 3 is a factor 2 too large for small ε. And Fact 1's tail bound $\Pr[L > \varepsilon']$ counts every output above ε' in full, ignoring the factor $(1 - e^{\varepsilon' - L})$ that discounts outputs just above it. Together these make the guarantee far more conservative than it says: at $k = 100$, $\varepsilon = 0.1$, the exact δ at the advanced $\varepsilon' = 5.85$ is 1.96×10^{-9}, not 10^{-5}.

21.4.5. Example: a hundred counts at $\varepsilon = 0.1$

Take $k = 100$, $\varepsilon = 0.1$ and $\delta' = 10^{-5}$. The Azuma term is $0.1\sqrt{200 \ln 10^5} = 0.1\sqrt{2302.6} = 4.7985$, and the mean term is $100 \times 0.1(e^{0.1} - 1) = 1.0517$, so $\varepsilon' = 5.8502$. With the tanh mean term, $100 \times 0.1 \tanh(0.05) = 0.4996$ and $\varepsilon' = 4.7985 + 0.4996 = 5.2981$. Basic composition gives 10, and the optimal answer of the next section is 4.3068. Table 21.3 compares the bounds over k, and Figure 21.2 plots them.

Table 21.3.: Total ε' at $\delta' = 10^{-5}$ for k mechanisms that are each 0.1-DP.

k	Basic $k\varepsilon$	Advanced, Equation 21.7	Advanced, Equation 21.8	Optimal
10	1.000	1.623	1.567	0.994
100	10.000	5.850	5.298	4.307
1,000	100.000	25.691	20.170	17.787
10,000	1,000.000	153.156	97.944	91.691

At $k = 10$, both advanced bounds are worse than basic composition, because $k = 10$ is below the crossover at 29; the optimal bound is slightly better, 0.994 against 1. At $k = 10{,}000$ the order of the terms flips. The Azuma term is $0.1\sqrt{20{,}000 \ln 10^5} = 48$, while the cruder mean term $k\varepsilon(e^\varepsilon - 1) = 1{,}000 \times 0.105$ costs 105 of the 153, against $1{,}000 \tanh(0.05) = 50$ for the tanh form. There the mean term matters more than the Azuma term, as the previous paragraph predicts: $\varepsilon\sqrt{k} = 10$ is well above 1.

21.4.6. Check yourself

- Why does Azuma apply in step 5 even though the mechanism at step i depends on earlier outputs? *(Azuma needs only that each centred increment has conditional mean zero and lies in an interval of fixed length given the past. Both hold for every history.)*
- For fixed ε, does advanced composition always beat basic composition for large enough k? *(Only if $e^\varepsilon - 1 < 1$, that is $\varepsilon < \ln 2$. Otherwise the mean term $k\varepsilon(e^\varepsilon - 1)$ alone is at least $k\varepsilon$. The* tanh *form beats basic composition for every ε, since* tanh < 1 *leaves room for the $\sqrt{k}$ term once k is large.)*

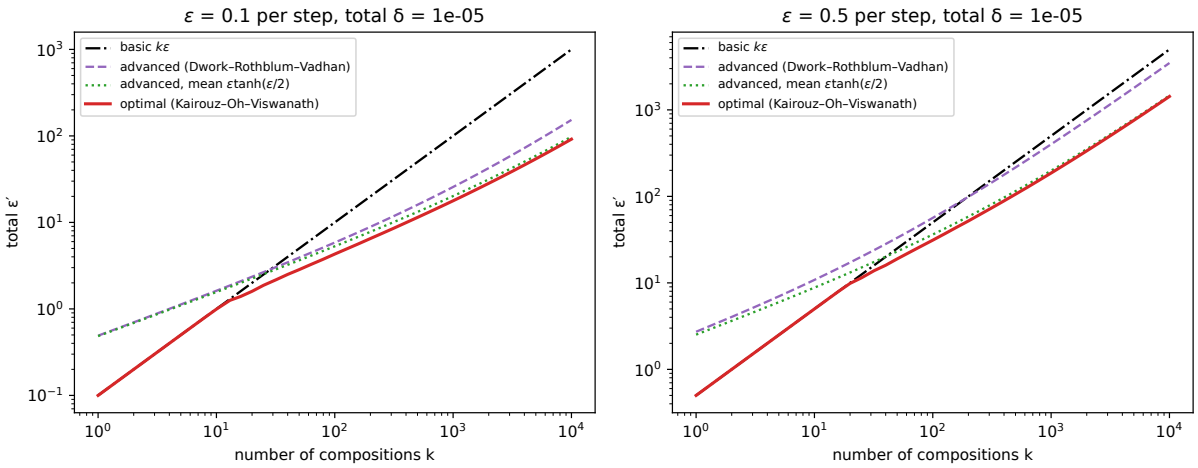


Figure 21.2.: Total ε' at $\delta = 10^{-5}$ after k compositions of ε-DP mechanisms, for $\varepsilon = 0.1$ (left) and $\varepsilon = 0.5$ (right), on log axes. Basic composition grows like k; advanced and optimal composition grow like $\sqrt{k}$ until the mean term takes over. The optimal curve is never above any other.

21.5. Optimal composition

21.5.1. Context

Advanced composition is one bound among many you could derive. Which is the best? Fix k, ε and δ, and ask for the smallest $\delta(\varepsilon')$ that is guaranteed for every composition of k mechanisms that are each (ε, δ)-DP. Kairouz, Oh & Viswanath (2015) answered this exactly, with a surprisingly simple argument built on **post-processing**: passing an output through any further randomized step that does not look at the data. Post-processing can only make two output distributions harder to tell apart, so it never increases $\delta(\varepsilon')$; this is the data-processing inequality. Kairouz, Oh & Viswanath showed that every (ε, δ)-DP mechanism is a post-processing of one particular mechanism, a version of randomized response. Then every composition of (ε, δ)-DP mechanisms is a post-processing of composed randomized response, and so can be no worse. The worst composition is composed randomized response, whose total loss Equation 21.5 you already know.

21.5.2. Statement

Randomized response is the worst. The worst mechanism needs to spend both parameters. It spends δ by, with probability δ, announcing which dataset it ran on, and ε by otherwise running randomized response at level ε. Its four outputs are listed in Table 21.4; call its two output distributions $P^\star$ (on D) and $Q^\star$ (on D'). A **channel** K is a table of probabilities $K(y \mid z)$: given an input z, it draws an output y. Feeding $P^\star$ through it gives the distribution $P^\star K(y) = \sum_z P^\star(z) K(y \mid z)$, and this is exactly what post-processing does. The claim is that every pair (P, Q) that satisfies (ε, δ)-DP in both directions can be produced this way: there is a channel K with $P = P^\star K$ and $Q = Q^\star K$.

For $\delta = 0$ the channel is explicit. Plain randomized response outputs 0, the report that points to D, with probability t on D and $1-t$ on D'. So the channel must satisfy, for every output y of the target pair, $p(y) = tK(y \mid 0) + (1-t)K(y \mid 1)$ and $q(y) = (1-t)K(y \mid 0) + tK(y \mid 1)$. These are two linear equations in the two unknowns $K(y \mid 0)$ and $K(y \mid 1)$. Solving them, and using $t/(1-t) = e^{\varepsilon}$, gives

$$K(y \mid 0) = \frac{e^{\varepsilon} p(y) - q(y)}{e^{\varepsilon} - 1}, \qquad K(y \mid 1) = \frac{e^{\varepsilon} q(y) - p(y)}{e^{\varepsilon} - 1}. \tag{21.10}$$

The input that points to D spreads its mass over the outputs in proportion to how much P favours them, and the other input does the same for Q.

Table 21.4.: The (ε, δ) randomized response of Kairouz, Oh & Viswanath, with $t = e^{\varepsilon}/(1+e^{\varepsilon})$. With probability δ it announces the dataset outright; otherwise it is randomized response at level ε.

Output	"flag D"	0	1	"flag D'"
$P^\star$ (on D)	δ	$(1-\delta)t$	$(1-\delta)(1-t)$	0
$Q^\star$ (on D')	0	$(1-\delta)(1-t)$	$(1-\delta)t$	δ
Privacy loss	$+\infty$	$+\varepsilon$	$-\varepsilon$	$-\infty$

Optimal composition. Compose k copies of the four-point pair. Either some step announces the dataset, and then the output is impossible under D' and counts in full towards $\delta(\varepsilon')$, or no step does, and then the run is k rounds of randomized response, whose loss is binomial by Equation 21.5. Fact 1 turns that binomial loss into a δ, which we call $\delta_{\mathrm{RR}}(\varepsilon')$: the average of $(1 - e^{\varepsilon' - L})_+$ over the $k+1$ possible values of J. For every composition of k mechanisms that are each (ε, δ)-DP, adaptive or not, and every $\varepsilon' \geq 0$,

$$\delta(\varepsilon') \leq 1 - (1-\delta)^k (1 - \delta_{\mathrm{RR}}(\varepsilon')), \qquad \delta_{\mathrm{RR}}(\varepsilon') = \sum_{j=0}^{k} \binom{k}{j} t^{k-j} (1-t)^j \left(1 - e^{\varepsilon' - (k-2j)\varepsilon}\right)_+, \tag{21.11}$$

with equality for k independent copies of the four-point pair. The binomial weight $\binom{k}{j} t^{k-j}(1-t)^j$ is the probability that exactly j of the k reports point to D'. At the points where ε' equals one of the values the loss can take, $\varepsilon' = (k-2i)\varepsilon$ for $i = 0, \ldots, \lfloor k/2 \rfloor$, the positive part can be dropped and the sum becomes a ratio of exponentials. This is the formula of Kairouz, Oh & Viswanath's main theorem:

$$\delta_{\mathrm{RR}}((k-2i)\varepsilon) = \frac{\sum_{\ell=0}^{i-1} \binom{k}{\ell} \left(e^{(k-\ell)\varepsilon} - e^{(k-2i+\ell)\varepsilon}\right)}{(1+e^{\varepsilon})^k}. \tag{21.12}$$

The sharp mean loss. The KL divergence, like $\delta(\varepsilon')$, can only shrink under post-processing. Since every ε-DP pair is a post-processing of randomized response, its mean loss is at most that of randomized response:

$$\mathbb{E}[L] = D_{\mathrm{KL}}(P \| Q) \leq \varepsilon \tanh(\varepsilon/2), \tag{21.13}$$

with equality for randomized response.

21.5.3. Derivation

The channel for $\delta = 0$. The formula came from solving two equations; here we check that it is a valid channel and that it does what it should. Recall that randomized response outputs 0 with probability t on D and $1 - t$ on D' (here 0 is the report that points to D).

1. *The rows are distributions.* A channel's rows must be non-negative and sum to 1. $K(y \mid 0) \geq 0$ exactly when $e^{\varepsilon} p(y) \geq q(y)$, that is $p(y) \geq e^{-\varepsilon} q(y)$, and $K(y \mid 1) \geq 0$ exactly when $p(y) \leq e^{\varepsilon} q(y)$. Both hold for every y because the pair is ε-DP: no output is more than e^{ε} times likelier under one side than the other. Since p and q each sum to 1, each row sums to $(e^{\varepsilon} - 1)/(e^{\varepsilon} - 1) = 1$.
2. *They reproduce P.* Mixing the rows with randomized response's probabilities on D gives $t\,K(y \mid 0) + (1-t)K(y \mid 1) = \frac{(te^{\varepsilon} - (1-t))p + ((1-t)e^{\varepsilon} - t)q}{e^{\varepsilon} - 1}$. With $t = e^{\varepsilon}/(1 + e^{\varepsilon})$, $(1-t)e^{\varepsilon} = t$, so the coefficient of q is 0 and the coefficient of p is $\frac{te^{\varepsilon} - (1-t)}{e^{\varepsilon} - 1} = \frac{(e^{2\varepsilon} - 1)/(1 + e^{\varepsilon})}{e^{\varepsilon} - 1} = 1$, since $e^{2\varepsilon} - 1 = (e^{\varepsilon} - 1)(e^{\varepsilon} + 1)$.
3. *They reproduce Q.* The same computation with t and $1 - t$ swapped.

The channel for $\delta > 0$. Here the construction is not a one-line formula, and we cite it, after explaining why it must exist. Think of telling P from Q as a hypothesis test. The **trade-off function** of a pair records, for each false positive rate α, the smallest false negative rate any test can reach: the higher the curve, the harder the pair is to test. The definition of differential privacy shows that a pair is (ε, δ)-DP exactly when its trade-off function lies above a fixed boundary $f_{\varepsilon,\delta}$, made of two straight lines that come from applying the DP inequality to a test's rejection set and to its complement. The four-point pair sits exactly on that boundary: its likelihood-ratio tests have errors at the corners of the boundary, and mixing them traces the lines. So every (ε, δ)-DP pair is at least as hard to test as $(P^{\star}, Q^{\star})$, at every level. Blackwell's theorem (1953) says that one pair is at least as hard to test as another at every level exactly when it is a post-processing of the other. Kairouz, Oh & Viswanath (2015) construct the channel explicitly.

Composition. The argument uses the data-processing inequality for the hockey-stick divergence (Divergences): post-processing never increases $\delta(\varepsilon')$. Here is why. Take any set S of final outputs. After the channel, $P^{\star}K(S) - e^{\varepsilon'} Q^{\star}K(S) = \sum_z \phi(z)(P^{\star}(z) - e^{\varepsilon'} Q^{\star}(z))$ with $\phi(z) = K(S \mid z) \in [0, 1]$. Each term is at most $\left(P^{\star}(z) - e^{\varepsilon'} Q^{\star}(z)\right)_+$, so the gap after the channel is at most the hockey-stick sum before it.

1. *Non-adaptive.* If step i is $P_i = P^{\star}K_i$ and $Q_i = Q^{\star}K_i$, then the product $P_1 \times \cdots \times P_k$ is $(P^{\star})^k$ followed by the channel that applies K_i to coordinate i, independently. So $\delta(\varepsilon')$ of the composition is at most that of $(P^{\star})^k$ against $(Q^{\star})^k$.
2. *Adaptive.* Now the mechanism of step i, and so its channel, may depend on the earlier outputs: $K_i^{y_{<i}}$. The trick is to draw all the randomized-response inputs first and apply the channels afterwards. Draw $z_1, \ldots, z_k$ from $(P^{\star})^k$, then $y_1 \sim K_1(\cdot \mid z_1)$, then $y_2 \sim K_2^{y_1}(\cdot \mid z_2)$, and so on. Given $y_{<i}$, z_i is still a fresh draw from $P^{\star}$, because it was drawn independently of $z_{<i}$ and $y_{<i}$ depends only on those. So y_i has law $P^{\star}K_i^{y_{<i}} = P_i(\cdot \mid y_{<i})$, and the sequence has exactly the law of the adaptive run on D. The same sampler fed with $(Q^{\star})^k$ produces the run on D'. So the adaptive run is still one post-processing of $(P^{\star})^k$ against $(Q^{\star})^k$.
3. *The four-point product.* Each coordinate of $(P^{\star})^k$ avoids "flag D" with probability $1 - \delta$, independently, so with probability $1 - (1 - \delta)^k$ some coordinate shows "flag D". $(Q^{\star})^k$ never produces that output, so there $L = +\infty$, the bracket $(1 - e^{\varepsilon' - L})_+$ equals 1, and every

such output contributes its full probability to Equation 21.2. Outputs with "flag D'" have probability 0 under $(P^\star)^k$ and contribute nothing. Otherwise, with probability $(1-\delta)^k$, all coordinates are plain randomized response. On these outputs both $P^\star$ and $Q^\star$ carry the same factor $(1-\delta)$ per coordinate, which cancels in the ratio, so $L = (k-2J)\varepsilon$ by Equation 21.5. Fact 1 on this part gives $(1-\delta)^k\delta_{\mathrm{RR}}(\varepsilon')$. Adding the two parts, $1-(1-\delta)^k+(1-\delta)^k\delta_{\mathrm{RR}}(\varepsilon')$, which is Equation 21.11.

4. *Equality.* The four-point pair is itself a mechanism, on a dataset of one bit, and it is (ε,δ)-DP. Composing k copies of it gives equality. No better bound can hold for the whole class.
5. *The grid formula.* At $\varepsilon' = (k-2i)\varepsilon$, the terms of δ_{RR} with $j \geq i$ vanish: their loss $(k-2j)\varepsilon$ is at most ε', so the bracket is 0. For $j = \ell < i$, the bracket is positive and the $(\cdot)_+$ can be dropped. Since $t = e^\varepsilon/(1+e^\varepsilon)$ and $1-t = 1/(1+e^\varepsilon)$, the weight is $\binom{k}{\ell}t^{k-\ell}(1-t)^\ell = \binom{k}{\ell}e^{(k-\ell)\varepsilon}/(1+e^\varepsilon)^k$. Multiplying it by $e^{\varepsilon'-(k-2\ell)\varepsilon} = e^{(2\ell-2i)\varepsilon}$ turns $e^{(k-\ell)\varepsilon}$ into $e^{(k-2i+\ell)\varepsilon}$. This is Equation 21.12.

The sharp mean loss. KL divergence also obeys the data-processing inequality: post-processing cannot increase it, for the same reason as above, since passing two distributions through the same channel can only make them more alike. An ε-DP pair is a post-processing of randomized response, so its KL is at most that of randomized response, $\varepsilon t - \varepsilon(1-t) = \varepsilon\tanh(\varepsilon/2)$ (step 3 of the derivation in Section 21.3 with $k=1$). That gives Equation 21.13, and with it Equation 21.8.

Advanced composition with $\delta > 0$. By step 2 it suffices to bound the four-point product. A flag shows with probability $1-(1-\delta)^k$, which is at most $k\delta$ because the chance that at least one of k events happens is at most the sum of their chances. Otherwise the run is k rounds of randomized response, a pure ε-DP composition, whose δ at the advanced ε' is at most δ' by the $\delta = 0$ derivation. So the total is at most $k\delta + \delta'$, which completes Equation 21.7.

21.5.4. Interpretation

"Optimal" means optimal for the class. Equation 21.11 is the best you can say about k mechanisms *if all you know about each is its* (ε,δ). Randomized response attains it. A Laplace mechanism with the same ε does a little better, because its loss is not always at the extremes. For a count with Laplace noise of scale $1/\varepsilon$, the loss is $\pm\varepsilon$ when the noisy answer lands outside the interval between the two true counts, and strictly inside $(-\varepsilon,\varepsilon)$ when it lands within it, which happens with probability $\frac{1}{2}(1-e^{-\varepsilon})$. Those middle outputs carry less evidence, so the mean loss is smaller: at $\varepsilon = 1$ it is 0.3679, against 0.4621.

A single (ε,δ) throws information away. A Gaussian mechanism has a whole curve $\delta(\varepsilon)$: for each ε there is a smallest δ, and no single pair describes it. Summarizing each step by one point of that curve and composing with Equation 21.11 is wasteful, because the summary forgets how the curve continues. The exact composition is easy here. The privacy loss of one Gaussian step with shift $1/\sigma$ is itself normal, and a sum of independent normal losses is normal, so k steps have exactly the loss of a single Gaussian step with shift $\sqrt{k}/\sigma$. Figure 21.4 shows how much the point summary costs: with noise $\sigma = 20$ on a sensitivity-1 query, $k = 1{,}000$ steps are exactly $(7.51, 10^{-5})$-DP, but the best per-step split of δ followed by optimal composition gives 62.7. Composing the curves themselves, instead of points on them, is the subject of Privacy accounting.

Computing the bound needs care. Equation 21.12 as printed divides two numbers that overflow: at $\varepsilon = 1$ and $k = 1{,}000$, $(1+e)^{1000} \approx 10^{570}$, beyond the largest double, which is about 10^{308}. The script works with logarithms of the binomial probabilities and adds the non-negative terms of

Equation 21.11 with a log-sum-exp: it subtracts the largest logarithm before exponentiating, so nothing overflows, and since every term is non-negative nothing cancels. To find the optimal ε' for a target δ, note that between two grid points the set of outputs whose loss exceeds ε' does not change. On that piece $\delta_{\mathrm{RR}}(\varepsilon') = A - e^{\varepsilon'} B$, where A and B are the P- and Q-masses of the outputs with loss above ε', and you can solve that exactly on the right piece.

Different ε_i per step are harder. The same argument shows that composed randomized response at levels $\varepsilon_1, \dots, \varepsilon_k$ is the worst case, but its loss is no longer a multiple of a binomial: it depends on which subset of steps lied, not just how many. Computing the optimal bound exactly is then #P-complete (Murtagh & Vadhan, 2016), the complexity class of hard counting problems, so no efficient exact algorithm is expected. For a few steps you can still enumerate (Exercise 7 in the exercise book).

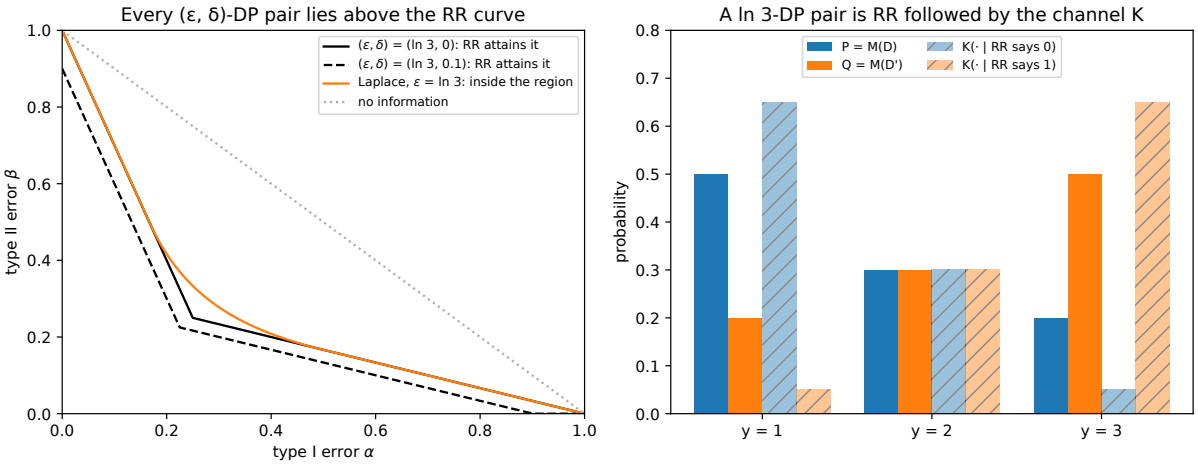


Figure 21.3.: Left: the trade-off curves $f_{\varepsilon,\delta}$ at $\varepsilon = \ln 3$ with $\delta = 0$ and $\delta = 0.1$, attained by randomized response, and the trade-off curve of a Laplace mechanism with the same ε, which lies inside the allowed region. Right: a $\ln 3$-DP pair on three outputs, $P = (0.5, 0.3, 0.2)$ and $Q = (0.2, 0.3, 0.5)$, and the two rows of the channel K of Equation 21.10 that produce it from randomized response.

21.5.5. Example: the channel, the hundred counts, and the checks

A channel by hand. Take $P = (0.5, 0.3, 0.2)$ and $Q = (0.2, 0.3, 0.5)$. The largest loss is $\ln(0.5/0.2) = \ln 2.5 = 0.9163$, below $\ln 3 = 1.0986$, so the pair is $\ln 3$-DP. With $e^{\varepsilon} = 3$, Equation 21.10 gives $K(\cdot \mid 0) = (3P - Q)/2 = (0.65, 0.30, 0.05)$ and $K(\cdot \mid 1) = (3Q - P)/2 = (0.05, 0.30, 0.65)$ (Figure 21.3, right); for the first output, $(3 \times 0.5 - 0.2)/2 = 0.65$. Check that the channel reproduces P on the first output: randomized response at $\ln 3$ has $t = \frac{3}{4}$, and $\frac{3}{4} \times 0.65 + \frac{1}{4} \times 0.05 = 0.5$. The data-processing inequality then says that every divergence of the pair is at most that of randomized response at $\ln 3$. Its KL is $0.5 \ln 2.5 + 0.3 \ln 1 + 0.2 \ln 0.4 = 0.2749$, against $\ln 3 \tanh(\frac{\ln 3}{2}) = \ln 3 \times \frac{1}{2} = 0.5493$. Its $\delta(0)$, the total variation, is $0.5 - 0.2 = 0.3$, against 0.5.

The hundred counts. For $k = 100$ and $\varepsilon = 0.1$, the optimal ε' at $\delta' = 10^{-5}$ is 4.3068. That is 19% below the improved advanced bound ($4.3068/5.2981 = 0.81$), 26% below the original one ($4.3068/5.8502 = 0.74$) and 57% below basic composition's 10. If each step also has $\delta = 5 \times 10^{-8}$,

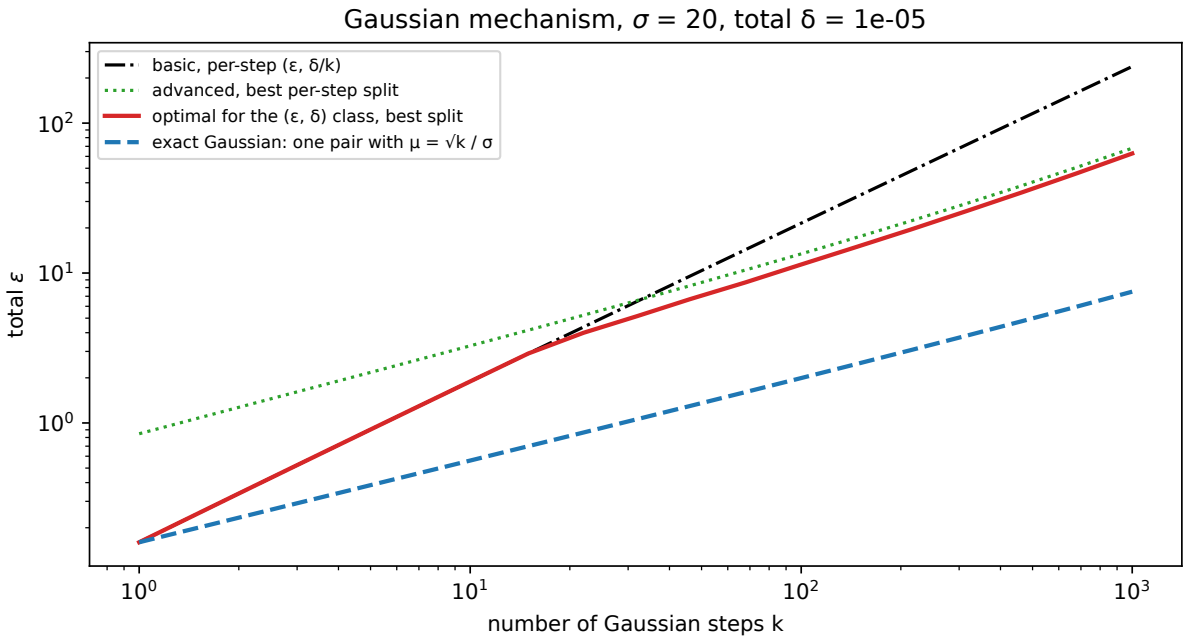


Figure 21.4.: Total ε at $\delta = 10^{-5}$ after k Gaussian mechanisms with $\sigma = 20$ on a query of sensitivity 1. The first three curves first convert each step to one (ε, δ) point, with the best split of δ, then compose with basic, advanced or optimal composition. The dashed curve composes the exact Gaussian curves, which is a single Gaussian pair with shift $\sqrt{k}/\sigma$.

the flags use up $1-(1-\delta)^{100} = 5.0 \times 10^{-6}$, about $100 \times 5 \times 10^{-8}$, of a total $\delta = 10^{-5}$, and the optimal ε' is 4.4469. Advanced composition with the remaining $\delta' = 5 \times 10^{-6}$ gives 5.9926.

Checks. The script compares Equation 21.11 with brute force: it lists every output sequence of the k-fold product and sums $(p - e^{\varepsilon'} q)_+$ directly (Table 21.5). With four outputs per step there are $4^6 = 4{,}096$ sequences for $k = 6$, and with two outputs (when $\delta = 0$) there are $2^{12} = 4{,}096$ for $k = 12$.

Table 21.5.: The log-space evaluation of Equation 21.11 against brute-force enumeration of the composed four-point pair, at several values of ε'.

Case	Sequences enumerated	Largest difference from Equation 21.11
$\varepsilon = 0.5$, $\delta = 0$, $k = 12$	4,096	3.9×10^{-16}
$\varepsilon = 0.3$, $\delta = 10^{-3}$, $k = 6$	4,096	1.1×10^{-16}
$\varepsilon = 1$, $\delta = 0.01$, $k = 5$	1,024	1.1×10^{-16}
$\varepsilon = \ln 3$, $\delta = 0$, $k = 8$, exact fractions	256	2.2×10^{-16}

The differences are at the level of floating-point rounding, about 10^{-16}. With $e^{\varepsilon} = 3$ the probabilities are fractions ($t = \frac{3}{4}$), and enumeration gives the exact value $\delta(0) = 1759/2048$ for $k = 8$. At $\varepsilon = 1$ and $k = 1{,}000$, the literal grid formula raises `OverflowError`, while the log-space sum gives $\delta(550) = 6.094 \times 10^{-4}$.

21.5.6. Check yourself

- Why does the composition of k four-point pairs have $\delta(\varepsilon') \geq 1 - (1-\delta)^k$ for every ε', however large? *(With that probability some step announces the dataset outright. Then the loss is infinite, and no finite ε' covers it.)*
- Can a mechanism that is (ε, δ)-DP have a composition with a worse $\delta(\varepsilon')$ than Equation 21.11? *(No. Every such mechanism is a post-processing of the four-point pair, and post-processing never increases $\delta(\varepsilon')$.)*

21.6. Adaptive composition

21.6.1. Context

In practice the next query depends on the last answer. An analyst looks at a noisy histogram and then zooms into its largest bin. A training loop computes the next gradient at parameters that came from earlier noisy gradients. A hyperparameter search picks the next configuration based on earlier noisy scores. Each step is private, but the *choice* of step depends on earlier outputs. Does composition still hold? The derivations above already said yes. This section makes the model explicit and says where adaptivity does break things.

21.6.2. Statement

The k-fold adaptive composition experiment (Dwork, Rothblum & Vadhan, 2010). An adversary interacts with one of two neighbouring datasets, D or D', for k rounds. In round i it chooses a mechanism M_i from a class of (ε, δ)-DP mechanisms, as any function of the outputs $y_1, \dots, y_{i-1}$ so far, and receives $y_i \sim M_i(D)$ or $M_i(D')$. Its view is $V = (y_1, \dots, y_k)$.

Every guarantee of this chapter holds for this experiment. For every adversary, the views on D and on D' satisfy each of the following:

- basic composition: $(k\varepsilon, k\delta)$-DP;
- advanced composition: $(\varepsilon', k\delta + \delta')$-DP with ε' from Equation 21.7 or Equation 21.8;
- optimal composition: Equation 21.11.

The parameters k, ε and δ must be fixed before the interaction starts. Only the choice of mechanisms may adapt.

21.6.3. Derivation

Each theorem was proved in a form that covers adaptivity. The reason is the same each time: every argument looked at one step at a time, given the history so far, and once the history is fixed, so is the adversary's choice.

1. **The loss of the view is a sum** (Fact 2), with each term $L_i(y_i \mid y_{<i})$ computed for the mechanism chosen after $y_{<i}$. The adversary's choice is a function of $y_{<i}$, so it is fixed once the history is, and the chain rule of Fact 2 applies unchanged.
2. **Basic composition.** For every history, step i is (ε, δ)-DP. For $\delta = 0$, $|L_i| \le \varepsilon$ pointwise, so $|L| \le k\varepsilon$. For $\delta > 0$, use the reduction of step 4 below: the run is a post-processing of k four-point pairs, a flag shows with probability at most $k\delta$, and otherwise $|L| \le k\varepsilon$, so $\delta(k\varepsilon) \le k\delta$.
3. **Advanced composition.** Azuma's inequality needs only conditional statements, given the past: a conditional mean of zero and a range of fixed length. That is the setting of steps 2–5 of the advanced-composition derivation.
4. **Optimal composition.** Step 2 of the composition derivation in Section 21.5 builds the adaptive run from the four-point product with history-dependent channels.

21.6.4. Interpretation: what adaptivity does not allow

Adaptivity does not allow choosing the privacy parameters on the fly. If the analyst picks ε_i after seeing earlier outputs, the per-step bound in step 2 of the advanced-composition derivation depends on the history, and so does the range length $d_i = 2\varepsilon_i$. Azuma's inequality needs the interval lengths d_i to be fixed in advance, so the theorem as stated no longer applies. A rule such as "keep querying until the budget is spent", with the budget computed from the realized ε_i, needs its own theorem: a **privacy filter** (Rogers, Roth, Ullman & Vadhan, 2016). Reporting the privacy spent so far, valid at every moment, needs a **privacy odometer**, and costs more. Both are treated in Adaptive composition. For pure DP with basic composition the naive rule is fine: each $|L_i| \le \varepsilon_i$ pointwise, so the bound $|L| \le \sum_i \varepsilon_i$ holds whatever order the ε_i came in, and no probability argument is involved.

Nor does it allow choices that depend on the data rather than the outputs. The adversary may use the outputs, which are public, because anything computed from them is post-processing. If the decision to stop, or the choice of the next query, looks at the raw data, it is no longer post-processing, and the decision leaks on its own. For example, a system that answers private counts *until the true count of some group crosses a threshold* reveals that count through the number of answers it gives. Every answer can be ε-DP and the whole run still not be DP for any finite ε.

21.6.5. Example: an adaptive analyst

The script simulates an analyst who, at each of $k = 100$ steps, chooses the next mechanism from the last output. If the previous output pointed to D, meaning its privacy loss was positive (a function of that output, so the analyst can compute it), the analyst asks a Laplace query at $\varepsilon = 0.1$; otherwise a randomized-response query at $\varepsilon = 0.1$. The choice depends on earlier outputs, so the steps are not independent. Over 100,000 simulated runs on D, Table 21.6 estimates $\delta(2)$ through Equation 21.2, by averaging $(1 - e^{2-L})_+$ over the simulated total losses L. The exact Laplace mean is 100 times the per-step KL divergence of two Laplace densities whose centres are ε scale units apart, $e^{-\varepsilon} + \varepsilon - 1$.

Table 21.6.: The privacy loss of 100 steps of 0.1-DP mechanisms, composed non-adaptively and by an adaptive analyst.

Composition of 100 steps at $\varepsilon = 0.1$	Mean total loss	$\delta(\varepsilon' = 2)$
Randomized response, exact (Equation 21.11)	0.4996	0.0201
Randomized response, simulated	—	0.0198
Laplace, simulated	0.4838 (exact 0.4837)	0.0189
Adaptive mix, simulated	0.4971	0.0196

The adaptive analyst's $\delta(2)$ lies between the two non-adaptive compositions, and below the optimal bound, as the theorem requires: each of its steps is one of the two 0.1-DP mechanisms, and the theorem covers every way of choosing among them. The Laplace composition has the smallest loss because its per-step loss is sometimes strictly between $-\varepsilon$ and ε, so it carries less evidence on average. Figure 21.5 shows the whole curves: the three compositions nearly coincide, and the advanced-composition points lie far to the right of them.

21.6.6. Check yourself

- An analyst chooses, after seeing each answer, whether the next query asks about men or about women. Each query is 0.1-DP. Do the theorems of this chapter cover the run? *(Yes. The choice of mechanism depends only on earlier outputs, and each choice is* 0.1*-DP for every history.)*

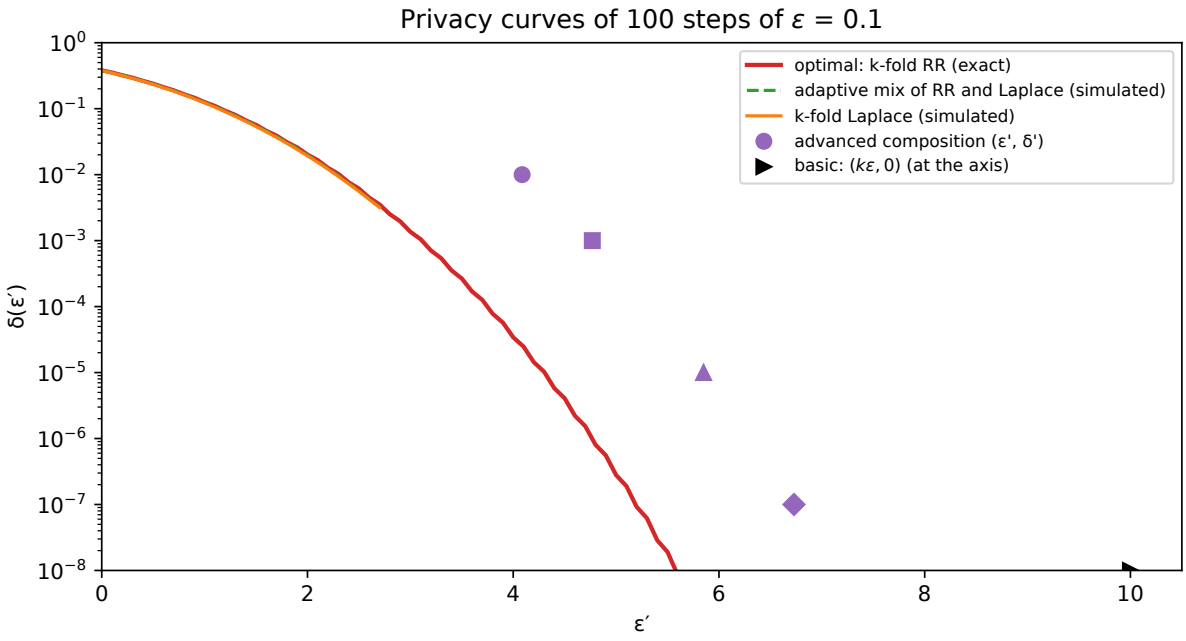


Figure 21.5.: The privacy curve $\delta(\varepsilon')$ of 100 steps at $\varepsilon = 0.1$: the exact curve of composed randomized response (optimal composition), simulated curves for composed Laplace mechanisms and for the adaptive analyst (shown where the simulation has enough samples), the advanced-composition pairs (ε', δ') for $\delta' = 10^{-2}, 10^{-3}, 10^{-5}, 10^{-7}$, and basic composition at $(10, 0)$.

- A system stops answering once the noisy answers have moved by less than 1% for three rounds. Is the number of answers it gives covered? *(Yes, if the stopping rule looks only at the noisy answers: the stopping time is then a post-processing of the outputs. If it looked at the true answers, it would not be.)*

21.7. Code: check every composition bound

The script code/composition.py computes every number in this chapter and draws every figure. The composition bounds are small typed functions. The optimal bound is evaluated in log space and checked against brute-force enumeration, in floating point and in exact fractions. Monte Carlo simulations of Laplace and adaptive compositions are compared with the exact bound. From the repository root:

```
uv run python code/part03-dp-foundations/composition.py
```

Output (seed 0), summarized in Table 21.7:

Table 21.7.: The script's output with the default seed.

Experiment	Result
One step, $\varepsilon = 0.1$	$t = 0.5250$; mean loss $\varepsilon \tanh(\varepsilon/2) = 0.004996$; bound $\varepsilon(e^{\varepsilon} - 1) = 0.010517$
One step, $\varepsilon = 1$	tanh form 0.4621; $\varepsilon(e^{\varepsilon} - 1) = 1.7183$; $\varepsilon^2/2 = 0.5$; Laplace mean loss 0.3679
Three-output pair	Largest loss 0.9163; channel rows $(0.65, 0.3, 0.05)$ and $(0.05, 0.3, 0.65)$; KL 0.2749 against 0.5493; $\delta(0)$ 0.3 against 0.5
100 rounds of randomized response, $\varepsilon = 0.1$	Mean 0.4996; standard deviation 0.9988; $k\varepsilon$ is 9.5 standard deviations above the mean; $t^k = 1.033 \times 10^{-28}$
100 steps, $\varepsilon = 0.1$, $\delta' = 10^{-5}$	Basic 10; advanced 5.8502; advanced (tanh) 5.2981; optimal 4.3068
Exact δ at the advanced ε'	1.956×10^{-9} at 5.8502; 6.249×10^{-8} at 5.2981
Crossover	Advanced beats basic from $k = 29$
Brute-force checks	Agreement to 4×10^{-16} (Table 21.5)
$\varepsilon = 0.01$, $k = 100{,}000$, $\delta' = 10^{-6}$	Optimal 19.4228; advanced (tanh) 21.6225
Gaussian, $\sigma = 5$, $k = 100$, $\delta = 10^{-5}$	Basic 93.178; advanced 89.843; optimal 76.223; exact 9.997
Gaussian, $\sigma = 20$, $k = 1{,}000$, $\delta = 10^{-5}$	Basic 238.578; advanced 68.008; optimal 62.720; exact 7.511

tests/test_composition.py checks that the output still matches.

Three things stand out. First, the exact formula is cheap and safe. In log space, Equation 21.11 costs $O(k)$ operations, one per value of J, agrees with enumeration to rounding error, and never

overflows. So there is no reason to use a looser bound when all you know is each step's (ε, δ). Second, advanced composition is a good approximation for small ε and large k, where the total loss is close to normal and the mean term is small. At $\varepsilon = 0.01$ and $k = 100{,}000$ the tanh form is 11% above optimal ($21.6225/19.4228 = 1.11$). At $k = 100$, $\varepsilon = 0.1$ it is 23% above ($5.2981/4.3068 = 1.23$). Third, the class bound is not the mechanism's bound. For Gaussian noise, every route through per-step (ε, δ) is off by a factor of about 8 ($76.223/9.997 = 7.6$ and $62.720/7.511 = 8.4$ in the two cases), because each route forgets the shape of the Gaussian curve. That gap motivates the accountants of the next part of the book.

21.8. References

- Dwork, Rothblum & Vadhan, "Boosting and Differential Privacy" (FOCS 2010). Introduces the adaptive composition experiment and advanced composition, Equation 21.7, with the bound $\varepsilon(e^{\varepsilon} - 1)$ on the mean loss.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.5: basic and advanced composition with full proofs (Theorem 3.20), including the $\delta > 0$ case through a decomposition of approximate DP pairs.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015; IEEE Transactions on Information Theory, 2017). The hypothesis-testing view, the four-point randomized response, the optimal composition formula Equation 21.12, and improved closed-form bounds of the advanced kind.
- Blackwell, "Equivalent Comparisons of Experiments" (Annals of Mathematical Statistics, 1953). The theorem that one experiment is uniformly harder to test than another exactly when it is a post-processing of it.
- Murtagh & Vadhan, "The Complexity of Computing the Optimal Composition of Differential Privacy" (TCC 2016). Optimal composition with different ε_i is #P-complete, with an efficient approximation.
- Vadhan, "The Complexity of Differential Privacy" (2017): composition theorems in the order used here, with the privacy loss as the central object.
- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016). Why adaptively chosen privacy parameters need new theorems, and the first filters and odometers.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS Series B, 2022). Composition of whole trade-off curves, and the central limit theorem for composition that the loss distribution of Section 21.3 foreshadows.

21.9. Cheat sheet

$L = \ln \frac{p_{M(D)}(Y)}{p_{M(D')}(Y)}$ with $Y \sim M(D)$ is the privacy loss random variable, $\delta(\varepsilon) = \mathbb{E}[(1 - e^{\varepsilon - L})_+]$, and randomized response at level ε is truthful with probability $t = e^{\varepsilon}/(1 + e^{\varepsilon})$. Table 21.8 collects the results for k steps that are each (ε, δ)-DP.

Table 21.8.: The composition results of this chapter.

Result	Statement	Holds for
Loss of a composition	$L = \sum_i L_i(y_i \mid y_{<i})$	Any sequence, adaptive or not
Mean loss of an ε-DP step	$\mathbb{E}[L] \le \varepsilon \tanh(\varepsilon/2) \le \min\{\varepsilon^2/2,\ \varepsilon(e^\varepsilon - 1)\}$	Equality for randomized response
Basic composition	$(k\varepsilon, k\delta)$	Adaptive
Advanced composition	$(\varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^\varepsilon - 1),\ k\delta + \delta')$; $\tanh(\varepsilon/2)$ in place of $e^\varepsilon - 1$ is also valid	Adaptive, parameters fixed in advance
Worst mechanism	The four-point randomized response of Table 21.4; every (ε, δ) pair is a post-processing of it	—
Optimal composition	$\delta(\varepsilon') = 1-(1-\delta)^k(1-\delta_{\mathrm{RR}}(\varepsilon'))$, Equation 21.11	Adaptive; tight for the class
Randomized response, k rounds	$L = (k-2J)\varepsilon$, $J \sim \mathrm{Bin}(k, 1-t)$	—

Rules of thumb

- Under composition the privacy loss adds up; its mean grows like $k\varepsilon^2/2$ and its spread like $\sqrt{k}\,\varepsilon$.
- Advanced composition helps once $k \gtrsim 2\ln(1/\delta')$, about 23 at $\delta' = 10^{-5}$; budget $\varepsilon \approx \varepsilon_{\mathrm{total}}/\sqrt{2k\ln(1/\delta')}$ per step.
- If all you know is each step's (ε, δ), compute the optimal bound exactly in log space; it costs nothing.
- If you know the mechanism, compose its whole privacy curve instead: the per-step (ε, δ) summary can be off by a factor of 8.
- Adaptive choice of queries is free; adaptive choice of privacy parameters, or stopping rules that look at the raw data, are not.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

22. Selection: exponential mechanism, report-noisy-max, sparse vector

22.1. Why this matters

Many questions about a dataset ask you to **pick** something rather than to compute a number. Which of 50 hyperparameter settings scores best on the validation set? Which word is most frequent in a corpus? Which split point gives the best decision stump? Which of a thousand monitored statistics is the first to cross an alarm level? The answer is a name from a list, and adding noise to it makes no sense: "model 7 plus Laplace noise" is not a model.

The Laplace and Gaussian mechanisms, the basic tools of differential privacy (DP), do not help directly. They add noise to numbers. You could use them to release a noisy score for every candidate and then take the largest, but the noise has to be scaled to how far one record can move the whole released vector. That distance is the ℓ_1 **sensitivity**: the largest total change, summed over all coordinates, that replacing, adding or removing one record can cause. If one record can move each of k scores by up to 1, it can move the vector by up to k in total, so the Laplace mechanism needs k times more noise than for a single score. Put differently, releasing k noisy scores is k releases, and under basic composition (Post-processing, group privacy and composition), which says that the privacy costs of several releases add up, the cost grows with the number of candidates. That number is often in the thousands.

This chapter shows that you never have to pay for the losers. Three mechanisms, all built from one move, make selection cost as much as answering a single query:

- The **exponential mechanism** samples a candidate with probability that grows exponentially in its score.
- **Report-noisy-max** adds noise to every score and releases only the name of the winner, never the noisy scores.
- The **sparse vector technique** goes through a long stream of queries and reports only which one first exceeds a threshold. Every query that stays below costs nothing.

22.1.1. The one idea: release the position, not the values

In each mechanism the output is a position (which candidate, which query), and the noise is added to scores that are never released. The privacy proof fixes everything except the noise of the candidate in question, and notices that this candidate is chosen exactly when its own noise is large enough to clear a bar set by the others. It then asks one question: how far can one record move that *bar*? The bar is a maximum over the other candidates, and a maximum of numbers that each move by at most Δ moves by at most Δ, however many numbers there are. That is why

the number of candidates k does not appear in the privacy cost. It appears only in the accuracy, and only as $\ln k$.

Table 22.1 lists the results of the chapter and where each is used.

Table 22.1.: The results of this chapter and where each is used.

Result	Where it is used
Exponential mechanism is ε-DP	Private model selection, private medians and quantiles, the multiplicative weights exponential mechanism for synthetic data in later parts
Its utility: gap $\leq \frac{2\Delta}{\varepsilon}(\ln k + t)$ with probability $1 - e^{-t}$	Choosing ε for a selection; the $\ln k$ that lower bounds show is necessary
Report-noisy-max with Laplace noise	Noisy argmax over vote counts, as in private aggregation of teacher ensembles in a later part
Gumbel noise turns report-noisy-max into the exponential mechanism	Sampling the exponential mechanism stably; top-c selection
AboveThreshold is ε-DP for any number of queries below the threshold	Private hyperparameter search, monitoring, the reusable holdout
Buggy variants of the sparse vector technique	Reviewing a privacy proof; what a correct implementation must not change

How to read this chapter. Read the toolkit, then the exponential mechanism and its utility, which are the core. Report-noisy-max and the Gumbel section are two views of the same mechanism and are short. The sparse vector section has the subtlest proof in this part; read its statement and the picture first, then the derivation slowly. The last section, on buggy variants, shows what goes wrong when one line of that proof is skipped.

What this chapter assumes, restated. A **mechanism** M is a randomized algorithm that takes a dataset and returns a random output. Two datasets are **neighbours**, written $D \simeq D'$, when they differ in one person's record: under **add/remove** one of them has one extra record, and under **replace** one record has been swapped for another. M is **ε-differentially private** (ε-DP) if, for every pair of neighbours and every set S of outputs, $\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S]$: no output becomes more than e^{ε} times more likely because one record changed. For a single output y, the **privacy loss** is $\ln\left(\Pr[M(D) = y] / \Pr[M(D') = y]\right)$, and ε-DP says its absolute value is at most ε. All of this is in The definition of differential privacy. Basic composition, from Post-processing, group privacy and composition, says that running an ε_1-DP and an ε_2-DP mechanism on the same data is $(\varepsilon_1 + \varepsilon_2)$-DP. The **Laplace distribution** $\mathrm{Lap}(b)$ with scale b has density $\frac{1}{2b}e^{-|z|/b}$, a two-sided exponential; its upper tail is $\Pr[Z > x] = \frac{1}{2}e^{-x/b}$ for $x \geq 0$, and so $\Pr[|Z| \geq x] = e^{-x/b}$ (Probability distributions, Concentration inequalities). The **Laplace mechanism** of The Laplace and Gaussian mechanisms adds $\mathrm{Lap}(\Delta/\varepsilon)$ noise to a number of sensitivity Δ.

Notation. A selection problem has a finite set $\mathcal{R}$ of k candidates and a **score function** $u(D, r)$: how good candidate r is on dataset D. How much noise a selection needs depends on how far one

record can move a score, so the key quantity is the **sensitivity** of the score, the largest change of any one candidate's score between any two neighbours:

$$\Delta u = \max_{r \in \mathcal{R}} \max_{D \simeq D'} \big|u(D, r) - u(D', r)\big|. \tag{22.1}$$

We write Δ for Δu when the score is clear, and $\mathrm{OPT}(D) = \max_r u(D, r)$ for the best score. Scores are **monotone** if for every pair of neighbours either $u(D', r) \geq u(D, r)$ for all r, or $u(D', r) \leq u(D, r)$ for all r: one record pushes all scores the same way. Counts under add/remove are monotone, because adding a record can only increase each count and removing one can only decrease each count. Logarithms are natural.

22.2. Toolkit: four facts every derivation uses

Every privacy proof in this chapter has the same shape: pick one output, fix all the randomness except one noise variable, see that the output happens exactly when that noise clears a bar, bound how far one record can move the bar, and pay for that move with a Laplace tail. The four facts below are the four pieces of that shape, in order. Each has a one-line proof. The second is the one readers stall on, so it gets a worked case.

22.2.1. Fact 1 — Pure DP can be checked outcome by outcome

The definition of ε-DP asks about every *set* of outputs, and there are far more sets than outputs. When the outputs can be listed (a candidate's name, the index of a query), it is enough to check single outputs. The reason is that the probability of a set is the sum of the probabilities of its outputs, and if each term on the left is at most e^{ε} times the matching term on the right, so is the sum.

If the output set is finite and $\Pr[M(D) = y] \leq e^{\varepsilon} \Pr[M(D') = y]$ for every outcome y and every ordered pair of neighbours, then M is ε-DP.

In symbols, for any set S, $\Pr[M(D) \in S] = \sum_{y \in S} \Pr[M(D) = y] \leq \sum_{y \in S} e^{\varepsilon} \Pr[M(D') = y] = e^{\varepsilon} \Pr[M(D') \in S]$. This is Fact 1 of The definition of differential privacy, which says the same thing from the other side: the worst set for a ratio is a single outcome. Every proof below therefore bounds the probability of one output, "candidate i wins" or "the run stops at query k".

22.2.2. Fact 2 — Fix the other randomness, then average

A mechanism often draws several random numbers, and only one of them matters for the output you are looking at. You would like to treat the others as fixed constants, prove the bound, and be done. That is allowed as long as the randomness you fix is drawn the same way whatever the data are. Then the probability under D and the probability under D' are the same weighted average of conditional probabilities, with the same weights, and if every conditional probability under D is at most e^{ε} times the one under D', the averages keep that ratio.

Let W be randomness inside the mechanism whose distribution does not depend on the data. If $\Pr[A \mid D, W = w] \leq e^{\varepsilon} \Pr[A \mid D', W = w]$ for every value w, then $\Pr[A \mid D] \leq e^{\varepsilon} \Pr[A \mid D']$.

The proof is one line of averaging, $\Pr[A \mid D] = \sum_w \Pr[W = w]\Pr[A \mid D, w] \le \sum_w \Pr[W = w]\, e^{\varepsilon}\Pr[A \mid D', w] = e^{\varepsilon}\Pr[A \mid D']$. The weights $\Pr[W = w]$ are the same on both sides only because W does not depend on the data; if they differed, the two averages could weight the conditional probabilities differently and the ratio could escape.

To see it with numbers, take a mechanism that flips a fair coin W and then does something that depends on the data. Table 22.2 shows one event A.

Table 22.2.: Fixing a data-independent coin: the ratio after averaging, 1.43, lies between the conditional ratios 1.25 and 1.50, so a bound on every conditional ratio bounds the average.

Coin w	$\Pr[W = w]$	$\Pr[A \mid D, w]$	$\Pr[A \mid D', w]$	Ratio
heads	0.5	0.30	0.20	1.50
tails	0.5	0.10	0.08	1.25
average		0.20	0.14	1.43

The averages are $0.5(0.30) + 0.5(0.10) = 0.20$ and $0.5(0.20) + 0.5(0.08) = 0.14$, and their ratio $0.20/0.14 = 1.43$ can never exceed the larger conditional ratio, 1.50, because a ratio of two averages taken with the same weights lies between the smallest and the largest of the individual ratios. In report-noisy-max, W will be the noise of every candidate except one; in the sparse vector technique, it will be the noise of every query before the one that stops the run.

22.2.3. Fact 3 — Moving a Laplace threshold by s changes its tail by at most $e^{|s|/b}$

After Fact 2 has fixed the other noise, the output will happen exactly when one Laplace variable Z lands above some bar t, and moving to a neighbouring dataset will move the bar to $t - s$. The question is how much that can change $\Pr[Z > t]$. The answer comes from the shape of the Laplace density: it falls by a factor $e^{-1/b}$ for each unit you move away from 0, so sliding it by s changes its height at any point by at most a factor $e^{|s|/b}$, and therefore changes the probability of any tail by at most that factor. For $Z \sim \mathrm{Lap}(b)$ and all real t and s,

$$\Pr[Z > t - s] \le e^{|s|/b}\,\Pr[Z > t]. \tag{22.2}$$

To prove it, compare the density at z with the density at $z - s$. They are $p(z) = \frac{1}{2b}e^{-|z|/b}$ and $p(z - s) = \frac{1}{2b}e^{-|z-s|/b}$, so $p(z-s)/p(z) = e^{(|z|-|z-s|)/b}$, and the triangle inequality says that two points s apart differ in distance from 0 by at most $|s|$: $\big||z|-|z-s|\big| \le |s|$. Hence $p(z-s) \le e^{|s|/b}p(z)$ at every z. Then change variables: $\Pr[Z > t - s] = \int_{t-s}^{\infty} p(z)\,dz = \int_{t}^{\infty} p(z-s)\,dz \le e^{|s|/b}\int_{t}^{\infty} p(z)\,dz$.

With $b = 1$ and $s = 1$, the script gives $\Pr[Z > -1] = 1 - \frac{1}{2}e^{-1} = 0.8161$ against $\Pr[Z > 0] = 0.5000$, a ratio of 1.6321, well below e. With $t = 2$ it gives $\Pr[Z > 1] = \frac{1}{2}e^{-1} = 0.1839$ against $\Pr[Z > 2] = \frac{1}{2}e^{-2} = 0.0677$, a ratio of $2.7183 = e$ exactly. The bound is attained there because both bars lie in the upper tail, where every probability has the form $\frac{1}{2}e^{-x/b}$ and moving the bar by 1 multiplies it by exactly $e^{1/b}$. Every privacy proof in this chapter ends with this step.

22.2.4. Fact 4 — A maximum moves no more than its arguments

The bar in Fact 3 will be a maximum over many other candidates. If each of them can move by up to Δ, you might fear that the maximum moves by more, perhaps by Δ for each of them. It does not, because the maximum is just the value of whichever term is on top, and no term can rise or fall by more than Δ.

If $|a'_j - a_j| \le \Delta$ for every j, then $|\max_j a'_j - \max_j a_j| \le \Delta$, for any number of terms.

For the proof, raise every term to its highest possible value: $\max_j a'_j \le \max_j (a_j + \Delta) = \max_j a_j + \Delta$. The same argument with the roles of a and a' swapped gives $\max_j a_j \le \max_j a'_j + \Delta$, and the two together are the claim. If all terms move the same way, say $a'_j \in [a_j, a_j + \Delta]$ for every j, then the same argument gives $\max_j a'_j - \max_j a_j \in [0, \Delta]$: the maximum moves the same way, by at most Δ. This direction will matter for monotone scores. The bar a candidate has to clear is a maximum over the others, and this fact is why the number of candidates, and the number of queries below a threshold, never enters the privacy cost.

22.3. The exponential mechanism

22.3.1. Context

You want to output a good candidate, and "good" is measured by a score with bounded sensitivity. The natural private answer is a random candidate, tilted toward high scores. How much tilting can you afford? McSherry and Talwar (2007) answered: give each candidate a weight $e^{\varepsilon u/(2\Delta)}$ and pick it with probability proportional to that weight. One record changes each score by at most Δ, so it changes every weight by at most a factor $e^{\varepsilon/2}$, and that is enough for ε-DP whatever the number of candidates.

22.3.2. Statement

The **exponential mechanism** M_E turns each score into a weight that grows exponentially with it, and picks a candidate with probability equal to its share of the total weight. The total weight $Z(D)$ is the **normalizer** that makes the probabilities add up to 1. So M_E outputs $r \in \mathcal{R}$ with probability

$$\Pr[M_E(D) = r] = \frac{\exp\left(\frac{\varepsilon\, u(D,r)}{2\Delta}\right)}{Z(D)}, \qquad Z(D) = \sum_{s \in \mathcal{R}} \exp\left(\frac{\varepsilon\, u(D,s)}{2\Delta}\right). \tag{22.3}$$

1. **Privacy.** M_E is ε-DP.
2. **Monotone scores.** If the scores are monotone, the mechanism with weights $\exp(\varepsilon u(D,r)/\Delta)$, without the 2, is ε-DP.

22.3.3. Derivation

By Fact 1 it is enough to bound the ratio of the probabilities of one output r on two neighbours. That probability is a weight divided by a total, and one record can change both: it can move r's own score, and it can move all the other scores that make up the total. So split the ratio into those two effects and bound each one. Fix neighbours $D \simeq D'$ and a candidate r:

$$\frac{\Pr[M_E(D) = r]}{\Pr[M_E(D') = r]} = \underbrace{\exp\Big(\frac{\varepsilon\big(u(D,r) - u(D',r)\big)}{2\Delta}\Big)}_{\text{numerator}} \cdot \underbrace{\frac{Z(D')}{Z(D)}}_{\text{normalizer}} . \tag{22.4}$$

1. **Bound the numerator, r's own weight.** By the definition of sensitivity, $u(D,r) - u(D',r) \le \Delta$, so this factor is at most $e^{\varepsilon\Delta/(2\Delta)} = e^{\varepsilon/2}$.
2. **Bound the normalizer, the total weight.** For every candidate s, $u(D',s) \le u(D,s) + \Delta$, again by sensitivity, so every term of $Z(D')$ is at most $e^{\varepsilon/2}$ times the matching term of $Z(D)$. Adding up the terms, $Z(D') \le e^{\varepsilon/2} Z(D)$.
3. **Multiply the two bounds.** The ratio is at most $e^{\varepsilon/2} \cdot e^{\varepsilon/2} = e^{\varepsilon}$. The pair of neighbours and the candidate were arbitrary, so Fact 1 gives ε-DP.

Monotone scores. Now the weights are $e^{\varepsilon u/\Delta}$, so each factor alone could be as large as e^{ε}, and their product as large as $e^{2\varepsilon}$. Monotonicity stops them from both being large at once. For r's own weight to grow from D' to D, its score must go down from D to D'; for the total to grow, the scores must go up. When all scores move the same way, only one of these can happen.

1. *If every score goes up from D to D',* the numerator $\exp(\varepsilon(u(D,r) - u(D',r))/\Delta) \le 1$, because the exponent is not positive. The normalizer $Z(D')/Z(D) \le e^{\varepsilon}$, by the argument of step 2 with Δ in place of 2Δ in the weights.
2. *If every score goes down,* the numerator is at most e^{ε}, by step 1 with the new weights, and every term of $Z(D')$ is at most the matching term of $Z(D)$, so the normalizer is at most 1.
3. *Either way* the product is at most e^{ε}. The neighbours in the other order are covered by the same two cases with the directions swapped.

22.3.4. Interpretation

The 2 in the exponent pays for two things that can both move. One record can raise the score of r and lower the scores of its competitors at the same time: the first makes r's weight larger, the second makes the normalizer smaller, and each effect alone can use up to $e^{\varepsilon/2}$. Exercise 1 in the exercise book shows a pair of neighbours where the ratio approaches $e^{2\varepsilon}$ if you drop the 2 for scores that are not monotone. With monotone scores the two effects pull in opposite directions, as the derivation showed, so half the budget is enough.

If you have met the **softmax** of machine learning, which turns scores x_r into probabilities proportional to $e^{x_r/\tau}$ for a temperature τ, you have met this mechanism: it is a softmax of the scores with temperature $2\Delta/\varepsilon$. As $\varepsilon \to 0$ the temperature grows, every weight tends to 1, and the output is uniform and reveals nothing. As $\varepsilon \to \infty$ the largest score's weight dominates and the mechanism becomes the argmax. Figure 22.1 shows the whole range on the model-selection example below.

Three things must hold for the proof to apply. First, the candidate set must not depend on the data. $\mathcal{R}$ is fixed in advance, because the proof compares the same list of candidates on D and D'; if the list were read off the data, a candidate's presence or absence would itself leak. Second, the candidate set may be infinite. On a continuous range, replace the sum by an integral against a fixed base measure μ: the density is proportional to $\mu(r)\, e^{\varepsilon u(D,r)/(2\Delta)}$, and the same two-factor proof goes through, since it never used that the sum was finite. The private median over an interval is the standard example; Exercise 2 in the exercise book does the discrete version. Third, and in the other direction, nothing in the proof uses the structure of $\mathcal{R}$. The candidates can be models, split points, strings or graphs. This is what makes the mechanism the general-purpose tool for non-numeric outputs.

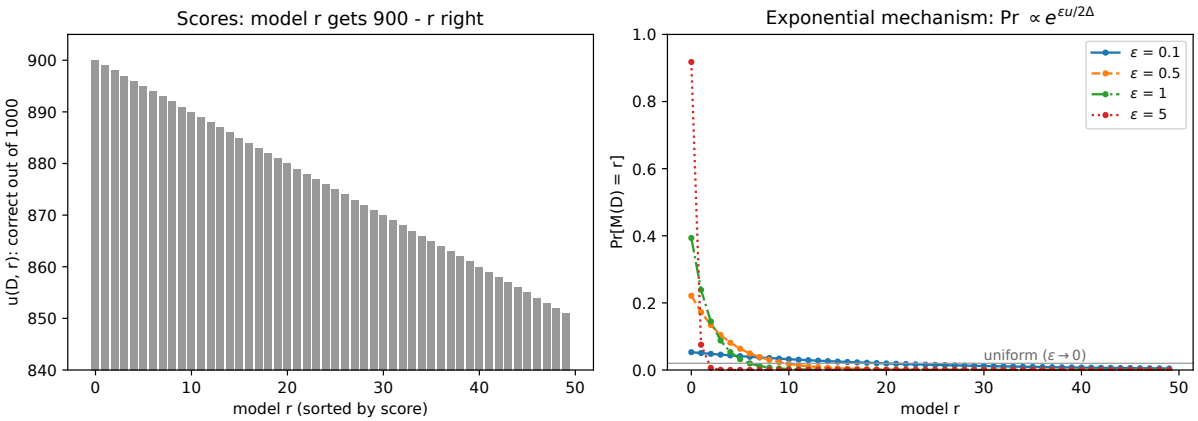


Figure 22.1.: Left: the scores of 50 models, model r getting $900 - r$ of 1,000 validation examples right. Right: the probability that the exponential mechanism selects each model, for four values of ε. Small ε spreads the probability toward uniform; large ε concentrates it on the best model.

22.3.5. Example: a vote with four options

Six people vote among options A, B, C, D. On dataset D the counts are $(3, 2, 1, 0)$; its neighbour D' has one more vote, for B, so the counts are $(3, 3, 1, 0)$. The score is the count. One added or removed vote changes one count by 1, so $\Delta = 1$ under add/remove. Take $\varepsilon = 1$, so the weights are $e^{u/2}$. Table 22.3 shows the script's output.

Table 22.3.: The exponential mechanism with $\varepsilon = 1$ on a vote and its neighbour with one extra vote for B.

Option	A	B	C	D
$\Pr[M_E(D) = r]$	0.4551	0.2760	0.1674	0.1015
$\Pr[M_E(D') = r]$	0.3859	0.3859	0.1420	0.0861
Ratio D/D'	1.1790	0.7151	1.1790	1.1790

Check the derivation's two factors by hand. The weights on D are $e^{1.5}$, e^{1}, $e^{0.5}$ and $e^{0} = 1$, which sum to $Z(D) = 9.8487$; on D' B's weight rises to $e^{1.5}$ and the sum to $Z(D') = 11.6121$. Only

B's score changed, so for A, C and D the numerator is 1 and the ratio is the normalizer ratio $Z(D')/Z(D) = 11.6121/9.8487 = 1.1790$, below its bound $e^{1/2} = 1.6487$. For B the numerator is $e^{(2-3)/2} = e^{-1/2}$, giving $1.1790/1.6487 = 0.7151$. The largest privacy loss on this pair is $|\ln 0.7151| = 0.3353$, a third of ε. That is no contradiction: the bound must hold for every pair, and this pair moves only a single score, so only one of the two factors is at work for each option. Counts are monotone here, so the weights e^u are also allowed; they give a sharper selection (A with probability 0.6439 on D) and a largest privacy loss of 0.6585, still below 1.

22.3.6. Check yourself

- If you add the same constant to every score, does the exponential mechanism change? *(No. Every weight is multiplied by the same factor, which cancels against the same factor in the normalizer. Only differences between scores matter.)*
- The score is the number of correct validation predictions, and you switch to accuracy, the same count divided by n. What happens to Δ and to the output distribution? *(Δ becomes $1/n$, and u/Δ is unchanged, so the distribution is the same. The mechanism is invariant to rescaling the score.)*

22.4. Utility of the exponential mechanism

22.4.1. Context

Privacy came for free in k. Accuracy does not: with more candidates there are more bad ones to pick by accident. How far below the best score can the selected candidate be? The answer is a tail bound with only a $\ln k$ cost. The idea of the proof is to compare weights. A bad candidate, one at least c below the best, has a weight at most $e^{-\varepsilon c/(2\Delta)}$ times the weight of the best candidate, which sits in the normalizer. So each bad candidate is picked with probability at most $e^{-\varepsilon c/(2\Delta)}$, and there are at most k of them.

22.4.2. Statement

Call the **gap** the difference between the best score and the score of the candidate the mechanism picks. The probability that the gap exceeds a few multiples of the temperature $2\Delta/\varepsilon$ falls exponentially. For every dataset D and every $t > 0$,

$$\Pr\left[u(D, M_E(D)) \le \mathrm{OPT}(D) - \frac{2\Delta}{\varepsilon}(\ln k + t)\right] \le e^{-t}, \tag{22.5}$$

and the expected gap is

$$\mathbb{E}\big[\mathrm{OPT}(D) - u(D, M_E(D))\big] \le \frac{2\Delta}{\varepsilon}(\ln k + 1). \tag{22.6}$$

If k_{OPT} candidates attain OPT, $\ln k$ can be replaced by $\ln(k/k_{\mathrm{OPT}})$. With monotone scores and weights $e^{\varepsilon u/\Delta}$, replace 2Δ by Δ.

22.4.3. Derivation

Write $G = \mathrm{OPT}(D) - u(D, M_E(D)) \geq 0$ for the gap, and let $B_c = \{r : u(D, r) \leq \mathrm{OPT} - c\}$ be the bad candidates, those at least c below the best. The gap is at least c exactly when the mechanism picks one of them, so $\Pr[G \geq c]$ is their total weight divided by $Z(D)$. Bound the top from above and the bottom from below.

1. **Bad candidates are light.** The weight grows with the score, so each $r \in B_c$ has weight $e^{\varepsilon u(D,r)/(2\Delta)} \leq e^{\varepsilon(\mathrm{OPT}-c)/(2\Delta)}$.
2. **The normalizer is heavy.** $Z(D)$ is a sum of positive weights that includes the weight of an optimal candidate, so $Z(D) \geq e^{\varepsilon\,\mathrm{OPT}/(2\Delta)}$, and k_{OPT} times that if there are k_{OPT} of them.
3. **Divide.** $\Pr[G \geq c] = \Pr[M_E(D) \in B_c] \leq |B_c|\, e^{\varepsilon(\mathrm{OPT}-c)/(2\Delta)} / e^{\varepsilon\,\mathrm{OPT}/(2\Delta)} = |B_c|\, e^{-\varepsilon c/(2\Delta)} \leq k\, e^{-\varepsilon c/(2\Delta)}$, since there are at most k bad candidates. (With k_{OPT} optimal candidates, the bad ones number at most k and the normalizer is k_{OPT} times larger, which gives the $\ln(k/k_{\mathrm{OPT}})$ version.)
4. **Solve for c.** Ask which gap c has probability at most e^{-t}. Setting $k\, e^{-\varepsilon c/(2\Delta)} = e^{-t}$ and taking logarithms gives $\ln k - \varepsilon c/(2\Delta) = -t$, so $c = \frac{2\Delta}{\varepsilon}(\ln k + t)$, which is Equation 22.5.
5. **Expectation.** For a non-negative variable, the mean is the integral of its tail, $\mathbb{E}G = \int_0^\infty \Pr[G \geq c]\, dc$. The bound of step 3 is larger than 1, and so useless, until $c_0 = \frac{2\Delta}{\varepsilon} \ln k$, where $k\, e^{-\varepsilon c_0/(2\Delta)} = 1$. So bound the integrand by 1 up to c_0 and by step 3 beyond: $\mathbb{E}G \leq c_0 + \int_{c_0}^\infty k\, e^{-\varepsilon c/(2\Delta)}\, dc = c_0 + \frac{2\Delta}{\varepsilon}$, since the integral equals $\frac{2\Delta}{\varepsilon} k\, e^{-\varepsilon c_0/(2\Delta)} = \frac{2\Delta}{\varepsilon}$.

22.4.4. Interpretation

The cost of k is logarithmic. Doubling the number of candidates adds $\frac{2\Delta}{\varepsilon} \ln 2$ to the gap, because $\ln(2k) = \ln k + \ln 2$. You can select among 2^{20} candidates at the price of $\ln 2^{20} = 20 \ln 2 \approx 14$ in units of $2\Delta/\varepsilon$. A lower bound shows that no ε-DP mechanism can avoid the $\ln k$.

How good is the bound? The derivation made two approximations. Step 3 replaced the number of bad candidates $|B_c|$ by k, and their weights by the largest possible bad weight; step 2 kept only one term of Z. Both are nearly exact when there is one best candidate and all $k-1$ others sit exactly c below it. Figure 22.2 (right) shows that case: once the bound is small, the exact probability of missing the best and the bound are almost the same (0.0022 against 0.0023 at a gap of 20). With $k = 50$ and $\varepsilon = 1$, the bound is $50e^{-10}$ and the exact probability is $49e^{-10}/(1 + 49e^{-10})$, so the two differ only in counting 50 bad candidates instead of 49 and in dropping the runners-up from the normalizer.

When scores are spread out, the bound is loose. Then few candidates are near the top, and most bad candidates are much further than c below the best and so much lighter than the bound assumes. Figure 22.2 (left) shows the bound several orders of magnitude above the truth for the model-selection scores below.

22.4.5. Example: choosing among 50 models

You have trained 50 models and want to release the best one according to a private validation set of 1,000 examples. Model r gets $900 - r$ right (Figure 22.1, left), so the best is at 90.0% accuracy

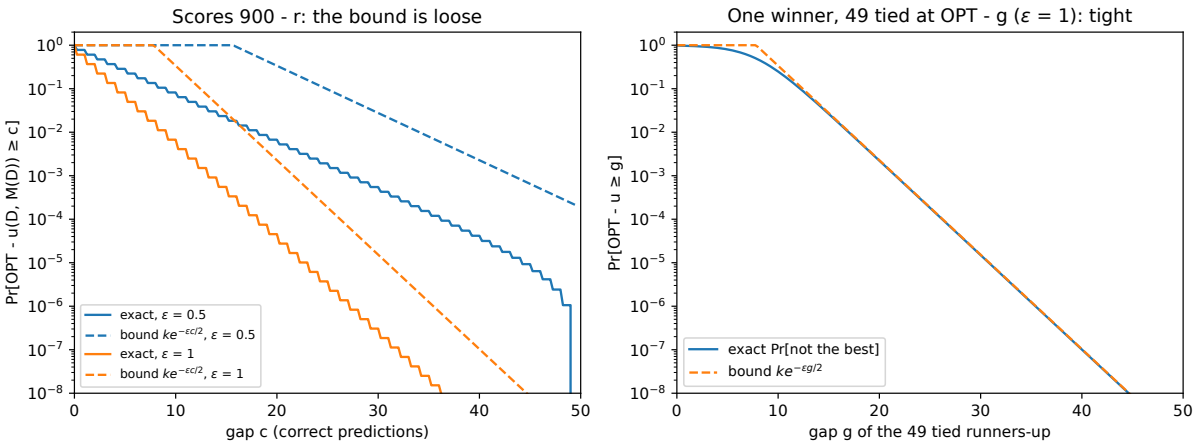


Figure 22.2.: Probability that the exponential mechanism's choice is at least c below the best score, exact against the bound $ke^{-\varepsilon c/2\Delta}$. Left: the 50 models with scores $900 - r$, for $\varepsilon = 0.5$ and 1; the bound is loose because the scores are spread out. Right: one best model and 49 runners-up all exactly g below it, with $\varepsilon = 1$; the bound is tight.

and neighbouring ranks differ by 0.1%. The score is the count of correct predictions; adding or removing one validation example changes each count by at most 1, so $\Delta = 1$. Table 22.4 shows the script's output.

Table 22.4.: The exponential mechanism selecting among 50 models with scores $900 - r$. The gap is in correct predictions out of 1,000.

ε	Pr[best]	$\mathbb{E}$[gap]	Bound $2(\ln 50 + 1)/\varepsilon$	$\mathbb{E}$[gap], monotone weights
0.1	0.0531	15.03	98.24	9.17
1	0.3935	1.54	9.82	0.58
5	0.9179	0.09	1.96	0.01

At $\varepsilon = 1$ you lose 1.54 correct predictions on average, 0.15% accuracy, although you pick the best model only 39% of the time: the mechanism mostly picks one of the near-ties, which cost only one or two correct predictions. The tail bound with $t = 3$ says the gap exceeds $2(\ln 50 + 3) = 13.82$ with probability at most $e^{-3} = 0.0498$; the exact probability is 9.12×10^{-4}, about fifty times smaller, because the scores are spread out. At $\varepsilon = 0.1$ the expected-gap bound, 98.24, exceeds the largest possible gap of 49 (the worst model has score $900 - 49$) and says nothing. On the tied profile of Figure 22.2 (right) with $\varepsilon = 1$, the exact probability of a gap of at least 20 is 0.0022 against a bound of 0.0023.

Counts of correct predictions are monotone under add/remove of a validation example, since one more example can only add correct predictions, so the weights $e^{\varepsilon u}$ are allowed too. They halve the temperature and cut the expected gap at $\varepsilon = 1$ from 1.54 to 0.58.

22.4.6. Check yourself

- You have $k = 10^6$ candidates instead of 50. By how much does the tail bound at $\varepsilon = 1$, $\Delta = 1$, $t = 3$ grow? *(From* $2(\ln 50 + 3) = 13.82$ *to* $2(\ln 10^6 + 3) = 33.63$*: a factor of 20,000 in candidates costs a factor of 2.4 in the gap.)*
- Why does step 3 of the derivation not need the bad candidates to be few? *(It bounds each bad candidate's probability by* $e^{-\varepsilon c/(2\Delta)}$ *and adds them up; there are at most* k *of them, which is where* $\ln k$ *comes from.)*

22.5. Report-noisy-max

22.5.1. Context

Here is the obvious way to select privately. Add independent noise to every score, and report the index of the largest noisy score. Release *only the index*. If you released the noisy scores as well, you would be releasing a vector, and its ℓ_1 sensitivity, the total change over all coordinates, would set the noise. For k counting queries that one record can all change (for example "how many records have feature j", for k features), that sensitivity is k, and the Laplace mechanism would need noise of scale k/ε. Releasing only the argmax needs noise of scale $1/\varepsilon$, for any k, because of the bar argument sketched at the start of this chapter.

22.5.2. Statement

Report-noisy-max adds an independent Laplace noise of scale b to every score and outputs the candidate with the largest noisy score:

$$M_N(D) = \arg\max_{r \in \mathcal{R}} \left(u(D, r) + Z_r\right), \qquad Z_r \sim \mathrm{Lap}(b) \text{ independent.} \tag{22.7}$$

Ties have probability zero, because the noise is continuous.

1. **General scores.** With $b = 2\Delta/\varepsilon$, M_N is ε-DP.
2. **Monotone scores** (Dwork & Roth, Claim 3.9, stated there for counting queries with $\Delta = 1$). With $b = \Delta/\varepsilon$, M_N is ε-DP.
3. **Accuracy.** With probability at least $1 - \beta$, $\mathrm{OPT}(D) - u(D, M_N(D)) \le 2b \ln(k/\beta)$.

22.5.3. Derivation

The proof below covers both privacy claims. It is the argument of Dwork & Roth's Claim 3.9, written so that the general and the monotone case differ in one step. It uses all four facts of the toolkit in order: look at one output (Fact 1), freeze the other candidates' noise (Fact 2), see that the output happens when one Laplace noise clears a bar, bound the bar's movement (Fact 4), and pay for it (Fact 3).

1. **Fix everything but one noise.** Fix neighbours $D \simeq D'$, a candidate i, and the noise z_{-i} of all other candidates. The noises are drawn the same way whatever the data, so by Fact 2 it is enough to bound $\Pr[M_N(D) = i \mid z_{-i}]$ against the same probability on D'.

2. **Candidate i wins when its noise clears a bar.** With the others' noise fixed, their noisy scores are fixed numbers, and i wins on D exactly when its own noisy score beats the best of them, $u(D,i)+Z_i > \max_{j\neq i}(u(D,j)+z_j)$. Moving $u(D,i)$ to the right-hand side, that is $Z_i > t(D)$ with the bar
$$t(D) = \max_{j\neq i}\,(u(D,j)+z_j) - u(D,i). \tag{22.8}$$
So $\Pr[M_N(D)=i \mid z_{-i}] = \Pr[Z_i > t(D)]$, and likewise for D'.
3. **The bar moves by at most 2Δ.** The first term of t is a maximum of numbers $u(\cdot,j)+z_j$, and each moves by at most Δ between D and D' because the z_j are fixed. By Fact 4 the maximum moves by at most Δ, however many terms it has. The second term, $u(\cdot,i)$, moves by at most Δ too. So $|t(D)-t(D')| \le 2\Delta$.
4. **With monotone scores, by at most Δ.** If every score goes up by an amount in $[0,\Delta]$, the maximum goes up by an amount in $[0,\Delta]$ (Fact 4, the same-direction case) and so does $u(\cdot,i)$. Their difference, the bar, changes by an amount in $[-\Delta,\Delta]$, because subtracting two numbers in $[0,\Delta]$ gives a number in $[-\Delta,\Delta]$. The same holds if every score goes down.
5. **Shift the threshold.** By Fact 3 with $s = t(D)-t(D')$, $\Pr[Z_i > t(D)] \le e^{|t(D)-t(D')|/b}\Pr[Z_i > t(D')]$. In the general case that factor is at most $e^{2\Delta/b} = e^{\varepsilon}$ with $b = 2\Delta/\varepsilon$, and in the monotone case at most $e^{\Delta/b} = e^{\varepsilon}$ with $b = \Delta/\varepsilon$.
6. **Average and collect.** Fact 2 removes the conditioning on z_{-i}, and Fact 1 turns the bound for each i into ε-DP.

Figure 22.3 draws steps 2 to 5 for $b=1$, $t(D)=0.7$ and a bar lowered by $s=1$ on D': the winning probability rises from $\Pr[Z_i > 0.7] = \frac{1}{2}e^{-0.7} = 0.2483$ to $\Pr[Z_i > -0.3] = 1-\frac{1}{2}e^{-0.3} = 0.6296$, a ratio of 2.5357, below $e = 2.7183$. The ratio falls short of e because the lowered bar has crossed 0, out of the exponential tail where Fact 3 is exact.

Accuracy. The idea is that if no noise is large, the noise cannot reorder candidates whose scores are far apart. A single Laplace noise exceeds x in absolute value with probability $e^{-x/b}$. By the union bound, the probability that at least one of k events happens is at most the sum of their probabilities, so some $|Z_r|$ reaches x with probability at most $k\,e^{-x/b}$. Setting this to β gives $x = b\ln(k/\beta)$: every $|Z_r| < b\ln(k/\beta)$ with probability at least $1-\beta$. This is the maximum-of-Laplace rule of Concentration inequalities. On that event the winner i beat a best candidate r^* despite noise of at most $b\ln(k/\beta)$ on each side, so $u(D,i)+b\ln(k/\beta) > u(D,i)+Z_i \ge u(D,r^*)+Z_{r^*} > \mathrm{OPT}-b\ln(k/\beta)$, and the gap is less than $2b\ln(k/\beta)$. This is Exercise 5 in the exercise book of that chapter.

22.5.4. Interpretation

The privacy cost is not $k\varepsilon$ because of step 3. That is where the losers would cost something, and they don't: they enter only through one maximum, which moves by at most Δ however many of them there are (Fact 4).

The noise must be $2\Delta/\varepsilon$ in general because the bar has two parts that can move in opposite directions: the competitors can rise while candidate i falls, and then the bar rises by both amounts. Monotone scores cannot do that, which halves the noise. This is the same factor of 2 as in the exponential mechanism, for the same reason.

Should you use Laplace report-noisy-max or the exponential mechanism? They are different mechanisms with the same privacy, and neither dominates. On the 50 models at $\varepsilon = 1$, Laplace

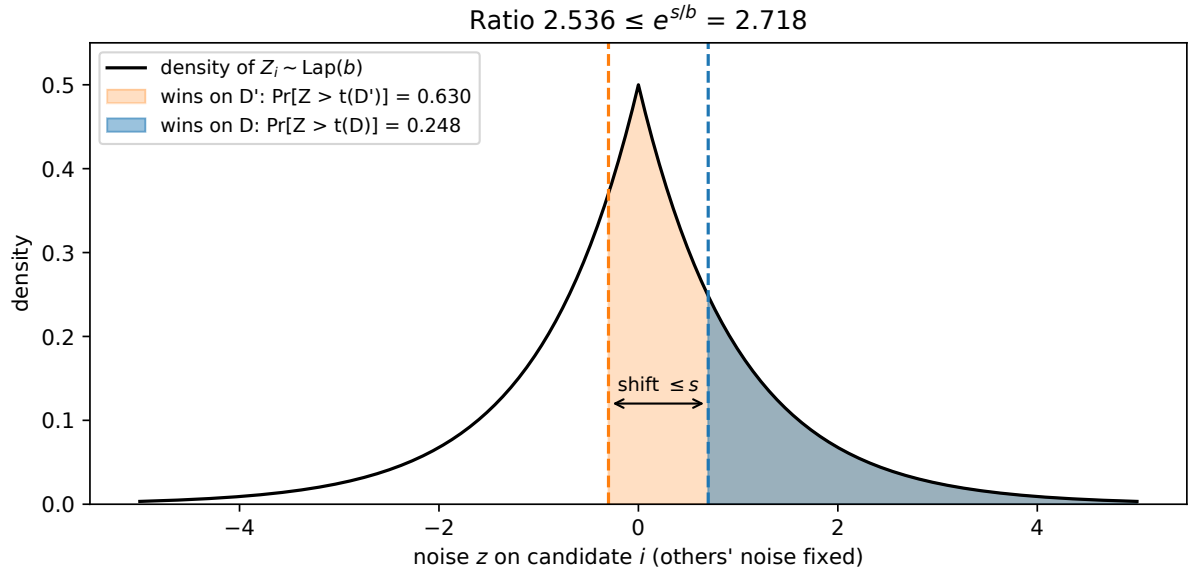


Figure 22.3.: The proof of report-noisy-max in one picture. With the other candidates' noise fixed, candidate i wins when its own noise $Z_i \sim \mathrm{Lap}(1)$ exceeds a bar t. Moving to a neighbouring dataset moves the bar by at most s, and the winning probability changes by at most a factor $e^{s/b}$.

report-noisy-max with $b = 2$ picks the best model with probability 0.4241 against 0.3935 and has expected gap 1.40 against 1.54. With two candidates the exponential mechanism is slightly better (Exercise 6 in the exercise book). Adding one-sided exponential noise instead of Laplace noise gives a third mechanism, which equals the "permute-and-flip" mechanism of McKenna & Sheldon (2020); its expected score is never below the exponential mechanism's.

Finally, release only the index. Releasing the winning noisy score as well breaks the proof: the bar argument conditions on everything but Z_i and averages over Z_i, and Z_i itself would then be published. To release the winner's score, spend extra budget on a fresh Laplace answer for that one query.

22.5.5. Example: the vote and the 50 models

On the vote of Table 22.3, counts are monotone, so $b = 1/\varepsilon = 1$ is allowed. The script integrates the winning probabilities exactly: on D they are $(0.6551, 0.2380, 0.0794, 0.0275)$, on D' $(0.4639, 0.4639, 0.0534, 0.0189)$, and the largest privacy loss over the four outputs, the largest $|\ln|$ of a ratio of matching entries, is 0.6674, below $\varepsilon = 1$. With the general scale $b = 2$ it is 0.3626.

On the 50 models at $\varepsilon = 1$, the general mechanism ($b = 2$) has expected gap 1.40 (1.39 in 200,000 simulated runs), and the monotone one ($b = 1$) has 0.54 and picks the best model with probability 0.6463. The accuracy bound at $\beta = 0.05$ is $2b\ln(50/0.05) = 2b\ln 1000$, which is 27.63 for $b = 2$ and 13.82 for $b = 1$. Like the exponential mechanism's bound, it is loose for spread-out scores.

22.5.6. Check yourself

- Where does the proof use that the noises are independent? *(In step 1: conditioning on z_{-i} must leave Z_i with its Laplace distribution, so that Fact 3 applies.)*
- If the scores are a histogram under replace neighbours (one count goes down by 1, another goes up by 1), can you use $b = 1/\varepsilon$? *(No. The scores move in opposite directions, so they are not monotone; use $b = 2/\varepsilon$.)*

22.6. Gumbel noise: report-noisy-max is the exponential mechanism

22.6.1. Context

Report-noisy-max with Laplace noise is close to the exponential mechanism but not equal to it (Figure 22.4). Is there a noise distribution for which "add noise and take the argmax" gives exactly the softmax probabilities of Equation 22.3? There is: the Gumbel distribution. This is the **Gumbel-max trick**, used throughout machine learning to sample from a softmax. For privacy it gives a second proof that the exponential mechanism is private, a numerically stable way to sample it, and a fast way to select the top c candidates.

22.6.2. Statement

The **Gumbel distribution** arises as the limit of the maximum of many independent variables with exponential tails, after shifting; what matters here is only its formula. $G \sim \text{Gumbel}(\beta)$, with scale β, has cumulative distribution function (CDF) $\Pr[G \le g] = \exp(-e^{-g/\beta})$; NumPy's `rng.gumbel(0, beta)` samples it. The claim is that adding independent Gumbel noise to each score and taking the argmax picks candidate i with exactly its softmax probability. For independent $G_r \sim \text{Gumbel}(\beta)$,

$$\Pr\left[\arg\max_r \left(u_r + G_r\right) = i\right] = \frac{e^{u_i/\beta}}{\sum_r e^{u_r/\beta}}. \tag{22.9}$$

With $\beta = 2\Delta/\varepsilon$ this is the exponential mechanism.

22.6.3. Derivation

Rescaling all scores and noises by $1/\beta$ does not change which one is largest, so take $\beta = 1$. The Gumbel CDF is then $F(g) = \exp(-e^{-g})$, and its derivative, the density, is $f(g) = e^{-g}\exp(-e^{-g})$.

Two candidates first. Candidate 1 wins when $G_2 - G_1 < u_1 - u_2$, so you need the distribution of the difference of two independent Gumbels. It is the **logistic** distribution, with CDF $\Pr[G_1 - G_2 \le x] = 1/(1+e^{-x})$, and $G_2 - G_1$ has the same distribution, because swapping the names of two identically distributed independent noises changes nothing. (To get it, integrate $F(g+x)$ against $f(g)$; it is the general calculation below with $k = 2$.) So

$$\Pr[u_1 + G_1 > u_2 + G_2] = \Pr[G_2 - G_1 < u_1 - u_2] = \frac{1}{1 + e^{-(u_1 - u_2)}} = \frac{e^{u_1}}{e^{u_1} + e^{u_2}}, \tag{22.10}$$

where the last step multiplies top and bottom by e^{u_1}. That is the softmax of two scores.

In general. The plan is to condition on the value of the winner's noise, use independence to multiply the probabilities that each other candidate falls short, and notice that the resulting integral can be done exactly.

1. **Condition on the winner's noise.** If candidate i's noise is $G_i = g$, it wins when every other candidate's noisy score is below $u_i + g$, that is, when every other $G_j < g + u_i - u_j$. Each of these has probability $F(g + u_i - u_j)$, and the noises are independent, so the probabilities multiply. Averaging over g with density f,

$$\Pr[i \text{ wins}] = \int_{-\infty}^{\infty} f(g) \prod_{j \neq i} F(g + u_i - u_j)\, dg. \tag{22.11}$$

2. **Collect the exponentials.** The density can be written $f(g) = e^{-g}F(g)$, and $F(g)$ is $F(g + u_i - u_j)$ for $j = i$. Each factor is $F(g + u_i - u_j) = \exp(-e^{-g}e^{u_j - u_i})$, and a product of exponentials is the exponential of the sum, so the product of all the F factors, including the one inside f, is $\exp\big(-e^{-g}\sum_j e^{u_j - u_i}\big) = \exp(-e^{-g}A)$ with $A = e^{-u_i}\sum_j e^{u_j}$. The integrand is $e^{-g}\exp(-e^{-g}A)$.
3. **Substitute** $w = e^{-g}$, so $dw = -e^{-g}\,dg$, and g from $-\infty$ to ∞ is w from ∞ to 0; the minus sign flips the limits back:

$$\Pr[i \text{ wins}] = \int_0^{\infty} e^{-wA}\, dw = \frac{1}{A} = \frac{e^{u_i}}{\sum_j e^{u_j}}. \tag{22.12}$$

22.6.4. Interpretation

The exponential mechanism now has two proofs. It can be analysed by its weights (Section 22.3) or as a noisy max. The bar argument of Section 22.5 also works with Gumbel noise of scale $2\Delta/\varepsilon$, because the Gumbel tail also satisfies $\Pr[G > t - s] \le e^{s/\beta}\Pr[G > t]$ for $s \ge 0$ (for $s < 0$ the probability only falls). To see why, write the tail as $1 - e^{-y}$ with $y = e^{-t/\beta}$. Lowering t by s multiplies y by $e^{s/\beta} \ge 1$, and because $1 - e^{-y}$ is zero at $y = 0$ and concave, multiplying y by a factor of at least 1 multiplies $1 - e^{-y}$ by at most that factor.

The noisy-max form also gives stable sampling. Computing $e^{\varepsilon u/(2\Delta)}$ for scores in the thousands overflows. The noisy max works in score units directly. (Subtracting the largest score before exponentiating, as the script does, is the other fix; it is allowed because adding a constant to every score does not change the mechanism.)

It also gives top-c selection in one shot. Taking the c largest of $u_r + G_r$ gives the same distribution as running the exponential mechanism c times, removing each winner (Durfee & Rogers 2019). Its privacy is that of c exponential mechanisms, $c\varepsilon$ by basic composition.

Laplace noise is close, not equal. Laplace and Gumbel noise of the same scale have similar bodies but different tails, so the selection probabilities differ by a few percent.

22.6.5. Example: five candidates, a million draws

For scores $(4, 3, 3, 1, 0)$ and $\varepsilon = 1$, $\Delta = 1$, the noise scale is $2\Delta/\varepsilon = 2$. Table 22.5 compares the three mechanisms.

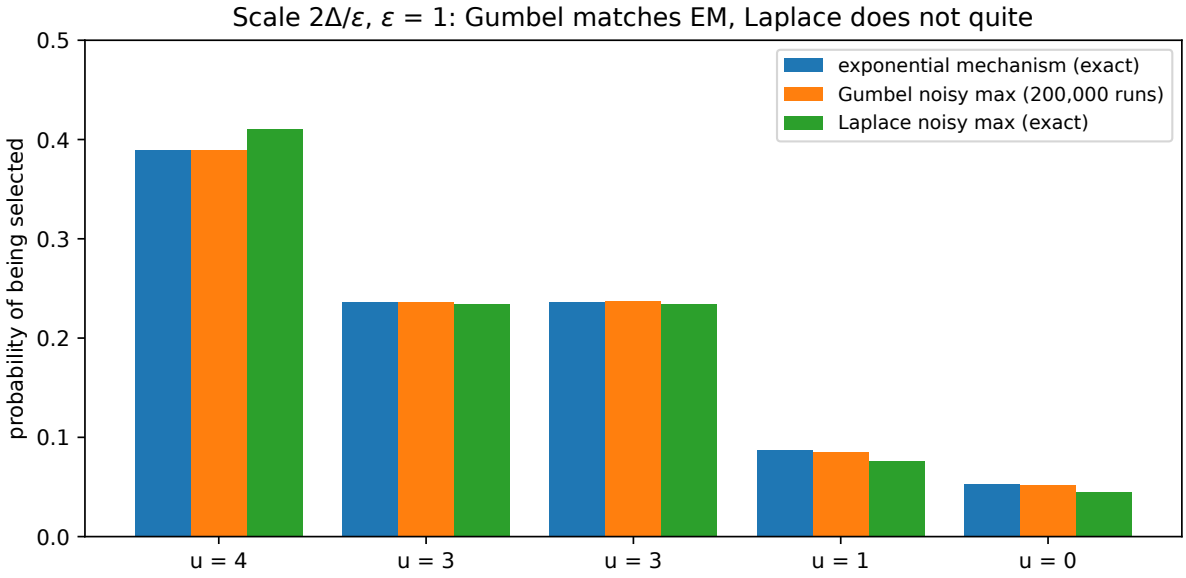


Figure 22.4.: Selection probabilities for five candidates with scores $(4, 3, 3, 1, 0)$ and noise scale $2\Delta/\varepsilon$ with $\varepsilon = 1$: the exponential mechanism's exact probabilities, the frequencies of the Gumbel noisy max in 200,000 runs, and the exact probabilities of the Laplace noisy max.

Table 22.5.: Selection probabilities with noise scale 2 for five candidates.

Candidate score	4	3	3	1	0
Exponential mechanism, exact	0.3889	0.2359	0.2359	0.0868	0.0526
Gumbel noisy max, 10^6 runs	0.3891	0.2351	0.2362	0.0871	0.0525
Laplace noisy max, exact	0.4105	0.2341	0.2341	0.0764	0.0449

The Gumbel frequencies are within 0.0008 of the exponential mechanism. That is about the sampling error of a million draws: a frequency estimated from n draws has standard deviation $\sqrt{p(1-p)/n}$, here $\sqrt{0.39 \times 0.61/10^6} \approx 0.0005$. The Laplace noisy max is further away. Measured by the total variation distance, half the sum of the absolute differences between the two rows, which is the largest difference in probability the two can assign to any set of candidates, it is at distance 0.0216 from the exponential mechanism. Here that is exactly the extra $0.4105 - 0.3889 = 0.0216$ it puts on the top candidate: it favours the top candidate more and the bottom ones less.

22.6.6. Check yourself

- How do you sample from the exponential mechanism over 10^7 candidates without computing a normalizer? *(Draw one Gumbel per candidate, add it to $\varepsilon u/(2\Delta)$, and take the argmax. It is one pass over the scores.)*
- Does adding Gumbel noise and releasing the top 3 cost ε or 3ε? *(3ε under basic composition: it has the distribution of three exponential mechanisms run one after the other.)*

22.7. The sparse vector technique

22.7.1. Context

Now the candidates arrive one at a time and you only care about the first one that is good enough. You monitor a thousand statistics and want to know which one first crosses an alarm level. You try hyperparameter settings until one reaches a target validation accuracy. Most queries in such a stream are uninteresting: their answer is "below". Answering each one with the Laplace mechanism costs ε per query, so a thousand queries would cost 1000ε under basic composition. The **sparse vector technique** (Dwork, Naor, Reingold, Rothblum & Vadhan 2009) compares each query with a single noisy threshold and pays only for the one that is above. Its basic form, **AboveThreshold**, is in Dwork & Roth (2014), Algorithm 1.

22.7.2. Statement

AboveThreshold adds noise to the threshold once, adds fresh noise to each query, and stops at the first noisy query that reaches the noisy threshold. It writes $\perp$ ("below") for each query that does not and $\top$ ("above") for the one that does.

AboveThreshold. Input: a dataset D, a stream of queries $f_1, f_2, ...$, each with sensitivity Δ (they may be chosen adaptively), a threshold T, and budgets $\varepsilon_1, \varepsilon_2 > 0$.

1. Draw the threshold noise $\rho \sim \mathrm{Lap}(\Delta/\varepsilon_1)$ once.
2. For each query $i = 1, 2, ...$: draw fresh noise $\nu_i \sim \mathrm{Lap}(2\Delta/\varepsilon_2)$. If $f_i(D) + \nu_i \geq T + \rho$, output $\top$ and **halt**. Otherwise output $\perp$ and continue.

The output is the sequence $\perp^{k-1}\top$ ($k-1$ answers "below", then one "above"), or $\perp^m$ if all m queries are below; equivalently, the index k of the first $\top$.

1. **Privacy** (Dwork & Roth Thm 3.23 for $\varepsilon_1 = \varepsilon_2 = \varepsilon/2$; Lyu, Su & Li 2017, Algorithm 1 with $c = 1$, for a general split). AboveThreshold is $(\varepsilon_1 + \varepsilon_2)$-DP, for any number of queries.
2. **Monotone queries** (Lyu, Su & Li 2017). If the queries are monotone, as counts under add/remove are, $\nu_i \sim \mathrm{Lap}(\Delta/\varepsilon_2)$ suffices.
3. **Accuracy** (Dwork & Roth Thm 3.24). With $\varepsilon_1 = \varepsilon_2 = \varepsilon/2$, that is $\rho \sim \mathrm{Lap}(2\Delta/\varepsilon)$ and $\nu_i \sim \mathrm{Lap}(4\Delta/\varepsilon)$, consider a run that stops at query k (or answers all k). The noisy comparisons can be wrong only for queries close to the threshold, and the margin α of

"close" is the size the noises stay below with high probability. With probability at least $1-\beta$, every $\perp$ query has $f_i(D) < T + \alpha$, and the $\top$ query has $f_k(D) \geq T - \alpha$, where

$$\alpha = \frac{8\Delta(\ln k + \ln(2/\beta))}{\varepsilon}. \tag{22.13}$$

4. c **answers above.** To find the first c queries above the threshold, restart AboveThreshold with fresh threshold noise after each $\top$, with budget ε/c per run. By basic composition this is ε-DP (Dwork & Roth's algorithm Sparse).

22.7.3. Derivation

Privacy. The output "stop at query k" says two things: the noisy threshold sits above every earlier noisy answer, and it sits at or below the k-th noisy answer. Moving from D to D' moves those noisy answers. The proof undoes that move by sliding the two noises that matter. It slides the threshold noise ρ by however much the largest earlier answer moved, so that the threshold keeps its place relative to all the earlier answers at once; by Fact 4 that is at most Δ, however many earlier queries there are. Then it slides the k-th query's noise ν_k so that query k keeps its place relative to the slid threshold, which takes at most 2Δ. Each slide is paid for by the Laplace density, as in Fact 3.

Fix neighbours $D \simeq D'$. Write $\nu = \nu_i$ for the noise of a single query and $g(D) = \max_{i<k}(f_i(D)+\nu_i)$ for the largest noisy answer before query k.

1. **Fix the noise of the earlier queries.** Consider the output "stop at query k". Fix $\nu_1, \dots, \nu_{k-1}$. By Fact 2 it is enough to bound the conditional probability, because these noises do not depend on the data. (With adaptive queries, the queries $f_1, \dots, f_k$ are fixed too: they can depend only on earlier answers, which on this output are all $\perp$.)
2. **Write the output as two conditions on (ρ, ν_k).** All earlier queries say $\perp$ exactly when even the largest of them is below the noisy threshold, $g(D) < T + \rho$. Query k says $\top$ when $f_k(D) + \nu_k \geq T + \rho$. The probability of the output is the probability that both hold, which is the integral of the two noise densities p_ρ and p_ν over the region where both indicators are 1:

$$\Pr[\text{stop at } k \mid D] = \iint p_\rho(r)\, p_\nu(v)\, \mathbf{1}[g(D) < T + r]\, \mathbf{1}[f_k(D) + v \geq T + r]\, dr\, dv. \tag{22.14}$$

3. **Shift both noises.** Substitute $r = r' + g(D) - g(D')$ and $v = v' + (g(D) - g(D')) - (f_k(D) - f_k(D'))$. Then $g(D) < T + r$ becomes $g(D') < T + r'$, and $f_k(D) + v \geq T + r$ becomes $f_k(D') + v' \geq T + r'$ (substitute and cancel the terms that appear on both sides). The two indicators are exactly those for D' in the variables (r', v'). The substitution is a translation, so $dr\, dv = dr'\, dv'$. What has changed is only the densities, which are now evaluated at shifted points: $p_\rho(r' + g(D) - g(D'))$ and $p_\nu(v' + \text{shift of } v)$.
4. **Size of the shifts.** g is a maximum of numbers $f_i + \nu_i$ that each move by at most Δ (the ν_i are fixed), so $|g(D) - g(D')| \leq \Delta$ by Fact 4, whatever k is. The shift of v is a difference of two terms of size at most Δ, so at most 2Δ.
5. **Pay for the shifts.** The Laplace density satisfies $p(x) \leq e^{|x-x'|/b} p(x')$, as shown in the proof of Fact 3. The threshold noise has $b = \Delta/\varepsilon_1$ and shift at most Δ: a factor e^{ε_1}. The query noise has $b = 2\Delta/\varepsilon_2$ and shift at most 2Δ: a factor e^{ε_2}. Pulling both factors out of the integral leaves exactly the integral for D', so $\Pr[\text{stop at } k \mid D, \nu_{<k}] \leq e^{\varepsilon_1+\varepsilon_2} \Pr[\text{stop at } k \mid D', \nu_{<k}]$.

6. **The output "all below".** Only the first indicator is present, with the maximum over all m queries. Shifting ρ alone costs e^{ε_1}.
7. **Collect.** Fact 2 removes the conditioning, and Fact 1 gives $(\varepsilon_1 + \varepsilon_2)$-DP.

Monotone queries. If all f_i move the same way by at most Δ, then g and f_k move the same way, each by an amount in $[0, \Delta]$ or each in $[-\Delta, 0]$ (Fact 4). The shift of v in step 3 is their difference, which is then at most Δ in size. A query noise with $b = \Delta/\varepsilon_2$ then costs e^{ε_2}.

Accuracy. A comparison of noisy numbers gives the same answer as the comparison of true numbers whenever the noises are smaller than the distance between the true numbers. So the proof finds a margin that all the noises stay below with probability $1 - \beta$, splitting the failure probability β equally between the threshold noise and the query noises. Let $\varepsilon_1 = \varepsilon_2 = \varepsilon/2$.

1. **Threshold noise.** The Laplace tail $\Pr[|\rho| \geq x] = e^{-x/b}$ with $b = 2\Delta/\varepsilon$ and $x = \alpha/2$ gives $\Pr[|\rho| \geq \alpha/2] = e^{-(\alpha/2)/(2\Delta/\varepsilon)} = e^{-\alpha\varepsilon/(4\Delta)}$.
2. **Query noise.** Each ν_i has scale $4\Delta/\varepsilon$, so $\Pr[|\nu_i| \geq \alpha/2] = e^{-\alpha\varepsilon/(8\Delta)}$. By the union bound over k queries, the probability that any of them is that large is $\Pr[\max_i |\nu_i| \geq \alpha/2] \leq k\, e^{-\alpha\varepsilon/(8\Delta)}$.
3. **Both small.** Setting each to at most $\beta/2$ needs $\alpha \geq 8\Delta \ln(2k/\beta)/\varepsilon$ for the second (the first then holds too, since its exponent is twice as large and it has no factor k), which is Equation 22.13 because $\ln(2k/\beta) = \ln k + \ln(2/\beta)$.
4. **Read off the answers.** If $|\rho| < \alpha/2$ and every $|\nu_i| < \alpha/2$, the noises together can move a comparison by less than α. A $\perp$ means $f_i + \nu_i < T + \rho$, so $f_i < T + \rho - \nu_i < T + \alpha$; the $\top$ means $f_k + \nu_k \geq T + \rho$, so $f_k > T - \alpha$.

22.7.4. Interpretation

The $\perp$ answers are free because the derivation pays for all of them at once, in step 4, through one maximum g and one shift of the threshold noise. Fact 4 makes that shift at most Δ whether there are 3 or 3 million queries below the threshold. The single $\top$ is paid for separately, by its own noise.

The run must halt because that payment covers only one $\top$. After the first $\top$, the remaining answers would still compare against the same ρ. The proof's shift of ρ was chosen to keep the $\perp$s consistent; each further $\top$ would need another shift of size up to 2Δ in its own noise, and so another factor e^{ε_2}. With c answers above, Lyu, Su & Li's Algorithm 1 keeps one threshold noise and uses $\nu_i \sim \mathrm{Lap}(2c\Delta/\varepsilon_2)$, so that the c shifts together cost e^{ε_2}; without any limit on c the cost is unbounded (Section 22.8).

AboveThreshold answers a yes/no question. It outputs positions, not values. To also release the value of the query that fired, spend extra budget on a fresh Laplace answer to that query.

The split between ε_1 and ε_2 matters for accuracy, not privacy. Any $\varepsilon_1 + \varepsilon_2 = \varepsilon$ is ε-DP. The equal split is simple but not optimal: Exercise 4 in the exercise book finds the split that minimizes α, and Lyu, Su & Li recommend $\varepsilon_1 : \varepsilon_2 = 1 : (2c)^{2/3}$ to maximize the probability of correct answers.

The proof is tight. On the neighbouring pair of the next section, the output "stop at query $m+1$" has privacy loss exactly ε for $m = 20$ (Table 22.7): both shifts in step 4 reach their maximum.

Figure 22.5 shows one run.

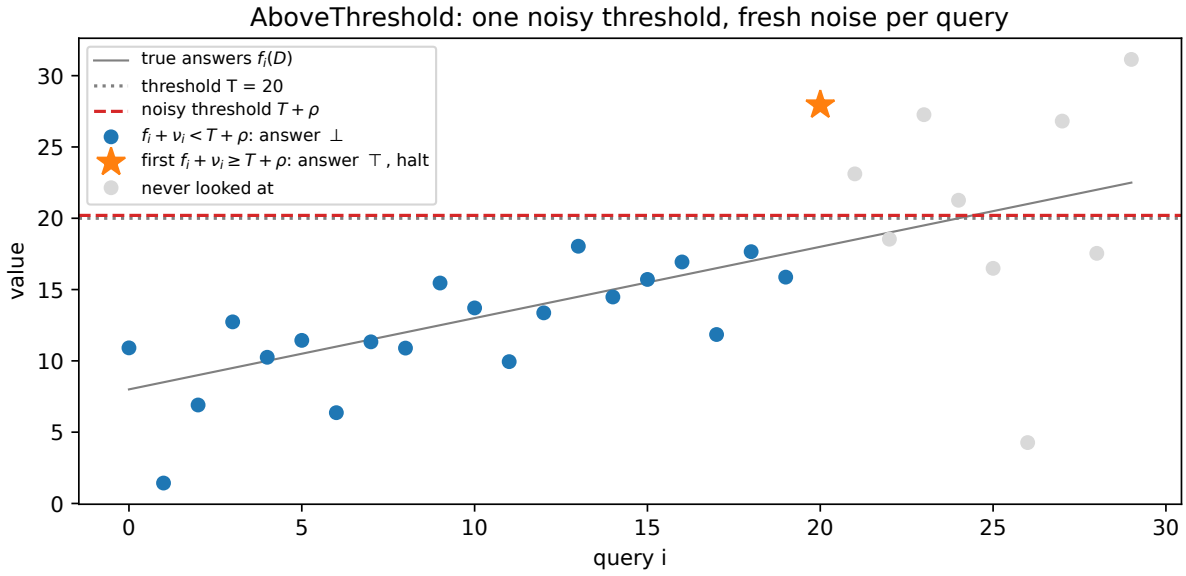


Figure 22.5.: One run of AboveThreshold with $\varepsilon = 1$ on 30 queries whose true answers rise from 8 to 22.5, with threshold $T = 20$. Each query's noisy answer is compared with the same noisy threshold. The first one above it is reported and the run halts; later queries are never looked at.

22.7.5. Example: an alarm among 1,000 statistics

You monitor $k = 1000$ counting queries ($\Delta = 1$) with threshold $T = 500$ at $\varepsilon = 1$. With $\beta = 0.05$, Equation 22.13 gives $\alpha = 8(\ln 1000 + \ln 40) = 84.77$, since $2/\beta = 40$. So with probability at least 0.95, no query below $500 - 84.77 = 415.23$ fires (a $\top$ needs $f_k \geq T - \alpha$) and no query above $500 + 84.77 = 584.77$ is skipped (a $\bot$ needs $f_i < T + \alpha$). In the script's stream, every query has count 400 except query 700, which has 600. The counts of 400 are below 415.23 and cannot fire, and the count of 600 is above 584.77 and cannot be skipped, so the guarantee says the output is query 700 with probability at least 0.95; in 10,000 simulated runs it was query 700 every time.

The union bound behind α is nearly tight. The margin that the noises actually needed is $2\max(|\rho|, \max_i |\nu_i|)$, the smallest α for which every noise is below $\alpha/2$. Its 95% quantile over 10,000 simulated runs is 78.59, against 84.77. The alternative, answering all 1,000 queries with the Laplace mechanism at $\varepsilon/1000$ each, adds noise of scale 1,000 to counts of a few hundred.

22.7.6. Check yourself

- AboveThreshold answers 999 queries with $\bot$ and halts at the 1000th. How much of its budget did the 999 $\bot$ answers use? *(All of them together used ε_1, through the threshold noise; the $\top$ used ε_2. With 9 or 999,999 $\bot$ answers the cost would be the same.)*
- Why does the accuracy bound depend on k, while the privacy does not? *(Accuracy needs every query noise to be small at once, which costs a union bound over k queries. Privacy needs only the maximum of the noisy answers to move by at most Δ, which holds for any k.)*

22.8. When the sparse vector technique goes wrong

22.8.1. Context

The sparse vector technique is easy to state and easy to break. Lyu, Su & Li (2017) surveyed published variants and found that several were not differentially private at all, although each looked like a harmless optimization of AboveThreshold. Each bug removes one ingredient that a step of the proof above relies on. Seeing which step fails is the fastest way to review an implementation.

22.8.2. Statement

Table 22.6 lists the variants analysed by Lyu, Su & Li (2017) and what each changes.

Table 22.6.: Variants of the sparse vector technique and their actual privacy, from Lyu, Su & Li (2017).

Variant (as numbered by Lyu, Su & Li)	What it changes	Privacy shown by Lyu, Su & Li
Algorithm 1 (theirs); Algorithm 2 (Dwork & Roth)	Correct, with c answers above	ε
Algorithm 3 (Roth's 2011 lecture notes)	Outputs the noisy value $f_i + \nu_i$ for each $\top$	∞
Algorithm 4 (Lee & Clifton 2014)	Query noise does not grow with c	$\frac{1+6c}{4}\varepsilon$
Algorithm 5 (Stoddard et al. 2014)	No noise on the queries	∞
Algorithm 6 (Chen et al. 2015)	Never halts	∞

This section proves the last row for the **no-halt variant** of AboveThreshold: the same noise ($\rho \sim \mathrm{Lap}(2\Delta/\varepsilon)$, $\nu_i \sim \mathrm{Lap}(4\Delta/\varepsilon)$), but every query is answered and the run never stops. To show a mechanism is not private, it is enough to find one pair of neighbours and one output whose probability ratio is too large. The pair below is built so that every answer of the output $\bot^m\top^m$ (m answers "below", then m answers "above") is a little more likely under D: on D the first m queries are low, which makes their $\bot$ easier, and the last m are high, which makes their $\top$ easier. Each of the $2m$ answers contributes a small factor, and without a halt nothing stops the factors from multiplying. Take $\Delta = 1$, $2m$ queries, and neighbours with

$$f(D) = (\underbrace{0, \dots, 0}_{m}, \underbrace{1, \dots, 1}_{m}), \qquad f(D') = (\underbrace{1, \dots, 1}_{m}, \underbrace{0, \dots, 0}_{m}). \tag{22.15}$$

Each query changes by 1 between the two, so each has sensitivity 1. For the output $\bot^m\top^m$, the privacy loss grows at least linearly in m:

$$\ln \frac{\Pr[\bot^m\top^m \mid D]}{\Pr[\bot^m\top^m \mid D']} \geq \frac{m\varepsilon}{4}. \tag{22.16}$$

The privacy loss grows without bound in m, so the variant is not ε'-DP for any ε'.

22.8.3. Derivation

The answers are not independent, because they all compare against the same noisy threshold. But once the threshold is fixed they are, since each then depends only on its own fresh noise. So the proof fixes the threshold, multiplies the per-query probabilities, shows that the ratio is at least $e^{1/b}$ per pair of queries for every threshold, and averages over the threshold at the end. Let $b = 4/\varepsilon$, the query-noise scale with $\Delta = 1$, and let F and $S = 1 - F$ be the CDF and the survival function of $\mathrm{Lap}(b)$: $F(x) = \Pr[\nu \le x]$ and $S(x) = \Pr[\nu > x]$.

1. **Fix the threshold.** Condition on $T + \rho = t$. The query noises are independent, so the answers are independent. A query with value f says $\perp$ when $f + \nu < t$, that is $\nu < t - f$, with probability $F(t - f)$, and $\top$ with probability $S(t - f)$.
2. **The two conditional probabilities.** Under D the output needs m queries of value 0 to say $\perp$ and m of value 1 to say $\top$; under D' it needs m of value 1 to say $\perp$ and m of value 0 to say $\top$. Multiplying,

$$a_D(t) = F(t)^m \, S(t-1)^m, \qquad a_{D'}(t) = F(t-1)^m \, S(t)^m. \tag{22.17}$$

3. **Their ratio is a power.** $a_D(t)/a_{D'}(t) = h(t)^m$ with $h(t) = \frac{F(t)}{F(t-1)} \cdot \frac{S(t-1)}{S(t)}$. Both factors are at least 1, because F increases and S decreases: a query of value 0 is more likely to say $\perp$ than one of value 1, and a query of value 1 is more likely to say $\top$.
4. $h(t) \ge e^{1/b}$ **for every** t**.** The Laplace CDF is $F(x) = \frac{1}{2}e^{x/b}$ for $x \le 0$ and the survival function is $S(x) = \frac{1}{2}e^{-x/b}$ for $x \ge 0$; on each of those half-lines, moving x by 1 changes the value by exactly the factor $e^{1/b}$. There are three cases, according to where t sits. If $t \le 0$, both t and $t - 1$ are on the half-line where F is exponential, so $F(t)/F(t-1) = e^{1/b}$, and the second factor is at least 1. If $t \ge 1$, both $t - 1$ and t are on the half-line where S is exponential, so $S(t-1)/S(t) = e^{1/b}$, and the first factor is at least 1. If $0 < t < 1$, the two points straddle 0, and each factor gets part of the shift: $F(t) \ge \frac{1}{2}$ and $F(t-1) = \frac{1}{2}e^{(t-1)/b}$, so the first factor is at least $e^{(1-t)/b}$; likewise $S(t-1) \ge \frac{1}{2}$ and $S(t) = \frac{1}{2}e^{-t/b}$, so the second is at least $e^{t/b}$. The product is at least $e^{(1-t)/b+t/b} = e^{1/b}$.
5. **Average over the threshold.** The threshold noise has the same distribution under D and D', so $\Pr[\perp^m \top^m \mid D] = \mathbb{E}[a_D(T+\rho)] \ge e^{m/b}\,\mathbb{E}[a_{D'}(T+\rho)] = e^{m\varepsilon/4}\Pr[\perp^m \top^m \mid D']$, using $h^m \ge e^{m/b}$ at every threshold and $m/b = m\varepsilon/4$.

Why AboveThreshold's proof does not apply. Every $\top$ in the second half is a new comparison with the same threshold. Step 3 of the AboveThreshold proof can shift ρ once to fix all the $\perp$ answers, but each $\top$ needs its own noise shifted by up to 2Δ, at a cost of $e^{\varepsilon/2}$ each with this noise scale. The halt is what limits the number of $\top$ answers to one.

No query noise (Algorithm 5). Without ν_i even the halting version fails, with two queries: $f(D) = (0, 1)$ and $f(D') = (1, 0)$. Without query noise, a query with value f says $\top$ exactly when $f \ge T + \rho$, a deterministic comparison with the noisy threshold. The output $\perp\top$ needs $0 < T + \rho \le 1$ on D, which has positive probability, and $1 < T + \rho \le 0$ on D', which is impossible. A probability ratio of ∞ cannot be bounded by any e^{ε}. Exercise 5 in the exercise book computes the first probability.

22.8.4. Interpretation

Each ingredient of AboveThreshold protects one step of its proof. The fresh noise ν_k pays for the $\top$ (step 5 of the AboveThreshold proof): remove it and the $\top$ reveals a deterministic comparison, as in Algorithm 5. The halt limits the $\top$s to one: remove it and the cost per $\top$ accumulates, as in the no-halt variant. Publishing $f_k + \nu_k$ releases the very noise that the proof integrates over, as in Algorithm 3.

The bug is in the accounting, not in the noise. The no-halt variant adds exactly as much noise per query as AboveThreshold. A reviewer who checks only noise scales will not find it.

Empirical audits catch it, and you do not need the exact probabilities to run one. If a mechanism is ε-DP, then every event, including "the test says D", is at most e^{ε} times as likely under D as under D'. So any test whose probability of firing under D is more than e^{ε} times its probability under D' proves the mechanism is not ε-DP, and the log of that ratio is a lower bound on the true ε. This is the hypothesis-testing form of DP in The definition of differential privacy. A simple test, "are there more $\top$ answers in the second half than in the first?", already exceeds what $\varepsilon = 1$ allows once m is a few dozen (Figure 22.6, right).

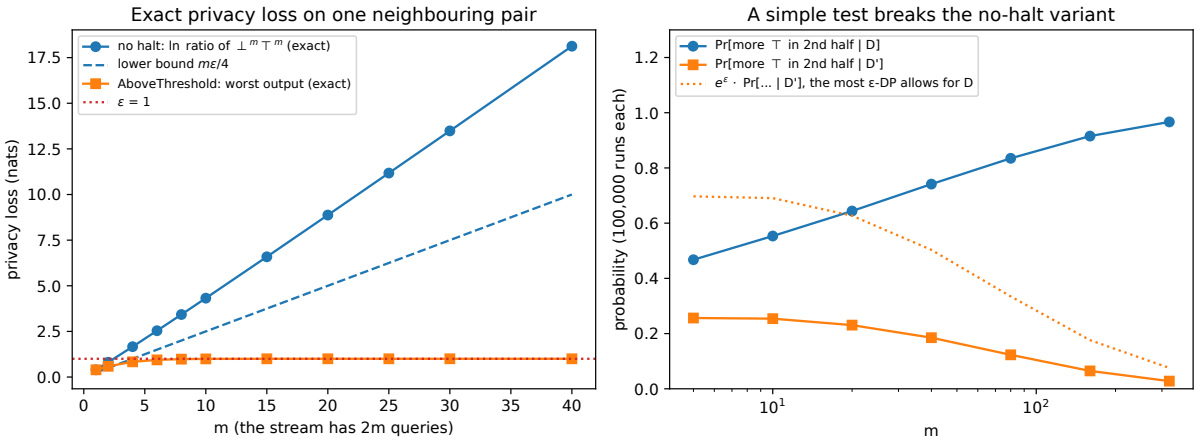


Figure 22.6.: Left: privacy loss on the neighbouring pair $f(D) = (0^m, 1^m)$, $f(D') = (1^m, 0^m)$ with $\varepsilon = 1$, computed exactly by integrating over the threshold noise. For the no-halt variant, the loss of the output $\bot^m\top^m$ grows linearly in m, above the lower bound $m\varepsilon/4$. For AboveThreshold, the largest loss over all outputs levels off at ε. Right: the no-halt variant under a simple test, the probability that the second half has more $\top$ answers than the first. Once the probability under D exceeds e^{ε} times the probability under D' (dotted), no ε-DP mechanism could produce it.

22.8.5. Example: the loss, computed and simulated

Table 22.7 gives the script's exact values for $\varepsilon = 1$ and $T = 0$. For AboveThreshold the table takes the largest privacy loss over all its possible outputs on this pair; for the no-halt variant it takes the loss of the one output $\bot^m\top^m$.

Table 22.7.: Exact privacy loss of AboveThreshold and of its no-halt variant on the counterexample, with $\varepsilon = 1$.

m	1	5	10	20	40
AboveThreshold, largest loss over outputs	0.3958	0.8998	0.9940	1.0000	1.0000
No halt, loss of $\bot^m\top^m$	0.3958	2.0985	4.3220	8.8741	18.1246
Lower bound $m\varepsilon/4$	0.25	1.25	2.50	5.00	10.00

At $m = 1$ the two mechanisms coincide, because $\bot\top$ is a complete AboveThreshold output: AboveThreshold halts after the $\top$ anyway. From then on the no-halt loss grows by about $18.1246/40 = 0.45$ nats (units of natural logarithm) for each pair of queries added, while AboveThreshold never exceeds ε. The lower bound is loose by a factor of a little under 2 (18.1246 against 10.00 at $m = 40$), because step 4 of the derivation used a lower bound on $h(t)$ that holds at every threshold, and at every threshold h is in fact larger.

The integrals are checked by simulation at $m = 2$: the exact probability of $\bot\bot\top\top$ under D is 0.0669, and 400,000 runs give 0.0671. The sign test of Figure 22.6, run 200,000 times per dataset in the script (the figure uses 100,000 runs per point), gives Pr[more $\top$ in the second half] of 0.5529 under D and 0.2552 under D' at $m = 10$, a log-ratio of 0.77; 0.7399 and 0.1832 at $m = 40$, a log-ratio of 1.40; and 0.9151 and 0.0659 at $m = 160$, a log-ratio of 2.63. From $m = 40$ on, the log-ratio is above 1, so this test alone proves that the variant is not 1-DP.

22.8.6. Check yourself

- A colleague's implementation of AboveThreshold reuses a single noise draw ν for all queries to "save randomness". Which step of the proof fails? *(Step 1: fixing $\nu_1, \dots, \nu_{k-1}$ would also fix ν_k, so the $\top$ could not be paid for by shifting its own noise. With one shared ν, every answer compares f_i with the same random level $T + \rho - \nu$, which is Algorithm 5's bug.)*
- Is releasing the index of the first $\top$ different from releasing the whole sequence $\bot^{k-1}\top$? *(No. Each determines the other. The bug in the no-halt variant is answering after the $\top$, not the output format.)*

22.9. Code: check every mechanism by computation and simulation

The script code/selection.py computes every number in this chapter and draws every figure. The exponential mechanism's probabilities are exact sums. Laplace report-noisy-max and the sparse vector technique are integrated exactly on a fine grid over one noise variable (the winner's noise, or the threshold noise), and simulations check the integrals. From the repository root:

```
uv run python code/part03-dp-foundations/selection.py
```

Output (seed 0), summarized in Table 22.8:

Table 22.8.: The script's output with the default seed.

Experiment	Result
Laplace threshold shift, $b = s = 1$	Ratio 1.6321 at $t = 0$, 2.5357 at $t = 0.7$, 2.7183 at $t = 1$ and $t = 2$
Exponential mechanism on the vote, $\varepsilon = 1$	Table 22.3; normalizer ratio 1.1790; largest loss 0.3353 (monotone weights: 0.6585)
Exponential mechanism on 50 models	Table 22.4; $\Pr[\text{gap} \geq 13.82] = 9.12 \times 10^{-4}$ against $e^{-3} = 0.0498$; tied profile at gap 20: 0.0022 against 0.0023
Laplace report-noisy-max, 50 models, $\varepsilon = 1$	$b = 2$: Pr[best] 0.4241, expected gap 1.40 (simulated 1.39). $b = 1$: 0.6463 and 0.54
Laplace report-noisy-max on the vote, $b = 1$	Largest loss 0.6674 ($b = 2$: 0.3626)
Gumbel-max, five candidates	Within 0.0008 of the exponential mechanism over 10^6 runs; Laplace at total variation 0.0216
AboveThreshold, $k = 1000$, $\beta = 0.05$	$\alpha = 84.77$; simulated 95% quantile 78.59; alarm stream correct in 10,000 of 10,000 runs
AboveThreshold against no halt	Table 22.7; sign test log-ratios 0.77, 1.40, 2.63 at $m = 10, 40, 160$

tests/test_selection.py checks that the output still matches.

Four things stand out. First, the privacy proofs are right, and some are tight: on every neighbouring pair the script tries, the largest privacy loss stays below ε, and AboveThreshold reaches it exactly. Second, the utility bounds are right but often loose. The exponential mechanism's tail bound is tight only when all runners-up are tied; for spread-out scores it overestimates the probability of a large gap by orders of magnitude, because it treats every bad candidate as if it were barely bad. Third, Gumbel noise is the exponential mechanism, and Laplace noise is a close relative; neither of the two private argmaxes dominates the other. Fourth, one missing line breaks the sparse vector technique. Without the halt, the privacy loss grows linearly in the number of queries, and a two-line test detects it.

22.10. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), §3.3 (report-noisy-max, Claim 3.9), §3.4 (the exponential mechanism, Theorems 3.10 and 3.11) and §3.6 (the sparse vector technique: AboveThreshold, Theorems 3.23 and 3.24, and the algorithms Sparse and NumericSparse). The proofs above follow theirs; go there for the version with $\sqrt{c}$ cost under advanced composition.

- McSherry & Talwar, "Mechanism Design via Differential Privacy" (FOCS 2007). The exponential mechanism, with the continuous version and its use in auctions.
- Lyu, Su & Li, "Understanding the Sparse Vector Technique for Differential Privacy" (Proceedings of the VLDB Endowment, 2017). The general algorithm with c answers above and a budget split, the monotone improvement, the survey of incorrect variants in Table 22.6, and the optimal split. The best single source on the technique.
- Dwork, Naor, Reingold, Rothblum & Vadhan, "On the Complexity of Differentially Private Data Release" (STOC 2009). Where the sparse vector idea first appears.
- Dwork, Feldman, Hardt, Pitassi, Reingold & Roth, "The Reusable Holdout: Preserving Validity in Adaptive Data Analysis" (Science 2015). Thresholdout, an application of the sparse vector technique to validation sets reused across many experiments.
- Durfee & Rogers, "Practical Differentially Private Top-k Selection with Pay-what-you-get Composition" (NeurIPS 2019). The one-shot Gumbel top-c and its equivalence to repeated exponential mechanisms.
- McKenna & Sheldon, "Permute-and-Flip: A New Mechanism for Differentially Private Selection" (NeurIPS 2020), and Ding et al., "The Permute-and-Flip Mechanism is Identical to Report-Noisy-Max with Exponential Noise" (2021).
- Maddison, Tarlow & Minka, "A* Sampling" (NeurIPS 2014): the Gumbel-max trick in machine learning, with its history going back to Gumbel and to Yellott (1977).
- Near & Abuah, *Programming Differential Privacy*, the chapters on the exponential mechanism and the sparse vector technique: short implementations alongside Dwork & Roth.

22.11. Cheat sheet

$\mathcal{R}$ has k candidates with scores $u(D, r)$ of sensitivity Δ; $\mathrm{OPT} = \max_r u(D, r)$. Monotone scores all move the same way between neighbours. Table 22.9 collects the mechanisms.

Table 22.9.: The selection mechanisms of this chapter.

Mechanism	Output	Privacy	Accuracy
Exponential mechanism	r with probability $\propto e^{\varepsilon u(D,r)/(2\Delta)}$	ε; $e^{\varepsilon u/\Delta}$ for monotone scores	Gap $\leq \frac{2\Delta}{\varepsilon}(\ln k + t)$ w.p. $1 - e^{-t}$; mean gap $\leq \frac{2\Delta}{\varepsilon}(\ln k + 1)$
Report-noisy-max, Laplace	$\arg\max_r(u + \mathrm{Lap}(2\Delta/\varepsilon))$	ε; $\mathrm{Lap}(\Delta/\varepsilon)$ for monotone scores	Gap $\leq 2b \ln(k/\beta)$ w.p. $1 - \beta$
Report-noisy-max, Gumbel	$\arg\max_r(u + \mathrm{Gumbel}(2\Delta/\varepsilon))$	Identical to the exponential mechanism	Same
AboveThreshold	Index of the first $f_i + \mathrm{Lap}(2\Delta/\varepsilon_2) \geq T + \mathrm{Lap}(\Delta/\varepsilon_1)$, then halt	$\varepsilon_1 + \varepsilon_2$, any number of $\perp$	$\alpha = 8\Delta(\ln k + \ln \frac{2}{\beta})/\varepsilon$ for the equal split

Mechanism	Output	Privacy	Accuracy
Sparse (c answers above)	AboveThreshold restarted c times at ε/c	ε by basic composition	α with ε/c in place of ε

Proof pattern. Fix every noise except one (Fact 2). The output happens when that noise clears a bar. The bar is built from a maximum, which moves by at most Δ however many terms it has (Fact 4). Shifting a Laplace variable by s costs $e^{s/b}$ (Fact 3).

Rules of thumb

- Release positions, not values. Every released noisy value needs its own budget.
- The number of candidates costs $\ln k$ in accuracy and nothing in privacy.
- Use 2Δ unless you have checked that the scores are monotone under your neighbouring relation; counts are monotone under add/remove, a histogram under replace is not.
- To sample the exponential mechanism, add Gumbel noise and take the argmax.
- An implementation of the sparse vector technique must draw fresh noise for every query, keep one threshold noise per run, halt after the allowed number of $\top$ answers, and never publish the noisy values it compared.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

23. Beyond global sensitivity

23.1. Why this matters

The Laplace and Gaussian mechanisms answer a numeric query f by adding random noise to the true answer, and they set the size of that noise by the **global sensitivity**: the most that changing one record can move f, looking at *every* pair of neighbouring datasets that could exist, not just at yours. In symbols, $\Delta f = \max_{D \simeq D'} |f(D) - f(D')|$, where $D \simeq D'$ means the two datasets are neighbours. For counts and bounded sums this is the right scale: one person changes a count by at most 1, whatever the data. For many other statistics it is hopeless, for three different reasons.

The first is that the worst case can be absurd. The **median** of ages in $[0, 100]$ has global sensitivity 100. To see it, take a dataset in which half the people are 0 and half are 100, with one more person at 0 so that the median is 0. Replacing that one person by a 100-year-old moves the median from one end to the other. So the Laplace mechanism must add noise of scale $100/\varepsilon$, which swamps any real answer, even though no realistic dataset of ages looks like this. The second is that the worst case can be infinite. The **mean of unbounded data** (incomes, session lengths, gradient norms) can be moved arbitrarily far by one record, because that record can be arbitrarily large, so no amount of Laplace noise makes it private. The third is that the worst case can be unknown. For a **statistic you can only compute**, such as an estimator fitted by an algorithm, nobody knows how to bound how far one record moves it.

Yet on the dataset you actually hold, the median of 1,001 ages typically moves by a tenth of a year when one person changes, because with that many people the ages near the middle are packed tightly together. That is the **local sensitivity**, and it is hundreds of times smaller than the global one. This chapter asks: **can you add noise on the scale of the data you have, rather than the worst dataset there could be, and still satisfy differential privacy (DP)?**

Recall what DP asks for. A randomized mechanism M is (ε, δ)-DP if, for every pair of neighbouring datasets $D \simeq D'$ and every set S of outputs, $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta$: one record can change the probability of any outcome by at most a factor e^{ε}, up to a small slack δ. With $\delta = 0$ this is **pure** ε-DP. The answer to the question is yes, with one idea that organizes the whole chapter:

The noise scale is itself a function of the data, so it must be private too.

If you calibrate noise to the local sensitivity of your dataset, the *size* of the noise reveals something about the dataset: an observer who sees a release with very wide noise learns that the data were of the kind that has a large local sensitivity. That is enough to break the guarantee. Each technique of this chapter repairs this in a different way:

- **Smooth sensitivity** replaces the local sensitivity by an upper bound that changes slowly between neighbours, so that the noise scale leaks little.

- **Propose-test-release** proposes a noise scale, and privately tests whether the dataset is far from any dataset where that scale would be too small.
- **Subsample-and-aggregate** makes the sensitivity small by construction: run the statistic on disjoint blocks and privately combine the block answers, each of which one record can affect only once.
- **Clipping** forces bounded sensitivity on unbounded data, at the price of a bias that you trade against the noise.

Table 23.1 lists where each result is used.

Table 23.1.: The results of this chapter, what each gives you, and where it is used.

Result	What it gives you	Where it is used
Local sensitivity and its leak	Why "noise proportional to how sensitive *this* dataset is" fails	The motivation for everything below
Smooth sensitivity and admissible noise	Data-dependent noise with a proof: Cauchy-like noise for ε-DP, Laplace for (ε, δ)-DP	Private medians, quantiles and robust statistics
Smooth sensitivity of the median	An exact $O(n^2)$ formula	Private quantiles; choosing clip bounds
Propose-test-release	Release with small noise if the data are stable, refuse otherwise	Robust statistics (Dwork & Lei 2009); stability-based release
Subsample-and-aggregate	Privacy for a black-box statistic	Private model training by ensembles of teachers; estimators without known sensitivity
Clipping	Bounded sensitivity for unbounded data, and the bias–variance trade-off of the clip bound	Private means and sums; per-example gradient clipping in private training

Throughout, datasets have a fixed size n and $D \simeq D'$ means that D' is obtained from D by **replacing** one record (the bounded relation of The definition of differential privacy). The **Hamming distance** $d(D, D')$ counts the records in which two datasets differ, so neighbours are datasets at distance 1, and $d(D, D'') = k$ means you can walk from D to D'' by changing one record at a time in k steps. Every proof below compares the output densities $p_{M(D)}$ and $p_{M(D')}$ of a mechanism on two neighbours, point by point. The natural quantity for that is the log of their ratio at an output y, the **privacy loss**: it says how much more likely y is under D than under D', and pure ε-DP holds when it never exceeds ε. In symbols, $L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}$.

23.2. Toolkit: four facts every derivation uses

23.2.1. Fact 1 — The Laplace mechanism with global sensitivity

If one record never moves f by more than Δ, then adding Laplace noise of scale Δ/ε is enough. Formally, if $|f(D) - f(D')| \le \Delta$ for all $D \simeq D'$, then $f(D) + \mathrm{Lap}(\Delta/\varepsilon)$ is ε-DP. This is the Laplace mechanism, and the reason is short. The Laplace density with centre μ and scale b is $\frac{1}{2b}e^{-|y-\mu|/b}$, so with $b = \Delta/\varepsilon$ the normalizing constants cancel in the ratio of the two densities, and the privacy loss is $(|y - f(D')| - |y - f(D)|)\varepsilon/\Delta$. The triangle inequality says $|y - f(D')| \le |y - f(D)| + |f(D) - f(D')|$, so the bracket is at most $|f(D) - f(D')| \le \Delta$, and the loss is at most ε at every y. A density that is at most e^{ε} times another at every point has at most e^{ε} times its probability on every set (integrate over the set), which is ε-DP. Every technique in this chapter ends by applying this fact, or a cousin of it, to something whose sensitivity *is* bounded.

23.2.2. Fact 2 — A tail bound on the privacy loss gives (ε, δ)

Some mechanisms in this chapter have a privacy loss that is at most ε for almost every output, but not for all of them. This fact says that is enough for (ε, δ)-DP, with δ the probability of the rare outputs where the loss is too large. For every set S of outputs,

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \Pr_{y \sim M(D)}\left[L_{D,D'}(y) > \varepsilon\right]. \tag{23.1}$$

The proof splits S into the outputs where the loss is small and those where it is large: $S_1 = S \cap \{L \le \varepsilon\}$ and $S_2 = S \cap \{L > \varepsilon\}$. On S_1 the loss is at most ε, which says exactly that the densities satisfy $p_{M(D)} \le e^{\varepsilon} p_{M(D')}$ there, so integrating over S_1 gives $\Pr[M(D) \in S_1] \le e^{\varepsilon} \Pr[M(D') \in S_1] \le e^{\varepsilon} \Pr[M(D') \in S]$. On S_2 you give up comparing and just bound the probability: $\Pr[M(D) \in S_2]$ is at most the probability that the loss exceeds ε at all, $\Pr_{M(D)}[L > \varepsilon]$. Adding the two pieces gives Equation 23.1.

So if the privacy loss exceeds ε only with probability δ, the mechanism is (ε, δ)-DP. This "bad event" argument is behind every δ in this chapter.

23.2.3. Fact 3 — Distances to a set have sensitivity 1

A question of the form "how many records would have to change before something bad happens?" has an answer that one record can change by at most 1. Formally, for any set $\mathcal{B}$ of datasets (the "bad" ones), let $\mathrm{dist}(D, \mathcal{B}) = \min_{D'' \in \mathcal{B}} d(D, D'')$ be the number of changes needed to reach the nearest of them. Then $|\,\mathrm{dist}(D, \mathcal{B}) - \mathrm{dist}(D', \mathcal{B})| \le 1$ for $D \simeq D'$.

The reason is that a path from D' to a bad dataset can always go through D first, at the cost of one extra step. In terms of the Hamming distance, which satisfies the triangle inequality, $d(D', D'') \le d(D', D) + d(D, D'') = 1 + d(D, D'')$ for every D''. Minimizing both sides over $D'' \in \mathcal{B}$ gives $\mathrm{dist}(D', \mathcal{B}) \le 1 + \mathrm{dist}(D, \mathcal{B})$, and the same argument with D and D' exchanged gives the other direction. So such a count can be released with $\mathrm{Lap}(1/\varepsilon)$ noise by Fact 1. Propose-test-release is built on it.

23.2.4. Fact 4 — Changing k records moves each order statistic by at most k places

If you change k records, the j-th smallest value can move, but only as far as the $(j-k)$-th or $(j+k)$-th smallest value of the original data. To state it, sort the values of D as $x_1 \le ... \le x_n$ (the **order statistics**), all in $[0, \Lambda]$. Ranks $j-k$ or $j+k$ can fall outside $1, ..., n$, so pad with $x_i = 0$ for $i \le 0$ and $x_i = \Lambda$ for $i > n$: "below the smallest" means "possibly as low as 0", and "above the largest" means "possibly as high as Λ". If D'' differs from D in at most k records and has sorted values $y_1 \le ... \le y_n$, then

$$x_{j-k} \le y_j \le x_{j+k} \quad \text{for every } j. \tag{23.2}$$

For the upper bound, count the values below x_{j+k}. At least $j+k$ values of D are $\le x_{j+k}$ (when $j+k > n$ the padding sets $x_{j+k} = \Lambda$, which bounds every value, so the claim is trivially true). Changing k records removes at most k of them, so at least j values of D'' are still $\le x_{j+k}$, and that says the j-th smallest value of D'' is at most x_{j+k}: $y_j \le x_{j+k}$. The lower bound is the same argument counting values above x_{j-k}. This fact pins down how far the median of a modified dataset can travel.

23.3. Local sensitivity, and why it leaks

23.3.1. Context

Global sensitivity takes a maximum over all pairs of neighbours, including pathological ones that look nothing like your data. The natural fix is to take the maximum only over the neighbours of the dataset you hold. That gives a much smaller number. But adding noise proportional to it is not private, and seeing exactly why is the key to everything else in this chapter.

23.3.2. Statement

The **local sensitivity** of f at D is the most that one record can move f starting from D itself, rather than from the worst dataset:

$$\mathrm{LS}_f(D) = \max_{D' :\ d(D,D')=1} |f(D) - f(D')|, \tag{23.3}$$

and the global sensitivity is its worst case over all datasets, $\Delta f = \max_D \mathrm{LS}_f(D)$.

For the median, one change can push the median at most one place up or down the sorted list, so its local sensitivity is the larger of the two gaps next to it. Take n values in $[0, \Lambda]$, with n odd and $m = (n+1)/2$ the rank of the median, so that the median is x_m. Then

$$\mathrm{LS}_{\mathrm{med}}(D) = \max\left(x_m - x_{m-1},\ x_{m+1} - x_m\right), \qquad \Delta\,\mathrm{med} = \Lambda. \tag{23.4}$$

23.3.3. Derivation

1. **The median of a neighbour lies between x_{m-1} and x_{m+1}.** This is Fact 4 with $k = 1$ change and rank $j = m$. So the median moves from x_m by at most the larger of the gaps to its two sorted neighbours: $|\text{med}(D') - x_m| \leq \max(x_m - x_{m-1}, x_{m+1} - x_m)$.
2. **Both ends are reached.** To show the bound is attained, move the median value itself to an extreme. Replace x_m by Λ: the sorted values become $x_1, \dots, x_{m-1}, x_{m+1}, \dots, x_n, \Lambda$, and the m-th of them is x_{m+1}. Replace x_m by 0 instead, and everything above it keeps its rank while the new 0 goes first, so the m-th value becomes x_{m-1}. So the bound of step 1 is attained, which is Equation 23.4.
3. **The global sensitivity is Λ.** Take m values equal to 0 and $m - 1$ equal to Λ. Then $x_m = 0$ and $x_{m+1} = \Lambda$, so Equation 23.4 gives Λ, and no dataset can do worse because every value lies in $[0, \Lambda]$, so no gap exceeds Λ.

23.3.4. Interpretation: the noise scale is a statistic

Consider the mechanism that adds Laplace noise scaled to the local sensitivity of the data it is given, $M_{\text{LS}}(D) = f(D) + \text{Lap}(\text{LS}_f(D)/\varepsilon)$. Its noise scale is the function LS_f of the data, and LS_f **itself can have large sensitivity**: one record can change the local sensitivity of the median from 0 to Λ. The mechanism then releases $f(D)$ with no noise on one dataset and with enormous noise on its neighbour, and the noise level alone tells the two apart, even when f is the same on both.

The degenerate case makes this stark (Nissim, Raskhodnikova & Smith 2007, Example 1). With $n = 5$ and $\Lambda = 100$, the dataset $D = (0, 0, 0, 0, 100)$ has median 0, and both gaps next to the median are 0, so $\text{LS} = 0$ and $M_{\text{LS}}(D) = 0$ exactly. Its neighbour $D' = (0, 0, 0, 100, 100)$ also has median 0, but the gap above the median is now 100, so $\text{LS} = 100$ and $M_{\text{LS}}(D')$ has a continuous distribution, which equals 0 with probability 0. The event "the output is exactly 0" has probability 1 under D and 0 under D'. No finite ε and no $\delta < 1$ cover that, because the DP inequality would need $1 \leq e^{\varepsilon} \cdot 0 + \delta$.

23.3.5. Example: two neighbours with the same median

The leak does not need a degenerate dataset. Take $\Lambda = 100$, $\varepsilon = 1$, and the two datasets of Table 23.2, which differ in one record and have the same median, but whose local sensitivities differ by a factor of six.

Table 23.2.: Two neighbouring datasets with the same median and very different local sensitivities.

Dataset	Values	Median	LS
D	10, 20, 30, 31, 90	30	$\max(30 - 20, 31 - 30) = 10$
D' (31 replaced by 90)	10, 20, 30, 90, 90	30	$\max(30 - 20, 90 - 30) = 60$

The two releases are $30 + \mathrm{Lap}(10)$ and $30 + \mathrm{Lap}(60)$. They have the same centre, so the privacy loss depends only on the distance $u = |y - 30|$ from it. Far from the centre the wide density is much larger than the narrow one, because the narrow one decays six times faster, so the loss should grow with u. Writing the two Laplace densities $\frac{1}{2b}e^{-u/b}$ with $b = 60$ and $b = 10$ and taking the log of their ratio:

$$L_{D',D}(y) = \ln \frac{e^{-u/60}/120}{e^{-u/10}/20} = -\ln 6 + u\Big(\frac{1}{10} - \frac{1}{60}\Big) = -1.792 + \frac{u}{12}. \tag{23.5}$$

The constant is $\ln(20/120) = -\ln 6 = -1.792$, and the slope is $\frac{6-1}{60} = \frac{1}{12}$. The loss grows without bound in u (Figure 23.1), so no pure ε can hold. It exceeds $\varepsilon = 1$ once $-\ln 6 + u/12 > 1$, that is once $u > 12(1 + \ln 6) = 33.50$. A Laplace variable of scale b lands more than u from its centre with probability $e^{-u/b}$, so this happens with probability $e^{-33.5/60} = 0.572$ under D' and $e^{-33.5/10} = 0.0351$ under D.

How large must δ be for this pair? The smallest δ for which the pair satisfies the $(1, \delta)$ inequality is the **hockey-stick divergence** of Divergences: the largest gap $\Pr[M(D') \in S] - e^1 \Pr[M(D) \in S]$ over all sets S, which is attained by taking S to be exactly the outputs where the loss exceeds 1. The script computes it as 0.477 in the direction D' against D (and 0.215 in the other direction). An adversary who sees a value more than 34 away from 30 can bet on D' and be right far more often than any reasonable (ε, δ) allows. This is the distinguishing game of Threat models and privacy games, in which an adversary who knows both candidate datasets sees one release and guesses which dataset produced it. Here it wins by looking at the spread of the noise, not its centre.

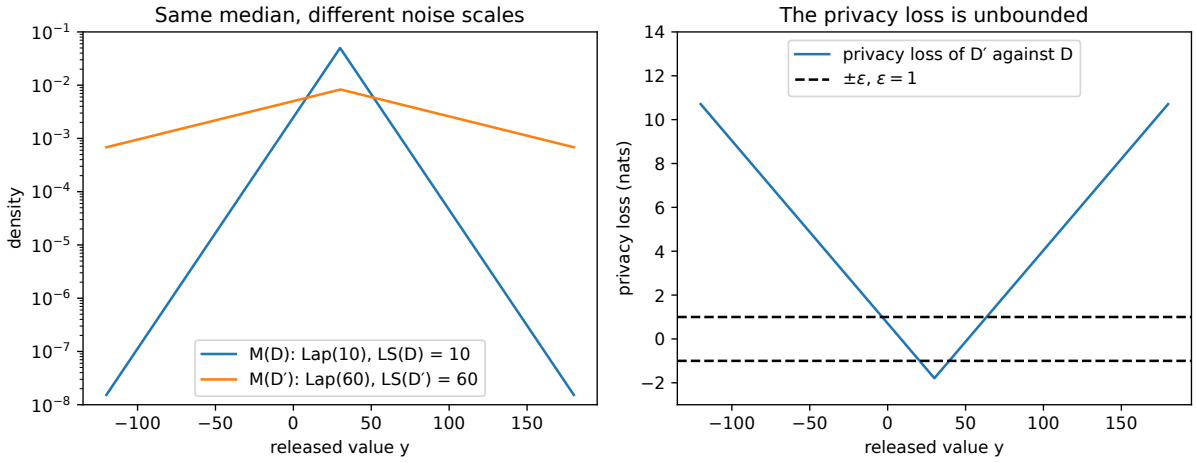


Figure 23.1.: Left: the densities of the median of D and of D' released with Laplace noise scaled to their own local sensitivities, 10 and 60. Right: the privacy loss $L_{D',D}(y)$ grows linearly in $|y - 30|$ and crosses $\varepsilon = 1$ at distance 33.5.

23.3.6. Check yourself

- Is $\mathrm{LS}_f(D) \le \Delta f$ always, and when are they equal? *(Always, since the global sensitivity is the maximum of the local one over all datasets. They are equal at the datasets where that maximum is attained, such as half zeros and half Λ for the median.)*

- Why doesn't the Laplace proof of Fact 1 go through for M_{LS}? *(It needs the same scale Δ on both sides. With scales $b \neq b'$ the loss contains $|y - f(D')|/b' - |y - f(D)|/b$, whose slopes no longer cancel, so it is unbounded in y.)*

23.4. Smooth sensitivity

23.4.1. Context

The leak came from the noise scale jumping between neighbours. So use a noise scale that cannot jump: an upper bound on the local sensitivity whose *logarithm* changes by at most β between neighbours, so that the scale itself changes by at most a factor e^{β}. A change by a bounded *factor* is the right kind of control, because a privacy loss is a log-ratio: rescaling the noise by e^{β} shifts log-densities by about β. Nissim, Raskhodnikova & Smith (2007) call such a bound smooth, find the smallest one, and show which noise distributions can be scaled by it. Two questions need answers: how to compute the bound, and which noise makes the result private.

23.4.2. Statement

A function S is a β-**smooth upper bound** on LS_f if it is never below the local sensitivity and changes by at most a factor e^{β} between neighbours:

$$S(D) \geq \mathrm{LS}_f(D) \quad \text{and} \quad S(D) \leq e^{\beta} S(D') \quad \text{for all } D \simeq D'. \tag{23.6}$$

The smallest such bound has an explicit form. If some dataset D'' at distance k has a large local sensitivity, a smooth bound cannot be small at D, because it can only decrease by a factor $e^{-\beta}$ per step on the way from D'' to D. So the bound at D must be at least $\mathrm{LS}_f(D'')e^{-\beta k}$ for every D'', and the β-**smooth sensitivity** takes exactly the largest of these:

$$S^{*}_{f,\beta}(D) = \max_{D''} \ \mathrm{LS}_f(D'')\, e^{-\beta\, d(D,D'')} \tag{23.7}$$

It is a β-smooth upper bound, and it is the smallest one: $S^{*}_{f,\beta}(D) \leq S(D)$ for every β-smooth upper bound S.

The mechanism adds noise from a fixed shape, stretched by $S(D)$:

$$M(D) = f(D) + \frac{S(D)}{\alpha}\, Z, \tag{23.8}$$

where Z is drawn from a fixed density h and $\alpha > 0$ is a constant that sets how many noise scales one record's shift is worth. Which h work is the content of the next two results. They differ in how heavy the tail of h is, and the derivation will show why that matters: the noise scale differs between neighbours, and a light-tailed density compared with a stretched copy of itself has a log-ratio that grows without bound in the tail, as in the example of Table 23.2.

Pure DP with Cauchy-like noise. Let $h_{\gamma}(z) \propto 1/(1 + |z|^{\gamma})$ with $\gamma > 1$, a density whose tails decay like the power $|z|^{-\gamma}$ ($\gamma = 2$ is the standard Cauchy). If S is a β-smooth upper bound and the sources of privacy loss together fit within the budget (the shift of the centre is worth $\gamma\alpha$,

and the change of scale is worth β through the density's normalizing factor plus $\gamma\beta$ through the stretch of its shape),

$$\gamma\alpha + (\gamma+1)\beta \le \varepsilon, \tag{23.9}$$

then Equation 23.8 is ε-DP. Nissim, Raskhodnikova & Smith (2007, Lemma 2.7) use $\alpha = \beta = \varepsilon/(2(\gamma+1))$, which satisfies Equation 23.9 with room to spare. For the Cauchy, $\gamma = 2$ gives $\alpha = \beta = \varepsilon/6$, the often-quoted release $f(D) + \frac{6}{\varepsilon} S^*_{f,\varepsilon/6}(D) \cdot Z$.

Approximate DP with Laplace noise. Let $Z \sim \mathrm{Lap}(1)$, the Laplace density $\frac{1}{2}e^{-|z|}$. Its tail is too light for pure DP, so the condition gains a term that charges for the rare draws of size above $\ln(1/\delta)$. If S is a β-smooth upper bound and

$$\beta + e^{\beta}\alpha + (e^{\beta} - 1)\ln(1/\delta) \le \varepsilon, \tag{23.10}$$

then Equation 23.8 is (ε, δ)-DP. Nissim, Raskhodnikova & Smith (2007, Lemma 2.9) prove a version with a slightly different split of the budget, $\alpha = \varepsilon/2$ and $\beta = \varepsilon/(2\ln(2/\delta))$. Because the third term is roughly $\beta\ln(1/\delta)$ for small β, here β must shrink like $1/\ln(1/\delta)$, and no $\beta > 0$ gives pure DP with Laplace noise.

23.4.3. Derivation

S^* is a smooth upper bound, and the smallest one.

1. *Upper bound.* One of the datasets in the maximum of Equation 23.7 is $D'' = D$ itself, at distance 0, which contributes $\mathrm{LS}_f(D)e^0 = \mathrm{LS}_f(D)$. A maximum is at least each of its terms, so $S^*_{f,\beta}(D) \ge \mathrm{LS}_f(D)$.
2. *Smooth.* Moving from D to a neighbour D' makes every dataset at most one step further away: $d(D', D'') \le d(D, D'') + 1$, by the triangle inequality of Fact 3. So each term $\mathrm{LS}_f(D'')e^{-\beta d(D', D'')}$ of $S^*(D')$ is at least $e^{-\beta}$ times the term for the same D'' in $S^*(D)$. Taking the maximum over D'' on both sides, $S^*(D') \ge e^{-\beta}S^*(D)$. Nothing in this argument depended on which of the two neighbours was called D, so running it with D and D' exchanged gives $S^*(D) \ge e^{-\beta}S^*(D')$, the smoothness condition.
3. *Smallest.* This is the argument that motivated the definition. Let S be any β-smooth upper bound and D'' any dataset at distance k from D. Walk from D'' to D through k neighbours; smoothness lets S drop by at most a factor $e^{-\beta}$ per step, so $S(D) \ge e^{-\beta k}S(D'')$, and $S(D'') \ge \mathrm{LS}_f(D'')$ because S is an upper bound. So $S(D) \ge e^{-\beta k}\mathrm{LS}_f(D'')$ for every D''; maximize over D''.

Pure DP for Cauchy-like noise. The plan is to write the privacy loss at an output y as a sum of three pieces, one for each way the two output distributions differ (their scales differ, their centres differ, and the noise draw that explains y is stretched), and to bound each piece by smoothness or by the upper-bound property.

1. *Write out the privacy loss.* Stretching Z by $S(D)/\alpha$ divides its density by that factor, so the density of $M(D)$ at y is $\frac{\alpha}{S(D)}h(z)$, where $z = \alpha\frac{y - f(D)}{S(D)}$ is the noise draw that would produce y from D. Likewise that of $M(D')$ is $\frac{\alpha}{S(D')}h(z')$ with $z' = \alpha\frac{y - f(D')}{S(D')}$. To compare the two

draws, substitute $y = f(D) + S(D)z/\alpha$ into z': $z' = \frac{S(D)}{S(D')}(z + \alpha\frac{f(D)-f(D')}{S(D)})$. That is, z' is z shifted and then stretched:

$$z' = e^\lambda(z + \Delta), \qquad e^\lambda = \frac{S(D)}{S(D')}, \quad \Delta = \alpha\,\frac{f(D) - f(D')}{S(D)}. \tag{23.11}$$

(Here Δ is this shift, not the global sensitivity.) The log of the ratio of the two densities splits into the ratio of the prefactors and the ratio of h at z and at z', and inserting $h(z+\Delta)$ in the middle splits the second part into the shift and the stretch:

$$L_{D,D'}(y) = \underbrace{\ln\frac{S(D')}{S(D)}}_{\text{scale}} + \underbrace{\ln\frac{h(z)}{h(z+\Delta)}}_{\text{sliding}} + \underbrace{\ln\frac{h(z+\Delta)}{h(e^\lambda(z+\Delta))}}_{\text{dilation}}. \tag{23.12}$$

2. *Size of the shift and the stretch.* Smoothness says $S(D)/S(D')$ lies between $e^{-\beta}$ and e^β, so $|\lambda| \le \beta$. And one record moves f by at most the local sensitivity, which is at most $S(D)$: $|f(D) - f(D')| \le \mathrm{LS}_f(D) \le S(D)$, so $|\Delta| \le \alpha$. This is the only place where "S is an upper bound on the local sensitivity" enters.
3. *Scale term:* at most β, by smoothness again.
4. *Sliding term.* The question is how fast $-\ln h$ can change when its argument shifts by $|\Delta|$. Write $\varphi(u) = \ln(1 + u^\gamma)$ for $u \ge 0$, so $-\ln h_\gamma(z) = \varphi(|z|) + \text{const}$. Its slope is $\varphi'(u) = \gamma u^{\gamma-1}/(1 + u^\gamma) \le \gamma$, because $u^{\gamma-1} \le 1 + u^\gamma$ (if $u \le 1$ the left side is at most 1; if $u > 1$ it is below u^γ). The mean value theorem says a function whose slope is at most γ in size changes by at most γ times the change in its argument. The argument changes from $|z|$ to $|z + \Delta|$, and these differ by at most $|\Delta|$ (the reverse triangle inequality). So $|\varphi(|z + \Delta|) - \varphi(|z|)| \le \gamma\big||z + \Delta| - |z|\big| \le \gamma|\Delta| \le \gamma\alpha$.
5. *Dilation term.* Here the question is how much h changes when its argument is stretched by e^λ. With $w = |z + \Delta|$ and $\lambda \ge 0$, $\frac{h(w)}{h(e^\lambda w)} = \frac{1+e^{\lambda\gamma}w^\gamma}{1+w^\gamma} \le e^{\lambda\gamma}$, since $(1 + ca)/(1 + a) \le c$ for $c \ge 1$ (multiply out: $1 + ca \le c + ca$). This bound holds for every w, however far in the tail, and that is what the power-law tail buys. For $\lambda < 0$ the stretch moves the argument towards 0, where h is larger, so the ratio is at most 1. Either way the term is at most $\gamma|\lambda| \le \gamma\beta$.
6. *Add up:* $L_{D,D'}(y) \le \beta + \gamma\alpha + \gamma\beta$ for **every** y. That is $\gamma\alpha + (\gamma+1)\beta$, so under Equation 23.9 it is at most ε, which is pure ε-DP. With $\alpha = \beta = \varepsilon/6$ and $\gamma = 2$ the bound is $\gamma\alpha + (\gamma+1)\beta = \frac{2}{6} + \frac{3}{6} = 0.8333\,\varepsilon$.

Approximate DP for Laplace noise. Steps 1–3 are unchanged. For $h(z) = \frac{1}{2}e^{-|z|}$ the log of a ratio of h values is just a difference of absolute values, so the sliding and dilation terms combine into one. Using $|e^\lambda(z + \Delta)| \le e^\lambda(|z| + |\Delta|)$, then $e^\lambda \le e^\beta$ and $|\Delta| \le \alpha$ from step 2:

$$\ln\frac{h(z)}{h(z')} = |e^\lambda(z + \Delta)| - |z| \le e^\lambda(|z| + |\Delta|) - |z| \le (e^\beta - 1)|z| + e^\beta\alpha. \tag{23.13}$$

Adding the scale term, $L_{D,D'}(y) \le \beta + e^\beta\alpha + (e^\beta - 1)|z|$, where z is the noise actually drawn, $Z \sim \mathrm{Lap}(1)$. This is **not** bounded: the stretch makes the Laplace log-density change by an amount proportional to $|z|$, so far in the tail the dilation term wins. But large draws are rare. A Lap(1) variable exceeds t in absolute value with probability e^{-t} (integrate the two tails of $\frac{1}{2}e^{-|z|}$; see Concentration inequalities), so $\Pr[|Z| > \ln(1/\delta)] = \delta$. Outside that event of probability δ, $|z| \le \ln(1/\delta)$ and the loss is at most the left side of Equation 23.10, hence at most ε. Fact 2 turns that into (ε, δ)-DP.

23.4.4. Interpretation: why heavy tails, and what admissibility means

Pure DP forces polynomial tails. The dilation step needed $h(w)/h(e^{\lambda}w)$ to be bounded for all w, however large. Write $h(w) \approx e^{-g(w)}$; the condition says $g(e^{\beta}w) - g(w)$ stays below some constant c. Applying it k times from $w = 1$ gives $g(e^{k\beta}) \le g(1) + kc$, so g can grow at most like $\frac{c}{\beta}\ln w$, and h can decay at most like a power of w: Cauchy-like. Laplace noise has $g(w) = w$, for which $g(e^{\beta}w) - g(w) = (e^{\beta} - 1)w$ grows without bound, and the Gaussian is worse. That is why both fail for pure DP and pay with δ.

The two conditions are what Nissim, Raskhodnikova & Smith call **admissibility**. They call h (α, β)-admissible when shifting it by at most α (sliding) and rescaling it by at most a factor $e^{\pm\beta}$ (dilation) each change probabilities by at most a factor $e^{\varepsilon/2}$ plus $\delta/2$. The derivation above bounds the two together, which gives slightly different constants.

The price of heavy tails shows up in the error. The Cauchy has no mean, so its error is reported through the median. For a standard Cauchy, $\Pr[|Z| \le t] = \frac{2}{\pi}\arctan t$, which is $\frac{1}{2}$ at $t = 1$ because $\arctan 1 = \pi/4$. So $|Z|$ has median 1, and the typical error is the noise scale S/α. But the tail is heavy: $\Pr[|Z| > 10] = 1 - \frac{2}{\pi}\arctan 10 = 0.0635$, so about one release in 16 is off by more than ten times the typical error. With Laplace noise the same probability is $e^{-10} \approx 4.5 \times 10^{-5}$.

Finally, β is a lever. A small β makes S^* larger, because the factor $e^{-\beta k}$ discounts distant datasets less and so lets more of them raise the maximum. In exchange it costs less privacy per unit of noise. Laplace noise needs $\beta \approx \varepsilon/(2\ln(1/\delta))$, much smaller than the Cauchy's $\varepsilon/6$, because in Equation 23.10 the change of scale is charged once for every unit of $\ln(1/\delta)$.

23.4.5. Computing S^* for the median

Equation 23.7 is a maximum over all datasets, which is not something you can enumerate. The way out is to group the datasets by their distance k from D: all datasets at distance k get the same discount $e^{-k\beta}$, so within a group you only need the largest local sensitivity, $A^{(k)}(D)$, and then the best discounted group:

$$S^*_\beta(D) = \max_{k=0,\dots,n} e^{-k\beta} A^{(k)}(D), \qquad A^{(k)}(D) = \max_{D'':\ d(D,D'')\le k} \mathrm{LS}(D''). \tag{23.14}$$

(Using "$\le k$" instead of "$= k$" changes nothing: a dataset at distance $j < k$ is counted again with the smaller weight $e^{-k\beta}$, which never beats its own weight $e^{-j\beta}$.)

For the median, $A^{(k)}$ asks: by changing k records, how wide a gap can you open next to the median? A gap is opened by moving the values that lie inside an interval of the sorted data out of it, to 0 or to Λ. With k changes you can empty out the k values between two order statistics $k + 1$ ranks apart, and you can choose how many to move down and how many up so that the emptied interval ends up next to the median. So the answer is the widest such window that sits close enough to rank m. With the padding of Fact 4, Nissim, Raskhodnikova & Smith (2007, Proposition 3.4) show

$$A^{(k)}(D) = \max_{0\le t\le k+1} (x_{m+t} - x_{m+t-k-1}). \tag{23.15}$$

Each window runs from rank $m + t - k - 1$ to rank $m + t$, and t slides it across the median. Each $A^{(k)}$ costs $O(k)$ to evaluate, one subtraction per window, and k never needs to exceed n: at $k = n$ the window with $t = m$ spans $x_0 = 0$ to $x_{n+1} = \Lambda$, so $A^{(n)} = \Lambda$, the largest possible value, and

larger k only lowers the weight. So the exact S^*_β takes $O(n^2)$ time. (An $O(n \log n)$ algorithm exists; the script uses the plain formula.)

Derivation of Equation 23.15. Steps 1–4 show that no dataset within distance k has a gap next to its median wider than the widest window, and step 5 shows that every window is achieved.

1. *What the gap looks like.* Let D'' be within distance k of D, with sorted values y_i. By Equation 23.4 its local sensitivity is one of its two gaps next to the median. Take the upper one, (y_m, y_{m+1}). The lower one needs no separate argument: reflecting every value, $x \mapsto \Lambda - x$, turns lower gaps into upper gaps and maps the set of windows in Equation 23.15 to itself.
2. *The gap contains at most k original values.* Assume the gap is positive (otherwise there is nothing to bound), so exactly m values of D'' are $\le y_m$. No value of D'' lies strictly inside (y_m, y_{m+1}), so every value of D strictly inside it must have been one of the k changed records. Call the values of D inside the gap $x_{a+1}, \dots, x_{a+j}$ with $j \le k$. Then $x_a \le y_m$ and $x_{a+j+1} \ge y_{m+1}$, so the gap is at most $x_{a+j+1} - x_a$, a window of the original data.
3. *The gap sits near rank m.* Now pin down a by counting values of D'' at or below y_m, of which there are exactly m. Each is either an unchanged value of D, and those are among $x_1, \dots, x_a$ (everything above x_a is inside the gap or beyond it), or one of at most k new values. So $m \le a + k$, that is $a \ge m - k$. In the other direction, at least $a - (k - j)$ of $x_1, \dots, x_a$ are unchanged (the j values inside the gap used up j of the k changes), and all of them are $\le y_m$, so $a - (k - j) \le m$, that is $a \le m + k - j$.
4. *Fit it in a window.* Let $i = \min(a, m)$, the lower end of the window to use. If $a \le m$, the window $[x_a, x_{a+k+1}]$ has $i = a \ge m - k$, so it is one of the windows of Equation 23.15 ($t = a + k + 1 - m \in [0, k + 1]$), and it contains $[x_a, x_{a+j+1}]$ because $j \le k$. If $a > m$, the window $[x_m, x_{m+k+1}]$ ($t = k + 1$) starts below x_a and contains $[x_a, x_{a+j+1}]$ because $a + j + 1 \le m + k + 1$ by step 3. Either way the gap is at most the right side of Equation 23.15.
5. *Every window is reached.* For a window $[x_\ell, x_u]$ with $\ell = m + t - k - 1$ and $u = m + t$, there are exactly $u - \ell - 1 = k$ values strictly between its ends. Change them: move $k + 1 - t$ of them to 0 and the rest to Λ (for $t = 0$, move all k to 0). Each value moved to 0 now sits below x_ℓ and raises its rank by one. So x_ℓ has rank $\ell + (k + 1 - t) = m$ and x_u rank $m + 1$ for $t \ge 1$ (for $t = 0$, ranks $m - 1$ and m). Either way the median sits at one end of an empty interval of length $x_u - x_\ell$, which is then one of its two gaps.

A walkthrough. Take $n = 5$ values on $[0, 10]$, $D = (1, 3, 4, 6, 9)$, so $m = 3$ and the median is 4, and pad with $x_i = 0$ for $i \le 0$ and $x_i = 10$ for $i \ge 6$. Table 23.3 evaluates Equation 23.15 window by window, with $\beta = 0.5$, and checks it against brute force: the script enumerates all $11^5 = 161{,}051$ datasets with values in $\{0, \dots, 10\}$, computes the local sensitivity of each and its distance from D, and takes the maximum for each k.

Table 23.3.: The smooth sensitivity of the median of $D = (1, 3, 4, 6, 9)$ on $[0, 10]$ with $\beta = 0.5$: the formula for $A^{(k)}$ against brute force over all datasets, and the discounted values whose maximum is $S^*_{0.5}$.

k	Windows $x_{m+t} - x_{m+t-k-1}$, $t = 0, \dots, k+1$	$A^{(k)}$	Brute force	$e^{-0.5k} A^{(k)}$
0	$4-3,\ 6-4$	2	2	2.00
1	$4-1,\ 6-3,\ 9-4$	5	5	**3.03**

k	Windows $x_{m+t} - x_{m+t-k-1}$, $t = 0, \dots, k+1$	$A^{(k)}$	Brute force	$e^{-0.5k} A^{(k)}$
2	$4-0,\ 6-1,\ 9-3,\ 10-4$	6	6	2.21
3	$4-0,\ 6-0,\ 9-1,\ 10-3,\ 10-4$	8	8	1.79
4	$\dots,\ 9-0,\ 10-1,\ \dots$	9	9	1.22
5	$\dots,\ 10-0,\ \dots$	10	10	0.82

The local sensitivity (the row $k = 0$) is 2, the gap from 4 up to 6. But one change, moving the 6 up to 10, opens a gap of $9 - 4 = 5$ next to the median, and even after the discount $e^{-0.5} = 0.6065$ for one step, $e^{-0.5} \times 5 = 3.03$ beats 2. Further rows open wider gaps but pay a larger discount. So $S^*_{0.5}(D) = 3.03$, attained at $k = 1$ (Figure 23.2, left).

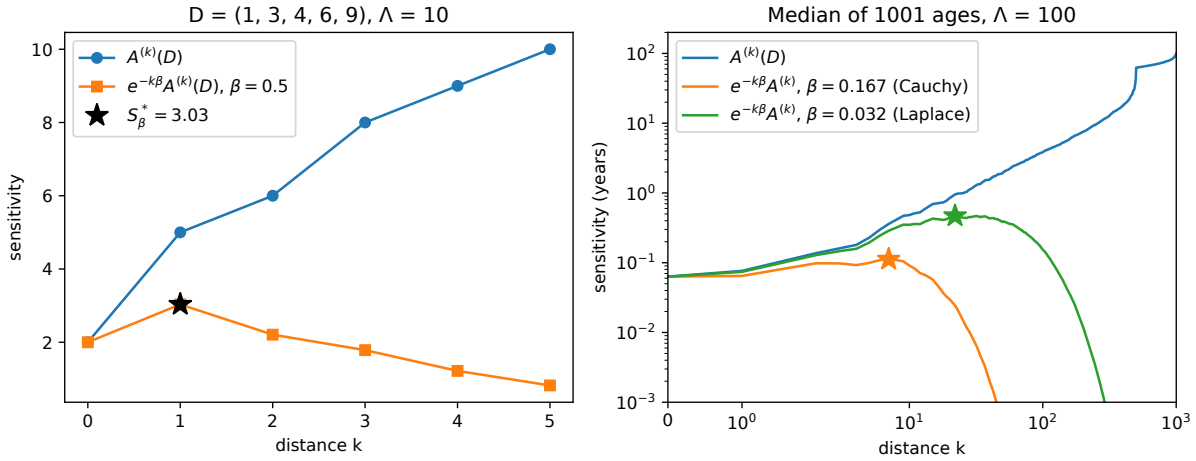


Figure 23.2.: Left: $A^{(k)}$ and $e^{-k\beta}A^{(k)}$ for the walkthrough dataset; the star marks $S^*_{0.5} = 3.03$. Right: the same for the median of 1,001 synthetic ages (a fresh draw) with $\beta = 1/6$ (Cauchy) and $\beta = 0.032$ (Laplace, $\delta = 10^{-6}$); the stars mark the maxima. The smaller β reaches further out.

23.4.6. Example: the median age

The script draws $n = 1001$ ages from a normal distribution with mean 38 and standard deviation 14, clipped to $[0, 100]$, and releases the median (36.96) with $\varepsilon = 1$. Each option is compared by its median absolute error. For noise bZ that is b times the median of $|Z|$: 1 for the Cauchy, as shown above, and $\ln 2$ for the Laplace, because $\Pr[|Z| > t] = e^{-t}$ equals $\frac{1}{2}$ at $t = \ln 2$.

With global sensitivity, Laplace noise has scale $\Lambda/\varepsilon = 100$ and median absolute value $100 \ln 2 = 69.31$ years: useless. The local sensitivity is 0.1386 years, but releasing with it is not private, as Section 23.3 showed.

Smooth sensitivity with Cauchy noise uses $\alpha = \beta = 1/6$. Then $S^*_{1/6} = 0.1632$, attained at $k = 7$: only slightly above the local sensitivity, because with this many people even seven changes cannot open a wide gap near the middle. The noise scale $6S^*/\varepsilon = 6 \times 0.1632 = 0.979$ is also the median absolute error (simulation: 0.980).

Smooth sensitivity with Laplace noise uses $\delta = 10^{-6}$ and $\alpha = 1/2$. The largest β allowed by Equation 23.10 is then 0.0322 (close to the 0.0345 of Nissim, Raskhodnikova & Smith). The smaller β lets more distant datasets count, so $S^*_{0.0322} = 0.5144$, attained at $k = 21$. The noise scale is $S^*/\alpha = 2S^*/\varepsilon = 1.029$ and the median absolute error $1.029 \ln 2 = 0.713$.

Both smooth releases are 70 to 100 times more accurate than the global one (69.31/0.979 and 69.31/0.713). Figure 23.3 shows the gain grows with n: the smooth noise falls roughly like $1/n$, the spacing of the data near the median, while the global noise stays put. For $n \lesssim 20$ the smooth noise is *worse* than the global one, because with few points a handful of changes can open a gap of nearly the whole range, so S^* can be close to Λ, and then the factor $1/\alpha$ multiplies it.

23.4.7. Check yourself

- Why does $S^*_\beta(D) > 0$ even when $\mathrm{LS}(D) = 0$? *(Some dataset at finite distance k has positive local sensitivity, and it contributes $e^{-k\beta}$ times that. For the median, $A^{(n)} = \Lambda$ gives $S^* \geq e^{-n\beta}\Lambda > 0$, which also means the scale in Equation 23.8 is never zero.)*
- In the Cauchy derivation, where is "upper bound" used and where is "smooth" used? *(Upper bound bounds the shift: $|\Delta| \leq \alpha$. Smoothness bounds the scale term and the stretch $|\lambda| \leq \beta$.)*

23.5. Propose-test-release

23.5.1. Context

Smooth sensitivity needs a formula like Equation 23.15, and those are known for only a few functions. Dwork & Lei (2009) take a cruder route that needs much less. Guess a bound b on the local sensitivity, check privately that the data are not near any dataset where the guess fails, and only then release with noise of scale b/ε. If the check fails, refuse to answer. The check is private because it is a "how many changes until something bad" count, which has sensitivity 1 by Fact 3.

23.5.2. Statement

Fix a proposed bound $b > 0$, a privacy parameter $\varepsilon > 0$ and a failure probability $\delta \in (0, 1)$. The quantity to test is the number of records you must change before the local sensitivity can exceed b, that is, how far D is from the nearest dataset where the proposal would be too small:

$$\mathrm{dist}_b(D) = \min\left\{ d(D, D'') : \ \mathrm{LS}_f(D'') > b \right\}. \tag{23.16}$$

Propose-test-release adds Laplace noise to this distance and compares it with a threshold $\ln(1/\delta)/\varepsilon$, chosen so that the noise alone pushes a distance of 0 over it only with probability $\delta/2$. It outputs one of two things:

- $\perp$ ("no answer") if $\text{dist}_b(D) + \text{Lap}(1/\varepsilon) \leq \ln(1/\delta)/\varepsilon$;
- $f(D) + \text{Lap}(b/\varepsilon)$ otherwise.

It is $(2\varepsilon, \delta/2)$-DP, so in particular $(2\varepsilon, \delta)$-DP. For the median, $\text{dist}_b(D) = \min\{k : A^{(k)}(D) > b\}$, directly from Equation 23.15: $A^{(k)}$ is the largest local sensitivity within k changes, so the first k where it exceeds b is the distance to the first dataset where b fails.

23.5.3. Derivation

The idea is to split neighbouring pairs into two cases. If the proposal b is right for this pair, the release is an ordinary Laplace mechanism. If it is wrong, both datasets sit right at an unstable point, the distance is 0, and the test almost always refuses.

1. **The test is ε-DP.** dist_b is a distance to the set of datasets whose local sensitivity exceeds b, so it has sensitivity 1 (Fact 3), and adding $\text{Lap}(1/\varepsilon)$ is the Laplace mechanism (Fact 1). Whether the test passes is a post-processing of that noisy value: a function of it that does not look at the data again, which cannot increase the privacy loss.
2. **Case 1: $|f(D) - f(D')| \leq b$.** Then the release step is also ε-DP (Fact 1 with $\Delta = b$). The output is a function of two independent ε-DP outputs, the noisy distance and the noisy value. Basic composition says that running two mechanisms with independent noise costs the sum of their εs, because the privacy loss of the pair is the sum of the two losses; post-processing says the function of them costs nothing more (Post-processing, group privacy and composition). So the mechanism is 2ε-DP on this pair.
3. **Case 2: $|f(D) - f(D')| > b$.** Then both $\text{LS}_f(D) > b$ and $\text{LS}_f(D') > b$, because each local sensitivity is the largest change over that dataset's neighbours, and each of D, D' is a neighbour of the other. So $\text{dist}_b(D) = 0$, and the test passes only if $\text{Lap}(1/\varepsilon) > \ln(1/\delta)/\varepsilon$. A $\text{Lap}(1/\varepsilon)$ variable exceeds t with probability $\frac{1}{2}e^{-\varepsilon t}$ (one tail of the density), so this has probability $\frac{1}{2}e^{-\ln(1/\delta)} = \delta/2$.
4. **Bound case 2.** For any set S of outputs, $M(D)$ lands in S either by refusing (if $\perp \in S$) or by releasing, and the release happens with probability at most $\delta/2$ by step 3. So $\Pr[M(D) \in S] \leq \Pr[M(D) = \perp]\,\mathbf{1}[\perp \in S] + \delta/2$, where $\mathbf{1}[\perp \in S]$ is 1 if $\perp$ is in S and 0 otherwise. The refusal is a post-processing of the ε-DP test of step 1, so the first term is at most $e^{\varepsilon}\Pr[M(D') = \perp]\,\mathbf{1}[\perp \in S] \leq e^{2\varepsilon}\Pr[M(D') \in S]$.
5. **Combine:** in both cases $\Pr[M(D) \in S] \leq e^{2\varepsilon}\Pr[M(D') \in S] + \delta/2$.

The "release the median for free" failure of the previous section is now confined to step 3: a dataset whose local sensitivity exceeds the proposal leaks only when a Laplace draw lands $\ln(1/\delta)/\varepsilon$ above zero.

23.5.4. Interpretation

Propose-test-release is all or nothing. When the data are stable, you get noise of scale b/ε, set by your proposal rather than by the data, so the noise scale reveals nothing. When they are not, you get $\perp$, and $\perp$ itself is a private answer: it says only that the data were probably near an unstable dataset.

The threshold the distance must clear is large. The test passes with probability $\frac{1}{2}$ when the distance equals the threshold, because the Laplace noise is positive half the time. With $\varepsilon = 1$

and $\delta = 10^{-6}$ that threshold is $\ln(10^6) = 13.82$, so the test needs $\mathrm{dist}_b(D) \gtrsim 13.82$ to pass with probability $\frac{1}{2}$. The data must be stable against more than a dozen changes, not just one.

The proposal b must not depend on the data. It is a proposal made before looking. Choosing b as, say, twice the observed local sensitivity makes the noise scale of the release a function of the data again, and reintroduces the leak of the previous section through the back door.

Compared with smooth sensitivity, propose-test-release needs only the distance to instability, which is often easier to compute or bound than S^*. It pays 2ε instead of ε, and it can refuse.

23.5.5. Example: the median age

On the 1,001 ages, propose $b = 1$ year. The first k with $A^{(k)} > 1$ is $k = 21$, so $\mathrm{dist}_1(D) = 21$, comfortably above the threshold 13.82. The test fails only if the noise drops the distance by more than $21 - 13.82$, which a Lap(1) draw does with probability $\frac{1}{2}e^{-(21-13.82)}$. So the pass probability is $1 - \frac{1}{2}e^{-(21-13.82)} = 0.999621$, and the release has noise Lap(1), with median absolute error $\ln 2 = 0.693$ years at a total budget of $2\varepsilon = 2$. On a sample of 101 ages, $\mathrm{dist}_1 = 1$, and the test passes only if the noise adds more than $13.82 - 1$, which has probability $\frac{1}{2}e^{-(13.82-1)} = 1.36 \times 10^{-6}$: there, one year is far too optimistic a proposal. Figure 23.3 (right) shows the pass probability switching from 0 to 1 between $n = 500$ and $n = 1000$.

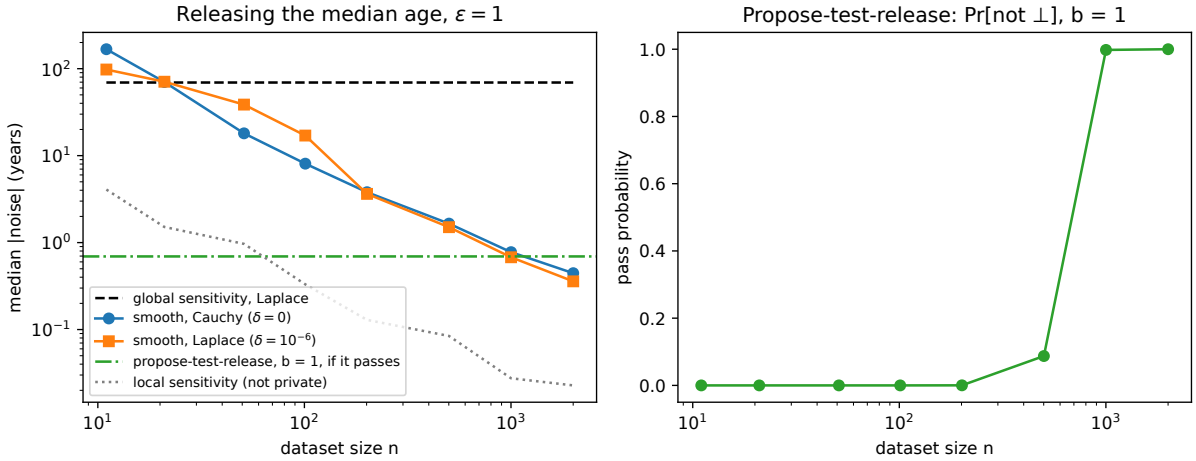


Figure 23.3.: Left: median absolute noise when releasing the median age with $\varepsilon = 1$, by global sensitivity, smooth sensitivity with Cauchy noise and with Laplace noise ($\delta = 10^{-6}$), and propose-test-release with $b = 1$ when it passes, averaged over 10 synthetic datasets per n. The dotted line is the non-private local sensitivity. Right: the probability that propose-test-release with $b = 1$ answers.

23.5.6. Check yourself

- Why does step 3 need *both* local sensitivities to exceed b, and why is that automatic? *(If either were at most b, the pair would fall into case 1. It is automatic because $|f(D) - f(D')|$ is one of the terms in the maximum defining each local sensitivity.)*

- Why does propose-test-release cost 2ε while smooth sensitivity costs ε? *(It runs two Laplace mechanisms, one for the test and one for the release, and composes them. Smooth sensitivity folds the cost of a data-dependent scale into the single noise draw.)*

23.6. Subsample-and-aggregate

23.6.1. Context

Sometimes you cannot analyse f at all: it is an estimator fitted by an optimizer, a trained model, or a pipeline you did not write. Subsample-and-aggregate (Nissim, Raskhodnikova & Smith 2007; Smith 2011) treats f as a black box. It runs f on disjoint pieces of the data and privately combines the answers. One record lives in one piece, so it can change only one answer, and the aggregator needs to be private only with respect to changing one of its inputs.

23.6.2. Statement

Split the n records into B disjoint blocks of n/B records, by a partition fixed in advance (for example by position, or by a hash of a record identifier). Compute the block answers $z_j = f(\text{block } j)$, clip each to a range $[0, R]$ (replace values below 0 by 0 and above R by R), and release their average plus Laplace noise. One record can move one clipped answer by at most R, so it moves the average by at most R/B, and that sets the noise:

$$M(D) = \frac{1}{B}\sum_{j=1}^{B} \operatorname{clip}_{[0,R]}(z_j) + \operatorname{Lap}\Big(\frac{R}{B\varepsilon}\Big). \tag{23.17}$$

This is ε-DP for any f.

How accurate is it, as an estimate of a target θ? Three things go wrong, and they add up in the mean squared error. Each block sees only n/B records, so f on a block can be biased in a way that f on the whole dataset is not; averaging the blocks keeps some random sampling error; and the Laplace noise has variance $2(R/(B\varepsilon))^2$. Writing z for the output of f on one block,

$$\underbrace{\big(\mathbb{E}[z] - \theta\big)^2}_{\text{bias of } f \text{ on } n/B \text{ records}} + \underbrace{\frac{\operatorname{Var}(z)}{B}}_{\text{sampling}} + \underbrace{2\Big(\frac{R}{B\varepsilon}\Big)^2}_{\text{noise}}, \tag{23.18}$$

when the clipping is inactive and the blocks are independent samples.

23.6.3. Derivation

1. **One record touches one block.** Replacing record i changes only the block that contains it, because the partition does not depend on the values. So the vector $(z_1, \dots, z_B)$ changes in at most one coordinate.
2. **The aggregate has sensitivity R/B.** After clipping, that coordinate moves by at most R, the width of the clip range, and dividing by B in the average divides that by B.
3. **Privacy:** Fact 1 with $\Delta = R/B$.

4. **Error.** The released value minus θ is the sum of three pieces: $\mathbb{E}[z]-\theta$ (a constant, the bias), $\frac{1}{B}\sum_j(z_j-\mathbb{E}z)$ (mean zero; it is an average of B independent terms, each with variance $\mathrm{Var}(z)$, so its variance is $\mathrm{Var}(z)/B$) and the Laplace noise (mean zero, variance $2(R/(B\varepsilon))^2$, since a $\mathrm{Lap}(s)$ variable has variance $2s^2$). Squaring the sum gives the three squares plus cross terms. Each cross term is a product involving at least one mean-zero piece that is independent of the other factor, so it averages to zero. That leaves Equation 23.18.

23.6.4. Interpretation

The number of blocks sets the trade-off: more blocks means less noise but more bias. The noise falls like $1/B$, because one record's influence on the average is diluted over more blocks. The sampling term hardly moves: for estimators whose variance scales like one over the sample size, a block estimate has variance about B times that of f on all n records, and averaging B of them divides it by B again, so the term stays near $\mathrm{Var}(f \text{ on } n \text{ records})$. The bias of f on small blocks grows with B, because the blocks get smaller. The best B balances the first and the last.

The privacy proof never looks inside f. This is what makes the method useful for trained models: an ensemble of "teacher" models, each trained on a disjoint shard and combined by a noisy vote (private aggregation of teacher ensembles, Papernot et al. 2017), is subsample-and-aggregate with a vote as the aggregator.

The range R is still a global bound, on the output of f rather than on the data, and a loose R means large noise. Nissim, Raskhodnikova & Smith replace the clipped average by a "center of attention" aggregator with smooth sensitivity, which adapts to how tightly the z_j cluster. Smith (2011) shows that for asymptotically normal estimators a version of this method loses nothing in the leading-order error.

23.6.5. Example: estimating a rate

You observe $n = 10{,}000$ inter-arrival times, independent Exponential with rate $\theta = 1$, and want to estimate θ privately with $\varepsilon = 1$. The maximum-likelihood estimate is one over the sample mean, $1/\bar{x}$, whose global sensitivity is infinite: a dataset of tiny times has a huge $1/\bar{x}$, and one record changes it by an arbitrary amount.

To fill in Equation 23.18 you need the mean and variance of $1/\bar{x}$ on a block of $n_b = n/B$ records. The block sum $n_b\bar{x}$ is a sum of n_b independent exponentials with rate θ, which has the gamma distribution with shape n_b and rate θ: $n_b\bar{x} \sim \mathrm{Gamma}(n_b, \theta)$. The inverse moments of a gamma variable G with shape k are $\mathbb{E}[1/G] = \theta/(k-1)$ and $\mathbb{E}[1/G^2] = \theta^2/((k-1)(k-2))$: integrate g^{-1} or g^{-2} against the gamma density $\theta^k g^{k-1}e^{-\theta g}/\Gamma(k)$, recognize what is left as a gamma density with shape $k-1$ or $k-2$, and use $\Gamma(k) = (k-1)\Gamma(k-1)$. Since $z = 1/\bar{x} = n_b/G$, multiplying by n_b and n_b^2 gives

$$\mathbb{E}[z] = \frac{n_b\theta}{n_b - 1}, \qquad \text{bias } \frac{\theta}{n_b - 1}, \qquad \mathrm{Var}(z) = \frac{n_b^2\theta^2}{(n_b-1)^2(n_b-2)}. \tag{23.19}$$

The variance is $\mathbb{E}[z^2] - \mathbb{E}[z]^2 = n_b^2\theta^2\left(\frac{1}{(n_b-1)(n_b-2)} - \frac{1}{(n_b-1)^2}\right)$, which simplifies to the form shown. With clip range $R = 5$, Table 23.4 and Figure 23.4 show the trade-off. For example, with $B = 10$

blocks the noise standard deviation is $\sqrt{2}\,R/(B\varepsilon) = \sqrt{2} \times 0.5 = 0.7071$, and the bias on blocks of 1,000 is $1/999 = 0.0010$.

Table 23.4.: Error of subsample-and-aggregate for the rate of an exponential distribution, $n =$ 10,000, $\theta = 1$, $R = 5$, $\varepsilon = 1$: the three terms of Equation 23.18, their combination, and a simulation with 1,000 repetitions.

Blocks B	Block size	Bias	Sampling sd	Noise sd	Root-mean-square error	Simulated
10	1,000	0.0010	0.0100	0.7071	0.7072	0.7205
100	100	0.0101	0.0102	0.0707	0.0722	0.0733
250	40	0.0256	0.0105	0.0283	0.0396	0.0401
1,000	10	0.1111	0.0124	0.0071	0.1120	0.1114

The best B on the script's grid is 250, with error 0.0396, where the bias and the noise are of similar size. A non-private estimate has standard deviation about $\theta/\sqrt{n} = 0.0100$, so privacy at $\varepsilon = 1$ costs a factor of four here, for an estimator whose sensitivity you never had to bound.

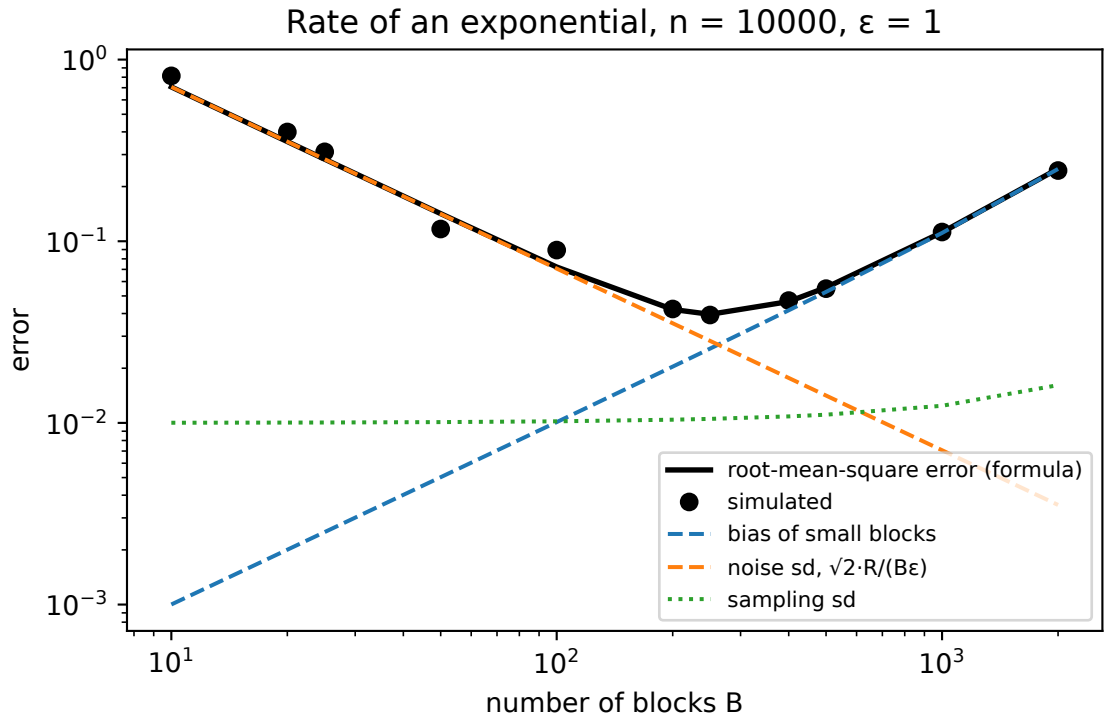


Figure 23.4.: Root-mean-square error of subsample-and-aggregate for the exponential rate against the number of blocks B (formula and simulation), with its three components: the bias of the estimator on small blocks, the Laplace noise, and the sampling error.

23.6.6. Check yourself

- Why must the partition into blocks not depend on the data values? *(Otherwise replacing one record could reshuffle many records between blocks and change many z_j, breaking step 1.)*
- Why does the sampling term in Table 23.4 barely move with B? *(The average of B block estimates, each with variance about $\theta^2 B/n$, has variance about θ^2/n whatever B is. Only the higher-order terms of the inverse-gamma variance depend on the block size.)*

23.7. Clipping

23.7.1. Context

The mean is the most common statistic of all, and on unbounded data its global sensitivity is infinite. **Clipping** forces a bound: replace each value x by $\min(x, C)$ before averaging. The sensitivity becomes C/n, and Laplace noise of scale $C/(n\varepsilon)$ makes the mean private. Nothing is free, though. Clipping changes the quantity you estimate, and the choice of C trades that bias against the noise. The same move, applied to per-example gradients rather than to values, is the heart of private model training in later parts.

23.7.2. Statement

Take non-negative data $X_1, \dots, X_n$ drawn independently from a distribution with mean μ. The clipped mean averages the clipped values and adds noise for sensitivity C/n:

$$M(D) = \frac{1}{n}\sum_{i=1}^{n} \min(x_i, C) + \mathrm{Lap}\Big(\frac{C}{n\varepsilon}\Big) \tag{23.20}$$

It is ε-DP. Its error has the same three parts as in subsample-and-aggregate. Clipping cuts off the part of each value above C, written $(x - C)_+ = \max(x - C, 0)$, and its average is the bias. The clipped values still vary from sample to sample, and the noise adds its own variance. So the mean squared error is

$$\mathrm{err}(C)^2 = \underbrace{\mathbb{E}[(X - C)_+]^2}_{\text{bias}^2} + \underbrace{\frac{\mathrm{Var}(\min(X, C))}{n}}_{\text{sampling}} + \underbrace{\frac{2C^2}{n^2\varepsilon^2}}_{\text{noise}}. \tag{23.21}$$

Raising C shrinks the bias and grows the noise, so the best C is where the two rates of change balance, which is where the derivative of Equation 23.21 in C is zero. Here $\Pr[X > C]$ is the fraction of values that clipping touches. The best C solves

$$\Pr[X > C]\Big(\mathbb{E}[(X - C)_+] - \frac{C - \mathbb{E}[\min(X, C)]}{n}\Big) = \frac{2C}{n^2\varepsilon^2}. \tag{23.22}$$

23.7.3. Derivation

1. **Sensitivity.** Each clipped value lies in $[0, C]$, so replacing one record changes the sum by at most C and the mean by at most C/n. Fact 1 gives ε-DP.
2. **The bias.** Clipping removes exactly the excess above C: $x - \min(x, C) = (x - C)_+$. Taking expectations, $\mu - \mathbb{E}[\min(X, C)] = \mathbb{E}[(X - C)_+]$: the mass that clipping cuts off.
3. **Error decomposition.** As in the subsample-and-aggregate derivation, the error is a constant (minus the bias), a mean-zero sampling term with variance $\mathrm{Var}(\min(X, C))/n$, and independent Laplace noise with variance $2(C/(n\varepsilon))^2$. Squaring and averaging kills the cross terms, since each contains a mean-zero factor independent of the other: Equation 23.21.
4. **Derivatives in C.** Raising C by a small amount h raises $\min(X, C)$ by h exactly when $X > C$ and leaves it alone otherwise, so $\frac{d}{dC}\mathbb{E}[\min(X, C)] = \Pr[X > C]$. By the same reasoning $\frac{d}{dC}\mathbb{E}[\min(X, C)^2] = 2C \Pr[X > C]$. The bias is $\mathbb{E}[(X - C)_+] = \mu - \mathbb{E}[\min(X, C)]$, whose derivative is $-\Pr[X > C]$, so by the chain rule the bias term has derivative $-2\,\mathbb{E}[(X - C)_+] \Pr[X > C]$. The variance is $\mathbb{E}[\min(X, C)^2] - \mathbb{E}[\min(X, C)]^2$, so the sampling term has derivative $\frac{1}{n}(2C \Pr[X > C] - 2\,\mathbb{E}[\min(X, C)] \Pr[X > C]) = \frac{2}{n} \Pr[X > C](C - \mathbb{E}[\min(X, C)])$. The noise term has derivative $\frac{4C}{n^2\varepsilon^2}$.
5. **Set the sum to zero** and divide by 2: Equation 23.22.

23.7.4. Interpretation

The bias does not shrink with n, but the noise does. So the best C grows with n: with more data you can afford to clip less. For large n the sampling part of Equation 23.22, which has an extra $1/n$, drops out, and the condition reads $\Pr[X > C]\,\mathbb{E}[(X - C)_+] \approx 2C/(n\varepsilon)^2$. The right side shrinks as $n\varepsilon$ grows, so the left side must shrink too, which pushes C out into the tail.

The bias has a sign. Clipping only from above always lowers values, so it always underestimates. For data on both sides of zero, clipping to $[-C, C]$ cuts off mass on both sides, and the two cuts cancel only for distributions symmetric about 0, so only those are estimated without bias.

Choosing C is itself a data-dependent decision. Setting C to the observed maximum, or to the empirical 99th percentile, makes the noise scale a function of the data, and the leak of Section 23.3 returns. Choose C from domain knowledge, or spend part of the budget on a private quantile: the exponential mechanism, or the smooth-sensitivity and propose-test-release methods above applied to an order statistic.

Clipping is the honest version of "bounded data". Every "assume $x \in [0, \Lambda]$" in earlier chapters is a clip in disguise, and its bias is the price of that assumption.

23.7.5. Example: mean income

Incomes (in thousands) follow a log-normal distribution with $\ln X \sim \mathcal{N}(\ln 40, 0.8^2)$: the log of an income is normal. The median is $e^{\ln 40} = 40$, because the exponential preserves the median. The mean is larger, $e^{\ln 40 + 0.32} = 55.085$, because a log-normal with $\ln X \sim \mathcal{N}(\eta, s^2)$ has mean $e^{\eta + s^2/2}$ and here $s^2/2 = 0.64/2 = 0.32$; the gap reflects a heavy right tail.

Every term of Equation 23.21 needs averages of X and X^2 over the values below C. For a log-normal these partial moments have a closed form, $\mathbb{E}[X^j \mathbf{1}\{X \le c\}] = e^{j\eta + j^2 s^2/2}\,\Phi((\ln c - \eta - j s^2)/s)$,

where Φ is the standard normal cumulative distribution function and $\mathbf{1}\{X \le c\}$ is 1 when $X \le c$ and 0 otherwise. To get it, write $X = e^Y$ with Y normal, so the integral is $\int_{-\infty}^{\ln c} e^{jy}\varphi(y)\,dy$ with φ the normal density of Y; completing the square in the exponent turns $e^{jy}\varphi(y)$ into $e^{j\eta + j^2 s^2/2}$ times a normal density with mean shifted to $\eta + js^2$, whose integral up to $\ln c$ is the Φ term (as in Probability distributions). Table 23.5 uses these with $n = 1000$ and $\varepsilon = 1$.

Table 23.5.: Error of the clipped private mean of log-normal incomes, $n = 1000$, $\varepsilon = 1$, for four clip bounds: the three terms of Equation 23.21, their combination, and a simulation with 5,000 repetitions.

Clip bound C	$\Pr[X > C]$	Bias	Sampling sd	Noise sd	Root-mean-square error	Simulated
40 (the median)	0.5000	23.415	0.335	0.057	23.417	23.411
257.23 (99th percentile)	0.0100	0.924	1.462	0.364	1.767	1.747
331.34 (optimal)	0.0041	0.438	1.538	0.469	1.666	1.649
1000	0.0000	0.006	1.645	1.414	2.169	2.134

Clipping at the median is a disaster: half the incomes are cut back, and the bias of 23 dwarfs everything. The optimum, $C^* = 331.34$, cuts off only 0.4% of the population and has total error 1.666, only 1% above the standard deviation 1.649 of the unclipped, non-private mean (the ratio is $1.666/1.649 \approx 1.01$). Above the optimum, the noise takes over: its standard deviation $\sqrt{2}\,C/(n\varepsilon)$ grows linearly in C, reaching 1.414 at $C = 1000$ (Figure 23.5, left). The optimum grows with n, as the interpretation predicted: 177.5 at $n = 100$, 557.3 at $n = 10{,}000$ and 876.3 at $n = 100{,}000$ (Figure 23.5, right).

23.7.6. Check yourself

- Under the add/remove relation, why is a clipped *sum* easier to release than a clipped mean? *(Adding or removing one record changes the sum by at most C, but it also changes n, so the mean's denominator is not public. One releases a noisy sum and a noisy count and divides.)*
- If you double n, roughly how does the noise term change, and which way does C^* move? *(The noise standard deviation halves at fixed C, so the balance in Equation 23.22 shifts toward a larger C^* with less bias.)*

23.8. Code: check every number by computation

The script code/sensitivity.py computes every number in this chapter and draws its figures. Local and smooth sensitivity, the noise parameters, propose-test-release, subsample-and-aggregate and the clipped mean are each small typed functions; the table calls them once and the figures loop over them. From the repository root:

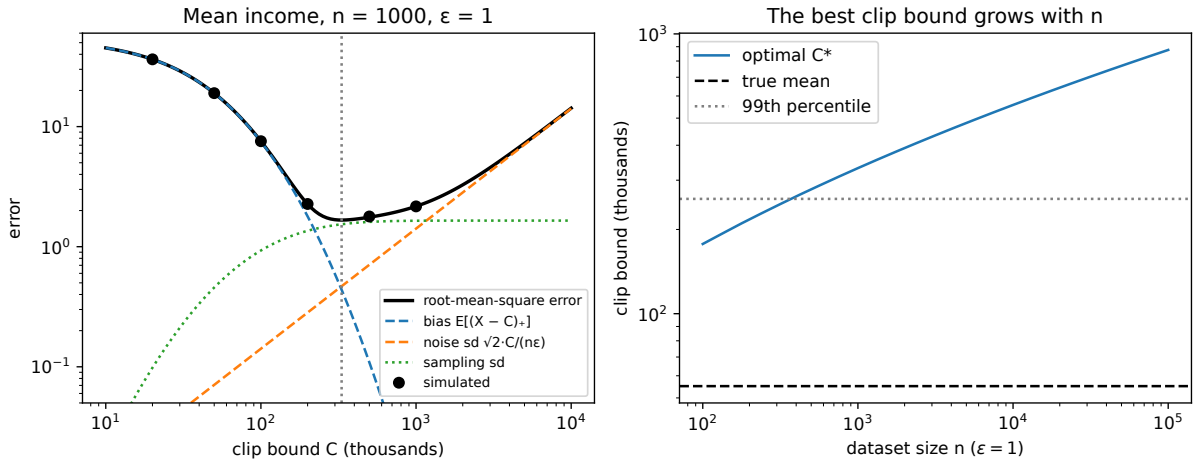


Figure 23.5.: Left: root-mean-square error of the clipped private mean of log-normal incomes against the clip bound C, with its bias, noise and sampling components and simulated points; the dotted line marks the optimum $C^* = 331$. Right: the optimal clip bound as a function of the dataset size.

```
uv run python code/part03-dp-foundations/sensitivity.py
```

Table 23.6 shows the output with seed 0.

Table 23.6.: Results of the chapter's script with seed 0.

Experiment	Result
Local-sensitivity noise on D, D' of Table 23.2, $\varepsilon = 1$	LS $= 10$ and 60; $\delta(1) = 0.477$ (D' against D) and 0.215
$A^{(k)}$ for $D = (1, 3, 4, 6, 9)$, $\Lambda = 10$	Formula equals brute force over 11^5 datasets for every k; $S^*_{0.5} = 3.03$ at $k = 1$
Noise parameters, $\varepsilon = 1$, $\delta = 10^{-6}$	Cauchy $\alpha = \beta = 0.1667$, loss bound 0.8333; Laplace $\alpha = 0.5$, $\beta = 0.0322$
Median of 1,001 ages	local sensitivity 0.1386; Cauchy scale 0.979 (simulated median error 0.980); Laplace median error 0.713; global 69.31
Propose-test-release, $b = 1$	$n = 1001$: distance 21, pass probability 0.999621; $n = 101$: distance 1, pass probability 1.36×10^{-6}
Subsample-and-aggregate, exponential rate	Best $B = 250$ on the grid, error 0.0396, against 0.0100 without privacy
Clipped mean of incomes, $n = 1000$	$C^* = 331.34$, error 1.666 (simulated 1.649); 99th percentile 1.767; median 23.417

tests/test_sensitivity.py checks that the output still matches this table.

What to take from it. The first lesson is that a data-dependent noise scale can leak more than the answer: in the leak example both datasets have the same median, so the whole $\delta = 0.477$ comes from the spread of the noise. The repair for the median rests on an exact formula, and the $O(n^2)$ formula for $A^{(k)}$ matches brute force at every distance k, so the smooth sensitivities it produces can be trusted. With it, smooth sensitivity and propose-test-release buy about two orders of magnitude over global sensitivity for the median of a thousand points, though they lose to it on tiny datasets, where a few changes can open a gap across the whole range. Finally, every method ends in a bias–noise trade-off you can compute: the number of blocks and the clip bound both have optima you can find in closed form before touching the data.

23.9. References

- Nissim, Raskhodnikova & Smith, "Smooth Sensitivity and Sampling in Private Data Analysis" (STOC 2007; full version on the authors' pages): local and smooth sensitivity, admissible noise (Lemmas 2.6–2.9), the median (Proposition 3.4) and the original sample-and-aggregate framework
- Dwork & Lei, "Differential Privacy and Robust Statistics" (STOC 2009): propose-test-release, and private robust estimators of scale, location and regression built on it
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 7: propose-test-release and subsample-and-aggregate in textbook form
- Near & Abuah, *Programming Differential Privacy*, chapter on local sensitivity: the same ideas with short code
- Smith, "Privacy-Preserving Statistical Estimation with Optimal Convergence Rates" (STOC 2011): subsample-and-aggregate for asymptotically normal estimators
- Bun & Steinke, "Average-Case Averages: Private Algorithms for Smooth Sensitivity and Mean Estimation" (NeurIPS 2019): better noise distributions for smooth sensitivity, under concentrated DP
- Amin, Kulesza, Muñoz Medina & Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (ICML 2019): the clipping trade-off when users contribute many records
- Papernot, Abadi, Erlingsson, Goodfellow & Talwar, "Semi-supervised Knowledge Transfer for Deep Learning from Private Training Data" (ICLR 2017): subsample-and-aggregate for model training

23.10. Cheat sheet

Table 23.7 collects the results. Datasets have fixed size n and neighbours differ by one replaced record.

Table 23.7.: Summary of the results in this chapter.

Result	Statement	Privacy
Local sensitivity	$\mathrm{LS}_f(D) = \max_{d(D,D')=1} \lvert f(D) - f(D') \rvert$; noise scaled by it is **not** private	—
Median	$\mathrm{LS} = \max(x_m - x_{m-1}, x_{m+1} - x_m)$; $\Delta = \Lambda$	—
Smooth sensitivity	$S^*_\beta(D) = \max_{D''} \mathrm{LS}(D'') e^{-\beta d(D,D'')}$, the smallest β-smooth upper bound	—
Median, exact	$S^*_\beta = \max_{k \le n} e^{-k\beta} \max_{0 \le t \le k+1} (x_{m+t} - x_{m+t-k-1})$, $O(n^2)$	—
Cauchy-like noise	$f + (S/\alpha)Z$, $h \propto 1/(1+\lvert z \rvert^\gamma)$, $\gamma\alpha + (\gamma+1)\beta \le \varepsilon$; Cauchy: $6S^*_{\varepsilon/6}/\varepsilon$	ε
Laplace noise	$f + (S/\alpha)\,\mathrm{Lap}(1)$, $\beta + e^\beta\alpha + (e^\beta - 1)\ln(1/\delta) \le \varepsilon$	(ε, δ)
Propose-test-release	$\perp$ if $\mathrm{dist}_b + \mathrm{Lap}(1/\varepsilon) \le \ln(1/\delta)/\varepsilon$, else $f + \mathrm{Lap}(b/\varepsilon)$	$(2\varepsilon, \delta/2)$
Subsample-and-aggregate	Mean of B clipped block outputs $+\mathrm{Lap}(R/(B\varepsilon))$; error = bias of small blocks + sampling + noise	ε
Clipped mean	$\frac{1}{n}\sum \min(x_i, C) + \mathrm{Lap}(C/(n\varepsilon))$; bias $\mathbb{E}[(X-C)_+]$	ε

Rules of thumb

- If the noise scale depends on the data, ask what the noise scale reveals. If you cannot answer, use one of the four repairs.
- Pure DP with a data-dependent scale needs polynomial tails; light tails need δ and a much smaller $\beta \approx \varepsilon/(2\ln(1/\delta))$.
- Smooth sensitivity pays off once the data are dense around the statistic: for the median, once n is in the hundreds.
- Choose clip bounds and block counts before looking at the data, by the error formula, or with a separate slice of the privacy budget.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

24. Discrete and floating-point-safe mechanisms

24.1. Why this matters

Every privacy proof so far was a statement about real numbers. The Laplace mechanism adds a real-valued $\mathrm{Lap}(b)$ draw to a real-valued $f(D)$, and the proof compares two densities on the real line. A computer has neither. It has floating-point numbers, a finite set with uneven gaps, and a source of random bits. The program that "samples $\mathrm{Lap}(b)$" outputs a distribution on that finite set, and nothing in the proof says anything about it.

The gap is not a rounding nuisance. Mironov (2012) showed that the textbook Laplace sampler, run in double precision, leaks $f(D)$ through the low-order bits of its output. Some outputs can be produced from D and not from D', and an observer who sees one knows which dataset was used. No ε covers that.

The organizing idea of this chapter is that **differential privacy (DP) is a property of the output distribution the code actually produces**, so the code must produce exactly the distribution the proof is about. There are two ways to get there. You can keep floating point and repair the output: round it to a coarse grid and clamp it, so that every reachable output is reachable from every input. Or you can leave floating point behind: work with integer-valued queries and sample integer noise exactly, using only integer and rational arithmetic and uniform random integers. The second route gives the discrete Laplace and the discrete Gaussian mechanisms. Their privacy proofs are as clean as the continuous ones, and their samplers produce exactly the distribution in the proof. Table 24.1 lists the results.

Table 24.1.: The results of this chapter, what each says, and where it is used.

Result	What it says	Where it is used
The support argument (Equation 24.4)	Outputs reachable from D but not from D' force $\delta \geq \Pr[M(D) \in S]$	Auditing any implementation; counterexample tests
Floating-point Laplace	The textbook sampler's outputs depend on $f(D)$ through their low-order bits	Why libraries don't ship it
The snapping mechanism	Rounding to a grid $\Lambda \geq b$ and clamping restores a bounded privacy loss	Laplace noise when the output must be a float
Exact Bernoulli($e^{-\gamma}$)	A coin with probability $e^{-\gamma}$ from uniform integers, for rational γ	The building block of both samplers below

Result	What it says	Where it is used
The discrete Laplace mechanism	$\varepsilon = \Delta_1 f / b$ for integer-valued f, sampled exactly	Counts and histograms
The discrete Gaussian mechanism	Same privacy loss as the continuous Gaussian at integer points; $\delta(\varepsilon)$ close to it; variance below σ^2	Census releases; noise added before secure aggregation
The exact discrete Gaussian sampler	Rejection from the discrete Laplace, at most about 2 proposals per draw	Any deployment of the discrete Gaussian

What this chapter assumes, restated. M is a mechanism: a randomized program that takes a dataset and returns an output. $D \simeq D'$ are neighbouring datasets, which differ in one person's record. A query f has ℓ_1 sensitivity $\Delta_1 f$ and ℓ_2 sensitivity $\Delta_2 f$: the largest change in f, measured as a sum of absolute coordinate changes or as a Euclidean length, when one record changes. M is (ε, δ)**-DP** if, for every pair of neighbours and every set S of outputs, $\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$: no event becomes more than e^{ε} times more likely when one record changes, except for an additive slack δ. With $\delta = 0$ this is **pure** ε-DP. The **privacy loss** of an output y is the log of the ratio of its probabilities under the two neighbours,

$$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}, \tag{24.1}$$

which measures how much evidence y gives for D over D'. For a discrete output, $p_{M(D)}$ is a probability mass function (PMF), the probability of each single output. Pure ε-DP is the statement $|L_{D,D'}(y)| \leq \varepsilon$ for every y, because the probability of a set is the sum of its outputs' probabilities, and if each term grows by at most e^{ε} so does the sum. The full treatment is in The definition of differential privacy, the Laplace and Gaussian mechanisms are in The Laplace and Gaussian mechanisms, and the Laplace and geometric distributions are in Probability distributions.

24.2. Toolkit: exact randomness from uniform integers

The exact samplers below need only one primitive: a uniform integer in $\{0, 1, \ldots, d-1\}$ for any $d \geq 1$. Python's `random.Random.randrange(d)` provides it exactly: it draws $\lceil \log_2 d \rceil$ random bits and redraws when the result is d or more, so every integer below d comes out with probability exactly $1/d$. A deployment draws the bits from the operating system with `secrets.SystemRandom()`, which has the same interface. Three facts turn this primitive into everything else: how to make a coin with a fraction as its bias, how to reshape one distribution into another by throwing some draws away, and how counting coin flips gives a geometric variable.

24.2.1. Fact 1: an exact coin with rational bias

To make a coin that comes up 1 with probability n/d, draw a uniform integer below d and call it a success when it is one of the first n values. For integers $0 \leq n \leq d$, let U be uniform on

$\{0, \dots, d-1\}$. Then

$$\Pr[U < n] = \frac{n}{d}, \tag{24.2}$$

because there are n favourable values out of d equally likely ones. So $\mathbf{1}[U < n]$, the indicator that is 1 when $U < n$ and 0 otherwise, is an exact Bernoulli(n/d) draw, with no rounding anywhere. A coin with an *irrational* bias such as e^{-1} can't be made this way, because no fraction n/d equals it, which is why Algorithm 1 below exists.

24.2.2. Fact 2: rejection sampling

Suppose you can sample from a PMF q, the **proposal**, but you want samples from a different PMF p. If $p(x)$ is proportional to $q(x)a(x)$ for some acceptance probability $0 \le a(x) \le 1$, you can draw from q and keep each draw x only with probability $a(x)$: values that p favours less than q get thrown away more often. Precisely, let $p(x) = q(x)a(x)/A$, where $A = \sum_x q(x)a(x)$ is the normalizer. Repeat: draw $Y \sim q$, then accept it with probability $a(Y)$; output the first accepted Y. Three steps show that this is exact and what it costs.

1. **Look at one round.** A round outputs x when it proposes x (probability $q(x)$) and accepts it (probability $a(x)$), so with probability $q(x)a(x)$. Adding over x, a round outputs something with probability A and nothing with probability $1 - A$.
2. **Add up the rounds.** The rounds are independent and identical, so the output is x when the first j rounds output nothing and round $j + 1$ outputs x, for some $j \ge 0$. That has probability $\sum_{j \ge 0}(1 - A)^j q(x)a(x)$, a geometric series with ratio $1 - A < 1$, which sums to $q(x)a(x)/A = p(x)$.
3. **Count the cost.** Each round succeeds with probability A independently, so the number of rounds is geometric with success probability A, and its mean is $1/A$.

The output is exact as long as each acceptance is an exact coin. What matters is that proposal times acceptance is proportional to the target. You never need to compute the normalizer A; it only sets how many rounds you wait.

24.2.3. Fact 3: counting successes gives a geometric variable

Flip independent coins with success probability r and let V be the number of successes before the first failure. The event $V = v$ means exactly v successes followed by one failure, and independent flips multiply, so

$$\Pr[V = v] = r^v(1 - r), \qquad v = 0, 1, 2, \dots \tag{24.3}$$

This is the geometric distribution of Probability distributions, counted from 0. Its tail is $\Pr[V \ge v] = r^v$, since $V \ge v$ just says the first v flips were successes.

24.3. Floating-point Laplace breaks differential privacy

24.3.1. Context

The textbook way to sample Lap(b) is inverse cumulative distribution function (CDF) sampling: turn a uniform number into a draw with the right tail by solving for where the tail probability

equals it. Draw U uniform on $(0, 1)$ and a fair sign $\xi \in \{-1, +1\}$, and output $\xi \cdot b \ln(1/U)$. The magnitude $b \ln(1/U)$ is exponential with mean b, written $\mathrm{Exp}(b)$, because $b \ln(1/U) > t$ exactly when $U < e^{-t/b}$, so $\Pr[b \ln(1/U) > t] = \Pr[U < e^{-t/b}] = e^{-t/b}$. A fair sign turns it into $\mathrm{Lap}(b)$, whose density $\frac{1}{2b} e^{-|z|/b}$ is the exponential mirrored around 0. The Laplace mechanism then outputs $y = f(D) + \xi \cdot b \ln(1/U)$. Run in floating point, U, the logarithm and the sum are each rounded. The question is whether the rounded mechanism is still DP.

24.3.2. Statement

The test is simple: if some outputs can only come from D, seeing one of them reveals D with certainty, and no multiplicative factor e^{ε} can account for that, so all of their probability has to be paid for by δ. Write $\mathcal{Y}_D$ for the set of outputs $M(D)$ can produce and let $S = \mathcal{Y}_D \setminus \mathcal{Y}_{D'}$ be the outputs reachable from D but not from D'. If M is (ε, δ)-DP, then for every ε

$$\delta \;\geq\; \Pr[M(D) \in S]. \tag{24.4}$$

In particular, if $\Pr[M(D) \in S] > 0$, then M is not ε-DP for any finite ε. The floating-point Laplace sampler has such outputs. In a toy model with a 12-bit significand that can be enumerated exactly, $\Pr[M(D) \in S] = 0.4791$ for $f(D) = 0$, $f(D') = 1$ and $b = 1$.

24.3.3. Derivation

1. **Apply the definition to the event S.** (ε, δ)-DP says $\Pr[M(D) \in S] \leq e^{\varepsilon} \Pr[M(D') \in S] + \delta$ for every event. Here $\Pr[M(D') \in S] = 0$ by the choice of S, so the multiplicative term is $e^{\varepsilon} \cdot 0 = 0$ however large e^{ε} is, and what remains is $\Pr[M(D) \in S] \leq \delta$. That is Equation 24.4.
2. **Read it as an attack.** Use the convention of The definition of differential privacy: a test decides between "the data were D" and "the data were D'", its false positive rate α is the chance of saying "D'" when the data were D, and its false negative rate β is the chance of saying "D" when the data were D'. Consider the test "say D when $y \in S$, and D' otherwise". Under D' the output never lands in S, so the test never says "D" and $\beta = 0$. Under D it says "D'" whenever y misses S, so $\alpha = 1 - \Pr[M(D) \in S]$. The hypothesis-testing form of DP in that chapter says that every test has $\alpha + e^{\varepsilon}\beta \geq 1 - \delta$; this is the definition applied to the event on which the test says "D". With $\beta = 0$ the factor e^{ε} multiplies nothing, and the inequality reads $1 - \Pr[M(D) \in S] \geq 1 - \delta$, which is again $\Pr[M(D) \in S] \leq \delta$. And whenever this test says "D", it is right.
3. **Floating-point outputs form a finite set that depends on $f(D)$.** A float with a p-bit significand (IEEE 754 doubles have $p = 53$) is $m \cdot 2^e$ with an integer $m < 2^p$ and an integer exponent e. Between 2^e and 2^{e+1} the floats are spaced 2^{e-p+1} apart, so they are dense near 0 and sparse far from it. The sampler computes $\mathrm{round}(f(D) + \xi \cdot \mathrm{round}(b \ln(1/U)))$. Its outputs are the rounded sums of $f(D)$ with a fixed, finite set of noise values. Shifting by $f(D') \neq f(D)$ before rounding lands on a different set of floats.
4. **Why the two sets differ.** Near 0 the noise values are fine-grained: the floats there are spaced far more closely than near 1. $M(D)$ with $f(D) = 0$ outputs those noise values as they are. $M(D')$ with $f(D') = 1$ must produce an output near 0 as $1 -$ (noise near 1), which is a multiple of the coarser spacing of the noise near 1. So most of the fine-grained outputs of $M(D)$ near 0 are impossible under $M(D')$. Away from the inputs the two grids

interleave and coincide only in part. Figure 24.1 (left) shows a narrow window: 3 of the 9 outputs of $M(D)$ there can't come from $M(D')$, and 4 of the 10 outputs of $M(D')$ can't come from $M(D)$.

Concretely, in a toy model. Doubles have too many values to enumerate, so the script uses p-bit floats with p from 6 to 14. U is an ideal uniform p-bit float: a real uniform rounded down to a p-bit float, so each float in $[2^{-40}, 1)$ has probability equal to its gap to the next one. The logarithm is computed and rounded to p bits, then the sum is rounded to p bits. Every output and its probability is computed exactly. Table 24.2 lists the results.

Table 24.2.: The toy floating-point Laplace sampler with $b = 1$, $f(D) = 0$ and $f(D') = 1$. S is the set of outputs of $M(D)$ that $M(D')$ can't produce, and S' the reverse.

Significand bits p	Outputs of $M(D)$	Outputs of $M(D')$	$\Pr[M(D) \in S]$	$\Pr[M(D') \in S']$
8	1,604	1,409	0.4795	0.2070
12	25,646	22,477	0.4791	0.2033
14	102,586	89,859	0.4826	0.2053

At $p = 12$, outputs with $|y| < 0.5$ carry probability 0.3933 under $M(D)$, and 0.3173 of it lands where $M(D')$ can't. Most of the leak is near $f(D)$, as step 4 predicts.

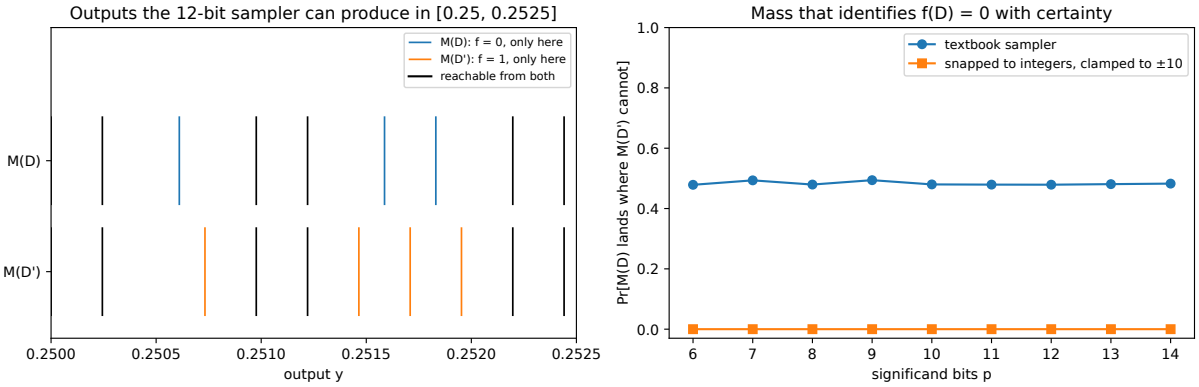


Figure 24.1.: The toy 12-bit floating-point Laplace sampler with $b = 1$. Left: every output that $f(D) = 0$ (top) and $f(D') = 1$ (bottom) can produce in a narrow window, coloured by whether the other input can produce it too. Right: the probability that $M(D)$ produces an output $M(D')$ cannot, as a function of the significand length p, for the textbook sampler and for the snapped one of Section 24.4.

24.3.4. Interpretation

More precision does not help. The right panel of Figure 24.1 is flat: from 6 to 14 bits, about 48% of the mass of $M(D)$ is on outputs $M(D')$ can't produce. Adding bits refines both grids together, so the mismatch stays. Mironov (2012) found the same effect at $p = 53$ in the samplers of several DP libraries of the time, and showed how to recover $f(D)$ from single outputs.

The leak sits in the low-order bits. It is not in where the output lands on the real line but in which float it is. Printing the output to a few decimals would hide it, and that is the intuition behind snapping in the next section.

There is a second, smaller leak: the tails are cut off. A 53-bit uniform $U = k \cdot 2^{-53}$, with k a positive integer, is never below 2^{-53}, so the noise magnitude $b \ln(1/U)$ never exceeds $b \ln(2^{53}) = b \cdot 53 \ln 2 \approx 36.74\, b$. Outputs between $f(D) + 36.74b$ and $f(D') + 36.74b$ are then reachable from only one input. Their probability is about 2^{-53}, but by Equation 24.4 it is still a floor on δ, and it rules out pure DP.

Finally, the model is a toy, but the effect is not. Real libraries differ from the toy in how they draw U and round the logarithm. Those details change the numbers, not the conclusion. Jin et al. (2022) found floating-point and timing leaks in DP libraries released after Mironov's paper.

24.3.5. Example: a count released with $\varepsilon = 1$

A hospital releases the number of patients with a rare diagnosis, a count with $\Delta_1 f = 1$, using the textbook sampler with $b = 1$, so that the exact mechanism would have $\varepsilon = \Delta_1 f / b = 1$. An analyst who knows that the count is either 0 or 1 (say, whether one particular patient is in the data) checks whether the released float can be produced from 1. In the 12-bit toy, the check identifies the count 0 with certainty 47.91% of the time: that is $\Pr[M(D) \in S] = 0.4791$ from Table 24.2. The proof of the Laplace mechanism promised that one output could move the analyst's odds by at most a factor e; the code delivers certainty almost half the time.

24.3.6. Check yourself

- Why doesn't the support argument need ε to be small? *(Because* $\Pr[M(D') \in S] = 0$*, the term* $e^{\varepsilon} \Pr[M(D') \in S]$ *vanishes for every* ε*, however large.)*
- Would the problem go away if $f(D)$ and $f(D')$ were both integers and b were a power of 2? *(No. The noise values* $b \ln(1/U)$ *are still rounded differently once added to different integers, because the spacing of floats depends on the magnitude of the sum. The toy uses* $b = 1$ *and integer* f*.)*

24.4. The snapping mechanism

24.4.1. Context

If the output must be a float, the low-order bits have to go. The snapping mechanism of Mironov (2012) removes them by rounding the noisy answer to a coarse grid, and clamps both the input and the output to a bounded range so that floating-point rounding errors stay small relative to the grid. This section gives the mechanism and its guarantee in outline.

24.4.2. Statement

In words: clamp the true answer into $[-B, B]$, add Laplace noise computed in floating point, round the result to the nearest multiple of a power of 2 at least as large as the noise scale, and clamp again. Let f have sensitivity 1, let λ be the noise scale, let Λ be the smallest power of 2 with $\Lambda \geq \lambda$, and let B be a clamping bound. The **snapping mechanism** outputs

$$M(D) = \mathrm{clamp}_B\Big(\lfloor \mathrm{clamp}_B(f(D)) \oplus \xi \otimes \lambda \otimes \mathrm{LN}(U^*) \rceil_\Lambda\Big), \tag{24.5}$$

where $\oplus$, $\otimes$ and LN are floating-point addition, multiplication and logarithm, ξ is a fair sign, U^* is an ideal uniform double, $\lfloor\cdot\rceil_\Lambda$ rounds to the nearest multiple of Λ, and clamp_B maps to $[-B, B]$. ($\mathrm{LN}(U^*)$ is negative, and the fair sign ξ makes the sign of the noise irrelevant.)

Its guarantee is the exact Laplace mechanism's $\varepsilon = 1/\lambda$ plus two small terms that pay for floating-point error. The error of one floating-point operation is at most a fraction $\eta = 2^{-53}$ of the number it produces, and clamping keeps every number below B in size, so the terms grow with $B\eta$. Mironov (2012, Theorem 1) proves that for $\lambda < B < 2^{46}\lambda$ the mechanism is ε-DP with

$$\varepsilon = \frac{1 + 12B\eta}{\lambda} + \frac{2^{-49}B}{\lambda}, \qquad \eta = 2^{-53}. \tag{24.6}$$

24.4.3. Derivation (outline)

1. **Clamping the input** bounds $|f(D)| \leq B$, so every sum is computed at magnitudes where the float spacing is at most about $B\eta$.
2. **Rounding to a multiple of** $\Lambda \geq \lambda$ discards every bit finer than Λ. The output is one of the multiples of Λ in $[-B, B]$, the same finite set for every input, so the support argument of Section 24.3 has nothing to bite on: no output is reachable from one input only.
3. **Each grid point collects an interval of noise values** of width Λ: all noise values that round to that point. In exact arithmetic, its probability under D and under D' is the probability that a Laplace variable lands in one of two intervals shifted by $|f(D) - f(D')| \leq 1$. The Laplace density changes by at most a factor $e^{1/\lambda}$ under a shift of 1, because $e^{-|z|/\lambda}$ and $e^{-|z-1|/\lambda}$ differ in the exponent by at most $1/\lambda$, so the ratio of the two interval probabilities is at most $e^{1/\lambda}$. In other words, rounding exact Laplace noise is post-processing of the exact Laplace mechanism, and post-processing never increases the privacy loss.
4. **Floating-point error moves each interval's endpoints** by a relative amount of order η, measured against magnitudes up to B. Bounding the effect of those moves on the interval probabilities gives the extra terms proportional to $B\eta/\lambda$. Mironov (2012, §5) does this bookkeeping.

24.4.4. Interpretation

The price is accuracy: rounding to Λ adds up to $\Lambda/2 \leq \lambda$ of error (since Λ is the smallest power of 2 at or above λ, it is below 2λ), and clamping biases answers near $\pm B$. The extra terms in ε are tiny for doubles. With $\lambda = 1$ and $B = 10$, the first term exceeds $1/\lambda$ by $12B\eta = 120 \cdot 2^{-53}$ and the second is $10 \cdot 2^{-49}$, so the excess is $120 \cdot 2^{-53} + 10 \cdot 2^{-49} \approx 3.1 \times 10^{-14}$. The guarantee also assumes a correctly behaved logarithm and an ideal uniform U^*, which a library has to supply.

24.4.5. Example: the toy, snapped

Snap the toy sampler to integers ($\Lambda = 1$ for $b = 1$) and clamp to $[-10, 10]$. Both inputs now reach the same 21 outputs, the integers from -10 to 10, so $\Pr[M(D) \in S] = 0$ at every precision (Figure 24.1, right). The largest privacy loss over those outputs, the largest log-ratio of an output's probabilities under the two inputs, is $\max_y |L_{D,D'}(y)| = 1.0010$ at $p = 12$ and 1.0002 at $p = 14$, against $\varepsilon = 1$ for the exact mechanism. The excess shrinks with precision, as the η terms predict.

24.4.6. Check yourself

- Why must the grid be at least as coarse as λ? *(Finer grids keep bits whose pattern still depends on $f(D)$. The proof needs each grid cell to collect a whole interval of noise values, wide enough that floating-point errors at its edges move only a tiny fraction of its probability.)*
- Is snapping post-processing of the floating-point Laplace mechanism? *(It is post-processing, but post-processing of a mechanism that isn't DP proves nothing. The proof compares the snapped output with snapped exact Laplace noise and bounds the difference.)*

24.5. Exact Bernoulli($e^{-\gamma}$)

24.5.1. Context

Both exact samplers below need a coin that comes up 1 with probability $e^{-\gamma}$ for a rational $\gamma \geq 0$. The obvious code, `random() < exp(-gamma)`, is inexact twice over: `exp` rounds, and `random()` has only 53 bits. Algorithm 1 of Canonne, Kamath and Steinke (2020), based on a method of von Neumann (1951), makes the coin from rational coins alone.

24.5.2. Statement

The idea is to build $e^{-\gamma}$ from its power series $e^{-\gamma} = 1 - \gamma + \frac{\gamma^2}{2!} - \frac{\gamma^3}{3!} + \cdots$, whose terms are probabilities you can produce with rational coins. Flip coins whose success probabilities shrink as $\gamma, \gamma/2, \gamma/3, \ldots$ until one fails. The chance of surviving k flips is then $\gamma \cdot \frac{\gamma}{2} \cdots \frac{\gamma}{k} = \gamma^k/k!$, the k-th term of the series. The series alternates in sign, and the parity of the stopping time produces exactly that alternation.

For $0 \leq \gamma \leq 1$: draw $A_k \sim \text{Bernoulli}(\gamma/k)$ for $k = 1, 2, \ldots$, and stop at the first $A_K = 0$, so K is the index of the first failure. Output 1 if K is odd and 0 if K is even. Then the survival probability, the output probability and the mean number of flips are

$$\Pr[K > k] = \frac{\gamma^k}{k!}, \qquad \Pr[K \text{ odd}] = e^{-\gamma}, \qquad \mathbb{E}[K] = e^{\gamma}. \tag{24.7}$$

For $\gamma > 1$ the first coin would need a probability above 1, so split γ into its integer part $\lfloor \gamma \rfloor$ and a remainder below 1: draw $\lfloor \gamma \rfloor$ coins with probability e^{-1}, stopping at the first 0, and then one coin with probability $e^{-(\gamma - \lfloor \gamma \rfloor)}$. Output 1 only if all of them are 1.

24.5.3. Derivation

1. **The survival probabilities.** $K > k$ means the first k coins all came up 1. They are independent with probabilities $\gamma/1, \gamma/2, \ldots, \gamma/k$, so $\Pr[K > k]$ is their product, $\gamma^k/k!$. For $\gamma = n/d$, each $\gamma/j = n/(dj)$ is a ratio of integers, at most 1 since $\gamma \le 1$, so each coin is exact by Fact 1.
2. **The point probabilities.** $K = k$ means the coins survived $k-1$ flips but not k, so $\Pr[K = k] = \Pr[K > k-1] - \Pr[K > k] = \frac{\gamma^{k-1}}{(k-1)!} - \frac{\gamma^k}{k!}$.
3. **Add over odd k.** With $k = 1, 3, 5, \ldots$ the terms are $(1-\gamma) + (\frac{\gamma^2}{2!} - \frac{\gamma^3}{3!}) + (\frac{\gamma^4}{4!} - \frac{\gamma^5}{5!}) + \cdots$. Every power γ^j appears exactly once: with a plus sign when $j = k-1$ is even and a minus sign when $j = k$ is odd. So the sum is the series $\sum_{j \ge 0} (-\gamma)^j / j! = e^{-\gamma}$.
4. **The mean.** A variable K on $\{1, 2, \ldots\}$ can be written as a count, $K = \sum_{k \ge 0} \mathbf{1}[K > k]$, since exactly K of the indicators $\mathbf{1}[K > 0], \mathbf{1}[K > 1], \ldots$ are 1. Taking expectations, $\mathbb{E}[K] = \sum_{k \ge 0} \Pr[K > k] = \sum_k \gamma^k/k! = e^\gamma$. Each round uses one uniform integer, so a coin costs $e^\gamma \le e$ uniform draws on average.
5. **Large γ.** $e^{-\gamma} = (e^{-1})^{\lfloor \gamma \rfloor} e^{-(\gamma - \lfloor \gamma \rfloor)}$, and a product of probabilities is the probability that independent coins all come up 1. Each coin has an exponent of at most 1, so steps 1–3 cover it. Stopping at the first 0 doesn't change the output, which is 0 as soon as one coin is 0.

Concretely, for $\gamma = 0.7$ (Table 24.3). The odd rows add up to $0.3000 + 0.1878 + 0.0086 + \cdots = 0.4966 = e^{-0.7}$.

Table 24.3.: Algorithm 1 for $\gamma = 0.7$, outcome by outcome. The output is the parity of the index of the first 0.

K	Coins seen	$\Pr[K = k]$	Output
1	$A_1 = 0$	$1 - 0.7 = 0.3000$	1
2	$A_1 = 1$, $A_2 = 0$	$0.7 - 0.7^2/2 = 0.4550$	0
3	$A_1 = A_2 = 1$, $A_3 = 0$	$0.245 - 0.0572 = 0.1878$	1
4	three 1s, then 0	0.0472	0
5	four 1s, then 0	0.0086	1

In row 3, $0.245 = 0.7^2/2!$ is the chance of surviving two flips and $0.0572 = 0.7^3/3!$ the chance of surviving three; the difference is the chance of stopping at exactly the third.

24.5.4. Interpretation

The coin is exact: no step rounds, and the only randomness is uniform integers. The cost is small and bounded: at most e draws for $\gamma \le 1$, and below $\frac{e}{1-e^{-1}} + 1 \approx 5.3$ for any γ (Exercise 1 in the exercise book). The cost curve in Figure 24.2 jumps at each integer. At $\gamma = 1$ the algorithm runs one full e^{-1} coin and then a coin for the fractional part 0, which costs one extra draw when the first coin comes up 1 (probability e^{-1}), so the cost goes from $e = 2.7183$ just below 1 to $e + e^{-1} = 3.0862$ at 1. Splitting γ differently would smooth this out; it doesn't matter for exactness.

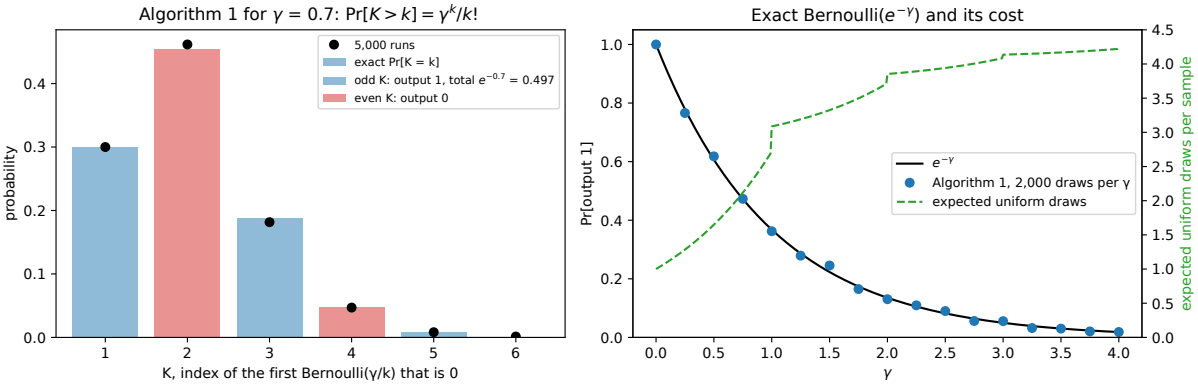


Figure 24.2.: Left: the distribution of K for $\gamma = 0.7$, exact and from 5,000 runs, with odd K (output 1) and even K (output 0) coloured. Right: the frequency of 1s from Algorithm 1 against $e^{-\gamma}$, with the expected number of uniform draws per coin on the right axis.

24.5.5. Example: one coin per proposal

Algorithm 3 below accepts a proposal y with probability $e^{-\gamma}$ for a rational $\gamma = (|y| - \sigma^2/t)^2/(2\sigma^2)$, where σ^2 is the target variance and t an integer scale. With $\sigma^2 = 4$, $t = 3$ and $y = 4$, that is $\gamma = (4 - 4/3)^2/8 = (8/3)^2/8 = 8/9$, so the coin is 1 with probability $e^{-8/9} = 0.4111$ and costs $e^{8/9} \approx 2.43$ draws on average. Everything stays a fraction: γ is computed with `fractions.Fraction`, and each γ/k is a ratio of integers.

24.5.6. Check yourself

- Why does the algorithm use γ/k rather than a fixed probability γ at every step? *(The decreasing probabilities make* $\Pr[K > k] = \gamma^k/k!$*, the terms of the exponential series. A fixed* γ *would give* γ^k*, a geometric series, and the alternating sum would be* $1/(1+\gamma)$*.)*
- What does the algorithm output for $\gamma = 0$? *(The first coin is* Bernoulli(0)*, so* $K = 1$*, which is odd, and the output is* $1 = e^0$*.)*

24.6. The discrete Laplace mechanism

24.6.1. Context

Most queries in DP are counts or histograms: integer-valued, with sensitivity 1. For them the natural noise is integer-valued too. Then the output is an integer, there are no low-order bits to leak, and the noise can be sampled exactly. The discrete Laplace distribution is the integer analogue of the Laplace, and adding it is the **geometric mechanism** of Ghosh, Roughgarden and Sundararajan (2009).

24.6.2. Statement

Take the Laplace density $e^{-|z|/b}$, keep only its values at the integers, and divide by their sum so they add up to 1. Each step away from 0 multiplies the probability by the same factor $r = e^{-1/b}$, just as the continuous density drops by $e^{-1/b}$ per unit. The **discrete Laplace** distribution with scale $b > 0$, written $\mathrm{Lap}_{\mathbb{Z}}(b)$, has PMF

$$\Pr[Z = z] = \frac{1-r}{1+r}\, r^{|z|}, \qquad z \in \mathbb{Z}, \qquad r = e^{-1/b}. \tag{24.8}$$

Its variance is $2r/(1-r)^2$, slightly below the $2b^2$ of $\mathrm{Lap}(b)$. Because shifting the input by one unit changes each output's probability by at most the factor $r^{-1} = e^{1/b}$, as for the continuous Laplace, the privacy guarantee is the same. For an integer-valued f with sensitivity $\Delta_1 f$, the mechanism $M(D) = f(D) + Z$ is ε-DP with

$$\varepsilon = \frac{\Delta_1 f}{b}. \tag{24.9}$$

For rational $b = t/s$, Algorithm 2 of Canonne, Kamath and Steinke (2020) samples $\mathrm{Lap}_{\mathbb{Z}}(b)$ exactly.

24.6.3. Derivation

The PMF and variance. Both come from sums of geometric series.

1. **Normalize.** The weights $r^{|z|}$ are 1 at $z = 0$ and appear twice, at $\pm z$, for each $z \geq 1$. So $\sum_{z\in\mathbb{Z}} r^{|z|} = 1 + 2\sum_{z\geq 1} r^z = 1 + \frac{2r}{1-r} = \frac{1+r}{1-r}$, and dividing by it gives the constant in Equation 24.8.
2. **Variance.** The PMF gives z and $-z$ the same probability, so their contributions $z\Pr[Z = z]$ and $-z\Pr[Z = -z]$ cancel and the mean is 0. The variance is then $\mathbb{E}Z^2$. Differentiating $\sum_{z\geq 0} r^z = \frac{1}{1-r}$ and multiplying by r gives $\sum_{z\geq 1} zr^z = \frac{r}{(1-r)^2}$; doing it once more gives $\sum_{z\geq 1} z^2 r^z = \frac{r(1+r)}{(1-r)^3}$. Counting $\pm z$ twice, $\mathbb{E}Z^2 = \frac{1-r}{1+r} \cdot 2 \cdot \frac{r(1+r)}{(1-r)^3} = \frac{2r}{(1-r)^2}$.

Privacy. The plan is to compute the privacy loss of each output and bound it by ε, which is pure ε-DP.

1. **The privacy loss.** For neighbours with $f(D) = a$ and $f(D') = a'$, an output y requires noise $y - a$ under D and $y - a'$ under D'. The normalizing constant is the same for both, so it cancels in the ratio:

$$L_{D,D'}(y) = \ln \frac{r^{|y-a|}}{r^{|y-a'|}} = \frac{|y-a'| - |y-a|}{b}, \tag{24.10}$$

   using $\ln r = -1/b$.
2. **Bound it.** The triangle inequality says that two numbers' distances to y differ by at most their distance to each other: $|y - a'| - |y - a| \leq |a - a'| \leq \Delta_1 f$. So $|L_{D,D'}(y)| \leq \Delta_1 f/b$ for every integer y, which is pure DP with Equation 24.9. The bound is reached for every $y \geq \max(a, a')$, so it is tight.

3. **Vectors.** For $f \in \mathbb{Z}^d$ with independent noise per coordinate, the probability of an output vector is a product over coordinates, so its log, the privacy loss, is a sum of per-coordinate losses. Each is at most $|a_i - a_i'|/b$, and $\sum_i |a_i - a_i'| \le \Delta_1 f$ gives the same ε with the ℓ_1 sensitivity.

Step 1 is where integers matter. The constant cancels because $\sum_z r^{|z-a|}$ is the same for every *integer* a: shifting by an integer only relabels the terms. Shifting by a non-integer would move the noise off the lattice, and the mechanism would have different supports for different inputs: the floating-point failure again.

The exact sampler (Algorithm 2). The plan is to build a one-sided geometric variable with ratio $e^{-1/b}$ out of exact pieces, then attach a random sign. A geometric with ratio $e^{-1/t}$ is assembled from a remainder U below t and a quotient V, the way an integer x is $u + tv$; dividing by s then turns ratio $e^{-1/t}$ into $e^{-s/t} = e^{-1/b}$. Input integers $s, t \ge 1$, so $b = t/s$. Repeat until a value is returned:

1. Draw U uniform on $\{0, \dots, t-1\}$ and accept it with probability $e^{-U/t}$ (Algorithm 1); on rejection, start over.
2. Count the successes V of independent e^{-1} coins before the first failure (Algorithm 1 with $\gamma = 1$).
3. Set $X = U + tV$ and $Y = \lfloor X/s \rfloor$.
4. Draw a fair sign bit B. If $B = 1$ and $Y = 0$, start over. Otherwise return $(1 - 2B)Y$.

Five steps show that the output is exactly $\mathrm{Lap}_{\mathbb{Z}}(b)$, following the algorithm's own order.

1. U **given acceptance** has $\Pr[U = u] \propto e^{-u/t}$ on $\{0, \dots, t-1\}$, by Fact 2 with a uniform proposal and acceptance $e^{-u/t}$.
2. V **is geometric** with $\Pr[V = v] = (1 - e^{-1})e^{-v}$, by Fact 3 with $r = e^{-1}$.
3. $X = U + tV$ **is geometric with ratio** $e^{-1/t}$**.** Each $x \ge 0$ is $u + tv$ for exactly one pair $(u = x \bmod t,\ v = \lfloor x/t \rfloor)$, and the independent weights multiply to $e^{-u/t}e^{-v} = e^{-(u+tv)/t} = e^{-x/t}$. So $\Pr[X = x] \propto e^{-x/t}$.
4. $Y = \lfloor X/s \rfloor$ **is geometric with ratio** $e^{-s/t} = e^{-1/b}$**.** $Y = y$ collects $x = sy, \dots, sy + s - 1$, with total weight $e^{-sy/t}(1 + e^{-1/t} + \cdots + e^{-(s-1)/t})$. The bracket does not depend on y, so the weight is $\propto e^{-sy/t} = r^y$.
5. **The sign.** $\pm Y$ with a fair sign gives $\pm y$ with probability $\frac{1}{2}\Pr[Y = y]$ each, but 0 would get both halves, since $+0$ and -0 are the same output. Rejecting $(B = 1, Y = 0)$ leaves $\frac{1}{2}\Pr[Y = y]$ for every $z = \pm y$, including 0. After renormalizing (Fact 2), $\Pr[Z = z] \propto r^{|z|}$.

Concretely, for $t = 3$ **and** $s = 2$ $(b = 1.5)$. Table 24.4 follows steps 3 and 4.

Table 24.4.: Algorithm 2 with $t = 3$, $s = 2$. Each x is one pair (u, v); pairs of consecutive x merge into one y.

x	$u = x \bmod 3$	$v = \lfloor x/3 \rfloor$	$\Pr[X = x] = (1 - e^{-1/3})e^{-x/3}$	$y = \lfloor x/2 \rfloor$
0	0	0	0.2835	0
1	1	0	0.2031	0
2	2	0	0.1455	1
3	0	1	0.1043	1

x	$u = x \bmod 3$	$v = \lfloor x/3 \rfloor$	$\Pr[X = x] = (1 - e^{-1/3})e^{-x/3}$	$y = \lfloor x/2 \rfloor$
4	1	1	0.0747	2
5	2	1	0.0535	2

$\Pr[Y = 0] = 0.2835 + 0.2031 = 0.4866$ and $\Pr[Y = 1] = 0.1455 + 0.1043 = 0.2498$. Their ratio is $0.5134 = e^{-2/3}$, a geometric with ratio $e^{-s/t}$, as step 4 says.

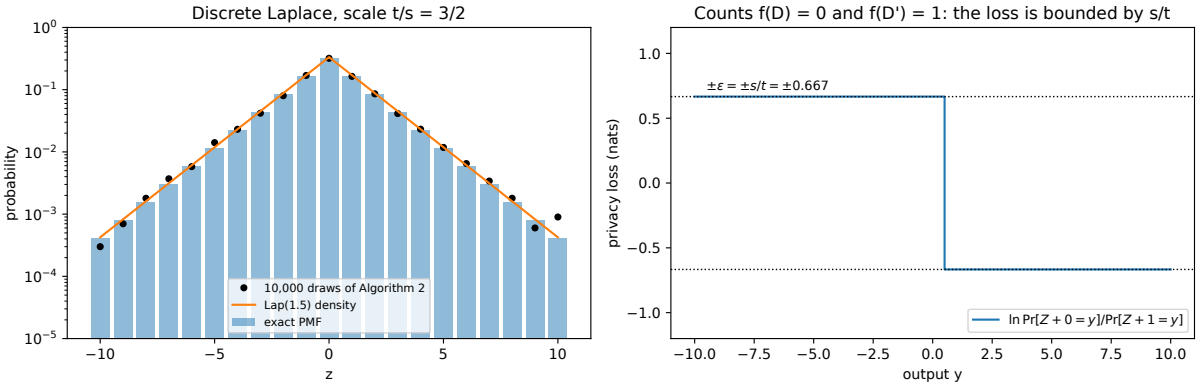


Figure 24.3.: Left: the PMF of $\mathrm{Lap}_{\mathbb{Z}}(1.5)$ on a log scale, exact and from 10,000 draws of Algorithm 2, with the Lap(1.5) density for comparison. Right: the privacy loss between counts 0 and 1, a step function bounded by $\varepsilon = s/t = 2/3$.

24.6.4. Interpretation

The discrete Laplace gives the same guarantee with less noise. It has the same ε as the continuous one at the same scale and a slightly smaller variance: 1.8413 against 2 at $b = 1$. Exercise 3 in the exercise book shows the gap tends to $1/6$ as b grows. Rounding $f(D) + \mathrm{Lap}(b)$ to the nearest integer would also be ε-DP in exact arithmetic, as post-processing, but it is not the same distribution, and it needs a floating-point sampler.

It is also the best possible mechanism for one count. Ghosh, Roughgarden and Sundararajan (2009) show that for a single count, the geometric mechanism is universally utility-maximizing: every user, whatever their loss function and prior, can post-process its output to do as well as with any other ε-DP mechanism.

The cost follows from Fact 2. A pass of Algorithm 2 survives step 1 with probability $A_U = \frac{1}{t}\sum_{u<t} e^{-u/t}$, the average acceptance over the t uniform values. It then survives step 4 unless $B = 1$ and $Y = 0$; since Y is geometric with ratio $e^{-s/t}$, $\Pr[Y = 0] = 1 - e^{-s/t}$, so that happens with probability $\frac{1}{2}(1 - e^{-s/t})$. The mean number of passes is one over the product of the two survival probabilities, $1/(A_U(1 - \frac{1}{2}(1 - e^{-s/t})))$. That is 1.462 passes at $b = 1$, and about 10 to 12 uniform draws per sample over the scales in Table 24.7.

24.6.5. Example: a count with $\varepsilon = 1$

Release a count with sensitivity 1 at $\varepsilon = 1$, so $b = \Delta_1 f/\varepsilon = 1$ and $r = e^{-1} = 0.3679$. The exact count is released when the noise is 0, which happens with probability $\Pr[Z = 0] = \frac{1-r}{1+r} = \frac{0.6321}{1.3679} = 0.4621$. The noise has variance $\frac{2r}{(1-r)^2} = 1.8413$, so its standard deviation is $\sqrt{1.8413} = 1.357$. The output is an integer, sampled exactly, so the analysis in this section is the analysis of the code.

24.6.6. Check yourself

- Why is the bound in step 2 of the privacy proof reached for $y \geq \max(a, a')$? *(There both $|y - a|$ and $|y - a'|$ are y minus the input, so their difference is exactly $|a - a'|$.)*
- In Algorithm 2, why can't you skip the rejection of $(B = 1, Y = 0)$? *(Then $\Pr[Z = 0] = \Pr[Y = 0]$ while $\Pr[Z = \pm y] = \frac{1}{2}\Pr[Y = y]$: zero would get double weight, and the PMF would not be Equation 24.8.)*

24.7. The discrete Gaussian mechanism

24.7.1. Context

The Gaussian mechanism is the workhorse of private machine learning, because its noise grows with the ℓ_2 sensitivity rather than the ℓ_1 sensitivity, and because sums of Gaussians stay Gaussian. Canonne, Kamath and Steinke (2020) define its integer analogue and show that it keeps the privacy of the continuous Gaussian. The US Census Bureau used it for the 2020 Census redistricting data (Abowd et al., 2022), and it is the noise of choice when clients add noise before secure aggregation, which works with integers modulo a fixed number (Kairouz, Liu & Steinke, 2021).

24.7.2. Statement

As with the discrete Laplace, take the Gaussian bell curve, keep its values at the integers and normalize. The **discrete Gaussian** with parameter $\sigma > 0$ and integer location μ, written $\mathcal{N}_{\mathbb{Z}}(\mu, \sigma^2)$, has PMF

$$\Pr[X = x] = \frac{e^{-(x-\mu)^2/(2\sigma^2)}}{\sum_{y \in \mathbb{Z}} e^{-y^2/(2\sigma^2)}}, \qquad x \in \mathbb{Z}. \tag{24.11}$$

The parameter σ^2 is the variance of the continuous bell curve, not quite the variance of X, as the last claim below shows. For an integer-valued f, consider $M(D) = f(D) + X$ with $X \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$, and neighbours with $f(D) - f(D') = \Delta$. Four claims describe how it compares with the continuous Gaussian mechanism.

First, **the privacy loss is the continuous Gaussian's**, evaluated at integers. The reason is that the normalizer is the same for every integer location, so the log-ratio of two PMFs is the log-ratio of the two bell curves:

$$L_{D,D'}(y) = \frac{(y - f(D'))^2 - (y - f(D))^2}{2\sigma^2}, \qquad y \in \mathbb{Z}. \tag{24.12}$$

Second, the smallest δ at each ε has the same formula as for the continuous Gaussian. That smallest δ is the probability mass by which $M(D)$ exceeds e^{ε} times $M(D')$, summed over outputs, and since the privacy loss is linear in y the outputs that contribute form a tail. The **exact** $\delta(\varepsilon)$ (Canonne, Kamath & Steinke 2020, Theorem 7) is

$$\delta(\varepsilon) = \Pr\left[X > \frac{\varepsilon\sigma^2}{\Delta} - \frac{\Delta}{2}\right] - e^{\varepsilon}\Pr\left[X > \frac{\varepsilon\sigma^2}{\Delta} + \frac{\Delta}{2}\right], \qquad X \sim \mathcal{N}_{\mathbb{Z}}(0,\sigma^2), \tag{24.13}$$

the same expression as the continuous Gaussian's analytic curve with $X \sim \mathcal{N}(0,\sigma^2)$, only with the tails of the discrete distribution in place of the continuous ones.

Third, the discrete Gaussian is never worse than the continuous one in Rényi divergence. The **Rényi divergence** of order $\alpha > 1$, $D_\alpha(P\|Q) = \frac{1}{\alpha-1}\ln\sum_x p(x)^\alpha q(x)^{1-\alpha}$, is a way of averaging the privacy loss that weights large losses more as α grows; for continuous Gaussians with means μ and ν and variance σ^2 it equals $\alpha(\mu-\nu)^2/(2\sigma^2)$. For every $\alpha > 1$,

$$D_\alpha(\mathcal{N}_{\mathbb{Z}}(\mu,\sigma^2)\,\|\,\mathcal{N}_{\mathbb{Z}}(\nu,\sigma^2)) \le \frac{\alpha(\mu-\nu)^2}{2\sigma^2}, \tag{24.14}$$

with equality for integer α.

Fourth, **the variance is below** σ^2. Restricting the bell curve to integers pulls a little mass towards 0, by an amount that the Poisson summation lemma in the derivation computes exactly:

$$\operatorname{Var} X = \sigma^2 - \frac{\sum_{k\neq 0} 4\pi^2\sigma^4 k^2 e^{-2\pi^2\sigma^2k^2}}{1+\sum_{k\neq 0} e^{-2\pi^2\sigma^2k^2}} < \sigma^2. \tag{24.15}$$

24.7.3. Derivation

Privacy loss and $\delta(\varepsilon)$. Take $f(D) = 0$ and $f(D') = \Delta$; the general case shifts everything by an integer, which changes nothing.

1. **The normalizer cancels.** Under D the output y has probability $e^{-y^2/(2\sigma^2)}/Z$, and under D' it has $e^{-(y-\Delta)^2/(2\sigma^2)}/Z$, with the same $Z = \sum_y e^{-y^2/(2\sigma^2)}$, because shifting the sum by the integer Δ only relabels its terms. So the ratio is the ratio of the two exponentials, and
$$L_{D,D'}(y) = \frac{(y-\Delta)^2 - y^2}{2\sigma^2} = \frac{\Delta^2 - 2\Delta y}{2\sigma^2}, \tag{24.16}$$
the formula of the continuous Gaussian, now only at integers y.
2. **δ as an excess.** (ε,δ)-DP asks $\Pr[M(D)\in S] - e^{\varepsilon}\Pr[M(D')\in S] \le \delta$ for every set S. The left side is a sum of $p(y) - e^{\varepsilon}q(y)$ over $y \in S$, where p is the PMF under D and q under D', so it is largest when S holds exactly the outputs with a positive term. The smallest valid δ is therefore $\delta(\varepsilon) = \sum_y \big(p(y) - e^{\varepsilon}q(y)\big)_+$, where $(\cdot)_+$ keeps positive values and replaces negative ones by 0. This is the hockey-stick divergence of Divergences. The summand is positive exactly when $p(y)/q(y) > e^{\varepsilon}$, that is when $L_{D,D'}(y) > \varepsilon$, which by step 1 means $\Delta^2 - 2\Delta y > 2\sigma^2\varepsilon$, or $y < \frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}$.
3. **The first sum.** $\sum_{y<\Delta/2-\varepsilon\sigma^2/\Delta} p(y) = \Pr[X < \frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}]$. The PMF of X gives x and $-x$ the same probability, so X and $-X$ have the same distribution, and $\Pr[X < c] = \Pr[-X < c] = \Pr[X > -c]$. With $c = \frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}$ this is $\Pr[X > \frac{\varepsilon\sigma^2}{\Delta} - \frac{\Delta}{2}]$.

4. **The second sum.** Under D' the output is $X + \Delta$, so $q(y) = p(y - \Delta)$ and $\sum_{y<\Delta/2-\varepsilon\sigma^2/\Delta} q(y) = \Pr[X + \Delta < \frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}] = \Pr[X < -\frac{\Delta}{2} - \frac{\varepsilon\sigma^2}{\Delta}] = \Pr[X > \frac{\varepsilon\sigma^2}{\Delta} + \frac{\Delta}{2}]$, by the same symmetry. Subtracting e^{ε} times step 4 from step 3 gives Equation 24.13.

A lemma from Poisson summation. The remaining two claims rest on one question: if you sum a Gaussian bump over the integers, how does the sum depend on where the bump is centred? The answer is that it is largest when the centre is an integer. The tool is the Poisson summation formula, which turns a sum of a function over the integers into a sum of its Fourier transform over the integers: $\sum_{x\in\mathbb{Z}} g(x) = \sum_{k\in\mathbb{Z}} \hat{g}(k)$ for a smooth, fast-decaying g with Fourier transform $\hat{g}(k) = \int g(x)e^{-2\pi ikx}dx$ (Stein & Shakarchi 2003, ch. 5). The Fourier transform of a Gaussian is again a Gaussian, and shifting g by c multiplies the transform by the phase $e^{-2\pi ikc}$: for $g(x) = e^{-(x-c)^2/(2\sigma^2)}$, $\hat{g}(k) = \sqrt{2\pi}\sigma e^{-2\pi^2\sigma^2k^2}e^{-2\pi ikc}$. Pairing k with $-k$ adds $e^{-2\pi ikc} + e^{2\pi ikc} = 2\cos(2\pi kc)$, which gives

$$\sum_{x\in\mathbb{Z}} e^{-(x-c)^2/(2\sigma^2)} = \sqrt{2\pi}\,\sigma \sum_{k\in\mathbb{Z}} e^{-2\pi^2\sigma^2k^2}\cos(2\pi kc). \tag{24.17}$$

Every coefficient is positive and $\cos \le 1$, with $\cos(2\pi kc) = 1$ for all k exactly when c is an integer. So the left side is largest at integer c: **shifting the lattice off the integers can only lower the sum.** At $\sigma = 0.5$ the script finds 1.271342 at $c = 0$ and 1.235287 at $c = 0.5$. At $\sigma = 1$ both are 2.506628, which is $\sqrt{2\pi}$ to six decimals: the $k \neq 0$ terms are below 10^{-8}.

Rényi divergences. The idea is that the sum defining D_α is, after completing a square, a Gaussian bump summed over the integers with centre c, and the lemma says that sum is at most its value at an integer centre. Write $p_\mu(x) = e^{-(x-\mu)^2/(2\sigma^2)}/Z$, with Z the same for every integer μ.

1. **Write out the summand.** $D_\alpha = \frac{1}{\alpha-1}\ln\sum_x p_\mu(x)^\alpha p_\nu(x)^{1-\alpha}$. The powers of Z combine to $Z^{-\alpha}Z^{\alpha-1} = 1/Z$, and the exponent is $-\frac{1}{2\sigma^2}(\alpha(x-\mu)^2 + (1-\alpha)(x-\nu)^2)$.
2. **Complete the square.** $\alpha(x-\mu)^2 + (1-\alpha)(x-\nu)^2 = (x-c)^2 - \alpha(\alpha-1)(\mu-\nu)^2$ with $c = \alpha\mu + (1-\alpha)\nu$. (Expand: the x^2 and x terms match by the choice of c, and the constant is what is left.)
3. **So** $\sum_x p_\mu^\alpha p_\nu^{1-\alpha} = e^{\alpha(\alpha-1)(\mu-\nu)^2/(2\sigma^2)} \cdot \frac{1}{Z}\sum_x e^{-(x-c)^2/(2\sigma^2)}$.
4. **Apply the lemma.** Z is the same sum centred at 0, so the last factor is the sum centred at c divided by the sum centred at an integer. By the lemma it is at most 1, with equality when c is an integer, which it is for integer α because μ and ν are integers. Taking $\frac{1}{\alpha-1}\ln$ gives Equation 24.14. The continuous Gaussian has exactly $\alpha(\mu-\nu)^2/(2\sigma^2)$ (Divergences), so the discrete Gaussian is never worse in Rényi divergence. In the language of Privacy accounting, where zero-concentrated DP is the guarantee that every Rényi divergence of order α is at most a constant times α, this is the statement that the discrete Gaussian satisfies the same zero-concentrated DP as the continuous one (Canonne, Kamath & Steinke 2020, Theorem 4).

Variance. The idea is that the normalizer, seen as a function of $1/(2\sigma^2)$, encodes the variance in its derivative, and the lemma gives the normalizer in closed form. Let $\tau = 1/(2\sigma^2)$ and $Z(\tau) = \sum_x e^{-\tau x^2}$.

1. **Variance from the normalizer.** The mean is 0 because x and $-x$ have the same probability, so $\operatorname{Var} X = \mathbb{E}X^2 = \sum_x x^2 e^{-\tau x^2}/Z$. Differentiating $e^{-\tau x^2}$ in τ brings down a factor $-x^2$, so $\frac{d}{d\tau}Z = -\sum_x x^2 e^{-\tau x^2}$, and $\mathbb{E}X^2 = -\frac{Z'(\tau)}{Z(\tau)} = -\frac{d}{d\tau}\ln Z(\tau)$.

2. **Poisson summation at $c = 0$.** Equation 24.17, with $\sigma = 1/\sqrt{2\tau}$, gives $Z(\tau) = \sqrt{\pi/\tau}\,(1 + \theta(\tau))$ with $\theta(\tau) = \sum_{k\neq 0} e^{-\pi^2 k^2/\tau}$.
3. **Differentiate.** The log splits into $\frac{1}{2}\ln\pi - \frac{1}{2}\ln\tau + \ln(1+\theta(\tau))$, so $-\frac{d}{d\tau}\ln Z = \frac{1}{2\tau} - \frac{\theta'(\tau)}{1+\theta(\tau)}$, where $\theta'(\tau) = \sum_{k\neq 0} \frac{\pi^2 k^2}{\tau^2} e^{-\pi^2 k^2/\tau} > 0$.
4. **Substitute $\tau = 1/(2\sigma^2)$:** $\frac{1}{2\tau} = \sigma^2$, $\frac{\pi^2 k^2}{\tau} = 2\pi^2\sigma^2 k^2$ and $\frac{\pi^2 k^2}{\tau^2} = 4\pi^2\sigma^4 k^2$. This is Equation 24.15. The correction is positive, so $\operatorname{Var} X < \sigma^2$, and its leading term, from $k = \pm 1$ with the denominator close to 1, is $8\pi^2\sigma^4 e^{-2\pi^2\sigma^2}$.

24.7.4. Interpretation

The privacy proof is the continuous one. The privacy loss is the same linear function, so every statement about it carries over, restricted to integer points. The only new ingredient is that integer shifts don't change the normalizer. That is why the query must be integer-valued. To use the discrete Gaussian for a real-valued query, scale it by $1/\gamma$ for a small step γ and round it to an integer first. Randomized rounding, which rounds up with probability equal to the fractional part, keeps it unbiased. Then account for the sensitivity of the rounded query, which can be slightly larger than the scaled original.

$\delta(\varepsilon)$ is close to the continuous curve, not equal to it. The privacy loss takes only the values $(\Delta^2 - 2\Delta y)/(2\sigma^2)$ for integer y; moving y by one changes the loss by $2\Delta/(2\sigma^2) = \Delta/\sigma^2$, so these values form a lattice with spacing Δ/σ^2 ($1/\sigma^2$ for $\Delta = 1$). As ε crosses one of these values, a whole output enters or leaves the sum for δ, so $\delta(\varepsilon)$ has a kink at each lattice value and oscillates around the continuous curve (Figure 24.4). At $\varepsilon = 1$ and $\Delta = 1$ the script finds $\delta = 0.6032$ against 0.5099 at $\sigma = 0.5$, 0.1414 against 0.1269 at $\sigma = 1$, and 0.0072 against 0.0068 at $\sigma = 2$. At other ε, such as just past a kink, the discrete curve is below. Neither dominates, and the gap closes as σ grows. In Rényi divergence, by contrast, the discrete Gaussian is never worse: at $\sigma = 0.5$, $D_{2.5} = 4.9808$ against 5.

The variance deficit is invisible in practice. Its relative size $1 - \operatorname{Var} X/\sigma^2$ has leading term $8\pi^2\sigma^2 e^{-2\pi^2\sigma^2}$, the leading term of Equation 24.15 divided by σ^2. At $\sigma = 1$ that is $8\pi^2 e^{-2\pi^2} = 78.96 \times 2.67 \times 10^{-9}$, so $1 - \operatorname{Var} X/\sigma^2 = 2.11 \times 10^{-7}$; at $\sigma = 2$ it is at the level of rounding error. Only below $\sigma \approx 0.6$ does the discrete Gaussian concentrate visibly more than σ^2 suggests: at $\sigma = 0.5$, $\operatorname{Var} X = 0.8601\,\sigma^2$, and at $\sigma = 0.25$ almost all the mass is at 0 (Figure 24.5).

Sums are not discrete Gaussians. Unlike the continuous case, the sum of two independent discrete Gaussians is not exactly a discrete Gaussian. For $\sigma \geq 1$ it is very close (Exercise 6 in the exercise book), which is what distributed noise addition, where each client adds its own share of the noise, relies on.

24.7.5. Example: the Rényi divergence, computed

The script sums the PMFs directly to get $D_\alpha(\mathcal{N}_\mathbb{Z}(0,\sigma^2)\|\mathcal{N}_\mathbb{Z}(1,\sigma^2))$ (Table 24.5). Integer orders match $\alpha/(2\sigma^2)$ exactly, because then the centre c of the completed square is an integer and the lemma holds with equality. The order 2.5 at $\sigma = 0.5$ has $c = 2.5\cdot 0 + (1-2.5)\cdot 1 = -1.5$, half-way between integers, and falls below the continuous value. At $\sigma = 1$ the lemma's slack is too small to show.

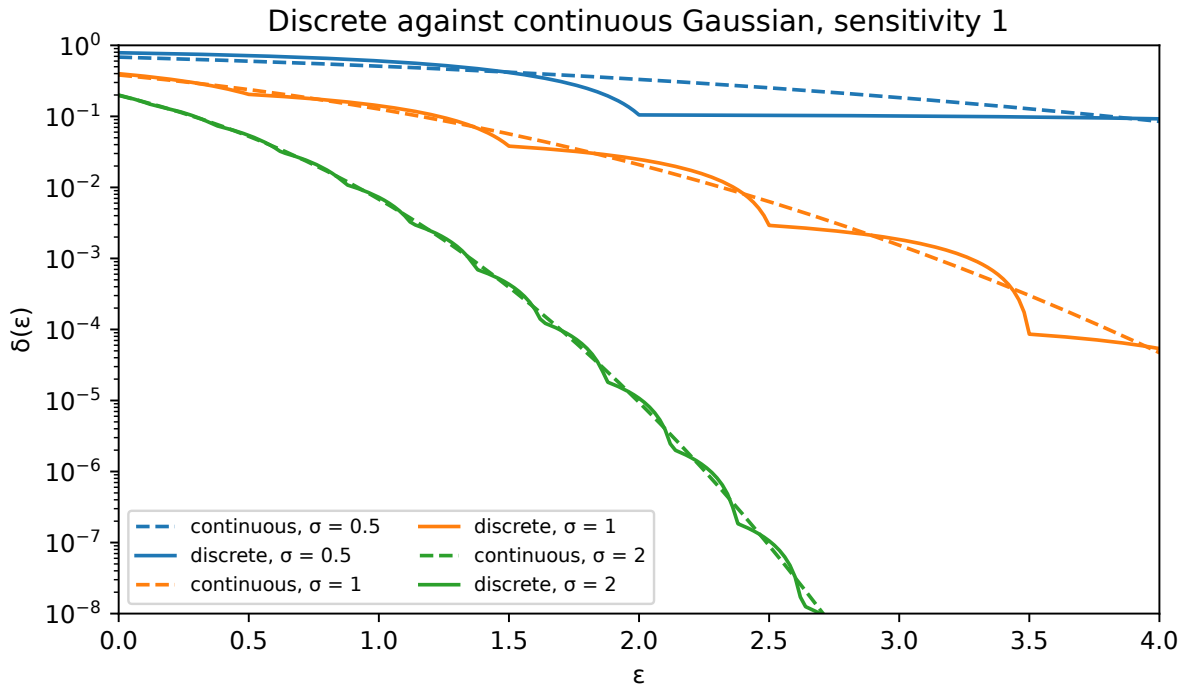


Figure 24.4.: Discrete against continuous Gaussian for sensitivity 1: $\delta(\varepsilon)$ from Equation 24.13 (solid) and the analytic curve of the continuous Gaussian (dashed), for $\sigma = 0.5$, 1 and 2. The discrete curve has a kink at each value of the lattice-valued privacy loss.

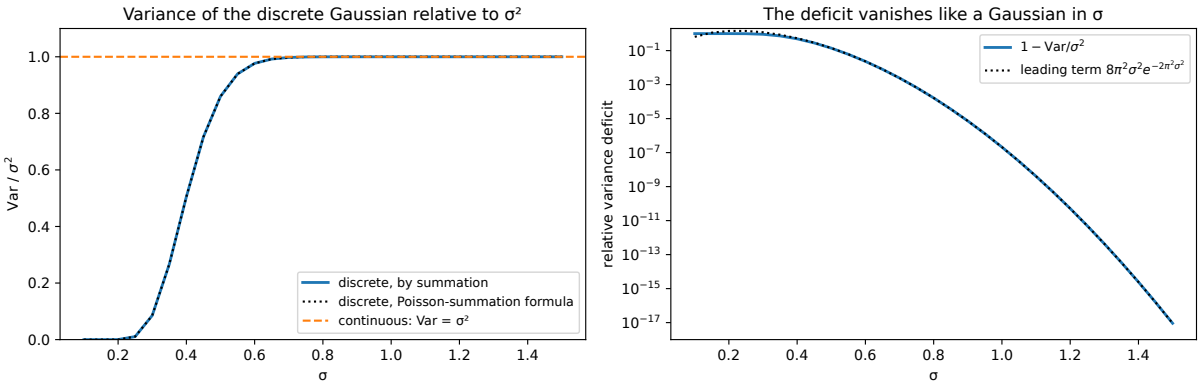


Figure 24.5.: Left: $\operatorname{Var} X/\sigma^2$ for $X \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$, by direct summation and from Equation 24.15, against 1 for the continuous Gaussian. Right: the relative deficit $1 - \operatorname{Var} X/\sigma^2$ on a log scale, with its leading term $8\pi^2\sigma^2 e^{-2\pi^2\sigma^2}$.

Table 24.5.: Rényi divergences between discrete Gaussians shifted by 1, by direct summation, with the continuous value $\alpha/(2\sigma^2)$ in parentheses.

σ	D_2 (continuous)	$D_{2.5}$ (continuous)	D_5 (continuous)
0.5	4.0000 (4.0000)	4.9808 (5.0000)	10.0000 (10.0000)
1	1.0000 (1.0000)	1.2500 (1.2500)	2.5000 (2.5000)
2	0.2500 (0.2500)	0.3125 (0.3125)	0.6250 (0.6250)

24.7.6. Check yourself

- Where does the proof of Equation 24.14 use that μ and ν are integers? *(In writing both PMFs with the same normalizer Z. For non-integer locations, the normalizers differ by the lemma, and the clean formula breaks.)*
- At $\sigma = 1$ and $\Delta = 1$, where are the kinks of the discrete $\delta(\varepsilon)$ curve? *(At the values $(1-2y)/2$ of the privacy loss that are positive: $\varepsilon = 0.5, 1.5, 2.5, \ldots$, as in Figure 24.4.)*

24.8. Sampling the discrete Gaussian exactly

24.8.1. Context

The privacy analysis is only as good as the sampler. Inverse-CDF sampling of $\mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$ needs the normalizer, which is irrational, and a floating-point uniform, which brings back the problems of Section 24.3. Algorithm 3 of Canonne, Kamath and Steinke (2020) instead proposes from the discrete Laplace and thins with an exact $e^{-\gamma}$ coin.

24.8.2. Statement

The discrete Laplace has heavier tails than the discrete Gaussian, so it can serve as the proposal of rejection sampling (Fact 2): draw from it, and throw draws away with a probability chosen so that proposal times acceptance is proportional to $e^{-y^2/(2\sigma^2)}$. The acceptance probability is a Gaussian factor in $|y|$, shifted so that it cancels the Laplace factor exactly.

Input a rational $\sigma^2 > 0$. Let $t = \lfloor \sigma \rfloor + 1$. Repeat: draw $Y \sim \mathrm{Lap}_{\mathbb{Z}}(t)$ with Algorithm 2 ($s = 1$), and accept it with probability

$$\exp\Big(- \frac{(|Y| - \sigma^2/t)^2}{2\sigma^2} \Big) \tag{24.18}$$

using Algorithm 1. Output the first accepted Y. The output is exactly $\mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$. The acceptance probability per proposal is the proposal's normalizing constant times the constant left over from the cancellation times the discrete Gaussian's unnormalized total mass:

$$A = \frac{1-r}{1+r}\, e^{-\sigma^2/(2t^2)} \sum_{y \in \mathbb{Z}} e^{-y^2/(2\sigma^2)}, \qquad r = e^{-1/t}, \tag{24.19}$$

which tends to $\sqrt{\pi/2}\, e^{-1/2} = 0.7602$ as σ grows.

24.8.3. Derivation

1. **Proposal times acceptance.** By Fact 2, the output is the target if proposal times acceptance is proportional to $e^{-y^2/(2\sigma^2)}$. The proposal has $\Pr[Y = y] \propto e^{-|y|/t}$. Multiply by the acceptance probability, add the exponents and expand the square:

$$-\frac{|y|}{t} - \frac{(|y| - \sigma^2/t)^2}{2\sigma^2} = -\frac{|y|}{t} - \frac{y^2}{2\sigma^2} + \frac{|y|}{t} - \frac{\sigma^2}{2t^2} = -\frac{y^2}{2\sigma^2} - \frac{\sigma^2}{2t^2}. \tag{24.20}$$

   The $|y|/t$ terms cancel, and what is left is $e^{-y^2/(2\sigma^2)}$ times a constant. By Fact 2 the output is $\mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$.
2. **The acceptance probability is valid.** The exponent $-(|y| - \sigma^2/t)^2/(2\sigma^2)$ is at most 0, so its exponential is at most 1 and is a probability.
3. **Everything is rational.** t is an integer, computed as $\lfloor\sqrt{\lfloor\sigma^2\rfloor}\rfloor + 1$ with integer square roots (an integer n has $n^2 \le \sigma^2$ exactly when $n^2 \le \lfloor\sigma^2\rfloor$, so this equals $\lfloor\sigma\rfloor + 1$ without computing σ). So $\gamma = (|y| - \sigma^2/t)^2/(2\sigma^2)$ is a ratio of integers, and Algorithm 1 applies.
4. **The acceptance rate.** $A = \sum_y \Pr[Y = y]\, e^{-(|y|-\sigma^2/t)^2/(2\sigma^2)}$, the average acceptance over proposals. With $\Pr[Y = y] = \frac{1-r}{1+r} r^{|y|}$, step 1 turns each term into $\frac{1-r}{1+r} e^{-\sigma^2/(2t^2)} e^{-y^2/(2\sigma^2)}$, and summing over y gives Equation 24.19. Exercise 5 in the exercise book derives the limit.

Concretely, for $\sigma^2 = 4$ ($t = \lfloor 2\rfloor + 1 = 3$, $r = e^{-1/3}$, $\frac{1-r}{1+r} = 0.1651$). Table 24.6 computes a few rows by hand.

Table 24.6.: Algorithm 3 for $\sigma^2 = 4$, by hand. The product is $\propto e^{-y^2/8}$: divide each row by the first and you get $e^{-y^2/8}$, for instance $0.1167/0.1322 = 0.883 = e^{-1/8}$.

y	Proposal $0.1651\, r^{\lvert y\rvert}$	$\gamma = (\lvert y\rvert - 4/3)^2/8$	Acceptance $e^{-\gamma}$	Product
0	0.1651	2/9	0.8007	0.1322
1	0.1183	1/72	0.9862	0.1167
2	0.0848	1/18	0.9460	0.0802
4	0.0435	8/9	0.4111	0.0179
6	0.0223	49/18	0.0657	0.0015

24.8.4. Interpretation

The sampler is exact and cheap. Every step is integer or rational arithmetic. The acceptance rate is 0.5185 at $\sigma^2 = \frac{1}{4}$, 0.6629 at $\sigma^2 = 4$ and 0.7532 at $\sigma^2 = 100$, so a draw takes fewer than two proposals on average ($1/A$ by Fact 2, at most $1/0.5185 < 2$ here), and between 15 and 27 uniform integers in the script's runs (Table 24.7).

The choice $t = \lfloor\sigma\rfloor + 1$ only affects cost. Any integer $t \ge 1$ gives an exact sampler, because step 1 of the derivation works for every t. For large σ, $\frac{1-r}{1+r} \approx \frac{1}{2t}$ (since $r = e^{-1/t} \approx 1 - 1/t$) and the sum in Equation 24.19 is about $\sqrt{2\pi}\sigma$ by Equation 24.17, so $A \approx \frac{\sqrt{2\pi}\sigma}{2t} e^{-\sigma^2/(2t^2)}$. The derivative of $\ln A$ in t is $-\frac{1}{t} + \frac{\sigma^2}{t^3}$, which is zero at $t = \sigma$, and $\lfloor\sigma\rfloor + 1$ is the nearest integer above it.

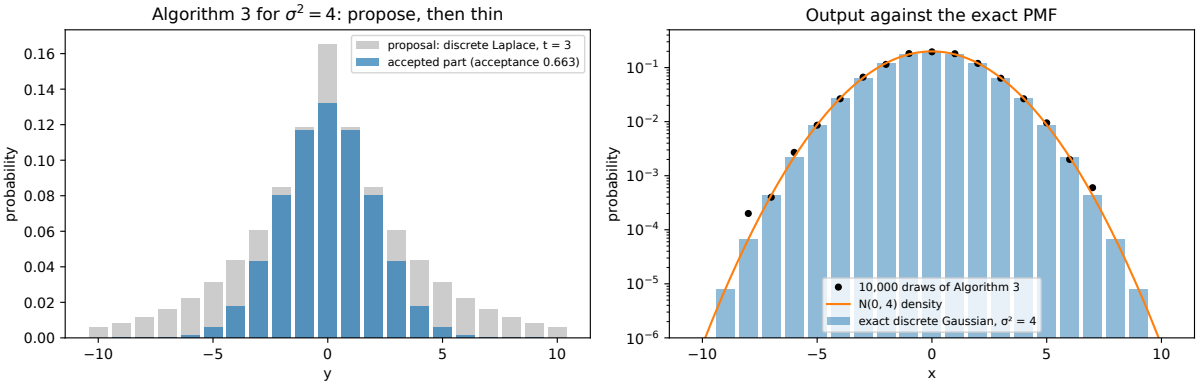


Figure 24.6.: Algorithm 3 for $\sigma^2 = 4$. Left: the discrete Laplace proposal with $t = 3$ and the part of it that is accepted, which is A times the discrete Gaussian PMF. Right: 10,000 draws of Algorithm 3 against the exact PMF on a log scale, with the $\mathcal{N}(0, 4)$ density for comparison.

Exactness is not timing safety. The number of rounds depends on the output, so the running time leaks information about the noise. Jin et al. (2022) show such leaks are exploitable; a deployment pads the running time or runs a fixed number of rounds.

24.8.5. Example: a histogram over 1,000 bins

Release a histogram of 1,000 bins under add/remove neighbours, where one record changes one bin by 1, so $\Delta_2 f = 1$. Add $\mathcal{N}_{\mathbb{Z}}(0, 100)$ to each bin. The noise in the 999 unchanged bins has the same distribution under both neighbours, so those bins add nothing to the privacy loss, and the privacy loss is that of one bin. Equation 24.13 with $\Delta = 1$ and $\sigma = 10$ gives its $\delta(\varepsilon)$, essentially the continuous Gaussian's at $\sigma = 10$. The script's sampler at $\sigma^2 = 100$ uses about 15.6 uniform integers per bin, so the whole histogram needs about $15.6 \times 1{,}000 = 15{,}600$ uniform integers and no floating-point operation.

24.8.6. Check yourself

- If you dropped the σ^2/t inside the acceptance probability, accepting with $e^{-y^2/(2\sigma^2)}$ instead, what would the output be? *(Proposal times acceptance would be $\propto e^{-|y|/t - y^2/(2\sigma^2)}$, a discrete Gaussian tilted by a Laplace factor, not $\mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$.)*
- Why does the sampler not need the normalizer $\sum_y e^{-y^2/(2\sigma^2)}$? *(Rejection sampling only needs proposal times acceptance to be proportional to the target, Fact 2. The normalizer sets the acceptance rate, not the output.)*

24.9. Code: check every sampler against its exact distribution

The script code/discrete_mechanisms.py implements Algorithms 1–3 with `fractions.Fraction` and integers, drawing uniform integers from a seeded `random.Random` (swap in `secrets.SystemRandom()`

to deploy). It also enumerates the toy floating-point sampler exactly, and computes the discrete Gaussian's PMF, variance, Rényi divergences and $\delta(\varepsilon)$ by summation. From the repository root:

```
uv run python code/part03-dp-foundations/discrete_mechanisms.py
```

Output (seed 0, 10,000 draws per sampler), summarized in Table 24.7. To compare a sampler with its exact distribution, the script uses the total variation (TV) distance: half the sum, over outputs, of the absolute difference between the exact probability and the observed frequency, which is the largest difference the two can show on any event. Even a perfect sampler shows some TV distance after 10,000 draws, from sampling noise alone. The "noise floor" measures how much: it is the TV distance between the exact PMF and the empirical PMF of 10,000 draws from it made with NumPy's sampler.

Table 24.7.: The script's output with the default seed.

Experiment	Result
Float Laplace, $p = 12$	$\Pr[M(D) \in S] = 0.4791$; snapped to integers: 0, with $\max\|L\| = 1.0010$
Algorithm 1, $\gamma = \frac{1}{2}, 1, \frac{5}{2}, 4$	Frequencies 0.6024, 0.3709, 0.0869, 0.0168 against 0.6065, 0.3679, 0.0821, 0.0183; draws per coin 1.662, 3.096, 3.976, 4.218 against 1.649, 3.086, 3.941, 4.240
Algorithm 2, $b = 1$	Variance 1.8305 (exact 1.8413); TV to exact 0.0114 (noise floor 0.0074); 11.49 draws per sample
Algorithm 2, $b = 3/2$	Variance 4.3834 (exact 4.3370); TV 0.0119 (noise floor 0.0243); 12.08 draws per sample
Algorithm 2, $b = 10$	Variance 195.2985 (exact 199.8334); TV 0.0339 (noise floor 0.0331); 10.25 draws per sample
Algorithm 3, $\sigma^2 = \frac{1}{4}$	Variance 0.2189 (exact 0.2150); TV 0.0028 (noise floor 0.0045); acceptance 0.5185; 26.6 draws
Algorithm 3, $\sigma^2 = 1$	Variance 0.9977 (exact 1.0000); TV 0.0076 (noise floor 0.0042); acceptance 0.5418; 24.8 draws
Algorithm 3, $\sigma^2 = 4$	Variance 4.0826 (exact 4.0000); TV 0.0154 (noise floor 0.0142); acceptance 0.6629; 18.9 draws
Algorithm 3, $\sigma^2 = 100$	Variance 99.4386 (exact 100.0000); TV 0.0243 (noise floor 0.0325); acceptance 0.7532; 15.6 draws

Experiment	Result
Discrete Gaussian variance	$\mathrm{Var}/\sigma^2 = 0.010728, 0.860051, 1.000000$ at $\sigma = 0.25, 0.5, 1$, by summation and by Equation 24.15 alike

tests/test_discrete_mechanisms.py checks that the output still matches.

What to take from it

The samplers are exact, up to sampling noise. Every TV distance is within about a factor 2 of the noise floor, sometimes below it, and every frequency and variance is within a few standard errors of its exact value. To see what a few standard errors means for a variance: a variance estimated from n draws has a relative standard error of about $\sqrt{(\kappa-1)/n}$, where κ is the kurtosis, the fourth moment divided by the squared variance. The Gaussian has $\kappa = 3$, which gives $\sqrt{2/10{,}000} \approx 1.4\%$; the Laplace has $\kappa = 6$, which gives $\sqrt{5/10{,}000} \approx 2.2\%$. The largest deviation, at $b = 10$, is $(199.8334 - 195.2985)/199.8334 \approx 2.3\%$ low, about one standard error. The noise floor varies a lot between runs, so only the order of magnitude matters.

The costs match the formulas. Algorithm 1's draws per coin match Equation 24.7 and its extension to $\gamma > 1$ to within 1%.

The float leak doesn't shrink with precision, and snapping removes it: the first row reproduces Table 24.2 and the snapped example.

The discrete Gaussian behaves like the continuous one for $\sigma \geq 1$, in privacy and in variance. Below that, its variance is smaller than σ^2 and its $\delta(\varepsilon)$ oscillates more around the continuous curve.

24.10. References

- Mironov, "On Significance of the Least Significant Bits for Differential Privacy" (CCS 2012). The floating-point attack on the textbook Laplace sampler, the snapping mechanism and its proof (Theorem 1 and §5).
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020; journal version in the Journal of Privacy and Confidentiality, 2022). Algorithms 1–3, the exact $\delta(\varepsilon)$ (Theorem 7), the Rényi and concentrated-DP guarantee (Theorem 4), the variance and tail bounds via Poisson summation. Read §5 alongside the code here.
- Ghosh, Roughgarden & Sundararajan, "Universally Utility-Maximizing Privacy Mechanisms" (STOC 2009; SIAM Journal on Computing, 2012). The geometric mechanism and its optimality for a single count.
- von Neumann, "Various Techniques Used in Connection with Random Digits" (1951). The original trick behind Algorithm 1: exponential probabilities from comparisons of uniform variables.
- Jin, McMurtry, Rubinstein & Ohrimenko, "Are We There Yet? Timing and Floating-Point Attacks on Differential Privacy Systems" (IEEE S&P 2022). Floating-point and timing attacks on DP libraries released after Mironov's paper, including attacks on samplers whose running time depends on the output.

- Kairouz, Liu & Steinke, "The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation" (ICML 2021). The discrete Gaussian when each client adds its own noise, and how close sums of discrete Gaussians are to a discrete Gaussian.
- Abowd et al., "The 2020 Census Disclosure Avoidance System TopDown Algorithm" (Harvard Data Science Review, 2022). The largest deployment of the discrete Gaussian.
- Stein & Shakarchi, *Fourier Analysis: An Introduction* (2003), ch. 5. The Poisson summation formula and the theta function behind Equation 24.17.

24.11. Cheat sheet

$r = e^{-1/b}$ for the discrete Laplace; $X \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$ for the discrete Gaussian; Δ is the integer change in f between neighbours. Table 24.8 collects the results.

Table 24.8.: Every result in this chapter.

Object	Formula	Fact to remember
Support argument	$\delta \geq \Pr[M(D) \in \mathcal{Y}_D \setminus \mathcal{Y}_{D'}]$	Outputs reachable from one input only rule out every ε
Floating-point Laplace	$\operatorname{round}(f(D) + \xi \operatorname{round}(b \ln(1/U)))$	Not DP: about 48% of the mass identifies $f(D)$ in the toy, at every precision
Snapping	$\operatorname{clamp}_B(\lfloor \operatorname{clamp}_B(f) \oplus \xi \otimes \lambda \otimes \mathrm{LN}(U^*) \rceil_\Lambda)$	$\varepsilon = \frac{1+12B\eta}{\lambda} + \frac{2^{-49}B}{\lambda}$ for sensitivity 1
Bernoulli$(e^{-\gamma})$, $\gamma \leq 1$	Parity of the first 0 among Bernoulli(γ/k)	$\Pr[K > k] = \gamma^k/k!$; e^γ draws on average
$\mathrm{Lap}_{\mathbb{Z}}(b)$	$\frac{1-r}{1+r} r^{\lvert z \rvert}$	Variance $\frac{2r}{(1-r)^2} \approx 2b^2 - \frac{1}{6}$
Geometric mechanism	$f(D) + \mathrm{Lap}_{\mathbb{Z}}(b)$	$\varepsilon = \Delta_1 f / b$, pure DP
$\mathcal{N}_{\mathbb{Z}}(0, \sigma^2)$	$\propto e^{-x^2/(2\sigma^2)}$ on $\mathbb{Z}$	$\mathrm{Var} = \sigma^2 - O(\sigma^4 e^{-2\pi^2\sigma^2})$
Discrete Gaussian mechanism	$L(y) = \frac{\Delta^2 - 2\Delta y}{2\sigma^2}$ at integers	$\delta(\varepsilon)$ by Equation 24.13; $D_\alpha \leq \frac{\alpha\Delta^2}{2\sigma^2}$
Algorithm 3	Propose $\mathrm{Lap}_{\mathbb{Z}}(\lfloor\sigma\rfloor + 1)$, accept w.p. $e^{-(\lvert y\rvert - \sigma^2/t)^2/(2\sigma^2)}$	Acceptance $\to 0.7602$; exact

Rules of thumb

- Never ship `f(D) + laplace(b)` in floating point. Either snap the output to a grid at least as coarse as b and clamp it, or make the query integer-valued and use an exact integer sampler.
- An exact sampler uses only integers, rationals and uniform integers from a cryptographic source. Any `exp`, `log` or `random()` in the sampler is a place where the analysed distribution and the real one can part.

- The privacy proofs of the discrete Laplace and Gaussian are the continuous proofs, because integer shifts leave the normalizer unchanged.
- For $\sigma \geq 1$, treat the discrete Gaussian as the continuous one for both accuracy and privacy; check $\delta(\varepsilon)$ with Equation 24.13 when it matters.
- Exactness of the distribution is not the end: running time that depends on the output is a separate side channel.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

25. Local, shuffle and distributed trust models

25.1. Why this matters

Every mechanism so far in this part assumed a **trusted curator**: someone who holds the raw data, runs the mechanism, and releases only its output. The guarantee of differential privacy (DP) is about that output. It says nothing about the curator, the database it keeps, the engineer who can read it, or the court order that can demand it. Threat models put it in one line: trust decides who the adversary is. In the central model the curator is outside the threat model. In many deployments it cannot be. A browser vendor collecting usage statistics, a keyboard app learning new words, or a phone operating system counting crashes is exactly the party users do not want to see their data.

This chapter answers one question: **how much accuracy does each weaker trust assumption cost, and what do you have to trust instead?** There are four answers, drawn in Figure 25.1. They are four parallel options, so they are listed:

- **Central.** Trust the curator with everything. The noise is added once, to the answer, and the error does not grow with the number of people n.
- **Local.** Trust nobody. Each person randomizes their own record before it leaves their device. The noise is added n times, and for counting the error is a factor of about $\sqrt{n}$ worse than central. No protocol can avoid this.
- **Shuffle.** Trust a shuffler only to cut the link between reports and senders. Local reports become anonymous, and anonymity amplifies privacy: reports randomized at a weak local level ε_0 give a strong central ε, so the error is close to central again.
- **Distributed.** Trust cryptography. Each person adds a small share of the noise, and a secure aggregation protocol reveals only the sum. The total noise equals the central noise, as long as enough people are honest.

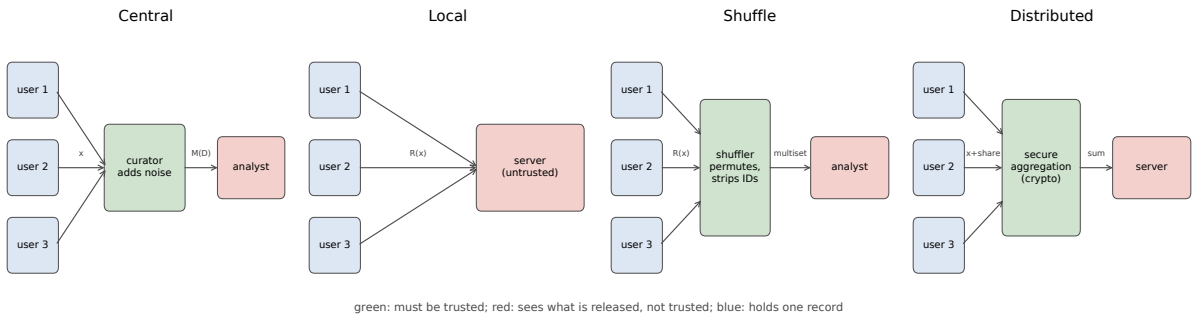


Figure 25.1.: The four trust models. Blue boxes hold one record each, green boxes must be trusted, and red boxes see what is released and are not trusted. Only the central model gives raw records to a single party.

25.1.1. The one idea: where the noise enters

All four models release a noisy statistic. They differ only in where the noise is added and who sees the data before it is. The further upstream the noise enters, the fewer parties you trust, and the more noise reaches the answer.

Here is why. A local report has to hide its owner from everyone, including the analyst, so each report carries noise of the size the central mechanism adds once in total. When you add up n independent noises, their variances add, so the total noise has standard deviation of order $\sqrt{n}$ times one report's. The signal, the true count, grows like n. So the relative error of a local count shrinks only like $\sqrt{n}/n = 1/\sqrt{n}$, while a central count with one dose of noise has relative error of order $1/n$. The shuffle and distributed models move the noise back downstream without giving the data back to one party: anonymity hides which report is whose, and cryptography hides everything except the sum.

Table 25.1 lists the results and where each one is used.

Table 25.1.: The results of this chapter and where each one is used.

Result	Where it is used
A local randomizer makes the whole protocol ε-DP	Deciding what a locally private deployment promises, and against whom
Unbiased estimation from randomized reports	Every local frequency estimate; debiasing any linear randomizer
The local lower bound: error $\gtrsim 1/((e^{\varepsilon}-1)\sqrt{n})$	Knowing when local DP is hopeless, and why the other models exist
k-ary randomized response and unary encoding	Frequency estimation over large domains, as in browser and phone telemetry
Amplification by shuffling (Feldman, McMillan & Talwar 2021)	Choosing the local ε_0 that a shuffler turns into a target central ε
Distributed noise with secure aggregation	Central accuracy without a trusted curator, in federated settings; how collusion erodes it

The chapter uses the definition, the hockey-stick form and randomized response from The definition of differential privacy, post-processing from Post-processing, group privacy and composition, and the Laplace and Gaussian mechanisms. From Part 1 it uses concentration (Concentration inequalities), Pinsker's inequality (Divergences) and additive secret sharing (Cryptography basics). Each of them is restated where it is used. The derivation of amplification by shuffling belongs to Privacy accounting; here it is a cited result that you compute with.

The guarantee, restated. A randomized mechanism M is (ε, δ)**-DP** if, for every pair of neighbouring datasets D, D' and every set S of possible outputs, $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta$. In words: changing one person's record changes the probability of any outcome by at most a factor e^{ε}, up to a small slack δ. With $\delta = 0$ this is **pure** ε-DP.

Notation. n people hold records $x_1, \dots, x_n$ from a domain $\mathcal{X}$. A **local randomizer** R maps one record to a random report, and $p_R(y \mid x)$ is the probability that it outputs report y when the record is x. When local and central guarantees appear together, ε_0 is the local level of one report and (ε, δ) the central guarantee of everything released. Neighbouring datasets differ by **replacing** one record, in every model of this chapter. The reason is that a report has to hide its owner's value, while the number of reports already reveals who took part, so no randomizer of the value could hide participation; The definition of differential privacy makes the same point for randomized response. Frequencies are fractions in $[0, 1]$; an error on a fraction becomes n times larger on a count.

25.2. Toolkit: three facts every derivation uses

25.2.1. Fact 1 — Debiasing a linear randomizer

A randomizer shrinks the signal: the mean of a report moves less with the true value than the true value itself does. If you know by how much it shrinks, you can undo it. Precisely, suppose a report Y has mean $\mathbb{E}[Y] = a + b\,\theta$ for an unknown quantity θ and known numbers a and $b \neq 0$. Then subtracting the offset and dividing by the slope,

$$\hat{\theta} = \frac{Y - a}{b}, \tag{25.1}$$

is unbiased for θ, with $\operatorname{Var}\hat{\theta} = \operatorname{Var}(Y)/b^2$.

The proof is two lines. Linearity of expectation gives $\mathbb{E}\hat{\theta} = (a + b\theta - a)/b = \theta$. Subtracting a constant does not change a variance, and scaling by $1/b$ multiplies it by $1/b^2$.

Every local estimator in this chapter is of this form. The slope b is the gap between "report 1 when the true value is v" and "report 1 when it is not". A strongly private randomizer makes that gap small, and the estimator then has to scale the report up by $1/b$, which multiplies the noise variance by $1/b^2$. That is where the cost of local privacy comes from.

25.2.2. Fact 2 — Variances of independent reports add

For independent reports $Y_1, \dots, Y_n$, the variance of a sum is the sum of the variances, and dividing by n divides the variance by n^2:

$$\operatorname{Var}\Big(\frac{1}{n}\sum_i Y_i\Big) = \frac{1}{n^2}\sum_i \operatorname{Var} Y_i. \tag{25.2}$$

A Bernoulli report, which is 1 with probability π_i and 0 otherwise, has variance $\pi_i(1 - \pi_i)$. Together with Fact 1, this gives the exact variance of every frequency estimate below, with no approximation.

25.2.3. Fact 3 — A test built from an estimator

A good estimator can tell two hypotheses apart, so if two hypotheses are hard to tell apart, no estimator can be good. This fact turns that into a number, and it is how the chapter proves lower bounds on error.

It uses the **total variation (TV) distance** between two distributions P and Q of the data: half the total absolute difference between their probability tables, $\mathrm{TV}(P,Q) = \frac{1}{2}\sum_z |P(z) - Q(z)|$. From Hypothesis testing, every test has false positive rate plus false negative rate at least $1 - \mathrm{TV}$. The reason is short: a test that decides "Q" on a set A of data has false positive rate $P(A)$ and false negative rate $1 - Q(A)$, so their sum is $1 - (Q(A) - P(A))$, and $Q(A) - P(A)$ can never exceed TV, which is the largest such difference.

Now the fact. Suppose an estimator $\hat{\theta}$ is within $\alpha/2$ of the truth with probability at least $1 - \beta$, both when the truth is θ_0 and when it is $\theta_1 = \theta_0 + \alpha$. Use it to test: decide θ_1 if $\hat{\theta} > \theta_0 + \alpha/2$, the midpoint between the two values. When the truth is θ_0, the test is wrong only if the estimate lands more than $\alpha/2$ above the truth, which happens with probability at most β. When the truth is θ_1, the test is wrong only if the estimate lands at least $\alpha/2$ below it, again with probability at most β (an estimate exactly at the midpoint can be sent either way; with continuous estimates it has probability 0). So both error rates are at most β, their sum is at most 2β, and the bound $1 - \mathrm{TV} \leq 2\beta$ gives that the TV distance between the two distributions of the data is at least $1 - 2\beta$.

Read the other way round, this turns an upper bound on how much n reports can reveal (a small TV) into a lower bound on any estimator's error.

25.3. Central and local differential privacy

25.3.1. Context

In the central model a curator holds $D = (x_1, \ldots, x_n)$ and releases $M(D)$. In the **local model** there is no curator. Each person runs a randomizer on their own record and sends only the result. The question of this section: what does a local randomizer promise, to whom, and what does the change cost on the simplest task, counting?

25.3.2. Statement

A local randomizer has to make each person's report look about the same whatever their record is, because whoever receives it may know whose it is. So the requirement is the DP definition applied to a dataset of one record, with every pair of records as neighbours. A randomizer $R : \mathcal{X} \to \mathcal{Y}$ is an ε_0**-DP local randomizer** if for every pair of records $x, x' \in \mathcal{X}$ and every event $S \subseteq \mathcal{Y}$,

$$\Pr[R(x) \in S] \leq e^{\varepsilon_0} \Pr[R(x') \in S]. \tag{25.3}$$

This is the definition of differential privacy for one record. The **local protocol** releases the transcript $(R(x_1), \ldots, R(x_n))$, with independent randomness for each person. Then:

1. The transcript is ε_0-DP in the central sense, under replace.

2. Each report on its own is ε_0-DP against anyone who sees it, including the party that collects it.

25.3.3. Derivation

The idea is that replacing one person's record changes only that person's report, so the privacy of the whole transcript is the privacy of one report.

1. **The transcript factorizes.** People use independent randomness, and the probability of independent events is the product of their probabilities: $p(y_1, \dots, y_n \mid D) = \prod_i p_R(y_i \mid x_i)$.
2. **Neighbours differ in one factor.** If D and D' differ only in record j, every factor with $i \neq j$ is the same under both and cancels in the ratio of the two probabilities. What remains, the **privacy loss** of the transcript, is $\ln \frac{p_R(y_j|x_j)}{p_R(y_j|x'_j)}$. Applying Equation 25.3 to the single outcome $S = \{y_j\}$, in both directions, bounds it by ε_0 in absolute value. A bound on every single outcome's ratio extends to every event by adding the outcomes in the event. This is the argument The definition of differential privacy made for randomized response, for any R.
3. **Who sees the report does not matter.** Equation 25.3 involves only the distribution of one report. It holds for the network, the collector, the analyst and anyone they hand the data to. Any later processing of the reports, including the curator's estimate, is covered by **post-processing** (Post-processing, group privacy and composition): a function applied to a DP output, without access to the data, is DP with the same parameters, because it can only blur the output's dependence on the data, never sharpen it.

25.3.4. Interpretation

The guarantee no longer depends on the curator. A breach of the collector's database, a malicious insider or a subpoena reveals reports, and reports are already private. That is why the large deployments of the 2010s (Google's RAPPOR in Chrome, Apple's telemetry in iOS and macOS) used local DP.

But every person pays separately. Each report must hide its owner from everyone, so each carries noise of the size the central mechanism adds once in total. Table 25.2 sets out what you trust in each model and what it costs. The error column is derived in the sections below; the local entry follows from the example right after the table, where the standard deviation of a count is n times that of a fraction, $n \cdot 1/(\varepsilon\sqrt{n}) = \sqrt{n}/\varepsilon$.

Table 25.2.: The four trust models: who is trusted, what the untrusted party sees, and the order of the error of a count.

Model	Who must be trusted	What the untrusted party sees	Error of a count, ε fixed
Central	The curator, with all raw records	$M(D)$	Order $1/\varepsilon$, independent of n
Local	Nobody	Every report $R(x_i)$	Order $\sqrt{n}/\varepsilon$

Model	Who must be trusted	What the untrusted party sees	Error of a count, ε fixed
Shuffle	The shuffler, to unlink reports from senders	The multiset of reports	Order $\sqrt{\ln(1/\delta)}/\varepsilon$
Distributed	The cryptography, and enough honest clients	The noisy sum	Order $\sqrt{\ln(1/\delta)}/\varepsilon$

25.3.5. Example: one survey, two trust models

Of $n = 10{,}000$ people, 3,000 hold a 1. Each person uses **randomized response** at $\varepsilon = \ln 3$: report the true bit with the keep probability $t = e^{\varepsilon}/(1 + e^{\varepsilon}) = 3/4 = 0.75$, and the flipped bit otherwise. The mean $\bar{y}$ of the reports has expectation $(1 - t) + \pi(2t - 1)$, where π is the true fraction of ones, so by Fact 1 the estimate $\hat{\pi} = (\bar{y} - (1 - t))/(2t - 1)$ is unbiased. Every report is a Bernoulli variable with variance $t(1 - t)$, whatever its true bit, so by Fact 2 and Fact 1 its standard deviation is $\sqrt{t(1-t)}/((2t-1)\sqrt{n})$, which The definition of differential privacy rewrites as $\sqrt{e^{\varepsilon}/(n(e^{\varepsilon} - 1)^2)}$. With $e^{\varepsilon} = 3$ and $(e^{\varepsilon} - 1)^2 = 4$ this is $\sqrt{0.75/10{,}000} = 0.00866$. In 20,000 simulated surveys the mean estimate is 0.3000 and the standard deviation 0.00861. On the count, that is a standard deviation of $10{,}000 \times 0.00866 = 86.6$ people.

A trusted curator would instead release the true count plus Laplace noise $\mathrm{Lap}(1/\varepsilon)$. The **Laplace mechanism** adds noise of scale $b = \Delta_1/\varepsilon$, where the sensitivity Δ_1 is the most one person can move the answer; under replace, changing one bit moves the count by at most 1, so $b = 1/\varepsilon$. A $\mathrm{Lap}(b)$ variable has variance $2b^2$, so the standard deviation is $\sqrt{2}/\varepsilon = 1.4142/1.0986 = 1.287$. The local survey is $86.6/1.287 = 67.3$ times noisier, at the same ε, on the same people.

25.3.6. Check yourself

- A company runs randomized response at $\varepsilon_0 = 1$ on each user's device, and then also stores each user's network address with the report. Is the report still 1-DP? *(As a function of the sensitive bit, yes: Equation 25.3 is about the bit. But the stored address links the report to the person, so anything else learned about them later can be combined with it. The guarantee covers the randomized value, not the metadata.)*
- Why does local DP use the replace relation and not add/remove? *(Each report is sent by a known device. Whether a report exists reveals participation, which no randomizer of the value can hide.)*

25.4. The price of local privacy: a $\sqrt{n}$ gap

25.4.1. Context

The example showed a factor of 67 at $n = 10{,}000$. Is that a weakness of randomized response, or of the local model? This section shows it is the model: no locally private protocol can estimate a

fraction with error below order $1/((e^{\varepsilon_0}-1)\sqrt{n})$, while the central Laplace mechanism reaches order $1/(n\varepsilon)$. The ratio grows like $\sqrt{n}$.

25.4.2. Statement

Randomized response (upper bound). The calculation of the previous example, at a general level ε_0, gives a standard deviation that does not depend on the data, because every report has the same variance $t(1-t)$ whatever its true bit. For every dataset, the unbiased estimate of a fraction has

$$\mathrm{sd}(\hat{\pi}) = \frac{e^{\varepsilon_0/2}}{(e^{\varepsilon_0}-1)\sqrt{n}}. \tag{25.4}$$

Any local protocol (lower bound). The lower bound needs a way to measure how much a set of reports reveals. It uses the **Kullback–Leibler (KL) divergence**, $D_{\mathrm{KL}}(M_0\|M_1) = \sum_y m_0(y)\ln\frac{m_0(y)}{m_1(y)}$: the expected log-likelihood ratio in favour of M_0 when the data come from M_0, which is 0 when the two distributions are equal and grows as they separate. KL is convenient because it adds over independent observations.

The key step is that a local randomizer contracts this information. Let each person's record be drawn independently from a population P_0 or P_1, and let each person apply any ε_0-DP local randomizer (possibly a different one per person). Because the randomizer can change each report's probability by at most a factor e^{ε_0}, the report distributions M_0, M_1 under the two populations can differ only a little, however different the populations are:

$$D_{\mathrm{KL}}(M_0\|M_1) \le (e^{\varepsilon_0}-1)^2\,\mathrm{TV}(P_0,P_1)^2. \tag{25.5}$$

As a consequence, n reports cannot tell apart two populations whose fractions are close. Take $\mathrm{Bern}(\frac{1}{2})$ and $\mathrm{Bern}(\frac{1}{2}+\alpha)$, populations in which a fraction $\frac{1}{2}$ or $\frac{1}{2}+\alpha$ hold a 1. No estimator built from n reports is within $\alpha/2$ of the true fraction with probability $1-\beta$ under both, unless

$$\frac{\alpha}{2} \ge \frac{1-2\beta}{(e^{\varepsilon_0}-1)\sqrt{2n}}. \tag{25.6}$$

25.4.3. Derivation

The upper bound is the randomized-response calculation of the previous example, which used Facts 1 and 2. The lower bound has two halves. Steps 1 to 4 show that one report carries little information about the population (Equation 25.5). Steps 5 to 7 add up n reports and turn "little information" into "large error" with Fact 3. Write $m_j(y) = \sum_x P_j(x)\,p_R(y \mid x)$ for the probability of report y under population j: draw a record from P_j, then randomize it. Assume a finite domain for simplicity.

1. **Pin the report probabilities in a band.** Fix a report y and let $r(y) = \min_x p_R(y \mid x)$ be its smallest probability over all records. By Equation 25.3, no record makes y more than e^{ε_0} times as likely as any other, so every $p_R(y \mid x)$ lies in $[r(y), e^{\varepsilon_0}r(y)]$. Subtracting the floor, $g(x) = p_R(y \mid x) - r(y)$ lies in $[0, (e^{\varepsilon_0}-1)r(y)]$.

2. **Only the difference of the populations matters.** Both populations are probability distributions, so $\sum_x (P_0(x) - P_1(x)) = 1 - 1 = 0$. Subtracting the constant $r(y)$ from $p_R(y \mid x)$ therefore changes nothing: $m_0(y) - m_1(y) = \sum_x (P_0(x) - P_1(x))\, p_R(y \mid x) = \sum_x (P_0(x) - P_1(x))\, g(x)$.
3. **Bound the sum.** Since the differences $P_0(x) - P_1(x)$ add to 0, their positive parts and their negative parts have the same total size, and each total is half the sum of absolute values, which is $\mathrm{TV}(P_0, P_1)$. With g between 0 and $G = (e^{\varepsilon_0} - 1)r(y)$, the sum is largest when $g = G$ on the positive parts and $g = 0$ on the negative ones, giving $\mathrm{TV} \cdot G$, and smallest the other way round, giving $-\mathrm{TV} \cdot G$. Finally, $r(y) \le m_1(y)$, because $m_1(y)$ is an average of the numbers $p_R(y \mid x)$, each at least $r(y)$. So

$$|m_0(y) - m_1(y)| \le (e^{\varepsilon_0} - 1)\, \mathrm{TV}(P_0, P_1)\, m_1(y). \tag{25.7}$$

4. **From the pointwise bound to KL.** The tool is the inequality $\ln u \le u - 1$, which holds because the line $u - 1$ is tangent to the concave curve $\ln u$ at $u = 1$ and a concave curve lies below its tangents. Applied to $u = m_0/m_1$:

$$D_{\mathrm{KL}}(M_0 \| M_1) = \sum_y m_0 \ln \frac{m_0}{m_1} \le \sum_y m_0 \Big(\frac{m_0}{m_1} - 1 \Big) = \sum_y \frac{(m_0 - m_1)^2}{m_1} \le (e^{\varepsilon_0} - 1)^2\, \mathrm{TV}^2 \sum_y m_1. \tag{25.8}$$

   The middle equality holds because both sides equal $\sum_y m_0^2/m_1 - 1$: the left is $\sum m_0^2/m_1 - \sum m_0$, and expanding the square on the right gives $\sum m_0^2/m_1 - 2\sum m_0 + \sum m_1$, with $\sum m_0 = \sum m_1 = 1$. The last inequality squares step 3. The last sum is 1, which is Equation 25.5.
5. **Add over people.** Records and randomness are independent across people, so the reports are independent. For independent observations the log of the joint likelihood ratio is the sum of the individual log ratios, so the KL divergence, its expectation, adds (Divergences). The two Bernoulli populations differ by α in the probability of a 1 and by α in the probability of a 0, so $\mathrm{TV}(\mathrm{Bern}(\frac{1}{2}), \mathrm{Bern}(\frac{1}{2} + \alpha)) = \frac{1}{2}(\alpha + \alpha) = \alpha$. So n reports have KL at most $n(e^{\varepsilon_0} - 1)^2 \alpha^2$.
6. **Pinsker.** Fact 3 needs TV, not KL. **Pinsker's inequality** (Divergences) converts one into the other: $\mathrm{TV} \le \sqrt{\mathrm{KL}/2}$ for any two distributions. TV does not add over independent samples, which is why the derivation goes through KL. Here $\mathrm{TV}(M_0^n, M_1^n) \le \sqrt{\mathrm{KL}/2} \le \alpha(e^{\varepsilon_0} - 1)\sqrt{n/2}$, where M_j^n is the distribution of all n reports.
7. **Fact 3.** An estimator within $\alpha/2$ with probability $1 - \beta$ under both populations forces $\mathrm{TV} \ge 1 - 2\beta$. Combining with step 6, $1 - 2\beta \le \alpha(e^{\varepsilon_0} - 1)\sqrt{n/2}$, so $\alpha \ge (1 - 2\beta)\sqrt{2}/((e^{\varepsilon_0} - 1)\sqrt{n})$. Halving both sides, and using $\sqrt{2}/2 = 1/\sqrt{2}$, gives Equation 25.6.

25.4.4. Interpretation

Randomized response is optimal up to a constant. Divide its standard deviation by the lower bound: the factor $(e^{\varepsilon_0} - 1)\sqrt{n}$ cancels, and what is left is $e^{\varepsilon_0/2}\sqrt{2}/(1 - 2\beta)$. At $\varepsilon_0 = 1$ and $\beta = 0.1$ this is $1.6487 \times 1.4142/0.8 = 2.91$, the same as the ratio $3.03 \times 10^{-2}/1.04 \times 10^{-2}$ of the two values in Table 25.5. No local protocol can do more than a constant factor better.

Local noise is a contraction. Equation 25.5 says a local randomizer shrinks the information about the population by a factor $(e^{\varepsilon_0} - 1)^2 \approx \varepsilon_0^2$ for small ε_0. Since KL adds over people, you need about $1/\varepsilon_0^2$ times as many people to learn the same thing. Centrally, privacy costs an additive

$1/(n\varepsilon)$ in the error of a fraction, which vanishes faster than the sampling error $1/\sqrt{n}$ that any survey has anyway.

The bound is loose by a constant. For one randomized-response report at $\varepsilon_0 = 1$, comparing Bern(0.5) with Bern(0.6), the exact KL is 0.00429, and Equation 25.5 gives $(e-1)^2 \times 0.1^2 = 2.952 \times 0.01 = 0.02952$, about seven times more. The slack comes from step 1: it allows every report probability to sit anywhere in the band $[r(y), e^{\varepsilon_0} r(y)]$, and randomized response does not use all of it. Duchi, Jordan & Wainwright (2018) prove the same contraction with a factor 4 and use it for minimax rates (the best worst-case error any estimator can reach) of many local estimation problems; Chan, Shi & Song (2012) prove the matching $\Omega(\sqrt{n})$ lower bound for counting.

Interaction does not help much. The derivation allowed each person a different randomizer. Lower bounds of the same order also hold for **sequentially interactive** protocols, in which each person still speaks once but their randomizer may be chosen after seeing the earlier reports.

25.4.5. Example: how the gap grows

Figure 25.2 plots the three curves at $\varepsilon = 1$: randomized response, the lower bound, and the central Laplace mechanism, whose error on a fraction is the count's $\sqrt{2}/\varepsilon$ divided by n. At $n = 1{,}000$, randomized response has standard deviation 3.03×10^{-2}, the central Laplace mechanism $\sqrt{2}/1000 = 1.41 \times 10^{-3}$, a ratio of 21. Dividing the two formulas, the ratio is $e^{1/2}\sqrt{n}/(\sqrt{2}(e-1)) = 0.678\sqrt{n}$, so it grows like $\sqrt{n}$: at $n = 100{,}000$ the ratio is 215, and at $n = 10^7$ it is 2146. Each factor of 100 in n multiplies it by $\sqrt{100} = 10$. For a standard deviation of 0.001 at $\varepsilon = 1$, randomized response needs $(e^{1/2}/((e-1) \times 0.001))^2$, that is 920,674 people, and the central Laplace mechanism needs $\sqrt{2}/n \le 0.001$, that is 1,415 (Exercise 3 in the exercise book). This is why local DP is used for the most frequent items of very large populations, and almost never for anything finer.

25.4.6. Check yourself

- In steps 2 and 3, why may g be shifted by $r(y)$, a quantity that depends on y? *(For fixed y it is a constant in x, and $\sum_x (P_0 - P_1)(x) \cdot c = 0$ for any constant c.)*
- The central Laplace mechanism at $\varepsilon = 1$ has fraction error $\sqrt{2}/n$. At what n does the sampling error $\sqrt{\pi(1-\pi)/n}$ of a fraction near $\frac{1}{2}$ exceed it, and what does that say about the cost of central privacy? *(Near $\frac{1}{2}$ the sampling error is $\sqrt{0.25/n} = 0.5/\sqrt{n}$. It exceeds $\sqrt{2}/n$ when $0.5/\sqrt{n} > \sqrt{2}/n$, that is $\sqrt{n} > 2\sqrt{2}$, or $n > 8$. Beyond a handful of people, central privacy costs less than sampling already does.)*

25.5. Frequency estimation: k-ary randomized response and unary encoding

25.5.1. Context

Counting one bit is the simplest task. The workhorse of deployed local DP is **frequency estimation**: each person holds one of k values (a home page, an emoji, a crashing app), and the analyst wants the fraction f_v of people holding each value v. Two natural generalizations of randomized response behave very differently as k grows.

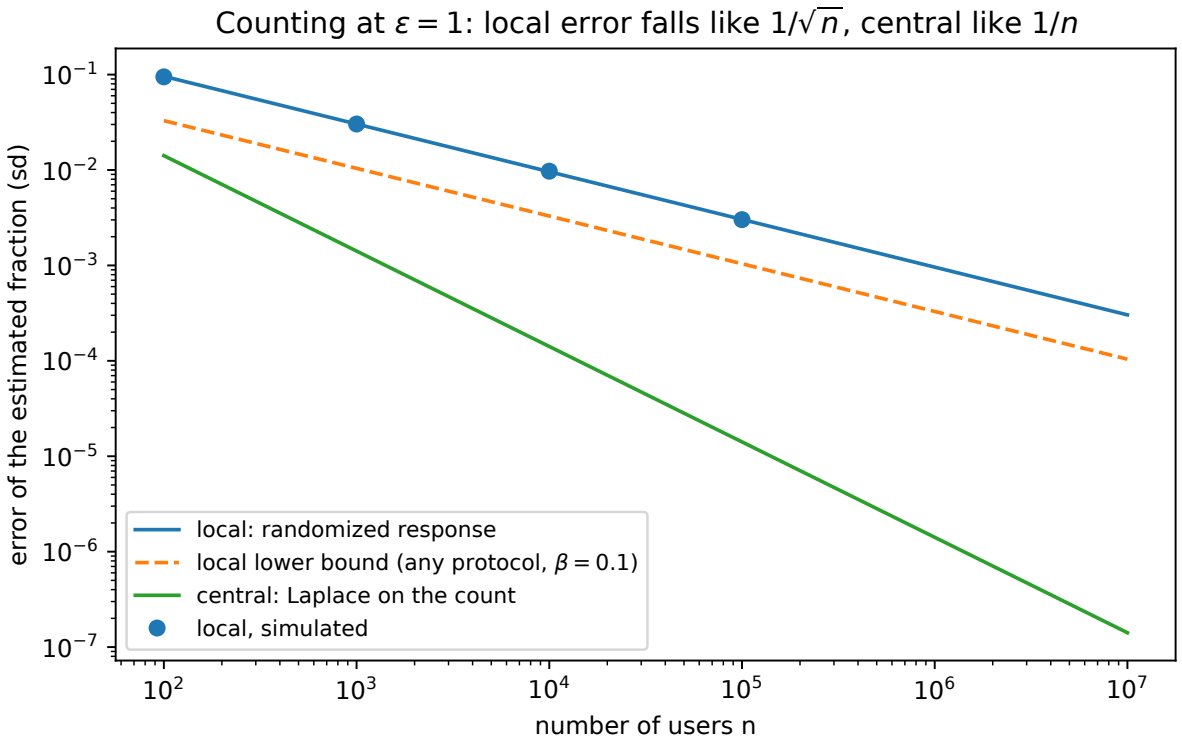


Figure 25.2.: Standard deviation of an estimated fraction at $\varepsilon = 1$, against the number of people. Randomized response (line and simulated dots) falls like $1/\sqrt{n}$ and stays a constant factor above the lower bound for any local protocol; the central Laplace mechanism falls like $1/n$.

25.5.2. Statement

Both schemes fit one template, and analysing the template once gives both. The idea is that each report either "supports" value v or not, and supports it more often when the person really holds v. Then the number of supporting reports is a shrunken version of the true count, and Fact 1 unshrinks it.

A pure frequency oracle. Suppose that for each value v there is a 0/1 "support" event in the report that has probability p when the person holds v and $q < p$ otherwise, independently across people. Let C_v be the number of reports that support v. Then the debiased estimate and its exact variance are

$$\hat{f}_v = \frac{C_v/n - q}{p - q}, \qquad \operatorname{Var} \hat{f}_v = \frac{f_v p(1-p) + (1-f_v)q(1-q)}{n(p-q)^2}. \tag{25.9}$$

k-ary randomized response. Report the true value with probability $p = \frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+k-1}$, and each of the other $k-1$ values with probability $q = \frac{1}{e^{\varepsilon_0}+k-1}$. The two probabilities are chosen so that $p/q = e^{\varepsilon_0}$ and $p + (k-1)q = 1$. The report supports v if it equals v. It is an ε_0-DP local randomizer.

Unary encoding. Encode the value as a one-hot vector of k bits (bit v is 1 and all others are 0), and send each bit independently through its own randomized response: a 1 is sent as 1 with probability p, a 0 is sent as 1 with probability q. The report supports v if bit v is 1. Changing the record changes two bits of the vector, so the privacy level is the product of the two bits' worst ratios:

$$\varepsilon_0 = \ln \frac{p(1-q)}{q(1-p)}. \tag{25.10}$$

Two choices of (p, q) are used. The **symmetric** one sets $p = e^{\varepsilon_0/2}/(e^{\varepsilon_0/2}+1)$ and $q = 1-p$, which is basic one-shot RAPPOR. The **optimized** one sets $p = \frac{1}{2}$ and $q = 1/(e^{\varepsilon_0}+1)$ (Wang et al. 2017).

Variances for a rare value. The regime that decides utility is $f_v \to 0$. Most values of a large domain are rare, and the variance at $f_v = 0$ is the noise floor: a value whose frequency is below a few standard deviations of it cannot be told apart from 0. Setting $f_v = 0$ in Equation 25.9 gives

$$k\text{-ary: } \frac{e^{\varepsilon_0}+k-2}{n(e^{\varepsilon_0}-1)^2}, \qquad \text{symmetric unary: } \frac{e^{\varepsilon_0/2}}{n(e^{\varepsilon_0/2}-1)^2}, \qquad \text{optimized unary: } \frac{4e^{\varepsilon_0}}{n(e^{\varepsilon_0}-1)^2}. \tag{25.11}$$

k-ary randomized response beats optimized unary encoding exactly when $k < 3e^{\varepsilon_0} + 2$.

25.5.3. Derivation

1. **The oracle is unbiased (Fact 1).** C_v is a sum of n independent Bernoulli variables, one per person: nf_v of them, the holders of v, have mean p, and the rest have mean q. So $\mathbb{E}[C_v/n] = f_v p + (1-f_v)q = q + f_v(p-q)$, which is linear in f_v with offset $a = q$ and slope $b = p - q$. Fact 1 gives the estimate in Equation 25.9.
2. **Its variance (Fact 2).** The Bernoulli variances are $p(1-p)$ for the nf_v holders and $q(1-q)$ for the others. Their sum is $n(f_v p(1-p) + (1-f_v)q(1-q))$. Dividing by n^2 (Fact 2, for the mean C_v/n) and by $(p-q)^2$ (Fact 1, for the debiasing) gives Equation 25.9.

3. **k-ary randomized response is ε_0-DP.** Any single report y has probability p (if y is the record) or q (if not) under any record, so the ratio between two records is at most $p/q = e^{\varepsilon_0}$, which is Equation 25.3 for single outcomes.
4. **Unary encoding is private at the stated level.** Changing the record from x to x' changes the one-hot vector in two bits, x and x'. All other bits have the same distribution under both records and cancel in the ratio. Bit x goes from "a 1" to "a 0", so its contribution to the ratio is p/q if it is sent as 1 and $(1-p)/(1-q)$ if sent as 0. Bit x' goes the other way: q/p or $(1-q)/(1-p)$. Since $p > q$, the largest product is $\frac{p}{q} \cdot \frac{1-q}{1-p}$, when bit x is sent as 1 and bit x' as 0. Plugging in the two choices gives exactly e^{ε_0}. For the symmetric one, $q = 1-p$ makes both factors equal to $p/(1-p) = e^{\varepsilon_0/2}$, and their product is e^{ε_0}. For the optimized one, $p = \frac{1}{2}$ makes $p/(1-p) = 1$, and $(1-q)/q = e^{\varepsilon_0}$. The script checks both at $\varepsilon_0 = 1$.
5. **Rare-value variances.** At $f_v = 0$, Equation 25.9 is $q(1-q)/(n(p-q)^2)$. For k-ary: $q(1-q) = \frac{e^{\varepsilon_0}+k-2}{(e^{\varepsilon_0}+k-1)^2}$ and $p-q = \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+k-1}$; the factors $(e^{\varepsilon_0}+k-1)^2$ cancel. For optimized unary: $q(1-q) = \frac{e^{\varepsilon_0}}{(e^{\varepsilon_0}+1)^2}$ and $p-q = \frac{1}{2} - \frac{1}{e^{\varepsilon_0}+1} = \frac{e^{\varepsilon_0}-1}{2(e^{\varepsilon_0}+1)}$; the factors $(e^{\varepsilon_0}+1)^2$ cancel and the 2 squared leaves the 4. The symmetric case is binary randomized response at level $\varepsilon_0/2$, whose rare-value variance is the $k = 2$ case of the k-ary formula with $\varepsilon_0/2$ in place of ε_0.
6. **Crossover.** The k-ary and optimized unary variances have the same denominator, so the first is smaller exactly when $e^{\varepsilon_0} + k - 2 < 4e^{\varepsilon_0}$, that is when $k < 3e^{\varepsilon_0} + 2$.

25.5.4. Interpretation

k-ary randomized response spreads its noise over the domain. With many values the true one is reported with probability $\approx e^{\varepsilon_0}/k$, so the report is almost uniform and carries little signal. Its rare-value variance has k in the numerator, so it grows linearly in k (Figure 25.3).

Unary encoding avoids this by paying ε_0 for two bits, once, whatever k is. Its variance does not depend on k, at the price of a k-bit report. Optimized unary encoding spends the budget asymmetrically: it keeps a 1 only half the time but almost never invents one. That helps because almost all bits of a one-hot vector are 0, so the noise that matters most is the false 1s, and those are the ones it suppresses. For large k, hashing schemes (local hashing, Hadamard response) reach the same variance with reports of $O(\log k)$ bits.

RAPPOR shows what a deployment adds. Erlingsson, Pihur & Korolova (2014) deployed a unary encoding over a Bloom filter (a hashed bit vector) in Chrome. It adds a **permanent** randomized response, memoized per value, on top of a fresh one per report. The reason is composition. By basic composition (Post-processing, group privacy and composition), releasing several outputs about the same record, each ε_0-DP, costs the sum of their levels, because the privacy losses of independent releases add. Without memoization, a user who reports the same value every day would pay ε_0 per day, and averaging the reports would reveal the value. With it, the daily reports are fresh noise around a fixed randomized value, and averaging can recover at most that randomized value.

The variance still hides a $\sqrt{n}$. Every row of Table 25.3 must be divided by n, and the square root is again of order $1/\sqrt{n}$: these oracles are optimal local protocols, still with the gap of the previous section.

Table 25.3.: n times the variance of the estimated frequency of a rare value, at $\varepsilon_0 = 1$. k-ary randomized response wins below $k = 3e + 2 = 10.15$.

k	k-ary randomized response	Symmetric unary	Optimized unary
2	0.921	3.918	3.683
8	2.953	3.918	3.683
32	11.082	3.918	3.683
128	43.596	3.918	3.683

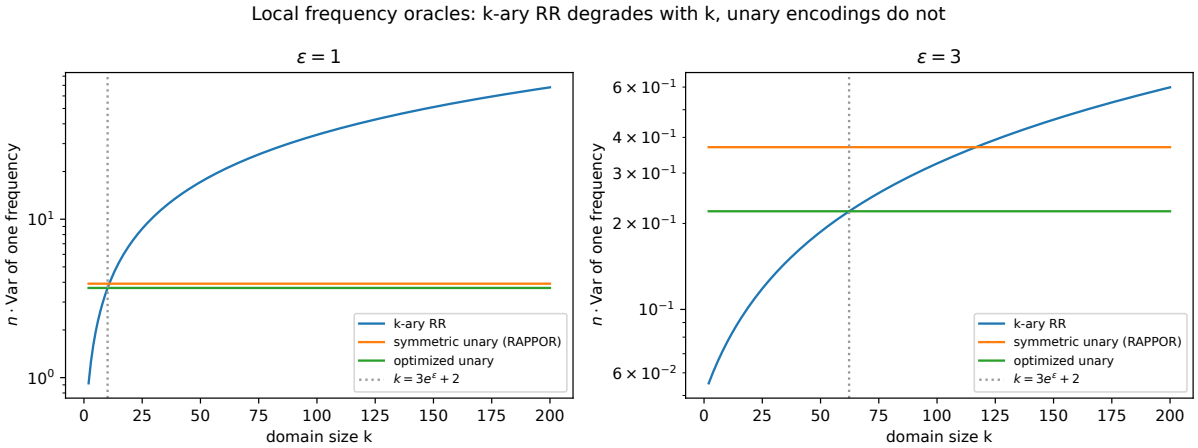


Figure 25.3.: n times the variance of a rare value's estimated frequency, against the domain size k, at $\varepsilon_0 = 1$ and $\varepsilon_0 = 3$. k-ary randomized response grows linearly in k; the unary encodings are flat; the dotted line is the crossover $k = 3e^{\varepsilon_0} + 2$.

25.5.5. Example: an emoji survey

A keyboard app wants the frequency of each of $k = 128$ emoji at $\varepsilon_0 = 1$ from $n = 10^6$ users. The standard deviation of a rare emoji's estimate is the square root of the table entry divided by n. k-ary randomized response gives $\sqrt{43.596/10^6} = 0.0066$ per emoji; optimized unary encoding gives $\sqrt{3.683/10^6} = 0.0019$. An estimate has to stand about three standard deviations above 0 to be told apart from noise, and $3 \times 0.0019 \approx 0.006$, so emoji used by fewer than about 0.6% of users are lost in the noise either way. With $k = 8$ choices instead, below the crossover, k-ary randomized response is better: $\sqrt{2.953/10^6} = 0.0017$.

25.5.6. Check yourself

- What is k-ary randomized response at $k = 2$, and why does Table 25.3 give 0.921 for it? *(It is binary randomized response. The rare-value formula becomes $e^{\varepsilon_0}/(e^{\varepsilon_0} - 1)^2$, the randomized-response factor, which is $e/(e-1)^2 = 0.921$ at $\varepsilon_0 = 1$.)*

- In unary encoding, why does flipping the two bits of x and x' not cost $2\varepsilon_0$, given that each bit is a randomized response? *(The level ε_0 is defined as the log of the product of the two worst ratios, so it already accounts for both bits. The symmetric scheme gives each bit $\varepsilon_0/2$.)*

25.6. The shuffle model and amplification by shuffling

25.6.1. Context

The local model loses a factor $\sqrt{n}$ because each report must hide its owner from an analyst who knows whose report it is. Remove that knowledge, and the reports hide each other. In the **shuffle model** (Bittau et al. 2017, the encode–shuffle–analyze architecture of Prochlo; Cheu et al. 2019), each person sends a locally randomized report to a **shuffler**, which strips identifiers and timing and forwards a uniformly random permutation of the reports. The analyst sees the multiset: which reports were sent, but not by whom. The shuffler can be a separate organization, a mix network, or a trusted execution environment. It must be trusted only to unlink, not to keep data secret: it sees reports, which are already ε_0-DP.

25.6.2. Statement

The claim is that anonymity amplifies privacy: the more people's reports the target's report is mixed with, the less the multiset reveals about the target, so the central ε falls as n grows. Erlingsson et al. (2019) showed that shuffling ε_0-DP reports gives central (ε, δ)-DP with $\varepsilon = O(\varepsilon_0 \sqrt{\ln(1/\delta)/n})$ for $\varepsilon_0 < 1$. The bound to compute with is Theorem 3.1 of Feldman, McMillan & Talwar (2021), quoted here with the range spaces of the randomizers left out. In it, $[n] = \{1, \dots, n\}$ and $z_{1:i-1}$ is the list of the first $i-1$ outputs:

> For any domain $\mathcal{D}$, let $\mathcal{R}^{(i)}$ for $i \in [n]$ be a sequence of algorithms such that $\mathcal{R}^{(i)}(z_{1:i-1}, \cdot)$ is an ε_0-DP local randomizer for all values of auxiliary inputs $z_{1:i-1}$. Let $\mathcal{A}_s$ be the algorithm that, given a dataset $x_{1:n}$, samples a uniform random permutation π over $[n]$, then sequentially computes $z_i = \mathcal{R}^{(i)}(z_{1:i-1}, x_{\pi(i)})$ for $i \in [n]$ and outputs $z_{1:n}$. Then for any $\delta \in [0, 1]$ such that $\varepsilon_0 \le \log\left(\frac{n}{16\log(2/\delta)}\right)$, $\mathcal{A}_s$ is (ε, δ)-DP, where
>
> $$\varepsilon \le \ln\left(1 + \frac{e^{\varepsilon_0} - 1}{e^{\varepsilon_0} + 1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(4/\delta)}}{\sqrt{n}} + \frac{8e^{\varepsilon_0}}{n}\right)\right). \tag{25.12}$$

The theorem is stated for replace neighbours, with log the natural logarithm. Shuffling the outputs of a fixed randomizer is the special case in which every $\mathcal{R}^{(i)}$ ignores its auxiliary input. Read the formula from the inside out: the bracket falls like $1/\sqrt{n}$, it is multiplied by a factor below 1 that depends only on ε_0, and $\ln(1+u) \approx u$ when u is small, so for large n the central ε is about that product.

25.6.3. Why it works, in outline

The full derivation is in Amplification by shuffling and by iteration. Its first step is elementary and carries the intuition: every other person's report has some chance of looking exactly like a report the target might have sent, and after shuffling those look-alikes hide the target's report.

Fix the two values x^0, x^1 that the target person might hold. For any other person's value x and any report y, Equation 25.3 says y is at most e^{ε_0} times less likely under x than under any other value, so $p_R(y \mid x) \geq e^{-\varepsilon_0} p_R(y \mid x^0)$ and $p_R(y \mid x) \geq e^{-\varepsilon_0} p_R(y \mid x^1)$. A number at least as large as two others is at least as large as their average, so $p_R(y \mid x) \geq \frac{e^{-\varepsilon_0}}{2}(p_R(y \mid x^0) + p_R(y \mid x^1))$. That lets you split the report distribution of x into three pieces:

$$R(x) = \frac{e^{-\varepsilon_0}}{2} R(x^0) + \frac{e^{-\varepsilon_0}}{2} R(x^1) + (1 - e^{-\varepsilon_0})\,\mathrm{LO}(x), \tag{25.13}$$

where the leftover distribution $\mathrm{LO}(x)$ is whatever remains after removing the first two pieces, rescaled to total mass 1. It is non-negative by the inequality, and its weight $1 - e^{-\varepsilon_0}$ is what the first two pieces leave of the total mass 1.

Read the split as a recipe for drawing the report: with probability $e^{-\varepsilon_0}/2$ report as if holding x^0, with the same probability as if holding x^1, and otherwise draw from the leftover. So every other person, with probability $e^{-\varepsilon_0}$, sends a report that looks exactly like a report of x^0 or of x^1: a **clone** of the target. The number of clones is a binomial count with mean about $(n-1)e^{-\varepsilon_0}$, split evenly between the two hypotheses at random. After shuffling, the analyst sees the target's report among those clones and cannot tell which one is the target's.

To turn this into (ε, δ), the proof uses the **hockey-stick divergence** of The definition of differential privacy, $E_\gamma(P\|Q) = \sum_y \max\left(P(y) - \gamma Q(y), 0\right)$, the total by which P exceeds γ times Q. A mechanism is (ε, δ)-DP exactly when $E_{e^\varepsilon} \leq \delta$ for the output distributions of every pair of neighbours, because the worst event S in the definition is the set of outputs where $P > e^\varepsilon Q$. The rest of the proof bounds the hockey-stick divergence of the resulting clone counts with binomial concentration (Concentration inequalities): a binomial count rarely strays from its mean by more than a few standard deviations, so the even split between x^0 and x^1 clones is rarely lopsided enough to expose the target. It gains the extra factor $\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}$ from writing the target's own report as a randomized response.

Figure 25.4 does the split for 4-ary randomized response at $\varepsilon_0 = \ln 3$, where $p = 3/(3+3) = \frac{1}{2}$ and $q = 1/6$. A person holding value 3 reports $(\frac{1}{6}, \frac{1}{6}, \frac{1}{2}, \frac{1}{6})$. The clone weight is $e^{-\varepsilon_0}/2 = \frac{1}{6}$, so the clone part is $\frac{1}{6}(R(1) + R(2)) = \frac{1}{6}(\frac{2}{3}, \frac{2}{3}, \frac{1}{3}, \frac{1}{3}) = (0.1111, 0.1111, 0.0556, 0.0556)$. Subtracting it from the person's own distribution leaves $(0.0556, 0.0556, 0.4444, 0.1111)$, of total mass $0.6667 = 1 - e^{-\varepsilon_0}$.

25.6.4. Interpretation

The scaling. The factor in front of the bracket is a hyperbolic tangent: dividing top and bottom by $e^{\varepsilon_0/2}$ gives $\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1} = \frac{e^{\varepsilon_0/2}-e^{-\varepsilon_0/2}}{e^{\varepsilon_0/2}+e^{-\varepsilon_0/2}} = \tanh(\varepsilon_0/2)$. For large n the second term of the bracket, of order $1/n$, is negligible next to the first, and $\ln(1+u) \approx u$, so Equation 25.12 is about $\tanh(\varepsilon_0/2) \cdot 8\sqrt{e^{\varepsilon_0} \ln(4/\delta)/n}$. It falls like $1/\sqrt{n}$ (Figure 25.5). For small ε_0, $\tanh(\varepsilon_0/2) \approx \varepsilon_0/2$ and $e^{\varepsilon_0} \approx 1$, so it is of order $\varepsilon_0\sqrt{\ln(1/\delta)/n}$; for large ε_0, tanh ≈ 1, so it is of order $\sqrt{e^{\varepsilon_0} \ln(1/\delta)/n}$.

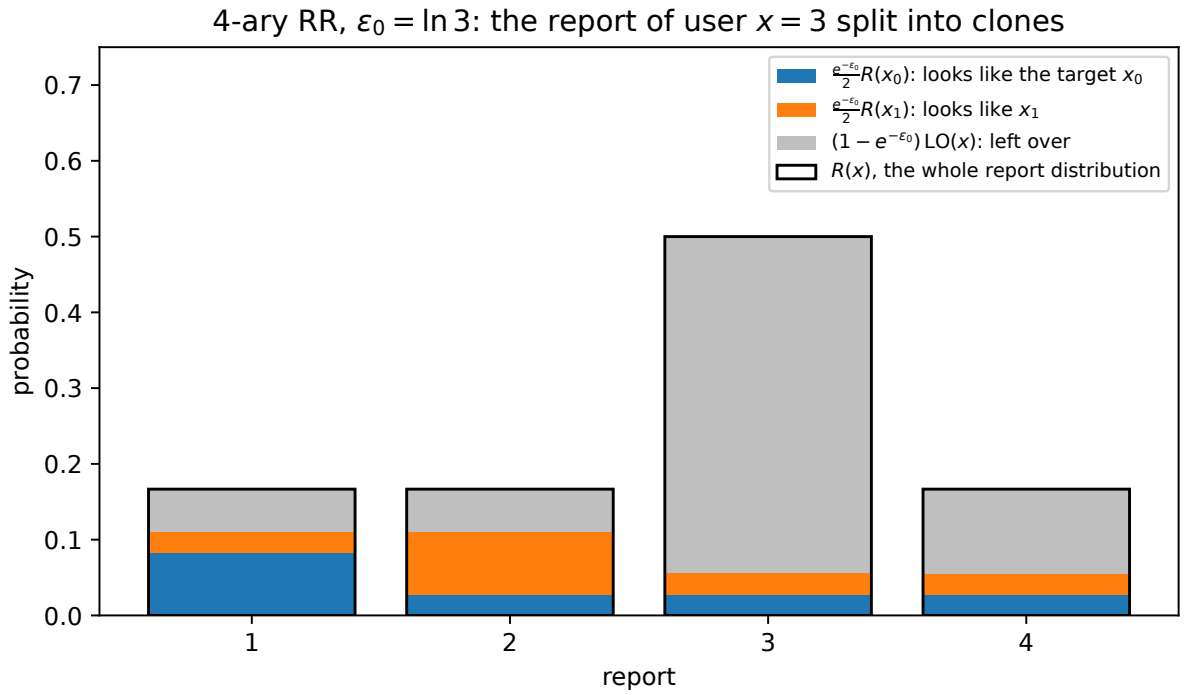


Figure 25.4.: The report distribution of a person holding value 3 under 4-ary randomized response at $\varepsilon_0 = \ln 3$, split into a part that looks like a report of value 1 (blue), a part that looks like value 2 (orange), and a leftover (grey). With probability $e^{-\varepsilon_0} = 1/3$, this person is a clone of a target whose value is 1 or 2.

Large local ε_0 becomes affordable. Invert the large-ε_0 form: a target central ε allows e^{ε_0} of order $\varepsilon^2 n/\ln(1/\delta)$. Randomized response at that level has error $e^{\varepsilon_0/2}/((e^{\varepsilon_0}-1)\sqrt{n}) \approx 1/\sqrt{ne^{\varepsilon_0}}$ on a fraction, since $e^{\varepsilon_0}-1 \approx e^{\varepsilon_0}$ when ε_0 is large. Substituting $e^{\varepsilon_0} \approx \varepsilon^2 n/\ln(1/\delta)$ gives $1/\sqrt{\varepsilon^2 n^2/\ln(1/\delta)} = \sqrt{\ln(1/\delta)}/(\varepsilon n)$: the central rate $1/(\varepsilon n)$, up to $\sqrt{\ln(1/\delta)}$. Multiplied by n, this is the shuffle row of Table 25.2.

The condition matters. $\varepsilon_0 \le \ln(n/(16\ln(2/\delta)))$ is the same as $e^{\varepsilon_0} \le n/(16\ln(2/\delta))$, or $ne^{-\varepsilon_0} \ge 16\ln(2/\delta)$: there must be enough expected clones. Below that, the target may have too few clones to hide among, and the theorem says nothing.

The constants are loose. Table 25.4 compares Equation 25.12 with two computed values at $n = 10^4$ and $\delta = 10^{-6}$. The **numeric** column computes the exact hockey-stick divergence of the pair of clone counts that Theorem 3.2 of the same paper reduces the problem to: the clone argument above turns any ε_0-DP randomizer into a comparison of two distributions of counts, and that comparison can be computed exactly. So the numeric column is still a valid bound for every ε_0-DP randomizer. The **exact** column is the true ε of shuffled binary randomized response on five dataset pairs (the other $n-1$ people hold 0, 2500, 5000, 7499 or 9999 ones), maximized. Since binary randomized response is one of the randomizers the theorem covers, no valid general bound can be below it. Dividing the columns, the closed form is 2.6 to 4.6 times the numeric bound (for example $0.2140/0.0530 = 4.0$ at $\varepsilon_0 = 1$), and the numeric bound is at most 1.9 times the exact value. Use the closed form to reason, and a numerical accountant to deploy.

Table 25.4.: Central ε after shuffling the reports of $n = 10^4$ people, each ε_0-DP, at $\delta = 10^{-6}$. The condition of the theorem requires $\varepsilon_0 \le 3.763$ here.

ε_0	Closed form, Equation 25.12	Numeric (Theorem 3.2)	Exact, shuffled binary randomized response
0.5	0.0939	0.0204	0.0162
1	0.2140	0.0530	0.0357
2	0.5009	0.1550	0.0870
3	0.8241	0.3207	0.1692

25.6.5. Example: choosing the local level

You want central $\varepsilon = 1$ at $\delta = 10^{-6}$. The right-hand side of Equation 25.12 increases with ε_0, so the largest allowed ε_0 is where it equals 1, and bisection finds it: halve an interval of ε_0 values, keeping the half where the bound crosses 1. Each person may randomize at $\varepsilon_0 = 3.504$ when $n = 10^4$, at 5.701 when $n = 10^5$, and at 7.992 when $n = 10^6$. At $\varepsilon_0 = 7.992$, a single report is almost the true value: k-ary randomized response over 8 values keeps the truth with probability $e^{7.992}/(e^{7.992}+7) = 0.998$. Yet the shuffled collection is $(1, 10^{-6})$-DP. Section 25.8 shows what this buys in accuracy.

25.6.6. Trust and its limits

The amplification holds only as long as the shuffler does its one job, and three things can undo it.

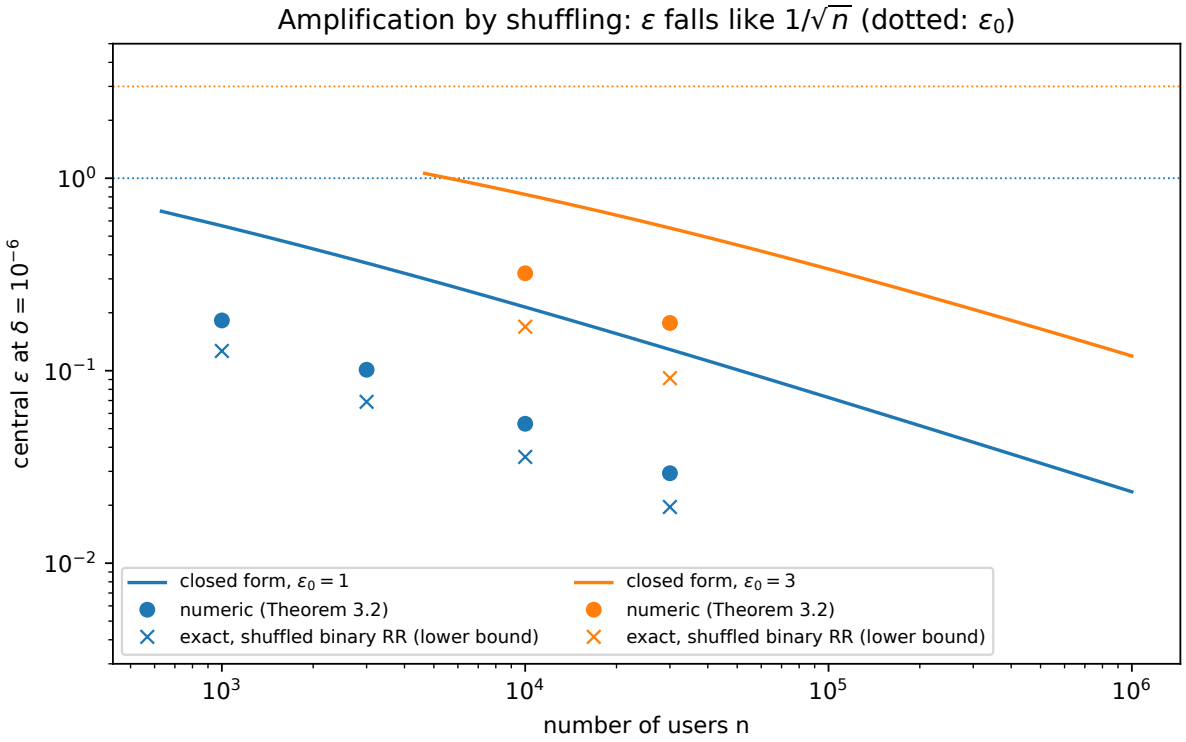


Figure 25.5.: Central ε at $\delta = 10^{-6}$ after shuffling, against n, for $\varepsilon_0 = 1$ and $\varepsilon_0 = 3$: the closed form of Feldman, McMillan & Talwar (lines, drawn where its condition holds), the numeric bound of their Theorem 3.2 (dots) and the exact ε of shuffled binary randomized response (crosses). The dotted lines mark ε_0, the guarantee without shuffling.

The first is collusion. If the shuffler colludes with the analyst, the analyst sees who sent what, and the guarantee falls back to the local ε_0 of each report, which may be large. Shuffling is a defence in depth, not a replacement for a reasonable ε_0.

The second is fake senders. If the adversary controls m of the senders, it knows their reports and can remove them from the multiset. Only the $n - m$ honest reports provide clones, so use $n - m$ in place of n in Equation 25.12 (Exercise 5 in the exercise book).

The third is side channels. The theorem covers only the multiset. Timing, message sizes and the number of messages per person must not reveal anything either, which is why real shufflers batch and pad.

25.6.7. Check yourself

- Why does the theorem allow each randomizer to depend on the earlier outputs $z_{1:i-1}$? *(So that adaptive protocols, such as a server that updates the query after each batch, are covered. The clone argument needs only that each step is ε_0-DP in its record, whatever the auxiliary input.)*
- At $n = 10^4$ and $\delta = 10^{-6}$, the theorem's condition allows $\varepsilon_0 \leq 3.763$. How many clones does the target expect at that limit? *(At the limit the condition holds with equality, so $ne^{-\varepsilon_0} = 16 \ln(2/\delta) = 16 \times 14.5 = 232$, since $\ln(2 \times 10^6) = 14.5$.)*

25.7. Distributed DP: splitting the noise

25.7.1. Context

The shuffle model still adds noise to every report. The **distributed model** adds the central amount of noise once, in pieces. Each client adds a small share of noise to its own value, and a **secure aggregation** protocol reveals only the sum of the noisy values to the server. In federated settings this is how a server can learn a noisy model update without seeing any client's update.

Treat secure aggregation as a black box here. It is a cryptographic protocol built from the additive secret sharing of Cryptography basics: each value is split into random shares that add up to it, so that any incomplete set of shares looks uniformly random and reveals nothing. The property used below is **simulation**: everything the server sees during the protocol could be generated from the sum alone, plus randomness of its own, so it learns nothing beyond the sum. Practical versions (Bonawitz et al. 2017) use pairwise masks that cancel in the sum and tolerate clients that drop out.

25.7.2. Statement

The idea is that independent Gaussian noises add up to a Gaussian whose variance is the sum of theirs. So n small Gaussians of variance σ^2/n add up to exactly the one Gaussian of variance σ^2 that the central mechanism would have added.

Recall the **Gaussian mechanism**: release $f(D) + \mathcal{N}(0, \sigma^2 I)$, where the ℓ_2 **sensitivity** Δ_2 is the largest Euclidean distance $\|f(D) - f(D')\|_2$ between neighbours. Its privacy depends only on the

ratio Δ_2/σ, and a larger σ gives a smaller ε at each δ. Let n clients hold vectors x_i with a sum $f(D) = \sum_i x_i$ of ℓ_2 sensitivity Δ_2. Each client sends $x_i + Z_i$ with $Z_i \sim \mathcal{N}(0, \frac{\sigma^2}{n} I)$ independent, through secure aggregation. Then:

1. The server's view is (ε, δ)-DP whenever the Gaussian mechanism with noise σ on f is.
2. If an adversary knows the noise of t clients (they collude with it, or they dropped out after their noise was planned for), the guarantee is that of the Gaussian mechanism with noise $\sigma\sqrt{(n-t)/n}$.

25.7.3. Derivation

1. **The noise adds up to the central noise.** Independent normals add their means and their variances, and their sum is again normal: $\sum_i Z_i \sim \mathcal{N}(0, n \cdot \frac{\sigma^2}{n} I) = \mathcal{N}(0, \sigma^2 I)$. So the sum of the noisy values, $f(D) + \sum_i Z_i$, is exactly the output of the central Gaussian mechanism on $f(D)$.
2. **The protocol reveals only the sum.** By the simulation property, the server's view is a function (a simulator) of the sum and independent randomness. A function of a DP output is DP with the same parameters (post-processing, Post-processing, group privacy and composition), so the view is as private as the sum.
3. **Colluders subtract what they know.** t colluding clients know their own x_i and Z_i and can remove them from the sum. What remains is $\sum_{\text{honest}} x_i$ plus $\sum_{\text{honest}} Z_i$, a sum of $n - t$ normals of variance σ^2/n each, which by step 1 is $\mathcal{N}(0, \frac{n-t}{n}\sigma^2 I)$. That is a Gaussian mechanism with the smaller noise $\sigma\sqrt{(n-t)/n}$. The privacy of a Gaussian at a given σ is the exact (ε, δ) curve of The Laplace and Gaussian mechanisms.

25.7.4. From real noise to integers: Skellam and the distributed discrete Gaussian

Secure aggregation adds integers modulo q, because only in a finite set like that can shares be uniformly random. Adding a fixed number modulo q is a bijection, a one-to-one shuffle of the set, and a bijection maps the uniform distribution to itself (Cryptography basics); there is no uniform distribution on the real numbers or on all integers for this to work with. Real-valued Gaussian noise cannot be sent. So the clients scale and round their vectors to integers, add integer noise, and the modulus is chosen large enough that the true sum never wraps around.

The integer noise must keep the property that made step 1 work: the sum of the shares must have a known distribution with the central privacy. Two answers are used.

The first is **Skellam noise** (Agarwal, Kairouz & Liu 2021). $\mathrm{Sk}(\mu)$ is the difference $N_1 - N_2$ of two independent Poisson(μ) variables. Each Poisson has mean and variance μ, so the difference has mean 0 and variance $\mu + \mu = 2\mu$. Sums of independent Poissons are Poisson with the summed mean. The reason is the **moment generating function** (MGF), $\mathbb{E}[e^{\lambda N}]$, which determines a distribution: for Poisson(μ) it is $e^{\mu(e^\lambda - 1)}$, the MGF of a sum of independent variables is the product of their MGFs, and multiplying these exponentials adds the μs. So if each of n clients adds $\mathrm{Sk}(\mu/n) = N_{1i} - N_{2i}$, the total is $\sum_i N_{1i} - \sum_i N_{2i}$, a difference of two independent Poisson(μ) variables, which is exactly $\mathrm{Sk}(\mu)$. Agarwal, Kairouz & Liu show that its privacy is close to that of a Gaussian with the same variance 2μ, with a correction that vanishes as μ grows. This section uses the Gaussian calibration as that approximation and does not derive the correction.

The second is **the distributed discrete Gaussian** (Kairouz, Liu & Steinke 2021). A discrete Gaussian puts probability proportional to $e^{-z^2/(2\sigma^2)}$ on each integer z, and each client adds one. Unlike Skellam, the sum of discrete Gaussians is not a discrete Gaussian, but it is close to one when each share's variance is not too small, and the paper bounds the gap.

25.7.5. Interpretation

Distributed noise gives central accuracy, with a δ. The distributed sum has the error of the central Gaussian mechanism. Against the central Laplace mechanism on a low-dimensional histogram it still loses a constant factor (see Section 25.8), because the Gaussian has to pay for its δ with a wider spread. Its advantage is in high dimension. The Laplace mechanism is calibrated to the ℓ_1 sensitivity, the sum of absolute changes, and the Gaussian to the ℓ_2 sensitivity. For a d-dimensional change spread evenly over all coordinates, the ℓ_1 norm is $\sqrt{d}$ times the ℓ_2 norm, so in the worst case the ℓ_2 sensitivity of a d-dimensional model update is $\sqrt{d}$ times smaller.

The guarantee depends on how many are honest. Figure 25.6 shows how fast it erodes. Deployments over-provision the noise: if at most a fraction ρ of clients may collude or drop out, each adds $\frac{\sigma^2}{n(1-\rho)}$. Then the $n(1-\rho)$ honest clients alone contribute $n(1-\rho)\cdot\frac{\sigma^2}{n(1-\rho)}=\sigma^2$, enough for the target, and when everyone is honest the server gets $\frac{1}{1-\rho}$ times the variance it needs.

Malicious clients are a different problem. A client can send any vector it likes, including one without noise or with an enormous value. Secure aggregation hides the inputs but does not check them; bounding them needs zero-knowledge proofs or verifiable aggregation, outside this chapter.

25.7.6. Example: 100 clients

A server aggregates a histogram from users through 100 clients and wants $(\varepsilon,\delta)=(1,10^{-6})$. Under replace, a user who changes value moves one count down by 1 and another up by 1, so $\Delta_2=\sqrt{1^2+1^2}=\sqrt{2}$. The **analytic Gaussian calibration** of The Laplace and Gaussian mechanisms computes the exact δ of the Gaussian mechanism at a given σ and picks the smallest σ that meets the target. It gives total noise $\sigma=5.975$, so each client adds noise of standard deviation $5.975/\sqrt{100}=0.597$, because 100 variances of $\sigma^2/100$ add to σ^2. If 20% of the clients collude with the server, the remaining noise is $5.975\sqrt{0.8}$ and gives $\varepsilon=1.128$; at 50%, $\varepsilon=1.455$; if only 10% are honest, $\varepsilon=3.525$. The script also checks the two pieces of the pipeline: 100 Skellam shares with total variance 25 sum to a variable with empirical mean 0.004 and variance 25.15, and the two-server additive sharing of the vectors $(3,-2)$, $(5,7)$, $(0,1)$ modulo 2^{20} reveals the sum $(8,6)$.

25.7.7. Check yourself

- Why can't each client simply add $\mathcal{N}(0,\sigma^2)$, the full central noise? *(It would work for privacy, but the sum would carry $n\sigma^2$ of variance, n times too much: that is the local model again, with its $\sqrt{n}$ loss.)*

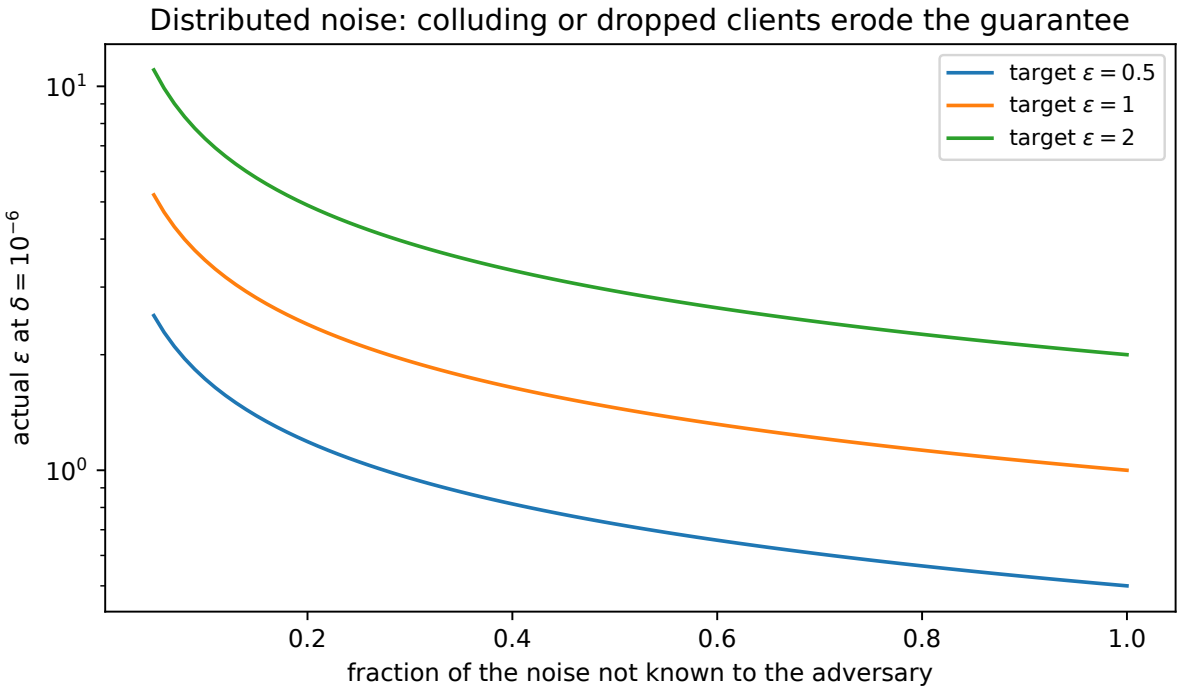


Figure 25.6.: The actual ε at $\delta = 10^{-6}$ of distributed Gaussian noise calibrated for a target ε, against the fraction of the total noise that the adversary does not know. At fraction 1 every client is honest and the target is met; below it, colluding or dropped clients leave less noise than planned.

- Is the sum of n independent Lap(b/n) variables a Lap(b) variable? *(No. Each has variance $2(b/n)^2$, so the sum has variance $n \cdot 2b^2/n^2 = 2b^2/n$, not $2b^2$, and by the central limit theorem it is close to normal for large n. Splitting Laplace noise needs a different decomposition, as a difference of Gamma variables (Exercise 6 in the exercise book).)*

25.8. Code: frequency estimation in four trust models

The script code/trust_models.py computes every number in this chapter and draws its figures. Each estimator, bound and accountant is a small typed function. The central comparison estimates a histogram over $k = 8$ values with Zipf frequencies ($f_v \propto 1/(v+1)$) at the same central guarantee $(\varepsilon, \delta) = (1, 10^{-6})$ under replace in all four models. The four set-ups are:

- **Central:** the Laplace mechanism on the counts, Lap$(2/\varepsilon)$ per count (the histogram has ℓ_1 sensitivity 2 under replace, since one count goes down by 1 and another up by 1). It is pure ε-DP, which is stronger than required.
- **Distributed:** one-hot vectors plus Skellam shares through secure aggregation, with total variance from the Gaussian calibration at $\Delta_2 = \sqrt{2}$. The simulation draws the aggregated Skellam noise directly, which Poisson additivity makes exact.
- **Shuffle:** the better local oracle at the largest ε_0 that Equation 25.12 allows, capped by its condition.
- **Local:** the better local oracle at $\varepsilon_0 = \varepsilon = 1$.

The error is the expected squared ℓ_2 distance between the estimated and the true histogram of fractions, computed exactly from Equation 25.9 and the noise variances, and checked against 200 simulated runs. For instance, the central Laplace noise has variance $2 \times 2^2 = 8$ per count, so 8 counts give 64 on the counts and $64/n^2$ on the fractions, which is 6.40×10^{-5} at $n = 10^3$. The distributed noise has variance $5.975^2 = 35.7$ per count, and $8 \times 35.7/10^6 = 2.86 \times 10^{-4}$. From the repository root:

```
uv run python code/part03-dp-foundations/trust_models.py
```

Table 25.5 shows the output (seed 0).

Table 25.5.: Output of the chapter's script (seed 0). The matched rows give the expected squared ℓ_2 error of the histogram; the script also prints the simulated values, which agree within the Monte Carlo error of 200 runs.

Experiment	Result
Randomized response, $\varepsilon = \ln 3$, $n = 10^4$, 3,000 ones	sd 0.00866 (simulated: mean 0.3000, sd 0.00861). Count sd: local 86.6, central 1.287, ratio 67.3
Fraction sd at $\varepsilon = 1$	$n = 10^3$: local 3.03×10^{-2}, central 1.41×10^{-3}, ratio 21, lower bound 1.04×10^{-2}. $n = 10^5$: ratio 215. $n = 10^7$: ratio 2146
One report, Bern(0.5) against Bern(0.6), $\varepsilon_0 = 1$	KL 0.00429; bound Equation 25.5 0.02952

Experiment	Result
Frequency oracles, $\varepsilon_0 = 1$	See Table 25.3; both unary encodings have $\varepsilon_0 = 1.0000$
Clones, 4-ary, $\varepsilon_0 = \ln 3$	Leftover mass 0.6667
Shuffling, $n = 10^4$, $\delta = 10^{-6}$	See Table 25.4. Largest ε_0 for $\varepsilon = 1$: 1.460 ($n = 10^3$, the condition's cap), 3.504 (10^4), 5.701 (10^5), 7.992 (10^6)
Distributed Gaussian, $(1, 10^{-6})$, 100 clients	$\sigma = 5.975$ total, 0.597 per client. ε with 80%, 50%, 10% honest: 1.128, 1.455, 3.525
Matched $(1, 10^{-6})$, $k = 8$, $n = 10^3$	Central 6.40×10^{-5}; distributed 2.86×10^{-4}; shuffle 9.35×10^{-3}; local 2.71×10^{-2}
Matched, $n = 10^4$	Central 6.40×10^{-7}; distributed 2.86×10^{-6}; shuffle 4.88×10^{-5}; local 2.71×10^{-3}
Matched, $n = 10^5$	Central 6.40×10^{-9}; distributed 2.86×10^{-8}; shuffle 4.76×10^{-7}; local 2.71×10^{-4}

tests/test_trust_models.py checks that the output still matches.

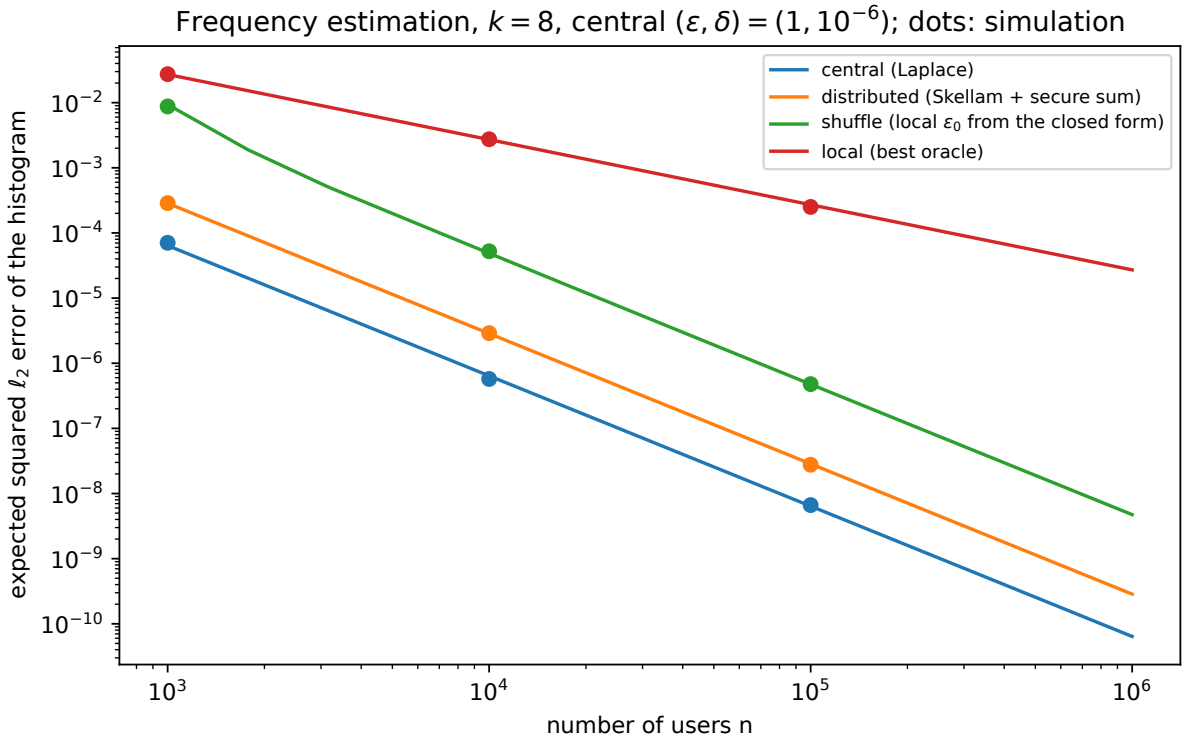


Figure 25.7.: Expected squared ℓ_2 error of an 8-value histogram against the number of people, at the same central $(\varepsilon, \delta) = (1, 10^{-6})$ in all four trust models. Lines are exact; dots are simulations. Central and distributed fall like $1/n^2$, local like $1/n$; shuffle bends from the local to the central slope as n grows.

What to take from it

Local is a different regime. At $n = 10^5$ the local error is $2.71 \times 10^{-4}/6.40 \times 10^{-9} \approx 42{,}000$ times the central error. Squared errors scale like $1/n$ locally and like $1/n^2$ centrally, so the gap grows linearly in n (Figure 25.7).

Shuffling recovers most of it. At $n = 10^4$ the shuffle error is $2.71 \times 10^{-3}/4.88 \times 10^{-5} \approx 56$ times smaller than local, and from $n = 10^4$ on it falls like $1/n^2$, parallel to central, because the allowed ε_0 keeps growing with n. At $n = 10^3$ the theorem's condition caps ε_0 at 1.460, and shuffle helps only by a factor of 3 ($2.71 \times 10^{-2}/9.35 \times 10^{-3}$). Its remaining distance from central is the loose constant of Equation 25.12 and the $\sqrt{\ln(1/\delta)}$; a numerical bound narrows it.

Distributed is central Gaussian. It is $2.86 \times 10^{-4}/6.40 \times 10^{-5} \approx 4.5$ times the central Laplace error for this 8-value histogram, because Gaussian noise for $\delta = 10^{-6}$ is wider than Laplace noise in low dimension. It needs no trusted party, only honest-majority cryptography.

Every model's guarantee is the same (ε, δ), against a different adversary. The local model protects against the collector; the others protect only against parties that do not see what the trusted component sees. Report both.

25.9. References

- Warner, "Randomized Response: A Survey Technique for Eliminating Evasive Answer Bias" (*Journal of the American Statistical Association* 1965): the original local randomizer.
- Kasiviswanathan, Lee, Nissim, Raskhodnikova & Smith, "What Can We Learn Privately?" (FOCS 2008): defines the local model and relates it to statistical-query learning.
- Duchi, Jordan & Wainwright, "Minimax Optimal Procedures for Locally Private Estimation" (*Journal of the American Statistical Association* 2018; conference version 2013): the KL contraction behind the local lower bound, and optimal local estimators.
- Chan, Shi & Song, "Optimal Lower Bound for Differentially Private Multi-Party Aggregation" (2012): the $\Omega(\sqrt{n})$ lower bound for counting without a trusted party.
- Erlingsson, Pihur & Korolova, "RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response" (CCS 2014): the first large deployment, with memoization.
- Wang, Blocki, Li & Jha, "Locally Differentially Private Protocols for Frequency Estimation" (USENIX Security 2017): the frequency-oracle framework, optimized unary encoding and local hashing.
- Bittau et al., "Prochlo: Strong Privacy for Analytics in the Crowd" (2017): the encode–shuffle–analyze architecture.
- Cheu, Smith, Ullman, Zeber & Zhilyaev, "Distributed Differential Privacy via Shuffling" (EUROCRYPT 2019): the shuffle model as a model of computation, and its separations from local and central.
- Erlingsson, Feldman, Mironov, Raghunathan, Talwar & Thakurta, "Amplification by Shuffling: From Local to Central Differential Privacy via Anonymity" (SODA 2019): the first amplification theorem.
- Balle, Bell, Gascón & Nissim, "The Privacy Blanket of the Shuffle Model" (CRYPTO 2019): a different decomposition of the randomizer, and tight bounds for specific protocols.
- Feldman, McMillan & Talwar, "Hiding Among the Clones: A Simple and Nearly Optimal Analysis of Privacy Amplification by Shuffling" (FOCS 2021): the closed form used here, the clone reduction and its numerical version.

- Bonawitz et al., "Practical Secure Aggregation for Privacy-Preserving Machine Learning" (CCS 2017): the secure aggregation protocol used in federated learning.
- Kairouz, Liu & Steinke, "The Distributed Discrete Gaussian Mechanism for Federated Learning with Secure Aggregation" (ICML 2021) and Agarwal, Kairouz & Liu, "The Skellam Mechanism for Differentially Private Federated Learning" (NeurIPS 2021): integer noise that survives secure aggregation.

25.10. Cheat sheet

Table 25.6 collects the results. ε_0 is the local level of one report, and errors are on fractions.

Table 25.6.: Summary of the results of this chapter.

Result	Statement	Note
Local randomizer	$\Pr[R(x) \in S] \le e^{\varepsilon_0} \Pr[R(x') \in S]$ for all x, x'	The transcript is ε_0-DP under replace, against everyone
Randomized response	$\mathrm{sd} = e^{\varepsilon_0/2}/((e^{\varepsilon_0}-1)\sqrt{n})$	Data-independent; $\approx 1/(\varepsilon_0\sqrt{n})$ for small ε_0
KL contraction	$D_{\mathrm{KL}}(M_0 \Vert M_1) \le (e^{\varepsilon_0}-1)^2 \mathrm{TV}(P_0, P_1)^2$	Holds for any local randomizer
Local lower bound	error $\ge (1-2\beta)/((e^{\varepsilon_0}-1)\sqrt{2n})$	Central Laplace: $\sqrt{2}/(n\varepsilon)$; the gap is $\sqrt{n}$
Frequency oracle	$\hat{f}_v = (C_v/n - q)/(p-q)$, variance Equation 25.9	Unbiased; exact variance
k-ary randomized response	$p = e^{\varepsilon_0}/(e^{\varepsilon_0}+k-1)$; rare variance $(e^{\varepsilon_0}+k-2)/(n(e^{\varepsilon_0}-1)^2)$	Best for $k < 3e^{\varepsilon_0}+2$
Optimized unary encoding	$p = \frac{1}{2}$, $q = 1/(e^{\varepsilon_0}+1)$; rare variance $4e^{\varepsilon_0}/(n(e^{\varepsilon_0}-1)^2)$	Independent of k; k-bit reports
Clones	$R(x) = \frac{e^{-\varepsilon_0}}{2}R(x^0) + \frac{e^{-\varepsilon_0}}{2}R(x^1) + (1-e^{-\varepsilon_0})\mathrm{LO}(x)$	The intuition behind amplification
Shuffling (Feldman, McMillan & Talwar 2021)	Equation 25.12, for $\varepsilon_0 \le \ln(n/(16\ln(2/\delta)))$	$\varepsilon \approx 8\tanh(\varepsilon_0/2)\sqrt{e^{\varepsilon_0}\ln(4/\delta)/n}$; 2.6–4.6 times loose at $n = 10^4$
Distributed Gaussian	n shares $\mathcal{N}(0, \sigma^2/n)$ sum to $\mathcal{N}(0, \sigma^2)$	t colluders leave $\sigma\sqrt{(n-t)/n}$
Skellam	n shares $\mathrm{Sk}(\mu/n)$ sum to $\mathrm{Sk}(\mu)$, variance 2μ	Integer noise for secure aggregation

Rules of thumb

- Local DP costs a factor of about $\sqrt{n}$ in error over central DP. Use it only for frequent items in very large populations.

- For frequency estimation, use k-ary randomized response when $k < 3e^{\varepsilon_0} + 2$ and optimized unary encoding (or a hashing variant) otherwise. Memoize reports of values that do not change.
- A shuffler lets each report use a large ε_0, roughly up to $\ln(\varepsilon^2 n / \ln(1/\delta))$. Compute the central ε numerically before deploying; the closed form is several times loose.
- With distributed noise, decide how many clients may collude or drop out, and over-provision the noise for it.
- Always state the trust model next to (ε, δ): the same numbers protect against different adversaries.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

26. Lower bounds: what privacy costs

26.1. Why this matters

The earlier chapters of this part built mechanisms and measured their error. The Laplace mechanism releases a histogram over N bins with error about $\ln N/\varepsilon$ in every bin. The Gaussian mechanism releases d attribute means with error about $\sqrt{d\ln(1/\delta)}/(n\varepsilon)$ each. These are **upper bounds**: they say what one mechanism achieves. They leave one question open. Is the error the price of privacy, or the price of a clumsy mechanism? A cleverer algorithm might do much better.

A **lower bound** answers it. It says that *no* mechanism satisfying the definition can be more accurate than some amount, whatever it does. When the lower bound matches an upper bound, the question is closed. You know the mechanism is optimal, and you know that asking for more accuracy means asking for less privacy or for more data. This chapter proves the three standard lower bounds of differential privacy (DP), and each one matches a mechanism you already know:

- **Packing** (pure ε-DP): an output that must pick one of m candidates costs error growing like $\ln m/\varepsilon$. Laplace histograms are optimal up to a constant.
- **Fingerprinting** (approximate (ε,δ)-DP): releasing d attribute means needs n growing like $\sqrt{d}$. The Gaussian mechanism has this rate.
- **Reconstruction** (any ε): answering about n random counting queries needs error of order $\sqrt{n}$ per answer. This is the Dinur–Nissim attack of Reconstruction attacks, read as a lower bound.

26.1.1. The one idea: accuracy is an attack

How can you prove that *every* mechanism fails, when there are infinitely many mechanisms and you cannot try them all? You do not look at how a mechanism works at all. You only use the two things you know about it, that it is accurate and that it is private, and show that the two cannot hold together. Every proof in this chapter does this in the same three steps.

1. **Accuracy gives an attack.** If a mechanism is accurate, its output tells you something about the data, and an attacker can use that: to pick out which of many datasets was used, to tell whether a person is in the dataset, or to reconstruct secret bits. The more accurate the output, the better the attack works.
2. **DP limits every attack.** The definition says that no test can tell the outputs on two neighbouring datasets apart much better than chance. Group privacy extends that to datasets that differ in k records, with a weaker limit that degrades with k.
3. **Compare.** If the attack built from an accurate mechanism would succeed more often than DP allows, the mechanism cannot be both accurate and private. Writing this as an inequality and solving it for the error, or for the number of records n, gives the lower bound.

So a lower bound on error is the same thing as an attack on accurate mechanisms. The fingerprinting proof makes this literal: its attack is a membership inference attack of the kind in Membership inference, and you will run it.

Table 26.1 lists the results and where each is used.

Table 26.1.: The results of this chapter and where each is used.

Result	Privacy	Where it is used
Packing lemma	Pure	Lower bounds for any output that must single out one of many candidates
Mode and histogram bounds	Pure	Showing Laplace histograms and report-noisy-max are optimal
Sample complexity	Pure and approximate	How many records a private analysis needs; when privacy is cheap
Packing fails under $\delta > 0$	Approximate	Why the Gaussian mechanism can beat every pure-DP mechanism
Fingerprinting lemma and the $\sqrt{d}$ bound	Approximate	Optimality of the Gaussian mechanism for many means; the cost of high-dimensional releases
Tracing attack	Any	Membership inference on released statistics; auditing noisy means
Dinur–Nissim as a lower bound	Any	Why every query answered privately must carry noise of order $\sqrt{n}$

How to read this chapter. The Toolkit has three facts; the packing and fingerprinting sections each use two of them. On a first pass, read the packing section, the statement and interpretation of the fingerprinting section, and the tracing section, which is the fingerprinting proof turned into an experiment. The fingerprinting derivation is the hardest piece of mathematics in the chapter. It is short, but come back to it after the tracing simulation has made the score concrete.

What this chapter assumes. The chapter restates what it uses where it uses it, so the links below are for more detail. The definitions of DP, its hypothesis-testing form and its neighbouring relations are in The definition of differential privacy. Group privacy is in Post-processing, group privacy and composition, and the Laplace and Gaussian mechanisms have their own chapter. Report-noisy-max is in Selection. From Part 1 the chapter uses Hoeffding's inequality (Concentration), the link between the advantage of a test and total variation (TV)

(Hypothesis testing) and Fano's method (Information theory), which is the non-private ancestor of packing.

Conventions. M is a randomized mechanism: it takes a dataset and returns a random output. Two datasets $D \simeq D'$ are **neighbours** when they differ in one record. Recall the definition: M is (ε, δ)-**DP** if for every pair of neighbours and every set S of outputs,

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{26.1}$$

In words, no event becomes more than e^{ε} times more likely when one record changes, up to a small additive slack δ. Because the neighbour relation is symmetric, the inequality also holds with D and D' swapped. **Pure** DP is the case $\delta = 0$, written ε-DP; **approximate** DP allows $\delta > 0$.

What "differ in one record" means depends on the setting. Histograms use **add/remove** neighbours: one dataset has one more record than the other, so one count changes by 1. The marginal, tracing and reconstruction sections use **replace** neighbours on datasets of a fixed size n: one record is swapped for another. They use it because they speak about the number of records n, which add/remove would change.

An accuracy guarantee has two parameters: the **error** α and the **failure probability** β. "M is (α, β)-accurate" means that, on *every* dataset, its error is at most α with probability at least $1 - \beta$. (In Hypothesis testing α and β were the error rates of a test. Here they are not, and tests are described instead by two rates: the **true positive rate** (TPR), how often a test fires when the data are D', the dataset with the record, and the **false positive rate** (FPR), how often it fires when the data are D, the dataset without it. This is the convention of The definition of differential privacy, where a test that says "D'" fires.) The examples use $\varepsilon = 1$, $\beta = 0.05$ and, for approximate DP, $\delta = 10^{-6}$.

26.2. Toolkit: three facts every derivation uses

The definition of DP talks about one event and one pair of neighbours at a time. The proofs below need three consequences of it: what happens when datasets are k records apart (Fact 1), how much the average of a score can move (Fact 2), and how well any test or guess can do (Fact 3). Each fact is stated, proved once, and given a number.

26.2.1. Fact 1 — Group privacy

DP only compares neighbours, but the packing proof needs to compare datasets that are k records apart. The idea is to walk from one to the other one record at a time. Each step lets probabilities grow by a factor e^{ε} and adds δ, so after k steps the factor is $e^{k\varepsilon}$ and the additive terms have piled up. If M is (ε, δ)-DP and D, D'' differ in k records (a chain of k neighbouring steps), then for every set of outputs S

$$\Pr[M(D) \in S] \le e^{k\varepsilon} \Pr[M(D'') \in S] + \delta_k, \qquad \delta_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} \le k\, e^{(k-1)\varepsilon} \delta. \tag{26.2}$$

Why it holds. Write the chain as $D = D_0 \simeq D_1 \simeq \cdots \simeq D_k = D''$ and apply the definition at each link: $\Pr[M(D_j) \in S] \le e^{\varepsilon} \Pr[M(D_{j+1}) \in S] + \delta$. Substitute the second inequality into the first,

the third into the result, and so on. For two steps this gives $\Pr[M(D_0) \in S] \le e^{\varepsilon}(e^{\varepsilon}\Pr[M(D_2) \in S] + \delta) + \delta = e^{2\varepsilon}\Pr[M(D_2) \in S] + \delta(1 + e^{\varepsilon})$. In general, the δ that enters at link j (counting from 0) is multiplied by e^{ε} once for each of the j links before it, which gives the sum $\delta \sum_{j=0}^{k-1} e^{j\varepsilon}$. Each of its k terms is at most the largest one, $e^{(k-1)\varepsilon}\delta$, which gives the simpler bound. Post-processing, group privacy and composition does this in full.

How fast it degrades. With $\varepsilon = 1$ and $\delta = 10^{-6}$, for $k = 5$ steps, $\delta_5 = 10^{-6}(1 + e + e^2 + e^3 + e^4) = 8.6 \times 10^{-5}$, still small. For $k = 20$, $\delta_{20} = 282$, a bound on a probability that is larger than 1 and so says nothing at all. The geometric growth of $e^{j\varepsilon}$ is what makes approximate DP useless for groups of more than about $\ln(1/\delta)/\varepsilon$ records. Under pure DP ($\delta = 0$) there is no additive term and so no such blow-up, only the factor $e^{k\varepsilon}$. The packing section uses this fact.

26.2.2. Fact 2 — DP bounds the expectation of a bounded score

DP bounds the probability of every event. The fingerprinting proof needs something slightly different: a bound on the *average* of a score h computed from the output. The bridge is that the average of a random variable can be written as a sum of probabilities of events, "h exceeds t", over all levels t. DP bounds each of those probabilities, and so it bounds the average. The result is that a score's mean on D can exceed its mean on a neighbour D' only by a multiple $e^{\varepsilon} - 1$ of its typical size on D', plus a small term from δ. Precisely, if M is (ε, δ)-DP, $D \simeq D'$, and h is a function of the output with $|h| \le T$, then

$$\mathbb{E}\, h(M(D)) \le \mathbb{E}\, h(M(D')) + (e^{\varepsilon} - 1)\,\mathbb{E}\,|h(M(D'))| + 2\delta T. \tag{26.3}$$

Derivation.

1. **Write the mean as an integral of tail probabilities.** A variable Y with $0 \le Y \le T$ has $\mathbb{E}\, Y = \int_0^T \Pr[Y > t]\, dt$. To see why, take Y equal to 1, 2 or 3 with probability $\frac{1}{3}$ each, so $\mathbb{E}\, Y = 2$. Then $\Pr[Y > t]$ is 1 for $t < 1$, $\frac{2}{3}$ for $1 \le t < 2$ and $\frac{1}{3}$ for $2 \le t < 3$, and the integral is $1 + \frac{2}{3} + \frac{1}{3} = 2$: each value y is counted once for every level t below it, which adds up to y. A score that can be negative is split into its positive part $h_+ = \max(h, 0)$ and negative part $h_- = \max(-h, 0)$, so $h = h_+ - h_-$ and
$$\mathbb{E}\, h = \int_0^T \Pr[h > t]\, dt - \int_0^T \Pr[h < -t]\, dt. \tag{26.4}$$
2. **Bound the positive part from above.** Apply the definition to the event $\{h(M(D)) > t\}$: $\Pr[h(M(D)) > t] \le e^{\varepsilon}\Pr[h(M(D')) > t] + \delta$.
3. **Bound the negative part from below.** It is subtracted, so you need a lower bound on it. Apply the definition the other way round, with D' on the left, to the event $\{h < -t\}$: $\Pr[h(M(D')) < -t] \le e^{\varepsilon}\Pr[h(M(D)) < -t] + \delta$. Solved for the term on D, this reads $\Pr[h(M(D)) < -t] \ge e^{-\varepsilon}(\Pr[h(M(D')) < -t] - \delta)$.
4. **Integrate over $t \in [0, T]$.** Write $h' = h(M(D'))$. Integrating the bounds of steps 2 and 3 turns the tail probabilities back into means, by step 1, and each δ into δT:
$$\mathbb{E}\, h(M(D)) \le e^{\varepsilon}\,\mathbb{E}\, h'_+ - e^{-\varepsilon}\,\mathbb{E}\, h'_- + (1 + e^{-\varepsilon})\delta T. \tag{26.5}$$

5. **Pull out the mean on D'.** Adding and subtracting $\mathbb{E}\,h'_+$ and $\mathbb{E}\,h'_-$ gives $e^\varepsilon \mathbb{E}h'_+ - e^{-\varepsilon}\mathbb{E}h'_- = \mathbb{E}h' + (e^\varepsilon - 1)\mathbb{E}h'_+ + (1 - e^{-\varepsilon})\mathbb{E}h'_-$, because $\mathbb{E}h' = \mathbb{E}h'_+ - \mathbb{E}h'_-$. The coefficient $1 - e^{-\varepsilon}$ is at most $e^\varepsilon - 1$ (their difference is $e^\varepsilon + e^{-\varepsilon} - 2 \geq 0$), so the two middle terms are at most $(e^\varepsilon - 1)(\mathbb{E}h'_+ + \mathbb{E}h'_-) = (e^\varepsilon - 1)\mathbb{E}|h'|$. Finally $1 + e^{-\varepsilon} \leq 2$.

What it allows. If $h(M(D'))$ has mean 0 and mean absolute value 1, then $h(M(D))$ can have mean at most $e - 1 = 1.72$ at $\varepsilon = 1$ (plus $2\delta T$). A score that is centred on one dataset can move only by a multiple of its typical size on the neighbour. The fingerprinting bound uses this fact, with h the tracing score.

26.2.3. Fact 3 — DP bounds every membership test and every guess of a bit

A test that tries to tell $M(D)$ from $M(D')$ is any rule that looks at the output and "fires" or not. Following The definition of differential privacy, the test fires when it says "D'", the dataset with the record. Its quality is its **advantage**, TPR − FPR: how much more often it fires on $M(D')$ than on $M(D)$. A test that ignores the output fires equally often on both and has advantage 0; a perfect test has advantage 1. DP caps the advantage, because it constrains how often a test can fire on one dataset given how often it fires on the other, and it does so in both directions. If M is (ε, δ)-DP and $D \simeq D'$, every test that tries to tell $M(D)$ from $M(D')$ has

$$\mathrm{TPR} - \mathrm{FPR} \leq \frac{e^\varepsilon - 1 + 2\delta}{e^\varepsilon + 1}. \tag{26.6}$$

The same cap gives a limit on guessing a secret. If one record holds a uniformly random secret bit and D, D' differ only in that bit, every guess of the bit from the output is right with probability at most $(e^\varepsilon + \delta)/(e^\varepsilon + 1)$.

Derivation.

1. **Two constraints from the definition.** Let the test fire with probability b on $M(D')$ (its TPR) and a on $M(D)$ (its FPR). The definition applied to the pair (D', D) and the event "fires" gives $b \leq e^\varepsilon a + \delta$. Applied in the other direction, to the pair (D, D') and the complement "does not fire", it gives $1 - a \leq e^\varepsilon(1 - b) + \delta$.
2. **Each constraint caps the advantage.** The first gives $b - a \leq (e^\varepsilon - 1)a + \delta$, a cap that grows with a. The second, solved for b, gives $b \leq 1 - e^{-\varepsilon}(1 - a - \delta)$, so $b - a \leq (1 - a)(1 - e^{-\varepsilon}) + e^{-\varepsilon}\delta$, a cap that shrinks with a. A test is limited by the smaller of the two caps.
3. **The best test sits where the caps meet.** The smaller of a rising and a falling line is largest at their crossing. Setting $(e^\varepsilon - 1)a + \delta = (1 - a)(1 - e^{-\varepsilon}) + e^{-\varepsilon}\delta$ and multiplying by e^ε gives $(e^\varepsilon - 1)(e^\varepsilon + 1)a = (e^\varepsilon - 1)(1 - \delta)$, so $a = (1 - \delta)/(e^\varepsilon + 1)$. There the first cap is $(e^\varepsilon - 1)(1 - \delta)/(e^\varepsilon + 1) + \delta = (e^\varepsilon - 1 + 2\delta)/(e^\varepsilon + 1)$.
4. **From tests to guesses of a bit.** A guess of the bit is a test: it says "1" on some set A of outputs. With the bit equal to 1 or 0 with probability $\frac{1}{2}$ each, it is right with probability $\frac{1}{2}\Pr[A \mid 1] + \frac{1}{2}(1 - \Pr[A \mid 0]) = \frac{1}{2} + \frac{1}{2}(b - a)$, where b and a are the probabilities of A in the two worlds. Inserting the cap of step 3 gives $\frac{1}{2} + \frac{1}{2} \cdot \frac{e^\varepsilon - 1 + 2\delta}{e^\varepsilon + 1} = \frac{e^\varepsilon + \delta}{e^\varepsilon + 1}$.

What it allows. At $\varepsilon = 1$, $\delta = 0$ the advantage is at most $(e - 1)/(e + 1) = 0.4621$, and a secret bit is guessed correctly at most $e/(e + 1) = 73.11\%$ of the time. The tracing attack and the reconstruction bound use this fact. The definition of differential privacy derives the full trade-off curve, the largest TPR any test can reach at each FPR, of which this bound is the single point where the advantage is largest.

26.3. Packing: pure DP pays the logarithm of the number of candidates

26.3.1. Context

Suppose the output of a mechanism must point to one of m candidates: the most common ZIP code among $N = 10^6$, the bin of a histogram that holds a spike, the corner of a cube that a dataset sits at. For each candidate i you can build a dataset D_i on which a good answer must point to i. If the mechanism is accurate, $M(D_i)$ lands in its own region G_i of outputs almost always.

Now run the mechanism on a single reference dataset D_0 instead. Its output must land somewhere, and its probabilities add up to 1, so on average each of the m regions gets at most a $1/m$ share. But under pure DP, group privacy (Fact 1) says that no event is more than $e^{k\varepsilon}$ times more likely under $M(D_i)$ than under $M(D_0)$, where k is the number of records that separate D_0 from D_i. Region G_i has probability almost 1 under $M(D_i)$, so it must have probability at least about $e^{-k\varepsilon}$ under $M(D_0)$. That holds for *every* region at once. The two requirements clash when m is large: a probability budget of 1 has to be shared among m regions, each of which demands $e^{-k\varepsilon}$. This counting argument is **packing**. It is the private version of Fano's method in Information theory, which shows that an estimator choosing among many hypotheses whose data distributions are hard to tell apart must often choose wrong.

26.3.2. Statement

The lemma turns the budget argument into one inequality. The m demands of size $(1-\beta)e^{-k\varepsilon}$ must fit in a total of 1, so k must be at least the logarithm of the number of candidates, divided by ε.

Packing lemma. Let $D_0, D_1, \dots, D_m$ be datasets with D_0 within k records of every D_i, and let $G_1, \dots, G_m$ be disjoint sets of outputs. If M is ε-DP and $\Pr[M(D_i) \in G_i] \geq 1-\beta$ for every i, then

$$m(1-\beta)e^{-k\varepsilon} \leq 1, \qquad \text{that is,} \qquad k \geq \frac{\ln\left(m(1-\beta)\right)}{\varepsilon}. \tag{26.7}$$

Two consequences follow by choosing the datasets. The **mode** is the most frequent element of the data. To report it reliably, the most frequent element must lead the others by enough records, and the lemma says how many. Any ε-DP mechanism (add/remove) that returns the most frequent element of a universe of size N with probability at least $1-\beta$, whenever that element appears k times and every other element appears 0 times, needs

$$k \geq \frac{\ln\left(N(1-\beta)\right)}{\varepsilon}. \tag{26.8}$$

A **histogram** is the vector of counts of each of N elements, and its ℓ_∞ error is the largest error over the bins. A histogram that is accurate to within α in every bin can be used to find a spike of height more than 2α, so its error inherits the mode's bound, halved. Any ε-DP mechanism (add/remove) that releases a histogram over N bins with ℓ_∞ error at most α with probability at least $1-\beta$ on every dataset has

$$\alpha \geq \frac{1}{2}\left(\frac{\ln\left(N(1-\beta)\right)}{\varepsilon} - 1\right). \tag{26.9}$$

26.3.3. Derivation

Packing lemma.

1. **Each region gets a share of $M(D_0)$.** Apply Fact 1 with $\delta = 0$ to D_i and D_0 and the set G_i. Accuracy says G_i is likely under D_i, and group privacy says it cannot be much less likely under D_0: $1 - \beta \le \Pr[M(D_i) \in G_i] \le e^{k\varepsilon} \Pr[M(D_0) \in G_i]$. So $\Pr[M(D_0) \in G_i] \ge (1-\beta)e^{-k\varepsilon}$.
2. **The shares fit into one distribution.** The G_i are disjoint, so no output is counted twice, and the probabilities add: $\sum_i \Pr[M(D_0) \in G_i] = \Pr[M(D_0) \in \cup_i G_i] \le 1$.
3. **Combine.** Adding the m lower bounds of step 1 and using step 2 gives $m(1-\beta)e^{-k\varepsilon} \le 1$. Multiplying by $e^{k\varepsilon}$ and taking logarithms gives $\ln(m(1-\beta)) \le k\varepsilon$.

Figure 26.1 draws the two sides of step 3. The required share per region falls like $e^{-k\varepsilon}$; the available share is $1/m$. Accuracy is only possible once k is large enough for the required share to fit.

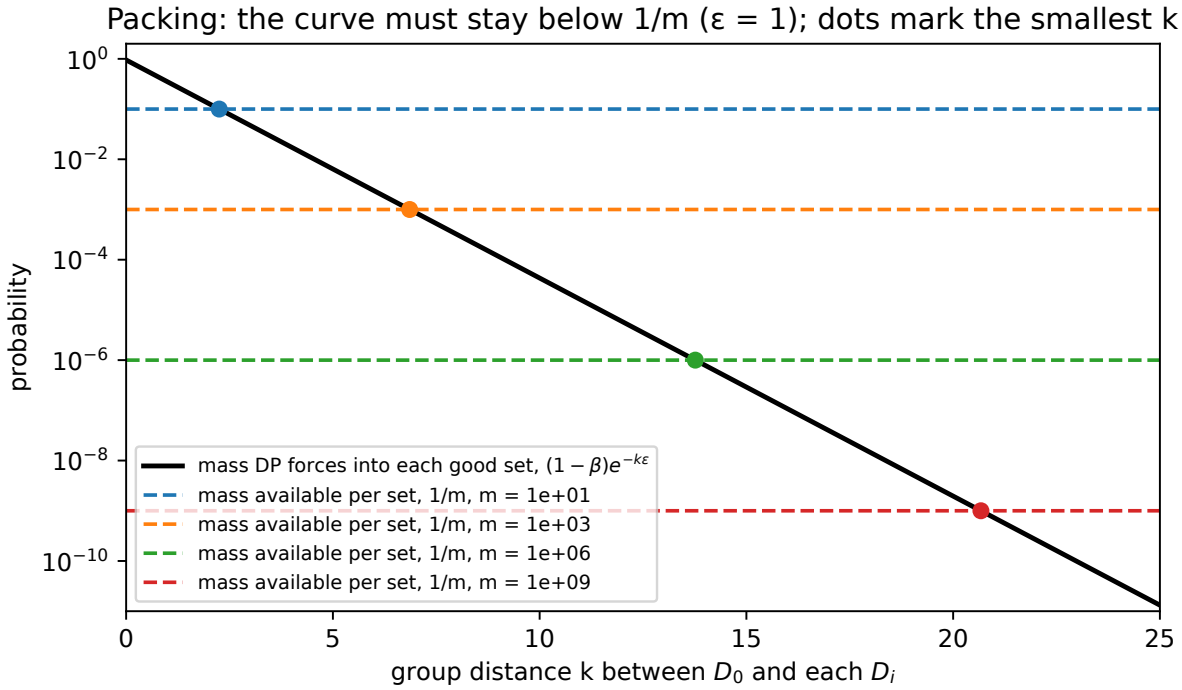


Figure 26.1.: The packing argument at $\varepsilon = 1$, $\beta = 0.05$. The black line is the probability that $M(D_0)$ must put on each good region, $(1-\beta)e^{-k\varepsilon}$. The dashed lines are the average share available when there are m regions. Accuracy at group distance k is impossible to the left of each dot, at $k = \ln(m(1-\beta))/\varepsilon$.

Mode. The task is to build the m candidate datasets and their regions. Take $D_0 = \varnothing$, the empty dataset, and, for each element x of the universe, let D_x be k copies of x. On D_x the mode is x, so a correct answer lands in $G_x = \{x\}$, and these regions are disjoint. Each D_x is k additions away from D_0, and there are $m = N$ of them. The lemma gives Equation 26.8.

Histogram. Use the same datasets. The true histogram of D_x is $k\,e_x$, which has k in bin x and 0 elsewhere.

1. **Disjoint regions.** A correct answer on D_x lies in the ball $G_x = \{\hat{h} : \|\hat{h} - k\,e_x\|_\infty \le \alpha\}$ around the true histogram. For $x \neq y$ the centres differ by $\|k\,e_x - k\,e_y\|_\infty = k$. A point in both balls would be within α of each centre, so the centres would be within 2α of each other, by the triangle inequality. Hence the balls are disjoint when $k > 2\alpha$.
2. **Pick the smallest such k.** The lemma's bound is strongest when the datasets are as close to D_0 as possible, so take the smallest integer above 2α: $k = \lfloor 2\alpha \rfloor + 1 \le 2\alpha + 1$.
3. **Apply the lemma.** It gives $k \ge \ln(N(1-\beta))/\varepsilon$, and with step 2, $2\alpha + 1 \ge \ln(N(1-\beta))/\varepsilon$. Subtracting 1 and halving gives Equation 26.9.

26.3.4. Interpretation

Laplace is optimal up to a factor of about 2. The Laplace mechanism adds independent $\mathrm{Lap}(1/\varepsilon)$ noise to every count, which suits add/remove neighbours because one record moves one count by 1 (sensitivity 1). A $\mathrm{Lap}(b)$ variable, with density $\frac{1}{2b}e^{-|z|/b}$, exceeds α in absolute value with probability $e^{-\alpha/b}$, here $e^{-\alpha\varepsilon}$. The N noises are independent, so all of them stay within α with probability $(1 - e^{-\alpha\varepsilon})^N$, and the ℓ_∞ error exceeds α with probability $1 - (1 - e^{-\alpha\varepsilon})^N \le Ne^{-\alpha\varepsilon}$, the right side being the union bound (the probability that one of N events happens is at most the sum of their probabilities). Setting the exact expression equal to β and solving for α gives the error at failure probability β, exactly $-\ln\left(1 - (1-\beta)^{1/N}\right)/\varepsilon$. Setting the union bound equal to β gives the simpler upper bound $\ln(N/\beta)/\varepsilon$. Both the lower bound and the upper bound grow like $\ln N/\varepsilon$ (Figure 26.2). The lower bound has slope $1/(2\varepsilon)$ in $\ln N$ and Laplace has slope $1/\varepsilon$, so the gap between them is a factor of about 2 in the slope, plus the constants from β.

Report-noisy-max is optimal for the mode. Report-noisy-max adds $\mathrm{Lap}(1/\varepsilon)$ to every count and returns the bin with the largest noisy count (Selection). On D_x, bin x has count k and every other bin has count 0. So the mechanism fails only if some other bin's noise exceeds bin x's noise by k or more. The difference of two independent $\mathrm{Lap}(1/\varepsilon)$ variables has density $\frac{\varepsilon}{4}(1 + \varepsilon|z|)e^{-\varepsilon|z|}$ (the convolution of the two Laplace densities). Substituting $u = \varepsilon z$, its tail beyond k is $\frac{1}{4}\int_{k\varepsilon}^{\infty}(1+u)e^{-u}\,du = \frac{1}{4}((1+k\varepsilon)e^{-k\varepsilon} + e^{-k\varepsilon}) = e^{-k\varepsilon}(2+k\varepsilon)/4$. A union bound over the $N-1$ other bins gives a failure probability at most $(N-1)e^{-k\varepsilon}(2+k\varepsilon)/4$. Up to the slowly growing factor $2 + k\varepsilon$, this falls like $Ne^{-k\varepsilon}$, the same rate at which the packing lemma allows failure.

Where packing is loose. Two steps of the lemma give something away. Step 2 throws away the probability that $M(D_0)$ puts outside the good regions, and step 1 uses the worst-case factor $e^{k\varepsilon}$ for every region. For the Laplace histogram, $M(\varnothing)$ is pure noise around the all-zero histogram, and almost all of its mass lies outside every G_x, because a ball G_x needs bin x to be near k. That wasted mass is the factor of 2.

Approximate DP escapes. Why does the lemma need $\delta = 0$? Repeat step 1 with $\delta > 0$. Fact 1 now gives $1 - \beta \le e^{k\varepsilon}\Pr[M(D_0) \in G_i] + \delta_k$, so $\Pr[M(D_0) \in G_i] \ge e^{-k\varepsilon}(1 - \beta - \delta_k)$. Using $\delta_k \le k\,e^{(k-1)\varepsilon}\delta$, the subtracted part $e^{-k\varepsilon}\delta_k$ is at most $k\,e^{-\varepsilon}\delta$, so $\Pr[M(D_0) \in G_i] \ge e^{-k\varepsilon}(1-\beta) - k\,e^{-\varepsilon}\delta$. The first term shrinks exponentially in k and the second grows. Once $e^{-k\varepsilon}$ is smaller than about $k\delta$, the right side is negative, step 1 demands nothing of $M(D_0)$, and the lemma says nothing, however many regions there are. So under approximate DP packing can force k only up to about $\ln(1/\delta)/\varepsilon$. This is not a weakness of the proof, because a mechanism really does better. The stability-based histogram (Vadhan 2017) adds noise only to the bins that are non-empty and

releases a bin only if its noisy count clears a threshold of order $\ln(1/\delta)/\varepsilon$. Empty bins are never released, so there is no union bound over all N bins, and its error does not grow with N at all.

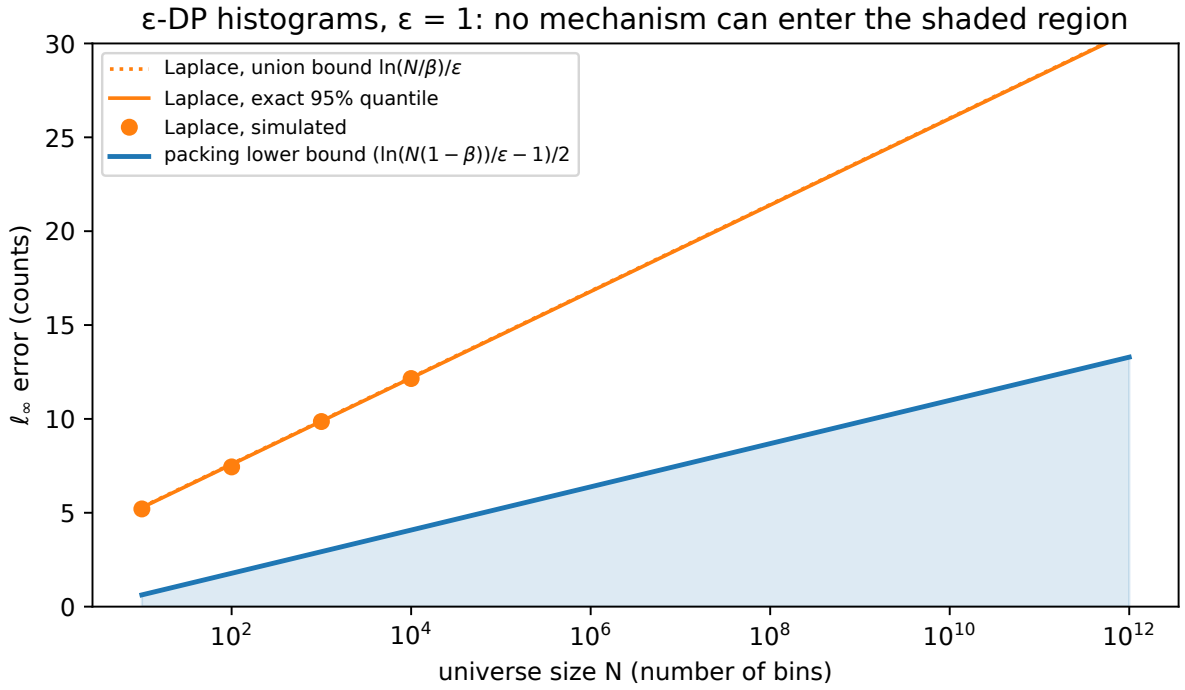


Figure 26.2.: ℓ_∞ error of ε-DP histograms over N bins at $\varepsilon = 1$, failure probability 0.05. Orange: the Laplace mechanism, exact (line), union bound (dotted, indistinguishable) and simulated (dots). Blue: the packing lower bound; no ε-DP mechanism has an error in the shaded region. Both grow like $\ln N$.

26.3.5. Example: a million ZIP codes

Table 26.2 compares the bounds at $\varepsilon = 1$ and $\beta = 0.05$ for three universe sizes. For $N = 10^6$, a typical number of fine-grained locations, an ε-DP release must allow the most common location to lead by at least $\ln(10^6 \times 0.95)/1 = 13.76$ records before it can be reported reliably; report-noisy-max needs a lead of 18.44, the k at which its failure bound $(N-1)e^{-k}(2+k)/4$ equals 0.05. Every ε-DP histogram has, at the 95% level, an error of at least $\frac{1}{2}(13.76 - 1) = 6.38$ in some bin; Laplace has 16.79. The script's simulation of the Laplace histogram at $N = 1000$ gives 9.84 against the exact 9.88, and report-noisy-max at the lead 11.09 fails 2.28% of the time against the union bound's 5%.

Table 26.2.: Packing lower bounds against the mechanisms that achieve them, $\varepsilon = 1$, $\beta = 0.05$, add/remove neighbours. Errors and leads are in records.

N	Mode: lower bound on lead k	Report-noisy-max needs	Histogram: lower bound on error	Laplace error (exact)	Laplace error (union bound)
10^3	6.86	11.09	2.93	9.88	9.90
10^6	13.76	18.44	6.38	16.79	16.81
10^9	20.67	25.65	9.84	23.69	23.72

Each factor of 1000 in N adds $\ln 1000 = 6.9$ to every column that grows like $\ln N$, and half that to the histogram lower bound, which carries the factor $\frac{1}{2}$.

26.3.6. Check yourself

- Why do the regions G_i have to be disjoint, and what happens to the lemma if they overlap? *(Disjointness is what lets the shares add up to at most 1 in step 2. If the regions overlap, one output can count for several candidates, and $M(D_0)$ can meet all the demands with less total mass.)*
- Under replace neighbours a histogram count moves by 1 in two bins. Does the lower bound change? *(Barely: the same datasets work with $D_0 = n$ copies of a fixed element, giving $N - 1$ candidates instead of N. The Laplace upper bound doubles, because the ℓ_1 sensitivity is 2. Exercise 1 in the exercise book does this.)*

26.4. Sample complexity: how many records privacy costs

26.4.1. Context

An error in counts becomes an error in fractions once you divide by n. An error of 6 records is large in a dataset of 50 and negligible in a dataset of a million. So a lower bound on error is a lower bound on the number of records: to get every frequency right to within α (a fraction), you need enough people. This view also shows how the cost of privacy compares with a cost you pay anyway, sampling error. And it is where pure and approximate DP part ways, as soon as the output has many dimensions.

26.4.2. Statement

Histograms. Translate the histogram bound of the previous section into fractions: an error of α in each frequency is an error of αn in each count, and the packing bound says this must be at least about $\ln N/(2\varepsilon)$. An ε-DP histogram whose ℓ_∞ error is at most αn with probability $1 - \beta$, on every dataset of up to n records, needs

$$n \geq \frac{\ln\left(N(1-\beta)\right) - \varepsilon}{2\alpha\varepsilon}. \tag{26.10}$$

For comparison, without privacy, estimating all N frequencies of a distribution to within α from a sample takes at most $n = \ln(2N/\beta)/(2\alpha^2)$ samples.

One-way marginals. Records are $x \in \{-1, +1\}^d$, a person's answers to d yes/no attributes, and the release is the vector of attribute means $\bar{x} \in [-1, 1]^d$. The datasets made of copies of one corner of the cube $\{\pm 1\}^d$ give 2^d candidates, so packing charges $\ln 2^d = d \ln 2$, a cost linear in d. Any ε-DP mechanism (replace) with ℓ_∞ error at most α with probability $1 - \beta$ needs

$$n \geq \frac{1}{\alpha}\left(\frac{\ln\left((2^d - 1)(1 - \beta)\right)}{\varepsilon} - 1\right) \approx \frac{d \ln 2}{\alpha\varepsilon}. \tag{26.11}$$

The approximation holds for large d, where $\ln(2^d - 1) \approx d \ln 2$ and the terms $\ln(1 - \beta)$ and -1 are small next to it.

Two mechanisms come close to this. The Laplace mechanism achieves it with $n = 2d \ln(d/\beta)/(\alpha\varepsilon)$, and the Gaussian mechanism, which is only (ε, δ)-DP, with $n = 2\sqrt{d}\, z/(\mu\alpha)$. Here z is the normal quantile with tail $\beta/(2d)$, that is, the number with $\Phi(-z) = \beta/(2d)$, where Φ is the standard normal cumulative distribution function. The number $\mu = \mu(\varepsilon, \delta)$ is the largest ratio of ℓ_2 sensitivity to noise standard deviation that the exact Gaussian calibration of The Laplace and Gaussian mechanisms allows. That chapter shows that Gaussian noise of standard deviation σ on a query of ℓ_2 sensitivity Δ_2 is (ε, δ)-DP exactly when $\delta \geq \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) - e^{\varepsilon}\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$ with $\mu = \Delta_2/\sigma$, a quantity that grows with μ; $\mu(\varepsilon, \delta)$ is the value where equality holds ($\mu = 0.2367$ at $\varepsilon = 1$, $\delta = 10^{-6}$).

26.4.3. Derivation

Histograms. Apply Equation 26.9 with the error αn in counts: $\alpha n \geq \frac{1}{2}(\ln(N(1 - \beta))/\varepsilon - 1)$. Multiplying by 2ε gives $2\alpha\varepsilon n \geq \ln(N(1 - \beta)) - \varepsilon$, and dividing by $2\alpha\varepsilon$ gives Equation 26.10. The packing datasets have $k \leq n$ records, so they are among the datasets the guarantee covers.

The sampling bound comes from Hoeffding's inequality (Concentration), which says that the average of n independent variables with values in $[0, 1]$ is more than α from its mean with probability at most $2e^{-2n\alpha^2}$. A frequency $\hat{\pi}$ is such an average of indicators, so $\Pr[|\hat{\pi} - \pi| > \alpha] \leq 2e^{-2n\alpha^2}$. A union bound over the N bins puts the probability that any bin is off by more than α at $2Ne^{-2n\alpha^2}$. Setting this to β and solving for n gives $n = \ln(2N/\beta)/(2\alpha^2)$.

Marginals, lower bound. This is packing again, now at a fixed size n, so the candidates are made by replacing records rather than adding them.

1. **Candidates.** Fix a corner $u_0 \in \{\pm 1\}^d$. D_0 is n copies of u_0. For each other corner v ($m = 2^d - 1$ of them), D_v is D_0 with k records replaced by v, so D_0 is k replacements from each D_v.
2. **Disjoint regions.** D_v has $n - k$ copies of u_0 and k copies of v, so its mean is $\frac{(n-k)u_0 + kv}{n} = u_0 + \frac{k}{n}(v - u_0)$. Two corners $v \neq v'$ differ in some coordinate by 2 (one is -1 there, the other $+1$), so their means differ there by $2k/n$. As in the histogram derivation, the ℓ_∞ balls of radius α around them are disjoint when the centres are more than 2α apart: $2k/n > 2\alpha$, that is $k > \alpha n$.
3. **Apply the lemma** with the smallest such k, $k = \lfloor \alpha n \rfloor + 1 \leq \alpha n + 1$: $\alpha n + 1 \geq \ln\left((2^d - 1)(1 - \beta)\right)/\varepsilon$. Subtracting 1 and dividing by α gives Equation 26.11.

Marginals, upper bounds. Both mechanisms add noise scaled to how much one record can move the answer. Replacing one record changes each coordinate of that record by at most 2 (from -1 to $+1$), so it moves each of the d means by at most $2/n$. Adding up over the coordinates, the ℓ_1 sensitivity is $\Delta_1 = d \cdot \frac{2}{n} = 2d/n$, and the ℓ_2 sensitivity is $\Delta_2 = \sqrt{d(2/n)^2} = 2\sqrt{d}/n$.

For the Laplace mechanism, the noise in each coordinate is $\mathrm{Lap}(b)$ with scale $b = \Delta_1/\varepsilon = 2d/(n\varepsilon)$. Each coordinate's noise exceeds α with probability $e^{-\alpha/b}$, so a union bound over the d coordinates gives $\Pr[\max_j |\eta_j| > \alpha] \le de^{-\alpha/b}$. Setting this to β gives $\alpha/b = \ln(d/\beta)$, that is $\alpha n\varepsilon/(2d) = \ln(d/\beta)$, and solving for n gives $n = 2d\ln(d/\beta)/(\alpha\varepsilon)$.

For the Gaussian mechanism, the noise in each coordinate has standard deviation $\sigma = \Delta_2/\mu$. A normal variable exceeds z standard deviations in absolute value with probability $2\Phi(-z)$, so a union bound gives $\Pr[\max_j |\eta_j| > \sigma z] \le 2d\,\Phi(-z) = \beta$, which is how z was chosen. Setting $\sigma z = \alpha$, that is $2\sqrt{d}\,z/(n\mu) = \alpha$, and solving for n gives $n = 2\sqrt{d}\,z/(\mu\alpha)$.

26.4.4. Interpretation

Privacy adds a term, it does not multiply. For histograms, the private requirement grows like $\ln N/(\alpha\varepsilon)$ and the statistical one like $\ln N/\alpha^2$. (Fano's method gives a matching statistical lower bound of the same order, so sampling really costs that much.) A data analyst needs the larger of the two. Their ratio, private over statistical, is about α/ε, so the private term dominates only when $\alpha \gtrsim \varepsilon$, that is, for coarse answers at strong privacy. At $\varepsilon = 1$ and any accuracy finer than 0.1, the records you need for sampling error alone already pay for privacy.

Pure DP pays linearly in the dimension. With d attributes the number of candidates is 2^d, so packing charges $\ln 2^d = d\ln 2$. The Laplace mechanism matches it up to the ratio of the two requirements, $\frac{2d\ln(d/\beta)/(\alpha\varepsilon)}{d\ln 2/(\alpha\varepsilon)} = 2\ln(d/\beta)/\ln 2$, a factor that grows only logarithmically in d.

Approximate DP pays only $\sqrt{d}$. The Gaussian mechanism's requirement grows like $\sqrt{d\ln d}$, because its noise is calibrated to the ℓ_2 sensitivity $2\sqrt{d}/n$ rather than the ℓ_1 sensitivity $2d/n$, and the quantile z grows like $\sqrt{\ln d}$. For large d it needs fewer records than packing says *any* pure-DP mechanism needs. There is no contradiction: packing breaks down once $\delta > 0$, as the previous section showed, because the additive δ_k of group privacy swamps the share $e^{-k\varepsilon}$ once k passes about $\ln(1/\delta)/\varepsilon$. At $\delta = 10^{-6}$ packing over 2^d corners can only force a group distance of 13, just below $\ln(10^6) = 13.82$, whatever d is, against $\ln(2^{1000} \times 0.95) = 693.10$ for pure DP at $d = 1000$. A lower bound for approximate DP needs a different idea, which is the next section.

26.4.5. Example: a survey with many questions

Table 26.3 gives the numbers at $\varepsilon = 1$ and $\beta = 0.05$. A histogram over $N = 10^6$ cells to within $\alpha = 0.1$ needs at least $(13.76 - 1)/(2 \times 0.1 \times 1) = 63.8$, so 64 records under ε-DP, and at most 875 for sampling error; at $\alpha = 0.01$ the numbers are 638 and 87,522. Privacy is the smaller cost in both cases.

Table 26.3.: Records needed to release d attribute means with ℓ_∞ error 0.1 (on the scale $[-1, 1]$) with probability 0.95, at $\varepsilon = 1$ and, for the Gaussian, $\delta = 10^{-6}$.

	Pure DP, packing (lower)	Pure DP, Laplace (upper)	Approximate DP, Gaussian (upper)
$d = 100$	683	15,202	2,941
$d = 1000$	6,921	198,070	10,836
$d = 10{,}000$	69,304	2,441,215	38,570
$d = 100{,}000$	693,137	29,017,315	134,299

The packing column grows tenfold with each tenfold increase in d, the Laplace column a little faster, and the Gaussian column only 3.7-, 3.6- and 3.5-fold, close to $\sqrt{10}$, so the Gaussian column must eventually drop below the packing column. It does so from $d = 2731$ attributes on: from there, the Gaussian mechanism needs fewer records than any pure-DP mechanism can. A survey of a few thousand yes/no items, or a model with a few thousand parameters, is already past that point. This is one reason why private machine learning, where d is the number of parameters, is done with the Gaussian mechanism and approximate DP.

26.4.6. Check yourself

- A histogram over $N = 10^6$ cells at $\varepsilon = 0.1$ instead of 1: roughly how does the private lower bound on n change, and does it now dominate the sampling term at $\alpha = 0.1$? *(By Equation 26.10 it becomes* $(\ln(950{,}000) - 0.1)/(2 \times 0.1 \times 0.1) = (13.76 - 0.1)/0.02 = 683$*, about ten times more. That is still below the sampling term of 875, but the two are now comparable.)*
- Why does the marginal lower bound Equation 26.11 not apply to the Gaussian mechanism? *(It uses the packing lemma, which needs pure DP. With $\delta > 0$ the δ_k term of group privacy swamps the $e^{-k\varepsilon}$ share once $k \gtrsim \ln(1/\delta)/\varepsilon$.)*

26.5. Fingerprinting: the $\sqrt{d}$ lower bound for approximate DP

26.5.1. Context

Packing looks for many datasets far from one reference, and approximate DP defeats it because group privacy breaks down at large distances. Fingerprinting only ever compares neighbours, so δ does not blow up. It looks at one random dataset and asks what an accurate answer reveals about each person in it. Draw a dataset at random, release its d attribute means accurately, and look at a single record. If the released means agree with that record more than with a stranger's record, the release has "fingerprinted" the person.

The argument has two halves that pull against each other. The **fingerprinting lemma** shows that accuracy *forces* such agreement: an estimate that follows the true means must follow the data, and the data are the records, so the release must correlate with the records, by a total amount of order d. DP, through Fact 2, says that each single record's correlation must be small,

no larger than a stranger's plus a little. Since there are only n records to carry the total, n must be large.

The idea comes from fingerprinting codes in cryptography (Boneh & Shaw 1998; Tardos 2003), which trace the source of a pirated copy; Bun, Ullman & Vadhan (2014) brought it to DP. The version here, with a uniform prior on the attribute means, follows Dwork et al. (2015) and Steinke & Ullman (2016).

26.5.2. Statement

The random dataset. Draw attribute means $p_1, \dots, p_d$ independently and uniformly from $[-1, 1]$. Given p, draw n records independently, each coordinate $x_{ij} \in \{\pm 1\}$ independently with mean p_j, that is, $+1$ with probability $(1 + p_j)/2$ and -1 otherwise.

Fingerprinting lemma. The lemma is about one attribute, so drop the index j. Let $x = (x_1, \dots, x_n)$ be the column of that attribute, $\bar{x}$ its mean, and $f : \{\pm 1\}^n \to \mathbb{R}$ any estimate of p computed from the column. The quantity on the left below is the correlation between the estimate and the records' deviations $x_i - p$ from the true mean, summed over the records. The lemma says that this correlation equals $2\,\mathbb{E}[p\, f(x)]$, which measures how well f follows p, and that it cannot be small unless the estimate is inaccurate:

$$\mathbb{E}\Big[f(x) \sum_{i=1}^{n} (x_i - p)\Big] = 2\,\mathbb{E}[p\, f(x)] \geq \frac{2}{3} - 2\,\mathbb{E}\,|f(x) - \bar{x}|. \tag{26.12}$$

The expectations are over p and the column.

Lower bound. Summing the lemma over the d attributes gives a total correlation of order d, and Fact 2 caps each record's share at about $(e^{\varepsilon} - 1)\sqrt{d} + 4\delta d$. So n times the cap must cover the total. Let M be (ε, δ)-DP under replace neighbours, with outputs in $[-1, 1]^d$, and with average error $\mathbb{E}\,\|M(D) - \bar{x}\|_1 \leq \alpha' d$ on the random dataset, where α' is the average error per attribute. Then

$$n \geq \frac{(2/3 - 2\alpha')\, d}{(e^{\varepsilon} - 1)\sqrt{d} + 4\delta d}. \tag{26.13}$$

The bound uses an average error, but the guarantees of the previous sections are of the (α, β) kind. They convert: an (α, β)-accurate mechanism in ℓ_∞ has $\alpha' \leq \alpha + 2\beta$. To see this, first clip its output to $[-1, 1]^d$. The true means lie in that cube, so clipping moves the output closer to them and never increases the error. Then each coordinate is off by at most α, except with probability β, when it is off by at most 2, the width of $[-1, 1]$. For $\delta \ll 1/\sqrt{d}$ and constant accuracy, the term $4\delta d$ is small next to $\sqrt{d}$, and the bound reads $n \gtrsim \sqrt{d}/(e^{\varepsilon} - 1)$.

26.5.3. Derivation

The lemma. The plan is to show that the correlation on the left is exactly the slope of the average estimate as a function of p, weighted by $1 - p^2$, and then to average that slope over p. An estimate that is accurate must rise from about -1 at $p = -1$ to about $+1$ at $p = 1$, so its slope cannot be small on average. Fix p and write $g(p) = \mathbb{E}[f(x) \mid p]$, the average estimate when the true mean is p.

1. **How the probability of the data responds to p.** The probability of the column x is $\prod_i \frac{1+x_i p}{2}$, because $\Pr[x_i = 1] = \frac{1+p}{2}$ and $\Pr[x_i = -1] = \frac{1-p}{2}$. Its log-derivative in p, called the **score** in statistics, says how strongly the data favour a larger p. It is $\sum_i \frac{x_i}{1+x_i p}$. For $x_i = 1$ the term is $\frac{1}{1+p} = \frac{1-p}{1-p^2}$, and for $x_i = -1$ it is $\frac{-1}{1-p} = \frac{-1-p}{1-p^2}$. In both cases it is $\frac{x_i - p}{1-p^2}$. So the score is the sum of the records' deviations from p, divided by $1-p^2$.
2. **Differentiate the average estimate.** Write $g(p) = \sum_x f(x) \Pr[x \mid p]$ and differentiate each term. Since $\frac{d}{dp}\Pr[x \mid p] = \Pr[x \mid p]\,\frac{d}{dp}\ln \Pr[x \mid p]$, this gives $g'(p) = \mathbb{E}\big[f(x)\,\frac{d}{dp}\ln \Pr[x \mid p] \;\big|\; p\big]$. Inserting the score of step 1 and multiplying by $1-p^2$,

$$(1-p^2)\,g'(p) = \mathbb{E}\Big[f(x)\sum_i (x_i - p) \;\Big|\; p\Big]. \tag{26.14}$$

   In words: an estimate that follows p ($g' > 0$) must be correlated with the deviations $x_i - p$ of the records, because the deviations are the only way the data carry information about p.
3. **Average over p and integrate by parts.** The density of p is $\frac{1}{2}$ on $[-1, 1]$, so averaging step 2 over p gives $\frac{1}{2}\int_{-1}^{1}(1-p^2)g'(p)\,dp$. Integration by parts moves the derivative from g to $1-p^2$, whose derivative is $-2p$:

$$\mathbb{E}\Big[f\sum_i (x_i - p)\Big] = \frac{1}{2}\int_{-1}^{1}(1-p^2)g'(p)\,dp = \frac{1}{2}\Big[(1-p^2)g(p)\Big]_{-1}^{1} + \frac{1}{2}\int_{-1}^{1} 2p\,g(p)\,dp = 2\,\mathbb{E}[p\,g(p)]. \tag{26.15}$$

   The boundary term vanishes because $1-p^2 = 0$ at ± 1 and g is finite there: each $\Pr[x \mid p]$ is a polynomial in p, so g is a polynomial. The last equality holds because $\frac{1}{2}\int_{-1}^{1} 2p\,g(p)\,dp = \int_{-1}^{1} p\,g(p)\,dp$ is twice the average $\mathbb{E}[p\,g(p)] = \frac{1}{2}\int_{-1}^{1} p\,g(p)\,dp$. Finally, $g(p)$ is the average of f given p, so $\mathbb{E}[p\,g(p)] = \mathbb{E}[p\,f(x)]$. This is the equality in Equation 26.12.
4. **Compare with the sample mean.** The sample mean is the benchmark accurate estimate. Its average given p is $\mathbb{E}[\bar{x} \mid p] = p$, so $\mathbb{E}[p\,\bar{x}] = \mathbb{E}[p^2] = \frac{1}{2}\int_{-1}^{1} p^2\,dp = \frac{1}{3}$. Any other estimate differs from it by $f - \bar{x}$, so $\mathbb{E}[p\,f] = \frac{1}{3} + \mathbb{E}[p(f-\bar{x})] \geq \frac{1}{3} - \mathbb{E}|f - \bar{x}|$, because $|p| \leq 1$. Multiply by 2.

The lower bound. The proof is an attack. Score each record x_i by how well it agrees with the release, measured as a correlation with the record's deviations from the true means:

$$Z_i = \langle M(D),\; x_i - p\rangle = \sum_{j=1}^{d} M(D)_j\,(x_{ij} - p_j). \tag{26.16}$$

A member should score high, because the release follows the data. A stranger's record, independent of the release, should score around 0. The four steps make this precise and play the two facts against each other.

1. **Accuracy makes the scores large on average.** Fix an attribute j. Condition on the other columns and on the other means p_{-j}. Columns are independent given p, so column j and p_j still have the distribution of the lemma. Let f be the average of $M(D)_j$ over the mechanism's random coins: with everything else fixed, it is a function of column j alone. The lemma gives $\mathbb{E}[M(D)_j \sum_i (x_{ij} - p_j)] \geq \frac{2}{3} - 2\,\mathbb{E}|M(D)_j - \bar{x}_j|$. Two remarks justify this. The coins are independent of the data, so averaging $M(D)_j$ over them first does not change the left side. And Jensen's inequality, here simply the fact that the absolute value of an average is at most the average of the absolute values, gives $|f - \bar{x}_j| \leq \mathbb{E}_{\text{coins}}|M(D)_j - \bar{x}_j|$.

Now sum over j. On the left, swapping the sums over j and i gives $\sum_i \mathbb{E}Z_i$. On the right, the errors add up to the ℓ_1 error:

$$\sum_{i=1}^{n} \mathbb{E}Z_i \geq \frac{2}{3}d - 2\,\mathbb{E}\|M(D) - \bar{x}\|_1 \geq \left(\frac{2}{3} - 2\alpha'\right)d. \tag{26.17}$$

2. **A stranger's score is small.** Let $D^{(i)}$ be D with record i replaced by a fresh record from the same distribution, and $Z'_i = \langle M(D^{(i)}), x_i - p\rangle$ the score of x_i against a release that never saw it. Given p, x_i is independent of $M(D^{(i)})$ and each deviation $x_{ij} - p_j$ has mean 0, so $\mathbb{E}Z'_i = 0$. For the spread, expand $Z_i'^2$ as a double sum over coordinates j, k. Given p and the release, the deviations $x_{ij} - p_j$ are independent with mean 0 and variance $1 - p_j^2 \leq 1$, so the cross terms ($j \neq k$) average to 0 and the diagonal terms give $\mathbb{E}Z_i'^2 \leq \sum_j \mathbb{E}\,M(D^{(i)})_j^2 \leq d$, because each output coordinate lies in $[-1, 1]$. By the Cauchy–Schwarz inequality, $\mathbb{E}|Z'_i| \leq \sqrt{\mathbb{E}Z_i'^2} \leq \sqrt{d}$.
3. **DP keeps a member's score close to a stranger's.** D and $D^{(i)}$ differ in exactly record i, so they are neighbours under replace. Fix the data and apply Fact 2 to the score $h(y) = \langle y, x_i - p\rangle$ of an output y. It is bounded: $|y_j| \leq 1$ and $|x_{ij} - p_j| \leq 2$, so $|h| \leq \|x_i - p\|_1 \leq 2d$, which makes $T = 2d$ and $2\delta T = 4\delta d$. Fact 2 then says that the member's score Z_i exceeds the stranger's Z'_i on average by at most $(e^\varepsilon - 1)\mathbb{E}|Z'_i| + 4\delta d$. Averaging over the data and inserting step 2: $\mathbb{E}Z_i \leq 0 + (e^\varepsilon - 1)\sqrt{d} + 4\delta d$.
4. **Combine.** Summing step 3 over the n records bounds the total from above, and step 1 bounds it from below: $(\frac{2}{3} - 2\alpha')d \leq n((e^\varepsilon - 1)\sqrt{d} + 4\delta d)$. Dividing by the bracket gives Equation 26.13.

Figure 26.3 shows the lemma at work for $n = 5$. The sample mean has $g(p) = p$, so $g' = 1$ and its correlation with the records is $(1 - p^2)$ at each p, averaging $1 - \frac{1}{3} = \frac{2}{3}$. Shrinking the estimate toward 0, to half the sample mean, halves g' and so halves the correlation, and a constant estimate has $g' = 0$ and no correlation. But those estimates pay in error, exactly as Equation 26.12 says.

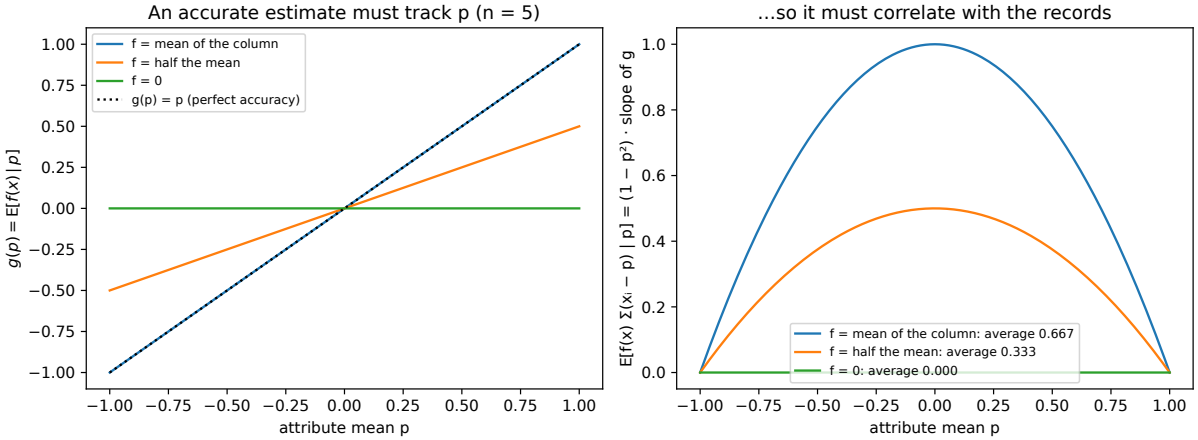


Figure 26.3.: The fingerprinting lemma for one attribute and $n = 5$ records. Left: the average estimate $g(p)$ for the sample mean, half the sample mean, and the constant 0. Right: the correlation of each estimate with the records' deviations, $(1-p^2)g'(p)$. Averaged over p it is $\frac{2}{3}$ for the mean; an estimate can only lower it by being less accurate.

26.5.4. Interpretation

The $\sqrt{d}$ is the gap between signal and noise. Step 1 says the total correlation is of order d, shared among n records, so a typical member has a score of order d/n. Step 2 says a stranger's score has spread of order $\sqrt{d}$. DP allows a member's score to exceed a stranger's by only a constant multiple of that spread. So $d/n \lesssim \sqrt{d}$, which is $n \gtrsim \sqrt{d}$. With pure DP the same bound holds with $\delta = 0$, but packing gives the stronger d.

The role of δ. The term $4\delta d$ is harmless only while $\delta \ll 1/\sqrt{d}$, so that it stays small next to $\sqrt{d}$. It cannot be removed. A mechanism that publishes each record outright with probability δ satisfies $(0, \delta)$-DP, because one record's presence changes any event's probability by at most δ (The definition of differential privacy). If δ were as large as $1/n$, it would publish about one record on average, and no lower bound of this kind could hold.

How tight is it? The $\sqrt{d}$ rate matches the Gaussian mechanism, but the constants in Equation 26.13 are poor. At $\alpha = 0.1$, $\beta = 0.05$, $\varepsilon = 1$, $\delta = 10^{-6}$ the bound says $n \geq 5.0$ for $d = 1000$ and $n \geq 15.8$ for $d =$ 10,000, against 10,836 and 38,570 records for the Gaussian mechanism (Figure 26.4). Three things separate the simple proof from the Gaussian rate. The first is a factor $1/\alpha$: Exercise 4 in the exercise book recovers it with a padding argument, and the numbers just quoted already include it. The second is a factor $\sqrt{\ln(1/\delta)}$, which Steinke & Ullman (2016) obtain with a sharper choice of the distribution of p, proving $n = \Omega(\sqrt{d \ln(1/\delta)}/(\alpha\varepsilon))$. The third is the $\sqrt{\ln d}$ from the union bound in the upper bound, which is a property of ℓ_∞ error, not of privacy. With all three, upper and lower bounds agree up to that logarithm.

26.5.5. Example: the lemma, computed exactly

For $n = 5$ the script sums over all $2^5 = 32$ possible columns and integrates over p exactly, so the numbers below have no simulation error. For the sample mean, the left side of Equation 26.12 is 0.6667, equal to $2\mathbb{E}[pf]$, and the bound is also 0.6667, because the error term $\mathbb{E}|f - \bar{x}|$ is 0: no slack. For half the sample mean, the correlation is half as large, 0.3333, and its error is $\mathbb{E}|f - \bar{x}| = \frac{1}{2}\mathbb{E}|\bar{x}| = 0.3000$, so the bound is $\frac{2}{3} - 0.6 = 0.0667$, well below the truth. For $f = 0$ the correlation is 0, the error is $\mathbb{E}|\bar{x}| = 0.6$, and the bound, $\frac{2}{3} - 1.2 = -0.5333$, is vacuous. The lemma is tight for unbiased estimators and loose for biased ones. A biased estimator $c\,\bar{x}$ loses correlation in proportion to its bias (its correlation is $2c/3$), while the bound charges it twice its absolute error.

26.5.6. Check yourself

- Why does the proof draw the means p_j at random instead of fixing them? *(For a fixed p, the constant estimate $f = p$ is accurate and uncorrelated with every record, so there is no lower bound. The mechanism must be accurate for every p at once, and averaging over p is what turns "follows p" into "correlates with the records" via the integration by parts.)*
- In step 3, why is the stranger's dataset $D^{(i)}$ a neighbour of D? *(It differs from D in exactly record i, which is the replace relation. That is why this section uses replace neighbours.)*

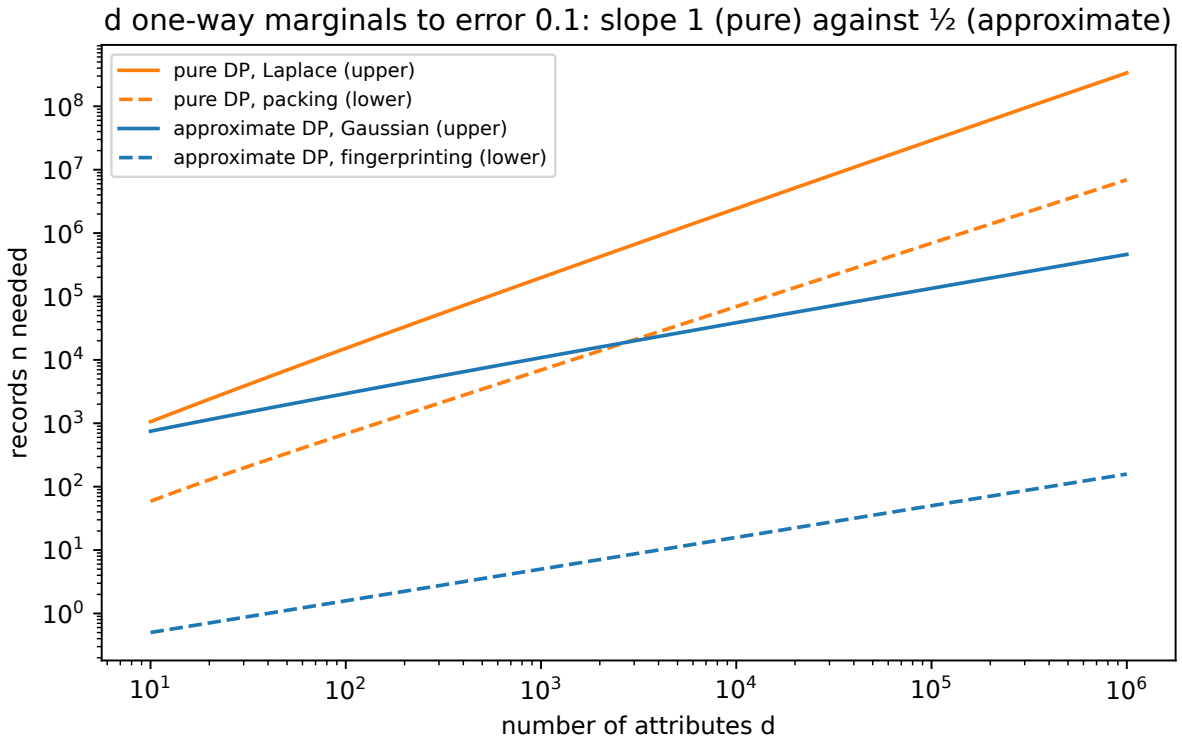


Figure 26.4.: Records needed to release d attribute means to ℓ_∞ error 0.1 with probability 0.95, at $\varepsilon = 1$ and $\delta = 10^{-6}$, on log–log axes. Orange: pure DP (Laplace upper bound, packing lower bound), both of slope 1. Blue: approximate DP (Gaussian upper bound, fingerprinting lower bound), both of slope ½. The constants of the fingerprinting bound are loose; the slopes are the point.

26.6. Tracing attacks: the lower bound is a membership inference attack

26.6.1. Context

In 2008 Homer et al. showed that from the published allele frequencies of a genome-wide association study (summary statistics of a few hundred people over hundreds of thousands of single-nucleotide polymorphisms) you could tell whether a given person's DNA was in the study. The US National Institutes of Health took such summaries off public access the same year. The test they used is, up to details, the score Z_i of the fingerprinting proof: compare the person's genome with the published frequencies, relative to the population frequencies, and see whether they agree more than chance would explain. This section runs that attack on synthetic data, computes its power, and compares it with what DP allows. Dwork et al. (2015) proved the attack is **robust** in a precise sense: it still works when the released means are noisy, and when the attacker knows the population only through a small reference sample.

26.6.2. Statement

The attacker sees the release $\hat{q} = \bar{x} + \eta$, the true means with Gaussian noise $\eta \sim \mathcal{N}(0, \sigma^2 I_d)$ of standard deviation σ in each coordinate. It also has a target record y and the population means p. It scores the target by the correlation of the release with the target's deviations from the population, $Z(y) = \langle \hat{q}, y - p \rangle$, and says "member" when the score is large. Each member pulls the release slightly toward its own record, by $1/n$ of its deviation, so a member's score is shifted up; a non-member's record is independent of the release, so its score is centred at 0. With the random dataset of the previous section, the shift and the spread of the scores are

$$\mathbb{E}[Z \mid \text{member}] - \mathbb{E}[Z \mid \text{non-member}] = \frac{2d}{3n}, \qquad \operatorname{Var}[Z \mid \text{non-member}] = d\left(\frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3}\right). \tag{26.18}$$

How well the attack works depends on the shift measured in units of the spread. If the score is approximately normal with this mean gap m and standard deviation s under both hypotheses, the best threshold has advantage $\mathrm{TPR} - \mathrm{FPR} = 2\Phi(m/(2s)) - 1$, where Φ is the standard normal cumulative distribution function.

26.6.3. Derivation

All averages are over p uniform on $[-1, 1]$, with density $\frac{1}{2}$, where $\mathbb{E}[p^2] = \frac{1}{2}\int_{-1}^{1} p^2\, dp = \frac{1}{3}$, $\mathbb{E}[p^4] = \frac{1}{2}\int_{-1}^{1} p^4\, dp = \frac{1}{5}$ and so $\mathbb{E}[(1-p^2)^2] = 1 - \frac{2}{3} + \frac{1}{5} = \frac{8}{15}$. Two facts about a ± 1 variable with mean p are used throughout: its square is always 1, and its variance is $1 - p^2$.

1. **Non-member mean.** A fresh y is independent of $\hat{q}$ given p, and $\mathbb{E}[y_j - p_j \mid p] = 0$, so each coordinate of Z averages to 0 and $\mathbb{E}Z = 0$.
2. **Member mean.** For $y = x_i$, coordinate j contributes $\mathbb{E}[\bar{x}_j(x_{ij} - p_j)]$; the noise is independent of everything and averages out. Write $\bar{x}_j = \frac{1}{n}\sum_k x_{kj}$. The terms $k \neq i$ are other people's records, independent of x_{ij}, and contribute 0. The term $k = i$ is the member's own record inside the mean, and contributes $\frac{1}{n}\mathbb{E}[x_{ij}(x_{ij} - p_j)] = \frac{1}{n}\mathbb{E}[1 - x_{ij}p_j] = \frac{1}{n}(1 - p_j^2)$, because

$x_{ij}^2 = 1$ and $\mathbb{E}[x_{ij}] = p_j$. Averaging over p_j gives $\frac{1}{n} \cdot \frac{2}{3}$, and summing over the d coordinates gives $\frac{2d}{3n}$.

3. **Non-member variance.** The mean is 0, so the variance is $\mathbb{E}Z^2$. The coordinates $y_j - p_j$ are independent of each other and of $\hat{q}$, with mean 0, so in the double sum $Z^2 = \sum_{j,k} \hat{q}_j \hat{q}_k (y_j - p_j)(y_k - p_k)$ the cross terms vanish and $\operatorname{Var} Z = \sum_j \mathbb{E}[\hat{q}_j^2(1-p_j^2)]$. Given p, the mean square of $\hat{q}_j$ is its squared mean plus its variance: $\mathbb{E}[\hat{q}_j^2] = p_j^2 + \frac{1-p_j^2}{n} + \sigma^2$ (squared mean, sampling variance of an average of n records, noise variance). Multiply by $1 - p_j^2$ and average over p_j: $\mathbb{E}[p^2 - p^4] + \frac{1}{n}\mathbb{E}[(1-p^2)^2] + \sigma^2 \mathbb{E}[1-p^2] = (\frac{1}{3} - \frac{1}{5}) + \frac{8}{15n} + \frac{2\sigma^2}{3} = \frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3}$ per coordinate.
4. **Advantage of a threshold.** For $\mathcal{N}(0, s^2)$ against $\mathcal{N}(m, s^2)$, the two densities cross at the midpoint $m/2$, and by the Neyman–Pearson lemma (the best test thresholds the ratio of the two densities, which here is increasing in the score) the best threshold is that crossing point. The test fires above $m/2$. A member's score exceeds it with probability $\Phi(\frac{m}{2s})$, since $m/2$ is $\frac{m}{2s}$ standard deviations below the member mean, and a non-member's with probability $\Phi(-\frac{m}{2s})$, by the mirror argument. So $\mathrm{TPR} - \mathrm{FPR} = \Phi(\frac{m}{2s}) - \Phi(-\frac{m}{2s}) = 2\Phi(\frac{m}{2s}) - 1$, using $\Phi(-t) = 1 - \Phi(t)$. This is the total variation of the two normal distributions, as in Hypothesis testing, where the best advantage of any test is shown to equal the TV distance.

26.6.4. Interpretation

Tracing needs $d \gtrsim n^2$, and noise of order $\sqrt{d}/n$ stops it. Without noise, and ignoring the small $8/(15n)$ term, the signal-to-noise ratio is $m/s \approx \frac{2d}{3n} \big/ \sqrt{2d/15} = \frac{2\sqrt{d}}{3n} \big/ \sqrt{2/15}$. The attack is strong once this ratio is a few units, that is, once $\sqrt{d}$ is a few times n: once there are many more attributes than the square of the number of people. Noise adds $2\sigma^2/3$ to the variance per coordinate, and dominates the sampling term $\frac{2}{15}$ once $\frac{2\sigma^2}{3} > \frac{2}{15}$, that is, $\sigma \gtrsim 1/\sqrt{5}$. The ratio then behaves like $\frac{2d}{3n} \big/ \sqrt{2d\sigma^2/3}$, which is of order $\sqrt{d}/(n\sigma)$. So the attack fails when $\sigma \gg \sqrt{d}/n$. The Gaussian mechanism adds $\sigma = \Delta_2/\mu = 2\sqrt{d}/(n\mu)$: the same order, as it must be for a mechanism that is both private and as accurate as fingerprinting allows.

DP bounds the attack for each target. The member-versus-stranger game is a test between $M(D)$ and $M(D^{(i)})$, neighbours under replace. Here the member's record is in D rather than in the second dataset, but Fact 3 does not care which of two neighbours plays which role, because the definition holds for both orders of the pair. So by Fact 3 its advantage is at most $(e^\varepsilon - 1 + 2\delta)/(e^\varepsilon + 1)$. For the Gaussian mechanism there is a sharper bound. Its outputs on the two neighbours are Gaussians with the same σ whose means differ by at most Δ_2, and step 4 with $m = \Delta_2$ and $s = \sigma$ gives their TV distance, at most $2\Phi(\Delta_2/(2\sigma)) - 1$.

This is the membership inference of Part 2, on statistics instead of models. The score is a likelihood-ratio-style statistic, and, as in Membership inference, the honest way to report it is the TPR at a low FPR, because an attack that identifies a few people with confidence is a privacy failure even if its average advantage is small. Released aggregates are not safe because they are aggregates: what protects members is the ratio of n to $\sqrt{d}$, or noise.

26.6.5. Example: 50 people, 2000 attributes

The script draws 200 datasets of $n = 50$ records with $d = 2000$ attributes, and scores the 50 members and 50 fresh non-members of each. Without noise, Equation 26.18 predicts a mean gap of $2 \times 2000/(3 \times 50) = 26.67$ and a spread of $\sqrt{2000(\frac{2}{15} + \frac{8}{750})} = 16.97$; the simulation gives 26.70 and 16.92. The normal approximation then gives the advantage $2\Phi(26.67/(2 \times 16.97)) - 1 = 2\Phi(0.786) - 1 = 0.568$. Table 26.4 shows the attack at three noise levels. The $(1, 10^{-5})$-DP Gaussian mechanism on these means has $\sigma = 6.67$.

Table 26.4.: The tracing attack on released attribute means, $n = 50$, $d = 2000$. The advantage is the best TPR − FPR over thresholds.

Noise σ	Advantage (simulated)	Advantage (normal approximation)	Worst-case TV bound	TPR at 1% FPR
0	0.578	0.568	1.000	0.242
1.00	0.264	0.259	0.629	0.052
6.67 (DP, $\varepsilon = 1$)	0.049	0.044	0.107	0.014

The worst-case TV column is $2\Phi(\Delta_2/(2\sigma)) - 1$ with $\Delta_2 = 2\sqrt{2000}/50 = 1.79$; without noise it is 1, because in the worst case the exact means of two neighbours differ, and an exact release then tells them apart perfectly. Without noise, a quarter of the members are identified at a 1% false-alarm rate, from means of 50 people. With $(1, 10^{-5})$-DP noise the advantage is 0.049, under both the Gaussian TV bound of 0.107 and the generic DP bound of Fact 3, 0.462. The TPR at 1% FPR is 0.014. DP bounds it too: applied to the event "the test fires", the definition says $\text{TPR} \le e^{\varepsilon}\,\text{FPR} + \delta = e^1 \times 0.01 + 10^{-5} = 0.0272$, and 0.014 is under it. The simulated advantage sits slightly above the normal approximation, because it takes the best of many thresholds on a finite sample, which picks up some luck.

Figure 26.5 shows the score distributions and Figure 26.6 the whole curve. The attack fades out around $\sigma = \sqrt{d}/n = \sqrt{2000}/50 = 0.89$ and is almost gone at the DP noise level.

26.6.6. Check yourself

- If you double both n and $\sqrt{d}$, what happens to the tracing advantage without noise? *(Almost nothing: $m/s \approx \frac{2\sqrt{d}}{3n}/\sqrt{2/15}$ depends on $\sqrt{d}/n$, apart from the small $8/(15n)$ term.)*
- The attacker needs p. Why is it not enough to replace p by the released $\hat{q}$ itself? *(Then the score is $\langle \hat{q}, y - \hat{q} \rangle$, and the member's own record is part of $\hat{q}$, so the subtraction cancels part of the signal. A separate reference sample keeps the mean gap $2d/(3n)$, at the cost of extra variance; see Exercise 6 in the exercise book.)*

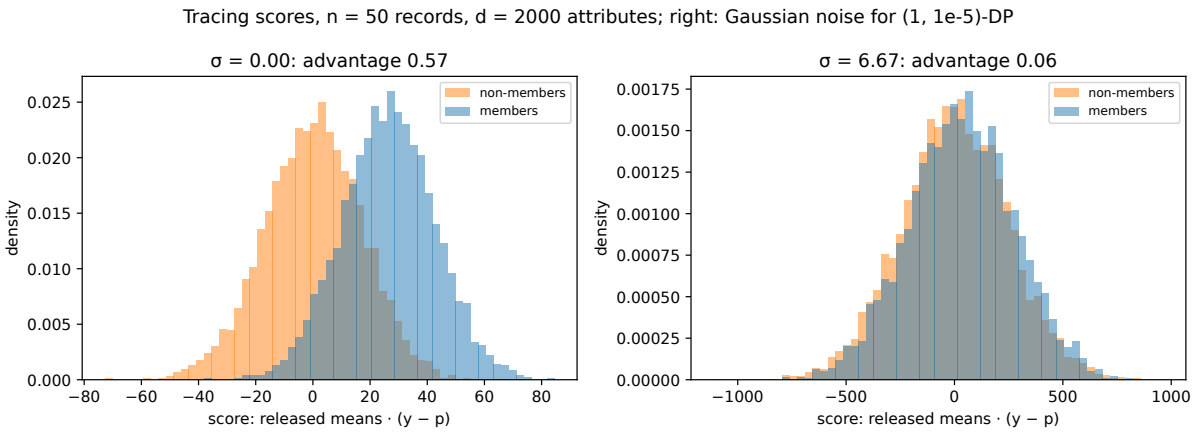


Figure 26.5.: Tracing scores of members (blue) and non-members (orange) for $n = 50$ records and $d = 2000$ attributes. Left: exact means; the members' scores are shifted by $2d/(3n) = 26.7$. Right: means released with the Gaussian noise of a $(1, 10^{-5})$-DP mechanism; the two distributions nearly coincide.

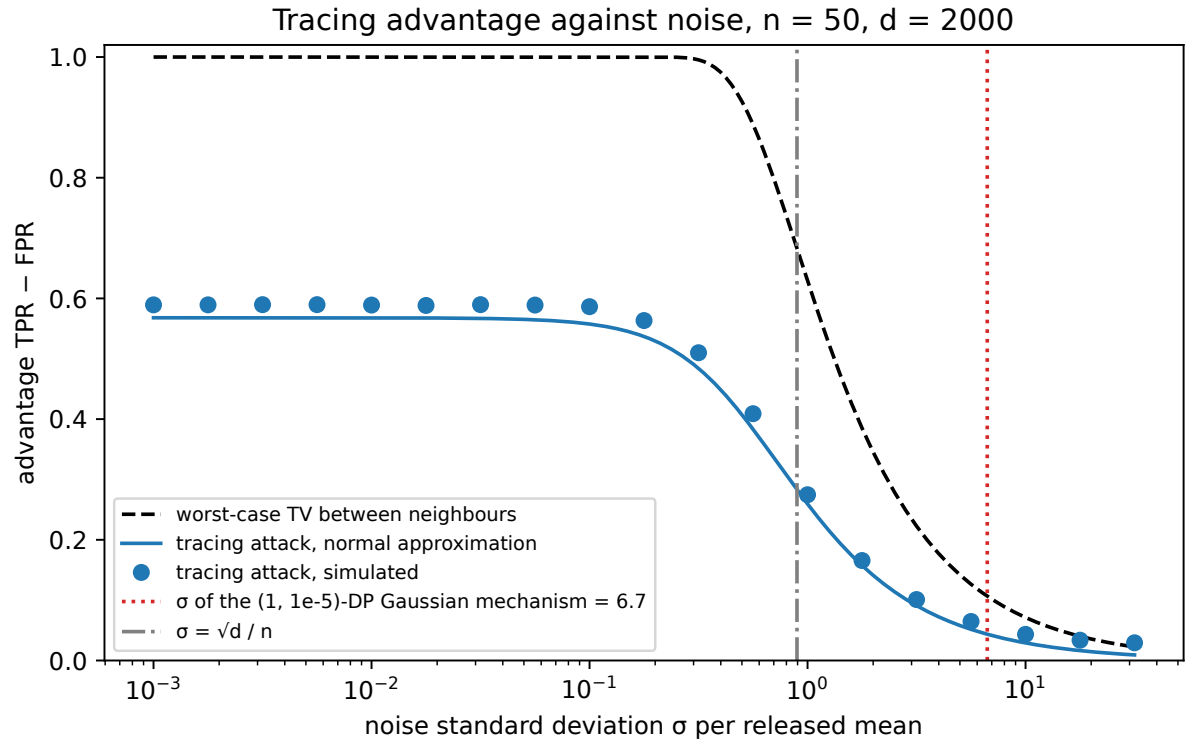


Figure 26.6.: Advantage of the tracing attack against the noise standard deviation, $n = 50$, $d = 2000$. Dots: simulation; solid line: the normal approximation from Equation 26.18; dashed: the TV distance between the outputs on neighbouring datasets in the worst case, which bounds every attack. The attack fails once σ is well above $\sqrt{d}/n$; the DP Gaussian mechanism adds $\sigma = 6.7$.

26.7. Dinur–Nissim as a lower bound

26.7.1. Context

Reconstruction attacks showed that a database of n secret bits can be almost entirely reconstructed from a number of random subset sums only polylogarithmically larger than n (Dinur and Nissim used $n(\log n)^2$; later analyses need only $O(n)$), answered with error $o(\sqrt{n})$, that is, error that grows more slowly than $\sqrt{n}$. Each query picks a random subset of the people and asks how many of them have bit 1. The attacker then finds the database of bits that best fits all the noisy answers, for example by least squares. If the errors are much smaller than $\sqrt{n}$, any database that fits them well must agree with the true one on almost every bit. There it was an attack. Here it is a lower bound: since DP limits how well any single bit can be guessed (Fact 3), a release from which most bits can be reconstructed cannot be private. Every mechanism that answers that many counting queries with (ε, δ)-DP must therefore add error of order $\sqrt{n}$ to its answers. The Gaussian mechanism adds exactly that order, so the bound is tight.

26.7.2. Statement

Let the database be n independent uniform bits, and call two databases neighbours when they differ in one bit. Fact 3 says each bit can be guessed correctly with probability at most $(e^\varepsilon + \delta)/(e^\varepsilon + 1)$, and the fraction of bits recovered is an average of such guesses, so it obeys the same limit. Read the other way, recovering a fraction r proves a lower bound on ε. If M is (ε, δ)-DP, then every reconstruction procedure applied to M's output recovers, in expectation, a fraction of the bits of at most

$$r \le \frac{e^\varepsilon + \delta}{e^\varepsilon + 1}, \qquad \text{so} \qquad \varepsilon \ge \ln \frac{r - \delta}{1 - r}. \tag{26.19}$$

By the Dinur–Nissim theorem, in the sharper form of Dwork, McSherry and Talwar (2007) that needs only $k = O(n)$ queries, answering k random subset-sum queries with error $o(\sqrt{n})$ allows a reconstruction with $r \to 1$. The right side of Equation 26.19 then grows without limit, so no such mechanism is (ε, δ)-DP for any fixed ε and $\delta < 1/2$. Error $\Omega(\sqrt{n})$, at least a constant times $\sqrt{n}$, is necessary.

26.7.3. Derivation

1. **One bit at a time.** Fix i and all bits except x_i. Since the bits are independent, x_i is still uniform. The two values of x_i give databases that differ in one bit, which are neighbours. The reconstruction's guess of x_i is computed from M's output alone (a post-processing), so it is a test between the two neighbours, and by Fact 3 it is right with probability at most $(e^\varepsilon + \delta)/(e^\varepsilon + 1)$. Averaging over the other bits keeps the bound.
2. **Average over the bits.** The expected fraction recovered is $\frac{1}{n}\sum_i \Pr[\text{bit } i \text{ is guessed right}]$, by linearity of expectation, and each term obeys the bound of step 1, so the average does too. To solve $r \le (e^\varepsilon + \delta)/(e^\varepsilon + 1)$ for ε, multiply by $e^\varepsilon + 1$: $re^\varepsilon + r \le e^\varepsilon + \delta$, that is $e^\varepsilon(1 - r) \ge r - \delta$. Dividing by $1 - r$ and taking logarithms gives the second form of Equation 26.19.
3. **Insert the attack.** If the errors are $o(\sqrt{n})$, the Dinur–Nissim decoder misses $o(n)$ bits, so $r \to 1$, the denominator $1 - r$ goes to 0, and the bound forces $\varepsilon \to \infty$.

26.7.4. Interpretation

The Gaussian mechanism matches it. Changing one bit changes by 1 the answer of every query whose subset contains that person, and leaves the others alone. At most k answers change, so the answer vector moves by at most $\sqrt{k}$ in ℓ_2 (in fact by about $\sqrt{k/2}$, since each person is in about half of the random subsets). The Gaussian mechanism therefore needs σ proportional to $\sqrt{k}/\mu(\varepsilon,\delta)$, which for $k = 4n$ is $2\sqrt{n}/\mu$, of order $\sqrt{n}$. Answering k counting queries costs error of order $\sqrt{k}$ per query under approximate DP, as the marginals section found for d means. Under pure DP the Laplace mechanism must be calibrated to the ℓ_1 sensitivity, which is k, and so pays error of order k per query, as packing says it must.

The bound is crude in one way. It looks only at the fraction of bits recovered, and least squares is not the best attack. So the ε it implies is far below the ε the Gaussian mechanism guarantees at the same noise. The true privacy loss of the release lies somewhere between them. Auditing, in a later part, narrows this gap with stronger attacks.

26.7.5. Example: 200 bits, 800 queries

The script answers $k = 800$ random subset sums of $n = 200$ uniform bits with Gaussian noise $\sigma = c\sqrt{n}$, decodes by least squares and rounds at $\frac{1}{2}$, over 40 random databases. Table 26.5 gives the fraction recovered, the ε it implies by Equation 26.19 at $\delta = 10^{-6}$, and the ε that the Gaussian mechanism with that noise guarantees (with ℓ_2 sensitivity $\sqrt{k}$, so $\mu = \sqrt{k}/\sigma = \sqrt{800}/(c\sqrt{200}) = 2/c$). For instance, recovering 0.9607 of the bits proves $\varepsilon \geq \ln\left((0.9607 - 10^{-6})/(1 - 0.9607)\right) = 3.20$.

Table 26.5.: Reconstruction from 800 noisy subset sums of 200 bits, and the privacy loss it proves, at $\delta = 10^{-6}$.

$\sigma/\sqrt{n}$	Fraction recovered	ε at least (attack)	ε at most (Gaussian mechanism)
0.1	1.0000	∞	294.17
0.25	0.9607	3.20	69.24
0.5	0.8058	1.42	26.36
1	0.6710	0.71	11.00
2	0.5810	0.33	4.89

At $\sigma = 0.1\sqrt{n} = 0.1 \times \sqrt{200} = 1.4$ every bit is recovered, so $r = 1$ and the release is not $(\varepsilon, 10^{-6})$-DP for any finite ε. A mechanism that wants $\varepsilon = 1$ can let an attacker recover at most $e/(e+1) = 73.11\%$ of the bits (Fact 3). In the table, 0.8058 at $0.5\sqrt{n}$ is above that and 0.6710 at $\sqrt{n}$ is below it, so such a mechanism needs noise between $0.5\sqrt{n}$ and $\sqrt{n}$ per answer. Figure 26.7 shows the whole range.

26.7.6. Check yourself

- Why does Equation 26.19 need the secret bits to be uniform? *(Fact 3's bit-guessing bound is for a uniform prior. With a biased prior, guessing the more likely value already beats $\frac{1}{2}$, and the bound must start from the prior instead.)*

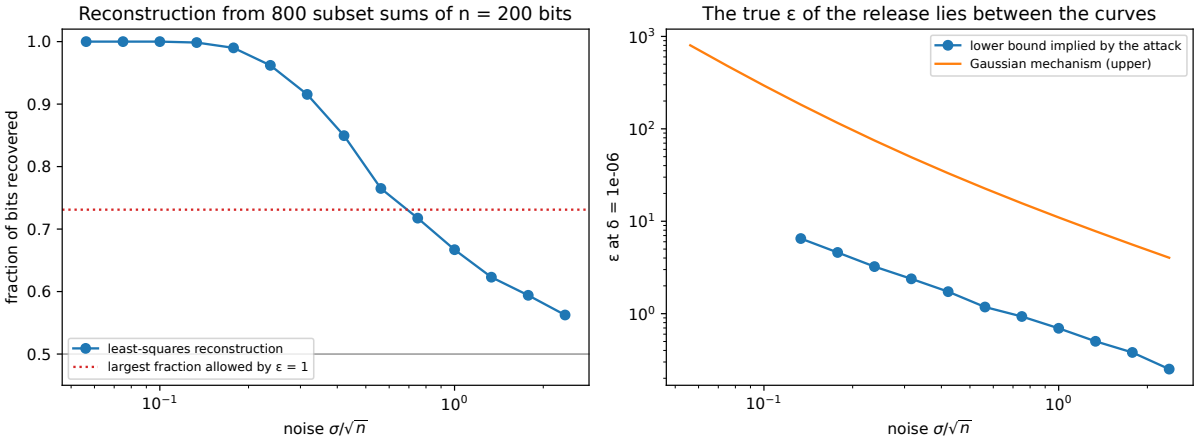


Figure 26.7.: Left: fraction of 200 secret bits recovered by least squares from 800 subset sums with Gaussian noise of standard deviation σ, against $\sigma/\sqrt{n}$. The dotted line is the most that any $\varepsilon = 1$ mechanism allows. Right: the ε that the reconstruction proves (blue) and the ε that the Gaussian mechanism guarantees (orange), at $\delta = 10^{-6}$; the true privacy loss of the release lies between them.

- The attack recovers 67% of the bits at $\sigma = \sqrt{n}$. Does that mean the release is $(0.71, 10^{-6})$-DP? *(No. The attack proves only $\varepsilon \geq 0.71$. A lower bound from one attack is never a privacy guarantee; the guarantee comes from the mechanism's analysis, here $\varepsilon \leq 11.00$.)*

26.8. Code

The script code/lower_bounds.py computes every number in this chapter and draws every figure. Each bound and each mechanism's error is a small function. The simulations check the Laplace histogram and report-noisy-max against their formulas, the fingerprinting lemma by exact summation, the tracing attack against Equation 26.18, and the reconstruction attack against Equation 26.19. From the repository root:

```
uv run python code/part03-dp-foundations/lower_bounds.py
```

Output (seed 0), summarized in Table 26.6:

Table 26.6.: The script's output with the default seed.

Experiment	Result
Toolkit, $\varepsilon = 1$	Advantage at most 0.4621; bit guessed at most 0.7311 of the time; group $\delta_5 = 8.6 \times 10^{-5}$, $\delta_{20} = 282$ at $\delta = 10^{-6}$
Mode and histogram, $N = 10^6$	Lead $k \geq 13.76$ (noisy-max needs 18.44); error ≥ 6.38 (Laplace 16.79); full table in Table 26.2

Experiment	Result
Simulations at $N = 1000$	Laplace 95% error 9.84 (exact 9.88); noisy-max failure 0.0228 at lead 11.09 (union bound 0.05)
Packing under $\delta = 10^{-6}$, 2^d candidates	Forces $k \geq 7$ at $d = 10$ and $k \geq 13$ at $d = 100$ and 1000 (pure: 6.88, 69.26, 693.10); $\ln(1/\delta)/\varepsilon = 13.82$
Sample complexity, histogram $N = 10^6$	$\alpha = 0.1$: 64 private, 875 sampling. $\alpha = 0.01$: 638 private, 87,522 sampling
Sample complexity, marginals	Table 26.3; the Gaussian beats the pure-DP lower bound from $d = 2731$; $\mu(1, 10^{-6}) = 0.2367$
Fingerprinting lower bound	$n \geq 1.6$, 5.0, 15.8, 50.1 at $d = 10^2, 10^3, 10^4, 10^5$
Fingerprinting lemma, $n = 5$	Mean: 0.6667 (bound 0.6667). Half mean: 0.3333 (bound 0.0667). Zero: 0 (bound -0.5333)
Tracing, $n = 50$, $d = 2000$	Gap 26.70 (predicted 26.67), spread 16.92 (16.97); Table 26.4
Dinur–Nissim, $n = 200$, $k = 800$	Table 26.5

tests/test_lower_bounds.py checks that the output still matches.

What to take from it. Packing is nearly tight under pure DP: the lower bounds and the Laplace and report-noisy-max mechanisms all grow like $\ln N/\varepsilon$, within a factor of about 2. Allowing $\delta > 0$ changes the rate, not just the constant. Packing cannot force more than $\ln(1/\delta)/\varepsilon$ under approximate DP, because group privacy degrades past that distance, and so the Gaussian mechanism overtakes every pure-DP mechanism from a few thousand dimensions on. The $\sqrt{d}$ lower bound that replaces packing is an attack you can run: the fingerprinting score identifies members of a 50-person dataset from its means, its power matches the formula, and DP noise of order $\sqrt{d}/n$ defeats it. Finally, lower bounds from attacks are lower bounds on ε. The reconstruction attack proves a release is not private below some ε; it never proves that it is.

26.9. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 8: packing lower bounds and reconstruction attacks as lower bounds, at textbook depth.
- Vadhan, "The Complexity of Differential Privacy" (2017), §5: packing, fingerprinting and reconstruction lower bounds side by side; also the stability-based histogram for approximate DP.
- Hardt & Talwar, "On the Geometry of Differential Privacy" (STOC 2010): packing generalized to any set of linear queries through volume arguments, with matching upper bounds for pure DP.

- Beimel, Brenner, Kasiviswanathan & Nissim, "Bounds on the Sample Complexity for Private Learning and Private Data Release" (Machine Learning, 2014): the sample-complexity view of packing.
- Bun, Ullman & Vadhan, "Fingerprinting Codes and the Price of Approximate Differential Privacy" (STOC 2014): the first $\sqrt{d}$ lower bound for approximate DP, through fingerprinting codes.
- Steinke & Ullman, "Between Pure and Approximate Differential Privacy" (Journal of Privacy and Confidentiality, 2016): the simplified fingerprinting lemma and the tight bound $\Omega(\sqrt{d\ln(1/\delta)}/(\alpha\varepsilon))$ for one-way marginals.
- Dwork, Smith, Steinke, Ullman & Vadhan, "Robust Traceability from Trace Amounts" (FOCS 2015): tracing attacks that tolerate noisy statistics and estimated population means.
- Dwork, Smith, Steinke & Ullman, "Exposed! A Survey of Attacks on Private Data" (Annual Review of Statistics and Its Application, 2017): reconstruction and tracing attacks for a statistics audience.
- Homer et al., "Resolving Individuals Contributing Trace Amounts of DNA to Highly Complex Mixtures Using High-Density SNP Genotyping Microarrays" (PLoS Genetics, 2008): the tracing attack on genomic summary statistics.
- Kamath & Ullman, "A Primer on Private Statistics" (2020, arXiv): fingerprinting lower bounds for private mean estimation, with the score-and-integrate-by-parts proof used here.
- Dinur & Nissim, "Revealing Information while Preserving Privacy" (PODS 2003): the reconstruction attack and the $\sqrt{n}$ threshold.
- Boneh & Shaw, "Collusion-Secure Fingerprinting for Digital Data" (IEEE Transactions on Information Theory, 1998) and Tardos, "Optimal Probabilistic Fingerprint Codes" (STOC 2003): the cryptographic codes that gave fingerprinting its name.

26.10. Cheat sheet

α is the error, β the failure probability, d the number of attributes or queries, N the universe size, n the number of records. Table 26.7 collects the results.

Table 26.7.: Every lower bound in this chapter, with the mechanism that matches it.

Result	Statement	Matching mechanism
Packing lemma (pure)	m disjoint good sets, all within k of D_0: $k \geq \ln(m(1-\beta))/\varepsilon$	—
Mode over N elements (pure)	Lead $k \geq \ln(N(1-\beta))/\varepsilon$	Report-noisy-max: $(N-1)e^{-k\varepsilon}(2+k\varepsilon)/4 \leq \beta$
Histogram, ℓ_∞ (pure)	$\alpha \geq \frac{1}{2}(\ln(N(1-\beta))/\varepsilon - 1)$	Laplace: $\alpha \leq \ln(N/\beta)/\varepsilon$
Packing under $\delta > 0$	Forces at most $k \approx \ln(1/\delta)/\varepsilon$	Stability-based histogram
Histogram, records	$n \geq (\ln(N(1-\beta)) - \varepsilon)/(2\alpha\varepsilon)$; sampling needs $\approx \ln(2N/\beta)/(2\alpha^2)$	Laplace
d marginals (pure)	$n \gtrsim d\ln 2/(\alpha\varepsilon)$	Laplace: $n = 2d\ln(d/\beta)/(\alpha\varepsilon)$
Fingerprinting lemma	$\mathbb{E}[f\sum_i(x_i - p)] = 2\mathbb{E}[pf] \geq \frac{2}{3} - 2\mathbb{E}\lvert f - \bar{x}\rvert$	—

Result	Statement	Matching mechanism
d marginals (approximate)	$n \geq (\frac{2}{3} - 2\alpha')d/((e^{\varepsilon} - 1)\sqrt{d} + 4\delta d)$	Gaussian: $n = 2\sqrt{d}\, z/(\mu\alpha)$
Tracing score	Gap $2d/(3n)$; variance $d(\frac{2}{15} + \frac{8}{15n} + \frac{2\sigma^2}{3})$	Defeated by $\sigma \gg \sqrt{d}/n$
Bit guessing / membership	Accuracy $\leq \frac{e^{\varepsilon}+\delta}{e^{\varepsilon}+1}$; advantage $\leq \frac{e^{\varepsilon}-1+2\delta}{e^{\varepsilon}+1}$	—
Dinur–Nissim	Recovering a fraction r proves $\varepsilon \geq \ln \frac{r-\delta}{1-r}$; error $o(\sqrt{n})$ on $O(n)$ queries is never private	Gaussian with $\sigma \propto \sqrt{k}$

Rules of thumb

- A lower bound on error is an attack on accurate mechanisms plus DP's limit on attacks.
- Pure DP pays ln(number of candidates)/ε; for d-dimensional outputs that is linear in d.
- Approximate DP pays about $\sqrt{d}$ for d statistics, and δ must stay far below $1/n$ for that to mean anything.
- Answering k counting queries privately costs error about $\sqrt{k}$ per query with $\delta > 0$, and about k without.
- Privacy is an additive cost in the number of records: it matters most for coarse answers at small ε.
- Aggregates over many attributes are not safe by being aggregates: without noise, tracing reaches advantage 0.5 once $d \approx 0.56\, n^2$ (Exercise 5 in the exercise book). Noise of order $\sqrt{d}/n$ stops it.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

Part IV.

Privacy accounting

The previous part defined differential privacy (DP), built the basic mechanisms and proved how (ε, δ) guarantees compose. This part answers the question that decides whether a trained model's privacy claim means anything: **after thousands of noisy steps, each on a random subset of the data, how large is ε, really?** The answer is never read off a single theorem. It comes from an **accountant**, a procedure that tracks some summary of each step's privacy loss, combines the summaries, and converts the result into the (ε, δ) you report. The running case throughout is stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD, and the workhorse summary is Rényi differential privacy (RDP). Two accountants given the same training run can report values of ε that differ by a large factor. Both can be correct upper bounds, but only the tighter one tells you what the noise actually bought.

Who this part is for

You are expected to have read the prerequisites and the part on DP foundations. From Divergences you need the Rényi divergence D_α, the hockey-stick divergence E_γ and the data-processing inequality. From Hypothesis testing you need trade-off functions $T(P, Q)$ and the Gaussian trade-off curve. From Concentration inequalities you need Hoeffding's lemma, and from Optimization the contraction of a gradient step. From the DP foundations you need randomized response and the hypothesis-testing reading of DP, basic composition, the privacy loss random variable with advanced and optimal composition from Composition, the Gaussian mechanism and its exact $\delta(\varepsilon)$ from The Laplace and Gaussian mechanisms, and the local and shuffle models from Trust models. Each chapter restates the facts it uses in one line and links to them.

The common thread

Composition is easy for the privacy loss itself. If you run k mechanisms, the privacy loss of the whole run is the sum of the k per-step losses, and its distribution is a convolution. Composition is hard for (ε, δ), because a pair of numbers forgets almost everything about that distribution. Advanced composition recovers part of what is lost. The rest of the field rests on one move: **track a richer summary of the privacy loss, one that composes simply, and convert to (ε, δ) only once, at the end.**

The summaries differ in how much they keep, and Table 26.8 lists them.

Table 26.8.: The chapters of this part, the question each one answers, the summary of the privacy loss it tracks, and the earlier chapters it uses.

Chapter	The question it answers	Summary it tracks	Builds on
Rényi DP	Which summary of the privacy loss adds up under composition?	The curve $\varepsilon(\alpha)$ of Rényi divergences	2, 3, 18–21

Chapter	The question it answers	Summary it tracks	Builds on
From Rényi DP to (ε, δ) and to tests	How much does converting an RDP curve to (ε, δ) lose, and what is unavoidable?	The same curve, read as a constraint on tests	3, 4, 18, 21, 27
Concentrated differential privacy	When does one number ρ describe the whole curve, and when not?	A line $\rho\alpha$ above the curve	2, 3, 19–21, 24, 27, 28; one formula from 31
f-DP and Gaussian differential privacy	What is the most informative description of a mechanism's privacy?	The full trade-off function; μ for Gaussian ones	3, 4, 18, 21, 27, 28
Amplification by subsampling	How much privacy does using each record only with probability q buy?	RDP of the subsampled Gaussian	3, 18–21, 27, 28
Numerical accounting with privacy loss distributions	How do you compute the exact ε of a composition, with error bars?	The discretized distribution of the privacy loss	3, 18, 20, 21, 27, 28, 30, 31
Amplification by shuffling and by iteration	What other sources of uncertainty amplify privacy?	Bounds for shuffled reports and hidden iterates	3, 6, 7, 18, 25, 27, 28, 30, 31
Adaptive composition: filters, odometers and individual accounting	What changes when the privacy parameters are chosen on the fly, or differ per record?	Running sums checked by filters and odometers	2, 21, 27, 29, 31
Choosing an accountant	Which accountant should you use for DP-SGD, and what should you report?	All of the above, side by side	3, 21, 27–34

Read the table from top to bottom as a ladder. An RDP curve is a lossy summary of the trade-off function, a zero-concentrated DP (zCDP) parameter ρ is a lossy summary of the RDP curve, and (ε, δ) is a lossy summary of all three. Numerical accounting keeps the whole privacy loss distribution and loses only discretization error, which it bounds. Amplification (chapters 31 and 33) is a separate axis. It is about which pair of distributions you account for in the first place, and every accountant above must use the pair that matches the sampling scheme the training code actually runs.

The running example

Every chapter that can compute it reports the same number: the ε at $\delta = 10^{-5}$ of DP-SGD with Poisson sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and up to $k = 10{,}000$ steps. That is about one hundred epochs at a typical setting. A single Gaussian mechanism with $\sigma = 1$ serves as the small example. Watching the same number move from chapter to chapter shows what each tool gains. Choosing an accountant puts all the values side by side.

How to read this part

Order. Read chapters 27 and 28 first: every later chapter uses the RDP curve and its conversion. Chapter 28 ends by reading the curve as a constraint on tests, and chapter 30 develops that trade-off view in full, so read it next. Chapters 31 and 32 form the practical core and should be read in order. Chapter 29 can be read any time after chapter 28, except its section on subsampling, which uses one formula from chapter 31; read it before chapter 34, which uses its conversion. Chapter 33 needs chapter 31. Chapter 34 needs chapters 29 and 31. Chapter 35 closes the part and uses all the others, so read it last.

Structure. The chapters follow the shape of the earlier parts: why the topic matters, a toolkit, one section per result with a derivation, interpretation, worked example and two check-yourself questions, then code, references and a cheat sheet. Exercises with worked solutions are collected in the exercise book, a companion website, with a link at the end of each chapter.

Run the code. Each chapter has a script in `code/` that prints every number its text quotes, except values attributed to an external library or paper, and regenerates its figures, for example

```
uv run python code/part04-privacy-accounting/renyi_dp.py
```

None of the scripts uses an accounting library. Each accountant is implemented from its definition in a few dozen lines of NumPy, so you can read exactly what it computes. As an exercise, compare them with Google's `dp_accounting` or Opacus.

Notation

The part uses the notation of the DP foundations part, extended in Table 26.9.

Table 26.9.: Notation used throughout this part, in addition to that of the earlier parts.

Symbol	Meaning
M, $D \simeq D'$	A mechanism; neighbouring datasets (add/remove-one unless replace-one is named)
y, $P = M(D)$, $Q = M(D')$	An output; the two output distributions being compared

Symbol	Meaning
$D' = D \cup \{x\}$; a, b	As in the DP foundations part, under add/remove D' is the dataset that holds the record x and D the one without it. A test of P against Q has type I error a, the false-positive rate of saying D' when the data were D, and type II error b, the rate of saying D when they were D'. A guarantee covers both orders of the pair; a chapter that needs one order says so, as the subsampling chapters do for the remove direction, which puts the dataset with the record first
$L_{D,D'}(y) = \ln \frac{p_{M(D)}(y)}{p_{M(D')}(y)}$	The privacy loss of output y; $L_{D,D'}(Y)$ with $Y \sim M(D)$ is the privacy loss random variable
$\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q)$	The smallest δ for a given ε, as a hockey-stick divergence
D_α, $\varepsilon(\alpha)$	Rényi divergence of order $\alpha > 1$; the RDP curve of a mechanism
ρ	The zCDP parameter
$T(P, Q)$, G_μ, μ	Trade-off function; the Gaussian trade-off function $T(\mathcal{N}(0,1), \mathcal{N}(\mu,1))$; the parameter of Gaussian differential privacy, so a Gaussian mechanism with noise multiplier σ has $\mu = 1/\sigma$
$\Delta_2 f$, σ	ℓ_2 sensitivity; noise multiplier (noise standard deviation per unit of sensitivity)
q, k	Sampling rate; number of composed steps
h	Grid width of a discretized privacy loss distribution

Two symbols do double duty, and each chapter warns you when it happens. α is the Rényi order throughout this part, but the f-DP literature uses it for the type I error, so f-DP and Gaussian differential privacy renames the errors. q is the sampling rate, so the chapters that subsample write output densities with capital letters, such as $Q(y)$ or $P_q(y)$, never $q(y)$.

What you will be able to do

By the end of this part you should be able to:

- compute the RDP curve of the Gaussian, Laplace and randomized-response mechanisms, compose them, and convert the result to (ε, δ) with both the classic and the improved conversion;
- explain what an RDP curve, a zCDP parameter and a trade-off function each keep and lose about the privacy loss;
- derive privacy amplification by subsampling, and compute the RDP of the Poisson-subsampled Gaussian;
- build a privacy-loss-distribution accountant with pessimistic and optimistic error bounds, and use it to check any other accountant;
- say when shuffling or hiding intermediate iterates gives extra privacy, and when it does not apply;
- run a privacy filter, and compute per-record privacy spend in DP-SGD;
- pick an accountant for a DP-SGD run, state its assumptions, and report ε in a way that someone else can check.

27. Rényi differential privacy

27.1. Why this matters

You train a model with 1000 steps of full-batch noisy gradient descent. Each step computes a quantity whose **sensitivity** is 1, meaning that changing one person's record moves it by at most 1, and releases it with Gaussian noise whose standard deviation is $\sigma = 20$ times that sensitivity; σ is the **noise multiplier**. What is the total differential privacy (DP) guarantee at $\delta = 10^{-5}$? Recall what such a guarantee says: a mechanism is (ε, δ)-DP if, for any two neighbouring datasets (datasets that differ in one person's record) and any set of outputs, the probability of that set under one dataset is at most e^{ε} times its probability under the other, plus a small slack δ.

Part 3 gives one answer. Turn each step into a single guarantee $(\varepsilon_0, \delta_0)$, then apply advanced composition (Composition), the theorem that says k such steps cost roughly $\varepsilon_0\sqrt{k}$ rather than $k\varepsilon_0$. With the best split of δ between the steps and the final tail, over the grid of splits Part 3 searches, that gives $\varepsilon = 68.01$. The true answer is $\varepsilon = 7.51$. You can compute it exactly because 1000 Gaussian steps behave like one Gaussian mechanism with noise $\sigma/\sqrt{1000}$: the 1000 shifts of 1 form a vector of Euclidean length $\sqrt{1000}$, and a Gaussian with shift $\sqrt{1000}$ and noise σ is the same as one with shift 1 and noise $\sigma/\sqrt{1000}$. The bound is nine times too large ($68.01/7.51 \approx 9$).

The loss comes from the very first move. A Gaussian step does not have one $(\varepsilon_0, \delta_0)$. For each ε_0 there is a smallest δ_0 that works, so it has a whole curve of valid pairs, its **privacy curve**. Collapsing the curve to a single point throws away the information about how the step behaves at every other point, and that is exactly the information composition needs. After that, no composition theorem can recover it.

27.1.1. The one idea: track a curve that adds

Rényi differential privacy (RDP) keeps a whole curve instead of a point, and it chooses a curve that is easy to compose. For each mechanism and each pair of neighbouring datasets, consider how much more likely an output is under one dataset than under the other. RDP summarizes that likelihood ratio by its moments: for each **order** $\alpha > 1$ it records the Rényi divergence D_α between the two output distributions, which is, up to a logarithm and a factor $1/(\alpha - 1)$, the average of the likelihood ratio raised to the power α (the Toolkit below restates it). Taking the worst pair of neighbours at each order gives a function $\alpha \mapsto \varepsilon(\alpha)$, the **RDP curve**.

Two facts make this curve the workhorse of privacy accounting. First, curves add under composition. When you run several mechanisms in sequence, even when each is chosen after seeing the earlier outputs, the curve of the whole run is at most the pointwise sum of their curves, with equality for independent steps. The reason is that a moment of a product of independent likelihood ratios is a product of moments, and the logarithm turns that product into a sum. No δ is spent along the way. Second, any point of the curve converts to an (ε, δ) guarantee at the end. The conversion

is a Chernoff bound, Markov's inequality applied to an exponential, which turns a bound on a moment of the privacy loss into a bound on how often the loss is large. So you compose curves, and choose the best order once, after composing.

For the 1000 Gaussian steps, adding curves and converting once gives $\varepsilon = 8.84$, 18% above the exact 7.51 ($8.84/7.51 = 1.177$). The same bookkeeping, under the name *moments accountant*, is what made differentially private deep learning practical.

Table 27.1 lists the results of this chapter and where each is used.

Table 27.1.: The results of this chapter and where each is used.

Result	Where it is used
Definition of (α, ε)-RDP; limits $\alpha \to 1$ and $\alpha \to \infty$	Every accountant in this part; ε-DP is the point $\alpha = \infty$
RDP curves of the Gaussian, Laplace and randomized response mechanisms	The inputs to every accountant; the Gaussian curve is the base of accounting for differentially private stochastic gradient descent (SGD), DP-SGD for short
Composition adds curves	Accounting for thousands of training steps in one line
Post-processing; weak group privacy	Releasing models and predictions; correlated records
The moments accountant is RDP	Reading Abadi et al. (2016) and the libraries built on it
Conversion RDP $\Rightarrow (\varepsilon, \delta)$	Reporting a guarantee; From RDP to (ε, δ) and to tests sharpens it

What you need. The Rényi divergence, its limits, its monotonicity in α and the data-processing inequality are in Divergences (Section 3.4, Section 3.10). The Chernoff method is in Concentration. From Part 3 you need the definition of DP (Definition), the Laplace and Gaussian mechanisms (Laplace and Gaussian mechanisms), randomized response (Definition, Section 18.5) and the privacy loss random variable (Composition). The Toolkit restates the facts this chapter uses, and each mechanism is restated where it first appears.

Notation. M is a mechanism and $D \simeq D'$ are neighbouring datasets. $P = M(D)$ and $Q = M(D')$ are the output distributions, with densities (or probability mass functions) p and q. The **privacy loss** at output y is $L(y) = \ln \frac{p(y)}{q(y)}$, the log of how much more likely y is under D than under D': positive when y points to D, negative when it points to D'. The **privacy loss random variable** is $L(Y)$ with $Y \sim P$, the loss of an output actually drawn from the mechanism on D. The Gaussian mechanism adds $\mathcal{N}(0, \sigma^2\Delta_2^2)$ noise to a query of ℓ_2-sensitivity Δ_2 (the largest Euclidean distance between the query's answers on neighbours), so σ is the noise multiplier. Logarithms are natural. Whenever this part optimizes over the order, it uses the same grid of orders: 99 orders from 1.1 to 10.9 in steps of 0.1, the 52 integers from 12 to 63, and 128, 256 and 512, which is $99 + 52 + 3 = 154$ orders from 1.1 to 512:

```
ORDERS = [1 + x / 10 for x in range(1, 100)] + list(range(12, 64)) + [128, 256, 512]
```

27.2. Toolkit: four facts from Part 1

Every derivation below reuses four facts about the Rényi divergence of order $\alpha > 1$. They are proved in Divergences and Concentration; each is restated here with the short reason it holds. Throughout, $r = p/q$ is the **likelihood ratio**, how many times more likely an output is under P than under Q, and $L = \ln r$ is the privacy loss.

27.2.1. Fact 1 — A moment of the likelihood ratio

The Rényi divergence of order α is defined as

$$D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \int p^\alpha q^{1-\alpha}\, dy. \tag{27.1}$$

The integral inside is an average of a power of the likelihood ratio, and you can take that average under either distribution. Multiply both sides of the definition by $\alpha - 1$ and exponentiate:

$$e^{(\alpha-1)D_\alpha(P\|Q)} = \int p^\alpha q^{1-\alpha}\, dy = \mathbb{E}_Q[r^\alpha] = \mathbb{E}_P[r^{\alpha-1}] = \mathbb{E}_P[e^{(\alpha-1)L}]. \tag{27.2}$$

The first equality after the integral holds because $p^\alpha q^{1-\alpha} = q\,(p/q)^\alpha$, and integrating q times something is averaging that something under Q. The next one is the **change of measure** $\mathbb{E}_P[g] = \mathbb{E}_Q[rg]$, which holds because $\int p\, g = \int q\,(p/q)\, g$: an average under P is an average under Q with each output reweighted by its likelihood ratio. Applied with $g = r^{\alpha-1}$, it turns $\mathbb{E}_Q[r \cdot r^{\alpha-1}]$ into $\mathbb{E}_P[r^{\alpha-1}]$. The last form uses $r^{\alpha-1} = e^{(\alpha-1)L}$. It says that $(\alpha-1)D_\alpha$ is the logarithm of the **moment generating function** (MGF) of L under P, the function $\lambda \mapsto \mathbb{E}_P[e^{\lambda L}]$, evaluated at $\lambda = \alpha - 1$ (Equation 3.12). That is why a Rényi divergence is a statement about the exponential moments of the privacy loss.

27.2.2. Fact 2 — Limits and monotonicity

D_α is non-decreasing in α. As $\alpha \to 1$ it tends to the Kullback–Leibler (KL) divergence $\mathbb{E}_P[L]$, the average privacy loss, and as $\alpha \to \infty$ it tends to the **max-divergence** $D_\infty = \sup_y L(y)$, the largest privacy loss of any output.

The reason is a picture. Write $\psi(\lambda) = \ln \mathbb{E}_P[e^{\lambda L}]$, the log-MGF of Fact 1. It passes through the origin, since $\psi(0) = \ln 1 = 0$, and it is convex, as every log-MGF is. By Fact 1, $D_\alpha = \psi(\alpha-1)/(\alpha-1)$, the slope of the chord of ψ from the origin to the point at $\lambda = \alpha - 1$. Chords of a convex function from a fixed point get steeper as their far end moves right, so D_α grows with α. As $\alpha \to 1$ the chord shrinks to the tangent at 0, whose slope is $\psi'(0) = \mathbb{E}_P[L]$, the KL divergence. As $\alpha \to \infty$, D_α is the log of $(\mathbb{E}_P[r^{\alpha-1}])^{1/(\alpha-1)}$, a power mean of r with a growing exponent, and such a mean is pulled toward the largest value of r; its logarithm tends to $\sup_y L(y)$.

27.2.3. Fact 3 — Data processing

Processing the output cannot make two distributions easier to tell apart. For any **channel** K, a randomized map applied to the output that does not depend on which distribution produced it, $D_\alpha(PK\|QK) \le D_\alpha(P\|Q)$, for every α including ∞ (Section 3.10). Here PK is the distribution of the processed output when the input is drawn from P. The idea is that the likelihood ratio of a processed output is an average of the ratios of the original outputs that could have produced it, and averaging pulls ratios toward each other. Since $x \mapsto x^\alpha$ is convex for $\alpha > 1$, Jensen's inequality (the power of an average is at most the average of the powers) says that averaging can only lower the moment $\mathbb{E}_Q[r^\alpha]$, and with it D_α.

27.2.4. Fact 4 — Markov on an exponential (the Chernoff method)

For any random variable X, any c and any $\lambda > 0$,

$$\Pr[X \ge c] \le e^{-\lambda c}\,\mathbb{E}\big[e^{\lambda X}\big]. \tag{27.3}$$

The reason is one line. On the event $X \ge c$ the non-negative quantity $e^{\lambda X}$ is at least $e^{\lambda c}$, so its average is at least $e^{\lambda c}\Pr[X \ge c]$; divide by $e^{\lambda c}$. This is how an exponential moment, which is what a Rényi divergence controls, becomes a bound on a tail probability.

27.3. Rényi differential privacy

27.3.1. Context

ε-DP bounds the privacy loss $L(y)$ at *every* output. That is a statement about the maximum of L, which is the max-divergence. (ε, δ)-DP allows the bound to be exceeded on outputs carrying a small amount of probability mass, measured by δ. Both summarize the distribution of the privacy loss by one point of its upper tail. The Rényi divergence of order α summarizes it differently, by an exponential moment $\mathbb{E}_P[e^{(\alpha-1)L}]$ (Fact 1). An exponential moment weights large losses more heavily than small ones, and the larger the order, the more heavily. So small orders look at the typical loss and large orders at the worst case (Fact 2). Requiring a bound at every order describes the whole upper tail of the loss, not one point of it.

27.3.2. Statement

The definition asks for the same bound on the Rényi divergence for every pair of neighbours. A mechanism M is (α, ε)-**RDP**, for an order $\alpha > 1$, if for all neighbouring datasets $D \simeq D'$

$$D_\alpha(M(D)\,\|\,M(D')) \le \varepsilon. \tag{27.4}$$

The **RDP curve** of M records, at each order, the smallest ε that works, which is the divergence of the worst pair of neighbours:

$$\varepsilon_M(\alpha) = \sup_{D \simeq D'} D_\alpha(M(D)\,\|\,M(D')). \tag{27.5}$$

The curve has three properties, which tie its two ends to familiar quantities.

1. $\alpha = \infty$ **is pure DP.** M is ε-DP if and only if $\varepsilon_M(\infty) \le \varepsilon$.
2. **Monotone.** $\varepsilon_M(\alpha)$ is non-decreasing in α. So an ε-DP mechanism is (α, ε)-RDP at every order.
3. $\alpha \to 1$ **is KL.** As $\alpha \to 1$, $\varepsilon_M(\alpha)$ tends to a limit at least the largest KL divergence between neighbours, with equality for every mechanism in this chapter.

27.3.3. Derivation

1. **Pure DP is the max-divergence.** ε-DP says $P(S) \le e^{\varepsilon} Q(S)$ for every event S, that is, $\ln \frac{P(S)}{Q(S)} \le \varepsilon$. The largest value of that log ratio over events equals the largest privacy loss of a single output, which is the event form of the max-divergence (Section 3.4). The reason: $\frac{P(S)}{Q(S)} = \frac{\int_S q\, r}{\int_S q}$ is an average of the likelihood ratio r over the outputs in S, weighted by q, so it can never exceed the largest r, and an event concentrated on the outputs where r is largest comes as close to it as you like. So $\sup_S \ln \frac{P(S)}{Q(S)} = D_\infty(P\|Q)$, and ε-DP holds exactly when $D_\infty(M(D)\|M(D')) \le \varepsilon$ for all neighbours, that is, when $\varepsilon_M(\infty) \le \varepsilon$.
2. **Monotonicity passes through the supremum.** For each pair, D_α is non-decreasing in α (Fact 2). A supremum of non-decreasing functions is non-decreasing, because if every function in the family is larger at α' than at α, so is the largest of them. Combined with step 1, $\varepsilon_M(\alpha) \le \varepsilon_M(\infty) \le \varepsilon$ for an ε-DP mechanism.
3. **The KL limit.** For each pair, $D_\alpha \ge \mathrm{KL}$ and $D_\alpha \to \mathrm{KL}$ as $\alpha \to 1$ (Fact 2). So $\varepsilon_M(\alpha) \ge \sup_{D \simeq D'} \mathrm{KL}$ at every order, and the limit is at least the largest KL. Equality would need the limit and the supremum to commute, which can fail in general: each pair's divergence could approach its KL at a different speed. For the mechanisms below the curves are explicit, and you can check the limit directly.

27.3.4. Interpretation

An RDP guarantee is a curve, not a number. "Randomized response with $p = 0.75$ is $(2, 0.8473)$-RDP" (from the example below) is one point of it. What accountants store is the curve on a grid of orders. Each point is a valid guarantee, and different orders serve different purposes: composition works at every order at once, and the conversion to (ε, δ) picks the best order at the end.

The curve sits between the average case and the worst case. As $\alpha \to 1$ it sees only the average loss, the KL divergence. A mechanism whose loss is usually small but occasionally large has a small KL divergence, because the rare large losses barely move the average, but large high-order divergences, because a high exponential moment is dominated by those rare large losses. So it is caught at large orders. Pure DP, at $\alpha = \infty$, is the most demanding point of the curve.

Why only $\alpha > 1$? Orders below 1 exist as divergences, but they are not used for privacy. The conversion to (ε, δ) applies the Chernoff method to the loss, which needs an exponential moment $\mathbb{E}_P[e^{\lambda L}]$ with $\lambda = \alpha - 1 > 0$, and only orders above 1 provide one (Section 27.8).

27.3.5. Example: randomized response, outcome by outcome

Randomized response reports one person's bit truthfully with probability $p > \frac{1}{2}$, the keep-probability, and flips it otherwise. The two neighbouring inputs are the two values of the bit. Take $p = 0.75$. The likelihood ratio of any output is $0.75/0.25 = 3$ or its inverse, so the mechanism is ε-DP with $\varepsilon = \ln \frac{p}{1-p} = \ln 3$. Table 27.2 lists the outputs when the true bit is 1 (P) and 0 (Q).

Table 27.2.: Randomized response with keep-probability $p = 0.75$. With an argument, $p(y)$ and $q(y)$ are output probabilities; p alone is the keep-probability. The last column gives the terms of $\int p^2 q^{-1}$, the order-2 moment of Fact 1.

Output y	$p(y)$ (bit is 1)	$q(y)$ (bit is 0)	$r(y)$	$L(y)$	$p(y)^2/q(y)$
1	0.75	0.25	3	1.0986	2.2500
0	0.25	0.75	1/3	−1.0986	0.0833

At order 2, Fact 1 says $e^{(2-1)D_2}$ is the sum of $p(y)^2 q(y)^{-1}$ over the outputs, which is the last column: $e^{D_2} = 2.2500 + 0.0833 = 2.3333$, so $\varepsilon(2) = \ln 2.3333 = 0.8473$. The same recipe works at any order: sum $p(y)^\alpha q(y)^{1-\alpha}$ over the two outputs, take the logarithm, and divide by $\alpha - 1$. The output 1 contributes $p^\alpha (1-p)^{1-\alpha}$ and the output 0 contributes $(1-p)^\alpha p^{1-\alpha}$, so

$$\varepsilon(\alpha) = \frac{1}{\alpha - 1} \ln \Big(p^\alpha (1-p)^{1-\alpha} + (1-p)^\alpha p^{1-\alpha} \Big). \tag{27.6}$$

Swapping the roles of the bits swaps the two rows of the table and gives the same sum, so both directions of the pair have the same curve. Table 27.3 gives it at a few orders.

Table 27.3.: The RDP curve of randomized response with $p = 0.75$.

α	1 (KL)	1.5	2	5	10	64	∞
$\varepsilon(\alpha)$	0.5493	0.7340	0.8473	1.0267	1.0666	1.0940	1.0986

The curve rises from the KL value to the pure DP level (Figure 27.1). The KL value is the average loss under P: the loss is $\ln 3$ with probability 0.75 and $-\ln 3$ with probability 0.25, so $\mathbb{E}_P[L] = (0.75 - 0.25) \ln 3 = 0.5 \ln 3 = 0.5493$. The pure DP level is the largest loss, $\ln 3 = 1.0986$. The script also evaluates the formula at $\alpha = 1.0001$ (0.5494) and $\alpha = 1000$ (1.0983), approaching both ends. Randomized response is already a pure DP mechanism, so here the curve says little that $\ln 3$ did not. The curve matters most for mechanisms whose D_∞ is large or infinite.

27.3.6. Check yourself

- A mechanism is (α, ε)-RDP. Is it (α', ε)-RDP for $\alpha' < \alpha$? For $\alpha' > \alpha$? *(For $\alpha' < \alpha$ yes, by monotonicity: the curve at α' is at most the curve at α. For $\alpha' > \alpha$ not in general: the curve can keep rising, as the Gaussian's does.)*
- Why is $\varepsilon(\alpha)$ for randomized response the same in both directions of the pair? *(Swapping the bit swaps $p(y)$ and $q(y)$ between the two outputs, and the sum $p^\alpha q^{1-\alpha}$ over both outputs is unchanged.)*

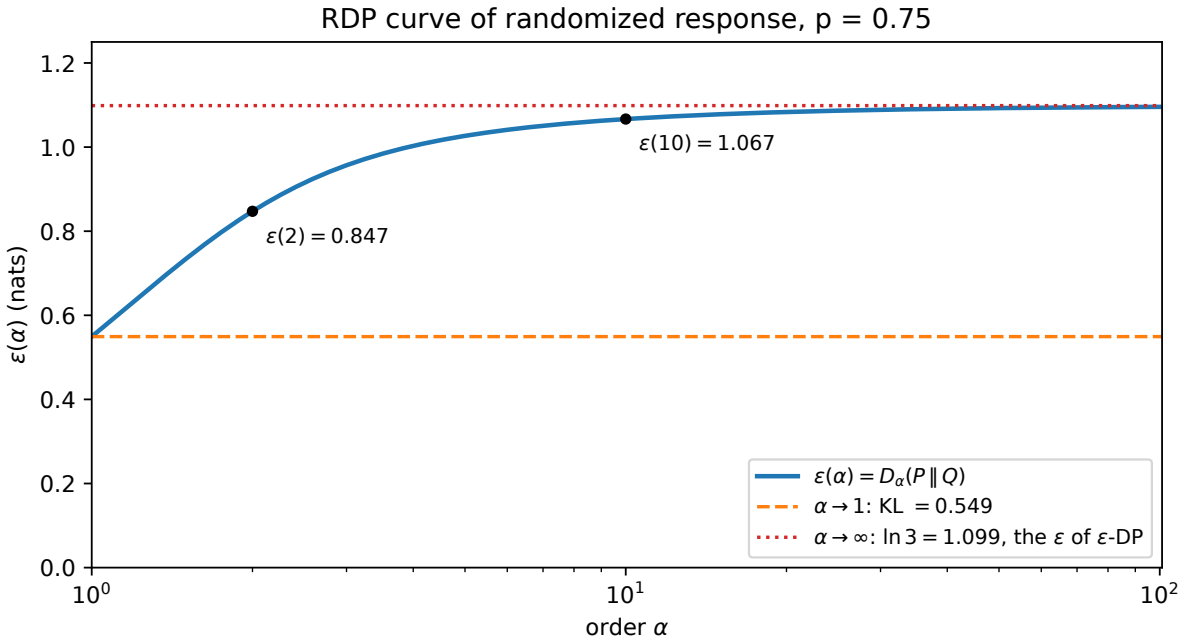


Figure 27.1.: The RDP curve $\varepsilon(\alpha)$ of randomized response with $p = 0.75$, on a log scale in α. It starts at the KL divergence 0.549 as $\alpha \to 1$ and rises to the pure DP level $\ln 3 = 1.099$ as $\alpha \to \infty$.

27.4. RDP curves of the basic mechanisms

27.4.1. Context

To account for a mechanism you need its curve. For a mechanism that adds noise to a query's answer, the two outputs on neighbours are the same noise distribution centred at two different answers. So the curve comes from the Rényi divergence between the noise distribution and a shifted copy of itself, which Part 1 computed for the normal and the Laplace distributions. The only new step is the supremum over neighbours: you need to know which pair of neighbouring inputs is the worst, and for these mechanisms it is the pair whose answers are furthest apart.

27.4.2. Statement

With sensitivity 1, the three curves are as follows. The Gaussian mechanism adds normal noise with standard deviation σ (the noise multiplier). The **Laplace mechanism** adds noise from the Laplace distribution with scale b, whose density $\frac{1}{2b}e^{-|y|/b}$ falls off exponentially on both sides. Randomized response has keep-probability p. The Gaussian curve is a straight line in α, the Laplace curve is the Part 1 formula for a shift of one scale unit $1/b$, and the randomized response curve is the sum over two outputs from the previous section:

$$\text{Gaussian:} \quad \varepsilon(\alpha) = \frac{\alpha}{2\sigma^2}, \tag{27.7}$$

$$\text{Laplace:}\quad \varepsilon(\alpha) = \frac{1}{\alpha-1}\ln\left(\frac{\alpha}{2\alpha-1}e^{(\alpha-1)/b} + \frac{\alpha-1}{2\alpha-1}e^{-\alpha/b}\right), \tag{27.8}$$

$$\text{randomized response:}\quad \varepsilon(\alpha) = \frac{1}{\alpha-1}\ln\left(p^\alpha(1-p)^{1-\alpha} + (1-p)^\alpha p^{1-\alpha}\right). \tag{27.9}$$

These are Corollaries 3 and 2 and the randomized response entry of Table 2 in Mironov (2017). The Laplace formula is for a real-valued query.

27.4.3. Derivation

Gaussian. The plan is to reduce any pair of neighbours to two one-dimensional normals a distance s apart, use the Part 1 formula, and then take the largest s.

1. *One pair.* With $\Delta_2 = 1$ the noise is $\mathcal{N}(0, \sigma^2 I)$. On neighbours with answers $f(D)$ and $f(D')$, the outputs are $\mathcal{N}(f(D), \sigma^2 I)$ and $\mathcal{N}(f(D'), \sigma^2 I)$.
2. *Divergence of the pair.* Translating and rotating the output space is a bijection, a relabelling of the outputs that can be undone, so it changes no divergence (apply Fact 3 in both directions). So you can move $f(D)$ to 0 and $f(D') - f(D)$ onto the first axis. The noise is the same in every direction, so after the rotation the two distributions differ only in the first coordinate; the other coordinates are identical, independent normals under both. Rényi divergence is additive over independent coordinates (Section 3.4), because the moment of Fact 1 factorizes into a product over coordinates, and identical coordinates contribute 0. What remains is the one-dimensional divergence $D_\alpha(\mathcal{N}(0,\sigma^2)\|\mathcal{N}(s,\sigma^2)) = \alpha s^2/(2\sigma^2)$ (Equation 3.15), with $s = \|f(D) - f(D')\|_2$. Part 1 derives it by noting that the privacy loss of a normal pair is itself normal, with mean $s^2/(2\sigma^2)$ and variance s^2/σ^2, and a normal variable's log-MGF is known in closed form.
3. *Supremum.* The divergence grows with s, and $s \le \Delta_2 = 1$ for neighbours. So the worst pair has $s = 1$ and gives Equation 27.7. The script also integrates $p^\alpha q^{1-\alpha}$ numerically and gets 1.0000 at $\alpha = 2$ and 5.0000 at $\alpha = 10$ for $\sigma = 1$, matching $2/2$ and $10/2$.

Laplace. The plan is the same: reduce to a shift, use Part 1's closed form, and show the worst shift is the largest.

1. *One pair.* The outputs are $\mathrm{Lap}(f(D), b)$ and $\mathrm{Lap}(f(D'), b)$. After a translation, this is $\mathrm{Lap}(0, b)$ against $\mathrm{Lap}(s, b)$ with $s = |f(D) - f(D')| \le 1$.
2. *Divergence of the pair.* Part 1 computes it by splitting the integral $\int p^\alpha q^{1-\alpha}$ at the two kinks of the densities, 0 and s, where the absolute values change sign; on each of the three pieces the integrand is a plain exponential (Equation 3.19). With $t = s/b$, the shift in units of the scale, the result is the formula above with $1/b$ replaced by t.
3. *Supremum.* You need the divergence to grow with the shift, so that the largest shift is the worst. Write the argument of the logarithm as $g(t) = \frac{\alpha}{2\alpha-1}e^{(\alpha-1)t} + \frac{\alpha-1}{2\alpha-1}e^{-\alpha t}$. Its derivative is $g'(t) = \frac{\alpha(\alpha-1)}{2\alpha-1}(e^{(\alpha-1)t} - e^{-\alpha t})$, which is positive for $t > 0$ because the first exponent is positive and the second negative. So $\ln g(t)/(\alpha-1)$ increases with t. The reverse direction, $\mathrm{Lap}(s, b)$ against $\mathrm{Lap}(0, b)$, gives the same value, because the reflection $y \mapsto s - y$ swaps the two distributions and changes no divergence. So the worst pair has $s = 1$, $t = 1/b$, and gives Equation 27.8.
4. *Numerical check.* The script integrates $p^\alpha q^{1-\alpha}$ on a grid of step 10^{-4}, with both kinks as grid points. For $b \in \{0.5, 1, 4\}$ and $\alpha \in \{1.5, 2, 10, 64\}$, the largest difference from Equation 27.8 is 1.3×10^{-8}.

Randomized response. There is only one pair of inputs, the two values of the bit, so no supremum is needed, and Table 27.2 is its distribution. Fact 1 sums $p^\alpha q^{1-\alpha}$ over the two outputs, as in the previous section.

27.4.4. Interpretation

Figure 27.2 draws the three curves. Two shapes appear, and which one you get depends on whether the privacy loss is bounded.

When the privacy loss is bounded, the curve levels off. For randomized response the loss is $\pm \ln 3$. For Laplace the loss at y is $(|y-1| - |y|)/b$, and by the triangle inequality $\big||y-1| - |y|\big| \le 1$, so $|L| \le 1/b$. Since every D_α is at most D_∞, the largest loss (Fact 2), the curves rise towards $1/b$ and $\ln 3$ and stay below them. For large α you can see the approach. The first term of Equation 27.8 dominates, because $e^{(\alpha-1)/b}$ grows while $e^{-\alpha/b}$ shrinks. Keeping only it, the logarithm splits as $\ln \frac{\alpha}{2\alpha-1} + \frac{\alpha-1}{b}$, and dividing by $\alpha - 1$ gives $\varepsilon(\alpha) \approx \frac{1}{b} + \frac{1}{\alpha-1} \ln \frac{\alpha}{2\alpha-1}$. As α grows, $\frac{\alpha}{2\alpha-1}$ tends to $\frac{1}{2}$, so $\varepsilon(\alpha) \approx \frac{1}{b} - \frac{\ln 2}{\alpha-1}$: the curve approaches $1/b$ from below at rate $1/(\alpha-1)$. At $\alpha = 512$ and $b = 1$, both the formula and this approximation give 0.9986.

When the privacy loss is unbounded, the curve grows linearly. The Gaussian privacy loss is a straight line in the output y (the squares in the two exponents cancel, leaving a linear term), so it has no maximum, and the curve $\alpha/(2\sigma^2)$ grows without bound. The Gaussian mechanism is therefore not ε-DP for any ε. It is still (α, ε)-RDP at every finite order, and that is all composition and conversion need.

The straight-line shape of the Gaussian curve has a meaning of its own. For a single pair, a curve of the form $\rho\alpha$ means the privacy loss is exactly normal, $\mathcal{N}(\rho, 2\rho)$. The reason is Fact 1: $(\alpha-1)\varepsilon(\alpha)$ is the log-MGF of the loss at $\lambda = \alpha - 1$, and with $\varepsilon = \rho\alpha = \rho(\lambda+1)$ it is $\rho\lambda + \rho\lambda^2$. The log-MGF of a normal variable with mean m and variance v is $m\lambda + v\lambda^2/2$, so this one has $m = \rho$ and $v = 2\rho$. Concentrated DP builds a definition on it.

27.4.5. Example: the three curves at a few orders

Table 27.4 gives the curves at four orders, from the script.

Table 27.4.: RDP curves of the three basic mechanisms with sensitivity 1.

Mechanism	$\varepsilon(1.5)$	$\varepsilon(2)$	$\varepsilon(10)$	$\varepsilon(64)$	$\varepsilon(\infty)$
Gaussian, $\sigma = 1$	0.7500	1.0000	5.0000	32.0000	∞
Laplace, $b = 1$	0.5129	0.6191	0.9287	0.9891	1
Randomized response, $p = 0.75$	0.7340	0.8473	1.0666	1.0940	1.0986

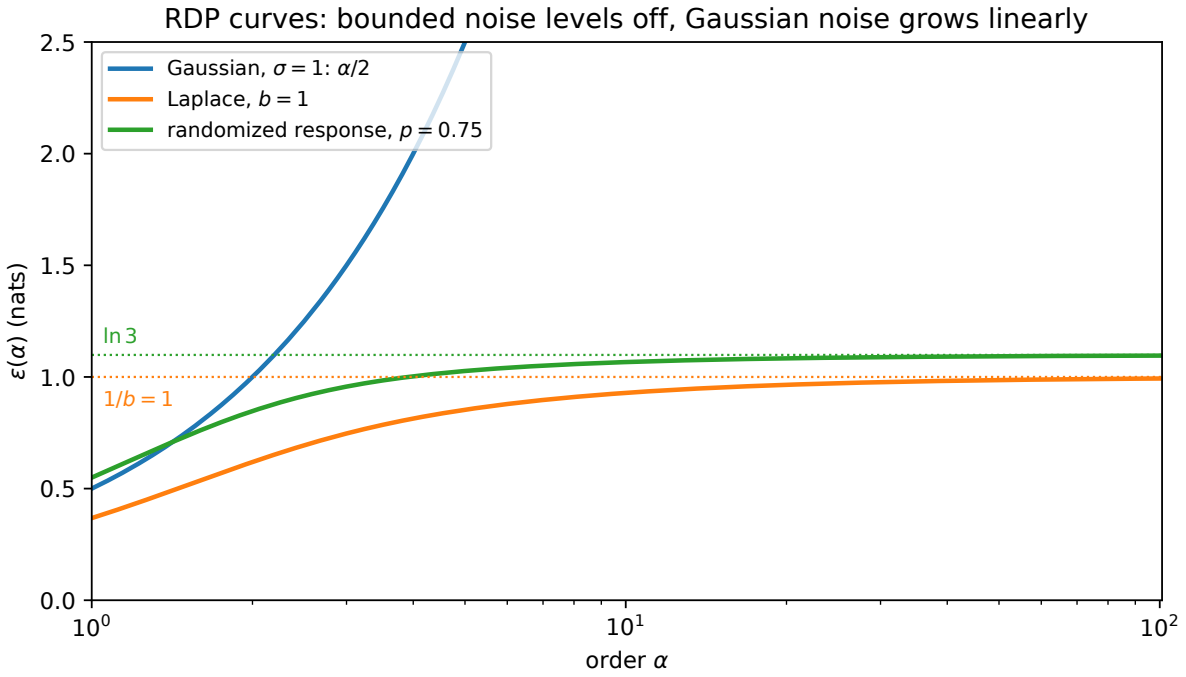


Figure 27.2.: RDP curves of the Gaussian mechanism ($\sigma = 1$), the Laplace mechanism ($b = 1$) and randomized response ($p = 0.75$), all with sensitivity 1, on a log scale in α (so the straight line $\alpha/2$ looks curved). The Laplace and randomized response curves level off at their pure DP levels $1/b$ and $\ln 3$ (dotted). The Gaussian curve $\alpha/2$ grows linearly.

The Gaussian row is $\alpha/2$, read off at each order. You can check the Laplace row by hand at order 2: the coefficients in Equation 27.8 are $\frac{2}{3}$ and $\frac{1}{3}$, the prefactor $1/(\alpha-1)$ is 1, and with $b=1$ the curve is $\ln\left(\frac{2}{3}e+\frac{1}{3}e^{-2}\right)=0.6191$, the same number as in Part 1's Laplace example. As $\alpha\to 1$ the Laplace curve tends to the KL divergence $1/b+e^{-1/b}-1=0.3679$. You can see this from Equation 27.8: at $\alpha=1$ the argument of the logarithm is 1, and its derivative in α there is $\frac{1}{b}-1+e^{-1/b}$, so for α near 1 the logarithm is about $(\alpha-1)$ times that derivative, and dividing by $\alpha-1$ leaves the derivative. At $b=1$ it is $1+e^{-1}-1=e^{-1}=0.3679$.

27.4.6. Check yourself

- What is the RDP curve of the Gaussian mechanism with $\sigma=2$ and sensitivity 1, and at which order is it 1? *($\varepsilon(\alpha)=\alpha/(2\cdot 2^2)=\alpha/8$, which equals 1 at $\alpha=8$.)*
- For the Gaussian mechanism on a vector query, why does only the ℓ_2 sensitivity matter, and not the ℓ_1? *(The divergence between two isotropic normals depends only on the Euclidean distance between their means, by rotation invariance.)*

27.5. Composition adds curves

27.5.1. Context

Training a model runs thousands of mechanisms, each chosen after seeing the outputs of the earlier ones. You need the curve of the whole run. For a product of independent distributions, Part 1 showed that Rényi divergences add: the moment of Fact 1 is an integral of a product, which splits into a product of integrals, and the logarithm turns the product into a sum. Adaptive composition is not a product of independent distributions, because the second mechanism depends on the first output. The chain rule below shows that the curves still add, as long as each step's curve is bounded whatever the earlier outputs were.

27.5.2. Statement

Let M_1 have RDP curve ε_1. Let $M_2(\cdot\,;y_1)$ be a mechanism chosen using the first output y_1, whose RDP curve is at most ε_2 for every y_1. The **adaptive composition** releases both outputs, $M(D)=(y_1,y_2)$, with $y_1\sim M_1(D)$ and $y_2\sim M_2(D;y_1)$. Its curve is at most the sum of the two curves:

$$\varepsilon_M(\alpha)\le\varepsilon_1(\alpha)+\varepsilon_2(\alpha)\qquad\text{for every }\alpha>1. \tag{27.10}$$

By induction, treating the first $k-1$ steps as one mechanism, k adaptively chosen mechanisms have curve at most $\sum_{i=1}^{k}\varepsilon_i(\alpha)$. This is Mironov (2017), Proposition 1.

27.5.3. Derivation

The idea is to write the order-α moment of the pair of outputs as a double integral, bound the inner integral (over y_2) by the worst case of the second step, and recognise what is left as the moment of the first step. Fix neighbours $D\simeq D'$ and an order $\alpha>1$. Write p_1,q_1 for the densities of y_1 under D and D', and $p_2(\cdot\mid y_1),q_2(\cdot\mid y_1)$ for those of y_2 given y_1.

1. **Factor the joint densities.** By the definition of the adaptive composition, y_1 is drawn first and y_2 is drawn given y_1, so $p(y_1, y_2) = p_1(y_1)\, p_2(y_2 \mid y_1)$, and the same for q.
2. **Write the order-α moment as a double integral** (Fact 1):

$$e^{(\alpha-1)D_\alpha(P\|Q)} = \iint p_1^\alpha q_1^{1-\alpha}\, p_2(y_2 \mid y_1)^\alpha q_2(y_2 \mid y_1)^{1-\alpha}\, dy_2\, dy_1. \tag{27.11}$$

   The integrand splits into a factor for y_1 and a factor for y_2 given y_1, because a power of a product is the product of the powers.
3. **Do the inner integral first, with y_1 fixed.** It is $e^{(\alpha-1)D_\alpha(P_2(\cdot|y_1)\|Q_2(\cdot|y_1))}$, by Fact 1 again. For fixed y_1, $P_2(\cdot \mid y_1)$ and $Q_2(\cdot \mid y_1)$ are the outputs of the mechanism $M_2(\cdot\,; y_1)$ on D and D', so the divergence is at most $\varepsilon_2(\alpha)$. Since $\alpha - 1 > 0$, the exponential is increasing, and the inner integral is at most $e^{(\alpha-1)\varepsilon_2(\alpha)}$, whatever y_1 is.
4. **The outer integral.** Pull that constant out. What is left is $\int p_1^\alpha q_1^{1-\alpha}\, dy_1$, which by Fact 1 is $e^{(\alpha-1)D_\alpha(P_1\|Q_1)} \le e^{(\alpha-1)\varepsilon_1(\alpha)}$. So the double integral is at most $e^{(\alpha-1)\varepsilon_2(\alpha)} e^{(\alpha-1)\varepsilon_1(\alpha)}$.
5. **Take logarithms and divide by $\alpha - 1$.** This gives $D_\alpha(P\|Q) \le \varepsilon_1(\alpha) + \varepsilon_2(\alpha)$ for this pair. The right side does not depend on the pair, so the supremum over neighbours satisfies the same bound.

To see the steps on numbers, take an adaptive pair of randomized responses. The first step reports the bit with keep-probability 0.75. The second reports it again, with keep-probability 0.75 if the first output was 1 and 0.6 if it was 0. Table 27.5 lists the four joint outcomes when the true bit is 1 (P) and 0 (Q). Each joint probability is the first step's probability times the second's, as in step 1.

Table 27.5.: Two adaptive randomized responses: the second keep-probability depends on the first output. The last column gives the terms of the order-2 moment.

(y_1, y_2)	$p(y_1, y_2)$	$q(y_1, y_2)$	p^2/q
(1, 1)	$0.75 \times 0.75 = 0.5625$	$0.25 \times 0.25 = 0.0625$	5.0625
(1, 0)	$0.75 \times 0.25 = 0.1875$	$0.25 \times 0.75 = 0.1875$	0.1875
(0, 1)	$0.25 \times 0.6 = 0.15$	$0.75 \times 0.4 = 0.3$	0.0750
(0, 0)	$0.25 \times 0.4 = 0.1$	$0.75 \times 0.6 = 0.45$	0.0222

The terms sum to 5.3472, so $D_2 = \ln 5.3472 = 1.6766$. The bound adds the first step's curve to the worst curve the second step can have. At order 2 the second step's curve is $\varepsilon_{0.75}(2) = 0.8473$ on the branch $y_1 = 1$ and $\varepsilon_{0.6}(2) = 0.1542$ on the branch $y_1 = 0$, writing ε_p for the curve of randomized response with keep-probability p. So the bound is $\varepsilon_1(2) + \max\big(\varepsilon_{0.75}(2), \varepsilon_{0.6}(2)\big) = 0.8473 + \max(0.8473, 0.1542) = 1.6946$. The bound holds, and it is not tight here, because step 3 replaced the inner divergence by its worst case, 0.8473, on the branch $y_1 = 0$ where it is really 0.1542. When the second step does not adapt (keep-probability 0.75 on both branches), the joint distribution is a product and $D_2 = 1.6946$ exactly.

27.5.4. Interpretation

The bound is tight for products and exact for the Gaussian. When the mechanisms are independent, the inner divergence in step 3 is the same for every y_1, so replacing it by its worst case loses

nothing and the curves add exactly (additivity, Section 3.4). A sum of independent Gaussian steps is again Gaussian, so for Gaussian steps the composed curve is exactly the curve of a single mechanism.

This is why composition works for RDP and not for (ε, δ) directly. The order-α moment is an integral of a product, so it factorizes over the steps, as step 2 used. The smallest δ that works for a given ε is the **hockey-stick divergence** $\delta(\varepsilon) = \int (p - e^{\varepsilon} q)_+$, the total probability by which P exceeds e^{ε} times Q, where $(x)_+ = \max(x, 0)$. A positive part of a difference does not factorize over the steps. That is why composing (ε, δ) points needs a separate theorem with its own slack.

One condition remains. The mechanisms can be chosen adaptively, but each step's curve bound ε_i must be fixed in advance; it must not depend on the data. When the budget itself is chosen on the fly, you need the filters and odometers of Adaptive composition.

27.5.5. Example: k Gaussian steps

With $\sigma = 20$, each step has $\varepsilon(\alpha) = \alpha/(2 \cdot 20^2) = \alpha/800$. After k steps, the curve is $k\alpha/800$. Setting $k\alpha/800 = \alpha/(2\sigma'^2)$ shows that this is the curve of one Gaussian step with noise multiplier $\sigma' = 20/\sqrt{k}$:

- $k = 10$: $0.0125\,\alpha$, the curve of one Gaussian with $\sigma = 6.3246$;
- $k = 100$: $0.1250\,\alpha$, one Gaussian with $\sigma = 2.0000$;
- $k = 1000$: $1.2500\,\alpha$, one Gaussian with $\sigma = 0.6325$.

In general, k Gaussian steps with noise multiplier σ have the curve of one Gaussian with noise multiplier $\sigma/\sqrt{k}$. The function `compose_rdp` adds 1000 copies of the curve on the grid of orders and returns 2.5000 at $\alpha = 2$, which is $1000 \times 2/800$.

Different mechanisms compose just as easily, because their curves are added order by order. A Laplace step with $b = 1$ followed by a Gaussian step with $\sigma = 1$ has $\varepsilon(2) = 0.6191 + 1 = 1.6191$ and $\varepsilon(10) = 0.9287 + 5 = 5.9287$, reading both terms from Table 27.4.

Full-batch DP-SGD. The part's running example runs 10,000 steps of DP-SGD with $\sigma = 1$. If each step used the whole dataset, each would have the curve $\alpha/2$, and the composed curve would be $10{,}000\,\alpha/2 = 5000\,\alpha$. The converted guarantee at $\delta = 10^{-5}$ would be $\varepsilon = 5425.5$ even when computed exactly. That is no guarantee at all. With the conversion of Section 27.8, which adds a tail term $\ln(1/\delta)/(\alpha - 1)$ with $\ln(1/\delta) = 11.5129$, RDP on `ORDERS` gives $5000 \times 1.1 + 11.5129/0.1 = 5615.1$ at $\alpha = 1.1$, the lowest order of the grid. The continuous optimum (Equation 27.18 with the composed noise multiplier $1/\sqrt{10{,}000} = 0.01$) is $\alpha^* = 1 + 0.01 \times 4.7985 = 1.048$, with $5000 + 4.7985/0.01 = 5479.9$. A curve this steep needs orders just above 1. DP-SGD is private only because each step touches a random 1% of the data. Amplification by subsampling computes the curve of a subsampled step.

27.5.6. Check yourself

- Two mechanisms with curves $\alpha/8$ and $\alpha/2$ are composed. Which single Gaussian mechanism has the same curve? *(The sum is* $5\alpha/8 = \alpha/(2\sigma^2)$ *with* $\sigma^2 = 0.8$*, so* $\sigma = 0.8944$*.)*

- In Table 27.5, where does the derivation lose $1.6946 - 1.6766 = 0.0180$ nats? *(In step 3, which bounds the inner divergence on the branch $y_1 = 0$ by the worst case 0.8473, while the keep-probability 0.6 gives only 0.1542.)*

27.6. Post-processing and group privacy

27.6.1. Context

Two properties of DP carry over to RDP. Post-processing matters because a released model or its predictions must not leak more than the training procedure that produced them. Group privacy matters because records are correlated: a family shares a household, and a user contributes many rows. For ε-DP, a group of k records gets $k\varepsilon$ (Properties of DP), because the likelihood ratios along a chain of k neighbours multiply. For RDP the answer is weaker: the Rényi divergence does not satisfy a plain triangle inequality, so going through an intermediate dataset costs a higher order, and the order of the group guarantee must shrink as the group grows.

27.6.2. Statement

Post-processing. Applying any randomized map g that does not look at the data to the output cannot raise the curve: $\varepsilon_{g\circ M}(\alpha) \le \varepsilon_M(\alpha)$ for every α.

Weak triangle inequality (Mironov 2017, Proposition 11). To bound the divergence from P to Q through an intermediate distribution R, you pay with higher orders on the two legs, and the exponents u and v choose how the cost is split between them. For distributions P, R, Q, an order $\alpha > 1$ and exponents $u, v > 1$ with $\frac{1}{u} + \frac{1}{v} = 1$,

$$D_\alpha(P\|Q) \le \frac{\alpha - 1/u}{\alpha - 1} D_{u\alpha}(P\|R) + D_{v(\alpha-1)+1}(R\|Q). \tag{27.12}$$

The even split $u = v = 2$ (Corollary 4 there) doubles the order on the first leg and nearly doubles it on the second:

$$D_\alpha(P\|Q) \le \frac{\alpha - 1/2}{\alpha - 1} D_{2\alpha}(P\|R) + D_{2\alpha-1}(R\|Q). \tag{27.13}$$

Group privacy (Mironov 2017, Proposition 2). If M is (α, ε)-RDP and $\alpha \ge 2^{c+1}$, then on inputs that differ in 2^c records M satisfies the RDP bound of order $\alpha/2^c$ with ε replaced by $3^c\varepsilon$.

27.6.3. Derivation

Post-processing. $g \circ M(D)$ is $M(D)$ passed through the channel g, the same channel whichever dataset was used, so D_α can only decrease (Fact 3). Take the supremum over neighbours. For a concrete case, pass the output of randomized response ($p = 0.75$) through a channel that flips it with probability 0.2. When the bit is 1, the final output is 1 if the first output was 1 and was kept, or was 0 and was flipped: $0.75 \times 0.8 + 0.25 \times 0.2 = 0.65$. When the bit is 0, it is $0.25 \times 0.8 + 0.75 \times 0.2 = 0.35$. The two outputs are now closer in probability, and D_2 drops from 0.8473 to $\ln(0.65^2/0.35 + 0.35^2/0.65) = 0.3333$, while D_∞ drops from 1.0986 to $\ln(0.65/0.35) = 0.6190$.

Weak triangle inequality. The idea is to write the moment for P against Q as an average under R of a product of two likelihood ratios, p/r and r/q, and then separate the two factors with Hölder's inequality. Write p, r, q for the densities of P, R, Q.

1. **Insert the middle distribution.** Multiply and divide by powers of r: since $(p/r)^\alpha (r/q)^{\alpha-1} r = p^\alpha r^{-\alpha+\alpha-1+1} q^{1-\alpha} = p^\alpha q^{1-\alpha}$, Fact 1 gives
$$e^{(\alpha-1)D_\alpha(P\|Q)} = \int p^\alpha q^{1-\alpha} = \mathbb{E}_R\Big[\Big(\frac{p}{r}\Big)^\alpha \Big(\frac{r}{q}\Big)^{\alpha-1}\Big]. \tag{27.14}$$
2. **Separate the two ratios with Hölder's inequality.** Hölder's inequality bounds the average of a product of two non-negative quantities by a product of their higher moments: for $u, v > 1$ with $\frac{1}{u} + \frac{1}{v} = 1$, $\mathbb{E}_R[XY] \le (\mathbb{E}_R X^u)^{1/u} (\mathbb{E}_R Y^v)^{1/v}$. (With $u = v = 2$ it is the Cauchy–Schwarz inequality.) Apply it with $X = (p/r)^\alpha$ and $Y = (r/q)^{\alpha-1}$.
3. **The first factor** is $\mathbb{E}_R[(p/r)^{u\alpha}]$, the moment of the ratio p/r at order $u\alpha$ under R. By Fact 1 at order $u\alpha$, it equals $e^{(u\alpha-1)D_{u\alpha}(P\|R)}$.
4. **The second factor** is $\mathbb{E}_R[(r/q)^{v(\alpha-1)}]$. By the change of measure ($\mathbb{E}_R[g] = \mathbb{E}_Q[(r/q)g]$) it equals $\mathbb{E}_Q[(r/q)^{v(\alpha-1)+1}]$, and by Fact 1 at order $v(\alpha-1)+1$ that is $e^{v(\alpha-1)D_{v(\alpha-1)+1}(R\|Q)}$.
5. **Combine.** Raise the two factors to the powers $1/u$ and $1/v$ and take logarithms: $(\alpha-1)D_\alpha(P\|Q) \le \frac{u\alpha-1}{u} D_{u\alpha}(P\|R) + (\alpha-1) D_{v(\alpha-1)+1}(R\|Q)$. Divide by $\alpha - 1$ to get Equation 27.12, since $\frac{u\alpha-1}{u} = \alpha - \frac{1}{u}$. With $u = v = 2$, $v(\alpha-1)+1 = 2\alpha - 1$, which gives Equation 27.13.

Group privacy, two records ($c = 1$). The idea is to walk from D to D' through a dataset that differs from each in one record, and apply the weak triangle inequality to the two neighbouring steps. Let D and D' differ in two records, and let D_1 be the dataset in between, a neighbour of both. Put $P = M(D)$, $R = M(D_1)$, $Q = M(D')$ and let $\beta = \alpha/2 \ge 2$ be the order of the group guarantee.

1. **Apply Equation 27.13 at order β.** Its orders become $2\beta = \alpha$ and $2\beta - 1 = \alpha - 1$: $D_\beta(P\|Q) \le \frac{\beta-1/2}{\beta-1} D_\alpha(P\|R) + D_{\alpha-1}(R\|Q)$.
2. **Both pairs are neighbours**, so $D_\alpha(P\|R) \le \varepsilon$, and $D_{\alpha-1}(R\|Q) \le D_\alpha(R\|Q) \le \varepsilon$ by monotonicity (Fact 2).
3. **Bound the factor.** The factor $\frac{\beta-1/2}{\beta-1} = 1 + \frac{1/2}{\beta-1}$ decreases in β and equals $\frac{1.5}{1} = \frac{3}{2}$ at $\beta = 2$, so it is at most $\frac{3}{2}$ for $\beta \ge 2$. So $D_\beta(P\|Q) \le \frac{3}{2}\varepsilon + \varepsilon = \frac{5}{2}\varepsilon \le 3\varepsilon$. Mironov states the bound with the constant 3; the argument gives $\frac{5}{2}$.

For $c > 1$, repeat: a group of 2^c records is two groups of 2^{c-1}, so each round halves the order and multiplies ε by 3. After c rounds the order is $\alpha/2^c$ and the bound is $3^c\varepsilon$; the condition $\alpha \ge 2^{c+1}$ keeps the order of the last round at least 2.

27.6.4. Interpretation

Each doubling of the group costs half the order as well as a factor in ε. The reason is step 2 of the Hölder argument: the divergence across two steps involves the product of two likelihood ratios, and to control the average of a product you need higher moments of each factor. A bound at order α for one record therefore buys a bound at only about order $\alpha/2$ for two.

How loose is this? Test it on the Gaussian, where the exact answer is known. A group of 2^c records shifts a Gaussian query of sensitivity 1 by up to 2^c, so the exact group curve is Equation 27.7 with the shift 2^c in place of 1, $\alpha(2^c)^2/(2\sigma^2)$. Table 27.6 compares it with the bounds at $\sigma = 1$. For two records, the weak triangle inequality loses little once $\beta \geq 4$, and Proposition 2's constant 3 costs a factor 1.5, because the exact value is $2\varepsilon(2\beta)$ ($4\beta/2 = 2 \cdot 2\beta/2$) and Proposition 2 gives $3\varepsilon(2\beta)$. The loss compounds with c: 3^c against 2^c gives a factor 1.5^c.

So use the mechanism when you can. For the Gaussian, the exact group curve is just the curve with sensitivity 2^c. The generic bound matters for mechanisms whose curve on far-apart inputs you cannot compute.

Table 27.6.: Group privacy for the Gaussian mechanism with $\sigma = 1$: the exact curve on inputs 2^c records apart against the two bounds.

Group size 2^c, order	Exact	Weak triangle, iterated	Proposition 2: $3^c\varepsilon(2^c\beta)$	Proposition 2 / exact
2, $\beta = 2$	4.0000	4.5000	6.0000	1.5000
2, $\beta = 4$	8.0000	8.1667	12.0000	1.5000
2, $\beta = 10$	20.0000	20.0556	30.0000	1.5000
4, $\beta = 2$	16.0000	18.5000	36.0000	2.2500
8, $\beta = 2$	64.0000	74.5000	216.0000	3.3750

You can check the first row by hand. The exact value is $2 \cdot 2^2/2 = 4$. The weak triangle inequality at $\beta = 2$ gives $\frac{1.5}{1}\varepsilon(4) + \varepsilon(3) = 1.5 \times 2 + 1.5 = 4.5$. Proposition 2 gives $3\varepsilon(4) = 3 \times 2 = 6$, and $6/4 = 1.5$.

27.6.5. Example: two records under Gaussian and Laplace noise

Figure 27.3 draws the two-record case for the Gaussian with $\sigma = 1$. The weak-triangle bound hugs the exact line 2β for $\beta \geq 3$ and blows up near $\beta = 1$, where its factor $\frac{\beta - 1/2}{\beta - 1}$ does. Proposition 2 applies only from $\beta = 2$, and its line has slope 3 instead of 2.

For the Laplace mechanism with $b = 1$ at order 2, two records apart, the exact value is the Laplace curve with shift 2, that is Equation 27.8 with $1/b$ replaced by 2: 1.5958. The weak triangle gives 1.9674 and Proposition 2 gives 2.4411. The pure DP group bound, $2/b = 2$, is better than Proposition 2 here. Because the Laplace mechanism is $1/b$-DP, a group of two is $2/b$-DP, and by monotonicity that bound holds at every order. In general, for a mechanism that is already ε-DP, the group bound $k\varepsilon$ of pure DP holds at every order, and it can beat the RDP route.

27.6.6. Check yourself

- A mechanism is $(8, 1)$-RDP. What does Proposition 2 give for groups of two and of four records? *(Two records: order $8/2 = 4$ and $\varepsilon = 3$. Four records: order $8/4 = 2$ and $\varepsilon = 3^2 = 9$.)*
- Why does post-processing need g to ignore the data? *(If g reads the data, $g \circ M$ is not a channel applied to M's output, and Fact 3 does not apply: g could simply print a record.)*



Figure 27.3.: RDP of the Gaussian mechanism ($\sigma = 1$) on inputs two records apart, as a function of the order β of the group guarantee. Black: the exact curve 2β. Dashed: the weak triangle inequality with $u = v = 2$. Dotted: Proposition 2's bound $3\varepsilon(2\beta)$, drawn from $\beta = 2$, where its condition $\alpha = 2\beta \geq 4$ starts to hold.

27.7. The moments accountant is RDP in disguise

27.7.1. Context

Abadi et al. (2016) introduced the **moments accountant** to train neural networks with DP. For each $\lambda > 0$ (in practice, the integers up to 32), it tracks the logarithm of the moment generating function (MGF) of the privacy loss, $\ln \mathbb{E}[e^{\lambda L}]$. It adds these numbers over the training steps and converts the total to (ε, δ) with a tail bound. Their paper predates RDP and is written in different notation. But Fact 1 already says that the log-MGF of the privacy loss at λ is $(\alpha - 1)D_\alpha$ at $\alpha = \lambda + 1$. This section makes that precise: the moments accountant tracks the same object as the RDP curve, so everything in this chapter applies to it.

27.7.2. Statement

Abadi et al. define the log-moment $\alpha_M(\lambda) = \sup_{D \simeq D'} \ln \mathbb{E}_{Y \sim M(D)}\left[e^{\lambda L(Y)}\right]$, the log-MGF of the privacy loss for the worst pair of neighbours. Their letter α clashes with our order, so write it $K_M(\lambda)$. It is the RDP curve one order up, multiplied by λ:

$$K_M(\lambda) = \lambda\, \varepsilon_M(\lambda + 1), \qquad \text{equivalently} \qquad \varepsilon_M(\alpha) = \frac{K_M(\alpha - 1)}{\alpha - 1}. \tag{27.15}$$

The two parts of their Theorem 2 (Abadi et al. 2016) then become the two results of this chapter that RDP needs.

1. **Composability:** $K_M(\lambda) \le \sum_i K_{M_i}(\lambda)$. This is Equation 27.10 at order $\lambda + 1$, multiplied by λ.
2. **Tail bound:** M is (ε, δ)-DP with $\delta = \min_\lambda \exp\big(K_M(\lambda) - \lambda\varepsilon\big)$. This is the conversion of Section 27.8, read in the other direction: it fixes ε and solves for δ instead of fixing δ and solving for ε (Exercise 3 in the exercise book).

27.7.3. Derivation

The identity is Fact 1 written out, with one change of measure.

1. **Write the moment as a sum** over outputs (for densities, an integral). Since $e^{\lambda L} = (p/q)^\lambda$, $\mathbb{E}_P[e^{\lambda L}] = \sum_y p(y)\Big(\frac{p(y)}{q(y)}\Big)^\lambda$.
2. **Change of measure.** Move one factor of p/q onto the weight: $p\,(p/q)^\lambda = q\,(p/q)^{\lambda+1}$. So the sum is an average under Q, $\mathbb{E}_Q[r^{\lambda+1}]$.
3. **Fact 1 at order $\alpha = \lambda + 1$:** $\mathbb{E}_Q[r^\alpha] = e^{(\alpha-1)D_\alpha(P\|Q)} = e^{\lambda D_{\lambda+1}(P\|Q)}$.
4. **Take logarithms and the supremum** over neighbours. The factor $\lambda > 0$ does not change which pair is worst, so $K_M(\lambda) = \lambda\,\varepsilon_M(\lambda + 1)$.

As a concrete check, take randomized response with $p = 0.75$ at $\lambda = 1$, using Table 27.2. The output 1 has $p(1) = 0.75$ and $e^L = 3$, the output 0 has $p(0) = 0.25$ and $e^L = 1/3$. So $\mathbb{E}_P[e^L] = 0.75 \times 3 + 0.25 \times \frac{1}{3} = 2.2500 + 0.0833 = 2.3333$, the same two terms as in the order-2 column of Table 27.2. Its logarithm, 0.8473, is $1 \cdot \varepsilon(2)$. At $\lambda = 2$, the direct sum and $2\varepsilon(3)$ both give 1.9136.

27.7.4. Interpretation

The two accountants give the same numbers with different bookkeeping. The moments accountant stores $(\alpha - 1)\varepsilon(\alpha)$ where RDP stores $\varepsilon(\alpha)$, and both add over steps. Libraries that say "moments accountant" and libraries that say "RDP accountant" compute the same thing.

The identity itself is simple, so what was new in 2016? The contribution of Abadi et al. was a bound on $K(\lambda)$ for the *subsampled* Gaussian step of DP-SGD, where each step uses a random fraction of the data. Amplification by subsampling derives it in RDP form.

There is also a statistical reading. The log of a moment generating function is called the **cumulant generating function**: its derivatives at 0 are the mean, the variance and the higher cumulants of the variable. So $K_M(\lambda)$ is the cumulant generating function of the privacy loss of the worst pair (more precisely, the supremum over neighbours of their cumulant generating functions). If the loss is normal, all cumulants beyond the variance vanish, K is a quadratic in λ, and the RDP curve is linear in α, as for the Gaussian.

27.7.5. Example: the Gaussian privacy loss

Take the Gaussian pair $P = \mathcal{N}(0, 1)$, $Q = \mathcal{N}(\mu, 1)$. The log-ratio of the two densities is $-\frac{y^2}{2} + \frac{(y-\mu)^2}{2} = \frac{\mu^2}{2} - \mu y$, so the privacy loss $L(Y) = \frac{\mu^2}{2} - \mu Y$ with $Y \sim P$ is a linear function of a

standard normal. It is therefore normal, with mean $\mu^2/2$ and variance μ^2 (Divergences). The log-MGF of a normal variable with mean m and variance v is $m\lambda + v\lambda^2/2$, so

$$K(\lambda) = \lambda\frac{\mu^2}{2} + \frac{\lambda^2\mu^2}{2} = \lambda \cdot \frac{(\lambda+1)\mu^2}{2} = \lambda\,\varepsilon(\lambda+1), \tag{27.16}$$

with $\varepsilon(\alpha) = \alpha\mu^2/2$, as Equation 27.15 says. At $\mu = 1$, the formula gives $K(1) = \frac{1}{2} + \frac{1}{2} = 1.0000$ and $K(2) = 1 + 2 = 3.0000$. Averaging $e^{\lambda L}$ over a million draws of Y gives 1.0000 and 3.0003.

27.7.6. Check yourself

- A mechanism has $K_M(\lambda) = \lambda(\lambda+1)/8$. What is its RDP curve? *($\varepsilon(\alpha) = K(\alpha-1)/(\alpha-1) = (\alpha-1)\alpha/(8(\alpha-1)) = \alpha/8$, a Gaussian with $\sigma = 2$.)*
- Why does Abadi et al.'s composability theorem need no new proof once you know Equation 27.10? *(Multiplying Equation 27.10 at order $\lambda+1$ by $\lambda > 0$ gives it.)*

27.8. From RDP to (ε, δ)

27.8.1. Context

After composing, you hold a curve. What you report is an (ε, δ) pair, because that is the guarantee with a direct meaning for an event: no set of outputs becomes more than e^{ε} times likelier, up to an additive δ (Definition). The conversion rests on one observation. Outputs whose privacy loss is at most some threshold c already satisfy the e^c bound, so only the outputs with loss above c can break it, and their total probability can be charged to δ. That probability is a tail of the privacy loss, and a Chernoff bound controls a tail with an exponential moment, which is exactly what each point of the RDP curve provides. Every order gives a valid (ε, δ), and you pick the best one.

27.8.2. Statement

Mironov's conversion (Mironov 2017, Proposition 3). The converted ε is the RDP value plus a tail term, which pays for δ and shrinks as the order grows. If M is $(\alpha, \varepsilon(\alpha))$-RDP, then for every $0 < \delta < 1$ it is $(\varepsilon_\delta, \delta)$-DP with

$$\varepsilon_\delta = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{27.17}$$

If you hold the whole curve, take the best order: $\varepsilon_\delta = \min_\alpha \left[\varepsilon(\alpha) + \ln(1/\delta)/(\alpha-1)\right]$. For the Gaussian curve $\alpha/(2\sigma^2)$, the first term grows with α and the second falls, and over all real $\alpha > 1$ the minimum is where they balance:

$$\alpha^* = 1 + \sigma\sqrt{2\ln(1/\delta)}, \qquad \varepsilon_\delta = \frac{1}{2\sigma^2} + \frac{\sqrt{2\ln(1/\delta)}}{\sigma}. \tag{27.18}$$

27.8.3. Derivation

The conversion. The plan is to split every event into outputs with small loss, which satisfy the e^c bound, and outputs with large loss, whose probability is the tail, then bound the tail with Chernoff. Fix neighbours, with privacy loss L, and a threshold c to be chosen.

1. **Split any event S by the size of the loss:** $P(S) = P(S \cap \{L \le c\}) + P(S \cap \{L > c\})$.
2. **Bound each part.** On $\{L \le c\}$, $\ln(p/q) \le c$, so $p \le e^c q$ by the definition of L. Integrating over $S \cap \{L \le c\}$, the first term is at most $e^c Q(S)$. The second term is at most $P[L > c]$, since $S \cap \{L > c\}$ is a subset of $\{L > c\}$.
3. **Chernoff on the loss.** Apply Fact 4 with $\lambda = \alpha - 1$ and then Fact 1, which says $\mathbb{E}_P[e^{(\alpha-1)L}] = e^{(\alpha-1)D_\alpha(P\|Q)}$: $P[L > c] \le e^{-(\alpha-1)c}\,\mathbb{E}_P[e^{(\alpha-1)L}] = e^{(\alpha-1)(D_\alpha(P\|Q)-c)} \le e^{(\alpha-1)(\varepsilon(\alpha)-c)}$.
4. **Choose c to make the tail δ.** Setting $(\alpha-1)(\varepsilon(\alpha)-c) = \ln\delta$ gives $c = \varepsilon(\alpha)+\ln(1/\delta)/(\alpha-1)$. Then $P(S) \le e^c Q(S) + \delta$ for every S and every pair of neighbours, which is (c, δ)-DP.

Mironov proves the same bound by a different route, through Hölder's inequality ("probability preservation").

The Gaussian optimum. The plan is to minimize the converted ε over the order with calculus. Write $a = \alpha - 1 > 0$ and $\ell = \ln(1/\delta)$. Substituting the Gaussian curve $\alpha/(2\sigma^2) = (a+1)/(2\sigma^2)$ into Equation 27.17 gives $\varepsilon_\delta(a) = \frac{a+1}{2\sigma^2} + \frac{\ell}{a}$.

1. **Set the derivative to zero:** $\frac{1}{2\sigma^2} - \frac{\ell}{a^2} = 0$, so $a^2 = 2\sigma^2\ell$ and $a = \sigma\sqrt{2\ell}$. The function is convex in $a > 0$ (a line plus ℓ/a, whose second derivative $2\ell/a^3$ is positive), so this is the minimum.
2. **Substitute:** $\frac{a}{2\sigma^2} = \frac{\sqrt{2\ell}}{2\sigma}$ and $\frac{\ell}{a} = \frac{\ell}{\sigma\sqrt{2\ell}} = \frac{\sqrt{2\ell}}{2\sigma}$. The two terms that depend on a are equal at the optimum, and they add to $\sqrt{2\ell}/\sigma$. With the constant $\frac{1}{2\sigma^2}$, this is Equation 27.18.

27.8.4. Interpretation

The conversion gives something away in two places. Step 2 charges the whole tail $P[L > c]$ to δ. The exact smallest δ at level c is the hockey-stick divergence, which can be written as an average over outputs, $\delta(c) = \mathbb{E}_P[(1 - e^{c-L})_+]$: factoring p out of $(p - e^c q)_+$ leaves $(1 - e^{c-L})_+$. So the exact δ charges an output with loss $L > c$ only the fraction $1 - e^{c-L}$ of its probability, not all of it. Step 3 then bounds the tail with a Chernoff bound, which overestimates it. From RDP to (ε, δ) and to tests removes much of both losses with sharper conversions.

The order trades two terms against each other. Small orders make the curve small but the tail term $\ln(1/\delta)/(\alpha-1)$ large; large orders do the opposite (Figure 27.4, left). The optimum moves to smaller orders as the composed curve grows, because a steeper curve makes large orders more expensive. By Equation 27.18 with $\sqrt{2\ln(1/\delta)} = 4.7985$ at $\delta = 10^{-5}$, one Gaussian step with $\sigma = 20$ wants $\alpha \approx 1 + 20 \times 4.7985 \approx 97$, while 1000 steps, with composed noise multiplier $20/\sqrt{1000} = 0.6325$, want $\alpha \approx 1 + 0.6325 \times 4.7985 \approx 4$. That is why the grid runs from 1.1 to 512.

For many Gaussian steps the conversion reproduces the square-root growth of advanced composition. For k steps, Equation 27.18 with $\sigma/\sqrt{k}$ in place of σ gives $\varepsilon_\delta = \frac{k}{2\sigma^2} + \frac{\sqrt{2k\ln(1/\delta)}}{\sigma}$. The second

term grows like $\sqrt{k}$, the same shape as advanced composition, but with constants derived from the Gaussian curve instead of from a worst-case $(\varepsilon_0, \delta_0)$ point.

27.8.5. Example: one Gaussian step, and many

One step, $\sigma = 1$, $\delta = 10^{-5}$. Here $\ln(1/\delta) = 11.5129$ and $\sqrt{2\ln(1/\delta)} = \sqrt{23.0259} = 4.7985$. The curve is $\alpha/2$, and Equation 27.17 adds $11.5129/(\alpha - 1)$. Three orders show the trade-off:

- $\alpha = 2$: $1.0000 + 11.5129 = 12.5129$;
- $\alpha = 5.8$: $2.9000 + 2.3985 = 5.2985$;
- $\alpha = 20$: $10.0000 + 0.6059 = 10.6059$.

At the small order the tail term dominates, at the large order the curve does, and in between they nearly balance. The grid's best order is 5.8, next to the continuous optimum $\alpha^* = 1 + 4.7985 = 5.7985$, and both give $\varepsilon = 5.2985$.

To see how good this is, compare it with the exact ε of the Gaussian mechanism at $\delta = 10^{-5}$. The exact privacy curve of a Gaussian pair whose means are $\mu = 1/\sigma$ standard deviations apart is

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) \tag{27.19}$$

(Balle & Wang 2018; Laplace and Gaussian mechanisms), where Φ is the standard normal cumulative distribution function, $\Phi(t)$ being the probability that a standard normal variable is below t. It is the hockey-stick divergence $P(S) - e^{\varepsilon}Q(S)$ evaluated on the set S of outputs whose privacy loss exceeds ε: since the loss is normal with mean $\mu^2/2$ and standard deviation μ under P (and mean $-\mu^2/2$ under Q), each of $P(S)$ and $Q(S)$ is a normal tail, which gives the two Φ terms. Solving $\delta(\varepsilon) = 10^{-5}$ for ε by bisection gives 4.3772. The conversion is 21% too large ($5.2985/4.3772 = 1.21$). Read the other way, the mechanism at $\varepsilon = 5.2985$ actually has $\delta = 1.300 \times 10^{-7}$, 76.9 times smaller than the conversion promises ($10^{-5}/(1.300 \times 10^{-7}) = 76.9$).

k steps, $\sigma = 20$, $\delta = 10^{-5}$. Table 27.7 compares four ways to account for k Gaussian steps. The exact value is available because k steps are one Gaussian with $\mu = \sqrt{k}/\sigma$. The two generic routes from Part 3 first turn each step into one $(\varepsilon_0, \delta_0)$ point of its exact curve, then compose the points. *Basic composition* adds the per-step guarantees, both the εs and the δs, so it spends δ/k per step and reports $k\varepsilon_0$. *Advanced composition* treats the total privacy loss as a sum of k independent bounded terms and bounds its tail. It spends a fraction s of δ on the k per-step slacks, $\delta_0 = s\delta/k$ each, and the rest, $\delta' = (1 - s)\delta$, on the tail of the sum, and it gives

$$\varepsilon_0\sqrt{2k\ln(1/\delta')} + k\varepsilon_0\tanh(\varepsilon_0/2). \tag{27.20}$$

The first term is the size of a fluctuation of the summed loss that is exceeded with probability at most δ', from Azuma's inequality for a sum of k terms each between $-\varepsilon_0$ and ε_0. The second term is a bound on the mean of the summed loss: k times the largest mean loss of one step, $\varepsilon_0\tanh(\varepsilon_0/2)$. The script picks the best s on the grid Part 3 uses, the powers $10^{-1/4}, 10^{-2/4}, \ldots, 10^{-10}$; at every k in Table 27.7 it lands on $s = 10^{-1/4} \approx 0.56$.

The second term is the sharper form from Part 3. The textbook form, $k\varepsilon_0(e^{\varepsilon_0} - 1)$ (Dwork & Roth 2014, Theorem 3.20), would give 105.86 at $k = 1000$ instead of 68.01.

Table 27.7.: Total ε at $\delta = 10^{-5}$ for k Gaussian steps with $\sigma = 20$, four ways.

k	Basic	Advanced	RDP + Mironov (best order)	Exact	RDP / exact	Advanced / exact
1	0.160	0.847	0.251 (128)	0.160	1.566	5.29
10	1.892	3.270	0.771 (31)	0.561	1.374	5.83
100	21.510	13.423	2.524 (10.6)	1.993	1.267	6.73
1000	238.578	68.008	8.838 (4)	7.511	1.177	9.05

Advanced composition is loose, and it gets looser as k grows: it is 5 to 9 times the exact value, because each step's curve was collapsed to one point before composing. For one step it is even worse than basic composition, since the factor $\sqrt{2\ln(1/\delta')}$ is paid even when $k = 1$, when there is no sum to concentrate.

RDP is 18% to 57% above exact in this table, and it gets tighter as k grows. At $k = 1$ the best order on the continuous scale would be 97.0 (giving 0.241), but the grid jumps from 63 to 128, so the grid's answer 0.251 is a little worse. For $k \geq 10$ the grid optimum agrees with Equation 27.18 to within 0.001 (0.771 at 31.3, 2.524 at 10.6, and 8.838 on the grid against 8.837 at $\alpha^* = 4.03$). The remaining gap to the exact value is the conversion's slack, which the next chapter reduces.

That slack depends on the composed $\mu = \sqrt{k}/\sigma$, the distance between the two composed Gaussians in standard deviations. At the continuous optimum, Mironov's conversion is 1.07 times exact at $\mu = 10$, 1.21 at $\mu = 1$, 1.42 at $\mu = 0.1$ and 1.76 at $\mu = 0.01$. Weak composed mechanisms pay the most, because as $\mu \to 0$ the exact ε falls faster than the Chernoff bound does.

Figure 27.4 shows both halves: the trade-off over orders for $k = 1000$, and the three methods as k grows.

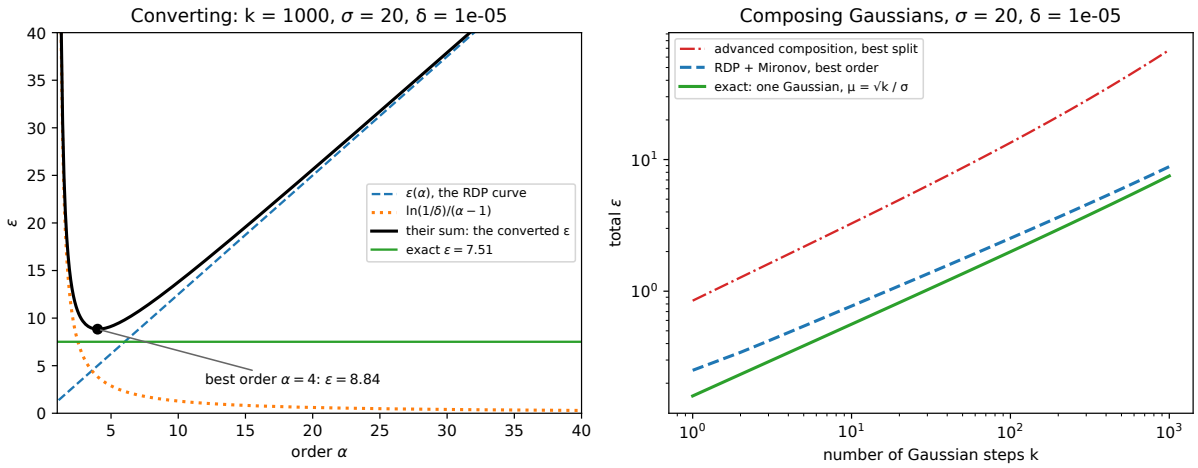


Figure 27.4.: Left: converting the curve of 1000 Gaussian steps with $\sigma = 20$ at $\delta = 10^{-5}$. The converted ε (black) is the sum of the RDP curve (dashed) and the tail term $\ln(1/\delta)/(\alpha - 1)$ (dotted). Its minimum over the grid, 8.84 at $\alpha = 4$, sits above the exact value 7.51 (green). Right: total ε against the number of steps k on log axes, for advanced composition, RDP with Mironov's conversion, and the exact value.

27.8.6. Check yourself

- A mechanism is $(10, 0.5)$-RDP. What $(\varepsilon, 10^{-5})$-DP does Equation 27.17 give at that order? *($0.5 + 11.5129/9 = 1.7792$.)*
- In Table 27.7, why is the best order 4 for $k = 1000$ steps but 128 for one step? *(The composed curve is 1000 times steeper, so large orders cost more. In Equation 27.18, replacing σ by $\sigma/\sqrt{k}$ shrinks $\alpha^* - 1$ by $\sqrt{k}$.)*

27.9. Code: check every curve and every conversion

The script code/renyi_dp.py computes every number in this chapter and draws every figure. The curves are small typed functions (`gaussian_rdp`, `laplace_rdp`, `randomized_response_rdp`). Composition is `compose_rdp`, a pointwise sum on the grid `ORDERS`. The conversion is `rdp_to_dp_mironov`, which returns the best ε and the order that attains it. The exact Gaussian privacy curve is `gaussian_delta_exact`, and `gaussian_eps_exact` inverts it by bisection. Later chapters import these functions. From the repository root:

```
uv run python code/part04-privacy-accounting/renyi_dp.py
```

Output (seed 0), summarized in Table 27.8:

Table 27.8.: The script's output with the default seed.

Experiment	Result
Randomized response, $p = 0.75$	$\varepsilon(2) = 0.8473$; KL 0.5493; $\varepsilon(\infty) = 1.0986$; curve in Table 27.3
Laplace curve against numerical integration	Largest difference 1.3×10^{-8} over $b \in \{0.5, 1, 4\}$ and four orders
Gaussian curve against numerical integration, $\sigma = 1$	1.0000 at $\alpha = 2$; 5.0000 at $\alpha = 10$
Adaptive randomized response, order 2	Exact 1.6766; bound 1.6946
Group privacy, Gaussian, two records, order 2	Exact 4.0000; weak triangle 4.5000; Proposition 2: 6.0000
Moments accountant, Gaussian $\mu = 1$	$K(1) = 1.0000$, $K(2) = 3.0000$; Monte Carlo 1.0000 and 3.0003
One Gaussian, $\sigma = 1$, $\delta = 10^{-5}$	RDP 5.2985 at $\alpha = 5.8$; exact 4.3772
1000 Gaussians, $\sigma = 20$, $\delta = 10^{-5}$	Advanced 68.01; RDP 8.84 at $\alpha = 4$; exact 7.51

tests/test_renyi_dp.py checks that the output still matches.

What to take from it. First, the closed forms are right. The Laplace and Gaussian curves agree with direct numerical integration, and the moments accountant's log-moments agree with the

RDP curve and with simulation. Second, composition is exact where it should be. Non-adaptive randomized responses and Gaussian steps compose with equality, because their outputs are independent and the moments factorize; the adaptive example shows where the bound gives something away, on the branch where the second step is less revealing than its worst case. Third, the conversion costs 18% to 57% in Table 27.7, while advanced composition costs 5 to 9 times. Most of the gain of RDP comes from composing curves rather than points, so even a loose conversion done once at the end beats a good theorem applied to points.

27.10. References

- Mironov, "Rényi Differential Privacy" (IEEE Computer Security Foundations Symposium, CSF 2017; arXiv:1702.07476). The source of this chapter: the definition (Definition 4), adaptive composition (Proposition 1), weak group privacy (Proposition 2), the conversion (Proposition 3), the curves of randomized response, Laplace and Gaussian (Table 2, Corollaries 2 and 3) and the weak triangle inequality (Proposition 11, Corollary 4). Short and readable.
- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016). DP-SGD and the moments accountant; Theorem 2 is composability and the tail bound. Read it for the subsampled Gaussian bound and the experiments.
- van Erven & Harremoës, "Rényi Divergence and Kullback–Leibler Divergence" (IEEE Transactions on Information Theory, 2014). The properties of D_α that RDP inherits, proved for general measures.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018). The exact privacy curve of the Gaussian mechanism used here as the ground truth (Theorem 8).
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (2014), ch. 3. Basic composition (Theorem 3.16) and advanced composition (Theorem 3.20), the baseline that RDP improves.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The curves that make DP-SGD accounting work, and the reference implementation behind most RDP accountants; used in Amplification by subsampling.

27.11. Cheat sheet

$P = M(D)$, $Q = M(D')$ for neighbours; $L = \ln(p/q)$ the privacy loss; orders $\alpha > 1$. Table 27.9 collects the results.

Table 27.9.: Every result of this chapter.

Result	Statement	Tight?
(α, ε)-RDP	$D_\alpha(M(D)\|M(D')) \le \varepsilon$ for all $D \simeq D'$	—

Result	Statement	Tight?
Limits	$\varepsilon(\alpha \to 1) \geq$ largest KL (equal for the mechanisms here); $\$\varepsilon(\infty) = \$$ pure DP ε; non-decreasing in α	—
Gaussian, noise multiplier σ	$\varepsilon(\alpha) = \alpha/(2\sigma^2)$	Exact
Laplace, scale b, sensitivity 1	$\frac{1}{\alpha-1}\ln\left(\frac{\alpha}{2\alpha-1}e^{(\alpha-1)/b} + \frac{\alpha-1}{2\alpha-1}e^{-\alpha/b}\right)$	Exact
Randomized response, keep p	$\frac{1}{\alpha-1}\ln\left(p^\alpha(1-p)^{1-\alpha} + (1-p)^\alpha p^{1-\alpha}\right)$	Exact
Composition	$\varepsilon_M(\alpha) \leq \sum_i \varepsilon_i(\alpha)$, adaptively	Equality for products
Post-processing	$\varepsilon_{g\circ M}(\alpha) \leq \varepsilon_M(\alpha)$	—
Weak triangle	$D_\alpha(P\|Q) \leq \frac{\alpha-1/u}{\alpha-1}D_{u\alpha}(P\|R) + D_{v(\alpha-1)+1}(R\|Q)$, $\frac{1}{u}+\frac{1}{v}=1$	Equality for Gaussians at the best u (Exercise 4 in the exercise book)
Group of 2^c	order $\alpha/2^c$, $3^c\varepsilon(\alpha)$, if $\alpha \geq 2^{c+1}$	Loose: factor 1.5 per doubling on the Gaussian
Moments accountant	$K(\lambda) = \lambda\,\varepsilon(\lambda+1)$	Identity
Conversion	$(\varepsilon(\alpha) + \ln(1/\delta)/(\alpha-1), \delta)$-DP, best α	Loose: 18–57% above exact in Table 27.7; $\times 1.76$ at composed $\mu = 0.01$
Gaussian optimum	$\alpha^* = 1 + \sigma\sqrt{2\ln(1/\delta)}$, $\varepsilon = \frac{1}{2\sigma^2} + \frac{\sqrt{2\ln(1/\delta)}}{\sigma}$	—

Rules of thumb

- Compose in RDP, convert once at the end. Never convert each step to $(\varepsilon_0, \delta_0)$ first.
- k Gaussian steps with noise multiplier σ are one Gaussian step with $\sigma/\sqrt{k}$.
- The best order shrinks as the composed curve grows; keep a grid from just above 1 to a few hundred.
- Mironov's conversion is loose by tens of percent at moderate μ, and by nearly a factor 2 for very weak composed mechanisms; for reporting, use the sharper conversions of the next chapter or a numerical accountant.
- Group privacy in RDP halves the order; for the Gaussian, use the curve with the group's sensitivity instead.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

28. From Rényi DP to (ε, δ) and to tests

28.1. Why this matters

Rényi differential privacy (RDP) is the quantity accountants track, because it composes by adding curves (previous chapter). Recall what the curve is. For two neighbouring datasets, which differ in one person's record, a mechanism produces two output distributions, P on the dataset without your record and Q on the one with it. For each **order** $\alpha > 1$, the Rényi divergence $D_\alpha(P\|Q)$ measures how far apart they are: it is a logarithmic average of the likelihood ratio p/q raised to the power $\alpha - 1$, so larger orders weigh the outputs where P is much more likely than Q more heavily. The **RDP curve** $\varepsilon(\alpha)$ bounds that divergence, over all neighbouring pairs, at every order. When mechanisms run one after another, their curves add.

The curve is not the quantity anyone reports. A paper, a model card or a regulator asks for (ε, δ)-differential privacy (DP), or for a statement about attacks: how much better than guessing can a test tell whether your record was used? So every RDP analysis ends with a conversion, and the conversion decides the number you publish. This chapter answers three questions: how much the standard conversion loses, and where; what the best (ε, δ) is that you can claim from an RDP curve, and which order to use for it; and what an RDP curve says about tests, and what it can never say.

One idea organizes all three answers: **an RDP curve is a summary of the privacy loss distribution, and converting it asks for the largest δ consistent with that summary.** The privacy loss of an output is the logarithm of how much more likely that output is under P than under Q. Drawn at random from the mechanism, it is the privacy loss random variable (PRV), and δ is a particular average over it. The RDP curve records the moment generating function (MGF) of the PRV, the average of $e^{\lambda L}$ for a loss L, at the points $\lambda = \alpha - 1$, and nothing else. A conversion has to bound the average that defines δ using only those MGF values. Mironov's conversion does this with Markov's inequality and throws away most of what the summary says. The improved conversion, and the hypothesis-testing view behind it, recover almost all of it: for the Gaussian mechanism the improved conversion agrees to three decimals with the best any method could do with the curve alone. What remains, 5 to 10% of ε, is not a defect of any formula. A two-outcome mechanism has an RDP curve no larger than the Gaussian's and a larger δ, so no conversion that sees only the curve can do better. Table 28.1 lists the results.

Table 28.1.: The results in this chapter and where they are used.

Result	Where it is used
Mironov's conversion $\varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$	The original moments accountant; still in older code and papers

Result	Where it is used
Improved conversion (Balle et al.; Canonne, Kamath & Steinke)	Every modern RDP accountant: Opacus, Google's `dp_accounting`
RDP as a constraint on tests	Reading an RDP guarantee as a bound on attack power; comparing with f-DP
Choosing the order α	Designing the order grid of an accountant
The slack no conversion removes	Knowing when to switch to a numerical accountant
(ε, δ) does not imply RDP; ε-DP does	Mixing pure-DP and RDP components in one analysis

Conventions. $P = M(D)$ and $Q = M(D')$ are the output distributions of the mechanism M on neighbouring datasets D and D', with densities p and q. As in Part 3, D' is the dataset that holds your record and D the one without it. A guarantee must hold for every ordered pair of neighbours, so every bound below holds just as well with the roles of P and Q swapped. The privacy loss at an output y is $L(y) = \ln(p(y)/q(y))$: positive when y is more likely under D than under D', negative when it is less likely. With $Y \sim P$, the random number $L(Y)$ is the PRV (composition). Throughout, α **is the Rényi order and never an error rate.** A test of "the output came from P" against "it came from Q", that is, of "your record was not used" against "it was", looks at the output and either accepts or rejects. It has **type I error** a, the probability that it rejects although the output came from P (for a membership test, its false-positive rate: saying D' when the data were D), and **type II error** b, the probability that it accepts although the output came from Q, as in hypothesis testing, where they were called α_ϕ and β_ϕ. Its **power** $1 - b$ is the probability that it correctly rejects when the output came from Q.

The running example is the Gaussian mechanism. One step adds noise $\mathcal{N}(0, \sigma^2)$ to a query of sensitivity 1, where σ is the noise multiplier. After dividing the output by σ, the two worlds are $\mathcal{N}(0, 1)$ and $\mathcal{N}(1/\sigma, 1)$. With k independent steps the outputs form a vector, and only its component along the direction of the shift differs between the worlds; that shift has length $\sqrt{k}/\sigma$. So every quantity in this chapter depends only on $\mu = \sqrt{k}/\sigma$, and the composed mechanism is the pair $\mathcal{N}(0, 1)$, $\mathcal{N}(\mu, 1)$. Its RDP curve is $\varepsilon(\alpha) = \alpha\mu^2/2$: one step has curve $\alpha/(2\sigma^2)$ (previous chapter), and k steps add k copies, $k\alpha/(2\sigma^2) = \alpha\mu^2/2$. All numbers use $\delta = 10^{-5}$ and the part's grid of 154 orders: 1.1 to 10.9 in steps of 0.1, every integer from 12 to 63, and then 128, 256 and 512.

28.2. Toolkit: four facts every derivation uses

28.2.1. Fact 1 — δ is an expectation over the privacy loss

(ε, δ)-DP for the ordered pair (P, Q) says $P(S) \le e^\varepsilon Q(S) + \delta$ for every event S (definition): any set of outputs is at most e^ε times more likely under D than under D', up to an additive slack δ. The smallest δ that works is therefore the largest value of $P(S) - e^\varepsilon Q(S)$ over all events. That largest value is reached by putting into S exactly the outputs with $p > e^\varepsilon q$, because each

such output adds a positive amount $p - e^{\varepsilon} q$ and every other output would subtract. The result, $\int (p - e^{\varepsilon} q)_+$, where $(x)_+ = \max(x, 0)$, is the hockey-stick divergence $E_{e^{\varepsilon}}(P\|Q)$ of divergences. Written in terms of the privacy loss, it is an average over the PRV:

$$\delta(\varepsilon) = E_{e^{\varepsilon}}(P\|Q) = \mathbb{E}_P\big[(1 - e^{\varepsilon - L})_+\big] = P[L > \varepsilon] - e^{\varepsilon} Q[L > \varepsilon]. \tag{28.1}$$

The second form comes from factoring p out of the integrand. Where $p > 0$, $(p - e^{\varepsilon} q)_+ = p\,(1 - e^{\varepsilon} q/p)_+$, and $q/p = e^{-L}$, so this is $p\,(1 - e^{\varepsilon - L})_+$; integrating p times a function of the output is taking its average under P. The integrand is positive exactly where $L > \varepsilon$, and integrating $p - e^{\varepsilon} q$ over that set gives the last form.

The second form is the one to remember. Each output contributes to δ only if its privacy loss exceeds ε, and then only the fraction $1 - e^{\varepsilon - L}$ of its probability. An output with loss $L = \varepsilon + \ln 2$ contributes half its probability, since $1 - e^{-\ln 2} = 1 - \frac{1}{2}$. An output with loss just above ε contributes almost nothing, and only an output with infinite loss contributes all of its probability.

28.2.2. Fact 2 — RDP is the MGF of the privacy loss

The Rényi divergence of order α is defined by $D_\alpha(P\|Q) = \frac{1}{\alpha - 1} \ln \int p^\alpha q^{1-\alpha}$. Write $p^\alpha q^{1-\alpha} = p\,(p/q)^{\alpha - 1}$ and $(p/q)^{\alpha - 1} = e^{(\alpha - 1)L}$: the integral is the average under P of $e^{(\alpha - 1)L}$, which is the MGF of the PRV at the point $\alpha - 1$. An RDP bound $D_\alpha(P\|Q) \le \varepsilon(\alpha)$ is therefore a bound on that MGF value:

$$\mathbb{E}_P\big[e^{(\alpha - 1)L}\big] = e^{(\alpha - 1) D_\alpha(P\|Q)} \le e^{(\alpha - 1)\varepsilon(\alpha)}. \tag{28.2}$$

This is Equation 3.12 of divergences, with the exponent multiplied out.

So knowing the RDP curve at the orders $\alpha_1, \alpha_2, \ldots$ is knowing (an upper bound on) the MGF of the PRV at $\alpha_1 - 1, \alpha_2 - 1, \ldots$. That is the whole summary. Every conversion is a way of bounding the average in Equation 28.1 using only the averages in Equation 28.2.

28.2.3. Fact 3 — Data processing on a test

A test ϕ maps the output to "reject" or "accept", so its outcome is a coin. Under P the coin shows "reject" with probability a; under Q it shows "reject" with probability $1 - b$. Turning the output into this coin is post-processing, and post-processing cannot increase a Rényi divergence: this is the data processing inequality of divergences. The short reason is that merging outputs can only shrink the sum $\int p^\alpha q^{1-\alpha}$. The function $x^\alpha y^{1-\alpha}$ is convex for $\alpha > 1$ and scales linearly when x and y are both multiplied by the same factor, and such a function satisfies $g(x_1 + x_2, y_1 + y_2) \le g(x_1, y_1) + g(x_2, y_2)$; merging two outputs replaces the right side by the left. (A test that randomizes its decision is handled the same way; divergences gives the general proof.) So, writing $\mathrm{Bern}(x)$ for "reject with probability x",

$$D_\alpha(\mathrm{Bern}(a)\,\|\,\mathrm{Bern}(1 - b)) \le D_\alpha(P\|Q). \tag{28.3}$$

In words: a test cannot tell the two worlds apart better, in Rényi divergence, than the full output does.

28.2.4. Fact 4 — The Gaussian pair, exactly

For the running example everything can be computed in closed form, which is what lets this chapter measure each conversion against the truth. Take $P = \mathcal{N}(0,1)$ and $Q = \mathcal{N}(\mu,1)$. The privacy loss is the difference of the two log densities, $L(y) = -\frac{y^2}{2} + \frac{(y-\mu)^2}{2} = \frac{\mu^2}{2} - \mu y$. It is a linear function of y, so under P, where y is standard normal, the PRV is normal with mean $\mu^2/2$ and standard deviation μ: $\mathcal{N}(\mu^2/2, \mu^2)$.

Now evaluate the last form of Equation 28.1. Write $\Phi(x) = \Pr[Z \le x]$ for the standard normal cumulative distribution function. Under P, $L > \varepsilon$ means $y < \mu/2 - \varepsilon/\mu$, which has probability $\Phi(\mu/2 - \varepsilon/\mu)$. Under Q, write $y = \mu + z$ with z standard normal; then $L = -\mu^2/2 - \mu z$, and $L > \varepsilon$ means $z < -\mu/2 - \varepsilon/\mu$. So

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big), \tag{28.4}$$

the closed form already derived in divergences. It falls as ε grows, because the integrand $(1-e^{\varepsilon-L})_+$ does, so inverting it by bisection gives the exact ε at any δ. The script evaluates the second term in logarithms so that it stays accurate when e^{ε} overflows. As a check, at $\mu = 1$ a Riemann sum of $\mathbb{E}_P[(1-e^{\varepsilon-L})_+]$ over the normal density gives 1.2694×10^{-1} at $\varepsilon = 1$ and 1.5372×10^{-3} at $\varepsilon = 3$, the same as the closed form to all printed digits.

28.3. Where Mironov's conversion loses

28.3.1. Context

Rényi DP derived Mironov's conversion: an $(\alpha, \varepsilon(\alpha))$-RDP mechanism, one whose Rényi divergence of order α is at most $\varepsilon(\alpha)$, is $(\varepsilon(\alpha) + \ln(1/\delta)/(\alpha-1), \delta)$-DP. It is easy to remember and easy to compute. This section rewrites that proof as two inequalities on the privacy loss and measures what each one costs on the Gaussian, so that you can see where the excess comes from.

28.3.2. Statement

The proof first replaces δ by the probability that the loss exceeds ε, and then bounds that probability with the MGF. For every order $\alpha > 1$ and every ε,

$$\delta(\varepsilon) \overset{(1)}{\le} P[L > \varepsilon] \overset{(2)}{\le} e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}, \tag{28.5}$$

and setting the right side equal to δ gives **Mironov's conversion**

$$\varepsilon' = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1}. \tag{28.6}$$

28.3.3. Derivation

The proof in Rényi DP has exactly these two steps. Written with Fact 1 and Fact 2:

1. **Drop the fraction.** Fact 1 says each output with $L > \varepsilon$ contributes the fraction $1 - e^{\varepsilon - L}$ of its probability. Step 1 rounds that fraction up to 1: $(1 - e^{\varepsilon - L})_+ \le \mathbf{1}[L > \varepsilon]$, because the left side is 0 when $L \le \varepsilon$ and below 1 otherwise. Averaging under P gives inequality (1). This is the step that charges the whole tail $P[L > \varepsilon]$ to δ.
2. **Bound the tail by the MGF (Chernoff).** Markov's inequality says a nonnegative random variable exceeds a level t with probability at most its average divided by t. Apply it to $e^{(\alpha-1)L}$, which exceeds $e^{(\alpha-1)\varepsilon}$ exactly when $L > \varepsilon$: $P[L > \varepsilon] \le \mathbb{E}_P[e^{(\alpha-1)L}]\, e^{-(\alpha-1)\varepsilon}$. Fact 2 bounds the average by $e^{(\alpha-1)\varepsilon(\alpha)}$, which gives inequality (2).
3. **Solve for ε.** Setting $e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)} = \delta$ and taking logarithms gives $(\alpha - 1)(\varepsilon(\alpha) - \varepsilon) = -\ln(1/\delta)$, which is Equation 28.6. DP also needs the inequality with the roles of the two datasets swapped, the pair (Q, P). RDP bounds $D_\alpha(Q\|P)$ by the same curve, because swapping two neighbours gives two neighbours, so the same argument applies.

28.3.4. Interpretation

The two steps lose in different places. Table 28.2 measures them on the Gaussian, converting each inequality of Equation 28.5 into an ε at $\delta = 10^{-5}$. The first row is the exact ε from Equation 28.4. The second is the ε at which the tail $P[L > \varepsilon] = \Phi(\mu/2 - \varepsilon/\mu)$ equals δ. Since $\Phi(x) = 10^{-5}$ at $x = -\Phi^{-1}(1 - 10^{-5}) = -4.2649$, where Φ^{-1} is the inverse of Φ, this is $\varepsilon = \mu^2/2 + 4.2649\,\mu$: $0.5 + 4.2649 \approx 4.765$ at $\mu = 1$ and $50 + 42.649 = 92.649$ at $\mu = 10$. The third row is Mironov's ε' minimized over the grid.

Table 28.2.: Mironov's conversion for the Gaussian at $\delta = 10^{-5}$, split into its two steps.

	$\mu = 1$ (one step, $\sigma = 1$)	$\mu = 10$ (100 steps, $\sigma = 1$)
Exact ε	4.377	91.817
After step 1, $P[L > \varepsilon] = \delta$	4.765	92.649
After step 2, Mironov	5.299 at $\alpha = 5.8$	98.026 at $\alpha = 1.5$
Loss from step 1	0.388	0.832
Loss from step 2	0.534	5.377 (0.041 of it from the grid)

Step 1 ignores how far past ε the loss goes. An output with L slightly above ε counts fully toward the tail but almost nothing toward δ. The density of the Gaussian PRV is falling steeply at ε, which sits several standard deviations above its mean, so most of the tail sits just above ε, and step 1 overcounts it.

Step 2 replaces a tail by an MGF, and that costs more. Markov's inequality is tight only for a variable that puts all its mass on 0 and on the level t; the Gaussian PRV is nothing like that, so the bound is far above the true tail at 4.765. At $\mu = 10$ this step costs 5.377 of the $98.026 - 91.817 = 6.209$ total excess (Table 28.2). Of that, 0.041 is the grid's: over all real orders Mironov's conversion gives 97.985 at $\alpha = 1.480$, against 98.026 at the grid's 1.5.

On the δ side the losses are multiplicative. At Mironov's $\varepsilon = 5.299$ for $\mu = 1$, the exact δ is 1.300×10^{-7}. The tail is 7.992×10^{-7}, 6.1 times larger, and Mironov's bound is 1.000×10^{-5}, 12.5 times larger again. You pay for $\delta = 10^{-5}$ and get 1.3×10^{-7}.

28.3.5. Example: one release of a mean

You release the mean of a bounded statistic with the Gaussian mechanism at noise multiplier $\sigma = 1$, so $\mu = 1$ and $\varepsilon(\alpha) = \alpha/2$. Mironov's conversion gives $\alpha/2 + \ln(10^5)/(\alpha - 1)$ at each order. The first term grows with the order and the second shrinks, so there is a best order in between. Over the continuous range of orders the sum is smallest at $\alpha = 5.799$, with $\varepsilon' = 5.299$ (Section 28.6 derives both); the grid's best order, 5.8, gives the same 5.299. The exact answer is 4.377. Reporting Mironov's number overstates ε by 0.921, which means $e^{0.921}$ more "allowed" likelihood ratio for every output.

28.3.6. Check yourself

- Why does step 2 lose about ten times more at $\mu = 10$ than at $\mu = 1$? *(The PRV has mean $\mu^2/2$ and standard deviation μ. At the best order, Mironov's $\varepsilon^* = \mu^2/2 + 2\sqrt{(\mu^2/2)\ln(1/\delta)} = \mu^2/2 + \mu\sqrt{2\ln(1/\delta)}$ sits $\sqrt{2\ln(1/\delta)} \approx 4.799$ standard deviations above the mean, while the true tail point sits $\Phi^{-1}(1-\delta) = 4.2649$ standard deviations above it. The gap is a fixed number of standard deviations, about 0.534, so in ε it grows like μ: 0.534 at $\mu = 1$, about ten times that at $\mu = 10$.)*
- Is step 1 ever tight? *(Only when every output with $L > \varepsilon$ has $L = \infty$, where $1 - e^{\varepsilon - L} = 1$, such as a mechanism that sometimes reveals the record outright, with the pair ordered so that P is the output on the dataset that holds the record.)*

28.4. The improved conversion

28.4.1. Context

Both of Mironov's steps together replace the integrand $(1 - e^{\varepsilon - z})_+$ of Equation 28.1, as a function of the loss value z, by an exponential $c\, e^{(\alpha-1)z}$ that lies above it. An exponential is used because it is the only kind of function whose average Fact 2 tells you. Mironov's choice is $c = e^{-(\alpha-1)\varepsilon}$, the exponential that equals 1 at $z = \varepsilon$ and so passes through the corner of the indicator. That is not the smallest exponential above the integrand, because the integrand is well below 1 near $z = \varepsilon$. Using the smallest one is the whole improvement.

28.4.2. Statement

An $(\alpha, \varepsilon(\alpha))$-RDP mechanism is (ε', δ)-DP with

$$\varepsilon' = \varepsilon(\alpha) + \ln\frac{\alpha - 1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha - 1}, \tag{28.7}$$

or equivalently, at a given ε,

$$\delta = \frac{e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}}{\alpha}\Big(1-\frac{1}{\alpha}\Big)^{\alpha-1}. \tag{28.8}$$

This is Theorem 21 of Balle, Barthe, Gaboardi, Hsu & Sato (2020) and Proposition 12 of Canonne, Kamath & Steinke (2020), who also give the δ form; Asoodeh, Liao, Calmon, Kosut & Sankar proved it independently (2020, Lemma 1, Eq. 20). We call it the **improved conversion**. Compared with Equation 28.6, its δ is Mironov's $e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}$ multiplied by the factor $\frac{1}{\alpha}(1-\frac{1}{\alpha})^{\alpha-1}$, which is below 1.

28.4.3. Derivation

The plan: find the smallest constant c for which $c\,e^{(\alpha-1)z}$ stays above the integrand, then average.

1. **Bound the integrand by an exponential.** If $(1-e^{\varepsilon-z})_+ \le c\,e^{(\alpha-1)z}$ for every real z, then the same holds with z replaced by the random loss L, and averaging under P keeps the inequality. By Fact 1 the left side averages to $\delta(\varepsilon)$, and by Fact 2 the right side averages to at most $c\,e^{(\alpha-1)\varepsilon(\alpha)}$. So $\delta(\varepsilon) \le c\,\mathbb{E}_P[e^{(\alpha-1)L}] \le c\,e^{(\alpha-1)\varepsilon(\alpha)}$.
2. **The smallest c.** The condition of step 1 says $c \ge (1-e^{\varepsilon-z})_+\,e^{-(\alpha-1)z}$ for every z, so the smallest valid c is $c^* = \sup_z (1-e^{\varepsilon-z})_+\,e^{-(\alpha-1)z}$. For $z \le \varepsilon$ the expression is 0, so only $z > \varepsilon$ matters. There it is $f(z) = e^{-(\alpha-1)z} - e^{\varepsilon-\alpha z}$.
3. **Find the maximum.** Differentiate: $f'(z) = -(\alpha-1)e^{-(\alpha-1)z} + \alpha e^{\varepsilon-\alpha z} = e^{-\alpha z}(\alpha e^{\varepsilon} - (\alpha-1)e^{z})$. The bracket equals $e^{\varepsilon} > 0$ at $z = \varepsilon$ and decreases as z grows, so f first rises and then falls. Its maximum is where the bracket vanishes, at $e^{z^*} = \frac{\alpha}{\alpha-1}e^{\varepsilon}$, that is, $z^* = \varepsilon - \ln(1-1/\alpha)$.
4. **Evaluate it.** Factor $f(z^*) = e^{-\alpha z^*}(e^{z^*} - e^{\varepsilon})$. At z^*, $e^{z^*} - e^{\varepsilon} = e^{\varepsilon}(\frac{\alpha}{\alpha-1} - 1) = e^{\varepsilon}\cdot\frac{1}{\alpha-1}$, and $e^{-\alpha z^*} = e^{-\alpha\varepsilon}(1-1/\alpha)^{\alpha}$. Multiplying,

$$c^* = \frac{e^{-(\alpha-1)\varepsilon}}{\alpha-1}\Big(1-\frac{1}{\alpha}\Big)^{\alpha}. \tag{28.9}$$

5. **Collect.** Step 1 gives $\delta \le c^* e^{(\alpha-1)\varepsilon(\alpha)}$. Since $(1-1/\alpha)/(\alpha-1) = 1/\alpha$, the constant in front simplifies to $\frac{1}{\alpha}(1-\frac{1}{\alpha})^{\alpha-1}$, and this is Equation 28.8.
6. **Solve for ε.** Set the bound equal to δ and take logarithms: $\ln\delta = (\alpha-1)(\varepsilon(\alpha)-\varepsilon) - \ln\alpha + (\alpha-1)\ln(1-1/\alpha)$. Divide by $\alpha-1$ and solve for ε, using $\ln(1-1/\alpha) = \ln\frac{\alpha-1}{\alpha}$, to get Equation 28.7. As before, the reverse pair (Q, P) follows because RDP bounds $D_\alpha(Q\|P)$ by the same curve.

The script checks step 4 by brute force: at $\alpha = 6$ and $\varepsilon = 4$ the formula gives $c^* = 1.380552\times10^{-10}$ at $z^* = 4 - \ln(5/6) = 4.1823$, and the maximum of f over a grid of two million points is the same to all seven digits.

Figure 28.1 draws the two exponentials. Mironov's passes through the corner of the indicator at $z = \varepsilon$; the smallest one sits lower and touches the integrand at z^*, slightly to the right of ε.

28.4.4. Interpretation

The gain depends only on the order. Both conversions add something to the same $\varepsilon(\alpha)$, so when you subtract Equation 28.7 from Equation 28.6 at the same α, the $\varepsilon(\alpha)$ terms cancel, and so do

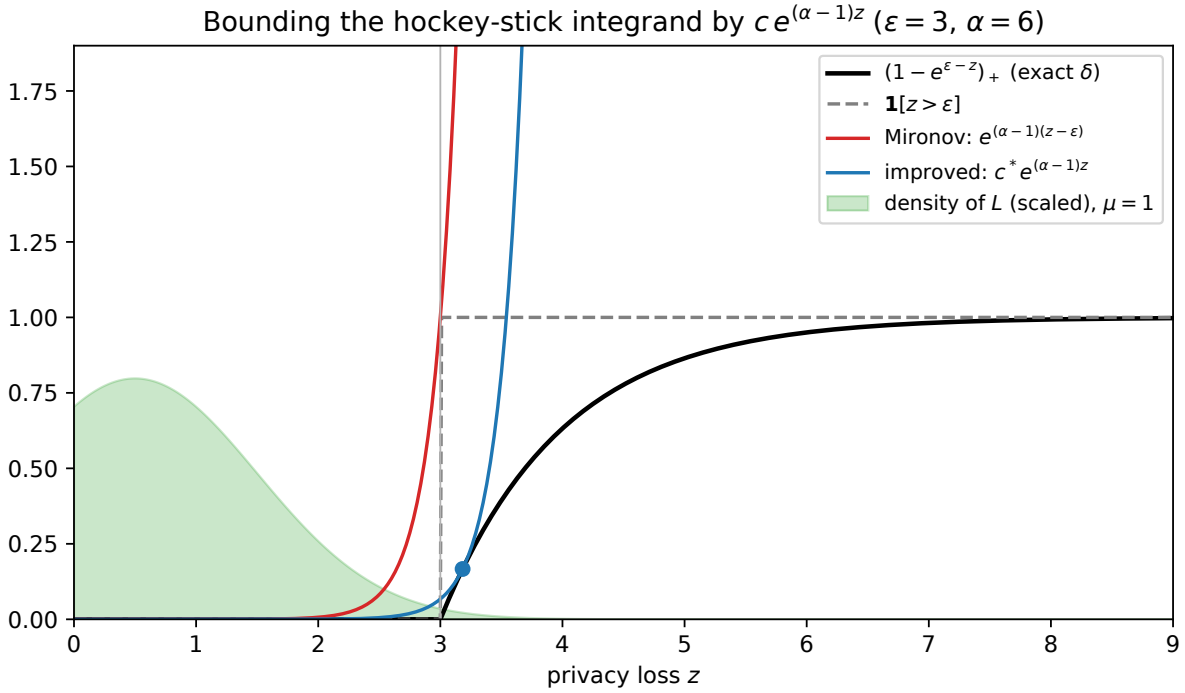


Figure 28.1.: The hockey-stick integrand $(1 - e^{\varepsilon - z})_+$ (black) at $\varepsilon = 3$, the indicator $\mathbf{1}[z > \varepsilon]$ that Mironov's step 1 replaces it with (dashed), and the two exponentials of order $\alpha = 6$ that bound it: Mironov's, through the corner of the indicator (red), and the smallest one, which touches the integrand at $z^* = \varepsilon - \ln(1 - 1/\alpha)$ (blue). The shaded curve is the density of the Gaussian privacy loss at $\mu = 1$: the conversions average the exponentials against it.

the $\ln(1/\delta)$ terms. What is left is how much smaller the improved ε' is:

$$\Delta(\alpha) = \frac{\ln \alpha}{\alpha - 1} + \ln \frac{\alpha}{\alpha - 1}, \tag{28.10}$$

whatever the mechanism and whatever δ. On the δ side, the improved δ is Mironov's times $\frac{1}{\alpha}(1-\frac{1}{\alpha})^{\alpha-1}$. Because $(1-1/\alpha)^{\alpha-1} \to 1/e$, that factor behaves like $1/(e\alpha)$ for large α. Table 28.3 gives values; at $\alpha = 2$, for instance, the factor is $\frac{1}{2} \cdot \frac{1}{2} = 0.25$ and the saving is $\ln 2 + \ln 2 = 1.3863$.

Table 28.3.: What the improved conversion gains over Mironov's at the same order.

α	δ factor	$1/(e\alpha)$	ε saving $\Delta(\alpha)$
2	0.2500	0.1839	1.3863
10	0.0387	0.0368	0.3612
100	0.0037	0.0037	0.0566

Over a grid, the saving is at least $\Delta(\alpha_M)$, where α_M is the order Mironov's conversion picks, for any RDP curve. The reason is that the improved conversion takes its own best order, so its minimum is at most its value at α_M, and that value is Mironov's minimum minus $\Delta(\alpha_M)$ by Equation 28.10. So you can read the saving of any accountant off Table 28.3 from its best order: about 1.39 at order 2, 0.36 at order 10, 0.06 at order 100. For the Gaussian at $\mu = 1$, Mironov picks order 5.8, $\Delta(5.8) = \frac{\ln 5.8}{4.8} + \ln \frac{5.8}{4.8} = 0.555$, and the actual saving is 0.570, a little more because the improved conversion moves to a better order. If two accountants report ε about 0.4 apart from the same curve at a best order near 10, one of them is using Mironov's conversion. Opacus and `dp_accounting` both use the improved one.

The improved conversion also beats both of Mironov's steps, not only the second. For the Gaussian at $\mu = 1$ it gives 4.729 (at $\alpha = 5.4$), below even the 4.765 of the tail $P[L > \varepsilon] = \delta$ after step 1 in Table 28.2. That is possible because it never replaces the integrand by an indicator: its exponential is allowed to dip below 1 just above ε, where the integrand is small. At $\mu = 10$ it gives 96.116 against Mironov's 98.026 and the exact 91.817. At $\varepsilon = 4.729$ the exact Gaussian δ is 2.097×10^{-6}, so the guarantee still holds with room to spare.

For one order, it is close to optimal. The next section computes the best possible conversion from the RDP values on a whole grid, and a single order is the special case of a grid with one point. Take only the forward constraint $D_\alpha(P\|Q) \le \varepsilon(\alpha)$ at one order, as the improved conversion does, at $\mu = 1$. At order 2, where $\varepsilon(2) = 1$, Mironov gives $1 + \ln(10^5) = 12.513$, the improved conversion $1 + \ln \frac{1}{2} + \ln(10^5) - \ln 2 = 11.127$, and the optimum 10.668. At orders 8 and 32 the improved conversion and the optimum agree to three decimals (5.214 and 16.228). Exercise 6 in the exercise book explains why the gap appears only at small orders.

Balle et al. derive the same formula geometrically, as a tangent line. In the plane of test errors (a, b), the (ε, δ) constraint $1 - a \le e^\varepsilon b + \delta$ is a straight line, and the best line that keeps a relaxed version of the region RDP allows (Section 28.5) on its feasible side touches the region's boundary. Figure 28.1 shows the same tangency in the privacy-loss variable: the blue exponential touches the integrand at z^*.

28.4.5. Example: the same release, converted better

For the mean released with $\sigma = 1$, the improved conversion over the grid gives $(4.729, 10^{-5})$-DP at order 5.4. Mironov's gave 5.299 at order 5.8, and the exact value is 4.377. The improved conversion removes 0.570 of the 0.921 excess, at no cost: it uses the same RDP curve and one extra line of code.

28.4.6. Check yourself

- Why must the bounding function be an exponential in z, rather than, say, a polynomial that hugs the integrand more closely? *(Fact 2 only tells you $\mathbb{E}_P[e^{(\alpha-1)L}]$. To bound $\mathbb{E}_P[g(L)]$ you need $g \le c\, e^{(\alpha-1)z}$; any other g has an expectation the RDP curve does not determine.)*
- The improved conversion can return a negative ε' when $\varepsilon(\alpha)$ is tiny. What should you report? *($\varepsilon' = 0$. $\delta(\varepsilon)$ can only fall as ε grows, so a bound at some $\varepsilon' < 0$ also holds at 0. The script clips at 0, as Canonne et al. do.)*

28.5. RDP as a constraint on tests

28.5.1. Context

An (ε, δ) statement is one straight line in the plane of test errors: every test has $1 - a \le e^{\varepsilon} b + \delta$. An RDP curve says more. By Fact 3, each order constrains the pair (a, b) of every test, and the constraints for different orders bend differently. This section draws the region an RDP curve allows, following Balle et al. (2020), compares it with the Gaussian's true trade-off curve, and uses it to compute the best possible conversion. The **trade-off function** $T(P, Q)(a)$ of hypothesis testing is the smallest type II error any test can have at type I error a; it describes the most powerful possible attack at each false-positive rate.

28.5.2. Statement

The statement turns Fact 3 into an explicit inequality on (a, b) for each order, in both directions. Let M have RDP curve $\varepsilon(\cdot)$ at the orders of a grid. Every test between $P = M(D)$ and $Q = M(D')$, with type I error a and type II error b, satisfies for every order α in the grid

$$a^{\alpha}(1-b)^{1-\alpha} + (1-a)^{\alpha} b^{1-\alpha} \le e^{(\alpha-1)\varepsilon(\alpha)}, \qquad (1-b)^{\alpha} a^{1-\alpha} + b^{\alpha}(1-a)^{1-\alpha} \le e^{(\alpha-1)\varepsilon(\alpha)}. \tag{28.11}$$

Call the set of (a, b) satisfying all of these the **RDP privacy region**, and let $f_{\mathrm{RDP}}(a)$ be the smallest b in it at type I error a. Two claims follow.

1. **Trade-off bound.** $T(P, Q)(a) \ge f_{\mathrm{RDP}}(a)$ for every a: no test is more powerful than the region allows.
2. **Optimal conversion.** Every pair consistent with the curve has

$$\delta(\varepsilon) \le \delta_{\mathrm{opt}}(\varepsilon) = \max_{a} \left[1 - a - e^{\varepsilon} f_{\mathrm{RDP}}(a)\right], \tag{28.12}$$

   and some two-outcome pair consistent with the curve attains it. So δ_{opt} is the best δ that any conversion using only these RDP values can certify.

28.5.3. Derivation

The first three steps compute the Rényi divergence of a test's outcome and apply the RDP curve to it. The last two rewrite δ as a statement about tests, so that the region bounds it.

1. **The outcome of a test is a coin (Fact 3).** Table 28.4 lists the outcome's distribution under each hypothesis.

Table 28.4.: The outcome of a test with errors (a, b), as a two-point distribution under each hypothesis.

Outcome	Probability under P	Probability under Q
reject	a	$1-b$
accept	$1-a$	b

2. **Compute its Rényi divergence.** The definition $D_\alpha = \frac{1}{\alpha-1}\ln\sum p^\alpha q^{1-\alpha}$ for a two-point distribution is a sum of two terms, one per row of Table 28.4: $D_\alpha(\text{outcome under } P \,\|\, \text{outcome under } Q) = \frac{1}{\alpha-1}\ln\left[a^\alpha(1-b)^{1-\alpha} + (1-a)^\alpha b^{1-\alpha}\right]$.
3. **Apply the RDP curve.** Data processing (Fact 3) gives this divergence $\le D_\alpha(P\|Q) \le \varepsilon(\alpha)$. Multiply by $\alpha - 1 > 0$ and exponentiate to get the first inequality of Equation 28.11. The second comes the same way from $D_\alpha(Q\|P) \le \varepsilon(\alpha)$, which RDP also guarantees because swapping two neighbours gives two neighbours; it is the same sum with the columns of the table swapped. Since every test's (a, b) lies in the region, its b is at least the smallest b the region allows at its a. This proves claim 1.
4. **Write δ as a best test.** Let S be the acceptance region of a test, the outputs on which it accepts P, so $P(S) = 1-a$ and $Q(S) = b$. Fact 1 says $\delta(\varepsilon)$ is the largest $P(S) - e^\varepsilon Q(S)$ over all events, so $\delta(\varepsilon) = \max_S[P(S) - e^\varepsilon Q(S)] = \max_\phi[1 - a - e^\varepsilon b]$. Every test's (a, b) is in the region, so $b \ge f_{\text{RDP}}(a)$, and replacing b by $f_{\text{RDP}}(a)$ can only increase the expression. That gives the bound in Equation 28.12.
5. **Show that the bound is attained.** Let (a^*, b^*) maximize Equation 28.12, with $b^* = f_{\text{RDP}}(a^*)$. Build a mechanism whose output is just the coin of Table 28.4 with these errors: $M(D) = \text{Bern}(1-a^*)$ and $M(D') = \text{Bern}(b^*)$, where the outcome 1 is "accept". By step 2 its Rényi divergences in both directions are exactly the left sides of Equation 28.11 at (a^*, b^*), which lie in the region, so the mechanism satisfies the curve. The test "accept on outcome 1" has errors (a^*, b^*), so this mechanism's $\delta(\varepsilon)$ is at least $1 - a^* - e^\varepsilon b^*$, which is $\delta_{\text{opt}}(\varepsilon)$.

Computing it. For fixed a, both left sides of Equation 28.11 equal 1 at $b = 1-a$: the first becomes $a^\alpha a^{1-\alpha} + (1-a)^\alpha(1-a)^{1-\alpha} = a + (1-a)$, the second likewise. That is where the test ignores the output and the two coins are identical. Both are convex in b (each term has the form $x^\alpha y^{1-\alpha}$, which is jointly convex in (x, y) for $\alpha > 1$), and their smallest value is at $b = 1-a$, so they decrease on $[0, 1-a]$. The constraints therefore hold for b above some cut-off and fail below it, and $f_{\text{RDP}}(a)$ is that cut-off, found by bisection. The script works with logarithms of a, $1-a$, b and $1-b$, because the interesting tests have errors like 10^{-6}. The objective in Equation 28.12 is concave in a, because f_{RDP} is convex: the region is an intersection of convex sets, one per constraint, and the lower boundary of a convex set is a convex function. So a grid search refined by a one-dimensional bracketing search finds the maximum. The ε at a given δ is the maximum

over a of $\ln\left((1-a-\delta)/f_{\mathrm{RDP}}(a)\right)$, the ε at which the line through the point $(a, f_{\mathrm{RDP}}(a))$ just reaches δ.

28.5.4. Interpretation

To see how loose the region is, compare it with the truth for the Gaussian. The best test between $\mathcal{N}(0,1)$ and $\mathcal{N}(1,1)$ rejects when the output exceeds a threshold t (the Neyman–Pearson lemma: the best test thresholds the likelihood ratio, which here increases with the output). Its type I error is $a = 1-\Phi(t)$, so $t = \Phi^{-1}(1-a)$, and its type II error is the probability that an output from $\mathcal{N}(1,1)$ falls below t, which is $G_1(a) = \Phi(\Phi^{-1}(1-a)-1)$. At $a = 0.1$, $\Phi^{-1}(0.9) = 1.2816$, so $G_1 = \Phi(0.2816) = 0.6109$ and the true power is 0.3891. Figure 28.2 plots, for the Gaussian at $\mu = 1$, the largest power $1 - f_{\mathrm{RDP}}(a)$ that its RDP curve allows, against this true power $1 - G_1(a)$.

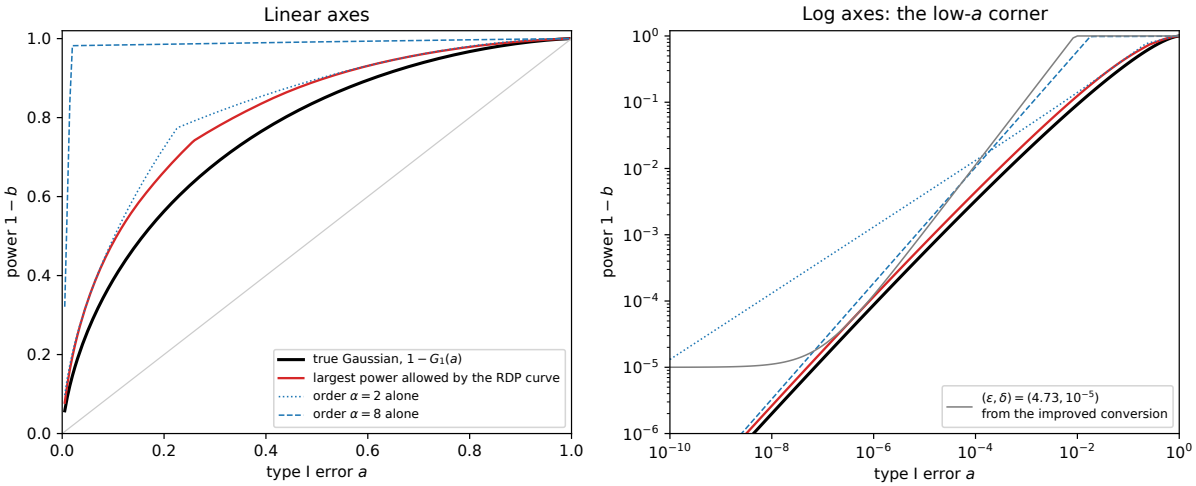


Figure 28.2.: Largest power $1-b$ at type I error a allowed by the Gaussian's RDP curve at $\mu = 1$ (red, all orders of the grid together), by the single orders 2 and 8 (blue), and achieved by the Gaussian itself (black). Left: linear axes. Right: log axes, with the (ε, δ) line $1-b \le e^{\varepsilon}a + \delta$ from the improved conversion (grey), which touches the red curve.

Each order guards a different range of a. For small a and b near 1, the term $(1-b)^{\alpha}a^{1-\alpha}$ of the second inequality of Equation 28.11 is by far the larger, because $a^{1-\alpha}$ blows up as $a \to 0$. Keeping only that term and solving for $1-b$ gives $1-b \lesssim e^{(\alpha-1)\varepsilon(\alpha)/\alpha}\, a^{(\alpha-1)/\alpha}$. On log axes this is a line of slope $(\alpha-1)/\alpha$, with intercept $(\alpha-1)\varepsilon(\alpha)/\alpha = (\alpha-1)/2$ when $\varepsilon(\alpha) = \alpha/2$. Order 2 has slope $\frac{1}{2}$ and is weak at small a: at $a = 10^{-6}$ it allows power 1.3118×10^{-3}, while order 8 (slope $\frac{7}{8}$) allows 1.8622×10^{-4}. In the bulk the roles swap: at $a = 0.1$, order 2 allows 0.4932 and order 8 allows 0.9838. The two lines meet where $\frac{1}{2} + \frac{1}{2}\ln a = \frac{7}{2} + \frac{7}{8}\ln a$, that is $-3 = \frac{3}{8}\ln a$, at $a = e^{-8} \approx 3.35 \times 10^{-4}$. The lines are bounds on the boundaries, not the boundaries themselves, because they drop a term. Order 8's line is exact at small a, but order 2's overstates its power: at $a = 10^{-6}$ it gives $e^{1/2} \cdot 10^{-3} \approx 1.6487 \times 10^{-3}$ against the exact 1.3118×10^{-3}. So the exact boundaries of the two orders cross a little lower, at $a \approx 1.87 \times 10^{-4}$, the value the script finds by solving for equal power. At $a = 10^{-3}$ neither is binding: the binding order is 3.7, which alone allows 2.4910×10^{-2}, the grid's value. Large orders guard small a, small orders the bulk.

The RDP region is a good but loose summary of the trade-off curve. Table 28.5 compares powers. The curve allows 1.24 to 1.36 times the power the Gaussian has, at every a shown; at $a = 0.1$, for instance, $0.48376/0.38914 = 1.24$.

Table 28.5.: Power of the best test against the Gaussian mechanism at $\mu = 1$, true and allowed by its RDP curve.

Type I error a	True power $1 - G_1(a)$	Largest power allowed by the RDP curve	Ratio
0.1	3.8914×10^{-1}	4.8376×10^{-1}	1.24
10^{-3}	1.8298×10^{-2}	2.4910×10^{-2}	1.36
10^{-6}	8.7218×10^{-5}	1.1636×10^{-4}	1.33

The RDP region is not an (ε, δ) region. The grey line in Figure 28.2 is the improved conversion's guarantee. It touches the red curve at one small a and lies far above it elsewhere, because an (ε, δ) pair is a single straight line, and the best it can do is be tangent to the curved region at one point. A whole trade-off curve is the envelope of such lines, one for each ε, which is the duality that Gaussian DP develops.

Nor does the region characterize RDP. A test squeezes the output into a single bit, and the Rényi divergence of a pair is in general larger than the largest Rényi divergence of any single test's outcome: information spread over many outputs is lost when they are merged into "accept" and "reject". So a pair can have every test inside the region and still violate the RDP curve. The region is what RDP implies about tests, not an equivalent definition.

28.5.5. Example: a membership test against a released statistic

An auditor runs a membership test against the mean released with $\sigma = 1$ and wants a false-positive rate of $a = 10^{-3}$. The RDP curve alone guarantees that no test detects membership with probability above 2.4910×10^{-2}. The Gaussian's exact trade-off curve says the best test reaches 1.8298×10^{-2}. The $(4.729, 10^{-5})$ guarantee from the improved conversion only says power $\leq e^{\varepsilon} a + \delta$, which is $e^{4.729} \times 10^{-3} + 10^{-5} = 0.1131$, more than four times the region's bound. Reading an RDP result through one (ε, δ) pair throws away most of what it says about low false-positive rates.

28.5.6. Check yourself

- Why are there two inequalities per order in Equation 28.11? *(One for $D_\alpha(P\|Q)$ and one for $D_\alpha(Q\|P)$. RDP bounds both, because the neighbouring relation is symmetric; the first matters for tests with a near 1, the second for a near 0.)*
- Why is the optimum in Equation 28.12 attained by a two-outcome mechanism, rather than needing a richer one? *(Any mechanism's δ is the best value of $1 - a - e^{\varepsilon} b$ over its tests, and each test only sees a coin. The coin with the extreme errors is itself a mechanism with the same constraints.)*

28.6. Choosing the order

28.6.1. Context

Both conversions hold at every order, so you take the best one. Each converted ε' is the RDP curve, which grows with the order, plus a term in $\ln(1/\delta)/(\alpha-1)$, which shrinks with it, so the best order balances the two. For the Gaussian the curve is linear in α, and the best order has a closed form or nearly so. The formulas show how the best order moves with the number of steps and with δ, and that tells you what an accountant's grid of orders must contain.

28.6.2. Statement

Let $\varepsilon(\alpha) = \rho\alpha$, with slope $\rho = \mu^2/2 = k/(2\sigma^2)$ for the composed Gaussian. Over continuous $\alpha > 1$, the best orders are as follows. For **Mironov's conversion**, the best order is $\alpha^* = 1 + \sqrt{\ln(1/\delta)/\rho}$, and $\varepsilon^* = \rho + 2\sqrt{\rho \ln(1/\delta)}$. For the **improved conversion**, the best order solves

$$\rho(\alpha-1)^2 = \ln\frac{1}{\alpha\delta}, \tag{28.13}$$

which the script solves by iterating $\alpha \leftarrow 1 + \sqrt{\ln(1/(\alpha\delta))/\rho}$ from Mironov's α^*. The only difference from Mironov's condition is the extra α inside the logarithm, which pulls the best order slightly lower.

28.6.3. Derivation

Each step sets a derivative to zero, as for any smooth minimum.

1. **Mironov.** The derivative of $\rho\alpha + \ln(1/\delta)/(\alpha-1)$ is $\rho - \ln(1/\delta)/(\alpha-1)^2$. Setting it to zero gives $(\alpha-1)^2 = \ln(1/\delta)/\rho$, which is α^*. Substitute back: $\rho\alpha^* = \rho + \rho\sqrt{\ln(1/\delta)/\rho} = \rho + \sqrt{\rho\ln(1/\delta)}$, and the second term is $\ln(1/\delta)/\sqrt{\ln(1/\delta)/\rho} = \sqrt{\rho\ln(1/\delta)}$ too. At the optimum the two terms contribute equally beyond ρ.
2. **Improved, derivative.** Write Equation 28.7 as $\rho\alpha + \ln(1-\frac{1}{\alpha}) + \frac{\ln(1/\delta)-\ln\alpha}{\alpha-1}$ and differentiate term by term: $\frac{d}{d\alpha}\ln(1-\frac{1}{\alpha}) = \frac{1}{\alpha(\alpha-1)}$, and $\frac{d}{d\alpha}\frac{\ln(1/\delta)-\ln\alpha}{\alpha-1} = -\frac{1}{\alpha(\alpha-1)} - \frac{\ln(1/\delta)-\ln\alpha}{(\alpha-1)^2}$ by the quotient rule. The $\frac{1}{\alpha(\alpha-1)}$ terms cancel, leaving $\rho - \frac{\ln(1/(\alpha\delta))}{(\alpha-1)^2}$. Setting it to zero gives Equation 28.13.
3. **It is a minimum.** The derivative of that expression is $\frac{1}{\alpha(\alpha-1)^2} + \frac{2\ln(1/(\alpha\delta))}{(\alpha-1)^3}$, which is positive whenever $\alpha < 1/\delta$, so the objective is convex there and the stationary point is its minimum.
4. **Scaling.** Ignoring the slowly varying $\ln\alpha$, $\alpha^* - 1 \approx \sqrt{\ln(1/\delta)/\rho} = \sigma\sqrt{2\ln(1/\delta)/k}$. More steps or less noise push the best order toward 1; smaller δ pushes it up, but only through $\sqrt{\ln(1/\delta)}$.

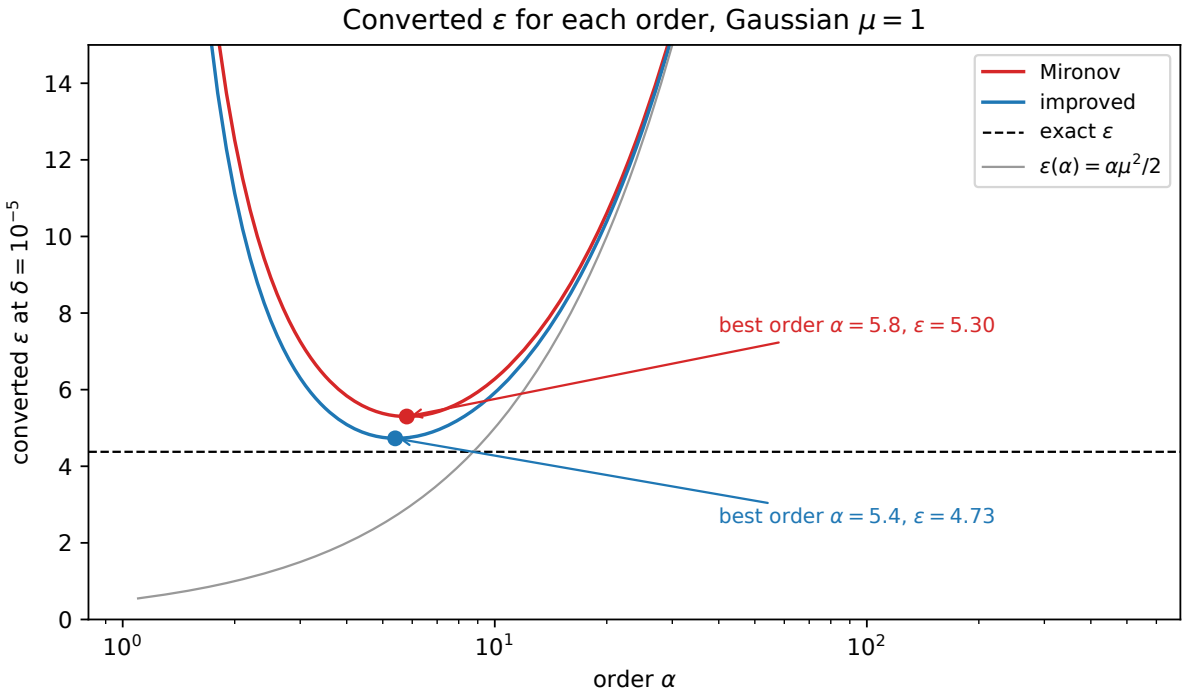


Figure 28.3.: Converted ε at $\delta = 10^{-5}$ for each order of the grid, for the Gaussian at $\mu = 1$, with Mironov's conversion (red) and the improved one (blue). Dots mark the best order of each. The dashed line is the exact ε, and the grey curve is the RDP curve $\varepsilon(\alpha) = \alpha/2$ that both conversions add to.

28.6.4. Interpretation

Figure 28.3 shows the two converted $\varepsilon'(\alpha)$ for $\mu = 1$. Both are U-shaped: at small α the $\ln(1/\delta)/(\alpha - 1)$ term explodes, at large α the curve $\varepsilon(\alpha)$ itself does. The improved curve sits below Mironov's everywhere by $\Delta(\alpha)$ of Equation 28.10, and its minimum is at a slightly smaller order.

Table 28.6 shows how the best order moves over four settings. In the first row, for instance, $\rho = 1/(2 \cdot 100) = 0.005$, and Mironov's best order is $1 + \sqrt{\ln(10^5)/0.005} = 1 + \sqrt{2302.6} = 48.985$; the improved conversion's extra $\ln \alpha$ brings it down to 40.525.

Table 28.6.: Best orders for the composed Gaussian at $\delta = 10^{-5}$.

σ	k	ρ	Best order, improved (continuous / grid)	ε continuous / grid	Mironov's best order
10	1	0.005	40.525 / 41	0.375 / 0.375	48.985
1	1	0.5	5.432 / 5.4	4.728 / 4.729	5.799
1	100	50	1.472 / 1.5	96.035 / 96.116	1.480
1	10,000	5000	1.048 / 1.1	5475.791 / 5611.778	1.048

The grid needs orders near 1 for long runs with little noise. At $\rho = 5000$ the best order is 1.048, below the grid's smallest order 1.1, and the grid's answer is $5611.778 - 5475.791 = 135.987$ larger than the continuous optimum. (That setting is a huge ε anyway; Exercise 5 in the exercise book measures the relative loss.)

The grid also needs large orders, for small total ρ. One step at $\sigma = 10$ already wants order 40.525, and at $\rho = 5 \times 10^{-5}$ the best order is 338.312 (Exercise 5 in the exercise book). Subsampled mechanisms (subsampling), which use each record only in a random fraction of the steps, have tiny curves at small orders, which is why the grid ends at 128, 256 and 512.

The value of δ matters little, because it enters only through $\sqrt{\ln(1/\delta)}$. At $\mu = 1$, moving from $\delta = 10^{-5}$ to 10^{-10} doubles $\ln(1/\delta)$ but moves the improved best order only from 5.432 to 7.483.

28.6.5. Example: tuning a grid for a training run

You plan $k = 100$ full-batch noisy gradient steps at $\sigma = 1$, so $\rho = 100/2 = 50$. Equation 28.13 puts the best order at 1.472, and the grid's 1.5 costs $96.116 - 96.035 = 0.081$ in ε. If you instead plan a single release at $\sigma = 10$, the best order is 40.525, and the grid's integer orders match it to three decimals of ε. One grid serves both because it is dense near 1 and runs from 1.1 to 512.

28.6.6. Check yourself

- Why does the best order go to 1 as $k \to \infty$? *(The curve's slope ρ grows with k, so the $\rho\alpha$ term punishes large orders more, while the $\ln(1/\delta)/(\alpha-1)$ term does not change. The balance point $\alpha - 1 \approx \sqrt{\ln(1/\delta)/\rho}$ shrinks like $1/\sqrt{k}$.)*
- If an accountant reports the best order at the edge of its grid, what should you do? *(Extend the grid in that direction: the true optimum lies outside, and the reported ε is larger than necessary.)*

28.7. The slack no conversion removes

28.7.1. Context

The improved conversion is better than Mironov's, and the optimal conversion of Equation 28.12 is better still, in principle. Yet both stay above the exact ε of the Gaussian. Is that a weakness of the formulas, or of the summary? This section shows it is the summary: a different mechanism satisfies the same RDP curve and has a larger δ, and a conversion that sees only the curve must cover both. It also looks at the opposite direction, from (ε, δ) to RDP.

28.7.2. Statement

The three claims are about what the RDP curve can and cannot carry.

1. **The curve does not determine δ.** For the Gaussian, the optimal conversion from the RDP values on the grid agrees with the improved conversion to three decimals, and both exceed the exact ε by 5 to 10% (Table 28.8). The excess is real: a two-outcome mechanism satisfies the Gaussian's RDP curve $\alpha/2$ at every order $\alpha > 1$ and has $\delta(\varepsilon) = 4.4698 \times 10^{-5}$ at the Gaussian's exact $\varepsilon = 4.377$.
2. **(ε, δ)-DP with $\delta > 0$ implies no RDP bound at any order.**
3. **ε-DP implies $(\alpha, \min(\varepsilon, \alpha\varepsilon^2/2))$-RDP for every $\alpha > 1$** (Bun & Steinke 2016, Proposition 3.3). The constant $\frac{1}{2}\varepsilon^2$ can be sharpened to $\varepsilon \tanh(\varepsilon/2)$ (Equation 29.7 in concentrated DP).

28.7.3. Derivation

Claim 1. The idea is to build the worst mechanism the curve allows and compute its δ. By Equation 28.12, the largest δ consistent with a set of orders is attained by the two-outcome pair at the maximizer. On the part's grid, at $\mu = 1$ and $\varepsilon = 4.377$, that pair has $\delta = 4.4700 \times 10^{-5}$. But it satisfies the curve only at the grid's orders: its Rényi divergence is a smooth function of the order, and between two grid orders it can poke above $\alpha/2$. To get a pair that satisfies the curve at every real order, the script maximizes over a dense set of orders (steps of 0.002 from 1.01 to 30, and every integer up to 512) and then raises $Q(\text{accept})$ by a relative 10^{-5}, which pushes the pair strictly inside the curve. Table 28.7 shows the result. The privacy loss column is $\ln(P/Q)$ for each outcome: $\ln(2.2779 \times 10^{-4}/2.2998 \times 10^{-6}) = 4.5956$ for "accept", and for "reject" a tiny negative number, because P puts slightly less mass there than Q.

Table 28.7.: A two-outcome mechanism that satisfies the Gaussian's RDP curve at $\mu = 1$ at every order and has nearly the largest δ at $\varepsilon = 4.377$ that any such mechanism can have.

Outcome	$P = M(D)$	$Q = M(D')$	Privacy loss L
accept	2.2779×10^{-4}	2.2998×10^{-6}	4.5956
reject	rest	rest	-2.255×10^{-4}

1. **Check that it satisfies the curve.** On a scan of α from $1 + 10^{-8}$ to 10^6 (steps of 10^{-4} between 2 and 30), refined around the maximum, the largest value of $D_\alpha(P\|Q) - \alpha/2$ is -9.97×10^{-6}, at $\alpha = 5.0955$; it is negative, so the curve holds everywhere scanned. The reverse divergences $D_\alpha(Q\|P)$ stay about 0.5 below the curve, closest as $\alpha \to 1$. The limits are safe too: as $\alpha \to \infty$, $D_\alpha(P\|Q)$ tends to the largest privacy loss, 4.5956, while $\alpha/2$ grows without bound. The rounded values in the table also satisfy the curve, with largest excess -1.93×10^{-5}.
2. **Compute its δ.** By Equation 28.1 only outcomes with $L > \varepsilon$ contribute, and here that is only "accept". So $\delta(4.377) = P(\text{accept}) - e^{4.377}\,Q(\text{accept})$; with $e^{4.377} \approx 79.6$ this is 2.2779×10^{-4} minus about 1.831×10^{-4}, which is 4.4698×10^{-5} with unrounded values, against the Gaussian's 10^{-5}.
3. **Conclude.** So no curve-based method can certify $(4.377, 10^{-5})$ for the Gaussian, because the same certificate would have to cover this mechanism, which has the same curve and about 4.5 times the δ. The improved conversion's δ at that ε is 4.4701×10^{-5}, within 10^{-8} of this pair's.

The pair puts almost all its mass at $L \approx 0$ and a sliver at $L = 4.5956$, near the touching point z^* of Figure 28.1. That is the distribution for which the improved conversion's exponential bound is nearly an equality, since the exponential and the integrand meet there. It is very unlike the Gaussian PRV, which spreads its tail smoothly over all large losses. The RDP curve cannot tell them apart: it only sees MGF values (Fact 2).

Claim 2. A single example suffices. Take the mechanism that outputs nothing, except that with probability δ it publishes your record. Order the pair with the dataset that holds you first, $P = M(D')$ and $Q = M(D)$, which a guarantee must also cover. Then P puts mass δ on an output that Q never produces, and otherwise matches Q. Moving mass δ can change the probability of any event by at most δ, so $P(S) \le Q(S) + \delta$ for every event, and the mechanism is $(0, \delta)$-DP. But the sum $\sum p^\alpha q^{1-\alpha}$ in the Rényi divergence contains the term $\delta^\alpha \cdot 0^{1-\alpha}$ for the published record, and $0^{1-\alpha} = \infty$ for $\alpha > 1$, so $D_\alpha(P\|Q) = \infty$ at every order. RDP, like pure DP, rules out catastrophic events however rare; (ε, δ)-DP allows them with probability δ.

Claim 3. The first bound, $\varepsilon(\alpha) \le \varepsilon$, follows directly from Fact 2. ε-DP says the privacy loss never exceeds ε, so $e^{(\alpha-1)L} \le e^{(\alpha-1)\varepsilon}$ for every output, and its average under P is at most $e^{(\alpha-1)\varepsilon}$; taking logarithms and dividing by $\alpha - 1$ gives $D_\alpha \le \varepsilon$. (Equivalently, Rényi divergence is non-decreasing in α and tends to the max-divergence, the largest privacy loss, which ε-DP bounds by ε; see divergences.) The second bound, $\alpha\varepsilon^2/2$, says an ε-DP mechanism is $\varepsilon^2/2$-zero-concentrated: the logarithm of the MGF of its privacy loss grows at most quadratically. Bun & Steinke (2016, Proposition 3.3) prove it through a two-point variable and an inequality between hyperbolic sines; concentrated DP gives a shorter proof with Hoeffding's lemma, and also proves the sharper line $\alpha\varepsilon \tanh(\varepsilon/2)$ of Equation 29.7, which randomized response attains as $\alpha \to 1$. This section uses both without proof. For randomized response at $\varepsilon = 0.5$, the bound at order 2 is

$\min(0.5,\ 2 \cdot 0.25/2) = 0.2500$, at order 4 it is $\min(0.5,\ 4 \cdot 0.25/2) = 0.5000$, and at order 10 it is $\min(0.5,\ 1.25) = 0.5000$; the script gives $D_2 = 0.2273$ against the bound 0.2500, and $D_4 = 0.3519$ and $D_{10} = 0.4473$ against 0.5000.

28.7.4. Interpretation

Table 28.8 and Figure 28.4 compare all four answers for the Gaussian with $\sigma = 10$ per step. The per-step noise is chosen so that $\mu = \sqrt{k}/10$ runs from 0.1 to 10 over $k = 1$ to 10,000, and the exact ε from 0.341 to 91.817: the range where accounting choices matter. Because everything depends only on μ, the row $k = 100$, where $\mu = 1$, repeats the single release at $\sigma = 1$ of the earlier sections.

Table 28.8.: ε at $\delta = 10^{-5}$ for the Gaussian with $\sigma = 10$ composed k times: exact, the best any conversion from the RDP grid can give, the improved conversion and Mironov's.

k	μ	Exact	Optimal from the curve	Improved	Mironov	Ratios to exact (opt. / impr. / Mir.)
1	0.100	0.341	0.375	0.375	0.485	1.10 / 1.10 / 1.42
10	0.316	1.199	1.308	1.308	1.568	1.09 / 1.09 / 1.31
100	1.000	4.377	4.729	4.729	5.299	1.08 / 1.08 / 1.21
1000	3.162	17.857	19.054	19.054	20.175	1.07 / 1.07 / 1.13
10,000	10.000	91.817	96.116	96.116	98.026	1.05 / 1.05 / 1.07

The optimal and improved conversions coincide. At $\mu = 1$ they are 4.728505 and 4.728507, a difference of 2×10^{-6}. The optimal conversion can use all orders at once, but that helps only if more than one order constrains the worst mechanism. Here it buys nothing measurable over the best single order: at $\varepsilon = 4.377$ the grid's worst pair is held back only by the constraint at order 5.1, and every other order of the grid leaves at least 0.0011 of room.

Mironov's excess shrinks fast with k, the unavoidable excess slowly. Mironov is 42% above the exact value at $k = 1$ and 7% at $k = 10{,}000$; the RDP summary costs 10% and 5%. Both shrink in relative terms because for large μ the $\rho\alpha$ part, which every method shares, dominates.

When the slack matters, change the summary, not the formula. To get the exact 4.377 you must track more than an MGF: the trade-off function (Gaussian DP) or the full privacy loss distribution (numerical accounting).

The 5 to 10% is for the plain Gaussian. The differentially private version of stochastic gradient descent (SGD), DP-SGD, composes subsampled Gaussian steps, in which each step uses a random fraction q of the records, and their RDP curve is not a line. Subsampling applies the same conversions to the part's running example ($q = 0.01$, $\sigma = 1$): 7.43 with Mironov's conversion and

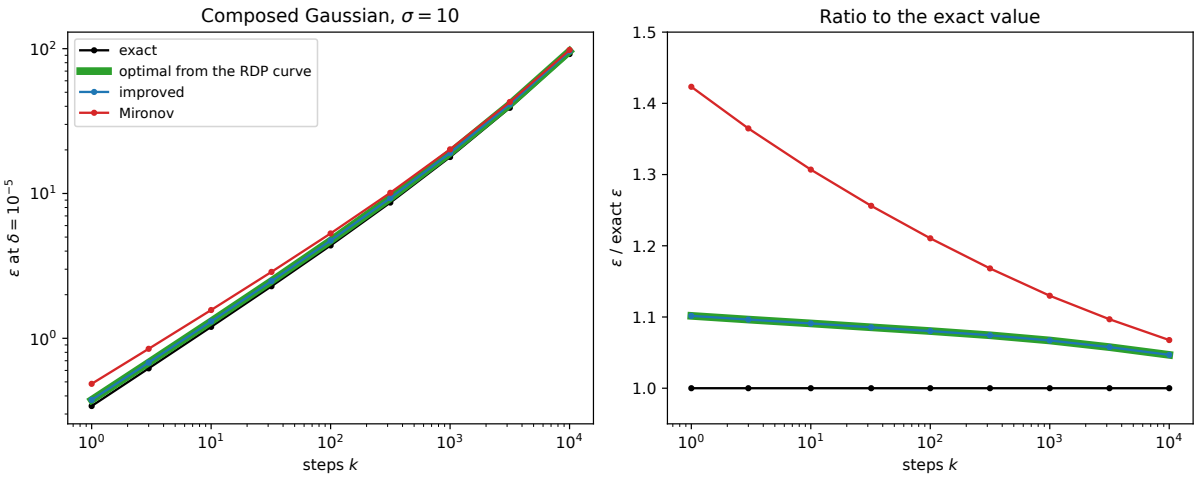


Figure 28.4.: Left: ε at $\delta = 10^{-5}$ for the Gaussian with $\sigma = 10$ composed k times, exact (black), optimal from the RDP curve (wide green), improved (blue, on top of the green) and Mironov (red). Right: each divided by the exact value. The improved conversion is optimal for the curve; the remaining 5 to 10% belongs to the RDP summary itself.

6.71 with the improved one at $k = 10{,}000$. There the gap to the exact value can be much larger, and choosing an accountant measures it.

28.7.5. Example: what to tell a reviewer

Your paper reports $(4.729, 10^{-5})$-DP from an RDP accountant for a mechanism that is in fact a Gaussian with $\mu = 1$. A reviewer asks whether a better conversion would lower it. The answer is no. The two-outcome mechanism of Table 28.7 satisfies the same RDP curve and has $\delta = 4.4698 \times 10^{-5}$ at $\varepsilon = 4.377$, so any certificate computed from the curve alone must also hold for it. The same construction at $\delta = 10^{-5}$ gives a pair inside the curve at every order whose ε is 4.728376, and the improved conversion at the best real order, 5.4318, gives 4.728387. The first is a mechanism that a certificate must cover, so the best certificate is at least 4.728376; the second is a valid certificate, so the best is at most 4.728387. So the best value any method can certify from the curve at every real order lies between 4.728376 and 4.728387, which is 4.7284 to four decimals; a method that sees only the grid's orders must report at least 4.728505. A lower number needs an accountant that sees the mechanism, and for the Gaussian that number is 4.377.

28.7.6. Check yourself

- Does the two-outcome mechanism contradict the Gaussian's $\delta = 10^{-5}$ at $\varepsilon = 4.377$? *(No. It is a different mechanism that satisfies the same RDP curve. The conversion must cover both, so it cannot certify the smaller δ for either.)*
- A mechanism is $(1, 10^{-8})$-DP. What RDP curve does that give you? *(None: $\delta > 0$ allows a rare event with infinite privacy loss, and claim 2 shows that makes every D_α infinite. You need to know how the δ arises, for example from Gaussian noise, to get an RDP curve.)*

28.8. Code: check every conversion by computation

The script code/rdp_conversions.py computes every number in this chapter and draws its figures. The conversions are small typed functions: `rdp_to_dp_mironov` and `rdp_to_dp_improved` return the best ε and order over a grid, `rdp_to_delta_improved` goes the other way, `rdp_tradeoff_lower_bound` computes f_{RDP} by bisection in logarithms, `rdp_to_dp_tradeoff` maximizes Equation 28.12, `continuous_worst_pair` builds the two-outcome pair of Table 28.7 and `max_gaussian_curve_excess` checks it against the curve at every real order, and `gaussian_delta_exact` and `gaussian_eps_exact` give the exact Gaussian answers. From the repository root:

```
uv run python code/part04-privacy-accounting/rdp_conversions.py
```

Output (seed 0; nothing is random):

Table 28.9.: Results printed by the script.

Experiment	Result
Fact 1, Gaussian $\mu = 1$, $\varepsilon = 1$	closed form 1.2694×10^{-1}; Riemann sum 1.2694×10^{-1}
Mironov at $\mu = 1$	5.299 at order 5.8; continuous optimum 5.299 at 5.799
Optimal constant at $\alpha = 6$, $\varepsilon = 4$	formula 1.380552×10^{-10}; grid 1.380552×10^{-10}
Improved at $\mu = 1$	4.729 at order 5.4; exact 4.377
Single order 2, $\varepsilon(2) = 1$	Mironov 12.513, improved 11.127, optimal 10.668
Power at $a = 10^{-3}$, $\mu = 1$	true 1.8298×10^{-2}; RDP allows 2.4910×10^{-2}
Best order, $\rho = 5000$	continuous 1.048; grid 1.1
Optimal from the curve vs improved, $\mu = 1$	4.728505 and 4.728507
Worst two-outcome pair at $\varepsilon = 4.377$	grid orders only: $\delta = 4.4700 \times 10^{-5}$; every real order: $\delta = 4.4698 \times 10^{-5}$, largest excess over $\alpha/2$ is -9.97×10^{-6}
Worst pair at $\delta = 10^{-5}$, every real order	$\varepsilon = 4.728376$; improved at order 5.4318: 4.728387
Randomized response $\varepsilon = 0.5$, order 2	$D_2 = 0.2273 \leq 0.2500$

tests/test_rdp_conversions.py checks that the output still matches Table 28.9 and every other number in the text.

What to take from it. The improved conversion is free and nearly optimal: it agrees with the optimal conversion from the curve to three decimals at every k of Table 28.8, and at $\mu = 1$ the two differ by 2×10^{-6}. The gap that remains to the exact value is real, because a concrete two-outcome mechanism with the same curve realizes it, so no formula can close it. What still matters is the grid, where it is sparse or ends: very long runs push the best order below 1.1, and very short ones push it into the sparse stretch between 63 and 512, or beyond 512.

28.9. References

- Mironov, "Rényi Differential Privacy" (IEEE CSF 2017): the definition, composition, the Gaussian and Laplace curves, and the conversion of Equation 28.6 (Proposition 3).
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020; arXiv:1905.09982): privacy regions of divergences, the test region of Equation 28.11 (§5.1.1), and the improved conversion as a tangent line (Theorem 21).
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020; arXiv:2004.00010): the short proof of the improved conversion through the optimal constant c^* (Proposition 12), and its use for concentrated DP (Corollary 13).
- Asoodeh, Liao, Calmon, Kosut & Sankar, "A Better Bound Gives a Hundred Rounds: Enhanced Privacy Guarantees via f-Divergences" (IEEE International Symposium on Information Theory, ISIT 2020; arXiv:2001.05990): an independent proof of the improved conversion (Lemma 1, Eq. 20), applied to noisy SGD.
- Asoodeh, Liao, Calmon, Kosut & Sankar, "Three Variants of Differential Privacy: Lossless Conversion and Applications" (IEEE Journal on Selected Areas in Information Theory, 2021; arXiv:2008.06529): the optimal conversion from one Rényi order through the joint range of two f-divergences, and RDP as a trade-off between type I and type II errors.
- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016): ε-DP implies $\varepsilon^2/2$-zero-concentrated DP (Proposition 3.3) and the standard zero-concentrated conversion.
- Steinke, "Composition of Differential Privacy & Privacy Amplification by Subsampling" (2022 survey, arXiv:2210.00597): a unified treatment of privacy loss, RDP, conversions and their tightness.
- Opacus (`opacus/accountants/analysis/rdp.py`) and Google's `dp_accounting` (`rdp_privacy_accountant.py`): production implementations of the improved conversion, citing Balle et al. Theorem 21 and Canonne et al. Proposition 12 respectively.

28.10. Cheat sheet

Table 28.10 collects every result in the chapter.

Table 28.10.: Every result in this chapter.

Result	Statement	Where it comes from
δ as an expectation	$\delta(\varepsilon) = \mathbb{E}_P[(1 - e^{\varepsilon - L})_+]$	Hockey-stick divergence
RDP as an MGF	$\mathbb{E}_P[e^{(\alpha-1)L}] = e^{(\alpha-1)D_\alpha(P\|Q)}$	Definition of D_α
Mironov	$\varepsilon' = \varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$	Indicator, then Markov
Improved	$\varepsilon' = \varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha-1}$; $\delta = \frac{e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}}{\alpha}(1 - \frac{1}{\alpha})^{\alpha-1}$	Smallest exponential above the integrand
Gain at a fixed order	$\Delta(\alpha) = \frac{\ln\alpha}{\alpha-1} + \ln\frac{\alpha}{\alpha-1}$ in ε; factor $\approx 1/(e\alpha)$ in δ	Subtracting the two

Result	Statement	Where it comes from
Test region	$a^\alpha(1-b)^{1-\alpha} + (1-a)^\alpha b^{1-\alpha} \le e^{(\alpha-1)\varepsilon(\alpha)}$, and with a, b swapped	Data processing on the test's outcome
Optimal conversion	$\delta_{\text{opt}} = \max_a [1 - a - e^\varepsilon f_{\text{RDP}}(a)]$, attained by a two-outcome pair	Region plus attainment
Best order, linear curve	Mironov $\alpha^* = 1 + \sqrt{\ln(1/\delta)/\rho}$; improved $\rho(\alpha-1)^2 = \ln(1/(\alpha\delta))$	Calculus
Unavoidable slack (Gaussian)	5 to 10% of ε at $\delta = 10^{-5}$	A two-outcome pair inside the same curve
(ε, δ) ⇒ RDP	False for any $\delta > 0$	An atom of P that Q lacks
ε-DP ⇒ RDP	$\varepsilon(\alpha) \le \min(\varepsilon, \alpha\varepsilon^2/2)$; sharply $\alpha\varepsilon \tanh(\varepsilon/2)$ in place of $\alpha\varepsilon^2/2$	Monotonicity; Bun & Steinke, proved in concentrated DP

Rules of thumb

- Always use the improved conversion. It saves $\Delta(\alpha)$, about 0.36 at order 10, and on the Gaussian it matches the best any method can do with the curve to three decimals.
- Check the best order. If it sits at either end of the grid, extend the grid.
- Expect an RDP accountant to overstate the plain Gaussian's ε by 5 to 10%, and subsampled DP-SGD's by more. If that matters, use a privacy loss distribution accountant.
- Read an RDP guarantee as a bound on tests with the whole region, not with one (ε, δ) line: at low false-positive rates the region is much tighter.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

29. Concentrated differential privacy

29.1. Why this matters

The first chapter of this part describes a mechanism by a whole curve. For two neighbouring datasets D and D', which differ in one person's record, the mechanism's outputs $M(D)$ and $M(D')$ are two probability distributions, and the **Rényi divergence** D_α of order $\alpha > 1$ measures how far apart they are; larger orders put more weight on the outputs where one distribution is much more likely than the other. The **Rényi differential privacy (RDP) curve** $\varepsilon(\alpha)$ is the largest D_α over all neighbouring pairs, one value per order. That is a lot to carry around. For the Gaussian mechanism, which adds normal noise of standard deviation σ to a query whose value moves by at most 1 between neighbours, the curve is a straight line through the origin, $\varepsilon(\alpha) = \alpha/(2\sigma^2)$, and a line through the origin is fixed by its slope. So for the Gaussian you could keep one number instead of a curve. The question of this chapter is: **when does one number describe the whole RDP curve, and what does that number buy you?**

The answer is **zero-concentrated differential privacy (zCDP)**, the most widely used form of concentrated differential privacy (CDP). A mechanism is ρ-zCDP when its RDP curve lies under the line $\rho\alpha$. Three things follow at once, and each comes from a property of the RDP curve that the line inherits. First, composition is addition. When you run several mechanisms one after another, their RDP curves add, so lines through the origin add to a line whose slope is the sum of the slopes: k steps of ρ-zCDP are $k\rho$-zCDP. Second, the privacy loss is "concentrated", which gives the definition its name. The privacy loss is the log of how much more likely an output is under D than under D', and a bound on every Rényi divergence turns out to be a bound on its moment generating function (MGF), the function $s \mapsto \mathbb{E}[e^{sL}]$ that controls how heavy the tails of a random variable L can be. The line $\rho\alpha$ says that the privacy loss has a tail no heavier than a normal variable with mean ρ and variance 2ρ. Third, the conversion to the usual (ε, δ) guarantee has a closed form, $\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}$, because the optimization over orders that the conversion requires can be done by hand for a line.

29.1.1. The one idea: a line under the curve

Everything in this chapter is about **how well a line $\rho\alpha$ fits a mechanism's RDP curve**. For the Gaussian the line is the curve, and nothing is lost. A pure differential privacy (DP) mechanism, one whose output probabilities change by at most a factor e^ε between neighbours, has a curve that bends over and flattens at ε, because no Rényi divergence can exceed the largest log-ratio of probabilities. A line through the origin fits such a curve only near $\alpha = 1$; the fit costs a little there and a lot at large orders. For the subsampled Gaussian of differentially private stochastic gradient descent (SGD), DP-SGD for short, where each step runs on a small random sample of the data, the curve is tiny at small orders and then bends sharply upwards. Any line that stays above it everywhere is as steep as the line of the Gaussian without subsampling, so

zCDP throws away all of the privacy that subsampling gained. **Truncated CDP** fixes this by asking for the line only on orders up to a cut-off ω.

Table 29.1 lists the results and where each is used.

Table 29.1.: The results of this chapter and where each is used.

Result	Where it is used
Definition of ρ-zCDP; Gaussian is $\Delta^2/(2\sigma^2)$-zCDP; composition adds ρ	Accounting for any pipeline of Gaussian mechanisms with one number, as in the 2020 US Census TopDown algorithm
ε-DP implies $(\varepsilon^2/2)$-zCDP, sharply $(\varepsilon \tanh(\varepsilon/2))$-zCDP; exponential mechanism is $(\varepsilon^2/8)$-zCDP	Mixing Laplace, randomized response and selection steps with Gaussian steps in one budget
ρ-zCDP to (ε, δ): simple and improved conversions	Reporting the final guarantee; calibrating σ to a target ε
Group privacy $k^2\rho$	Protecting households or users with several records; lower bounds
Approximate zCDP	Pipelines that include mechanisms that are only (ε, δ)-DP, such as stability-based ones
No amplification by subsampling; truncated CDP	Why DP-SGD is accounted in RDP, not zCDP; a one-number summary that does amplify

How to read this chapter. The definition and the conversion sections are the core; read them first. The pure-DP section shows how other mechanisms enter a zCDP budget. The subsampling section explains why the rest of this part works with RDP curves and privacy loss distributions instead of a single ρ.

29.2. Toolkit: four facts used below

Each fact is stated here once, with the short reason it holds and a link to where it is derived in full. Throughout, $P = M(D)$ and $Q = M(D')$ are the output distributions on two neighbouring datasets, and $P(y)$, $Q(y)$ are their probabilities (or densities) at an output y.

29.2.1. Fact 1 — Rényi divergence is a log-MGF of the privacy loss

The **privacy loss** at an output y is $\ln(P(y)/Q(y))$: how much more likely y is when the data are D than when they are D', on a log scale. Drawing the output from P makes it a random variable, the **privacy loss random variable** $L = L_{D,D'}(Y)$ with $Y \sim P$. The Rényi divergence of order α is defined as $D_\alpha(P\|Q) = \frac{1}{\alpha-1} \ln \sum_y P(y)^\alpha Q(y)^{1-\alpha}$. Write $P(y)^\alpha Q(y)^{1-\alpha} = P(y)\,(P(y)/Q(y))^{\alpha-1} =$

$P(y)\,e^{(\alpha-1)L(y)}$: the sum is then an average over $Y \sim P$ of $e^{(\alpha-1)L}$. So the Rényi divergence, times $\alpha - 1$, is the log of the MGF of the privacy loss at the point $\alpha - 1$:

$$(\alpha-1)D_\alpha(P\|Q) = \ln \mathbb{E}_P[e^{(\alpha-1)L}]. \tag{29.1}$$

This is Equation 3.12 in Divergences, with the log-likelihood ratio renamed the privacy loss.

29.2.2. Fact 2 — RDP composes by adding curves, and survives post-processing

If M_1 has RDP curve $\varepsilon_1(\alpha)$ and M_2, possibly chosen after seeing the output of M_1, has curve $\varepsilon_2(\alpha)$, the pair has curve at most $\varepsilon_1(\alpha)+\varepsilon_2(\alpha)$. The reason is that the sum $\sum P^\alpha Q^{1-\alpha}$ over pairs of outputs (y_1, y_2) factors: for each first output y_1, the inner sum over y_2 is at most $e^{(\alpha-1)\varepsilon_2(\alpha)}$, whatever M_2 was chosen, and the remaining sum over y_1 is at most $e^{(\alpha-1)\varepsilon_1(\alpha)}$. Exponents multiply, so the divergences add. Also, any function of the output has curve at most $\varepsilon(\alpha)$: this is the data-processing inequality, which says that passing two distributions through the same random map can only make them harder to tell apart. Both are derived in Rényi DP, from additivity and the data-processing inequality for D_α.

29.2.3. Fact 3 — Mironov's conversion from RDP to (ε, δ)

A mechanism is (ε, δ)-DP when, for every set S of outputs, $P(S) \le e^\varepsilon Q(S) + \delta$. One way to get there is to show that the privacy loss exceeds ε with probability at most δ: on outputs where $L \le \varepsilon$ you have $P(y) \le e^\varepsilon Q(y)$, and the rest of S has P-probability at most δ. Markov's inequality applied to $e^{(\alpha-1)L}$ bounds that probability, $\Pr[L > \varepsilon] \le \mathbb{E}[e^{(\alpha-1)L}]e^{-(\alpha-1)\varepsilon} \le e^{(\alpha-1)(\varepsilon(\alpha)-\varepsilon)}$ by Fact 1, and setting the right side equal to δ and solving for ε gives the conversion. A mechanism with $\varepsilon(\alpha)$ at order α is $(\varepsilon(\alpha) + \ln(1/\delta)/(\alpha-1), \delta)$-DP, for every $\delta \in (0, 1)$. Every order gives a valid guarantee, and you pick the order that makes ε smallest. An improved conversion, which avoids the slack of Markov's inequality, subtracts $\ln(\alpha)/(\alpha-1) - \ln(1-1/\alpha)$ from this. Both are derived in From Rényi DP to (ε, δ) and to tests.

29.2.4. Fact 4 — Hoeffding's lemma

A random variable confined to an interval cannot have heavy tails, so its MGF is no larger than that of a normal variable with the same mean and a variance set by the interval's length. Precisely: if $X \in [a, b]$, then $\ln \mathbb{E}[e^{\lambda X}] \le \lambda\mathbb{E}[X] + \lambda^2(b-a)^2/8$ for every real λ. (A normal variable with variance v has $\ln \mathbb{E}[e^{\lambda X}] = \lambda\mathbb{E}[X] + \lambda^2 v/2$, so this is the normal bound with $v = (b-a)^2/4$.) Derived in Concentration.

29.3. The definition of zCDP

29.3.1. Context

You add Gaussian noise to a gradient, a count or a mean, again and again, and you want one number that says how much privacy you have spent so far. The RDP curve of the Gaussian is a line through the origin, and lines add. Bun and Steinke turned this observation into a definition:

describe a mechanism by the slope of a line that lies above its RDP curve. The definition also has a reading in terms of the privacy loss itself, and that reading gives the name.

29.3.2. Statement

A mechanism M is **ρ-zero-concentrated differentially private** (ρ-zCDP) if its RDP curve lies below the line of slope ρ: for all neighbouring $D \simeq D'$ and all orders $\alpha > 1$,

$$D_\alpha(M(D) \,\|\, M(D')) \le \rho\alpha. \tag{29.2}$$

Multiplying both sides by $\alpha - 1$ and using Fact 1 turns this into a statement about the privacy loss random variable L: its MGF at every point $\alpha - 1 > 0$ is bounded,

$$\ln \mathbb{E}\big[e^{(\alpha-1)L}\big] \le (\alpha-1)\alpha\rho \quad \text{for all } \alpha > 1. \tag{29.3}$$

A bounded MGF limits how often L can be large, so the privacy loss has a Gaussian-type upper tail: the chance that it exceeds its mean ρ by more than λ falls off like a normal tail, as $e^{-\lambda^2/(4\rho)}$,

$$\Pr[L > \rho + \lambda] \le e^{-\lambda^2/(4\rho)} \quad \text{for all } \lambda > 0. \tag{29.4}$$

Three properties make the definition useful. The Gaussian mechanism, which adds $\mathcal{N}(0, \sigma^2)$ to a query with ℓ_2 sensitivity $\Delta = \Delta_2 f$ (the largest Euclidean distance between the query's values on neighbouring datasets), is $\Delta^2/(2\sigma^2)$-zCDP. With the part's noise multiplier convention, which measures σ in units of the sensitivity ($\Delta = 1$), $\rho = 1/(2\sigma^2)$. Composition adds the parameters: if M_1 is ρ_1-zCDP and M_2 is ρ_2-zCDP, possibly chosen adaptively, the pair is $(\rho_1 + \rho_2)$-zCDP. And post-processing is free: any function of a ρ-zCDP output is ρ-zCDP.

29.3.3. Derivation

The first three steps show that the three displays say the same thing at increasing strength; the last two prove the three properties.

1. **The two forms agree.** Multiply Equation 29.2 by $\alpha - 1$, which is positive, so the inequality keeps its direction. By Fact 1 the left side becomes $\ln \mathbb{E}[e^{(\alpha-1)L}]$, and the right side is $(\alpha-1)\alpha\rho$.
2. **The tail bound is a Chernoff bound.** To bound the chance that L is large, apply Markov's inequality to the positive variable $e^{(\alpha-1)L}$, which is large exactly when L is. For any threshold t, $\Pr[L > t] = \Pr[e^{(\alpha-1)L} > e^{(\alpha-1)t}] \le \mathbb{E}[e^{(\alpha-1)L}]\, e^{-(\alpha-1)t}$. By Equation 29.3 this is at most $\exp\big((\alpha-1)(\alpha\rho - t)\big)$.
3. **Optimize the order.** Step 2 holds for every order, so use the one that gives the smallest bound. Put $t = \rho + \lambda$ and write $s = \alpha - 1$. The exponent is $s(\rho + s\rho - \rho - \lambda) = s^2\rho - s\lambda$, a parabola in s that is smallest at $s = \lambda/(2\rho)$, where it equals $\lambda^2/(4\rho) - \lambda^2/(2\rho) = -\lambda^2/(4\rho)$. This is Equation 29.4.
4. **Gaussian mechanism.** The output distributions are $\mathcal{N}(f(D), \sigma^2)$ and $\mathcal{N}(f(D'), \sigma^2)$ with $|f(D) - f(D')| \le \Delta$. Two normal distributions with the same variance σ^2 and means s apart have Rényi divergence $D_\alpha = \alpha s^2/(2\sigma^2)$ (Equation 3.15 in Divergences; the sum in Fact 1 becomes a Gaussian integral that you evaluate by completing the square). So $D_\alpha = \alpha(f(D) - f(D'))^2/(2\sigma^2) \le \alpha\Delta^2/(2\sigma^2)$. The same holds in d dimensions with ℓ_2

sensitivity, because the Rényi divergence of two isotropic Gaussians depends only on the distance between their means: rotate the coordinates so the means differ along one axis, and the other axes contribute nothing.

5. **Composition and post-processing.** By Fact 2, the composed curve is at most $\rho_1\alpha+\rho_2\alpha = (\rho_1+\rho_2)\alpha$, a line of slope $\rho_1+\rho_2$, and post-processing does not raise the curve.

Why "concentrated": the Gaussian case by hand. The tail bound says the privacy loss behaves no worse than a normal variable. For the Gaussian mechanism it *is* a normal variable, and the bound holds with equality in every exponential moment. To see this, take $\Delta = 1$, $P = \mathcal{N}(0,\sigma^2)$, $Q = \mathcal{N}(1,\sigma^2)$. The privacy loss at output y is the log of the ratio of the two normal densities; the normalizing constants cancel and only the exponents remain:

$$L(y) = \frac{(y-1)^2 - y^2}{2\sigma^2} = \frac{1-2y}{2\sigma^2}. \tag{29.5}$$

With $Y \sim \mathcal{N}(0,\sigma^2)$, L is a linear function of a normal variable, so it is normal. Its mean is the constant term, $1/(2\sigma^2) = \rho$, and its variance is the square of the coefficient of y times σ^2, that is $4\sigma^2/(4\sigma^4) = 1/\sigma^2 = 2\rho$. A normal variable with mean m and variance v has MGF $e^{sm+s^2v/2}$, so the MGF of $\mathcal{N}(\rho,2\rho)$ at s is $e^{s\rho+s^2\rho}$; at $s=\alpha-1$ it is $e^{(\alpha-1)(1+\alpha-1)\rho} = e^{(\alpha-1)\alpha\rho}$. So Equation 29.3 holds **with equality** for every α. A ρ-zCDP mechanism is one whose privacy loss is, in every exponential moment, no worse than a normal with mean ρ and variance 2ρ.

Where the definition came from. Dwork and Rothblum (2016) introduced **concentrated differential privacy** with two parameters (μ,τ). The mean of the privacy loss is at most μ, and the privacy loss minus its mean is **sub-Gaussian** with parameter τ, which means its MGF is no larger than that of a centred normal with variance τ^2: $\mathbb{E}[e^{\lambda(L-\mathbb{E}L)}] \le e^{\lambda^2\tau^2/2}$ for every real λ. Bun and Steinke (2016) call this mean-concentrated DP and replaced it with Equation 29.2, for two reasons. First, the Rényi form follows from properties of D_α (data processing, additivity), so post-processing and composition are immediate; their Appendix A shows that the mean-concentrated definition is *not* closed under post-processing. Second, zCDP is a relaxation of mean-concentrated DP, so nothing that was mean-concentrated is lost. To see this, write $L = \mathbb{E}L + (L-\mathbb{E}L)$ and apply the sub-Gaussian bound with $\lambda = \alpha-1$: this gives $\ln\mathbb{E}[e^{(\alpha-1)L}] \le (\alpha-1)\mu + (\alpha-1)^2\tau^2/2$. Dividing by $\alpha-1$ and using Fact 1 gives $D_\alpha \le \mu + (\alpha-1)\tau^2/2 = (\mu-\tau^2/2) + (\tau^2/2)\alpha$, with equality for the Gaussian mechanism (their Lemma 4.2), where $\mu=\rho$ and $\tau^2 = 2\rho$ make the constant term vanish. In general that is a line with an intercept; Bun and Steinke allow one in a two-parameter (ξ,ρ)-zCDP, but mostly use $\xi = 0$, the one-parameter ρ-zCDP that this book follows. (This μ is unrelated to the μ of Gaussian differential privacy later in this part.)

29.3.4. Interpretation

zCDP is "RDP restricted to lines". Every ρ-zCDP mechanism has an RDP curve, and the curve is at most $\rho\alpha$. Going from a curve to ρ loses information: the smallest valid ρ is the largest ratio $\varepsilon(\alpha)/\alpha$, wherever it occurs, and the rest of the curve is forgotten. The rest of the chapter is about how much that costs.

The tail bound is loose by a polynomial factor. Figure 29.1 shows, for $\sigma = 1$ ($\rho = 0.5$), the simulated privacy loss against $\mathcal{N}(\rho,2\rho)$ and the exact tail against Equation 29.4. Since the loss is exactly $\mathcal{N}(\rho,2\rho)$, the event $L > \rho+\lambda$ is the event that a standard normal exceeds $z = \lambda/\sqrt{2\rho}$, so the exact tail is $\Phi(-\lambda/\sqrt{2\rho})$, where Φ is the standard normal cumulative distribution function.

Both tails fall off like $e^{-\lambda^2/(4\rho)} = e^{-z^2/2}$; the bound misses only the Gaussian tail's polynomial factor. For large z the Mills ratio approximation $\Phi(-z) \approx e^{-z^2/2}/(z\sqrt{2\pi})$ says the exact tail is the bound divided by $z\sqrt{2\pi} = \lambda\sqrt{\pi/\rho}$, so the ratio of the exact tail to the bound is about $\sqrt{\rho/\pi}/\lambda$. At $\lambda = 4$, $z = 4$, the exact tail is $\Phi(-4) = 3.167 \times 10^{-5}$ and the bound $e^{-16/2} = 3.355 \times 10^{-4}$: their ratio is 0.094, against $\sqrt{0.5/\pi}/4 = 0.100$ from the Mills ratio. That gap is one reason the simple conversion of the conversion section (Section 29.5) is loose.

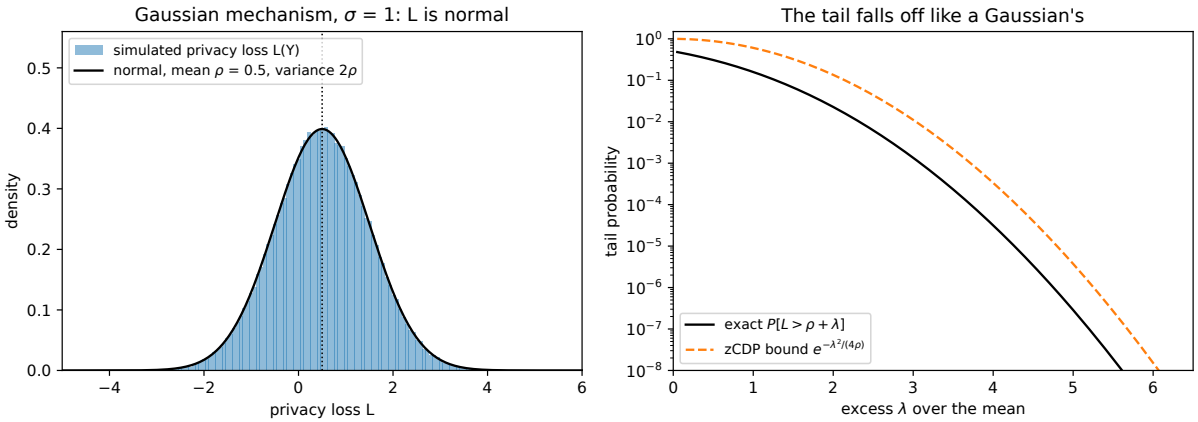


Figure 29.1.: The privacy loss of the Gaussian mechanism with $\sigma = 1$. Left: a histogram of $L(Y)$ from simulation against the normal density with mean $\rho = 0.5$ and variance $2\rho = 1$. Right: the exact upper tail $\Pr[L > \rho + \lambda]$ against the zCDP bound $e^{-\lambda^2/(4\rho)}$, on a log scale.

Where zCDP is used. zCDP is the accounting of choice when a system is built from Gaussian mechanisms on exact, non-subsampled data: statistical releases, query systems and synthetic data from noisy marginals. It is not the accounting of choice for DP-SGD, for the reason of Section 29.8. An example is the 2020 US Census TopDown algorithm, which protects the redistricting data (Abowd et al., 2022). It measures noisy versions of a set of queries ("measurements") under zCDP, using the discrete Gaussian mechanism, which adds integer-valued noise shaped like a normal density so that counts stay integers. It then post-processes the measurements together with **invariants**, statistics that the Census Bureau excluded from the privacy accounting as a matter of policy, into a microdata file. The discrete Gaussian and its zCDP guarantee are from Canonne, Kamath and Steinke (2020), the same paper as the improved conversion; Discrete mechanisms covers it. zCDP suits this setting for three reasons. The budget is split over many queries and geographic levels, and zCDP makes the split a sum of ρ's. The Gaussian's RDP curve is a line, so zCDP loses nothing before the final conversion. And the final conversion to (ε, δ) can be done once, at whatever δ the publisher chooses to report. The invariants are a reminder that the accounted ρ covers only the noisy measurements: anything released exactly, without noise, sits outside the guarantee.

29.3.5. Example: the Gaussian with $\sigma = 1$

With $\rho = 1/(2\sigma^2)$, one Gaussian step with $\sigma = 1$ is 0.5-zCDP; ten of them are $10 \times 0.5 = 5$-zCDP; one step with $\sigma = 2$ is $1/8 = 0.125$-zCDP. A million simulated privacy losses (seed 0) have mean 0.4990 and variance 1.0013, against $\rho = 0.5$ and $2\rho = 1$, and $\ln \mathbb{E}[e^L]$ is 1.0000, against

$(\alpha - 1)\alpha\rho = 1$ at $\alpha = 2$. Table 29.2 compares the tail with its bound. With $\rho = 0.5$ the exact column is $\Phi(-\lambda)$ and the bound is $e^{-\lambda^2/2}$: at $\lambda = 1$, $\Phi(-1) = 0.1587$ against $e^{-1/2} = 0.6065$.

Table 29.2.: Upper tail of the Gaussian mechanism's privacy loss, $\sigma = 1$, $\rho = 0.5$: exact, simulated with 10^6 draws, and the zCDP bound Equation 29.4.

λ	Exact $\Pr[L > \rho + \lambda]$	Simulated	zCDP bound $e^{-\lambda^2/(4\rho)}$
1	1.587×10^{-1}	1.586×10^{-1}	6.065×10^{-1}
2	2.275×10^{-2}	2.261×10^{-2}	1.353×10^{-1}
4	3.167×10^{-5}	2.700×10^{-5}	3.355×10^{-4}

29.3.6. Check yourself

- A mechanism adds $\mathcal{N}(0, 4)$ noise to a count with sensitivity 1, and then a second mechanism adds $\mathcal{N}(0, 1)$ noise to another count. What is the total ρ? *($1/8 + 1/2 = 0.625$. The first has variance 4, so $\sigma = 2$ and $\rho = 1/(2 \cdot 4)$; the second has $\sigma = 1$ and $\rho = 1/2$; composition adds them.)*
- Does ρ-zCDP bound the lower tail of the privacy loss, $\Pr[L < -t]$? *(Yes, through the other order of the pair. Equation 29.3 only uses positive exponents, so it says nothing directly about large negative values. But the definition also holds with D and D' swapped, and a change of measure connects the two: for $s > 1$, $\mathbb{E}_P[e^{-sL}] = \sum_y P(y)(Q(y)/P(y))^s = \sum_y P(y)^{1-s}Q(y)^s = e^{(s-1)D_s(Q\|P)} \le e^{(s-1)s\rho}$. Chernoff, as in steps 2 and 3, applied to $-L$ then gives $\Pr[L < -t] \le e^{-(t+\rho)^2/(4\rho)}$ for $t \ge \rho$. A weaker bound holds for every pair, zCDP or not: $\Pr_P[L < -t] \le e^{-t}$, because $P(y) < e^{-t}Q(y)$ on that event, and the Q-probabilities sum to at most 1.)*

29.4. Pure DP implies zCDP

29.4.1. Context

A real pipeline mixes mechanisms. A Laplace count, a randomized-response survey bit and an exponential-mechanism selection are ε-DP, not Gaussian: for neighbouring datasets, the probability of any output changes by at most a factor e^{ε}. To put them in the same budget as Gaussian steps, you need their ρ. A pure ε-DP mechanism has a bounded privacy loss, $|L| \le \varepsilon$, because the log of a ratio between $e^{-\varepsilon}$ and e^{ε} lies between $-\varepsilon$ and ε. A bounded variable has light, Gaussian-like tails (Hoeffding's lemma, Fact 4). So it should be zCDP with ρ of order ε^2. The question is the constant.

29.4.2. Statement

If M is ε-DP, then M is $(\varepsilon^2/2)$-zCDP (Bun & Steinke 2016, Proposition 1.4). The RDP curve of an ε-DP mechanism is also capped at ε, because every Rényi divergence is at most the max-divergence

D_∞, the largest log-ratio $\ln(P(y)/Q(y))$, which is at most ε: $D_\alpha \le D_\infty \le \varepsilon$. Together, its RDP curve lies under both the line and the cap:

$$\varepsilon(\alpha) \le \min\left(\varepsilon,\ \frac{\alpha\varepsilon^2}{2}\right). \tag{29.6}$$

The constant can be sharpened. The worst ε-DP mechanism turns out to be randomized response, which reports a bit truthfully with probability $e^\varepsilon/(1+e^\varepsilon)$ and flips it otherwise, and its slope gives the sharp constant. Every ε-DP mechanism is $(\varepsilon\tanh(\varepsilon/2))$-zCDP,

$$\varepsilon(\alpha) \le \alpha\,\varepsilon\tanh(\varepsilon/2), \tag{29.7}$$

and randomized response shows that no smaller ρ holds for every ε-DP mechanism. Since $\tanh x \le x$ for $x \ge 0$, $\varepsilon\tanh(\varepsilon/2) \le \varepsilon\cdot\varepsilon/2 = \varepsilon^2/2$, with ratio tending to 1 as $\varepsilon \to 0$, where $\tanh x \approx x$.

For the exponential mechanism, which selects one candidate from a list and favours candidates with higher scores, the constant is four times smaller: an ε-DP exponential mechanism is $(\varepsilon^2/8)$-zCDP (Cesar & Rogers 2021, the remark after Lemma 3.4). The reason is the bounded-range property of Durfee & Rogers (2019): its privacy loss lies in an interval of length ε, not 2ε.

29.4.3. Derivation

Fix neighbours, write $P = M(D)$, $Q = M(D')$, and $L = \ln(P(y)/Q(y))$ under P, where $P(y)$ and $Q(y)$ are the densities (capital letters, because q is the sampling rate later in this chapter).

The constant $\frac{1}{2}$, by Hoeffding. The plan is to bound the MGF of L with Hoeffding's lemma. Hoeffding needs the mean of L, so the first job is to bound that mean, and Hoeffding does that too.

1. **The privacy loss is bounded.** ε-DP for both orders of the pair means $e^{-\varepsilon} \le P(y)/Q(y) \le e^\varepsilon$ wherever either density is positive. So $L \in [-\varepsilon, \varepsilon]$, and P, Q have the same support: neither can be positive where the other is zero.
2. **The mean is at most $\varepsilon^2/2$.** Apply Hoeffding's lemma (Fact 4) to $X = -L$, which lies in an interval of length 2ε, with $\lambda = 1$: $\ln \mathbb{E}_P[e^{-L}] \le -\mathbb{E}_P[L] + (2\varepsilon)^2/8$. The left side is known exactly. $e^{-L} = Q(Y)/P(Y)$, and its average under P is $\sum_y P(y)\,Q(y)/P(y) = \sum_y Q(y) = 1$, because the supports agree (Fact 1 of Divergences). So $0 \le -\mathbb{E}_P[L] + \varepsilon^2/2$: the Kullback–Leibler (KL) divergence $\mathbb{E}_P[L]$, the average privacy loss, is at most $\varepsilon^2/2$.
3. **The MGF is sub-Gaussian.** Hoeffding's lemma again, now for L itself with $\lambda = \alpha - 1$: $\ln \mathbb{E}_P[e^{(\alpha-1)L}] \le (\alpha-1)\mathbb{E}_P[L] + (\alpha-1)^2(2\varepsilon)^2/8 = (\alpha-1)\mathbb{E}_P[L] + (\alpha-1)^2\varepsilon^2/2$.
4. **Combine.** Put the mean bound of step 2 into step 3: the right side is at most $(\alpha-1)\varepsilon^2/2 + (\alpha-1)^2\varepsilon^2/2 = (\alpha-1)(1+\alpha-1)\,\varepsilon^2/2 = (\alpha-1)\alpha\,\varepsilon^2/2$. That is Equation 29.3 with $\rho = \varepsilon^2/2$.
5. **The cap.** D_α is non-decreasing in α with limit $D_\infty \le \varepsilon$ (Divergences). Larger orders weight the largest likelihood ratios more, and in the limit only the largest one, at most e^ε, counts. That gives the other term of the minimum.

The sharp constant, through randomized response. Hoeffding treats L as an arbitrary bounded variable, which wastes something. The sharper route shows that randomized response is the worst case, then computes its MGF exactly.

6. **Reduce to randomized response.** Every ε-DP pair is a post-processing of ε-randomized response. That is, if $P^\star$ and $Q^\star$ are the output distributions of randomized response on the bits 0 and 1, there is a single random map K from $\{0,1\}$ to outputs with $P = P^\star K$ and $Q = Q^\star K$. Composition (Section 21.5) gives the channel K explicitly (Equation 21.10). By the data-processing inequality for D_α (Fact 2), passing both distributions through K cannot increase their divergence, so $D_\alpha(P\|Q) \le D_\alpha(P^\star\|Q^\star)$ at every order. It is enough to bound the curve of randomized response.
7. **The MGF of randomized response.** Under $P^\star$ the truthful answer has probability $t = e^\varepsilon/(1+e^\varepsilon)$, and its likelihood ratio is $t/(1-t) = e^\varepsilon$. So the privacy loss is $+\varepsilon$ with probability t and $-\varepsilon$ otherwise. With $s = \alpha - 1$, the MGF is $t\,e^{s\varepsilon} + (1-t)\,e^{-s\varepsilon}$, which is

$$\mathbb{E}[e^{sL}] = \frac{e^{(s+1)\varepsilon} + e^{-s\varepsilon}}{1+e^\varepsilon} = \frac{\cosh\left((s+\frac{1}{2})\varepsilon\right)}{\cosh(\varepsilon/2)}. \tag{29.8}$$

   For the second equality, factor $e^{\varepsilon/2}$ out of the numerator and the denominator: the numerator is $e^{\varepsilon/2}(e^{(s+\frac{1}{2})\varepsilon} + e^{-(s+\frac{1}{2})\varepsilon}) = 2e^{\varepsilon/2}\cosh\left((s+\frac{1}{2})\varepsilon\right)$ and the denominator is $e^{\varepsilon/2}(e^{-\varepsilon/2}+e^{\varepsilon/2}) = 2e^{\varepsilon/2}\cosh(\varepsilon/2)$.
8. **A concavity bound.** You want the log of step 7 to be at most $\rho\, s(s+1)$. Write $x = (s+\frac{1}{2})\varepsilon$ and $a = \varepsilon/2$. Then $x^2 - a^2 = \varepsilon^2((s+\frac{1}{2})^2 - \frac{1}{4}) = \varepsilon^2(s^2+s)$ is exactly a multiple of $s(s+1)$, so a bound on $\ln\cosh x - \ln\cosh a$ that is linear in $x^2 - a^2$ gives the right shape. That suggests looking at $\ln\cosh$ as a function of x^2. Let $h(u) = \ln\cosh\sqrt{u}$ for $u > 0$. By the chain rule its derivative is $h'(u) = \tanh(\sqrt{u})/(2\sqrt{u})$, which is decreasing, because $\tanh(y)/y$ is decreasing for $y > 0$ (its derivative has the sign of $y - \sinh y\cosh y < 0$). A function with decreasing derivative is concave, and a concave function lies below its tangent line. The tangent at $u_0 = a^2$ has slope $h'(a^2) = \tanh(a)/(2a)$, so

$$\ln\cosh x - \ln\cosh a \le (x^2 - a^2)\frac{\tanh a}{2a}. \tag{29.9}$$

   With $x^2 - a^2 = \varepsilon^2(s^2+s)$ and $\tanh(a)/(2a) = \tanh(\varepsilon/2)/\varepsilon$, the right side is $s(s+1)\,\varepsilon\tanh(\varepsilon/2) = (\alpha-1)\alpha\,\varepsilon\tanh(\varepsilon/2)$. With step 7 that is Equation 29.3 with $\rho = \varepsilon\tanh(\varepsilon/2)$, and step 6 carries it to every ε-DP pair: Equation 29.7.
9. **No smaller ρ.** As $\alpha \to 1$, any RDP curve tends to the KL divergence, the mean privacy loss. For randomized response that mean is $\varepsilon t - \varepsilon(1-t) = \varepsilon(2t-1) = \varepsilon\frac{e^\varepsilon - 1}{e^\varepsilon + 1} = \varepsilon\tanh(\varepsilon/2)$. So $\varepsilon(\alpha)/\alpha$ tends to $\varepsilon\tanh(\varepsilon/2)$, and any smaller slope fails near $\alpha = 1$.

So the naive Hoeffding route already gives the constant $\frac{1}{2}$; the only care needed is to bound the mean with Hoeffding too, through $\mathbb{E}_P[e^{-L}] = 1$. Bun and Steinke prove the same result differently: they replace the likelihood ratio by a two-point variable on $\{e^{-\varepsilon}, e^\varepsilon\}$ with the same mean (Jensen's inequality, which says that for a convex function the average of the function is at least the function of the average, so spreading the ratio to the endpoints can only increase the moments), compute its moments in closed form with sinh, and finish with an inequality between hyperbolic sines. Earlier, Dwork and Rothblum bounded the mean by the slightly weaker $\frac{1}{2}\varepsilon(e^\varepsilon - 1)$. You might hope to get the sharp constant just by putting the sharp mean bound into step 4. That bound, Equation 21.13, says the KL divergence of any ε-DP pair is at most $\varepsilon\tanh(\varepsilon/2)$, by the same data-processing argument as step 6. It is not enough on its own: it gives $\varepsilon(\alpha) \le \varepsilon\tanh(\varepsilon/2) + (\alpha-1)\varepsilon^2/2$, a line with the same slope $\varepsilon^2/2$ that starts a little lower, and not of the form $\rho\alpha$. The slope comes from Hoeffding's variance term, which is what steps 6 to 8 avoid: they bound every exponential moment at once.

The exponential mechanism. Its privacy loss lies in an interval $[c-\varepsilon, c]$ of length ε, not 2ε, for some c that depends on the pair (the bounded-range property). Steps 2 to 4 go through with the shorter interval, and the constant $(b-a)^2/8$ in Hoeffding's lemma drops from $(2\varepsilon)^2/8$ to $\varepsilon^2/8$. Hoeffding with $\lambda = 1$ and $\mathbb{E}_P[e^{-L}] = 1$ give $0 \le -\mathbb{E}_P[L] + \varepsilon^2/8$, so $\mathbb{E}_P[L] \le \varepsilon^2/8$. (The same identity forces $c - \varepsilon \le 0 \le c$: if L were always positive, e^{-L} would always be below 1 and could not average to 1, and likewise if it were always negative. So the interval contains 0, the supports agree, and the mechanism is also ε-DP.) Step 3 with length ε gives $(\alpha-1)^2\varepsilon^2/8$, and the sum is $(\alpha-1)\varepsilon^2/8 + (\alpha-1)^2\varepsilon^2/8 = (\alpha-1)\alpha\,\varepsilon^2/8$: the mechanism is $(\varepsilon^2/8)$-zCDP. Cesar and Rogers bound the mean with a result of Dong, Durfee & Rogers (2020) instead. The bound is nearly tight. The script computes $D_\alpha/(\alpha\varepsilon^2/8)$, the fraction of the bound actually used, for the two-outcome bounded-range mechanisms of Cesar & Rogers's Definition 2.4, over $c \in (0, \varepsilon)$, $\alpha \in (1, 10^2]$ and both directions. The largest value is 0.9965 at $\varepsilon = 0.5$, 0.9864 at $\varepsilon = 1$ and 0.8348 at $\varepsilon = 4$, each at the smallest order checked: the worst case is the mean, as $\alpha \to 1$.

29.4.4. Interpretation

One curve covers every ε-DP mechanism. Step 6 says more than Equation 29.7. Since every ε-DP pair is a post-processing of randomized response, the exact RDP curve of ε-randomized response is a valid RDP curve for *every* ε-DP mechanism, at every order. An RDP accountant can charge each pure-DP step that curve, which is below both the cap ε and the line $\alpha\varepsilon\tanh(\varepsilon/2)$, instead of $\min(\varepsilon, \alpha\varepsilon^2/2)$. Randomized response is the worst ε-DP mechanism at every order.

$\varepsilon^2/2$ is sharp only as $\varepsilon \to 0$. For randomized response the smallest valid ρ is $\varepsilon\tanh(\varepsilon/2)$ (steps 8 and 9): 0.1225 against $\varepsilon^2/2 = 0.1250$ at $\varepsilon = 0.5$, and 0.4621 against 0.5000 at $\varepsilon = 1$. The script confirms it numerically: the largest $\varepsilon(\alpha)/\alpha$ on a fine grid of orders is the KL divergence. Since $\tanh(x) \approx x$ for small x, the ratio tends to 1. Mechanisms other than randomized response can need less: the Laplace mechanism needs 0.3679 at $\varepsilon = 1$ (Exercise 1 in the exercise book).

The line is a poor fit far from $\alpha = 1$. Figure 29.2 draws the exact curve of randomized response, both zCDP lines and the cap. At $\varepsilon = 1$ the line $\alpha\varepsilon^2/2$ is close near $\alpha = 1$ but crosses the cap where $\alpha\varepsilon^2/2 = \varepsilon$, that is at $\alpha = 2/\varepsilon = 2$. Beyond that, the pure-DP cap ε is the better bound, and the true curve is below both. At $\varepsilon = 3$ the line $\alpha\varepsilon^2/2$ starts above the cap: its slope $\varepsilon^2/2 = 4.5$ already exceeds $\varepsilon = 3$ at $\alpha = 1$, so it is worse than ε at every order. The sharp line starts at the KL divergence, below the cap, but crosses it almost at once. An RDP accountant keeps the exact curve, or $\min(\varepsilon, \alpha\varepsilon^2/2)$; a zCDP accountant can keep only a line. For mechanisms with large ε, as in local DP, where each user randomizes their own data, describe them by their RDP curve or their ε, not by ρ.

29.4.5. Example: randomized response at $\varepsilon = 1$

The truthful-report probability is $e/(1+e) = 0.7311$. Table 29.3 compares the exact curve with the bounds. The second column is the Hoeffding bound with the sharp mean, $\varepsilon\tanh(\varepsilon/2) + (\alpha-1)\varepsilon^2/2$ with $\varepsilon\tanh(1/2) = 0.4621$; it sits $0.5 - 0.4621 = 0.0379$ below the zCDP line at every order, because both have slope $\varepsilon^2/2 = 0.5$.

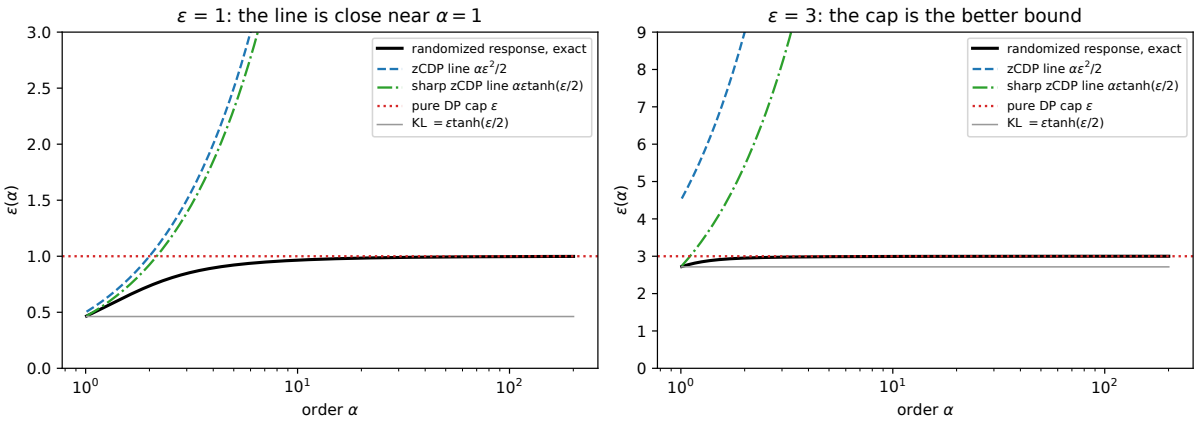


Figure 29.2.: The RDP curve of randomized response (black) against the zCDP line $\alpha\varepsilon^2/2$ (dashed), the sharp zCDP line $\alpha\varepsilon\tanh(\varepsilon/2)$ (dash-dotted) and the pure-DP cap ε (dotted), for $\varepsilon = 1$ (left) and $\varepsilon = 3$ (right). The grey line is the KL divergence $\varepsilon\tanh(\varepsilon/2)$, the curve's value at $\alpha \to 1$.

Table 29.3.: Randomized response at $\varepsilon = 1$: the exact RDP curve against the Hoeffding bound with the sharp mean, the zCDP line and the line capped at ε.

α	Exact $\varepsilon(\alpha)$	$\varepsilon\tanh(\varepsilon/2) + (\alpha-1)\varepsilon^2/2$	$\alpha\varepsilon^2/2$	$\min(\varepsilon, \alpha\varepsilon^2/2)$
1.5	0.6273	0.7121	0.7500	0.7500
2	0.7353	0.9621	1.0000	1.0000
5	0.9217	2.4621	2.5000	1.0000
10	0.9652	4.9621	5.0000	1.0000
50	0.9936	24.9621	25.0000	1.0000

Up to $\alpha = 2$ the line is within 36% of the exact value ($1.0000/0.7353 = 1.360$); at $\alpha = 50$ it is 25 times too large ($25.0000/0.9936 = 25.2$), while the cap is within 1% ($1.0000/0.9936 = 1.0064$).

29.4.6. Check yourself

- A Laplace mechanism is 0.1-DP. What ρ do you charge for it in a zCDP budget? *($\varepsilon^2/2 = 0.01/2 = 0.005$, or slightly less with the sharp $\varepsilon\tanh(\varepsilon/2)$.)*
- Why can't zCDP describe a 5-DP mechanism well? *(Charged $\varepsilon^2/2 = 12.5$, its line lies above the cap $\varepsilon = 5$ at every order $\alpha > 1$ (because $12.5\alpha > 5$). The sharp $\rho = 5\tanh(2.5) = 4.9331$ helps little: the line 4.9331α passes 5 at $\alpha = 5/4.9331 = 1.0136$. Converting either gives a worse ε than the 5 you started with.)*

29.5. From zCDP to (ε, δ)

29.5.1. Context

Whatever you account in, you report (ε, δ). Fact 3 turns an RDP curve into (ε, δ): each order α gives a valid $\varepsilon = \varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$, and you take the best order. For a ρ-zCDP mechanism the RDP curve is known in closed form, $\rho\alpha$, so that optimization over α can be done by hand. The result is the formula most papers quote. The improved conversion of Fact 3 has no closed form but is easy to minimize numerically.

29.5.2. Statement

Putting the line $\rho\alpha$ into Mironov's conversion and choosing the best order gives the simple conversion. A ρ-zCDP mechanism is (ε, δ)-DP for every $\delta \in (0, 1)$ with

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{29.10}$$

attained at $\alpha^* = 1 + \sqrt{\ln(1/\delta)/\rho}$ (Bun & Steinke 2016, Proposition 1.3). Putting the same line into the improved conversion (Canonne, Kamath & Steinke 2020, Proposition 12 and Corollary 13) gives a smaller ε, as a minimum over orders that has to be computed numerically:

$$\varepsilon = \min_{\alpha > 1} \left[\rho\alpha + \frac{\ln(1/\delta)}{\alpha - 1} + \ln\left(1 - \frac{1}{\alpha}\right) - \frac{\ln \alpha}{\alpha - 1} \right]. \tag{29.11}$$

For calibration you need the reverse: the largest ρ whose simple conversion stays within a target ε at δ. Solving Equation 29.10 for ρ, a target ε at δ is met by any

$$\rho \le \left(\sqrt{\varepsilon + \ln(1/\delta)} - \sqrt{\ln(1/\delta)} \right)^2. \tag{29.12}$$

29.5.3. Derivation

1. **Plug the line into Mironov's conversion.** By Fact 3 with $\varepsilon(\alpha) = \rho\alpha$, every $\alpha > 1$ gives a valid ε: $g(\alpha) = \rho\alpha + \ell/(\alpha - 1)$, where $\ell = \ln(1/\delta)$. The first term grows with the order and the second shrinks, so there is a best order in between.
2. **Minimize.** $g'(\alpha) = \rho - \ell/(\alpha - 1)^2$ is zero at $\alpha - 1 = \sqrt{\ell/\rho}$. Both terms of g are convex on $\alpha > 1$ (a line, and a positive multiple of $1/(\alpha - 1)$), so g is convex and this stationary point is the minimum.
3. **Evaluate.** At $\alpha^* = 1 + \sqrt{\ell/\rho}$, the first term is $\rho\alpha^* = \rho + \rho\sqrt{\ell/\rho} = \rho + \sqrt{\rho\ell}$ and the second is $\ell/\sqrt{\ell/\rho} = \ell\sqrt{\rho/\ell} = \sqrt{\rho\ell}$. Their sum is $\rho + 2\sqrt{\rho\ell}$, which is Equation 29.10. Bun and Steinke reach the same formula through the tail bound: by Equation 29.4 with $\lambda = 2\sqrt{\rho\ell}$, $\Pr[L > \varepsilon] \le e^{-4\rho\ell/(4\rho)} = \delta$ for this ε, and a privacy loss that exceeds ε with probability at most δ gives (ε, δ)-DP (the argument of Fact 3; see also Composition).
4. **The improved form** is Fact 3's improved conversion with $\varepsilon(\alpha) = \rho\alpha$. Its derivation is in From Rényi DP to (ε, δ) and to tests. The script minimizes it over a fine continuous grid of 20,001 log-spaced orders from $1 + 10^{-4}$ to $1 + 10^4$, and, for comparison, over the part's grid `ORDERS`, the fixed list of orders on which this part's accountants store RDP curves.
5. **Inversion.** Write $x = \sqrt{\rho}$. Equation 29.10 reads $x^2 + 2x\sqrt{\ell} = \varepsilon$. Adding ℓ to both sides completes the square: $(x + \sqrt{\ell})^2 = \varepsilon + \ell$. Take the positive root, $x = \sqrt{\varepsilon + \ell} - \sqrt{\ell}$, and square it. Since the simple ε grows with ρ, any smaller ρ also meets the target.

29.5.4. Interpretation

For a Gaussian, every bit of slack is in the conversion. Dividing the output of a Gaussian mechanism by σ changes nothing about privacy, because the map can be undone. After the division the two output distributions are $\mathcal{N}(0,1)$ and $\mathcal{N}(\mu,1)$ with $\mu = \Delta/\sigma$, and $\rho = \Delta^2/(2\sigma^2) = \mu^2/2$. So a ρ-zCDP Gaussian mechanism is the pair $\mathcal{N}(0,1)$ vs $\mathcal{N}(\mu,1)$ with $\mu = \sqrt{2\rho}$, the μ of μ-Gaussian differential privacy (GDP) in f-DP and Gaussian differential privacy. For this pair the exact (ε,δ) trade-off is known. The **privacy profile** $\delta(\varepsilon)$ is the smallest δ for which the mechanism is (ε,δ)-DP, the largest value of $P(S) - e^{\varepsilon}Q(S)$ over sets of outputs S. The best set is the outputs where the privacy loss exceeds ε, so $\delta(\varepsilon) = \Pr_P[L > \varepsilon] - e^{\varepsilon}\Pr_Q[L > \varepsilon]$. The privacy loss is normal with mean $\mu^2/2$ and variance μ^2 under P (as in Section 29.3), and with mean $-\mu^2/2$ under Q, so each probability is a value of Φ (The Laplace and Gaussian mechanisms):

$$\delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{29.13}$$

The zCDP line is exact for the Gaussian, so the gap between Equation 29.10 and the exact ε from Equation 29.13 is the price of the conversion alone. Figure 29.3 shows it. The simple formula is 21% too large at $\rho = 0.5$ (ratio $5.299/4.377 = 1.210$), the improved conversion 8% (ratio $4.728/4.377 = 1.080$). Both gaps shrink, in relative terms, as ρ grows.

The grid hardly matters here. On `ORDERS`, the improved conversion agrees with the fine grid to within 0.001 for $\rho \le 1$. At $\rho = 50$ the best order on the fine grid is 1.472, between the grid points 1.4 and 1.5, and `ORDERS` gives 96.116 against 96.035.

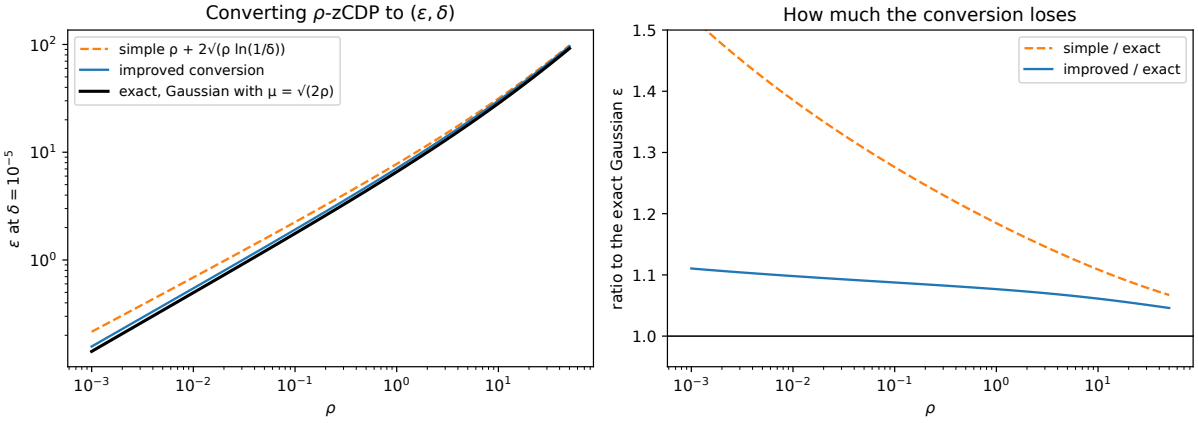


Figure 29.3.: Left: ε at $\delta = 10^{-5}$ for a ρ-zCDP mechanism, from the simple formula Equation 29.10, the improved conversion Equation 29.11, and the exact value for the Gaussian mechanism with $\mu = \sqrt{2\rho}$. Right: each conversion divided by the exact Gaussian value.

29.5.5. Example: ε for k Gaussian steps

k Gaussian steps with $\sigma = 1$ are $(k/2)$-zCDP, since each costs $1/2$ and composition adds. Table 29.4 converts a range of ρ at $\delta = 10^{-5}$, where $\ln(1/\delta) = 11.5129$. The row $\rho = 0.5$ is one step of the part's small example. For it the best order of the simple formula is $\alpha^* = 1 + \sqrt{11.5129/0.5} = 5.799$,

and the simple ε is $0.5 + 2\sqrt{0.5 \times 11.5129} = 5.299$. The row $\rho = 5$ is ten steps, and $\rho = 50$ a hundred.

Table 29.4.: ε at $\delta = 10^{-5}$ for ρ-zCDP: simple formula, improved conversion on a fine grid of orders and on the part's `ORDERS`, and the exact ε of the Gaussian mechanism with the same ρ.

ρ	$\mu = \sqrt{2\rho}$	Simple Equation 29.10	Improved, fine grid	Improved, `ORDERS`	Exact Gaussian
0.01	0.1414	0.689	0.546	0.546	0.497
0.1	0.4472	2.246	1.914	1.914	1.760
0.5	1.0000	5.299	4.728	4.729	4.377
1	1.4142	7.786	7.077	7.077	6.573
5	3.1623	20.174	19.047	19.054	17.857
50	10.0000	97.985	96.035	96.116	91.817

Calibrating the noise. To make a single Gaussian step $(1, 10^{-5})$-DP through Equation 29.12, take $\rho = (\sqrt{12.5129} - \sqrt{11.5129})^2 = 0.02082$, that is $\sigma = 1/\sqrt{2\rho} = 4.901$.

29.5.6. Check yourself

- You have a budget of $\rho = 1$ and spend 0.5 on one query. How much ε at $\delta = 10^{-5}$ is left for the rest? *(The question has no clean answer: ε does not add, because the conversion is a square root of ρ, not a multiple of it. Budgets are kept in ρ, which adds, and converted once at the end. The whole run is 7.077-DP by the improved conversion whatever the split.)*
- Why does the simple formula's best order $\alpha^* = 1 + \sqrt{\ln(1/\delta)/\rho}$ grow as ρ shrinks? *(A small ρ makes the term $\rho\alpha$ cheap, so the conversion can afford a high order, which shrinks the $\ln(1/\delta)/(\alpha - 1)$ term.)*

29.6. Group privacy

29.6.1. Context

A household, a user with many records, or a hospital contributes several records. Group privacy asks how the guarantee degrades for datasets that differ in m records. (This section writes m for the group size, because k is the number of steps in this part.) Two such datasets are joined by a chain of m neighbouring steps, each changing one record. For pure DP the answer is $m\varepsilon$: each step multiplies probabilities by at most e^{ε}, so m steps multiply them by at most $e^{m\varepsilon}$ (Properties of DP). For (ε, δ)-DP, δ blows up exponentially in m, because each step's δ is multiplied by the e^{ε} factors of the later steps. For RDP the chain argument works only if you give up orders, because Rényi divergence has no triangle inequality: $D_\alpha(P\|Q)$ is not bounded by $D_\alpha(P\|R) + D_\alpha(R\|Q)$ at the same order. zCDP gets a clean answer because it constrains every order at once, so it can pay for the chain in higher orders.

29.6.2. Statement

The answer is that the slope grows with the square of the group size, exactly as it does for the Gaussian, where a group shifts the mean m times as far. If M is ρ-zCDP and D, D' are joined by a chain of m neighbouring steps, then for every $\alpha > 1$

$$D_\alpha(M(D) \,\|\, M(D')) \le m^2 \rho\, \alpha, \tag{29.14}$$

so M is $m^2\rho$-zCDP for groups of size m (Bun & Steinke 2016, Proposition 1.9). The Gaussian mechanism attains it. The tool is a **triangle-like inequality** for Rényi divergence (their Lemma 5.2). It lets you go from P to Q through an intermediate R, at the price of using higher orders on the two legs and a factor slightly above 1 on the first: for distributions P, Q, R and $m > 1$,

$$D_\alpha(P\|Q) \le \frac{m\alpha}{m\alpha - 1} D_{\frac{m\alpha-1}{m-1}}(P\|R) + D_{m\alpha}(R\|Q). \tag{29.15}$$

For comparison, Mironov's group bound for plain RDP gives up orders instead: an (α, ε)-RDP mechanism is $(\alpha/2^c, 3^c\varepsilon)$-RDP for groups of 2^c, when $\alpha \ge 2^{c+1}$ (Mironov 2017, Proposition 2; see Rényi DP).

29.6.3. Derivation

The triangle-like inequality, by Hölder. Write $P(y)$, $Q(y)$, $R(y)$ for the densities. The idea is to write the sum that defines $D_\alpha(P\|Q)$ as an average under R of a product of two ratios, P/R and R/Q, and then split the product with Hölder's inequality. Each factor then becomes a Rényi divergence of one leg of the chain.

1. **Insert R.** $e^{(\alpha-1)D_\alpha(P\|Q)} = \sum_y P(y)^\alpha Q(y)^{1-\alpha} = \mathbb{E}_R\big[(P/R)^\alpha\,(R/Q)^{\alpha-1}\big]$, where the ratios are evaluated at $Y \sim R$. To check it, multiply out one term: $R \cdot P^\alpha R^{-\alpha} \cdot R^{\alpha-1}Q^{1-\alpha} = P^\alpha Q^{1-\alpha}$. Rewriting a sum under one distribution as an average under another like this is a change of measure (Fact 1 of Divergences).
2. **Hölder's inequality.** For positive random variables X, Y and exponents $u, v > 1$ with $\frac{1}{u} + \frac{1}{v} = 1$, $\mathbb{E}[XY] \le (\mathbb{E}[X^u])^{1/u}(\mathbb{E}[Y^v])^{1/v}$. Choose v so that the second factor becomes a divergence of order $m\alpha$, the order the induction below can afford, and u to match: $u = \frac{m\alpha-1}{\alpha(m-1)}$ and $v = \frac{m\alpha-1}{\alpha-1}$, which satisfy $\frac{1}{u} + \frac{1}{v} = \frac{\alpha(m-1)+\alpha-1}{m\alpha-1} = 1$. Then
$$\mathbb{E}_R\big[(P/R)^\alpha (R/Q)^{\alpha-1}\big] \le \big(\mathbb{E}_R[(P/R)^{u\alpha}]\big)^{1/u} \big(\mathbb{E}_R[(R/Q)^{v(\alpha-1)}]\big)^{1/v}. \tag{29.16}$$
3. **Recognize two Rényi divergences.** By Fact 1 in its sum form, an average under R of a power of P/R is a Rényi divergence of P from R: $\mathbb{E}_R[(P/R)^\beta] = \sum_y R(y)^{1-\beta}P(y)^\beta = e^{(\beta-1)D_\beta(P\|R)}$ with $\beta = u\alpha = \frac{m\alpha-1}{m-1}$. In the same way $\mathbb{E}_R[(R/Q)^{\gamma-1}] = \sum_y R(y)^\gamma Q(y)^{1-\gamma} = e^{(\gamma-1)D_\gamma(R\|Q)}$ with $\gamma = v(\alpha-1)+1 = m\alpha$.
4. **Take logarithms and divide by $\alpha - 1$.** The left side becomes $D_\alpha(P\|Q)$. The second factor contributes $\frac{1}{v} \cdot v(\alpha-1)D_{m\alpha}(R\|Q)/(\alpha-1) = D_{m\alpha}(R\|Q)$. The first contributes $\frac{1}{u}(u\alpha-1)D_\beta(P\|R)/(\alpha-1)$, a coefficient $\frac{u\alpha-1}{u(\alpha-1)}$. Here $u\alpha - 1 = \frac{m\alpha-1-(m-1)}{m-1} = \frac{m(\alpha-1)}{m-1}$, so the coefficient is $\frac{m(\alpha-1)}{m-1} \cdot \frac{\alpha(m-1)}{(m\alpha-1)(\alpha-1)} = \frac{m\alpha}{m\alpha-1}$. This is Equation 29.15.

Group privacy, by induction on m. Now walk along the chain one record at a time, using the triangle-like inequality to add one step to a group of $m-1$.

5. **Base case.** $m = 1$ is the definition.
6. **Step.** Let $D \simeq_{m-1} \hat{D} \simeq D'$: $\hat{D}$ is $m-1$ steps from D and one step from D'. Apply Equation 29.15 with $P = M(D)$, $R = M(\hat{D})$, $Q = M(D')$. The first term involves the group of $m-1$, so by the induction hypothesis at order $\beta = \frac{m\alpha-1}{m-1}$ it is at most $\frac{m\alpha}{m\alpha-1}(m-1)^2\rho\frac{m\alpha-1}{m-1} = m(m-1)\rho\alpha$; the factor $\frac{m\alpha-1}{m-1}$ from the order cancels the coefficient's denominator. The second term is a single neighbouring step, so by ρ-zCDP at order $m\alpha$ it is at most $m\rho\alpha$. The sum is $(m^2 - m + m)\rho\alpha = m^2\rho\alpha$, which is Equation 29.14.
7. **Tightness.** For the Gaussian mechanism a group of m shifts the mean by $m\Delta$, and the Gaussian Rényi divergence $\alpha s^2/(2\sigma^2)$ with $s = m\Delta$ (Equation 3.15) gives exactly $\alpha m^2\Delta^2/(2\sigma^2) = m^2\rho\alpha$. Along the chain $\mathcal{N}(0,1)$, $\mathcal{N}(1,1)$, $\mathcal{N}(2,1)$, Equation 29.15 holds with equality: at $\alpha = 3$, $m = 2$, the left side is D_3 for a shift of 2, $3 \cdot 4/2 = 6$; on the right, the first leg has order $(6-1)/1 = 5$ and coefficient $6/5$, giving $\frac{6}{5} \cdot \frac{5}{2} = 3$, and the second leg has order 6, giving $6/2 = 3$. Both sides are 6.

29.6.4. Interpretation

Quadratic in m, and that is right. With $\rho \approx \varepsilon^2/2$ for pure DP, $m^2\rho \approx (m\varepsilon)^2/2$: zCDP's group privacy matches pure DP's $m\varepsilon$ after the translation from ε to ρ. The RDP route is weaker on two counts. It halves the order at each doubling of the group, so a curve known on all orders becomes a curve known only on orders up to $\alpha/2^c$. And it multiplies ε by 3 per doubling, so the value $\rho\alpha$ at order α becomes $3^c\rho\alpha$ at order $\alpha' = \alpha/2^c$. In terms of the new order that is $3^c 2^c \rho\alpha' = 6^c\rho\alpha'$: a slope of $6^c\rho$, against zCDP's $m^2\rho = 4^c\rho$.

Strong group privacy is also a weakness. Bun and Steinke use exactly this property to prove lower bounds for zCDP that are as strong as those for pure DP. For example, stability-based algorithms, which are (ε, δ)-DP with sample sizes independent of the domain size, cannot be zCDP. That is the motivation for the next two sections: approximate zCDP and truncated CDP both weaken group privacy for large groups to escape these bounds.

29.6.5. Example: households under the Gaussian mechanism

Table 29.5 takes the Gaussian mechanism with $\sigma = 1$ ($\rho = 0.5$) and groups of size m. The RDP route starts from the curve at order 64, where $\varepsilon(64) = 0.5 \times 64 = 32$. For a group of $m = 2^c$ it gives order $64/2^c$, $\varepsilon = 3^c \times 32$, and slope $6^c \times 0.5$: for $m = 2$ that is order 32, $\varepsilon = 96$ and slope 3.

Table 29.5.: Group privacy of the Gaussian mechanism with $\sigma = 1$. zCDP gives slope $m^2\rho$ at every order, and its conversion is within about 8% of the exact value. Mironov's RDP group bound, started from order 64, gives slope $6^c\rho$ and only on orders up to $64/m$.

Group size m	zCDP $m^2\rho$	ε at 10^{-5} (improved)	Exact Gaussian, shift m	RDP route: order, ε, slope
1	0.5	4.728	4.377	—
2	2	10.725	9.997	32, 96, 3
4	8	25.919	24.382	16, 288, 18
8	32	68.616	65.319	8, 864, 108

For comparison, a 1-DP mechanism gives 8-DP for groups of 8.

29.6.6. Check yourself

- A user contributes up to 4 records to a 0.1-zCDP release. What ρ protects the user? *($4^2 \times 0.1 = 1.6$.)*
- Why does the induction in step 6 need ρ-zCDP at order $m\alpha$, larger than α? *(Hölder trades one divergence of order α for two of higher orders. zCDP controls every order, so the trade is free; a bound known only up to some order would run out after a few steps, which is what happens to truncated CDP.)*

29.7. Approximate zCDP

29.7.1. Context

Some useful mechanisms are (ε, δ)-DP but not zCDP at all. Propose-test-release and stability-based histograms check, privately, that the data are far from any dataset where the answer would change, release an exact answer when they are, and fail only with probability about δ. The group-privacy lower bound above shows they cannot be zCDP. You still want to compose them with Gaussian steps in one budget. Bun and Steinke's answer is to allow zCDP to fail on an event of probability δ.

29.7.2. Statement

M is **δ-approximately ρ-zCDP** if, after throwing away a bad event of probability at most δ in each world, the rest is ρ-zCDP. Precisely: for all neighbours $D \simeq D'$ there are events E (for $M(D)$) and E' (for $M(D')$), each of probability at least $1 - \delta$, such that for every $\alpha > 1$

$$D_\alpha(M(D)|_E \,\|\, M(D')|_{E'}) \le \rho\alpha \quad \text{and} \quad D_\alpha(M(D')|_{E'} \,\|\, M(D)|_E) \le \rho\alpha, \tag{29.17}$$

where $M(D)|_E$ is the output distribution conditioned on E (Bun & Steinke 2016, Definition 1.12). With $\delta = 0$ the events are certain and this is ρ-zCDP.

Three more properties make it a usable ledger. First, it absorbs (ε, δ)-DP: an (ε, δ)-DP mechanism is δ-approximately $(\varepsilon^2/2)$-zCDP (their Proposition 1.13 and Corollary 8.7). Second, it composes column by column: δ_1-approximate ρ_1-zCDP followed by δ_2-approximate ρ_2-zCDP is $(\delta_1 + \delta_2 - \delta_1\delta_2)$-approximately $(\rho_1 + \rho_2)$-zCDP (their Lemma 8.2). For many steps the failure probability is $1 - \prod_i (1 - \delta_i) \le \sum_i \delta_i$. Third, it converts to (ε, δ) by adding the failure probability to the conversion's δ: a δ_a-approximately ρ-zCDP mechanism is $(\varepsilon, \delta_a + (1 - \delta_a)\delta')$-DP, where ε is any conversion of ρ-zCDP at δ' (their Lemma 8.8, stated there with their own refinement of the conversion; the argument below works with any of them).

29.7.3. Derivation

This derives the conversion, which is what you use at the end. The idea: on the good event the mechanism is ρ-zCDP and the usual conversion applies; the bad event can add at most its probability. Let $\Pr[E], \Pr[E'] \geq 1 - \delta_a$ and fix a set of outputs S.

1. **Split on the event.** $\Pr[M(D) \in S] \leq \Pr[M(D) \in S \mid E] \Pr[E] + (1 - \Pr[E])$, because the probability of S on the complement of E is at most 1. Write $a = \Pr[M(D) \in S \mid E]$; the right side is $a \Pr[E] + 1 - \Pr[E] = 1 - \Pr[E](1 - a)$, which can only shrink as $\Pr[E]$ grows. So its value at the smallest allowed $\Pr[E]$ bounds it: it is at most $a(1 - \delta_a) + \delta_a$.
2. **Use the conditional zCDP.** The conditioned pair is ρ-zCDP, so by any conversion, $\Pr[M(D) \in S \mid E] \leq e^{\varepsilon} \Pr[M(D') \in S \mid E'] + \delta'$.
3. **Remove the conditioning on D'.** Dropping the outputs outside E' can only lower a probability: $\Pr[M(D') \in S] \geq \Pr[M(D') \in S \mid E'] \Pr[E'] \geq \Pr[M(D') \in S \mid E'](1 - \delta_a)$.
4. **Combine.** Put step 2 into step 1 and use step 3 for the term with e^{ε}: $\Pr[M(D) \in S] \leq (1 - \delta_a)(e^{\varepsilon} \Pr[M(D') \in S \mid E'] + \delta') + \delta_a \leq e^{\varepsilon} \Pr[M(D') \in S] + (1 - \delta_a)\delta' + \delta_a$.

The other two properties have shorter reasons. The conversion from (ε, δ)-DP comes from a decomposition of any (ε, δ)-DP pair: with probability $1 - \delta$ it behaves like an ε-DP pair, and Section 29.4 turns that into $\varepsilon^2/2$. Composition holds because the conditioned mechanisms compose as zCDP (Fact 2), and the good events of both steps hold together with probability at least $(1 - \delta_1)(1 - \delta_2) = 1 - (\delta_1 + \delta_2 - \delta_1\delta_2)$.

29.7.4. Interpretation

Approximate zCDP is a way to keep one ledger with two columns, ρ and δ_a. The ρ column composes like zCDP, with its $\sqrt{k}$ behaviour after conversion; the δ_a column adds up, like δ in basic composition. It is only worth using when some steps are genuinely (ε, δ) and not zCDP. For a step that is zCDP, putting it in the δ_a column wastes δ.

29.7.5. Example: a pipeline with stability-based steps

A pipeline runs 10 stability-based steps, each $(0.5, 10^{-7})$-DP, and 10 Gaussian steps with $\sigma = 1$. Each stability step enters the ρ column as $0.5^2/2 = 0.125$ and the δ_a column as 10^{-7}. The total is $\rho = 10 \times 0.125 + 10 \times 0.5 = 6.25$ and $\delta_a = 1 - (1 - 10^{-7})^{10} = 1.0000 \times 10^{-6}$. Converting with the improved conversion at $\delta' = 10^{-5} - \delta_a$ gives $\varepsilon = 22.100$ at total $\delta = 10^{-5}$. Without approximate zCDP you would convert the Gaussians alone ($\varepsilon = 19.119$ at $\delta = 9 \times 10^{-6}$) and add the stability steps by basic composition ($\varepsilon = 10 \times 0.5 = 5$, $\delta = 10^{-6}$), for $19.119 + 5 = 24.119$. The unified ledger saves about 2 because the $\varepsilon = 0.5$ steps compose in ρ as $\varepsilon^2/2$ each, not as ε.

29.7.6. Check yourself

- Is $(\varepsilon, 0)$-DP the same as 0-approximate $(\varepsilon^2/2)$-zCDP? *(No. ε-DP implies $(\varepsilon^2/2)$-zCDP, but zCDP does not imply pure DP: the Gaussian is zCDP and not ε-DP for any finite ε, because its privacy loss is normal and so unbounded.)*

- Why do the δ_a add while the ρ add only under a square root in ε? *(The δ_a are failure probabilities of events and combine by the union bound. The ρ add, and the conversion turns ρ into $\varepsilon \approx 2\sqrt{\rho \ln(1/\delta)}$.)*

29.8. Subsampling, and truncated CDP

29.8.1. Context

DP-SGD, the running example of this part, runs each step on a Poisson sample of the data: each record is used independently with probability q. Subsampling makes a step much more private than the Gaussian it contains, because a record is usually not in the batch. Does zCDP see this? In this section q is the sampling rate, never a density.

Take add/remove neighbours (two datasets differ by one record being present or absent), sensitivity 1 and noise multiplier σ, and reduce to one dimension. Without the record the step outputs $Q = \mathcal{N}(0, \sigma^2)$. With it, the record is in the sample with probability q, shifting the output's mean by 1, and absent otherwise, so the output is the mixture $P = (1-q)\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2)$. The RDP curve compares P against Q; the other order, Q against P, is never larger (both facts are derived in Amplification by subsampling, Equation 31.9). For integer orders the curve has a closed form, a binomial sum over how many of α copies of the record's contribution are present:

$$\varepsilon(\alpha) = \frac{1}{\alpha - 1} \ln A_\alpha, \qquad A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\left(\frac{j^2 - j}{2\sigma^2}\right) \tag{29.18}$$

(Mironov, Talwar & Zhang 2019). To see where it comes from, let r be the likelihood ratio of $\mathcal{N}(1, \sigma^2)$ to Q. Then $P/Q = 1 - q + q\,r$, and by Fact 1, $A_\alpha = e^{(\alpha-1)D_\alpha(P\|Q)} = \mathbb{E}_Q[(P/Q)^\alpha] = \mathbb{E}_Q[(1 - q + q\,r)^\alpha]$. Expand the power binomially. Each term needs $\mathbb{E}_Q[r^j]$, which by Fact 1 is $e^{(j-1)D_j}$ for the pair $\mathcal{N}(1, \sigma^2)$, $\mathcal{N}(0, \sigma^2)$; with $D_j = j/(2\sigma^2)$ (Equation 3.15 at order j) that is $e^{(j^2-j)/(2\sigma^2)}$.

29.8.2. Statement

No amplification for zCDP. For the Poisson-subsampled Gaussian with any $q \in (0, 1]$, the smallest ρ with $\varepsilon(\alpha) \le \rho\alpha$ for all α is $1/(2\sigma^2)$, the same as without subsampling.

Truncated CDP. Since the problem comes from large orders, ask for the line only on orders below a cut-off ω. A mechanism is (ρ, ω)**-tCDP** if for all neighbours and all $\alpha \in (1, \omega)$

$$D_\alpha(M(D) \,\|\, M(D')) \le \rho\alpha \tag{29.19}$$

(Bun, Dwork, Rothblum & Steinke 2018, Definition 1). ρ-zCDP is the case $\omega = \infty$, that is (ρ, ∞)-tCDP. Composition works as for zCDP on the orders both steps cover: (ρ_1, ω_1) and (ρ_2, ω_2) compose to $(\rho_1 + \rho_2, \min(\omega_1, \omega_2))$ (their Lemma 2). The conversion to (ε, δ) is the simple zCDP conversion when its best order is below ω, and uses the order ω itself otherwise (their Lemma 6):

$$\varepsilon = \begin{cases} \rho + 2\sqrt{\rho \ln(1/\delta)} & \text{if } \ln(1/\delta) \le (\omega - 1)^2 \rho, \\ \rho\omega + \dfrac{\ln(1/\delta)}{\omega - 1} & \text{otherwise.} \end{cases} \tag{29.20}$$

And, unlike zCDP, tCDP is amplified by subsampling (their Theorem 11): let $\rho, s \in (0, 0.1]$ with $\ln(1/s) \geq 3\rho(2 + \log_2(1/\rho))$, and let M be (ρ, ω')-tCDP with $\omega' \geq \ln(1/s)/(2\rho) \geq 3$. Running M on a uniformly random subset of a fraction s of the records (fixed size, without replacement) is $(13s^2\rho,\ \omega)$-tCDP with $\omega = \ln(1/s)/(4\rho)$.

29.8.3. Derivation

No amplification for zCDP. Write $\rho_0 = 1/(2\sigma^2)$, the slope of the Gaussian without subsampling. The plan is to show that ρ_0 is enough (subsampling never hurts) and that nothing smaller is enough (at large orders, subsampling stops helping).

1. **Upper bound, by convexity.** $1 - q + q\,r$ is an average of the values 1 and r with weights $1 - q$ and q. Since $x \mapsto x^\alpha$ is convex, the power of an average is at most the average of the powers: $(1 - q + q\,r)^\alpha \leq (1 - q) + q\,r^\alpha$. Take $\mathbb{E}_Q$; with $\mathbb{E}_Q[r^\alpha] = e^{(\alpha-1)\alpha\rho_0}$ from the Gaussian curve, $A_\alpha \leq 1 - q + q\,e^{(\alpha-1)\alpha\rho_0} \leq e^{(\alpha-1)\alpha\rho_0}$, the last step because an average of 1 and a number at least 1 is at most that number. So $D_\alpha(P\|Q) \leq \rho_0\alpha$. The other order obeys the same bound, by a convexity argument on pairs. For any pair (A, B), $e^{(\alpha-1)D_\alpha(A\|B)} = \sum_y B(y)\,(A(y)/B(y))^\alpha$ is the perspective of the convex function x^α, which makes it jointly convex in (A, B): the value at an average of pairs is at most the average of the values. Writing the pair (Q, P) as the mixture $(1 - q)\,(Q, Q) + q\,(Q, \mathcal{N}(1, \sigma^2))$ gives $e^{(\alpha-1)D_\alpha(Q\|P)} \leq (1 - q) + q\,e^{(\alpha-1)\alpha\rho_0}$, the same bound. Subsampling never hurts.
2. **Lower bound, by one term.** Every term of Equation 29.18 is positive, so dropping terms gives a lower bound. Keep only $j = \alpha$, the event that the record is present in all α copies: $A_\alpha \geq q^\alpha e^{(\alpha^2-\alpha)\rho_0}$. Taking logs and dividing by $\alpha - 1$,
$$\varepsilon(\alpha) \geq \rho_0\alpha + \frac{\alpha \ln q}{\alpha - 1}. \tag{29.21}$$
3. **Divide by α and let $\alpha \to \infty$.** $\varepsilon(\alpha)/\alpha \geq \rho_0 + \ln q/(\alpha - 1) \to \rho_0$: the penalty $\ln q$ for the record being rarely sampled is fixed, while the Gaussian part grows with the order, so the penalty's share of the slope vanishes. A line $\rho\alpha$ above the whole curve needs $\rho \geq \rho_0$, and by step 1, ρ_0 suffices.

The tCDP conversion. This is the zCDP conversion of Section 29.5 with the orders restricted to $(1, \omega]$.

4. **Restrict Mironov's conversion.** As in Section 29.5, every $\alpha \in (1, \omega]$ gives a valid $g(\alpha) = \rho\alpha + \ell/(\alpha - 1)$ with $\ell = \ln(1/\delta)$. (The endpoint ω is allowed: D_α is non-decreasing in α and lower semicontinuous, as a supremum over finite partitions of functions continuous in α, so $D_\omega \leq \lim_{\alpha\uparrow\omega} D_\alpha \leq \rho\omega$.)
5. **Constrained minimum of a convex function.** The unconstrained minimizer is $\alpha^* = 1 + \sqrt{\ell/\rho}$. If $\alpha^* \leq \omega$, that is $\ell \leq (\omega - 1)^2\rho$, use it and get Equation 29.10. Otherwise α^* lies beyond ω; a convex function is decreasing to the left of its minimizer, so g is decreasing on $(1, \omega]$, the best allowed order is ω, and it gives $\rho\omega + \ell/(\omega - 1)$.

The subsampling theorem has a longer proof (their Section 3): write the subsampled pair as mixtures $sP + (1 - s)R$ and $sQ + (1 - s)R$, expand $(1 + sZ)^\alpha$ with $Z = (P - Q)/(sQ + (1 - s)R)$, and bound the second moment of Z through D_2 and the higher moments through the tCDP bound at orders up to ω'. The second-moment term carries the factor s^2 that gives the amplification. The truncation is essential: the higher-order terms are controlled only for α up to about $\ln(1/s)/\rho$.

29.8.4. Interpretation

The curve bends, and a line cannot. Figure 29.4 shows the curve of one DP-SGD step, $q = 0.01$, $\sigma = 1$, on integer orders. Up to $\alpha = 8$ it is tiny and almost linear. Its slope there is close to $q^2(e^{1/\sigma^2} - 1)/2 = 10^{-4} \times 1.71828/2 = 8.5914 \times 10^{-5}$. That value comes from the first terms of Equation 29.18. If every exponential in the sum were 1, the sum would be $((1-q)+q)^\alpha = 1$; the $j = 0$ and $j = 1$ exponentials are exactly 1, so the excess over 1 starts at $j = 2$, and for small q it is $A_\alpha \approx 1 + \binom{\alpha}{2} q^2(e^{1/\sigma^2} - 1)$. Then $\ln A_\alpha/(\alpha - 1) \approx \alpha q^2(e^{1/\sigma^2} - 1)/2$. Between $\alpha = 8$ and $\alpha = 16$ the curve jumps by more than three orders of magnitude, from 8.9364×10^{-4} to 3.0879 (Table 29.6). Beyond that it follows the lower bound Equation 29.21, which turns positive where $\rho_0(\alpha - 1) = \ln(1/q)$, at $\alpha = 1 + 2\sigma^2 \ln(1/q) = 1 + 2 \ln 100 = 10.21$. At large orders the term $j = \alpha$ dominates: it is the event that the record is in every one of α imagined samples, rare but with the full Gaussian loss. A line through the origin above the whole curve must have the slope at infinity, $\rho_0 = 0.5$. A line that only has to stay above the curve up to $\omega = 8$ can have slope 1.1171×10^{-4}, $0.5/(1.1171 \times 10^{-4}) = 4476$ times smaller.

Small orders are what matter for DP-SGD. After $k = 10{,}000$ steps the RDP curve is $k\varepsilon(\alpha)$, and the conversion picks the order where $k\varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$ is smallest. With per-step values around 10^{-4}, the first term is small at small orders (from $10^4 \times 1.7181 \times 10^{-4} = 1.7181$ at order 2 to 8.9364 at order 8), and the order is pushed up only until the curve's jump makes the first term expensive. That is a small order: on the integer orders of `ORDERS` the best is $\alpha = 4$, for Mironov's conversion and for the improved one. The large orders that set the zCDP slope are never used. So the right one-number summary of DP-SGD keeps ρ only on small orders, which is what (ρ, ω)-tCDP does. RDP accounting keeps the whole curve and does at least as well.

tCDP's theorem is loose in its constant. Theorem 11 holds for every tCDP mechanism, so its constant 13 is generous. With $\sigma = \sqrt{5}$ ($\rho = 1/(2 \cdot 5) = 0.1$, so the theorem applies) and $s = 0.01$, the side condition reads $3\rho(2 + \log_2(1/\rho)) = 1.597 \le \ln(1/s) = 4.605$, and the theorem gives $(13 \times 0.01^2 \times 0.1,\ 4.605/0.4) = (1.300 \times 10^{-4},\ 11.51)$-tCDP. The Poisson-subsampled Gaussian curve on integer orders up to 11.51 needs only $\rho = 1.128 \times 10^{-5}$, about 11 times less. After 10,000 steps the theorem's parameters convert to $\varepsilon = 9.037$ by Equation 29.20, while RDP accounting of the Poisson curve on the integer orders of `ORDERS`, with the improved conversion, gives 2.046.

This comparison is only indicative. The theorem is for fixed-size sampling and replace-one neighbours, the curve for Poisson sampling and add/remove neighbours, and Amplification by subsampling explains why the two must not be mixed in an accountant.

29.8.5. Example: the running DP-SGD example in zCDP and tCDP

Table 29.6 lists the per-step curve. All values are on integer orders, which is what Equation 29.18 computes; a guarantee between grid points needs the fractional-order formula of Amplification by subsampling.

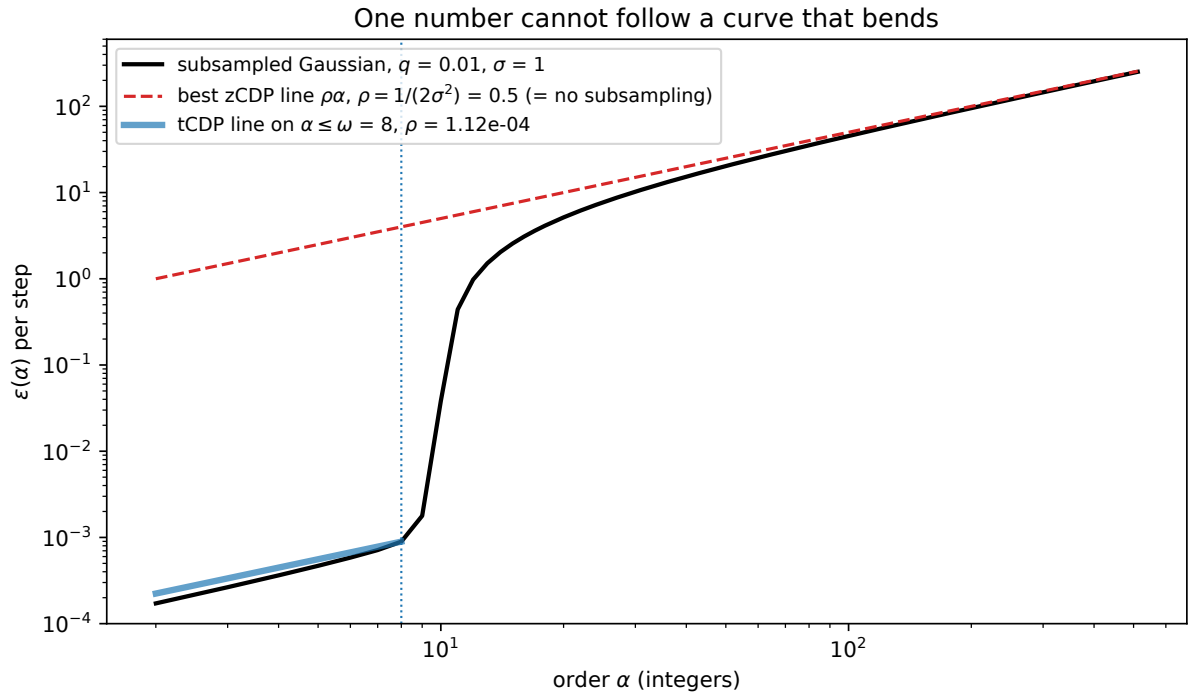


Figure 29.4.: The RDP curve of one Poisson-subsampled Gaussian step with $q = 0.01$, $\sigma = 1$, on integer orders (black), against the best zCDP line $\rho\alpha$ with $\rho = 1/(2\sigma^2) = 0.5$ (red dashed), which is the unsubsampled Gaussian's line $\alpha/2$, and the tCDP line on orders up to $\omega = 8$ (blue). Both axes are logarithmic.

Table 29.6.: The RDP curve of one DP-SGD step ($q = 0.01$, $\sigma = 1$) on integer orders: its value, its slope from the origin, the lower bound Equation 29.21, and the Gaussian without subsampling.

α	$\varepsilon(\alpha)$	$\varepsilon(\alpha)/\alpha$	Lower bound Equation 29.21	No subsampling, $\alpha/2$
2	1.7181×10^{-4}	8.5907×10^{-5}	-8.2103	1
4	3.6315×10^{-4}	9.0789×10^{-5}	-4.1402	2
8	8.9364×10^{-4}	1.1171×10^{-4}	-1.2631	4
10	3.8270×10^{-2}	3.8270×10^{-3}	-0.1169	5
12	9.7801×10^{-1}	8.1501×10^{-2}	0.9762	6
16	3.0879	1.9299×10^{-1}	3.0878	8
64	2.7322×10^{1}	4.2690×10^{-1}	27.3217	32
512	2.5139×10^{2}	4.9099×10^{-1}	251.3858	256

zCDP. The steepest ratio on the integer orders of `ORDERS` is 0.4910, at $\alpha = 512$, but that is not a valid zCDP parameter: zCDP needs the line at every order, and the ratio tends to 0.5 as $\alpha \to \infty$. So zCDP must charge $\rho = 0.5$ per step, the same as without subsampling. After 10,000 steps, $\rho = 5000$, and $\varepsilon = 5000 + 2\sqrt{5000 \times 11.5129} = 5479.9$ at $\delta = 10^{-5}$ by Equation 29.10, or 5475.8 with the improved conversion. That is the number zCDP reports for a training run that RDP accounting, on the integer orders of `ORDERS`, puts at $\varepsilon = 7.469$ with Mironov's conversion and 6.719 with the improved one. Amplification by subsampling also uses the fractional orders of `ORDERS` and gets slightly lower values (Table 31.9).

tCDP. Table 29.7 takes the per-step slope on orders up to ω, composes 10,000 steps by adding the slopes, and converts with Equation 29.20.

Table 29.7.: tCDP accounting of the running example, $q = 0.01$, $\sigma = 1$, $k = 10{,}000$, $\delta = 10^{-5}$, for several truncation points ω. Slopes are taken on integer orders up to ω.

ω	ρ per step	ρ after 10^4 steps	ε at 10^{-5}
2	8.5907×10^{-5}	0.8591	13.231
3	8.8213×10^{-5}	0.8821	8.403
4	9.0789×10^{-5}	0.9079	7.469
5	9.3733×10^{-5}	0.9373	7.507
6	9.7250×10^{-5}	0.9725	7.665
8	1.1171×10^{-4}	1.1171	8.289
10	3.8270×10^{-3}	38.2704	80.252
16	1.9299×10^{-1}	1929.9067	2228.027

The best integer choice, $\omega = 4$, reproduces RDP with Mironov's conversion, 7.469. A smaller ω lowers the slope only a little but forbids the orders the conversion needs. With $\omega = 4$, $\ln(1/\delta) = 11.51 > (\omega - 1)^2\rho = 9 \times 0.9079 = 8.17$, so the second branch of Equation 29.20 applies: $4 \times 0.9079 + 11.5129/3 = 7.469$, which is Mironov's conversion at order 4. Using the first branch anyway would give 7.374, but that needs order $\alpha^* = 1 + \sqrt{11.5129/0.9079} = 4.561 > \omega$, where the tCDP bound says nothing. Once ω passes the bend near $\alpha = 10$, tCDP collapses towards zCDP.

29.8.6. Check yourself

- Why do the largest orders decide zCDP's ρ for a subsampled Gaussian, when no conversion ever uses them? *(zCDP requires the line at every order. The ratio $\varepsilon(\alpha)/\alpha$ keeps growing towards $1/(2\sigma^2)$ as $\alpha \to \infty$, so the limit sets the slope, and $\rho = 1/(2\sigma^2)$.)*
- Two DP-SGD runs, one $(10^{-4}, 4)$-tCDP per step and one $(10^{-5}, 20)$-tCDP per step, are composed. What is the result? *($(\rho, \omega) = (1.1 \times 10^{-4}, 4)$ per pair of steps: ρ adds and ω is the minimum, because the line is guaranteed for both steps only on the orders both cover.)*

29.9. Code: check every number

The script code/concentrated_dp.py computes every number in this chapter and draws every figure. The Gaussian privacy loss is checked against simulation, randomized response and the binary bounded-range mechanisms are evaluated exactly in log space, the conversions are minimized on a fine grid and on `ORDERS` and compared with the exact Gaussian $\delta(\varepsilon)$, and the subsampled-Gaussian curve is computed with Equation 29.18 by a log-sum-exp over the binomial terms. From the repository root:

```
uv run python code/part04-privacy-accounting/concentrated_dp.py
```

Output (seed 0), summarized in Table 29.8:

Table 29.8.: The script's output with the default seed.

Experiment	Result
Gaussian privacy loss, $\sigma = 1$	Simulated mean 0.4990, variance 1.0013, $\ln \mathbb{E}[e^L] = 1.0000$; tails in Table 29.2; exact/bound 0.094 at $\lambda = 4$
Pure DP to zCDP	Smallest ρ of randomized response on a fine grid of orders $= \varepsilon \tanh(\varepsilon/2)$ (Equation 29.7): 0.1225 ($\varepsilon = 0.5$), 0.4621 ($\varepsilon = 1$), against $\varepsilon^2/2$ = 0.1250, 0.5000; curve in Table 29.3
Bounded range, $\varepsilon^2/8$	Largest $D_\alpha/(\alpha\varepsilon^2/8)$: 0.9965, 0.9864, 0.8348 at $\varepsilon = 0.5$, 1, 4
zCDP to $(\varepsilon, 10^{-5})$	Table 29.4; at $\rho = 0.5$ the simple formula is 1.210 and the improved 1.080 times the exact 4.377
Group privacy, $\sigma = 1$	Table 29.5; triangle-like inequality on Gaussians: both sides 6
Approximate zCDP pipeline	$\rho = 6.25$, $\delta_a = 1.0000 \times 10^{-6}$, $\varepsilon = 22.100$; without it 24.119

Experiment	Result
DP-SGD, $q = 0.01$, $\sigma = 1$, $k = 10^4$	zCDP: $\rho = 5000$, $\varepsilon = 5479.9$ (simple), 5475.8 (improved). tCDP, $\omega = 4$: $\varepsilon = 7.469$. RDP on integer orders: 7.469 (Mironov), 6.719 (improved), both at $\alpha = 4$

tests/test_concentrated_dp.py checks that the output still matches.

What to take from it. For Gaussian mechanisms zCDP is exact, and all the slack is in the conversion: 8% with the improved conversion at $\rho = 0.5$. For pure DP mechanisms the sharp constant is $\varepsilon \tanh(\varepsilon/2)$, and $\varepsilon^2/2$ is close to it for small ε, but any line ignores the cap ε; for large ε keep the RDP curve, for instance that of randomized response. For subsampled mechanisms zCDP is useless: it reports ε in the thousands for a run that is $\varepsilon \approx 7$, because its slope is set by orders the conversion never uses. tCDP, or the full RDP curve, recovers the amplification.

29.10. References

- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016; arXiv:1605.02065). The source for this chapter: the definition (1.1), the Gaussian (1.6), pure DP to zCDP (1.4, proved as 3.3), conversion (1.3, 3.5, 3.6), group privacy (1.9, 5.3), approximate zCDP (1.12, Section 8), and the lower bounds. Read Sections 1–3 first.
- Dwork & Rothblum, "Concentrated Differential Privacy" (2016, arXiv:1603.01887). The original two-parameter definition and its motivation from advanced composition.
- Bun, Dwork, Rothblum & Steinke, "Composable and Versatile Privacy via Truncated CDP" (STOC 2018). Truncated CDP: definition, composition, conversion (Lemma 6), amplification by subsampling (Theorem 11) and the sinh-normal mechanism.
- Canonne, Kamath & Steinke, "The Discrete Gaussian for Differential Privacy" (NeurIPS 2020). The discrete Gaussian and its zCDP guarantee, and the improved conversion from RDP and zCDP to (ε, δ) (Proposition 12, Corollary 13), with a numerically stable way to compute it.
- Cesar & Rogers, "Bounding, Concentrating, and Truncating: Unifying Privacy Loss Composition for Data Analytics" (Algorithmic Learning Theory 2021). Composition of pure DP, bounded-range (exponential mechanism) and CDP steps in one analysis, including the $\varepsilon^2/8$ bound for bounded-range mechanisms.
- Durfee & Rogers, "Practical Differentially Private Top-k Selection with Pay-what-you-get Composition" (NeurIPS 2019). Introduces the bounded-range property of the exponential mechanism.
- Dong, Durfee & Rogers, "Optimal Differential Privacy Composition for Exponential Mechanisms" (ICML 2020). Exact composition of bounded-range mechanisms, and the mean bound that Cesar & Rogers use.
- Mironov, "Rényi Differential Privacy" (CSF 2017). The RDP framework this chapter restricts to lines, and the RDP group-privacy bound used for comparison.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (2019, arXiv:1908.10530). The subsampled-Gaussian curve Equation 29.18.

- Abowd et al., "The 2020 Census Disclosure Avoidance System TopDown Algorithm" (Harvard Data Science Review, 2022; arXiv:2204.08986). zCDP accounting with the discrete Gaussian at national scale, and the role of invariants.

29.11. Cheat sheet

Table 29.9 collects the results. $\ell = \ln(1/\delta)$.

Table 29.9.: Every result of this chapter.

Result	Formula	Tight?
ρ-zCDP	$D_\alpha \le \rho\alpha$ for all $\alpha > 1$; $\ln \mathbb{E}[e^{(\alpha-1)L}] \le (\alpha-1)\alpha\rho$	Equality for the Gaussian
Privacy-loss tail	$\Pr[L > \rho + \lambda] \le e^{-\lambda^2/(4\rho)}$	Off by the Mills-ratio factor $\sqrt{\rho/\pi}/\lambda$ for the Gaussian
Gaussian mechanism	$\rho = \Delta^2/(2\sigma^2)$; privacy loss $\sim \mathcal{N}(\rho, 2\rho)$	Exact
Composition, post-processing	ρ's add; functions of the output keep ρ	Exact for Gaussians
Pure DP	ε-DP $\Rightarrow$ $(\varepsilon^2/2)$-zCDP; curve $\le \min(\varepsilon, \alpha\varepsilon^2/2)$	Sharp only as $\varepsilon \to 0$
Pure DP, sharp	ε-DP $\Rightarrow$ $(\varepsilon\tanh(\varepsilon/2))$-zCDP; curve $\le$ that of ε-randomized response	Attained by randomized response
Exponential mechanism	$(\varepsilon^2/8)$-zCDP, by Hoeffding on an interval of length ε	Within 0.4% at $\varepsilon = 0.5$ on binary pairs
To (ε, δ), simple	$\varepsilon = \rho + 2\sqrt{\rho\ell}$ at $\alpha^* = 1 + \sqrt{\ell/\rho}$	21% above exact at $\rho = 0.5$
To (ε, δ), improved	$\min_\alpha[\rho\alpha + \ell/(\alpha-1) + \ln(1 - 1/\alpha) - \ln\alpha/(\alpha-1)]$	8% above exact at $\rho = 0.5$
Calibration	$\rho = (\sqrt{\varepsilon+\ell} - \sqrt{\ell})^2$, $\sigma = 1/\sqrt{2\rho}$	Via the simple conversion
Group of m	$m^2\rho$	Exact for Gaussians
Approximate zCDP	ρ adds, δ_a adds; $(\varepsilon(\rho, \delta'), \delta_a + \delta')$-DP	—
Subsampled Gaussian	Smallest $\rho = 1/(2\sigma^2)$: no amplification	Exact
(ρ, ω)-tCDP	$D_\alpha \le \rho\alpha$ for $\alpha \in (1, \omega)$; compose to $(\sum \rho_i, \min \omega_i)$; convert by Equation 29.20	Matches RDP-Mironov on the running example

Rules of thumb

- Keep budgets in ρ, which adds; convert to (ε, δ) once, at the end, with the improved conversion.
- One Gaussian step with σ costs $\rho = 1/(2\sigma^2)$: $\sigma = 1$ is $\rho = 0.5$, about $\varepsilon = 4.7$ at $\delta = 10^{-5}$.
- A pure ε-DP step costs $\varepsilon^2/2$, or $\varepsilon \tanh(\varepsilon/2)$ with the sharp bound; an exponential-mechanism step costs $\varepsilon^2/8$. For $\varepsilon \geq 2$ the line $\alpha\varepsilon^2/2$ is above the plain ε at every order, and the sharp line crosses it soon after $\alpha = 1$; keep the RDP curve or the plain ε.
- Never account a subsampled mechanism in zCDP. Use RDP, tCDP with a small ω, or a privacy loss distribution.
- zCDP's group privacy is quadratic in the group size, exactly as for the Gaussian.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

30. f-DP and Gaussian differential privacy

30.1. Why this matters

Every privacy definition in this part answers the same question: how well can an adversary who sees the output of a mechanism M tell whether it ran on a dataset D or on a neighbour D' that differs from it in one record? Differential privacy (DP) in its (ε, δ) form answers with two numbers. Rényi differential privacy (RDP) answers with a curve over orders α (Rényi DP), and concentrated DP with one number ρ (concentrated DP). But Part 1 already gave the complete answer to "how well can anyone tell P from Q". It is the **trade-off function** $T(P, Q)$: for each rate at which a test may wrongly reject P (its type I error), the smallest rate at which it can wrongly accept P when the truth is Q (its type II error) (Hypothesis testing, Section 4.5). This chapter makes that curve the privacy guarantee itself.

The organizing idea: **the most informative description of a mechanism's privacy is its whole trade-off curve between $M(D)$ and $M(D')$, and every other notion is a lossy summary of it.** An (ε, δ) pair is one straight line lying under the curve, an RDP curve is a set of moments of it, and both throw information away. Two facts make the curve practical rather than just complete. First, Gaussian mechanisms give a one-parameter family of curves G_μ that is closed under composition: running a mechanism with curve G_{μ_1} and then one with curve G_{μ_2} gives the curve $G_{\sqrt{\mu_1^2+\mu_2^2}}$. Second, a central limit theorem (CLT) says that composing many mechanisms, each of which leaks little, gives a curve close to some G_μ, whatever the mechanisms were. Table 30.1 lists the results.

Table 30.1.: The results in this chapter and where they are used.

Result	Where it is used
f-DP and the curve $f_{\varepsilon,\delta}$	Reading any (ε, δ) guarantee as a bound on every attacker's errors
μ-Gaussian differential privacy (GDP); the Gaussian mechanism is $1/\sigma$-GDP	A one-number guarantee that is exact for Gaussian noise
Duality: $\delta(\varepsilon) = 1 + f^*(-e^\varepsilon)$; the GDP privacy profile	Reporting a GDP guarantee as (ε, δ) without loss
Composition: $G_{\mu_1} \otimes G_{\mu_2} = G_{\sqrt{\mu_1^2+\mu_2^2}}$; the tensor product	Exact accounting for full-batch noisy gradient descent; what numerical accountants compute
CLT for composition	Approximating many small mechanisms by one G_μ

Result	Where it is used
Subsampled Gaussian CLT, $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$	A quick, approximate ε for differentially private stochastic gradient descent (SGD), DP-SGD for short
RDP as a lossy summary of the curve	Why RDP accounting leaves slack, and how much

A clash of letters. The f-DP papers write α for the type I error and β for the type II error, as Part 1 does. In this part α is the Rényi order. So this chapter writes a **for the type I error and** b **for the type II error** of a test, and keeps α for Rényi orders only.

Notation. Throughout, $P = M(D)$ and $Q = M(D')$ are the two output distributions, with densities p and q. The one exception is Section 30.8, where q is a sampling rate and the densities are written $P(y)$ and $Q(y)$. A test ϕ looks at the output y and "fires", deciding that the data was D', with probability $\phi(y)$. Its type I error is how often it fires when the data was D, $a = \mathbb{E}_P[\phi]$, and its type II error is how often it stays silent when the data was D', $b = 1 - \mathbb{E}_Q[\phi]$. The **privacy loss** of an output y is the log of how much more likely y is under D than under D', $L(y) = \ln \frac{p(y)}{q(y)}$. Drawing the output from P makes it random: $L(Y)$ with $Y \sim P$ is the **privacy loss random variable (PRV)** (composition).

30.2. Toolkit: three facts every derivation uses

30.2.1. Fact 1 — The curve is fixed by the law of the privacy loss

The trade-off curve is about the best tests, and the best tests only ever look at the privacy loss. The Neyman–Pearson lemma of Part 1 says that, at any type I error, the test with the smallest type II error fires on the outputs that are most typical of Q relative to P, that is, where the likelihood ratio q/p is largest. Since $q/p = e^{-L}$, these are the outputs where the privacy loss L is smallest. So every optimal test has the form "fire when $L < l$" (with a coin flip at $L = l$ when L has an atom there), and its errors are the probabilities of the event $\{L < l\}$ under P and under Q. The curve $T(P, Q)$ therefore depends on the pair only through the law of L under P and under Q.

Those two laws are not independent of each other. To get a probability under Q from one under P, weight each output by $q/p = e^{-L}$. This is Part 1's change of measure (Fact 2 of Hypothesis testing, where L names the ratio q/p; here that ratio is e^{-L}): for any event A defined through L,

$$Q(A) = \mathbb{E}_P[e^{-L}\mathbf{1}_A], \tag{30.1}$$

when Q puts no mass where $p = 0$. The reason is one line: $\mathbb{E}_P[e^{-L}\mathbf{1}_A] = \int_A p \cdot \frac{q}{p} = \int_A q$. So the law of the PRV under P alone fixes the curve.

The Gaussian pair shows this at work. Take $P = \mathcal{N}(0, 1)$ and $Q = \mathcal{N}(\mu, 1)$, a unit-variance normal and the same normal shifted by μ. Taking the log of the ratio of the two densities, the y^2

terms cancel and

$$L(y) = \ln \frac{p(y)}{q(y)} = -\frac{y^2}{2} + \frac{(y-\mu)^2}{2} = \frac{\mu^2}{2} - \mu y. \tag{30.2}$$

The loss is a linear function of a normal output, so it is normal. Under P, y has mean 0 and variance 1, so $L \sim \mathcal{N}(\frac{\mu^2}{2}, \mu^2)$. Under Q, y has mean μ, so $L \sim \mathcal{N}(\frac{\mu^2}{2} - \mu^2, \mu^2) = \mathcal{N}(-\frac{\mu^2}{2}, \mu^2)$. The change of measure with $A =$ everything says $\mathbb{E}_P[e^{-L}]$ should be $Q(\text{everything}) = 1$. The moment generating function of a normal, $\mathbb{E}[e^{tX}] = e^{tm+t^2v/2}$ for $X \sim \mathcal{N}(m, v)$, with $t = -1$, gives $\mathbb{E}_P[e^{-L}] = e^{-\mu^2/2+\mu^2/2} = 1$, as it must.

One consequence is used several times below: the slope of the curve is set by the loss at the threshold. Part 1 showed $T'(a) = -q/p$ at the threshold of the test of size a. The reason is that moving the threshold to include one more sliver of outputs near y raises the type I error by $p(y)\,dy$ and lowers the type II error by $q(y)\,dy$, so b changes by $-q/p$ per unit of a. In terms of the loss, this is the **slope rule**

$$|T'(a)| = e^{-l(a)}, \tag{30.3}$$

where $l(a)$ is the threshold of the size-a test, the a-quantile of L under P (the test fires when $L < l(a)$, which happens with probability a under P).

30.2.2. Fact 2 — The best "gain" of a test is a hockey-stick divergence

Suppose each correct detection of D' earns one unit and each false alarm costs γ units. A test's net gain is then its power $1-b$ minus γa. The best gain over all tests is a divergence between P and Q that the book has already met: the **hockey-stick divergence** $E_\gamma(Q\|P) = \sup_S (Q(S) - \gamma P(S))$, the largest amount by which some event is more likely under Q than γ times its probability under P (Divergences). For any pair and any $\gamma \geq 0$,

$$\max_{a\in[0,1]} (1 - T(P,Q)(a) - \gamma a) = \sup_S (Q(S) - \gamma P(S)) = E_\gamma(Q\|P). \tag{30.4}$$

To see why, write the gain of a test ϕ as one integral: $1 - b - \gamma a = \mathbb{E}_Q\phi - \gamma\mathbb{E}_P\phi = \int \phi\,(q - \gamma p)$. The integrand is positive exactly where $q > \gamma p$, so the gain is largest when the test fires on those outputs and nowhere else, $\phi = \mathbf{1}[q > \gamma p]$. That test is the event $S = \{q > \gamma p\}$, and its gain is $\int (q - \gamma p)_+ = E_\gamma(Q\|P)$. The left side of Equation 30.4 reaches the same number in two stages: at each type I error a the largest power is $1 - T(a)$ by the definition of T, and maximizing over a then picks the best test overall.

At $\gamma = 1$ the gain is $1 - a - b$, the test's advantage, and $E_1(Q\|P)$ is the total variation (TV) distance. So Fact 2 contains Part 1's statement that the best advantage of any test is the TV distance.

30.2.3. Fact 3 — A Gaussian shift in d dimensions is a Gaussian shift in one

When the output is a vector with independent unit-variance Gaussian noise, only the direction of the shift between the two means carries evidence; the noise in every perpendicular direction is the same under both hypotheses. So the d-dimensional problem is a one-dimensional one with shift equal to the length of the mean difference. For $m \in \mathbb{R}^d$,

$$T(\mathcal{N}(0, I_d), \mathcal{N}(m, I_d)) = G_{\|m\|_2}, \qquad G_\mu(a) = \Phi\big(\Phi^{-1}(1-a) - \mu\big). \tag{30.5}$$

Here Φ is the standard normal distribution function, $\Phi(t) = \Pr[Z \le t]$ for $Z \sim \mathcal{N}(0,1)$, and Φ^{-1} its inverse.

Start with $d = 1$, which is Part 1's Gaussian trade-off curve (Equation 4.10). For $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$ the loss $\frac{\mu^2}{2} - \mu y$ is small when y is large, so the best test fires when $y > t$. Its type I error is $a = \Pr[Z > t] = 1 - \Phi(t)$, so the threshold of the size-a test is $t = \Phi^{-1}(1-a)$. Its type II error is the chance that a draw from $\mathcal{N}(\mu,1)$ stays below t, $b = \Phi(t-\mu)$. Substituting t gives G_μ.

For general d, the likelihood ratio is $\exp\left(\langle m, y\rangle - \frac{1}{2}\|m\|^2\right)$, from the same cancellation of squares as in Fact 1. It depends on y only through the single number $s = \langle m, y\rangle / \|m\|$, the length of the projection of y on the direction of the shift. By Neyman–Pearson, the optimal tests are thresholds on s. Under $\mathcal{N}(0, I)$, $s \sim \mathcal{N}(0,1)$; under $\mathcal{N}(m, I)$, $s \sim \mathcal{N}(\|m\|, 1)$. Both hold because the inner product of a unit vector with a standard normal vector is a standard normal number (its variance is the squared length of the unit vector, 1), and the mean m contributes $\langle m, m\rangle / \|m\| = \|m\|$. So every optimal test is a threshold test on $\mathcal{N}(0,1)$ against $\mathcal{N}(\|m\|, 1)$, whose curve is $G_{\|m\|}$.

Figure 30.3 (left) shows the picture: 400 draws from each of $\mathcal{N}(0, I_2)$ and $\mathcal{N}((1,1), I_2)$. The best boundary is the dashed line perpendicular to the shift. Nothing about the other direction helps.

30.3. f-DP

30.3.1. Context

A definition of privacy should say what an adversary cannot do. In hypothesis-testing terms the strongest such statement is a lower bound on the adversary's type II error at every type I error: however the adversary tunes its test, it must miss at least this often. That bound is a curve, so the definition takes a curve as its parameter.

30.3.2. Statement

Call f a trade-off function if it is the curve of some pair of distributions. Part 1 (Section 4.5) showed this means f is non-increasing, convex, continuous and satisfies $f(a) \le 1 - a$ (a coin flip that fires with probability a already achieves $b = 1 - a$). A mechanism M is **f-differentially private (f-DP)** if no neighbouring pair is easier to tell apart than f allows,

$$T(M(D), M(D')) \ge f \quad \text{for all neighbours } D \simeq D' \tag{30.6}$$

(Dong, Roth & Su 2022, Definition 2.3; cited as Dong et al. from here on).

Two consequences follow. Because the neighbouring relation is symmetric, an f-DP mechanism is also $\max(f, f^{-1})$**-DP**, where $f^{-1}(a) = \inf\{t \in [0,1] : f(t) \le a\}$ is the inverse of f, its graph reflected across the diagonal $a = b$ (Dong et al., Proposition 2.4). A curve with $f = f^{-1}$ is called **symmetric**. And the old definition is a special case: (ε, δ)-DP is exactly $f_{\varepsilon,\delta}$-DP for the curve made of two straight lines,

$$f_{\varepsilon,\delta}(a) = \max\left(0,\ 1 - \delta - e^{\varepsilon} a,\ e^{-\varepsilon}(1 - \delta - a)\right), \tag{30.7}$$

the curve that Part 3 derived in Section 18.7 (Equation 18.12; Wasserman & Zhou 2010; Kairouz, Oh & Viswanath 2015; Dong et al., Proposition 2.5).

30.3.3. Derivation

1. **Swap the roles, then invert.** Since $D' \simeq D$ is also a neighbouring pair, f-DP gives $T(M(D'), M(D)) \geq f$ as well. Now relate the curve of (Q, P) to the curve of (P, Q). Take a test ϕ of P against Q with errors (a, b) and use its opposite, $1 - \phi$, as a test of Q against P: it fires when the data was D' exactly when ϕ stays silent, so its type I error is $\mathbb{E}_Q[1 - \phi] = b$ and its type II error is $1 - \mathbb{E}_P[1 - \phi] = a$. So the achievable error pairs of (Q, P) are those of (P, Q) with the coordinates swapped, and $T(Q, P)$ is the inverse function of $T(P, Q)$, in the generalized sense of f^{-1} above when the curve has flat stretches. Part 1 proves this as Exercise 2 in the exercise book of Hypothesis testing.
2. **Reflecting keeps the order.** A curve lying above f means its region of achievable errors sits inside the region above f. Reflecting both regions across the diagonal keeps that inclusion, so $T(M(D'), M(D)) \geq f$ turns into $T(M(D), M(D')) \geq f^{-1}$. Together with $T(M(D), M(D')) \geq f$ this gives $\geq \max(f, f^{-1})$. The maximum of two convex, non-increasing functions is again convex and non-increasing, so this is a trade-off function.
3. **(ε, δ)-DP is a curve.** This is Part 3's hypothesis-testing reading (Section 18.7), restated in this chapter's letters. (ε, δ)-DP says that for every event S, $Q(S) \leq e^{\varepsilon} P(S) + \delta$ and $P(S) \leq e^{\varepsilon} Q(S) + \delta$. Read S as the region where a test fires, so $a = P(S)$ and $1 - b = Q(S)$. The first inequality becomes $1 - b \leq e^{\varepsilon} a + \delta$, that is $b \geq 1 - \delta - e^{\varepsilon} a$. The second, applied to the complement of S, where $P = 1 - a$ and $Q = b$, becomes $1 - a \leq e^{\varepsilon} b + \delta$, that is $b \geq e^{-\varepsilon}(1 - \delta - a)$. Together with $b \geq 0$ these are the three pieces of Equation 30.7. A randomized test is a mixture of such region tests, so its errors obey the same linear bounds. Conversely, if every test has $b \geq f_{\varepsilon,\delta}(a)$, then reading each event S as a test gives back the two DP inequalities. So $T(P, Q) \geq f_{\varepsilon,\delta}$ for every neighbouring pair exactly when M is (ε, δ)-DP.

30.3.4. Interpretation

f-DP contains (ε, δ)-DP: with $f = f_{\varepsilon,\delta}$ you get back the old definition, and other curves express guarantees that no single (ε, δ) pair can.

The shape of $f_{\varepsilon,\delta}$ shows what the two parameters do. It has two straight pieces, with slopes $-e^{\varepsilon}$ and $-e^{-\varepsilon}$. They meet on the diagonal $a = b$: setting $a = 1 - \delta - e^{\varepsilon} a$ gives $a = \frac{1-\delta}{1+e^{\varepsilon}}$. The curve starts at $1 - \delta$ on the b-axis, not at 1. That gap is what δ means: a test may catch D' with probability δ at no type I cost.

Post-processing comes for free. Processing the outputs can only make two distributions harder to tell apart, because any test on the processed output is also a test on the raw output, so the trade-off curve can only rise under processing (Part 3, Section 19.3; Dong et al., Lemma 2.9). So f-DP is closed under post-processing.

The curve is also the most informative summary there is. Blackwell's theorem (Dong et al., Theorem 2.10) says that $T(P, Q) \leq T(P', Q')$ holds exactly when some randomized map sends P to P' and Q to Q' at the same time. So comparing curves is the same as asking whether one pair can be turned into the other by post-processing. Any quantity that respects post-processing, such as a divergence, can therefore only be a coarsening of the curve.

30.3.5. Example: three mechanisms with "$\varepsilon = 1$"

Binary randomized response with $\varepsilon = 1$ reports the true bit with probability $e/(1+e) = 0.7311$ and flips it otherwise (randomized response; as a mechanism it protects one person's bit under replace-one neighbours, and here it serves as a pair of distributions). If the bit is 0 under D and 1 under D', the output distributions are $P = (0.7311, 0.2689)$ and $Q = (0.2689, 0.7311)$ over the outputs (0, 1). The only useful test is "fire when the output is 1", with $a = 0.2689$ and $b = 0.2689$; mixing it with "never fire" and "always fire" fills in the straight lines to $(0, 1)$ and $(1, 0)$. So the curve has a single interior corner at $(0.2689, 0.2689)$. That is exactly $f_{1,0}$, whose corner is at $1/(1+e) = 0.2689$ by the formula $\frac{1-\delta}{1+e^{\varepsilon}}$ above. The Laplace mechanism with sensitivity 1 and scale $1/\varepsilon$ is also $(1, 0)$-DP, and the Gaussian mechanism with $\sigma = 1$ is shown for comparison.

Table 30.2.: Smallest type II error at three type I errors for three mechanisms. The Laplace curve is Part 1's Equation 4.14 with $\kappa = 1$.

Type I error a	$f_{1,0}$ (randomized response)	Laplace, $\varepsilon = 1$	Gaussian, $\sigma = 1$ (G_1)
0.01	0.9728	0.9728	0.9076
0.1	0.7282	0.7282	0.6109
0.25	0.3204	0.3679	0.3724

The $f_{1,0}$ column is the first line $1 - e\,a$: at $a = 0.1$, $1 - 0.2718 = 0.7282$. Randomized response sits exactly on $f_{1,0}$: it is the worst $(1, 0)$-DP mechanism. The Laplace mechanism matches it at small a, where both have slope $-e$, but is strictly more private in the middle (0.3679 against 0.3204 at $a = 0.25$, Table 30.2). The label "$(1, 0)$-DP" cannot tell the two apart; their curves can. Figure 30.1 also shows $f_{1,0.1}$, which starts at $b = 0.9$ instead of 1: at $a = 0.1$ it allows $b = 0.9 - 0.2718 = 0.6282$, exactly $\delta = 0.1$ below $f_{1,0}$.

30.3.6. Check yourself

- Why must $f_{\varepsilon,\delta}$ be symmetric, $f_{\varepsilon,\delta} = f_{\varepsilon,\delta}^{-1}$? *(Its two pieces are reflections of each other across the diagonal: solving $b = 1 - \delta - e^{\varepsilon}a$ for a gives $a = e^{-\varepsilon}(1 - \delta - b)$, which is the second piece with a and b swapped. This mirrors the two DP inequalities, one for each order of D, D'.)*
- The Laplace curve in Table 30.2 coincides with $f_{1,0}$ for small a and lies above it in the middle. Is $\varepsilon = 1$ the smallest ε with $\delta = 0$ it satisfies? *(Yes. Near $a = 0$ both curves are $1 - e\,a$, and any $f_{\varepsilon,0}$ with $\varepsilon < 1$ is $1 - e^{\varepsilon}a$ there, which lies above the Laplace curve.)*

30.4. Gaussian differential privacy

30.4.1. Context

The Gaussian mechanism is the workhorse of private machine learning. By Fact 3, its curve is a Gaussian trade-off curve G_{μ}, which has one parameter. Using that family as the yardstick gives a one-number definition that is exact for the Gaussian mechanism.

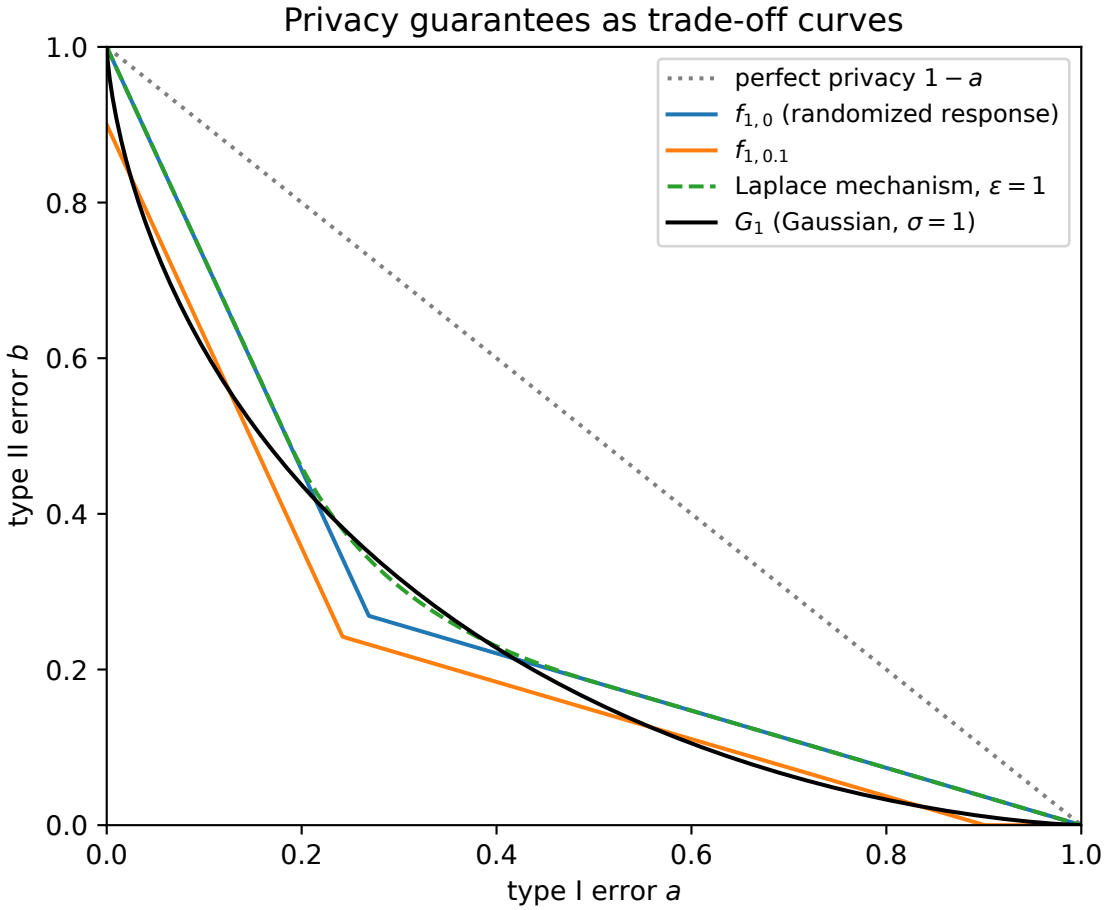


Figure 30.1.: Trade-off curves of $(1, 0)$-DP (randomized response), $(1, 0.1)$-DP, the Laplace mechanism with $\varepsilon = 1$ and the Gaussian mechanism with $\sigma = 1$. A mechanism is f-DP when its curve lies on or above f; the dotted diagonal is perfect privacy.

30.4.2. Statement

A mechanism is **μ-GDP** if it is G_μ-DP, with G_μ as in Equation 30.5 (Dong et al., Definition 2.6): no test can separate its neighbouring outputs better than one can separate $\mathcal{N}(0,1)$ from $\mathcal{N}(\mu,1)$. The Gaussian mechanism adds noise to a query g, $M(D) = g(D) + \mathcal{N}(0, \sigma^2\Delta_2^2 I)$, where Δ_2 is the ℓ_2 sensitivity of g (the largest Euclidean distance $\|g(D) - g(D')\|_2$ over neighbours) and σ the noise multiplier (the noise standard deviation in units of Δ_2). It is

$$\mu\text{-GDP with } \mu = \frac{1}{\sigma}, \tag{30.8}$$

and this is tight: when the sensitivity is attained by some neighbouring pair, that pair has trade-off curve exactly $G_{1/\sigma}$. μ-GDP is closed under post-processing.

30.4.3. Derivation

The plan is to turn the two output distributions, by invertible maps that cannot change any test's errors, into the standard pair of Fact 3, and then read off the shift.

1. **Standardize.** Divide the output by $\sigma\Delta_2$. The map is invertible, so any test on the scaled output is a test on the original and vice versa, and no error changes. After it, the noise has identity covariance: $P = \mathcal{N}(g(D)/(\sigma\Delta_2), I)$ and $Q = \mathcal{N}(g(D')/(\sigma\Delta_2), I)$.
2. **Reduce to one dimension.** Subtract $g(D)/(\sigma\Delta_2)$ from both, another invertible map, so that P is centred at 0. Fact 3 then gives $T(P,Q) = G_\nu$ with $\nu = \|g(D) - g(D')\|_2/(\sigma\Delta_2)$, the length of the remaining shift.
3. **Bound the shift.** By the definition of sensitivity, $\|g(D) - g(D')\|_2 \le \Delta_2$, so $\nu \le 1/\sigma$.
4. **A bigger shift is easier to detect.** $G_\mu(a) = \Phi(\Phi^{-1}(1-a) - \mu)$ decreases as μ grows, because Φ is increasing. So $G_\nu \ge G_{1/\sigma}$ for every neighbouring pair, which is the statement that the mechanism is $1/\sigma$-GDP. A pair with $\|g(D) - g(D')\|_2 = \Delta_2$ has $\nu = 1/\sigma$ and gives equality.
5. **Post-processing.** It is the special case $f = G_\mu$ of the post-processing property of f-DP in Section 30.3.

30.4.4. Interpretation

For the Gaussian mechanism, μ-GDP is not a bound with slack; it is the curve. Every other guarantee for this mechanism, in particular every (ε, δ) pair, follows from μ (Section 30.5).

μ has a direct reading: it is the distance between the two output means in units of the noise standard deviation. Part 1's Table 4.4 gives a feel for it. At a 5% false-alarm rate the threshold is $\Phi^{-1}(0.95) = 1.6449$, so at $\mu = 1$ the best test misses with probability $\Phi(1.6449 - 1) = 0.7405$ and has power $1 - 0.7405 = 0.2595$; at $\mu = 2$ it misses with probability $\Phi(-0.3551) = 0.3612$ and has power 0.6388. Dong et al. suggest that $\mu \le 0.5$ is a reasonable guarantee and $\mu \ge 6$ promises almost nothing (their discussion of Figure 3, after Definition 2.6).

The guarantee is set by the worst pair. A pair whose outputs differ by half the sensitivity has curve $G_{0.5}$ at $\sigma = 1$, more private than the guarantee G_1, which comes from the pairs that attain the full sensitivity.

30.4.5. Example: one noisy query

A count query has $\Delta_2 = 1$, since adding or removing one record changes the count by 1. Released with $\sigma = 1$, it is 1-GDP; with $\sigma = 4$, it is 0.25-GDP. For $\sigma = 1$, Table 30.3 evaluates G_1 at three type I errors, each by setting the threshold $\Phi^{-1}(1-a)$ and subtracting the shift 1. For example, at $a = 0.01$ the threshold is $\Phi^{-1}(0.99) = 2.3263$ and $G_1(0.01) = \Phi(1.3263) = 0.9076$.

Table 30.3.: The 1-GDP guarantee of the Gaussian mechanism with $\sigma = 1$, read as attack errors.

Type I error a	Smallest type II error $G_1(a)$	Best power $1 - G_1(a)$
0.01	0.9076	0.0924
0.05	0.7405	0.2595
0.1	0.6109	0.3891

A simulation of the threshold test at $a = 0.05$ with 1,000,000 draws per hypothesis gives $\hat{a} = 0.0502$ and $\hat{b} = 0.7408$, against the exact 0.7405 in Table 30.3. Read as a membership attack, the first row says: whatever the attacker knows about the rest of the data, at a 1% false-positive rate it detects the presence of a record at most 9.24% of the time.

30.4.6. Check yourself

- A query has ℓ_2 sensitivity 3 and you add $\mathcal{N}(0, 6^2 I)$. What is μ? *(The noise multiplier is the noise standard deviation in units of the sensitivity, $\sigma = 6/3 = 2$, so $\mu = 1/\sigma = 0.5$.)*
- Why is it enough to look at the one-dimensional statistic $\langle m, y\rangle / \|m\|$ when the query returns a vector? *(Fact 3: the likelihood ratio depends on y only through it, so by Neyman–Pearson no test can do better than thresholding it.)*

30.5. Duality with (ε, δ)

30.5.1. Context

Regulators, papers and libraries report (ε, δ). So you need to turn a curve into (ε, δ) pairs without losing anything, and you need the (ε, δ) profile of μ-GDP in closed form.

30.5.2. Statement

The idea is geometric. An (ε, δ) guarantee is the line $b = 1 - \delta - e^{\varepsilon} a$ (and its mirror image), and a curve satisfies it when the curve lies above the line. For a given slope $-e^{\varepsilon}$, the smallest δ that works is the one where the line just touches the curve. Doing this for every slope gives the function $\delta(\varepsilon)$, and since a convex curve is the upper envelope of the lines that touch it, these lines together carry all of the curve.

The tool that computes "the line of slope y that just touches f" is the **convex conjugate** $f^*(y) = \sup_a (ya - f(a))$: for each slope y it gives (minus) the intercept of the touching line.

In these terms: for a symmetric trade-off function f, a mechanism is f-DP if and only if it is $(\varepsilon, \delta(\varepsilon))$-DP for every $\varepsilon \geq 0$, with

$$\delta(\varepsilon) = 1 + f^*(-e^\varepsilon), \qquad f^*(y) = \sup_{a \in [0,1]} (ya - f(a)), \tag{30.9}$$

(Dong et al., Proposition 2.12). For $f = G_\mu$ the touching point can be found in closed form, and

$$\delta(\varepsilon) = \Phi\Big(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}\Big) - e^\varepsilon\, \Phi\Big(-\frac{\varepsilon}{\mu} - \frac{\mu}{2}\Big) \tag{30.10}$$

(Dong et al., Corollary 2.13). This is the exact privacy profile of the Gaussian mechanism, the same formula that Part 3 derived as a hockey-stick divergence (Equation 20.17 in Section 20.6; Balle & Wang 2018, Theorem 8). Here it comes out of the curve by convex duality.

30.5.3. Derivation of the duality

1. **One (ε, δ) pair is one line.** By Equation 30.7, $f \geq f_{\varepsilon,\delta}$ requires in particular $f(a) \geq 1 - \delta - e^\varepsilon a$ for all a. Rearranged, $\delta \geq 1 - f(a) - e^\varepsilon a$ for every a, so the smallest δ that works is $\sup_a (1 - f(a) - e^\varepsilon a)$. With $y = -e^\varepsilon$ in the definition of the conjugate, this is $1 + f^*(-e^\varepsilon)$, the quantity in Equation 30.9. The other piece of $f_{\varepsilon,\delta}$ is the mirror image of this line across the diagonal, and f is its own mirror image (it is symmetric), so the other piece gives the same condition.
2. **So f-DP implies every $(\varepsilon, \delta(\varepsilon))$.** If $T(M(D), M(D')) \geq f$ and, by step 1 with the smallest δ, $f \geq f_{\varepsilon,\delta(\varepsilon)}$, then $T(M(D), M(D')) \geq f_{\varepsilon,\delta(\varepsilon)}$.
3. **Conversely, the lines rebuild the curve.** A convex function is the supremum of the lines that touch it from below. For each $\varepsilon \geq 0$ the line $1 - \delta(\varepsilon) - e^\varepsilon a$ touches f where the slope of f is $-e^\varepsilon$. As ε runs over $[0, \infty)$ the slopes run over every value from -1 down, which covers the part of the curve left of the diagonal (a symmetric curve has slope -1 where it crosses the diagonal and is steeper to the left). The mirror pieces cover the rest by symmetry. So $\sup_{\varepsilon \geq 0} f_{\varepsilon,\delta(\varepsilon)} = f$, and a mechanism that is $(\varepsilon, \delta(\varepsilon))$-DP for all ε has $T \geq f_{\varepsilon,\delta(\varepsilon)}$ for every ε, hence $T \geq f$ (Dong et al., Proposition 2.11).
4. **The conjugate is a hockey-stick divergence.** When $f = T(P, Q)$, Fact 2 (Equation 30.4) with $\gamma = e^\varepsilon$ says $1 + f^*(-e^\varepsilon) = \max_a (1 - f(a) - e^\varepsilon a) = E_{e^\varepsilon}(Q \| P)$. So $\delta(\varepsilon)$ is the familiar privacy profile $\delta(\varepsilon) = E_{e^\varepsilon}$ of composition, the smallest δ that goes with each ε. For a symmetric pair the two orders, $E_{e^\varepsilon}(Q \| P)$ and $E_{e^\varepsilon}(P \| Q)$, agree.

30.5.4. Derivation of the Gaussian profile through the conjugate

To evaluate $1 + G_\mu^*(-e^\varepsilon) = \max_a (1 - G_\mu(a) - e^\varepsilon a)$, find the test where the curve has slope $-e^\varepsilon$, then read off its two errors.

1. **Find the touching point.** The expression $1 - G_\mu(a) - e^\varepsilon a$ is concave in a, because G_μ is convex, so its maximum is where its derivative vanishes, $G_\mu'(a) = -e^\varepsilon$. By the slope rule (Fact 1), the slope at the size-a threshold $t = \Phi^{-1}(1 - a)$ is minus the likelihood ratio there. For $\mathcal{N}(0, 1)$ against $\mathcal{N}(\mu, 1)$ that ratio is $q(t)/p(t) = e^{\mu t - \mu^2/2}$. Setting it equal to e^ε gives $\mu t - \mu^2/2 = \varepsilon$, so $t = \frac{\varepsilon}{\mu} + \frac{\mu}{2}$.

2. **Read off the errors.** The test fires when $y > t$. Its type I error is $a^* = 1 - \Phi(t) = \Phi(-\frac{\varepsilon}{\mu} - \frac{\mu}{2})$, using $1 - \Phi(t) = \Phi(-t)$. Its power is the chance a draw from $\mathcal{N}(\mu, 1)$ exceeds t, $1 - b^* = 1 - \Phi(t - \mu) = \Phi(-\frac{\varepsilon}{\mu} + \frac{\mu}{2})$.
3. **Substitute** into $\delta = 1 - b^* - e^{\varepsilon} a^*$ to get Equation 30.10.
4. **Agreement with Part 3.** By Fact 2 this route computes $E_{e^{\varepsilon}}(Q\|P)$. Part 3 instead computed $E_{e^{\varepsilon}}(P\|Q) = P[L > \varepsilon] - e^{\varepsilon} Q[L > \varepsilon]$ from the Gaussian law of the loss (Fact 1: $L \sim \mathcal{N}(\frac{\mu^2}{2}, \mu^2)$ under P and $\mathcal{N}(-\frac{\mu^2}{2}, \mu^2)$ under Q). The two agree because the Gaussian pair is symmetric: reflecting the line $y \mapsto \mu - y$ swaps $\mathcal{N}(0, 1)$ and $\mathcal{N}(\mu, 1)$, so swapping P and Q changes nothing.

30.5.5. Interpretation

The duality is lossless both ways. Reporting the whole profile $\varepsilon \mapsto \delta(\varepsilon)$ loses nothing, because the lines rebuild the curve. Reporting one point, such as ε at $\delta = 10^{-5}$, keeps one touching line and drops the rest of the curve (Figure 30.2, right).

At $\varepsilon = 0$ the profile is the TV distance. The line of slope -1 touches a symmetric curve on the diagonal, at the test with the best advantage $1 - a - b$. For G_{μ} that test has threshold $t = \mu/2$, halfway between the means, and Equation 30.10 at $\varepsilon = 0$ gives $\delta(0) = \Phi(\mu/2) - \Phi(-\mu/2) = 2\Phi(\mu/2) - 1$, Part 1's best advantage.

Small δ costs ε, in proportion to μ. To make δ small, the first term of Equation 30.10, a Gaussian tail $\Phi(-x)$ with $x = \frac{\varepsilon}{\mu} - \frac{\mu}{2}$, must be small. A normal tail falls roughly like $e^{-x^2/2}$, so reaching δ needs x of about $\sqrt{2\ln(1/\delta)}$, and ε grows roughly like $\mu\sqrt{2\ln(1/\delta)}$. Each further factor of ten in δ therefore costs extra ε, a little less with each decade (a square root grows ever more slowly), and about μ times as much as at $\mu = 1$. At $\mu = 1$, going from $\delta = 10^{-4}$ to 10^{-5} to 10^{-6} raises ε from 3.8044 to 4.3772 to 4.8866 (steps of 0.57 and 0.51); at $\mu = 2$, from 8.8769 to 9.9973 to 10.9972 (steps of 1.12 and 1.00, about twice as large). Figure 30.2 (left) and Table 30.4 show the same growth.

30.5.6. Example: reading $\delta(1)$ of 1-GDP off the curve

At $\mu = 1$ and $\varepsilon = 1$ the touching threshold is $t = \frac{1}{1} + \frac{1}{2} = 1.5000$. The best test where the slope is $-e$ has type I error $a^* = \Phi(-1.5) = 0.066807$ and power $1 - b^* = \Phi(1 - 1.5) = \Phi(-0.5) = 0.308538$, so $\delta(1) = 0.308538 - e \times 0.066807 = 0.308538 - 0.181601 = 0.126937$. These are the numbers of Part 3's Table 20.6, read in a new way. There, 0.3085 was the probability of the bad event $\{L > 1\}$, where the privacy loss exceeds ε, and 0.1816 its correction. Here they are the power of the best test and the price of its type I error at the exchange rate e. As a check on step 1, maximizing $1 - G_1(a) - e\,a$ over a grid of 20,001 values of a gives the same 0.126937, at $a = 0.0668$.

Going the other way, from a target δ to ε, has no closed form, but $\delta(\varepsilon)$ decreases in ε, so the function `gdp_eps` finds the ε where it equals the target by bisection. At $\delta = 10^{-5}$:

Table 30.4.: The ε of μ-GDP at $\delta = 10^{-5}$.

μ	0.5	1	2
ε at $\delta = 10^{-5}$	1.9931	4.3772	9.9973

To check step 3 of the duality, the script rebuilds G_1 from 1001 lines $f_{\varepsilon,\delta(\varepsilon)}$ with $\varepsilon \in [0, 10]$: their supremum is within 4.9×10^{-6} of G_1 on a grid of 999 values of a.

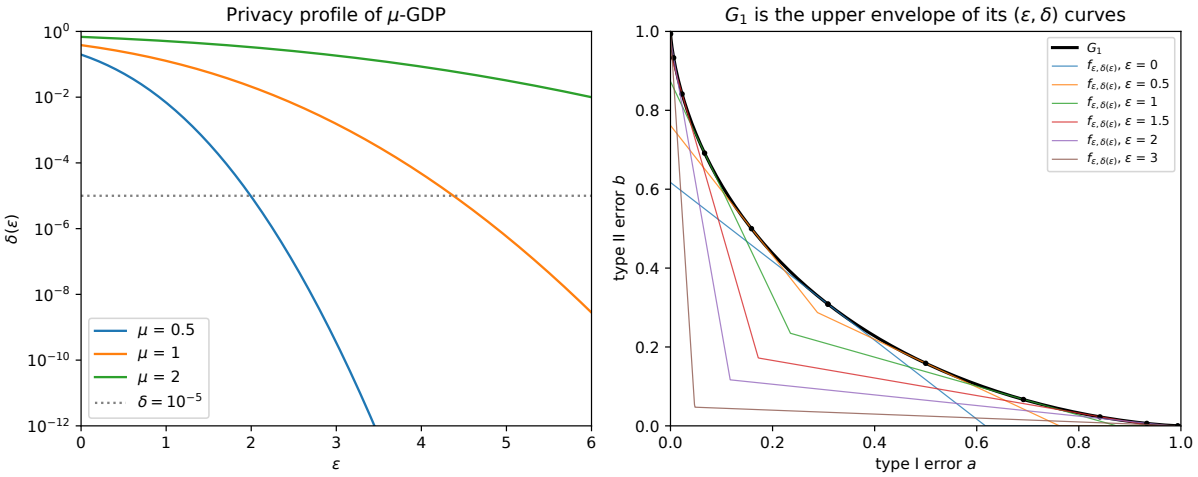


Figure 30.2.: Left: the privacy profile $\delta(\varepsilon)$ of μ-GDP for $\mu = 0.5, 1, 2$, with the level $\delta = 10^{-5}$ dotted. Right: G_1 and six of its $(\varepsilon, \delta(\varepsilon))$ curves. Each touches G_1 at two mirror-image points (dots); their upper envelope is G_1.

30.5.7. Check yourself

- From Figure 30.2 (right), which part of G_1 does the $\varepsilon = 0$ line describe? *(The point where the slope is -1, on the diagonal at $a = b = \Phi(-1/2) = 0.3085$: the best-advantage test. Larger ε describe points closer to the axes.)*
- Is "1-GDP" the same guarantee as "$(4.3772, 10^{-5})$-DP"? *(No. 1-GDP implies it, but the single curve $f_{4.3772,10^{-5}}$ allows far more power: at $a = 0.1$ its second piece allows $b = e^{-4.3772}(1 - 10^{-5} - 0.1) = 0.0113$, while $G_1(0.1) = 0.6109$.)*

30.6. Composition

30.6.1. Context

Training runs a mechanism thousands of times on the same data. Composing (ε, δ) guarantees directly is lossy (composition). Composing curves is exact, and for Gaussian curves it is a one-line formula.

30.6.2. Statement

When an adversary sees the outputs of two mechanisms run with independent randomness, it faces the pair of joint distributions: $P \times P'$ (both outputs drawn on D) against $Q \times Q'$ (both drawn

on D'). The curve of that joint pair defines the composition of curves. The **tensor product** of two trade-off functions $f = T(P, Q)$ and $g = T(P', Q')$ is

$$f \otimes g = T(P \times P', Q \times Q'), \tag{30.11}$$

the curve of the pair of product distributions. It does not depend on which pairs are used to represent f and g (Dong et al., Definition 3.1). If each mechanism M_i is f_i-DP, even when it is chosen adaptively from earlier outputs, then the composition $(M_1, \dots, M_k)$ is $f_1 \otimes \cdots \otimes f_k$-DP, and no better bound holds in general (Dong et al., Theorem 3.2). For Gaussian curves the product is again Gaussian, with the shifts adding like the sides of a right-angled triangle:

$$G_{\mu_1} \otimes G_{\mu_2} \otimes \cdots \otimes G_{\mu_k} = G_\mu, \qquad \mu = \sqrt{\mu_1^2 + \cdots + \mu_k^2} \tag{30.12}$$

(Dong et al., Corollary 3.3).

30.6.3. Derivation

1. **Non-adaptive composition through Blackwell.** Suppose $T(P_i, Q_i) \geq f_i = T(\tilde{P}_i, \tilde{Q}_i)$, where $(\tilde{P}_i, \tilde{Q}_i)$ is some pair whose curve is exactly f_i. By Blackwell's theorem (Dong et al., Theorem 2.10), a curve lies above another exactly when its pair can be produced from the other pair by post-processing, so some randomized map K_i sends $\tilde{P}_i \mapsto P_i$ and $\tilde{Q}_i \mapsto Q_i$. Apply the maps coordinatewise and independently: $(K_1, \dots, K_k)$ sends $\prod \tilde{P}_i \mapsto \prod P_i$ and $\prod \tilde{Q}_i \mapsto \prod Q_i$. Post-processing cannot help a test, so $T(\prod P_i, \prod Q_i) \geq T(\prod \tilde{P}_i, \prod \tilde{Q}_i) = \bigotimes f_i$.
2. **Adaptive composition.** When M_2 depends on the output of M_1, the second coordinate is not independent of the first, and step 1 does not apply. Dong et al. prove the same bound with the Neyman–Pearson lemma alone, conditioning on the first output (their Theorem 3.2 and Appendix C). This chapter uses the result without re-proving it.
3. **Tightness.** If the mechanisms ignore each other's outputs and some neighbouring pair attains each f_i, the composed pair is exactly the product pair, so its curve is exactly $\bigotimes f_i$.
4. **Gaussian curves.** $G_{\mu_i} = T(\mathcal{N}(0,1), \mathcal{N}(\mu_i, 1))$, so $\bigotimes G_{\mu_i}$ is the curve of k independent coordinates, $T(\mathcal{N}(0, I_k), \mathcal{N}(m, I_k))$ with $m = (\mu_1, \dots, \mu_k)$. By Fact 3, this is $G_{\|m\|_2}$, and $\|m\|_2 = \sqrt{\mu_1^2 + \cdots + \mu_k^2}$, which is Equation 30.12.

30.6.4. Interpretation

Under composition, the quantity that adds up is μ^2, just as variances of independent noises add. With k identical steps, μ grows like $\sqrt{k}$. Full-batch noisy gradient descent with clipped gradients and noise multiplier σ for k steps is a composition of k Gaussian mechanisms, each $1/\sigma$-GDP, so it is $\sqrt{k/\sigma^2} = \sqrt{k}/\sigma$-GDP. By step 3 it is in the worst case no better: no accountant can improve on it, and none needs to do more.

Concentrated DP is the same arithmetic. In zero-concentrated DP the Gaussian mechanism has $\rho = \mu^2/2$, and ρ adds under composition (concentrated DP), which is the same as μ^2 adding. The difference is what you can read off at the end: μ-GDP gives the exact curve, ρ only an RDP bound on it (Section 30.9).

For other curves the tensor product is hard. For $f_{\varepsilon,\delta}$ the product is a supremum of many $(\varepsilon_i, \delta_i)$ curves, which is the optimal composition theorem of Kairouz, Oh & Viswanath. Computing it exactly is #P-complete, a class of counting problems believed to have no efficient algorithm, when the $(\varepsilon_i, \delta_i)$ differ from step to step (Murtagh & Vadhan 2016; Part 3, Section 21.5). So numerical accountants compute $\bigotimes f_i$ another way: since the composed privacy loss is the sum of the independent per-step losses, they discretize the PRV and convolve its law with itself. These are the privacy loss distribution (PLD) and PRV accountants of numerical accounting.

30.6.5. Example: two concrete products

Gaussian. By Equation 30.12, two 1-GDP releases compose to $G_{\sqrt{2}} = G_{1.4142}$; three releases with $\mu = (0.5, 0.5, 1)$ to $G_{\sqrt{0.25+0.25+1}} = G_{1.2247}$; four 1-GDP releases to $G_{\sqrt{4}} = G_2$; and 100 releases with $\sigma = 10$ each, so $\mu_i = 0.1$, to $G_{\sqrt{100\times 0.01}} = G_1$. A simulation of the projected test in two dimensions, $\mu = (1, 1)$, at $a = 0.05$ with 1,000,000 draws per hypothesis gives $\hat{a} = 0.0497$ and $\hat{b} = 0.5913$, against $G_{\sqrt{2}}(0.05) = 0.5912$. The composed release sits strictly between one release, $G_1(0.05) = 0.7405$, and a single release with twice the shift, $G_2(0.05) = 0.3612$ (Figure 30.3).

Randomized response twice. This shows the tensor product for curves that are not Gaussian. Run randomized response with $\varepsilon = 1$ on the same bit twice, independently. Part 3 tabulated the same product at $\varepsilon = \ln 3$ as a sum of privacy losses (Table 21.2); here it is read as a curve. Each run keeps the bit with probability 0.7311. Under P (bit 0) and Q (bit 1) the four output pairs have the probabilities of Table 30.5; for example, 00 has probability $e^2/(1+e)^2 = 0.5344$ under P (both runs keep the bit) and $1/(1+e)^2 = 0.0723$ under Q (both runs flip it).

Table 30.5.: Two independent runs of randomized response with $\varepsilon = 1$.

Outputs	p	q	q/p
00	0.5344	0.0723	0.1353
01	0.1966	0.1966	1.0000
10	0.1966	0.1966	1.0000
11	0.0723	0.5344	7.3891

By Neyman–Pearson, the best tests add outcomes to the firing region in decreasing order of the likelihood ratio q/p (Table 30.5): first 11 ($q/p = e^2$), then the two mixed pairs ($q/p = 1$), then 00 (e^{-2}). Each addition raises a by the outcome's p and lowers b by its q. Firing on 11 alone gives $(a, b) = (0.0723, 1 - 0.5344) = (0.0723, 0.4656)$; adding one mixed pair gives $(0.2689, 0.2689)$; adding both gives $(0.4656, 0.0723)$. So the curve $f_{1,0} \otimes f_{1,0}$ is the polygon through $(0, 1)$, $(0.0723, 0.4656)$, $(0.2689, 0.2689)$, $(0.4656, 0.0723)$ and $(1, 0)$, with straight segments in between from randomizing at the boundary outcome. Its slopes are minus the likelihood ratios, $-e^2$, -1 and $-e^{-2}$. An $f_{\varepsilon,\delta}$ curve has only two slopes, so this curve is not $f_{\varepsilon,\delta}$ for any pair. It is the supremum of two such curves: $f_{2,0}$, whose first line $1 - e^2 a$ is the steep segment, and $f_{0,0.4621}$, whose line $1 - \delta - a$ is the middle segment, with $\delta = 1 - 0.0723 - 0.4656 = 0.4621$, which equals $\tanh(1/2)$ (Exercise 3 in the exercise book). This is the smallest instance of the fact that composed (ε, δ) guarantees are a family of pairs, not one pair.

Composing points loses. Four 1-GDP releases are 2-GDP, so Table 30.4 gives $\varepsilon = 9.9973$ at $\delta = 10^{-5}$. Suppose instead you first turn each release into one (ε, δ) point, giving each a quarter

of the δ budget, $(4.6902, 2.5 \times 10^{-6})$, and then compose the points with basic composition, which adds the ε's and the δ's. You get $\varepsilon = 18.7607$ at $\delta = 4 \times 2.5 \times 10^{-6} = 10^{-5}$: almost twice as large. Even optimal composition of the points loses, because each point has already dropped most of its curve; Part 3's Figure 21.4 shows by how much for many steps.

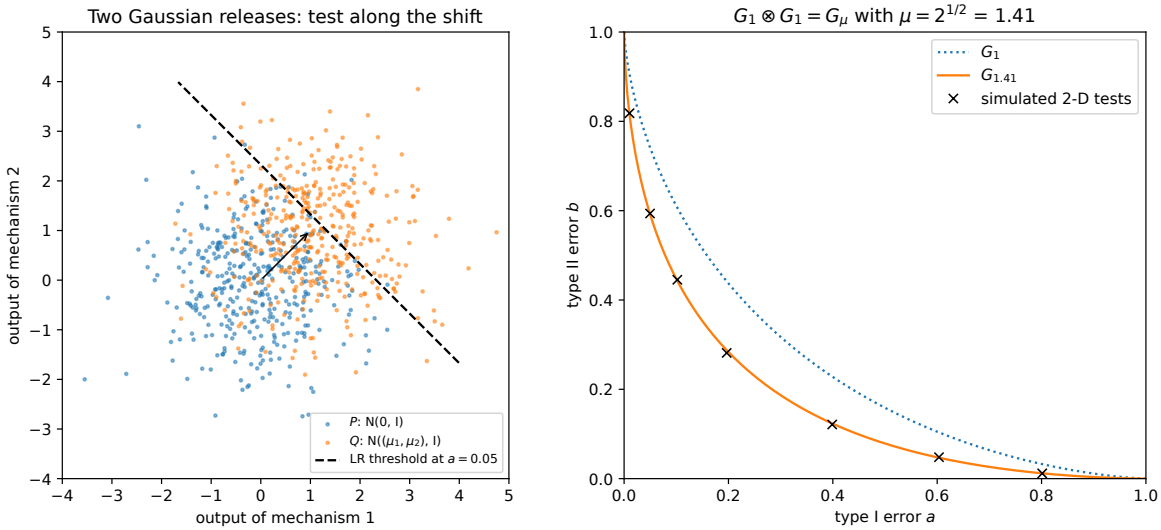


Figure 30.3.: Left: 400 draws each from $\mathcal{N}(0, I_2)$ and $\mathcal{N}((1,1), I_2)$, the outputs of two Gaussian releases on D and D'. The best test thresholds the projection on the shift (arrow); the dashed line is its boundary at $a = 0.05$. Right: G_1 for one release, $G_{\sqrt{2}}$ for two, and simulated errors of the projected test (crosses, 20,000 draws per point).

30.6.6. Check yourself

- You may release a vector query with $\Delta_2 = 1$ either once with $\sigma = 1$ or twice, independently, with $\sigma = \sqrt{2}$ each. Which is more private? *(Neither: each of the two releases is $1/\sqrt{2}$-GDP, and $\sqrt{(1/\sqrt{2})^2 + (1/\sqrt{2})^2} = 1$, the same G_1.)*
- Why does the product of two $f_{\varepsilon,0}$ curves have a segment of slope -1? *(Outcomes with equal likelihood under P and Q, here the mixed pairs 01 and 10, carry no evidence; adding them to the firing region trades type I for type II error one for one.)*

30.7. The central limit theorem for composition

30.7.1. Context

Most mechanisms are not Gaussian, so Equation 30.12 does not apply directly. But the privacy loss of a composition is the sum of the independent per-step losses, because the densities of independent outputs multiply and logs turn products into sums. Sums of many small independent terms are nearly normal, and by Fact 1 a normal privacy loss means a Gaussian curve. The CLT for f-DP turns this into a statement about curves.

30.7.2. Statement

To use the CLT you need the mean, the variance and a third-moment size of the privacy loss. Dong et al. write them as functionals of the curve itself, through its slope: by the slope rule of Fact 1, $-\ln|f'(a)|$ is the threshold loss $l(a)$, so averaging powers of it over $a \in [0,1]$ averages powers of the loss (the derivation makes this precise). For a trade-off function $f = T(P,Q)$ define (Dong et al., §3.2)

$$\mathrm{kl}(f) = -\int_0^1 \ln|f'(a)|\,da, \quad \kappa_2(f) = \int_0^1 \ln^2|f'(a)|\,da, \quad \kappa_3(f) = \int_0^1 \big|\ln|f'(a)|\big|^3 da, \quad \bar\kappa_3(f) = \int_0^1 \big|\ln|f'(a)| + \tag{30.13}$$

In words: kl is the mean of the loss, κ_2 its second moment, κ_3 its third absolute moment and $\bar\kappa_3$ its third absolute moment about the mean.

Berry–Esseen form (Dong et al., Theorem 3.4). For k curves, collect each functional into a vector, $\mathrm{kl} = (\mathrm{kl}(f_1),\dots,\mathrm{kl}(f_k))$ and so on, and write $\|\cdot\|_1$ for the sum of the entries and $\|\cdot\|_2^2$ for the sum of their squares. Then $\|\mathrm{kl}\|_1$ is the mean of the summed loss and $\|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2$ its variance. The theorem matches the composition to the Gaussian curve whose loss has the same ratio of mean to standard deviation, and bounds the error by the Berry–Esseen ratio of third moment to variance$^{3/2}$. Let $f_1,\dots,f_k$ be symmetric with $\kappa_3(f_i) < \infty$, and put

$$\mu = \frac{2\|\mathrm{kl}\|_1}{\sqrt{\|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2}}, \qquad \gamma = \frac{0.56\,\|\bar\kappa_3\|_1}{\left(\|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2\right)^{3/2}}. \tag{30.14}$$

If $\gamma < \frac{1}{2}$, then for all $a \in [\gamma, 1-\gamma]$ the composed curve lies in a band around G_μ of width γ in both directions:

$$G_\mu(a+\gamma) - \gamma \le f_1 \otimes \cdots \otimes f_k(a) \le G_\mu(a-\gamma) + \gamma. \tag{30.15}$$

Limit form (Dong et al., Theorem 3.5). As the number of curves grows and each one leaks less, the band shrinks to zero. Precisely, for a triangular array $f_{k1},\dots,f_{kk}$ of symmetric curves (a fresh list of k curves for each k) with $\sum_i \mathrm{kl}(f_{ki}) \to K \ge 0$, $\max_i \mathrm{kl}(f_{ki}) \to 0$, $\sum_i \kappa_2(f_{ki}) \to s^2 > 0$ and $\sum_i \kappa_3(f_{ki}) \to 0$, the composition converges uniformly to $G_{2K/s}$. For $(\varepsilon_i, 0)$-DP mechanisms with $\sum_i \varepsilon_i^2 \to \mu^2$ and $\max_i \varepsilon_i \to 0$, the limit is G_μ (Dong et al., Theorem 3.6).

30.7.3. Derivation (the idea; the proof is in Dong et al., Appendix D)

1. **The functionals are moments of the PRV.** By the slope rule (Fact 1), $-\ln|f'(a)| = l(a)$, the a-quantile of L under P. Now use the quantile transform: if U is uniform on $[0,1]$, then $l(U)$ has the law of L, just as feeding uniform random numbers into the inverse of a distribution function produces samples from that distribution. So $\int_0^1 g(l(a))\,da = \mathbb{E}[g(l(U))] = \mathbb{E}_P[g(L)]$ for any function g. Taking $g(x) = x$, x^2 and $|x - \mathbb{E}_P L|^3$ gives

$$\mathrm{kl}(f) = \mathbb{E}_P[L] = D_{\mathrm{KL}}(P\|Q), \quad \kappa_2(f) = \mathbb{E}_P[L^2], \quad \bar\kappa_3(f) = \mathbb{E}_P\big|L - \mathbb{E}_P L\big|^3. \tag{30.16}$$

   The first is the Kullback–Leibler divergence, the expected log likelihood ratio. And $\kappa_2 - \mathrm{kl}^2$ is the second moment minus the squared mean, the variance of the PRV.

2. **Composition adds independent losses.** For product distributions the densities multiply, so the composed loss is $L_1 + \cdots + L_k$ with independent terms. Means add and, for independent terms, variances add. So under P the composed loss has mean $K = \|\mathrm{kl}\|_1$ and variance $V = \|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2$.
3. **The ordinary CLT.** When no term dominates, the sum is close to $\mathcal{N}(K, V)$. The Berry–Esseen theorem says how close: the distribution function of a sum of independent terms differs from that of the matching normal by at most a constant times the summed third absolute central moments divided by $V^{3/2}$. Here that is $\|\bar{\kappa}_3\|_1/V^{3/2}$, and with the constant 0.56 it is the γ of Equation 30.14.
4. **Match a Gaussian pair.** By Fact 1 the pair behind G_μ has PRV $\mathcal{N}(\frac{\mu^2}{2}, \mu^2)$ under P, whose mean-to-standard-deviation ratio is $\frac{\mu^2/2}{\mu} = \mu/2$. Matching that ratio to $K/\sqrt{V}$ gives $\mu = 2K/\sqrt{V}$, Equation 30.14.
5. **Why match the ratio and not the variance alone.** A normal law $\mathcal{N}(m, v)$ is the law of some PRV only if $\mathbb{E}_P[e^{-L}] = 1$ (the change of measure of Fact 1 with $A =$ everything). By the normal moment generating function, $\mathbb{E}[e^{-L}] = e^{-m+v/2}$, so the condition is $m = v/2$, as in the Gaussian pair, where $m = \mu^2/2$ and $v = \mu^2$. Sums of small losses satisfy this only approximately, so at finite k different matches give slightly different μ; matching the ratio is Dong et al.'s choice. In the limit the two agree: $K = s^2/2$, and then $2K/s = s$. Dong et al. observe this in all their examples, and for symmetric curves it follows in a few lines. Symmetry means the law of the summed loss under Q is the mirror image of its law under P, so if $L \to \mathcal{N}(K, s^2)$ under P, then $L \to \mathcal{N}(-K, s^2)$ under Q. For every $c > 0$ the change of measure gives $\mathbb{E}_P[e^{-L}\mathbf{1}\{L > -c\}] = Q[L > -c]$. The integrand on the left is bounded by e^c and jumps only at $-c$, a point of probability 0 under the normal limit, so both sides pass to the limit: $\mathbb{E}[e^{-Z}\mathbf{1}\{Z > -c\}] = \Pr[Z' > -c]$ with $Z \sim \mathcal{N}(K, s^2)$ and $Z' \sim \mathcal{N}(-K, s^2)$. Letting $c \to \infty$ gives $\mathbb{E}[e^{-Z}] = 1$, that is $e^{-K+s^2/2} = 1$, so $K = s^2/2$.
6. **From the law of the loss to the curve.** By Fact 1 the curve is a functional of the law of the loss under P. Since that law is close to the Gaussian pair's, so is the curve.

30.7.4. Interpretation

This is why GDP is the natural family. The limit of many small compositions is always some G_μ, and the family is closed under composition. The (ε, δ) family is not: two runs of randomized response already leave it (Table 30.5).

For ε-DP pieces, μ is the root sum of squares, $\mu = \sqrt{\sum \varepsilon_i^2}$, in the limit. That is the $\sqrt{k}$ behaviour of advanced composition, with the exact constant.

The CLT is an approximation, not a bound. The Berry–Esseen band γ is a bound, but it is wide: 0.0562 at $k = 100$ in the example below. The point estimate G_μ can be above or below the true curve, and the error is largest in the tails. That is where small δ lives, because a small δ is decided by rare, large values of the loss, and a CLT controls the bulk of a distribution much better than its tails.

30.7.5. Example: composing randomized response and Laplace

Take k mechanisms, each ε_0-DP with $\varepsilon_0 = 1/\sqrt{k}$, so that $\sum \varepsilon_i^2 = k \cdot \frac{1}{k} = 1$ and the limit is G_1. To judge the CLT you need the true $\delta(1)$ of the composition, and that requires the exact law of

the summed loss.

For randomized response this law is simple. Under P each run reports the true bit with probability $e^{\varepsilon_0}/(1+e^{\varepsilon_0})$, giving loss $+\varepsilon_0$, and flips it otherwise, giving $-\varepsilon_0$. If $B \sim \mathrm{Bin}(k, e^{\varepsilon_0}/(1+e^{\varepsilon_0}))$ counts the truthful runs, the summed loss is $\varepsilon_0 B - \varepsilon_0(k-B) = \varepsilon_0(2B-k)$. The profile is the hockey-stick divergence $\int (p - e^{\varepsilon} q)_+$, which after dividing inside by p is $\delta(\varepsilon) = \mathbb{E}_P[(1 - e^{\varepsilon - L})_+]$, an exact finite sum over the $k+1$ values of B. For the Laplace mechanism the loss has two atoms at $\pm\varepsilon_0$ and a continuous part in between. The script rounds the continuous part down, and then up, to a lattice of spacing $\varepsilon_0/400$, convolves k copies by the fast Fourier transform, and reports both values. Since $\delta(\varepsilon)$ only grows when any loss grows, the two values bracket the exact one.

Table 30.6.: $\delta(1)$ for k-fold randomized response and Laplace with $\varepsilon_0 = 1/\sqrt{k}$, exactly (or bracketed) and from the CLT. The limit G_1 has $\delta(1) = 0.1269$.

k	ε_0	Randomized response, exact $\delta(1)$	Randomized response: μ, γ of Equation 30.14	Randomized response: GDP $\delta(1)$	Laplace $\delta(1)$	Laplace: μ, γ	Laplace: GDP $\delta(1)$
1	1.0000	0.0000	1.0422, 0.7663	0.1420	0.0000	0.9075, 0.7348	0.0956
10	0.3162	0.1323	1.0042, 0.1837	0.1284	[0.1112, 0.1114]	0.9569, 0.1889	0.1120
100	0.1000	0.1257	1.0004, 0.0562	0.1271	[0.1211, 0.1214]	0.9844, 0.0569	0.1215
1000	0.0316	0.1269	1.0000, 0.0177	0.1270	[0.1250, 0.1252]	0.9948, 0.0178	0.1251

Three things stand out in Table 30.6. The CLT with the finite-k μ of Equation 30.14 is accurate in the bulk from $k = 100$ on (0.1271 against 0.1257 for randomized response; 0.1215 against the bracket $[0.1211, 0.1214]$ for Laplace). At $k = 10$ the randomized-response value 0.1323 lies *above* the CLT's 0.1284: the CLT under-reports δ there. And Laplace stays on the private side of G_1 at every k shown (its $\delta(1)$ is below 0.1269), while its CLT μ (0.9569, 0.9844, 0.9948) rises toward 1 from below.

The tails are different. At $\delta = 10^{-5}$:

Table 30.7.: ε at $\delta = 10^{-5}$ for k-fold randomized response with $\varepsilon_0 = 1/\sqrt{k}$.

k	Randomized response, exact ε	GDP with the μ of Equation 30.14	GDP with $\mu = 1$	Basic composition $k\varepsilon_0$
10	3.1599	4.3983	4.3772	3.1623
100	4.3068	4.3793	4.3772	10.0000
1000	4.3683	4.3774	4.3772	31.6228

At $k = 10$ the true loss can never exceed $k\varepsilon_0 = 10/\sqrt{10} = 3.1623$, so the exact ε (3.1599) is far below the Gaussian's 4.3983: a Gaussian tail does not know that the loss is bounded. By $k = 1000$

the three values agree to within 0.01 (Table 30.7). In the tail the CLT erred upward at every k. In the bulk (Table 30.6) it erred downward at $k = 10$ and slightly upward from $k = 100$ on. Figure 30.4 shows both effects.

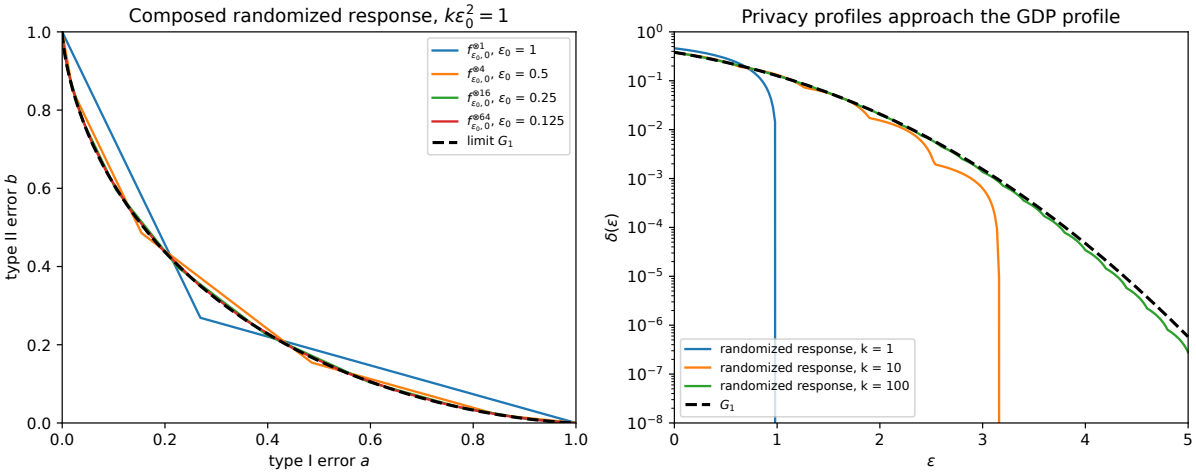


Figure 30.4.: Left: the exact curve $f_{\varepsilon_0,0}^{\otimes k}$ of k-fold randomized response with $\varepsilon_0 = 1/\sqrt{k}$, for $k = 1, 4, 16, 64$, approaching G_1. Right: their privacy profiles $\delta(\varepsilon)$ on a log scale for $k = 1, 10, 100$; each drops to 0 at $\varepsilon = k\varepsilon_0 = \sqrt{k}$ (off the plot for $k = 100$), while the GDP profile has a Gaussian tail.

30.7.6. Check yourself

- Why is $\kappa_2(f_{\varepsilon,0}) = \varepsilon^2$ exactly? *(By step 1, κ_2 is the second moment of the loss. The loss of randomized response is $\pm\varepsilon$, so its square is always ε^2.)*
- In Table 30.6, why is the CLT useless at $k = 1$? *(One term is not a sum: $\gamma = 0.7663 > \frac{1}{2}$, so the theorem gives no band, and the true $\delta(1)$ is 0, since one $(1, 0)$-DP run never violates $(1, 0)$.)*

30.8. The subsampled Gaussian and DP-SGD

30.8.1. Context

DP-SGD (subsampling) composes k steps. Each step forms a batch by Poisson subsampling, which includes each record independently with probability q, clips each gradient to norm at most the sensitivity, and adds Gaussian noise with noise multiplier σ to their sum. Each step leaks little when q is small, because the target record is usually not even in the batch, so the CLT applies. Under add/remove neighbours, where D' is D with one record added, one subsampled Gaussian step reduces to the pair of step 1 below: a Gaussian against a two-component mixture (subsampling derives it). The reason is short: the other records contribute the same to both outputs and can be subtracted, and the added record's clipped gradient, at worst a shift of one sensitivity, is present with probability q and absent otherwise. So the derivation here stands on

its own. In this section q is the sampling rate; the two output densities of one step are $P(y)$ and $Q(y)$.

30.8.2. Statement

Under add/remove neighbours, if $k \to \infty$ and $q\sqrt{k} \to c$ with σ fixed, k steps of the Poisson-subsampled Gaussian mechanism are asymptotically μ-GDP with

$$\mu = q\sqrt{k(e^{1/\sigma^2} - 1)} \tag{30.17}$$

(Bu, Dong, Long & Su 2020, Section 3, proved as their Theorem 5; Dong et al., Theorem 5.2 and Corollary 5.4, prove the analogous limit for their fixed-size batches under replace-one, with a different constant). This is a limit, used in practice as an approximation at finite k.

30.8.3. Derivation (heuristic, through Equation 30.14)

The plan is to compute the mean and variance of one step's privacy loss for small q, add them over k steps, and plug them into $\mu = 2K/\sqrt{V}$ from the CLT.

1. **One step.** Work in units of the sensitivity, so a present record shifts the mean by 1. The pair is $P = \mathcal{N}(0, \sigma^2)$ (the record is absent) and $Q = (1-q)\mathcal{N}(0, \sigma^2) + q\mathcal{N}(1, \sigma^2)$ (present, so with probability q it is in the batch). The ratio of the $\mathcal{N}(1, \sigma^2)$ density to the $\mathcal{N}(0, \sigma^2)$ density is $e^{(2y-1)/(2\sigma^2)} = e^{(y-1/2)/\sigma^2}$, from the same cancellation of squares as in Fact 1. So the likelihood ratio is

$$\frac{Q(y)}{P(y)} = 1 - q + q\, e^{(y-1/2)/\sigma^2} = 1 + qW(y), \qquad W(y) = e^{(y-1/2)/\sigma^2} - 1. \tag{30.18}$$

   Writing it as $1 + qW$ isolates the small parameter q: W measures how much the output favours "the record was in the batch", and q scales it down.
2. **Moments of W under P.** The mean is 0: $\mathbb{E}_P[W] = \mathbb{E}_P[\mathcal{N}(1, \sigma^2)\text{-density}/P(y)] - 1 = 1 - 1 = 0$, because averaging a density ratio under the denominator integrates the numerator density, which is 1. For the second moment, use the moment generating function of $Y \sim \mathcal{N}(0, \sigma^2)$, $\mathbb{E}[e^{tY}] = e^{t^2\sigma^2/2}$, with $t = 2/\sigma^2$: $\mathbb{E}_P[e^{2(Y-1/2)/\sigma^2}] = e^{2/\sigma^2} e^{-1/\sigma^2} = e^{1/\sigma^2}$. Expanding the square, $\mathbb{E}_P[W^2] = \mathbb{E}_P[(W+1)^2] - 2\,\mathbb{E}_P[W+1] + 1 = e^{1/\sigma^2} - 2 + 1 = e^{1/\sigma^2} - 1$. This is the χ^2 divergence between the two Gaussians, the mean squared deviation of their density ratio from 1.
3. **Expand the loss.** The loss is $L = \ln \frac{P(y)}{Q(y)} = -\ln(1 + qW)$. For small q, the Taylor series $\ln(1+x) = x - \frac{x^2}{2} + O(x^3)$ gives $L = -qW + \frac{q^2W^2}{2} + O(q^3W^3)$. Taking moments under P with step 2: the first term has mean 0, so $\mathbb{E}_P[L] \approx \frac{q^2}{2}\mathbb{E}_P[W^2] = \frac{q^2}{2}(e^{1/\sigma^2} - 1)$, and to leading order the variance is that of $-qW$, $\mathrm{Var}_P[L] \approx q^2(e^{1/\sigma^2} - 1)$.
4. **Sum over k steps.** Means and variances of independent losses add, so $K \approx \frac{kq^2}{2}(e^{1/\sigma^2} - 1)$ and $V \approx kq^2(e^{1/\sigma^2} - 1)$. Then $\mu = 2K/\sqrt{V} = kq^2(e^{1/\sigma^2} - 1)/(q\sqrt{k(e^{1/\sigma^2} - 1)}) = q\sqrt{k(e^{1/\sigma^2} - 1)}$, Equation 30.17.
5. **The regime.** With $q\sqrt{k} \to c$, each step's moments are $O(q^2) = O(1/k)$, so K and V converge to finite limits, $\max_i \mathrm{kl} \to 0$, and $\sum \kappa_3 = O(kq^3) \to 0$. These are the conditions of the limit form of the CLT. The other order of P and Q gives the same leading terms.

30.8.4. Interpretation

Subsampling buys a factor of about q. Full-batch training ($q = 1$) for k steps is $\sqrt{k}/\sigma$-GDP. Dividing Equation 30.17 by $\sqrt{k}/\sigma$, subsampling multiplies this by about $q\sqrt{e^{1/\sigma^2} - 1} \cdot \sigma$, which for $\sigma = 1$ is $q \times 1.3108$.

It is an approximation, not a bound. Two approximations stack: the CLT itself, and the expansion in step 3 (the exact per-step moments are smaller than the leading terms). Neither is controlled in the tail, which is where $\delta = 10^{-5}$ lives. Gopi, Lee & Wutschitz (2021) showed that the CLT value can lie below the true ε of DP-SGD. The example below confirms it for the running example; numerical accounting computes certified values and choosing an accountant compares all the accountants on this example.

Watch the neighbouring relation. Corollary 5.4 of Dong et al. analyses fixed-size batches under replace-one neighbours, where D' swaps one record of D for another, and gets a different constant ($\mu = 1.7101$ at $k = 10{,}000$ in the running example). Do not mix the two (subsampling explains why).

30.8.5. Example: the running DP-SGD example

The running example has sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and target $\delta = 10^{-5}$. Then $e^{1/\sigma^2} - 1 = 1.7183$ and $\sqrt{1.7183} = 1.3108$, so Equation 30.17 gives $\mu = 0.01 \times 1.3108\sqrt{k}$; at $k = 100$, for example, $\mu = 0.01 \times 1.3108 \times 10 = 0.1311$. Table 30.8 turns each μ into ε with Equation 30.10 and compares it with a numerical accountant.

Table 30.8.: The running DP-SGD example: the CLT's μ and ε, and a numerical bracket for the true ε.

Steps k	$q\sqrt{k}$	CLT μ	CLT ε at 10^{-5}	Numerical accountant ε	Full batch, $\mu = \sqrt{k}$
100	0.10	0.1311	0.4575	[0.7175, 0.7185]	10.0
1,000	0.32	0.4145	1.6177	[1.8232, 1.8332]	31.6
10,000	1.00	1.3108	6.0071	[6.1377, 6.2377]	100.0

The bracket in Table 30.8 comes from the same lattice-and-convolve method as the Laplace example, applied to the per-step loss on a lattice of spacing $h = 10^{-5}$, taking the worse of the two orders of neighbours. Rounding each step's loss down or up by at most h moves the composed loss by at most kh, so the bracket's width is kh by construction (0.0010, 0.0100 and 0.1000), and the informative number is its lower end. That lower end is exact up to the quadrature error in the per-step weights. In all three rows the CLT value lies below the lower end of the bracket. The ratio of the CLT value to that lower end is 0.638 at $k = 100$, 0.887 at $k = 1{,}000$ and 0.979 at $k = 10{,}000$, where $q\sqrt{k}$ is largest and the CLT regime fits best.

Computing the exact per-step moments by quadrature shows the second approximation at work. In the order of the derivation ($P = \mathcal{N}(0, 1)$, the record absent), the mean of the loss is 8.1890×10^{-5}

against the leading term $q^2(e-1)/2 = 8.5914 \times 10^{-5}$, and the variance 1.6009×10^{-4} against $q^2(e-1) = 1.7183 \times 10^{-4}$. Plugging the exact moments into Equation 30.14 gives $\mu = 1.2944$ at $k = 10{,}000$ and $\varepsilon = 5.9184$. The other order (the record present in D) gives $\mu = 1.2795$ and $\varepsilon = 5.8379$. Both are below the CLT's 6.0071, and further below the lower end 6.1377. So a smaller "more exact" μ is not more conservative; only a certified accountant is.

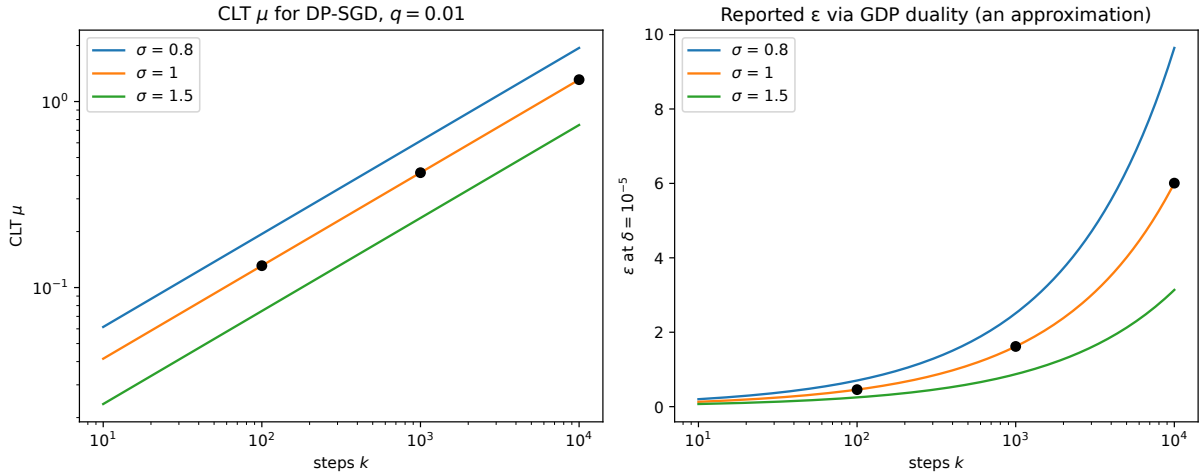


Figure 30.5.: CLT approximation for DP-SGD with $q = 0.01$. Left: $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$ against the number of steps for three noise multipliers. Right: the ε at $\delta = 10^{-5}$ it implies through Equation 30.10. Dots mark the running example at $k = 100$, 1,000 and 10,000.

Figure 30.5 shows how quickly ε grows with k once μ passes 1, and how strongly it depends on σ: $e^{1/\sigma^2} - 1$ is 3.7707 at $\sigma = 0.8$, 1.7183 at $\sigma = 1$ and 0.5596 at $\sigma = 1.5$.

30.8.6. Check yourself

- At fixed q and σ, how does the CLT μ scale with the number of epochs $E = kq$? *(As $\sqrt{k}$, so as $\sqrt{E/q}$: ten times more epochs multiplies μ by $\sqrt{10}$.)*
- Why can you not report "$\varepsilon = 6.0071$" for the running example at $k = 10{,}000$ as a DP guarantee? *(It is an asymptotic approximation with no error bound, and a numerical accountant shows the true ε is at least 6.1377, up to quadrature error.)*

30.9. RDP as a lossy summary of the curve

30.9.1. Context

Rényi DP tracks $\varepsilon(\alpha) = D_\alpha(P\|Q)$ over orders $\alpha > 1$, where the Rényi divergence $D_\alpha(P\|Q) = \frac{1}{\alpha-1} \ln \mathbb{E}_P\left[(p/q)^{\alpha-1}\right]$ is a log-moment of the likelihood ratio. Conversions (Section 28.5) turned an RDP curve into the region of test errors it allows, with lower boundary f_{RDP}, and showed that each point of that boundary is attained by a two-outcome pair. This section reads that

construction in the language of f-DP. What kind of guarantee is f_{RDP}, and where does it sit between the true curve and a single (ε, δ) pair?

30.9.2. Statement

The construction rests on one observation: a test turns any pair into a pair of coins. Its output is "fire" with probability a under P and $1-b$ under Q, so it maps P to $\mathrm{Bern}(a)$ and Q to $\mathrm{Bern}(1-b)$, and processing cannot increase a Rényi divergence. So the test's errors must keep these two coins within the RDP budget at every order. The lowest type II error that passes this check at each a is f_{RDP}.

Let $\varepsilon(\alpha)$ be an RDP curve, known at the orders of the part's grid (the fixed list of orders that the accountants of this part evaluate) and in both orders of neighbours, and let

$$f_{\mathrm{RDP}}(a) = \min\left\{b : D_\alpha(\mathrm{Bern}(a) \,\|\, \mathrm{Bern}(1-b)) \le \varepsilon(\alpha) \text{ and } D_\alpha(\mathrm{Bern}(1-b) \,\|\, \mathrm{Bern}(a)) \le \varepsilon(\alpha) \text{ for every order } \alpha\right. \tag{30.19}$$

be the lowest type II error it allows (Equation 28.11). Then:

1. The trade-off curve of a pair determines its RDP curve.
2. f_{RDP} is a symmetric trade-off function, and a mechanism with this RDP curve is f_{RDP}-DP. At each a, no higher value follows from the RDP values alone.
3. The best $\delta(\varepsilon)$ that the RDP values certify is the privacy profile of f_{RDP}: $\delta_{\mathrm{RDP}}(\varepsilon) = 1 + f^*_{\mathrm{RDP}}(-e^\varepsilon)$, which is the optimal conversion Equation 28.12 of Conversions.

For the Gaussian pair, $\varepsilon(\alpha) = \alpha\mu^2/2$ (Part 1). At $\mu = 1$, f_{RDP} lies below G_1 at every a the script computed.

30.9.3. Derivation

1. **The curve determines the Rényi divergence.** By Fact 1 the curve fixes the law of L under P. Since $p/q = e^L$, the Rényi divergence is $D_\alpha(P\|Q) = \frac{1}{\alpha-1}\ln \mathbb{E}_P[e^{(\alpha-1)L}]$, a functional of that law (Divergences).
2. **RDP implies f_{RDP}-DP.** A test with errors (a, b) maps P to $\mathrm{Bern}(a)$ and Q to $\mathrm{Bern}(1-b)$. By data processing for Rényi divergence (post-processing cannot increase it), the two Bernoullis satisfy the constraints whenever P and Q do. So every point of the true curve is feasible, and $T(P, Q) \ge f_{\mathrm{RDP}}$.
3. **f_{RDP} is a trade-off function.** Each constraint is convex in (a, b), because each term of the Bernoulli divergence has the form $x^\alpha y^{1-\alpha}$, which is jointly convex for $\alpha > 1$ ("Computing it" in Conversions). So the feasible set is convex and its lower boundary f_{RDP} is convex. The point $(a, 1-a)$ is feasible, because two equal Bernoullis have divergence 0, so $f_{\mathrm{RDP}}(a) \le 1-a$ and $f_{\mathrm{RDP}}(1) = 0$. A convex function on $[0, 1]$ whose minimum is at $a = 1$ is non-increasing: for $a < a' < 1$, write $a' = \lambda a + (1-\lambda)\cdot 1$, and then $f(a') \le \lambda f(a) + (1-\lambda)f(1) \le f(a)$. Swapping a and b swaps the two constraints (the coins $\mathrm{Bern}(a)$ and $\mathrm{Bern}(1-b)$ trade places after relabelling their outcomes), so the feasible set is symmetric about the diagonal and $f_{\mathrm{RDP}} = f^{-1}_{\mathrm{RDP}}$.

4. **No higher value at any a.** For each feasible (a, b) the pair $(\mathrm{Bern}(a), \mathrm{Bern}(1-b))$ is itself a mechanism that satisfies the constraints at every order of the grid, and its test "fire on outcome 1" has errors (a, b), so its curve passes through (a, b) (step 5 of the derivation in Conversions). So at each a some pair consistent with the RDP values reaches $f_{\mathrm{RDP}}(a)$. Different values of a need different pairs.
5. **The optimal conversion is the conjugate.** Equation 28.12 writes the best certified δ as $\max_a [1 - a - e^{\varepsilon} f_{\mathrm{RDP}}(a)]$, with the roles of a and b the other way round from the duality of Section 30.5. Because f_{RDP} is symmetric, its graph contains (b, a) whenever it contains (a, b), so swapping the roles changes nothing: the maximum equals $\max_a [1 - f_{\mathrm{RDP}}(a) - e^{\varepsilon} a] = 1 + f^*_{\mathrm{RDP}}(-e^{\varepsilon})$, the duality of Equation 30.9 applied to f_{RDP}.
6. **Below G_1.** The script computes f_{RDP} for $\mu = 1$ by bisection on b over the grid of orders, and finds it below G_1 everywhere it looked. This is a numerical observation, not a proof. More orders impose more constraints and can only raise f_{RDP}, but not by much: at $a = \Phi(-1/2) = 0.3085$, a dense set of orders in $[1.01, 3000]$ raises it from 0.2201 to 0.2219, still far below $G_1(0.3085) = 0.3085$.

30.9.4. Interpretation

The results form a ladder of summaries. For the Gaussian mechanism, G_μ is the exact curve, f_{RDP} is what its RDP curve keeps, and an (ε, δ) pair is one touching line of f_{RDP}. So the slack of an RDP conversion has two parts: the gap between the line and f_{RDP}, which the optimal conversion removes, and the gap between f_{RDP} and the true curve, which no conversion removes (Section 28.7).

RDP loses because it keeps only the exponential moments $\mathbb{E}_P[e^{(\alpha-1)L}]$ at a few orders. Different laws of the loss share them, and f_{RDP} has to allow the worst of those laws. The absolute gap in b is largest in the bulk. Relative to the attacker's power $1 - b$, though, the RDP curve allows roughly the same factor more power than the Gaussian has from the bulk down to $a = 10^{-6}$ (Table 28.5), so the loss is not confined to the bulk.

30.9.5. Example: what 1-GDP's RDP curve forgets

Table 30.9 compares G_1 with f_{RDP} computed from $\varepsilon(\alpha) = \alpha/2$. The third row is the best advantage, which for a symmetric curve sits at the diagonal point $a = b$, where it equals $1 - a - b = 1 - 2a$; for G_1 that is $1 - 2 \times 0.3085 = 0.3829$. The last row is the duality of Section 30.5 applied to each curve.

Table 30.9.: The trade-off curve of 1-GDP against the best curve its RDP curve implies over the part's grid of orders, read at the diagonal and through the duality.

Quantity	Exact curve G_1	f_{RDP} from $\varepsilon(\alpha) = \alpha/2$
b at $a = \Phi(-1/2) = 0.3085$	0.3085	0.2201
Diagonal point, $a = b$	0.3085	0.2585
Best advantage $\delta(0) = 1 - 2a$	0.3829	0.4830
at the diagonal point		
$\delta(1)$	0.1269	0.2149

In the bulk the RDP summary gives a lot away (Table 30.9). It allows an attacker an advantage of 0.4830 where the best possible is 0.3829, and the best $\delta(1)$ it can certify is 0.2149, 1.69 times the exact 0.1269. Figure 30.6 draws both curves with their touching lines at $\varepsilon = 1$.

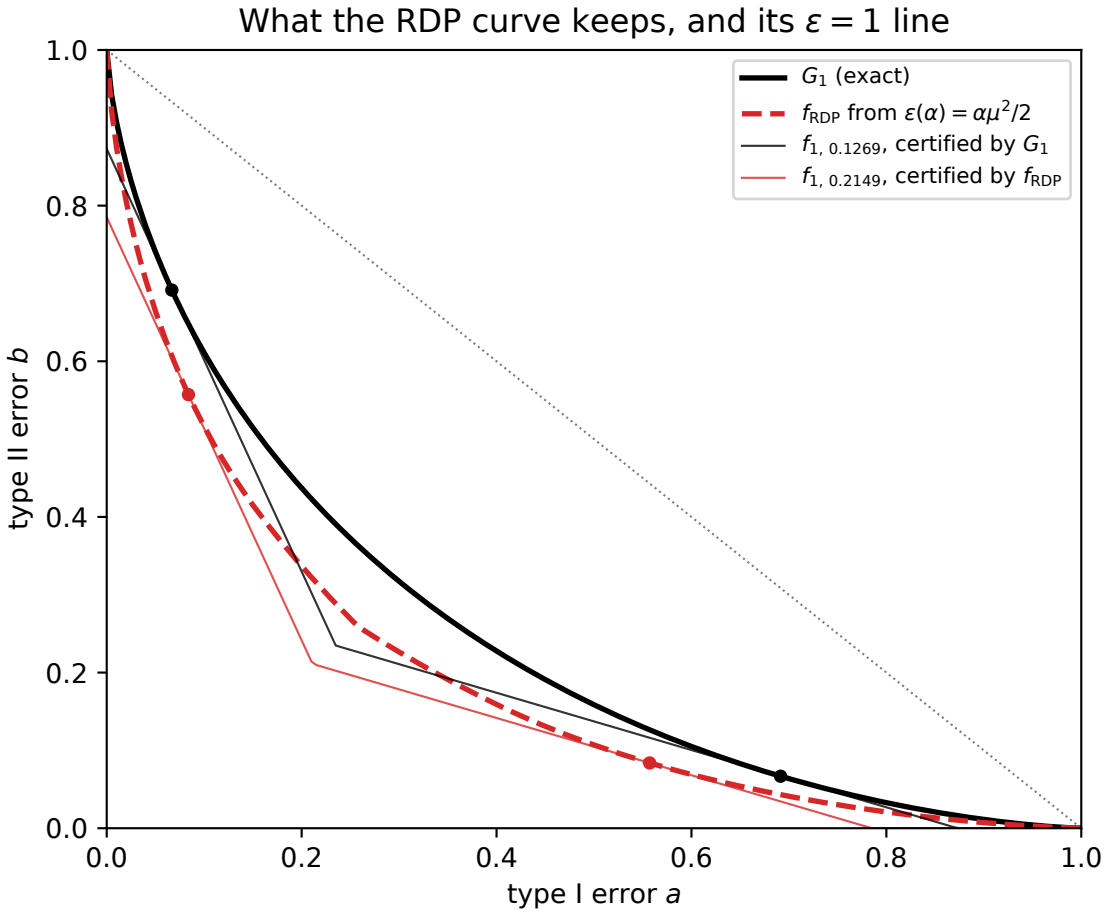


Figure 30.6.: G_1 (black) and the lowest curve f_{RDP} compatible with its RDP curve $\varepsilon(\alpha) = \alpha/2$ over the part's grid of orders (red, dashed). The thin lines are the (ε, δ) curves at $\varepsilon = 1$ that each one certifies, $f_{1,0.1269}$ for G_1 and $f_{1,0.2149}$ for f_{RDP}; each touches its own curve at two mirror-image points (dots). An RDP conversion can only report lines under f_{RDP}.

30.9.6. Check yourself

- If a mechanism's RDP curve is no larger than the Gaussian's at every order of the grid, must its trade-off curve lie on or above G_μ? *(No. By step 4, for each a the two-outcome pair through $(a, f_{\mathrm{RDP}}(a))$ satisfies the Gaussian's RDP curve at every order of the grid, yet its curve passes below G_μ at a.)*
- Why is GDP accounting for full-batch noisy gradient descent strictly better than RDP accounting for it? *(Both compose exactly, but GDP keeps the whole curve $G_{\sqrt{k}/\sigma}$, while RDP keeps only $\varepsilon(\alpha) = \alpha k/(2\sigma^2)$, from which only f_{RDP}, which lies below $G_{\sqrt{k}/\sigma}$, can be*

recovered.)

30.10. Code: check every result by computation and simulation

The script code/gaussian_dp.py computes every number quoted in this chapter and draws every figure. The public functions `gdp_delta`, `gdp_eps`, `gaussian_tradeoff`, `eps_delta_tradeoff`, `compose_gdp` and `clt_mu_subsampled_gaussian` are the ones later chapters import. It uses the standard library and NumPy only: Φ comes from `math.erfc` and Φ^{-1} from bisection. From the repository root:

```
uv run python code/part04-privacy-accounting/gaussian_dp.py
```

Output (seed 0):

Table 30.10.: Results printed by the script with seed 0.

Experiment	Result
$f_{1,0}$, Laplace and G_1 at $a = 0.25$	0.3204, 0.3679, 0.3724
Simulated threshold test on G_1 at $a = 0.05$	$\hat{a} = 0.0502$, $\hat{b} = 0.7408$; exact 0.7405
$\delta(1)$ of 1-GDP: formula, hockey-stick integral, conjugate on a grid	0.126937 each
ε at $\delta = 10^{-5}$ for $\mu = 0.5, 1, 2$	1.9931, 4.3772, 9.9973
Envelope of 1001 (ε, δ) lines against G_1	max gap 4.9×10^{-6}
Simulated 2-D test, $\mu = (1, 1)$, $a = 0.05$	$\hat{b} = 0.5913$; $G_{\sqrt{2}}(0.05) = 0.5912$
Four 1-GDP releases at $\delta = 10^{-5}$	exact 9.9973; composing points 18.7607
Randomized response, $k = 100$, $\varepsilon_0 = 0.1$: $\delta(1)$	exact 0.1257; CLT 0.1271
DP-SGD, $k = 10{,}000$	CLT $\mu = 1.3108$, $\varepsilon = 6.0071$; numerical $[6.1377, 6.2377]$
1-GDP vs its RDP curve: best advantage; $\delta(1)$	0.3829 vs 0.4830; 0.1269 vs 0.2149

tests/test_gaussian_dp.py checks that the output still matches Table 30.10 and every other number in the text.

What to take from it. The GDP profile is exact: the closed form, the hockey-stick integral of the PRV and the conjugate of the curve agree to six digits, three routes to one number. Composing curves rather than points matters: turning each step into one (ε, δ) point before composing nearly doubled ε. The CLT is good in the bulk and unreliable in the tail: it erred in both directions on randomized response and under-reported ε for the running DP-SGD example at every k. And RDP forgets part of the curve, even for the Gaussian mechanism, where it is as good as it gets.

30.11. References

- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022; arXiv:1905.02383): the source of f-DP and GDP. Definition 2.3 (f-DP), Proposition 2.4 (symmetrization), Proposition 2.5 ($f_{\varepsilon,\delta}$), Propositions 2.11–2.12 and Corollary 2.13 (duality), Theorem 2.14 (group privacy), Theorem 3.2 and Corollary 3.3 (composition), Theorems 3.4–3.6 (CLTs), Theorem 5.2 and Corollary 5.4 (CLT for their fixed-size subsampling under replace-one, NoisySGD). The discussion sections are a good read on why the curve is the right object.
- Bu, Dong, Long & Su, "Deep Learning with Gaussian Differential Privacy" (Harvard Data Science Review, 2020): the Poisson-subsampling CLT of Equation 30.17 (Section 3, proved as Theorem 5 in the appendix) and its use for DP-SGD, with experiments.
- Wasserman & Zhou, "A Statistical Framework for Differential Privacy" (Journal of the American Statistical Association, 2010): the hypothesis-testing reading of (ε, δ)-DP behind Equation 30.7.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015): optimal composition of (ε, δ)-DP through testing regions and Blackwell's theorem; the tensor product of $f_{\varepsilon,\delta}$ curves in other words.
- Balle & Wang, "Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising" (ICML 2018): Theorem 8 is the exact (ε, δ) profile of the Gaussian mechanism, Equation 30.10, and its use to calibrate σ.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021): the PRV accountant, with error bounds, and examples where the GDP CLT under-reports ε.
- Murtagh & Vadhan, "The Complexity of Computing the Optimal Composition of Differential Privacy" (TCC 2016): exact optimal composition of (ε, δ) guarantees is #P-complete when the $(\varepsilon_i, \delta_i)$ differ, which is why accountants discretize.

30.12. Cheat sheet

Table 30.11 collects every result in the chapter. Type I error is a, type II error b; α is a Rényi order.

Table 30.11.: Every result in this chapter.

Result	Statement	Where it comes from
f-DP	$T(M(D), M(D')) \geq f$ for all neighbours; then also $\max(f, f^{-1})$-DP	Neyman–Pearson; swap D, D'
(ε, δ)-DP as a curve	$f_{\varepsilon,\delta}(a) = \max(0, 1-\delta-e^{\varepsilon}a, e^{-\varepsilon}(1-\delta-a))$	The two DP inequalities on a rejection region
μ-GDP	$G_{\mu}(a) = \Phi(\Phi^{-1}(1-a)-\mu)$; Gaussian mechanism: $\mu = 1/\sigma$	Projection on the shift (Fact 3)

Result	Statement	Where it comes from
Duality	f-DP $\iff$ $(\varepsilon, 1 + f^*(-e^\varepsilon))$-DP for all $\varepsilon \geq 0$	Supporting lines of a convex curve
GDP profile	$\delta(\varepsilon) = \Phi(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}) - e^\varepsilon \Phi(-\frac{\varepsilon}{\mu} - \frac{\mu}{2})$	Conjugate, or hockey stick of $L \sim \mathcal{N}(\frac{\mu^2}{2}, \mu^2)$
Composition	$\bigotimes f_i$, tight; $\bigotimes G_{\mu_i} = G_{\sqrt{\sum \mu_i^2}}$	Blackwell / Neyman–Pearson; rotation invariance
CLT	$\bigotimes f_i \approx G_\mu$, $\mu = 2\|\mathrm{kl}\|_1 / \sqrt{\|\kappa_2\|_1 - \|\mathrm{kl}\|_2^2}$, error band γ	Berry–Esseen for the summed PRV
Subsampled Gaussian	$\mu \approx q\sqrt{k(e^{1/\sigma^2} - 1)}$ as $q\sqrt{k} \to c$	Second-order expansion of $\ln(1 + qW)$
RDP from the curve	$\varepsilon(\alpha)$ is a functional of T; RDP implies f_{RDP}-DP, a symmetric trade-off function below T; the optimal conversion is its conjugate	Data processing on $(\mathrm{Bern}(a), \mathrm{Bern}(1-b))$; duality

Rules of thumb

- The trade-off curve is the guarantee; (ε, δ), RDP and ρ are summaries of it. Summarize as late as possible.
- For Gaussian noise without subsampling, account in GDP: μ^2 adds, and the (ε, δ) profile at the end is exact.
- $\mu = 1$ means $\varepsilon \approx 4.4$ at $\delta = 10^{-5}$; $\mu = 0.5$ means about 2.
- Use the CLT for intuition and quick estimates, never as the reported guarantee: it under-reported ε for the running DP-SGD example at every k. Report a certified numerical accountant instead.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

31. Amplification by subsampling

31.1. Why this matters

Differential privacy (DP) for deep learning would be useless without one trick. Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, runs ten thousand steps or more. Each step is a Gaussian mechanism: it clips every example's gradient, sums them and adds Gaussian noise whose standard deviation is the **noise multiplier** σ times the clipping norm. Composing ten thousand such steps with $\sigma = 1$, and paying for every record at every step, gives $\varepsilon = 5425.5$ at $\delta = 10^{-5}$, as Rényi DP computes: a privacy loss nobody would report.

The trick is that each step only looks at a small random batch. If a record is in the batch only with probability q, an adversary who sees the step's output must first guess whether the record was used at all. That uncertainty is worth a lot. With $q = 0.01$, it turns a step that costs $\varepsilon = 1$ into one that costs about 0.017. In the language of Rényi differential privacy (RDP), which describes a mechanism by a curve $\varepsilon(\alpha)$ giving its privacy cost at each order $\alpha > 1$, it turns the curve of a Gaussian step from $\alpha/2$ into something of order $10^{-4}\alpha$ for small orders.

This chapter answers three questions. How much privacy does random sampling buy, exactly? Which sampling scheme does the answer hold for? And how do you compute it for the Gaussian steps of DP-SGD?

Notation. In this chapter q is always the **sampling rate**, never a density. Output distributions are capital letters, as in the rest of this part, and the same letter names a distribution and its density. P_0 is a step's output without the record and P_1 its output with it. For the subsampled Gaussian, $P_0 = \mathcal{N}(0, \sigma^2)$, $P_1 = \mathcal{N}(1, \sigma^2)$, and the output with the record sampled at rate q is the mixture $P_q = (1-q)P_0 + qP_1$: with probability $1-q$ the record is left out and the output is drawn from P_0, with probability q it is used and the output is drawn from P_1. The mixture lemma of the toolkit uses three generic distributions R, P, P' and a mixing weight η; the letter γ is taken, because it is the parameter of the hockey-stick divergence E_γ defined in Fact 1. An output is y.

31.1.1. The one idea: the output is a mixture, and mixtures are closer

Fix a dataset D and add one record x to get $D' = D \cup \{x\}$. Under **Poisson sampling**, each record enters the batch independently with probability q, by its own coin flip. Condition on which records of D were drawn. On D, the mechanism then outputs some distribution P_0. On D', it outputs P_1 if x was drawn and P_0 otherwise. Since x is drawn with probability q, the output on D' is the **mixture**, compared with the output on D:

$$(1-q)P_0 + qP_1 \quad \text{against} \quad P_0. \tag{31.1}$$

With probability $1-q$ the two runs are identical. The difference between the runs is diluted by a factor q, and every likelihood ratio, the ratio of the two densities at an output y that measures

how much more likely y is in one run than in the other, is pulled toward 1. Divide the mixture by P_0 to see it: the ratio P_1/P_0 becomes $(1-q)+q\,P_1/P_0$, a weighted average of 1 and the old ratio. All of DP-SGD's privacy rests on this dilution, and the dilution depends on how the batch was drawn. A different sampling scheme gives a different mixture, and an accountant that assumes one scheme while the training code runs another reports a number that does not apply.

Table 31.1 lists the results of the chapter and where each is used.

Table 31.1.: The results of this chapter and where each is used.

Result	Where it is used
The mixture lemma	Every amplification statement in this chapter
(ε, δ) amplification, Poisson sampling, add/remove	One-step guarantees; the naive DP-SGD accountant
The same for a fixed-size batch, replace-one	Accounting for fixed-size batches; why the scheme must match the neighbouring relation
RDP of the Poisson-subsampled Gaussian	The RDP accountant for DP-SGD, and the input of numerical accounting
Small-q asymptotics, the moments accountant	Quick estimates; reading Abadi et al. (2016)
The RDP accountant for DP-SGD, against per-step composition	The running example; Choosing an accountant
No tight generic RDP amplification	Why accountants special-case the Gaussian

How to read this chapter. Read the toolkit and Section 31.3, then Section 31.4 on sampling schemes. It holds the warning that matters most in practice. Section 31.5 is the computation every DP-SGD accountant performs; do its $\alpha = 2$ walk-through by hand. Section 31.6 and Section 31.8 can wait for a second reading.

The chapter uses the Rényi divergence D_α and the hockey-stick divergence E_γ from Divergences, RDP and its composition from Rényi DP, and the conversions to (ε, δ) from From RDP to (ε, δ). Each is restated where it is used.

31.2. Toolkit: three facts every derivation uses

31.2.1. Fact 1 — (ε, δ) is a statement about events, and averaging preserves it

A mechanism is (ε, δ)-DP when, for every pair of neighbouring datasets taken in both orders, the output distributions P and Q satisfy $P(A) \le e^\varepsilon Q(A) + \delta$ for every event A (Definition): no set of outputs becomes more than e^ε times more likely, except for a slack δ. The **hockey-stick divergence** packs this into one number. It is the largest amount by which P exceeds γ times Q on any event, $E_\gamma(P\|Q) = \max_A [P(A) - \gamma Q(A)]$, so the DP condition for the pair is exactly $E_{e^\varepsilon}(P\|Q) \le \delta$.

Because the condition is a bound on each event separately, it survives averaging. Suppose $P = \sum_t w_t P_t$ and $Q = \sum_t w_t Q_t$ are averages with the same weights w_t, for instance over some random choice t that both runs share. For every event the excess is the average of the excesses, $P(A) - \gamma Q(A) = \sum_t w_t [P_t(A) - \gamma Q_t(A)]$. If each bracket is at most δ, so is their weighted average. So $E_\gamma(P\|Q) \le \max_t E_\gamma(P_t\|Q_t)$. This is **joint convexity** of E_γ, restricted to a shared mixing variable.

The subsampled mechanism is exactly such an average, over which other records were drawn. Fact 1 lets you fix those records and prove the bound for one draw.

31.2.2. Fact 2 — The mixture lemma

This is the fact that does the amplifying. Picture two runs whose outputs are mixtures with the same weight η: with probability $1-\eta$ both draw from a common distribution R, where the runs agree, and with probability η the first draws from P and the second from P', where they may differ. Where they agree the likelihood ratio is 1; where they differ it is at most about e^ε. The mixture's ratio should then be at most the weighted average $(1-\eta)\cdot 1 + \eta e^\varepsilon = 1 + \eta(e^\varepsilon - 1)$, and the slack δ, which only enters through the differing part, should shrink to $\eta\delta$. The lemma makes this precise, with one subtlety: P must be close not only to P' but also to R, because a high-probability event of P could otherwise be one that R never produces.

Let R, P, P' be three distributions, $\varepsilon \ge 0$, $\delta \in [0,1]$ and $\eta \in [0,1]$. Suppose that for every event A,

$$P(A) \le e^\varepsilon P'(A) + \delta \qquad \text{and} \qquad P(A) \le e^\varepsilon R(A) + \delta. \tag{31.2}$$

Then for every event A,

$$(1-\eta)R(A) + \eta P(A) \le e^{\varepsilon'}\left[(1-\eta)R(A) + \eta P'(A)\right] + \eta\delta, \qquad e^{\varepsilon'} = 1 + \eta(e^\varepsilon - 1). \tag{31.3}$$

So the ε shrinks to ε' and the δ to $\eta\delta$. This is an (ε,δ) form of what Balle, Barthe & Gaboardi (2018, Theorem 2) call **advanced joint convexity**.

Proof. The plan is to treat the three probabilities of A as unknowns, see that the gap between the two sides gets larger only when $P'(A)$ and $R(A)$ get smaller, and push them down to the smallest values the assumptions allow. Fix A and write $a = P(A)$, $b = P'(A)$, $c = R(A)$, and F for the left side minus the right side without the $\eta\delta$. The claim is $F \le \eta\delta$.

1. *Expand.* The $(1-\eta)c$ on the left and the $e^{\varepsilon'}(1-\eta)c$ on the right combine to $(1-\eta)c(1-e^{\varepsilon'})$, and $1 - e^{\varepsilon'} = -\eta(e^\varepsilon - 1)$. So
$$F = \eta a - \eta(1-\eta)(e^\varepsilon - 1)\,c - \eta\big(1 + \eta(e^\varepsilon - 1)\big)\,b. \tag{31.4}$$
2. *Signs.* The coefficients of b and c are negative, so F is largest when b and c are as small as the assumptions allow. Solving each assumption for the smaller side gives $b \ge e^{-\varepsilon}(a-\delta)$ and $c \ge e^{-\varepsilon}(a-\delta)$, and both are also ≥ 0, being probabilities. Which of the two lower bounds binds depends on whether a exceeds δ.
3. *Case $a \le \delta$.* Drop the negative terms: $F \le \eta a \le \eta\delta$.
4. *Case $a > \delta$.* Put $b = c = e^{-\varepsilon}(a-\delta)$ into step 1. The two coefficients add up to $(1-\eta)(e^\varepsilon - 1) + 1 + \eta(e^\varepsilon - 1) = e^\varepsilon$, so $F \le \eta a - \eta e^{-\varepsilon}(a-\delta)e^\varepsilon = \eta\delta$.

The script checks the lemma on random examples. It draws 20,000 random triples (R, P, P') on four outcomes, computes the smallest δ for which the two assumptions hold in both directions at $\varepsilon = 1$, and checks Equation 31.3 with $\eta = 0.1$. For each triple it divides the left side's excess over $e^{\varepsilon'}$ times the bracket by $\eta\delta$. The largest value of this ratio is 1.0000: the lemma holds, and some triples meet it with equality to four digits, so it cannot be improved.

31.2.3. Fact 3 — Group privacy

A guarantee for datasets one record apart also covers datasets two records apart, at a price. If a mechanism is (ε, δ)-DP for datasets one step apart, apply the definition twice along the chain D_0, D_1, D_2: $P_0(A) \le e^{\varepsilon} P_1(A) + \delta \le e^{\varepsilon}(e^{\varepsilon} P_2(A) + \delta) + \delta = e^{2\varepsilon} P_2(A) + (1 + e^{\varepsilon})\delta$. So it is $(2\varepsilon, (1 + e^{\varepsilon})\delta)$-DP for datasets two steps apart (Properties). Replacing a record is a removal followed by an addition, so a guarantee for add/remove neighbours gives one for replace-one neighbours at this price.

31.3. Amplification for (ε, δ)-DP under Poisson sampling

31.3.1. Context

You have a mechanism M that is (ε, δ)-DP for add/remove neighbours, datasets that differ by one record being present in one and absent from the other. You run it on a Poisson sample of the data, each record kept with probability q. What are the privacy parameters of the combined mechanism? This is the oldest amplification result. Kasiviswanathan et al. (2008) used it to learn privately from a sample, and the tight form below is due to Li, Qardaji & Su (2012) and Balle, Barthe & Gaboardi (2018).

31.3.2. Statement

The answer is the mixture lemma with the sampling rate as the weight: the likelihood ratio becomes a weighted average of 1 and e^{ε}, and δ is paid only when the record is drawn. Let M be (ε, δ)-DP for add/remove neighbours and let $M \circ S_q$ run M on a Poisson sample with rate q. Then $M \circ S_q$ is (ε', δ')-DP for add/remove neighbours with

$$\varepsilon' = \ln\left(1 + q(e^{\varepsilon} - 1)\right), \qquad \delta' = q\delta. \tag{31.5}$$

The bound is tight: for every ε and q some mechanism attains it (Balle, Barthe & Gaboardi 2018, Theorems 8 and 13).

31.3.3. Derivation

The idea is to fix every random choice except whether x is drawn, apply the mixture lemma once for each order of the pair, and then average the fixed choices back in with Fact 1. Let $D' = D \cup \{x\}$.

1. **Condition on the rest of the sample.** Let T be the Poisson sample of D. Because each record has its own coin, the sample of D' is T or $T \cup \{x\}$, the second with probability q, independently of T. Given T, write $P_0 = M(T)$ and $P_1 = M(T \cup \{x\})$. Then $M \circ S_q(D)$ given T is P_0, and $M \circ S_q(D')$ given T is $(1-q)P_0 + qP_1$.
2. **Direction "D' against D".** Here the mixture is on the left. Apply the mixture lemma (Fact 2) with $R = P_0$, $P = P_1$, $P' = P_0$ and $\eta = q$. Both of its assumptions then read $P_1(A) \le e^{\varepsilon} P_0(A) + \delta$, which holds because T and $T \cup \{x\}$ are neighbours and M is DP. The right side of Equation 31.3 becomes $e^{\varepsilon'}[(1-q)P_0(A) + qP_0(A)] + q\delta = e^{\varepsilon'} P_0(A) + q\delta$. This is the **remove** direction in the naming of Numerical accounting: the dataset with the record comes first, and its neighbour is obtained by removing x.
3. **Direction "D against D'".** Now the mixture is on the right. Apply the lemma with $R = P_0$, $P = P_0$, $P' = P_1$. The assumptions are $P_0(A) \le e^{\varepsilon} P_1(A) + \delta$, which is DP of M again, and $P_0(A) \le e^{\varepsilon} P_0(A) + \delta$, which is always true. The left side is $(1-q)P_0(A) + qP_0(A) = P_0(A)$ and the right side $e^{\varepsilon'}[(1-q)P_0(A) + qP_1(A)] + q\delta$. This is the **add** direction.
4. **Average over T.** Both bounds hold for every T with the same $(\varepsilon', q\delta)$, and both runs average over T with the same weights, so they hold for the averages over T (Fact 1).

Tightness. To show ε' cannot be smaller, you need a mechanism whose likelihood ratio actually reaches $1 + q(e^{\varepsilon} - 1)$. Randomized response does it. It reports a bit truthfully with probability $e^{\varepsilon}/(1+e^{\varepsilon})$ and flipped otherwise (Randomized response), so the probability of any report changes by a factor of exactly e^{ε} when the true bit changes. Let M apply it to the bit "the sample contains a flagged record". Take D with no flagged record and x flagged. Then the bit is 0 whenever x is not drawn and 1 whenever it is, so P_0 and P_1 are the same for every T, and $P_1(1)/P_0(1) = e^{\varepsilon}$. The likelihood ratio of the mixture to P_0 at the output "1" is

$$\frac{(1-q)P_0(1) + qP_1(1)}{P_0(1)} = (1-q) + q\frac{P_1(1)}{P_0(1)} = 1 + q(e^{\varepsilon} - 1). \tag{31.6}$$

With $\delta = 0$ the event $\{1\}$ forces $e^{\varepsilon'} \ge 1 + q(e^{\varepsilon} - 1)$, so ε' in Equation 31.5 cannot be improved.

31.3.4. Interpretation

The formula behaves differently for small and large ε. For small ε, amplification is linear in q. When $\varepsilon \ll 1$, $e^{\varepsilon} - 1 \approx \varepsilon$ and $\ln(1+y) \approx y$ for small y, so $\varepsilon' \approx q\varepsilon$. A 1% sample makes each step a hundred times cheaper.

For large ε, amplification only subtracts. When $qe^{\varepsilon} \gg 1$, the 1 inside the logarithm is negligible and $1 + q(e^{\varepsilon} - 1) \approx qe^{\varepsilon}$, so $\varepsilon' \approx \ln(qe^{\varepsilon}) = \varepsilon - \ln(1/q)$. Sampling can never remove more than $\ln(1/q)$ nats, because in the worst case the record was drawn, and that happens with probability q. At $q = 0.01$ and $\varepsilon = 10$, Table 31.3 gives 5.3993, close to $10 - \ln 100 = 5.395$. Figure 31.1 shows the switch between the two regimes around $\varepsilon \approx \ln(1/q)$, where qe^{ε} reaches 1.

The δ always shrinks by q, because the failure event of M only matters when the record was drawn. Finally, Equation 31.5 is a statement about one step, and it is tight for one step. Composing per-step (ε', δ') pairs is not tight, and the running example shows by how much (Section 31.7).

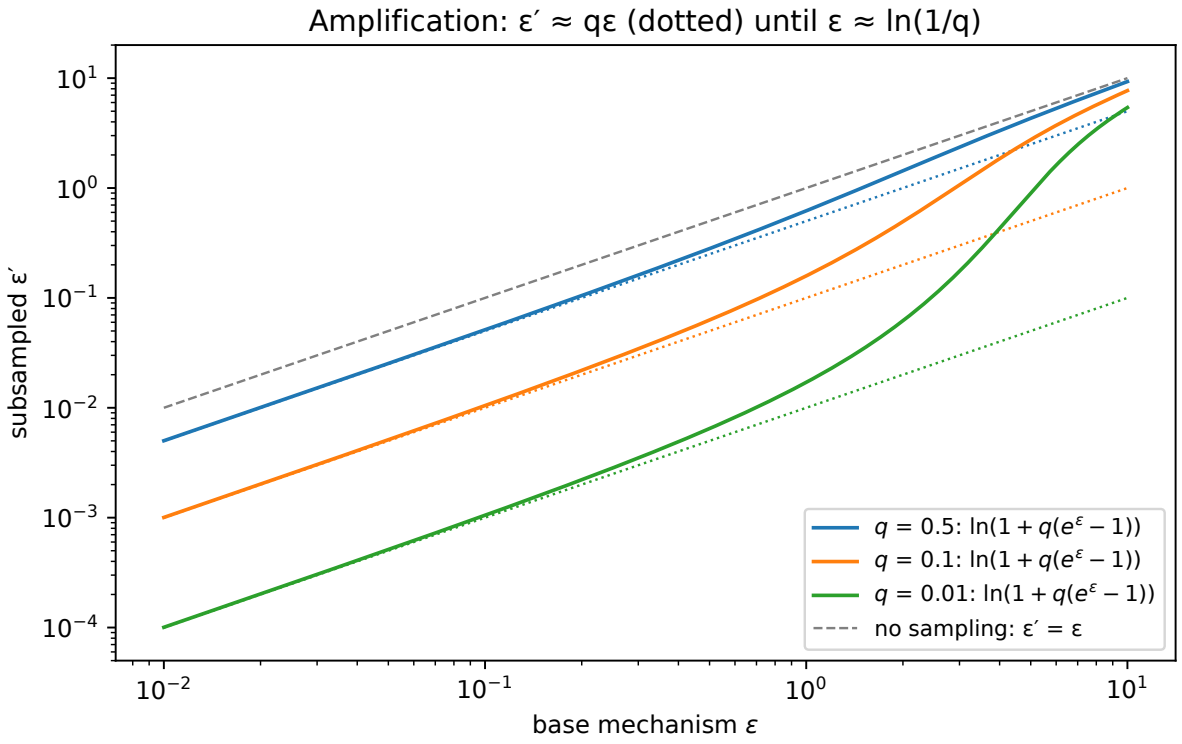


Figure 31.1.: The subsampled ε' against the base ε for three sampling rates, with the small-ε approximation $q\varepsilon$ dotted and no sampling dashed. Amplification is a factor q for small ε and only a shift by $\ln(1/q)$ for large ε.

31.3.5. Example: randomized response, sampled at 10%

Take $\varepsilon = 1$ and $q = 0.1$, and the randomized-response mechanism of the tightness argument. When the record is not drawn the true bit is 0, which is reported as 0 with probability $e/(1+e) = 0.7311$ and as 1 with probability $1/(1+e) = 0.2689$; when it is drawn the two probabilities swap. The mixture weights these by 0.9 and 0.1: for the output 1, $0.9 \times 0.2689 + 0.1 \times 0.7311 = 0.3152$. Table 31.2 shows both outputs and the likelihood ratios in both directions.

Table 31.2.: Randomized response with $\varepsilon = 1$ on a 10% Poisson sample. The likelihood ratios of the unsampled mechanism are $e^{\pm 1}$; sampling pulls them to at most 1.1718.

Output y	$P_0(y)$, record not drawn	$P_1(y)$, record drawn	Mixture $0.9P_0 + 0.1P_1$	Mixture / P_0	P_0 / mixture
0	0.7311	0.2689	0.6848	0.9368	1.0675
1	0.2689	0.7311	0.3152	1.1718	0.8534

Without sampling the ratios are $e = 2.718$ and $1/e$. With sampling the largest is $1.1718 = 1 + 0.1(e-1)$, so $\varepsilon' = \ln 1.1718 = 0.1586$, as Equation 31.5 says, against $q\varepsilon = 0.1$. The other direction's largest ratio, 1.0675, is smaller: removing the record from a mixture changes less than adding it, because the mixture never puts less than 0.9 times P_0's probability on any output. Table 31.3 gives ε' for more cases.

Table 31.3.: The amplified $\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$ for three sampling rates.

q	$\varepsilon = 0.1$	$\varepsilon = 1$	$\varepsilon = 5$	$\varepsilon = 10$
0.01	0.0011	0.0170	0.9059	5.3993
0.1	0.0105	0.1586	2.7563	7.6978
0.5	0.0512	0.6201	4.3136	9.3069

31.3.6. Check yourself

- At $q = 0.01$, roughly how large must ε be before amplification stops being a factor of 100? *(Around $\ln(1/q) = 4.6$, where $q(e^{\varepsilon} - 1)$ reaches 1. At $\varepsilon = 5$, Table 31.3 gives $\varepsilon' = 0.9059$, not 0.05.)*
- Why does the derivation need T and x to be drawn independently? *(So that, given T, the record is included with probability exactly q. The mixture weights must not depend on the rest of the sample.)*

31.4. The sampling scheme must match the neighbouring relation

31.4.1. Context

Poisson sampling gives batches of random size. Most training code draws fixed-size batches: either a random batch of m out of n records at each step, or a random permutation of the data

cut into n/m consecutive batches per epoch (**shuffle and partition**). Does the amplification of Equation 31.5 still apply? Only if you change what you compare. A fixed batch size pairs naturally with **replace-one** neighbours, datasets of the same size that differ in one record's value, because with a fixed batch size adding a record would change the size. Accounting for one scheme while running another is a bug that can multiply ε several times over.

31.4.2. Statement

Fixed-size batches, replace-one. A uniformly random batch of m out of n contains a given record with probability m/n, so m/n plays the role of q, and the same formula holds, as long as the base guarantee is for replace-one neighbours. Draw a uniformly random batch of m records from n, and let M be (ε, δ)-DP for replace-one neighbours on datasets of size m. Then the subsampled mechanism is (ε', δ')-DP for replace-one neighbours on datasets of size n, with

$$\varepsilon' = \ln\Big(1 + \frac{m}{n}(e^{\varepsilon} - 1)\Big), \qquad \delta' = \frac{m}{n}\delta \tag{31.7}$$

(Balle, Barthe & Gaboardi 2018, Theorem 9). The formula is that of Equation 31.5 with $q = m/n$, but the base guarantee is a different one.

The matching rule. Each combination of sampling scheme and neighbouring relation produces its own pair of output distributions, and needs its own base guarantee. Table 31.4 lists them. Its last column applies them to DP-SGD's noisy gradient sum, where every gradient is clipped to norm C and the noise has standard deviation σC; what matters is how far one record can move the sum, measured in units of the noise. The zero-out relation in the last row compares a dataset with the same dataset in which the target's gradient is set to 0.

Table 31.4.: What to account for under each combination of sampling scheme and neighbouring relation.

Sampling	Neighbours	Pair to account for	Base guarantee needed	Gaussian sum, clipping norm C
Poisson, rate q	add/remove	$(1-q)P_0 + qP_1$ against P_0	add/remove	means differ by C: noise multiplier σ
Fixed size m of n	replace-one	$(1-\eta)R + \eta P$ against $(1-\eta)R + \eta P'$, $\eta = m/n$	replace-one	means differ by $2C$: noise multiplier $\sigma/2$
Fixed size m of n	add/remove	dominated by the Poisson pair with noise multiplier $\sigma/2$ (Lebeda et al. 2024, Proposition 9)	replace-one	$\sigma/2$

Sampling	Neighbours	Pair to account for	Base guarantee needed	Gaussian sum, clipping norm C
Poisson, rate q	replace-one	both sides are mixtures, base pair two steps apart	$(2\varepsilon, (1+e^{\varepsilon})\delta)$ by group privacy	means differ by up to $2C$
Shuffle and partition	zero-out (the target's gradient set to 0)	the record is in exactly one batch per epoch	no simple formula; see below	measured, not derived

31.4.3. Derivation

Equation 31.7 by a coupling. A fixed-size batch is not drawn record by record, so there is no independent coin for the differing record. The trick, called a **coupling**, is to build the batch in a way that has such a coin, and then to use the same coin and the same other records for both datasets. Let D and D' differ in record i: x in D, x' in D'.

1. **Build a batch that avoids i in two stages.** Draw B, a uniform $(m-1)$-subset of the other $n-1$ records, then z, a uniform record from the $(n-1)-(m-1)=n-m$ others not in B. The batch $B\cup\{z\}$ is a uniform m-subset that avoids i: each such subset arises from m choices of which element is z, equally often.
2. **Write both outputs as mixtures over (B, z).** A uniform m-subset of n contains i with probability $\eta = m/n$. Given that it does, the rest of it is a uniform $(m-1)$-subset of the others, which is B; given that it does not, it is a uniform m-subset avoiding i, which is $B\cup\{z\}$. So flip an independent coin with probability η: on heads the batch is $B\cup\{x\}$ in D and $B\cup\{x'\}$ in D', on tails it is $B\cup\{z\}$ in both. Given (B, z), the outputs are $(1-\eta)R+\eta P$ and $(1-\eta)R+\eta P'$ with $R = M(B\cup\{z\})$, $P = M(B\cup\{x\})$, $P' = M(B\cup\{x'\})$.
3. **Check the lemma's assumptions.** The three batches $B\cup\{x\}$, $B\cup\{x'\}$, $B\cup\{z\}$ have size m and differ pairwise in one element, so they are pairwise replace-one neighbours. Hence $P \le e^{\varepsilon}P' + \delta$ and $P \le e^{\varepsilon}R + \delta$ on every event, and the same with P and P' swapped.
4. **Apply the mixture lemma and average.** Fact 2 with weight η gives the bound in one direction, and again with P and P' swapped in the other. Both runs average over (B, z) with the same weights, so Fact 1 carries the bound to the full mechanism.

The Gaussian column. For a sum of gradients clipped to norm C, adding or removing a record moves the sum by at most C, and replacing one moves it by up to $2C$: the old gradient leaves and a new one, possibly pointing the opposite way, comes in. A Gaussian mechanism's privacy depends only on the shift divided by the noise's standard deviation, and $2C/(\sigma C) = C/((\sigma/2)C)$, so noise σC with shift $2C$ is the same as shift C with noise multiplier $\sigma/2$. Lebeda, Regehr, Kamath & Steinke (2024, Proposition 9) show, using Zhu, Dong & Wang (2022, Theorem 11), that for fixed-size batches this shift of $2C$ is the worst case under the add relation and under the remove relation, each taken separately, because a record entering the batch pushes another one out.

Poisson with replace-one. Given the rest of the sample T, the target is drawn with probability q in both runs, so the outputs are $(1-q)R+qP$ and $(1-q)R+qP'$ with $R = M(T)$, $P = M(T\cup\{x\})$, $P' = M(T\cup\{x'\})$. The pair (P, R) is one add/remove step apart, but (P, P') is two: remove x,

then add x'. The mixture lemma needs one guarantee for both pairs, so use the weaker one. Fact 3 gives the base guarantee $(2\varepsilon, (1+e^{\varepsilon})\delta)$ for both, and Fact 2 then gives $\varepsilon' = \ln(1+q(e^{2\varepsilon}-1))$ and $\delta' = q(1+e^{\varepsilon})\delta$. For small q both logarithms are close to their arguments' excess over 1, and since $e^{2\varepsilon}-1 = (e^{\varepsilon}-1)(e^{\varepsilon}+1)$, this bound is about $e^{\varepsilon}+1$ times the add/remove value.

The bound is not tight: the pair (P, R) is only one step apart, but the lemma charges it the two-step guarantee. For pure DP ($\delta = 0$) the exact value follows from likelihood ratios. Pure DP of M says that every likelihood ratio between neighbours lies in $[e^{-\varepsilon}, e^{\varepsilon}]$, so both P/R and P'/R do. Divide the two mixtures by R: their ratio is $\frac{(1-q)+qP/R}{(1-q)+qP'/R}$. It is largest when the numerator is largest and the denominator smallest, that is with $P/R = e^{\varepsilon}$ and $P'/R = e^{-\varepsilon}$, which gives at most $\frac{1-q+qe^{\varepsilon}}{1-q+qe^{-\varepsilon}}$. Taking the logarithm, and writing $1-q+qe^{-\varepsilon} = 1-q(1-e^{-\varepsilon})$,

$$\varepsilon' = \ln\left(1+q(e^{\varepsilon}-1)\right) - \ln\left(1-q(1-e^{-\varepsilon})\right). \tag{31.8}$$

The Laplace mechanism on a sum attains it (Exercise 3 in the exercise book). For small ε each of the two logarithms is about $q\varepsilon$, so the value is about twice the add/remove value. At $\varepsilon = 1$ and $q = 0.01$ it is 0.02338, against 0.06193 for the group-privacy bound and 0.01704 for add/remove: a ratio of 1.37, not 3.64.

31.4.4. Interpretation

Why mixing them is a bug. Each row of Table 31.4 is correct on its own. The bug is to train with one row and report the number from another. Two cases are common.

The first is fixed-size sampled batches with a Poisson accountant. The accountant assumes a shift of C; the training run has a shift of $2C$. You are reporting the privacy of twice as much noise as you added. Lebeda et al. (2024, Table 1) give an example with $q = 0.001$, $\sigma = 0.8$ and 10,000 steps: at $\delta = 10^{-5}$ they report $\varepsilon = 0.80$ for Poisson sampling and $\varepsilon = 12.98$ for fixed-size batches, from a numerical accountant. (The tight privacy-loss-distribution accountants of the `dp_accounting` and `prv_accountant` libraries give 0.78 for Poisson sampling; that value comes from those libraries, not from this chapter's script.) The RDP accountant of this chapter reproduces the gap: 1.38 against 15.30.

The second is shuffle and partition with a Poisson accountant, and it is worse, because the mechanism of amplification is missing altogether. In one epoch the record is in exactly one batch, never absent, so no step's output is a mixture with "not used". The only uncertainty is *which* batch held the record, and the other records can give that away.

Chua et al. (2024) build a bad case. Every other record has gradient -1, the target has $+1$, and the neighbouring dataset zeroes the target out. Recentre each batch sum by adding the batch size, which removes the -1 contributions of a full batch of other records. Batches without the target then output $\mathcal{N}(0, \sigma^2)$. The target's batch has one -1 fewer and a $+1$ instead, so it outputs $\mathcal{N}(2, \sigma^2)$, or $\mathcal{N}(1, \sigma^2)$ when the target is zeroed. You don't know which batch holds it, so with b batches the epoch's output is a mixture, over the batch index t, of $\mathcal{N}(2e_t, \sigma^2 I)$ against the same mixture of $\mathcal{N}(e_t, \sigma^2 I)$, where e_t is the vector with 1 in position t and 0 elsewhere.

A natural test looks at the largest of the b batch outputs and says "target present" when it exceeds a threshold τ. The largest output stays below τ only if every batch output does, and the batches are independent. The target's batch stays below τ with probability $\Phi(\frac{\tau-2}{\sigma})$, where Φ is the standard normal distribution function, and each of the $b-1$ others with probability $\Phi(\frac{\tau}{\sigma})$.

This does not depend on which batch holds the target, so mixing over t changes nothing, and the test fires with probability $1 - \Phi(\frac{\tau-2}{\sigma})\Phi(\frac{\tau}{\sigma})^{b-1}$ under the first distribution and $1 - \Phi(\frac{\tau-1}{\sigma})\Phi(\frac{\tau}{\sigma})^{b-1}$ under the second. Recall that $\delta(\varepsilon)$, the smallest δ that works with a given ε, is the largest value of $P(A) - e^{\varepsilon}Q(A)$ over events A (Fact 1). Any one event, such as "the largest output exceeds τ", gives a value no larger, so it gives a lower bound on $\delta(\varepsilon)$. Maximizing over τ gives the best such lower bound, and the smallest ε it does not rule out at a given δ is a lower bound on the true ε.

This is the zero-out relation, not add/remove. Under zero-out, a Poisson step's clipped sum also moves by at most C, as under add/remove, so the Poisson accountant's number applies to the same relation, and the comparison below is like with like.

The script reproduces Chua et al.'s headline check: with $\sigma = 0.4$ and $b = 10{,}000$ batches, the test gives $\delta(4) \geq 0.226$ for shuffling, against $\delta(4) = 0.244$ for a fixed, non-random batch order. They report that a Poisson accountant gives $\delta(4) \leq 1.18 \times 10^{-5}$ for the same parameters.

31.4.5. Example: one epoch of the running example

Take $q = 0.01$, so one epoch is $b = 100$ batches, and $\delta = 10^{-5}$. Table 31.5 compares three batch samplers for one epoch. With deterministic batches the record is used once, in a known batch, so the epoch is a plain Gaussian mechanism and its ε is exact. For shuffle and partition the table gives the lower bound from the test above, and for Poisson sampling the upper bound from the RDP accountant of Section 31.7.

Table 31.5.: The ε at $\delta = 10^{-5}$ of one epoch of 100 batches under three batch samplers. Shuffling is a lower bound and Poisson an upper bound, so wherever the shuffled column exceeds the Poisson column, the Poisson number is provably wrong for shuffled training.

σ	Deterministic batches (exact)	Shuffled batches (lower bound)	Poisson sampling (upper bound)
0.5	10.00	10.00	8.03
1	4.38	4.06	1.21
1.5	2.75	1.46	0.47
2	1.99	0.48	0.26
2.5	1.55	0.20	0.17
3	1.27	0.10	0.14

At the running example's $\sigma = 1$, shuffled training has $\varepsilon \geq 4.06$ after one epoch, while the Poisson accountant reports at most 1.21. Shuffling buys almost nothing over a fixed order (4.38). The gap closes as σ grows (Figure 31.2). At $\sigma = 3$ the shuffled lower bound falls below the Poisson upper bound, which says only that this particular test is too weak there to show a gap. For fixed-size sampled batches over many epochs, the running example gives $\varepsilon = 47.42$ after 10,000 steps against 6.71 for Poisson (both by RDP; the first uses noise multiplier $\sigma/2 = 0.5$).

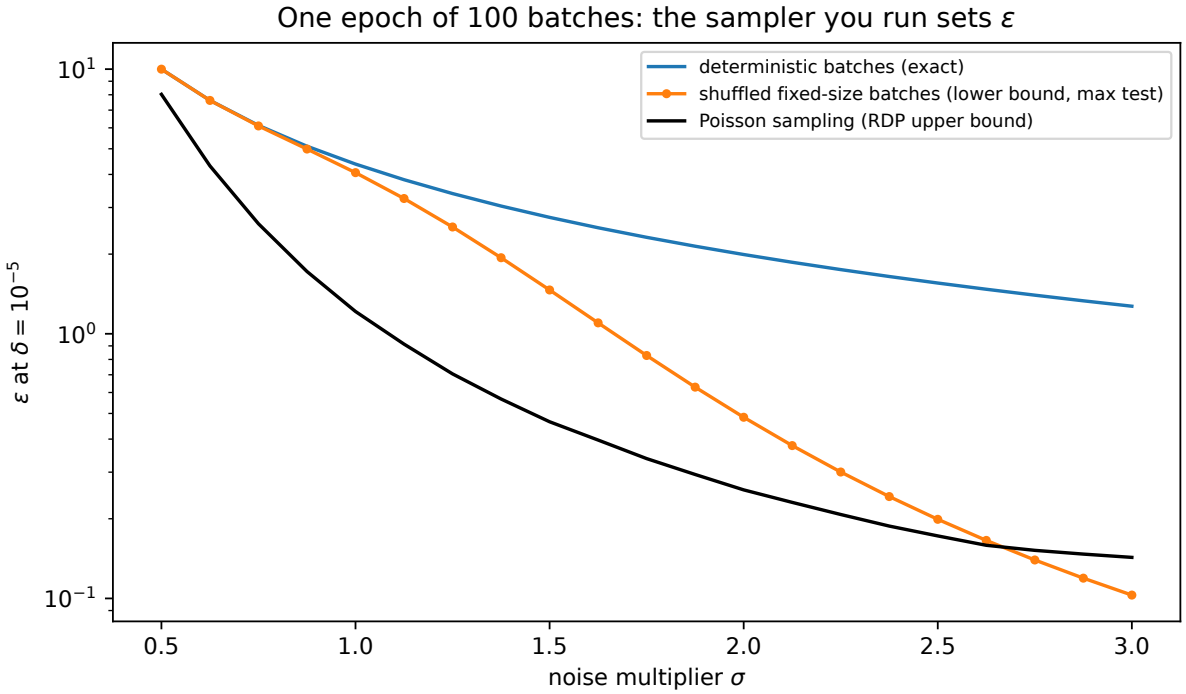


Figure 31.2.: The ε at $\delta = 10^{-5}$ of one epoch of 100 batches as a function of the noise multiplier: deterministic batches (exact), shuffled fixed-size batches (a lower bound from the largest-batch-output test of Chua et al. 2024), and Poisson sampling with $q = 0.01$ (an RDP upper bound). For σ up to about 2.5, shuffled training is provably less private than the Poisson accountant claims.

31.4.6. Check yourself

- Your code draws a fresh random batch of exactly 256 out of 25,600 records at each step. Which noise multiplier should you give a Poisson-sampling RDP accountant with $q = 0.01$? *(Half of the one you use, $\sigma/2$: the base mechanism's shift is $2C$.)*
- Why can't the mixture lemma be applied to one step of shuffled training? *(Given the other records' positions, the record's batch is fixed, so the step's output on D' is not a mixture of "used" and "not used". The randomness is only in which step uses it.)*

31.5. RDP of the Poisson-subsampled Gaussian

31.5.1. Context

Per-step (ε, δ) amplification composes badly (Section 31.7). RDP composes better. A mechanism is $(\alpha, \varepsilon(\alpha))$-RDP when the Rényi divergence of order α between its outputs on any two neighbouring datasets is at most $\varepsilon(\alpha)$. The Rényi divergence $D_\alpha(P\|Q) = \frac{1}{\alpha-1} \ln \mathbb{E}_{y\sim Q}\left[(P(y)/Q(y))^\alpha\right]$ is a log-average of a power of the likelihood ratio; a larger order α weights large ratios more heavily. Under composition the RDP curves of the steps simply add (Rényi DP), so you want the RDP curve of one subsampled Gaussian step. That is a Rényi divergence between a Gaussian mixture and a Gaussian. It has no closed form, but it has an exact series for integer orders and a stable numerical integral for the rest. Figure 31.3 shows the pair and its privacy loss, the logarithm of the likelihood ratio at each output.

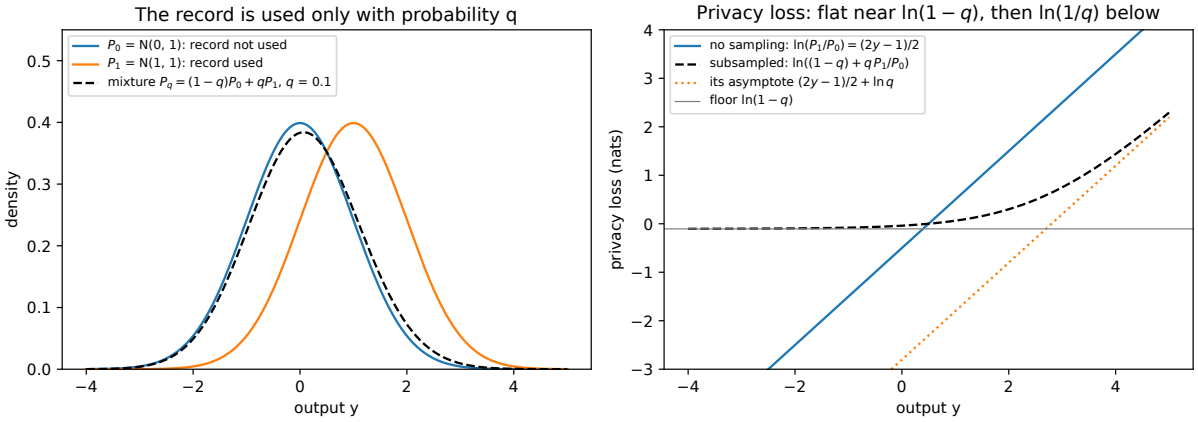


Figure 31.3.: Left: one subsampled Gaussian step with $q = 0.1$. The output with the record is the mixture (dashed), barely distinguishable from P_0. Right: the privacy loss $\ln\left((1-q) + q\,P_1/P_0\right)$. Where the record is unlikely to have been used it is flat near $\ln(1-q)$; in the far right tail it runs parallel to the unsubsampled loss, $\ln(1/q)$ lower.

31.5.2. Statement

The statement reduces the whole problem to one dimension and one direction. Whatever the data, the worst case is a single coordinate in which the record moves the sum by the full sensitivity, and

of the two orders of the pair the one with the mixture first is always the larger. That divergence is the average under P_0 of the mixture's likelihood ratio raised to the power α, and for integer α the binomial theorem expands it into a finite sum.

Let $P_0 = \mathcal{N}(0, \sigma^2)$, $P_1 = \mathcal{N}(1, \sigma^2)$ and $P_q = (1-q)P_0 + qP_1$. The Poisson-subsampled Gaussian mechanism with ℓ_2 sensitivity 1 (one record moves the sum by Euclidean length at most 1) and noise multiplier σ is $(\alpha, \varepsilon(\alpha))$-RDP for add/remove neighbours with

$$\varepsilon(\alpha) = \max\left\{D_\alpha(P_q\|P_0),\ D_\alpha(P_0\|P_q)\right\} = D_\alpha(P_q\|P_0) = \frac{\ln A_\alpha}{\alpha - 1}, \tag{31.9}$$

$$A_\alpha = \mathbb{E}_{y\sim P_0}\Big[\Big((1-q) + q\frac{P_1(y)}{P_0(y)}\Big)^\alpha\Big], \qquad \frac{P_1(y)}{P_0(y)} = e^{(2y-1)/(2\sigma^2)}. \tag{31.10}$$

For integer α,

$$A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\Big(\frac{j^2 - j}{2\sigma^2}\Big). \tag{31.11}$$

The first direction, $D_\alpha(P_q\|P_0)$, puts the dataset with the record first: it is the remove direction of Section 31.3 and of Numerical accounting. The reduction to one dimension is Mironov, Talwar & Zhang (2019, Theorem 4), the dominance of the first direction their Theorem 5 and Corollary 7, and Equation 31.11 their Section 3.3.

31.5.3. Derivation

The derivation first removes everything except the one coordinate along which the record moves the output, then evaluates the divergence of the remaining one-dimensional pair.

1. **Condition on the rest of the sample** (Mironov, Talwar & Zhang 2019, Theorem 4). Let f be the clipped gradient sum, divided by C so that the sensitivity is 1. Condition on the rest of the sample T. The outputs are $\mathcal{N}(f(T), \sigma^2 I)$ and the mixture $(1-q)\mathcal{N}(f(T), \sigma^2 I) + q\mathcal{N}(f(T \cup \{x\}), \sigma^2 I)$, and both runs average these over T with the same weights.
2. **Averaging cannot increase the divergence.** This is the RDP counterpart of Fact 1. For $\alpha > 1$, $e^{(\alpha-1)D_\alpha(P\|Q)} = \int P(y)^\alpha Q(y)^{1-\alpha}\, dy$. Write the integrand as $Q\,(P/Q)^\alpha$. The function t^α is convex, and a function of the form $Q\,g(P/Q)$ with g convex, called the **perspective** of g, is jointly convex in the pair (P, Q): the value at an average of pairs is at most the average of the values. So for averages with shared weights w_t, $\int (\sum_t w_t P_t)^\alpha (\sum_t w_t Q_t)^{1-\alpha} \le \sum_t w_t \int P_t^\alpha Q_t^{1-\alpha} \le \max_t \int P_t^\alpha Q_t^{1-\alpha}$, in either order of the pair. The divergence of the averages over T is at most the largest divergence for one T. This is the **quasi-convexity** of the Rényi divergence (van Erven & Harremoës 2014, Theorem 13).
3. **Shift, rotate and rescale.** Moving both distributions by the same amount or rotating both does not change any likelihood ratio, so it does not change the divergence. Shift by $f(T)$ and rotate so the difference $f(T \cup \{x\}) - f(T)$ lies along the first axis. The other coordinates have the same distribution in both runs and are independent of the first, and the Rényi divergence of independent coordinates is the sum of theirs (additivity, Divergences), so the identical ones contribute 0 and drop out. A difference of length $c < 1$ is, after rescaling by $1/c$, the same as length 1 with noise $\sigma/c > \sigma$. That pair is the length-1 pair with extra independent noise added, and no processing of the outputs can increase a divergence (data processing), so it cannot be worse. What is left is the pair (P_q, P_0) in both orders.

4. **Write the first direction as an expectation under P_0.** By the definition of D_α given in the context, $D_\alpha(P_q\|P_0) = \frac{1}{\alpha-1}\ln \mathbb{E}_{P_0}[(P_q/P_0)^\alpha]$, and dividing the mixture by P_0 gives $P_q/P_0 = (1-q)+q\,P_1/P_0$. This is Equation 31.10. The ratio P_1/P_0 is the Gaussian likelihood ratio: the normalizing constants cancel and $\exp\left(\frac{-(y-1)^2+y^2}{2\sigma^2}\right) = e^{(2y-1)/(2\sigma^2)}$.
5. **Expand for integer α.** Write $r = P_1/P_0$. The binomial theorem gives $(1-q+qr)^\alpha = \sum_j \binom{\alpha}{j}(1-q)^{\alpha-j}q^j r^j$, and the expectation of a sum is the sum of the expectations, so A_α is this sum with each r^j replaced by $\mathbb{E}_{P_0}[r^j]$.
6. **Compute each moment.** $\mathbb{E}_{P_0}[r^j] = \mathbb{E}[e^{j(2y-1)/(2\sigma^2)}]$ for $y \sim \mathcal{N}(0,\sigma^2)$. The moment generating function of this normal is $\mathbb{E}[e^{sy}] = e^{s^2\sigma^2/2}$, and with $s = j/\sigma^2$ it gives $e^{j^2/(2\sigma^2)}$. Multiplying by the constant factor $e^{-j/(2\sigma^2)}$, the j-th moment is $e^{(j^2-j)/(2\sigma^2)}$, which gives Equation 31.11. Equivalently, $\mathbb{E}_{P_0}[(P_1/P_0)^j] = e^{(j-1)D_j(P_1\|P_0)}$ by the definition of D_j, and $D_j(P_1\|P_0) = j/(2\sigma^2)$ for two Gaussians with unit shift (Divergences, Equation 3.15).
7. **Fractional α: integrate numerically.** For non-integer α the binomial sum does not terminate, so compute Equation 31.10 as an integral. Its integrand is $P_0(y)$ times a power of the mixture ratio. Huge and tiny factors would overflow, so work with its logarithm, $\ln P_0(y)+\alpha\,\mathrm{logaddexp}(\ln(1-q), \ln q+\frac{2y-1}{2\sigma^2})$, where $\mathrm{logaddexp}(u,v) = \ln(e^u+e^v)$ is computed without forming e^u or e^v. Sum on a grid of step $\sigma/100$ with log-sum-exp, the same trick for a sum of many exponentials. The integrand has a bump near $y=0$ and, for large α, a second one near $y=\alpha$. The second comes from the part $q^\alpha e^{\alpha(2y-1)/(2\sigma^2)}$: multiplied by $P_0(y) \propto e^{-y^2/(2\sigma^2)}$, it gives an exponent $(-y^2+2\alpha y)/(2\sigma^2)$, a Gaussian centred at $y=\alpha$ (the tilted distributions of Divergences). So the grid runs from -12σ to $\alpha+12\sigma$.
8. **The other direction.** By the definition, $D_\alpha(P_0\|P_q) = \frac{1}{\alpha-1}\ln \mathbb{E}_{P_q}[(P_0/P_q)^\alpha]$. Changing the expectation from P_q to P_0 multiplies the integrand by P_q/P_0, which turns $(P_0/P_q)^\alpha$ into $(P_0/P_q)^{\alpha-1}$, so $D_\alpha(P_0\|P_q) = \frac{1}{\alpha-1}\ln \mathbb{E}_{P_0}[(P_q/P_0)^{-(\alpha-1)}]$. The power is negative, so there is no finite binomial sum; integrate it numerically. Mironov, Talwar & Zhang (2019, Theorem 5) prove $D_\alpha(P_q\|P_0) \geq D_\alpha(P_0\|P_q)$ for every $\alpha \geq 1$. Their argument uses the reflection $y \mapsto 1-y$, which swaps P_0 and P_1. The script computes both and takes the larger, which is always the first. This dominance is for Rényi divergences. For $\delta(\varepsilon)$, Section 32.7 computes both directions, because neither dominates in general.

Walk-through at $\alpha = 2$. With $q = 0.01$ and $\sigma = 1$, Equation 31.11 has three terms, one for each possible number j of factors r (Table 31.6).

Table 31.6.: The binomial sum for A_2 with $q = 0.01$, $\sigma = 1$. Without the exponential factors the terms would sum to exactly 1.

j	$\binom{2}{j}(1-q)^{2-j}q^j$	$e^{(j^2-j)/2}$	Term
0	$(0.99)^2 = 0.9801$	1	0.980100
1	$2 \times 0.99 \times 0.01 = 0.0198$	1	0.019800
2	$(0.01)^2 = 0.0001$	$e = 2.718$	0.000272

Without the exponential factors the three coefficients add up to $(1-q+q)^2 = 1$. The exponential factor only changes the last term, from q^2 to $q^2 e^{1/\sigma^2}$, so the terms sum to $A_2 = 1+q^2(e^{1/\sigma^2}-1) = 1+0.0001\times 1.71828 = 1.00017183$, and $\varepsilon(2) = \ln A_2/(2-1) = 1.7181\times 10^{-4}$. Without sampling, $\varepsilon(2) = 2/(2\sigma^2) = 1$. Only the $j=2$ term carries any privacy cost, because the terms $j=0$ and 1 have the same factor 1 that a mixture of identical distributions would have.

31.5.4. Interpretation

Two regimes. Figure 31.4 shows the curve for $q = 0.01$, $\sigma = 1$. Up to about $\alpha = 8$ the curve stays within 30% of the small-q line, the $j = 2$ term alone, worked out in Section 31.6 (1.30 times it at $\alpha = 8$, Table 31.8): $\varepsilon(8) = 8.9364 \times 10^{-4}$, against 4 without sampling. Between $\alpha = 8$ and 16 it climbs steeply: $\varepsilon(16) = 3.0879$ and $\varepsilon(32) = 11.246$, against 8 and 16 without sampling. The reason is in the right panel of Figure 31.3. Low orders average the privacy loss over the bulk, where sampling flattens it. High orders look at the far tail, where the loss is the unsubsampled one minus $\ln(1/q)$. In the tail, sampling saves only $\ln(1/q)$, as for (ε, δ) at large ε. The one-term lower bound Equation 29.21 of Concentrated DP makes this precise. Every term of Equation 31.11 is positive, so A_α is at least its last term, $q^\alpha e^{\alpha(\alpha-1)/(2\sigma^2)}$, and dividing its logarithm by $\alpha - 1$ gives $\varepsilon(\alpha) \geq \frac{\alpha \ln q}{\alpha - 1} + \frac{\alpha}{2\sigma^2}$: the unsubsampled curve minus about $\ln(1/q)$.

Which orders matter. An accountant turns the RDP curve into an (ε, δ) guarantee by a conversion that holds at every order, such as $\varepsilon = k\,\varepsilon(\alpha) + \ln(1/\delta)/(\alpha - 1)$ for k steps, and it only uses the order that makes this smallest. For long DP-SGD runs that order sits in the quadratic regime. For a few steps it sits at the foot of the steep rise, where $k\,\varepsilon(\alpha)$ is still small next to the conversion term (Section 31.7).

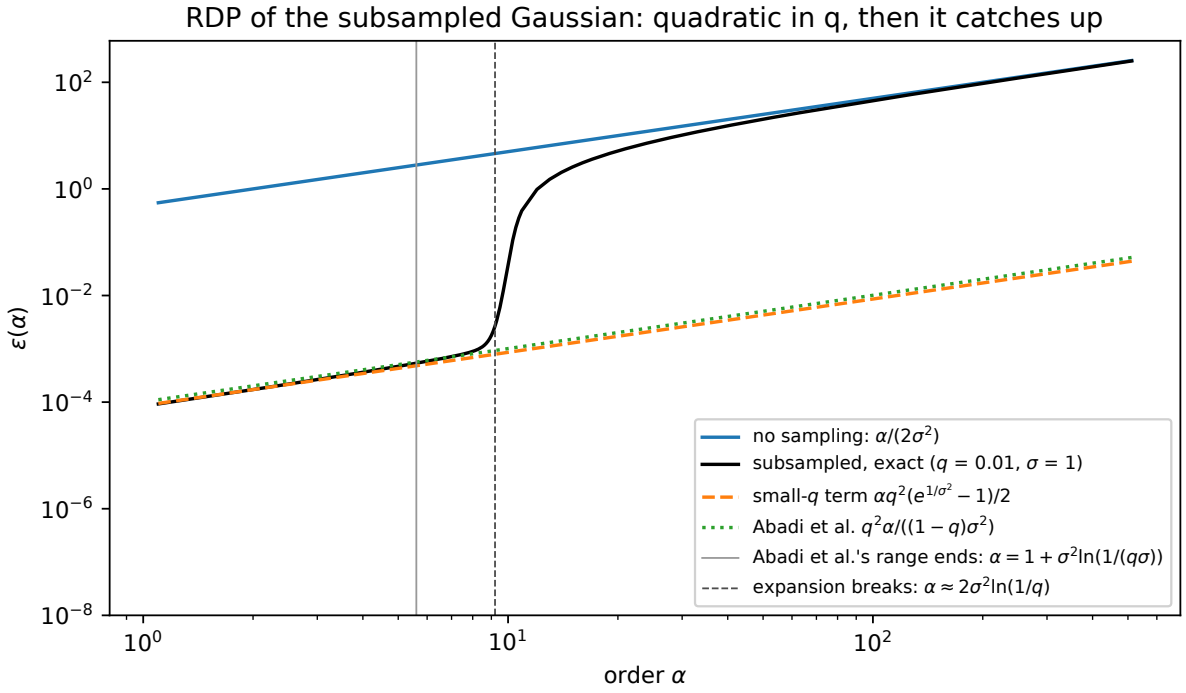


Figure 31.4.: The RDP curve of the Poisson-subsampled Gaussian with $q = 0.01$, $\sigma = 1$ (black), against the unsubsampled Gaussian $\alpha/(2\sigma^2)$, the small-q term $\alpha q^2(e^{1/\sigma^2} - 1)/2$ and the leading term of Abadi et al.'s bound. The vertical line marks where Abadi et al.'s bound stops applying. The dashed vertical line marks $\alpha = 2\sigma^2 \ln(1/q)$, where the quadratic expansion breaks. The exact curve stays near the quadratic term up to about $\alpha = 8$, then rises toward the unsubsampled line.

31.5.5. Example: checking the computation

The binomial sum is exact but only exists at integer orders; the numerical integral covers every order but could be wrong if the grid were too coarse or too short. Comparing them where both exist tests the integral. Table 31.7 does this at integer orders, and also compares the two directions at several orders, for $q = 0.01$ and $\sigma = 1$.

Table 31.7.: The two ways of computing $D_\alpha(P_q \| P_0)$ agree to seven digits, and the reverse direction is always smaller. $q = 0.01$, $\sigma = 1$.

α	Binomial sum	Integral	$D_\alpha(P_q \| P_0)$	$D_\alpha(P_0 \| P_q)$
1.5	—	—	1.2725×10^{-4}	1.2148×10^{-4}
2	1.718134×10^{-4}	1.718134×10^{-4}	1.7181×10^{-4}	1.6022×10^{-4}
3	2.646376×10^{-4}	2.646376×10^{-4}	—	—
8	—	—	8.9364×10^{-4}	5.7239×10^{-4}
10	3.827042×10^{-2}	3.827042×10^{-2}	—	—
32	1.124628×10^{1}	1.124628×10^{1}	1.1246×10^{1}	1.6923×10^{-3}

The integral agrees with the exact sum to all printed digits, from 10^{-4} to 10^{1}, so the grid is fine enough. The reverse direction is smaller at every order. At $\alpha = 32$ it is nearly four orders of magnitude smaller (1.6923×10^{-3} against 11.246), because it raises the mixture ratio to a negative power and so looks at the left tail, where the ratio P_0/P_q is bounded by $1/(1-q)$: the mixture always keeps at least $1 - q$ of P_0. The script also checks every order of the grid for $\sigma = 0.5$ and $\sigma = 2$: the largest excess of the reverse direction is negative in both cases (-5.09×10^{-4} and -5.67×10^{-8}).

31.5.6. Check yourself

- In Equation 31.11, why do the terms $j = 0$ and $j = 1$ never cost privacy? *(Their exponential factor is $e^0 = 1$. They are the terms in which the record's influence enters at most linearly, and a linear term averages to the same value under both distributions, since $\mathbb{E}_{P_0}[P_1/P_0] = 1$.)*
- Why does the numerical grid for $D_\alpha(P_q \| P_0)$ extend to $y = \alpha + 12\sigma$, far beyond where P_0 has any mass? *(For large α the integrand is dominated by $q^\alpha P_0(y) e^{\alpha(2y-1)/(2\sigma^2)}$, which is a Gaussian bump centred at $y = \alpha$. Cutting the grid at 12σ would miss it.)*

31.6. Small q and the moments accountant

31.6.1. Context

Abadi et al. (2016) introduced DP-SGD together with the **moments accountant**. Its central quantity is built from the **privacy loss**, $L = \ln\left(P(y)/Q(y)\right)$ for an output y drawn from P: the log-likelihood ratio of the output, which is large when the output gives the record away. The accountant tracks the logarithm of the moment generating function of L, which Abadi et al. call

$\alpha_M(\lambda)$ and Rényi DP renames $K_M(\lambda) = \ln \mathbb{E}[e^{\lambda L}]$, because α is our order. It adds K_M over steps and converts the total to (ε, δ) with a tail bound.

This is RDP in disguise, as Rényi DP shows. Since $e^{\lambda L} = (P/Q)^\lambda$, the expectation under P is $\mathbb{E}_P[(P/Q)^\lambda] = \mathbb{E}_Q[(P/Q)^{\lambda+1}] = e^{\lambda D_{\lambda+1}(P\|Q)}$, so $K_M(\lambda) = \lambda\,\varepsilon(\lambda+1)$. The tail bound is Markov's inequality applied to $e^{\lambda L}$: $\Pr[L > \varepsilon] \le \mathbb{E}[e^{\lambda L}]e^{-\lambda\varepsilon}$, which gives $\delta = \min_\lambda e^{K_M(\lambda)-\lambda\varepsilon}$, Mironov's conversion. Abadi et al. did not have Equation 31.11 in general. They proved an asymptotic bound for small q. How good is it, and where does it stop holding?

31.6.2. Statement

When q is small, the walk-through at $\alpha = 2$ generalizes: the terms $j = 0$ and 1 of Equation 31.11 cost nothing, every term from $j = 3$ on carries q^3 or a higher power, and the $j = 2$ term dominates. For $q \to 0$ at fixed α and σ, the leading term of the subsampled Gaussian's RDP curve is

$$\varepsilon(\alpha) \approx \frac{\alpha q^2(e^{1/\sigma^2} - 1)}{2} \approx \frac{\alpha q^2}{2\sigma^2} \quad \text{for large } \sigma. \tag{31.12}$$

Abadi et al. (2016, Lemma 3) prove a bound of the same shape, under conditions that limit the order. For $\sigma \ge 1$, $q < 1/(16\sigma)$ and integer $\lambda \le \sigma^2 \ln \frac{1}{q\sigma}$,

$$K_M(\lambda) \le \frac{q^2\lambda(\lambda+1)}{(1-q)\sigma^2} + O\Big(\frac{q^3\lambda^3}{\sigma^3}\Big), \quad \text{that is} \quad \varepsilon(\alpha) \lesssim \frac{q^2\alpha}{(1-q)\sigma^2} \ \text{ for } \ \alpha \le 1 + \sigma^2 \ln \frac{1}{q\sigma}. \tag{31.13}$$

The second form divides by λ and substitutes $\alpha = \lambda + 1$.

31.6.3. Derivation

1. **Subtract what costs nothing.** Without the exponential factors, Equation 31.11 is $\sum_j \binom{\alpha}{j}(1-q)^{\alpha-j}q^j = (1-q+q)^\alpha = 1$ by the binomial theorem. Subtracting this 1 from A_α leaves each term multiplied by its exponential factor minus 1:

$$A_\alpha = 1 + \sum_{j=2}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \big(e^{(j^2-j)/(2\sigma^2)} - 1\big). \tag{31.14}$$

   The $j = 0$ and $j = 1$ terms are gone, because their factor is $e^0 - 1 = 0$.
2. **Keep the lowest power of q.** The $j = 2$ term is $\binom{\alpha}{2}q^2(1-q)^{\alpha-2}(e^{1/\sigma^2} - 1)$. Every other term carries q^3 or more, so for small q it is negligible next to this one.
3. **Take the logarithm.** For small y, $\ln(1+y) = y + O(y^2)$, and $(1-q)^{\alpha-2} = 1 + O(q)$. So $(\alpha-1)\varepsilon(\alpha) = \ln A_\alpha = \frac{\alpha(\alpha-1)}{2}q^2(e^{1/\sigma^2} - 1) + O(q^3)$.
4. **Divide by** $\alpha - 1$ to get Equation 31.12. For large σ, $e^{1/\sigma^2} - 1 \approx 1/\sigma^2$, since $e^u \approx 1 + u$ for small u.
5. **Where does the expansion break?** The expansion assumes the $j = 2$ term dominates, so look at how the terms change with j. Going from j to $j+1$, the binomial coefficient is multiplied by $\frac{\alpha-j}{j+1}$, the power of q by q, and the exponential by $e^{((j+1)^2-(j+1)-j^2+j)/(2\sigma^2)} = e^{j/\sigma^2}$. So the terms first fall, and grow once $q\,e^{j/\sigma^2}$ passes 1, near $j \approx \sigma^2 \ln(1/q)$. The expansion fails only later, when the last term, $q^\alpha e^{\alpha(\alpha-1)/(2\sigma^2)}$, overtakes the $j = 2$ term,

about $q^2 e^{1/\sigma^2}$. Ignoring the binomial coefficient and comparing logarithms, that happens when $\alpha \ln q + \frac{\alpha(\alpha-1)}{2\sigma^2} > 2\ln q + \frac{1}{\sigma^2}$. Move everything to one side: the terms in $\ln q$ give $-(\alpha-2)\ln\frac{1}{q}$, and the others give $\frac{\alpha^2-\alpha-2}{2\sigma^2}$. Since $\alpha^2-\alpha-2=(\alpha-2)(\alpha+1)$, the condition is $(\alpha-2)(\frac{\alpha+1}{2\sigma^2}-\ln\frac{1}{q})>0$, that is $\alpha > 2\sigma^2\ln(1/q)-1$. So the expansion holds up to about

$$\alpha \approx 2\sigma^2 \ln(1/q), \tag{31.15}$$

and the curve leaves the quadratic regime abruptly there. At $q=0.01$, $\ln(1/q)=4.605$, so this is 9.2 for $\sigma=1$ and 147.4 for $\sigma=4$. The exact curve first exceeds twice the leading term at 8.9 and 147.4 (Table 31.8). Abadi et al.'s condition $\lambda \le \sigma^2 \ln\frac{1}{q\sigma}$ has the same form with about half the constant, so it is conservative.

31.6.4. Interpretation

The first thing to notice is that the cost is quadratic, not linear, in q. The (ε,δ) amplification of Section 31.3 gives a per-step ε' linear in q. In RDP the cost per step is quadratic in q, about $\alpha q^2/(2\sigma^2)$. Since $k=E/q$ steps make E epochs, the total RDP after E epochs is about $(E/q)\times\alpha q^2/(2\sigma^2) = E\,q\,\alpha/(2\sigma^2)$. It shrinks as batches get smaller at a fixed number of epochs.

Abadi et al.'s bound is about twice the leading term for large σ: $q^2\alpha/\sigma^2$ against $q^2\alpha/(2\sigma^2)$. It is a valid but loose upper bound, and it is the source of the "$\varepsilon(\alpha)\approx q^2\alpha/\sigma^2$" you will see quoted.

The real limit of the expansion is not its constant but its order range. Up to $\alpha=\sigma^2\ln(1/q)$ the leading term is within 8% (1.08 at $\alpha=4.6$ for $\sigma=1$, 1.05 at $\alpha=73.7$ for $\sigma=4$). Beyond $\alpha\approx 2\sigma^2\ln(1/q)$ (Equation 31.15) it is wrong by orders of magnitude, 2246 times too small at $\alpha=16$ for $\sigma=1$, and an accountant that uses it there under-reports ε.

31.6.5. Example: where the expansion breaks

Table 31.8 compares the exact curve with Equation 31.12 and Equation 31.13 at $q=0.01$. For instance, at $\alpha=2$ and $\sigma=1$ the Abadi column is $q^2\alpha/((1-q)\sigma^2) = 0.0002/0.99 = 2.0202\times10^{-4}$.

Table 31.8.: The exact RDP of the subsampled Gaussian against its small-q expansion and Abadi et al.'s leading term, $q=0.01$.

α	Exact, $\sigma=1$	Leading term, $\sigma=1$	Abadi, $\sigma=1$	Exact / leading, $\sigma=1$	Exact / leading, $\sigma=4$
2	1.7181×10^{-4}	1.7183×10^{-4}	2.0202×10^{-4}	1.00	1.00
4	3.6315×10^{-4}	3.4366×10^{-4}	4.0404×10^{-4}	1.06	1.00
6	5.8350×10^{-4}	5.1548×10^{-4}	6.0606×10^{-4}	1.13	1.00
8	8.9364×10^{-4}	6.8731×10^{-4}	8.0808×10^{-4}	1.30	1.00
10	3.8270×10^{-2}	8.5914×10^{-4}	1.0101×10^{-3}	44.54	1.01
32	1.1246×10^{1}	2.7493×10^{-3}	3.2323×10^{-3}	4090.67	1.02

At $\sigma=1$, Abadi et al.'s range ends at $\alpha = 1+\sigma^2\ln\frac{1}{q\sigma} = 1+\ln 100 = 5.6$, and the exact curve first exceeds twice the leading term at the grid order $\alpha=8.9$ (8.9 also by bisection over fractional

orders), just below $2\sigma^2 \ln(1/q) = 9.2$. Inside its range Abadi et al.'s value is above the exact one, as an upper bound should be. At $\alpha = 8$, outside it, the leading term of their bound (8.0808×10^{-4}) is already below the exact 8.9364×10^{-4}.

Step 5 of the derivation predicted terms that first fall and then grow; at $\alpha = 10$ you can see it. The binomial terms for $j = 2, \dots, 10$ are 1.1×10^{-2}, 2.2×10^{-3}, 8.0×10^{-4}, 5.3×10^{-4}, 6.6×10^{-4}, 1.5×10^{-3}, 6.4×10^{-3}, 4.3×10^{-2} and 3.5×10^{-1}: they first fall and then grow, and the last one dominates. At $\sigma = 4$ the range ends at $\alpha = 52.5$ and the leading term is within 2% of the exact value up to $\alpha = 32$; the exact curve first exceeds twice the leading term between the grid orders 128 and 256, at $\alpha = 147.4$ by bisection, where Equation 31.15 puts it.

Abadi et al.'s headline number. With $q = 0.01$, $\sigma = 4$, $\delta = 10^{-5}$ and 10,000 steps they report $\varepsilon \approx 1.26$ from the moments accountant. The exact RDP curve, composed over the 10,000 steps and converted with Mironov's conversion, gives 1.2586, at $\alpha = 20$, which matches their number to the digits they report. The improved conversion of From RDP to (ε, δ) gives 1.0355, at $\alpha = 17$.

31.6.6. Check yourself

- At a fixed number of epochs, does halving the batch size (and so q) raise or lower the RDP accountant's total, to first order? *(Lower it. There are twice as many steps, each costing a quarter as much, so the total halves, as long as the orders used stay in the quadratic regime.)*
- Why does a small σ shrink the range where Equation 31.12 holds? *(The higher terms grow like $q^j e^{j^2/(2\sigma^2)}$. A small σ makes the exponential overtake the power of q at a smaller j, and so at a smaller α.)*

31.7. The running example: DP-SGD

31.7.1. Context

Everything is now in place to account for the running example: DP-SGD with Poisson rate $q = 0.01$, noise multiplier $\sigma = 1$, $\delta = 10^{-5}$, and k up to 10,000 steps. How large is ε, and how much does the accounting route matter?

31.7.2. Statement

The RDP accountant adds the per-step curve of Equation 31.9 over the k steps and converts the total to (ε, δ) once. A conversion takes an RDP guarantee at one order α and returns an ε for the target δ. It holds at every order, so the accountant takes the order that gives the smallest ε. Mironov's conversion adds $\ln(1/\delta)/(\alpha - 1)$ to the RDP value; the improved conversion subtracts two small corrections from that amount. DP-SGD is (ε, δ)-DP for add/remove neighbours with

$$\varepsilon = \min_{\alpha} \left[k\,\varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha - 1} \right] \quad \text{(Mironov)}, \qquad \varepsilon = \min_{\alpha} \left[k\,\varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1} \right] \quad \text{(improved)}, \tag{31.16}$$

with $\varepsilon(\alpha)$ from Equation 31.9 and the minimum over the grid of orders shared by this part.

31.7.3. Derivation

1. **One step is a subsampled Gaussian.** Clipping each per-example gradient to norm C and adding $\mathcal{N}(0, \sigma^2 C^2 I)$ to the sum makes one step the mechanism of Section 31.5, after dividing by C. Its RDP curve is $\varepsilon(\alpha)$.
2. **Compose.** Each step may depend on the previous outputs, through the model it starts from. RDP curves add under such adaptive composition (Rényi DP), so k steps have curve $k\,\varepsilon(\alpha)$.
3. **Convert.** Mironov's conversion is from Rényi DP: it is the Markov bound on the privacy loss recalled in Section 31.6, solved for ε. The improved one (Balle et al. 2020, Theorem 21) is from From RDP to (ε, δ). Each holds at every order, so take the smallest.

The naive routes, for comparison, amplify each step in (ε, δ) form with Equation 31.5 and then compose the per-step pairs.

1. **Per step.** Give each step a share δ_0 of δ before amplification. The plain Gaussian mechanism with sensitivity 1 and noise σ is $(\varepsilon_0, \delta_0)$-DP exactly when $\delta_0 = \Phi(\frac{1}{2\sigma} - \varepsilon_0\sigma) - e^{\varepsilon_0}\Phi(-\frac{1}{2\sigma} - \varepsilon_0\sigma)$, the hockey-stick divergence of the two Gaussians, with Φ the standard normal distribution function (The Laplace and Gaussian mechanisms). Solve this for ε_0, then amplify with Equation 31.5 to $(\varepsilon_1, q\delta_0)$.
2. **Basic composition.** The εs and δs of the steps add, so k steps are $(k\varepsilon_1, kq\delta_0)$-DP (Composition). Setting $kq\delta_0 = \delta$ fixes $\delta_0 = \delta/(kq)$.
3. **Advanced composition.** Advanced composition trades a little extra δ' for an ε that grows like $\sqrt{k}$ instead of k: k steps are $(\varepsilon_1\sqrt{2k\ln(1/\delta')} + k\varepsilon_1(e^{\varepsilon_1} - 1),\ kq\delta_0 + \delta')$-DP (Composition). The script splits δ between $kq\delta_0$ and δ', trying shares of δ' from 0.01 to 0.99, and keeps the best.

31.7.4. Interpretation

RDP is the only usable route for long runs. After 10,000 steps it gives $\varepsilon = 6.71$; the per-step routes give numbers in the thousands (Table 31.9). The example below shows where the per-step routes lose.

For one step, though, RDP loses. At $k = 1$ the per-step route gives 0.20 and RDP 0.96. The reason is the conversion: it costs $\ln(1/\delta)/(\alpha - 1)$ no matter how small the curve is. RDP pays this once, so it wins once there are a handful of steps to compose: from $k = 4$ here (Figure 31.5).

The improved conversion saves from 27.5% at $k = 1$ to 9.6% at $k = 10{,}000$: 0.96 against 1.32, and 6.71 against 7.43.

The best order falls as k grows, from 9.9 to 4.1, because the curve term $k\,\varepsilon(\alpha)$ grows with k and pulls the minimum toward smaller orders, where the curve is lower. For a few steps the best order sits at the foot of the steep rise in Figure 31.4: at the best orders for $k = 1$, 10 and 100, the exact curve is 30.61, 5.44 and 1.82 times the quadratic term. It is chosen there because $k\,\varepsilon(\alpha)$ is still small next to the conversion term (0.026, 0.044 and 0.138), so a larger order, which shrinks the conversion term, is worth it. Only for long runs does the best order move into the quadratic regime: the ratio is 1.26 at $k = 1000$ and 1.06 at $k = 10{,}000$.

How tight is 6.71? It is an upper bound with two sources of slack: the conversion from RDP, and the RDP curve itself, which keeps only one summary of the privacy loss distribution. Numerical accounting computes the tight value for the same run, and Choosing an accountant compares all the accountants of this part on it.

31.7.5. Example: the running example at k = 10,000

Table 31.9 and Figure 31.5 give ε for the running example under the four routes.

Table 31.9.: The ε at $\delta = 10^{-5}$ of DP-SGD with $q = 0.01$, $\sigma = 1$ under four accounting routes.

Steps k	RDP, Mironov	RDP, improved	Best order (improved)	Per step, basic	Per step, advanced
1	1.32	0.96	9.9	0.20	1.13
10	1.41	1.04	9.4	3.64	7.88
100	1.61	1.21	8.8	58.01	78.10
1,000	2.54	2.10	7.8	839.39	1258.64
10,000	7.43	6.71	4.1	11296.93	24379.55

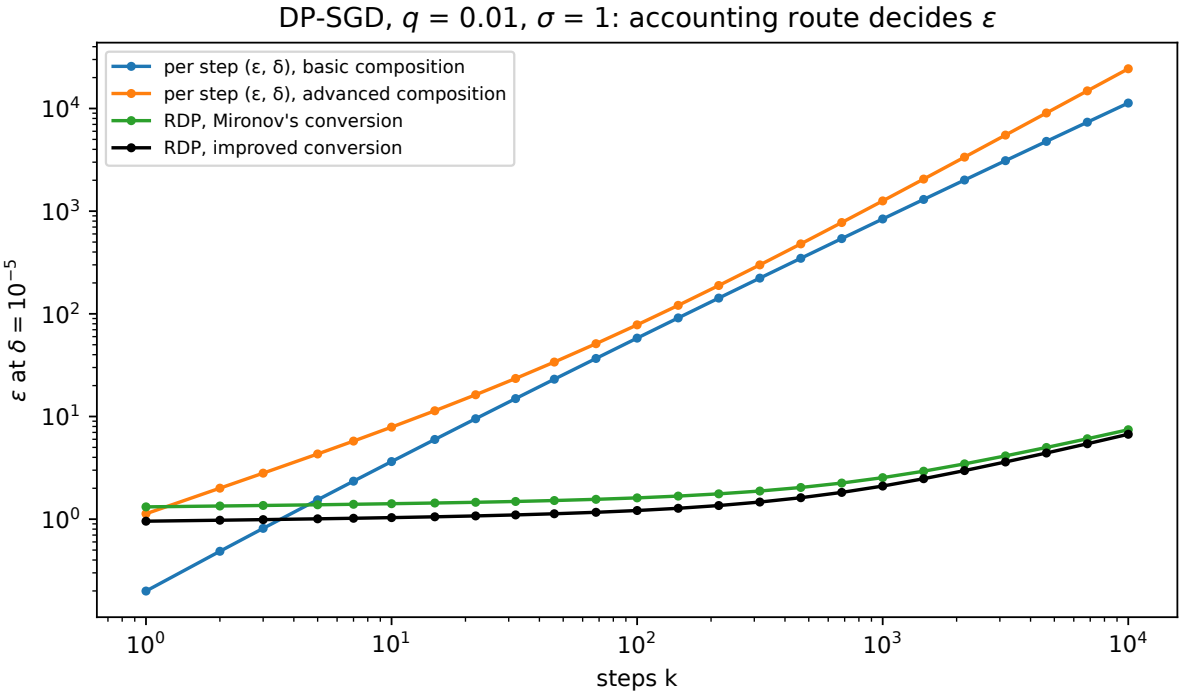


Figure 31.5.: The ε at $\delta = 10^{-5}$ of DP-SGD with $q = 0.01$, $\sigma = 1$ against the number of steps, under four accounting routes. The RDP accountant grows slowly; per-step amplification followed by basic or advanced composition grows almost linearly and is useless beyond a few steps.

Follow the per-step route at $k = 10{,}000$ to see where it loses. The first loss is weak amplification. With basic composition the route must spend $\delta_0 = \delta/(kq) = 10^{-5}/100 = 10^{-7}$ per step. That forces

$\varepsilon_0 = 5.35$, in the regime where amplification only subtracts (Figure 31.1): $\ln\left(1 + 0.01(e^{5.35} - 1)\right)$ gives the amplified $\varepsilon_1 = 1.130$, only about five times smaller, and basic composition then multiplies it by 10,000.

Advanced composition does not rescue it; it is worse than basic at every k here. Its first term, $\varepsilon_1\sqrt{2k\ln(1/\delta')}$, beats $k\varepsilon_1$ only when $k \gg 2\ln(1/\delta')$. Its second term, $k\varepsilon_1(e^{\varepsilon_1} - 1)$, alone exceeds $k\varepsilon_1$ once $e^{\varepsilon_1} - 1 > 1$, that is $\varepsilon_1 > \ln 2$, as here. The sharper form of Composition, with $\tanh(\varepsilon_1/2)$ in place of $e^{\varepsilon_1} - 1$, gives 6433: better than basic composition, and still useless.

The RDP accountant, by contrast, composes the whole curve, where each step costs almost nothing at moderate orders, and converts once, at the order 4.1.

31.7.6. Check yourself

- At $k = 10{,}000$ the best order is about 4. If you only had orders $\alpha \geq 8$ available, would the reported ε go up or down? *(Up. The conversion is a minimum over orders, and removing orders can only raise it.)*
- Why does the RDP column barely move between $k = 1$ and $k = 100$? *(For small k the conversion term $\ln(1/\delta)/(\alpha - 1)$ dominates and $k\varepsilon(\alpha)$ is negligible at the chosen order.)*

31.8. No tight amplification theorem for every mechanism

31.8.1. Context

For (ε, δ), Equation 31.5 is one formula, and it is tight over the class of (ε, δ)-DP mechanisms: for every ε and q, randomized response attains it. Is there an RDP analogue: a function that takes any mechanism's RDP curve and q and returns the subsampled curve, tightly? No. The best general results are tight only up to small constant factors, and accountants special-case the Gaussian because it is the case where the exact answer is known.

31.8.2. Statement

The general bound starts from the same binomial expansion as the Gaussian case, with each moment of the likelihood ratio bounded by the mechanism's own RDP curve at that order, and pays a factor 3 on the higher terms to cover the direction the expansion does not reach. Let M be any mechanism with RDP curve $\varepsilon(\alpha)$ for add/remove neighbours, and let q be the Poisson rate. For integer $\alpha \geq 2$, Zhu & Wang (2019, Theorem 5) show that the subsampled mechanism has RDP at most

$$\frac{1}{\alpha - 1}\ln\left[(1-q)^{\alpha-1}(\alpha q - q + 1) + \binom{\alpha}{2}q^2(1-q)^{\alpha-2}e^{\varepsilon(2)} + 3\sum_{j=3}^{\alpha}\binom{\alpha}{j}(1-q)^{\alpha-j}q^j e^{(j-1)\varepsilon(j)}\right]. \tag{31.17}$$

The first term collects $j = 0$ and $j = 1$: $(1-q)^\alpha + \alpha q(1-q)^{\alpha-1} = (1-q)^{\alpha-1}(\alpha q - q + 1)$. Without the factor 3 the same expression is a lower bound for some mechanisms (their Theorem 6). It is exact under a condition on the privacy loss that the Gaussian and Laplace mechanisms satisfy (their Theorem 8 and Proposition 10). For sampling a fixed batch without replacement under

replace-one neighbours, Wang, Balle & Kasiviswanathan (2019, Theorem 9) give a bound of the form $\frac{1}{\alpha-1}\ln\left(1+\sum_{j\geq 2}\binom{\alpha}{j}\eta^j c_j\right)$ with $\eta = m/n$, where $c_2 \leq 4(e^{\varepsilon(2)}-1)$ and $c_j \leq 2e^{(j-1)\varepsilon(j)}$ for $j \geq 3$, and a lower bound (Proposition 11) that matches it up to constant factors.

31.8.3. Derivation (why the Gaussian is special)

1. **The first direction expands for any mechanism.** The binomial expansion (step 5 of the derivation in Section 31.5) never used the Gaussian: for integer α, $D_\alpha((1-q)P_0 + qP_1\|P_0)$ expands into moments $\mathbb{E}_{P_0}[(P_1/P_0)^j]$. By the definition of the Rényi divergence each moment equals $e^{(j-1)D_j(P_1\|P_0)}$, and RDP of M bounds it by $e^{(j-1)\varepsilon(j)}$. Bounding each term this way gives Equation 31.17 without the factor 3.
2. **But each term may come from a different pair.** $\varepsilon(j)$ is a maximum over neighbouring pairs, taken separately for each j. The bound in step 1 is attained only if one pair attains $\varepsilon(j)$ for every j at once. For the Gaussian it does: the pair whose sums differ by the full sensitivity is the worst for every order.
3. **The second direction does not expand.** $D_\alpha(P_0\|(1-q)P_0 + qP_1)$ has the mixture in the denominator, raised to a negative power, as in step 8 of the derivation in Section 31.5, so there is no finite binomial sum. For a general mechanism this direction can exceed the first. A general bound must cover it too, and Zhu & Wang's bound carries a factor 3 that the first-direction expansion of step 1 does not need. For the Gaussian, the reflection argument of Mironov, Talwar & Zhang (2019, Theorem 5) shows that the first direction always dominates in Rényi divergence.

31.8.4. Interpretation

For the Gaussian, the general bound Equation 31.17 is close but not exact, and the factor 3 costs most at the orders in between (see the example). That is why accountants use Equation 31.11 for the Gaussian and keep the general bounds for other mechanisms.

Lebeda et al. (2024) add two further warnings. For add/remove neighbours, the worst pair of datasets for one step need not be the worst for the composition of many steps; they give a counterexample with the subsampled Laplace mechanism, so a numerical accountant must check its worst-case pair, not assume it. And under replace-one neighbours there may be several candidate worst-case pairs, all of which an accountant must consider.

31.8.5. Example: the general bound on the Gaussian

Table 31.10 evaluates Equation 31.17 with the Gaussian's curve $\varepsilon(j) = j/(2\sigma^2)$ against the exact curve.

Table 31.10.: Zhu & Wang's general bound applied to the Gaussian against the exact RDP, $q = 0.01$, $\sigma = 1$.

α	2	4	8	16	32
General bound Equation 31.17	1.7181×10^{-4}	4.1880×10^{-4}	1.4036×10^{-3}	3.1611	11.282
Exact Equation 31.9	1.7181×10^{-4}	3.6315×10^{-4}	8.9364×10^{-4}	3.0879	11.246
Ratio	1.00	1.15	1.57	1.02	1.00

The factor 3 costs nothing at $\alpha = 2$, where there are no $j \geq 3$ terms. It costs little at high orders, where the last term dominates the sum inside the logarithm, so multiplying it by 3 adds only about $\ln 3$ to the logarithm and $\ln 3/(\alpha - 1)$ to the RDP value. It costs most in between, a factor 1.57 at $\alpha = 8$, near where DP-SGD's best orders lie for $k \approx 100$ to 1000 (8.8 and 7.8 in Table 31.9).

31.8.6. Check yourself

- Why is the (ε, δ) amplification of Equation 31.5 tight in the worst case, when no RDP formula is? *(An (ε, δ) guarantee is one number per event, and randomized response attains the worst case of the mixture lemma for every ε. An RDP curve constrains infinitely many orders at once, and different mechanisms with the same curve can subsample differently.)*
- In Table 31.10, why is the ratio 1.00 at $\alpha = 2$? *(For $\alpha = 2$ the sum over $j \geq 3$ is empty, so the factor 3 never enters, and the remaining terms are exact for the Gaussian.)*

31.9. Code: check every number by computation

The script code/subsampling.py computes every number in this chapter and draws every figure. The amplification formulas are small functions. The RDP of the subsampled Gaussian is computed by the binomial sum for integer orders and by integration on a grid for the rest, and the two are compared. The comparison of batch samplers reproduces checks from Chua et al. (2024) and Lebeda et al. (2024). From the repository root:

```
uv run python code/part04-privacy-accounting/subsampling.py
```

Its public functions are used by later chapters: `amplify_eps_delta(eps, delta, q)`, `subsampled_gaussian_rdp(alpha, q, sigma)`, `subsampled_gaussian_rdp_curve(q, sigma, orders)` (cached per (q, σ)) and `dpsgd_eps_rdp(q, sigma, steps, delta, conversion)`.

Output (seed 0), summarized in Table 31.11:

Table 31.11.: The script's output with the default seed.

Experiment	Result
Mixture lemma, 20,000 random triples	Largest ratio of the left side's excess to $\eta\delta$: 1.0000
Randomized response, $\varepsilon = 1$, $q = 0.1$	Exact $\varepsilon' = 0.1586$; formula 0.1586
Replace-one under Poisson, $\varepsilon = 1$, $q = 0.01$	Group-privacy bound 0.06193 (ratio 3.64 to add/remove's 0.01704); exact pure-DP value 0.02338 (ratio 1.37)
One epoch, 100 batches, $\sigma = 1$	Poisson ≤ 1.21; shuffled ≥ 4.06; deterministic 4.38
Chua et al. check, $\sigma = 0.4$, 10,000 batches	Shuffled $\delta(4) \geq 0.226$; deterministic 0.244
Lebeda et al. check, $q = 0.001$, $\sigma = 0.8$, 10,000 steps	RDP: Poisson 1.38, fixed-size 15.30
Subsampled Gaussian RDP, $q = 0.01$, $\sigma = 1$	Binomial and integral agree to seven digits; reverse direction always smaller (Table 31.7)
Small-q breakdown, $q = 0.01$	Exact first exceeds twice the leading term at $\alpha = 8.9$ ($\sigma = 1$) and 147.4 ($\sigma = 4$); $2\sigma^2 \ln(1/q) = 9.2$ and 147.4
Abadi et al. example, $\sigma = 4$	Mironov 1.2586 at $\alpha = 20$; improved 1.0355 at $\alpha = 17$
Running example, $k = 10{,}000$	RDP improved 6.71 at $\alpha = 4.1$; Mironov 7.43; fixed-size batches 47.42; RDP beats per-step basic composition from $k = 4$

tests/test_subsampling.py checks that the output still matches.

What to take from it. Three things. First, the numerical integral is trustworthy: it matches the exact binomial sum at every integer order checked, from 10^{-4} to 10^{1}, so the fractional orders of the grid can use it. Second, published numbers reproduce: Abadi et al.'s 1.26, Chua et al.'s 0.226 and the direction of Lebeda et al.'s gap all come out of a few dozen lines. Third, the sampling scheme moves ε more than the accountant does. Switching from Mironov's to the improved conversion saves about a tenth at 10,000 steps (7.43 to 6.71), while accounting for the batch sampler you actually run can multiply ε by seven ($47.42/6.71 \approx 7.1$).

31.10. References

- Kasiviswanathan, Lee, Nissim, Raskhodnikova & Smith, "What Can We Learn Privately?" (FOCS 2008; SIAM Journal on Computing 2011). The first use of sampling to amplify privacy, in private learning. Read it for the origin of the idea; the bound there is not tight.
- Li, Qardaji & Su, "On Sampling, Anonymization, and Differential Privacy: Or, k-Anonymization Meets Differential Privacy" (ACM Symposium on Information, Computer

and Communications Security, ASIACCS 2012). The first proof of the tight (ε, δ) amplification for Poisson sampling.

- Balle, Barthe & Gaboardi, "Privacy Amplification by Subsampling: Tight Analyses via Couplings and Divergences" (NeurIPS 2018). Advanced joint convexity (Theorem 2), the tight (ε, δ) amplification for Poisson sampling (Theorem 8), fixed-size sampling with replace-one neighbours (Theorem 9) and sampling with replacement, and tightness via randomized response (Theorem 13). The reference for Section 31.3 and Section 31.4.
- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016). DP-SGD and the moments accountant; Lemma 3 is the small-q bound of Section 31.6. Read it to see where the practice started and which assumptions it made, including Poisson sampling.
- van Erven & Harremoës, "Rényi Divergence and Kullback-Leibler Divergence" (IEEE Transactions on Information Theory, 2014). The properties of the Rényi divergence, including joint quasi-convexity (Theorem 13), used in step 2 of Section 31.5.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The reduction to one dimension (Theorem 4), the dominance of the first direction (Theorem 5 and Corollary 7), and the numerically stable computation of A_α (Section 3.3). The reference for Section 31.5 and the basis of most RDP accountants.
- Wang, Balle & Kasiviswanathan, "Subsampled Rényi Differential Privacy and Analytical Moments Accountant" (AISTATS 2019). General RDP amplification for sampling without replacement (Theorem 9) and a lower bound (Proposition 11).
- Zhu & Wang, "Poisson Subsampled Rényi Differential Privacy" (ICML 2019). General RDP amplification for Poisson sampling (Theorem 5), the matching lower bound (Theorem 6), and exactness under a condition that the Gaussian and Laplace mechanisms satisfy (Theorem 8 and Proposition 10).
- Steinke, "Composition of Differential Privacy & Privacy Amplification by Subsampling" (arXiv 2022). A self-contained survey chapter with clean proofs of composition and of subsampling; the best second reading after this chapter.
- Chua, Ghazi, Kamath, Kumar, Manurangsi, Sinha & Zhang, "How Private are DP-SGD Implementations?" (ICML 2024). Shows that shuffle-and-partition batching can be much less private than the Poisson accountant reports, with the largest-batch-output test used in Table 31.5.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (arXiv:2405.20769, 2024; IEEE SaTML 2025). Fixed-size against Poisson sampling (the factor two in the shift, and $\varepsilon \approx 1$ against $\varepsilon > 10$), and worst-case datasets that fail under composition.
- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022). Dominating pairs for subsampled mechanisms (Theorem 11), used by Lebeda et al. and by numerical accounting.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). The improved conversion used in Equation 31.16 (Theorem 21).

31.11. Cheat sheet

q is the sampling rate; $P_0 = \mathcal{N}(0, \sigma^2)$ and $P_1 = \mathcal{N}(1, \sigma^2)$ are a step's output without and with the record, and η is the mixing weight of the mixture lemma. Table 31.12 collects the results.

Table 31.12.: Every result of the chapter.

Result	Formula	Conditions	Tight?
Mixture lemma	$(1-\eta)R + \eta P \le e^{\varepsilon'}[(1-\eta)R + \eta P'] + \eta\delta$, $e^{\varepsilon'} = 1 + \eta(e^{\varepsilon} - 1)$	$P \le e^{\varepsilon} P' + \delta$ and $P \le e^{\varepsilon} R + \delta$ on every event	Yes
Poisson, add/remove	$\varepsilon' = \ln(1 + q(e^{\varepsilon} - 1))$, $\delta' = q\delta$	Base (ε, δ) for add/remove	Yes, per step
Fixed size m of n, replace-one	Same with $q = m/n$	Base (ε, δ) for replace-one	Yes, per step
Poisson, replace-one	$\varepsilon' = \ln(1 + q(e^{2\varepsilon} - 1))$, $\delta' = q(1 + e^{\varepsilon})\delta$	Group privacy on the base	No: an upper bound. For pure DP the exact value is Equation 31.8, about twice the add/remove cost for small ε
Fixed size, add/remove	Account the Poisson pair with noise multiplier $\sigma/2$	Clipped Gaussian sums (Lebeda et al. 2024)	Dominating pair, each relation separately
Shuffle and partition	Max test: $1 - \Phi(\frac{\tau-2}{\sigma})\Phi(\frac{\tau}{\sigma})^{b-1}$ against $1 - \Phi(\frac{\tau-1}{\sigma})\Phi(\frac{\tau}{\sigma})^{b-1}$	b batches, zero-out	A lower bound on $\delta(\varepsilon)$
Subsampled Gaussian RDP	$\varepsilon(\alpha) = \frac{1}{\alpha-1} \ln \mathbb{E}_{P_0}[(1 - q + qP_1/P_0)^{\alpha}]$	Poisson, add/remove, sensitivity 1	Exact
Integer orders	$A_\alpha = \sum_j \binom{\alpha}{j}(1-q)^{\alpha-j} q^j e^{(j^2-j)/(2\sigma^2)}$	Integer α	Exact
Small q	$\varepsilon(\alpha) \approx \alpha q^2 (e^{1/\sigma^2} - 1)/2$	$\alpha \lesssim 2\sigma^2 \ln(1/q)$ (Equation 31.15)	Leading term; within 8% up to $\sigma^2 \ln(1/q)$ here
Abadi et al.	$\varepsilon(\alpha) \lesssim q^2 \alpha / ((1-q)\sigma^2)$	$\alpha \le 1 + \sigma^2 \ln \frac{1}{q\sigma}$, $q < \frac{1}{16\sigma}$, $\sigma \ge 1$	About twice the exact value for large σ ($2.0202/1.7181 = 1.18$ times at $\sigma = 1$, $\alpha = 2$); below it outside its order range
High orders	$\varepsilon(\alpha) \ge \frac{\alpha \ln q}{\alpha-1} + \frac{\alpha}{2\sigma^2}$ (Equation 29.21)	Any α	Nearly exact once $\alpha \gtrsim 2\sigma^2 \ln(1/q)$; negative, so useless, well below it

Result	Formula	Conditions	Tight?
DP-SGD, RDP accountant	Equation 31.16: $k\,\varepsilon(\alpha)$, converted at the best order	Poisson, add/remove, adaptive steps	Upper bound; 6.71 for the running example
Any mechanism, Poisson	Equation 31.17	Integer α	Within a factor 3 inside the logarithm

Rules of thumb

- Sampling at rate q multiplies a small ε by q, but takes at most $\ln(1/q)$ off a large one.
- In RDP the per-step cost is quadratic in q: about $\alpha q^2/(2\sigma^2)$ for large σ, valid up to $\alpha \approx 2\sigma^2 \ln(1/q)$, and it breaks abruptly there.
- Never compose per-step (ε, δ) for DP-SGD. Compose RDP curves, or privacy loss distributions, and convert once.
- Account for the sampler you run. Poisson pairs with add/remove. Fixed-size batches need the base mechanism's replace-one guarantee, which for clipped Gaussian sums means half the noise multiplier. Shuffle and partition has no simple amplification theorem, and the Poisson number can be far too small for it.
- For the Gaussian, use the exact binomial sum and the numerical integral; keep the general bounds for other mechanisms.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

32. Numerical accounting with privacy loss distributions

32.1. Why this matters

You trained a model with differential privacy (DP): 10,000 steps of stochastic gradient descent (SGD), each on a Poisson sample of the data with rate $q = 0.01$, each with clipped gradients and Gaussian noise of multiplier $\sigma = 1$. This is DP-SGD. You need one number: the smallest ε such that the whole run is $(\varepsilon, 10^{-5})$-DP, which means that for any two datasets that differ in one person's record, the probability of any set of outcomes of the run changes by at most a factor e^{ε}, plus a slack of 10^{-5}. Rényi differential privacy (RDP), from Rényi DP, gives an upper bound: it tracks one Rényi divergence per order and adds them up over the steps, but converting that Rényi curve to (ε, δ) loses something you cannot see. The Gaussian central limit theorem (CLT) of f-DP and Gaussian DP says that many composed steps look like one Gaussian mechanism, which gives an approximation, but not a bound. Neither tells you how far it is from the truth. This chapter computes the truth itself, to any precision you are willing to pay for, together with a lower and an upper bound that bracket it.

32.1.1. The one idea: δ is an average over the privacy loss, and composition adds losses

Let P be the distribution of the output when the run uses your dataset D, and Q the distribution when it uses the neighbour D'. Composition measured how much one output y gives the two datasets away by the log of the ratio of its two probabilities, the **privacy loss** $L(y) = \ln \frac{P(y)}{Q(y)}$. It is positive when y is more likely under D, negative when it is more likely under D', and zero when y says nothing. With the output drawn as $Y \sim P$, the loss $L(Y)$ is a random variable, the privacy loss random variable (PRV). Two facts make it the right object to track.

The first is that the guarantee $\delta(\varepsilon)$, the smallest δ that works for a given ε, depends on the pair (P, Q) only through the distribution of $L(Y)$. Each output contributes to $\delta(\varepsilon)$ an amount that depends only on its own loss: nothing if its loss is at most ε, a fraction $(1 - e^{\varepsilon - \ell})_+$ of its probability if its loss ℓ is larger. So $\delta(\varepsilon)$ is the average of that fixed function of the loss. We call the distribution of $L(Y)$ the **privacy loss distribution (PLD)**.

The second is that the privacy loss of a composition is the sum of the per-step losses, because the probability of a sequence of independent outputs is a product and the logarithm turns the product into a sum. For independently run steps the terms are independent, and the distribution of a sum of independent variables is the convolution of their distributions. So the PLD of k steps is the k-fold convolution of the per-step PLDs (Section 32.4 handles steps chosen adaptively).

Everything in this chapter follows from these two facts. Discretize the per-step loss on a grid of width h, convolve with a fast Fourier transform (FFT), and read $\delta(\varepsilon)$ off the result. Since a

larger loss never contributes less to δ, rounding moves δ in a known direction. If you **round each loss up** to the grid, the δ you compute is an upper bound on the true δ for every composition. If you **round down**, it is a lower bound. The true value is sandwiched, and the width of the sandwich is known in advance: each step's two roundings are at most h apart, so after k steps the two totals are at most kh apart, which makes the sandwich at most kh wide in ε, and exactly kh for a continuous PLD without truncation.

Table 32.1 lists the results and where each is used.

Table 32.1.: The results of this chapter and where each is used.

Result	Where it is used
$\delta(\varepsilon) = \mathbb{E}_{Y\sim P}[(1 - e^{\varepsilon - L(Y)})_+]$ (Equation 32.3)	Every accountant in this chapter; reading (ε, δ) off any PLD
The Gaussian PLD $\mathcal{N}(\mu^2/2, \mu^2)$	Checking the code; the Gaussian mechanism; Gaussian DP
Composition is convolution (Equation 32.5), dominating pairs	Accounting for k steps, adaptive or not
Pessimistic and optimistic rounding (Equation 32.6)	Turning a numerical answer into a certified upper bound and a lower bound
FFT convolution, padding, repeated squaring	Fast accountants: the PLD accountant, the PRV accountant
PLD of the Poisson-subsampled Gaussian, two directions	DP-SGD accounting
The running example	Comparing PLD against RDP and the CLT in Choosing an accountant

Conventions. In this chapter q is the sampling rate. Since the lowercase letter is taken, the densities of the two output distributions are written with the same letter as the distribution, $P(y)$ and $Q(y)$. σ is the noise multiplier and the sensitivity is 1: one record moves the quantity being noised by at most 1. A **PLD** ω is the law of $L(Y)$ under $Y \sim P$. It may have an atom at $+\infty$, which is the P-mass of the outputs where $Q(y) = 0$: those outputs are impossible under D', so seeing one proves the dataset was D. Φ is the standard normal distribution function, $\Phi(x) = \Pr[Z \le x]$ for $Z \sim \mathcal{N}(0, 1)$. Logs are natural. The running example is DP-SGD with $q = 0.01$, $\sigma = 1$, $\delta = 10^{-5}$ and up to $k = 10{,}000$ steps. The RDP accountant of Amplification by subsampling and the CLT approximation of Gaussian differential privacy (GDP) serve as comparisons.

32.2. Toolkit: three facts the derivations reuse

32.2.1. Fact 1 — Change of measure: $Q(y) = e^{-L(y)}P(y)$

You often need a probability under Q but only know the distribution of the loss under P. The way across is to reweight: an output's Q-probability is its P-probability times the ratio $Q(y)/P(y)$, and that ratio is $e^{-L(y)}$ by the definition of the loss. Wherever $P(y) > 0$ and $Q(y) > 0$, $Q(y) = P(y)\,\frac{Q(y)}{P(y)} = e^{-L(y)}P(y)$. Summing any function f of the loss against both sides gives, for any f,

$$\mathbb{E}_Q[f(L)\,\mathbf{1}\{P > 0\}] = \mathbb{E}_P[f(L)\,e^{-L}]. \tag{32.1}$$

The indicator $\mathbf{1}\{P > 0\}$ is there because the reweighting only reaches outputs that P can produce; the outputs with $P(y) = 0$ are invisible from the P side. With $f = 1$ the left side is the total Q-mass on outputs that P can produce: $\mathbb{E}_P[e^{-L}] = Q(P > 0) \le 1$, with equality when Q puts no mass where P puts none. This is Fact 1 of Divergences, read with the likelihood ratio $r = e^L$.

Two things follow that the chapter uses. The law of L under Q is determined by the PLD, because every Q-probability of a loss event is a reweighted P-probability. And a distribution ω can only be a PLD if $\mathbb{E}_\omega[e^{-L}] \le 1$.

32.2.2. Fact 2 — Monotone coupling

If you move every value of a random variable up, any non-decreasing function of it can only go up too, and so can its average. Formally, let g be non-decreasing. If two random variables, built on the same random draw, satisfy $X \le X'$ for every outcome, then $\mathbb{E}[g(X)] \le \mathbb{E}[g(X')]$. This is just $g(X) \le g(X')$ outcome by outcome, averaged. Building two variables from one draw like this, so that they can be compared outcome by outcome, is called a **coupling**. The chapter uses it to show that moving privacy loss upward can only increase δ (Section 32.5).

32.2.3. Fact 3 — The discrete convolution theorem

Convolving two sequences directly means summing $a_i b_j$ over every pair (i, j) with the same $i + j$, which costs a number of operations proportional to the product of their lengths. The Fourier transform turns this into a pointwise product. The reason is that the transform attaches to index i the power ζ^i of a fixed complex number, and powers multiply by adding exponents, $\zeta^i\zeta^j = \zeta^{i+j}$: indices add exactly the way losses add. For sequences $a_0, \dots, a_{N-1}$, the discrete Fourier transform of length N is $\hat{a}_m = \sum_{i=0}^{N-1} a_i\,\zeta^{im}$ with $\zeta = e^{-2\pi\mathrm{i}/N}$, an N-th root of unity. Multiplying two transforms and grouping the terms by $i + j$ gives, since $\zeta^N = 1$ means exponents only matter modulo N,

$$\hat{a}_m\,\hat{b}_m = \sum_{i,j} a_i b_j\,\zeta^{(i+j)m} = \sum_{n=0}^{N-1} \Big(\sum_{i+j \equiv n \bmod N} a_i b_j \Big) \zeta^{nm}. \tag{32.2}$$

The bracket is the coefficient at index n, so the inverse transform of $\hat{a}\hat{b}$ is the **circular** convolution: index sums are taken modulo N, and a sum that reaches N or more folds back to the start. To get the ordinary convolution you make the window long enough that nothing folds back. If a has n_a nonzero entries at the start, b has n_b, and $N \ge n_a + n_b - 1$, then $i + j \le n_a + n_b - 2 < N$ always, no index wraps around, and the circular convolution equals the ordinary one. The FFT

computes such a transform in $O(N \log N)$ operations instead of $O(N^2)$. The chapter uses it to convolve PLDs (Section 32.6).

32.3. The privacy loss distribution

32.3.1. Context

The guarantee you report is $\delta(\varepsilon) = E_{e^\varepsilon}(P\|Q)$, the **hockey-stick divergence** between the output distributions on two neighbouring datasets (see Divergences and Definition). It is the smallest δ for which $P(S) \le e^\varepsilon Q(S) + \delta$ holds for every set S of outputs. The largest gap $P(S) - e^\varepsilon Q(S)$ comes from the set that holds exactly the outputs with $P(y) > e^\varepsilon Q(y)$, since each of them adds a positive amount and any other output would subtract, so the divergence is the sum $\sum_y \big(P(y) - e^\varepsilon Q(y)\big)_+$, where $(x)_+ = \max\{x, 0\}$. That is a sum over outputs, and the output space of a training run is enormous. This section shows that you never need the output space: the one-dimensional distribution of the privacy loss carries all of $\delta(\varepsilon)$.

32.3.2. Statement

Each output adds to δ only if its loss exceeds ε, and then it adds the fraction $1 - e^{\varepsilon - \ell}$ of its probability, which grows from 0 at $\ell = \varepsilon$ towards 1 for a very large loss. So δ is an average of that weight over the PLD. Equivalently, it is the chance under P that the loss exceeds ε, minus e^ε times the same chance under Q: the second form is the hockey-stick gap of the best set $S = \{L > \varepsilon\}$. Let ω be the PLD of (P, Q), the law of $L(Y)$ with $Y \sim P$, including its atom $\omega(\{+\infty\})$. Then for every ε,

$$\delta(\varepsilon) = \mathbb{E}_{L\sim\omega}\big[(1 - e^{\varepsilon - L})_+\big] = P[L > \varepsilon] - e^\varepsilon\, Q[L > \varepsilon], \tag{32.3}$$

with the convention $(1 - e^{\varepsilon - \infty})_+ = 1$: an output impossible under D' counts fully. The right side also needs the law of L under Q, but that is no extra information. By Fact 1 it is the PLD reweighted by $e^{-\ell}$, namely $e^{-\ell}\,\omega(d\ell)$ on finite ℓ, plus an atom at $-\infty$ of size $1 - \mathbb{E}_\omega[e^{-L}]$ for the Q-mass that sits where P is zero. So ω alone determines both sides.

For the **Gaussian pair** $P = \mathcal{N}(0, \sigma^2)$, $Q = \mathcal{N}(s, \sigma^2)$, the two outputs of a Gaussian mechanism whose query moves by s, write $\mu = s/\sigma$ for the shift measured in noise standard deviations. The privacy loss turns out to be a linear function of the output, so it is itself Gaussian, with mean $\mu^2/2$ and variance μ^2. Plugging its two tails into Equation 32.3 gives δ in closed form:

$$L \sim \mathcal{N}\big(\tfrac{\mu^2}{2}, \mu^2\big) \text{ under } P, \qquad \delta(\varepsilon) = \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^\varepsilon\, \Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big). \tag{32.4}$$

32.3.3. Derivation

δ as an average over the PLD. The plan is to start from the hockey-stick sum, factor each output's probability out of its term so that what remains depends only on its loss, and then recognize an average.

1. **Write the hockey-stick divergence as a sum over outputs.** As the Context explains (Fact 3 of Divergences), the best set of outputs is the one where $P > e^{\varepsilon}Q$, so $\delta(\varepsilon) = \sum_y \big(P(y) - e^{\varepsilon}Q(y)\big)_+$.
2. **Sort the outputs into three kinds, because the loss behaves differently on each.** Where $P(y) = 0$ the term is $(-e^{\varepsilon}Q(y))_+ = 0$, so these outputs drop out. Where $Q(y) = 0 < P(y)$ the term is $P(y)$, and $L(y) = +\infty$. Elsewhere both are positive and the loss is finite.
3. **Factor out $P(y)$ on the third kind,** using $Q(y)/P(y) = e^{-L(y)}$: $P(y)(1 - e^{\varepsilon}\frac{Q(y)}{P(y)})_+ = P(y)(1 - e^{\varepsilon - L(y)})_+$.
4. **Recognize an expectation under P.** A sum of $P(y)$ times something is the average of that something with $Y \sim P$. Adding up the second and third kinds gives $\mathbb{E}_P[(1 - e^{\varepsilon - L(Y)})_+]$, where an infinite loss gets the value 1, as step 2 requires. The thing being averaged depends on y only through $L(y)$, so the expectation is an average over ω. This is the first form of Equation 32.3.
5. **Split the positive part into two terms.** When $L > \varepsilon$ the bracket $1 - e^{\varepsilon - L}$ is positive and the positive part keeps it; when $L \le \varepsilon$ it is zero. So $(1 - e^{\varepsilon - L})_+ = \mathbf{1}\{L > \varepsilon\} - e^{\varepsilon}e^{-L}\mathbf{1}\{L > \varepsilon\}$. The first term averages to $P[L > \varepsilon]$.
6. **Change measure in the second term.** Fact 1 with $f = \mathbf{1}\{\cdot > \varepsilon\}$ turns the P-average reweighted by e^{-L} into a Q-probability: $\mathbb{E}_P[e^{-L}\mathbf{1}\{L > \varepsilon\}] = Q[L > \varepsilon]$. The outputs where $P = 0$, which Fact 1 cannot reach, have $L = -\infty$ and are not in the event anyway. This gives the second form of Equation 32.3.
7. **Find the law of L under Q the same way.** Fact 1 with $f = \mathbf{1}\{L \in A\}$ gives $Q[L \in A] = \int_A e^{-\ell}\omega(d\ell)$ for sets A of finite losses. The remaining Q-mass sits where $P = 0$, at $L = -\infty$.

The Gaussian PLD. The point is that for two Gaussians with the same variance the log-ratio of the densities is linear in the output, because the quadratic terms cancel.

1. **The loss is linear in y.** The normalizing constants are the same and cancel: $L(y) = \ln \frac{e^{-y^2/2\sigma^2}}{e^{-(y-s)^2/2\sigma^2}} = \frac{(y-s)^2 - y^2}{2\sigma^2} = \frac{s^2 - 2sy}{2\sigma^2}$.
2. **A linear function of a Gaussian is Gaussian.** With $Y \sim \mathcal{N}(0, \sigma^2)$, the mean of L is the constant term $\frac{s^2}{2\sigma^2} = \frac{\mu^2}{2}$, and the variance is the squared slope times the variance of Y, $\frac{4s^2}{4\sigma^4}\sigma^2 = \mu^2$.
3. **Under Q, $Y \sim \mathcal{N}(s, \sigma^2)$** has mean s instead of 0, which moves the mean of L by the slope times s, $-\frac{2s \cdot s}{2\sigma^2} = -\mu^2$, so $L \sim \mathcal{N}(-\frac{\mu^2}{2}, \mu^2)$. Step 7 above predicts the same. Multiplying the density of $\mathcal{N}(m, v)$ by $e^{-\ell}$ and completing the square, $-\frac{(\ell - m)^2}{2v} - \ell = -\frac{(\ell - (m - v))^2}{2v} + \text{const}$, gives $\mathcal{N}(m - v, v)$; here $\frac{\mu^2}{2} - \mu^2 = -\frac{\mu^2}{2}$.
4. **Read off the tails.** For $L \sim \mathcal{N}(m, \mu^2)$, standardizing gives $\Pr[L > \varepsilon] = \Pr\left[Z > \frac{\varepsilon - m}{\mu}\right] = \Phi(\frac{m - \varepsilon}{\mu})$, because the standard normal is symmetric about 0. So $P[L > \varepsilon] = \Phi(\frac{\mu^2/2 - \varepsilon}{\mu}) = \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu})$, and $Q[L > \varepsilon] = \Phi(\frac{-\mu^2/2 - \varepsilon}{\mu}) = \Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$. Insert them in Equation 32.3. This is the exact $\delta(\varepsilon)$ of the Gaussian mechanism, and the (ε, δ) curve of μ-GDP in f-DP and Gaussian DP, where a mechanism is μ-GDP if telling its two outputs apart is never easier than telling $\mathcal{N}(0, 1)$ from $\mathcal{N}(\mu, 1)$.

32.3.4. Interpretation

The PLD is a complete summary for (ε, δ). Two pairs with the same PLD have the same $\delta(\varepsilon)$ at every ε, whatever their output spaces, because Equation 32.3 reads δ off the PLD and nothing else. A DP-SGD run with millions of parameters and a one-dimensional Gaussian pair can have the same PLD.

Not every distribution is a PLD, though. Fact 1 requires $\mathbb{E}_\omega[e^{-L}] \le 1$, since that average is the Q-mass where P is positive. The converse holds too: any ω with $\mathbb{E}_\omega[e^{-L}] \le 1$ is the PLD of some pair, and you can build the pair directly. Take $P = \omega$ on the real line, except that its atom at $+\infty$, if any, goes on an extra point y_+ where Q is zero. Take $Q(d\ell) = e^{-\ell}\omega(d\ell)$ on the finite losses, which has total mass at most 1 by assumption, and put the missing Q-mass $1 - \mathbb{E}_\omega[e^{-L}]$ on a second extra point y_- where P is zero. Then $\ln \frac{P}{Q}$ is ℓ at each finite ℓ and $+\infty$ at y_+, so the PLD is ω. Section 32.5 uses this to show that a rounded PLD still belongs to a real pair of distributions.

The weight $(1 - e^{\varepsilon - \ell})_+$ is non-decreasing in ℓ: it is 0 up to $\ell = \varepsilon$ and rises to 1. So $\delta(\varepsilon)$ is the PLD's upper tail beyond ε, down-weighted near ε (Figure 32.1). Because the weight is at most 1, $\delta(\varepsilon) \le P[L > \varepsilon]$, and because losses just above ε get a weight close to 0, that tail bound is loose (Exercise 2 in the exercise book).

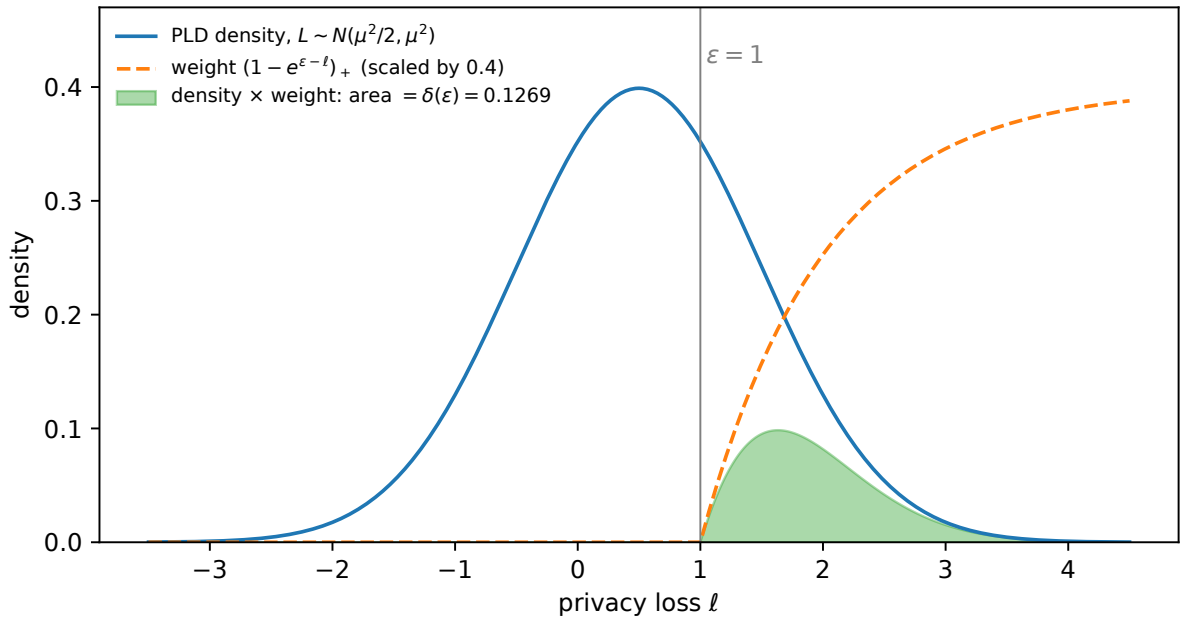


Figure 32.1.: The Gaussian PLD for $\mu = 1$ (blue), the weight $(1 - e^{\varepsilon - \ell})_+$ at $\varepsilon = 1$ (orange, scaled by 0.4 to fit), and their product (green). The green area is $\delta(1) = 0.1269$.

32.3.5. Example: the Gaussian mechanism with $\sigma = 1$

With sensitivity 1 and $\sigma = 1$, the shift is one noise standard deviation, $\mu = 1/1 = 1$, and the PLD is $\mathcal{N}(\frac{1}{2}, 1) = \mathcal{N}(0.5, 1)$. From a million samples of Y, the script finds a mean loss of 0.4990, a variance of 1.0013 and $\mathbb{E}[e^{-L}] = 1.0015$, against the exact 0.5, 1 and 1. The green area of

Figure 32.1 comes from Equation 32.4 at $\varepsilon = 1$: $\Phi(\frac{1}{2} - 1) - e\,\Phi(-\frac{1}{2} - 1) = \Phi(-0.5) - e\,\Phi(-1.5) = 0.3085 - 2.7183 \times 0.0668 = 0.3085 - 0.1816 = 0.1269$. At $\varepsilon = 0$ the second term has no factor e^{ε} and the formula is $\Phi(0.5) - \Phi(-0.5) = 0.6915 - 0.3085 = 0.3829$. Table 32.2 checks Equation 32.4 against numerical integration of $(1 - e^{\varepsilon - \ell})_+$ over the PLD density and against Monte Carlo.

Table 32.2.: $\delta(\varepsilon)$ of the Gaussian pair with $\mu = 1$, computed three ways.

ε	Closed form Equation 32.4	Quadrature over the PLD	Monte Carlo (10^6 draws)
0	0.382925	0.382925	0.382644
0.5	0.238422	0.238422	0.238748
1	0.126937	0.126937	0.126967
2	0.020924	0.020924	0.020922

The quadrature agrees to all six digits and Monte Carlo to within its sampling error. Since δ falls as ε grows, you can also go the other way and solve $\delta(\varepsilon) = 10^{-5}$ for ε: the mechanism is $(4.3772, 10^{-5})$-DP.

For a finite pair the two forms of Equation 32.3 can be checked by hand. Take $P = (0.1, 0.2, 0.3, 0.4)$ and $Q = (0.4, 0.3, 0.2, 0.1)$ at $\varepsilon = 0.5$, where $e^{0.5} = 1.6487$. Only the last output has $P > e^{0.5}Q$, so the sum is $\sum (P - e^{0.5}Q)_+ = 0.4 - 1.6487 \times 0.1 = 0.2351$. The losses are $\ln \frac{0.1}{0.4}$, $\ln \frac{0.2}{0.3}$, $\ln \frac{0.3}{0.2} = 0.405$ and $\ln \frac{0.4}{0.1} = 1.386$, so only the last exceeds 0.5, and $P[L > 0.5] - e^{0.5}Q[L > 0.5] = 0.4 - 1.6487 \times 0.1 = 0.2351$ as well.

32.3.6. Check yourself

- Why does an output with $Q(y) = 0$ count fully in δ, whatever ε is? *(Seeing it proves the dataset was D, not D'. No e^{ε} factor can cover a ratio of $P(y)/0$, so its whole probability goes into δ.)*
- A Gaussian mechanism with noise standard deviation 2 on a count (sensitivity 1), and a 1000-dimensional Gaussian mechanism with noise standard deviation 4 on a query of ℓ_2 sensitivity 2: do they have the same (ε, δ) curve? *(Yes. Both shift by half a noise standard deviation, $\mu = 1/2 = 2/4 = 0.5$, so both have PLD $\mathcal{N}(\frac{0.25}{2}, 0.25) = \mathcal{N}(0.125, 0.25)$, and by Equation 32.3 δ depends only on the PLD.)*

32.4. Composition is convolution

32.4.1. Context

DP-SGD releases k noisy gradients. The output is the tuple $(y_1, \ldots, y_k)$, and you want its $\delta(\varepsilon)$. The bounds of Composition (basic, advanced, and the optimal composition theorem of Kairouz, Oh and Viswanath, which is the best bound possible for steps known only by their $(\varepsilon_i, \delta_i)$) start from per-step $(\varepsilon_i, \delta_i)$ pairs, and so throw away the shape of each step's PLD. With PLDs, composition is exact.

32.4.2. Statement

Run k steps independently. The probability of the whole tuple of outputs is the product of the per-step probabilities, so the log-ratio of the whole tuple is the sum of the per-step log-ratios: the total privacy loss is the sum of the step losses. Those step losses are independent, and the distribution of a sum of independent variables is the **convolution** of their distributions. For independent steps with pairs (P_i, Q_i) and PLDs ω_i, the product pair $(P_1 \times \cdots \times P_k,\ Q_1 \times \cdots \times Q_k)$ therefore has PLD

$$\omega_{1..k} = \omega_1 * \omega_2 * \cdots * \omega_k, \qquad \omega^{*k} \text{ for } k \text{ identical steps,} \tag{32.5}$$

where $*$ is convolution of distributions (the law of a sum of independent variables), and the atom at $+\infty$ absorbs any sum that contains $+\infty$: once one step gives the dataset away completely, the whole run does. For k Gaussian steps, means and variances add, so $\mathcal{N}(\frac{\mu^2}{2}, \mu^2)^{*k} = \mathcal{N}(\frac{k\mu^2}{2}, k\mu^2)$, which is the PLD of a single Gaussian with $\mu_k = \sqrt{k}\,\mu$.

Adaptive composition. In DP-SGD each step's gradient depends on the model, which depends on the earlier outputs, so the steps are not independent. The way around this is to replace each step by a fixed pair that is at least as easy to tell apart, whatever happened before. To compare pairs at every level of strictness, use the hockey-stick divergence with a general weight γ in place of e^ε, $E_\gamma(P\|Q) = \sum_y \big(P(y) - \gamma Q(y)\big)_+$. A pair (P, Q) **dominates** a mechanism M if $E_\gamma(M(D)\|M(D')) \le E_\gamma(P\|Q)$ for all neighbours $D \simeq D'$ and all $\gamma \ge 0$ (Zhu, Dong & Wang 2022, Definition 7). If (P_i, Q_i) dominates step i for every value of the earlier outputs, then $(P_1 \times \cdots \times P_k, Q_1 \times \cdots \times Q_k)$ dominates the adaptive composition (their Theorem 10). So the convolution of the dominating PLDs gives a valid $\delta(\varepsilon)$ for the adaptive run.

32.4.3. Derivation

The derivation follows the probability of the tuple from a product, to a sum of losses, to a convolution.

1. **Densities multiply.** For independent steps the probability of a tuple is the product of the probabilities of its entries: $P(y_1, \dots, y_k) = \prod_i P_i(y_i)$, and likewise for Q.
2. **Losses add.** The ratio of two products is the product of the ratios, and the logarithm of a product is the sum of logarithms: $L(y_1, \dots, y_k) = \sum_i L_i(y_i)$, with $L_i = \ln(P_i/Q_i)$.
3. **The terms are independent.** Under P, the Y_i are independent with $Y_i \sim P_i$. Each $L_i(Y_i)$ is a function of its own Y_i only, so the $L_i(Y_i)$ are independent too, with laws ω_i.
4. **The law of a sum of independent variables is the convolution of their laws.** For two discrete variables, the chance that the sum is ℓ adds up, over all ways of splitting $\ell = a + b$, the chance of a times the chance of b, and that is what convolution computes. This is Equation 32.5.
5. **Gaussian check.** A sum of independent Gaussians is Gaussian, and means and variances add: $k\frac{\mu^2}{2}$ and $k\mu^2$. This is $\mathcal{N}(\frac{\mu_k^2}{2}, \mu_k^2)$ with $\mu_k^2 = k\mu^2$.
6. **Adaptivity breaks step 3, and domination repairs it.** If the second mechanism is chosen after seeing y_1, its pair depends on y_1, so its loss depends on the first output and the losses are no longer independent. The dominating-pair theorem bounds every pair the second mechanism could have by one fixed pair, and shows that the hockey-stick curves of the product of the fixed pairs still lie above those of the adaptive run. Its proof is in

Zhu, Dong & Wang (2022, Theorem 10). It uses E_γ for all $\gamma \geq 0$, not only $\gamma \geq 1$ (which corresponds to $\varepsilon \geq 0$), which is why Definition 7 asks for all $\gamma \geq 0$.

32.4.4. Interpretation

Equation 32.5 has no slack at all. It is exact, so the only errors in a PLD accountant are the ones you introduce to compute the convolution: discretization (Section 32.5) and floating point (Section 32.6).

It also shows why the CLT is plausible. A k-fold convolution is the law of a sum of k independent terms, and the ordinary central limit theorem says that such a sum of anything with finite variance looks Gaussian for large k. That is the GDP central limit theorem of f-DP and Gaussian DP. The PLD accountant does not need that approximation; it computes the convolution.

The dominating pair is a choice, and the bound is only as tight as that choice. For the Gaussian mechanism with ℓ_2 sensitivity 1, the pair $\mathcal{N}(0, \sigma^2)$, $\mathcal{N}(1, \sigma^2)$ dominates and is attained by some neighbouring datasets, so the result is tight. For subsampled mechanisms you need one pair per direction (Section 32.7).

32.4.5. Example: ten Gaussian steps

One Gaussian step with $\sigma = 1$ has $\mu = 1$ and PLD $\mathcal{N}(0.5, 1)$. Ten steps add ten means and ten variances, so their PLD is $\mathcal{N}(5, 10)$, which is a single Gaussian with $\mu = \sqrt{10}$. Solving Equation 32.4 with that μ for $\delta = 10^{-5}$, the composition is $(17.8566, 10^{-5})$-DP. For 100 steps, $\mu = \sqrt{100} = 10$ and $\varepsilon = 91.8173$. Section 32.6 recovers both numbers from a discretized PLD.

32.4.6. Check yourself

- You compose a Gaussian step with $\mu = 1$ and one with $\mu = 2$. What is the PLD? *(Means $\frac{\mu^2}{2}$ and variances μ^2 add: $\mathcal{N}(\frac{1}{2} + 2, 1 + 4) = \mathcal{N}(2.5, 5)$, a single Gaussian with $\mu = \sqrt{5}$.)*
- Why is the convolution of PLDs exact while advanced composition is not? *(Advanced composition, and even optimal composition, see each step only through $(\varepsilon_i, \delta_i)$, so they must cover the worst PLD with those values. Convolution uses the actual PLDs.)*

32.5. Discretization: pessimistic and optimistic

32.5.1. Context

A computer cannot convolve continuous distributions. You must put the loss on a finite grid $\{ih : i \in \mathbb{Z}\}$ of spacing h. Any rounding changes δ. The question is whether you can round so that the error has a known sign, for every number of compositions, so that the answer is still a guarantee.

32.5.2. Statement

The weight $(1 - e^{\varepsilon-\ell})_+$ that turns a loss into δ never decreases as the loss grows. So if you move every loss up, δ can only grow, and if you move every loss down, it can only shrink. That gives two ways to round, one for each direction. Given a PLD ω and a grid width h, define two discretizations.

- **Pessimistic** $\omega^{\uparrow}$: move every loss $\ell \in ((i-1)h, ih]$ up to ih. Truncate the top tail by moving its mass (at most a tiny τ) to $+\infty$, and the bottom tail by moving it up to the lowest kept grid point. Every move is upward.
- **Optimistic** $\omega^{\downarrow}$: move every loss $\ell \in [ih, (i+1)h)$ down to ih. Truncate the top tail by moving it down to the highest kept grid point, and drop the bottom tail. Every move is downward or removes mass.

Write $\delta_\omega(\varepsilon)$ for Equation 32.3 evaluated on ω. Rounding each step before convolving keeps the direction of every move in the total, so for every k and every ε,

$$\delta_{(\omega^{\downarrow})^{*k}}(\varepsilon) \;\le\; \delta_{\omega^{*k}}(\varepsilon) \;\le\; \delta_{(\omega^{\uparrow})^{*k}}(\varepsilon), \tag{32.6}$$

and the same holds for convolutions of different PLDs, each rounded the same way.

Two more things hold. First, the upper bound is not just a number but a real guarantee: $\omega^{\uparrow}$ is a valid PLD, $\mathbb{E}_{\omega^{\uparrow}}[e^{-L}] \le 1$, of a pair that dominates (P, Q), so the upper bound also holds for adaptive composition. Second, the sandwich is narrow. Write $\varepsilon^{\uparrow}(\delta)$ and $\varepsilon^{\downarrow}(\delta)$ for the values of ε at which the pessimistic and optimistic curves reach δ. Without truncation the two are at most kh apart, because each step's two roundings differ by at most h: $\varepsilon^{\uparrow}(\delta) \le \varepsilon^{\downarrow}(\delta) + kh$.

32.5.3. Derivation

1. **Build both roundings of one step from the same draw.** Draw $L \sim \omega$ and set $L^{\uparrow} = h\lceil L/h \rceil$ (round up) and $L^{\downarrow} = h\lfloor L/h \rfloor$ (round down). Then $L^{\uparrow}$ has law $\omega^{\uparrow}$, $L^{\downarrow}$ has law $\omega^{\downarrow}$, and, since a number lies between its two roundings and they are at most one grid step apart, $L^{\downarrow} \le L \le L^{\uparrow} \le L^{\downarrow} + h$ for every outcome. This is a coupling in the sense of Fact 2.
2. **Add up k steps.** Draw the k steps independently and apply step 1 to each. Adding the k inequalities gives $\sum L_i^{\downarrow} \le \sum L_i \le \sum L_i^{\uparrow} \le \sum L_i^{\downarrow} + kh$, and each sum has the convolved law of its rounded PLDs (Equation 32.5).
3. **Use that δ grows with the loss.** By Equation 32.3, $\delta(\varepsilon) = \mathbb{E}[g_\varepsilon(\text{total loss})]$ with $g_\varepsilon(\ell) = (1 - e^{\varepsilon-\ell})_+$, a non-decreasing function. Fact 2 applied to the inequalities of step 2 gives Equation 32.6.
4. **Truncation keeps the sign.** The pessimistic truncation moves mass up (to a higher grid point, or to $+\infty$ where $g_\varepsilon = 1$), so Fact 2 still applies. The optimistic truncation moves mass down, which Fact 2 covers in the other direction, or deletes it, which removes a non-negative term from δ. Both can be repeated after every convolution without breaking the inequality.
5. **The pessimistic PLD belongs to a real pair.** Rounding up raises every loss, so it lowers e^{-L}: $\mathbb{E}_{\omega^{\uparrow}}[e^{-L}] \le \mathbb{E}_{\omega}[e^{-L}] \le 1$ (mass moved to $+\infty$ contributes $e^{-\infty} = 0$). The Interpretation in Section 32.3 showed that any distribution with $\mathbb{E}[e^{-L}] \le 1$ is the PLD of some pair, by building the pair from it. So there is a pair $(P^{\uparrow}, Q^{\uparrow})$ with PLD $\omega^{\uparrow}$ (Doroshenko et al. 2022, Lemma 4.2, prove it for this rounding).

6. **That pair dominates the original.** Step 3 with $k = 1$ says $E_{e^\varepsilon}(P\|Q) \le E_{e^\varepsilon}(P^\uparrow\|Q^\uparrow)$ for every real ε, that is, for every $\gamma = e^\varepsilon > 0$. At $\gamma = 0$ the divergence is $\sum_y P(y) = 1$ for both pairs. So $(P^\uparrow, Q^\uparrow)$ dominates (P, Q) at every $\gamma \ge 0$, and by the dominating-pair theorem of Section 32.4 the convolution of rounded-up PLDs bounds adaptive compositions too.
7. **The width.** By step 2, the pessimistic total is the optimistic total plus at most kh. Adding a constant c to the loss shifts its δ curve by c to the right, because g_ε only sees the difference $\varepsilon - \ell$: $\mathbb{E}[g_\varepsilon(X + c)] = \mathbb{E}[g_{\varepsilon-c}(X)]$. With Fact 2 this gives $\delta^\uparrow(\varepsilon) \le \delta^\downarrow(\varepsilon - kh)$. So wherever the optimistic curve has dropped to a given δ, the pessimistic curve has dropped to it at most kh further right, and the ε at which each crosses δ differ by at most kh.

32.5.4. Interpretation

The lower bound needs care about what it bounds. Rounding down lowers losses, which raises e^{-L}, so it can push $\mathbb{E}[e^{-L}]$ above 1, and then $\omega^\downarrow$ is not the PLD of any pair. That does not matter for a lower bound: $\delta^\downarrow$ is still below the δ of k non-adaptive copies of (P, Q), by the sandwich. It bounds the mechanism's δ from below only if some neighbouring datasets actually produce (P, Q). That holds for the Gaussian and the subsampled Gaussian.

The width is almost exactly kh, not just at most. A loss that is not on the grid is moved by exactly h between its two roundings, and a continuous PLD puts no mass on the grid points, so $L^\uparrow = L^\downarrow + h$ almost surely and the totals differ by exactly kh. The error therefore grows linearly in k. To keep the sandwich width fixed at 10^4 steps, you need a grid 10^4 times finer than for one step.

The midpoint of the two sides is a good estimate, but not a bound. A loss spread evenly inside a grid cell is moved up by about $h/2$ on average when rounded up, and down by the same amount when rounded down. So the pessimistic total is biased up by about $h/2$ per step and the optimistic one down by the same amount, and their midpoint is nearly unbiased. Only the two sides are guarantees.

Mass at $+\infty$ is cheap but not free. Each truncation of τ moves at most τ of mass to a place where its weight is 1, so it adds at most τ to δ. With $\tau = 10^{-15}$ and a few dozen truncations, this is far below any δ you would report.

32.5.5. Example: randomized response composed three times

Randomized response with $\varepsilon_0 = \ln 3$ reports the true bit with probability $\frac{3}{4}$ and the flipped bit with probability $\frac{1}{4}$ (see Randomized response). The true report is $\frac{3/4}{1/4} = 3$ times likelier under D than under D', so its loss is $\ln 3$, and the flipped one has loss $-\ln 3$. Its PLD therefore has two points: loss $+1.0986$ with probability 0.7500 and -1.0986 with probability 0.2500. On the grid $h = 0.5$, the loss 1.0986 lies between 1 and 1.5 and -1.0986 between -1.5 and -1, so rounding up gives $\{-1, 1.5\}$ and rounding down gives $\{-1.5, 1\}$, with the same probabilities. The rounded-up PLD has $\mathbb{E}[e^{-L}] = 0.75\, e^{-1.5} + 0.25\, e^{1} = 0.1673 + 0.6796 = 0.8469 \le 1$, a valid PLD. The rounded-down PLD has $\mathbb{E}[e^{-L}] = 0.75\, e^{-1} + 0.25\, e^{1.5} = 0.2759 + 1.1204 = 1.3963 > 1$, which no pair of distributions can have.

Table 32.3 follows the four outcomes of three steps, indexed by the number j of steps with a positive loss. The probability of j is binomial, for instance $(\frac{1}{4})^3 = 0.0156$ for $j = 0$ and

$3 \times \frac{3}{4} \times (\frac{1}{4})^2 = 0.1406$ for $j = 1$. The exact total is $j \ln 3 - (3-j) \ln 3 = (2j-3) \ln 3$, the rounded-up total is $1.5j - (3-j)$, and the rounded-down total is $j - 1.5(3-j)$.

Table 32.3.: Three-fold randomized response with $\varepsilon_0 = \ln 3$. Each outcome's total loss, exactly and after rounding each step's loss to the grid $h = 0.5$.

j	Probability	Exact total loss	Rounded up	Rounded down
0	0.0156	-3.2958	-3	-4.5
1	0.1406	-1.0986	-0.5	-2
2	0.4219	1.0986	2	0.5
3	0.4219	3.2958	4.5	3

Every rounded-up total sits above the exact total, and every rounded-down total below it. The gap between the two is $3 \times 0.5 = 1.5$ in every row, as derivation step 7 predicts. Feeding each column into Equation 32.3 gives the three δ curves. At $\varepsilon = 0$, for instance, only the two positive exact totals count, with weights $1 - e^{-3.2958} = 1 - \frac{1}{27}$ and $1 - e^{-1.0986} = 1 - \frac{1}{3}$, and each of them has probability $\frac{27}{64} = 0.4219$, so the exact $\delta(0) = \frac{27}{64} \times \frac{26}{27} + \frac{27}{64} \times \frac{2}{3} = \frac{26}{64} + \frac{18}{64} = \frac{44}{64} = 0.6875$. The resulting values of δ are bracketed accordingly (Table 32.4 and Figure 32.2).

Table 32.4.: $\delta(\varepsilon)$ of three-fold randomized response, exact and from the rounded PLDs on the grid $h = 0.5$.

ε	Exact δ	Pessimistic	Optimistic
0	0.6875	0.7820	0.5669
1	0.4190	0.6758	0.3648
2	0.3064	0.3872	0.2667

32.5.6. Check yourself

- Would it be safe to round each loss to the *nearest* grid point? *(Not as a guarantee. Some losses move down, so neither side of Equation 32.6 holds in general. Nearest rounding is a good estimate. The PRV accountant starts from it and then shifts the whole grid so that the mean loss is preserved (Section 32.6).)*
- If the grid happens to contain every possible loss, how wide is the sandwich? *(Zero. With $h = \ln 3/2$ every loss of randomized response, $\pm \ln 3 = \pm 2h$, is on the grid, and both roundings return the exact PLD; Exercise 3 in the exercise book checks it.)*

32.6. Composition by FFT

32.6.1. Context

A discretized PLD is a vector of masses $m_0, \ldots, m_{n-1}$ at losses $(o+i)h$, where the integer offset o says where the vector starts on the grid, plus an atom at $+\infty$. Convolving two of them

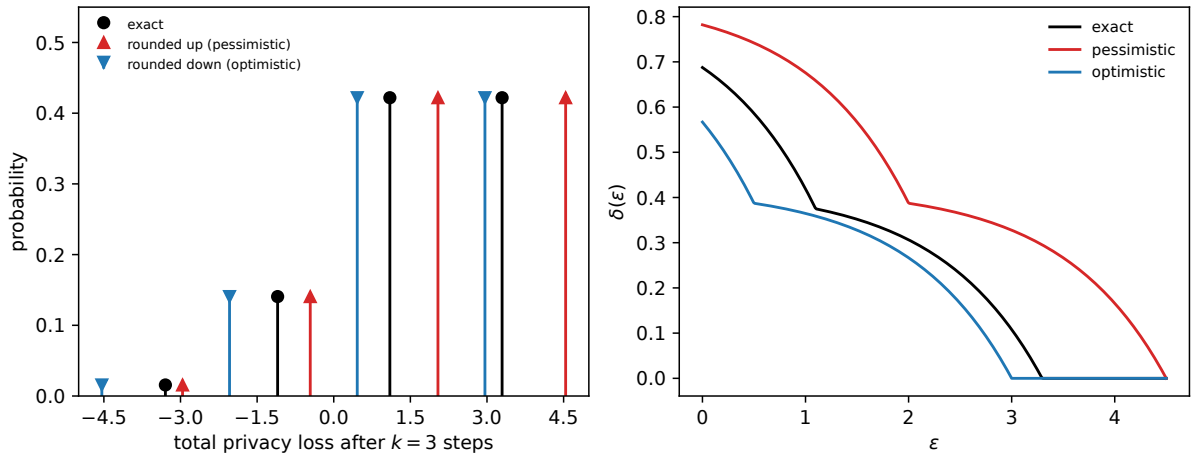


Figure 32.2.: Left: the PLD of three-fold randomized response with $\varepsilon_0 = \ln 3$ (black), rounded up (red) and rounded down (blue) on the grid $h = 0.5$; the rounded atoms are offset slightly sideways so that they do not overlap. Right: the three $\delta(\varepsilon)$ curves. The pessimistic curve lies above the exact one and the optimistic curve below it at every ε.

directly costs $O(n_a n_b)$ operations for vectors of lengths n_a and n_b. With a grid of $h = 10^{-5}$, a single DP-SGD step of the running example already has hundreds of thousands of bins (329590, Section 32.7), and after 10,000 steps the composed PLD has 2103251, a count the script prints along with the bins at every k it composes. The FFT makes this cheap. Koskela, Jälkö and Honkela (2020) introduced FFT-based PLD accounting.

32.6.2. Statement

On a common grid, adding two losses adds their grid indices, so the masses of the sum are the discrete convolution of the two mass vectors, and Fact 3 computes that convolution as a product of transforms, provided the window is long enough not to wrap. Let a and b be the mass vectors of two discretized PLDs with offsets o_a, o_b and atoms a_∞, b_∞ at $+\infty$, on the same grid. Their convolution has offset $o_a + o_b$, atom $a_\infty + b_\infty - a_\infty b_\infty$, and masses

$$c = \mathrm{IDFT}_N(\mathrm{DFT}_N(a) \cdot \mathrm{DFT}_N(b)), \qquad N \geq n_a + n_b - 1, \tag{32.7}$$

where DFT_N is the discrete Fourier transform of length N, IDFT_N its inverse, and the transforms are of the zero-padded vectors. For k identical steps, the transform of ω^{*k} is the k-th power of the transform, since each convolution multiplies by one more copy. The composed vector reaches index $k(n-1)$, so with $N \geq k(n-1)+1$,

$$\omega^{*k} = \mathrm{IDFT}_N(\mathrm{DFT}_N(m)^k). \tag{32.8}$$

If N is smaller, the result is the PLD of the total loss **modulo** Nh: mass that falls beyond the window wraps around to the other end.

32.6.3. Derivation

1. **Grid losses add as indices.** A loss $(o_a + i)h$ plus a loss $(o_b + j)h$ is $(o_a + o_b + i + j)h$. So the mass at index n of the sum collects every pair with $i + j = n$, $c_n = \sum_{i+j=n} a_i b_j$, which is the discrete convolution.
2. **The atom at $+\infty$.** The sum is finite only when both terms are finite, which has probability $(1 - a_\infty)(1 - b_\infty)$. The sum is $+\infty$ otherwise, with probability $1 - (1 - a_\infty)(1 - b_\infty) = a_\infty + b_\infty - a_\infty b_\infty$.
3. **Padding removes wraparound.** Pad a and b with zeros to length $N \geq n_a + n_b - 1$ and apply Fact 3. Since $i + j \leq n_a + n_b - 2 < N$, no index wraps, and the circular convolution is c.
4. **Powers.** Applying step 3 $k - 1$ times multiplies the transform by itself each time and gives $\mathrm{DFT}(m)^k$. The composed vector has $k(n - 1) + 1$ entries, because its largest index is k times the largest index $n - 1$, so that is the padding needed.
5. **Wraparound.** With N too small, Fact 3 still holds but indices add modulo N. Mass at total index $n \geq N$ lands at $n - N$: the total loss is reduced modulo Nh.

Repeated squaring. Taking the k-th power in one shot needs the full padded window, which is k times wider than one step. The script instead builds the power by squaring. It forms $\omega^{*2}, \omega^{*4}, \omega^{*8}, \ldots$ by convolving each with itself, and multiplies together the powers in the binary expansion of k; for $k = 10 = 8 + 2$, $\omega^{*10} = \omega^{*8} * \omega^{*2}$. After every convolution it truncates both tails at mass $\tau = 10^{-15}$, moving the mass in the safe direction of Section 32.5. The vector then grows like the spread of the composed PLD, not like the worst-case range $k(n - 1)$. The spread is the standard deviation of a sum of k independent losses, which grows like $\sqrt{k}$, and truncation at a fixed τ keeps a window of a roughly fixed number of standard deviations. Each convolution is padded to its two inputs, so nothing can wrap. In the example below, ten Gaussian steps take the vector from 15884 to 50257 bins, a ratio of $50257/15884 = 3.16$, which is $\sqrt{10}$. The script also cuts every loss below -100 or above 200 in the same safe direction, whatever its mass, because of floating-point noise (next paragraph): for a sharply peaked PLD, the noise in millions of bins can add up to more than τ, and then truncation by mass alone never trims the ends.

32.6.4. Interpretation

One FFT convolution of vectors of length N costs $O(N \log N)$. Repeated squaring needs at most $2 \log_2 k$ of them: about $\log_2 k$ squarings, and at most one multiplication for each binary digit of k.

The FFT is not exact in floating point. It computes each mass with an absolute error of order 10^{-16} times the largest mass, so tiny masses far in the tails come out as noise, sometimes negative. The script sets negative masses to zero. Summed over the two million bins of the 10,000-step PLD, this noise is many orders of magnitude below a δ of 10^{-5}, but it is not part of the certified bound. The PRV accountant of Gopi, Lee and Wutschitz (2021) also leaves it out of its error analysis, arguing that it is negligible for δ above 10^{-10}, the smallest δ its implementation accepts (their Section 6 and Appendix A).

To choose the grid, pick h from the sandwich width you can accept, which is kh (Section 32.5). Pick the range from the tails. The composed PLD has mean and variance k times those of one step, so a window of many standard deviations around the mean holds all but τ of the mass.

Chebyshev's inequality, which says that a variable lands at least t away from its mean with probability at most its variance divided by t^2, bounds the mass outside a window from the variance alone. It is loose but needs nothing else (Exercise 4 in the exercise book).

Three variants of the same idea are in use. The **PRV accountant** (Gopi, Lee & Wutschitz 2021) also discretizes the privacy loss random variable and convolves with the FFT. It truncates the loss to a window chosen with tail bounds, rounds each loss to the nearest grid point, and then shifts the whole grid by one constant so that the discretized loss has the same mean as the truncated one. Rounding to the nearest point moves some losses up and some down; once the means match, the per-step errors have mean zero, so over k steps they partly cancel and their sum grows like $\sqrt{k}\,h$ instead of kh (their Lemma 5.3). That is why a mesh $h \approx 1/\sqrt{k}$ suffices instead of $h \approx 1/k$. It proves explicit error bounds on ε, and for k identical steps its running time is of order $\sqrt{k}$ up to logarithmic factors, against $k^{1.5}$ for the earlier FFT accountant of Koskela et al.

Connect the dots (Doroshenko, Ghazi, Kamath, Kumar & Manurangsi 2022) builds the pessimistic PLD on a given grid differently: from the values of $\delta(\varepsilon)$ at the grid points, interpolating the hockey-stick curve between them, instead of rounding each loss. They prove it is the best pessimistic PLD supported on that grid, and it allows much coarser grids than rounding up.

Characteristic-function accounting (Zhu, Dong & Wang 2022) skips the grid of masses and works with the Fourier transform of the PLD directly. The characteristic function $\mathbb{E}[e^{\mathrm{i}tL}]$ of a sum of independent losses is the product of the per-step ones, for the same reason as in Fact 3, and δ is recovered by numerical integration in the frequency domain. Google's `dp_accounting` library implements PLD accounting with pessimistic and optimistic rounding and connect-the-dots.

32.6.5. Example: Gaussian steps on a grid

Discretize the Gaussian PLD of $\mu = 1$, which is $\mathcal{N}(0.5, 1)$, on the grid $h = 10^{-3}$, truncated at $\tau = 10^{-15}$ per tail. It has 15884 bins. After ten steps by repeated squaring, the truncated vector has 50257 bins, while the one-shot k-th power needs a padded window of $10 \times (15884 - 1) + 1 = 158831$. Both give the same pessimistic $\delta(1) = 0.81891018$, against the exact 0.81851782 from Equation 32.4 with $\mu = \sqrt{10}$. Turning this into ε at $\delta = 10^{-5}$, Table 32.5 compares with the exact values from Section 32.4.

Table 32.5.: Composed Gaussian steps with $\mu = 1$ on the grid $h = 10^{-3}$: ε at $\delta = 10^{-5}$ from the pessimistic and optimistic PLDs against the exact value.

k	Exact ε	Pessimistic	Optimistic	Mean of the pessimistic PLD
10	17.8566	17.8616	17.8516	5.0050 (exact 5)
100	91.8173	91.8673	91.7673	50.0500 (exact 50)

The sandwich width is kh: $10 \times 10^{-3} = 0.01$ at $k = 10$ and $100 \times 10^{-3} = 0.1$ at $k = 100$. The exact value sits in the middle, $17.8616 - 17.8566 = 0.005 = kh/2$ at $k = 10$, because rounding up moves each step's loss up by $h/2$ on average and rounding down moves it down by as much. The pessimistic means show the shift: $5 + 10 \times 0.0005$ and $50 + 100 \times 0.0005$.

Without padding, wraparound appears. Compute the tenth power on a circular window centred at the mean 5. A window of width 10, from loss 0 to 10, gives $\delta(1) = 0.83478562$, wrong in the second decimal, because losses below 0 wrap to the top of the window and losses above 10 wrap to the bottom. Width 20 gives 0.81890091, and width 40 gives 0.81891018, the padded value.

32.6.6. Check yourself

- Why does the circular window of width 10 *overestimate* $\delta(1)$? *(The two tails hold the same mass, 0.0569 each, because the window is centred on the mean: the composed PLD has standard deviation $\sqrt{10}$, so each tail beyond 5 from the mean has mass $\Phi(-5/\sqrt{10}) = \Phi(-1.581) = 0.0569$. Negative totals, which count nothing towards $\delta(1)$, wrap to just below 10, where they count almost fully. Totals above 10 wrap to between 0 and 10. Those that land below $\varepsilon = 1$ lose their weight, but about half of them (a share of 0.51) land above 1 and keep part of it. So the window gains more than it loses. In general the sign of the wraparound error is unknown, so it cannot be part of a certified bound.)*
- You double k. How much longer does one squaring step take, roughly? *(The composed PLD's spread grows like $\sqrt{k}$, so the vectors grow by about $\sqrt{2}$ and the FFT cost by a bit more than that, because of the $\log N$ factor.)*

32.7. The subsampled Gaussian

32.7.1. Context

One DP-SGD step with Poisson sampling at rate q and noise multiplier σ includes each record independently with probability q, sums the clipped gradients of the sampled records, and adds Gaussian noise. Look at it from the point of view of one record, the one that may or may not be in the dataset. Without it, the noisy sum (in the worst-case direction, scaled by the clipping norm) is a Gaussian centred at 0. With it, the record is sampled with probability q and then shifts the centre by 1, the sensitivity; otherwise it changes nothing. So the output is a mixture: centred at 1 with probability q, at 0 with probability $1 - q$. Amplification by subsampling derives this reduction and computes its RDP. Here you need its PLD, in closed form, so that the discretization can be exact up to rounding.

32.7.2. Statement

Under the add/remove relation, neighbouring datasets differ by removing one record or by adding one, and each direction gives its own pair. Write $D' = D \cup \{x\}$ for the dataset that holds the record, as in the rest of the part; a guarantee must cover both orders of the pair. The **remove** direction puts the dataset with the record first, $P = M(D')$ and $Q = M(D)$, so P is the mixture and Q the plain Gaussian; its neighbour is obtained by removing x. The **add** direction is the order $P = M(D)$, $Q = M(D')$, and the roles swap. The two pairs that matter are (Zhu, Dong & Wang 2022, Theorem 11 and the paragraph after it)

$$\text{remove: } P = (1-q)\,\mathcal{N}(0,\sigma^2) + q\,\mathcal{N}(1,\sigma^2), \quad Q = \mathcal{N}(0,\sigma^2); \qquad \text{add: } P = \mathcal{N}(0,\sigma^2), \quad Q = (1-q)\,\mathcal{N}(0,\sigma^2) + q\,\mathcal{N} \tag{32.9}$$

Each dominates one direction of neighbouring datasets (Theorem 11 states the remove pair with the mixture component at 0 and the base at 1; the reflection $y \mapsto 1 - y$ turns it into the form above and changes no divergence), and the k-step guarantee is the larger of the two δ curves, each composed k times.

The privacy loss of the remove pair is the log of the mixture density over the base density. The mixture over the base is $(1 - q)$ plus q times the ratio of the shifted Gaussian to the base, and that ratio is an exponential in y, as in the Gaussian PLD of Section 32.3. The add pair is the remove pair with P and Q swapped, which flips the sign of the log-ratio. So

$$L_{\text{rem}}(y) = \ln\Big((1-q) + q\, e^{(2y-1)/(2\sigma^2)}\Big), \qquad L_{\text{add}}(y) = -L_{\text{rem}}(y), \tag{32.10}$$

with $Y \sim P$ of the respective pair. L_{rem} is increasing in y, ranges over $(\ln(1-q), \infty)$, and so can be inverted: the output that produces loss ℓ is

$$y(\ell) = \sigma^2 \ln \frac{e^\ell - 1 + q}{q} + \frac{1}{2}. \tag{32.11}$$

Because the loss is increasing in the output, a bin of losses is an interval of outputs, and its mass is the probability of that interval under the mixture, a weighted difference of normal distribution functions:

$$P[L_{\text{rem}} \in (a, b]] = (1-q)\Big[\Phi\Big(\tfrac{y(b)}{\sigma}\Big) - \Phi\Big(\tfrac{y(a)}{\sigma}\Big)\Big] + q\Big[\Phi\Big(\tfrac{y(b)-1}{\sigma}\Big) - \Phi\Big(\tfrac{y(a)-1}{\sigma}\Big)\Big], \tag{32.12}$$

and $P[L_{\text{add}} \in (a, b]] = \Phi(y(-a)/\sigma) - \Phi(y(-b)/\sigma)$ with $Y \sim \mathcal{N}(0, \sigma^2)$.

32.7.3. Derivation

1. **Divide the mixture by the base.** Write φ for the standard normal density, so that $\mathcal{N}(c, \sigma^2)$ has density proportional to $\varphi((y-c)/\sigma)$ with the same constant for both centres. Dividing each part of the mixture by the base gives $\frac{(1-q)\varphi(y/\sigma) + q\varphi((y-1)/\sigma)}{\varphi(y/\sigma)} = (1-q) + q\,\frac{\varphi((y-1)/\sigma)}{\varphi(y/\sigma)}$.
2. **Compute the ratio of two Gaussians.** The densities are exponentials of minus a square, so their ratio is the exponential of the difference of the squares: $\frac{\varphi((y-1)/\sigma)}{\varphi(y/\sigma)} = \exp\big(\frac{y^2 - (y-1)^2}{2\sigma^2}\big) = e^{(2y-1)/(2\sigma^2)}$. Taking logs of step 1 gives L_{rem}. Swapping P and Q flips the sign of the log-ratio, which gives L_{add}.
3. **The loss is increasing.** $e^{(2y-1)/(2\sigma^2)}$ is increasing in y and positive, so L_{rem} is increasing. As $y \to -\infty$ the exponential vanishes and the loss tends to $\ln(1-q)$; as $y \to \infty$ it tends to ∞.
4. **Invert it.** Solve $e^\ell = (1-q) + q\, e^{(2y-1)/(2\sigma^2)}$ for y: subtract $1-q$, divide by q, take logs, multiply by σ^2 and add $\frac{1}{2}$.
5. **Turn loss bins into output intervals.** Because the map is increasing, $\{L_{\text{rem}} \in (a, b]\} = \{Y \in (y(a), y(b)]\}$. Its probability under the mixture is $1-q$ times the probability under $\mathcal{N}(0, \sigma^2)$ plus q times that under $\mathcal{N}(1, \sigma^2)$, which is the stated sum. For the add pair, $L_{\text{add}} \in (a, b]$ exactly when $L_{\text{rem}} \in [-b, -a)$, that is, exactly when $Y \in [y(-b), y(-a))$, with $Y \sim \mathcal{N}(0, \sigma^2)$. Where $-b \le \ln(1-q)$, no output has that loss, and $y(-b) = -\infty$.
6. **Single-step δ in closed form, as a check.** For the remove pair, $L > \varepsilon$ exactly when $Y > y(\varepsilon)$, so by Equation 32.3, $\delta(\varepsilon) = P[Y > y(\varepsilon)] - e^\varepsilon\, Q[Y > y(\varepsilon)]$, two normal tail probabilities. The add pair is the same with $Y < y(-\varepsilon)$.

7. **Why two directions.** Under add/remove, the neighbour of a dataset can be that dataset with one record removed or with one added, so both orders of the pair, $(M(D'), M(D))$ and $(M(D), M(D'))$, must be covered. The two cases give different pairs, and neither one's PLD is a rounding of the other, so bounding one does not bound the other. Zhu, Dong & Wang (2022) prove each pair dominates its case. A k-step run's guarantee is $\max\{\delta_{\mathrm{rem},k}(\varepsilon), \delta_{\mathrm{add},k}(\varepsilon)\}$, and the corresponding ε is the larger of the two values of ε.

The script computes the bin masses in step 5 from whichever of Φ and $1-\Phi$ is smaller, so masses deep in either tail keep their relative precision.

32.7.4. Interpretation

The two directions have very different losses (Figure 32.3). The remove loss is bounded below by $\ln(1-q)$ and has an unbounded upper tail: a rare large output betrays that the record was used. The add loss is its mirror image, bounded above by $-\ln(1-q)$. Since δ only counts losses above ε, a single add step has $\delta(\varepsilon) = 0$ once $\varepsilon \geq -\ln(1-q)$.

Which direction dominates? In the running example the remove direction gives the larger ε at every k in Table 32.7: the add direction's upper bound is only 0.0091 at $k = 1$ and 5.6615 at $k = 10{,}000$. That is an observation, not a theorem; the accountant computes both and takes the larger, as `pld_epsilon_subsampled_gaussian` does. The remove pair alone is not enough in general: Zhu, Dong & Wang (2022) point out that PLD accountants that use only the remove pair cover only the "remove" half of the add/remove relation.

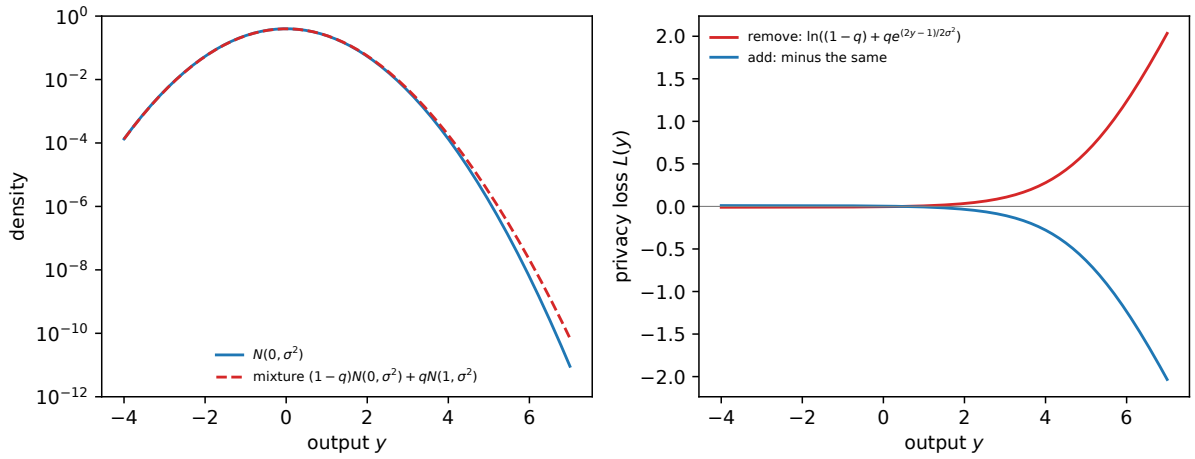


Figure 32.3.: Left: the densities of $\mathcal{N}(0,\sigma^2)$ and of the mixture $(1-q)\mathcal{N}(0,\sigma^2)+q\mathcal{N}(1,\sigma^2)$ for $q = 0.01$, $\sigma = 1$, on a log scale. They differ only in the far right tail. Right: the privacy loss as a function of the output for the remove pair (red) and the add pair (blue). Both are nearly flat near zero and steep in the right tail.

32.7.5. Example: one DP-SGD step

For $q = 0.01$ and $\sigma = 1$, the lowest possible remove loss is $\ln(0.99) = -0.0101$. The remove loss is in $[-0.0101, 3.2858]$ except for 10^{-15} of mass, and the add loss is in $[-2.8926, 0.0101]$. On the grid

$h = 10^{-5}$, that is 329590 and 290268 bins; for the remove direction, $3.2858 - (-0.0101) = 3.2959$ divided by 10^{-5}. The rounded-up remove PLD has $\mathbb{E}[e^{-L}] = 0.999995$, below 1 and so a valid PLD, as derivation step 5 of Section 32.5 requires. Table 32.6 compares the pessimistic PLD's δ with the closed form of step 6.

Table 32.6.: One subsampled-Gaussian step ($q = 0.01$, $\sigma = 1$): δ(ε) in closed form and from the pessimistic PLD on the grid $h = 10^{-5}$.

ε	Remove: exact δ	Remove: pessimistic	Add: exact δ	Add: pessimistic
0	3.829249e-03	3.830792e-03	3.829249e-03	3.832688e-03
0.5	2.214519e-07	2.214633e-07	0	0
1	2.732009e-09	2.732117e-09	0	0

The pessimistic values are slightly above the exact ones, as they must be. At $\varepsilon = 0$ both directions have the same δ, and the reason is a symmetry. At $\varepsilon = 0$, δ is $\sum_y (P(y) - Q(y))_+$, the total variation distance. Since P and Q both sum to 1, the total amount by which P exceeds Q equals the total amount by which Q exceeds P, so the value does not change when the two are swapped, and the add pair is the remove pair swapped. Beyond $\varepsilon = 0.0101$ the add direction's δ is 0, as the Interpretation explains. At $\delta = 10^{-5}$, one step has $\varepsilon = 0.1995$ in the remove direction and 0.0091 in the add direction (exact, by bisection on the closed form).

32.7.6. Check yourself

- What is $\delta(\varepsilon)$ of one add step at $\varepsilon = 0.02$? *(Zero, since the add loss never exceeds* $-\ln(0.99) = 0.0101$*, and δ only counts losses above ε.)*
- Why is the closed-form δ of one step not enough for k steps? *(The composed PLD is a k-fold convolution of a non-Gaussian distribution, which has no closed form. Only the per-step PLD is explicit.)*

32.8. The running example: DP-SGD

32.8.1. Context

The question from the start of the chapter: DP-SGD with $q = 0.01$, $\sigma = 1$, at $\delta = 10^{-5}$, after k steps. Every other chapter of this part gives its own number for the same run. This section gives the PLD bounds and compares them with two accountants from earlier chapters.

32.8.2. Statement

Putting the pieces together, discretize each direction's subsampled-Gaussian PLD up and down, compose it k times by FFT, and read off ε; the true value lies between the optimistic and the pessimistic answer. On the grid $h = 10^{-5}$ with tails truncated at 10^{-15}, Table 32.7 gives ε from the pessimistic PLD (an upper bound, valid for adaptive composition) and the optimistic PLD

(a lower bound for the non-adaptive run), each the larger over the two directions. It compares them with the RDP accountant with the improved conversion and with the GDP central-limit approximation.

Table 32.7.: DP-SGD with $q = 0.01$, $\sigma = 1$: ε at $\delta = 10^{-5}$ after k steps, from the pessimistic and optimistic PLDs on the grid $h = 10^{-5}$ (the larger of the two directions; the width is their difference, kh), the pessimistic PLD of the add direction alone, the RDP accountant, and the GDP central-limit approximation.

k	PLD upper	PLD lower	Width	Add direction, upper	RDP	GDP CLT
1	0.1995	0.1994	1.0e-05	0.0091	0.9555	0.0368
10	0.3800	0.3799	1.0e-04	0.0655	1.0353	0.1304
100	0.7185	0.7175	1.0e-03	0.3387	1.2141	0.4575
1000	1.8332	1.8232	1.0e-02	1.4217	2.1014	1.6177
10000	6.2377	6.1377	1.0e-01	5.6615	6.7127	6.0071

32.8.3. Derivation

1. **PLD columns.** Discretize each direction's PLD (Section 32.7) up and down, compose k times by FFT (Section 32.6), and solve $\delta(\varepsilon) = 10^{-5}$ on each result. The solve can be exact. For ε between two grid points, the set of grid losses above ε does not change, so by Equation 32.3 $\delta(\varepsilon) = m_\infty + A - e^\varepsilon B$, where m_∞ is the atom at $+\infty$, A the total mass of the grid points above ε, and B the sum of those masses times $e^{-\ell}$. Setting this equal to 10^{-5} and solving for e^ε gives the root directly. Take the larger ε of the two directions.
2. **RDP column.** RDP tracks, for each order $\alpha > 1$, the Rényi divergence $\frac{1}{\alpha-1}\ln \mathbb{E}_Q\big[(P/Q)^\alpha\big]$, a log-moment of the likelihood ratio; these divergences add up over composed steps, and a conversion turns the total into (ε, δ). As in Amplification by subsampling, for the remove pair the moment is $A_\alpha = \mathbb{E}_{y\sim\mathcal{N}(0,\sigma^2)}\big[\big((1-q) + q\,e^{(2y-1)/(2\sigma^2)}\big)^\alpha\big]$, the likelihood ratio of step 1 of Section 32.7 raised to the power α and averaged under the base $Q = \mathcal{N}(0, \sigma^2)$. The RDP of one step at order α is $\frac{1}{\alpha-1}\ln A_\alpha$. For integer α, expanding the power with the binomial theorem and using $\mathbb{E}_Q[e^{j(2y-1)/(2\sigma^2)}] = e^{(j^2-j)/(2\sigma^2)}$ (the Gaussian moment generating function at j/σ^2) gives $A_\alpha = \sum_{j=0}^{\alpha}\binom{\alpha}{j}(1-q)^{\alpha-j}q^j e^{(j^2-j)/(2\sigma^2)}$. For fractional α the script computes the expectation by numerical integration. Multiply by k, convert with $\varepsilon = \varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha-1}$ (From Rényi DP to (ε, δ) and to tests), where $\varepsilon(\alpha)$ is the composed RDP at order α, and minimize over the part's grid of orders. As a check, integration and the binomial sum agree at $\alpha = 2$ ($\ln A_2 = 0.00017181$) and $\alpha = 10$ ($\ln A_{10} = 0.34443377$).
3. **CLT column.** The GDP central limit theorem replaces the k composed steps by one Gaussian pair with shift $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$ (f-DP and Gaussian DP). One way to see where the formula comes from: one Gaussian step with shift 1 has χ^2-divergence $e^{1/\sigma^2} - 1$ (the average squared deviation of the likelihood ratio from 1), subsampling at rate q scales the ratio's deviation from 1 by q and so this divergence by q^2, and for Gaussians μ^2 adds over composed steps, which gives the factor k. Then ε comes from Equation 32.4. At $k = 10{,}000$, $\mu = 0.01 \times \sqrt{10^4 \times (e-1)} = 0.01 \times 100 \times 1.3108 = 1.3108$.

32.8.4. Interpretation

The PLD pins down the truth (Figure 32.4). At $k = 1000$ the true ε is between 1.8232 and 1.8332. At $k = 10{,}000$ the bracket is $[6.1377, 6.2377]$, wider because the width is kh.

RDP is a valid bound, but loose. It exceeds the PLD upper bound by $0.9555 - 0.1995 = 0.7560$ at $k = 1$, $2.1014 - 1.8332 = 0.2682$ at $k = 1000$ and $6.7127 - 6.2377 = 0.4750$ at $k = 10{,}000$. The single step shows why most clearly: it gives 0.9555 against 0.1995, because the conversion from RDP pays a term of about $\frac{\ln(1/\delta)}{\alpha - 1}$ whatever the curve, the price of turning one moment of the loss into a tail bound.

The CLT is not a bound, and here it is too optimistic. It falls below the PLD *lower* bound at every k, by $1.8232 - 1.6177 = 0.2055$ at $k = 1000$ and $6.1377 - 6.0071 = 0.1306$ at $k = 10{,}000$. The subsampled PLD has a heavy right tail (Figure 32.5), and at $\delta = 10^{-5}$ you are reading exactly that tail, where the Gaussian approximation is worst. Reporting the CLT number would understate the privacy loss.

Pick h for the largest k you will run. The width is kh, and $h = 10^{-5}$ makes it $10^4 \times 10^{-5} = 0.1$ at $k = 10{,}000$. Each tenfold refinement of h makes the vectors ten times longer, because the same range of losses is cut into ten times as many bins: after 1000 steps the pessimistic remove PLD has 7687, 76894 and 767721 bins for $h = 10^{-3}$, 10^{-4} and 10^{-5}. Time and memory grow at least as fast.

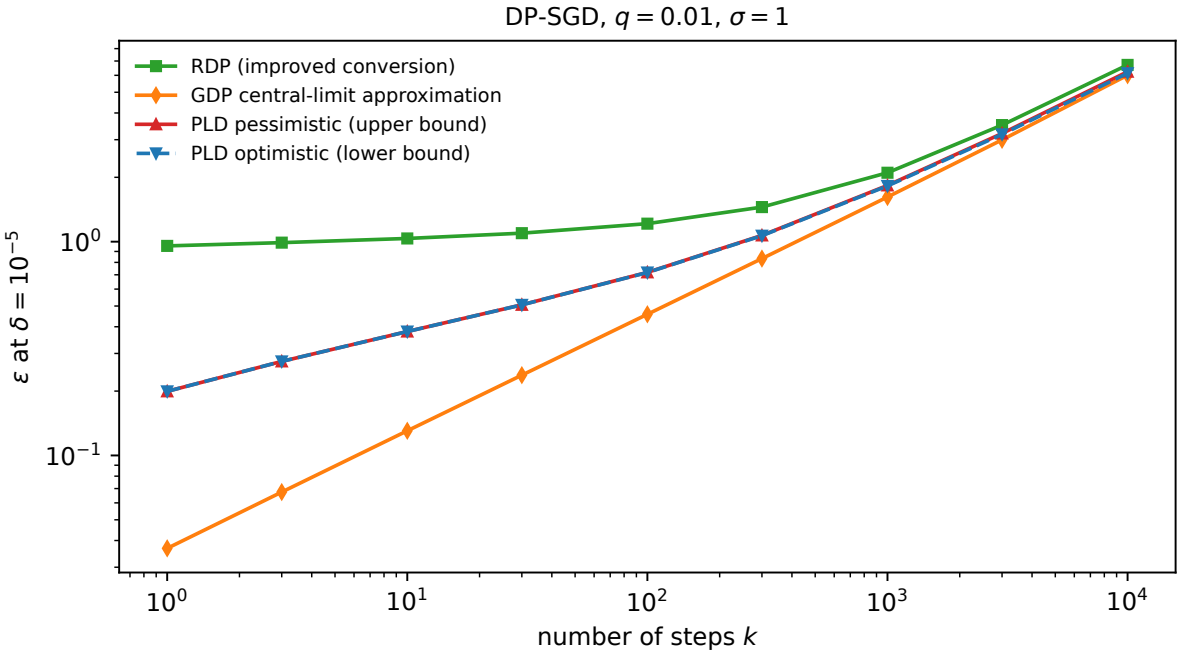


Figure 32.4.: ε at $\delta = 10^{-5}$ against the number of steps k for DP-SGD with $q = 0.01$, $\sigma = 1$: the RDP accountant (green), the GDP central-limit approximation (orange), and the pessimistic (red) and optimistic (blue) PLD bounds on the grid $h = 10^{-5}$. The two PLD curves overlap at this scale; RDP lies above them and the CLT below them.

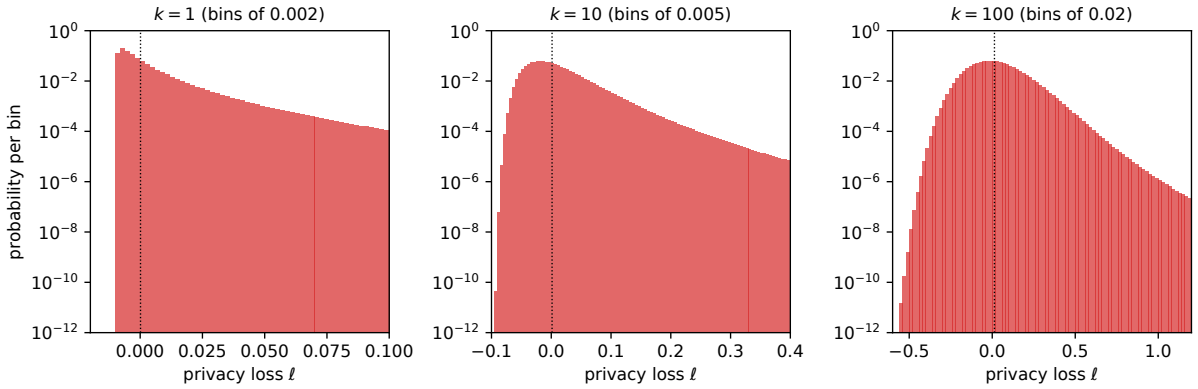


Figure 32.5.: The pessimistic remove-direction PLD of DP-SGD with $q = 0.01$, $\sigma = 1$ after 1, 10 and 100 steps (grid $h = 10^{-4}$, re-binned for display; log scale). The dotted line is the mean. One step piles its mass just above its lower limit $\ln(1-q) = -0.0101$ and has a long right tail; convolution moves the bulk towards a bell shape, but the right tail stays heavier than the left.

32.8.5. Example: sandwich width against grid size

The sandwich width should be kh, so at a fixed k it should shrink tenfold with each tenfold finer grid. Table 32.8 repeats $k = 1000$ on three grids.

Table 32.8.: DP-SGD after $k = 1000$ steps: the PLD bounds on ε at $\delta = 10^{-5}$ for three grid widths.

h	Upper	Lower	Width	kh
10^{-3}	2.3279	1.3279	1.0000	1.0000
10^{-4}	1.8782	1.7782	0.1000	0.1000
10^{-5}	1.8332	1.8232	0.0100	0.0100

The width equals kh to four decimals, as derivation step 7 of Section 32.5 predicts. On the coarsest grid the upper bound 2.3279 is worse than RDP's 2.1014. A PLD accountant is only as good as its grid.

32.8.6. Check yourself

- A colleague reports $\varepsilon = 1.62$ for 1000 steps from the CLT. What do you tell them? *(That is below the PLD lower bound 1.8232, so the run is not* $(1.62, 10^{-5})$*-DP. The CLT is an approximation and underestimates here.)*
- You need the upper bound at $k = 10{,}000$ within 0.01 of the truth. What grid do you need? *(A width of at most 0.01 needs* $kh \le 0.01$*, so* $h \le 0.01/10^4 = 10^{-6}$*, ten times finer than the script's grid.)*

32.9. Code

The script code/numerical_accounting.py implements the whole accountant in NumPy: the PLD dataclass, pessimistic and optimistic discretization, FFT convolution with padding and truncation, repeated squaring, the exact solve for ε, the subsampled-Gaussian PLD in both directions, and, for comparison, the RDP accountant and the CLT. The public functions are `gaussian_pld_delta`, `discretize_subsampled_gaussian_pld`, `compose_pld`, `pld_delta`, `pld_epsilon` and `pld_epsilon_subsampled_gaussian`. From the repository root:

```
uv run python code/part04-privacy-accounting/numerical_accounting.py
```

It takes from half a minute to a few minutes, depending on the machine. Nearly all of that is the four $k = 10{,}000$ compositions on the grid $h = 10^{-5}$. **Output** (seed 0), summarized in Table 32.9:

Table 32.9.: The script's output with the default seed.

Experiment	Result
Gaussian PLD, $\mu = 1$	Monte Carlo mean 0.4990, variance 1.0013, $\mathbb{E}[e^{-L}]$ 1.0015; δ(ε) in Table 32.2; $\varepsilon = 4.3772$ at $\delta = 10^{-5}$
Finite pair, $\varepsilon = 0.5$	Both forms of Equation 32.3 give 0.2351
Randomized response, three steps, $h = 0.5$	Table 32.3 and Table 32.4; $\mathbb{E}[e^{-L}]$ 0.8469 (up), 1.3963 (down)
Gaussian steps on $h = 10^{-3}$	Table 32.5; squaring and one-shot power agree, $\delta(1) = 0.81891018$
Wraparound, $k = 10$	Window 10: 0.83478562; window 20: 0.81890091; window 40: 0.81891018
One subsampled step	Table 32.6; exact ε 0.1995 (remove) and 0.0091 (add)
RDP check	$\ln A_2 = 0.00017181$ and $\ln A_{10} = 0.34443377$ by both methods
DP-SGD	Table 32.7 and Table 32.8

tests/test_numerical_accounting.py checks that the output still matches.

What to take from it. First, the pieces are right: the Gaussian PLD reproduces the closed form, the FFT squaring reproduces the one-shot power to eight digits, and the subsampled discretization brackets the closed-form single-step δ. Because each piece is checked against something exact, you can trust the composed numbers where nothing exact exists. Second, the sandwich is certified and its width is at most kh: nothing else in the accountant introduces a one-sided error you cannot see, so the two PLD columns of Table 32.7 are an honest error bar. Third, RDP is safe but loose, and the CLT is neither safe nor, here, accurate in the tail. The gap between them and the PLD bounds is what a loose accountant costs you.

32.10. References

- Sommer, Meiser & Mohammadi, "Privacy Loss Classes: The Central Limit Theorem in Differential Privacy" (Proceedings on Privacy Enhancing Technologies, PoPETs 2019). Defines the privacy loss distribution, proves that composition convolves PLDs, and shows that composed PLDs tend to Gaussians. Read it for the PLD formalism of Section 32.3 and Section 32.4.
- Meiser & Mohammadi, "Tight on Budget? Tight Bounds for r-Fold Approximate Differential Privacy" (CCS 2018). The privacy buckets: discretize the loss into buckets, round up for a certified upper bound and down for a lower bound. The origin of Section 32.5.
- Koskela, Jälkö & Honkela, "Computing Tight Differential Privacy Guarantees Using FFT" (AISTATS 2020). FFT-based PLD accounting on a fixed window, with error analysis for the truncation and wraparound. Read it for Section 32.6.
- Koskela, Jälkö, Prediger & Honkela, "Tight Differential Privacy for Discrete-Valued Mechanisms and for the Subsampled Gaussian Mechanism Using FFT" (AISTATS 2021). Rigorous upper and lower bounds with the FFT accountant, applied to the subsampled Gaussian; the source for the setting of Section 32.7 and Section 32.8.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021). The PRV accountant: nearest-point discretization with a mean-matching shift, explicit error bounds, and a running time of order $\sqrt{k}$ (up to logarithmic factors) for k identical steps. Read it when you need certified error bars on a fast accountant.
- Doroshenko, Ghazi, Kamath, Kumar & Manurangsi, "Connect the Dots: Tighter Discrete Approximations of Privacy Loss Distributions" (PoPETs 2022). Frames discretization with dominating pairs, proves rounding up gives a valid PLD (Lemma 4.2) and builds the optimal pessimistic PLD on a grid. Read it for the cleanest account of why the pessimistic bound is valid.
- Zhu, Dong & Wang, "Optimal Accounting of Differential Privacy via Characteristic Function" (AISTATS 2022). Dominating pairs (Definition 7), their adaptive composition (Theorem 10), the two directions of Poisson subsampling (Theorem 11), and accounting through the characteristic function of the privacy loss.
- Google, `dp_accounting` library documentation (the `pld` module). A production PLD accountant with pessimistic and optimistic estimates and connect-the-dots discretization; use it to check the numbers here.

32.11. Cheat sheet

$L(y) = \ln \frac{P(y)}{Q(y)}$; the PLD ω is the law of $L(Y)$ with $Y \sim P$; q is the sampling rate; h the grid width; k the number of steps. Table 32.10 collects the results.

Table 32.10.: The results of this chapter.

Result	Formula	Note
δ from the PLD	$\delta(\varepsilon) = \mathbb{E}_\omega[(1 - e^{\varepsilon - L})_+] = P[L > \varepsilon] - e^\varepsilon Q[L > \varepsilon]$	Atom at $+\infty$ counts fully
Law under Q	$e^{-\ell}\omega(d\ell)$, rest at $-\infty$	A PLD needs $\mathbb{E}_\omega[e^{-L}] \le 1$

Result	Formula	Note
Gaussian PLD	$\mathcal{N}(\mu^2/2, \mu^2)$; $\delta = \Phi(\frac{\mu}{2} - \frac{\varepsilon}{\mu}) - e^{\varepsilon}\Phi(-\frac{\mu}{2} - \frac{\varepsilon}{\mu})$	k steps: $\mu \to \sqrt{k}\mu$
Composition	$\omega_1 * \cdots * \omega_k$	Adaptive: convolve dominating pairs
Pessimistic / optimistic	Round losses up (tail to $+\infty$) / down (bottom tail dropped)	$\delta^{\downarrow} \le \delta \le \delta^{\uparrow}$ for every k
Sandwich width	$\varepsilon^{\uparrow} - \varepsilon^{\downarrow} \le kh$	Equality for continuous PLDs without truncation
FFT	$\mathrm{IDFT}(\mathrm{DFT}(a)\,\mathrm{DFT}(b))$, pad to $n_a + n_b - 1$	Too short a window wraps losses modulo Nh
Subsampled Gaussian loss	$\pm \ln\left((1-q) + qe^{(2y-1)/(2\sigma^2)}\right)$	Remove: $+$, $Y \sim$ mixture; add: $-$, $Y \sim \mathcal{N}(0, \sigma^2)$; take the max

Rules of thumb

- Report the pessimistic ε, and the optimistic one next to it as the error bar.
- Choose h so that kh is the error you can accept; check the sandwich width rather than trusting the grid.
- Compute both directions of add/remove for subsampled mechanisms; do not assume one dominates.
- Pad or truncate; never let an FFT wrap around silently.
- Use RDP and the CLT as quick estimates. RDP over-reports; the CLT can under-report in the tail that $\delta = 10^{-5}$ looks at.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

33. Amplification by shuffling and by iteration

33.1. Why this matters

Amplification by subsampling showed that uncertainty is worth privacy. If an adversary cannot tell **whether** your record was used in a step, the step costs far less than its noise alone suggests: a step that touches your record only with probability q leaks roughly q times as much. That is not the only uncertainty an algorithm can create. Two others matter in practice, and each has its own accountant.

The first is uncertainty about **which report is yours**. In the shuffle model, n users each randomize their own data on their own device and send the noisy report through a shuffler, a trusted component that strips identities and mixes the order. The analyst sees a bag of n anonymous reports. Each report on its own satisfies local differential privacy (DP) with some level ε_0: whoever sees that one report, even the collector, learns little about its sender, and the smaller ε_0, the less. To keep the reports useful, ε_0 is usually large, which makes this a weak guarantee. But the analyst does not know which report in the bag is yours, so your report hides among the others. The result is central DP, the guarantee about everything the analyst sees, with an ε that falls like $1/\sqrt{n}$. This is shuffling of anonymous, locally randomized reports. It is not the shuffle-and-partition batching used to train a model privately in the central model, where each record is used in exactly one batch per epoch and the shuffle only hides which one: Amplification by subsampling shows that no comparable amplification theorem covers that case, and that the Poisson number is wrong for it.

The second is uncertainty about **when your record was used**. Noisy gradient descent (GD) on a convex loss forgets. Each later step adds fresh noise and pulls nearby parameters together, so the effect of your record on the final model is washed out step by step. If only the final model is released, a record used early in training is much better hidden than the composition accounting of Rényi DP says, because composition charges every step as if the adversary had seen every intermediate model. For some problems, the privacy loss stops growing with the number of steps.

33.1.1. The one idea: the adversary's view is a post-processing of something simpler

Every result in this chapter has the same shape. You show that what the adversary sees can be **generated from a much simpler pair of distributions** by a procedure that does not depend on your record. Anything the adversary can learn from the real output, it could then have learned from the simple pair, by running that procedure itself. This is the data-processing inequality, and it means the simple pair's privacy holds for the real output. For shuffling, the simple pair is a pair of **counts**: how many reports look like each of your two possible values. For iteration, it is a pair of **Gaussians**, or a pair of distributions that a short chain of noisy steps keeps close. The work is

in finding the simple pair and computing how far apart its two distributions are. Table 33.1 lists the results.

Table 33.1.: The results of this chapter and where each is used.

Result	Where it is used
Exact $\delta(\varepsilon)$ of shuffled binary randomized response	Ground truth for any shuffling bound; deploying randomized-response surveys
The clone reduction and bound of Feldman, McMillan & Talwar	Central guarantees for any local randomizer behind a shuffler
Shifted Rényi divergence, shift reduction, contraction	Privacy of the last iterate of contractive noisy iterations
Amplification by iteration for one-pass noisy gradient descent	Per-record guarantees that depend on when the record was used
Hidden-state bounds that plateau	Convex training run for many epochs, releasing only the final model
Exact last-iterate privacy of noisy GD on a quadratic	A worked case in which every quantity is in closed form

How to read this chapter. The first two sections are about shuffling and the last three about iteration; the two halves are independent. In each half, read the exact computation first (shuffled randomized response; the quadratic loss), because it shows what the general bounds approximate. The derivations of the shift-reduction lemma and of the bounded-domain bound can wait for a second pass.

What you need. The Toolkit below restates every outside fact the derivations use, with a short reason for each, so you can read this chapter on its own. For more detail: the hockey-stick divergence E_γ, the Rényi divergence D_α and the data-processing inequality are in Divergences. The non-expansiveness of a gradient step is Fact 4 of Stability and generalization, and projection onto a convex set is Fact 3 of Optimization. Randomized response is in The definition of differential privacy, and the shuffle model in Trust models. Rényi differential privacy (RDP) and its composition are in Rényi DP, and its conversion to (ε, δ) in From RDP to (ε, δ). Gaussian differential privacy (GDP) is in f-DP and Gaussian DP.

Conventions. Throughout this chapter the neighbouring relation is replace-one: D and D' have the same size and differ in the value of one record, yours. Both shuffling theorems and both iteration theorems below are stated for it. P and Q are the distributions of the adversary's view on D and on D'. A mechanism is (ε, δ)-DP when $P(S) \le e^\varepsilon Q(S) + \delta$ and $Q(S) \le e^\varepsilon P(S) + \delta$ for every set S of outputs: no event becomes more than e^ε times likelier when your record changes, up to a small slack δ. In the iteration sections, some symbols differ from the part's notation table. σ is the noise standard deviation in parameter units, and $\tilde{\sigma}$ the noise multiplier. L_f is the Lipschitz constant of the loss and β its smoothness constant; L stays the privacy loss. d_K is the diameter of the convex set K that the parameters live in, so that D always means a dataset.

33.2. Toolkit: three facts every derivation uses

33.2.1. Fact 1 — $\delta(\varepsilon)$ of a discrete pair is a sorted tail sum

For a fixed ε, the smallest δ that makes $P(S) \le e^{\varepsilon}Q(S) + \delta$ true for every set S is the largest gap $P(S) - e^{\varepsilon}Q(S)$ over all sets. That largest gap is the **hockey-stick divergence** $E_{e^{\varepsilon}}(P\|Q)$. To find it, look at what each outcome y contributes to the gap: $P(y) - e^{\varepsilon}Q(y)$. This is positive exactly when $P(y)/Q(y) > e^{\varepsilon}$, that is, when the **privacy loss** $L(y) = \ln \frac{P(y)}{Q(y)}$, the log of how much likelier y is under D than under D', exceeds ε. So the best set collects every outcome with $L(y) > \varepsilon$ and nothing else, and for distributions P, Q on a finite set

$$\delta(\varepsilon) = E_{e^{\varepsilon}}(P\|Q) = \sum_{y:\, L(y)>\varepsilon} (P(y) - e^{\varepsilon}Q(y)). \tag{33.1}$$

This form makes the computation cheap. Sort the outcomes by $L(y)$ once and keep running sums of P and Q from the top. Then $\delta(\varepsilon)$ for any ε is one lookup: find where L drops below ε and read off the two running sums. The ε for a target δ is a bisection over these lookups. The script works this way, and builds the binomial probabilities in log space so that values of 10^{-300} cause no trouble.

The definition of (ε, δ)-DP has two directions, because the adversary may try to tell D from D' or D' from D. It needs $E_{e^{\varepsilon}}(P\|Q) \le \delta$ and $E_{e^{\varepsilon}}(Q\|P) \le \delta$, so the exact $\delta(\varepsilon)$ of a pair is the larger of the two sums.

33.2.2. Fact 2 — Post-processing a simpler pair

Suppose there is a randomized map f, the same for both datasets, that turns a sample of $\tilde{P}$ into a sample of P and a sample of $\tilde{Q}$ into a sample of Q; write $P = f(\tilde{P})$ and $Q = f(\tilde{Q})$. Then the real pair can be no further apart than the simple one: every divergence of (P, Q) is at most that of $(\tilde{P}, \tilde{Q})$,

$$E_{\gamma}(P\|Q) \le E_{\gamma}(\tilde{P}\|\tilde{Q}) \qquad \text{and} \qquad D_{\alpha}(P\|Q) \le D_{\alpha}(\tilde{P}\|\tilde{Q}). \tag{33.2}$$

This is the **data-processing inequality**. The reason, for E_{γ}, is that a test on the output can be pulled back to a test on the input. Take any set S of outputs, and let $g(u) = \Pr[f(u) \in S]$, a number between 0 and 1 for each input u. Then $P(S) - \gamma Q(S) = \mathbb{E}_{\tilde{P}}[g] - \gamma\, \mathbb{E}_{\tilde{Q}}[g]$. The right side is a sum of $g(u)(\tilde{P}(u) - \gamma\tilde{Q}(u))$ over u, and it is largest when $g = 1$ wherever the bracket is positive and $g = 0$ elsewhere, which gives exactly $E_{\gamma}(\tilde{P}\|\tilde{Q})$. The Rényi divergence D_{α}, defined in Fact 3, satisfies the same inequality, by Jensen's inequality applied to the convex function $t \mapsto t^{\alpha}$ inside its sum (Divergences proves it).

In this chapter the shuffled view will be a post-processing of counts, and the final model a post-processing of a coupled pair of noisy iterates.

33.2.3. Fact 3 — Gaussian pairs

Gaussian pairs come up in both halves of the chapter, so it pays to know everything about them once. Suppose that on D the adversary sees $Y \sim \mathcal{N}(m, v)$ and on D' it sees $Y \sim \mathcal{N}(m', v)$: the same spread, a shifted centre. Subtracting m and dividing by $\sqrt{v}$ is an invertible map that is the same for both datasets, so by Fact 2 applied in both directions it changes no divergence. It turns the pair into $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu, 1)$ with

$$\mu = \frac{|m - m'|}{\sqrt{v}}, \tag{33.3}$$

the shift measured in standard deviations. A pair like this is **μ-GDP**: exactly as hard to tell apart as two unit-variance normals μ apart. Everything else follows from the privacy loss of $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$, which is the difference of the two log densities, $L(y) = \frac{(y-\mu)^2 - y^2}{2} = \frac{\mu^2}{2} - \mu y$. It is linear in y, so it is normal: when $y \sim \mathcal{N}(0,1)$ it has mean $\mu^2/2$ and variance μ^2, and when $y \sim \mathcal{N}(\mu,1)$ it has mean $-\mu^2/2$ and the same variance.

The $\delta(\varepsilon)$ curve comes from the continuous form of Fact 1, $\delta(\varepsilon) = \Pr_P[L > \varepsilon] - e^{\varepsilon} \Pr_Q[L > \varepsilon]$. Write Φ for the standard normal cumulative distribution function, $\Phi(x) = \Pr[\mathcal{N}(0,1) \le x]$. Under P, $L > \varepsilon$ means a $\mathcal{N}(\mu^2/2, \mu^2)$ variable exceeds ε, which has probability $\Phi((\mu^2/2 - \varepsilon)/\mu)$; under Q the mean is $-\mu^2/2$. So $\delta(\varepsilon) = \Phi(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}) - e^{\varepsilon}\Phi(-\frac{\varepsilon}{\mu} - \frac{\mu}{2})$.

The RDP curve needs one more definition. The **Rényi divergence of order** $\alpha > 1$ measures how far apart two distributions are by a moment of the likelihood ratio: $D_\alpha(P\|Q) = \frac{1}{\alpha-1} \ln \mathbb{E}_Q[(P/Q)^\alpha] = \frac{1}{\alpha-1} \ln \mathbb{E}_P[e^{(\alpha-1)L}]$. A mechanism whose pair has $D_\alpha \le \varepsilon(\alpha)$ for all neighbours is $(\alpha, \varepsilon(\alpha))$-RDP. For a normal L with mean $\mu^2/2$ and variance μ^2, $\mathbb{E}[e^{(\alpha-1)L}] = \exp\left((\alpha-1)\frac{\mu^2}{2} + (\alpha-1)^2\frac{\mu^2}{2}\right) = \exp\left(\alpha(\alpha-1)\frac{\mu^2}{2}\right)$, using the fact that a normal variable X has $\mathbb{E}[e^{tX}] = e^{t\,\mathbb{E}[X] + t^2 \mathrm{Var}(X)/2}$. So $\varepsilon(\alpha) = \alpha\mu^2/2$.

Two more properties are used throughout. Sums of independent Gaussians are Gaussian, with means and variances adding. Releasing k independent Gaussian pairs with parameters $\mu_1, \dots, \mu_k$ is like seeing a k-dimensional normal shifted by $(\mu_1, \dots, \mu_k)$; only the direction of the shift matters, and along it the shift is $\sqrt{\mu_1^2 + \cdots + \mu_k^2}$ standard deviations. So the composition is $\sqrt{\mu_1^2 + \cdots + \mu_k^2}$-GDP (f-DP and Gaussian DP proves this for adaptively chosen steps too).

In RDP terms, hiding a shift of size a under noise of standard deviation σ is a Gaussian pair with $\mu = a/\sigma$, so it costs $\alpha\mu^2/2$:

$$R_\alpha(a) = D_\alpha(\mathcal{N}(0,\sigma^2) \,\|\, \mathcal{N}(a,\sigma^2)) = \frac{\alpha a^2}{2\sigma^2}. \tag{33.4}$$

33.3. Shuffled binary randomized response, exactly

33.3.1. Context

Each of n users holds a bit and reports it with binary randomized response: the true bit with probability $\theta = e^{\varepsilon_0}/(1 + e^{\varepsilon_0})$ (written t in Part 3), the flipped bit otherwise. Each report on its own is ε_0-DP. A report of 1 has probability θ if you hold 1 and $1 - \theta$ if you hold 0, and the reverse for a report of 0, so the likelihood ratio of any report is at most $\theta/(1-\theta) = e^{\varepsilon_0}$. A shuffler

permutes the reports before the analyst sees them. How private is the shuffled bag? For this one randomizer the question has an exact answer, and that answer is the yardstick for every general bound in the next section.

33.3.2. Statement

Once the order is gone, the only thing a bag of bits can tell the analyst is how many of them are 1. So the analyst's view is equivalent to the **number Y of 1-reports**. Your report moves that count by at most one, and the other users' reports add random noise to it. Fix the other users' bits, and let your bit be 0 in D and 1 in D'. If the other $n-1$ users all hold 0, each of them reports 1 only when their coin flips the bit, so their number of 1-reports is $S \sim \mathrm{Bin}(n-1, 1-\theta)$, with probability mass function b. The count $Y = y$ arises either when your report is 0 and the others sent y ones, or when your report is 1 and the others sent $y-1$. Weighting the two cases by the probability of your report gives

$$P(y) = \theta\, b(y) + (1-\theta)\, b(y-1), \qquad Q(y) = (1-\theta)\, b(y) + \theta\, b(y-1), \qquad y = 0, \dots, n. \quad (33.5)$$

The exact $\delta(\varepsilon)$ of shuffled randomized response for this pair is the larger of $E_{e^\varepsilon}(P\|Q)$ and $E_{e^\varepsilon}(Q\|P)$, computed by Fact 1.

33.3.3. Derivation

1. **The bag is the count.** A shuffled sequence of bits is a uniformly random arrangement of its 1s and 0s. Given the count Y, the arrangement carries no further information about anyone's data, because every arrangement with Y ones is equally likely under both D and D'. A summary with this property is a **sufficient statistic**. Fact 2 applies in both directions: the count is a function of the bag, and the bag can be rebuilt from the count by arranging Y ones and $n-Y$ zeros in a random order, which uses randomness that does not depend on the data. So the divergences of the bag equal those of Y.
2. **Split the count.** Write $Y = S + B$, where S is the number of 1-reports among the others and B is your report. S does not depend on your bit. $B \sim \mathrm{Bern}(1-\theta)$ under D, because you hold 0 and report 1 only when you flip, and $B \sim \mathrm{Bern}(\theta)$ under D'.
3. **The others' count.** A user holding 0 reports 1 with probability $1-\theta$, independently of everyone else, so the number of 1-reports among $n-1$ such users is $S \sim \mathrm{Bin}(n-1, 1-\theta)$.
4. **Condition on B.** By the law of total probability, split on the value of your report: $P(Y=y) = \Pr[B=0]\, b(y) + \Pr[B=1]\, b(y-1)$. With $\Pr[B=0] = \theta$ under D and $1-\theta$ under D', this is Equation 33.5.
5. **Other pairs.** The pair above assumes the other users all hold 0. If m of them hold 1, those m report 1 with probability θ each, so S becomes $\mathrm{Bin}(m, \theta) + \mathrm{Bin}(n-1-m, 1-\theta)$, the distribution of a sum of two independent counts (a convolution). A guarantee has to hold for the worst m. You only need to check half of them. Swapping 0 and 1 for every user, in both the bits and the reports, is a relabelling that the analyst could undo. It turns "m others hold 1, you hold 0 or 1" into "$n-1-m$ others hold 1, you hold 1 or 0", the same pair with D and D' exchanged. Since $\delta(\varepsilon)$ already takes the larger of the two directions, m and $n-1-m$ give the same ε, and $m \le (n-1)/2$ covers every pair. The script scans every such m at $n = 1000$ and $\delta = 10^{-6}$. For $\varepsilon_0 = 1$ and 4 the largest ε is at $m = 0$. For $\varepsilon_0 = 2$ it is at $m = 2$: 0.323324 against 0.323279 at $m = 0$, because the discrete count makes ε ripple

slightly in m. Large m is much better (0.2858 at $m = 499$), because then about half the others report 1 and the count is at its noisiest. At $n = 10^4$ and 10^5 the script scans $m \le 30$, and the largest value it finds exceeds the $m = 0$ value by at most 4.1×10^{-6}. The rest of the chapter uses the pair $m = 0$ and calls its ε the exact one. Strictly, it is exact for that pair and a lower bound on the worst case, within 10^{-4} of the largest value found: every entry of Table 33.4 is the same to four decimals.

A walk-through with $n = 3$. Take $\varepsilon_0 = \ln 3$, so $\theta = \frac{3}{1+3} = \frac{3}{4}$. The two other users hold 0, so each reports 1 with probability $\frac{1}{4}$, and $S \sim \mathrm{Bin}(2, \frac{1}{4})$ has $b = ((\frac{3}{4})^2, 2 \cdot \frac{1}{4} \cdot \frac{3}{4}, (\frac{1}{4})^2) = (\frac{9}{16}, \frac{6}{16}, \frac{1}{16})$. Table 33.2 fills in Equation 33.5 outcome by outcome. For example, $Q(1) = \frac{1}{4} \cdot \frac{6}{16} + \frac{3}{4} \cdot \frac{9}{16} = \frac{6+27}{64}$.

Table 33.2.: Shuffled binary randomized response with $n = 3$ users and $\varepsilon_0 = \ln 3$; the two other users hold 0.

Count y	0	1	2	3
$b(y)$, others' 1-reports	9/16	6/16	1/16	0
$P(y)$, you hold 0	27/64	27/64	9/64	1/64
$Q(y)$, you hold 1	9/64	33/64	19/64	3/64
$P(y)/Q(y)$	3.0000	0.8182	0.4737	0.3333

Now apply Fact 1 at $\varepsilon = 0.5$, where $e^{0.5} = 1.6487$. Only $y = 0$ has a ratio P/Q above 1.6487, so only it enters the sum: $E_{e^{0.5}}(P\|Q) = \frac{27-9e^{0.5}}{64} = 0.1900$. In the other direction, the ratios Q/P are the reciprocals of the last row, and $y = 2$ and $y = 3$ exceed 1.6487, so $E_{e^{0.5}}(Q\|P) = \frac{(19-9e^{0.5})+(3-e^{0.5})}{64} = 0.0861$. The other direction is smaller, so $\delta(0.5) = 0.1900$. Compare a single unshuffled report, whose pair is $(\frac{3}{4}, \frac{1}{4})$ against $(\frac{1}{4}, \frac{3}{4})$: only one output has a ratio above $e^{0.5}$, and $\delta(0.5) = \frac{3}{4} - \frac{1}{4}e^{0.5} = 0.3378$. Even two other users nearly halve δ. At $\varepsilon = 1$ the two values are 0.0396 and 0.0704. A simulation of 200,000 shuffles under D gives count frequencies 0.4226, 0.4203, 0.1413, 0.0158 against the exact 0.4219, 0.4219, 0.1406, 0.0156; the largest gap, 0.0015, is within sampling error.

33.3.4. Interpretation

Shuffling never improves pure DP. Look at the count $y = 0$. Its ratio is $P(0)/Q(0) = \theta b(0)/((1-\theta)b(0)) = e^{\varepsilon_0}$ for every n, because $b(-1) = 0$ removes the second term of both P and Q. In words: if nobody reports a 1, you were probably the one holding 0, and no number of other users changes that ratio. What changes is how often it happens. That outcome has probability θ^n under D (everyone, you included, reports 0), which is tiny for large n. So shuffling amplifies by moving the bad outcomes into δ, as the first column of Table 33.2 shows.

A central limit view. Why should the privacy fall like $1/\sqrt{n}$? Under both datasets Y is the sum of many independent bits, so by the central limit theorem it is close to normal, and Fact 3 says a pair of normals with equal variance is μ-GDP with μ equal to the mean difference divided by the standard deviation. The mean of Y differs between D and D' only through your report, by $\mathbb{E}[B'] - \mathbb{E}[B] = \theta - (1-\theta) = 2\theta - 1$, where B' is your report under D'. The variance is $n\theta(1-\theta)$

under both, since every report, yours included, is a Bernoulli variable that is 1 with probability θ or $1-\theta$, and either way has variance $\theta(1-\theta)$. Dividing,

$$\mu \approx \frac{2\theta-1}{\sqrt{n\theta(1-\theta)}} = \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0/2}\sqrt{n}} = \frac{2\sinh(\varepsilon_0/2)}{\sqrt{n}}, \tag{33.6}$$

where the middle form uses $2\theta-1 = \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}$ and $\sqrt{\theta(1-\theta)} = \sqrt{\frac{e^{\varepsilon_0}}{(e^{\varepsilon_0}+1)^2}} = \frac{e^{\varepsilon_0/2}}{e^{\varepsilon_0}+1}$, so that the factors $e^{\varepsilon_0}+1$ cancel. The last form divides top and bottom by $e^{\varepsilon_0/2}$: $\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0/2}} = e^{\varepsilon_0/2}-e^{-\varepsilon_0/2} = 2\sinh(\varepsilon_0/2)$. The privacy of shuffled randomized response falls like $1/\sqrt{n}$, with an ε_0-dependent constant that grows like $e^{\varepsilon_0/2}$.

When the approximation fails. The normal approximation is good when there are many 1-reports to hide among. At $n = 10^5$ and $\varepsilon_0 = 2$ it gives $\varepsilon = 0.0244$ against the exact 0.0247 (Table 33.4). At $n = 1000$ and $\varepsilon_0 = 4$, each of the 999 others reports 1 with probability $1-\theta = 1/(1+e^4)$, so on average they send only $999/(1+e^4) = 18.0$ 1-reports. A count that small is far from normal, and the approximation gives 0.9667 against the exact 1.6910. Use the exact computation, which costs milliseconds.

Figure 33.1 draws the two count distributions and the exact $\delta(\varepsilon)$ curves.

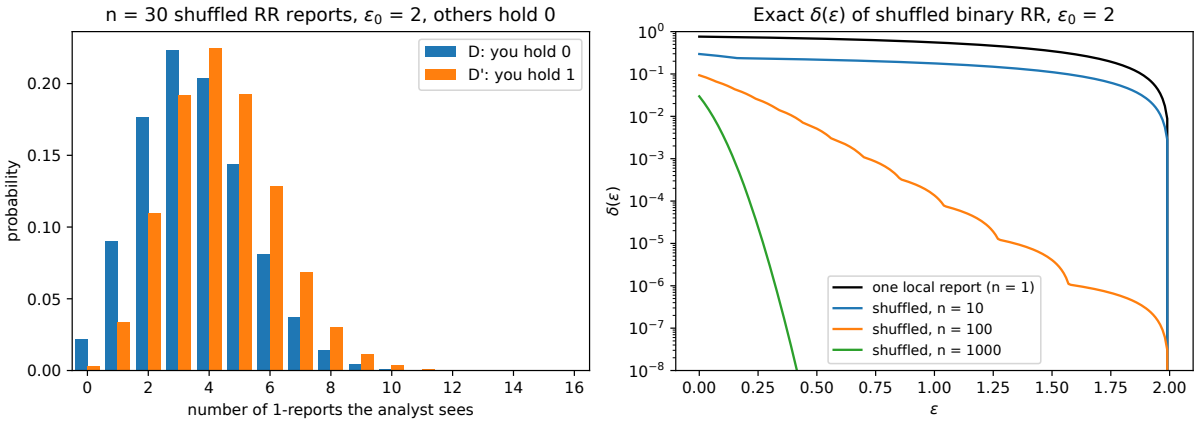


Figure 33.1.: Left: the distribution of the number of 1-reports among $n = 30$ shuffled randomized-response reports with $\varepsilon_0 = 2$, when you hold 0 (blue) and when you hold 1 (orange); the other users hold 0. Your report moves the count by at most one, against a spread of about $\sqrt{n\theta(1-\theta)}$. Right: the exact $\delta(\varepsilon)$ for one unshuffled report and for $n = 10, 100, 1000$ shuffled reports. All curves reach 0 only at $\varepsilon = \varepsilon_0$, but for large n they are negligible far earlier.

33.3.5. Example: a survey of 10,000 people

A company asks 10,000 users whether they used a feature, through randomized response with $\varepsilon_0 = 2$ and a shuffler. Each user's report alone is 2-DP, a weak guarantee: the truth is reported with probability $\theta = e^2/(1+e^2) = 0.8808$, so a single report is the truth seven times out of eight. The shuffled bag is $(0.0870, 10^{-6})$-DP (Table 33.4). The shuffler bought central-model privacy without anyone seeing raw data.

33.3.6. Check yourself

- Why does the analysis of the count not depend on the order in which reports arrive? *(The shuffler outputs a uniformly random order, which carries no information. The count is sufficient for the bag.)*
- With $n = 3$ and $\varepsilon_0 = \ln 3$, what is the smallest ε for which $\delta(\varepsilon) = 0$? *(It is* $\ln 3$*: the outcome* $y = 0$ *has likelihood ratio 3, so for any smaller* ε *it contributes a positive amount to Fact 1's sum. Shuffling never lowers the pure-DP level.)*

33.4. The clone bound for any local randomizer

33.4.1. Context

Randomized response on bits is one randomizer. In practice users send histograms, k-ary responses or noisy vectors, and the shuffle is sometimes applied to adaptively chosen randomizers, as in shuffled stochastic gradient descent (SGD), where each user randomizes their own gradient before it is shuffled. You want one bound that holds for **every** ε_0-DP local randomizer, that is, every randomized map R from a user's data to a report whose output probabilities change by at most a factor e^{ε_0} when the input changes. Erlingsson et al. (2019) gave the first such bound; Balle et al. (2019) gave another with the "privacy blanket" idea, which writes each randomizer as a mixture of an input-independent part and the rest. The bound to use today is Feldman, McMillan & Talwar (2021), and its proof is short enough to follow.

33.4.2. Statement

The bound says that shuffling turns ε_0-local privacy into a central ε that shrinks like $1/\sqrt{n}$, with a constant that grows like $e^{\varepsilon_0/2}$ and a factor $\sqrt{\ln(1/\delta)}$ for the slack. It needs enough users relative to e^{ε_0}, for a reason the derivation makes clear.

Theorem 3.1 of Feldman, McMillan & Talwar (2021). Let each user apply an ε_0-DP local randomizer (possibly chosen adaptively from the previous outputs) to the data in a uniformly random order. For any $\delta \in [0, 1]$ with $\varepsilon_0 \le \ln\left(\frac{n}{16\ln(2/\delta)}\right)$, the output is (ε, δ)-DP with

$$\varepsilon \le \ln\left(1 + \frac{e^{\varepsilon_0} - 1}{e^{\varepsilon_0} + 1}\left(\frac{8\sqrt{e^{\varepsilon_0}\ln(4/\delta)}}{\sqrt{n}} + \frac{8e^{\varepsilon_0}}{n}\right)\right). \tag{33.7}$$

The theorem rests on a reduction, which is the useful part. Any ε_0-DP randomizer, run on any other user's data, produces with probability $e^{-\varepsilon_0}$ a report that looks exactly like yours could, a **clone**. Once the order is hidden, the adversary is left counting how many reports look like your D-version and how many look like your D'-version, and the clones blur that count. To state it, let C be the number of other users who act as clones, A the number of those clones that look like your D-version, and Δ a bit that says whether your own report points the right way.

Theorem 3.2 of the same paper (the reduction). Let $C \sim \mathrm{Bin}(n-1, e^{-\varepsilon_0})$, $A \sim \mathrm{Bin}(C, \frac{1}{2})$ and $\Delta \sim \mathrm{Bern}(\frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+1})$, independent. The shuffled output on D and on D' is a post-processing, by one and the same randomized map, of

$$\tilde{P} = (A + \Delta,\ C - A + 1 - \Delta) \qquad \text{and} \qquad \tilde{Q} = (A + 1 - \Delta,\ C - A + \Delta). \tag{33.8}$$

In $\tilde{P}$ your report adds to the first count when $\Delta = 1$, and in $\tilde{Q}$ it adds to the second. By Fact 2, any (ε, δ) that holds for this pair of two-dimensional counts holds for the shuffled output. Computing the pair's $\delta(\varepsilon)$ exactly, with Fact 1, gives the **numeric clone bound**; Equation 33.7 is an analytic upper bound on it.

33.4.3. Derivation

Steps 1 to 4 prove the reduction, and step 1 carries the idea. Step 5 bounds the resulting pair.

1. **Every report contains a copy of yours.** Fix the two values x^0 (in D) and x^1 (in D') that you might hold, and let $R(x)$ be the distribution of the report of a user holding x. Being ε_0-DP means that no output is more than e^{ε_0} times likelier for one input than for another. Read from the other side, every output of $R(x)$ keeps at least a fraction $e^{-\varepsilon_0}$ of its probability under $R(x^0)$, and likewise for x^1: $R(x)(y) \geq e^{-\varepsilon_0} R(x^0)(y)$ and $R(x)(y) \geq e^{-\varepsilon_0} R(x^1)(y)$ for every output y. Average the two inequalities: $R(x) \geq \frac{e^{-\varepsilon_0}}{2}(R(x^0) + R(x^1))$ pointwise. The difference between the two sides is non-negative, and its total mass is $1 - \frac{e^{-\varepsilon_0}}{2}(1+1) = 1 - e^{-\varepsilon_0}$, so it is $(1 - e^{-\varepsilon_0})$ times a distribution $\mathrm{LO}(x)$, the **leftover**:
$$R(x) = \frac{e^{-\varepsilon_0}}{2} R(x^0) + \frac{e^{-\varepsilon_0}}{2} R(x^1) + (1 - e^{-\varepsilon_0})\mathrm{LO}(x). \tag{33.9}$$
2. **Read the mixture as a three-sided coin.** A mixture is sampled by first choosing a component with its weight, then drawing from that component. So sampling from Equation 33.9 means: flip a coin that says "0" with probability $e^{-\varepsilon_0}/2$, "1" with probability $e^{-\varepsilon_0}/2$, "leftover" otherwise; then report a draw from $R(x^0)$, $R(x^1)$ or $\mathrm{LO}(x)$. A user whose coin says "0" sends a report distributed exactly like yours on D: a **clone**. Each of the other $n-1$ users is a clone (face "0" or "1") with probability $e^{-\varepsilon_0}$, independently, so the number of clones is $C \sim \mathrm{Bin}(n-1, e^{-\varepsilon_0})$. Each clone is a 0-clone or a 1-clone with probability $\frac{1}{2}$, so $A \sim \mathrm{Bin}(C, \frac{1}{2})$ of them are 0-clones.
3. **After shuffling, only the counts of the coin faces matter.** Generate the output in two stages. First flip every other user's three-sided coin; then draw each report given its face. A user whose face is 0 or 1 draws from $R(x^0)$ or $R(x^1)$, which does not depend on who they are. A user whose face is "leftover" draws from $\mathrm{LO}(x_i)$, which depends only on that user's own data x_i, not on yours. Give yourself the face 0 on D and 1 on D', since your report is a draw from $R(x^0)$ or $R(x^1)$. After the shuffle nobody can tell whose report is whose. So someone who knows every other user's data, but not your bit, can produce the shuffled bag from just the number of 0-faces and 1-faces: draw that many reports from $R(x^0)$ and $R(x^1)$, draw a leftover report for every other user, and shuffle (Lemma 3.3 of the paper makes this precise). Those numbers are $(A+1, C-A)$ on D and $(A, C-A+1)$ on D'. By Fact 2, the output is no more distinguishable than these counts.
4. **Your own report is a randomized response.** The counts of step 3 still give your face away exactly, by one. The last improvement uses the fact that your own report is itself noisy. Any ε_0-DP randomizer restricted to two inputs is a post-processing of binary randomized

response (Kairouz, Oh & Viswanath 2015, Lemma 3.4 of the paper). Concretely, $R(x^0)$ and $R(x^1)$ are mixtures of two distributions Q_0, Q_1 with weights $(\frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+1}, \frac{1}{e^{\varepsilon_0}+1})$ and the reverse: your report is drawn from the distribution "on your side" with probability $\frac{e^{\varepsilon_0}}{e^{\varepsilon_0}+1}$ and from the other side otherwise, exactly like a randomized-response bit. Replacing your face by that randomized-response bit Δ gives Equation 33.8. This is where the factor $\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}$ in Equation 33.7 comes from: it is $\Pr[\Delta = 1] - \Pr[\Delta = 0]$, how much more often randomized response points the right way than the wrong way. Equivalently, randomized response passes on your bit only with that probability, and outputs a fair coin otherwise.

5. **Bound the pair.** The last step bounds $E_{e^\varepsilon}(\tilde{P}\|\tilde{Q})$ with binomial concentration (Chernoff bounds on C and on A given C; Lemma 3.5 and Appendix A of the paper). These bounds need many clones to hide among: the condition on ε_0 is the same as $ne^{-\varepsilon_0} \geq 16\ln(2/\delta)$, an expected number of clones large enough for C to be close to its mean except with probability about δ. We do not repeat the calculation; the script computes the pair's $\delta(\varepsilon)$ exactly instead.

Step 1 on a concrete randomizer. Binary randomized response with $\varepsilon_0 = \ln 3$ has $R(0) = (\frac{3}{4}, \frac{1}{4})$ and $R(1) = (\frac{1}{4}, \frac{3}{4})$ on outputs $(0, 1)$, and $e^{-\varepsilon_0} = \frac{1}{3}$, so each clone weight is $\frac{e^{-\varepsilon_0}}{2} = \frac{1}{6}$ and the leftover weight is $1 - \frac{1}{3} = \frac{2}{3}$. Take another user who holds $x = 0$, with your values $x^0 = 0$, $x^1 = 1$. Table 33.3 decomposes this user's report.

Table 33.3.: The clone decomposition Equation 33.9 for binary randomized response with $\varepsilon_0 = \ln 3$, for a user holding 0. With probability $\frac{1}{6}$ the user's report is a draw from $R(0)$, with probability $\frac{1}{6}$ a draw from $R(1)$, and otherwise a draw from the leftover.

Output y	$R(0)(y)$	$\frac{1}{6}R(x^0)(y)$	$\frac{1}{6}R(x^1)(y)$	$\frac{2}{3}\mathrm{LO}(0)(y)$	$\mathrm{LO}(0)(y)$
0	3/4	1/8	1/24	7/12	7/8
1	1/4	1/24	1/8	1/12	1/8
Total	1	1/6	1/6	2/3	1

The fourth column is what remains of $R(0)$ after the two clone parts are taken out, and the last column rescales it by $\frac{3}{2}$ to a distribution. Each row adds up: $\frac{1}{8} + \frac{1}{24} + \frac{7}{12} = \frac{3}{4}$. The script prints the clone part $(0.1667, 0.1667)$ and $\mathrm{LO}(0) = (0.875, 0.125)$.

33.4.4. Interpretation

Why $1/\sqrt{n}$ and $e^{\varepsilon_0/2}$. The central limit view of the previous section explains the shape of the bound. Given $C = c$ clones, the first coordinates of $\tilde{P}$ and $\tilde{Q}$ are $A + \Delta$ and $A + 1 - \Delta$. Their means differ by $\mathbb{E}[\Delta] - \mathbb{E}[1 - \Delta] = 2\Pr[\Delta = 1] - 1 = \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}$. The noise that hides this difference is the spread of $A \sim \mathrm{Bin}(c, \frac{1}{2})$, a standard deviation of $\sqrt{c \cdot \frac{1}{2} \cdot \frac{1}{2}} = \sqrt{c}/2$. The expected number of clones is $c \approx ne^{-\varepsilon_0}$, and dividing the mean difference by the spread, as Fact 3 does for a Gaussian pair, gives $\mu \approx \frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1} \cdot \frac{2}{\sqrt{ne^{-\varepsilon_0}}}$. That is the shape of Equation 33.7: $\varepsilon = O(\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0}+1}\sqrt{e^{\varepsilon_0}\ln(1/\delta)/n})$. For $\varepsilon_0 \leq 1$ the first factor is about $\varepsilon_0/2$ and e^{ε_0} is at most e, so this is of order $\varepsilon_0\sqrt{\ln(1/\delta)/n}$; above 1 the first factor tends to 1, leaving $\sqrt{e^{\varepsilon_0}\ln(1/\delta)/n}$.

What the general bound gives up. Divide the clone μ by the randomized-response μ of Equation 33.6, written as $\frac{e^{\varepsilon_0}-1}{e^{\varepsilon_0/2}\sqrt{n}}$. The factors $e^{\varepsilon_0} - 1$ and $\sqrt{n}$ cancel, and what is left is

$\frac{2e^{\varepsilon_0/2}}{e^{\varepsilon_0}+1} \cdot e^{\varepsilon_0}/2 = \frac{2e^{\varepsilon_0}}{e^{\varepsilon_0}+1} = \frac{2}{1+e^{-\varepsilon_0}}$. The ratio is above 1 for a clear reason. In binary randomized response every other report is a bit that could be yours, while the clone argument counts only the $e^{-\varepsilon_0}$ fraction of reports guaranteed to look like yours, split into two halves. The script gives this ratio as 1.46, 1.76 and 1.96 for $\varepsilon_0 = 1, 2, 4$; for instance $2e/(e+1) = 1.46$. The measured ratios of the numeric clone bound to the exact ε in Table 33.4 are 1.44 to 2.00 wherever the expected number of clones $(n-1)e^{-\varepsilon_0}$ is large, close to this prediction. At $n = 1000$ and $\varepsilon_0 = 4$ there are only about $999e^{-4} = 18.3$ expected clones, far from the central limit regime, and the ratio is 2.36. No general bound can go below the exact column, since binary randomized response is one of the randomizers it must cover.

The closed form is loose by another factor of 3 to 5. Equation 33.7 comes from Chernoff bounds with constant 8, which are safe but not sharp. Where its condition holds in Table 33.4, it is 3.10 to 4.75 times the numeric clone bound and 4.47 to 7.55 times the exact ε. Use it to reason about scaling; compute the numeric bound to deploy.

The condition matters. Below $ne^{-\varepsilon_0} = 16\ln(2/\delta)$ there may be too few clones, and Equation 33.7 says nothing. Solving for ε_0 gives the largest allowed level, $\ln\left(n/(16\ln(2/\delta))\right)$. At $\delta = 10^{-6}$, $16\ln(2 \times 10^6) = 232.1$, so ε_0 must be at most $\ln(1000/232.1) = 1.460$ for $n = 1000$, 3.763 for $n = 10^4$ and 6.066 for $n = 10^5$.

The first bound was much weaker. Erlingsson et al. (2019, Theorem 7) prove $\varepsilon \le \varepsilon_1\sqrt{2n\ln(1/\delta)} + n\varepsilon_1(e^{\varepsilon_1} - 1)$ with $\varepsilon_1 = 2e^{2\varepsilon_0}(e^{\varepsilon_0} - 1)/n$, through a swap argument, subsampling and advanced composition. Substituting ε_1 into the first term, its leading term is $2e^{2\varepsilon_0}(e^{\varepsilon_0} - 1)\sqrt{2\ln(1/\delta)/n}$, so its dependence on ε_0 is $e^{3\varepsilon_0}$ instead of $e^{\varepsilon_0/2}$, a factor $e^{2.5\varepsilon_0}$ worse. In Table 33.4 it is below ε_0, that is, it amplifies at all, only for $n = 10^5$ and $\varepsilon_0 = 1$, where it gives 0.4285.

Table 33.4.: Central ε at $\delta = 10^{-6}$ after shuffling n reports that are each ε_0-DP. "n/a" marks where the condition of Equation 33.7 fails; "—" marks where the bound of Erlingsson et al. is at least ε_0. The $n = 10^4$ rows agree with the table in Trust models.

n	ε_0	Exact, binary randomized response	Normal approx., Equation 33.6	Clones, numeric	Clones, closed form	Erlingsson et al.
1,000	1	0.1266	0.1212	0.1824	0.5662	—
1,000	2	0.3233	0.2890	0.5455	n/a	—
1,000	4	1.6910	0.9667	3.9900	n/a	—
10,000	1	0.0357	0.0352	0.0530	0.2140	—
10,000	2	0.0870	0.0844	0.1550	0.5009	—
10,000	4	0.3146	0.2816	0.6009	n/a	—
100,000	1	0.0101	0.0101	0.0153	0.0726	0.4285
100,000	2	0.0247	0.0244	0.0450	0.1862	—
100,000	4	0.0847	0.0822	0.1698	0.5346	—

Figure 33.2 draws the same comparison over n. All curves fall like $1/\sqrt{n}$ once n is large; the constants are what separates them. Multiplying by $\sqrt{n}$ removes the common $1/\sqrt{n}$ and leaves the constant: the exact ε times $\sqrt{n}$ is 3.566, 8.702 and 31.464 at $n = 10^4$ for $\varepsilon_0 = 1, 2, 4$, growing with ε_0 roughly as $2\sinh(\varepsilon_0/2)$ in Equation 33.6 does.

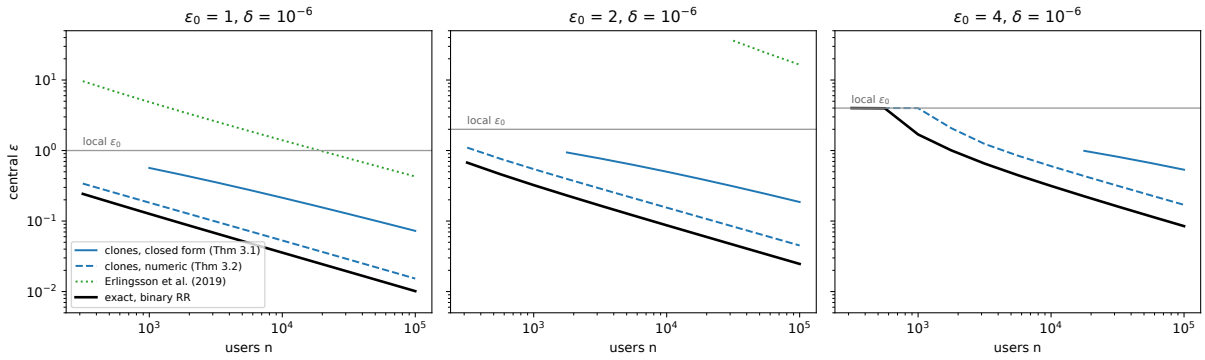


Figure 33.2.: Central ε at $\delta = 10^{-6}$ against the number of users n, for $\varepsilon_0 = 1, 2, 4$: the closed-form clone bound (drawn only where its condition holds), the numeric clone bound of Theorem 3.2, the bound of Erlingsson et al. (drawn where it is below 50, so it appears only for large n when $\varepsilon_0 = 2$ and not at all when $\varepsilon_0 = 4$, where it exceeds 7×10^6 for every n shown) and the exact ε of shuffled binary randomized response. The grey line is the local ε_0.

33.4.5. Example: how large may each user's ε_0 be?

You collect a bit from $n = 10^5$ users behind a shuffler and want central $\varepsilon \leq 0.2$ at $\delta = 10^{-6}$. Of the values in Table 33.4, the closed form allows $\varepsilon_0 = 2$ (0.1862), the numeric clone bound allows $\varepsilon_0 = 4$ (0.1698), and for binary randomized response the exact value at $\varepsilon_0 = 4$ is 0.0847. A larger ε_0 means less noise per report, so it matters what $\varepsilon_0 = 4$ buys. Each report is the truth with probability $\theta = e^4/(1+e^4) = 0.9820$. Suppose the users are a random sample from a population in which a proportion p hold 1. A report is then 1 with probability $q = p\theta + (1-p)(1-\theta) = (1-\theta) + p(2\theta - 1)$, so the fraction $\bar{Y}$ of 1-reports estimates q, and solving for p gives the debiased estimate $\hat{p} = (\bar{Y} - (1-\theta))/(2\theta - 1)$. Its variance is $q(1-q)/(n(2\theta-1)^2)$, and expanding $q(1-q) = (2\theta-1)^2 p(1-p) + \theta(1-\theta)$ gives $(p(1-p) + \frac{\theta(1-\theta)}{(2\theta-1)^2})/n$. The first term is the ordinary sampling variance, which you would pay without any privacy. The term added by the noise is $\frac{0.9820 \times 0.0180}{0.9640^2} = 0.0190$, against up to 0.25 for the sampling term $p(1-p)$, so the noise costs little accuracy. Which accountant you use decides whether a deployment is feasible.

33.4.6. Check yourself

- In step 1, why average the two inequalities instead of using just one? *(Using only x^0 would give clones of D's version of you but none of D''s. The analyst could then count reports that look like x^1. Splitting the clones evenly between the two makes the situation symmetric.)*
- Why does the analysis need a uniformly random order, rather than a fixed one? *(With a fixed order the analyst knows which report is yours, and the guarantee falls back to ε_0. The random order is what lets the faces be described by their counts.)*

33.5. Amplification by iteration

33.5.1. Context

Projected noisy SGD makes one pass over the data: at step t it takes a gradient step on record x_t, adds Gaussian noise, and projects back onto a convex set of allowed parameters. If every iterate is released, the record x_t is protected only by the noise of step t, as in one Gaussian mechanism: a value with bounded sensitivity plus Gaussian noise. If only the final iterate w_n is released, the noise of steps $t+1, \dots, n$ also covers x_t. Feldman, Mironov, Talwar & Thakurta (2018) turned this into a guarantee. The obstacle is that the steps in between are arbitrary maps applied to random points, not additions, so you cannot just add up variances as for a sum of Gaussians. Their tool is a divergence that allows for a bounded shift.

33.5.2. Statement

The argument needs every step to be a map that never pushes two points further apart. A **contractive noisy iteration** is $X_t = \psi_t(X_{t-1}) + Z_t$, where each ψ_t is a **contraction**, $\|\psi_t(u) - \psi_t(v)\| \le \|u - v\|$, and $Z_t \sim \mathcal{N}(0, \sigma^2 I)$ is fresh noise. Two kinds of maps in noisy SGD have this property.

A gradient step $w \mapsto w - \eta \nabla f(w)$ on a convex β-smooth f is a contraction when $\eta \le 2/\beta$ (Stability and generalization, Fact 4). The reason is that for such f the gradient is **co-coercive**: $\langle \nabla f(u) - \nabla f(v), u - v \rangle \ge \frac{1}{\beta} \|\nabla f(u) - \nabla f(v)\|^2$, so the gradients of two points differ in a direction that brings them together. Writing $g = \nabla f(u) - \nabla f(v)$ and expanding the square, $\|u - v - \eta g\|^2 = \|u - v\|^2 - 2\eta \langle g, u - v \rangle + \eta^2 \|g\|^2 \le \|u - v\|^2 - \eta(\frac{2}{\beta} - \eta)\|g\|^2$, which is at most $\|u - v\|^2$ when $\eta \le 2/\beta$.

The projection Π onto a closed convex set K, which maps each point to the nearest point of K, is a contraction too. The proof of Fact 3 of Optimization shows that the nearest point makes an obtuse angle with every other point of K: $\langle u - \Pi u, w - \Pi u \rangle \le 0$ for every $w \in K$. Take $w = \Pi v$, then exchange u and v, and add the two inequalities: $\|\Pi u - \Pi v\|^2 \le \langle u - v, \Pi u - \Pi v \rangle \le \|u - v\| \, \|\Pi u - \Pi v\|$ by Cauchy–Schwarz, so $\|\Pi u - \Pi v\| \le \|u - v\|$. A composition of contractions is a contraction, so a projected gradient step is one too.

Now the divergence. The two runs, on D and on D', are driven apart at the step that uses your record, and after that the contractions keep the gap from growing while the noise blurs it. The quantity to track is therefore how far apart the two runs might be (a **shift**) and how much divergence would remain if you were allowed to move one of them by that shift. To move a random variable by at most z means to pair each of its values with a value at most z away; such a pairing of two random variables is a **coupling**. The **shifted Rényi divergence** allows the first distribution to be moved by at most z in this way before it is compared, with the Rényi divergence D_α of Fact 3:

$$D_\alpha^{(z)}(X \| X') = \inf \{ D_\alpha(Y \| X') : \text{some coupling of } X \text{ and } Y \text{ has } \|X - Y\| \le z \text{ almost surely} \}. \tag{33.10}$$

With $z = 0$ no move is allowed, and it is D_α. Two lemmas drive everything (Feldman et al. 2018, Lemmas 20 and 21): noise lets you pay off part of the shift, and contractions only add the gap between the two maps to the shift, at no cost.

Shift reduction. Adding noise lets you buy down the shift. Let $R_\alpha(a)$ be the largest D_α between the noise and a copy of it shifted by at most a; for Gaussian Z it is Equation 33.4. Adding Z to both sides lowers the allowed shift by a at a price $R_\alpha(a)$: for any $a \geq 0$,

$$D_\alpha^{(z)}(X + Z \,\|\, X' + Z) \leq D_\alpha^{(z+a)}(X \| X') + R_\alpha(a). \tag{33.11}$$

Contraction. Applying contractions ψ, ψ' to the two sides cannot move points apart, but the two maps themselves may differ (they do at the step that uses your record). If they differ by at most s at every point, $\sup_u \|\psi(u) - \psi'(u)\| \leq s$, the shift needed grows by at most s and the divergence does not grow:

$$D_\alpha^{(z+s)}(\psi(X) \,\|\, \psi'(X')) \leq D_\alpha^{(z)}(X \| X'). \tag{33.12}$$

Chaining the two lemmas over T steps gives their Theorem 22. Let the two runs differ by $s_t = \sup_u \|\psi_t(u) - \psi'_t(u)\|$ at step t, and choose payments $a_1, \ldots, a_T \geq 0$ such that the unpaid shift $z_t = \sum_{i \leq t}(s_i - a_i)$ never goes negative and is 0 at the end, $z_T = 0$. Then $D_\alpha(X_T \| X'_T) \leq \sum_t R_\alpha(a_t)$: the divergence of the last iterate is at most the total price of the payments. For noisy SGD this becomes the following.

Theorem 23 of Feldman et al. (2018). Let the losses $f(\cdot, x)$ be convex, L_f-Lipschitz and β-smooth on a convex set K, and run projected noisy SGD $w_t = \Pi_K(w_{t-1} - \eta(\nabla f(w_{t-1}, x_t) + Z_t))$, $Z_t \sim \mathcal{N}(0, \sigma^2 I)$, $\eta \leq 2/\beta$, once over $x_1, \ldots, x_n$. Releasing only w_n is $(\alpha, \frac{2\alpha L_f^2}{\sigma^2(n-t+1)})$-RDP for the record x_t, for every $\alpha > 1$.

The guarantee is easier to read in terms of the **noise multiplier** $\tilde{\sigma} = \sigma/(2L_f)$, the noise per unit of the gradient's replace-one sensitivity. That sensitivity is $2L_f$ because an L_f-Lipschitz loss has gradients of norm at most L_f, so the gradients on x_t and x'_t differ by at most $2L_f$. Substituting $\sigma^2 = 4L_f^2\tilde{\sigma}^2$, the RDP of the record at step t reads

$$\varepsilon_t(\alpha) = \frac{\alpha}{2\tilde{\sigma}^2} \cdot \frac{1}{n - t + 1} : \tag{33.13}$$

one step's Gaussian RDP curve, divided by the number of noisy steps from t to the end.

33.5.3. Derivation

Shift reduction for one Gaussian step, concretely. The lemma is easiest to see on two point masses. Let $X = 0$ and $X' = z$, a distance z apart, and add $Z \sim \mathcal{N}(0, \sigma^2)$ to both.

1. With no shift allowed, the two outputs are $\mathcal{N}(0, \sigma^2)$ and $\mathcal{N}(z, \sigma^2)$, a Gaussian pair with $\mu = z/\sigma$, so $D_\alpha = \frac{\alpha z^2}{2\sigma^2}$ (Fact 3).
2. Now allow a shift of $z - a$, and use it: move the first output by $z - a$ towards the second. That is, $Y = X + Z + (z - a) \sim \mathcal{N}(z - a, \sigma^2)$, coupled to $X + Z$ at distance exactly $z - a$. Only a of the gap is left, so $D_\alpha(Y \| X' + Z) = \frac{\alpha a^2}{2\sigma^2}$ by Fact 3 again. Hence $D_\alpha^{(z-a)}(X + Z \| X' + Z) \leq R_\alpha(a)$.
3. Now read Equation 33.11 with shift $z - a$ on the left. Its right side is $D_\alpha^{(z)}(X \| X') + R_\alpha(a)$, and $D_\alpha^{(z)}(X \| X') = 0$ because moving X by z turns it into X'. So the lemma gives exactly the bound of step 2. The noise **pays for a of the shift at price** $\alpha a^2/(2\sigma^2)$; the rest of the shift is carried forward. The general proof (Lemma 20 of the paper) does the same with a coupling $U + W$, $\|W\| \leq a$, and bounds the Rényi divergence of the joint pair $(U + W, Z - W)$.

Spreading the shift. Now let the two runs differ only at step t, by s, and apply identity maps afterwards ($\psi_k(u) = u$, a contraction). You decide how much shift to pay at each later step, $a_t, \dots, a_T$ with $\sum_k a_k = s$. Since the price is quadratic in each payment, small equal payments should be cheapest.

1. By Equation 33.12 at step t, the shift grows from 0 to s. By Equation 33.11 at each step $k \geq t$ it shrinks by a_k at cost $R_\alpha(a_k)$. At step T it is 0, so the shifted divergence is the ordinary D_α.
2. The total cost is $\frac{\alpha}{2\sigma^2}\sum_k a_k^2$. Cauchy–Schwarz, applied to the vectors $(a_t, \dots, a_T)$ and $(1, \dots, 1)$ of length $T - t + 1$, gives $(\sum_k a_k)^2 \leq (T - t + 1)\sum_k a_k^2$, that is, $\sum_k a_k^2 \geq \frac{(\sum_k a_k)^2}{T-t+1}$, with equality for the even schedule $a_k = s/(T - t + 1)$.
3. So $D_\alpha(X_T \| X'_T) \leq \frac{\alpha s^2}{2\sigma^2(T-t+1)}$.
4. **The general induction (Theorem 22).** With arbitrary contractions in between, write $z_k = \sum_{i \leq k}(s_i - a_i)$ for the shift still unpaid after step k. The claim is that the divergence, with the unpaid shift allowed, is at most what has been paid so far: $D_\alpha^{(z_k)}(X_k \| X'_k) \leq \sum_{i \leq k} R_\alpha(a_i)$. It holds at $k = 0$, where both runs start at the same point. For the step from k to $k + 1$, write the new iterates as the maps plus noise, peel off the noise with shift reduction (paying a_{k+1}), rewrite the shift using $z_{k+1} + a_{k+1} = z_k + s_{k+1}$, and peel off the maps with the contraction lemma:

$$\begin{aligned} D_\alpha^{(z_{k+1})}(X_{k+1} \| X'_{k+1}) &= D_\alpha^{(z_{k+1})}(\psi_{k+1}(X_k) + Z \,\|\, \psi'_{k+1}(X'_k) + Z) \\ &\leq D_\alpha^{(z_{k+1}+a_{k+1})}(\psi_{k+1}(X_k) \,\|\, \psi'_{k+1}(X'_k)) + R_\alpha(a_{k+1}) && \text{(shift reduction)} \\ &= D_\alpha^{(z_k+s_{k+1})}(\psi_{k+1}(X_k) \,\|\, \psi'_{k+1}(X'_k)) + R_\alpha(a_{k+1}) && \text{(definition of } z_{k+1}) \\ &\leq D_\alpha^{(z_k)}(X_k \| X'_k) + R_\alpha(a_{k+1}) && \text{(contraction).} \end{aligned} \tag{33.14}$$

   Applying the claim for step k to the last line proves it for step $k + 1$. At $k = T$ the shift is $z_T = 0$, and the shifted divergence is the ordinary one. The contractions never enter the cost: they can only keep the shift from growing beyond $z_k + s_{k+1}$.
5. For noisy SGD, the steps on x_t and x'_t differ by $s = \eta\|\nabla f(w, x_t) - \nabla f(w, x'_t)\| \leq 2\eta L_f$, since each gradient has norm at most L_f (Lipschitz). The update subtracts ηZ_t, so the noise in parameter space has standard deviation $\eta\sigma$. Step 3, with the induction of step 4 in place of the identity maps and n in place of T, gives $\frac{\alpha(2\eta L_f)^2}{2\eta^2\sigma^2(n-t+1)} = \frac{2\alpha L_f^2}{\sigma^2(n-t+1)}$, which is Theorem 23. The step size η cancels, because it scales the shift and the noise alike.

Table 33.5 walks through steps 1 to 3 with $s = 1$, $\sigma = 1$, $\alpha = 2$ and $T - t + 1 = 4$ noisy steps. Each step pays $a = \frac{1}{4}$ at price $2 \cdot (\frac{1}{4})^2/2 = 0.0625$.

Table 33.5.: Paying off a shift of 1 over four Gaussian steps with $\sigma = 1$, at $\alpha = 2$. The total equals $D_2(\mathcal{N}(0, 4)\|\mathcal{N}(1, 4)) = 0.25$.

Step	1	2	3	4	Total
Shift before the step	1	0.75	0.5	0.25	
Shift paid, a_k	0.25	0.25	0.25	0.25	1
Cost $\alpha a_k^2/(2\sigma^2)$	0.0625	0.0625	0.0625	0.0625	0.2500

The total matches the exact divergence because four independent steps of variance 1 add up to variance 4, and by Fact 3 a shift of 1 under variance 4 costs $\frac{2\cdot 1^2}{2\cdot 4} = 0.25$. Uneven schedules cost more, because squares punish large payments: $(0.8, 0.2, 0, 0)$ costs $0.8^2 + 0.2^2 = 0.6800$ (with $\alpha/(2\sigma^2) = 1$), and paying everything at once costs 1. Figure 33.3 draws the schedule and its cost as the number of steps grows.

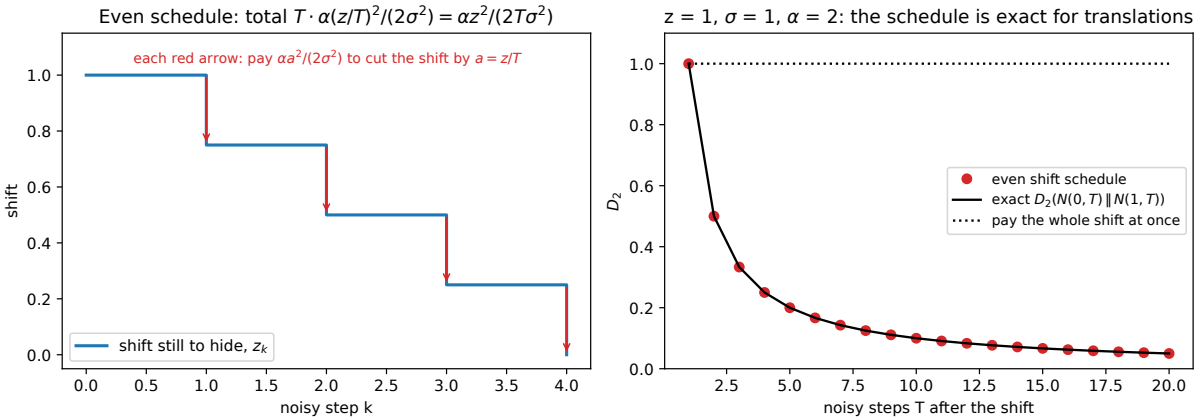


Figure 33.3.: Left: the even shift schedule over $T = 4$ noisy steps; each red arrow is a payment of $a = z/T$ at price $\alpha a^2/(2\sigma^2)$. Right: the total cost of the even schedule (dots) against the exact $D_2(\mathcal{N}(0,T)\|\mathcal{N}(1,T))$ (line), for a shift $z = 1$ and $\sigma = 1$; the dotted line is the cost of paying the whole shift in one step.

33.5.4. Interpretation

Exact for translations from a point. The even schedule loses nothing when the maps after step t are translations (they move every point by the same vector, so they keep the shift exactly as it is) and the two runs start from the same point at step t. Then the last iterate on D and D' is a pair of Gaussians shifted by s with variance $(T - t + 1)\sigma^2$ added after the shift, whose exact D_α is $\frac{\alpha s^2}{2(T-t+1)\sigma^2}$ by Fact 3. That is exactly the even schedule's cost, for every T (Figure 33.3, right; the script checks $T = 1, 2, 4, 10$).

Two things Theorem 23 ignores. Noise added **before** step t also hides the shift when the maps preserve it, and a **strict** contraction, one that brings points closer, shrinks the shift for free. Figure 33.4 compares the bound with the exact μ of a one-dimensional run, $w_k = c\, w_{k-1} + b_k + \mathcal{N}(0, \sigma^2)$, in which $c \le 1$ is the contraction factor and your record shifts only the offset b_t, by $s = \sigma$. Unrolling the run as in Section 33.7, the last iterate is Gaussian, your shift reaches the end damped to $c^{T-t}s$, and the noise has standard deviation $\sigma\sqrt{\sum_{j=0}^{T-1} c^{2j}}$; their ratio is the exact μ (Fact 3). For $c = 1$ the shift persists unchanged to the end and all T noise draws pile up on top of it, so the exact μ is $\frac{s}{\sigma\sqrt{T}} = 1/\sqrt{T} = 0.1000$ at every position, with $T = 100$. The bound, $1/\sqrt{T - t + 1}$, counts only the noise after step t and rises to 1 at $t = T$. For $c = 0.95$ the exact μ is far smaller (0.0240 at $t = 50$ against the bound's $1/\sqrt{51} = 0.1400$).

When it is exact. The bound is attained when step t first maps every point to a single point, which is a contraction, so that the earlier noise is erased. Then only the noise from step t on

hides the shift, and the exact μ is $s/(\sigma\sqrt{T-t+1})$, which matches Theorem 23 in its μ form. So Theorem 23 is exact in the worst case over contractions, and loose for typical ones.

Early records are safer, late ones are not. The last record is protected by one step only. A randomly chosen stopping time spreads this out: Theorem 26 of the paper gives $(\alpha, \frac{4\alpha L_f^2 \ln n}{n\sigma^2})$-RDP for every record when the number of steps is uniform on $\{1, \dots, n\}$, provided $\sigma \geq L_f\sqrt{2(\alpha-1)\alpha}$. Substituting $\sigma^2 = 4L_f^2\tilde{\sigma}^2$, that is $\alpha \ln n/(n\tilde{\sigma}^2)$ in noise-multiplier form. The first record gets $\frac{\alpha}{2\tilde{\sigma}^2 n}$ from Equation 33.13 with $t = 1$, so the random stopping time gives every record $2\ln n$ times the guarantee of the first record.

One pass, no sampling. Each record is used once, at a fixed position. Several passes need the general Theorem 22 with several shifts; with random sampling, Altschuler & Talwar (2022) combine iteration with subsampling.

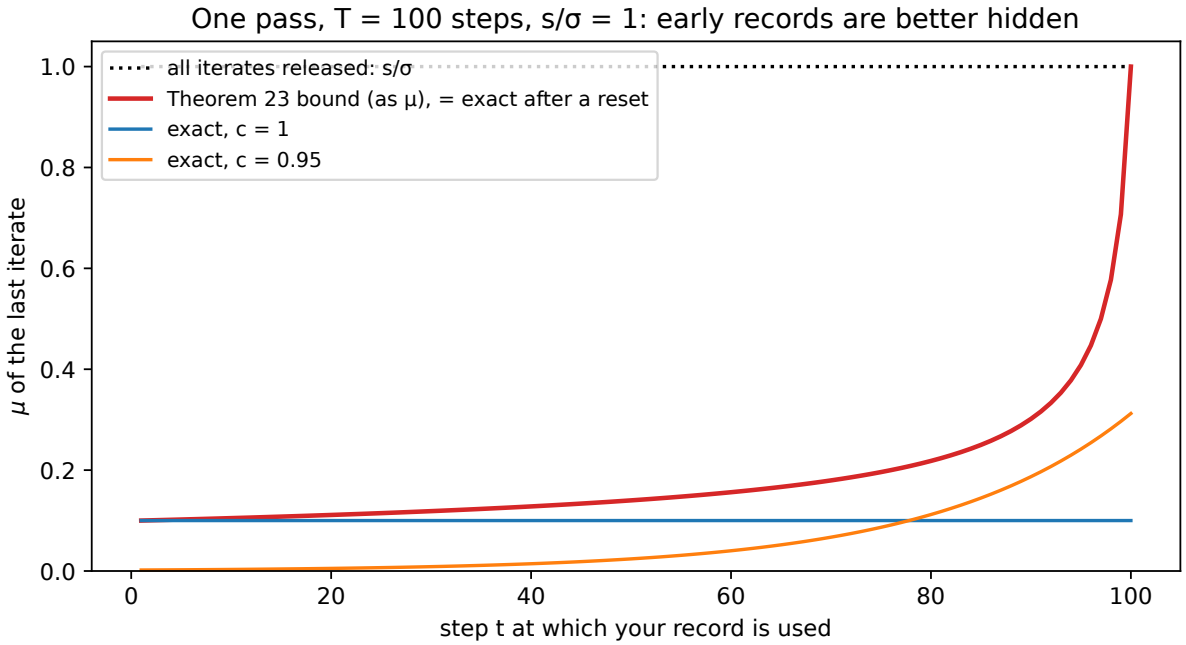


Figure 33.4.: The μ of the last iterate against the step t at which your record is used, for a one-pass run of $T = 100$ steps with per-step shift equal to the noise standard deviation ($s/\sigma = 1$). Releasing all iterates gives $\mu = 1$ for every record (dotted). Theorem 23 (red, as $\mu = 1/\sqrt{T-t+1}$) is exact when the run is reset just before step t; with $c = 1$ (blue) the earlier noise also helps, and with a strict contraction $c = 0.95$ (orange) the shift itself decays.

33.5.5. Example: a one-pass run over 10,000 records

Projected noisy SGD makes one pass over $n = 10{,}000$ records with noise multiplier $\tilde{\sigma} = 1$. An RDP curve becomes an (ε, δ) guarantee through Mironov's conversion: at each order α, the probability that the privacy loss exceeds ε is bounded by Markov's inequality applied to $e^{(\alpha-1)L}$, whose mean the RDP curve controls, and this gives $\varepsilon = \varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha-1}$. The first term grows with α and the second shrinks, so you minimize over the part's grid of orders. At $\delta = 10^{-5}$ the second

term is $\ln(10^5)/(\alpha-1) = 11.51/(\alpha-1)$. Converting Equation 33.13 this way gives a different answer for each position.

The last record ($t = 10{,}000$) has $n-t+1 = 1$ and gets $\varepsilon = 5.2985$. Its RDP curve is $\alpha/2$, the same as a single Gaussian mechanism with $\tilde{\sigma} = 1$, which is also what every record gets if all iterates are released. (That mechanism is a Gaussian pair with $\mu = 1$, and its exact ε, read from the $\delta(\varepsilon)$ curve of Fact 3, is 4.3772: the RDP conversion is not tight.) The record used 100 steps before the end ($t = 9901$) gets $\varepsilon = 0.4849$, and the record in the middle ($t = 5000$) gets $\varepsilon = 0.0707$. The first record gets $\varepsilon = 0.0481$, at the largest order $\alpha = 512$ of the grid. Its RDP curve is $\alpha/20{,}000$, so small ($\varepsilon(2) = 0.000100$) that the conversion term $\ln(1/\delta)/(\alpha-1)$ stays as large as the RDP term even at the end of the grid: 0.0225 against $512/20{,}000 = 0.0256$ at $\alpha = 512$. A finer conversion from From RDP to (ε, δ) would give less.

The guarantee is per record and depends on position. A report must state the worst record's value, or randomize the order and the stopping time.

33.5.6. Check yourself

- Why does the theorem need the loss to be convex and smooth, with $\eta \le 2/\beta$? *(So that each gradient step is a contraction. An expanding map could amplify a small shift faster than the noise can pay it off.)*
- If every iterate is released, what does the record at step 1 get? *(Only the noise of its own step: $\varepsilon(\alpha) = \alpha/(2\tilde{\sigma}^2)$, as for the last record. The later iterates reveal w_1 directly.)*

33.6. Hidden state: privacy that stops growing

33.6.1. Context

Full-batch noisy GD uses every record at every step. Composition charges every step, so the RDP of T steps grows linearly in T. Amplification by iteration does not help by itself: with every step adding a fresh shift s, the payments of Theorem 22 must cover Ts in total, and the cheapest schedule pays s at every step, which is composition again. Yet if the parameters live in a bounded set, two runs can never be further apart than its diameter, however long they run. So the shift never has to exceed the diameter, and only the last stretch of steps can matter. Can the privacy loss of the last iterate then stop growing with T? Altschuler & Talwar (2022) show that it can.

33.6.2. Statement

The idea is to ignore everything that happened before some late step τ: at that point the two runs are somewhere inside the set, at most d_K apart, and only the steps after τ need to pay off that distance plus the new shifts. The cost then depends on how many steps are needed to pay, not on T.

Theorem 3.1 of Altschuler & Talwar (2022), in shape. For convex, L_f-Lipschitz, β-smooth losses on a convex set K of diameter d_K, step size $\eta \le 2/\beta$, noise $\mathcal{N}(0, \sigma^2 I)$ added to the batch

gradient, any batch size b and T steps, noisy SGD releasing only the last iterate is (α, ε)-RDP with

$$\varepsilon \lesssim \frac{\alpha L_f^2}{n^2\sigma^2} \min\{T, \bar{T}\}, \qquad \bar{T} = \left\lceil \frac{d_K n}{L_f \eta} \right\rceil, \tag{33.15}$$

for α up to a threshold and $\sigma > 8\sqrt{2}L_f/b$; $\lesssim$ hides a constant, and $\lceil\cdot\rceil$ rounds up. Their Theorem 4.1 shows the shape is tight up to constants. The bound grows like composition until $\bar{T}$ and then stays flat.

The key lemma is their Proposition 3.2. It is Theorem 22 started at step τ instead of 0, with an initial shift of d_K instead of 0. In a contractive noisy iteration confined to a set of diameter d_K, with noise standard deviation σ_t at step t, for any $\tau < T$ and payments $a_{\tau+1}, \dots, a_T$ whose unpaid shift $z_t = d_K + \sum_{i=\tau+1}^{t}(s_i - a_i)$ stays non-negative and ends at $z_T = 0$,

$$D_\alpha(X_T \| X'_T) \le \sum_{t=\tau+1}^{T} \frac{\alpha a_t^2}{2\sigma_t^2}. \tag{33.16}$$

Specializing it to **full-batch noisy GD** with per-step shift s and per-step noise standard deviation σ (both in parameter units) gives the bound below. It takes the better of two strategies, starting the accounting at the beginning (cost Ts^2, composition) or m steps before the end (cost $(d_K + ms)^2/m$), each times $\frac{\alpha}{2\sigma^2}$. This specialization is our derivation, below; the paper proves Theorem 3.1 by a different route.

$$\varepsilon(\alpha) \le \frac{\alpha}{2\sigma^2} \min\left\{ Ts^2, \min_{m\in\{1,\dots,T\}} \frac{(d_K + ms)^2}{m} \right\} \le \frac{2\alpha d_K s}{\sigma^2} \quad \text{when } d_K/s \text{ is an integer.} \tag{33.17}$$

For other d_K/s the right-hand bound is $\frac{\alpha}{2\sigma^2}(4d_K s + s^2)$ (step 5).

33.6.3. Derivation

1. **Start the accounting late.** At step τ both runs lie in a set of diameter d_K, so they are at most d_K apart, and $D_\alpha^{(d_K)}(X_\tau \| X'_\tau) = 0$: shifting X_τ by at most d_K can turn it into X'_τ (couple the two runs and move each point onto its partner). This is the only place where boundedness enters.
2. **Run Theorem 22 from τ.** The induction of Theorem 22, started at τ with shift d_K and divergence 0 instead of shift 0, gives Proposition 3.2. Projections onto K are contractions (Section 33.5), so by Equation 33.12 they cost nothing.
3. **Pay evenly over the last $m = T - \tau$ steps.** The shift to pay off is the initial d_K plus s per step, $d_K + ms$ in total. The even schedule $a = (d_K + ms)/m$ pays s more than each step adds, so the unpaid shift falls linearly from d_K to 0 and $z_t \ge 0$ holds. It costs $m \cdot \frac{\alpha a^2}{2\sigma^2} = \frac{\alpha(d_K+ms)^2}{2m\sigma^2}$.
4. **Or start at the beginning.** With $\tau = 0$ both runs start at the same point, the starting shift is 0, and the even schedule over all T steps pays s per step: $\frac{\alpha T s^2}{2\sigma^2}$, exactly the composition of T Gaussian steps. Take the better of the two choices.
5. **Optimize m.** A short window means few payments, each large; a long window means more shift to pay. Expanding the square, $(d_K + ms)^2/m = d_K^2/m + 2d_K s + ms^2$, the sum of a term falling in m and one growing in m. Over real $m > 0$ it is minimized where the two balance, at $m = d_K/s$ (setting the derivative $-d_K^2/m^2 + s^2$ to 0), where

it equals $d_K s + 2d_K s + d_K s = 4d_K s$. When d_K/s is an integer and $T \geq d_K/s$, that m is allowed, so the bound never exceeds $\frac{\alpha}{2\sigma^2} \cdot 4d_K s = \frac{2\alpha d_K s}{\sigma^2}$, whatever T. Otherwise take $m = \lceil d_K/s \rceil$: writing $m = d_K/s + r$ with $0 < r < 1$, $d_K + ms = 2d_K + rs$, and $(d_K + ms)^2/m = 4d_K s + r^2 s^2/m < 4d_K s + s^2$. The bound reaches its plateau when the composition branch catches up with it, $Ts^2 = 4d_K s$, at $T = 4d_K/s$.

6. **Back to their parameters.** With $s = 2\eta L_f/n$ (the full-batch gradient is an average over n records, so replacing one moves it by at most $2L_f/n$) and noise $\eta\sigma$ in parameter space, the plateau is $\frac{2\alpha d_K \cdot 2\eta L_f/n}{\eta^2\sigma^2} = \frac{\alpha L_f^2}{n^2\sigma^2} \cdot \frac{4d_K n}{L_f \eta}$, the shape of Theorem 3.1 with $\bar{T} \propto d_K n/(L_f \eta)$.

33.6.4. Interpretation

Why the bounded domain caps the loss. Composition pays for every step because it assumes the adversary sees every iterate. The last iterate forgets: whatever happened before step $T - m$ is summarized by a starting point inside the set, and the starting points of the two runs differ by at most d_K. The last m noisy steps can hide that difference for a cost that does not depend on T.

The constant is loose. To see how loose, take a linear loss on $K = [0, 1]$, with $s = 0.01$ and $\sigma = 0.1$ per step. The two runs are then random walks drifting in opposite directions and projected onto the interval (the construction behind Altschuler & Talwar's lower bound). The script computes their D_2 on a grid of 1000 cells (500 cells give 0.3779 against 0.3786 at $T = 2000$, so the discretization error is small). Table 33.6 compares it with composition and with Equation 33.17. The walk's divergence flattens at 0.3786 after about 100 steps, the bound at 4.0000 after 400 steps (Figure 33.5, right). This walk is one pair of datasets with one starting point, so its divergence is a lower bound on the worst case: the shape of the bound is right, and its constant is at most $4.0000/0.3786 = 10.57$ times too large.

Strongly convex losses converge faster. A loss is λ-strongly convex when it curves upward at least as fast as $\frac{\lambda}{2}\|w\|^2$. Then each gradient step is a strict contraction by $1 - \eta\lambda$, so the shift shrinks by that factor every step even when nobody pays for it. The plateau is reached after about $1/(\eta\lambda)$ steps instead of $d_K n/(L_f \eta)$, with no diameter needed. Chourasia, Ye & Shokri (2021, Corollary 1) had shown earlier, by a different route, that for full-batch noisy GD (Langevin dynamics) the RDP of the last iterate is at most $\frac{\alpha S_g^2}{\lambda\sigma^2 n^2}(1 - e^{-\lambda\eta T/2})$ after T steps, where S_g is the sensitivity of the summed gradient and their noise is $\sqrt{2\eta}\sigma$ per step. As T grows the bracket tends to 1, so the bound levels off. Ye & Shokri (2022) extended this to mini-batch SGD, concurrently with Altschuler & Talwar. The next section checks their limit against an exact computation.

Only the last iterate. All these results need **only the last iterate to be released**. Checkpoints, logged intermediate models or a model released after each epoch break the argument, because the adversary then sees the run before it has forgotten, and you are back to composition.

Contractive steps. They also need **contractive steps**, which require convexity. For non-convex losses, Annamalai (2024) constructs a loss for which the last iterate of DP-SGD leaks as much as all iterates together, so no hidden-state amplification holds for all non-convex losses. Empirically, Nasr et al. (2021) found that their strongest attacks match the DP-SGD bound when the adversary sees every step or can craft gradients, and fall short of it when the adversary only sees the final model. That gap is an observation about their attacks, not a guarantee.

Do not report a hidden-state ε for deep learning.

Table 33.6.: D_2 of the last iterate of full-batch noisy GD on a linear loss over $K = [0, 1]$ ($d_K = 1$), with per-step shift $s = 0.01$ and noise $\sigma = 0.1$, by composition, by the bounded-domain bound, and computed numerically for the two projected random walks.

Steps T	10	100	400	1000	2000
Composition, $\alpha T s^2/(2\sigma^2)$	0.1000	1.0000	4.0000	10.0000	20.0000
Bounded-domain bound, Equation 33.17	0.1000	1.0000	4.0000	4.0000	4.0000
Projected random walks, numerical	0.0961	0.3693	0.3786	0.3786	0.3786

33.6.5. Example: two thousand steps on the unit interval

Take the setting of Table 33.6: diameter $d_K = 1$, $s = 0.01$, $\sigma = 0.1$, so $d_K/s = 100$ is an integer. Each composed step costs $\frac{\alpha s^2}{2\sigma^2} = \frac{2\times 0.0001}{2\times 0.01} = 0.01$ at $\alpha = 2$, so composition charges $0.01T$. Step 5 gives the plateau $2\alpha d_K s/\sigma^2 = 2\alpha \cdot \frac{0.01}{0.01} = 2\alpha$, which is 4.0000 at $\alpha = 2$, reached at $T = 4d_K/s = 400$ steps. After 2000 steps, composition charges 20.0000; the bounded-domain bound charges 4.0000, five times less, and for the hard instance of the projected random walks the value is below 0.4. Running ten times longer does not change the bound; under composition it would cost ten times more.

33.6.6. Check yourself

- Why does an unconstrained linear loss get no hidden-state amplification? *(Without a bounded set or a strict contraction, the runs drift apart by s per step forever: after T steps the means differ by Ts, the variances are $T\sigma^2$, and $\mu = Ts/(\sigma\sqrt{T}) = \sqrt{T}s/\sigma$, the same as composition. The next section computes this.)*
- A training pipeline saves a checkpoint every epoch and publishes the best one. Does the plateau apply? *(No. The adversary may see any of the checkpoints, so the analysis must cover all of them; the published model is selected using all of them.)*

33.7. An exact example: noisy GD on a quadratic

33.7.1. Context

All the results above are bounds. In one dimension, with a quadratic loss, each step of noisy GD is a linear map plus Gaussian noise, so the last iterate is Gaussian on both datasets and its privacy can be computed exactly with Fact 3. The computation shows the plateau directly, shows that it needs contraction, and gives a yardstick for the strongly convex bounds.

33.7.2. Statement

Consider the run $w_k = c\,w_{k-1} + b_k + \xi_k$, $\xi_k \sim \mathcal{N}(0, \sigma^2)$, for $k = 1, \dots, T$ from a fixed w_0. Here c is the contraction factor, b_k is a data-dependent offset and ξ_k is the noise of step k. The offsets on D and D' differ by s at every step $k \geq t$ and agree before: your record enters at step t and stays. Unrolling the run, the last iterate is a weighted sum of all offsets and all noise draws, the step-k term weighted by c^{T-k}. So it is Gaussian with the same variance on both datasets, and by Fact 3 its μ is the weighted sum of the offset differences divided by the standard deviation of the weighted noise. The last iterate is exactly μ-GDP with

$$\mu_T = \frac{s}{\sigma} \cdot \frac{\sum_{k=t}^{T} c^{T-k}}{\sqrt{\sum_{j=0}^{T-1} c^{2j}}}, \qquad \text{and for } t = 1,\ c < 1: \qquad \lim_{T\to\infty} \mu_T = \frac{s}{\sigma}\sqrt{\frac{1+c}{1-c}}. \tag{33.18}$$

Releasing all iterates instead is $\sqrt{T-t+1}\,s/\sigma$-GDP. For $c = 1$ and $t = 1$ the two agree.

This is full-batch noisy GD on the quadratic loss $\ell(w; x) = \frac{\lambda}{2}(w - x)^2$, whose gradient is $\lambda(w - x)$. Averaged over the records, the gradient is $\lambda(w - \bar{x})$, where $\bar{x}$ is the mean of the records. So the step $w - \eta(\lambda(w - \bar{x}) + \text{noise})$ is $(1 - \eta\lambda)w + \eta\lambda\bar{x}$ plus noise: it has $c = 1 - \eta\lambda$ and offset $b = \eta\lambda\bar{x}$. Replacing one of n records by one that differs by Δx changes $\bar{x}$ by $\Delta x/n$, so $s = \eta\lambda\Delta x/n$. Starting the record at step $t > 1$ models a record that joins the training set late.

33.7.3. Derivation

1. **Unroll.** Substituting each step into the next, by induction on T, $w_T = c^T w_0 + \sum_{k=1}^{T} c^{T-k}(b_k + \xi_k)$: each step's offset and noise are multiplied by c once for every later step.
2. **The noise part** is a sum of independent Gaussians, so it is Gaussian with mean 0 and variance $\sigma^2 \sum_k c^{2(T-k)} = \sigma^2 \sum_{j=0}^{T-1} c^{2j}$, the same on D and D' (Fact 3).
3. **The mean part** differs between the datasets by $\sum_{k\geq t} c^{T-k} s$; the start w_0 and the offsets before step t are the same in both runs and cancel.
4. **Gaussian pair.** By Fact 3 the pair is exactly μ-GDP with $\mu =$ (mean difference)/(standard deviation), which is Equation 33.18.
5. **The limit.** For $t = 1$ and $c < 1$, both sums are geometric: $\sum_{k=1}^{T} c^{T-k} = (1 - c^T)/(1 - c)$ and $\sum_{j=0}^{T-1} c^{2j} = (1 - c^{2T})/(1 - c^2)$. So $\mu_T = \frac{s(1-c^T)/(1-c)}{\sigma\sqrt{(1-c^{2T})/(1-c^2)}} \to \frac{s}{\sigma} \cdot \frac{\sqrt{1-c^2}}{1-c}$, since $c^T \to 0$. Writing $1 - c^2 = (1 - c)(1 + c)$, this is $\frac{s}{\sigma}\sqrt{\frac{(1-c)(1+c)}{(1-c)^2}} = \frac{s}{\sigma}\sqrt{\frac{1+c}{1-c}}$.
6. **Composition.** Releasing all iterates is $T - t + 1$ Gaussian mechanisms, each s/σ-GDP given the past (the offset shifts that step's output by s under noise σ), so by the composition rule of Fact 3 it is $\sqrt{T-t+1}\,s/\sigma$-GDP. For $c = 1$, every power of c is 1, and step 4 gives $\frac{(T-t+1)s}{\sigma\sqrt{T}}$, which for $t = 1$ is $\sqrt{T}s/\sigma$: hiding the state buys nothing without contraction.

33.7.4. Interpretation

Contraction is what forgets. With $c < 1$, the offset of step k reaches the end damped by c^{T-k}, while fresh noise keeps arriving. The signal saturates at the geometric limit $s/(1 - c)$ and the

noise standard deviation at $\sigma/\sqrt{1-c^2}$. With $c = 1$ both keep growing, the signal like T and the noise standard deviation like $\sqrt{T}$, so μ grows like $\sqrt{T}$ (Figure 33.5, left).

The plateau arrives after about $1/(1-c)$ steps. That is how long it takes c^T to become small. With $c = 0.9$, μ is 4.3588 at $T = 100$ and its limit is $\sqrt{1.9/0.1} = \sqrt{19} = 4.3589$; with $c = 0.99$ it is 9.6101 at $T = 100$, 14.1061 at $T = 1000$, and the limit is $\sqrt{1.99/0.01} = \sqrt{199} = 14.1067$. In general, a gradient step with $\eta = 1/\beta$ on a λ-strongly convex, β-smooth loss contracts by $c = 1 - \lambda/\beta$, so $1/(1-c)$ is the condition number β/λ, which is the threshold, up to logarithmic factors, that Altschuler & Talwar give for strongly convex losses.

Recent records are hardly hidden. A record that joins at step 41 of 50, with $c = 0.9$ and $s/\sigma = 1$, gets $\mu = 2.8391$ against $\sqrt{10} = 3.1623$ for releasing its 10 steps: only the last steps carry its signal, and a little earlier noise helps.

How tight is the strongly convex bound? In this example the limit of Chourasia, Ye & Shokri's Corollary 1, translated into the s, σ and c of this section, becomes $\frac{2\alpha s^2}{(1-c)\sigma^2}$, while the exact limiting RDP is $\alpha\mu^2/2$ (Fact 3) with the limiting μ of Equation 33.18, that is, $\frac{\alpha}{2} \cdot \frac{s^2}{\sigma^2} \cdot \frac{1+c}{1-c}$. Their ratio is $\frac{1+c}{4}$: at $c = 0.99$, $\alpha = 2$ and $s/\sigma = 1$ the exact value is $199 = 199.0000$ and the bound $4/0.01 = 400.0000$, a ratio of $1.99/4 = 0.4975$. The bound is within a factor of about 2.

A check by simulation. 200,000 runs with $c = 0.9$, $T = 50$ and $s/\sigma = 1$ give an estimated μ of 4.3381 against the exact 4.3365.

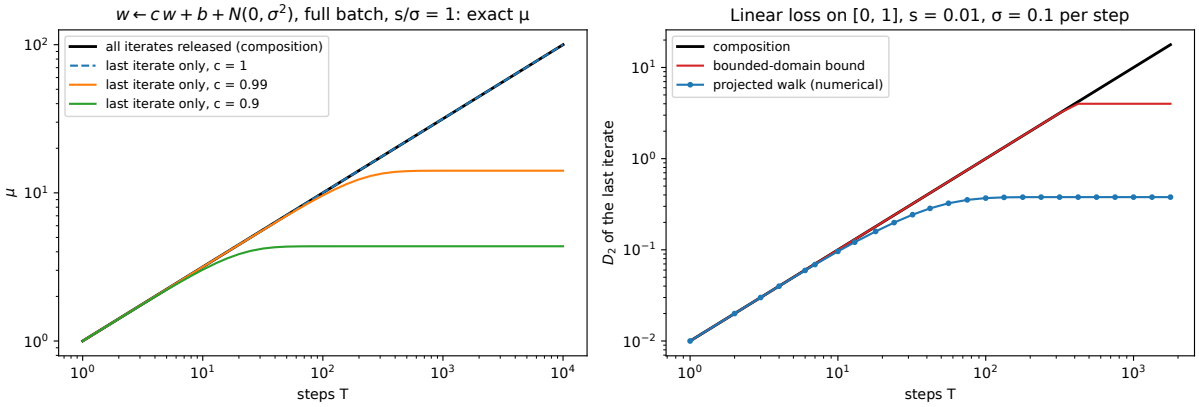


Figure 33.5.: Left: the exact μ of the last iterate of one-dimensional full-batch noisy GD with per-step shift equal to the noise standard deviation, against the number of steps T, for $c = 1$ (dashed, on top of the black composition line), $c = 0.99$ and $c = 0.9$. Right: D_2 of the last iterate for a linear loss on $[0, 1]$ with $s = 0.01$ and $\sigma = 0.1$ per step: composition, the bounded-domain bound Equation 33.17, and the numerical value for two projected random walks.

33.7.5. Example: ten thousand steps with noise multiplier 10

Run $k = 10{,}000$ steps of full-batch noisy GD with noise multiplier $\sigma/s = 10$ on a quadratic with $c = 0.99$. Releasing every iterate is $\mu = \sqrt{10{,}000}/10 = 10.0000$-GDP, which is $\varepsilon = 91.8173$ at $\delta = 10^{-5}$ (from the $\delta(\varepsilon)$ curve of Fact 3): no guarantee at all. Releasing only the final model is, after this many steps, at the limit of Equation 33.18, $\mu = 0.1 \times 14.1067 = 1.4107$-GDP, $\varepsilon = 6.5534$.

The noise is the same; the difference is entirely in what the adversary is allowed to see. Compare the DP-SGD running example of this part (under add/remove neighbours), where subsampling rather than hidden state does the work: there each step uses only 1% of the data, which is why noise multiplier 1 suffices.

33.7.6. Check yourself

- In Equation 33.18, why does the start w_0 not matter? *(It enters both runs as the same constant $c^T w_0$, so it cancels in the mean difference and adds no variance.)*
- Doubling the noise σ in this example: what happens to the plateau μ? *(It halves: μ is proportional to s/σ at every T.)*

33.8. Code: check every number by computation

The script code/shuffling_iteration.py computes every number in this chapter and draws every figure. Shuffled randomized response and the clone pair are computed exactly as discrete distributions in log space, with $\delta(\varepsilon)$ read from sorted prefix sums (Fact 1). The iteration results are closed forms for Gaussian pairs, checked by Monte Carlo, plus one grid computation for the projected random walks. From the repository root:

```
uv run python code/part04-privacy-accounting/shuffling_iteration.py
```

Output (seed 0), summarized in Table 33.7:

Table 33.7.: The script's output with the default seed.

Experiment	Result
Shuffled randomized response, $n = 3$, $\varepsilon_0 = \ln 3$	$\delta(0.5) = 0.1900$ against 0.3378 for one report; simulation within 0.0015 of the exact count distribution
Worst dataset pair, all m at $n = 1000$, $m \le 30$ at $n = 10^4, 10^5$	The pair where the others all hold 0 is the worst for $\varepsilon_0 = 1, 4$ at $n = 1000$, and within 4.5×10^{-5} of the largest ε found everywhere
Clone decomposition, $\varepsilon_0 = \ln 3$	Clone part $(0.1667, 0.1667)$, leftover $(0.875, 0.125)$
Shuffling at $\delta = 10^{-6}$	Table 33.4; numeric clone bound 1.44 to 2.36 times the exact value
Shift reduction, $z = 1$, $\sigma = 1$, $\alpha = 2$	Even schedule equals the exact D_2 at $T = 1, 2, 4, 10$
One pass over 10^4 records, $\tilde{\sigma} = 1$	ε from 0.0481 (first record) to 5.2985 (last) at $\delta = 10^{-5}$
Bounded domain, $d_K = 1$, $s = 0.01$, $\sigma = 0.1$	Table 33.6; plateau 4.0000 against 0.3786 for the projected random walks
Quadratic, $s/\sigma = 1$	Limits 4.3589 ($c = 0.9$) and 14.1067 ($c = 0.99$); Monte Carlo 4.3381 against 4.3365

tests/test_shuffling_iteration.py checks that the output still matches.

What to take from it. Shuffling amplifies like $1/\sqrt{n}$, and the constant matters: the exact value for binary randomized response is about half the best general bound and a quarter to an eighth of the closed form. Since **exact computations are cheap** (the count distributions have $n+1$ points, and even $n = 10^5$ takes milliseconds), do not settle for a closed form when the exact value is at hand. On the iteration side, **iteration amplifies only what is hidden**: the same noisy run is worth $\mu = 10$ or $\mu = 1.41$ depending on whether intermediate iterates are released. And **hidden-state gains need contraction or a bounded domain**; without either, the last iterate is as revealing as the whole trajectory.

33.9. References

- Feldman, McMillan & Talwar, "Hiding Among the Clones: A Simple and Nearly Optimal Analysis of Privacy Amplification by Shuffling" (FOCS 2021; arXiv:2012.12803). Theorems 3.1 and 3.2 and Lemmas 3.3–3.5 are the clone bound and reduction used here; Section 6 gives the numerical method, and their code computes it. Also covers approximate-DP randomizers (Theorem 3.8) and RDP of shuffled mechanisms.
- Erlingsson, Feldman, Mironov, Raghunathan, Talwar & Thakurta, "Amplification by Shuffling: From Local to Central Differential Privacy via Anonymity" (SODA 2019; arXiv:1811.12469). The first amplification-by-shuffling theorem (Theorem 7 of the arXiv version), for adaptive randomizers; read it for the swap argument and for the encode–shuffle–analyze motivation.
- Balle, Bell, Gascón & Nissim, "The Privacy Blanket of the Shuffle Model" (CRYPTO 2019). Writes each randomizer as a mixture with an input-independent "blanket" and derives amplification bounds from it, including a closed form tailored to k-ary randomized response.
- Cheu, Smith, Ullman, Zeber & Zhilyaev, "Distributed Differential Privacy via Shuffling" (EUROCRYPT 2019). An early formal study of the shuffle model; its analysis of shuffled binary randomized response already has the $e^{\varepsilon_0/2}/\sqrt{n}$ dependence.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015). Shows that every ε_0-DP randomizer on two inputs is a post-processing of binary randomized response, the fact used in step 4 of the clone derivation.
- Feldman, Mironov, Talwar & Thakurta, "Privacy Amplification by Iteration" (FOCS 2018; arXiv:1808.06651). Shifted Rényi divergence, the shift-reduction lemma (Lemma 20), the contraction lemma (Lemma 21), Theorem 22, and noisy SGD (Theorems 23, 24 and 26).
- Altschuler & Talwar, "Privacy of Noisy Stochastic Gradient Descent: More Iterations without More Privacy Loss" (NeurIPS 2022; arXiv:2205.13710). The bounded-domain plateau (Proposition 3.2, Theorem 3.1; the full-batch form Equation 33.17 is this chapter's specialization of Proposition 3.2), a matching lower bound (Theorem 4.1), and extensions to strongly convex losses and cyclic batches.
- Chourasia, Ye & Shokri, "Differential Privacy Dynamics of Langevin Diffusion and Noisy Gradient Descent" (NeurIPS 2021; arXiv:2102.05855). Converging RDP of the last iterate of noisy GD for smooth strongly convex losses (Corollary 1), proved through Langevin diffusion and log-Sobolev inequalities.
- Ye & Shokri, "Differentially Private Learning Needs Hidden State (Or Much Faster Convergence)" (NeurIPS 2022; arXiv:2203.05363). Extends the converging bound to mini-batch noisy SGD with shuffle-and-partition and sampling without replacement.

- Nasr, Song, Thakurta, Papernot & Carlini, "Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning" (IEEE S&P 2021; arXiv:2101.04535). Attacks that match the DP-SGD bound for strong adversaries and fall short when the adversary sees only the final model.
- Annamalai, "It's Our Loss: No Privacy Amplification for Hidden State DP-SGD With Non-Convex Loss" (ACM Workshop on Artificial Intelligence and Security, AISec 2024; arXiv:2407.06496). A non-convex loss for which the last iterate leaks as much as all iterates.

33.10. Cheat sheet

Table 33.8 collects the results. ε_0 is the local level, n the number of users or records, $\tilde{\sigma}$ the noise multiplier, s and σ the per-step shift and noise in parameter units, c the contraction factor, d_K the diameter. All use replace-one neighbours.

Table 33.8.: The results of this chapter.

Result	Statement	Tightness
Shuffled binary randomized response	Count pair Equation 33.5; $\mu \approx 2\sinh(\varepsilon_0/2)/\sqrt{n}$	Exact for the pair where the others hold 0, within 10^{-4} of the worst pair found; normal approximation fails for few 1-reports
Clone reduction (Thm 3.2)	Output is a post-processing of $(A+\Delta, C-A+1-\Delta)$ vs $(A+1-\Delta, C-A+\Delta)$, $C \sim \mathrm{Bin}(n-1, e^{-\varepsilon_0})$	Numeric: 1.44–2.36 × exact for binary randomized response ($\approx 2/(1+e^{-\varepsilon_0})$ with many clones)
Clone bound (Thm 3.1)	Equation 33.7, needs $\varepsilon_0 \le \ln \frac{n}{16\ln(2/\delta)}$	4.47–7.55 × exact
Shift reduction	$D_\alpha^{(z)}(X+Z\Vert X'+Z) \le D_\alpha^{(z+a)}(X\Vert X') + \frac{\alpha a^2}{2\sigma^2}$	Exact for translations from a common point
One-pass noisy SGD (Thm 23)	$\varepsilon_t(\alpha) = \frac{\alpha}{2\tilde{\sigma}^2(n-t+1)}$ for the record at step t, last iterate only	Exact after a reset; loose for strict contractions
Bounded domain (from Prop. 3.2)	$\varepsilon(\alpha) \le \frac{\alpha}{2\sigma^2}\min\{Ts^2, \min_m(d_K + ms)^2/m\} \le \frac{2\alpha d_K s}{\sigma^2}$ (d_K/s an integer)	Right shape; constant at most about 10 × too large in the example
Quadratic, exact	μ_T of Equation 33.18; limit $\frac{s}{\sigma}\sqrt{\frac{1+c}{1-c}}$	Exact; strongly convex bound within a factor $\frac{4}{1+c}$

Rules of thumb

- Shuffling turns ε_0-local into central $\varepsilon \approx$ (a constant growing like $e^{\varepsilon_0/2}$) $\cdot \sqrt{\ln(1/\delta)/n}$. It never lowers the pure-DP level.

- For binary randomized response, compute the exact value. For other randomizers, compute the numeric clone bound; use the closed form only to reason.
- Amplification by iteration needs contractive steps (convex, smooth, $\eta \leq 2/\beta$) and a hidden state. The record used last gets no amplification.
- With a bounded domain or strong convexity, the last iterate's privacy plateaus; without them it grows like composition.
- Never claim hidden-state privacy for non-convex training or when checkpoints are released.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

34. Adaptive composition: filters, odometers and individual accounting

34.1. Why this matters

A composition theorem tells you how much privacy a sequence of steps loses in total, given what each step loses. Every composition theorem so far assumes that the privacy parameters of the steps are fixed before the run starts. The mechanisms themselves may be chosen adaptively, after seeing earlier outputs, as in Composition (Section 21.6), but the list of per-step budgets $\varepsilon_1, \dots, \varepsilon_k$, and the number of steps k itself, must be known in advance. Real systems break this assumption in three ways:

- **Data-dependent stopping.** You train until the validation loss stops improving, or an analyst keeps asking queries until the answer looks clear. The number of steps is a function of the outputs.
- **Adaptive noise.** You lower the noise when training stalls, or spend more on a query whose first answer was surprising. Each ε_t depends on the earlier outputs.
- **Records that contribute differently.** Training with differential privacy (DP) usually means stochastic gradient descent (SGD) with clipped per-record gradients and Gaussian noise, called DP-SGD. A record whose clipped gradient is small moves the noisy sum less than the clip allows. Its actual privacy cost per step is smaller than the worst case, and how much smaller depends on the current model, which depends on the earlier outputs.

In each case the per-step parameters are random variables: you only learn them as the run goes on. It is tempting to wait until the run ends, plug the values that actually occurred into a fixed-parameter theorem, and report the result. That does not work. Section 34.3 shows an analyst who makes the plug-in advanced composition bound fail with probability more than three times its nominal $\delta = 0.01$, just by choosing when to stop.

34.1.1. The one idea: a running budget needs a theorem about stopping

Once the parameters are adaptive, the question "what is the ε of these k steps?" no longer makes sense, because neither k nor the ε_t are known in advance. It splits into two questions, and each has its own object. A **privacy filter** answers "may I run one more step without exceeding the budget B?" It is a stopping rule, and its guarantee is a fixed (ε, δ) for whatever transcript the stopped run produces. A **privacy odometer** answers "how much have I spent so far?" It is a running bound, like the odometer of a car, and because you may stop whenever you like, it must hold at every step simultaneously.

Both need theorems of their own, and the tool behind all of them is the same. Track a quantity built from the privacy loss that, on average, never goes up from one step to the next: a

supermartingale, a process whose expected next value, given everything seen so far, is at most its current value. A fair betting game is the picture to keep in mind: your expected wealth after the next bet equals your wealth now. The key fact is that stopping such a process at a time of your choosing cannot raise its mean, so a clever stopping rule cannot manufacture privacy loss. In Rényi differential privacy (RDP), which measures each step by a curve $\varepsilon_t(\alpha)$ of Rényi divergences indexed by an order $\alpha > 1$, this is almost free. If the RDP spends at a fixed order α add up to at most B on every path, the run is (α, B)-RDP, exactly as if the parameters had been fixed. That one fact gives a filter with the constants of advanced composition, an odometer that costs only a union bound over orders, and per-record accounting in DP-SGD. Table 34.1 lists the results.

Table 34.1.: The results of this chapter and where each is used.

Result	Where it is used
Plugging realized ε_t into advanced composition fails under data-dependent stopping	Why "report advanced composition of what you spent" is not a valid odometer
Filters for pure DP: basic, Rogers et al., Whitehouse et al.	Interactive query systems with a global budget; choosing ε_t on the fly
The Rényi filter	Any RDP accountant with adaptive noise or adaptive stopping; the base of the next two rows
Odometers, and why they cost more	Early stopping; reporting the privacy spent so far
Individual (per-record) accounting in DP-SGD	Dropping records once their own budget is spent; per-record ε and its caveats

What you need. The privacy loss of an adaptive composition is a sum of conditional losses (Equation 21.3 in Composition), and advanced composition is Azuma's inequality applied to that sum (Equation 21.7). Azuma's inequality and martingales are in Concentration. From this part you need RDP, its composition and Mironov's conversion (Equation 27.10, Equation 27.17 in Rényi DP), the RDP curves of randomized response and the Gaussian mechanism (Equation 27.9, Equation 27.7), and two facts from Concentrated differential privacy: an ε-DP step is $(\alpha, \alpha\varepsilon^2/2)$-RDP for every α (Equation 29.6), and ρ-zero-concentrated differential privacy (ρ-zCDP) converts to $(\rho + 2\sqrt{\rho \ln(1/\delta)}, \delta)$-DP (Equation 29.10). Each is restated below where it is used.

Notation. Two datasets are **neighbours** if they differ in one person's record. A run has steps $t = 1, 2, ...$; step t releases an output y_t, using a mechanism chosen from the earlier outputs $y_{<t} = (y_1, \dots, y_{t-1})$. $P = M(D)$ and $Q = M(D')$ are the distributions of the whole transcript on two neighbours D and D'. The probability (or density) that step t outputs y_t after the history $y_{<t}$ is $P_t(y_t \mid y_{<t})$ on D and $Q_t(y_t \mid y_{<t})$ on D'. The **privacy loss** of an output is the log of how much likelier it is on D than on D': positive when it favours D, negative when it favours D'. The conditional loss of step t is therefore $L_t = \ln \frac{P_t(y_t|y_{<t})}{Q_t(y_t|y_{<t})}$, and the loss after t steps is $L_{1:t} = L_1 + \cdots + L_t$. The parameter ε_t (or $\varepsilon_t(\alpha)$ for RDP) may depend on the history $y_{<t}$ but not on the output y_t it is about to price: it is **predictable**, known just before the step runs. Probabilities and expectations are under P unless written otherwise. Randomized response at level ε reports a person's bit truthfully with probability $e^\varepsilon/(1 + e^\varepsilon)$ and flips it otherwise.

34.2. Toolkit: stopping a supermartingale

Each derivation below reuses three facts. The first says that stopping early does not change how you compute the loss. The second says that stopping early cannot raise the mean of a supermartingale. The third turns the second into a bound on the whole path of the process. The first is Fact 2 of Composition, restated for a stopped run; the other two are proved here.

34.2.1. Fact 1 — The loss of a stopped run is the sum of the losses of the steps it ran

The probability of a whole transcript is built one step at a time: the probability of y_1, times the probability of y_2 given y_1, and so on. This is the chain rule of probability. So the transcript $(y_1, \dots, y_t)$ has density $\prod_{j \le t} p_j(y_j \mid y_{<j})$ under P and the same product of q_j under Q. The ratio of the two products is the product of the step-by-step ratios, and the logarithm turns that product into a sum, so $L_{1:t} = \sum_{j \le t} L_j$ (Equation 21.3).

Now let the run stop at a random time T that is decided from the outputs alone. The stopped transcript $(y_1, \dots, y_T)$ still has density $\prod_{j \le T} p_j$. The reason is that the decision to stop after y_T is a function of $y_{\le T}$: given those outputs, it happens with probability 1 or 0, and that probability is the same under P and Q. A factor that is equal in the numerator and the denominator cancels from the ratio. So the loss of the stopped run is $L_{1:T}$, the sum of the losses of the steps it ran.

34.2.2. Fact 2 — Optional stopping: a stopped supermartingale keeps its mean at most where it started

Think of N_t as your wealth after t bets in a game that is fair or unfavourable to you. Each bet, on average, leaves your wealth where it was or lowers it. You may decide when to quit, looking at everything that has happened, but not at the future. The fact says that no such quitting rule can make your expected final wealth larger than your starting wealth.

In symbols, a process $N_0, N_1, \dots$ with N_t a function of $y_{\le t}$ is a **supermartingale** if $\mathbb{E}[N_t \mid y_{<t}] \le N_{t-1}$ for every t: given the history, the expected next value is at most the current one. It is a **martingale** if equality holds, the fair game. A **stopping time** T is a random step such that the event $\{T \ge t\}$, "the run takes step t", is decided by $y_{<t}$: you decide whether to take a step before seeing its result. If $T \le n$ for a fixed n, then

$$\mathbb{E}[N_T] \le \mathbb{E}[N_0]. \tag{34.1}$$

Proof. The idea is to write the stopped process as a running total of the increments it actually collects, and to show that each collected increment has non-positive mean. Write $T \wedge t = \min(T, t)$ for the time the stopped process has reached by step t.

1. **Write the stopped process as a sum of increments.** Up to step t, the stopped process gains the increment $N_t - N_{t-1}$ only if the run takes step t, that is if $T \ge t$. So $N_{T \wedge t} = N_{T \wedge (t-1)} + \mathbf{1}\{T \ge t\}(N_t - N_{t-1})$, where the indicator $\mathbf{1}\{T \ge t\}$ is 1 when the run takes step t and 0 otherwise.

2. **Each collected increment has non-positive mean.** The indicator is decided by $y_{<t}$, so once you condition on $y_{<t}$ it is a known constant and comes out of the conditional expectation: $\mathbb{E}[\mathbf{1}\{T \geq t\}(N_t - N_{t-1}) \mid y_{<t}] = \mathbf{1}\{T \geq t\}(\mathbb{E}[N_t \mid y_{<t}] - N_{t-1}) \leq 0$, because the bracket is non-positive by the supermartingale property. This is where it matters that the stopping decision does not look at y_t.
3. **Add up the steps.** Take expectations and sum over $t = 1, \ldots, n$. Each step adds a non-positive amount to the mean, so $\mathbb{E}[N_{T \wedge n}] \leq \mathbb{E}[N_0]$. Since $T \leq n$, $T \wedge n = T$.

Within a fixed horizon, then, no rule for when to quit turns a fair coin game (N_t = your wealth) into a winning one; this is the reason a filter works. The horizon matters in general: with unlimited time and unlimited credit, "quit when you are one ahead" wins a fair coin game with probability 1, because the walk eventually reaches +1, at the price of arbitrarily deep debt along the way. For the non-negative processes used below ($N_t \geq 0$, so no debt) the horizon can be dropped. If T may be infinite, apply Equation 34.1 to $T \wedge n$ and let $n \to \infty$: $N_{T \wedge n} \to N_T$ on $\{T < \infty\}$, and Fatou's lemma (the mean of a limit of non-negative variables is at most the limit of the means) gives $\mathbb{E}[N_T \mathbf{1}\{T < \infty\}] \leq \mathbb{E}[N_0]$. The three-step table in Section 34.5 walks through the computation outcome by outcome.

34.2.3. Fact 3 — Ville's inequality: a supermartingale rarely ever gets large

Markov's inequality says that a non-negative variable with mean 1 is at least c with probability at most $1/c$. Applied to N_n at one fixed time n, it bounds the chance of being large at that time. Ville's inequality says the same bound holds for the chance of being large at *any* time up to n. The reason is Fact 2: stop the first time the process reaches c. That is a legitimate stopping rule, so the stopped value still has mean at most 1, and it is at least c on every path that ever got there. For example, in a fair game you start with wealth 1, so the chance that your wealth ever reaches 10 is at most 1/10, however long you play.

If $N_t \geq 0$ is a supermartingale with $N_0 = 1$, then for every $c > 0$ and every horizon n,

$$\Pr\left[\exists\, t \leq n : N_t \geq c\right] \leq \frac{1}{c}. \tag{34.2}$$

Proof.

1. **Stop at the first crossing.** Let T be the first t with $N_t \geq c$, or n if there is none. Whether $T \geq t$ depends only on $N_1, \ldots, N_{t-1}$ (it means none of them reached c), so T is a stopping time bounded by n.
2. **The stopped process keeps its mean.** By Fact 2 (Equation 34.1), $\mathbb{E}[N_T] \leq N_0 = 1$.
3. **Apply Markov's argument to the stopped value.** On the event that the process reaches c by time n, the run stopped there, so $N_T \geq c$; elsewhere $N_T \geq 0$. So N_T is at least c times the indicator of that event, and taking means, $c \Pr[\exists\, t \leq n : N_t \geq c] \leq \mathbb{E}[N_T] \leq 1$.

The events $\{\exists\, t \leq n : N_t \geq c\}$ grow with n, and their union is $\{\exists\, t : N_t \geq c\}$, so the same bound holds with no horizon at all. This is Markov's inequality, but for the whole path at once instead of one fixed time, and at no extra cost. It is the tool for odometers.

34.3. Why adaptively chosen parameters break the usual theorems

34.3.1. Context

Advanced composition says that k steps of ε-DP, with k and ε fixed, have privacy loss at most $\varepsilon\sqrt{2k\ln(1/\delta)} + k\varepsilon\tanh(\varepsilon/2)$ except with probability δ (Equation 21.8). The first term is the typical spread of a sum of k bounded losses, the second is their mean. A tempting shortcut for adaptive runs is to apply it **ex post**: stop whenever you like, then report advanced composition of the parameters you actually used. Rogers, Roth, Ullman & Vadhan (2016) showed that this shortcut is not valid, and that a correct running bound must be larger. The problem is the choice of the stopping time: a bound that fails with probability δ at each fixed time can fail far more often at a time chosen by looking at the outputs.

34.3.2. Statement

The two objects of this chapter are defined by Rogers et al. (2016); the form below follows Whitehouse, Ramdas, Rogers & Wu (2023). Let each step be ε_t-DP conditionally on the past, meaning that whatever the history $y_{<t}$, the mechanism run at step t is ε_t-DP, with ε_t predictable.

A **privacy filter** with budget (ε_g, δ) is a rule that, before each step, looks at the parameters $\varepsilon_1, \dots, \varepsilon_t$ so far, including the one about to run, and says *continue* or *halt*. It is valid if the stopped run $y_{1:T}$ is (ε_g, δ)-DP for every analyst, however the analyst picks the steps and their parameters.

A **privacy odometer** with confidence δ is a sequence of functions $U_t(\varepsilon_1, \dots, \varepsilon_t)$, a running bound computed from the parameters spent so far. It is valid if, for every analyst and every pair of neighbours, the probability that the loss ever exceeds the running bound is at most δ:

$$\Pr\left[\exists\, t : L_{1:t} > U_t\right] \le \delta. \tag{34.3}$$

The "there exists a t" is the point. Because you may stop at any time, the bound you report at the time you stop is only safe if it holds at all times at once.

The plug-in odometer fails. Let every step be randomized response with $\varepsilon = 0.1$ on the same person's bit, and let $U_t = t\varepsilon\tanh(\varepsilon/2) + \varepsilon\sqrt{2t\ln(1/\delta)}$, advanced composition of the realized parameters. For each fixed t, $\Pr[L_{1:t} > U_t] \le \delta$. But an analyst who stops the first time $L_{1:t} > U_t$ makes the bound fail with the probability in Table 34.2, which exceeds δ and keeps growing with the horizon.

Table 34.2.: Exact probability that randomized response with $\varepsilon = 0.1$ crosses the plug-in advanced composition bound by step n, computed by dynamic programming over the loss. The last column is the largest probability of exceeding the bound at a single fixed step, over every $t \le 1000$, and the step where it occurs.

Horizon n	10	100	1000	10,000	worst single fixed $t \le 1000$
$\delta = 0.05$: $\Pr[\exists\, t \le n : L_{1:t} > U_t]$	0.0110	0.0575	0.1004	0.1424	0.0110 ($t = 17$)
$\delta = 0.01$: $\Pr[\exists\, t \le n : L_{1:t} > U_t]$	0.0000	0.0090	0.0202	0.0318	0.0016 ($t = 21$)
$\delta = 10^{-5}$: $\Pr[\exists\, t \le n : L_{1:t} > U_t]$	0	3.12×10^{-6}	1.80×10^{-5}	3.69×10^{-5}	8.18×10^{-7} ($t = 758$)

34.3.3. Derivation

The argument has two halves. The first checks that the plug-in bound is a correct bound at every fixed time. The second shows that an analyst who stops on the data turns those correct fixed-time bounds into a wrong running bound.

The fixed-t bound holds.

1. **The loss is a walk.** Take a person whose bit is 1 in D and 0 in D'. Each randomized-response report of 1 has probability $e^\varepsilon/(1+e^\varepsilon)$ on D and $1/(1+e^\varepsilon)$ on D', so its loss is $\ln e^\varepsilon = +\varepsilon$, and a report of 0 has loss $-\varepsilon$. Under D the loss is $+\varepsilon$ with probability $e^\varepsilon/(1+e^\varepsilon)$, so its mean is $\varepsilon(\frac{e^\varepsilon}{1+e^\varepsilon} - \frac{1}{1+e^\varepsilon}) = \varepsilon\frac{e^\varepsilon-1}{e^\varepsilon+1} = \varepsilon\tanh(\varepsilon/2)$, which is $0.1 \times \tanh(0.05) = 0.004996$ for $\varepsilon = 0.1$ (step 3 of Section 21.3). The total loss $L_{1:t}$ is a random walk that moves up or down by ε at each step, with a slight upward drift.
2. **Center it.** Subtract the drift: $M_t = L_{1:t} - t\varepsilon\tanh(\varepsilon/2)$. Each step adds a term with mean zero given the past, so M_t is a martingale, and its increments lie in an interval of length 2ε (from $-\varepsilon$ to $+\varepsilon$, shifted by the mean).
3. **Azuma.** Azuma's inequality (Concentration) says that a martingale with t increments, each confined to an interval of length ℓ, exceeds s with probability at most $\exp(-2s^2/(t\ell^2))$. With $\ell = 2\varepsilon$ this is $\Pr[M_t \ge s] \le \exp(-2s^2/(4t\varepsilon^2))$. Setting this to δ and solving, $s^2 = 2t\varepsilon^2\ln(1/\delta)$, so $s = \varepsilon\sqrt{2t\ln(1/\delta)}$. Adding the drift back, $\Pr[L_{1:t} > U_t] \le \delta$ at each fixed t. The exact probabilities are smaller still: over every $t \le 1000$ the largest is 0.0110 (at $t = 17$) for $\delta = 0.05$, because Azuma assumes the worst variance an increment in that interval could have.

Stopping when ahead breaks it.

4. **The analyst's rule.** Run steps until $L_{1:t} > U_t$, then stop. The analyst sees only outputs, and $L_{1:t}$ is a function of them (it is ε times the number of reports of 1 minus the number of reports of 0), so this is a legitimate stopping rule: it decides whether to continue from what has already been released.

5. **The event the odometer must control.** At the stopping time T, the claim "$L_{1:T} \le U_T$" fails exactly when the walk crossed the boundary at some $t \le n$, because that is when the analyst stops. So the failure probability is the crossing probability $\Pr[\exists\, t \le n : L_{1:t} > U_t]$, not the fixed-$t$ probability. The first is a probability over the whole path and can be much larger than the largest of the second.
6. **Compute it exactly.** $L_{1:t}/\varepsilon$ moves on the integers by ± 1. Propagate its distribution step by step, and at each t remove the probability mass that sits above U_t/ε, since those paths have crossed and stopped. The total removed mass is the crossing probability (Table 34.2).
7. **It tends to 1.** The boundary sits $\varepsilon\sqrt{2t\ln(1/\delta)}$ above the mean, a fixed multiple of $\sqrt{t}$. The centered steps take values $\pm\varepsilon$ minus the mean $\varepsilon\tanh(\varepsilon/2)$, so their variance is the mean square minus the squared mean, $\varepsilon^2 - \varepsilon^2\tanh^2(\varepsilon/2) = \varepsilon^2/\cosh^2(\varepsilon/2)$, and their standard deviation is $\varepsilon\,\mathrm{sech}(\varepsilon/2)$. The law of the iterated logarithm (a cited fact; see Durrett in the References) says how far a random walk reaches along its whole path: for any $\eta > 0$, the centered walk exceeds $(1-\eta)\,\varepsilon\,\mathrm{sech}(\varepsilon/2)\sqrt{2t\ln\ln t}$ infinitely often with probability 1. That curve grows slightly faster than any fixed multiple of $\sqrt{t}$, because of the slowly growing $\ln\ln t$. Since $\ln\ln t$ eventually exceeds $\ln(1/\delta)/\mathrm{sech}^2(\varepsilon/2)$, the walk crosses the boundary with probability 1, and the crossing probability by step n tends to 1 as $n \to \infty$.
8. **Every odometer pays for it.** Rogers et al. (2016, Theorem 6.1) turn this into a lower bound: no valid odometer can exceed the mean term by only $o(\sqrt{\sum_t \varepsilon_t^2\,\ln(\ln n/\delta)})$, that is, by an amount that becomes negligible next to that square root. Their construction is this one, stop when ahead, with n steps of $\varepsilon = 1/n$. So the price is an extra $\ln\ln n$ inside the logarithm, where a fixed-time bound has $\ln(1/\delta)$ alone.

34.3.4. Interpretation

The theorem was never wrong; its use was. Advanced composition bounds the loss at a time fixed in advance. A data-dependent time is a different random variable, and an analyst who stops when the loss happens to be high selects exactly the paths where the bound fails.

Fixed-t bounds have slack, but not enough to absorb this. At $\delta = 0.05$ the plug-in bound fails at a single time $t \le 1000$ with probability at most 0.0110, but over 10^4 steps with probability 0.1424. A valid odometer must pay for the union over times, which, by step 8, costs at least an extra $\ln\ln n$ inside the logarithm (Rogers et al., Theorem 6.1).

A small δ shrinks the price, but not the failure. Rogers et al. point out that for δ far below $1/n$, $\ln(\ln n/\delta) = \ln(1/\delta) + \ln\ln n$ is $\ln(1/\delta)$ plus a small additive term, so a valid odometer costs little more than the fixed-time bound. The plug-in bound is still invalid there: at $\delta = 10^{-5}$ it is crossed by step 10^4 with probability 3.69×10^{-5}, 3.69 times δ (Table 34.2).

Filters are different. A filter fixes the budget in advance and only asks whether the next step fits, so there is no running claim for the analyst to catch out at a bad moment. That question turns out to be answerable with no $\ln\ln$ loss (Section 34.4). So filters and odometers are genuinely different objects once parameters are adaptive, which they are not when parameters are fixed.

34.3.5. Example

With $\delta = 0.01$, the plug-in bound after $t = 100$ steps is the mean term plus the spread term: $100 \times 0.004996 + 0.1\sqrt{200\ln 100} = 3.534$. Figure 34.1 (left) shows typical runs. Most never come

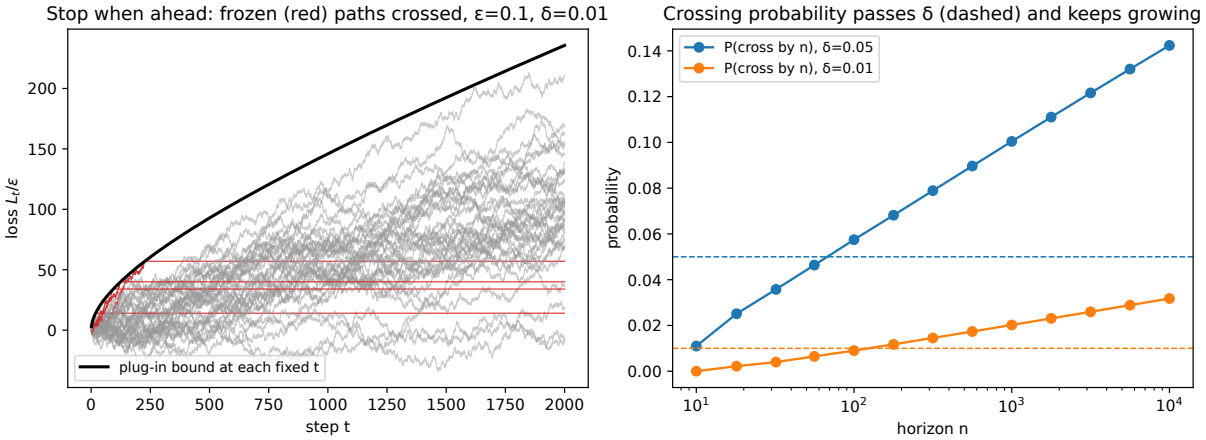


Figure 34.1.: Left: forty runs of randomized response with $\varepsilon = 0.1$, the loss in units of ε, and the plug-in advanced-composition bound at $\delta = 0.01$. An analyst who stops at the first crossing freezes the red paths there. Right: the exact probability that the bound is crossed by step n, for $\delta = 0.05$ and 0.01 (dashed lines); it passes δ and keeps growing.

near the boundary, but a few cross it early, where it is narrow because $\sqrt{t}$ is still small, and the analyst freezes them there. By $n = 10^4$, a fraction 0.0318 of runs have crossed (Table 34.2), so a report "$\varepsilon = U_T$ with $\delta = 0.01$" is wrong $0.0318/0.01 = 3.18$ times as often as it claims. At $\delta = 0.05$ the crossing probability reaches 0.1424, $0.1424/0.05 = 2.85$ times δ.

34.3.6. Check yourself

- Basic composition, $|L_{1:t}| \le \sum_{j \le t} \varepsilon_j$, does not suffer from this problem. Why? *(It holds for every output, not with high probability, so there is no bad event for the stopping rule to select. Summing realized parameters is a valid odometer and filter for pure DP, as Section 21.6 notes.)*
- Is the stop-when-ahead analyst unrealistic, since it needs the privacy loss? *(No. Here the loss is a function of the outputs, and in general an analyst can stop on any statistic correlated with it, such as "the answers look unusual". The definitions quantify over all analysts, so the bound must survive the worst one.)*

34.4. Filters for pure differential privacy

34.4.1. Context

You run an interactive system with a global budget (ε_g, δ). Each query comes with its own ε_t, chosen after seeing the earlier answers. Before each query you need a rule that says whether it may run. The simplest rule sums the ε_t; it is exact but its total grows like k, the number of steps. Advanced composition grows like $\sqrt{k}$ but, as Section 34.3 showed, cannot be applied ex post without care. Rogers et al. (2016) gave the first valid $\sqrt{k}$ filter, with worse constants than

advanced composition. Whitehouse et al. (2023), and independently the Rényi route of Feldman & Zrnic (2021), closed the gap.

34.4.2. Statement

All three filters below are valid for steps that are ε_t-DP conditionally on the past (pure DP, $\delta_t = 0$), with ε_t predictable. Each computes one statistic of $\varepsilon_1, \dots, \varepsilon_t$, including the step about to run, and says *continue* while it is at most ε_g. They differ in the statistic: the first adds the budgets, the other two add a mean term and a square-root spread term, as advanced composition does.

- **Basic filter** (Rogers et al. 2016, Theorem 3.4). The statistic is the plain sum of the budgets: continue while $\sum_t \varepsilon_t \le \varepsilon_g$. The stopped run is $(\varepsilon_g, 0)$-DP.
- **Rogers et al. filter** (2016, Theorem 5.1), for $\delta < 1/e$. The statistic is a mean term, the sum of each step's bound on its expected loss, plus a spread term like advanced composition's $\sqrt{2V\ln(1/\delta)}$, where $V = \sum_t \varepsilon_t^2$ plays the role of the variance. Because V is now random, the spread term is inflated: V is padded by $x = \varepsilon_g^2/(28.04\ln(1/\delta))$, and multiplied by a logarithmic factor in V/x. Continue while

$$K = \sum_t \frac{\varepsilon_t(e^{\varepsilon_t}-1)}{2} + \sqrt{2(V+x)\Big(1+\tfrac{1}{2}\ln\big(\tfrac{V}{x}+1\big)\Big)\ln(1/\delta)} \le \varepsilon_g. \tag{34.4}$$

- **Whitehouse et al. filter** (2023, Theorem 2). The statistic is advanced composition itself, spread term $\sqrt{2\ln(1/\delta)\sum_t \varepsilon_t^2}$ plus mean term $\frac{1}{2}\sum_t \varepsilon_t^2$, with no penalty for adaptivity. Continue while

$$\sqrt{2\ln(1/\delta)\textstyle\sum_t \varepsilon_t^2} + \tfrac{1}{2}\textstyle\sum_t \varepsilon_t^2 \le \varepsilon_g. \tag{34.5}$$

The last two give (ε_g, δ)-DP. Compare Equation 34.5 with advanced composition for unequal steps: it is the same formula with the tanh mean term $\varepsilon_t \tanh(\varepsilon_t/2)$ replaced by the slightly larger $\varepsilon_t^2/2$. It is larger because $\tanh x \le x$, so $\varepsilon_t \tanh(\varepsilon_t/2) \le \varepsilon_t \cdot \varepsilon_t/2$; the difference is about $\varepsilon_t^4/24$ per step, from the next term of the series $\tanh x = x - x^3/3 + \dots$. For fixed parameters it is exactly the bound you get by composing in zCDP, so adaptivity costs essentially nothing.

34.4.3. Derivation

Basic filter. The idea is that a pure-DP step bounds the loss of every single output, not just its typical size, so the sum of the budgets bounds the loss of every stopped transcript.

1. **Each step's loss is bounded on every output.** ε_t-DP says no output is more than e^{ε_t} times likelier on one neighbour than on the other, in either direction. Taking logarithms, $|L_t| \le \varepsilon_t$ for every output.
2. **So is the stopped run's loss.** By Fact 1 the stopped run has loss $L_{1:T} = \sum_{t \le T} L_t$, so $|L_{1:T}| \le \sum_{t \le T} \varepsilon_t \le \varepsilon_g$ on every path, because the filter never lets the sum pass ε_g.
3. **A bounded loss is pure DP.** A loss bounded by ε_g on every output means $P(y) \le e^{\varepsilon_g} Q(y)$ for every transcript y. Summing over the transcripts in any event gives $P(E) \le e^{\varepsilon_g} Q(E)$, which is $(\varepsilon_g, 0)$-DP.

Rogers et al. filter (what it needs; the proof is in their Section 5). It uses the centered loss of step 2 in Section 34.3, a martingale whose increments lie in intervals of length $2\varepsilon_t$; the difference is that these lengths are now random. Azuma's inequality needs the lengths fixed in advance, so Rogers et al. replace it with a **self-normalized** inequality (their Theorem 4.4, built on a result of de la Peña, Klass & Lai): a bound on the martingale divided by $\sqrt{V+x}$, where V, the sum of squared increment ranges up to a constant, may itself be random. Dividing by the realized spread is what lets the bound hold whatever V turns out to be, and the price is the extra x inside the square root and the factor $1+\frac{1}{2}\ln(V/x+1)$; the constant 28.04 is their choice of x. The mean term bounds each step's expected loss given the past, $\mathbb{E}[L_t \mid y_{<t}]$, by $\varepsilon_t(e^{\varepsilon_t}-1)/2$.

Whitehouse et al. filter, through the Rényi filter of the next section. The idea is to measure each pure-DP step in zCDP, where adaptive budgets add up for free at every order (the Rényi filter, Equation 34.6), and to convert to (ε,δ)-DP only at the end. Recall that a mechanism is $(\alpha,\varepsilon(\alpha))$-RDP if the Rényi divergence of order α between its outputs on any two neighbours is at most $\varepsilon(\alpha)$, and ρ-zCDP if it is $(\alpha,\alpha\rho)$-RDP at every order $\alpha>1$: its RDP curve is at most a line through the origin with slope ρ.

1. **Each step is zCDP.** An ε_t-DP step is $(\alpha,\alpha\varepsilon_t^2/2)$-RDP for every $\alpha>1$ (Equation 29.6), that is, $\varepsilon_t^2/2$-zCDP. The derivation is in Concentrated differential privacy; it uses that the step's loss lies in $[-\varepsilon_t,\varepsilon_t]$, so its exponential moments are controlled by its range.
2. **The rule is a Rényi filter at every order at once.** Let $\rho^*=(\sqrt{\ln(1/\delta)+\varepsilon_g}-\sqrt{\ln(1/\delta)})^2$ (Equation 29.12; step 3 shows where it comes from) and continue while $\frac{1}{2}\sum_t\varepsilon_t^2\le\rho^*$. For any fixed α, multiplying both sides by α shows this is the same as $\sum_t\alpha\varepsilon_t^2/2\le\alpha\rho^*$: a Rényi filter with budget $\alpha\rho^*$ on the spends of step 1. By Equation 34.6, the stopped run is $(\alpha,\alpha\rho^*)$-RDP. Since this holds for every α, the stopped run is ρ^*-zCDP.
3. **Convert.** ρ^*-zCDP gives $(\rho^*+2\sqrt{\rho^*\ln(1/\delta)},\delta)$-DP (Equation 29.10). This is Mironov's conversion (Equation 27.17), $\varepsilon(\alpha)+\ln(1/\delta)/(\alpha-1)$, at the best order: with $a=\alpha-1$ and $\ell=\ln(1/\delta)$ it reads $\rho+\rho a+\ell/a$, whose minimum over a is at $a=\sqrt{\ell/\rho}$, where it equals $\rho+2\sqrt{\rho\ell}$. The choice of ρ^* makes this exactly ε_g: $\rho+2\sqrt{\rho\ell}=\varepsilon_g$ is $(\sqrt{\rho}+\sqrt{\ell})^2=\varepsilon_g+\ell$, whose solution is $\sqrt{\rho}=\sqrt{\varepsilon_g+\ell}-\sqrt{\ell}$.
4. **Rewrite the rule.** $\rho\mapsto\rho+2\sqrt{\rho\ln(1/\delta)}$ is increasing, so "$\rho\le\rho^*$" is the same as "$\rho+2\sqrt{\rho\ln(1/\delta)}\le\varepsilon_g$". With $\rho=\frac{1}{2}\sum_t\varepsilon_t^2$, the square root is $2\sqrt{\frac{1}{2}\sum_t\varepsilon_t^2\ln(1/\delta)}=\sqrt{2\ln(1/\delta)\sum_t\varepsilon_t^2}$, so the left side is $\frac{1}{2}\sum_t\varepsilon_t^2+\sqrt{2\ln(1/\delta)\sum_t\varepsilon_t^2}$, which is Equation 34.5.

This is the theorem that Composition (Section 21.6) said a budget-spending rule needs. Whitehouse et al. prove the same filter directly, and also for steps with $\delta_t>0$, through approximate zCDP; Feldman & Zrnic (2021, Section 4.3) state the ρ^* form.

34.4.4. Interpretation

A plug-in filter on the Dwork–Rothblum–Vadhan form of advanced composition is valid after all. That form, Equation 21.7, written for unequal steps, has the statistic $\sqrt{2\ln(1/\delta)\sum_t\varepsilon_t^2}+\sum_t\varepsilon_t(e^{\varepsilon_t}-1)$. Since $e^x-1\ge x$, every step has $\varepsilon_t(e^{\varepsilon_t}-1)\ge\varepsilon_t^2\ge\varepsilon_t^2/2$, so this statistic is never below that of Equation 34.5. A filter that halts on it therefore halts no later than the Whitehouse filter, and stopping earlier is post-processing of the longer run: anything computed from a prefix of a private transcript is at least as private. The argument does not cover the tanh

form (Equation 21.8) used in Section 34.3, whose statistic is slightly below Whitehouse's. What fails there is the odometer use, not a filter.

Rogers et al. lose a constant factor. In Table 34.3 the Rogers filter needs 1.40 to 1.51 times the budget of the Whitehouse filter, and it lets through fewer than half as many steps (Table 34.4). The loss comes from the self-normalized inequality, which has to work for every value of the variance proxy V at once.

All of this is for pure DP. For steps with $\delta_t > 0$, Whitehouse et al. add a second rule, $\sum_t \delta_t \le \delta''$, and the guarantee becomes $(\varepsilon_g, \delta + \delta'')$. Rogers et al. (Lemma 3.3) instead run their pure-DP filter on the doubled parameters $2\varepsilon_t$ and halt once $\sum_t 2\delta_t/(\varepsilon_t e^{\varepsilon_t})$ exceeds δ'', with the same $(\varepsilon_g, \delta + \delta'')$.

34.4.5. Example

Take k steps of ε_0 each at $\delta = 10^{-6}$ and ask what global budget each rule needs to let them all through. For the basic filter that is $k\varepsilon_0$; for the others, it is the value of the statistic after k steps. Table 34.3 compares basic composition, advanced composition with k fixed in advance (Equation 21.7), and the two $\sqrt{k}$ filters; Figure 34.2 plots the same over k.

Table 34.3.: Global budget ε_g needed for k steps of ε_0 at $\delta = 10^{-6}$. For the Rogers filter, the smallest ε_g at which Equation 34.4 holds.

ε_0	k	Basic	Advanced (fixed k)	Rogers filter	Whitehouse filter	Rogers / Whitehouse
0.01	100	1.000	0.536	0.803	0.531	1.51
0.01	1000	10.000	1.763	2.570	1.712	1.50
0.01	10,000	100.000	6.262	8.450	5.757	1.47
0.1	100	10.000	6.308	8.472	5.757	1.47
0.1	1000	100.000	27.140	30.244	21.623	1.40

Turned around: with a budget of $\varepsilon_g = 1$, how many steps of ε_0 does each rule allow (Table 34.4)? For the Whitehouse filter, $\rho^* = (\sqrt{\ln(10^6) + 1} - \sqrt{\ln(10^6)})^2 = 0.017469$, and the rule $\frac{1}{2}k\varepsilon_0^2 \le \rho^*$ allows $\lfloor 2\rho^*/\varepsilon_0^2 \rfloor$ steps.

Table 34.4.: Number of steps of ε_0 allowed by a global budget $\varepsilon_g = 1$ at $\delta = 10^{-6}$.

ε_0	Basic	Advanced (fixed k)	Rogers filter	Whitehouse filter
0.01	100	337	154	349
0.001	1000	33,785	15,481	34,937

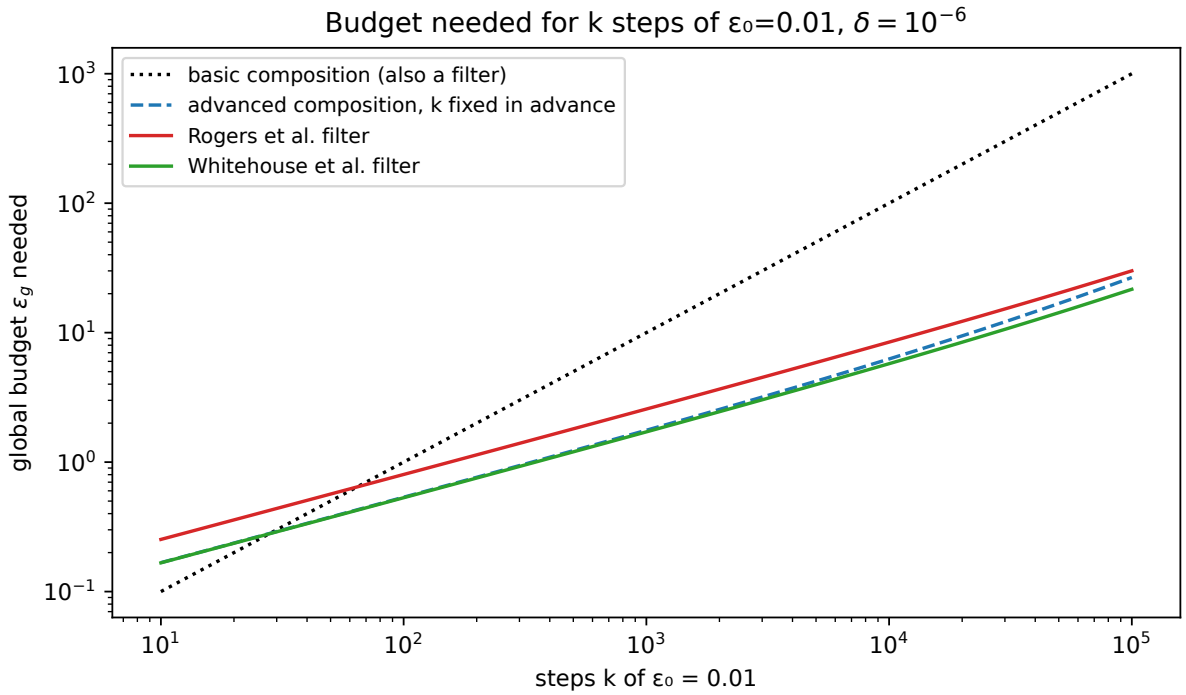


Figure 34.2.: Global budget needed to run k steps of $\varepsilon_0 = 0.01$ at $\delta = 10^{-6}$, on log–log axes: basic composition, advanced composition with k fixed in advance, and the Rogers et al. and Whitehouse et al. filters. The Whitehouse filter lies on or below fixed-k advanced composition; the Rogers filter sits a constant factor above.

34.4.6. Check yourself

- Why does the Whitehouse filter beat fixed-k advanced composition from Equation 21.7, even though it allows adaptivity? *(Its mean term is $\varepsilon_t^2/2$ instead of $\varepsilon_t(e^{\varepsilon_t}-1)$. It comes from the zCDP view of a pure-DP step, which is tighter than the Dwork–Rothblum–Vadhan mean bound. Against the* tanh *form Equation 21.8 the two are nearly equal, since $\varepsilon\tanh(\varepsilon/2)\approx\varepsilon^2/2$.)*
- In step 2 of the Whitehouse derivation, why is it important that the halting rule does not depend on α? *(Each Rényi filter is valid at one fixed order. Because $\frac{1}{2}\sum\varepsilon_t^2\le\rho^*$ is the same rule for every α, one run is a valid filter at all orders simultaneously, so the conversion may pick the best α afterwards.)*

34.5. The Rényi filter

34.5.1. Context

Rényi differential privacy describes a mechanism by a curve. The Rényi divergence of order $\alpha>1$ between the output distributions on two neighbours is $D_\alpha(P\|Q)=\frac{1}{\alpha-1}\ln\mathbb{E}_P\big[e^{(\alpha-1)L}\big]$, a log of an exponential moment of the privacy loss L, and a mechanism is $(\alpha,\varepsilon(\alpha))$-RDP if this is at most $\varepsilon(\alpha)$ for every pair of neighbours. RDP composition adds curves: if step t is $(\alpha,\varepsilon_t(\alpha))$-RDP with ε_t fixed in advance, the run is $(\alpha,\sum_t\varepsilon_t(\alpha))$-RDP (Equation 27.10). What if $\varepsilon_t(\alpha)$ depends on the earlier outputs, because the noise is adapted, because the run stops on a data-dependent rule, or because it is one record's individual spend? Feldman & Zrnic (2021) and Lécuyer (2021) showed that nothing is lost, at a fixed order.

34.5.2. Statement

Fix an order and a budget, add up the RDP spends as the run goes, and stop before the sum would pass the budget. The result is that the stopped run has the guarantee the budget names, exactly as if the spends had been fixed in advance.

Rényi filter (Feldman & Zrnic 2021, Section 4; Lécuyer 2021, Section 3). Fix an order $\alpha>1$ and a budget B. Let step t be $(\alpha,\varepsilon_t(\alpha))$-RDP conditionally on $y_{<t}$, with $\varepsilon_t(\alpha)$ predictable. Run step t only if the spends so far, including its own, fit in the budget:

$$\sum_{j\le t}\varepsilon_j(\alpha)\le B. \tag{34.6}$$

Then the stopped run is (α,B)-RDP. Equivalently: if $\sum_t\varepsilon_t(\alpha)\le B$ holds on every path, the adaptive composition is (α,B)-RDP.

34.5.3. Derivation

The plan is to build a supermartingale out of the privacy loss and the spends, stop it with the filter, and read the RDP bound off its mean. The quantity that works is $e^{(\alpha-1)L}$, because its

mean is exactly what the Rényi divergence measures, divided by what the spends allow it to be. Fix neighbours D, D' and the order α, and define

$$N_t = \exp\Big((\alpha-1)\Big(L_{1:t} - \sum_{j\le t}\varepsilon_j(\alpha)\Big)\Big), \qquad N_0 = 1. \tag{34.7}$$

1. **One step's exponential moment is the RDP parameter.** Given $y_{<t}$, step t is a fixed mechanism, so by the definition of D_α as a moment of the loss (Equation 29.1), $\mathbb{E}[e^{(\alpha-1)L_t} \mid y_{<t}] = e^{(\alpha-1)D_\alpha(P_t\|Q_t)}$. The RDP assumption $D_\alpha(P_t\|Q_t) \le \varepsilon_t(\alpha)$ then bounds it by $e^{(\alpha-1)\varepsilon_t(\alpha)}$. The moment form holds because $\mathbb{E}_P[e^{(\alpha-1)L}] = \sum_y P(y)(P(y)/Q(y))^{\alpha-1} = \sum_y P(y)^\alpha Q(y)^{1-\alpha}$, the sum inside the Rényi divergence.
2. **N_t is a supermartingale.** Going from $t-1$ to t adds L_t to the loss and $\varepsilon_t(\alpha)$ to the spend, so $N_t = N_{t-1}\, e^{(\alpha-1)(L_t - \varepsilon_t(\alpha))}$. Both N_{t-1} and $\varepsilon_t(\alpha)$ are functions of $y_{<t}$, so given $y_{<t}$ they are constants and come out of the conditional expectation. What remains is $\mathbb{E}[e^{(\alpha-1)L_t} \mid y_{<t}]\, e^{-(\alpha-1)\varepsilon_t(\alpha)} \le 1$ by step 1, so $\mathbb{E}[N_t \mid y_{<t}] \le N_{t-1}$. This is where predictability is used.
3. **Stop it.** The filter's last step T is a stopping time: whether step t runs is decided by $\varepsilon_1(\alpha), \dots, \varepsilon_t(\alpha)$, which are functions of $y_{<t}$. By Fact 2, $\mathbb{E}[N_T] \le N_0 = 1$.
4. **Use the budget on every path.** Undo the definition of N_T: $e^{(\alpha-1)L_{1:T}} = N_T\, e^{(\alpha-1)\sum_{j\le T}\varepsilon_j(\alpha)}$. The filter guarantees $\sum_{j\le T}\varepsilon_j(\alpha) \le B$ on every path, so this is at most $N_T\, e^{(\alpha-1)B}$. Take expectations: $\mathbb{E}\big[e^{(\alpha-1)L_{1:T}}\big] \le e^{(\alpha-1)B}\,\mathbb{E}[N_T] \le e^{(\alpha-1)B}$.
5. **Read off the divergence.** By Fact 1, $L_{1:T}$ is the privacy loss of the stopped transcript, so by the moment form again the left side is $e^{(\alpha-1)D_\alpha(P_{1:T}\|Q_{1:T})}$, and taking logs gives $D_\alpha \le B$. Swapping the roles of D and D' gives the other direction.

Step 4 is where "on every path" matters. If $\sum_j \varepsilon_j(\alpha) \le B$ held only on average, you could not pull $e^{(\alpha-1)\sum_j \varepsilon_j}$ out of the expectation as the constant $e^{(\alpha-1)B}$: paths with a large spend may also be paths with a large N_T, and the expectation of a product is not the product of expectations.

The supermartingale, outcome by outcome. Rényi DP (Table 27.5) already composed two adaptive randomized responses, $\ln 3$ and then $\ln 3$ or $\ln 1.5$ depending on y_1. With two steps, fixed-curve composition (Equation 27.10, charging step 2 its worst branch) gives the same bound as the filter, the largest path spend. Three steps whose branches swap costs show what the filter adds.

Take $\alpha = 2$, so $\alpha - 1 = 1$ and $N_t = e^{L_{1:t} - \text{spend}}$, and one person's bit, 1 in D and 0 in D'. Step 1 is randomized response with $\varepsilon = \ln 3$: it reports the true bit with probability $3/(1+3) = 3/4$. Its RDP at order 2, from the randomized-response curve (Equation 27.9) with $p = 3/4$, is $\varepsilon_1(2) = \ln\left(p^2/(1-p) + (1-p)^2/p\right) = \ln(2.25 + 0.0833) = 0.8473$. After $y_1 = 1$, steps 2 and 3 are randomized response with $\ln 3$ (strong) and then $\ln 1.5$ (weak, keep-probability $1.5/2.5 = 0.6$, RDP $\ln(0.36/0.4 + 0.16/0.6) = 0.1542$); after $y_1 = 0$, weak and then strong. Every path runs one strong step after the first, so every path spends $B = 2\varepsilon_1(2) + 0.1542 = 1.8487$ (computed before rounding), the filter's budget. Fixed-curve composition must charge each of steps 2 and 3 its worst branch, 0.8473, and gives $3 \times 0.8473 = 2.5419$.

Table 34.5 lists the eight transcripts. Read the first row as a template. On D the bit is 1, so the three reports of 1 have probabilities 0.75 (strong), 0.75 (strong) and 0.6 (weak), and $P = 0.75 \times 0.75 \times 0.6 = 0.3375$. On D' each report of 1 is a flip, so $Q = 0.25 \times 0.25 \times 0.4 = 0.0250$. The loss is $\ln(0.3375/0.0250) = \ln 13.5 = 2.6027$, and $N_3 = e^{2.6027 - 1.8487} = 2.1254$.

Table 34.5.: Three adaptive randomized-response steps at $\alpha = 2$. The first output decides the order of a strong ($\ln 3$) and a weak ($\ln 1.5$) step, so every path spends the same; $N_3 = e^{L_{1:3} - \text{spend}}$.

(y_1, y_2, y_3)	P	Q	$L_{1:3}$	spend $\sum_j \varepsilon_j(2)$	N_3	$P \times N_3$
(1, 1, 1)	0.3375	0.0250	+2.6027	1.8487	2.1254	0.7173
(1, 1, 0)	0.2250	0.0375	+1.7918	1.8487	0.9446	0.2125
(1, 0, 1)	0.1125	0.0750	+0.4055	1.8487	0.2362	0.0266
(1, 0, 0)	0.0750	0.1125	−0.4055	1.8487	0.1050	0.0079
(0, 1, 1)	0.1125	0.0750	+0.4055	1.8487	0.2362	0.0266
(0, 1, 0)	0.0375	0.2250	−1.7918	1.8487	0.0262	0.0010
(0, 0, 1)	0.0750	0.1125	−0.4055	1.8487	0.1050	0.0079
(0, 0, 0)	0.0250	0.3375	−2.6027	1.8487	0.0117	0.0003

The table checks two things. First, the mean of N_3 is 1. The mean is the sum of the last column, and the unrounded entries sum to 1.0000. You can also see the supermartingale step by step. After one step, $N_1 = e^{L_1 - 0.8473}$: given $y_1 = 1$ it is $e^{\ln 3 - 0.8473} = 1.2857$, and given $y_1 = 0$ it is $e^{-\ln 3 - 0.8473} = 0.1429$. Its mean is $0.75 \times 1.2857 + 0.25 \times 0.1429 = 1.0000$, and the conditional mean of N_3 on each branch equals that branch's N_1. For randomized response, step 1 of the derivation holds with equality, because the RDP parameter is the exact divergence, so N is a martingale rather than merely a supermartingale.

Second, the filter bound is attained, and it beats fixed curves. The joint D_2 of the transcript is $\ln \mathbb{E}_P[e^{L_{1:3}}] = \ln \sum P^2/Q = 1.8487$, exactly B, because N is a martingale and every path spends B. Fixed-curve composition gives 2.5419: it charges both later steps as if both were strong, though no path runs two strong steps after the first.

34.5.4. Interpretation

At a fixed order, the filter is free. Its bound is the largest path spend, which is never more than fixed-parameter composition gives, and less when branches trade costs (Table 34.5). When every path spends exactly B and every step's exponential moment equals its RDP parameter, as for Gaussian steps and randomized response, N is a martingale and the bound is attained. In Figure 34.3, adaptive Gaussian steps (shift 0.3 after a positive output, 0.6 otherwise, in units of the noise; $\alpha = 2$, $B = 1$) are stopped by the filter. Each path's $\ln N_t$ wanders, but the mean of N_t stays at 1. When the last step is shrunk to spend the remaining budget exactly, a Monte Carlo estimate over 10^6 paths gives $D_2 = 1.0004$, the budget up to sampling error. When the filter simply stops, paths end with a mean spend of 0.8998 and a largest spend of 0.9900, and $D_2 = 0.9305$: below the budget, because no path uses all of it.

The price is the order. The filter is valid at an order fixed before the run. You can track several orders and stop when any one of them is exhausted; then the budget held at every order on every path, and every one of them is certified. What you cannot do is let the run continue as long as *some* order still fits, with the order chosen after seeing the spends, without paying for it; Section 34.6 prices that choice with a union bound.

At the end you convert. The stopped run is (α, B)-RDP, so by Mironov's conversion (Equation 27.17) it is $(B + \ln(1/\delta)/(\alpha - 1), \delta)$-DP for every δ. That conversion is a Chernoff bound: the exponential moment $e^{(\alpha-1)B}$ bounds the probability that the loss exceeds $B + \ln(1/\delta)/(\alpha - 1)$ by δ, and outputs whose loss is below that threshold satisfy the e^{ε} ratio. A DP target (ε_g, δ) therefore fixes $B = \varepsilon_g - \ln(1/\delta)/(\alpha - 1)$ at the chosen order.

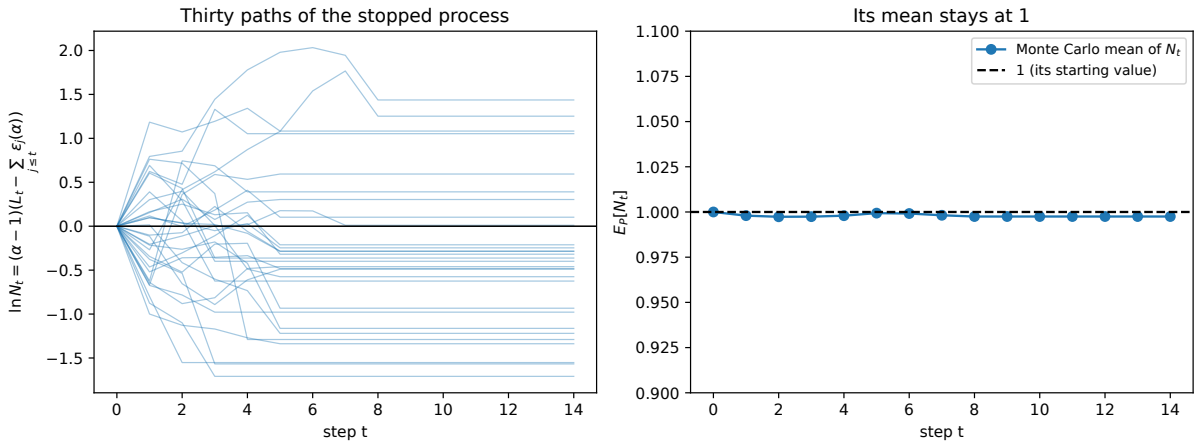


Figure 34.3.: Adaptive Gaussian steps under a Rényi filter with $\alpha = 2$ and $B = 1$. Left: $\ln N_t$ along thirty paths; each path freezes when the filter stops it. Right: the Monte Carlo mean of N_t over 2×10^5 paths stays at its starting value 1, as optional stopping says.

34.5.5. Example

You train with full-batch noisy gradient descent, noise multiplier σ, and you want to lower the noise when progress stalls. With a budget of $(\varepsilon_g, \delta) = (5.299, 10^{-5})$ and the order $\alpha = 5.8$, the RDP budget is $B = 5.299 - \ln(10^5)/4.8 = 2.900$ (these are the numbers of Section 34.7). A Gaussian step with noise multiplier σ_t spends $\alpha/(2\sigma_t^2)$ at order α (Equation 27.7), because two Gaussians one noise-width apart have Rényi divergence $\alpha/2$ in units of the noise, and dividing the shift by σ_t divides that by σ_t^2. With $\sigma_t = 10$ throughout each step spends $5.8/200 = 0.0290$, and you get $2.900/0.0290 = 100$ steps. If you switch to $\sigma_t = 5$ after a plateau at step 60, each step costs $5.8/50 = 0.1160$, and the filter allows $(2.900 - 60 \times 0.0290)/0.1160 = 1.160/0.1160 = 10$ more steps. Either way the run is $(5.8, 2.900)$-RDP, whatever rule decided the switch.

34.5.6. Check yourself

- In step 2 of the derivation, what goes wrong if $\varepsilon_t(\alpha)$ may depend on y_t, the output of step t itself? *(It no longer comes out of the conditional expectation given $y_{<t}$. A mechanism could then spend little exactly on the outputs that reveal the most, and N need not be a supermartingale.)*
- In Table 34.5 the joint D_2 equals the budget exactly, while in the two-step run of Table 27.5 it is 1.6766, below the largest path spend 1.6946. What makes the difference? *(In the two-step run the paths through the weak branch spend only 1.0014 and pull $\mathbb{E}_P[e^L]$ below e^B. Here every path spends B and N is a martingale, so $\mathbb{E}_P[e^{L_{1:3}}] = e^B\,\mathbb{E}_P[N_3] = e^B$.)*

34.6. Odometers

34.6.1. Context

Early stopping, "accuracy first" training and open-ended analyses do not have a budget to stop at. They need a running answer to "how much have I spent?" that stays valid however the stopping time is chosen. That is the odometer of Equation 34.3: a bound U_t such that the loss exceeds it at *some* time with probability at most δ. Section 34.3 showed that plugging the realized parameters into a fixed-time bound is not one. The cost of an odometer is a union over the times at which you might stop, or over the orders you might pick.

34.6.2. Statement

Each odometer below takes the running RDP spend at a few orders and turns it into a running ε. Let step t be $(\alpha, \varepsilon_t(\alpha))$-RDP conditionally on the past, for each order α in a finite grid Λ fixed in advance ($|\Lambda|$ is the number of orders), and write $S_t(\alpha) = \sum_{j \le t} \varepsilon_j(\alpha)$ for the spend so far at order α.

Ville odometer. Report, at every step, Mironov's conversion of the spend so far, at the best order in the grid, with δ replaced by $\delta/|\Lambda|$ to pay for choosing the order late. With probability at least $1 - \delta$, simultaneously for all t,

$$L_{1:t} \le U_t = \min_{\alpha \in \Lambda} \Big[S_t(\alpha) + \frac{\ln(|\Lambda|/\delta)}{\alpha - 1} \Big]. \tag{34.8}$$

Lécuyer odometer (Lécuyer 2021, Section 4). Instead of applying Ville's inequality to the spend directly, run a sequence of Rényi filters with doubling budgets, and report the budget of the first filter that has not yet run out, plus a conversion term at a confidence that shrinks with the filter's index. For each order, the budgets are $B_f(\alpha) = 2^{f-1} \ln(2|\Lambda|/\delta)/(\alpha - 1)$, $f = 1, 2, ...$, and $f(\alpha)$ is the first f whose budget covers $S_t(\alpha)$. Then with probability at least $1 - \delta$,

$$L_{1:t} \le \min_{\alpha \in \Lambda} \Big[B_{f(\alpha)}(\alpha) + \frac{\ln(2|\Lambda| f(\alpha)^2/\delta)}{\alpha - 1} \Big]. \tag{34.9}$$

Pure-DP odometers. Rogers et al. (2016, Theorem 6.5) give an advanced-composition odometer whose square-root term carries $\ln(\ln n/\delta)$ instead of $\ln(1/\delta)$, the $\ln\ln$ price of Section 34.3. Whitehouse et al. (2023, Theorem 3) give tighter ones, matching advanced composition at a preselected point or, up to a $\ln\ln$ factor, at all times.

34.6.3. Derivation

Ville odometer. The supermartingale of the Rényi filter does not need a filter to be a supermartingale; the filter only stopped it. Ville's inequality says it rarely ever gets large, and "N_t large" unpacks to "the loss is far above the spend".

1. **At a fixed order, N_t of Equation 34.7 is a non-negative supermartingale with $N_0 = 1$.** Steps 1–2 of the Rényi filter derivation showed this using only that the spends are predictable; no filter is needed.

2. **Ville** (Fact 3, Equation 34.2, with no horizon) with $c = |\Lambda|/\delta$: $\Pr[\exists\, t : N_t \geq |\Lambda|/\delta] \leq \delta/|\Lambda|$.
3. **Unpack.** Take logarithms and divide by $\alpha - 1$: $N_t \geq |\Lambda|/\delta$ is the event $L_{1:t} \geq S_t(\alpha) + \ln(|\Lambda|/\delta)/(\alpha - 1)$. So, at this order, the bound $S_t(\alpha) + \ln(|\Lambda|/\delta)/(\alpha - 1)$ holds for all t at once except with probability $\delta/|\Lambda|$.
4. **Union over the orders.** The probability that the bound fails at some order is at most the sum of the failure probabilities, $|\Lambda| \times \delta/|\Lambda| = \delta$ (the union bound). Outside the failure event the bound holds at every order and every time, so it holds at the minimizing order, chosen after the fact.

With a single order ($|\Lambda| = 1$) the Ville odometer is $S_t(\alpha) + \ln(1/\delta)/(\alpha - 1)$, which is Mironov's fixed-time conversion: at one fixed order, a running bound is free. The cost of an odometer is entirely the freedom to choose the order late.

Lécuyer odometer (what it needs). Each filter in the sequence certifies (α, B_f)-RDP of the run truncated at its halting time, and is converted with a tail bound at confidence $\delta/(2|\Lambda|f^2)$, which gives the term $\ln(2|\Lambda|f^2/\delta)/(\alpha - 1)$. The confidences shrink with f so that their total stays below δ: the sum over f and α is $\delta \sum_f 1/(2f^2)$, and $\sum_f 1/(2f^2) = \pi^2/12 < 1$. Because the budgets double, the certified $B_{f(\alpha)}$ is at most twice the actual spend once the spend exceeds B_1, so rounding up to the next filter costs at most a factor 2 in that term.

34.6.4. Interpretation

At a single order, the Ville odometer is exactly tight for Gaussian steps. To see why, look at the loss in the right units. A Gaussian step with shift μ, in units of the noise, has privacy loss that is itself normal, with mean $\mu^2/2$ and variance μ^2, so after t steps the loss is normal with mean $t\mu^2/2$ and variance $t\mu^2$. Write $s = t\mu^2$ for the accumulated variance. Then $L_{1:t} = s/2 + W_s$, where W is a Brownian motion (the continuous-time limit of a random walk, with $W_s \sim \mathcal{N}(0, s)$ and independent increments) observed at the variance times s. The spend at order α is $\alpha\mu^2/2$ per step, so the Ville boundary is the line $\alpha s/2 + \ln(1/\delta)/(\alpha - 1)$. Subtracting the drift $s/2$, the walk crosses it when $W_s \geq a + bs$ with $a = \ln(1/\delta)/(\alpha - 1)$ and $b = (\alpha - 1)/2$.

Ville's inequality gives the crossing probability in continuous time too. The process $\exp(2bW_s - 2b^2 s)$ is a non-negative martingale with mean 1, because the Gaussian moment generating function gives $\mathbb{E}[e^{2bW_s}] = e^{(2b)^2 s/2} = e^{2b^2 s}$. On the event $W_s \geq a + bs$ its exponent is at least $2b(a + bs) - 2b^2 s = 2ab$, so that event is the event that the martingale reaches e^{2ab}, and the probability is at most e^{-2ab}. It is exactly e^{-2ab}: a continuous path meets the line exactly, where the martingale equals e^{2ab}, and on paths that never cross, the martingale tends to 0, so optional stopping holds with equality. Here $2ab = \ln(1/\delta)$, so $e^{-2ab} = \delta$. A Monte Carlo check with $\mu = 0.2$, $\alpha = 2$, $\delta = 0.05$ over 1000 steps gives a crossing probability of 0.0461, against the continuous-time limit 0.0500: checking only at whole steps misses crossings between them. At any single fixed time the same line is crossed with probability at most 0.0072. The slack that Mironov's conversion has at a fixed time is precisely what pays for time-uniformity. With more than one order the union bound adds slack, and the odometer is no longer tight.

A small grid is cheaper. The union costs $\ln|\Lambda|$ in the numerator: $\ln(1/\delta) = \ln(10^5) = 11.513$ against $\ln(|\Lambda|/\delta) = 11.513 + \ln 154 = 16.550$ for the part's 154 orders at $\delta = 10^{-5}$. Heuristically, a coarse grid of a few orders per factor of two in spend is enough: if the spend may range over $[S_{\min}, S_{\max}]$, such a grid has $|\Lambda| \approx \log_2(S_{\max}/S_{\min})$ orders, and the union costs $\ln|\Lambda|$, a $\ln\ln$ of the range. This is the same shape as the $\ln\ln$ term in Rogers et al.'s lower bound.

Odometers are tail bounds, not DP guarantees. Equation 34.3 bounds the tail of the loss at whatever time you stop: reporting "$\varepsilon = U_T$" means "the loss of this run exceeds U_T with probability at most δ", with U_T itself random. To turn it into an ordinary (ε_g, δ)-DP guarantee, stop by a rule that keeps $U_T \le \varepsilon_g$, and split any event E by whether the loss is above ε_g: where it is not, $P(y) \le e^{\varepsilon_g} Q(y)$, and where it is, the probability is at most $\Pr[L > \varepsilon_g]$. That gives $\Pr[E] \le e^{\varepsilon_g} Q(E) + \Pr[L > \varepsilon_g]$ (steps 1–2 of Section 27.8). The second check-yourself question below does this.

34.6.5. Example

Take full-batch Gaussian steps with $\sigma = 10$, so $S_t(\alpha) = \alpha t/(2\sigma^2)$, $\delta = 10^{-5}$ and the part's grid of 154 orders. Table 34.6 compares the fixed-time conversion, valid only if t was chosen in advance, with the two odometers and with a Ville odometer at the single order 5.8, chosen in advance for $t = 100$. The last column is easy to check by hand: at order 5.8 each step spends $5.8/200 = 0.029$ and the conversion term is $\ln(10^5)/4.8 = 2.399$, so at $t = 10$ it is $0.29 + 2.399 = 2.689$ and at $t = 1000$ it is $29 + 2.399 = 31.399$. Figure 34.4 plots them over t.

Table 34.6.: Running ε at $\delta = 10^{-5}$ for t full-batch Gaussian steps with $\sigma = 10$. Only the last three columns are valid when t is chosen from the outputs.

Steps t	Fixed-time conversion	Ville odometer (154 orders)	Lécuyer odometer	Ville at $\alpha = 5.8$ only
10	1.568	1.869	1.916	2.689
100	5.299	6.253	6.507	5.299
300	9.812	11.465	11.892	11.099
1000	20.175	23.194	24.633	31.399

The Ville odometer costs 15–19% over the fixed-time value (ratios 1.15 to 1.19 across the rows): 6.253 against 5.299 at $t = 100$. The single-order odometer is free at its design point and poor elsewhere, 31.399 at $t = 1000$, because its order is far from the best one there. For the part's running example ($q = 0.01$, $\sigma = 1$, 10^4 steps, $\delta = 10^{-5}$, with the subsampled-Gaussian curve of Amplification by subsampling, where each step uses a random fraction q of the records), Mironov's fixed-time conversion gives 7.43; an odometer valid whenever the run stops costs 8.82 (Ville) or 9.08 (Lécuyer).

34.6.6. Check yourself

- Why is a filter cheaper than an odometer? *(A filter only certifies one budget fixed in advance, so it can use one order and one stopping time. An odometer must be correct at whatever time and order turn out best, and pays a union bound over them.)*
- Can you build a filter for a DP target (ε_g, δ) that continues while *some* order in Λ still fits, $\min_\alpha[S_t(\alpha) + \ln(|\Lambda|/\delta)/(\alpha - 1)] \le \varepsilon_g$? *(Yes: it stops before the Ville odometer passes ε_g, so the stopped loss exceeds ε_g with probability at most δ, and the tail split gives (ε_g, δ)-DP. The $|\Lambda|$ inside the logarithm is the price of choosing the order late.)*

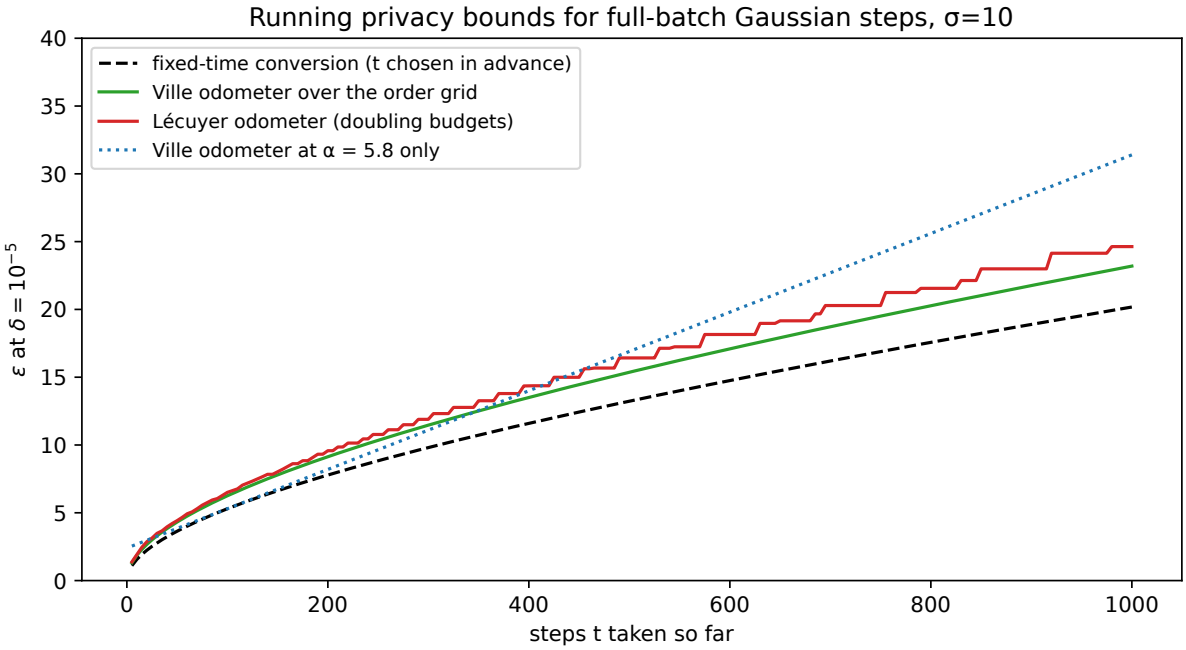


Figure 34.4.: Running privacy bounds at $\delta = 10^{-5}$ for full-batch Gaussian steps with $\sigma = 10$: Mironov's conversion at a time fixed in advance (dashed, not a valid odometer), the Ville odometer over the part's 154 orders, Lécuyer's odometer with doubling budgets, and a Ville odometer at the single order 5.8.

34.7. Individual privacy accounting in DP-SGD

34.7.1. Context

A step of DP-SGD computes each record's gradient, **clips** it (scales it down, if needed, so that its norm is at most C), adds the clipped gradients, and adds Gaussian noise before releasing the sum. The usual analysis charges every record the worst case at every step, as if its clipped gradient always had norm C. Most records do not: once the model fits them, their gradients shrink. Feldman & Zrnic (2021) observed that a record's actual privacy cost is a random, predictable quantity, so the Rényi filter applies record by record. You can track each record's own spend and drop it from the gradient sum once its budget is used, while every record keeps the same guarantee. This section does the full-batch case, where every record is used at every step and every number is exact.

34.7.2. Statement

A record's privacy cost at a step depends on how far it moves the released sum, which is the norm of its clipped gradient. Measured at order α, that cost is the Gaussian RDP for that shift, at most the worst-case cost when the norm is C.

Individual RDP of a Gaussian step (Feldman & Zrnic 2021, Section 2). At step t, the mechanism releases $\sum_i \bar{g}_{i,t} + \mathcal{N}(0, \sigma^2 C^2 I)$, where $\bar{g}_{i,t}$ is record i's gradient at the current model,

clipped to norm C. Removing record i moves the mean by $\bar{g}_{i,t}$, so record i's spend at order α is

$$\varepsilon_{i,t}(\alpha) = \frac{\alpha\|\bar{g}_{i,t}\|^2}{2\sigma^2 C^2} \leq \frac{\alpha}{2\sigma^2}. \tag{34.10}$$

Individual filter (Feldman & Zrnic 2021, Section 4.1). Fix α and B. At step t, record i takes part only if $\sum_{j\leq t} \varepsilon_{i,j}(\alpha) \leq B$, counting only the steps it took part in. Then the whole run is (α, B)-RDP.

34.7.3. Derivation

The plan is to compute a record's spend exactly, check that it is predictable, and then apply the Rényi filter separately to each pair of neighbours, which differ in a single record.

1. **The individual spend is a Gaussian Rényi divergence.** With record i present the output is $\mathcal{N}(m + \bar{g}_{i,t}, \sigma^2 C^2 I)$, without it $\mathcal{N}(m, \sigma^2 C^2 I)$, where m is the sum of the other records' clipped gradients. Rotate so $\bar{g}_{i,t}$ lies along one axis; the noise is the same in every direction, so the other coordinates have the same distribution in both cases and contribute nothing. The remaining one-dimensional pair is two normals with standard deviation σC whose means differ by $\|\bar{g}_{i,t}\|$, a shift of $\|\bar{g}_{i,t}\|/(\sigma C)$ in units of the noise. Two normals with shift μ in units of the noise have $D_\alpha = \alpha\mu^2/2$ (Equation 3.15), in both directions, which gives $\alpha\|\bar{g}_{i,t}\|^2/(2\sigma^2 C^2)$. Clipping makes $\|\bar{g}_{i,t}\| \leq C$, which gives the bound $\alpha/(2\sigma^2)$.
2. **It is predictable and does not depend on the other records.** $\bar{g}_{i,t}$ is a function of record i and the current model, and the model is a function of the earlier releases $y_{<t}$. So $\varepsilon_{i,t}(\alpha)$ is the RDP of step t for the pair (any dataset containing record i, the same dataset without it), conditionally on $y_{<t}$.
3. **Dropping a record costs it nothing.** Whether record i takes part at step t depends only on its own spends so far, which are functions of $y_{<t}$ and record i. If it is dropped, the outputs with and without it have the same law at that step, and its spend for the step is 0.
4. **Apply the Rényi filter per record.** For the pair (with i, without i), record i's spends are predictable and their sum never exceeds B, so by Equation 34.6 the transcript is (α, B)-RDP for this pair. Every pair of neighbours differs in one record, so the run is (α, B)-RDP.

34.7.4. Interpretation

The filter gives the same guarantee with more training. Every record ends with (α, B)-RDP, as in a worst-case run of $B/(\alpha/(2\sigma^2))$ steps. But records whose gradients shrink spend less per step, so they keep taking part long after the worst-case run would have stopped.

The guarantee is at one order. The filter certifies one order and one budget, (α, B), for every record; converted at that order it gives the run's (ε, δ). A per-record ε computed from the realized spend, at that order or with the order optimized afterwards, is an estimate, not a guarantee, for the reasons of Section 34.3: it is a fixed-time bound evaluated at a random quantity.

Individual ε is itself sensitive. A record's spend is a function of its gradient norms, so a small spend says "this record is well fitted". Feldman & Zrnic note that it can be shown to the record's owner without hurting anyone else, because it depends on the other records only through the private outputs, but publishing all of them leaks.

Without a filter, per-record numbers are ex post estimates. In an ordinary run of fixed length, a record's realized spend $\sum_t \varepsilon_{i,t}(\alpha)$ is random, and Equation 34.6 needs a bound on every path. Yu et al. (2023) study these ex post, output-specific guarantees for DP-SGD; Koskela, Tobaben & Honkela (2023) extend individual filtering from RDP to Gaussian differential privacy, which describes a mechanism by how hard it is to tell its two output distributions apart with a hypothesis test, for tighter accounting.

There is a fairness cost. Records that the model fits poorly, often from under-represented groups, keep large gradients, spend their budget first and are dropped first, so the model sees even less of them. Yu et al. (2023) find that the classes a model serves worst also get the weakest individual guarantees. This is one mechanism behind the disparate impact of DP training on accuracy, discussed in a later part.

The argument extends to subsampling. For Poisson-subsampled DP-SGD, where each record joins each step's batch independently with probability q, the individual spend of a record with clipped norm $\|\bar{g}_{i,t}\|$ is the RDP of the subsampled Gaussian with noise multiplier $\sigma C/\|\bar{g}_{i,t}\|$ (Amplification by subsampling), since the record's shift is $\|\bar{g}_{i,t}\|$ against noise of standard deviation σC. The filter argument is unchanged, because that norm, and so the spend, is still predictable. Feldman & Zrnic (2021, Section 5) note that the gain then needs every record's gradient at every step, sampled or not; Koskela, Tobaben & Honkela (2023) carry out individual accounting for subsampled DP-SGD.

34.7.5. Example

A simulation with an explicit model of gradient norms. There are 10,000 records, clip $C = 1$, $\sigma = 10$ and full-batch steps. Two kinds of record differ in whether the model learns them. **Typical records** (90%) have gradient norms $\|g_{i,t}\| = s_i e^{-t/40} e^{0.25 Z_{i,t}}$, with a per-record scale $s_i \sim U(0.3, 2)$ (uniform between 0.3 and 2) and $Z_{i,t}$ standard normal noise. The factor $e^{-t/40}$ makes them shrink as training proceeds, as the model fits them. **Hard records** (10%; 962 in this draw) have $\|g_{i,t}\| = s_i e^{0.25 Z_{i,t}}$ with $s_i \sim U(0.7, 1.5)$, with no shrinking factor: they are never fitted.

Gradients are then clipped at $C = 1$. The worst-case run has $k_0 = 100$ steps. Each Gaussian step with noise multiplier σ is $1/(2\sigma^2)$-zCDP, so 100 steps are $\rho = 100/(2 \cdot 10^2) = 0.500$-zCDP, and converting gives $\varepsilon = 5.299$ at $\delta = 10^{-5}$, best at order $\alpha^* = 5.8$, where the spend is $B = 2.900$. Every per-step spend is at most $\alpha^*/(2\sigma^2) = 0.0290$.

Individual ε after the same 100 steps. At the fixed order $\alpha^* = 5.8$, record i's ε is Mironov's conversion of its own spend, $\sum_t \varepsilon_{i,t}(5.8) + \ln(10^5)/4.8$. The second term does not depend on the record, so an individual ε can never fall below the floor $\ln(10^5)/4.8 = 2.399$, even for a record that spends nothing. Table 34.7 and Figure 34.5 show the distribution.

Table 34.7.: Individual ε at $\delta = 10^{-5}$ after 100 full-batch steps with $\sigma = 10$, at the fixed order $\alpha^* = 5.8$ and, in the last column, with the order optimized per record over the part's grid. The worst case is 5.299.

Records	10th percentile	Median	90th percentile	Median, order chosen per record
All	2.56	3.25	3.81	2.75
Typical	2.55	3.17	3.67	2.61
Hard	4.23	5.02	5.26	5.01

The typical median is 3.17, against 5.299 for the worst case; 16.3% of all records are below half the worst case. Hard records sit just under the worst case, 5.02 at the median, because their gradients are usually near or above the clip. These realized values are ex post estimates (Yu et al. 2023): the filter certifies only its budget, $\varepsilon = 5.299$, for every record. Optimizing the order per record lowers the typical median further, to 2.61, because a small spend is best converted at a larger order, where the floor term is smaller.

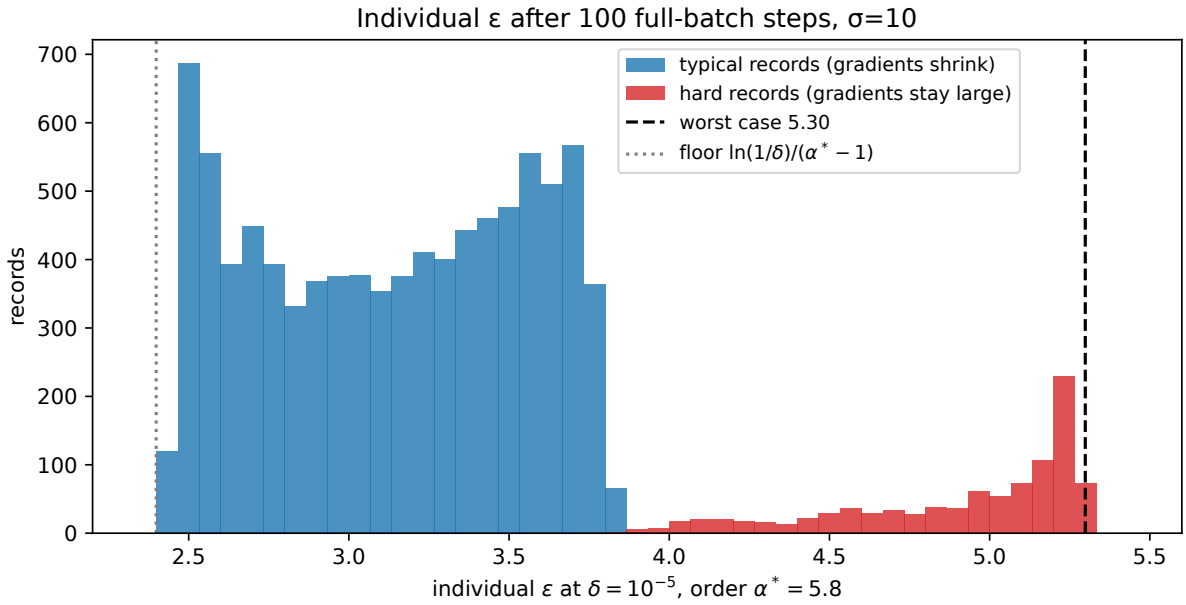


Figure 34.5.: Individual ε at $\delta = 10^{-5}$ and order $\alpha^* = 5.8$ after 100 full-batch steps with $\sigma = 10$. Typical records (blue) spend far less than the worst case (dashed); hard records (red) pile up just below it. The dotted line is the floor $\ln(1/\delta)/(\alpha^* - 1)$ that a fixed order cannot go below.

The individual filter over 300 steps. With budget $B = 2.900$ at $\alpha^* = 5.8$, everyone takes part in the first 100 steps, since no step costs more than 0.0290 and $100 \times 0.0290 = 2.900$. After that, hard records run out: the first is excluded at step 101, the median one at step 111, and by step 200 none takes part (0.1% are back in at step 300, on steps where their gradient happened to be small enough to fit in what was left of the budget). No typical record is ever excluded; at step 300 their median spend is 0.778, about a quarter of the budget. Every record, typical or hard, has the same $(5.8, 2.900)$-RDP guarantee, the same as the 100-step worst-case run (Figure 34.6).

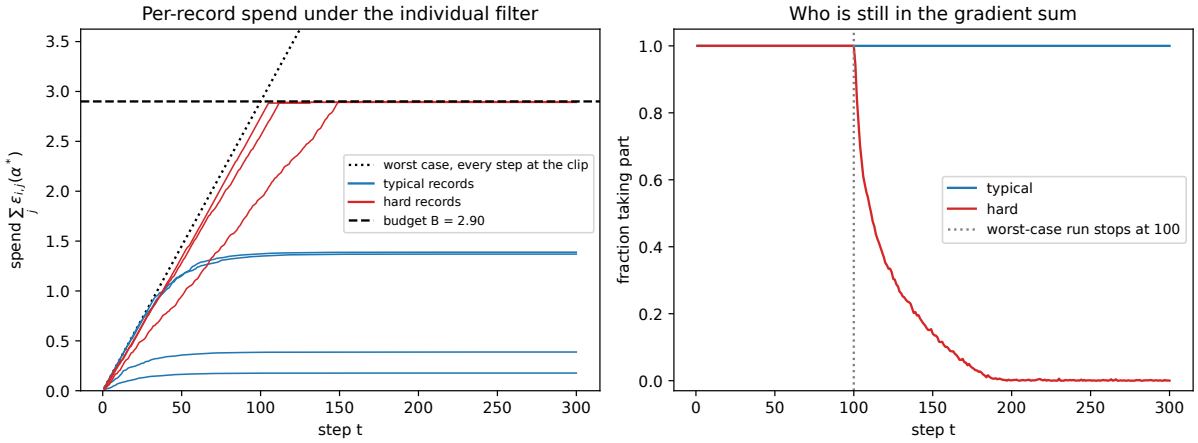


Figure 34.6.: Left: cumulative spend at order 5.8 under the individual filter, for four typical records (blue) and three hard records (red), against the worst case (dotted) and the budget $B = 2.90$ (dashed). Hard records reach the budget and stop; typical records level off far below it. Right: the fraction of each group that takes part in the gradient sum at each step.

34.7.6. Check yourself

- Why must a record's spend be computed from that record and the past outputs alone, and not from the whole dataset? *(If record j's spend depended on the whole dataset, removing record i would change the spends of other records and so which of them take part. The outputs with and without i would then differ even at steps where i itself is dropped, and step 3 fails. Feldman & Zrnic make this point about the per-instance definition, relative to the fixed dataset. For a clipped-gradient sum the issue does not arise, since $\bar{g}_{i,t}$ depends only on record i and the model.)*
- If every gradient is clipped, is the individual filter ever worse than the worst-case analysis? *(No. Each per-step spend is at most $\alpha/(2\sigma^2)$, so every record survives at least $B/(\alpha/(2\sigma^2))$ steps, the length of the worst-case run, and the guarantee is the same.)*

34.8. Code: filters, odometers and per-record spend

The script code/adaptive_composition.py computes every number in this chapter and draws every figure. The plug-in failure is computed exactly by dynamic programming over the randomized-response walk. The filters are short functions of the realized parameters; the Rogers et al. budget is found by bisection. The Rényi filter is checked on the three-step table and by Monte Carlo on adaptive Gaussian steps; the Ville odometer's tightness is checked against the Brownian line-crossing formula. The individual-accounting simulation uses the explicit gradient-norm model above. From the repository root:

```
uv run python code/part04-privacy-accounting/adaptive_composition.py
```

Output (seed 0), summarized in Table 34.8:

Table 34.8.: The script's output with the default seed.

Experiment	Result
Plug-in advanced composition, randomized response $\varepsilon = 0.1$	Crossing by $n = 10^4$: 0.1424 at $\delta = 0.05$, 0.0318 at $\delta = 0.01$, 3.69×10^{-5} at $\delta = 10^{-5}$; at a single fixed $t \le 1000$ at most 0.0110, 0.0016 and 8.18×10^{-7}
Budget for $k = 1000$ steps of 0.01, $\delta = 10^{-6}$	Basic 10.000; advanced (fixed k) 1.763; Rogers filter 2.570; Whitehouse filter 1.712
Steps allowed at $\varepsilon_g = 1$, $\varepsilon_0 = 0.01$	Basic 100; advanced 337; Rogers 154; Whitehouse 349
Three-step randomized response, $\alpha = 2$	$\mathbb{E}[N_3] = 1.0000$; joint $D_2 = 1.8487$ = budget; fixed-curve bound 2.5419
Adaptive Gaussian filter, $\alpha = 2$, $B = 1$, 10^6 paths	With top-up $D_2 = 1.0004$; without, $D_2 = 0.9305$
Odometers, $\sigma = 10$, $t = 100$, $\delta = 10^{-5}$	Fixed time 5.299; Ville 6.253; Lécuyer 6.507
Odometers, running example ($q = 0.01$, $\sigma = 1$, 10^4 steps)	Fixed time 7.43; Ville 8.82; Lécuyer 9.08
Ville crossing, Gaussian steps, $\delta = 0.05$	Monte Carlo 0.0461; Brownian limit 0.0500
Individual ε, 100 steps, $\sigma = 10$	Worst case 5.299; median typical 3.17, hard 5.02
Individual filter, 300 steps	Hard records excluded from step 101 (median 111); typical never; typical median spend 0.778 of 2.900

tests/test_adaptive_composition.py checks that the output still matches.

Four things stand out. First, stopping is the whole difficulty: the same bound that is valid at every fixed time fails about three times as often as claimed (2.85 times at $\delta = 0.05$, 3.18 at 0.01) once the analyst may choose when to stop. Second, filters cost nothing in RDP: the Rényi filter's bound is attained in the Monte Carlo check, and the derived DP filter matches fixed-parameter zCDP composition. Third, odometers cost a union bound, about 15–19% in ε here, which is the price of choosing the order after the fact. Fourth, per-record spend varies widely: most records spend far less than the worst case, while the minority that the model never fits spend all of it and are the first to be dropped.

34.9. References

- Rogers, Roth, Ullman & Vadhan, "Privacy Odometers and Filters: Pay-as-you-Go Composition" (NeurIPS 2016; arXiv:1605.08294). Defines filters and odometers, proves basic composition works for both (Theorem 3.4), gives the first advanced-composition filter (Theorem 5.1), the lower bound separating odometers from filters (Theorem 6.1) and an odometer (Theorem 6.5). Read Sections 1 and 3 for the model.

- Whitehouse, Ramdas, Rogers & Wu, "Fully-Adaptive Composition in Differential Privacy" (ICML 2023; arXiv:2203.05481). Filters that match advanced composition including constants (Theorems 1–2, through approximate zCDP), and odometers from time-uniform martingale concentration (Theorem 3). The place to look for steps with $\delta_t > 0$.
- Feldman & Zrnic, "Individual Privacy Accounting via a Rényi Filter" (NeurIPS 2021; arXiv:2008.11193). The fully adaptive RDP composition theorem and the Rényi filter (Sections 3–4), individual RDP and the individual filter, and private gradient descent with filtering (Section 5).
- Lécuyer, "Practical Privacy Filters and Odometers with Rényi Differential Privacy and Applications to Differentially Private Deep Learning" (2021, arXiv:2103.01379). An independent proof of the Rényi filter, the filter over a set of orders, and the RDP odometer with doubling budgets; experiments with adaptive training.
- Koskela, Tobaben & Honkela, "Individual Privacy Accounting with Gaussian Differential Privacy" (ICLR 2023). Individual filtering with Gaussian differential privacy and privacy loss distributions, tighter than RDP for the Gaussian mechanism.
- Yu, Kamath, Kulkarni, Liu, Yin & Zhang, "Individual Privacy Accounting for Differentially Private Stochastic Gradient Descent" (TMLR 2023). Output-specific per-example guarantees for DP-SGD, and the link between a record's training loss, its privacy and group fairness.
- Dwork, Rothblum & Vadhan, "Boosting and Differential Privacy" (FOCS 2010). Advanced composition with parameters fixed in advance, the theorem this chapter makes adaptive.
- Howard, Ramdas, McAuliffe & Sekhon, "Time-uniform Chernoff Bounds via Nonnegative Supermartingales" (Probability Surveys 2020). Ville's inequality and the time-uniform bounds behind the Whitehouse et al. odometers.
- Williams, *Probability with Martingales* (1991), chs. 10–11. Optional stopping (ch. 10) and martingale convergence (ch. 11), with proofs.
- Durrett, *Probability: Theory and Examples* (5th ed., Cambridge University Press, 2019), the chapter on Brownian motion. The law of the iterated logarithm used in Section 34.3, and Brownian hitting probabilities.

34.10. Cheat sheet

Steps have predictable parameters ε_t (pure DP) or $\varepsilon_t(\alpha)$ (RDP at a fixed order); $V = \sum_t \varepsilon_t^2$; $S_t(\alpha) = \sum_{j \le t} \varepsilon_j(\alpha)$. Table 34.9 collects the results.

Table 34.9.: Filters, odometers and individual accounting.

Object	Rule or bound	Guarantee	Source
Basic filter / odometer	$\sum_t \varepsilon_t \le \varepsilon_g$; $U_t = \sum_{j \le t} \varepsilon_j$	$(\varepsilon_g, 0)$-DP; holds on every path	Rogers et al., Thm 3.4
Rogers filter	Equation 34.4 $\le \varepsilon_g$	(ε_g, δ)-DP, $\delta < 1/e$	Rogers et al., Thm 5.1
Whitehouse filter	$\sqrt{2\ln(1/\delta)V} + V/2 \le \varepsilon_g$, i.e. $V/2 \le \rho^*$	(ε_g, δ)-DP	Whitehouse et al., Thm 2
Rényi filter	$S_T(\alpha) \le B$ on every path	(α, B)-RDP	Feldman & Zrnic; Lécuyer
Ville odometer	$U_t = \min_{\alpha \in \Lambda}[S_t(\alpha) + \ln(\lvert\Lambda\rvert/\delta)/(\alpha - 1)]$	$\Pr[\exists\, t : L_{1:t} > U_t] \le \delta$	Fact 3 + union bound

Object	Rule or bound	Guarantee	Source
Lécuyer odometer	Equation 34.9	same	Lécuyer, Section 4
Individual spend, Gaussian sum	$\varepsilon_{i,t}(\alpha) = \alpha\|\bar{g}_{i,t}\|^2/(2\sigma^2 C^2)$	per-record filter gives (α, B)-RDP	Feldman & Zrnic

Rules of thumb

- Never report a fixed-time bound evaluated at a data-dependent time. Use a filter if you have a budget, an odometer if you do not.
- In RDP, a filter at a fixed order costs nothing. Fix the order before the run; track a few orders if you want the best one, and stop when any is exhausted.
- An odometer costs a union over the orders you might choose: replace $\ln(1/\delta)$ by $\ln(|\Lambda|/\delta)$, and keep the grid small.
- For pure-DP steps, the Whitehouse filter is advanced composition with the mean term $\varepsilon_t^2/2$, and it is valid for adaptive parameters.
- Individual accounting gives every record the same guarantee and lets well-fitted records keep training; the records it drops first are the ones the model already serves worst.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

35. Choosing an accountant

35.1. Why this matters

You have trained a model with differential privacy (DP). The training loop was stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD. At each step every record joins the batch independently with probability q (Poisson sampling), each record's gradient is clipped to norm at most C, and Gaussian noise of standard deviation σC is added to the sum, so σ, the **noise multiplier**, says how large the noise is compared with one record's largest possible contribution. Your run used $q = 0.01$, $\sigma = 1$ and $k = 10{,}000$ steps. You must now write one sentence in a paper or a model card: "the model is $(\varepsilon, 10^{-5})$-DP". That sentence means: for any two datasets that differ in one record, and for any event about what training released, the probability of the event on one dataset is at most e^{ε} times its probability on the other, plus $\delta = 10^{-5}$. A small ε means one record barely changes what anyone can see; δ is a small allowance for that promise to fail.

The run is fixed, so its true ε is one number. But nobody can compute it exactly, and a procedure that bounds or estimates it is called an **accountant**. The chapters of this part give you six of them. Composing per-step guarantees gives 11296.93. Rényi differential privacy (RDP) gives 7.4287 or 6.7127, depending on the conversion. The Gaussian differential privacy (GDP) central limit theorem (CLT) gives 6.0071. A privacy loss distribution (PLD) accountant gives an interval, $[6.1377, 6.2377]$. These numbers describe the same training run. Which one do you report, and what else must you say so that someone else can check it?

35.1.1. The one idea: bounds and approximations

The numbers differ because the accountants make two different kinds of statement, and you can only report one kind.

An **upper bound** comes with a proof that holds for every pair of neighbouring datasets. It may be loose, so the true ε may be smaller, but the run really is (ε, δ)-DP with the ε it reports. Per-step composition, RDP with any of its conversions, and the pessimistic PLD are upper bounds. An **approximation** has no such proof. It may land close to the truth, but nothing stops it from landing below, and then it reports a number for which the run is *not* (ε, δ)-DP. The GDP central-limit approximation is of this kind, and on the running example it does land below: it falls under the PLD *lower* bound at every k.

The optimistic PLD makes a third kind of statement, a certified **lower bound**: the true ε is at least this large. It is not a privacy guarantee, since it says the run is no better than this, but it tells you how far any valid upper bound can fall. The true ε sits between the pessimistic and the optimistic PLD.

So the rule is short. **Report the tightest upper bound whose assumptions hold for the code you ran, and report enough that someone else can recompute it.** Each section of this chapter unpacks one clause of that rule. Table 35.1 lists the sections.

Table 35.1.: The sections of this chapter and where each one is used.

Section	Question	Where you use it
Section 35.2	How do the accountants compare on one DP-SGD run, and which are bounds?	Picking an accountant
Section 35.3	How much noise does each accountant ask for at a target ε?	Calibrating σ before training
Section 35.4	Which accountant is tight for which mechanism?	Accounting for anything other than DP-SGD
Section 35.5	Do the accountant's assumptions match the training code?	Code review of a private training pipeline
Section 35.6	What must a privacy statement contain?	Papers, model cards, audits
Section 35.7	How do you check an accountant?	Testing your own, or someone else's, accountant

This chapter adds little new theory: the ordering of the bounds and the monotonicity behind the inverse problem are the only new derivations. Every accountant is the one derived earlier in the part, and the script imports it from that chapter's code rather than re-implementing it. Each section below restates what an accountant does where it uses it, so you can read this chapter on its own. The running example and the order grid are those of the whole part.

35.2. Every accountant on one run

35.2.1. Context

This is the question the part has built up to: for DP-SGD with $q = 0.01$, $\sigma = 1$ and up to $k = 10{,}000$ steps, compute ε at $\delta = 10^{-5}$ with every accountant, and see what choosing a loose one costs. Amplification by subsampling and Numerical accounting each computed some of these numbers. This section puts them in one table, adds the optimal conversion of an RDP curve, and says for each accountant what it assumes and whether it is a bound.

35.2.2. Statement

All accountants start from the same object: what one step of DP-SGD reveals about one record. Look at the noisy gradient sum along the direction of that record's clipped gradient, and measure

it in units of the clipping norm C. Without the record, the output is pure noise, $\mathcal{N}(0, \sigma^2)$. With the record, the record is in the batch with probability q, and then its gradient shifts the output by 1; otherwise it is absent and the output is the same noise. So the output is the mixture $(1-q)\mathcal{N}(0, \sigma^2) + q\mathcal{N}(1, \sigma^2)$. The accountants differ in what they keep of this pair and how they combine k copies of it. Here they are, each with where it was derived:

1. **Per step, then composition** (Amplification by subsampling, Section 31.7). First find an $(\varepsilon_0, \delta_0)$ guarantee for one Gaussian step on its own. Then use subsampling: because the record is missing from the batch with probability $1-q$, the factor e^{ε_0} by which it can change a probability is diluted to $1 + q(e^{\varepsilon_0} - 1)$, which is Equation 31.5, $\varepsilon_1 = \ln(1 + q(e^{\varepsilon_0} - 1))$. Finally compose k steps, either by basic composition, where the ε's simply add to $k\varepsilon_1$, or by advanced composition, which grows like $\sqrt{k}$ at the price of an extra δ'.
2. **RDP, Mironov's conversion** (Rényi DP). RDP describes one step by a curve $\varepsilon(\alpha)$, the Rényi divergence of order $\alpha > 1$ between the two output distributions; for the subsampled Gaussian it is Equation 31.9. Its appeal is that curves of composed steps simply add, so k steps have curve $k\,\varepsilon(\alpha)$. To get back to (ε, δ), Mironov's conversion uses the fact that δ is at most the chance that the privacy loss L (defined in item 6) exceeds ε. The curve fixes the moment $\mathbb{E}[e^{(\alpha-1)L}] = e^{(\alpha-1)k\varepsilon(\alpha)}$, so Markov's inequality gives $\Pr[L > \varepsilon] \le e^{(\alpha-1)(k\varepsilon(\alpha)-\varepsilon)}$. Setting the right side equal to δ and solving gives, at each order, $\varepsilon = k\,\varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha-1}$, and the conversion takes the best order: $\varepsilon = \min_\alpha \left[k\,\varepsilon(\alpha) + \frac{\ln(1/\delta)}{\alpha-1}\right]$.
3. **RDP, improved conversion** (From Rényi DP to (ε, δ) and to tests, Equation 28.7). The same curve, converted with a sharper analysis of the same tail (Balle et al. 2020) that saves a fixed amount at each order: $\varepsilon = \min_\alpha \left[k\,\varepsilon(\alpha) + \ln\frac{\alpha-1}{\alpha} - \frac{\ln\delta + \ln\alpha}{\alpha-1}\right]$.
4. **RDP, optimal conversion** (Section 28.5). The best any conversion of the curve can do: the smallest ε that every pair of distributions with the curve $k\,\varepsilon(\alpha)$ satisfies at δ.
5. **GDP central-limit approximation** (f-DP and Gaussian DP, Equation 30.17). A mechanism is **μ-GDP** if telling its two output distributions apart is no easier than telling $\mathcal{N}(0,1)$ from $\mathcal{N}(\mu,1)$. The privacy loss of k steps is a sum of k small losses, so by the central limit theorem it is close to Gaussian, and the approximation treats the run as μ-GDP with $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$. It then reads ε off the exact (ε, δ) curve of a μ-GDP mechanism, Equation 30.10.
6. **PLD, pessimistic and optimistic** (Numerical accounting, Equation 32.6). The **privacy loss** at an output y is the log of how much more likely y is on one of the two neighbouring datasets than on the other, and its distribution over outputs is the PLD. Each order of the pair gives its own PLD: the **remove** direction puts the dataset with the record first, so the loss measures how much more likely y is with the record than without it, and the **add** direction puts it second. Under composition the losses add, so the PLD of k steps is the k-fold convolution of one step's PLD, which a computer does quickly with the fast Fourier transform (FFT). The convolution needs the loss on a grid, here of width $h = 10^{-5}$. Rounding every loss up gives the pessimistic PLD, rounding it down the optimistic one. Solve each for ε, and take the larger of the add and remove directions, since the guarantee must hold whichever of the two datasets has the record.

Table 35.2 gives every accountant's ε on the running example.

Table 35.2.: DP-SGD with $q = 0.01$, $\sigma = 1$: ε at $\delta = 10^{-5}$ after k steps under every accountant of this part. Basic and advanced are per-step amplification followed by composition; the PLD columns use the grid $h = 10^{-5}$.

k	Basic	Advanced	RDP, Mironov	RDP, improved	RDP, optimal	GDP CLT	PLD upper	PLD lower
1	0.20	1.13	1.3175	0.9555	0.4899	0.0368	0.1995	0.1994
10	3.64	7.88	1.4145	1.0353	0.7951	0.1304	0.3800	0.3799
100	58.01	78.10	1.6118	1.2141	1.1538	0.4575	0.7185	0.7175
1,000	839.39	1258.64	2.5380	2.1014	2.1009	1.6177	1.8332	1.8232
10,000	11296.93	24379.55	7.4287	6.7127	6.7127	6.0071	6.2377	6.1377

35.2.3. Derivation: what each accountant assumes, and why it is a bound

Every row of Table 35.2 assumes the same model of the training run: Poisson sampling at rate q, add/remove neighbours (the two datasets differ by one record being present or absent), per-example clipping to norm C and Gaussian noise of standard deviation σC on the summed gradients, and every intermediate noisy gradient released. Under that model, one step is the Poisson-subsampled Gaussian pair of the statement, $(1-q)\mathcal{N}(0,\sigma^2)+q\mathcal{N}(1,\sigma^2)$ against $\mathcal{N}(0,\sigma^2)$, in units of C (Amplification by subsampling). The accountants differ only in what they keep of this pair, and each step below says whether what is kept still gives a proof.

1. **Per step, then composition: an upper bound.** One step is $(\varepsilon_1, q\delta_0)$-DP by the amplification theorem (Section 31.3), and basic and advanced composition hold even when each step is chosen after seeing the earlier outputs, which is how SGD works (Composition). So the result is a proof. It is very loose because each step's whole privacy loss distribution is replaced by just two numbers, ε_1 and $q\delta_0$, and composition has to assume the worst distribution consistent with them at every step.
2. **RDP with Mironov's or the improved conversion: an upper bound.** Three facts make this a proof. The subsampled curve Equation 31.9 is the exact Rényi divergence of the pair, not an estimate. RDP curves add under adaptive composition (Equation 27.10), because the Rényi divergence is the log of a moment of the likelihood ratio, and the moment of a product of independent factors is the product of their moments. And both conversions are proved for every pair with the curve (Section 28.3, Section 28.4). The accountant evaluates the curve only on a grid of orders, and minimizing over fewer orders can only give a larger ε, so the grid costs tightness, never validity.
3. **RDP with the optimal conversion: an upper bound.** It computes the largest $\delta(\varepsilon)$ over all pairs of distributions with the given RDP curve. The real pair has that curve, so it is in the set, and its $\delta(\varepsilon)$ is no larger than the maximum (Section 28.5). The same argument shows it is the tightest bound any accountant that sees only the curve can give: some pair in the set attains the maximum, and such an accountant cannot tell that pair from the real one.
4. **The GDP central-limit approximation: not a bound.** Equation 30.17 comes from two approximations (Section 30.8). It is a limit as $k \to \infty$ with $q\sqrt{k}$ held fixed, and it uses a second-order expansion of the per-step loss, keeping its mean and variance and dropping

the rest. Neither step comes with an error bound in the tail of the loss distribution, and $\delta = 10^{-5}$ is read exactly there: it asks about the chance of a rare, large loss, which is where a Gaussian approximation is least reliable.

5. **The pessimistic PLD: an upper bound; the optimistic PLD: a lower bound.** Rounding every loss up describes a pair that leaks at least as much as the real one (a dominating pair), and composing two dominating pairs gives a dominating pair, so the rounding error keeps its sign through all k convolutions. Rounding down does the same in the other direction. So $\delta^{\downarrow}(\varepsilon) \le \delta(\varepsilon) \le \delta^{\uparrow}(\varepsilon)$ for every ε (Equation 32.6). The upper bound holds under adaptive composition of the fixed per-step pair. How far apart are the two? Each step's loss moves by less than one grid width h between rounding down and rounding up, so after k steps the two composed losses differ by at most kh, and so do the ε's they give, as long as no tail is truncated (Section 32.5). Here $kh = 10{,}000 \times 10^{-5} = 0.1$.

Steps 2, 3 and 5 put the bounds in two orders, which together read

$$\varepsilon_{\text{RDP, Mironov}} \;\ge\; \varepsilon_{\text{RDP, improved}} \;\ge\; \varepsilon_{\text{RDP, optimal}} \;\ge\; \varepsilon_{\text{true}}, \qquad \varepsilon^{\uparrow}_{\text{PLD}} \;\ge\; \varepsilon_{\text{true}} \;\ge\; \varepsilon^{\downarrow}_{\text{PLD}}. \tag{35.1}$$

The second chain is the sandwich of step 5. In the first chain, the last inequality is step 3. Mironov's conversion is above the improved one because at every order the improved one is smaller by $\Delta(\alpha) = \frac{\ln \alpha}{\alpha - 1} + \ln \frac{\alpha}{\alpha - 1} > 0$ (Equation 28.10); both terms are positive for $\alpha > 1$, and if one expression is above another at every order, its minimum over the grid is above too. The improved conversion is above the optimal one because the optimal one is, by definition, the smallest ε valid for every pair with the curve, and the improved one is one such valid ε.

The theory does not order the two chains against each other. You might expect the RDP bounds to always sit above the PLD upper bound, because RDP throws away information. But the pessimistic PLD describes a rounded-up pair, not the real one, and its ε may exceed the truth by up to kh. A bound from the curve alone that came very close to the truth could therefore fall below it. Without truncation, all that follows is $\varepsilon_{\text{RDP, optimal}} \ge \varepsilon_{\text{true}} \ge \varepsilon^{\uparrow}_{\text{PLD}} - kh$. On the running example the script checks both chains at all nine values of k from 1 to 10,000. It also finds RDP optimal $\ge$ PLD upper at every one of them, because the curve slack (at least 0.2677, from Table 35.4 below) is far larger than kh (at most 0.1000). That is an observation about this example, not a theorem. The script also checks that the CLT value is below the PLD lower bound at every k.

35.2.4. Interpretation

Figure 35.1 plots the table on a log scale, with each accountant's ratio to the PLD upper bound beside it. Table 35.3 lists the ratios. Since the PLD upper bound is within kh of the truth, a ratio well above 1 measures how much an accountant overstates ε, and a ratio below the PLD lower column means the number is not a valid guarantee.

Table 35.3.: Each accountant's ε divided by the PLD upper bound, from Table 35.2.

k	Basic	Advanced	RDP, Mironov	RDP, improved	RDP, optimal	GDP CLT	PLD lower
1	1.00	5.66	6.61	4.79	2.46	0.18	1.00
10	9.58	20.74	3.72	2.72	2.09	0.34	1.00

k	Basic	Advanced	RDP, Mironov	RDP, improved	RDP, optimal	GDP CLT	PLD lower
100	80.73	108.69	2.24	1.69	1.61	0.64	1.00
1,000	457.88	686.57	1.38	1.15	1.15	0.88	0.99
10,000	1811.07	3908.41	1.19	1.08	1.08	0.96	0.98

Per-step composition is useless beyond a handful of steps. It is the right answer only at $k = 1$, where there is nothing to compose and basic composition is just the amplified one-step guarantee. At $k = 10{,}000$ it is off by a factor of over a thousand, because adding ε's assumes every step leaks its worst case, while in reality the losses of different steps mostly cancel. Advanced composition is worse than basic here, for the reason given in Section 31.7. Its bound is $\varepsilon_1\sqrt{2k\ln(1/\delta')}+k\varepsilon_1(e^{\varepsilon_1}-1)$: a $\sqrt{k}$ term with a large constant (at $\delta' = 10^{-5}$, $\sqrt{2\ln(10^5)} \approx 4.80$), plus a term that still grows like k. The $\sqrt{k}$ only wins when ε_1 is so small that the second term is negligible, and the amplified per-step ε_1 is too large for that.

RDP is a safe bound, and its slack has two sources. RDP with the improved conversion exceeds the PLD upper bound for two separate reasons, and Table 35.4 splits the excess accordingly. The first is the conversion: turning the curve into one (ε, δ) pair with the improved formula loses against the best possible conversion of the same curve, and that loss is improved minus optimal. The second is the curve itself: even the best conversion of the curve is above the PLD, because the curve keeps only a family of moments of the privacy loss, not its whole distribution, and that loss is optimal minus PLD upper.

Conversion slack dominates short runs. At $k = 1$ the conversion loses 0.4656 of the 0.7561 excess. The reason is the term $\ln(1/\delta)/(\alpha-1)$ in the conversion (about $11.5/(\alpha-1)$ at $\delta = 10^{-5}$), which you pay however small the curve is. When the curve is tiny, as after one step, this fixed toll is most of the answer.

Curve slack dominates long runs. After many steps the curve $k\,\varepsilon(\alpha)$ is large, the best order moves up, and the fixed toll shrinks in comparison. From $k = 1000$ on, the conversion slack is negligible (0.0005 at $k = 1000$, zero to four decimals at $k = 10{,}000$), and almost all the slack is in the curve. That last 7.6% at $k = 10{,}000$ is what only a numerical accountant removes, because only it keeps the whole distribution.

The CLT under-reports here. It is below the certified lower bound at every k: at $k = 10{,}000$, 6.0071 against 6.1377. Its ratio climbs toward 1 as k grows, as a central limit theorem should, since a sum of more terms is closer to Gaussian. But at no k in the table is it a valid guarantee.

The PLD bracket is the truth, to within kh. At $k = 10{,}000$ the width is $6.2377 - 6.1377 = 0.1000$ on this grid, exactly kh. If that is too wide, refine h. Its only other error, truncating the far tails of the loss, is pushed in the safe direction on each side (mass is moved so that the upper bound only rises and the lower bound only falls), so h is the one knob to turn.

Table 35.4.: Where the RDP accountant (improved conversion) loses against the PLD upper bound. Conversion slack is improved minus optimal conversion; curve slack is optimal conversion minus PLD upper bound.

k	Conversion slack	Curve slack	Total	Excess over PLD upper
1	0.4656	0.2905	0.7561	379.1%
10	0.2402	0.4151	0.6554	172.5%
100	0.0603	0.4353	0.4956	69.0%
1,000	0.0005	0.2677	0.2681	14.6%
10,000	0.0000	0.4750	0.4750	7.6%

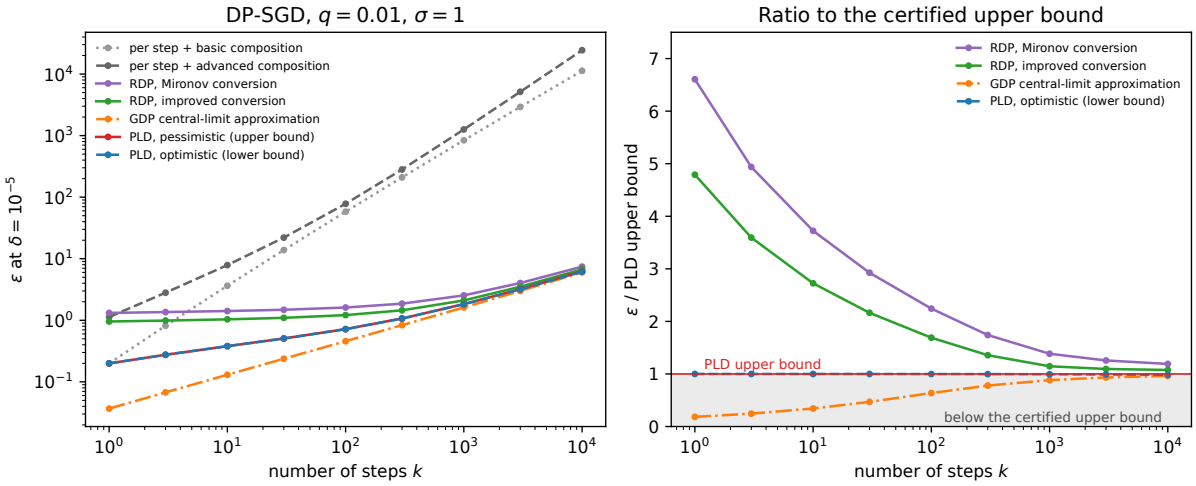


Figure 35.1.: Left: ε at $\delta = 10^{-5}$ against the number of steps k for DP-SGD with $q = 0.01$, $\sigma = 1$, under per-step basic and advanced composition, RDP with Mironov's and the improved conversion, the GDP central-limit approximation, and the pessimistic and optimistic PLD on the grid $h = 10^{-5}$ (the two PLD curves overlap). Right: RDP (both conversions), the CLT and the PLD lower bound divided by the PLD upper bound; per-step composition (ratios 1 to 3908) is off the scale. Ratios above 1 are valid but loose; the shaded region below 1 is where only the PLD lower bound may lie, and the CLT lies there at every k.

35.2.5. Example: the number you would have reported

Suppose you ran 1,000 steps. Depending on the accountant, your paper says $\varepsilon = 2.54$ (RDP with Mironov's conversion, as in older code), 2.10 (RDP with the improved conversion, as in Opacus's RDP accountant), 1.83 (a PLD accountant), or 1.62 (the CLT). The first three are true statements about the run, because each is an upper bound; the third is the most informative, because it is the smallest. The fourth is false: the optimistic PLD proves $\varepsilon \geq 1.8232$, so the true ε is larger than 1.62 and the run is not $(1.62, 10^{-5})$-DP.

35.2.6. Check yourself

- At $k = 1$, per-step basic composition gives 0.20 and RDP gives 0.9555. Does that make the RDP accountant wrong? *(No. Both are upper bounds, and the basic one happens to be tighter for a single step, because RDP's conversion pays* $\ln(1/\delta)/(\alpha - 1)$ *even when the curve is tiny. RDP wins as soon as there is something to compose.)*
- Why can the optimal conversion of an RDP curve never beat the pessimistic PLD by more than kh? *(It must cover every pair with the same RDP curve, and the real pair is one of them, so it is at least the true* ε*. The pessimistic PLD is at most* kh *above the true* ε*. So the optimal conversion can fall below the PLD upper bound, but by at most* kh *(without truncation). Here the curve slack is much larger than* kh*, and the script finds optimal* $\geq$ *PLD upper at every* k*.)*

35.3. The inverse problem: noise for a target ε

35.3.1. Context

Practitioners rarely compute ε after the fact. They fix a budget, say $\varepsilon = 3$ at $\delta = 10^{-5}$, fix the number of steps and the batch size, and ask how much noise they must add. The noise multiplier is then the knob that sets utility. The noise added to the gradient sum has standard deviation σC, and the sum is divided by the expected batch size qn for n records, so the noise in the averaged gradient has standard deviation $\sigma C/(qn)$. A loose accountant overstates ε at every σ, so it asks for a larger σ to meet the same budget: its slack turns directly into extra noise.

35.3.2. Statement

For each accountant A, write $\varepsilon_A(\sigma)$ for the ε it reports at δ for the run with noise multiplier σ. The noise it requires for a target ε^* is the smallest σ at which its reported ε meets the target:

$$\sigma_A(\varepsilon^*) = \min\{\sigma > 0 : \varepsilon_A(\sigma) \leq \varepsilon^*\}. \tag{35.2}$$

Table 35.5 gives it for $k = 10{,}000$, $q = 0.01$, $\delta = 10^{-5}$ and three targets.

Table 35.5.: The noise multiplier σ each accountant requires for a target ε at $\delta = 10^{-5}$, with $k = 10{,}000$ steps at $q = 0.01$. PLD on the grid $h = 10^{-5}$; the PLD lower column is the σ at which the optimistic bound reaches the target.

Target ε	Advanced	RDP, Mironov	RDP, improved	GDP CLT	PLD upper	PLD lower
1	24.682	4.974	4.126	3.796	3.988	3.654
3	10.505	1.854	1.662	1.549	1.584	1.547
8	5.711	0.962	0.917	0.868	0.885	0.880

35.3.3. Derivation

The plan is to show that more noise never raises any accountant's ε, so that Equation 35.2 asks for the point where a decreasing function crosses a level, and then to find that point numerically.

1. **Every bound decreases in σ.** The idea is that a noisier run can be produced from a less noisy one without touching the data, and that cannot reveal more. Fix $\sigma < \sigma'$. (a) *Post-processing:* the variances of independent Gaussians add, so adding independent $\mathcal{N}(0, \sigma'^2 - \sigma^2)$ noise to the output of the step with noise σ gives exactly the step with noise σ', in both runs and in every step. That map does not look at the data. (b) *Data processing:* the data-processing inequality (Divergences) says that passing two distributions through the same data-independent map can only make them harder to tell apart, so every divergence between them can only fall. That applies to the per-step and the composed pair, and to both the Rényi divergences and the hockey-stick divergence that defines $\delta(\varepsilon)$. Every accountant is built from those divergences, so its ε falls too. For the discretized PLD this holds only up to grid effects of order kh, since rounding can move a value by up to that much. The CLT's $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$ falls in σ directly, because e^{1/σ^2} does.
2. **So Equation 35.2 is a root of a decreasing function,** $\varepsilon_A(\sigma) - \varepsilon^*$, and any bracketing method finds it: start from a low σ where the function is positive and a high one where it is negative, and shrink the interval while keeping the sign change inside. The script uses false position with the Illinois modification. False position places each new guess where the straight line through the two ends crosses zero, rather than at the midpoint as bisection does. The Illinois modification halves the function value kept at an end that has not moved for two rounds, which stops one end from getting stuck. It needs only a handful of evaluations. That matters for the PLD, where each evaluation is two FFT compositions of 10,000 steps. The script stops when the bracket is narrower than 10^{-4} and returns the upper end, where $\varepsilon_A \leq \varepsilon^*$. The method does not need exact monotonicity for this. Up to discretization, the returned σ may not be the minimum in Equation 35.2, but it was checked to satisfy $\varepsilon_A(\sigma) \leq \varepsilon^*$, which is all safety needs. For an upper-bound accountant the returned σ is therefore safe.
3. **The bracket for the PLD comes from the other accountants.** The RDP value serves as the upper end: RDP was above the PLD upper bound at every k in Table 35.2, so it should need more noise, though the theory does not guarantee it (Equation 35.1). Ninety percent of the smaller of the RDP and CLT values serves as the lower end. Because neither end is guaranteed, the solver checks that the two ends really bracket the root, with the function positive at one end and negative at the other, and stops with an error if they do not.
4. **The CLT has a closed form.** First find the μ^* whose GDP curve Equation 30.10 gives ε^* at δ. Then solve $\mu = q\sqrt{k(e^{1/\sigma^2} - 1)}$ for σ: squaring gives $q^2k(e^{1/\sigma^2} - 1) = \mu^{*2}$, so $e^{1/\sigma^2} = 1 + \mu^{*2}/(q^2k)$, and taking logs and square roots, $\sigma = 1/\sqrt{\ln(1 + \mu^{*2}/(q^2k))}$.

As a check, the RDP column reproduces $\sigma = 4.126$ for $\varepsilon = 1$ from Exercise 4 in the exercise book of Amplification by subsampling, and the PLD upper bound at $\sigma = 3.9884$ is 1.0000, as the solution of $\varepsilon_A(\sigma) = 1$ should give.

35.3.4. Interpretation

Figure 35.2 shows each accountant's σ relative to the PLD upper bound's, the tightest certified one.

Advanced composition needs six times the noise. It asks for 6.19 to 6.63 times the σ of the PLD ($24.682/3.988 \approx 6.19$ at $\varepsilon = 1$, $10.505/1.584 \approx 6.63$ at $\varepsilon = 3$), a noise level at which DP-SGD rarely trains a useful model.

The RDP conversions cost a few percent of noise. Mironov's conversion costs 8.6 to 24.7% more noise, most at the strictest target, where its fixed $\ln(1/\delta)/(\alpha - 1)$ toll is the largest share of a small ε. RDP with the improved conversion costs 3.4 to 4.9% more noise. This is the σ-side image of the 7.6% ε-excess in Table 35.4, and it is smaller because ε falls faster than σ rises: in this range a small relative increase in σ buys a larger relative decrease in ε, so closing a gap in ε costs a smaller gap in σ. Seen from the other side, at $\sigma = 0.8851$, where the PLD certifies $\varepsilon = 8$, the RDP accountant reports 8.6330.

The CLT asks for less noise than the certified bound needs, 2.0 to 4.8% less. At $\varepsilon = 8$ this is provably too little: at the CLT's $\sigma = 0.8676$ the *optimistic* PLD, a lower bound on the true ε, already gives 8.2594, so the run is not $(8, 10^{-5})$-DP. At $\varepsilon = 1$ and 3 the grid is too coarse to tell. At the CLT's σ the PLD bracket is $[0.9550, 1.0550]$ and $[2.9938, 3.0938]$, and both contain the target, so the truth could be on either side.

The sandwich width matters most at small targets. The width $kh = 0.1$ is a fixed amount of ε, and it is 10% of $\varepsilon = 1$, so the PLD brackets σ only to $[3.654, 3.988]$. For a tight σ at a small target, refine h.

35.3.5. Example: what 4% of noise is worth

With $n = 50{,}000$ records and $q = 0.01$, the expected batch holds $0.01 \times 50{,}000 = 500$ records. At $\varepsilon = 8$, the RDP accountant sets $\sigma = 0.917$, so the averaged gradient carries noise of standard deviation $0.917C/500$. The PLD accountant sets $\sigma = 0.885$: the RDP accountant asks for 3.6% more noise for the same certified budget (the ratio 1.036 in Figure 35.2). The same budget, with a tighter accountant, buys a less noisy model. In the other direction, the PLD lets you keep σ and train longer; the exercises quantify that.

35.3.6. Check yourself

- Why does the script return the upper end of the final bracket rather than its midpoint? *(The upper end is a σ at which the accountant's ε was checked to be at most the target. For an upper-bound accountant that σ certifiably meets the budget; the midpoint might not.)*
- A colleague calibrates σ with the CLT and then reports the ε of the PLD upper bound at that σ. Is their report valid? *(Yes. The reported number comes from a bound, so it is valid. It is just above the target they meant to meet: 8.3594 instead of 8 at the $\varepsilon = 8$ target.)*

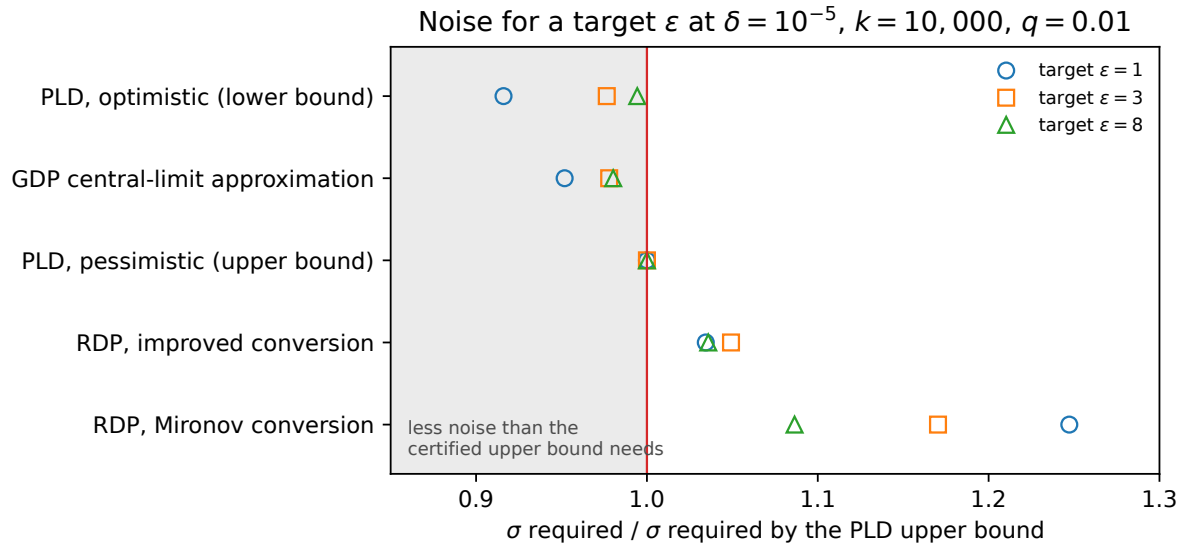


Figure 35.2.: For each target ε at $\delta = 10^{-5}$ ($k = 10{,}000$, $q = 0.01$), the noise multiplier each accountant requires, divided by the one the PLD upper bound requires. RDP with Mironov's conversion asks for up to 25% more noise and RDP with the improved conversion about 4% more. The CLT and the PLD lower bound fall in the shaded region: they ask for less noise than any certified accountant can justify. Advanced composition (6.19 to 6.63) is off the chart.

35.4. Where each accountant is tight

35.4.1. Context

DP-SGD is one mechanism. You will also release statistics with the Gaussian mechanism, answer queries with pure-DP mechanisms (those with $\delta = 0$), shuffle locally randomized reports, or stop training adaptively. The ranking of accountants changes with the mechanism, because what makes an accountant tight is whether the summary it keeps describes that mechanism's privacy loss exactly. This section collects, from the earlier chapters of this part, which accountant is tight for which mechanism, and puts numbers on three of the rows.

35.4.2. Statement

Table 35.6 is the decision table.

Table 35.6.: Which accountant to use for which mechanism, with the chapter or section that establishes it.

Mechanism	Use	Why, and how tight	Avoid
Gaussian, no subsampling (any number of steps, any sensitivity mix)	GDP or zero-concentrated differential privacy (zCDP), converted exactly with Equation 30.10	μ^2 adds and the Gaussian profile is exact (f-DP and Gaussian DP). RDP and zCDP keep the curve exactly but the conversion loses 5 to 10% (Section 28.7)	Per-step (ε, δ) composition
Poisson-subsampled Gaussian (DP-SGD)	PLD, pessimistic, with the optimistic value as error bar	Certified bracket of width kh (Numerical accounting). RDP with the improved conversion is 7.6 to 69% above it on the running example for $k \geq 100$	zCDP, which cannot express amplification (Section 29.8); the CLT as a guarantee
Pure-DP steps (randomized response, Laplace, exponential mechanism)	PLD (exact for finite outputs), or RDP with the mechanism's own curve	The privacy loss takes few values, so the PLD is exact or nearly so. zCDP with $\rho = \varepsilon^2/2$ is convenient but loose for large ε (Section 29.4)	Advanced composition when a tighter accountant is available
Adaptive choice of noise, stopping or per-record budgets	RDP filter at a fixed order (Adaptive composition)	Free at a fixed order (Section 34.5); an odometer costs a union bound over orders	Plugging realized ε_t into a fixed-parameter theorem
Local randomizers behind a shuffler	Exact computation for binary randomized response; numeric clone bound otherwise (Amplification by shuffling and by iteration)	The closed-form clone bound is 4.47 to 7.55 times the exact value there	The Poisson-subsampling accountant

Mechanism	Use	Why, and how tight	Avoid
Only the last iterate released	Amplification by iteration (Amplification by shuffling and by iteration)	Needs a convex, smooth loss, contractive steps and no released checkpoints; rarely applies to deep learning	Any hidden-state claim for non-convex training

35.4.3. Derivation: three rows with numbers

The script computes three rows at $\delta = 10^{-5}$, one where the composed loss is exactly Gaussian, one where it takes only a few values, and the DP-SGD case where it is neither.

1. **Gaussian without subsampling.** A Gaussian step with sensitivity 1 and noise multiplier σ is exactly μ-GDP with $\mu = 1/\sigma$, and under composition the squares of μ add, just as variances of independent Gaussians do (f-DP and Gaussian DP). So one hundred steps with $\sigma = 10$ compose to $\mu = \sqrt{100 \times (1/10)^2} = \sqrt{100}/10 = 1$. The exact ε from Equation 30.10 is 4.3772. RDP's curve for this run, $\alpha\mu^2/2$, is also exact, but converting it costs: it gives 4.7285 with the improved conversion, 8.0% above, and 5.2985 with Mironov's. Here zCDP, which describes the curve by a line $\rho\alpha$, is the same as RDP, because the curve already is a line.
2. **Pure DP: one hundred steps of randomized response with $\varepsilon_0 = 0.1$.** In randomized response each output is e^{ε_0} times more likely under one dataset than the other, in one direction or the other, so its privacy loss is $\pm\varepsilon_0$. The composed loss is ε_0 times (number of + steps minus number of − steps), so the composed PLD is a binomial and the exact $\delta(\varepsilon)$ is a finite sum (Numerical accounting). The exact ε is 4.3068. RDP with the exact randomized-response curve gives 4.6154. zCDP needs a ρ with $\varepsilon(\alpha) \le \rho\alpha$; the smallest that works for randomized response is $\varepsilon_0 \tanh(\varepsilon_0/2)$ per step, which is also the sharp charge every ε_0-DP mechanism satisfies (Equation 29.7 in Concentrated DP), and it gives 4.7263. Advanced composition with $\delta' = \delta$ gives 5.8502, and basic composition $100 \times 0.1 = 10.0000$.
3. **The subsampled Gaussian at $k = 10{,}000$.** From Table 35.2, the PLD brackets ε in $[6.1377, 6.2377]$, RDP (improved) gives 6.7127, 7.6% above the upper bound, and the CLT gives 6.0071, below the bracket.

35.4.4. Interpretation

The more structure the accountant keeps, the tighter it is. The ranking in each row follows the ladder of the part's introduction. An (ε, δ) pair keeps two numbers, zCDP one line, RDP a whole curve, GDP the whole trade-off function (the best type II error of any test at each type I error) when that function is Gaussian, and the PLD the entire distribution of the privacy loss. Each rung loses information the next one keeps, and lost information has to be filled in with a worst case.

The right tool depends on what is Gaussian. When the composed pair is exactly Gaussian, as in the first row, GDP is exact and costs nothing. When subsampling makes it a mixture, as in DP-SGD, no closed form is exact, and only the PLD is both certified and tight.

Adaptivity is the one place where RDP is the natural tool. If the noise or the stopping time is chosen while training runs, based on what has been released, the per-step pair is not fixed in advance. The Rényi filter of Section 34.5 handles that and loses nothing at a fixed order, whereas the PLD accountant above assumes the per-step pair is fixed in advance. The references of Adaptive composition include filters built on GDP and PLDs (Koskela, Tobaben & Honkela 2023) for when the RDP slack matters.

35.4.5. Example: an analytics pipeline

A team releases 100 noisy counts with the Gaussian mechanism at $\sigma = 10$ and reports $\varepsilon = 5.2985$, from an old RDP accountant with Mironov's conversion. The same release is exactly $(4.3772, 10^{-5})$-DP, as GDP shows. The reported number is 21.0% above the truth ($5.2985/4.3772 \approx 1.210$): the team could have answered more queries, or added less noise, at no cost in privacy.

35.4.6. Check yourself

- For the randomized-response row, why is the PLD exact rather than a bracket? *(The privacy loss takes only the values $\pm\varepsilon_0$, so the composed loss lies on the lattice $\{(2j-k)\varepsilon_0\}$, where j is the number of $+$ steps, and no rounding is needed; $\delta(\varepsilon)$ is a finite binomial sum.)*
- Why is zCDP the wrong accountant for DP-SGD even though it is exact for the Gaussian? *(The ratio $\varepsilon(\alpha)/\alpha$ of the subsampled curve tends to $1/(2\sigma^2)$ as α grows, so the smallest ρ with $\varepsilon(\alpha) \le \rho\alpha$ for all α is the unsubsampled $1/(2\sigma^2)$, even though the curve is far below that line at small orders. zCDP cannot express the amplification. See Section 29.8.)*

35.5. Matching the accountant to the code

35.5.1. Context

Every number in Table 35.2 assumes a model of the training run. The accountant cannot check that model against your code: it takes q, σ and k and trusts that the code does what the theorem assumes. Ponomareva et al. (2023, Section 5.3.1) put the point in one heading: "privacy accounting assumptions should match training reality". They note that many papers have reported the Poisson-sampling number while training with shuffled fixed-size batches, going back to Abadi et al. (2016). A mismatch does not make the number a little loose. It makes it a statement about a different algorithm.

35.5.2. Statement

Before reporting ε, check each item of Table 35.7 against the code.

Table 35.7.: A checklist for matching the accountant to the training code.

Item	What the accountant assumes	What to check in the code	If it does not match
Sampling scheme	Poisson sampling at rate q	Is each record included independently with probability q, giving variable batch sizes?	Fixed-size random batches: account the Poisson pair with $\sigma/2$ (Table 31.4). Shuffle and partition: no amplification theorem applies; report the guarantee without amplification, for zero-out neighbours
Neighbouring relation	Add/remove one record	Is the guarantee stated for add/remove, replace-one or zero-out?	Replace-one under Poisson sampling costs more per step: the group-privacy bound is about $e^{\varepsilon} + 1$ times the add/remove value, and the exact pure-DP cost is about twice it for small ε (Section 31.4, Equation 31.8)
Privacy unit	One example	Can one person contribute several examples?	Account for users: clip and sample per user, or pay group privacy
Released outputs	Every noisy gradient	Are checkpoints, logs or gradients exposed?	Nothing to fix: the guarantee covers them all. Claiming less leakage needs Section 33.6 and its conditions
Hyperparameter tuning	Not counted	Were learning rate, clipping norm or σ chosen by looking at results on the private data?	Account for the tuning runs too, or tune on public data
Adaptive parameters	σ, q and k fixed in advance	Early stopping, noise schedules, per-record budgets?	Use a filter (Section 34.5)

Item	What the accountant assumes	What to check in the code	If it does not match
Implementation	Exact real-valued Gaussian noise, exact clipping	Floating-point noise sampling, gradient accumulation across devices, clipping applied before or after accumulation	Out of scope of the accountant: a question for the implementation and its verification, not for this part

35.5.3. Derivation: the price of three mismatches

1. **Fixed-size batches accounted as Poisson.** Suppose each step draws a fresh random batch of exactly qn records. With a fixed batch size, adding a record has to push another one out, so in effect one record is swapped for another, and the batch sum can move by up to $2C$ instead of C. Lebeda et al. (2024, Proposition 9; Table 31.4) show that such a run is covered by the Poisson pair with noise multiplier $\sigma/2$: the same noise, measured against a shift twice as large. At $k = 10{,}000$ the RDP accountant then gives 47.42 instead of 6.71, as in Amplification by subsampling, and the PLD upper bound gives 43.42 instead of 6.24.
2. **Shuffle and partition.** Here each epoch shuffles the data once and cuts it into consecutive batches, so the record is used exactly once per epoch, and there is no amplification theorem that holds for every dataset (Section 31.4). What remains certain is the composition of the record's own steps, one per epoch, for zero-out neighbours, the relation of Table 31.4 for this sampler: the two datasets differ in that the record's gradient is replaced by zero, so the batch that holds it moves by at most C in each epoch. With noise σC, that is a Gaussian mechanism with $\mu = 1/\sigma$. Then 100 epochs at $\sigma = 1$ are 100 Gaussian mechanisms with $\mu = 1$, whose squares add to $\mu = \sqrt{100} = 10$ (Section 30.6), so Equation 30.10 gives $\varepsilon = 91.8173$. This needs the zero-out relation. Under add/remove there is no clean bound at all, because inserting a record shifts the fixed partition into batches and changes every batch after it. Under replace-one, the batch sum moves by up to $2C$, so $\mu = 2$ per epoch, $\mu = \sqrt{100} \times 2 = 20$ in total, and $\varepsilon = 284.39$. Ponomareva et al. recommend reporting such a guarantee without amplification whenever the sampling assumption does not hold.
3. **Tuning on the private data.** If you train 10 configurations on the private data and release the best, which one is best depends on all 10 runs, so the release is a post-processing of all 10 of them, and its privacy cost is that of the 10 runs composed, ten times the run. With RDP, ten times the curve converts to $\varepsilon = 27.23$; for 100 configurations, 145.17. Papernot & Steinke (2022) give much better bounds for tuning procedures that try a random number of configurations. NIST SP 800-226 (Section 4.3) recommends tuning on non-sensitive data, or with a differentially private algorithm.

35.5.4. Interpretation

The sampling scheme is the largest single risk. A factor of 6.96 in ε ($43.42/6.24 \approx 6.96$) from a one-line change in the data loader is more than the choice between the RDP and PLD accountants (at most 1.19 here, from Table 35.3). Opacus, for one, uses Poisson sampling by default when you make a model private.

The privacy unit decides what is protected. The guarantee is about datasets that differ in one unit, so it protects one unit. NIST SP 800-226 (Section 2.4) calls a guarantee whose unit is one event, when a person contributes many, a privacy hazard. Example-level DP-SGD on a dataset with many examples per user protects each example, not each user.

The accountant does not see the implementation. The theorems assume exact real-valued Gaussian noise. Floating-point sampling of noise can break the guarantee of a mechanism that is correct on paper (NIST SP 800-226, Section 4.3). That is a question for formal methods and careful implementation, not for an accountant.

35.5.5. Example: a code review

A training script uses a `DataLoader` with `shuffle=True` and `batch_size=500` on 50,000 records for 100 epochs, and calls an RDP accountant with `sample_rate=0.01`. The checklist flags the first row. The sampler is shuffle and partition, so the Poisson accountant's 6.71 does not apply. The certified guarantee is the unamplified 91.8173, for zero-out neighbours. Switching the loader to Poisson sampling, a change of a few lines, makes the accountant's assumptions true and the guarantee 6.24 with a PLD accountant.

35.5.6. Check yourself

- Your code draws fixed-size random batches, and you want the Poisson accountant's 6.24 to be honest. What noise multiplier must you train with? *(Twice the one in the accountant: $\sigma = 2$, since fixed-size batches are dominated by the Poisson pair with $\sigma/2$, and $2/2 = 1$ is the σ the accountant used.)*
- Does releasing every checkpoint of a DP-SGD run cost extra privacy? *(No. The accountant already assumes every noisy gradient is released, and checkpoints are a post-processing of them.)*

35.6. What to report

35.6.1. Context

An ε without its assumptions cannot be checked, compared or even read: the same number means different things for one example or one user, with or without tuning, at $\delta = 10^{-5}$ or 10^{-10}. Two guidance documents say what a statement should contain. NIST SP 800-226, "Guidelines for Evaluating Differential Privacy Guarantees" (Near, Darais, Lefkovitz & Howarth, March 2025), is written for anyone evaluating a DP claim. Ponomareva et al., "How to DP-fy ML" (JAIR 2023), Section 5.3.3, is written for machine learning.

35.6.2. Statement

From Ponomareva et al. (2023, Section 5.3.3), a report should give:

1. **The DP setting:** central or local, and who is trusted.
2. **The instantiated definition:** which data accesses are covered (one training run, or tuning too), what the mechanism's output is, the unit of privacy, and the neighbouring relation.
3. **Accounting details:** the type of accounting (RDP, PLD, …), its assumptions and whether they hold, and the formal statement, such as the (ε, δ) or ρ values, for the final model and for the tuning.
4. **Transparency:** code, when possible, using standard libraries for the mechanism and the accountant.

On δ, both documents want it small compared with $1/n$ for n units of privacy. The reason is that δ is the probability with which the promise may fail. A mechanism that publishes one randomly chosen record in full changes its output distribution by only about $1/n$ when one record changes, so it meets a δ of about $1/n$ with $\varepsilon = 0$, and yet it exposes someone completely. A δ near $1/n$ therefore allows guarantees that protect nobody. Ponomareva et al. (Section 5.3.2) give the convention $\delta \ll 1/n$, for example $\delta = n^{-1.1}$, with n counted in units of privacy. NIST SP 800-226 (Section 2.3) gives $\delta \leq 1/n^2$ as a common recommendation, calls values above 10^{-5} suspicious, and asks you to report the original parameters of an RDP, zCDP or GDP guarantee alongside any (ε, δ) conversion, so that others can interpret it more precisely.

For DP-SGD, add what the accountant needs to recompute ε: the sampling scheme, q (or batch size and n), k (or epochs), σ, and the accountant's version and discretization. Report the clipping norm C too, although ε does not depend on it.

35.6.3. Derivation: the numbers in the statement

Take the running example with $n = 50{,}000$ records. Then the expected batch is $qn = 500$, $k = 10{,}000$ steps are $kq = 100$ epochs (each step touches a fraction q of the data), and the conventions give $1/n = 2.0 \times 10^{-5}$, $n^{-1.1} = 6.8 \times 10^{-6}$ and $1/n^2 = 4.0 \times 10^{-10}$. The pessimistic PLD composed for $k = 10{,}000$ is computed once. Solving it at several δ is cheap, because it only means evaluating $\delta(\varepsilon)$ on the same composed distribution, and Table 35.8 gives the result.

Table 35.8.: The running example's privacy profile: ε at several δ for $k = 10{,}000$, $q = 0.01$, $\sigma = 1$.

δ	PLD upper	PLD lower	RDP, improved
2.0×10^{-5} $(1/n)$	6.0068	5.9068	6.4884
1.0×10^{-5}	6.2377	6.1377	6.7127
6.8×10^{-6} $(n^{-1.1})$	6.3641	6.2641	6.8353
1.0×10^{-6}	6.9574	6.8574	7.4140
4.0×10^{-10} $(1/n^2)$	9.0502	8.9378	9.4540

$\delta = 10^{-5}$ is half of $1/n = 2.0 \times 10^{-5}$ for this dataset, so it does not meet Ponomareva et al.'s "much smaller than $1/n$". At $1/n^2$, as NIST suggests, the certified ε rises to 9.0502.

The PLD bracket has width 0.1000, the $kh = 10{,}000 \times 10^{-5}$ of Numerical accounting, at every δ down to 10^{-6}, and $9.0502 - 8.9378 = 0.1124$ at $1/n^2$, slightly more. Without truncation the width is at most kh (Section 32.5), so the excess comes from the tails the accountant truncates. Each step's PLD moves up to 10^{-15} of tail mass on each side, in the safe direction. Over 10^4 steps that is of order $10^4 \times 10^{-15} = 10^{-11}$, a few percent of $\delta = 4 \times 10^{-10}$, so it starts to matter only at such a small δ.

The RDP accountant's original parameter, the thing NIST asks you to report, is its curve. At the best order $\alpha = 4.1$ the composed curve is $\varepsilon(4.1) = 3.7336$, which the improved conversion turns into 6.7127.

35.6.4. Interpretation

Report the profile, not a point. A single (ε, δ) pair cannot be converted to a smaller δ (Exercise 3 in the exercise book), because many different mechanisms share that one pair and they behave differently at smaller δ. So a reader who wants $\delta = 1/n^2$ needs either the PLD or the RDP curve, or your table. Table 35.8 costs one line.

Report the accountant's error bar. For a PLD accountant, the optimistic value says how much the number could still fall. For RDP, nothing does: it has no matching lower bound. That is one more reason to prefer the PLD.

State the fallback. If any item of Table 35.7 is in doubt, give the guarantee that does not rely on it, as Ponomareva et al. recommend for amplification. A reader can then see what holds even in the worst case.

35.6.5. Example: a filled-in privacy statement

> **Privacy guarantee.** The model was trained with DP-SGD in the central model: the training server is trusted to run the code below, and the guarantee covers every intermediate noisy gradient, hence every checkpoint and the final model. **Unit of privacy:** one training example; each person contributes at most one example. **Neighbouring relation:** add or remove one example. **Training:** Poisson sampling with rate $q = 0.01$ ($n = 50{,}000$ examples, expected batch size 500), $k = 10{,}000$ steps (100 epochs), per-example clipping norm $C = 1$, Gaussian noise multiplier $\sigma = 1$. **Accountant:** privacy loss distributions, pessimistic discretization on the grid $h = 10^{-5}$, both add and remove directions (this book's `numerical_accounting.py`). **Guarantee:** $(6.24, 10^{-5})$-DP and $(9.06, 4.0 \times 10^{-10})$-DP. The optimistic discretization gives 6.1377 and 8.9378, so the accountant's own error is at most 0.1000 and 0.1124. For comparison, the RDP accountant with the improved conversion gives 6.7127 at $\delta = 10^{-5}$. **Not covered:** hyperparameters were tuned on a public proxy dataset; no tuning on the private data was performed. **If the sampling assumption failed** (for example, shuffled fixed-size batches), the guarantee without amplification is $(91.82, 10^{-5})$-DP for zero-out neighbours (one example's gradient replaced by zero).

Each number in the statement is in Table 35.8 or Section 35.5. Upper bounds are rounded up, never to the nearest value: 9.0502 becomes 9.06, because 9.05 would be a claim the accountant has not certified.

35.6.6. Check yourself

- Why does the statement give the clipping norm even though ε does not depend on it? *(The noise added is σC, so C with σ tells a reader the actual noise level and lets them check the implementation. Ponomareva et al. (Section 4.2) note that the common convention is to report σ and C separately, so that the noise reads as a fraction of the clipping norm and transfers between models.)*
- A paper reports "$\varepsilon = 3$" and nothing else. Name two items without which you cannot interpret it. *(Any two of: δ, the unit of privacy, the neighbouring relation, the sampling scheme, the accountant.)*

35.7. Checking your accountant

35.7.1. Context

An accountant is code, and code has bugs. Before you trust a number, whether from your own accountant or a library's, run checks whose answers you know. A good check is a case where the truth is known exactly, or a property every correct accountant must have.

35.7.2. Statement

Four checks, each run by the script:

1. **Set $q = 1$ and compare with the closed form.** A bound must lie above the exact ε of k full-batch Gaussian steps.
2. **Run the PLD machinery on a pair with a closed form.** The PLD bracket for the plain Gaussian must contain the exact ε.
3. **Check both chains of Equation 35.1 at every k,** and that no upper bound falls below the optimistic PLD.
4. **Round-trip δ.** The composed pessimistic PLD, evaluated at its own ε, must give back the δ it was solved at, and its ε must increase with k.

35.7.3. Derivation: why each answer is known

1. **$q = 1$.** Without subsampling every record is in every batch, so each step is the plain Gaussian mechanism. That mechanism is exactly μ-GDP with $\mu = 1/\sigma$, and since squares of μ add under composition, k of them are exactly $\sqrt{k \times (1/\sigma)^2} = \sqrt{k}/\sigma$-GDP (Section 30.6). Equation 30.10 then gives the exact ε, and every upper bound must be at least this.
2. **The plain Gaussian PLD.** The same closed form is the truth for the composed Gaussian pair. The pessimistic and optimistic discretizations bracket the truth, and without truncation they are within kh of each other (Equation 32.6), so the bracket must contain the closed form.
3. **The chains.** They were proved above for every correct implementation, so a violation is a bug, not slack.

4. **The round trip.** Solving $\delta(\varepsilon) = \delta$ for ε and evaluating δ at the solution are inverse operations on the same decreasing profile, so they must give back δ. For the growth in k: whatever the first k steps release is part of what the first $k+1$ steps release, so the k-step transcript is obtained from the $(k+1)$-step one by dropping the last output. Dropping an output is a data-independent map, so by the data-processing inequality the k-step pair is no easier to tell apart, and no correct accountant can decrease in k.

35.7.4. Example: the checks on this book's accountants

1. For $\sigma = 1$, $k = 10$, $q = 1$: the exact μ is $\sqrt{10}/1 = 3.1623$, and the exact ε is 17.8566. The RDP accountant gives 19.0536, above it as a bound must be. The CLT formula, with $q = 1$, gives $\mu = \sqrt{10(e-1)} = 4.1452$ instead of 3.1623 and $\varepsilon = 25.5853$: it fails the check, because its derivation assumes small q (Section 30.8).
2. On the plain Gaussian with $\mu = 1$ and $k = 10$ at $h = 10^{-3}$, the PLD bracket $[17.8516, 17.8616]$ contains the closed form 17.8566. Its width is $10 \times 10^{-3} = 0.01$, which is kh.
3. Both chains hold on the running example at all nine values of k, and the CLT lies below the PLD lower bound at every one.
4. Evaluating the composed pessimistic PLD at its own ε gives back $\delta = 1.000 \times 10^{-5}$, and the PLD upper bound increases with k.

35.7.5. Interpretation: comparing with independent libraries

This book's accountants use no library, so that every line can be read. Production code should use a maintained one and compare against a second. Three are widely used: Google's `dp_accounting` (RDP and PLD accountants), Opacus (RDP, privacy loss random variable (PRV) and GDP accountants; its default is the PRV accountant), and Microsoft's `prv_accountant` (Gopi, Lee & Wutschitz 2021), which returns a lower bound, an estimate and an upper bound for ε. What agreement should you expect?

RDP accountants agree when the grid and the conversion agree. Two RDP accountants differ only in the orders they try and the formula they convert with. Opacus's default orders are this part's grid without 128, 256 and 512, and its conversion is the improved one (Balle et al. 2020, Theorem 21). On the running example the best orders lie between 4.1 and 9.9, inside both grids, so the results should agree to the precision of the fractional-order computation. If they differ by about $\Delta(\alpha)$ of Equation 28.10, the fixed amount by which the improved conversion beats Mironov's at order α, one of them uses Mironov's conversion. At order 10 that gap is $\frac{\ln 10}{9} + \ln\frac{10}{9} = 0.3612$ (Table 28.3).

Fractional orders are a known weak spot. At an integer order the subsampled Gaussian's RDP is a finite sum, but at a fractional order it needs an infinite series with terms of both signs. `dp_accounting` 0.6.0 computes it with a series whose negative terms it counts as positive, so it returns a loose upper bound there, not the exact value. Opacus keeps the signs and is exact. Checking the fixed-size-batch value 47.42 of Section 35.5 with `dp_accounting` 0.6.0 gives 49.43 (computed with that library, not by this book's scripts). Compare at integer orders, or against Opacus.

PLD and PRV accountants agree within their error bars. Different discretizations give different numbers, but every certified upper bound must lie above the truth and every certified lower bound below it, so every upper bound must lie above every lower bound. If two libraries' brackets do not overlap, one of them has a bug.

No accountant may fall below a certified lower bound. The optimistic PLD is the referee: any reported upper bound below it, like the CLT's, is wrong.

A cross-check against the libraries, run outside this book's scripts, gives a clean result. The RDP and PLD numbers of this chapter agree with Opacus 1.6.0, `dp_accounting` 0.6.0 and `prv_accountant` 0.2.0 to the printed precision, and the libraries' tight PLD estimate at $k = 10{,}000$ is about 6.19, inside the bracket $[6.1377, 6.2377]$. These library values (and the 49.43 above) are quoted from the libraries; the book's scripts do not import them and cannot print them.

35.7.6. Check yourself

- Your RDP accountant reports 7.43 for the running example at $k = 10{,}000$ and a colleague's reports 6.71. Which conversion does each use? *(7.43 is Mironov's, 6.71 the improved one; see Table 35.2.)*
- Why is $q = 1$ a good test case for a DP-SGD accountant? *(The composed pair is exactly Gaussian, so the true ε has a closed form. A bound must lie above it, and an approximation that misses it is caught at once.)*

35.8. Code

The script code/choosing_an_accountant.py imports every accountant from the chapter that derived it and implements nothing new except the comparison, the inverse problem and the checks. Its public functions are `accountant_table(q, sigma, delta, ks)`, which returns one `AccountantRow` per k with every accountant's ε, and `sigma_for_target(eps_target, q, steps, delta, accountant)`, which solves Equation 35.2 for any accountant named in `ACCOUNTANTS`. From the repository root:

```
uv run python code/part04-privacy-accounting/choosing_an_accountant.py
```

It runs in about two and a half minutes, almost all of it in PLD compositions. **Output** (seed 0; nothing is random), summarized in Table 35.9:

Table 35.9.: The script's output with the default seed.

Experiment	Result
Every accountant, $k = 1$ to 10,000	Table 35.2, Table 35.3, Table 35.4; both chains of Equation 35.1 hold at every k, and RDP optimal $\geq$ PLD upper is observed at every k; CLT below the PLD lower bound at every k

Experiment	Result
Noise for a target ε	Table 35.5; PLD upper bound at $\sigma = 3.9884$ is 1.0000
Other mechanisms	Gaussian $\mu = 1$: 4.3772 exact, 4.7285 RDP; randomized response: 4.3068 exact, 4.6154 RDP, 4.7263 zCDP, 5.8502 advanced
Mismatches	Fixed-size batches: 47.42 (RDP), 43.42 (PLD); no amplification: 91.8173 (zero-out), 284.39 (replace-one); 10 tuning runs: 27.23
Privacy profile	Table 35.8
Sanity checks	$q = 1$: 17.8566 exact, 19.0536 RDP, 25.5853 CLT; Gaussian PLD $[17.8516, 17.8616]$

tests/test_choosing_an_accountant.py checks that the output still matches, and calls the two public functions directly.

What to take from it. First, the accountants of the part agree where they must. Every number shared with Amplification by subsampling and Numerical accounting is reproduced to the printed digits, and the bounds satisfy both proved chains of Equation 35.1. Second, the gaps between them are the cost of the choice: per-step composition is off by orders of magnitude, RDP by 7.6% at 10,000 steps and much more for short runs, and the CLT is invalid on this example. Third, mismatched assumptions cost more than the choice between RDP and PLD: a factor of 6.96 over the Poisson PLD bound for fixed-size batches, and 14.72 without amplification (91.8173/6.2377).

35.9. References

- Ponomareva, Hazimeh, Kurakin, Xu, Denison, McMahan, Vassilvitskii, Chien & Thakurta, "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (Journal of Artificial Intelligence Research 77, 2023; arXiv:2303.00654). Section 5.3.1 on matching accounting to data processing (Table 3), Section 5.3.2 on setting δ, Section 5.3.3 on what to report, Section 5.2.1 on target ε tiers. The practical companion to this chapter.
- Near, Darais, Lefkovitz & Howarth, "Guidelines for Evaluating Differential Privacy Guarantees" (NIST Special Publication 800-226, March 2025). The unit of privacy (Section 2.4), the choice of δ and reporting original parameters of RDP, zCDP and GDP guarantees (Section 2.3), floating-point and hyperparameter-tuning hazards (Section 4.3). Read it for what an evaluator will ask.
- Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021). The PRV accountant with certified error bounds, and examples where the CLT under-reports; the basis of `prv_accountant` and Opacus's default accountant.

- Doroshenko, Ghazi, Kamath, Kumar & Manurangsi, "Connect the Dots: Tighter Discrete Approximations of Privacy Loss Distributions" (PoPETs 2022). The pessimistic PLD discretization used in Google's `dp_accounting`.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The RDP of the subsampled Gaussian, computed exactly; the core of every RDP accountant for DP-SGD.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). Theorem 21 is the improved conversion that Opacus and `dp_accounting` use.
- Dong, Roth & Su, "Gaussian Differential Privacy" (Journal of the Royal Statistical Society Series B, 2022), and Bu, Dong, Long & Su, "Deep Learning with Gaussian Differential Privacy" (Harvard Data Science Review, 2020). The GDP CLT for DP-SGD; read with this chapter's warning that it is not a bound.
- Lebeda, Regehr, Kamath & Steinke, "Avoiding Pitfalls for Privacy Accounting of Subsampled Mechanisms under Composition" (arXiv:2405.20769, 2024; IEEE SaTML 2025). Fixed-size batches versus Poisson sampling, and the factor that the wrong accountant costs.
- Chua, Ghazi, Kamath, Kumar, Manurangsi, Sinha & Zhang, "How Private are DP-SGD Implementations?" (ICML 2024). Shuffled batches can be much less private than the Poisson accountant reports.
- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022). Privacy cost of tuning with a random number of trials, far below naive composition.
- Google, `dp_accounting` library; Meta, Opacus; Microsoft, `prv_accountant`. Documentation of the production RDP, PLD and PRV accountants to check this book's numbers against.

35.10. Cheat sheet

Table 35.10 collects the chapter's results for DP-SGD with $q = 0.01$, $\sigma = 1$, $\delta = 10^{-5}$.

Table 35.10.: The accountants on the running example, what kind of statement each makes, and when to use it.

Accountant	Kind	ε at $k = 10{,}000$	σ for $\varepsilon = 3$	When to use it
Per step + advanced composition	Upper bound	24379.55	10.505	Never for DP-SGD
RDP, Mironov conversion	Upper bound	7.4287	1.854	Never: the improved conversion is free
RDP, improved conversion	Upper bound	6.7127	1.662	Quick bounds, adaptive filters, many heterogeneous mechanisms

Accountant	Kind	ε at $k = 10{,}000$	σ for $\varepsilon = 3$	When to use it
PLD, pessimistic	Upper bound, within kh of the truth	6.2377	1.584	The number to report
PLD, optimistic	Lower bound	6.1377	1.547	The error bar, and the referee for other accountants
GDP CLT	Approximation	6.0071	1.549	Intuition only; under-reports here

Rules of thumb

- Report the pessimistic PLD, with the optimistic value beside it. Use RDP with the improved conversion when you need adaptivity or a quick bound, and expect it to overstate ε by 5 to 10% for long runs and by far more for short ones.
- Never report a CLT number as a guarantee. Check any reported number against the PLD lower bound.
- Calibrate σ with the accountant you will report: a looser accountant costs noise, an approximation can cost the guarantee.
- Check the sampling scheme first. Fixed-size or shuffled batches accounted as Poisson is the most expensive error in practice.
- Report the privacy unit, the neighbouring relation, q, k, σ, C, the accountant and its discretization, ε at several δ including one well below $1/n$, and what was not accounted for.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

Part V.

Differential privacy in machine learning

The previous parts built the guarantee and the tools to account for it. This part trains models with it. It asks the questions that decide whether a private model is worth deploying: **how do you train with differential privacy (DP), how much accuracy does it cost, which knobs move that cost, and how do you check that the trained model is as private as the accountant claims?** The answers are mostly about one algorithm, stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD, and about everything around it: how its hyperparameters are chosen, what replaces it when its assumptions fail, how public data and the choice of privacy unit change it, who pays for its noise, which ε to target, and how to audit the result.

Who this part is for

You are expected to have read the earlier parts. From Optimization you need SGD, strong convexity and the sensitivity of regularized empirical risk minimization (ERM). From Membership inference you need the likelihood-ratio attack (LiRA) and the true and false positive rates of an attack. From the DP foundations you need the Gaussian mechanism (The Laplace and Gaussian mechanisms), clipping (Beyond global sensitivity), the exponential mechanism and report-noisy-max (Selection) and the lower bounds of Lower bounds. From the accounting part you need Gaussian differential privacy (GDP) and trade-off functions (f-DP and Gaussian DP), amplification by subsampling and the Rényi differential privacy (RDP) accountant for DP-SGD (Amplification by subsampling), and what to report (Choosing an accountant). Each chapter restates the facts it uses and links to them.

The common thread

A private training algorithm has two halves. The **privacy half** says which quantity has bounded sensitivity and how much noise covers it; it is a theorem, and the accountants of the previous part turn it into a number. The **utility half** says how much the noise and the bounding cost; it is mostly measured, rarely proved. Every chapter of this part moves one of the two.

The first two chapters build the algorithm: Private ERM compares three places to inject noise into training and shows why only noise on the gradients survives non-convex models, and DP-SGD adds clipping, minibatches and accounting. The next three ask what else spends the budget or could spend it better: hyperparameter search; mechanisms that do not rely on random batches, such as the follow the regularized leader (FTRL) algorithm run on noisy gradient sums, DP-FTRL; and private prediction instead of private training, such as private aggregation of teacher ensembles (PATE). The next three change the setting: public data, a different privacy unit, and the effect on under-represented groups. The last two close the loop between theory and practice: which ε to choose, and how to test a claim empirically. Table 35.11 lists them.

Table 35.11.: The chapters of this part, the question each answers, and the earlier chapters it uses.

Chapter	The question it answers	Builds on
Private empirical risk minimization	Where can noise enter training, and what does each choice cost?	6, 7, 20, 23, 26, 30
DP-SGD	How do clipping, noise and sampling make deep-learning training private, and what does each knob do?	6, 20, 23, 31, 35, 36
Hyperparameter tuning and private selection	How much privacy does tuning leak, and how do you pay for it honestly?	22, 27, 37
DP-FTRL and matrix factorization	How do you train privately without trusting random batches?	20, 30, 37
Private prediction: PATE and in-context learning	When is it better to privatize answers than weights?	22, 23, 27, 37
Public data and private fine-tuning	How much does a public pretrained model buy, and why does fine-tuning fewer parameters help?	37, 38
User-level differential privacy	What changes when one person contributes many records?	19, 37
Disparate impact	Who pays for the noise and the clipping?	37
Choosing ε	Which ε should you target, and how do you justify it?	18, 30, 35, 37
Auditing private training	Is the model as private as the accountant says?	10, 30, 37, 44

The running examples

Two models recur. The first is **regularized logistic regression** in $d = 10$ dimensions on synthetic data, where the exact minimizer can be computed and every method can be compared with the non-private optimum; Private ERM introduces it. The second is a **small neural network** trained with DP-SGD on synthetic data, small enough that every experiment of the part runs on a laptop in seconds, yet non-convex, so that clipping and noise behave as they do in deep learning; DP-SGD introduces it. The study plan's exercise, DP-SGD on CIFAR-10 audited with LiRA, needs a GPU and PyTorch, which are not dependencies of the repository; the chapters scale it down and say how.

How to read this part

Order. Read chapters 36 and 37 first, in order: every later chapter uses DP-SGD. Chapters 38 and 39 change how the algorithm is run, and can be read in either order. Chapters 40 to 43 are independent of each other. Chapter 44 uses the vocabulary of all the earlier ones, and chapter 45 should come last, because it checks what the others claim.

Structure. The chapters follow the shape of the earlier parts: why the topic matters, a toolkit, one section per result with a derivation, interpretation, worked example and two check-yourself questions, then code, references and a cheat sheet. Exercises with worked solutions are in the exercise book, a companion website, with a link at the end of each chapter.

Run the code. Each chapter has a script in `code/` that prints every number its text quotes, except values attributed to a paper, and regenerates its figures, for example

```
uv run python code/part05-dp-ml/dp_erm.py
```

Every model is written in NumPy, gradients included, so you can see exactly what each algorithm computes.

Notation

The part uses the notation of the earlier parts, extended in Table 35.12.

Table 35.12.: Notation used throughout this part, in addition to that of the earlier parts.

Symbol	Meaning
$D = (z_1, \dots, z_n)$, $z_i = (x_i, y_i)$	A training set of n records with features x_i and labels y_i
w, $\ell(w, z)$, $J(w; D)$	Model weights; the loss on one record; the training objective
w^*, $\tilde{w}$	The non-private minimizer; a private release
$g_i = \nabla_w \ell(w, z_i)$	The gradient of record i's loss, a per-example gradient
C	The clipping norm: per-example gradients are rescaled to length at most C
σ	The noise multiplier: the noise standard deviation in units of C
q, B, T	The sampling rate; the expected batch size qn; the number of steps
$\mu(\varepsilon, \delta)$	The largest ratio of sensitivity to noise standard deviation for which a Gaussian mechanism is (ε, δ)-DP

What you will be able to do

By the end of this part you should be able to:

- train a convex model privately by output, objective or gradient perturbation, and predict its excess risk;
- implement DP-SGD from scratch, with per-example clipping, Poisson sampling and an RDP accountant, and explain what each hyperparameter trades;
- account for hyperparameter search, and run a private selection procedure;
- explain when DP-FTRL or a matrix-factorization mechanism beats DP-SGD, and why;
- train a teacher ensemble and answer queries with PATE or a private in-context learner;
- decide whether to fine-tune all parameters or a few, with a public pretrained model;
- account for user-level privacy, and bound each user's contribution;
- measure and explain the disparate impact of private training on small groups;
- choose an ε and justify it to someone who will hold you to it;
- audit a private training run with canaries and turn the attack's success into an empirical lower bound on ε.

36. Private empirical risk minimization

36.1. Why this matters

Most of machine learning is **empirical risk minimization (ERM)**: choose the parameters w that minimize the average loss on the training data, usually plus a penalty on the size of w. The trained weights are a function of every training record, and Membership inference showed that they give some of those records away. Differential privacy (DP) promises the opposite: the distribution of whatever you release changes by at most a factor e^{ε}, plus a slack δ, when one record is replaced. To keep that promise for a trained model, randomness has to enter somewhere between the data and the released weights. This chapter asks where.

There are three natural places, and each gives a classic algorithm. **Output perturbation** trains as usual and adds noise to the final weights. **Objective perturbation** adds a random linear term to the objective, then minimizes it exactly. **Gradient perturbation** adds noise to every gradient during training. All three were analysed for convex losses between 2008 and 2014, by Chaudhuri, Monteleoni and Sarwate, by Kifer, Smith and Thakurta, and by Bassily, Smith and Thakurta, and the third is the ancestor of the algorithm almost every private deep-learning system runs today: stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, the subject of the next chapter.

36.1.1. The one idea: privacy needs a bound on how far one record can move what you release

Every DP mechanism in this book works the same way. Find a quantity whose change, when one record is replaced, is bounded by a known number, its **sensitivity**. Then add noise large enough to hide a change of that size. The three methods differ only in *which* quantity they bound.

Output perturbation bounds the minimizer itself. Strong convexity, the curvature a ridge penalty $\frac{\lambda}{2}\|w\|^2$ adds, guarantees that replacing one of n records moves the minimizer by at most $2L/(\lambda n)$, where L bounds the length of each record's loss gradient (Optimization). Objective perturbation bounds the *gradient* of the objective at every point, which also moves by at most $2L/n$, and turns that into a bound on the output's density through a change of variables. Gradient perturbation bounds the gradient at the points the algorithm actually visits, one step at a time, and composes the steps.

The order matters because each step back toward the data needs fewer assumptions. Output perturbation needs an exact, unique minimizer of a strongly convex objective. Objective perturbation needs that and a loss with bounded curvature. Gradient perturbation, for its privacy, needs only that each record's gradient is bounded: no convexity, no exact solution. That is why it is the one that survives in deep learning.

Table 36.1 lists the results and where each is used.

Table 36.1.: The results of this chapter and where each is used.

Result	Where it is used
Output perturbation: noise of scale $2L/(\lambda n)$ on the minimizer	Convex models trained to convergence; the baseline the others are measured against
Sensitivity of an approximate minimizer	Why output perturbation needs the exact solution, and how much an inexact one costs
Objective perturbation: a random linear term, minimized exactly	Private logistic regression and linear support vector machines; why curvature in the data helps
Noisy gradient descent: Gaussian noise on every gradient	The privacy argument of DP-SGD; every iterative private trainer
Excess risk of all three, and the optimal rates	Choosing between them; what no method can beat (Lower bounds)
What survives without convexity	Why deep learning uses gradient perturbation, with clipping

Notation. A training set $D = (z_1, \dots, z_n)$ holds n records $z_i = (x_i, y_i)$ with features $x_i \in \mathbb{R}^d$ and labels $y_i \in \{-1, +1\}$. The loss of weights w on a record is $\ell(w, z)$, and the regularized objective is

$$J(w; D) = \frac{1}{n} \sum_{i=1}^{n} \ell(w, z_i) + \frac{\lambda}{2} \|w\|^2, \tag{36.1}$$

with minimizer $w^*(D)$. Neighbouring datasets $D \simeq D'$ differ by **replacing** one record, z in D by z' in D', so both have size n; the next chapter switches to adding or removing one, and says why. L bounds the length of every gradient $\nabla_w \ell(w, z)$, $\|\cdot\|$ is the Euclidean norm, and a released, private version of the weights is written $\tilde{w}$. The **excess risk** of a release is $J(\tilde{w}; D) - J(w^*; D)$, how much worse it fits the training objective than the exact minimizer.

The running example. Regularized logistic regression, with loss $\ell(w, (x, y)) = \ln(1 + e^{-y\langle w, x\rangle})$, in $d = 10$ dimensions with ridge $\lambda = 0.01$. The features are uniform on the unit sphere, $\|x\| = 1$, and the labels come from a logistic model whose true weights have length 5. The derivative of $\ln(1 + e^{-m})$ in the margin $m = y\langle w, x\rangle$ lies in $(-1, 0)$, so each gradient has length at most $\|x\| = 1$ and $L = 1$. Every method targets $\varepsilon = 1$, with $\delta = 10^{-5}$ for those that need a δ, on n from 1,000 to 100,000 records.

How to read this chapter. Read the toolkit, then Section 36.3, which sets the pattern. Section 36.4 has the chapter's one long derivation; its one-dimensional example shows each step on numbers. Section 36.5 is the one to know well, because the next chapter builds on it. Section 36.6 and Section 36.7 say which method to use when.

36.2. Toolkit: five facts every derivation uses

36.2.1. Fact 1 — The Gaussian mechanism, calibrated exactly

To release a vector-valued function $f(D)$ whose value moves by at most Δ in Euclidean length between neighbours (its ℓ_2 **sensitivity**), add independent normal noise $\mathcal{N}(0, \sigma^2)$ to each coordinate. What matters for privacy is only the ratio $\mu = \Delta/\sigma$, the shift measured in noise standard deviations: rotate so the shift lies along one axis, and the other coordinates have the same distribution under both datasets. The mechanism is then (ε, δ)-DP exactly when

$$\delta \geq \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big), \tag{36.2}$$

where Φ is the standard normal distribution function (The Laplace and Gaussian mechanisms, Balle & Wang 2018). The right side increases with μ, so for each (ε, δ) there is a largest admissible ratio $\mu(\varepsilon, \delta)$, and the smallest noise is $\sigma = \Delta/\mu(\varepsilon, \delta)$. For $\varepsilon = 1$ and $\delta = 10^{-5}$, $\mu = 0.2681$, so $\sigma = 3.7306\,\Delta$. The classic formula $\sigma = \Delta\sqrt{2\ln(1.25/\delta)}/\varepsilon$ gives $4.8448\,\Delta$, 30% more noise for the same guarantee; this chapter always uses the exact calibration.

36.2.2. Fact 2 — The regularized minimizer moves by at most $2L/(\lambda n)$

Optimization proves this, and the argument is short enough to restate. Replacing z by z' changes J by $h(v) = \frac{1}{n}(\ell(v, z') - \ell(v, z))$, whose gradient has length at most $2L/n$: one gradient leaves, another arrives. At the new minimizer $w' = w^*(D')$, the old objective's gradient is therefore $\nabla J(w'; D) = -\nabla h(w')$, of length at most $2L/n$. The ridge makes $J(\cdot; D)$ **λ-strongly convex**: its gradient grows by at least λ per unit of distance in every direction, so a point where the gradient has length g is within g/λ of the minimizer. Hence

$$\|w^*(D) - w^*(D')\| \leq \frac{2L}{\lambda n}. \tag{36.3}$$

The same argument gives a bound for any point, not only the exact minimizer: if $\|\nabla J(w; D)\| \leq g$, then $\|w - w^*(D)\| \leq g/\lambda$.

36.2.3. Fact 3 — Noise with density proportional to $e^{-\|b\|/s}$

Pure ε-DP, with $\delta = 0$, needs noise whose density falls off no faster than exponentially in every direction. The standard choice in d dimensions has density $\nu(b) \propto e^{-\|b\|/s}$: it depends on b only through its length. If f has ℓ_2 sensitivity Δ and $s = \Delta/\varepsilon$, then $f(D) + b$ is ε-DP. At any output y, the ratio of the two output densities is $e^{(\|y - f(D')\| - \|y - f(D)\|)/s}$. By the triangle inequality the exponent is at most $\|f(D) - f(D')\|/s \leq \Delta/s = \varepsilon$.

To sample it, split b into a direction and a length. The density depends only on the length r, and the sphere of radius r has surface area proportional to r^{d-1}, so the length has density proportional to $r^{d-1}e^{-r/s}$. That is a **gamma distribution** with shape d and scale s, and the direction is uniform on the sphere. Its moments are $\mathbb{E}\|b\| = ds$ and $\mathbb{E}\|b\|^2 = d(d+1)s^2$. Compare the Gaussian, whose squared length has mean $d\sigma^2$: the pure-DP noise is longer by a factor that grows with d.

36.2.4. Fact 4 — Change of variables for densities

If a random vector b has density ν and w is obtained from b by a smooth invertible map, write the inverse as $b = \phi(w)$. Then w has density

$$p(w) = \nu(\phi(w)) \left| \det \nabla \phi(w) \right|. \tag{36.4}$$

The determinant corrects for stretching: if ϕ squeezes a small box around w into a box of half the volume around b, the probability of that box is the same, so the density at w is half the density at b. In one dimension this is the familiar $p(w) = \nu(\phi(w))|\phi'(w)|$.

36.2.5. Fact 5 — Adaptive Gaussian steps compose into one Gaussian

Run T Gaussian mechanisms in sequence, each allowed to depend on the outputs of the earlier ones, with sensitivity-to-noise ratios $\mu_1, \dots, \mu_T$. The whole sequence is exactly as private as a single Gaussian mechanism with ratio

$$\mu = \sqrt{\mu_1^2 + \cdots + \mu_T^2}. \tag{36.5}$$

This is composition of Gaussian differential privacy (GDP) from f-DP and Gaussian differential privacy. The reason is that the privacy loss of each step is a normal random variable with variance μ_t^2 and mean $\mu_t^2/2$, given the past. Their sum is normal with variance $\sum_t \mu_t^2$ and half that as its mean, which is the privacy loss of one Gaussian with ratio μ. So T equal steps with ratio μ_{step} give $\mu = \sqrt{T}\,\mu_{\text{step}}$, and Equation 36.2 converts the total to (ε, δ) with no slack.

36.3. Output perturbation

36.3.1. Context

The simplest private trainer treats training as a black box: compute the exact minimizer $w^*(D)$, then add noise. Fact 2 says the minimizer is a function with ℓ_2 sensitivity $2L/(\lambda n)$, so this is just the Gaussian mechanism (or Fact 3's mechanism) applied to it. Chaudhuri & Monteleoni (2008) introduced it for logistic regression. Two questions remain: how much accuracy does it cost, and what happens when, as always in practice, the minimizer is only computed approximately?

36.3.2. Statement

Release

$$\tilde{w} = w^*(D) + Z, \qquad Z \sim \mathcal{N}(0, \sigma^2 I), \qquad \sigma = \frac{2L}{\lambda n\, \mu(\varepsilon, \delta)}. \tag{36.6}$$

This is (ε, δ)-DP for replace-one neighbours. Replacing Z by noise with density $\propto e^{-\|b\|/s}$ and $s = 2L/(\lambda n \varepsilon)$ gives pure ε-DP. If the objective has Hessian H at w^*, the excess risk is, to second order,

$$\mathbb{E}\big[J(\tilde{w}) - J(w^*)\big] \approx \frac{1}{2}\,\mathbb{E}\big[Z^\top H Z\big] = \frac{\mathbb{E}\|Z\|^2}{2d}\,\operatorname{tr} H, \tag{36.7}$$

for any noise whose direction is uniform, so $\frac{\sigma^2}{2}\operatorname{tr} H$ for the Gaussian. An approximate minimizer w with $\|\nabla J(w; D)\| \le g$ has sensitivity at most

$$\frac{2L}{\lambda n} + \frac{2g}{\lambda}. \tag{36.8}$$

36.3.3. Derivation

1. **Privacy.** By Fact 2, w^* moves by at most $\Delta = 2L/(\lambda n)$ between neighbours. Adding Gaussian noise with $\sigma = \Delta/\mu(\varepsilon, \delta)$ is (ε, δ)-DP by Fact 1, and adding noise $\propto e^{-\|b\|/s}$ with $s = \Delta/\varepsilon$ is ε-DP by Fact 3. Nothing else about the data reaches the output.
2. **Expand the objective around its minimizer.** The gradient vanishes at w^*, so a second-order Taylor expansion gives $J(w^* + Z) - J(w^*) \approx \frac{1}{2} Z^\top H Z$, with $H = \nabla^2 J(w^*)$. The noise is small next to the scale on which the curvature changes in the cases of interest; Section 36.6 checks the approximation against simulation.
3. **Average over the direction.** Write $Z = \|Z\| u$ with u a unit vector. For a uniformly random direction, $\mathbb{E}[u_i u_j]$ is $1/d$ when $i = j$, because the d squared coordinates of a unit vector add up to 1 and are exchangeable, and 0 otherwise, by symmetry under flipping one coordinate's sign. So $\mathbb{E}[u^\top H u] = \sum_i H_{ii}/d = \operatorname{tr} H/d$, and if the length is independent of the direction, as for both noises here, $\mathbb{E}[Z^\top H Z] = \mathbb{E}\|Z\|^2 \operatorname{tr} H/d$. For the Gaussian, $\mathbb{E}\|Z\|^2 = d\sigma^2$. This is Equation 36.7.
4. **Approximate minimizers.** Let w and w' be the outputs of the optimizer on D and D', with gradients of length at most g. By the last line of Fact 2, each is within g/λ of its exact minimizer. The triangle inequality along the path $w \to w^*(D) \to w^*(D') \to w'$ gives $\|w - w'\| \le g/\lambda + 2L/(\lambda n) + g/\lambda$, which is Equation 36.8. The stopping rule must be a gradient check, not a step count: only the gradient bound is guaranteed on every dataset.

36.3.4. Interpretation

The noise is calibrated to the worst case, the damage to the actual case. The sensitivity $2L/(\lambda n)$ uses only the curvature λ that the ridge guarantees. The damage, Equation 36.7, is measured by the actual curvature H, which includes the data's own. Put the two together for the Gaussian: the excess risk is about $\frac{\operatorname{tr} H}{2} \cdot \frac{4L^2}{\lambda^2 n^2 \mu^2}$. If every eigenvalue of H is close to some h, this is $\frac{2dL^2}{\mu^2 n^2} \cdot \frac{h}{\lambda^2}$. The factor h/λ^2 grows as the ridge shrinks, whatever the data look like. A small ridge buys a better non-private fit and costs a lot of noise.

It falls like $1/n^2$. The noise standard deviation falls like $1/n$, so its square, and the excess risk with it, falls like $1/n^2$. The statistical error of the non-private minimizer falls only like $1/n$, so for large n privacy eventually costs less than sampling does, in this convex, low-dimensional setting.

It needs the exact solution. Equation 36.8 adds $2g/\lambda$ to the sensitivity. To keep that below the exact term, the optimizer must reach a gradient of length $g \le L/n$, which shrinks with the dataset. An optimizer stopped after a fixed number of steps has no such guarantee on every dataset, and then the sensitivity bound, and the privacy proof, do not apply. Wu et al. (2017) analyse output perturbation of SGD run for a fixed number of passes directly, with a different sensitivity bound.

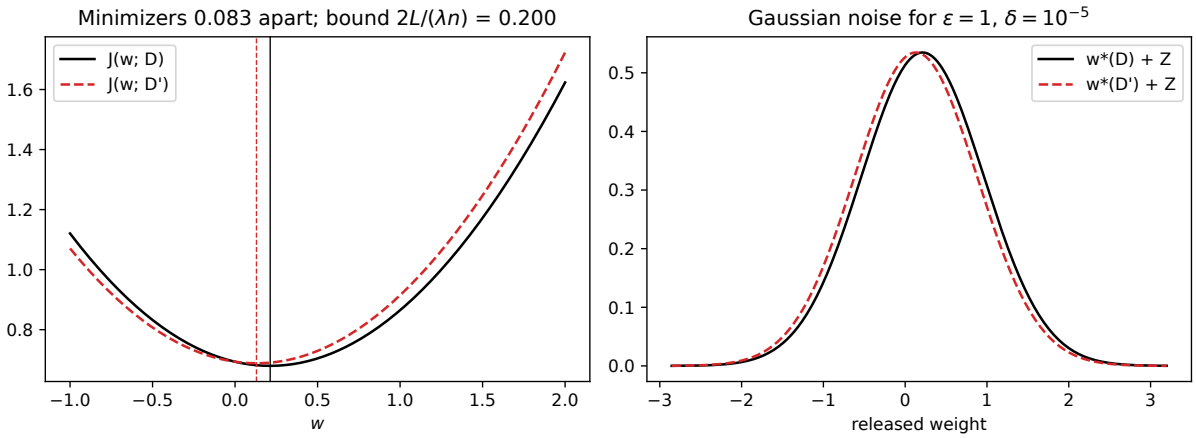


Figure 36.1.: Left: two neighbouring regularized logistic objectives in one dimension ($n = 20$, $\lambda = 0.5$), their minimizers (vertical lines) 0.083 apart, below the bound $2L/(\lambda n) = 0.2$. Right: the released weight's density under each dataset with Gaussian noise for $\varepsilon = 1$, $\delta = 10^{-5}$; the noise must hide a shift as large as the bound, not the actual shift.

36.3.5. Example: one dimension, then the running example

Take one feature, $n = 20$ records and $\lambda = 0.5$ (Figure 36.1). Replacing the last record, which has $x = 1$, by the same feature with the opposite label moves the minimizer from 0.2128 to 0.1296, by 0.0832. The bound is $2/(0.5 \times 20) = 0.2$, and that is what the noise is calibrated to: $\sigma = 3.7306 \times 0.2 = 0.7461$. The two output densities overlap almost completely, as they must.

In the running example ($d = 10$, $\lambda = 0.01$), $n = 10{,}000$ gives sensitivity $2/(0.01 \times 10{,}000) = 0.02$ and $\sigma = 0.07461$ per coordinate, next to a minimizer of length 2.886. The Hessian at w^* has eigenvalues between 0.0263 and 0.0330, and trace 0.3129. Equation 36.7 predicts $\frac{1}{2} \times 0.07461^2 \times 0.3129 = 8.711 \times 10^{-4}$, and 20 simulated releases average 9.274×10^{-4}. The pure-DP version, with noise $\propto e^{-\|b\|/s}$ and $s = 0.02$, has $\mathbb{E}\|b\|^2 = 110 \times 0.02^2$; the prediction 6.885×10^{-4} and the simulated 8.321×10^{-4} differ by more, because the squared length of gamma noise varies a lot from draw to draw and 20 draws average it only roughly.

Stopping early is costly. With a gradient tolerance of $g = 10^{-6}$ the sensitivity is 0.0202, essentially exact. With $g = 10^{-4}$ it doubles to 0.04, and with $g = 10^{-3}$ it is 0.22, eleven times the exact value. A tolerance that looks tight, 10^{-3}, multiplies the noise by eleven and the excess risk by about 121.

36.3.6. Check yourself

- You halve the ridge λ. By what factor does the Gaussian noise grow, and roughly by what factor does the excess risk grow if the data's curvature dominates H? *(The noise doubles, since $\sigma \propto 1/\lambda$. With H unchanged the excess risk, proportional to $\sigma^2 \operatorname{tr} H$, grows fourfold.)*
- Why must the stopping rule be "stop when $\|\nabla J\| \le g$" rather than "stop after 50 Newton steps"? *(The privacy proof needs the sensitivity bound on every pair of neighbouring datasets.*

A gradient check guarantees it everywhere; a step count only guarantees it on datasets where 50 steps happen to be enough.)

36.4. Objective perturbation

36.4.1. Context

Output perturbation wastes the data's curvature: it pays for the worst case λ and is judged on the actual H. Chaudhuri, Monteleoni & Sarwate (2011) put the noise inside the optimization instead. Add a random linear term $\frac{1}{n}\langle b, w\rangle$ to the objective and minimize exactly. A linear term is a constant force; the objective's curvature, all of it, resists that force. Where the data add curvature, the minimizer moves less. The price is a subtler privacy proof, because the output is no longer "something fixed plus noise".

36.4.2. Statement

Let the loss have the form $\ell(w,(x,y)) = g(y\langle w, x\rangle)$ with g convex and twice differentiable, $|g'| \le 1$ and $0 \le g'' \le c$, and let $\|x\| \le 1$; so the gradients have length at most $L = 1$. Logistic regression qualifies with $c = \frac{1}{4}$. Set

$$\varepsilon' = \varepsilon - 2\ln\Big(1 + \frac{c}{n\lambda}\Big), \tag{36.9}$$

draw b with density $\nu(b) \propto e^{-\varepsilon'\|b\|/2}$ (Fact 3 with $s = 2/\varepsilon'$), and release

$$\tilde{w} = \arg\min_{w}\Big[J(w; D) + \frac{1}{n}\langle b, w\rangle\Big]. \tag{36.10}$$

If $\varepsilon' > 0$, this is ε-DP for replace-one neighbours (Chaudhuri, Monteleoni & Sarwate 2011, Algorithm 2). If not, increase the ridge to $\lambda + \lambda_+$ with $\lambda_+ = \frac{c}{n(e^{\varepsilon/4}-1)} - \lambda$, which makes the logarithm $\varepsilon/4$ and lets $\varepsilon' = \varepsilon/2$.

The minimizer moves, to first order, by $-H^{-1}b/n$, so the excess risk is about

$$\mathbb{E}\big[J(\tilde{w}) - J(w^*)\big] \approx \frac{1}{2n^2}\mathbb{E}\big[b^\top H^{-1} b\big] = \frac{\mathbb{E}\|b\|^2}{2dn^2}\operatorname{tr} H^{-1}. \tag{36.11}$$

36.4.3. Derivation

The plan is to write the density of $\tilde{w}$ explicitly with Fact 4, then bound the ratio of the two densities, for D and D', at every point. The ratio splits into a noise factor and a stretching factor, and each consumes part of ε.

1. **From the output back to the noise.** The perturbed objective is λ-strongly convex, so it has exactly one minimizer, where its gradient is zero: $\nabla J(\tilde{w}; D) + b/n = 0$. Solving for b,

$$b = \phi_D(\tilde{w}) = -n\nabla J(\tilde{w}; D) = -\sum_{i=1}^{n}\nabla\ell(\tilde{w}, z_i) - n\lambda\tilde{w}. \tag{36.12}$$

   Every w is the output for exactly one b, namely $\phi_D(w)$, so the map from noise to output is invertible and ϕ_D is its inverse.

2. **The density of the output.** By Fact 4, $p_D(w) = \nu(\phi_D(w))\,|\det \nabla\phi_D(w)|$, and the same for D' with $\phi_{D'}$. The ratio at any w is
$$\frac{p_D(w)}{p_{D'}(w)} = \underbrace{\frac{\nu(\phi_D(w))}{\nu(\phi_{D'}(w))}}_{\text{noise}} \cdot \underbrace{\frac{|\det \nabla\phi_D(w)|}{|\det \nabla\phi_{D'}(w)|}}_{\text{Jacobian}}. \tag{36.13}$$
3. **The noise factor is at most $e^{\varepsilon'}$.** The sums in ϕ_D and $\phi_{D'}$ share every term except the replaced record's, so $\phi_{D'}(w) - \phi_D(w) = \nabla\ell(w,z) - \nabla\ell(w,z')$, of length at most 2. The noise factor is $e^{\varepsilon'(\|\phi_{D'}(w)\| - \|\phi_D(w)\|)/2}$, and by the triangle inequality the exponent is at most $\varepsilon' \cdot 2/2 = \varepsilon'$.
4. **The Jacobian is the Hessian.** $\nabla\phi_D(w) = -n\nabla^2 J(w;D) = -A$, with $A = \sum_i g''(m_i)x_i x_i^\top + n\lambda I$, where $m_i = y_i\langle w, x_i\rangle$ is the margin; the chain rule gives each loss's Hessian as $g''(m_i)$ times $y_i^2 x_i x_i^\top = x_i x_i^\top$. Every term is positive semi-definite and $n\lambda I$ is added, so all eigenvalues of A are at least $n\lambda$. For D' the matrix is $A' = A + E$ with $E = g''(m')x'x'^\top - g''(m)xx^\top$: one rank-one term in, one out.
5. **The Jacobian factor is at most $(1 + c/(n\lambda))^2$.** The ratio of determinants is $\det(A')/\det(A) = \det(I + A^{-1}E) = \det(I + M)$ with $M = A^{-1/2}EA^{-1/2}$, since $A^{-1}E$ and M are similar matrices. M is symmetric and is the difference of two rank-one positive semi-definite matrices, $g''(m')\tilde{x}'\tilde{x}'^\top - g''(m)\tilde{x}\tilde{x}^\top$ with $\tilde{x} = A^{-1/2}x$. A rank-one matrix $vv^\top$ has one nonzero eigenvalue, $\|v\|^2$, and here $\|\tilde{x}\|^2 = x^\top A^{-1}x \le \|x\|^2/(n\lambda) \le 1/(n\lambda)$. So M has at most two nonzero eigenvalues, each between $-a$ and a with $a = c/(n\lambda)$, and $\det(I+M) = (1+\lambda_1)(1+\lambda_2) \le (1+a)^2$. Exchanging the roles of D and D' gives the same bound for the inverse ratio, because A' also has all eigenvalues at least $n\lambda$.
6. **Add the two budgets.** The log of the density ratio is at most $\varepsilon' + 2\ln(1 + c/(n\lambda)) = \varepsilon$ by Equation 36.9, at every w and in both directions. A bound on the density ratio everywhere is pure ε-DP.
7. **When $\varepsilon' \le 0$.** The Jacobian factor alone would exceed ε. A larger ridge shrinks a: with $\lambda + \lambda_+$ in place of λ, $c/(n(\lambda+\lambda_+)) = e^{\varepsilon/4} - 1$, so $2\ln(1+a) = \varepsilon/2$, and the noise gets the other half.
8. **Excess risk.** To first order in b, the perturbed minimizer solves $H(\tilde{w} - w^*) + b/n = 0$, so $\tilde{w} - w^* = -H^{-1}b/n$. Substituting this into the second-order expansion $J(\tilde{w}) - J(w^*) \approx \frac{1}{2}(\tilde{w} - w^*)^\top H(\tilde{w} - w^*)$ of step 2 of Section 36.3 gives $\frac{1}{2n^2}b^\top H^{-1}HH^{-1}b = \frac{1}{2n^2}b^\top H^{-1}b$. Averaging over the uniform direction, as in step 3 there, replaces $b^\top H^{-1}b$ by $\mathbb{E}\|b\|^2 \operatorname{tr} H^{-1}/d$, which is Equation 36.11.

Walk-through in one dimension. With $d = 1$ everything in steps 1–5 is a number, and the script computes both densities exactly on a grid of outputs. Take $n = 5$ records with features in $[-1, 1]$, $\lambda = 0.1$ and $\varepsilon = 1$. Then $c/(n\lambda) = 0.25/0.5 = 0.5$, the Jacobian budget is $2\ln 1.5 = 0.8109$, and the noise gets $\varepsilon' = 0.1891$. Figure 36.2 shows one dataset: the two densities, and their log ratio, which stays inside $\pm\varepsilon$. Over 500 random datasets and replacements, the largest log ratio anywhere on the grid is 0.3779. Its noise part never exceeds 0.0992, about half its budget of 0.1891, and its Jacobian part never exceeds 0.3395, well under its budget of 0.8109, and under even $\ln 1.5 = 0.4055$, half of it.

36.4.4. Interpretation

Where the slack is. Both factors in step 2 are bounded by worst cases that cannot happen together, and mostly cannot happen at all for logistic regression. The noise factor reaches $e^{\varepsilon'}$ only

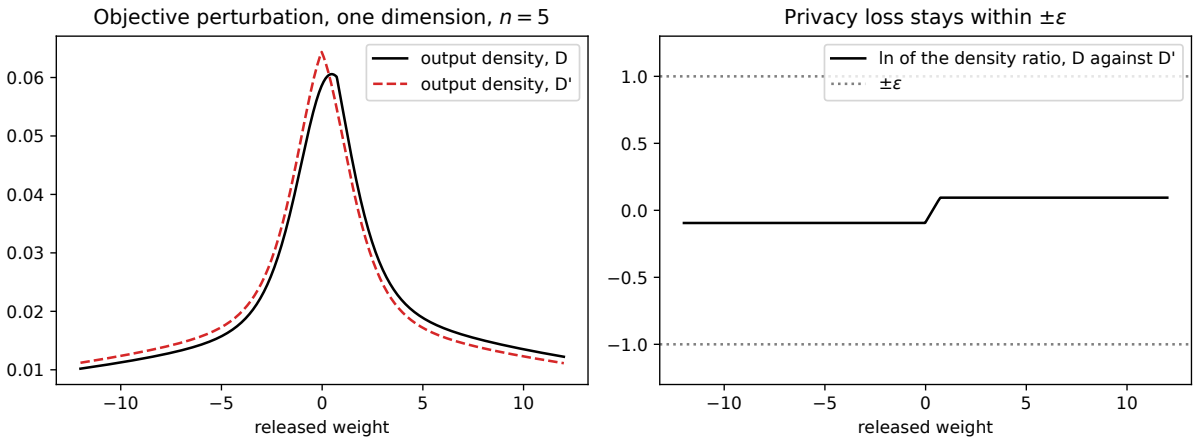


Figure 36.2.: Objective perturbation in one dimension, $n = 5$, $\lambda = 0.1$, $\varepsilon = 1$. Left: the exact densities of the released weight under a dataset and its neighbour, computed by change of variables. Right: their log ratio, the privacy loss, which stays well inside $\pm\varepsilon$.

if the old and new record have full-length gradients pointing in opposite directions. A logistic gradient has full length only when its margin is very negative, and two records whose gradients point in opposite directions have opposite margins, so at most one of them can be saturated: the difference has length at most about 1, not 2, which is why the noise part stops at half its budget. The Jacobian bound charges both the record that leaves and the one that arrives the full curvature c, while their eigenvalues $\lambda_1 \geq 0 \geq \lambda_2$ have opposite signs, so the product $(1 + \lambda_1)(1 + \lambda_2)$ is at most $1 + a$: the Jacobian needs only half its budget (Exercise 6 in the exercise book). The analysis is safe, not tight.

Curvature helps. Compare Equation 36.11 with Equation 36.7. In output perturbation the noise has length about $1/(\lambda n)$ and the damage is weighted by H; in objective perturbation the noise has length about 1, is divided by n, and the damage is weighted by H^{-1}. If every eigenvalue of H is close to h, the ratio of the two excess risks is about $(h/\lambda)^2$ (with $\varepsilon' \approx \varepsilon$). When the ridge supplies all the curvature, $h = \lambda$ and the two are equivalent. When the data supply more, objective perturbation wins quadratically.

What it needs. Exact minimization, twice-differentiable losses (the hinge loss of a support vector machine must be smoothed first), a bound c on the curvature, and a loss that depends on w through one inner product, which makes each record's Hessian rank one. Kifer, Smith & Thakurta (2012) extend it to Gaussian noise with (ε, δ)-DP and to constrained problems, and Iyengar et al. (2019) give a version that tolerates approximate minimization by adding a little output noise on top.

36.4.5. Example: the running example at $n = 10{,}000$

At $n = 10{,}000$ the Jacobian budget is $2\ln(1 + 0.25/100) = 0.0050$, so $\varepsilon' = 0.9950$: almost all of ε goes to the noise. (At $n = 100$ the split would be $\varepsilon' = 0.5537$, and the Jacobian would take almost half.) The noise has $\mathbb{E}\|b\|^2 = d(d + 1)(2/\varepsilon')^2$, and $\operatorname{tr} H^{-1} = 320.7$, so Equation 36.11 predicts an excess risk of 7.127×10^{-5}; 20 simulated releases average 8.156×10^{-5}. The pure-DP output

perturbation of Section 36.3, with the same ε, had 8.321×10^{-4}: objective perturbation is ten times better. The rule of thumb $(h/\lambda)^2$ predicts this: the objective's curvature, ridge included, lies between 0.0263 and 0.0330, about three times the ridge's 0.01, and $3^2 = 9$.

36.4.6. Check yourself

- Why does the proof need the minimizer to be unique? *(Step 1 inverts the map from noise to output. If two weights both minimized the perturbed objective, the output would not be a function of b, and Fact 4 would not apply.)*
- If you double n, what happens to the share of ε that the Jacobian consumes? *(It roughly halves: $2\ln(1 + c/(n\lambda)) \approx 2c/(n\lambda)$ for large n.)*

36.5. Gradient perturbation: noisy gradient descent

36.5.1. Context

The first two methods release one point and need the exact minimizer. Real training is iterative and stops early. Gradient perturbation, analysed by Song, Chaudhuri & Sarwate (2013) and Bassily, Smith & Thakurta (2014), makes each step of the optimizer private and composes the steps. Here is its simplest form, full-batch gradient descent with Gaussian noise. Its privacy argument uses nothing about convexity; convexity enters only when you ask how accurate it is.

36.5.2. Statement

Start from $w_0 = 0$ and run T steps of

$$w_{t+1} = w_t - \eta(\nabla J(w_t; D) + \xi_t), \qquad \xi_t \sim \mathcal{N}(0, \sigma^2 I) \text{ independent}, \qquad \sigma = \frac{2L}{n} \cdot \frac{\sqrt{T}}{\mu(\varepsilon, \delta)}. \tag{36.14}$$

Releasing all the iterates $w_1, \dots, w_T$ is (ε, δ)-DP for replace-one neighbours, for any loss whose per-record gradients have length at most L, convex or not.

For a λ-strongly convex, β-smooth objective (curvature between λ and β everywhere) and step $\eta = 1/\beta$, the excess risk of the last iterate is approximately

$$\mathbb{E}\big[J(w_T) - J(w^*)\big] \approx \Big(1 - \frac{\lambda}{\beta}\Big)^T \big(J(w_0) - J(w^*)\big) + \frac{d\sigma^2}{4\beta}, \tag{36.15}$$

an optimization error that shrinks with T plus a noise floor that grows with T, because $\sigma^2 \propto T$.

36.5.3. Derivation

1. **One step is a Gaussian mechanism.** Given w_t, the step releases $\nabla J(w_t; D) + \xi_t$. The ridge term λw_t is the same on both datasets, and the averages differ only in the replaced record, so the gradient moves by $\frac{1}{n}\|\nabla\ell(w_t, z) - \nabla\ell(w_t, z')\| \le 2L/n$. Its ratio is $\mu_{\text{step}} = 2L/(n\sigma)$. The point w_t was itself computed from earlier noisy gradients, but it is public at that moment, which is what adaptive composition allows.
2. **Compose.** By Fact 5, T such steps are exactly as private as one Gaussian with $\mu = \sqrt{T} \cdot 2L/(n\sigma)$. Setting this to $\mu(\varepsilon, \delta)$ gives the σ of Equation 36.14. The iterates are computed from the noisy gradients and w_0 alone, so releasing them is post-processing.
3. **Where convexity was not used.** Steps 1 and 2 bounded the gradient's change by $2L/n$ at the point w_t, whatever w_t is. Nothing required a minimizer, convexity or smoothness.
4. **Optimization error.** Without noise, gradient descent with step $1/\beta$ on a λ-strongly convex, β-smooth function shrinks $J(w_t) - J(w^*)$ by at least a factor $1 - \lambda/\beta$ per step (Optimization), which gives the first term.
5. **The noise floor.** Near the minimizer, the gradient is about $H(w_t - w^*)$, so the error $e_t = w_t - w^*$ evolves as $e_{t+1} \approx (I - \eta H)e_t - \eta\xi_t$. Along an eigenvector of H with curvature h, the error's variance v settles where one step leaves it unchanged: $v = (1 - \eta h)^2 v + \eta^2\sigma^2$, so $v = \eta^2\sigma^2/(1 - (1 - \eta h)^2) = \eta\sigma^2/(h(2 - \eta h))$. Its cost is $\frac{1}{2}hv = \eta\sigma^2/(2(2 - \eta h))$, about $\eta\sigma^2/4$ when ηh is small. Summing over the d directions with $\eta = 1/\beta$ gives $d\sigma^2/(4\beta)$, and the curvature h has cancelled: directions with little curvature wander farther but cost less per unit of wandering.

36.5.4. Interpretation

The step count is a privacy parameter. Every step spends budget, so the noise per step grows like $\sqrt{T}$ and the noise floor like T. Too few steps leave optimization error, too many drown the solution in noise. Figure 36.3 shows the trade-off on the running example. And you cannot pick the best T by trying several and keeping the one with the best validation score without paying for it: that selection is itself a computation on the data, the subject of Hyperparameter tuning.

A public rule for T. Equation 36.15 can be evaluated without looking at the data, using only public quantities: λ, the curvature bound $\beta = \frac{1}{4} + \lambda$ for logistic loss with unit features, $J(w_0) = \ln 2$ for $w_0 = 0$ (each loss is $\ln 2$ at $w = 0$, and $J(w^*) \ge 0$), d, n and the budget. The rule picks the T that minimizes it. It is pessimistic, because it only knows that the curvature is at least λ.

Why it is the one that generalizes. Gradient perturbation needs a gradient bound, not a minimizer. For a neural network no such bound holds, because gradients can be arbitrarily long, but you can force one by **clipping**: rescale each record's gradient to length at most C before averaging, exactly as Beyond global sensitivity clips a value before averaging it. That, with minibatches instead of full batches, is DP-SGD.

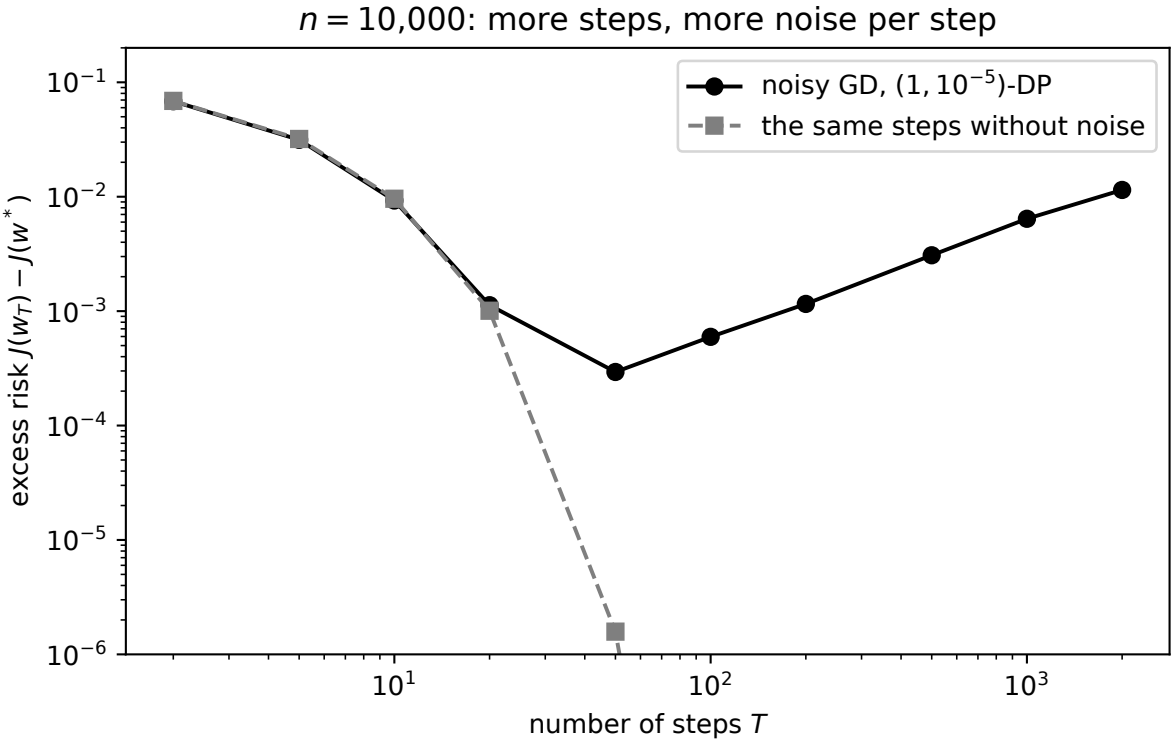


Figure 36.3.: Excess risk of noisy gradient descent at $n = 10{,}000$, $(\varepsilon, \delta) = (1, 10^{-5})$, against the number of steps T, averaged over datasets, with the same steps without noise (dashed). The noiseless error falls geometrically and reaches machine precision; the noisy run is limited by noise that grows with T, so its error is smallest near $T = 50$.

36.5.5. Example: choosing T for the running example

For $n = 10{,}000$, the public rule picks $T = 218$, which gives per-step noise $\sigma = 0.01102$ on the mean gradient and an estimated excess risk of 1.301×10^{-3}. Simulated runs average 1.384×10^{-3}, close to the estimate.

Figure 36.3 shows that the rule overshoots. With the actual curvature of the data, between 0.0263 and 0.0330 rather than the 0.01 the rule assumes, noiseless gradient descent is already within 2.679×10^{-11} of the minimum after 100 steps. Measured over 10 datasets, the excess risk is 1.027×10^{-3} at $T = 20$, 2.875×10^{-4} at $T = 50$, 7.137×10^{-4} at $T = 100$ and 6.018×10^{-3} at $T = 1{,}000$. The best T is near 50, four times smaller than the rule's, and gives five times less excess risk. You may only use that knowledge if you obtained it privately, or from public data that resembles yours.

Table 36.2.: Noisy gradient descent on the running example: the step count chosen by minimizing Equation 36.15 with public quantities only, the resulting noise, and the estimated excess risk.

n	Steps T (public rule)	Per-step σ on the mean gradient	Estimated excess risk
1,000	100	0.07461	6.725×10^{-2}
3,000	156	0.03106	1.080×10^{-2}
10,000	218	0.01102	1.301×10^{-3}
30,000	274	0.004117	1.779×10^{-4}
100,000	335	0.001366	1.930×10^{-5}

The step count grows slowly with n, like $\ln n$, because the optimization error shrinks geometrically while the target, the noise floor, shrinks like $1/n^2$ (Table 36.2).

36.5.6. Check yourself

- Noisy gradient descent with $T = 100$ steps is $(1, 10^{-5})$-DP. You run it for $T = 400$ steps with the same per-step noise. What is μ now? *(Twice as large: $\mu = \sqrt{T}\,\mu_{step}$, and $\sqrt{400/100} = 2$. So $\mu = 2 \times 0.2681 = 0.5362$, a weaker guarantee.)*
- In step 1 of the derivation, why is it allowed that the point w_t at which the gradient is evaluated depends on the data? *(Because w_t is computed from earlier released outputs only. Adaptive composition covers mechanisms chosen based on earlier outputs, so the bound holds conditionally on the past, at whatever w_t it produced.)*

36.6. How the three compare

36.6.1. Context

Each method is private by its own argument. Which should you use? Theory answers by rates, how the excess risk falls with n, d and ε, and by lower bounds that no method can beat. Simulation

answers for the running example. They agree on the rate and disagree on the constant, and the constant is what you choose by.

36.6.2. Statement

For λ-strongly convex objectives with L-Lipschitz losses, Bassily, Smith & Thakurta (2014) show that the best achievable expected excess risk under (ε, δ)-DP is

$$\Theta\Big(\frac{L^2\, d\, \ln(1/\delta)}{\lambda\, n^2 \varepsilon^2}\Big) \tag{36.16}$$

up to logarithmic factors in n, and that a version of noisy gradient descent attains it. For convex objectives over a ball of radius R, without strong convexity, the rate is $\Theta(LR\sqrt{d\ln(1/\delta)}/(n\varepsilon))$ up to logarithms. The lower bounds come from the fingerprinting codes of Lower bounds: a private algorithm whose output is too accurate would let an adversary trace which records were used.

On the running example, Table 36.3 gives the measured excess risk of the four private releases.

Table 36.3.: Mean excess risk $J(\tilde{w}) - J(w^*)$ over 20 datasets of the running example ($d = 10$, $\lambda = 0.01$, $\varepsilon = 1$, $\delta = 10^{-5}$ for the Gaussian methods), and the slope of a straight-line fit of log excess risk against $\log n$.

n	Output, Gaussian	Output, pure	Objective, pure	Noisy gradient descent (public T)
1,000	8.170×10^{-2}	7.060×10^{-2}	1.009×10^{-2}	6.558×10^{-2}
3,000	1.211×10^{-2}	7.206×10^{-3}	6.303×10^{-4}	8.888×10^{-3}
10,000	9.274×10^{-4}	8.321×10^{-4}	8.156×10^{-5}	1.384×10^{-3}
30,000	9.973×10^{-5}	8.943×10^{-5}	7.135×10^{-6}	1.538×10^{-4}
100,000	8.543×10^{-6}	6.379×10^{-6}	8.798×10^{-7}	1.621×10^{-5}
Slope in $\log n$	-2.01	-2.00	-2.01	-1.79

36.6.3. Derivation

The upper bounds for output and objective perturbation are Equation 36.7 and Equation 36.11 with the noise of each method substituted. Both have $\mathbb{E}\|\text{noise}\|^2 \propto 1/n^2$ (the output noise has scale $1/(\lambda n)$; the objective noise has scale 1 and is divided by n), which gives the $1/n^2$. For noisy gradient descent, the noise floor $d\sigma^2/(4\beta)$ has $\sigma^2 \propto T/n^2$, and the optimization error must be pushed below it, which takes $T \propto \frac{\beta}{\lambda}\ln n$ steps; this gives $1/n^2$ times a logarithm. The proofs of the optimal rate and of the lower bound are long; Bassily, Smith & Thakurta (2014) give both.

36.6.4. Interpretation

The rate is the same; the constants are not. All three methods fall like $1/n^2$, and noisy gradient descent a little slower, -1.79, because its step count grows with $\ln n$. On this problem objective perturbation is about ten times better than output perturbation at every n, the $(h/\lambda)^2$

of Section 36.4, and noisy gradient descent with the public step rule is the worst of the three for $n \geq 10{,}000$. Tuned privately to $T \approx 50$, it would sit between the two perturbation methods.

Pure against approximate DP. At $d = 10$ the pure-DP noise of Fact 3 and the Gaussian are close: their mean squared lengths are $d(d+1)s^2 = 110\,\Delta^2$ at $\varepsilon = 1$ and $d\sigma^2 = 10 \times 3.7306^2\,\Delta^2 = 139.2\,\Delta^2$, which is why the pure-DP column is slightly better here. At $d = 100$ they are $10{,}100\,\Delta^2$ against $1{,}392\,\Delta^2$: pure DP costs a factor that grows like d, the price of $\delta = 0$ that Lower bounds proves unavoidable.

Accuracy is forgiving. Excess risk is a fine measure, but you care about predictions. On held-out data the non-private model has accuracy 0.747 at $n = 1{,}000$; output perturbation drops it to 0.681, noisy gradient descent to 0.695 and objective perturbation only to 0.736. By $n = 10{,}000$ all four are within 0.002 of the non-private 0.753. A low-dimensional convex model on ten thousand records can be made private almost for free.

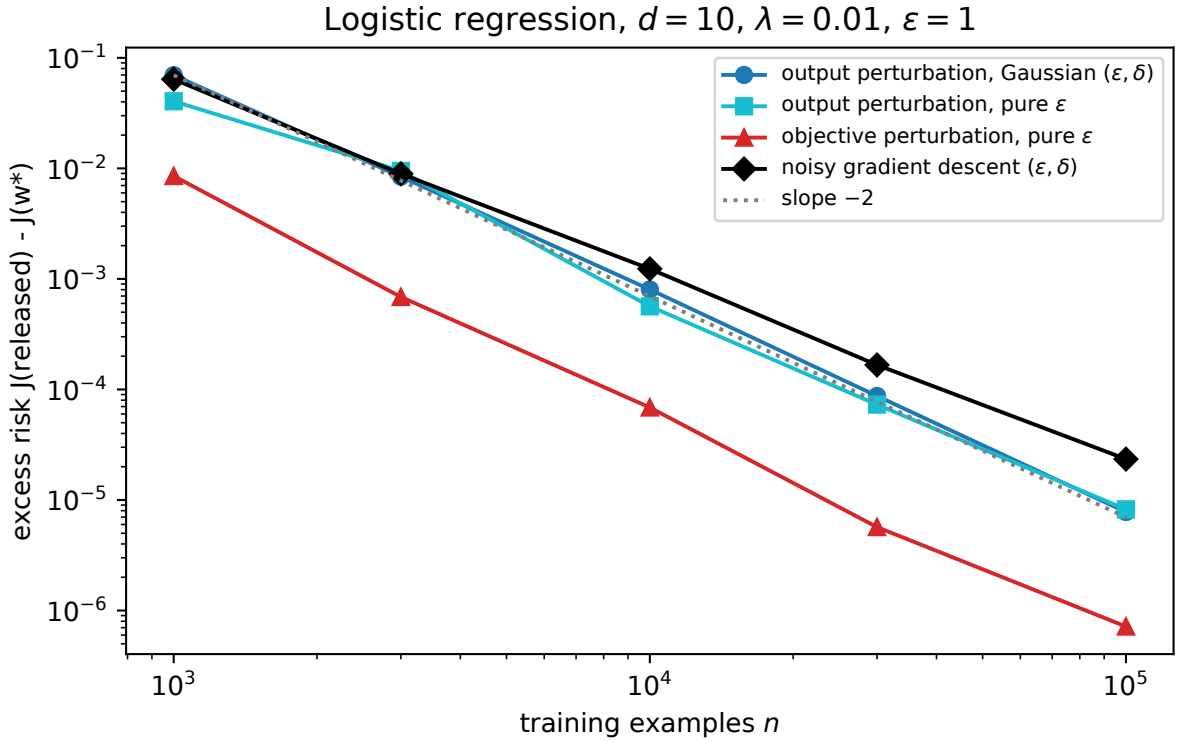


Figure 36.4.: Excess risk of the four private releases against the dataset size on the running example, log-log, with a line of slope -2. All fall like $1/n^2$; objective perturbation is about ten times better than output perturbation throughout.

36.6.5. Example: reading the rate

Equation 36.16 predicts how much data you need. Suppose an excess risk of 10^{-4} is acceptable, and output perturbation gives 9.274×10^{-4} at $n = 10{,}000$. The rate $1/n^2$ says that multiplying n by $\sqrt{9.274} = 3.05$ brings it to about 10^{-4}; Table 36.3 confirms it, with 9.973×10^{-5} at $n = 30{,}000$.

Equivalently, halving ε costs the same as halving n, because n and ε appear only as the product $n\varepsilon$: to keep the same accuracy at half the budget, you need twice the data.

36.6.6. Check yourself

- In Equation 36.16, why do n and ε appear only as the product $n\varepsilon$? *(Every sensitivity in this chapter is proportional to $1/n$ and every noise to sensitivity over ε, so the noise scale is proportional to $1/(n\varepsilon)$.)*
- Objective perturbation beat output perturbation by a factor of ten. Name a problem on which it would not help at all. *(Any problem whose data add no curvature beyond the ridge, such as a linear loss: then $H = \lambda I$, $h = \lambda$, and the ratio $(h/\lambda)^2$ is 1.)*

36.7. When the model is not convex

36.7.1. Context

A neural network's loss is not convex in its weights. What survives of the three methods?

36.7.2. Statement

Output perturbation and objective perturbation have no general privacy guarantee for non-convex objectives: both need a unique minimizer that depends continuously on the data. Noisy gradient descent keeps its guarantee, Equation 36.14, for any loss whose per-record gradients have length at most L, and loses only its accuracy guarantee. When no such L exists, clipping each per-record gradient to length C makes $L = C$.

36.7.3. Derivation

1. **The minimizer can jump.** Optimization gives a one-dimensional loss with two wells, $\frac{1}{4}(w^2-1)^2-yw$ with labels $y = \pm 1$, whose empirical minimizer sits in the well favoured by the majority label. With 501 labels $+1$ and 500 labels -1, changing one label flips the majority and moves the minimizer by 2.0010. No $1/n$ appears: the sensitivity of the minimizer is a constant, so output perturbation would need noise with $\sigma = 3.7306 \times 2.0010 = 7.465$ at $\varepsilon = 1$, more than three times the distance between the wells, whatever the dataset size.
2. **The objective-perturbation proof breaks at step 1.** Without strong convexity the perturbed objective can have several minimizers, the map from noise to output is no longer invertible, and Fact 4 cannot be applied.
3. **Gradient perturbation survives.** Its derivation in Section 36.5 used only the gradient bound at the visited points (step 3 there).
4. **Clipping supplies the bound.** Replace each per-record gradient $\nabla\ell(w, z_i)$ by $\nabla\ell(w, z_i)\min(1, C/\|\nabla\ell(w, z_i)\|)$, which has length at most C and is unchanged when it is already shorter. The average of clipped gradients changes by at most $2C/n$ when one record is replaced, and the derivation goes through with $L = C$.

36.7.4. Interpretation

What the non-convex case loses is the guarantee of accuracy, not of privacy. Noisy gradient descent on a neural network is private whatever the network does; whether it learns anything is an empirical question, and the answer depends on the clipping norm, the noise and the number of steps, which the next chapter studies. Clipping also changes what is being optimized: the average of clipped gradients is not the gradient of J, so the algorithm no longer minimizes J, a bias that the next chapter measures.

Full-batch gradient descent is also too slow for large datasets: each step touches every record. Using a random minibatch at each step makes each step cheaper and, by Amplification by subsampling, more private per step, because each record is used only with probability q. Clipping, noise, minibatches and a subsampled-Gaussian accountant together are DP-SGD.

36.7.5. Example: two wells and output perturbation

Take the double-well loss with $n = 1{,}001$, whose minimizer sits in one of the two wells near ± 1. Output perturbation with $\sigma = 7.465$ releases a weight whose standard deviation is more than seven times the distance from either well to the origin; the released value carries almost no information about which well the data chose, which is exactly what privacy demands, since one record decides it. Noisy gradient descent with clipping still moves toward whichever well the noisy average gradient points to; on datasets where many records agree, the average gradient is large compared with the noise and the method finds the majority well with high probability.

36.7.6. Check yourself

- Why does clipping not break the privacy proof, even though it changes the gradients? *(The proof needs only that each step releases a function of the data with bounded sensitivity plus Gaussian noise. The clipped average is such a function, with sensitivity $2C/n$, whatever it is the gradient of.)*
- Does noisy gradient descent on a non-convex loss need to find the global minimum to be private? *(No. Privacy holds for every run, whatever point it reaches; finding a good point is a question of accuracy only.)*

36.8. Code: check every number by computation

The script code/dp_erm.py implements the three methods for regularized logistic regression with exact Newton solves, computes the exact output densities of objective perturbation in one dimension, and runs the comparison. From the repository root:

```
uv run python code/part05-dp-ml/dp_erm.py
```

Its public functions are used by later chapters of this part: `make_logistic_data`, `fit_logistic`, `logistic_objective`, `logistic_grad`, `excess_risk`, `gaussian_delta(eps, mu)`, `gaussian_mu(eps, delta)` and `gaussian_sigma(sensitivity, eps, delta)`.

Output (seed 0), summarized in Table 36.4:

Table 36.4.: The script's output with the default seed.

Experiment	Result
Gaussian calibration, $\varepsilon = 1$, $\delta = 10^{-5}$	$\mu = 0.2681$; exact $\sigma = 3.7306\,\Delta$ against classic $4.8448\,\Delta$
Output perturbation, one dimension	Minimizers 0.0832 apart, bound 0.2
Output perturbation, $n = 10{,}000$	Predicted excess risk 8.711×10^{-4}, measured 9.274×10^{-4}
Approximate minimizer, $g = 10^{-3}$	Sensitivity 0.22 against 0.02 exact
Objective perturbation, one dimension, 500 datasets	Largest log ratio 0.3779 against $\varepsilon = 1$; noise part 0.0992 of 0.1891, Jacobian part 0.3395 of 0.8109
Objective perturbation, $n = 10{,}000$	Predicted 7.127×10^{-5}, measured 8.156×10^{-5}
Noisy gradient descent, $n = 10{,}000$	Public rule $T = 218$, measured 1.384×10^{-3}; best $T \approx 50$ gives 2.875×10^{-4}
Slopes in $\log n$	-2.01, -2.00, -2.01, -1.79

tests/test_dp_erm.py checks that the output still matches.

What to take from it. Three things. First, the privacy proofs can be checked numerically where the output distribution is computable: in one dimension the change-of-variables density is exact, and the log ratio stays inside its bound with room to spare. Second, the second-order predictions of excess risk match simulation to within the sampling error, so you can predict the cost of privacy before training. Third, all methods have the same $1/n^2$ rate on this problem, and the constant, a factor of ten here, comes from how each method uses the data's curvature.

36.9. References

- Chaudhuri & Monteleoni, "Privacy-preserving logistic regression" (NeurIPS 2008). Output and objective perturbation for logistic regression, the first private ERM algorithms.
- Chaudhuri, Monteleoni & Sarwate, "Differentially Private Empirical Risk Minimization" (JMLR 2011). The general analysis of both: output perturbation (Algorithm 1), objective perturbation (Algorithm 2) and their excess-risk bounds. The reference for Section 36.3 and Section 36.4.
- Kifer, Smith & Thakurta, "Private Convex Empirical Risk Minimization and High-dimensional Regression" (COLT 2012). Objective perturbation with Gaussian noise, constraints and non-smooth losses.
- Song, Chaudhuri & Sarwate, "Stochastic gradient descent with differentially private updates" (IEEE GlobalSIP 2013). Gradient perturbation for SGD, with minibatches.
- Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014). Noisy gradient methods that attain the optimal rates, and the matching lower bounds; the reference for Section 36.6.

- Wu, Li, Kumar, Chaudhuri, Jha & Naughton, “Bolt-on Differential Privacy for Scalable Stochastic Gradient Descent-based Analytics” (SIGMOD 2017). Output perturbation for SGD run for a fixed number of passes, without an exact minimizer.
- Iyengar, Near, Song, Thakkar, Thakurta & Wang, “Towards Practical Differentially Private Convex Optimization” (IEEE S&P 2019). Approximate minima perturbation, an objective perturbation that tolerates inexact solvers, and a benchmark of all three methods.
- Balle & Wang, “Improving the Gaussian Mechanism for Differential Privacy: Analytical Calibration and Optimal Denoising” (ICML 2018). The exact calibration of Fact 1.
- Dong, Roth & Su, “Gaussian Differential Privacy” (JRSS-B 2022). The composition of Fact 5.

36.10. Cheat sheet

n records, ridge λ, per-record gradients of length at most L, replace-one neighbours, $\mu(\varepsilon,\delta)$ the largest admissible Gaussian ratio (0.2681 at $\varepsilon=1$, $\delta=10^{-5}$), H the Hessian at the minimizer. Table 36.5 collects the results.

Table 36.5.: Every result of the chapter.

Result	Formula	Needs	Guarantee
Sensitivity of the minimizer	$2L/(\lambda n)$	λ-strong convexity, exact minimizer	—
Approximate minimizer	$2L/(\lambda n)+2g/\lambda$	$\lVert\nabla J\rVert\le g$ checked at stopping	—
Output perturbation	$w^*+\mathcal{N}(0,\sigma^2 I)$, $\sigma=2L/(\lambda n\mu)$	As above	(ε,δ)-DP
Output, pure	w^*+b, $\nu(b)\propto e^{-\lVert b\rVert\lambda n\varepsilon/(2L)}$	As above	ε-DP
Excess risk, output	$\approx \mathbb{E}\lVert Z\rVert^2\operatorname{tr}H/(2d)$	Second order	—
Objective perturbation	$\arg\min J+\langle b,w\rangle/n$, $\nu(b)\propto e^{-\varepsilon'\lVert b\rVert/2}$, $\varepsilon'=\varepsilon-2\ln(1+c/(n\lambda))$	Loss $g(y\langle w,x\rangle)$, $\lvert g'\rvert\le 1$, $0\le g''\le c$, $\lVert x\rVert\le 1$, exact minimizer	ε-DP
Excess risk, objective	$\approx \mathbb{E}\lVert b\rVert^2\operatorname{tr}H^{-1}/(2dn^2)$	Second order	—
Noisy gradient descent	$\sigma=(2L/n)\sqrt{T}/\mu$ on the mean gradient	Per-record gradients $\le L$ (or clipped)	(ε,δ)-DP, all iterates
Excess risk, noisy gradient descent	$\approx(1-\lambda/\beta)^T(J(w_0)-J^*)+d\sigma^2/(4\beta)$	Strong convexity, smoothness, $\eta=1/\beta$	—
Optimal rate	$\tilde{\Theta}(L^2 d\ln(1/\delta)/(\lambda n^2\varepsilon^2))$	Strongly convex	Bassily, Smith & Thakurta (2014)

Rules of thumb

- All three methods give excess risk falling like $1/(n\varepsilon)^2$ on strongly convex problems. Halving ε costs as much as halving the data.
- Output perturbation pays for the ridge's curvature, objective perturbation benefits from the data's. If the data add curvature h beyond λ, expect objective perturbation to win by about $(h/\lambda)^2$.
- Output and objective perturbation need the exact minimizer; stop on a gradient tolerance well below L/n, never on a step count.
- The number of steps of noisy gradient descent is part of the privacy budget. Choose it from public quantities, or pay for choosing it.
- Only gradient perturbation is private without convexity. Clip per-record gradients to get the bound it needs.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

37. DP-SGD

37.1. Why this matters

Private empirical risk minimization ended with one method left standing for non-convex models: gradient descent with Gaussian noise on every step. Its privacy proof needs only that one record moves each step's gradient by a bounded amount, and clipping each record's gradient supplies that bound for any model. Two more changes make it practical for differential privacy (DP). Each step uses a random minibatch instead of the full dataset, which makes steps cheap and, by amplification, more private. And an accountant composes thousands of such steps into one (ε, δ). The result is stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, introduced by Abadi et al. (2016). Nearly every private deep-learning system runs it, through libraries such as Opacus, TensorFlow Privacy and JAX Privacy.

Part 4 computed the privacy of DP-SGD without running it: Amplification by subsampling and Choosing an accountant report $\varepsilon = 6.71$ at $\delta = 10^{-5}$ for sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and 10,000 steps. This chapter runs that configuration on a neural network, and asks what the noise and the clipping cost and how to choose the knobs that control them.

37.1.1. The one idea: the analysis never looks inside the model

Each step of DP-SGD releases a noisy **sum of clipped per-example gradients**. Clipping rescales each record's gradient to length at most C, so adding or removing one record changes the sum by at most C, whatever the network, the loss or the current weights. The privacy proof stops there. Everything else, the architecture, the data, whether training converges, affects only accuracy.

That split organizes the chapter. The knobs divide into two groups. The noise multiplier σ, the sampling rate q and the number of steps T determine ε through the accountant, and nothing else does. The clipping norm C and the step size η do not change ε at all; they change how much of the gradient survives clipping and how much the noise moves the weights. The cost of privacy comes in two forms: a **bias** from clipping, which changes the direction of the average gradient, and a **variance** from the noise, which makes each step partly random.

Table 37.1 lists the results and where each is used.

Table 37.1.: The results of this chapter and where each is used.

Result	Where it is used
The algorithm and its (ε, δ) guarantee	Every chapter of this part; what to check in an implementation

Result	Where it is used
Clipping: the expected update, scale invariance and the choice of C	Tuning DP-SGD; disparate impact
Batch size at a fixed budget	Choosing q and T; large-batch private training
Per-example gradient norms without per-example gradients	Implementing DP-SGD at scale; public data and fine-tuning
Privacy against accuracy on the running example	The baseline every later chapter improves on

Notation. A training set D has n records $z_i = (x_i, y_i)$. The model has p weights w, and $g_i = \nabla_w \ell(w, z_i)$ is the gradient of record i's loss, its **per-example gradient**. C is the clipping norm, σ the noise multiplier (the noise standard deviation in units of C), q the sampling rate, $B = qn$ the expected batch size, T the number of steps and η the step size. Neighbouring datasets now differ by **adding or removing** one record, $D' = D \cup \{x\}$, because Poisson sampling pairs with that relation (Amplification by subsampling); the previous chapter used replacement.

The running example. Inputs are 50-dimensional standard normal vectors, and labels are the sign of the output of a fixed random **teacher** network with 8 hidden tanh units, so a perfect classifier exists. The **student** is a network with one hidden layer of 64 tanh units and a logistic loss, $p = 3{,}329$ weights including biases, trained on $n = 10{,}000$ records and tested on 20,000 more. DP-SGD runs with $q = 0.01$ (expected batch $B = 100$), $C = 0.5$, $\eta = 0.5$, $\sigma = 1$ and $T = 10{,}000$ steps, 100 epochs: exactly Part 4's running example. The dataset is fixed; only the training randomness changes between runs.

How to read this chapter. Read the toolkit and Section 37.3, which states what the guarantee covers and what it assumes. Section 37.4 and Section 37.5 say how to set the knobs. Section 37.6 is about implementation and can be skipped on a first reading. Section 37.7 puts the pieces together.

37.2. Toolkit: four facts every derivation uses

37.2.1. Fact 1 — A Gaussian mechanism is described by one ratio

Add noise $\mathcal{N}(0, s^2 I)$ to a vector-valued function whose value moves by at most Δ in Euclidean length between neighbours. Its privacy depends only on $\mu = \Delta/s$, the shift measured in noise standard deviations, because rotating the space puts the shift on one axis and leaves identical noise on the others (The Laplace and Gaussian mechanisms). For a clipped sum with noise σC, $\Delta = C$ and $\mu = 1/\sigma$: the clipping norm cancels.

37.2.2. Fact 2 — The Poisson-subsampled Gaussian and its accountant

If each record enters the batch independently with probability q, one step's output with the record is a mixture: with probability $1 - q$ the record was not drawn and the output is the same as without it, with probability q it was drawn and the output is shifted. In the worst case the

two distributions to compare are the mixture $(1-q)\mathcal{N}(0,\sigma^2)+q\mathcal{N}(1,\sigma^2)$ and $\mathcal{N}(0,\sigma^2)$, in one dimension. Its Rényi differential privacy (RDP) curve, the Rényi divergence of order α between them, has for integer α the exact form

$$\varepsilon_{q,\sigma}(\alpha)=\frac{1}{\alpha-1}\ln\sum_{j=0}^{\alpha}\binom{\alpha}{j}(1-q)^{\alpha-j}q^{j}e^{(j^2-j)/(2\sigma^2)}, \tag{37.1}$$

and a numerical integral for other orders (Amplification by subsampling). RDP curves add under composition, even when each step depends on the previous outputs (Rényi DP), and the composed curve converts to (ε,δ) at the best order,

$$\varepsilon=\min_{\alpha}\left[T\varepsilon_{q,\sigma}(\alpha)+\ln\frac{\alpha-1}{\alpha}-\frac{\ln\delta+\ln\alpha}{\alpha-1}\right] \tag{37.2}$$

(Balle et al. 2020; From Rényi DP to (ε, δ)). For small q and orders that are not too large, only the $j=2$ term of Equation 37.1 matters beyond the first two, and the curve is close to $\frac{\alpha}{2}q^2(e^{1/\sigma^2}-1)$.

37.2.3. Fact 3 — Post-processing

Any computation on a private output, which does not touch the data again, is as private as the output itself (Post-processing, group privacy and composition). Dividing a noisy sum by a public constant, taking a gradient step with it, and releasing the resulting weights are all post-processing.

37.2.4. Fact 4 — A dense layer's gradient is an outer product

For a dense layer that maps an input $a\in\mathbb{R}^{p_{\text{in}}}$ to Wa, the gradient of a loss with respect to W is $\delta a^{\top}$, where δ is the gradient with respect to the layer's output, the **back-propagated error**. Each entry is $\partial\ell/\partial W_{jk}=\delta_j a_k$, because W_{jk} multiplies a_k and contributes to output j only. Gradient inversion uses the same fact to reconstruct inputs from gradients; here it makes per-example gradient norms cheap.

37.3. The algorithm and its guarantee

37.3.1. Context

What exactly does DP-SGD compute, and which (ε,δ) does it satisfy? The answer has to be precise, because each detail of the algorithm is an assumption of the proof, and implementations that change a detail can lose the guarantee without any visible symptom.

37.3.2. Statement

Start from weights w_0 chosen independently of the data. For $t = 0, \dots, T-1$:

1. **Sample.** Put each record in the batch S_t independently with probability q.
2. **Per-example gradients.** For each $i \in S_t$, compute $g_i = \nabla_w \ell(w_t, z_i)$.
3. **Clip.** Rescale each to length at most C: $\bar{g}_i = g_i \min(1, C/\|g_i\|)$.
4. **Noisy sum.** $G_t = \sum_{i \in S_t} \bar{g}_i + \xi_t$ with $\xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I)$.
5. **Step.** $w_{t+1} = w_t - \eta G_t / B$, with $B = qn$ the expected batch size.

Releasing all of $w_1, \dots, w_T$ is (ε, δ)-DP for add/remove neighbours, with ε given by Equation 37.2. The guarantee does not depend on the loss, the model, C or η.

37.3.3. Derivation

1. **One record moves the clipped sum by at most C.** Fix the weights w_t and the batch of the other records. If the record x is in the batch, the sum contains one extra term $\bar{g}_x$, of length at most C; every other term is the same for D and D', because each is computed from its own record alone.
2. **One step is a subsampled Gaussian mechanism.** With noise σC, Fact 1 gives the ratio $1/\sigma$ for a step on which x is drawn. Under Poisson sampling, x is drawn with probability q independently of the other records, so the step's output on D' is the mixture of Fact 2. Rescaling by C and rotating, the worst case over the direction and length of $\bar{g}_x$ is the one-dimensional pair of Fact 2, a shift of exactly C along one axis; a shorter shift is the same pair with extra noise, which cannot be less private.
3. **Dividing by B is post-processing.** $B = qn$ is a fixed public number, so w_{t+1} is computed from w_t and G_t alone (Fact 3).
4. **Compose.** Step t is evaluated at w_t, which depends on the earlier noisy sums. RDP composition holds for such adaptive sequences, so the T steps together have RDP curve $T\varepsilon_{q,\sigma}(\alpha)$.
5. **Convert.** Equation 37.2 turns the composed curve into (ε, δ). Every iterate is a function of the released noisy sums, so releasing all of them, or only the last, is covered.

37.3.4. Interpretation

What the proof assumes, and how implementations break it. Each step of the derivation is a condition on the code.

- *Step 1 needs per-example independence.* Batch normalization computes means and variances over the batch, so one record changes every other record's activations and gradients, and the sum can move by much more than C. Private models use normalizations that act on one example at a time, such as group or layer normalization.
- *Step 1 needs a fixed C.* Choosing C as, say, the median gradient norm of the current batch makes the noise scale depend on the data. Choose C in advance, from public data or a private estimate.

- *Step 2 needs Poisson sampling.* Shuffling the data and cutting it into fixed batches is a different mechanism, which Amplification by subsampling shows can be much less private than the Poisson accountant reports.
- *Step 3 needs a public denominator.* Dividing by the actual batch size $|S_t|$, which reveals how many records were drawn and changes when x is added, is not post-processing of G_t. Divide by B, or release a noisy count and spend budget on it.
- *Step 4 needs the accountant to see every step*, including the steps of runs that were discarded during tuning: that is the subject of Hyperparameter tuning.

What it does not cover. The guarantee is for one record. A person with many records needs user-level privacy. Preprocessing that uses the data, such as normalizing features by their mean, must itself be private or use public statistics.

37.3.5. Example: the running example

With $q = 0.01$, $\sigma = 1$, $T = 10{,}000$ and $\delta = 10^{-5}$, the accountant gives $\varepsilon = 6.71$, attained at order $\alpha = 4.1$, the same value Part 4 reports. Table 37.2 gives the noise multiplier that the same q and T need for other budgets.

Table 37.2.: The noise multiplier that DP-SGD with $q = 0.01$ and $T = 10{,}000$ steps needs for each budget, from the RDP accountant.

ε at $\delta = 10^{-5}$	0.5	1	2	4	6.71	8
Noise multiplier σ	7.72	4.13	2.28	1.36	1	0.917

How large is the noise? The averaged update G_t/B carries noise with standard deviation $\sigma C/B = 1 \times 0.5/100 = 0.005$ on each of the 3,329 coordinates. A Gaussian vector with p independent coordinates of standard deviation s has length close to $s\sqrt{p}$, so the noise has length about $0.005 \times \sqrt{3{,}329} = 0.288$. The averaged clipped gradient has length at most $C = 0.5$, and in the run it is 0.070 at the first step and between 0.015 and 0.027 from step 1,000 on. The noise in every step is about ten times longer than the signal.

Training works anyway, because the signal points in roughly the same direction from one step to the next, while the noise is independent. Over k steps, the signals add up to about k times their length, and the noise to about $\sqrt{k}$ times its length, since independent vectors add in squares. Signal overtakes noise when $k \cdot 0.026 > \sqrt{k} \cdot 0.288$, that is after $k > (0.288/0.026)^2 \approx 123$ steps. The final model reaches test accuracy 0.9589, against 0.9627 for non-private SGD.

37.3.6. Check yourself

- A colleague's implementation divides each noisy sum by the number of records actually in the batch. Which step of the derivation fails? *(Step 3: the divisor depends on the data, so the update is no longer a function of the noisy sum alone, and the accountant's analysis does not apply to it.)*
- Does changing C from 0.5 to 5 change ε? *(No. The sum's sensitivity and the noise both scale with C, so their ratio $1/\sigma$ and the accountant's input are unchanged. It changes accuracy.)*

37.4. Clipping: bias, noise and the choice of C

37.4.1. Context

The clipping norm does not affect privacy, but it controls both forms of cost. Small C clips more gradients, which biases the update; large C adds more noise, because the noise is σC. What does the update estimate, and how should C be chosen?

37.4.2. Statement

Averaged over the sampling and the noise, the update direction is a **reweighted gradient**:

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \frac{1}{n}\sum_{i=1}^{n} \min\Big(1, \frac{C}{\|g_i\|}\Big) g_i, \tag{37.3}$$

the true average gradient $\frac{1}{n}\sum_i g_i$ with each record weighted by $\min(1, C/\|g_i\|)$. Its noise has standard deviation $\sigma C/B$ per coordinate.

Scale invariance. If every per-example gradient in a run is clipped, that is $\|g_i\| \geq C$ at every step, then the update is

$$w_{t+1} = w_t - \frac{\eta C}{B}\Big(\sum_{i \in S_t} \frac{g_i}{\|g_i\|} + \sigma\zeta_t\Big), \qquad \zeta_t \sim \mathcal{N}(0, I), \tag{37.4}$$

which depends on η and C only through their product ηC.

37.4.3. Derivation

1. **Average over the sampling.** Each record is in the batch with probability q, so $\mathbb{E}\sum_{i \in S_t} \bar{g}_i = \sum_{i=1}^{n} q\, \bar{g}_i$. The noise has mean zero.
2. **Divide by $B = qn$.** The q cancels and leaves $\frac{1}{n}\sum_i \bar{g}_i$, which is Equation 37.3 by the definition of $\bar{g}_i$. This is why the denominator is the expected batch size: with it, the update is an unbiased estimate of the average clipped gradient.
3. **The noise.** ξ_t/B has standard deviation $\sigma C/B$ per coordinate.
4. **Scale invariance.** If $\|g_i\| \geq C$, then $\bar{g}_i = C g_i/\|g_i\|$. Write the noise as $\sigma C \zeta_t$ with ζ_t standard normal. Then $G_t = C(\sum_i g_i/\|g_i\| + \sigma\zeta_t)$, and $\eta G_t/B$ is Equation 37.4. Two runs with the same ηC, the same seed and every gradient clipped produce the same weights.

37.4.4. Interpretation

Below the gradient norms, only ηC matters. When C is smaller than every per-example gradient, DP-SGD is normalized SGD, which averages unit vectors, plus noise, and its behaviour depends on the product ηC. Lowering C further changes nothing once η is scaled up to match. This is why practitioners often clip aggressively and tune only the step size; De et al. (2022) report that small clipping norms work well, which this explains.

Above them, noise grows and signal does not. Once C exceeds most gradient norms, the clipped average stops growing with C, but the noise $\sigma C/B$ keeps growing in proportion. The signal-to-noise ratio falls like $1/C$. A large C is never better than the smallest C that clips almost nothing.

The bias reweights records, not uniformly. Equation 37.3 gives each record a weight that falls as its gradient grows. Records the model fits badly, such as rare examples, members of small groups or mislabelled points, have large gradients and are down-weighted, while well-fitted records keep their full weight. The update is not the gradient of the training loss but of a reweighted one, whose weights change as training proceeds. Disparate impact measures who pays for this.

Gradient norms change during training. In the DP-SGD run of the running example the median per-example gradient norm is 3.592 at the first step, 0.006 at step 1,000, and rounds to 0.000 from step 3,000 on; non-private training behaves the same way (Figure 37.1, right). The logistic loss flattens on records that are classified with a wide margin, so their gradients vanish. With $C = 0.5$, every gradient is clipped at the start, and between 9% and 25% of the batch later: the few records still near the decision boundary. A fixed C is in the scale-invariant regime early and in the unbiased regime late.

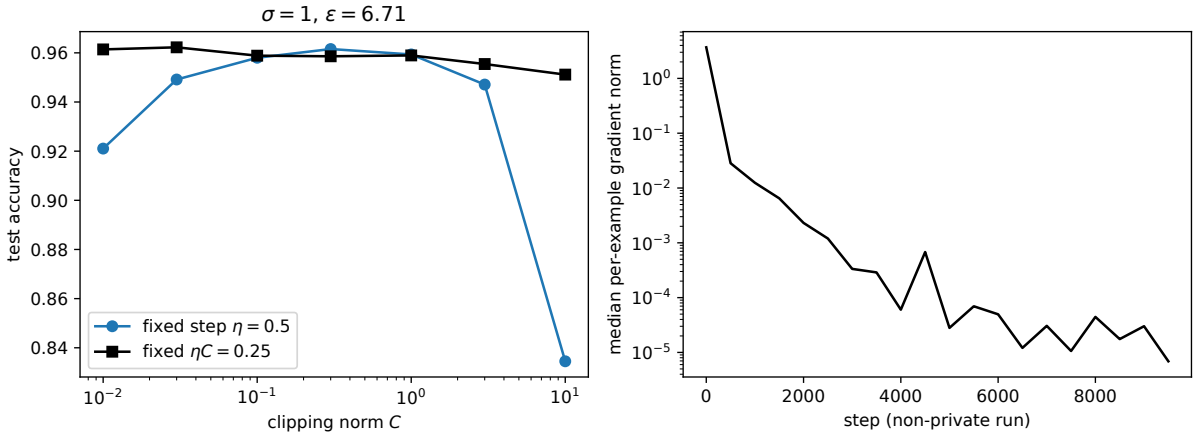


Figure 37.1.: Left: test accuracy of the running example at $\sigma = 1$ ($\varepsilon = 6.71$) against the clipping norm, with the step size fixed at $\eta = 0.5$ (circles) and with ηC fixed at 0.25 (squares). Right: the median per-example gradient norm during non-private training, on a log scale; it falls by about five orders of magnitude.

37.4.5. Example: sweeping C

Table 37.3 trains the running example at $\varepsilon = 6.71$ with seven clipping norms, once with the step size fixed at 0.5 and once with ηC fixed at $0.5 \times 0.5 = 0.25$.

Table 37.3.: Test accuracy of the running example at $\sigma = 1$ for seven clipping norms, one run each. With ηC fixed, small clipping norms all give the same accuracy.

C	0.01	0.03	0.1	0.3	1	3	10
Fixed $\eta = 0.5$	0.9244	0.9510	0.9567	0.9607	0.9583	0.9523	0.8347
Fixed $\eta C = 0.25$	0.9627	0.9623	0.9605	0.9599	0.9590	0.9558	0.9494

With the step size fixed, a small C is too slow (0.9244 at $C = 0.01$: every step has length proportional to $\eta C = 0.005$) and a large one too noisy (0.8347 at $C = 10$). With ηC fixed, accuracy is flat for $C \leq 0.3$, as scale invariance predicts, and falls slowly for large C, where the noise grows while the clipped gradients do not. The best accuracy, 0.9627 at $C = 0.01$, equals the non-private model's.

37.4.6. Check yourself

- You double C and halve η. When is the training run unchanged, and when not? *(Unchanged in distribution if every gradient is clipped under both values of C, by Equation 37.4. If some gradients are shorter than the new C, they now keep more of their length relative to the others, and the update changes.)*
- Why does dividing by B rather than $|S_t|$ make the expected update unbiased for the clipped average? *(Because $\mathbb{E}\sum_{i\in S_t} \bar{g}_i = q\sum_i \bar{g}_i$, and dividing by qn gives exactly the average. Dividing by $|S_t|$ is a ratio of random quantities, whose expectation is not the ratio of expectations.)*

37.5. Batch size and the number of steps at a fixed budget

37.5.1. Context

Suppose the privacy budget (ε, δ) and the compute budget, a number of epochs E, are fixed. Each epoch is $1/q$ steps, so $T = E/q$. Should you take many small steps or few large ones? The accountant answers how much noise each choice needs, and the noise per averaged gradient decides how useful each step is.

37.5.2. Statement

In the regime where the small-q approximation of Fact 2 holds, keeping ε fixed while changing q at a fixed number of epochs requires

$$\sigma \propto \sqrt{q}, \qquad \text{so} \qquad \frac{\sigma C}{B} \propto \frac{C}{\sqrt{qn}\,\sqrt{n}} = \frac{C}{\sqrt{Bn}} : \tag{37.5}$$

the noise on the averaged gradient falls like $1/\sqrt{B}$. Outside that regime, at small σ, small batches need even more noise than Equation 37.5 says, so large batches do better than $1/\sqrt{B}$.

37.5.3. Derivation

1. **Per-step cost.** By Fact 2, for small q the per-step RDP is about $\frac{\alpha}{2}q^2(e^{1/\sigma^2}-1)$, and for σ well above 1, $e^{1/\sigma^2}-1 \approx 1/\sigma^2$, so about $\frac{\alpha q^2}{2\sigma^2}$.
2. **Total cost.** $T = E/q$ steps add up to $\frac{E}{q}\cdot\frac{\alpha q^2}{2\sigma^2} = \frac{\alpha E q}{2\sigma^2}$. The conversion of Equation 37.2 depends only on this curve, so ε stays fixed when q/σ^2 does, that is $\sigma \propto \sqrt{q}$.
3. **Noise per averaged gradient.** $\sigma C/B$ with $B = qn$ and $\sigma \propto \sqrt{q}$ is proportional to $\sqrt{q}/q = 1/\sqrt{q}$, which is Equation 37.5.
4. **Small σ.** When σ is near or below 1, $e^{1/\sigma^2}-1$ is much larger than $1/\sigma^2$: the per-step cost is higher than step 1 says, by the factor $\sigma^2(e^{1/\sigma^2}-1)$. Small batches are in this regime, because they need small σ, so they pay this factor and need more noise than the $\sqrt{q}$ law predicts.

37.5.4. Interpretation

Amplification is most efficient at large σ. The ratio $\sigma/\sqrt{q}$, which Equation 37.5 says is constant, falls from 22.3 at $B = 10$ to 10.8 at $B = 10{,}000$ in Table 37.4. The factor $\sigma^2(e^{1/\sigma^2}-1)$ explains most of it: 3.23 at $\sigma = 0.704$, 1.33 at $\sigma = 1.36$ and 1.04 at $\sigma = 3.78$. Steps with little noise are expensive per unit of signal, because in the tail of the privacy loss, the high orders that the conversion uses, sampling buys little (Amplification by subsampling).

Large batches help, but not for free. For a fixed number of epochs, large batches give less noise per step and fewer steps. Fewer steps means less progress if the step size cannot grow in proportion, and the best step size grows with the batch: in Table 37.5 the best step size goes from 0.25 at $B = 30$ to between 1 and 2 at $B = 100$ to $B = 1{,}000$. De et al. (2022) and Anil et al. (2022) train with batches of thousands to hundreds of thousands, and report that large batches are a main ingredient of accurate private models. Large batches also parallelize well, which matters more on accelerators than in this small example.

37.5.5. Example: one budget, seven batch sizes

Table 37.4 fixes $\varepsilon = 4$, $\delta = 10^{-5}$ and 100 epochs on $n = 10{,}000$ records, and finds the noise multiplier for each sampling rate. For full batches, $q = 1$, there is no sampling and the 100 Gaussian steps compose exactly, like a single Gaussian with ratio $\sqrt{100}/\sigma$ (f-DP and Gaussian DP).

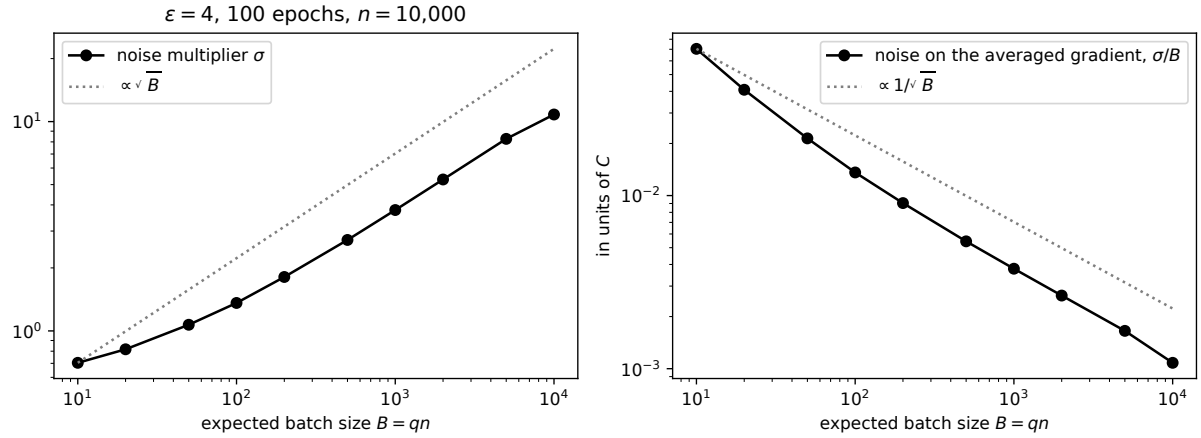


Figure 37.2.: DP-SGD at $\varepsilon = 4$, $\delta = 10^{-5}$, for 100 epochs on $n = 10{,}000$ records, as a function of the expected batch size. Left: the noise multiplier the accountant requires, against the $\sqrt{B}$ law of Equation 37.5. Right: the noise standard deviation on the averaged gradient, in units of C, against the $1/\sqrt{B}$ law. Small batches need more noise than the law predicts.

Table 37.4.: Noise needed for $\varepsilon = 4$ at $\delta = 10^{-5}$ over 100 epochs, for seven sampling rates. The noise on the averaged gradient, in units of C, falls faster than $1/\sqrt{B}$.

q	B	Steps T	σ	Noise per averaged gradient σ/B	$\sigma/\sqrt{q}$	$\sigma^2(e^{1/\sigma^2} - 1)$
0.001	10	100,000	0.704	0.0704	22.3	3.23
0.003	30	33,333	0.909	0.0303	16.6	1.95
0.01	100	10,000	1.36	0.0136	13.6	1.33
0.03	300	3,333	2.16	0.0072	12.5	1.12
0.1	1,000	1,000	3.78	0.00378	12	1.04
0.3	3,000	333	6.44	0.00215	11.8	1.01
1	10,000	100	10.8	0.00108	10.8	1

From $B = 10$ to $B = 1{,}000$, a hundredfold increase, the noise per averaged gradient falls from 0.0704 to 0.00378, a factor of 18.6, against the factor of 10 that $1/\sqrt{B}$ predicts. Table 37.5 trains four of these configurations with three step sizes each.

Table 37.5.: Test accuracy of the running example at $\varepsilon = 4$ and 100 epochs, for four batch sizes and three step sizes each, one run each.

B	Step sizes and test accuracy
30	$\eta = 0.25$: 0.9544; 0.5: 0.9487; 1: 0.9453
100	$\eta = 0.5$: 0.9522; 1: 0.9569; 2: 0.9465
300	$\eta = 1$: 0.9591; 2: 0.9565; 4: 0.9577

B	Step sizes and test accuracy
1,000	$\eta = 2$: 0.9589; 4: 0.9565; 8: 0.9576

At its best step size, each batch size reaches between 0.9544 and 0.9591, and the two largest batches do best, with 100 and 30 times fewer steps than $B = 10$ would take. (Choosing the best of three step sizes after looking at test accuracy is itself a leak; it is done here to compare batch sizes, and Hyperparameter tuning shows how to do it privately.)

37.5.6. Check yourself

- At a fixed number of epochs and fixed ε, you multiply the batch size by 4. Roughly how does the noise multiplier change in the regime of Equation 37.5, and the noise on the averaged gradient? *(The noise multiplier doubles, $\sigma \propto \sqrt{q}$, and the noise on the averaged gradient halves, $\propto 1/\sqrt{B}$.)*
- Why does the full-batch row not need the subsampling accountant? *(With $q = 1$ every record is in every step, the steps are plain Gaussian mechanisms, and Gaussian mechanisms compose exactly into one with ratio $\sqrt{T}/\sigma$.)*

37.6. Per-example gradient norms without per-example gradients

37.6.1. Context

Ordinary backpropagation computes the gradient of the *sum* of the batch's losses, never the gradient of each record. Clipping needs each record's gradient norm. Forming all B per-example gradients costs B times the memory of the model's gradient, which for large models is prohibitive. Can the norms, and the clipped sum, be had more cheaply?

37.6.2. Statement

For a dense layer with inputs a_i and back-propagated errors δ_i, record i's gradient is $\delta_i a_i^\top$ (Fact 4), and

$$\left\| \delta_i a_i^\top \right\|_F = \|\delta_i\| \, \|a_i\|. \tag{37.6}$$

The whole network's per-example gradient norm is the square root of the sum of these squared norms over layers. The clipped sum is $\sum_i c_i \delta_i a_i^\top = (c \circ \Delta)^\top A$, where $c_i = \min(1, C/\|g_i\|)$, Δ and A stack the errors and inputs as rows, and $c \circ \Delta$ multiplies row i of Δ by c_i. For a layer applied to a sequence of T tokens, record i's gradient is $\sum_t \delta_{it} a_{it}^\top$, and

$$\left\| \sum_t \delta_{it} a_{it}^\top \right\|_F^2 = \sum_{t,s} (a_{it} \cdot a_{is})(\delta_{it} \cdot \delta_{is}). \tag{37.7}$$

37.6.3. Derivation

1. **One token.** The squared Frobenius norm is the sum of squared entries: $\sum_{j,k} \delta_j^2 a_k^2 = (\sum_j \delta_j^2)(\sum_k a_k^2) = \|\delta\|^2 \|a\|^2$.
2. **Layers.** The full gradient is the concatenation of the layers' gradients, so its squared length is the sum of theirs.
3. **The clipped sum.** $\sum_i c_i \delta_i a_i^\top$ is linear in the errors: scale each record's error by c_i and do the usual batch product of errors and inputs. One backward pass computes the errors, the norms come from Equation 37.6, and one more matrix product per layer gives the clipped sum.
4. **Sequences.** Expand the square: $\| \sum_t \delta_t a_t^\top \|_F^2 = \sum_{t,s} \operatorname{tr}(a_t \delta_t^\top \delta_s a_s^\top) = \sum_{t,s} (\delta_t \cdot \delta_s)(a_s \cdot a_t)$, using $\operatorname{tr}(uv^\top) = v \cdot u$. This is the entrywise product of two $T \times T$ Gram matrices, summed.

37.6.4. Interpretation

This is **ghost clipping** (Goodfellow 2015 for single inputs; Li et al. 2022 for sequences): the per-example gradients are "ghosts" that are never formed. For one token per record it stores $B(p_{\text{in}} + p_{\text{out}})$ numbers per layer instead of $Bp_{\text{in}}p_{\text{out}}$. For sequences the Gram matrices cost $2BT^2$, which beats the explicit $Bp_{\text{in}}p_{\text{out}}$ only when $2T^2 < p_{\text{in}}p_{\text{out}}$, so long sequences in narrow layers favour forming the gradients. Bu, Mao & Xu (2022) choose per layer. Either way, privacy is untouched: these are exact computations of the same quantities.

37.6.5. Example: the running example and a transformer layer

For the 256 records of a test batch, the norms from Equation 37.6 agree with explicitly formed per-example gradients to a relative difference of 2.2×10^{-16}, rounding error. For a random 16-token sequence, Equation 37.7 gives 33.144228 and the explicit gradient's norm is the same. Table 37.6 compares the memory for several layers.

Table 37.6.: Numbers stored to clip one dense layer, explicitly and with ghost norms.

Layer	Explicit $Bp_{\text{in}}p_{\text{out}}$	Ghost	Ratio
Running example, first layer: $B = 100$, $p = 51 \to 64$	326,400	11,500	28.4
$B = 32$, one token, $768 \to 768$	18,874,368	49,152	384.0
$B = 32$, 512 tokens, $768 \to 768$	18,874,368	16,777,216	1.1
$B = 32$, 512 tokens, $768 \to 3{,}072$	75,497,472	16,777,216	4.5
$B = 8$, 4,096 tokens, $4{,}096 \to 4{,}096$	134,217,728	268,435,456	0.5

37.6.6. Check yourself

- Why does ghost clipping need a second pass, or a second matrix product, after the norms are known? *(The clipping factors c_i depend on the norms, which need the errors of the whole network. Only then can the errors be rescaled and multiplied by the inputs.)*
- For a convolutional layer, is the per-example gradient an outer product? *(Not of one pair of vectors. It is a sum over spatial positions of outer products, like the sequence case, so Equation 37.7 applies with positions in place of tokens.)*

37.7. The running example: privacy against accuracy

37.7.1. Context

Put the pieces together: for the running example, how much accuracy does each budget cost, and where does it go?

37.7.2. Statement

Table 37.7 trains the running example with $q = 0.01$, $C = 0.5$, $\eta = 0.5$ and $T = 10{,}000$ steps, at each budget's noise multiplier from Table 37.2, three times each.

Table 37.7.: Test accuracy of the running example under DP-SGD at five budgets, with $\delta = 10^{-5}$.

	Noise multiplier	Test accuracy (mean ± standard deviation over 3 runs)
$\varepsilon = 0.5$	7.72	0.8209 ± 0.0180
$\varepsilon = 1$	4.13	0.9175 ± 0.0161
$\varepsilon = 2$	2.28	0.9441 ± 0.0051
$\varepsilon = 4$	1.36	0.9548 ± 0.0019
$\varepsilon = 8$	0.917	0.9604 ± 0.0019
Clipped, no noise	0	0.9649 ± 0.0009
Non-private SGD, 1,000 steps	—	0.9610 ± 0.0005

37.7.3. Derivation

Each row runs the algorithm of Section 37.3 from a fresh random initialization, with the noise multiplier from the accountant (Fact 2). The non-private baseline uses the same sampling and step size with neither clipping nor noise; its accuracy is already at its best after 1,000 steps (0.9627) and stays there (0.9617 after 10,000).

37.7.4. Interpretation

Clipping costs nothing here, noise costs everything. Clipping without noise reaches 0.9649, slightly above non-private SGD: on this problem clipping only limits the influence of the few records with large gradients. The whole loss comes from the noise, and it is steeply non-linear in ε. From $\varepsilon = 8$ to 4 the accuracy drops by half a point, from 2 to 1 by 2.7 points, and from 1 to 0.5 by almost 10.

Why the collapse at small ε. At $\varepsilon = 1$ the noise multiplier is 4.13, so the noise on the averaged gradient has length about $4.13 \times 0.5/100 \times \sqrt{3{,}329} = 1.19$, against a signal of about 0.026. By the argument of Section 37.3, signal overtakes noise only after about $(1.19/0.026)^2 \approx 2{,}100$ steps, so the 10,000 steps contain only a few independent "useful" moves. Figure 37.4 shows the result: with $\sigma = 4$ the model reaches its accuracy early and then wanders.

Run-to-run variation grows with the noise. The standard deviation over runs is 0.0019 at $\varepsilon = 8$ and 0.018 at $\varepsilon = 0.5$. A single private run is a sample from a wide distribution, and comparing two configurations by one run each, as Table 37.3 does, is reliable only for differences well above that spread.

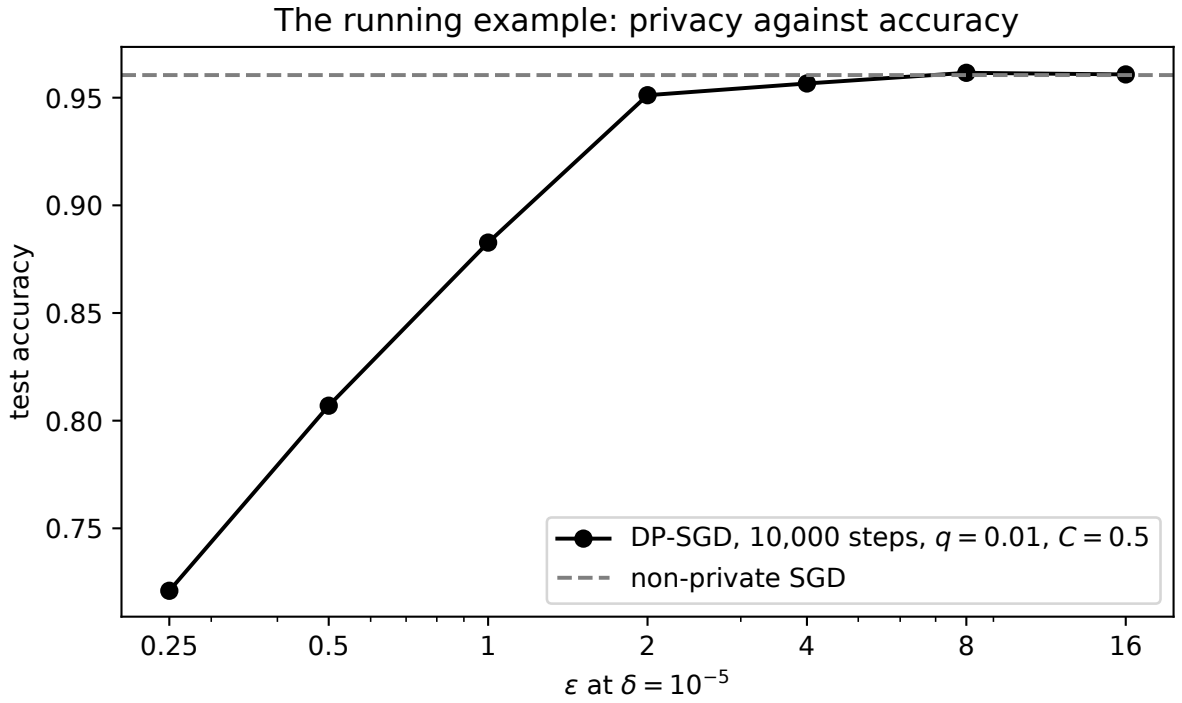


Figure 37.3.: Test accuracy of DP-SGD on the running example at $\delta = 10^{-5}$ against ε (log scale), with the non-private model dashed. Accuracy is close to non-private above $\varepsilon = 4$ and falls steeply below $\varepsilon = 1$.

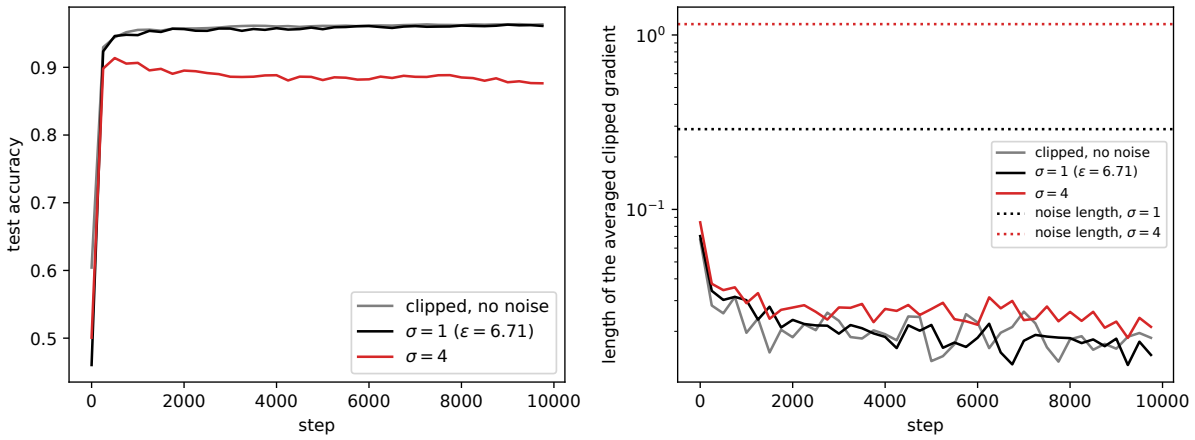


Figure 37.4.: Left: test accuracy during training with clipping alone, with $\sigma = 1$ ($\varepsilon = 6.71$) and with $\sigma = 4$. Right: the length of the averaged clipped gradient at each step (solid, log scale) against the length of the noise on it (dotted). The noise is about ten times longer than the signal at $\sigma = 1$, and about forty times at $\sigma = 4$.

37.7.5. Example: what $\varepsilon = 6.71$ buys

Part 4 asked how private the configuration $q = 0.01$, $\sigma = 1$, $T = 10{,}000$ is, and answered $\varepsilon = 6.71$. This chapter adds the other half: on this task it costs 0.4 points of accuracy, 0.9589 against 0.9627. To halve ε to about 3 you would need σ between 1.36 and 2.28 and accept about one more point of loss; to reach $\varepsilon = 1$, four points. Choosing ε asks which of these trades is worth making, and Auditing checks whether the trained model is as private as the accountant says.

37.7.6. Check yourself

- Why does the non-private baseline use 1,000 steps rather than 10,000? *(Its accuracy is the same, 0.9627 against 0.9617, and the shorter run is the fair comparison: it is the non-private model at its best, not one that was given more steps.)*
- The training curve at $\sigma = 4$ is flat after the first few hundred steps. Would more steps at the same σ help? *(They would add signal, but they would also raise ε. At a fixed budget, more steps require more noise per step, and the break-even calculation of the interpretation gets worse, not better.)*

37.8. Code: check every number by computation

The script code/dp_sgd.py implements the two-layer network, its per-example gradients by ghost norms, DP-SGD with Poisson sampling, and the RDP accountant of Part 4, rewritten in a few dozen lines so the part stands on its own. From the repository root:

```
uv run python code/part05-dp-ml/dp_sgd.py
```

Later chapters import its model and accountant: `make_task()`, `train_dpsgd(rng, data, config)`, `accuracy`, `clipped_grad_sum`, `dpsgd_epsilon(q, sigma, steps, delta)` and `noise_multiplier_for(q, steps, eps, delta)`.

Output (seed 0), summarized in Table 37.8:

Table 37.8.: The script's output with the default seed.

Experiment	Result
Accountant, $q = 0.01$, $\sigma = 1$, 10,000 steps	$\varepsilon = 6.71$ at $\alpha = 4.1$, as in Part 4
Noise on the averaged gradient	0.005 per coordinate, length about 0.288; signal 0.015 to 0.027
Ghost norms	Agree with explicit gradients to 2.2×10^{-16}
Running example, $\varepsilon = 6.71$	0.9589 against 0.9627 non-private
Clipping sweep	With ηC fixed, 0.9599 to 0.9627 for $C \leq 0.3$
Batch sweep, $\varepsilon = 4$	Noise per averaged gradient falls 18.6 times from $B = 10$ to 1,000
Privacy against accuracy	From 0.8209 at $\varepsilon = 0.5$ to 0.9604 at $\varepsilon = 8$

tests/test_dp_sgd.py checks that the output still matches.

What to take from it. DP-SGD is short: sampling, clipping, a noisy sum and a step take about twenty lines, and the accountant a few dozen more. What takes care is everything around it, the conditions of Section 37.3 and the choice of knobs. On this task the clipping norm hardly matters once ηC is tuned, the batch size matters a little, and the noise multiplier, set by the budget, matters most.

37.9. References

- Abadi, Chu, Goodfellow, McMahan, Mironov, Talwar & Zhang, "Deep Learning with Differential Privacy" (CCS 2016). DP-SGD and the moments accountant; the paper this chapter implements.
- Song, Chaudhuri & Sarwate, "Stochastic gradient descent with differentially private updates" (IEEE GlobalSIP 2013), and Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014). The noisy SGD that DP-SGD extends with clipping.
- Mironov, Talwar & Zhang, "Rényi Differential Privacy of the Sampled Gaussian Mechanism" (arXiv 2019). The accountant of Fact 2.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023). The practitioner's reference for everything in this chapter: what to clip, how to tune, what to report.
- De, Berrada, Hayes, Smith & Balle, "Unlocking High-Accuracy Differentially Private Image Classification through Scale" (arXiv 2022). Large batches, small clipping norms and other practices that close much of the gap on image classification.
- Anil, Ghazi, Gupta, Kumar & Manurangsi, "Large-Scale Differentially Private BERT" (Findings of EMNLP 2022). Very large batches for private language-model pretraining.

- Goodfellow, "Efficient Per-Example Gradient Computations" (arXiv 2015). The norm trick of Equation 37.6.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022). Ghost clipping for sequence models, Equation 37.7.
- Bu, Mao & Xu, "Scalable and Efficient Training of Large Convolutional Neural Networks with Differential Privacy" (NeurIPS 2022). Choosing per layer between ghost norms and explicit gradients.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). The conversion of Equation 37.2.

37.10. Cheat sheet

n records, p weights, clipping norm C, noise multiplier σ, sampling rate q, expected batch $B = qn$, T steps, step size η, add/remove neighbours. Table 37.9 collects the results.

Table 37.9.: Every result of the chapter.

Result	Formula	Conditions
Update	$w_{t+1} = w_t - \frac{\eta}{B}\big(\sum_{i\in S_t} g_i \min(1, \frac{C}{\|g_i\|}) + \mathcal{N}(0, \sigma^2 C^2 I)\big)$	Poisson batches, per-example clipping
Privacy	RDP $T\varepsilon_{q,\sigma}(\alpha)$, converted by Equation 37.2	Fixed C, public B, no batch statistics
Running example	$\varepsilon = 6.71$ at $\delta = 10^{-5}$	$q = 0.01$, $\sigma = 1$, $T = 10{,}000$
Expected update	$\frac{1}{n}\sum_i \min(1, C/\|g_i\|) g_i$	Divide by B, not $\|S_t\|$
Noise	$\sigma C/B$ per coordinate, length about $\sigma C\sqrt{p}/B$	—
Scale invariance	Depends on ηC only	Every gradient clipped
Batch size	$\sigma \propto \sqrt{q}$, noise $\propto 1/\sqrt{B}$	Fixed ε and epochs, small q, $\sigma \gtrsim 1$
Ghost norms	$\|\delta_i a_i^\top\|_F = \|\delta_i\|\|a_i\|$; sequences: $\langle AA^\top, \Delta\Delta^\top\rangle$	Dense layers
Signal against noise	Signal overtakes noise after about (noise length/signal length)^2 steps	Signal roughly constant in direction

Rules of thumb

- Only σ, q and T set ε. C and η are free for accuracy.
- Clip small, tune ηC. Below the typical gradient norm, the clipping norm stops mattering.
- Prefer large batches at a fixed number of epochs; scale the step size up with them.
- Check the implementation against Section 37.3: Poisson sampling, per-example gradients, no batch statistics, a public denominator, a fixed C.

- Expect accuracy to be close to non-private at large ε and to collapse below some threshold; locate that threshold for your task before promising a budget.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

38. Hyperparameter tuning and private selection

38.1. Why this matters

The previous chapter proved a guarantee for one training run. Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, run once with a fixed step size, clipping norm, batch size and number of steps, satisfies differential privacy (DP) with the (ε, δ) that the accountant reports; this algorithm is DP-SGD. Nobody trains once. A practitioner tries a grid of step sizes, perhaps several clipping norms and architectures, scores each trained model on a validation set, and ships the best one. The validation set and the training set are private data, and every run touched them. **How much does the search leak, and how do you pay for it honestly?**

The question has a sharp answer, and it is not "nothing" and not always "the sum of all runs". Releasing the best of k private runs can cost exactly k times one run: there are mechanisms for which the best-of-k release is no better than releasing all k runs. That is what composition, the bound that adds up the costs of all runs in Rényi differential privacy (RDP), charges anyway, and for a grid of 10 DP-SGD runs of the running example it turns $\varepsilon = 6.71$ into 27.23. But a small change to the procedure changes the answer completely. If the *number* of runs is random, drawn from a suitable distribution before the search starts, the best-of release costs a constant factor (two or three times one run for pure DP) plus a term that grows only with the logarithm of the expected number of runs. For 10 runs in expectation, the cost falls from 27.23 to between 10.08 and 13.67 depending on the distribution, and for 1,000 runs from 1,035.93 to 13.46.

38.1.1. The one idea: the release is a function of every run, but a random number of runs hides which ones

The model you ship was chosen by comparing all the runs, so it is a function of all of them, and post-processing says its privacy is at most that of all the runs together. That is the composition bound. It can be tight because a fixed number of runs gives the adversary a sharp event to test for: "every one of the k runs came out badly", whose probability multiplies k per-run likelihood ratios together. A random number of runs blurs that event. If K is random, the adversary does not know how many runs failed, and the event "all runs failed" is mostly explained by "there were few runs" rather than by the data. The mathematics of this is the **generating function** $\mathbb{E}[x^K]$ of the number of runs, which turns "the best of K draws" into a formula, and the right choice of its distribution trades a slightly worse search for a much cheaper guarantee.

Table 38.1 lists the results and where each is used.

Table 38.1.: The results of this chapter and where each is used.

Result	Where it is used
The best of k private runs can cost k runs	Why tuning must be accounted for; the common reporting bug
Composition: k runs cost k times the RDP curve	The baseline; small grids
Random stopping: the best of a geometric number of runs is 3ε-DP	Pure-DP candidates; the idea behind everything that follows
Rényi bounds for a random number of runs (Poisson, truncated negative binomial)	Tuning DP-SGD with sublinear cost
Selecting by a noisy validation score	Choosing among fixed candidates; protecting the validation set
Tuning the running example, and what to report	Every private training project; Choosing ε and Auditing

Notation. One **run** is a randomized algorithm Q that trains a model on the private data with hyperparameters drawn from a fixed set of **candidates**, and outputs the model together with its **score**, a number used to rank runs, such as a validation accuracy. The outputs of Q are totally ordered by score, so "the best" is well defined. $Q(D)$ also denotes the distribution of one run's output on dataset D, and $D \simeq D'$ are neighbours. K is the number of runs, fixed ($K = k$) or random, and $f(x) = \mathbb{E}[x^K]$ its generating function. The **best-of-K** algorithm A draws K, runs Q independently K times, and releases the best output. $\varepsilon(\lambda)$ is an RDP curve at order λ (the papers this chapter follows write λ for the order, and so does this chapter), ε_A the curve of A, and γ, η, μ parameters of distributions of K. The running example is that of DP-SGD: a network with 3,329 weights trained on $n = 10{,}000$ records, sampling rate $q = 0.01$, clipping norm $C = 0.5$, $\delta = 10^{-5}$.

How to read this chapter. The toolkit has the two facts every derivation uses, post-processing and the distribution of a maximum. Section 38.3 shows why tuning is not free, and Section 38.4 what the obvious fix costs. Section 38.5 is the key idea, in its simplest form, and Section 38.6 its version for DP-SGD; read the statements and the examples first, the derivations on a second pass. Section 38.7 covers the classical selection mechanisms, Section 38.8 runs everything on the running example, and Section 38.9 says what to do and what to report.

38.2. Toolkit: four facts every derivation uses

38.2.1. Fact 1 — RDP curves add, and convert to (ε, δ)

A mechanism M satisfies $(\lambda, \varepsilon(\lambda))$-**Rényi differential privacy** if, for all neighbours $D \simeq D'$, the Rényi divergence of order $\lambda > 1$ between its output distributions is at most $\varepsilon(\lambda)$:

$$D_\lambda(M(D) \,\|\, M(D')) = \frac{1}{\lambda - 1} \ln \sum_y P_D[y]^\lambda P_{D'}[y]^{1-\lambda} \le \varepsilon(\lambda), \tag{38.1}$$

where $P_D[y]$ is the probability that $M(D)$ outputs y. The sum is the average of the likelihood ratio $P_D[y]/P_{D'}[y]$ raised to the power $\lambda - 1$, averaged over outputs drawn from D; large orders weight the rare, large ratios more. As $\lambda \to \infty$ the divergence becomes the largest log ratio, so pure ε-DP is the point $\lambda = \infty$. Three properties are used here (Rényi differential privacy). Running mechanisms in sequence, even adaptively, adds their curves. The divergence grows with the order, so a bound at order λ_2 also holds at every $\lambda_1 < \lambda_2$. And a curve converts to (ε, δ)-DP at the best order,

$$\varepsilon = \min_{\lambda} \Big[\varepsilon(\lambda) + \ln \frac{\lambda - 1}{\lambda} - \frac{\ln \delta + \ln \lambda}{\lambda - 1}\Big], \tag{38.2}$$

or, read the other way, $(\lambda, \varepsilon(\lambda))$-RDP gives $(\hat{\varepsilon}, \hat{\delta})$-DP for every $\hat{\varepsilon}$ with $\hat{\delta} = e^{(\lambda-1)(\varepsilon(\lambda)-\hat{\varepsilon})}(1 - 1/\lambda)^{\lambda-1}/\lambda$ (From RDP to (ε, δ)). DP-SGD with $q = 0.01$, $\sigma = 1$ and 10,000 steps has $\varepsilon = 6.71$ at $\delta = 10^{-5}$ by this conversion.

38.2.2. Fact 2 — Post-processing, and why "the best" is a post-processing of all runs

A computation that uses only a private output, and not the data, is as private as that output (Post-processing, group privacy and composition). The best of k runs is computed from the k outputs alone: compare their scores and keep the winner. So the best-of-k release is a post-processing of the k runs, and it is at least as private as releasing all of them, which by composition costs k times one run. The question of this chapter is whether it can be *more* private than that. Post-processing never makes things worse, but it does not promise to make them better.

38.2.3. Fact 3 — The best of K draws, and the generating function

Draw an output Y from a distribution Q on outcomes ordered from best to worst. The best of k independent draws is no better than y exactly when every draw is no better than y, so

$$P[\text{best of } k \preceq y] = Q(\preceq y)^k, \tag{38.3}$$

where $Q(\preceq y)$ is the probability that one draw is no better than y and $Q(\prec y)$ the probability that it is strictly worse. The best is exactly y when all draws are no better than y but not all are strictly worse: $Q(\preceq y)^k - Q(\prec y)^k$. Now let the number of draws K be random, independent of the draws. Averaging over K replaces x^k by $\mathbb{E}[x^K]$, which is the **generating function** of K, written $f(x) = \sum_k P[K = k]\, x^k$. So

$$P[A = y] = f(Q(\preceq y)) - f(Q(\prec y)). \tag{38.4}$$

A fixed $K = k$ has $f(x) = x^k$; a Poisson number of runs with mean μ has $f(x) = e^{\mu(x-1)}$; a geometric number on $\{1, 2, \dots\}$, $P[K = k] = \gamma(1-\gamma)^{k-1}$, has $f(x) = \gamma x/(1 - (1-\gamma)x)$, by summing the geometric series.

A die. The best of two rolls of a fair die is a 6 unless both rolls are below 6: $1 - (5/6)^2 = 11/36 = 0.3056$. In the form of Equation 38.4 with outcome $y = 6$ the best, $Q(\preceq 6) = 1$ and $Q(\prec 6) = 5/6$, so $P = 1^2 - (5/6)^2$. If instead you roll once or twice with probability $\frac{1}{2}$ each, $f(x) = \frac{1}{2}x + \frac{1}{2}x^2$ and $P[\text{best} = 6] = f(1) - f(5/6) = 1 - \frac{1}{2} \cdot \frac{5}{6} - \frac{1}{2} \cdot \frac{25}{36} = 17/72 = 0.2361$, the average of the one-roll answer $1/6$ and the two-roll answer $11/36$.

38.2.4. Fact 4 — Disjoint parts of the data cost the larger of their costs

Suppose a dataset is split into two disjoint parts, a training set and a validation set, and each record lives in exactly one of them. An algorithm that is ε_1-DP in the training records (with the validation set fixed) and ε_2-DP in the validation records (with the training set fixed) is $\max(\varepsilon_1, \varepsilon_2)$-DP for neighbours that add or remove one record anywhere, because a pair of neighbours differs in one record, which sits in one part only. The same holds order by order for RDP curves: the curve is the pointwise maximum. This is the idea of **parallel composition**, and it is why a held-out validation set does not add its cost to training's cost: it adds its own cost only where it is larger.

38.3. Best of k is k times worse

38.3.1. Context

The intuitive argument for ignoring tuning goes: the release is one model, chosen among a handful by one comparison, and a choice among eight step sizes carries at most three bits. That argument is wrong in two ways. The released model is not the choice; it is a whole private run, picked *because* of how it came out. And the comparison is a function of every run. How bad can it be?

38.3.2. Statement

Let one run be **randomized response** on two outcomes, "good" and "bad" (best first). On D the run comes out good with probability $\frac{1}{1+e^\varepsilon}$, and on its neighbour D' with probability $\frac{e^\varepsilon}{1+e^\varepsilon}$. One run is exactly ε-DP. Releasing the best of k runs is exactly $k\varepsilon$-DP:

$$\frac{P_D[\text{best of } k \text{ is bad}]}{P_{D'}[\text{best of } k \text{ is bad}]} = \frac{\left(e^\varepsilon/(1+e^\varepsilon)\right)^k}{\left(1/(1+e^\varepsilon)\right)^k} = e^{k\varepsilon}. \tag{38.5}$$

In RDP, Papernot & Steinke (2022, Proposition 17) show that the same release has $D_\lambda \geq k\varepsilon - k\ln(1+e^{-\varepsilon})/(\lambda-1)$, so at large orders it also costs about k times one run.

38.3.3. Derivation

1. **Best of k is bad only if every run is bad.** By Fact 3 with outcome "bad" the worst, $P[\text{best is bad}] = Q(\text{bad})^k$.
2. **Each run's ratio is e^ε.** $Q_D(\text{bad}) = e^\varepsilon/(1+e^\varepsilon)$ and $Q_{D'}(\text{bad}) = 1/(1+e^\varepsilon)$, a ratio of e^ε.
3. **The k ratios multiply.** Raising both to the power k gives Equation 38.5. The other outcome, "good", has the ratio $(1-Q_D(\text{bad})^k)/(1-Q_{D'}(\text{bad})^k)$, which is between $e^{-\varepsilon}$ and 1, so the largest log ratio over outcomes is $k\varepsilon$.
4. **The Rényi version.** Keep only the "bad" term in the sum of Fact 1: $e^{(\lambda-1)D_\lambda} \geq P_D[\text{bad}]^\lambda P_{D'}[\text{bad}]^{1-\lambda} = e^{k\varepsilon\lambda}(1+e^\varepsilon)^{-k}$, which is $(e^{k\varepsilon})^\lambda$ times $P_{D'}[\text{bad}]$. Taking logarithms and dividing by $\lambda-1$ gives $D_\lambda \geq (k\varepsilon\lambda - k\ln(1+e^\varepsilon))/(\lambda-1) = k\varepsilon - k\ln(1+e^{-\varepsilon})/(\lambda-1)$, using $\ln(1+e^\varepsilon) = \varepsilon + \ln(1+e^{-\varepsilon})$.

38.3.4. Interpretation

Where the loss comes from. The whole loss sits on one event: every run failed. Its probability is small on both datasets, but its ratio is the product of k per-run ratios, because the runs are independent and the event requires all of them. Figure 38.1 (right) shows the privacy loss distribution for $\varepsilon = 1$: one run has losses ± 1; the best of 10 has a loss of -0.0446 with probability 0.9564 and a loss of 10 with probability 0.0436. A test that looks for "the released model is bad" detects membership with a likelihood ratio of e^{10} whenever it fires.

This is not an artefact of a contrived mechanism. A DP-SGD run that sometimes diverges, a learning rate that sometimes produces a useless model, a candidate that rarely clears a threshold: each is a run with a "bad" outcome whose probability depends a little on the data. Best-of-k multiplies those small dependences. The example is extreme because it puts all of each run's privacy loss on a single binary outcome. A particular DP-SGD search may leak much less, but no analysis that knows only each run's guarantee can promise it, so the general bound for a fixed k is composition, which grows linearly (Section 38.4).

The bound for a fixed k is tight, so the procedure must change. Composition says best-of-k costs at most $k\varepsilon$, and Equation 38.5 says some mechanism attains it. No better analysis of "the best of exactly k runs" exists for all mechanisms. A better guarantee needs a different procedure, and the next sections change one thing: the number of runs.

Non-private tuning leaks too. If you tune without any noise and then train privately once with the chosen hyperparameters, the hyperparameters are a non-private function of the data, and the final run's ε does not cover them. Papernot & Steinke (2022, Section 2) build a support vector machine whose best regularization weight changes when a few outliers are added, so the chosen value reveals whether the outliers were in the training set, a membership inference through a single number.

38.3.5. Example: the numbers for $\varepsilon = 1$

With $\varepsilon = 1$ a run is good with probability 0.2689 on D and 0.7311 on D'. The script computes the best-of-k distributions exactly. The largest log ratio is 2.0000 for $k = 2$, 3.0000 for $k = 3$ and 10.0000 for $k = 10$, exactly $k\varepsilon$. At order $\lambda = 2$, the Rényi divergence of the best of 10 is 6.8683, between the lower bound of step 4, 6.8674, and composition's upper bound, ten times one run's 0.7353, which is 7.3533. Composition overstates the exact value by 7%; at order 2 it is nearly tight, as at order ∞.

38.3.6. Check yourself

- Releasing only the index of the best run, not the trained model, would it fix Equation 38.5? *(No. In the example the output is "good" or "bad", and the index of the best run carries the same event: if all runs are bad, every index is a bad run, and the chosen one reveals it. What matters is that the release depends on every run.)*
- Suppose each run is ε-DP and you release the best of k runs together with all k scores. What is the cost? *(At most $k\varepsilon$ by composition, the same as the best alone in the worst case; releasing more does not cost more here because the best of k already attains the bound.)*

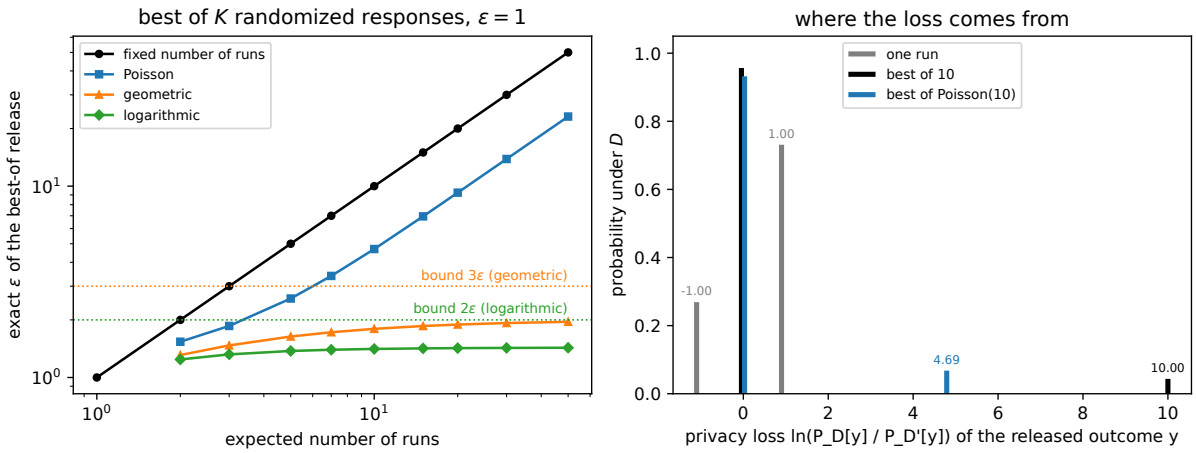


Figure 38.1.: Left: the exact ε of releasing the best of K randomized-response runs, each 1-DP, against the expected number of runs, for a fixed number of runs, a Poisson, a geometric and a logarithmic number. A fixed number grows linearly; the geometric and logarithmic numbers stay below their bounds 3ε and 2ε. Right: the privacy loss of the released outcome under D for one run, the best of 10 runs and the best of a Poisson number with mean 10; the best of 10 puts a loss of 10 on an event of probability 0.0436.

38.4. Paying by composition

38.4.1. Context

The safe answer is to charge for every run. What does it cost on the running example, and how much more noise does each run need to keep the total fixed?

38.4.2. Statement

If each of k runs has RDP curve $\varepsilon(\lambda)$, releasing anything computed from all of them, in particular the best, has curve at most

$$\varepsilon_A(\lambda) \le k\,\varepsilon(\lambda), \tag{38.6}$$

converted to (ε, δ) by Equation 38.2. For DP-SGD, k runs of T steps at noise multiplier σ cost the same as one run of kT steps, so keeping the total at a target ε means choosing σ for kT steps.

38.4.3. Derivation

1. **The release is a post-processing of the k runs** (Fact 2).
2. **The k runs compose.** Each run uses fresh randomness, and RDP curves add (Fact 1), so their joint output has curve $k\varepsilon(\lambda)$. This holds even if the candidates are chosen adaptively, each after seeing the earlier results, since RDP composition covers adaptive sequences.
3. **DP-SGD.** One run of T steps has curve $T\varepsilon_{q,\sigma}(\lambda)$, where $\varepsilon_{q,\sigma}$ is one subsampled Gaussian step. k runs give $kT\varepsilon_{q,\sigma}(\lambda)$, the curve of one run of kT steps.

38.4.4. Interpretation

Composition is cheaper than $k\varepsilon$, but still grows like k. In (ε, δ) terms, k runs cost less than k times one run's ε, because the conversion of Equation 38.2 pays its $\ln(1/\delta)$ term once, and because adding curves behaves like advanced composition, roughly $\sqrt{k}$ at small ε. On the running example, Table 38.2, 10 runs cost 4.06 times one run, not 10. But the curve grows linearly, and so does ε once the budget is large: from 30 to 100 runs, ε grows by a factor 2.5.

Raising the noise is the honest way to keep the budget. To keep the total at $\varepsilon = 6.71$, each of 10 runs needs $\sigma = 2.45$ instead of 1. DP-SGD showed what that costs on this task: more noise per step means fewer useful steps, and accuracy falls steeply below some budget.

When composition is the right tool. For a handful of candidates, two or three, composition costs little more than the alternatives below and needs no change to the procedure. Figure 38.2 shows that the random-K methods only pull away from composition from a few runs on.

38.4.5. Example: the running example's grid

Table 38.2 takes the running example, $q = 0.01$, $\sigma = 1$, 10,000 steps, one run $\varepsilon = 6.71$ at $\delta = 10^{-5}$, and composes k runs. The values for 10 and 100 runs are those Choosing an accountant reported.

Table 38.2.: Composition of k DP-SGD runs of the running example, at $\delta = 10^{-5}$.

Runs k	1	3	10	30	100
ε of k runs at $\sigma = 1$	6.71	12.73	27.23	58.25	145.17
Ratio to one run	1.00	1.90	4.06	8.68	21.63
σ per run for a total of 6.71	1	1.46	2.45	4.12	7.44

38.4.6. Check yourself

- Why does k runs of T steps cost exactly as much as one run of kT steps, if the runs start from different initializations? *(The accountant does not look at the weights. Each step costs $\varepsilon_{q,\sigma}(\lambda)$ whatever it is applied to, and curves add.)*
- Your team ran 30 configurations, each at $\varepsilon = 6.71$, and reports 6.71. What does composition say the release costs? *(58.25. The best-of release can be at most that; Section 38.3 shows that without more structure no smaller number is guaranteed.)*

38.5. Random stopping: the best of a geometric number of runs

38.5.1. Context

Liu & Talwar (2019) asked whether the best of many ε-DP runs can cost a constant multiple of ε, independent of how many runs are made. They found that it can, with one change: after each

run, flip a coin and stop with probability γ. The number of runs is then geometric with mean $1/\gamma$, and releasing the best run seen is 3ε-DP whatever γ is.

38.5.2. Statement

Algorithm (random stopping). Repeat: run Q on the data and keep its output; with probability γ, stop and release the best output kept so far.

If Q is ε-DP, the release is 3ε-DP (Liu & Talwar 2019, Theorem 3.2). Its output distribution is explicit: for an outcome y with probability p under one run, $p_>$ the probability that one run is strictly better than y and $p_\geq = p_> + p$ the probability that it is at least as good,

$$P[A = y] = \frac{\gamma\, p}{(\gamma + (1-\gamma)p_>)(\gamma + (1-\gamma)p_\geq)}. \tag{38.7}$$

Its utility: for any fraction π, the release falls below the top π-quantile of one run's score with probability

$$P[\text{miss}] = \frac{\gamma(1-\pi)}{\gamma + (1-\gamma)\pi} \leq \frac{\gamma}{\pi}. \tag{38.8}$$

38.5.3. Derivation

The idea: the probability of each output is a ratio built from three probabilities of one run, p, $p_>$ and $p_\geq$. Each moves by at most e^ε between neighbours, and the parameter γ enters in a way that cannot amplify those moves.

1. **The number of runs.** The algorithm stops after run j with probability $\gamma(1-\gamma)^{j-1}$: it continued $j-1$ times and stopped once. So K is geometric with mean $1/\gamma$ and generating function $f(x) = \gamma x/(1-(1-\gamma)x)$ (Fact 3).
2. **Apply Equation 38.4.** One run is no better than y with probability $1-p_>$ and strictly worse with probability $1-p_\geq$. So $P[A=y] = f(1-p_>) - f(1-p_\geq)$. Since $1-(1-\gamma)(1-p_>) = \gamma + (1-\gamma)p_>$, this is
$$\frac{\gamma(1-p_>)}{\gamma + (1-\gamma)p_>} - \frac{\gamma(1-p_\geq)}{\gamma + (1-\gamma)p_\geq}. \tag{38.9}$$
3. **Simplify.** Over the common denominator, the numerator is γ times $(1-p_>)(\gamma + (1-\gamma)p_\geq) - (1-p_\geq)(\gamma + (1-\gamma)p_>)$. Expanding, the terms with γ give $\gamma(p_\geq - p_>) = \gamma p$, and the terms with $1-\gamma$ give $(1-\gamma)(p_\geq - p_> p_\geq - p_> + p_\geq p_>) = (1-\gamma)p$. Their sum is p, which gives Equation 38.7.
4. **Bound each factor.** Write primes for the same quantities on D'. Each of p, $p_>$ and $p_\geq$ is the probability of an event about one run's output, so ε-DP of Q gives $p \leq e^\varepsilon p'$, $p'_> \leq e^\varepsilon p_>$ and $p'_\geq \leq e^\varepsilon p_\geq$. The first bounds the numerator ratio. For the denominators, $\gamma + (1-\gamma)p'_> \leq e^\varepsilon(\gamma + (1-\gamma)p_>)$, because $\gamma \leq e^\varepsilon\gamma$ and $p'_> \leq e^\varepsilon p_>$; the same for $p_\geq$.
5. **Multiply.** $P[A(D)=y]/P[A(D')=y] = \frac{p}{p'} \cdot \frac{\gamma+(1-\gamma)p'_>}{\gamma+(1-\gamma)p_>} \cdot \frac{\gamma+(1-\gamma)p'_\geq}{\gamma+(1-\gamma)p_\geq} \leq e^{3\varepsilon}$. This holds for every outcome, so the release is 3ε-DP.
6. **Utility.** The release is below the top π-quantile only if every run is: $\sum_j \gamma(1-\gamma)^{j-1}(1-\pi)^j = \gamma(1-\pi)/(1-(1-\gamma)(1-\pi))$, by the geometric series, which is the equality in Equation 38.8. It is at most γ/π because $\pi(1-\pi) \leq \gamma(1-\pi) + \pi$.

38.5.4. Interpretation

Why the fixed-k disaster disappears. In Equation 38.4 with $f(x) = x^k$, the ratio for the worst outcome is $(Q(\preceq y)/Q'(\preceq y))^k$, a k-th power. The geometric generating function is a ratio of linear functions of x, so the ratio of outputs is a product of three ratios of linear functions, each moved by at most e^ε. The event "every run was bad" is now mostly the event "the coin stopped early", which does not depend on the data.

The 3 is real. The bound is attained in the limit. Take three outcomes, best first, with probabilities $(a, b, 1-a-b)$ on D and $(ae^\varepsilon, be^{-\varepsilon}, \dots)$ on D', with $\gamma \ll b \ll a$ small. For the middle outcome, the numerator ratio $p/p' = e^\varepsilon$ and, since γ is negligible next to $p_>$ and $p_\geq \approx p_>$, both denominator ratios approach e^ε. The example below computes it.

Two outcomes give only 2ε. On randomized response, the "good" outcome has $p_> = 0$, so one denominator factor is γ on both datasets, and only two factors can move. Figure 38.1 (left) shows the exact ε of geometric stopping on randomized response rising toward 2ε as the mean number of runs grows.

The utility guarantee is about quantiles, and it costs runs. Equation 38.8 does not promise the best candidate. It promises that, with probability at least $1 - \gamma/\pi$, the release is at least as good as the top π fraction of a single run's outcomes. To find a candidate in the top $1/m$ of a grid of m with probability 0.9, you need $\gamma/\pi \leq 0.1$, so $\gamma = 0.1/m$, a mean of $10m$ runs, ten times the grid. The exact miss probability is smaller than the bound, but not by much when π is small. This is the price of the method: you buy a constant privacy cost with extra runs, and the geometric distribution often stops after one or two.

Approximate DP and threshold variants. Liu & Talwar also give a version with a known score threshold, which stops as soon as a run clears it and costs 2ε plus a small term (their Theorem 3.1), and versions for (ε, δ)-DP runs and with a hard cap on the number of runs (Theorems 3.4 and 3.5). For DP-SGD, which is analysed in RDP, Section 38.6 is the version to use.

38.5.5. Example: a 3ε pair, and the utility of stopping

On randomized response with $\varepsilon = 1$, geometric stopping gives an exact ε of 1.3106 at $\gamma = 0.5$, 1.7957 at $\gamma = 0.1$, 1.9769 at $\gamma = 0.01$ and 1.9977 at $\gamma = 0.001$, approaching 2. On the three-outcome pair, with $\varepsilon = 1$ for one run, Table 38.3 shows the exact ε of the release approaching 3, and the logarithmic distribution of Section 38.6 approaching 2.

Table 38.3.: Exact ε of the best-of release on a three-outcome mechanism that is exactly 1-DP, computed from Equation 38.4. The middle outcome attains the maximum, and Equation 38.7 gives the same value.

a	b	γ	Geometric stopping	Logarithmic number, same γ
0.01	0.001	0.1	1.2598	1.1299
0.01	0.0001	0.01	2.2316	1.6158
0.01	10^{-6}	10^{-4}	2.9874	1.9937
0.001	10^{-9}	10^{-6}	2.9987	1.9994

For utility, suppose a grid of 8 candidates with one best. With $\gamma = 1/8$, a mean of 8 runs, the chance of never trying the top eighth is 0.4667, and the bound $\gamma/\pi = 1$ says nothing. With $\gamma = 1/80$, a mean of 80 runs, the chance falls to 0.0805 against a bound of 0.1. To have the same chance of hitting a top 1/80 fraction you would again need ten times more runs: with $\gamma = 1/80$ the miss probability for $\pi = 1/80$ is 0.4969.

38.5.6. Check yourself

- In step 4, why does $\gamma \le e^{\varepsilon}\gamma$ matter? *(The denominator is a mixture of the constant γ, which does not depend on the data, and $p_>$, which does. Each part grows by at most e^{ε}, so the sum does. The constant part is what blurs the "all runs failed" event.)*
- If you wanted to run exactly 8 candidates, could you emulate it by setting $\gamma = 1/8$? *(Only in mean. The geometric distribution stops after one run with probability 1/8 and after more than 16 runs with probability $(7/8)^{16} = 0.118$; Figure 38.3 shows how spread out it is. That spread is what buys the privacy.)*

38.6. Rényi bounds for a random number of runs

38.6.1. Context

DP-SGD is not usefully ε-DP for any small ε; its guarantee is an RDP curve that converts to (ε, δ). Papernot & Steinke (2022) extended random stopping to RDP and to a family of distributions for K, which lets you choose between more runs and less privacy cost. What do their bounds say, and what do they cost on the running example?

38.6.2. Statement

The generic bound (Papernot & Steinke 2022, Lemma 7). Let K have generating function f, and let A release the best of K independent runs of Q (if $K = 0$, an output that does not depend on the data). For neighbours with run distributions Q, Q' and best-of distributions A, A',

$$D_\lambda(A\|A') \le D_\lambda(Q\|Q') + \frac{1}{\lambda - 1}\ln\Big[f'(q)^\lambda f'(q')^{1-\lambda}\Big] \tag{38.10}$$

for some numbers $q, q' \in [0,1]$ that are the probabilities of one and the same randomized test accepting a run's output, on D and on D'.

Poisson (Theorem 6). If K is Poisson with mean μ, and Q is $(\lambda, \varepsilon(\lambda))$-RDP and $(\hat\varepsilon, \hat\delta)$-DP with $e^{\hat\varepsilon} \le 1 + \frac{1}{\lambda - 1}$, then

$$\varepsilon_A(\lambda) \le \varepsilon(\lambda) + \mu\hat\delta + \frac{\ln \mu}{\lambda - 1}. \tag{38.11}$$

Truncated negative binomial (Theorem 2). For $\eta > -1$ and $\gamma \in (0,1)$, let K on $\{1, 2, \dots\}$ have $P[K = k] = \frac{(1-\gamma)^k}{\gamma^{-\eta} - 1}\prod_{\ell=0}^{k-1}\frac{\ell+\eta}{\ell+1}$, and for $\eta = 0$ the **logarithmic distribution** $P[K = k] = \frac{(1-\gamma)^k}{k\ln(1/\gamma)}$. The case $\eta = 1$ is the geometric distribution. If Q is $(\lambda, \varepsilon(\lambda))$-RDP and $(\hat\lambda, \varepsilon(\hat\lambda))$-RDP, then

$$\varepsilon_A(\lambda) \le \varepsilon(\lambda) + (1+\eta)\Big(1 - \frac{1}{\hat\lambda}\Big)\varepsilon(\hat\lambda) + \frac{(1+\eta)\ln(1/\gamma)}{\hat\lambda} + \frac{\ln \mathbb{E}[K]}{\lambda - 1}, \tag{38.12}$$

for every $\hat{\lambda} \geq 1$, with $\mathbb{E}[K] = \frac{\eta(1-\gamma)}{\gamma(1-\gamma^\eta)}$ ($\eta \neq 0$) or $\frac{1/\gamma - 1}{\ln(1/\gamma)}$ ($\eta = 0$). For an ε-DP run, letting both orders go to infinity gives $(2+\eta)\varepsilon$-DP (their Corollary 3): 3ε for the geometric distribution, as in Section 38.5, and 2ε for the logarithmic.

38.6.3. Derivation

The generic bound comes from writing the best-of distribution of Fact 3 as a derivative, then pulling the derivative out of the Rényi sum. The Poisson bound then follows in a few lines. Take outputs finite and $P[K = 0] = 0$ for simplicity (the paper handles the general case).

1. **The best-of probability is an average of f'.** By Equation 38.4 and the fundamental theorem of calculus, $A(y) = f(Q(\preceq y)) - f(Q(\prec y)) = \int_{Q(\prec y)}^{Q(\preceq y)} f'(x)\,dx$. The interval has length $Q(y)$, so $A(y) = Q(y)\,c(y)$, where $c(y)$ is the average of f' over that interval, the expectation of $f'(X)$ for X uniform on it. Likewise $A'(y) = Q'(y)\,c'(y)$.
2. **Put it in the Rényi sum.** $e^{(\lambda-1)D_\lambda(A\|A')} = \sum_y A(y)^\lambda A'(y)^{1-\lambda} = \sum_y Q(y)^\lambda Q'(y)^{1-\lambda}\,c(y)^\lambda c'(y)^{1-\lambda}$.
3. **Bound the averages by a worst case.** Couple X and X' at the same relative position $t \in [0,1]$ in their intervals. The function $(u,v) \mapsto u^\lambda v^{1-\lambda}$ is jointly convex for $\lambda > 1$, so by Jensen's inequality $c(y)^\lambda c'(y)^{1-\lambda} \leq \mathbb{E}[f'(X)^\lambda f'(X')^{1-\lambda}]$, which is at most the largest value of $f'(q)^\lambda f'(q')^{1-\lambda}$ over t, with $q = Q(\prec y) + tQ(y)$ and $q' = Q'(\prec y) + tQ'(y)$.
4. **Pull the worst case out of the sum.** Bounding every $c^\lambda c'^{1-\lambda}$ by the largest one over y and t leaves $\sum_y Q(y)^\lambda Q'(y)^{1-\lambda} = e^{(\lambda-1)D_\lambda(Q\|Q')}$ times that maximum. Taking logarithms and dividing by $\lambda - 1$ gives Equation 38.10.
5. **q and q' come from one test.** The test "accept an output worse than y; accept y itself with probability t; reject anything better" accepts with probability q on Q and q' on Q'. Any privacy guarantee of Q therefore constrains the pair, by post-processing (Fact 2).
6. **Poisson: compute f'.** $f(x) = e^{\mu(x-1)}$, so $f'(x) = \mu e^{\mu(x-1)}$ and $\ln[f'(q)^\lambda f'(q')^{1-\lambda}] = \ln\mu + \mu\big[\lambda(q-1) + (1-\lambda)(q'-1)\big] = \ln\mu + \mu\big[(\lambda-1)(1-q') - \lambda(1-q)\big]$.
7. **Use $(\hat{\varepsilon}, \hat{\delta})$-DP on the rejection event.** $1-q'$ and $1-q$ are the probabilities that the test of step 5 rejects, on D' and D, so $1 - q' \leq e^{\hat{\varepsilon}}(1-q) + \hat{\delta}$. The bracket is at most $(1-q)\big[(\lambda-1)e^{\hat{\varepsilon}} - \lambda\big] + (\lambda-1)\hat{\delta}$.
8. **The condition on $\hat{\varepsilon}$ kills the first term.** $e^{\hat{\varepsilon}} \leq 1 + \frac{1}{\lambda-1}$ is the same as $(\lambda-1)e^{\hat{\varepsilon}} \leq \lambda$, so the first term is not positive and the bracket is at most $(\lambda-1)\hat{\delta}$. Divide by $\lambda - 1$ and add $D_\lambda(Q\|Q') \leq \varepsilon(\lambda)$: that is Equation 38.11.
9. **Truncated negative binomial.** Here $f'(x) = \gamma^{\eta+1}\mathbb{E}[K]\,(1-(1-\gamma)x)^{-\eta-1}$. The same steps apply, except that step 7 bounds the probabilities $1-q$ and $1-q'$ through the run's RDP guarantee at a second order $\hat{\lambda}$, again by post-processing, which gives Equation 38.12; Papernot & Steinke (2022, Appendix B.2) carry out the algebra.

38.6.4. Interpretation

What each term pays for. In Equation 38.11 and Equation 38.12 the first term, $\varepsilon(\lambda)$, is one run: you never pay less than one run, since the release contains a run. The last term, $\ln\mathbb{E}[K]/(\lambda-1)$, is the price of the number of runs, and it is logarithmic: multiplying the expected number of runs by 10 adds $\ln 10/(\lambda-1)$. The middle terms are the price of the tail of K: how much the distribution of K lets the adversary tell "many runs, all bad" from "few runs". For the truncated negative binomial it is a constant times the run's own privacy at another order, so the total is a

constant multiple of one run plus the logarithm. For the Poisson distribution it is $\mu\hat{\delta}$, which is linear in μ but multiplied by the run's $\hat{\delta}$ at a small $\hat{\varepsilon}$, and so is small only when one run is very private.

Choosing η: privacy against the shape of K. Smaller η means a smaller multiple, $(1+\eta)$, of the second term, so better privacy; the logarithmic distribution ($\eta = 0$) is the best of the family that keeps $K \geq 1$ with a finite mean. But smaller η also means a more skewed distribution of K. With mean 10, the logarithmic distribution has $\gamma = 0.02692$ and stops after a single run with probability 0.2692, against 0.1000 for the geometric ($\eta = 1$) and 0.1562 for $\eta = 0.5$. The Poisson distribution is the limit $\eta \to \infty$ at a fixed mean, and is concentrated near its mean: with mean 10, $P[K \leq 1] = 4.99 \times 10^{-4}$. Figure 38.3 shows the four distributions with mean 8. A skewed K often runs few candidates, which Section 38.8 shows costs accuracy.

How tight are the bounds? They are upper bounds for every run with the given curve, so they can be loose for a particular one. On randomized response with $\varepsilon = 0.5$ and a logarithmic number of runs with mean 10, Table 38.4 compares the exact Rényi divergence of the release with Equation 38.12 (best $\hat{\lambda}$) and with composition of 10 runs. At order 2, Equation 38.12 is worse than composition, because its constant terms dominate at small orders. At orders 5 and 20, where conversions to (ε, δ) usually land, it is two to four times below composition and two to three times above the exact value.

Table 38.4.: Randomized response with $\varepsilon = 0.5$, best of a logarithmic number of runs with mean 10: the exact Rényi divergence against the bound and against composition.

Order λ	Exact D_λ	Bound Equation 38.12	Composition of 10 runs
2	0.1438	3.0561	2.2734
5	0.3912	1.4860	3.8424
20	0.6379	1.1224	4.7505

For the Poisson bound on the same mechanism with mean 10, randomized response is $\hat{\delta} = 0$ at $\hat{\varepsilon} = 0.5$, so Equation 38.11 applies at the order where $e^{0.5} = 1 + 1/(\lambda - 1)$, $\lambda = 2.5415$. There the exact divergence is 0.4421, the bound 1.7654 and composition 2.7165.

What the theorems do not cover. Each run must draw its candidate independently from a fixed distribution, typically uniform on the grid: this is random search. Adaptive methods that pick the next candidate from the scores so far, such as Bayesian optimization, are not covered and need adaptive composition (Adaptive composition). The score must be part of each run's private output: if it is computed on private validation data, it must be private too (Section 38.7). And for zero-concentrated DP runs, whose curve is a line $\varepsilon(\lambda) = \rho\lambda$, the paper's Corollary 4 gives the closed form $\varepsilon_A(\lambda) \leq 2\sqrt{\rho \ln \mathbb{E}[K]} + 2(1+\eta)\sqrt{\rho \ln(1/\gamma)} - \eta\rho$ for orders up to $1 + \sqrt{\ln \mathbb{E}[K]/\rho}$, when $\rho \leq \ln(1/\gamma)$: the overhead grows like the square root of the logarithm of the expected number of runs.

38.6.5. Example: the running example

The script evaluates Equation 38.11 and Equation 38.12 on the RDP curve of one DP-SGD run of the running example, at every order of the accountant, takes the best $\hat{\lambda}$ for Equation 38.12 and,

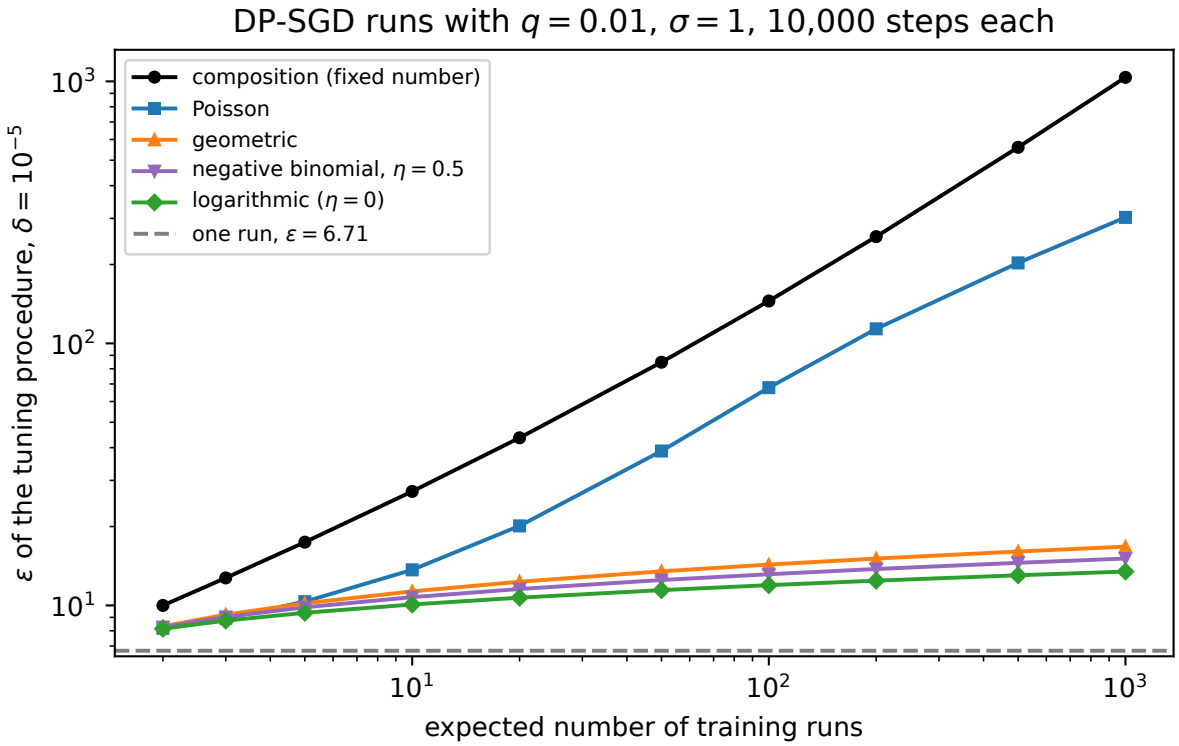


Figure 38.2.: The ε at $\delta = 10^{-5}$ of releasing the best of several DP-SGD runs of the running example ($q = 0.01$, $\sigma = 1$, 10,000 steps, one run $\varepsilon = 6.71$), against the expected number of runs, by composition, by the Poisson bound and by the truncated negative binomial bound with $\eta = 1$ (geometric), 0.5 and 0 (logarithmic). Both axes are logarithmic.

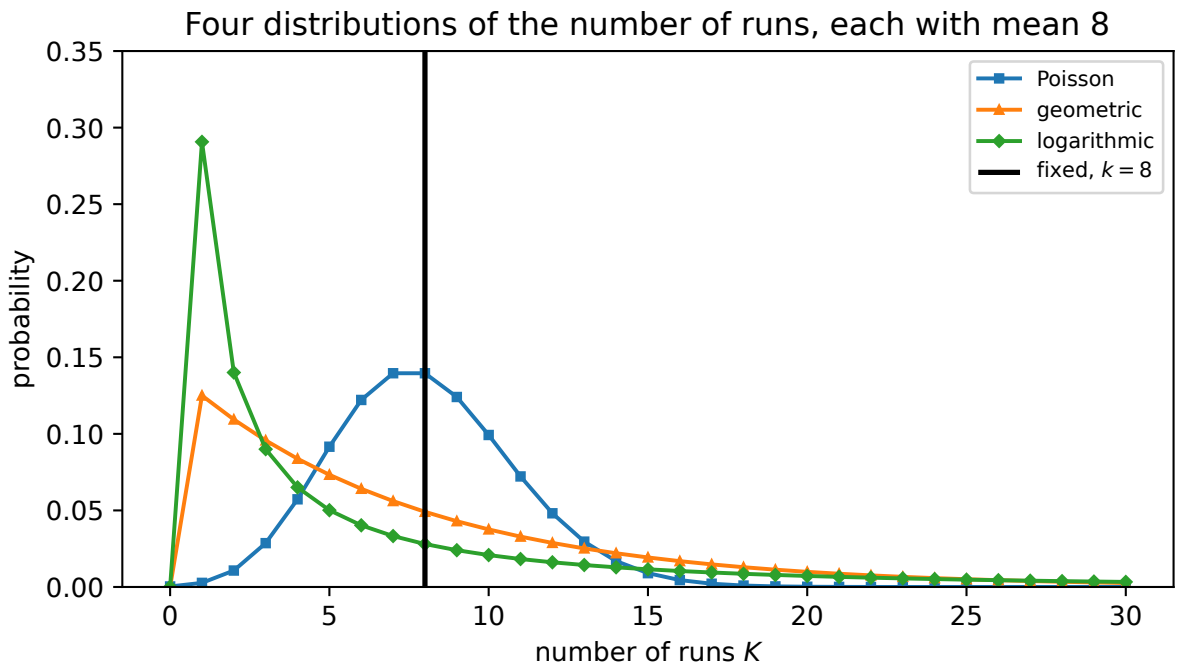


Figure 38.3.: Four distributions of the number of runs, each with mean 8: Poisson, geometric, logarithmic and a fixed 8. The logarithmic distribution stops after one run with probability 0.29.

for Equation 38.11, the best $\hat{\delta}$ that the curve gives at $\hat{\varepsilon} = \ln(1 + 1/(\lambda - 1))$ by the conversion of Fact 1, uses that a bound at a larger order holds at a smaller one, and converts with Equation 38.2. Table 38.5 and Figure 38.2 show the result.

Table 38.5.: The ε at $\delta = 10^{-5}$ of tuning with DP-SGD runs of the running example (one run: $\varepsilon = 6.71$), by the number of runs and how it is chosen.

Expected runs	3	10	30	100	1,000
Composition	12.73	27.23	58.25	145.17	1035.93
Poisson	8.93	13.67	26.43	67.77	302.57
Geometric ($\eta = 1$)	9.22	11.31	12.85	14.32	16.76
$\eta = 0.5$	9.03	10.75	11.98	13.15	15.09
Logarithmic ($\eta = 0$)	8.75	10.08	11.04	11.95	13.46

For 10 runs in expectation, the logarithmic distribution costs 10.08, 1.5 times one run, against 27.23 for composition. From 10 to 1,000 runs it adds only 3.38 more. The Poisson bound starts well, 8.93 for 3 runs, but grows nearly linearly, because one run at $\varepsilon = 6.71$ is far from private enough to make $\mu\hat{\delta}$ small: at the orders the conversion uses, $\hat{\varepsilon}$ must be small and the run's $\hat{\delta}$ there is not. At a smaller per-run budget the Poisson bound does much better, as the experiment of Section 38.8 shows.

38.6.6. Check yourself

- In Equation 38.12, why can't the cost fall below one run, $\varepsilon(\lambda)$, however few runs are expected? *(The release is the output of one run. If $K = 1$ it is exactly one run, so no bound can be below one run's divergence.)*
- The Poisson distribution has $P[K = 0] = e^{-\mu}$. What must the algorithm release then, and why does it not matter much? *(Something that does not depend on the data, such as the untrained initialization. With $\mu = 8$ this happens with probability 3.35×10^{-4}.)*

38.7. Selecting by a noisy validation score

38.7.1. Context

The methods so far treat each run as a black box that outputs a model and its score. A different situation is common too: the candidates are already fixed, for example public pretrained checkpoints or models whose training is accounted for separately, and only the choice among them uses private data, a held-out validation set. That is a **selection** problem, and Selection solved it with the exponential mechanism and report-noisy-max. How do those tools apply to tuning, and what do they leave out?

38.7.2. Statement

Let the score of candidate r be its number of correct predictions $u(r)$ on a private validation set of m records. Adding or removing one validation record changes every count by at most $\Delta = 1$. The **exponential mechanism** picks candidate r with probability proportional to $e^{\varepsilon u(r)/(2\Delta)}$; it is ε-DP for the validation records, and its expected gap to the best count among k candidates is at most $\frac{2\Delta}{\varepsilon}(\ln k + 1)$, an accuracy gap of $\frac{2}{\varepsilon m}(\ln k + 1)$. Report-noisy-max, which adds Laplace or Gumbel noise to each count and releases the index of the largest, has the same guarantee (with Gumbel noise it is the same mechanism).

If the candidates were trained on the private training set, with runs whose RDP curves are $\varepsilon_1(\lambda), \dots, \varepsilon_k(\lambda)$, and the validation set is disjoint from it, releasing the selected model costs

$$\varepsilon_A(\lambda) \le \max\Big(\sum_{j=1}^{k} \varepsilon_j(\lambda),\ \varepsilon_{\text{sel}}(\lambda) \Big), \tag{38.13}$$

where ε_{sel} is the selection mechanism's curve in the validation records.

38.7.3. Derivation

1. **The exponential mechanism.** One record changes each count by at most 1, so each weight $e^{\varepsilon u(r)/2}$ by at most $e^{\varepsilon/2}$ and the normalizing sum by at most $e^{\varepsilon/2}$; the ratio of probabilities is at most e^{ε} (Selection). Candidates more than c below the best have weight at most $e^{-\varepsilon c/2}$ times the best's, so the probability of picking one is at most $ke^{-\varepsilon c/2}$, and integrating the tail gives the expected gap (Utility of the exponential mechanism).

2. **Training records.** For a training record, the validation set and the selection noise are fixed and independent of it. The released model and index are a post-processing of the k trained models (Fact 2), which compose (Fact 1): $\sum_j \varepsilon_j(\lambda)$.
3. **Validation records.** For a validation record, the trained models do not depend on it, and the release is the selection mechanism applied to fixed candidates: $\varepsilon_{\text{sel}}(\lambda)$.
4. **Combine.** Each record is in one of the two sets, so Fact 4 gives the maximum.

38.7.4. Interpretation

Selection is cheap; training is not. On a validation set of thousands of records, selecting privately costs almost nothing: the scores differ by tens of correct predictions and the mechanism's temperature, $2/\varepsilon$, is a few. The expensive part of tuning is that each candidate was *trained* on private data, and Equation 38.13 still charges for all k trainings by composition. The exponential mechanism has nothing to say about it, because a trained model is not a low-sensitivity score: Liu & Talwar's starting point was exactly that the candidates' quality is only known through private, randomized runs.

When the classical tools are the right ones. If the candidates do not depend on the private training data, the training term in Equation 38.13 vanishes. Choosing which public checkpoint to fine-tune, which public model to deploy, or which hyperparameters to use from a list validated elsewhere, by their accuracy on a private validation set, is a pure selection problem, and the exponential mechanism or report-noisy-max is the tool, with $\ln k$ utility cost and no linear privacy cost.

The validation set needs protection under every method. The best-of methods of Section 38.5 and Section 38.6 assume that the score is part of each run's private output. If it is an exact validation accuracy, the validation records are not protected at all: the choice and the released score depend on them with no noise. Adding Gaussian noise of standard deviation σ_{val} to each validation count makes each run's score a Gaussian mechanism with RDP curve $\lambda/(2\sigma_{\text{val}}^2)$, and by Fact 4 a run's curve is the larger of its training curve and this one.

A cheaper score is not a cheaper run. Each noisy count adds to the cost in validation records, but by Fact 4 it adds nothing as long as it stays below the training curve at every order. With $\sigma_{\text{val}} = 20$ correct predictions, one noisy count alone has $\varepsilon = 0.182$ and eight composed have 0.546, well below the training runs in the example below.

38.7.5. Example: choosing a step size on 5,000 validation records

In the experiment of Section 38.8, eight DP-SGD runs at $\sigma = 2.07$, with step sizes from 0.0625 to 8, get these counts out of $m = 5{,}000$: 4,486, 4,604, 4,681, 4,694, 4,600, 4,470, 4,232 and 4,123. The best, step size 0.5, beats the second, 0.25, by 13 correct predictions. Table 38.6 selects among them with the exponential mechanism.

Table 38.6.: The exponential mechanism selecting among eight trained models by their correct predictions on 5,000 private validation records.

ε_{sel}	P[best]	Expected gap (correct predictions)	Accuracy gap	Bound $2(\ln 8 + 1)/\varepsilon_{\text{sel}}$
0.01	0.2478	87.62	1.752%	615.9
0.1	0.6484	5.61	0.112%	61.6
1	0.9985	0.02	0.000%	6.2

At $\varepsilon_{\text{sel}} = 0.1$ the selection costs about a tenth of a point of accuracy, and at 1 nothing measurable. The training costs much more: each run at $\sigma = 2.07$ and 1,000 steps has $\varepsilon = 0.657$, and the eight together 1.994. Adding the eight noisy validation counts, run by run, changes nothing: the total stays at 1.994, because the counts' curve is below the training curve at the orders that matter.

38.7.6. Check yourself

- Why is the sensitivity of a count of correct predictions 1 under add/remove, while an accuracy has a sensitivity that depends on m? *(Adding a record adds one prediction, which is right or wrong, so the count moves by at most 1. The accuracy is the count divided by the number of records, and both change; the count avoids dividing by a private number.)*
- Fifty public checkpoints, a private validation set of 2,000 records and $\varepsilon = 0.5$: what expected accuracy gap does the exponential mechanism guarantee? *(At most* $2(\ln 50 + 1)/(0.5 \times 2{,}000) = 0.0098$*, about one point.)*

38.8. Tuning the running example

38.8.1. Context

Put the strategies side by side on the running example: tune the step size of DP-SGD over a grid of eight values, at the same total budget, and compare what each strategy delivers. The budget must cover the training runs and the validation scores.

38.8.2. Statement

Each run trains the running example's network with DP-SGD ($q = 0.01$, $C = 0.5$) for **1,000 steps instead of the 10,000 of DP-SGD**, to keep the script's runtime at a few minutes, with one of eight step sizes $\eta \in \{0.0625, 0.125, \ldots, 8\}$. The first 5,000 of the 20,000 held-out records are a private validation set, and the other 15,000 the test set. A run's score is its number of correct validation predictions plus Gaussian noise with $\sigma_{\text{val}} = 20$. The total budget is $\varepsilon = 2$ at $\delta = 10^{-5}$, and each strategy chooses its noise multiplier σ so that the whole procedure meets it, by the accountant of Section 38.4 or Section 38.6.

Table 38.7.: Tuning the step size of the running example (1,000 steps per run) at a total budget of $\varepsilon = 2$, $\delta = 10^{-5}$. Test accuracy of the released model, mean and standard deviation over repetitions of the whole procedure, estimated by resampling from three runs per step size; runs on private data is the average number of training runs.

Strategy	σ	ε of one run	Total ε	Test accuracy	Runs on private data
No tuning: step size 0.5 from DP-SGD	1.03	1.97	1.97	0.9509 ± 0.0015	1
Grid of 8, single-run ε reported	1.03	1.97	5.60	0.9479 ± 0.0030	8
Tune on proxy data, then one run	1.03	1.97	1.97	0.9487 ± 0.0025	1
Grid of 8, composition	2.07	0.66	1.99	0.9430 ± 0.0022	8
Random search, Poisson, mean 8	1.53	0.98	1.99	0.9433 ± 0.0060	8.01
Random search, logarithmic, mean 8	1.36	1.17	2.00	0.9372 ± 0.0200	7.95

38.8.3. Derivation

The rows are computed as follows; each uses the facts and results named.

1. **No tuning.** One run with the step size the previous chapter used, at the noise multiplier for $\varepsilon = 2$, $\sigma = 1.03$ (the accountant rounds up, so the run costs 1.97).
2. **The reporting bug.** The same eight runs at $\sigma = 1.03$, one per step size, best noisy validation count released, and the single-run ε reported. Its honest cost is the composition of eight runs, 5.60 (Equation 38.6).
3. **Proxy data.** A second task built the same way with a different random teacher stands in for public data from a related distribution. The grid is run there, the best step size by validation on the proxy task is kept, and one private run is trained with it at $\sigma = 1.03$. The proxy runs touch no private data, so they cost nothing (Fact 2).
4. **Composition.** The eight runs at the σ for which eight runs together cost 2, which is 2.07 (Equation 38.6 with Fact 4 for the noisy counts).

5. **Random search.** K runs, each with a step size drawn uniformly from the grid, with K Poisson or logarithmic with mean 8, σ chosen by Equation 38.11 or Equation 38.12 so that the procedure costs 2.
6. **Estimating accuracy.** Training is the slow part, so the script trains three independent runs per step size for each σ, and repeats each tuning procedure 4,000 times by drawing runs from this pool, with fresh validation noise and, for random search, a fresh K. Drawing with replacement from three runs understates the variety of a real search a little.

38.8.4. Interpretation

The honest strategies cost about a point of accuracy here, and the bug hides it. At the same total budget, composition reaches 0.9430 and Poisson random search 0.9433, against 0.9509 without tuning. Reporting the single-run budget makes the grid search look free: it claims 1.97 and spends at most 5.60.

Tuning did not pay on this task, for an instructive reason. Even with the budget misreported, the grid search, 0.9479, is below the untuned step size of 0.5, 0.9509. The step size 0.5 is the best of the grid here, and the noisy validation count sometimes picks a neighbour. It is the best because DP-SGD chose its running example's settings with knowledge of this very task, a leak of exactly the kind this chapter is about, which no accountant in the book has charged for. On a new task, you would not have such a guess, and a search would be needed.

Random search with a Poisson number of runs beats composition only slightly here. At a total of $\varepsilon = 2$ and a mean of 8 runs, Poisson random search allows $\sigma = 1.53$ instead of 2.07, each run costing 0.98 instead of 0.66. But it tries a given step size with probability only $1 - e^{-1}$, since a candidate is missed with probability 0.3679, and its accuracy varies more from one search to the next, 0.0060 against 0.0022. The two effects roughly cancel at this small grid. With more candidates, Table 38.5 says the gap in σ grows.

The logarithmic distribution has the cheapest bound and the worst search. It allows the least noise of the honest strategies, $\sigma = 1.36$, but with mean 8 it stops after a single run with probability 0.2907, and then releases a random step size. That is why its accuracy is lowest and most variable, 0.9372 ± 0.0200. Papernot & Steinke report the same trade-off, and found the Poisson distribution best in their own small search. Figure 38.4 repeats the comparison at $\varepsilon = 1$ and 4 (one run per step size, so those points are noisier).

Tuning on proxy data is nearly free, when proxy data exist. The proxy task has a different teacher, so its best step size is not always the private task's, and the released model reaches 0.9487 for a total cost of one private run. Its quality depends on how close the proxy is.

38.8.5. Example: what the bug changes in a paper's table

A paper reports "test accuracy 0.948 at $\varepsilon = 1.97$" for a model chosen from eight step sizes. A reader comparing it with a paper that reports 0.943 at $\varepsilon = 1.99$ after an honest grid search concludes that the first method is better at the same budget. In Table 38.7 both rows are the same algorithm; the difference is the accounting. At its honest cost of 5.60, the first row should be compared with other methods at $\varepsilon = 5.60$, not at 1.97; DP-SGD shows how much accuracy a budget that large buys.

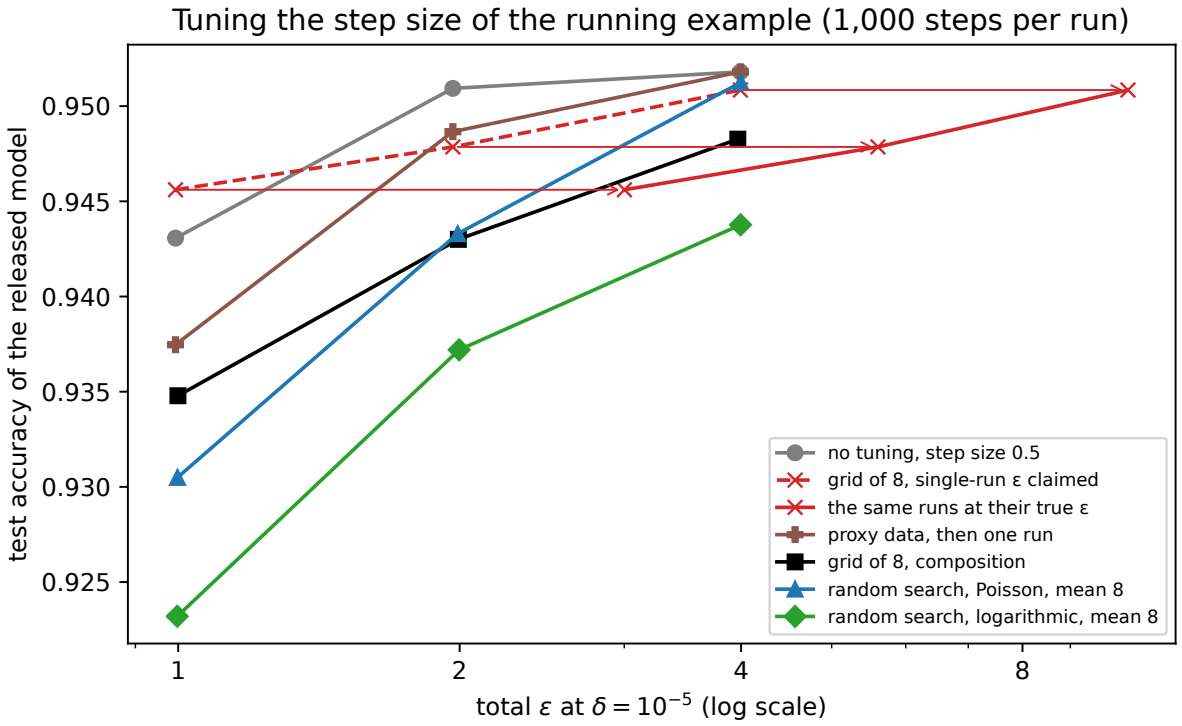


Figure 38.4.: Test accuracy of the model released by each tuning strategy against the total ε at $\delta = 10^{-5}$ it costs (1,000 DP-SGD steps per run). The red dashed line places the grid search at the single-run ε that the reporting bug claims, and the arrows move each point to its honest cost by composition. Points at $\varepsilon = 2$ use three runs per step size; those at 1 and 4 one run.

38.8.6. Check yourself

- Why does the composition row use $\sigma = 2.07$ while the logarithmic row can use 1.36, both at a total of 2? *(Composition charges eight full runs; Equation 38.12 charges one run plus a constant multiple of one run at another order plus* $\ln 8/(\lambda - 1)$*, which is less than seven more runs.)*
- The bootstrap draws runs with replacement from three per step size. Which strategy's accuracy does this distort most? *(Those that run the same step size many times, random search with large K: repeated draws of the same three runs make the best of many runs look less good than independent runs would. With a mean of 8 runs over 8 step sizes, the effect is small.)*

38.9. What to do and what to report

38.9.1. Context

The results of this chapter turn into a short procedure. What should a team training a private model actually do about hyperparameters, and what must a reported ε cover for a reader to rely on it?

38.9.2. Statement

1. **Spend private runs only where you must.** Tune on public data or on proxy data from a related distribution when it exists. Reuse hyperparameters that worked on similar tasks. Shrink the search: DP-SGD showed that below the typical gradient norm only the product of step size and clipping norm matters, so one of the two need not be searched.
2. **Fix the search before touching the private data.** Write down the candidate set, the distribution of the number of runs, the score and the validation set.
3. **Make the score private.** Use a held-out validation set and a noisy count, or spend part of the budget on a private score.
4. **Account for the whole search.** For a few candidates, compose. For more, run random search with a random number of runs and use Equation 38.11 (small per-run budget, moderate number of runs) or Equation 38.12 (many runs). Account for every run on the private data, including runs that crashed, runs for debugging, and evaluations used for early stopping.
5. **Report both numbers.** The ε of the released run, and the ε of the whole procedure with how it was computed: the candidate set, the distribution of K, the accountant, δ and the neighbouring relation.

38.9.3. Derivation

Each item follows from a result above.

1. Runs on public data cost nothing (Fact 2), and every private run costs something by Section 38.4 and Section 38.6. A smaller candidate set gives the same utility with fewer runs, which lowers $\ln \mathbb{E}[K]$ or k.
2. The theorems of Section 38.5 and Section 38.6 require that the candidates are drawn from a fixed distribution and that K is drawn from its distribution independently of the data. Choosing them after looking at results makes them functions of the data that no theorem covers.
3. Every best-of theorem assumes that the score is part of a private output (Section 38.7). An exact validation score makes the validation records unprotected.
4. Composition and the random-K bounds are both valid; which is smaller depends on the number of runs and the per-run budget (Table 38.5, Table 38.7). A crashed run still computed on the data, and if its failure influenced what was done next, it is part of the adaptive sequence.
5. The released run's ε says how much the final model alone leaks if the hyperparameters were public; the procedure's ε is what the release actually guarantees. Papernot & Steinke (2022) suggest stating both when the second is much larger.

38.9.4. Interpretation

The cheapest fix is usually upstream. In Table 38.7, the strategies that spend the whole budget on one private run, no tuning and proxy tuning, beat every honest search. That will not always hold, since a poor guess can be far worse than a noisy search, but it says where to start: get good hyperparameters without the private data, and search privately only over what remains uncertain.

Pre-registration is part of the guarantee. The random-K theorems are statements about a procedure. An engineer who looks at the first few results and decides to "try a few more" has changed the procedure, and the theorem no longer applies. Logging the search configuration before it runs is to tuning what fixing the clipping norm in advance is to DP-SGD.

Reporting standards exist. Ponomareva et al. (2023, Section 5.3.3) ask for the formal guarantee of the final model and of the tuning, and Choosing an accountant lists tuning among the checks of a reported ε. Auditing cannot catch the bug: an audit tests the trained model, not how it was chosen.

38.9.5. Example: the report for the running example

For the Poisson row of Table 38.7, a complete report reads: "DP-SGD with Poisson sampling rate 0.01, clipping norm 0.5, noise multiplier 1.53, 1,000 steps; one run satisfies $(0.98, 10^{-5})$-DP. The step size was selected by random search over $\{0.0625, 0.125, \ldots, 8\}$ with a Poisson number of runs with mean 8, scored by correct predictions on a disjoint validation set of 5,000 records with Gaussian noise of standard deviation 20; the whole procedure satisfies $(1.99, 10^{-5})$-DP by the Rényi bound of Papernot & Steinke (2022, Theorem 6), add/remove neighbours, record level."

38.9.6. Check yourself

- A colleague tunes on the private data, then retrains the chosen configuration from scratch with fresh noise and releases only the retrained model. Does the retraining make the tuning free? *(No. The chosen hyperparameters are a function of the tuning runs, so the release is a post-processing of the tuning runs plus the new run; the tuning must be accounted for, and the new run adds its own cost.)*
- Why can't an audit of the released model detect that tuning was not accounted for? *(An audit estimates how much the trained model reveals about its training records. The best-of-k leak is in the event "every run was bad", which an audit of one released model at one fixed configuration does not replicate unless it reruns the whole search on neighbouring datasets.)*

38.10. Code: check every number by computation

The script code/hyperparameter_tuning.py computes the exact best-of-K distributions of Equation 38.4 for randomized response and the three-outcome mechanism, evaluates the bounds of Liu & Talwar and Papernot & Steinke on the RDP accountant of DP-SGD, whose model and accountant it imports from code/dp_sgd.py, and runs the tuning experiment. From the repository root:

```
uv run python code/part05-dp-ml/hyperparameter_tuning.py
```

It trains about 200 DP-SGD runs of 1,000 steps each and takes two to four minutes. Its accountants are reusable: `tuned_epsilon(q, sigma, steps, strategy)` gives the total ε of a search with `strategy` one of `("fixed", k)`, `("poisson", mean)` or `("tnb<η>", mean)`, and `sigma_for_tuning(q, steps, eps, strategy)` the noise multiplier that keeps a search within a budget.

Output (seed 0), summarized in Table 38.8:

Table 38.8.: The script's output with the default seed.

Experiment	Result
Best of 10 randomized responses, $\varepsilon = 1$	Exactly $\varepsilon = 10$; loss 10 with probability 0.0436
Composition, running example	27.23 for 10 runs, 145.17 for 100
Geometric stopping, three outcomes	Exact ε up to 2.9987, below the bound 3
Logarithmic number of runs, running example	10.08 for 10 runs, 13.46 for 1,000
Exponential mechanism over 8 models	$P[\text{best}] = 0.6484$ at $\varepsilon = 0.1$
Tuning at $\varepsilon = 2$	Honest search 0.9430 to 0.9433; bug claims 1.97 and spends 5.60

tests/test_hyperparameter_tuning.py checks that the output still matches.

What to take from it. The exact computations show that the bounds are attained in the worst case: $k\varepsilon$ for a fixed number of runs, 3ε and 2ε for the geometric and logarithmic numbers. The accountant shows that for DP-SGD the random-K bounds grow logarithmically while composition grows linearly. The experiment shows that on a small grid the accuracy gained by tuning can be smaller than the accuracy lost by paying for it, and that the cheapest strategy is to need fewer private runs.

38.11. References

- Liu & Talwar, "Private Selection from Private Candidates" (STOC 2019; arXiv:1811.07971). Random stopping and the 3ε bound of Section 38.5; threshold and (ε, δ) variants; the fixed-k counterexample (their Appendix B).
- Papernot & Steinke, "Hyperparameter Tuning with Renyi Differential Privacy" (ICLR 2022; arXiv:2110.03620). The generic bound, the Poisson and truncated negative binomial theorems of Section 38.6, the lower bound of Section 38.3, and an experiment on image classification that recommends the Poisson distribution for small searches.
- Chaudhuri & Vinterbo, "A Stability-Based Validation Procedure for Differentially Private Machine Learning" (NeurIPS 2013). The first improvement over composition for tuning, for learning algorithms that are stable.
- Mohapatra, Sasy, He, Kamath & Thakkar, "The Role of Adaptive Optimizers for Honest Private Hyperparameter Selection" (arXiv:2111.04906, 2021). Honest tuning with composition, and optimizers that need less tuning.
- McSherry & Talwar, "Mechanism Design via Differential Privacy" (FOCS 2007). The exponential mechanism of Section 38.7.
- Papernot, Thakurta, Song, Chien & Erlingsson, "Tempered Sigmoid Activations for Deep Learning with Differential Privacy" (arXiv:2007.14191, 2020). Architecture choices that are best without privacy are not best with it, so tuning cannot simply be done non-privately.
- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023). What to tune, how, and what to report.
- Bergstra & Bengio, "Random Search for Hyper-Parameter Optimization" (JMLR 2012). Why random search, which the random-K theorems require, is a sound tuning method in its own right.

38.12. Cheat sheet

K runs of a private algorithm Q with RDP curve $\varepsilon(\lambda)$, best output released, generating function $f(x) = \mathbb{E}[x^K]$. Table 38.9 collects the results.

Table 38.9.: Every result of the chapter.

Result	Formula	Conditions
Best of K	$P[A = y] = f(Q(\preceq y)) - f(Q(\prec y))$	Independent runs, K independent of them
Fixed k	$\varepsilon_A(\lambda) \le k\varepsilon(\lambda)$; attained for $\lambda = \infty$	Always valid

Result	Formula	Conditions
Random stopping	3ε-DP; misses top π fraction w.p. $\le \gamma/\pi$	ε-DP runs, K geometric with mean $1/\gamma$
Generic	$\varepsilon_A(\lambda) \le \varepsilon(\lambda) + \frac{1}{\lambda-1}\ln[f'(q)^\lambda f'(q')^{1-\lambda}]$	q, q' from one test
Poisson	$\varepsilon(\lambda) + \mu\hat{\delta} + \frac{\ln\mu}{\lambda-1}$	$(\hat{\varepsilon}, \hat{\delta})$-DP with $e^{\hat{\varepsilon}} \le 1 + \frac{1}{\lambda-1}$
Truncated negative binomial	$\varepsilon(\lambda) + (1+\eta)(1-\frac{1}{\hat{\lambda}})\varepsilon(\hat{\lambda}) + \frac{(1+\eta)\ln(1/\gamma)}{\hat{\lambda}} + \frac{\ln\mathbb{E}[K]}{\lambda-1}$	Any $\hat{\lambda} \ge 1$; pure DP: $(2+\eta)\varepsilon$
Selection among fixed candidates	Exponential mechanism, gap $\le \frac{2}{\varepsilon m}(\ln k + 1)$ in accuracy	Count score on m validation records
Trained candidates, disjoint validation	$\max(\sum_j \varepsilon_j(\lambda), \varepsilon_{\text{sel}}(\lambda))$	Each record in one set

Rules of thumb

- The best of k runs is not cheaper than k runs unless k is random. Report the cost of the search, not of the released run.
- For DP-SGD, random search with a Poisson number of runs suits small searches at small per-run budgets; the logarithmic or $\eta = 0.5$ distributions suit large searches, at the price of often running few candidates.
- Validation scores on private data must be noisy; a few thousand validation records make the noise cheap.
- The best tuning is the tuning you do not do on private data: public or proxy data, reused settings, fewer knobs.
- Fix the candidate set and the distribution of the number of runs in advance; adaptive searches need adaptive composition.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

39. DP-FTRL and matrix factorization

39.1. Why this matters

Stochastic gradient descent (SGD) with differential privacy (DP), the DP-SGD of the previous chapter, gets its privacy from two sources. Clipping and Gaussian noise make each step a Gaussian mechanism, and **Poisson sampling** makes each step much more private than that: a record is in the batch only with probability q, and the accountant credits the run for the uncertainty about whether the record was used. For the running example of this part, with $q = 0.01$, that amplification is what turns a noise multiplier of 1 into $\varepsilon = 6.71$ over 10,000 steps.

Many training pipelines cannot honestly claim Poisson sampling. Data loaders shuffle the dataset once, or once per epoch, and cut it into fixed batches; streaming pipelines read the data in the order it arrives; and in federated learning, where the records live on phones, the server cannot choose which devices check in, so it cannot sample them independently with a known probability. Amplification by subsampling shows that applying the Poisson accountant to such pipelines can understate ε considerably. The honest alternative is to give up amplification. Each step is then a plain Gaussian mechanism, and on the running example, for $\varepsilon = 2$ over five passes through the data, the noise multiplier has to grow from 0.928 to 4.458, and test accuracy falls from 0.9460 to 0.8941.

This chapter asks: **how do you train privately without trusting random batches, and can correlated noise beat independent noise?** It can, by a lot, and the reason is a change of viewpoint. The first algorithm to exploit it ran the follow the regularized leader (FTRL) algorithm on noisy gradient sums and is called DP-FTRL; it adds its noise through a binary tree, a construction from the theory of private counters. The modern form, the **matrix factorization mechanism**, contains the tree as a special case, and on the running example it closes nearly all of the gap to amplified DP-SGD without any sampling at all.

39.1.1. The one idea: training releases prefix sums, so put the noise where the prefix sums need it

Gradient descent without momentum adds up its steps. After t steps the weights are the starting point minus the step size times the **prefix sum** $S_t = x_1 + \cdots + x_t$ of the clipped gradient sums x_s of the batches seen so far. What the models reveal about the data is what these prefix sums reveal, and the error in a model is the error in its prefix sum. SGD with independent noise on every step, DP-SGD without amplification, makes the noise in S_t a random walk, whose standard deviation grows like $\sqrt{t}$. Nothing forces the noise to be independent. If the noise added at step t partly cancels the noise added before it, the prefix sums stay accurate. The price is that each record's gradient must then be hidden in several noisy quantities at once.

The bookkeeping is linear algebra. Releasing all T prefix sums is the linear map $x \mapsto Ax$, with A the lower-triangular matrix of ones. Write $A = LR$, add Gaussian noise to Rx, and release $L(Rx + z) = Ax + Lz$. The **encoder** R sets the privacy, because the noise must hide one record's effect on Rx; the **decoder** L sets the error, because it is what multiplies the noise. Independent noise, the binary tree and the best known mechanisms are all choices of this factorization, and the question "how much noise does private training need without amplification?" becomes "how small can the sensitivity of R times the size of L be?". Table 39.1 lists the results.

Table 39.1.: The results of this chapter and where each is used.

Result	Where it is used
Training as a prefix-sum workload; the two naive mechanisms	Why DP-SGD without amplification is noisy; what DP-FTRL releases
The matrix mechanism: privacy from the columns of R, error from the rows of L, and a lower bound	Comparing any two correlated-noise schemes; streaming and adaptivity
Tree aggregation and DP-FTRL	The first practical mechanism; federated training
The square-root factorization and the optimal one	The default correlated-noise mechanism; how close it is to optimal
Several participations: sensitivity as the norm of a sum of columns; banded factorizations	Training for more than one epoch; why the data order matters
The running example: matrix mechanisms against DP-SGD	When to use which; user-level privacy in federated learning

Notation. A run has T steps with a fixed batch size B. At step t the clipped per-example gradients of the batch are summed into $x_t \in \mathbb{R}^p$, one entry per weight; stacking the x_t as rows gives the $T \times p$ matrix x. C is the clipping norm, so each record adds a vector of length at most C to each x_t it takes part in. $S_t = \sum_{s \le t} x_s$ is the t-th prefix sum and $\hat{S}_t$ its private estimate. A is the $T \times T$ lower-triangular matrix of ones. A **factorization** is $A = LR$, with R an $m \times T$ encoder and L a $T \times m$ decoder; r_j is the j-th column of R and ℓ_t the t-th row of L. The noise z is $m \times p$ with independent $\mathcal{N}(0, s^2C^2)$ entries; s is the **noise scale**, in units of C like DP-SGD's noise multiplier σ. μ is the ratio of sensitivity to noise standard deviation of a Gaussian mechanism, and E the number of epochs.

The running example. The part's network from DP-SGD: 50-dimensional inputs, labels from a fixed random teacher network, a student with 64 tanh hidden units and $p = 3{,}329$ weights, $n = 10{,}000$ training and 20,000 test records, clipping norm $C = 0.5$, $\delta = 10^{-5}$. The **fixed-order** runs use one public random permutation of the record slots, drawn independently of the data and kept for every epoch, cut into 100 batches of $B = 100$. Each epoch is 100 steps, and each record participates once per epoch, exactly 100 steps apart. The DP-SGD runs use Poisson sampling with $q = 0.01$, so their expected batch is also 100.

How to read this chapter. Read the toolkit and Section 39.3, which sets up the problem, then Section 39.4, the general framework. Section 39.5 and Section 39.6 are two instances, the

historical one and the one to use. Section 39.7 is needed as soon as you train for more than one epoch. Section 39.8 puts the pieces together on the running example.

39.2. Toolkit: four facts every derivation uses

39.2.1. Fact 1 — A Gaussian mechanism is described by one ratio

Release $f(D) + Z$, where f takes values in $\mathbb{R}^{m\times p}$ and Z has independent $\mathcal{N}(0, \varsigma^2)$ entries. Let Δ be the largest change of f between neighbouring datasets, measured in the Frobenius norm: the square root of the sum of squared entries, the Euclidean length of the matrix read as one long vector. The privacy of the release depends only on $\mu = \Delta/\varsigma$, because rotating the output space puts the shift on one axis, and the noise on every other axis has the same distribution under both datasets (The Laplace and Gaussian mechanisms). Such a mechanism is μ-Gaussian differential privacy (GDP), and it is (ε, δ)-DP exactly when

$$\delta \geq \Phi\Big(\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big) - e^{\varepsilon}\,\Phi\Big(-\frac{\mu}{2} - \frac{\varepsilon}{\mu}\Big), \tag{39.1}$$

with Φ the standard normal distribution function (f-DP and Gaussian DP). The right side grows with μ, so each (ε, δ) has a largest admissible ratio $\mu(\varepsilon, \delta)$. At $\delta = 10^{-5}$, $\mu(2, \delta) = 0.502$ and $\mu(8, \delta) = 1.666$. The noise standard deviation for sensitivity Δ is then $\varsigma = \Delta/\mu(\varepsilon, \delta)$.

39.2.2. Fact 2 — Post-processing

Any computation on a private output that does not touch the data again is as private as the output (Post-processing, group privacy and composition). Multiplying a noisy matrix by a fixed matrix, and taking gradient steps with the result, are post-processing.

39.2.3. Fact 3 — A linear combination of independent Gaussian noise

If z has independent $\mathcal{N}(0, \varsigma^2)$ entries and ℓ is a fixed row vector, then ℓz, one number for each of the p columns, has independent coordinates, each normal with variance

$$\varsigma^2 \|\ell\|^2 = \varsigma^2 \sum_j \ell_j^2, \tag{39.2}$$

because a weighted sum of independent normals is normal and the variances of independent terms add, each multiplied by its weight squared. Applied to the rows ℓ_t of a decoder, the error of the t-th released prefix sum has standard deviation $\varsigma\|\ell_t\|$ in each coordinate. Two rows give correlated errors, with covariance $\varsigma^2\, \ell_t \cdot \ell_u$.

39.2.4. Fact 4 — Adaptive Gaussian steps compose into one Gaussian

If each of k steps is a Gaussian mechanism with ratio μ_i, and each step may depend on the outputs of the earlier ones, the run is $\sqrt{\mu_1^2 + \cdots + \mu_k^2}$-GDP (f-DP and Gaussian DP; Private empirical risk minimization restates the argument). Conditionally on the earlier outputs, each step's privacy loss is normal with the mean and variance of a one-step Gaussian loss, so the losses add like independent normals, and their variances, which are the μ_i^2, add.

39.3. Training is a stream of prefix sums

39.3.1. Context

What does a training run actually have to release, and what does privacy cost when the data order is fixed and nothing is sampled? Pinning this down turns training into a classic problem, the private release of running sums, for which much better mechanisms exist than fresh noise on every step.

39.3.2. Statement

Run gradient descent with a constant step size η and no momentum, $w_{t+1} = w_t - \frac{\eta}{B}\hat{x}_t$, where $\hat{x}_t$ is the possibly noisy clipped gradient sum of step t. Then

$$w_{t+1} = w_0 - \frac{\eta}{B}\hat{S}_t, \qquad \hat{S}_t = \sum_{s \le t} \hat{x}_s, \tag{39.3}$$

so the whole run is a post-processing of the prefix sums $\hat{S}_1, \dots, \hat{S}_T$, and the noise in w_{t+1} is η/B times the noise in $\hat{S}_t$. **FTRL** with linearized losses and a quadratic regularizer, $w_{t+1} = \arg\min_w \left(\langle \hat{S}_t, w\rangle + \frac{B}{2\eta}\|w - w_0\|^2\right)$, produces the same weights. Releasing all prefix sums is the linear workload

$$S = Ax, \qquad A = \begin{pmatrix} 1 & & & \\ 1 & 1 & & \\ \vdots & & \ddots & \\ 1 & 1 & \cdots & 1 \end{pmatrix} \in \mathbb{R}^{T \times T}, \tag{39.4}$$

where row t of Ax is S_t. For a record that takes part in one step, two mechanisms suggest themselves, each with the noise that makes it μ-GDP for its own sensitivity:

- **Independent noise on each step**, $\hat{S}_t = \sum_{u \le t}(x_u + z_u)$: sensitivity C, and error standard deviation $\sqrt{t}$ in units of C/μ for $\hat{S}_t$.
- **Independent noise on each prefix sum**, $\hat{S}_t = S_t + z_t$: sensitivity $\sqrt{T}C$, and error standard deviation $\sqrt{T}$ in the same units, for every t.

39.3.3. Derivation

1. **Unroll the recursion.** Adding the updates $w_{u+1} - w_u = -\frac{\eta}{B}\hat{x}_u$ for $u = 1, \dots, t$ telescopes to Equation 39.3. The weights at every step are a fixed function of the released prefix sums, so releasing the prefix sums privately makes every model private (Fact 2).
2. **FTRL.** The FTRL objective is a linear term plus a quadratic; setting its gradient $\hat{S}_t + \frac{B}{\eta}(w - w_0)$ to zero gives $w = w_0 - \frac{\eta}{B}\hat{S}_t$, the same point.
3. **Independent noise on each step: privacy.** The record changes only x_τ, its step, by a vector of length at most C. The released noisy gradients $x_u + z_u$ are one Gaussian mechanism per step, and only step τ moves, so the run is a Gaussian mechanism with sensitivity C; the prefix sums are post-processing of the noisy gradients. Noise $z_u \sim \mathcal{N}(0, C^2/\mu^2)$ per coordinate gives μ-GDP.

4. **Its error.** $\hat{S}_t - S_t = z_1 + \cdots + z_t$, a sum of t independent normals with variance C^2/μ^2 each, so its standard deviation is $\sqrt{t}C/\mu$ (Fact 3 with $\ell = (1, \ldots, 1, 0, \ldots, 0)$).
5. **Independent noise on each prefix sum: privacy.** The record's gradient appears in every S_t with $t \geq \tau$, so $T - \tau + 1$ of the released vectors move by the same vector of length up to C. The total Frobenius change is $\sqrt{T - \tau + 1}\, C$, at most $\sqrt{T}C$ for $\tau = 1$. By Fact 1 the noise must be $\sqrt{T}C/\mu$, and that is the error of every $\hat{S}_t$.

39.3.4. Interpretation

DP-SGD without amplification is the first mechanism. Fixed-order noisy SGD adds independent noise to every step, so its model at step t carries a random walk of t noise vectors. At the end of a run of T steps the walk has standard deviation $\sqrt{T}$ times the per-step noise. The second mechanism makes every prefix sum equally noisy, but no better than the worst prefix sum of the first. Neither exploits the structure of A: the first ignores that the noise accumulates, the second ignores that most of S_t was already released in S_{t-1}.

Why prefix sums, and not just the last model? If only the final model were released, one prefix sum, S_T, would suffice, with sensitivity C and error C/μ. But training is adaptive: the gradient x_t is computed at w_t, which depends on the earlier noisy prefix sums. The mechanism has to produce all the intermediate models anyway, and each must be private, because the later gradients are functions of them. So the workload is all T rows of A, released one at a time as the stream arrives.

Other optimizers are other workloads. With momentum or a learning-rate schedule the weights are still fixed linear combinations of the noisy gradients, only with a different lower-triangular matrix in place of A. Denisov, McMahan, Rush, Smith & Thakurta (2022) factor those matrices too. This chapter keeps plain gradient descent, so the workload is always A.

39.3.5. Example: one epoch of the running example

Take one epoch, $T = 100$ steps, and normalize $\mu = 1$, $C = 1$. Independent noise on each step gives a final prefix sum with error standard deviation $\sqrt{100} = 10$ and a root-mean-square error over the 100 prefix sums of $\sqrt{(1 + 2 + \cdots + 100)/100} = \sqrt{50.5} = 7.11$. Independent noise on each prefix sum has error 10 on every one. At $\varepsilon = 2$, $\mu = 0.502$, so the first mechanism's noise scale is $1/0.502 = 1.994$ per step, in units of C, and its final prefix sum carries noise of standard deviation 19.94 in units of C. DP-SGD with Poisson sampling at the same ε and T needs only $\sigma = 0.827$, and its final prefix sum carries $0.827\sqrt{100} = 8.27$: amplification buys a factor of 2.4 here. The rest of the chapter shows that correlated noise can beat even that, without any sampling.

39.3.6. Check yourself

- Why is the noise in the model after t steps of fixed-order noisy SGD of order $\sqrt{t}$ and not t? *(The per-step noises are independent with mean zero, so their variances add: the sum of t of them has variance t times one, and standard deviation $\sqrt{t}$ times one.)*

- With momentum, are the weights still a post-processing of the prefix sums of noisy gradients? *(Yes. Momentum combines the past noisy gradients linearly with fixed coefficients, and the noisy gradients are the differences of consecutive noisy prefix sums. A factorization of the momentum matrix itself can do better than going through prefix sums.)*

39.4. The matrix mechanism

39.4.1. Context

The two naive mechanisms put noise at opposite ends: on the inputs, or on the answers. The **matrix mechanism** of Li, Hay, Rastogi, Miklau & McGregor (2010) puts it in between, on some linear transformation Rx of the inputs from which the answers can be recovered linearly. How private is such a release, how accurate, and what does it need to work on a stream whose inputs are chosen adaptively?

39.4.2. Statement

Let $A = LR$, with $R \in \mathbb{R}^{m\times T}$ and $L \in \mathbb{R}^{T\times m}$, and release

$$\hat{S} = L(Rx + z) = Ax + Lz, \qquad z_{jk} \sim \mathcal{N}(0, s^2C^2) \text{ independent.} \tag{39.5}$$

Privacy. If a record takes part in one step, at a position fixed in advance, the release is μ-GDP with

$$\mu = \frac{\operatorname{sens}(R)}{s}, \qquad \operatorname{sens}(R) = \max_j \|r_j\|, \tag{39.6}$$

the largest Euclidean length of a column of R. This holds even when each x_t is chosen after seeing $\hat{S}_1, \dots, \hat{S}_{t-1}$, provided L and R are lower-triangular (streaming).

Error. Each coordinate of $\hat{S}_t - S_t$ is normal with mean zero and standard deviation $sC\|\ell_t\|$. At a fixed privacy level μ this is

$$\frac{C}{\mu}\, e_t, \qquad e_t = \operatorname{sens}(R)\, \|\ell_t\|, \tag{39.7}$$

and e_t, the **error per unit of privacy**, is unchanged if R is multiplied by any number $c > 0$ and L divided by it. Two summaries are used below: the root-mean-square error $\left(\frac{1}{T}\sum_t e_t^2\right)^{1/2} = \operatorname{sens}(R)\, \|L\|_F/\sqrt{T}$, and the maximum error $\max_t e_t$.

Lower bound. For every factorization,

$$\left(\frac{1}{T}\sum_t e_t^2\right)^{1/2} = \frac{\operatorname{sens}(R)\, \|L\|_F}{\sqrt{T}} \geq \frac{\|A\|_*}{T}, \tag{39.8}$$

where $\|A\|_* = \sum_k \sigma_k(A)$ is the sum of the singular values of A, its **trace norm**. For the prefix-sum matrix, $\sigma_k(A) = 1/(2\sin\frac{(2k-1)\pi}{4T+2})$ for $k = 1, \dots, T$.

39.4.3. Derivation

1. **The release is a function of $Rx + z$.** L is fixed, so $\hat{S}$ is post-processing of $Rx + z$ (Fact 2), and $LRx = Ax$ by the factorization.
2. **Sensitivity, non-adaptive inputs.** Neighbouring streams differ in the record's contribution g to x_τ, a vector of length at most C: row τ of x changes by g and nothing else. Then Rx changes by $r_\tau g^\top$, column τ of R times the row g, whose Frobenius norm is $\|r_\tau\| \|g\| \le \|r_\tau\| C$ (each entry is $r_{j\tau} g_k$, and the sum of their squares factorizes). The worst position gives $\max_j \|r_j\| C$, and Fact 1 with noise sC gives Equation 39.6.
3. **Adaptive inputs, one participation.** In training, x_t is computed at w_t, which depends on the earlier outputs. With L and R lower-triangular, row j of $Rx + z$ depends on $x_1, \dots, x_j$ only, so the mechanism can release row j of $Rx + z$ at step j, and $\hat{S}_j$ with it. Compare the two streams step by step. Before step τ the record has contributed nothing, so the outputs before τ have the same distribution under both. Condition on them: they fix w_τ, hence the record's contribution g. From then on, at any step, the other records' gradients are the same functions of the earlier outputs in both runs, and the record contributes nothing more, so given the same history the mean of each later row of $Rx + z$ differs between the two runs by the matching row of the fixed shift $r_\tau g^\top$. Released row by row, these are Gaussian mechanisms whose squared shifts add up to $\|r_\tau g^\top\|_F^2 \le \|r_\tau\|^2 C^2$, so by Fact 4 the rest of the run is Gaussian with ratio at most $\|r_\tau\| C/(sC)$, as in step 2. Averaging over the history before τ keeps the bound. Denisov et al. (2022, Theorem 2.1) prove the general statement: any factorization of a lower-triangular A that is μ-GDP, or (ε, δ)-DP, for fixed streams stays so for adaptively chosen ones, with the same parameters.
4. **Error.** $\hat{S}_t - S_t = \ell_t z$, so Fact 3 with noise sC gives standard deviation $sC\|\ell_t\|$; substituting $s = \mathrm{sens}(R)/\mu$ from Equation 39.6 gives Equation 39.7. Replacing R by cR and L by L/c multiplies $\mathrm{sens}(R)$ by c and $\|\ell_t\|$ by $1/c$.
5. **Lower bound, first half.** For any matrix U whose largest singular value is at most 1, $\mathrm{tr}(U^\top A) \le \|A\|_*$, with equality for U built from the singular vectors of A; this is the dual description of the trace norm. Write $\mathrm{tr}(U^\top LR) = \langle L^\top U, R\rangle_F \le \|L^\top U\|_F \|R\|_F \le \|L\|_F \|R\|_F$: the first step is the Cauchy–Schwarz inequality for the Frobenius inner product. For the second, $\|L^\top U\|_F = \|U^\top L\|_F$, and $U^\top$ does not lengthen any vector, since its singular values are at most 1; applied to each column of L it gives $\|U^\top L\|_F \le \|L\|_F$. So $\|L\|_F \|R\|_F \ge \|A\|_*$.
6. **Second half.** R has T columns, each of length at most $\mathrm{sens}(R)$, so $\|R\|_F^2 \le T\,\mathrm{sens}(R)^2$. Combining, $\mathrm{sens}(R)\|L\|_F \ge \|A\|_*/\sqrt{T}$; dividing by $\sqrt{T}$ gives Equation 39.8. The singular values follow from A^{-1}, which has 1 on the diagonal and -1 just below it; the eigenvalues of $(A^{-1})^\top A^{-1}$ are $4\sin^2 \frac{(2k-1)\pi}{4T+2}$. The script checks the closed form against a numerical singular value decomposition at $T = 256$: they agree to 2.0×10^{-13}.

39.4.4. Interpretation

Privacy lives in the columns, error in the rows. A column of R is where one step's gradient ends up in the noisy quantities; its length says how much noise is needed to hide it. A row of L says which noisy quantities one prefix sum is assembled from; its length says how much of the noise lands in that answer. The best factorization balances the two, and the scale invariance of step 4 means only the product matters.

The two naive mechanisms are the extreme factorizations. Independent noise on each step is $R = I$, $L = A$: columns of length 1, rows of length $\sqrt{t}$. Noise on each prefix sum is $R = A$, $L = I$: the first column has length $\sqrt{T}$, rows have length 1. For $T = 100$ their root-mean-square errors are 7.11 and 10, while Equation 39.8 says no factorization can go below 2.18. The rest of the chapter is about factorizations near that bound.

Streaming needs lower-triangular factors. If L or R had entries above the diagonal, $\hat{S}_t$ would need noise or data from future steps. Honaker (2015) improved the tree's estimates by using all of its nodes at the end, which is not lower-triangular, and for that reason his estimator cannot be used during training (Denisov et al. 2022 make this point). For any factorization there is a lower-triangular one with the same distribution of $\hat{S}$ (Denisov et al. 2022, Proposition 2.2), so nothing is lost by insisting on it.

When R is square and invertible, the mechanism is noisy SGD with correlated noise. Then $L = AR^{-1}$ and $\hat{S} = A(x + R^{-1}z)$: the mechanism adds the noise $\zeta = R^{-1}z$ to the gradients and then sums. Each step's noise ζ_t is a fixed combination of $z_1, \dots, z_t$, so implementations generate it on the fly from the stored z's, without forming any $T \times T$ matrix.

What neighbouring means here. The record's slot in the fixed order is public, and the neighbouring stream has the slot empty, contributing zero gradient. If neighbours instead replace the record by another, the contribution changes by up to $2C$, and every sensitivity doubles.

39.4.5. Example: the scale of the gap

Table 39.2 collects the error per unit of privacy, e_t of Equation 39.7, for one participation, for the mechanisms of this and the next two sections. The rows for the tree, the square root and the optimal factorization are derived in Section 39.5 and Section 39.6.

Table 39.2.: Error of each released prefix sum per unit of privacy ($\mu = 1$, $C = 1$), for one participation per record. The optimal factorization is normalized to columns of length 1 and was computed for $T \leq 256$ only.

Mechanism	$T = 100$: sens(R)	root-mean-square e_t	max e_t	$T = 10{,}000$: sens(R)	root-mean-square e_t	max e_t
Independent noise ($R = I$)	1	7.11	10.00	1	70.71	100.00
Noise on prefix sums ($R = A$)	10	10.00	10.00	100	100.00	100.00
Binary tree	2.828	5.05	6.93	3.873	9.84	13.96
Square root	1.591	2.37	2.53	2.000	3.84	4.00
Optimal (Frobenius)	1	2.24	2.45	—	—	—
Lower bound Equation 39.8	—	2.18	—	—	3.63	—

From $T = 100$ to $T = 10{,}000$, a hundredfold longer run, the independent-noise error grows tenfold, the tree's by a factor of 2 and the square root's by a factor of 1.6. At $T = 10{,}000$, independent noise is 18 times worse than the square root in root-mean-square error.

39.4.6. Check yourself

- You rescale an encoder by 2, $R \to 2R$, and the decoder by $1/2$. What happens to the noise scale needed for $\mu = 1$, and to the error? *(The sensitivity doubles, so the noise scale doubles; the decoder halves, so the error $sC\|\ell_t\|$ is unchanged.)*
- Why can the lower bound of Equation 39.8 not be reached by a factorization whose columns have very different lengths? *(Step 6 bounds $\|R\|_F^2$ by T times the largest squared column length, which is tight only when all columns are equally long. Unequal columns waste privacy on the short ones.)*

39.5. Tree aggregation and DP-FTRL

39.5.1. Context

The first mechanism with error far below $\sqrt{T}$ was the **binary tree**, found independently by Dwork, Naor, Pitassi & Rothblum (2010) and Chan, Shi & Song (2010) for counting events in a stream. Kairouz, McMahan, Song, Thakkar, Thakurta & Xu (2021) used it to release the prefix sums of gradients, with FTRL as the optimizer, and called the result DP-FTRL. How does it work, and how good is it?

39.5.2. Statement

Build a binary tree over the steps $1, \dots, T$. Level 0 has one node per step; each node at level l covers one of the blocks $1, \dots, 2^l$, then $2^l + 1, \dots, 2 \cdot 2^l$, and so on, of 2^l consecutive steps; there are $L_T = \lceil \log_2 T \rceil + 1$ levels. Release each node's sum of gradients plus independent noise. A prefix $1, \dots, t$ is the union of one node for each 1 in the binary expansion of t, so

$$\hat{S}_t = \sum_{\text{nodes in the decomposition of } t} (\text{node sum} + \text{noise}), \qquad \text{sens}(R) = \sqrt{L_T}, \qquad e_t = \sqrt{L_T}\sqrt{\text{popcount}(t)}, \tag{39.9}$$

where $\text{popcount}(t)$ is the number of 1s in the binary expansion of t, at most $L_T - 1$ for $t \le T$. Both factors grow like $\sqrt{\log T}$, so the error grows like $\log T$.

39.5.3. Derivation

1. **The factorization.** R has one row per node, with 1 in the columns of the steps the node covers: R is $m \times T$ with m about $2T$. Row t of L has a 1 for each node of the decomposition of t. Then $LR = A$, because the decomposition's nodes cover each step $1, \dots, t$ exactly once.

2. **The decomposition.** Write t in binary. Going from the highest bit down, each 1 at bit l takes the next block of 2^l steps, which starts right after the sum of the higher bits, a multiple of 2^{l+1}, so the block is a node of the tree. For example $7 = 4+2+1$ uses the nodes covering steps 1 to 4, 5 and 6, and 7; 8 uses a single node covering 1 to 8.
3. **Sensitivity.** A step lies in exactly one node per level, so column j of R has L_T ones and length $\sqrt{L_T}$.
4. **Error.** Row t of L has popcount(t) ones, so $\|\ell_t\| = \sqrt{\text{popcount}(t)}$, and Equation 39.7 gives Equation 39.9.
5. **Privacy during training.** L and R can be ordered so that each node is released when its last step has arrived, and $\hat{S}_t$ uses only completed nodes: the factorization is a streaming one, and Section 39.4 applies.

39.5.4. Interpretation

Why the tree helps. Independent noise sums t noise terms for the t-th prefix; the tree sums at most $\log_2 t + 1$ of them, each somewhat larger because every step is hidden in L_T nodes. Trading a factor $\sqrt{L_T}$ in noise for a reduction from $\sqrt{t}$ terms to $\sqrt{\log t}$ is a large win for long runs.

Why it is not the end. The tree's error jumps around: at $t = 127 = 1111111_2$ it uses seven nodes, at $t = 128$ one (Figure 39.2, left). Its encoder is also wasteful: the nodes at the top levels are used by few prefixes, yet every step pays for being in them. Honaker (2015) showed how to combine all nodes with weights to reduce the variance, but not in a streaming way (Section 39.4). The square-root factorization of the next section has smaller error everywhere.

DP-FTRL. Kairouz et al. (2021) run FTRL, which by Equation 39.3 is gradient descent from w_0 on noisy prefix sums, with the prefix sums released by the tree; they also add momentum and restart the tree after a number of steps. They report that DP-FTRL, without any amplification, compares favourably with DP-SGD with amplification in their experiments, and that it allows much more flexible data access. Xu et al. (2023) describe training production language models for the Gboard keyboard with DP-FTRL in federated learning, where Poisson sampling of devices is not available.

39.5.5. Example: $T = 100$ and $T = 10{,}000$

For $T = 100$, the tree has $L_T = 8$ levels, so $\text{sens}(R) = \sqrt{8} = 2.828$. The largest popcount up to 100 is 6 (at $t = 63$), so the largest error per unit of privacy is $\sqrt{8}\sqrt{6} = 6.93$, and the root-mean-square error is 5.05 (Table 39.2). For $T = 10{,}000$ there are 15 levels, $\text{sens}(R) = 3.873$, the maximum error is 13.96 and the root-mean-square error 9.84, against 70.71 for independent noise.

39.5.6. Check yourself

- Which prefix sums does the tree release most accurately? *(Those whose length is a power of two, which use one node, such as $t = 64$; their error is $\sqrt{L_T}$ per unit of privacy.)*
- For $T = 8$, is the tree better than independent noise? *(No. With 4 levels, $\text{sens}(R) = 2$, and the root-mean-square error is $2\sqrt{13/8} = 2.55$, against $\sqrt{4.5} = 2.12$ for independent noise. The tree only pays off for longer runs; Exercise 1 in the exercise book finds the crossover.)*

39.6. The square-root factorization

39.6.1. Context

The tree treats privacy and error asymmetrically: a tall, thin encoder and a sparse decoder. A natural symmetric choice is $L = R$, a matrix whose square is A. Is there a lower-triangular square root of A, what does it look like, and how close is it to the best possible factorization?

39.6.2. Statement

The lower-triangular matrix $R = A^{1/2}$ with $R^2 = A$ is the Toeplitz matrix (constant along each diagonal) with entries

$$R_{ij} = r_{i-j} \text{ for } i \geq j, \qquad r_k = \binom{2k}{k} 4^{-k} : \quad r = 1, \tfrac{1}{2}, \tfrac{3}{8}, \tfrac{5}{16}, \tfrac{35}{128}, \dots, \tag{39.10}$$

the coefficients of the power series $(1-x)^{-1/2} = \sum_k r_k x^k$, computed by $r_0 = 1$, $r_k = r_{k-1}\frac{2k-1}{2k}$. With $L = R$,

$$\operatorname{sens}(R)^2 = \|\ell_T\|^2 = \sum_{k<T} r_k^2, \qquad 1 + \frac{\ln \frac{2T+1}{3}}{\pi} \leq \sum_{k<T} r_k^2 \leq 1 + \frac{0.8 + \ln \frac{4T-3}{5}}{\pi}, \tag{39.11}$$

and the largest error per unit of privacy is $\max_t e_t = e_T = \sum_{k<T} r_k^2$, which grows like $\ln(T)/\pi$. As noisy SGD, the noise added to the gradients is $\zeta = R^{-1}z$, with R^{-1} the Toeplitz matrix of the coefficients h_k of $(1-x)^{1/2}$: $h = 1, -\frac{1}{2}, -\frac{1}{8}, -\frac{1}{16}, -\frac{5}{128}, \dots$.

39.6.3. Derivation

1. **Lower-triangular Toeplitz matrices are power series.** Identify the matrix with first column $(c_0, c_1, \dots, c_{T-1})$ with the series $c(x) = \sum_k c_k x^k$. The product of two such matrices has, in position (i,j), the sum $\sum_k a_{i-k} b_{k-j}$ over $j \leq k \leq i$, which is the coefficient of x^{i-j} in $a(x)b(x)$: products of matrices are products of series, truncated after x^{T-1}.
2. A **is the series** $1/(1-x)$, since its first column is all ones and $1 + x + x^2 + \cdots = 1/(1-x)$.
3. **A square root.** The series $r(x) = (1-x)^{-1/2}$ satisfies $r(x)^2 = 1/(1-x)$, so its Toeplitz matrix squares to A. Its coefficients follow from the binomial series, $(1-x)^{-1/2} = \sum_k \binom{-1/2}{k}(-x)^k$, with $(-1)^k\binom{-1/2}{k} = \frac{\frac{1}{2}\cdot\frac{3}{2}\cdots\frac{2k-1}{2}}{k!} = \frac{1\cdot 3\cdots(2k-1)}{2^k k!}$. Multiplying top and bottom by $2\cdot 4\cdots 2k = 2^k k!$ gives $\frac{(2k)!}{4^k(k!)^2} = \binom{2k}{k}4^{-k}$, and the ratio of consecutive terms is $\frac{2k-1}{2k}$.
4. **The inverse.** R^{-1} corresponds to $1/r(x) = (1-x)^{1/2}$, whose coefficients are $1, -\frac{1}{2}, -\frac{1}{8}, -\frac{1}{16}, -\frac{5}{128}, \dots$, by the binomial series again; all after the first are negative.
5. **Sensitivity and error.** Column j of R holds $r_0, \dots, r_{T-j}$, so the first column is the longest, with squared length $\sum_{k<T} r_k^2$. Row t of $L = R$ holds $r_{t-1}, \dots, r_0$, so its squared length is $\sum_{k<t} r_k^2$, largest at $t = T$. The product of the two at $t = T$ is $\sum_{k<T} r_k^2$.

6. **Bounds.** A Wallis-type inequality gives $\frac{1}{\pi(k+1/2)} \le r_k^2 \le \frac{1}{\pi(k+1/4)}$ for $k \ge 1$ (it is quoted, with a reference, by Fichtenberger, Henzinger & Upadhyay 2023; the script checks it for all $k < 100{,}000$). Compare the sums with integrals of the decreasing functions $1/(x+\frac{1}{2})$ and $1/(x+\frac{1}{4})$: $\sum_{k=1}^{T-1} \frac{1}{k+1/2} \ge \int_1^T \frac{dx}{x+1/2} = \ln\frac{2T+1}{3}$, and $\sum_{k=1}^{T-1} \frac{1}{k+1/4} \le \frac{1}{5/4} + \int_1^{T-1} \frac{dx}{x+1/4} = 0.8 + \ln\frac{4T-3}{5}$. Adding the $k=0$ term, 1, gives Equation 39.11.

39.6.4. Interpretation

Correlated noise that cancels. The noise added to step t's gradient is $z_t - \frac{1}{2}z_{t-1} - \frac{1}{8}z_{t-2} - \ldots$: fresh noise, minus a decaying share of the noise already injected. Each step's noise is a little larger than independent noise, with standard deviation $\sqrt{\sum_k h_k^2}$ times the noise scale, which tends to $\sqrt{4/\pi} = 1.128$ (the script sums 100,000 coefficients: $\sum h_k^2 = 1.2732 = 4/\pi$). But the running sums of the noise no longer drift: the negative coefficients pull the accumulated noise back toward zero. Figure 39.1 shows one coordinate at $\mu = 1$ for 1,000 steps. The per-step noises of the two mechanisms look alike; their running sums do not.

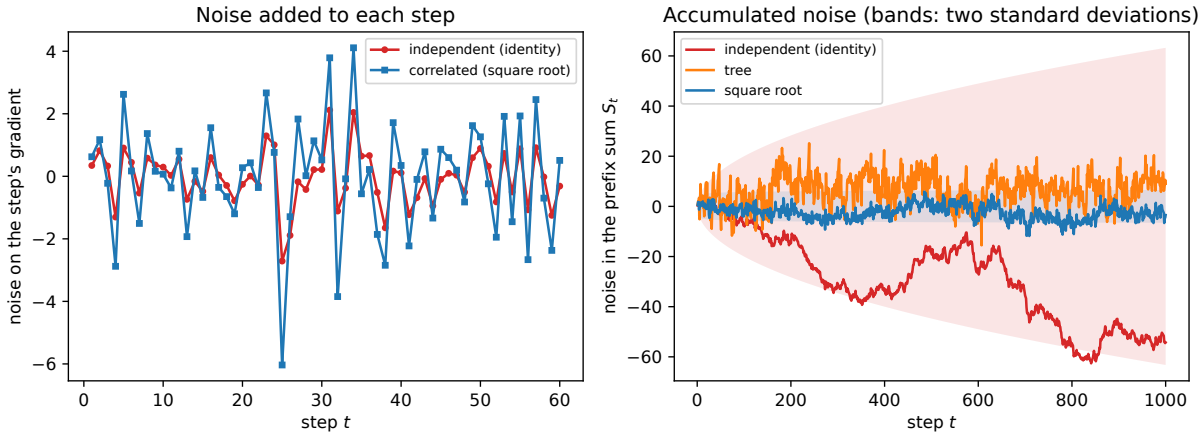


Figure 39.1.: Left: one coordinate of the noise added to each of the first 60 steps' gradients, independent (identity factorization) and square-root correlated, both calibrated to $\mu = 1$ for one participation. Right: the noise in the prefix sums over 1,000 steps, for independent noise, the binary tree and the square root, with bands of two standard deviations. Independent noise drifts like a random walk; the square root's stays within a narrow band.

How close to optimal. The lower bound Equation 39.8 is 2.18 at $T = 100$ and 3.63 at $T = 10{,}000$, and the square root's root-mean-square error is 2.37 and 3.84, within 9% and 6%. Fichtenberger, Henzinger & Upadhyay (2023) bound its maximum error with explicit constants and compare it with a lower bound that holds for every factorization, and Henzinger, Upadhyay & Upadhyay (2023) give almost tight bounds for its mean squared error; the factorization itself already appears in Bennett (1977), as they note. (The closed form printed in Theorem 1 of the arXiv version, $1 + \ln\frac{4T-3}{5}/\pi$, omits the first term of the integral comparison of step 6 and is slightly below the true sum, 2.392 against 2.531 at $T = 100$; the corrected form is the upper bound of Equation 39.11.) A finer expansion of r_k^2 around $1/(\pi k)$ gives the approximation

$\sum_{k<T} r_k^2 \approx (\ln T + \gamma + 4\ln 2)/\pi$, with γ Euler's constant, stated here without proof; the script finds 2.532 against the exact 2.531 at $T = 100$.

The optimal factorization. For the root-mean-square error, the best encoder minimizes $\|L\|_F$ subject to columns of length at most 1. Writing $X = R^\top R$, the problem is to minimize $\operatorname{tr}(A^\top A X^{-1})$ over positive definite X with diagonal at most 1, which is convex. Denisov et al. (2022, Theorem 3.2) show that the solution is $X = V^{-1/2}(V^{1/2}A^\top A V^{1/2})^{1/2}V^{-1/2}$, with V the diagonal matrix of a vector v that is the fixed point of $v \mapsto \operatorname{diag}\left((V^{1/2}A^\top A V^{1/2})^{1/2}\right)$, and they iterate that map. Setting the derivative of the Lagrangian $\operatorname{tr}(A^\top A X^{-1}) + \sum_i \lambda_i (X_{ii} - 1)$ to zero gives $X\Lambda X = A^\top A$, and the constraint $X_{ii} = 1$ turns that into exactly this fixed point. A lower-triangular R with $R^\top R = X$ comes from a Cholesky factorization of X with rows and columns reversed. The script's iteration converges in 155 steps at $T = 100$ and 193 at $T = 256$, to a root-mean-square error of 2.24 and 2.52, against 2.37 and 2.67 for the square root. The optimum costs a cubic-time computation per iteration, and the square root, which is free and nearly as good, is the usual default for long runs.

Figure 39.2 shows the error per unit of privacy across one run and against the horizon, and Figure 39.3 shows the factors for $T = 16$. The square root and the optimal factorization look alike: lower-triangular, largest on the diagonal, decaying away from it.

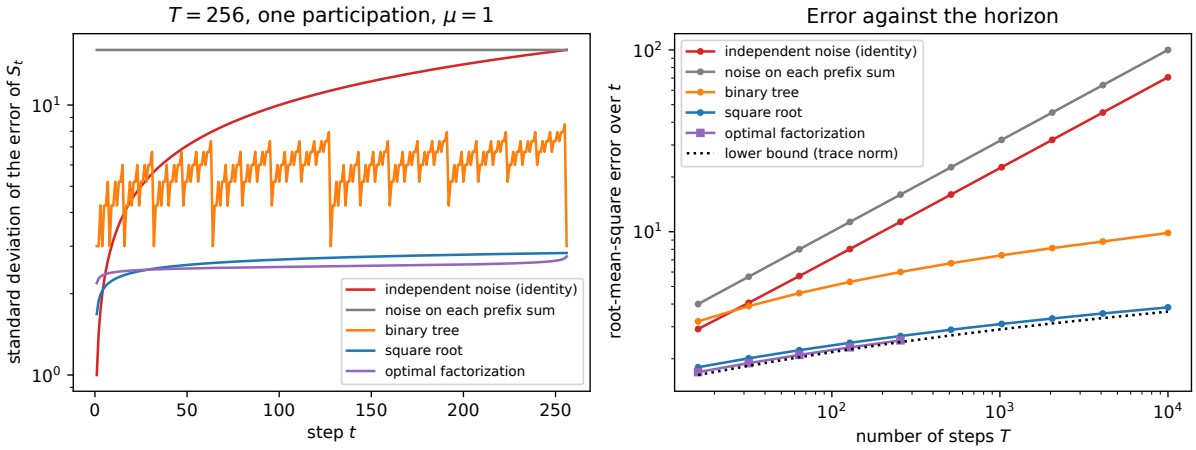


Figure 39.2.: Left: standard deviation of the error of each released prefix sum S_t for $T = 256$, one participation, at $\mu = 1$ and $C = 1$, for five factorizations. Right: the root-mean-square error over a run against its length T, with the trace-norm lower bound of Equation 39.8 dotted; the optimal factorization is computed up to $T = 256$.

39.6.5. Example: $T = 4$ by hand

With $r = (1, \frac{1}{2}, \frac{3}{8}, \frac{5}{16})$,

$$R = \begin{pmatrix} 1 & & & \\ \frac{1}{2} & 1 & & \\ \frac{3}{8} & \frac{1}{2} & 1 & \\ \frac{5}{16} & \frac{3}{8} & \frac{1}{2} & 1 \end{pmatrix}. \tag{39.12}$$

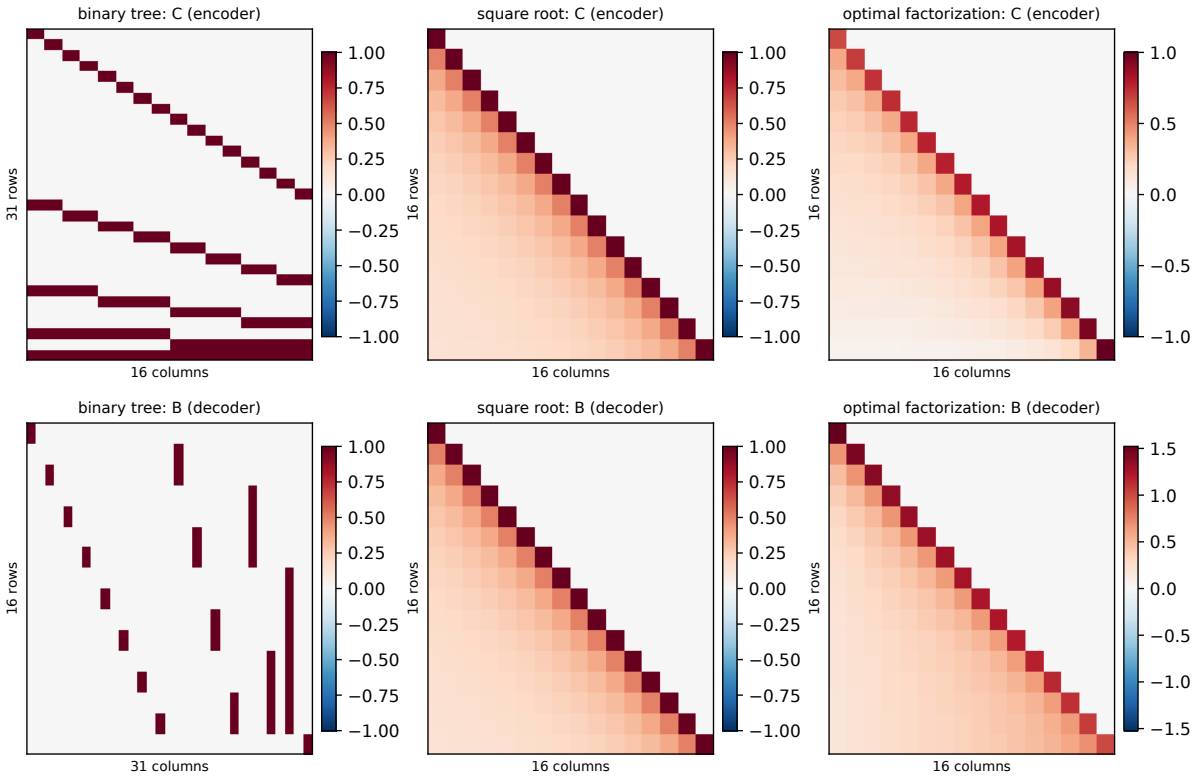


Figure 39.3.: The encoder R (top) and decoder L (bottom) of three factorizations of the 16×16 prefix-sum matrix. The tree's encoder has one row per node (31 rows), and its decoder picks the nodes of each prefix. The square root uses the same matrix for both; the optimal factorization's encoder has columns of length exactly 1.

Row 3 times column 1 of R is $\frac{3}{8}+\frac{1}{2}\cdot\frac{1}{2}+\frac{3}{8}=1$, and row 4 times column 1 is $\frac{5}{16}+\frac{3}{8}\cdot\frac{1}{2}+\frac{1}{2}\cdot\frac{3}{8}+\frac{5}{16}=1$, as $R^2=A$ requires; the script confirms $R^2=A$ exactly. The first column has squared length $1+\frac{1}{4}+\frac{9}{64}+\frac{25}{256}=\frac{381}{256}=1.488$, so $\mathrm{sens}(R)=1.220$, and the last prefix sum has error 1.488 per unit of privacy, against $\sqrt{4}=2$ for independent noise. At $T=100$ the same computation gives $\sum r_k^2=2.531$, so $\mathrm{sens}(R)=1.591$ and the final prefix sum's error is 2.53, a quarter of independent noise's 10.

39.6.6. Check yourself

- Why are the noise coefficients h_k after the first all negative, and what would happen to the prefix-sum noise if they were positive? *(They are the coefficients of $(1-x)^{1/2}$, whose binomial terms after the first are negative. Positive coefficients would add past noise again, so the prefix sums would drift faster than a random walk.)*
- The square root has $\mathrm{sens}(R)=1.591$ at $T=100$, so it needs 59% more noise per step than independent noise. Why is it still better? *(Its decoder rows have length at most 1.591 instead of up to 10: each prefix sum collects a decaying, partly cancelling combination of the noise, not t independent terms.)*

39.7. Several participations

39.7.1. Context

So far each record took part in one step, as in a single pass over the data. Training usually makes several passes, and then a record's gradient enters x at several steps, each time through a different column of R. How large is the sensitivity then, and how should the factorization change?

39.7.2. Statement

Let a record take part at steps $\tau_1<\cdots<\tau_k$, contributing vectors $g_1,\dots,g_k$ of length at most C, possibly chosen adaptively. Then the change in Rx is $\sum_i r_{\tau_i} g_i^\top$, and

$$\mathrm{sens}(R)^2=\frac{1}{C^2}\max_{\text{patterns}}\sup_{\|g_i\|\le C}\Big\|\sum_i r_{\tau_i}g_i^\top\Big\|_F^2=\max_{\text{patterns}}\Big\|\sum_i r_{\tau_i}\Big\|^2\quad\text{if } R^\top R\ge 0 \text{ entrywise,}\tag{39.13}$$

the maximum over the possible participation patterns of the length of the sum of the participating columns. As before the sensitivity is in units of C, and the release is μ-GDP with $\mu=\mathrm{sens}(R)/s$. Two special cases:

- **Independent noise**, $R=I$: $\mathrm{sens}(R)=\sqrt{k}$.
- **Banded encoders.** If R is lower-triangular with nonzero entries only within b diagonals of the main one, $R_{ij}=0$ for $i-j\ge b$, and any two participations are at least b steps apart (b-**min-separation**), then the columns of distinct participations do not overlap, and $\mathrm{sens}(R)=\big(\sum_i\|r_{\tau_i}\|^2\big)^{1/2}\le\sqrt{k}\max_j\|r_j\|$.

39.7.3. Derivation

1. **The change in Rx.** Row τ_i of x changes by g_i, so Rx changes by $\sum_i r_{\tau_i} g_i^\top$, a sum of the same column-times-row products as in Section 39.4.
2. **Expand the norm.** $\|\sum_i r_{\tau_i} g_i^\top\|_F^2 = \sum_{i,j} (r_{\tau_i} \cdot r_{\tau_j})(g_i \cdot g_j)$, because the Frobenius inner product of $ab^\top$ and $cd^\top$ is $(a \cdot c)(b \cdot d)$.
3. **The worst gradients.** Each $g_i \cdot g_j$ lies in $[-C^2, C^2]$. If every $r_{\tau_i} \cdot r_{\tau_j}$ is nonnegative, the sum is largest when every $g_i \cdot g_j = C^2$, that is when all g_i are the same vector of length C; the sum is then $C^2 \sum_{i,j} r_{\tau_i} \cdot r_{\tau_j} = C^2 \|\sum_i r_{\tau_i}\|^2$. The entries of $R^\top R$ are exactly the inner products of columns, so the condition is $R^\top R \geq 0$, which holds for any R with nonnegative entries, such as the identity, the tree and the square root. Adaptive choices of the g_i cannot exceed this supremum; Denisov et al. (2022, Theorem 2.1) state their adaptive result for exactly this sensitivity, the supremum over neighbouring streams.
4. **Independent noise.** The columns of I are orthonormal, so $\|\sum_i e_{\tau_i}\|^2 = k$.
5. **Banded encoders.** Column τ of a b-banded R is nonzero only in rows $\tau, \ldots, \tau + b - 1$. Participations at least b apart have columns with disjoint supports, so their inner products are zero, the cross terms of step 2 vanish whatever the g_i, and the sum is $\sum_i \|r_{\tau_i}\|^2 \|g_i\|^2$.

39.7.4. Interpretation

Correlated noise makes repeated participation more expensive. Under independent noise, k participations cost $\sqrt{k}$, the usual composition of Gaussian mechanisms (Fact 4). Under the square root, the columns of nearby steps overlap and point in similar directions, so a record that appears again adds its contribution on top of its earlier one, and the sensitivity grows faster than $\sqrt{k}$. Under the tree it is worse still: a record in two steps shares all the high-level nodes covering both, and in those nodes its two contributions add.

Banding restores $\sqrt{k}$. Cutting the square root's encoder to its first b diagonals, with b the separation between participations, removes the overlaps. The decoder $L = AR^{-1}$ of the banded encoder is less accurate than the square root's for a single participation, because the band no longer spreads the noise across the whole run, but with many participations the sensitivity saving dominates. Choquette-Choo, McMahan, Rush & Thakurta (2023) optimize the factorization directly under a participation constraint for multi-epoch training, and Choquette-Choo, Ganesh, McKenna, McMahan, Rush, Thakurta & Xu (2023) show that banded factorizations can also use amplification by sampling, so that they match DP-SGD where it is strong and beat it elsewhere.

The data order is part of the mechanism. The sensitivity depends on which steps a record can occupy. A fixed permutation repeated every epoch guarantees a separation of exactly n/B steps; reshuffling every epoch can put a record at the end of one epoch and the start of the next, one step apart, which breaks the banded bound. The order must be chosen independently of the data, and fixed in the analysis, but it need not be random.

39.7.5. Example: up to 20 epochs

Table 39.3 takes the running example's order: 100 steps per epoch, separation $b = 100$, E epochs, so $T = 100E$ and every record takes part E times. The banded encoder keeps the first 100

coefficients of the square root. The last column is the error of the final prefix sum per unit of privacy, e_T of Equation 39.7 with the multi-epoch sensitivity.

Table 39.3.: Sensitivity (in units of C) and error of the final prefix sum per unit of privacy, for E participations 100 steps apart, $T = 100E$.

Epochs E	Independent: sens / final e_T	Tree	Square root	Banded square root
1	1.000 / 10.00	2.828 / 4.90	1.591 / 2.53	1.591 / 2.53
2	1.414 / 20.00	4.690 / 8.12	2.531 / 4.20	2.250 / 4.16
5	2.236 / 50.00	8.944 / 21.91	5.080 / 8.86	3.558 / 8.56
10	3.162 / 100.00	16.186 / 39.65	9.154 / 16.54	5.031 / 15.69
20	4.472 / 200.00	30.430 / 74.54	17.191 / 32.09	7.115 / 29.85

The banded sensitivity at 20 epochs is $\sqrt{20} \times 1.591 = 7.115$, as step 5 predicts. The full square root's is 17.191, more than twice as large, because its long columns overlap. Yet the final errors are close, 29.85 against 32.09, because the banded decoder is less accurate; both are between a sixth and a seventh of independent noise's 200.

39.7.6. Check yourself

- Does Equation 39.13 apply to the optimal factorization of Section 39.6? *(Only if its $R^\top R = X$ has no negative entries; otherwise the worst case over the g_i must be computed with signs, and can be larger than the aligned case.)*
- Why does the tree's sensitivity grow so fast with epochs? *(A node covering many steps contains several participations of the same record, and those add before the norm is taken: a top node with 20 participations contributes 20^2 to the squared sensitivity, not 20.)*

39.8. The running example: matrix mechanisms against DP-SGD

39.8.1. Context

The previous sections compared mechanisms by the error in released sums. Does that translate into accuracy when a network is trained, and how do correlated-noise mechanisms without sampling compare with DP-SGD with sampling at the same ε?

39.8.2. Statement

Table 39.4 gives the noise each method needs for $\varepsilon = 2$ and 8 at $\delta = 10^{-5}$, and the standard deviation of the noise carried by the final prefix sum, in units of C. Table 39.5 gives the test accuracy, at the better of two step sizes, 0.5 and 2, averaged over three runs.

Table 39.4.: Noise needed by each method at equal ε ($\delta = 10^{-5}$, 100 steps per epoch) and the resulting noise in the final prefix sum, all in units of C. DP-SGD uses Poisson sampling with $q = 0.01$ and the Rényi differential privacy (RDP) accountant; the others use a fixed order and exact Gaussian accounting.

ε, epochs	DP-SGD σ	Independent s	Tree s	Square root s	Banded s	Final prefix noise: DP-SGD / independent / tree / square root / banded
2, 1	0.827	1.994	5.639	3.172	3.172	8.27 / 19.94 / 9.77 / 5.05 / 5.05
2, 5	0.928	4.458	17.833	10.129	7.093	20.75 / 99.69 / 43.68 / 17.67 / 17.06
2, 10	1.03	6.305	32.273	18.251	10.031	32.57 / 199.38 / 79.05 / 32.98 / 31.28
8, 1	0.501	0.600	1.698	0.955	0.955	5.01 / 6.00 / 2.94 / 1.52 / 1.52
8, 5	0.573	1.342	5.369	3.049	2.135	12.81 / 30.01 / 13.15 / 5.32 / 5.14
8, 10	0.616	1.898	9.716	5.495	3.020	19.48 / 60.02 / 23.80 / 9.93 / 9.42

Table 39.5.: Test accuracy (mean ± standard deviation over three runs) at equal ε, $\delta = 10^{-5}$, batch 100, clipping norm 0.5, best of step sizes 0.5 and 2. Without noise, fixed-order training with clipping reaches 0.9420, 0.9580 and 0.9606 after 1, 5 and 10 epochs.

ε, epochs	DP-SGD (Poisson)	Independent	Tree	Square root	Banded square root
2, 1	0.9343 ± 0.0044	0.9066 ± 0.0031	0.9134 ± 0.0038	0.9361 ± 0.0046	0.9358 ± 0.0019

ε, epochs	DP-SGD (Poisson)	Independent	Tree	Square root	Banded square root
2, 5	0.9460 ± 0.0021	0.8941 ± 0.0071	0.9183 ± 0.0084	0.9430 ± 0.0011	0.9456 ± 0.0019
2, 10	0.9493 ± 0.0012	0.8641 ± 0.0130	0.9096 ± 0.0074	0.9398 ± 0.0029	0.9491 ± 0.0024
8, 1	0.9371 ± 0.0016	0.9395 ± 0.0018	0.9412 ± 0.0006	0.9461 ± 0.0022	0.9434 ± 0.0017
8, 5	0.9524 ± 0.0026	0.9420 ± 0.0029	0.9461 ± 0.0012	0.9497 ± 0.0025	0.9542 ± 0.0014
8, 10	0.9520 ± 0.0006	0.9410 ± 0.0032	0.9484 ± 0.0004	0.9530 ± 0.0015	0.9549 ± 0.0024

39.8.3. Derivation

Each entry is computed as the earlier sections prescribe. For DP-SGD, the RDP accountant of DP-SGD finds the smallest σ for $q = 0.01$, $T = 100E$ steps and the target ε; its final prefix sum carries T independent noises, $\sigma\sqrt{T}$. For the fixed-order methods, the sensitivity of Equation 39.13 for E participations 100 steps apart (Table 39.3) is divided by $\mu(\varepsilon, \delta)$ from Fact 1 to give the noise scale s; for independent noise this is Fact 4, $s = \sqrt{E}/\mu$. The final prefix sum's noise is s times the length of the last row of the decoder (Fact 3). Training then runs Equation 39.3 with the noise of each factorization: the per-step noise $R^{-1}z$ for the square-root and banded encoders, generated by a convolution in time, and the tree's node noises summed along the binary decompositions, with consecutive differences taken as the per-step noise.

39.8.4. Interpretation

Without amplification, correlated noise is essential. At $\varepsilon = 2$, fixed-order SGD with independent noise loses 2.8 points to DP-SGD after one epoch and 8.5 points after ten (0.8641 against 0.9493), and it gets worse with more epochs, because its noise grows like $\sqrt{E}$ per step and accumulates like $\sqrt{T}$. The tree recovers between a quarter and a half of the gap. The square root and the banded square root recover nearly all of it: at ten epochs the banded encoder reaches 0.9491, within 0.0002 of DP-SGD.

Matrix mechanisms can beat amplified DP-SGD. For short runs and large ε, amplification gains little. At $\varepsilon = 8$ and one epoch, DP-SGD needs $\sigma = 0.501$ and independent noise without sampling 0.600, barely more, and the square root, whose final prefix sum carries 1.52 against DP-SGD's 5.01, wins outright (0.9461 against 0.9371). At $\varepsilon = 8$ the banded encoder has the highest mean accuracy at five and ten epochs. At $\varepsilon = 2$ and one epoch the square root's final noise, 5.05, is also below DP-SGD's 8.27, and its accuracy is the same within the run-to-run spread.

Where DP-SGD keeps its lead. At small ε over many epochs, amplification is strongest: DP-SGD's per-step noise grows only from 0.827 to 1.03 between one and ten epochs at $\varepsilon = 2$, while every fixed-order mechanism pays for ten participations. The full square root then falls behind (0.9398 against 0.9493), because its overlapping columns make repeated participation

expensive, and only banding keeps up. The final-prefix noise is a good guide: the method with the least of it is the most accurate or within the spread of the most accurate in every row.

The comparison is slightly tilted. DP-SGD is accounted with the RDP accountant, which overstates ε somewhat compared with a numerical accountant (Choosing an accountant), while the fixed-order mechanisms use the exact Gaussian curve of Fact 1. A tighter DP-SGD accountant would lower its noise somewhat. The step size was chosen per method from two values by test accuracy, which is itself a leak that Hyperparameter tuning accounts for; here it serves only to compare methods at their best. The runs are short, at most 1,000 steps, to keep the script under a few minutes. The noise gaps of Table 39.3 keep widening with the number of epochs, so longer runs should separate the methods more, not less.

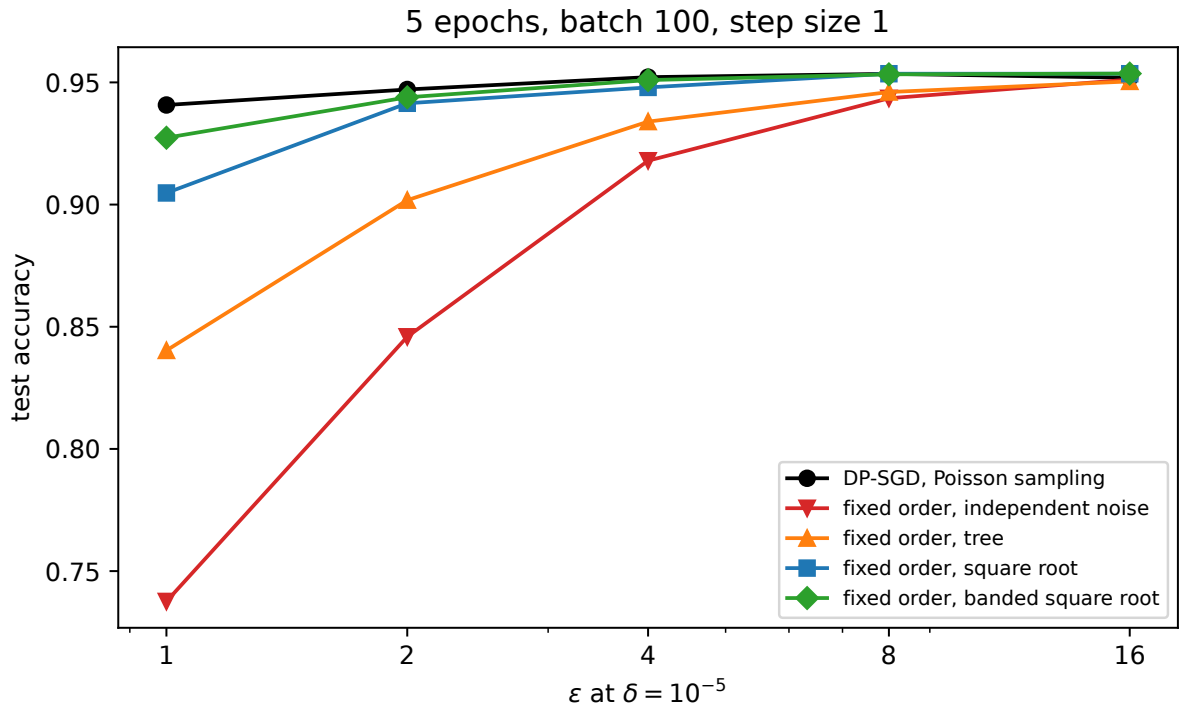


Figure 39.4.: Test accuracy of the running example after 5 epochs (500 steps of batch 100) at step size 1, against ε at $\delta = 10^{-5}$, one run per point. DP-SGD uses Poisson sampling; the other four use a fixed data order and differ only in how their noise is correlated.

Figure 39.4 shows the same pattern across budgets at five epochs: independent noise collapses at small ε, the tree is better, and the two square-root encoders track DP-SGD, with the banded one closest.

39.8.5. Example: what a federated deployment gains

In federated learning a "record" is a device, which checks in when it is idle, charging and on Wi-Fi; the server cannot sample devices independently with a known rate, so the Poisson accountant does not apply. Suppose the server can at least enforce that a device contributes at most once every b rounds, a b-min-separation. Then the banded matrix mechanism of Section 39.7 gives a provable

guarantee with sensitivity $\sqrt{k}$ times one column for k participations, and on the running example's scale it is as accurate as DP-SGD with sampling would be. That is why production federated training of language models uses DP-FTRL (Xu et al. 2023), and why later work optimizes matrix factorizations for user-level training of language models (Choquette-Choo, McMahan, Rush & Thakurta 2023). The privacy unit there is a user, not an example, which User-level differential privacy treats.

39.8.6. Check yourself

- In Table 39.5, independent noise at $\varepsilon = 2$ gets worse from one epoch to ten. Why does more training hurt it? *(Each extra epoch adds a participation, so the noise scale grows like $\sqrt{E}$, from 1.994 to 6.305 between one and ten epochs: every update carries three times more noise, while each update's signal stays the same size or shrinks as the model converges.)*
- When would you still choose DP-SGD over a matrix mechanism? *(When you can implement Poisson sampling honestly and ε is small with many epochs, where amplification is strongest; or when the participation pattern is unknown, so no sensitivity for a correlated mechanism can be computed. Banded mechanisms with amplification combine both.)*

39.9. Code: check every number by computation

The script code/matrix_mechanisms.py builds the identity, prefix, tree, square-root and optimal factorizations, computes their sensitivities and errors (by structured formulas, checked against dense matrices at $T = 256$ to a relative difference of 3.3×10^{-16}), computes multi-epoch sensitivities, and trains the running example in a fixed order with each mechanism's correlated noise, against DP-SGD from code/dp_sgd.py. From the repository root:

```
uv run python code/part05-dp-ml/matrix_mechanisms.py
```

Output (seed 0), summarized in Table 39.6:

Table 39.6.: The script's output with the default seed.

Experiment	Result
Square-root coefficients and inverse	$r = 1, 0.5, 0.375, 0.3125, ...$; $h = 1, -0.5, -0.125, -0.0625, ...$; $R^2 = A$ exactly at $T = 4$
One participation, $T = 100$	Root-mean-square error 7.11 (independent), 5.05 (tree), 2.37 (square root), 2.24 (optimal), bound 2.18
One participation, $T = 10{,}000$	70.71, 9.84, 3.84; bound 3.63
20 epochs, sensitivity	4.472 (independent), 30.430 (tree), 17.191 (square root), 7.115 (banded)
Accuracy, $\varepsilon = 2$, 10 epochs	DP-SGD 0.9493, independent 0.8641, tree 0.9096, square root 0.9398, banded 0.9491

Experiment	Result
Accuracy, $\varepsilon = 8$, 1 epoch	DP-SGD 0.9371, independent 0.9395, square root 0.9461

tests/test_matrix_mechanisms.py checks that the output still matches. The script takes about two minutes of processor time, most of it for the 198 training runs behind Table 39.5.

What to take from it. A correlated-noise mechanism is a few lines on top of noisy SGD: store the past noise, and add a fixed combination of it to each step. What takes care is the sensitivity, which depends on the data order and the number of participations, and must be computed for the order you actually use.

39.10. References

- Dwork, Naor, Pitassi & Rothblum, "Differential Privacy Under Continual Observation" (STOC 2010), and Chan, Shi & Song, "Private and Continual Release of Statistics" (ACM Transactions on Information and System Security, 2011; conference version 2010). The binary tree for private running counts.
- Li, Hay, Rastogi, Miklau & McGregor, "Optimizing Linear Counting Queries Under Differential Privacy" (PODS 2010); journal version Li, Miklau, Hay, McGregor & Rastogi, "The Matrix Mechanism" (VLDB Journal 2015). Factoring a query workload to choose where noise goes.
- Kairouz, McMahan, Song, Thakkar, Thakurta & Xu, "Practical and Private (Deep) Learning without Sampling or Shuffling" (ICML 2021). DP-FTRL with tree aggregation; the comparison with amplified DP-SGD.
- Denisov, McMahan, Rush, Smith & Thakurta, "Improved Differential Privacy for SGD via Optimal Private Linear Operators on Adaptive Streams" (NeurIPS 2022). The matrix mechanism for adaptive streams (Theorem 2.1), lower-triangular factorizations, and the fixed point for the optimal factorization.
- Fichtenberger, Henzinger & Upadhyay, "Constant Matters: Fine-grained Error Bound on Differentially Private Continual Observation" (ICML 2023), and Henzinger, Upadhyay & Upadhyay, "Almost Tight Error Bounds on Differentially Private Continual Counting" (SODA 2023). The square-root factorization and how close it is to optimal, for the maximum and the mean squared error.
- Choquette-Choo, McMahan, Rush & Thakurta, "Multi-Epoch Matrix Factorization Mechanisms for Private Machine Learning" (ICML 2023). Sensitivity under several participations and factorizations optimized for it.
- Choquette-Choo, Ganesh, McKenna, McMahan, Rush, Thakurta & Xu, "(Amplified) Banded Matrix Factorizations: A Unified Approach to Private Training" (NeurIPS 2023). Banded encoders, min-separation, and amplification for matrix mechanisms.
- Honaker, "Efficient Use of Differentially Private Binary Trees" (2015). The variance-optimal, but not streaming, estimator from a tree.
- Xu, Zhang, Andrew, Choquette-Choo, Kairouz, McMahan, Rosenstock & Zhang, "Federated Learning of Gboard Language Models with Differential Privacy" (ACL 2023, industry track). DP-FTRL in a production federated system.

39.11. Cheat sheet

T steps, clipping norm C, workload A (prefix sums), factorization $A = LR$, noise z with standard deviation sC, one participation unless stated. Table 39.7 collects the results.

Table 39.7.: Every result of the chapter.

Result	Formula	Conditions
Training releases prefix sums	$w_{t+1} = w_0 - \frac{\eta}{B}\hat{S}_t$	Gradient descent or FTRL, no momentum
Matrix mechanism	$\hat{S} = L(Rx + z) = Ax + Lz$	L, R lower-triangular for streaming
Privacy	$\mu = \max_j \|r_j\|/s$	One participation; adaptive inputs allowed
Error	$\mathrm{sd}(\hat{S}_t - S_t) = sC\|\ell_t\| = \frac{C}{\mu}\max_j \|r_j\|\,\|\ell_t\|$	Per coordinate
Lower bound	$\max_j \|r_j\|\,\|L\|_F/\sqrt{T} \geq \|A\|_*/T$	Any factorization
Independent noise	$R = I$: sens 1, error $\sqrt{t}$	DP-SGD without amplification
Binary tree	sens $\sqrt{\lceil \log_2 T \rceil + 1}$, error $\times\sqrt{\mathrm{popcount}(t)}$	DP-FTRL
Square root	$r_k = \binom{2k}{k}4^{-k}$; $\mathrm{sens}^2 = \max$ error $= \sum_{k<T} r_k^2 \approx (\ln T + 3.35)/\pi$	$L = R = A^{1/2}$; noise $\zeta = R^{-1}z$
Several participations	$\mathrm{sens} = \max \|\sum_i r_{\tau_i}\|$	$R^\top R \geq 0$
Banded encoder	$\mathrm{sens} = \sqrt{\sum_i \|r_{\tau_i}\|^2}$	Band $\leq$ separation of participations

Rules of thumb

- If you cannot implement Poisson sampling, do not account as if you did. Fix the data order in advance and use a matrix mechanism.
- For one pass over the data, use the square-root factorization: it is within about 10% of the best possible error and costs nothing to compute.
- For several passes, fix the order so participations are evenly separated, and use a banded encoder whose band is at most the separation.
- Compute the sensitivity for the participation pattern you actually have; reshuffling every epoch changes it.
- Compare methods by the noise in the final prefix sum at equal ε: on the running example, the method with the least of it was the most accurate, or within the run-to-run spread of the most accurate, in every configuration.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

40. Private prediction: PATE and in-context learning

40.1. Why this matters

Training a model with differential privacy (DP) by stochastic gradient descent (SGD) with clipped and noisy gradients, DP-SGD (DP-SGD), makes the trained weights private. Every later use of the model is then free: anyone may download the weights and query them as often as they like, because releasing the weights was the private step and everything after it is post-processing. The price is paid during training. Every step adds noise to every gradient, the training code must compute and clip per-example gradients, and you must own the training loop.

Sometimes that price is the wrong one to pay, or cannot be paid at all. The model may be a large language model (LLM) behind an application programming interface, which you can prompt but not train. It may be a model that someone else trains, or one with no gradients, such as a forest of decision trees. Or it may answer only a few dozen questions in its whole life, a medical consultation tool for one study, for which a private training run is overkill. In each case there is another option: train ordinary, non-private models, **never release them**, and make each *answer* private instead. A query goes in, a differentially private answer comes out, and the models themselves stay behind a wall.

This chapter asks when that is the better choice and how to make it work. It answers with DP applied to predictions rather than to weights. Two questions organize it. How much does each private answer cost, and how many can you afford? And how do you turn a stream of private answers into a private model, so that the cost stops growing? The answer to the second question is private aggregation of teacher ensembles (PATE), introduced by Papernot, Abadi, Erlingsson, Goodfellow and Talwar (2017). Its modern descendant privatizes in-context learning: the private examples are placed in an LLM's prompt rather than used for training.

40.1.1. The one idea: one record, one vote

Split the private records into k disjoint shards, and train one model, a **teacher**, on each. To answer a query x, let every teacher vote for a label and release only the label with the most votes after adding noise to the counts. A record lives in exactly one shard, so it influences exactly one teacher, and that teacher casts exactly one vote. Adding or removing the record can therefore move at most one vote from one label to another, whatever the teachers are, however they were trained, and however badly one of them overfits its shard. The privacy proof stops there. It is the subsample-and-aggregate idea of Beyond global sensitivity, with a vote as the aggregator.

Everything else is about accuracy, and it comes down to one comparison: the **gap** between the top two vote counts against the noise. When the teachers agree, the gap is large and the noisy winner is almost always the true plurality. When they disagree, the noise decides. And the noise must grow like the square root of the number of answered queries, because each answer spends

budget. So private prediction is cheap for a few queries on which the teachers agree, and expensive otherwise. Table 40.1 lists the results.

Table 40.1.: The results of this chapter and where each is used.

Result	Where it is used
The noisy vote, LNMax and GNMax, and its cost per answer	Every private-prediction system; the noise needed for Q queries
The accuracy of a noisy vote: the gap against the noise	Choosing the number of teachers
Private answers against a private model: the break-even number of queries	Deciding between private prediction and DP-SGD
PATE: a student trained on privately labelled public data	Turning answers into a model whose use is free
Data-dependent analysis and Confident-GNMax	Paying less for answers the teachers agree on; what must be sanitized
Private in-context learning	Prompting an LLM with private examples

Notation. A private dataset D has n records and is split into k disjoint shards of n/k records, one per teacher. A query x has m possible labels; $n_i(x)$ is the number of teachers that vote for label i, so $\sum_i n_i(x) = k$, and i^* is the label with the most votes, the **plurality**. The **gap** is the difference between the top count and the runner-up; on two labels it is $|n_1 - n_2|$. The noise on each count is $\mathcal{N}(0, \sigma^2)$ for GNMax and $\mathrm{Lap}(b)$ for LNMax. Q is the number of answered queries, $\alpha > 1$ an order of Rényi differential privacy (RDP), and $\tilde{q}$ an upper bound on the probability that the noisy winner differs from the plurality. Neighbouring datasets differ by **adding or removing** one record, as in DP-SGD, and $\delta = 10^{-5}$ throughout.

The running example. The part's neural-network task (DP-SGD): 50-dimensional standard normal inputs labelled ± 1 by a fixed random network, $n = 10{,}000$ private training records. Its 20,000 test records are split in two. The first 10,000 form a **public pool**: their inputs are treated as public and unlabelled, as PATE requires, and their labels are used only to measure how good the private labels are. The last 10,000 form the **evaluation set**, on which every accuracy in this chapter is measured. The teachers are regularized logistic regressions, the model of Private empirical risk minimization, each fitted on one shard of consecutive records. Logistic regression is cheap and, on this task, as accurate as the part's network on the same few records.

How to read this chapter. Read the toolkit and Section 40.3, which give the mechanism and its cost. Section 40.4 and Section 40.5 answer the question in the title: when private answers beat a private model. Section 40.6 builds PATE, and Section 40.7 the analysis that makes it competitive in practice; the second is the most technical section and can be skimmed for its statement and interpretation. Section 40.8 applies everything to in-context learning.

40.2. Toolkit: four facts every derivation uses

40.2.1. Fact 1 — One record changes one vote

Fix the partition of records into shards *before* looking at the data, by a hash of a record identifier rather than by position. Adding or removing a record then changes one shard, the one its hash points to, and leaves every other shard, and so every other teacher, exactly as it was. That teacher may now vote differently on a query: one count goes down by 1 and another goes up by 1, or nothing changes. The vote histogram $(n_1(x), \dots, n_m(x))$ therefore moves by at most 2 in ℓ_1 norm, the sum of absolute changes, and by at most $\sqrt{1^2 + 1^2} = \sqrt{2}$ in ℓ_2 norm, the Euclidean length of the change. This is the step of subsample-and-aggregate in Beyond global sensitivity: one record touches one block.

The partition rule matters. If shards were cut by position, inserting one record at the front would shift every later record into the next shard, change every shard, and possibly change every teacher. The derivations below assume a fixed hash; the script splits the records by position only because they are never added or removed in its experiments.

40.2.2. Fact 2 — The Gaussian mechanism in Rényi DP, and its accountant

Adding $\mathcal{N}(0, s^2)$ noise to every coordinate of a vector-valued function whose value moves by at most Δ in ℓ_2 norm between neighbours gives a mechanism whose Rényi divergence of order α between neighbours is at most $\alpha\Delta^2/(2s^2)$ (Rényi DP). The **Rényi divergence** of order α between two distributions P and P' on a discrete set of outputs is

$$D_\alpha(P\|P') = \frac{1}{\alpha - 1} \ln \sum_y P(y)^\alpha P'(y)^{1-\alpha}, \tag{40.1}$$

and a mechanism is $(\alpha, \varepsilon(\alpha))$-RDP if this is at most $\varepsilon(\alpha)$ for every pair of neighbours. Three properties make RDP the accountant here. Any computation on a private output that does not touch the data again, **post-processing**, cannot increase the divergence. RDP curves **add** under composition, even when each mechanism is chosen after seeing the earlier outputs. And a composed curve converts to (ε, δ)-DP at the best order,

$$\varepsilon = \min_\alpha \left[\varepsilon(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1} \right] \tag{40.2}$$

(Balle et al. 2020; From Rényi DP to (ε, δ)). When the curve is a line, $\varepsilon(\alpha) = \rho\alpha$, the result depends only on its slope ρ: at $\delta = 10^{-5}$, the largest slope that converts to $\varepsilon = 1$ is $\rho = 0.0306$, and to $\varepsilon = 4$ it is $\rho = 0.3731$. A pure Gaussian mechanism also has an exact conversion: its privacy is that of one Gaussian shift by $\mu = \Delta/s$ standard deviations, Gaussian differential privacy (GDP) with parameter μ, and composing Q of them gives $\mu\sqrt{Q}$ (f-DP and Gaussian DP).

40.2.3. Fact 3 — Report-noisy-max, and the RDP curve of any pure-DP step

Report-noisy-max adds independent Laplace noise $\mathrm{Lap}(b)$ to every candidate's score and releases only the index of the largest noisy score (Selection). If one record moves each score by at most Δ, it is $(2\Delta/b)$-DP. The reason is that, with every other candidate's noise held fixed, candidate i

wins exactly when its own noise clears a bar, the best of the others' noisy scores minus its own score. One record moves that bar by at most 2Δ, Δ through the maximum of the others and Δ through candidate i's own score, and moving a Laplace threshold by s changes its tail probability by at most $e^{s/b}$. When all scores move in the same direction, the factor 2 disappears. A vote histogram is not of that kind: one count goes up while another goes down.

An ε_0-DP mechanism also has an RDP curve. The worst ε_0-DP mechanism at every order is randomized response, which reports a bit truthfully with probability $p = e^{\varepsilon_0}/(1+e^{\varepsilon_0})$, so its curve,

$$\varepsilon_{\mathrm{RR}}(\alpha) = \frac{1}{\alpha-1}\ln\left(p^\alpha(1-p)^{1-\alpha} + (1-p)^\alpha p^{1-\alpha}\right), \tag{40.3}$$

is a valid RDP curve for any ε_0-DP step (Concentrated differential privacy). This lets Laplace answers enter the same accountant as Gaussian ones.

40.2.4. Fact 4 — The difference of two independent Gaussians

If Z_1 and Z_2 are independent $\mathcal{N}(0,\sigma^2)$, then $Z_1 - Z_2$ is $\mathcal{N}(0,2\sigma^2)$: variances of independent variables add, and subtracting does not change a variance. So $\Pr[Z_1 - Z_2 > t] = \Phi(-t/(\sqrt{2}\sigma))$, where Φ is the standard normal cumulative distribution function. In terms of the complementary error function erfc, which numerical libraries provide, $\Phi(-u) = \frac{1}{2}\operatorname{erfc}(u/\sqrt{2})$, so $\Pr[Z_1 - Z_2 > t] = \frac{1}{2}\operatorname{erfc}(t/(2\sigma))$. Every accuracy statement about GNMax uses this, because whether a noisy vote picks label i over label j depends only on the difference of their two noises.

40.3. The noisy vote: LNMax and GNMax

40.3.1. Context

A query arrives, the k teachers vote, and you must release one label. Which mechanism releases it, and how much does each answer cost? Two mechanisms are used in practice, and they differ in their noise: Laplace noise in the original PATE paper, Gaussian noise in its successor. The question is also how the cost of Q answers grows with Q, since that is what limits private prediction.

40.3.2. Statement

LNMax releases $\arg\max_i\,(n_i(x) + \mathrm{Lap}(b))$ and **GNMax** releases $\arg\max_i\,(n_i(x) + \mathcal{N}(0,\sigma^2))$, with independent noise on every count. Under add/remove neighbours and a fixed partition:

1. Each LNMax answer is $(2/b)$-DP (Papernot et al. 2017).
2. Each GNMax answer is $(\alpha, \alpha/\sigma^2)$-RDP for every $\alpha > 1$ (Papernot, Song, Mironov, Raghunathan, Talwar & Erlingsson 2018, Proposition 8).
3. Q GNMax answers, even with queries chosen adaptively, are $(\alpha, Q\alpha/\sigma^2)$-RDP. To meet a budget (ε,δ) the noise must therefore be

$$\sigma = \sqrt{\frac{Q}{\rho(\varepsilon,\delta)}}, \tag{40.4}$$

where $\rho(\varepsilon, \delta)$ is the largest slope whose line $\rho\alpha$ converts to (ε, δ) by Equation 40.2. The noise grows like $\sqrt{Q}$.

40.3.3. Derivation

1. **The histogram's sensitivity.** By Fact 1, one record moves the vote vector by at most 2 in ℓ_1 norm and $\sqrt{2}$ in ℓ_2 norm.
2. **LNMax is report-noisy-max.** The counts are the scores, each moves by at most $\Delta = 1$, and they can move in opposite directions. Fact 3 with $\Delta = 1$ gives $(2/b)$-DP.
3. **GNMax is post-processing of a Gaussian histogram.** Adding $\mathcal{N}(0, \sigma^2)$ to every count is the Gaussian mechanism on a vector with ℓ_2 sensitivity $\sqrt{2}$, so by Fact 2 its RDP is $\alpha(\sqrt{2})^2/(2\sigma^2) = \alpha/\sigma^2$. Taking the argmax and discarding the noisy counts is post-processing, which cannot increase it.
4. **Compose.** Each answer is $(\alpha, \alpha/\sigma^2)$-RDP whatever the earlier answers were, so Q answers add up to $Q\alpha/\sigma^2$ (Fact 2). This holds even if the analyst chooses query t after seeing answers $1, \dots, t-1$.
5. **Calibrate.** The composed curve is the line $\rho\alpha$ with $\rho = Q/\sigma^2$, and its ε depends only on ρ. Setting $Q/\sigma^2 = \rho(\varepsilon, \delta)$ and solving for σ gives Equation 40.4.

40.3.4. Interpretation

The cost is per answer, not per teacher. Neither the number of teachers nor what they are enters the privacy cost. A teacher can be a neural network trained without any noise, a decision forest, or a human annotator who saw one shard. The cost is set by the noise and the number of answers, and the noise can be large in absolute terms because the counts themselves are large: with 250 teachers, noise of standard deviation 40 on each count is noticeable but not overwhelming.

Gaussian noise wins for many queries, Laplace for very few. A Laplace answer is analysed as a pure-DP step, which pays for the worst case of every answer; a Gaussian answer has a privacy loss that is itself Gaussian and composes exactly. For one answer the pure-DP analysis is cheap, and as answers accumulate the Gaussian's analysis wins. Table 40.2 compares the noise each needs, expressed as a standard deviation ($\sqrt{2}b$ for Laplace). For a single answer, Laplace needs less noise. From 100 answers on, Gaussian noise is smaller by a factor close to 2 at both budgets, and the gap persists, since both grow like $\sqrt{Q}$ once the composition is in its square-root regime.

Only the label is released. As in report-noisy-max, publishing the noisy counts as well would release a vector, and step 3 would no longer apply to the published output; the analysis would then have to pay for all m counts as a release in their own right. GNMax happens to have a valid analysis for the noisy counts too, since it is a Gaussian mechanism, but the data-dependent analysis of Section 40.7 needs the label alone.

How tight is the RDP accountant here? A Gaussian histogram has an exact conversion (Fact 2). At the σ the RDP accountant picks for $\varepsilon = 1$, the exact ε is 0.915, and for $\varepsilon = 4$ it is 3.698: the accountant overstates ε by about 9% and 8%. The ratio is the same for every Q because both depend only on Q/σ^2.

40.3.5. Example: the noise for Q answers

Table 40.2 gives the noise for four numbers of answers at $\varepsilon = 1$ and $\varepsilon = 4$.

Table 40.2.: Noise per count for Q answers at $(\varepsilon, 10^{-5})$, from the RDP accountant; Laplace answers are charged the randomized-response curve of Fact 3.

Budget	Answers Q	GNMax σ	LNMax b	LNMax standard deviation $\sqrt{2}b$
$\varepsilon = 1$	1	5.72	2.02	2.85
	100	57.2	79.7	113
	1,000	181	255	361
	10,000	572	809	1,140
$\varepsilon = 4$	1	1.64	0.501	0.709
	100	16.4	22.7	32.1
	1,000	51.8	73.1	103
	10,000	164	231	327

The $\sqrt{Q}$ law of Equation 40.4 is exact here: from 100 to 10,000 answers, a hundredfold increase, σ grows tenfold, from 57.2 to 572. That is what makes 10,000 answers at $\varepsilon = 1$ hopeless for any ensemble of fewer than a few thousand teachers: noise of standard deviation 572 on each count swamps any vote.

Figure 40.1 shows what the noise does to real votes. With $k = 100$ teachers on the running example and the noise for 1,000 answers at $\varepsilon = 4$, $\sigma = 51.8$, a query on which 98 teachers vote -1 and 2 vote $+1$, a gap of 96, gets the plurality label with probability 0.9051. A query split 70 to 30, gap 40, gets it with probability 0.7076, and one split 52 to 48, gap 4, with probability 0.5218, barely better than a coin. Across the public pool, the fraction of teachers voting $+1$ piles up near 0 and 1, but a large share of queries sits in between.

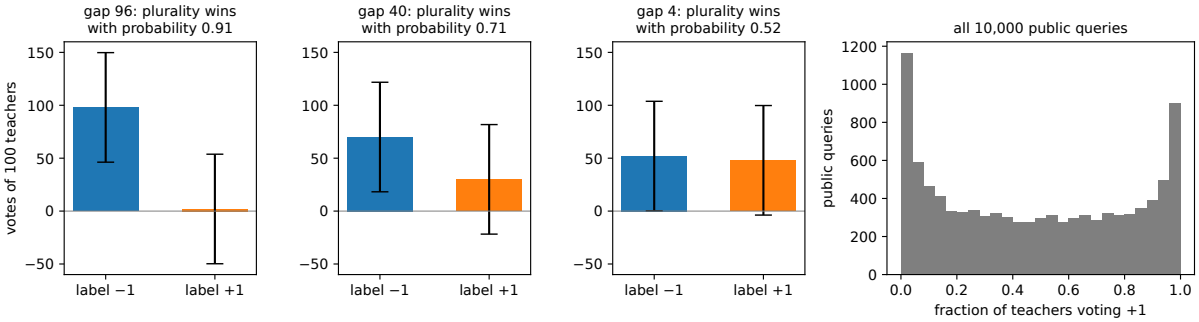


Figure 40.1.: Left three panels: the votes of 100 teachers on three public queries, with bars of one noise standard deviation ($\sigma = 51.8$, the GNMax noise for 1,000 answers at $\varepsilon = 4$); the title gives the probability that the noisy vote returns the plurality label. Right: across all 10,000 public queries, the fraction of the 100 teachers voting $+1$.

40.3.6. Check yourself

- You double the number of teachers and keep Q and ε. Does σ change? *(No. Equation 40.4 does not contain k: one record still moves one vote. What changes is the counts, which are now twice as large, so the same noise matters less if the teachers agree as much as before.)*
- Why would publishing the noisy winning count along with the label break the LNMax analysis? *(The report-noisy-max proof conditions on the other candidates' noise and integrates over the winner's own noise. Publishing the winner's noisy count publishes that noise, which the proof assumed stays hidden.)*

40.4. How accurate is a noisy vote

40.4.1. Context

The cost of an answer does not depend on the teachers. Its accuracy depends on little else. How likely is the noisy vote to return the plurality label, and how should you choose the number of teachers k, given that more teachers means fewer records each?

40.4.2. Statement

On two labels, GNMax returns the plurality label with probability

$$\Pr[\text{plurality}] = \Phi\Big(\frac{g}{\sqrt{2}\,\sigma}\Big), \tag{40.5}$$

where g is the gap. On m labels, the probability that it returns anything other than the plurality i^* is at most

$$\tilde{q} = \sum_{i \neq i^*} \frac{1}{2}\operatorname{erfc}\left(\frac{n_{i^*} - n_i}{2\sigma}\right) \tag{40.6}$$

(Papernot et al. 2018, Proposition 7). If a fraction $a(x)$ of the k teachers vote for the correct label on a two-label query x, the gap measured toward the correct label is $k(2a(x)-1)$, and the expected accuracy of the private answers is the average of $\Phi(k(2a(x)-1)/(\sqrt{2}\sigma))$ over queries.

40.4.3. Derivation

1. **Two labels.** GNMax returns label 1 when $n_1 + Z_1 > n_2 + Z_2$, that is when $Z_2 - Z_1 < n_1 - n_2$. By Fact 4, $Z_2 - Z_1$ is $\mathcal{N}(0, 2\sigma^2)$, so this has probability $\Phi((n_1 - n_2)/(\sqrt{2}\sigma))$. With label 1 the plurality, $n_1 - n_2 = g$: Equation 40.5.
2. **Many labels.** GNMax returns some $i \neq i^*$ only if $n_i + Z_i > n_{i^*} + Z_{i^*}$ for at least one i. The probability that at least one of several events happens is at most the sum of their probabilities (the union bound), and each has probability $\frac{1}{2}\operatorname{erfc}((n_{i^*} - n_i)/(2\sigma))$ by Fact 4: Equation 40.6.
3. **Accuracy toward the truth.** Measure the gap toward the correct label instead of the plurality: it is the number of teachers that are right minus the number that are wrong, $ka - k(1-a) = k(2a-1)$, negative when most teachers are wrong. Step 1 with this signed gap gives the probability of a correct answer, and averaging over queries gives the expected accuracy.

40.4.4. Interpretation

The trade-off in k. For a fixed number of answers and budget, σ is fixed (Equation 40.4), and the signal is $k(2a-1)$. More teachers multiply the signal by k; but each teacher sees n/k records, is less accurate, and agrees less with the others, so $2a-1$ shrinks. Whether to add teachers depends on which effect wins. When the teachers' accuracy falls slowly with shard size, as it does for a model that learns from few examples, more teachers win; when a teacher needs many records to be any good, fewer teachers are better.

The ceiling. Without noise, the plurality vote is an ensemble, and its accuracy is the best any private answer can reach. Noise can only pull answers toward a coin flip. On queries the teachers find hard, where $a(x)$ is near $\frac{1}{2}$, any noise at all makes the answer close to random; this is the part of the accuracy that noise destroys first.

How tight is Equation 40.6? For two labels it is exact, since there is only one competitor. For many labels it adds the probabilities of beating the plurality one competitor at a time, which double-counts the outcomes where several competitors beat it. It is close when one competitor dominates the error and loose when many are nearly tied with the plurality.

40.4.5. Example: teachers on the running example

Table 40.3 splits the 10,000 private records among 25 to 500 teachers.

Table 40.3.: Teachers on the running example, measured on the evaluation set: each teacher's accuracy, the accuracy of the noiseless plurality vote, and the median gap as a fraction of the number of teachers.

Teachers k	Records each	Mean teacher accuracy	Plurality vote accuracy	Median $\lvert g\rvert/k$
25	400	0.8954	0.9403	1.000
50	200	0.8518	0.9389	0.880
100	100	0.7896	0.9384	0.660
250	40	0.7031	0.9336	0.408
500	20	0.6459	0.9339	0.276

The teachers get much worse as their shards shrink, from 0.8954 with 400 records to 0.6459 with 20, but the plurality vote hardly moves, between 0.9336 and 0.9403. Averaging many weak linear classifiers recovers most of what one classifier on all the data would learn. The relative gap falls from 1.000 to 0.276, but the absolute median gap grows, from 25 to $0.276 \times 500 = 138$. Since the noise does not depend on k, more teachers win here. Table 40.4 puts numbers on it.

Table 40.4.: Expected accuracy of Q GNMax answers on the evaluation set, computed exactly from Equation 40.5 and each query's vote gap.

Budget	Answers Q	σ	$k=25$	$k=50$	$k=100$	$k=250$	$k=500$
$\varepsilon=1$	100	57.2	0.5961	0.6651	0.7461	0.8250	0.8588

Budget	Answers Q	σ	$k = 25$	$k = 50$	$k = 100$	$k = 250$	$k = 500$
	1,000	180.9	0.5308	0.5546	0.5889	0.6489	0.6987
	10,000	572.1	0.5097	0.5173	0.5285	0.5498	0.5709
$\varepsilon = 4$	100	16.4	0.7899	0.8803	0.9064	0.9185	0.9237
	1,000	51.8	0.6059	0.6805	0.7641	0.8378	0.8682
	10,000	163.7	0.5340	0.5602	0.5979	0.6625	0.7140

With 100 answers at $\varepsilon = 4$, 500 teachers answer with accuracy 0.9237, about 1 point below their noiseless vote, 0.9339. With 1,000 answers at $\varepsilon = 1$, even 500 teachers fall to 0.6987, and 25 teachers, whose gaps never exceed 25, to 0.5308, a coin flip with a slight bias.

40.4.6. Check yourself

- A query has 250 teachers split 175 to 75. With $\sigma = 40$, what is the probability that GNMax returns the plurality? *(The gap is 100, so* $\Phi(100/(\sqrt{2} \times 40)) = \Phi(1.77) \approx 0.96$*.)*
- Why does the plurality accuracy in Table 40.3 barely depend on k, when the teachers' own accuracy drops by 25 points? *(Each teacher's error is mostly its own sampling noise on a small shard, and these errors are close to independent across shards; a majority of many weak but independent voters is right much more often than any one of them.)*

40.5. Private answers against a private model

40.5.1. Context

Here is the question of the title in its sharpest form. You have a budget (ε, δ). You can spend it on training one model with DP-SGD, whose answers are then free, or on answering queries one at a time with a teacher ensemble. Which gives more accurate answers, and how does the answer depend on how many queries you expect?

40.5.2. Statement

Let $A_{\text{model}}(\varepsilon)$ be the accuracy of a privately trained model at budget ε; it does not depend on how many queries the model answers. Let $A_{\text{vote}}(Q, \varepsilon)$ be the expected accuracy of Q private answers at the same budget, with σ from Equation 40.4. Then $A_{\text{vote}}(Q, \varepsilon)$ decreases in Q, from the noiseless vote's accuracy toward $\frac{1}{2}$ (on two labels), and private answers are better exactly for

$$Q < Q^*(\varepsilon) = \max\{Q : A_{\text{vote}}(Q, \varepsilon) \geq A_{\text{model}}(\varepsilon)\}, \tag{40.7}$$

with $Q^* = 0$ when even one answer is worse than the model.

40.5.3. Derivation

1. **The model's accuracy is constant in Q.** Every query to a released model is post-processing of its weights (Fact 2), so its answers cost nothing and are exactly as accurate as the model.
2. **The vote's accuracy falls with Q.** By Equation 40.4, σ grows like $\sqrt{Q}$. Each query's probability of a correct answer, $\Phi(g/(\sqrt{2}\sigma))$ with its signed gap g, decreases in σ when $g > 0$ and increases toward $\frac{1}{2}$ when $g < 0$; as $\sigma \to \infty$ every term tends to $\frac{1}{2}$. On this task the plurality is correct on most queries, and the average decreases in Q throughout the range shown here.
3. **Break-even.** A decreasing function crosses a constant at most once, at Q^*.

40.5.4. Interpretation

Private prediction wins when the budget is small and the queries are few. DP-SGD pays a fixed price whatever the number of queries, and at small ε that price is a model much worse than the non-private one. The vote pays per query; its first answers are almost as good as the noiseless ensemble. So at small ε there is a range of Q where the vote wins, and at large ε, where DP-SGD loses little, there may be none.

The ensemble's ceiling sets the large-ε limit. Even one noiseless answer is only as good as the plurality vote. If the teachers, each trained on a small shard, cannot match one model trained on all the data, no amount of budget closes the gap. On the running example the ensemble of linear teachers reaches 0.9403 at best, below what DP-SGD reaches at $\varepsilon = 2$.

What the comparison leaves out. It assumes you *can* run DP-SGD. When the teachers are LLMs behind an interface, models without gradients, or models trained by different parties who will not pool their data, the comparison is not with DP-SGD but with nothing, and private prediction is the only private option. It also ignores cost: DP-SGD trains once, while private prediction runs k models on every query.

40.5.5. Example: the running example

The DP-SGD baseline is the configuration of DP-SGD: $q = 0.01$, $C = 0.5$, step size 0.5, 10,000 steps, with the noise multiplier for each budget, retrained here and evaluated on this chapter's evaluation set (two runs each). Table 40.5 gives its accuracy and the break-even number of queries, taking for each budget the number of teachers that answers the most queries.

Table 40.5.: DP-SGD on the running example against private answers by GNMax: the largest number of queries for which the answers' expected accuracy is at least the private model's.

ε	DP-SGD accuracy (2 runs)	Break-even Q^*	Best k
0.5	0.8164 ± 0.0060	58	500
1	0.9128 ± 0.0207	18	500
2	0.9438 ± 0.0039	0	—

ε	DP-SGD accuracy (2 runs)	Break-even Q^*	Best k
4	0.9573 ± 0.0024	0	—
8	0.9605 ± 0.0004	0	—

At $\varepsilon = 0.5$, where DP-SGD reaches only 0.8164, 500 teachers answer up to 58 queries more accurately than the private model. At $\varepsilon = 1$ the range shrinks to 18 queries. From $\varepsilon = 2$ on, DP-SGD beats even a single noiseless vote, because the ensemble's ceiling, 0.9336 to 0.9403, is below its 0.9438. Figure 40.2 shows the whole curve.

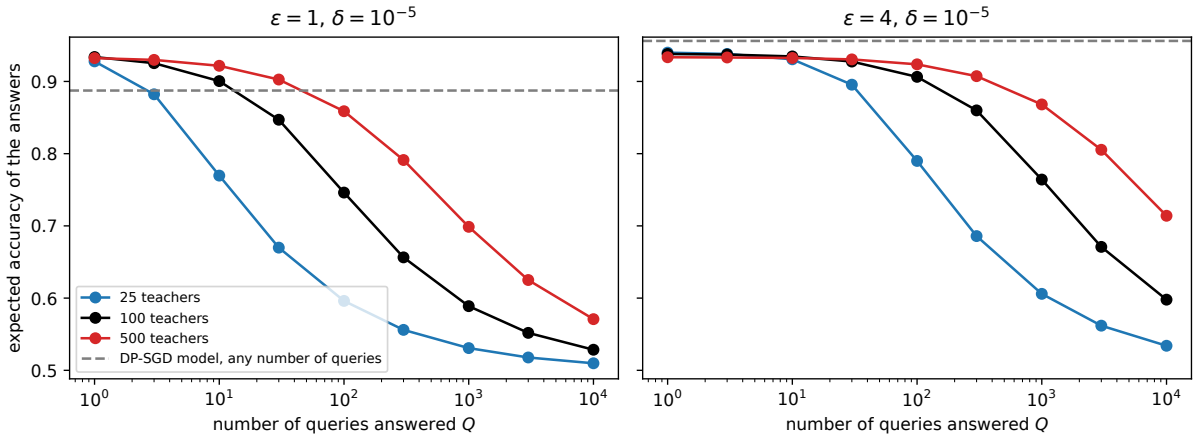


Figure 40.2.: Expected accuracy of Q private answers by GNMax against Q, for 25, 100 and 500 teachers, at $\varepsilon = 1$ (left) and $\varepsilon = 4$ (right), $\delta = 10^{-5}$. The dashed line is a DP-SGD model trained at the same budget, whose accuracy does not depend on the number of queries.

40.5.6. Check yourself

- If DP-SGD at $\varepsilon = 1$ were much worse on your task, say because you had only 1,000 records, would Q^* grow or shrink? *(It would grow, because the model's accuracy, the constant in Equation 40.7, falls. The teachers would also be weaker, though, so the vote's curve falls too; which effect wins has to be measured.)*
- A product team expects a model to answer a million queries. Should it use private prediction? *(Not query by query: a million answers need σ a hundred times that of 100 answers. It should either train privately or turn a few hundred private answers into a model, which is the next section.)*

40.6. PATE: from private answers to a private model

40.6.1. Context

Private answers cost budget forever; a private model costs it once. PATE combines them. Spend the budget on labelling a fixed number of *public, unlabelled* inputs with private answers, then train a new model, the **student**, on those inputs and labels. The student is released; the teachers never are. How private is the student, and how good can it be?

40.6.2. Statement

Let a student be trained, by any algorithm, on Q public inputs $x_1, \dots, x_Q$ and their GNMax labels with noise σ, and on any other public data, possibly unlabelled. Then the student, and every prediction it ever makes, is $(\alpha, Q\alpha/\sigma^2)$-RDP with respect to the private data, and so (ε, δ)-DP with ε from Equation 40.2. The privacy cost depends on the number of labelled queries Q and on σ, and on nothing about the student: not its size, its training algorithm or how many predictions it makes.

40.6.3. Derivation

1. **The only private output is the labels.** The student's training algorithm sees the public inputs, which do not depend on the private data, and the Q labels. It never touches the teachers or the private records.
2. **The labels are Q GNMax answers.** The inputs to be labelled may be chosen adaptively, for example by the student asking about inputs it is unsure of, because RDP composition holds for adaptively chosen mechanisms. Their total cost is $Q\alpha/\sigma^2$ (Section 40.3).
3. **Post-processing.** The student's weights are a function of the labels and of public data, so they are as private as the labels (Fact 2), and so is every prediction computed from the weights.

40.6.4. Interpretation

Three things limit the student. It is trained on Q examples rather than n, so it learns less than a model trained on all the private data. Its labels are noisy versions of the plurality vote, wrong on some fraction of the queries. And the plurality vote itself has the ensemble's ceiling. Budget pulls these in opposite directions: more labels mean more noise per label by Equation 40.4, so for each ε there is a best Q.

What PATE needs from you. Public, unlabelled inputs from the same distribution as the private data. In some domains this is easy (unlabelled images or text are plentiful; the labels are what is sensitive), in others impossible (there is no public pool of medical records). PATE also needs the public pool to be really public: an input that is itself a private record of someone else is protected by nothing in this analysis.

Semi-supervised students. The student may use every unlabelled public input, not only the Q it paid to label. Papernot et al. (2017) train the student as a semi-supervised generative

adversarial network (GAN) on the labelled and unlabelled inputs, and Papernot et al. (2018) use semi-supervised methods for their image tasks. This chapter's student uses only its Q labelled inputs, which is the simplest choice and the least favourable to PATE.

What PATE protects, and against whom. The guarantee covers the released student and its predictions. The teachers are non-private models of the data and must stay secret: an attacker who steals one can run any attack of Part 2 on its shard. PATE moves the trust from the training algorithm to the server that holds the teachers.

40.6.5. Example: students on the running example

Take $k = 250$ teachers. Their plurality vote labels the public pool with accuracy 0.9322. The student is the part's network (64 hidden units, 3,329 weights), trained non-privately on the first Q public inputs and their GNMax labels for 2,000 steps. Table 40.6 gives the results; the first two rows are references without privacy.

Table 40.6.: Test accuracy of a PATE student with 250 teachers, against the number of labelled public queries Q, one run each; in parentheses, the fraction of the student's GNMax labels that are correct.

Labels	$Q = 250$	$Q = 500$	$Q = 1{,}000$	$Q = 2{,}000$
True labels (not private)	0.8726	0.8990	0.9188	0.9410
Noiseless plurality (not private)	0.8913	0.9106	0.9229	0.9297
GNMax, $\varepsilon = 1$	0.8238 (0.7760)	0.7595 (0.6880)	0.6995 (0.6350)	0.6749 (0.6260)
GNMax, $\varepsilon = 2$	0.7980 (0.8040)	0.8393 (0.8140)	0.8141 (0.7490)	0.7318 (0.6885)
GNMax, $\varepsilon = 4$	0.8633 (0.8720)	0.8754 (0.8740)	0.8898 (0.8360)	0.8259 (0.7840)
GNMax, $\varepsilon = 8$	0.8950 (0.9160)	0.9037 (0.9040)	0.9049 (0.8870)	0.8884 (0.8525)

The best student reaches 0.8238 at $\varepsilon = 1$ ($Q = 250$), 0.8393 at $\varepsilon = 2$ ($Q = 500$), 0.8898 at $\varepsilon = 4$ ($Q = 1{,}000$) and 0.9049 at $\varepsilon = 8$ ($Q = 1{,}000$). DP-SGD, at 0.9128, 0.9438, 0.9573 and 0.9605, is better at every budget (Figure 40.3). The table shows why. With true labels, the student needs 2,000 examples to reach 0.9410; at $\varepsilon = 4$, 2,000 labels need $\sigma = 73.2$, and only 0.7840 of them are right. With fewer labels the noise is lower, but the student has too few examples. (The best Q for each budget is chosen here by looking at test accuracy, which is itself a leak; Hyperparameter tuning shows how to pay for it.)

On this task PATE loses, and it is worth saying why, because the reasons are the conditions under which it wins elsewhere. The task is binary, so a wrong label is the other label and label noise is as damaging as it can be. The teachers are weak (0.7031 each with 40 records) and disagree on many queries, so many gaps are small against the noise. The student needs thousands of labels to learn the decision boundary. And DP-SGD does well on this task even at $\varepsilon = 1$. PATE's published successes are on tasks where teachers agree on most inputs and a student learns from few labels plus many unlabelled inputs: on a handwritten-digit benchmark (MNIST), Papernot et

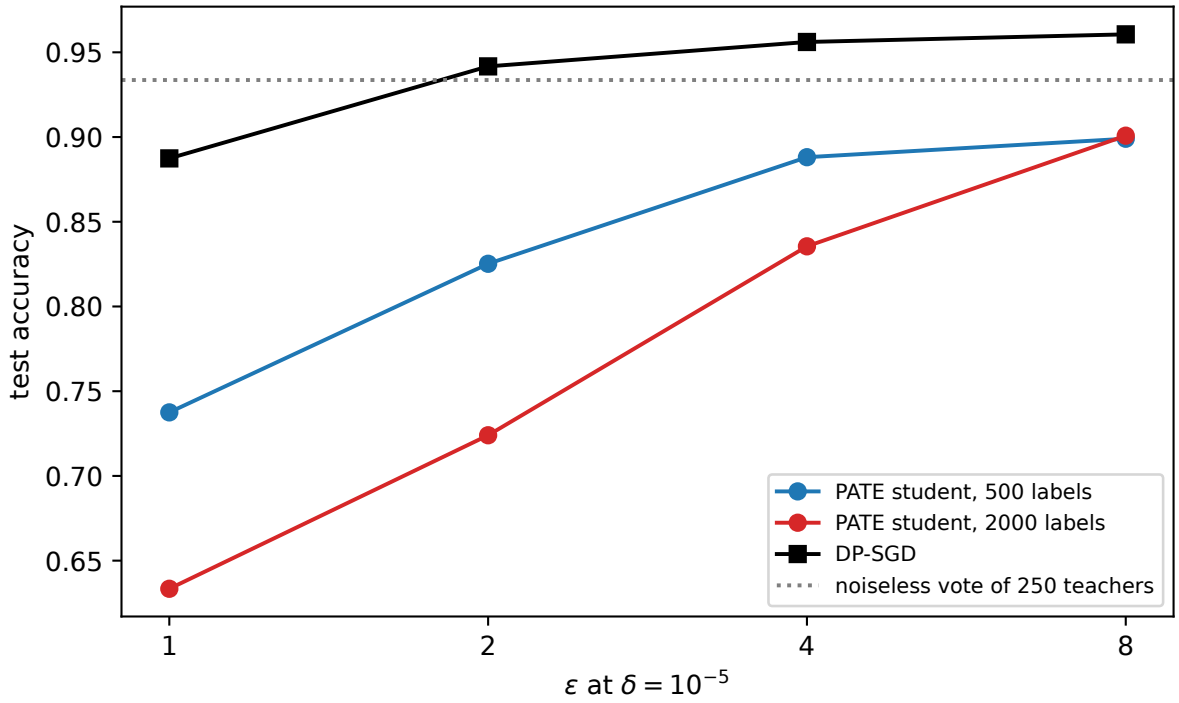


Figure 40.3.: Test accuracy of the PATE student with 250 teachers and 500 or 2,000 labelled public queries, and of DP-SGD, against ε at $\delta = 10^{-5}$, one run each. The dotted line is the accuracy of the teachers' noiseless plurality vote.

al. (2018, Table 1) report a student with 98.5% accuracy against a non-private 99.2%, at $\varepsilon = 1.97$ with 286 answered queries. The next section explains the analysis behind such small ε.

40.6.6. Check yourself

- The student is trained with 2,000 steps of non-private SGD and has 3,329 weights. Does either number change its ε? *(No. The student's training is post-processing of the labels; only Q and σ enter.)*
- Could the student query the teachers about inputs it is uncertain about, after a first round of training? *(Yes. The queries are public inputs chosen from earlier private answers, which adaptive composition allows; each answer still costs α/σ^2.)*

40.7. Data-dependent analysis and Confident-GNMax

40.7.1. Context

The cost α/σ^2 of a GNMax answer is a worst case over all vote histograms, including those where the vote is tied and one teacher decides it. But when 249 of 250 teachers agree, one record, which changes one teacher, can hardly change the answer: the noisy vote returns the plurality with overwhelming probability, with or without that record. Can a query on which the teachers agree be charged less? And can a mechanism be designed to answer only such queries?

40.7.2. Statement

The pure-DP form (Papernot et al. 2017, Theorem 3, written in RDP). Let M be ε_0-DP and suppose that on the actual dataset D there is an output i^* with $\Pr[M(D) \neq i^*] = q < e^{-\varepsilon_0}$. Then for every neighbour D',

$$D_\alpha(M(D)\|M(D')) \le \frac{1}{\alpha-1}\ln\Big[(1-q)^\alpha(1-e^{\varepsilon_0}q)^{1-\alpha} + q\,e^{(\alpha-1)\varepsilon_0}\Big]. \tag{40.8}$$

When only an upper bound $\tilde{q} \ge q$ is known, the bound holds with the largest value of the right side over $q \in (0, \tilde{q}]$. For LNMax, $\varepsilon_0 = 2/b$.

The Gaussian form (Papernot et al. 2018, Theorem 6, with their orders from Proposition 10). For GNMax with noise σ and $\tilde{q}$ from Equation 40.6, let $\mu_2 = \sigma\sqrt{\ln(1/\tilde{q})}$, $\mu_1 = \mu_2 + 1$, $\varepsilon_1 = \mu_1/\sigma^2$ and $\varepsilon_2 = \mu_2/\sigma^2$. If $\alpha \le \mu_1$, $\mu_2 > 1$ and $\tilde{q} \le e^{(\mu_2-1)\varepsilon_2}\big/\big(\frac{\mu_1}{\mu_1-1}\cdot\frac{\mu_2}{\mu_2-1}\big)^{\mu_2}$, then for every neighbour D'

$$D_\alpha(M(D)\|M(D')) \le \frac{1}{\alpha-1}\ln\Big[(1-\tilde{q})A^{\alpha-1} + \tilde{q}B^{\alpha-1}\Big], \quad A = \frac{1-\tilde{q}}{1-(\tilde{q}e^{\varepsilon_2})^{(\mu_2-1)/\mu_2}}, \quad B = \frac{e^{\varepsilon_1}}{\tilde{q}^{1/(\mu_1-1)}}. \tag{40.9}$$

Each answer is charged the smaller of this and α/σ^2, and the charges add up over answers.

Confident-GNMax (Papernot et al. 2018, Algorithm 1). For each query, release $\perp$ (no answer) unless $\max_i n_i + \mathcal{N}(0, \sigma_1^2) \geq T$; if the check passes, release a GNMax answer with noise σ_2. With data-independent accounting, P checks and A answers cost

$$\varepsilon(\alpha) = \Big(\frac{P}{2\sigma_1^2} + \frac{A}{\sigma_2^2}\Big)\alpha, \tag{40.10}$$

provided the total is kept below a budget fixed in advance.

40.7.3. Derivation

The pure-DP form shows where all data-dependent bounds come from; the Gaussian form follows the same plan with Rényi bounds at higher orders in place of the pure-DP ratio, and its proof is in Papernot et al. (2018, Appendix A).

1. **Write the divergence as a sum over outputs.** By Equation 40.1, $e^{(\alpha-1)D_\alpha} = \sum_y P(y)^\alpha P'(y)^{1-\alpha}$, with $P = M(D)$ and $P' = M(D')$. Split the sum into the likely output i^* and the rest. Write $q = P(\text{not } i^*)$ and $p = P'(\text{not } i^*)$.
2. **The unlikely outputs.** For $y \neq i^*$, $P(y)^\alpha P'(y)^{1-\alpha} = P(y)\big(P(y)/P'(y)\big)^{\alpha-1} \leq P(y)e^{(\alpha-1)\varepsilon_0}$, because ε_0-DP bounds the ratio by e^{ε_0}. Summed over $y \neq i^*$, this is at most $q\, e^{(\alpha-1)\varepsilon_0}$.
3. **The likely output.** Its term is $(1-q)^\alpha(1-p)^{1-\alpha}$. Pure DP, applied to the event "not i^*", gives $p \leq e^{\varepsilon_0}q$, so $1-p \geq 1-e^{\varepsilon_0}q > 0$. The exponent $1-\alpha$ is negative, so a larger base gives a smaller power: $(1-p)^{1-\alpha} \leq (1-e^{\varepsilon_0}q)^{1-\alpha}$.
4. **Collect.** Adding the two parts and taking $\frac{1}{\alpha-1}\ln$ gives Equation 40.8 with the true q. Since only a bound $\tilde{q}$ is computable, take the largest value over all q the bound allows; the right side increases in q in every example of this chapter, so the largest value is at $q = \tilde{q}$.
5. **Small q.** At $q = 0$ the bracket is 1 and the bound is 0. Its derivative there is $e^{(\alpha-1)\varepsilon_0} + (\alpha-1)e^{\varepsilon_0} - \alpha$, so for small q the bound is about q times a constant: the cost of an answer is proportional to the probability that the noise overturns the plurality.
6. **Confident-GNMax.** The check releases a noisy version of $\max_i n_i$, which one record changes by at most 1 (one count up or down by 1 moves the maximum by at most 1), so it is a Gaussian mechanism with sensitivity 1: $(\alpha, \alpha/(2\sigma_1^2))$-RDP for every query, answered or not. The second step is GNMax, $(\alpha, \alpha/\sigma_2^2)$-RDP, run only when the check passes. Whether it runs is decided by a released output, so its cost is chosen adaptively. Both costs are lines in α, and the Rényi filter of Adaptive composition says that adaptively chosen RDP costs that stay below a fixed budget on every path compose as if fixed. Summing gives Equation 40.10.

40.7.4. Interpretation

Agreement is cheap, disagreement costs full price. Figure 40.4 draws the cost of one GNMax answer with $\sigma = 40$ at order 8 against the gap. At gaps up to 100 the data-dependent bound is no better than the data-independent $\alpha/\sigma^2 = 0.005$; then it falls steeply, to 1.23×10^{-3} at a gap of 150 and 8.37×10^{-5} at 200. The dashed curve is the exact data-dependent cost on two labels, which the script computes directly: the output is then a coin with probability $\Phi(g/(\sqrt{2}\sigma))$, one record moves g by 2, and Equation 40.1 applies to two outcomes. It lies below the bound

everywhere: at a gap of 150, 2.42×10^{-4}, five times below. Theorem 6 is valid but not tight, and it is far from tight for small gaps, where on two labels even the data-independent cost overstates the truth.

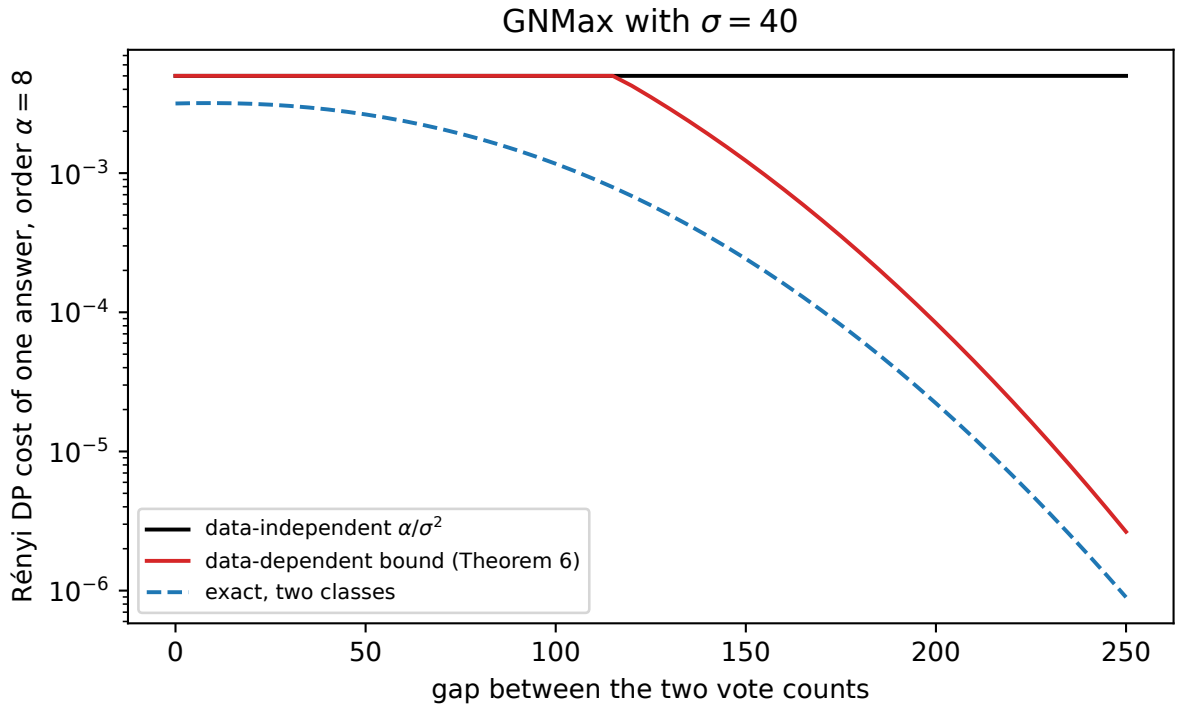


Figure 40.4.: The RDP cost at order 8 of one GNMax answer with $\sigma = 40$, against the gap between the two vote counts, on a log scale: the data-independent α/σ^2, the data-dependent bound of Equation 40.9, and the exact data-dependent cost on two labels.

Why it helps little here and a lot elsewhere. The total is a sum over queries, so the saving depends on how many queries have large gaps. With 250 teachers on the running example, only 0.2879 of the public queries have a gap of at least 150, so most answers pay close to full price. On a task where nearly all teachers agree on nearly all inputs, as on handwritten digits, almost every answer is nearly free, and ε falls by a large factor.

The data-dependent ε is itself private information. It is computed from the votes, so publishing it reveals something about the teachers and so about the data. Worse, its sensitivity is large: one record changes one teacher, and that teacher's votes can change on *every* query, moving every term of the sum. Papernot et al. (2018, Appendix B) therefore release the data-dependent ε with Gaussian noise scaled to its **smooth sensitivity**, the device of Beyond global sensitivity for functions whose worst-case sensitivity is far above their typical one, and pay for that release too. On MNIST, with 250 teachers of 200 examples, 640 queries, $T = 200$, $\sigma_1 = 150$ and $\sigma_2 = 40$, they report an expected 283 answered queries and a data-dependent $\varepsilon = 2.00$ at $\delta = 10^{-5}$, which becomes 2.52 ± 0.385 once sanitized (their Table 2). Without that step, a data-dependent ε is a measurement, not a guarantee.

A one-directional bound. Equation 40.8 and Equation 40.9 bound $D_\alpha(M(D)\|M(D'))$ for the actual dataset D against every neighbour, the direction the accounting of Papernot et al. uses.

The exact two-label computation above takes the worse of both directions, and is still below the bound.

Confident-GNMax answers what is cheap and correct. Queries with a strong consensus are cheap under the data-dependent analysis, and their plurality label is more likely to be right. The noisy check selects them. It costs something on every query, but with σ_1 much larger than σ_2 that cost is small per query, and the expensive, likely-wrong queries are mostly skipped. The price is that the student never sees labels for the hard inputs near the decision boundary.

40.7.5. Example: data-dependent costs on the running example

The pure-DP form. For an LNMax answer with $\varepsilon_0 = 0.1$, at order 8 the data-independent cost is 0.0367. If the plurality fails with probability $\tilde{q} = 0.1$, Equation 40.8 gives 0.0235; at $\tilde{q} = 0.01$, 0.00248; at $\tilde{q} = 0.001$, 0.00025. Below $\tilde{q} = 0.1$ the cost falls in proportion to $\tilde{q}$, as step 5 predicts.

GNMax on 250 teachers. Table 40.7 compares the costs of answering the first Q public queries with $\sigma = 40$.

Table 40.7.: ε at $\delta = 10^{-5}$ for Q GNMax answers with 250 teachers and $\sigma = 40$ on the running example's public pool.

Answers Q	Data-independent ε	Data-dependent ε (not sanitized)
250	2.45	1.93
1,000	5.38	4.37
4,000	12.30	10.23

The saving is about 20% at every Q, the fraction of queries with large gaps. Figure 40.5 shows the same over Q, with LNMax at the same noise standard deviation for comparison: Laplace answers cost more at every Q shown, and the ratio grows with Q. The data-dependent ε also has the instability described above: if one teacher among the 250 switched every one of its votes on the first 1,000 queries, the data-dependent ε would move from 4.375 to 4.389.

Confident-GNMax. With the parameters Papernot et al. use on MNIST, $T = 200$, $\sigma_1 = 150$ and $\sigma_2 = 40$, Table 40.8 runs the 250 teachers on pools of 1,000 and 4,000 public queries.

Table 40.8.: Confident-GNMax with 250 teachers, $T = 200$, $\sigma_1 = 150$, $\sigma_2 = 40$, on the running example; the student is trained on the answered queries only.

Pool P	Answered A	Label accuracy	Data-independent ε	Data-dependent ε	Student accuracy
1,000	459	0.8693	3.59	2.82	0.8894
4,000	1,773	0.8917	7.87	6.28	0.9043

At a data-independent $\varepsilon = 3.59$, Confident-GNMax trains a student to 0.8894, about what plain GNMax reached at $\varepsilon = 4$ (0.8898 in Table 40.6). It improves on GNMax a little, not enough

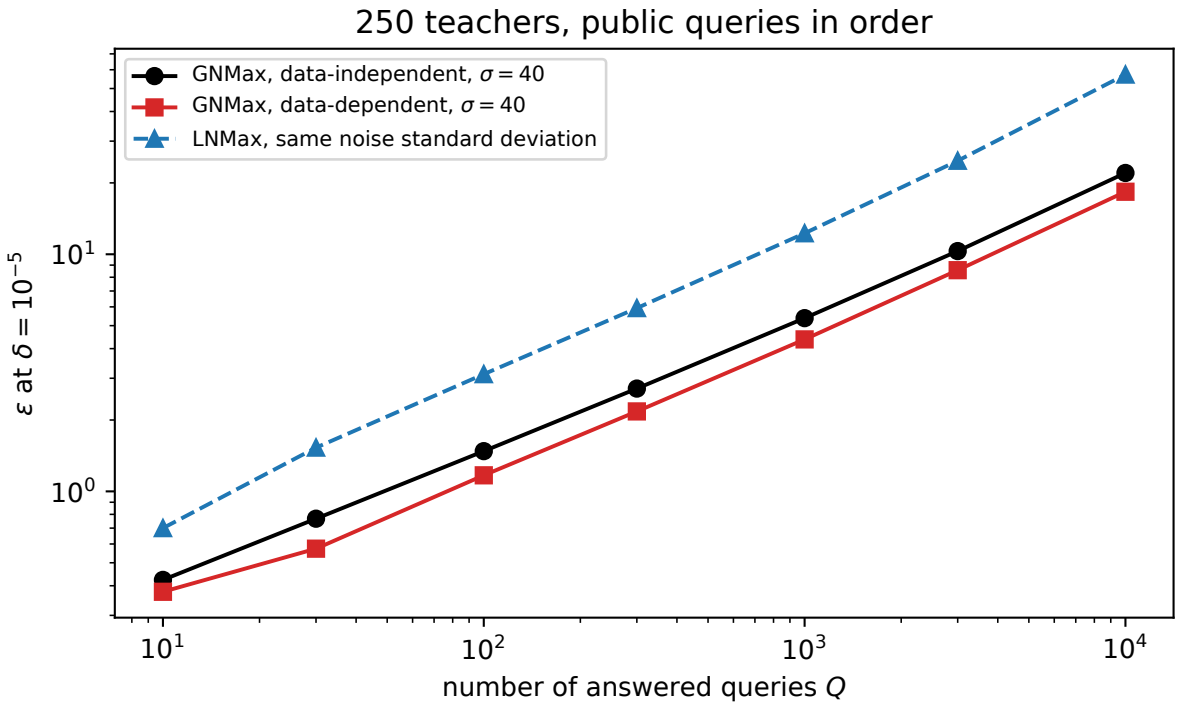


Figure 40.5.: ε at $\delta = 10^{-5}$ for Q answers with 250 teachers on the running example's public queries, on log–log axes: GNMax with $\sigma = 40$ under the data-independent and data-dependent analyses, and LNMax with the same noise standard deviation.

to catch DP-SGD at 0.9573. The check costs little: $1{,}000/(2 \times 150^2) = 0.022$ in slope, against $459/40^2 = 0.287$ for the answers.

40.7.6. Check yourself

- Why can't the data-dependent ε simply be released with noise calibrated to its global sensitivity? *(Its global sensitivity is huge. One record changes one teacher, whose votes may change on every query; on a histogram near a tie, each such change moves that query's cost from almost 0 to the full α/σ^2, so the worst-case change of the sum is close to the data-independent total.)*
- In Confident-GNMax, why is σ_1 chosen much larger than σ_2? *(The check is paid on every query and the answer only on answered ones. A large σ_1 makes the frequent cost small; the check only needs to separate strong consensus from weak, which a noisy maximum can do even with a lot of noise when T is set well.)*

40.8. Private in-context learning

40.8.1. Context

An LLM can learn a task from examples placed in its prompt, without any training: show it four labelled movie reviews and it will label the fifth. This **in-context learning** has its own privacy problem. If the examples, the **exemplars**, are private records, every answer is computed from them, and an attacker who can query the prompted model can learn about them. Duan, Dziedzic, Papernot and Boenisch (NeurIPS 2023) demonstrate this with a membership inference attack (MIA): querying GPT-3 prompted with one private example, the model's probability for the correct label is markedly higher on the example that was in the prompt than on other records. Training the LLM with DP-SGD is not an option when it is only reachable through an interface. How do you answer privately from a prompt?

40.8.2. Statement

Private in-context learning (Wu, Panda, Wang & Mittal, ICLR 2024; Duan et al. 2023): split the private exemplars into disjoint groups by a fixed rule, prompt the model once per group with that group's exemplars and the query, and release the GNMax label of the k answers. Each answer is $(\alpha, \alpha/\sigma^2)$-RDP, exactly as in Section 40.3. If, as in Wu et al., each query first draws a fresh Poisson sample of the exemplars, each record included independently with probability r, and the sample is split into groups by the fixed rule, then each answer's RDP curve is that of the Poisson-subsampled Gaussian with sampling rate r and noise multiplier $\sigma/\sqrt{2}$, and Q answers add up:

$$\varepsilon(\alpha) = Q\, \varepsilon_{r,\,\sigma/\sqrt{2}}(\alpha), \tag{40.11}$$

with $\varepsilon_{q,s}$ the subsampled-Gaussian curve of Amplification by subsampling, the same accountant DP-SGD uses. **PromptPATE** (Duan et al.) turns the answers into a public prompt instead: label a few public inputs with Confident-GNMax over an ensemble of prompted models, and use the labelled public inputs as the exemplars of a single prompt that is then deployed with no further cost, the student of PATE in prompt form.

40.8.3. Derivation

1. **One exemplar, one prompt.** With the groups fixed by a rule that does not depend on the data (a hash, as in Fact 1), adding or removing an exemplar changes the content of one group's prompt and so at most one answer: one vote moves. The histogram has ℓ_2 sensitivity $\sqrt{2}$.
2. **Without subsampling.** GNMax on this histogram is $(\alpha, \alpha/\sigma^2)$-RDP (Section 40.3), whatever the language model computes from its prompt.
3. **With Poisson subsampling.** The record is in the query's sample with probability r, independently of the others. If it is not, the output is distributed as without it; if it is, the histogram shifts by a vector of length at most $\sqrt{2}$ against noise of standard deviation σ on each coordinate. Rotating the shift onto one axis and rescaling by $\sqrt{2}$, this is the subsampled Gaussian of DP-SGD with sensitivity 1 and noise multiplier $\sigma/\sqrt{2}$.
4. **Compose.** The Q answers use independent samples and add up in RDP: Equation 40.11.
5. **PromptPATE.** The deployed prompt contains public inputs and their private labels. It is post-processing of the labels, so its cost is that of the labels alone, however often it is used.

40.8.4. Interpretation

Answering from a prompt is private prediction. Everything in the previous sections carries over: the cost grows with the number of answered queries, the accuracy depends on how much the prompted models agree, and the data-dependent analysis applies. The difference is the teacher. A prompted LLM is a strong learner from very few examples, because most of what it knows comes from pretraining; its errors with different exemplars are correlated, so its votes agree far more than independently trained small models do. Duan et al. observe this high consensus and suggest that it may come from all teachers sharing the same underlying model. High consensus is exactly what makes data-dependent costs small.

Subsampling amplifies; disjointness is still needed. Subsampling lets a large exemplar set serve many queries cheaply, since each record takes part in a fraction r of them. But within one query the groups must be disjoint, or a record in two groups moves two votes.

Text generation needs another aggregator. Votes work for labels. For free text, Wu et al. map each group's answer into a space where averaging makes sense, either sentence embeddings, which they privatize with the Gaussian mechanism, or keyword counts, from which they select keywords privately with propose-test-release or a joint exponential mechanism, and then ask the model to produce an answer from the private aggregate.

What the papers report. These numbers are from the papers, not from this chapter's script. Duan et al. report, with GPT-3 as the base model, 92.7% accuracy on a movie-review sentiment task at $(\varepsilon, \delta) = (0.147, 10^{-6})$ with PromptPATE, against 95.2% without privacy. Wu et al. (arXiv version) report on the same sentiment task, with 10,000 queries each answered by a vote of ten 4-shot prompts, 91.97% at $\varepsilon = 1$ and 92.83% at $\varepsilon = 3$ with GPT-3 Babbage, against 91.89% for a single non-private 4-shot prompt and 86.58% with no examples at all.

40.8.5. Example: a simulated in-context learner

No language model is available here, so the script simulates the mechanics with a **stand-in**: an "in-context learner" that, given exemplars (x_j, y_j), labels a query by the sign of $\langle \sum_j y_j x_j, x \rangle$. It has no pretraining at all, which makes it far weaker than an LLM; what carries over is the accounting and the dependence on agreement, not the accuracy.

The leak. An attacker who knows a candidate record (x, y) scores it by $y\langle w, x \rangle$, the learner's confidence in the right label. The area under the curve (AUC) of this attack, the probability that a random member scores above a random non-member, is 1.000 against a 1-shot learner, 0.996 against a 4-shot learner and 0.889 against a 16-shot learner (400 trials each). A prompt with few exemplars reveals them almost perfectly, as Duan et al. found for GPT-3.

Private answers. Each query draws a Poisson sample of the 10,000 private records at rate $r = 0.04$, about 400 records, and splits it by record index into 25 groups of about 16. One 16-shot learner is right on 0.6232 of the evaluation set (50 draws); the 25-vote without noise, on 0.8090 of 2,000 queries. Table 40.9 adds the noise for a budget spread over Q queries, from Equation 40.11.

Table 40.9.: Private in-context learning with the stand-in learner: 25 groups of about 16 exemplars per query, Poisson sampling rate 0.04, GNMax noise for Q answers at $(\varepsilon, 10^{-5})$, accuracy on 2,000 evaluation queries.

Budget	Answers Q	σ	Accuracy
$\varepsilon = 1$	100	2.75	0.7780
	1,000	7.39	0.6800
	10,000	22.93	0.5615
$\varepsilon = 4$	100	1.30	0.8080
	1,000	2.36	0.7910
	10,000	6.65	0.7025

Subsampling makes the noise small: 1,000 answers at $\varepsilon = 1$ need $\sigma = 7.39$ here, against 181 without subsampling (Table 40.2). But the vote has only 25 voters, so even $\sigma = 7.39$ is large against its gaps, and accuracy falls from 0.8090 to 0.6800. Figure 40.6 shows the decline over Q.

A student prompt. Split all 10,000 records into 625 disjoint 16-shot learners; their noiseless vote is right on 0.9366 of the evaluation set. Label the first Q public inputs with GNMax at $\varepsilon = 1$ and use them as the exemplars of one public learner. With 16 labelled inputs ($\sigma = 22.9$) the student prompt reaches 0.6315; with 64 ($\sigma = 45.8$), 0.7237; with 256 ($\sigma = 91.5$, labels 0.8086 correct), 0.7954, about the accuracy of 100 private answers at the same budget, but for any number of queries. The stand-in needs hundreds of exemplars because it has no prior knowledge; an LLM needs a handful, which is why PromptPATE can stop at very few labelled public inputs and a very small ε.

40.8.6. Check yourself

- A developer puts 200 private exemplars into one long prompt and answers 1,000 user queries with it. Is anything private? *(No. Every answer is a deterministic or sampled function*

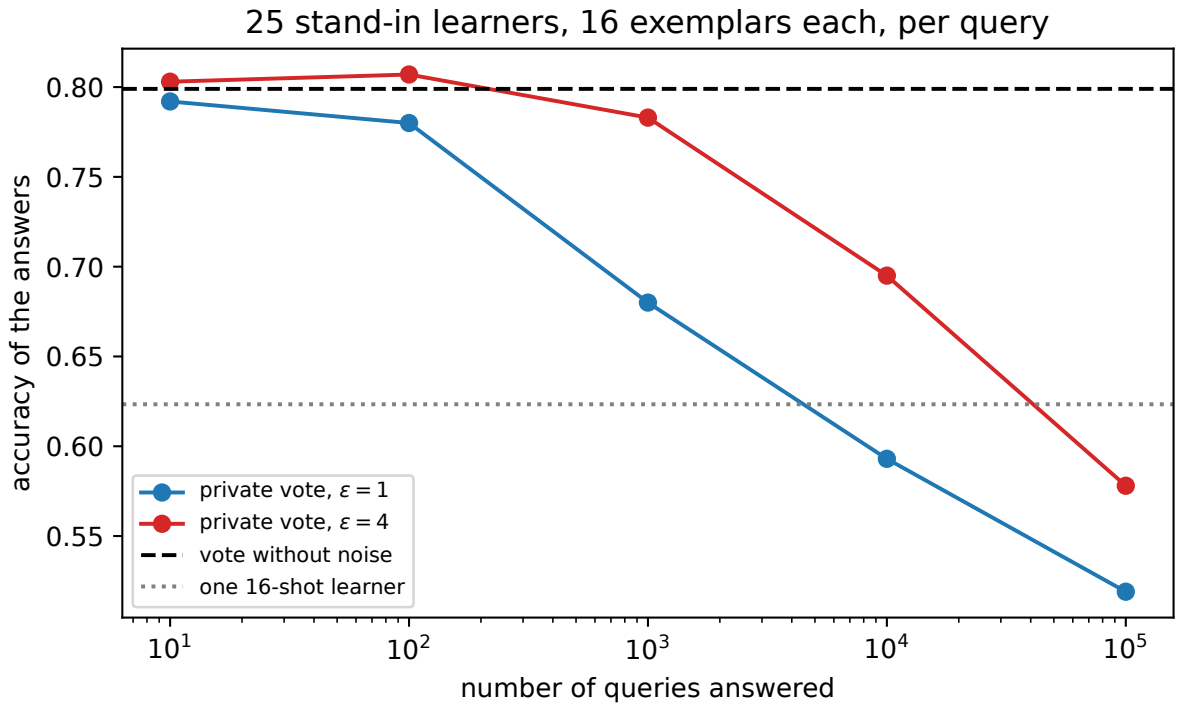


Figure 40.6.: Accuracy of private answers from the stand-in in-context learner against the number of queries answered at $\varepsilon = 1$ and $\varepsilon = 4$ ($\delta = 10^{-5}$), with 25 groups of about 16 exemplars drawn per query at rate 0.04. The dashed line is the vote without noise and the dotted line a single 16-shot learner.

of all 200 exemplars, with no noise and no bound on any one exemplar's influence; the membership attack above works against it.)

- Why does Equation 40.11 use noise multiplier $\sigma/\sqrt{2}$ rather than σ? *(The vote histogram moves by $\sqrt{2}$ in ℓ_2 norm when one exemplar's group changes its vote. The subsampled-Gaussian curve is stated for a shift of 1, and a shift of $\sqrt{2}$ against noise σ is the same as a shift of 1 against noise $\sigma/\sqrt{2}$.)*

40.9. Code: check every number by computation

The script code/private_prediction.py trains the teachers, implements LNMax, GNMax, Confident-GNMax and their accountants (data-independent, the data-dependent bounds of Equation 40.8 and Equation 40.9, and the exact two-label cost), trains the PATE students and the DP-SGD baselines, and simulates private in-context learning. It imports the running example, the network, DP-SGD and the subsampled-Gaussian accountant from code/dp_sgd.py, and the logistic regression from code/dp_erm.py. From the repository root:

```
uv run python code/part05-dp-ml/private_prediction.py
```

Output (seed 0), summarized in Table 40.10:

Table 40.10.: The script's output with the default seed.

Experiment	Result
Noise for Q answers	$\sigma \propto \sqrt{Q}$: 57.2 for 100 answers at $\varepsilon = 1$, 572 for 10,000
Teachers	Plurality accuracy 0.9336 to 0.9403 for 25 to 500 teachers
Answers against DP-SGD	Answers win for up to 58 queries at $\varepsilon = 0.5$, 18 at $\varepsilon = 1$, none from $\varepsilon = 2$
PATE student, 250 teachers	Best 0.8238 at $\varepsilon = 1$ to 0.9049 at $\varepsilon = 8$; DP-SGD 0.9128 to 0.9605
Data-dependent analysis	About 20% lower ε on this task; Theorem 6 above the exact two-label cost
Confident-GNMax	459 of 1,000 queries answered, student 0.8894 at $\varepsilon = 3.59$
In-context learning, stand-in	Membership AUC 0.889 for a 16-shot learner; 0.7780 accuracy for 100 private answers at $\varepsilon = 1$

tests/test_private_prediction.py checks that the output still matches.

What to take from it. The privacy of private prediction is easy: one record, one vote, a Gaussian histogram, an accountant. Its accuracy is the hard part, and it is decided by the teachers' agreement against noise that grows like $\sqrt{Q}$. On a binary task where DP-SGD works well, private prediction wins only at small budgets and for a few dozen queries, and PATE's student loses to

DP-SGD at every budget. That is not a verdict on the method but a description of when to use it: when you cannot train privately, when the queries are few, and when the teachers agree, which is what pretrained models in in-context learning provide.

40.10. References

- Papernot, Abadi, Erlingsson, Goodfellow & Talwar, "Semi-supervised Knowledge Transfer for Deep Learning from Private Training Data" (ICLR 2017). PATE: teachers on disjoint shards, LNMax, a student trained on privately labelled public data, and the first data-dependent analysis (their Theorem 3, Equation 40.8).
- Papernot, Song, Mironov, Raghunathan, Talwar & Erlingsson, "Scalable Private Learning with PATE" (ICLR 2018). GNMax and its RDP (Proposition 8), the union bound Equation 40.6 (Proposition 7), the data-dependent bound Equation 40.9 (Theorem 6, Proposition 10), Confident- and Interactive-GNMax, and the smooth-sensitivity release of the data-dependent ε (Appendix B).
- Nissim, Raskhodnikova & Smith, "Smooth Sensitivity and Sampling in Private Data Analysis" (STOC 2007). Sample-and-aggregate, of which PATE is an instance, and smooth sensitivity.
- Duan, Dziedzic, Papernot & Boenisch, "Flocks of Stochastic Parrots: Differentially Private Prompt Learning for Large Language Models" (NeurIPS 2023). The membership attack on prompts and PromptPATE.
- Wu, Panda, Wang & Mittal, "Privacy-Preserving In-Context Learning for Large Language Models" (ICLR 2024). Disjoint exemplar groups with Poisson subsampling and a Gaussian noisy vote for classification; embedding- and keyword-space aggregation for text generation.
- Dwork & Feldman, "Privacy-preserving Prediction" (COLT 2018). Private prediction as a learning problem in its own right, with sample-complexity bounds for answering predictions privately.
- Mironov, "Rényi Differential Privacy" (CSF 2017). The Gaussian mechanism's RDP and composition used throughout.
- Balle, Barthe, Gaboardi, Hsu & Sato, "Hypothesis Testing Interpretations and Rényi Differential Privacy" (AISTATS 2020). The conversion of Equation 40.2.

40.11. Cheat sheet

k teachers on disjoint shards fixed by a hash, m labels, vote counts n_i, gap g, Q answered queries, add/remove neighbours. Table 40.11 collects the results.

Table 40.11.: Every result of the chapter.

Result	Formula	Conditions
Histogram sensitivity	ℓ_1: 2, ℓ_2: $\sqrt{2}$	One record in one shard
LNMax	$(2/b)$-DP per answer	Release the label only
GNMax	$(\alpha, \alpha/\sigma^2)$-RDP per answer	Release the label only

Result	Formula	Conditions
Noise for Q answers	$\sigma = \sqrt{Q/\rho(\varepsilon,\delta)}$; $\rho = 0.0306$ for $\varepsilon = 1$, 0.3731 for $\varepsilon = 4$ at $\delta = 10^{-5}$	Adaptive queries allowed
Accuracy, two labels	$\Phi(g/(\sqrt{2}\sigma))$	Exact
Failure, m labels	$\tilde{q} = \sum_{i \neq i^*} \frac{1}{2}\operatorname{erfc}((n_{i^*} - n_i)/(2\sigma))$	Union bound
Break-even	Answers beat a private model for $Q < Q^*(\varepsilon)$	Compare at the same budget
PATE student	Cost of its Q labels; predictions free	Public unlabelled inputs
Data-dependent, pure DP	$\frac{1}{\alpha-1}\ln[(1-q)^\alpha(1-e^{\varepsilon_0}q)^{1-\alpha} + qe^{(\alpha-1)\varepsilon_0}]$	$q < e^{-\varepsilon_0}$; must be sanitized to publish
Data-dependent, GNMax	Equation 40.9	Conditions of Theorem 6; must be sanitized
Confident-GNMax	$(P/(2\sigma_1^2) + A/\sigma_2^2)\alpha$	Budget fixed in advance
Private in-context learning	$Q\,\varepsilon_{r,\sigma/\sqrt{2}}(\alpha)$	Poisson rate r, disjoint groups per query

Rules of thumb

- Private prediction costs per answer; private training costs once. Count your queries before choosing.
- The noise grows like $\sqrt{Q}$ and does not depend on the number of teachers; add teachers as long as their vote gaps grow.
- Use Gaussian noise for more than a handful of answers, Laplace only for one or two.
- A teacher ensemble is only as good as its noiseless vote; measure that ceiling first.
- Data-dependent ε is not a guarantee until it is released privately; report the data-independent one, or pay for the release.
- Never put private exemplars in a prompt that answers users directly; vote over disjoint groups with noise, or distil into a public prompt.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

41. Public data and private fine-tuning

41.1. Why this matters

The previous chapters trained models with differential privacy (DP) from random weights, by stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, called DP-SGD, as in DP-SGD: every step adds Gaussian noise to a sum of clipped per-example gradients, and an accountant turns the steps into one (ε, δ). On the part's running example it costs little at $\varepsilon = 8$ and a great deal at $\varepsilon = 1$ and below. The most accurate private models in the literature are not trained that way. They start from a model **pretrained** non-privately on a large **public** dataset, such as text or images collected from the web, and only **fine-tune** it privately on the sensitive data. Private image classifiers and private language models fine-tuned this way are far more accurate, at the same ε, than the same models trained privately from scratch (Tramèr & Boneh 2021; Yu et al. 2022; Li et al. 2022; De et al. 2022).

This chapter asks why that helps so much, and why a second practice helps on top of it: fine-tuning only a small part of the model, such as the last layer or a low-rank update to each weight matrix, often beats fine-tuning all of it under DP, although without privacy it would usually be the other way round.

41.1.1. The one idea: the noise is paid per trainable direction, the signal is not

DP-SGD adds noise of the same size to every coordinate it updates: with noise multiplier σ, clipping norm C and expected batch size B, the noise on the averaged gradient has standard deviation $\sigma C/B$ in each of the k trainable coordinates, so its length is about $\sigma C\sqrt{k}/B$. The useful part of the gradient, the average over the records, does not grow with k: it has length at most C whatever the number of coordinates, and it is usually concentrated in a few directions. Every trainable coordinate that the task does not need is a coordinate that only collects noise.

Public data helps in two ways that both follow from this. First, it moves the starting point close to a good model, so that what remains to be learned privately is small and lies in few directions. Second, it tells you which directions those are, before you touch the private data and at no privacy cost: the last layer on top of good features, a low-rank update, or the top principal directions of public gradients. Restricting training to those directions cuts the noise without cutting the signal. Whether it is worth restricting depends on how much of the signal the restriction throws away, and that is what decides between fine-tuning all weights and fine-tuning a few.

Table 41.1 lists the results and where each is used.

Table 41.1.: The results of this chapter and where each is used.

Result	Where it is used
The noise's length grows like $\sqrt{k}$; its cost grows with the number of directions it has time to fill, d_{eff}	Deciding how many weights to train; why full fine-tuning can still work
A linear model on fixed features is private convex empirical risk minimization (ERM)	Linear probing; when features from public data suffice
A low-rank adapter has $r(d_{\text{in}} + d_{\text{out}})$ weights and outer-product gradients	Parameter-efficient private fine-tuning; ghost clipping of adapters
Private gradients lie near a public low-dimensional subspace	Projected DP-SGD and gradient embedding; why fine-tuning is low-dimensional
Pretraining and fine-tuning on the running example	What each strategy buys at each ε
The guarantee covers the private data only	What to check about "public" data before relying on it

Notation. As in DP-SGD: n private records, C the clipping norm, σ the noise multiplier, q the sampling rate, $B = qn$ the expected batch size, T the number of steps and η the step size. The model has p weights, of which k **are trainable** during private fine-tuning; the others are frozen at their pretrained values. $s = \sigma C/B$ is the noise standard deviation per coordinate on the averaged gradient. D_{pub} is the public dataset, used freely, and D the private one. H is the curvature (Hessian) of the training loss at the starting point, with eigenvalues $h_1 \geq h_2 \geq \ldots$. Neighbouring private datasets differ by adding or removing one record.

The running example. The private task is the part's running example: 50-dimensional standard normal inputs labelled by a fixed random teacher network with 8 hidden tanh units, $n = 10{,}000$ training records, and a student with 64 hidden tanh units and $p = 3{,}329$ weights. The public task, built for this chapter, has 50,000 inputs from the same distribution labelled by a **related teacher**, which shares most of the private teacher's structure but not all of it; Section 41.7 gives the details. Pretraining on it helps on the private task without solving it. Fine-tuning runs DP-SGD with $q = 0.01$, $C = 0.5$ and $T = 10{,}000$ steps, the part's configuration, at $\delta = 10^{-5}$.

How to read this chapter. Read the toolkit and Section 41.3, which explains the rest. Section 41.4 and Section 41.5 are the two standard ways to train few weights; Section 41.6 explains why training all of them can still work. Section 41.7 runs everything on the running example, and Section 41.8 says what the guarantee does and does not cover. Read the last one even if you skip the others.

41.2. Toolkit: four facts every derivation uses

41.2.1. Fact 1 — DP-SGD's guarantee depends on the clipped sum, not on which weights it updates

Each step of DP-SGD releases $\sum_{i\in S_t}\bar{g}_i+\mathcal{N}(0,\sigma^2C^2I)$, where $\bar{g}_i$ is record i's gradient clipped to length at most C and S_t is a Poisson-sampled batch. Adding or removing one record changes the sum by one clipped gradient, of length at most C, so the noise is σ times the sensitivity, and the accountant composes the steps into (ε,δ) from q, σ and T alone (DP-SGD). Nothing in this argument depends on what the gradient is a gradient *of*: all weights, the last layer, the factors of a low-rank update, or a projection of the gradient onto a fixed subspace. The clipping simply applies to the vector of trainable coordinates, whatever it is. With $q=0.01$ and $T=10{,}000$, the accountant gives $\sigma=7.72$, 4.13, 2.28 and 0.917 for $\varepsilon=0.5$, 1, 2 and 8 at $\delta=10^{-5}$.

41.2.2. Fact 2 — A Gaussian vector's length grows like the square root of its dimension, and a projection keeps it Gaussian

If $\xi\sim\mathcal{N}(0,s^2I_k)$, then $\mathbb{E}\|\xi\|^2=ks^2$, a sum of k squared coordinates each with mean s^2, and $\|\xi\|$ concentrates near $s\sqrt{k}$ (Concentration inequalities). If P is the orthogonal projection onto a fixed m-dimensional subspace, then $P\xi$ is again Gaussian, with standard deviation s along each of m orthonormal directions of the subspace and nothing outside it. The reason is rotation invariance: the density of ξ depends only on $\|\xi\|$, so in any orthonormal basis its coordinates are independent $\mathcal{N}(0,s^2)$, and the projection keeps m of them. So noise restricted to a subspace has length about $s\sqrt{m}$.

41.2.3. Fact 3 — Public data is free, and so is anything computed from it

DP bounds how the output's distribution changes when one *private* record changes. A computation that reads only public data, such as pretraining, choosing a subspace from public gradients or tuning on a public validation set, produces a fixed object as far as the private data are concerned, and using it as the starting point or a hyperparameter of a private algorithm costs nothing. In the DP-SGD proof this enters as the condition that the initial weights w_0 are chosen independently of the private data (DP-SGD). Anything computed afterwards from the private algorithm's output is post-processing (Post-processing, group privacy and composition).

41.2.4. Fact 4 — A dense layer's per-example gradient is an outer product

For a layer that maps an input $a\in\mathbb{R}^{d_{\text{in}}}$ to Wa, the gradient of record i's loss with respect to W is $\delta_i a_i^\top$, where δ_i is the gradient with respect to the layer's output, and its length is $\|\delta_i\|\,\|a_i\|$, because the squared entries $\delta_{ij}^2a_{il}^2$ sum to $(\sum_j\delta_{ij}^2)(\sum_l a_{il}^2)$. Clipping can therefore compute each record's gradient norm without forming the gradient, **ghost clipping** (DP-SGD).

41.3. Why dimension costs, and when it does not

41.3.1. Context

Every bound on private learning in this part grows with the dimension. Private empirical risk minimization shows that the best excess risk achievable under (ε, δ)-DP grows like $\sqrt{d}$ for convex Lipschitz losses and like d for strongly convex ones, and that a matching lower bound holds. The noise in DP-SGD has length $\sigma C\sqrt{k}/B$, growing with the number of trainable weights. Yet large pretrained language models with hundreds of millions of weights are fine-tuned privately with good results. How much does each trainable direction really cost, and what decides it?

41.3.2. Statement

The noise on the averaged gradient, restricted to the k trainable coordinates, has length

$$\|\xi_t\|/B \approx s\sqrt{k} = \frac{\sigma C\sqrt{k}}{B}. \tag{41.1}$$

Near the starting point, model the loss as a quadratic with curvature H. Then T steps of noisy gradient descent with step size η and noise of standard deviation s per coordinate raise the expected loss, compared with the same steps without noise, by

$$\Delta = \sum_{j=1}^{k} \frac{\eta s^2}{2(2-\eta h_j)}\Big(1-(1-\eta h_j)^{2T}\Big) \approx \frac{\eta s^2}{4}\, d_{\text{eff}}, \qquad d_{\text{eff}} = \sum_{j=1}^{k}\Big(1-(1-\eta h_j)^{2T}\Big), \tag{41.2}$$

where h_j are the eigenvalues of H restricted to the trainable coordinates, $0 \le h_j < 2/\eta$, and the approximation assumes $\eta h_j \ll 1$. The **effective dimension** d_{eff} satisfies

$$d_{\text{eff}} \le \min\Big(k,\ 2\eta T \operatorname{tr} H\Big). \tag{41.3}$$

41.3.3. Derivation

The idea is to follow the noise separately from the signal. On a quadratic, the dynamics are linear, so the noise's contribution to the weights is a sum of independent Gaussian kicks, each shrunk by the curvature. Directions with high curvature forget old kicks quickly and reach a steady state; directions with almost no curvature keep accumulating kicks but charge very little for them.

1. **Separate the noise.** On the quadratic $J(w) = J(w_*) + \frac{1}{2}(w-w_*)^\top H(w-w_*)$ the gradient is $H(w-w_*)$. Let w_t follow the noisy steps $w_{t+1} = w_t - \eta(H(w_t - w_*) + \xi_t)$ and $\bar{w}_t$ the same steps without noise, from the same start. Their difference $e_t = w_t - \bar{w}_t$ obeys $e_{t+1} = (I - \eta H)e_t - \eta\xi_t$ with $e_0 = 0$, because the terms with w_* are the same for both and cancel. So e_t is a linear function of the noise alone, a Gaussian with mean zero.
2. **The loss increase.** Expanding the quadratic around $\bar{w}_T$, $J(w_T) = J(\bar{w}_T) + \nabla J(\bar{w}_T)^\top e_T + \frac{1}{2}e_T^\top H e_T$. The middle term has mean zero, because e_T has mean zero and is independent of the deterministic $\bar{w}_T$. So $\Delta = \frac{1}{2}\mathbb{E}[e_T^\top H e_T]$.

3. **One direction at a time.** In the eigenbasis of H the coordinates decouple, and by Fact 2 each coordinate of the noise is still $\mathcal{N}(0, s^2)$, independently. Along an eigenvector with curvature h, the coordinate e of the error obeys $e_{t+1} = (1-\eta h)e_t - \eta\xi$, so after T steps it is $-\eta\sum_{t<T}(1-\eta h)^{T-1-t}\xi_t$, with variance

$$v_T = \eta^2 s^2 \sum_{t=0}^{T-1}(1-\eta h)^{2t} = \eta^2 s^2\,\frac{1-(1-\eta h)^{2T}}{1-(1-\eta h)^2}, \tag{41.4}$$

   a geometric sum.
4. **Its cost.** The direction contributes $\frac{1}{2}hv_T$ to Δ. Since $1-(1-\eta h)^2 = \eta h(2-\eta h)$, this is $\frac{\eta s^2}{2(2-\eta h)}(1-(1-\eta h)^{2T})$, and summing over the directions gives Equation 41.2. With $\eta h \ll 1$ the first factor is $\eta s^2/4$.
5. **Two limits.** For $2\eta hT \gg 1$ the bracket is 1: the direction is at its steady state, the case Private empirical risk minimization analyses, and it costs $\eta s^2/4$ whatever its curvature (stiffer directions wander less but charge more per unit of wandering). For $2\eta hT \ll 1$ the bracket is about $2\eta hT$, by $(1-x)^{2T} \approx 1-2Tx$, and the direction costs about $\frac{\eta s^2}{4}\cdot 2\eta hT$: the noise has random-walked a distance of variance $T\eta^2 s^2$, charged at rate $h/2$, which is small when the direction is flat.
6. **The bound.** Each bracket is at most 1, which gives $d_{\text{eff}} \le k$. And $1-(1-x)^{2T} \le 2Tx$ for $0 \le x \le 1$ (Bernoulli's inequality, $(1-x)^{2T} \ge 1-2Tx$), so $d_{\text{eff}} \le 2\eta T\sum_j h_j = 2\eta T\,\operatorname{tr} H$.

41.3.4. Interpretation

The count of trainable weights is an upper bound, not the price. Equation 41.2 says that noise in a direction the loss does not care about, $h_j \approx 0$, costs nothing, however long it is. What you pay for is roughly the number of directions whose curvature exceeds $1/(2\eta T)$, the directions the noise has time to fill. A large pretrained model whose loss is curved in only a few directions near the pretrained weights can be fine-tuned in all its weights at a cost set by those few. This is the explanation Li, Liu, Hashimoto, Inan, Kulkarni, Lee & Thakurta (2022) give for why private fine-tuning of large language models does not suffer from their size: the optimal rates of Private empirical risk minimization are worst cases over all losses with a given Lipschitz constant, and they replace the dimension d by a quantity that measures how the gradients spread over directions.

Fewer trainable weights still help, for two reasons. Freezing weights removes their directions from the sum, so d_{eff} can only fall. And clipping happens to the trainable gradient: with fewer coordinates, per-example gradients are shorter, fewer are clipped, and the average clipped gradient, the signal, can be longer. Both effects appear in the example below. Against them stands the **bias** of freezing: if the task needs a direction that is frozen, no amount of private data can move it.

The step size and the number of steps are part of the dimension. d_{eff} grows with ηT: a longer or more aggressive run fills more flat directions with noise. This is one reason private fine-tuning works best with smaller step sizes than private training from scratch, as Section 41.7 finds.

How good is the model? The quadratic is a local model, accurate while fine-tuning stays near its starting point; the loss of a network is not quadratic far from it. And the loss increase is

not an accuracy loss: accuracy changes only when noise flips predictions. Use Equation 41.2 to compare strategies, not to predict accuracy.

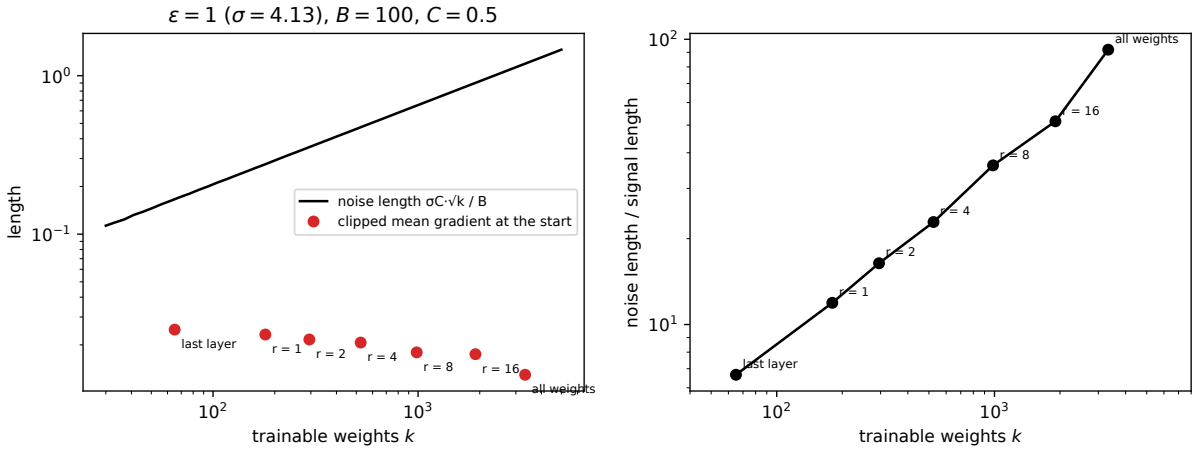


Figure 41.1.: Left: at $\varepsilon = 1$ ($\sigma = 4.13$, $B = 100$, $C = 0.5$), the noise length $\sigma C\sqrt{k}/B$ (line) against the number of trainable weights k, and the length of the average clipped gradient at the pretrained model (points) for the last layer, low-rank adapters of ranks 1 to 16, and all weights. Right: their ratio. The noise grows like $\sqrt{k}$ while the signal shrinks, because longer gradient vectors are clipped more.

41.3.5. Example: the running example after pretraining

Table 41.2 gives the noise length Equation 41.1 for the three trainable sets of Section 41.7, with $B = 100$ and $C = 0.5$.

Table 41.2.: Length of the noise on the averaged gradient, $\sigma C\sqrt{k}/B$, for three numbers of trainable weights.

ε	σ	All weights, $k = 3{,}329$	Rank-4 adapter and last layer, $k = 525$	Last layer, $k = 65$
0.5	7.72	2.227	0.884	0.311
1	4.13	1.191	0.473	0.166
2	2.28	0.658	0.261	0.092
8	0.917	0.265	0.105	0.037

The signal is the average clipped gradient over the private records at the start of fine-tuning. From the pretrained weights its length is 0.0130 when all weights are trained, 0.0207 for the adapter and 0.0250 for the last layer alone (Figure 41.1). The last layer's 65 coordinates carry a *longer* average clipped gradient than all 3,329 coordinates together, because each record's gradient restricted to 65 coordinates is shorter and less often clipped. At $\varepsilon = 1$ the noise is 92.0 times the signal for all weights, 22.9 times for the adapter and 6.7 times for the last layer. By the break-even argument of DP-SGD, a signal of constant direction outgrows a random walk of noise

after (noise/signal)^2 steps: 8,455, 523 and 44 steps. From a random initialization the signal is 0.0618, five times longer than from the pretrained weights, because a random network gets many records wrong and their gradients are long; but it shrinks quickly as training proceeds, to 0.026 after 1,000 steps in DP-SGD.

The curvature tells a less pessimistic story. Take the **Gauss–Newton curvature** of the private loss at the pretrained weights: the Hessian of the logistic loss with the network replaced by its linearization around the current weights, the average over records of $\pi_i(1-\pi_i)\nabla f_i \nabla f_i^\top$, with f_i the network's output on record i and π_i its predicted probability. It is positive semi-definite, and near a good fit it is close to the full Hessian, which is what the quadratic model needs. Estimated from 2,000 private records, it has largest eigenvalue 0.723 and trace 16.310 over all weights; for the last layer alone the largest eigenvalue is 0.142 and the trace 0.681. With $\eta = 0.1$ and $T = 10{,}000$, only 406 of the 3,329 directions have curvature above $1/(2\eta T) = 5 \times 10^{-4}$, and $d_{\text{eff}} = 451$; for the last layer, $d_{\text{eff}} = 60.2$ of 65. The full model's noise is $\sqrt{3{,}329/65} = 7.2$ times longer than the last layer's, but by Equation 41.2 it costs only $451/60.2 = 7.5$ times more, not $3{,}329/65 = 51$ times. At $\varepsilon = 1$, where $s = 0.0206$, Equation 41.2 predicts a loss increase of 0.0048 for all weights at $\eta = 0.1$, against 0.0355 if every direction had reached its steady state, and 0.00064 for the last layer alone.

41.3.6. Check yourself

- You double the number of steps at the same σ (ignoring that this would raise ε). Which directions' noise cost changes? *(Only the flat ones, $2\eta h T \lesssim 1$, whose cost doubles roughly; directions already at their steady state cost the same.)*
- Why does $d_{\text{eff}} \le 2\eta T \operatorname{tr} H$ not help much in the running example? *(Because $2\eta T \operatorname{tr} H = 2 \times 0.1 \times 10{,}000 \times 16.31 = 32{,}620$, far above $k = 3{,}329$. The trace is dominated by a few large eigenvalues; the bound is useful only when the total curvature is small compared with $1/(\eta T)$.)*

41.4. Good features: the last layer as private convex ERM

41.4.1. Context

The simplest way to use a pretrained model is to freeze everything except the last layer and train a linear classifier on the frozen features, **linear probing**. For the running example that is logistic regression on the 64 hidden activations plus a bias, 65 weights. It turns private deep learning back into the convex problem of Private empirical risk minimization, where the rates and the methods are known. Tramèr & Boneh (2021) showed that this simple recipe beats end-to-end private training of convolutional networks on standard image benchmarks at moderate budgets, even with handcrafted features (the ScatterNet wavelet transform), and that features learned on public data do better still. What does the private part of a linear probe cost, and what limits it?

41.4.2. Statement

Let $\phi(x) \in \mathbb{R}^k$ be the frozen features, computed from public data only, and $\ell(w,(x,y)) = \ln(1+e^{-y\langle w,\phi(x)\rangle})$. Then:

1. The per-record gradient has length at most $\|\phi(x)\|$. For $k-1$ tanh features and a constant 1, $\|\phi(x)\| \le \sqrt{k}$, so the loss is $\sqrt{k}$-Lipschitz in w and clipping is optional.
2. The excess risk of a private probe splits into a **feature gap** and a **privacy cost**:

$$R(\tilde{w}) - R_{\text{best}} = \underbrace{R(w^\phi) - R_{\text{best}}}_{\text{features}} + \underbrace{R(\tilde{w}) - R(w^\phi)}_{\text{privacy}}, \tag{41.5}$$

   where R is the risk, R_{best} the best risk of any model, and w^ϕ the best linear model on the features ϕ. The first term does not depend on n or ε. The second is private convex ERM in k dimensions, whose best rate for losses that are L-Lipschitz over a ball of radius R_w is $\tilde{\Theta}(LR_w\sqrt{k\ln(1/\delta)}/(n\varepsilon))$ (Bassily, Smith & Thakurta 2014).

41.4.3. Derivation

1. **The gradient bound.** $\nabla_w \ell = \ell'(m)\, y\, \phi(x)$ with $m = y\langle w, \phi(x)\rangle$ the margin and $\ell'(m) = -1/(1+e^m) \in (-1,0)$. So $\|\nabla_w \ell\| \le \|\phi(x)\|$. Each tanh activation lies in $(-1,1)$, so $\|\phi(x)\|^2 < (k-1)+1 = k$.
2. **The split.** Add and subtract $R(w^\phi)$. The first difference compares two non-private quantities and contains no noise.
3. **The privacy term.** By Fact 3, ϕ is a fixed function, so the probe is an ordinary convex ERM problem on the transformed records $(\phi(x_i), y_i)$, with dimension k and Lipschitz constant $\sqrt{k}$ or the clipping norm. The rate is the one Private empirical risk minimization states, proved by Bassily, Smith & Thakurta (2014), and noisy gradient descent attains it. The Lipschitz constant and the dimension enter through the noise $\propto L\sqrt{k}$ that every private method must add.

41.4.4. Interpretation

Features cap accuracy; privacy only decides how close you get to the cap. The feature gap is the price of freezing, paid even with infinite data and no privacy. The privacy cost falls like $\sqrt{k}/(n\varepsilon)$. With few features and good ones, both are small; with few features and poor ones, the probe is private, cheap and bad. This is the trade-off behind the title of Tramèr & Boneh's paper: private learning needs better features, or much more data. End-to-end training can eventually beat a probe, because it can learn better features, but it pays for every one of its weights to do so, and at small ε and small n it does not catch up.

Why linear probing is the safe first baseline. The problem is convex, so the optimizer cannot wander into a bad region, and the theory of Private empirical risk minimization applies: output perturbation, objective perturbation and noisy gradient descent all work, with known rates and no clipping bias if you use the Lipschitz bound. It has few hyperparameters, and with very few weights it can even be run full-batch. If a probe on public features is already within your accuracy target, there is little reason to fine-tune more.

Clipping below the Lipschitz bound still pays. The bound $\sqrt{k}$ is a worst case: it is reached only by a record with every activation saturated and a large negative margin. Most records' gradients are much shorter, because a record classified with a wide margin has $|\ell'(m)|$ close to 0. Clipping at a smaller C biases the gradient of the few long ones and divides the noise by $\sqrt{k}/C$; Exercise 3 in the exercise book compares the two on the running example.

41.4.5. Example: the ceiling and the cost on the running example

The pretrained network's 64 hidden units plus a constant give $k = 65$ features; the largest feature norm on the private training records is 7.035, below the bound $\sqrt{65} = 8.062$. The pretrained network itself, with its public last layer, labels 0.9035 of the private test inputs correctly (the report half, Section 41.7). Fitting the last layer exactly and non-privately on the 10,000 private records, by Newton's method, raises this to 0.9536: that is the probe's ceiling, w^{ϕ} in Equation 41.5. A network trained non-privately on the private data from scratch reaches 0.9606, so the feature gap is 0.007 in accuracy. The private probe reaches 0.9478 at $\varepsilon = 0.5$ and 0.9514 at $\varepsilon = 8$ (Table 41.5): at every budget it is within 0.006 of its ceiling, and at no budget can it pass it.

41.4.6. Check yourself

- The public features are replaced by 650 features of the same quality (same ceiling). What happens to the privacy cost of the probe? *(The noise length grows by $\sqrt{10} \approx 3.2$ at the same σ, so the privacy term of Equation 41.5 grows by about that factor at a fixed $n\varepsilon$; you need about 3.2 times more data for the same cost.)*
- Is the probe private if the features were computed by a network pretrained on data that included some of the private records? *(The probe is private with respect to the private dataset given the features; but the features themselves are a function of those records and were not trained privately, so the released model as a whole has no guarantee for them. See Section 41.8.)*

41.5. Low-rank adapters

41.5.1. Context

The last layer is often not enough: the features of a model pretrained on one task are not exactly right for another, and the probe stops at its ceiling. Training all weights fixes the features but pays for every weight. **Low-rank adaptation (LoRA)** sits in between: it freezes each pretrained weight matrix W_0 and learns a low-rank correction, $W = W_0 + UV^{\top}$ with $U \in \mathbb{R}^{d_{\text{out}} \times r}$ and $V \in \mathbb{R}^{d_{\text{in}} \times r}$ (Hu et al. 2022). Yu et al. (2022) showed that private fine-tuning of language models with LoRA and similar parameter-efficient methods matches or beats private full fine-tuning at a fraction of the cost. How many weights does it train, what are its per-example gradients, and in which directions can it move W?

41.5.2. Statement

A rank-r adapter on a $d_{\text{out}} \times d_{\text{in}}$ matrix has

$$k = r(d_{\text{in}} + d_{\text{out}}) \tag{41.6}$$

trainable weights, against $d_{\text{in}} d_{\text{out}}$ for the matrix. With δ_i the gradient of record i's loss with respect to the layer's output and a_i its input, the per-example gradients are outer products,

$$\nabla_U \ell_i = \delta_i \, (V^\top a_i)^\top, \qquad \nabla_V \ell_i = a_i \, (U^\top \delta_i)^\top, \tag{41.7}$$

with lengths $\|\delta_i\| \|V^\top a_i\|$ and $\|a_i\| \|U^\top \delta_i\|$, so ghost clipping applies. One step with noisy gradient sums $G_U = \sum_i \overline{\nabla_U \ell_i} + \xi_U$ and $G_V = \sum_i \overline{\nabla_V \ell_i} + \xi_V$ and step η/B changes W by

$$\Delta W = -\frac{\eta}{B}(G_U V^\top + U G_V^\top) + O\Big(\frac{\eta^2}{B^2}\Big). \tag{41.8}$$

In particular, from the usual start $U = 0$, the first step's change has rank at most r, and to first order its rows lie in the span of the columns of V. The privacy guarantee is that of DP-SGD with the same q, σ and T (Fact 1).

41.5.3. Derivation

1. **Counting.** U has $d_{\text{out}} r$ entries and V has $d_{\text{in}} r$.
2. **The chain rule through the product.** By Fact 4 the gradient with respect to W is $G_i = \delta_i a_i^\top$, and the loss depends on U and V only through W. A small change dU changes W by $dU\, V^\top$, which changes the loss by $\operatorname{tr}(G_i^\top dU\, V^\top) = \operatorname{tr}(V^\top G_i^\top dU) = \langle G_i V, dU \rangle$, using the cyclic property of the trace. So $\nabla_U \ell_i = G_i V = \delta_i (a_i^\top V) = \delta_i (V^\top a_i)^\top$. In the same way a change dV changes W by $U\, dV^\top$ and the loss by $\operatorname{tr}(G_i^\top U\, dV^\top) = \langle G_i^\top U, dV \rangle$, so $\nabla_V \ell_i = G_i^\top U = a_i (\delta_i^\top U) = a_i (U^\top \delta_i)^\top$. This is Equation 41.7.
3. **Norms.** Each is an outer product of two vectors, so its length is the product of their lengths, as in Fact 4. The norm of record i's whole gradient is the square root of the sum of the squared norms over U, V and the other trainable layers, computed from vectors of length d_{out}, d_{in} and r without forming any matrix.
4. **The step.** After the step, $W' = W_0 + (U - \frac{\eta}{B} G_U)(V - \frac{\eta}{B} G_V)^\top$. Expanding the product, $W' - W = -\frac{\eta}{B}(G_U V^\top + U G_V^\top) + \frac{\eta^2}{B^2} G_U G_V^\top$, which is Equation 41.8.
5. **From $U = 0$.** Then $\nabla_V \ell_i = 0$ for every record, so $G_V = \xi_V$ is pure noise, and $\Delta W = -\frac{\eta}{B} G_U V^\top + \frac{\eta^2}{B^2} G_U \xi_V^\top$. The first term is a $d_{\text{out}} \times r$ matrix times $V^\top$, of rank at most r, with rows that are combinations of the columns of V. The second term is of order η^2/B^2 and tilts those rows slightly. The whole ΔW is a product of a $d_{\text{out}} \times r$ and an $r \times d_{\text{in}}$ matrix, so its rank is at most r exactly.

41.5.4. Interpretation

An adapter is DP-SGD in a smaller, moving space. The noise of DP-SGD on (U, V) lives in $k = r(d_{\text{in}} + d_{\text{out}})$ coordinates, so Equation 41.1 applies with that k. Seen from W, each step moves the matrix within a set of dimension about k: the directions $XV^\top + UY^\top$ for arbitrary X

and Y, which change as U and V change. The adapter restricts the update to that set; noise in the other $d_{\text{in}}d_{\text{out}} - k$ directions of W never arises.

What the restriction costs. At the start, with $U = 0$ and V random, the update is the true gradient G multiplied on the right by $VV^\top$, which for a random V with small r is close to a multiple of the projection onto the span of its columns: the gradient is projected onto a random r-dimensional subspace of the inputs. Random subspaces capture a fraction of about r/d_{in} of a fixed vector's energy, which for small r is little. As V moves (its gradient is no longer zero once $U \neq 0$) the subspace aligns with the useful directions. Low-rank adaptation works when the change from the pretrained weights that the new task needs is itself close to low rank, as Hu et al. (2022) observed for language models. The rank r trades bias against noise like the number of trainable weights does.

Full fine-tuning is not ruled out. Li, Tramèr, Liang & Hashimoto (2022) fine-tuned all the weights of pretrained language models with DP-SGD and matched or beat the parameter-efficient methods, once the hyperparameters were right: large batches, a learning rate tuned for private training, and a fine-tuning objective aligned with pretraining. Equation 41.2 explains why this can work: what full fine-tuning pays is d_{eff}, not p. Whether an adapter or full fine-tuning wins is an empirical question that depends on the model, the data and the tuning, and the published evidence goes both ways. The adapter's advantages that do not depend on accuracy are certain: it stores and communicates only k numbers per task, and it computes and clips much smaller gradients.

41.5.5. Example: counting, and checking the gradients

The running example's first layer maps 51 inputs (50 features and a constant) to 64 hidden units: 3,264 weights. A rank-4 adapter trains $4 \times (51 + 64) = 460$ of them, plus the 65 weights of the last layer, $k = 525$; rank 1 gives 180 and rank 16 gives 1,905. For a 768×768 projection of a transformer, rank 8 trains $8 \times 1{,}536 = 12{,}288$ weights instead of 589,824, a factor of 48 fewer, so the noise is $\sqrt{48} = 6.9$ times shorter. On 256 private records, the ghost norms of Equation 41.7 agree with explicitly formed per-example gradients of the adapter to a relative difference of 3.3×10^{-16}. Exercise 4 in the exercise book checks the rank of the first step.

41.5.6. Check yourself

- Why does LoRA start with $U = 0$ rather than both factors random? *(So that $W = W_0$ at the start: the fine-tuned model begins exactly at the pretrained one. With both random, the first forward pass would use a randomly perturbed matrix.)*
- An adapter of rank $r = \min(d_{\text{in}}, d_{\text{out}})$ can represent any change of W. Does it then have the same noise as training W directly? *(No, more: it has $r(d_{in} + d_{out})$ coordinates, more than $d_{in}d_{out}$ when r is this large. Adapters save noise only at small rank.)*

41.6. Gradients near a public subspace

41.6.1. Context

Section 41.3 says the noise costs little in directions the loss does not use. If you knew the directions the private gradients do use, you could add noise only there. Public data gives an estimate: compute per-example gradients on public records at the current or pretrained weights, find their top principal directions, and project the private gradients onto them. Zhou, Wu & Banerjee (2021) do this as **projected DP-SGD**, and Yu, Zhang, Chen & Liu (2021) as **gradient embedding perturbation**, which also releases the residual outside the subspace with separate noise. Li et al. (2022) measured that the gradients of private fine-tuning are dominated by a few principal components. How much does a projection save, and what does it lose?

41.6.2. Statement

Let P_m be the orthogonal projection onto an m-dimensional subspace computed from public data only. Releasing

$$P_m\Big(\sum_{i\in S_t}\bar{g}_i\Big)+\mathcal{N}(0,\sigma^2C^2P_m) \tag{41.9}$$

(noise of standard deviation σC along each of m orthonormal directions of the subspace) at every step has the same (ε,δ) as DP-SGD with the same q, σ and T. If $\bar{m}$ is the average clipped gradient and $f_m=\|P_m\bar{m}\|^2/\|\bar{m}\|^2$ the fraction of its energy that the subspace captures, the kept signal has length $\sqrt{f_m}\|\bar{m}\|$, the noise on the averaged gradient has length about $s\sqrt{m}$, and signal outgrows noise after about

$$T_{\text{even}}(m)=\Big(\frac{s\sqrt{m}}{\sqrt{f_m}\,\|\bar{m}\|}\Big)^2 \tag{41.10}$$

steps, against $(s\sqrt{p}/\|\bar{m}\|)^2$ without projection. The lost fraction $1-f_m$ of the signal is a bias that no number of steps recovers.

41.6.3. Derivation

1. **Sensitivity.** A projection never lengthens a vector: $\|P_mx\|\le\|x\|$, because $x=P_mx+(x-P_mx)$ with the two parts orthogonal, so $\|x\|^2=\|P_mx\|^2+\|x-P_mx\|^2$. Adding or removing one record changes the projected sum by $P_m\bar{g}$, of length at most C.
2. **The mechanism.** In an orthonormal basis of the subspace, Equation 41.9 is m coordinates of a vector of sensitivity at most C plus independent $\mathcal{N}(0,\sigma^2C^2)$ noise on each: a Gaussian mechanism with ratio $1/\sigma$, exactly as in DP-SGD, and Poisson sampling makes it the same subsampled Gaussian. The subspace is fixed before the private data are read (Fact 3), so it is a public parameter of the mechanism. Equivalently, Equation 41.9 is the projection of the ordinary DP-SGD release onto the subspace (Fact 2), and projection is post-processing.
3. **Signal and noise.** Dividing by B, the kept signal is $P_m\bar{m}$, of length $\sqrt{f_m}\|\bar{m}\|$ by the definition of f_m, and the noise has m coordinates of standard deviation s, length about $s\sqrt{m}$ (Fact 2).
4. **Break-even.** As in DP-SGD, over j steps a signal of constant direction adds up to j times its length and independent noise to $\sqrt{j}$ times; they are equal at Equation 41.10.

41.6.4. Interpretation

A projection trades bias for variance, and public data sets the exchange rate. The noise falls by $\sqrt{p/m}$, the signal by $\sqrt{f_m}$. If the private gradients lie near the public subspace, f_m is close to 1 for $m \ll p$ and the trade is excellent. If the public data are unrelated, the subspace is no better than a random one, $f_m \approx m/p$, and the projection removes as much signal as noise, gaining nothing. The quality of the public data appears here directly as f_m.

A fixed subspace is a cap; methods refresh it or add the residual. Since P_m removes the part $1 - f_m$ of the signal at every step, training with a fixed projection can only move the weights within the subspace. Projected DP-SGD recomputes the subspace from public gradients as training proceeds; gradient embedding perturbation releases the residual $\bar{g} - P_m\bar{g}$ too, with its own noise, so the bias disappears at the price of a little more noise. LoRA, in this light, chooses a subspace by structure (low-rank changes of each matrix) and lets training move it.

Why this explains full fine-tuning. The same concentration that makes a public subspace useful makes d_{eff} small in Equation 41.2: gradients near a low-dimensional subspace are gradients of a loss curved in few directions. Projection makes the saving explicit and certain; full fine-tuning gets part of it implicitly, if the step size and number of steps are small enough that flat directions do not fill up with noise.

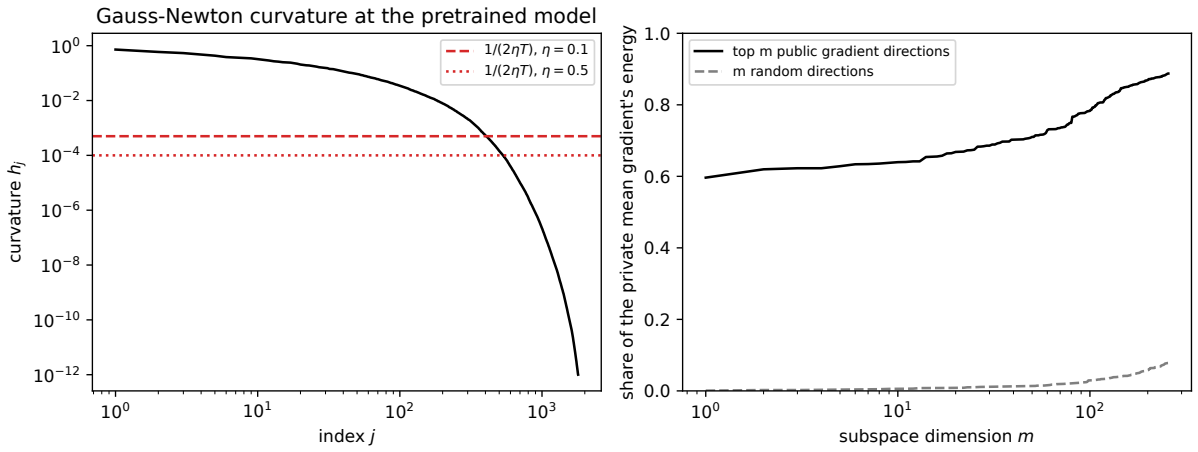


Figure 41.2.: Left: eigenvalues of the Gauss–Newton curvature of the private loss at the pretrained weights, estimated from 2,000 private records, on log scales, with the thresholds $1/(2\eta T)$ for $\eta = 0.1$ and 0.5 and $T = 10{,}000$; directions below a threshold contribute little noise cost. Right: the fraction of the energy of the private average clipped gradient that lies in the top m principal directions of 500 public per-example gradients (solid), and in m random directions (dashed).

41.6.5. Example: how concentrated the running example's gradients are

At the pretrained weights the average clipped private gradient has length 0.0130. The top principal directions of 500 clipped public per-example gradients capture Table 41.3's fractions of its energy; random subspaces of the same dimension capture about k/p.

Table 41.3.: How much of the private average clipped gradient lies in public subspaces, at the pretrained weights, and the steps after which the kept signal outgrows the noise at $\varepsilon = 1$ ($s = 0.0206$).

Dimension m	1	4	16	64	256	3,329 (all)
Public top-m directions, f_m	0.596	0.623	0.655	0.732	0.887	1
m random directions	0.001	0.001	0.004	0.022	0.080	1
Break-even steps at $\varepsilon = 1$, Equation 41.10	4	16	62	222	733	8,455

A single public direction holds 60% of the private gradient's energy, and 64 directions, 2% of the dimensions, hold 73%. At $\varepsilon = 1$ the kept signal outgrows the noise after 222 steps in the 64-dimensional subspace, against 8,455 steps for all weights. The cost is the 27% of the energy outside it, which a fixed projection never learns. The left panel of Figure 41.2 shows the curvature: its eigenvalues fall from 0.723 to below 5×10^{-4} within the first 406 directions and keep falling steeply after, so the noise of a run with $\eta = 0.1$ fills only the directions above the dashed line.

41.6.6. Check yourself

- The public subspace is computed from the private data instead, by a non-private principal component analysis. Is Equation 41.9 still (ε, δ)-DP? *(No. The subspace then depends on the private records, the projection is no longer a fixed public map, and the subspace itself leaks information about them; it must be computed privately, at a cost in budget, or from public data.)*
- When does projection onto m public directions not shorten the break-even time? *(When $f_m \leq m/p$ or close to it, so that the signal shrinks as much as the noise: for public data unrelated to the private task, or for a subspace computed at weights far from where training now is.)*

41.7. Pretraining and fine-tuning the running example

41.7.1. Context

How much does a related public dataset buy on the running example, at each budget, and which part of the model should be fine-tuned?

41.7.2. Statement

The public task. The private teacher has a first layer W_1^{priv} (8 hidden units, 51 inputs) and an output layer w_2^{priv}. The public teacher has first layer $W_1^{\text{priv}} + 0.15Z$ and output layer $\cos(0.3)\, w_2^{\text{priv}} + \sin(0.3)\, z$, where Z and z are fixed standard normal draws: the same hidden units, each slightly perturbed, read out with weights turned by 0.3 radians toward a random vector. It labels 50,000 inputs drawn from the private input distribution. The design makes pretraining useful but not sufficient: the public student must learn features close to the ones the private task needs, but cannot learn the private task itself. The public teacher agrees with the private one on 0.9004 of the private test inputs.

Pretraining. The 64-unit student is trained on the public data by non-private SGD, 10,000 steps with batches of 100 and step size 0.5, to 0.9861 training accuracy on the public task. On the private task it reaches 0.9035 without any private data.

Fine-tuning. From the pretrained weights, DP-SGD with $q = 0.01$, $C = 0.5$ and $T = 10{,}000$ trains: all 3,329 weights (*full*); the last layer, $k = 65$ (*last layer*); a rank-4 adapter $W_1 = W_1^{\text{pre}} + UV^\top$ and the last layer, $k = 525$ (*adapter*). The baseline trains all weights from a random initialization (*scratch*), the setting of DP-SGD. The private test set of 20,000 records is split in half: the step size of each method is chosen from three values on the first half at $\varepsilon = 1$ (Table 41.4) and used at every budget, and accuracy is reported on the second half.

Table 41.4.: Step-size selection at $\varepsilon = 1$, one run each, on the validation half of the private test set.

Method	Step sizes tried, accuracy on the validation half	Chosen
Scratch	0.25: 0.9318; 0.5: 0.9303; 1: 0.8582	0.25
Full	0.03: 0.9559; 0.1: 0.9567; 0.3: 0.9433	0.1
Last layer	0.1: 0.9479; 0.3: 0.9499; 1: 0.9460	0.3
Adapter, rank 4	0.03: 0.9556; 0.1: 0.9619; 0.3: 0.9472	0.1

Table 41.5 gives the results.

Table 41.5.: Test accuracy on the report half of the private test set, mean and standard deviation over two runs, $\delta = 10^{-5}$. The pretrained model without fine-tuning has 0.9035. Non-private: SGD for 1,000 steps from scratch and from the pretrained weights, and the exact logistic regression on the frozen features.

	Scratch	Full	Last layer	Adapter, rank 4
Trainable weights k	3,329	3,329	65	525
$\varepsilon = 0.5$	0.8589 ± 0.0075	0.9395 ± 0.0006	0.9478 ± 0.0010	0.9464 ± 0.0005
$\varepsilon = 1$	0.9246 ± 0.0050	0.9599 ± 0.0013	0.9497 ± 0.0000	0.9616 ± 0.0001
$\varepsilon = 2$	0.9518 ± 0.0010	0.9663 ± 0.0001	0.9519 ± 0.0005	0.9653 ± 0.0002
$\varepsilon = 8$	0.9611 ± 0.0012	0.9677 ± 0.0001	0.9514 ± 0.0005	0.9655 ± 0.0001
Non-private	0.9606	0.9704	0.9536	—

41.7.3. Derivation

Each entry is a measurement, and each method's privacy follows from Fact 1 and Fact 3: the pretrained weights and the adapter's random V are fixed before the private data are read, the clipped sum of each step has sensitivity C in the method's trainable coordinates, and the accountant gives $\sigma = 7.72$, 4.13, 2.28 and 0.917 for the four budgets, the same for every method because q and T are the same. The step sizes were chosen by looking at private data, the validation half, without paying for it; Hyperparameter tuning shows how to do that privately, and the comparison here is of methods at their best, as in DP-SGD.

41.7.4. Interpretation

Pretraining is worth about a factor of eight in ε. Every fine-tuning method at $\varepsilon = 0.5$ (0.9395 to 0.9478) beats training from scratch at $\varepsilon = 1$ (0.9246), and the best of them comes within 0.004 of training from scratch at $\varepsilon = 2$ (0.9518). The adapter at $\varepsilon = 1$ (0.9616) matches training from scratch at $\varepsilon = 8$ (0.9611) and the non-private model trained from scratch (0.9606), and full fine-tuning at $\varepsilon = 2$ (0.9663) beats both. The public data carry information about the private task that 10,000 private records do not, and that information is free.

Which weights to train depends on the budget. At $\varepsilon = 0.5$ the last layer is best: with 65 trainable weights its noise is $\sqrt{3{,}329/65} = 7.2$ times shorter than full fine-tuning's, and its ceiling, 0.9536, is not yet binding. From $\varepsilon = 1$ on, the last layer is stuck near its ceiling (0.9497 to 0.9519) while the adapter and full fine-tuning keep improving: they can correct the features, and the noise is now small enough to let them. The adapter, with 525 weights, is never more than 0.0022 below the best method at any budget: the middle choice is the robust one. Full fine-tuning is behind at $\varepsilon = 0.5$, by 0.0083, but not far behind elsewhere once its step size is right, as Li et al. (2022) found for language models; at $\varepsilon = 2$ and 8 it is the best, by 0.0010 and 0.0022. Figure 41.3, a separate set of single runs, shows the same overall picture, with the order of the three pretrained methods at $\varepsilon = 0.5$ and 1 changed: differences of a few thousandths between them are within run-to-run variation.

Step sizes matter as much as the method. In Table 41.4 full fine-tuning loses 1.3 points at a step size three times too large, and the adapter 1.5 points. The best step size for full fine-tuning and the adapter, 0.1, is 2.5 times smaller than for training from scratch. That is the effective-dimension argument of Section 41.3 at work: a larger η fills more flat directions with noise and undoes what pretraining provided.

What the run-to-run spread says. The standard deviations over two runs are small for the pretrained methods, 0.0013 or less, but two runs estimate a spread only roughly: the rank sweep below repeats the adapter at $\varepsilon = 1$ and finds a standard deviation of 0.0023, and a mean of 0.9624 against the table's 0.9616. Differences between the pretrained methods smaller than about 0.005 should not be read as rankings.

41.7.5. Example: the adapter's rank

At $\varepsilon = 1$, with the adapter's step size 0.1, two runs each give 0.9590 ± 0.0038 at rank 1 ($k = 180$), 0.9624 ± 0.0023 at rank 4 ($k = 525$) and 0.9586 ± 0.0031 at rank 16 ($k = 1{,}905$). Rank 4 is best, but the differences are about the size of the run-to-run spread. Two forces pull in opposite

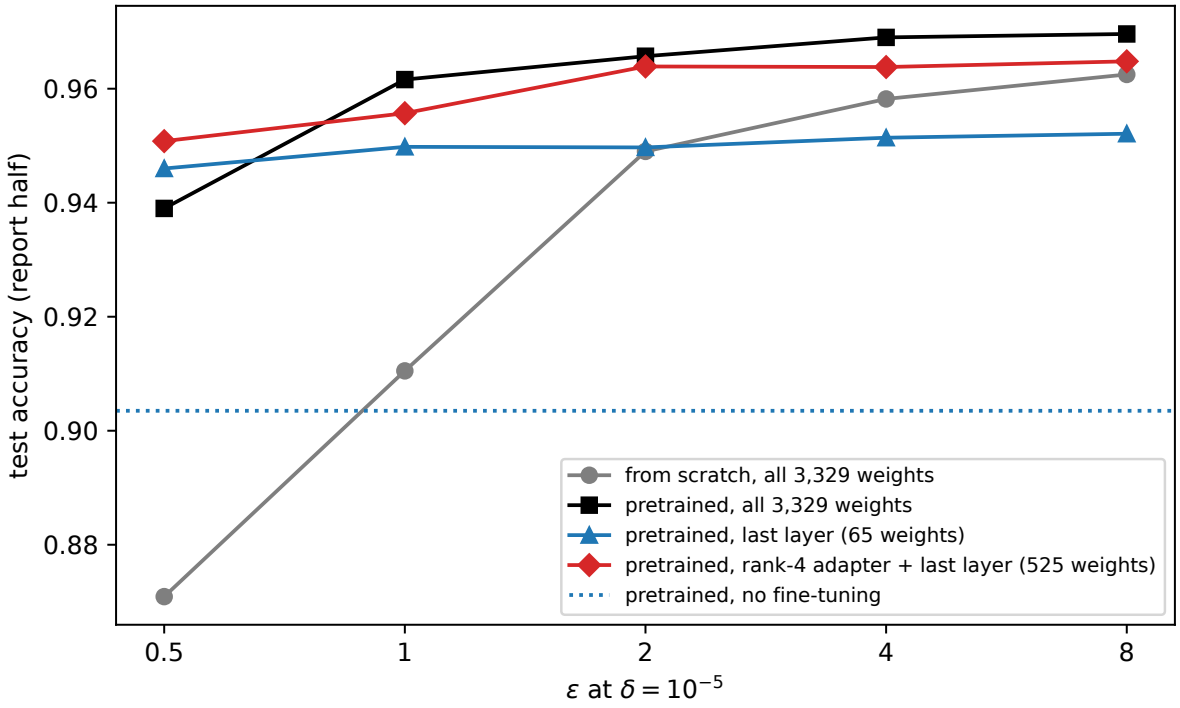


Figure 41.3.: Test accuracy of the four strategies against ε at $\delta = 10^{-5}$, one run each with the step sizes of Table 41.4 and a different seed from Table 41.5, with the pretrained model's accuracy before fine-tuning (dotted). The pretrained methods are far ahead of training from scratch at small budgets; the last layer stays near its ceiling from $\varepsilon = 1$ on, and training from scratch passes it at larger budgets.

directions: a higher rank lets the update follow more of the directions the task needs, and it adds noise, whose length grows like $\sqrt{k}$ (Figure 41.1). On this problem neither dominates over a sixteenfold range of ranks, so the rank is a cheap hyperparameter to get roughly right.

41.7.6. Check yourself

- Why is the last layer's accuracy at $\varepsilon = 8$ (0.9514) not higher than at $\varepsilon = 2$ (0.9519)? *(It is at its ceiling, the best linear model on the frozen features, 0.9536 non-privately. Less noise cannot help a model that cannot represent a better classifier.)*
- If the public teacher were identical to the private one, which strategy would you expect to win at $\varepsilon = 0.5$, and why? *(The last layer, or no fine-tuning at all: the pretrained model would already be close to perfect, so there would be nothing for the extra weights to learn and only noise for them to collect. Exercise 5 in the exercise book measures it.)*

41.8. What public data does not buy

41.8.1. Context

Everything so far treated the public data as free. Formally that is right: the guarantee of a private fine-tuning run does not depend on what the public data are. But "public" is a label someone chose, and the guarantee says nothing about the records under that label. Tramèr, Kamath & Carlini (2024) argue that the field's reliance on large web-scraped pretraining data needs scrutiny on several counts. What exactly is guaranteed, and what is not?

41.8.2. Statement

Let $\mathcal{A}(D_{\text{pub}}, D)$ pretrain on D_{pub} without privacy and fine-tune on D with an (ε, δ)-DP algorithm. Then for every fixed D_{pub}, the map $D \mapsto \mathcal{A}(D_{\text{pub}}, D)$ is (ε, δ)-DP. There is no guarantee of any kind with respect to D_{pub}. A person with a record x in both datasets is protected only for the copy in D: removing it from D changes the output's distribution by at most (ε, δ), while the copy in D_{pub} can be revealed completely.

41.8.3. Derivation

1. **Fixed public data.** With D_{pub} fixed, pretraining produces fixed weights w_0 (or a fixed distribution over them, if pretraining is randomized, independent of D). The DP-SGD proof of DP-SGD holds for any w_0 chosen independently of the private data (Fact 3), so it holds conditionally on w_0, and therefore for the mixture over w_0.
2. **Nothing for the public data.** Changing one record of D_{pub} can change w_0 arbitrarily, and the fine-tuned model inherits whatever w_0 encodes; non-private training can memorize individual records (Memorization and extraction). No noise was calibrated to that change.

3. **A record in both.** The neighbouring datasets that DP compares are D and D without x, with D_{pub} the same in both. A test of whether x was in D has true positive rate at most e^{ε} times its false positive rate plus δ (The definition). A test of whether x is in the training data at all can use the copy in D_{pub}, which nothing bounds.

41.8.4. Interpretation

Overlap and contamination. Web-scale "public" datasets contain personal data: posts, profiles, documents put online by others, data that leaked. A private fine-tuning set of medical notes or emails may overlap with them, and the people whose records appear in both are protected only as well as the pretraining protects them, that is, not at all. Calling the resulting model "differentially private" is accurate about D and misleading about those people. Tramèr, Kamath & Carlini (2024) argue for this reason that the label should be used with care, and that the privacy of pretraining data deserves its own consideration.

Distribution shift. Public data help as much as they resemble the private task. Benchmarks in which the "private" set is a standard dataset whose distribution is close to well-curated public data, for example two collections of natural images, can overstate what the method will do on genuinely sensitive data, which is often unlike anything public: medical images, rare languages, internal logs. Table 41.5 used a public task close to the private one by design; Exercise 5 in the exercise book shows the gain shrinking, and for the last layer turning into a loss against training from scratch, as the public teacher drifts away.

Public is not the same as non-sensitive. Even when every pretraining record is public, a model trained on them can reproduce them, link them, or make inferences about people that the original publication did not allow. And large pretrained models are often too large to fine-tune locally, so the private data are sent to whoever has the compute, which is a trust question DP does not address (Trust models).

What to report. State what the public data are, how they were obtained, whether overlap with the private data was checked (deduplication against the private set is the minimum), and that the (ε, δ) covers the private dataset given the pretrained model. Any choice made by looking at private data, such as which public model to start from or the step size, is part of the private computation and must be accounted for.

41.8.5. Example: one record, two copies

At $\varepsilon = 1$ and $\delta = 10^{-5}$, any test of whether a record was in the private fine-tuning set that has false positive rate 0.01 has true positive rate at most $e \times 0.01 + 10^{-5} = 0.0272$. If the same record is also in the public set, a membership test against the pretrained model, such as the likelihood-ratio attack of Membership inference, is bounded by nothing; it may still fail in practice, but no theorem says it will. If the record appears twice in the private set, group privacy gives $(2\varepsilon, 2e^{\varepsilon}\delta) = (2, 5.44 \times 10^{-5})$ for the pair (Post-processing, group privacy and composition), a weaker but real guarantee. A copy in the public set is worse than a duplicate in the private set.

41.8.6. Check yourself

- You choose between three public pretrained models by fine-tuning each privately and keeping the one with the best private validation accuracy. What have you spent? *(Three private training runs plus a selection on private data: the budget of all three, composed, or a private selection procedure from Hyperparameter tuning. Choosing by public validation data would have been free.)*
- A company deduplicates its private dataset against the public pretraining corpus by exact string matching. What can still go wrong? *(Near-duplicates and paraphrases of the same personal information survive exact matching, and records about the same person need not be duplicates at all; the deduplication protects copies, not people.)*

41.9. Code: check every number by computation

The script code/public_data.py builds the public task, pretrains the student, implements the last-layer and low-rank-adapter versions of DP-SGD with their per-example gradients (ghost norms included), estimates the curvature and the public gradient subspace, and runs the comparison. It imports the model, DP-SGD and the accountant from code/dp_sgd.py. From the repository root:

```
uv run python code/part05-dp-ml/public_data.py
```

It takes about four minutes on a laptop, most of it in about 70 training runs of 10,000 steps.

Output (seed 0), summarized in Table 41.6:

Table 41.6.: The script's output with the default seed.

Experiment	Result
Public task	Teacher agreement 0.9004; pretrained student 0.9035 on the private task
Noise length at $\varepsilon = 1$	1.191, 0.473 and 0.166 for $k = 3{,}329$, 525 and 65
Signal at the pretrained weights	0.0130, 0.0207 and 0.0250; break-even 8,455, 523 and 44 steps
Curvature	Largest eigenvalue 0.723, trace 16.310; $d_{\text{eff}} = 451$ against 60.2 for the last layer at $\eta = 0.1$
Public subspace	60% of the private gradient's energy in one direction, 73% in 64
Adapter ghost norms	Agree with explicit gradients to 3.3×10^{-16}
Fine-tuning, $\varepsilon = 0.5$	Last layer 0.9478, adapter 0.9464, full 0.9395, scratch 0.8589
Fine-tuning, $\varepsilon = 1$	Adapter 0.9616, full 0.9599, last layer 0.9497, scratch 0.9246

tests/test_public_data.py checks that the output still matches.

What to take from it. A related public dataset is worth more than any choice made afterwards: it moves accuracy at $\varepsilon = 1$ from 0.9246 to about 0.96. After that, the right number of trainable weights depends on the budget: few at very small ε, more as the noise allows. The noise length predicts the ranking at small budgets; the feature ceiling and the effective dimension explain the rest.

41.10. References

- Tramèr & Boneh, "Differentially Private Learning Needs Better Features (or Much More Data)" (ICLR 2021). Linear models on handcrafted ScatterNet features beat end-to-end private networks at moderate budgets; features from public data do better. The source of Section 41.4.
- Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang & Chen, "LoRA: Low-Rank Adaptation of Large Language Models" (ICLR 2022). The adapter of Section 41.5, without privacy.
- Yu, Naik, Backurs, Gopi, Inan, Kamath, Kulkarni, Lee, Manoel, Wutschitz, Yekhanin & Zhang, "Differentially Private Fine-tuning of Language Models" (ICLR 2022). Private fine-tuning with LoRA and other parameter-efficient methods; reports, for example, 87.8% accuracy on a language-inference benchmark at $\varepsilon = 6.7$ against 90.2% without privacy, for a large RoBERTa model.
- Li, Tramèr, Liang & Hashimoto, "Large Language Models Can Be Strong Differentially Private Learners" (ICLR 2022). Full fine-tuning with the right hyperparameters matches parameter-efficient methods; ghost clipping for sequence models.
- Li, Liu, Hashimoto, Inan, Kulkarni, Lee & Thakurta, "When Does Differentially Private Learning Not Suffer in High Dimensions?" (NeurIPS 2022). Dimension-free bounds when gradients concentrate in few directions, and the measurement that fine-tuning gradients are dominated by a few principal components. The background of Section 41.3.
- Zhou, Wu & Banerjee, "Bypassing the Ambient Dimension: Private SGD with Gradient Subspace Identification" (ICLR 2021). Projected DP-SGD with a subspace estimated from public gradients.
- Yu, Zhang, Chen & Liu, "Do Not Let Privacy Overbill Utility: Gradient Embedding Perturbation for Private Learning" (ICLR 2021). Projection onto a public anchor subspace plus a separately noised residual.
- Tramèr, Kamath & Carlini, "Position: Considerations for Differentially Private Learning with Large-Scale Public Pretraining" (ICML 2024). What the public-pretraining paradigm does and does not protect, and how benchmarks can mislead. The source of Section 41.8.
- Bassily, Smith & Thakurta, "Private Empirical Risk Minimization: Efficient Algorithms and Tight Error Bounds" (FOCS 2014). The rates for private convex ERM used in Section 41.4.
- De, Berrada, Hayes, Smith & Balle, "Unlocking High-Accuracy Differentially Private Image Classification through Scale" (arXiv 2022). Private fine-tuning of image models pretrained on large public datasets, among other practices.

41.11. Cheat sheet

p weights, k trainable; noise standard deviation $s = \sigma C/B$ per coordinate on the averaged gradient; curvature eigenvalues h_j; step size η, T steps. Table 41.7 collects the results.

Table 41.7.: Every result of the chapter.

Result	Formula	Conditions
Noise length	$\sigma C\sqrt{k}/B$	k trainable coordinates
Noise cost of T steps	$\approx \frac{\eta s^2}{4} d_{\text{eff}}$, $d_{\text{eff}} = \sum_j (1-(1-\eta h_j)^{2T})$	Quadratic loss near the start, $\eta h_j \ll 1$
Effective dimension	$d_{\text{eff}} \le \min(k, 2\eta T \operatorname{tr} H)$	—
Linear probe	Private convex ERM in k dimensions, $L \le \sqrt{k}$ for bounded features	Features computed from public data
Probe error	Feature gap (fixed) + privacy cost $\tilde{\Theta}(\sqrt{k \ln(1/\delta)}/(n\varepsilon))$	Convex, Lipschitz
LoRA weights	$r(d_{\text{in}} + d_{\text{out}})$ per matrix	Rank r
LoRA gradients	$\nabla_U = \delta(V^\top a)^\top$, $\nabla_V = a(U^\top \delta)^\top$	Ghost norms apply
Projection	Same (ε, δ); break-even $(s\sqrt{m}/(\sqrt{f_m}\|\bar{m}\|))^2$; bias $1 - f_m$	Subspace from public data only
Guarantee	(ε, δ)-DP in D for every fixed D_{pub}; nothing for D_{pub}	—

Rules of thumb

- Start from a model pretrained on public data that resembles the private task. On the running example this is worth about a factor of eight in ε.
- At very small budgets, train the last layer: few weights, a convex problem, a known ceiling. Measure that ceiling non-privately on public data to know what is at stake.
- At moderate budgets, train a low-rank adapter or all weights; the adapter is the robust middle choice. Tune the step size, smaller than from scratch, on public data.
- Count the trainable weights, but think in effective dimension: noise in directions the loss does not use is cheap; noise in directions it does use is what you pay for.
- The guarantee covers the private data given the public model. Check the public data for overlap with the private data, and say what they are when you report ε.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

42. User-level differential privacy

42.1. Why this matters

Every differential privacy (DP) guarantee in this part so far protects one record. Stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, clips each record's gradient, adds noise scaled to that clip, and reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$ for the part's running example: an adversary who sees the trained model learns little about whether any single training example was used. But the people behind the data do not contribute one record each. A user of a messaging app contributes hundreds of messages, a patient dozens of visits, a photographer thousands of photos. Each of those records is protected by DP, and the person is not, because what the adversary learns about all of a person's records together can be much larger than what they learn about one.

This chapter asks two questions. What does a record-level ε say about a person who contributed k records? And how do you train a model with a guarantee for the person instead?

42.1.1. The one idea: the neighbouring relation is the privacy unit

A DP guarantee compares the outputs on two **neighbouring** datasets, and whatever differs between neighbours is what is protected. If neighbours differ in one record, records are protected; if they differ in all of one person's records, people are. Nothing else in the definition changes. The work is to bound how much the *unit*, now a person, can move the computation, and to account for that bound.

There are two ways to get a guarantee for a person. The first keeps the record as the unit and pays for the person afterwards, by group privacy or by an accountant that knows the person has k records. It is cheap to implement and expensive in privacy, because the noise was calibrated to one record while the person moves the output by up to k. The second makes the person the unit from the start: sample people instead of records, bound each person's total contribution by clipping it, and run the usual accountant with people in place of records. On the running example the first route gives a useless model at any reasonable budget, and the second gives one within a few points of the non-private model.

Table 42.1 lists the results and where each is used.

Table 42.1.: The results of this chapter and where each is used.

Result	Where it is used
The privacy unit and group privacy	Reading a record-level guarantee as a statement about people

Result	Where it is used
The accountant for a user with k records under example-level sampling	Auditing what an existing DP-SGD model guarantees to heavy users
User-level DP-SGD	Training with a guarantee per person; federated learning
Contribution bounding and its bias–variance trade-off	Keeping an example-level pipeline while protecting people
Heterogeneous users	Deciding how to weight heavy and light users
The strategies compared on the running example	Choosing among the three in practice

Notation. The training set has n records belonging to U users; user u owns n_u records. Two datasets are **example-level neighbours** if they differ by adding or removing one record, and **user-level neighbours** if they differ by adding or removing all the records of one user. k is the number of records a user contributes, or the cap on it. In a batch drawn by Poisson sampling at rate q, M is the number of a given user's records that are drawn. As in DP-SGD, C is the clipping norm, σ the noise multiplier, T the number of steps and B the expected batch size, now counted in whichever unit is sampled.

The running example. The part's network (one hidden layer of 64 tanh units, 3,329 weights) learns labels produced by a fixed teacher network from 10,000 training records of 50 features, and is tested on 20,000 more. This chapter assigns the 10,000 records to $U = 1{,}000$ users. Each user gets one record, and the other 9,000 are spread over users with probabilities proportional to log-normal weights, so user sizes are very uneven: the mean is 10 records, the median 4, and the largest user holds 348 (Figure 42.1). A user's records are a nearest-neighbour cluster around a random record, so they resemble each other the way one person's messages do: the label balance $|\bar{y}|$ is 0.21 on average in the ten largest users, against 0.06 in the whole set. The population is fixed, like the dataset.

How to read this chapter. The toolkit restates three facts from earlier parts. Section 42.3 and Section 42.4 say what a record-level guarantee gives a person; the second has the chapter's main derivation, the accountant for a user with k records. Section 42.5 and Section 42.6 are the two ways to train with a user-level guarantee, Section 42.7 asks what they do to heavy users, and Section 42.8 compares everything on the running example.

42.2. Toolkit: three facts every derivation uses

42.2.1. Fact 1 — Group privacy

If a mechanism is (ε, δ)-DP for neighbours that differ in one record, then for two datasets that differ in k records it satisfies, for every event S,

$$\Pr[M(D) \in S] \le e^{k\varepsilon} \Pr[M(D'') \in S] + \delta_k, \qquad \delta_k = \delta \sum_{j=0}^{k-1} e^{j\varepsilon} = \delta \, \frac{e^{k\varepsilon} - 1}{e^{\varepsilon} - 1}. \tag{42.1}$$

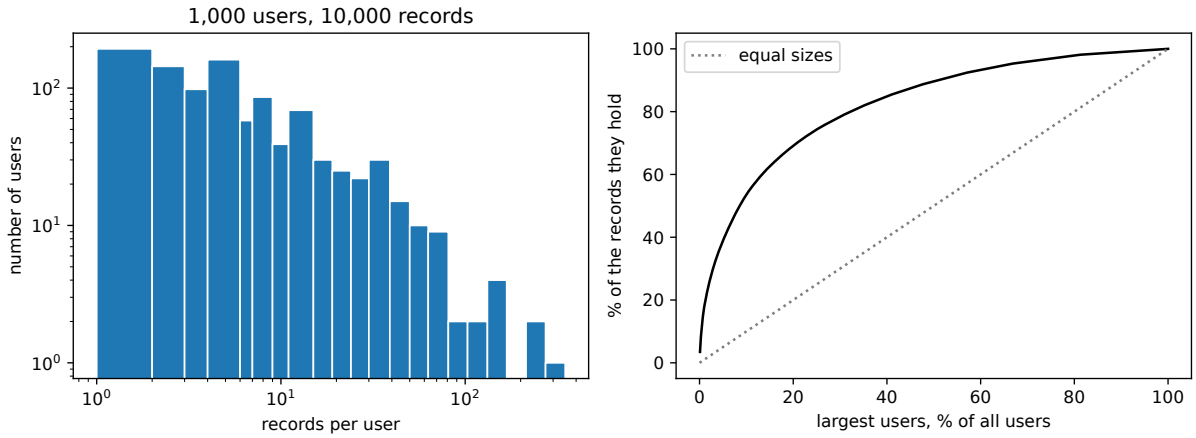


Figure 42.1.: Left: the number of records per user in the running example's population of 1,000 users, on log–log axes; 193 users have one record and the largest has 348. Right: the share of all records held by the largest users. The largest 10 users hold 17.9% of the records and the smallest half of the users 10.3%.

The reason is a chain: D and D'' are joined by a path of k steps, each between neighbouring datasets. Applying the one-record guarantee once per link multiplies the probability bound by e^{ε} each time, and each link's slack δ is then multiplied by the e^{ε} of every later link, which is where the geometric sum comes from. The bound is attained by some mechanisms, so no better one holds for every (ε, δ)-DP mechanism (Post-processing, group privacy and composition). For Rényi differential privacy (RDP) the chain is weaker still, because the Rényi divergence has no triangle inequality: an (α, ε)-RDP mechanism is $(\alpha/2^c, 3^c\varepsilon)$-RDP for groups of 2^c records, when $\alpha \geq 2^{c+1}$ (Mironov 2017; Rényi DP). Each doubling of the group halves the order and triples ε.

42.2.2. Fact 2 — The accountant of DP-SGD

One step of DP-SGD adds Gaussian noise of standard deviation σC to a sum of clipped per-example gradients computed on a Poisson batch: each record enters independently with probability q. In units of C, the worst case for one record is the pair $P_0 = \mathcal{N}(0, \sigma^2)$, the output without the record, and the mixture $(1-q)P_0 + q\mathcal{N}(1, \sigma^2)$, the output with it. Its RDP curve at order α, the Rényi divergence of order α between the two, is $\varepsilon_{q,\sigma}(\alpha) = \frac{1}{\alpha - 1} \ln A_\alpha$, and for integer α

$$A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\left(\frac{j^2 - j}{2\sigma^2}\right). \tag{42.2}$$

The sum comes from expanding $\big((1-q) + q\,r\big)^{\alpha}$, with r the Gaussian likelihood ratio, by the binomial theorem and averaging each power r^j under P_0 (Amplification by subsampling). RDP curves add over the T steps, even when each step depends on earlier outputs, and the total converts to (ε, δ) at the best order:

$$\varepsilon = \min_{\alpha} \left[T\varepsilon_{q,\sigma}(\alpha) + \ln \frac{\alpha - 1}{\alpha} - \frac{\ln \delta + \ln \alpha}{\alpha - 1} \right]. \tag{42.3}$$

The terms $j = 0$ and 1 carry no privacy cost, since their exponential factor is 1. For small q and moderate orders only the $j = 2$ term matters beyond them, so $A_\alpha \approx 1 + \binom{\alpha}{2} q^2 (e^{1/\sigma^2} - 1)$ and $\varepsilon_{q,\sigma}(\alpha) \approx \frac{\alpha}{2} q^2 (e^{1/\sigma^2} - 1)$.

42.2.3. Fact 3 — The Gaussian moment generating function

If $Y \sim \mathcal{N}(0, \sigma^2)$, then $\mathbb{E}[e^{sY}] = e^{s^2\sigma^2/2}$ for every real s (Distributions). Every Rényi divergence between Gaussians with the same variance reduces to it, because their likelihood ratio is the exponential of a linear function of the output: for $P_m = \mathcal{N}(m, \sigma^2)$,

$$\frac{P_m(y)}{P_0(y)} = \exp\Big(\frac{-(y-m)^2 + y^2}{2\sigma^2}\Big) = \exp\Big(\frac{2my - m^2}{2\sigma^2}\Big). \tag{42.4}$$

42.3. The privacy unit

42.3.1. Context

A model trained with record-level DP-SGD at $\varepsilon = 6.71$ is released. One of the people in the training set contributed k of its records. What, exactly, does the guarantee promise that person? The question has to be answered before a deployment can say "your data are protected", because the promise is about whatever the neighbouring relation changes, and that relation was chosen by the engineer, not by the person.

42.3.2. Statement

A mechanism is **user-level** (ε, δ)-DP if for every pair of datasets D, D' that differ by adding or removing all the records of one user, and every event S,

$$\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta. \tag{42.5}$$

It is **example-level** DP if the same holds for datasets that differ in one record. A mechanism that is example-level (ε, δ)-DP is user-level $(k\varepsilon, \delta_k)$-DP for users with at most k records, with δ_k from Equation 42.1. Without more information about the mechanism, nothing better can be said.

42.3.3. Derivation

1. **User-level neighbours are k example-level steps apart.** If D' is D plus the k records of one user, add them one at a time. The datasets $D, D \cup \{x_1\}, D \cup \{x_1, x_2\}, \dots, D'$ form a chain in which each is an example-level neighbour of the next.
2. **Apply group privacy.** Fact 1 along this chain gives Equation 42.1 with $D'' = D'$, in both directions, since the chain can be walked either way.
3. **Nothing better holds in general.** The bound is attained by a mechanism built for the purpose, which outputs an alarm with probabilities chosen so that every link's inequality is an equality (Post-processing, group privacy and composition). A guarantee that is only known as "(ε, δ)-DP" could come from that mechanism.

42.3.4. Interpretation

The definition is the same; the unit is not. User-level DP is not a new privacy notion. It is DP with a coarser neighbouring relation, so every property proved for DP holds for it: post-processing, composition and the accountants of Part 4. The only thing that changes is the sensitivity: how far one unit can move the computation. Records, users, households, devices and organizations are all possible units, and the choice should follow the question "whose participation do I want to hide?" A deployment that says "DP" without naming the unit leaves that question open, and Choosing an accountant lists the unit among the things a guarantee must report.

Group privacy is linear in ε and exponential in δ. For pure DP a user with k records gets $k\varepsilon$, already weak: at $\varepsilon = 1$ per record, a user with 10 records gets $\varepsilon = 10$, a likelihood ratio of $e^{10} \approx 22{,}000$. With $\delta > 0$ it is worse. In Equation 42.1 each δ is multiplied by up to $e^{(k-1)\varepsilon}$, so to keep the user's δ_k at 10^{-5} the per-record δ must be tiny, and a smaller δ means a larger per-record ε. For DP-SGD this trade can be made at any δ, because the accountant's RDP curve converts to (ε, δ) at every δ: choose the per-record δ so that $\delta_k = 10^{-5}$ exactly.

For DP-SGD it can fail altogether. The per-record ε grows as the per-record δ shrinks, so $\delta_k = \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1}$ is a race between a falling δ and a rising $e^{(k-1)\varepsilon}$. For small k the falling δ wins. For larger k the rising ε wins at every δ, and no choice brings δ_k down to 10^{-5}. Group privacy then gives no $(\varepsilon, 10^{-5})$ guarantee at all (Exercise 2 in the exercise book traces the race).

42.3.5. Example: the running example's users

The running example's model is example-level $(6.71, 10^{-5})$-DP. Read naively as a user-level statement, a user with two records gets $\varepsilon = 2 \times 6.71 = 13.42$ with $\delta_2 = 10^{-5}(1 + e^{6.71}) = 0.0082$: a probability of almost one in a hundred that the guarantee fails outright, which is no guarantee. Re-converting the accountant's RDP curve at a smaller per-record δ does better. The per-record δ that makes $\delta_2 = 10^{-5}$ gives a per-record ε of 9.30, so the user gets $(18.61, 10^{-5})$. For five records the same procedure gives a per-record ε of 23.62 and a user-level ε of 118.12. For ten records it gives nothing: no per-record δ makes δ_{10} as small as 10^{-5}. In the running example's population, 244 of the 1,000 users have at least ten records (the median is 4), and 30 have at least 50.

42.3.6. Check yourself

- A dataset has one row per hospital visit, and a mechanism is 0.2-DP (pure) at the visit level. What does a patient with 7 visits get? *(1.4-DP, by group privacy with $k = 7$; for pure DP the bound is just $k\varepsilon$.)*
- Why does a user-level guarantee still compose and survive post-processing? *(Because it is DP with a different neighbouring relation, and those proofs hold for any fixed relation: they never use what a neighbour is.)*

42.4. A user with k records in example-level DP-SGD

42.4.1. Context

Group privacy treats DP-SGD as a black box. It is not one: we know exactly how a user with k records enters each step. Each of the user's records is drawn into the batch independently with probability q, so in each step a random number of them, between 0 and k, contributes to the noisy sum. An accountant that knows this can compute the user's guarantee directly, as Amplification by subsampling did for one record. How large is that guarantee, and how does it grow with k?

42.4.2. Statement

Let $M \sim \text{Binomial}(k, q)$ be the number of the user's records in a Poisson batch, and $b_m = \binom{k}{m} q^m (1-q)^{k-m}$ its probabilities. In units of C, one step of example-level DP-SGD compares

$$P_0 = \mathcal{N}(0, \sigma^2) \qquad \text{and} \qquad P_{(k)} = \sum_{m=0}^{k} b_m \, \mathcal{N}(m, \sigma^2), \tag{42.6}$$

the output without the user and with the user, whose drawn records all push the sum by the full C in the same direction. Charles et al. (2024, Theorem 3) prove that this pair, composed over the T steps, bounds the user's privacy loss, and that the bound is attained by a loss for which the user's gradients are aligned. The user's RDP curve is therefore $\varepsilon_k(\alpha) = \max\{D_\alpha(P_{(k)} \| P_0), D_\alpha(P_0 \| P_{(k)})\}$. The first divergence is $\frac{1}{\alpha-1} \ln A_\alpha$ with

$$A_\alpha = \mathbb{E}\Big[\exp\Big(\frac{1}{\sigma^2} \sum_{1 \le i < j \le \alpha} M_i M_j \Big) \Big], \qquad M_1, \dots, M_\alpha \text{ independent copies of } M, \tag{42.7}$$

for integer α. For small kq the curve is close to that of a single record sampled at rate kq:

$$\varepsilon_k(\alpha) \approx \frac{\alpha}{2} (kq)^2 (e^{1/\sigma^2} - 1). \tag{42.8}$$

The user's (ε, δ) after T steps is Equation 42.3 applied to $T\varepsilon_k(\alpha)$.

42.4.3. Derivation

The derivation follows the one for a single record: find the distribution of one step's output with and without the user, write the Rényi divergence as an average of a power of the likelihood ratio, and evaluate the average. The new ingredient is that the power of a mixture over m becomes a product of independent copies of M, which makes the average computable by Fact 3.

1. **How many of the user's records are drawn.** Poisson sampling draws each record independently with probability q, so the number of the user's k records in the batch is a sum of k independent coins: $M \sim \text{Binomial}(k, q)$, independent of which other records are drawn.

2. **What they do to the sum.** Fix the other records' contribution, which is the same with and without the user. Given $M = m$, the user adds m clipped gradients, each of length at most C, so their sum has length at most mC, with equality when they all point the same way. In units of C and along that direction, the output is $\mathcal{N}(m, \sigma^2)$ with probability b_m: the mixture of Equation 42.6. That the aligned case is the worst over all configurations of the user's gradients, and that the reduction to one dimension is valid, is the content of Charles et al.'s Theorem 3; the intuition is that gradients pointing in different directions add in squares rather than in length, and spread the user's influence over several coordinates where it is harder to detect.
3. **The divergence as an average.** By the definition of the Rényi divergence, $e^{(\alpha-1)D_\alpha(P_{(k)}\|P_0)} = \mathbb{E}_{y\sim P_0}\big[(P_{(k)}(y)/P_0(y))^\alpha\big]$. Dividing the mixture by P_0 term by term with Equation 42.4,

$$\frac{P_{(k)}(y)}{P_0(y)} = \sum_m b_m \exp\Big(\frac{2my - m^2}{2\sigma^2}\Big) = \mathbb{E}_M\Big[\exp\Big(\frac{2My - M^2}{2\sigma^2}\Big)\Big], \tag{42.9}$$

   an average over the number of drawn records.
4. **The power of an average is an average over independent copies.** For integer α, $(\mathbb{E}_M[f(M)])^\alpha = \mathbb{E}[f(M_1)f(M_2)\cdots f(M_\alpha)]$ with $M_1, \dots, M_\alpha$ independent copies of M: multiplying out α copies of the sum $\sum_m b_m f(m)$ gives one term for every choice $(m_1, \dots, m_\alpha)$, with weight $b_{m_1}\cdots b_{m_\alpha}$, which is the joint distribution of independent copies. The product of exponentials is the exponential of the sum, so with $S = M_1 + \cdots + M_\alpha$,

$$\Big(\frac{P_{(k)}(y)}{P_0(y)}\Big)^\alpha = \mathbb{E}\Big[\exp\Big(\frac{2Sy - \sum_i M_i^2}{2\sigma^2}\Big)\Big]. \tag{42.10}$$

5. **Average over y.** Exchange the two averages, which is allowed because everything is positive. For fixed copies, Fact 3 with $s = S/\sigma^2$ gives $\mathbb{E}_{y\sim P_0}[e^{Sy/\sigma^2}] = e^{S^2/(2\sigma^2)}$, so

$$A_\alpha = \mathbb{E}\Big[\exp\Big(\frac{S^2 - \sum_i M_i^2}{2\sigma^2}\Big)\Big]. \tag{42.11}$$

   Expanding the square, $S^2 = \sum_i M_i^2 + 2\sum_{i<j} M_i M_j$, and the squares cancel: this is Equation 42.7. Only products of *different* copies remain.
6. **Check against one record.** For $k = 1$, each M_i is a coin, 0 or 1. If J of the α coins are 1, the sum $\sum_{i<j} M_i M_j$ counts the pairs among them, $\binom{J}{2} = (J^2 - J)/2$, and $J \sim \text{Binomial}(\alpha, q)$. The average of $e^{(J^2-J)/(2\sigma^2)}$ over J is Equation 42.2.
7. **Small kq.** When kq is small, most copies are 0. The terms of Equation 42.7 with at most one non-zero copy have $\sum_{i<j} M_i M_j = 0$ and contribute their probability times 1. The first term that costs privacy has exactly two copies equal to 1, the most likely way for a product to be non-zero: there are $\binom{\alpha}{2}$ pairs of copies, each copy is 1 with probability about kq, and the factor is e^{1/σ^2}. So $A_\alpha \approx 1 + \binom{\alpha}{2}(kq)^2(e^{1/\sigma^2} - 1)$, the same as Equation 42.2's first three terms with q replaced by kq. Taking $\ln(1+x) \approx x$ and dividing by $\alpha - 1$ gives Equation 42.8.
8. **Other orders and the other direction.** For non-integer α, and for $D_\alpha(P_0\|P_{(k)})$, there is no finite expansion. Compute both as one-dimensional integrals of $P_0(y)$ times a power of the mixture ratio of step 3, on a grid of step $\sigma/10$ wide enough to contain the bump near $y = \alpha k$ that large orders create, with log-sum-exp to avoid overflow, exactly as for one record.

42.4.4. Interpretation

A user with k records looks like one record sampled k times as often. Equation 42.8 says that, in the regime where amplification works well, the only thing that matters is the probability that some record of the user is in the batch, about kq. Halving the sampling rate exactly compensates doubling the user's records. Since the per-step RDP grows like $(kq)^2$, and ε after conversion grows roughly like the square root of the RDP slope, the user's ε grows roughly linearly in k while kq is small.

Beyond that regime it grows faster. Two things break the approximation. First, kq is no longer small: for $k = 50$ and $q = 0.01$, half of all steps include one of the user's records, and subsampling, which helps only because the unit is usually absent, stops helping. Second, the higher terms of Equation 42.7 grow like $e^{M_i M_j/\sigma^2}$, and $M_i M_j$ can be as large as k^2. A step that draws all k records shifts the sum by kC, a mechanism with noise multiplier σ/k, whose privacy loss has a tail that no amount of rarity can hide at high orders. The accountant then uses orders close to 1, where the conversion is expensive.

Group privacy is the wrong tool for DP-SGD. It ignores both the structure of the mechanism and the fact that the RDP curve is known. The direct accountant gives a smaller ε for every k and a finite one for every k.

The accountant is a worst case over gradients, not over users. It assumes the user's drawn records push the sum in the same direction by the full C. Real users' gradients are partly aligned, which is precisely why their records are correlated, and the theorem says some loss attains the bound. An auditor should therefore treat it as the guarantee, not as a pessimistic estimate.

42.4.5. Example: the running example's guarantee for its users

Table 42.2 applies the accountant to the running example's DP-SGD run, $q = 0.01$, $\sigma = 1$, $T = 10{,}000$, $\delta = 10^{-5}$.

Table 42.2.: The user-level ε at $\delta = 10^{-5}$ of the running example's example-level DP-SGD, for a user with k records. "None" means that group privacy gives no $(\varepsilon, 10^{-5})$ guarantee.

Records of the user k	1	2	5	10	50
User accountant, Equation 42.6	6.72	15.54	52.36	146.20	2161.43
Best order α	4	2.5	1.65	1.3	1.05
Group privacy, Equation 42.1	6.71	18.61	118.12	none	none
One record at rate kq	6.71	15.32	50.11	134.50	1673.08

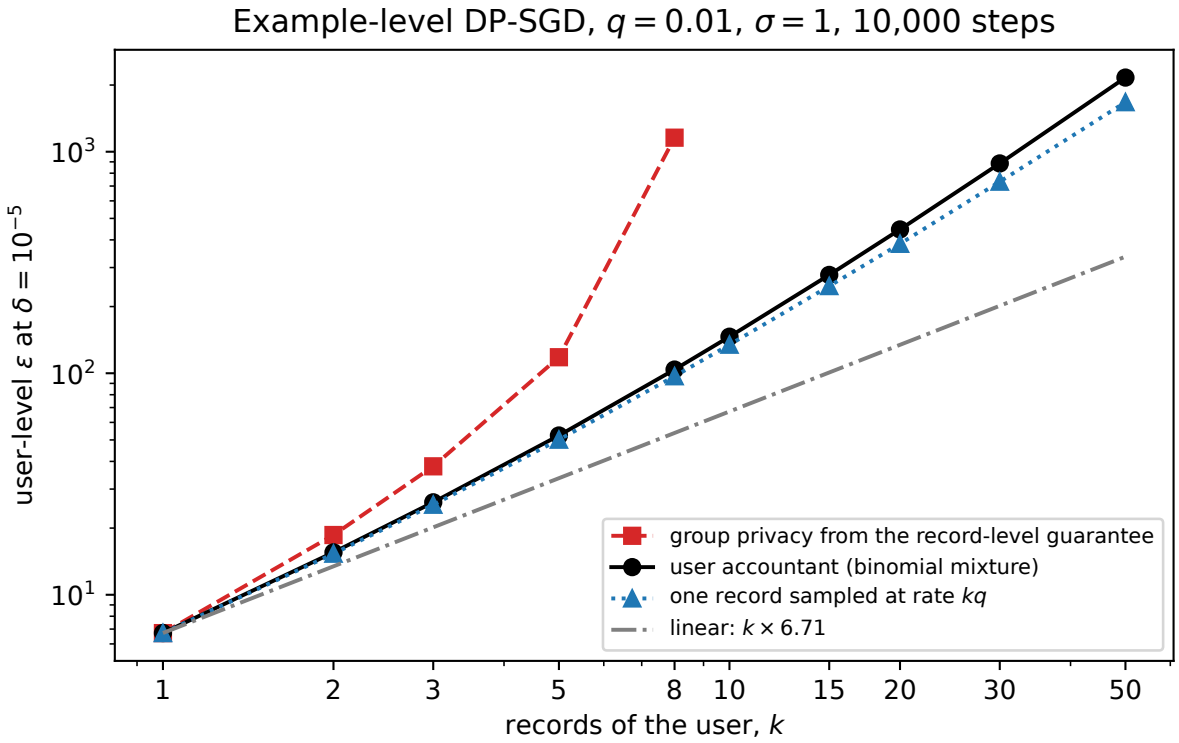


Figure 42.2.: The user-level ε at $\delta = 10^{-5}$ of the running example's DP-SGD ($q = 0.01$, $\sigma = 1$, 10,000 steps; $\varepsilon = 6.71$ per record) for a user with k records. The direct accountant of Equation 42.6 (black) is close to a single record sampled at rate kq (blue, dotted) and grows faster than linearly (grey). Group privacy from the record-level guarantee (red, dashed) is larger, and gives no guarantee at all for $k = 10$ and above.

The first column checks the accountant: with one record it is the DP-SGD accountant, 6.72 against 6.71, the difference coming from the coarser grid of orders this chapter's script uses (its best order is 4, against 4.1 on the finer grid). At $\alpha = 4.1$ the two curves agree to all printed digits, 3.733633×10^{-4}. For $k = 2$ and 5 at $\alpha = 2$, the numerical integral and the exact formula Equation 42.7, $A_2 = \mathbb{E}[e^{M_1 M_2/\sigma^2}]$, agree to seven digits (6.992273×10^{-4} and 4.626224×10^{-3}), and the reverse direction is smaller (6.119238×10^{-4} and 3.412123×10^{-3}).

Already at two records, the person behind them is protected at $\varepsilon = 15.54$, more than twice the record's 6.71. At ten, $\varepsilon = 146.20$ means no protection at all. The rate-kq approximation is within 1.5% at $k = 2$ (15.32 against 15.54) and 8% at $k = 10$ (134.50 against 146.20), so it is a quick estimate of the user's exposure: rerun the record-level accountant with q replaced by kq. In every row it errs on the low side, because it drops the terms in which several of the user's records are drawn together.

42.4.6. Check yourself

- Your model was trained with example-level DP-SGD at $q = 0.001$, and its heaviest user has 20 records. Which single-record accountant run gives a quick estimate of that user's ε, and in which direction is it off? *(The run with $q = 0.02$ and the same σ and T. It underestimates slightly, because it ignores steps that draw several of the user's records at once.)*
- In Equation 42.7, why do only products of different copies appear? *(Because the $\sum_i M_i^2$ in the exponent of the likelihood ratio cancels the squares in S^2 after the Gaussian average. Each copy alone averages to a ratio whose expectation under P_0 is 1, as for one record.)*

42.5. User-level DP-SGD

42.5.1. Context

Accounting for users after the fact charges a heavy user for every one of their records. The alternative changes the algorithm, so that the unit DP-SGD samples, clips and accounts for is the user. The idea is due to McMahan, Ramage, Talwar & Zhang (2018), who used it to train a language model with federated learning, where each user's device computes an update on its own data and the server only sees the updates. What does the algorithm compute, and what does it guarantee?

42.5.2. Statement

Start from weights w_0 chosen independently of the data. For $t = 0, \dots, T-1$:

1. **Sample users.** Put each user in the set S_t independently with probability q.
2. **Per-user update.** For each $u \in S_t$, compute one vector from user u's records alone, for example the average gradient $\bar{g}_u = \frac{1}{n_u} \sum_{i \in u} \nabla_w \ell(w_t, z_i)$, or the change in weights after a few local steps of SGD on those records.
3. **Clip per user.** $\tilde{g}_u = \bar{g}_u \min(1, C/\|\bar{g}_u\|)$.
4. **Noisy sum.** $G_t = \sum_{u \in S_t} \tilde{g}_u + \xi_t$ with $\xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I)$.

5. **Step.** $w_{t+1} = w_t - \eta\, G_t/(qU)$.

Releasing $w_1, \dots, w_T$ is user-level (ε, δ)-DP with ε given by Equation 42.3 for the curve $T\varepsilon_{q,\sigma}(\alpha)$ of Fact 2: exactly the accountant of example-level DP-SGD, with users in place of records and q the user sampling rate.

A common variant replaces steps 2 and 3 by **per-record clipping and a cap**: keep at most k records per user, clip each record's gradient to C, and sum them. One user then moves the sum by at most kC, and the guarantee is that of noise multiplier σ/k:

$$A_\alpha = \sum_{j=0}^{\alpha} \binom{\alpha}{j} (1-q)^{\alpha-j} q^j \exp\Big(\frac{k^2(j^2-j)}{2\sigma^2}\Big), \tag{42.12}$$

Equation 42.2 with σ replaced by σ/k.

42.5.3. Derivation

The proof of DP-SGD used four properties of the clipped sum. Each holds with users in place of records.

1. **One user moves the clipped sum by at most C.** Fix the weights w_t and the other sampled users. Each $\tilde{g}_v$ is computed from user v's records alone, so adding user u to the dataset leaves every other term unchanged and adds $\tilde{g}_u$, of length at most C by step 3. However many records u has, they enter only through $\tilde{g}_u$.
2. **One step is a subsampled Gaussian mechanism over users.** User u enters S_t with probability q, independently of the other users. So the step's output with u is a mixture: with probability $1-q$ the same as without u, with probability q shifted by $\tilde{g}_u$. After rescaling by C and rotating the shift onto one axis, the worst case is the pair of Fact 2, and the step's RDP curve is $\varepsilon_{q,\sigma}(\alpha)$.
3. **Dividing by qU is post-processing**, since the number of users U is treated as public, as n is in example-level DP-SGD.
4. **Compose and convert.** RDP composition over the T adaptive steps and Equation 42.3 finish the proof, as before.
5. **The capped-sum variant.** With per-record clipping, user u's contribution is a sum of at most k vectors of length at most C, so its length is at most kC, and the user is still sampled as a whole with probability q. In units of C the pair is $\mathcal{N}(0, \sigma^2)$ against $(1-q)\mathcal{N}(0, \sigma^2) + q\mathcal{N}(k, \sigma^2)$. Dividing the output by k, an invertible map that changes no divergence, gives the pair with shift 1 and noise σ/k, whose curve is Equation 42.2 with σ/k: that is Equation 42.12. The script checks it at $k = 2$, $\alpha = 3$: both give 8.219438×10^{-2}.

42.5.4. Interpretation

The count that matters is the number of units. User-level DP-SGD has the privacy of example-level DP-SGD on a dataset of U records. To keep the same expected batch of B units, the sampling rate must be B/U, larger than B/n by the average number of records per user, and amplification is correspondingly weaker. On the running example, with 1,000 steps and an expected batch of 100 units, a record-level $\varepsilon = 4$ needs $\sigma = 0.778$ at $q = 0.01$, and a user-level

$\varepsilon = 4$ needs $\sigma = 3.78$ at $q = 0.1$. The noise on the averaged update, $\sigma C/B$, is 0.0189 against 0.00389, about five times larger. This is the honest price of protecting people: there are ten times fewer of them than records, and DP's cost per unit is fixed.

Records inside a user are free. The guarantee does not depend on n_u. A user's records can be averaged, used for several local steps, or subsampled, all without changing ε, as long as the result is one clipped vector computed from that user's data alone. More records per user make each $\bar{g}_u$ a better estimate of the user's gradient, so the signal improves while the noise stays fixed. Levy et al. (2021) show that, when each user's records come from the same distribution, user-level private learning gains from having more records per user.

Average, or sum then clip? Averaging before clipping gives every user a vector of the same kind regardless of size, which keeps the clipping norm meaningful across users. Clipping each record and summing at most k of them uses per-example gradient code that already exists, but pays a factor k in noise multiplier (Equation 42.12). Summing all of a user's records and then clipping the sum is also valid (sensitivity C), and gives heavy users more weight than averaging does, as Section 42.7 shows.

What the proof assumes. Step 1 needs every user's contribution to be computed from that user alone, which rules out cross-user normalization and requires that one user's records are not spread across several "users" in the data. If one person has several accounts, the unit is the account, and the person is again a group. Step 2 needs users to be Poisson-sampled; an implementation that iterates over users in a fixed shuffled order is a different mechanism, as for records.

42.5.5. Example: the noise each strategy needs

Table 42.3 fixes 1,000 steps with an expected batch of 100 sampled units and gives the noise multiplier for four user-level budgets. The column "record-level" is not a user-level guarantee; it shows the noise the same schedule would need to protect only records. Column (a) is example-level DP-SGD on all 10,000 records ($q = 0.01$), accounted for the largest user with $k = 348$ by Equation 42.6; (b) is user-level DP-SGD ($q = 0.1$); (c) keeps at most 2 records per user, 1,807 records in all, and runs example-level DP-SGD on them with $q = 100/1{,}807 = 0.0553$, accounted for $k = 2$ (Section 42.6).

Table 42.3.: Noise multipliers for 1,000 steps with an expected batch of 100 units, at $\delta = 10^{-5}$.

User-level ε	Record-level (not user-level)	(a) All records, $k = 348$	(b) User-level DP-SGD	(c) At most 2 records per user
1	1.52	449	12.9	14.4
2	1.03	237	6.89	7.62
4	0.778	128	3.78	4.18
8	0.616	70.3	2.18	2.4

Column (a) is the cost of keeping the record as the unit when one user has 348 records: in each step this user has on average $348 \times 0.01 = 3.48$ records in the batch, so subsampling hardly hides them, and the noise must cover a shift of several clipping norms in nearly every step. At $\varepsilon = 4$ the noise on the averaged gradient has standard deviation $128 \times 0.5/100 = 0.64$ per coordinate, so length about $0.64 \times \sqrt{3{,}329} = 36.9$, against a clipped average of length at most 0.5.

42.5.6. Check yourself

- In user-level DP-SGD, a user with 300 records runs five local steps of SGD on them and sends the clipped change in weights. Does that change ε? *(No. The user's contribution is still one vector computed from their records alone, clipped to C; the local computation is invisible to the accountant.)*
- Why does user-level DP-SGD on 1,000 users need more noise than example-level DP-SGD on 10,000 records, at the same expected batch, steps and ε? *(To keep the batch at 100 units it must sample at rate 0.1 instead of 0.01. Amplification by subsampling is weaker at a higher rate, so the same ε needs a larger σ.)*

42.6. Contribution bounding

42.6.1. Context

User-level DP-SGD needs a data pipeline that groups records by user and computes one vector per user. Many training pipelines are built around records: shuffled files of examples, per-example gradient code, fixed-length batches. Can you keep that pipeline and still protect users? You can, if you limit how many records each user contributes. The question is what limit to choose, because the limit trades the noise the accountant requires against the data you throw away.

42.6.2. Statement

Contribution bounding keeps at most k records of each user, chosen without looking at other users' data, for example at random, and discards the rest. Run example-level DP-SGD on the $n_k = \sum_u \min(n_u, k)$ remaining records with sampling rate q. The result is user-level (ε, δ)-DP with ε given by the accountant of Equation 42.6 for a user with k records. The trade-off:

- the **noise** grows with k: in the small-kq regime the user's RDP is that of a record sampled at rate kq (Equation 42.8), so the noise multiplier must grow roughly in proportion to k at a fixed q;
- the **bias** shrinks with k: the training set loses $n - n_k$ records, all of them from users with more than k records, so the data are both smaller and reweighted toward light users.

42.6.3. Derivation

1. **Bounding is applied per user.** Write the bounded dataset as $\Pi(D) = \bigcup_u \pi(D_u)$, where D_u is user u's records and π keeps at most k of them using randomness that is independent of everything else. Adding user u to D adds $\pi(D_u)$ to $\Pi(D)$ and changes nothing else, so user-level neighbours map to datasets that differ in at most k records, all of one user.
2. **Account for those records.** Example-level DP-SGD on $\Pi(D)$ sees the user's at most k records, each drawn independently with probability q. By Section 42.4, the pair for one step is Equation 42.6 with k. A user with fewer records is covered too: they are a k-record user some of whose clipped gradients happen to be 0, and the worst case of Section 42.4 is over all gradients of length at most C, including 0. Composition and conversion are as before.

3. **The noise grows with k.** At fixed q, Equation 42.8 depends on k and q through kq only. Keeping the user's RDP fixed while k doubles therefore requires either halving q, which halves the batch, or raising σ. In the regime where the per-step RDP is about $\frac{\alpha}{2}(kq)^2/\sigma^2$, holding it fixed means $\sigma \propto kq$.
4. **The bias.** Before bounding, the expected update of DP-SGD averages over all n records; after it, over n_k records in which each user has weight $\min(n_u, k)/n_k$ instead of n_u/n. A user with $n_u > k$ loses weight, and their records beyond k are not used at all.

42.6.4. Interpretation

Choosing k is a bias–variance trade-off. Amin, Kulesza, Muñoz Medina & Vassilvitskii (2019) studied exactly this choice for private empirical risk minimization, and showed that the best cap depends on the distribution of user sizes: a cap above most users' sizes buys little data at a large noise cost, and a cap below them throws away most of the data. Their framing is the same as that of choosing the clipping norm in Beyond global sensitivity: a smaller bound means less noise and more distortion of what is computed. Because the cap depends on the data, it should be chosen from public knowledge of the population or privately; choosing it by trying several caps and looking at test accuracy, as the example below does to show the trade-off, spends privacy that the accountant does not see (Hyperparameter tuning).

Which records to keep matters. Random selection keeps the proof simple. Chua et al. (2024) compare selection rules for fine-tuning language models under contribution bounding and report that keeping each user's longest records worked best on one of their datasets. Any rule that looks only at the user's own records is allowed, by step 1.

Bounding plus user-level DP-SGD. The two techniques combine: cap each user at k records to bound compute, then compute one clipped vector per user. With user sampling, the cap no longer costs noise, only data, and users can contribute their full records again by subsampling them inside each step.

42.6.5. Example: the cap on the running example

Table 42.4 fixes the user-level budget at $\varepsilon = 4$, $\delta = 10^{-5}$ and 1,000 steps with an expected batch of 100 records. For each cap, the sampling rate is $q = 100/n_k$, so kq is the probability scale at which a full user enters a batch.

Table 42.4.: Contribution bounding at user-level $\varepsilon = 4$: example-level DP-SGD on at most k records per user, accounted for k records, one run each.

Cap k	Records kept	Sampling rate q	kq	Noise multiplier σ	Test accuracy
1	1,000 (10.0%)	0.1	0.100	3.78	0.8917
2	1,807 (18.1%)	0.0553	0.111	4.18	0.8920
5	3,506 (35.1%)	0.0285	0.143	5.34	0.8790
10	5,063 (50.6%)	0.0198	0.198	7.34	0.8276
20	6,631 (66.3%)	0.0151	0.302	11.2	0.7562
50	8,437 (84.4%)	0.0119	0.593	21.8	0.6401

Raising the cap from 1 to 50 multiplies the data by 8.4 and the noise multiplier by 5.8, and accuracy falls from 0.8917 to 0.6401. On this task the noise dominates: a 1,000-record dataset, one record per user, already contains most of what the network can learn privately at this budget, and every extra record per user costs more in noise than it adds in signal. The caps 1 and 2 are tied. The cap $k = 1$ row is user-level DP-SGD on one record per user in disguise: with one record per user, sampling records at rate 0.1 is sampling users at rate 0.1, and the accountant is the same, $\sigma = 3.78$.

42.6.6. Check yourself

- Why must the cap be applied with randomness independent of other users' data? *(The derivation needs adding one user to change only that user's kept records. If, say, the cap kept the k records most similar to the rest of the dataset, adding a user could change which records other users keep, and neighbours would differ in more than k records.)*
- In Table 42.4, kq rises from 0.100 to 0.593 as the cap goes from 1 to 50. What does Equation 42.8 predict for the noise multiplier, and why is the actual one smaller? *(Proportionality to kq predicts about $3.78 \times 5.93 = 22.4$. The rule is only approximate: it holds where σ is large and kq small, and at $kq = 0.593$ neither the rate-kq approximation nor the large-σ law is accurate, so the accountant's 21.8 differs from it.)*

42.7. Heterogeneous users

42.7.1. Context

User sizes are rarely equal. In the running example's population, 30 users with at least 50 records each hold 30.6% of the records, while the smallest half of the users hold 10.3% (Figure 42.1). Example-level training weights each user by their number of records; user-level training does not. What objective does user-level DP-SGD optimize, and what does that mean for heavy users?

42.7.2. Statement

Averaged over the user sampling and the noise, the update direction of user-level DP-SGD is

$$\mathbb{E}\Big[\frac{G_t}{qU}\Big] = \frac{1}{U}\sum_{u=1}^{U} \min\Big(1, \frac{C}{\|\bar{g}_u\|}\Big)\bar{g}_u, \tag{42.13}$$

a sum in which each user has weight at most $1/U$ whatever their number of records. Example-level DP-SGD gives user u weight n_u/n in its expected update, $\frac{1}{n}\sum_u \sum_{i\in u} \min(1, C/\|g_i\|)g_i$.

42.7.3. Derivation

1. **Average over the sampling.** User u is in S_t with probability q, so $\mathbb{E}\sum_{u\in S_t} \tilde{g}_u = q\sum_u \tilde{g}_u$. The noise has mean zero.
2. **Divide by qU.** The q cancels, leaving $\frac{1}{U}\sum_u \tilde{g}_u$, which is Equation 42.13 by the definition of $\tilde{g}_u$.

3. **The weight of one user.** $\|\tilde{g}_u\| \leq C$ for every user, so each term has length at most C/U, and the share of any group of users in the update is at most their share of the users.
4. **Example-level.** The same computation for DP-SGD over records (DP-SGD) gives each record weight $1/n$, so user u's records together have weight n_u/n.

42.7.4. Interpretation

User-level training changes the objective, not only the noise. With averaged per-user gradients, the model minimizes (a clipped version of) the average over users of each user's average loss, $\frac{1}{U}\sum_u \frac{1}{n_u}\sum_{i\in u}\ell(w, z_i)$, rather than the average over records. Whether that is what you want depends on the application. For a keyboard model, weighting people equally keeps a few very active users from defining everyone's suggestions. For a model meant to fit the overall distribution of records, it under-weights heavy users' content, and the bias can matter when heavy users differ from light ones, for example by language, topic or device.

Clipping caps heavy users twice. A heavy user's average gradient is a good estimate, typically short when the model already fits their data, and is rarely clipped. But if the user's data are unusual, their gradient is long and is clipped to C, so their influence is capped at that of any other user. This is the user-level analogue of what clipping does to records with large gradients (Disparate impact).

Sum-then-clip sits in between. Summing a user's gradients before clipping makes a heavy user's vector up to n_u times longer than one record's, so heavy users are clipped first, and their influence rises with n_u until it reaches C. It interpolates between record weighting, for users whose sum stays below C, and user weighting, for those above it (Exercise 5 in the exercise book).

42.7.5. Example: heavy users in the running example

The 30 users with at least 50 records hold 30.6% of the records, and so 30.6% of the weight in the expected example-level update. Under user-level DP-SGD their weight is at most 30/1,000 = 3.0%. Does that hurt the model where heavy users' data are? The script splits the test set by the owner of each test point's nearest training record. Because a heavy user's records were chosen as the nearest neighbours of one seed in 50 dimensions, they spread widely, and 68.3% of test points have a heavy user's record as their nearest neighbour. At user-level $\varepsilon = 4$, user-level DP-SGD reaches 0.9361 on those points and 0.9319 on the others; the non-private model, 0.9607 and 0.9622. On this task, down-weighting heavy users costs nothing measurable, because one teacher network labels every record, so the light users' records teach the same function. On data where heavy users' records follow their own patterns, the same reweighting would cost accuracy on those patterns.

42.7.6. Check yourself

- A user-level model will be used to predict for each record equally, and you know that heavy users' records differ from light users'. What can you change without changing ε? *(How each user's vector is computed: summing instead of averaging before clipping, or a larger C relative to typical averaged gradients. Both are inside the per-user computation and leave the sensitivity at C.)*

- Why is each user's share of the user-level update at most $1/U$ rather than exactly $1/U$? *(Because a user's clipped vector has length at most C; users whose average gradient is shorter than C, typically those the model already fits, contribute less.)*

42.8. Comparing the strategies on the running example

42.8.1. Context

Three ways to train the running example with a user-level guarantee are now on the table: (a) example-level DP-SGD on all records, with the user accountant for the heaviest user; (b) user-level DP-SGD; (c) contribution bounding to two records per user, then example-level DP-SGD. At the same user-level (ε, δ), which model is best?

42.8.2. Statement

Table 42.5 trains each strategy for 1,000 steps with an expected batch of 100 units, $C = 0.5$ and $\eta = 0.5$, at the noise multipliers of Table 42.3, one run each.

Table 42.5.: Test accuracy at the same user-level budget, $\delta = 10^{-5}$, one run each.

User-level ε	(a) Example-level, all records	(b) User-level DP-SGD	(c) At most 2 records per user
1	0.5318	0.8065	0.7084
2	0.5582	0.9028	0.8374
4	0.5713	0.9347	0.8810
8	0.5478	0.9442	0.9333
Non-private, 1,000 steps	0.9612		

42.8.3. Derivation

Each strategy's guarantee was derived above: (a) and (c) by the accountant of Section 42.4 with $k = 348$ and $k = 2$, and (b) by Section 42.5. The cap $k = 2$ is the best cap at $\varepsilon = 4$ in Table 42.4, tied with $k = 1$; choosing it from test accuracy is a leak that a real deployment would have to pay for. Run-to-run variation is of the order of a point at these budgets: the cap $k = 1$ and user-level DP-SGD restricted to one random record per user are the same algorithm with the same σ, and two runs of it gave 0.8917 and 0.9070.

42.8.4. Interpretation

Keeping the record as the unit fails. Strategy (a) is at chance, between 0.53 and 0.57, at every budget up to $\varepsilon = 8$. The heaviest user dictates the noise for everyone: σ between 70.3 and 449 drowns every gradient. That is the practical meaning of Figure 42.2: an example-level pipeline on data with heavy users needs either an enormous ε or a cap.

Changing the unit wins. User-level DP-SGD is the best strategy at every budget, 0.9347 at $\varepsilon = 4$ against 0.9612 for the non-private model. It needs a little less noise than the cap of 2 (Table 42.3) and, unlike the cap, it uses every record, through the per-user average. Its advantage is largest at small budgets: 9.8 points at $\varepsilon = 1$, 1.1 points at $\varepsilon = 8$. Its cost is compute: each step processes all the records of about 100 users, about 1,000 records, ten times what the example-level strategies process.

Bounding is the fallback. Strategy (c) keeps the example-level pipeline and loses 5 to 10 points against (b) below $\varepsilon = 8$. It comes close at $\varepsilon = 8$, where the noise is small enough that the discarded data, not the noise, limits accuracy.

These findings agree with published comparisons on language models, which use different data, models and accountants. Chua et al. (2024) compare contribution bounding with group-privacy accounting ("Group Privacy") against user-level DP-SGD with per-user averaging ("User-wise DP-SGD") for fine-tuning, and report that user-wise DP-SGD generally does better, especially at small budgets. Charles et al. (2024), who call the two sampling schemes example-level and user-level sampling, report that user-level sampling usually does better when each user's data are varied and when strong privacy is required, and that example-level sampling sometimes wins. Note that Chua et al. account for the cap with the generic group privacy of Equation 42.1; the direct accountant of Section 42.4 would give them less noise.

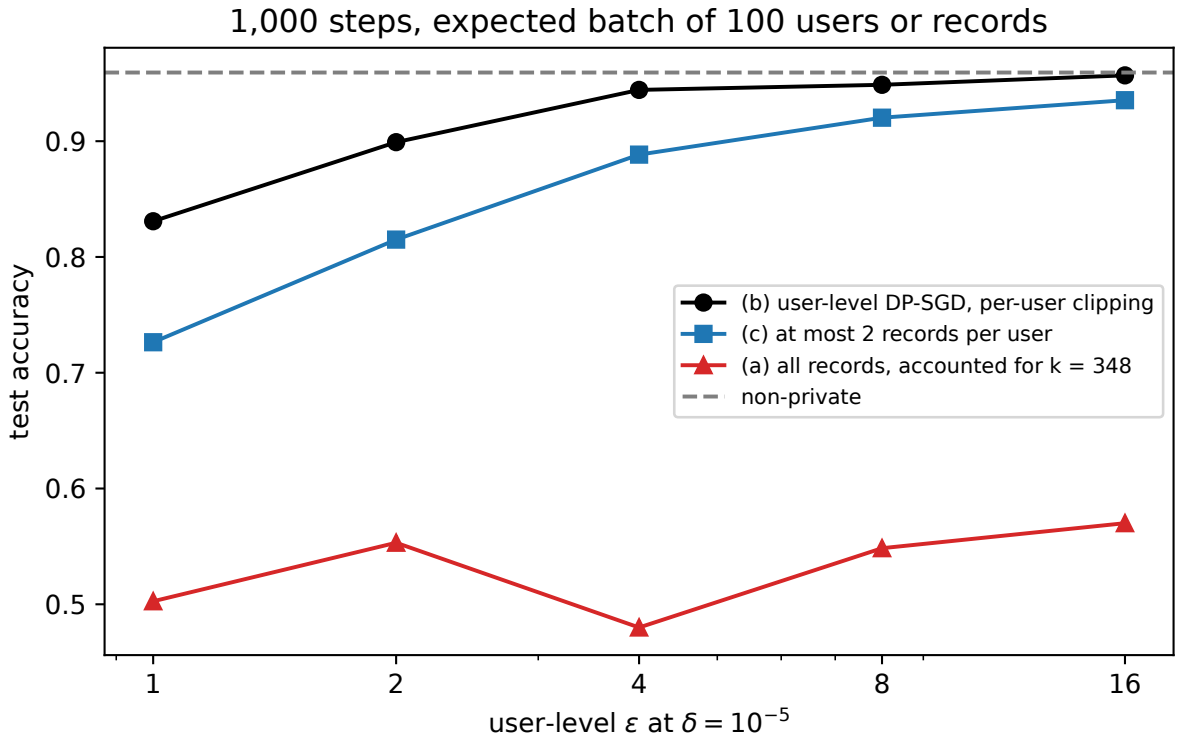


Figure 42.3.: Test accuracy of the running example against the user-level ε at $\delta = 10^{-5}$, for the three strategies (1,000 steps, expected batch 100, one run each; the runs differ from Table 42.5 in their random seeds). User-level DP-SGD is best at every budget; example-level DP-SGD on all records, accounted for its heaviest user, stays near chance.

42.8.5. Example: what the user-level guarantee costs

At $\varepsilon = 4$ the running example loses 2.7 points to protect users (0.9347 against 0.9612). The record-level model of DP-SGD at the same ε, trained for 10,000 steps, reached 0.9548, and this chapter's 1,000-step schedule would need only $\sigma = 0.778$ for a record-level guarantee (Table 42.3). The difference is the price of the unit: 1,000 people instead of 10,000 records, and a guarantee that now covers the person with 348 records as well as the one with a single record.

42.8.6. Check yourself

- Strategy (a) uses every record and the tightest available accountant. Why does it do worst? *(Because the guarantee must hold for the heaviest user, whose 348 records put about 3.5 of them into every batch; the noise has to hide a shift of several clipping norms in almost every step, which takes σ in the hundreds.)*
- Would strategy (a) look better if the heaviest user were removed from the data? *(Only a little: the accountant would be for the next-largest user, who has 258 records, so about 2.6 of them would still be in every batch. Removing a user because of their size is itself a data-dependent choice that must be made with public information or paid for, and it is a crude form of contribution bounding.)*

42.9. Code: check every number by computation

The script code/user_level.py builds the user population, implements the user accountant of Equation 42.6 by numerical integration and checks it against Equation 42.7 and Equation 42.2, compares it with group privacy, and trains the three strategies with the part's network. It imports the model, DP-SGD and the record-level accountant from code/dp_sgd.py. From the repository root:

```
uv run python code/part05-dp-ml/user_level.py
```

It takes about a minute and a half, most of it spent in user-level DP-SGD, which processes about 1,000 records per step, and in finding the noise multiplier for the user with 348 records.

Output (seed 0), summarized in Table 42.6:

Table 42.6.: The script's output with the default seed.

Experiment	Result
Population	1,000 users; mean 10, median 4, largest 348 records; 193 users with one record
Accountant checks	$k = 1$ matches the subsampled Gaussian; integral matches Equation 42.7 to seven digits
Running example, user with k records	$\varepsilon = 15.54$ at $k = 2$, 146.20 at $k = 10$; group privacy 18.61 and none

Experiment	Result
Noise at user-level $\varepsilon = 4$	$\sigma = 128$ (all records), 3.78 (user-level), 4.18 (cap 2)
Contribution bounding at $\varepsilon = 4$	0.8917 at $k = 1$ down to 0.6401 at $k = 50$
Strategies at $\varepsilon = 4$	0.5713 (all records), 0.9347 (user-level), 0.8810 (cap 2); non-private 0.9612
Heavy users	30.6% of records, at most 3.0% of the user-level update

tests/test_user_level.py checks that the output still matches.

What to take from it. The record-level accountant answers the wrong question when people contribute many records, and asking it the right question, about a user with k records, gives answers that grow faster than k. The fix is in the algorithm, not the accountant: make the person the unit that is sampled and clipped. Implementation needs only a per-user aggregation before clipping; the accountant is unchanged.

42.10. References

- McMahan, Ramage, Talwar & Zhang, "Learning Differentially Private Recurrent Language Models" (ICLR 2018). User-level DP-SGD in federated learning: sample users, clip each user's update, add Gaussian noise, account with users as the unit.
- Charles, Ganesh, McKenna, McMahan, Mitchell, Pillutla & Rush, "Fine-Tuning Large Language Models with User-Level Differential Privacy" (arXiv 2407.07737, 2024). The tight accountant for example-level sampling (their Theorem 3, the mixture of Equation 42.6), and an empirical comparison of example-level and user-level sampling for language models.
- Chua, Ghazi, Huang, Kamath, Kumar, Liu, Manurangsi, Sinha & Zhang, "Mind the Privacy Unit! User-Level Differential Privacy for Language Model Fine-Tuning" (Conference on Language Modeling 2024; arXiv 2406.14322). Contribution bounding with group privacy against user-wise DP-SGD, with record-selection rules and the choice of the cap.
- Amin, Kulesza, Muñoz Medina & Vassilvitskii, "Bounding User Contributions: A Bias-Variance Trade-off in Differential Privacy" (ICML 2019). The cap on records per user as a bias–variance trade-off, and how the best cap depends on the distribution of user sizes.
- Levy, Sun, Amin, Kale, Kulesza, Mohri & Suresh, "Learning with User-Level Privacy" (NeurIPS 2021). Theory of user-level private learning when each user holds several samples.
- Mironov, "Rényi Differential Privacy" (CSF 2017). Group privacy for RDP, Fact 1.
- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (Foundations and Trends in Theoretical Computer Science, 2014). Group privacy for pure and approximate DP.

42.11. Cheat sheet

U users, user u with n_u records, cap k, sampling rate q, noise multiplier σ, clipping norm C, T steps. Table 42.7 collects the results.

Table 42.7.: Every result of the chapter.

Result	Formula	Conditions
Group privacy	$(k\varepsilon, \delta\frac{e^{k\varepsilon}-1}{e^{\varepsilon}-1})$	Any example-level (ε, δ) mechanism
User with k records, example-level DP-SGD	$\mathcal{N}(0, \sigma^2)$ against $\sum_m \binom{k}{m} q^m (1-q)^{k-m} \mathcal{N}(m, \sigma^2)$	Tight (Charles et al. 2024, Theorem 3)
Its moments	$A_\alpha = \mathbb{E}[\exp(\sigma^{-2} \sum_{i<j} M_i M_j)]$, $M_i \sim \text{Binomial}(k, q)$	Integer α
Quick estimate	Record-level accountant at rate kq	Small kq; slightly optimistic
User-level DP-SGD	DP-SGD's accountant with users as units, q = user sampling rate	One clipped vector per user, from that user's data alone
Capped sum per user	Noise multiplier σ/k	Per-record clipping, at most k records, users sampled whole
Contribution bounding	Accountant for k records on $n_k = \sum_u \min(n_u, k)$ records	Cap chosen per user, independently of others
Expected user-level update	$\frac{1}{U} \sum_u \min(1, C/\|\bar{g}_u\|)\bar{g}_u$	Each user weighs at most $1/U$
Running example	Records: $\varepsilon = 6.71$; user with 2 records 15.54, with 10 records 146.20	$q = 0.01$, $\sigma = 1$, $T = 10{,}000$

Rules of thumb

- Name the privacy unit with every ε. A record-level ε says little about a person with many records.
- To estimate what an example-level model gives a user with k records, rerun the accountant at rate kq, and expect the truth to be a little worse.
- Don't use group privacy for DP-SGD; account for the user's records directly, or better, sample and clip users.
- Prefer user-level DP-SGD with per-user averaging. It pays for the number of people, not their records, and uses all their data.
- If the pipeline must stay record-based, cap records per user, choose the cap from public knowledge, and account for the cap with the user accountant.
- Decide deliberately how heavy users should count: averaging weights people equally, summing before clipping weights records until the clip.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

43. Disparate impact

43.1. Why this matters

The chapter on private gradient descent showed that training the part's running example with differential privacy (DP) at $\varepsilon = 4$ costs less than one point of test accuracy: 0.9548 against 0.9610 without privacy. That number is an average over the test set. It says nothing about who loses the point. Bagdasaryan, Poursaeed & Shmatikov (2019) asked that question and found an uncomfortable answer: when a network is trained with stochastic gradient descent (SGD) with per-example clipping and Gaussian noise, DP-SGD, the accuracy it loses is concentrated on the classes and subgroups that are under-represented in the training data. In their gender classifier, for example, the drop in accuracy was much larger on Black faces than on white faces, and the gap between groups that the non-private model already had grew under DP. They called this the **disparate impact** of DP on model accuracy.

This matters in two ways. A model that is "less than a point worse" can be ten points worse on a group of people, and an average hides it. And the groups that pay are often the ones a privacy guarantee is meant to protect: small populations, rare conditions, unusual records. This chapter explains why it happens, from the two ingredients of DP-SGD, measures it on the running example, and asks what can be done about it.

43.1.1. The one idea: every record gets the same budget of influence, and every group gets the same noise

DP-SGD caps each record's pull on the update at C, then adds noise whose size depends only on C and σ, not on the data. Both rules are blind to groups, and that is exactly why they are not neutral. A group's pull on the expected update is at most its share of the records times C, however badly the model fits it, so a group whose gradients are large is cut back more than a group whose gradients are small: that is the **clipping** mechanism. And the noise is the same for everyone, while a group's signal is proportional to its size, so a small group's signal-to-noise ratio is smaller by its share: that is the **noise** mechanism. Small groups and groups the model finds hard suffer from both, and the two compound, because clipping shrinks a group's effective share and the noise then acts on the shrunk share.

Table 43.1 lists the results and where each is used.

Table 43.1.: The results of this chapter and where each is used.

Result	Where it is used
Measuring disparate impact: privacy cost per group	Reporting a private model; choosing ε

Result	Where it is used
Clipping reweights groups and turns the update	Choosing C; group-aware clipping
The noise floor of a small group	Deciding whether a group is large enough to train on privately
Confidence intervals for per-group accuracy	Every per-group evaluation, private or not
Mitigations and what each costs	Training a private model that serves every group

Notation. As in DP-SGD, n records $z_i = (x_i, y_i)$ train a model with weights w, g_i is record i's gradient, C the clipping norm, σ the noise multiplier, q the sampling rate, $B = qn$ the expected batch size, T the number of steps and η the step size. The records are split into **groups** $g = 1, \dots, K$; group g has n_g records and **share** $s_g = n_g/n$. A record's **clipping weight** is $\min(1, C/\|g_i\|)$, the factor clipping multiplies its gradient by, and a group's clipping weight w_g is its average over the group's records. acc_g is a model's accuracy on test records of group g. A subscript i always names a record and a subscript g or h a group, so g_i is a record's gradient and $\bar{g}_g$, defined below, a group's average gradient.

The running example. The part's running example classifies 50-dimensional inputs with a two-layer network (DP-SGD). This chapter adds a minority group. Of the $n = 10{,}000$ training records, 9,500 are the first records of the running example's fixed training set: standard normal inputs, labelled by the random teacher network. The other 500, a share of 5%, come from a different region and follow a different rule. Their inputs are $x = 8u + z$, where u is a fixed unit vector and z is standard normal in a fixed 15-dimensional subspace orthogonal to u, and their label is the sign of one coordinate of z. So the minority's records lie far from the majority's, have larger inputs (average length 8.89 against 7.04), and need a rule that the majority does not teach. The non-private model learns both: it reaches 0.9529 on the majority and 0.9411 on the minority. Each group is tested on 20,000 fresh records of its own, so that the test error is small compared with the effects being measured. DP-SGD runs with the running example's settings: $q = 0.01$, $C = 0.5$, $\eta = 0.5$, $T = 10{,}000$ steps and $\delta = 10^{-5}$.

How to read this chapter. Section 43.3 defines what to measure and shows the effect. Section 43.4 and Section 43.5 explain it, one mechanism each; they are the core of the chapter. Section 43.6 is about measuring per-group accuracy honestly, and applies to any model. Section 43.7 discusses what can be done and what each fix costs.

43.2. Toolkit: four facts every derivation uses

43.2.1. Fact 1 — The expected update is a reweighted average gradient

Each DP-SGD step adds up the clipped gradients of a Poisson batch, adds Gaussian noise of standard deviation σC to each coordinate, and divides by the public expected batch size $B = qn$. Each record is in the batch with probability q, so the expected sum is q times the sum over all records, and dividing by qn gives

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \frac{1}{n}\sum_{i=1}^{n} \min\Big(1, \frac{C}{\|g_i\|}\Big) g_i, \tag{43.1}$$

the average gradient with each record weighted by its clipping weight (DP-SGD). The noise on the averaged update has standard deviation $\tau = \sigma C/B$ per coordinate, whatever the data. In the running example at $\varepsilon = 4$, $\sigma = 1.36$ and $\tau = 1.36 \times 0.5/100 = 0.0068$.

43.2.2. Fact 2 — A drift beats a random walk after (noise/signal)² steps

Suppose each step moves the weights by a signal of length ℓ_{sig} that keeps pointing the same way, plus independent noise of length ℓ_{noise}. After k steps the signals add up to length $k\ell_{\text{sig}}$. The noise vectors are independent with mean zero, so their squared lengths add, and their sum has length about $\ell_{\text{noise}}\sqrt{k}$. The signal is longer once $k\ell_{\text{sig}} > \ell_{\text{noise}}\sqrt{k}$, that is after

$$k > \left(\frac{\ell_{\text{noise}}}{\ell_{\text{sig}}}\right)^2 \tag{43.2}$$

steps. DP-SGD uses this to explain why training works when every step's noise is ten times longer than its signal: at $\varepsilon = 4$ the noise on the averaged gradient has length $\tau\sqrt{p}$ for $p = 3{,}329$ weights, and the signal emerges after a few hundred steps.

43.2.3. Fact 3 — A shrinking recursion with noise settles at a fixed variance

Let a number evolve as $x_{t+1} = (1 - a)x_t + e_t$, where $0 < a < 2$ and the e_t are independent with mean zero and variance v. Each step shrinks x toward zero by the factor $1-a$ and adds fresh noise. Because the noise is independent of the current value, variances add: $\operatorname{Var} x_{t+1} = (1-a)^2 \operatorname{Var} x_t + v$. Since $(1 - a)^2 < 1$, this converges, and the limit V solves $V = (1 - a)^2 V + v$:

$$V = \frac{v}{1 - (1 - a)^2} = \frac{v}{a(2 - a)}. \tag{43.3}$$

For example, with $a = 0.1$ and $v = 1$ the variance settles at $1/(0.1 \times 1.9) = 5.26$: a weak pull toward zero lets the noise accumulate to more than five times one step's variance. Halving the pull roughly doubles the stationary variance. The number of steps to forget the starting point is about $1/a$, since $(1 - a)^t \approx e^{-at}$.

43.2.4. Fact 4 — An accuracy measured on m records is a binomial proportion

If a model classifies each test record of a group correctly with probability p, independently, then the number correct out of m is binomial with mean mp and variance $mp(1 - p)$, and the measured accuracy k/m has standard deviation $\sqrt{p(1 - p)/m}$. The **Clopper–Pearson interval** (Hypothesis testing) inverts the binomial distribution exactly: at the 95% level its upper end u is the accuracy at which k or fewer correct answers would have probability 0.025, and its lower end l the accuracy at which k or more would have probability 0.025. It covers the true accuracy with probability at least 0.95. The smaller the group, the wider the interval, in proportion to $1/\sqrt{m}$.

43.3. Disparate impact, measured

43.3.1. Context

What exactly should you measure to say that DP hurts one group more than another, and does it happen on the running example? An accuracy gap between groups is not enough, because the non-private model may already have one. The question is how much *privacy* adds to each group's error.

43.3.2. Statement

The **privacy cost** of group g is the accuracy the group loses when the same training recipe is made private:

$$\pi_g = \mathrm{acc}_g(\text{non-private}) - \mathbb{E}\big[\mathrm{acc}_g(\text{private})\big], \tag{43.4}$$

where the expectation is over the randomness of private training (Esipova et al. 2023 use this definition; Bagdasaryan et al. 2019 compare the same differences). DP has **disparate impact** when the privacy costs of groups differ, $\pi_a \neq \pi_b$, and the **privacy cost gap** $|\pi_a - \pi_b|$ measures how much.

On the running example with a 5% minority, Table 43.2 gives each group's accuracy over three training runs per row.

Table 43.2.: Test accuracy of each group (mean ± standard deviation over 3 runs) of the running example with a 5% minority, trained with DP-SGD at four budgets ($\delta = 10^{-5}$), with clipping alone, and without privacy. The privacy cost is Equation 43.4.

Training	Majority accuracy	Minority accuracy	Gap	Privacy cost: majority	minority
Non-private SGD	0.9529 ± 0.0018	0.9411 ± 0.0052	0.0119	—	—
Clipping only, $\sigma = 0$	0.9581 ± 0.0015	0.9007 ± 0.0048	0.0574	−0.0051	0.0404
$\varepsilon = 8$, $\sigma = 0.917$	0.9550 ± 0.0006	0.8553 ± 0.0110	0.0997	−0.0020	0.0858
$\varepsilon = 4$, $\sigma = 1.36$	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0993	0.0059	0.0934
$\varepsilon = 2$, $\sigma = 2.28$	0.9366 ± 0.0082	0.7574 ± 0.0379	0.1791	0.0163	0.1836
$\varepsilon = 1$, $\sigma = 4.13$	0.8822 ± 0.0032	0.6133 ± 0.0173	0.2689	0.0707	0.3278

43.3.3. Derivation

This statement is a definition and a measurement, so its "derivation" is the experiment's design, and each choice in it is there to make the comparison fair.

1. **Same data, same recipe.** Every row trains on the same 10,000 records with the same sampling rate, step size and number of steps. The only differences are the clipping (absent in the non-private row) and the noise multiplier, which the accountant of DP-SGD sets for each ε.
2. **Large test sets.** Each group is tested on 20,000 records, so by Fact 4 a measured accuracy near 0.9 has a standard deviation of $\sqrt{0.9 \times 0.1/20{,}000} = 0.0021$. The differences in the table are much larger than that.
3. **Several runs.** Private training is random, and its spread differs between groups. Each row averages three runs and reports their standard deviation; the privacy cost subtracts the average private accuracy from the average non-private accuracy.
4. **A non-private model that serves both groups.** The minority's non-private accuracy, 0.9411, is within 1.2 points of the majority's, 0.9529. Any large gap in the private rows is therefore caused by privacy, not inherited from the data.

43.3.4. Interpretation

The minority pays almost all of it. At $\varepsilon = 4$ the majority loses 0.59 points of accuracy and the minority 9.34, 15.7 times as much. At $\varepsilon = 8$, a budget at which DP-SGD found the running example nearly as accurate as non-private training, the majority loses nothing (its accuracy is 0.20 points higher) and the minority loses 8.58 points. The ratio is smaller at $\varepsilon = 1$, 4.6, only because there the majority starts to lose too: 7.07 points against the minority's 32.78. An average over a test set drawn like the training set, 95% majority, would report a loss of $0.95 \times 0.0059 + 0.05 \times 0.0934 = 0.0103$ at $\varepsilon = 4$, about one point.

Both mechanisms are visible. The clipping-only row has no noise at all, yet the minority loses 4.04 points while the majority gains 0.51: clipping alone is disparate. Adding noise then widens the gap steadily as ε falls. The next two sections explain each part.

The minority's runs also vary more. At $\varepsilon = 4$ the standard deviation over runs is 0.0211 for the minority and 0.0037 for the majority. A single private run says less about a small group than about a large one, a point Section 43.6 returns to.

43.3.5. Example: what the published experiments found

Bagdasaryan, Poursaeed & Shmatikov (2019) trained classifiers with DP-SGD on images and text and found the same pattern: the under-represented classes and subgroups, and the more complex ones, lost the most accuracy, and an existing gap between groups grew. Esipova et al. (2023) report, for MNIST digits where class 8 was undersampled to about 1% of the training set, a privacy cost of 8.9 points for the typical class 2 and 57.9 points for class 8 under DP-SGD (their Table 2, $\varepsilon = 5.90$). On the running example, Figure 43.1 shows the same shape: the majority's curve is the curve of DP-SGD, the minority's lies far below it.

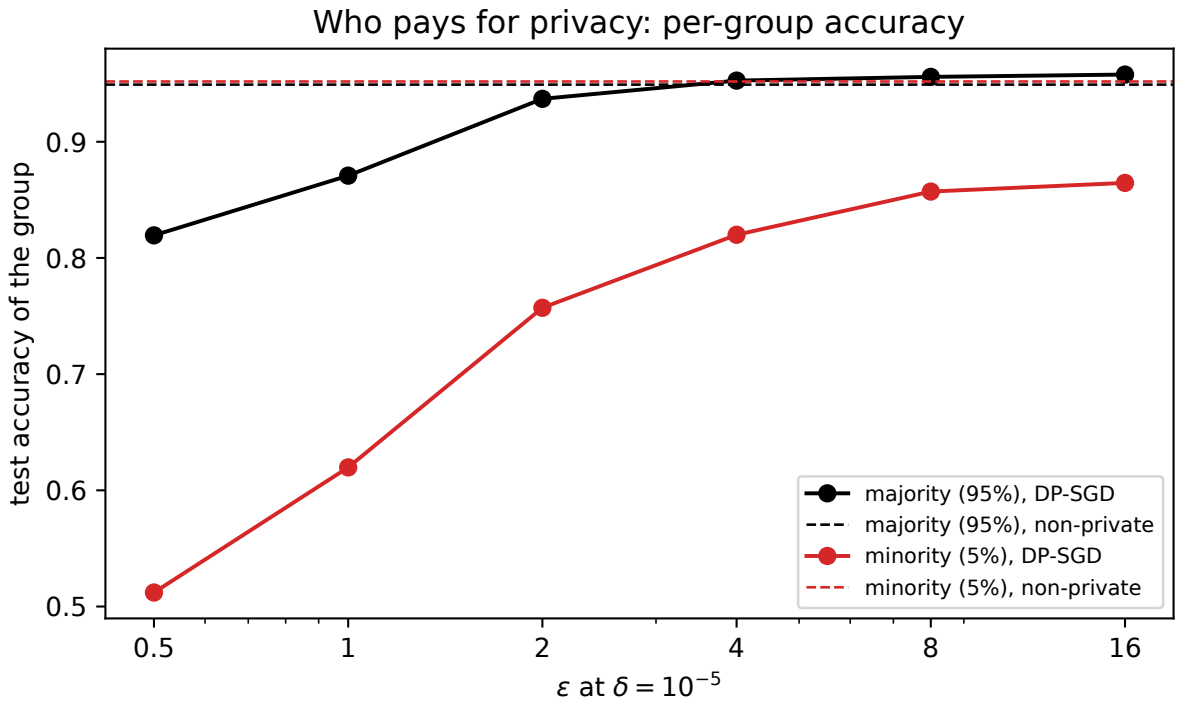


Figure 43.1.: Test accuracy of each group of the running example against ε (log scale, one run per point, $\delta = 10^{-5}$), with each group's non-private accuracy dashed. The majority is close to its non-private accuracy down to $\varepsilon = 2$; the minority falls far below it at every budget.

43.3.6. Check yourself

- Why is the privacy cost a better measure than the gap between the private model's group accuracies? *(Because part of the gap may be inherited from non-private training, which DP did not cause. The privacy cost subtracts each group's own non-private accuracy.)*
- At $\varepsilon = 4$, a test set with the training proportions shows an overall loss of under one point. What would a test set with equal numbers of both groups show? *(The average of the two privacy costs,* $(0.0059 + 0.0934)/2 = 0.0497$*: about five points. The overall number depends on the test set's mix as much as on the model.)*

43.4. Clipping reweights the groups

43.4.1. Context

Clipping without noise changes nothing about privacy (without noise there is none), yet in Table 43.2 it costs the minority 4.04 points. How does a rule that treats every record the same way end up treating groups differently?

43.4.2. Statement

Split the expected update of Fact 1 by group. Write $\bar{g}_g = \frac{1}{n_g}\sum_{i\in g} g_i$ for group g's average gradient and $m_g = \frac{1}{n_g}\sum_{i\in g} \min(1, C/\|g_i\|)\, g_i$ for its average *clipped* gradient. Then

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \sum_g s_g\, m_g, \qquad \|s_g\, m_g\| \le s_g\, C : \tag{43.5}$$

each group pulls with its share times its average clipped gradient, and that pull is never longer than its share times C, however large the group's gradients are.

When the records within each group have parallel gradients, each group's average clipped gradient is its average gradient scaled by a number, and the expected update becomes the gradient of a **reweighted** mixture of the groups:

$$\mathbb{E}\Big[\frac{G_t}{B}\Big] = \Big(\sum_h s_h\, c_h\Big) \sum_g \tilde{s}_g\, \bar{g}_g, \qquad \tilde{s}_g = \frac{s_g\, c_g}{\sum_h s_h\, c_h}, \tag{43.6}$$

where $c_g = \|m_g\|/\|\bar{g}_g\|$ is the fraction of the group's average gradient that survives clipping and $\tilde{s}_g$ is its **effective share**. When all of a group's records have the same norm, c_g is the group's clipping weight w_g.

43.4.3. Derivation

1. **Split the sum.** Fact 1's expected update is $\frac{1}{n}\sum_i \min(1, C/\|g_i\|) g_i$. Group the records: $\frac{1}{n}\sum_g \sum_{i\in g}(\cdots) = \sum_g \frac{n_g}{n} \cdot \frac{1}{n_g}\sum_{i\in g}(\cdots) = \sum_g s_g m_g$.

2. **Bound the pull.** Each clipped gradient has length at most C, and the length of an average is at most the average of the lengths (the triangle inequality), so $\|m_g\| \le C$ and $\|s_g m_g\| \le s_g C$.
3. **Parallel gradients.** If every g_i in group g points along the same unit vector e_g, then $g_i = \|g_i\| e_g$ and $\min(1, C/\|g_i\|) g_i = \min(\|g_i\|, C) e_g$. Averaging, m_g and $\bar{g}_g$ both point along e_g, so $m_g = c_g \bar{g}_g$ with $c_g = \|m_g\|/\|\bar{g}_g\|$.
4. **Factor out the total.** $\sum_g s_g c_g \bar{g}_g = (\sum_h s_h c_h) \sum_g \frac{s_g c_g}{\sum_h s_h c_h} \bar{g}_g$, which is Equation 43.6. The effective shares are non-negative and add up to 1.
5. **Equal norms.** If all of group g's records also have the same norm ν_g, then $c_g = \min(\nu_g, C)/\nu_g = \min(1, C/\nu_g) = w_g$.

43.4.4. Interpretation

Two errors: length and direction. Equation 43.6 separates what clipping does into two parts. The factor $\sum_h s_h c_h$ shortens the update, which a larger step size can undo. The effective shares change *which* mixture of the groups is being descended, and no step size undoes that. Esipova et al. (2023) make this distinction for a whole batch: they compare training with the clipped batch gradient's length but the true direction against the true length but the clipped direction, and find the direction error far more harmful, especially to the under-represented class (their Table 1). In Figure 43.2, clipping shortens the update and also turns it toward the majority.

Which groups are cut back. A group's effective share falls when its clipping weight falls, that is when its gradients are long compared with C. Gradients are long for two reasons. The model fits the group badly: for the logistic loss, the derivative with respect to the output is $-y\,\mathrm{sigmoid}(-yf)$, close to 1 in size for a misclassified record and close to 0 for a record classified with a wide margin. And the group's inputs are large: a dense layer's per-example gradient is the outer product of the back-propagated error and the layer's input, whose norm is the product of the two norms (DP-SGD), so at the same error a record with a longer input has a longer gradient. Tran, Dinh & Fioretto (2021) identify both factors, gradient norms and the input norms behind them, as the source of the clipping part of disparate impact. A small group the model has not yet learned has both.

A feedback loop. Early in training every record is clipped and every group has a similar weight. As the majority is learned, its gradients shrink below C and its weight rises toward 1, while the minority, not yet learned, keeps long gradients and a low weight. Its effective share falls below its true share, so it is learned more slowly, so its gradients stay long. The loop ends only when the minority's gradients fall below C too.

Where training stops. Clipped gradient descent stops where the expected update is zero, $\sum_g s_g m_g = 0$, not where the true average gradient $\sum_g s_g \bar{g}_g$ is zero. If the groups' gradients conflict, the stopping point is that of the reweighted objective, which has moved toward the majority.

43.4.5. Example: two dimensions, then the running example

Take 95 majority records whose gradient is $(0.2, 0)$, already short, and 5 minority records whose gradient is $(0, 4)$, long because the model fits them badly. Without clipping the average gradient

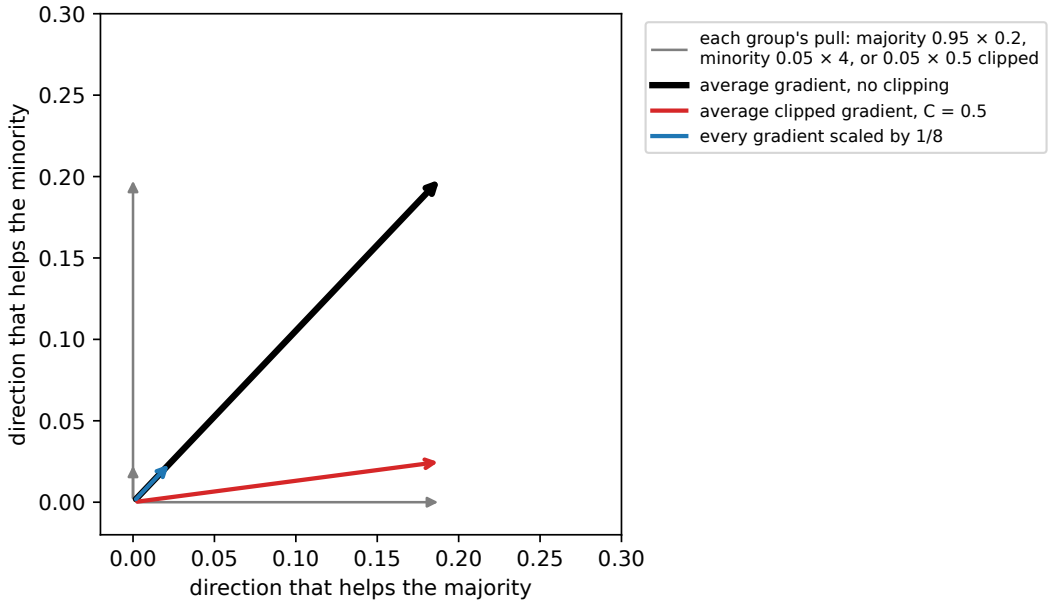


Figure 43.2.: The clipping example in two dimensions: 95 majority records with gradient (0.2, 0) and 5 minority records with gradient (0, 4). Without clipping the two groups pull about equally hard (black); clipping at C = 0.5 cuts the minority's pull eightfold and turns the update toward the majority (red). Scaling every gradient by the same factor 1/8 instead keeps the direction (blue).

is $(0.95 \times 0.2, 0.05 \times 4) = (0.19, 0.2)$, of length 0.2759: the five minority records pull as hard as the ninety-five others, and the minority's component is a fraction $0.2/0.39 = 0.513$ of the sum of the two components. Clip at $C = 0.5$. The majority's gradients are untouched; the minority's shrink to length 0.5. The average becomes $(0.19, 0.05 \times 0.5) = (0.19, 0.025)$, of length 0.1916, and the minority's fraction falls to 0.116. The update has turned by 39.0 degrees toward the majority. Here $c_{\text{maj}} = 1$ and $c_{\text{min}} = 0.5/4 = 0.125$, and Equation 43.6 gives the minority an effective share of $0.05 \times 0.125/(0.05 \times 0.125 + 0.95) = 0.0065$ instead of 0.05. Scaling every gradient by C/Z with $Z = 4$, the largest norm, instead gives $(0.0238, 0.0250)$: eight times shorter, but at an angle of 0.0 degrees to the true average.

In the running example, Table 43.3 follows each group's gradients during training with clipping and no noise, on 1,000 majority and the 500 minority training records.

Table 43.3.: Per-group gradient statistics during training with clipping at $C = 0.5$ and no noise. "Clipped" is the fraction of records with gradient norm above C. The effective share is $s\,w_{\min}/(s\,w_{\min} + (1-s)\,w_{\mathrm{maj}})$ with $s = 0.05$, Equation 43.6 with $c_g = w_g$.

Step	Median norm: majority	Median norm: minority	Clipped: majority	Clipped: minority	Weight w_g: majority	Weight w_g: minority	Effective share of the minority	Test accuracy: majority	Test accuracy: minority
0	3.4	3.7	1.00	1.00	0.165	0.163	0.0495	0.5064	0.5019
2,500	3.8×10^{-4}	2.7	0.18	0.70	0.869	0.424	0.0251	0.9569	0.6724
5,000	1.4×10^{-4}	0.71	0.14	0.54	0.891	0.579	0.0331	0.9607	0.8137
7,500	7.7×10^{-5}	0.30	0.13	0.45	0.906	0.651	0.0364	0.9621	0.8661
10,000	7.0×10^{-5}	0.16	0.11	0.39	0.917	0.702	0.0387	0.9613	0.8900

At step 0 both groups are fully clipped, with similar weights, and the minority's effective share, 0.0495, is its true share. By step 2,500 the majority's median gradient norm has fallen to 3.8×10^{-4}, though 18% of its records, those still near the decision boundary, are clipped. The minority's median norm is still 2.7 and 70% of its records are clipped. Its effective share has fallen to 0.0251, half its true share, exactly while it most needs to be learned: its test accuracy is 0.6724 against the majority's 0.9569. It recovers only as the minority is learned, to 0.0387 at the end. The same happens with noise (Figure 43.3): at $\varepsilon = 4$ the minority's effective share is 0.0261 at step 2,500 and 0.0358 at the end, where its weight is 0.637 against the majority's 0.902. The noise keeps the minority's gradients long, so clipping keeps cutting them back.

43.4.6. Check yourself

- If you multiply the minority's loss by 10, does its pull on the update grow tenfold? *(Not for clipped records. Their gradients grow tenfold and are then rescaled back to length C; only records whose upweighted gradient stays below C gain. Exercise 1 in the exercise book works it out.)*
- Why does a larger clipping norm reduce the clipping part of disparate impact? *(It raises the clipping weights of the groups with long gradients more than those of groups whose gradients are already below C, whose weight is already 1, so the effective shares move back toward the true shares.)*

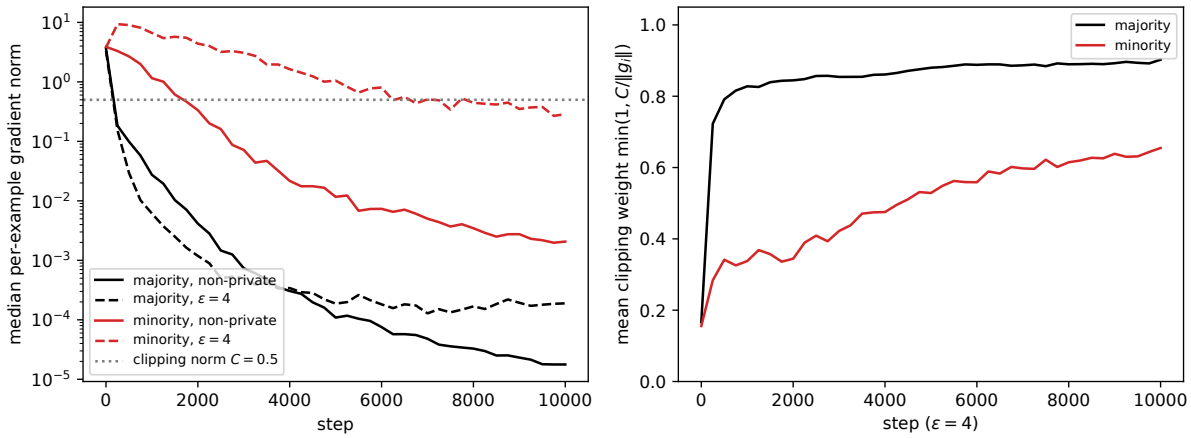


Figure 43.3.: Left: the median per-example gradient norm of each group during non-private training (solid) and DP-SGD at $\varepsilon = 4$ (dashed), on a log scale, with the clipping norm dotted. The majority's gradients fall far below C within the first thousand steps; the minority's stay above it much longer. Right: each group's mean clipping weight during DP-SGD at $\varepsilon = 4$.

43.5. The noise floor of a small group

43.5.1. Context

Noise is added to the sum, the same for everyone. Why should it hurt a small group more than a large one? Because what the noise competes with is each group's signal, and a group's signal in the sum is proportional to its size.

43.5.2. Statement

A drift that is a fraction of the population's. If the part of the expected update that serves group g is a fraction s_g of what the whole population's signal would be, and the noise is unchanged, Fact 2 says that group's signal emerges from the noise only after

$$k_g = \frac{1}{s_g^2}\left(\frac{\ell_{\text{noise}}}{\ell_{\text{sig}}}\right)^2 \tag{43.7}$$

steps, $1/s_g^2$ times as many as for a signal the whole population shares.

A quadratic model of the noise floor. Suppose group g's loss depends on its own block of k weights, which the other groups' losses do not touch, as $L_g = \frac{\lambda}{2}\|w_g - w_g^*\|^2$, and the training objective is the share-weighted average $\sum_g s_g L_g$. Run gradient descent with noise of standard deviation τ per coordinate on the averaged gradient, as DP-SGD adds. Then group g's excess loss settles at

$$\mathbb{E}[L_g] = \frac{k\,\eta\,\tau^2}{2\,s_g\,(2 - \eta s_g \lambda)} \approx \frac{k\,\eta\,\tau^2}{4\,s_g}, \tag{43.8}$$

and it takes about $1/(\eta s_g \lambda)$ steps to get there. With clipping, s_g is replaced by $s_g c_g$, the group's share times the fraction of its gradient that survives clipping; when the majority is hardly clipped, this is close to the effective share $\tilde{s}_g$ of Equation 43.6.

43.5.3. Derivation

1. **The group's gradient.** The objective's gradient with respect to w_g is $s_g \lambda (w_g - w_g^*)$: only group g's loss depends on these weights, and it enters with weight s_g.
2. **One step.** With $x = w_g - w_g^*$ and noise $\tau\xi$ on the averaged gradient, ξ standard normal, the update $w_g \leftarrow w_g - \eta(s_g \lambda x + \tau\xi)$ gives $x \leftarrow (1 - \eta s_g \lambda)x - \eta\tau\xi$, coordinate by coordinate.
3. **Stationary variance.** This is Fact 3 with $a = \eta s_g \lambda$ and $v = \eta^2 \tau^2$: each coordinate settles at variance $\eta^2\tau^2/(a(2-a)) = \eta\tau^2/(s_g\lambda(2 - \eta s_g \lambda))$.
4. **Excess loss.** $L_g = \frac{\lambda}{2} \sum_j x_j^2$ over the k coordinates, so its expectation is $\frac{\lambda}{2} k$ times the variance, which is Equation 43.8. For a small group $\eta s_g \lambda$ is small and $2 - \eta s_g \lambda \approx 2$.
5. **Time scale.** By Fact 3 the starting point is forgotten after about $1/a = 1/(\eta s_g \lambda)$ steps.
6. **The break-even law.** If the population's signal has length ℓ_{sig} per step, the part that serves group g has length $s_g \ell_{\text{sig}}$ and the noise length is still ℓ_{noise}; Fact 2 with $s_g \ell_{\text{sig}}$ in place of ℓ_{sig} gives Equation 43.7.
7. **With clipping.** By Equation 43.6, with parallel gradients within the group, group g's term in the expected update is $s_g c_g \bar{g}_g$ instead of $s_g \bar{g}_g$, while the noise is unchanged. Treating c_g as constant over the steps considered, steps 1 to 6 apply with $s_g c_g$ in place of s_g. If the majority's c is close to 1, then $\sum_h s_h c_h$ is close to 1 and $s_g c_g$ is close to $\tilde{s}_g$.

43.5.4. Interpretation

The floor scales like one over the share. A group with a twentieth of the records sits at about twenty times the excess loss of a group with all of them. The curvature λ cancels: a group whose loss is sharper is pulled back harder, but each unit of noise also costs it more, and the two effects balance. What does not cancel is the share, because the noise is not scaled by it. It also takes the small group $1/s_g$ times longer to reach even that floor.

Why waiting does not help. At a fixed budget, more steps require more noise per step. Under the small-q law of DP-SGD, multiplying the number of steps by m multiplies the noise variance per step by about m, so the break-even count of Equation 43.7 grows as fast as the run (Exercise 3 in the exercise book).

The two mechanisms compound. Clipping lowers a group's pull from s_g to $s_g c_g$; the noise floor scales like one over the pull. A group that is 5% of the data and whose gradients clipping halves pays the floor of a group of 2.5%.

How good is the model. The quadratic model is exact for what it assumes, as the simulation below confirms, but a network is not quadratic and its groups do not use disjoint weights. Where the minority's task overlaps the majority's, the majority's gradient does part of the minority's work, and the penalty is smaller. Tran, Dinh & Fioretto (2021) derive a related per-step decomposition for DP-SGD without assuming disjoint weights: the expected change in a group's loss has a clipping term, driven by the group's gradient norms, and a noise term proportional to $\eta^2\sigma^2C^2$ times the trace of the group's loss Hessian, which they relate to the group's distance to the

decision boundary and its input norms. The quadratic model makes the role of the share explicit, which their per-step terms leave implicit.

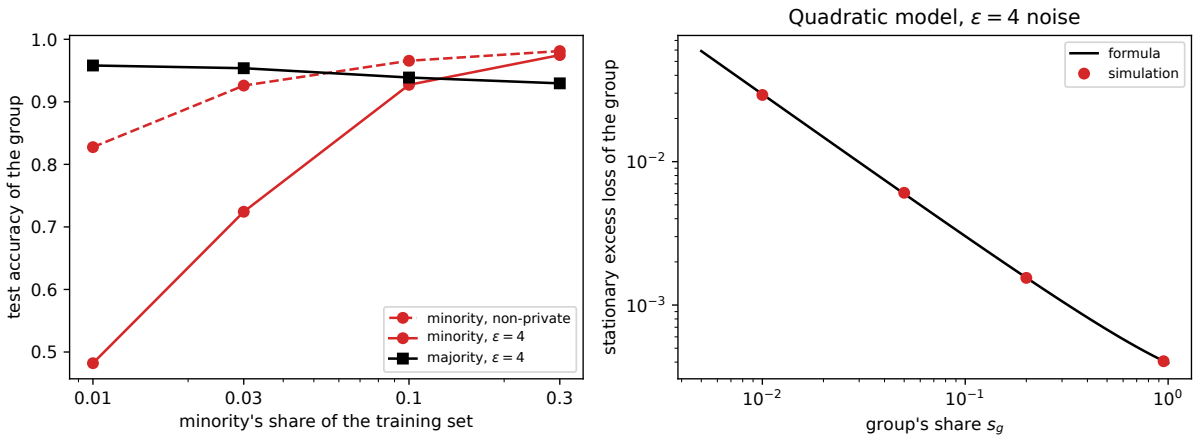


Figure 43.4.: Left: each group's test accuracy of the running example against the minority's share of the 10,000 training records (one run per point), non-private and at $\varepsilon = 4$. Right: the stationary excess loss of a group in the quadratic model, Equation 43.8, against its share, with the noise of $\varepsilon = 4$, compared with simulation.

43.5.5. Example: the model, then the running example

At $\varepsilon = 4$ the noise on the averaged gradient has $\tau = 0.0068$ per coordinate. Take $k = 51$ weights per group, $\lambda = 1$ and $\eta = 0.5$. Table 43.4 compares Equation 43.8 with a simulation of 40,000 noisy steps, averaged over the second half.

Table 43.4.: The noise floor of a group in the quadratic model with $\varepsilon = 4$ noise ($\tau = 0.0068$, $k = 51$, $\lambda = 1$, $\eta = 0.5$).

Share s_g	Excess loss, Equation 43.8	Simulated	Steps to settle, $1/(\eta s_g)$
0.95	4.07×10^{-4}	4.07×10^{-4}	2
0.2	1.55×10^{-3}	1.55×10^{-3}	10
0.05	5.97×10^{-3}	5.99×10^{-3}	40
0.01	2.96×10^{-2}	2.97×10^{-2}	200

The formula and the simulation agree to within 1%, and the floor grows from 4.07×10^{-4} for a group with 95% of the data to 2.96×10^{-2} for a group with 1%, a factor of 73. The approximate form of Equation 43.8 would say 95, the ratio of the shares; the exact form is lower because the factor $2 - \eta s_g \lambda$ is 1.525 for the large group rather than 2. The break-even law is harsher. In DP-SGD the averaged clipped gradient of the running example has length about 0.026 during training, and at $\varepsilon = 4$ the noise has length $\tau\sqrt{3{,}329} = 0.392$. A signal shared by the whole population emerges after $(0.392/0.026)^2 = 228$ steps. A signal that is 20% of it needs 5,693 steps, 5% needs 91,085, nine times the entire run, and 1% needs 2,277,115. A group whose rule the rest of the data does not teach is in this position.

In the network, Table 43.5 varies the minority's share with everything else fixed.

Table 43.5.: Each group's test accuracy for three sizes of the minority, non-private and at $\varepsilon = 4$ (mean ± standard deviation over 2 runs; the 5% row is from Table 43.2, over 3 runs).

Minority share	Records	Non-private: majority	minority	$\varepsilon = 4$: majority	minority	Privacy cost: majority	minority
1%	100	0.9556 ± 0.0015	0.8498 ± 0.0021	0.9558 ± 0.0036	0.5009 ± 0.0102	−0.0002	0.3488
5%	500	0.9529 ± 0.0018	0.9411 ± 0.0052	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0059	0.0934
20%	2,000	0.9395 ± 0.0016	0.9787 ± 0.0004	0.9355 ± 0.0019	0.9507 ± 0.0038	0.0041	0.0280

The minority's privacy cost falls steeply with its share: 34.88 points at 1%, 9.34 at 5%, 2.80 at 20%. Multiplying the share by 5, from 1% to 5%, divides the cost by $0.3488/0.0934 = 3.7$, and multiplying it by 4, from 5% to 20%, divides it by $0.0934/0.0280 = 3.3$: a little less than the ratio of the shares, which is what Equation 43.8 predicts for the loss. Accuracy is not loss, and it saturates: at 1% the private model is at chance on the minority, 0.5009, and cannot lose more. The majority's cost stays below one point throughout. With 20% of the data, the minority is no longer a minority in any sense that matters to the noise, and its privacy cost is within three points of the majority's (Figure 43.4, left).

43.5.6. Check yourself

- The noise multiplier is doubled. By how much does the noise floor of Equation 43.8 grow, and does it grow more for the small group? *(By a factor of 4, since it is proportional to τ^2, for every group alike. The gap between groups grows by the same factor in absolute terms; the ratio between them, s_{major}/s_{minor}, is unchanged.)*
- Why does the curvature λ drop out of the approximate noise floor? *(The pull toward the optimum and the cost of a displacement are both proportional to λ; the stationary variance falls like $1/\lambda$ and the loss per unit of variance rises like λ.)*

43.6. Measuring per-group accuracy

43.6.1. Context

Every number so far was measured on 20,000 test records per group. Real evaluations are not so generous: a test split of 2,000 records with a 5% minority holds 100 minority records. How precisely can a group's accuracy, and the gap between two groups, be measured, and how much does a single training run say?

43.6.2. Statement

The test-set error of a group's accuracy is binomial (Fact 4): with k correct of m records, report the Clopper–Pearson interval. For the gap between two groups measured on disjoint test records, the two estimates are independent and their variances add, so an approximate 95% interval is

$$(\hat{p}_a - \hat{p}_b) \pm 1.96\sqrt{\frac{\hat{p}_a(1-\hat{p}_a)}{m_a} + \frac{\hat{p}_b(1-\hat{p}_b)}{m_b}}, \tag{43.9}$$

with $\hat{p}_g$ the measured accuracies and m_g the numbers of test records. Training randomness is a second, separate source of variation: a group's accuracy varies from run to run, more for small groups, and is estimated only by training several times.

43.6.3. Derivation

1. **One group.** Each test record is classified correctly or not, independently given the model, with probability equal to the group's true accuracy p_g. The count is binomial, and Clopper–Pearson inverts it exactly (Hypothesis testing).
2. **The gap.** The two counts come from disjoint test records, so they are independent given the model. The variance of a difference of independent estimates is the sum of their variances, $p_a(1-p_a)/m_a + p_b(1-p_b)/m_b$. For large counts the difference is approximately normal by the central limit theorem, and 1.96 standard deviations on each side cover 95%.
3. **Two sources.** Conditional on the trained model, the spread comes from the test set alone. Over training runs, the model itself varies; by the law of total variance the spread of a single run's measured accuracy is the test-set variance plus the run-to-run variance of the true accuracy. Only repeated training estimates the second.

43.6.4. Interpretation

Small groups need intervals. The width of the Clopper–Pearson interval falls like $1/\sqrt{m}$ (Figure 43.5). A 5-point gap between groups can be seen only with a few hundred minority test records, and a 1-point gap only with thousands (Exercise 4 in the exercise book). Report the interval, not only the point estimate, for every group.

Report the spread over runs. In Table 43.2 the run-to-run standard deviation at $\varepsilon = 4$ is 0.0211 for the minority against 0.0037 for the majority, almost six times as large. A comparison of two training recipes on one run each can easily be reversed for the small group. This is the same lesson as for the overall accuracy of private models (DP-SGD), sharper for small groups.

The evaluation leaks too. Per-group accuracies computed on private test data are statistics of that data. If they are published, they need their own privacy analysis, which Choosing ε and the selection mechanisms of Hyperparameter tuning address.

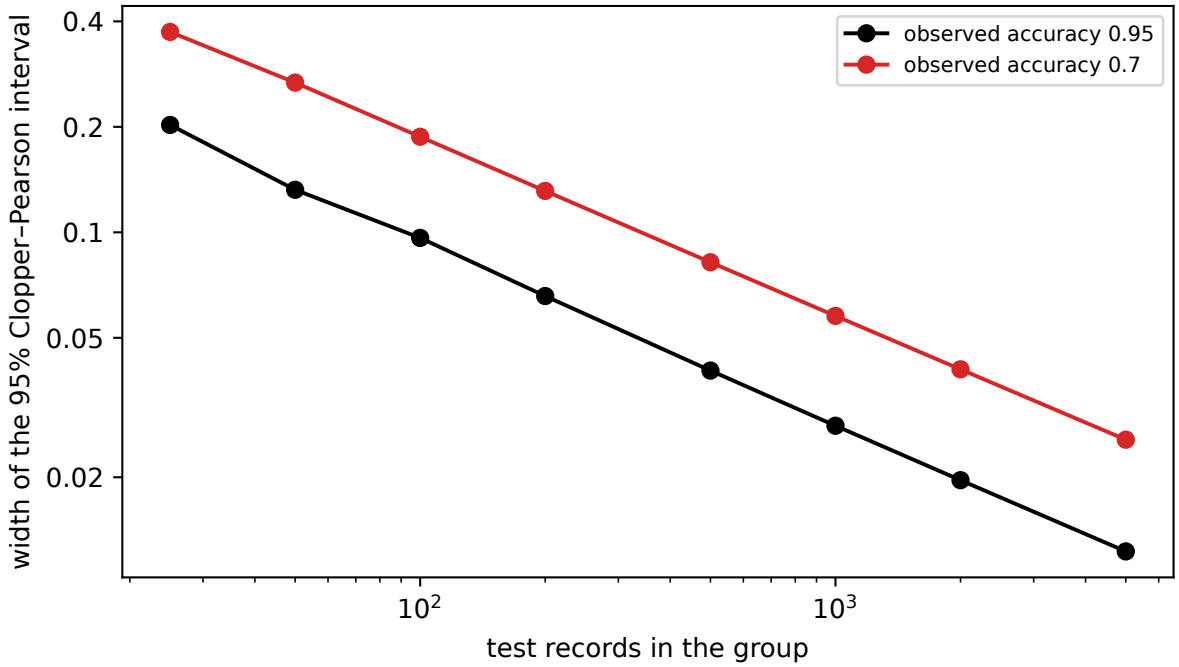


Figure 43.5.: Width of the 95% Clopper–Pearson interval for a group's accuracy against the number of test records in the group, for observed accuracies 0.95 and 0.7, on log scales. The width falls like one over the square root of the number of records.

43.6.5. Example: one model, three test sets

Train the running example at $\varepsilon = 4$ once. On all 20,000 minority test records its accuracy is 0.8641. Table 43.6 measures it on random subsets.

Table 43.6.: Minority accuracy of one model trained at $\varepsilon = 4$, measured on random subsets of the minority's test records, with the majority on 1,900 records for comparison.

Test records	Correct	Estimate	95% Clopper–Pearson interval
100 minority	88	0.880	$[0.800, 0.936]$
500 minority	424	0.848	$[0.813, 0.878]$
2,000 minority	1,742	0.871	$[0.856, 0.885]$
1,900 majority	1,812	0.954	$[0.943, 0.963]$

With 100 minority records, the split a 2,000-record test set with a 5% minority would give, the estimate is 0.880, 1.6 points above the true 0.8641, and the interval is 13.6 points wide. The true accuracy is inside it, as it should be 95% of the time, and the non-private model's 0.9411 from Table 43.2 lies only just above its upper end, 0.936: on this test set alone the minority's privacy cost could be anywhere from about half a point to fourteen points. With 500 records the interval is 6.5 points wide and still contains the true value. The majority's interval, from 1,900 records at a higher accuracy, is 2 points wide. By Equation 43.9 the gap between the groups, measured on the 1,900 and the 100 records, is $0.954 - 0.880 = 0.074$ with

a standard error of $\sqrt{0.954 \times 0.046/1{,}900 + 0.88 \times 0.12/100} = 0.0329$, so its 95% interval is $0.074 \pm 1.96 \times 0.0329 = 0.074 \pm 0.064$, from 0.010 to 0.138: the gap is real, but it could be anywhere from one point to fourteen. And this is one training run: by Table 43.2 the minority's accuracy at $\varepsilon = 4$ varies by 0.0211 from run to run on top of this.

43.6.6. Check yourself

- A report gives a private model's accuracy as 0.84 on a subgroup of 60 test records and 0.85 non-private on the same records. What can you conclude? *(Almost nothing about this subgroup. Each accuracy's 95% interval is about* $2 \times 1.96\sqrt{0.84 \times 0.16/60} = 0.19$ *wide, nineteen times the one-point difference. Because both models were tested on the same records, the difference could be measured more precisely from the records on which they disagree, but the report does not say how many there are.)*
- Why doesn't a larger test set fix the run-to-run variation? *(It removes the test-set error for the model at hand, but a different training run produces a different model. Only repeated training estimates how much a group's accuracy varies with the training noise.)*

43.7. Mitigations and what they cost

43.7.1. Context

Both mechanisms come from DP-SGD's two knobs. Can you choose the knobs, or change the algorithm, so that small groups pay less, without spending more privacy? Each fix in this section moves cost around rather than removing it, and it is worth knowing where to.

43.7.2. Statement

Global scaling is a large clipping norm. Scaling every per-example gradient by C/Z, where no gradient is longer than Z, and adding noise σC, is the same algorithm as DP-SGD with clipping norm Z, the same noise multiplier and step size $\eta C/Z$:

$$\frac{\eta}{B}\Big(\frac{C}{Z}\sum_{i\in S_t} g_i + \sigma C\xi\Big) = \frac{\eta C/Z}{B}\Big(\sum_{i\in S_t} g_i + \sigma Z\xi\Big). \tag{43.10}$$

Oversampling a group costs that group's privacy. If group g's records are sampled at rate vq instead of q, its pull on the update grows by the factor v, and each of its records has the privacy of the subsampled Gaussian at rate vq:

$$\varepsilon_g = \varepsilon_{\text{acct}}(vq, \sigma, T, \delta) \;>\; \varepsilon_{\text{acct}}(q, \sigma, T, \delta), \tag{43.11}$$

where $\varepsilon_{\text{acct}}$ is the accountant of DP-SGD. The algorithm's guarantee is the largest over records, so a fixed target ε requires the noise multiplier for rate vq.

43.7.3. Derivation

1. **Global scaling.** Factor C/Z out of the left-hand side: $\frac{C}{Z}\sum_i g_i + \sigma C\xi = \frac{C}{Z}(\sum_i g_i + \sigma Z\xi)$. With clipping norm Z and no gradient longer than Z, DP-SGD's clipped sum is $\sum_i g_i$ and its noise $\sigma Z\xi$, which is the right-hand side. Both have sensitivity-to-noise ratio $1/\sigma$, so the same accountant applies (Exercise 6 in the exercise book).
2. **Oversampling: the pull.** The expected clipped sum is $\sum_i q_i \min(1, C/\|g_i\|)g_i$, with q_i record i's sampling rate. With $q_i = vq$ for group g and division by the public $B = qn$, group g's term becomes $vs_g m_g$.
3. **Oversampling: the privacy.** The add/remove analysis of record x in DP-SGD compares the output with and without x; the mixture depends only on x's own inclusion probability, because the other records' terms are the same in both. A record sampled at rate vq therefore has the privacy of the subsampled Gaussian at rate vq. The sampling rate may depend on the record's own group, which is part of the record, but the rule must be fixed in advance, not chosen by looking at the data.
4. **The guarantee.** (ε, δ)-DP must hold for every record, so it is the largest of the per-record values, which is Equation 43.11's.

43.7.4. Interpretation

Global scaling trades direction for noise. By Equation 43.10, scaling all gradients by the same factor is the large-C end of the clipping-norm curve of DP-SGD with ηC fixed: nothing is clipped, so the update points in the true direction (the blue arrow of Figure 43.2), but the noise σZ is as large as possible relative to the gradients. Esipova et al. (2023) propose a version that clips the rare gradients above Z instead of discarding them and adapts Z during training from a privately estimated count of how many gradients exceed it; they report that it reduces the privacy cost gap and improves accuracy compared with DP-SGD and with earlier mitigations on MNIST, CelebA and two tabular datasets. Because it treats every record alike, it needs no group labels.

Group-aware methods need group labels. Xu, Du & Wu (2021) adjust each group's contribution according to how much that group is being clipped, giving groups that are clipped more a larger clipping bound, so that the groups' clipping biases are equalized. Tran, Dinh & Fioretto (2021) add to the training objective penalties on the differences between groups in the quantities their decomposition identifies. Both need each training record's group, which is itself sensitive and may not be available or lawful to collect, and the per-group statistics they use must be computed privately.

Oversampling buys little at a fixed budget. Fixed at $\varepsilon = 4$ for every record, sampling the minority four times as often requires $\sigma = 4.71$ instead of 1.36: the minority's pull grows by 4 and the noise by $4.71/1.36 = 3.46$. Its signal-to-noise ratio improves by only $4/3.46 = 1.16$, while the majority's falls by 3.46, and the majority's records are then far more private than the target, $\varepsilon = 0.86$, which is wasted budget (Exercise 5 in the exercise book). Oversampling without changing σ would silently give the minority $\varepsilon = 20.96$: the group that is already disadvantaged in accuracy would also get the weakest privacy.

Some of the cost is unavoidable. Learning a group's own rule privately is, in part, a private estimation problem on that group's n_g records, and private estimation has a price that no algorithm avoids. Lower bounds proves it for the simplest case: releasing d attribute means with constant

accuracy under (ε, δ)-DP needs at least about $\sqrt{d}/(e^{\varepsilon}-1)$ records, by the fingerprinting argument, whatever the mechanism. Put differently, the error that n_g records allow shrinks only like $1/(n_g\varepsilon)$, and the Gaussian mechanism, with error about $\sqrt{d\ln(1/\delta)}/(n_g\varepsilon)$, is close to the best possible. A smaller group is a smaller dataset, and no clever clipping makes it larger. Feldman (2020) argues the deeper point for any learner: when the data have a long tail of small subpopulations, fitting them well requires memorizing their few examples, which is exactly what DP forbids. What helps is more data from the group, a larger ε (with its own costs), or a starting point that already serves the group, such as a public pretrained model (Public data and private fine-tuning).

43.7.5. Example: the clipping norm and oversampling on the running example

Table 43.7 compares three changes at $\varepsilon = 4$ for every record.

Table 43.7.: Mitigations at $\varepsilon = 4$ on the running example with a 5% minority (mean $\pm$ standard deviation). The first row is from Table 43.2 (3 runs); the clipping norm rows keep $\eta C = 0.25$, 2 runs each; the oversampling row samples minority records at rate $4q$ and sets σ so that they meet $\varepsilon = 4$, 3 runs.

Change	σ	Majority accuracy	Minority accuracy	Gap
None: $C = 0.5$, $\eta = 0.5$	1.36	0.9470 ± 0.0037	0.8477 ± 0.0211	0.0993
Smaller clipping norm: $C = 0.05$, $\eta = 5$	1.36	0.9541 ± 0.0008	0.8313 ± 0.0157	0.1229
Larger clipping norm: $C = 5$, $\eta = 0.05$	1.36	0.9463 ± 0.0027	0.8133 ± 0.0166	0.1330
Oversample the minority four times	4.71	0.8669 ± 0.0079	0.8270 ± 0.0111	0.0399

None of the three changes helps the minority. A ten times smaller clipping norm clips the minority more, and its accuracy falls by 1.64 points while the majority's rises by 0.71: the clipping mechanism of Section 43.4 at work. A ten times larger one clips almost nothing, so the update's direction is right, but with ηC fixed the noise in each step, $\eta\sigma C/B$, is unchanged while the step along every gradient shorter than 0.5 is ten times smaller: the noise is ten times larger relative to that part of the signal, and the minority falls by 3.44 points: the noise mechanism of Section 43.5 at work. The running example's $C = 0.5$ sits between the two failures. These differences are of the size of the run-to-run spread, 0.0157 to 0.0211, so only their direction is clear, not their size.

Oversampling narrows the gap from 9.93 to 3.99 points, but by **levelling down**: the minority's accuracy is 2.07 points lower than without it, and the majority's 8.01 points lower. A smaller gap is not by itself a fairer model. Report both groups' accuracy and privacy cost, never the gap alone.

43.7.6. Check yourself

- Why does oversampling a group require more noise, while upweighting its loss does not? *(Upweighting leaves every record's contribution bounded by C and its sampling rate at q, so the accountant is unchanged, but clipping undoes it. Oversampling raises the group's sampling rate, which the subsampled Gaussian's privacy depends on.)*
- A team proposes to fix disparate impact by setting C to the largest per-example gradient norm observed in the training data. What is wrong? *(The clipping norm then depends on the data, and the sensitivity argument of DP-SGD assumes it is fixed in advance. A data-dependent C must itself be chosen privately, as in Esipova et al.'s adaptive threshold, and the estimate charged to the budget.)*

43.8. Code: check every number by computation

The script code/disparate_impact.py builds the running example with a minority group, trains it with the model, clipping and accountant of DP-SGD through a training loop that samples each group at its own rate and logs per-group gradient statistics, and computes the two-dimensional example, the quadratic model and the Clopper–Pearson intervals. From the repository root:

```
uv run python code/part05-dp-ml/disparate_impact.py
```

Output (seed 0), summarized in Table 43.8:

Table 43.8.: The script's output with the default seed.

Experiment	Result
Per-group accuracy, 5% minority	Privacy cost at $\varepsilon = 4$: 0.0059 majority, 0.0934 minority; clipping alone: -0.0051 and 0.0404
Two-dimensional clipping example	Minority's fraction of the update 0.513 $\rightarrow$ 0.116; the update turns by 39.0 degrees
Clipping weights during training	Minority's effective share 0.0495 at the start, 0.0251 at step 2,500, 0.0387 at the end
Quadratic noise floor	Formula and simulation agree; 4.07×10^{-4} at share 0.95, 2.96×10^{-2} at 0.01
Group size at $\varepsilon = 4$	Minority privacy cost 0.3488 at 1%, 0.0934 at 5%, 0.0280 at 20%
Clopper–Pearson on 100 minority records	88 correct, $[0.800, 0.936]$, against a true accuracy of 0.8641
Mitigations at $\varepsilon = 4$	No change of C helps the minority; oversampling four times needs $\sigma = 4.71$ and levels both groups down

tests/test_disparate_impact.py checks that the output still matches.

What to take from it. The disparate impact of DP-SGD needs no special data to appear: a 5% group that lives in another region of input space and follows its own rule is enough. Its two causes can be separated by switching the noise off, and each follows the arithmetic of this chapter: clipping lowers the group's effective share while the majority is learned, and the noise penalizes a small share roughly in proportion to one over the share. Neither knob of DP-SGD removes the effect at a fixed budget; it only moves it between the groups and between the two mechanisms.

43.9. References

- Bagdasaryan, Poursaeed & Shmatikov, "Differential Privacy Has Disparate Impact on Model Accuracy" (NeurIPS 2019). The observation this chapter explains: DP-SGD's accuracy loss falls on under-represented and complex groups, and existing gaps grow.
- Tran, Dinh & Fioretto, "Differentially Private Empirical Risk Minimization under the Fairness Lens" (NeurIPS 2021). Per-step decompositions of a group's excess risk into clipping and noise terms, for output perturbation and DP-SGD, and the role of gradient norms, input norms, the loss Hessian and the distance to the decision boundary.
- Esipova, Ghomi, Luo & Cresswell, "Disparate Impact in Differential Privacy from Gradient Misalignment" (ICLR 2023). Separates clipping's direction error from its length error, finds the direction error the main cause, and proposes global scaling with an adaptive threshold.
- Xu, Du & Wu, "Removing Disparate Impact on Model Accuracy in Differentially Private Stochastic Gradient Descent" (KDD 2021). Group-dependent clipping that equalizes the groups' clipping bias.
- Feldman, "Does Learning Require Memorization? A Short Tale about a Long Tail" (STOC 2020). Why fitting small subpopulations well requires memorizing their examples, and so conflicts with privacy.
- Clopper & Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial" (Biometrika, 1934). The exact interval of Section 43.6.

43.10. Cheat sheet

Group g has share s_g, average gradient $\bar{g}_g$ and clipping weight w_g, the mean of $\min(1, C/\|g_i\|)$ over its records; $\tau = \sigma C/B$. Table 43.9 collects the results.

Table 43.9.: Every result of the chapter.

Result	Formula	Conditions
Privacy cost of a group	$\pi_g = \text{acc}_g(\text{non-private}) - \mathbb{E}[\text{acc}_g(\text{private})]$	Same data and recipe
A group's pull	$s_g m_g$, of length at most $s_g C$	Any gradients
Effective share	$\tilde{s}_g = s_g c_g / \sum_h s_h c_h$, with $c_g = w_g$ for equal norms	Parallel gradients within each group
Break-even steps	$(\ell_{\text{noise}}/\ell_{\text{sig}})^2/s_g^2$	Group's signal a fraction s_g of the population's
Noise floor	$\mathbb{E}[L_g] = k\eta\tau^2/(2s_g(2 - \eta s_g\lambda)) \approx k\eta\tau^2/(4s_g)$	Quadratic loss on the group's own k weights

Result	Formula	Conditions
Accuracy gap interval	$\hat{p}_a - \hat{p}_b \pm 1.96\sqrt{\sum_g \hat{p}_g(1-\hat{p}_g)/m_g}$	Disjoint test records, large counts
Global scaling	Same as clipping norm Z, step $\eta C/Z$	No gradient above Z
Oversampling at vq	Those records' ε is the accountant's at rate vq	Rate fixed in advance per group

Rules of thumb

- Always report accuracy per group, with Clopper–Pearson intervals and the spread over several training runs, next to the non-private model's.
- Expect the privacy cost of a group to grow like one over its share of the data, and faster for groups with long gradients: large inputs or a rule the rest of the data does not teach.
- Clipping alone is disparate. A larger clipping norm (with ηC fixed) reduces the clipping part and raises the noise part.
- Do not upweight a group's loss to help it: clipping undoes it. Do not oversample a group without accounting at its higher sampling rate.
- When a group is too small to learn privately, get more data from it or start from a public model that already serves it.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

44. Choosing ε

44.1. Why this matters

Every model trained with differential privacy (DP) ships with a number, its budget ε. DP is the guarantee that one record changes the probability of any output by at most a factor e^{ε}, up to a slack δ. Chapter 37 trained the part's running example, a small neural network on 10,000 records, with stochastic gradient descent (SGD) made private by per-example clipping and Gaussian noise, DP-SGD, at five budgets, and measured what each costs in accuracy: test accuracy 0.9175 at $\varepsilon = 1$, 0.9441 at $\varepsilon = 2$, 0.9548 at $\varepsilon = 4$ and 0.9604 at $\varepsilon = 8$, against 0.9610 without privacy, all at $\delta = 10^{-5}$. That is half of a decision. The other half is what each budget means for the people in the data: what an attacker who sees the model can learn about one of them, and how sure it can be. Without that half, the choice of ε is made by habit ("everyone uses 8") or by the accuracy curve alone, and neither survives a reviewer, a regulator or a data subject who asks what the number promises.

This chapter supplies the privacy half. It reads the guarantee in three operational ways: as a bound on any membership test, as a bound on how far an attacker's belief can move, and as a bound on reconstructing a record. For each it compares two numbers: what the single pair (ε, δ) promises, and what the actual training algorithm, DP-SGD, really allows. It then turns to δ, to the budgets that real deployments have published and the units they are stated in, and to a procedure for choosing and defending a number.

44.1.1. The one idea: ε is one line under a curve

An attacker who wants to know whether a record was in the training set runs a test, and every test has two error rates. The definition of DP is equivalent to a straight-line bound on those rates: at false positive rate α, the true positive rate is at most $e^{\varepsilon}\alpha + \delta$. But a training algorithm has a whole **trade-off curve**, the best true positive rate at every false positive rate, and (ε, δ) is only one line that lies above it, touching it at one point. Far from that point, the line can be useless: at $\varepsilon = 8$ it allows a test that catches every member while almost never accusing a non-member, while the curve of DP-SGD at the same ε caps the true positive rate at 0.0589 when the false positive rate is 10^{-3}. Choosing ε well means computing and reporting the curve, not just the line, and reading it at the error rates that matter for your threat.

Table 44.1 lists the results and where each is used.

Table 44.1.: The results of this chapter and where each is used.

Result	Where it is used
What one (ε, δ) pair bounds: true positive rate at most e^{ε} times the false positive rate plus δ, and the advantage	Reading any published budget; the baseline every refinement improves on
The whole trade-off curve of DP-SGD: exactly from the privacy loss distribution, bounded by Rényi DP, approximated by Gaussian DP	Stating what a trained model allows at low false positive rates; Auditing
The Bayesian reading: posterior bounds, the realized privacy loss, the precision of a flagged member	Explaining a budget to someone who thinks in probabilities
Reconstruction: success is at most the membership true positive rate at a false positive rate equal to the baseline	Arguing that large budgets can still prevent reconstruction
What δ means and how small it must be	Choosing δ; reading guarantees with large δ
Deployed budgets and their privacy units	Comparing your budget with practice, honestly
A procedure for choosing and justifying ε	Writing the privacy statement of a model

Notation. A **membership test** sees the trained model (or the whole sequence of iterates) and says "member" or "non-member" about a target record z. Its **false positive rate (FPR)** α is the probability that it says "member" when z was not in the training set, and its **true positive rate (TPR)** is the probability that it says "member" when z was. Q is the distribution of the output when z is in the data, P the distribution when it is not, and $L = \ln(Q(y)/P(y))$ is the **privacy loss** of an output y. $\delta(\varepsilon)$ is the smallest δ for which a mechanism is (ε, δ)-DP at a given ε, its **privacy profile**. For DP-SGD, q is the sampling rate, σ the noise multiplier and T the number of steps. The running example has $q = 0.01$, $T = 10{,}000$ and $n = 10{,}000$ records, and its noise multiplier is the one the Rényi differential privacy (RDP) accountant of DP-SGD requires for each target ε at $\delta = 10^{-5}$.

How to read this chapter. The toolkit and Section 44.3 restate what The definition of differential privacy proves, in the form this chapter needs. Section 44.4 is the core: the curve of DP-SGD against the line. Section 44.5 and Section 44.6 read the same curve as beliefs and as reconstruction. Section 44.7, Section 44.8 and Section 44.9 are about practice and can be read on their own.

44.2. Toolkit: four facts every derivation uses

44.2.1. Fact 1 — A membership attacker is a test, and DP bounds the probability of every event

Fix the training set without the target, D^-, and the target record z. The attacker sees an output y, drawn from $Q = M(D^- \cup \{z\})$ if z is a member and from $P = M(D^-)$ if not, and says "member" on some set S of outputs. Then FPR $= P(S)$ and TPR $= Q(S)$: both error rates are probabilities

of one event. A mechanism M is (ε, δ)-DP when, for every pair of neighbouring datasets and every event S, in both orders,

$$Q(S) \le e^{\varepsilon} P(S) + \delta. \tag{44.1}$$

So DP is a statement about tests before it is a statement about anything else. This chapter uses add/remove neighbours, as DP-SGD with Poisson sampling does (DP-SGD). An attacker that randomizes, saying "member" with probability $\phi(y)$, has $\mathrm{TPR} = \mathbb{E}_Q[\phi]$ and $\mathrm{FPR} = \mathbb{E}_P[\phi]$, and the same bound holds for it, because a randomized test is an average of events (The definition of differential privacy).

44.2.2. Fact 2 — The best test thresholds the privacy loss

Among all tests with a given FPR, the one with the largest TPR says "member" when the likelihood ratio $Q(y)/P(y)$ is large, that is, when the privacy loss $L(y)$ exceeds a threshold t, randomizing at the threshold if needed. This is the Neyman–Pearson lemma (Hypothesis testing). The reason is an exchange argument: if a test said "member" on an output with a low ratio and "non-member" on one with a higher ratio, swapping the two (in equal amounts of P-probability) would keep its FPR and raise its TPR.

Two consequences follow. First, the best TPR at FPR α depends only on the distribution of L. Write ω for the distribution of L when $y \sim Q$, the **privacy loss distribution (PLD)**. Then the best test with threshold t has

$$\mathrm{TPR}(t) = Q[L > t], \qquad \mathrm{FPR}(t) = P[L > t] = \mathbb{E}_Q\big[e^{-L};\ L > t\big], \tag{44.2}$$

where the second form is a **change of measure**: $P(y) = e^{-L(y)}Q(y)$ by the definition of L, so the P-probability of any event is the Q-average of e^{-L} over it. Second, the best achievable pairs $(\mathrm{FPR}, \mathrm{TPR})$ form a region whose upper boundary, the **trade-off curve** written here as $\mathrm{TPR}^*(\alpha)$, is increasing and concave: increasing because a test with a higher FPR can always say "member" more often, concave because randomizing between two tests reaches every point of the segment between them.

44.2.3. Fact 3 — Beliefs multiply by the likelihood ratio

An attacker who believes beforehand that z is a member with probability p, the **prior**, and then sees y, believes afterwards

$$\frac{\Pr[\text{member} \mid y]}{\Pr[\text{non-member} \mid y]} = \frac{p}{1-p} \cdot \frac{Q(y)}{P(y)} = \frac{p}{1-p} e^{L(y)}. \tag{44.3}$$

This is Bayes' rule in odds form: the posterior odds are the prior odds times the likelihood ratio, and the likelihood ratio is e^L. Converting odds o back to a probability, $o/(1+o)$, gives the posterior $pe^L/(pe^L + 1 - p)$.

44.2.4. Fact 4 — DP-SGD's worst case is one pair of distributions, composed

Each step of DP-SGD adds Gaussian noise of standard deviation σC to a sum of per-example gradients clipped to length C, over a batch that contains each record independently with probability q. DP-SGD showed that, for the target record, the worst case over models, data and directions is a one-dimensional pair in units of C: without the record the step's output is $\mathcal{N}(0, \sigma^2)$, with it the output is the mixture $(1-q)\mathcal{N}(0,\sigma^2) + q\mathcal{N}(1,\sigma^2)$. The privacy loss of one step is $\ell(y) = \ln\left((1-q) + q\, e^{(2y-1)/(2\sigma^2)}\right)$, and T steps release T outputs whose losses add, so the PLD of the whole run is the distribution of a sum of T such losses. Three accountants summarize it (Choosing an accountant):

- the **RDP accountant** bounds the Rényi divergence of order α by $R(\alpha) = T\varepsilon_{q,\sigma}(\alpha)$ and converts it to (ε, δ); it is a valid upper bound, and it is what DP-SGD used to set σ;
- the **numerical PLD accountant** discretizes the one-step loss on a grid of spacing h, rounding each loss up (pessimistic) or down (optimistic), and composes by fast Fourier transform (FFT) convolution; the truth lies between the two answers, which differ by at most a shift of Th in the total loss (Numerical accounting);
- the **Gaussian differential privacy (GDP)** central limit theorem (CLT) approximates the run by a single Gaussian pair $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$ with $\mu = q\sqrt{T(e^{1/\sigma^2}-1)}$; it is an approximation, not a bound (f-DP and Gaussian DP).

Table 44.2 gives the noise multipliers of the running example and what the three accountants say about them.

Table 44.2.: The running example's noise multipliers at $q = 0.01$, $T = 10{,}000$, $\delta = 10^{-5}$, chosen by the RDP accountant, and the ε each accountant reports for them. The upper end of the PLD bracket is the larger pessimistic value over the two neighbouring directions, the lower end the optimistic value of the direction a membership test uses.

Target ε (RDP)	σ	RDP ε	PLD ε, bracket	Grid h	GDP μ	GDP ε
0.5	7.72	0.500	[0.445, 0.465]	2×10^{-6}	0.130	0.454
1	4.13	0.999	[0.903, 0.923]	2×10^{-6}	0.246	0.909
2	2.28	1.998	[1.825, 1.845]	2×10^{-6}	0.461	1.819
4	1.36	3.993	[3.629, 3.729]	10^{-5}	0.847	3.615
8	0.917	7.997	[7.317, 7.417]	10^{-5}	1.511	7.116
16	0.695	15.955	[14.582, 14.682]	10^{-5}	2.632	14.097

Against the upper end of the PLD bracket, the RDP accountant overstates the true ε by between 0.035 (at $\varepsilon = 0.5$) and 0.264 (at $\varepsilon = 4$), and by 0.58 at $\varepsilon = 8$, consistent with Part 4. Everything below uses these six values of σ, and calls each run by its target, "the $\varepsilon = 4$ run", even where the true ε is a little smaller.

44.3. What one (ε, δ) pair says about a membership test

44.3.1. Context

A model card says "trained with $(\varepsilon, \delta) = (8, 10^{-5})$". Taken alone, what does that number forbid an attacker to do? This is the question to answer first, because it is the only one you can answer for a model whose algorithm you have not seen.

44.3.2. Statement

If M is (ε, δ)-DP, every membership test, against every target, satisfies

$$\mathrm{TPR} \le \min\left\{1,\ e^{\varepsilon}\,\mathrm{FPR} + \delta,\ 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})\right\}, \tag{44.4}$$

and its **advantage**, $\mathrm{TPR} - \mathrm{FPR}$, the amount by which it beats guessing, satisfies

$$\mathrm{TPR} - \mathrm{FPR} \le \frac{e^{\varepsilon} - 1 + 2\delta}{e^{\varepsilon} + 1}. \tag{44.5}$$

44.3.3. Derivation

1. **The first line.** Apply the definition (Fact 1) with $D^- \cup \{z\}$ first and the event S on which the test says "member": $Q(S) \le e^{\varepsilon} P(S) + \delta$, that is $\mathrm{TPR} \le e^{\varepsilon}\mathrm{FPR} + \delta$.
2. **The second line.** Apply it in the other order to the complement S^c, where the test says "non-member": $P(S^c) \le e^{\varepsilon} Q(S^c) + \delta$, that is $1 - \mathrm{FPR} \le e^{\varepsilon}(1 - \mathrm{TPR}) + \delta$. Solving for TPR gives $\mathrm{TPR} \le 1 - e^{-\varepsilon}(1 - \delta - \mathrm{FPR})$. A probability is at most 1, which is the third term of Equation 44.4.
3. **The advantage.** $\mathrm{TPR} - \mathrm{FPR}$ is largest where the two lines of steps 1 and 2 cross, because below the crossing the first line rises faster than the diagonal (slope $e^{\varepsilon} \ge 1$) and above it the second line rises more slowly (slope $e^{-\varepsilon} \le 1$). Setting $e^{\varepsilon}\alpha + \delta = 1 - e^{-\varepsilon}(1 - \delta - \alpha)$ and multiplying by e^{ε} gives $\alpha(e^{2\varepsilon} - 1) = e^{\varepsilon} - 1 + \delta - \delta e^{\varepsilon}$, so $\alpha = (1 - \delta)/(e^{\varepsilon} + 1)$. There the TPR is $e^{\varepsilon}(1 - \delta)/(e^{\varepsilon} + 1) + \delta$, and subtracting α gives Equation 44.5.

44.3.4. Interpretation

The bound has three regimes. At small FPR the line $e^{\varepsilon}\mathrm{FPR} + \delta$ binds, and the ratio TPR/FPR is at most e^{ε} plus δ/FPR. Below $\mathrm{FPR} \approx \delta e^{-\varepsilon}$ the δ term dominates, and the guarantee rests on δ alone. At large FPR the second line binds. And once $e^{\varepsilon}\mathrm{FPR} + \delta \ge 1$, which at $\varepsilon = 8$ happens from $\mathrm{FPR} = (1 - 10^{-5})e^{-8} = 3.35 \times 10^{-4}$ on, the bound allows a perfect true positive rate: the pair says nothing at all at the false positive rates where attacks are usually evaluated.

Low FPR is where it matters. Membership inference evaluates attacks by their TPR at an FPR of 10^{-3} or below, because an attack that flags members while almost never flagging non-members is one that confidently identifies people. At $\varepsilon = 1$ the pair caps that TPR at $e \times 10^{-3} + 10^{-5} = 0.002728$, a real constraint. At $\varepsilon = 4$ the cap is 0.05461, at $\varepsilon = 8$ it is 0.9997.

The advantage is an average, and DP's guarantee is not about averages. Equation 44.5 bounds the advantage at a single point of the curve, the one with equal error rates. It is 0.4621 at

$\varepsilon = 1$ and 0.9640 at $\varepsilon = 4$, numbers that make $\varepsilon = 4$ sound hopeless. They are tight only for the worst mechanism with this ε, randomized response, whose curve is exactly the two lines. DP-SGD is not that mechanism, as the next section shows.

44.3.5. Example: the pair at six budgets

Table 44.3 evaluates Equation 44.4 and Equation 44.5 for the six budgets of the running example. For $\varepsilon = 1$ and FPR $= 0.01$, the first line gives $e \times 0.01 + 10^{-5} = 0.02719$.

Table 44.3.: The largest TPR and advantage of any membership test that the pair $(\varepsilon, 10^{-5})$ allows, from Equation 44.4 and Equation 44.5.

ε	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}	TPR at FPR 10^{-1}	Advantage
0.5	0.001659	0.0165	0.1649	0.2449
1	0.002728	0.02719	0.2718	0.4621
2	0.007399	0.0739	0.7389	0.7616
4	0.05461	0.546	0.9835	0.9640
8	0.9997	0.9997	0.9997	0.9993
16	1	1	1	1.0000

Read this way, the pair says that $\varepsilon \leq 2$ is a strong guarantee everywhere, that $\varepsilon = 4$ is meaningful only at very low FPR, and that $\varepsilon \geq 8$ is no guarantee at all against membership inference. The rest of the chapter shows that, for DP-SGD, the second and third conclusions are far too pessimistic.

44.3.6. Check yourself

- An audit reports TPR 0.02 at FPR 0.001 against a model claimed to be $(4, 10^{-5})$-DP. Does it refute the claim? *(No. The pair allows up to 0.05461 there. An audit refutes a claim only when the measured point lies above the line, and for $\varepsilon = 4$ that needs a TPR above about 5.5% at this FPR.)*
- Why can the advantage bound be close to 1 while the TPR at FPR 10^{-3} is tiny? *(They are different points of the curve. The advantage is read where the two error rates are equal, around FPR 0.27 for $\varepsilon = 1$; the low-FPR point is governed by the line $e^{\varepsilon}\text{FPR} + \delta$, which is small there.)*

44.4. The whole curve of DP-SGD

44.4.1. Context

The pair (ε, δ) is one number out of a whole profile: the accountant could equally have reported ε at $\delta = 10^{-6}$ or 10^{-3}, and each of those pairs is another line that every test must satisfy. The best TPR an attacker can reach against DP-SGD is the curve that all those lines enclose, and for DP-SGD it can be computed. How far below the single line does it lie, and how well do cheaper accountants approximate it?

44.4.2. Statement

For a mechanism with PLD ω (the law of L under Q), the best TPR at FPR α is reached by thresholding the total loss (Fact 2), and equals the lower envelope of all the lines of its profile:

$$\mathrm{TPR}^*(\alpha) = \inf_{\varepsilon\in\mathbb{R}} \big(e^\varepsilon \alpha + \delta(\varepsilon)\big), \qquad \delta(\varepsilon) = Q[L > \varepsilon] - e^\varepsilon P[L > \varepsilon]. \tag{44.6}$$

For DP-SGD the curve is traced exactly by

$$(\mathrm{FPR}(t), \mathrm{TPR}(t)) = (\mathbb{E}_\omega[e^{-L};\ L > t],\ \omega(L > t)), \qquad t \in \mathbb{R}, \tag{44.7}$$

where ω is the T-fold convolution of the one-step PLD of Fact 4. Two cheaper curves bracket or approximate it. From the RDP curve $R(\alpha)$ of the whole run, every test satisfies

$$\mathrm{TPR} \le \min_{\alpha>1} \big(\mathrm{FPR}\cdot e^{R(\alpha)}\big)^{(\alpha-1)/\alpha}, \tag{44.8}$$

a valid bound. The GDP central limit theorem approximates the curve by that of $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$,

$$\mathrm{TPR}^*(\alpha) \approx 1 - \Phi(\Phi^{-1}(1-\alpha) - \mu), \qquad \mu = q\sqrt{T(e^{1/\sigma^2}-1)}, \tag{44.9}$$

with Φ the standard normal distribution function; this is not a bound.

44.4.3. Derivation

1. **Every line is valid.** For each ε, the mechanism is $(\varepsilon, \delta(\varepsilon))$-DP in this direction by the definition of the profile, so by Equation 44.4 every test has $\mathrm{TPR} \le e^\varepsilon \mathrm{FPR} + \delta(\varepsilon)$. Taking the smallest right-hand side over ε gives "$\le$" in Equation 44.6.
2. **The profile in terms of the loss.** For the event $\{L > \varepsilon\}$, $Q(L > \varepsilon) - e^\varepsilon P(L > \varepsilon) = \mathbb{E}_Q[(1 - e^{\varepsilon - L});\ L > \varepsilon]$ by the change of measure of Fact 2, and no other event does better, because adding an output with $L \le \varepsilon$ adds $Q(y) - e^\varepsilon P(y) = Q(y)(1 - e^{\varepsilon-L}) \le 0$ and removing one with $L > \varepsilon$ removes a positive amount. This is the formula for $\delta(\varepsilon)$.
3. **Each line touches the curve.** Take the Neyman–Pearson test with threshold t and the line with $\varepsilon = t$. By step 2, $\delta(t) = \mathrm{TPR}(t) - e^t \mathrm{FPR}(t)$, so $\mathrm{TPR}(t) = e^t \mathrm{FPR}(t) + \delta(t)$: the test lies on the line. Since the tests with thresholds t (and randomization between them) are the best ones by Fact 2, the infimum is attained and Equation 44.6 holds with equality.
4. **Composition adds losses.** The T steps of DP-SGD use independent noise, so the likelihood ratio of all T outputs is the product of the per-step ratios, and the total loss is the sum of the per-step losses. Even though each step's input depends on the earlier outputs, the worst-case pair of Fact 4 is the same at every step, so the per-step losses are independent and identically distributed under Q, and ω is the T-fold convolution of one step's PLD (Numerical accounting). Fact 2 then gives Equation 44.7.
5. **Computing it with a guarantee.** Round each one-step loss up to the grid $h\mathbb{Z}$ and compose: the total loss is rounded up by at most Th, and the resulting pair is less private than the true one, so its curve lies above the true curve. Rounding down gives a curve below it. The script uses $h = 2 \times 10^{-6}$ for the three smallest budgets and $h = 10^{-5}$ for the others, so that the total loss is pinned within 0.02 and 0.1.

6. **The RDP bound.** The Rényi divergence can only shrink when both distributions are passed through the same test (data processing, Rényi DP). After the test, the two distributions are coins with "member" probabilities TPR and FPR, so $e^{(\alpha-1)R(\alpha)} \geq \mathrm{TPR}^\alpha \mathrm{FPR}^{1-\alpha} + (1-\mathrm{TPR})^\alpha (1-\mathrm{FPR})^{1-\alpha}$. Dropping the second, non-negative term and solving for TPR gives $\mathrm{TPR}^\alpha \leq e^{(\alpha-1)R(\alpha)} \mathrm{FPR}^{\alpha-1}$, which is Equation 44.8 after taking the power $1/\alpha$; it holds at every order, so at the best one.
7. **The GDP approximation.** For $\mathcal{N}(0,1)$ against $\mathcal{N}(\mu,1)$ the best test thresholds the output at $\Phi^{-1}(1-\alpha)$, and the shifted distribution exceeds that threshold with probability $1-\Phi(\Phi^{-1}(1-\alpha)-\mu)$. The value of μ is the central-limit approximation of f-DP and Gaussian DP, which matches the mean and variance of the summed loss to leading order in q.

44.4.4. Interpretation

The pair overstates the attacker, and more so the larger ε. Table 44.5 puts the four curves side by side at FPR 10^{-3}. At $\varepsilon = 1$ the pair allows 0.002728 and the true curve is between 0.002209 and 0.00225, a factor of about 1.2. At $\varepsilon = 4$ the pair allows 0.05461 and the curve at most 0.01312, a factor of about 4. At $\varepsilon = 8$ the pair allows 0.9997 and the curve at most 0.0589, a factor of about 17. The reason is visible in Figure 44.1: the curve of DP-SGD is smooth and close to a Gaussian curve, and the line that the accountant reports at $\delta = 10^{-5}$ touches it only at a false positive rate of at most $e^{-\varepsilon} Q[L > \varepsilon]$, below 10^{-4} for every run here (Table 44.8). Everywhere else the line is a loose summary.

The RDP bound is the cheap middle ground. Equation 44.8 uses the same accountant that produced ε, costs nothing extra, and is a guarantee. At $\varepsilon = 8$ it caps the TPR at FPR 10^{-3} at 0.09016 instead of 0.9997. It is still looser than the exact curve, because one Rényi order summarizes the loss distribution by one moment, and because the conversion drops a term; the PLD keeps everything.

GDP is close, but not on the safe side. For budgets up to 8 the GDP curve lies inside the exact bracket at FPR 10^{-3}, and within 0.001 of it at the other operating points: 0.05719 at $\varepsilon = 8$ and FPR 10^{-3}, against the bracket [0.05555, 0.0589]. At $\varepsilon = 16$ it overshoots, 0.3234 against at most 0.2808, because $\mu = 2.632$ is far from the small-$q\sqrt{T}$ regime where the central limit theorem holds. Use it to explain, not to certify.

The curve is the worst case of a strong attacker. It is the best test of an attacker who knows every other record, the architecture, the initialization and every hyperparameter, sees every intermediate iterate, and targets a record whose clipped gradient is as large as clipping allows at every step on which it is sampled. Nasr et al. (2021) showed that an adversary who can choose the target's gradient and see every iterate does reach the accountant's guarantee, so the curve is attainable in that threat model. A typical record attacked through the final model alone is much safer. Table 44.4 measures it for one attack: the loss-threshold attack of Membership inference, which calls a training record a member when its loss is below a threshold, run on one model trained at each budget, with the 10,000 training records as members and the 20,000 test records as non-members.

Table 44.4.: A loss-threshold membership attack on the running example, one trained model per budget. Members are the 10,000 training records, non-members the 20,000 test records. The best advantage is the largest TPR − FPR over 999 thresholds.

ε	Test accuracy	TPR at FPR 10^{-3}	TPR at FPR 10^{-2}	TPR at FPR 10^{-1}	Best advantage
0.5	0.8186	0.0010	0.0102	0.0989	0.0155
1	0.9266	0.0013	0.0108	0.1019	0.0085
2	0.9502	0.0011	0.0096	0.0983	0.0059
4	0.9538	0.0008	0.0072	0.0963	0.0116
8	0.9584	0.0007	0.0098	0.1007	0.0162
16	0.9632	0.0013	0.0102	0.0992	0.0158
Non-private	0.9597	0.0004	0.0099	0.0956	0.0221

The attack is at chance at every budget, and nearly at chance against the non-private model too: the task is noiseless and the network generalizes, so an average record's loss says little about membership. That is a statement about this attack, these records and this task, not a guarantee; Auditing private training builds the attacks that come close to the curve, with canaries chosen to be as exposed as possible.

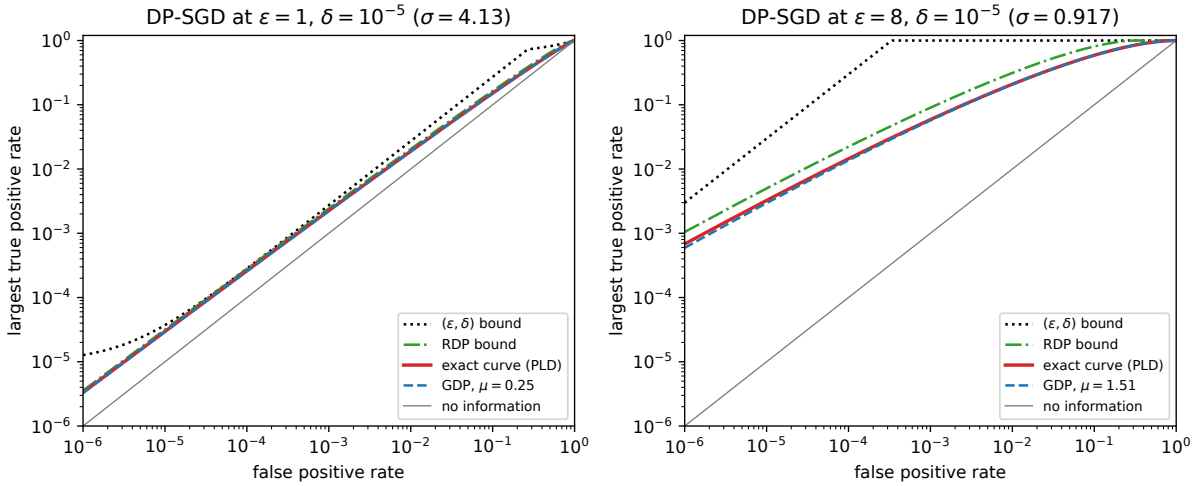


Figure 44.1.: The largest TPR of a membership test against DP-SGD on the running example, on log axes, at $\varepsilon = 1$ (left) and $\varepsilon = 8$ (right), $\delta = 10^{-5}$: the bound from the single pair (dotted), the bound from the whole RDP curve (dash-dotted), the exact curve from the pessimistic PLD (solid red), and the GDP approximation (dashed). The grey diagonal is an attacker with no information.

44.4.5. Example: the curve at six budgets

Table 44.5 gives the four answers at FPR 10^{-3} and Table 44.6 the exact bracket at higher FPR and for the advantage.

Table 44.5.: The largest TPR of a membership test against the running example's DP-SGD at FPR 10^{-3}, $\delta = 10^{-5}$: from the pair, from the RDP curve, exactly (optimistic and pessimistic PLD), and from the GDP approximation.

ε	σ	Pair, Equation 44.4	RDP, Equation 44.8	Exact (PLD bracket)	GDP, Equation 44.9
0.5	7.72	0.001659	0.00161	[0.001524, 0.001554]	0.001537
1	4.13	0.002728	0.002427	[0.002209, 0.00225]	0.002224
2	2.28	0.007399	0.005033	[0.004267, 0.004342]	0.004273
4	1.36	0.05461	0.01668	[0.01215, 0.01312]	0.01244
8	0.917	0.9997	0.09016	[0.05555, 0.0589]	0.05719
16	0.695	1	0.4971	[0.2714, 0.2808]	0.3234

Table 44.6.: The exact trade-off curve of the running example's DP-SGD at two more false positive rates, and its largest advantage, against the advantage the pair allows.

ε	Exact TPR at FPR 10^{-2}	Exact TPR at FPR 10^{-1}	Exact advantage	Pair advantage
0.5	[0.01391, 0.01418]	[0.1236, 0.126]	[0.0472, 0.0567]	0.2449
1	[0.01859, 0.01893]	[0.1489, 0.1515]	[0.0933, 0.1023]	0.4621
2	[0.03088, 0.03141]	[0.2043, 0.2076]	[0.1778, 0.1859]	0.7616
4	[0.06739, 0.0724]	[0.3211, 0.3416]	[0.3094, 0.3429]	0.9640
8	[0.1987, 0.2092]	[0.5682, 0.5901]	[0.5271, 0.5498]	0.9993
16	[0.5447, 0.5588]	[0.8637, 0.8752]	[0.7647, 0.7758]	1.0000

The advantage tells the same story as the low-FPR column. The pair allows 0.9640 at $\varepsilon = 4$; DP-SGD allows at most 0.3429. Figure 44.2 draws the six exact curves together. Each is nearly a straight line on log axes at low FPR, with a slope a little below 1, which means TPR/FPR, the factor by which the attacker's flag is more likely on a member, grows slowly as FPR falls: at $\varepsilon = 8$ it is about 21 at FPR 10^{-2} and about 59 at 10^{-3}, nowhere near $e^8 \approx 2{,}981$.

44.4.6. Check yourself

- Why does the line $e^{\varepsilon}\text{FPR} + \delta$ at $\delta = 10^{-5}$ touch the curve only at a tiny FPR? *(By step 3 it touches the curve at the Neyman–Pearson test with threshold $t = \varepsilon$, whose FPR is*

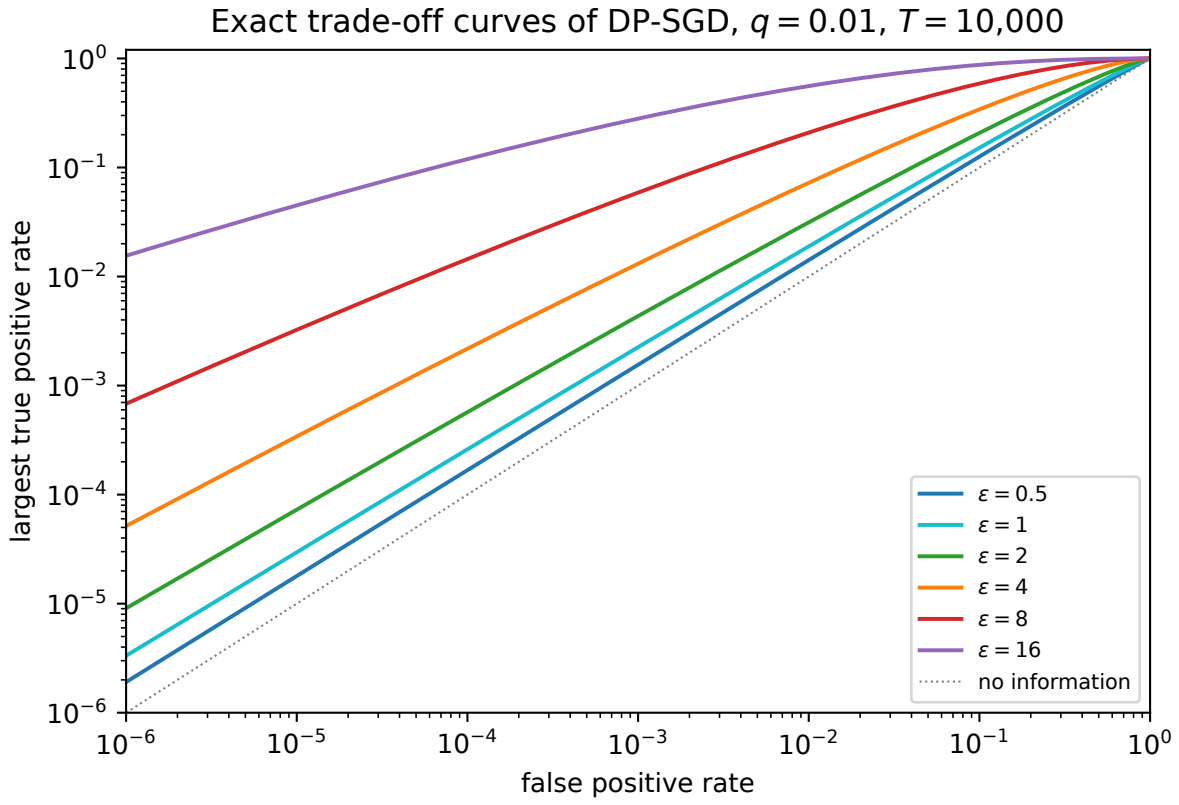


Figure 44.2.: The exact trade-off curves (pessimistic PLD) of DP-SGD on the running example at the six budgets, on log axes. The dotted diagonal is an attacker with no information.

$P[L > \varepsilon] \le e^{-\varepsilon} Q[L > \varepsilon]$, *because* $P = e^{-L}Q$ *and* $L > \varepsilon$ *on that event. The accountant chose* ε *so that the loss rarely exceeds it, so that FPR is tiny.)*

- A colleague proposes to report only the GDP μ of a run with $\mu = 2.6$. What do you tell them? *(That the GDP curve is an approximation that can be too optimistic, as at* $\varepsilon = 16$ *here, or too pessimistic. Report the PLD curve or the RDP bound as the guarantee, and* μ *as an explanation.)*

44.5. The Bayesian reading

44.5.1. Context

People who are not statisticians rarely think in error rates. They ask "if someone sees the model, how sure can they be that I was in the data?" That is a question about posterior probabilities. What do ε, and the actual curve of DP-SGD, say about it, and why does the answer depend so much on the prior?

44.5.2. Statement

If every output has privacy loss at most ε, then for every prior p the posterior probability of membership is at most

$$\Pr[\text{member} \mid y] \le \frac{p\, e^{\varepsilon}}{p\, e^{\varepsilon} + 1 - p}. \tag{44.10}$$

Under (ε, δ)-DP this holds at a level $\varepsilon' > \varepsilon$ except on outputs whose total probability is at most $\delta/(1 - e^{\varepsilon - \varepsilon'})$. For DP-SGD the posterior after the run is exactly $p\, e^{L}/(p\, e^{L} + 1 - p)$, with L drawn from the PLD ω. An attacker who acts only on a flag ("the test said member") is right with probability

$$\Pr[\text{member} \mid \text{flag}] = \frac{p\,\mathrm{TPR}}{p\,\mathrm{TPR} + (1-p)\,\mathrm{FPR}} \le \frac{p\,(e^{\varepsilon}\mathrm{FPR} + \delta)}{p\,(e^{\varepsilon}\mathrm{FPR} + \delta) + (1-p)\,\mathrm{FPR}}. \tag{44.11}$$

44.5.3. Derivation

1. **The posterior is a function of the loss.** By Fact 3, the posterior odds are $\frac{p}{1-p} e^{L(y)}$, and the posterior is $pe^{L}/(pe^{L} + 1 - p)$. This is increasing in L.
2. **Pure bound.** If $L \le \varepsilon$, substitute ε for L: Equation 44.10.
3. **Rare outputs.** Let B be the outputs with $L > \varepsilon'$. On B, $P(y) < e^{-\varepsilon'} Q(y)$, so $P(B) < e^{-\varepsilon'} Q(B)$. The definition gives $Q(B) \le e^{\varepsilon} P(B) + \delta < e^{\varepsilon - \varepsilon'} Q(B) + \delta$, so $Q(B)(1 - e^{\varepsilon - \varepsilon'}) < \delta$ (The definition of differential privacy).
4. **The precision of a flag.** Bayes' rule with the event "flag" in place of y: the flag has probability TPR for a member and FPR for a non-member, so the posterior odds are $\frac{p}{1-p} \cdot \frac{\mathrm{TPR}}{\mathrm{FPR}}$. Bounding TPR by Equation 44.4, which can only increase the odds, gives Equation 44.11. With $\delta = 0$ the bound equals Equation 44.10: a flag can never be more convincing than the most convincing single output.

44.5.4. Interpretation

DP bounds the change of belief, not the belief. Small priors stay small at small ε. Table 44.7 gives Equation 44.10 at three priors. At $\varepsilon = 1$ an attacker who thinks there is a 1% chance that you are in the data can reach at most 2.67%. At $\varepsilon = 4$ it can reach 35.55%, at $\varepsilon = 8$ 96.79%: read through the pure bound, $\varepsilon = 8$ permits near certainty.

Table 44.7.: The largest posterior probability of membership when the privacy loss is at most ε, Equation 44.10.

ε	Prior 0.01	Prior 0.1	Prior 0.5
0.5	0.0164	0.1548	0.6225
1	0.0267	0.2320	0.7311
2	0.0695	0.4509	0.8808
4	0.3555	0.8585	0.9820
8	0.9679	0.9970	0.9997
16	1.0000	1.0000	1.0000

The realized loss is usually much smaller than ε. The bound is attained only by outputs whose loss is ε. For DP-SGD, the loss is a random variable, and Table 44.8 reads its distribution off the pessimistic PLD for a member: the median, and the values exceeded with probability 10^{-3} and 10^{-5}. In the $\varepsilon = 8$ run, half of all members have total loss at most 1.136, and the posterior of an attacker with prior 1% moves to at most 0.0305 for them; only one member in a thousand has loss above 5.971 (posterior 0.7984), and one in a hundred thousand above 7.910 (posterior 0.9649). Figure 44.3 draws these curves against the prior.

Table 44.8.: The total privacy loss of a worst-case member in the running example's DP-SGD runs, from the pessimistic PLD (so slightly overstated), and the posterior it produces from a prior of 1%.

Run	Median loss	Loss exceeded w.p. 10^{-3}	Loss exceeded w.p. 10^{-5}	$Q[L > \varepsilon]$	Posterior from prior 0.01: median, 10^{-3}, 10^{-5}
$\varepsilon = 0.5$	0.018	0.421	0.575	1.10×10^{-4}	0.0102, 0.0152, 0.0176
$\varepsilon = 1$	0.040	0.802	1.093	5.04×10^{-5}	0.0104, 0.0220, 0.0293
$\varepsilon = 2$	0.115	1.549	2.099	2.52×10^{-5}	0.0112, 0.0454, 0.0762
$\varepsilon = 4$	0.401	3.059	4.092	1.59×10^{-5}	0.0149, 0.1771, 0.3768
$\varepsilon = 8$	1.136	5.971	7.910	7.86×10^{-6}	0.0305, 0.7984, 0.9649
$\varepsilon = 16$	3.032	11.661	15.314	3.78×10^{-6}	0.1732, 0.9991, 1.0000

This is the Bayesian face of the gap between the line and the curve. The pure bound treats every output as the worst one; the actual loss is concentrated far below ε, with a thin tail that reaches it. The column $Q[L > \varepsilon]$ shows how thin: the probability that the run's loss for the worst-case member exceeds the reported ε is of the order of δ, between 3.78×10^{-6} and 1.10×10^{-4}.

Base rates make flags unconvincing. In Equation 44.11 the attacker's precision depends on the prior through $p/(1-p)$, and at a small prior even a strong ratio TPR/FPR leaves most flags wrong. At $\varepsilon = 8$, prior 1% and FPR 10^{-2}, the pair allows a precision of 0.5024, while the exact curve allows 0.1744: five flags out of six are false. At $\varepsilon = 4$ and FPR 10^{-3}, the pair allows 0.3555 and the curve 0.1170. Who the attacker is matters: an insider who already knows that you are one of two candidates has prior 0.5; a stranger scanning a population has a prior near the fraction of the population in the dataset.

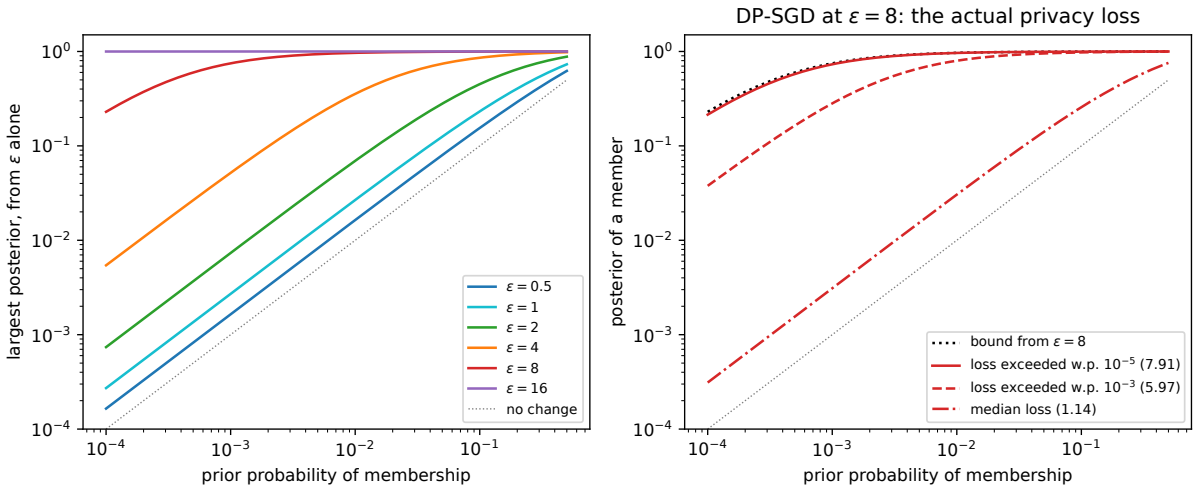


Figure 44.3.: Left: the largest posterior probability of membership against the prior, from Equation 44.10, for the six budgets (log axes). Right: for the $\varepsilon = 8$ run, the bound (dotted) and the posterior of a member whose total privacy loss is the median of the PLD, or is exceeded with probability 10^{-3} or 10^{-5}.

44.5.5. Example: an insider and a stranger

An insider knows that the training set is one of two: with you, or with your colleague in your place. Its prior is 0.5. Against the $\varepsilon = 2$ run, Table 44.7 caps its posterior at 0.8808 on the worst output. Against the same run, a stranger with prior 0.01 reaches at most 0.0695 on the worst output and, for a typical member, $0.01e^{0.115}/(0.01e^{0.115} + 0.99) = 0.0112$, the value in Table 44.8. The guarantee is the same; what it protects against depends on how much the attacker knew before.

44.5.6. Check yourself

- Can DP make an attacker less sure than its prior? *(Yes, for outputs with negative loss, which are typical for non-members. Equation 44.10 bounds the increase; the lower bound $pe^{-\varepsilon}/(pe^{-\varepsilon} + 1 - p)$ bounds the decrease.)*

- Why is $Q[L > \varepsilon]$ for the $\varepsilon = 8$ run below $\delta = 10^{-5}$, while for the $\varepsilon = 4$ run it is above? *(Because $\delta(\varepsilon) = \mathbb{E}_Q[(1 - e^{\varepsilon - L})_+]$ is at most $Q[L > \varepsilon]$, but not equal to it; and because the RDP accountant overstates ε, by more at $\varepsilon = 8$, so the true loss rarely reaches the reported value.)*

44.6. Reconstruction

44.6.1. Context

Membership is a demanding standard: the attacker is given the target record and only has to decide whether it was used. Many harms need more: recovering a record the attacker does not know, such as a patient's diagnosis or a person's face. Balle, Cherubin & Hayes (2022) formalize this as **reconstruction robustness**: an informed adversary knows every training record except the target Z, knows a prior π from which Z was drawn, sees the trained model, and outputs a guess; it succeeds if the guess is within error η of Z. The baseline is the best success without seeing the model,

$$\kappa = \sup_{z_0} \Pr_{Z \sim \pi} \left[\mathrm{err}(Z, z_0) \le \eta\right], \tag{44.12}$$

for example $\kappa = 1/m$ when Z is uniform over m candidates and only an exact guess counts. How much can the model raise the success above κ?

44.6.2. Statement

If every neighbouring pair of M has trade-off curve at or below TPR^*, then every reconstruction attack succeeds with probability at most

$$\Pr[\text{success}] \le \mathrm{TPR}^*(\kappa). \tag{44.13}$$

With the pair (ε, δ) this is $e^\varepsilon \kappa + \delta$; with pure ε-DP, $e^\varepsilon \kappa$, and with one RDP order, $(\kappa e^{R(\alpha)})^{(\alpha-1)/\alpha}$, which are Balle, Cherubin & Hayes's bounds; with the exact curve of DP-SGD it is the bound of Hayes, Mahloujifar & Balle (2023), who write it as a "blow-up function", the largest probability under Q of an event whose probability under P is at most κ, which is the trade-off curve under another name.

44.6.3. Derivation

1. **A reconstruction attack defines one test per candidate.** Fix D^- and the attack R. For a candidate z, let S_z be the set of models θ for which $R(\theta)$ is within η of z. The attack succeeds on z exactly when the model trained on $D^- \cup \{z\}$ falls in S_z.
2. **Each test is bounded by the curve.** "Say member when $\theta \in S_z$" is a membership test for z against the pair $Q_z = M(D^- \cup \{z\})$, $P = M(D^-)$. Its FPR is $a_z = P(S_z)$, so its TPR satisfies $Q_z(S_z) \le \mathrm{TPR}^*(a_z)$.
3. **Average over the prior.** The success probability is $\mathbb{E}_{Z \sim \pi}[Q_Z(S_Z)] \le \mathbb{E}_Z[\mathrm{TPR}^*(a_Z)] \le \mathrm{TPR}^*(\mathbb{E}_Z[a_Z])$, by Jensen's inequality, since the curve is concave (Fact 2).

4. **The average FPR is at most the baseline.** $\mathbb{E}_Z[a_Z] = \Pr_{Z\sim\pi,\,\theta\sim M(D^-)}[\mathrm{err}(Z, R(\theta)) \le \eta]$. Here θ is trained without Z, so it is independent of Z, and for each fixed θ the guess $R(\theta)$ is a fixed z_0, which succeeds with probability at most κ. So $\mathbb{E}_Z[a_Z] \le \kappa$, and since the curve is increasing, Equation 44.13 follows.
5. **The special cases.** Substitute the line of Equation 44.4, or the RDP bound Equation 44.8 at one order, for TPR*.

The (ε, δ) form $e^\varepsilon\kappa + \delta$ is this chapter's corollary; Balle, Cherubin & Hayes state the pure-DP, RDP and zero-concentrated cases.

44.6.4. Interpretation

Reconstruction is membership at FPR κ. The derivation turns any reconstruction attack into a membership test whose false positive rate is the baseline. So the trade-off curve, read at FPR $1/m$, answers "can the attacker pick the right one of m candidates?". A large prior uncertainty is a small FPR, and that is where the curve of DP-SGD is far below the line.

Large budgets can still prevent reconstruction. Table 44.9 reads the curves of Figure 44.2 at $\kappa = 1/m$. At $\varepsilon = 8$ the pair gives no bound for any m up to 1,000, but the curve caps the success of choosing among 1,000 equally likely candidates at 0.0589, against 0.001 by guessing. Balle, Cherubin & Hayes observed the same in experiments: in their words, DP with large values of ε can protect against reconstruction attacks if the target is high-dimensional and the adversary's prior contains a large degree of uncertainty. Hayes, Mahloujifar & Balle found their bound close to their best attack in all their settings, and that different DP-SGD hyperparameters with the same ε can give quite different reconstruction success: the curve, not ε, is what matters.

Small priors are the honest assumption only if you can defend them. If the attacker can narrow the target to two candidates, $\kappa = 0.5$ and reconstruction is as easy as membership at FPR 0.5. A defensible reconstruction claim states the prior: "an attacker who knows the target is one of m records".

Guo, Karrer, Chaudhuri & van der Maaten (2022) bound reconstruction differently, for continuous targets: under RDP of order 2 at level ε, the expected mean squared error of any reconstruction is at least of order $1/(e^\varepsilon - 1)$, and Fisher information gives a tighter, data-dependent bound for unbiased reconstructions.

44.6.5. Example: one of m candidates

Table 44.9.: The largest probability of reconstructing a record drawn uniformly from m candidates (baseline $1/m$), from the pair $e^\varepsilon\kappa + \delta$ and from the exact (pessimistic) curve of the running example's DP-SGD.

ε	$m = 10$: pair	$m = 10$: curve	$m = 100$: pair	$m = 100$: curve	$m = 1{,}000$: pair	$m = 1{,}000$: curve
0.5	0.1649	0.126	0.0165	0.01418	0.001659	0.001554
1	0.2718	0.1515	0.02719	0.01893	0.002728	0.00225
2	0.7389	0.2076	0.0739	0.03141	0.007399	0.004342

ε	$m = 10$: pair	$m = 10$: curve	$m = 100$: pair	$m = 100$: curve	$m = 1{,}000$: pair	$m = 1{,}000$: curve
4	1	0.3416	0.546	0.0724	0.05461	0.01312
8	1	0.5901	1	0.2092	1	0.0589
16	1	0.8752	1	0.5588	1	0.2808

Ten candidates is a strong prior: at $\varepsilon = 4$ the curve allows the attacker 0.3416 against a baseline of 0.1. At a thousand candidates even $\varepsilon = 16$ keeps the success at 0.2808.

44.6.6. Check yourself

- Why does the derivation need the curve to be concave? *(To move the average over candidates inside: the average of the per-candidate bounds is at most the bound at the average FPR. Without concavity, a few candidates with large FPR could do better than their average.)*
- An attacker knows a patient's record except one binary field. What is κ, and what does $\varepsilon = 4$ guarantee? *(If both values are equally likely a priori, $\kappa = 0.5$; the exact curve at FPR 0.5 is far above 0.5, so little. Attribute inference with a strong prior is membership inference in disguise.)*

44.7. Interpreting δ

44.7.1. Context

Every number so far was at $\delta = 10^{-5}$. What does δ mean, how small must it be, and what does a smaller one cost in ε for DP-SGD?

44.7.2. Statement

The mechanism that publishes each record in full, independently with probability δ, is $(0, \delta)$-DP and publishes at least one of n records with probability

$$1 - (1 - \delta)^n \approx 1 - e^{-n\delta}. \tag{44.14}$$

So $\delta = 1/n$ permits exposing a record with probability about 0.63, and a meaningful guarantee needs $n\delta \ll 1$. For a Gaussian-type mechanism such as DP-SGD, δ is not a probability of catastrophe: the privacy loss has a light tail, $\delta(\varepsilon) \leq Q[L > \varepsilon]$, and shrinking δ by orders of magnitude raises ε only moderately.

44.7.3. Derivation

1. **The release mechanism is $(0, \delta)$-DP.** Run it on $D^- \cup \{z\}$ by running it on D^- and tossing one more coin for z. The two outputs differ only when that coin publishes z, which has probability δ, so for every event S the two probabilities differ by at most δ in either direction (The definition of differential privacy).
2. **Exposure.** Each record escapes with probability $1 - \delta$, independently, so all escape with probability $(1-\delta)^n = e^{n\ln(1-\delta)} \approx e^{-n\delta}$, which gives Equation 44.14.
3. **δ is at most the tail of the loss.** $\delta(\varepsilon) = \mathbb{E}_Q[(1 - e^{\varepsilon - L})_+]$ (step 2 of Section 44.4), and the integrand is at most 1 and is 0 unless $L > \varepsilon$.
4. **Smaller δ for DP-SGD.** Run the RDP conversion of DP-SGD at several δ for the same σ; Table 44.10 and Figure 44.4 show the result.

44.7.4. Interpretation

Two kinds of δ. In the release mechanism, δ is spent at infinite loss: the published record is identified with certainty. In DP-SGD it is spent at moderate losses, just above ε, and the loss distribution has a Gaussian-like tail (Table 44.8). That is why NIST's guidelines recommend, where possible, guarantees such as Rényi DP, zero-concentrated differential privacy (zCDP) or GDP, which do not allow catastrophic failure, and why a full privacy profile, or the curve, is better than any one pair.

How small. Near, Darais, Lefkovitz & Howarth (NIST SP 800-226, 2025) write that a common recommendation is $\delta \le 1/n^2$, that $\delta \le 1/(n \log n)$ is another sensible choice, and that values of δ exceeding 10^{-5} are suspicious and should be justified. The running example, with $n = 10{,}000$ and $\delta = 10^{-5}$, has $n\delta = 0.1$: acceptable for DP-SGD, whose δ is not catastrophic, but not by the $1/n^2 = 10^{-8}$ rule. Table 44.10 shows what the stricter rule costs: the $\varepsilon = 4$ run is $(5.209, 10^{-8})$-DP.

The unit decides n. If one person can contribute many records, n is the number of people and δ must be small relative to that; and the per-record guarantee is not a per-person guarantee at all (User-level differential privacy).

Table 44.10.: The ε of three DP-SGD runs of the running example at five values of δ, from the RDP accountant.

σ (run)	$\delta = 10^{-4}$	10^{-5}	10^{-6}	10^{-8}	10^{-10}
4.13 ($\varepsilon = 1$)	0.852	0.999	1.129	1.354	1.548
1.36 ($\varepsilon = 4$)	3.501	3.993	4.432	5.209	5.890
0.917 ($\varepsilon = 8$)	7.088	7.997	8.821	10.301	11.627

44.7.5. Example: ten thousand records

For $n = 10{,}000$, Equation 44.14 gives an exposure probability of 0.6321 at $\delta = 10^{-4} = 1/n$, 0.09516 at $\delta = 10^{-5}$ and 0.00995 at 10^{-6} for the release mechanism. For DP-SGD the same tenfold reductions of δ cost little: from 10^{-5} to 10^{-6} the $\varepsilon = 4$ run's budget rises from 3.993 to

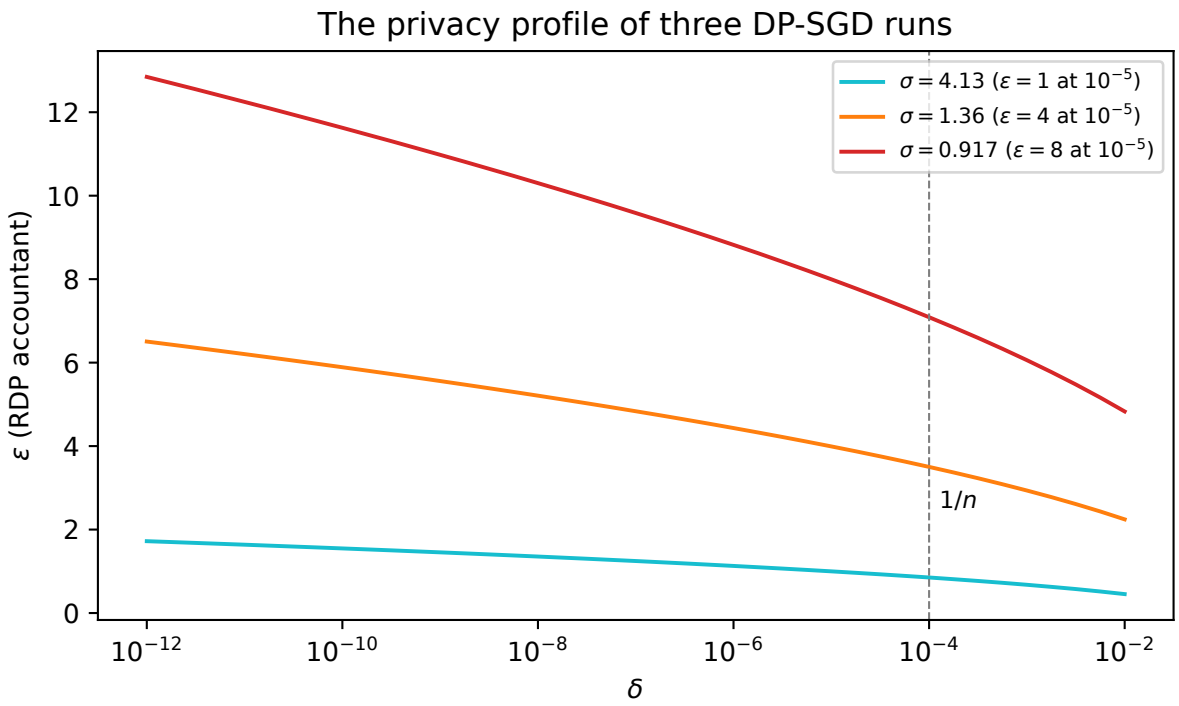


Figure 44.4.: The privacy profile $\varepsilon(\delta)$ of three DP-SGD runs of the running example, from the RDP accountant. The dashed line marks $\delta = 1/n$.

4.432, and down to 10^{-10} it reaches 5.890. Every factor of 100 in δ costs between about 0.2 and 0.9 in ε for these two runs. When in doubt, use a smaller δ; it is cheap.

44.7.6. Check yourself

- A model is reported as $(1, 10^{-3})$-DP on 50,000 records. What is wrong? *(It allows $n\delta = 50$: the release mechanism with this δ would publish about 50 records. Unless the loss distribution is shown to be light-tailed, the guarantee permits catastrophic failure for some people.)*
- Why does the ε in Table 44.10 grow roughly like $\sqrt{\ln(1/\delta)}$? *(For a nearly Gaussian loss with variance v, the tail beyond ε falls like $e^{-(\varepsilon-m)^2/(2v)}$, so reaching a tail of δ needs $\varepsilon - m$ of order $\sqrt{2v\ln(1/\delta)}$.)*

44.8. Deployed values and their units

44.8.1. Context

"Industry uses ε between 1 and 10" is a common justification. What have real deployments published, and in which units? An ε means nothing without the **privacy unit** (what one neighbouring change is: one record, one user's data for a day, one user's data forever) and the **period** over which the budget is spent.

44.8.2. Statement

Table 44.11 lists deployments whose parameters are documented in a primary source. The privacy unit and period differ from row to row, and so do the accounting notions, so the ε values are not comparable without conversion. The zCDP guarantees convert to (ε, δ) by the simple bound

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{44.15}$$

of Bun & Steinke (2016), and tighter numerical conversions give somewhat smaller values; pure ε-DP guarantees for independent reports compose by adding. The Gboard models were trained by follow the regularized leader (FTRL) with noise added to sums of gradients, DP-FTRL (DP-FTRL and matrix factorization), in federated learning, where the data stay on users' devices.

Table 44.11.: Published differential-privacy parameters of real deployments, with their privacy units. Sources are in the references; values are as stated there, except the RAPPOR ε_∞, computed here from the paper's formula.

Deployment	Mechanism and accounting	Privacy unit and period	Published parameters
US Census Bureau, 2020 Census redistricting data	Discrete Gaussian, zCDP	One person's (or housing unit's) record, replace-one; one release	$\rho = 2.63$ in total, 2.56 for persons and 0.07 for housing units; reported as $\varepsilon = 17.14$ (persons) plus 2.47 (housing), 19.61 in total, at $\delta = 10^{-10}$
US Census Bureau, 2020 Demographic and Housing Characteristics file	Discrete Gaussian, zCDP	As above	$\rho \approx 4.96$ for persons, ≈ 7.70 for households
Google Gboard language models	DP-FTRL in federated learning, zCDP	One device's data, whole training run	ρ from 0.25 to 1.86 across the models of the paper's privacy statement; $\rho = 0.25$ is $\varepsilon = 4.49$ and $\rho = 1.86$ is $\varepsilon = 13.69$ at $\delta = 10^{-10}$ (numerical conversion)
Google Gboard, Spanish model (2022)	As above	As above	$\rho = 0.81$, $(8.9, 10^{-10})$; later models down to $\varepsilon = 0.994$
Apple, local DP on iOS and macOS	Local randomizers, pure ε	One donation; limited donations per day; data kept up to three months	Emoji $\varepsilon = 4$, one donation per day; QuickType $\varepsilon = 8$, two per day; Health types $\varepsilon = 2$, one per day; Lookup Hints $\varepsilon = 4$, two per day

Deployment	Mechanism and accounting	Privacy unit and period	Published parameters
Google RAPPOR in Chrome (homepages)	Local randomized response, pure ε	One report (ε_1); one unchanging value over all reports (ε_∞)	$\varepsilon_1 = 0.5343$; $\varepsilon_\infty = 2.043$ from the paper's formula
Microsoft Windows app-usage telemetry	Local, pure ε, with memoization	One user, one collection round every 6 hours	$\varepsilon = 1$ per round and app; 0.686 after output perturbation; 1.672 across all apps
LinkedIn Audience Engagements API	Gaussian and Laplace noise on counts	One member, per analyst, per month	$(0.15, 10^{-10})$ per returned result; $(34.9, 7 \times 10^{-9})$ per month
Facebook URLs dataset for researchers	Gaussian noise, zCDP	One action (a user and a URL); user level for users with at most 100 actions	Per action, $\rho = 0.005$ gives $\varepsilon = 0.485$ at $\delta = 10^{-5}$; user level $\varepsilon = 0.45$ at $\delta = 10^{-5}$

44.8.3. Derivation: making the rows comparable

1. **zCDP to (ε, δ).** Equation 44.15 with $\rho = 2.56$ and $\delta = 10^{-10}$ gives $2.56 + 2\sqrt{2.56 \times 23.026} = 17.915$, the persons' figure of Kazan & Reiter (2022); the Bureau's 17.14 comes from a tighter conversion. For the Spanish Gboard model, $\rho = 0.81$ gives 9.447 by Equation 44.15, against the 8.9 Google reports from a numerical conversion. The Bureau's 19.61 adds the ε of persons and housing units; converting the summed $\rho = 2.63$ once would give a smaller number (18.194 by Equation 44.15).
2. **Replace-one to add/remove.** The Census guarantee is for replacing one person's record, which is a removal followed by an addition, so its ε is not directly comparable with the add/remove ε of DP-SGD; an add/remove guarantee of ε implies a replace-one guarantee of 2ε (The definition of differential privacy).
3. **Local reports over time.** A local randomizer with ε per donation, run on fresh data k times, composes to $k\varepsilon$ for a person whose data are correlated across reports. Apple's per-donation numbers are per day as stated: QuickType's two donations of $\varepsilon = 8$ are 16 per day. Tang, Korolova et al. (2017) reverse-engineered macOS 10.12 and found a daily privacy loss of up to 16 for four applications, renewing every day. RAPPOR's memoization makes a value that does not change cost at most $\varepsilon_\infty = 2h \ln \frac{1-f/2}{f/2}$ over all its reports; with $h = 2$ hash functions and $f = 0.75$ that is $4 \ln(5/3) = 2.043$, but a value that changes is reported afresh.
4. **Per day versus total.** A budget per user per day of $\varepsilon = 1$ adds up to 365 over a year for someone whose behaviour is the same every day. NIST SP 800-226 uses exactly this example to warn that event-level and user-day units can be surprisingly weak.

44.8.4. Interpretation

The model rows are the comparable ones. For training a model on user data, the Gboard models are the best documented: device-level guarantees for a whole training run, ε between about 1 and 14 at $\delta = 10^{-10}$. The running example's budgets of 1 to 8 at $\delta = 10^{-5}$ are in that range, but at a much larger δ and with records, not users, as the unit.

Large published values are not rare. The Census Bureau published $\varepsilon = 17.14$ for persons. Many local deployments spend several units of ε per day. NIST SP 800-226 notes that many deployments have used $1 < \varepsilon \leq 20$. These are facts about practice, not evidence that a value is safe; Dwork, Kohli & Mulligan (2019) found no clear consensus on how deployments choose ε, and proposed an Epsilon Registry, a public record of deployed parameters, so that choices could be compared and criticized.

Compare like with like. Desfontaines' list of real-world uses of DP converts guarantees to $\delta = 10^{-5}$ and, for Apple, to a user-day unit, so its numbers differ from the primary sources on purpose. When you compare your budget with a deployment, convert both to the same unit, neighbouring relation, accounting and δ, or say that you did not.

44.8.5. Example: the Census as a training run

Could the running example's $\varepsilon = 8$ run be described as "less than half the Census budget"? The Census $\varepsilon = 17.14$ is replace-one at $\delta = 10^{-10}$; the run's 7.997 is add/remove at $\delta = 10^{-5}$. At $\delta = 10^{-10}$ the run is at 11.627 (Table 44.10), and replace-one doubles an add/remove guarantee in the worst case. After converting, the honest statement is that the two are of the same order, which is a weaker claim than the slogan.

44.8.6. Check yourself

- A product reports "$\varepsilon = 2$ per user per day" for data kept a year. What total should you assume for a user whose behaviour never changes? *(Up to* $2 \times 365 = 730$ *by basic composition, unless the mechanism memoizes or the accounting shows otherwise.)*
- Why can the Census Bureau's 19.61 be larger than the ε of the combined $\rho = 2.63$? *(Because adding two* ε *values computed at the same* δ *is a valid but loose composition, while zCDP composes by adding* ρ *and converts once.)*

44.9. Justifying a choice

44.9.1. Context

You must pick a target for the running example and write a statement that a sceptical reviewer will check. What should the procedure be, and what does it give here?

44.9.2. Statement

A defensible choice of ε fixes the claim before the number:

1. **The unit and the neighbouring relation.** Record, user, device; add/remove or replace-one; the period over which the budget is spent, including every training run and hyperparameter search that touched the data (Hyperparameter tuning).
2. **The threat.** Membership, attribute inference or reconstruction; the attacker's knowledge and prior; what it observes (the final model, the iterates, an API).
3. **The whole guarantee.** The privacy profile $\varepsilon(\delta)$ or the trade-off curve, computed by a tight accountant, with $\delta \ll 1/n$, and read at the operating points of the threat: TPR at FPR 10^{-3} and below for membership, at FPR κ for reconstruction.
4. **The evidence.** An empirical audit (Auditing private training) whose lower bound is consistent with the claim, and a check of the implementation against the assumptions of the proof (DP-SGD).
5. **The utility.** The accuracy curve against ε, and where it bends: the smallest budget past the bend is the natural candidate.
6. **The comparison.** The tiers of Ponomareva et al. (2023): $\varepsilon \leq 1$ gives strong formal guarantees; $\varepsilon \leq 10$ is the widely used goal they call a reasonable level of anonymization for many applications; above 10, the guarantee alone cannot be taken as sufficient evidence of anonymization and additional measures such as auditing may be necessary. NIST SP 800-226 offers no specific rule for choosing ε, cites a study suggesting that ε of 0.1 generally provides strong protection and values below 1 are reasonable, and warns that large values may not provide meaningful privacy, that releases with ε of 10 or more can leave outliers vulnerable, and that organizations should evaluate the risk of the final data, especially for ε above 1. It also recommends publishing the parameters, which it says creates no additional risk.

44.9.3. Derivation: why each step is needed

1. **Unit.** Without it, the same number can mean protection of a single click or of a person's whole history (Section 44.8).
2. **Threat.** The curve answers different questions at different points: a prior of 0.5 reads the curve near FPR 0.5, a thousand candidates read it at 10^{-3} (Section 44.5, Section 44.6).
3. **Whole guarantee.** The single pair can overstate the attacker by a factor of 17 at FPR 10^{-3} and $\varepsilon = 8$ (Table 44.5), so a statement based on the pair alone either undersells the model or tempts the reader to dismiss DP as vacuous.
4. **Evidence.** The curve is a theorem about an algorithm; whether the code runs that algorithm is an empirical fact, and audits are how bugs that break the proof are found.
5. **Utility.** The accuracy curve is steeply non-linear (Figure 44.5), so the cost of halving ε varies by an order of magnitude along it.
6. **Comparison.** Tiers and practice give a reader a reference point; they cannot replace steps 1 to 5.

44.9.4. Interpretation

The accuracy curve and the risk curve together. Figure 44.5 puts the accuracies of DP-SGD against the largest membership TPR at FPR 1%, from the pair and from the exact curve. Going from $\varepsilon = 1$ to 2 gains 2.66 points of accuracy (0.9175 to 0.9441) and moves the exact TPR at FPR 1% from at most 0.01893 to at most 0.03141. From 2 to 4 gains 1.07 points for a TPR of at most 0.0724. From 4 to 8 gains 0.56 points (0.9548 to 0.9604), the gap to the non-private model falling from 0.0062 to 0.0006, and nearly triples the TPR, to at most 0.2092. Past $\varepsilon = 4$, each unit of accuracy is bought with a much larger increase in worst-case risk.

Rules of thumb are where the reasoning stops, not where it starts. "$\varepsilon \leq 1$ is strong" is true of every mechanism, because Equation 44.4 is already tight enough. "1 to 10 is common in machine learning" is true of practice, and this chapter shows why it can be reasonable for DP-SGD: at those budgets the actual curve is far below the line. "Above 10 needs justification" is true because there even the actual curve allows attacks with substantial power on worst-case records (TPR above 0.27 at FPR 10^{-3} at $\varepsilon = 16$).

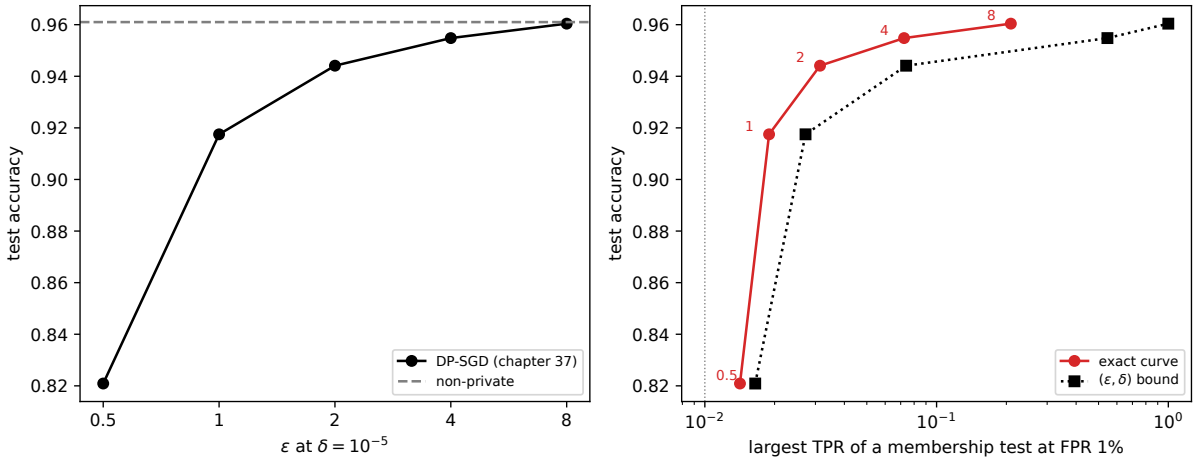


Figure 44.5.: Left: test accuracy of the running example against ε at $\delta = 10^{-5}$, from the DP-SGD chapter's table (three runs each), with the non-private model dashed. Right: the same accuracies against the largest TPR of a membership test at FPR 1%, from the exact curve (circles, labelled by ε) and from the pair alone (squares). The dotted vertical line is an attacker with no information.

44.9.5. Example: a privacy statement for the running example

Following the procedure, a team might choose the $\varepsilon = 4$ run and write:

> The model was trained with DP-SGD (Poisson sampling rate 0.01, noise multiplier 1.36, clipping norm 0.5, 10,000 steps) on 10,000 records, one record per person. The privacy unit is one record under add/remove neighbours; no other run touched the training data. The RDP accountant gives $(3.993, 10^{-5})$-DP and $(5.209, 10^{-8})$-DP; a PLD accountant gives $\varepsilon \leq 3.729$ at $\delta = 10^{-5}$. Against an attacker who knows every other record and sees every iterate, any membership test has TPR at most 0.01312 at

FPR 10^{-3} and at most 0.0724 at FPR 10^{-2}, and any attack that picks a record out of 100 equally likely candidates succeeds with probability at most 0.0724. Test accuracy is 0.9548, against 0.9610 without privacy.

Every number in it is checkable, and each comes with its unit, its accountant and the attacker it refers to. A reviewer who thinks $\varepsilon = 4$ is too large can now argue with the TPR at a specific FPR, which is the substance, rather than with the number.

44.9.6. Check yourself

- Your accuracy curve is flat above $\varepsilon = 2$. Which budget should you choose, and why? *(About 2, or slightly above to leave room for run-to-run variation: past the bend, more budget buys almost no accuracy and raises the worst-case risk.)*
- A reviewer asks why you report the TPR at FPR 10^{-3} rather than the advantage. *(Because an attack is dangerous when its flags are reliable, which is a low-FPR property, and because the advantage is read at one point near the middle of the curve, where it says little about confident identification.)*

44.10. Code: check every number by computation

The script code/choosing_epsilon.py calibrates the running example's noise multipliers with the RDP accountant of DP-SGD, discretizes the one-step privacy loss of the subsampled Gaussian pessimistically and optimistically, composes it 10,000 times by FFT, and reads off the trade-off curve, the privacy loss quantiles and the bounds of this chapter. It also trains one model per budget and runs the loss-threshold attack. From the repository root:

```
uv run python code/part05-dp-ml/choosing_epsilon.py
```

It takes two to three minutes, most of it in the PLD compositions. **Output** (seed 0), summarized in Table 44.12:

Table 44.12.: The script's output with the default seed.

Experiment	Result
Accountants	RDP ε = target; PLD brackets 0.02 to 0.1 wide and below it; GDP ε below the PLD lower end except at $\varepsilon = 0.5$ and 1 (Table 44.2)
Membership at FPR 10^{-3}	Pair 0.002728 / 0.05461 / 0.9997 at $\varepsilon = 1$ / 4 / 8; exact curve at most 0.00225 / 0.01312 / 0.0589
Advantage	Pair 0.9640 at $\varepsilon = 4$; exact at most 0.3429
Bayes	Median total loss 0.401 at $\varepsilon = 4$, 1.136 at $\varepsilon = 8$; $Q[L > \varepsilon]$ between 3.78×10^{-6} and 1.10×10^{-4}
Reconstruction, 1,000 candidates	At most 0.0589 at $\varepsilon = 8$, 0.2808 at $\varepsilon = 16$

Experiment	Result
δ	$\varepsilon = 4$ run: 3.993 at 10^{-5}, 5.209 at 10^{-8}, 5.890 at 10^{-10}
Conversions	$\rho = 2.56$ gives 17.915 at $\delta = 10^{-10}$ by Equation 44.15; RAPPOR $\varepsilon_\infty = 2.043$
Loss-threshold attack	TPR within 0.004 of the FPR at every budget and every FPR (Table 44.4)

tests/test_choosing_epsilon.py checks that the output still matches.

What to take from it. The exact trade-off curve of DP-SGD is cheap to compute, with the same machinery as a numerical accountant, and it changes the conversation: at the budgets machine learning uses, the curve is far below the line that the reported (ε, δ) draws, and it, not the line, is what an attacker faces.

44.11. References

- Dwork & Roth, *The Algorithmic Foundations of Differential Privacy* (Foundations and Trends in Theoretical Computer Science, 2014). The definition and its testing and Bayesian readings.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015), and Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS B, 2022). The testing view of DP, trade-off curves, the duality of Equation 44.6 and the central limit theorem for composition.
- Koskela, Jälkö & Honkela, "Computing Tight Differential Privacy Guarantees Using FFT" (AISTATS 2020), and Gopi, Lee & Wutschitz, "Numerical Composition of Differential Privacy" (NeurIPS 2021). Numerical PLD accounting, which Equation 44.7 reuses for the curve.
- Nasr, Song, Thakurta, Papernot & Carlini, "Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning" (IEEE S&P 2021). Attacks that reach the accountant's guarantee when the adversary controls the target's gradient and sees every iterate.
- Balle, Cherubin & Hayes, "Reconstructing Training Data with Informed Adversaries" (IEEE S&P 2022). Reconstruction robustness and its bounds from pure DP, RDP and zCDP.
- Hayes, Mahloujifar & Balle, "Bounding Training Data Reconstruction in DP-SGD" (NeurIPS 2023). The reconstruction bound from the trade-off curve of DP-SGD, Equation 44.13, and experiments showing it is nearly tight.
- Guo, Karrer, Chaudhuri & van der Maaten, "Bounding Training Data Reconstruction in Private (Deep) Learning" (ICML 2022). Lower bounds on the mean squared error of reconstruction from RDP and Fisher information.
- Near, Darais, Lefkovitz & Howarth, *Guidelines for Evaluating Differential Privacy Guarantees* (NIST SP 800-226, March 2025). Privacy units, $\delta \leq 1/n^2$, the hazards of large ε and of event-level units, and the case for publishing parameters.

- Ponomareva et al., "How to DP-fy ML: A Practical Guide to Machine Learning with Differential Privacy" (JAIR 2023). The three tiers of ε for machine learning and what to report.
- Dwork, Kohli & Mulligan, "Differential Privacy in Practice: Expose your Epsilons!" (Journal of Privacy and Confidentiality, 2019). Interviews with practitioners and the proposal of an Epsilon Registry.
- Desfontaines, "A list of real-world uses of differential privacy" (blog, desfontain.es, updated 2025). A maintained list of deployments, converted to a common δ and unit.
- Abowd et al., "The 2020 Census Disclosure Avoidance System TopDown Algorithm" (Harvard Data Science Review, 2022), and US Census Bureau, "Census Bureau Sets Key Parameters to Protect Privacy in 2020 Census Results" (press release, June 2021). The redistricting data's ρ and ε. Cumings-Menon et al., "Disclosure Avoidance for the 2020 Census Demographic and Housing Characteristics File" (HDSR, 2023). Kazan & Reiter, "Assessing Statistical Disclosure Risk for Differentially Private, Hierarchical Count Data" (arXiv 2022), for the 17.91 conversion.
- Xu, Zhang, Andrew, Choquette-Choo, Kairouz, McMahan, Rosenstock & Zhang, "Federated Learning of Gboard Language Models with Differential Privacy" (ACL 2023 Industry Track); McMahan & Thakurta, "Federated Learning with Formal Differential Privacy Guarantees" (Google AI blog, 2022); Xu & Zhang, "Advances in private training for production on-device language models" (Google Research blog, 2024). The Gboard rows.
- Apple Differential Privacy Team, "Learning with Privacy at Scale" (Apple Machine Learning Journal, 2017), and Apple, "Differential Privacy Overview" (2017). Tang, Korolova, Bai, Wang & Wang, "Privacy Loss in Apple's Implementation of Differential Privacy on MacOS 10.12" (arXiv 2017). The Apple rows.
- Erlingsson, Pihur & Korolova, "RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response" (CCS 2014). Ding, Kulkarni & Yekhanin, "Collecting Telemetry Data Privately" (NeurIPS 2017). Rogers et al., "LinkedIn's Audience Engagements API: A Privacy Preserving Data Analytics System at Scale" (Journal of Privacy and Confidentiality, 2021). Messing et al., "Facebook Privacy-Protected Full URLs Data Set" (Harvard Dataverse, 2020). The other rows of Table 44.11.
- Bun & Steinke, "Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds" (TCC 2016). The conversion Equation 44.15.

44.12. Cheat sheet

Q, P: output laws with and without the target; $L = \ln(Q/P)$ its privacy loss; ω the law of L under Q; $\delta(\varepsilon)$ the privacy profile; p the attacker's prior; κ the reconstruction baseline. Table 44.13 collects the results.

Table 44.13.: Every result of the chapter.

Result	Formula	Conditions
One pair	$\mathrm{TPR} \le e^{\varepsilon}\mathrm{FPR} + \delta$; advantage $\le \frac{e^{\varepsilon}-1+2\delta}{e^{\varepsilon}+1}$	Any (ε, δ)-DP mechanism
Whole curve	$\mathrm{TPR}^*(\alpha) = \inf_{\varepsilon}(e^{\varepsilon}\alpha + \delta(\varepsilon))$	Exact, from the full profile

Result	Formula	Conditions
Curve from the PLD	$(\mathbb{E}_\omega[e^{-L}; L > t], \omega(L > t))$	Composition: convolve per-step PLDs
From RDP	$\text{TPR} \leq \min_\alpha(\text{FPR}\, e^{R(\alpha)})^{(\alpha-1)/\alpha}$	Valid bound, cheap
GDP	$\text{TPR} \approx 1 - \Phi(\Phi^{-1}(1-\alpha) - \mu)$, $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}$	Approximation, not a bound
Posterior	$\leq pe^\varepsilon/(pe^\varepsilon + 1 - p)$; exactly $pe^L/(pe^L + 1 - p)$	Fails with probability $\leq \delta/(1 - e^{\varepsilon - \varepsilon'})$ at level ε'
Precision of a flag	$p\,\text{TPR}/(p\,\text{TPR} + (1-p)\text{FPR})$	Base rates dominate at small p
Reconstruction	Success $\leq \text{TPR}^*(\kappa)$; pair: $e^\varepsilon\kappa + \delta$	Informed adversary, prior baseline κ
δ	Release mechanism exposes $1 - (1-\delta)^n$; use $\delta \ll 1/n$, NIST: $\leq 1/n^2$	Per privacy unit
zCDP to (ε, δ)	$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}$	Simple, slightly loose
Running example, $\varepsilon = 4$	$\text{TPR} \leq 0.01312$ at FPR 10^{-3} (pair: 0.05461); accuracy 0.9548	$q = 0.01$, $\sigma = 1.36$, $T = 10{,}000$
Running example, $\varepsilon = 8$	$\text{TPR} \leq 0.0589$ at FPR 10^{-3} (pair: 0.9997); accuracy 0.9604	$\sigma = 0.917$

Rules of thumb

- Report the curve, not just the pair: the TPR at FPR 10^{-3} and 10^{-2}, computed with a PLD accountant or bounded with RDP.
- State the unit, the neighbouring relation, the period and δ with every ε; convert before comparing with anyone else's.
- $\varepsilon \leq 1$ is strong for any mechanism. Between 1 and 10, DP-SGD's actual curve is far below the pair's line, and that curve is your justification. Above 10, show the curve and an audit, and expect worst-case records to be exposed.
- Choose δ well below $1/n$ for the right n; for Gaussian-type mechanisms a smaller δ costs little ε.
- Put the accuracy curve next to the risk curve, and pick the smallest budget past the bend in accuracy.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

45. Auditing private training

45.1. Why this matters

Every chapter of this part ended with a number: train with stochastic gradient descent (SGD) under differential privacy (DP), that is, with per-example clipping and Gaussian noise (DP-SGD), at sampling rate $q = 0.01$, noise multiplier $\sigma = 1$ and 10,000 steps, and the accountant reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$. That number is the conclusion of a proof. The proof is about an algorithm on paper, and what ships is code. Between the two sit a dozen places where an implementation can quietly differ from the algorithm the proof analysed: noise scaled for the averaged gradient but added to the sum, the wrong dataset size handed to the accountant, batches sampled one way and accounted another, a random seed reused. None of these changes the accuracy much, so none is caught by the usual tests. **Is the trained model as private as the accountant says?**

An **audit** answers this the only way an empirical check can: it attacks. It plants records of its own, **canaries**, trains with and without them, runs a membership test, and counts how often the test is right. The definition of DP limits how often any test can be right, so the measured error rates imply a **lower bound** on ε: the algorithm cannot be private with any smaller ε. A lower bound cannot prove privacy. It can disprove a claim, when it lands above the claimed value, and it says how much of the accountant's worst case a given attacker can actually realize.

45.1.1. The one idea: privacy caps every test, so a measured test bounds privacy from below

An (ε, δ)-DP algorithm guarantees that every test, for every pair of neighbouring datasets, has a true positive rate (TPR) of at most e^{ε} times its false positive rate (FPR) plus δ. Read backwards, a test with a measured FPR and TPR forces $\varepsilon \geq \ln \frac{\mathrm{TPR}-\delta}{\mathrm{FPR}}$. Everything in this chapter is about making that inequality bite. The rates are estimated from a finite number of trials, so they come with confidence intervals, and the bound must use the pessimistic end of each; this caps what any audit with n trials can certify at about $\ln n$. The test is only as strong as the attack, so the auditor chooses the most detectable canary and the most informative view of training; with the right canary and a view of every step, the audit faces exactly the pair of distributions the accountant analyses. And training thousands of models is unaffordable, so two tricks get many trials out of one or two runs: many canaries in one run, or one observation per step.

Table 45.1 lists the results and where each is used.

Table 45.1.: The results of this chapter and where each is used.

Result	Where it is used
A test's FPR and TPR, with Clopper–Pearson intervals, give an ε lower bound	Every audit; reading audit results in papers
The ceiling: n trials certify at most about $\ln(n/\ln(2/\beta))$	Planning an audit; why audits of large ε are expensive
A Dirac canary seen at every step faces exactly the accountant's pair	Strongest audits of DP-SGD; testing implementations
The likelihood-ratio statistic, and what the sum of projections loses	Choosing the attack's score
Where ε at $\delta = 10^{-5}$ lives on the trade-off curve	Why audits stop short of the accountant
One-run auditing: correct guesses are dominated by a binomial	Auditing real training runs
Black-box audits with input canaries	What a realistic attacker learns from a released model
Per-step and end-to-end audits find bugs	Testing DP-SGD implementations before release

Notation. The training algorithm is M, the dataset without the canary D and with it $D' = D \cup \{c\}$; neighbours differ by adding or removing one record. A **trial** is one run of M on D or on D', with fresh randomness, followed by the attack's score s. The attack says "canary in" when $s \geq \tau$, a threshold. Its **false positive rate** is $\mathrm{FPR} = \Pr[s \geq \tau \mid D]$ and its **true positive rate** $\mathrm{TPR} = \Pr[s \geq \tau \mid D']$; the false negative rate (FNR) is $1 - \mathrm{TPR}$. With n trials of each kind, the counts are k_0 false positives and k_1 true positives. β is the probability that an audit's bound is wrong (5% throughout), and $\varepsilon_{\mathrm{lo}}$ is the bound itself. DP-SGD's notation is that of DP-SGD: clipping norm C, noise multiplier σ, sampling rate q, expected batch size B (the sampling rate times the number of records, 100 here), step size η and T steps. In this chapter n always counts trials, not records.

The running example. The part's network: a two-layer tanh network trained on the 10,000 records of DP-SGD with $q = 0.01$, $C = 0.5$, $\sigma = 1$, step size 0.5 and $T = 10{,}000$ steps. The Rényi differential privacy (RDP) accountant reports $\varepsilon = 6.71$ at $\delta = 10^{-5}$; this is the **claim** every audit below tests.

How to read this chapter. Read the toolkit and Section 45.3, which turns an attack into a bound and says what it costs. Section 45.4 chooses the attack, Section 45.5 runs the strongest one many times and explains why even it stops short of 6.71. Section 45.6 and Section 45.7 audit a real training run in one go, and Section 45.8 uses audits to catch broken implementations. This chapter comes last in the part because it checks the claims of all the others.

45.2. Toolkit: four facts every derivation uses

45.2.1. Fact 1 — Differential privacy caps every test, in both directions

M is (ε, δ)-DP when, for all neighbours D, D' and every set of outputs S, $\Pr[M(D') \in S] \leq e^{\varepsilon} \Pr[M(D) \in S] + \delta$, and the same with D and D' swapped. A test that says "canary in" on the event S has $\mathrm{TPR} = \Pr[M(D') \in S]$ and $\mathrm{FPR} = \Pr[M(D) \in S]$, so the definition reads $\mathrm{TPR} \leq e^{\varepsilon}\,\mathrm{FPR} + \delta$. Applying the definition with the roles swapped to the complement of S, where the test says "canary out", gives $1 - \mathrm{FPR} \leq e^{\varepsilon}(1 - \mathrm{TPR}) + \delta$: a test cannot be almost always right about absent canaries while rarely missing present ones. Together,

$$\mathrm{TPR} \leq e^{\varepsilon}\,\mathrm{FPR} + \delta, \qquad 1 - \mathrm{FPR} \leq e^{\varepsilon}\,(1 - \mathrm{TPR}) + \delta. \tag{45.1}$$

These are the two lines of the hypothesis-testing interpretation of DP (The definition of differential privacy); they hold for randomized tests too.

45.2.2. Fact 2 — Clopper–Pearson bounds on a rate

If k of n independent trials succeed, each with unknown probability p, then k is binomial, $\mathrm{Bin}(n, p)$. The **one-sided Clopper–Pearson upper bound** at level a is the rate u at which seeing k or fewer successes has probability exactly a, $\Pr[\mathrm{Bin}(n, u) \leq k] = a$; the **lower bound** l solves $\Pr[\mathrm{Bin}(n, l) \geq k] = a$. Then $\Pr[p \leq u] \geq 1 - a$ and $\Pr[p \geq l] \geq 1 - a$, for every true p. The reason is that a true rate above u would make the observed count implausibly small, and the probability of an observation that implausible is at most a (Hypothesis testing proves it). With $k = 10$ and $n = 10{,}000$, the bounds at $a = 0.025$ are 0.00048 and 0.00184, the two ends of the usual two-sided 95% interval. With $k = 0$ and $n = 3{,}000$, the 95% upper bound is 9.9808×10^{-4}, about $3/n$: the **rule of three**.

45.2.3. Fact 3 — The best test thresholds the likelihood ratio

Among all tests with a given FPR, the one with the largest TPR says "canary in" when the likelihood ratio, the density of the observation under D' divided by its density under D, exceeds a threshold. This is the Neyman–Pearson lemma (Hypothesis testing). The reason is a budgeting argument: each observation costs its probability under D in false positives and earns its probability under D' in true positives, and the best way to spend a fixed false-positive budget is on the observations with the best ratio of earnings to cost. The curve of the best TPR against FPR is the pair's **trade-off curve** (written as best FNR against FPR in f-DP and Gaussian DP). For two unit-variance normals at distance μ, the best TPR at FPR a is

$$\mathrm{TPR}_{\mu}(a) = 1 - \Phi\big(\Phi^{-1}(1 - a) - \mu\big), \tag{45.2}$$

where Φ is the standard normal distribution function; a mechanism whose curve is this one, or better, satisfies Gaussian differential privacy (GDP) with parameter μ, or **μ-GDP**.

45.2.4. Fact 4 — One DP-SGD step, seen along one direction

A DP-SGD step releases $G_t = \sum_{i \in S_t} \bar{g}_i + \xi_t$, the sum of the clipped gradients of the records in the batch S_t plus Gaussian noise $\xi_t \sim \mathcal{N}(0, \sigma^2 C^2 I)$. Project G_t on a fixed unit vector u and divide by C. The noise becomes σz_t with z_t standard normal, because the projection of an isotropic Gaussian vector on a unit vector is a one-dimensional Gaussian with the same standard deviation. A record whose clipped gradient is exactly Cu adds 1 when it is sampled, which under Poisson sampling happens with probability q, independently at every step. So if no other record's gradient has a component along u, the projection is

$$x_t \sim \mathcal{N}(0, \sigma^2) \text{ without the record}, \qquad x_t \sim (1-q)\,\mathcal{N}(0, \sigma^2) + q\,\mathcal{N}(1, \sigma^2) \text{ with it}, \tag{45.3}$$

independently over steps, because both the sampling and the noise are drawn afresh at every step. This is the pair of distributions the accountant of Amplification by subsampling analyses: the worst case for one step under add/remove neighbours.

45.3. From an attack to a lower bound on ε

45.3.1. Context

You ran an attack: n training runs without the canary and n with it, a score per run, a threshold. You counted k_0 false positives and k_1 true positives. What can you conclude about ε, with what confidence, and how much could you possibly conclude with n trials?

45.3.2. Statement

The measured rates are k_0/n and k_1/n. Write $\bar{\alpha}$ and $\underline{\tau}$ for the one-sided Clopper–Pearson upper bound on the FPR and lower bound on the TPR, each at level $\beta/2$ (Fact 2). If the threshold was chosen without looking at these $2n$ trials, then with probability at least $1-\beta$ the algorithm is not (ε, δ)-DP for any ε below

$$\varepsilon_{\text{lo}} = \max\left\{ \ln \frac{\underline{\tau} - \delta}{\bar{\alpha}},\ \ln \frac{1 - \bar{\alpha} - \delta}{1 - \underline{\tau}} \right\}. \tag{45.4}$$

The most that n trials per hypothesis can certify is the bound of a perfect attack, $k_0 = 0$ and $k_1 = n$:

$$\varepsilon_{\max}(n) = \ln \frac{(\beta/2)^{1/n} - \delta}{1 - (\beta/2)^{1/n}} \approx \ln \frac{n}{\ln(2/\beta)}. \tag{45.5}$$

45.3.3. Derivation

1. **The rates bound ε.** Solve the two inequalities of Equation 45.1 for ε: $e^\varepsilon \geq (\text{TPR} - \delta)/\text{FPR}$ and $e^\varepsilon \geq (1 - \text{FPR} - \delta)/(1 - \text{TPR})$. Any (ε, δ)-DP algorithm satisfies both, at the true rates of every test.
2. **Both expressions grow with the TPR and shrink with the FPR.** So replacing the true FPR by something larger and the true TPR by something smaller can only lower them.

3. **The intervals hold together with probability** $1-\beta$**.** By Fact 2, FPR $\le \bar{\alpha}$ fails with probability at most $\beta/2$, and TPR $\ge \underline{\tau}$ fails with probability at most $\beta/2$. The probability that at least one of them fails is at most the sum, β (the union bound). This needs the counts to be binomial: independent trials, and a threshold fixed before the counting.
4. **Combine.** On the event that both intervals hold, step 2 gives $\varepsilon_{\text{lo}} \le \max\{\ln \frac{\text{TPR}-\delta}{\text{FPR}}, \ln \frac{1-\text{FPR}-\delta}{1-\text{TPR}}\}$, and step 1 says this is at most the ε of any valid guarantee. So the event "M is (ε,δ)-DP with $\varepsilon < \varepsilon_{\text{lo}}$" has probability at most β.
5. **The ceiling.** With $k_0 = 0$, $\Pr[\text{Bin}(n,u) \le 0] = (1-u)^n = \beta/2$ gives $\bar{\alpha} = 1-(\beta/2)^{1/n}$. With $k_1 = n$, $\Pr[\text{Bin}(n,l) \ge n] = l^n = \beta/2$ gives $\underline{\tau} = (\beta/2)^{1/n}$. Substituting in the first form of Equation 45.4 gives Equation 45.5; the second form gives the same value here. For large n, $(\beta/2)^{1/n} = e^{-\ln(2/\beta)/n} \approx 1 - \ln(2/\beta)/n$, so $\bar{\alpha} \approx \ln(2/\beta)/n$, $\underline{\tau} \approx 1$ and $\varepsilon_{\max} \approx \ln(n/\ln(2/\beta))$; with $\beta = 0.05$, $\ln(2/\beta) = \ln 40 = 3.69$.

45.3.4. Interpretation

A lower bound, and only that. ε_{lo} says "no smaller ε is possible", never "this ε is enough". A weak attack gives a small ε_{lo} for a leaky model as easily as for a private one. That asymmetry is the whole logic of auditing: a bound above the claim refutes the claim, and a bound below it only says that this attacker, on this canary, could not do better.

The ceiling grows like $\ln n$**.** Each factor of e in the bound costs a factor of e in trials, because a false positive rate of α cannot be distinguished from zero with fewer than about $3/\alpha$ trials (the rule of three), and ε is about $\ln(1/\alpha)$ when the TPR is near 1. Table 45.2 shows what that means. Certifying the running example's 6.71 needs at least 3,038 trials per hypothesis, so 6,076 training runs, even for an attack that never errs. Each run of the running example takes a few seconds; for a real network each is a full training job.

Table 45.2.: The most a perfect attack can certify with n trials per hypothesis, at 95% confidence and $\delta = 10^{-5}$. Conversely, certifying $\varepsilon = 1$, 2, 4, 6.71 and 8 takes at least 12, 30, 204, 3,038 and 10,999 trials per hypothesis.

Trials per hypothesis n	100	1,000	10,000	100,000	1,000,000
Largest certifiable ε, Equation 45.5	3.28	5.60	7.90	10.21	12.51

Choose the threshold on separate trials. Step 3 assumes the threshold does not depend on the counts. Sweeping the threshold over the same data and reporting the best bound is the norm in parts of the literature, and it makes the confidence level wrong: among many thresholds, one is lucky. Every audit in this chapter chooses its threshold on half of the trials and counts on the other half.

Both inequalities matter. The first form of Equation 45.4 rewards tests that flag few absent canaries; the second rewards tests that rarely miss present ones. A test that says "canary in" almost always, with FPR 0.5 and TPR 0.999, has a useless first form, $\ln(0.999/0.5) = 0.69$, and a second form of $\ln(0.5/0.001) = 6.21$. An audit should try both (Exercise 1 in the exercise book).

45.3.5. Example: what confidence costs

Suppose 1,000 runs with the canary and 1,000 without give 500 true positives and 1 false positive. The point estimate is $\ln \frac{0.5-10^{-5}}{0.001} = 6.21$. The Clopper–Pearson bounds at $\beta/2 = 0.025$ are $\bar{\alpha} = 0.00556$ and $\underline{\tau} = 0.4685$: one false positive in 1,000 is consistent with a true FPR five times larger. The certified bound is $\ln \frac{0.4685-10^{-5}}{0.00556} = 4.43$, almost two less than the point estimate. With ten times the trials and the same rates, 10 false positives and 5,000 true positives out of 10,000 each, $\bar{\alpha} = 0.00184$, $\underline{\tau} = 0.4902$, and the bound rises to 5.59. The point estimate does not change; the price of confidence falls, slowly, as the counts grow.

45.3.6. Check yourself

- An audit with 500 trials per hypothesis reports $\varepsilon_{\text{lo}} = 7$ at 95% confidence. What went wrong? *(500 trials can certify at most about* $\ln(500/3.69) = 4.9$*, by Equation 45.5. The threshold was probably chosen on the same trials, or the trials were not independent.)*
- Why does the bound use the upper end of the FPR interval and the lower end of the TPR interval? *(Because the bound must hold whatever the true rates are within their intervals, and* $\ln \frac{\text{TPR}-\delta}{\text{FPR}}$ *is smallest when the FPR is as large and the TPR as small as the data allow.)*

45.4. Canaries and threat models

45.4.1. Context

DP is a promise about the worst pair of neighbouring datasets and the worst test. An audit must instantiate both: it picks the dataset, the canary record and what the attacker sees, then the score. A typical training record and a loss threshold give a valid but nearly useless bound (Membership inference shows that most records are barely exposed). What is the strongest audit of DP-SGD, and what does it face?

45.4.2. Statement

The auditor has two choices of canary and two threat models.

An **input-space canary** is a record (x, y), inserted into the training data like any other: a mislabeled example, or one far from the data, so that its gradient is large and points where few others do. A **gradient-space canary** lets the auditor write the canary's gradient directly, as if it controlled the training code; the strongest is a **Dirac canary**, a gradient equal to C on one coordinate and zero elsewhere (Nasr et al. 2023), on a coordinate where the other records' gradients are zero. In the **black-box** threat model the attacker sees only the final model; in the **white-box** model it sees every iterate $w_0, \dots, w_T$, and therefore every noisy sum $G_t = -B(w_{t+1} - w_t)/\eta$.

For a Dirac canary seen white-box, the audit faces the accountant's pair exactly. The projections $x_t = G_{t,j}/C$ on the canary's coordinate j follow Equation 45.3, independently over

the T steps. The most powerful test at every FPR thresholds the **likelihood-ratio statistic**

$$\Lambda = \sum_{t=1}^{T} \ln\left(1 - q + q\, e^{(x_t - 1/2)/\sigma^2}\right), \tag{45.6}$$

and by the central limit theorem (CLT) its trade-off curve is close to that of μ-GDP with $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}$. The simpler **sum of projections** $\sum_t x_t$, which is the canary coordinate of the total displacement $w_0 - w_T$ times $B/(\eta C)$, has a curve close to μ-GDP with $\mu_{\text{sum}} = q\sqrt{T}/\sigma$.

45.4.3. Derivation

1. **The canary is the only difference.** With a Dirac canary on a coordinate j no other record's gradient touches, every other record contributes 0 to $G_{t,j}$. The canary's clipped gradient is itself, since its norm is exactly C, and it contributes C when sampled. Dividing by C gives Fact 4: $x_t = \sigma z_t$ without the canary and $\sigma z_t + b_t$ with it, $b_t \sim \text{Bernoulli}(q)$.
2. **The steps are independent.** The sampling coin b_t and the noise z_t are fresh at each step, and the canary's gradient is the same at every step whatever the weights are. So $(x_1, \ldots, x_T)$ are independent and identically distributed under each hypothesis, even though training is adaptive.
3. **The likelihood ratio of one step.** The density of $\mathcal{N}(1, \sigma^2)$ divided by that of $\mathcal{N}(0, \sigma^2)$ at x is $e^{(2x-1)/(2\sigma^2)} = e^{(x-1/2)/\sigma^2}$, because the squares x^2 cancel in the ratio of the two exponentials. The mixture's density over the null's is therefore $(1 - q) + q\, e^{(x-1/2)/\sigma^2}$.
4. **Multiply over steps.** Independence (step 2) makes the joint likelihood ratio a product, and its logarithm the sum Equation 45.6. By Fact 3, thresholding Λ is the most powerful test at every FPR. No attacker, whatever it sees, can do better against this canary: the whole training run is a function of the projections x_t and of randomness that has the same distribution under both hypotheses (the other records' sampling and the noise on the other coordinates), so it carries no evidence beyond the x_t.
5. **The two statistics in Gaussian terms.** The CLT applied to the per-step privacy loss gives $\mu = q\sqrt{T(e^{1/\sigma^2} - 1)}$ for Λ (f-DP and Gaussian DP derives it from the mean and variance of $\ln(1 + qW)$, with $W = e^{(x-1/2)/\sigma^2} - 1$). For the sum, the shift between the hypotheses is the expected number of times the canary is sampled, qT, and the noise standard deviation is $\sigma\sqrt{T}$; their ratio is $q\sqrt{T}/\sigma$. (With the canary the sum has a little extra variance, $Tq(1 - q)$, from the random number of samples.)

45.4.4. Interpretation

Why the likelihood ratio beats the sum. The sum treats every step alike. The likelihood ratio does not: a step with $x_t = 3$ is much stronger evidence of the canary than three steps with $x_t = 1$, because a value three noise standard deviations out is very unlikely without the canary. The exponential in Equation 45.6 weighs it accordingly. The two coincide when σ is large, since then the exponential is nearly linear, and differ most at small σ; the ratio of their μ is $\sigma\sqrt{e^{1/\sigma^2} - 1}$, 1.311 at $\sigma = 1$ (Exercise 4 in the exercise book).

Why black-box audits are weaker. Three things separate a black-box audit with an input canary from the white-box Dirac audit. The final model is one function of all the noisy sums, so it carries at most as much information as all of them (post-processing), and usually much less:

the noise of 10,000 steps is summed into the weights, and later steps partly overwrite earlier ones. An input canary's gradient is not of norm C at every step: once the model fits it, or gives up on it, its gradient shrinks or turns. And it shares directions with the other records' gradients, whose contributions act as extra noise. Nasr, Song, Thakurta, Papernot & Carlini (2021) built a ladder of such adversaries for DP-SGD and found the lower bound tight for the strongest, which sees every iterate and chooses a worst-case dataset and canary, and much weaker as the adversary was restricted towards a realistic one. Jagielski, Ullman & Oprea (2020), an early audit of DP-SGD, used black-box poisoning canaries crafted to have large gradients in a direction where the other data's gradients are small (their ClipBKD), with Clopper–Pearson intervals; they report bounds about ten times better than standard membership inference and within about a factor of ten of the theoretical ε.

Which threat model to audit. They answer different questions. A white-box gradient audit tests the *mechanism*: it asks whether the code adds the noise and clips the gradients the proof assumes, and its bound should match the accountant. A black-box input audit tests the *deployment*: it measures what an attacker with a released model and a planted record can do. A gap between the black-box bound and the accountant is not a bug; it is a measure of how far this attacker is from the proof's worst case.

45.4.5. Example: the running example's two statistics

At $q = 0.01$, $\sigma = 1$ and $T = 10{,}000$, the likelihood-ratio statistic has $\mu = 0.01 \times \sqrt{10{,}000 \times 1.7183} = 1.3108$, and the sum of projections $\mu_{\text{sum}} = 0.01 \times 100 = 1.00$ (0.9951 with the extra variance). Read through the Gaussian privacy profile, the conversion from μ to ε at a given δ (f-DP and Gaussian DP), these are $\varepsilon = 6.01$ and 4.35 at $\delta = 10^{-5}$. Choosing the sum lowers μ from 1.31 to 1.00, and the ε the audit can ever reach from 6.01 to 4.35, before a single trial is run.

45.4.6. Check yourself

- Why must the Dirac canary sit on a coordinate the other records do not touch? *(Otherwise their gradients add an unknown amount to x_t, which acts as extra noise and as a bias that differs from step to step; the audit no longer faces Equation 45.3 but a harder problem, and its bound drops.)*
- An attacker sees only w_0 and w_T of a run with a Dirac canary. Which statistic can it compute, and what does it lose? *(The total displacement on the canary coordinate, which is the sum of projections; it loses the per-step values, so it cannot weigh the large steps more, and its μ falls from 1.31 to 1.00.)*

45.5. Tight audits of the mechanism

45.5.1. Context

The white-box Dirac audit faces the accountant's own pair. With enough trials, does it recover the accountant's 6.71? With the trials an auditor can afford, how close does it get?

45.5.2. Statement

For any pair of distributions P (without the canary) and Q (with it), the smallest ε for which $Q(S) \le e^{\varepsilon} P(S) + \delta$ holds for every event S is

$$\varepsilon^*(\delta) = \sup_S \ \ln \frac{Q(S) - \delta}{P(S)}, \tag{45.7}$$

the supremum over events with $Q(S) > \delta$, and the supremum may be taken over likelihood-ratio thresholds only. For a μ-GDP curve the supremum is reached at the threshold whose false positive rate is

$$a^* = \Phi\Big(-\frac{\varepsilon}{\mu} - \frac{\mu}{2}\Big), \qquad \mathrm{TPR}^* = \Phi\Big(-\frac{\varepsilon}{\mu} + \frac{\mu}{2}\Big). \tag{45.8}$$

For the running example, $\mu = 1.3108$ and $\varepsilon = 6.01$ give $a^* = 8.1 \times 10^{-8}$ and $\mathrm{TPR}^* = 4.3 \times 10^{-5}$: certifying the full ε at $\delta = 10^{-5}$ needs to measure a false positive rate below one in ten million.

45.5.3. Derivation

1. **Equation 45.7 is the definition, solved for ε.** If ε is at least the supremum, then $\ln \frac{Q(S)-\delta}{P(S)} \le \varepsilon$ for every S, which rearranges to $Q(S) \le e^{\varepsilon} P(S) + \delta$. If ε is below it, some event has $\ln \frac{Q(S)-\delta}{P(S)} > \varepsilon$ and violates the inequality.
2. **Only likelihood-ratio thresholds matter.** For a fixed $P(S) = a$, the ratio grows with $Q(S)$, and by Fact 3 the event with the largest $Q(S)$ among those with $P(S) = a$ is a likelihood-ratio threshold.
3. **So a perfect audit is exact.** An audit that thresholds Λ (Equation 45.6) and measures FPR and TPR without error at every threshold computes Equation 45.7, the true $\varepsilon^*(\delta)$ of the canary's pair. For the Dirac canary this is the pair the accountant bounds, so the audit converges to DP-SGD's true ε in this direction of neighbours.
4. **The touching point.** For a μ-GDP curve, the ratio $\frac{\mathrm{TPR}-\delta}{\mathrm{FPR}}$ is largest where the line $\mathrm{TPR} = e^{\varepsilon}\,\mathrm{FPR} + \delta$ touches the curve, that is where the curve's slope is e^{ε}. The slope of a trade-off curve at a threshold is the likelihood ratio there (Fact 3's budgeting argument at the margin), which for two normals at distance μ with threshold t is $e^{\mu t - \mu^2/2}$. Setting it to e^{ε} gives $t = \varepsilon/\mu + \mu/2$, whose FPR is $1 - \Phi(t) = \Phi(-t)$ and TPR $1 - \Phi(t - \mu)$: Equation 45.8.

45.5.4. Interpretation

The accountant's number is not the audit's target. The audit converges to the true ε^*, not to what an accountant reports. The RDP accountant's 6.71 is an upper bound, and a loose one: the numerical accountant of f-DP and Gaussian DP puts the true value of the running example between 6.14 and 6.24, and the CLT approximation gives 6.01. Even an audit with infinitely many trials stops about half a unit below 6.71. That gap belongs to the accountant, not to the audit.

ε at small δ lives in the extreme tail. Equation 45.8 is the main reason audits fall short. With n trials you can measure false positive rates down to about $3/n$; below that, the threshold admits no false positive and its rate is only bounded by the rule of three. At the rates you can measure, the curve's ratio TPR/FPR is far below e^{ε}. For $\mu = 1.3108$ it is $\ln(\mathrm{TPR}/\mathrm{FPR}) = 2.74$ at FPR 10^{-2}, 3.63 at 10^{-3} and 4.38 at 10^{-4}, by Equation 45.2. Reaching 6 needs FPR near 10^{-7},

and about 1.2×10^8 trials without the canary to see ten false positives there (Exercise 3 in the exercise book). Figure 45.1 shows both sides: the measured trade-off curve sits well inside the $(6.71, 10^{-5})$ limit, and the ε each threshold certifies keeps rising until FPR 10^{-7}.

Assume the shape, and extrapolate. Nasr, Hayes, Steinke, Balle, Tramèr, Jagielski, Carlini & Terzis (2023) avoid the tail by assuming the trade-off curve has a known shape. If the mechanism is μ-GDP for some μ, a single test bounds μ from below: by Equation 45.2, $\mathrm{TPR} \le \mathrm{TPR}_\mu(\mathrm{FPR})$ rearranges to

$$\mu \ge \Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR}), \tag{45.9}$$

and a lower bound on μ converts to a lower bound on ε through the Gaussian privacy profile. The rates can be measured at a moderate FPR, where trials are cheap, and the shape carries the bound into the tail. The statement "M is not μ-GDP for any μ below this" is rigorous; the conversion to ε is valid only if the curve really is Gaussian. For the subsampled Gaussian of DP-SGD it is not exactly: its tail is heavier, and the μ that Equation 45.9 gives depends on where you read it. In the example below, the measured rates give 1.300 at FPR 10^{-1}, 1.303 at 10^{-2}, 1.371 at 10^{-3} and 1.492 at 2×10^{-4}, against the CLT's 1.3108. Their audits mostly insert the canary at every step, so that each step is an exact Gaussian mechanism, and compose the per-step result with an accountant; Section 45.8 does the same.

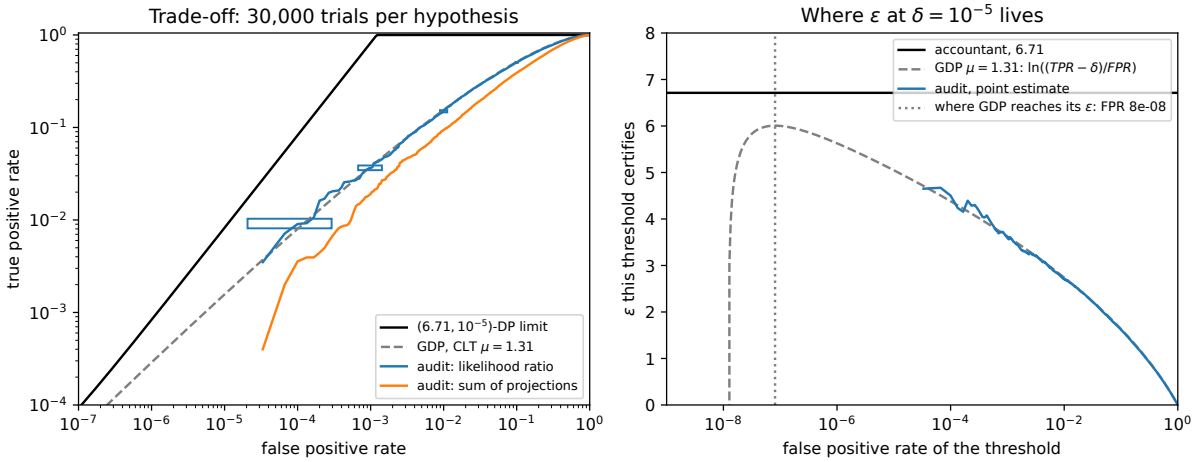


Figure 45.1.: Left: the trade-off of the white-box audit of the running example on log axes, from 30,000 trials per hypothesis. The black line is the most any test can achieve under $(6.71, 10^{-5})$-DP; the dashed grey curve is μ-GDP with the CLT's $\mu = 1.31$. The blue curve is the likelihood-ratio statistic, the orange one the sum of projections; boxes are 95% Clopper–Pearson regions at four thresholds. Right: the ε a threshold would certify if its rates were known exactly, $\ln((\mathrm{TPR} - \delta)/\mathrm{FPR})$, against its false positive rate. The Gaussian curve reaches its ε of 6.01 only at FPR 8×10^{-8} (dotted line), far below the rates a feasible audit can measure.

45.5.5. Example: auditing the running example's mechanism

The script simulates the white-box Dirac audit directly from Equation 45.3: each trial draws 10,000 projections, adds the canary on a $\mathrm{Bin}(10{,}000, 0.01)$ number of steps when it is present,

and computes Λ. This is statistically identical to training the network with a Dirac canary on an untouched coordinate, and thousands of times cheaper.

With 50,000 trials per hypothesis, the measured curve is heavier-tailed than the Gaussian approximation. At the threshold that lets through 500 false positives (FPR 10^{-2}) the TPR is 0.1530 and $\ln((\text{TPR} - \delta)/\text{FPR}) = 2.73$, as the Gaussian predicts; at 10 false positives (FPR 2.0×10^{-4}) the TPR is 0.0203 and the point value 4.62, and at 5 false positives (FPR 10^{-4}) it is 4.84, above the Gaussian's 4.38. These are point estimates, without confidence intervals. Table 45.3 gives the certified bounds, each with the threshold chosen on the first half of the trials and the counts taken on the second.

Table 45.3.: The white-box Dirac audit of the running example with the likelihood-ratio statistic, against the accountant's $\varepsilon = 6.71$. All bounds hold with probability 95%.

Trials per hypothesis	1,000	3,000	10,000	50,000
ε_{lo}, Equation 45.4	1.64	1.88	2.84	3.63
μ_{lo}, Equation 45.9 with Clopper–Pearson	0.795	1.064	1.147	1.185
ε of μ_{lo}-GDP (assumes the Gaussian shape)	3.36	4.70	5.13	5.33
Ceiling, Equation 45.5, at half the trials	4.91	6.01	7.21	8.82

The rigorous bound reaches 3.63 with 100,000 simulated training runs, about half the claim. With 10,000 runs per hypothesis its threshold lets through 6 false positives among 5,000 and catches 254 of 5,000 canaries; with 50,000 it lets through 3 among 25,000 and catches 368. The Gaussian-shape estimate climbs faster, to 5.33, and approaches the CLT's 6.01 from below. The sum of projections needs far more trials for less: its rigorous bound is 1.97 with 10,000 trials per hypothesis and 3.26 with 100,000. With 1,000,000 it is 2.27 in this run, lower than with 100,000, because a threshold chosen on 500,000 trials to let through a handful of false positives let through more on the other half. Bounds read in the far tail are noisy, and Figure 45.2 shows the same scatter. Every number in Table 45.3 is below the claim, as it must be for a correct implementation.

45.5.6. Check yourself

- A white-box audit with unlimited trials converges to 6.2, not 6.71. Is the implementation broken? *(No. The audit converges to the true $\varepsilon^*(\delta)$ of the pair, which the numerical accountant puts between 6.14 and 6.24; the RDP accountant's 6.71 is an upper bound that is loose by about half a unit.)*

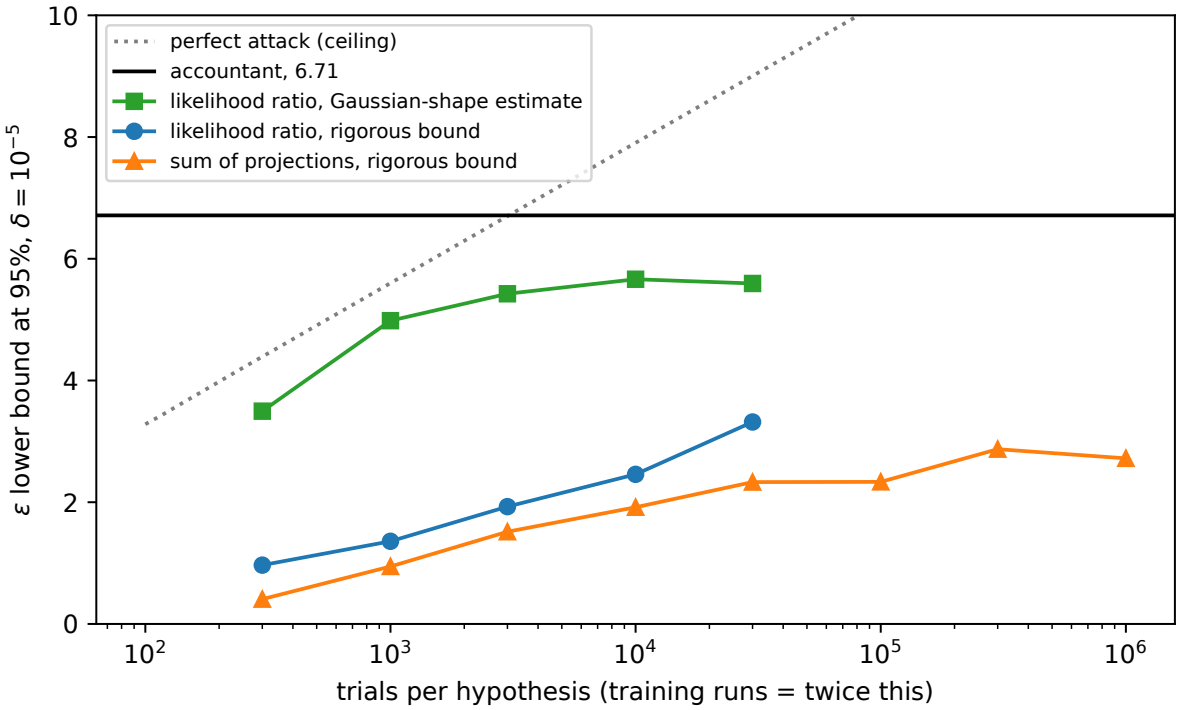


Figure 45.2.: The certified ε lower bound of the white-box audit of the running example against the number of trials per hypothesis, at 95% confidence and $\delta = 10^{-5}$: the likelihood-ratio statistic (circles), the sum of projections (triangles) and the Gaussian-shape estimate from the likelihood ratio (squares), with the accountant's 6.71 and the ceiling of a perfect attack. The bounds grow roughly like the logarithm of the number of trials.

- Why does the Gaussian-shape estimate exceed the rigorous bound with the same trials? *(It measures μ where rates are well estimated and extrapolates to the tail using the Gaussian curve; the rigorous bound only uses what was measured. The estimate is a valid bound only if the curve really is Gaussian.)*

45.6. One-run auditing

45.6.1. Context

The audits so far need one training run per trial, thousands of runs in all. For a model that takes a day to train, that is out of the question. Can one training run give a valid bound?

45.6.2. Statement

Insert m canaries, each independently with probability $\frac{1}{2}$: $S_i = +1$ if canary i is in the training set and -1 if not. Train once. Score every canary, and guess $\hat{S}_i = +1$ ("in") for the $r/2$ highest scores, $\hat{S}_i = -1$ ("out") for the $r/2$ lowest, and abstain ($\hat{S}_i = 0$) on the rest. Let W be the number of correct guesses. If training is ε-DP, then for every v

$$\Pr[W \geq v] \leq \Pr\left[\mathrm{Bin}(r, p_\varepsilon) \geq v\right], \qquad p_\varepsilon = \frac{e^\varepsilon}{1 + e^\varepsilon}. \tag{45.10}$$

If training is (ε, δ)-DP, then with $f(v) = \Pr[\mathrm{Bin}(r, p_\varepsilon) \geq v]$,

$$\Pr[W \geq v] \leq f(v) + 2m\delta \max_{i=1,\ldots,m} \frac{f(v-i) - f(v)}{i} \tag{45.11}$$

(Steinke, Nasr & Jagielski 2023, Corollary 5.4). Observing v correct guesses rejects every ε whose right-hand side is below β; the largest such ε is the bound.

45.6.3. Derivation (pure DP; the δ term is in the paper)

1. **Each inclusion bit is protected.** Fix every inclusion bit except S_i. Changing S_i adds or removes one record, so the training run, and the guesses computed from it (post-processing), are ε-DP as a function of S_i: for every vector of guesses s, $\Pr[\hat{S} = s \mid S_i = +1, S_{-i}] \leq e^\varepsilon \Pr[\hat{S} = s \mid S_i = -1, S_{-i}]$.
2. **Average over the later bits.** Order the guessed canaries. Conditioning only on the earlier bits $S_{<i}$, average step 1 over the later ones $S_{>i}$: they are independent of S_i, so the same weights appear on both sides, and $\Pr[\hat{S} = s \mid S_i = +1, S_{<i}] \leq e^\varepsilon \Pr[\hat{S} = s \mid S_i = -1, S_{<i}]$.
3. **No guess is right more than p_ε of the time.** By Bayes' rule with the prior $\frac{1}{2}$ on S_i, $\Pr[S_i = +1 \mid \hat{S} = s, S_{<i}] = \frac{x}{x+y}$ with $x = \Pr[s \mid S_i = +1, S_{<i}]$ and $y = \Pr[s \mid S_i = -1, S_{<i}]$. The fraction grows with x, and step 2 says $x \leq e^\varepsilon y$, so it is at most $\frac{e^\varepsilon y}{e^\varepsilon y + y} = \frac{e^\varepsilon}{e^\varepsilon + 1} = p_\varepsilon$. The same holds for $S_i = -1$. So whichever way canary i is guessed, it is correct with probability at most p_ε given the guesses and the earlier bits, which fix whether the earlier guesses were correct.

4. **A sum of such indicators is dominated by a binomial.** Write the j-th guess's correctness as $\mathbf{1}[U_j \le p_j]$ with U_j independent uniform numbers and $p_j \le p_\varepsilon$ its conditional probability from step 3. Then $\mathbf{1}[U_j \le p_j] \le \mathbf{1}[U_j \le p_\varepsilon]$, and the right-hand indicators sum to a $\mathrm{Bin}(r, p_\varepsilon)$ variable. So W is at most a binomial variable, and Equation 45.10 follows.

45.6.4. Interpretation

Randomized response is the worst case, and that sets the scale. The binomial of Equation 45.10 is what you get when each guess is ε-randomized response on the true bit (The definition of differential privacy), so the bound is tight for that mechanism (Exercise 6 in the exercise book). The paper's worked example: 75 correct out of 100 is what $\varepsilon = \ln 3$ produces on average, and Equation 45.10 gives the lower bound 0.702 at 95% confidence; the script reproduces it, and the paper's 0.699 and 0.673 for $\delta = 10^{-4}$ with 100 and 1,000 canaries.

The ceiling is again logarithmic. If all r guesses are right, the bound is the ε with $p_\varepsilon^r = \beta$, about $\ln(r/\ln(1/\beta))$: 3.49 for 100 canaries, 5.81 for 1,000 and 8.11 for 10,000, with $\delta = 0$. Certifying 6.71 needs about 2,500 canaries guessed without a single error (Exercise 5 in the exercise book). Each canary is a trial, so a one-run audit has the same ln ceiling as the many-run audit with $n \approx m$; it saves runs, not trials.

The δ term grows with the number of canaries. The correction in Equation 45.11 is at most $2m\delta$. With $m = 1{,}000$ and $\delta = 10^{-5}$ that is 0.02, small next to $\beta = 0.05$; with 10,000 canaries all guessed right the bound falls from 8.11 to 7.83. When $2m\delta$ approaches the significance level, the correction can swallow the whole budget: no number of correct guesses rejects anything.

One run is not tight for DP-SGD. Even with white-box Dirac canaries, the one-run bound stays well below the many-run bound. Each canary is guessed once, at a fixed fraction of guesses, so the audit reads the trade-off curve near FPR $1/m$ at best; and the binomial is calibrated to randomized response, whose curve has a sharp corner, while the curve of DP-SGD is smooth. Steinke et al. show that their bound is exact for randomized response; for DP-SGD their one-run bounds stay below the theoretical ε even with white-box gradient canaries.

45.6.5. Example: Dirac canaries in one run

Simulate one run of the running example with 1,000 Dirac canaries on distinct untouched coordinates, about half included, each scored by Λ. The auditor tries six list lengths r and pays for trying them with a Bonferroni correction: each test runs at level $\beta/6$, so that the chance that any of the six is wrong is at most β. Table 45.4 gives the result.

Table 45.4.: One-run audit of the running example's mechanism with 1,000 white-box Dirac canaries, at 95% overall confidence and $\delta = 10^{-5}$.

Guesses r	20	50	100	200	500	1,000
Correct, white-box Dirac canaries	20	50	92	185	425	745
ε_{lo} (Bonferroni over the six)	1.21	2.22	1.58	1.88	1.43	0.90

All 50 most confident guesses are right, which certifies 2.22; the 100-guess list makes 8 mistakes and certifies less. With 5,000 canaries the best bound is 2.53, from 479 correct out of 500. There the δ correction shows its teeth: $2m\delta = 0.1$ exceeds the per-test level $\beta/8$, and 20 correct guesses out of 20 certify nothing at all. One run certifies 2.22, against 2.84 for 10,000 runs per hypothesis in Table 45.3, at a twenty-thousandth of the cost.

45.6.6. Check yourself

- Why must each canary be included by an independent fair coin, rather than, say, the first half included? *(Step 3 uses the prior $\frac{1}{2}$ on S_i and step 2 the independence of the bits. With a deterministic split, the guesses could be right for reasons that have nothing to do with the training run, for example the order of the canaries.)*
- 960 correct guesses out of 1,000 under pure DP: is $\varepsilon = 2$ rejected? *(Yes. Under $\varepsilon = 2$, $p_\varepsilon = 0.881$, so the binomial has mean 881 and standard deviation about $\sqrt{1{,}000 \times 0.881 \times 0.119} = 10.2$; 960 is almost eight standard deviations above.)*

45.7. Black-box audit of the trained network

45.7.1. Context

The white-box Dirac audit tests the mechanism. What does an attacker learn from the released network itself, with canaries that are ordinary records?

45.7.2. Statement: the protocol

Take 1,000 records from the held-out test set of the running example and flip their labels: **mislabeled canaries**, which the model can only fit by memorizing them. Include each in the 10,000 training records with probability $\frac{1}{2}$, train once with DP-SGD at the claimed configuration, and score each canary by minus its loss on the final model: a canary the model fits well was probably trained on. Guess with the six list lengths of Section 45.6 and bound ε with Equation 45.11,

Bonferroni-corrected. As controls, run the same audit on a model trained with clipping but no noise, and on a model trained without clipping or noise.

The bound is valid for the add/remove neighbours of any canary, because including a canary is adding a record, and all the guarantees of DP-SGD hold for every dataset, including one with planted records.

45.7.3. Interpretation

Before the numbers, what to expect. The black-box attacker sees one function of the 10,000 noisy sums, the canary's gradient changes as it is fitted, and its direction is shared with the gradients of the 10,000 real records, which pull the other way since the canary's label is wrong. Each of these weakens the audit relative to Section 45.5. Nothing in the accountant knows about them: the accountant bounds the worst dataset, the worst record and an attacker who sees every step. So the black-box bound should be far below 6.71, and the gap measures this attacker, not the privacy of the algorithm.

The no-noise control separates the two. A model trained with clipping and no noise has no finite ε at all. If the black-box audit of that model also gives a small bound, then the audit's weakness, not the noise, is what keeps the bound small.

45.7.4. Example: three training runs

Table 45.5 gives the three audits, one training run each.

Table 45.5.: Black-box one-run audit of the running example's network with 1,000 mislabeled canaries, one training run each, at 95% overall confidence and $\delta = 10^{-5}$. Test accuracy is on the 19,000 test records not used as canaries.

Training	Test accuracy	Mean loss, in	Mean loss, out	Correct of $r = 100$	Correct of $r = 1{,}000$	Best ε_{lo}
DP-SGD, $\sigma = 1$, claim $\varepsilon = 6.71$	0.9645	5.565	6.013	59	556	0.13
Clipping only, $\sigma = 0$, $\varepsilon = \infty$	0.9638	5.511	6.155	67	589	0.24
Non-private, no clipping	0.9155	0.023	9.041	91	967	2.96

The non-private model memorizes the canaries it trained on: their mean loss is 0.023 against 9.041 for those it did not see, 967 of 1,000 guesses are right, and the audit certifies 2.96; 1,000 perfect

guesses would certify 5.78 at $\delta = 10^{-5}$ even without the Bonferroni correction, so 33 mistakes cost almost three units. Memorizing 481 contradictory labels costs it almost five points of test accuracy, 0.9155 against the DP model's 0.9645. DP-SGD does not memorize: the losses of included and excluded canaries differ by less than half a unit on average, 59 of the 100 most confident guesses are right, and the certified bound is 0.13 (from 293 of 500, the best list). The clipping-only model is barely easier, 0.24. So with $\sigma = 1$ removed, the black-box attacker still learns almost nothing. Clipping alone bounds each canary's pull on the weights to C per step, against the combined pull of 10,000 correctly labelled records, and that is what defeats this attacker. Figure 45.3 shows the loss distributions and how the fraction of correct guesses falls as the list grows.

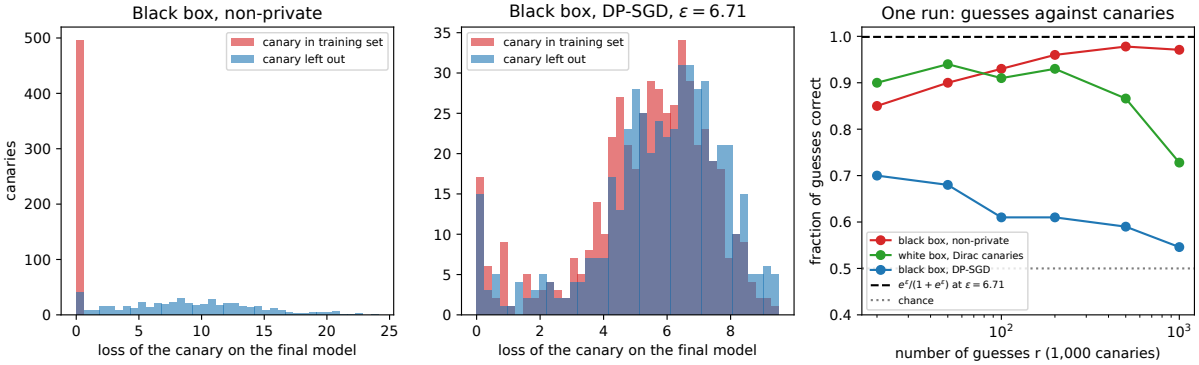


Figure 45.3.: One-run audits with 1,000 canaries. Left and middle: the loss of each mislabeled canary on the final model, for canaries in the training set (red) and left out (blue), without privacy and with DP-SGD at $\varepsilon = 6.71$. Right: the fraction of the r guesses that are correct, against r, for the non-private network (black box), the DP-SGD network (black box) and the mechanism with white-box Dirac canaries; the dashed line is the most a 6.71-DP algorithm allows on average, $e^{\varepsilon}/(1 + e^{\varepsilon})$.

What this says about the claim. Nothing contradicts 6.71, and nothing confirms it. The black-box bound of 0.13 says that a realistic attacker, with one model and planted mislabeled records, extracts very little; the white-box audits of Section 45.5 say that an attacker who sees every step and controls a gradient extracts much more. Reporting both, with the threat model of each, is what an honest privacy statement does (Choosing an accountant lists what else it must contain).

45.7.5. Check yourself

- The audit of the clipping-only model gives 0.24. Does that mean clipping alone is 0.24-DP? *(No. The model has no finite ε: a Dirac gradient canary seen white-box would identify membership perfectly. The bound says only that this black-box attack, with these canaries, cannot show more.)*
- Why mislabeled canaries rather than random test records? *(A correctly labelled record from the data distribution has a small gradient and nearly the same loss whether or not it was trained on, so the audit would learn even less; a mislabeled one can only be fitted by memorizing it, which is the leakage the audit looks for.)*

45.8. Finding bugs

45.8.1. Context

An audit bound below the claim proves nothing. A bound above it proves, with confidence $1-\beta$, that the claim is false. That makes audits a test suite for private training code: the test passes silently on a correct implementation and fails loudly on a broken one. Which bugs does it catch, and how cheaply?

45.8.2. Statement

A **per-step audit** inserts a Dirac canary at every step (canary sampling rate $q_c = 1$) in one run and leaves it out in another, and treats each step's projection as one trial: T trials per hypothesis from two runs. In a correct implementation each step is a Gaussian mechanism with sensitivity C and noise σC, so the projections are

$$x_t \sim \mathcal{N}(0, \sigma^2) \text{ without the canary}, \qquad x_t \sim \mathcal{N}(1, \sigma^2) \text{ with it}, \tag{45.12}$$

and each step is exactly μ-GDP with $\mu = 1/\sigma$. The audit's μ_{lo} from Equation 45.9 with Clopper–Pearson rates is a rigorous lower bound on the per-step μ. If $\mu_{\text{lo}} > 1/\sigma$, the implementation does not add the noise it claims.

An **end-to-end audit** with the many-trial protocol of Section 45.5 tests everything at once, sampling and accounting included, but needs many runs.

45.8.3. Derivation

1. **One step with the canary always present is a Gaussian mechanism.** With $q_c = 1$ there is no sampling coin for the canary, so Fact 4 gives Equation 45.12: a shift of 1 in noise of standard deviation σ. By Fact 3 the best test thresholds x_t itself, and its trade-off curve is Equation 45.2 with $\mu = 1/\sigma$.
2. **The steps are independent trials.** The noise is fresh at every step and the canary's gradient is fixed, so the T projections of each run are independent, and the counts at a fixed threshold are binomial.
3. **The bound on μ.** For any threshold, a μ-GDP step has $\text{TPR} \le 1 - \Phi(\Phi^{-1}(1-\text{FPR}) - \mu)$, which rearranges to Equation 45.9. Replacing the rates by their Clopper–Pearson bounds can only lower the right-hand side (it grows with the TPR and shrinks with the FPR), so with probability $1-\beta$ the true per-step μ is at least μ_{lo}.
4. **A refutation.** A correct implementation has $\mu = 1/\sigma$, so $\mu_{\text{lo}} > 1/\sigma$ happens with probability at most β. Observing it rejects the hypothesis that the code adds noise σC to a sum with sensitivity C.

45.8.4. Interpretation

Per-step audits test the noise and the clipping; they cannot see the sampling or the accountant. With the canary in every step, the audit never exercises the sampling code, and it compares the measured noise with the claimed σ, not with the claimed ε. A bug in how batches are drawn, or in what the accountant is told, passes it. Nasr et al. (2023) audit one step for this reason and say so; they caught clipping applied after averaging, noise drawn from a seed with only 100 possible values (a similar error appeared in a canonical JAX example of DP-SGD), and a wrong noise scale.

End-to-end audits see everything, but only large bugs. An end-to-end audit tests the claimed ε directly, so it catches an accounting bug as readily as a noise bug, provided the true ε exceeds the claim by more than the audit's shortfall. From Section 45.5, that shortfall is about three units at 100,000 runs for the running example. Tramèr, Terzis, Steinke, Song, Jagielski & Carlini (2022) used such an audit to show that an open-source implementation of a differentially private deep-learning algorithm violated its claimed guarantee, at a confidence they report as $1 - 10^{-10}$.

What no canary audit catches. An audit tests one dataset, one canary and one attack. A bug that only shows on datasets or records the audit did not try, or that raises ε by less than the audit's shortfall, passes. Sampling bugs are the instructive case. Batches drawn by shuffling but accounted as Poisson can have an ε several times the reported one (Amplification by subsampling derives a lower bound that shows it), yet a per-step audit with the canary in every step never exercises the sampling, and an end-to-end audit sees the gap only where it exceeds the audit's shortfall. Passing an audit is evidence, never a certificate.

45.8.5. Example: three implementations and an accounting bug

Table 45.6 audits three implementations of the running example's noise, each with two runs of 10,000 steps, and an accounting bug.

Table 45.6.: Per-step audits of three implementations of the running example's DP-SGD step, from 10,000 steps with and 10,000 without a Dirac canary, at 95% confidence. The last column is what the RDP accountant would report for the noise the audit measured.

Implementation	Noise on the sum	Claimed per-step μ	Audited μ_{lo}	Accountant at $\sigma = 1/\mu_{\text{lo}}$	Verdict
Correct	σC	1	0.886	5.38	consistent
Variance passed as a standard deviation	$\sigma^2 C^2 = 0.5\,C$	1	1.745	29.27	refuted
Noise scaled for the mean, added to the sum	$\sigma C/B = 0.01\,C$	1	6.359	over 4,700	refuted

The correct implementation passes: 0.886 is below its claimed 1, and plugged into the accountant it gives 5.38, close to but below 6.71. The two noise bugs fail decisively. Passing $\sigma^2 C^2 = 0.25$ where the code expects a standard deviation halves the noise when $C = 0.5$, and the audit measures $\mu_{\text{lo}} = 1.745$ against the claimed 1. Scaling the noise for the averaged gradient, whose sensitivity is C/B, and then adding it to the sum, whose sensitivity is C, makes it $B = 100$ times too small, and 10,000 steps separate the two hypotheses perfectly: 6.359 is the most Equation 45.9 can certify from 5,000 error-free trials per hypothesis, the evaluation half. The same bug also fails an end-to-end audit with the sum of projections: with 10,000 runs per hypothesis, the threshold catches all 5,000 canaries of the evaluation half with one false positive, and $\varepsilon_{\text{lo}} = 7.21 > 6.71$.

The accounting bug: the code is correct, but the accountant is told the dataset has 20,000 records instead of 10,000, as can happen when augmented copies are counted, so it computes $q = 0.005$ and reports $\varepsilon = 3.06$. The per-step audit measures the noise, which is right, and composing its μ_{lo} at the claimed $q = 0.005$ gives 2.47, which does not contradict 3.06. The end-to-end audit of Table 45.3 does: its 3.63 with 50,000 trials per hypothesis exceeds 3.06, so the claim is false with 95% confidence. Figure 45.4 shows why the two noise bugs are easy to see.

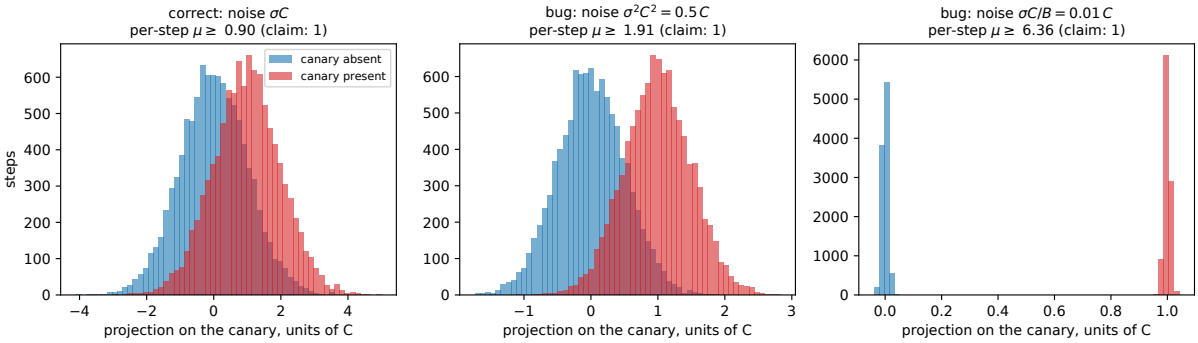


Figure 45.4.: Per-step audits of three implementations: the projections of 10,000 steps on a Dirac canary, without it (blue) and with it at every step (red), in units of C. A correct implementation has noise of standard deviation σC; the two bugs shrink it by a factor of 2 and of 100. The titles give the certified lower bound on the per-step μ; the claim is 1.

45.8.6. Check yourself

- A per-step audit of your implementation gives $\mu_{\text{lo}} = 0.97$ for a claimed $\sigma = 1$. Can you conclude the implementation is correct? *(No. A lower bound on μ can show that μ is too large, never that it is small enough: 0.97 is consistent with the claimed $\mu = 1$, and also with noise a little too small. And the audit says nothing about the sampling, the accountant or bugs that do not affect a Dirac canary.)*
- The opposite mistake adds noise σC to the averaged gradient instead of to the sum. Would an audit flag it? *(No. Noise σC on the mean is noise σCB on the sum, B times more than claimed: the implementation is more private than its claim and every audit passes. It is still a bug, since it wastes accuracy, but audits only catch errors in the unsafe direction.)*

45.9. Code: run every audit

The script code/auditing.py implements the Clopper–Pearson bounds from the incomplete beta function, the bound of Equation 45.4 with thresholds chosen on half the trials, the white-box simulation of Equation 45.3 with both statistics, the one-run bound of Equation 45.11, the black-box audit of the running example's network (importing the model and DP-SGD from code/dp_sgd.py) and the per-step audits. From the repository root:

```
uv run python code/part05-dp-ml/auditing.py
```

It takes a few minutes, most of it simulating 100,000 white-box trials of 10,000 steps each and training three networks.

Output (seed 0), summarized in Table 45.7:

Table 45.7.: The script's output with the default seed.

Experiment	Result
Clopper–Pearson, $k = 10$ of 10,000	$[0.00048, 0.00184]$, as in Part 1
One-run bound, the paper's examples	0.702, 0.699 and 0.673, as published
Ceiling of a perfect attack	5.60 with 1,000 trials, 7.90 with 10,000; 3,038 needed for 6.71
White-box Dirac audit, likelihood ratio	$\varepsilon_{\text{lo}} = 3.63$ with 50,000 trials per hypothesis
Same, Gaussian-shape estimate	5.33, against the CLT's 6.01
One run, 1,000 white-box Dirac canaries	2.22
One run, black box, mislabeled canaries	0.13 with DP-SGD, 0.24 with clipping only, 2.96 non-private
Per-step audits	Correct 0.886 against a claim of 1; bugs 1.745 and 6.359

tests/test_auditing.py checks that the output still matches.

What to take from it. The bounds are valid but conservative, and they grow only like the logarithm of the effort. An audit is most useful as a test: the strongest attack you can mount, white-box with a gradient canary, aimed at the claim you want to check. A correct implementation passes it with room to spare, and the two noise bugs fail it with margins no amount of bad luck explains.

45.10. References

- Jagielski, Ullman & Oprea, "Auditing Differentially Private Machine Learning: How Private is Private SGD?" (NeurIPS 2020). An early audit of DP-SGD: poisoning canaries, Clopper–Pearson intervals, and the lower bound of Equation 45.4.

- Nasr, Song, Thakurta, Papernot & Carlini, "Adversary Instantiation: Lower Bounds for Differentially Private Machine Learning" (IEEE S&P 2021). A ladder of threat models from black box to gradient canaries with every iterate visible; tight bounds for the strongest.
- Nasr, Hayes, Steinke, Balle, Tramèr, Jagielski, Carlini & Terzis, "Tight Auditing of Differentially Private Machine Learning" (USENIX Security 2023). Dirac canaries, per-step audits with the canary at every step, Gaussian-shape bounds, and three bugs found.
- Steinke, Nasr & Jagielski, "Privacy Auditing with One (1) Training Run" (NeurIPS 2023). The one-run audit of Section 45.6; Corollary 5.4 is Equation 45.11, and its appendix gives code for the bound.
- Tramèr, Terzis, Steinke, Song, Jagielski & Carlini, "Debugging Differential Privacy: A Case Study for Privacy Auditing" (arXiv 2022). An audit that refuted the guarantee of a published implementation.
- Carlini, Chien, Nasr, Song, Terzis & Tramèr, "Membership Inference Attacks From First Principles" (IEEE S&P 2022). The likelihood ratio attack (LiRA) and the case for reporting TPR at a low FPR, which an audit's bound formalizes.
- Dong, Roth & Su, "Gaussian Differential Privacy" (JRSS B 2022). Trade-off curves, μ-GDP and the CLT for composition used in Section 45.5.
- Kairouz, Oh & Viswanath, "The Composition Theorem for Differential Privacy" (ICML 2015). The hypothesis-testing reading of (ε, δ)-DP behind Equation 45.1.

45.11. Cheat sheet

Canary in (D') or out (D); FPR and TPR of the attack's threshold test; n trials per hypothesis; Clopper–Pearson bounds $\bar{\alpha}$ and $\underline{\tau}$ at $\beta/2$. Table 45.8 collects the results.

Table 45.8.: Every result of the chapter.

Result	Formula	Conditions
DP caps every test	$\mathrm{TPR} \le e^{\varepsilon}\mathrm{FPR} + \delta$, $1 - \mathrm{FPR} \le e^{\varepsilon}(1 - \mathrm{TPR}) + \delta$	Any test, any neighbours
Certified bound	$\max\{\ln \frac{\underline{\tau}-\delta}{\bar{\alpha}}, \ln \frac{1-\bar{\alpha}-\delta}{1-\underline{\tau}}\}$	Independent trials; threshold chosen on other trials
Ceiling	$\approx \ln(n/\ln(2/\beta))$: 5.60 at $n = 1{,}000$, 7.90 at 10,000	Perfect attack
Best statistic, Dirac canary	$\Lambda = \sum_t \ln(1-q+qe^{(x_t-1/2)/\sigma^2})$	White box, untouched coordinate
Gaussian approximations	$\mu = q\sqrt{T(e^{1/\sigma^2}-1)}$; sum of projections $q\sqrt{T}/\sigma$	CLT regime
Where ε lives	FPR $\Phi(-\varepsilon/\mu - \mu/2)$: 8.1×10^{-8} for the running example	μ-GDP
GDP bound from one test	$\mu \ge \Phi^{-1}(\mathrm{TPR}) - \Phi^{-1}(\mathrm{FPR})$	Rigorous for μ; for ε only if the curve is Gaussian
One run	$W \preceq \mathrm{Bin}(r, \frac{e^{\varepsilon}}{1+e^{\varepsilon}})$, plus $2m\delta$-type correction	Canaries included by independent fair coins

Result	Formula	Conditions
Per-step audit	$x_t \sim \mathcal{N}(0, \sigma^2)$ vs $\mathcal{N}(1, \sigma^2)$; claim $\mu = 1/\sigma$	Canary at every step; tests noise and clipping only

Rules of thumb

- An audit can refute a claim, never prove one. Report its threat model with its number.
- Budget trials before you start: certifying ε needs at least $e^{\varepsilon} \ln(2/\beta)$ trials per hypothesis, and in practice far more, because ε at small δ lives at false positive rates you cannot measure.
- To test an implementation, use the strongest attack: white box, Dirac gradient canary, likelihood ratio. Audit the noise per step and the claim end to end.
- Choose thresholds and list lengths on separate data, or pay for the choice with a Bonferroni correction.
- Read a small black-box bound as a statement about the attacker, not as the model's privacy.

Practice. Exercises for this chapter, each with a worked solution, are in the exercise book in the exercise book.

A. Glossary

Acronyms used in the book and the pages that use them.

AI Artificial intelligence. Used in Choosing ε.

API Application programming interface. Used in Threat models and privacy games, Choosing ε.

AUC Area under the curve. Used in Hypothesis testing, Threat models and attacks, Membership inference, Private prediction: PATE and in-context learning.

CDF Cumulative distribution function. Used in Probability distributions, Hypothesis testing, Threat models and privacy games, Syntactic anonymity and how it breaks, The Laplace and Gaussian mechanisms, Selection: exponential mechanism, report-noisy-max, sparse vector, Discrete and floating-point-safe mechanisms.

CDP Concentrated differential privacy. Used in Concentrated differential privacy.

CLT Central limit theorem. Used in Probability distributions, Concentration inequalities, f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant, Choosing ε, Auditing private training.

CPA Chosen-plaintext attack. Used in Cryptography basics.

DLG Deep Leakage from Gradients. Used in Gradient inversion.

DNA Deoxyribonucleic acid. Used in Lower bounds: what privacy costs.

DP Differential privacy. Used in Differential privacy foundations, The definition of differential privacy, Post-processing, group privacy and composition, The Laplace and Gaussian mechanisms, Advanced and optimal composition, Selection: exponential mechanism, report-noisy-max, sparse vector, Beyond global sensitivity, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training.

DPI Data processing inequality. Used in Divergences, Reconstruction attacks, Post-processing, group privacy and composition.

EMD Earth mover's distance. Used in Syntactic anonymity and how it breaks.

ERM Empirical risk minimization. Used in Optimization, Algorithmic stability and generalization, Differential privacy in machine learning, Private empirical risk minimization, Public data and private fine-tuning.

FFT Fast Fourier transform. Used in Numerical accounting with privacy loss distributions, Choosing an accountant, Choosing ε.

FL Federated learning. Used in Gradient inversion.

FNR False negative rate. Used in The definition of differential privacy, Auditing private training.

FPR False positive rate. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference, Attribute inference and model inversion, Property inference, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training.

FTRL Follow the regularized leader. Used in Differential privacy in machine learning, DP-FTRL and matrix factorization, Choosing ε.

GAN Generative adversarial network. Used in Divergences, Private prediction: PATE and in-context learning.

GD Gradient descent. Used in Optimization, Amplification by shuffling and by iteration.

GDP Gaussian differential privacy. Used in Concentrated differential privacy, f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Choosing ε, Auditing private training.

GPU Graphics processing unit. Used in Threat models and attacks, Membership inference, Differential privacy in machine learning.

HIV Human immunodeficiency virus. Used in Syntactic anonymity and how it breaks.

HMAC Hash-based message authentication code. Used in Cryptography basics.

IND Indistinguishability, as in IND-CPA. Used in Cryptography basics.

KL Kullback–Leibler. Used in Prerequisites, Divergences, Hypothesis testing, Information theory, Advanced and optimal composition, Local, shuffle and distributed trust models, Rényi differential privacy, Concentrated differential privacy.

LLM Large language model. Used in Private prediction: PATE and in-context learning.

LP Linear programming. Used in Reconstruction attacks.

LR Likelihood ratio. Used in Hypothesis testing.

LWE Learning with errors. Used in Cryptography basics.

MAP Maximum a posteriori. Used in Information theory, Attribute inference and model inversion, Linkage and re-identification.

MGF Moment generating function. Used in Probability distributions, Concentration inequalities, Divergences, Local, shuffle and distributed trust models, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy.

MIA Membership inference attack. Used in Private prediction: PATE and in-context learning.

ML Machine learning. Used in Cryptography basics, Membership inference, Choosing an accountant, DP-SGD, Hyperparameter tuning and private selection, Choosing ε.

NN Nearest neighbour. Used in Algorithmic stability and generalization.

PATE Private aggregation of teacher ensembles. Used in Differential privacy in machine learning, Private prediction: PATE and in-context learning.

PIN Personal identification number. Used in Information theory.

PLD Privacy loss distribution. Used in f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing an accountant, Choosing ε.

PMF Probability mass function. Used in Probability distributions, Discrete and floating-point-safe mechanisms.

PPT Probabilistic polynomial time. Used in Cryptography basics.

PRF Pseudorandom function. Used in Cryptography basics.

PRG Pseudorandom generator. Used in Cryptography basics.

PRP Pseudorandom permutation. Used in Cryptography basics.

PRV Privacy loss random variable. Used in From Rényi DP to (ε, δ) and to tests, f-DP and Gaussian differential privacy, Numerical accounting with privacy loss distributions, Choosing

an accountant.

RDP Rényi differential privacy. Used in Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, User-level differential privacy, Choosing ε, Auditing private training.

ROC Receiver operating characteristic. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference, The definition of differential privacy.

SGD Stochastic gradient descent. Used in Optimization, Algorithmic stability and generalization, Gradient inversion, Privacy accounting, Rényi differential privacy, From Rényi DP to (ε, δ) and to tests, Concentrated differential privacy, f-DP and Gaussian differential privacy, Amplification by subsampling, Numerical accounting with privacy loss distributions, Amplification by shuffling and by iteration, Adaptive composition: filters, odometers and individual accounting, Choosing an accountant, Differential privacy in machine learning, Private empirical risk minimization, DP-SGD, Hyperparameter tuning and private selection, DP-FTRL and matrix factorization, Private prediction: PATE and in-context learning, Public data and private fine-tuning, User-level differential privacy, Disparate impact, Choosing ε, Auditing private training.

TPR True positive rate. Used in Hypothesis testing, Threat models and attacks, Threat models and privacy games, Membership inference, Attribute inference and model inversion, Property inference, The definition of differential privacy, Lower bounds: what privacy costs, Choosing ε, Auditing private training.

TV Total variation. Used in Divergences, Hypothesis testing, Cryptography basics, Threat models and privacy games, Property inference, Syntactic anonymity and how it breaks, The definition of differential privacy, The Laplace and Gaussian mechanisms, Discrete and floating-point-safe mechanisms, Local, shuffle and distributed trust models, Lower bounds: what privacy costs, f-DP and Gaussian differential privacy.

XOR Exclusive or. Used in Cryptography basics.

ZIP Zone Improvement Plan code, the US postal code. Used in Linkage and re-identification, Syntactic anonymity and how it breaks, Lower bounds: what privacy costs.

B. Index

AboveThreshold, 571, 586, 587, 589, 592–596
adaptive composition, 515, 521, 523, 549, 568, 703, 718, 730, 794, 830, 849, 862, 916, 921, 922, 929, 958, 960, 1010, 1048, 1049
advanced composition, 469, 549, 553, 556, 564, 566, 568, 569, 594, 702, 728, 730, 779, 830, 831, 847, 892, 896, 898, 899, 902, 903, 915–917, 920, 921, 924, 928–930, 941
amplification by iteration, 865, 888, 890, 930
amplification by shuffling, 647, 670, 703, 888, 929, 930
amplification by subsampling, 466, 471, 703, 706, 730, 753, 756, 779, 836, 858, 920, 933, 944, 972, 1094
analytic Gaussian mechanism, 527, 541–545, 547
approximate differential privacy, 468, 479, 482, 500, 520, 524, 568, 685, 690, 698–700, 862, 888, 1101
approximate zCDP, 756, 778–780, 916
attribute inference, 250, 251, 253, 303, 306, 323, 1141, 1147
auxiliary knowledge, 254, 259–261, 275, 308
Azuma's inequality, 57, 555

background knowledge attack, 436
basic composition, 469, 503, 517, 521, 523–525, 535, 553, 556, 557, 564, 566, 569, 586, 587, 596, 610, 702, 730, 799, 830, 835, 898, 902, 915, 920, 925, 930, 1146
Bernoulli distribution, 7, 31, 32, 42, 46, 56, 57, 135, 153, 332, 621, 630, 644, 653, 656, 804, 1064, 1159
binary symmetric channel, 130, 142, 148, 155–157
binomial distribution, 7, 12, 14, 15, 19, 31, 32, 41, 128, 129, 265, 270, 332, 333, 411, 495, 552, 824, 826, 827, 833, 835, 838, 858, 873, 930, 931, 987, 996, 998, 1009, 1010, 1021, 1025, 1087, 1088, 1092, 1102, 1117, 1123, 1154, 1157, 1166, 1167, 1170
black-box attack, 255, 259, 260, 262, 263, 273, 274, 278, 303, 329, 345, 598, 1154, 1168, 1169, 1175
blatantly non-private, 454, 463

canary, 252, 253, 348, 358–362, 367, 947, 1154, 1159–1161, 1165, 1167–1175
central limit theorem, 32, 265, 288, 336, 568, 668, 782, 783, 798–803, 807–809, 840, 858–860, 862, 863, 874, 920–930, 938–942, 1117, 1128, 1150, 1159, 1162, 1173, 1174
central limit theorem for composition, 568, 1150
channel capacity, 130, 156
Chernoff bound, 7, 32, 33, 41, 46, 49, 54, 56–58, 95, 128, 726, 736, 758, 761, 873, 916
clipping, 450, 469, 527, 598, 612, 616–620, 702, 817, 818, 830, 911, 932, 933, 936, 937, 944–947, 949, 963, 964, 969, 974, 975, 980, 982–984, 1006, 1028, 1061, 1067, 1080, 1092, 1097, 1100, 1102, 1104, 1106, 1107, 1111–1114, 1120–1124, 1148, 1168, 1169, 1173, 1175
Clopper–Pearson interval, 125–129, 1117, 1118, 1122, 1124, 1154, 1162, 1163, 1170, 1173
collision probability, 232, 401
composition, 253, 418, 434, 436, 437, 466, 467, 469, 471, 474, 500, 503, 509,

515, 517, 520, 521, 523–525, 535, 549, 553, 555–557, 559, 562–566, 568, 569, 586, 587, 594–596, 610, 650, 665, 702, 703, 708, 718, 728, 730, 731, 753, 756, 759, 761, 763, 766, 779, 780, 782, 791, 794, 798, 799, 808, 809, 811, 830, 831, 835, 836, 840, 847, 849, 862, 863, 882, 884–886, 888, 890, 892, 896, 898, 899, 902, 903, 915–917, 920–922, 924–926, 928–930, 933, 937, 940, 941, 958, 960, 966, 971, 987, 990–992, 997–999, 1003, 1005–1010, 1045, 1048, 1049, 1058, 1085, 1092, 1094, 1131, 1146, 1150, 1152, 1174
composition attack, 253, 418, 434, 437
conditional entropy, 130, 139, 153, 157
contractive noisy iteration, 865

data processing inequality, 61, 75, 90, 92, 94, 96, 130, 140, 153, 155, 156, 330, 331, 456, 464, 466, 506, 525, 539, 702, 763, 926, 938
deduplication, 348, 357, 1079
Deep Leakage from Gradients, 371, 380, 382, 383, 386–390
differential privacy, 1, 250, 324, 415, 463, 466, 468, 471, 500, 521, 524, 546, 568, 594, 595, 619, 624, 643, 650, 670, 671, 698, 699, 702, 703, 705, 708, 730, 753, 779, 782, 783, 808, 835, 836, 862, 888, 889, 891, 900, 916, 929, 933, 940, 941, 944, 945, 965, 966, 983, 984, 1009, 1028, 1032, 1058, 1080, 1101, 1123, 1128, 1136, 1142, 1144, 1145, 1150, 1151, 1173, 1174
Dinur–Nissim attack, 440, 673, 674, 695, 698, 700
discoverable memorization, 357, 367
discrete Gaussian distribution, 469, 622, 636, 639, 641, 643–645, 671, 753, 779, 780, 1144
discrete Laplace distribution, 469, 622, 641, 645
distributed differential privacy, 670, 888
distribution inference, 345
dominating pair, 836, 837, 840, 846, 849, 862, 863, 922
double backpropagation, 381, 388
DP-SGD, 702–704, 706, 708, 730, 754, 756, 777–779, 783, 802, 803, 807–809, 811, 831, 836, 838, 840, 858–861, 889, 891, 892, 916, 919, 921, 924, 929, 931, 934, 936, 939, 941, 944–947, 949, 969, 972, 977, 980, 981, 983, 987, 991, 992, 998, 999, 1003, 1005, 1006, 1010, 1012, 1028–1033, 1035, 1043, 1044, 1047, 1054, 1057, 1061, 1071, 1077, 1080, 1083, 1089–1095, 1097–1099, 1101, 1102, 1106, 1107, 1113, 1120, 1122, 1123, 1126, 1128, 1131, 1133–1135, 1137, 1140, 1142, 1143, 1145, 1147, 1148, 1150, 1152, 1154, 1161, 1168, 1169, 1171, 1173

earth mover's distance, 429, 431, 437
eccentricity, 393, 409, 414–416
empirical risk minimization, 4, 159, 184, 188–190, 192, 201, 204, 215–217, 297, 944, 945, 965, 983, 1061, 1064, 1067, 1080, 1081, 1123
entropy, 6, 130, 131, 135, 137–139, 141, 144, 148, 150–154, 156, 157, 285, 348, 390, 402, 403, 425, 435, 437, 456, 457
exponential mechanism, 467, 469, 570, 571, 576, 577, 579, 585, 586, 594–596, 756, 779–781, 929, 944, 1000, 1002, 1008–1010
extractable memorization, 253, 368

f-divergence, 60, 61, 85, 87, 89, 96, 97, 753
f-DP, 471, 500, 703, 705, 733, 782, 786, 788, 789, 791, 808, 809, 843, 858, 920, 929, 930, 944, 1128, 1132, 1159
FFT accountant, 862
fingerprinting code, 699
fingerprinting lemma, 674, 688, 698, 699
frequency oracle, 469, 647, 669–672
fully adaptive composition, 916
Fundamental Law of Information Recovery, 251, 463

Gaussian differential privacy, 471, 500, 546, 568, 703, 705, 782, 786, 787, 789–794, 796, 798–801, 803, 805–809,

840, 843, 858, 859, 885, 916, 920–926, 929, 930, 933, 937, 938, 940–942, 944, 966, 1014, 1017, 1126, 1128, 1132–1134, 1136, 1149, 1150, 1152, 1159, 1161–1163, 1170, 1174
Gaussian mechanism, 467, 469, 527, 541, 542, 544, 546, 547, 562, 622, 644, 665, 671, 673, 674, 685, 694, 696, 697, 702, 704, 705, 716, 717, 719, 722, 723, 730, 744, 756, 758, 760, 761, 767, 768, 770, 779, 780, 782, 788, 790, 808, 830, 836, 840, 843, 845, 862, 885, 916, 933, 937, 941, 944, 946, 958, 966, 971, 978, 983, 1014, 1017, 1038, 1049, 1058, 1071, 1092, 1170
Gaussian trade-off function, 705
generalization gap, 206, 253, 278, 296, 301, 303, 323
geometric mechanism, 643, 644
global sensitivity, 467, 527, 546, 601, 602, 604, 611, 944, 1053
gradient inversion, 251, 389
group privacy, 466, 469, 474, 476, 503, 512, 513, 520, 521, 523–525, 535, 610, 650, 665, 673, 679, 685, 708, 722, 729–731, 756, 770, 778, 779, 781, 808, 818, 835, 837, 932, 1082, 1085, 1086, 1089, 1090, 1100–1102
Gumbel-max trick, 594, 595

hidden state, 865, 884, 888–890, 930
hockey-stick divergence, 6, 82, 84, 86, 466, 468, 482, 519, 541, 635, 702, 705, 753, 791, 830, 843, 926
Hoeffding's inequality, 462, 555, 765
homomorphic encryption, 389
honest-but-curious, 259, 260, 275

imputation baseline, 252, 323, 324
individual privacy accounting, 914–916

k-anonymity, 250, 251, 253, 415, 416, 418, 422, 424, 436, 437
Kullback–Leibler divergence, 64, 96, 139, 154, 551, 554, 555, 653, 711, 713, 730, 763, 765, 797

l-diversity, 251, 253, 418, 427, 436, 437
label leakage, 371, 389
label-only attack, 255, 259, 260, 262, 263, 273, 275, 306, 322, 323
Laplace distribution, 5, 6, 583
Laplace mechanism, 500, 527, 533, 541, 544, 546, 547, 561, 566, 610, 612, 622, 627, 628, 654, 655, 668, 681, 716, 765, 788, 836
learning with errors, 218, 245, 247, 248
likelihood ratio attack, 251–253, 278, 288, 290, 291, 294, 295, 298, 300, 302–304, 944, 945, 1174
likelihood-ratio test, 98, 106, 251, 270, 278
linkage attack, 436
Lipschitz, 158, 163, 164, 166, 172, 188–190, 206, 216, 878, 1067, 1081
local differential privacy, 471, 647, 651, 671, 1144
local randomizer, 647, 659, 670, 671, 865, 929, 1144, 1145
local sensitivity, 597, 598, 605, 607, 609–611, 618–620
logit scaling, 303
loss attack, 262, 274, 278, 283, 291, 302–304, 1150

malicious adversary, 272
martingale, 54, 55, 57, 555, 896, 906, 916
max-divergence, 60, 61, 68, 71, 77, 84, 96, 97, 466, 471, 482, 711
McDiarmid's inequality, 33, 55, 57, 192, 200, 217, 393, 401, 403, 414, 416
membership advantage, 262, 272, 273, 306, 323, 326, 334, 468, 471, 490, 499–501
membership inference, 250–252, 274, 275, 303, 323, 346, 348, 466, 674, 944, 1141, 1174
meta-classifier, 326, 338–341, 343–346
Mills ratio, 49, 537, 780
min-entropy, 6, 131, 135, 144, 148, 150–152, 156, 157, 348
model inversion, 250, 303, 306, 323
moment generating function, 7, 9, 15, 19, 32, 40, 42, 43, 45, 47, 57, 71, 74, 555, 714, 736, 753, 758, 761–763, 798, 801, 824, 858

moments accountant, 708, 729–732, 811, 836, 983
Mondrian, 424, 427, 430, 436
mutual information, 4, 6, 130, 139, 141–143, 147, 156, 157, 456

Narayanan–Shmatikov attack, 415
negligible function, 218, 224
neighbouring datasets, 466, 468, 470, 471, 500, 501, 503, 525, 530, 534, 546, 547, 582, 596, 601, 673, 694, 704, 744, 811, 817, 920, 932, 935–937, 942, 953, 1006, 1008, 1078, 1086, 1147, 1152
Neyman–Pearson lemma, 98, 128, 250, 251, 253, 268, 270, 285, 363, 405, 692, 794
normal distribution, 5, 466, 539, 692, 758, 830, 857

objective perturbation, 949, 956, 962, 963, 965–967
one-way marginal, 699
onion effect, 298, 303
optimal composition, 467, 469, 471, 500, 503, 521, 524, 549, 553, 557, 562, 564, 566, 568, 569, 702, 808, 847
optimistic discretization, 936, 937
output perturbation, 949, 962, 963, 965–967, 1123, 1145

packing lower bound, 679, 681, 682, 690, 698
Paley–Zygmund inequality, 439, 446, 464
parallel composition, 503, 520, 524, 525
permutation invariance, 344, 345
permute-and-flip, 595
pessimistic discretization, 936
Pinsker's inequality, 61, 81, 92–94, 96, 97, 653
Poisson distribution, 7, 14–16, 19, 31, 32, 42, 418, 421, 423, 425, 437, 637, 643, 644, 668, 704, 706, 776, 808, 811, 814, 816–818, 820–822, 825, 835–838, 840, 862, 929, 932–934, 936, 941, 942, 947, 971, 972, 984, 987, 991, 996, 998–1000, 1003, 1004, 1006, 1009, 1010, 1028–1031, 1033, 1054, 1055, 1058, 1059, 1071, 1087, 1148
Poisson sampling, 704, 808, 811, 820–822, 835, 836, 862, 929, 932, 936, 941, 947, 971, 972, 984, 1028, 1030, 1031, 1033, 1054, 1055, 1058, 1071, 1087, 1148
Poisson summation formula, 637, 643, 644
post-processing, 466, 469, 471, 474, 503, 508, 509, 517, 521, 523–525, 535, 539, 559, 563, 564, 567–569, 610, 627, 628, 650, 665, 695, 708, 722, 731, 759, 763, 780, 789, 794, 804, 872, 888, 889, 926, 933, 934, 958, 971, 972, 991, 996, 1001, 1008, 1014, 1016, 1017, 1038, 1043, 1045, 1048, 1054, 1071, 1085, 1086, 1092, 1165
privacy filter, 521, 523, 706, 916
privacy game, 4, 250, 252, 255, 264, 267, 274, 275, 283, 286, 287, 310, 323, 345, 490
privacy loss, 466–468, 480, 482, 484, 485, 497, 499, 501, 506, 510, 514–516, 518, 519, 527, 533, 534, 536, 545, 547, 549, 551, 553, 558, 565, 568, 569, 592, 593, 602, 604, 610, 621, 622, 627, 631–633, 638, 639, 650, 696, 697, 702, 703, 705, 706, 711, 714, 739, 749, 751, 753, 754, 760–763, 766, 772, 778–781, 822, 830, 838, 856, 862, 888, 898, 904, 916, 920, 921, 929–931, 936, 941, 956, 991, 1126, 1137, 1138, 1145, 1151, 1159
privacy loss distribution, 703, 705, 706, 754, 781, 838, 840, 844, 845, 847–851, 853, 854, 856–863, 916, 920–931, 933, 935–942, 1126, 1128, 1131, 1133–1138, 1148–1150, 1152
privacy loss random variable, 467, 468, 549, 702, 705
privacy odometer, 524, 568, 915
privacy profile, 467, 508, 509, 541, 546, 547, 566, 569, 730, 782, 791, 793, 800, 804, 935, 940, 1143, 1147
property inference, 250, 251, 274, 345, 346
propose-test-release, 469, 598, 611, 612, 618–620
PRV accountant, 808, 840, 850, 862, 940, 941
pseudorandom function, 237, 246, 247

pure differential privacy, 468, 474, 482, 492, 500, 501, 519, 525, 541, 546, 547, 620, 631, 644, 674, 685, 690, 698, 700, 711, 713, 716, 731, 733, 765, 772, 778–780, 835, 837, 871, 889, 892, 898, 899, 903, 917, 929, 930, 932, 987, 1010, 1049, 1059, 1086, 1150, 1167

quantitative information flow, 156
quasi-identifier, 253, 393, 396–398, 415, 418, 422–424, 436

randomized response, 471, 480–482, 484, 487, 489–491, 499–501, 503, 507, 509, 515, 516, 518, 523, 549, 551, 553, 558, 559, 561, 565–569, 647, 650, 651, 655, 657–659, 661–663, 668, 670–672, 702, 705, 708, 712, 713, 715, 716, 718, 729–731, 752, 756, 763, 765, 778, 780, 787, 788, 795, 799, 800, 807, 816, 834–836, 850, 851, 861, 865, 869, 870, 872–875, 887–890, 896, 898, 905, 915, 929–931, 940, 991, 997, 1008, 1039, 1145
RDP curve, 703–706, 708, 712, 713, 715–717, 725, 728, 741–744, 746, 748, 749, 751, 763, 765, 766, 776, 777, 781, 804–807, 825, 830, 834, 838, 921, 925, 971, 987, 991, 1092, 1133, 1134
re-identification, 251, 393, 407, 408, 415, 433, 463
reconstruction attack, 251, 463, 673, 698, 699, 1139
rejection sampling, 61, 641
report-noisy-max, 467, 469, 527, 570, 571, 582, 594, 595, 674, 682, 699, 944, 1038, 1040
ROC curve, 121, 128, 269, 274, 275, 293, 295, 300, 303, 304
Rényi differential privacy, 466, 702–706, 708, 711–717, 720–726, 728–731, 733, 736, 738, 741–744, 746, 748–754, 756, 763, 765, 766, 770, 776, 777, 779–781, 783, 804–807, 809, 811, 821–823, 825, 828–838, 840, 858, 859, 861, 863, 888, 892, 900, 904, 906, 911, 916, 917, 920–926, 928–930, 933, 935, 936, 938–942, 944, 947, 971, 972, 976, 983, 984, 987, 991, 996, 1007, 1009, 1028, 1037–1039, 1045, 1049, 1050, 1054, 1058, 1092, 1094, 1095, 1101, 1126, 1128, 1132–1134, 1136, 1139, 1140, 1142, 1143, 1148–1150, 1152, 1163, 1171
Rényi divergence, 61, 67, 68, 71, 72, 77, 78, 96, 636, 639, 702, 705, 714, 730, 742, 758, 769, 770, 804, 823, 824, 833, 836, 858, 865, 877, 888, 904, 911, 920, 921, 926, 997, 1088, 1128, 1132
Rényi filter, 892, 900, 903, 906, 907, 911, 916, 1049

sample complexity, 674, 698, 699, 1058
sampling without replacement, 811, 821, 835, 836, 838, 888, 933, 934, 936, 940, 941
secure aggregation, 4, 469, 622, 644, 646, 647, 668, 671
semantic security, 246
sensitivity, 4, 159, 184, 190, 415, 466–469, 500, 524, 527, 531, 533, 543, 545–547, 562, 575, 597, 598, 601, 602, 604, 605, 607, 609–612, 616, 618–620, 632, 638, 644, 668, 682, 705, 715–717, 731, 758, 761, 789, 790, 801, 823, 830, 833, 837, 845, 929, 930, 944, 946, 949, 953, 963–966, 972, 1002, 1012, 1014, 1017, 1019–1021, 1026, 1027, 1031–1033, 1038, 1049, 1053, 1054, 1058, 1071, 1097, 1120, 1122, 1170
shadow model, 252, 278, 285, 288, 290, 291, 294, 297, 299–304, 326, 345
shifted Rényi divergence, 865, 888
shuffle model, 646, 647, 651, 670, 672, 702, 865, 871, 888, 929
simulation paradigm, 218, 247
Skellam mechanism, 668, 671
skewness attack, 431
smooth sensitivity, 469, 597, 598, 607, 611, 612, 619, 620, 1058
smoothness, 163, 189, 604, 605, 609, 958, 966

snapping mechanism, 621, 643
sparse vector technique, 467, 469, 547, 570, 571, 590, 594–596
staircase mechanism, 546
stochastic gradient descent, 3, 6, 158, 162, 168, 169, 171, 172, 179, 180, 182, 183, 188–190, 192, 209, 210, 215–217, 702–704, 706, 708, 730, 753, 754, 756, 777–779, 783, 801–803, 807–809, 811, 831, 836, 838, 840, 858–861, 878, 888, 889, 891, 892, 916, 919–921, 924, 929, 931, 934, 936, 938, 939, 941, 944–947, 949, 965, 966, 969, 972, 977, 980, 981, 983, 987, 991, 992, 998, 999, 1003, 1005, 1006, 1010, 1012, 1015, 1028–1033, 1035, 1043, 1044, 1047, 1048, 1054, 1057, 1061, 1071, 1074, 1077, 1080, 1083, 1089–1095, 1097–1099, 1101, 1102, 1106, 1107, 1113, 1120, 1122, 1123, 1126, 1128, 1131, 1133–1135, 1137, 1140, 1142, 1143, 1145, 1147, 1148, 1150, 1152, 1154, 1161, 1168, 1169, 1171, 1173
strong convexity, 158, 163–166, 172, 173, 175, 184, 188–190, 203, 206, 215, 888–890, 944, 954, 958, 963, 966, 967
sub-Gaussian, 47, 57, 762
subsample-and-aggregate, 469, 598, 614, 616, 618–620, 1058
subsampled Gaussian mechanism, 703, 706, 730, 776, 778–780, 783, 809, 811, 822, 825, 828, 830, 835, 837, 840, 857, 862, 863, 920, 929, 930, 941, 971, 991, 1054, 1057, 1071, 1092, 1100, 1120, 1122
subset-sum query, 464
sufficient statistic, 97, 333, 868

t-closeness, 251, 253, 418, 430, 437
threat model, 1, 4, 250, 252–254, 260, 275, 286, 306, 345, 466, 1174, 1175
total variation distance, 6, 60, 61, 80–82, 85, 93, 94, 98, 115, 118, 250, 251, 253, 270, 293, 306, 309, 324, 326, 389, 418, 430, 431, 594, 692
tracing attack, 463, 469, 674, 693, 694, 699
trade-off function, 6, 98, 110, 112, 113, 118, 122, 128, 129, 250, 253, 255, 268, 466, 500, 546, 561, 568, 702, 703, 705, 706, 786, 788, 804–806, 809, 944, 1126, 1134, 1135, 1147, 1150, 1154, 1161, 1170, 1174
training-data extraction, 251, 348
truncated concentrated differential privacy, 756, 771, 776–781, 929, 931
type I error, 268, 705, 743, 744, 786, 787, 790, 792
type II error, 109, 111, 128, 268, 705, 753, 786, 787, 790, 796

unary encoding, 647, 657–659, 669–672
uniform stability, 200, 214
union bound, 58, 124, 231–233, 237, 245, 433, 435, 440, 447–449, 456, 464, 588, 589, 681, 682, 698, 773, 908, 909, 916, 929, 1040, 1058, 1059, 1157
unit of privacy, 468, 470, 474, 476, 500, 501, 935–937, 940, 1018, 1020, 1027, 1083, 1089, 1091–1095, 1098, 1099, 1101, 1102

variance, 3, 5, 7, 12, 15, 19, 22, 31–33, 38, 39, 47, 57, 58, 66, 67, 69, 74, 77, 78, 81, 84, 127, 158, 177–179, 181, 188, 190, 265, 291, 293, 294, 296, 300–304, 328, 332, 333, 344, 403, 446, 450, 454, 468, 536, 539, 543, 553, 554, 598, 613, 615, 616, 619, 622, 631, 636, 642–644, 656–658, 665, 666, 668, 671, 688, 692, 693, 700, 714, 758, 760, 761, 778, 797, 798, 801, 843, 846, 847, 861, 884, 885, 887, 896, 897, 899, 921, 926, 930, 958, 971, 1015, 1032, 1064, 1083, 1101, 1114, 1116, 1117, 1132, 1143, 1159, 1171
vulnerability, 131, 146, 156, 157

weak triangle inequality, 723, 730
white-box attack, 255, 259, 260, 262, 263, 273–275, 303, 329, 1162–1164, 1167, 1169, 1173

zero-concentrated differential privacy, 703, 705, 706, 756, 760, 761, 765, 767, 768, 770–772, 776, 778–781, 900, 903, 916, 929–931, 940, 1144–1146, 1150–1152